

Analyse statique de programmes React par interprétation abstraite

Anatomie complète de `reactant-analyzer`

rboudrouss

Instantané du dépôt au commit e67b10a (27 septembre 2026)

Avant-propos

Ce manuscrit décrit `reactant-analyzer`, un analyseur statique de programmes React fondé sur l'interprétation abstraite, tel qu'il existe au commit e67b10a du dépôt (27 septembre 2026). Il a été écrit pour qu'un lecteur qui connaît Rust et React côté utilisateur, sans formation préalable en analyse statique, en sorte en connaissant très bien la codebase : ses fichiers, ses types, ses algorithmes, ses décisions d'architecture et ses limites.

Parcours. L'ordre des chapitres va du concret vers l'abstrait. La première partie pose le problème : les bugs React visés, le modèle d'exécution de React que l'on abstrait, les rappels d'interprétation abstraite nécessaires, puis un tour d'architecture de la codebase. Les parties suivantes suivent un fichier source dans l'ordre où l'analyseur le traite : le parseur et la détection des composants, la représentation intermédiaire et le lowering, les domaines abstraits, le moteur de point fixe et ses relations, l'analyse inter-composants, puis les règles qui transforment ces faits en diagnostics. La dernière partie décrit l'outil lui-même (ligne de commande, distribution, méthode de mesure), l'histoire de ses décisions et ses limites connues. Trois annexes servent de référence : les fiches des décisions d'architecture, le glossaire et le catalogue des règles. La carte de la codebase prévue en quatrième annexe n'a pas été rédigée dans cette version ; le tour d'architecture (chapitre 4) en tient lieu.

Un lecteur pressé peut lire les chapitres 1 à 4 puis piocher ; un lecteur qui veut contribuer au moteur lira dans l'ordre jusqu'au chapitre 16 ; un lecteur qui veut écrire une règle lira les chapitres 1 à 4, 17 et 22.

Conventions. Les extraits de code Rust sont pris tels quels dans le dépôt, avec leurs numéros de ligne réels ; les programmes React d'exemple sont donnés en TSX et les sorties de l'analyseur ont toutes été observées sur le binaire construit au commit photographié. Les encadrés *Rappel React* rappellent un point de la sémantique de React nécessaire à la compréhension ; les encadrés *Décision* résument une décision d'architecture (ADR) et les alternatives refusées ; les encadrés *Piège* signalent une subtilité ou une erreur classique ; les encadrés *Exercice* proposent un travail au lecteur, avec solution. Les identifiants, noms de fichiers et noms de règles restent en anglais, tels qu'ils apparaissent dans le code ; les notions mathématiques sont en français avec une notation unifiée (ordre $\sqsubseteq$, join $\sqcup$, meet $\sqcap$, widening ∇ , concrétisation γ).

État du texte. Chaque chapitre a été rédigé à partir de dossiers techniques établis sur le code, puis relu par un vérificateur qui a recontrôlé chaque extrait et rejoué chaque sortie. Les constats faits en cours de rédaction sur le code lui-même (faux négatifs ou faux positifs observés, écarts entre une ADR et le code) sont rassemblés au chapitre 27 et présentés comme des observations au commit e67b10a, non comme des jugements. Une passe finale d'harmonisation reste à faire ; les renvois croisés et l'index peuvent encore comporter des irrégularités.

Sources. La documentation officielle de React [Met26b], la sémantique formelle React-tRace [LAY25] et les travaux fondateurs de l'interprétation abstraite [CC77, CC79, RY20] sont les références externes de ce manuscrit ; tout le reste vient du dépôt [rbo26].

Table des matières

Avant-propos	ii
I Le problème et son contexte	1
1 Introduction : pourquoi analyser statiquement React ?	2
1.1 Trois programmes React fautifs	2
1.1.1 Premier défaut : la boucle infinie	3
1.1.2 Deuxième défaut : la dépendance oubliée	3
1.1.3 Troisième défaut : la fermeture figée	4
1.1.4 Ce que ces trois exemples ont en commun	4
1.2 Ce qu'un linter syntaxique voit, et ce qu'il ne voit pas	5
1.2.1 Ce que la forme suffit à voir	5
1.2.2 Ce que la forme ne voit pas	6
1.3 Le pari : interpréter abstraitement le composant	9
1.3.1 Exécuter sur des ensembles	9
1.3.2 Pas seulement des nombres	10
1.3.3 Quel React on abstrait	11
1.3.4 Le pipeline	11
1.4 La promesse : trois niveaux de diagnostic et la soundness	12
1.4.1 Trois niveaux, trois promesses	13
1.4.2 Une Error se construit à partir d'une preuve	14
1.4.3 La soundness : pas de faux négatif	15
1.5 Première prise en main	16
1.5.1 Construire et lancer	16
1.5.2 Lire la sortie humaine	16
1.5.3 Lire la sortie JSON	18
1.6 Le projet en chiffres	20
1.6.1 Le code	20
1.6.2 Les règles	21
1.6.3 Le corpus	22
1.7 Plan du manuscrit et conseils de lecture	22
1.7.1 Parcours de lecture	23

1.7.2	Conventions	23
	Résumé du chapitre	23
1.8	Exercices	24
2	React vu par l'analyste : le modèle d'exécution que l'on abstrait	26
2.1	Un composant est une fonction du rendu	27
2.1.1	Un premier programme : Counter	27
2.1.2	Le rendu doit être pur	28
2.1.3	Ce que l'analyseur voit de Counter	29
2.2	L'état : <code>useState</code> , la file d'updaters et le batching	29
2.2.1	Un setter ne modifie pas la variable	29
2.2.2	Le bail-out <code>Object.is</code>	30
2.2.3	L'état initial et l'initialiseur paresseux	31
2.2.4	Ce que fait le code	31
2.3	Les effets : après le commit, gardés par les deps	35
2.3.1	Quand un effet s'exécute	35
2.3.2	<code>SelfCounter</code> et <code>Inf</code> : le cycle Check-Effect	36
2.3.3	<code>Flicker</code> : un rendu de trop	37
2.3.4	<code>Toggle</code> : une boucle que la valeur abstraite ne voit pas	37
2.3.5	<code>Cleanup</code> et effets de mise en page	38
2.3.6	Ce que fait le code	39
2.4	L'identité référentielle	41
2.4.1	Même contenu, autre objet	41
2.4.2	<code>ObjChurn</code> : une boucle par l'identité	41
2.4.3	Ce que fait le code	42
2.5	Les règles des hooks et l'identité positionnelle	46
2.5.1	Un hook est identifié par sa position	46
2.5.2	Ce que fait le code	47
2.5.3	Appeler un setter pendant le rendu	48
2.6	Strict Mode : le double rendu	49
2.7	Le contexte et la propagation du rendu	51
2.7.1	Rendre un parent, c'est rendre ses enfants	51
2.8	Server Components et la directive <code>"use client"</code>	53
2.9	React-tRace : une sémantique formelle des hooks	54
2.9.1	Le langage	54
2.9.2	Les objets sémantiques	54
2.9.3	La boucle de rendu	55
2.9.4	Les règles d'évaluation	55
2.9.5	Les théorèmes	56
2.10	La sémantique concrète du projet : ce qu'on garde, ce qu'on abstrait	57
2.10.1	Le contrat : ADR-001	57

2.10.2	De la boucle de rendu à la boucle de point fixe	57
2.10.3	Ce qu'on garde, ce qu'on abstrait	58
2.10.4	Les callbacks : dans le cycle ou hors du cycle	60
2.10.5	Trois écarts vérifiés	61
2.11	Les exemples canoniques du manuscrit	62
	Résumé du chapitre	64
	Exercices	64
3	Rappels d'interprétation abstraite	67
3.1	Pourquoi approximer	67
3.1.1	Une question que l'analyseur doit trancher	67
3.1.2	Pourquoi on ne peut pas calculer la réponse exacte	68
3.1.3	Sur-approximer, et ce que cela coûte	68
3.2	Ordres, treillis, top et bottom	69
3.2.1	Un premier exemple : que sait-on d'un booléen ?	69
3.2.2	Définitions	70
3.2.3	Ce que fait le code : le trait <code>AbstractDomain</code>	70
3.2.4	Autres treillis du projet, en un coup d'œil	73
3.3	Concrétisation et correction	73
3.3.1	Ce que « veut dire » une valeur abstraite	73
3.3.2	Ce que fait le code : l'intervalle	74
3.3.3	« Absent » vaut $\top$ ou $\perp$ selon le contexte	75
3.4	Exemple filé : le compteur en intervalles	75
3.4.1	Le treillis des intervalles	75
3.4.2	Un tour abstrait de <code>Runaway</code>	76
3.5	Transfert, équations de flot, point fixe	77
3.5.1	Une boucle locale	77
3.5.2	Fonctions de transfert	77
3.5.3	Équations de flot	78
3.5.4	La worklist : n'itérer que ce qui change	79
3.5.5	Ce que fait le code : <code>analyze_cfg</code>	79
3.5.6	Le point fixe des tours de <code>React</code>	82
3.6	Hauteur infinie et widening	83
3.6.1	Le problème	83
3.6.2	Ce que fait le code : <code>widen</code> et <code>widen_to</code>	84
3.6.3	D'où viennent les seuils	85
3.6.4	Retour aux compteurs	86
3.6.5	Le narrowing, et pourquoi <code>reactant-analyzer</code> s'en passe	87
3.7	Must, may, et la polarité d'un fait	88
3.7.1	Deux setters dans le rendu	88
3.7.2	Pourquoi une sur-approximation ne suffit pas	88

3.7.3	Ce que fait le code : des verdicts typés par leur polarité	89
3.8	Soundness	90
3.9	CFG, dominance, « sur tous les chemins »	91
3.9.1	Un hook après un return anticipé	91
3.9.2	Ce que fait le code	92
3.10	Intra- et inter-procédural, inlining	94
3.11	Mises à jour fortes et faibles	96
3.12	Ce que reactant-analyzer fait, et ne fait pas	97
	Résumé du chapitre	98
	Exercices	98
4	Tour d'architecture : lire la codebase	101
4.1	La carte : crates, modules et fichiers	102
4.1.1	Un workspace, deux crates, un paquet npm	102
4.1.2	La racine : lib.rs et main.rs	103
4.1.3	Les dix modules de la bibliothèque	103
4.1.4	Fichier par fichier	104
4.2	Le flot d'une invocation, de bout en bout	107
4.2.1	De main au driver	109
4.2.2	Contexte, découverte, lowering	109
4.2.3	L'analyse : registres, point fixe, relations	112
4.2.4	Les règles, le tri, la sortie	114
4.3	Cinq couches et leurs frontières	116
4.3.1	Front-end → IR : plus d'AST après le lowering	117
4.3.2	IR → moteur : un interpréteur générique, un cycle cassé	117
4.3.3	Moteur → règles : les règles lisent, elles ne parcourent pas	118
4.3.4	Règles → coquille : un driver pour tous les hôtes	119
4.4	Les conventions transversales	120
4.4.1	Fail-closed : l'absence n'est pas une preuve	120
4.4.2	Must et may comme types	121
4.4.3	Identités internées : FileId, ComponentId	121
4.4.4	Positions : SourceRange	122
4.4.5	Les alias de base	122
4.5	Les trois principes de CLAUDE.md, dans le code	123
4.6	Comment naviguer	124
4.6.1	Les tests comme documentation exécutable	124
4.6.2	Les options qui montrent l'intérieur	126
4.6.3	Les documents du dépôt	127
4.7	Feuille de route des chapitres suivants	127
	Résumé du chapitre	127
4.8	Exercices	128

II	Du source à la représentation intermédiaire	130
5	Le front-end : oxc et la détection des composants, hooks et utilitaires	131
5.1	oxc en quelques pages	132
5.1.1	Une arène par fichier	132
5.1.2	ParserReturn : panicked n'est pas diagnostics	133
5.1.3	Le type de source se choisit par extension	134
5.1.4	Spans et positions	135
5.1.5	Ce que le front-end lit dans l'AST	135
5.2	Le contrat du front-end	135
5.2.1	Ce qui entre, ce qui sort	135
5.2.2	L'invariant de frontière	137
5.2.3	Un lowerer, pas à pas	138
5.3	Le marcheur commun et le candidat	140
5.3.1	Le problème : combien de façons d'écrire une fonction ?	140
5.3.2	L'interface du marcheur	140
5.3.3	La marche	141
5.3.4	Le candidat	143
5.4	Trois classes de fonctions	143
5.4.1	Le hook custom : un critère purement nominal	144
5.4.2	L'utilitaire : une double négation	144
5.4.3	Le composant : quatre règles	145
5.4.4	Les trois classes, formellement	148
5.5	Deux faux négatifs corrigés	149
5.5.1	La flèche concise perdait sa valeur de retour (#5)	149
5.5.2	Le composant toujours null n'était pas détecté (#122)	151
5.6	La provenance des hooks	154
5.6.1	Le problème : un nom de hook ne dit pas d'où il vient	154
5.6.2	HookOrigin : ce que la déclaration d'import prouve	154
5.6.3	ImportCtx et classify_callee	157
5.7	Les faits de module	159
5.7.1	Les constantes de module	159
5.7.2	Les contextes et leur identité	160
5.7.3	ModuleFacts : directives et arêtes d'import	161
5.7.4	Les props typées DOM	163
5.8	L'identité des callees JSX	164
5.8.1	Le problème : qui est <Widget/> ?	164
5.8.2	CompOrigin et JsxOrigins	164
5.9	Les limites du front-end sont des questions de soundness	166
5.9.1	Absent n'est pas $\top$	166
5.9.2	Quand l'utilisateur est-il averti ?	167

5.9.3	Les frontières tracées	169
	Résumé du chapitre	170
	Exercices	170
6	La représentation intermédiaire	172
6.1	Pourquoi une IR, et pourquoi un CFG	172
6.1.1	Un compteur et ses formes équivalentes	172
6.1.2	La décision : une IR dédiée, en graphe de flot de contrôle	173
6.1.3	Le périmètre : seize fichiers	174
6.1.4	Les alias de base	174
6.2	Le squelette : blocs, terminateurs, arêtes, instructions	175
6.2.1	Le CFG du compteur	175
6.2.2	Les types	175
6.2.3	Deux accessibilités et une validation	178
6.2.4	Les instructions	180
6.3	Les expressions	181
6.3.1	Littéraux et opérateurs	181
6.3.2	L'énumération Expr	182
6.3.3	Les sites d'allocation	185
6.3.4	Deux aides sur les littéraux d'objet	186
6.3.5	Le parcours canonique	187
6.4	Les hooks dans l'IR	188
6.4.1	Labels positionnels et slots	188
6.4.2	La table HookEntry	189
6.4.3	Marqueurs : ancrer un hook sans valeur	192
6.4.4	Les tableaux de deps	193
6.4.5	La provenance	196
6.5	Les unités de l'IR et les faits de module	197
6.5.1	Composant, hook custom, utilitaire	197
6.5.2	Constantes de module et contextes	198
6.5.3	Les faits de module	200
6.6	Les identités internées	200
6.6.1	Le patron : un identifiant de 4 octets et une table	200
6.6.2	Le nom affiché n'est pas une identité	202
6.7	Les positions	203
6.8	Analyses syntaxiques sur l'IR	204
6.8.1	Variables libres	205
6.8.2	Chemins d'accès	206
6.8.3	Couverture et épinglage	207
6.8.4	Certificats de liaison	208
6.9	Où l'IR choisit le côté sûr	210

Résumé du chapitre	211
Exercices	212
7 Le lowering : de l'AST au CFG	214
7.1 Le lowering dans le pipeline	215
7.1.1 Trois fichiers, trois passes	215
7.1.2 La chaîne d'appel	215
7.1.3 Pourquoi un CFG	216
7.2 La machine à blocs	217
7.2.1 Un corps linéaire	217
7.2.2 Le constructeur	218
7.2.3 La boucle des instructions et le code mort	220
7.2.4 La sortie : into_cfg et la chute en fin de corps	220
7.3 Le if et le retour anticipé	221
7.3.1 La table des instructions	221
7.3.2 Le gabarit du if	223
7.3.3 Le retour anticipé et la dominance	224
7.4 Les expressions : garder chaque lecture	226
7.4.1 Le contrat de lower_expr	226
7.4.2 Deux outils transverses	227
7.4.3 Tour des formes	227
7.4.4 Objets, tableaux, clés synthétiques	230
7.4.5 JSX	231
7.5 Les diamants : ternaire et court-circuits	232
7.5.1 Le problème : des effets conditionnels dans une expression	232
7.5.2 Le code	233
7.5.3 Deux subtilités	235
7.6 Les boucles, break et continue	236
7.6.1 while : l'arête arrière	236
7.6.2 Les trois autres boucles	237
7.6.3 break, continue, labels	238
7.7 switch et try : deux sous-approximations corrigées	240
7.7.1 Le switch : une chaîne de dispatchs opaques	240
7.7.2 Le try : une branche sur l'inconnu	243
7.8 Motifs, déstructurations et temporaires	244
7.8.1 Le principe : un temporaire, puis des projections	244
7.8.2 Défauts et paramètres	245
7.8.3 Les cibles d'affectation	247
7.9 Fonctions imbriquées, classes et sites d'allocation	248
7.9.1 Un corps par fonction	248
7.9.2 Le compteur d'allocation	249

7.10	await, arêtes Await et positions synthétiques	250
7.10.1	Le problème : ce qui suit un await n'est plus dans la phase	250
7.10.2	Positions synthétiques	252
7.11	L'extraction des hooks	253
7.11.1	Avant, après	253
7.11.2	Qui est un hook ? La provenance	255
7.11.3	L'entrée et le marqueur	256
7.12	Handlers et subscriptions	258
7.12.1	La règle : par fuite, pas par nom	258
7.12.2	Les subscriptions	259
7.13	Hoist de terminateur et flèches concises	260
7.14	Limites du lowering	263
7.14.1	Écarts vérifiés	263
7.14.2	Pièges pour qui modifie ce code	264
	Résumé du chapitre	265
	Exercices	265
8	Splice, remap et inlining	267
8.1	Pourquoi inliner	267
8.1.1	Un hook n'a pas d'analyse à lui	267
8.1.2	Cinq façons de franchir un appel	269
8.1.3	Où la greffe se place dans l'analyse d'un composant	270
8.2	Décaler les identités : le remap	270
8.2.1	Quatre espaces numérotés à partir de zéro	270
8.2.2	Offsets : un seul porteur pour deux décalages	271
8.2.3	La largeur d'une greffe : alloc_id_span	272
8.2.4	Appliquer le décalage : remap_expr, remap_cfg, remap_hooks	272
8.2.5	La réserve d'identités d'un composant : SpliceIds	272
8.3	La primitive de splice	273
8.3.1	Un utilitaire à deux retours	273
8.3.2	Le contrat : Splice	274
8.3.3	Étapes 1 à 4 : renommer, vérifier, couper, allouer	275
8.3.4	Étape 5 : réécrire les terminateurs	277
8.3.5	Étapes 6 à 9 : insérer et raccorder	278
8.3.6	Étape 10 : les arêtes	279
8.3.7	Déroulé complet : un utilitaire inter-fichiers	280
8.4	Hygiène : α -renommage et capture	281
8.4.1	Un espace de noms plat	281
8.4.2	Le code : bound_vars et callee_rename_map	281
8.4.3	Renommer sans capturer : les fermetures imbriquées	283
8.4.4	Le sel et l'affichage : SPLICE_MARK	284

8.4.5	Substituer les arguments dans les initialiseurs d'état	284
8.5	Expansion des hooks custom	285
8.5.1	Ce que le lowering a laissé	285
8.5.2	L'algorithme	286
8.5.3	Trouver le hook, se garder de la récursion	286
8.5.4	Préparer la copie	288
8.5.5	Enregistrer la région et la provenance	289
8.5.6	Le garde-throw, rejoué	290
8.5.7	Le prix de la garde : un hook appelé deux fois	290
8.6	Inlining des utilitaires	291
8.6.1	Le registre des utilitaires	291
8.6.2	Trouver les appels, épuiser le budget	292
8.6.3	Ce qui n'est pas une greffe : fermetures locales (B5/B6)	293
8.6.4	Ce qui n'est pas une greffe : les composants enfants	294
8.7	Les hooks de bibliothèque : des résumés	294
8.7.1	Sans source, pas de greffe	294
8.7.2	Connu n'est pas modélisé	294
8.7.3	Ce que fait le moteur d'un résumé	295
8.8	Attribuer les findings des hooks inlinés	297
8.8.1	Un défaut, trois consommateurs	297
8.8.2	La décision	297
8.8.3	Un finding sans position	298
8.9	Cas limites et défauts observés	298
8.9.1	Un return jeté	298
8.9.2	Des sites en collision	300
8.9.3	Trois lectures fausses sans finding démontré	300
	Résumé du chapitre	301
	Exercices	302

III Les domaines abstraits 304

9 Les treillis simples : plats, ensembles bornés, intervalles 305

9.1	Le contrat d'un domaine : le trait AbstractDomain	306
9.1.1	Une question concrète	306
9.1.2	Le trait, méthode par méthode	306
9.1.3	Le contrat	308
9.2	Les treillis plats : BoolVal et SetterVal	309
9.2.1	Deux exemples React	309
9.2.2	Ce que fait le code : la macro flat_lattice!	310
9.2.3	BoolVal : un treillis plat exact	311
9.2.4	SetterVal : un treillis plat à identité	312

9.3	Un ensemble borné de chaînes : <code>StrConst</code>	313
9.3.1	Le problème	313
9.3.2	Ce que fait le code	314
9.3.3	Où naissent les ensembles de chaînes	316
9.4	Les intervalles : représentation, ordre, join	317
9.4.1	Le problème	317
9.4.2	Ce que fait le code	317
9.4.3	Ordre, join, meet	319
9.5	Arithmétique d'intervalles	322
9.5.1	Addition, soustraction, multiplication, négation	322
9.5.2	<code>NaN</code> , $\pm\infty$ et la division	323
9.5.3	Reste et puissance (commit 548f922)	323
9.5.4	La table des opérateurs binaires	325
9.6	Les opérateurs bit à bit	327
9.6.1	Un faux négatif : les décalages sur des non-entiers	330
9.7	Widening : du saut à l'infini aux seuils	331
9.7.1	La chronologie d'un compteur	331
9.7.2	Les deux opérateurs	332
9.7.3	D'où viennent les seuils	333
9.7.4	Le même widening dans les boucles locales	334
9.8	Raffinement sur gardes	334
9.8.1	Le besoin	334
9.8.2	Ce que fait le code : les raffinements numériques	335
9.8.3	Qui appelle les raffinements	336
9.8.4	La véracité, et une garde impossible qui n'élague rien	337
9.9	Bilan : la soundness opération par opération	339
9.10	Limites et défauts observés	339
9.10.1	$0 \times \pm\infty$: un $\perp$ inattendu	339
9.10.2	<code>Infinity</code> comme valeur	340
9.10.3	Les cycles finis	341
9.10.4	Imprécisions assumées	341
	Résumé du chapitre	341
	Exercices	342
10	Le produit <code>StateValue</code> et la stabilité référentielle	345
10.1	Un enum plat perd <code>null</code> $\cup$ <code>number</code>	345
10.1.1	Le problème	345
10.1.2	Le domaine d'ADR-008 : une somme plate	346
10.1.3	Trois rustines, et un FN qui résiste	346
10.2	Sortes disjointes : le produit <code>StateValue</code>	347
10.2.1	L'observation	347

10.2.2	Le code	347
10.2.3	$\perp$, $\top$ et les constructeurs	349
10.2.4	KindMask : une seule énumération des sortes	350
10.2.5	L'arithmétique : as_arith et ToNumber(null)	352
10.2.6	Ordre, join, meet et widening ponctuels	354
10.2.7	from_init : un nom trompeur	356
10.3	La stabilité référentielle : un premier treillis, et son impasse	357
10.3.1	Ce que React compare	357
10.3.2	Le treillis d'ADR-002	357
10.3.3	Le faux positif F5	358
10.3.4	Le faux négatif caché dessous	358
10.4	Traces de changement, bornes may et must	359
10.4.1	L'objet concret : la trace de changement	359
10.4.2	Deux bornes, deux usages	359
10.4.3	Le treillis Stability	360
10.4.4	Canonisation	361
10.4.5	Ordre, join, meet, widen	362
10.5	Deux vues d'un même état : la conversion côté lecture	365
10.5.1	Le problème	365
10.5.2	Le code : un seul point de conversion	365
10.5.3	Le churn d'objet, certifié par la structure	367
10.5.4	Lire un champ d'un objet versionné	368
10.6	Littéraux, useRef, useMemo, useCallback	368
10.6.1	Où naît une référence	368
10.6.2	La valeur d'un mémo : recompute_memo	369
10.7	Projections vers les règles	371
10.7.1	Quatre questions différentes	371
10.7.2	to_stability : la règle « motion-wins »	372
10.7.3	Les prédicats	374
10.7.4	Côté règles : StabilityVerdict	375
10.8	Les contextes : la passerelle vers le moteur	377
10.8.1	Ce dont une fonction de transfert a besoin	377
10.8.2	AnalysisCtx : le paquet d'état mutable	377
10.8.3	QueryContext et ses deux implémentations	378
10.8.4	InterCtx et AnalyzeChildFn	380
10.9	Limites et défauts observés	381
10.9.1	Un mémo sur un état numérique : un FP de projection	381
10.9.2	Les cycles finis	382
10.9.3	Une chaîne qui grandit : is_unbounded ignore str	382
10.9.4	Un élément JSX natif classé Stable : un FN sans trace	383
10.9.5	Limites connues et suivies	383

Résumé du chapitre	384
Exercices	385
11 Fonctions de transfert, interpréteur d'expressions et stores	387
11.1 Le problème : évaluer $n + 1$ sans exécuter le programme	388
11.1.1 Deux compteurs	388
11.1.2 Ce que voit l'interpréteur	389
11.1.3 Le calcul, à la main	389
11.1.4 Le contrat : le trait <code>Transfer</code>	390
11.1.5 Le chemin des appels	391
11.2 L'environnement abstrait	392
11.2.1 Un exemple : ce qu'on sait d'une variable	392
11.2.2 Quatre tables	393
11.2.3 Le <code>join</code>	394
11.3 L'évaluateur d'expressions	396
11.3.1 Un banc d'essai	396
11.3.2 La fonction <code>eval_state_value</code>	397
11.3.3 Littéraux, variables et hooks	398
11.3.4 Allocations et éléments	399
11.3.5 Appels	399
11.3.6 Opérateurs	400
11.3.7 Membres et annotations	402
11.4 Ce que le lowering a déjà fait	402
11.5 Les stores	403
11.5.1 <code>StateStore</code> : l'état React, sujet du point fixe	403
11.5.2 <code>MemoStore</code> : des valeurs dérivées, sans point fixe propre	405
11.5.3 <code>SharedStateStore</code> : l'état vu par tout le programme	405
11.5.4 <code>Heap</code> : le tas abstrait	406
11.5.5 Synthèse	406
11.6 L'instruction : liaisons, écritures de champ, setters	408
11.6.1 Le squelette : pré-passe, puis cœur	408
11.6.2 Lier une variable : <code>bind_rhs</code>	409
11.6.3 Écrire un champ : une mise à jour faible	410
11.6.4 Tirer les effets d'une expression	412
11.7 La conversion côté lecture	414
11.8 Les callbacks : classe de déclenchement, pré-passe, corps	415
11.8.1 Le piège de l' <code>onClick</code>	415
11.8.2 La classe de déclenchement	415
11.8.3 La pré-passe	417
11.8.4 Exécuter un corps : <code>exec_body_impl</code>	419
11.8.5 Retour sur l'exemple	420

11.9	Le tas par site d'allocation	421
11.9.1	Le besoin	421
11.9.2	Sites, valeurs du tas, vue EnvVal	422
11.9.3	Allouer une fermeture : un point unique	422
11.9.4	Suivre une chaîne de membres : un walker unique	423
11.9.5	Lire un membre	424
11.9.6	Exécuter une fonction par son nom : B5 et B6	425
11.10	La valeur d'un mémo	427
11.11	L'inter-composants, vu du transfert	427
11.11.1	Un parent, deux enfants	428
11.11.2	eval_comp_app, pas à pas	428
11.11.3	Le havoc des setters qui s'échappent	430
11.12	La soundness en pratique : garanties et défauts	431
11.12.1	Les garanties par construction	431
11.12.2	D1 : un corps qui voit l'état d'un bloc en retard	432
11.12.3	D2 et D3 : où et comment un setter écrit	433
11.12.4	D4 et D7 : le contexte d'un enfant, oublié	434
11.12.5	D5, D6, D9, D11, D13, D14 : le tas	434
11.12.6	D8, D10, D12 : trois modélisations	435
11.12.7	Synthèse	435
	Résumé du chapitre	435
	Exercices	436

IV Le moteur 439

12 Interpréter un CFG : worklist, arcs arrière, dominance 440

12.1	Le problème : un corps, plusieurs chemins	440
12.1.1	Qui appelle analyze_cfg	441
12.2	Un bloc, puis une séquence	442
12.2.1	Deux tables, un store	442
12.2.2	Le transfert d'un bloc	443
12.2.3	Les effets de bord d'un Return	444
12.2.4	L'environnement de fin d'un corps	445
12.3	Le diamant : joindre aux points de jonction	445
12.3.1	Ce que reçoit chaque successeur	445
12.3.2	Un diamant, pas à pas	446
12.3.3	Le join des environnements, et pourquoi $\perp$ n'y est pas neutre	447
12.3.4	Détecter un changement	447
12.4	La boucle : arcs arrière, compteur par en-tête, widening à seuils	448
12.4.1	La propagation	448
12.4.2	L'ensemble des seuils	449

12.4.3	Le compteur de boucle, pas à pas	449
12.4.4	L'algorithme complet	451
12.5	Raffiner par les gardes	451
12.5.1	Le problème	451
12.5.2	Ce que fait le code	453
12.5.3	Pourquoi ces raffinements sont sound	454
12.5.4	Une garde réfutée n'élimine pas sa branche	455
12.5.5	Les autres utilisateurs du raffinement	456
12.6	L'état dans une passe : un store insensible au flot	456
12.7	Relire une passe terminée	457
12.7.1	Ce que garantit une passe	457
12.7.2	entry_env_of : reconstruire l'entrée d'un bloc	457
12.8	Dominance : ce qui s'exécute sur tous les chemins	459
12.8.1	Le problème	459
12.8.2	compute_dominators	459
12.8.3	DominatorTree et dominates	462
12.8.4	Les sorties atteignables : ExitDominance	462
12.8.5	on_all_paths : un ensemble sur tous les chemins	464
12.9	Sonder après convergence : eval_in_stores et ConvergedEval	465
12.9.1	Le problème	465
12.9.2	Le cœur : eval_in_stores	465
12.9.3	La couche commune : ConvergedEval	466
12.10	Terminaison et complexité	467
12.10.1	Pourquoi une passe termine	467
12.10.2	Complexité	468
12.10.3	Deux interpréteurs de CFG	468
	Résumé du chapitre	469
	Exercices	469
13	Le point fixe d'un composant	472
13.1	Le problème : un ensemble d'états, pas une simulation	473
13.1.1	Un compteur qui ne s'arrête jamais	473
13.1.2	Pourquoi pas simuler	473
13.1.3	Ce qu'un ensemble ne dit pas	474
13.1.4	Les quatre portes d'entrée et la configuration	475
13.2	La préparation : ce qui se passe une fois	477
13.2.1	Les constantes de module	479
13.2.2	L'expansion : les utilitaires, puis les hooks personnalisés	480
13.2.3	Les seuils de widening	485
13.2.4	Les corps de useCallback	486
13.2.5	L'amorçage des slots	486

13.3	Un tour de la boucle externe	489
13.3.1	La passe de rendu	489
13.3.2	Les mémos : une valeur qui n'est qu'une stabilité	489
13.3.3	Les effets : tous, à chaque tour, depuis la sortie du rendu	492
13.3.4	Les handlers : dans le point fixe, hors du signal	494
13.3.5	Le join et le test de convergence	496
13.4	Convergence, widening et terminaison	497
13.4.1	Le compteur et les deux opérateurs	497
13.4.2	Trois déroulés	499
13.4.3	iterations et widen_trace	501
13.4.4	Terminaison	502
13.5	Après la convergence	503
13.5.1	Rafraîchir le rendu	503
13.5.2	Les écritures pures des effets	504
13.5.3	La tranche des relations	506
13.5.4	Le programme : phases 1 et 2	507
13.6	Correspondance avec React-tRace et argument de soundness	508
13.6.1	Un tour abstrait, quatre transitions concrètes	508
13.6.2	L'argument de soundness	509
13.7	Ce que le moteur garantit, ce qu'il ne garantit pas	510
	Résumé du chapitre	512
	Exercices	514
14	Les relations du moteur	515
14.1	Pourquoi des relations	516
14.1.1	Un fait calculé deux fois est deux faits qui dérivent	516
14.1.2	Les trois couches	517
14.1.3	Le vocabulaire de polarité	518
14.1.4	Où les relations naissent : la dernière tranche du point fixe	518
14.1.5	Le catalogue	520
14.2	La marche des setters	521
14.2.1	Le problème : quatre écritures, quatre moments	521
14.2.2	Région et phase : deux colonnes pour deux faits	521
14.2.3	La classe de marche	523
14.2.4	L'état d'une marche	524
14.2.5	Parcourir un CFG	526
14.2.6	Un appel : le site, le helper local, l'IIFE	527
14.2.7	Les arguments fonctionnels	529
14.2.8	Comment une FnLit est-elle entrée ?	530
14.2.9	Le témoin	531
14.2.10	Ce que la marche garantit	531

14.3	La table des registrars : contrat contre devinette	531
14.4	La ligne SlotWriter, colonne par colonne	533
14.4.1	Une ligne par site	533
14.4.2	La structure	534
14.4.3	La cible et le propriétaire	535
14.4.4	same_tick : la co-exécution comme fait de ligne	537
14.4.5	Updater : l'argument 0 est-il une fonction prouvée ?	538
14.4.6	block et guard_block	539
14.4.7	via : qui a écrit l'appel	539
14.4.8	Tri et déterminisme	541
14.5	La colonne written : ce que l'écriture stocke	542
14.5.1	Les trois cas de classify	543
14.5.2	Où évaluer l'argument : SiteEnvs	544
14.6	La relation effect _t riggers : identité, pas dépendance	545
14.7	La relation slot _s eed _s	547
14.7.1	Le pli	549
14.8	La relation registrations et l'appariement	551
14.8.1	L'appariement, un fait à trois valeurs	553
14.9	Deux canaux de plus : slot _r eadset	554
14.10	La preuve mono-site : une écriture qui éteint ses gardes	555
14.10.1	Le problème : écrire pendant le rendu, une seule fois	555
14.10.2	Les gardes d'un site	556
14.10.3	Trois arguments de mort	558
14.10.4	La fonction et son appelant	559
14.11	ProgramRelations et la frontière de certification	560
14.11.1	Les relations programme-entier	560
14.11.2	La frontière de certification	562
14.11.3	Le cliquet	562
14.11.4	Ce que la frontière ne garantit pas encore	563
14.12	Limites, pièges et dette documentaire	564
14.12.1	Pièges de lecture	564
14.12.2	Une ligne étrangère lue comme écrivain local	564
14.12.3	Faux négatifs et faux positifs connus	565
14.12.4	Coûts et déterminisme	565
14.12.5	Dette documentaire au commit e67b10a	565
	Résumé du chapitre	566
	Exercices	566
15	Le graphe de churn et la preuve multi-site	568
15.1	Le problème : deux effets qui se relancent	569
15.1.1	Un programme de huit lignes	569

15.1.2	Pourquoi les autres bras sont aveugles	569
15.1.3	Les trois jambes d'une arête	570
15.2	Le graphe : slots qualifiés, arêtes Must et May	570
15.2.1	Les nœuds : des slots qualifiés	570
15.2.2	Les types	570
15.2.3	La force d'une arête	572
15.2.4	L'effet sans deps et son auto-arête	573
15.2.5	Deux bras, une relation : <code>self_slot</code>	574
15.2.6	Déduplication et déterminisme	575
15.2.7	Trois graphes	576
15.3	Trouver les cycles : Tarjan en deux passes	577
15.3.1	Pourquoi deux passes	577
15.3.2	Composantes et reconstruction	578
15.3.3	Ce que prouve un cycle tout-Must	578
15.4	Le kill de convergence, première époque : un seul écrivain	579
15.4.1	Le motif « fetch once » en paire	579
15.4.2	Pourquoi la garde meurt	580
15.4.3	Le reviver : une écriture voisine ranime la garde	580
15.4.4	La réponse d'ADR-018 : un seul écrivain	581
15.5	Qu'est-ce qu'un site ?	581
15.5.1	Qui peut ranimer une garde ?	581
15.5.2	L'effet de montage d'un enfant	583
15.5.3	La vue du site pour la preuve : <code>WriteSite</code>	584
15.6	La preuve multi-site	585
15.6.1	La question, en une phrase	585
15.6.2	Première moitié : préconditions et ensemble tueur	586
15.6.3	Seconde moitié : les revivers	587
15.6.4	Ce qui tient à travers la boucle : <code>Invariance</code>	589
15.6.5	Retour sur les exemples	590
15.6.6	Ce que la preuve garantit	590
15.7	Revivers et plus petit point fixe des sites convergents	591
15.7.1	Un reviver qui tire une fois par requête	591
15.7.2	Le point fixe des sites convergents	592
15.7.3	Pourquoi pas le plus grand point fixe	593
15.7.4	Le kill des arêtes : la partie référence	595
15.8	Plusieurs composants	596
15.8.1	Un setter passé en prop	596
15.8.2	Quand les props bougent : <code>props_hold</code>	597
15.8.3	Ce qui ne remonte pas	598
15.9	De l'arête au diagnostic : la frontière de certification	599
15.9.1	Le bras graphe et <code>must_effect_cycle</code>	599

15.9.2	Le bras self-churn re-dérive must-reach	600
15.9.3	L'ancre churn_cycles des règles déclaratives	601
15.9.4	Construit une fois	601
15.10	Limites connues au commit e67b10a	602
15.10.1	Faux négatifs connus	602
15.10.2	Faux positifs assumés (Warning au plus)	602
15.10.3	Granularité et forme du résultat	602
15.10.4	Dette documentaire	602
	Résumé du chapitre	603
15.11	Exercices	603
16	L'analyse inter-composants et inter-fichiers	606
16.1	Le problème : des props inconnues, un setter qui remonte	607
16.1.1	Un badge qui relance son parent	607
16.1.2	Ce qu'un analyseur intra ne voit pas	607
16.1.3	Deux questions, un sous-système	608
16.2	L'identité d'un composant	609
16.2.1	Trois façons de nommer un composant	609
16.2.2	ComponentId et ComponentTable	610
16.2.3	Le nom d'affichage n'est qu'un rendu	611
16.2.4	L'origine prouvée d'un élément JSX	612
16.2.5	resolve_child : trois réponses	613
16.3	Les racines de l'analyse	615
16.3.1	L'heuristique : ce que personne ne rend	615
16.3.2	-all-roots et -entry	617
16.3.3	Un trou de la collecte, sans danger	618
16.4	L'inlining descendant	618
16.4.1	Le contexte partagé InterCtx	618
16.4.2	eval_comp_app, pas à pas	619
16.4.3	Les props dans le tas	622
16.4.4	Combien de fois un élément est-il évalué ?	623
16.4.5	La récursion : une coupure, pas un havoc	623
16.5	Le flux ascendant : setters, store partagé, havoc	624
16.5.1	Un setter qui porte son propriétaire	624
16.5.2	Le store partagé, et pourquoi il n'y a pas de point fixe de programme	625
16.5.3	Le havoc des setters qui s'échappent	626
16.6	Le cache de composants	627
16.6.1	Une analyse par valeur de props	627
16.6.2	Un « déjà vu », pas une table de résultats	629
16.7	Deux phases	630
16.7.1	La discipline de lecture de phase1_reached	631

16.8	Ce que lit une règle	632
16.8.1	AnalysisResult : le résultat d'un composant	632
16.8.2	ProgramAnalysisResult et ses satellites	632
16.8.3	RuleCtx et ProgramCache	633
16.9	Le graphe de symboles : prévu, réalisé	634
16.10	La dépendance de rendu	634
16.10.1	Pourquoi une seconde analyse	634
16.10.2	Les sources et le treillis Deps	635
16.10.3	L'analyse avant d'un composant	638
16.10.4	La collecte : utiliser ou transmettre	640
16.10.5	Handlers gardés : le fait gated	642
16.10.6	Contextes	642
16.10.7	Bindings de module	643
16.11	Composer sur l'arbre d'éléments	644
16.11.1	L'index de rendu et la résolution d'un site	644
16.11.2	uses : descendre en traduisant la question	644
16.11.3	home_of : le plus petit sous-arbre	644
16.12	Soundness : ce qui est garanti, ce qui fuit	646
	Résumé du chapitre	647
	Exercices	648

V Les règles 651

17 La couche règles : requêtes typées, sévérité certifiée, témoins 652

17.1	Le problème : dire vrai, avec la bonne certitude	653
17.1.1	Deux prédicats qui n'étaient pas complémentaires	653
17.1.2	Deux corrections, et la leçon	654
17.2	Le trait Rule et une première règle	655
17.2.1	Le contrat	655
17.2.2	L'objet de contexte RuleCtx	656
17.2.3	Exemple minimal : conditional-hook	657
17.2.4	Les dix-neuf règles natives	658
17.3	Diagnostic et Severity : le sceau	660
17.3.1	Trois niveaux, deux ordres	660
17.3.2	La structure et son champ scellé	661
17.3.3	Pourquoi un module feuille	661
17.4	Le typestate : Certified, MustResult, May	663
17.4.1	La provenance et le jeton	663
17.4.2	Les verdicts polarisés	665
17.4.3	Les classifieurs totaux	666
17.4.4	Le graphe des types	667

17.5	Les primitives must	669
17.5.1	Dominance sur les sorties : <code>hook_is_conditional</code>	669
17.5.2	Une analyse must avant : <code>must_setter_on_all_paths</code>	671
17.5.3	Le bon quantificateur : <code>all_deps_provably_stable</code>	674
17.5.4	Frapper au point de connaissance : <code>classify_motion</code> puis <code>must_frozen_seed</code>	675
17.5.5	Re-dériver plutôt que croire : <code>must_effect_cycle</code>	678
17.5.6	La preuve de toute la conclusion : <code>must_stale_capture</code>	678
17.5.7	Les autres primitives	679
17.6	Les témoins	679
17.6.1	Un exemple entre deux fichiers	679
17.6.2	Le vocabulaire fermé	680
17.6.3	L'identité du fichier : <code>FileId</code>	682
17.6.4	Les producteurs partagés	683
17.6.5	Du témoin à la position : <code>located</code>	683
17.6.6	Le rendu	684
17.7	Le registre	685
17.7.1	La passe par composant	685
17.7.2	Surcharges et <code>clamp</code>	686
17.7.3	<code>off</code> , <code>allow</code> et la discipline des noms	688
17.7.4	La suspension des assurances	688
17.7.5	Options et validation bruyante	689
17.7.6	La documentation générée	690
17.8	Le cache programme	691
17.9	Les helpers programme	692
17.10	La frontière « walk-free »	694
17.11	La façade, le Tier A et l'ouverture	695
	Résumé du chapitre	695
	Exercices	696
18	Les règles sur l'état	699
18.1	Le socle commun	700
18.1.1	Du nom au slot	700
18.1.2	Applicabilité et assurance	701
18.1.3	Signaler ou affirmer	701
18.2	<code>lazy-init</code> : un appel dans l'initialiseur	702
18.2.1	La condition : un appel syntaxiquement présent	702
18.2.2	Graduer par la nature de l'appel	703
18.2.3	La seule <code>ERROR</code> : un setter dans l'initialiseur	704
18.2.4	Exemples	705
18.2.5	Ce que la règle ne voit pas	706
18.3	<code>unstable-context-value</code> : une valeur neuve à chaque rendu	707

18.3.1	Trouver les providers prouvés	707
18.3.2	Le fait must : FreshEveryRender	708
18.3.3	Exemples	709
18.4	redundant-set-state : écrire ce que l'état contient déjà	711
18.4.1	Le test : stable et stable	711
18.4.2	Les slots contestés	712
18.4.3	Exemples	713
18.5	setter-in-render : écrire pendant le rendu	715
18.5.1	Quels noms sont des setters	715
18.5.2	Les sites et leur phase	716
18.5.3	La preuve : deux must, puis la dominance	717
18.5.4	L'idiome d'ajustement : se taire sur une convergence prouvée	718
18.5.5	Messages et ancrages	719
18.5.6	Exemples	719
18.5.7	Ce que la règle ne voit pas	721
18.6	derived-state : un effet qui recopie un état	723
18.6.1	Cinq conditions	723
18.6.2	Exemples	724
18.6.3	Un faux négatif par collision de labels	725
18.7	state-mutation : muter l'état en place	727
18.7.1	Racines, sites et portées	727
18.7.2	La poursuite d'identité	728
18.7.3	Les deux bras et la preuve	731
18.7.4	Exemples	731
18.8	frozen-initial-state : un état figé sur la première prop	734
18.8.1	Le seed et sa synchronisation : une relation du moteur	734
18.8.2	La prop bouge-t-elle ?	735
18.8.3	Le montage	736
18.8.4	La stratification	737
18.8.5	Exemples	739
18.9	Synthèse	741
	Résumé du chapitre	742
18.10	Exercices	744
19	Étude de cas : infinite-loop et state-lifted-too-high	747
19.1	Le bug visé et l'anatomie de la règle	748
19.1.1	Une boucle que React n'arrête pas	748
19.1.2	Les quatre bras	748
19.2	Bras 1 : le widening comme signal	752
19.2.1	Trois compteurs	752
19.2.2	La frise des itérations	753

19.2.3	Ce que le point fixe laisse à la règle	753
19.2.4	Les deux portes	755
19.2.5	La garde $\forall$ -stable sur les deps	756
19.2.6	Le diagnostic et ses témoins	758
19.2.7	Pourquoi WARNING et pas ERROR	760
19.3	Bras 2 : le setter d'un parent	761
19.4	Bras 3 : les cycles du graphe de churn	763
19.4.1	Quatre programmes, quatre verdicts	763
19.4.2	Ce que la règle lit et ce qu'elle re-dérive	765
19.4.3	Deux passes de Tarjan, un plafond pour le cross	768
19.5	Bras 4 : le self-churn	768
19.5.1	Une référence recrée dans ses propres deps	768
19.5.2	La partition et le verdict	769
19.5.3	Meilleur verdict par slot, et les trois textes	771
19.5.4	Le re-déclenchement sensible aux champs	772
19.5.5	Un faux négatif de soundness : l'écriture d'une valeur dérivée	774
19.6	Synthèse : témoins, pièges, écarts	775
19.6.1	Les témoins produits	775
19.6.2	Faux positifs assumés, faux négatifs observés	775
19.6.3	Mesure sur le corpus	776
19.7	state-lifted-too-high : un état porté trop haut	776
19.7.1	Le bug visé : le <i>prop drilling</i>	776
19.7.2	Le foyer d'un slot	777
19.7.3	L'absence d'usage vaut preuve	778
19.7.4	La règle, ligne à ligne	780
19.7.5	Pourquoi WARNING, et ce qui arrête la cascade	783
	Résumé du chapitre	783
20	Les règles sur les effets	786
20.1	Vue d'ensemble : sept familles, une grille	786
20.1.1	La carte des règles	786
20.1.2	La grille de lecture	787
20.1.3	Ce que React fait d'un tableau de deps	788
20.1.4	Trois états d'un argument de deps, deux manières de le lire	788
20.1.5	L'environnement de sortie du rendu	789
20.2	Les règles Info : annoncer ce que l'on n'a pas vu	790
20.2.1	Un composant sur lequel l'analyse abdique	790
20.2.2	widening-info : la trace du widening	791
20.2.3	analysis-limit : sept renoncements	793
20.2.4	La suspension des assurances	794
20.3	conditional-hook : la règle des hooks par dominance	795

20.3.1	Le problème : un retour anticipé	796
20.3.2	La formulation par dominance	796
20.3.3	Ce que fait le code	796
20.3.4	Niveau et preuve	800
20.3.5	Cas limites observés	800
20.4	always-unstable-deps : quand Object.is échoue à chaque rendu	801
20.4.1	Le problème : un objet frais à côté d'un memo	802
20.4.2	La borne must PerRender	802
20.4.3	Ce que fait le code	803
20.4.4	Ce que la règle tait, et pourquoi	804
20.4.5	Niveau : un fait certain au coût incertain	805
20.5	missing-deps : ce que la closure lit et que les deps taisent	806
20.5.1	Le problème : trois lectures, trois verdicts	806
20.5.2	Chemins d'accès et couverture par préfixe	806
20.5.3	L'algorithme	809
20.5.4	Les trois kills de stabilité	810
20.5.5	Niveau, message et assurance	812
20.5.6	Faux positifs assumés	813
20.5.7	Faux négatifs observés	814
20.6	unnecessary-rerender : l'effet de montage qui écrase l'initialisation	814
20.6.1	Le problème et sa sortie	815
20.6.2	Deux évaluations, deux moments	815
20.6.3	L'idiome SSR	817
20.6.4	Niveau, alias et assurance	818
20.7	missing-cleanup : un enregistrement que rien ne défait	818
20.7.1	Le problème et sa sortie	818
20.7.2	La relation d'enregistrement	819
20.7.3	Le cleanup, fait trois-valué	821
20.7.4	La règle	822
20.7.5	Ce que la règle ne voit pas	823
20.8	stale-closure : prouver toute la conclusion	823
20.8.1	Le problème : le compteur figé	824
20.8.2	Prouver la conséquence	825
20.8.3	Le chemin d'un finding	825
20.8.4	La preuve d'Error : cinq faits must	827
20.8.5	Les silences, et ce qui les justifie	830
20.8.6	Niveaux, limites et assurance	832
20.9	Synthèse : ce que les règles d'effets se partagent	833
	Résumé du chapitre	834
20.10	Exercices	835

21 Rendu et Server Components : wasted-subtree-render, server-component-hook	837
21.1 Deux règles à côté du domaine des valeurs	837
21.2 Le rendu gaspillé : le problème	838
21.3 Ce que la règle consomme : dépendance de rendu et arbre d'éléments	840
21.3.1 Le résumé d'un composant	840
21.3.2 L'index programme	842
21.3.3 L'arbre d'éléments de <code>typing.tsx</code>	842
21.4 L'algorithme de <code>wasted-subtree-render</code> pas à pas	843
21.4.1 Étape 1 : collecter les déclencheurs	844
21.4.2 Étape 2 : la fréquence, un classement et non une preuve	848
21.4.3 Étape 3 : les frères gaspillés	850
21.4.4 Étape 4 : compter les rendus	852
21.4.5 Étape 5 : seuil et message	853
21.5 Des options sur une règle native	853
21.6 Cas limites : ce que les fixtures épinglent	854
21.7 Niveau et soundness d'une affirmation d'absence	856
21.8 Server Components : le graphe serveur de <code>Next.js</code>	857
21.8.1 Les entrées : une convention de noms	858
21.8.2 Le graphe serveur : une atteignabilité avec frontière	858
21.9 L'algorithme de <code>server-component-hook</code>	860
21.10 Pourquoi <code>Warning</code> et pas <code>Error</code>	863
21.11 Pièges et limites connues	864
Résumé du chapitre	865
21.12 Exercices	865
22 Les règles déclaratives et les packs	867
22.1 Pourquoi un langage de règles, et pourquoi pas <code>ESLint</code>	868
22.1.1 Deux demandes d'équipe	868
22.1.2 Le principe de périmètre	869
22.1.3 Trois tiers, un seul est livré	870
22.1.4 Place dans le pipeline	870
22.2 Anatomie d'une règle	871
22.2.1 La plus petite règle utile	871
22.2.2 Le fichier de pack	872
22.2.3 Une grammaire	873
22.2.4 Paramètres et options du consommateur	874
22.2.5 Le gabarit de message	876
22.3 Le système de sortes	877
22.3.1 Un pack qui compile en apparence	877
22.3.2 Ancres et arêtes	878
22.3.3 Gardes	879

22.3.4	Champs	881
22.3.5	Le refus du point de programme	882
22.4	La validation en trois étages	883
22.4.1	Le point d'entrée	883
22.4.2	L'ordre des vérifications	884
22.4.3	Typer une garde	885
22.4.4	Les contrôles transverses	886
22.4.5	Le catalogue des erreurs et des avertissements	887
22.5	L'exécution d'une règle	889
22.5.1	Ce que voit l'exécuteur	889
22.5.2	L'adaptateur vers les relations	890
22.5.3	Énumérer les candidats	891
22.5.4	Une conjonction court-circuitée	891
22.5.5	Exemple : écrivains d'un même tick	892
22.5.6	Position, label, ordre	893
22.6	La sévérité : pin $\sqcap$ polarité	893
22.6.1	Une règle, deux niveaux	893
22.6.2	Garde certifiante, keep et drop	895
22.6.3	L'émission	895
22.7	Les quantificateurs	897
22.7.1	any_of : toutes les branches sont évaluées	897
22.7.2	every : un refus, puis un amendement	898
22.7.3	none : la négation d'un existentiel	901
22.8	La croissance du vocabulaire	906
22.8.1	Mesurer ce qu'on sait dire	906
22.8.2	L'élargissement nommé	906
22.8.3	Les packs livrés	907
22.9	Limites, et une brèche ouverte	907
22.9.1	La brèche de conjonction (#143)	907
22.9.2	Les limites connues	909
22.9.3	Écarts constatés entre documentation et code	909
	Résumé du chapitre	910
	Exercices	910

VI L'outil et sa méthode 913

23 Projets, configuration, driver et CLI 914

23.1	L'invocation vue de l'utilisateur	915
23.1.1	Deux premiers runs	915
23.1.2	Sous-commandes et forme historique	916
23.1.3	Codes de sortie	918

23.2	La configuration	918
23.2.1	Un fichier découvert à la racine	918
23.2.2	Chargement : configuration, registre, packs	920
23.2.3	Précédence : les drapeaux combient, la configuration complète	922
23.2.4	Overrides de règles : off, allowlist, options	923
23.2.5	Sévérité : un plafond, jamais un plancher	924
23.3	Le driver : run_check comme composition	925
23.3.1	Hôte et driver	925
23.3.2	Le flot de run_check	927
23.4	La découverte des fichiers	930
23.4.1	Un exemple : scripts/build n'est pas du build	930
23.4.2	Quels fichiers, quels répertoires	931
23.4.3	Un lecteur de .gitignore pour une seule question	932
23.5	Projets et résolution d'imports	933
23.5.1	Un projet Vite	933
23.5.2	Type de projet, racine de configuration, racine de découverte	934
23.5.3	Les tsconfig : fichier, répertoire, ancêtres	937
23.5.4	Résoudre un spécificateur	939
23.5.5	Pièges de la détection	941
23.6	Parse, lowering et canal d'erreurs	942
23.7	Blind spots : pas de clean bill pour du code non lu	945
23.7.1	Un run rétréci	945
23.7.2	-follow-imports : lire ce que les chemins nommés importent	946
23.8	Next.js et le graphe serveur	948
23.9	Le rendu : humain et JSON	951
23.9.1	Nommer le bon fichier	951
23.9.2	Localisations et attributions	951
23.9.3	Le schéma JSON v2	953
23.10	Le registre des résumés de bibliothèques	954
23.10.1	Connu n'est pas modélisé	954
23.10.2	Le trait et le registre commun	955
	Résumé du chapitre	957
23.11	Exercices	958
24	Distribution : WASM, npm, GitHub Action, CI	961
24.1	Deux artefacts, un cœur : les features Cargo	962
24.1.1	Un workspace, deux crates	962
24.1.2	Construire le paquet : npm/build.sh	963
24.2	Le cœur WASM et l'hôte non fiable	964
24.2.1	Le problème : un hôte qui pourrait mentir	964
24.2.2	Les cinq exports	965

24.2.3	L'enveloppe d'entrée	966
24.2.4	run_inner : tout refaire dans le cœur	966
24.2.5	Deux séquences de chargement, un point de convergence	968
24.3	Le paquet npm	969
24.3.1	Ce que publie npm/package.json	969
24.3.2	L'architecture en couches de npm/lib/	969
24.3.3	Un seul module, deux façons de l'amorcer	970
24.3.4	La table des fichiers : un sur-ensemble	971
24.3.5	Résoudre les packs : deux implémentations	971
24.3.6	reactant packs build : écrire un pack en JS, committer du JSON	972
24.4	La parité native/WASM, testée	974
24.5	L'Action GitHub	976
24.5.1	Trois lignes dans un workflow	976
24.5.2	Ce que fait scripts/gh-action.mjs	976
24.5.3	Les angles morts dans une CI	977
24.5.4	Pièges de l'Action	978
24.6	La CI du dépôt	978
24.7	Versions et stabilité	979
24.7.1	Une version, cinq endroits	979
24.7.2	Ce qui est un contrat	979
	Résumé du chapitre	980
24.8	Exercices	980
25	Tests, corpus et campagnes : la méthode de mesure	982
25.1	Pourquoi mesurer : le silence et son témoin	983
25.1.1	Un test qui passe pour de mauvaises raisons	983
25.1.2	Six altitudes, six portes d'entrée	984
25.2	La sévérité comme contrat	984
25.2.1	Trois niveaux, une définition chacun	984
25.2.2	Un exemple gradué : trois formes, trois verdicts	985
25.3	La pyramide des tests	987
25.3.1	Étage 1 : les tests unitaires et src/test_support.rs	987
25.3.2	Étage 2 : le harnais « source → règles »	988
25.3.3	Étage 3 : le binaire, et le déterminisme	990
25.3.4	Étage 4 : la parité, et les méta-tests	991
25.4	La discipline des paires	992
25.4.1	Le problème : une correction de précision retire des findings	992
25.4.2	Un exemple : une écriture qui règle sa propre garde (#91)	993
25.4.3	Retirer une preuve indue : try/catch/finally (#2)	995
25.4.4	Le test par retrait (<i>gate-by-removal</i>)	996
25.5	Le silence n'est une preuve que si l'on a regardé	997

25.5.1	Un défaut que la sortie cachait	997
25.5.2	Trois propriétés tenues ensemble	997
25.5.3	L'angle mort vu par la mesure	999
25.6	Le corpus : quatorze dépôts épinglés	999
25.6.1	Pourquoi un corpus	999
25.6.2	Épingler : une mesure sur des sources fixes	999
25.6.3	Un seul arbre, et ce que cela coûte	1001
25.7	La métrique : rangées, localisations, digest	1001
25.7.1	Une rangée n'est pas un défaut	1001
25.7.2	La baseline commitée	1002
25.7.3	La porte : comparer, ou refuser de comparer	1004
25.7.4	corpus-diff.py : compter les deux côtés	1005
25.7.5	Ce que la métrique ne voit pas	1005
25.8	Étude de cas : l'erreur de comptage du 3 septembre 2026	1006
25.8.1	Les faits	1007
25.8.2	Le raisonnement	1007
25.8.3	Deux erreurs symétriques	1007
25.8.4	Les ruptures de série	1008
25.9	Le cycle de vie d'un chiffre	1008
25.9.1	Le workflow corpus	1008
25.9.2	Rouge, puis vert : une séquence réelle	1009
25.9.3	Ce que fait l'humain entre le rouge et le vert	1011
25.10	Les campagnes	1011
25.10.1	La campagne FP classique	1011
25.10.2	La campagne « wish-list à l'aveugle »	1012
25.11	L'oracle runtime	1014
25.11.1	Une vérité concrète, comptée	1014
25.11.2	Une paire, mesurée puis analysée	1015
25.11.3	Les onze paires au commit étudié	1015
25.12	Classer ce que l'on ne corrige pas	1016
25.12.1	Défaut ou compromis	1017
25.12.2	Les issues fermées wontfix	1017
25.12.3	La dette du sous-système lui-même	1018
	Résumé du chapitre	1018
25.13	Exercices	1019
26	Histoire et doctrine du projet	1021
26.1	Un sous-système qui n'est pas du code	1022
26.1.1	Quatre lieux pour une décision	1022
26.1.2	Un corpus de décisions	1022
26.2	Le problème de départ	1023

26.2.1	Un prototype, puis une réécriture	1023
26.2.2	Deux CFG par composant, et une sémantique de référence	1023
26.2.3	La frise	1024
26.3	Les premières semaines : un pipeline en un jour	1025
26.3.1	L'ordre des commits est l'ordre du manuscrit	1025
26.3.2	Un choix pragmatique qui a survécu : <code>Unknown</code> $\rightarrow$ <code>skip</code>	1025
26.4	Le tournant du corpus	1026
26.4.1	ADR-015 : un FN documenté devient une détection	1027
26.4.2	ADR-017 : un couple FP/FN	1028
26.4.3	ADR-018 et ADR-019 : le graphe de churn et les témoins	1029
26.5	La codification : <code>CLAUDE.md</code> et ADR-020	1029
26.5.1	Des principes écrits après coup	1029
26.5.2	L'histoire du fichier lui-même	1030
26.5.3	La campagne de dette et ses onze refus	1031
26.6	La polarité devient un type	1032
26.6.1	Un FN corrigé deux fois	1032
26.6.2	Le sceau : <code>Certified</code>	1034
26.6.3	Les packs, et la même idée appliquée aux règles des utilisateurs	1035
26.7	Les relations : un fait, un endroit	1035
26.7.1	Le symptôme : deux marches qui divergent	1035
26.7.2	La vague de septembre	1036
26.7.3	ADR-042 : la frontière et son cliquet	1037
26.8	Identité et attribution	1038
26.9	Les cascades de rendu : une seconde analyse plutôt qu'un champ	1040
26.10	La méthode	1040
26.10.1	Le cycle d'une décision	1040
26.10.2	La mesure comme juge, et la mesure corrigée	1041
26.10.3	Les ADR rétrogradés et les numéros réutilisés	1041
26.10.4	Les refus citables	1042
26.10.5	Le graphe des ADR	1042
26.10.6	Quand l'ADR et le code divergent	1043
26.11	Les fils rouges	1043
	Résumé du chapitre	1044
26.12	Exercices	1044
27	Limites connues, défauts observés et perspectives	1046
27.1	Deux registres, et ce que le projet documente déjà	1046
27.1.1	Défaut ou compromis : la grille de lecture	1046
27.1.2	La page <code>docs/limitations.md</code>	1047
27.1.3	Les limites tranchées « on ne corrige pas »	1047
27.2	Les défauts observés au commit <code>e67b10a</code>	1049

27.2.1	L1 — L’updater fonctionnel sur le chemin programme	1050
27.2.2	L2 — Un appel de setter hors position d’instruction	1051
27.2.3	L3 — useReducer : l’action écrite à la place de l’état	1052
27.2.4	L4 — Les écritures d’un nettoyage n’entrent pas dans le store	1053
27.2.5	L5 — $\top$ par join n’est pas un signal	1053
27.2.6	L6 — Une chaîne qui grandit sans borne	1054
27.2.7	L7 — Le return d’un appelé greffé sans liaison	1055
27.2.8	L8 — Deux instances, un seul résultat	1056
27.2.9	L9 — Un alias de setter appelé pendant le rendu	1057
27.2.10	L10 — Le provider React 19	1058
27.2.11	L11 — Les formes que la détection ne reconnaît pas	1059
27.2.12	L12 à L14 — Trois faux positifs	1061
27.2.13	Écarts entre ADR, documentation et code	1063
27.2.14	Récapitulatif	1064
27.3	Causes communes et dette	1065
27.3.1	Quelques racines pour quatorze constats	1065
27.3.2	L’état de la dette	1066
27.4	Perspectives de recherche	1067
27.4.1	Une sémantique concrète écrite	1067
27.4.2	Réducteurs, contexte et cycle de vie	1067
27.4.3	Précision relationnelle et information de transition	1067
27.4.4	Une ERROR pour la divergence de valeur	1068
27.4.5	Le vocabulaire des packs	1068
	Résumé du chapitre	1068
27.5	Exercices	1068

VII Annexes 1070

A	Fiches des décisions d’architecture (ADR) 1071
A.1	ADR-001 — La sémantique concrète : React-tRace 1072
A.2	ADR-002 — Treillis de stabilité et trois stores 1072
A.3	ADR-003 — Une IR dédiée à base de CFG 1073
A.4	ADR-004 — render_cfg et effect_cfg séparés 1073
A.5	ADR-005 — Portée intra-procédurale et registre de hooks 1074
A.6	ADR-006 — Les règles en post-passe 1074
A.7	ADR-007 — Requêtes inter-domaines 1075
A.8	ADR-008 — StateValue : enum plat et TypedStateStore 1075
A.9	ADR-009 — Traversée sémantique des callbacks 1076
A.10	ADR-010 — Modèle de tas par site d’allocation 1077
A.11	ADR-011 — SourceRange et notes 1077
A.12	ADR-012 — Analyse inter-composants 1078

A.13 ADR-013 — Analyse cross-file	1078
A.14 ADR-014 — Widening à seuils ; narrowing abandonné	1079
A.15 ADR-015 — Domaine produit sur sortes disjointes	1079
A.16 ADR-016 — CLI, sortie JSON, détection de projet Vite	1081
A.17 ADR-017 — Stabilité versionnée	1081
A.18 ADR-018 — Graphe de churn multi-effets	1082
A.19 ADR-019 — Chaînes de témoins typées	1083
A.20 ADR-020 — Dette technique : onze non-changements délibérés	1083
A.21 ADR-021 — Surface de requêtes typée	1084
A.22 ADR-022 — Frontends de règles, packs déclaratifs, WASM	1086
A.23 ADR-023 — Croissance du vocabulaire Tier-A	1086
A.24 ADR-024 — Attribution des findings inlinés	1087
A.25 ADR-025 — Fall-through = return undefined	1087
A.26 ADR-026 — Projets Next.js	1089
A.27 ADR-027 — Relation slot-writer, résumés de phase, provenance	1089
A.28 ADR-028 — writers par site, Updater, same_tick	1090
A.29 ADR-029 — Ancre churn_cycles	1091
A.30 ADR-030 — Lignes render-setter qualifiées par propriétaire	1092
A.31 ADR-031 — Relation slot_seeds	1092
A.32 ADR-032 — Relation context_consumers	1093
A.33 ADR-033 — Bit d'exactitude de la chasse aux liaisons	1093
A.34 ADR-034 — Relation d'enregistrement, une table de registrars	1094
A.35 ADR-035 — Frontière de phase await	1095
A.36 ADR-036 — Relation d'appels	1096
A.37 ADR-037 — Relation de lecture de slot	1096
A.38 ADR-038 — Une écriture est une écriture où qu'elle soit écrite	1097
A.39 ADR-039 — Une liaison synthétique a une position réelle	1098
A.40 ADR-040 — Identité de composant = ComponentId	1098
A.41 ADR-041 — Dépendance de rendu	1099
A.42 ADR-042 — Les relations sont des produits du moteur	1100
Résumé du chapitre	1102
Exercices	1103
B Glossaire	1104
Glossaire	1105
Résumé du chapitre	1113
B.1 Exercices	1113
C Catalogue des règles	1115
C.1 Lire ce catalogue	1115
C.1.1 Deux espaces de noms	1115
C.1.2 Ce que reactant rules et reactant explain affichent	1116

C.2	Les dix-neuf règles natives	1116
C.2.1	Les règles sur l'état (chapitre 18)	1117
C.2.2	Le point fixe et le prop drilling (chapitre 19)	1117
C.2.3	Les règles sur les effets et les hooks à deps (chapitre 20)	1119
C.2.4	Le rendu et les Server Components (chapitre 21)	1119
C.3	Les règles des packs livrés	1119
C.3.1	guardrails : le pack de démonstration	1119
C.3.2	Les packs community/* : des preuves de vocabulaire, pas des règles à activer	1120
C.3.3	La sortie de reactant rules	1120
	Résumé du chapitre	1121
C.4	Exercices	1121

Table des figures

1.1	Le cycle d'exécution d'un composant React. Les flèches pointillées sont les retours possibles : un effet ou un handler qui appelle un setter planifie un nouveau rendu.	3
1.2	Valeur abstraite de l'état n au fil des tours abstraits (déroulé simplifié ; le détail des itérations et le seuil du widening sont au chapitre 13). Les valeurs finales $[0, +\infty)$ et $[0, 3]$ sont celles que le moteur calcule effectivement au commit e67b10a.	9
1.3	Le pipeline d'une invocation <code>reactant check</code> . Chaque boîte est une couche du code ; sur les flèches, le type principal qui passe d'une couche à la suivante. Le chapitre 4 détaille chaque étape et ses frontières.	12
2.1	Chronologie d'un tour de React. Un setter appelé dans un effet relance immédiatement un tour (flèche pleine) : c'est le mécanisme de la boucle infinie d'effets. Un setter appelé dans un handler ne relance un tour qu'après un événement extérieur (flèche tiretée). Un setter appelé pendant le rendu fait jeter le rendu et recommencer.	28
2.2	Déroulement concret de <code>SelfCounter</code> . Chaque rendu est suivi d'un commit puis de l'effet, car la <code>dep s</code> a changé. Le quatrième effet tourne encore (la <code>dep</code> a changé de 2 à 3) mais n'écrit rien : aucun tour n'est relancé et React attend un événement. Pour <code>Inf</code> , la chaîne ne s'arrête pas.	37
2.3	Le treillis <code>Stability</code> (ADR-017). À gauche, la chaîne <code>may</code> « ne change qu'aux sets de S », dont la hauteur est bornée par le seuil <code>VERSIONED_LABELS_THRESHOLD = 4</code> ; à droite, le fait <code>must</code> « neuf à chaque rendu ». Leur <code>join</code> est $\top$.	44
2.4	Les modes de <code>React-tRace</code> et ses quatre transitions. La flèche d'entrée est <code>STEPI-NIT</code> (évaluation de l'expression principale et initialisation de l'arbre). La boucle infinie d'effets est le cycle <code>rendered</code> $\rightarrow$ <code>check</code> $\rightarrow$ <code>rendered</code> qui ne passe jamais par <i>event loop</i> ; un handler, lui, n'est exécuté qu'en mode <i>event loop</i> .	55
3.1	Sur-approximation. Le concret $\mathcal{C}$ (ensemble des états réellement atteignables) est inclus dans l'abstrait A . Tant que A évite la zone de défaut, le programme est prouvé correct ; si seul A la touche, l'analyse émet un faux positif. Jamais un défaut de $\mathcal{C}$ ne peut échapper à A .	69
3.2	Deux treillis plats. À gauche, <code>BoolVal</code> (<code>src/domains/impls/bool_val.rs</code>) avec la concrétisation de chaque élément (section 3.3) ; à droite, <code>SetterVal</code> (<code>src/domains/impls/setter_val.rs</code>), dont les éléments intermédiaires nomment un setter précis (composant, label de hook). Deux éléments intermédiaires distincts sont incomparables ; leur <code>join</code> est $\top$.	70
3.3	Le treillis des intervalles, tronqué. Un intervalle est au-dessus de ceux qu'il contient. La chaîne $[0, 0] \sqsubseteq [0, 1] \sqsubseteq [0, 2] \sqsubseteq \dots$ ne s'arrête jamais : la hauteur est infinie, ce qui obligera au widening (section 3.6).	76

3.4	CFG schématique du corps de l'effet de liste 3.10. L'arc en pointillés est l'arc arrière de la boucle; le lowering le marque <code>EdgeKind::Back</code> (chapitre 7). La numérotation des blocs est celle de l'exposé, pas nécessairement celle du lowering. . .	78
3.5	CFG schématique du rendu de <code>Profile</code> . $\text{Dom}(B_1) = \{B_0, B_1\}$ et $\text{Dom}(B_2) = \{B_0, B_2\}$: B_2 , qui porte les hooks, ne domine pas la sortie B_1	92
4.1	Le pipeline de <code>reactant check</code> , module par module. À gauche, de la ligne de commande à l'IR; à droite, de l'IR à la sortie. Sur chaque flèche, le type principal transmis. L'AST d'oxc n'existe qu'entre le parse et le lowering, et seulement le temps d'un fichier : aucune flèche ne franchit la ligne pointillée avec un AST. .	108
4.2	Les cinq couches de <code>reactant-analyzer</code> (de bas en haut, dans l'ordre du flot) et, à droite, la frontière entre deux couches consécutives avec ce qui la fait respecter.	117
5.1	Le front-end d'un fichier. Les six fonctions d'entrée lisent le même <code>Program</code> indépendamment; l'AST est libéré à la fin de l'itération. La passe inter-fichiers ne voit plus que des structures possédées.	136
5.2	Arbre de décision de <code>is_component</code> (<code>component_detector.rs</code> , l. 47–73). La règle 1 est subsumée par le test de majuscule; la règle 4 est postérieure à ADR-3 (#122).	146
5.3	Les trois classes du front-end. Colonnes : le nom; lignes : ce que les règles 2 à 4 trouvent dans le corps et le type. Les cases en tirets ne sont vues par aucun détecteur.	149
5.4	Arbre de décision de <code>classify_callee</code> (<code>hook_extractor.rs</code> , l. 562–637). Toute autre forme de <code>callee</code> est un appel ordinaire.	158
5.5	Le graphe d'import de <code>tests/fixtures/next_project</code> . En orange, l'ensemble <code>reachable_from(départs, Some("use client"))</code> : les modules serveur. Le parcours s'arrête sur <code>counter.tsx</code> , qui déclare la frontière, et l'exclut.	163
6.1	Le <code>render_cfg</code> du compteur (liste 6.1) et sa table de hooks. Le bloc B1 et le bloc B3 sont deux sorties; le corps du handler est un CFG à part.	176
6.2	Diagramme de Hasse de <code>Arity</code> ordonnée par concrétisation. Chaque <code>Exact(n)</code> est minimal; la chaîne des <code>AtLeast</code> descend sans fin.	195
7.1	Le lowering dans le pipeline. La construction du CFG est commune aux trois classes de fonctions; l'extraction des hooks, des handlers et des subscriptions ne s'applique qu'aux composants et aux hooks custom. À droite de la ligne rouge, aucun consommateur ne voit l'AST.	217
7.2	Gabarit du <code>if</code> . Les trois blocs sont réservés avant d'abaisser les branches; une branche qui se termine (à droite, <code>return</code>) n'a pas d'arête vers la jonction.	224
7.3	Les deux diamants. Le temporaire <code>__tN</code> est tiré du compteur du constructeur; la valeur de l'expression est <code>Var(__tN)</code> dans la jonction.	235
7.4	Les gabarits des quatre boucles. Les arêtes arrière (en bleu) sont les seules où le point fixe applique le widening. Un <code>break</code> va toujours à X par une arête <code>Unconditional</code>	238
7.5	Gabarits du <code>switch</code> et du <code>try</code> . Les dispatchs du <code>switch</code> portent <code>Lit(true)</code> , que le point fixe ne raffine pas; les arêtes pointillées (<code>fall-through</code>) n'existent que pour une case sans <code>break</code> , les autres sautent à X. Le <code>try</code> branche sur la sentinelle <code>⊥</code> et place le <code>finally</code> sur la jonction.	244

7.6	Gabarit de <code>const r = await f(x)</code> . Le contrôle est inconditionnel — dominance et accessibilité sont inchangées —, mais le successeur s'exécute après une suspension.	252
7.7	Le compteur avant et après les passes d'extraction. Le <code>let</code> du temporaire d'état disparaît ; ses deux projections sont réécrites en <code>StateVal(0)</code> et <code>StateSetter(0)</code> ; la table des hooks reçoit l'état (label 0) puis le handler (label 1) ; la provenance n'a qu'une ligne, car un handler n'en a jamais.	253
8.1	Avant et après la greffe de <code>clamp</code> dans <code>Counter</code> . Les blocs de l'appelé (fond coloré) sont décalés de 1 (le plus grand bloc de l'appelant plus un) ; la jointure B5 reçoit les instructions qui suivaient l'appel et l'ancien terminateur ; chaque <code>return e</code> devient <code>let shown = e</code> suivi d'un saut vers la jointure ; le paramètre est lié par <code>let n#0 = count</code> en tête de l'entrée.	274
9.1	Les huit composantes du produit <code>StateValue</code> . En bleu, les treillis étudiés dans ce chapitre ; en gris, ceux du chapitre 10 (la stabilité et les trois bits de sorte).	305
9.2	À gauche, <code>BoolVal</code> , dont la concrétisation est une bijection avec $\mathcal{P}(\{\text{true}, \text{false}\})$. À droite, le treillis plat engendré par <code>flat_lattice!</code> pour un ensemble quelconque d'éléments intermédiaires, avec le bras de <code>partial_cmp</code> qui établit chaque relation.	309
9.3	Un fragment de <code>StrConst</code> (quelques arêtes seulement). Les ensembles de taille 1 à 4 sont ordonnés par inclusion ; au-delà, le join saute à $\top : \{a, b, c\} \sqcup \{d, e\} = \top$ (flèche), car l'union a cinq éléments.	316
9.4	Un fragment du treillis des intervalles. Les points sont les atomes ; le join de $[0, 0]$ et $[2, 2]$ est $[0, 2]$, qui contient aussi 1 (convexité). Toute chaîne $[0, 0] \sqsubset [0, 1] \sqsubset \dots$ monte sans fin vers $[0, +\infty]$: hauteur infinie.	321
9.5	Chronologie du slot <code>count</code> dans la boucle externe. Aux tours 1 et 2, le nouvel état remplace l'ancien ; au tour 3 (<code>iteration = 3</code>), il est élargi à seuils : la borne haute saute au plus petit seuil ≥ 3 , qui est $+\infty$ pour <code>Unbounded</code> ($T = \{0, 1\}$) et 10 pour <code>Guarded</code> ($T = \{0, 1, 10\}$). Au tour 4, le nouvel état est inclus dans l'ancien : post-point fixe ($\checkmark$).	332
10.1	Le même join dans les deux domaines. À gauche, la somme plate d'ADR-008 : deux sortes différentes n'ont que $\top$ pour majorant commun (les sous-treillis de chaque sorte sont repliés en un nœud). À droite, le produit d'ADR-015 : chaque sorte garde sa composante, le join se fait composante par composante (seules deux composantes sont montrées).	347
10.2	Le carré <code>may/must</code> de l'ADR-017. Chaque point est une paire (borne <code>may</code> , borne <code>must</code>) sur la trace de changement C ; seules les paires avec <code>must</code> $\leq$ <code>may</code> ont un sens. Les quatre points encadrés en bleu sont retenus (avec $\perp$, non figuré) ; les deux points en pointillés, « doit changer aux sets », sont écartés. Le join affaiblit les deux bornes : il va vers la droite et vers le bas.	360
10.3	Diagramme de Hasse de <code>Stability</code> (d'après le commentaire de <code>src/domains/impls/stability.rs</code> , l. 20–30). À gauche, la chaîne <code>may</code> ordonnée par inclusion des ensembles de slots, bornée par <code>VERSIONED_LABELS_THRESHOLD = 4</code> ; à droite, le fait <code>must PerRender</code> . Les deux branches ne se rejoignent qu'en $\top$ et en $\perp$. Les étiquettes <code>b</code> et <code>c</code> abrègent <code>Versioned(b)</code> et <code>Versioned(c)</code> .	362

10.4	La double vue de l'état sur <code>ObjectStateDep</code> . Le store range le join des valeurs <i>écrites</i> , deux allocations fraîches ; toute lecture du slot rend la vue <i>inter-rendus</i> , ce que <code>Object.is</code> compare d'un rendu à l'autre. Les consommateurs qui comparent des valeurs écrites lisent le store ; ceux qui raisonnent sur les deps lisent l'évaluation.	366
11.1	Du moteur à l'évaluateur. Les boîtes blanches appartiennent au moteur (chapitres 12 et 16), les bleues à ce chapitre. La flèche tiretée est la récursion des corps imbriqués : un corps de callback ou de <i>functional updater</i> est exécuté instruction par instruction par le même <code>exec_full_stmt</code> , à la profondeur $d + 1$, plafonnée à <code>MAX_INLINE_DEPTH = 3</code> .	392
11.2	La carte des stores. Les flèches vers l'environnement sont des lectures faites par l'évaluateur ; la flèche « appel de setter » est l'écriture faible d'un setter local (section 11.6), la flèche tiretée celle d'un setter reçu d'un ancêtre ; la flèche « recalcul » est la mise à jour des mémos depuis l'environnement de sortie du rendu. Le store partagé est le seul dont la portée dépasse le composant.	407
11.3	Le tas de Heapy, d'après la sonde. Les flèches pleines vont d'une variable à ses sites ; les tiretées sont les lectures de membres qui ont produit une liaison. <code>bag.onClear</code> lit la valeur <i>propre</i> du membre, <code>Stable</code> , et non celle du conteneur ; <code>nested.inner.f</code> s'arrête faute de site pour le littéral intérieur, et <code>g</code> vaut $\top$.	425
11.4	Le défaut D1 sur le troisième callback de liste 11.38. <code>env_at</code> est indexé par le bloc <i>destinataire</i> , mais chaque bloc lit la table sous les noms de ses <i>prédécesseurs</i> : il voit l'entrée de son prédécesseur, soit l'état « d'un bloc en retard ».	433
12.1	Le diamant de <code>diamond_joins_at_merge_point</code> , annoté des environnements calculés par <code>analyze_cfg</code> .	446
12.2	<code>counting_loop_cfg</code> et ses environnements finaux (composante numérique de <code>i</code>). L'arc arrière est en pointillé.	450
12.3	Un losange suivi d'une boucle, et un bloc orphelin. Sous chaque bloc, son ensemble de dominateurs, tel que <code>compute_dominators</code> le rend (relevé à la sonde). RPO : $[0, 2, 1, 3, 5, 4]$.	461
13.1	Chronogramme de Runaway : les tours de React (en haut, infinis) et les tours de la boucle externe (en bas, quatre). Le store abstrait S_k sur-approxime toutes les valeurs de <code>x</code> atteignables en au plus k tours ; après le widening, S_3 les contient toutes.	474
13.2	Le flot de <code>analyze_component_impl</code> (<code>src/engine/fixpoint.rs</code>). Les numéros de ligne sont ceux du commit <code>e67b10a</code> . La préparation s'exécute une fois ; la boucle externe répète les quatre passes jusqu'à $N \sqsubseteq S$; la post-convergence recalcule ce que les règles liront.	478
14.1	Les trois couches d'ADR-42. Le moteur interprète ; la couche des relations dérive des faits du résultat convergé, chacun calculé une fois, avec sa polarité ; les règles transforment ces faits en diagnostics. La frontière basse est tenue par le cliquet <code>tests/layer_boundary.rs</code> .	517

14.2	La marche sur l'exemple 14.3. Le corps de l'IIFE est entré en Sync au site d'appel; <code>const v = await load()</code> est abaissé en <code>Let __t0 = load()</code> , fin de bloc, puis <code>Let v = __t0</code> dans le bloc suivant (blocs tels que les imprime la sonde <code>/tmp/verif07/irprobe</code>), et l'arête <code>EdgeKind::Await</code> qui les relie fait basculer la marche en Deferred pour le bloc 1; la fonction retournée est marchée en Cleanup. Les trois sites héritent du bloc de provenance 0, seul identifiant qui ait un sens dans le CFG de l'effet.	529
14.3	Provenance d'une écriture splicée (exemple 14.6). Blocs tels que la sonde les imprime. Le splice émet le préfixe de paramètres <code>let setter#0 = setN</code> en tête du premier bloc du callee, avec le span du site d'appel (ligne 5 de <code>App.tsx</code> , ADR-39 §2), enregistre la plage de blocs du callee, et <code>resolve_setter_aliases</code> rattache <code>setter#0</code> au slot 0. Une chaîne <code>putState → helper → setX</code> donnerait <code>Via(["putState", "helper"])</code>	540
14.4	Le treillis Freshness. Ce n'est pas un treillis d'approximation au sens du chapitre 3 : <code>Not</code> et <code>Fresh</code> sont deux affirmations <code>must</code> opposées, <code>Maybe</code> leur join.	542
14.5	L'arbre de décision de <code>pair</code> (<code>src/engine/registrations.rs</code> , l. 393–455). L'absence de cleanup est tranchée <i>avant</i> toute comparaison : « it is the one case where the absence is total ». Les disposers reconnus forment un ensemble fermé, <code>DISPOSERS</code> (<code>src/engine/registrations.rs</code> , l. 478–490).	554
14.6	Les chaînes de gardes des deux sites de l'exemple 14.11 (numéros de blocs de la sonde). Le site de <code>setN</code> n'a aucune garde : rien ne peut mourir, <code>converges_once_written</code> rend <code>false</code> et la règle passe à la dominance de sortie.	557
15.1	Trois graphes de churn (<code>two_effects.tsx</code> , <code>three.tsx</code> , <code>dag.tsx</code>). Les nœuds sont des slots qualifiés du même composant ; <code>eN</code> est le label du hook porteur. Une arête pleine est <code>Must</code> , une arête pointillée <code>May</code>	576
15.2	Le corps du premier effet de l'exemple 15.10 après lowering (blocs de la sonde IR), la chaîne de gardes du bloc 3 et l'ensemble tueur de l'écriture différée. Le site différé n'a pas de block mais hérite du <code>guard_block</code> de l'instruction qui le planifie.	592
15.3	Le graphe de <code>props_move.tsx</code> : un cycle de trois arêtes <code>May</code> dont un nœud appartient à <code>Parent</code> (<code>cross_component</code>). <code>Child</code> porte les trois arêtes ; comme un de ses déclencheurs nomme un slot du parent, <code>props_hold</code> est faux et les faits sur <code>p</code> ne sont pas lus.	598
16.1	Le pipeline programme. L'IR de tous les fichiers est abaissée d'abord ; le registre interne l'identité des composants ; la stratégie choisit les racines ; <code>analyze_program</code> analyse les racines en inlinant leurs enfants à la volée (<code>eval_comp_app</code>), puis balaie les composants non atteints. Les résumés de dépendance de rendu sont calculés à la demande, une fois par programme, dans la couche règles.	609
16.2	Diagramme de séquence de l'évaluation d'un élément <code><Child/></code> sous un <code>InterCtx</code> . Les retours sont en pointillés. L'écriture de l'enfant dans le store partagé n'est lue par le parent qu'au test de convergence de <i>sa</i> boucle, qui relance un tour si elle fait monter son état.	622
16.3	Les deux phases sur l'exemple de la liste 16.3. En phase 1, les racines <code>App</code> et <code>Page</code> sont analysées ; <code>App</code> inline <code>Widget@b</code> par son origine ; la référence nue de <code>Page</code> est ambiguë, donc havoquée. Personne n'atteint <code>Widget@a</code> , que la phase 2 analyse seul. L'ensemble <code>phase1_reached</code> (cadre pointillé) est photographié entre les deux.	631

16.4	Le treillis Deps pour trois sources : parties ordonnées par inclusion, plus un sommet $\top$ qui vide <code>set</code> et <code>gated</code> . Le treillis est fini, donc la montée s'arrête sans <code>widening</code> . La composante <code>gated</code> va dans le sens dual : elle ne peut que rétrécir quand on joint.	636
16.5	L'arbre d'éléments de <code>drill.tsx</code> et la question <code>Slot(0)</code> , <code>Setter(0)</code> d'App, traduite à chaque saut dans le repère de l'enfant. Les trois composants du chemin (bord orangé) ne font que transmettre ; <code>Field</code> (bleu) est le <i>home</i> ; <code>Content</code> est un frère qui re-rend aussi (<code>siblings = 1</code>).	645
17.1	L'arbre des modules de <code>src/rules/</code> . Un item privé est visible de son module et de ses descendants, jamais de ses sœurs ni de leurs enfants : ni la règle <code>conditional_hook</code> ni aucune autre ne voit le champ <code>severity</code> , les constructeurs privés de <code>Diagnostic</code> ou <code>Certified::mint</code> . Si <code>Diagnostic</code> était défini dans <code>crate::rules</code> lui-même, toute règle, descendante, les verrait.	662
17.2	Le graphe des types de la sévérité. En bleu, le seul chemin vers une <code>ERROR</code> : une primitive <code>must</code> frappe un jeton, que <code>Diagnostic::error</code> consomme. En gris, les chemins libres : un fait <code>may</code> ou une évidence dégradée ne mène qu'à <code>warn</code> ou <code>info</code> , et <code>clamp</code> ne fait que descendre. En rouge, les flèches interdites, avec le code d'erreur que <code>rustc</code> rend réellement (exemple 17.1) : passer un fait <code>may</code> à <code>error</code> (E0308, types incompatibles), frapper un jeton hors de <code>query.rs</code> (E0624, fonction privée ; E0451 pour un littéral de structure), remonter une sévérité en écrivant le champ (E0616, champ privé) ou par <code>with_severity</code> (E0624).	668
17.3	Un CFG idéalisé de corps d'effet avec une boucle. L'appel <code>setB(a * 2)</code> est placé soit dans <code>B0</code> (avant la boucle), soit dans <code>B2</code> (dans la boucle), soit dans <code>B3</code> (après). Le lowering réel produit davantage de blocs, mais la même forme.	673
17.4	La séquence de <code>check_component</code> . <code>safe_check</code> ne voit que la sortie <i>brute</i> de <code>check</code> , avant tout filtre ; la suspension lit les diagnostics <i>non filtrés</i> ; le <code>clamp</code> précède le tri, dont la sévérité est une clé. Le filtre <code>-info</code> vient ensuite, dans le driver. . . .	686
17.5	Le pipeline du <code>clamp</code> : $final = pin \sqcap construit$, calculé sur le rang. À gauche, une descente honorée ; à droite, une montée demandée et refusée par construction. Les deux cas sont ceux de la configuration <code>/tmp/ch17/cfg/reactant.config.json</code>	687
18.1	CFG de rendu de <code>Unconditional</code> et <code>Conditional</code> (liste 18.21), tels que les imprime la sonde d'IR (élagués). La preuve d' <code>ERROR</code> demande que le bloc de l'appel domine <i>toutes</i> les sorties atteignables. Dans <code>Conditional</code> , <code>B0</code> est le dominateur immédiat de <code>B1</code> , <code>B2</code> et <code>B3</code> ; <code>B1</code> ne domine aucune sortie.	718
18.2	Les strates de <code>frozen-initial-state</code> (<code>src/rules/impls/frozen_initial_state.rs</code> , l. 97–283). Les rétrogradations ne font que descendre ; seule la dernière étape consomme le jeton de <code>classify_motion</code> . Le message ne dépend que de la branche <code>Proven/Unproven</code> , pas des rétrogradations.	738
19.1	Le chemin d'un effet à travers les quatre bras. Les deux premiers filtres (montage seul, $\forall$ -stable) ne concernent que les bras 1 et 2 ; le graphe de <code>churn</code> a appliqué ses propres filtres au moment de construire ses arêtes (chapitre 15). Les flèches pointillées sont les deux canaux de déduplication.	751
19.2	Frise des itérations de la boucle externe pour <code>Counter</code> et <code>Bounded</code> . Les deux slots entrent dans <code>widen_trace</code> au même tour ; seule la valeur rejouée depuis $\perp$ après convergence les distingue : $[1, +\infty]$ est non bornée, $[1, 10]$ est bornée.	753

19.3	Les graphes de churn des quatre composants de liste 19.18, avec le verdict du bras 3. <code>eN</code> est le label du hook porteur ; le bloc est celui de l'écriture dans le corps de l'effet (bloc 0 = entrée = tous les chemins). La ligne <code>setB(null)</code> de <code>MultiWriter</code> n'est pas fraîche et ne porte pas d'arête, mais elle est un <i>site</i> qui empêche le kill de l'arête $a \rightarrow b$	765
19.4	L'arbre d'éléments de <code>drill.tsx</code> vu par <code>state-lifted-too-high</code> : le chemin du foyer (bord orangé) descend d' <code>App</code> à <code>Field</code> (bleu), le foyer ; <code>Content</code> (pointillé) est un frère compté dans <code>siblings</code> . En dessous, les quantités que la règle calcule et le seuil qu'elles franchissent. La construction du chemin par <code>home_of</code> est détaillée au chapitre 16.	777
20.1	Dominance et hooks conditionnels. À gauche, <code>Profile</code> (liste 20.12) ; à droite, un losange qui se referme avant le hook. Les sorties (blocs <code>Return</code> atteignables) sont en trait épais. La numérotation des blocs est illustrative : aucune sortie de la CLI n'expose les identifiants réels.	797
20.2	Les préfixes de deux chemins lus par <code>Search</code> (liste 20.21), annotés de la stabilité de leur valeur dans l'environnement de sortie. Un cadre vert est un préfixe stable, qui suffit à éteindre le finding ; un cadre rouge, un préfixe qui peut changer. Le préfixe 0 (la racine) relève du kill 1 ; le kill 2 examine les préfixes $n = 1, \dots, p $	811
20.3	La table <code>REGISTRARS</code> en matrice <code>Firing × Timing</code> . Les lignes « méthodes seulement » (<code>method_only</code>) ne valent que sous la forme <code>recv.nom(...)</code> ; <code>then</code> , <code>catch</code> , <code>finally</code> aussi. <code>queueMicrotask</code> et les continuations de promesse n'ont pas de <code>teardown</code>	820
20.4	La capture figée de <code>Timer</code> (liste 20.38). Le callback <code>cb</code> est créé une fois, au rendu 1 ; toutes ses exécutions (flèches pointillées) lisent la même copie $n = 0$. L'état passe à 1 au premier tick, puis n'avance plus.	825
20.5	La conjonction de <code>must_stale_capture</code> (liste 20.41). À droite, pour chaque feuille, le test de <code>tests/stale_closure.rs</code> qui la rend fausse (toutes les autres restant vraies) et exige un <code>WARNING</code> au lieu de l' <code>ERROR</code> . La feuille 2 est aussi réfutée par <code>set_timeout_self_write_warns</code> (l. 200).	829
21.1	L'arbre d'éléments de <code>typing.tsx</code> et la question <code>Relevance {Slot(0)}</code> . Le handler de l' <code><input></code> écrit le slot (flèche pointillée) ; <code><Preview></code> est touché (bord orangé) ; <code><ExpensiveTree></code> ne l'est pas, se résout, et coûte son propre rendu plus un <code><Row></code> compté une fois pour toute la liste (bleu).	843
21.2	Le filtre de <code>wasted_siblings</code> sur un site du propriétaire. Toute sortie à droite est un refus de conclure ; seule la sortie du bas produit une affirmation.	851
21.3	Le graphe de modules de <code>/tmp/ch21/nx/</code> . Fond bleu : <code>server_modules</code> . La frontière "use client" (pointillés orangés) arrête le parcours : <code>search.tsx</code> n'est pas visité. L'arête pointillée de <code>search.tsx</code> vers <code>price.tsx</code> n'est jamais suivie depuis le serveur ; <code>price.tsx</code> est serveur par l'arête directe de la page.	860
22.1	Le flot d'une règle déclarative, de son texte à son diagnostic. Les étiquettes <code>En</code> et <code>Wn</code> renvoient au catalogue des erreurs et avertissements de la section 22.4 : chaque étage rejette ce qu'il sait rejeter, avec un chemin JSON exact.	870

22.2	Le diagramme entité-relation des sortes. En bleu, les sortes que lie l'ancre <code>hook_calls</code> (selon <code>kind</code>) ou <code>elements</code> ; en blanc, les sortes d'éléments; chaque flèche est une arête navigable par <code>forEach</code> ou quantifiable par <code>none</code> (<code>every</code> ne quantifie que <code>deps</code>). Les sortes <code>Hook(ref)</code> et <code>Hook(None)</code> n'ont aucune arête. Une même sorte d'élément peut être atteinte par deux chemins : <code>Call</code> est aussi la sorte de l'ancre <code>render_calls</code> , <code>JsxProp</code> celle de <code>jsx_props</code>	879
23.1	Le flot de <code>run_check</code> . À droite, en rouge, les trois points de sortie <code>EXIT_USAGE</code> du driver (les autres codes 2 naissent avant, dans l'hôte : <code>clap</code> , <code>configuration</code> , <code>overrides</code>). À gauche, en pointillés, les trois étapes qui alimentent la liste des <code>blind spots</code>	928
23.2	L'arbre de décision de <code>build_context</code> . En orange, les deux feuilles qui posent un <code>alias_warning</code> , que le driver transforme en <code>blind spot unresolved-aliases</code>	936
23.3	Les trois niveaux de la recherche des <code>paths</code> . Traits pleins : la chaîne <code>extends</code> , suivie en premier, où un parent à <code>baseUrl</code> seul est mis de côté. Tirets : le saut <code>references</code> , tenté si la chaîne n'a pas de motifs. Pointillés : la remontée vers les ancêtres de la racine de configuration. Un ensemble <code>visited</code> partagé coupe les cycles.	939
23.4	Le graphe d'imports de <code>tests/fixtures/next_project/src</code> (chemins relatifs à <code>src/</code>). En gras, les trois entrées serveur. Le graphe serveur est <code>{page, layout, dashboard/page, sidebar, use-nav-items, stats}</code> ; il s'arrête à <code>counter.tsx</code> , qui porte "use client".	949
24.1	Séquences de chargement native (à gauche) et WASM (à droite). Au-dessus du tronc commun, chaque hôte ne fait que trouver et lire des octets; le tronc commun (en bleu) est le même code Rust, appelé depuis <code>src/cli/</code> d'un côté et depuis <code>run_inner</code> de l'autre. Seul le choix de l'implémentation de <code>FileSystem</code> et de la fonction d'affichage diffère.	968
25.1	La pyramide des mesures. Les quatre étages grisés tournent dans <code>cargo test</code> ; les deux du haut mesurent des résultats plutôt que des mécanismes.	987
25.2	Le flot de la mesure corpus : un manifeste épinglé, un run JSON, une porte à trois codes de sortie. L'empreinte du corpus est vérifiée <i>avant</i> le digest.	1001
25.3	Le cycle de vie d'un chiffre, avec les runs réels du 27 septembre 2026. Deux pushes rouges, un <code>chore(corpus)</code> régénéré depuis l'artefact de 548f922, puis un vert; la PR #163 régénère sa baseline elle-même, faute d'artefact avant fusion.	1010
25.4	L'entonnoir de la campagne à l'aveugle (<code>docs/campaign/README.md</code> , <code>triage-2026-09-02-wave2.md</code>). Chaque étage est une mesure datée; les triages ne sont pas réécrits quand le vocabulaire change.	1013
26.1	Chronologie du projet, du 8 mai au 27 septembre 2026 : histogramme des commits par mois (<code>git log au commit e67b10a</code>), phases numérotées comme au tableau 26.1, pauses, releases <code>v0.1.0</code> à <code>v0.6.0</code> , et arrivée de <code>CLAUDE.md</code> le 21 juillet.	1024
26.2	La migration des faits de la couche règles vers le moteur (dates des commits d'ADR). Chaque flèche remplace une marche propre à une ou plusieurs règles par une relation calculée une fois, à la fin de la convergence, avec des colonnes polarisées (<code>docs/rerelations.md</code>).	1036
26.3	Le cycle d'une décision, reconstitué à partir de l'historique (notes du dossier 15, §4.1). Un delta de corpus qu'on ne sait pas attribuer renvoie à la localisation.	1041

26.4 Remplacements et amendements entre ADR (notes du dossier 15, §5.3 ; en-têtes Supersedes/Amends des ADR). Une flèche va de l'ADR ancien vers celui qui le modifie ; l'étiquette nomme la section touchée.	1042
27.1 Où naissent les constats de ce chapitre. Chaque étiquette renvoie à une ligne du tableau 27.2. Un constat est placé dans la couche qui contient sa <i>cause</i> , pas celle où il devient visible : presque tous se voient dans les règles.	1049
27.2 Les constats L1–L14 et leurs racines. La flèche pointillée rappelle que L1 cumule deux causes. Le regroupement est une lecture de ce chapitre, pas une décision du projet.	1065

Liste des extraits de code

1.1	Counter : un effet sans deps qui écrit l'état qu'il lit (/tmp/ch01/loop.tsx)	3
1.2	Profile : l'effet lit userId mais ne le déclare pas (/tmp/ch01/deps.tsx)	3
1.3	Timer : le callback de setInterval lit une valeur figée au montage (/tmp/ch01/timer.tsx)	4
1.4	Panel : un useState après un retour anticipé (/tmp/ch01/cond.tsx)	5
1.5	Counter avec des deps complètes (/tmp/ch01/deps_ok.tsx)	6
1.6	Alias de setter (/tmp/ch01/alias.tsx)	6
1.7	Boucle cachée dans un hook personnalisé (/tmp/ch01/custom_hook.tsx)	7
1.8	Setter du parent appelé par l'enfant (/tmp/ch01/prop_setter.tsx)	7
1.9	Bounded : la même boucle, gardée (/tmp/ch01/bounded.tsx)	9
1.10	Un objet recréé à chaque exécution de l'effet (/tmp/ch01/churn.tsx)	10
1.11	src/rules/api/diagnostic.rs, l. 23–38 — les trois niveaux et leur contrat	13
1.12	src/rules/api/diagnostic.rs, l. 157–191 — le constructeur d'une ERROR, contre les constructeurs libres d'un WARNING et d'une INFO	14
1.13	Une boucle certaine que la CLI ne signale pas (/tmp/ch01/updater_dep.tsx)	15
1.14	Sortie check -format json timer.tsx (élaguée : le diagnostic missing-deps est remplacé par ...)	18
2.1	Counter (fichier /tmp/ch02/counter.tsx)	27
2.2	Batch : trois appels au setter dans un même handler (/tmp/ch02/batching.tsx)	29
2.3	SameValue : un effet qui réécrit la valeur courante (/tmp/ch02/samevalue.tsx, imports omis)	30
2.4	Todos : initialiseur appelé à chaque rendu, et sa forme paresseuse (/tmp/ch02/lazy.tsx, imports omis)	31
2.5	src/lowering/hook_extractor.rs, l. 667–681 — un useState devient HookEntry::State, un useEffect un HookEntry::Effect	32
2.6	src/engine/fixpoint.rs, l. 336–350 — amorçage d'un slot, initialiseur paresseux compris	32
2.7	src/domains/stores/state_store.rs, l. 24–33 — lecture et mise à jour du store d'état	32
2.8	src/domains/interp/interpreter.rs, l. 341–366 — exec_setter_call : un setter est un join, un updater est exécuté sur le store	33
2.9	src/lowering/hook_extractor.rs, l. 715–719 — useReducer : le réducteur est jeté	34
2.10	Trois formes de deps pour le même effet (/tmp/ch02/deps.tsx, imports omis)	35
2.11	Inf et SelfCounter (/tmp/ch02/selfcounter.tsx)	36
2.12	Flicker (/tmp/ch02/flicker.tsx, imports omis)	37
2.13	Toggle (/tmp/ch02/toggle.tsx)	38
2.14	Width : un abonnement sans cleanup (/tmp/ch02/listener.tsx, imports omis)	38
2.15	src/lowering/hook_extractor.rs, l. 720–729 — useLayoutEffect et useInsertionEffect sont des HookEntry::Effect	39
2.16	src/engine/fixpoint.rs, l. 415–449 — les passes d'effets de la boucle de point fixe	39
2.17	src/engine/triggers.rs, l. 33–44 — une ligne de la relation effect _t riggers	40

2.18	Search : une dep fraîche à chaque rendu, et sa version mémorisée (/tmp/ch02/search.tsx, imports omis)	41
2.19	ObjChurn et ObjStateDep (/tmp/ch02/objchurn.tsx)	42
2.20	src/domains/transfer/state_value.rs, l. 142–156 — l'identité des marqueurs de hooks et des littéraux	42
2.21	src/domains/impls/stability.rs, l. 13–52 — le treillis Stability	43
2.22	src/domains/transfer/state_value.rs, l. 122–134 — la conversion « côté lecture » de l'ADR-017	44
2.23	src/domains/transfer/state_value.rs, l. 63–77 — recompute_memo : les cas sans deps lisibles et []	45
2.24	Cond, transcrit de React-tRace (/tmp/ch02/cond.tsx)	46
2.25	src/ir/hooks.rs, l. 247–259 — les deux premières variantes de HookEntry	47
2.26	src/rules/impls/conditional_hook.rs, l. 3–8 — conditional-hook : un hook conditionnel ne domine pas toutes les sorties	47
2.27	Inf2 : l'updater identité, appelé dans le corps (/tmp/ch02/setter_in_render.tsx)	48
2.28	List : le motif « adjust state during render » (/tmp/ch02/adjust.tsx)	49
2.29	src/rules/impls/setter_in_render.rs, l. 18–36 — les niveaux de setter-in-render	49
2.30	Impure : le rendu modifie une variable de module (/tmp/ch02/strict.tsx)	50
2.31	Un provider dont la valeur est neuve à chaque rendu (/tmp/ch02/context.tsx)	51
2.32	src/lowering/hook_extractor.rs, l. 733–744 — un hook React non modélisé devient une entrée Custom	52
2.33	src/rules/impls/unstable_context_value.rs, l. 4–13 — le défaut visé par unstable-context-value	52
2.34	app/page.tsx : un état dans un Server Component (/tmp/ch02next/app/page.tsx)	53
2.35	src/domains/interp/callbacks.rs, l. 6–23 — la classe de déclenchement d'un callee	60
2.36	src/engine/setters.rs, l. 57–70 — les phases d'un appel de setter	60
2.37	D : un updater dans un effet gardé par l'état qu'il écrit (/tmp/ch02/updater.tsx, imports omis)	61
2.38	R : un réducteur qui ignore l'action noop (/tmp/ch02/reducer.tsx, imports omis)	62
3.1	Deux compteurs pilotés par un effet (/tmp/ch03/counter.tsx, repris de tests/fixtures/widening.tsx)	67
3.2	src/domains/mod.rs, l. 16–38 — le trait des domaines abstraits	70
3.3	src/domains/impls/bool_val.rs, l. 1–13 — BoolVal, un treillis plat	71
3.4	src/domains/impls/mod.rs, l. 10–40 — la macro flat_lattice! (ordre et join)	72
3.5	src/domains/stores/mod.rs, l. 19–25 — $a \sqsubseteq b$ dans un ordre partiel	72
3.6	src/domains/impls/interval.rs, l. 5–27 — les intervalles et leur concrétisation	74
3.7	src/domains/impls/interval.rs, l. 69–83 — le join des intervalles	74
3.8	src/domains/stores/abstract_env.rs, l. 78–81 — une variable inconnue vaut $\top$	75
3.9	src/domains/stores/state_store.rs, l. 24–27 — un slot jamais écrit vaut $\perp$	75
3.10	Une boucle bornée dans un effet (/tmp/ch03/loop.tsx, identique à BoundedLocalLoop de tests/fixtures/widening.tsx)	77
3.11	src/engine/cfg_analyzer.rs, l. 47–58 — analyze_cfg : initialisation	79
3.12	src/engine/cfg_analyzer.rs, l. 60–89 — analyze_cfg : transfert d'un bloc	80
3.13	src/engine/cfg_analyzer.rs, l. 91–121 — analyze_cfg : propagation aux successeurs	81
3.14	src/engine/fixpoint.rs, l. 485–497 — le test de post-point fixe de la boucle externe	82
3.15	src/domains/impls/interval.rs, l. 84–106 — le widening des intervalles	84
3.16	src/domains/impls/interval.rs, l. 108–146 — le widening à seuils	84
3.17	src/engine/fixpoint.rs, l. 1185–1206 — la récolte des seuils	85
3.18	src/engine/fixpoint.rs, l. 514–529 — le widening de la boucle externe	86
3.19	Une garde à borne symbolique (/tmp/ch03/limit.tsx, élagué de sa ligne d'import)	87

3.20	Un setter dans le rendu, inconditionnel puis conditionnel (/tmp/ch03/setter.tsx, élagué de sa ligne d'import)	88
3.21	src/rules/api/query.rs, l. 111–135 — les verdicts polarisés	89
3.22	src/rules/impls/setter_in_render.rs, l. 147–162 — setter-in-render : l'ERROR demande deux musts	89
3.23	Un hook sauté par un return anticipé (/tmp/ch03/hook.tsx, élagué de sa ligne d'import)	91
3.24	src/engine/dominance.rs, l. 22–57 — le plus grand point fixe de la dominance .	92
3.25	src/rules/api/query.rs, l. 672–706 — ExitDominance : négation may et certification must	93
3.26	Un hook personnalisé qui boucle (/tmp/ch03/custom.tsx, élagué de sa ligne d'import)	95
3.27	Deux écritures successives (/tmp/ch03/weak.tsx, élagué de sa ligne d'import) . .	96
3.28	src/domains/stores/state_store.rs, l. 29–33 — l'écriture d'un slot est un join .	96
4.1	Le programme témoin du chapitre (/tmp/tour/counter.tsx)	101
4.2	Cargo.toml, l. 13–33 — workspace, features, binaire et bibliothèque	102
4.3	src/main.rs — le binaire entier	103
4.4	src/lib.rs — la surface de la bibliothèque	103
4.5	src/cli/check.rs, l. 192–202 — le binaire appelle le driver, puis écrit les flux . .	109
4.6	src/driver/mod.rs, l. 85–90 — la sortie tamponnée du driver	109
4.7	src/driver/mod.rs, l. 102–112 — la signature de run_check	109
4.8	src/driver/mod.rs, l. 236–237 — un seul appel pour tout le front-end	110
4.9	src/resolver/mod.rs, l. 289–312 — parse d'un fichier et décision de le garder . .	110
4.10	src/lowering/mod.rs, l. 128–144 — le candidat produit par les détecteurs	111
4.11	src/lowering/mod.rs, l. 463–470 — du candidat au CFG et à la table des hooks	111
4.12	src/ir/component.rs, l. 59–83 — l'IR d'un composant	111
4.13	src/driver/mod.rs, l. 370–374 — la configuration réelle du moteur et l'appel de l'analyse	112
4.14	src/resolver/mod.rs, l. 439–452 — des IR aux registres, puis au moteur	112
4.15	src/engine/fixpoint.rs, l. 484–497 — le test de convergence de la boucle externe	113
4.16	Les champs d'AnalysisResult, élagués et regroupés (src/engine/analysis_result.rs, l. 171–272)	113
4.17	src/engine/program_relations.rs, l. 19–42 — les relations de programme, construites à la première demande	114
4.18	src/driver/mod.rs, l. 407–410 — un seul cache pour tout le programme	114
4.19	src/rules/registry.rs, l. 254–274 — la passe des règles sur un composant (début)	114
4.20	src/rules/api/diagnostic.rs, l. 55–73 — le diagnostic et son champ de sévérité scellé	115
4.21	src/driver/mod.rs, l. 480–484 — la politique de code de sortie	116
4.22	src/domains/context.rs, l. 17–25 — le pointeur de fonction qui casse le cycle transfert / point fixe	118
4.23	tests/layer_boundary.rs, l. 19–41 — la liste d'autorisation et les marqueurs du cliquet	118
4.24	src/engine/function_registry.rs, l. 51–63 — résolution fail-closed d'un utilitaire	120
4.25	src/rules/api/query.rs, l. 115–129 — un résultat must porte un jeton de preuve ; un fait may n'en porte pas	121
4.26	src/rules/api/diagnostic.rs, l. 157–167 — le seul constructeur d'un Error . . .	121
4.27	src/ir/source_range.rs, l. 3–9 — l'identité d'un fichier	122
4.28	src/ir/component_id.rs, l. 23–29 — l'identité d'un composant	122
4.29	src/ir/source_range.rs, l. 36–42 — une position source	122
4.30	src/ir/types.rs, l. 1–9 — les alias de base de l'IR	122

4.31	src/rules/registry.rs, l. 356–369 — la position manquante est complétée au niveau central	123
4.32	src/rules/api/diagnostic.rs, l. 24–39 — les trois niveaux de diagnostic, définis dans le code	124
4.33	tests/corpus_fp_fixes.rs, l. 14–69 — le pipeline en miniature, dans un test . .	124
5.1	Quatre fonctions de niveau supérieur (/tmp/ch05/classes/App.tsx)	131
5.2	src/resolver/mod.rs, l. 289–312 — le parse d’un fichier et le traitement des erreurs	133
5.3	src/resolver/mod.rs, l. 193–200 — source_type_for	134
5.4	src/ir/source_range.rs, l. 51–57 — SourceMap	135
5.5	src/ir/component.rs, l. 59–83 — ComponentIR	137
5.6	src/lowering/mod.rs, l. 437–491 — lower_program_with_resolver	138
5.7	src/lowering/detector.rs, l. 15–35 — FnItem, Classify, DefaultHandler	140
5.8	src/lowering/detector.rs, l. 41–71 — detect_fns, le marcheur commun	141
5.9	src/lowering/detector.rs, l. 85–106 — consider_var	141
5.10	Formes limites du marcheur (/tmp/ch05/walker/A.tsx)	142
5.11	src/lowering/mod.rs, l. 128–160 — Candidate et son unique point de dispatch . .	143
5.12	src/lowering/mod.rs, l. 46–61 — is_hook_name, la frontière hook / utilitaire . . .	144
5.13	src/lowering/utility_detector.rs, l. 17–47 — le détecteur d’utilitaires	144
5.14	src/lowering/component_detector.rs, l. 47–73 — is_component	145
5.15	src/lowering/jsx_detect.rs, l. 14–45 — stmt_has_jsx_return	146
5.16	src/lowering/jsx_detect.rs, l. 47–63 — expr_contains_jsx	147
5.17	src/lowering/cfg_builder.rs, l. 985–1001 — le CFG d’une flèche concise	150
5.18	tests/fixtures/concise_arrow/App.tsx, l. 8–17 — une fabrique concise d’objets frais	150
5.19	tests/fixtures/concise_arrow/App.tsx, l. 30–42 — le témoin à corps bloc	150
5.20	src/lowering/hook_call_detect.rs, l. 1–12 — le principe de la règle 4	151
5.21	src/lowering/component_detector.rs, l. 248–264 — le test de #122	152
5.22	src/lowering/hook_call_detect.rs, l. 60–88 — expr_calls_hook	152
5.23	src/lowering/hook_call_detect.rs, l. 102–109 — callee_is_hook	153
5.24	/tmp/ch05/shadow/Shadow.tsx	154
5.25	src/lowering/import_resolution.rs, l. 35–55 — HookOrigin	154
5.26	src/lowering/import_resolution.rs, l. 74–130 — build_hook_origins	155
5.27	/tmp/ra-ex/e5/Page.tsx (fichiers hooks/useData.ts, ui/Widget.tsx, types.ts présents)	156
5.28	src/lowering/hook_extractor.rs, l. 531–543 — ImportCtx	157
5.29	src/lowering/hook_extractor.rs, l. 562–582 — classify_callee (début) : un hook local masque tout	157
5.30	/tmp/ch05/consts/Consts.tsx	159
5.31	src/ir/component.rs, l. 12–41 — ModuleConstInit	160
5.32	src/ir/module.rs, l. 21–36 — ModuleFacts	161
5.33	src/ir/module.rs, l. 100–126 — reachable_from, parcours en largeur arrêté par une frontière	161
5.34	/tmp/ra-ex/e3b/Dom.tsx	163
5.35	src/lowering/import_resolution.rs, l. 237–262 — build_jsx_origins	164
5.36	src/lowering/expr_lower.rs, l. 832–842 — le callee JSX reçoit son origine	165
5.37	src/engine/component_registry.rs, l. 114–127 — resolve_child, trois réponses . .	165
5.38	/tmp/ra-ex/e6/Limits.tsx	166
5.39	/tmp/ra-ex/e7/App.tsx	167
6.1	Un compteur avec retour anticipé (/tmp/ch06/counter.tsx)	172
6.2	src/ir/types.rs, l. 1–25 — alias de base et clé d’allocation	174
6.3	src/ir/cfg.rs, l. 5–65 — blocs, terminateurs, arêtes et graphe	175
6.4	src/ir/cfg.rs, l. 146–163 — les blocs atteignables depuis l’entrée	178

6.5	src/ir/cfg.rs, l. 181–227 — la bonne formation structurelle	178
6.6	src/ir/stmt.rs, l. 7–29 — les quatre formes d’instruction	180
6.7	src/ir/stmt.rs, l. 47–54 — la cible d’une écriture membre	180
6.8	src/ir/expr.rs, l. 10–18 — les littéraux primitifs	181
6.9	src/ir/expr.rs, l. 44–56 — un variant par opérateur, plus de Unknown	181
6.10	src/ir/expr.rs, l. 174–228 — Expr, première partie	182
6.11	src/ir/expr.rs, l. 230–300 — Expr, seconde partie	183
6.12	src/ir/expr.rs, l. 89–113 — lire un membre par son nom, en respectant les spreads	186
6.13	src/ir/expr.rs, l. 463–511 — l’énumération canonique des enfants directs . . .	187
6.14	src/ir/hooks.rs, l. 247–313 — une ligne de la table des hooks	189
6.15	src/ir/hooks.rs, l. 341–357 — le corps d’une ligne, sans bras par défaut	191
6.16	src/ir/expr.rs, l. 302–331 — ce que lit une liaison de marqueur	192
6.17	Quatre formes de deps (/tmp/ch06/exo.tsx)	193
6.18	src/ir/hooks.rs, l. 88–107 — les trois états de l’argument deps	193
6.19	src/ir/hooks.rs, l. 160–178 — un tableau de deps écrit	194
6.20	src/ir/hooks.rs, l. 44–58 — l’arité d’un tableau écrit	194
6.21	src/ir/hooks.rs, l. 192–215 — les entrées qui couvrent une lecture	195
6.22	src/ir/hooks.rs, l. 10–42 — une ligne de provenance	196
6.23	src/ir/component.rs, l. 59–83 — l’IR d’un composant	197
6.24	src/ir/hook_ir.rs, l. 9–24 — l’IR d’un hook custom	198
6.25	src/ir/function_ir.rs, l. 15–22 — l’IR d’un utilitaire	198
6.26	src/ir/component.rs, l. 12–41 — ce que l’on sait d’un initialiseur de const de module	199
6.27	src/ir/component.rs, l. 43–57 — l’identité canonique d’un contexte	199
6.28	src/ir/module.rs, l. 21–36 — ce qu’un fichier déclare de lui-même	200
6.29	src/ir/source_range.rs, l. 3–16 — l’identité d’un fichier et sa table	200
6.30	src/ir/component_id.rs, l. 23–29 — l’identité d’un composant	201
6.31	src/ir/component_id.rs, l. 55–75 — la table d’internement des composants . . .	201
6.32	src/ir/component_id.rs, l. 138–154 — le nom à <i>montrer</i>	202
6.33	src/ir/source_range.rs, l. 36–42 — une position source	203
6.34	src/ir/source_range.rs, l. 94–102 — d’un offset à une position	203
6.35	Deps, chemins et alias (/tmp/ch06/deps.tsx)	204
6.36	src/ir/free_vars.rs, l. 91–107 — les variables libres d’un corps	205
6.37	src/ir/free_vars.rs, l. 6–23 — un chemin d’accès	206
6.38	src/ir/free_vars.rs, l. 280–311 — extraire la chaîne maximale enracinée à une variable	206
6.39	src/ir/free_vars.rs, l. 253–278 — les chemins déclarés et la couverture par préfixe	207
6.40	src/ir/bindings.rs, l. 128–149 — l’unique membre droit d’un nom	208
6.41	src/ir/bindings.rs, l. 49–56 — les deux orthographes d’une liaison fonctionnelle	209
6.42	src/ir/bindings.rs, l. 151–179 — le certificat qui autorise à exécuter un corps .	209
7.1	src/lowering/mod.rs, l. 463–474 — la chaîne de lowering d’un composant	215
7.2	src/lowering/mod.rs, l. 146–160 — le seul endroit qui choisit entre corps-bloc et corps-expression	216
7.3	Un corps linéaire, sans return (/tmp/ch07/c1_linear.tsx)	217
7.4	src/lowering/cfg_builder.rs, l. 78–100 — l’état du constructeur de blocs	218
7.5	src/lowering/cfg_builder.rs, l. 172–203 — sceller, ouvrir, couper à un await . .	219
7.6	src/lowering/cfg_builder.rs, l. 270–277 — la boucle d’instructions s’arrête au premier terminateur	220
7.7	src/lowering/cfg_builder.rs, l. 213–240 — la sortie du constructeur	220
7.8	src/lowering/cfg_builder.rs, l. 445–464 — les instructions sans code, en extension	222
7.9	src/lowering/cfg_builder.rs, l. 286–297 — instruction-expression et return . . .	223

7.10	src/lowering/cfg_builder.rs, l. 544–585 — l’abaissement d’un if	223
7.11	Retour anticipé, for...of, continue, ternaire (/tmp/ch07/e2_control.tsx)	224
7.12	Une garde-throw (/tmp/ch07/c3_throw.tsx)	225
7.13	src/lowering/expr_lower.rs, l. 18–36 — la valeur opaque et la lecture pour effet	227
7.14	src/lowering/expr_lower.rs, l. 256–277 — i++ et -i	227
7.15	src/lowering/expr_lower.rs, l. 617–634 — la liste d’arguments	229
7.16	src/lowering/expr_lower.rs, l. 493–518 — affectation à un identifiant	229
7.17	src/lowering/expr_lower.rs, l. 352–394 — le littéral d’objet	230
7.18	src/lowering/expr_lower.rs, l. 799–830 — un élément JSX : composant ou élément hôte	231
7.19	?, ??, && en instruction, useEffect (/tmp/ch07/e3_logical.tsx)	232
7.20	src/lowering/expr_lower.rs, l. 675–700 — le ternaire (début ; les deux bras suivent le même motif)	233
7.21	src/lowering/expr_lower.rs, l. 739–795 — &&, et ??	234
7.22	Une boucle while (/tmp/ch07/c2_while.tsx)	236
7.23	src/lowering/cfg_builder.rs, l. 603–633 — la boucle while	236
7.24	src/lowering/cfg_builder.rs, l. 381–397 — break et continue sont de vraies arêtes	238
7.25	switch avec fall-through, puis try/catch/finally (/tmp/ch07/e4_switch_try.tsx) .	240
7.26	src/lowering/cfg_builder.rs, l. 786–838 — la chaîne de dispatchs du switch . .	241
7.27	src/lowering/cfg_builder.rs, l. 492–510 — le try : un Branch sur T	243
7.28	src/lowering/cfg_builder.rs, l. 869–894 — le motif tableau	244
7.29	src/lowering/cfg_builder.rs, l. 938–946 — un défaut de motif est émis, sans être modélisé	246
7.30	src/lowering/cfg_builder.rs, l. 952–970 — le préambule des paramètres	246
7.31	src/lowering/expr_lower.rs, l. 442–457 — une flèche : un corps à part	248
7.32	src/lowering/cfg_builder.rs, l. 20–58 — le compteur partagé et le contexte par fichier	249
7.33	Un effet asynchrone (/tmp/ch07/e6_async.tsx)	251
7.34	src/lowering/expr_lower.rs, l. 577–606 — await : lier l’argument, puis couper .	251
7.35	src/lowering/hook_extractor.rs, l. 264–304 — la passe extract_hooks	253
7.36	src/lowering/hook_extractor.rs, l. 369–417 — process_stmt : le cas Let	254
7.37	src/lowering/hook_extractor.rs, l. 809–821 — le corps d’un hook : le FnLit, ou l’appel du callee	257
7.38	src/lowering/hook_extractor.rs, l. 84–100 — les handlers se décident par fuite .	259
7.39	src/lowering/hook_extractor.rs, l. 18–41 — les subscriptions : addEventListener dans un corps d’effet	259
7.40	src/lowering/cfg_builder.rs, l. 985–1001 — le corps d’une flèche concise est un Return	260
7.41	src/lowering/hook_extractor.rs, l. 321–341 — sortir un appel de hook d’un terminateur	261
8.1	Un hook custom qui boucle (/tmp/ch08/hook/Counter.tsx)	267
8.2	Le même corps, deux faits incomparables (/tmp/ch08/step/Step.tsx)	268
8.3	src/engine/fixpoint.rs, l. 197–229 — les deux passes d’expansion, dans l’ordre	270
8.4	src/ir/remap.rs, l. 11–37 — ce qu’une greffe décale	271
8.5	src/engine/fixpoint.rs, l. 837–867 — la réserve de sels et de sites d’une composante	272
8.6	Un utilitaire à deux return (/tmp/ch08/clamp/Counter.tsx)	273
8.7	src/ir/splice.rs, l. 33–54 — le côté appelé d’une greffe	274
8.8	src/ir/splice.rs, l. 77–134 — renommage, préconditions, coupe du bloc d’appel, allocation des blocs et liaisons de paramètres	275
8.9	src/ir/splice.rs, l. 136–186 — décalage des blocs et réécriture des retours . . .	277

8.10	src/ir/splice.rs, l. 188–214 — liaisons en tête, insertion, jointure	278
8.11	tests/fixtures/splice_span/App.tsx, l. 1–6 — le composant appelant	280
8.12	tests/fixtures/splice_span/lib/heavy.ts, l. 1–5 — l'utilitaire, dans un autre fichier	280
8.13	src/ir/splice.rs, l. 259–272 — les liaisons d'un corps	281
8.14	src/ir/splice.rs, l. 302–330 — la table de renommage (sa doc, src/ir/splice.rs, l. 274–301, est résumée ci-dessous)	282
8.15	src/ir/splice.rs, l. 431–447 — renommage d'une expression, cas de la fermeture	283
8.16	src/ir/splice.rs, l. 336–349 — le séparateur et le nom affiché	284
8.17	src/ir/splice.rs, l. 636–663 — substitution de lectures, attentive aux fermetures	284
8.18	src/engine/fixpoint.rs, l. 925–942 — garde de récursion et recherche dans le registre	286
8.19	src/engine/fixpoint.rs, l. 996–1026 — décalage, substitution, sel, table de re- nommage	288
8.20	src/engine/fixpoint.rs, l. 1028–1047 — la table de l'appelé, copiée et réécrite .	288
8.21	src/engine/setters.rs, l. 806–820 — une région greffée	289
8.22	Deux appels du même hook (/tmp/ch08/step/ Twice.tsx, useStep du liste 8.2) . .	290
8.23	src/engine/function_registry.rs, l. 51–63 — résolution fail-closed d'un utilitaire	291
8.24	src/engine/fixpoint.rs, l. 1698–1718 — ce qu'est un appel greffable	292
8.25	src/engine/fixpoint.rs, l. 1763–1790 — la greffe d'un appel d'utilitaire	292
8.26	src/domains/interp/interpreter.rs, l. 521–527 — B6 : appel direct d'une ferme- ture locale	294
8.27	src/registry/summary.rs, l. 69–77 — le registre par défaut	294
8.28	src/engine/fixpoint.rs, l. 949–977 — le résumé d'un hook introuvable	295
8.29	Hook résumé par membres, hook connu, hook inconnu (/tmp/ch08/summary/Form.tsx)	296
8.30	tests/fixtures/shared_hook_repeat/hooks/useShared.ts, l. 6–12 — un hook par- tagé qui boucle	297
8.31	src/rules/registry.rs, l. 356–369 — un finding prend la première position de sa chaîne	298
8.32	Un setter appelé dans un return (/tmp/ircheck/ret/Ret.tsx)	299
9.1	Deux compteurs (/tmp/ch09/counter.tsx, repris de tests/fixtures/widening.tsx)	306
9.2	src/domains/mod.rs, l. 40–85 — ponts vers StateValue et raffinements par défaut	306
9.3	Un booléen d'état et un setter choisi dynamiquement (exemple d'illustration) .	309
9.4	src/domains/impls/mod.rs, l. 1–53 — la macro flat_lattice!	310
9.5	src/domains/impls/bool_val.rs, l. 1–13 — le treillis des booléens	311
9.6	src/domains/impls/setter_val.rs, l. 3–37 — le treillis plat des setters	312
9.7	Un mode d'affichage (exemple d'illustration)	313
9.8	src/domains/impls/str_const.rs, l. 7–39 — StrConst, ses constructeurs, $k = 4$. .	314
9.9	src/domains/impls/str_const.rs, l. 42–62 — l'ordre de StrConst : l'inclusion . . .	314
9.10	src/domains/impls/str_const.rs, l. 75–102 — join, meet et widen de StrConst . .	315
9.11	src/domains/impls/interval.rs, l. 5–27 — la structure Interval	317
9.12	src/domains/impls/interval.rs, l. 29–67 — constructeurs et tests élémentaires .	318
9.13	src/domains/impls/interval.rs, l. 312–328 — l'ordre des intervalles : l'inclusion des bornes	319
9.14	src/domains/impls/interval.rs, l. 332–361 — impl AbstractDomain for Interval (début)	319
9.15	src/domains/impls/interval.rs, l. 148–187 — addition, soustraction, multiplication	322
9.16	src/domains/impls/interval.rs, l. 189–198 — négation	322
9.17	src/domains/impls/interval.rs, l. 200–253 — reste et puissance	323
9.18	src/domains/transfer/state_value.rs, l. 752–811 — la table des opérateurs binaires	325
9.19	src/domains/transfer/state_value.rs, l. 813–844 — plages int32/uint32 et utilitaires	327

9.20	src/domains/transfer/state_value.rs, l. 846–886 — eval_bitwise : plancher, &, et ^	327
9.21	src/domains/transfer/state_value.rs, l. 887–939 — eval_bitwise : décalages, et all_ones_above	328
9.22	src/domains/transfer/state_value.rs, l. 978–993 — la négation bit à bit	329
9.23	Décalages sur un état non entier (/tmp/ch09/shift2.tsx, élagué de sa ligne d’import)	330
9.24	src/domains/impls/state_value.rs, l. 670–676 — le widening à seuils du produit	333
9.25	src/engine/fixpoint.rs, l. 1212–1224 — la récolte des littéraux, fermetures comprises	333
9.26	src/domains/impls/interval.rs, l. 255–309 — les raffinements numériques des intervalles	335
9.27	src/engine/cfg_analyzer.rs, l. 212–241 — choix du raffinement numérique selon la condition et la branche	336
9.28	src/domains/impls/state_value.rs, l. 565–582 — le raffinement de véracité (branche vraie)	337
9.29	Garde de véracité (/tmp/ch09/limits.tsx, extrait)	338
9.30	Un produit par zéro d’un état non borné (/tmp/ch09/limits.tsx, extrait)	339
9.31	Deux cycles finis (/tmp/ch09/limits.tsx, extrait)	341
10.1	Un compteur nullable (/tmp/ch10/nullable.tsx, extrait)	346
10.2	L’ancien StateValue (docs/adr/ADR-008-value-domain.md, élagué des commentaires)	346
10.3	src/domains/impls/state_value.rs, l. 16–44 — le produit ponctuel des sortes JavaScript	348
10.4	src/domains/impls/state_value.rs, l. 190–206 — $\perp$: toutes les composantes à $\perp$	349
10.5	src/domains/impls/state_value.rs, l. 512–523 — $\top$: toutes les composantes à $\top$, résidu compris	349
10.6	src/domains/impls/state_value.rs, l. 125–130 — la forme commune des constructeurs	350
10.7	src/domains/impls/state_value.rs, l. 46–78 — le masque des sortes peuplées	350
10.8	src/domains/impls/state_value.rs, l. 80–121 — l’unique énumération des sortes	351
10.9	src/domains/impls/state_value.rs, l. 270–287 — deux prédicats dérivés du masque	352
10.10	src/domains/transfer/state_value.rs, l. 708–732 — la vue numérique d’un opérande	353
10.11	src/domains/impls/state_value.rs, l. 479–501 — l’ordre produit	354
10.12	src/domains/impls/state_value.rs, l. 631–676 — join, meet, widen et widen_to ponctuels	355
10.13	src/domains/impls/state_value.rs, l. 289–306 — la valeur syntaxique d’un initialiseur	356
10.14	Un état objet en dep (/tmp/ch10/stability.tsx, extrait)	358
10.15	Une boucle de churn d’objet (/tmp/ch10/stability.tsx, extrait)	358
10.16	src/domains/impls/stability.rs, l. 9–52 — le treillis de stabilité versionnée	360
10.17	src/domains/impls/stability.rs, l. 54–70 — les constructeurs canonisants	361
10.18	src/domains/impls/stability.rs, l. 72–101 — l’ordre de Stability	362
10.19	src/domains/impls/stability.rs, l. 108–146 — join, meet et widen de Stability	363
10.20	src/domains/transfer/state_value.rs, l. 122–135 — la conversion côté lecture (ADR-017)	365
10.21	src/engine/written.rs, l. 173–200 — la fraîcheur d’une valeur écrite, lue sur la composante référence	367
10.22	src/domains/transfer/state_value.rs, l. 617–628 — un champ d’objet versionné garde les étiquettes	368
10.23	src/domains/transfer/state_value.rs, l. 153–156 — les littéraux alloués et les éléments natifs	368
10.24	src/domains/transfer/state_value.rs, l. 176–181 — appels et new	368

10.25	src/domains/transfer/state_value.rs, l. 142–147 — les marqueurs de hooks . . .	369
10.26	src/domains/transfer/state_value.rs, l. 56–101 — la valeur d’un mémo, stabilité de ses deps	369
10.27	Un mémo primitif sur une dep fraîche (/tmp/ch10/memo.tsx, extrait)	371
10.28	src/domains/impls/state_value.rs, l. 320–369 — la projection vers la stabilité .	372
10.29	Un nombre à deux valeurs (/tmp/ch10/limits.tsx, extrait)	373
10.30	src/domains/impls/state_value.rs, l. 371–401 — croissance non bornée et finitude	374
10.31	src/domains/impls/state_value.rs, l. 403–417 — garder les étiquettes, tester la stabilité	374
10.32	Deux états optionnels (/tmp/ch10/kinds.tsx, extrait)	375
10.33	src/rules/api/query.rs, l. 137–179 — le verdict de stabilité côté règles	375
10.34	src/domains/context.rs, l. 105–125 — le paquet d’état passé à chaque fonction de transfert	377
10.35	src/domains/context.rs, l. 127–145 — un contexte sans requête ni inter-composants	377
10.36	src/domains/context.rs, l. 27–49 — le canal de requête entre domaines	378
10.37	src/domains/context.rs, l. 147–171 — le contexte de requête du point fixe . . .	379
10.38	src/domains/context.rs, l. 15–25 — le pointeur qui casse le cycle transfert–point fixe	380
10.39	src/domains/context.rs, l. 51–103 — le contexte inter-composants	380
10.40	Deux mémos sur un état (/tmp/ch10/memo.tsx, extrait)	381
10.41	Un booléen qui bascule (/tmp/ch10/kinds.tsx, extrait)	382
10.42	Un élément natif en dep (/tmp/ch10/limits.tsx, extrait)	383
11.1	Deux compteurs (/tmp/ch11/counter.tsx)	388
11.2	src/domains/mod.rs, l. 114–144 — le contrat entre le moteur et un domaine (la quatrième méthode, recompute_memo, suit jusqu’à la ligne 161)	390
11.3	src/domains/transfer/state_value.rs, l. 24–54 — l’unique implémentation du trait	391
11.4	Ce qu’on sait d’une variable (/tmp/ch11/env.tsx)	392
11.5	src/domains/stores/abstract_env.rs, l. 46–60 — l’environnement abstrait	393
11.6	src/domains/stores/abstract_env.rs, l. 78–93 — la lecture d’une variable	394
11.7	src/domains/stores/abstract_env.rs, l. 130–173 — le join de deux environnements	395
11.8	Des opérateurs (/tmp/ch11/ex1.tsx)	396
11.9	src/domains/transfer/state_value.rs, l. 106–136 — l’évaluateur : littéraux, variables, hooks	397
11.10	src/domains/transfer/state_value.rs, l. 137–163 — l’évaluateur : marqueurs, allocations, éléments	398
11.11	src/domains/transfer/state_value.rs, l. 238–259 — les appels dont le résultat est toujours neuf	399
11.12	src/domains/transfer/state_value.rs, l. 941–977 — les opérateurs unaires (début)	400
11.13	src/domains/stores/state_store.rs, l. 7–45 — le store d’état : lecture, mise à jour faible, fusion	403
11.14	src/engine/fixpoint.rs, l. 336–348 — l’amorçage d’un slot par son initialiseur .	404
11.15	src/domains/stores/memo_store.rs, l. 27–35 — le store des mémos : lecture et écriture	405
11.16	src/domains/stores/shared_state_store.rs, l. 10–17 — le store inter-composants (get et update suivent, l. 24–34)	405
11.17	src/domains/stores/shared_state_store.rs, l. 57–66 — la tranche d’un composant	406
11.18	src/domains/interp/interpreter.rs, l. 69–99 — une instruction : pré-passe des callbacks, puis cœur	408
11.19	src/domains/interp/interpreter.rs, l. 214–245 — lier une variable (1/2) : setters, callbacks, allocations	409

11.20	src/domains/interp/interpreter.rs, l. 246–292 — lier une variable (2/2) : résumés, alias, chaînes de membres, valeur	409
11.21	src/domains/interp/interpreter.rs, l. 155–200 — l’écriture de champ, mise à jour faible du tas	411
11.22	src/domains/interp/interpreter.rs, l. 114–126 — la définition unique de « exécuter une expression pour ses effets »	412
11.23	src/domains/interp/interpreter.rs, l. 341–392 — l’appel de setter : slot local, puis slot d’un propriétaire	412
11.24	Cinq façons d’appeler un setter depuis un effet (/tmp/ch11/ex9.tsx)	415
11.25	src/domains/interp/callbacks.rs, l. 6–56 — la classe de déclenchement d’un appel	415
11.26	src/domains/interp/interpreter.rs, l. 477–540 — la pré-passe des callbacks . . .	417
11.27	src/domains/interp/interpreter.rs, l. 394–469 — l’exécution d’un corps de fonction imbriqué	419
11.28	Membres, sites et fermetures (/tmp/ch11/ex10.tsx, extrait du corps de Heapy) . .	421
11.29	src/domains/stores/heap.rs, l. 17–34 — le tas abstrait et ses valeurs	422
11.30	src/domains/stores/heap.rs, l. 45–70 — l’allocation d’une fermeture et de ses captures	422
11.31	src/domains/stores/heap.rs, l. 134–165 — les sites que désigne une expression .	423
11.32	src/domains/transfer/state_value.rs, l. 598–628 — la lecture d’un membre par le tas	424
11.33	src/domains/interp/interpreter.rs, l. 544–583 — exécuter un callback désigné par une variable	425
11.34	Un enfant connu, un enfant inconnu (/tmp/ch11/ex13.tsx)	428
11.35	src/domains/transfer/state_value.rs, l. 559–580 — le tas et l’environnement initiaux d’un enfant	429
11.36	src/domains/transfer/state_value.rs, l. 265–297 — joindre $\top$ dans les slots dont le setter s’échappe	430
11.37	src/domains/transfer/state_value.rs, l. 385–392 — un corps n’est parcouru qu’une fois par environnement capturé	430
11.38	Trois callbacks à deux blocs (/tmp/ch11/ex3.tsx, extrait de l’effet)	432
12.1	Un effet avec une boucle locale (/tmp/ch12/loop.tsx)	441
12.2	src/engine/cfg_analyzer.rs, l. 34–46 — signature d’analyze_cfg	442
12.3	src/engine/cfg_analyzer.rs, l. 47–58 — initialisation	442
12.4	src/engine/cfg_analyzer.rs, l. 60–89 — visite d’un bloc	443
12.5	Une flèche concise (/tmp/ch12/concise.tsx)	444
12.6	src/engine/fixpoint.rs, l. 1226–1237 — l’environnement de sortie d’un CFG . .	445
12.7	src/engine/cfg_analyzer.rs, l. 126–150 — l’environnement passé à chaque successeur	445
12.8	Le diamant du test, en source équivalent	446
12.9	src/domains/stores/abstract_env.rs, l. 130–142 — join des valeurs : une variable d’un seul côté vaut $\top$	447
12.10	src/engine/cfg_analyzer.rs, l. 91–121 — fusion et propagation vers les successeurs	448
12.11	src/engine/fixpoint.rs, l. 1185–1206 — récolte des seuils du widening	449
12.12	Un compteur gardé (/tmp/ch12/guarded.tsx)	452
12.13	La même garde, écrite limit > count (/tmp/ch12/unguarded.tsx)	453
12.14	src/engine/cfg_analyzer.rs, l. 201–217 — narrow_env_for_branch : une variable à gauche, sinon rien	453
12.15	src/engine/cfg_analyzer.rs, l. 242–293 — raffinements de nullité et de vérité . .	453
12.16	Une branche morte (/tmp/ch12/dead.tsx)	455
12.17	src/engine/cfg_analyzer.rs, l. 152–185 — relire l’environnement d’entrée d’un bloc	457
12.18	Trois questions de dominance (/tmp/ch12/early.tsx)	459

12.19	src/engine/dominance.rs, l. 6–58 — le plus grand point fixe de la dominance . .	459
12.20	src/engine/dominance.rs, l. 113–135 — ordre postfixe inverse	461
12.21	src/engine/dominance.rs, l. 94–111 — la relation précalculée	462
12.22	src/rules/api/query.rs, l. 647–671 — les sorties sont les Return atteignables . .	462
12.23	src/engine/dominance.rs, l. 69–92 — l’ensemble post-domine-t-il l’entrée?	464
12.24	src/engine/eval.rs, l. 31–46 — le cœur mécanique de toute sonde	465
12.25	src/engine/eval.rs, l. 61–80 — le cas commun : stores et tas convergés	466
12.26	src/engine/eval.rs, l. 89–105 — un évaluateur réutilisable	466
13.1	La boucle infinie canonique (/tmp/ch13/runaway.tsx)	473
13.2	Un objet recréé dans ses propres deps (/tmp/ch13/fresh.tsx)	474
13.3	src/engine/fixpoint.rs, l. 83–118 — les deux entrées intra-composant	475
13.4	src/engine/fixpoint.rs, l. 38–60 — les paramètres du moteur et leurs défauts .	476
13.5	src/engine/fixpoint.rs, l. 120–138 — la doc d’en-tête et la signature de analyze_component_impl	477
13.6	src/engine/fixpoint.rs, l. 153–190 — les constantes de module amorcent deux environnements	479
13.7	src/engine/fixpoint.rs, l. 192–229 — les deux expansions, dans cet ordre . . .	480
13.8	Un hook personnalisé qui boucle (/tmp/ch13/ticker.tsx)	481
13.9	src/engine/fixpoint.rs, l. 828–867 — les identités d’une greffe : sel et curseur d’ExprId	482
13.10	src/engine/fixpoint.rs, l. 1611–1634 — la boucle de greffes d’un CFG et son budget	483
13.11	src/engine/fixpoint.rs, l. 1185–1224 — la récolte des seuils : tous les littéraux numériques, y compris dans les fermetures	485
13.12	src/engine/fixpoint.rs, l. 297–312 — le store sujet du point fixe et les accumula- teurs de la boucle	486
13.13	src/engine/fixpoint.rs, l. 314–353 — l’amorçage : un join par useState, l’initiali- seur paresseux exécuté	487
13.14	tmp/ch13/lazy.tsx	488
13.15	src/engine/fixpoint.rs, l. 355–380 — la passe de rendu d’un tour	489
13.16	src/engine/fixpoint.rs, l. 382–413 — le recalcul des mémos depuis l’environne- ment de sortie du rendu	489
13.17	src/engine/fixpoint.rs, l. 1226–1237 — exit_env : le join des sorties, par reduce et non par fold	490
13.18	src/domains/stores/memo_store.rs, l. 27–30 — un mémo pas encore calculé vaut $\top$ — à l’inverse du StateStore, où l’absent vaut $\perp$	491
13.19	tmp/ch13/memoloop.tsx	491
13.20	src/engine/fixpoint.rs, l. 415–449 — les passes d’effets d’un tour	492
13.21	tmp/ch13/loader.tsx	493
13.22	tmp/ch13/mutual.tsx	493
13.23	src/engine/fixpoint.rs, l. 451–483 — les passes de handlers : même schéma que les effets, avec le commentaire décisif	494
13.24	tmp/ch13/counter.tsx	495
13.25	src/engine/fixpoint.rs, l. 485–497 — le join des quatre sources et le test d’arrêt	496
13.26	src/domains/stores/shared_state_store.rs, l. 57–66 — la tranche d’un composant dans le SharedStateStore	496
13.27	src/domains/stores/state_store.rs, l. 69–77 — $S \sqsubseteq S'$: chaque label de S est en dessous de sa valeur dans S'	496
13.28	src/engine/fixpoint.rs, l. 499–530 — join tant que le compteur est bas, widening à seuils ensuite, garde-fou à cent	497

13.29	src/domains/stores/state_store.rs, l. 35–62 — join, widen, widen_to : un seul parcours, trois combinateurs	498
13.30	src/domains/impls/state_value.rs, l. 657–676 — le widening du produit : intervalles et chaînes ont un vrai widening, le reste est un join	499
13.31	tmp/ch13/guarded.tsx	499
13.32	tmp/ch13/dead.tsx	501
13.33	src/engine/analysis_result.rs, l. 18–28 — la provenance d’un widening	501
13.34	src/engine/fixpoint.rs, l. 532–563 — une passe de rendu de plus, sur les stores convergés	503
13.35	src/engine/fixpoint.rs, l. 565–594 — rejouer les effets depuis un store $\perp$ pour isoler ce qu’ils écrivent	504
13.36	src/rules/impls/infinite_loop.rs, l. 128–136 — les deux portes du bras intra	504
13.37	src/domains/impls/state_value.rs, l. 371–382 — ce que « non borné » veut dire pour une valeur du produit	505
13.38	src/engine/fixpoint.rs, l. 647–667 — la relation <i>effect_itriggers</i> , évaluée dans <i>env_exit</i> contre la store convergée	506
13.39	src/engine/fixpoint.rs, l. 670–698 — l’ <i>AnalysisResult</i> rendu	506
13.40	src/engine/fixpoint.rs, l. 757–794 — la photographie de la phase 1 et le balayage intra de la phase 2	507
13.41	tmp/ch13/updater.tsx	510
13.42	tmp/ch13/cleanup.tsx	511
13.43	tmp/ch13/reducer.tsx	512
14.1	la sonde de lecture des relations (élagué, /tmp/relprobe/src/main.rs)	516
14.2	/tmp/verif07/e12.tsx	516
14.3	src/engine/fixpoint.rs, l. 596–646 — la dernière tranche : setters étrangers, environnements de site, puis les quatre relations	519
14.4	/tmp/relprobe/ex2.tsx	521
14.5	src/engine/setters.rs, l. 641–648 — la région lexicale d’une écriture	521
14.6	src/engine/setters.rs, l. 681–701 — la phase d’exécution, un verdict may	522
14.7	src/engine/setters.rs, l. 1319–1344 — la classe interne de la marche	523
14.8	src/engine/setters.rs, l. 978–986 — de la classe à la phase	523
14.9	src/engine/setters.rs, l. 1232–1283 — l’état d’une marche, champ par champ	524
14.10	src/engine/setters.rs, l. 1696–1716 — la boucle sur les blocs	526
14.11	src/engine/setters.rs, l. 1882–1919 — un site d’écriture, puis le bras B6	527
14.12	tmp/verif07/e11.tsx	528
14.13	src/engine/setters.rs, l. 1626–1651 — la classe d’un argument fonctionnel	529
14.14	src/engine/setters.rs, l. 1870–1875 — les <i>FnLit</i> ne sont jamais traversées de l’extérieur	530
14.15	src/engine/registrations.rs, l. 63–78 — une ligne de la table des registrars	532
14.16	tmp/relprobe/ex1.tsx	533
14.17	src/engine/setters.rs, l. 728–786 — la ligne <i>SlotWriter</i>	534
14.18	src/engine/setters.rs, l. 245–259 — un setter reçu d’un autre composant	535
14.19	src/engine/setters.rs, l. 1098–1143 — <i>same_tick</i> , <i>block</i> et <i>written</i> pour chaque site	537
14.20	src/engine/setters.rs, l. 709–720 — la colonne <i>Updater</i>	538
14.21	src/engine/setters.rs, l. 1303–1316 — comment l’argument est classé	538
14.22	src/engine/setters.rs, l. 789–804 — la provenance d’une écriture	539
14.23	src/engine/setters.rs, l. 847–858 — de la plage de blocs à la chaîne de wrappers	540
14.24	tmp/ch14/via/helpers.ts et /tmp/ch14/via/App.tsx	540
14.25	src/rules/api/query.rs, l. 763–774 — la certification d’une écriture directe	541
14.26	src/engine/written.rs, l. 37–58 — <i>Freshness</i> et <i>Written</i>	542
14.27	src/engine/written.rs, l. 66–106 — le calcul de la colonne <i>written</i>	543

14.28	src/engine/written.rs, l. 185–200 — fraîcheur d’une valeur stockée	544
14.29	src/engine/written.rs, l. 248–274 — l’environnement d’un site	544
14.30	src/engine/triggers.rs, l. 33–44 — une ligne de <i>effect_ttriggers</i>	545
14.31	tmp/relprobe/ex4.tsx	546
14.32	tmp/relprobe/ex3.tsx	547
14.33	src/engine/seeds.rs, l. 35–78 — SeedSync et la ligne SlotSeed	547
14.34	src/engine/seeds.rs, l. 108–170 — les deux kills de slot et la couverture par graine	549
14.35	tmp/ch14/regs.tsx	551
14.36	src/engine/registrations.rs, l. 218–236 — Pairing : seul Paired est une affirmation	553
14.37	src/engine/setters.rs, l. 2017–2051 — BodyCall et SlotRead	554
14.38	tmp/ch14/guarded.tsx	555
14.39	src/engine/guards.rs, l. 655–669 — la chaîne à prédécesseur unique	556
14.40	src/engine/guards.rs, l. 671–699 — les conjoints d’un site, dépliés	556
14.41	src/engine/guards.rs, l. 714–780 — les trois bras, sur des gardes conjonctives .	558
14.42	src/engine/guards.rs, l. 60–89 — la preuve mono-site	559
14.43	src/engine/program_relations.rs — les relations programme-entier, paresseuses	560
14.44	tests/layer_boundary.rs, l. 19–31 — les fichiers de règles qui touchent encore la syntaxe	562
14.45	tmp/verif07/alias.tsx	563
15.1	tmp/ch15/two_effects.tsx	569
15.2	src/engine/churn.rs, l. 85–129 — les quatre types publics du graphe	570
15.3	src/engine/churn.rs, l. 530–543 — la jambe must-reach et la force	572
15.4	src/engine/dominance.rs, l. 72–92 — on_all_paths, la primitive must-reach . . .	572
15.5	tmp/ch15/nodeps.tsx	573
15.6	src/engine/churn.rs, l. 569–576 — l’auto-arête d’un effet sans deps	574
15.7	src/engine/churn.rs, l. 577–594 — les arêtes guidées par deps, exactes puis versionnées	575
15.8	tmp/ch15/three.tsx	576
15.9	src/engine/churn.rs, l. 635–662 — find_cycles : sous-graphe Must d’abord, puis tout	577
15.10	tmp/ch15/fetch_once_pair.tsx	579
15.11	tmp/ch15/multi_writer.tsx	580
15.12	tmp/ch15/settle_each_other.tsx	581
15.13	tmp/ch15/render_write.tsx	582
15.14	src/engine/churn.rs, l. 259–287 — la collecte des sites	582
15.15	tmp/ch15/remount.tsx	583
15.16	src/engine/guards.rs, l. 92–121 — WriteSite, la vue d’un site pour la preuve . .	584
15.17	src/engine/guards.rs, l. 137–197 — converges_under_all_writes : signature et première moitié	586
15.18	src/engine/guards.rs, l. 198–258 — converges_under_all_writes, seconde moitié	587
15.19	src/engine/guards.rs, l. 264–273 — contradicts	588
15.20	src/engine/guards.rs, l. 305–315 — les faits d’invariance d’un composant	589
15.21	tmp/ch15/reviver.tsx	591
15.22	src/engine/churn.rs, l. 450–485 — le plus petit point fixe des sites convergents .	592
15.23	tmp/ch15/null_updater.tsx	593
15.24	src/engine/churn.rs, l. 502–529 — le kill d’une arête	595
15.25	src/engine/written.rs, l. 202–209 — reference_part	595
15.26	tmp/ch15/cross.tsx	596
15.27	tmp/ch15/props_move.tsx	597

15.28	src/rules/impls/infinite_loop.rs, l. 281–292 — le bras multi-effets choisit la règle et demande la preuve	599
15.29	src/rules/api/query.rs, l. 1002–1031 — must_effect_cycle re-dérive les deux faits	599
15.30	src/rules/impls/infinite_loop.rs, l. 441–460 — le bras self-churn ne fait pas confiance à e.strength	600
15.31	src/rules/helpers/cycles.rs, l. 72–93 — une ligne churn_cycles	601
16.1	/tmp/ch16/intro.tsx	607
16.2	tests/fixtures/state_lifted_too_high/drill.tsx, l. 6–19 — un état tenu trois niveaux trop haut	608
16.3	Deux Widget, un alias, une référence nue (/tmp/ch16/amb/)	609
16.4	src/ir/component_id.rs, l. 23–29 — l'identité d'un composant	610
16.5	src/ir/component_id.rs, l. 61–74 — la table d'interning	610
16.6	src/engine/component_registry.rs, l. 42–55 — construction du registre, internant en ordre de clé trié	611
16.7	src/ir/component_id.rs, l. 138–154 — le nom affiché, suffixé seulement en cas de collision	611
16.8	src/rules/api/query.rs, l. 393–404 — l'identité pour comparer, le nom pour écrire	612
16.9	src/ir/expr.rs, l. 245–258 — l'élément de composant dans l'IR	612
16.10	src/ir/expr.rs, l. 160–172 — l'origine : fichier de définition et nom exporté	613
16.11	src/engine/component_registry.rs, l. 8–20 — la réponse du registre sur un nom JSX	613
16.12	src/engine/component_registry.rs, l. 114–127 — résoudre un élément, sans jamais deviner	613
16.13	src/engine/root_detector.rs, l. 25–36 — les trois stratégies	615
16.14	src/engine/root_detector.rs, l. 57–91 — la détection des racines, stratégie par stratégie	615
16.15	/tmp/ch16/roots.tsx	616
16.16	src/engine/root_detector.rs, l. 101–110 — les noms d'entrée qui ne sélectionnent rien	617
16.17	src/domains/context.rs, l. 59–77 — le contexte d'une analyse inter-composants	618
16.18	src/domains/transfer/state_value.rs, l. 476–518 — sans contexte inter, puis résolution de l'enfant	619
16.19	src/domains/transfer/state_value.rs, l. 520–556 — récursion, props abstraites, cache	620
16.20	src/domains/transfer/state_value.rs, l. 558–593 — l'environnement de l'enfant, son analyse, l'enregistrement	621
16.21	/tmp/ch16/rec.tsx	623
16.22	src/domains/interp/interpreter.rs, l. 368–391 — l'appel d'un setter porteur de propriétaire écrit le store partagé	624
16.23	src/domains/stores/shared_state_store.rs, l. 29–34 — une mise à jour monotone	625
16.24	src/engine/fixpoint.rs, l. 486–497 — le parent importe les écritures de ses enfants	625
16.25	src/domains/transfer/state_value.rs, l. 287–296 — un setter propre au composant ou d'un ancêtre	626
16.26	/tmp/ch16/hv.tsx	627
16.27	src/engine/component_cache.rs, l. 19–27 — le cache, indexé par identité et props abstraites	627
16.28	src/engine/component_cache.rs, l. 98–119 — égalité stricte : mêmes clés, $\sqsubseteq$ dans les deux sens	627
16.29	src/engine/component_cache.rs, l. 65–92 — insertion, et éviction par join	628
16.30	/tmp/ch16/overwrite.tsx	629
16.31	src/engine/fixpoint.rs, l. 720–755 — phase 1 : les racines, sous un InterCtx	630

16.32	src/engine/fixpoint.rs, l. 757–762 — ce que la phase 1 a atteint, photographié avant la phase 2	631
16.33	src/engine/program_result.rs, l. 52–68 — ce que la phase 1 a atteint, et comment le lire	631
16.34	tmp/ch16/ctrl1.tsx	634
16.35	src/engine/render_deps.rs, l. 53–84 — une entrée de rendu, dans le repère d’un composant	635
16.36	src/engine/render_deps.rs, l. 119–131 — un ensemble may de sources, et deux compléments	636
16.37	src/engine/render_deps.rs, l. 153–170 — le join : may pour set, must pour gated	637
16.38	src/engine/render_deps.rs, l. 603–625 — un nom non lié : contexte prouvé, binding écrit, ou rien	638
16.39	src/engine/render_deps.rs, l. 689–704 — la condition de chemin d’un bloc, puis le transfert	639
16.40	src/engine/render_deps.rs, l. 301–322 — le résumé d’un composant	640
16.41	src/engine/render_deps.rs, l. 252–277 — un élément de composant construit par le rendu	640
16.42	src/engine/render_deps.rs, l. 961–979 — la garde d’un site : le chemin, et le type d’élément s’il est une valeur	641
16.43	tmp/ch16/foreach.tsx (extrait)	641
16.44	tests/fixtures/state_lifted_too_high/context.tsx, l. 6–21 — un état qui atteint son consommateur par contexte	642
16.45	src/rules/helpers/render_tree.rs, l. 226–244 — descendre tant qu’un seul enfant utilise	644
17.1	src/rules/mod.rs, l. 1–16 — la découpe de la couche règles	652
17.2	Un effet qui s’auto-relance tant que data bouge (/tmp/ch17/topdep.tsx)	653
17.3	src/rules/mod.rs, l. 75–106 — le trait Rule	655
17.4	src/rules/mod.rs, l. 61–73 — SafeCheck, une assurance positive	656
17.5	src/rules/api/query.rs, l. 314–340 — RuleCtx et sa référence au cache	656
17.6	Un hook sous condition (/tmp/ch17/profile.tsx)	657
17.7	src/rules/impls/conditional_hook.rs, l. 14–46 — la règle conditional-hook en entier	658
17.8	src/rules/mod.rs, l. 108–131 — les règles natives	658
17.9	src/rules/api/diagnostic.rs, l. 24–52 — les niveaux et leur rang	660
17.10	src/rules/api/diagnostic.rs, l. 54–73 — Diagnostic, dont seule la sévérité est privée	661
17.11	src/rules/api/diagnostic.rs, l. 1–12 — le raisonnement du module feuille	661
17.12	src/rules/api/diagnostic.rs, l. 157–190 — les trois portes	662
17.13	src/rules/api/query.rs, l. 52–57 — la provenance d’une preuve	663
17.14	src/rules/api/query.rs, l. 74–109 — le jeton de preuve Certified	664
17.15	src/rules/api/query.rs, l. 111–135 — MustResult et May	665
17.16	Forme usuelle de consommation d’un MustResult (pseudo-code)	666
17.17	src/rules/api/query.rs, l. 143–179 — StabilityVerdict, classifieur total	666
17.18	src/rules/api/query.rs, l. 221–227 — la sonde T-sûre	667
17.19	src/rules/api/query.rs, l. 417–421 — le contrat normatif des primitives	669
17.20	src/rules/api/query.rs, l. 652–683 — ExitDominance : les sorties atteignables, et leur négation	669
17.21	src/rules/api/query.rs, l. 472–511 — hook_is_conditional, la négation $\forall$ -sorties par hook	670
17.22	src/rules/api/query.rs, l. 527–552 — collecte des sites d’appel du setter	671
17.23	src/rules/api/query.rs, l. 576–621 — l’analyse must avant et sa conclusion	672
17.24	src/rules/helpers/mod.rs, l. 211–235 — la porte $\forall$ -stable	674

17.25	Un état qui gèle une prop mobile (/tmp/ch17/frozen.tsx)	675
17.26	src/rules/api/query.rs, l. 839–890 — classify_motion frappe là où il constate	675
17.27	src/rules/api/query.rs, l. 904–916 — must_frozen_seed rend ou dégrade la preuve	676
17.28	src/rules/api/query.rs, l. 942–959 — must_stale_capture : les premières portes	678
17.29	Card.tsx et api.ts	680
17.30	src/rules/api/witness.rs, l. 25–41 — Note, une étape ancrée	680
17.31	src/rules/api/witness.rs, l. 81–130 — Step, le vocabulaire des jugements	680
17.32	src/ir/source_range.rs, l. 36–49 — une position porte son fichier	682
17.33	src/rules/registry.rs, l. 356–369 — located : la première position du témoin	683
17.34	src/driver/human.rs, l. 23–35 — la ligne principale et les étapes partagent la même position	684
17.35	src/rules/registry.rs, l. 254–294 — la passe par composant, jusqu’aux filtres	685
17.36	src/rules/registry.rs, l. 29–49 — les surcharges du consommateur	686
17.37	src/rules/api/agnostic.rs, l. 104–114 — le clamp ne sait que descendre	687
17.38	src/rules/registry.rs, l. 11–17 — pourquoi off filtre à l’émission	688
17.39	src/rules/api/query.rs, l. 276–288 — la déclaration typée d’une option	689
17.40	src/rules/docs.rs, l. 11–24 — la documentation d’un nom de diagnostic	690
17.41	src/rules/api/cache.rs, l. 26–41 — ProgramCache	691
17.42	tests/layer_boundary.rs, l. 19–41 — la liste qui ne peut que rétrécir	694
18.1	src/engine/setters.rs, l. 515–528 — les deux tables syntaxiques du rendu	700
18.2	src/engine/setters.rs, l. 621–635 — la table fermée par alias, sur tous les corps	700
18.3	src/rules/impls/lazy_init.rs, l. 107–115 — le déclencheur : un appel dans l’initialiseur, sans poursuite des liaisons	702
18.4	src/rules/impls/lazy_init.rs, l. 206–234 — classification de l’initialiseur, avec sa précedence	703
18.5	src/rules/api/query.rs, l. 722–740 — la primitive qui certifie un setter dans l’initialiseur	704
18.6	src/rules/impls/lazy_init.rs, l. 148–158 — le bras Setter de useState	704
18.7	Initialiseurs gradués (/tmp/ch18/lazy.tsx)	705
18.8	Un appel caché, deux fois (/tmp/ch18/lazy.tsx, suite)	706
18.9	src/rules/helpers/providers.rs, l. 47–86 — les sites de providers prouvés, dans un ordre déterministe	707
18.10	src/rules/helpers/jsx.rs, l. 267–298 — l’identité d’une valeur à un site du rendu	708
18.11	src/domains/impls/state_value.rs, l. 281–287 — un fait must : une référence neuve, et rien d’autre	709
18.12	src/rules/impls/unstable_context_value.rs, l. 50–71 — la règle entière	709
18.13	Cinq providers (/tmp/ch18/ctx.tsx)	710
18.14	src/rules/impls/redundant_set_state.rs, l. 253–279 — le test d’un appel de setter	711
18.15	src/rules/impls/redundant_set_state.rs, l. 57–81 — les slots sur lesquels la règle n’a pas qualité pour parler	713
18.16	Écritures redondantes ou non (/tmp/ch18/redundant.tsx)	713
18.17	src/rules/impls/setter_in_render.rs, l. 63–97 — les setters locaux et étrangers	715
18.18	src/rules/impls/setter_in_render.rs, l. 109–121 — une ligne Handler ou Deferred est une preuve de non-rendu	716
18.19	src/rules/impls/setter_in_render.rs, l. 147–162 — l’unique chemin vers l’ERROR	717
18.20	src/rules/impls/setter_in_render.rs, l. 164–183 — le silence de l’idiome « adjust during render »	718
18.21	Six manières d’appeler un setter depuis le rendu (/tmp/ch18/setter.tsx)	719
18.22	src/engine/setters.rs, l. 136–157 — le repli : un site par variable, le plus petit dans l’ordre de WalkClass	721

18.23	Deux effets du repli (/tmp/ch18/timing.tsx et /tmp/ch18/tworender.tsx, élagués) .	722
18.24	src/rules/impls/derived_state.rs, l. 76–128 — les conditions de derived-state . .	723
18.25	Dérivations, vraies et fausses (/tmp/ch18/derived.tsx)	724
18.26	src/engine/setters.rs, l. 2356–2364 — « quelqu’un d’autre écrit-il ce slot ? » . .	725
18.27	Le slot 1 du parent masque le slot 1 de l’enfant (/tmp/ch18/derived_foreign.tsx)	726
18.28	src/rules/impls/state_mutation.rs, l. 37–65 — racine d’identité, site enregistré, portée lexicale	727
18.29	src/rules/impls/state_mutation.rs, l. 384–407 — un conteneur par corps de dé- clenchement	728
18.30	src/rules/impls/state_mutation.rs, l. 112–167 — chase : remonter une référence jusqu’à sa racine	729
18.31	src/rules/impls/state_mutation.rs, l. 295–334 — l’updater : son paramètre <i>est</i> le slot	730
18.32	src/rules/api/query.rs, l. 1039–1057 — la primitive : un conteneur commun . .	731
18.33	src/rules/impls/state_mutation.rs, l. 433–440 — la seule voie vers l’ERROR de state-mutation	731
18.34	Mutations d’état et de props (/tmp/ch18/mutation.tsx)	731
18.35	src/engine/seeds.rs, l. 34–47 — le verdict de synchronisation, may dans un sens	734
18.36	src/rules/impls/frozen_initial_state.rs, l. 106–115 — pas de seed, ou un chemin de synchronisation : silence	734
18.37	src/rules/impls/frozen_initial_state.rs, l. 140–174 — classer le mouvement de chaque seed	735
18.38	src/rules/helpers/mount.rs, l. 44–67 — ce que les sites d’appel prouvent du montage	736
18.39	src/rules/impls/frozen_initial_state.rs, l. 195–206 — le niveau de base d’une preuve de mouvement	737
18.40	src/rules/impls/frozen_initial_state.rs, l. 231–255 — les trois rétrogradations	737
18.41	src/rules/impls/frozen_initial_state.rs, l. 263–285 — l’ERROR consomme le jeton, que must_frozen_seed rend ou dégrade	738
18.42	Un parent, six enfants (/tmp/ch18/frozen.tsx)	739
18.43	Props $\top$ (/tmp/ch18/frozen_roots.tsx, élagué de l’import)	741
19.1	src/rules/impls/infinite_loop.rs, l. 26–37 — le type de la règle et un doc- comment en retard sur le code	748
19.2	src/rules/impls/infinite_loop.rs, l. 44–54 — l’applicabilité et l’assurance publiée sous -info	749
19.3	src/rules/impls/infinite_loop.rs, l. 56–83 — le prélude de check : tables de setters, retour anticipé, ensemble de déduplication	749
19.4	src/rules/impls/infinite_loop.rs, l. 230–241 — les bras 3 et 4, après la boucle sur les effets	750
19.5	/tmp/ch19/counter.tsx	752
19.6	src/engine/analysis_result.rs, l. 18–28 — la provenance d’un widening forcé .	753
19.7	src/engine/fixpoint.rs, l. 485–529 — le test de convergence et l’enregistrement du widening, sur render $\sqcup$ effets seulement	754
19.8	src/domains/impls/state_value.rs, l. 371–382 — is_unbounded : les trois formes de la croissance non bornée	755
19.9	src/rules/impls/infinite_loop.rs, l. 115–136 — le filtre Handler et les deux portes du bras intra	755
19.10	src/rules/impls/infinite_loop.rs, l. 96–113 — montage seul, puis suppression seulement si <i>tous</i> les deps sont prouvés stables	756
19.11	src/rules/helpers/mod.rs, l. 225–235 — all_deps_provably_stable : $\forall$ -stable, clé sur stability_verdict_of	757

19.12/tmp/ch19/deps.tsx	757
19.13src/rules/impls/infinite_loop.rs, l. 143–156 — le WARNING du bras intra . . .	758
19.14src/rules/api/witness.rs, l. 529–560 — slot_history : un Write par effet écrivain au tour du widening, puis le Widen	759
19.15/tmp/ch19/handlerfp.tsx	760
19.16/tmp/ch19/cross.tsx	761
19.17src/rules/impls/infinite_loop.rs, l. 195–226 — le bras cross-component	761
19.18/tmp/ch19/cycles.tsx	763
19.19src/rules/impls/infinite_loop.rs, l. 267–296 — une arête du cycle, un diagnostic ; la preuve est re-dérivée	765
19.20src/rules/impls/infinite_loop.rs, l. 298–334 — les six textes du bras 3 ; le texte suit la preuve, pas all_must	766
19.21/tmp/ch19/churn.tsx	768
19.22src/rules/impls/infinite_loop.rs, l. 417–433 — la partition du bras 4 : arêtes pilotees par deps, locales de bout en bout	770
19.23src/rules/impls/infinite_loop.rs, l. 439–471 — le verdict par arête : la preuve est re-dérivée des lignes d'écrivains	770
19.24src/rules/impls/infinite_loop.rs, l. 493–523 — les trois strates du bras 4 . . .	772
19.25/tmp/ch19/fields.tsx	772
19.26src/engine/churn.rs, l. 828–852 — can_retrigger : false seulement si chaque dep réactif lit un membre préservé	773
19.27/tmp/ch19/derived157.tsx	774
19.28tests/fixtures/state_lifted_too_high/drill.tsx, l. 6–19 — la fixture drill.tsx .	776
19.29src/rules/helpers/render_tree.rs, l. 34–64 — Hop, Home et le compte des rendus gaspillés	778
19.30src/rules/helpers/render_tree.rs, l. 531–590 — la descente d'uses : garde touchée, liste, élément non résolu, profondeur, récursion — tout inconnu est un usage — puis la question traduite dans le repère de l'enfant	778
19.31src/rules/impls/state_lifted_too_high.rs, l. 22–43 — les deux seuils, déclarés comme options typées	780
19.32src/rules/impls/state_lifted_too_high.rs, l. 54–77 — un foyer par slot useState, trois filtres	781
19.33src/rules/impls/state_lifted_too_high.rs, l. 96–117 — le conseil suit le nombre de sites qui montent le foyer, et le contexte éventuel	781
19.34tests/fixtures/state_lifted_too_high/context.tsx, l. 6–21 — la fixture context.tsx, première moitié	782
20.1 src/ir/hooks.rs, l. 102–107 — les trois états de l'argument de deps	788
20.2 src/ir/hooks.rs, l. 203–215 — la liste qui couvre une lecture (le commentaire de tête, l. 192–202, est résumé dans le texte)	789
20.3 src/engine/analysis_result.rs, l. 279–288 — l'environnement de sortie du rendu	789
20.4 /tmp/ch20/limits.tsx : un hook de bibliothèque, des deps illisibles, un compteur qui boucle	790
20.5 src/engine/fixpoint.rs, l. 514–529 — l'enregistrement d'un widening dans widen_trace	791
20.6 src/engine/analysis_result.rs, l. 20–28 — un événement de widening	791
20.7 src/rules/impls/widening_info.rs, l. 15–40 — le corps de check, tout ce que fait la règle	792
20.8 src/rules/registry.rs, l. 364–369 — un diagnostic sans position prend celle de sa première note	792
20.9 src/rules/impls/analysis_limit_info.rs, l. 99–108 — le hook opaque : la condition	793

20.10	src/rules/impls/analysis_limit_info.rs, l. 131–154 — les deps illisibles	793
20.11	src/rules/registry.rs, l. 287–291 — un seul analysis-limit retire toutes les assurances du composant	794
20.12	tmp/ch20/profile.tsx : un effet placé après un retour anticipé	796
20.13	src/rules/impls/conditional_hook.rs, l. 32–45 — la règle : une ERROR par jeton	796
20.14	src/rules/api/query.rs, l. 475–511 — la primitive hook_is_conditional	797
20.15	src/rules/api/query.rs, l. 652–683 — les sorties atteignables et le test may_be_skipped	798
20.16	src/rules/api/query.rs, l. 1092–1105 — le choix de la garde (début de guard_site)	799
20.17	tmp/ch20/feed.tsx : la même dep, fraîche puis mémorisée	802
20.18	src/domains/impls/stability.rs, l. 35–52 — le treillis de stabilité référentielle	802
20.19	src/domains/impls/state_value.rs, l. 281–287 — le prédicat de la règle	803
20.20	src/rules/impls/always_unstable_deps.rs, l. 71–109 — la boucle de la règle	803
20.21	tmp/ch20/search.tsx : une prop oubliée, un ref atteint par un objet frais, un objet non déclaré	806
20.22	src/ir/free_vars.rs, l. 6–23 — un chemin d'accès : une racine et une suite de champs	807
20.23	src/ir/free_vars.rs, l. 253–278 — les chemins déclarés et la couverture	807
20.24	src/engine/analysis_result.rs, l. 95–120 — la vue « deps » d'un effet, d'un memo ou d'un callback	808
20.25	La boucle de missing-deps, élaguée des commentaires (src/rules/impls/missing_deps.rs, l. 49–98)	809
20.26	src/rules/impls/missing_deps.rs, l. 139–157 — un préfixe stable suffit	810
20.27	src/rules/impls/missing_deps.rs, l. 159–181 — identité contre comportement	811
20.28	src/rules/helpers/mod.rs, l. 42–52 — des stabilités aux mots	812
20.29	tmp/ch20/width.tsx, composant Theme : deux écritures de montage	815
20.30	src/rules/impls/unnecessary_rerender.rs, l. 62–82 — la valeur initiale, évaluée avec des stores et un tas vides	815
20.31	src/rules/impls/unnecessary_rerender.rs, l. 110–146 — le balayage des écritures de montage	816
20.32	src/rules/impls/unnecessary_rerender.rs, l. 148–156 — le drapeau de montage	817
20.33	tmp/ch20/width.tsx, composant Width : un listener jamais retiré	818
20.34	src/engine/registrations.rs, l. 31–57 — comment et quand un callback enregistré tire	819
20.35	src/engine/registrations.rs, l. 257–269 — la fin de la ligne Registration	820
20.36	src/rules/api/query.rs, l. 1147–1178 — le verdict de cleanup (le type CleanupVerdict, l. 1135–1145, a trois variantes : Present, Absent, Unknown)	821
20.37	src/rules/impls/missing_cleanup.rs, l. 56–84 — un effet sans cleanup qui enregistre quelque chose de répétitif	822
20.38	tmp/ch20/timer.tsx : la forme canonique et deux variantes	824
20.39	src/rules/impls/stale_closure.rs, l. 76–114 — du callback écrit à un corps	826
20.40	src/rules/impls/stale_closure.rs, l. 343–354 — la sévérité vient du jeton, et de lui seul	827
20.41	src/rules/api/query.rs, l. 942–989 — la primitive must_stale_capture	827
21.1	Un état de saisie au-dessus d'un sous-arbre qui ne le lit pas	839
21.2	src/engine/render_deps.rs, l. 301–322 — le résumé : ce qui est utilisé, ce qui est construit	840
21.3	src/engine/render_deps.rs, l. 252–277 — un élément de composant construit par le rendu	841

21.4	src/rules/helpers/render_tree.rs, l. 766–777 — un élément ne se résout que par son origine prouvée	842
21.5	tests/fixtures/wasted_subtree_render/typing.tsx, l. 5–20 — un champ texte à côté d'un sous-arbre coûteux	842
21.6	src/rules/impls/wasted_subtree_render.rs, l. 263–292 — ce qui fait de deux écritures un seul lot	844
21.7	src/rules/helpers/render_tree.rs, l. 77–97 — l'endroit où une capacité d'écriture est appelée	845
21.8	tests/fixtures/wasted_subtree_render/child_setter.tsx, l. 5–22 — le setter descend dans <Field>	846
21.9	src/rules/impls/wasted_subtree_render.rs, l. 315–333 — peut-il appeler le setter ? derrière un test ?	846
21.10	Deux écritures dans un même tick	847
21.11	src/rules/helpers/render_tree.rs, l. 720–751 — la fréquence d'un événement sur une cible	848
21.12	src/domains/impls/state_value.rs, l. 384–401 — un slot à peu de valeurs re-rend au rythme de ses transitions	849
21.13	src/rules/helpers/render_tree.rs, l. 266–326 — les éléments que le lot ne peut pas changer	850
21.14	src/rules/helpers/render_tree.rs, l. 328–372 — le coût d'un sous-arbre gaspillé	852
21.15	src/rules/impls/wasted_subtree_render.rs, l. 33–48 — les deux options de la règle	853
21.16	Un survol à côté d'un sous-arbre (/tmp/ch21/cfg/hover_inline.tsx)	854
21.17	tests/fixtures/wasted_subtree_render/children.tsx, l. 3–16 — l'élément lourd est construit plus haut	855
21.18	src/project/nextjs.rs, l. 13–38 — les entrées App Router	858
21.19	src/project/nextjs.rs, l. 40–55 — les modules du graphe serveur	858
21.20	Le projet /tmp/ch21/nx/ (fichiers concaténés)	859
21.21	src/rules/impls/server_component_hook.rs, l. 67–111 — qu'est-ce qui prouve un hook client ?	861
21.22	src/rules/impls/server_component_hook.rs, l. 10–47 — les hooks documentés comme client	862
22.1	src/rules/declarative/mod.rs, l. 1–16 — le contrat du sous-système	867
22.2	Un setter aliasé (/tmp/ch22/alias.tsx)	868
22.3	Un hook retiré caché derrière un wrapper (/tmp/ch22/wrapper.tsx)	868
22.4	packs/guardrails.json, l. 6–18 — la règle effect-without-deps-array	871
22.5	Un titre synchronisé après chaque rendu (/tmp/ch22/ex1_no_deps.tsx)	871
22.6	src/rules/declarative/schema.rs, l. 17–56 — PackFile et RuleDef	872
22.7	Grammaire abrégée d'un pack (pseudo-EBNF, rédigée pour ce chapitre)	873
22.8	packs/guardrails.json, l. 65–71 — oversized-effect (docs omises)	874
22.9	packs/guardrails.json, l. 81–87 — banned-hook (docs omises)	874
22.10	La configuration des exemples du chapitre	874
22.11	Un hook retiré importé sous alias, un effet trop large (/tmp/ch22/ex4_banned.tsx)	874
22.12	src/rules/declarative/schema.rs, l. 899–936 — PVal, une feuille qui accepte une valeur ou une référence de paramètre	875
22.13	src/rules/declarative/entity.rs, l. 1069–1078 — le nom affiché d'un label	876
22.14	/tmp/ch22/bad/bad-sort.json — une garde stability sur un argument	877
22.15	src/rules/declarative/validate.rs, l. 59–98 — les seize sortes	877
22.16	src/rules/declarative/schema.rs, l. 238–247 — ForEach, une arête et une liaison	879
22.17	src/rules/declarative/schema.rs, l. 305–308 — le préfixe must_ rend la polarité visible	880

22.18	src/rules/declarative/validate.rs, l. 448–477 — la table unique des champs . .	881
22.19	src/rules/declarative/validate.rs, l. 669–671 — là où le refus est appliqué . . .	883
22.20	src/rules/declarative/mod.rs, l. 47–87 — load_pack, trois étages puis la cuisson	883
22.21	src/rules/declarative/validate.rs, l. 1514–1539 — le typage de la garde stability	885
22.22	src/rules/declarative/validate.rs, l. 1321–1366 — trois contrôles sur l’arbre de gardes résolu	886
22.23	src/rules/declarative/validate.rs, l. 953–964 — l’IR résolue d’une règle	889
22.24	src/rules/declarative/exec.rs, l. 37–50 — TierARule et Proof	889
22.25	src/rules/declarative/entity.rs, l. 123–153 — EntityCtx	890
22.26	src/rules/declarative/exec.rs, l. 227–245 — le début de TierARule::check . . .	891
22.27	src/rules/declarative/exec.rs, l. 408–428 — évaluer un candidat	891
22.28	packs/community/state.json, l. 14–21 — same-tick-slot-collapse (docs omises) .	892
22.29	Deux incréments qui n’en font qu’un (/tmp/ch22/ex7_same_tick.tsx)	892
22.30	packs/guardrails.json, l. 48–55 — self-retriggering-effect (docs omises) . . .	894
22.31	Deux effets qui s’auto-relancent (/tmp/ch22/ex2_self_retrigger.tsx)	894
22.32	src/rules/declarative/exec.rs, l. 688–695 — une garde certifiante	895
22.33	src/rules/declarative/exec.rs, l. 868–904 — emit, pin $\sqcap$ polarité	895
22.34	src/rules/api/diagnostic.rs, l. 104–114 — Diagnostic::clamp	896
22.35	src/rules/declarative/exec.rs, l. 696–707 — la disjonction	897
22.36	packs/guardrails.json, l. 27–38 — inert-single-dep (docs omises)	898
22.37	Des deps prouvées stables, et une dep inconnue (/tmp/ch22/ex3_every.tsx) . . .	899
22.38	src/rules/declarative/exec.rs, l. 757–785 — every	899
22.39	src/rules/declarative/exec.rs, l. 649–681 — count sur une arité exacte ou minorée	900
22.40	src/rules/declarative/schema.rs, l. 641–659 — none : la direction non sûre est ici la bonne	901
22.41	src/rules/declarative/exec.rs, l. 727–756 — none sur une arête d’une ligne de hook	902
22.42	packs/community/wave2.json, l. 349–388 — controlled-input-without-a-writer (docs et params omis)	903
22.43	tmp/ch22/ex5_input.tsx	903
22.44	packs/community/wave2.json, l. 93–131 — acquired-resource-not-released (docs et params omis)	904
22.45	tmp/ch22/ex6_resource.tsx	905
22.46	tmp/ch22/i143/pack.json — la règle de l’issue #143	908
22.47	Un setter confié à une bibliothèque, un autre à un handler (/tmp/ch22/i143/form.tsx)	908
23.1	tests/fixtures/clean.tsx, l. 7–20 — un compteur sans défaut	915
23.2	tests/fixtures/config_project/App.tsx, l. 7–14 — un setter en plein rendu et une dep manquante	915
23.3	src/cli/mod.rs, l. 31–48 — la structure clap de la ligne de commande	916
23.4	src/cli/mod.rs, l. 110–127 — le dispatch des sous-commandes	917
23.5	tests/fixtures/config_discover/reactant.config.json (verbatim)	918
23.6	src/config.rs, l. 26–67 — le type de la configuration	919
23.7	src/config.rs, l. 96–104 — un réglage de règle normalisé	920
23.8	src/cli/config_load.rs, l. 55–91 — chargement des packs, chaque échec est bruyant	921
23.9	src/cli/check.rs, l. 62–75 — trois Option sans défaut clap	922
23.10	src/config.rs, l. 250–269 — la fusion drapeaux / configuration	922
23.11	src/config.rs, l. 271–293 — la résolution des overrides	923
23.12	src/rules/api/diagnostic.rs, l. 104–114 — le clamp, qui ne fait que baisser . . .	924
23.13	src/cli/check.rs, l. 192–202 — l’hôte natif appelle le driver, imprime, sort . . .	925
23.14	src/driver/mod.rs, l. 1–10 — le contrat du driver	925
23.15	src/driver/mod.rs, l. 61–90 — options résolues et sortie tamponnée	926

23.16	src/resolver/filesystem.rs, l. 12–19 — la couture en lecture seule	927
23.17	src/driver/mod.rs, l. 115–129 — un premier argument qui ne nomme rien . . .	927
23.18	src/driver/mod.rs, l. 307–321 — un <code>-entry</code> qui ne sélectionne rien est une erreur	929
23.19	src/driver/mod.rs, l. 366–374 — le seul branchement de <code>new_with_common</code> dans un run réel	929
23.20	src/driver/mod.rs, l. 480–484 — la politique de sortie	930
23.21	Les constantes de la découverte (élagué : quatre lignes non contiguës, src/resolver/mod.rs, l. 509, 515, 526, 533)	931
23.22	src/resolver/mod.rs, l. 559–575 — la politique de répertoires	931
23.23	src/resolver/gitignore.rs, l. 98–114 — les <code>.gitignore</code> au-dessus de la racine de marche	932
23.24	src/resolver/gitignore.rs, l. 59–75 — verdict d'un fichier : la dernière règle gagne	933
23.25	tests/fixtures/vite_project/src/App.tsx, l. 5–10 — un import par l'alias <code>@/*</code> . .	933
23.26	tests/fixtures/vite_project/src/hooks/useData.ts, l. 4–10 — le hook, qui boucle	934
23.27	src/project/mod.rs, l. 26–37 — les trois conventions reconnues	934
23.28	src/project/mod.rs, l. 80–105 — la remontée vers le marqueur le plus proche . .	935
23.29	src/project/mod.rs, l. 171–215 — <code>build_context</code> , première moitié	935
23.30	src/project/tsconfig.rs, l. 16–27 — <code>baseUrl</code> et <code>paths</code> , en forme absolue	937
23.31	src/project/tsconfig.rs, l. 221–245 — l'héritage par <code>extends</code>	938
23.32	src/project/mod.rs, l. 121–133 — le plus proche ancêtre qui déclare de vrais <code>paths</code>	938
23.33	src/project/paths_resolver.rs, l. 78–116 — résolution TypeScript : exacts, plus long préfixe, puis <code>baseUrl</code>	940
23.34	src/resolver/mod.rs, l. 164–178 — deux erreurs de parse qui ne se valent pas . .	942
23.35	src/resolver/mod.rs, l. 270–312 — lecture, parse, et le seul critère d'abandon .	942
23.36	src/resolver/mod.rs, l. 193–200 — le type source, JSX partout sauf dans <code>.ts</code> . .	943
23.37	src/driver/mod.rs, l. 236–260 — bruit récupéré contre perte silencieuse	944
23.38	src/driver/blind_spots.rs, l. 22–31 — une raison pour laquelle le silence n'est pas une preuve	945
23.39	src/driver/mod.rs, l. 262–280 — les arêtes résolues qui sortent de l'ensemble analysé	945
23.40	src/resolver/closure.rs, l. 44–66 — la fermeture transitive des imports	946
23.41	src/driver/mod.rs, l. 440–451 — un composant hors des chemins nommés : compté, pas montré	947
23.42	src/project/nextjs.rs, l. 26–55 — entrées serveur et graphe serveur	948
23.43	src/driver/human.rs, l. 23–35 — <code>(line L:C)</code> ou <code>(chemin:L:C)</code>	951
23.44	src/driver/locations.rs, l. 24–40 — la clé de groupage et l'index	952
23.45	src/driver/json.rs, l. 17–31 — la racine du rapport JSON	953
23.46	Deux <code>useQuery</code> , deux provenances (<code>/tmp/ch23/sum/Nav.tsx</code>)	954
23.47	src/registry/summary.rs, l. 52–60 — la clé des résumés	955
23.48	src/registry/summary.rs, l. 166–177 — la recherche : paquet exact, puis non scopé	955
23.49	src/registry/summary.rs, l. 10–31 — le contrat abstrait d'un hook sans source (début)	955
24.1	Cargo.toml, l. 13–33 — workspace, features et cibles de la crate racine	962
24.2	crates/reactant-wasm/Cargo.toml, l. 1–15 — le manifeste de la crate WASM . . .	962
24.3	npm/build.sh, l. 13–34 — les étapes de construction du paquet npm	963
24.4	crates/reactant-wasm/src/lib.rs, l. 1–10 — l'en-tête du cœur WASM	964
24.5	crates/reactant-wasm/src/lib.rs, l. 50–60 — l'hôte ne parse jamais la configuration	965
24.6	crates/reactant-wasm/src/lib.rs, l. 85–105 — l'enveloppe d'entrée	966
24.7	crates/reactant-wasm/src/lib.rs, l. 170–212 — la revalidation de la configuration et des packs	966

24.8	crates/reactant-wasm/src/lib.rs, l. 237–251 — la commande check sous WASM (extrait)	967
24.9	npm/package.json, l. 9–21 — points d’entrée du paquet npm	969
24.10	npm/lib/core-node.js, l. 45–55 — amorcer la glue web sous Node	970
24.11	Usage navigateur de l’API (extrait de docs/usage.md, élagué)	970
24.12	npm/lib/host.js, l. 95–103 — les fichiers que l’hôte charge	971
24.13	src/cli/config_load.rs, l. 98–108 — la résolution native d’un nom de pack . . .	971
24.14	npm/lib/host.js, l. 55–74 — la résolution npm d’un nom de pack	972
24.15	npm/test/fixtures/team.pack.cjs, l. 12–29 — engendrer une règle par ligne d’une table	973
24.16	npm/lib/packs.js, l. 72–91 — valider par le cœur avant d’écrire	973
24.17	npm/test/smoke.sh, l. 28–40 — les treize invocations comparées	974
24.18	Usage de l’Action (extrait du README.md, élagué)	976
24.19	scripts/gh-action.mjs, l. 18–25 — l’invocation de npx	976
24.20	scripts/gh-action.mjs, l. 46–62 — une annotation par erreur de parse et par diagnostic	976
24.21	Annotations reconstituées à la main à partir de scripts/gh-action.mjs (non exécuté, élagué)	977
24.22	action.yml, l. 53–60 — la sortie blind-spots	977
24.23	github/workflows/ci.yml, l. 94–106 — épingler wasm-bindgen-cli sur Cargo.lock	978
25.1	tests/fixtures/clean.tsx, l. 7–20 — un compteur correct	983
25.2	src/rules/api/diagnostic.rs, l. 24–39 — les trois niveaux et leur contrat	984
25.3	src/rules/api/diagnostic.rs, l. 104–114 — le seul changement de niveau exposé descend	985
25.4	Trois formes de churn d’objet (/tmp/ch25/churn.tsx, construit d’après tests/corpus_fp_fixes.rs, l. 472–521)	986
25.5	src/test_support.rs, l. 26–52 — les constructeurs de CFG et d’identités	987
25.6	tests/corpus_fp_fixes.rs, l. 14–69 — le harnais source → règles	988
25.7	tests/cli.rs, l. 360–398 — la sortie doit être identique octet pour octet d’un run à l’autre	990
25.8	tests/layer_boundary.rs, l. 19–41 — la liste qui ne peut que rétrécir, et ce qui compte comme parcourir la syntaxe	991
25.9	tests/layer_boundary.rs, l. 54–59 — la détection, purement textuelle	992
25.10	tests/corpus_fp_fixes.rs, l. 1694–1713 — la forme corrigée : la garde est réglée par l’écriture	993
25.11	tests/corpus_fp_fixes.rs, l. 1760–1800 — les deux formes voisines qui doivent tirer encore	993
25.12	La paire #91 et ses deux témoins (/tmp/ch25/settle.tsx, d’après tests/corpus_fp_fixes.rs, l. 1698–1800)	994
25.13	tests/fixtures/try_catch_finally/App.tsx, l. 29–49 — la rétrogradation et son témoin	995
25.14	tests/fixtures/blind_spots/outside_root/shared/useThing.ts, l. 1–10 — une boucle infinie que personne ne voit quand on analyse app/ seul	997
25.15	tests/blind_spots.rs, l. 31–51 — le témoin et le retrait de la coche	997
25.16	tests/blind_spots.rs, l. 63–80 — ce que l’angle mort cachait, et ce qu’il ne change pas	998
25.17	scripts/setup-test-repo.sh, l. 8–17 — pourquoi chaque entrée est épinglée . . .	999
25.18	scripts/setup-test-repo.sh, l. 102–107 — un clone superficiel sur un SHA . . .	1000
25.19	scripts/corpus-baseline.py, l. 36–40 — l’unité comparable, dupliquée à l’identique dans corpus-diff.py	1002

25.20	docs/corpus-baseline.json, l. 1–23 — le total, le digest et la ventilation par règle	1002
25.21	scripts/corpus-baseline.py, l. 53–79 — le digest et l’empreinte	1003
25.22	scripts/corpus-baseline.py, l. 114–130 — l’identité du corpus avant le digest . .	1004
25.23	scripts/corpus-diff.py, l. 84–97 — deux ensembles, trois nombres, une réconciliation	1005
25.24	tests/corpus_baseline.rs, l. 33–67 — la baseline doit être cohérente avec elle-même	1006
25.25	github/workflows/corpus.yml, l. 60–86 — vérifier le corpus, mesurer, comparer, garder l’artefact	1009
25.26	tests/fixtures/community_wave2/state7.tsx, l. 1–21 — un slot écrit mais jamais lu au rendu, et son quasi-voisin	1012
25.27	scripts/rerender-bench/run.mjs, l. 13–31 — le compteur est injecté au build, les scénarios restent du React pur	1014
25.28	scripts/rerender-bench/scenarios/02-drill-before.tsx, l. 1–21 — prop drilling : un état possédé trois niveaux trop haut	1015
26.1	Trois formes syntaxiques pour un même flot de contrôle	1023
26.2	Un callback passé à une fonction inconnue	1025
26.3	src/domains/interp/callbacks.rs, l. 6–23 — les quatre classes de déclencheurs d’ADR-009	1026
26.4	/tmp/ch26/ex1.tsx	1027
26.5	src/domains/impls/state_value.rs, l. 16–24 — la justification du produit, en tête du type	1027
26.6	/tmp/ch26/ex2.tsx	1028
26.7	CLAUDE.md, l. 5–28 — les trois principes et les deux invariants	1029
26.8	/tmp/ch26/ex8.tsx	1032
26.9	src/rules/helpers/mod.rs, l. 211–235 — la garde quantifiée $\forall$ -stable	1033
26.10	src/rules/api/query.rs, l. 74–93 — le jeton de preuve et son constructeur privé	1034
26.11	/tmp/ch26/ex7.tsx	1035
26.12	src/engine/registrations.rs, l. 40–57 — les trois moments d’exécution d’un callback enregistré	1036
26.13	/tmp/ch26/ex5.tsx, composant Wrapped (extrait)	1037
26.14	tests/layer_boundary.rs, l. 19–41 — la liste qui ne peut que rétrécir, et les marqueurs de syntaxe	1037
26.15	src/ir/component_id.rs, l. 23–29 — l’identité d’un composant	1039
26.16	/tmp/ch26/ex4.tsx (extrait)	1040
27.1	/tmp/r27/updater/Updater.tsx : la même boucle, deux écritures	1050
27.2	src/domains/interp/interpreter.rs, l. 368–391 — le second bras	1050
27.3	/tmp/r27/misc/Misc.tsx et son témoin /tmp/r27/ctrl/Ctrl.tsx (extraits)	1051
27.4	src/domains/interp/interpreter.rs, l. 81–89 — seul ExprStmt déclenche l’écriture d’état	1052
27.5	/tmp/r27/reducer/Reducer.tsx	1052
27.6	src/lowering/hook_extractor.rs, l. 715–719 — useReducer devient un HookEntry::State	1053
27.7	/tmp/r27/memo/MemoLoop.tsx	1053
27.8	src/domains/stores/memo_store.rs, l. 27–30 — un mémo pas encore calculé vaut $\top$	1054
27.9	src/engine/fixpoint.rs, l. 514–526 — widen_trace n’enregistre que les slots élargis	1054
27.10	/tmp/r27/misc/Misc.tsx, composant StrGrow	1054
27.11	src/domains/impls/state_value.rs, l. 371–382 — is_unbounded ignore l’emplace- ment str	1055
27.12	/tmp/r27/ret/Ret.tsx; le témoin /tmp/r27/ctrl/Ctrl.tsx écrit set(n + 1); sans return	1055
27.13	src/ir/splice.rs, l. 158–174 — le bras Return du splice	1055

27.14/tmp/r27/persp/Persp.tsx (extrait); le témoin /tmp/r27/persp2/One.tsx ne garde que la première instance	1056
27.15src/domains/transfer/state_value.rs, l. 582–590 — le résultat d'un enfant écrase le précédent	1056
27.16/tmp/r27/alias/Alias.tsx	1057
27.17src/rules/impls/setter_in_render.rs, l. 63–86 — les setters locaux de setter-in-render	1057
27.18/tmp/r27/ctx19/Ctx.tsx	1058
27.19src/rules/helpers/providers.rs, l. 88–99 — un provider se reconnaît à .Provider	1058
27.20/tmp/r27/switch/App.tsx	1059
27.21src/lowering/jsx_detect.rs, l. 14–45 — les formes de retour qui comptent pour la règle 2	1060
27.22/tmp/r27/fp/Fp.tsx, composant TwoConsts	1061
27.23/tmp/r27/fp/Fp.tsx, composant ExclusiveBranches	1061
27.24src/rules/api/query.rs, l. 1039–1057 — la preuve « même conteneur » de state-mutation	1062
27.25/tmp/r27/fp/Fp.tsx, composant Derived	1062
27.26/tmp/r27/persp/Persp.tsx, composant Toggle	1067
A.1 src/domains/interp/callbacks.rs, l. 6–23 — TriggerClass — la classification qui décide si un appelé est descendu	1076
A.2 src/domains/interp/interpreter.rs, l. 21–21 — la profondeur d'inlining des appels locaux	1077
A.3 src/domains/impls/state_value.rs, l. 16–44 — StateValue — le domaine produit sur sortes disjointes	1079
A.4 Compteur initialisé à null	1080
A.5 src/domains/impls/stability.rs, l. 35–64 — Stability — le treillis versionné : Versioned/VersionedTop (may) et PerRender (must)	1081
A.6 src/rules/api/query.rs, l. 111–129 — MustResult<T> — le verdict must à trois valeurs, et May<T>, sans chemin vers une ERROR	1084
A.7 src/rules/api/query.rs, l. 74–84 — Certified<E> — le jeton scellé : constructeur privé au module	1085
A.8 src/ir/cfg.rs, l. 12–30 — Terminator — Return sans span, Unreachable réservé à l'arrêt de contrôle	1087
A.9 useCart : garde par exception avant les hooks	1088
A.10 src/engine/setters.rs, l. 681–701 — WriterPhase — la phase d'exécution d'une écriture, un verdict may	1089
A.11 src/engine/setters.rs, l. 728–745 — SlotWriter — une ligne par site, et pourquoi le renversement est monotone	1090
A.12 src/engine/registrations.rs, l. 40–57 — Timing — « le contrat, pas le nom »	1094
A.13 src/ir/cfg.rs, l. 32–45 — EdgeKind — l'arête Await, contrôle inchangé, phase rompue	1095
A.14 Setter enveloppé, appelé en rendu	1097
A.15 src/ir/component_id.rs, l. 23–29 — ComponentId — interné, comparable en une instruction	1098
A.16 src/engine/program_relations.rs, l. 21–44 — ProgramRelations — le graphe de churn construit une fois, en aval du point fixe	1100
A.17 src/engine/setters.rs, l. 770–786 — Les colonnes owner, block, written de SlotWriter	1100
A.18 src/engine/written.rs, l. 37–58 — Freshness et Written — ce qu'une écriture stocke, pour les preuves de churn	1101

A.19	src/engine/triggers.rs, l. 33–44 — EffectTrigger — une ligne par dépendance d’effet qu’un slot fait bouger	1101
C.1	src/rules/registry.rs, l. 11–17 — la discipline de nommage : deux espaces de noms, une seule raison	1115
C.2	src/rules/docs.rs, l. 165–178 — une entrée de RULE_DOCS : le résumé, l’explication, l’exemple et la correction d’infinite-loop	1116

Première partie

Le problème et son contexte

Chapitre 1

Introduction : pourquoi analyser statiquement React ?

Objectifs du chapitre.

- Reconnaître trois défauts React typiques et ce qui se passe réellement à l'exécution.
- Comprendre pourquoi un linter syntaxique ne suffit pas.
- Saisir l'idée directrice : interpréter abstraitement un composant.
- Connaître la promesse de l'outil : trois niveaux de diagnostic et la soundness.
- Lancer `reactant-analyzer` et lire ses sorties humaine et JSON.
- Situer le projet (taille, règles, décisions) et choisir un parcours de lecture.

Prérequis : aucun. Il suffit de savoir écrire un composant React avec `useState` et `useEffect`, et de lire du Rust.

Un composant React est une fonction courte. On la lit en quelques secondes, on la teste en quelques minutes, et pourtant les défauts qu'elle contient sont parmi les plus coûteux d'une application web : un onglet qui gèle, un écran qui affiche les données d'un autre utilisateur, un chronomètre qui reste bloqué. Ces défauts ont un point commun : ils ne se voient pas dans le texte du composant, mais dans la *suite de ses exécutions*. Ce chapitre explique pourquoi il faut, pour les trouver, raisonner sur toutes ces exécutions à la fois, et comment `reactant-analyzer` s'y prend. Il reste volontairement informel : les chapitres suivants reprennent chaque idée avec sa définition exacte et le code qui l'incarne.

1.1 Trois programmes React fautifs

Commençons par trois composants très simples. Chacun compile, s'affiche, et passe un test qui vérifie le premier rendu. Chacun est pourtant faux. Pour comprendre en quoi, il faut d'abord rappeler dans quel ordre React exécute un composant.

Rappel React : le cycle rendu, commit, effets

React exécute un composant en trois temps [Met26c].

1. **Déclenchement.** Un rendu est demandé soit parce que le composant apparaît pour la première fois (le *montage*), soit parce qu'un état a été mis à jour par un setter (`setN(...)`) dans ce composant ou dans l'un de ses ancêtres.
2. **Rendu.** React *appelle* la fonction du composant. Elle lit l'état courant, calcule le JSX et ne doit rien faire d'autre : le rendu est un calcul pur.
3. **Commit.** React reporte dans le DOM les différences entre l'ancien et le nouveau JSX.

Viennent ensuite les **effets** : après le commit, React exécute les fonctions passées à `useEffect`. Sans tableau de deps, un effet s'exécute après *chaque* commit ; avec `[]`, une seule fois après le montage ; avec `[a, b]`, seulement si l'un des deps a changé depuis le rendu précédent, la comparaison se faisant par `Object.is`. Enfin, un setter ne modifie pas la variable du rendu en cours : il *planifie* un nouveau rendu, dans lequel `useState` renverra la nouvelle valeur. Si cette valeur est identique à l'ancienne (encore au sens de `Object.is`), React peut s'épargner le re-rendu.

Figure 1.1 résume ce cycle. La flèche qui remonte des effets vers le rendu est la source de tous les ennuis de ce chapitre : un effet qui appelle un setter relance le cycle.

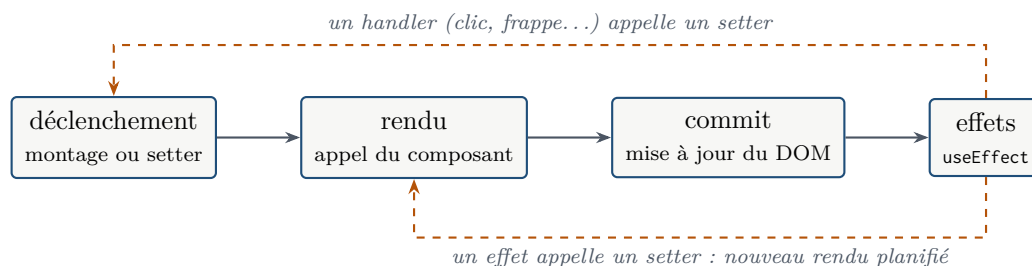


FIGURE 1.1 – Le cycle d'exécution d'un composant React. Les flèches pointillées sont les retours possibles : un effet ou un handler qui appelle un setter planifie un nouveau rendu.

1.1.1 Premier défaut : la boucle infinie

Extrait 1.1 – Counter : un effet sans deps qui écrit l'état qu'il lit (`/tmp/ch01/loop.tsx`)

```

1 import { useEffect, useState } from "react";
2
3 export function Counter() {
4   const [n, setN] = useState(0);
5   useEffect(() => {
6     setN(n + 1);
7   });
8   return <p>{n}</p>;
9 }

```

Déroulons la liste 1.1 à la main. Au montage, `n` vaut `0` ; le rendu produit `<p>0</p>`, React le commit, puis exécute l'effet, qui appelle `setN(1)`. Un rendu est planifié : `n` vaut `1`, commit, et comme l'effet n'a pas de tableau de deps, il s'exécute de nouveau et appelle `setN(2)`. Et ainsi de suite : `n` prend les valeurs `0, 1, 2, 3, ...` sans fin, chaque valeur étant différente de la précédente, si bien que React ne peut jamais s'épargner le re-rendu. Le navigateur passe son temps à rendre ce composant ; en développement, React finit par se plaindre dans la console d'un nombre excessif de mises à jour imbriquées.

Ce défaut est *certain* : aucune entrée, aucun clic, aucune prop ne l'empêche. Il n'apparaît pourtant pas dans un test qui ne regarde que le premier rendu, puisque celui-ci est correct.

1.1.2 Deuxième défaut : la dépendance oubliée

Extrait 1.2 – Profile : l'effet lit `userId` mais ne le déclare pas (`/tmp/ch01/deps.tsx`)

```

1 import { useEffect, useState } from "react";
2

```

```

3 export function Profile({ userId }: { userId: string }) {
4   const [user, setUser] = useState(null);
5   useEffect(() => {
6     fetch(`/api/users/${userId}`)
7       .then((r) => r.json())
8       .then(setUser);
9   }, []);
10  return <p>{user ? user.name : "..."}</p>;
11 }

```

Ici, pas de boucle. Au montage, l'effet charge l'utilisateur `userId` et l'affiche. Mais le tableau de deps est vide : l'effet ne s'exécutera *plus jamais*. Si le parent rend ensuite `<Profile userId="bob" />` à la place de `<Profile userId="alice" />`, le composant est re-rendu avec la nouvelle prop, mais l'effet ne tourne pas, et l'écran continue d'afficher Alice. Rien ne plante ; l'interface ment, en silence.

Ce défaut-là n'est pas certain au même sens que le premier : il ne se manifeste que si `userId` change effectivement pendant la vie du composant, ce que le composant seul ne permet pas de savoir.

1.1.3 Troisième défaut : la fermeture figée

Extrait 1.3 – Timer : le callback de `setInterval` lit une valeur figée au montage (`/tmp/ch01/timer.tsx`)

```

1 import { useEffect, useState } from "react";
2
3 export function Timer() {
4   const [seconds, setSeconds] = useState(0);
5   useEffect(() => {
6     const id = setInterval(() => {
7       setSeconds(seconds + 1);
8     }, 1000);
9     return () => clearInterval(id);
10  }, []);
11  return <p>{seconds} s</p>;
12 }

```

L'effet de la liste 1.3 s'exécute une fois, au montage, et installe un intervalle. Le callback passé à `setInterval` est une fermeture (*closure*) : il capture la variable `seconds` du rendu de montage, qui vaut 0, et la gardera toute sa vie. À la première seconde, il appelle `setSeconds(0 + 1)` : l'écran affiche « 1 s ». À la deuxième, il appelle encore `setSeconds(0 + 1)`, car sa variable `seconds` vaut toujours 0. L'état reste à 1, React constate par `Object.is` que rien n'a changé, et le chronomètre reste bloqué à « 1 s » pour toujours. On parle de *stale closure* (fermeture périmée).

La correction consiste à ne plus lire la variable capturée, mais à passer au setter une *fonction de mise à jour* (un *updater*) : `setSeconds((s) => s + 1)`. React appelle cette fonction avec la valeur courante de l'état, et non celle du montage.

1.1.4 Ce que ces trois exemples ont en commun

Tableau 1.1 met les trois défauts côte à côte. Aucun n'est visible au premier rendu ; tous dépendent de la *succession* des rendus, commits et effets ; et chacun demande de savoir quelque chose sur *toutes* les exécutions possibles du composant : que `n` ne se stabilise jamais, que `userId` peut changer, que `seconds` lu par le callback est toujours la valeur du montage.

Composant	Symptôme à l'exécution	Condition d'apparition	Nature
Counter	rendus sans fin, onglet saturé	toujours, dès le montage	certain
Profile	données d'un autre utilisateur	si la prop <code>userId</code> change	possible
Timer	affichage bloqué à « 1 s »	toujours, dès la deuxième seconde	certain

TABLE 1.1 – Les trois défauts d'ouverture. La dernière colonne annonce la distinction entre défaut certain et défaut possible, qui structure les niveaux de diagnostic (section 1.4).

1.2 Ce qu'un linter syntaxique voit, et ce qu'il ne voit pas

L'outil que tout projet React installe d'abord est un *linter* : `eslint-plugin-react-hooks`, avec ses deux règles historiques `rules-of-hooks` et `exhaustive-deps` [Met26d]. Il est rapide, il tourne dans l'éditeur, et il attrape une partie des défauts précédents. Pour comprendre ce qu'il ne peut pas attraper, il faut préciser ce qu'il regarde.

Définition 1.1: Analyse syntaxique d'un composant

Une *analyse syntaxique* (ou *lexicale*) décide à partir de la *forme* du texte : elle parcourt l'arbre de syntaxe d'une fonction, dans un fichier, et reconnaît des motifs (un appel `useX` sous un `if`, un identifiant lu dans un effet et absent du tableau de `deps`). Elle ne calcule pas les *valeurs* que prennent les variables quand le programme s'exécute.

1.2.1 Ce que la forme suffit à voir

Deux des défauts de la section 1.1 sont écrits « en toutes lettres ». Dans `Profile` (liste 1.2), l'identifiant `userId` apparaît dans le corps de l'effet et pas dans `[]` : il suffit de comparer deux listes de noms, et c'est exactement ce que fait `exhaustive-deps`. De même, un hook appelé après un retour anticipé est reconnaissable sur l'arbre de syntaxe :

Extrait 1.4 – `Panel` : un `useState` après un retour anticipé (`/tmp/ch01/cond.tsx`)

```
1 import { useState } from "react";
2
3 export function Panel({ open }: { open: boolean }) {
4   if (!open) return null;
5   const [tab, setTab] = useState(0);
6   return <button onClick={() => setTab(tab + 1)}>{tab}</button>;
7 }
```

Rappel React : les hooks sont identifiés par leur rang

React ne connaît pas le *nom* des variables d'état : il range les hooks d'un composant dans une liste, dans l'ordre où ils sont appelés, et les retrouve au rendu suivant par leur *position* [Met26d]. Si un rendu appelle un hook de moins (ici, quand `open` vaut `false`), tous les hooks suivants sont décalés d'un cran. D'où la règle : les hooks s'appellent au premier niveau du composant, jamais sous une condition, une boucle ou après un `return` conditionnel.

`reactant-analyzer` signale ces deux composants, comme le ferait le linter :

```
Panel (2 hooks) /tmp/ch01/cond.tsx
  error conditional-hook [hook:0] (line 5:8) this hook is called conditionally (not on every render path)
    (1 trace step(s), rerun with --trace)

Profile (2 hooks) /tmp/ch01/deps.tsx
  warn missing-deps [hook:1] var:userId (line 5:2) `userId` is used in this effect but not in its deps array, and
    ↳ its value may change between renders
```

```
(1 trace step(s), rerun with --trace)
```

(Sorties de deux exécutions séparées, check `-no-color` sur chacun des fichiers, juxtaposées ; les lignes de bilan sont omises.) On lira ces lignes en détail à la section 1.5. Remarquons seulement que les deux diagnostics n'ont pas le même niveau : `error` pour le hook conditionnel, `warn` pour la dépendance oubliée. Ce n'est pas un hasard, et la section 1.4 en donne la raison.

1.2.2 Ce que la forme ne voit pas

Reprenons maintenant la boucle infinie de Counter, mais écrite « proprement », avec un tableau de deps complet :

Extrait 1.5 – Counter avec des deps complètes (/tmp/ch01/deps_ok.tsx)

```
1 import { useEffect, useState } from "react";
2
3 export function Counter() {
4   const [n, setN] = useState(0);
5   useEffect(() => {
6     setN(n + 1);
7   }, [n]);
8   return <p>{n}</p>;
9 }
```

Pour exhaustive-deps, tout va bien : l'effet lit `n`, et `n` est dans ses deps. Et pourtant le composant boucle exactement comme la liste 1.1 : à chaque exécution, l'effet écrit une valeur différente de `n`, donc le dep `n` change, donc l'effet se ré-exécute. La règle a vérifié une propriété de la *forme* (« la liste est complète ») ; le défaut est une propriété de la *suite des valeurs* (« `n` ne se stabilise jamais »). Aucun motif syntaxique ne peut trancher ce second point en général : il faudrait savoir que `n + 1` n'est jamais égal à `n`, et que rien ne vient arrêter la progression. Comparez avec la variante bornée `if (n < 3) setN(n + 1)`, syntaxiquement presque identique, qui s'arrête d'elle-même à 3.

Même si l'on acceptait une règle approximative, du genre « un effet qui appelle le setter d'un état figurant dans ses deps », il suffit d'une indirection pour que le motif ne corresponde plus. Voici les trois indirections que le plan de ce chapitre annonçait, chacune sur un programme de quelques lignes.

Un alias de setter. Le setter est rangé dans une autre variable avant d'être appelé :

Extrait 1.6 – Alias de setter (/tmp/ch01/alias.tsx)

```
1 import { useEffect, useState } from "react";
2
3 export function Counter() {
4   const [n, setN] = useState(0);
5   const bump = setN;
6   useEffect(() => {
7     bump(n + 1);
8   }, [n]);
9   return <p>{n}</p>;
10 }
```

Syntaxiquement, `bump` est une variable locale quelconque. Pour savoir qu'appeler `bump` écrit l'état `n`, il faut suivre la *valeur* de `bump` depuis sa définition.

Un hook personnalisé. L'état et l'effet vivent dans un hook `useTicker`, que le composant se contente d'appeler :

Extrait 1.7 – Boucle cachée dans un hook personnalisé (/tmp/ch01/custom_hook.tsx)

```
1 import { useEffect, useState } from "react";
2
3 function useTicker() {
4   const [n, setN] = useState(0);
5   useEffect(() => {
6     setN(n + 1);
7   }, [n]);
8   return n;
9 }
10
11 export function Clock() {
12   const n = useTicker();
13   return <p>{n}</p>;
14 }
```

Le défaut est dans `useTicker`, mais c'est `Clock` qui boucle : un hook n'a pas de cycle de vie propre, il s'exécute *dans* le composant qui l'appelle. Si `useTicker` est dans un autre fichier, une règle qui travaille fichier par fichier ne voit plus du tout l'effet depuis `Clock`.

Un setter passé en prop. Le parent possède l'état et donne son setter à un enfant, qui l'appelle dans un effet :

Extrait 1.8 – Setter du parent appelé par l'enfant (/tmp/ch01/prop_setter.tsx)

```
1 import { useEffect, useState } from "react";
2
3 function Child({ value, onChange }: { value: number; onChange: (v: number) => void }) {
4   useEffect(() => {
5     onChange(value + 1);
6   }, [value, onChange]);
7   return <p>{value}</p>;
8 }
9
10 export function Parent() {
11   const [n, setN] = useState(0);
12   return <Child value={n} onChange={setN} />;
13 }
```

Ici, aucun des deux composants n'est fautif isolément. `Parent` ne fait que transmettre ; `Child` appelle une fonction `onChange` qu'il reçoit, et ses deps sont complètes. La boucle n'existe que dans la *composition* : l'effet de `Child` écrit l'état de `Parent`, `Parent` se re-rend et re-rend `Child` avec une nouvelle valeur, l'effet se ré-exécute.

`reactant-analyzer` signale les trois (une seule invocation sur les trois fichiers, avec `-trace`) :

```
Child (1 hooks) prop_setter.tsx
  warn cross-component-infinite-loop [hook:0] (line 4:2) this effect calls `onChange`, a state setter of parent `
    ↳ Parent` (its deps do not provably gate it, so the effect can re-run every render). Parent re-renders → child re
    ↳ -renders → effect fires again: infinite loop
Clock (1 hooks) custom_hook.tsx
  warn infinite-loop [hook:1] (line 5:2) this effect keeps pushing state `n` (its deps do not provably gate it, so
    ↳ the effect can re-run every render) to new values on every run. Potential infinite render loop
    ↳ state `n` is written here [hook:2] (line 5:2)
    ↳ the abstract value of state `n` kept growing and was widened at iteration 3
Counter (2 hooks) alias.tsx
  warn infinite-loop [hook:0] (line 6:2) this effect keeps pushing state `n` (its deps do not provably gate it, so
    ↳ the effect can re-run every render) to new values on every run. Potential infinite render loop
```

```

    → state `n` is written here [hook:1] (line 6:2)
    → the abstract value of state `n` kept growing and was widened at iteration 3
1 clean component(s) hidden, rerun with --show-clean

! 3 warning(s) across 3 file(s).

```

Trois détails de cette sortie annoncent la suite du manuscrit. Le finding de `useTicker` est rapporté sur le composant `Clock`, qui est celui qui boucle : le hook a été *inliné* dans son appelant (chapitre 8). Celui de `Child` nomme le parent `Parent` dont il écrit l'état : l'analyse a suivi le setter à travers la prop (chapitre 16). Enfin, deux lignes parlent de la « valeur abstraite » de `n`, qui « a grandi » puis a été « élargie à l'itération 3 » : c'est le vocabulaire de l'interprétation abstraite, que la section 1.3 présente.

Le hook personnalisé placé dans un autre fichier se comporte de même. Avec `useTicker` déplacé dans `/tmp/ch01/xfile/useTicker.ts` et `Clock` dans `/tmp/ch01/xfile/Clock.tsx`, l'analyse du dossier entier donne :

```

Clock (1 hooks) Clock.tsx
warn  infinite-loop [hook:1] (useTicker.ts:5:2) this effect keeps pushing state `n` (its deps do not provably
    ↪ gate it, so the effect can re-run every render) to new values on every run. Potential infinite render loop
    → state `n` is written here [hook:2] (useTicker.ts:5:2)
    → the abstract value of state `n` kept growing and was widened at iteration 3

! 1 warning(s) across 2 file(s).

```

La position (`useTicker.ts:5:2`) désigne un autre fichier que celui du composant : le finding est ancré dans le hook, mais porté par le composant qui le subit (ADR-24).

Programme	Ce que la forme montre	Ce qu'il faut calculer
Panel (liste 1.4)	hook après un <code>return</code> conditionnel	rien de plus
Profile (liste 1.2)	<code>userId</code> lu, absent des deps	que <code>userId</code> peut changer
Counter (liste 1.5)	deps complètes	la suite des valeurs de <code>n</code>
alias (liste 1.6)	appel d'une variable locale	que <code>bump</code> est le setter
Clock (liste 1.7)	appel d'un hook	le corps du hook, dans son appelant
Child/Parent (liste 1.8)	appel d'une prop	que la prop est le setter du parent

TABLE 1.2 – Ce que la syntaxe montre et ce qu'il faut calculer pour conclure.

Tableau 1.2 résume la leçon : dès qu'on quitte les deux premières lignes, conclure demande de connaître des *valeurs* (ce que contient `n`, ce que désigne `bump` ou `onChange`) sur *toutes* les exécutions possibles.

Remarque

Le README.md du dépôt situe `reactant-analyzer` par rapport à `eslint-plugin-react-hooks` et au React Compiler : le linter reste l'outil juste pour les règles lexicales des hooks, dans l'éditeur ; le compilateur mémorise automatiquement et fait disparaître certains symptômes de références instables, sans corriger les défauts de logique. `reactant-analyzer` ne remplace ni l'un ni l'autre : il est plus lent (il calcule un point fixe sur tout le programme) et vise les défauts que la forme ne montre pas. Le README annonce par exemple 528 fichiers analysés en 1,8 s sur `excalidraw` et 4195 fichiers en 76 s sur `dub` (README.md, l. 121–122 ; chiffres du README, non re-mesurés pour ce manuscrit).

Piège

« Le linter ne dit rien » signifie au mieux « les deps sont complètes », jamais « l'effet se stabilise » : la liste 1.5 a un tableau de deps irréprochable et boucle. Inversement, compléter

les deps pour faire taire le linter peut *créer* la boucle. Partons de `useEffect(() => { setN(n + 1); }, [])`, qui s'exécute une fois : exhaustive-deps, comme missing-deps, réclame `n` (exercice 1.2); l'ajouter donne exactement la liste 1.5.

1.3 Le pari : interpréter abstraitement le composant

Si la forme ne suffit pas, que reste-t-il ? Exécuter le composant. Mais une exécution ne prouve rien sur les autres : elle choisit une valeur de prop, un ordre d'événements, un nombre de clics. Et on ne peut pas les essayer toutes : une prop `number` a des milliards de valeurs, et une boucle de rendus peut ne jamais s'arrêter. Le pari de `reactant-analyzer` est celui de l'*interprétation abstraite* [CC77] : exécuter le composant *une fois*, mais sur des *descriptions d'ensembles de valeurs* plutôt que sur des valeurs.

1.3.1 Exécuter sur des ensembles

Revenons à Counter (liste 1.5) et à sa variante bornée :

Extrait 1.9 – Bounded : la même boucle, gardée (`/tmp/ch01/bounded.tsx`)

```
1 import { useEffect, useState } from "react";
2
3 export function Bounded() {
4   const [n, setN] = useState(0);
5   useEffect(() => {
6     if (n < 3) setN(n + 1);
7   }, [n]);
8   return <p>{n}</p>;
9 }
```

Au lieu de dire « `n` vaut 0 », l'analyseur dit « `n` est un nombre entier compris entre 0 et 0 », ce qu'on note par l'intervalle $[0, 0]$. Il rejoue alors un *tour* de React : il interprète le corps du composant (le rendu), puis le corps de l'effet. Dans Counter, l'effet écrit `n + 1`, soit un nombre de $[1, 1]$. L'état `n` peut donc valoir, à l'entrée du tour suivant, soit l'ancienne valeur, soit la nouvelle : on réunit les deux en $[0, 1]$. Au tour suivant l'effet écrit $[1, 2]$ et l'état devient $[0, 2]$, puis $[0, 3]$... La suite ne se stabilise pas, et l'analyseur ne peut pas attendre. Il applique alors une opération d'accélération, le *widening* : la borne qui grandit est projetée d'un coup vers une borne sûre. Pour Counter, cette borne est $+\infty$; l'intervalle $[0, +\infty)$ est stable (l'effet y écrit $[1, +\infty)$, déjà inclus), l'analyse s'arrête. Pour Bounded, la borne est le seuil 3 lu dans la garde `n < 3` ; dans $[0, 3]$, l'effet n'écrit que lorsque `n < 3`, donc des valeurs de $[1, 3]$, et l'intervalle $[0, 3]$ est lui aussi stable.

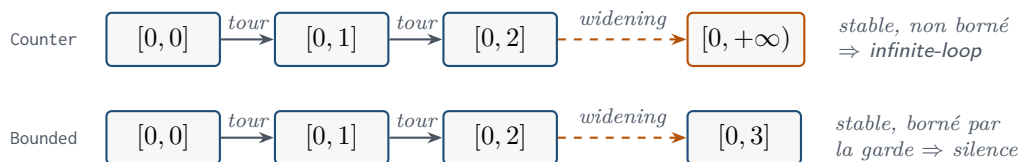


FIGURE 1.2 – Valeur abstraite de l'état `n` au fil des tours abstraits (déroulé simplifié ; le détail des itérations et le seuil du widening sont au chapitre 13). Les valeurs finales $[0, +\infty)$ et $[0, 3]$ sont celles que le moteur calcule effectivement au commit e67b10a.

Figure 1.2 résume ces deux déroulés. Ils ont un point commun essentiel : le résultat final *contient* toutes les valeurs que `n` peut réellement prendre. Dans Counter, `n` prend les valeurs

$0, 1, 2, \dots$, toutes dans $[0, +\infty)$; dans Bounded, les valeurs $0, 1, 2, 3$, toutes dans $[0, 3]$. C'est cette inclusion qui rend le raisonnement valable : l'analyse *sur-approxime*, elle peut se tromper en imaginant des valeurs impossibles, jamais en oubliant des valeurs possibles.

Définition 1.2: Valeur abstraite, point fixe (informel)

Une *valeur abstraite* décrit un ensemble de valeurs concrètes (un intervalle décrit les nombres qu'il contient). L'analyse d'un composant répète des tours abstraits (rendu, effets, handlers), en réunissant à chaque tour ce que l'état pouvait valoir et ce qui vient d'y être écrit, jusqu'à ce qu'un tour n'ajoute plus rien : c'est le *point fixe*. Le widening garantit qu'on l'atteint en un nombre fini de tours. Les définitions exactes (treillis, join, concrétisation, widening) sont au chapitre 3.

Le point fixe atteint, les règles n'ont plus qu'à *lire* le résultat. Pour Counter, l'état a dû être élargi et l'effet y écrit sans borne : c'est la signature d'une boucle de rendus, et `infinite-loop` la signale. Pour Bounded, l'état a aussi été élargi (le widening est une approximation, l'analyseur le dit sous `-info`), mais l'écriture reste bornée par la garde : pas de boucle, et la règle l'atteste.

```
Bounded (2 hooks) fixed.tsx
info widening-info (line 26:2) state `n` kept changing during analysis and was approximated to converge, so
  ↳ findings that depend on it may be imprecise
    (2 trace step(s), rerun with --trace)
verified always-unstable-deps no deps array is defeated by an always-fresh reference
verified conditional-hook all hooks run unconditionally, in a stable order
verified derived-state no effect merely mirrors other state
verified infinite-loop no effect diverges into an infinite render loop
...
```

(Extrait de `check -no-color -info -show-clean` sur `/tmp/ch01/fixed.tsx`, qui regroupe les versions corrigées des exemples du chapitre; les six autres lignes `verified` sont élaguées.)

1.3.2 Pas seulement des nombres

Les intervalles ne suffisent pas pour React. Les trois défauts de la section 1.1 demandent chacun une information différente sur les valeurs, et `reactant-analyzer` les abstrait toutes à la fois.

- **La valeur.** $n + 1$ diffère-t-il de n ? Le domaine des valeurs combine un intervalle pour les nombres, des ensembles bornés de chaînes, des booléens, `null`, `undefined`, dans un produit (chapitres 9 et 10).
- **L'identité.** React compare les deps et les états avec `Object.is`, qui teste l'identité des objets et non leur contenu. Un objet littéral `{ . . . }` est une référence *neuve* à chaque évaluation. Le domaine suit donc aussi la *stabilité* d'une valeur : stable, changeant seulement quand un état change, ou neuve à chaque rendu (chapitre 10).
- **Le moment.** Qui écrit quel état, pendant le rendu, dans un effet, ou plus tard dans un callback (`setInterval`, `.then`)? Avec quelle valeur capturée? Ces faits sont calculés une fois par le moteur sous forme de *relations* (chapitre 14).

Exemple 1.1: L'identité, pas la valeur

L'effet suivant ne « change » rien au contenu de `query` dès le deuxième passage (`trim` d'une chaîne déjà nettoyée), et pourtant il boucle :

Extrait 1.10 – Un objet recréé à chaque exécution de l'effet (`/tmp/ch01/churn.tsx`)

```
1 import { useEffect, useState } from "react";
2
```



```

3 export function Filters() {
4   const [query, setQuery] = useState({ term: "" });
5   useEffect(() => {
6     setQuery({ ...query, term: query.term.trim() });
7   }, [query]);
8   return <input value={query.term} />;
9 }

```

Le littéral `{ ...query, term: ... }` est un objet neuf : pour `Object.is`, le dep `query` a changé, l'effet repart. L'analyse le sait sans rien connaître du contenu des chaînes :

```

Filters (2 hooks) /tmp/ch01/churn.tsx
error infinite-loop [hook:0] (line 5:2) this effect recreates object state `query` it depends on. Every run
↳ stores a fresh reference (`Object.is` always fails) and re-triggers itself: infinite render loop
(1 trace step(s), rerun with --trace)

```

Notons le niveau : `error`, alors que la boucle de `Counter` n'était qu'un `warn`. La section 1.4 explique cette différence.

1.3.3 Quel React on abstrait

Pour que « sur-approximer » ait un sens, il faut savoir ce qu'on sur-approxime : une *sémantique concrète* de React, qui dit précisément ce qu'une exécution peut faire. React n'en publie pas ; sa documentation décrit le comportement en prose [Met26b]. `reactant-analyzer` a fait le choix d'adosser son modèle à une sémantique formelle publiée.

Décision : ADR-001

Problème. Sans sémantique concrète explicite, on ne peut ni établir la soundness de l'analyseur ni écrire ses fonctions de transfert autrement qu'à l'intuition. **Décision.** Adopter comme sémantique concrète React-tRace (Lee, Ahn et Yi, OOPSLA 2025), dont les étapes *init*, *effect* et *check* correspondent à une itération du point fixe. **Limite acceptée.** React-tRace ne couvre qu'un noyau (`useState`, `useEffect`) ; les extensions du projet (deps, mémos, contexte, props...) n'ont pas de garantie formelle équivalente. Le chapitre 2 présente cette sémantique et ce que le projet en garde (ADR-1).

Concrètement, le tour abstrait que rejoue le moteur est celui de la figure 1.1, légèrement grossi pour rester sûr : à chaque tour, il interprète le rendu, les mémos, *tous* les effets (qu'ils soient ou non re-déclenchés, ce qui couvre toutes les combinaisons de deps) et les handlers (qu'un utilisateur pourrait déclencher à tout moment), puis réunit les états écrits avec les états précédents. Le chapitre 13 décrit cette boucle, et en quoi chaque grossissement préserve la sur-approximation.

1.3.4 Le pipeline

Interpréter un composant demande d'abord de le lire. `reactant-analyzer` n'interprète pas directement le texte source : il le traduit en une représentation intermédiaire (IR) conçue pour l'analyse, un graphe de flot de contrôle par composant, où les hooks sont des objets de première classe. La figure 1.3 montre les étapes d'une invocation, avec le module Rust qui porte chacune et le type qui passe à l'étape suivante.

On y lit trois décisions qui structurent tout le projet.

1. **Après le lowering, plus aucun AST.** Le front-end utilise le parseur `oxc` [The26], détecte les composants, hooks et utilitaires, et produit un `ComponentIR` par composant ; le moteur ne voit jamais la syntaxe d'origine (ADR-3, chapitres 5 à 7).

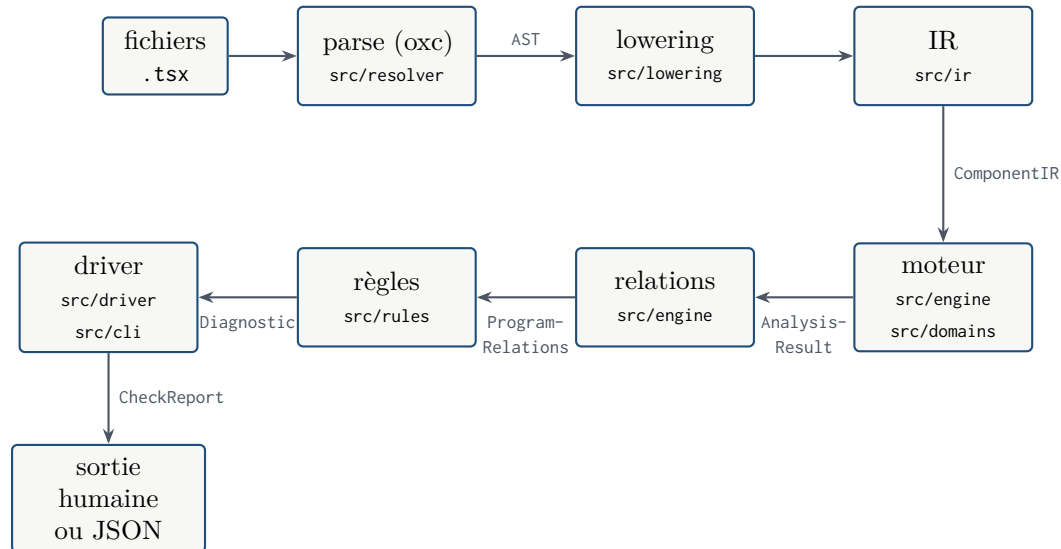


FIGURE 1.3 – Le pipeline d’une invocation `reactant check`. Chaque boîte est une couche du code ; sur les flèches, le type principal qui passe d’une couche à la suivante. Le chapitre 4 détaille chaque étape et ses frontières.

2. **Le moteur calcule, les règles lisent.** Le moteur exécute le point fixe de chaque composant et dérive des relations (qui écrit quel état, quand, avec quelle valeur ; quels effets se re-déclenchent) ; les règles ne parcourent pas l’IR, elles interrogent ces résultats (ADR-42, chapitres 14 et 17).
3. **L’analyse est de programme.** Un parent est analysé avec ses enfants, les hooks personnalisés et utilitaires sont inlinés dans leurs appelants, à travers les fichiers (chapitres 8 et 16). C’est ce qui a permis de voir les trois indirections de la section 1.2.

1.4 La promesse : trois niveaux de diagnostic et la soundness

Rassemblons les niveaux observés jusqu’ici :

Programme	Diagnostic	Niveau
Panel (liste 1.4)	conditional-hook	ERROR
Filters (liste 1.10)	infinite-loop	ERROR
Timer (liste 1.3)	stale-closure	ERROR
Timer (liste 1.3)	missing-deps	WARNING
Profile (liste 1.2)	missing-deps	WARNING
Counter (listes 1.1 et 1.5)	infinite-loop	WARNING
Bounded (liste 1.9)	widening-info	INFO

(La sortie de `Timer` est commentée à la section 1.5.) Le niveau n’est pas une impression de gravité : c’est une promesse sur la *nature* de ce que l’outil sait.

1.4.1 Trois niveaux, trois promesses

Définition 1.3: Niveaux de diagnostic

- **ERROR** : le défaut est *certain* dès que le code signalé s'exécute. L'outil détient une preuve de *toute* la conclusion, pas seulement d'une de ses conditions.
- **WARNING** : le défaut est *possible* (il dépend d'un chemin, d'une entrée, ou l'analyse a sur-approximé), ou bien un fait est certain mais son coût ne l'est pas (une référence neuve à chaque rendu, un rendu gaspillé).
- **INFO** : une limite de l'analyse (un widening, un hook qu'elle n'a pas pu lire) ou un motif qui a l'air voulu. Masqué par défaut, affiché avec `-info`.

Cette définition n'est pas seulement une convention de documentation : elle est écrite dans le type qui porte le niveau, et c'est la même que celle des invariants du projet dans `CLAUDE.md`.

Extrait 1.11 – `src/rules/api/diagnostic.rs`, l. 23–38 — les trois niveaux et leur contrat

```

23
24 /// Confidence level of a diagnostic.
25 ///
26 /// - `Error` the defect is certain whenever the flagged code runs. Built only
27 /// from a proof of the whole claim, not of one of its conjuncts (#142).
28 /// - `Warning` a possible defect (conditional path, over-approximation), or a
29 /// certain fact whose cost is not (a fresh reference, one wasted render).
30 /// - `Info` not actionable without context: a known analysis limitation
31 /// (widening, depth cap), or a pattern that looks intentional (a seed-once
32 /// prop name, a cheap pure initializer). Hidden by default; show with --info.
33 #[derive(Debug, Clone, Copy, PartialEq, Eq, Default)]
34 pub enum Severity {
35     Error,
36     #[default]
37     Warning,
38     Info,

```

Relisons nos exemples avec cette grille.

- **Profile** : le défaut n'existe que si `userId` change pendant la vie du composant ; rien, dans le programme analysé, ne dit qu'il change ni qu'il ne change pas. Défaut possible : **WARNING**.
- **Panel** : le `useState` n'est pas sur tous les chemins du rendu (le retour anticipé l'évite). C'est un fait de structure, prouvable en regardant le graphe de flot : aucun chemin de l'entrée à la sortie ne peut être oublié. **ERROR**.
- **Filters** : trois faits certains se composent. L'effet écrit `query` sur tous ses chemins ; il y écrit toujours un objet neuf ; `query` est exactement l'un de ses deps. La conclusion « il se re-déclenche lui-même à chaque exécution » est entièrement prouvée. **ERROR**.
- **Counter** : la boucle est pourtant tout aussi certaine. Mais ce que l'analyse sait, c'est que l'intervalle de `n` a dû être élargi et que l'effet y écrit sans borne. Un widening peut aussi venir d'une simple imprécision sur une boucle qui converge ; il ne constitue pas une preuve. L'outil ne dispose pas, au commit `e67b10a`, d'une primitive qui prouverait « la valeur écrite diffère toujours de la valeur courante » : il s'en tient à **WARNING** (#144).

Le cas **Counter** enseigne l'asymétrie essentielle : un niveau trop *bas* est permis, un niveau trop *haut* ne l'est pas. Un **WARNING** sur un défaut certain est une perte de précision ; une **ERROR** sur un défaut seulement possible est une promesse rompue.

1.4.2 Une Error se construit à partir d'une preuve

Comment le code empêche-t-il une règle de promettre trop ? Par le typage. Le constructeur d'une `ERROR` exige un argument de type `Certified<E>`, un « jeton de preuve » que seules les primitives de preuve du moteur savent fabriquer (chapitre 17) ; un `WARNING` ou une `INFO` se construisent librement.

Extrait 1.12 – `src/rules/api/diagnostic.rs`, l. 157–191 — le constructeur d'une `ERROR`, contre les constructeurs libres d'un `WARNING` et d'une `INFO`

```

157    /// The only constructor of an `Error` (ADR-021 §2). Builds the finding
158    /// *from* a proof: the certified evidence's span/label/witness ride along, so
159    /// they need not be re-threaded by hand. A `May<_>`/`MustResult::Some` value
160    /// has no `Certified` to pass here – Error-on-may is a type error.
161    ///
162    /// Further `.with_*` builders may still refine/override the absorbed fields.
163    pub fn error<E>(
164        rule: impl Into<Cow<'static, str>>,
165        proof: Certified<E>,
166        message: impl Into<String>,
167    ) -> Self {
168        let prov = proof.provenance();
169        Diagnostic {
170            severity: Severity::Error,
171            rule: rule.into(),
172            message: message.into(),
173            hook_label: prov.hook_label,
174            var: None,
175            range: prov.range,
176            notes: prov.notes.clone(),
177        }
178    }
179
180    /// A Warning: a safe over-claim (conditional path / over-approximation). Free
181    /// to construct – a Warning asserts no MUST fact.
182    pub fn warn(rule: impl Into<Cow<'static, str>>, message: impl Into<String>) -> Self {
183        Diagnostic::new(rule, message).with_severity(Severity::Warning)
184    }
185
186    /// An Info: a known analysis limitation, or a pattern that looks
187    /// intentional. Makes no must/may claim; hidden unless `--info`.
188    pub fn info(rule: impl Into<Cow<'static, str>>, message: impl Into<String>) -> Self {
189        Diagnostic::new(rule, message).with_severity(Severity::Info)
190    }
191 }
```

Une règle qui n'a qu'un fait `may` (vrai sur certains chemins seulement) n'a pas de `Certified` à passer : écrire une `ERROR` à partir de lui est une erreur de compilation, pas une faute de relecture.

Décision : ADR-021

Problème. Un faux négatif réel s'était glissé dans un utilitaire partagé des règles, et n'importe quel booléen pouvait devenir une `Severity::Error`. **Décision.** Faire de la *polarité* d'un fait (vrai sur tous les chemins, sur certains, ou inconnu) un type : `MustResult`, `May`, et un jeton `Certified` que seules les primitives `must_*` produisent ; `Diagnostic::error` n'accepte que ce jeton. Les règles existantes ont été migrées d'un bloc plutôt que progressivement, pour ne pas laisser de fenêtre ouverte aux faux négatifs. **Précision ultérieure (#142).** La preuve doit couvrir *toute* la conclusion : une `Error` de `stale-closure` avait été produite à

partir d'une preuve ne couvrant qu'une de ses trois conditions, ce qui a conduit à resserrer la règle et la doctrine des niveaux (ADR-21, chapitres 17 et 26).

1.4.3 La soundness : pas de faux négatif

Reste la promesse la plus forte, qui porte sur le *silence*. Parlons d'abord vocabulaire.

Définition 1.4: Faux positif, faux négatif, soundness (informel)

Un *faux positif* (FP) est un diagnostic sur un défaut qui n'existe pas ; un *faux négatif* (FN) est un défaut réel que l'outil ne signale pas. Une analyse est *sound* (on parle de sa *soundness*) si elle calcule un sur-ensemble des comportements possibles du programme : tout ce qui peut arriver est dans ce qu'elle a calculé. Conséquence : ses FN sont exclus, ses FP tolérés. La définition exacte est au chapitre 3.

La soundness est l'invariant numéro un du projet (CLAUDE.md) : « faux positifs tolérés, faux négatifs INTERDITS ». Elle se lit à plusieurs endroits de la sortie.

- **Le silence est une affirmation.** Quand une règle applicable n'a rien trouvé, -info l'écrit : `verified infinite-loop no effect diverges into an infinite render loop`. Cette ligne n'est publiée que si l'analyse du composant n'a pas été tronquée.
- **Ce qui n'a pas été lu est dit.** Si l'on analyse `/tmp/ch01/xfile/Clock.tsx` seul, sans le fichier du hook qu'il importe, l'outil ne conclut pas « aucun problème » :

```
1 clean component(s) hidden, rerun with --show-clean

! 1 file(s), no findings, but parts of this run were not analyzed, so this is not a clean bill.
not analyzed:
  • 1 imported file(s) resolved outside the analysed set and were never read. Pass them on the command line to
    ↳ analyse them (useTicker.ts)
```

Le code de sortie reste `0` (aucun diagnostic), mais la phrase finale refuse le « ✓ no issues found » : on ne délivre pas de certificat de propreté pour du code qu'on n'a pas lu (chapitre 23).

- **L'inconnu est $\top$, pas « rien ».** Un hook de bibliothèque que l'outil ne sait pas modéliser renvoie une valeur *inconnue* (le top du treillis, $\top$), qui peut être n'importe quoi. Les règles qui la lisent doivent alors supposer le pire, et une INFO `analysis-limit` indique la limite (chapitre 20).

Piège

La soundness est un *invariant visé*, pas un théorème démontré sur le code. La page `docs/limitations.md` du dépôt recense des limites connues, et le tracker du projet porte des issues étiquetées `soundness-bug` ouvertes au commit `e67b10a` (par exemple `#157`, `#143`, `#7`). La rédaction de ce manuscrit a aussi mis au jour des faux négatifs. Le plus simple tient en six lignes :

Extrait 1.13 – Une boucle certaine que la CLI ne signale pas (`/tmp/ch01/updater_dep.tsx`)

```
1 import { useEffect, useState } from "react";
2
3 export function D() {
4   const [count, setCount] = useState(0);
5   useEffect(() => {
6     setCount((c) => c + 1);
7   }, [count]);
8   return <p>{count}</p>;
```

```
9 }
```

L'updater fait croître `count` à chaque exécution, `count` est un `dep`, l'effet boucle. Pourtant :

```
1 clean component(s) hidden, rerun with --show-clean
✓ 1 file(s) no issues found.
```

La cause (le chemin « programme » du moteur écrit la fonction `(c) => c + 1` elle-même dans l'état, au lieu d'appliquer l'updater) est analysée au chapitre 27. Le lecteur doit donc retenir deux choses à la fois : la conception entière vise l'absence de FN, et les écarts à cette visée sont des défauts à corriger à la racine, pas des comportements acceptés.

1.5 Première prise en main

Il est temps de lancer l'outil soi-même. Toutes les sorties de ce manuscrit ont été produites au commit `e67b10a`, depuis la racine du dépôt, par le binaire de développement.

1.5.1 Construire et lancer

```
cargo build --release          # binaire : target/release/reactant
cargo run -q -- check /tmp/ch01/timer.tsx
cargo run -q -- check src/     # un dossier : parcours récursif des .ts/.tsx/.js/.jsx
```

`check` est la sous-commande par défaut (`reactant src/` équivaut à `reactant check src/`). Sans chemin, c'est le dossier courant qui est analysé. Les autres sous-commandes sont `rules` (le catalogue), `explain <rule>` (la documentation d'un diagnostic, avec un exemple et une correction), `schemas` (les schémas JSON de la configuration et des packs). Côté JavaScript, le même analyseur est distribué sur npm, compilé en WebAssembly : `npx reactant-analyzer` (chapitre 24).

1.5.2 Lire la sortie humaine

Lançons l'analyse sur `Timer` (liste 1.3), en demandant les chaînes causales avec `-trace`. On se place dans `/tmp/ch01` pour que les chemins affichés soient courts.

```
Timer (2 hooks) timer.tsx
warn  missing-deps [hook:1] var:seconds (line 5:2) `seconds` is used in this effect but not in its deps array,
  ↳ and it is recreated on every render
  ↳ `seconds` is read here [hook:1] (line 5:2)
error  stale-closure [hook:1] var:seconds (line 6:10) the `setInterval` callback registered by this mount-only
  ↳ effect reads `seconds` and writes it back. `seconds` was captured once at mount, so every firing recomputes
  ↳ from the same frozen value and the state can never advance past its first update
  ↳ `setInterval` has side effects (subscriptions/requests/timers re-fire on every call) [hook:1] (line 6:10)
  ↳ `seconds` is captured at registration time, so the callback keeps this value, not the latest one [hook:0] (
  ↳ line 6:10)
  ↳ state `seconds` is written here [hook:0] (line 7:6)

! 1 error(s), 1 warning(s) across 1 file(s).
```

Décomposons.

- **L'en-tête** `Timer (2 hooks) timer.tsx` nomme le composant, le nombre de hooks que l'IR lui connaît et son fichier. Les hooks sont numérotés dans l'ordre d'appel, à partir de 0 : `[hook:0]` est le `useState`, `[hook:1]` le `useEffect`. Ce numéro est le *label* du hook, celui qui identifie un slot d'état pour React comme pour l'analyseur (chapitre 6).

- **Chaque diagnostic** donne son niveau (error, warn, info), le nom de la règle, le hook concerné, la variable en cause (var:seconds), la position (line L:C) — ligne comptée à partir de 1, colonne à partir de 0 — et un message. Ici, (line 6:10) pointe sur l'appel `setInterval(...)`.
- **Les lignes « → »** sont la *chaîne de témoins* du diagnostic : les faits, typés et localisés, qui ont conduit à la conclusion (ADR-19, chapitre 17). Pour `stale-closure`, ce sont exactement les trois ingrédients du défaut raconté à la section 1.1 : `setInterval` rappellera le callback ; le callback a capturé `seconds` au moment de l'enregistrement ; il réécrit l'état `seconds`. Sans `-trace`, ces lignes sont remplacées par un rappel (3 trace step(s), rerun with `-trace`).
- **La dernière ligne** fait le bilan. Le code de sortie est 1 : par défaut (`-fail-on warning`), une `ERROR` ou un `WARNING` fait échouer la commande, ce qui la rend utilisable telle quelle en intégration continue.

Les deux diagnostics racontent le même défaut à deux niveaux de preuve. `missing-deps` constate une propriété locale (`seconds` est lu et n'est pas déclaré), en parité avec `exhaustive-deps` : défaut possible, `WARNING`. `stale-closure` prouve la *conséquence* : avec des `deps []` et un callback répétitif qui réécrit ce qu'il lit, l'état ne peut jamais dépasser sa première mise à jour. C'est ce que l'on a constaté à la main (l'affichage bloqué à « 1 s »), et la preuve couvre toute la conclusion : `ERROR` (chapitre 20).

Piège

La formulation du motif ajouté par `missing-deps` (« and it is recreated on every render », « and its value may change between renders ») n'est pas un jugement de l'auteur de la règle : elle traduit en anglais courant la valeur abstraite que le moteur a calculée pour la variable (`describe_value`, `src/rules/helpers/mod.rs`, l. 42). Elle peut donc surprendre, comme ici pour un nombre : `seconds` n'a pas la forme d'une référence, mais son intervalle ne s'est jamais réduit à un point pendant l'analyse (section 1.3), ce qui suffit à le classer « recreated on every render » au même titre qu'un objet neuf. Les chapitres 10 et 20 expliquent cette traduction.

La correction est celle de la section 1.1 : un updater, `setSeconds((s) => s + 1)`. Le fichier `/tmp/ch01/fixed.tsx` rassemble les versions corrigées de `Timer`, de `Profile` (avec `[userId]`) et de `Bounded`. Sans option, la sortie est :

```
3 clean component(s) hidden, rerun with --show-clean

✓ 1 file(s) no issues found.
```

Avec `-info -show-clean`, chaque composant affiche aussi les vérifications applicables qui sont passées ; pour `Timer`, on lit entre autres :

```
Timer (2 hooks) fixed.tsx ✓
  verified conditional-hook all hooks run unconditionally, in a stable order
  ...
  verified stale-closure no long-lived callback captures a stale state value
  ...
```

Le signe ✓ après le fichier marque un composant sans diagnostic.

Tableau 1.3 rassemble les options les plus utiles. `-verbose` est une fenêtre sur le moteur : sur la boucle de la liste 1.1, elle affiche, en plus du diagnostic,

```
[verbose] symbol graph: 1 nodes, topo order = [Counter@loop.tsx]
[verbose] 1 components analyzed
[verbose] cache hits: 0, misses: 0
[verbose] Counter: 3 iteration(s), widened: [0]
```

Option	Effet
-trace	affiche la chaîne de témoins de chaque diagnostic
-info	affiche les INFO et les lignes <code>verified</code> / <code>suspended</code>
-show-clean	affiche aussi les composants sans diagnostic
-format json	un unique document JSON sur la sortie standard
-fail-on error warning never	seuil du code de sortie 1 (défaut <code>warning</code>)
-rule, -ignore-rule	ne garder / retirer qu'un nom de diagnostic
-follow-imports	lire aussi les fichiers importés, transitivement
-verbose	statistiques du moteur sur la sortie d'erreur
-no-color	sortie sans couleurs ANSI

TABLE 1.3 – Les options de check utilisées dans ce manuscrit (extrait de `docs/usage.md` ; la liste complète et la configuration sont au chapitre 23).

c'est-à-dire trois tours abstraits, et le slot de label 0 (n) élargi, comme sur la figure 1.2.

1.5.3 Lire la sortie JSON

Pour une machine (intégration continue, éditeur, agent), `-format json` écrit un document unique au schéma versionné. Voici celui de `Timer`, où le premier diagnostic (`missing-deps`) a été élagué :

Extrait 1.14 – Sortie check `-format json timer.tsx` (élaguée : le diagnostic `missing-deps` est remplacé par ...)

```

1 {
2   "version": 2,
3   "files_analyzed": 1,
4   "parse_errors": [],
5   "diagnostics": [
6     ...,
7     {
8       "rule": "stale-closure",
9       "severity": "error",
10      "component": "Timer",
11      "file": "timer.tsx",
12      "component_file": "timer.tsx",
13      "line": 6,
14      "col": 10,
15      "hook_label": 1,
16      "var": "seconds",
17      "message": "the `setInterval` callback registered by this mount-only effect reads `seconds` and
↳ writes it back. `seconds` was captured once at mount, so every firing recomputes from the same
↳ frozen value and the state can never advance past its first update",
18      "notes": [
19        {
20          "message": "`setInterval` has side effects (subscriptions/requests/timers re-fire on every call)"
↳ ,
21          "kind": "call",
22          "hook_label": 1,
23          "file": "timer.tsx",
24          "line": 6,
25          "col": 10,
26          "callee": "setInterval",
27          "effect_class": "effectful"
28        },
29      ]

```



```

30     "message": "`seconds` is captured at registration time, so the callback keeps this value, not the
    ↪ latest one",
31     "kind": "capture",
32     "hook_label": 0,
33     "file": "timer.tsx",
34     "line": 6,
35     "col": 10,
36     "what": "seconds"
37   },
38   {
39     "message": "state `seconds` is written here",
40     "kind": "write",
41     "hook_label": 0,
42     "file": "timer.tsx",
43     "line": 7,
44     "col": 6,
45     "slot": 0,
46     "value_class": "unknown"
47   }
48 ]
49 }
50 ],
51 "blind_spots": [],
52 "summary": {
53   "errors": 1,
54   "warnings": 1,
55   "infos": 0,
56   "components_analyzed": 1,
57   "exit_code": 1
58 }
59 }

```

On retrouve champ pour champ la sortie humaine, plus quelques informations structurées.

- `version` est la version du schéma (2) ; `files_analyzed` et `parse_errors` disent ce qui a été lu, et si le parseur a dû récupérer d'erreurs de syntaxe.
- Dans chaque diagnostic, `file` est le fichier où pointe la position, `component_file` celui où le composant est défini ; les deux diffèrent quand le finding est ancré dans un hook inline depuis un autre fichier (section 1.2, exemple Clock). `line` commence à 1, `col` à 0.
- `notes` est la chaîne de témoins ; chaque étape porte un `kind` pris dans un vocabulaire fermé (`call`, `capture`, `write`, `read`, `widen`...) et les champs propres à ce genre (`callee`, `what`, `slot`...).
- `blind_spots` liste ce que l'exécution sait ne pas avoir lu. Non vide, il signifie que les compteurs du `summary` sont une borne inférieure, pas un verdict ; c'est la version machine de la phrase « this is not a clean bill » vue plus haut.
- `summary.exit_code` reproduit le code de sortie du processus.

Code	Signification
0	aucun diagnostic au niveau du seuil <code>-fail-on</code> ou au-dessus
1	au moins un diagnostic au niveau du seuil ou au-dessus
2	erreur d'usage ou d'entrée : aucun fichier, chemin inexistant, règle inconnue

TABLE 1.4 – Codes de sortie de `reactant check` (docs/usage.md). Les INFO et les `blind_spots` ne changent jamais le code de sortie.

Tableau 1.4 donne les codes de sortie. Enfin, `reactant explain <rule>` affiche la documentation d'un diagnostic. Pour `stale-closure`, son premier paragraphe reprend ce que nous avons constaté,

et la correction proposée est celle que nous avons appliquée :

```

stale-closure
  long-lived callback keeps a state value frozen at registration time

A callback handed to `setInterval`, `addEventListener`, `subscribe`, `setTimeout` or a promise `.then` inside an effect
  ↪ closes over the variable values from the render that last ran the effect. [...]

Example:
const [n, setN] = useState(0);
useEffect(() => { setInterval(() => setN(n + 1), 1000); }, []);

Fix:
Use the functional updater (`setN(n => n + 1)`) so the callback never reads the captured value; mirror the latest value
  ↪ into a `useRef` and read `ref.current`; or add the value to the deps array and return a cleanup so the
  ↪ callback is re-registered with a fresh capture.

```

(Sortie élaguée : la suite de l'explication, marquée [...], détaille la condition de l'ERROR.)

1.6 Le projet en chiffres

Avant d'entrer dans le code, prenons la mesure de ce qu'il y a à lire. Tous les chiffres de cette section sont comptés sur le dépôt au commit e67b10a (find, wc -l, git rev-list).

1.6.1 Le code

La bibliothèque reactant (src/) compte 142 fichiers Rust et 59 757 lignes, commentaires et tests unitaires compris. Tableau 1.5 les répartit par module. Deux modules pèsent à eux seuls plus de la moitié du total : src/rules (les règles, leur API typée, le front-end des règles déclaratives) et src/engine (le point fixe, les relations, l'analyse de programme).

Module	Fichiers	Lignes	Rôle (chapitres)
src/lowering	13	6 692	détection et lowering AST → IR (5, 7)
src/ir	16	4 423	représentation intermédiaire, splice (6, 8)
src/domains	21	7 393	domaines abstraits, transfert, stores (9–11)
src/engine	22	13 913	point fixe, relations, churn, programme (12–16)
src/rules	41	20 432	API des règles, règles natives, packs (17–22)
src/resolver	4	1 930	découverte, parse, résolution d'imports (23)
src/project	4	1 432	projets Vite, Next.js, tsconfig (23)
src/driver	7	1 666	run_check, rendus humain et JSON (23)
src/registry	3	691	résumés de hooks de bibliothèques (8)
src/cli	7	570	sous-commandes, chargement de la configuration (23)
racine de src/	4	615	config.rs, lib.rs, main.rs, test_support.rs
total src/	142	59 757	

TABLE 1.5 – Répartition du code de src/ par module au commit e67b10a (fichiers .rs, lignes brutes).

Autour de la bibliothèque :

- tests/ : 69 fichiers Rust de tests d'intégration (26 212 lignes) et 131 fixtures (programmes React d'exemple, projets miniatures). Au total, le dépôt déclare 1 529 fonctions #[test] (632 dans src/, 897 dans tests/).
- crates/reactant-wasm : le point d'entrée WebAssembly (323 lignes), et npm/ : le paquet reactant-analyzer (chapitre 24).

- docs/ : l’usage (usage.md), la page des limites (limitations.md), les règles personnalisées (custom-rules.md), le journal de précision, et surtout docs/adr/ : **42 ADR** (*Architecture Decision Records*), numérotées de ADR-001 (« concrete semantics ») à ADR-042 (« relations are engine products »). Chacune fait l’objet d’une fiche en annexe (chapitre A).

L’historique compte 322 commits, du 8 mai au 27 septembre 2026, et six versions publiées (v0.1.0 à v0.6.0, la version de Cargo.toml). Le chapitre 26 raconte cette histoire ; retenons-en qu’elle commence par une réécriture complète, décidée par la conception (ADR-3 : une IR dédiée plutôt que des fonctions de transfert sur l’AST), et que le pipeline actuel a été posé en une journée, le 1^{er} juin, dans l’ordre même où ce manuscrit le présente.

1.6.2 Les règles

reactant-analyzer enregistre 19 règles natives (la fonction `all_rules`, `src/rules/mod.rs`, l. 109), qui émettent 21 *noms de diagnostic* : `infinite-loop` émet aussi `cross-component-infinite-loop`, et `setter-in-render` émet aussi `cross-setter-in-render`. Tableau 1.6 les liste tous, avec le résumé que donne reactant rules et le niveau le plus haut que chaque nom peut atteindre au commit e67b10a (lu dans les constructeurs de diagnostic de chaque règle, `src/rules/impls/`). Une configuration peut abaisser ces niveaux, jamais les relever. L’annexe C donne une fiche par règle.

Diagnostic	Max	Résumé (reactant rules, verbatim)
<code>always-unstable-deps</code>	WARNING	a dep is a fresh reference every render, so the deps array never matches
<code>analysis-limit</code>	INFO	the analyzer deliberately truncated analysis here (potential false negatives)
<code>conditional-hook</code>	ERROR	hook called inside a conditional branch
<code>cross-component-infinite-loop</code>	WARNING	child effect sets parent state, parent re-renders child, effect refires
<code>cross-setter-in-render</code>	ERROR	parent’s setter (received as prop) called during render
<code>derived-state</code>	WARNING	effect only mirrors another state, so compute it during render
<code>frozen-initial-state</code>	ERROR	useState seeded from a prop that changes, so the state freezes at the first value
<code>infinite-loop</code>	ERROR	effect sets state that re-triggers the effect, and the state diverges
<code>lazy-init</code>	ERROR	useState initializer calls a function on every render
<code>missing-cleanup</code>	WARNING	effect starts something long-lived and returns no teardown
<code>missing-deps</code>	WARNING	effect body captures a variable not listed in its deps array
<code>redundant-set-state</code>	WARNING	setState called with the value the state already holds
<code>server-component-hook</code>	WARNING	a hook is called in a Next.js Server Component
<code>setter-in-render</code>	ERROR	setState called during the render body
<code>stale-closure</code>	ERROR	long-lived callback keeps a state value frozen at registration time
<code>state-lifted-too-high</code>	WARNING	state is used only deep in one child subtree ; the components above re-render to pass it down
<code>state-mutation</code>	ERROR	state or prop object mutated in place, same reference, no re-render
<code>unnecessary-rerender</code>	WARNING	mount-only effect immediately overwrites the initial state
<code>unstable-context-value</code>	WARNING	context provider hands consumers a new object every render
<code>wasted-subtree-render</code>	WARNING	a state written on every keystroke or pointer move re-renders subtrees that do not depend on it
<code>widening-info</code>	INFO	state slot required widening to converge (precision lost here)

TABLE 1.6 – Le catalogue des diagnostics natifs au commit e67b10a. La colonne « Max » est le niveau le plus haut atteignable ; pour `cross-component-infinite-loop`, le plafond WARNING est expliqué dans le code (`src/rules/impls/infinite_loop.rs`, l. 281–283).

Remarque

Un niveau maximal `ERROR` ne veut pas dire que la règle émet souvent des `ERROR` : c'est seulement le cas où une primitive de preuve a fourni son jeton. `infinite-loop`, par exemple, émet un `WARNING` sur `Counter` et une `ERROR` sur `Filters`.

À ces règles natives s'ajoutent des *règles déclaratives* : des règles écrites en JSON contre les relations du moteur, sans une ligne de Rust, regroupées en *packs* (`packs/guardrails.json` et cinq packs sous `packs/community/`). Le test `tests/catalogue.rs` mesure l'expressivité de ce langage sur un catalogue figé de 22 classes de règles : 21 sont exprimables au commit `e67b10a` (`EXPRESSIBLE_NOW` = 21, `tests/catalogue.rs`, l. 1094 ; chapitre 22).

1.6.3 Le corpus

Les tests unitaires vérifient des mécanismes ; ils ne disent pas ce que l'outil produit sur du vrai code. Pour cela, le projet épingle un *corpus* de 14 dépôts open source (`ai-chatbot`, `bulletproof-react`, `chakra-ui`, `commerce`, `dub`, `excalidraw`, `mantine`, `memos`, `next-shadcn-dashboard-starter`, `novel`, `precedent`, `shadcn-admin`, `twenty`, `zustand`), clonés à des commits fixés par `scripts/setup-test-repo.sh`. Le nombre de diagnostics distincts produits sur ce corpus est commité dans `docs/corpus-baseline.json` : **1 498** localisations au commit `e67b10a`, dont 476 `missing-deps`, 381 `always-unstable-deps`, 222 `lazy-init` et 35 `infinite-loop`. Un job d'intégration continue compare chaque exécution à ce chiffre ; toute variation doit être expliquée, et consignée dans `docs/precision-log.md`. Le chapitre 25 décrit cette méthode, et pourquoi un chiffre qui *baisse* n'est pas forcément une bonne nouvelle : pour un outil *sound*, un diagnostic qui disparaît peut être un faux positif corrigé, ou un faux négatif introduit.

1.7 Plan du manuscrit et conseils de lecture

Le manuscrit suit le trajet d'un fichier source dans le pipeline de la figure 1.3, précédé du contexte nécessaire et suivi de l'outil et de sa méthode. Il est organisé en six parties et quatre annexes (tableau 1.7) ; la difficulté croît d'une partie à l'autre, et à l'intérieur de chaque chapitre.

Partie	Contenu	Difficulté
I. Le problème et son contexte	ce chapitre ; le modèle d'exécution de React (2) ; rappels d'interprétation abstraite (3) ; tour d'architecture (4)	1–2
II. Du source à l'IR	oxc et détection (5), IR (6), lowering (7), splice et inlining (8)	2–3
III. Les domaines abstraits	treillis simples (9), produit <code>StateValue</code> et stabilité (10), transfert et stores (11)	2–4
IV. Le moteur	CFG et dominance (12), point fixe d'un composant (13), relations (14), churn (15), inter-composants (16)	3–5
V. Les règles	API typée (17), règles d'état (18), <code>infinite-loop</code> (19), effets (20), rendu et RSC (21), règles déclaratives (22)	3–5
VI. L'outil et sa méthode	CLI et driver (23), distribution (24), tests et corpus (25), histoire et doctrine (26), limites (27)	2–3
Annexes	fiches ADR (A), glossaire (B), catalogue des règles (C)	—

TABLE 1.7 – Les parties du manuscrit ; les numéros entre parenthèses renvoient aux chapitres.

1.7.1 Parcours de lecture

Le manuscrit se lit dans l'ordre, mais tous les lecteurs n'ont pas les mêmes besoins. Quelques parcours :

Vous utilisez l'outil et voulez comprendre ses diagnostics. Ce chapitre, puis le chapitre 2 ; ensuite, pour chaque diagnostic rencontré, sa fiche dans l'annexe C et sa section dans les chapitres 18 à 21 ; enfin le chapitre 23 (configuration, projets) et le chapitre 27 (ce que l'outil rate).

Vous voulez écrire une règle. La partie I, puis le chapitre 17 (l'API typée et le jeton de preuve), le chapitre 14 (les faits disponibles), une ou deux règles de la partie V comme modèles, et le chapitre 22 si une règle déclarative suffit. Le chapitre 25 dit comment mesurer l'effet d'une nouvelle règle.

Vous voulez modifier le moteur. Tout, dans l'ordre, avec une attention particulière aux chapitres 13 à 15 et aux encadrés « piège » qui signalent les trous de soundness connus.

Vous connaissez l'analyse statique, pas React. Le chapitre 2 est indispensable ; le chapitre 3 peut être survolé pour la notation. Les chapitres propres au projet sont ensuite les chapitres 10, 13, 15 et 16.

Vous connaissez React, pas l'analyse statique. Le chapitre 3 est indispensable : il pose le vocabulaire (treillis, join, widening, must/may, soundness) que tout le reste emploie.

Vous vous intéressez aux choix de conception. Le chapitre 26 et les fiches ADR (chapitre A), puis les encadrés « décision » des chapitres.

1.7.2 Conventions

- **Code cité.** Les extraits pris dans le dépôt portent en légende leur fichier et leurs vrais numéros de ligne au commit e67b10a (par exemple liste 1.11). Les programmes React d'exemple sont écrits pour le manuscrit ; leur chemin sous `/tmp` est indiqué pour pouvoir les réanalyser. Un extrait élagué le dit dans sa légende.
- **Sorties.** Toute sortie de l'analyseur a été observée réellement sur ce commit ; les éliminations sont signalées. Elles sont reproduites en anglais, telles que l'outil les écrit.
- **Encadrés.** « React » rappelle un comportement de React nécessaire à la suite ; « Décision » résume une ADR et les alternatives écartées ; « Piège » signale une erreur naturelle ou une subtilité, y compris les écarts observés entre la visée et le code.
- **Vocabulaire.** Les noms du code restent en anglais (`StateValue`, `SlotWriter`) ; les concepts ont un nom français fixé, recensé dans le glossaire (chapitre B). Quelques mots anglais sont gardés tels quels parce qu'ils sont d'usage : *widening*, *join*, *meet*, *soundness*, *slot*, *handler*, *deps*.

Résumé du chapitre

- Les défauts React les plus coûteux — boucle de rendus, dépendance oubliée, fermeture figée — ne se voient pas au premier rendu : ils sont des propriétés de la *suite* des rendus, commits et effets (tableau 1.1).
- Un linter syntaxique juge la forme d'une fonction, dans un fichier. Il voit un hook conditionnel ou un dep absent ; il ne voit ni la suite des valeurs d'un état, ni un alias de setter, ni un hook personnalisé dans son appelant, ni un setter passé en prop (tableau 1.2).
- `reactant-analyzer` interprète abstraitement chaque composant : il rejoue les tours de React sur des *ensembles* de valeurs (intervalles, stabilité des références, écrivains de chaque état) jusqu'à un point fixe, que le widening garantit fini ; les règles lisent ce résultat.

- Pipeline : parse (oxc) → lowering → IR (ComponentIR) → moteur (AnalysisResult) → relations → règles (Diagnostic) → driver (sortie humaine ou JSON) (figure 1.3).
- Trois niveaux : ERROR = défaut certain, prouvé en entier (seul `Diagnostic::error`, qui exige un `Certified`, en produit); WARNING = défaut possible ou coût incertain; INFO = limite de l'analyse. Sous-déclarer est permis (`Counter`, #144), sur-déclarer ne l'est pas.
- Soundness : l'analyse sur-approxime; FP tolérés, FN interdits. Le silence est une affirmation (`verified`), et l'outil refuse le « no issues found » sur du code qu'il n'a pas lu. C'est une visée du code, avec des écarts connus et documentés (liste 1.13).
- Fichiers rencontrés : `src/rules/api/diagnostic.rs` (`Severity`, `Diagnostic::error`); `src/rules/mod.rs` (`all_rules`); `src/driver/mod.rs` (`run_check`); `docs/usage.md`; `docs/adr/`.
- Chiffres au commit e67b10a : 142 fichiers et 59 757 lignes dans `src/`, 1 529 tests, 19 règles natives et 21 noms de diagnostic, 42 ADR, 14 dépôts de corpus, 1 498 localisations de référence.

Le chapitre 2 précise le modèle d'exécution de React que l'analyse abstrait ; le chapitre 3 donne les définitions exactes de ce que ce chapitre a présenté en mots.

1.8 Exercices

Les sorties citées dans les solutions ont été observées avec `check -no-color -info -trace` sur les fichiers indiqués.

Exercice 1.1: Un seuil plus lointain

On remplace la garde de `Bounded` (liste 1.9) par `if (n < 100) setN(n + 1) (/tmp/ch01/exo_hundred.tsx)`. (a) Que fait React ? (b) En raisonnant comme sur la figure 1.2, quel intervalle final attendez-vous pour `n` ? (c) `infinite-loop` doit-elle se déclencher ?

Solution

(a) Cent rendus supplémentaires après le montage, puis l'effet cesse d'écrire : `n` s'arrête à 100. (b) Le widening projette la borne qui grandit sur le plus petit seuil qui l'englobe ; les littéraux du composant fournissent 100, d'où $[0, 100]$, stable parce que l'effet n'écrit que sous `n < 100`. Le moteur calcule bien `number[0, 100]` (sonde hors dépôt). (c) Non : la sortie contient l'INFO `widening-info` (l'état a été élargi) et la ligne `verified infinite-loop no effect diverges into an infinite render loop`. Le widening seul n'est pas un diagnostic de boucle.

Exercice 1.2: Élargi, et pourtant sans boucle

Soit l'effet `useEffect(() => { setN(n + 1); }, [])` dans un composant `Once` dont l'état vaut initialement 0 (`/tmp/ch01/exo_once.tsx`). (a) Combien de rendus React fait-il ? (b) L'analyseur élargit l'état `n` à $[0, +\infty)$: pourquoi, alors que l'effet ne s'exécute qu'une fois ? (c) Quels diagnostics attendez-vous ?

Solution

(a) Deux : le montage (`n = 0`), puis le rendu provoqué par `setN(1)` ; les deps `[]` n'ayant pas changé, l'effet ne repart pas. (b) Le tour abstrait exécute *tous* les effets à chaque tour, qu'ils soient re-déclenchés ou non (section 1.3) : c'est une sur-approximation, qui fait croître `n` à chaque tour abstrait. La règle `infinite-loop`, elle, tient compte des deps et ne conclut pas à une boucle. (c) La sortie observée contient un WARNING `missing-deps` sur `n` (lu, non déclaré), l'INFO `widening-info`, et `verified infinite-loop`. Le code de sortie est 1, à cause du WARNING.

Exercice 1.3: Setter dans le corps du rendu

Quel niveau reactant-analyzer doit-il donner au composant suivant (/tmp/ch01/exo_render.tsx, ligne d'import omise), et pourquoi ?

```
1 export function Eager() {  
2   const [n, setN] = useState(0);  
3   setN(n + 1);  
4   return <p>{n}</p>;  
5 }
```

Solution

Un setter appelé pendant le rendu planifie un nouveau rendu avant même le commit ; appelé inconditionnellement, il relance le rendu à chaque fois, et React interrompt la boucle par une erreur (chapitre 2). Le défaut est certain dès que le composant est rendu, et la preuve est structurelle : l'appel est dans le bloc d'entrée du rendu, qui est sur tous les chemins. reactant-analyzer donne `error setter-in-render [hook:0] (line 5:2) setter 'setN' called directly in the render body, move this call into a useEffect or an event handler (chapitre 18).`

Exercice 1.4: Une constante de module

Dans /tmp/ch01/exo_const.tsx, un effet de deps [page] lit une prop page et une constante de module `const LIMIT = 10`. `missing-deps` doit-elle signaler `LIMIT` ? Que répond l'outil ?

Solution

Non : une constante de module ne change jamais entre deux rendus, l'omettre des deps ne peut pas figer une valeur périmée. L'analyse le sait parce qu'elle évalue les constantes de module (chapitre 5). Sortie observée : `1 clean component(s) hidden puis ✓ 1 file(s) no issues found.`

Exercice 1.5: Classifier des énoncés

Pour chacun des énoncés suivants, dites s'il peut justifier une `ERROR`, un `WARNING` ou seulement une `INFO` : (a) « cet effet n'a pas de tableau de deps et écrit, sur tous ses chemins, un objet littéral neuf dans un état » ; (b) « cet état a dû être élargi pendant l'analyse » ; (c) « cette variable, lue dans l'effet, peut changer entre deux rendus » ; (d) « ce hook importé n'a pas pu être lu ».

Solution

(a) `ERROR` possible : chaque conjoint est un fait certain (écriture sur tous les chemins, référence toujours neuve, re-déclenchement à chaque commit sans deps), la conclusion est prouvée en entier. Sur la variante de `Filters` (exemple 1.1) sans tableau de deps (/tmp/ch01/exo_nodeps_churn.tsx), l'outil répond `error infinite-loop [hook:1] (line 5:2) this effect has no dependency array and stores a fresh reference into state 'query', so it re-runs after every render and re-triggers itself: infinite render loop.` (b) `INFO` seulement : un widening peut venir d'une imprécision, c'est l'`INFO widening-info`. (c) `WARNING` : défaut possible, qui dépend de l'évolution réelle de la variable (`missing-deps`). (d) `INFO` : c'est une limite de l'analyse (`analysis-limit`), qui suspend aussi les lignes `verified` du composant.

Chapitre 2

React vu par l'analyste : le modèle d'exécution que l'on abstrait

Objectifs du chapitre.

- Décrire précisément ce que React fait d'un composant : déclenchement, rendu, commit, effets, et la façon dont un `setState` relance le cycle.
- Connaître les faits de React sur lesquels repose toute l'analyse : file d'updaters et batching, comparaison `Object.is` des deps et de l'état, identité fraîche des littéraux, ordre positionnel des hooks, Strict Mode, contexte, Server Components.
- Lire la sémantique formelle React-tRace (syntaxe, mémoire d'arbre, transitions `StepInit/StepEffect/StepCheck/StepEvent`, théorèmes) et savoir pourquoi le projet l'a prise comme sémantique concrète de référence (ADR-1).
- Savoir, pour chaque fait de React, ce que `reactant-analyzer` en garde, ce qu'il en abstrait, où cela se trouve dans le code, et pourquoi l'abstraction reste une sur-approximation.
- Connaître les exemples canoniques (`Counter`, `SelfCounter`, `Inf`, `Timer`, `Toggle`, `ObjChurn`...) réutilisés dans tout le manuscrit, avec la sortie réelle de l'analyseur sur chacun.

Prérequis : chapitre 1 (les trois programmes fautifs et la promesse `ERROR/WARNING/INFO`). Aucune connaissance d'analyse statique n'est requise : les notions de treillis, de join et de soundness sont seulement nommées ici et définies au chapitre 3.

Un analyseur par interprétation abstraite ne calcule pas « dans le vide » : il sur-approxime une *sémantique concrète*, c'est-à-dire une description exacte de ce que fait le programme lorsqu'il s'exécute. Pour un programme React, « ce que fait le programme » n'est pas seulement ce que fait le code JavaScript du composant : c'est ce que fait React *avec* ce code — quand il appelle la fonction, ce qu'il garde entre deux appels, quand il exécute les effets, ce qu'il compare pour décider de recommencer. Ce chapitre décrit ce modèle d'exécution du point de vue de quelqu'un qui doit l'abstraire.

Trois sources sont distinguées tout au long du chapitre, et il importe de ne jamais les confondre :

- **React** — la documentation officielle [Met26b], qui fixe le comportement attendu (les citations sont reprises en anglais, telles quelles) ;
- **React-tRace** — l'article de Lee, Ahn et Yi (OOPSLA 2025) [LAY25], qui donne une sémantique opérationnelle formelle d'un noyau de React ;
- **le code** — ce que `reactant-analyzer` calcule réellement au commit e67b10a, localisé par fichier et lignes.

Les huit premières sections suivent la progression d'un utilisateur de React : rendu, état, effets, identité, règles des hooks, Strict Mode, contexte, Server Components. Chacune se termine par

ce que l'analyseur en fait. Les trois dernières prennent de la hauteur : la sémantique formelle (section 2.9), la sémantique concrète que le projet adopte réellement (section 2.10) et le catalogue des exemples du manuscrit (section 2.11).

2.1 Un composant est une fonction du rendu

2.1.1 Un premier programme : Counter

Commençons par le programme React le plus simple qui ait un état. C'est le premier exemple de l'article React-tRace, transcrit en TSX.

Exemple 2.1: Counter : un état, un handler

Extrait 2.1 – Counter (fichier /tmp/ch02/counter.tsx)

```
1 import { useState } from "react";
2
3 export function Counter() {
4   const [count, setCount] = useState(0);
5   return (
6     <button onClick={() => setCount((c) => c + 1)}>
7       {count}
8     </button>
9   );
10 }
```

Déroulons ce qui se passe à l'exécution.

1. *Montage*. React appelle la fonction Counter(). L'appel useState(0) ne trouve aucun état existant : React en crée un, lui donne la valeur initiale 0, et renvoie la paire [0, setCount]. La fonction renvoie un élément JSX, c'est-à-dire une *description* d'un bouton dont le texte vaut 0.
2. *Commit*. React compare cette description à ce qui est affiché (rien) et crée le nœud DOM <button>.
3. *Clic*. Le navigateur déclenche le handler () => setCount((c) => c + 1). L'appel à setCount ne modifie pas count : il *met en file* une mise à jour et demande un nouveau rendu.
4. *Second rendu*. React rappelle Counter(). Cette fois useState(0) retrouve l'état existant, applique la mise à jour en file (c => c + 1 appliqué à 0) et renvoie [1, setCount]. L'argument 0 est ignoré.
5. *Second commit*. React compare la nouvelle description à l'ancienne; seul le texte du bouton diffère, seul ce nœud texte est modifié.

Rappel React : déclencher, rendre, valider

La documentation décrit trois étapes [Met26c] : « Triggering a render », « Rendering the component » et « Committing to the DOM ». Il n'y a que deux raisons de rendre un composant : « It's the component's initial render » et « The component's (or one of its ancestors') state has been updated ». Le rendu est récursif : rendre un composant rend les composants qu'il renvoie. Enfin, « React only changes the DOM nodes if there's a difference between renders ».

Pour l'analyste, la conséquence est simple et structurante : *le composant est une fonction que React appelle autant de fois qu'il le veut*. Entre deux appels, rien ne survit dans les variables locales de la fonction ; tout ce qui survit est rangé par React, à côté du composant, et rendu à

l'appel suivant par les hooks.

Définition 2.1: Tour de React

Un *tour* de React est la séquence

déclencheur $\rightarrow$ rendu $\rightarrow$ commit $\rightarrow$ effect

où le *déclencheur* est le montage ou une mise à jour d'état ; le *rendu* (render) appelle les fonctions des composants concernés et produit une description de l'interface ; le *commit* (commit) applique au DOM la différence avec la description précédente ; les *effets* (effect) exécutent les callbacks `useEffect` dont les deps ont changé. Si les effets appellent un setter, un nouveau tour est déclenché. Entre deux tours, React attend un événement extérieur (clic, réponse réseau, minuterie) : c'est la *boucle d'événements*.

La figure 2.1 représente un tour et les deux façons d'en déclencher un autre. Elle servira de référence dans tout le manuscrit : les phases *render*, *commit*, *effect* et le temps « plus tard » (tick, un tour ultérieur de la boucle d'événements) sont celles que la relation des écrivains distingue (chapitre 14).

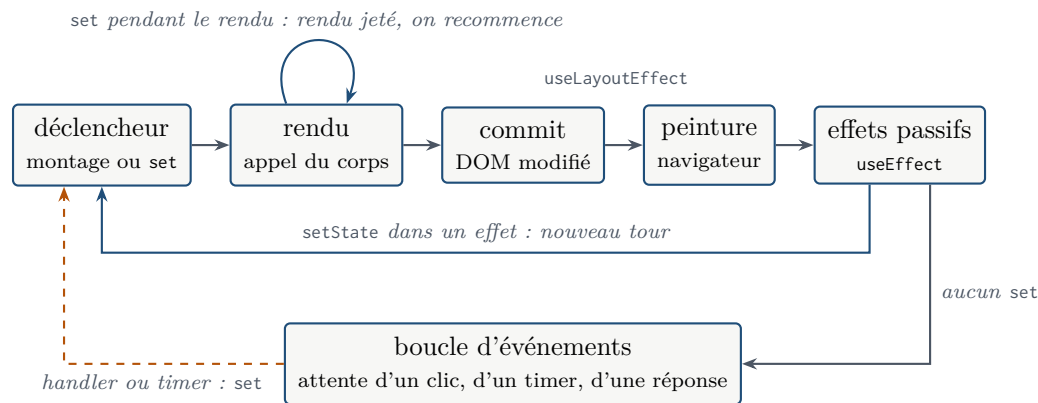


FIGURE 2.1 – Chronologie d'un tour de React. Un setter appelé dans un effet relance immédiatement un tour (flèche pleine) : c'est le mécanisme de la boucle infinie d'effets. Un setter appelé dans un handler ne relance un tour qu'après un événement extérieur (flèche tiretée). Un setter appelé pendant le rendu fait jeter le rendu et recommencer.

2.1.2 Le rendu doit être pur

Rappel React : pureté du rendu

« Rendering must always be a pure calculation » [Met26c] : mêmes entrées, même sortie (« Same inputs, same output »), et le composant ne modifie rien qui existait avant son appel (« It minds its own business »). Les entrées d'un rendu sont les props, l'état et le contexte.

La pureté est ce qui rend l'analyse possible : si le corps d'un composant est une fonction de (props, état, contexte), l'analyste peut l'évaluer abstraitement sur des *ensembles* de props et d'états sans se soucier du nombre de fois où React l'appelle. Mais la pureté n'est qu'une convention — rien n'empêche d'appeler un setter dans le corps ou de muter un objet d'état. *reactant-analyser* la traite donc comme une *hypothèse* dont chaque violation a sa propre règle : *setter-in-render* pour un setter appelé pendant le rendu, *state-mutation* pour une mutation en place. Ce motif — une hypothèse du modèle abstrait, doublée d'une règle qui signale sa violation — reviendra plusieurs fois ; le projet l'appelle *soundness en couches* (section 2.4).

2.1.3 Ce que l'analyseur voit de Counter

Le front-end de `reactant-analyzer` (chapitre 5) détecte Counter comme un composant, et le lowering (chapitre 7) en produit deux choses : un graphe de flot de contrôle du corps, le `render_cfg`, et une liste d'entrées de hooks. Pour Counter, cette liste contient un `HookEntry::State` (le `useState`, label 0) et un `HookEntry::Handler` (le `onClick`, label 1) : le handler n'est pas un hook React, mais l'IR l'extrait comme un *point d'entrée* séparé (section 2.10). D'où le « 2 hooks » de la sortie :

```
$ reactant check --no-color --info --show-clean --project plain counter.tsx
```

```
Counter (2 hooks) counter.tsx ✓
verified conditional-hook all hooks run unconditionally, in a stable order
verified lazy-init no useState/useRef initializer re-runs work on every render
verified setter-in-render no setter is called during render
verified state-mutation no state or prop object is mutated in place
```

```
✓ 1 file(s) no issues found.
```

Aucun diagnostic : un setter dans un handler est l'usage normal de React. Ce silence n'est pas une évidence pour un analyseur. Le handler peut s'exécuter un nombre quelconque de fois, et rien ne borne la valeur de `count` ; il faut donc que le moteur sache qu'une croissance causée par des clics n'est pas une boucle infinie. C'est exactement la distinction « déclenché par le cycle » / « déclenché de l'extérieur » de la figure 2.1, étudiée à la section 2.10.

Piège

Le corps d'un composant n'est pas « le programme ». Un lecteur qui aborde l'analyse comme celle d'une fonction JavaScript ordinaire oublie que React l'appelle en boucle, garde l'état *hors* de la fonction, et exécute d'autres morceaux du code (effets, handlers) à d'autres moments. Toute la difficulté de l'analyse — et tout ce chapitre — tient dans ce décalage.

2.2 L'état : useState, la file d'updaters et le batching

2.2.1 Un setter ne modifie pas la variable

Le premier malentendu sur React porte sur ce que fait un setter. Le programme suivant l'expose.

Extrait 2.2 – Batch : trois appels au setter dans un même handler (`/tmp/ch02/batching.tsx`)

```
1 import { useState } from "react";
2
3 export function Batch() {
4   const [n, setN] = useState(0);
5   function onClick() {
6     setN(n + 1);
7     setN(n + 1);
8     setN((m) => m + 1);
9   }
10  return <button onClick={onClick}>{n}</button>;
11 }
```

Un lecteur qui lit `setN` comme une affectation s'attend à ce qu'un clic fasse passer `n` de 0 à 3. Il n'en est rien : `n` est une constante du rendu courant, elle vaut 0 pendant tout le handler, et chaque appel au setter *ajoute une mise à jour à une file*. La file du premier clic contient trois entrées, que React rejoue au rendu suivant, dans l'ordre :

mise à jour en file	valeur reçue	valeur produite
« remplacer par $0 + 1$ »	0 (ignorée)	1
« remplacer par $0 + 1$ »	1 (ignorée)	1
$m \Rightarrow m + 1$	1	2

Le bouton affiche donc 2. Les deux premières entrées sont des *remplacements* calculés avec la valeur figée $n = 0$; seule la troisième, un *updater* (une fonction de l'état précédent vers le suivant), lit la valeur produite par les mises à jour qui la précèdent.

Rappel React : set, file d'updaters, batching

La documentation de `useState` [Met26b] est explicite : « The set function only updates the state variable for the *next* render. If you read the state variable after calling the set function, you will still get the old value that was on the screen before your call. » Les mises à jour sont regroupées (*batching*) : « React batches state updates. It updates the screen after all the event handlers have run and have called their set functions. » Pour les updaters, « React queues this function to be processed after all the other code in the event handler has run. During the next render, React goes through the queue and gives you the final updated state. » Enfin, le regroupement ne franchit pas les événements : « React does not batch across multiple intentional events like clicks ». L'exemple canonique de la page « Queueing a Series of State Updates » : `setNumber(number + 5)` suivi de `setNumber(n => n + 1)` donne 6.

Piège

« set modifie l'état » est faux ; « set planifie le prochain rendu » est juste. Deux conséquences qui trompent régulièrement les contributeurs de l'analyseur. D'abord, lire l'état juste après l'avoir écrit, dans le même corps, donne l'*ancienne* valeur : un modèle abstrait qui ferait « voir » la nouvelle valeur à la ligne suivante serait faux (nous verrons que *reactant-analyzer* voit l'ancienne *et* la nouvelle, ce qui est permis). Ensuite, `setN(n + 1)` et `setN(m => m + 1)` ne sont pas interchangeables : le premier capture n au moment où la fermeture a été créée, le second lit la file. C'est tout le ressort de la *stale closure* du Timer de l'introduction (liste 1.3).

2.2.2 Le bail-out `Object.is`

Un setter qui écrit la valeur déjà présente ne relance pas le rendu.

Rappel React : bail-out

« If the new value you provide is identical to the current state, as determined by an `Object.is` comparison, React will skip re-rendering the component and its children. » [Met26b] (page `useState`).

Ce détail décide de la terminaison de nombreux programmes. Dans le composant suivant, l'effet n'a pas de deps et tourne donc après chaque commit ; mais il écrit 0 dans un état qui vaut déjà 0, React compare, ne rend pas, et il n'y a pas de second commit.

Extrait 2.3 – `SameValue` : un effet qui réécrit la valeur courante (`/tmp/ch02/samevalue.tsx`, imports omis)

```

1 export function SameValue() {
2   const [n, setN] = useState(0);
3   useEffect(() => {
4     setN(0);
5   });
6   return <div>{n}</div>;

```

```
7 }
```

```
SameValue (2 hooks) samevalue.tsx
warn   redundant-set-state [hook:0] (line 5:2) state `n` is set to a stable value it already holds, so the update
      ↪ is redundant
```

L'analyseur ne signale pas de boucle (la valeur abstraite de n reste le point $[0, 0]$ et ne croît pas), et la règle `redundant-set-state` relève l'écriture inutile.

Remarque

Conventions pour les sorties de ce chapitre. Toutes ont été obtenues au commit `e67b10a` par `reactant check -no-color -info -show-clean -project plain <fichier>` (le binaire de `target/debug`). Elles sont élaguées, sans que ce soit redit, des lignes `verified` qui ne portent pas l'information, des lignes de bilan final et des lignes « (k trace step(s), rerun with `-trace`) » ; quand une sortie montre des lignes en flèche ($\rightarrow$), celles-ci viennent de la même commande relancée avec `-trace`. Les lignes `[verbose]` viennent de `-verbose`.

2.2.3 L'état initial et l'initialiseur paresseux

L'argument de `useState` n'est lu qu'une fois.

Rappel React : état initial

« React saves the initial state once and ignores it on the next renders. » Une fonction passée en argument est un *initialiseur* : « If you pass a function as `initialState`, it will be treated as an initializer function. It should be pure, should take no arguments, and should return a value of any type. » La documentation oppose `useState(createInitialTodos)` (la fonction est passée, appelée une seule fois) à `useState(createInitialTodos())` (l'appel est évalué à chaque rendu, son résultat jeté après le premier) [Met26b].

Extrait 2.4 – Todos : initialiseur appelé à chaque rendu, et sa forme paresseuse (`/tmp/ch02/lazy.tsx`, imports omis)

```
1 function createTodos() {
2   return [{ id: 1, done: false }];
3 }
4
5 export function Todos() {
6   const [todos] = useState(createTodos());
7   const [lazy] = useState(() => createTodos());
8   return <ul>{todos.length + lazy.length}</ul>;
9 }
```

```
Todos (2 hooks) lazy.tsx
warn   lazy-init [hook:0] (line 8:8) this useState is initialised by a direct function call. The call runs on
      ↪ every render but the result is only used on mount; wrap as `useState(() => ...)` to defer it
```

2.2.4 Ce que fait le code

Le lowering. Chaque appel de hook reconnu comme venant de React devient une entrée `HookEntry` (la liste complète est au section 2.5). Pour `useState`, l'entrée ne garde que l'expression initiale :

Extrait 2.5 – `src/lowering/hook_extractor.rs`, l. 667–681 — un `useState` devient `HookEntry::State`, un `useEffect` un `HookEntry::Effect`

```

667     match name {
668         "useState" => {
669             let init = it.next().unwrap_or(Expr::Lit(Prim::Unit));
670             Some(HookEntry::State { label, init, span })
671         }
672         "useEffect" => {
673             let body_cfg = hook_body_cfg(it.next());
674             let deps = it.next().map_or(DepsArg::Absent, DepsArg::from_expr);
675             Some(HookEntry::Effect {
676                 label,
677                 body_cfg,
678                 deps,
679                 span,
680             })
681         }

```

La valeur lue de l'état, `n`, devient dans le CFG du rendu l'expression `Expr::StateVal(label)` et le setter `Expr::StateSetter(label)`; le `label` est la position du hook (section 2.5).

L'amorçage. Avant la boucle de point fixe, le moteur évalue une fois chaque initialiseur et en fait la valeur de départ du slot. Un initialiseur paresseux (une fonction sans paramètre) est *exécuté*, et c'est sa valeur de retour qui amorce le slot, conformément à la documentation :

Extrait 2.6 – `src/engine/fixpoint.rs`, l. 336–350 — amorçage d'un slot, initialiseur paresseux compris

```

336     match init {
337         // Lazy initializer `useState(() => expr)`: React runs
338         // the thunk once at mount and stores its RETURN value
339         // - the state is never the closure itself. Abstracting
340         // the FnLit (reference(Unstable)) made every lazy-init
341         // state slot an "unstable dep" (corpus FP, TODO.md F2).
342         Expr::FnLit {
343             params, body_cfg, ..
344         } if params.is_empty() => crate::domains::interp::exec_body(
345             transfer, body_cfg, &init_env, &mut ac,
346         ),
347         _ => transfer.eval_expr(init, &init_env, &mut ac),
348     }
349 };
350 state.update(*label, init_val);

```

Cet amorçage a lieu *hors* de la boucle (`src/engine/fixpoint.rs`, l. 314–353) : l'initialiseur ne ré-influence jamais le store, exactement comme React « ignores it on the next renders ». La règle `lazy-init` exploite l'autre moitié du fait React : l'expression non paresseuse, elle, est ré-évaluée à chaque rendu.

Le store d'état. L'état abstrait d'un composant est un `StateStore`, une table de `label` vers valeur abstraite :

Extrait 2.7 – `src/domains/stores/state_store.rs`, l. 24–33 — lecture et mise à jour du store d'état

```

24     /// Returns `D::bottom()` for labels not yet updated by any setter call.
25     pub fn get(&self, label: HookLabel) -> D {
26         map_get_or(&self.0, &label, D::bottom)

```

```

27     }
28
29     /// Monotone update: `self[label] = self[label]  $\sqcup$  val`.
30     pub fn update(&mut self, label: HookLabel, val: D) {
31         let current = self.get(label);
32         self.0.insert(label, current.join(&val));
33     }

```

Tout tient dans la seconde fonction. Écrire dans un slot ne *remplace* pas sa valeur : on en prend le join avec la valeur écrite. C'est une *mise à jour faible* (*weak update*), et le store ne fait que croître selon l'ordre du treillis. Un slot jamais écrit ni amorcé vaut $\perp$.

L'exécution d'un setter. Quand l'interprète abstrait rencontre un appel `setX(arg)`, il exécute `exec_setter_call`, dont le commentaire de tête (`src/domains/interp/interpreter.rs`, l. 335–340) dit l'intention : « weak-update the corresponding state label », en traitant les updaters fonctionnels par l'exécution de leur corps avec le paramètre lié à la valeur courante de l'état.

Extrait 2.8 – `src/domains/interp/interpreter.rs`, l. 341–366 — `exec_setter_call` : un setter est un join, un updater est exécuté sur le store

```

341 fn exec_setter_call<T: Transfer>(
342     transfer: &T,
343     expr: &Expr,
344     env: &AbstractEnv<T::Domain>,
345     ctx: &mut AnalysisCtx<T::Domain>,
346     depth: usize,
347 ) {
348     if let Expr::Call { fn_, args } = expr
349         && let Expr::Var(name) = fn_.as_ref()
350         && let Some(label) = env.setter_label(name)
351     {
352         let arg_val = match args.first() {
353             Some(Expr::FnLit {
354                 params, body_cfg, ..
355             }) => {
356                 let mut sub_env = env.clone();
357                 if let Some(param) = params.first() {
358                     sub_env.extend(param.clone(), ctx.state.get(label));
359                 }
360                 exec_body_depth(transfer, body_cfg, &sub_env, ctx, depth + 1)
361             }
362             Some(a) => transfer.eval_expr(a, env, ctx),
363             None => T::Domain::top(),
364         };
365         ctx.state.update(label, arg_val);
366     }

```

Lisons-le branche par branche.

- La garde (l. 348–350) reconnaît un appel dont le callee est une variable liée à un setter de ce composant (`env.setter_label`).
- Si l'argument est une fonction littérale (l. 353–361), c'est un updater : son paramètre est lié à `ctx.state.get(label)`, la valeur *courante du store*, et son corps est exécuté ; la valeur écrite est ce que l'updater renvoie.
- Sinon (l. 362), l'argument est évalué dans l'environnement courant ; sans argument (l. 363), on écrit $\top$.
- Dans tous les cas, l. 365, la valeur est jointe au slot.

La suite de la fonction (`src/domains/interp/interpreter.rs`, l. 368–392) traite l'appel d'un setter d'un autre composant, en analyse inter-composants : l'argument est évalué et joint dans un `SharedStateStore` indexé par (`composant`, `label`). Nous y reviendrons au section 2.10, car cette branche est aussi le siège d'un défaut vérifié.

Exemple 2.2: Batch dans le domaine abstrait

Rejouons à la main la passe du handler de Batch (liste 2.2) lors de la première itération, en ne suivant que l'intervalle numérique. Le store vaut $\{n \mapsto [0, 0]\}$ et, dans l'environnement du handler, `n` vaut $[0, 0]$.

1. `setN(n + 1)` : on écrit $[1, 1]$; le store devient $[0, 0] \sqcup [1, 1] = [0, 1]$.
2. `setN(n + 1)` : `n` est lu dans l'environnement, qui n'a pas changé ($[0, 0]$) ; on écrit encore $[1, 1]$; le store reste $[0, 1]$.
3. `setN((m) => m + 1)` : `m` est lié au store, $[0, 1]$; l'updater renvoie $[1, 2]$; le store devient $[0, 2]$.

React produit 2 ; l'analyste a calculé $[0, 2]$, qui contient 2 mais aussi 0 et 1. Il ne représente ni la file, ni son ordre, ni le moment où elle est rejouée : il joint tout, tout de suite. (Ce calcul est celui de la première branche d'`exec_setter_call`, seule active en analyse intra ; sur le chemin programme de la CLI, la seconde branche s'y ajoute pour l'updater, avec l'effet décrit au section 2.10.)

Ce calcul est imprécis, mais il est *sûr* : tout état concret atteignable après n'importe quelle suite de clics et de rendus est obtenu en appliquant des mises à jour à des valeurs que le store contient déjà, et le store, fermé par join, contient leurs résultats. L'argument est développé au section 2.10 ; il repose sur deux choix que l'on retrouvera partout : la mise à jour est un join, et l'updater lit un sur-ensemble de la valeur que React lui passerait.

Remarque

Lire l'état après l'avoir écrit dans le même corps donne, dans l'analyste, l'ancienne valeur *jointe* à la nouvelle, alors que React donne l'ancienne. Le join contient la réponse de React : le résultat est imprécis mais correct. C'est le sens du premier piège de la section.

Le bail-out `Object.is`, lui, n'est pas modélisé dans le point fixe : le moteur suppose que tout set peut relancer un rendu. C'est encore une sur-approximation (on considère plus de rendus que React n'en fait), et le fait React n'est exploité que par des règles qui en ont besoin, comme `redundant-set-state` ci-dessus ou `state-mutation`, dont la documentation rappelle que muter un objet en place laisse `Object.is` conclure à l'égalité et saute le rendu (`src/rules/impls/state_mutation.rs`, l. 21–26).

useReducer. Le lowering range `useReducer` dans la même case que `useState` :

Extrait 2.9 – `src/lowering/hook_extractor.rs`, l. 715–719 — `useReducer` : le réducteur est jeté

```

715     "useReducer" => {
716         let _reducer = it.next(); // skip reducer fn
717         let init = it.next().unwrap_or(Expr::Lit(Prim::Unit));
718         Some(HookEntry::State { label, init, span })
719     }

```

Le réducteur n'est jamais exécuté. `dispatch(action)` est donc lu comme un setter qui écrit l'action dans le slot. Le section 2.10 montre la conséquence observée : une `ERROR` sur un programme qui ne boucle pas.

2.3 Les effets : après le commit, gardés par les deps

2.3.1 Quand un effet s'exécute

Un effet est la seule partie du composant que React exécute *après* le commit. C'est aussi la seule qui puisse, à elle seule, relancer un tour sans intervention extérieure : c'est donc elle qui porte les boucles infinies.

Rappel React : `useEffect`

Signature : `useEffect(setup, dependencies?)`. « React will compare each dependency with its previous value using the `Object.is` comparison. If you omit this argument, your Effect will re-run after every commit of the component. » Avec `[]`, l'effet ne tourne qu'après le commit initial. « After every commit with changed dependencies, React will first run the cleanup function (if you provided it) with the old values, and then run your setup function with the new values » ; au démontage, le cleanup tourne une dernière fois. Le moment exact : « If your Effect wasn't caused by an interaction (like a click), React will generally let the browser paint the updated screen first before running your Effect. » Enfin, « Effects only run on the client » [Met26b].

Les trois formes du tableau de deps donnent trois comportements, que le programme suivant met côte à côte avec le même corps d'effet.

Extrait 2.10 – Trois formes de deps pour le même effet (`/tmp/ch02/deps.tsx`, imports omis)

```
1 export function EveryCommit() {
2   const [n, setN] = useState(0);
3   useEffect(() => {
4     setN(n + 1);
5   });
6   return <div>{n}</div>;
7 }
8
9 export function MountOnly() {
10  const [n, setN] = useState(0);
11  useEffect(() => {
12    setN(n + 1);
13  }, []);
14  return <div>{n}</div>;
15 }
16
17 export function OnChange() {
18  const [n, setN] = useState(0);
19  useEffect(() => {
20    setN(n + 1);
21  }, [n]);
22  return <div>{n}</div>;
23 }
```

À l'exécution :

- `EveryCommit` boucle : chaque commit exécute l'effet, qui écrit une valeur nouvelle, donc un nouveau rendu, donc un nouveau commit ;
- `MountOnly` rend deux fois (0 puis 1) et s'arrête : l'effet ne tourne qu'une fois ;
- `OnChange` boucle aussi : `n` change à chaque tour, et c'est justement sa dep.

EveryCommit (2 hooks) deps.tsx

warn infinite-loop [hook:0] (line 5:2) this effect keeps pushing state `n` to new values on every run. Potential
 ↳ infinite render loop

```

info widening-info (line 5:2) state `n` kept changing during analysis and was approximated to converge, so
  ↳ findings that depend on it may be imprecise
MountOnly (2 hooks) deps.tsx
warn missing-deps [hook:1] var:n (line 13:2) `n` is used in this effect but not in its deps array, and it is
  ↳ recreated on every render
info widening-info (line 13:2) state `n` kept changing during analysis and was approximated to converge, so
  ↳ findings that depend on it may be imprecise
verified infinite-loop no effect diverges into an infinite render loop
OnChange (2 hooks) deps.tsx
warn infinite-loop [hook:0] (line 21:2) this effect keeps pushing state `n` (its deps do not provably gate it,
  ↳ so the effect can re-run every render) to new values on every run. Potential infinite render loop
info widening-info (line 21:2) state `n` kept changing during analysis and was approximated to converge, so
  ↳ findings that depend on it may be imprecise

```

Les deux boucles sont signalées en WARNING, jamais en ERROR : la divergence d'une valeur numérique ne donne pas, à ce jour, la preuve complète qu'exige le niveau ERROR (#144). MountOnly n'a pas de boucle, mais un INFO widening-info y apparaît : le piège de la fin de section explique pourquoi.

2.3.2 SelfCounter et Inf : le cycle Check–Effect

L'article React-tRace [LAY25] illustre l'effet par deux composants presque identiques. Leur transcription TSX :

Exemple 2.3: Inf et SelfCounter

Extrait 2.11 – Inf et SelfCounter (/tmp/ch02/selfcounter.tsx)

```

1 import { useEffect, useState } from "react";
2
3 export function Inf() {
4   const [s, setS] = useState(0);
5   useEffect(() => {
6     setS(s + 1);
7   });
8   return <div>{s}</div>;
9 }
10
11 export function SelfCounter() {
12   const [s, setS] = useState(0);
13   useEffect(() => {
14     if (s < 3) setS(s + 1);
15   }, [s]);
16   return <div>{s}</div>;
17 }

```

SelfCounter rend quatre fois de suite sans aucune interaction (s vaut 0, 1, 2, 3) puis s'arrête, comme le montre la figure 2.2; Inf, privé de la garde, ne s'arrête jamais. L'article résume le mécanisme : « Infinite render loop occurs because setting state within an Effect triggers the runtime to Check if the state has changed [...]. If the state whose setter function is called is actually modified, the component re-renders and decides to run the Effect again. Essentially, this Check-Effect decision cycle creates a render loop. » Les mots *Check* et *Effect* sont ici des termes techniques, les *décisions* de la sémantique formelle (section 2.9).

```

Inf (2 hooks) selfcounter.tsx
warn infinite-loop [hook:0] (line 5:2) this effect keeps pushing state `s` to new values on every run. Potential
  ↳ infinite render loop
  ↳ state `s` is written here [hook:1] (line 5:2)
  ↳ the abstract value of state `s` kept growing and was widened at iteration 3

```

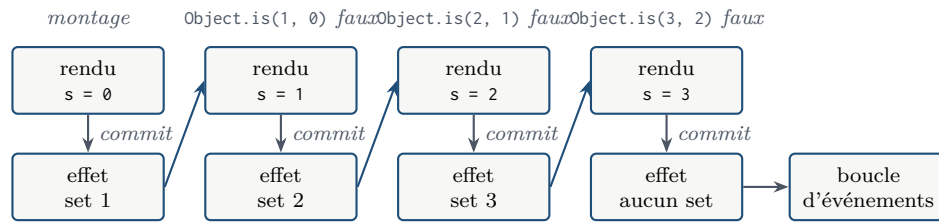


FIGURE 2.2 – Déroulement concret de SelfCounter. Chaque rendu est suivi d'un commit puis de l'effet, car la dep `s` a changé. Le quatrième effet tourne encore (la dep a changé de 2 à 3) mais n'écrit rien : aucun tour n'est relancé et React attend un événement. Pour Inf, la chaîne ne s'arrête pas.

```

info widening-info (line 5:2) state `s` kept changing during analysis and was approximated to converge, so
  ↳ findings that depend on it may be imprecise
SelfCounter (2 hooks) selfcounter.tsx
info widening-info (line 13:2) state `s` kept changing during analysis and was approximated to converge, so
  ↳ findings that depend on it may be imprecise
verified infinite-loop no effect diverges into an infinite render loop
  
```

La sortie (obtenue avec `-trace` pour les lignes en flèche) raconte le calcul abstrait. Pour Inf, le slot `s` vaut $[0, 0]$, puis $[0, 1]$, puis $[0, 2]$; au troisième tour de la boucle abstraite, le moteur applique un *widening* et le slot passe à $[0, +\infty]$; ce slot entre dans la trace d'élargissement (*widen_trace*), et la règle *infinite-loop* en conclut que l'effet pousse l'état vers des valeurs toujours nouvelles. Pour SelfCounter, le slot est aussi élargi (d'où l'INFO), mais l'écriture elle-même, recalculée après convergence, reste bornée par la garde `s < 3` : pas de boucle. Le mécanisme exact (*widening* à seuils, écritures « pures » des effets) est l'objet du chapitre 13.

2.3.3 Flicker : un rendu de trop

Un effet de montage qui écrit une constante ne boucle pas, mais coûte un rendu : la valeur initiale s'affiche une fraction de seconde. L'article appelle ce composant Flicker (« A swift user might even see the transient state 0 »).

Exemple 2.4: Flicker

Extrait 2.12 – Flicker (/tmp/ch02/flicker.tsx, imports omis)

```

1 export function Flicker() {
2   const [s, setS] = useState(0);
3   useEffect(() => {
4     setS(42);
5   }, []);
6   return <div>{s}</div>;
7 }
  
```

```

Flicker (2 hooks) flicker.tsx
warn unnecessary-rerender [hook:0] (line 5:2) mount-only effect sets state `s` to a constant different from
  ↳ its initial value, which costs one extra rerender on mount; consider initialising directly with the target
  ↳ value
  
```

2.3.4 Toggle : une boucle que la valeur abstraite ne voit pas

Le dernier exemple canonique de cette section est un avertissement pour le lecteur, et une limite de l'outil au commit étudié.

Exemple 2.5: Toggle**Extrait 2.13 – Toggle (/tmp/ch02/toggle.tsx)**

```

1 import { useEffect, useState } from "react";
2
3 export function Toggle() {
4   const [on, setOn] = useState(false);
5   useEffect(() => {
6     setOn(!on);
7   }, [on]);
8   return <div>{on ? 1 : 0}</div>;
9 }

```

En React, ce composant boucle : on passe de `false` à `true`, la dep a changé, l'effet repasse, on revient à `false`, et ainsi de suite, un rendu par tour. Pourtant l'ensemble des valeurs atteintes est fini : `{false, true}`. L'analyseur, qui calcule précisément cet ensemble, converge sans widening :

```

[verbose] Toggle: 1 iteration(s), widened: []
✓ 1 file(s) no issues found.

```

Piège

Un point fixe abstrait atteint ne prouve pas que la boucle de React s'arrête. La sémantique que le moteur sur-approxime est l'ensemble des *états* atteignables, pas la longueur des suites de rendus. Une boucle qui oscille entre des valeurs déjà vues (Toggle) a un ensemble d'états fini ; sa détection ne peut venir que d'un argument de *structure* (la dep est le slot écrit, et la valeur écrite diffère toujours de la valeur lue), du même genre que celui du churn d'objets (section 2.4). Au commit e67b10a, ce raisonnement existe pour les références fraîches, pas pour une négation booléenne : Toggle est un faux négatif observé.

2.3.5 Cleanup et effets de mise en page

Un effet peut renvoyer une fonction de nettoyage. React l'appelle avant chaque ré-exécution et au démontage. Un effet qui s'abonne sans se désabonner empile les abonnements :

Extrait 2.14 – Width : un abonnement sans cleanup (/tmp/ch02/listener.tsx, imports omis)

```

1 export function Width() {
2   const [w, setW] = useState(0);
3   const box = useRef(null);
4   useEffect(() => {
5     window.addEventListener("resize", () => setW(window.innerWidth));
6   }, [box]);
7   return <div ref={box}>{w}</div>;
8 }

```

Width (4 hooks) listener.tsx

```

warn  missing-cleanup [hook:2] (line 7:4) this effect calls `window.addEventListener` but returns no cleanup. The
  ↳ registration is repeated every time the effect re-runs (and on every mount, twice under StrictMode) and nothing
  ↳ ever undoes it; return a function that tears it down
verified infinite-loop no effect diverges into an infinite render loop

```

Deux détails de la sortie comptent pour ce chapitre. Les « 4 hooks » sont le `useState`, le `useRef`, l'effet et le callback d'`addEventListener`, extrait comme un handler (`HookEntry::Handler`) par le lowering (extract_subscriptions, src/lowering/hook_extractor.rs, l. 16–26) : il ne tourne que sur un événement extérieur. Et `infinite-loop` est vérifiée : l'écriture `setW` n'a lieu que sur un

redimensionnement, pas dans le cycle rendu-effet (section 2.10).

`useLayoutEffect` et `useInsertionEffect` s'exécutent plus tôt que `useEffect` (avant la peinture). Pour l'analyseur, la différence de moment ne change rien à ce qui compte — l'effet ne s'exécute jamais pendant le rendu, et il s'exécute après le commit — et le lowering les confond :

Extrait 2.15 — `src/lowering/hook_extractor.rs`, l. 720–729 — `useLayoutEffect` et `useInsertionEffect` sont des `HookEntry::Effect`

```
720     "useLayoutEffect" | "useInsertionEffect" => {
721         let body_cfg = hook_body_cfg(it.next());
722         let deps = it.next().map_or(DepsArg::Absent, DepsArg::from_expr);
723         Some(HookEntry::Effect {
724             label,
725             body_cfg,
726             deps,
727             span,
728         })
729     }
```

2.3.6 Ce que fait le code

Tous les effets, à chaque itération. La boucle de point fixe (chapitre 13) exécute, à chaque itération, le rendu puis chaque effet, sur le même instantané du store et dans l'environnement de sortie du rendu :

Extrait 2.16 — `src/engine/fixpoint.rs`, l. 415–449 — les passes d'effets de la boucle de point fixe

```
415     // -- Effect passes -----
416     let mut state_from_effects = StateStore::bottom();
417     slot_writers.clear();
418     for hook in &hooks {
419         if let HookEntry::Effect {
420             label, body_cfg, ..
421         } = hook
422         {
423             let (eff_bs, eff_state) = {
424                 let ctx = FixpointCtx {
425                     state: &state_store,
426                     memo: &memo_store,
427                     callbacks: &callback_bodies,
428                 };
429                 analyze_cfg:::<T>{
430                     comp_id,
431                     body_cfg,
432                     env_exit.clone(),
433                     &state_store,
434                     &memo_store,
435                     transfer,
436                     config.widen_threshold,
437                     &thresholds,
438                     &mut heap,
439                     &ctx,
440                     inter,
441                 }
442             };
443             effect_block_states.insert(*label, eff_bs);
444             for slot in eff_state.labels() {
445                 slot_writers.entry(slot).or_default().push(*label);
446             }
447         }
448     }
```

```

447         state_from_effects = state_from_effects.join(&eff_state);
448     }
449 }

```

Le motif `HookEntry::Effect label, body_cfg, ..` de la l. 419 ignore le champ `deps` : *tous* les effets sont exécutés à chaque itération, quels que soient leurs `deps`. C'est la décision de modélisation la plus importante de la section.

Décision : tous les effets à chaque tour abstrait

React exécute un effet *si et seulement si* l'une de ses `deps` a changé selon `Object.is` (ou s'il n'a pas de `deps`, ou au montage). Le point fixe l'exécute *toujours*. L'ensemble des écritures possibles en est un sur-ensemble : c'est sound. La précision perdue est récupérée dans les règles, qui relisent les `deps` : `infinite-loop` écarte les effets de montage et ceux dont toutes les `deps` sont prouvées stables (`src/rules/impls/infinite_loop.rs`, l. 96–113), et la relation `effect_triggers` dit quand un effet *doit* repartir. L'ADR-004 (ADR-4) décrivait une passe « `for each Effect whose decision = Effect` » ; le code n'a pas de notion de décision, et c'est plus simple et plus sûr.

Chaque effet démarre du même store et du même environnement (celui du rendu de cette itération) : l'ordre dans lequel React exécute les effets (enfants d'abord, puis ordre syntaxique) est abstrait par le `join`, qui est commutatif ; l'influence d'un effet sur un autre est vue à l'itération suivante. Et un effet voit les valeurs *du rendu qui l'a créé* : c'est une fermeture, comme dans React.

Quand un effet doit repartir. Pour transformer « l'effet *peut* repartir » en « l'effet *doit* repartir », il faut savoir qu'une `dep` change à coup sûr. Le moteur calcule pour cela une relation, une ligne par couple (`dep`, `slot`) :

Extrait 2.17 – `src/engine/triggers.rs`, l. 33–44 — une ligne de la relation `effect_triggers`

```

33 /// One dep of an effect that a state slot moves.
34 #[derive(Debug, Clone, PartialEq, Eq)]
35 pub struct EffectTrigger {
36     pub hook: HookLabel,
37     /// Index of the dep in the effect's deps list.
38     pub dep: usize,
39     pub slot: QualifiedSlot,
40     /// The dep IS the slot value (`StateVal(1)` or an alias of it): the effect
41     /// must re-run whenever a fresh value is stored. `false`: the dep is
42     /// versioned by the slot, the effect may re-run.
43     pub exact: bool,
44 }

```

Le bit `exact` est le cœur : la `dep` est la valeur du slot (ou un alias), donc si l'on stocke dans ce slot une valeur fraîche, `Object.is` échoue sur cette `dep` et l'effet repart. Sans ce bit, la `dep` est seulement *versionnée* par le slot (un champ, un mémoire, une prop calculée par le parent) et l'effet *peut* repartir. Le commentaire de module précise l'intention : « This answers identity, not dependence » (`src/engine/triggers.rs`, l. 1–17). La relation est étudiée au chapitre 14 ; retenons qu'elle est la traduction abstraite de la phrase « React compare chaque `dep` avec `Object.is` ».

Piège

`widen_trace` ne signifie pas « boucle React ». Dans `MountOnly`, le slot est élargi (l'INFO `widening-info` le dit) parce que le point fixe exécute l'effet à chaque itération, `deps []` ou non. La trace d'élargissement dit « la valeur abstraite croît sous l'hypothèse que tout effet

peut toujours repartir » ; c'est la règle, en relisant les deps, qui décide s'il y a boucle. Lire `widen_trace` seul produirait un faux positif sur chaque effet de montage.

2.4 L'identité référentielle

2.4.1 Même contenu, autre objet

React compare les deps, l'état et la valeur d'un contexte avec `Object.is`. Pour un nombre, une chaîne ou un booléen, c'est une comparaison de valeur ; pour un objet, un tableau ou une fonction, c'est une comparaison d'*identité* : deux objets de même contenu sont différents s'ils ont été alloués séparément.

Rappel React : identité et mémorisation

« In JavaScript, a function `()` or `() =>` always creates a *different* function, similar to how the object literal always creates a new object » (page `useCallback`). La documentation de `useEffect` en tire la conséquence : un objet construit dans le corps du composant et placé dans les deps fait repartir l'effet à chaque rendu ; le remède est de le construire *dans* l'effet, ou de le mémoriser. `useMemo` et `useCallback` renvoient la valeur en cache tant que leurs deps sont égales au sens de `Object.is`, et la recalculent sinon ; sans tableau, à chaque rendu. `useRef` renvoie le même objet à tous les rendus, et modifier `ref.current` ne déclenche pas de rendu [Met26b].

Extrait 2.18 – Search : une dep fraîche à chaque rendu, et sa version mémorisée (/tmp/ch02/search.tsx, imports omis)

```
1 export function Search({ query }: { query: string }) {
2   const [hits, setHits] = useState(0);
3   const options = { query, limit: 10 };
4   const stable = useMemo(() => ({ query, limit: 10 }), [query]);
5   useEffect(() => {
6     console.log(options.limit);
7   }, [options]);
8   useEffect(() => {
9     console.log(stable.limit, hits);
10  }, [stable, hits]);
11  return <button onClick={() => setHits(hits + 1)}>{hits}</button>;
12 }
```

Search (5 hooks) search.tsx

warn always-unstable-deps [hook:2] (line 7:2) this effect depends on `options`, a new reference every render, so
 ↳ `Object.is` always differs and the effect re-runs on every render regardless of the other deps

Le premier effet repart à chaque rendu : `options` est un littéral objet, donc un objet neuf à chaque appel du corps. Le second ne repart que si `query` ou `hits` change : `stable` est la même référence tant que `query` ne bouge pas. Les « 5 hooks » sont l'état, le mémo, les deux effets et le handler du bouton.

2.4.2 ObjChurn : une boucle par l'identité

L'identité peut aussi fabriquer une boucle infinie sans qu'aucune valeur ne « croisse ». C'est l'exemple qui a motivé l'ADR-017 (ADR-17).

Exemple 2.6: ObjChurn et ObjStateDep**Extrait 2.19 – ObjChurn et ObjStateDep (/tmp/ch02/objchurn.tsx)**

```

1 import { useEffect, useState } from "react";
2
3 export function ObjChurn() {
4   const [obj, setObj] = useState({ a: 1 });
5   useEffect(() => {
6     setObj({ ...obj, b: 2 });
7   }, [obj]);
8   return <div>{obj.a}</div>;
9 }
10
11 export function ObjStateDep() {
12   const [ctx] = useState({ locale: "en" });
13   useEffect(() => {
14     document.title = ctx.locale;
15   }, [ctx]);
16   return <div>{ctx.locale}</div>;
17 }

```

Dans `ObjChurn`, l'effet écrit un objet neuf; `Object.is` échoue sur la dep `obj`; l'effet repart, et ainsi de suite : c'est une boucle certaine, alors que le *contenu* de l'objet est le même dès le deuxième tour. Dans `ObjStateDep`, l'état est aussi un objet littéral, mais rien ne l'écrit : sa référence est la même à chaque rendu, et l'effet ne tourne qu'au montage.

```

ObjChurn (2 hooks) objchurn.tsx
  error infinite-loop [hook:0] (line 5:2) this effect recreates object state `obj` it depends on. Every run stores
    ↳ a fresh reference (`Object.is` always fails) and re-triggers itself: infinite render loop
    ↳ → a fresh value is written to state `obj` here [hook:1] (line 6:4)
ObjStateDep (2 hooks) objchurn.tsx ✓
  verified always-unstable-deps no deps array is defeated by an always-fresh reference
  verified infinite-loop no effect diverges into an infinite render loop

```

Les deux verdicts sont justes, et chacun demande un fait différent. Pour `ObjChurn`, la divergence de valeur (le mécanisme de `Inf`) ne sert à rien : la valeur abstraite de la référence ne croît pas, « fraîche » joint « fraîche » reste « fraîche », et le point fixe converge dès le premier test (`-verbose: ObjChurn: 0 iteration(s), widened: []`). Pour `ObjStateDep`, il faut au contraire savoir que *lire* un état objet ne donne pas une référence fraîche à chaque rendu, même si l'état a été initialisé par un littéral.

2.4.3 Ce que fait le code

Les allocations. L'évaluateur abstrait donne à chaque forme d'expression sa stabilité. Les littéraux objet, tableau et fonction sont frais à chaque rendu; un élément JSX natif, un `useRef` sont stables; un hook non résolu est $\top$:

Extrait 2.20 – `src/domains/transfer/state_value.rs`, l. 142–156 — l'identité des marqueurs de hooks et des littéraux

```

142 Expr::HookMarker(_, MarkerVal::Undefined) => StateValue::undefined(),
143 Expr::HookMarker(_, MarkerVal::Unknown) => StateValue::top(),
144 // `useRef` hands back the same container every render – a reference,
145 // and a stable one. Both halves matter: `undefined` was stable too, but
146 // it was not a reference, so the identity was invisible.
147 Expr::HookMarker(_, MarkerVal::StableRef) => StateValue::reference(Stability::Stable),
148 // A summarized library hook reads exactly as its summary; the marker
149 // is kept (rather than replaced by a bare `SummaryVal`) so the label

```



```

150     // stays anchored at the call site.
151     Expr::HookMarker(_, MarkerVal::Summary(sv)) => summary_value(sv),
152
153     Expr::ObjectLit { .. } => StateValue::reference(Stability::PerRender),
154     Expr::ArrayLit { .. } => StateValue::reference(Stability::PerRender),
155     Expr::FnLit { .. } => StateValue::reference(Stability::PerRender),
156     Expr::NativeElem { .. } => StateValue::reference(Stability::Stable),

```

S'y ajoutent `new X()` (`src/domains/transfer/state_value.rs`, l. 181) et les appels dont le résultat est prouvé neuf, comme `map`, `filter` ou `JSON.parse` (`returns_fresh_reference`, `src/domains/transfer/state_value.rs`, l. 238–259). La liste exclut à dessein `slice` et `concat`, qui renvoient une primitive sur une chaîne, et les méthodes en place (`sort`, `reverse...`) qui renvoient le receveur.

Le treillis de stabilité. La stabilité est une borne sur la *trace de changement* d'une valeur, l'ensemble des rendus où `Object.is` échoue entre deux rendus consécutifs, qui est la seule chose que React observe :

Extrait 2.21 – `src/domains/impls/stability.rs`, l. 13–52 — le treillis Stability

```

13 /// Stability lattice: bounds on the *change trace* of a value – the set of
14 /// renders where `Object.is(vi, vi-1)` fails (the only thing React observes).
15 ///
16 /// Two kinds of bounds coexist (ADR-017):
17 /// - may bound (over-approx): used by rules to stay silent soundly.
18 /// - must bound (under-approx): used by rules to fire without FPs.
19 ///
20 /// ```text
21 ///           Unknown  (⊤)
22 ///           /       \
23 ///   VersionedTop   PerRender
24 ///       |           |
25 ///   Versioned(S) ⊆-chains |
26 ///       |           |
27 ///     Stable         |
28 ///           \       /
29 ///           Bottom  (⊥)
30 /// ```
31 ///
32 /// `Versioned`, `VersionedTop` and `PerRender` are incomparable – not
33 /// opposites, different bounds: `join = Unknown` (guarantees nothing in
34 /// either direction).
35 #[derive(Debug, Clone, PartialEq, Eq)]
36 pub enum Stability {
37     /// ⊥ no information (unreachable path / uninitialized).
38     Bottom,
39     /// Never changes: the same reference on every render (safe as a dep).
40     Stable,
41     /// Changes only at setter events of these state slots (may bound).
42     /// Invariant: non-empty (canonicalised – `Versioned(⌀) ≡ Stable`) and
43     /// `len() ≤ VERSIONED_LABELS_THRESHOLD` (widened to `VersionedTop` above).
44     Versioned(BTreeSet<QualifiedSlot>),
45     /// Versioned by unknown state slots (threshold-widened `Versioned`).
46     VersionedTop,
47     /// A fresh reference every render, guaranteed (must bound).
48     /// For non-reference kinds via `to_stability`: "may change every render".
49     PerRender,
50     /// ⊤ no bound in either direction.
51     Unknown,
52 }

```

La figure 2.3 redessine le diagramme du commentaire. Le treillis est défini et étudié au chapitre 10 (définition 10.3) ; ce qui nous importe ici est la lecture React de chaque point. **Stable** : la même référence à chaque rendu. **PerRender** : une référence neuve à chaque rendu, à *coup sûr* — c'est une borne must, qui permet de tirer. **Versioned(S)** : ne change *qu'aux* événements de set des slots de S — c'est une borne may, qui permet de se taire. **PerRender** et **Versioned** ne sont pas opposés : ce sont deux bornes différentes, incomparables, dont le join ne garantit rien (**Unknown**).

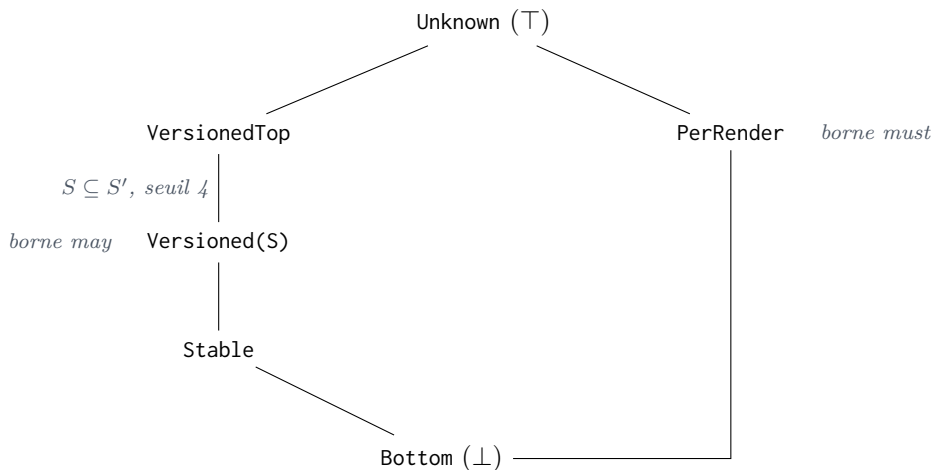


FIGURE 2.3 – Le treillis Stability (ADR-017). À gauche, la chaîne may « ne change qu'aux sets de S », dont la hauteur est bornée par le seuil `VERSIONED_LABELS_THRESHOLD = 4` ; à droite, le fait must « neuf à chaque rendu ». Leur join est $\top$.

La lecture d'un état. Voici le fait React qui règle `ObjStateDep` : un état ne change *que* lorsque son setter est appelé. Le moteur le traduit à un seul endroit, la lecture d'un slot :

Extrait 2.22 – `src/domains/transfer/state_value.rs`, l. 122–134 — la conversion « côté lecture » de l'ADR-017

```

122 Expr::StateVal(label) => {
123     let mut val = ctx.state.get(*label);
124     // ADR-017 read-side conversion: the store holds the join of
125     // *written* values (event view); what a render *reads* can only
126     // change at setter events of this slot (cross-render view). The
127     // written value's allocation freshness (`PerRender`) must not
128     // leak into reads. Assumes sets happen outside render – the
129     // violation has its own diagnostic (`setter-in-render`).
130     if val.reference != Stability::Bottom {
131         val.reference = Stability::versioned_by(ctx.component, *label);
132     }
133     val
134 }
```

Le store contient le join des valeurs *écrites* ; pour `ObjStateDep`, c'est la référence `PerRender` du littéral d'initialisation. Mais ce qu'un rendu *lit* ne peut changer qu'aux événements de set de ce slot : la lecture remplace donc la composante référence par `Versioned((composant, label))`. Une dep `Versioned` ne déclenche pas `always-unstable-deps`, qui exige `PerRender`.

Invariant 2.1: Double vue de l'état

Le store d'état est la *vue événement* : pour chaque slot, le join des valeurs écrites. L'évaluation de `Expr::StateVal` est la *vue inter-rendus* : ce que `Object.is` compare d'un rendu à l'autre, dont la composante référence est toujours `Versioned` par le slot lui-même (sauf si elle est $\perp$). Les règles qui comparent des valeurs écrites (`redundant-set-state`) lisent le store ; les règles de deps lisent la vue inter-rendus.

Pour `ObjChurn`, la certitude vient d'ailleurs : de la *structure*. La dep `obj` est le slot (ligne exact de `effect_triggers`, section 2.3), l'effet écrit dans ce slot une valeur `PerRender` sur tous ses chemins, et React ré-exécute un effet dont une dep a changé. Trois faits `must`, donc une `ERROR`. Le bras correspondant d'`infinite-loop` et le graphe de churn qui le généralise sont l'objet du chapitre 15.

Décision : ADR-017 — stabilité versionnée

Contexte. Cinq faux positifs `always-unstable-deps` du premier corpus avaient tous la forme de `ObjStateDep` : un état objet lu comme « neuf à chaque rendu ». Sous ce FP se cachait un FN : `ObjChurn` n'était pas détecté, parce que le join de deux références fraîches ne croît pas. *Décision.* Un treillis à six points, fragment d'un produit `may × must` ; la conversion côté lecture dans `StateVal` ; un bras « churn » dans `infinite-loop`. *Alternatives refusées.* Rendre les lectures d'état stables sans nouveau signal (rouvrir le FN) ; encoder un compteur d'événements dans le store pour faire tirer le widening (« a non-standard hack ») ; garder dans le treillis deux points « change à coup sûr au set », qui n'avaient qu'un consommateur, lequel a de toute façon besoin d'un raisonnement d'atteignabilité et vit donc dans la règle.

Les mémos. Le moteur ne calcule pas la *valeur* d'un mémo, seulement son identité, comme join des stabilités de ses deps :

Extrait 2.23 – `src/domains/transfer/state_value.rs`, l. 63–77 — `recompute_memo` : les cas sans deps lisibles et `[]`

```
63      // No readable deps array: the memo may recompute on any render and may
64      // equally never recompute – T. `Stable` here would be a *must* claim
65      // about a list the IR never saw, which is how `useMemo(fn, deps)` used
66      // to read as pinned forever.
67      let Some(deps) = deps.list() else {
68          return StateValue::reference(Stability::Unknown);
69      };
70      if deps.arity == Arity::Exact(0) {
71          // `[]` pins the memo – but only an array *known* to be empty.
72          return StateValue::reference(Stability::Stable);
73      }
74      if deps.is_empty() {
75          // Every entry is a spread whose source the fold cannot reach.
76          return StateValue::reference(Stability::Unknown);
77      }
```

Sans tableau lisible, le mémo peut être recalculé à chaque rendu comme ne jamais l'être : `T`. Avec `[]` (d'arité exactement nulle), il est figé : `Stable`. Sinon (`src/domains/transfer/state_value.rs`, l. 78–100), chaque dep contribue : une dep qui est un slot contribue `Versioned(slot)`, les autres leur stabilité évaluée. Pour `Search`, `stable` hérite ainsi de la stabilité de la prop query, qui n'est pas prouvée `PerRender` : silence.

Définition 2.2: Soundness en couches

Un argument de soundness est *en couches* lorsqu'il repose sur une hypothèse du modèle abstrait dont chaque violation est signalée par une règle dédiée. La conversion côté lecture suppose que les sets ont lieu hors du rendu : si un setter est appelé *pendant* le rendu avec une référence fraîche, le slot change réellement à chaque rendu et Versioned le sous-estime. Cette violation est signalée à part par **setter-in-render**. De même, lire un champ d'un objet d'état suppose qu'on ne le mute pas en place, violation signalée par **state-mutation**.

Piège

Versioned n'est pas Stable. Un état jamais écrit se lit Versioned, pas Stable (#41) : une dep sur lui ne peut pas être omise, ce qui est aussi la position d'eslint. Et le fait qu'une dep Versioned « garde » un effet dans infinite-loop n'est sound qu'*avec* le bras churn : retirer l'un sans l'autre rouvre le faux négatif d'ObjChurn. L'ADR-017 appelle ce couplage « a load-bearing soundness dependency ».

2.5 Les règles des hooks et l'identité positionnelle

2.5.1 Un hook est identifié par sa position

Rien, dans `const [s] = useState(0)`, ne dit à React *quel* état rendre : ni nom, ni clé. React s'en remet à l'ordre des appels. Au k -ième appel de hook d'un rendu, il rend le k -ième état de sa liste. Cet ordre doit donc être le même à chaque rendu.

Rappel React : règles des hooks

« Don't call Hooks inside loops, conditions, nested functions, or try/catch/finally blocks. Instead, always use Hooks at the top level of your React function, before any early returns. » [Met26d] React-tRace donne la raison : « each Hook call is identified by its position in a linked list of internal bookkeeping objects » [LAY25].

Exemple 2.7: Cond

Extrait 2.24 – Cond, transcrit de React-tRace (/tmp/ch02/cond.tsx)

```
1 import { useState } from "react";
2
3 export function Cond() {
4   const [b, setB] = useState(false);
5   if (b) {
6     const [s] = useState(0);
7     return <p>{s}</p>;
8   }
9   return <button onClick={() => setB((v) => !v)}>Show</button>;
10 }
```

```
Cond (3 hooks) cond.tsx
  error conditional-hook [hook:1] (line 6:10) this hook is called conditionally (not on every render path)
    → guarded by a condition evaluated here, so some render paths skip the hook (line 5:6)
```

Au premier rendu, `b` est faux, un seul hook est appelé. Après le clic, `b` est vrai et un deuxième `useState` apparaît, que React n'a jamais vu ; dans les mots de l'article, « instead of displaying the initial state of `s` which is 0, the UI breaks down at runtime ». L'analyseur le signale en **ERROR** : le défaut se produit dès que le code s'exécute sur l'autre branche, et l'appel conditionnel est

prouvé.

2.5.2 Ce que fait le code

Le label d'un hook est un entier (`src/ir/types.rs`, l. 2 : `pub type HookLabel = usize;`), attribué par le lowering dans l'ordre des appels du corps. C'est l'exact analogue du « ℓ » que React-tRace attache à chaque `useState` ; les handlers extraits partagent cet espace de labels (d'où `[hook:1]` pour le `useState` conditionnel de `Cond`, et « 3 hooks » en comptant le handler du bouton). L'IR range chaque hook dans une variante de `HookEntry`, dont voici les deux premières :

Extrait 2.25 – `src/ir/hooks.rs`, l. 247–259 — les deux premières variantes de `HookEntry`

```
247 #[derive(Debug, Clone)]
248 pub enum HookEntry {
249     State {
250         label: HookLabel,
251         init: Expr,
252         span: Option<SourceRange>,
253     },
254     Effect {
255         label: HookLabel,
256         body_cfg: CFG,
257         deps: DepsArg,
258         span: Option<SourceRange>,
259     },
```

Variante	Hooks React correspondants	Champs propres
State	<code>useState</code> , <code>useReducer</code> (réducteur jeté)	<code>init</code>
Effect	<code>useEffect</code> , <code>useLayoutEffect</code> , <code>useInsertionEffect</code>	<code>body_cfg</code> , <code>deps</code>
Memo	<code>useMemo</code>	<code>body_cfg</code> , <code>deps</code>
Callback	<code>useCallback</code>	<code>body_cfg</code> , <code>params</code> , <code>deps</code>
Ref	<code>useRef</code>	<code>init</code>
Custom	tout autre <code>use*</code> : hook du projet, de bibliothèque, ou hook React non modélisé (<code>useContext</code> ...)	<code>name</code> , <code>args</code> , <code>binding</code> , <code>provenance</code>
Handler	aucun : handler JSX <code>onX</code> ou callback d' <code>addEventListener</code>	<code>event</code> , <code>body_cfg</code>

TABLE 2.1 – Les variantes de `HookEntry` (`src/ir/hooks.rs`, l. 247–313) et les hooks React qu'elles représentent. Toutes portent un `label` et un `span` ; le classement est fait par `make_hook_entry` (`src/lowering/hook_extractor.rs`, l. 642–747).

La tableau 2.1 donne la carte complète ; le détail des champs est au chapitre 6. La règle `conditional-hook` traduit la règle de React en termes de CFG :

Extrait 2.26 – `src/rules/impls/conditional_hook.rs`, l. 3–8 — `conditional-hook` : un hook conditionnel ne domine pas toutes les sorties

```
3 /// Fires when a hook is called inside a conditional branch.
4 ///
5 /// Detection: hook at block H is conditional iff H does NOT dominate at least
6 /// one exit (Return-terminated) block. A hook dominates all exits  $\iff$  every
7 /// path from entry to every return must pass through H  $\iff$  unconditional call.
8 pub struct ConditionalHook;
```

« Appelé à chaque rendu » devient « le bloc du hook domine toutes les sorties du rendu » : tout chemin de l'entrée à un `return` passe par lui. Dans `Cond`, le bloc `then` ne domine pas le

return du bouton. La dominance est définie au chapitre 3 (définition 3.9) ; c'est un fait sur *tous* les chemins, donc un fait must, qui peut porter une ERROR.

Décision : ADR-025 — la fin d'un corps est un return

Un corps qui « tombe » à la fin sans return renvoie undefined en JavaScript. Le lowering scellait autrefois ce bloc par Unreachable, ce qui, après inlining, coupait certains composants de leur propre sortie : 208 composants du corpus avaient ainsi un CFG de rendu « sectionné », et la dominance des sorties se calculait sur moins de chemins que le programme n'en a. *Décision* (ADR-25) : la fin de corps est Return(undefined) ; Unreachable ne veut plus dire que « le contrôle s'arrête » (un throw) ; un return inatteignable n'est pas une sortie. *Alternative refusée* : raccorder tout Unreachable d'un callee à la jonction, « just an over-approximation ». Mesurée, elle inventait un chemin qui atteint la sortie sans passer par les hooks du callee et signalait en ERROR l'idiome conforme « guard-throw » (if (ctx === undefined) throw ... puis useState). En React, un throw pendant le rendu *abandonne* le rendu : aucun chemin ne continue vers les hooks suivants.

2.5.3 Appeler un setter pendant le rendu

La seconde règle de pureté concerne le setter. L'appeler pendant le rendu est autorisé, mais à une condition précise.

Rappel React : set pendant le rendu

« Calling the set function *during rendering* is only allowed from within the currently rendering component. React will discard its output and immediately attempt to render it again with the new state. » Un appel inconditionnel donne l'erreur « Too many re-renders. React limits the number of renders to prevent an infinite loop. » [Met26b] React-tRace (§3.1.2) : « an unconditional top-level call to a setter function causes an infinite loop and is never correct », même quand la valeur écrite est toujours la même ; le runtime lève une exception après 25 tentatives.

Exemple 2.8: Inf2

Extrait 2.27 – Inf2 : l'updater identité, appelé dans le corps (/tmp/ch02/setter_in_render.tsx)

```
1 import { useState } from "react";
2
3 export function Inf2() {
4   const [s, setS] = useState(0);
5   setS((x) => x);
6   return <div>{s}</div>;
7 }
```

```
Inf2 (1 hooks) setter_in_render.tsx
error setter-in-render [hook:0] (line 5:2) setter `setS` called directly in the render body, move this call
↳ into a useEffect or an event handler
→ `setS` is a state setter, so calling it writes state (line 5:2)
```

L'appel est dans le bloc d'entrée, qui domine toutes les sorties : ERROR. L'exemple est instructif pour le bail-out : l'updater identité ne change rien, et pourtant React boucle, parce qu'un set pendant le rendu fait *jeter* le rendu avant toute comparaison (section 2.9).

La documentation de React prévoit pourtant un usage légitime : « ajuster l'état pendant le rendu » quand une prop change, à la place d'un effet (page « You Might Not Need an Effect »).

Extrait 2.28 – List : le motif « adjust state during render » (/tmp/ch02/adjust.tsx)

```

1 import { useState } from "react";
2
3 export function List({ items }: { items: string[] }) {
4   const [prevItems, setPrevItems] = useState(items);
5   const [selection, setSelection] = useState<string | null>(null);
6   if (items !== prevItems) {
7     setPrevItems(items);
8     setSelection(null);
9   }
10  return <ul onClick={() => setSelection(items[0])}>{selection}</ul>;
11 }

```

List (3 hooks) adjust.tsx

warn setter-in-render [hook:1] (line 8:4) setter `setSelection` called directly in the render body, move this
 ↳ call into a useEffect or an event handler

La règle fait trois choses différentes, décrites par sa documentation :

Extrait 2.29 – src/rules/impls/setter_in_render.rs, l. 18–36 — les niveaux de setter-in-render

```

18 /// Fires when a state setter is called directly in the render body either a
19 /// local setter (`useState`) or a parent-component setter passed as a prop.
20 ///
21 /// Rule names emitted in diagnostics:
22 /// - `setter-in-render` local `useState` setter called in render.
23 /// - `cross-setter-in-render` parent's setter (prop) called in render.
24 ///
25 /// Severity:
26 /// - `Error` call block dominates all render exits (unconditional).
27 /// - `Warning` conditional path or nested FnLit (dominance unknowable).
28 ///
29 /// Silence: the sanctioned "adjust state during render" idiom – a
30 /// **conditional local** call whose dominating guards read the slot it
31 /// writes, such that once the written value sits in the slot the guard is
32 /// dead (`if (changed && open) setOpen(false)`). React explicitly allows
33 /// this shape (one convergent extra render, no loop); warning on it is
34 /// noise. Cross-component calls are never silenced – setting another
35 /// component's state during render is a runtime error regardless of guards.
36 pub struct SetterInRender;

```

`setPrevItems(items)` est tu : la garde qui le domine (`items !== prevItems`) lit le slot qu'il écrit, et devient fausse dès que la valeur écrite y est (`converges_once_written`, `src/engine/guards.rs`, l. 60–67). `setSelection(null)` est signalé en `WARNING` alors qu'il fait partie du même motif documenté : sa garde ne lit pas `selection`, et la règle ne peut pas prouver que l'appel converge. C'est un faux positif de niveau `WARNING`, que la doctrine tolère.

2.6 Strict Mode : le double rendu

En développement, React peut exécuter volontairement deux fois une partie du code pour faire apparaître les impuretés.

Rappel React : `<StrictMode>`

Strict Mode active des comportements *de développement* : « Your components will re-render an extra time to find bugs caused by impure rendering », « Your components will re-run Effects an extra time to find bugs caused by missing Effect cleanup », et de même pour les

ref callbacks. Sont appelés deux fois : « Your component function body (only top-level logic, so this doesn't include code inside event handlers) » et les fonctions passées à `useState`, aux fonctions `set`, à `useMemo` et à `useReducer`. Pour les effets, « React will run one extra development-only setup+cleanup cycle before the first real setup ». « All of these checks are development-only and do not impact the production build. » [Met26b]

Un composant impur montre la différence. Celui-ci compte ses rendus dans une variable de module :

Extrait 2.30 – Impure : le rendu modifie une variable de module (/tmp/ch02/strict.tsx)

```
1 import { useState } from "react";
2
3 let renders = 0;
4
5 export function Impure() {
6   const [n, setN] = useState(0);
7   renders = renders + 1;
8   return <button onClick={() => setN(n + 1)}>{renders}</button>;
9 }
```

Sous Strict Mode, le compteur avance de deux par rendu en développement et de un en production : c'est exactement ce que le double appel veut révéler. `reactant-analyzer` ne dit rien de ce composant :

```
Impure (2 hooks) strict.tsx ✓
verified conditional-hook all hooks run unconditionally, in a stable order
verified lazy-init no useState/useRef initializer re-runs work on every render
verified setter-in-render no setter is called during render
verified state-mutation no state or prop object is mutated in place
```

Aucune règle du catalogue ne porte sur l'écriture d'une variable de module pendant le rendu ; `state-mutation` ne vise que les objets d'état et de props, comme sa ligne `verified` le dit.

Ce que fait le code. Rien : Strict Mode n'est pas modélisé par le moteur. La seule trace est documentaire, dans `missing-cleanup`, qui rappelle qu'« under StrictMode it runs it twice on the first mount » (`src/rules/impls/missing_cleanup.rs`, l. 9–13) et reprend le fait dans son message (liste 2.14).

Ce silence est-il sound ? L'argument suivant n'est écrit nulle part dans le code ; c'est une déduction. Le point fixe calcule un ensemble d'états atteignables et joint les écritures. Exécuter deux fois un rendu, un initialiseur ou un updater *purs* produit deux fois les mêmes écritures, et $x \sqcup x = x$: l'ensemble calculé est inchangé. Pour les effets, le cycle supplémentaire `setup-cleanup` n'ajoute que des exécutions d'un corps que le point fixe exécute déjà à chaque itération. Strict Mode ne rend donc visible que ce qui viole la pureté, c'est-à-dire des hypothèses que le modèle abstrait fait déjà (section 2.1).

Piège

Le double rendu de Strict Mode n'est pas un rendu « de trop » au sens des règles. Un contributeur qui voudrait le modéliser comme deux tours ferait tirer `unnecessary-rerender` ou compterait deux fois une écriture de montage ; or ces règles parlent de la production, où le double appel n'existe pas. Inversement, un utilisateur qui voit son effet de montage s'exécuter deux fois en développement ne doit pas en conclure qu'il boucle, ni que `reactant-analyzer` se trompe en le traitant comme un effet de montage : l'effet est exécuté une fois de plus *par construction*, et la règle qui s'en préoccupe est `missing-cleanup`, pas `infinite-loop`.

2.7 Le contexte et la propagation du rendu

2.7.1 Rendre un parent, c'est rendre ses enfants

Un rendu est récursif : rendre un composant rend les composants qu'il renvoie (section 2.1). Un set dans un parent re-rend donc tout son sous-arbre, sauf les parties que React peut prouver inchangées (un élément reçu en children dont l'identité est conservée, un composant enveloppé par memo dont les props sont égales). Le contexte est l'autre canal de propagation : il traverse l'arbre sans passer par les props.

Rappel React : useContext

« useContext returns the context value for the calling component. It is determined as the value passed to the closest SomeContext above the calling component in the tree. » À défaut de provider, c'est la defaultValue de createContext. « React automatically re-renders all the children that use a particular context starting from the provider that receives a different value. The previous and the next values are compared with the Object.is comparison. Skipping re-renders with memo does not prevent the children receiving fresh context values. » D'où le conseil d'envelopper la valeur fournie dans useMemo et les fonctions dans useCallback [Met26b].

Extrait 2.31 – Un provider dont la valeur est neuve à chaque rendu (/tmp/ch02/context.tsx)

```
1 import { createContext, useContext, useState } from "react";
2
3 const ThemeCtx = createContext({ dark: false, toggle: () => {} });
4
5 export function ThemeProvider({ children }: { children: any }) {
6   const [dark, setDark] = useState(false);
7   return (
8     <ThemeCtx.Provider value={{ dark, toggle: () => setDark((d) => !d) }}>
9       {children}
10    </ThemeCtx.Provider>
11  );
12 }
13
14 export function Button() {
15   const { dark } = useContext(ThemeCtx);
16   return <button>{dark ? "dark" : "light"}</button>;
17 }
```

```
Button (1 hooks) context.tsx
info analysis-limit [hook:0] (line 15:8) hook `useContext` was not found in the registry. Pass its source file
  ↳ or add a HookSummary to analyse it (FN possible)
suspended analysis-limit 1 passing check(s) withheld: the analysis was truncated in this component, so they are not
  ↳ guaranteed

ThemeProvider (1 hooks) context.tsx
info analysis-limit component `ThemeCtx.Provider` was not found in the analysis registry. Pass its file on the
  ↳ command line to analyse it (FN possible)
warn unstable-context-value (line 8:4) `ThemeCtx.Provider` is given a newly allocated value on every render. `
  ↳ Object.is` fails for every consumer, so each `useContext(ThemeCtx)` re-renders whenever this component does,
  ↳ even when nothing in the value changed; wrap the value in `useMemo`
suspended analysis-limit 4 passing check(s) withheld: the analysis was truncated in this component, so they are not
  ↳ guaranteed
```

Les deux côtés du contexte sont traités très différemment.

Côté consommateur : T. useContext n'est pas modélisé. Le lowering le range, comme tout

hook React sans modèle, dans une entrée Custom dont la provenance garde le spécificateur react (commentaire de `src/lowering/hook_extractor.rs`, l. 730–732) :

Extrait 2.32 – `src/lowering/hook_extractor.rs`, l. 733–744 — un hook React non modélisé devient une entrée Custom

```

733         let args: Vec<Expr> = it.collect();
734         Some(HookEntry::Custom {
735             label,
736             name: name.to_string(),
737             args,
738             deps: DepsArg::Absent,
739             binding: None,
740             import_source: call.import_source(),
741             resolved_file: None,
742             span,
743         })
744     }

```

Son site d'appel est un marqueur `HookMarker(_, Unknown)`, qui s'évalue en $\top$; c'est la l. 143 de `src/domains/transfer/state_value.rs`, déjà citée au section 2.4. Le consommateur lit donc « n'importe quelle valeur », ce qui est sound, et l'INFO analysis-limit rend le trou visible ; les vérifications qui auraient réussi sont « suspendues » plutôt qu'affichées comme prouvées. C'est, d'après `docs/limitations.md`, la première source de limites d'analyse du corpus (363 sites, #28).

Côté provider : l'identité. La valeur passée à `value` est un littéral objet, donc `PerRender` : fait must. La règle `unstable-context-value` en tire un WARNING :

Extrait 2.33 – `src/rules/impls/unstable_context_value.rs`, l. 4–13 — le défaut visé par `unstable-context-value`

```

4  /// Fires when a context provider hands its consumers a brand-new object on
5  /// every render.
6  ///
7  /// `useContext` re-renders a consumer whenever the provided value changes
8  /// identity, and React compares with `Object.is`. A value allocated in the
9  /// provider's own render body – `value={{ a, b }}`, or a `const v = { a, b }`
10 /// that is never memoized – is a different object every time, so *every*
11 /// consumer re-renders whenever the providing component does, however deep the
12 /// tree and however little actually changed. The fix is `useMemo` on the value
13 /// (or lifting it out of render when it is constant).

```

WARNING et non ERROR : la référence fraîche est certaine, mais son coût (des rendus de consommateurs) ne l'est pas — c'est la seconde moitié de la définition du WARNING (« fait certain au coût incertain »). Au-delà de cette règle, la relation `context_consumers` (ADR-32) et la dépendance de rendu (ADR-41) suivent un contexte prouvé de son provider jusqu'aux consommateurs placés sous lui, pour les règles de cascade `state-lifted-too-high` et `wasted-subtree-render` (chapitre 16). Un élément que l'analyse ne peut pas ouvrir, en particulier un composant enveloppé par `memo` ou `forwardRef` (#64), y est compté comme un usage possible.

Remarque

La première ligne info de `ThemeProvider` signale que `ThemeCtx.Provider` n'est pas un composant du registre : l'analyseur voit une balise JSX dont il ne connaît pas le corps, et le dit. Le provider n'en est pas moins reconnu par la règle, qui repère `ThemeCtx` comme un `createContext(...)` de niveau module.

2.8 Server Components et la directive "use client"

Tout ce qui précède suppose que le composant vit dans le navigateur, où il a un état et des effets. Les Server Components (RSC), popularisés par l'App Router de Next.js, rompent cette hypothèse.

Rappel React : Server et Client Components

Un Server Component est rendu une fois, sur le serveur, et ne persiste pas : « Server Components cannot use most Hooks. When Server Components are rendered, their output is essentially a list of components for the client to render. Server Components do not persist in memory after render and cannot have their own state. » La frontière est une directive de module : « Add 'use client' at the top of a file to mark the module and its transitive dependencies as client code », et elle « must be at the very beginning of a file, above any imports or other code ». Il n'y a pas de directive pour le côté serveur, qui est le défaut ; et « a single module may be evaluated on the server when imported from server code and on the client when imported from client code » [Met26b].

La dernière phrase est la difficulté : être « serveur » n'est pas une propriété d'un fichier, mais de la façon dont on l'atteint dans le graphe d'imports. Considérons un projet Next minimal (/tmp/ch02next/) : un next.config.js vide, une page app/page.tsx sans directive, et un composant components/like-button.tsx qui commence par "use client".

Extrait 2.34 – app/page.tsx : un état dans un Server Component (/tmp/ch02next/app/page.tsx)

```
1 import { useState } from "react";
2 import { LikeButton } from "../components/like-button";
3
4 export default function Page() {
5   const [open, setOpen] = useState(false);
6   return (
7     <main onClick={() => setOpen(!open)}>
8       <LikeButton />
9     </main>
10  );
11 }
```

```
$ cd /tmp/ch02next && reactant check --no-color --info --show-clean .
```

```
[warn] no tsconfig `paths` found, so aliased imports (e.g. `@/...`) stay unresolved and their targets are NOT analyzed (
  ↳ possible false negatives). Aliases declared only in next.config are not read.
LikeButton (2 hooks) components/like-button.tsx ✓
  verified conditional-hook all hooks run unconditionally, in a stable order
  verified lazy-init no useState/useRef initializer re-runs work on every render
  verified setter-in-render no setter is called during render
  verified state-mutation no state or prop object is mutated in place
Page (2 hooks) app/page.tsx
  warn server-component-hook [hook:0] (line 5:8) `useState` is called in a Server Component. this file is an App
  ↳ Router `page` and no `use client` directive covers it, so React renders it on the server, where hooks do not
  ↳ exist; add `use client` at the top of the file, or move the stateful part into a child component that declares
  ↳ it
```

Le projet est reconnu comme Next.js par son next.config.js. La page est une entrée de l'App Router, aucune directive ne la couvre : c'est un module serveur, et son useState est un défaut. LikeButton, sous "use client", est un composant client ordinaire.

Décision : ADR-026 — analyser les Server Components quand même

Décision (ADR-26). Chaque module garde ses faits bruts (`ModuleFacts directives, imports`); un module est serveur s'il est atteignable depuis une entrée de l'App Router (`page, layout, template, default, not-found, loading`) sans franchir de "use client"; la règle `server-component-hook` y signale les hooks, au niveau `WARNING`; elle se tait si le programme n'utilise jamais "use client". *Alternatives refusées*. Sauter les modules serveur : décider « serveur, on passe » sur un graphe d'imports incomplet ferait disparaître leurs défauts, un faux négatif. Une preuve `must` (`Certified`) pour la règle : « would dress a path convention as a domain proof », d'où le plafond `WARNING` : les deux faits porteurs (une convention de noms de fichiers, un graphe dont le résolveur a pu manquer des arêtes) sont hors du domaine abstrait. Un seul diagnostic par composant, car le défaut est la directive manquante du module. Supprimer les autres diagnostics dans un module serveur : toute erreur de classification deviendrait un faux négatif. Analyser un Server Component ne coûte rien en soundness : il n'a pas d'état, et l'interprétation abstraite le traite comme un composant dont les props sont $\top$.

La règle sous-déclare quand un import sur le chemin d'un module serveur n'est pas résolu (#29), et le premier avertissement de la sortie rappelle justement que des imports aliés peuvent rester non résolus.

2.9 React-tRace : une sémantique formelle des hooks

Les sections précédentes décrivent React à partir de sa documentation, qui est informelle : elle ne dit pas, par exemple, *quand exactement* la file d'updaters est rejouée (« sometime during the next render », relèvent les auteurs). Un analyseur par interprétation abstraite a besoin d'une sémantique concrète précise à sur-approximer. React-tRace, de Jay Lee, Joongwon Ahn et Kwangkeun Yi (OOPSLA 2025) [LAY25], en fournit une pour le cœur des hooks : `useState` et `useEffect`. Cette section en résume les éléments dont le projet se sert ; les notations sont celles de l'article.

2.9.1 Le langage

L'article travaille sur un petit langage applicatif, découplé de JavaScript (« we decouple ourselves from the subtleties of the complex semantics of JS »). Sa syntaxe (Fig. 1 de l'article) :

$$\begin{aligned}
 P &::= D \ e & D &::= \text{let } C(x) = e \\
 e &::= () \mid \text{true} \mid \text{false} \mid n \mid x \mid C \mid e \oplus e \mid [e] \mid \text{print } e \mid \text{if } e \text{ then } e \text{ else } e \mid e; e \\
 &\quad \mid \text{fun } x \rightarrow e \mid e \ e \mid \text{let } x = e \text{ in } e \mid \text{let } (x, x_{\text{set}}) = \text{useState}_{\ell} \ e \text{ in } e \mid \text{useEffect } e
 \end{aligned}$$

Un programme est une suite de définitions de composants suivie d'une expression principale. `[e]` est une vue composée (l'analogue du JSX imbriqué) ; une fermeture utilisée comme vue représente un handler (un bouton *est* son handler). Les hooks ne peuvent apparaître qu'au niveau supérieur d'un corps de composant, ce que l'interpréteur vérifie à l'analyse syntaxique ; chaque `useState` porte un label unique ℓ . L'effet n'a pas de deps : il s'exécute après chaque rendu, les deps étant « a straightforward extension ».

2.9.2 Les objets sémantiques

Les valeurs de base sont les constantes, les fermetures $\langle \lambda x.e, \sigma \rangle$ et un tampon de sortie ω . La machinerie React ajoute :

- la *spécification de composant* $\langle C, v \rangle$ (un nom et son argument : appliquer un composant ne fait que l’empaqueter) ;
- le *setter* $\langle \ell, p \rangle$: le label du `useState` d’origine et le chemin p de la vue qui le possède ;
- la *mémoire d’arbre* $m = [p \mapsto \pi]$, qui associe à chaque chemin une *vue* $\pi = \{spec, dec, sttst, effq, child\}$: sa spécification, un ensemble de *décisions* $d \in \{\text{Check}, \text{Effect}\}$, un store d’état $\rho = [\ell \mapsto \{val : v, sttq : q\}]$ (valeur courante et file d’updaters), une file d’effets et un arbre d’enfants ;
- une *phase* d’évaluation $\phi \in \{\text{Init}, \text{Succ}, \text{Normal}\}$: premier appel d’un corps, appels suivants, ou expression principale, effets et handlers ;
- un *mode* du moteur : *rendered* (l’interface est à l’écran, des effets attendent), *check* (il faut vérifier si un rendu est nécessaire) ou *event loop* (on attend une entrée).

Les décisions sont les deux mots de la citation du section 2.3 : **Check** marque une vue à re-vérifier (un set l’a visée), **Effect** une vue dont les effets doivent tourner après ce rendu.

2.9.3 La boucle de rendu

Le moteur est un système de transitions à quatre règles (Fig. 4 de l’article). En omettant le tampon de sortie et la table des définitions δ , qui ne change pas :

$$\begin{array}{lcl}
\text{STEPINIT} & \frac{[], [] \vdash e \Downarrow^{\text{Normal}} s \quad [] \vdash \text{init}(s) = \langle t, m \rangle}{\langle e \rangle \hookrightarrow \langle t, m, \text{rendered} \rangle} \\
\text{STPEFFECT} & \frac{m \vdash \text{commitEffe}(t) = m'}{\langle t, m, \text{rendered} \rangle \hookrightarrow \langle t, m', \text{check} \rangle} \\
\text{STEPCHECK} & \frac{m \vdash \text{check}(t) = \langle \mu, m' \rangle}{\langle t, m, \text{check} \rangle \hookrightarrow \langle t, m', \mu \rangle} \\
\text{STPEVENT} & \frac{\langle \lambda x.e, \sigma \rangle \in \text{handlers}(m, t) \quad m, \sigma[x \mapsto ()] \vdash e \Downarrow^{\text{Normal}} v, m'}{\langle t, m, \text{event loop} \rangle \hookrightarrow \langle t, m', \text{check} \rangle}
\end{array}$$

avec $\mu \in \{\text{rendered}, \text{event loop}\}$. La figure 2.4 en donne l’automate. On y lit la figure 2.1 sous forme exacte : STEPINIT est le montage ; STPEFFECT exécute les effets en file ; STEPCHECK re-rend les vues marquées **Check** et revient en mode *rendered* si un état a changé, en *event loop* sinon ; STPEVENT exécute un handler, ce qui impose une nouvelle vérification.

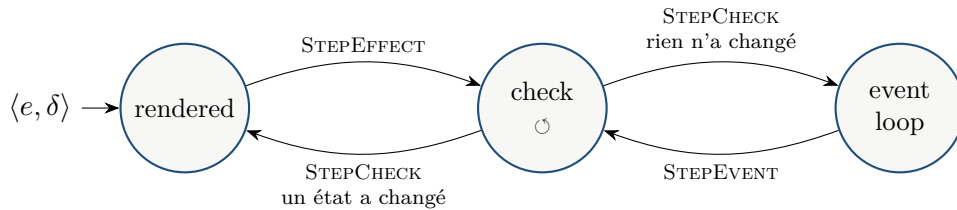


FIGURE 2.4 – Les modes de React-tRace et ses quatre transitions. La flèche d’entrée est STEPINIT (évaluation de l’expression principale et initialisation de l’arbre). La boucle infinie d’effets est le cycle $\text{rendered} \rightarrow \text{check} \rightarrow \text{rendered}$ qui ne passe jamais par *event loop* ; un handler, lui, n’est exécuté qu’en mode *event loop*.

2.9.4 Les règles d’évaluation

Sous la boucle, l’évaluation d’une expression $\Sigma, \sigma \vdash e \Downarrow_p^\phi v, \Sigma', \omega$ est une sémantique à grands pas. Les règles propres à React sont les suivantes (Fig. 5 à 10 de l’article).

- SttBind** (phase Init) : `useState` évalue son argument v_1 , crée l'entrée $\ell \mapsto \{val : v_1, sttq : []\}$ et lie x_{set} au setter $\langle \ell, p \rangle$.
- SttReBind** (phase Succ) : `useState` rejoue la file $[\langle \lambda x_i.e_i, \sigma_i \rangle]_{i=1}^n$ dans l'ordre, $v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_n$, vide la file, stocke v_n , et ajoute la décision **Effect** si $v_n \neq v_0$.
- AppSetComp** (set pendant le corps, phases Init et Succ) : le chemin du setter doit être celui de la vue courante ; l'updater est ajouté à la file et la décision **Check** posée.
- AppSetNormal** (set dans un effet ou un handler, phase Normal) : l'updater est ajouté à la file de la vue $m[p]$ désignée par le setter, quelle qu'elle soit, et **Check** y est posée. C'est la seule règle qui voie toute la mémoire : un enfant peut mettre à jour son parent.
- Eff** : `useEffect` e ajoute la fermeture $\langle \lambda _ . e, \sigma \rangle$ à la file d'effets de la vue — l'effet capture l'environnement *du rendu*.
- EvalOnce** / **EvalMult** : un corps est évalué, puis ré-évalué en phase Succ tant qu'il pose **Check** (file d'effets vidée à chaque essai) ; React lève une exception après 25 essais.
- CommitEfsPath** / **CommitEfsPathIdle** : les effets d'une vue ne sont exécutés que si sa décision contient **Effect**, enfants d'abord (post-ordre), puis dans l'ordre syntaxique.
- CheckIdle** / **CheckNoEffect** / **CheckEffect** : une vue sans **Check** est sautée ; avec **Check**, son corps est ré-évalué ; si la nouvelle décision ne contient pas **Effect** (l'état est revenu au même), rien n'est rendu ; sinon l'enfant est réconcilié et le mode redevient *rendered*.
- ReconcileComEffect** / **ReconcileComNew** : un enfant du même composant est ré-évalué en Succ ; un composant différent à la même place est initialisé à neuf.

Exemple 2.9: SelfCounter en React-tRace

Au montage, **INITCOM** évalue le corps en phase Init (**STTBIND** : $s = 0$; **EFF** met l'effet en file) et pose **Effect** : mode *rendered*. **STPEFFECT** exécute l'effet en phase Normal : $s < 3$, donc **APPSETNORMAL** met $\lambda s. s + 1$ en file et pose **Check** ; mode *check*. **STEPCHECK** ré-évalue le corps : **STTREBIND** rejoue la file, $0 \rightarrow 1$, $1 \neq 0$ donc **Effect** : **CHECKEFFECT**, mode *rendered*. Le cycle se répète jusqu'à $s = 3$; alors l'effet tourne mais ne pose pas **Check**, et **STEPCHECK** (**CHECKIDLE**) mène en *event loop*.

2.9.5 Les théorèmes

L'article prouve deux propriétés que la documentation de React énonce informellement (§6.1).

Théorème 2.1: Ré-évaluation par appel d'un setter (Theorem 1)

La dérivation de l'évaluation retentée d'un corps contient plusieurs évaluations de ce corps si et seulement si **APPSETCOMP** apparaît dans la dérivation de la première évaluation.

Théorème 2.2: Condition d'exécution des effets (Theorem 2)

Si $\langle t, m, \omega, \delta, \text{check} \rangle \hookrightarrow \langle t, m', \omega', \delta, \mu \rangle$, alors pour tout chemin p atteignable depuis la racine dans m et m' , **COMMITEFFSPATH** en p apparaît dans la dérivation de la transition suivante si et seulement s'il existe un ancêtre p' de p tel que (1) pour un ℓ du store de $m'[p']$, $m[p'].sttst[\ell].val \neq m'[p'].sttst[\ell].val$; ou (2) la dérivation de la transition contient **APPSETCOMP** de chemin p' .

Le premier théorème est le mécanisme d'Inf2 : un set dans le corps fait jeter l'évaluation et recommencer, indépendamment de la valeur écrite. Le second dit quand les effets tournent : après un changement d'état de la vue *ou d'un de ses ancêtres* (c'est la cascade de rendu), ou si un set a eu lieu pendant le corps. Un troisième résultat (Theorem 8, appuyé sur le Lemma 7) concerne une optimisation de React, qui applique par avance les updaters en tête de file sans

ré-évaluer le corps : si ces updaters sont purs (Definition 3), l'état final est le même. Il justifie que la sémantique ignore cette optimisation. La validité d'une vue (Definition 6) repose enfin sur le fait que « each useState call in a component body is executed exactly once during initialization » — c'est ici que la règle des hooks entre dans la formalisation.

Conformité. La suite de tests couvre les 44 règles d'évaluation par 38 tests répartis en 18 scénarios (Table 1), rejoués sur React 16.14.0, 17.0.2, 18.3.1 et 19.1.0 ; une seule différence, due à l'optimisation ci-dessus (scénario S17, set depuis un événement utilisateur). L'interpréteur et le visualiseur sont publiés (Zeta611/react-trace).

Extensions esquissées. Le §7 de l'article esquisse useRef (un store de refs sans Check), useMemo (valeur et deps, recalcul en Succ si les deps différent), useContext (recherche ascendante du provider, Check des consommateurs quand la valeur change), useEffect (un mode supplémentaire) et les hooks personnalisés (de simples compositions). Aucune n'est formalisée : c'est précisément la part que reactant-analyzer doit ajouter sans garantie formelle.

2.10 La sémantique concrète du projet : ce qu'on garde, ce qu'on abstrait

2.10.1 Le contrat : ADR-001

Décision : ADR-001 — React-tRace comme sémantique concrète de référence

Contexte. Une interprétation abstraite dérive une sémantique abstraite $C^\#$ d'une sémantique concrète C ; sans C explicite, la soundness ne peut pas être établie et les fonctions de transfert « are written by guesswork ». *Décision* (ADR-1). React-tRace est la sémantique concrète C ; les fonctions de transfert en dérivent ; les extensions (deps, useMemo, useCallback, useRef, objets) sont spécifiées comme extensions de React-tRace. *Justification.* C'est la seule formalisation publique de React accompagnée d'une preuve de conformité ; la boucle STEPINIT $\rightarrow$ STEPEFFECT $\rightarrow$ STEP CHECK correspond directement à l'itération de point fixe ; STTREBIND, CHECKEFFECT et CHECKNOEFFECT définissent exactement les conditions de re-rendu. *Limites acceptées.* React-tRace ne couvre que useState et useEffect sans deps ; son langage n'est pas JavaScript ; les extensions n'ont pas de garantie formelle équivalente.

L'ADR annonce aussi trois conséquences que le dépôt, au commit e67b10a, ne réalise pas. Le fichier docs/semantics.md censé spécifier les extensions n'existe pas ; aucune fonction de src/domains/ ne cite de règle React-tRace (grep -rn "tRace" src ne trouve rien) ; aucun test ne compare l'analyseur à l'interpréteur de l'article sur ses exemples (aucun test ne mentionne Flicker, SelfCounter ou Inf2). React-tRace est donc la référence *conceptuelle* du projet — l'ADR-004 et l'ADR-012 s'y réfèrent, et ce chapitre en tire la table de correspondance —, pas un oracle outillé.

2.10.2 De la boucle de rendu à la boucle de point fixe

Décision : ADR-004 — un CFG de rendu, un CFG par effet

Chaque composant est représenté par un render_cfg et une liste de HookEntry, dont chaque effet porte son propre body_cfg (ADR-4). L'alternative refusée est un méta-CFG unique, rendu et effets reliés par des arêtes rendu $\rightarrow$ effet $\rightarrow$ check : un grand graphe difficile à maintenir, qui rend implicite la séparation rendu/effet et « risks incorrect modeling of

React's execution order ». Or la séparation est un invariant de React (« the effect never executes during the render »), et les défauts visés sont des propriétés de l'état au point fixe, pas des chemins d'un graphe unifié.

La boucle de `analyze_component_impl` (`src/engine/fixpoint.rs`, l. 130–698) joue abstraitement les tours de React. L'algorithme 1 en donne une lecture fidèle, réduite à ce qui concerne ce chapitre ; son étude complète est au chapitre 13.

Algorithme 1 : La boucle de point fixe d'un composant (lecture de `src/engine/fixpoint.rs`, l. 355–593, élaguée : garde-fou à 100 itérations, relations calculées à la fin)

Entrées : un composant : `render_cfg`, `hooks`, environnement initial (`props`)

Sorties : les stores convergés, `widen_trace`, `effect_setter_writes`

```

1  $S \leftarrow \perp$ ;
2 pour chaque  $\text{State}\{\ell, \text{init}\}$  faire
3    $S[\ell] \leftarrow S[\ell] \sqcup \llbracket \text{init} \rrbracket$  ;                                // SttBind, l. 314–353
4 tant que vrai faire
5    $(\text{env}, R) \leftarrow$  passe de rendu sur  $S$  ;                        // StepInit/Succ
6   recalculer les mémos depuis  $\text{env}$ ;
7    $E \leftarrow \sqcup_{\text{tout effet}}$  passe d'effet depuis  $\text{env}$  sur  $S$  ;        // StepEffect, deps ignorées
8    $H \leftarrow \sqcup_{\text{tout handler}}$  passe de handler depuis  $\text{env}$  sur  $S$  ;    // StepEvent, 0..N fois
9    $N \leftarrow R \sqcup E \sqcup H$  écritures des autres composants;
10  si  $N \sqsubseteq S$  alors
11     $\sqcup$  sortir ;                                                    // StepCheck : rien n'a changé
12   $i \leftarrow i + 1$ ;
13  si  $i \geq 3$  alors
14     $\text{widen\_trace} \sqcup =$  slots changés dans  $R \sqcup E$  (pas  $H$ );
15     $S \leftarrow S \nabla N$  (à seuils);
16  sinon
17     $\sqcup$   $S \leftarrow N$ ;
18 rejouer le rendu sur les stores convergés ; rejouer les effets depuis  $\perp \rightarrow$ 
    effect_setter_writes;
```

Une itération abstraite n'est pas un tour de React : elle en représente *tous* à la fois. Le store S de l'itération k sur-approxime les valeurs que l'état peut avoir après au plus k tours, de n'importe quel type (déclenché par un effet ou par un handler), et la boucle s'arrête quand un tour de plus n'ajoute rien — c'est la version abstraite de `CHECKNOEFFECT` partout. La tableau 2.2 fait la correspondance règle par règle.

2.10.3 Ce qu'on garde, ce qu'on abstrait

La tableau 2.3 résume les choix. L'argument de soundness qui les réunit tient en une phrase : chaque abstraction *ajoute* des comportements (plus d'effets exécutés, plus de rendus, plus de valeurs dans chaque slot), jamais n'en retire, et la mise à jour faible garantit que ce qu'un tour concret peut écrire est contenu dans le store. La définition précise de la soundness est au chapitre 3 (définition 3.8) ; la dernière ligne du tableau en marque la portée : ce qui est sur-approximé, c'est l'ensemble des états, pas la terminaison.

React-tRace	reactant-analyzer (commit e67b10a)
STTBIND (phase Init)	amorçage hors boucle, <code>src/engine/fixpoint.rs</code> , l. 314–353
APPSETCOMP, APPSETNORMAL (mise en file)	<code>exec_setter_call</code> : join immédiat, <code>src/domains/interp/interpreter.rs</code> , l. 341–366 ; setter d'un autre composant : <code>SharedStateStore</code> , l. 368–392
STTREBIND (rejeu de la file, $v_n \neq v_0$)	lecture <code>StateVal</code> (store joint, référence <code>Versioned</code>), <code>src/domains/transfer/state_value.rs</code> , l. 122–134 ; « a changé ? » = <code>test N ⊆ S</code> , <code>src/engine/fixpoint.rs</code> , l. 495
EFF, COMMITEFFSPATH STEPCHECK, CHECKNOEFFECT	passes d'effets, tous les effets, <code>src/engine/fixpoint.rs</code> , l. 415–449 test de convergence, <code>src/engine/fixpoint.rs</code> , l. 495
cycle Check–Effect infini (§3.1.1)	<code>widen_trace</code> , <code>src/engine/fixpoint.rs</code> , l. 514–527, et <code>effect_setter_writes</code> , l. 565–593 ; <code>churn</code> : lignes exact de <code>effect_triggers</code>
EVALMULT infini (§3.1.2) STPEVENT	<code>setter-in-render</code> , ERROR si l'appel domine toutes les sorties <code>HookEntry::Handler</code> , passes de handlers hors <code>widen_trace</code> , <code>src/engine/fixpoint.rs</code> , l. 451–483
setter $\langle \ell, p \rangle$	<code>Expr::StateSetter</code> s'évalue en <code>component_setter(composant, label)</code> , <code>src/domains/transfer/state_value.rs</code> , l. 135 (ADR-012 : « Corresponds to <code>Set_clos label, path</code> of React-tRace »)
APPCOM, <i>init</i> , <i>reconcile</i>	<code>eval_comp_app</code> : analyse descendante des enfants avec leurs props abstraites, <code>src/domains/transfer/state_value.rs</code> , l. 470–594

TABLE 2.2 – Table de correspondance entre les règles de React-tRace et les mécanismes de reactant-analyzer.

Fait de React	Ce que reactant-analyzer en fait	Pourquoi c'est sûr
Le rendu, les effets et les handlers s'exécutent à des moments distincts	gardé : un CFG par corps, trois sortes de passes	structure
Un effet capture l'environnement de son rendu	gardé : chaque effet part de l'environnement de sortie du rendu	exact
L'identité d'un hook est sa position	gardé : <code>label</code> = ordre d'appel	exact
Un setter est une valeur liée à un slot d'un composant	gardé : <code>component_setter</code> , <code>SharedStateStore</code>	exact
File d'updaters, ordre, batching	abstraits : join immédiat, <code>updater</code> sur le store	le join contient tout résultat de file
« l'effet tourne ssi une dep a changé »	abstrait : tous les effets à chaque itération ; deps relues par les règles	plus d'exécutions que React
Bail-out <code>Object.is</code>	abstrait : tout set peut re-rendre	plus de rendus que React
Ordre enfants-parent des effets	abstrait : join commutatif, effets croisés vus à l'itération suivante	ordre quelconque couvert
Nombre de clics	abstrait : handlers exécutés 0..N fois dans le point fixe	toute séquence couverte
DOM, réconciliation fine, Strict Mode	ignorés (un élément natif vaut <code>Stable</code>)	sans effet sur l'état
Durée d'une suite de rendus	non calculée	voir le piège de <code>Toggle</code>

TABLE 2.3 – Les faits de React et leur sort dans l'analyseur.

2.10.4 Les callbacks : dans le cycle ou hors du cycle

Un effet qui appelle un setter dans un `.then` ou un `setTimeout` relance un tour sans intervention extérieure; un effet qui s'abonne avec `addEventListener` ne le fait qu'à chaque événement. L'analyseur doit distinguer les deux, sinon tout compteur de clics serait une boucle infinie.

Extrait 2.35 – `src/domains/interp/callbacks.rs`, l. 6–23 — la classe de déclenchement d'un callee

```

6  /// How a call's closure arguments should be treated by the side-effect pre-pass.
7  #[derive(Debug, Clone, Copy, PartialEq, Eq)]
8  pub enum TriggerClass {
9      /// Callee is a bound state setter handled by the core exec path (functional
10     /// updaters), so the pre-pass must NOT descend its closure.
11     Setter,
12     /// Runs as a consequence of the current render/effect: synchronous HOFs
13     /// (`map`, `forEach`, ...) and scheduled async (`then`/`catch`/`finally`,
14     /// `setTimeout`/`setInterval`, `queueMicrotask`, `requestAnimationFrame`).
15     /// Its closure arguments ARE descended into.
16     InCycle,
17     /// Event subscription (`addEventListener`/`removeEventListener`) triggered
18     /// externally, NOT part of the render→effect→render cycle. Not descended.
19     Subscription,
20     /// Unrecognized callee (custom helper/hook). Conservatively NOT descended
21     /// (FP-averse: avoids flagging custom subscription wrappers).
22     Unknown,
23 }
```

La classe est décidée par le nom du callee (`classify_callee`, `src/domains/interp/callbacks.rs`, l. 28–56) : `setTimeout`, `setInterval`, `queueMicrotask`, `requestAnimationFrame`, `then/catch/finally` et les fonctions d'ordre supérieur synchrones (`map`, `forEach`...) sont `InCycle` et leurs arguments fonctionnels sont exécutés; `addEventListener` est `Subscription`; tout autre callee est `Unknown`.

Décision : ADR-009 — points d'entrée et classe de déclenchement

Décision (ADR-9). La descente dans les callbacks est *sémantique* (le store bouge), pas seulement structurelle; les handlers JSX et les abonnements sont des *points d'entrée* séparés, exécutés 0..N fois dans le point fixe, dont les écritures entrent dans le store mais pas dans `widen_trace` : « clicking 1000× isn't a bug ». *Alternative refusée*. Descendre uniformément dans tout callback aurait armé la valeur sur `addEventListener('click', () => setCount(c => c + 1))` dans un effet, un faux positif d'infinite-loop sur du React correct. *Choix explicite* : `Unknown` n'est pas descendu, « for a linter, false positives are more costly than false negatives [...] We accept the FN ».

Piège

Le choix « `Unknown` → skip » de l'ADR-009 est en tension avec l'invariant du projet, « faux négatifs INTERDITS ». Un setter appelé dans `myUtil(() => setX(x + 1))` ne fait pas bouger le store; un mécanisme ultérieur (l'inlining des fonctions locales connues) réduit ce trou sans le fermer. La relation des écrivains, elle, garde ces écritures avec une phase `T` (`SetterCallPhase::Unknown`), que `setter-in-render` signale en `WARNING`. Le store et la relation n'ont donc pas la même politique : « absent du store » ne veut pas dire « jamais écrit ».

Le moment d'une écriture, vu par la relation des écrivains (chapitre 14), est une phase :

Extrait 2.36 – `src/engine/setters.rs`, l. 57–70 — les phases d'un appel de setter

```

57 pub enum SetterCallPhase {
58     /// Runs when the enclosing body runs.
59     Sync,
```

```

60    /// Runs only on an external event – a registered DOM listener, a reified
61    /// JSX handler. A user event stands between the body and this write.
62    Handler,
63    /// A later tick of the same mount: a known deferring registrar took the
64    /// callback (`setTimeout`, `queueMicrotask`), or it is an effect cleanup.
65    /// Proven not to run in the enclosing body's own pass.
66    Deferred,
67    /// T – the callee that received the callback has no summary, so every
68    /// phase is possible, the enclosing body's own pass included.
69    Unknown,
70 }

```

Handler dit qu'un événement utilisateur s'interpose ; Deferred, qu'un tour ultérieur de la boucle d'événements s'interpose (minuterie, microtâche, cleanup). Depuis l'ADR-035 (ADR-35), une écriture placée après un `await` est aussi Deferred : le lowering coupe le bloc à l'`await` par une arête `EdgeKind::Await`, et la marche des setters promeut Sync en Deferred dans les blocs qui suivent. C'est la traduction exacte d'un fait du langage : après un `await`, on est sorti de toute phase de React.

2.10.5 Trois écarts vérifiés

L'exactitude impose de montrer aussi où la sémantique abstraite du commit étudié *ne* sur-approxime *pas* la sémantique concrète, ou la sur-approxime au mauvais niveau. Ce sont des défauts, pas des choix ; ils sont présentés ici parce que chacun se comprend par un fait React de ce chapitre.

Piège

L'updater fonctionnel sur le chemin programme (faux négatif).

Extrait 2.37 – D : un updater dans un effet gardé par l'état qu'il écrit (`/tmp/ch02/updater.tsx`, imports omis)

```

1  export function D() {
2    const [count, setCount] = useState(0);
3    useEffect(() => {
4      setCount((c) => c + 1);
5    }, [count]);
6    return <div>{count}</div>;
7  }

```

En React, c'est une boucle certaine. La CLI ne dit rien :

```

[verbose] D: 2 iteration(s), widened: []
✓ 1 file(s) no issues found.

```

Pourtant le test `functional_updater_with_state_dep_is_flagged` de `tests/functional_updater.rs`, qui analyse le même composant, passe (cargo test `-test functional_updater : 5 passed`). Le test appelle `analyze_component` (analyse intra, `inter = None`) ; la CLI passe par l'analyse de programme (`inter` présent). La seconde branche d'`exec_setter_call` (`src/domains/interp/interpreter.rs`, l. 368–392) s'applique alors à tout callee qui s'évalue en `setter`, y compris le `setter` du composant *lui-même*, et joint dans le `SharedStateStore` l'argument *évalué*, c'est-à-dire la fermeture `c => c + 1` (`PerRender`) et non la valeur qu'elle renvoie ; ce store est rejoint au test de convergence, le slot devient un mélange nombre-référence sur lequel `c + 1` vaut `⊤`, et plus rien ne croît.

Piège

useReducer (faux positif au niveau Error). Parce que le réducteur est jeté (section 2.2), `dispatch(action)` écrit l'action dans le slot :

Extrait 2.38 – R : un réducteur qui ignore l'action `noop` (`/tmp/ch02/reducer.tsx`, imports omis)

```
1 function reducer(s: { n: number }, action: { type: string }) {
2   if (action.type === "noop") return s;
3   return { n: s.n + 1 };
4 }
5
6 export function R() {
7   const [state, dispatch] = useReducer(reducer, { n: 0 });
8   useEffect(() => {
9     dispatch({ type: "noop" });
10  }, [state]);
11   return <div>{state.n}</div>;
12 }
```

```
R (2 hooks) reducer.tsx
error infinite-loop [hook:0] (line 10:2) this effect recreates object state `state` it depends on. Every run
  ↳ stores a fresh reference (`Object.is` always fails) and re-triggers itself: infinite render loop
  ↳ a fresh value is written to state `state` here [hook:1] (line 11:4)
```

En React, le réducteur renvoie `s` inchangé, `Object.is` conclut à l'égalité, pas de rendu : aucune boucle. L'analyseur voit un objet frais (l'action) écrit dans le slot dont l'effet dépend, et conclut au churn. C'est une ERROR fausse, ce que `docs/limitations.md` exclut (« A false positive never carries an Error »).

Piège

La boucle oscillante (faux négatif). C'est `Toggle` (exemple 2.5) : la valeur abstraite converge parce que l'ensemble des états est fini, et aucune preuve structurelle ne couvre une négation booléenne.

2.11 Les exemples canoniques du manuscrit

Les chapitres suivants reviennent sans cesse à une poignée de programmes. Ils sont définis dans ce chapitre (ou, pour `Timer`, dans l'introduction) et réunis dans la tableau 2.4, avec le comportement de React et le verdict de `reactant-analyzer` au commit `e67b10a`. Tous les fichiers sont sous `/tmp/ch02/` et les sorties ont été obtenues par `reactant check -no-color -info -show-clean -project plain <fichier>`.

Il manquait au chapitre la sortie de `Timer`. Le fichier `/tmp/ch02/timer.tsx` contient une variante du `Timer` de l'introduction (liste 1.3) : même structure, un état nommé `n` et un callback écrit en une ligne, `setInterval(() => setN(n + 1), 1000)`; et sa correction `TimerFixed`, qui passe l'updater `(m) => m + 1` :

```
Timer (2 hooks) timer.tsx
warn missing-deps [hook:1] var:n (line 5:2) `n` is used in this effect but not in its deps array, and it is
  ↳ recreated on every render
error stale-closure [hook:1] var:n (line 6:10) the `setInterval` callback registered by this mount-only effect
  ↳ reads `n` and writes it back. `n` was captured once at mount, so every firing recomputes from the same frozen
  ↳ value and the state can never advance past its first update
info widening-info (line 5:2) state `n` kept changing during analysis and was approximated to converge, so
  ↳ findings that depend on it may be imprecise
TimerFixed (2 hooks) timer.tsx ✓
verified stale-closure no long-lived callback captures a stale state value
```

Le callback de `setInterval` est `InCycle` (section 2.10) : son écriture entre dans le point fixe, d'où l'élargissement. Le fait React qui fonde l'`ERROR` est celui de la section 2.2 : `n` est une constante du rendu de montage, et l'effet, de `deps []`, ne tourne qu'une fois.

TABLE 2.4 – Les exemples canoniques. « Ex. » renvoie à l'encadré ou à l'extrait qui les définit.

Composant (fichier)	À l'exécution (React)	reactant-analyzer (e67b10a)
Counter (counter.tsx), exemple 2.1	compte les clics	aucun diagnostic
Batch (batching.tsx), liste 2.2	+2 par clic	aucun diagnostic
SameValue (samevalue.tsx), liste 2.3	bail-out, un seul rendu	WARNING redundant-set-state
Todos (lazy.tsx), liste 2.4	appel inutile à chaque rendu	WARNING lazy-init
EveryCommit, OnChange (deps.tsx)	boucle infinie	WARNING infinite-loop
MountOnly (deps.tsx)	deux rendus puis arrêt	WARNING missing-deps, INFO widening-info
Inf (selfcounter.tsx), exemple 2.3	boucle infinie	WARNING infinite-loop
SelfCounter (id.)	quatre rendus puis arrêt	INFO widening-info seulement
Flicker (flicker.tsx), exemple 2.4	un rendu de trop au montage	WARNING unnecessary-rerender
Toggle (toggle.tsx), exemple 2.5	boucle infinie (oscillation)	aucun diagnostic (FN)
Width (listener.tsx), liste 2.14	abonnements empilés	WARNING missing-cleanup
Search (search.tsx), liste 2.18	un effet repart à chaque rendu	WARNING always-unstable-deps
ObjChurn (objchurn.tsx), exemple 2.6	boucle infinie (identité)	ERROR infinite-loop
ObjStateDep (id.)	effet de montage seul	aucun diagnostic
Cond (cond.tsx), exemple 2.7	interface cassée après le clic	ERROR conditional-hook
Inf2 (setter_in_render.tsx), exemple 2.8	« Too many re-renders »	ERROR setter-in-render
List (adjust.tsx), liste 2.28	motif documenté, converge	WARNING setter-in-render sur setSelection (FP)
Impure (strict.tsx), liste 2.30	rendu impur (Strict Mode le révèle)	aucun diagnostic
ThemeProvider, Button (context.tsx)	consommateurs re-rendus	WARNING unstable-context-value, INFO analysis-limit
Page (app/page.tsx de /tmp/ch02next)	hook dans un Server Component	WARNING server-component-hook
Timer (timer.tsx), liste 1.3	chronomètre bloqué à 1	ERROR stale-closure, WARNING missing-deps
TimerFixed (id.)	correct	aucun diagnostic
D (updater.tsx), liste 2.37	boucle infinie	aucun diagnostic (FN)
R (reducer.tsx), liste 2.38	pas de boucle (bail-out)	Error infinite-loop (FP)

Résumé du chapitre

- Un composant est une fonction que React appelle autant de fois qu'il le veut. Un tour : déclencheur, rendu, commit, effets ; un set dans un effet relance un tour, un set dans un handler attend un événement, un set pendant le rendu fait jeter le rendu (figure 2.1).
- set planifie le prochain rendu ; les mises à jour sont mises en file et regroupées ; un updater lit la file ; Object.is fait sauter un rendu inutile. L'analyseur ne garde ni file ni ordre : une écriture est un join dans le StateStore, un updater est exécuté sur la valeur du store (exec_setter_call).
- Un effet tourne après le commit si une dep a changé selon Object.is. Le point fixe exécute tous les effets à chaque itération ; les deps sont relues par les règles et par la relation effect_triggers (bit exact : l'effet doit repartir).
- L'identité compte autant que la valeur : littéraux PerRender, lecture d'un état Versioned par son slot (double vue de l'état, ADR-017), mémos comme join des deps. ObjChurn est une ERROR par trois faits must, pas par divergence.
- Un hook est identifié par sa position (HookLabel) ; un hook conditionnel ne domine pas toutes les sorties ; la fin d'un corps est un return (ADR-025).
- Strict Mode n'est pas modélisé, le contexte côté consommateur est $\top$, les Server Components sont analysés quand même et un hook y est un WARNING (ADR-026).
- React-tRace fournit la sémantique concrète de référence (ADR-001) : mémoire d'arbre, décisions Check/Effect, quatre transitions STEPINIT, STEPEFFECT, STEP CHECK, STEP EVENT ; la tableau 2.2 la relie au code. Le contrat est conceptuel : ni docs/semantics.md, ni oracle de tests.
- Le point fixe sur-approxime l'ensemble des états, pas la durée d'une suite de rendus (Toggle) ; trois écarts vérifiés au commit étudié : Toggle et D (FN), R (ERROR fausse).

Fichiers rencontrés : src/engine/fixpoint.rs, src/domains/interp/interpreter.rs, src/domains/interp/callbacks.rs, src/domains/transfer/state_value.rs, src/domains/impls/stability.rs, src/domains/stores/state_store.rs, src/engine/triggers.rs, src/engine/setters.rs, src/ir/hooks.rs, src/lowering/hook_extractor.rs. Le chapitre 3 donne le cadre mathématique (treillis, concrétisation, point fixe, widening, soundness) dans lequel toutes ces approximations se formulent exactement.

Exercices

Exercice 2.1: Les itérations d'Inf

Pour Inf (exemple 2.3), donner la valeur du slot s (composante numérique) au début de chaque itération de l'algorithme 1, sachant que `widen_threshold = 3`, et dire à quelle itération `widen_trace` reçoit le slot. Comparer avec la sortie `-verbose : Inf: 3 iteration(s), widened: [0]`.

Solution

Amorçage : $[0, 0]$. Passe 1 : l'effet écrit $[1, 1]$, $N = [0, 1] \not\sqsubseteq [0, 0]$, $i = 1$, $S \leftarrow [0, 1]$. Passe 2 : écriture $[1, 2]$, $N = [0, 2]$, $i = 2$, $S \leftarrow [0, 2]$. Passe 3 : écriture $[1, 3]$, $N = [0, 3]$, $i = 3 \geq 3$: le slot entre dans `widen_trace` avec l'itération 3 (d'où « widened at iteration 3 ») et $S \leftarrow [0, 2] \nabla [0, 3]$, non borné à droite. Passe 4 : plus rien ne croît, sortie. Le compteur vaut 3.

Exercice 2.2: Deux updaters

Un handler exécute `setN(c) => c + 1`; `setN(c) => c + 1`; avec un store $\{n \mapsto [0, 0]\}$. Que vaut le slot après la passe selon `exec_setter_call`? Que donne React? Pourquoi le résultat abstrait est-il correct bien qu'il contienne une valeur que React ne produit jamais après ce clic?

Solution

Premier updater : $c = [0, 0]$, écriture $[1, 1]$, slot $[0, 1]$. Second : $c = [0, 1]$, écriture $[1, 2]$, slot $[0, 2]$. React rejoue la file : $0 \rightarrow 1 \rightarrow 2$, soit 2. $[0, 2]$ contient 2; les valeurs 0 et 1 en trop sont le prix du join (imprécision), pas une erreur : une sur-approximation peut contenir des états impossibles, elle ne doit pas en omettre de possibles.

Exercice 2.3: Tous les effets

En vous appuyant sur les règles `COMMITEFFSPATH` et `COMMITEFFSPATHIDLE`, montrer qu'exécuter tous les effets à chaque itération (liste 2.16) produit un store qui contient toutes les écritures des effets que React exécute. Qu'est-ce que cette sur-approximation coûte en précision, et où le projet la récupère-t-il?

Solution

React exécute un sous-ensemble des effets (ceux dont la vue porte `Effect`); l'analyseur les exécute tous, sur un store qui contient déjà toutes les valeurs atteignables à ce point; comme les écritures sont jointes, l'ensemble obtenu contient celui de n'importe quel sous-ensemble. Le coût : un effet de montage semble tourner à chaque tour (`MountOnly` est élargi). Le projet récupère la précision dans les règles, qui relisent les deps (`infinite-loop` écarte `[]` et les deps toutes stables) et dans `effect_triggers`.

Exercice 2.4: Les trois faits de l'Error d'ObjChurn

Énumérer les trois faits `must` qui fondent l'ERROR d'ObjChurn. Si la dep est `[obj.a]` au lieu de `[obj]`, lequel manque, et pourquoi?

Solution

`Must-change` : la dep est le slot exact et la valeur écrite est `PerRender`. `Must-reach` : l'écriture a lieu sur tous les chemins du corps de l'effet. `Must-rerun` : React ré-exécute un effet dont une dep a changé. Avec `[obj.a]`, le premier manque : `...obj, b: 2` conserve la valeur du champ `a`, la dep peut ne pas changer; l'ADR-017 plafonne ce cas au `WARNING`.

Exercice 2.5: ObjStateDep

L'état de `ObjStateDep` est initialisé par un littéral objet, donc le store contient une référence `PerRender`. Expliquer pourquoi `always-unstable-deps` ne tire pas, en citant la ligne de code responsable, et dire quelle hypothèse cette ligne fait sur le programme.

Solution

La lecture `Expr::StateVal` remplace la composante référence par `Versioned((composant, label))` (liste 2.22, l. 130–132); la règle exige `PerRender`. L'hypothèse : les sets ont lieu hors du rendu; sa violation est signalée par `setter-in-render` (`soundness` en couches).

Exercice 2.6: Pourquoi Toggle converge

Calculer la suite des valeurs booléennes abstraites du slot `on` de `Toggle` et expliquer la sortie 1 iteration(s), widened: []. Quel fait *structurel*, à la manière du bras churn, permettrait de prouver la boucle ? (Question ouverte : il n'existe pas de réponse dans le code.)

Solution

Amorçage `false` ; la passe d'effet écrit `!on = true` ; le slot devient `false`, `true`, c'est-à-dire le top du treillis booléen ; $i = 1$; la passe suivante n'ajoute rien, sortie. Aucune croissance, donc pas de widening. Il faudrait établir que la dep est le slot exact (ligne exact de `effect_triggers`) et que la valeur écrite diffère *toujours* de la valeur lue (ici $\neg v \neq v$ pour tout booléen), sur tous les chemins de l'effet : un must-change par valeur plutôt que par identité.

Exercice 2.7: Inf2 de bout en bout

Pour Inf2 (exemple 2.8), donner la règle de React-tRace qui s'applique à l'appel `sets((x) => x)`, le théorème qui explique la boucle, et le fait de CFG que `setter-in-render` utilise pour émettre une `ERROR` plutôt qu'un `WARNING`.

Solution

`APPSETCOMP` (set pendant l'évaluation du corps) pose `Check` ; par le théorème 2.1, l'évaluation est retentée (`EVALMULT`), et comme chaque essai rappelle le setter, indéfiniment, quelle que soit la valeur. `setter-in-render` émet une `ERROR` parce que le bloc de l'appel domine toutes les sorties du rendu : l'appel a lieu à chaque rendu.

Chapitre 3

Rappels d'interprétation abstraite

Objectifs du chapitre.

- Comprendre pourquoi un analyseur statique *approxime*, et dans quel sens il a le droit de se tromper.
- Maîtriser le vocabulaire des treillis (ordre, join, meet, top, bottom, hauteur) et la concrétisation.
- Savoir calculer un point fixe par itération de Kleene et par worklist, et savoir pourquoi le widening est nécessaire.
- Distinguer un fait must d'un fait may, et relier cette polarité aux niveaux ERROR, WARNING, INFO.
- Connaître la dominance, l'inlining, les mises à jour fortes et faibles, et situer `reactant-analyzer` parmi ces choix.

Prérequis : chapitre 1 et chapitre 2 (le cycle rendu $\rightarrow$ commit $\rightarrow$ effets, le batching des setters). Aucune connaissance préalable d'analyse statique n'est supposée ; la lecture de Rust est supposée.

3.1 Pourquoi approximer

3.1.1 Une question que l'analyseur doit trancher

Partons de deux composants presque identiques. Ils sont au cœur de ce chapitre ; nous les suivrons jusqu'au bout.

Extrait 3.1 – Deux compteurs pilotés par un effet (`/tmp/ch03/counter.tsx`, repris de `tests/fixtures/widening.tsx`)

```
1 import { useState, useEffect } from "react";
2
3 export function Runaway() {
4   const [count, setCount] = useState(0);
5   useEffect(() => {
6     setCount(count + 1);
7   }, [count]);
8   return <div>{count}</div>;
9 }
10
11 export function Guarded() {
12   const [count, setCount] = useState(0);
13   useEffect(() => {
14     if (count < 10) setCount(count + 1);
15   }, [count]);
16   return <div>{count}</div>;
17 }
```

À l'exécution (chapitre 2), Runaway ne s'arrête jamais : chaque exécution de l'effet écrit `count + 1`, la `dep count` change, l'effet repart. Guarded s'arrête après dix tours, quand la garde `count < 10` devient fausse. La question que l'analyseur doit trancher est donc : *l'état count peut-il grandir sans borne ?*

Regardons ce que React fait vraiment, en notant l'ensemble des valeurs que `count` prend au fil d'une exécution. Pour Runaway, c'est $\{0, 1, 2, 3, \dots\} = \mathbb{N}$; pour Guarded, c'est $\{0, 1, \dots, 10\}$. Ces ensembles sont la réponse exacte. Si l'analyseur savait les calculer, il n'aurait plus qu'à demander « est-il fini ? ».

Définition 3.1: Sémantique concrète, sémantique collectrice

La *sémantique concrète* d'un programme décrit ses exécutions : suites d'états, un état donnant la valeur de chaque variable et de chaque slot d'état. Sa *sémantique collectrice* en garde l'essentiel pour un analyste : pour chaque point du programme, l'ensemble des états qu'une exécution quelconque peut y atteindre. On la note $\mathcal{C}$; un élément de $\mathcal{C}$ est une partie de l'ensemble des états.

Pour *reactant-analyzer*, la sémantique concrète de référence est celle de React décrite au chapitre 2 (React-tRace étendue, ADR-1) ; un « point » est un bloc du rendu, d'un effet ou d'un handler, et un état comprend l'environnement local et la valeur de chaque slot.

3.1.2 Pourquoi on ne peut pas calculer la réponse exacte

Deux obstacles s'y opposent.

- **La taille.** L'ensemble des valeurs de `count` dans Runaway est infini. On ne peut pas l'énumérer, et un ensemble quelconque d'entiers n'a pas de représentation finie.
- **L'indécidabilité.** Même en se limitant à des questions à réponse oui/non, la réponse exacte est hors d'atteinte. Le théorème de Rice affirme que toute propriété non triviale du comportement d'un programme (et « ce composant boucle » en est une) est indécidable : aucun algorithme ne la tranche correctement sur tous les programmes. Un composant React contient un langage Turing-complet (JavaScript) ; on ne peut donc pas espérer un analyseur exact.

Il reste à renoncer à l'exactitude, mais *dans un sens choisi*. L'*interprétation abstraite* [CC77, CC79] est la théorie qui organise ce renoncement : on remplace les ensembles d'états par des *descriptions* finies, calculables, qui contiennent au moins tous les états possibles. On calcule ces descriptions en « exécutant » le programme sur elles, et l'on raisonne sur le résultat. Le manuel de Rival et Yi [RY20] en donne un exposé complet ; ce chapitre n'en retient que ce dont *reactant-analyzer* se sert, et montre à chaque fois où le code l'implémente.

3.1.3 Sur-approximer, et ce que cela coûte

Choisir le « sens » de l'erreur, c'est choisir entre deux familles d'analyses (figure 3.1).

- Une *sur-approximation* calcule un ensemble A qui *contient* tous les comportements possibles $\mathcal{C}$: $\mathcal{C} \subseteq A$. Si A ne rencontre pas la zone des comportements fautifs, le programme est correct : le silence de l'analyse est une preuve. En revanche A peut rencontrer la zone fautive alors que $\mathcal{C}$ ne la rencontre pas : l'analyse signale un défaut qui n'existe pas, c'est un *faux positif*.
- Une *sous-approximation* calcule un ensemble contenu dans $\mathcal{C}$ (c'est ce que fait un test : il exécute quelques comportements). Tout défaut qu'elle trouve est réel, mais son silence ne prouve rien : elle peut manquer un défaut, c'est un *faux négatif*.

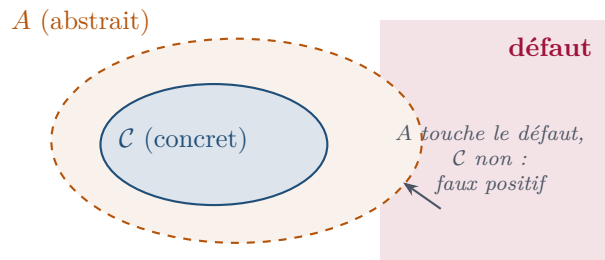


FIGURE 3.1 – Sur-approximation. Le concret $\mathcal{C}$ (ensemble des états réellement atteignables) est inclus dans l’abstrait A . Tant que A évite la zone de défaut, le programme est prouvé correct ; si seul A la touche, l’analyse émet un faux positif. Jamais un défaut de $\mathcal{C}$ ne peut échapper à A .

reactant-analyzer a fait le premier choix et l’a érigé en invariant (CLAUDE.md) : « faux positifs tolérés, faux négatifs INTERDITS ». Nous formaliserons cette promesse à la section 3.8, une fois les outils en place. Retenons dès maintenant qu’elle a une conséquence inattendue : quand l’outil veut affirmer un défaut *certain* (niveau ERROR), il lui faut un autre type de raisonnement, celui des faits *must* (section 3.7), car une sur-approximation ne prouve jamais qu’un comportement a lieu.

Remarque

Le mot « approximation » ne veut pas dire « à peu près ». Une sur-approximation est une *borne* : elle est fausse au sens où elle contient des états impossibles, mais vraie au sens où elle contient tous les états possibles. Toute la suite du chapitre consiste à rendre cette borne calculable (treillis, points fixes, widening) sans jamais la rendre fausse dans l’autre sens.

3.2 Ordres, treillis, top et bottom

3.2.1 Un premier exemple : que sait-on d’un booléen ?

Soit un composant qui lit une prop booléenne `open`. Sans rien savoir de l’appelant, l’analyseur doit décrire les valeurs possibles de `open`. Il y a quatre descriptions utiles :

- « `open` ne peut valoir que `true` » ;
- « `open` ne peut valoir que `false` » ;
- « `open` peut valoir l’un ou l’autre » : on ne sait rien, c’est l’information minimale, notée $\top$ (*top*) ;
- « aucune valeur n’est possible » : le point est inatteignable, notée $\perp$ (*bottom*).

Ces descriptions sont ordonnées par la *quantité de valeurs qu’elles admettent* : $\perp$ en admet le moins, $\top$ le plus ; `true` et `false` ne sont pas comparables. Figure 3.2 dessine cet ordre : un trait montant de a vers b signifie « a est plus précis que b ». C’est un *diagramme de Hasse*.

Si un chemin du rendu donne `open` = `true` et un autre `open` = `false`, ce que l’on sait après la jonction des deux chemins est la plus petite description qui admette les deux : $\top$. Cette opération de « réunion d’informations » s’appelle le *join*.

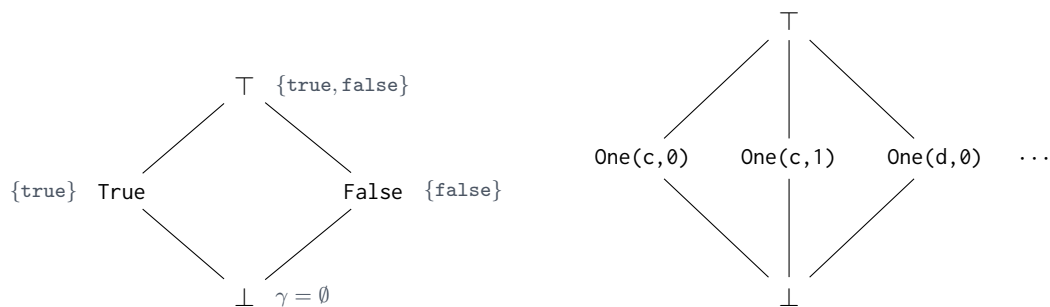


FIGURE 3.2 – Deux treillis plats. À gauche, BoolVal (src/domains/impls/bool_val.rs) avec la concrétisation de chaque élément (section 3.3); à droite, SetterVal (src/domains/impls/setter_val.rs), dont les éléments intermédiaires nomment un setter précis (composant, label de hook). Deux éléments intermédiaires distincts sont incomparables; leur join est $\top$.

3.2.2 Définitions

Définition 3.2: Ordre partiel, treillis, join, meet

Un *ordre partiel* sur un ensemble L est une relation $\sqsubseteq$ réflexive, antisymétrique et transitive; deux éléments peuvent être incomparables. Pour $a, b \in L$:

- le *join* $a \sqcup b$ est la plus petite borne supérieure de a et b : $a \sqsubseteq a \sqcup b$, $b \sqsubseteq a \sqcup b$, et tout c qui majore a et b vérifie $a \sqcup b \sqsubseteq c$;
- le *meet* $a \sqcap b$ est, dualement, la plus grande borne inférieure.

$(L, \sqsubseteq)$ est un *treillis* si toute paire admet un join et un meet; il est *complet* si toute partie (même infinie) admet une borne supérieure (notée $\sqcup$) et une borne inférieure. Le plus petit élément, s'il existe, est le *bottom* $\perp$; le plus grand est le *top* $\top$. Une *chaîne ascendante* est une suite $x_0 \sqsubseteq x_1 \sqsubseteq \dots$; la *hauteur* de L est la longueur maximale d'une chaîne strictement croissante (elle peut être infinie).

En analyse, on lit l'ordre comme un ordre d'*information*: $a \sqsubseteq b$ signifie « a est plus précis que b », ou « b admet au moins tous les comportements de a ». $\perp$ décrit un point inatteignable, $\top$ un point dont on ne sait rien (on dira aussi « inconnu »). Le join est l'opération des *jonctions* du programme (après un if, en tête de boucle, entre deux tours de React): il fusionne ce que disent plusieurs chemins. Le meet raffinerait une information par une autre; nous verrons que reactant-analyzer ne s'en sert presque pas.

Exemple 3.1: Le treillis plat des booléens

Dans BoolVal, $\text{True} \sqcup \text{False} = \top$, $\text{True} \sqcap \text{False} = \perp$, $\top \sqcup \text{True} = \top$ et $\perp \sqsubseteq x \sqsubseteq \top$ pour tout x . La hauteur est 2 (plus longue chaîne stricte: $\perp \sqsubset \text{True} \sqsubset \top$). Les valeurs ont été vérifiées sur le code (sonde ops, $\text{True vs False} = \text{None}$, $\text{True join False} = \text{Top}$).

3.2.3 Ce que fait le code : le trait AbstractDomain

Dans reactant-analyzer, un domaine abstrait est un type Rust qui implémente le trait AbstractDomain. Sa tête traduit exactement la définition 3.2 :

Extrait 3.2 – src/domains/mod.rs, l. 16–38 — le trait des domaines abstraits

```
16 /// Core abstract domain trait.
17 ///
18 /// Supertrait bounds:
```

```

19 /// - `Clone + Copy` values are small, freely copyable
20 /// - `PartialEq` needed for convergence checks
21 /// - `PartialOrd` lattice order ( $a \leq b = a \sqsubseteq b$ )
22 /// - `Debug` required for diagnostics and derive macros on generic containers
23 pub trait AbstractDomain: Clone + PartialEq + PartialOrd + std::fmt::Debug {
24     fn bottom() -> Self;
25     fn top() -> Self;
26     fn is_bottom(&self) -> bool;
27     fn join(&self, other: &Self) -> Self;
28     fn meet(&self, other: &Self) -> Self;
29     fn widen(&self, other: &Self) -> Self;
30
31     /// Widening "up to" a finite set of thresholds (ASTRÉE-style).
32     ///
33     /// A growing bound jumps to the tightest enclosing threshold rather than
34     /// straight to  $\top/\pm\infty$ , recovering precision on guarded growth. Default falls
35     /// back to plain `widen` (sound, threshold-unaware). Numeric domains override.
36     fn widen_to(&self, other: &Self, _thresholds: &[f64]) -> Self {
37         self.widen(other)
38     }

```

- **L'ordre est PartialOrd.** Le supertrait PartialOrd porte l'ordre du treillis : $a \leq b$ signifie $a \sqsubseteq b$, et partial_cmp rend None pour deux éléments incomparables. Ce n'est pas un ordre total (Ord serait faux).
- **bottom, top, join, meet** sont les quatre ingrédients de la définition. is_bottom teste l'inatteignabilité (un domaine peut avoir plusieurs représentations de $\perp$, c'est le cas des intervalles).
- **widen et widen_to** sont les opérateurs de widening de la section 3.6. Par défaut widen_to ignore ses seuils et délègue à widen.
- **PartialEq** sert au test de convergence des points fixes.

Piège

Le commentaire en tête du trait annonce Clone + Copy, mais la borne réelle est Clone + PartialEq + PartialOrd + Debug : pas de Copy. Le commentaire date d'avant le domaine produit (ADR-15) et la stabilité versionnée (ADR-17), dont les valeurs contiennent des ensembles. Se fier à la ligne pub trait, pas au commentaire.

Le domaine le plus simple du code est le treillis plat des booléens :

Extrait 3.3 – src/domains/impls/bool_val.rs, l. 1–13 — BoolVal, un treillis plat

```

1 /// Flat lattice for the boolean slot of `StateValue`:  $\perp < \{\text{True}, \text{False}\} < \top$ ,
2 /// with `True`/`False` incomparable. Lattice ops via the `flat_lattice!` macro.
3 #[derive(Debug, Clone, Copy, PartialEq, Eq)]
4 pub enum BoolVal {
5     ///  $\perp$  unreachable.
6     Bottom,
7     True,
8     False,
9     ///  $\top$  may be either.
10    Top,
11 }
12
13 flat_lattice!(BoolVal, bottom = Bottom, top = Top);

```

La macro flat_lattice! engendre l'ordre et les opérations de tout treillis plat (un $\perp$, un $\top$, des éléments intermédiaires deux à deux incomparables). Son partial_cmp et son join sont la

définition 3.2 écrite en match :

Extrait 3.4 – src/domains/impls/mod.rs, l. 10–40 — la macro flat_lattice! (ordre et join)

```

10 macro_rules! flat_lattice {
11     ($ty:ident, bottom = $bottom:ident, top = $top:ident) => {
12         impl ::std::cmp::PartialOrd for $ty {
13             fn partial_cmp(&self, other: &Self) -> ::core::option::Option<::std::cmp::Ordering> {
14                 use ::std::cmp::Ordering;
15                 match (self, other) {
16                     (a, b) if a == b => Some(Ordering::Equal),
17                     ($ty::$bottom, _) | (_, $ty::$top) => Some(Ordering::Less),
18                     ($ty::$top, _) | (_, $ty::$bottom) => Some(Ordering::Greater),
19                     _ => None,
20                 }
21             }
22         }
23
24         impl $crate::domains::AbstractDomain for $ty {
25             fn bottom() -> Self {
26                 $ty::$bottom
27             }
28             fn top() -> Self {
29                 $ty::$top
30             }
31             fn is_bottom(&self) -> bool {
32                 matches!(self, $ty::$bottom)
33             }
34             fn join(&self, other: &Self) -> Self {
35                 match (self, other) {
36                     (a, b) if a == b => a.clone(),
37                     ($ty::$bottom, x) | (x, $ty::$bottom) => x.clone(),
38                     _ => $ty::$top,
39                 }
40             }
41         }
42     }
43 }

```

L'ordre des bras compte : l'égalité est testée d'abord, puis « $\perp$ à gauche ou $\top$ à droite » (donc Less) ; le cas ($\perp$, $\top$) tombe ainsi sur Less. Deux éléments intermédiaires différents n'ont aucun bras : None. Le join suit le même schéma : égaux, on garde ; l'un est $\perp$, on garde l'autre ; sinon $\top$. Le meet (src/domains/impls/mod.rs, l. 41–47) est le dual, et widen est le join (src/domains/impls/mod.rs, l. 48–50) : un treillis de hauteur finie n'a pas besoin de mieux (section 3.6).

Les stores comparent leurs valeurs slot par slot avec un seul utilitaire, qui dit bien que l'ordre est partiel :

Extrait 3.5 – src/domains/stores/mod.rs, l. 19–25 — $a \sqsubseteq b$ dans un ordre partiel

```

19 /// `a ⊆ b` in a (possibly partial) lattice order: `Less`/`Equal` hold,
20 /// incomparable/`Greater` do not. Centralises the pointwise  $\leq$  check the store
21 /// `leq`s share, spelled with `partial_cmp` (the form clippy prefers over a
22 /// negated comparison operator).
23 pub(crate) fn leq_pointwise<D: AbstractDomain>(a: &D, b: &D) -> bool {
24     matches!(a.partial_cmp(b), Some(Ordering::Less | Ordering::Equal))
25 }

```

Piège

Écrire $!(a > b)$ pour tester $a \sqsubseteq b$ est faux dans un ordre partiel : deux éléments incomparables ne sont ni $>$ ni $<=$. `leq_pointwise` ne retient que Less ou Equal, et un couple incomparable

n'est donc *pas* considéré comme convergé.

3.2.4 Autres treillis du projet, en un coup d'œil

Le chapitre 9 détaillera les domaines de base ; il suffit ici d'en connaître la forme.

- `SetterVal` : treillis plat dont les éléments intermédiaires sont les setters `One(ComponentId, HookLabel)` (figure 3.2, à droite).
- `StrConst` : ensembles d'au plus quatre chaînes constantes, ordonnés par inclusion, avec $\top$ au-delà ; hauteur finie.
- `Interval` : intervalles de nombres, ordonnés par inclusion ; hauteur *infinie* (section 3.4).
- `Stability` : la stabilité référentielle versionnée d'ADR-17 (chapitre 10).
- `StateValue` : le produit des précédents, une composante par sorte de valeur JavaScript (ADR-15). Un produit de treillis est un treillis, ordonné composante par composante.

3.3 Concrétisation et correction

3.3.1 Ce que « veut dire » une valeur abstraite

Dans `Guarded`, l'analyseur finit par attribuer à `count` l'intervalle $[0, 10]$. Pour que cette phrase soit une affirmation vérifiable, il faut dire *quels nombres* elle autorise : les entiers de 0 à 10. La fonction qui associe à chaque valeur abstraite l'ensemble des valeurs concrètes qu'elle décrit est la concrétisation.

Définition 3.3: Concrétisation, abstraction, correction

Soit $(L, \sqsubseteq)$ un domaine abstrait et $\mathcal{D}$ l'ensemble des valeurs (ou des états) concrets. La *concrétisation* $\gamma : L \rightarrow \mathcal{P}(\mathcal{D})$ associe à chaque valeur abstraite l'ensemble des valeurs concrètes qu'elle représente. Elle est *monotone* : $a \sqsubseteq b \Rightarrow \gamma(a) \subseteq \gamma(b)$. On demande $\gamma(\perp) = \emptyset$ et, souvent, $\gamma(\top) = \mathcal{D}$.

Une *abstraction* $\alpha : \mathcal{P}(\mathcal{D}) \rightarrow L$ associe à un ensemble concret la meilleure description disponible. Quand $\alpha(X) \sqsubseteq a \iff X \subseteq \gamma(a)$, le couple (α, γ) est une *correspondance de Galois*.

Une opération abstraite $f^\# : L \rightarrow L$ est *correcte* vis-à-vis d'une opération concrète f si, pour tout a , $f(\gamma(a)) \subseteq \gamma(f^\#(a))$: ce que f peut produire à partir de valeurs décrites par a est décrit par $f^\#(a)$. Pour le join : $\gamma(a) \cup \gamma(b) \subseteq \gamma(a \sqcup b)$.

La condition de correction est la seule qui compte pour la soundness : une opération abstraite peut être imprécise (rendre plus que nécessaire), jamais rendre moins. Une abstraction α n'est qu'une commodité de présentation : `reactant-analyzer` ne définit aucune fonction α générale. Il abstrait les littéraux au cas par cas (`Interval::point(0.0)` pour $\emptyset$) et calcule tout le reste par des opérations abstraites ; γ n'apparaît, lui, que dans les commentaires du code, sous forme de phrases.

Exemple 3.2: Deux concrétisations

- `BoolVal` : $\gamma(\perp) = \emptyset$, $\gamma(\text{True}) = \{\text{true}\}$, $\gamma(\text{False}) = \{\text{false}\}$, $\gamma(\top) = \{\text{true}, \text{false}\}$. Ici γ est une bijection avec les parties de $\{\text{true}, \text{false}\}$; le join est *exact* : $\gamma(a \sqcup b) = \gamma(a) \cup \gamma(b)$.
- `Interval` : $\gamma([a, b]) = \{x \mid a \leq x \leq b\}$, restreint aux entiers si l'intervalle est marqué entier. Le join $[0, 0] \sqcup [10, 10] = [0, 10]$ est correct ($\{0\} \cup \{10\} \subseteq \gamma([0, 10])$) mais *pas* exact : $\gamma([0, 10])$ contient aussi $1, \dots, 9$. C'est la perte de précision inhérente à un

domaine convexe.

3.3.2 Ce que fait le code : l'intervalle

La structure `Interval` documente sa propre concrétisation :

Extrait 3.6 – `src/domains/impls/interval.rs`, l. 5–27 — les intervalles et leur concrétisation

```

5  /// Closed interval [lo, hi] over f64. `lo > hi` = bottom (empty).
6  ///
7  /// `is_int` records whether every concrete value the interval denotes is an
8  /// integer (`true` = proven all-integer; `false` = may contain non-integers).
9  /// It is a *precision* annotation only: it lets `narrow_lt`/`narrow_gt` apply
10 /// the integer tightening  $x < 5 \Rightarrow x \leq 4$  (ADR-014 threshold widening relies on
11 /// this to bound counting loops) while staying sound over reals when the flag is
12 /// clear – a float state  $x = 1.7$  under  $x < 2$  must keep 1.7, not drop it.
13 /// Because it carries no set-membership information beyond `[lo, hi]`, it is
14 /// deliberately excluded from `PartialEq`/`PartialOrd`: two intervals with the
15 /// same bounds are equal (and the fixpoint converges) regardless of `is_int`.
16 #[derive(Debug, Clone, Copy)]
17 pub struct Interval {
18     pub lo: f64,
19     pub hi: f64,
20     pub is_int: bool,
21 }
22
23 impl PartialEq for Interval {
24     fn eq(&self, other: &Self) -> bool {
25         self.lo == other.lo && self.hi == other.hi
26     }
27 }
```

- `lo, hi` : bornes fermées, en `f64` ; $\pm\infty$ représente l'absence de borne. Un intervalle avec `lo > hi` est $\perp$ (vide).
- `is_int` : « toute valeur concrète est entière ». C'est une annotation de *précision* : elle autorise le raffinement $x < 5 \Rightarrow x \leq 4$, que nous retrouverons avec le widening. Elle est volontairement exclue de l'égalité (et de l'ordre) : le point fixe converge quelles que soient les valeurs de ce bit.

Le join des intervalles est l'enveloppe convexe (*hull*) :

Extrait 3.7 – `src/domains/impls/interval.rs`, l. 69–83 — le join des intervalles

```

69  /// Least upper bound: smallest interval containing both. Integer-valued only
70  /// if both operands are (the union of two integer sets is integer-valued).
71  pub fn hull(&self, other: &Self) -> Self {
72      if self.is_bottom() {
73          return *other;
74      }
75      if other.is_bottom() {
76          return *self;
77      }
78      Interval {
79          lo: self.lo.min(other.lo),
80          hi: self.hi.max(other.hi),
81          is_int: self.is_int && other.is_int,
82      }
83  }
```


Sa correction se lit ligne à ligne : si $x \in \gamma(a)$, alors $\min(a.lo, b.lo) \leq a.lo \leq x \leq a.hi \leq \max(a.hi, b.hi)$; et si les deux opérandes ne contiennent que des entiers, leur réunion aussi. Les deux premiers tests traitent $\perp$ comme élément neutre, ce que demande $\gamma(\perp) = \emptyset$.

Piège

$\perp$ a plusieurs représentations. Le $\perp$ canonique est $[\!+\infty, -\infty]$, mais un raffinement peut produire, par exemple, $[5, 0]$ (c'est ce que la sonde `w1` observe dans la boucle de la section 3.5). Les combinateurs testent `is_bottom()` d'abord et restent corrects; en revanche `PartialEq` et `PartialOrd` comparent les bornes brutes, et $[5, 0] \neq [5, 1]$ pour eux alors que les deux sont vides. Conséquence pratique : une worklist peut revisiter un bloc inatteignable pour rien. C'est une imprécision de performance, pas une faute de soundness.

3.3.3 « Absent » vaut $\top$ ou $\perp$ selon le contexte

Deux conteneurs de `reactant-analyzer` donnent une valeur par défaut aux clefs absentes, et ils ne donnent pas la même.

Extrait 3.8 – `src/domains/stores/abstract_env.rs`, l. 78–81 — une variable inconnue vaut $\top$

```
78     /// Conservative lookup: `D::top()` for any variable not in the map.
79     pub fn lookup(&self, var: &str) -> D {
80         self.stabs.get(var).cloned().unwrap_or_else(D::top)
81     }
```

Extrait 3.9 – `src/domains/stores/state_store.rs`, l. 24–27 — un slot jamais écrit vaut $\perp$

```
24     /// Returns `D::bottom()` for labels not yet updated by any setter call.
25     pub fn get(&self, label: HookLabel) -> D {
26         map_get_or(&self.0, &label, D::bottom)
27     }
```

Les deux choix sont corrects, parce que les deux questions diffèrent. Une variable que l'environnement ne connaît pas (un import non résolu, un global du navigateur) peut valoir n'importe quoi : répondre autre chose que $\top$ inventerait une information. Un slot d'état, au contraire, ne reçoit de valeur que par ses écritures; le store accumule ces écritures par `join` à partir de $\perp$, et le moteur l'amorce dès le départ avec la valeur initiale de chaque `useState` (chapitre 13). « Pas d'écriture vue » y signifie « pas de valeur ajoutée », donc $\perp$.

Piège

Confondre les deux conventions est une source classique de faux négatif : lire un slot absent comme $\perp$ *dans une règle* reviendrait à dire « cet état ne prend aucune valeur », donc « il ne change pas ». Les règles lisent l'état convergé, où tout slot a été amorcé; hors de ce cadre, une absence est une absence d'information, et l'information minimale est $\top$.

3.4 Exemple filé : le compteur en intervalles

3.4.1 Le treillis des intervalles

Pour répondre à la question de la section 3.1, un domaine qui décrit des ensembles de nombres par leurs bornes suffit : le treillis des intervalles (figure 3.3). Son ordre est l'inclusion, $[a, b] \sqsubseteq [c, d] \iff c \leq a \wedge b \leq d$ (`src/domains/impls/interval.rs`, l. 312–328); son `join` est l'enveloppe convexe, son `meet` l'intersection. Deux intervalles qui se chevauchent sans s'inclure, comme $[1, 3]$ et $[2, 9]$, sont incomparables (`partial_cmp` rend `None`, vérifié avec la sonde `ops`).

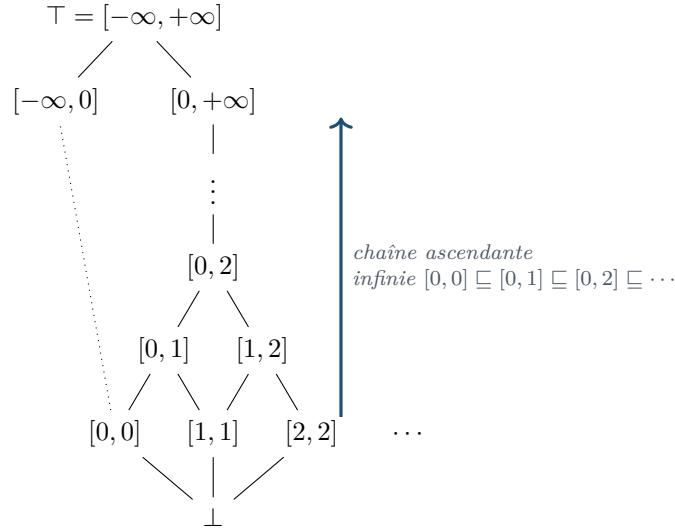


FIGURE 3.3 – Le treillis des intervalles, tronqué. Un intervalle est au-dessus de ceux qu'il contient. La chaîne $[0, 0] \subseteq [0, 1] \subseteq [0, 2] \subseteq \dots$ ne s'arrête jamais : la hauteur est infinie, ce qui obligera au widening (section 3.6).

Les opérations arithmétiques se relèvent borne à borne. Pour l'addition, $[a, b] + [c, d] = [a + c, b + d]$: si $x \in [a, b]$ et $y \in [c, d]$, alors $a + c \leq x + y \leq b + d$. Le code est la formule (src/domains/impls/interval.rs, l. 148–157), précédée du cas $\perp$: une somme dont un opérande est inatteignable est inatteignable.

3.4.2 Un tour abstrait de Runaway

Exécutons « à la main » un tour de React sur des intervalles, comme le fait le moteur (chapitre 13 donnera le détail).

1. **Amorçage.** `useState(0)` : le slot 0 (count) vaut $[0, 0]$.
2. **Rendu.** La variable count lit le slot : $[0, 0]$. Le rendu n'écrit aucun état.
3. **Effet.** Le corps évalue `count + 1`, soit $[0, 0] + [1, 1] = [1, 1]$, et appelle le setter.
4. **Écriture.** Le setter *ne remplace pas* la valeur du slot : il la joint, $[0, 0] \sqcup [1, 1] = [0, 1]$.
Nous justifierons ce choix à la section 3.11 ; retenons que le slot décrit toutes les valeurs que count a pu prendre jusqu'ici.

Au tour suivant, le rendu lit $[0, 1]$, l'effet écrit $[1, 2]$, le slot devient $[0, 2]$, et ainsi de suite. Au tour k (en partant de 0), le slot vaut $[0, k + 1]$: c'est exactement l'ensemble des valeurs que React donne à count au cours des $k + 1$ premières exécutions de l'effet. Chaque tour abstrait est correct (il contient ce que React peut faire), mais la suite ne se stabilise jamais.

Pour Guarded, un ingrédient s'ajoute : la garde. Sur la branche « vraie » de `count < 10`, l'analyseur sait que count est strictement inférieur à 10 ; sur un intervalle d'entiers, il le *raffine* en $[0, k] \sqcap [-\infty, 9]$ (le code calcule ce meet directement, par `Interval::narrow_lt`). C'est le *raffinement de garde* (`narrow_env_for_branch`, chapitre 12). Tant que $k < 10$, le raffinement ne change rien. Au tour où le slot vaut $[0, 10]$, la branche vraie ne voit plus que $[0, 9]$, l'effet écrit $[1, 10]$, et $[0, 10] \sqcup [1, 10] = [0, 10]$: le tour n'ajoute plus rien.

Tableau 3.1 pose les deux questions des sections suivantes. D'abord : quand la suite *se stabilise*, comme pour Guarded, pourquoi l'intervalle obtenu est-il une réponse correcte ? C'est la théorie du point fixe (section 3.5). Ensuite : quand elle *ne se stabilise pas*, comme pour Runaway, comment s'arrêter en temps fini sans perdre la correction ? C'est le widening (section 3.6).

tour	Runaway		Guarded		
	slot lu	slot après le tour	slot lu	slot après le tour	
0	[0, 0]	[0, 1]	[0, 0]	[0, 1]	
1	[0, 1]	[0, 2]	[0, 1]	[0, 2]	
2	[0, 2]	[0, 3]	[0, 2]	[0, 3]	
⋮					
9	[0, 9]	[0, 10]	[0, 9]	[0, 10]	
10	[0, 10]	[0, 11]	[0, 10]	[0, 10]	(stable)
⋮					
29	[0, 29]	[0, 30]			

TABLE 3.1 – Les tours abstraits des deux compteurs *sans widening*. Valeurs observées avec la sonde probe, qui réplique la boucle externe du moteur (src/engine/fixpoint.rs, l. 355–530) en désactivant le widening (WT=1000) : Guarded s’arrête au tour 10, Runaway croît encore au tour 29, dernier tour exécuté par la sonde.

3.5 Transfert, équations de flot, point fixe

3.5.1 Une boucle locale

Le compteur de la section précédente itère sur les *tours de React*. Une boucle écrite dans le code pose le même problème à plus petite échelle, à l’intérieur d’un seul corps de fonction. Le programme suivant est repris de la fixture tests/fixtures/widening.tsx :

Extrait 3.10 – Une boucle bornée dans un effet (/tmp/ch03/loop.tsx, identique à BoundedLocalLoop de tests/fixtures/widening.tsx)

```

1 import { useState, useEffect } from "react";
2
3 export function BoundedLocalLoop() {
4   const [total, setTotal] = useState(0);
5   useEffect(() => {
6     let i = 0;
7     while (i < 5) {
8       i = i + 1;
9     }
10    setTotal(i);
11  }, []);
12  return <div>{total}</div>;
13 }

```

Concrètement, i vaut $0, 1, \dots, 5$ en tête de boucle et 5 à la sortie ; l’effet écrit 5. L’analyseur doit retrouver au moins cela, sans exécuter la boucle un nombre de fois qu’il ne connaît pas à l’avance. Le chapitre 7 explique comment le corps de l’effet devient un graphe de flot de contrôle (CFG, définition 6.1) ; figure 3.4 en donne la forme.

3.5.2 Fonctions de transfert

À chaque bloc, l’analyseur associe une *fonction de transfert* abstraite : elle prend la valeur abstraite à l’entrée du bloc (ici, un environnement qui donne un intervalle à i) et rend celle de la sortie. Pour B_2 , $i = i + 1$ devient $f_2^\#(\rho) = \rho[i \mapsto \rho(i) + [1, 1]]$, avec l’addition d’intervalles de la section 3.4. Pour une arête issue d’une garde, la fonction de transfert est le raffinement

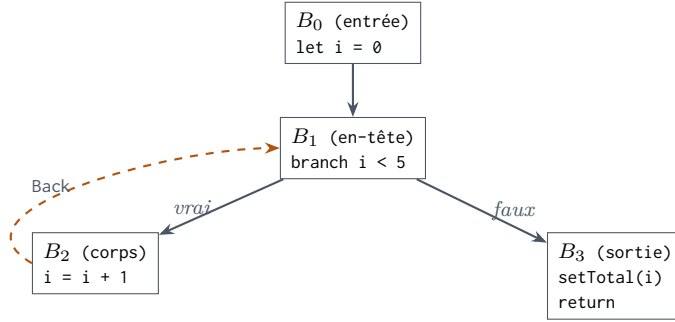


FIGURE 3.4 – CFG schématique du corps de l'effet de liste 3.10. L'arc en pointillés est l'arc arrière de la boucle ; le lowering le marque `EdgeKind::Back` (chapitre 7). La numérotation des blocs est celle de l'exposé, pas nécessairement celle du lowering.

: sur l'arête « vrai » de B_1 , $\rho(i)$ devient $\rho(i) \sqcap [-\infty, 4]$ (i est entier), sur l'arête « faux », $\rho(i) \sqcap [5, +\infty]$.

Définition 3.4: Fonction monotone, fonction correcte

Une fonction $f^\sharp : L \rightarrow L$ est *monotone* si $a \sqsubseteq b \Rightarrow f^\sharp(a) \sqsubseteq f^\sharp(b)$: plus d'information en entrée ne donne pas moins d'information en sortie. Elle est *correcte* vis-à-vis de l'instruction concrète f au sens de la définition 3.3 : $f(\gamma(a)) \sqsubseteq \gamma(f^\sharp(a))$.

L'addition d'intervalles est monotone et correcte (la preuve tient en une ligne, donnée plus haut). Le raffinement de garde est correct parce qu'il ne retire que des valeurs pour lesquelles la garde est fausse. La monotonie est la propriété confortable des manuels ; la correction est la seule dont dépend la soundness, et c'est elle que les commentaires du code revendiquent (« *Sound: result $\sqsubseteq$ self.hull(other)* » sur `widen_to`, `src/domains/impls/interval.rs`, l. 108–113).

3.5.3 Équations de flot

Posons une inconnue X_i par bloc, la valeur abstraite à son entrée. Le CFG donne directement un système d'équations :

$$\begin{aligned}
 X_0 &= \rho_0 \quad (\text{l'environnement d'entrée de l'effet}) \\
 X_1 &= f_0^\sharp(X_0) \sqcup f_2^\sharp(X_2) \\
 X_2 &= \text{vrai}_{i < 5}^\sharp(X_1) \\
 X_3 &= \text{faux}_{i < 5}^\sharp(X_1)
 \end{aligned}$$

L'en-tête B_1 a deux prédécesseurs : l'entrée et l'arc arrière. Son équation les *joint*. Regroupons les inconnues en un vecteur $X = (X_0, \dots, X_3)$ et les membres droits en une fonction $F^\sharp$; le système s'écrit $X = F^\sharp(X)$. Une solution est un *point fixe* de $F^\sharp$.

Définition 3.5: Point fixe, post-point fixe, itération de Kleene

Soit $F : L \rightarrow L$ sur un treillis L . Un *point fixe* est un x tel que $F(x) = x$; le *plus petit point fixe* se note $\text{lfp}F$. Un *post-point fixe* est un x tel que $F(x) \sqsubseteq x$: appliquer F n'ajoute plus rien. L'*itération de Kleene* est la suite

$$x_0 = \perp, \quad x_{n+1} = F(x_n) \text{ (ou, en accumulant, } x_n \sqcup F(x_n)\text{)}.$$

Théorème 3.1: Tarski, Kleene

Si L est un treillis complet et F monotone, alors F a un plus petit point fixe, et c'est le plus petit post-point fixe : $\text{lfp}F$ est la borne inférieure de $\{x \mid F(x) \sqsubseteq x\}$ (Tarski). Si de plus F préserve les bornes supérieures des chaînes, la suite de Kleene est croissante et $\text{lfp}F = \bigsqcup_n x_n$; si L est de hauteur finie, elle atteint $\text{lfp}F$ en un nombre fini de pas.

La conséquence qui compte pour un analyseur est la suivante.

Théorème 3.2: Un post-point fixe abstrait est sound

Soit F la fonction concrète de la sémantique collectrice (sur les parties d'états) et $F^\sharp$ une fonction abstraite correcte : $F(\gamma(a)) \subseteq \gamma(F^\sharp(a))$ pour tout a . Si a est un post-point fixe de $F^\sharp$, alors $\text{lfp}F \subseteq \gamma(a)$.

Preuve. Par correction puis monotonie de γ , $F(\gamma(a)) \subseteq \gamma(F^\sharp(a)) \subseteq \gamma(a) : \gamma(a)$ est un post-point fixe de F , donc il contient le plus petit (Tarski). $\square$

Ce théorème est précieux parce qu'il ne demande *ni* que $F^\sharp$ soit monotone, *ni* que a soit le plus petit point fixe abstrait. Il suffit de s'arrêter sur un a tel que $F^\sharp(a) \sqsubseteq a$: on peut y arriver par n'importe quel chemin, y compris en sautant des étapes (c'est ce que fera le widening). Tout le moteur de `reactant-analyzer` repose sur ce seul test ; algorithme 2 en donne la forme la plus simple.

Algorithme 2 : Itération de Kleene (forme accumulante)

Input : $F^\sharp : L \rightarrow L$

Output : un post-point fixe x de $F^\sharp$

```

1  $x \leftarrow \perp$ ;
2 repeat
3    $y \leftarrow F^\sharp(x)$ ;
4   if  $y \sqsubseteq x$  then return  $x$ ;
5    $x \leftarrow x \sqcup y$ ;
6 until  $y \sqsubseteq x$ ;
```

3.5.4 La worklist : n'itérer que ce qui change

Itérer $F^\sharp$ en bloc recalcule tous les blocs à chaque pas, même ceux dont l'entrée n'a pas bougé. Une *worklist* ne recalcule qu'un bloc dont l'entrée a changé, et ne propage qu'aux successeurs concernés (algorithme 3). C'est une stratégie d'itération chaotique : elle calcule le même résultat que Kleene quand $F^\sharp$ est monotone et le domaine de hauteur finie, avec beaucoup moins de travail.

3.5.5 Ce que fait le code : `analyze_cfg`

`analyze_cfg` (`src/engine/cfg_analyzer.rs`) est la *algorithme 3* en Rust. Le chapitre 12 la détaille ; en voici les trois temps. D'abord l'initialisation :

Extrait 3.11 – `src/engine/cfg_analyzer.rs`, l. 47–58 — `analyze_cfg` : initialisation

```

47 let mut entry_envs: HashMap<BlockId, AbstractEnv<T::Domain>> = HashMap::new();
48 let mut exit_envs: HashMap<BlockId, AbstractEnv<T::Domain>> = HashMap::new();
49 let mut state_out = state.clone();
50
```

Algorithme 3 : Worklist d'un CFG, telle que l'implémente `analyze_cfg`**Input** : un CFG, un environnement d'entrée ρ_0 , un seuil k **Output** : l'environnement de sortie de chaque bloc atteint

```

1  $In[entry] \leftarrow \rho_0$ ;  $W \leftarrow [entry]$ ;
2 while  $W$  non vide do
3    $b \leftarrow$  tête de  $W$  (FIFO);
4    $Out[b] \leftarrow f_b^\#(In[b])$ ;
5   foreach successeur  $s$  de  $b$ , avec l'environnement raffiné  $\rho_s$  do
6     if  $In[s]$  absent then
7        $\nu \leftarrow \rho_s$ ;
8     else if  $b \rightarrow s$  est un arc arrière et c'est son  $k$ -ième passage ou plus then
9        $\nu \leftarrow In[s] \nabla \rho_s$ ;
10    else
11       $\nu \leftarrow In[s] \sqcup \rho_s$ ;
12    if  $\nu \neq In[s]$  then  $In[s] \leftarrow \nu$ ; enfile  $s$  s'il n'est pas déjà dans  $W$ ;

```

```

51 entry_envs.insert(cfg.entry, entry_env);
52
53 let mut worklist: VecDeque<BlockId> = VecDeque::new();
54 let mut in_worklist: HashSet<BlockId> = HashSet::new();
55 worklist.push_back(cfg.entry);
56 in_worklist.insert(cfg.entry);
57
58 let mut back_edge_counts: HashMap<BlockId, usize> = HashMap::new();

```

- `entry_envs` est la table In , `exit_envs` la table Out . Seul le bloc d'entrée reçoit un environnement : un bloc que rien n'atteint n'est jamais visité et n'a pas d'entrée dans `exit_envs`, ce qui revient à lui donner $\perp$.
- `state_out` est une seule copie du store d'état pour toute la passe. Nous y reviendrons à la section 3.12 : l'état n'est pas suivi bloc par bloc.
- `in_worklist` évite d'enfiler deux fois le même bloc ; `back_edge_counts` compte les passages par arc arrière, bloc cible par bloc cible (ce compteur déclenchera le widening).

Ensuite l'application de la fonction de transfert d'un bloc :

Extrait 3.12 – `src/engine/cfg_analyzer.rs`, l. 60–89 — `analyze_cfg` : transfert d'un bloc

```

60 while let Some(b) = worklist.pop_front() {
61   in_worklist.remove(&b);
62
63   let env_in = entry_envs[&b].clone();
64   let mut env_out = env_in;
65   // Memo is fixed for this pass; local mutations are discarded.
66   let mut memo_local = memo.clone();
67
68   if let Some(block) = cfg.blocks.get(&b) {
69     let mut ac = AnalysisCtx {
70       component,
71       state: &mut state_out,
72       memo: &mut memo_local,
73       heap,
74       query: ctx,
75       inter,

```

```

76     };
77     for stmt in &block.stmts {
78         transfer.exec_stmt(stmt, &mut env_out, &mut ac);
79     }
80     // A concise-arrow body `() => setN(1)` lowers its expression to a
81     // `Return` terminator, not an `ExprStmt` – but its side effects must
82     // still fire. Run them through the shared effect-firing path instead
83     // of fabricating a throwaway `Stmt::ExprStmt`.
84     if let Terminator::Return(return_expr) = &block.term {
85         transfer.exec_expr_effects(return_expr, &mut env_out, &mut ac);
86     }
87 }
88
89 exit_envs.insert(b, env_out.clone());

```

Les instructions du bloc sont exécutées une à une par le Transfer (chapitre 11); si le bloc se termine par `return e`, les effets de bord de `e` sont aussi exécutés (un effet `() => setN(1)` a son appel dans le `return`). Enfin, la propagation :

Extrait 3.13 – `src/engine/cfg_analyzer.rs`, l. 91–121 — `analyze_cfg` : propagation aux successeurs

```

91     for (succ, outgoing_env) in outgoing(cfg, b, &env_out) {
92         let is_back = cfg
93             .edges
94             .iter()
95             .any(|e| e.from == b && e.to == succ && matches!(e.kind, EdgeKind::Back));
96
97         let new_entry = match entry_envs.get(&succ) {
98             None => outgoing_env,
99             Some(existing) => {
100                 if is_back {
101                     let cnt = back_edge_counts.entry(succ).or_insert(0);
102                     *cnt += 1;
103                     if *cnt >= widen_threshold {
104                         existing.widen_to(&outgoing_env, thresholds)
105                     } else {
106                         existing.join(&outgoing_env)
107                     }
108                 } else {
109                     existing.join(&outgoing_env)
110                 }
111             }
112         };
113
114         if entry_envs.get(&succ) != Some(&new_entry) {
115             entry_envs.insert(succ, new_entry);
116             if in_worklist.insert(succ) {
117                 worklist.push_back(succ);
118             }
119         }
120     }
121 }

```

`outgoing` (`src/engine/cfg_analyzer.rs`, l. 126–150) rend chaque successeur avec l'environnement raffiné par la garde du Branch. La nouvelle entrée est l'environnement propagé s'il n'y en avait pas, son `join` avec l'ancienne sinon, et son `widening` à seuils sur un arc arrière dès le `widen_threshold`-ième passage (3 par défaut, `src/engine/fixpoint.rs`, l. 51–60). Le successeur n'est ré-enfilé que si son entrée a changé.

Exemple 3.3: La worklist sur la boucle locale

Déroulons `analyze_cfg` sur figure 3.4, en ne montrant que la composante numérique de `i`. Les seuils du composant sont les littéraux numériques du programme, $T = \{0, 1, 5\}$ (section 3.6). Le déroulé est fait à la main d'après le code; son résultat final est épinglé par le test `bounded_local_loop_is_precise` (`tests/widening_e2e.rs`, l. 124–137).

pas	bloc	entrée → sortie	effet sur les successeurs
1	B_0	$\rho_0 \rightarrow [0, 0]$	$In[B_1] := [0, 0]$
2	B_1	$[0, 0]$	$In[B_2] := [0, 0], In[B_3] := [5, 0] (\perp)$
3	B_2	$[0, 0] \rightarrow [1, 1]$	arrière n°1 : $[0, 0] \sqcup [1, 1] = [0, 1]$
4	B_3	$\perp$	<code>setTotal</code> n'ajoute rien
5	B_1	$[0, 1]$	$In[B_2] := [0, 1], In[B_3] := [5, 1] (\perp)$
6	B_2	$[0, 1] \rightarrow [1, 2]$	arrière n°2 : $[0, 1] \sqcup [1, 2] = [0, 2]$
7	B_3	$\perp$	rien
8	B_1	$[0, 2]$	$In[B_2] := [0, 2], In[B_3] := [5, 2] (\perp)$
9	B_2	$[0, 2] \rightarrow [1, 3]$	arrière n°3 : $[0, 2] \nabla_T [1, 3] = [0, 5]$
10	B_3	$\perp$	rien
11	B_1	$[0, 5]$	$In[B_2] := [0, 4], In[B_3] := [5, 5]$
12	B_2	$[0, 4] \rightarrow [1, 5]$	arrière n°4 : $[0, 5] \nabla_T [1, 5] = [0, 5]$, inchangé
13	B_3	$[5, 5]$	<code>setTotal</code> écrit $[5, 5]$; le slot devient $[0, 0] \sqcup [5, 5] = [0, 5]$

La worklist est vide après le pas 13. Au pas 11, l'arête « vrai » raffine $[0, 5]$ en $[0, 4]$ (`i` est entier, `i < 5` donne `i <= 4`) et l'arête « faux » en $[5, 5]$. Le slot `total` vaut $[0, 5]$: la valeur initiale 0 et la valeur écrite 5, réunies par `join`. C'est exactement la valeur qu'assère le test.

Les pas 4, 7 et 10 visitent B_3 pour rien : son entrée est vide à chaque fois, mais elle passe de $[5, 0]$ à $[5, 1]$ puis $[5, 2]$, et ces trois représentations de $\perp$ sont différentes pour `PartialEq` (section 3.3, piège sur les $\perp$ multiples). C'est un coût, pas une faute.

3.5.6 Le point fixe des tours de React

Le même schéma existe à l'échelle du composant. La boucle externe de `analyze_component_impl` (chapitre 13, et chapitre 2 pour sa lecture comme sémantique de React) exécute un tour complet (rendu, mémos, effets, handlers) à partir du store courant, et compare :

Extrait 3.14 – `src/engine/fixpoint.rs`, l. 485–497 — le test de post-point fixe de la boucle externe

```

485 // -- Convergence check -----
486 let new_state_incycle = state_from_render.join(&state_from_effects);
487 // Include cross-component state updates made by child effects/callbacks.
488 let external_updates = inter
489     .map(|i| i.shared_state.borrow().slice(comp_id))
490     .unwrap_or_else(StateStore::bottom);
491 let new_state = new_state_incycle
492     .join(&state_from_handlers)
493     .join(&external_updates);
494
495 if new_state.leq(&state) {
496     break;
497 }
```

`new_state` est $F^\sharp(\text{state})$: ce qu'un tour de React abstrait produit à partir de `state`. Le test `new_state.leq(&state)` est littéralement « $F^\sharp(x) \sqsubseteq x$ », le test de post-point fixe du théorème 3.2.

Tant qu'il échoue, `state` est remplacé par `new_state` (les deux premières fois) puis par un widening (section 3.6).

Piège

Le théorème ne garantit rien pour ce que le test *ne regarde pas*. Le test de la boucle externe porte sur le store d'état et lui seul ; le tas et les mémos sont recalculés à chaque tour mais n'entrent pas dans la comparaison (chapitre 13). De même, au-delà de 100 tours, le moteur fait un widening sans seuils puis sort *sans* refaire le test (`src/engine/fixpoint.rs`, l. 499–512) : l'état retenu n'est alors pas vérifié comme post-point fixe. L'argument de terminaison (section 3.6) dit que ce garde-fou ne devrait jamais servir. Au commit e67b10a, aucun test du dépôt ne le déclenche : c'est un filet de sécurité non exercé, pas un chemin validé.

3.6 Hauteur infinie et widening

3.6.1 Le problème

Le théorème 3.1 promet un nombre fini de pas si le treillis est de hauteur finie. Les intervalles ne le sont pas (figure 3.3), et tableau 3.1 montre la conséquence : sur Runaway, la suite $[0, 1], [0, 2], [0, 3], \dots$ est une chaîne strictement croissante qui ne s'arrête jamais. Le plus petit point fixe abstrait existe, c'est $[0, +\infty]$, mais Kleene ne l'atteint qu'à la limite. Il faut *extrapoler* : deviner une borne qui majore toute la suite, et vérifier ensuite que c'est un post-point fixe.

Définition 3.6: Widening, widening à seuils

Un *widening* sur L est un opérateur binaire ∇ tel que :

1. il majore le join : $a \sqcup b \sqsubseteq a \nabla b$ pour tous a, b ;
2. pour toute suite $y_0, y_1, \dots$, la suite $x_0 = y_0, x_{n+1} = x_n \nabla y_{n+1}$ est ultimement stationnaire.

Remplacer le join de Kleene par ∇ donne une suite qui se stabilise en un nombre fini de pas sur un post-point fixe, donc sur une réponse sound (théorème 3.2).

Le widening des intervalles saute à l'infini dès qu'une borne grandit :

$$[a, b] \nabla [c, d] = [c < a ? -\infty : a, d > b ? +\infty : b].$$

Le *widening à seuils* ∇_T , pour un ensemble fini T de nombres, saute au seuil le plus proche qui englobe la nouvelle borne, et à l'infini seulement s'il n'y en a pas :

$$[a, b] \nabla_T [c, d] = [\ell, h] \quad \text{avec} \quad \begin{aligned} \ell &= \max\{t \in T \cup \{-\infty\} \mid t \leq c\} \text{ si } c < a, \text{ sinon } a, \\ h &= \min\{t \in T \cup \{+\infty\} \mid t \geq d\} \text{ si } d > b, \text{ sinon } b. \end{aligned}$$

La première propriété garantit la correction : on ne perd jamais une valeur que le join aurait gardée. La seconde garantit la terminaison : pour ∇ , chaque borne ne peut sauter qu'une fois (vers l'infini) ; pour ∇_T , chaque borne ne peut prendre que des valeurs de $T \cup \{\pm\infty\}$ après son premier saut, et ne fait que croître, donc au plus $|T| + 1$ sauts par borne.

Exemple 3.4: Widening à la main

Avec $T = \{0, 1, 10\}$: $[0, 2] \nabla [0, 3] = [0, +\infty]$ (la borne haute grandit) ; $[0, 2] \nabla_T [0, 3] = [0, 10]$ (le plus petit seuil ≥ 3 est 10) ; $[0, 10] \nabla_T [0, 10] = [0, 10]$ (rien ne grandit) ; $[0, 10] \nabla_T [0, 11] = [0, +\infty]$ (aucun seuil ≥ 11). Avec $T = \{0, 1\}$, $[0, 2] \nabla_T [0, 3] = [0, +\infty]$.

3.6.2 Ce que fait le code : widen et widen_to

Le widening simple des intervalles est la formule ci-dessus, précédée des cas $\perp$:

Extrait 3.15 – src/domains/impls/interval.rs, l. 84–106 — le widening des intervalles

```

84
85  /// Widening: if other grows the bound beyond self, jump to  $\pm\infty$ .
86  pub fn widen(&self, other: &Self) -> Self {
87      if self.is_bottom() {
88          return *other;
89      }
90      if other.is_bottom() {
91          return *self;
92      }
93      Interval {
94          lo: if other.lo < self.lo {
95              f64::NEG_INFINITY
96          } else {
97              self.lo
98          },
99          hi: if other.hi > self.hi {
100              f64::INFINITY
101          } else {
102              self.hi
103          },
104          is_int: self.is_int && other.is_int,
105      }
106  }
```

Le widening à seuils (ADR-14) remplace $\pm\infty$ par le seuil le plus serré :

Extrait 3.16 – src/domains/impls/interval.rs, l. 108–146 — le widening à seuils

```

108  /// Threshold widening ("widening up to"). A bound that grows jumps to the
109  /// tightest enclosing threshold instead of  $\pm\infty$ ;  $\pm\infty$  is used only when no finite
110  /// threshold encloses the grown bound. `thresholds` need not be sorted.
111  ///
112  /// Sound: result  $\sqsubseteq$  self.hull(other) (bounds only ever loosen), and the set is
113  /// finite so the ascending chain still stabilises.
114  pub fn widen_to(&self, other: &Self, thresholds: &[f64]) -> Self {
115      if self.is_bottom() {
116          return *other;
117      }
118      if other.is_bottom() {
119          return *self;
120      }
121      let lo = if other.lo < self.lo {
122          // Largest threshold  $\leq$  other.lo, else  $-\infty$ .
123          thresholds
124              .iter()
125              .copied()
126              .filter(|&t| t <= other.lo)
127              .fold(f64::NEG_INFINITY, f64::max)
128      } else {
129          self.lo
130      };
131      let hi = if other.hi > self.hi {
132          // Smallest threshold  $\geq$  other.hi, else  $+\infty$ .
133          thresholds
134              .iter()
135              .copied()
```

```

136         .filter(|&t| t >= other.hi)
137         .fold(f64::INFINITY, f64::min)
138     } else {
139         self.hi
140     };
141     Interval {
142         lo,
143         hi,
144         is_int: self.is_int && other.is_int,
145     }
146 }

```

- Une borne qui n’a pas grandi est conservée telle quelle (`self.lo`, `self.hi`).
- Une borne haute qui grandit prend le plus petit seuil $\geq \text{other.hi}$: `filter` garde les candidats, `fold` avec `f64::min` à partir de $+\infty$ prend le minimum, et rend $+\infty$ si aucun seuil ne convient. La borne basse est symétrique. Les seuils n’ont pas besoin d’être triés.
- `is_int` reste vrai si les deux opérandes sont entiers ; une borne infinie ne rend pas l’intervalle non entier.

Aux autres composantes du produit `StateValue`, qui sont de hauteur finie (treillis plats, ensembles bornés de chaînes, `Stability`), le widening par défaut délègue au `join` (liste 3.2, et `src/domains/impls/mod.rs`, l. 48–50 pour les treillis plats).

3.6.3 D’où viennent les seuils

Les seuils sont récoltés une fois par composant, avant le point fixe (`src/engine/fixpoint.rs`, l. 280) :

Extrait 3.17 – `src/engine/fixpoint.rs`, l. 1185–1206 — la récolte des seuils

```

1185 /// Harvest the finite threshold set for "widening up-to" (see ADR-014).
1186 ///
1187 /// Collects numeric literals from the render CFG, all hook bodies, and useState
1188 /// init expressions – the constants against which guarded state growth is
1189 /// bounded. Over-collecting is harmless: the set stays finite (termination) and
1190 /// extra thresholds only add candidate bounds (precision, never unsoundness).
1191 fn collect_thresholds(render_cfg: &CFG, hooks: &[HookEntry]) -> Vec<f64> {
1192     let mut out: Vec<f64> = Vec::new();
1193     collect_lits_cfg(render_cfg, &mut out);
1194     for hook in hooks {
1195         match hook {
1196             HookEntry::State { init, .. } => collect_lits_expr(init, &mut out),
1197             HookEntry::Effect { body_cfg, .. } | HookEntry::Handler { body_cfg, .. } => {
1198                 collect_lits_cfg(body_cfg, &mut out)
1199             }
1200         }
1201     }
1202 }
1203 out.sort_by(|a, b| a.partial_cmp(b).unwrap_or(std::cmp::Ordering::Equal));
1204 out.dedup();
1205 out
1206 }

```

Tous les littéraux numériques du rendu, des corps d’effets et de handlers, et des initialiseurs de `useState` sont pris. L’ADR ne prévoyait que les littéraux des gardes et des initialiseurs ; en prendre plus est sans danger : l’ensemble reste fini (terminaison) et un seuil de plus n’est qu’une borne candidate de plus (précision, jamais soundness). Pour `Guarded`, $T = \{0, 1, 10\}$;

pour Runaway, $T = \{0, 1\}$.

3.6.4 Retour aux compteurs

Où le widening intervient-il ? Aux deux endroits où une suite peut croître sans fin : les arcs arrière d'un CFG (liste 3.13) et la boucle des tours de React :

Extrait 3.18 – `src/engine/fixpoint.rs`, l. 514–529 — le widening de la boucle externe

```

514     if iteration >= config.widen_threshold {
515         // widen_trace: render+effects only handler widening is not a bug.
516         // `or_insert_with` keeps the FIRST widening iteration (the most
517         // informative one for the witness chain).
518         for label in new_state_incycle.changed_labels(&state) {
519             widen_trace
520                 .entry(label)
521                 .or_insert_with(|| crate::engine::WidenEvent {
522                     iteration,
523                     writers: slot_writers.get(&label).cloned().unwrap_or_default(),
524                 });
525         }
526         state = state.widen_to(&new_state, &thresholds);
527     } else {
528         state = new_state;
529     }

```

Le compteur `iteration` n'augmente que lorsqu'un tour a changé l'état. Les deux premiers changements sont pris tels quels ; à partir du troisième (`widen_threshold = 3`), l'état est élargi. Rejouons tableau 3.1 avec le widening :

- Runaway. Tours 0 et 1 : le slot passe à $[0, 1]$ puis $[0, 2]$. Tour 2 : le tour produit $[0, 3]$, `iteration` vaut 3, donc $\text{state} = [0, 2] \nabla_{\{0,1\}} [0, 3] = [0, +\infty]$. Tour 3 : l'effet écrit $[1, +\infty]$, le join donne $[0, +\infty]$, le test de post-point fixe réussit. Le slot convergé vaut $[0, +\infty]$, ce qu'épingle `unbounded_counter_is_flagged_and_grows_to_infinity` (tests/widening_e2e.rs, l. 79–92).
- Guarded. Même début ; au tour 2, $[0, 2] \nabla_{\{0,1,10\}} [0, 3] = [0, 10]$. Tour 3 : la garde raffine $[0, 10]$ en $[0, 9]$, l'effet écrit $[1, 10]$, le join rend $[0, 10]$: post-point fixe. Le test `guarded_counter_converges_bounded` (tests/widening_e2e.rs, l. 94–122) épingle $[0, 10]$.

La sortie de l'analyseur porte la trace de ces deux calculs. Avec `--info --trace` (sortie élaguée des lignes `verified`) :

```

$ reactant check /tmp/ch03/counter.tsx --info --trace
Guarded (2 hooks) /tmp/ch03/counter.tsx
info widening-info (line 13:2) state `count` kept changing during analysis and was approximated to converge, so
  ↳ findings that depend on it may be imprecise
  ↳ state `count` is written here [hook:1] (line 13:2)
  ↳ the abstract value of state `count` kept growing and was widened at iteration 3
Runaway (2 hooks) /tmp/ch03/counter.tsx
warn infinite-loop [hook:0] (line 5:2) this effect keeps pushing state `count` (its deps do not provably gate it
  ↳ , so the effect can re-run every render) to new values on every run. Potential infinite render loop
  ↳ state `count` is written here [hook:1] (line 5:2)
  ↳ the abstract value of state `count` kept growing and was widened at iteration 3
info widening-info (line 5:2) state `count` kept changing during analysis and was approximated to converge, so
  ↳ findings that depend on it may be imprecise
  ↳ state `count` is written here [hook:1] (line 5:2)
  ↳ the abstract value of state `count` kept growing and was widened at iteration 3

! 1 warning(s) across 1 file(s).

```

Les deux composants ont subi un widening au tour où `iteration` a atteint 3 (l'Info widening-info le dit, car un widening est une perte de précision possible). Seul `Runaway` déclenche `infinite-loop` : son slot est non borné, celui de `Guarded` ne l'est pas. Le chapitre 19 détaille comment la règle lit cette différence ; le niveau `WARNING` (et non `ERROR`) s'expliquera à la section 3.7.

3.6.5 Le narrowing, et pourquoi `reactant-analyzer` s'en passe

La théorie classique complète le widening par une phase descendante, le *narrowing* : une fois un post-point fixe x atteint, on réapplique $F^\sharp$ quelques fois, $x \sqcap F^\sharp(x)$, pour redescendre vers le plus petit point fixe et récupérer la précision perdue par le saut à l'infini.

Décision : ADR-14 — seuils oui, narrowing non

ADR-14 prévoyait les deux : widening à seuils (partie 1) et narrowing (partie 2). La partie 1 est implémentée, sur la boucle externe et sur les arcs arrière d'`analyze_cfg`. La partie 2 a été abandonnée (révision du 28 juin 2026) sur un constat empirique : le compteur gardé est déjà précis ($[0, 10]$) grâce au raffinement de garde pendant la montée, et la boucle locale l'est devenue dès que le widening à seuils a été porté sur l'arc arrière interne ($[0, 5]$). Les deux techniques ne récupèrent que des bornes *littérales*, que les seuils capturent déjà en montant. Surtout, une phase descendante ne pourrait pas faire redescendre le store d'état, puisque les écritures s'y accumulent par `join` (section 3.11). Alternative écartée : un opérateur `narrow` dans `AbstractDomain` et une boucle descendante bornée ; il reste une infrastructure différée, à reconsidérer seulement si des bornes symboliques deviennent nécessaires.

Ce que ni les seuils ni le narrowing ne savent faire, c'est une borne symbolique. Remplaçons le littéral par une prop :

Extrait 3.19 – Une garde à borne symbolique (`/tmp/ch03/limit.tsx`, élagué de sa ligne d'import)

```
1 export function Limited({ limit }: { limit: number }) {
2   const [count, setCount] = useState(0);
3   useEffect(() => {
4     if (count < limit) setCount(count + 1);
5   }, [count, limit]);
6   return <div>{count}</div>;
7 }
```

```
$ reactant check /tmp/ch03/limit.tsx
Limited (2 hooks) /tmp/ch03/limit.tsx
  warn  infinite-loop [hook:0] (line 5:2) this effect keeps pushing state `count` (its deps do not provably gate it
    ↪ , so the effect can re-run every render) to new values on every run. Potential infinite render loop
```

Pour une valeur finie de `limit`, ce composant s'arrête. Mais la garde `count < limit` compare deux variables : le raffinement de garde ne reconnaît que « variable comparée à un littéral » (chapitre 12), aucun seuil ne vaut `limit`, et le widening saute à $+\infty$. Le `WARNING` est un faux positif, toléré par l'invariant. Le corriger demanderait un domaine *relationnel* (qui retient $\text{count} \leq \text{limit}$), ce que `reactant-analyzer` n'a pas (section 3.12).

Piège

Le widening n'est pas monotone et dépend de l'ordre des calculs : $[0, 1] \sqsubseteq [0, 2]$, et pourtant $[0, 1] \nabla [0, 2] = [0, +\infty]$ alors que $[0, 2] \nabla [0, 2] = [0, 2]$; un argument gauche plus précis donne un résultat moins précis. Élargir au tour 2 ou au tour 3 ne donne donc pas forcément le même résultat. C'est sans conséquence sur la soundness, qui ne repose que sur le test de post-point fixe final (théorème 3.2). Mais un changement anodin (un seuil de plus, un

widen_threshold différent, un ordre de visite des blocs différent) peut changer la précision, donc faire apparaître ou disparaître un faux positif.

3.7 Must, may, et la polarité d'un fait

3.7.1 Deux setters dans le rendu

Appeler un setter pendant le rendu est un défaut (chapitre 2). Comparons deux façons de le faire :

Extrait 3.20 – Un setter dans le rendu, inconditionnel puis conditionnel (/tmp/ch03/setter.tsx, élagué de sa ligne d'import)

```

1 export function Unconditional() {
2   const [n, setN] = useState(0);
3   setN(n + 1);
4   return <p>{n}</p>;
5 }
6
7 export function Conditional({ flag }: { flag: boolean }) {
8   const [n, setN] = useState(0);
9   if (flag) setN(1);
10  return <p>{n}</p>;
11 }
```

```

$ reactant check /tmp/ch03/setter.tsx
Conditional (1 hooks) /tmp/ch03/setter.tsx
  warn  setter-in-render [hook:0] (line 11:12) setter `setN` called directly in the render body, move this call
        ↳ into a useEffect or an event handler
        (1 trace step(s), rerun with --trace)
Unconditional (1 hooks) /tmp/ch03/setter.tsx
  error  setter-in-render [hook:0] (line 5:2) setter `setN` called directly in the render body, move this call into
        ↳ a useEffect or an event handler
        (1 trace step(s), rerun with --trace)

! 1 error(s), 1 warning(s) across 1 file(s).
```

Même règle, deux niveaux. Dans `Unconditional`, *tout* rendu appelle `setN` : le défaut a lieu dès que le composant s'affiche. Dans `Conditional`, il n'a lieu que si `flag` est vrai, ce que l'analyseur ne sait pas (`flag` est une prop, donc $\top$). Le premier fait est vrai sur *tous* les chemins, le second sur *certains*.

3.7.2 Pourquoi une sur-approximation ne suffit pas

Tout ce qui précède calcule des sur-approximations : l'ensemble abstrait contient tous les comportements possibles, et peut-être d'autres. Une sur-approximation prouve des *absences* (« count ne dépasse jamais 10 »), jamais des *présences*. Si l'abstrait dit « `setN` est peut-être appelé », c'est peut-être à cause d'un chemin impossible. Pour affirmer qu'un défaut *a lieu*, il faut l'autre sens : un fait vrai de toutes les exécutions, donc une sous-approximation de ce qui arrive à coup sûr.

Définition 3.7: Fait must, fait may, polarité

Un fait est *must* s'il est établi pour *toutes* les exécutions concrètes que l'abstraction décrit (sur tous les chemins, à tous les tours). Il est *may* s'il vaut pour *au moins une* exécution que l'abstraction admet, qui peut être une exécution impossible introduite par l'approximation.

$\top$ (l'inconnu) n'est ni l'un ni l'autre ; lu comme réponse à la question « le défaut a-t-il lieu ? », il ne peut compter que comme may. La *polarité* d'un fait, d'une relation ou d'une colonne de relation (définition 14.1) dit lequel des deux elle porte : exacte, must, may ou $\top$.

Les deux polarités se calculent avec les mêmes outils, en sens inverse. Une analyse may joint aux jonctions (union des possibilités) et part de $\perp$: plus petit point fixe. Une analyse must intersecte aux jonctions (ce qui est vrai de *chaque* prédécesseur) et part de « tout est vrai » : plus grand point fixe. Nous en verrons un exemple avec la dominance (section 3.9).

3.7.3 Ce que fait le code : des verdicts typés par leur polarité

La couche règles (chapitre 17) inscrit la polarité dans les types (ADR-21). Une primitive must rend un `MustResult` :

Extrait 3.21 – `src/rules/api/query.rs`, l. 111–135 — les verdicts polarisés

```

111 /// A three-valued MUST verdict. `All` carries the certified token (the only way
112 /// to obtain a `Certified` for a single-verdict primitive); `Some`/`None` are
113 /// MAY facts with no path to an `Error`.
114 ///
115 /// (Deviates from the ADR-021 §1 literal `All(T)`: the token lives *inside* `All`
116 /// so `must_*` and the `Vec<Certified<_>>` primitives share one minting story.)
117 #[derive(Debug, Clone, PartialEq)]
118 pub enum MustResult<T> {
119     /// Proven on all paths – carries the minted proof.
120     All(Certified<T>),
121     /// Proven on some but not all paths – a MAY fact (raw payload).
122     Some(T),
123     /// No qualifying evidence at all.
124     None,
125 }
126
127 /// A MAY fact. There is no path from `May<_>` to an `Error`.
128 #[derive(Debug, Clone, Copy, PartialEq, Eq)]
129 pub struct May<T>(pub T);
130
131 impl<T> May<T> {
132     pub fn into_inner(self) -> T {
133         self.0
134     }
135 }
```

- `All` : établi sur tous les chemins. C'est le seul cas qui transporte un `Certified` (définition 17.1), le jeton sans lequel aucune règle ne peut émettre une `ERROR`.
- `Some` : établi sur certains chemins seulement, un fait may, sans jeton.
- `None` : aucune évidence.
- `May<T>` enveloppe un fait may ; aucune fonction ne le transforme en `Certified`.

Le contrat de toutes les primitives tient en trois clauses (`src/rules/api/query.rs`, l. 417–421) : chaque primitive rend un verdict typé par sa polarité ; $\top$ se replie en may à l'intérieur ; seules les primitives must frappent un `Certified`. La deuxième clause est celle qui protège la soundness des `ERROR` : un inconnu ne devient jamais une certitude.

Voici comment `setter-in-render` fabrique la différence observée plus haut :

Extrait 3.22 – `src/rules/impls/setter_in_render.rs`, l. 147–162 — `setter-in-render` : l'`ERROR` demande deux musts

```

147      // Unconditional call  $\implies$  a `Certified` proof – the only path to
148      // Error, and it takes two musts. The phase must be `Sync`: a  $\top$ 
149      // row is never certain, whatever block it sits in, because the
150      // proof would be about the block, not about the timing. And
151      // calling the variable must provably write: a local helper that
152      // only *carries* a captured setter into a JSX handler writes
153      // nothing when called, and dominance cannot upgrade a may.
154      let certain_write = call.class == SetterCallPhase::Sync
155        && cs_vars.get(&call.var).is_none_or(|p| p.must_write);
156      let proof = certain_write
157        .then_some(call.block_id)
158        .flatten()
159        .and_then(|bid| match exit_dom.certify(bid) {
160          MustResult::All(c) => Some(c),
161          _ => None,
162        });

```

La preuve a deux conjoints `must` : l'appel a lieu pendant le rendu, et il écrit à coup sûr (`certain_write`) ; puis son bloc domine toutes les sorties du rendu (`exit_dom.certify`, section 3.9). Si l'un manque, `proof` vaut `None` et le diagnostic reste un `WARNING`. Dans `Conditional`, le bloc `then` ne domine pas le `return` : pas de jeton, `WARNING`.

Piège

Un fait `must` sur *une partie* de la conclusion ne suffit pas. La doctrine du projet (CLAUDE.md) exige, pour une `ERROR`, la preuve « de *toute* la conclusion, pas d'un seul conjoint ». `infinite-loop` en donne un exemple : pour `Runaway`, la croissance du slot jusqu'à $+\infty$ est le résultat d'un widening, c'est-à-dire d'une extrapolation sur-approximante ; elle ne prouve pas que l'effet re-tourne indéfiniment. La règle n'émet donc qu'un `WARNING` ; seule la preuve d'un cycle d'effets (chapitre 19) atteint l'`ERROR`. L'écart entre divergence de valeur et divergence de référence est suivi par l'#144.

3.8 Soundness

Nous pouvons maintenant donner à la promesse de l'introduction (définition 1.4) sa forme exacte.

Définition 3.8: Soundness

Une analyse est *sound* lorsque ce qu'elle calcule *sur-approxime* la sémantique concrète : pour tout programme, $\text{lfp}F \subseteq \gamma(a)$ où a est le résultat abstrait. Pour un outil de diagnostic, cela se traduit ainsi : si le programme peut exhiber un défaut qu'une règle vise, l'analyse le signale (à un niveau ou à un autre), ou signale explicitement qu'elle n'a pas pu regarder. Un défaut réel non signalé, un *faux négatif* (FN), est interdit ; un signalement sans défaut réel, un *faux positif* (FP), est toléré.

Chaque niveau de diagnostic en tire une contrainte différente.

- **Le silence.** Une règle muette affirme l'absence du défaut. Cette affirmation est une propriété de sur-approximation : elle n'est vraie que si tous les calculs amont ont été corrects (définition 3.3) et ont atteint un post-point fixe (théorème 3.2). Sous `--info`, les lignes `verified` rendent ce silence visible.
- **Error.** Un défaut certain dès que le code s'exécute. C'est une affirmation `must`, qui ne peut pas venir d'une sur-approximation : elle demande un `Certified` (section 3.7).
- **Warning.** Un défaut possible (fait `may`), ou un fait certain au coût incertain. Tout ce que

la sur-approximation ne permet pas d'exclure et que rien ne permet de certifier tombe ici.

- **Info.** Une limite de l'analyse, ou un motif qui a l'air voulu. Quand l'analyse n'a pas pu lire du code (un inlining tronqué, par exemple), elle le dit par une INFO et suspend ses assurances *verified* (chapitre 20) : un silence qui ne vaut plus preuve est retiré plutôt que maintenu.

Exemple 3.5: Un faux positif toléré

```
1 export function Dead() {
2   const [n, setN] = useState(0);
3   const k = 3;
4   useEffect(() => {
5     if (k > 10) setN(n + 1);
6   }, [n]);
7   return <div>{n}</div>;
8 }
9
```

```
$ reactant check /tmp/ch03/dead.tsx
Dead (2 hooks) /tmp/ch03/dead.tsx
  warn infinite-loop [hook:0] (line 6:2) this effect keeps pushing state `n` (its deps do not provably gate
    ↳ it, so the effect can re-run every render) to new values on every run. Potential infinite render loop
```

La branche est morte : k vaut 3. Le raffinement de $k > 10$ donne bien $\perp$ à k sur la branche vraie, mais un raffinement vers $\perp$ ne rend pas le bloc inatteignable : le bloc est exécuté, seule la variable gardée y vaut $\perp$, et $\text{setN}(n + 1)$ écrit une vraie valeur (chapitre 12). Le WARNING est un FP : imprécis, sound.

Remarque

La soundness de *reactant-analyzer* n'est pas un théorème prouvé une fois pour toutes ; c'est un invariant tenu opération par opération : join et widening majorent le join ($\nabla_T \ll \sqsubseteq_{\text{hull}}$), une variable inconnue vaut $\top$, une écriture de slot est jointe et jamais remplacée, un appel opaque rend $\top$, un must ne compte pas un chemin d'exception comme couvert. Le chapitre 13 en dresse l'inventaire. Là où la soundness dépend d'une hypothèse du modèle, une règle dédiée surveille l'hypothèse (soundness en couches, définition 2.2). Les décisions de ne *pas* changer certains comportements pour des raisons de soundness sont consignées dans ADR-20.

Piège

« Tolérer les FP » ne veut pas dire « les ignorer ». Un FP au niveau ERROR est une faute : l'ERROR promet un défaut certain, et le contrat de la section 3.7 existe pour l'empêcher. Les FP tolérés sont des WARNING (ou des INFO) produits par l'imprécision de la sur-approximation.

3.9 CFG, dominance, « sur tous les chemins »

3.9.1 Un hook après un return anticipé

Extrait 3.23 – Un hook sauté par un return anticipé (/tmp/ch03/hook.tsx, élagué de sa ligne d'import)

```
1 export function Profile({ user }: { user: string | null }) {
2   if (user === null) {
```

```

3   return <p>anonymous</p>;
4   }
5   const [name, setName] = useState(user);
6   useEffect(() => {
7     document.title = name;
8   }, [name]);
9   return <p>{name}</p>;
10  }

```

```

$ reactant check /tmp/ch03/hook.tsx --trace
Profile (2 hooks) /tmp/ch03/hook.tsx
  error conditional-hook [hook:0] (line 7:8) this hook is called conditionally (not on every render path)
    → guarded by a condition evaluated here, so some render paths skip the hook (line 4:6)
  error conditional-hook [hook:1] (line 8:2) this hook is called conditionally (not on every render path)
    → guarded by a condition evaluated here, so some render paths skip the hook (line 4:6)

! 2 error(s) across 1 file(s).

```

Les règles des hooks exigent qu'un hook soit appelé à chaque rendu. Sur le CFG du rendu (figure 3.5), cela veut dire : *tout* chemin de l'entrée à une sortie passe par le bloc du hook. Ce « tout » est une quantification sur une infinité potentielle de chemins ; la dominance la ramène à un calcul fini.

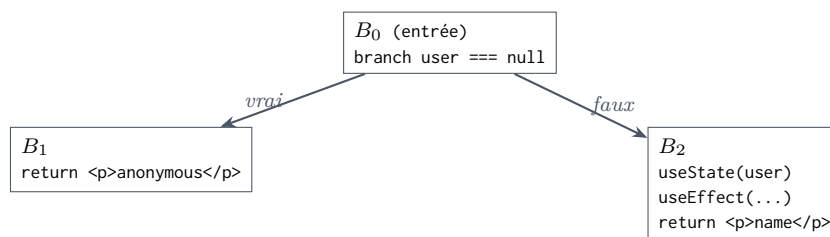


FIGURE 3.5 – CFG schématique du rendu de Profile. $\text{Dom}(B_1) = \{B_0, B_1\}$ et $\text{Dom}(B_2) = \{B_0, B_2\}$: B_2 , qui porte les hooks, ne domine pas la sortie B_1 .

Définition 3.9: Dominance, post-dominance

Dans un CFG d'entrée e , un bloc a *domine* un bloc b si tout chemin de e à b passe par a . On note $\text{Dom}(b)$ l'ensemble des dominateurs de b ; c'est la plus grande solution du système

$$\text{Dom}(e) = \{e\}, \quad \text{Dom}(b) = \{b\} \cup \bigcap_{p \in \text{pred}(b)} \text{Dom}(p).$$

Un ensemble de blocs S est *sur tous les chemins* (il post-domine l'entrée) si tout chemin de e à une sortie rencontre S . Un bloc s'exécute à chaque passage dans le CFG exactement quand il domine toutes les sorties atteignables.

Le système de la dominance est une analyse must : l'opérateur de jonction est l'intersection, et l'on part de « tous les blocs dominant tout » pour descendre. C'est donc le *plus grand* point fixe qu'on cherche, par une itération duale de Kleene (définition 3.5) : le treillis est celui des parties de blocs ordonné par $\supseteq$, fini, et l'itération termine.

3.9.2 Ce que fait le code

`compute_dominators` est cet algorithme itératif classique, parcouru en ordre postfixe inverse (RPO) :

Extrait 3.24 – `src/engine/dominance.rs`, l. 22–57 — le plus grand point fixe de la dominance

```

22     let rpo = rpo(cfg);
23
24     let mut changed = true;
25     while changed {
26         changed = false;
27         for &b in &rpo {
28             if b == cfg.entry {
29                 continue;
30             }
31             let preds = cfg.predecessors(b);
32             if preds.is_empty() {
33                 continue;
34             }
35
36             let new_dom_base: HashSet<BlockId> = preds
37                 .iter()
38                 .filter_map(|p| dom.get(p))
39                 .fold(None::

```

Chaque bloc autre que l'entrée part de `all_blocks` (lignes 16 à 20, hors extrait) et descend par intersection. Un bloc inatteignable n'est pas dans le RPO et garde « tous les blocs » : il est dominé par tout, ce qui est neutre pour un fait `must` (il ne s'exécute jamais). Pour éviter de refaire ce calcul à chaque question, `DominatorTree` (`src/engine/dominance.rs`, l. 94–111) le mémorise.

La couche règles consomme la dominance sous une forme typée, qui choisit les sorties une fois pour toutes :

Extrait 3.25 – `src/rules/api/query.rs`, l. 672–706 — `ExitDominance` : négation `may` et certification `must`

```

672
673     /// `true` when `block` may be skipped on some render path – the *rule-facing*
674     /// negation of [Self::certify], and deliberately not its MustResult::None
675     /// case: a CFG with no reachable exit proves nothing in either direction, so
676     /// nothing is skippable there.
677     pub fn may_be_skipped(&self, block: BlockId) -> bool {
678         !self.exits.is_empty()
679         && self
680             .exits
681             .iter()
682             .any(|&exit| !self.domtree.dominates(block, exit))

```

```

683 }
684
685 /// `All` iff `block` dominates every render exit (executes unconditionally);
686 /// `None` otherwise.
687 pub fn certify(&self, block: BlockId) -> MustResult<DominatesAllExits> {
688     if self.exits.is_empty() {
689         return MustResult::None;
690     }
691     if self
692         .exits
693         .iter()
694         .all(|&exit| self.domtree.dominates(block, exit))
695     {
696         // No provenance: a dominance relation over exits has no single
697         // position – the block it holds is on the evidence.
698         MustResult::All(Certified::mint(
699             DominatesAllExits { block },
700             Provenance::default(),
701         ))
702     } else {
703         MustResult::None
704     }
705 }
706 }

```

- `certify` rend `All` (avec un `Certified`) si le bloc domine *toutes* les sorties : c'est l'arme de *setter-in-render* (liste 3.22).
- `may_be_skipped` en est la négation : il existe une sortie que le bloc ne domine pas. Pour *Profile*, B_2 ne domine pas B_1 ; le hook est sauté sur au moins un chemin. Le défaut visé est structurel (la règle des hooks porte sur la forme du code, pas sur les valeurs) : l'existence de ce chemin dans le CFG suffit à l'établir, et *conditional-hook* l'émet en `ERROR` (définition 20.3).
- Un CFG sans sortie atteignable ne prouve rien dans aucun sens : `certify` rend `None`, et `may_be_skipped` rend faux.

Piège

Les sorties comptées sont les *return atteignables* (`src/rules/api/query.rs`, l. 652–671). Le *lowering* scelle la fin d'un corps par un `return undefined` (ADR-25) ; quand les deux branches d'un `if/else` ont déjà retourné, cette queue est orpheline. La compter ferait croire que tout hook antérieur est sauté, au niveau `ERROR` : un FP interdit par la section 3.8.

`on_all_paths` (`src/engine/dominance.rs`, l. 69–92) répond à la question voisine « ces blocs sont-ils sur tous les chemins ? » par un parcours qui évite l'ensemble : atteindre une feuille (un `return`, mais aussi un `throw`) sans le rencontrer, c'est exhiber un chemin qui s'échappe. Sa polarité est *must* : un chemin d'exception compte comme un chemin qui évite l'ensemble. Une troisième implémentation du même quantificateur existe, l'analyse *must* avant de `must_setter_on_all_paths` (`src/rules/api/query.rs`, l. 576–621), qui calcule elle aussi un plus grand point fixe booléen ; le chapitre 17 la présente.

3.10 Intra- et inter-procédural, inlining

Tout ce qui précède opère sur un CFG. Or un composant appelle des fonctions : des hooks personnalisés, des utilitaires, des composants enfants. Une analyse *intra-procédurale* traite chaque fonction isolément et remplace un appel par ce qu'elle sait sans le lire, c'est-à-dire $\top$:

sound, mais aveugle. Une analyse *inter-procédurale* suit les appels. Deux familles de techniques existent : calculer un *résumé* de chaque fonction, réutilisé à chaque appel ; ou *inliner*, c'est-à-dire greffer le corps de la fonction appelée au site d'appel et analyser le tout comme un seul CFG. L'inlining est la plus précise (chaque appel est analysé dans son contexte) et la plus coûteuse (le CFG grossit, la récursion doit être coupée).

Extrait 3.26 – Un hook personnalisé qui boucle (/tmp/ch03/custom.tsx, élagué de sa ligne d'import)

```

1 function useTicker(start: number) {
2   const [t, setT] = useState(start);
3   useEffect(() => {
4     setT(t + 1);
5   }, [t]);
6   return t;
7 }
8
9 export function Clock() {
10  const t = useTicker(0);
11  return <p>{t}</p>;
12 }

```

```

$ reactant check /tmp/ch03/custom.tsx
Clock (1 hooks) /tmp/ch03/custom.tsx
  warn  infinite-loop [hook:1] (line 5:2) this effect keeps pushing state `t` (its deps do not provably gate it, so
    ↳ the effect can re-run every render) to new values on every run. Potential infinite render loop
      (2 trace step(s), rerun with --trace)

```

Clock ne contient qu'un hook, l'appel à useTicker. Le diagnostic désigne pourtant [hook:1], à la ligne 5 de useTicker : le moteur a greffé le corps du hook dans le rendu de Clock, décalé ses labels de hooks, et analysé le compteur comme s'il était écrit dans le composant. reactant-analyzer fait ce choix partout où il dispose du source.

- **Hooks personnalisés** : greffés entiers au site d'appel, labels décalés, paramètres substitués (expand_custom_hooks, src/engine/fixpoint.rs, l. 884–1142). Un hook qui s'appelle lui-même, ou un second appel du même hook dans le même composant, reste opaque ($\top$). Le commentaire du code l'avoue (« *second call stays opaque FN, not FP* ») ; la sortie le compense : appeler useTicker deux fois (/tmp/ch03/twice.tsx) produit une INFO analysis-limit sur le second appel (« *hook 'useTicker' was not found in the registry* », message trompeur puisque le hook y est) et la ligne suspended analysis-limit 9 passing check(s) withheld.
- **Hooks de bibliothèque sans source** : un registre de résumés (SummaryRegistry) donne leur valeur abstraite ; c'est la technique par résumé, réservée à ce cas.
- **Utilitaires** : greffés en position d'instruction, au plus une fois par nom et par CFG, dans un budget de max_inline_depth = 8 greffes par CFG (src/engine/fixpoint.rs, l. 1611–1634) ; un budget épuisé laisse les appels restants opaques et suspend les assurances du composant (section 3.8, rubrique INFO).
- **Composants enfants** : analysés de haut en bas avec les props que le parent leur passe, et leurs écritures dans l'état du parent remontent par un store partagé (ADR-12).

Les mécanismes de greffe (*splice*, renommage, identités) sont l'objet du chapitre 8, l'analyse entre composants et entre fichiers celui du chapitre 16. Retenons ici leur statut théorique : l'inlining ne change rien au cadre de ce chapitre, il agrandit le CFG sur lequel on calcule le point fixe ; et chaque endroit où il renonce (récursion, budget, callee inconnu) retombe sur $\top$, donc reste sound.

3.11 Mises à jour fortes et faibles

Extrait 3.27 – Deux écritures successives (/tmp/ch03/weak.tsx, élagué de sa ligne d'import)

```
1 export function Twice() {
2   const [n, setN] = useState(0);
3   useEffect(() => {
4     let x = 1;
5     x = 5;
6     setN(x);
7     setN(7);
8   }, []);
9   return <div>{n}</div>;
10 }
```

Après $x = 5$, l'ancienne valeur de x est perdue : il est exact de *remplacer* $[1, 1]$ par $[5, 5]$. C'est une *mise à jour forte*. Elle n'est correcte que si l'écriture désigne un unique lieu de stockage concret, qu'elle a certainement lieu, et que l'abstraction ne décrit pas plusieurs instants à la fois. Quand l'une de ces conditions manque, il faut une *mise à jour faible* : joindre la nouvelle valeur à l'ancienne, $v \leftarrow v \sqcup v'$, ce qui dit « ce lieu vaut l'ancienne valeur ou la nouvelle ».

reactant-analyzer emploie les deux, selon ce que le lieu abstrait représente.

- **Variables locales : forte.** L'environnement est calculé bloc par bloc ; une affectation remplace la liaison (AbstractEnv::extend est un insert, src/domains/stores/abstract_env.rs, l. 100–103). Les chemins sont réunis par join aux jonctions du CFG.
- **Slots d'état : faible.** Un slot abstrait décrit *toutes* les valeurs que l'état prend au fil de *tous* les tours de React, pas sa valeur à un instant. Écrire ne peut donc qu'ajouter :

Extrait 3.28 – src/domains/stores/state_store.rs, l. 29–33 — l'écriture d'un slot est un join

```
29 /// Monotone update: `self[label] = self[label] \sqcup val`.
30 pub fn update(&mut self, label: HookLabel, val: D) {
31   let current = self.get(label);
32   self.0.insert(label, current.join(&val));
33 }
```

- **Champs du tas : faible.** Un objet abstrait est identifié par son site d'allocation (définition 6.2) et représente tous les objets créés à ce site ; une écriture $\text{obj.f} = v$ peut n'avoir lieu que sur un chemin, et obj peut désigner plusieurs sites. Le code joint donc, et le dit : « *join, never replace* » (src/domains/interp/interpreter.rs, l. 150–154).

Sur Twice, le slot n reçoit $[0, 0]$ (amorçage), puis $[5, 5]$, puis $[7, 7]$: il vaut $[0, 0] \sqcup [5, 5] \sqcup [7, 7] = [0, 7]$ (calcul fait d'après liste 3.28). En React, les deux appels sont regroupés (batching) et le rendu suivant voit 7 ; l'abstrait contient 0 et 7, et aussi 1, ..., 6 : une perte de précision, pas d'information fausse. C'est aussi pourquoi le narrowing ne pourrait pas faire redescendre un slot (section 3.6) : aucune écriture n'est jamais retirée.

```
$ reactant check /tmp/ch03/weak.tsx
Twice (2 hooks) /tmp/ch03/weak.tsx
warn unnecessary-rerender [hook:0] (line 5:2) mount-only effect sets state `n` to a constant different from its
  ↳ initial value, which costs one extra rerender on mount; consider initialising directly with the target value
(1 trace step(s), rerun with --trace)
```

La règle qui répond ici ne lit pas le slot joint ; elle parcourt les appels de setter de l'effet de montage et évalue leurs arguments un par un (src/rules/impls/unnecessary_rerender.rs, l. 110–144, et chapitre 18). La mise à jour faible est la bonne abstraction du store, pas la seule source

d'information des règles : quand une question porte sur *une* écriture, le moteur la conserve à part, site par site (définition 14.2).

Piège

Rendre fortes les écritures d'état « parce que le dernier `setN` gagne » serait un faux négatif en puissance : le slot ne décrirait plus les valeurs vues par les rendus intermédiaires, ni celles écrites par un autre effet ou un handler au même tour. La règle « dernier set gagne » vaut pour un tour concret donné ; le slot abstrait les résume tous.

3.12 Ce que `reactant-analyzer` fait, et ne fait pas

Un analyseur par interprétation abstraite se caractérise par une poignée de choix : ce qu'il distingue (flot, chemins, contextes), ce que ses domaines savent exprimer, et comment il force la terminaison. Tableau 3.2 résume ceux de `reactant-analyzer` ; chacun a été rencontré dans ce chapitre.

Dimension	Choix de <code>reactant-analyzer</code>
Sensibilité au flot (variables)	Oui : un environnement par entrée et sortie de bloc (<code>entry_envs</code> , <code>exit_envs</code>), mises à jour fortes.
Sensibilité au flot (état)	Non : un seul store par passe (<code>state_out</code>), partagé par tous les blocs, et un seul store pour tous les tours ; mises à jour faibles.
Sensibilité aux chemins	Non, sauf le raffinement de garde : les chemins sont joints aux jonctions ; une garde « variable comparée à un littéral » raffine chaque branche, sans jamais rendre une branche morte.
Forme de l'IR	Pas de SSA : les variables sont des noms, une affectation rebinde le nom dans l'environnement (chapitre 6).
Domaines numériques	Non relationnels : un intervalle par variable ou par slot ; aucune relation entre deux variables (d'où le FP de <code>Limited</code>).
Inter-procédural	Inlining des hooks, des utilitaires et des enfants, coupé à la récursion et par un budget ; résumés pour les hooks de bibliothèque ; $\top$ ailleurs.
Terminaison	Widening à seuils après 3 changements, sur les arcs arrière et entre tours ; widening sans seuils et arrêt au-delà de 100 tours. Pas de narrowing (ADR-14).
Polarité	Sur-approximation (<code>may</code>) pour le silence et les <code>WARNING</code> ; faits must certifiés (dominance, analyses must avant) pour les <code>ERROR</code> .

TABLE 3.2 – Les choix d'interprétation abstraite de `reactant-analyzer`.

Deux de ces choix méritent une phrase de justification. L'état est insensible au flot parce que la question que posent les règles porte sur l'ensemble des valeurs qu'un slot prend au cours de la vie du composant, pas sur sa valeur en un point du rendu : `React` ne change un slot qu'entre deux rendus, et le résumé de tous les tours est précisément ce qu'il faut. Les domaines sont non relationnels parce que les bornes que ADR-14 a rencontrées sont des littéraux, que les seuils capturent déjà ; un domaine relationnel (octogones, polyèdres) ne se justifierait que pour des bornes symboliques comme celle de `Limited`, cas que l'ADR laisse ouvert.

Les chapitres suivants reprennent chaque ligne du tableau à la *taille réelle* du code : les domaines de base (chapitre 9), le produit `StateValue` et la stabilité (chapitre 10), le transfert et les stores (chapitre 11), la `worklist` et la dominance (chapitre 12), la boucle externe (chapitre 13).

Résumé du chapitre

- Aucun analyseur ne peut calculer exactement les comportements d'un composant (taille, indécidabilité). *reactant-analyzer* *sur-approxime* : l'abstrait contient tout le concret. Son silence est une preuve ; ses erreurs sont des faux positifs, jamais des faux négatifs (définition 3.8).
- Un domaine abstrait est un treillis (définition 3.2) : ordre partiel (`PartialOrd`), `join`, `meet`, $\perp$, $\top$. Sa concrétisation γ dit ce que signifie chaque valeur ; une opération est correcte si elle ne perd aucune valeur concrète (définition 3.3). Trait `AbstractDomain` (`src/domains/mod.rs`), `BoolVal`, `flat_lattice!`, `Interval`.
- Le résultat d'une analyse est un post-point fixe d'équations de flot (définition 3.5) ; tout post-point fixe d'une fonction abstraite correcte est *sound* (théorème 3.2). `analyze_cfg` est une worklist FIFO sur un CFG ; la boucle externe de `analyze_component_impl` teste `new_state.leq(&state)`.
- Les intervalles sont de hauteur infinie : le widening à seuils (définition 3.6, `Interval::widen_to`, seuils récoltés par `collect_thresholds`) force la terminaison après `widen_threshold = 3` changements. Pas de narrowing (ADR-14) : le raffinement de garde et les seuils récupèrent les bornes littérales, pas les bornes symboliques.
- Une `ERROR` demande un fait *must* (définition 3.7) ; `MustResult::All` est le seul porteur de `Certified` ; $\top$ se replie en *may*. La dominance (définition 3.9, `src/engine/dominance.rs`, `ExitDominance`) est l'analyse *must* de référence, un plus grand point fixe.
- L'environnement est sensible au flot et mis à jour fortement ; l'état et le tas sont insensibles au flot et mis à jour faiblement ; l'inlining rend l'analyse inter-procédurale, et chaque renoncement retombe sur $\top$.

Le chapitre 4 situe ces mécanismes dans l'architecture du code ; la partie III (chapitre 9 à chapitre 11) reprend les domaines un par un, la partie IV (chapitre 12 et suivants) le moteur.

Exercices

Exercice 3.1: Le treillis plat

(a) Avec liste 3.4, que rend `partial_cmp` sur $(\perp, \top)$, sur $(\top, \perp)$, sur $(\text{True}, \text{False})$? (b) Calculer `One(c, 0)  $\sqcup$  One(c, 1)` et `One(c, 0)  $\sqcap$   $\top$` dans `SetterVal`. (c) Pourquoi le widening d'un treillis plat peut-il être le `join` ?

Solution

(a) $(\perp, \top)$: les éléments diffèrent, le deuxième bras s'applique (« $\perp$ à gauche »), `Some(Less)` ; $(\top, \perp)$: ni égalité, ni deuxième bras, le troisième donne `Some(Greater)` ; $(\text{True}, \text{False})$: aucun bras, `None`. (b) Deux éléments intermédiaires distincts : $\top$. Le `meet` avec $\top$ rend l'autre opérande, `One(c, 0)`. (c) La hauteur est 2 : toute chaîne strictement croissante a au plus trois éléments, donc la suite des joins est stationnaire ; le `join` vérifie les deux conditions de la définition 3.6.

Exercice 3.2: Calculs d'intervalles

Calculer `[1, 3]  $\sqcup$  [2, 9]`, `[1, 3]  $\sqcap$  [2, 9]`, `[0, 4]  $\sqcup$   $\perp$` , `[1, 3] + [2, 9]`, et dire si `[1, 3]  $\sqsubseteq$  [2, 9]`. Le `join` `[1, 3]  $\sqcup$  [7, 9]` est-il exact au sens de l'exemple 3.2 ?

Solution

$[1, 9]$; $[2, 3]$; $[0, 4]$ ($\perp$ est neutre); $[3, 12]$; non, les deux intervalles sont incomparables (`partial_cmp` rend `None`). $[1, 3] \sqcup [7, 9] = [1, 9]$ n'est pas exact : $\gamma([1, 9])$ contient 5, qui n'est dans aucun des deux opérandes.

Exercice 3.3: Widening à seuils

Avec $T = \{10, 20, 100\}$, calculer successivement $[0, 5] \nabla_T [0, 12]$, puis le résultat $\nabla_T [0, 25]$, puis le résultat $\nabla_T [0, 101]$. Calculer aussi $[0, 5] \nabla_T [-3, 5]$. Combien de sauts la borne haute peut-elle faire au plus ?

Solution

$[0, 20]$ (plus petit seuil ≥ 12); $[0, 100]$; $[0, +\infty]$ (aucun seuil ≥ 101). Pour la borne basse, aucun seuil n'est ≤ -3 : $[-\infty, 5]$. Après son premier saut, la borne haute ne prend que des valeurs de $T \cup \{+\infty\}$ et ne fait que croître : au plus $|T| + 1 = 4$ sauts. C'est l'argument de terminaison de ADR-14.

Exercice 3.4: Une worklist à la main

On analyse le corps `let i = 10; while (i > 0) { i = i - 1; }`, de CFG identique à figure 3.4, avec $T = \{0, 1, 10\}$ et un widening au troisième passage par l'arc arrière, comme `analyze_cfg`. Donner l'entrée finale de l'en-tête et la valeur de `i` à la sortie. Refaire le calcul avec le widening sans seuils.

Solution

En-tête : $[10, 10]$, puis $[9, 10]$ et $[8, 10]$ (joins, passages 1 et 2). Passage 3 : le corps rend $[7, 9]$ et $[8, 10] \nabla_T [7, 9] = [1, 10]$ (plus grand seuil ≤ 7). Passage 4 : la branche vraie raffine $[1, 10]$ par `i > 0` (entier) en $[1, 10]$, le corps rend $[0, 9]$, et $[1, 10] \nabla_T [0, 9] = [0, 10]$. L'en-tête vaut $[0, 10]$; la branche vraie y donne encore $[1, 10]$ (inchangé, la worklist se vide) et la branche fausse $[0, 10] \sqcap [-\infty, 0] = [0, 0]$: `i` vaut $[0, 0]$ à la sortie, la réponse exacte. Sans seuils, le passage 3 donne $[-\infty, 10]$, qui est déjà un post-point fixe, et la sortie vaut $[-\infty, 0]$: correct, mais la borne basse est perdue.

Exercice 3.5: Dominance et niveau

Pour Conditional (liste 3.20), dessiner le CFG du rendu, calculer Dom de chaque bloc, et en déduire ce que rendent `certify` et `may_be_skipped` pour le bloc de `setN(1)`. Retrouver le niveau observé.

Solution

`lower_if` (`src/lowering/cfg_builder.rs`, l. 544–585) scelle toujours un bloc `then` et un bloc `else`, même sans `else` source : quatre blocs, pas trois. L'entrée E porte le `useState` et le test de `flag`; S est le `then` (`setN(1)`, puis saut); X est l'`else` vide (rien à `lowerer`, saut immédiat); R est le bloc de jonction, qui porte le `return` et n'est atteint que depuis S et X (jamais directement depuis E). $\text{Dom}(E) = \{E\}$, $\text{Dom}(S) = \{E, S\}$, $\text{Dom}(X) = \{E, X\}$, $\text{Dom}(R) = \{R\} \cup (\{E, S\} \cap \{E, X\}) = \{E, R\}$: le bloc vide X ne change rien au résultat, puisqu'il hérite tel quel du dominateur de E . S ne domine pas la seule sortie R : `certify` rend `None` (pas de `Certified`), `may_be_skipped` rend `vrai`. Sans preuve, la règle `setter-in-render` s'arrête au `WARNING` observé.

Exercice 3.6: Forte ou faible ?

Un objet est alloué par un littéral `const o = { v: 0 }` dans le rendu, puis un handler fait `o.v = 1`. La variable `o` ne désigne qu'un site d'allocation. Pourquoi l'écriture sur le tas doit-elle néanmoins rester faible ?

Solution

Le site d'allocation résume *tous* les objets créés à cet endroit, un par rendu : un seul objet concret reçoit l'écriture, les autres gardent 0. Et le handler peut ne jamais s'exécuter, ou s'exécuter sur un seul chemin. Remplacer 0 par 1 affirmerait que tous ces objets valent 1 : une sous-approximation, donc un FN possible.

Chapitre 4

Tour d'architecture : lire la codebase

Objectifs du chapitre.

- Savoir situer chacun des 142 fichiers de `src/` (et le crate `reactant-wasm`) : son module, son rôle en une phrase, le chapitre qui le traite.
- Suivre une invocation `reactant check counter.tsx` de `main` jusqu'au code de sortie, en nommant à chaque étape la fonction appelée et le type qui passe à l'étape suivante.
- Connaître les cinq couches du projet et les frontières qui les séparent : plus d'AST après le lowering, des règles qui ne parcourent pas l'IR, un driver partagé par tous les hôtes.
- Reconnaître les conventions transversales qui reviennent partout : `fail-closed`, `must` et `may` comme types, identités internées, positions `SourceRange`.
- Lire les trois principes de `CLAUDE.md` dans le code, et savoir où chercher (`tests`, `fixtures`, `-trace`, `-verbose`, `explain`) quand on veut comprendre un comportement.

Prérequis : chapitre 1 (ce que fait l'outil, sa sortie) ; chapitre 2 (`rendu`, `commit`, effets) pour les exemples ; chapitre 3 pour le vocabulaire `must / may` et la `soundness` (définitions 3.7 et 3.8). Aucun chapitre technique n'est requis : celui-ci est la carte qui les précède tous.

Un lecteur qui ouvre le dépôt pour la première fois trouve environ soixante mille lignes de Rust réparties en 142 fichiers. Aucune n'est inutile, mais toutes ne se lisent pas au même moment. Ce chapitre ne détaille aucun algorithme : il donne la carte. Il répond à trois questions dans l'ordre où un nouveau contributeur se les pose. Où est quoi ? Par où passe une exécution ? Quelles règles du jeu tiennent l'ensemble ?

Le fil conducteur est le plus petit programme qui fasse réagir l'analyseur : un compteur dont l'effet incrémente l'état qu'il observe.

Extrait 4.1 – Le programme témoin du chapitre (`/tmp/tour/counter.tsx`)

```
1 import { useState, useEffect } from "react";
2
3 export function Counter() {
4   const [count, setCount] = useState(0);
5   useEffect(() => {
6     setCount(count + 1);
7   }, [count]);
8   return <p>{count}</p>;
9 }
```

L'analyseur, lancé sur ce fichier, répond (sortie observée, `-no-color`) :

```
Counter (2 hooks) counter.tsx
  warn  infinite-loop [hook:0] (line 5:2) this effect keeps pushing state `count` (its deps do not provably gate it
    ↳ , so the effect can re-run every render) to new values on every run. Potential infinite render loop
    (2 trace step(s), rerun with --trace)
```

```
| ! 1 warning(s) across 1 file(s).
```

et sort avec le code 1. Entre la ligne de commande et cette ligne `warn`, le fichier a été découvert, parsé, abaissé en IR, interprété jusqu'à un point fixe ; des relations ont été dérivées ; dix-neuf règles ont été consultées ; le résultat a été trié, compté et rendu. La section 4.2 suit ce trajet pas à pas. Avant cela, il faut savoir où se trouvent les pièces.

4.1 La carte : crates, modules et fichiers

4.1.1 Un workspace, deux crates, un paquet npm

Le dépôt est un workspace Cargo à deux membres. Le manifeste racine déclare aussi deux *features* et sépare explicitement la bibliothèque du binaire :

Extrait 4.2 – Cargo.toml, l. 13–33 — workspace, features, binaire et bibliothèque

```
13 [workspace]
14 members = [".", "crates/reactant-wasm"]
15
16 [features]
17 default = ["cli", "schema-gen"]
18 # The CLI (clap and friends) is binary-only; the wasm build uses
19 # `default-features = false` to keep the dependency tree to oxc + serde.
20 cli = ["dep:clap"]
21 # JSON Schema generation (ADR-022 §6): only the `reactant schemas` emitter
22 # needs schemars – validation is serde + hand checks, so the wasm build
23 # drops it.
24 schema-gen = ["dep:schemars"]
25
26 [[bin]]
27 name = "reactant"
28 path = "src/main.rs"
29 required-features = ["cli"]
30
31 [lib]
32 name = "reactant"
33 path = "src/lib.rs"
```

Trois choses sont à retenir.

- **La bibliothèque reactant** (`src/lib.rs`) contient tout l'analyseur. Le binaire `reactant` (`src/main.rs`) n'existe qu'avec la feature `cli`, qui tire `clap`.
- **Le crate reactant-wasm** (`crates/reactant-wasm/`) dépend de la bibliothèque avec `default-features = false` : ni `clap` ni `schemars`. Il expose, via `wasm-bindgen`, cinq fonctions (`hostConstants`, `helpPage`, `packSpecs`, `validatePack`, `run`) ; `run` appelle le même `driver::run_check` que la CLI native (`crates/reactant-wasm/src/lib.rs`, l. 249).
- **Le paquet npm reactant-analyzer** (`npm/`) emballe ce module WASM : un exécutable `npm/bin/reactant.js`, une API JavaScript (`npm/lib/`), des tests de fumée (`npm/test/`). Il est traité au chapitre 24.

Les dépendances de la bibliothèque sont volontairement minces : les quatre crates `oxc_parser`, `oxc_ast`, `oxc_span`, `oxc_allocator` en version 0.138.0 (Cargo.toml, l. 36–39), `serde`, `serde_json`, `serde_path_to_error`, plus `clap` et `schemars` optionnels. Le commentaire des lignes 18–19 en donne la raison : le build WASM doit se contenter d'« `oxc + serde` ».

4.1.2 La racine : `lib.rs` et `main.rs`

Le binaire tient en cinq lignes :

Extrait 4.3 – `src/main.rs` — le binaire entier

```
1 mod cli;
2
3 fn main() {
4     std::process::exit(cli::run());
5 }
```

et la bibliothèque en treize :

Extrait 4.4 – `src/lib.rs` — la surface de la bibliothèque

```
1 pub mod config;
2 pub mod domains;
3 pub mod driver;
4 pub mod engine;
5 pub mod ir;
6 pub mod lowering;
7 pub mod project;
8 pub mod registry;
9 pub mod resolver;
10 pub mod rules;
11
12 #[cfg(test)]
13 mod test_support;
```

Le module `cli` n'est *pas* dans la bibliothèque : il est déclaré par `main.rs`, côté binaire, et c'est lui qui dépend de `clap` (ADR-16). Tout le reste — y compris le `driver`, qui compose une exécution complète — est dans la bibliothèque, donc réutilisable par le crate `WASM` et par les tests. `test_support.rs` n'est compilé qu'en test.

4.1.3 Les dix modules de la bibliothèque

Les dix modules publics se rangent en cinq couches (section 4.3). En une phrase chacun :

- `src/lowering/`** le front-end : lit l'AST d'oxc, détecte composants, hooks custom et utilitaires, et produit l'IR. C'est, avec `src/resolver/`, le seul module qui importe oxc.
- `src/ir/`** la représentation intermédiaire : CFG, instructions, expressions, table des hooks, identités (`FileId`, `ComponentId`), et la primitive de splice.
- `src/domains/`** les domaines abstraits (treillis, produit `StateValue`), l'interpréteur des expressions et des instructions, les stores.
- `src/engine/`** le moteur : worklist sur un CFG, point fixe d'un composant, analyse de programme, relations dérivées à la convergence.
- `src/registry/`** le registre générique indexé par (fichier, nom) et les résumés de hooks de bibliothèques (`SummaryRegistry`).
- `src/rules/`** la couche règles : API typée, dix-neuf règles natives, registre, documentation, packs déclaratifs.
- `src/resolver/`** découverte des fichiers, résolution des imports, parse et lowering de tous les fichiers, point d'entrée de l'analyse.
- `src/project/`** conventions de projet (`Vite`, `Next.js`, `tsconfig`) qui paramètrent la découverte et la résolution.

src/driver/ la composition complète de check et les rendus humain et JSON, partagés par tous les hôtes.

src/config.rs le fichier `reactant.config.json` : types, lecture, précedence avec les options de ligne de commande.

4.1.4 Fichier par fichier

La tableau 4.1 donne, pour chaque fichier, sa taille (lignes brutes, tests unitaires compris), son rôle en une phrase et le chapitre qui le traite. Elle tient lieu de carte de la codebase pour cette version du manuscrit ; les types et fonctions publics de chaque fichier sont détaillés dans le chapitre qui le traite. Deux conseils de lecture.

- Les tailles sont trompeuses : `fixpoint.rs` (2 815 lignes) contient environ mille lignes de tests, et `transfer/state_value.rs` (2 749 lignes) n'a que 1 041 lignes de production, le reste étant des tests.
- Beaucoup de fichiers commencent par un commentaire `//!` qui résume leur rôle et cite l'ADR fondateur : c'est la première chose à lire en ouvrant un fichier.

TABLE 4.1 – Les fichiers de `src/` (chemins relatifs à `src/`) et du crate WASM, au commit `e67b10a`. La dernière colonne renvoie au chapitre qui traite le fichier.

Fichier	Lignes	Rôle	Ch.
Racine			
<code>main.rs</code>	5	le binaire : <code>std::process::exit(cli::run())</code>	23
<code>lib.rs</code>	13	déclaration des dix modules publics	23
<code>config.rs</code>	455	<code>reactant.config.json</code> : types, JSONC, précedence, surcharges de sévérité	23
<code>test_support.rs</code>	142	constructeurs de fixtures partagés par les tests unitaires	25
lowering/ — le front-end			
<code>mod.rs</code>	565	façade : <code>Candidate</code> , <code>is_hook_name</code> , constantes de module, <code>lowerers</code> des composants et des hooks	5
<code>detector.rs</code>	169	marcheur commun des fonctions de niveau supérieur	5
<code>component_detector.rs</code>	352	classification « composant », props typées DOM	5
<code>hook_detector.rs</code>	162	classification « hook custom »	5
<code>utility_detector.rs</code>	114	classification « utilitaire »	5
<code>hook_call_detect.rs</code>	109	« ce corps appelle-t-il un hook ? » (#122)	5
<code>jsx_detect.rs</code>	63	« un chemin de retour produit-il du JSX ? »	5
<code>import_resolution.rs</code>	262	provenance des imports : <code>HookOrigin</code> , <code>ResolvedImport</code> , <code>JsxOrigins</code>	5
<code>module_facts.rs</code>	163	directives et arêtes d'import d'un fichier (<code>ModuleFacts</code>)	5
<code>utility_lowerer.rs</code>	62	lowering des utilitaires en <code>FunctionIR</code>	5
<code>cfg_builder.rs</code>	1 381	construction des blocs, lowering des instructions et des motifs de liaison	7
<code>expr_lower.rs</code>	1 667	lowering des expressions, fente des blocs sur <code>?:</code> , <code>&&</code> , <code> </code> , <code>??</code> , <code>await</code>	7
<code>hook_extractor.rs</code>	1 623	passé IR vers IR : appels de hooks, handlers, subscriptions deviennent des <code>HookEntry</code>	7
ir/ — la représentation intermédiaire			
<code>mod.rs</code>	34	ré-exports de la surface publique de l'IR	6
<code>types.rs</code>	25	alias de base (<code>Symbol</code> , <code>HookLabel</code> , <code>BlockId</code> , <code>Var</code> , <code>QualifiedSlot</code>) et <code>ExprId</code>	6
<code>source_range.rs</code>	102	<code>FileId</code> , <code>FileTable</code> , <code>SourceRange</code> , <code>SourceMap</code>	6
<code>cfg.rs</code>	228	le CFG : <code>BasicBlock</code> , <code>Terminator</code> , <code>EdgeKind</code> , <code>Edge</code>	6

Fichier	Lignes	Rôle	Ch.
stmt.rs	54	les instructions Stmt	6
expr.rs	548	les expressions Expr, opérateurs, valeurs marquées des hooks	6
hooks.rs	358	la table des hooks : HookEntry, deps, provenance	6
component.rs	83	ComponentIR, ModuleConstInit, ContextId	6
hook_ir.rs	24	HookIR, l'IR d'un hook custom	6
function_ir.rs	22	FunctionIR, l'IR d'un utilitaire	6
component_id.rs	274	identité internée ComponentId, ComponentTable (ADR-40)	6
module.rs	222	ModuleFacts, ModuleTable	6
bindings.rs	212	certificats de liaison syntaxique (« ce nom est-il lié à coup sûr ? »)	6
free_vars.rs	630	variables libres, chemins d'accès AccessPath, couverture par les deps	6
remap.rs	451	décalage des labels de hooks et des sites d'allocation	8
splice.rs	1 156	la primitive de greffe d'un CFG dans un autre, α -renommage	8
domains/ — valeurs abstraites, interpréteur, stores			
mod.rs	162	les traits AbstractDomain et Transfer	9
context.rs	171	contextes de transfert : AnalysisCtx, InterCtx, AnalyzeChildFn	10
impls/mod.rs	65	macro flat_lattice!, ré-exports des domaines	9
impls/bool_val.rs	13	treillis plat des booléens	9
impls/setter_val.rs	72	treillis plat des setters, avec identité	9
impls/str_const.rs	153	ensembles bornés de constantes de chaînes	9
impls/interval.rs	614	intervalles de flottants avec bit d'intégralité	9
impls/stability.rs	367	stabilité référentielle versionnée (ADR-17)	10
impls/state_value.rs	1 262	le produit StateValue des sortes de valeurs JS (ADR-15)	10
transfer/mod.rs	3	ré-export de StateValueTransfer	11
transfer/state_value.rs	2 749	sémantique des expressions, mémos, inlining d'un enfant (eval_comp_app)	11
interp/mod.rs	9	ré-exports de l'interpréteur	11
interp/interpreter.rs	583	sémantique des instructions : liaisons, appels de setter, callbacks	11
interp/callbacks.rs	131	classification d'un callee (TriggerClass, ADR-9)	11
interp/cfg.rs	26	ordre topologique d'un corps de fonction	11
stores/mod.rs	35	déclarations, ordre ponctuel partagé par les stores	11
stores/abstract_env.rs	366	l'environnement local AbstractEnv	11
stores/state_store.rs	173	les valeurs écrites des slots useState	11
stores/memo_store.rs	70	les valeurs des useMemo et useCallback	11
stores/shared_state_store.rs	204	les écritures inter-composants	11
stores/heap.rs	165	le tas abstrait, indexé par site d'allocation	11
engine/ — le moteur			
mod.rs	49	déclaration des sous-modules, surface publique	13
cfg_analyzer.rs	1 019	worklist sur un CFG, widening sur arcs arrière, raffinement sur les gardes	12
dominance.rs	304	dominateurs, « sur tous les chemins »	12
eval.rs	105	évaluer une expression contre les stores convergés	12
fixpoint.rs	2 815	point fixe d'un composant, analyse de programme, expansion des hooks et utilitaires	13
hook_registry.rs	101	registre (fichier, nom) des HookIR	8
function_registry.rs	125	registre des FunctionIR, résolution fail-closed	8
triggers.rs	110	relation effect_triggers : quel slot fait bouger quelle dep	14
setters.rs	2 490	marche des setters, relations slot_writers, slot_reads, body_calls	14

Fichier	Lignes	Rôle	Ch.
written.rs	415	ce qu'une écriture stocke (fraîcheur, valeur)	14
guards.rs	1 058	preuve « une écriture éteint ses propres gardes »	14
seeds.rs	552	relation slot_seeds (useState amorcé par une prop)	14
registrations.rs	683	relation registrations (callbacks confiés à un registrar)	14
program_relations.rs	42	ProgramRelations : relations de programme, paresseuses	14
churn.rs	927	le graphe de churn du programme et ses cycles	15
render_deps.rs	1 276	dépendance de rendu (ADR-41)	16
symbol_graph.rs	416	graphe de symboles et ordre topologique (lu par -verbose seulement)	16
root_detector.rs	329	choix des racines (RootStrategy)	16
component_cache.rs	327	cache des analyses d'enfants, indexé par props abstraites	16
component_registry.rs	256	registre des composants, résolution d'un callee JSX	16
analysis_result.rs	289	AnalysisResult, le résultat par composant	16
program_result.rs	225	ProgramAnalysisResult, le résultat par programme	16
registry/			
mod.rs	5	ré-exports	23
keyed.rs	113	registre générique indexé par (fichier, nom)	23
summary.rs	573	résumés abstraits des hooks de bibliothèques (sans leur source)	8
rules/ — la couche règles			
mod.rs	131	trait Rule, SafeCheck, all_rules	17
registry.rs	808	registre dynamique (natives puis packs), passe check_component	17
docs.rs	426	documentation statique de chaque nom de diagnostic	17
api/mod.rs	13	déclaration de la surface typée	17
api/query.rs	1 259	requêtes typées : Certified, MustResult, May, RuleCtx	17
api/witness.rs	560	vocabulaire fermé des témoins (Step, Note)	17
api/diagnostic.rs	218	Diagnostic scellé, Severity	17
api/cache.rs	77	ProgramCache : structures de programme paresseuses	17
helpers/mod.rs	267	nommage des valeurs et des slots, scans, porte de stabilité des deps	17
helpers/jsx.rs	365	relations d'éléments JSX, verdict d'identité d'un site	17
helpers/context_flow.rs	234	relation context_consumers (ADR-32)	17
helpers/providers.rs	107	providers de contexte prouvés	17
helpers/purity.rs	155	qu'est-ce qu'une mutation en place	17
helpers/mount.rs	566	durée de montage : où un composant est rendu	17
helpers/cycles.rs	248	nommage et projection du graphe de churn	15
helpers/render_tree.rs	777	vue de programme de la dépendance de rendu (RenderIndex)	21
impls/mod.rs	44	un module par règle, rien d'autre	17
impls/lazy_init.rs	644	lazy-init : initialiseur recalculé à chaque rendu	18
impls/unstable_context_value.rs	72	unstable-context-value : valeur de provider fraîche	18
impls/redundant_set_state.rs	629	redundant-set-state : écrire la valeur déjà présente	18
impls/setter_in_render.rs	586	setter-in-render : setter appelé pendant le rendu	18
impls/derived_state.rs	169	derived-state : effet qui recopie un autre état	18
impls/state_mutation.rs	521	state-mutation : objet d'état ou de props muté en place	18
impls/frozen_initial_state.rs	336	frozen-initial-state : état figé sur la première prop	18
impls/infinite_loop.rs	1 667	infinite-loop : boucle de rendu infinie	19
impls/state_lifted_too_high.rs	146	state-lifted-too-high : état placé trop haut	19
impls/widening_info.rs	41	widening-info : slot élargi pour converger (Info)	20
impls/analysis_limit_info.rs	158	analysis-limit : analyse tronquée (Info)	20
impls/conditional_hook.rs	581	conditional-hook : hook appelé sous condition	20
impls/always_unstable_deps.rs	489	always-unstable-deps : dep toujours fraîche	20
impls/missing_deps.rs	701	missing-deps : variable capturée absente des deps	20

Fichier	Lignes	Rôle	Ch.
impls/unnecessary_rerender.rs	446	unnecessary-rerender : effet de montage qui écrase l'état initial	20
impls/missing_cleanup.rs	113	missing-cleanup : effet sans teardown	20
impls/stale_closure.rs	469	stale-closure : callback durable figé sur un état	20
impls/wasted_subtree_render.rs	333	wasted-subtree-render : sous-arbres re-rendus pour rien	21
impls/server_component_hook.rs	308	server-component-hook : hook dans un Server Component	21
declarative/mod.rs	87	load_pack, seule surface publique des packs	22
declarative/schema.rs	958	modèle serde de pack.json	22
declarative/validate.rs	2 544	validation sémantique d'un pack (typage des entités)	22
declarative/entity.rs	1 261	adaptateur entre entités du schéma et relations du moteur	22
declarative/exec.rs	918	l'exécuteur TierARule	22
resolver/, project/ — fichiers et projets			
resolver/mod.rs	1 164	découverte, résolution d'imports, parse et lowering de chaque fichier, analyze_lowered	23
resolver/filesystem.rs	166	la couture FileSystem (OsFileSystem, MemFileSystem)	23
resolver/gitignore.rs	383	.gitignore, pour la seule question « répertoire généré ? »	23
resolver/closure.rs	217	fermeture transitive des imports (-follow-imports)	23
project/mod.rs	464	détection du type de projet, contexte d'analyse	23
project/tsconfig.rs	563	lecture JSONC des tsconfig, paths et baseUrl	23
project/paths_resolver.rs	254	résolveur d'imports fondé sur les alias tsconfig	23
project/nextjs.rs	151	conventions de l'App Router : graphe serveur	21
driver/, cli/ — la coquille			
driver/mod.rs	704	run_check et les sous-commandes d'aide, rules, explain	23
driver/report.rs	68	CheckReport, ce que lisent les deux rendus	23
driver/human.rs	362	rendu humain	23
driver/json.rs	307	rendu JSON (schéma version 2)	23
driver/locations.rs	103	groupage des findings par position source (#129)	23
driver/blind_spots.rs	73	ce que l'exécution sait ne pas avoir lu	23
driver/palette.rs	49	séquences de couleur ANSI	23
cli/mod.rs	127	parsing clap, dispatch des sous-commandes	23
cli/check.rs	202	check : fusion options et config, appel du driver	23
cli/config_load.rs	128	chargement de la config et du registre (natives, puis packs)	23
cli/rules_cmd.rs	19	sous-commande rules	23
cli/explain.rs	17	sous-commande explain	23
cli/schemas_cmd.rs	63	sous-commande schemas (schémas JSON)	23
cli/color.rs	14	activation des couleurs (option, NO_COLOR, tty)	23
Crate reactant-wasm			
crates/reactant-wasm/src/lib.rs	323	point d'entrée WASM : cinq fonctions wasm-bindgen, dont run	24

4.2 Le flot d'une invocation, de bout en bout

Reprenons `reactant check counter.tsx` et suivons l'exécution. La figure 4.1 en donne la vue d'ensemble : les modules traversés et, sur les flèches, le type qui passe d'une étape à la suivante. Les sous-sections qui suivent la parcourent dans l'ordre.

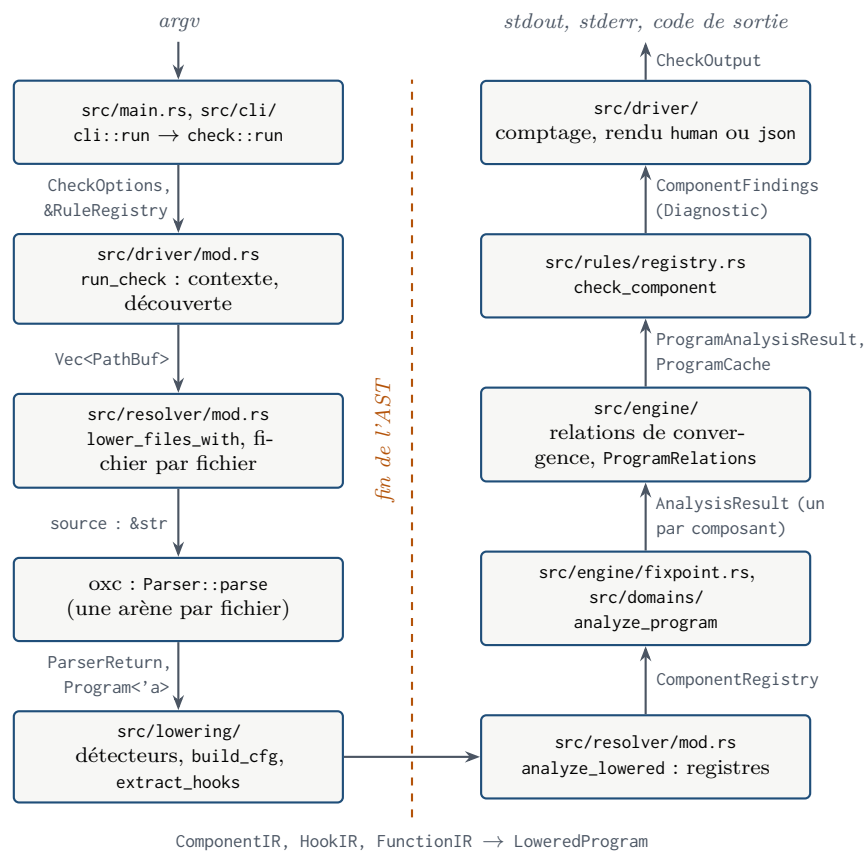


FIGURE 4.1 – Le pipeline de reactant check, module par module. À gauche, de la ligne de commande à l'IR ; à droite, de l'IR à la sortie. Sur chaque flèche, le type principal transmis. L'AST d'oxc n'existe qu'entre le parse et le lowering, et seulement le temps d'un fichier : aucune flèche ne franchit la ligne pointillée avec un AST.

4.2.1 De main au driver

`main` appelle `cli::run` (`src/cli/mod.rs`, l. 111), qui parse les arguments avec `clap` et dispatche. `reactant counter.tsx` sans sous-commande est un `check`, comme `reactant check counter.tsx`. `check::run` (`src/cli/check.rs`, l. 99) charge la configuration et le registre des règles (`load_config_and_registry`, `src/cli/config_load.rs`, l. 16), fusionne options et configuration (les options gagnent, ADR-22), puis passe la main :

Extrait 4.5 – `src/cli/check.rs`, l. 192–202 — le binaire appelle le driver, puis écrit les flux

```
192 let out = driver::run_check(
193     Arc::new(OsFileSystem),
194     &args.paths,
195     &registry,
196     &opts,
197     &|p| super::display_relative(p),
198 );
199 eprint!("{}", out.stderr);
200 print!("{}", out.stdout);
201 out.exit_code
202 }
```

Le driver ne touche jamais les flux du processus. Il reçoit un système de fichiers derrière une couture (`Arc<dyn FileSystem>`), les chemins positionnels, le registre prêt à l'emploi, les options résolues et une fonction d'affichage des chemins ; il rend trois valeurs tamponnées :

Extrait 4.6 – `src/driver/mod.rs`, l. 85–90 — la sortie tamponnée du driver

```
85 /// Buffered run result: the host writes the streams and exits.
86 pub struct CheckOutput {
87     pub stdout: String,
88     pub stderr: String,
89     pub exit_code: i32,
90 }
```

Extrait 4.7 – `src/driver/mod.rs`, l. 102–112 — la signature de `run_check`

```
102 /// The full check pipeline. `registry` arrives ready (natives + packs +
103 /// overrides installed); `paths` are the positional arguments, replayed
104 /// identically by every host; `display` renders paths for output (the CLI
105 /// passes a cwd-relativizer, WASM the identity).
106 pub fn run_check(
107     fs: Arc<dyn FileSystem>,
108     paths: &[String],
109     registry: &RuleRegistry,
110     opts: &CheckOptions,
111     display: &dyn Fn(&Path) -> String,
112 ) -> CheckOutput {
```

C'est cette signature qui permet au crate WASM d'appeler la même fonction avec un `MemFileSystem` et une fonction d'affichage identité. Les deux hôtes ne diffèrent que par ce qu'ils passent et par ce qu'ils font des trois champs rendus.

4.2.2 Contexte, découverte, lowering

`run_check` commence par des vérifications d'usage (premier argument qui ressemble à une commande mal tapée), construit le contexte de projet (Vite, Next.js ou simple parcours, alias `tsconfig`), puis découvre les fichiers : un répertoire est parcouru récursivement à la recherche de

.ts, .tsx, .js, .jsx; un fichier nommé est pris tel quel. Pour counter.tsx, la liste contient un seul chemin. Le chapitre 23 détaille ces étapes; ici, seule compte la suivante :

Extrait 4.8 – src/driver/mod.rs, l. 236–237 — un seul appel pour tout le front-end

```
236 // -- Lower -----
237 let mut lowered = lower_files_with(fs.as_ref(), &files, ctx.resolver.as_ref());
```

lower_files_with (src/resolver/mod.rs, l. 247–349) boucle sur les fichiers. Pour chacun, elle lit la source à travers la couture, crée une arène, parse, puis appelle trois lowerers et deux collecteurs de faits de module. Le cœur du parse est court :

Extrait 4.9 – src/resolver/mod.rs, l. 289–312 — parse d'un fichier et décision de le garder

```
289 let alloc = Allocator::default();
290 let ret = OxcParser::new(&alloc, &source, source_type_for(path))
291     .with_options(ParseOptions::default())
292     .parse();
293 // `panicked` is oxc's only "this AST is unusable" signal (the program
294 // is empty). The parser recovers from every other syntax error and
295 // still returns a lowerable program, so skipping on a non-empty
296 // diagnostic list would drop whole files from the analysis – a
297 // forbidden false negative, and a growing one: oxc keeps moving TS
298 // semantic checks into the parser.
299 if ret.panicked || !ret.diagnostics.is_empty() {
300     lowered.parse_errors.push(ParseError {
301         file: path.clone(),
302         message: ret
303             .diagnostics
304             .first()
305             .map(|d| d.message.to_string())
306             .unwrap_or_else(|| "the parser produced no usable program".to_string()),
307         analyzed: !ret.panicked,
308     });
309 }
310 if ret.panicked {
311     continue;
312 }
```

Premier type : ParserReturn. Le parseur d'oxc [The26] rend un ParserReturn dont le projet lit trois champs : program (l'AST, alloué dans l'arène alloc), diagnostics (les erreurs de syntaxe) et panicked (vrai quand l'AST est inutilisable). La décision codée ici est une décision de soundness : un fichier dont le parseur a *recupéré* une erreur est quand même abaissé, et seul un panicked fait sauter le fichier — lequel est alors signalé sur stderr ([skipped]) et interdit le « tout va bien » final. Sur un fichier contenant `const x = ;` passé à côté de counter.tsx, on observe :

```
[skipped] broken.tsx: Unexpected token, the file was not analyzed
Counter (2 hooks) counter.tsx
  warn  infinite-loop [hook:0] (line 5:2) this effect keeps pushing state `count` (its deps do not provably gate it
  ↳ , so the effect can re-run every render) to new values on every run. Potential infinite render loop
  (2 trace step(s), rerun with --trace)

! 1 warning(s) across 1 file(s).
not analyzed:
  • 1 file(s) could not be parsed and were dropped, so nothing in them was analysed
```

Deuxième type : Candidate. Chaque lowerer commence par demander aux détecteurs quelles fonctions de niveau supérieur l'intéressent. La réponse est une liste de Candidate, une vue

empruntée dans l'AST :

Extrait 4.10 – `src/lowering/mod.rs`, l. 128–144 — le candidat produit par les détecteurs

```

128 /// A top-level function picked out by one of the detectors, ready for lowering:
129 /// its binding name, parameter list, and body. Shared by the component, hook,
130 /// and utility detectors – they differ in how they *classify* a function, not
131 /// in this output shape (all three feed `Candidate::build_cfg`).
132 #[derive(Debug)]
133 pub struct Candidate<'a> {
134     pub name: String,
135     pub params: &'a FormalParameters<'a>,
136     pub body: &'a FunctionBody<'a>,
137     /// `true` for a concise arrow (`x => expr`), whose body oxc stores as a
138     /// single `ExpressionStatement` indistinguishable from `x => { expr; }`.
139     /// The difference is the whole return value, so it has to travel with the
140     /// body: lowered as a statement, `const useThing = () => useState(0)`
141     /// returns unit, and every consumer of `useThing` is left with an
142     /// unresolvable hook (#5).
143     pub expression: bool,
144 }
```

Pour `counter.tsx`, le détecteur de composants rend un candidat `Counter` : un nom en majuscule, un corps qui retourne du JSX (chapitre 5). Le champ `expression` distingue `x => expr` de `x => { expr; }`, qu'oxc représente de la même façon ; l'oublier a coûté un faux négatif (#5), d'où le commentaire.

Troisième type : `ComponentIR`. Le candidat est ensuite abaissé. La chaîne, pour un composant, tient en quatre appels :

Extrait 4.11 – `src/lowering/mod.rs`, l. 463–470 — du candidat au CFG et à la table des hooks

```

463 detect_components(program)
464     .into_iter()
465     .map(|candidate| {
466         let (param_names, mut render_cfg) = candidate.build_cfg(&ctx);
467         let (mut hooks, hook_provenance, mut next_label) =
468             extract_hooks(&mut render_cfg, &imports);
469         extract_handlers(&render_cfg, &mut hooks, &mut next_label);
470         extract_subscriptions(&mut hooks, &mut next_label);
471     })
```

`build_cfg` produit le CFG du corps de rendu (chapitre 7) ; `extract_hooks` y reconnaît les appels de hooks, les remplace par des valeurs marquées et fabrique la table des `HookEntry` ; `extract_handlers` et `extract_subscriptions` y ajoutent les handlers JSX et les `addEventListener` des effets. Les labels sont attribués dans cet ordre : pour `Counter`, le `useState` reçoit le label 0 et le `useEffect` le label 1 — ce sont les `[hook:0]` et `[hook:1]` de la sortie. Le résultat est un `ComponentIR` :

Extrait 4.12 – `src/ir/component.rs`, l. 59–83 — l'IR d'un composant

```

59 #[derive(Debug, Clone)]
60 pub struct ComponentIR {
61     /// Source file this component was lowered from. Used as part of the
62     /// `(file, name)` registry key so two components named `Page` in
63     /// different files don't collide.
64     pub file: PathBuf,
65     pub name: Symbol,
66     pub param: Var,
67     /// Names of props whose TypeScript type is a DOM interface
68     /// (`HTMLCanvasElement`, `SVGElement`, `Node`...). Mutating these is
```

```

69  /// imperative DOM manipulation, not a write into React-owned data -
70  /// the state-mutation rule exempts them.
71  pub dom_props: Arc<HashSet<Var>>,
72  pub render_cfg: CFG,
73  pub hooks: Vec<HookEntry>,
74  /// Provenance row per hook call in `hooks` (ADR-023 step 1):
75  /// `label → (origin hook, source, direct|inlined)`. Grows during
76  /// analysis as custom hooks are expanded.
77  pub hook_provenance: Vec<crate::ir::hooks::HookProvenance>,
78  /// Module-level `const` bindings of the source file with syntactically
79  /// known kinds, keyed by name. Function-valued consts (arrow/function
80  /// expressions) are excluded: those are components/utilities/handlers
81  /// with their own machinery.
82  pub module_consts: Arc<HashMap<Var, ModuleConstInit>>,
83  }

```

Ce type ne contient plus aucune référence vers l'AST : PathBuf, String, Arc, un CFG possédé. Dès la fin de l'itération, l'arène du fichier est libérée. Les hooks custom deviennent de la même façon des HookIR, les utilitaires des FunctionIR. Le tout est rangé dans un LoweredProgram (src/resolver/mod.rs, l. 208–233) qui porte aussi la table des fichiers (FileTable), la table des faits de module (ModuleTable), les erreurs de parse et les arêtes d'import des utilitaires.

4.2.3 L'analyse : registres, point fixe, relations

Revenu dans le driver, après quelques contrôles (imports non lus, stratégie de racines, validité de -entry), l'analyse proprement dite est lancée :

Extrait 4.13 – src/driver/mod.rs, l. 370–374 — la configuration réelle du moteur et l'appel de l'analyse

```

370  let config = Config {
371      summary_registry: crate::registry::SummaryRegistry::new_with_common(),
372      ..Config::default()
373  };
374  let program_result = analyze_lowered(lowered, strategy, config);

```

analyze_lowered range les trois sortes d'IR dans leurs registres, puis délègue au moteur :

Extrait 4.14 – src/resolver/mod.rs, l. 439–452 — des IR aux registres, puis au moteur

```

439  pub fn analyze_lowered(
440      lowered: LoweredProgram,
441      strategy: RootStrategy,
442      mut config: Config,
443  ) -> ProgramAnalysisResult {
444      config.function_registry =
445          FunctionRegistry::from_functions_and_imports(lowered.utilities, lowered.utility_imports);
446      let registry = ComponentRegistry::from_components(lowered.components);
447      let hook_registry = HookRegistry::from_hooks(lowered.hooks);
448      let mut result = analyze_program(registry, hook_registry, strategy, &config);
449      result.file_table = lowered.file_table;
450      result.module_table = lowered.module_table;
451      result
452  }

```

Le point fixe. analyze_program (src/engine/fixpoint.rs, l. 704) choisit les racines, puis analyse chacune par analyze_component_impl (src/engine/fixpoint.rs, l. 130). Un enfant rencontré

dans le JSX d'un parent est analysé à la demande, avec les props abstraites du parent ; les composants qu'aucune racine n'atteint sont analysés ensuite, isolément (chapitre 16). Pour un composant, `analyze_component_impl` greffe d'abord les utilitaires puis les hooks custom dans le CFG (chapitre 8), puis itère : passe de rendu, recalcul des mémos, passes des effets et des handlers, join des états écrits, test de convergence. Le test est lisible tel quel :

Extrait 4.15 – `src/engine/fixpoint.rs`, l. 484–497 — le test de convergence de la boucle externe

```

484
485     // -- Convergence check -----
486     let new_state_incycle = state_from_render.join(&state_from_effects);
487     // Include cross-component state updates made by child effects/callbacks.
488     let external_updates = inter
489         .map(|i| i.shared_state.borrow().slice(comp_id))
490         .unwrap_or_else(StateStore::bottom);
491     let new_state = new_state_incycle
492         .join(&state_from_handlers)
493         .join(&external_updates);
494
495     if new_state.leq(&state) {
496         break;
497     }

```

Pour Counter, chaque tour écrit `count + 1` dans le slot 0 ; l'intervalle du slot grandit, le moteur applique un widening, et la boucle converge au troisième tour. `-verbose` le montre :

```

[verbose] symbol graph: 1 nodes, topo order = [Counter@counter.tsx]
[verbose] 1 components analyzed
[verbose] cache hits: 0, misses: 0
[verbose] Counter: 3 iteration(s), widened: [0]

```

Quatrième type : `AnalysisResult`. À la convergence, `analyze_component_impl` calcule dans une « dernière tranche » les relations propres au composant (`src/engine/fixpoint.rs`, l. 596–667), puis range tout dans un `AnalysisResult`. Ses champs, élagués de leurs commentaires, se regroupent ainsi :

Extrait 4.16 – Les champs d'`AnalysisResult`, élagués et regroupés (`src/engine/analysis_result.rs`, l. 171–272)

```

1 pub struct AnalysisResult<D: AbstractDomain> {
2     // identité et contexte du composant
3     pub component: ComponentId, pub file: PathBuf, pub param: Var,
4     pub dom_props: Arc<HashSet<Var>>, pub module_consts: Arc<HashMap<Var, ModuleConstInit>>,
5     // stores convergés et environnements par bloc
6     pub state_store: StateStore<D>, pub memo_store: MemoStore<D>, pub heap: Heap,
7     pub block_states: HashMap<BlockId, AbstractEnv<D>>,
8     pub effect_block_states: ..., pub handler_block_states: ...,
9     // l'IR après expansion, et ce qu'on en a extrait
10    pub render_cfg: CFG, pub hooks: Vec<HookEntry>, pub hook_provenance: Vec<HookProvenance>,
11    pub hook_calls: Vec<HookCallInfo>, pub effect_info: ..., pub handler_info: ...,
12    pub inline_origins: Vec<InlineOrigin>, pub custom_arg_returns: ...,
13    // histoire du point fixe
14    pub widen_trace: HashMap<HookLabel, WidenEvent>, pub effect_setter_writes: StateStore<D>,
15    pub iterations: usize,
16    // relations calculées à la convergence
17    pub slot_writers: Vec<SlotWriter>, pub slot_seeds: Vec<SlotSeed>,
18    pub registrations: Vec<Registration>, pub effect_triggers: Vec<EffectTrigger>,
19 }

```

Tous les résultats sont réunis dans un `ProgramAnalysisResult` (`src/engine/program_result.rs`, l. 28–69) : la table `components` indexée par `ComponentId`, le store partagé inter-composants, le graphe d'appels, les statistiques, et les tables d'identité (`component_table`, `file_table`, `module_table`).

Cinquième type : `ProgramRelations`. Certaines relations ne sont pas celles d'un composant mais du programme : le graphe de churn relie les écritures d'un composant aux effets d'un autre. Elles vivent dans une structure paresseuse, construite une fois par programme :

Extrait 4.17 – `src/engine/program_relations.rs`, l. 19–42 — les relations de programme, construites à la première demande

```
19 /// Program-scoped, lazily-computed relations shared by every reader of one
20 /// [ `ProgramAnalysisResult` ].
21 pub struct ProgramRelations<'a> {
22     program: &'a ProgramAnalysisResult,
23     churn: OnceLock<ChurnGraph>,
24 }
25
26 impl<'a> ProgramRelations<'a> {
27     pub fn new(program: &'a ProgramAnalysisResult) -> Self {
28         ProgramRelations {
29             program,
30             churn: OnceLock::new(),
31         }
32     }
33
34     pub fn program(&self) -> &'a ProgramAnalysisResult {
35         self.program
36     }
37
38     /// The program's churn graph and its cycles, built on first request.
39     pub fn churn(&self) -> &ChurnGraph {
40         self.churn.get_or_init(|| ChurnGraph::build(self.program))
41     }
42 }
```

4.2.4 Les règles, le tri, la sortie

Le driver construit *un* cache pour toute l'exécution — un `ProgramCache`, qui contient un `ProgramRelations` et trois autres structures de programme encore construites côté règles (`src/rules/api/cache.rs`, l. 26–41) — puis, pour chaque composant dans l'ordre lexicographique des noms affichés, appelle le registre :

Extrait 4.18 – `src/driver/mod.rs`, l. 407–410 — un seul cache pour tout le programme

```
407 // One cache for the whole run: rules needing whole-program structure (the
408 // churn graph of `infinite-loop`) build it once here instead of once per
409 // component, which used to make the rules phase quadratic (issue #86).
410 let rule_cache = ProgramCache::new(&program_result);
```

Extrait 4.19 – `src/rules/registry.rs`, l. 254–274 — la passe des règles sur un composant (début)

```
254 pub fn check_component<'a>(
255     &self,
256     cache: &'a ProgramCache<'a>,
257     component: ComponentId,
258 ) -> ComponentFindings {
```



```

259     let program = cache.program();
260     let mut diags: Vec<Diagnostic> = Vec::new();
261     let mut safe_checks: Vec<SafeCheck> = Vec::new();
262     for r in &self.rules {
263         let ctx = RuleCtx::cached(cache, component, self.options_for(r.name()));
264         let produced = r.check(&ctx);
265         // `safe_check` is consulted on the *raw* output: a rule whose
266         // findings were all filtered away still ran and found something –
267         // it is not "verified safe".
268         if produced.is_empty()
269             && let Some(sc) = r.safe_check(&ctx)
270         {
271             safe_checks.push(sc);
272         }
273         diags.extend(produced.into_iter().map(located).map(|d| self.clamped(d)));
274     }

```

Chaque règle reçoit un `RuleCtx` (le cache, l'identifiant du composant, ses options), rend un `Vec<Diagnostic>` et, si elle n'a rien trouvé, peut publier une assurance positive (`SafeCheck`, affichée « verified » sous `-info`). La suite de `check_component` (`src/rules/registry.rs`, l. 276–341) retire les assurances si un `analysis-limit` a été émis, applique les filtres `off/-rule`, puis trie selon un ordre total pour que deux exécutions produisent des octets identiques.

Sixième type : `Diagnostic`. C'est l'unité de sortie de la couche règles :

Extrait 4.20 – `src/rules/api/diagnostic.rs`, l. 55–73 — le diagnostic et son champ de sévérité scellé

```

55 #[derive(Debug, Clone, PartialEq)]
56 pub struct Diagnostic {
57     /// Private (ADR-021): the seal. Set only by `error`/`warn`/`info`; read
58     /// through [ `Diagnostic::severity` ]. A `pub` field here would let any code
59     /// forge an Error by mutation or struct literal.
60     severity: Severity,
61     /// Diagnostic name. `Cow`: native rules pass string literals (borrowed,
62     /// zero-cost), dynamically loaded rules own their `pack/rule` id.
63     pub rule: Cow<'static, str>,
64     pub message: String,
65     /// Hook label most directly involved, if any.
66     pub hook_label: Option<HookLabel>,
67     /// Variable name most directly involved, if any.
68     pub var: Option<Var>,
69     /// Source location of the primary finding, if available.
70     pub range: Option<SourceRange>,
71     /// Secondary evidence items explaining the causal chain.
72     pub notes: Vec<Note>,
73 }

```

Le champ `severity` est privé : on ne le fixe qu'à la construction, par trois constructeurs, dont un seul produit un `ERROR` (section 4.4). Pour `Counter`, `infinite-loop` rend un diagnostic `WARNING` ancré au `useEffect` (ligne 5, colonne 2), dont `hook_label` vaut 0 (le slot count) et dont les deux notes forment la chaîne de témoins que `-trace` déroule :

```

Counter (2 hooks) counter.tsx
warn  infinite-loop [hook:0] (line 5:2) this effect keeps pushing state `count` (its deps do not provably gate it
↳ , so the effect can re-run every render) to new values on every run. Potential infinite render loop
    → state `count` is written here [hook:1] (line 5:2)
    → the abstract value of state `count` kept growing and was widened at iteration 3

```

Comptage et code de sortie. Le driver filtre les INFO (sauf `-info`), compte ERROR et WARNING, et fixe le code de sortie :

Extrait 4.21 – `src/driver/mod.rs`, l. 480–484 — la politique de code de sortie

```
480 let exit_code = match opts.fail_on {
481     FailOn::Error if errors > 0 => EXIT_FINDINGS,
482     FailOn::Warning if errors + warnings > 0 => EXIT_FINDINGS,
483     _ => EXIT_OK,
484 };
```

Par défaut `-fail-on warning` : le WARNING de Counter donne le code 1 (EXIT_FINDINGS). Enfin un CheckReport est construit et rendu par `human::render` ou `json::render` (`src/driver/mod.rs`, l. 510–522). La même exécution avec `-format json` produit un document unique sur stdout ; en voici le début, observé :

```
{
  "version": 2,
  "files_analyzed": 1,
  "parse_errors": [],
  "diagnostics": [
    {
      "rule": "infinite-loop",
      "severity": "warning",
      "component": "Counter",
      "file": "counter.tsx",
      "component_file": "counter.tsx",
      "line": 5,
      "col": 2,
      "hook_label": 0,
      "var": null,
      "message": "this effect keeps pushing state `count` (its deps ...",
```

(message tronqué ici ; les notes suivent, puis `blind_spots` et `summary`).

Remarque

La chaîne complète des types rencontrés est donc : `ParserReturn` (oxc, emprunté dans l'arène) → `Candidate` (vue empruntée de l'AST) → `ComponentIR` (possédé, plus d'AST) → `LoweredProgram` → `AnalysisResult` par composant, réunis en `ProgramAnalysisResult` → `ProgramRelations` (dans `ProgramCache`) → `Diagnostics` (dans `ComponentFindings`) → `CheckReport` → `CheckOutput`. Retenir cette liste suffit pour savoir, devant n'importe quelle fonction du dépôt, à quel étage on se trouve.

4.3 Cinq couches et leurs frontières

Définition 4.1: Couche

Une *couche* de `reactant-analyzer` est un groupe de modules qui ne consomme de la couche précédente qu'un type de sortie fixé, et n'en connaît rien d'autre. Le projet en a cinq : le *front-end* (`src/lowering/`, avec le parse de `src/resolver/`), l'*IR* (`src/ir/`), le *moteur* (`src/engine/`, `src/domains/`, `src/registry/`), les *règles* (`src/rules/`) et la *coquille* (`src/driver/`, `src/cli/`, `src/project/`, découverte de `src/resolver/`, `src/config.rs`).

La figure 4.2 empile ces couches et nomme, entre chacune, la frontière et le mécanisme qui la tient. Chaque frontière est une décision documentée ; chacune a une raison de soundness autant que de propreté.

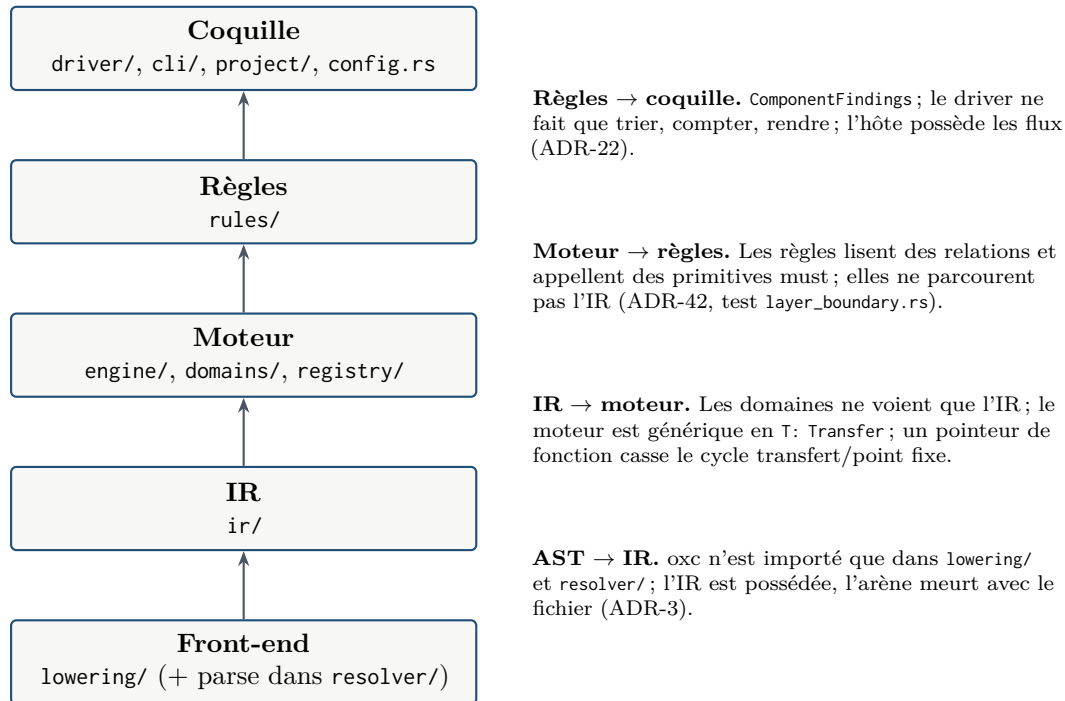


FIGURE 4.2 – Les cinq couches de `reactant-analyzer` (de bas en haut, dans l'ordre du flot) et, à droite, la frontière entre deux couches consécutives avec ce qui la fait respecter.

4.3.1 Front-end → IR : plus d'AST après le lowering

Invariant 4.1: Aucun domaine ne voit l'AST

Aucun type sorti du front-end ne porte de durée de vie liée à l'arène d'`oxc`. Hors de `src/lowering/` et de `src/resolver/`, aucun fichier n'importe un crate `oxc_*`.

La seconde moitié se vérifie mécaniquement : un `grep -r1 'oxc_' src` au commit `e67b10a` ne renvoie que les treize fichiers de `src/lowering/` et deux fichiers de `src/resolver/` (`mod.rs`, qui parse, et `closure.rs`, qui re-parse les fichiers suivis par `-follow-imports` pour n'en lire que les imports). La première moitié se lit sur les types : `Candidate<'a>` emprunte l'AST, `ComponentIR` (liste 4.12) ne l'emprunte plus.

Décision : ADR-3

ADR-3 choisit une IR dédiée, un CFG de blocs de base sur un langage d'expressions réduit, plutôt que d'interpréter l'AST d'`oxc`. Sa conséquence la plus citée est une phrase : « Abstract domains never see the Oxc AST, only the IR. » Alternative écartée : interpréter directement l'AST, ce qui obligerait chaque domaine à connaître toutes les formes de sucre syntaxique de JS et TS (déstructurations, court-circuits, `switch`, `try`, `await...`). Le lowering les normalise une fois ; le moteur n'en voit que quatre formes d'instructions. Effet secondaire utile : l'arène d'un fichier est libérée dès la fin de son lowering, et la mémoire de pointe côté AST est celle d'un seul fichier.

4.3.2 IR → moteur : un interpréteur générique, un cycle cassé

Le moteur n'est pas écrit pour un domaine particulier : `analyze_cfg` est générique en `T: Transfer` et `analyze_component_impl` en `T: Transfer<Domain = StateValue>`. Le seul `Transfer` concret du dépôt est `StateValueTransfer` (chapitre 11). Une difficulté de dépendance appa-

raît pourtant : quand l'évaluation d'une expression rencontre `<Child a={x} />`, le transfert (`src/domains/transfer/`) doit relancer l'analyse d'un composant, qui vit dans le moteur (`src/engine/fixpoint.rs`) — lequel dépend lui-même du transfert. Le cycle est cassé par un pointeur de fonction que le moteur fournit au transfert :

Extrait 4.22 – `src/domains/context.rs`, l. 17–25 — le pointeur de fonction qui casse le cycle transfert / point fixe

```
17 /// Function pointer for inlining a child component's analysis. Breaks the
18 /// circular dep between `domains::transfer` and `engine::fixpoint`.
19 pub type AnalyzeChildFn = fn(
20     &ComponentIR,
21     ComponentId,
22     AbstractEnv<StateValue>,
23     Heap,
24     &InterCtx<'_>,
25 ) -> AnalysisResult<StateValue>;
```

`analyze_program` y range `analyze_component_inter` (`src/engine/fixpoint.rs`, l. 729–741). `src/domains/` ne nomme `src/engine/` qu'à deux endroits : `context.rs` importe le type de ce contexte, et `transfer/state_value.rs` importe `ChildLookup` et `CallSite` pour résoudre l'identité d'un enfant JSX et enregistrer l'appel (chapitre 16) — toujours dans le sens IR/moteur vers transfert, jamais l'inverse.

4.3.3 Moteur → règles : les règles lisent, elles ne parcourent pas

Exemple 4.1: Un même fait, deux calculs

Considérons le fait « cet effet écrit le slot count ». Il faut suivre l'appel `setCount` à travers les alias (`const s = setCount`), les callbacks (`.then(() => setCount(. . .))`), les hooks custom inlinés. Si deux règles écrivent chacune leur propre parcours du CFG pour le retrouver, elles finissent par ne pas voir les mêmes appels ; l'une manque un `.then` que l'autre voit. C'est exactement ce qui s'est produit (#26).

Invariant 4.2: Frontière walk-free

Une règle lit des lignes de relations et appelle des primitives `must` ; elle ne parcourt ni CFG ni expression. Tout parcours vit dans le moteur et produit une relation nommée dont les colonnes déclarent leur polarité.

Cet invariant n'est pas un vœu : il est tenu par un test à cliquet. Le test lit le texte de chaque fichier de `src/rules/`, hors modules de test, et y cherche des marqueurs de parcours syntaxique ; seuls les fichiers d'une liste figée ont le droit d'en contenir, et la liste ne peut que rétrécir :

Extrait 4.23 – `tests/layer_boundary.rs`, l. 19–41 — la liste d'autorisation et les marqueurs du cliquet

```
19 /// The rule files still walking syntax, as of ADR-042 slice 5. Shrink only.
20 const ALLOWED: &[&str] = &[
21     "src/rules/api/query.rs",
22     "src/rules/api/witness.rs",
23     "src/rules/helpers/jsx.rs",
24     "src/rules/helpers/mod.rs",
25     "src/rules/helpers/mount.rs",
26     "src/rules/helpers/purity.rs",
27     "src/rules/impls/redundant_set_state.rs",
28     "src/rules/impls/setter_in_render.rs",
```

```

29     "src/rules/impls/state_mutation.rs",
30     "src/rules/impls/unnecessary_rerender.rs",
31 ];
32
33 /// What counts as touching syntax: iterating a CFG's blocks, matching
34 /// statements or terminators, or descending an expression tree.
35 const MARKERS: &[&str] = &[
36     "cfg.blocks",
37     ".blocks.values()",
38     "Stmt::",
39     "for_each_child",
40     "Terminator::",
41 ];

```

Décision : ADR-42

ADR-42 pose la frontière en une phrase : « a fact computed in two places is two facts that drift, and #26 is the drift ». Les relations (écrivains de slots, seeds, enregistrements, déclencheurs d'effets, churn) sont des *produits du moteur*, calculées une fois à la convergence ; la couche règles les lit. ADR-6 avait déjà fait des règles des « post-passes pures » sur le point fixe ; ADR-42 ajoute qu'elles ne refont pas de parcours. Alternative refusée : laisser chaque règle marcher la syntaxe à sa guise, ce qui fait diverger les faits d'une règle à l'autre et, en pratique, a produit des faux négatifs.

Piège

La liste de l'ADR et celle du test ne coïncident pas au commit étudié. ADR-42 §1 cite `impls/conditional_hook.rs` parmi les fichiers autorisés et ne cite pas `helpers/mod.rs` ; le test (`tests/layer_boundary.rs`, l. 20–31) autorise `helpers/mod.rs` et ne contient plus `impls/conditional_hook.rs`. C'est le test qui fait foi — c'est lui qui échoue. De même, ADR-42 §5 annonce que `ProgramRelations` « remplace » `ProgramCache` ; en réalité `ProgramCache` existe toujours et *contient* un `ProgramRelations`, en attendant que ses trois autres structures descendent dans le moteur. Quand un ADR et le code divergent, lire le code, puis chercher dans l'historique quel amendement manque.

4.3.4 Règles → coquille : un driver pour tous les hôtes

La dernière frontière sépare ce qui est *calculé* de ce qui est *montré*. Le driver ne crée aucun diagnostic et ne peut élever aucune sévérité ; il filtre les INFO, compte, choisit le code de sortie et rend. Surtout, tout ce qui dépend du système d'exploitation reste hors de lui : parsing d'argv, détection du terminal et de NO_COLOR, écriture des flux, sortie du processus. Le commentaire d'en-tête de `src/driver/mod.rs` le dit : une seule fonction, « used verbatim by the native CLI and the WASM entry point, so their behavior cannot fork » (ADR-22 §6). Le test `tests/memfs_parity.rs` vérifie que la sortie est identique octet pour octet entre `OsFileSystem` et `MemFileSystem` sur plusieurs projets de test.

Remarque

Les dépendances entre modules suivent les couches, avec deux nuances qu'il vaut mieux connaître. `src/engine/render_deps.rs` et `src/rules/helpers/render_tree.rs` importent deux prédicats de nom de `src/lowering/` (`is_hook_name`, `is_event_prop`) : ce sont des fonctions sur des chaînes, pas un accès à l'AST. Et aucun module du moteur, des domaines, de l'IR ou du front-end n'importe `crate::rules` : la couche règles est une feuille, qu'on peut retirer sans toucher au reste.

4.4 Les conventions transversales

Quatre conventions reviennent dans presque tous les fichiers. Les connaître évite de les redécouvrir une par une.

4.4.1 Fail-closed : l'absence n'est pas une preuve

Exemple 4.2: Un nom importé

Dans `import { useData } from "../hooks"`, le moteur veut savoir si `useData` est un hook custom défini dans le projet, pour en greffer le corps. Si la résolution échoue (fichier absent, alias `tsconfig` non chargé), deux lectures sont possibles : « ce n'est pas un hook du projet, on l'ignore », ou « on ne sait pas ce que c'est ». La première rend l'appel invisible, et tout ce qu'il fait avec lui ; c'est un faux négatif. La seconde le traite comme inconnu ($\top$), ce qui est sûr, et le dit. C'est ce que fait l'analyseur sur un composant `Panel` qui appelle `useData()` importé d'un `./hooks` absent (sortie observée avec `-info`) :

```
Panel (2 hooks) panel.tsx
info  analysis-limit [hook:0] (line 5:8) hook `useData` was not found in the registry. Pass its source file
      ↳ or add a HookSummary to analyse it (FN possible)
suspended analysis-limit 4 passing check(s) withheld: the analysis was truncated in this component, so they
      ↳ are not guaranteed
```

Les assurances « verified » du composant sont retirées : l'absence de finding n'y vaut plus preuve d'absence de défaut.

Définition 4.2: Fail-closed

Une question est tranchée *fail-closed* quand l'absence de preuve est lue comme « non prouvé » — donc comme $\top$ ou comme l'hypothèse qui préserve les diagnostics — et jamais comme une preuve du contraire.

Le dépôt applique la règle partout où une table peut être incomplète. `FunctionRegistry::resolve` ne devine jamais un utilitaire par son seul nom à travers les fichiers :

Extrait 4.24 – `src/engine/function_registry.rs`, l. 51–63 — résolution fail-closed d'un utilitaire

```
51  /// Resolve a bare callee name at a call site, fail-closed (ADR-027 §3):
52  /// a definition in the caller's own file, else the caller's resolved
53  /// import edge for that local name – never a by-name guess across files
54  /// (the first-match fallback silently spliced the wrong body on a name
55  /// collision, and an aliased import did not resolve at all).
56  pub fn resolve(&self, caller_file: &Path, name: &str) -> Option<&FunctionIR> {
57      let key = (caller_file.to_path_buf(), name.to_string());
58      self.functions.get(&key).or_else(|| {
59          self.aliases
60              .get(&key)
61              .and_then(|target| self.functions.get(target))
62      })
63  }
```

Les commentaires de types répètent la même discipline : la variante `ModuleConstInit::Context` est « Two-valued on purpose — absence means ‘not proven’, never ‘not a context’ » (`src/ir/component.rs`, l. 30–33) ; le champ `module_table` de `ProgramAnalysisResult` doit être lu comme « absent = *unproven* » ; le champ `phase1_reached` existe précisément pour qu'un composant sans appelant connu ne soit pas pris pour une racine prouvée (`src/engine/program_result.rs`, l. 52–68). Du côté du driver, le même réflexe produit les *blind spots* : un fichier abandonné, un alias non résolu ou un import non lu interdisent le « no issues found » final.

4.4.2 Must et may comme types

Le niveau ERROR promet un défaut certain ; il ne doit donc être construit que sur un fait must (définition 3.7). Plutôt que de le confier à la vigilance de chaque auteur de règle, le projet l'inscrit dans les types :

Extrait 4.25 – `src/rules/api/query.rs`, l. 115–129 — un résultat must porte un jeton de preuve ; un fait may n'en porte pas

```
115 /// (Deviates from the ADR-021 §1 literal `All(T)`: the token lives *inside* `All`
116 /// so `must_*` and the `Vec<Certified<_>>` primitives share one minting story.)
117 #[derive(Debug, Clone, PartialEq)]
118 pub enum MustResult<T> {
119     /// Proven on all paths – carries the minted proof.
120     All(Certified<T>),
121     /// Proven on some but not all paths – a MAY fact (raw payload).
122     Some(T),
123     /// No qualifying evidence at all.
124     None,
125 }
126
127 /// A MAY fact. There is no path from `May<_>` to an `Error`.
128 #[derive(Debug, Clone, Copy, PartialEq, Eq)]
129 pub struct May<T>{pub T};
```

Le jeton `Certified<E>` n'a qu'un constructeur, privé au module `query.rs` ; seules les primitives must de ce module peuvent en frapper un. Et `Diagnostic::error` en exige un :

Extrait 4.26 – `src/rules/api/diagnostic.rs`, l. 157–167 — le seul constructeur d'un Error

```
157 /// The only constructor of an `Error` (ADR-021 §2). Builds the finding
158 /// *from* a proof: the certified evidence's span/label/witness ride along, so
159 /// they need not be re-threaded by hand. A `May<_>`/`MustResult::Some` value
160 /// has no `Certified` to pass here – Error-on-may is a type error.
161 ///
162 /// Further `.with_*` builders may still refine/override the absorbed fields.
163 pub fn error<E>(<
164     rule: impl Into<Cow<'static, str>>,
165     proof: Certified<E>,
166     message: impl Into<String>,
167 ) -> Self {
```

Un `May<T>` ou un `MustResult::Some` n'a pas de `Certified` à passer : produire un ERROR sur un fait may est une *erreur de compilation*. Le `Diagnostic` lui-même vit dans un module feuille pour que ses champs privés restent hors de portée des règles (liste 4.20). Le chapitre 17 détaille cette surface typée (ADR-21).

4.4.3 Identités internées : `FileId`, `ComponentId`

Exemple 4.3: Deux Widget

Un projet contient `a/Widget.tsx` et, plus tard, `b/Widget.tsx`. Le rapport doit distinguer les deux : il affiche `Widget@a/Widget.tsx`. Si l'analyse utilisait cette chaîne d'affichage comme clé, l'ajout du second fichier renommerait le premier composant dans toutes les tables — résultats, store partagé, labels de stabilité — et une recherche écrite avec l'ancienne orthographe manquerait.

La règle du projet est de séparer l'identité de l'affichage. Une identité est un petit entier

Copy, interné une fois dans une table; le nom affiché est calculé au rendu seulement :

Extrait 4.27 – src/ir/source_range.rs, l. 3–9 — l'identité d'un fichier

```
3 /// Identity of a source file, interned in a [`FileTable`] (ADR-019).
4 ///
5 /// 4 bytes and `Copy`, so [`SourceRange`] stays `Copy` while carrying file
6 /// identity – the fix for the ADR-011 limitation where a span inside a
7 /// cross-file inlined CFG could not name the file it points into.
8 #[derive(Debug, Clone, Copy, PartialEq, Eq, Hash, PartialOrd, Ord)]
9 pub struct FileId(u32);
```

Extrait 4.28 – src/ir/component_id.rs, l. 23–29 — l'identité d'un composant

```
23 /// A component's identity, interned in a [`ComponentTable`].
24 ///
25 /// `Copy` and 4 bytes, so it sits inside a `BTreeSet` label or a store key
26 /// without the allocation a name costs, and two ids compare in one
27 /// instruction.
28 #[derive(Debug, Clone, Copy, PartialEq, Eq, Hash, PartialOrd, Ord)]
29 pub struct ComponentId(u32);
```

Le ComponentTable est construit par ComponentRegistry::from_components, le FileTable pendant le lowering; tous deux voyagent jusqu'au driver dans le ProgramAnalysisResult. L'historique de cette règle (ADR-19 pour les fichiers, ADR-40 pour les composants) est raconté au chapitre 26.

4.4.4 Positions : SourceRange

Toute position que l'utilisateur voit ((line 5:2)) vient d'un SourceRange :

Extrait 4.29 – src/ir/source_range.rs, l. 36–42 — une position source

```
36 /// A `(file, line, col)` source position. `line` is 1-indexed, `col` 0-indexed.
37 #[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
38 pub struct SourceRange {
39     pub file: FileId,
40     pub line: u32,
41     pub col: u32,
42 }
```

La position est calculée pendant le lowering, à partir de l'offset de début de chaque nœud d'oxc et d'une table des débuts de ligne construite une fois par fichier (SourceMap). Parce qu'elle porte un FileId, une position à l'intérieur d'un hook custom inliné depuis un autre fichier nomme le bon fichier. La ligne est comptée à partir de 1, la colonne à partir de 0 : le useEffect de liste 4.1, en retrait de deux espaces, est en 5:2.

4.4.5 Les alias de base

Enfin, quelques noms de types ne sont que des alias, qu'il faut connaître pour lire les signatures :

Extrait 4.30 – src/ir/types.rs, l. 1–9 — les alias de base de l'IR

```
1 pub type Symbol = String;
2 pub type HookLabel = usize;
3 pub type BlockId = usize;
4 pub type Var = String;
5
```



```

6 /// A state slot qualified by the component that owns it. `HookLabel` is
7 /// per-component, so every cross-component fact – a `Versioned` label set, a
8 /// churn node, a foreign writer row – names a slot this way (ADR-042 §2).
9 pub type QualifiedSlot = (super::component_id::ComponentId, HookLabel);

```

Un `HookLabel` est l'indice d'un hook dans sa table, propre à un composant ; un slot se désigne donc hors de son composant par un `QualifiedSlot`. `ExprId` (`src/ir/types.rs`, l. 11–15) identifie un *site* d'allocation (définition 6.2) ; il dérive `Ord` pour que les parcours d'ensembles de sites aient un ordre stable — le déterminisme de la sortie est une convention à part entière (#120).

4.5 Les trois principes de CLAUDE.md, dans le code

Le fichier `CLAUDE.md`, à la racine du dépôt, énonce trois principes de conception « non négociables » et deux invariants. Ajouté après les dix-neuf premiers ADR, il codifie des pratiques déjà à l'œuvre (chapitre 26). Voici comment chacun se lit dans le code qu'on vient de parcourir.

1. Pas de workarounds. « Corriger la cause racine, jamais le symptôme. » Exemple : un diagnostic dont la règle a mis une position dans un témoin mais pas sur le diagnostic lui-même s'affichait sans position. Plutôt que de corriger les règles une à une, le registre complète la position pour toutes, une fois :

Extrait 4.31 – `src/rules/registry.rs`, l. 356–369 — la position manquante est complétée au niveau central

```

356 /// A finding with no position of its own takes the first position its witness
357 /// chain names.
358 ///
359 /// The chain is the finding's evidence, and its first located step is where
360 /// that evidence starts – a rule that threaded a span into a `Step` but never
361 /// onto the diagnostic was withholding a position it already had. Applied here
362 /// rather than in each rule, because "a finding renders somewhere" is a
363 /// property of every finding (#131).
364 fn located(mut d: Diagnostic) -> Diagnostic {
365     if d.range.is_none() {
366         d.range = d.notes.iter().find_map(|n| n.range);
367     }
368     d
369 }

```

2. La règle du paragraphe unique. « Si une décision demande plus d'un paragraphe pour être justifiée, c'est une mauvaise idée. » Les ADR récents portent leur justification en une phrase explicite ; ADR-42 §1 écrit littéralement « One sentence justifies it », puis la phrase citée plus haut. Même la décision de garder un fichier dont le parseur a récupéré une erreur (liste 4.9) tient en une phrase dans son commentaire : sauter le fichier serait un faux négatif interdit.

3. Modulaire et général d'abord. « Un problème qui touche plusieurs règles se corrige une fois, au niveau central. » Trois exemples dans ce chapitre : `ProgramRelations` (« adding a program-level relation means one more field here — never a rebuild in a rule ») ; les `blind spots`, « collected here, once, at the driver level » (`src/driver/blind_spots.rs`) ; et la primitive de `splice`, « one CFG-splice primitive, shared by utility inlining and custom-hook expansion » (`src/ir/splice.rs`), qui a remplacé deux copies divergentes dont l'une perdait des blocs — un faux négatif.

Les invariants. Le premier est la soundness : « Faux positifs tolérés, faux négatifs INTERDITS » (définition 3.8). Le second fixe le sens des niveaux ; le code en porte la définition, mot pour mot, sur l'énumération `Severity` :

Extrait 4.32 – `src/rules/api/diagnostic.rs`, l. 24–39 — les trois niveaux de diagnostic, définis dans le code

```
24 /// Confidence level of a diagnostic.
25 ///
26 /// - `Error` the defect is certain whenever the flagged code runs. Built only
27 ///   from a proof of the whole claim, not of one of its conjuncts (#142).
28 /// - `Warning` a possible defect (conditional path, over-approximation), or a
29 ///   certain fact whose cost is not (a fresh reference, one wasted render).
30 /// - `Info` not actionable without context: a known analysis limitation
31 ///   (widening, depth cap), or a pattern that looks intentional (a seed-once
32 ///   prop name, a cheap pure initializer). Hidden by default; show with --info.
33 #[derive(Debug, Clone, Copy, PartialEq, Eq, Default)]
34 pub enum Severity {
35     Error,
36     #[default]
37     Warning,
38     Info,
39 }
```

C'est pourquoi Counter reçoit un `WARNING` et non un `ERROR` : le message le dit, « its deps do not provably gate it ». Une partie de la conclusion n'est pas prouvée `must`, et un `ERROR` exige la preuve de *toute* la conclusion (chapitre 19).

Décision : ADR-20

CLAUDE.md renvoie aussi à ADR-20, qui clôt une campagne de dette technique en listant les simplifications *refusées* parce qu'elles auraient coûté de la soundness, et au tracker GitHub, où les limites tranchées « on ne corrige pas » sont des issues fermées `wontfix`. Règle pratique pour un contributeur : avant de proposer un nettoyage ou un correctif, lire ces deux sources ; beaucoup de « duplications » apparentes sont des variations voulues.

4.6 Comment naviguer

4.6.1 Les tests comme documentation exécutable

Le dépôt compte 632 tests unitaires (blocs `#[cfg(test)]` sous `src/`) et 69 fichiers de tests d'intégration sous `tests/`, plus 43 fixtures sous `tests/fixtures/` (fichiers TSX et petits projets complets). Chaque suite d'intégration porte un nom qui dit ce qu'elle fixe : `effect_cycles.rs`, `stale_closure.rs`, `cross_file_context.rs`, `memfs_parity.rs`... Pour comprendre un comportement, le plus rapide est souvent de trouver le test qui le fixe et de le lancer seul :

```
cargo test --test effect_cycles      # une suite d'intégration
cargo test --lib engine::dominance  # les tests unitaires d'un module
```

Les tests d'intégration « bibliothèque » rejouent le pipeline sans le driver. Le harnais de `tests/corpus_fp_fixes.rs` en est la version la plus lisible : parse, lowering, internement des identités, analyse intra-composant, puis toutes les règles.

Extrait 4.33 – `tests/corpus_fp_fixes.rs`, l. 14–69 — le pipeline en miniature, dans un test

```
14 fn diagnostics(src: &str) -> Vec<(String, String)> {
15     let alloc = Allocator::default();
```

```

16     let ret = Parser::new(&alloc, src, SourceType::tsx())
17         .with_options(ParseOptions::default())
18         .parse();
19     assert!(
20         ret.diagnostics.is_empty(),
21         "parse errors: {:?}",
22         ret.diagnostics
23     );
24     let components = lower_program(
25         &ret.program,
26         src,
27         std::path::Path::new("test.tsx"),
28         &mut Default::default(),
29     );
30     assert!(!components.is_empty(), "no component detected");
31
32     let mut components_map = std::collections::HashMap::new();
33     let mut names = Vec::new();
34
35     let mut component_table = reactant::ir::ComponentTable::default();
36     for comp in components {
37         let id = component_table.intern(reactant::ir::CompOrigin {
38             file: comp.file.clone(),
39
40             name: comp.name.clone(),
41         });
42
43         let result = reactant::engine::analyze_component_as(
44             comp,
45             id,
46             &StateValueTransfer,
47             &Config::default(),
48         );
49
50         components_map.insert(id, result);
51
52         names.push(id);
53     }
54     let prog = reactant::engine::ProgramAnalysisResult {
55         components: components_map,
56         component_table,
57         ..Default::default()
58     };
59
60     let mut out = Vec::new();
61     for name in &names {
62         for rule in all_rules() {
63             for d in rule.check(&RuleCtx::new(&prog, *name)) {
64                 out.push((d.rule.to_string(), d.message.clone()));
65             }
66         }
67     }
68     out
69 }

```

On y retrouve les étapes de la section 4.2, à une différence près : `analyze_component_as` analyse chaque composant isolément, sans parents ni enfants, alors que la CLI passe par `analyze_program`. Les tests inter-composants appellent `analyze_program`; `tests/memfs_parity.rs` et les tests du binaire (`env!("CARGO_BIN_EXE_reactant")`) passent par le driver. `src/test_support.rs` fournit aux tests unitaires des constructeurs de CFG à un bloc et de résultats d'analyse. Le chapitre 25

décrit ces six altitudes de test et le corpus de mesure.

4.6.2 Les options qui montrent l'intérieur

La CLI offre quatre fenêtres sur le fonctionnement interne, toutes utilisées dans ce chapitre.

- trace** déroule la chaîne de témoins de chaque diagnostic (les lignes $\rightarrow$), au lieu du seul décompte « 2 trace step(s) ».
- verbose** écrit sur stderr le graphe de symboles, le nombre de composants analysés, les hits et misses du cache inter-composants et, par composant, le nombre d'itérations du point fixe et les labels élargis.
- info** montre les diagnostics INFO et, par composant affiché, les assurances « verified » des règles applicables qui n'ont rien trouvé. Avec **-show-clean**, les composants sans finding sont aussi listés.
- format json** donne le rapport complet sous forme d'un seul document, avec les notes typées (kind, slot, iteration...).

Sur Counter, **-info** ajoute ceci (observé, avec **-verbose**) :

```
info widening-info (line 5:2) state `count` kept changing during analysis and was approximated to converge, so
  ↳ findings that depend on it may be imprecise
  (2 trace step(s), rerun with --trace)
verified always-unstable-deps no deps array is defeated by an always-fresh reference
verified conditional-hook all hooks run unconditionally, in a stable order
verified derived-state no effect merely mirrors other state
verified lazy-init no useState/useRef initializer re-runs work on every render
verified missing-cleanup every effect that starts something long-lived also tears it down
verified missing-deps every effect declares the variables it reads
verified redundant-set-state no setState writes the value the state already holds
verified setter-in-render no setter is called during render
verified state-mutation no state or prop object is mutated in place
```

Deux sous-commandes documentent les règles sans lire le code : **reactant rules** liste les 21 noms de diagnostic (les dix-neuf règles natives en émettent deux de plus, **cross-component-infinite-loop** et **cross-setter-in-render**) ; **reactant explain <règle>** affiche la documentation complète d'un nom, tirée de `src/rules/docs.rs`. Extrait observé :

```
infinite-loop
  effect sets state that re-triggers the effect, and the state diverges

The render → effect → setState → render cycle never stabilizes: the abstract value of the state slot required
  ↳ widening and the effect's write stays unbounded, the structural signature of an infinite render loop. [...]

Example:
useEffect(() => { setCount(count + 1); }, [count]);
```

Piège

Il n'existe *aucune* option pour afficher l'IR ou les valeurs abstraites convergées : **-verbose** ne donne que des statistiques, et les types de l'IR ne dérivent que Debug, sans pretty-printer. Pour voir l'IR d'un composant, il faut écrire un petit programme qui appelle `reactant::lowering::lower_program` et imprime le résultat — c'est ce qu'ont fait les dossiers de préparation de ce manuscrit. Autre écart à connaître : l'aide de **-format** annonce « schema v1, see docs/usage.md », alors que le document émis porte "version": 2 et que `src/driver/json.rs` se décrit comme « schema v2 ». C'est la sortie qui fait foi.

4.6.3 Les documents du dépôt

Hors du code, cinq sources servent en permanence : docs/adr/ (les 42 ADR, résumées en annexe chapitre A), docs/relations.md (le catalogue des relations et de leurs polarités, chapitre 14), docs/limitations.md (les limites connues, chapitre 27), docs/usage.md (la CLI et le format JSON) et le tracker GitHub, dont les labels soundness-bug, precision-fn, precision-fp classent le backlog.

4.7 Feuille de route des chapitres suivants

La tableau 4.1 dit où chaque fichier est traité. Le tableau inverse, plus court, dit ce que chaque chapitre ajoute au flot qu'on vient de suivre.

Chapitre	Étape du flot et fichiers principaux
5	quelles fonctions sont des composants, hooks, utilitaires : lowering/mod.rs, les détecteurs, import_resolution.rs
6	les types de l'IR : ir/cfg.rs, expr.rs, hooks.rs, component.rs, component_id.rs
7	de l'AST au CFG : cfg_builder.rs, expr_lower.rs, hook_extractor.rs
8	greffer hooks custom et utilitaires : ir/splice.rs, remap.rs, expansions de fixpoint.rs
9–10	les valeurs abstraites : domains/impls/
11	l'interpréteur et les stores : domains/transfer/, interp/, stores/
12	un CFG jusqu'au point fixe local : cfg_analyzer.rs, dominance.rs, eval.rs
13	la boucle externe d'un composant : analyze_component_impl
14–15	la dernière tranche et le churn : setters.rs, written.rs, guards.rs, churn.rs
16	parents, enfants, fichiers : analyze_program, registres, render_deps.rs
17	la surface typée des règles : rules/api/, registry.rs
18–21	les dix-neuf règles natives : rules/impls/
22	les packs JSON : rules/declarative/
23–24	la coquille et les hôtes : driver/, cli/, project/, resolver/, reactant-wasm, npm/
25–27	la méthode : tests, corpus, histoire, limites

TABLE 4.2 – Ce que chaque chapitre ajoute au flot de la section 4.2.

Trois parcours de lecture sont possibles. Le parcours *linéaire* suit l'ordre des chapitres, c'est-à-dire le flot lui-même. Le parcours *règles d'abord* lit les chapitres 17 à 19 en revenant aux chapitres du moteur quand une relation est citée : il convient à qui veut écrire une règle. Le parcours *outil* lit les chapitres 23 à 25 : il convient à qui intègre l'analyseur dans une chaîne de CI.

Résumé du chapitre

- Deux crates (reactant, reactant-wasm) et un paquet npm. La bibliothèque contient tout, y compris le driver ; le binaire n'ajoute que clap et l'écriture des flux.
- Le flot d'une invocation : `main` → `cli::run` → `check::run` → `run_check` → `lower_files_with` (parse oxc, lowering) → `analyze_lowered` → `analyze_program` (point fixe, relations) → `ProgramCache` → `check_component` → `rendu` → `CheckOutput`.
- Les types qui passent d'une couche à l'autre : `ParserReturn`, `Candidate`, `ComponentIR`, `LoweredProgram`, `AnalysisResult`, `ProgramAnalysisResult`, `ProgramRelations`, `Diagnostic`, `CheckReport`, `CheckOutput`.
- Cinq couches, quatre frontières : plus d'AST après le lowering (ADR-3) ; un moteur générique en `Transfer`, cycle cassé par `AnalyzeChildFn` ; des règles qui lisent des relations

sans parcourir l'IR (ADR-42, `tests/layer_boundary.rs`); un driver partagé par tous les hôtes (ADR-22).

- Quatre conventions : `fail-closed`, `must` et `may` comme types (`Certified`, `Diagnostic::error`), identités internées (`FileId`, `ComponentId`), positions `SourceRange`.
- Les principes de `CLAUDE.md` se lisent dans le code : correction au niveau central (`located`, `blind spots`, `splice unique`), justification en une phrase, `soundness`, niveaux définis sur `Severity`.
- Pour naviguer : le test qui fixe un comportement, `-trace`, `-verbose`, `-info`, `-format json`, `rules`, `explain`. Pas de dump d'IR.

Le chapitre suivant ouvre la première couche : comment le front-end décide, dans un fichier, quelles fonctions sont des composants, des hooks et des utilitaires.

4.8 Exercices

Exercice 4.1: Situer un fichier

Pour chacun des fichiers suivants, dire à quelle couche il appartient et quel type il consomme ou produit à la frontière : `src/lowering/hook_extractor.rs`, `src/engine/triggers.rs`, `src/rules/api/diagnostic.rs`, `src/driver/json.rs`.

Solution

`hook_extractor.rs` : front-end ; il réécrit le CFG et produit les `HookEntry` d'un `ComponentIR`.
`triggers.rs` : moteur ; il produit la relation `effect_triggers`, rangée dans `AnalysisResult`.
`diagnostic.rs` : règles ; il définit `Diagnostic`, la sortie de la couche. `json.rs` : coquille ; il consomme un `CheckReport` et produit le texte de `stdout`.

Exercice 4.2: Où ajouter un calcul ?

Une nouvelle règle a besoin de savoir, pour chaque effet, quels `setTimeout` il programme. Un contributeur propose de parcourir le CFG de l'effet dans `check`. Que répond le cliquet de `tests/layer_boundary.rs` ? Où faut-il faire ce calcul ?

Solution

Parcourir les blocs d'un CFG dans un fichier de `src/rules/` introduit un marqueur (`.blocks.values()`, `Stmt::...`) : si le fichier n'est pas dans la liste, le test échoue, et la liste ne peut pas grandir. Le calcul doit être une relation du moteur, calculée à la convergence et lue par la règle. En l'occurrence, la relation `registrations` (`src/engine/registrations.rs`) décrit déjà les callbacks qu'un effet confie à un registrar ; la bonne solution est de la lire, ou d'y ajouter une colonne.

Exercice 4.3: Un fichier illisible

On lance `reactant check a.tsx b.tsx`, où `b.tsx` contient une erreur de syntaxe dont `oxc` se remet, et `a.tsx` une erreur qui le fait paniquer. Lequel des deux est analysé ? Que voit l'utilisateur, et pourquoi ce choix est-il imposé par la `soundness` ?

Solution

D'après liste 4.9, `b.tsx` est abaissé et analysé malgré son erreur (le message `[parse error]` n'apparaît qu'en format humain) ; `a.tsx` est sauté, signalé par `[skipped]` sur `stderr` dans tous les formats, et un `blind spot` « `could not be parsed and were dropped` » interdit le « `no issues found` ». Sauter `b.tsx` parce qu'il contient une erreur rendrait invisibles tous ses

défauts : un faux négatif. Pour `a.tsx`, l'analyse est impossible ; la soundness impose alors de le dire, pas de se taire.

Exercice 4.4: Suivre un label

Dans la sortie JSON de Counter, le diagnostic porte `"hook_label": 0` et sa première note `"hook_label": 1`. D'où viennent ces nombres, et pourquoi ne suffisent-ils pas, seuls, à désigner un slot dans le graphe de churn du programme ?

Solution

Ce sont des `HookLabel`, attribués par `extract_hooks` dans l'ordre des appels de hooks du corps de rendu : 0 pour le `useState` (le slot count), 1 pour le `useEffect`. Un `HookLabel` est propre à un composant ; deux composants ont chacun un label 0. Toute relation de programme désigne donc un slot par un `QualifiedSlot = (ComponentId, HookLabel)` (`src/ir/types.rs`, l. 6–9).

Deuxième partie

Du source à la représentation intermédiaire

Chapitre 5

Le front-end : oxc et la détection des composants, hooks et utilitaires

Objectifs du chapitre.

- Savoir ce que le parseur oxc livre au projet (arène, Program<'a>, spans, ParserReturn) et pourquoi un fichier dont le parseur a récupéré une erreur est quand même analysé.
- Connaître le contrat du front-end : ce qui entre (un AST par fichier), ce qui sort (des IR possédées et des faits de module), et la frontière « aucun domaine ne voit l'AST ».
- Lire le marcheur commun `detect_fns`, le type `Candidate` et les trois prédicats qui définissent un *composant*, un *hook custom* et un *utilitaire*.
- Comprendre deux faux négatifs corrigés (#5, #122) et la leçon qu'ils portent : au front-end, une détection trop étroite est un défaut de soundness.
- Suivre la provenance d'un appel de hook (`HookOrigin`, `ImportCtx`, `classify_callee`), les faits de module (constantes, contextes, directives, arêtes d'import) et l'identité d'un callee JSX (`CompOrigin`, `resolve_child`).
- Savoir énumérer les formes que le front-end ne voit pas, et dire pour chacune si l'utilisateur en est averti.

Prérequis : chapitre 4 (le pipeline et ses couches), chapitre 2 (composant, règles des hooks, Context, Server Components), chapitre 3 (soundness, définition 3.8 ; fait `must` / `may`, définition 3.7).

Tout commence par une question qui semble triviale : dans ce fichier, quelles fonctions sont des composants ? Considérons le programme suivant.

Extrait 5.1 – Quatre fonctions de niveau supérieur (/tmp/ch05/classes/App.tsx)

```
1 import { useState, useEffect } from "react";
2
3 export function formatCount(n: number) {
4   return `count: ${n}`;
5 }
6
7 export function useCounter(initial: number) {
8   const [n, setN] = useState(initial);
9   return { n, inc: () => setN(n + 1) };
10 }
11
12 export function Counter() {
13   const { n, inc } = useCounter(0);
14   return <button onClick={inc}>{formatCount(n)}</button>;
15 }
16
17 export function Beacon({ id }: { id: string }) {
18   const [n, setN] = useState(0);
```

```

19  useEffect(() => { setN(n + 1); });
20  return null;
21 }
22
23 function helper() {
24   return <span />;
25 }

```

Un lecteur React répond sans hésiter : Counter et Beacon sont des composants, useCounter est un hook custom, formatCount et helper sont des fonctions ordinaires. Mais sur quoi repose cette réponse ? Beacon ne renvoie jamais de JSX ; helper en renvoie, mais son nom commence par une minuscule ; rien, dans JavaScript, ne distingue une fonction « composant » d’une autre. React lui-même n’en sait rien avant de l’appeler : c’est le site d’usage `<Beacon/>` qui en fait un composant. Un analyseur statique, lui, doit trancher *avant* toute exécution, fichier par fichier, et sa réponse décide de ce qui sera analysé. Voici ce que `reactant-analyzer` répond :

```

$ reactant check /tmp/ch05/classes --no-color --show-clean
Beacon (2 hooks) /tmp/ch05/classes/App.tsx
  warn  infinite-loop [hook:0] (line 19:2) this effect keeps pushing state `n` to new values on every run.
        ↳ Potential infinite render loop
          (2 trace step(s), rerun with --trace)
Counter (1 hooks) /tmp/ch05/classes/App.tsx ✓

! 1 warning(s) across 1 file(s).

```

et, avec `-verbose`, le graphe de symboles construit sur les IR abaissées :

```

[verbose] symbol graph: 3 nodes, topo order = [useCounter@/tmp/ch05/classes/App.tsx, Counter@/tmp/ch05/classes/App.tsx,
        ↳ Beacon@/tmp/ch05/classes/App.tsx]
[verbose] 2 components analyzed

```

Deux composants, un hook ; helper n’apparaît nulle part. Le reste du chapitre explique pourquoi, et pourquoi ce « nulle part » est la décision la plus lourde de conséquences de tout le pipeline : une fonction que le front-end ne reconnaît pas n’est pas analysée approximativement, elle n’est *pas analysée du tout*.

Le sous-système tient dans dix fichiers de `src/lowering/` : `mod.rs` (565 lignes, la façade et les deux lowerers principaux), `detector.rs` (169, le marcheur commun), `component_detector.rs` (352), `hook_detector.rs` (162), `utility_detector.rs` (114), `jsx_detect.rs` (63), `hook_call_detect.rs` (109), `module_facts.rs` (163), `import_resolution.rs` (262) et `utility_lowerer.rs` (62). Le lowering des *corps* (`cfg_builder.rs`, `expr_lower.rs`, `hook_extractor.rs`) est l’objet du chapitre 7 ; nous le traitons ici comme une boîte noire, sauf là où le front-end lui transmet une information.

5.1 oxc en quelques pages

5.1.1 Une arène par fichier

Le projet ne contient pas de parseur : il utilise oxc [The26], un ensemble de crates Rust pour l’outillage JavaScript et TypeScript. Quatre crates sont déclarées dans `Cargo.toml`, toutes en version 0.138.0 : `oxc_allocator`, `oxc_parser`, `oxc_ast` et `oxc_span`. Le projet *n’utilise pas* `oxc_semantic` : il n’a ni table de symboles ni analyse de portées fournies par oxc. Toutes les résolutions de noms que ce chapitre décrit sont donc *syntactiques* et limitées au niveau supérieur du module ; nous verrons ce que cela coûte.

`oxc_allocator::Allocator` est une *arène* (allocateur « bump ») : tous les nœuds de l’AST y sont alloués, et libérés d’un bloc quand l’arène est détruite. D’où la durée de vie ‘a qui décore tous les types d’oxc : `Program<’a>`, `Statement<’a>`, `Expression<’a>`, `FunctionBody<’a>`,

FormalParameters<'a>. Un nœud emprunte l'arène et ne peut pas lui survivre. Voici la boucle qui, dans `src/resolver/mod.rs`, parse chaque fichier de l'analyse :

Extrait 5.2 – `src/resolver/mod.rs`, l. 289–312 — le parse d'un fichier et le traitement des erreurs

```

289     let alloc = Allocator::default();
290     let ret = OxcParser::new(&alloc, &source, source_type_for(path))
291         .with_options(ParseOptions::default())
292         .parse();
293     // `panicked` is oxc's only "this AST is unusable" signal (the program
294     // is empty). The parser recovers from every other syntax error and
295     // still returns a lowerable program, so skipping on a non-empty
296     // diagnostic list would drop whole files from the analysis – a
297     // forbidden false negative, and a growing one: oxc keeps moving TS
298     // semantic checks into the parser.
299     if ret.panicked || !ret.diagnostics.is_empty() {
300         lowered.parse_errors.push(ParseError {
301             file: path.clone(),
302             message: ret
303                 .diagnostics
304                 .first()
305                 .map(|d| d.message.to_string())
306                 .unwrap_or_else(|| "the parser produced no usable program".to_string()),
307             analyzed: !ret.panicked,
308         });
309     }
310     if ret.panicked {
311         continue;
312     }

```

L'arène est créée *dans* la boucle (`src/resolver/mod.rs`, l. 289) : l'AST d'un fichier ne survit pas à son itération, et la mémoire de pointe consacrée à l'AST est celle d'un seul fichier. Tout ce que le front-end veut garder doit donc être recopié dans des structures *possédées* (`String`, `PathBuf`, `Arc`) avant la fin de l'itération. C'est exactement ce que font les `IR` : `ComponentIR`, `HookIR`, `FunctionIR`, `ModuleFacts` n'ont aucun paramètre de durée de vie.

5.1.2 ParserReturn : panicked n'est pas diagnostics

`Parser::new(&alloc, source, source_type).with_options(ParseOptions::default()).parse()` renvoie un `ParserReturn`, dont le projet lit trois champs : `program`, `diagnostics` et `panicked`. La documentation d'oxc précise le sens du dernier : il vaut `false` si le parse a réussi *ou si le parseur a pu récupérer d'une erreur de syntaxe* ; quand il vaut `true`, `program` est vide et `diagnostics` contient au moins une erreur.

La distinction est décisive, et le commentaire de `src/resolver/mod.rs`, l. 293–298 (liste 5.2) en donne la raison : sauter un fichier dès que la liste de diagnostics est non vide ferait disparaître des fichiers entiers de l'analyse, alors que le programme récupéré est parfaitement abaissable. Ce serait un faux négatif, le seul type d'erreur que le projet s'interdit (définition 3.8), et un faux négatif croissant, puisqu'oxc déplace progressivement des vérifications sémantiques de TypeScript dans le parseur. Le projet ne saute donc que les fichiers `panicked`, et garde la trace des deux cas dans `ParseError` (`src/resolver/mod.rs`, l. 164–178), dont le drapeau `analyzed` vaut `true` quand le parseur a récupéré et que le fichier a quand même été abaissé : « a recovered syntax error is noise, while a dropped file means every finding it held is a silent false negative ».

Exemple 5.1: Erreur récupérée contre fichier abandonné

Le fichier `/tmp/ch05/parse/Broken.tsx` contient, au-dessus d'un composant `Loop` qui boucle, la ligne fonction `helper(a?: number = 1) return a;`, qu'oxc refuse (un paramètre ne peut être à la fois optionnel et initialisé) mais dont il récupère :

```
[parse error] /tmp/ch05/parse/Broken.tsx: A parameter cannot have a question mark and an initializer.
  Loop (2 hooks) /tmp/ch05/parse/Broken.tsx
    warn infinite-loop [hook:0] (line 7:2) this effect keeps pushing state `n` to new values on every run.
      ↪ Potential infinite render loop
        (2 trace step(s), rerun with --trace)

! 1 warning(s) across 1 file(s).
```

Le fichier est signalé *et* analysé : le défaut de `Loop` est trouvé. À l'inverse, `/tmp/ch05/parse2/Dead.tsx` (un JSX tronqué, `return <p>{</p>;` sans accolade fermante) fait paniquer le parseur :

```
[skipped] /tmp/ch05/parse2/Dead.tsx: Unexpected token, the file was not analyzed
! 0 file(s), no components detected, and parts of this run were not analyzed.
not analyzed:
  • 1 file(s) could not be parsed and were dropped, so nothing in them was analysed
```

Le fichier perdu devient un *blind spot* (« not analyzed »), et le rapport refuse de se déclarer propre.

Piège

Un fichier récupéré est abaissé depuis un AST *partiel* : oxc a pu sauter une partie du texte pour se resynchroniser, et les fonctions de cette partie manquent sans autre signal que la ligne `[parse error]` du canal humain. En JSON, seuls les fichiers *sautés* deviennent un blind spot `unparsed-files` (`src/driver/mod.rs`, l. 239–259). Un fichier `[parse error]` n'est donc pas une garantie de couverture complète.

5.1.3 Le type de source se choisit par extension

Le troisième argument du parseur est un `oxc_span::SourceType` : TypeScript ou JavaScript, JSX ou non, module ou script. Le projet le choisit sur la seule extension du fichier :

Extrait 5.3 – `src/resolver/mod.rs`, l. 193–200 — `source_type_for`

```
193 pub fn source_type_for(path: &Path) -> oxc_span::SourceType {
194     use oxc_span::SourceType;
195     match path.extension().and_then(|e| e.to_str()) {
196         Some("tsx") => SourceType::tsx(),
197         Some("ts") | Some("mts") | Some("cts") => SourceType::ts(),
198         _ => SourceType::unambiguous().with_jsx(true),
199     }
200 }
```

Deux décisions s'y lisent, que le commentaire de la fonction explique (`src/resolver/mod.rs`, l. 180–192). D'abord, `.js` et `.jsx` sont *tous deux* JSX-activés : du JSX dans un fichier non-JSX fait paniquer le parseur, et le commentaire rappelle que trois composants réels d'excalidraw ont été perdus ainsi. Ensuite, `.ts` ne peut pas partager ce drapeau, parce que `<T>expr` est une assertion de type en `.ts` et un élément JSX en `.tsx`. La fonction est publique pour une raison que son commentaire énonce : elle décide si un fichier est analysé, aucun appelant ne doit en garder une copie privée qui dériverait.

5.1.4 Spans et positions

Un `oxc_span::Span` est un couple `start: u32, end: u32` d'offsets d'octets. Le projet n'en utilise que `start`, qu'il convertit en position (`file`, `line`, `col`) grâce à une table des débuts de ligne calculée une fois par fichier :

Extrait 5.4 – `src/ir/source_range.rs`, l. 51–57 — `SourceMap`

```
51 /// Everything the lowering needs to mint [`SourceRange`]s for one file:
52 /// the line-start table plus the file's interned identity.
53 #[derive(Debug, Clone)]
54 pub struct SourceMap {
55     line_starts: Vec<u32>,
56     file: FileId,
57 }
```

La table est construite par `compute_line_starts` (`src/ir/source_range.rs`, l. 84–92), qui note l'offset suivant chaque `\n`, et `SourceMap::span_at` convertit un offset par recherche dans cette table (`src/ir/source_range.rs`, l. 76–80).

La ligne est comptée à partir de 1, la colonne à partir de 0 (`src/ir/source_range.rs`, l. 36) : dans la sortie de l'introduction, (line 19:2) désigne le `useEffect` de `Beacon`, indenté de deux espaces. Le `FileId` est un identifiant interné dans une `FileTable` (ADR-19) ; `FileTable::intern` est une recherche linéaire idempotente, si bien que les trois lowerers d'un même fichier obtiennent le même identifiant.

5.1.5 Ce que le front-end lit dans l'AST

Le front-end parcourt un petit sous-ensemble de l'AST d'oxc : `Program::directives` (le prologue de directives, qu'oxc range à part de `body`), `Program::body` et ses `Statement`, `Declaration`, `Expression`, `ExportDefaultDeclarationKind`, `ImportDeclarationSpecifier`, `ImportOrExportKind`, les types `TypeScript` `TSType`, `TSTypeName`, `TSSignature`, et les enfants `JSX` `JSXChild`. Deux détails d'oxc gouvernent des comportements que nous rencontrerons :

- `ArrowFunctionExpression::expression: bool` (« Is the function body an arrow expression ? i.e. `() => expr` instead of `() => { }` », dans `oxc_ast-0.138.0/src/ast/js.rs`) est le *seul* moyen de distinguer une flèche concise d'une flèche à corps bloc : oxc range l'expression d'une flèche concise dans un `FunctionBody` d'une seule `ExpressionStatement`. C'est l'origine de `#5` (section 5.5).
- `ParseOptions::default()` a `preserve_parens: true` : les parenthèses deviennent des nœuds `ParenthesizedExpression`. Tout code qui filtre une forme d'expression doit donc les « peler ». Certains le font, d'autres non (section 5.9).

5.2 Le contrat du front-end

5.2.1 Ce qui entre, ce qui sort

Le chapitre 4 a présenté le pipeline d'ensemble. Le front-end en est la première couche qui *comprend* React : il lit l'AST oxc pour décider *quoi* abaisser (quelles fonctions sont des composants, des hooks custom, des utilitaires) et pour extraire des *faits syntaxiques de module* (directives, arêtes d'import, constantes de module, origines des imports et des calées JSX). La figure 5.1 montre ce qui se passe pour un fichier.

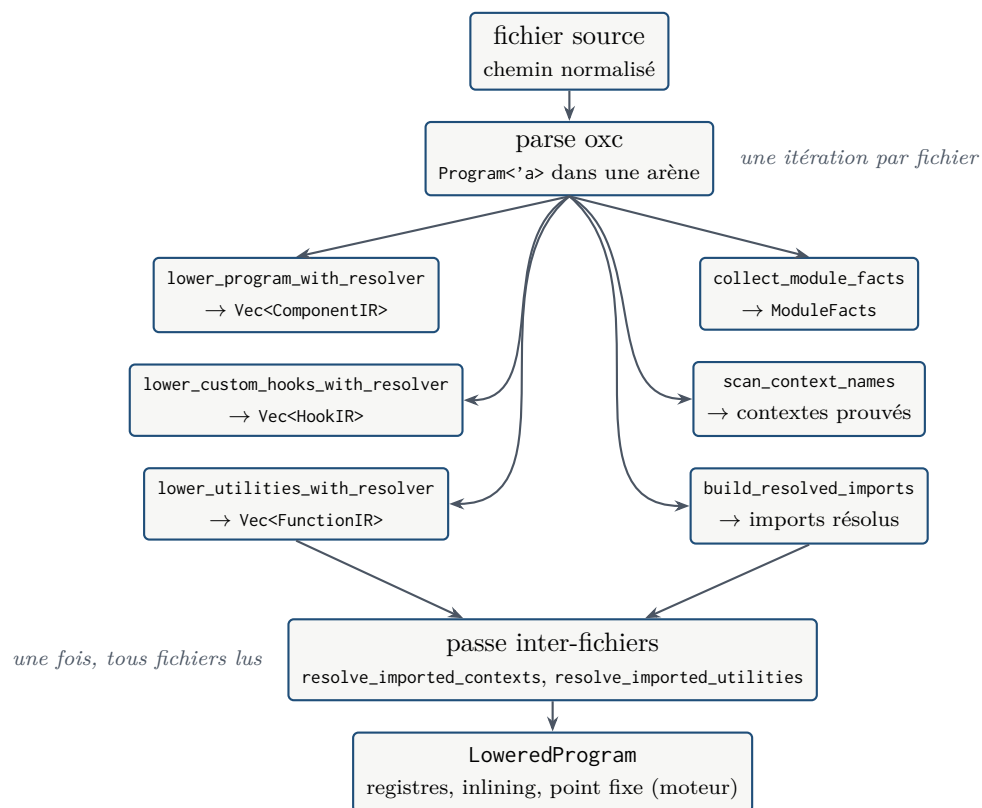


FIGURE 5.1 – Le front-end d’un fichier. Les six fonctions d’entrée lisent le même `Program` indépendamment ; l’AST est libéré à la fin de l’itération. La passe inter-fichiers ne voit plus que des structures possédées.

Les six appels sont faits, dans cet ordre, par la boucle de `lower_files_with` (`src/resolver/mod.rs`, l. 313–345) ; le tableau 5.1 les récapitule.

Fonction d'entrée	Sortie	Consommateur
<code>lower_program_with_resolver</code> (<code>src/lowering/mod.rs</code> , l. 437)	<code>Vec<ComponentIR></code>	<code>ComponentRegistry</code>
<code>lower_custom_hooks_with_resolver</code> (<code>src/lowering/mod.rs</code> , l. 373)	<code>Vec<HookIR></code>	<code>HookRegistry</code> (inlining des hooks custom)
<code>lower_utilities_with_resolver</code> (<code>src/lowering/utility_lowerer.rs</code> , l. 38)	<code>Vec<FunctionIR></code>	<code>FunctionRegistry</code> (inlining des utilitaires)
<code>collect_module_facts</code> (<code>src/lowering/module_facts.rs</code> , l. 17)	<code>ModuleFacts</code>	<code>ModuleTable</code> : blind spot unread-imports, server-component-hook, -follow-imports
<code>scan_context_names</code> (<code>src/lowering/mod.rs</code> , l. 197)	<code>HashSet<String></code>	<code>resolve_imported_contexts</code>
<code>build_resolved_imports</code> (<code>src/lowering/import_resolution.rs</code> , l. 140)	<code>HashMap<String, ResolvedImport></code>	<code>resolve_imported_contexts</code> , <code>resolve_imported_utilities</code>

TABLE 5.1 – Les fonctions d'entrée du front-end, par fichier.

Les variantes sans suffixe (`lower_program`, `lower_custom_hooks`, `lower_utilities`) appellent les précédentes avec `DefaultImportResolver::default()`, qui lit le système de fichiers réel ; elles servent aux tests et aux intégrations simples. Le suffixe `_with_resolver` existe pour l'interface « plugin » : le résolveur d'imports est un trait (`ImportResolver`, ADR-13 §2), que le driver instancie selon le type de projet (alias `tsconfig`, `Vite`, `Next.js` ; voir chapitre 23).

5.2.2 L'invariant de frontière

Invariant 5.1: Les domaines ne voient jamais l'AST

Aucune structure produite par le front-end ne contient de référence dans l'arène d'oxc. `ComponentIR`, `HookIR`, `FunctionIR`, `ModuleFacts`, `ResolvedImport` ne portent que des données possédées ; les emprunts (`Candidate<'a>`, `FnItem<'a>`) ne vivent que le temps d'un fichier. Au-delà du front-end et du lowering des corps, plus rien ne parle d'oxc.

C'est la conséquence directe d'ADR-3 (« Abstract domains never see the Oxc AST, only the IR »), et c'est ce qui rend possible une arène par fichier. L'IR la plus riche est celle d'un composant :

Extrait 5.5 – `src/ir/component.rs`, l. 59–83 — `ComponentIR`

```

59 #[derive(Debug, Clone)]
60 pub struct ComponentIR {
61     /// Source file this component was lowered from. Used as part of the
62     /// `(file, name)` registry key so two components named `Page` in
63     /// different files don't collide.
64     pub file: PathBuf,
65     pub name: Symbol,
66     pub param: Var,
67     /// Names of props whose TypeScript type is a DOM interface
68     /// (`HTMLCanvasElement`, `SVGElement`, `Node`...). Mutating these is
69     /// imperative DOM manipulation, not a write into React-owned data –
70     /// the state-mutation rule exempts them.
71     pub dom_props: Arc<HashSet<Var>>,
72     pub render_cfg: CFG,

```

```

73     pub hooks: Vec<HookEntry>,
74     /// Provenance row per hook call in `hooks` (ADR-023 step 1):
75     /// `label` → (origin hook, source, direct|inlined)`. Grows during
76     /// analysis as custom hooks are expanded.
77     pub hook_provenance: Vec<crate::ir::hooks::HookProvenance>,
78     /// Module-level `const` bindings of the source file with syntactically
79     /// known kinds, keyed by name. Function-valued consts (arrow/function
80     /// expressions) are excluded: those are components/utilities/handlers
81     /// with their own machinery.
82     pub module_consts: Arc<HashMap<Var, ModuleConstInit>>,
83 }

```

Champ par champ, du point de vue du front-end :

- `file` est le chemin normalisé du fichier ; avec `name`, il forme la clé (`file`, `name`) des registres (ADR-13 §1), qui empêche deux Page de fichiers différents de se confondre ; le moteur l’interne ensuite en `ComponentId` (ADR-40).
- `name` est le nom de liaison trouvé par le détecteur (section 5.3), ou `DefaultExport` pour un export default anonyme.
- `param` est le premier nom de paramètre renvoyé par le lowering du corps, ou `"props"` si le composant n’a pas de paramètre ; un paramètre déstructuré reçoit le nom synthétique `__p0`.
- `dom_props` est calculé par `collect_dom_props` (section 5.7).
- `render_cfg`, `hooks` et `hook_provenance` viennent du lowering du corps et de l’extraction des hooks (chapitre 7) ; le front-end leur fournit la table de provenance des imports (section 5.6).
- `module_consts` est la table des constantes de module du fichier, partagée par un `Arc` entre tous ses composants.

HookIR (`src/ir/hook_ir.rs`, l. 9–24) a la même forme, sans `dom_props` ni `module_consts` et avec tous les paramètres ; FunctionIR (`src/ir/function_ir.rs`, l. 15–22) n’a même pas de table de hooks, parce qu’un utilitaire ne peut légalement en appeler aucun (« utilities cannot contain hook calls », `src/ir/function_ir.rs`, l. 1–6).

5.2.3 Un lowerer, pas à pas

Voici le lowerer des composants, que celui des hooks reproduit presque ligne pour ligne (`src/lowering/mod.rs`, l. 373–414) :

Extrait 5.6 – `src/lowering/mod.rs`, l. 437–491 — `lower_program_with_resolver`

```

437 pub fn lower_program_with_resolver(
438     program: &Program,
439     source: &str,
440     file: &Path,
441     files: &mut FileTable,
442     resolver: &dyn ImportResolver,
443 ) -> Vec<ComponentIR> {
444     // One context per file: it carries the shared allocation-site counter
445     // (`ExprIds`, #134) and the JSX callee origins (#7) into every body.
446     let ctx = LowerCtx::new(
447         SourceMap::new(source, files.intern(file)),
448         build_jsx_origins(program, file, resolver),
449     );
450     let origins = build_hook_origins(program, file, resolver);
451     let react_ns = build_react_ns(program);
452     let module_consts = Arc::new(collect_module_consts(program, &react_ns, file));

```



```

453     let local_hooks: HashSet<String> = detect_custom_hooks(program)
454         .iter()
455         .map(|c| c.name.clone())
456         .collect();
457     let imports = hook_extractor::ImportCtx {
458         origins: &origins,
459         react_ns: &react_ns,
460         local_hooks: &local_hooks,
461         current_file: Some(file),
462     };
463     detect_components(program)
464         .into_iter()
465         .map(|candidate| {
466             let (param_names, mut render_cfg) = candidate.build_cfg(&ctx);
467             let (mut hooks, hook_provenance, mut next_label) =
468                 extract_hooks(&mut render_cfg, &imports);
469             extract_handlers(&render_cfg, &mut hooks, &mut next_label);
470             extract_subscriptions(&mut hooks, &mut next_label);
471             let param = param_names
472                 .into_iter()
473                 .next()
474                 .unwrap_or_else(|| "props".to_string());
475             let dom_props = Arc::new(component_detector::collect_dom_props(
476                 candidate.params,
477                 program,
478             ));
479             ComponentIR {
480                 file: file.to_path_buf(),
481                 name: candidate.name,
482                 param,
483                 dom_props,
484                 render_cfg,
485                 hooks,
486                 hook_provenance,
487                 module_consts: module_consts.clone(),
488             }
489         })
490         .collect()
491 }

```

La fonction se lit en deux temps. D'abord, elle construit les tables *par fichier* : le contexte de lowering LowerCtx (table des lignes, compteur de sites d'allocation, origines des callees JSX, section 5.8), la provenance des imports de hooks (origins), les liaisons locales du module react (react_ns), les constantes de module, et l'ensemble local_hooks des hooks custom définis dans le fichier. Ensuite, pour chaque candidat que detect_components a retenu, en ordre source : abaisser le corps en CFG (build_cfg), en extraire les hooks, les handlers et les abonnements, choisir param, calculer dom_props, émettre l'IR.

Remarque

Les trois lowerers sont appelés *indépendamment* sur le même Program, et chacun reconstruit ses propres tables : detect_custom_hooks tourne deux fois par fichier, build_jsx_origins trois fois, et chaque déclaration d'import est résolue au moins cinq fois. Chacun crée aussi son propre LowerCtx, donc son propre compteur de sites (le commentaire « One context per file » doit se lire « un par fichier et par lowerer »); les collisions d'identifiants entre un composant et le hook qu'il inline sont neutralisées au moment du splice par les décalages de src/ir/remap.rs (chapitre 8). Au commit e67b10a, aucun de ces coûts redondants n'est mesuré : ni le driver ni les ADR ne rapportent de décomposition du temps d'un run par

phase, et le projet n'a pas de banc de performance dédié au front-end. L'affirmation « le moteur domine » est donc une hypothèse de conception, pas une mesure ; le chapitre 25 indique si la méthode de mesure du projet permet de la trancher.

5.3 Le marcheur commun et le candidat

5.3.1 Le problème : combien de façons d'écrire une fonction ?

JavaScript offre de nombreuses façons de lier un nom à une fonction au niveau supérieur d'un module : `function F()` , `const F = () => ...` , `const F = function ()` , les mêmes précédées d'`export`, `export default function F()` , `export default () => ...`, sans compter les formes enveloppées (`memo(...)`, `forwardRef(...)`), les classes, les déstructurations et les fonctions imbriquées. Les trois détecteurs (composants, hooks, utilitaires) doivent tous énumérer ces formes, et ne diffèrent que par le *critère* qu'ils appliquent à chaque fonction trouvée. Jusqu'au commit `b39ee0e` (« `shared detector walker, drop triplicated scaffolding` », recensé par ADR-20), cette énumération était recopiée trois fois ; elle vit désormais dans `src/lowering/detector.rs`.

5.3.2 L'interface du marcheur

Extrait 5.7 – `src/lowering/detector.rs`, l. 15–35 — `FnItem`, `Classify`, `DefaultHandler`

```

15 /// A top-level function-like binding the walker found: its name, parameters,
16 /// body, and (folded) return-type annotation. A detector's [`Classify`] reads
17 /// whichever of these it needs – the component detector uses the return type,
18 /// the hook detector only the name.
19 pub(crate) struct FnItem<'a> {
20     pub name: &'a str,
21     pub params: &'a FormalParameters<'a>,
22     pub body: &'a FunctionBody<'a>,
23     pub return_type: Option<&'a TTypeAnnotation<'a>>,
24     /// `true` for a concise arrow body (`x => expr`) – see
25     /// [`Candidate::expression`](crate::lowering::Candidate::expression).
26     pub expression: bool,
27 }
28
29 /// Predicate deciding whether a walked function is a candidate of this kind.
30 pub(crate) type Classify = fn(&FnItem) -> bool;
31
32 /// How a detector treats `export default`. Runs in source order alongside the
33 /// other statements. `None` (utility) ignores default exports entirely.
34 pub(crate) type DefaultHandler =
35     for<'a> fn(&'a ExportDefaultDeclaration<'a>, Classify, &mut Vec<Candidate<'a>>);

```

Un `FnItem` est la vue qu'a un détecteur d'une fonction trouvée : son nom de liaison, ses paramètres, son corps (tous empruntés à l'arène), son annotation de type de retour *repliée*, et le drapeau `expression`. Le repli mérite un mot : `return_type` vaut l'annotation de retour de la fonction si elle en a une, sinon celle du `const` qui la reçoit (`const Foo: React.FC = ...`). Les deux ne désignent pas la même chose (le type de ce que la fonction renvoie, le type de la fonction elle-même), mais toutes deux alimentent la même règle 3 du détecteur de composants (section 5.4).

Un détecteur est entièrement décrit par deux pointeurs de fonction : un prédicat `Classify` et, facultativement, un traitement de l'`export default`. Ni trait ni objet dynamique : c'est la forme la plus simple qui laisse les trois détecteurs diverger là où ils divergent vraiment.

5.3.3 La marche

Extrait 5.8 – src/lowering/detector.rs, l. 41–71 — detect_fns, le marcheur commun

```

41 pub(crate) fn detect_fns<'a>(<
42     program: &'a Program<'a>,
43     classify: Classify,
44     default: Option<DefaultHandler>,
45 ) -> Vec<Candidate<'a>> {
46     let mut out = Vec::new();
47     for stmt in &program.body {
48         match stmt {
49             Statement::FunctionDeclaration(func) => {
50                 consider_fn(func, None, None, classify, &mut out)
51             }
52             Statement::VariableDeclaration(decl) => {
53                 for vd in &decl.declarations {
54                     consider_var(vd, classify, &mut out);
55                 }
56             }
57             Statement::ExportNamedDeclaration(exp) => {
58                 if let Some(decl) = &exp.declaration {
59                     consider_decl(decl, classify, &mut out);
60                 }
61             }
62             Statement::ExportDefaultDeclaration(exp) => {
63                 if let Some(handler) = default {
64                     handler(exp, classify, &mut out);
65                 }
66             }
67             _ => {}
68         }
69     }
70     out
71 }

```

La marche est *linéaire* sur `program.body` : elle ne descend jamais dans un corps de fonction, une classe ou un bloc. Quatre formes d’instruction sont reconnues ; toute autre (`ClassDeclaration`, `ExpressionStatement`, déclarations TypeScript, imports) tombe dans le `_ => {}`. Une déclaration de variable est examinée déclarateur par déclarateur :

Extrait 5.9 – src/lowering/detector.rs, l. 85–106 — consider_var

```

85 fn consider_var<'a>(<
86     vd: &'a VariableDeclarator<'a>,
87     classify: Classify,
88     out: &mut Vec<Candidate<'a>>,
89 ) {
90     let BindingPattern::BindingIdentifier(id) = &vd.id else {
91         return;
92     };
93     let name = id.name.as_str();
94     // A `const Foo: React.FC = ...` annotation feeds the component return-type rule.
95     let type_ann = vd.type_annotation.as_deref();
96     let Some(init) = &vd.init else { return };
97     match init {
98         Expression::ArrowFunctionExpression(arrow) => {
99             consider_arrow(name, arrow, type_ann, classify, out);
100         }
101         Expression::FunctionExpression(func) => {
102             consider_fn(func, Some(name), type_ann, classify, out);

```

```

103     }
104     _ => {}
105 }
106 }

```

Trois filtres s’y succèdent : le motif doit être un identifiant nu (une déstructuration `const [A, B] = ...` est ignorée), l’initialiseur doit exister, et il doit être *directement* une flèche ou une expression de fonction. Le kind de la déclaration n’est pas testé : `var` et `let` sont acceptés comme `const`. `consider_fn` (`src/lowering/detector.rs`, l. 108–136) prend le nom de la fonction ou un nom imposé (celui du `const`, ou `DefaultExport`), replie l’annotation, et saute les fonctions sans corps, c’est-à-dire les déclarations ambiantes et les signatures de surcharge TypeScript (`let Some(body) = func.body.as_deref() else return; ;`). `consider_arrow` (`src/lowering/detector.rs`, l. 140–158) fait de même pour une flèche et recopie son drapeau `expression`. `push_if` applique le prédicat et, s’il accepte, construit le candidat (`src/lowering/detector.rs`, l. 160–169). La sortie est en ordre source ; son coût est linéaire en nombre d’instructions de niveau supérieur, plus celui des prédicats.

Exemple 5.2: Les formes que le marcheur voit, et les autres

Le fichier `/tmp/ch05/walker/A.tsx` rassemble des formes limites, toutes capitalisées et toutes renvoyant du JSX ou annotées :

Extrait 5.10 – Formes limites du marcheur (`/tmp/ch05/walker/A.tsx`)

```

1 import React, { useState, useEffect, memo } from "react";
2
3 const Paren = (() => <div />);
4 const Typed: React.FC = () => { return null as any; };
5 var VarComp = function () { return <p />; };
6 declare function Ambient(): JSX.Element;
7 function Over(a: number): JSX.Element;
8 function Over(a: any) { return <b />; }
9 const [Left, Right] = [() => <i />, () => <u />];
10 export const Memo = memo(function Inner() { return <i />; });
11 export function Outer() {
12     function Nested() { return <s />; }
13     return <Nested />;
14 }
15 export default () => <main />;

```

```

$ reactant check /tmp/ch05/walker --no-color --verbose --info --show-clean
[verbose] symbol graph: 5 nodes, topo order = [VarComp@/tmp/ch05/walker/A.tsx, Typed@/tmp/ch05/walker/A.tsx, Over@/
    ↳ tmp/ch05/walker/A.tsx, Outer@/tmp/ch05/walker/A.tsx, DefaultExport@/tmp/ch05/walker/A.tsx]
[verbose] 5 components analyzed
...
Outer (0 hooks) /tmp/ch05/walker/A.tsx
  info  analysis-limit component `Nested` was not found in the analysis registry. Pass its file on the command
    ↳ line to analyse it (FN possible)

✓ 1 file(s) no issues found.

```

Cinq composants sur onze liaisons. `VarComp` est retenu (le `var` n’est pas un obstacle) ; `Typed` l’est grâce à l’annotation du `const` repliée dans `return_type` ; `Over` n’apparaît qu’une fois (la signature sans corps est sautée) ; la flèche par défaut devient `DefaultExport`. Sont absents : `Ambient` (pas de corps), `Paren` (l’initialiseur est une `ParenthesizedExpression`, que `consider_var` ne pèle pas), `Left` et `Right` (déstructuration), `Memo` et `Inner` (enveloppe `memo`, #64), `Nested` (fonction imbriquée, #56). Seul le dernier est signalé, en INFO, parce qu’un composant détecté le rend.

5.3.4 Le candidat

Le marcheur rend des Candidate, la sortie commune des trois détecteurs :

Extrait 5.11 – src/lowering/mod.rs, l. 128–160 — Candidate et son unique point de dispatch

```

128 /// A top-level function picked out by one of the detectors, ready for lowering:
129 /// its binding name, parameter list, and body. Shared by the component, hook,
130 /// and utility detectors – they differ in how they *classify* a function, not
131 /// in this output shape (all three feed `Candidate::build_cfg`).
132 #[derive(Debug)]
133 pub struct Candidate<'a> {
134     pub name: String,
135     pub params: &'a FormalParameters<'a>,
136     pub body: &'a FunctionBody<'a>,
137     /// `true` for a concise arrow (`x => expr`), whose body oxc stores as a
138     /// single `ExpressionStatement` indistinguishable from `x => { expr; }`.
139     /// The difference is the whole return value, so it has to travel with the
140     /// body: lowered as a statement, `const useThing = () => useState(0)`
141     /// returns unit, and every consumer of `useThing` is left with an
142     /// unresolvable hook (#5).
143     pub expression: bool,
144 }
145
146 impl Candidate<'_> {
147     /// Lower this candidate's body to a CFG, honouring a concise arrow's
148     /// implicit return.
149     ///
150     /// The single place that dispatch happens, so a new consumer of
151     /// [Candidate] cannot forget it – which is how the flag came to be
152     /// dropped in the first place.
153     pub(crate) fn build_cfg(&self, ctx: &LowerCtx) -> (Vec<String>, crate::ir::cfg::CFG) {
154         if self.expression {
155             cfg_builder::build_expr_fn_body_cfg(self.params, self.body, ctx)
156         } else {
157             cfg_builder::build_fn_body_cfg(self.params, self.body, ctx)
158         }
159     }
160 }

```

name est possédé, parce qu'il part dans l'IR; params et body sont des emprunts dans l'arène; expression vaut toujours false pour une fonction (src/lowering/detector.rs, l. 130) et recopie le drapeau d'oxc pour une flèche (src/lowering/detector.rs, l. 153). Les alias ComponentCandidate, HookCandidate et UtilityCandidate ne sont que des pub type ... = Candidate<'a>. La méthode build_cfg est l'unique endroit où le corps d'un candidat est abaissé; nous verrons au section 5.5 pourquoi cette unicité est une décision, et non une commodité.

5.4 Trois classes de fonctions

Rappel React : ce que React impose, et ce qu'il n'impose pas

Pour React, un composant fonction est une fonction appelée avec un objet props et qui renvoie un nœud (JSX, null, une chaîne, un tableau, un portail...). React n'exige *aucun* nom particulier de la fonction; c'est JSX qui impose la casse, au *site d'usage* : <foo/> désigne un élément hôte (la chaîne "foo"), <Foo/> et <ns.Foo/> des références de variables. Le lowering applique la même règle (src/lowering/expr_lower.rs, l. 808). Les règles des hooks [Met26d] ajoutent deux conventions : un hook ne s'appelle qu'au niveau supérieur d'un composant

fonction ou d'un hook custom ; un hook custom est une fonction dont le nom commence par use suivi d'une majuscule. Le linter eslint-plugin-react-hooks [Met26a] s'appuie sur ces conventions, et le front-end de reactant-analyzer en fait des critères de détection.

5.4.1 Le hook custom : un critère purement nominal

Le plus simple des trois détecteurs ne regarde que le nom.

Extrait 5.12 – src/lowering/mod.rs, l. 46–61 — is_hook_name, la frontière hook / utilitaire

```

46 /// React's naming rule for a custom hook: `use` followed by an uppercase letter
47 /// or a digit (`useCounter`, `use2FA`). A lowercase 4th char (`useful`,
48 /// `userId`) is NOT a hook – such a function cannot legally call hooks (Rules of
49 /// Hooks), so it is a plain utility.
50 ///
51 /// Single source of truth for the hook/utility classification boundary: the
52 /// hook detector and the utility detector used to hard-code divergent rules
53 /// (`starts_with("use")` && len > 3` vs this one), so `useful` was classified as
54 /// BOTH a hook and a utility. Both now call this.
55 pub(crate) fn is_hook_name(name: &str) -> bool {
56     name.starts_with("use")
57     && name
58         .chars()
59         .nth(3)
60         .is_some_and(|c| c.is_ascii_uppercase() || c.is_ascii_digit())
61 }
```

Un nom de hook commence par use et son quatrième caractère est une majuscule ASCII ou un chiffre : useCounter, use2FA sont des hooks ; useful, userId, et use tout court n'en sont pas. Le commentaire raconte une dette soldée : les détecteurs de hooks et d'utilitaires codaient autrefois deux règles divergentes (starts_with("use") && len > 3 d'un côté), si bien que useful était classée à la fois hook et utilitaire. Il n'y a plus qu'un prédicat, partagé par les détecteurs, par la règle 4 des composants, par l'extracteur de hooks et jusque par le moteur (src/engine/render_deps.rs, l. 51) ; c'est le principe 3 de CLAUDE.md (« modulaire et général d'abord ») appliqué.

Le prédicat du détecteur de hooks se réduit à is_custom_hook(item.name), qui n'est lui-même qu'un appel à is_hook_name (hook_detector.rs, l. 15–19 et 46–48) : le corps n'importe pas, un hook peut renvoyer du JSX ou non. Un export default n'est retenu que s'il est une déclaration de fonction nommée (src/lowering/hook_detector.rs, l. 21–32), puisqu'une flèche anonyme n'a pas de nom à confronter à la convention. Point subtil, documenté par le commentaire de is_custom_hook (src/lowering/hook_detector.rs, l. 36–48) : une fonction locale nommée comme un hook de React (function useMemo(name, options)) est un hook custom. La portée lexicale de JavaScript fait qu'elle masque l'import ; le section 5.6 montre comment cette information est exploitée. (Le commentaire renvoie à ImportCtx::callee_is_react, nom obsolète : la méthode s'appelle aujourd'hui classify_callee.)

5.4.2 L'utilitaire : une double négation

Extrait 5.13 – src/lowering/utility_detector.rs, l. 17–47 — le détecteur d'utilitaires

```

17 /// Detect every top-level utility function in `program`.
18 ///
19 /// `export default` is not visited: a default-exported top-level function is
20 /// (by React convention) a component, never a utility, so there is nothing to
21 /// classify here.
22 pub fn detect_utilities<'a>(program: &'a Program<'a>) -> Vec<UtilityCandidate<'a>> {
```

```

23     detector::detect_fns(program, classify, None)
24 }
25
26 /// A function is a utility iff its name is neither a hook nor a component and
27 /// its body does not return JSX (a JSX-returning function is a component).
28 fn classify(item: &FnItem) -> bool {
29     is_utility(item.name) && !body_returns_jsx(&item.body.statements)
30 }
31
32 /// Returns `true` iff `name` is a utility (not a hook, not a component).
33 /// Components have an uppercase first letter; hooks start with `use`.
34 fn is_utility(name: &str) -> bool {
35     if name.is_empty() {
36         return false;
37     }
38     // Hooks (shared predicate – see `super::is_hook_name`).
39     if super::is_hook_name(name) {
40         return false;
41     }
42     // Components uppercase first letter
43     if name.chars().next().is_some_and(|c| c.is_uppercase()) {
44         return false;
45     }
46     true
47 }

```

Un utilitaire est une fonction de niveau supérieur dont le nom n'est ni un nom de hook ni capitalisé, et dont aucun chemin de retour n'est JSX ; l'export default n'est jamais visité (`None` à la place du traitement), au motif qu'une fonction exportée par défaut est, par convention React, un composant. Les utilitaires sont abaissés en `FunctionIR` (`src/lowering/utility_lowerer.rs`, l. 38–62) sans extraction de hooks, pour être inlinés par le moteur aux sites d'appel en position d'instruction (ADR-13 §5, chapitre 8).

5.4.3 Le composant : quatre règles

Le détecteur de composants est le seul dont le critère combine le nom, le corps et le type :

Extrait 5.14 – `src/lowering/component_detector.rs`, l. 47–73 — `is_component`

```

47 fn is_component(name: &str, stmts: &[Statement], return_type: Option<&TSTypeAnnotation>) -> bool {
48     // Rule 1: `use` prefix → hook, never a component
49     if name.starts_with("use") {
50         return false;
51     }
52     // React convention: component names must start with uppercase
53     if !name.chars().next().is_some_and(|c| c.is_uppercase()) {
54         return false;
55     }
56     // Rule 2: any return path yields JSX
57     if body_returns_jsx(stmts) {
58         return true;
59     }
60     // Rule 3: TypeScript component type annotation on the return type
61     if let Some(ann) = return_type
62         && ts_type_is_component(&ann.type_annotation)
63     {
64         return true;
65     }
66     // Rule 4: it calls a React hook (#122). The Rules of Hooks read backwards
67     // – nothing but a component or a custom hook may call one, and rule 1 has

```

```

68 // already excluded custom hooks by name. This is what finds a component
69 // that returns `null` on every path: an analytics mount, a portal host, a
70 // keyboard-shortcut registrar. Those are exactly where lifecycle bugs live,
71 // and before this rule they were not lowered at all.
72 body_calls_hook(stmts)
73 }

```

La figure 5.2 en donne l'arbre de décision. L'évaluation est court-circuitante : le test JSX, le plus fréquent, passe avant l'annotation, et la marche des appels de hooks, la plus coûteuse, en dernier.

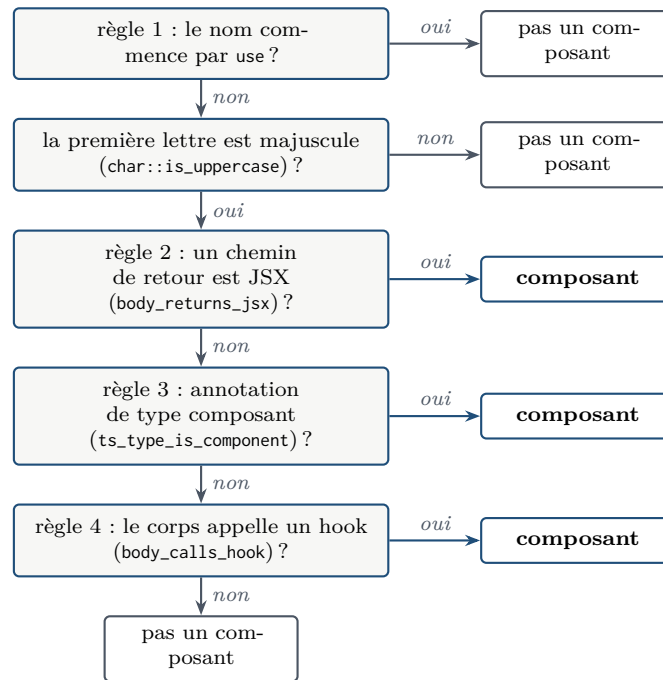


FIGURE 5.2 – Arbre de décision de `is_component` (`component_detector.rs`, l. 47–73). La règle 1 est subsumée par le test de majuscule ; la règle 4 est postérieure à ADR-3 (#122).

Règle 1 et majuscule. `name.starts_with("use")` exclut les hooks. Elle est logiquement *subsumée* par le test suivant (un nom qui commence par `use` commence par un `u` minuscule) : elle documente l'intention, héritée de la « Priority 0 » d'ADR-3. Le test de majuscule utilise `char::is_uppercase`, qui est Unicode (Élan est accepté), alors que `is_hook_name` teste l'ASCII ; l'asymétrie est sans conséquence connue.

Règle 2 : un chemin de retour est JSX. La marche est structurale sur les instructions :

Extrait 5.15 – `src/lowering/jsx_detect.rs`, l. 14–45 — `stmt_has_jsx_return`

```

14 fn stmt_has_jsx_return(stmt: &Statement) -> bool {
15     match stmt {
16         Statement::ReturnStatement(ret) => {
17             ret.argument.as_ref().is_some_and(|e| expr_contains_jsx(e))
18         }
19         // Expression-body arrows (`() => <div/>`) store the expression as an ExpressionStatement
20         Statement::ExpressionStatement(es) => expr_contains_jsx(&es.expression),
21         Statement::BlockStatement(block) => body_returns_jsx(&block.body),
22         Statement::IfStatement(if_) => {
23             stmt_has_jsx_return(&if_.consequent)

```



```

24         || if_
25         .alternate
26         .as_ref()
27         .is_some_and(|alt| stmt_has_jsx_return(alt))
28     }
29     Statement::WhileStatement(w) => stmt_has_jsx_return(&w.body),
30     Statement::ForStatement(f) => stmt_has_jsx_return(&f.body),
31     Statement::LabeledStatement(l) => stmt_has_jsx_return(&l.body),
32     Statement::TryStatement(tr) => {
33         body_returns_jsx(&tr.block.body)
34         || tr
35         .handler
36         .as_ref()
37         .is_some_and(|h| body_returns_jsx(&h.body.body))
38         || tr
39         .finalizer
40         .as_ref()
41         .is_some_and(|f| body_returns_jsx(&f.body))
42     }
43     _ => false,
44 }
45 }

```

Extrait 5.16 – src/lowering/jsx_detect.rs, l. 47–63 — expr_contains_jsx

```

47 fn expr_contains_jsx(expr: &Expression) -> bool {
48     match expr {
49         Expression::JSXElement(_) | Expression::JSXFragment(_) => true,
50         Expression::ConditionalExpression(c) => {
51             expr_contains_jsx(&c.consequent) || expr_contains_jsx(&c.alternate)
52         }
53         Expression::LogicalExpression(l) => {
54             expr_contains_jsx(&l.left) || expr_contains_jsx(&l.right)
55         }
56         Expression::ParenthesizedExpression(p) => expr_contains_jsx(&p.expression),
57         Expression::TSAsExpression(a) => expr_contains_jsx(&a.expression),
58         Expression::TSNonNullExpression(a) => expr_contains_jsx(&a.expression),
59         Expression::TSSatisfiesExpression(a) => expr_contains_jsx(&a.expression),
60         Expression::TSTypeAssertion(a) => expr_contains_jsx(&a.expression),
61         _ => false,
62     }
63 }

```

Il suffit d'un chemin (disjonction sur les branches d'un if, d'un try, d'une conditionnelle ou d'un opérateur logique) ; les enveloppes TypeScript et les parenthèses sont traversées. La marche ne descend pas dans les fonctions imbriquées (une `FunctionDeclaration` tombe dans `_ => false`). Le bras `ExpressionStatement` sert les flèches concises, dont `oxc` range l'expression dans une instruction ; il accepte au passage une instruction JSX isolée dans un corps bloc, ce qui est inoffensif. Retenons dès maintenant ce qui *manque* : `SwitchStatement`, `DoWhileStatement`, `for...in`, `for...of` parmi les instructions ; les appels (`items.map(i => <li/>)`, `createPortal(<div/>, el)`), les tableaux et les séquences parmi les expressions. Le section 5.9 mesure ce que ces trous coûtent.

Règle 3 : une annotation de type composant. Seules les références de type nommées sont examinées (`ts_type_is_component` et `ts_type_name_is_component`, `src/lowering/component_detector.rs`, l. 77–109) : `React.FC`, `React.FunctionComponent`, `React.ReactElement`, `React.ReactNode`, `JSX.Element`, et les noms nus `ReactElement`, `FC`, `FunctionComponent`. Une union (`JSX.Element | null`) n'est pas

une `TTypeReference` et n'est pas reconnue ; `ReactNode` nu n'est pas dans la liste. La règle est surtout utile pour les composants qui ne renvoient pas de JSX syntaxique (`return null as any`, retour délégué), et pour le repli de l'annotation du `const`.

Règle 4 : le corps appelle un hook. Elle fait l'objet du section 5.5.

L'export par défaut. Le traitement propre aux composants (`src/lowering/component_detector.rs`, l. 22–43) nomme les formes anonymes : `export default function App()` est enregistré sous `App` ; une fonction ou une flèche anonyme sous `DefaultExport`. `export default memo(Foo)`, `export default Foo` ; et `export default class` tombent dans le `_ =>` ; si `Foo` est déclaré dans le fichier, il est détecté sous son propre nom.

Décision : #65, wontfix : le nom générique des exports par défaut anonymes

« `export default () => <div/>` is mapped to "DefaultExport". Multi-file collisions are possible ... mitigated by (file, name) keying, but the user-visible name stays generic. The *identity* half is handled ; what remains is cosmetic naming. » Le projet ne cherche donc pas de meilleur nom : la clé (file, name) d'ADR-13 §1 suffit à l'identité, et seul l'affichage reste générique. Le nom `DefaultExport` ne peut être écrit par aucun JSX (`src/lowering/mod.rs`, l. 97–100) ; un tel composant n'est donc atteint que comme racine.

5.4.4 Les trois classes, formellement

Rassemblons les trois critères.

Définition 5.1: Composant, hook custom, utilitaire

Soit f une fonction liée au niveau supérieur d'un module sous une forme que reconnaît `detect_fns` (déclaration de fonction ; `const`, `let` ou `var` à motif identifiant, initialisé *directement* par une flèche ou une expression de fonction ; l'une des deux précédée d'`export` ; et, selon le détecteur, `export default`), munie d'un corps. On note n son nom de liaison.

- f est un *hook custom* si `is_hook_name(n)` : n commence par `use` suivi d'une majuscule ASCII ou d'un chiffre.
- f est un *composant* si n ne commence pas par `use`, si sa première lettre est majuscule, et si au moins l'une des conditions suivantes est vraie : (règle 2) un chemin de retour du corps est syntaxiquement JSX ; (règle 3) la fonction ou le `const` qui la reçoit porte une annotation de type composant ; (règle 4) le corps appelle directement un hook.
- f est un *utilitaire* si elle n'est pas un `export default`, si n n'est ni un nom de hook ni capitalisé, et si aucun chemin de retour de son corps n'est JSX.

Une fonction qui n'appartient à aucune des trois classes n'est pas abaissée.

Proposition 5.1: Partition non exhaustive

Les trois classes sont deux à deux disjointes, mais leur union ne couvre pas l'ensemble des fonctions de niveau supérieur, même parmi celles que rend React.

Démonstration. Un composant a une première lettre majuscule, un hook commence par un `u` minuscule, un utilitaire n'a ni l'un ni l'autre : les trois conditions sur le nom s'excluent. Pour la non-exhaustivité, trois témoins suffisent : `function helper() return <span/>` ; (minuscule et JSX : ni composant, ni utilitaire) ; `function useful() return <u/>` ; (use suivi d'une minuscule et JSX : ni hook, ni utilitaire, ni composant) ; `function List() return items.map(...)` ; (majuscule, mais aucune des règles 2 à 4). Le troisième est un composant React parfaitement valide. □

La figure 5.3 dessine cette partition sur deux axes : ce que dit le nom, et ce que disent le corps et le type.

	use[A-Z0-9]...	majuscule initiale	autre nom (y compris use+minuscule)
retour JSX détecté	hook custom (le corps n'importe pas)	composant (règle 2)	non abaissé helper, useful
pas de JSX, mais annotation ou hook		composant (règles 3, 4)	utilitaire
rien de tout cela		non abaissé List, Switchy, Portal	utilitaire (sauf export default)

FIGURE 5.3 – Les trois classes du front-end. Colonnes : le nom ; lignes : ce que les règles 2 à 4 trouvent dans le corps et le type. Les cases en tirets ne sont vues par aucun détecteur.

Exemple 5.3: Les classes du programme d'ouverture

Reprenons le liste 5.1. Counter est un composant par la règle 2 (il renvoie un `<button>`), avec `param = "props"` faute de paramètre. Beacon ne renvoie que `null` et ne porte pas d'annotation : il est composant par la règle 4, parce qu'il appelle `useState` ; son paramètre déstructuré devient `__p0`. `useCounter` est un hook custom. `formatCount` est un utilitaire (minuscule, pas de JSX). `helper`, minuscule et JSX, tombe dans la case en tirets de droite : aucun détecteur ne le retient, et le graphe de symboles de l'introduction ne le mentionne pas. L'avertissement `infinite-loop` de Beacon n'existerait pas sans la règle 4.

Piège

Le test unitaire `utility_that_returns_jsx_indirectly_is_component_not_utility` (`src/lowering/utility_detector.rs`, l. 109–112) porte un nom trompeur. Il vérifie seulement que `function widget() return <div/>` ; n'est pas un utilitaire ; or `widget`, minuscule, n'est pas non plus un composant. La fonction n'est abaissée par personne, contrairement à ce que suggère le nom du test.

5.5 Deux faux négatifs corrigés

Deux corrections récentes ont façonné le front-end. Elles portent sur des objets différents (la *forme* du corps pour l'une, la *classification* pour l'autre) mais elles ont la même nature : une fonction que l'utilisateur a écrite disparaissait de l'analyse, ou en ressortait vidée de son sens, sans aucun signal.

5.5.1 La flèche concise perdait sa valeur de retour (#5)

Soit `const useFlag = (init: boolean) => useState(init);`. `.oxc` range le corps de cette flèche comme un `FunctionBody` contenant *une* `ExpressionStatement` : exactement la même forme que `(init) => useState(init);`. Seul le drapeau `ArrowFunctionExpression::expression` distingue les deux, et la différence est toute la valeur de retour. Avant le commit `30a00c5` (« a hook in a terminator is still a hook (#4, #5) »), Candidate ne portait pas ce drapeau : le corps était abaissé comme une instruction, la fonction renvoyait `unit`, et chaque consommateur de `useFlag` se retrouvait avec un hook sans valeur. L'issue s'intitule « Concise-arrow bodies lose their return value, so the component or hook disappears ».

La correction tient en trois pièces. Le marcheur recopie le drapeau dans `FnItem` puis dans `Candidate` (liste 5.11); `Candidate::build_cfg` choisit entre deux constructeurs de CFG; le constructeur des corps concis scelle le bloc d'entrée par un `Return` de l'expression :

Extrait 5.17 – `src/lowering/cfg_builder.rs`, l. 985–1001 — le CFG d'une flèche concise

```

985 /// Like [build_fn_body_cfg] for concise arrow bodies (`x => expr`).
986 pub fn build_expr_fn_body_cfg(
987     params: &FormalParameters,
988     body: &FunctionBody,
989     ctx: &LowerCtx,
990 ) -> (Vec<String>, CFG) {
991     let mut builder = BlockBuilder::new(ctx);
992     builder.start_block(0);
993     let param_names = inject_param_preamble(params, &mut builder);
994     if let Some(Statement::ExpressionStatement(es)) = body.statements.first() {
995         let expr = lower_expr(&es.expression, &mut builder);
996         if !builder.is_terminated() {
997             builder.seal_with(Terminator::Return(expr));
998         }
999     }
1000     (param_names, builder.into_cfg(0))
1001 }

```

La fixture `tests/fixtures/concise_arrow/App.tsx` vérifie la *parité* entre les deux orthographes :

Extrait 5.18 – `tests/fixtures/concise_arrow/App.tsx`, l. 8–17 — une fabrique concise d'objets frais

```

8 // Fresh object every call.
9 const makeConfig = (id: string) => ({ id, retries: 3 });
10
11 export function UsesObject() {
12     const cfg = makeConfig("a");
13     useEffect(() => {
14         console.log(cfg);
15     }, [cfg]);
16     return <div />;
17 }

```

Extrait 5.19 – `tests/fixtures/concise_arrow/App.tsx`, l. 30–42 — le témoin à corps bloc

```

30 // The block-bodied control: same value, explicit return. Both spellings must
31 // reach the same verdict.
32 const makeConfigBlock = (id: string) => {
33     return { id, retries: 3 };
34 };
35
36 export function UsesObjectBlock() {
37     const cfg = makeConfigBlock("a");
38     useEffect(() => {
39         console.log(cfg);
40     }, [cfg]);
41     return <div />;
42 }

```

Le fichier contient encore `UsesFunction`, une fabrique concise de fonctions (`tests/fixtures/concise_arrow/App.tsx`, l. 19–28), et `UsesMemo`, dont le hook concis `useThing` renvoie un `useMemo` et doit rester silencieux (`tests/fixtures/concise_arrow/App.tsx`, l. 44–54).

```
$ reactant check tests/fixtures/concise_arrow/App.tsx --no-color --show-clean
UsesFunction (1 hooks) tests/fixtures/concise_arrow/App.tsx
  warn  always-unstable-deps [hook:0] (line 24:2) this effect depends on `h`, a new reference every render, so `
    ↳ Object.is` always differs and the effect re-runs on every render regardless of the other deps
    (1 trace step(s), rerun with --trace)
UsesMemo (2 hooks) tests/fixtures/concise_arrow/App.tsx ✓
UsesObject (1 hooks) tests/fixtures/concise_arrow/App.tsx
  warn  always-unstable-deps [hook:0] (line 13:2) this effect depends on `cfg`, a new reference every render, so `
    ↳ Object.is` always differs and the effect re-runs on every render regardless of the other deps
    (1 trace step(s), rerun with --trace)
UsesObjectBlock (1 hooks) tests/fixtures/concise_arrow/App.tsx
  warn  always-unstable-deps [hook:0] (line 38:2) this effect depends on `cfg`, a new reference every render, so `
    ↳ Object.is` always differs and the effect re-runs on every render regardless of the other deps
    (1 trace step(s), rerun with --trace)

! 3 warning(s) across 1 file(s).
```

UsesObject (fabrique concise) et UsesObjectBlock (fabrique à corps bloc) reçoivent le même verdict ; UsesMemo, dont le hook concis renvoie un useMemo, reste silencieux : la correction ne doit pas rendre instable toute valeur de corps concis. (Les fabriques makeConfig et makeHandler, minuscules et sans JSX, sont des utilitaires inlinés ; c'est leur valeur de retour qui porte l'objet frais.)

Décision : Principe 1 de CLAUDE.md : un seul point de dispatch

Le drapeau aurait pu être testé par chacun des trois lowerers. Il est au contraire porté par Candidate et consulté en un seul lieu, Candidate::build_cfg : « The single place that dispatch happens, so a new consumer of Candidate cannot forget it — which is how the flag came to be dropped in the first place » (src/lowering/mod.rs, l. 147–152). Corriger chaque consommateur aurait été un contournement ; corriger le type qu'ils partagent supprime la classe de bogues.

5.5.2 Le composant toujours null n'était pas détecté (#122)

Revenons à Beacon (liste 5.1). Il renvoie null sur tous ses chemins, ne porte aucune annotation : les règles 2 et 3 le manquent toutes deux. Or ce genre de composant n'est pas rare : un montage d'analytics, un hôte de portail, un enregistreur de raccourcis clavier. Et c'est précisément là que vivent les bogues de cycle de vie (effets sans cleanup, boucles d'écriture). L'issue #122, étiquetée soundness-bug et precision-fn, l'énonce sans détour : « a component that returns null on every path is not detected at all ».

La correction ajoute la règle 4, justifiée en une phrase par le module qui l'implémente :

Extrait 5.20 – src/lowering/hook_call_detect.rs, l. 1–12 — le principe de la règle 4

```
1  /// Does a function body call a React hook?
2  ///
3  /// The Rules of Hooks read backwards: nothing but a component or a custom hook
4  /// may call one. [crate::lowering::component_detector`] has already excluded
5  /// custom hooks by name before it asks, so for a capitalised function the
6  /// answer is componenthood — which is how a component that returns `null` on
7  /// every path gets detected at all (#122).
8  ///
9  /// The walk deliberately does not descend into nested function bodies. A
10 /// `useState` inside a callback is that callback's problem, not this function's
11 /// hook call, and crediting it would let an ordinary utility holding a
12 /// component literal read as a component.
```

« The Rules of Hooks read backwards » : puisque seuls un composant ou un hook custom peuvent appeler un hook, et que la règle 1 a déjà écarté les hooks custom par leur nom, une

fonction capitalisée qui appelle un hook *est* un composant. Le test de référence reprend Beacon :

Extrait 5.21 – src/lowering/component_detector.rs, l. 248–264 — le test de #122

```

248  /// #122: a component that returns `null` on every path yields no JSX and
249  /// carries no TS annotation, so rules 2 and 3 both miss it. It calls hooks,
250  /// which nothing but a component or a custom hook may do – and rule 1 has
251  /// already excluded custom hooks by name.
252  #[test]
253  fn always_null_component_is_detected_by_its_hook_calls() {
254      assert_eq!(
255          names(
256              "function Beacon({ id }) { \
257                  const [n, setN] = useState(0); \
258                  useEffect(() => { track(id, n); }, [id, n]); \
259                  return null; \
260              }"
261          ),
262          vec!["Beacon"]
263      );
264  }

```

La marche des appels suit le même schéma structurel que celle du JSX, avec davantage de formes, parce qu'un appel de hook peut se nicher partout où une expression est évaluée au rendu. Au niveau des instructions (src/lowering/hook_call_detect.rs, l. 22–58), elle visite les expressions, les return, les initialiseurs de variables, les blocs, les if (y compris leur *test* : un hook dans une condition n'est pas du React légal, mais c'est exactement ce que #4 signalait comme perdu, donc il compte), les while, les corps de for, les étiquettes, les switch et les try. Au niveau des expressions :

Extrait 5.22 – src/lowering/hook_call_detect.rs, l. 60–88 — expr_calls_hook

```

60  fn expr_calls_hook(expr: &Expression) -> bool {
61      match expr {
62          Expression::CallExpression(call) => {
63              callee_is_hook(&call.callee) || expr_calls_hook(&call.callee)
64          }
65          Expression::ConditionalExpression(c) => {
66              expr_calls_hook(&c.test)
67              || expr_calls_hook(&c.consequent)
68              || expr_calls_hook(&c.alternate)
69          }
70          Expression::LogicalExpression(l) => expr_calls_hook(&l.left) || expr_calls_hook(&l.right),
71          Expression::SequenceExpression(s) => s.expressions.iter().any(expr_calls_hook),
72          Expression::AwaitExpression(a) => expr_calls_hook(&a.argument),
73          Expression::UnaryExpression(u) => expr_calls_hook(&u.argument),
74          Expression::ParenthesizedExpression(p) => expr_calls_hook(&p.expression),
75          Expression::TSAsExpression(a) => expr_calls_hook(&a.expression),
76          Expression::TSNonNullExpression(a) => expr_calls_hook(&a.expression),
77          Expression::TSSatisfiesExpression(a) => expr_calls_hook(&a.expression),
78          Expression::TSTypeAssertion(a) => expr_calls_hook(&a.expression),
79          // `useRouter().push` / `useThing().value` – the hook call is the object.
80          Expression::StaticMemberExpression(m) => expr_calls_hook(&m.object),
81          Expression::ComputedMemberExpression(m) => expr_calls_hook(&m.object),
82          // JSX children can hold one (`<div>{useLabel()}</div>`), and #4's
83          // repro is exactly that shape.
84          Expression::JSXElement(el) => el.children.iter().any(jsx_child_calls_hook),
85          Expression::JSXFragment(fr) => fr.children.iter().any(jsx_child_calls_hook),
86          _ => false,
87      }
88  }

```

Extrait 5.23 – src/lowering/hook_call_detect.rs, l. 102–109 — callee_is_hook

```

102 fn callee_is_hook(callee: &Expression) -> bool {
103     match callee {
104         Expression::Identifier(id) => is_hook_name(id.name.as_str()),
105         // `React.useState(...)` , and any namespace import of the same shape.
106         Expression::StaticMemberExpression(m) => is_hook_name(m.property.name.as_str()),
107         _ => false,
108     }
109 }

```

Trois propriétés sont à retenir.

- La marche *ne descend pas* dans les fonctions imbriquées. Un `useState` dans un callback est le problème du callback ; le compter ferait passer pour composant un utilitaire capitalisé qui contient un littéral de composant (test `rule_four_does_not_widen_past_real_hook_calls`, src/lowering/component_detector.rs, l. 266–278).
- Le prédicat est purement *nominal* : `is_hook_name` sur un identifiant nu ou sur la propriété d'un accès membre, sans résolution d'import. Tout `obj.useX()` compte, y compris sur un objet qui n'a rien de React. Détecter trop largement ne coûte, au pire, que des faux positifs.
- Seul le *callee* d'un appel est inspecté, pas ses arguments. `useRouter().push` est vu (l'objet d'un accès membre est visité), `track(useId())` ne l'est pas. Nous y reviendrons au section 5.9.

Décision : #122 et ADR-3 : un écart documentaire

ADR-3 fixe la première définition du composant en trois priorités : « Priority 0: name starts with use → custom hook, never a component. Priority 1: at least one return path produces a JSXElement → component. Priority 2: annotated `React.FC` / `React.ReactElement` / `JSX.Element` → component. » Le code ajoute la contrainte de majuscule et la règle 4 (commit 31f08ef), que l'ADR ne mentionne pas. La reproduction de #122 mesurait `components_analyzed: 0` pour un composant `NullComp` qui renvoie `null`, contre un composant et quatre diagnostics, dont une `ERROR stale-closure`, pour le même corps terminé par `return <i />`. Au commit étudié, `NullComp` (/tmp/ch05/i122/A.tsx) reçoit les mêmes quatre diagnostics que son jumeau `JSX` :

```

NullComp (2 hooks) /tmp/ch05/i122/A.tsx
warn  missing-cleanup [hook:1] (line 5:20) this effect calls `setInterval` but returns no cleanup. ...
warn  missing-deps [hook:1] var:ms (line 5:2) `ms` is used in this effect but not in its deps array, ...
warn  missing-deps [hook:1] var:n (line 5:2) `n` is used in this effect but not in its deps array, ...
error  stale-closure [hook:1] var:n (line 5:20) the `setInterval` callback registered by this mount-only
      ↳ effect reads `n` and writes it back. ...

```

(Messages tronqués, lignes « trace step » éliminées.)

Piège

Deux leçons générales se lisent dans ces corrections. D'abord, au front-end, un défaut de *précision* apparent (une forme non reconnue) est en réalité un défaut de *soundness* : #122 porte les deux étiquettes. Ensuite, la règle 4 compte les hooks appelés conditionnellement, alors qu'ils sont illégaux : le détecteur n'a pas à juger le code, seulement à le trouver ; l'illégalité est l'affaire de la règle `conditional-hook`.

5.6 La provenance des hooks

5.6.1 Le problème : un nom de hook ne dit pas d'où il vient

Détecter les hooks custom *définis* dans un fichier ne suffit pas : il faut aussi savoir, à chaque *appel* de hook, ce qu'on appelle. Le moteur modélise précisément les hooks de React qu'il connaît (`useState`, `useEffect`, `useMemo`...), inline les hooks custom du projet, applique des résumés aux hooks de paquets connus, et traite tout le reste comme inconnu (ADR-5 : un hook non reconnu vaut $\top$, avec une INFO). Se tromper de catégorie a deux faces. Classer React un appel qui ne l'est pas, c'est lui appliquer une sémantique qu'il n'a pas ; classer inconnu un vrai hook de React, c'est perdre tout ce que le moteur sait de lui. Voici le cas qui a fixé la règle.

Exemple 5.4: Un `useMemo` local masque celui de React

Extrait 5.24 – `/tmp/ch05/shadow/Shadow.tsx`

```
1 import { useState, useEffect } from "react";
2
3 function useMemo(name: string) {
4   const [v, setV] = useState(0);
5   useEffect(() => { setV(v + 1); });
6   return v;
7 }
8
9 export function C() {
10   const v = useMemo("k");
11   return <div>{v}</div>;
12 }
```

```
$ reactant check /tmp/ch05/shadow --no-color --show-clean
C (1 hooks) /tmp/ch05/shadow/Shadow.tsx
  warn  infinite-loop [hook:1] (line 5:2) this effect keeps pushing state `v` to new values on every run.
        ↳ Potential infinite render loop
        (2 trace step(s), rerun with --trace)
```

Le `useMemo` de ce fichier n'est pas celui de React : la définition locale le masque. Il est classé hook custom, inliné dans `C`, et la boucle de son effet est trouvée (le témoin pointe la ligne 5, dans le hook). L'avoir classé `useMemo` de React aurait fait disparaître l'effet et la boucle. C'est le scénario du test `locally_defined_use_hook_shadows_react` (`tests/hook_classification.rs`).

5.6.2 HookOrigin : ce que la déclaration d'import prouve

Le commit 27538cd (« hook identity by provenance — ADR-023 step 1 ») a remplacé une classification par le nom, avec une devinette « fail-open » pour les cas douteux, par une table de provenance « fail-closed » :

Extrait 5.25 – `src/lowering/import_resolution.rs`, l. 35–55 — HookOrigin

```
35 /// Provenance of one hook-relevant imported binding, decided fail-closed:
36 /// each variant records what was *proven* about the origin, and the raw
37 /// specifier is retained on every non-React variant so a package-scoped
38 /// lookup ([`crate::registry::SummaryRegistry`]) still matches when a
39 /// self-aliasing tsconfig path resolves the package to a local file.
40 #[derive(Debug, Clone, PartialEq, Eq)]
41 pub enum HookOrigin {
42     /// The literal specifier was "react" — decided *before* the resolver is
43     /// consulted, so a project aliasing react` to a file keeps React's hooks
44     /// classified as React's (ADR-023 step 1: the mass-FN hazard).
```



```

45   React { imported: String },
46   /// The specifier resolved to a local file. `specifier` is the raw import
47   /// text (``zustand`` under a self-alias, ``./hooks/useData``).
48   File {
49       file: PathBuf,
50       specifier: String,
51       imported: String,
52   },
53   /// The specifier did not resolve – an npm package, or a missing file.
54   Package { specifier: String, imported: String },
55 }

```

Chaque variante enregistre ce qui a été *prouvé* de l'origine. `React` : le spécificateur littéral était "react". `File` : il a été résolu vers un fichier local. `Package` : il n'a pas été résolu (paquet npm ou fichier manquant). Le spécificateur brut est conservé dans les deux dernières variantes, parce que la recherche des résumés de paquets (SummaryRegistry, ADR-5 et ADR-26 §5) se fait par spécificateur, y compris quand un alias tsconfig fait pointer un nom de paquet vers un fichier local. La table est construite par :

Extrait 5.26 – `src/lowering/import_resolution.rs`, l. 74–130 — `build_hook_origins`

```

74 pub fn build_hook_origins(
75     program: &Program,
76     current_file: &Path,
77     resolver: &dyn ImportResolver,
78 ) -> HashMap<String, HookOrigin> {
79     let mut map = HashMap::new();
80     for stmt in &program.body {
81         let Statement::ImportDeclaration(decl) = stmt else {
82             continue;
83         };
84         let Some(specifiers) = &decl.specifiers else {
85             continue;
86         };
87         let source = decl.source.value.as_str();
88         // The literal specifier decides React-ness BEFORE the resolver runs:
89         // a self-aliasing tsconfig `paths` entry mapping "react" to a file
90         // must not demote React's hooks to opaque Custom rows.
91         let is_react = source == "react";
92         // Resolved lazily: only consulted when a hook-relevant specifier
93         // exists on a non-react declaration.
94         let mut resolved: Option<Option<PathBuf>> = None;
95         for spec in specifiers {
96             let (local, imported) = match spec {
97                 ImportDeclarationSpecifier::ImportSpecifier(s) => {
98                     (s.local.name.as_str(), s.imported.name().to_string())
99                 }
100                 ImportDeclarationSpecifier::ImportDefaultSpecifier(s) => {
101                     (s.local.name.as_str(), s.local.name.to_string())
102                 }
103                 ImportDeclarationSpecifier::ImportNamespaceSpecifier(_) => continue,
104             };
105             if !(super::is_hook_name(local) || super::is_hook_name(&imported)) {
106                 continue;
107             }
108             let origin = if is_react {
109                 HookOrigin::React { imported }
110             } else {
111                 let file = resolved
112                     .get_or_insert_with(|| resolver.resolve(current_file, source))
113                     .clone();

```

```

114         match file {
115             Some(file) => HookOrigin::File {
116                 file,
117                 specifier: source.to_string(),
118                 imported,
119             },
120             None => HookOrigin::Package {
121                 specifier: source.to_string(),
122                 imported,
123             },
124         }
125     };
126     map.insert(local.to_string(), origin);
127 }
128 }
129 map
130 }

```

Quatre points décident de tout.

1. La clé est le nom *local*, mais l'entrée est retenue si le nom local *ou* le nom importé est un nom de hook : `import useData as fetchData` donne une entrée `fetchData`, et le hook garde son nom d'origine `useData`.
2. La « React-ité » se décide sur la chaîne littérale `"react"`, *avant* tout appel au résolveur (`src/lowering/import_resolution.rs`, l. 88–91). Un projet dont le `tsconfig` fait pointer `react` vers un fichier (cela existe) garde ainsi les hooks de React classés comme tels. Dans l'ordre inverse, ils deviendraient tous des Custom opaques : ce que le code appelle « the mass-FN hazard ».
3. La résolution est paresseuse et faite au plus une fois par déclaration.
4. Les imports d'espace de noms (`import * as R`) sont sautés ; ils passent par `react_ns` (liaisons locales du module `react` lui-même, `src/lowering/mod.rs`, l. 162–190).

Exemple 5.5: Quatre provenances dans un fichier

Extrait 5.27 – `/tmp/ra-ex/e5/Page.tsx` (fichiers `hooks/useData.ts`, `ui/Widget.tsx`, `types.ts` présents)

```

1 import { useMemo as useM, useState } from "react";
2 import { useData as fetchData } from "../hooks/useData";
3 import { useQuery } from "@tanstack/react-query";
4 import type { T } from "../types";
5 import { Widget as Panel } from "../ui/Widget";
6 export * from "../ui/Widget";
7
8 export function Page() {
9     const [n, setN] = useState(0);
10    const { d } = fetchData("x");
11    const q = useQuery({ queryKey: ["k"] });
12    const v = useM(() => ({ n }), [n]);
13    return <Panel onChange={setN} />;
14 }

```

Par lecture de liste 5.26, la table contient quatre entrées : `useM` $\mapsto$ `React` `imported: "useMemo"` (l'alias ne trompe pas), `useState` $\mapsto$ `React`, `fetchData` $\mapsto$ `File` (fichier `hooks/useData.ts`, nom importé `useData`), `useQuery` $\mapsto$ `Package` `specifier: "@tanstack/react-query"`, `Panel` et `T` n'y sont pas (ni l'un ni l'autre n'a une forme de hook). Le dossier de notes a vérifié cette table par une sonde. La ligne de commande confirme ce qui en résulte : `Page` a quatre hooks,

et le composant enfant est bien identifié (nous y reviendrons au section 5.8) :

```
$ reactant check /tmp/ra-ex/e5 --no-color --show-clean
Page (4 hooks) /tmp/ra-ex/e5/Page.tsx ✓
Widget (0 hooks) /tmp/ra-ex/e5/ui/Widget.tsx
  error cross-setter-in-render var:onChange (line 3:2) prop `onChange` (a state setter of parent `Page`)
    ↳ called during render of `Widget`, which triggers a parent re-render on every render
      (1 trace step(s), rerun with --trace)

! 1 error(s) across 4 file(s).
```

5.6.3 ImportCtx et classify_callee

Les tables du front-end sont remises à l'extracteur de hooks sous la forme d'un ImportCtx :

Extrait 5.28 – src/lowering/hook_extractor.rs, l. 531–543 — ImportCtx

```
531 pub struct ImportCtx<'a> {
532     /// Hook-relevant imported bindings → proven origin.
533     pub origins: &'a HashMap<String, HookOrigin>,
534     /// Local names bound to the `react` module itself
535     /// (`import React from "react"`, `import * as R from "react"`).
536     pub react_ns: &'a HashSet<String>,
537     /// `use*` functions defined in this file (JS scoping: a local
538     /// definition shadows any same-named import or global).
539     pub local_hooks: &'a HashSet<String>,
540     /// File being lowered – provenance of locally-defined hooks, so the
541     /// `(file, name)` registry lookup stays precise. `None` for hand-built IR.
542     pub current_file: Option<&'a Path>,
543 }
```

origins vient de build_hook_origins, react_ns de build_react_ns, local_hooks des noms que detect_custom_hooks a trouvés dans le fichier (liste 5.6). L'extracteur, qui appartient au lowering des corps (chapitre 7), classe chaque callee par classify_callee :

Extrait 5.29 – src/lowering/hook_extractor.rs, l. 562–582 — classify_callee (début) : un hook local masque tout

```
562     /// Resolve a callee to a hook identity, or `None` for a plain call.
563     ///
564     /// - bare `name(...)` : a local `use*` definition shadows everything
565     ///   (JS scoping); then the [HookOrigin] map decides by provenance –
566     ///   including a non-`use` alias of a hook import and an aliased React
567     ///   hook; an unimported hook-shaped name stays React's by convention
568     ///   (test sources, globals).
569     /// - `ns.useX(...)` : React's iff `ns` is bound to the `react` module or
570     ///   is the conventional unimported global `React`; any other receiver
571     ///   is a custom hook with unknown provenance.
572     fn classify_callee(&self, fn_: &Expr) -> Option<ResolvedHookCall> {
573         match fn_ {
574             Expr::Var(name) => {
575                 if self.local_hooks.contains(name) {
576                     return Some(ResolvedHookCall {
577                         origin_name: name.clone(),
578                         is_react: false,
579                         specifier: None,
580                         resolved_file: self.current_file.map(Path::to_path_buf),
581                     });
582                 }
583             }
584         }
```

La suite (`src/lowering/hook_extractor.rs`, l. 583–611) traduit chaque variante de `HookOrigin` en `ResolvedHookCall` : `React` donne `is_react`: `true` et le spécificateur `"react"`, `File` donne `resolved_file`: `Some(file)`, `Package` garde le spécificateur sans fichier. Viennent ensuite la présomption conventionnelle pour un nom nu non importé (`src/lowering/hook_extractor.rs`, l. 612–620) et le cas d'un accès membre `Expr::FieldAccess` (`src/lowering/hook_extractor.rs`, l. 622–634).

L'ordre de priorité est celui de la portée lexicale de JavaScript, et la figure 5.4 le résume : (1) un hook défini dans le fichier masque tout import et tout global de même nom ; (2) sinon, la table `HookOrigin` décide par provenance ; (3) sinon, un nom de hook non importé est présumé `React` (sources de test, globaux) ; (4) pour un appel `ns.useX(...)`, c'est `React` si et seulement si `ns` est lié au module `react` ou est le global conventionnel `React`, et un hook custom de provenance inconnue sinon. Un `Custom` est ensuite résolu par le moteur, par inlining (`HookRegistry`) ou par résumé (`SummaryRegistry`) ; à défaut, il donne l'INFO « hook ... was not found in the registry ».

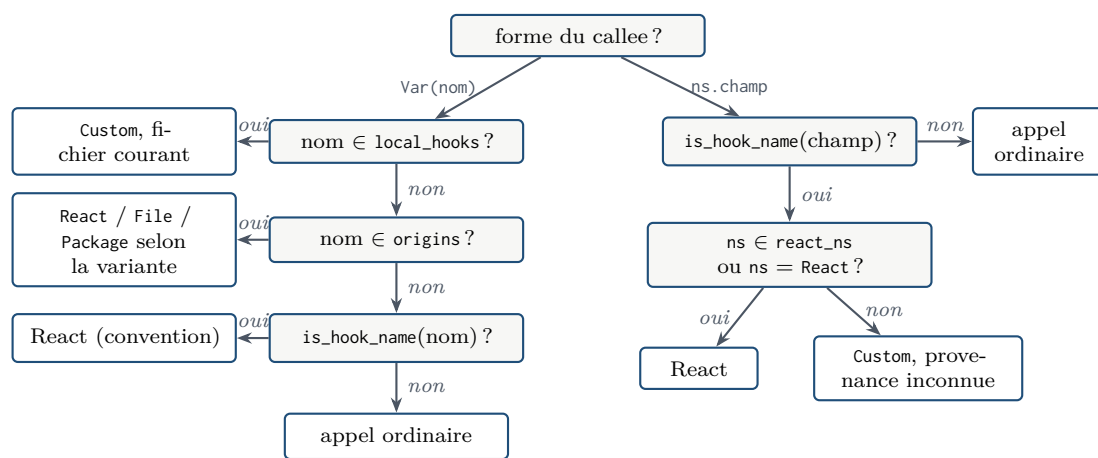


FIGURE 5.4 – Arbre de décision de `classify_callee` (`hook_extractor.rs`, l. 562–637). Toute autre forme de callee est un appel ordinaire.

Remarque

Le `Custom` d'un import de fichier porte `resolved_file` mais un `import_source` vide : `import_source` ne garde que les spécificateurs non relatifs, c'est-à-dire les noms de paquets (`src/lowering/hook_extractor.rs`, l. 494–498). Un hook local reçoit `resolved_file` égal au fichier courant, ce qui garde précise la recherche (`file`, `name`) dans le registre.

Piège

La présomption (3), « nom de hook non importé $\Rightarrow$ `React` », est une convention, pas une preuve. Elle est sûre parce qu'elle est la *dernière* : une définition locale ou un import non-`React` priment toujours. Autre subtilité : le masquage ne vaut qu'au niveau supérieur. Une liaison `const useMemo = ... à l'intérieur` d'un composant n'entre pas dans `local_hooks`, et un appel à ce `useMemo`-là serait classé `React`. Enfin, l'API `use(...)` de `React 19` n'est pas un nom de hook au sens de `is_hook_name` (pas de quatrième caractère) : le front-end ne la classe jamais comme hook, et `classify_callee` ne peut donc jamais produire de `ResolvedHookCall` dont le nom d'origine est `"use"` — ni par `local_hooks` (qui ne contient que des noms passant `is_hook_name`), ni par la table `HookOrigin` (même garde à l'insertion), ni par la convention (même garde). Le moteur, lui, anticipe pourtant ce cas : `reads_unnamed_context` teste `name == "use"` sur un `HookEntry::Custom` (`src/engine/render_deps.rs`, l. 662), une branche qu'aucun appel à `use(...)` ne peut atteindre par ce chemin au commit `e67b10a`. Le point d'évaluation

des appels ordinaires, lui, compense : eval sur un `Expr::Call` traite tout callee nommé "use" comme `is_hook_name` l'aurait fait (`src/engine/render_deps.rs`, l. 897), qu'il s'agisse ou non d'un `HookEntry`. Le filet de sécurité existe donc, mais un cran plus bas que ce que le commentaire de `reads_unnamed_context` laisse penser.

5.7 Les faits de module

Certaines informations ne concernent aucune fonction en particulier, mais le module entier : les constantes de niveau supérieur, les contextes qu'il crée, les directives qu'il déclare, les fichiers qu'il importe. Le front-end les extrait avec la même prudence : *n'enregistrer que ce qui est syntaxiquement certain*.

5.7.1 Les constantes de module

Exemple 5.6: Une constante de module n'est pas un littéral frais

Extrait 5.30 – /tmp/ch05/consts/Consts.tsx

```

1 import { useEffect } from "react";
2
3 const OPTIONS = { retries: 3 } as const;
4 let mutable = { retries: 3 };
5
6 export function Stable() {
7   useEffect(() => { console.log(OPTIONS); }, [OPTIONS]);
8   return <div />;
9 }
10
11 export function Fresh() {
12   const options = { retries: 3 };
13   useEffect(() => { console.log(options); }, [options]);
14   return <div />;
15 }
16
17 export function ViaLet() {
18   useEffect(() => { console.log(mutable); }, [mutable]);
19   return <div />;
20 }
```

```

$ reactant check /tmp/ch05/consts --no-color --show-clean
Fresh (1 hooks) /tmp/ch05/consts/Consts.tsx
  warn  always-unstable-deps [hook:0] (line 13:2) this effect depends on `options`, a new reference every
  ↳ render, so `Object.is` always differs and the effect re-runs on every render regardless of the other deps
    (1 trace step(s), rerun with --trace)
Stable (1 hooks) /tmp/ch05/consts/Consts.tsx ✓
ViaLet (1 hooks) /tmp/ch05/consts/Consts.tsx ✓

! 1 warning(s) across 1 file(s).
```

Le même littéral objet est instable dans le corps de `Fresh` (recréé à chaque rendu) et stable au niveau du module (évalué une fois, au chargement). `ViaLet` n'est pas signalé non plus, mais pour une autre raison : `mutable` n'est pas une constante collectée, sa valeur est inconnue (T), et une règle `WARNING` qui affirme « toujours frais » ne peut pas s'appuyer sur un inconnu.

Ce que le front-end sait d'un initialiseur de constante est décrit par :

Extrait 5.31 – src/ir/component.rs, l. 12–41 — ModuleConstInit

```

12 /// What is known about a module-level `const` initializer.
13 ///
14 /// Only initializers whose JS *kind* is syntactically certain are collected:
15 /// the product value domain expresses "constant across renders" as either an
16 /// exact primitive or a `Stable` reference slot – an opaque value of unknown
17 /// kind (`const X = f()`) has no sound encoding (a wide primitive slot reads
18 /// as "changes per render") and is left out, falling back to  $\top$ .
19 #[derive(Debug, Clone)]
20 pub enum ModuleConstInit {
21     /// Primitive literal: the exact value.
22     Prim(Prim),
23     /// Object/array/new/JSX literal: a reference, allocated once per module
24     /// lifetime  $\rightarrow$  identity Stable across renders.
25     Ref,
26     /// `createContext(...)` , proven to be React's by its import specifier. A
27     /// reference like [Ref`](ModuleConstInit::Ref) – the exception to "opaque
28     /// calls stay  $\top$ " is earned by knowing the callee, hence the kind.
29     ///
30     /// The variant exists because the *role* is what a provider rule needs:
31     /// `<X.Provider>` is a context provider only if `X` is a context, and
32     /// nothing else in the IR can say so. Two-valued on purpose – absence
33     /// means "not proven", never "not a context".
34     ///
35     /// It carries the [ContextId`] of the cell, not just the role (#109): two
36     /// files that import the same context bind it under whatever local name
37     /// they like, and pairing a consumer with a provider needs to know they
38     /// mean the same cell. The resolver already had the origin in hand when it
39     /// marked an imported name; it used to drop it.
40     Context(ContextId),
41 }

```

`collect_module_consts` (src/lowering/mod.rs, l. 243–341) parcourt les instructions de niveau supérieur, ne garde que les déclarations `const` (nues ou exportées) à motif identifiant, pèle les enveloppes TypeScript et les parenthèses (`peel_ts`, src/lowering/mod.rs, l. 248–259), puis classe l'initialiseur : un littéral primitif (booléen, null, nombre, chaîne) donne `Prim` ; un objet, un tableau, un `new`, une expression régulière ou un JSX donne `Ref` ; un appel de `createContext` prouvé React donne `Context`. Tout le reste est ignoré, et c'est voulu : « the value domain has no sound encoding for "unknown kind, constant across renders" — they stay $\top$ » (src/lowering/mod.rs, l. 240–242, #34). Les initialiseurs fonctionnels sont exclus aussi, car ils relèvent des détecteurs. Notons que `const N = -1` n'est pas collecté (`-1` est une `UnaryExpression`) et que `let` et `var` ne le sont jamais (réaffectables). Le moteur amorce son environnement initial avec ces valeurs, `Ref` et `Context` devenant des références `Stable` (src/engine/fixpoint.rs, l. 171–188, chapitre 10).

5.7.2 Les contextes et leur identité

La variante `Context` existe parce qu'une règle de provider a besoin du *rôle* : `<X.Provider>` ne fournit un contexte que si `X` en est un, et rien d'autre dans l'IR ne peut le dire. `createContext` n'est reconnu que s'il est atteint par une liaison React : un nom importé de "react" (éventuellement renommé) ou `ns.createContext` avec `ns` lié au module `react` ou égal à `React` (src/lowering/mod.rs, l. 277–293). Un `createContext` importé d'un autre paquet n'est pas un contexte (test `other_calls_are_still_skipped`).

La variante porte un `ContextId` (src/ir/component.rs, l. 43–57) : le fichier qui appelle `createContext` et le nom qu'il lie. Deux fichiers qui importent le même contexte sous des alias différents désignent ainsi la même cellule (#109). Pour cela, `scan_context_names` projette la table sur

les contextes de *chaque* fichier, y compris ceux sans composant (un `contexts/ctx.ts` qui n'exporte que le contexte est la forme courante), puis `resolve_imported_contexts` (`src/resolver/mod.rs`, l. 389–431) croise chaque import résolu avec la table du fichier d'origine. La résolution ne va qu'à *un* niveau : un contexte réexporté par un fichier tiers n'est pas suivi (#49), ce qui peut manquer un appariement consommateur–fournisseur, jamais en créer un faux.

5.7.3 ModuleFacts : directives et arêtes d'import

Extrait 5.32 – `src/ir/module.rs`, l. 21–36 — `ModuleFacts`

```
21 /// What one lowered file declares about itself.
22 #[derive(Debug, Clone, Default, PartialEq, Eq)]
23 pub struct ModuleFacts {
24     /// Directive prologue strings, in source order, unquoted: `use client`,
25     /// `use strict`, ... Only the prologue – a string expression statement
26     /// after the first non-directive statement is not a directive.
27     pub directives: Vec<String>,
28     /// Files this module imports, deduped, in first-seen order. Only edges
29     /// the `ImportResolver` mapped to a real file: npm packages and
30     /// unresolvable aliases leave no edge.
31     ///
32     /// Covers `import`, side-effect `import "./x"`, and the re-export forms
33     /// (`export ... from`, `export * from`), because a barrel re-export carries
34     /// a directive boundary exactly like a plain import does.
35     pub imports: Vec<PathBuf>,
36 }
```

`collect_module_facts` (`src/lowering/module_facts.rs`, l. 17–65) remplit les deux champs. Les directives viennent de `Program::directives`, qu'oxc restreint au *prologue* : un `'use client'` placé après une instruction réelle n'est pas une directive. Les `imports` sont les fichiers vers lesquels le résolveur a pu mapper un spécificateur, dédoublonnés dans l'ordre de première apparition, pour les import de valeur, les imports à effet de bord (`import "./x"`) et les réexportations (`export ... from`, `export * from`), parce qu'un barrel réexportateur tire le module exactement comme un import. `import type` et `export type ... from` ne sont *pas* des arêtes : ils sont effacés avant l'exécution et ne transportent aucun environnement.

La table `ModuleTable` (`src/ir/module.rs`, l. 44–52) associe ces faits à chaque fichier abaissé ; elle est délibérément *non interprétée* (`src/ir/module.rs`, l. 11–16). Une table vide répond « inconnu » partout, et chaque appelant doit lire « inconnu » comme absence de preuve, jamais comme preuve du contraire. Le seul mécanisme partagé est un parcours :

Extrait 5.33 – `src/ir/module.rs`, l. 100–126 — `reachable_from`, parcours en largeur arrêté par une frontière

```
100 pub fn reachable_from<'p>(<
101     &self,
102     seeds: impl IntoIterator<Item = &'p Path>,
103     boundary: Option<&str>,
104 ) -> HashSet<PathBuf> {
105     let blocked = |p: &Path| {
106         boundary.is_some_and(|d| self.files.get(p).is_some_and(|f| f.has_directive(d)))
107     };
108     let mut seen: HashSet<PathBuf> = HashSet::new();
109     let mut queue: VecDeque<PathBuf> = VecDeque::new();
110     for s in seeds {
111         if !blocked(s) && seen.insert(s.to_path_buf()) {
112             queue.push_back(s.to_path_buf());
113         }
114     }
115 }
```

```

114     }
115     while let Some(path) = queue.pop_front() {
116         let Some(facts) = self.files.get(&path) else {
117             continue;
118         };
119         for dep in &facts.imports {
120             if !blocked(dep) && seen.insert(dep.clone()) {
121                 queue.push_back(dep.clone());
122             }
123         }
124     }
125     seen
126 }

```

Le parcours part des points de départ (seeds), suit les arêtes d'import, et s'arrête *sur* un module qui déclare la directive frontière, en l'excluant. Il termine sur les cycles d'import grâce à `seen` et coûte $O(V + E)$. C'est toute la règle des Server Components en une fonction : une directive, écrite une fois en tête d'un module, gouverne tout ce qui est importé en dessous.

Exemple 5.7: Le graphe serveur d'un projet Next.js

La fixture `tests/fixtures/next_project` (alias `@/*` vers `./src/*`) est dessinée en figure 5.5. `src/app/page.tsx` n'a pas de directive et appelle `useState`; `src/components/counter.tsx` commence par `"use client"`; `src/components/sidebar.tsx` n'a pas de directive et n'est importé que par le layout serveur.

```

$ reactant check tests/fixtures/next_project --no-color --rule server-component-hook
HomePage (2 hooks) tests/fixtures/next_project/src/app/page.tsx
  warn server-component-hook [hook:0] (line 7:8) `useState` is called in a Server Component. this file is an
    ↳ App Router `page` and no `use client` directive covers it, so React renders it on the server, where
    ↳ hooks do not exist; add `use client` at the top of the file, or move the stateful part into a child
    ↳ component that declares it
Sidebar (2 hooks) tests/fixtures/next_project/src/components/sidebar.tsx
  warn server-component-hook [hook:0] (line 8:8) `usePathname`, `useState`, `useMemo` are called in a Server
    ↳ Component. this module is imported into the App Router's server graph and no `use client` directive
    ↳ covers it, so React renders it on the server, where hooks do not exist; add `use client` at the top of
    ↳ the file, or move the stateful part into a child component that declares it
1 clean component(s) hidden, rerun with --show-clean

! 2 warning(s) across 7 file(s).

```

Le front-end a fourni le nom `HomePage` (par export default fonction `HomePage`), les directives de chaque fichier et les arêtes résolues par alias; `server_modules` (`src/project/nextjs.rs`, l. 48–55) appelle `reachable_from` depuis les entrées serveur de l'App Router (`layout`, `page...`), et la règle `server-component-hook` (chapitre 21) font le reste. `Counter`, derrière sa frontière, n'est pas un Server Component : il n'est pas signalé par cette règle (sans le filtre `-rule`, il reçoit un infinite-loop sans rapport avec le front-end).

Décision : ADR-26 §1–2 : des faits de module dans l'IR

Les directives et les arêtes sont des faits de *fichier* et vivent hors de `ComponentIR` : « a directive governs every symbol in the module at once, and an import edge has no component to hang off ». La table est non interprétée; les arêtes de type sont exclues; `build_resolved_imports` perd son pré-filtre « relatifs seulement » (un alias résolu ne peut qu'ajouter des arêtes). L'alternative structurante refusée était d'*ignorer* les Server Components : « Server-ness is not a property of a file, it is a property of *how the file is reached* », et décider sur des arêtes incomplètes serait un faux négatif. La règle qui en découle est `WARNING`, pas `ERROR`, parce que ses faits viennent du front-end syntaxique et non du domaine abstrait.

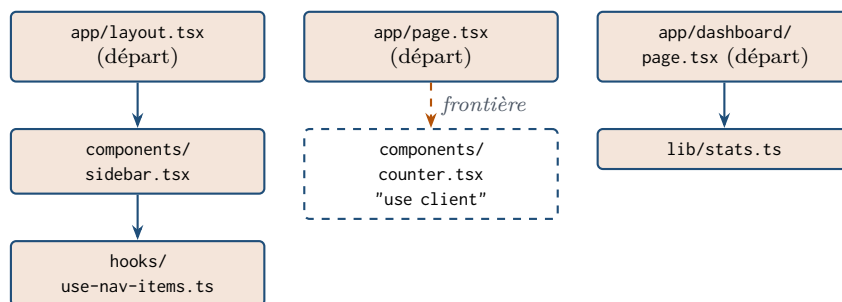


FIGURE 5.5 – Le graphe d'import de tests/fixtures/next_project. En orange, l'ensemble `reachable_from(départs, Some("use client"))` : les modules serveur. Le parcours s'arrête sur `counter.tsx`, qui déclare la frontière, et l'exclut.

Remarque

Contrairement à `collect_module_facts`, `build_resolved_imports` (`src/lowering/import_resolution.rs`, l. 140–177) ne filtre pas `import type` : un nom de type entre dans les imports résolus. C'est sans effet, un type ne correspondant à aucun composant, contexte ni utilitaire. À l'inverse, `import type T` (modificateur par spécificateur) garde `import_kind == Value` au niveau de la déclaration et *reste* une arête : un fichier qui n'exporte que des types peut ainsi déclencher le blind spot `unread-imports`. C'est une sur-approximation, jamais un faux négatif.

5.7.4 Les props typées DOM

Dernier fait extrait par le front-end, local à un composant : la fonction `collect_dom_props` (`src/lowering/component_detector.rs`, l. 113–139) lit l'annotation du *premier* paramètre (littéral de type, ou type ou interface du même fichier) et retient les membres dont le type est une interface DOM (`HTML...Element`, `SVG...Element`, `Element`, `Node`, `EventTarget`, `Document`; `src/lowering/component_detector.rs`, l. 196–208). Muter ces props est de la manipulation impérative du DOM, et la règle `state-mutation` les exempte. Les unions (`HTMLCanvasElement | null`) et les interface `... extends` ne sont pas suivies ; le commentaire l'assume, puisqu'une exemption manquée ne coûte qu'un `WARNING`.

Exemple 5.8: L'exemption DOM et la déstructuration

Extrait 5.34 – `/tmp/ra-ex/e3b/Dom.tsx`

```
1 import { useEffect } from "react";
2 type Props = { canvas: HTMLCanvasElement };
3 export function ChartA(props: Props) {
4   useEffect(() => { props.canvas.width = 3; }, [props.canvas]);
5   return <div />;
6 }
7 export function ChartB({ canvas }: Props) {
8   useEffect(() => { canvas.width = 3; }, [canvas]);
9   return <div />;
10 }
```

ChartA (1 hooks) /tmp/ra-ex/e3b/Dom.tsx ✓

ChartB (1 hooks) /tmp/ra-ex/e3b/Dom.tsx

warn state-mutation var:canvas (line 8:20) `canvas` roots in this component's props, so mutating it writes
 ↳ into an object owned by the parent; copy it before changing
 (1 trace step(s), rerun with --trace)

collect_dom_props rend "canvas" dans les deux cas. Mais la règle n'exempte que le motif props.canvas sur le paramètre du composant (src/rules/impls/state_mutation.rs, l. 121–127); un paramètre déstructuré s'appelle __p0, et canvas y est lié par le préambule de paramètres. C'est un faux positif WARNING, sur la forme la plus courante. Le dossier de notes le signale comme non répertorié.

5.8 L'identité des callees JSX

5.8.1 Le problème : qui est <Widget/> ?

Quand un composant écrit <Panel onChange=setN/>, le moteur doit trouver le corps de Panel pour l'analyser comme enfant, avec les props que le parent lui passe. Le nom seul ne suffit pas : Panel peut être un alias d'import, et un projet réel contient souvent vingt composants Form. Avant #7, le moteur résolvait un callee JSX par get_by_name, qui rendait la première clé (file, name) *dans l'ordre de tri* : un leurre a/Widget.tsx pouvait prendre la place du ./b/Widget importé, et l'ERROR qui dépendait du vrai corps disparaissait.

Exemple 5.9: Un alias, un leurre, et une référence ambiguë

Deux fichiers définissent Widget : a/Widget.tsx (inoffensif) et b/Widget.tsx, qui appelle sa prop onChange pendant le rendu. App.tsx importe import Widget as Panel from ".b/Widget" et rend <Panel onChange=setN />; Guess.tsx rend <Widget onChange=setN /> sans rien importer.

```
$ reactant check /tmp/ch05/ident --no-color --show-clean --info
App (1 hooks) /tmp/ch05/ident/App.tsx ✓
  verified ...
Guess (1 hooks) /tmp/ch05/ident/Guess.tsx
  info analysis-limit several analysed files define a component called `Widget` and this reference does not
    ↳ resolve to one of them, so the child is treated as unknown. Import it explicitly from its file (FN possible)
    ↳ )
  suspended analysis-limit 4 passing check(s) withheld: the analysis was truncated in this component, so they
    ↳ are not guaranteed
Widget@/tmp/ch05/ident/b/Widget.tsx (0 hooks) /tmp/ch05/ident/b/Widget.tsx
  error cross-setter-in-render var:onChange (line 2:2) prop `onChange` (a state setter of parent `App`)
    ↳ called during render of `Widget@/tmp/ch05/ident/b/Widget.tsx`, which triggers a parent re-render on every
    ↳ render
    (1 trace step(s), rerun with --trace)
```

(Les quatre lignes verified de App sont éliminées.) L'alias Panel est résolu vers le Widget de b/, dont l'ERROR est attribuée au bon parent. La référence de Guess, qu'aucun import ne tranche, n'est pas devinée : l'enfant devient inconnu, et une INFO le dit. Le nom affiché Widget@<fichier> n'est qu'un rendu, qui désambiguïse deux composants homonymes (ADR-40 §2).

5.8.2 CompOrigin et JsxOrigins

Un callee JSX porte désormais « ce que son propre fichier a prouvé » (ADR-40 §3) : un CompOrigin (src/ir/expr.rs, l. 160–172), c'est-à-dire le fichier qui définit le composant et le nom sous lequel ce fichier le connaît. Les deux moitiés comptent : le fichier sépare les homonymes, le nom résout l'alias (<Panel/> ici, Widget là-bas). La table JsxOrigins (src/lowering/import_resolution.rs, l. 207–224), un Arc<HashMap<Symbol, Arc<CompOrigin>> partagé par tous les corps du fichier, est construite une fois par fichier :

Extrait 5.35 – src/lowering/import_resolution.rs, l. 237–262 — build_jsx_origins

```
237 pub fn build_jsx_origins(
238     program: &Program,
```

```

239     current_file: &Path,
240     resolver: &dyn ImportResolver,
241 ) -> JsxOrigins {
242     let mut map: HashMap<Symbol, Arc<CompOrigin>> = HashMap::new();
243     for name in super::top_level_binding_names(program) {
244         map.insert(
245             name.clone(),
246             Arc::new(CompOrigin {
247                 file: current_file.to_path_buf(),
248                 name,
249             }),
250         );
251     }
252     for (local, origin) in build_resolved_imports(program, current_file, resolver) {
253         map.insert(
254             local,
255             Arc::new(CompOrigin {
256                 file: origin.file,
257                 name: origin.imported,
258             }),
259         );
260     }
261     JsxOrigins(Arc::new(map))
262 }

```

On y pose d'abord *tous* les noms liés au niveau supérieur (`top_level_binding_names`, `src/lowering/mod.rs`, l. 63–126 : fonctions, classes, variables, y compris sous `export`), délibérément pas « tous les composants » : un nom qui ne se révèle pas composant ne correspond simplement à rien dans le registre. On pose ensuite les imports résolus, avec leur nom d'origine (`ResolvedImport`, `src/lowering/import_resolution.rs`, l. 18–33). Un nom absent de la table est « non résolu », pas absent du programme : imports d'espace de noms, barrels et paquets npm y tombent, et le moteur se rabat alors sur le nom.

La table voyage dans le `LowerCtx` (`src/lowering/cfg_builder.rs`, l. 45–74), qui regroupe la table des lignes, le compteur de sites d'allocation et les origines JSX : un corps imbriqué (un `FnLit`) hérite des trois sans passe dédiée. Le tampon est posé au `lowering` de chaque élément capitalisé :

Extrait 5.36 – `src/lowering/expr_lower.rs`, l. 832–842 — le callee JSX reçoit son origine

```

832     // Who `<Name/>` refers to is settled by this file's imports and its own
833     // declarations, and nowhere else – resolving it later, from the bare
834     // name, is how a same-named component in an unrelated file came to be
835     // inlined in its place (#7).
836     let origin = builder.ctx.jsx.get(&name).cloned();
837     Expr::CompApp {
838         name,
839         props: Box::new(props),
840         span,
841         origin,
842     }

```

et consommé en un seul endroit du moteur :

Extrait 5.37 – `src/engine/component_registry.rs`, l. 114–127 — `resolve_child`, trois réponses

```

114     pub fn resolve_child(&self, name: &Symbol, origin: Option<&CompOrigin>) -> ChildLookup {
115         if let Some(o) = origin {
116             let key = (o.file.clone(), o.name.clone());
117             if self.entries.contains(&key) {

```

```

118         return ChildLookup::Resolved(key);
119     }
120 }
121 let mut matches = self.entries.keys().filter(|(_, n)| n == name);
122 match (matches.next(), matches.next()) {
123     (Some(key), None) => ChildLookup::Resolved(key.clone()),
124     (Some(_), Some(_)) => ChildLookup::Ambiguous,
125     _ => ChildLookup::Unknown,
126 }
127 }

```

Origine prouvée d'abord ; sinon le nom, s'il n'est défini que dans un fichier ; sinon *Ambiguous*, qui rend l'enfant inanalysable et fait porter au parent une *analysis-limit*. Jamais de « premier par ordre de tri ».

Décision : ADR-40 §3–5 : l'identité d'un callee se prouve dans son fichier

Le callee JSX porte l'origine que son fichier a établie ; la résolution vit en un seul lieu et refuse de deviner (« The old first-match was not non-deterministic, it was *wrong* ») ; tout chemin est normalisé une fois, en amont (*normalize* dans *lower_files_with*), pour que clés de registre et réponses du résolveur s'accordent. Alternative refusée : analyser *tous* les candidats d'un nom ambigu. « Sound and strictly more precise, and dropped on measurement » : 1 347 références ambiguës sur quatorze dépôts du corpus, <Button/> seul en comptant 1 453 quand le corpus est analysé d'un bloc, et une analyse complète de chaque candidat à chaque site reproduirait le blocage quadratique de #86. Le changement d'identité a été vérifié neutre : digest identique sur 35 541 fichiers.

5.9 Les limites du front-end sont des questions de soundness

5.9.1 Absent n'est pas $\top$

Partout ailleurs dans le pipeline, ce que l'analyse ne sait pas est sur-approximé : une valeur opaque vaut $\top$, un hook inconnu vaut $\top$ avec une *INFO*. Le front-end est le seul endroit où une décision *binaire et syntaxique* conditionne l'existence même de l'objet analysé. Une fonction que les trois détecteurs refusent n'est pas approximée : elle est *absente*.

Invariant 5.2: Asymétrie du front-end

Détecter trop largement (une fonction capitalisée qui renvoie du JSX sans jamais être rendue, un `obj.useX()` qui n'est pas un hook) coûte au pire des faux positifs, tolérés. Détecter trop étroitement retire du code à l'analyse : c'est un risque de faux négatif, donc un défaut de *soundness* au sens de définition 3.8, pas de précision.

C'est exactement la requalification faite par #122. Mesurons ce que la définition actuelle laisse passer.

Exemple 5.10: Huit composants, un seul vu

Extrait 5.38 – /tmp/ra-ex/e6/Limits.tsx

```

1 import React, { useState, memo } from "react";
2 import { createPortal } from "react-dom";
3
4 export function List({ items }: { items: string[] }) {
5     return items.map((i) => <li key={i}>{i}</li>);
6 }

```

```

7 export function Switchy({ k }: { k: number }) {
8   switch (k) {
9     case 1: return <a />;
10    default: return <b />;
11  }
12 }
13 export function Portal({ el }: { el: Element }) {
14   return createPortal(<div />, el);
15 }
16 export function Logger() {
17   console.log(useState(0));
18   return null;
19 }
20 export const Memo = memo(function Inner() { return <i />; });
21 export class Klass extends React.Component { render() { return <p />; } }
22 export function Dyn({ c }: { c: boolean }) {
23   const C = c ? List : Switchy;
24   return <C items={[]} k={1} />;
25 }
26 export function useful() { return <u />; }

```

```

$ reactant check /tmp/ra-ex/e6 --no-color --verbose --show-clean --info
[verbose] symbol graph: 1 nodes, topo order = [Dyn@/tmp/ra-ex/e6/Limits.tsx]
...
Dyn (0 hooks) /tmp/ra-ex/e6/Limits.tsx
info analysis-limit component `C` was not found in the analysis registry. Pass its file on the command line
  ↳ to analyse it (FN possible)

✓ 1 file(s) no issues found.

```

Sept définitions sur huit ne sont vues par aucun détecteur : `List` (retour par un appel `.map`), `Switchy` (`switch`), `Portal` (`createPortal(...)`), `Logger` (hook en argument d'appel), `Memo` (#64), `Klass` (composant de classe), `useful` (hors des trois classes). La ligne finale affirme « no issues found » alors que six composants React valides n'ont pas été analysés.

Le tableau 5.2 recense les formes non détectées au commit étudié. Les premières sont documentées par des issues ; les autres ont été observées par le dossier de notes et ne correspondent, à notre connaissance, à aucune issue.

5.9.2 Quand l'utilisateur est-il averti ?

Une absence n'est signalée que si un composant *détecté* rend le composant manquant : l'INFO `analysis-limit` « component ... was not found in the analysis registry ... (FN possible) » apparaît alors, visible avec `-info`. Si personne ne le rend, rien ne le signale. Le cas suivant montre les deux faces sur un même défaut.

Exemple 5.11: Un faux négatif de bout en bout

Extrait 5.39 – /tmp/ra-ex/e7/App.tsx

```

1 import { useState } from "react";
2
3 function ViaIf({ k, onChange }: { k: number; onChange: (n: number) => void }) {
4   onChange(1);
5   if (k === 1) return <a />;
6   return <b />;
7 }
8 function ViaSwitch({ k, onChange }: { k: number; onChange: (n: number) => void }) {

```

Forme	Cause dans le code
const C = cond ? A : B; <C/> (#63)	le CompApp C n'a pas d'origine et ne résout vers rien
memo(...), forwardRef(...) (#64)	consider_var n'accepte qu'une flèche ou une fonction nue
fonction imbriquée (#56)	la marche est au niveau supérieur seulement
utilitaire export default (#55)	detect_utilities passe None
node_modules (#51)	jamais abaissé, par conception
retour dans un switch, do...while, for...of	pas de bras dans stmt_has_jsx_return
return items.map(...), createPortal(...), [<a/>,]	expr_contains_jsx ne regarde ni appels ni tableaux
hook seulement en argument (track(useId())), sous ?., dans un gabarit, dans un for...of	expr_calls_hook n'inspecte que le callee; pas de bras ChainExpression, gabarit, ForOf
composant de classe	detect_fns ignore ClassDeclaration
annotation en union (JSX.Element null)	seules les TTypeReference sont examinées
initialiseur parenthésé (() => <div/>)	consider_var ne pèle pas les parenthèses
composant n'appelant que use(...) de React 19	is_hook_name("use") est faux

TABLE 5.2 – Formes non détectées au commit e67b10a. Au-dessus du trait : issues ouvertes ou décisions. En dessous : observées, non répertoriées à notre connaissance.

```

9   onChange(1);
10  switch (k) {
11    case 1: return <a />;
12    default: return <b />;
13  }
14 }
15 export function App() {
16   const [n, setN] = useState(0);
17   return (
18     <div>
19       <ViaIf k={n} onChange={setN} />
20       <ViaSwitch k={n} onChange={setN} />
21     </div>
22   );
23 }

```

```

$ reactant check /tmp/ra-ex/e7 --no-color
ViaIf (0 hooks) /tmp/ra-ex/e7/App.tsx
  error cross-setter-in-render var:onChange (line 4:2) prop `onChange` (a state setter of parent `App`)
    ↳ called during render of `ViaIf`, which triggers a parent re-render on every render
      (1 trace step(s), rerun with --trace)
1 clean component(s) hidden, rerun with --show-clean

! 1 error(s) across 1 file(s).

```

Le même défaut certain (un setter du parent appelé pendant le rendu de l'enfant) est trouvé dans `ViaIf` et manqué dans `ViaSwitch`; la seule différence est la forme du contrôle qui mène au `return`. Avec `-info`, App porte « component 'ViaSwitch' was not found in the analysis registry. Pass its file on the command line to analyse it (FN possible) », un conseil trompeur puisque le fichier a été passé.

Un dernier défaut vient du résolveur par défaut, que toutes les tables du front-end consultent.

`DefaultImportResolver::resolve` essaie `base.with_extension(ext)` pour chaque extension source (`src/resolver/mod.rs`, l. 659–665), et `Path::with_extension` *remplace* la dernière extension : `"/Button.styles"` devient `Button.ts`, `Button.tsx`... Avec `Button.tsx` et `Button.styles.ts` présents, l'import est résolu vers le mauvais fichier :

```
$ reactant check /tmp/ch05/dots/App.tsx --no-color --show-clean --info
App (1 hooks) /tmp/ch05/dots/App.tsx
info analysis-limit [hook:0] (line 4:8) hook `useTheme` was not found in the registry. Pass its source file or
  ↳ add a HookSummary to analyse it (FN possible)
suspended analysis-limit 1 passing check(s) withheld: the analysis was truncated in this component, so they are not
  ↳ guaranteed

! 1 file(s), no findings, but parts of this run were not analyzed, so this is not a clean bill.
not analyzed:
  • 1 imported file(s) resolved outside the analysed set and were never read. Pass them on the command line to analyse
    ↳ them (/tmp/ch05/dots/Button.tsx)
```

Le blind spot nommé `Button.tsx`, alors que `useTheme` vit dans `Button.styles.ts`. Quand le répertoire entier est analysé, la boucle de `useTheme` est bien trouvée (le moteur retombe sur la résolution par nom du hook) ; mais la même arête fautive peut aussi retirer silencieusement un composant entier du graphe serveur de `server-component-hook`, sans blind spot cette fois, parce que le fichier vers lequel elle pointe à tort est bien dans l'ensemble analysé. Sur un projet App Router où `app/page.tsx` importe `"/util.data"` (résolu vers un `util.tsx` sans rapport, présent à côté, plutôt que vers `util.data.ts`), le composant `Deep` que `util.data.ts` importe à son tour — il appelle `useState` sans `"use client"` — disparaît de la preuve :

```
$ reactant check /tmp/ch05/dots-graph --no-color --rule server-component-hook --show-clean
Deep (1 hooks) /tmp/ch05/dots-graph/app/deep.tsx ✓

✓ 4 file(s) no issues found.
```

En ne changeant que le nom du fichier importé (`"/util-data"` sans point, aucun `util.tsx` pour s'y substituer), le même arbre est correctement clos et `Deep` reçoit le `WARNING server-component-hook` attendu. Ce n'est donc pas seulement un blind spot mal nommé : c'est un faux négatif silencieux, par le même mécanisme.

5.9.3 Les frontières tracées

Décision : #63 et #51, wontfix

#63 (« Out of scope — dynamic components ») : « `const C = cond ? A : B; <C /> → no CompApp is generated, so it is not analyzed. Reopen with a design for resolving a component reference through a join.` » Au commit étudié, un `CompApp C` est bien généré, sans origine, et ne résout vers rien : l'effet est celui que l'issue décrit, par un autre mécanisme. #51 (« By design — `node_modules` utilities/hooks/components are never lowered ») : « the summary registry is the supported extension point for third-party behaviour ». Les alternatives, un join de références de composants et l'abaissement des sources de dépendances, restent ouvertes « avec un design » pour la première et hors périmètre pour la seconde ; dans les deux cas l'enfant non résolu est signalé par une `INFO analysis-limit`, et le comportement des paquets passe par les résumés.

Deux issues ouvertes sont rédigées comme des décisions : #64 (`React.memo` et `forwardRef`, « the one most likely to be worth lifting ») et #55 (imports par défaut résolus par nom local, utilitaires `export default` non détectés). Les formes du bas du tableau 5.2, en revanche, ne relèvent d'aucune décision : ce sont des trous de prédicats structurels, et chacun ne peut, une fois comblé, qu'ajouter des composants, donc des faux positifs possibles et aucun faux négatif (exercice 5.2).

Résumé du chapitre

- oxc parse chaque fichier dans sa propre arène ; seul panicked fait sauter un fichier, une erreur récupérée est signalée et le fichier est abaissé (`ParseError::analyzed`). `source_type_for` active JSX pour `.js/.jsx` ; `preserve_parens` matérialise les parenthèses.
- Le front-end rend, par fichier, des IR possédées (`ComponentIR`, `HookIR`, `FunctionIR`) et des faits de module (`ModuleFacts`, contextes, imports résolus) ; aucun domaine ne voit l'AST (invariant 5.1).
- Un marcheur unique, `detect_fns`, énumère les fonctions de niveau supérieur ; chaque détecteur n'apporte qu'un prédicat `Classify` et un traitement de l'export `default`. `Candidate::build_cfg` est l'unique point de dispatch corps bloc / corps concis (#5).
- Hook custom : `is_hook_name`. Utilitaire : ni hook ni majuscule, pas de retour JSX. Composant : majuscule et l'une des règles 2 (retour JSX), 3 (annotation), 4 (appel de hook, #122). Les classes sont disjointes, pas exhaustives (définition 5.1, proposition 5.1).
- La provenance d'un appel de hook est décidée fail-closed : hook local, puis `HookOrigin` ("react" testé avant le résolveur), puis convention, puis espace de noms (figure 5.4).
- Les faits de module n'enregistrent que le certain : le type `ModuleConstInit` (`Prim`, `Ref`, `Context(ContextId)`), directives du prologue, arêtes d'import de valeur ; `ModuleTable::reachable_from` porte toute la logique des frontières RSC (ADR-26).
- Un callee JSX porte l'origine que son fichier a prouvée (`CompOrigin`) ; `resolve_child` répond origine, nom unique ou `Ambiguous`, jamais une devinette (ADR-40).
- Au front-end, absent n'est pas `T` : une forme non détectée est un risque de faux négatif, souvent silencieux (invariant 5.2, tableau 5.2).

Fichiers rencontrés : dans `src/lowering/`, `mod.rs`, `detector.rs`, `component_detector.rs`, `hook_detector.rs`, `utility_detector.rs`, `jsx_detect.rs`, `hook_call_detect.rs`, `module_facts.rs`, `import_resolution.rs`, `utility_lowerer.rs` ; leurs voisins `src/ir/component.rs`, `src/ir/module.rs`, `src/ir/source_range.rs` et `src/resolver/mod.rs`. Le chapitre 6 décrit l'IR que ces fonctions remplissent, et le chapitre 7 le lowering des corps que `Candidate::build_cfg` déclenche ; l'usage inter-fichiers des faits produits ici est l'objet du chapitre 16.

Exercices

Exercice 5.1: Classer à la main

Pour chacune des huit définitions de l'exemple 5.10, dire quel détecteur la rejette et à quelle étape (marcheur, nom, règle 2, 3 ou 4).

Solution

`List`, `Switchy`, `Portal` : nom accepté, règles 2 à 4 en échec (appel, `switch`, appel), aucun hook direct. `Logger` : règle 4 en échec, le hook est un argument d'appel. `Memo` : rejeté par `consider_var` (l'initialiseur est un appel). `Klass` : rejeté par le marcheur (`ClassDeclaration`). `Dyn` : composant par la règle 2. `useful` : pas un hook (quatrième caractère minuscule), pas un composant (minuscule), pas un utilitaire (retour JSX).

Exercice 5.2: Comblé le trou du switch

Proposer la modification minimale de `stmt_has_jsx_return` qui fait détecter `ViaSwitch` (exemple 5.11). Montrer qu'elle ne peut introduire que des faux positifs.

Solution

Ajouter un bras `Statement::SwitchStatement(sw) => sw.cases.iter().any(|c| body_returns_jsx(&c.consequent))`, sur le modèle de `stmt_calls_hook`. La modification n'agrandit que l'ensemble des fonctions classées composants; par invariant 5.2, une fonction ajoutée à tort est analysée pour rien (faux positif possible), et aucune fonction auparavant analysée ne disparaît. Elle ne peut donc créer aucun faux négatif.

Exercice 5.3: L'ordre « react » puis résolveur

Expliquer pourquoi `build_hook_origins` teste la chaîne "react" avant d'appeler le résolveur. Décrire une configuration `tsconfig.json` qui casserait l'ordre inverse, et l'effet observable.

Solution

Une entrée `"paths": "react": [".../src/shims/react.ts"]` fait résoudre "react" vers un fichier local. Avec l'ordre inverse, tous les imports de React deviendraient `HookOrigin::File`, donc des hooks Custom : `useState` ne créerait plus de slot, `useEffect` plus d'effet, et toutes les règles qui en dépendent se tairaient. C'est le « mass-FN hazard » du commentaire.

Exercice 5.4: Types et arêtes

Pourquoi `import type` est-il exclu de `ModuleFacts::imports` mais pas de `build_resolved_imports`? Est-ce un problème de soundness?

Solution

Une arête de `ModuleFacts` a une sémantique d'exécution (elle propage un environnement serveur ou client) : une arête de type ferait entrer à tort un module serveur dans la clôture client. Un import résolu n'est qu'une clé de recherche dans les tables de composants, contextes et utilitaires; un nom de type n'y rencontre rien. Aucun problème de soundness dans les deux cas.

Exercice 5.5: Un pelage central

`const P = (() => <div/>);` n'est pas détecté, alors que `collect_module_consts` sait peler les parenthèses. Proposer une correction conforme au principe 3 de `CLAUDE.md`, plutôt qu'un cas spécial dans `consider_var`.

Solution

Extraire `peel_ts` de `collect_module_consts` en une fonction du module `lowering` (elle ne dépend que d'oxc), et l'appliquer à l'initialiseur dans `consider_var` comme dans `collect_module_consts`. Un seul pelage partagé : la prochaine enveloppe (`satisfies`, `as const`) sera traitée partout à la fois.

Exercice 5.6: Tracer un alias

Pour `import Widget as Panel from "../b/Widget"` (exemple 5.9), suivre le nom `Panel` de `build_resolved_imports` jusqu'à `resolve_child`, en nommant chaque structure traversée.

Solution

`build_resolved_imports` produit `Panel` $\mapsto$ `ResolvedImport` `file: b/Widget.tsx, imported: "Widget"`; `build_jsx_origins` en fait `Panel` $\mapsto$ `CompOrigin` `file: b/Widget.tsx, name: "Widget"` dans `JsxOrigins`; `LowerCtx` transporte la table jusqu'à `expr_lower.rs`, qui pose l'origine sur `Expr::CompApp`. Au moteur, `resolve_child` trouve la clé (`b/Widget.tsx`, `Widget`) dans le registre et répond `Resolved`, sans consulter le nom `Panel`.

Chapitre 6

La représentation intermédiaire

Objectifs du chapitre.

- Comprendre pourquoi `reactant-analyzer` n'interprète pas l'AST d'oxc mais une représentation intermédiaire (IR) dédiée, en graphe de flot de contrôle, et quelles alternatives ont été écartées (ADR-3, ADR-4).
- Lire sans hésiter les types du squelette : CFG, BasicBlock, Terminator, EdgeKind, Stmt, et connaître leurs invariants (validate, ordre des blocs, blocs orphelins).
- Connaître l'énumération Expr famille par famille, la notion de *site* d'allocation (ExprId) et la discipline du parcours canonique for_each_child.
- Savoir comment un appel de hook devient une ligne HookEntry, un label positionnel (le *slot*), un marqueur HookMarker, et comment un tableau de deps est lu (DepsArg, DepsList, Arity).
- Situer les unités (ComponentIR, HookIR, FunctionIR), les faits de module et les identités internées (FileId, ComponentId, ADR-40).
- Savoir d'où vient une position de diagnostic (SourceRange, ADR-11, ADR-39) et utiliser les analyses syntaxiques de l'IR : variables libres, chemins d'accès, couverture par préfixe, certificats de liaison.

Prérequis : chapitre 5 (d'où viennent les unités abaissées), chapitre 2 (règles des hooks, deps, handlers), chapitre 3 (treillis, définition 3.2; soundness, définition 3.8; dominance, définition 3.9).

Ce chapitre décrit le *vocabulaire commun* du projet. Le lowering (chapitre 7) l'écrit ; le moteur l'interprète ; les relations et les règles le relisent ; le driver n'en consomme que les positions et les identités. Tout ce qui vit sous `src/ir/` est décrit ici, sauf la greffe de CFG (`splice.rs`) et le décalage d'identités (`remap.rs`), qui ont leur chapitre (chapitre 8). Nous partons d'un compteur de dix lignes, et nous le suivons jusqu'à sa forme abaissée.

6.1 Pourquoi une IR, et pourquoi un CFG

6.1.1 Un compteur et ses formes équivalentes

Voici le programme qui nous accompagnera tout au long du chapitre.

Extrait 6.1 – Un compteur avec retour anticipé (`/tmp/ch06/counter.tsx`)

```
1 import { useState } from "react";
2
3 export function Counter() {
4   const [count, setCount] = useState(0);
5   if (count > 10) {
6     return null;
7   }
8   return <button onClick={() => setCount(count + 1)}>{count}</button>;
9 }
```

Pour analyser ce composant, l'interpréteur abstrait doit savoir trois choses : que `count` est la valeur d'un emplacement d'état et `setCount` son écrivain ; qu'il existe deux chemins vers la sortie, dont l'un s'arrête avant le JSX ; que la flèche passée à `onClick` n'est pas exécutée pendant le rendu, mais plus tard, un nombre quelconque de fois.

L'AST d'oxc ne dit rien de tout cela directement. Il dit qu'il y a une `VariableDeclaration` dont le motif est un `ArrayPattern`, qu'il y a un `IfStatement` dont le bloc contient un `ReturnStatement`, et qu'il y a un `JSXElement` avec un `JSXAttribute`. Pire, JavaScript offre des dizaines d'orthographe pour la même chose : `if (c) return null;` et `return c ? null : <button/>` ; une déstructuration et deux accès indexés ; `a && f()` et `if (a) f()`. Une fonction de transfert écrite sur l'AST devrait traiter chacune d'elles, et chacune serait une occasion d'oublier un cas — c'est-à-dire, pour un analyseur qui se veut *sound*, un faux négatif (FN).

6.1.2 La décision : une IR dédiée, en graphe de flot de contrôle

Décision : ADR-3 — une IR dédiée, fondée sur un CFG

Contexte. Appliquer les fonctions de transfert sur l'AST oxc oblige à traiter « des dizaines de formes équivalentes » (déstructuration, court-circuits, ternaires, retours anticipés).

Décision. Une IR dédiée, inspirée de `React-tRace` [LAY25] mais représentée en CFG. Le *lowering* est la seule couche qui lit l'AST ; « Abstract domains never see the Oxc AST, only the IR » (section *Consequences*).

Alternatives refusées. (1) Travailler sur l'AST : refusé pour la raison ci-dessus. (2) Une IR arborescente à la `React-tRace` : elle exige une transformation CPS pour les retours anticipés et ne représente pas les boucles sans arcs arrière. Le CFG représente nativement `return`, boucles et `switch`, identifie structurellement les en-têtes de boucle (où placer le *widening*, définition 3.6), et rend la dominance — nécessaire pour détecter un hook conditionnel — calculable par un algorithme standard.

Écarts avec le code d'aujourd'hui. L'ADR renvoie à un `docs/ir.md` qui n'existe pas ; elle décrit des nœuds `useState/useStateEffect/hookCall` (aujourd'hui : une table `HookEntry` et des marqueurs, section 6.4), un `TsAnnotated expr, ty` (aujourd'hui sans type) et un désucre de `a && b` en expression `If` (aujourd'hui : un diamant de blocs). La décision de fond, elle, tient.

Invariant 6.1: L'IR est la frontière de l'AST

Aucun type de `src/ir/` ne référence oxc. Toutes les structures sont *possédées* (`String`, `PathBuf`, `Arc<...>`), sans durée de vie liée à l'arène du parseur : l'AST d'un fichier est libéré dès la fin de son *lowering* (un `Allocator` par fichier dans `lower_files_with`, `src/resolver/mod.rs`, l. 247–349). Toutes dérivent `Debug` et `Clone` : le moteur clone et réécrit librement l'IR (greffe, décalage).

Décision : ADR-4 — un composant, c'est un rendu et une table de hooks

Décision. Un composant est représenté par un `render_cfg` et une table `hooks` dont chaque effet porte son propre CFG. Il n'y a pas de méta-CFG unique « rendu → effet → vérification ».

Alternative refusée. Le méta-CFG : graphe énorme, séparation rendu/effet rendue implicite, risque de mal modéliser l'ordre d'exécution de React. L'argument décisif : les bugs visés sont des propriétés de l'état *au point fixe*, pas de chemins dans un graphe unifié. La séparation est un invariant sémantique de React (un effet ne s'exécute jamais pendant le rendu) ; l'inscrire dans la structure le préserve sans effort.

Écart. Le squelette de l'ADR (`deps: Option<Vec<Expr>`, pas de handlers, pas de positions) est antérieur au code ; nous décrivons le code.

6.1.3 Le périmètre : seize fichiers

Tableau 6.1 donne la carte de `src/ir/` (4 423 lignes au total). Le module `mod.rs` ré-exporte la surface utilisée par le reste du crate (`src/ir/mod.rs`, l. 17–34) ; certains types importants ne sont *pas* ré-exportés et se nomment par leur chemin complet : `ExprId` (`crate::ir::types`), `MarkerVal` et `SummaryValue` (`crate::ir::expr`), `DepsArg` et `HookProvenance` (`crate::ir::hooks`), `AccessPath` (`crate::ir::free_vars`), `ClosureBinding` (`crate::ir::bindings`).

Fichier	Lignes	Rôle
<code>types.rs</code>	25	alias de base, <code>ExprId</code> , <code>QualifiedSlot</code>
<code>cfg.rs</code>	228	CFG, blocs, terminateurs, arêtes, accessibilité, <code>validate</code>
<code>stmt.rs</code>	54	instructions <code>Stmt</code> , <code>MemberKey</code>
<code>expr.rs</code>	548	expressions, opérateurs, valeurs de hooks, parcours canonique
<code>hooks.rs</code>	358	<code>HookEntry</code> , <code>DepsArg</code> , <code>DepsList</code> , <code>Arity</code> , <code>HookProvenance</code>
<code>component.rs</code>	83	<code>ComponentIR</code> , <code>ModuleConstInit</code> , <code>ContextId</code>
<code>hook_ir.rs</code> , <code>function_ir.rs</code>	24, 22	<code>HookIR</code> , <code>FunctionIR</code>
<code>component_id.rs</code>	274	<code>ComponentId</code> , <code>ComponentTable</code> (ADR-40)
<code>module.rs</code>	222	<code>ModuleFacts</code> , <code>ModuleTable</code>
<code>source_range.rs</code>	102	<code>FileId</code> , <code>FileTable</code> , <code>SourceRange</code> , <code>SourceMap</code>
<code>free_vars.rs</code>	630	variables libres, <code>AccessPath</code> , épingleage, couverture
<code>bindings.rs</code>	212	certificats de liaison, <code>ClosureBinding</code>
<code>splice.rs</code> , <code>remap.rs</code>	1156, 451	greffe, α -renommage, décalage (chapitre 8)
<code>mod.rs</code>	34	déclarations et ré-exports

TABLE 6.1 – Les fichiers de `src/ir/` au commit `e67b10a` (`wc -l`).

6.1.4 Les alias de base

Le fichier le plus court fixe le vocabulaire :

Extrait 6.2 – `src/ir/types.rs`, l. 1–25 — alias de base et clé d'allocation

```

1 pub type Symbol = String;
2 pub type HookLabel = usize;
3 pub type BlockId = usize;
4 pub type Var = String;
5
6 /// A state slot qualified by the component that owns it. `HookLabel` is
7 /// per-component, so every cross-component fact – a `Versioned` label set, a
8 /// churn node, a foreign writer row – names a slot this way (ADR-042 §2).
9 pub type QualifiedSlot = (super::component_id::ComponentId, HookLabel);
10
11 /// Allocation-site key. `Ord` so that a walk over a set of sites has one
12 /// stable order – the first match over a `HashSet<ExprId>` used to depend on
13 /// the process hash seed (#120).
14 #[derive(Debug, Clone, Copy, PartialEq, Eq, Hash, PartialOrd, Ord)]
15 pub struct ExprId(pub usize);
16
17 impl ExprId {
18     /// Allocate a fresh `ExprId` not derived from any AST node.
19     /// Used when creating synthetic heap values during inter-component analysis.
20     pub fn fresh() -> Self {
21         use std::sync::atomic::{AtomicUsize, Ordering};
22         static COUNTER: AtomicUsize = AtomicUsize::new(1_000_000_000);
23         ExprId(COUNTER.fetch_add(1, Ordering::Relaxed))
24     }
25 }
```

```
25 }
```

Symbol et Var sont tous deux des String ; par convention, Symbol nomme une entité (composant, champ, tag JSX) et Var une variable d'un corps, mais aucun typage fort ne les distingue. BlockId identifie un bloc *dans un* CFG. HookLabel est le rang d'un hook *dans un* composant (section 6.4) ; comme il n'est pas global, tout fait qui traverse les composants nomme un slot par la paire QualifiedSlot (ADR-42 §2). ExprId est la clé d'un site d'allocation, que nous définirons en section 6.3.3.

6.2 Le squelette : blocs, terminateurs, arêtes, instructions

6.2.1 Le CFG du compteur

Avant les définitions, regardons ce que le lowering produit pour le liste 6.1. L'impression suivante vient d'une sonde de lecture (un petit programme hors dépôt qui appelle `resolver::lower_files` et imprime l'IR) ; la notation est simple : `B0` : ouvre un bloc, `->` introduit le terminateur, `let x = e @fFileId(0):4:9` est une instruction Let placée en ligne 4, colonne 9 du fichier 0, `Obj#0{...}` et `Fn#1(...)` sont des nœuds allouants avec leur ExprId.

```
ComponentIR Counter file=counter.tsx param=props dom_props={} module_consts=[]
render_cfg:
  entry=B0
  B0:
    let count = StateVal(0)    @fFileId(0):4:9
    let setCount = StateSetter(0) @fFileId(0):4:16
    -> branch (count Gt 10) ? B1 : B2 @fFileId(0):5:6
  B1:
    -> return null
  B2:
    -> jump B3
  B3:
    -> return Native<button>(Obj#0{onClick: Fn#1() {
      entry=B0
      B0:
        -> return setCount((count Add 1))
      edges:
    }}; [count]) prop_spans=["onClick@fFileId(0):8:17"] @fFileId(0):8:9
  edges: B0->B1:IfTrue B0->B2:IfFalse B2->B3:Unconditional
hook: State #0 init=0 @fFileId(0):4:8
hook: Handler #1 event=click @fFileId(0):8:17
  entry=B0
  B0:
    -> return setCount((count Add 1))
  edges:
prov: HookProvenance { label: 0, origin_hook: "useState", react: true, specifier: Some("react"), file: None, inlined:
  ↪ false, span: Some(SourceRange { file: FileId(0), line: 4, col: 8 }) }
```

Figure 6.1 dessine le même graphe. Plusieurs traits de l'IR s'y lisent déjà. L'appel `useState(0)` a disparu du rendu : il est devenu une ligne `State #0` de la table des hooks, et les deux liaisons du tuple lisent `StateVal(0)` et `StateSetter(0)`. Le `if` est un terminateur Branch à deux arêtes typées. Le bras « sinon » implicite est un bloc vide B2 qui saute vers la suite. Enfin, la flèche de `onClick` est présente *deux fois* : dans les props de l'élément (c'est une valeur créée à chaque rendu) et comme point d'entrée Handler #1.

6.2.2 Les types

Extrait 6.3 – `src/ir/cfg.rs`, l. 5–65 — blocs, terminateurs, arêtes et graphe

```
5 #[derive(Debug, Clone)]
```

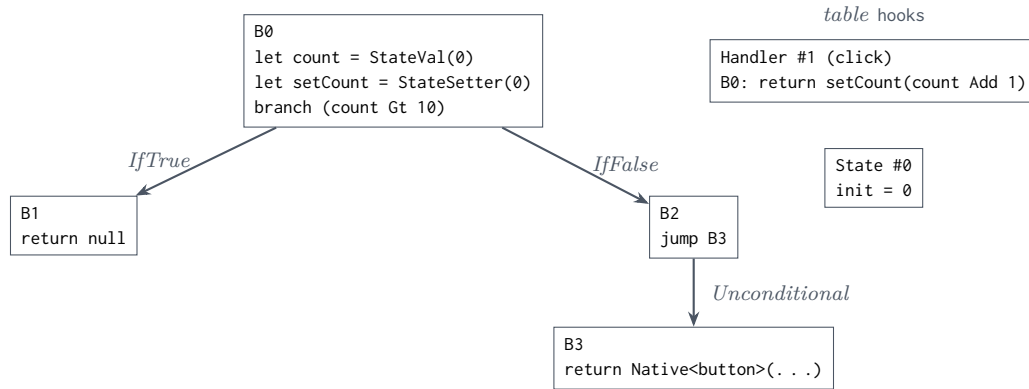


FIGURE 6.1 – Le `render_cfg` du compteur (liste 6.1) et sa table de hooks. Le bloc B1 et le bloc B3 sont deux sorties ; le corps du handler est un CFG à part.

```

6 pub struct BasicBlock {
7     pub id: BlockId,
8     pub stmts: Vec<Stmt>,
9     pub term: Terminator,
10 }
11
12 #[derive(Debug, Clone)]
13 pub enum Terminator {
14     Jump(BlockId),
15     Branch {
16         cond: Expr,
17         then_: BlockId,
18         else_: BlockId,
19         /// Where the condition is evaluated in the source (None for
20         /// synthetic branches and manual-IR tests).
21         span: Option<crate::ir::SourceRange>,
22     },
23     /// Unlike [Terminator::Branch], this carries no span: nothing needed the
24     /// position of a `return` until a hook could be extracted from one (#4),
25     /// and adding it now is a 40-site IR change tracked separately. The cost is
26     /// that a hook reached only through a return yields findings with no line
27     /// number – visible but unlocated, which is still strictly better than the
28     /// silence it replaced.
29     Return(Expr),
30     Unreachable,
31 }
32
33 #[derive(Debug, Clone, PartialEq, Eq)]
34 pub enum EdgeKind {
35     Unconditional,
36     IfTrue,
37     IfFalse,
38     Back,
39     /// The split at an `await` (#117, ADR-035). Control is the same – the edge
40     /// is unconditional – but the successor runs on a later turn of the event
41     /// loop, so `sync_phase`'s "lexis = execution, provably" stops holding
42     /// across it. Consumers ask [CFG::post_await_blocks] rather than reading
43     /// this variant directly.
44     Await,
45 }
46
47 #[derive(Debug, Clone)]
48 pub struct Edge {

```

```

49     pub from: BlockId,
50     pub to: BlockId,
51     pub kind: EdgeKind,
52 }
53
54 /// The block map is a [BTreeMap], not a HashMap, on purpose: every walk
55 /// over `blocks` then visits them in ascending [BlockId] – i.e. lowering
56 /// order – so a pass that picks a *representative* block (the first setter
57 /// call site of a witness, say) reports the same one on every run. Under a
58 /// `HashMap` that choice followed the per-process hash seed and diagnostics
59 /// were not reproducible.
60 #[derive(Debug, Clone)]
61 pub struct CFG {
62     pub entry: BlockId,
63     pub blocks: BTreeMap<BlockId, BasicBlock>,
64     pub edges: Vec<Edge>,
65 }

```

Définition 6.1: Graphe de flot de contrôle (CFG)

Un *CFG* de *reactant-analyzer* est un triplet $(entry, B, E)$ où B associe à chaque *BlockId* un *bloc de base* — une séquence linéaire d'instructions *Stmt* suivie d'un unique *terminateur* — et où E est une liste d'arêtes $(from, to, kind)$. Le terminateur est *Jump(t)* (saut), *Branch cond, then_, else_* (choix sur une expression), *Return(e)* (sortie du corps avec la valeur de e) ou *Unreachable* (le contrôle ne continue pas : *throw*, *break* orphelin). La nature d'une arête, *EdgeKind*, vaut *Unconditional*, *IfTrue*, *IfFalse*, *Back* (arc arrière d'une boucle) ou *Await* (le successeur s'exécute à un tour ultérieur de la boucle d'événements).

Reprenons les champs un par un.

- *BasicBlock.id* doit être égal à la clé sous laquelle le bloc est rangé ; *stmts* est la séquence d'instructions ; *term* le terminateur unique.
- *Branch.cond* est une expression quelconque ; pour un court-circuit *&&/||*, c'est une variable temporaire *Var("__tN")* (le diamant est expliqué plus bas). *Branch.span* situe la condition dans la source.
- *Return(e)* ne porte *pas* de position. Le commentaire (*src/ir/cfg.rs*, l. 23–28) l'assume : l'ajouter touche une quarantaine de sites de construction ; le prix est qu'un hook atteint seulement par un *return* produit des findings sans numéro de ligne. C'est l'issue ouverte #140.
- *Unreachable* n'est jamais la chute en fin de corps : depuis ADR-25, une fonction qui « tombe » de sa dernière instruction se termine par *Return(Lit(Unit))*, c'est-à-dire *return undefined*.
- *EdgeKind* : *IfTrue/IfFalse* servent au raffinement par les gardes, *Back* au placement du widening (ADR-14), *Await* à la frontière de phase des corps asynchrones (ADR-35).
- *CFG.blocks* est une *BTreeMap* : le commentaire (*src/ir/cfg.rs*, l. 54–59) en donne la raison, la reproductibilité. Tout parcours visite les blocs par *BlockId* croissant, donc dans l'ordre du lowering, et une passe qui choisit un bloc « représentatif » (le premier site d'appel d'un setter, pour un témoin) choisit le même à chaque exécution. Sous une *HashMap*, ce choix suivait la graine de hachage du processus.

Piège

La navigation lit les arêtes, pas les terminateurs. *successors* et *predecessors* parcourent *CFG.edges*. Un terminateur *Jump(7)* sans arête $(b, 7)$ correspondante rend le bloc 7 invisible au *worklist*, bien qu'il soit présent dans *blocks*. C'est la troisième clause de

validate, ci-dessous.

Piège

Sur un diamant de court-circuit, `EdgeKind` n'est pas une polarité. Pour `a && b`, le lowering produit `let __t = a; branch __t ? rhs : join`, puis `__t := b` dans `rhs`. Mais les *deux* arêtes sortantes reçoivent `IfTrue`, et pour `||/??` les deux reçoivent `IfFalse` (`src/lowering/expr_lower.rs`, l. 763–780) : la nature de l'arête encode l'*opérateur* (le sens dans lequel on entre dans l'opérande droit), et c'est ainsi que `guards.rs` la relit pour déplier une garde conjonctive (`src/engine/guards.rs`, l. 1019–1028). Observé sur `ok && setX(1)` : `edges: ... B4->B8:IfTrue B4->B9:IfTrue`. L'interpréteur, lui, raffine l'environnement d'après `then_/else_` du terminateur (`src/engine/cfg_analyzer.rs`, l. 135–140), ce qui est correct. Un nouveau consommateur qui lirait `IfTrue` comme « la condition est vraie sur cette arête » se tromperait sur l'arête qui court-circuite.

6.2.3 Deux accessibilités et une validation

Trois méthodes de CFG méritent d'être connues par cœur.

Extrait 6.4 – `src/ir/cfg.rs`, l. 146–163 — les blocs atteignables depuis l'entrée

```
146  /// Blocks reachable from [`Self::entry`] by following edges.
147  ///
148  /// Splicing and `if`/`else` lowering both leave orphan blocks behind (a
149  /// join whose every predecessor returned, an inlined body after a `throw`),
150  /// so "every block of the CFG" and "every block that can execute" are not
151  /// the same set. Anything quantifying over program points – exits above
152  /// all – must use this one.
153  pub fn reachable_blocks(&self) -> std::collections::HashSet<BlockId> {
154      let mut seen = std::collections::HashSet::new();
155      let mut stack = vec![self.entry];
156      while let Some(id) = stack.pop() {
157          if !seen.insert(id) {
158              continue;
159          }
160          stack.extend(self.successors(id));
161      }
162      seen
163  }
```

Le commentaire est la leçon : le lowering d'un `if/else` et la greffe laissent des *blocs orphelins* (une jointure dont tous les prédécesseurs ont retourné, un corps inliné après un `throw`). « Tous les blocs du CFG » et « tous les blocs qui peuvent s'exécuter » ne sont pas le même ensemble ; toute quantification sur les points de programme — et d'abord sur les sorties — doit passer par `reachable_blocks`.

La seconde accessibilité est `post_await_blocks` (`src/ir/cfg.rs`, l. 111–144) : la fermeture, par les successeurs, des cibles de toutes les arêtes `Await` du corps. Elle sur-approxime « exécuté après une suspension » : l'en-tête d'une boucle dont le corps attend en fait partie, car la deuxième itération s'exécute bien après la suspension. Un `await` dans un callback imbriqué n'affecte que le CFG de ce callback, qui est un graphe distinct.

Extrait 6.5 – `src/ir/cfg.rs`, l. 181–227 — la bonne formation structurelle

```
181  /// Structural well-formedness, for `debug_assert!` after a transformation
182  /// that rewrites blocks and edges (splicing, remapping).
183  ///
```



```

184     /// Returns the first violation as prose, or `Ok(())`. Three things have to
185     /// hold for the engine to see the whole program:
186     ///
187     /// 1. The entry block exists – without it nothing is analysed at all.
188     /// 2. Every terminator target names a block that exists – a dangling target
189     ///    is code the fixpoint silently never enters.
190     /// 3. Every terminator target also has an edge. `successors` reads `edges`,
191     ///    not terminators, so a target with no edge is invisible to the
192     ///    worklist even though the block is there.
193     ///
194     /// Cheap enough to run under `debug_assert!` but linear in edges per block,
195     /// so it stays out of release builds.
196     pub fn validate(&self) -> Result<(), String> {
197         if !self.blocks.contains_key(&self.entry) {
198             return Err(format!("entry block {} is missing", self.entry));
199         }
200         for edge in &self.edges {
201             if !self.blocks.contains_key(&edge.from) {
202                 return Err(format!("edge from missing block {}", edge.from));
203             }
204             if !self.blocks.contains_key(&edge.to) {
205                 return Err(format!("edge to missing block {}", edge.to));
206             }
207         }
208         for (&id, block) in &self.blocks {
209             if block.id != id {
210                 return Err(format!("block keyed {} reports id {}", id, block.id));
211             }
212             let targets: Vec<BlockId> = match &block.term {
213                 Terminator::Jump(t) => vec![*t],
214                 Terminator::Branch { then_, else_, .. } => vec![*then_, *else_],
215                 Terminator::Return(_) | Terminator::Unreachable => vec![],
216             };
217             for t in targets {
218                 if !self.blocks.contains_key(&t) {
219                     return Err(format!("block {} targets missing block {}", id, t));
220                 }
221                 if !self.edges.iter().any(|e| e.from == id && e.to == t) {
222                     return Err(format!("block {} targets {} with no edge", id, t));
223                 }
224             }
225         }
226         Ok(())
227     }

```

`validate` annonce trois clauses et en vérifie en fait quatre : l'entrée existe ; chaque arête relie deux blocs existants ; chaque bloc est rangé sous son propre `id` ; chaque cible de terminateur existe *et* possède une arête. Elle ne vérifie pas la réciproque (une arête sans terminateur correspondant), ni la densité des identifiants. Son seul appel est un `debug_assert!` en fin de greffe (`src/ir/splice.rs`, l. 251–255) : actif dans les tests et en build debug, absent en release.

Remarque

Le dossier source relève un invariant *implicite* : les `BlockId` d'un CFG forment l'intervalle $0..n$ sans trou. Le lowering alloue les blocs par un compteur (le bloc 0 est l'entrée) et la greffe place les blocs de l'appelé à la suite du maximum ; elle en dépend pour placer son bloc de jointure (chapitre 8). Cet invariant n'est ni documenté ni vérifié par `validate`. Au e67b10a, il tient sans exception sur les 930 CFG des fixtures et les 5 512 CFG de cinq corpus externes ; cette mesure, empirique et non refaite pour ce chapitre, est celle du dossier source

(§3.3), et ne prouve donc rien au-delà de ces 6 442 CFG.

6.2.4 Les instructions

Extrait 6.6 – `src/ir/stmt.rs`, l. 7–29 — les quatre formes d’instruction

```

7 #[derive(Debug, Clone)]
8 pub enum Stmt {
9     Let {
10         var: Var,
11         rhs: Expr,
12         span: Option<SourceRange>,
13     },
14     Assign {
15         var: Var,
16         rhs: Expr,
17         span: Option<SourceRange>,
18     },
19     /// In-place write through a member expression: `obj.f = v`, `arr[i] = v`,
20     /// `obj.f++`, `delete obj.f`. The heap identity of `obj` is unchanged –
21     /// that is the semantic payload: a mutation, not a rebinding.
22     MemberWrite {
23         obj: Expr,
24         key: MemberKey,
25         rhs: Expr,
26         span: Option<SourceRange>,
27     },
28     ExprStmt(Expr, Option<SourceRange>),
29 }

```

Quatre formes seulement, toutes avec une position optionnelle :

- `Let var, rhs` introduit une liaison locale (`const`, `let`, un temporaire du lowering, un paramètre greffé).
- `Assign var, rhs` écrit dans une liaison existante. La distinction avec `Let` n’est pas cosmétique : un `Assign` sans `Let` du même nom dans le corps est « l’orthographe IR d’une écriture dans une liaison extérieure » (`src/ir/splice.rs`, l. 160–163), ce que la greffe doit respecter.
- `MemberWrite obj, key, rhs` est une écriture *en place* : `obj.f = v`, `arr[i] = v`, `obj.f++`, `delete obj.f`. L’identité de l’objet dans le tas ne change pas — c’est une mutation, pas une re-liaison — et c’est exactement ce que la règle `state-mutation` doit voir. La clé est un `MemberKey` :

Extrait 6.7 – `src/ir/stmt.rs`, l. 47–54 — la cible d’une écriture membre

```

47 /// Which member of the object a [Stmt::MemberWrite] targets. `Index` keeps
48 /// the index expression alive: it is evaluated at runtime, so its reads count
49 /// (free variables, callbacks).
50 #[derive(Debug, Clone)]
51 pub enum MemberKey {
52     Field(Symbol),
53     Index(Expr),
54 }

```

- `ExprStmt(e)` évalue une expression pour ses effets (un appel, typiquement).

Exemple 6.1: Une affectation en position d'instruction est émise deux fois

Dans l'effet `useEffect(() => document.title = String(total); )`, la sonde imprime :

```
B0:
document.title <- String(total)  @fFileId(0):11:4
String(total);  @fFileId(0):11:4
-> return undefined
```

Le `MemberWrite` est suivi d'un `ExprStmt` qui ré-évalue le membre droit. La cause est vérifiée dans le lowering : une expression d'affectation pousse son écriture puis *renvoie sa valeur* (`src/lowering/expr_lower.rs`, l. 519–535), et une instruction-expression enveloppe toujours cette valeur dans un `ExprStmt` (`src/lowering/cfg_builder.rs`, l. 286–289). Même chose pour `x = e;`, qui donne `Assign` puis `ExprStmt(Var(x))`. C'est sans danger pour la soundness : un appel sur-compté n'est pas un appel perdu. Mais un lecteur qui compte les appels d'un corps doit le savoir.

Remarque

`Stmt::span` (`src/ir/stmt.rs`, l. 31–45) documente qu'un `None` est « un bug à corriger, pas une forme à contourner » (ADR-39). Un résidu subsiste : le `let __obj_N = __p0` produit pour un paramètre déstructuré (fonction `C( x )`) n'a pas de position (`@-` dans les impressions de la section 6.8), parce que `inject_param_preamble` passe `None` au lowering du motif (`src/lowering/cfg_builder.rs`, l. 963–965). La conséquence est bénigne : ce `Let` ne fait que copier une variable.

6.3 Les expressions

6.3.1 Littéraux et opérateurs

Extrait 6.8 – `src/ir/expr.rs`, l. 10–18 — les littéraux primitifs

```
10 #[derive(Debug, Clone)]
11 pub enum Prim {
12     String(String),
13     Int(i32),
14     Float(f64),
15     Bool(bool),
16     Null,
17     Unit,
18 }
```

`Unit` est `undefined`. Un littéral numérique devient `Int` s'il est entier et de valeur absolue inférieure à `i32::MAX`, `Float` sinon (`src/lowering/expr_lower.rs`, l. 148–154). `Prim` ne dérive pas `PartialEq` (un `f64` interdirait de toute façon `Eq`).

Les opérateurs binaires (`src/ir/expr.rs`, l. 20–63) ont chacun leur variant : `Add`, `Sub`, `Mul`, `Div`, `Mod`, `Pow`, les comparaisons, `In`, `InstanceOf` et les six opérateurs bit à bit. Le commentaire de `BitAnd` énonce la politique, qui vaut pour tout l'IR :

Extrait 6.9 – `src/ir/expr.rs`, l. 44–56 — un variant par opérateur, plus de `Unknown`

```
44 /// Bitwise and shift: `&`, `|`, `^`, `<<`, `>>`, `>>>`.
45 ///
46 /// Real variants rather than one opaque `Unknown`, because they carry
47 /// information `Unknown` could not express: JS coerces both operands to
48 /// int32 (uint32 for `>>>`), so the result is always a number in a known
49 /// range. Folding them into `Unknown` erased which operator it was, and
```

```

50    /// with it every one of those guarantees.
51    ///
52    /// This is the shared decision for every operator: give it a variant that
53    /// says what is known about it. There is no `Unknown` any more – every
54    /// binary operator of the language has its own variant, and `lower_binop`
55    /// is exhaustive so a new one cannot slip in silently.
56    BitAnd,

```

`lower_binop` n'a pas de bras `_` : un opérateur ajouté au langage serait une erreur de compilation, pas une valeur silencieusement fausse. Deux détails comptent. D'une part `Eq/Neq` couvrent à la fois `==` et `===` (`src/lowering/expr_lower.rs`, l. 1187–1188) : l'IR ne distingue pas égalité stricte et non stricte. D'autre part `And/Or` existent dans `BinOp`, mais les `&&/||` du source sont désués en diamant de blocs (ADR-20, non-changement n°1), pour ne pas perdre les effets de bord d'un opérande droit court-circuité. Côté unaire (`src/ir/expr.rs`, l. 65–80), `UnaryOp::Unknown` subsiste pour les opérateurs restants ; il s'évalue à $\top$, « jamais comme l'identité, qui falsifierait la valeur de l'opérande ».

6.3.2 L'énumération Expr

Voici la première moitié de l'énumération : littéraux, nœuds allouants, variables, accès et opérateurs.

Extrait 6.10 – `src/ir/expr.rs`, l. 174–228 — Expr, première partie

```

174 #[derive(Debug, Clone)]
175 pub enum Expr {
176     // Primitive literals
177     Lit(Prim),
178
179     // Composites each allocating node carries an ExprId (allocation-site key for the heap).
180     ObjectLit {
181         id: ExprId,
182         fields: Vec<(Symbol, Expr)>,
183     },
184     ArrayLit {
185         id: ExprId,
186         elems: Vec<Expr>,
187         /// How long the *source* array is, which `elems.len()` stops telling
188         /// once lowering flattens a `SpreadElement` into its source (one
189         /// element standing for however many it holds) or drops an elision
190         /// (no element at all). An elision still leaves the length countable,
191         /// so only a spread makes it a lower bound. This is the last point
192         /// where any of it is knowable, which is why it is recorded here
193         /// rather than derived later.
194         arity: Arity,
195         /// Positions in `elems` that came from a spread. Such an element is a
196         /// *container* of entries, not an entry: a reader that treats it as
197         /// one claims the array holds `rows` where it holds `rows[0], ...`.
198         spread_at: Vec<usize>,
199     },
200     FnLit {
201         id: ExprId,
202         params: Vec<Var>,
203         body_cfg: Arc<CFG>,
204     },
205
206     // Vars
207     Var(Symbol),
208

```

```

209 // Accesses
210 FieldAccess {
211     obj: Box<Expr>,
212     field: Symbol,
213 },
214 IndexAccess {
215     arr: Box<Expr>,
216     idx: Box<Expr>,
217 },
218
219 // Ops
220 BinOp {
221     op: BinOp,
222     lhs: Box<Expr>,
223     rhs: Box<Expr>,
224 },
225 UnaryOp {
226     op: UnaryOp,
227     arg: Box<Expr>,
228 },

```

Puis les appels, le JSX, l'annotation TypeScript et les valeurs de hooks :

Extrait 6.11 – src/ir/expr.rs, l. 230–300 — Expr, seconde partie

```

230 // Calls
231 Call {
232     fn_: Box<Expr>,
233     args: Vec<Expr>,
234 },
235 /// `new X(args)`: a call that always allocates. The value domain reads it
236 /// as a fresh reference whose members are  $\top$ , and every syntactic proof
237 /// reads it where it reads an object literal – lowered to a plain `Call`
238 /// it was a value that "holds still" and that no freshness check could
239 /// see (#158). `id` is the allocation site, as for `ObjectLit`.
240 New {
241     id: ExprId,
242     fn_: Box<Expr>,
243     args: Vec<Expr>,
244 },
245 CompApp {
246     name: Symbol,
247     props: Box<Expr>,
248     /// Span of the element's opening tag. A diagnostic about a JSX element
249     /// (a context provider's `value`, an identity-keyed prop) has nowhere
250     /// else to point: unlike `NativeElem`, whose handler props are reached
251     /// through `HookEntry::Handler`, a component element owns no hook.
252     span: Option<SourceRange>,
253     /// Which component `name` was proven to name, when the file's
254     /// imports (or its own declarations) settle it – the same fact
255     /// `HookEntry::Custom::resolved_file` records for a custom hook call.
256     /// `None` is ignorance, never a proven negative.
257     origin: Option<Arc<CompOrigin>>,
258 },
259 NativeElem {
260     tag: Symbol,
261     props: Box<Expr>,
262     children: Vec<Expr>,
263     /// Span of the element's opening tag – what a finding about the
264     /// element points at (#125), the same fact `CompApp` carries.
265     span: Option<SourceRange>,
266     /// Spans of JSX event-handler props (`onX={fn}`), by prop name.

```

```

267     /// Populated during lowering; consumed by `hook_extractor` to set
268     /// `HookEntry::Handler.span`. A list, not a map: an element has a
269     /// handful of handlers, `Expr` is the IR's hottest type, and a `HashMap`
270     /// header costs it twice what the whole payload does.
271     prop_spans: Vec<(Symbol, Option<SourceRange>)>,
272 },
273
274     /// TypeScript annotation marker (`x as T`, `useState<T>(..)`). The declared
275     /// type itself is not retained: TS types are erased at runtime, so narrowing
276     /// an abstract value by a type annotation would be unsound (`useState<number>`
277     /// can hold `undefined`/`any`-cast values). The wrapper is kept only so
278     /// `peel_ts` can see through it to the underlying expression.
279     TSAnnotated(Box<Expr>),
280
281     /// React Hooks
282     StateVal(HookLabel),
283     StateSetter(HookLabel),
284     MemoVal(HookLabel),
285     CallbackVal(HookLabel),
286
287     /// Marks the call site of a hook whose result carries no tracked value
288     /// (`useEffect`, `useRef`, custom hooks, ...). Its primary role is to keep
289     /// the hook's label anchored in the CFG so call-site blocks survive
290     /// inlining and renumbering (`collect_hook_calls`, conditional-hook).
291     /// Every extracted hook leaves its label in the CFG – value-bearing kinds
292     /// via `StateVal`/`MemoVal`/..., all others via this. The [ `MarkerVal` ] says
293     /// what the *binding* reads as.
294     HookMarker(HookLabel, MarkerVal),
295
296     /// Injected by `expand_custom_hooks` for library hooks with a `HookSummary`.
297     /// Evaluates directly to the encoded abstract value without going through the
298     /// concrete expression language (avoids a circular dep between `ir` and `domains`).
299     SummaryVal(SummaryValue),
300 }

```

Le tableau 6.2 regroupe les variants en familles.

Quelques champs demandent un commentaire.

- `ArrayLit.arity` et `spread_at` : dès que le lowering aplatit un `spread` (`[...rows]` devient un seul élément `rows`) ou omet une élation (`[a, , b]`), `elems.len()` cesse de dire la longueur du tableau. Le commentaire l'explique : c'est le dernier point où cette information est connue, d'où son enregistrement. Nous retrouverons ces deux champs dans les deps (section 6.4.4).
- `New` existe depuis le commit photographié (#158) : abaissé en `Call`, `new Map()` était une valeur qui « tient en place », invisible à tout test de fraîcheur. C'est désormais un nœud allouant, comme un littéral d'objet.
- `CompApp.origin` est la preuve d'identité du composant nommé (ADR-40 §3, section 6.6) ; `None` est une ignorance, jamais une négation prouvée. `CompApp.props` est un `ObjectLit` où les enfants JSX sont rangés sous la clé `children`, comme en React ; `NativeElem` garde ses enfants à part, parce que le DOM n'a pas de prop `children` à passer à du code utilisateur.
- `NativeElem.prop_spans` est une liste et non une table, pour une raison de taille : `Expr` est le type le plus fréquent de l'IR, et un en-tête de `HashMap` lui coûterait deux fois sa charge utile.
- `TSAnnotated` ne garde pas le type (ADR-20, non-changement n°10) : les types TypeScript sont effacés à l'exécution, et raffiner une valeur abstraite par `useState<number>` serait unsound (l'état peut contenir `undefined` ou une valeur forcée par `as any`). Le nœud ne sert qu'à être pelé par `peel_ts`.

Famille	Variants	Source typique et remarques
littéral	Lit	1, "a", null; l'identifiant <code>undefined</code> devient <code>Lit(Unit)</code> .
allouant	ObjectLit, ArrayLit, FnLit, New	portent un <code>id: ExprId</code> (section 6.3.3). Un spread d'objet est rangé sous une clé préfixée par "..."; <code>ArrayLit.arity</code> garde la longueur <i>source</i> ; <code>FnLit.body_cfg</code> est un CFG séparé, partagé par <code>Arc</code> .
variable	Var	après une greffe, peut porter un suffixe <code>#n</code> (chapitre 8).
accès	FieldAccess, IndexAccess	<code>a?.b</code> s'abaisse comme <code>a.b</code> (<code>src/lowering/expr_lower.rs</code> , l. 649–653).
opérateur	BinOp, UnaryOp	un variant par opérateur.
appel	Call	callee opaque $\Rightarrow \top$, sauf inlining ou résumé.
JSX	CompApp, NativeElem	composant (props dont <code>children</code>) / élément natif (enfants à part).
annotation	TSAnotated	<code>x as T, useState<T>()</code> ; le type est effacé.
hook	StateVal, StateSetter, MemoVal, CallbackVal, HookMarker, SummaryVal	ce que lit une liaison de hook (section 6.4).

TABLE 6.2 – Les familles de Expr.

Rappel React : pourquoi les handlers ne sont pas exécutés au rendu

Une fonction passée à `onClick` est créée à chaque rendu mais n'est appelée qu'à l'arrivée d'un événement, un nombre quelconque de fois. L'IR reflète les deux faits : la `FnLit` reste dans les props de l'élément (une référence fraîche à chaque rendu, ce qui compte pour la stabilité référentielle) et son corps est réifié en `HookEntry::Handler`, un point d'entrée que le moteur exécute « plus tard, n'importe quand » dans son point fixe (chapitre 13).

6.3.3 Les sites d'allocation

Considérons `const style = color: "red"` ; dans un rendu. À chaque rendu, React réexécute le corps et crée un *nouvel* objet ; mais c'est toujours le *même* littéral du source. L'interprétation abstraite ne peut pas suivre les objets concrets (il y en a un par rendu, donc potentiellement une infinité) ; elle suit les littéraux qui les créent.

Définition 6.2: Site

Un *site* est une occurrence syntaxique, dans le code abaissé, d'une opération qui alloue une référence (littéral d'objet, de tableau, de fonction, `new`) ou qui écrit un état. Pour les allocations, le site est matérialisé par le champ `id: ExprId` des variants `ObjectLit`, `ArrayLit`, `FnLit`, `New` (et `SummaryValue::Shape`). Tous les objets concrets créés par un même site sont représentés par une seule entrée du tas abstrait, indexée par cet `ExprId` (ADR-10).

Dans le compteur, `Obj#0` (l'objet des props du `<button>`) et `Fn#1` (la flèche du handler) sont deux sites. Le modèle de tas lui-même relève de chapitre 11 ; retenons ici trois règles de fabrication.

1. Les `ExprId` du lowering viennent d'un compteur *par fichier* (`ExprIds`, un `Rc<Cell<usize>`» partagé par tous les corps abaissés d'un fichier, `src/lowering/cfg_builder.rs`, l. 20–42). Le « compteur dans `BlockBuilder` » décrit par ADR-10 §1 est obsolète depuis #134.
2. Un hook custom, abaissé par un autre appel, recommence à 0 : quand on greffe son corps

dans un composant, il faut décaler ses identifiants (`Offsets::ids`, chapitre 8), sans quoi l'objet de l'appelé partagerait l'entrée de tas d'un objet de l'appelant.

3. `ExprId::fresh` (liste 6.2) puise dans un compteur global au processus commençant à 10^9 , pour ne jamais croiser les identifiants du lowering. Il sert aux valeurs synthétiques de l'analyse inter-composants (l'objet props d'un enfant, `src/domains/transfer/state_value.rs`, l. 563).

`ExprId` dérive `Ord` depuis `#120` : le premier élément d'un parcours sur un `HashSet<ExprId>` dépendait de la graine de hachage.

Piège

« **Unique par fichier** » n'est pas « **unique dans le programme** ». Le dossier source a reproduit une collision : quand un parent passe à un enfant d'un *autre* fichier un objet de son tas, les entrées copiées portent les identifiants du fichier parent, alors que l'enfant alloue avec ceux de son propre fichier, et l'insertion dans le tas écrase (`src/domains/stores/heap.rs`, l. 41–43). Quand les deux numéros coïncident, la closure du parent est remplacée et une écriture d'état disparaît : un faux négatif. La collision elle-même (`Heap::insert` qui écrase, `src/domains/stores/heap.rs`, l. 41–43, et la copie par `ExprId` du fichier parent, `src/domains/transfer/state_value.rs`, l. 563) est vérifiée dans le code cité ; sa reproduction pas à pas sur un cas concret, faite par le dossier source (§6.8(b)/§8.1), n'a pas été rejouée pour ce chapitre. Ce défaut est détaillé au chapitre 16 et au chapitre 27.

6.3.4 Deux aides sur les littéraux d'objet

Un spread d'objet (`a: 1, ...opts`) est gardé comme un membre synthétique dont la clé commence par "...", préfixe qu'aucune propriété du source ne peut avoir. Une lecture par nom doit en tenir compte :

Extrait 6.12 – `src/ir/expr.rs`, l. 89–113 — lire un membre par son nom, en respectant les spreads

```

89 /// The suffix of `fields` a name lookup may read: everything after the last
90 /// spread. Members written before one may have been overwritten by something
91 /// the per-member map cannot see, so they answer nothing.
92 ///
93 /// Shared so the two readers that resolve members by name cannot drift apart
94 /// on the rule – the abstract interpreter's `obj_members` and the binding
95 /// chase in [crate::engine::seeds].
96 pub fn members_after_last_spread(fields: &[(Symbol, Expr)]) -> &[(Symbol, Expr)] {
97     let start = fields
98         .iter()
99         .rposition(|(k, _)| k.starts_with(SPREAD_KEY_PREFIX))
100         .map_or(0, |i| i + 1);
101     &fields[start..]
102 }
103
104 /// The expression bound to `key` in an object literal, exactly – `None` when
105 /// no readable member has that name, which includes every member a spread may
106 /// have overwritten.
107 pub fn object_member<'e>(fields: &'e [(Symbol, Expr)], key: &str) -> Option<&'e Expr> {
108     members_after_last_spread(fields)
109         .iter()
110         .rev()
111         .find(|(k, _)| k == key)
112         .map(|(_, v)| v)
113 }
```


Un membre écrit *avant* un spread a pu être écrasé par ce spread : il « ne répond rien » (None), plutôt que de répondre une valeur peut-être fausse. Et `rev()` retient la *dernière* écriture d’une clé dupliquée, comme JavaScript. La fonction est partagée par l’interpréteur (`obj_members`) et par la poursuite de liaisons du moteur (`engine::seeds`), pour que les deux lecteurs ne divergent pas. Dans le même esprit, `MUTATING_METHODS` (`src/ir/expr.rs`, l. 121–135) est la liste *unique* des méthodes qui mutent leur receveur (`push`, `splice`, `sort`, `set...`), partagée par *state-mutation*, la garde de pureté et la dépendance de rendu.

6.3.5 Le parcours canonique

Les analyses de l’IR sont des parcours récursifs d’expressions. Chacun traite spécialement quelques variants (une variable, une fonction) et descend structurellement dans les autres. Le risque est connu : un parcours qui écrit `_ => {}` pour « tout le reste » absorbe silencieusement un variant ajouté plus tard comme « sans enfants », et perd tout ce qui est dessous. Pour une analyse sound, c’est un faux négatif en attente. L’IR impose donc un point unique :

Extrait 6.13 – `src/ir/expr.rs`, l. 463–511 — l’énumération canonique des enfants directs

```

463 pub fn for_each_child<'a>(&'a self, f: &mut impl FnMut(&'a Expr)) {
464     match self {
465         Expr::Lit(_)
466         | Expr::Var(_)
467         | Expr::FnLit { .. }
468         | Expr::StateVal(_)
469         | Expr::StateSetter(_)
470         | Expr::MemoVal(_)
471         | Expr::CallbackVal(_)
472         | Expr::HookMarker(..)
473         | Expr::SummaryVal(_) => {}
474     Expr::ObjectLit { fields, .. } => {
475         for (_, v) in fields {
476             f(v);
477         }
478     }
479     Expr::ArrayLit { elems, .. } => {
480         for e in elems {
481             f(e);
482         }
483     }
484     Expr::FieldAccess { obj, .. } => f(obj),
485     Expr::IndexAccess { arr, idx } => {
486         f(arr);
487         f(idx);
488     }
489     Expr::BinOp { lhs, rhs, .. } => {
490         f(lhs);
491         f(rhs);
492     }
493     Expr::UnaryOp { arg, .. } => f(arg),
494     Expr::Call { fn_, args } | Expr::New { fn_, args, .. } => {
495         f(fn_);
496         for a in args {
497             f(a);
498         }
499     }
500     Expr::CompApp { props, .. } => f(props),
501     Expr::NativeElem {
502         props, children, ..
503     } => {

```

```

504         f(props);
505         for c in children {
506             f(c);
507         }
508     }
509     Expr::TSAnnotated(inner) => f(inner),
510 }
511 }

```

Invariant 6.2: Pas de bras `_` dans l'énumération des enfants

`Expr::for_each_child` énumère les enfants directs de chaque variant par un `match` exhaustif, sans bras `_`. Les parcoureurs n'écrivent que les bras qu'ils traitent spécialement et délèguent le reste à `for_each_child`. Ajouter un variant à `Expr` casse alors la compilation à une vingtaine d'endroits (mesure citée dans le commentaire, `src/ir/expr.rs`, l. 385–402), tous guidés par le compilateur, et aucun parcourer ne le traite mal en silence.

Deux conséquences. D'abord, `FnLit` est une *feuille* : son corps est un CFG, pas une sous-expression, et franchir la frontière de fonction est une décision de chaque parcourer. Ensuite, le même principe est appliqué ailleurs : `HookEntry::body_cfg` (section 6.4) énumère lui aussi ses variants un par un.

Piège

Le commentaire qui décrit `for_each_child` (`src/ir/expr.rs`, l. 385–402) est placé au-dessus de `subscription_listener`, et enchaîne sans ligne vide sur la documentation de celle-ci : `rustdoc` attache donc l'ensemble à `subscription_listener`, et `for_each_child` n'a pas de documentation propre dans la doc générée. Lisez le source.

Le compagnon de `for_each_child` au niveau d'un corps entier est `CFG::for_each_expr` (`src/ir/cfg.rs`, l. 67–109) : il applique une fonction à chaque expression *de premier niveau* — membres droits des instructions, objet, index et valeur d'un `MemberWrite`, expressions des terminateurs `Return` et `Branch` — en visitant les blocs par identifiant croissant, orphelins compris. La visite des terminateurs n'est pas un détail : un ancien défaut les ignorait, et une variable lue seulement dans une condition n'était pas libre (`test src/engine/fixpoint.rs`, l. 2734). La variante `for_each_expr_where` filtre les instructions visitées ; elle sert à ignorer un `let` d'alias (section 6.8).

Parmi les autres méthodes de `Expr` : `is_call_free` (`src/ir/expr.rs`, l. 516–528) est faux dès qu'apparaît un `Call`, un `New`, un `CompApp` ou un `NativeElem` (le commentaire omet ce dernier, le code l'inclut) ; `peel_ts` retire les annotations ; `describe` (`src/ir/expr.rs`, l. 432–461) fabrique une description courte pour un message, en lisant les racines par `source_name` pour ne jamais montrer un suffixe de greffe.

6.4 Les hooks dans l'IR

6.4.1 Labels positionnels et slots

Rappel React : l'identité d'un hook est son rang

React ne connaît pas le nom des variables d'état : il associe le *i*-ème appel de hook d'un rendu à la *i*-ème cellule de la fibre. C'est la raison d'être de la règle des hooks (pas d'appel conditionnel, pas d'appel dans une boucle) : l'ordre d'appel doit être le même à chaque rendu (chapitre 2).

L'IR reproduit exactement ce modèle. L'extraction des hooks (`extract_hooks`, chapitre 7) parcourt le CFG et numérote les appels dans l'ordre de parcours; `extract_handlers` puis `extract_subscriptions` poursuivent la numérotation (`src/lowering/mod.rs`, l. 465–470), si bien que les handlers ont des labels *après* tous les hooks React. Dans le compteur, `useState` a le label 0 et le handler le label 1.

Définition 6.3: Slot, label de hook

Le *label* d'un hook (`HookLabel`) est son rang d'extraction dans le corps d'un composant. Un *slot* est l'emplacement d'état créé par un `useState` (ou un `useReducer`), identifié par le label de ce hook : `StateVal(1)` lit sa valeur courante, `StateSetter(1)` est son setter. Hors de son composant, un slot se nomme par `QualifiedSlot = (ComponentId, HookLabel)`.

Le label est local : deux composants ont chacun un slot 0. Quand un hook custom est greffé dans un composant, ses labels sont décalés à la suite de ceux de l'appelant — c'est la traduction directe de ce que fait React, pour qui les hooks d'un hook custom *sont* ceux de son appelant, dans l'ordre (chapitre 8).

6.4.2 La table `HookEntry`

Extrait 6.14 – `src/ir/hooks.rs`, l. 247–313 — une ligne de la table des hooks

```

247 #[derive(Debug, Clone)]
248 pub enum HookEntry {
249     State {
250         label: HookLabel,
251         init: Expr,
252         span: Option<SourceRange>,
253     },
254     Effect {
255         label: HookLabel,
256         body_cfg: CFG,
257         deps: DpsArg,
258         span: Option<SourceRange>,
259     },
260     Memo {
261         label: HookLabel,
262         body_cfg: CFG,
263         /// `None` when the deps argument is absent or unreadable. React makes
264         /// the argument mandatory in practice, but the IR must not invent an
265         /// empty list for one it could not parse.
266         deps: DpsArg,
267         span: Option<SourceRange>,
268     },
269     Callback {
270         label: HookLabel,
271         body_cfg: CFG,
272         /// Parameters of the memoized function. Unlike effect/memo bodies
273         /// (zero-arg), a `useCallback` fn takes arguments; its params must be
274         /// subtracted from the body's free variables or they read as captures
275         /// of any same-named outer binding (`(options) => ...` shadowing a
276         /// component-scope `options`).
277         params: Vec<Var>,
278         /// See [ `HookEntry::Memo` ]'s `deps`.
279         deps: DpsArg,
280         span: Option<SourceRange>,
281     },
282     Ref {
283         label: HookLabel,

```

```

284     init: Expr,
285     span: Option<SourceRange>,
286   },
287   Custom {
288     label: HookLabel,
289     name: Symbol,
290     args: Vec<Expr>,
291     deps: DepsArg,
292     /// Variable in the caller's render CFG that receives the hook's return value.
293     binding: Option<Var>,
294     /// NPM package the hook was imported from, if determinable at parse
295     /// time (`"@tanstack/react-query"`). Retained even when a self-aliasing
296     /// tsconfig path also resolves the package to a local file, so
297     /// `SummaryRegistry` package scoping survives. `None` when the hook is
298     /// defined locally or came through a relative specifier.
299     import_source: Option<String>,
300     /// File the hook's definition resolved to via `ImportResolver` –
301     /// relative or aliased specifier – or the current file for a local
302     /// definition. `None` for unresolved (plain npm) imports.
303     resolved_file: Option<PathBuf>,
304     span: Option<SourceRange>,
305   },
306   Handler {
307     label: HookLabel,
308     /// DOM event name without the "on" prefix, lowercased: "click", "change", "submit"...
309     event: String,
310     body_cfg: CFG,
311     span: Option<SourceRange>,
312   },
313 }

```

Tous les variants portent `label` et `span` (le site d'appel ; `None` si le hook n'est atteint que par un `return`, #140). Les autres champs :

- **State** `init` : l'expression d'initialisation. Ce n'est pas un CFG ; un initialiseur paresseux `useState(() => ...)` est une `FnLit` dans `init`.
- **Effect** `body_cfg`, `deps` : `useEffect`, mais aussi `useLayoutEffect` et `useInsertionEffect`, réunis. Pour les bugs visés (boucles d'état, captures périmées), le moment exact d'exécution ne change pas le point fixe ; la distinction reste lisible dans la provenance.
- **Memo, Callback** : le corps est *sorti* du rendu, qui ne garde que `MemoVal(1)/CallbackVal(1)`. `Callback.params` existe parce qu'une fonction mémorisée prend des arguments, qu'il faut retirer de ses variables libres sous peine de les lire comme des captures d'une liaison homonyme. Le commentaire de `Memo.deps` parle encore d'un `None` : il est antérieur au passage à `DepsArg`.
- **Ref** `init` : `useRef(init)`.
- **Custom** : tout appel de hook que le lowering ne modélise pas — hook utilisateur, hook de paquet, et même hooks de React non modélisés comme `useContext`. `import_source` garde le spécificateur npm brut (clé de la table de résumés), `resolved_file` le fichier de définition (clé du registre des hooks, avec le nom). Le champ `binding` est écrit par le lowering mais n'est lu nulle part en aval au commit étudié (le dossier source l'a vérifié par `grep`) : c'est un champ mort, que la greffe ne renomme d'ailleurs pas.
- **Handler** `event`, `body_cfg` : un pseudo-hook, point d'entrée événementiel. Il vient des props JSX (`onX=...`, et `ref` sur un élément natif ; toute prop fonctionnelle d'un composant) ou d'un `addEventListener` dans un corps d'effet. Il n'a pas de ligne de provenance.

Tableau 6.3 résume ce que devient chaque appel, d'après `make_hook_entry` et `hook_result_expr`

(src/lowering/hook_extractor.rs, l. 642–788). Le nom considéré est toujours le nom d'*origine* (import useEffect as ue est un useEffect), et l'appartenance à React vient de la résolution des imports (chapitre 5).

Appel (origine)	Ligne HookEntry	Ce que lit la liaison
hook non React	Custom	HookMarker(1, Unknown)
useState(init)	State (init absent : Lit(Unit))	StateVal(1), StateSetter(1)
useReducer(r, init)	State (le réducteur est ignoré)	StateVal(1)
useEffect et variantes	Effect	HookMarker(1, Undefined)
useMemo(fn, deps)	Memo	MemoVal(1)
useCallback(fn, deps)	Callback	CallbackVal(1)
useRef(init)	Ref (init absent : Lit(Null))	HookMarker(1, StableRef)
autre use* de React	Custom (import_source: "react")	HookMarker(1, Unknown)

TABLE 6.3 – Classification d'un appel de hook au lowering.

Le corps d'un effet ou d'un memo est fixé par hook_body_cfg (src/lowering/hook_extractor.rs, l. 809–835) : une FnLit donne son propre CFG ; tout autre argument (une variable, un membre, un appel) donne un bloc unique ExprStmt(Call fn_: arg, args: []); return undefined — le hook *invoque* son argument, le corps est donc exactement cette invocation. L'ancien repli était un CFG Unreachable, c'est-à-dire $\perp$ au lieu de $\top$: useEffect(handler) sortait propre, et même certifié.

La projection body_cfg applique la discipline de invariant 6.2 :

Extrait 6.15 – src/ir/hooks.rs, l. 341–357 — le corps d'une ligne, sans bras par défaut

```

341  /// The body CFG of hook kinds that have one (`Effect`, `Memo`, `Callback`,
342  /// `Handler`); `None` for `State`/`Ref`/`Custom`.
343  ///
344  /// Every variant is spelled out on purpose. A `_ => None` arm here reads as
345  /// "this kind has no body", which is exactly what a new body-bearing
346  /// variant would silently inherit: its statements would then be invisible
347  /// to every caller of this function, with no compiler error anywhere. The
348  /// exhaustive match makes adding such a variant a build failure instead.
349  pub fn body_cfg(&self) -> Option<&CFG> {
350      match self {
351          HookEntry::Effect { body_cfg, .. }
352          | HookEntry::Memo { body_cfg, .. }
353          | HookEntry::Callback { body_cfg, .. }
354          | HookEntry::Handler { body_cfg, .. } => Some(body_cfg),
355          HookEntry::State { .. } | HookEntry::Ref { .. } | HookEntry::Custom { .. } => None,
356      }
357  }
```

Piège

Un Handler n'a pas de paramètres. Quand onClick=onPick désigne un useCallback((opts) => opts.id, deps), la ligne Handler reçoit une copie du corps du callback, mais le variant n'a pas de champ params : dans l'exemple de la section 6.8, la sonde imprime Handler #4 event=click avec le corps return opts.id, où opts était le paramètre du callback. Le moteur exécute ce corps dans l'environnement de sortie du rendu (src/engine/fixpoint.rs, l. 453–481) et calcule ses variables libres sans retrait (src/engine/fixpoint.rs, l. 1498–1521) : le paramètre est lu comme la prop homonyme du composant, au lieu de $\top$. Même situation pour une flèche inline (n) => setN(n). C'est un constat nouveau, propre à la rédaction de ce chapitre, absent du dossier source. Rejoué sur deux variantes d'un composant à deux

slots d'état (/tmp/ch06/hparam2.tsx, où le paramètre `n` du handler porte le même nom qu'un slot du composant, et /tmp/ch06/hparam3.tsx, où il s'appelle `e`), `reactant check` rapporte le *même* `WARNING infinite-loop` dans les deux cas : sur ces deux cas, la confusion de `n` avec le slot homonyme ne change aucun diagnostic. Un faux négatif ou un faux positif demeure possible sur un autre programme, mais n'est pas démontré au e67b10a.

6.4.3 Marqueurs : ancrer un hook sans valeur

Un `useEffect` ne produit aucune valeur suivie ; pourtant son site d'appel doit rester dans le CFG. La règle `conditional-hook` demande si ce site domine toutes les sorties (définition 3.9), et la greffe d'un hook custom a besoin de retrouver le bloc où il était appelé. C'est le rôle de `HookMarker(1, v)` : dans le rendu du liste 6.1, rien ne l'illustre, mais l'exemple de la section 6.8 en montre deux (`HookMarker(0, Undefined)`). La seconde composante dit ce que la *liaison* lit :

Extrait 6.16 – `src/ir/expr.rs`, l. 302–331 — ce que lit une liaison de marqueur

```

302 /// What a [Expr::HookMarker`] binding reads as.
303 ///
304 /// The distinction is decided at lowering, where "is this React's own hook?"
305 /// is still known: a React hook with no tracked result really does return
306 /// `undefined`, while an unresolved custom hook returns something the engine
307 /// has no information about. Collapsing the two – as a single `undefined`
308 /// once did – makes an opaque hook's return *provably stable*
309 /// (`to_stability` joins `Stable` for `undef`), which silences every
310 /// stability-gated rule on it: a false negative, and the forbidden direction.
311 #[derive(Debug, Clone, PartialEq, Eq)]
312 pub enum MarkerVal {
313     /// Reads as `undefined` – React's own value-less hooks (an effect returns
314     /// nothing).
315     Undefined,
316     /// Reads as a reference whose identity is constant across renders –
317     /// `useRef`. Reading it as `Undefined` instead was *stable enough* for the
318     /// deps rules (`to_stability` joins `Stable` for `undef` too), but it threw
319     /// the identity away: the container is a reference, and every rule that
320     /// reasons about references rather than about stability saw nothing there.
321     StableRef,
322     /// Reads as `T` – a custom hook the engine could neither inline nor
323     /// summarize. Paired with the `analysis-limit/unknown-hook` Info.
324     Unknown,
325     /// Reads as the library hook's [`HookSummary`](crate::registry::HookSummary).
326     /// Retagged onto the marker by `expand_custom_hooks` rather than replacing
327     /// it: overwriting the marker with a bare `SummaryVal` erased the label,
328     /// and with it the call site every rules-of-hooks check needs – a
329     /// conditional `useAtom()` was invisible to `conditional-hook`.
330     Summary(SummaryValue),
331 }
```

Décision : la valeur d'un marqueur se décide au lowering

Le commentaire de tête donne l'argument : c'est au lowering qu'on sait encore si le hook est celui de React. Un hook de React sans résultat suivi renvoie vraiment `undefined` ; un hook custom non résolu renvoie quelque chose dont le moteur ne sait rien. Confondre les deux — comme le faisait un unique `undefined` — rendait le retour d'un hook opaque *prouvablement stable*, et faisait taire toutes les règles gardées par la stabilité : un faux négatif. D'où quatre valeurs : `Undefined` seulement pour un Effect, `StableRef` pour un Ref, `Unknown` pour tout le reste, y compris `useContext` ou `useId` (`src/lowering/hook_extractor.rs`, l. 766–772) ; `Summary`

est posé plus tard par le moteur.

Le moteur *re-étiquette* (« retag ») le marqueur d'un hook de bibliothèque connu : `retag_marker` (`src/engine/fixpoint.rs`, l. 1247–1275) remplace `Unknown` par `Summary(...)` plutôt que de remplacer le marqueur par un `SummaryVal` nu. Le commentaire de `Summary` rappelle pourquoi : écraser le marqueur effaçait le label, donc le site d'appel dont toute vérification des règles des hooks a besoin — un `useAtom()` conditionnel était invisible à `conditional-hook`. Les résumés eux-mêmes (`SummaryValue`, `src/ir/expr.rs`, l. 333–382 : `Top`, `StableRef`, `UnstableRef`, `Wrapper`, `Shape`, `Held`, `Navigator`) vivent dans l'IR pour éviter une dépendance circulaire entre `ir` et `domains` ; leur sens abstrait est donné au chapitre 11.

6.4.4 Les tableaux de deps

Rappel React : comment React compare les deps

Pour un effet, un memo ou un callback, React compare *chaque élément* du tableau de deps avec `Object.is` entre deux rendus. Sans tableau, l'effet s'exécute après chaque rendu. Un `spread [...rows]` fait comparer `rows[0]`, `rows[1]`, ..., jamais `rows` lui-même.

Trois situations sont donc à distinguer, et l'exemple suivant les réunit.

Extrait 6.17 – Quatre formes de deps (`/tmp/ch06/exo.tsx`)

```
1 export function Exo({ a, b, deps }: { a: number; b: number[]; deps: any[] }) {
2   useEffect(() => { console.log(a); }, [a, , ...b]);
3   useEffect(() => { console.log(a); }, deps);
4   useEffect(() => { console.log(a); });
5   useEffect(() => { console.log(a); }, [a] as const);
6   return null;
7 }
```

```
hook: Effect #0 deps=List[a, b] arity=AtLeast(2) spread_at=[1] @fFileId(0):4:2
hook: Effect #1 deps=Opaque @fFileId(0):5:2
hook: Effect #2 deps=Absent @fFileId(0):6:2
hook: Effect #3 deps=List[a] arity=Exact(1) spread_at=[] @fFileId(0):7:2
```

Extrait 6.18 – `src/ir/hooks.rs`, l. 88–107 — les trois états de l'argument deps

```
88 /// The deps argument of a hook call, as the IR could read it.
89 ///
90 /// The three states are three different facts, and folding any two of them
91 /// together has cost findings in both directions:
92 ///
93 /// - [DepsArg::Absent] – no argument at all. The hook re-runs on every
94 ///   render, so nothing it captures can go stale.
95 /// - [DepsArg::Opaque] – an argument the IR cannot read (`useMemo(fn, deps)`).
96 ///   The hook **is** gated by a list, so its captures can go stale exactly like
97 ///   a declared-but-incomplete array; the engine simply cannot see one element
98 ///   of it. Reading this as `Absent` skips the hook; reading it as an empty
99 ///   list claims it declares nothing. Both are wrong, in opposite directions.
100 /// - [DepsArg::List] – a written array literal, with whatever the lowering
101 ///   could keep of it.
102 #[derive(Debug, Clone)]
103 pub enum DepsArg {
104     Absent,
105     Opaque,
106     List(DepsList),
107 }
```

Définition 6.4: Argument deps

L'argument `deps` d'un hook est lu dans l'un de trois états : `Absent` (aucun argument : le hook se réexécute à chaque rendu, rien de ce qu'il capture ne peut être périmé) ; `Opaque` (un argument que l'IR ne sait pas lire, par exemple une variable : le hook *est* gardé par une liste, mais le moteur n'en voit pas les éléments) ; `List(1)` (un littéral de tableau écrit, avec ce que le lowering a pu en garder).

Le commentaire insiste : replier deux de ces états l'un sur l'autre « a coûté des findings dans les deux sens ». Lire `Opaque` comme `Absent` fait sauter le hook par les règles de `deps` (FN) ; le lire comme une liste vide prétend qu'il ne déclare rien (FP massif). `from_expr` (`src/ir/hooks.rs`, l. 109–126) pèle d'abord les annotations : `[a]` `as const` est un littéral de tableau (effet 3 ci-dessus). Et `is_declared` (`src/ir/hooks.rs`, l. 145–150) répond vrai pour `Opaque` : l'appelant a écrit un tableau, dire le contraire afficherait « cet effet se réexécute après chaque rendu » sur un hook gardé.

Extrait 6.19 – `src/ir/hooks.rs`, l. 160–178 — un tableau de `deps` écrit

```
160 /// A written dependency array, with whatever lowering could keep of it.
161 ///
162 /// `elems` holds the elements the IR can see: an elision contributes none, and
163 /// a spread contributes its *source* as one element standing for however many
164 /// it holds. `arity` says how long the source array actually is.
165 ///
166 /// Enumerating `elems` is sound whenever the enumeration makes a rule **fire** –
167 /// each element over-approximates what it stands for. It is NOT sound when the
168 /// enumeration makes a rule *stop*: crediting `[...rows]` as declaring `rows`
169 /// suppresses a stale-capture finding React itself would report, because React
170 /// compares `rows[0]`, `rows[1]`, ... and never `rows`.
171 #[derive(Debug, Clone)]
172 pub struct DepsList {
173     pub elems: Vec<Expr>,
174     pub arity: Arity,
175     /// Positions in `elems` that came from a spread – see
176     /// [`DepsList::covering`], which is the reason the field exists.
177     pub spread_at: Vec<usize>,
178 }
```

`elems` contient les éléments visibles : une élision n'en fournit aucun, un spread fournit sa *source* comme un seul élément. `arity` dit combien d'entrées le tableau source contient vraiment :

Extrait 6.20 – `src/ir/hooks.rs`, l. 44–58 — l'arité d'un tableau écrit

```
44 /// How many entries a written deps array has.
45 ///
46 /// Elisions and spreads are not the same kind of ignorance, and collapsing them
47 /// cost real findings. An elision drops an element from `elems` but the source
48 /// array's length is still perfectly countable at lowering (`[a, , b]` has
49 /// three entries), so the arity stays **exact**. Only a spread is open-ended:
50 /// `[a, ...rest]` holds one entry plus however many `rest` does, which is a
51 /// lower bound and nothing more.
52 #[derive(Debug, Clone, Copy, PartialEq, Eq)]
53 pub enum Arity {
54     /// The source array holds exactly this many entries.
55     Exact(usize),
56     /// The source array holds at least this many; a spread supplies the rest.
57     AtLeast(usize),
58 }
```

Dans l'effet 0, `[a, , ...b]` a trois composantes : `a` compte pour un, l'élision pour un (la

longueur reste *comptable*), le spread pour « au moins zéro ». D'où `elems = [a, b]`, `AtLeast(2)` et `spread_at = [1]`.

Arité est un petit treillis. Ordonnons les arités par inclusion de leurs concrétisations : $\gamma(\text{Exact}(n)) = n$ et $\gamma(\text{AtLeast}(n)) = m \mid m \geq n$. Alors $\text{Exact}(n) \sqsubseteq \text{AtLeast}(m)$ ssi $m \leq n$, et $\text{AtLeast}(n) \sqsubseteq \text{AtLeast}(m)$ ssi $m \leq n$; `AtLeast(0)` est le top (figure 6.2). La méthode `may_be` (`src/ir/hooks.rs`, l. 76–85) est exactement le test d'appartenance $n \in \gamma(a)$, et `at_least` le minorant garanti, qui permet encore à une liste ouverte de *réfuter* une affirmation d'arité. Le code ne définit ni `join` ni `meet` : l'arité est un fait syntaxique d'un site, jamais fusionné. Si l'on en voulait un, $\text{Exact}(a) \sqcup \text{Exact}(b) = \text{AtLeast}(\min(a, b))$ pour $a \neq b$.

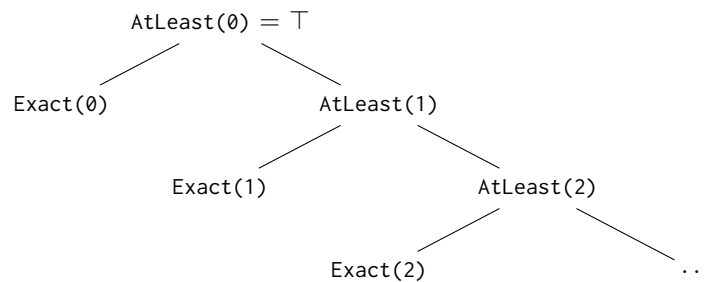


FIGURE 6.2 – Diagramme de Hasse de Arity ordonné par concrétisation. Chaque `Exact(n)` est minimal ; la chaîne des `AtLeast` descend sans fin.

Lire pour tirer, lire pour taire. Reste la question qui justifie `spread_at` :

Extrait 6.21 – `src/ir/hooks.rs`, l. 192–215 — les entrées qui couvrent une lecture

```

192  /// The entries that actually cover a read – every visible element
193  /// except a spread's source.
194  ///
195  /// This is the line between the two ways a rule uses a deps list. Reading
196  /// `elems` to make a rule fire is sound however the list was truncated,
197  /// because each element over-approximates what it stands for. Reading it
198  /// to make a rule stop is not: React compares `rows[0], rows[1], ...` and
199  /// never `rows`, so crediting a flattened `[...rows]` as declaring `rows`
200  /// suppresses a stale capture React itself reports. The elements written
201  /// beside the spread still cover their own reads, which is why this
202  /// filters rather than refusing the whole list.
203  pub fn covering(&self) -> std::borrow::Cow<'_, [Expr]> {
204      if self.spread_at.is_empty() {
205          return std::borrow::Cow::Borrowed(&self.elems);
206      }
207      std::borrow::Cow::Owned(
208          self.elems
209              .iter()
210              .enumerate()
211              .filter(|(i, _)| !self.spread_at.contains(i))
212              .map(|(_, e)| e.clone())
213              .collect(),
214      )
215  }
  
```

Invariant 6.3: Lire pour tirer, lire pour taire

Énumérer *elems* pour faire *tirer* une règle est sound quelle que soit la troncature, car chaque élément sur-approxime ce qu'il représente. L'énumérer pour faire *taire* une règle ne l'est pas : React compare `rows[0]`, `rows[1]`, ... et jamais `rows`, si bien que créditer `[...rows]` comme déclarant `rows` supprimerait une capture périmée que React lui-même rapporte. Une suppression passe donc toujours par `covering()`, qui retire les sources de spread et garde les éléments écrits à côté.

C'est ce que fait le moteur : l'épinglage des lectures (section 6.8) est calculé sur `deps.covering()` (`src/engine/fixpoint.rs`, l. 1435), et `missing-deps` déclare ses chemins à partir de la même couverture (`src/rules/impls/missing_deps.rs`, l. 62). Les passes IR vers IR (décalage, renommage, substitution) traversent les `deps` par `map_exprs`, qui réécrit chaque élément en conservant `arity` et `spread_at` (`src/ir/hooks.rs`, l. 224–245) : ce que le source contenait ne change pas.

6.4.5 La provenance

`HookEntry` dit ce que le moteur *modélise* (`useLayoutEffect` devient `Effect`). Une seconde table dit ce qui a été *prouvé* de l'identité de chaque appel :

Extrait 6.22 – `src/ir/hooks.rs`, l. 10–42 — une ligne de provenance

```

10 /// Provenance of one hook call: `label → (origin hook, source, direct|inlined)`.
11 ///
12 /// `HookEntry` records what the engine *models* (`useLayoutEffect` collapses
13 /// into `Effect`, an aliased import into its origin's entry kind); this row
14 /// records where the call's identity was *proven*, and survives
15 /// `expand_custom_hooks` so a component that reaches `useLayoutEffect` only
16 /// through an inlined wrapper is distinguishable from one calling it directly.
17 #[derive(Debug, Clone, PartialEq, Eq)]
18 pub struct HookProvenance {
19     pub label: HookLabel,
20     /// Name the origin defines the hook under (`useLayoutEffect`, `useData`) –
21     /// the *imported* name for an aliased import, never the local alias.
22     pub origin_hook: Symbol,
23     /// `true` iff the call was classified as React's own hook.
24     pub react: bool,
25     /// Raw import specifier at the call's import site (`"zustand"` even when a
26     /// self-aliasing tsconfig path resolves it to a local file). `None` for a
27     /// local definition or an unimported name.
28     pub specifier: Option<String>,
29     /// File the hook's definition resolved to; the current file for a local
30     /// definition.
31     pub file: Option<PathBuf>,
32     /// `false` = written in the component itself; `true` = reached through an
33     /// inlined custom hook.
34     pub inlined: bool,
35     /// Call-site span, pointing into the file the row was lowered from (for
36     /// an inlined row, the custom hook's own file – ADR-024 renders the
37     /// origin). Provenance-anchored findings need it because the row's label
38     /// can dangle: `expand_custom_hooks` keeps the wrapper call's direct row
39     /// but splices its `HookEntry` away, so there is no `hook_calls` row left
40     /// to join back to for a `SourceRange` (ADR-027 §7).
41     pub span: Option<SourceRange>,
42 }

```

Dans le compteur, la ligne unique dit : label 0, hook d'origine `useState`, de React, importé

de "react", écrit directement dans le composant. Trois remarques. La table survit à l'expansion des hooks custom : chaque ligne d'un hook greffé y est fusionnée, labels décalés et `inlined: true`, ce qui distingue un composant qui atteint `useLayoutEffect` par un wrapper d'un composant qui l'appelle directement (ADR-23). La ligne du wrapper lui-même reste, alors que sa ligne `HookEntry` a été remplacée par le corps greffé : son label *pend*, d'où le champ `span`, qui permet encore de la situer (ADR-27 §7). Enfin, les handlers n'ont pas de ligne de provenance (le compteur a deux hooks et une seule ligne).

6.5 Les unités de l'IR et les faits de module

Le lowering produit trois sortes d'unités, une par classe de fonction détectée (définition 5.1), plus des faits par fichier.

6.5.1 Composant, hook custom, utilitaire

Extrait 6.23 – `src/ir/component.rs`, l. 59–83 — l'IR d'un composant

```

59 #[derive(Debug, Clone)]
60 pub struct ComponentIR {
61     /// Source file this component was lowered from. Used as part of the
62     /// `(file, name)` registry key so two components named `Page` in
63     /// different files don't collide.
64     pub file: PathBuf,
65     pub name: Symbol,
66     pub param: Var,
67     /// Names of props whose TypeScript type is a DOM interface
68     /// (`HTMLCanvasElement`, `SVGElement`, `Node`...). Mutating these is
69     /// imperative DOM manipulation, not a write into React-owned data –
70     /// the state-mutation rule exempts them.
71     pub dom_props: Arc<HashSet<Var>>,
72     pub render_cfg: CFG,
73     pub hooks: Vec<HookEntry>,
74     /// Provenance row per hook call in `hooks` (ADR-023 step 1):
75     /// `label → (origin hook, source, direct|inlined)`. Grows during
76     /// analysis as custom hooks are expanded.
77     pub hook_provenance: Vec<crate::ir::hooks::HookProvenance>,
78     /// Module-level `const` bindings of the source file with syntactically
79     /// known kinds, keyed by name. Function-valued consts (arrow/function
80     /// expressions) are excluded: those are components/utilities/handlers
81     /// with their own machinery.
82     pub module_consts: Arc<HashMap<Var, ModuleConstInit>>,
83 }
```

Le chapitre 5 a montré comment ces champs sont remplis ; rappelons leur sens pour l'analyse.

- `file` et `name` forment la clé `(file, name)` des registres (ADR-13) et l'origine `CompOrigin` d'où l'on interne l'identité (section 6.6).
- `param` est *un seul* nom : celui du premier paramètre s'il est un identifiant, "props" si le composant n'en a pas (`src/lowering/mod.rs`, l. 471–474), et un temporaire `__p0` s'il est déstructuré ; le rendu commence alors par `let __obj_N = __p0` suivi d'un `let` par propriété.
- `render_cfg` est le corps, hooks remplacés par leurs valeurs ; `hooks` et `hook_provenance` sont les deux tables de la section 6.4.
- `dom_props` : les props typées comme interfaces DOM, que `state-mutation` exempte (muter un `HTMLCanvasElement` n'est pas écrire dans des données de React).

- `module_consts` : les `const` de module dont le *genre* est syntaxiquement certain, partagées (Arc) par tous les composants du fichier.

Un hook custom et un utilitaire ont chacun leur unité :

Extrait 6.24 – `src/ir/hook_ir.rs`, l. 9–24 — l'IR d'un hook custom

```

9  /// Lowered representation of a user-defined custom hook function.
10 /// Analogous to `ComponentIR` but for `use*` functions.
11 #[derive(Debug, Clone)]
12 pub struct HookIR {
13     /// Source file this hook was lowered from.
14     pub file: PathBuf,
15     pub name: Symbol,
16     pub params: Vec<Var>,
17     pub body_cfg: CFG,
18     /// Hook calls declared inside this hook's body (useState, useEffect, etc.).
19     pub hooks: Vec<HookEntry>,
20     /// Provenance row per hook call in `hooks` (ADR-023 step 1). Merged –
21     /// labels remapped, marked `inlined` – into each consumer that expands
22     /// this hook.
23     pub hook_provenance: Vec<HookProvenance>,
24 }
```

Extrait 6.25 – `src/ir/function_ir.rs`, l. 15–22 — l'IR d'un utilitaire

```

15 #[derive(Debug, Clone)]
16 pub struct FunctionIR {
17     /// Source file this function was lowered from.
18     pub file: PathBuf,
19     pub name: Symbol,
20     pub params: Vec<Var>,
21     pub body_cfg: CFG,
22 }
```

Les différences structurelles traduisent la règle des hooks. Un `HookIR` a plusieurs paramètres, une table de hooks et une provenance : ses hooks deviendront ceux de l'appelant. Un `FunctionIR` n'a ni l'une ni l'autre, parce qu'un utilitaire (une fonction dont le nom ne commence pas par `use`) n'appelle pas de hook. Un `ComponentIR` n'a qu'un paramètre, les props. Les deux premières unités servent de cibles à la greffe (chapitre 8) ; un composant enfant, lui, n'est jamais greffé : il est analysé avec des props abstraites (ADR-12, chapitre 16).

Piège

Le commentaire de tête de `function_ir.rs` cite `doOrNot(setX(...))` comme cible d'inlining. Au commit étudié, seules les instructions `let x = f(...)` et `f(...)` ; dont le callee est une variable sont inlinées (`src/engine/fixpoint.rs`, l. 1698–1718) ; un appel en position d'expression ne l'est pas (#52).

6.5.2 Constantes de module et contextes

Exemple 6.2: Une constante de module est stable

Dans `const EMPTY = []` ; `function List() useEffect(..., [EMPTY])` ; , le tableau est créé une fois, au chargement du module : sa référence est la même à chaque rendu, et le `dep` ne change jamais. À l'inverse, `const x = f()` ; peut être n'importe quoi.

Extrait 6.26 – `src/ir/component.rs`, l. 12–41 — ce que l'on sait d'un initialiseur de `const` de module

```

12 /// What is known about a module-level `const` initializer.
13 ///
14 /// Only initializers whose JS *kind* is syntactically certain are collected:
15 /// the product value domain expresses "constant across renders" as either an
16 /// exact primitive or a `Stable` reference slot – an opaque value of unknown
17 /// kind (`const X = f()`) has no sound encoding (a wide primitive slot reads
18 /// as "changes per render") and is left out, falling back to  $\top$ .
19 #[derive(Debug, Clone)]
20 pub enum ModuleConstInit {
21     /// Primitive literal: the exact value.
22     Prim(Prim),
23     /// Object/array/new/JSEX literal: a reference, allocated once per module
24     /// lifetime  $\rightarrow$  identity Stable across renders.
25     Ref,
26     /// `createContext(...)` proven to be React's by its import specifier. A
27     /// reference like [Ref`](ModuleConstInit::Ref) – the exception to "opaque
28     /// calls stay  $\top$ " is earned by knowing the callee, hence the kind.
29     ///
30     /// The variant exists because the *role* is what a provider rule needs:
31     /// `X.Provider` is a context provider only if X` is a context, and
32     /// nothing else in the IR can say so. Two-valued on purpose – absence
33     /// means "not proven", never "not a context".
34     ///
35     /// It carries the [ContextId`] of the cell, not just the role (#109): two
36     /// files that import the same context bind it under whatever local name
37     /// they like, and pairing a consumer with a provider needs to know they
38     /// mean the same cell. The resolver already had the origin in hand when it
39     /// marked an imported name; it used to drop it.
40     Context(ContextId),
41 }

```

Le commentaire de tête pose le critère : seuls les initialiseurs dont le genre JavaScript est certain sont collectés, car le domaine produit exprime « constant d'un rendu à l'autre » soit par une primitive exacte, soit par une référence `Stable` (chapitre 10) ; une valeur opaque n'a pas d'encodage sound et reste absente de la table, donc $\top$. Le moteur sème ces constantes dans l'environnement d'entrée : `Prim(p)` donne la valeur exacte, `Ref` et `Context(_)` une référence stable (`src/engine/fixpoint.rs`, l. 152–190).

Le variant `Context` porte l'identité de la cellule de contexte :

Extrait 6.27 – `src/ir/component.rs`, l. 43–57 — l'identité canonique d'un contexte

```

43 /// Canonical identity of a React context cell: the file that called
44 /// `createContext` and the name it bound the result to.
45 ///
46 /// **As deep as the resolution chain, and no deeper.** A context re-exported
47 /// through a third file resolves only one level (#49), so two importers that
48 /// reach the same cell by different depths get different ids – a missed
49 /// pairing, never a wrong one.
50 #[derive(Debug, Clone, PartialEq, Eq, Hash, PartialOrd, Ord)]
51 pub struct ContextId {
52     /// The file whose module scope holds the `createContext` call.
53     pub origin_file: PathBuf,
54     /// The name bound there – the exported name, not the importer's local
55     /// alias.
56     pub origin_name: String,
57 }

```

« Aussi profonde que la chaîne de résolution, et pas plus » : un contexte ré-exporté par un troisième fichier n'est résolu que d'un niveau (#49), si bien que deux importeurs qui atteignent la même cellule par des profondeurs différentes obtiennent deux identifiants — un appariement manqué, jamais un appariement faux. La relation qui apparie fournisseurs et consommateurs est au chapitre 14.

6.5.3 Les faits de module

Extrait 6.28 – `src/ir/module.rs`, l. 21–36 — ce qu'un fichier déclare de lui-même

```
21 /// What one lowered file declares about itself.
22 #[derive(Debug, Clone, Default, PartialEq, Eq)]
23 pub struct ModuleFacts {
24     /// Directive prologue strings, in source order, unquoted: "use client",
25     /// "use strict", ... Only the prologue – a string expression statement
26     /// after the first non-directive statement is not a directive.
27     pub directives: Vec<String>,
28     /// Files this module imports, deduped, in first-seen order. Only edges
29     /// the `ImportResolver` mapped to a real file: npm packages and
30     /// unresolvable aliases leave no edge.
31     ///
32     /// Covers `import`, side-effect `import "./x"`, and the re-export forms
33     /// (`export ... from`, `export * from`), because a barrel re-export carries
34     /// a directive boundary exactly like a plain import does.
35     pub imports: Vec<PathBuf>,
36 }
```

`directives` est le prologue du module ("use client", "use strict"), et seulement le prologue ; `imports` les fichiers importés que le résolveur a pu associer à un vrai fichier, ré-exports compris (un barrel porte une frontière de directive exactement comme un import). La table `ModuleTable` (`src/ir/module.rs`, l. 44–126) associe à chaque chemin abaissé ses faits ; elle est « délibérément non interprétée » (`src/ir/module.rs`, l. 11–16) : le sens de "use client" est donné par l'appelant, à travers un mécanisme unique, `reachable_from(seeds, boundary)`, la fermeture des imports arrêtée à tout module qui déclare la frontière.

Invariant 6.4: Une table vide ne prouve rien

`ModuleTable` est vide quand l'IR est construite à la main (tests unitaires). Toute requête répond alors « inconnu », et chaque appelant doit le lire comme *absence de preuve*, jamais comme preuve du contraire. `any_declares(directive)` est la porte de toute règle dérivée d'une directive : un code qui n'écrit jamais "use client" n'est pas un code Server Components (`server-component-hook`, chapitre 21).

6.6 Les identités internées

6.6.1 Le patron : un identifiant de 4 octets et une table

Deux entités de l'IR ont besoin d'une identité comparable, bon marché et indépendante de tout le reste : les fichiers et les composants. Le projet utilise pour les deux le même patron, l'*internement* : un *newtype* `Copy` de 4 octets, sans constructeur public, fabriqué uniquement par une table qui en garde l'inverse.

Extrait 6.29 – `src/ir/source_range.rs`, l. 3–16 — l'identité d'un fichier et sa table

```
3 /// Identity of a source file, interned in a [ `FileTable` ] (ADR-019).
```

```

4 ///
5 /// 4 bytes and `Copy`, so [`SourceRange`] stays `Copy` while carrying file
6 /// identity – the fix for the ADR-011 limitation where a span inside a
7 /// cross-file inlined CFG could not name the file it points into.
8 #[derive(Debug, Clone, Copy, PartialEq, Eq, Hash, PartialOrd, Ord)]
9 pub struct FileId(u32);
10
11 /// Interning table `FileId ↔ PathBuf`, built once during lowering (one entry
12 /// per parsed file) and carried through to the renderers.
13 #[derive(Debug, Default, Clone)]
14 pub struct FileTable {
15     paths: Vec<PathBuf>,
16 }

```

`FileTable::intern` (`src/ir/source_range.rs`, l. 18–25) renvoie l’identifiant existant ou en crée un nouveau ; c’est une recherche linéaire, sans importance à l’échelle d’un projet. `SourceMap::empty` utilise la valeur réservée `u32::MAX`, qui n’est jamais résolue. L’identité de fichier est née d’une limite d’ADR-11 (une position dans un CFG inliné depuis un autre fichier ne pouvait pas nommer ce fichier) et a été introduite par ADR-19.

Extrait 6.30 – `src/ir/component_id.rs`, l. 23–29 — l’identité d’un composant

```

23 /// A component's identity, interned in a [`ComponentTable`].
24 ///
25 /// `Copy` and 4 bytes, so it sits inside a `BTreeSet` label or a store key
26 /// without the allocation a name costs, and two ids compare in one
27 /// instruction.
28 #[derive(Debug, Clone, Copy, PartialEq, Eq, Hash, PartialOrd, Ord)]
29 pub struct ComponentId(u32);

```

Extrait 6.31 – `src/ir/component_id.rs`, l. 55–75 — la table d’internement des composants

```

55 /// Interning table `ComponentId ↔ (file, name)`, built once from the component
56 /// registry and carried on the analysis result.
57 ///
58 /// It also owns [`Self::display_name`], because minting that string needs the
59 /// collision counts and nothing else does. Keeping the two together is what
60 /// stops a second, disagreeing spelling from being invented elsewhere.
61 #[derive(Debug, Default, Clone)]
62 pub struct ComponentTable {
63     /// A map, not a vector indexed by id: a result analysed before any table
64     /// existed carries [`ComponentId::SYNTHETIC`], and registering that id
65     /// is how such a result joins a program without its interior labels
66     /// having to be rewritten.
67     origins: HashMap<ComponentId, CompOrigin>,
68     by_origin: HashMap<CompOrigin, ComponentId>,
69     next: u32,
70     /// How many files define each bare name – the only input to
71     /// [`Self::display_name`], precomputed because that answer is asked once
72     /// per rendered finding.
73     name_counts: HashMap<String, usize>,
74 }

```

La table est construite une fois par le registre des composants, qui interne chaque clé (`file`, `name`) *en ordre de clé trié* : l’ordre des identifiants est donc reproductible. Elle voyage sur le résultat d’analyse, pour que règles et renderers résolvent contre la table même qui a indexé l’analyse. `ComponentId::SYNTHETIC` (`u32::MAX`) est l’identifiant partagé par les IR construites à la main, que `register` permet d’enregistrer sans le renuméroter.

6.6.2 Le nom affiché n'est pas une identité

Extrait 6.32 – `src/ir/component_id.rs`, l. 138–154 — le nom à *montrer*

```

138  /// The name to *show* for `id`: the bare name when only one file defines
139  /// it, `name@<file>` when several do.
140  ///
141  /// A rendering concern and nothing else. It is content-dependent by
142  /// design – the suffix exists precisely to tell two same-named components
143  /// apart in a report – which is exactly why no table may be keyed by it.
144  /// `--entry` accepts the qualified form, and
145  /// [ `Self::resolve_display_name` ] is its inverse.
146  pub fn display_name(&self, id: ComponentId) -> Option<String> {
147      let origin = self.origin(id)?;
148      let count = self.name_counts.get(&origin.name).copied().unwrap_or(0);
149      Some(if count <= 1 {
150          origin.name.clone()
151      } else {
152          format!("{ }@{ }", origin.name, origin.file.display())
153      })
154  }

```

Exemple 6.3: Un alias et un leurre homonyme

Trois fichiers sous `/tmp/ch06/ident/` :

```

1  // App.tsx
2  import { useState } from "react";
3  import { Widget as Panel } from "../b/Widget";
4
5  export function App() {
6      const [n, setN] = useState(0);
7      return <Panel onChange={setN} value={n} />;
8  }
9  // a/Widget.tsx
10 export function Widget() {
11     return <div>clean</div>;
12 }
13 // b/Widget.tsx
14 export function Widget({ onChange }) {
15     onChange(1);
16     return <div>buggy</div>;
17 }

```

Le rendu de `App` contient `CompApp<Panel>(Obj#0{onChange: setN, value: n})` `origin=Some("b/Widget.tsx::Widget")` : le nom écrit (`Panel`) n'est pas le nom enregistré (`Widget`), et seule l'origine prouvée par l'import permet la résolution. Sans elle, `Widget` serait ambigu (deux fichiers le définissent). La commande `reactant check . -trace -show-clean` donne :

```

App (1 hooks) App.tsx ✓
Widget@b/Widget.tsx (0 hooks) b/Widget.tsx
  error cross-setter-in-render var:onChange (line 2:2) prop `onChange` (a state setter of parent `App`)
    ↳ called during render of `Widget@b/Widget.tsx`, which triggers a parent re-render on every render
    → `onChange` is a state setter, so calling it writes state (line 2:2)

! 1 error(s) across 3 file(s).

```

Le suffixe `@b/Widget.tsx` est frappé au rendu par `display_name`, parce que deux fichiers définissent `Widget`.

Décision : ADR-40 — l'identité est un identifiant interne, le nom affiché est un rendu

Contexte. Un composant était nommé de trois façons : la clé (`PathBuf`, `Symbol`) du registre, le nom affiché (`Name` ou `Name@file`) et le `Symbol` nu de `CompApp`. Après ADR-38 §5, c'est le nom affiché qui indexait l'analyse (stores partagés, étiquettes `Versioned`, `setters`) ; or il dépend du *contenu* du projet : ajouter un fichier sans rapport qui définit un second `Widget` renomme le premier et ré-indexe toutes ces tables. Deux défauts en découlaient, qui finissaient tous deux en « ✓ no issues found » : un enfant `JSX` résolu par le premier nom en ordre de tri (un leurre `a/Widget.tsx` déplaçait le vrai `./b/Widget`), et des chemins orthographiés différemment par la découverte et par le résolveur.

Décision. (1) `ComponentId` interne dans une `ComponentTable`, sur le modèle de `FileId`; (2) le nom affiché est frappé au rendu et rien ne le compare; (3) `CompApp` porte *origin* (fichier et nom exporté); (4) la résolution vit en un seul lieu, `resolve_child`, qui refuse de deviner (`Ambiguous`, avec un `analysis-limit`); (5) une seule orthographe de chemin.

Alternative refusée. Analyser tous les candidats d'un nom ambigu : *sound* et plus précis, mais 1 347 références ambiguës dans le corpus, et la forme même du blocage quadratique de #86.

Trois tests de `component_id.rs` fixent ces propriétés :

- `interning_is_idempotent_and_two_files_are_two_ids`;
- `the_display_suffix_appears_only_on_a_collision`;
- `interning_a_namesake_changes_an_existing_display_name_but_not_its_id` (`src/ir/component_id.rs`, l. 227), qui est *la* raison pour laquelle aucune table n'est indexée par le nom affiché.

Piège

`ComponentTable::register` ne refuse qu'un *identifiant* déjà présent, pas une *origine* déjà internée sous un autre identifiant (`src/ir/component_id.rs`, l. 94–101). Enregistrer sous `SYNTHETIC` une origine déjà internée écraserait l'entrée inverse et compterait deux fois le nom, qui prendrait un suffixe `@file` sans homonyme réel. Aucun appelant ne le fait au commit étudié : les deux appels hors du module enregistrent un seul composant dans une table vide (`src/engine/program_result.rs`, l. 91–99, `src/test_support.rs`, l. 86).

6.7 Les positions

Extrait 6.33 – `src/ir/source_range.rs`, l. 36–42 — une position source

```
36 /// A `(file, line, col)` source position. `line` is 1-indexed, `col` 0-indexed.
37 #[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
38 pub struct SourceRange {
39     pub file: FileId,
40     pub line: u32,
41     pub col: u32,
42 }
```

Malgré son nom, un `SourceRange` est un *point*, pas un intervalle : `(file, line, col)`, ligne comptée à partir de 1, colonne à partir de 0. Le lowering le frappe à partir de l'offset de début d'un nœud `oxc`, par la table des débuts de ligne de la `SourceMap` du fichier (chapitre 5) :

Extrait 6.34 – `src/ir/source_range.rs`, l. 94–102 — d'un offset à une position

```
94 /// Convert a byte offset into a 1-indexed (line, col) pair using a pre-built line-starts table.
95 pub fn offset_to_range(starts: &[u32], offset: u32, file: FileId) -> SourceRange {
96     let line = starts.partition_point(|&s| s <= offset).saturating_sub(1);
```

```

97   SourceRange {
98       file,
99       line: line as u32 + 1,
100      col: offset - starts[line],
101   }
102 }

```

La recherche est un `partition_point` en $O(\log L)$ (et non un `binary_search` comme l'écrit ADR-11). La colonne est une différence d'*octets* UTF-8, puisque les offsets d'`oxc` sont des octets : sur une ligne qui contient des caractères non ASCII avant la position, la colonne affichée est en octets, pas en caractères. Vérifié sur `const s = "café"; const y = 2; ;` é occupant deux octets, la sonde située y en colonne 27, quand un compte par caractères en donnerait 26. `SourceRange` n'implémente pas `Ord`; l'ordre dans un fichier passe par `pos_key`.

Où l'IR porte-t-elle des positions ? Sur chaque `Stmt`, chaque `HookEntry`, chaque `Branch`, sur `CompApp` et `NativeElem` (la balise ouvrante) et sur les props événementielles (`prop_spans`). Pas sur `Return`. Une position n'est jamais une identité : c'est ce qu'un diagnostic affiche et ce qui le regroupe.

Décision : ADR-11, ADR-19, ADR-39 — trois étapes vers « tout finding a une position »

ADR-11 introduit `SourceRange line, col`, une position optionnelle sur chaque `Stmt` et chaque `HookEntry` (option pour les IR construites à la main) et les notes de diagnostic.

ADR-19 §1 ajoute le `FileId` : les positions d'un corps greffé gardent le fichier où elles ont été abaissées, ce qui permet de nommer le fichier d'origine d'un finding « avec zéro logique dans la greffe » (ADR-24). **ADR-39** traite le résidu : 82 findings sur 7 146 n'avaient aucune position. Règle : *une liaison synthétique est synthétique, sa position ne l'est pas*. Tout temporaire (hoist d'un `await`, bras de ternaire, opérande de `||`, défaut de déstructuration) prend la position de l'expression qu'il lie ; ce que le source ne peut nommer, la greffe le nomme par le site d'appel ; un corps sans instruction propre hérite de la position par laquelle on y est entré, en repli et jamais en écrasement ; un finding sans position prend la première position de sa chaîne de témoins, une fois pour toutes les règles. Résultat mesuré : zéro finding sans position, ensemble des findings inchangé.

Alternative refusée. Donner une position à `Terminator::Return` : « mieux d'une ligne, au prix d'une centaine de sites de construction » ; laissé ouvert (#140).

Le compteur illustre le troisième point : le corps concis du handler, `() => setCount(count + 1)`, est un `Return` seul, sans instruction ni position ; un diagnostic sur ce setter hérite de la position de la prop `onClick` (8:17), par laquelle le parcours est entré.

6.8 Analyses syntaxiques sur l'IR

Plusieurs règles et plusieurs parties du moteur posent à l'IR des questions purement syntaxiques : quelles variables ce corps capture-t-il ? Quels membres lit-il ? Ce tableau de `deps` couvre-t-il ces lectures ? Ce nom désigne-t-il à coup sûr cette fonction ? Les réponses vivent dans `free_vars.rs` et `bindings.rs`. Un exemple d'abord.

Extrait 6.35 – Deps, chemins et alias (/tmp/ch06/deps.tsx)

```

1  import { useEffect, useMemo, useCallback } from "react";
2
3  export function Search({ opts, rows, deps }: { opts: any; rows: any[]; deps: any[] }) {
4      const cfg = opts.config;
5      useEffect(() => {
6          console.log(cfg.theme.color, opts.page[rows.length].id);

```

```

7   }, [opts.config.theme]);
8   const total = useMemo(() => rows.length, [...rows]);
9   const onPick = useCallback((opts: any) => opts.id, deps);
10  useEffect(() => {
11    document.title = String(total);
12  });
13  return <div onClick={onPick}>{total}</div>;
14 }

```

Le rendu abaissé (extrait de la sonde) montre le préambule de paramètres, les liaisons de hooks et les deux marqueurs d'effet :

```

B0:
  let __obj_81 = __p0 @-
  let opts = __obj_81.opts @fFileId(0):3:25
  let rows = __obj_81.rows @fFileId(0):3:31
  let deps = __obj_81.deps @fFileId(0):3:37
  let cfg = opts.config @fFileId(0):4:8
  HookMarker(0, Undefined); @fFileId(0):5:2
  let total = MemoVal(1) @fFileId(0):8:8
  let onPick = CallbackVal(2) @fFileId(0):9:8
  HookMarker(3, Undefined); @fFileId(0):10:2
  -> return Native<div>(Obj#6{onClick: onPick}; [total]) prop_spans=["onClick@fFileId(0):13:14"] @fFileId(0):13:9
hook: Effect #0 deps=List[opts.config.theme] arity=Exact(1) spread_at=[] @fFileId(0):5:2
hook: Memo #1 deps=List[rows] arity=AtLeast(0) spread_at=[0] @fFileId(0):8:8
hook: Callback #2 params=["opts"] deps=Opaque @fFileId(0):9:8
hook: Effect #3 deps=Absent @fFileId(0):10:2
hook: Handler #4 event=click @fFileId(0):13:14

```

et l'analyseur rapporte :

```

Search (5 hooks) deps.tsx
warn missing-deps [hook:0] var:cfg (line 5:2) `cfg` is used in this effect but not in its deps array, and its
  ↳ value may change between renders
warn missing-deps [hook:0] var:opts (line 5:2) `opts.page` is used in this effect but not in its deps array,
  ↳ and its value may change between renders
warn missing-deps [hook:0] var:rows (line 5:2) `rows` is used in this effect but not in its deps array, and its
  ↳ value may change between renders
warn missing-deps [hook:1] var:rows (line 8:8) `rows` is used in this memo but not in its deps array, and its
  ↳ value may change between renders

```

(les lignes « 1 trace step(s) » sont omises). Chacun de ces quatre WARNING s'explique par une fonction de cette section.

6.8.1 Variables libres

Définition 6.5: Variable libre d'un corps

Les *variables libres* d'un CFG C sont les variables lues quelque part dans C moins celles qui y sont définies (cible d'un Let ou d'un Assign) : $libres(C) = (\bigcup_{e \in C} lues(e)) \setminus définies(C)$, avec $lues(FnLit(ps, C')) = libres(C') \setminus ps$.

Extrait 6.36 – src/ir/free_vars.rs, l. 91–107 — les variables libres d'un corps

```

91 /// Compute free variables of a CFG: variables read anywhere minus variables locally defined.
92 pub fn compute_free_vars(cfg: &CFG) -> HashSet<Var> {
93   let mut used: HashSet<Var> = HashSet::new();
94   cfg.for_each_expr(&mut |e| collect_used_vars(e, &mut used));
95
96   // Locally-defined roots (`let`/assignment targets) are bound, not free.
97   let mut defined: HashSet<Var> = HashSet::new();
98   for block in cfg.blocks.values() {
99     for stmt in &block.stmts {

```

```

100         if let Stmt::Let { var, .. } | Stmt::Assign { var, .. } = stmt {
101             defined.insert(var.clone());
102         }
103     }
104 }
105
106 used.difference(&defined).cloned().collect()
107 }

```

Le calcul est *insensible au flot* : toute la liste des lues, moins toute la liste des définies. Il visite les terminateurs (par `for_each_expr`) et les orphelins, ce qui sur-approxime les lectures — le bon côté pour des captures. `collect_used_vars` (`src/ir/free_vars.rs`, l. 403–423) traite deux bras : `Var` et `FnLit`, dont les paramètres masquent les liaisons extérieures (`(open) => !open` ne lit aucun `open` extérieur); tout le reste est délégué à `for_each_child`. Un `Assign` à une variable extérieure la rend « définie » : on ne *lit* pas `x` en l'écrivant. Ces variables libres alimentent les `HandlerInfo` du moteur et la capture d'une closure dans le tas (`src/domains/stores/heap.rs`, l. 57).

6.8.2 Chemins d'accès

La granularité de la variable ne suffit pas à `missing-deps` : avec `[x.b]` en `deps`, un corps qui lit `x.a` a bien une dépendance manquante, un corps qui lit `x.b` non.

Extrait 6.37 – `src/ir/free_vars.rs`, l. 6–23 — un chemin d'accès

```

6 /// A read access rooted at a variable, refined by a chain of field names.
7 ///
8 /// `x` → `{root: x, segments: []}`; `x.a.b` → `{root: x, segments: [a, b]}`.
9 /// This is the granularity `missing-deps` matches at (TODO.md F1b): a dep
10 /// `[x.b]` covers a use of `x.a` only if it is a *prefix* of it, so `use(x.a)`
11 /// with deps [x.b] is (correctly) reported while `use(x.b)` with [x.b]
12 /// and `use(x.a)` with [x] are not.
13 ///
14 /// Dynamic member access (`x[i]`) cannot name the touched element, so on the
15 /// *use* side the path stops at the last named handle above the index:
16 /// `x.a[i].b` → `{root: x, segments: [a]} - "touches all of `x.a`", coverable
17 /// by a whole-`x` or whole-`x.a` dep but not by `x.a.b`. On the *dep* side such
18 /// an access declares nothing (see [dep_paths]).
19 #[derive(Clone, PartialEq, Eq, PartialOrd, Ord, Hash, Debug)]
20 pub struct AccessPath {
21     pub root: Var,
22     pub segments: Vec<String>,
23 }

```

Le calcul des chemins libres, `compute_free_paths`, suit la même structure que celui des variables, avec deux raffinements. D'abord, une chaîne de membres est lue *entière* par `extract_path` :

Extrait 6.38 – `src/ir/free_vars.rs`, l. 280–311 — extraire la chaîne maximale enracinée à une variable

```

280 /// Extract the maximal member chain rooted at a variable, pushing any
281 /// off-chain sub-expressions (index expressions, non-variable bases) into
282 /// `side` for the caller to recurse into. Returns `(root, segments, opaque)`
283 /// where `opaque` marks a dynamic index encountered in the chain (segments
284 /// are then meaningless – collapsed to the bare root).
285 fn extract_path<'e>(e: &'e Expr, side: &mut Vec<&'e Expr>) -> Option<(Var, Vec<String>, bool)> {
286     match e {
287         Expr::Var(v) => Some((v.clone(), Vec::new(), false)),

```

```

288 Expr::TSAnnotated(inner) => extract_path(inner, side),
289 Expr::FieldAccess { obj, field } => {
290     extract_path(obj, side).map(|(root, mut segs, opaque)| {
291         if !opaque {
292             segs.push(field.clone());
293         }
294         (root, segs, opaque)
295     })
296 }
297 Expr::IndexAccess { arr, idx } => {
298     side.push(idx);
299     // The chain *above* the index is still named. `theme.snackBar[v].color`
300     // reads an unknown member of `theme.snackBar`, so that prefix is the
301     // handle the read passes through, and a `[theme.snackBar]` dep covers
302     // it exactly as `[x.a]` already covers `x.a.b`. Only the segments
303     // *below* the index are lost – `opaque` stops the caller pushing more.
304     extract_path(arr, side).map(|(root, segs, _)| (root, segs, true))
305 }
306 other => {
307     side.push(other);
308     None
309 }
310 }
311 }

```

Un index dynamique arrête la chaîne. Ce qui est *au-dessus* reste nommé : l'accès `opts.page[rows.length].id` lit le chemin `opts.page`, par où la lecture passe. Ce qui est dessous est perdu, et l'expression d'index est lue à part, d'où le chemin `rows.length`. Le commentaire de tête (« collapsed to the bare root ») est antérieur à ce comportement : depuis #89, le préfixe au-dessus de l'index est conservé.

Ensuite, les *alias locaux* sont résolus. Un nom lié exactement une fois dans le corps, par un `let`, à une chaîne de membres sans index (`const c = x.a`) est un renommage, pas une lecture : son `let` est ignoré (par `for_each_expr_where`), et les chemins enracinés à `c` sont réécrits vers `x.a` (`src/ir/free_vars.rs`, l. 158–207). La réécriture est *intra-corps* : dans l'exemple, `cfg` est un alias défini dans le rendu, et le corps de l'effet ne le voit pas ; la lecture `cfg.theme.color` reste enracinée à `cfg`, alors que `opts.config.theme` est déclaré. C'est le premier WARNING : un faux positif (FP) toléré, documenté dans les limites connues.

6.8.3 Couverture et épinglage

Extrait 6.39 – `src/ir/free_vars.rs`, l. 253–278 — les chemins déclarés et la couverture par préfixe

```

253 /// Access paths *declared* by a deps array. A dep with a dynamic index
254 /// (`x[i]`) declares nothing coverable – we can't prove which element it
255 /// pins – so it is dropped (leaning to a false positive, never a negative).
256 /// Non-variable deps (literals, calls) yield no path.
257 pub fn dep_paths(deps: &[Expr]) -> Vec<AccessPath> {
258     deps.iter()
259         .filter_map(|e| {
260             let mut side = Vec::new();
261             match extract_path(e, &mut side) {
262                 Some((root, segments, opaque)) if !opaque => Some(AccessPath { root, segments }),
263                 _ => None,
264             }
265         })
266     .collect()

```

```

267 }
268
269 /// A used path is covered when some declared path is a *prefix* of it:
270 /// `[x]` covers `x.a`, `[x.a]` covers `x.a` and `x.a.b`, but `[x.b]` covers
271 /// neither `x.a` nor whole `x`.
272 pub fn path_covered(used: &AccessPath, declared: &[AccessPath]) -> bool {
273     declared.iter().any(|d| {
274         d.root == used.root
275         && d.segments.len() <= used.segments.len()
276         && used.segments[..d.segments.len()] == d.segments[..]
277     })
278 }

```

Définition 6.6: Couverture par préfixe

Un chemin lu u est *couvert* par un ensemble de chemins déclarés D s'il existe $d \in D$ de même racine dont la suite de segments est un préfixe de celle de u . $[x]$ couvre $x.a$; $[x.a]$ couvre $x.a$ et $x.a.b$; $[x.b]$ ne couvre ni $x.a$ ni x .

Côté usage, un index garde le préfixe au-dessus de lui ; côté déclaration, un dep indexé ($[x[i]]$) ne déclare *rien*, car on ne sait pas quel élément il épingle. Les deux choix penchent vers le faux positif. Avec l'invariant 6.3, cela explique les trois derniers WARNING de l'exemple : `opts.page` et `rows.length` ne sont pas couverts par `opts.config.theme` ; et pour le memo, `[...rows]` a une couverture vide, donc `rows` n'est pas déclaré. Le callback 2 (deps `Opaque`) ne produit rien ; ses lectures de `opts` sont celles de son propre paramètre, retiré des chemins libres (`src/engine/fixpoint.rs`, l. 1476–1479). L'effet 3 (deps `Absent`) non plus : il ne peut rien capturer de périmé.

Dernier raffinement : l'*épinglage*. Dans `useCallback(() => ... searchParams.get("sort") ...`, `[searchParams.get("sort")]`, le corps lit `searchParams`, mais seulement à l'intérieur d'une sous-expression qui apparaît *verbatim* dans les deps. React compare la valeur de cette sous-expression, le hook est recréé dès qu'elle change : la lecture ne peut pas être périmée. `free_paths_and_pinned` (`src/ir/free_vars.rs`, l. 120–156) renvoie les chemins libres et ceux qui sont ainsi épinglés, par égalité de *clés canoniques* (`src/ir/free_vars.rs`, l. 374–401). « Verbatim est toute la revendication » : `[JSON.stringify(o)]` n'épingle pas `o`, car une sérialisation perd de l'information. `missing-deps` combine les deux faits : une lecture est acceptée si elle est couverte ou épinglée (`src/rules/impls/missing_deps.rs`, l. 75).

6.8.4 Certificats de liaison

La dernière question est plus forte : ce nom désigne-t-il *à coup sûr* cette valeur ? Résoudre `useStore(sel)` vers le corps de `const sel = s => ...` n'est sound que si `sel` ne peut pas signifier autre chose au moment de l'appel. `bindings.rs` répond à trois forces croissantes, toutes *fail closed* : un nom non prouvé n'est pas « pas une fonction », c'est un nom dont on refuse de fixer le sens, et l'appelant répond $\top$ (`src/ir/bindings.rs`, l. 1–19).

Extrait 6.40 – `src/ir/bindings.rs`, l. 128–149 — l'unique membre droit d'un nom

```

128 /// The single right-hand side `var` is bound to in `cfg`, `None` when it is
129 /// bound zero times or more than once. Conditional or repeated re-binding is
130 /// not "some value we haven't found" – it is a name whose captured environment
131 /// is no longer syntactically certain, so the readers above fail closed.
132 fn sole_binding_in<'c>(var: &str, cfg: &'c CFG) -> Option<&'c Expr> {
133     let mut found: Option<&Expr> = None;
134     for block in cfg.blocks.values() {
135         for stmt in &block.stmts {
136             let (Stmnt::Let { var: v, rhs, .. } | Stmnt::Assign { var: v, rhs, .. }) = stmt else {

```

```

137         continue;
138     };
139     if v != var {
140         continue;
141     }
142     if found.is_some() {
143         return None;
144     }
145     found = Some(rhs.peel_ts());
146 }
147 }
148 found
149 }

```

Le premier niveau, `binding_of` (`src/ir/bindings.rs`, l. 105–126), part de cette liaison unique et la suit à travers les renommages (`bump` range le membre comme `Var("bump")`), au plus huit sauts. Si un nom intermédiaire n'a pas de liaison unique, la poursuite s'arrête sur ce nom, « qu'aucun appelant ne prend pour une valeur ». `closure_binding_of` ajoute les membres d'un `ObjectLit` (un hook custom qui renvoie `clearFieldError` livre la même closure qu'un `const clearFieldError = useCallback(...)`) et répond dans l'une des deux orthographes d'une fonction :

Extrait 6.41 – `src/ir/bindings.rs`, l. 49–56 — les deux orthographes d'une liaison fonctionnelle

```

49 /// The two spellings a function-valued binding can have. Hook extraction
50 /// rewrites `useCallback(fn, deps)` to `CallbackVal(label)` and lifts `fn` out
51 /// of the render CFG, so the second spelling answers a label and a consumer
52 /// that wants the body asks the hook table for it.
53 pub enum ClosureBinding<'c> {
54     Lit { params: &'c [Var], body: &'c CFG },
55     Callback(HookLabel),
56 }

```

Le certificat renforcé, enfin, exige en plus qu'aucun corps imbriqué ne re-lie le nom :

Extrait 6.42 – `src/ir/bindings.rs`, l. 151–179 — le certificat qui autorise à exécuter un corps

```

151 /// [ `fn_binding_in` ], certified against every nested function body: the name
152 /// must be bound exactly once, to a function literal, and never re-bound or
153 /// assigned inside any closure below `root`.
154 ///
155 /// That last clause is what makes the binding usable at an arbitrary later
156 /// program point. A handler or effect that reassigns the name
157 /// (`useEffect(() => { sel = other })`) means the value flowing into a call is
158 /// not the literal this returns, and no order of evaluation makes it so – so
159 /// the certificate is refused rather than qualified.
160 pub fn certified_fn_binding<'c>(<
161     var: &str,
162     root: &'c CFG,
163     also: &[&'c CFG],
164 ) -> Option<(&'c [Var], &'c CFG)> {
165     let binding = fn_binding_in(var, root)?;
166     if bound_below(var, root) {
167         return None;
168     }
169     // Hook extraction lifts effect/callback/memo bodies OUT of the render CFG,
170     // so they are not reachable by descending it – they must be handed in, or
171     // a `useEffect(() => { sel = other })` rebinding would be invisible and the
172     // certificate would certify a body the call never receives.
173     for body in also {
174         if binds(var, body) || bound_below(var, body) {

```



```

175         return None;
176     }
177 }
178 Some(binding)
179 }

```

Le paramètre `also` est indispensable pour une raison propre à l'IR : l'extraction des hooks a sorti les corps d'effet, de memo et de callback du rendu (section 6.4). Un `useEffect(() => sel = other)` serait invisible à une descente du seul rendu, et le certificat certifierait un corps que l'appel ne reçoit pas.

Lecture	Force	Consommateur principal
<code>local_bindings</code>	tous les membres droits de chaque nom, sans unicité	gardes, seeds, churn
<code>binding_of</code> , <code>closure_binding_of</code> , <code>fn_binding_in</code>	liaison unique dans <i>ce</i> corps, renommages et membres suivis	missing-deps, stale-closure, gardes
<code>certified_fn_binding</code>	idem, et jamais re-liée dans un corps imbriqué ni dans un corps de hook sorti	arguments des hooks custom (src/engine/fixpoint.rs, l. 261)

TABLE 6.4 – Les trois lectures de liaison de `bindings.rs`.

Piège

`sole_binding_in` compte les `Assign` comme des liaisons : `let x = ...; x = ...` n'a pas de liaison unique, et la réponse est `None`. C'est voulu (fail closed), mais un lecteur qui attend « la valeur de `x` » après une réaffectation n'obtiendra rien.

6.9 Où l'IR choisit le côté sûr

Récapitulons. Presque chaque choix de ce chapitre tranche entre précision et soundness (définition 3.8), et presque toujours du même côté : tableau 6.5 les rassemble.

Il reste des limites connues, qui ne relèvent pas toutes de ce chapitre mais que l'IR rend visibles :

- #140 : `Terminator::Return` sans position ; un hook atteint seulement par un `return` donne un `finding` sans ligne.
- #49 : les ré-exports ne sont résolus qu'à un niveau, ce qui touche `ContextId` et `CompOrigin`.
- #63 (*wontfix*) : un composant dynamique (`const C = cond ? A : B; <C/>`) donne un `CompApp` sans origine, que rien ne résout. L'issue et ADR-12 disent qu'aucun `CompApp` n'est produit ; le dossier source observe qu'il en est produit un, non résolu.
- #28 : `useContext` est un `Custom` non modélisé.
- Le champ `HookEntry::Custom.binding` est mort ; la ligne de provenance d'un wrapper greffé a un label pendant ; les handlers n'ont ni paramètres ni ligne de provenance.

Les défauts qui naissent de la greffe (collision de sites entre fichiers, un hook custom appelé deux fois n'est greffé qu'une fois, expression de `return` perdue pour un appel sans liaison) sont exposés au chapitre 8.

Lieu	Choix	Erreur résiduelle
DepsList::covering	un spread ne couvre rien	FP
DepsArg::Opaque	gardé mais illisible	ni FN ni FP massif
MarkerVal::Unknown	hook opaque = $\top$, pas undefined	FP
ModuleConstInit	genres certains seulement	FP ($\top$)
object_member après spread	ne répond rien	FP
dep_paths avec index	ne déclare rien	FP
extract_path (usage)	garde le préfixe au-dessus de l'index	FP limité
alias locaux	intra-corps seulement	FP
certified_fn_binding	refuse au moindre re-binding	précision perdue, $\top$
chute en fin de corps	Return(undefined) (ADR-25)	évite un FN
hook_body_cfg	un argument non littéral est invoqué	évite un FN
CompApp.origin, resolve_child	Ambiguous plutôt que deviner	limite signalée
ModuleTable vide	« pas de preuve »	aucune confusion
TSAnotated	le type n'est jamais lu	évite un FN

TABLE 6.5 – Précision contre soundness dans l'IR.

Résumé du chapitre

L'IR de `reactant-analyzer` est un ensemble de types possédés, sans AST, que lisent le moteur, les relations et les règles (ADR-3). Un corps est un CFG : une `BTreeMap` de `BasicBlock` (instructions `Let`, `Assign`, `MemberWrite`, `ExprStmt`, puis un `Terminator` `Jump`, `Branch`, `Return` ou `Unreachable`) et une liste d'arêtes typées (`Unconditional`, `IfTrue`, `IfFalse`, `Back`, `Await`) qui fait foi pour la navigation. Un composant est un `render_cfg` plus une table de hooks dont chaque effet a son CFG (ADR-4).

`Expr` se lit par familles ; les nœuds allouants portent un `ExprId`, le *site* qui indexe le tas abstrait (ADR-10) ; `for_each_child` est l'énumération canonique sans bras par défaut. Un appel de hook devient une ligne `HookEntry` indexée par son rang (le label, qui nomme le *slot*), une valeur dans le rendu (`StateVal`, `MemoVal`... ou `HookMarker` avec un `MarkerVal`) et une ligne de provenance. Les deps sont `Absent`, `Opaque` ou `List` ; une liste garde son arité et ses spreads, et seule sa couverture peut faire taire une règle.

Les unités sont `ComponentIR`, `HookIR`, `FunctionIR` ; les faits de module (`ModuleConstInit`, `ContextId`, `ModuleFacts`) ne disent que ce qui est certain, et une absence n'est jamais une preuve. Fichiers et composants ont une identité internée de 4 octets (`FileId`, `ComponentId`) ; le nom affiché est un rendu (ADR-40). Les positions sont des points (`file`, `line`, `col`) ; toute liaison synthétique prend la position de ce qu'elle lie (ADR-39). Enfin `free_vars.rs` et `bindings.rs` répondent aux questions syntaxiques : variables et chemins libres, couverture par préfixe, épinglage, certificats de liaison.

Fichiers rencontrés (sous `src/ir/`) : `types.rs`, `cfg.rs`, `stmt.rs`, `expr.rs`, `hooks.rs`, `component.rs`, `hook_ir.rs`, `function_ir.rs`, `component_id.rs`, `module.rs`, `source_range.rs`, `free_vars.rs`, `bindings.rs`.

Suite : le chapitre 7 montre comment le lowering construit ces structures à partir de l'AST, forme syntaxique par forme syntaxique ; le chapitre 8 montre comment le moteur les réécrit pour greffer hooks custom et utilitaires.

Exercices

Exercice 6.1: Prédire DepsArg

Pour chacun des appels suivants, donner le DepsArg produit (état, elems, arity, spread_at), puis la couverture covering() : (a) useEffect(f, [a, , ...b]); (b) useEffect(f, deps); (c) useEffect(f); (d) useEffect(f, [a] as const); (e) useMemo(g, [...xs, y]).

Solution

Les quatre premiers cas sont ceux du liste 6.17, observés : (a) List, elems = [a, b], AtLeast(2), spread_at = [1], couverture [a]; (b) Opaque, couverture vide mais is_declared() vrai; (c) Absent; (d) List, [a], Exact(1) (l'annotation est pelée). Pour (e), par la même règle de comptage (et la sonde le confirme sur /tmp/ch06/e.tsx) : elems = [xs, y], AtLeast(1), spread_at = [0], couverture [y].

Exercice 6.2: Dessiner un CFG

Dessiner le CFG du rendu suivant, en nommant la nature de chaque arête, puis le comparer à la sortie de la sonde donnée en solution.

```
1 export function List({ items, ok }: { items: number[]; ok: boolean }) {
2   const [x, setX] = useState(0);
3   let total = 0;
4   for (let i = 0; i < items.length; i++) {
5     if (items[i] < 0) break;
6     total = total + items[i];
7   }
8   ok && setX(1);
9   return <p>{total}</p>;
10 }
```

Solution

La sonde (/tmp/ch06/loop.tsx) imprime, préambule de paramètres omis :

```
B0: ... let total = 0; let i = 0; -> jump B1
B1: -> branch (i Lt items.length) ? B2 : B4
B2: -> branch (items[i] Lt 0) ? B5 : B6
B3: i := (i Add 1); i; -> jump B1
B4: let __t0 = ok; -> branch __t0 ? B8 : B9
B5: -> jump B4
B6: -> jump B7
B7: total := (total Add items[i]); total; -> jump B3
B8: __t0 := setX(1); -> jump B9
B9: __t0; -> return Native<p>(...)
edges: B0->B1:Unconditional B1->B2:IfTrue B1->B4:IfFalse B2->B5:IfTrue B2->B6:IfFalse
       B5->B4:Unconditional B6->B7:Unconditional B7->B3:Unconditional B3->B1:Back
       B4->B8:IfTrue B4->B9:IfTrue B8->B9:Unconditional
```

L'incrément B3 ferme la boucle par l'unique arête Back, vers l'en-tête B1 ; le break (B5) saute vers la sortie de boucle B4. Les deux arêtes du diamant && sont étiquetées IfTrue : voir le piège de la section 6.2. L'ordre des identifiants n'est pas l'ordre d'exécution (B3 est exécuté après B7).

Exercice 6.3: Ajouter un variant à Expr

On ajoute un variant Expr::Await(Box<Expr>). (a) Que faut-il écrire dans for_each_child ? (b) Pourquoi un parcourreur qui écrirait _ => {} au lieu de déléguer serait-il un faux négatif en attente ? (c) Citer deux autres match de ce chapitre qui profitent de la même discipline.

Solution

- (a) Un bras `Expr::Await(inner) => f(inner)` : le compilateur l'exige, faute de bras par défaut.
- (b) Parce que le parcourreur traiterait l'expression attendue comme « sans enfants » : un setter appelé dans `await setX(...)` deviendrait invisible, par exemple aux variables libres.
- (c) `HookEntry::body_cfg`, `HookEntry::label/span`, et `lower_binop` côté lowering.

Exercice 6.4: Chemins libres, déclarés, couverts

Pour `useEffect(() => f(x.a[i].b); , [x.a.b])`, où `x`, `i` et `f` sont des props, donner les chemins libres du corps, les chemins déclarés, et les lectures non couvertes. Quels deps faut-il écrire pour faire taire `missing-deps` ?

Solution

Chemins libres : `f`, `x.a` (le préfixe au-dessus de l'index), `i`. Chemin déclaré : `x.a.b`, qui n'est préfixe d'aucun d'eux. Les trois lectures sont non couvertes ; l'analyseur (`/tmp/ch06/paths.tsx`) rapporte trois `WARNING missing-deps`, dont « `x.a` is used in this effect but not in its deps array ». Avec `[x.a, i, f]`, il ne rapporte plus rien.

Exercice 6.5: Pourquoi ne jamais indexer par le nom affiché

Un projet contient un seul composant `Widget`, dans `a/Widget.tsx`. On ajoute `b/Widget.tsx`, qui définit un autre `Widget` sans rapport. (a) Que devient `display_name` du premier ? son `ComponentId` ? (b) Qu'arriverait-il à un store partagé indexé par le nom affiché ? (c) Quel test de `component_id.rs` fixe ce comportement ?

Solution

- (a) Le nom affiché passe de `Widget` à `Widget@a/Widget.tsx` ; l'identifiant ne change pas.
- (b) Toutes ses clés changeraient à cause d'une modification lointaine et sans rapport : c'est exactement le défaut qu'ADR-40 corrige.
- (c) `interning_a_namesake_changes_an_existing_display_name_but_not_its_id`.

Exercice 6.6: Tirer ou taire

Expliquer, en un paragraphe, pourquoi `missing-deps` peut énumérer `elems` pour *signaler* une lecture, mais doit passer par `covering()` pour la *déclarer*. Construire un effet où confondre les deux ferait un faux négatif.

Solution

Signaler est une affirmation « peut-être » : chaque élément sur-approxime ce qu'il représente, et une énumération trop large ne crée au pire qu'un faux positif. Déclarer fait taire la règle : il faut que React compare vraiment la valeur lue. Avec `useEffect(() => ref.current = rows; , [...rows])`, React compare `rows[0]`, `rows[1]`, ... et jamais l'identité de `rows` ; si `rows` est remplacé par un nouveau tableau aux mêmes éléments, l'effet ne se réexécute pas et `ref.current` garde l'ancien tableau. Créditer `rows` comme déclaré tairait ce défaut : un faux négatif. C'est le memo du liste 6.35, signalé à bon droit.

Chapitre 7

Le lowering : de l’AST au CFG

Objectifs du chapitre.

- Savoir où le lowering se place dans le pipeline, quelles fonctions l’appellent et dans quel ordre s’enchaînent ses trois passes (construction du CFG, extraction des hooks, extraction des handlers et subscriptions).
- Maîtriser la machine à blocs `BlockBuilder` : réserver, sceller, ouvrir, relier ; savoir pourquoi un corps qui « tombe du bout » se termine par `Return(undefined)` (ADR-25).
- Connaître le gabarit de CFG de chaque construction de contrôle : `if`, `while`, `for`, `do...while`, `for...of`, `switch`, `try`, ternaire, `&&/||/??`, `await` ; savoir quelle nature d’arête (`EdgeKind`) chacune produit, et pourquoi.
- Comprendre le principe qui gouverne l’abaissement des expressions : ne jamais perdre une lecture, une écriture ni un chemin, quitte à sur-approximer.
- Suivre l’extraction des hooks sur un exemple, avant et après : classification par provenance, marqueurs, destructuration d’état, corps de hooks, handlers, subscriptions, hoist de terminateur.
- Connaître les limites vérifiées du lowering au commit e67b10a, et les pièges qui attendent qui modifie ce code.

Prérequis : chapitre 6 (les types CFG, `Stmt`, `Expr`, `HookEntry` : définition 6.1, définition 6.2, définition 6.3), chapitre 5 (d’où viennent les fonctions candidates, définition 5.1), chapitre 3 (soundness, définition 3.8 ; dominance, définition 3.9 ; widening, définition 3.6).

Le chapitre précédent a décrit le vocabulaire de l’IR ; celui-ci explique comment on l’écrit. Le *lowering* est la traduction de l’AST d’oxc — du JavaScript ou du TypeScript complet, riche en sucre syntaxique — vers ce vocabulaire réduit. C’est la couche où la syntaxe disparaît : après elle, ni les domaines abstraits ni le moteur ne voient jamais un nœud d’AST. Toutes les formes équivalentes d’un programme (une destructuration et ses projections, un ternaire et un `if`, un `&&` en instruction et un `if` sans `else`) doivent donc y être ramenées à une forme unique, sans rien perdre de ce que le programme *fait*.

Cette dernière exigence est le fil rouge du chapitre. Le lowering n’est pas une simple traduction : c’est le premier endroit où l’on peut introduire un faux négatif. Oublier une lecture, c’est affirmer qu’une variable n’est pas utilisée ; oublier un chemin, c’est affirmer qu’un hook s’exécute toujours ; oublier une écriture, c’est affirmer qu’une variable garde sa valeur. Chacune de ces affirmations est fausse, et aucune couche en aval ne peut la corriger. Le code en porte la trace : presque chaque commentaire de `src/lowering/` justifie un choix par « ne pas perdre une lecture, un chemin ou une écriture ».

Nous procéderons du plus simple au plus difficile : la machine à blocs sur un corps linéaire, puis le `if`, les expressions, les diamants, les boucles, le `switch` et le `try`, les motifs, les fonctions imbriquées, l’`await`, et enfin la passe qui reconnaît les hooks. Chaque construction est montrée sur un programme court, avec l’IR réellement produite.

Remarque

Le dépôt ne fournit aucune option pour imprimer l'IR : `-verbose` n'écrit que des lignes de diagnostic (racine de découverte, imports suivis, itérations du point fixe), et les types de l'IR ne dérivent que `Debug`. Les impressions de ce chapitre viennent d'une petite sonde hors dépôt qui appelle `reactant::lowering::lower_program` et `lower_custom_hooks`, puis imprime l'IR dans une notation compacte. `B3:` ouvre un bloc; `let v = e`, `v := e`, `o.f <- e` et `expr e` sont respectivement un `Let`, un `Assign`, un `MemberWrite` et un `ExprStmt`; `@l:c` est la position (ligne, colonne); `=> ...` est le terminateur; `edges:` liste les arêtes avec leur nature (`T`, `F`, `U`, `Back`, `Await` pour `IfTrue`, `IfFalse`, `Unconditional`, `Back`, `Await`). Un littéral d'objet s'imprime `{#k ...}` avec son `ExprId` `k`, un tableau `[#k ...]<Exact(n)>`, une fonction `fn#k(p) {...}`, et la valeur opaque `SummaryVal(Top)` s'imprime `T`. Une variante `-raw` de la sonde imprime le CFG d'une fonction *avant* l'extraction des hooks, sans positions.

7.1 Le lowering dans le pipeline

7.1.1 Trois fichiers, trois passes

Trois fichiers portent le travail, pour environ 4 700 lignes dont un tiers de tests unitaires (67 tests) :

- `src/lowering/cfg_builder.rs` contient le constructeur de blocs (`BlockBuilder`) et l'abaissement des *instructions* : contrôle, déclarations, motifs de liaison, préambule des paramètres.
- `src/lowering/expr_lower.rs` abaisse les *expressions*, dont celles qui fendent des blocs (`?:`, `&&`, `||`, `??`, `await`).
- `src/lowering/hook_extractor.rs` est une passe *IR vers IR* : elle reconnaît les appels de hooks dans le CFG déjà produit, les remplace par des valeurs marquées (`StateVal(ℓ)`, `StateSetter(ℓ)`, `MemoVal(ℓ)`, `CallbackVal(ℓ)`, `HookMarker(ℓ, __)`) et fabrique la table des `HookEntry`, handlers et subscriptions compris.

Les deux premiers fichiers sont mutuellement dépendants : une instruction contient des expressions, et une expression (une flèche, une classe) contient des instructions. Le troisième ne connaît pas l'AST ; il ne lit que l'IR.

7.1.2 La chaîne d'appel

Le driver lit chaque fichier, le fait analyser par `oxc`, puis appelle trois points d'entrée :

- `lower_program_with_resolver` pour les composants (`src/lowering/mod.rs`) ;
- `lower_custom_hooks_with_resolver` pour les hooks custom (même fichier) ;
- `lower_utilities_with_resolver` pour les utilitaires (`src/lowering/utility_lowerer.rs`).

La répartition des fonctions entre ces trois classes a été faite avant, par les détecteurs du chapitre 5, qui produisent des `Candidate`. Pour un composant, la chaîne est exactement celle-ci :

Extrait 7.1 – `src/lowering/mod.rs`, l. 463–474 — la chaîne de lowering d'un composant

```

463     detect_components(program)
464         .into_iter()
465         .map(|candidate| {
466             let (param_names, mut render_cfg) = candidate.build_cfg(&ctx);
467             let (mut hooks, hook_provenance, mut next_label) =
468                 extract_hooks(&mut render_cfg, &imports);
469             extract_handlers(&render_cfg, &mut hooks, &mut next_label);
470             extract_subscriptions(&mut hooks, &mut next_label);

```

```

471         let param = param_names
472         .into_iter()
473         .next()
474         .unwrap_or_else(|| "props".to_string());

```

L'ordre est porteur de sens. `extract_handlers` passe *après* `extract_hooks`, parce qu'il résout `onClick=cb` quand `cb` est lié à un `CallbackVal(ℓ)`, que seul `extract_hooks` a produit. `extract_subscriptions` passe en dernier, parce qu'il fouille les corps des `HookEntry::Effect`. Les labels sont donc contigus : les hooks d'abord ($0..k$), puis les handlers, puis les subscriptions. Le paramètre du composant est le premier nom de paramètre, ou "props" s'il n'y en a pas (`src/lowering/mod.rs`, l. 471–474).

La même chaîne de trois passes est appliquée au corps de chaque hook custom (`src/lowering/mod.rs`, l. 396–404). Les utilitaires, eux, ne passent que par la construction du CFG, sans aucune extraction : un appel `useX()` dans un utilitaire reste un `Call` ordinaire, ce qui est cohérent avec la définition d'un utilitaire (une fonction qui n'appelle pas de hook).

La construction du CFG elle-même passe par un point de dispatch unique :

Extrait 7.2 – `src/lowering/mod.rs`, l. 146–160 — le seul endroit qui choisit entre corps-bloc et corps-expression

```

146 impl Candidate<'_> {
147     /// Lower this candidate's body to a CFG, honouring a concise arrow's
148     /// implicit return.
149     ///
150     /// The single place that dispatch happens, so a new consumer of
151     /// [Candidate] cannot forget it – which is how the flag came to be
152     /// dropped in the first place.
153     pub(crate) fn build_cfg(&self, ctx: &LowerCtx) -> (Vec<String>, crate::ir::cfg::CFG) {
154         if self.expression {
155             cfg_builder::build_expr_fn_body_cfg(self.params, self.body, ctx)
156         } else {
157             cfg_builder::build_fn_body_cfg(self.params, self.body, ctx)
158         }
159     }
160 }

```

Le commentaire rappelle une histoire que nous retrouverons à la section 7.13 : le drapeau `expression` d'une flèche concise (`x => expr`) a déjà été oublié par un consommateur, et la correction a consisté à ne laisser qu'un endroit où il est lu. Les fonctions imbriquées (flèches, `function`, méthodes, blocs `static`) ne passent pas par `Candidate` : `lower_expr` les abaisse récursivement dans leur propre `BlockBuilder` et les emballe dans un `Expr::FnLit` (section 7.9).

Figure 7.1 résume la position du lowering. La frontière à retenir est verticale : à gauche l'AST, à droite l'IR, et rien ne la franchit en sens inverse.

7.1.3 Pourquoi un CFG

Décision : ADR-3 — une IR dédiée, en CFG

ADR-3 décide une IR propre au projet, inspirée de `React-tRace` [LAY25] mais représentée comme un graphe de flot de contrôle plutôt que comme un arbre. L'alternative refusée, une IR arborescente, exige une transformation CPS pour les retours anticipés et ne sait pas représenter une boucle sans arc arrière. Le CFG représente nativement retours anticipés, boucles et `switch`; ses en-têtes de boucle sont structurels, ce qui place naturellement le `widening`; la dominance, nécessaire pour les hooks conditionnels, y est un algorithme

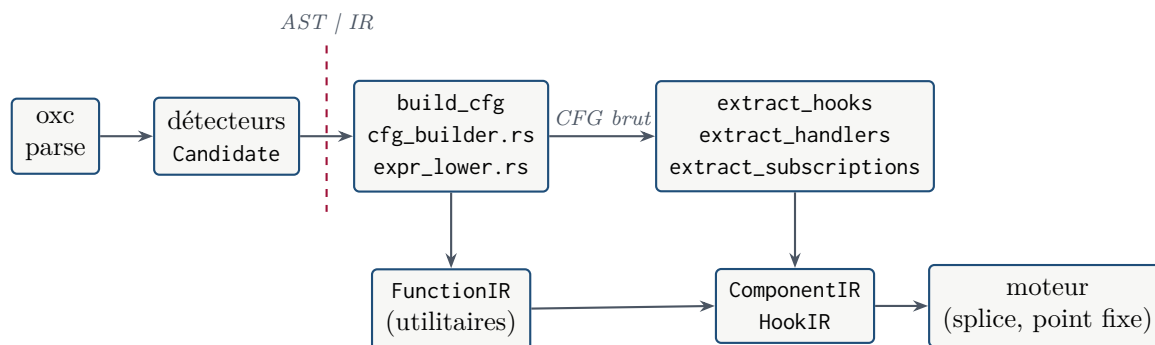


FIGURE 7.1 – Le lowering dans le pipeline. La construction du CFG est commune aux trois classes de fonctions ; l'extraction des hooks, des handlers et des subscriptions ne s'applique qu'aux composants et aux hooks custom. À droite de la ligne rouge, aucun consommateur ne voit l'AST.

standard.

L'ADR a vieilli sur plusieurs points de détail, qu'il faut connaître pour ne pas la lire comme une spécification : sa table de désucrage annonce $a \&\& b \rightarrow \text{If}(a, b, \text{Lit}(\text{false}))$ et $a ? b : c \rightarrow \text{If}(a, b, c)$, alors que le code produit des diamants de blocs (section 7.5) ; les « nœuds de hooks » `useState`, `useEffect` n'existent pas sous ces noms (ce sont des `HookEntry` et des marqueurs) ; le terminateur `Unreachable` n'y figure pas ; le lowering n'est plus une passe unique, puisque `hook_extractor` le suit ; enfin `docs/ir.md`, cité comme grammaire complète, n'existe pas dans le dépôt.

Décision : ADR-4 — un CFG de rendu, des corps d'effets séparés

ADR-4 fixe la forme d'un composant : un `render_cfg` et une table `hooks` dont les entrées d'effets, de memos et de callbacks portent chacune leur propre CFG. L'alternative refusée est un méta-CFG unique `rendu → effets → vérification`, jugé gros, difficile à maintenir et porteur d'un ordre `React` implicite. C'est ce choix qui explique que `extract_hooks` *déplace* le corps d'un `useEffect()` $\Rightarrow \dots$ hors du CFG de rendu (section 7.11). La structure de données décrite par l'ADR (`deps: Option<Vec<Expr>`) est périmée : c'est aujourd'hui un `DepsArg` (chapitre 6).

7.2 La machine à blocs

7.2.1 Un corps linéaire

Commençons par le cas le plus simple : un corps sans aucun branchement. Le hook custom suivant calcule un titre et l'applique dans un effet.

Extrait 7.3 – Un corps linéaire, sans return (`/tmp/ch07/c1_linear.tsx`)

```

1 import { useEffect } from "react";
2
3 export function useTitle(title: string) {
4   const full = title + " | App";
5   useEffect(() => {
6     document.title = full;
7   }, [full]);
8 }

```

```

=== custom hook useTitle (params ["title"]) ===
B0:
  let full = (title Add " | App") @4:8
  expr HookMarker(0, Undefined) @5:2
  => return undefined
edges:
hooks:
#0 Effect deps=List[full]<Exact(1)> @5:2
  B0:
    document.title <- full @6:4
    expr full @6:4
    => return undefined
edges:

```

Trois choses se lisent déjà. Le paramètre simple `title` ne produit aucune instruction : il sera lié par l'appelant (section 7.8). Le corps ne contient pas de `return`, et pourtant son unique bloc se termine par `return undefined`. Enfin, l'affectation `document.title = full` devient deux instructions : l'écriture en place (`MemberWrite`) puis la valeur de l'expression, gardée en `ExprStmt` (section 7.4).

7.2.2 Le constructeur

Tout CFG est fabriqué par une instance de `BlockBuilder`.

Extrait 7.4 – `src/lowering/cfg_builder.rs`, l. 78–100 — l'état du constructeur de blocs

```

78 pub(super) struct BlockBuilder {
79     blocks: BTreeMap<BlockId, BasicBlock>,
80     edges: Vec<Edge>,
81     current: BlockId,
82     counter: usize,
83     current_stmts: Vec<Stmt>,
84     terminated: bool,
85     temp_counter: usize,
86     /// Enclosing breakables, innermost last. A `switch` is breakable but not
87     /// continuable, so `continue_to` is `None` there.
88     loop_stack: Vec<LoopFrame>,
89     /// Label of the statement being lowered, when it labels a loop directly.
90     pending_label: Option<String>,
91     pub(super) ctx: LowerCtx,
92 }
93
94 /// One enclosing `break`/`continue` target.
95 struct LoopFrame {
96     label: Option<String>,
97     break_to: BlockId,
98     /// `None` for a `switch`: `continue` skips past it to the enclosing loop.
99     continue_to: Option<BlockId>,
100 }

```


Champ	Rôle
blocks, edges	le CFG en cours ; un bloc n'entre dans blocks qu'au moment où il est scellé
current	identifiant du bloc ouvert
counter	prochain identifiant libre ; initialisé à 1, car le bloc 0 est l'entrée (src/lowering/cfg_builder.rs, l. 108)
current_stmts	instructions du bloc ouvert
terminated	le bloc courant est scellé ; push_stmt devient alors sans effet (src/lowering/cfg_builder.rs, l. 166–170)
temp_counter	suffixe des temporaires __tN (fresh_temp), propre à chaque constructeur
loop_stack	pile des cibles de break/continue (section 7.6)
pending_label	label posé par une instruction étiquetée, consommé par la boucle suivante
ctx	le contexte partagé par tous les corps du fichier (LowerCtx, section 7.9)

Deux opérations élémentaires font tout le travail.

Extrait 7.5 – src/lowering/cfg_builder.rs, l. 172–203 — sceller, ouvrir, couper à un await

```

172  /// Seal current block with terminator. Returns sealed block id.
173  pub(super) fn seal_with(&mut self, term: Terminator) -> BlockId {
174      debug_assert(!self.terminated, "sealing already-terminated block");
175      let id = self.current;
176      let stmts = std::mem::take(&mut self.current_stmts);
177      self.blocks.insert(id, BasicBlock { id, stmts, term });
178      self.terminated = true;
179      id
180  }
181
182  /// Switch active block. Clears terminated flag.
183  pub(super) fn start_block(&mut self, id: BlockId) {
184      self.current = id;
185      self.terminated = false;
186  }
187
188  /// Seal the current block and continue in a fresh one across an `Await`
189  /// edge (#117). Control is unconditional – dominance and reachability are
190  /// unchanged – but the successor is marked as running after a suspension.
191  ///
192  /// A no-op on an already-terminated block: a `return await f()` has no
193  /// successor to defer, and sealing twice would trip the builder's own
194  /// invariant.
195  pub(super) fn split_at_await(&mut self) {
196      if self.terminated {
197          return;
198      }
199      let next = self.new_block();
200      let from = self.seal_with(Terminator::Jump(next));
201      self.add_edge(from, next, EdgeKind::Await);
202      self.start_block(next);
203  }

```

Définition 7.1: Sceller, ouvrir, réserver

Réserver un bloc, c'est tirer un identifiant neuf (new_block) sans rien créer. *Sceller* le bloc courant, c'est lui donner son terminateur et l'insérer dans blocks (seal_with) ; le constructeur est alors dans l'état « terminé ». *Ouvrir* un bloc réservé, c'est en faire le bloc courant et quitter l'état terminé (start_block). Chaque terminateur Jump ou Branch scellé doit être accompagné des arêtes correspondantes (add_edge).

Toutes les constructions de contrôle suivent le même protocole, qu'il faut avoir en tête pour

lire le fichier :

1. réserver les identifiants des blocs cibles ;
2. sceller le bloc courant avec un terminateur et ajouter les arêtes correspondantes ;
3. ouvrir chaque cible et y abaisser le sous-arbre ;
4. si le sous-arbre n'a pas terminé le bloc, le sceller par un Jump vers la jonction, sous la garde `if !builder.is_terminated()` ;
5. ouvrir la jonction et continuer.

Invariant 7.1: Arêtes et terminateurs

Toute cible d'un terminateur `Jump` ou `Branch` produit par le lowering a une arête dans `CFG.edges`. Le lowering le garantit par construction (chaque `seal_with(Jump|Branch)` est suivi d'`add_edge`), mais `CFG::validate`, qui vérifie cette propriété, n'est appelé qu'en `debug_assert!` après un `splice` (`src/ir/splice.rs`, l. 251–255), jamais directement sur la sortie du lowering.

Cet invariant n'est pas décoratif : les fonctions de navigation (`successors`, `predecessors`, `reachable_blocks`) lisent `edges`, pas les terminateurs (chapitre 6). Un `Jump` sans arête rendrait sa cible invisible au point fixe.

7.2.3 La boucle des instructions et le code mort

Une liste d'instructions s'abaisse séquentiellement, avec un arrêt :

Extrait 7.6 – `src/lowering/cfg_builder.rs`, l. 270–277 — la boucle d'instructions s'arrête au premier terminateur

```
270 fn lower_stmts(stmts: &[Statement], builder: &mut BlockBuilder) {
271     for stmt in stmts {
272         if builder.is_terminated() {
273             break;
274         }
275         lower_stmt(stmt, builder);
276     }
277 }
```

Dès qu'une instruction a scellé le bloc (`return`, `throw`, `break`, `continue`), le reste de la liste n'est pas abaissé du tout. C'est raisonnable pour du vrai code mort, mais JavaScript hisse les déclarations de fonctions : une fonction déclarée après un `return` est bel et bien définie. Nous verrons à la section 7.14 ce que cela coûte.

7.2.4 La sortie : `into_cfg` et la chute en fin de corps

Extrait 7.7 – `src/lowering/cfg_builder.rs`, l. 213–240 — la sortie du constructeur

```
213 pub(super) fn into_cfg(mut self, entry: BlockId) -> CFG {
214     if !self.terminated {
215         let id = self.current;
216         let stmts = std::mem::take(&mut self.current_stmts);
217         self.blocks.insert(
218             id,
219             BasicBlock {
220                 id,
221                 stmts,
222                 // A body that falls off the end returns `undefined` – that
223                 // is a `Return`, not `Unreachable`. Sealing it `Unreachable`
224                 // told the splice that control never came back, so the join
```

```

225          // block carrying the post-call statements *and the caller's
226          // own terminator* was left with no predecessor: 198 corpus
227          // components were severed from their own `Return`, and every
228          // `stability_verdict` on them read an exit env missing a
229          // real path (a false negative). `Unreachable` now means only
230          // what it says – a `throw`, a stray `break`.
231          term: Terminator::Return(Expr::Lit(Prim::Unit)),
232      },
233  );
234  }
235  CFG {
236      entry,
237      blocks: self.blocks,
238      edges: self.edges,
239  }
240  }

```

Si le dernier bloc ouvert n'a pas été scellé, c'est que le corps « tombe du bout » : `into_cfg` le scelle par `Return(Lit(Unit))`, c'est-à-dire `return undefined`. Le commentaire raconte pourquoi ce n'est pas `Unreachable`.

Décision : ADR-25 — la chute en fin de corps est un `return undefined`

Avant ADR-25, `into_cfg` scellait la queue en `Unreachable`. Or trois situations produisaient ce terminateur : un corps qui tombe du bout (le contrôle revient, avec `undefined`), un `throw` (il ne revient pas) et un `break` sans cible (JavaScript invalide). Le splice (chapitre 8) lit les terminateurs du appelé pour décider où le contrôle reprend : chaque `Return(e)` devient `[let bound = e;] Jump(join)`, et un `Unreachable` est laissé tel quel. Un hook custom sans return produisait donc une jonction sans prédécesseur, et l'appelant était coupé de sa propre sortie : 208 composants du corpus avaient un CFG de rendu sectionné, et toutes les règles qui lisent l'environnement de sortie joignaient sur moins de chemins que le programme n'en a — la direction interdite.

Décisions : (1) la queue est scellée en `Return(undefined)`, et `Unreachable` ne veut plus dire que « le contrôle ne continue pas » ; (2) `throw` garde `Unreachable` et le splice continue de le laisser seul. L'alternative tentante — relier tout `Unreachable` du appelé à la jonction, puisqu'un chemin de plus n'est « qu'une » sur-approximation — a été implémentée, mesurée et refusée : elle invente un chemin qui atteint la sortie de l'appelant *sans passer par les hooks du appelé*, et signalait l'idiome « garde-throw » comme `conditional-hook` au niveau `ERROR` sur du code conforme. (3) Un `Return` inatteignable n'est pas une sortie : l'ensemble des sorties passe par `CFG::reachable_blocks`. Mesure : composants sectionnés `208 → 3` ; 12 findings révélés, aucun perdu. Tests : `tests/cfg_exit_integrity.rs`.

Piège

Un chemin de plus n'est pas toujours gratuit. Le réflexe « dans le doute, ajoutons une arête, c'est sound » est juste pour les faits `may` et faux pour les faits `must` : une arête inventée vers la sortie peut faire croire qu'un hook est évitable, et produire un `ERROR` faux. Le lowering sur-approxime les *comportements*, pas les *graphes* : une arête n'est ajoutée que si le programme peut réellement la suivre.

7.3 Le if et le retour anticipé

7.3.1 La table des instructions

`lower_stmt` (`src/lowering/cfg_builder.rs`, l. 279–466) est un `match` sur les variantes de

Statement d'oxc. Tableau 7.1 en donne la carte ; les sections suivantes détaillent chaque ligne.

Instruction	IR produite
const/let/var	pour chaque déclarateur, la valeur (ou <code>Lit(Unit)</code> sans initialiseur) puis le motif de liaison (section 7.8)
expression e;	<code>ExprStmt(lower_expr(e))</code> ; l'expression a pu émettre ses propres instructions et fendre des blocs
return e / return	scelle <code>Return(e)</code> / <code>Return(Lit(Unit))</code>
if	<code>lower_if</code> : trois blocs réservés, <code>Branch</code>
...	la liste à plat : aucune portée lexicale n'est modélisée
boucles	<code>lower_while</code> , <code>lower_for</code> , <code>do...while</code> en ligne, <code>lower_iter_loop</code> (section 7.6)
throw e	<code>ExprStmt(e)</code> puis <code>Unreachable</code>
try, switch	<code>lower_try</code> , <code>lower_switch</code> (section 7.7)
l: boucle	le label est mis en attente, puis la boucle ; sur tout autre corps, le label est ignoré
break, continue	un <code>Jump</code> vers la cible de la pile (<code>Unconditional / Back</code>) ; sans cible, <code>Unreachable</code>
function f()	<code>Let f = FnLit</code> à sa position textuelle (pas de hispage)
class X	<code>Let X = lower_class(...)</code> , ou <code>ExprStmt</code> si anonyme
with (o) body	<code>ExprStmt(o)</code> puis le corps, les noms traités comme des liaisons ordinaires
vide, debugger, types TS, import/export	rien — mais écrit en extension

TABLE 7.1 – Ce que `lower_stmt` produit pour chaque instruction.

La dernière ligne mérite qu'on s'y arrête. La liste des instructions « sans code » est écrite variante par variante, sans bras `_` :

Extrait 7.8 – `src/lowering/cfg_builder.rs`, l. 445–464 — les instructions sans code, en extension

```

445     // Nothing executable – and spelled out, with no `_` arm, so a statement
446     // kind that *does* carry code cannot join this list by accident. That is
447     // how class bodies were lost in the first place.
448     Statement::EmptyStatement(_)
449     | Statement::DebuggerStatement(_)
450     // Types erase before anything runs.
451     | Statement::TSTypeAliasDeclaration(_)
452     | Statement::TSInterfaceDeclaration(_)
453     | Statement::TSEnumDeclaration(_)
454     | Statement::TSMODULEDeclaration(_)
455     | Statement::TSGlobalDeclaration(_)
456     | Statement::TSImportEqualsDeclaration(_)
457     // `import`/`export` are module-level only: a syntax error inside the
458     // function bodies this lowers, so they never reach here.
459     | Statement::ImportDeclaration(_)
460     | Statement::ExportAllDeclaration(_)
461     | Statement::ExportDefaultDeclaration(_)
462     | Statement::ExportNamedDeclaration(_)
463     | Statement::TSEXPORTAssignment(_)
464     | Statement::TSNamespaceExportDeclaration(_) => {}

```

Le commentaire donne la raison : c'est ainsi que les corps de classes ont été perdus (#77). Une nouvelle variante d'oxc qui porterait du code ne peut pas rejoindre silencieusement ce bras : le compilateur refusera le `match` non exhaustif. Une remarque au passage : `TSEnumDeclaration` produit un objet à l'exécution en TypeScript, et le lowering le traite comme effacé — le nom de l'enum n'est jamais lié. Au commit e67b10a, un composant qui lit un membre de l'enum

(e22_enum.tsx) est certifié sans réserve par toutes les règles : `Status` est une variable libre non liée, que l’environnement lit comme $\top$ (`lookup_missing_returns_top`), une sur-approximation sound qui ne coûte donc qu’en précision, jamais en soundness.

Les deux bras les plus fréquents sont courts :

Extrait 7.9 – `src/lowering/cfg_builder.rs`, l. 286–297 — instruction-expression et return

```

286     Statement::ExpressionStatement(es) => {
287         let expr = lower_expr(&es.expression, builder);
288         builder.push_stmt(Stmt::ExprStmt(expr, builder.span_at(es.span.start)));
289     }
290     Statement::ReturnStatement(ret) => {
291         let expr = ret
292             .argument
293             .as_ref()
294             .map(|e| lower_expr(e, builder))
295             .unwrap_or(Expr::Lit(Prim::Unit));
296         builder.seal_with(Terminator::Return(expr));
297     }

```

7.3.2 Le gabarit du if

Extrait 7.10 – `src/lowering/cfg_builder.rs`, l. 544–585 — l’abaissement d’un if

```

544 fn lower_if(
545     test: &Expression,
546     consequent: &Statement,
547     alternate: Option<&Statement>,
548     builder: &mut BlockBuilder,
549 ) {
550     let cond = lower_expr(test, builder);
551     let span = builder.span_at(test.span().start);
552     let then_block = builder.new_block();
553     let else_block = builder.new_block();
554     let join_block = builder.new_block();
555
556     let branch_id = builder.seal_with(Terminator::Branch {
557         cond,
558         then_: then_block,
559         else_: else_block,
560         span,
561     });
562     builder.add_edge(branch_id, then_block, EdgeKind::IfTrue);
563     builder.add_edge(branch_id, else_block, EdgeKind::IfFalse);
564
565     // Then branch
566     builder.start_block(then_block);
567     lower_stmt(consequent, builder);
568     if !builder.is_terminated() {
569         let b = builder.seal_with(Terminator::Jump(join_block));
570         builder.add_edge(b, join_block, EdgeKind::Unconditional);
571     }
572
573     // Else branch
574     builder.start_block(else_block);
575     if let Some(alt) = alternate {
576         lower_stmt(alt, builder);
577     }
578     if !builder.is_terminated() {

```

```

579     let b = builder.seal_with(Terminator::Jump(join_block));
580     builder.add_edge(b, join_block, EdgeKind::Unconditional);
581   }
582
583   // Continue at join block (may have no predecessors if both branches terminated)
584   builder.start_block(join_block);
585 }

```

Le code suit le protocole à la lettre. La condition est abaissée *avant* la réservation des blocs : si elle contient un `&&`, son diamant est construit d'abord, et le Branch ne porte plus que la variable temporaire du diamant. Trois blocs sont toujours réservés, même sans `else` : le bloc « sinon » vide contient alors un simple `Jump(join)`. Chaque branche qui ne s'est pas terminée d'elle-même saute vers la jonction. Figure 7.2 dessine le gabarit.

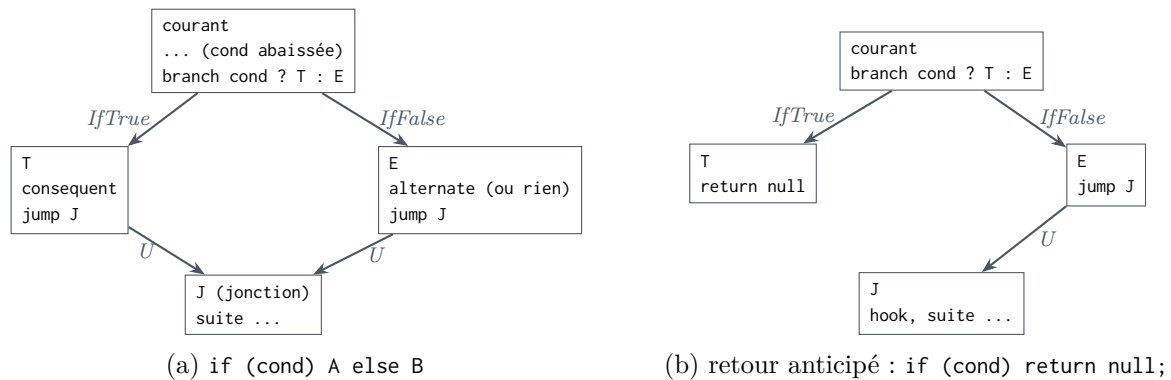


FIGURE 7.2 – Gabarit du `if`. Les trois blocs sont réservés avant d'abaisser les branches ; une branche qui se termine (à droite, `return`) n'a pas d'arête vers la jonction.

7.3.3 Le retour anticipé et la dominance

Le retour anticipé est le premier motif où la forme du CFG décide d'un diagnostic. Reprenons le programme du dossier de notes, qui contient aussi une boucle et un ternaire (nous les lirons plus loin) :

Extrait 7.11 – Retour anticipé, `for...of`, `continue`, ternaire (`/tmp/ch07/e2_control.tsx`)

```

1  import { useState } from "react";
2
3  export function Gate({ kind, items }: { kind: string; items: string[] }) {
4    if (kind === "none") {
5      return null;
6    }
7    const [n, setN] = useState(0);
8    let total = 0;
9    for (const it of items) {
10     if (!it) continue;
11     total += 1;
12   }
13   const label = n > 0 ? "some" : "none";
14   return <div title={label}>{total}</div>;
15 }

```

```

=== component Gate (param __p0) ===
render_cfg:
  B0:

```

```

    let __obj_56 = __p0
    let kind = __obj_56.kind @3:23
    let items = __obj_56.items @3:29
    => branch (kind Eq "none") ? B1 : B2
B1:
    => return null
B2:
    => jump B3
B3:
    let n = StateVal(0) @7:9
    let setN = StateSetter(0) @7:12
    let total = 0 @8:6
    expr items @9:19
    => jump B4
B4:
    => branch true ? B5 : B6
B5:
    let it = T @9:13
    => branch Not(it) ? B7 : B8
B6:
    => branch (n Gt 0) ? B10 : B11
B7:
    => jump B4
B8:
    => jump B9
B9:
    total := (total Add 1) @11:4
    expr total @11:4
    => jump B4
B10:
    let __t0 = "some" @13:24
    => jump B12
B11:
    let __t0 = "none" @13:33
    => jump B12
B12:
    let label = __t0 @13:8
    => return Native<div>({#0 title: label})[total]
edges: 0->1[T] 0->2[F] 2->3[U] 3->4[U] 4->5[T] 4->6[F] 5->7[T] 5->8[F]
        7->4[Back] 8->9[U] 9->4[Back] 6->10[T] 6->11[F] 10->12[U] 11->12[U]

```

Le if sans else donne les trois blocs B1 (le return null), B2 (le « sinon » vide) et B3 (la jonction). Le useState atterrit dans B3, que seule la branche B2 atteint. Il existe donc un chemin de l'entrée vers une sortie ($B0 \rightarrow B1$) qui ne passe pas par le hook : le bloc du hook ne domine pas toutes les sorties (définition 3.9), et c'est exactement ce que conditional-hook vérifie.

```

Gate (1 hooks) e2_control.tsx
error conditional-hook [hook:0] (line 7:8) this hook is called conditionally (not on every render path)

```

Comparons avec une garde qui lève une exception au lieu de retourner :

Extrait 7.12 – Une garde-throw (/tmp/ch07/c3_throw.tsx)

```

1 import { useState } from "react";
2
3 export function Guarded({ id }: { id?: string }) {
4   if (!id) throw new Error("id required");
5   const [v, setV] = useState(id);
6   return <b onClick={() => setV("")}>{v}</b>;
7 }

```

```

B0:
    let __obj_59 = __p0
    let id = __obj_59.id @3:26
    => branch Not(id) ? B1 : B2
B1:
    expr new#0 Error("id required") @4:11

```

```

=> unreachable
B2:
=> jump B3
B3:
  let v = StateVal(0) @5:9
  ...
edges: 0->1[T] 0->2[F] 2->3[U]

```

Le bloc B1 se termine par `Unreachable` : ce n'est pas une sortie. La seule sortie atteignable est dans B3, que le hook domine. L'analyseur le certifie (extrait de la sortie `-info -show-clean`) :

```

Guarded (2 hooks) c3_throw.tsx
warn frozen-initial-state [hook:0] var:id (line 5:8) state `v` is seeded from `id` and never re-synced. ...
verified conditional-hook all hooks run unconditionally, in a stable order

```

C'est la sémantique de React : un rendu qui lève n'est pas un rendu dans lequel un hook a été sauté, c'est un rendu avorté. Les deux programmes ne diffèrent que par `return` contre `throw`, et le lowering est le seul endroit où cette différence est traduite.

Définition 7.2: Bloc de jonction, bloc orphelin

Le *bloc de jonction* d'une construction est le bloc réservé où convergent ses branches (`if`, `ternaire`, `court-circuit`, `try`). Quand toutes les branches se sont terminées (`if (c) return 1; else return 2;`), la jonction est tout de même ouverte puis scellée — par `into_cfg` en `Return(undefined)` ou par les instructions qui suivent — et reste dans `blocks` sans aucun prédécesseur : c'est un *bloc orphelin*.

Piège

« **Tous les blocs** » n'est pas « **tous les blocs exécutables** ». Un bloc orphelin terminé par `Return` n'est pas une sortie. Tout consommateur qui quantifie sur les sorties (dominance, environnement de sortie) doit passer par `CFG::reachable_blocks`, comme le fixe ADR-25 §3; le test `reachable_blocks_excludes_an_orphaned_join` (`src/lowering/cfg_builder.rs`, l. 1348–1369) en garde la trace.

Remarque

Les identifiants de blocs sont alloués au moment de la réservation, *avant* l'abaissement des sous-structures. Dans `Gate`, la sortie de boucle B6 a été réservée avant les blocs du corps B7–B9. Les blocs imbriqués d'une branche ont donc des identifiants supérieurs à la jonction qui la suit : l'ordre des identifiants n'est pas l'ordre du texte. Comme `CFG.blocks` est une `BTreeMap`, cet ordre est au moins déterministe (chapitre 6).

7.4 Les expressions : garder chaque lecture

7.4.1 Le contrat de `lower_expr`

`lower_expr` (`src/lowering/expr_lower.rs`, l. 143–615) prend une `Expression` d'oxc et rend une `Expr`. Son commentaire de tête fixe le contrat : « Branching expressions (`ternary`, `&&`, `||`, `??`) split builder into new blocks and return a `Var temp`. All other expressions lower structurally. » Une expression peut donc faire deux choses de plus que rendre une valeur : *émettre* des instructions dans le bloc courant (une écriture, une lecture conservée) et *fendre* le bloc courant (les diamants de la section 7.5, la coupure d'await de la section 7.10). En revanche, elle ne scelle jamais un bloc en `Return` ou `Unreachable` : après `lower_expr`, un bloc est toujours ouvert. L'ordre d'évaluation est celui de JavaScript — gauche à droite, callee avant arguments, objet avant clé — à une exception près pour le JSX (plus bas).

7.4.2 Deux outils transverses

Extrait 7.13 – `src/lowering/expr_lower.rs`, l. 18–36 — la valeur opaque et la lecture pour effet

```
18 /// An opaque value of unknown kind (T). Used for expressions the analysis does
19 /// not model (exotic operators, `this`/`super`, unhandled syntax). A typed
20 /// sentinel rather than a magic `Expr::Var("__opaque")`: it evaluates directly
21 /// to `StateValue::top()` instead of relying on a name lookup missing in the
22 /// env, so a real user variable can never collide with it and it never shows up
23 /// as a spurious free-variable capture.
24 fn opaque() -> Expr {
25     Expr::SummaryVal(SummaryValue::Top)
26 }
27
28 /// Lower an expression the enclosing composite cannot represent, keeping it as
29 /// a statement so its reads and side effects stay visible. Dropping a
30 /// sub-expression outright is not an over-approximation: the deps rules consume
31 /// a missing read as "this variable is not used" – a claim, not ignorance.
32 fn lower_for_effect(expr: &Expression, builder: &mut BlockBuilder) {
33     let span = builder.span_at(expr.span().start);
34     let lowered = lower_expr(expr, builder);
35     builder.push_stmt(Stmt::ExprStmt(lowered, span));
36 }
```

`opaque()` est la sentinelle $\top$: une valeur inconnue, typée (`SummaryVal(Top)`) plutôt qu'un nom magique, pour qu'aucune variable de l'utilisateur ne puisse entrer en collision avec elle et qu'elle n'apparaisse jamais comme une capture. `lower_for_effect` garde en `ExprStmt` une sous-expression que le composite englobant ne sait pas représenter. Son commentaire énonce le principe le plus important du fichier.

Invariant 7.2: Ne perdre aucune lecture

Toute sous-expression évaluée à l'exécution est présente dans l'IR, au moins comme `ExprStmt`. Jeter une sous-expression n'est pas une sur-approximation : les règles de deps lisent une lecture absente comme « cette variable n'est pas utilisée » — une affirmation, pas une ignorance.

Cet invariant explique une série de formes qui surprennent à la première lecture de l'IR : le défaut d'un motif émis en `expr`, la clé calculée d'un objet lue avant sa valeur, l'argument d'un `void` gardé, l'expression itérée d'un `for...of` lue dans le pré-en-tête, un argument `...spread` sorti de la liste d'arguments. Toutes appliquent la même règle. Section 7.14 montrera les quelques endroits où elle est encore violée.

7.4.3 Tour des formes

Tableau 7.2 résume ce que devient chaque famille d'expressions. Les lignes marquées d'une étoile sont détaillées ensuite.

Incréments. Le bras de `UpdateExpression` émet l'écriture puis rend la variable :

Extrait 7.14 – `src/lowering/expr_lower.rs`, l. 256–277 — `i++` et `-i`

```
256 // `i++` / `--i`: emit the write (`i = i ± 1`), then yield the variable.
257 // Prefix/postfix differ only in the *value* expression (new vs old); we
258 // return the post-write `Var` for both – a sound approximation for the
259 // numeric domain (the rare `a = i++` over-counts by one, never under).
260 Expression::UpdateExpression(upd) => match &upd.argument {
261     SimpleAssignmentTarget::AssignmentTargetIdentifier(id) => {
```

Source	IR
booléen, null, nombre, chaîne	Lit; entier de magnitude $< 2^{31} - 1$ en Int, sinon Float
gabarit 'q0\$e0q1'	chaîne d'additions gauche-associée (("q0" Add e0) Add "q1")
undefined, identifiant	Lit(Unit), Var(nom)
this, super	T
binaire	BinOp par lower_binop, exhaustif; ==/=== en Eq, !=/!== en Neq
unaire	Neg, Not, BitNot, TypeOf, Plus; void e $\rightarrow$ ExprStmt(e) puis Lit(Unit); delete o.f $\rightarrow$ MemberWriteo, f, Unit puis Lit(true); le reste en Unknown
i++, -i *	Assign i := i + 1 (ou -), valeur Var(i); sur un membre, MemberWrite de T
appel, new *	Call, Newid (nœud allouant depuis #158); argument spread émis pour effet
gabarit étiqueté	Call(tag, [interpolations])
affectation *	Assign / MemberWrite / motif, puis la valeur
séquence (a, b, c)	a, b en ExprStmt, valeur c
a?.b, f?.(x)	comme a.b, f(x)
x as T; x!, satisfies, <T>x, parenthèses	TAnnotated(x); transparents
objet, tableau *	ObjectLitid, ArrayLitid, arity, spread_at
flèche, fonction, classe	FnLitid, lower_class (section 7.9)
JSX *	CompApp ou NativeElem
?:, &&, , ??	diamants (section 7.5)
await e	coupure Await (section 7.10)
yield e	la valeur de e (générateurs non modélisés)
autre	T (_ => opaque())

TABLE 7.2 – L'abaissement des expressions, famille par famille.

```

262     let name = id.name.to_string();
263     let op = match upd.operator {
264         UpdateOperator::Increment => IrBinOp::Add,
265         UpdateOperator::Decrement => IrBinOp::Sub,
266     };
267     builder.push_stmt(Stmt::Assign {
268         var: name.clone(),
269         rhs: Expr::BinOp {
270             op,
271             lhs: Box::new(Expr::Var(name.clone())),
272             rhs: Box::new(Expr::Lit(Prim::Int(1))),
273         },
274         span: builder.span_at(upd.span.start),
275     });
276     Expr::Var(name)
277 }

```

Préfixe et postfixe rendent tous deux la valeur *après* écriture. Le commentaire parle d'une approximation sound « pour le domaine numérique » (« the rare `a = i++` over-counts by one, never under »). Le domaine numérique (chapitre 9) étant un treillis d'intervalles, décaler la valeur de `a` de `+1` ne l'*élargit* pas : cela lui substitue une autre valeur ponctuelle, qui peut se trouver hors de tout intervalle sound pour la vraie valeur. Vérifié au commit e67b10a : pour `let i = 5; const a = i++; if (a === 5) setX(1);`, `a` vaut 5 en JavaScript et l'appel est certain ; le témoin littéral (`const a = 5; if (a === 5) ...`) est bien signalé (`warn setter-in-render`), mais avec `i++` l'analyseur calcule `a = 6`, le raffinement d'égalité déclare la branche `then` morte, et le composant est `verified setter-in-render` — un faux négatif, au sens strict où l'invariant de soundness du projet l'interdit, provoqué par le lowering et non par le moteur.

Piège

« **Sound pour le domaine numérique** » n'est pas vrai en général. Le commentaire de liste 7.14 est optimiste : un test d'égalité sur la valeur *avant* écriture (`a === 5` ci-dessus) compare deux valeurs ponctuelles distinctes de l'intervalle, et le décalage de `+1` peut faire passer une branche certaine pour morte. L'exemple `e27_postinc.tsx` en est la preuve : c'est un faux négatif que le lowering introduit, indépendamment de tout ce que le moteur fait ensuite correctement.

Sur une cible membre (`o.f++`, `a[i]-`), la cellule n'est pas suivie : `MemberWrite` de `T` et valeur `T`. Toute autre cible tombe dans le bras `_ => opaque()` — nous y reviendrons à la section 7.14.

Arguments. Un argument `...spread` ne peut pas garder de position : au splice, les paramètres se lient par position, et deviner lequel reçoit l'étalement serait une affirmation. Il est donc émis pour effet et retiré de la liste :

Extrait 7.15 – `src/lowering/expr_lower.rs`, l. 617–634 — la liste d'arguments

```
617 /// Lower a call/`new` argument list. A `...spread` argument cannot keep a
618 /// position – parameters bind positionally when a callee is inlined – so it is
619 /// emitted as a statement instead of guessed into a slot, which preserves its
620 /// reads without claiming which parameter receives it.
621 fn lower_arguments(arguments: &[Argument], builder: &mut BlockBuilder) -> Vec<Expr> {
622     let mut args = vec![];
623     for a in arguments {
624         match a {
625             Argument::SpreadElement(sp) => lower_for_effect(&sp.argument, builder),
626             other => {
627                 if let Some(e) = other.as_expression() {
628                     args.push(lower_expr(e, builder));
629                 }
630             }
631         }
632     }
633     args
634 }
```

Un appel à arguments de type explicites (`useState<T>(x)`) est enveloppé dans `TSAnnotated` (`src/lowering/expr_lower.rs`, l. 636–647), que l'extracteur de hooks regarde à travers.

Extrait 7.16 – `src/lowering/expr_lower.rs`, l. 493–518 — affectation à un identifiant

Affectations

```
493 Expression::AssignmentExpression(assign) => {
494     let rhs_val = lower_expr(&assign.right, builder);
495     match assign_target_ident(&assign.left) {
496         Some(name) => {
497             // Reconstruct `x op= e` → `x = x op e`, but only for operators
498             // `lower_binop` maps faithfully (Add/Sub/Mul/Div). Other
499             // compounds (%=, **=, bitwise, logical) would alias onto the
500             // wrong IR op → unsound; havoc the target to Top instead.
501             let rhs = if assign.operator.is_assign() {
502                 rhs_val
503             } else if let Some(op) = faithful_compound_binop(assign.operator) {
504                 Expr::BinOp {
505                     op,
506                     lhs: Box::new(Expr::Var(name.clone())),
507                     rhs: Box::new(rhs_val),
508                 }
509             } else {
```

```

510         opaque()
511     };
512     builder.push_stmt(Stmt::Assign {
513         var: name.clone(),
514         rhs,
515         span: builder.span_at(assign.span.start),
516     });
517     Expr::Var(name)
518 }

```

Une affectation composée $x \text{ op} = e$ est reconstruite en $x := x \text{ op } e$, mais seulement quand l'opérateur a une traduction fidèle; sinon la cible est mise à $\top$. Le commentaire est *périmé* : il ne cite que Add/Sub/Mul/Div, alors que `faithful_compound_binop` (`src/lowering/expr_lower.rs`, l. 1161–1179) couvre aujourd'hui les quatre opérations, le reste %, la puissance **, les trois opérateurs bit à bit et les trois décalages. Seuls $\&\&=$, $||=$ et $??=$ tombent encore sur $\top$. Une cible membre produit un `MemberWrite` (de $\top$ pour un composé, la cellule n'étant pas suivie); une cible motif $[a, b] = xs$ passe par `lower_assignment_target`, miroir des motifs de liaison (section 7.8). Dans tous les cas, l'expression rend ensuite sa valeur; en position d'instruction, cette valeur devient un `ExprStmt`, d'où les paires `total := (total Add 1) / expr total` de Gate.

Chânage optionnel. $a?.b$ et $f?.(x)$ s'abaissent comme leurs versions non optionnelles (`src/lowering/expr_lower.rs`, l. 649–657). L'argument du commentaire vaut pour la *valeur* : un receveur non nul est un objet, et tout autre receveur donne déjà $\top$, qui couvre `undefined`. Pour les *effets*, $f?.(x)$ devient un appel inconditionnel; si f est nul, l'appel ne s'exécute pas en vrai. Cela ne peut qu'ajouter des comportements (des faux positifs possibles), jamais en retirer.

7.4.4 Objets, tableaux, clés synthétiques

Extrait 7.17 – `src/lowering/expr_lower.rs`, l. 352–394 — le littéral d'objet

```

352 Expression::ObjectExpression(obj) => {
353     let id = builder.next_expr_id();
354     let mut fields: Vec<(String, Expr)> = vec![];
355     for prop in &obj.properties {
356         match prop {
357             ObjectPropertyKind::ObjectProperty(p) => {
358                 let key = match &p.key {
359                     // A getter/setter runs code on every read: `o.x`
360                     // is whatever the body returns, not the function
361                     // literal sitting in the property. Kept under a
362                     // synthetic key, like a spread – the body stays
363                     // visible, the name resolves to nothing.
364                     _ if p.kind != PropertyKind::Init => {
365                         synthetic_key(builder, "[accessor]")
366                     }
367                     PropertyKey::StaticIdentifier(ident) => ident.name.to_string(),
368                     PropertyKey::StringLiteral(s) => s.value.to_string(),
369                     // Computed key `{ [k]: v }`: the key expression
370                     // runs, and `v` is still in the object – under a
371                     // synthetic name, since the real one is unknown.
372                     other => {
373                         if let Some(e) = other.as_expression() {
374                             lower_for_effect(e, builder);
375                         }
376                         synthetic_key(builder, "[computed]")
377                     }
378                 }

```

```

379         let value = lower_expr(&p.value, builder);
380         fields.push((key, value));
381     }
382     // `{ ...opts }` forwards every one of `opts`' fields. Keep
383     // it under a synthetic key exactly as JSX spread does -
384     // dropping it lost the read of `opts` and any setter it
385     // carries.
386     ObjectPropertyKind::SpreadProperty(sp) => {
387         let key = synthetic_key(builder, SPREAD_KEY_PREFIX);
388         let value = lower_expr(&sp.argument, builder);
389         fields.push((key, value));
390     }
391 }
392 }
393 Expr::ObjectLit { id, fields }
394 }

```

L'identifiant d'allocation est tiré *avant* les valeurs des champs. Trois sortes de champs ne peuvent pas être rangées sous leur vrai nom : un accesseur (`get x()`), dont la lecture exécute un corps ; une clé calculée (`[k]: v`), dont le nom est inconnu ; un spread (`...opts`), qui apporte un nombre inconnu de champs. Tous trois sont rangés sous une *clé synthétique* fabriquée par `synthetic_key` (`src/lowering/expr_lower.rs`, l. 38–44) : un préfixe (`[accessor]`, `[computed]`, ...) suivi d'un `ExprId` consommé pour l'occasion. Aucun `FieldAccess` réel ne peut atteindre ces clés, mais les parcours qui visitent les champs *par valeur* (variables libres, setters qui s'échappent) les voient. Une clé numérique (`1: x`) passe par le bras « calculé ». L'accès à un membre écrit avant un spread est traité à part par le domaine (`members_after_last_spread`, chapitre 6).

Les tableaux (`src/lowering/expr_lower.rs`, l. 395–439) suivent la même logique : `elems` garde les éléments visibles, un spread y figure comme un élément « conteneur » dont la position est notée dans `spread_at`, une élision compte dans la longueur sans produire d'élément, et `arity` vaut `Exact(n)` sauf en présence d'un spread (`AtLeast(n)`). C'est le dernier endroit où la longueur de la source est connaissable ; elle alimente `DepsArg` (chapitre 6, #104).

7.4.5 JSX

Extrait 7.18 – `src/lowering/expr_lower.rs`, l. 799–830 — un élément JSX : composant ou élément hôte

```

799 fn lower_jsx_element(jsx: &JSXElement, builder: &mut BlockBuilder) -> Expr {
800     let name = jsx_element_name(&jsx.opening_element.name);
801     let children: Vec<Expr> = jsx
802         .children
803         .iter()
804         .filter_map(|c| lower_jsx_child(c, builder))
805         .collect();
806     let (props, prop_spans) = lower_jsx_props(&jsx.opening_element.attributes, builder);
807
808     if name.chars().next().is_some_and(|c| c.is_uppercase()) || name.contains('.') {
809         // React semantics: nested JSX children ARE `props.children`.
810         // Dropping them here would erase the whole subtree from the CFG
811         // (<Dialog><Select onChange={setX}/></Dialog> - the Select
812         // would never be visited, its escaping setter never havocked).
813         let mut props = props;
814         if !children.is_empty()
815             && let Expr::ObjectLit { fields, .. } = &mut props
816         {
817             let id = builder.next_expr_id();
818             fields.push((

```

```

819         "children".to_string(),
820         Expr::ArrayLit {
821             id,
822             // Synthetic container, not a written array literal: it
823             // stands for the children the walk kept, so it claims only
824             // a lower bound on anything in the source.
825             arity: Arity::AtLeast(children.len()),
826             elems: children,
827             spread_at: vec![],
828         },
829     ));
830 }

```

Rappel React : composants, éléments hôtes, children

En JSX, un nom qui commence par une majuscule ou qui est pointé (`<Ctx.Provider>`, `<motion.div>`) désigne un composant ; un nom en minuscule désigne un élément hôte du DOM. Les enfants d'un composant lui sont passés dans `props.children`, et c'est lui qui décide de les rendre ; les enfants d'un élément hôte sont rendus par React DOM.

Le lowering suit cette règle : `CompApp` si le nom commence par une majuscule (au sens Unicode) ou contient un point, `NativeElem` sinon. Pour un `CompApp`, les enfants rejoignent les props sous la clé `children`, dans un tableau synthétique d'arité `AtLeast` ; les jeter effacerait tout le sous-arbre du CFG, avec les setters qui s'y échappent. L'origine du composant (`origin`) est lue dans les imports du fichier au moment du lowering, et nulle part ailleurs (#7). Les props forment un `ObjectLit` ; un attribut sans valeur vaut `Lit(true)`, un spread d'attribut prend une clé `...N`. Pour un `NativeElem`, `prop_spans` enregistre la position de chaque prop `onX`, que l'extracteur de handlers relira. Un fragment `<>...</>` devient un `NativeElem` de balise "Fragment" ; un `<React.Fragment>` explicite, pointé, devient un `CompApp`.

Piège

Les enfants sont abaissés avant les props. `lower_jsx_element` abaisse les enfants puis les attributs, alors que JavaScript (`jsx(type, props)`) évalue les attributs d'abord. C'est sans effet sur les valeurs, mais visible dans l'ordre des `ExprId` et des instructions : un ternaire dans un enfant fend les blocs avant celui d'une prop. Autre détail d'ordre : `lower_jsx_props` tire l'identifiant de l'objet de props avant d'abaisser ses valeurs, si bien que dans le compteur (section 7.11) l'objet a l'identifiant 0 et la flèche de `onClick` l'identifiant 1.

7.5 Les diamants : ternaire et court-circuits

7.5.1 Le problème : des effets conditionnels dans une expression

Un `&&` en instruction est un `if` déguisé : `open && console.log(name)` n'appelle `console.log` que si `open` est vrai. Une IR qui représenterait `a && b` par un nœud d'expression plat perdrait ce fait : il faudrait que l'évaluateur rejoue l'exécution conditionnelle de `b`, ou bien il compterait l'appel comme inconditionnel. Le lowering choisit de fendre le bloc.

Extrait 7.19 – `?.`, `??`, `&&` en instruction, `useEffect` (`/tmp/ch07/e3_logical.tsx`)

```

1 import { useState, useEffect } from "react";
2
3 export function Profile({ user, fallback }: { user?: { name?: string }; fallback: string }) {
4     const [open, setOpen] = useState(false);
5     const name = user?.name ?? fallback;
6     open && console.log(name);

```

```

7   useEffect(() => {
8       setOpen(true);
9   }, []);
10  return <span>{name}</span>;
11  }

```

```

render_cfg:
  B0:
    let __obj_70 = __p0
    let user = __obj_70.user @3:26
    let fallback = __obj_70.fallback @3:32
    let open = StateVal(0) @4:9
    let setOpen = StateSetter(0) @4:15
    let __t0 = user.name @5:15
    => branch __t0 ? B2 : B1
  B1:
    __t0 := fallback @5:29
    => jump B2
  B2:
    let name = __t0 @5:8
    let __t1 = open @6:2
    => branch __t1 ? B3 : B4
  B3:
    __t1 := console.log(name) @6:10
    => jump B4
  B4:
    expr __t1 @6:2
    expr HookMarker(1, Undefined) @7:2
    => return Native<span>({#2 })[name]
edges: 0->2[F] 0->1[F] 1->2[U] 2->3[T] 2->4[T] 3->4[U]

```

`user?.name` est un simple `FieldAccess`. Le `??` et le `&&` ont chacun produit un *diamant* : un temporaire reçoit l'opérande gauche, un `Branch` sur ce temporaire choisit entre un bloc qui évalue l'opérande droit (et réécrit le temporaire) et la jonction. L'instruction `open && ...` se termine par `expr __t1` dans la jonction B4.

7.5.2 Le code

Le ternaire lie son temporaire par deux `Let`, un par bras :

Extrait 7.20 – `src/lowering/expr_lower.rs`, l. 675–700 — le ternaire (début ; les deux bras suivent le même motif)

```

675 /// `a ? b : c` splits into three blocks:
676 ///
677 ///   current: Branch(a, then, else)
678 ///   then:    Let __tN = b; Jump(join)
679 ///   else:    Let __tN = c; Jump(join)
680 ///   join:    Var(__tN) ← returned
681 ///
682 /// Analysis correctly joins stability(b) ∪ stability(c) at the join block.
683 fn lower_ternary(cond: &ConditionalExpression, builder: &mut BlockBuilder) -> Expr {
684     let test = lower_expr(&cond.test, builder);
685     let then_id = builder.new_block();
686     let else_id = builder.new_block();
687     let join_id = builder.new_block();
688     let tmp = builder.fresh_temp();
689
690     let span = builder.span_at(cond.test.span().start);
691     let bid = builder.seal_with(Terminator::Branch {
692         cond: test,
693         then_: then_id,
694         else_: else_id,

```

```

695     span,
696   });
697   builder.add_edge(bid, then_id, EdgeKind::IfTrue);
698   builder.add_edge(bid, else_id, EdgeKind::IfFalse);
699
700   builder.start_block(then_id);

```

La suite (`src/lowering/expr_lower.rs`, l. 701–724) ouvre `then_id`, abaisse la conséquence, émet `Let __tN = cons` avec la position de la conséquence, scelle un `Jump(join)`; fait de même pour l'alternative; ouvre la jonction et rend `Var(__tN)`. Les court-circuits lient le temporaire par un `Let` de l'opérande gauche *puis* un `Assign` dans le bloc droit :

Extrait 7.21 – `src/lowering/expr_lower.rs`, l. 739–795 — `&&`, `||` et `??`

```

739 fn lower_logical(log: &LogicalExpression, builder: &mut BlockBuilder) -> Expr {
740     let tmp = builder.fresh_temp();
741     let span = builder.span_at(log.left.span().start);
742     let left = lower_expr(&log.left, builder);
743     builder.push_stmt(Stmt::Let {
744         var: tmp.clone(),
745         rhs: left,
746         span,
747     });
748
749     let rhs_id = builder.new_block();
750     let join_id = builder.new_block();
751
752     let (then_, else_) = match log.operator {
753         LogicalOperator::And => (rhs_id, join_id), // truthy → rhs; falsy → join (keep a)
754         LogicalOperator::Or | LogicalOperator::Coalesce => (join_id, rhs_id), // truthy → join (keep a);
755         ↪ falsy → rhs
756     };
757
758     let bid = builder.seal_with(Terminator::Branch {
759         cond: Expr::Var(tmp.clone()),
760         then_,
761         else_,
762         span,
763     });
764     builder.add_edge(
765         bid,
766         then_,
767         if then_ == rhs_id {
768             EdgeKind::IfTrue
769         } else {
770             EdgeKind::IfFalse
771         },
772     );
773     builder.add_edge(
774         bid,
775         else_,
776         if else_ == rhs_id {
777             EdgeKind::IfFalse
778         } else {
779             EdgeKind::IfTrue
780         },
781     );
782
783     builder.start_block(rhs_id);
784     let right_span = builder.span_at(log.right.span().start);
785     let right = lower_expr(&log.right, builder);

```

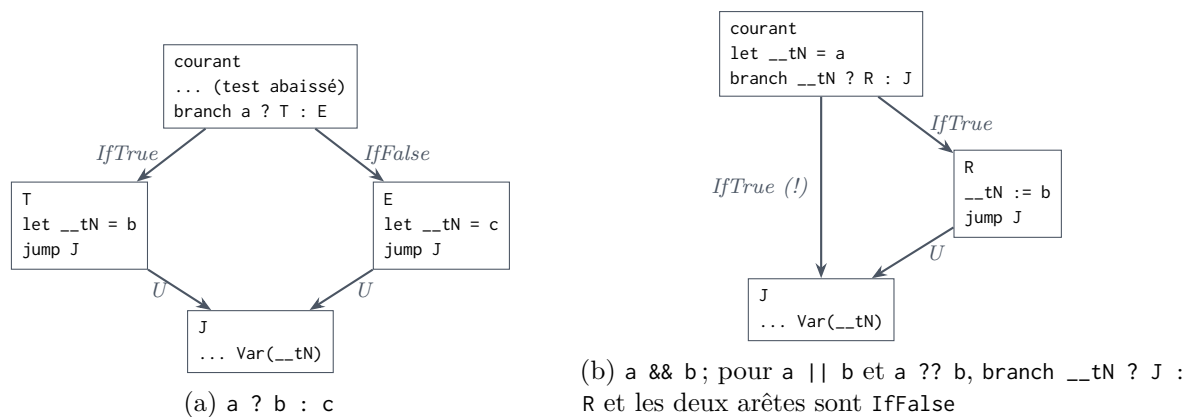


```

785     builder.push_stmt(Stmt::Assign {
786         var: tmp.clone(),
787         rhs: right,
788         span: right_span,
789     });
790     let r = builder.seal_with(Terminator::Jump(join_id));
791     builder.add_edge(r, join_id, EdgeKind::Unconditional);
792
793     builder.start_block(join_id);
794     Expr::Var(tmp)
795 }

```

Figure 7.3 dessine les deux gabarits.

FIGURE 7.3 – Les deux diamants. Le temporaire `__tN` est tiré du compteur du constructeur ; la valeur de l’expression est `Var(__tN)` dans la jonction.**Décision : ADR-20 §1 — garder le diamant**

ADR-20 range le diamant parmi les « non-changements délibérés » : ne pas introduire de nœud plat `LogicalOp`. Le diamant modélise les *effets* du court-circuit ($a \ \&\& \ \text{setX}()$ n’exécute `setX()` que sur la branche vraie) ; un nœud plat obligerait l’évaluateur à rejouer cette exécution conditionnelle ou à perdre l’appel, un faux négatif. Il n’achèterait pas de précision : le raffinement relationnel à travers $\&\&/||$ est déjà perdu, puisque le `Branch` porte `Var(__tN)` et jamais les comparaisons des opérandes. Le coût accepté est que `infinite_loop::expand_guard` doit reconstruire la structure des opérandes à partir du diamant.

7.5.3 Deux subtilités**Piège**

La polarité des arêtes d’un court-circuit. Dans `Profile`, les deux arêtes sortantes du `??` sont *IfFalse* ($0 \rightarrow 2[F] \ 0 \rightarrow 1[F]$) et les deux arêtes du $\&\&$ sont *IfTrue* ($2 \rightarrow 3[T] \ 2 \rightarrow 4[T]$). Le code (`src/lowering/expr_lower.rs`, l. 763–780) calcule la nature des deux arêtes à partir de l’arête vers `rhs` : l’arête vers la jonction reçoit la même. La nature de l’arête encode donc l’opérateur — dans quel sens on entre dans l’opérande droit —, pas la valeur de vérité de la condition sur cette arête. `expand_guard` (`src/engine/guards.rs`, l. 1018–1029) ne lit que l’arête vers `rhs`, et `site_guards` ne lit les polarités que sur des chaînes à prédécesseur unique, alors que la jonction d’un diamant en a toujours deux : le défaut est latent. Le point fixe, lui, raffine l’environnement d’après `then_/else_` du terminateur, ce qui est correct. Un nouveau consommateur qui lirait *IfTrue* comme « la condition est vraie ici » se tromperait sur l’arête

qui court-circuite.

Piège

?? n'est pas ||. Le lowering donne à `a ?? b` exactement la forme de `a || b` : `b` est évalué quand `a` est *falsy*, alors que JavaScript ne l'évalue que quand `a` est *nullish*. Pour le graphe, c'est un chemin de trop, donc sans danger. Mais le point fixe raffine le temporaire sur la branche `then_` (qui va à la jonction pour `??`) par « `__tN` est truthy », ce qui retire `0`, `""` et `false` — or `0 ?? b` vaut `0` et arrive à la jonction *sans* passer par le bloc droit. La valeur `0` disparaît : c'est une sous-approximation, vérifiée à la section 7.14.

Rappel React : conditions et règles des hooks

Un hook appelé dans l'opérande droit d'un court-circuit ou dans un bras de ternaire (flag `&& useEffect(...), c ? useState(1) : null`) est un hook conditionnel, interdit par les règles des hooks [Met26d]. Le diamant place l'appel dans un bloc qui ne domine pas la sortie ; encore faut-il que l'extracteur le reconnaisse comme un hook (sections 7.11 et 7.14).

7.6 Les boucles, `break` et `continue`

7.6.1 `while` : l'arête arrière

Extrait 7.22 – Une boucle `while` (/tmp/ch07/c2_while.tsx)

```
1 export function Digits({ n }: { n: number }) {
2   let k = n;
3   let count = 0;
4   while (k > 9) {
5     k = k / 10;
6     count++;
7   }
8   return <span>{count}</span>;
9 }
```

```
B0:
  let __obj_23 = __p0
  let n = __obj_23.n @1:25
  let k = n @2:6
  let count = 0 @3:6
  => jump B1
B1:
  => branch (k Gt 9) ? B2 : B3
B2:
  k := (k Div 10) @5:4
  expr k @5:4
  count := (count Add 1) @6:4
  expr count @6:4
  => jump B1
B3:
  => return Native<span>({#0 })[count]
edges: 0->1[U] 1->2[T] 1->3[F] 2->1[Back]
```

Extrait 7.23 – `src/lowering/cfg_builder.rs`, l. 603–633 — la boucle `while`

```
603 fn lower_while(test: &Expression, body: &Statement, builder: &mut BlockBuilder) {
604   let header = builder.new_block();
605   let body_block = builder.new_block();
606   let exit_block = builder.new_block();
607 }
```

```

608     let pre = builder.seal_with(Terminator::Jump(header));
609     builder.add_edge(pre, header, EdgeKind::Unconditional);
610
611     builder.start_block(header);
612     let cond = lower_expr(test, builder);
613     let span = builder.span_at(test.span().start);
614     let h = builder.seal_with(Terminator::Branch {
615         cond,
616         then_: body_block,
617         else_: exit_block,
618         span,
619     });
620     builder.add_edge(h, body_block, EdgeKind::IfTrue);
621     builder.add_edge(h, exit_block, EdgeKind::IfFalse);
622
623     builder.start_block(body_block);
624     builder.push_loop(exit_block, Some(header));
625     lower_stmt(body, builder);
626     builder.pop_loop();
627     if !builder.is_terminated() {
628         let b = builder.seal_with(Terminator::Jump(header));
629         builder.add_edge(b, header, EdgeKind::Back);
630     }
631
632     builder.start_block(exit_block);
633 }

```

Trois blocs sont réservés : l'*en-tête* (B1), qui porte le test ; le corps (B2) ; la sortie (B3). Le bloc qui précède l'en-tête — le *pré-en-tête*, ici B0 — y saute sans condition. La condition est abaissée *dans* l'en-tête : si elle contient un `&&` ou un ternaire, le diamant naît dans l'en-tête, et c'est sa jonction qui porte le Branch de la boucle ; l'arête de retour du corps vise toujours le premier bloc de l'en-tête, donc le test est bien réévalué à chaque tour. Le corps qui termine normalement revient à l'en-tête par une arête Back.

Définition 7.3: Arête arrière

Une *arête arrière* (`EdgeKind::Back`) est une arête que le lowering émet pour *revenir* vers l'en-tête d'une boucle : fin de corps d'un `while` ou d'un `for...of`, retour de l'`update` d'un `for`, branche vraie du test d'un `do...while`, et tout continue. C'est le seul endroit où le point fixe applique le widening (définition 3.6, chapitre 12) : le lowering décide donc, par la nature des arêtes qu'il émet, où l'analyse accélère sa convergence.

7.6.2 Les trois autres boucles

for (init; test; update) (`src/lowering/cfg_builder.rs`, l. 635–702). `init` est abaissé dans le bloc courant ; une déclaration `let i = 0` passe par le déclarateur ordinaire et garde sa position, une expression est émise en `ExprStmt` sans position. Quatre blocs sont réservés, dans cet ordre : en-tête, corps, `update`, sortie. Un test absent vaut `Lit(Bool(true))`. Le corps qui termine normalement va à l'`update` par une arête `Unconditional`, et l'`update` revient à l'en-tête par une arête `Back`. Le point décisif est la cible d'un `continue` : « `continue` in a `for` runs the `update` before the next test », d'où le cadre `push_loop(exit_block, Some(update_block))`. Aucune portée lexicale n'est modélisée : le `i` de `for (let i...)` est lié par un `let` dans le pré-en-tête et reste visible après la boucle pour l'analyse.

do...while (`src/lowering/cfg_builder.rs`, l. 322–354). Le test reçoit son propre bloc, parce qu'un `continue` l'exécute et qu'il doit donc être une cible de saut ; la sortie a ainsi un vrai

prédécesseur même quand le corps sort toujours plus tôt. Le pré-en-tête saute au corps, le corps au test, et le test porte un `Branch(cond, corps, sortie)` dont l'arête « vraie » est de nature `Back`, pas `IfTrue` : c'est l'arête de retour.

for...in / for...of (`lower_iter_loop`, `src/lowering/cfg_builder.rs`, l. 713–772). Le nombre d'itérations est inconnu. L'expression itérée est évaluée une fois dans le pré-en-tête, en `ExprStmt` (une lecture conservée pour les deps) ; l'en-tête porte un `Branch(Lit(true))` sans position, que le point fixe ne raffine pas (il ne raffine que les `BinOp`, `Var` et `Not(Var)`, `src/engine/cfg_analyzer.rs`, l. 212–292) ; et la variable de boucle est liée à $\top$ en tête de corps, de sorte qu'une variable de boucle qui masque une liaison extérieure n'est pas lue comme une capture de celle-ci. Dans `Gate` (liste 7.11), on lit `expr items` dans B3, `branch true` dans B4 et `let it = T` dans B5.

Figure 7.4 rassemble les quatre gabarits.

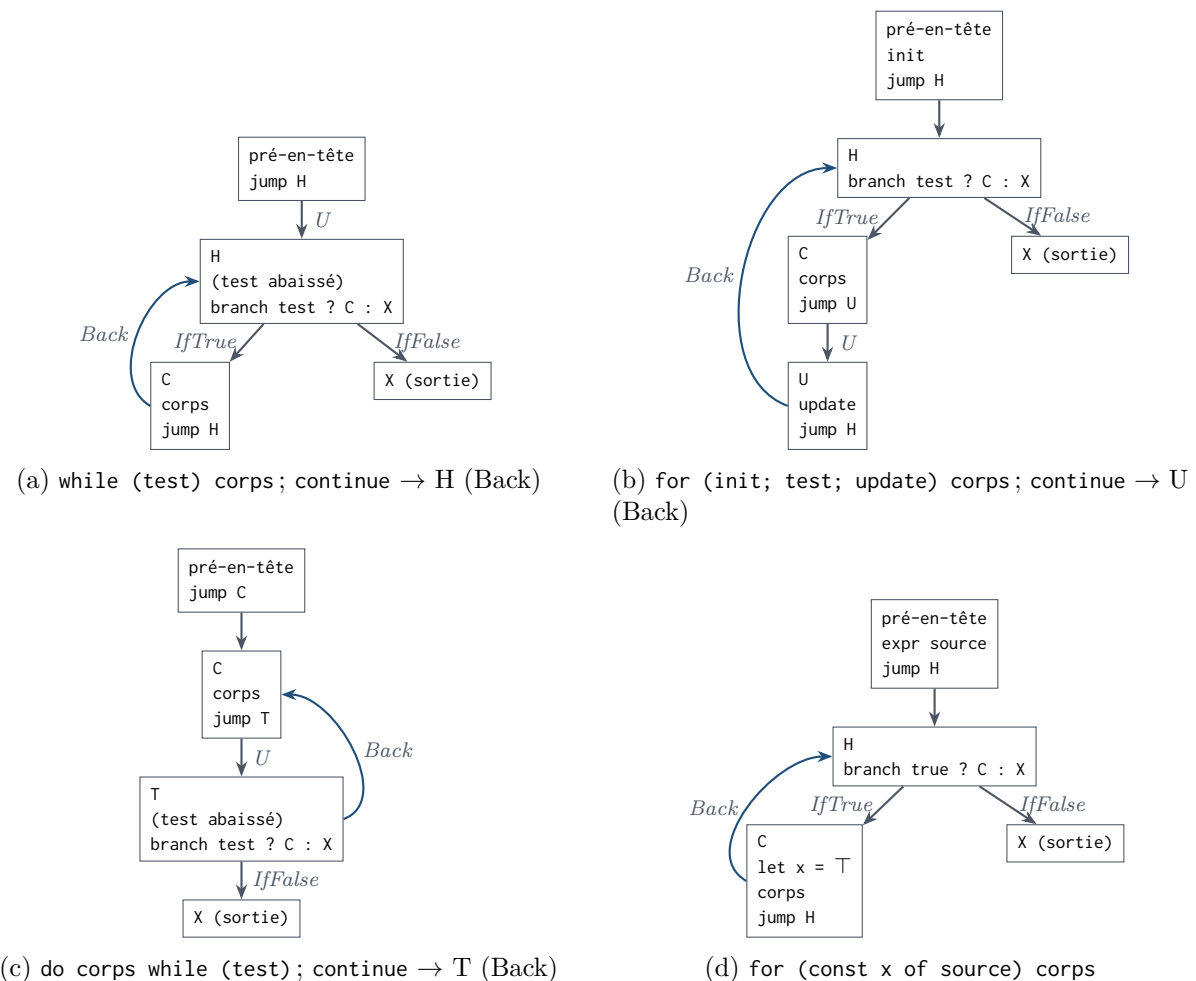


FIGURE 7.4 – Les gabarits des quatre boucles. Les arêtes arrière (en bleu) sont les seules où le point fixe applique le widening. Un `break` va toujours à X par une arête `Unconditional`.

7.6.3 break, continue, labels

Extrait 7.24 – `src/lowering/cfg_builder.rs`, l. 381–397 — `break` et `continue` sont de vraies arêtes

```
381 // A `break`/`continue` is a real edge to the loop's exit or header.
382 // Sealing `Unreachable` instead dropped the state the jump carries out
```

```

383 // of the loop, and left an exit the CFG says is unreachable while the
384 // real one has no edge – all-paths reasoning then reads a phantom exit
385 // set ( `ExitDominance`, `must_setter_on_all_paths` ).
386 Statement::BreakStatement(b) => {
387   let target = builder.break_target(b.label.as_ref().map(|l| l.name.as_str()));
388   jump_out(builder, target, EdgeKind::Unconditional);
389 }
390 // A `continue` is a back edge of the loop it continues: the fixpoint
391 // widens only on `Back` edges, and a loop whose counter advances on
392 // the `continue` path alone ( `left += 1; continue;` ) never converged
393 // through an `Unconditional` one.
394 Statement::ContinueStatement(c) => {
395   let target = builder.continue_target(c.label.as_ref().map(|l| l.name.as_str()));
396   jump_out(builder, target, EdgeKind::Back);
397 }

```

Chaque construction « cassable » empile un `LoopFrame` (liste 7.4) : sa cible de `break`, sa cible de `continue` (None pour un `switch`) et son label éventuel. La résolution (`src/lowering/cfg_builder.rs`, l. 132–149) remonte la pile : avec un label, le cadre qui porte ce label ; sans label, le cadre le plus proche pour `break`, et le cadre le plus proche qui *accepte* un `continue` — on saute donc les `switch` — pour `continue`. `jump_out` (`src/lowering/cfg_builder.rs`, l. 587–601) scelle un `Jump` vers la cible, ou `Unreachable` s’il n’y en a pas (JavaScript invalide : on n’invente pas d’arête).

Le commentaire donne les deux raisons de cette forme. Sceller un `break` en `Unreachable`, comme le faisait une version antérieure, perdait l’état que le saut emporte hors de la boucle, et laissait une sortie que le CFG déclarait inatteignable pendant que la vraie n’avait pas d’arête : tout raisonnement « sur tous les chemins » lisait un ensemble de sorties fantôme. Et un `continue` est une arête `Back` parce que le widening n’a lieu que sur ces arêtes : une boucle dont le compteur n’avance que sur le chemin du `continue` (`left += 1; continue;`) ne convergeait pas à travers une arête `Unconditional`.

Un label n’est honoré que s’il étiquette directement une boucle (`src/lowering/cfg_builder.rs`, l. 364–380) : `set_pending_label` le met en attente, et le prochain `push_loop` le consomme. Un label de bloc (`1: ... break 1;`) ou de `switch` est ignoré, et le `break 1` correspondant, sans cible, devient `Unreachable` — une sous-approximation assumée par le commentaire (« *vanishingly rare in components* »).

Exemple 7.1: Boucles imbriquées étiquetées

```

1 export function Loops({ rows }: any) {
2   let n = 0;
3   outer: for (let i = 0; i < 3; i++) {
4     do {
5       if (rows[i]) continue outer;
6       n++;
7     } while (n < 10);
8   }
9   return <div>{n}</div>;
10 }

```

```

B0:
  let __obj_22 = __p0
  let rows = __obj_22.rows @1:24
  let n = 0 @2:6
  let i = 0 @3:18
  => jump B1
B1:
  => branch (i Lt 3) ? B2 : B4

```

```

B2:
=> jump B5
B3:
i := (i Add 1) @3:32
expr i
=> jump B1
B4:
=> return Native<div>({#0 })[n]
B5:
=> branch rows[i] ? B8 : B9
B6:
=> branch (n Lt 10) ? B5 : B7
B7:
=> jump B3
B8:
=> jump B3
B9:
=> jump B10
B10:
n := (n Add 1) @6:6
expr n @6:6
=> jump B6
edges: 0->1[U] 1->2[T] 1->4[F] 2->5[U] 5->8[T] 5->9[F] 8->3[Back] 9->10[U]
10->6[U] 6->5[Back] 6->7[F] 7->3[U] 3->1[Back]

```

Le `for` a réservé B1 (en-tête), B2 (corps), B3 (update) et B4 (sortie) avant d'abaisser son corps; le `do...while` a ensuite réservé B5 (corps), B6 (test) et B7 (sortie). Le corps du `for` ne contient qu'un `jump B5`: `do...while` scelle d'abord le bloc courant vers son propre corps. `continue` outer vise l'update B3 du `for`, par une arête `Back` (8->3); un `continue` non étiqueté aurait visé le test B6. On compte trois arêtes arrière (8->3, 6->5, 3->1), donc trois points de widening potentiels. Enfin, `i++` donne `i := (i Add 1)` avec position, puis `expr i` sans position : l'`ExprStmt` de l'update est émis avec `None` (`src/lowering/cfg_builder.rs`, l. 696). L'analyseur ne signale rien sur ce composant.

Remarque

Dans un `for`, un chemin `continue` traverse *deux* arêtes arrière : `continue` → `update`, puis `update` → `en-tête`. C'est sans conséquence pour la correction (le widening est idempotent sur un point fixe atteint), mais utile à savoir quand on compte les arêtes `Back` d'un CFG.

7.7 switch et try : deux sous-approximations corrigées

Ces deux constructions ont une histoire commune : leur premier lowering était une ligne droite, et dans les deux cas cette ligne droite affirmait des choses fausses. Leur forme actuelle vient de #1 et #2.

7.7.1 Le switch : une chaîne de dispatchs opaques

Extrait 7.25 – `switch` avec fall-through, puis `try/catch/finally` (`/tmp/ch07/e4_switch_try.tsx`)

```

1 import { useState } from "react";
2
3 export function Status({ code }: { code: number }) {
4   const [msg, setMsg] = useState("");
5   switch (code) {
6     case 1:
7       setMsg("one");
8       break;
9     case 2:

```

```

10     setMsg("two");
11     default:
12     setMsg("other");
13 }
14 try {
15     risky();
16 } catch (e) {
17     report(e);
18 } finally {
19     done();
20 }
21 return <p>{msg}</p>;
22 }

```

```

B0:
  let __obj_58 = __p0
  let code = __obj_58.code @3:25
  let msg = StateVal(0) @4:9
  let setMsg = StateSetter(0) @4:14
  expr code
  => branch true ? B2 : B5
B1:
  => branch T ? B7 : B8
B2:
  expr setMsg("one") @7:6
  => jump B1
B3:
  expr setMsg("two") @10:6
  => jump B4
B4:
  expr setMsg("other") @12:6
  => jump B1
B5:
  => branch true ? B3 : B6
B6:
  => branch true ? B4 : B1
B7:
  expr risky() @15:4
  => jump B9
B8:
  let e = T @16:11
  expr report(e) @17:4
  => jump B9
B9:
  expr done() @19:4
  => return Native<p>({#0 })[msg]
edges: 0->2[T] 0->5[F] 5->3[T] 5->6[F] 6->4[T] 6->1[F] 2->1[U] 3->4[U] 4->1[U]
        1->7[T] 1->8[F] 7->9[U] 8->9[U]

```

Extrait 7.26 – src/lowering/cfg_builder.rs, l. 786–838 — la chaîne de dispatchs du switch

```

786 // One block per case, fronted by a chain of opaque dispatches: dispatch `i`
787 // either enters case `i` or moves on to dispatch `i + 1`, and the last one
788 // falls to the exit. Which case matches is not a truthiness test on the
789 // discriminant, so every dispatch is opaque, and no case has to match – the
790 // exit is reachable without entering any of them (a `default` clause is
791 // over-approximated as optional).
792 //
793 // Chaining the consequents into one straight line instead was an
794 // under-approximation twice over: entering `case 2` without running `case 1`
795 // was unrepresentable, and the first `break` sealed the chain, so every
796 // later case was dropped from the CFG entirely.
797 let case_blocks: Vec<BlockId> = sw.cases.iter().map(|_| builder.new_block()).collect();
798 // The first dispatch is the block the discriminant was evaluated in; the
799 // rest need blocks of their own.

```

```

800     let later_dispatch: Vec<BlockId> = (1..case_blocks.len())
801         .map(|_| builder.new_block())
802         .collect();
803
804     for (i, &case_block) in case_blocks.iter().enumerate() {
805         let skip_to = later_dispatch.get(i).copied().unwrap_or(exit_block);
806         let d = builder.seal_with(Terminator::Branch {
807             cond: Expr::Lit(Prim::Bool(true)),
808             then_: case_block,
809             else_: skip_to,
810             span: None,
811         });
812         builder.add_edge(d, case_block, EdgeKind::IfTrue);
813         builder.add_edge(d, skip_to, EdgeKind::IfFalse);
814         if let Some(&next) = later_dispatch.get(i) {
815             builder.start_block(next);
816         }
817     }
818
819     // Lower each case's body into its own block. `break` is the generic
820     // statement arm now that the switch pushes a frame, so a guarded
821     // `if (x) break;` reaches the exit too. A case that runs off its end falls
822     // through to the next one, as JavaScript does.
823     builder.push_loop(exit_block, None);
824     for (i, case) in sw.cases.iter().enumerate() {
825         builder.start_block(case_blocks[i]);
826         for stmt in &case.consequent {
827             if builder.is_terminated() {
828                 break;
829             }
830             lower_stmt(stmt, builder);
831         }
832         if !builder.is_terminated() {
833             let fallthrough = case_blocks.get(i + 1).copied().unwrap_or(exit_block);
834             let b = builder.seal_with(Terminator::Jump(fallthrough));
835             builder.add_edge(b, fallthrough, EdgeKind::Unconditional);
836         }
837     }
838     builder.pop_loop();

```

Le discriminant est évalué une fois, en ExprStmt sans position (src/lowering/cfg_builder.rs, l. 776). La sortie B1 est réservée en premier, avant même le discriminant ; viennent ensuite un bloc par case (B2, B3, B4), puis les dispatchs suivants (B5, B6), le premier dispatch étant le bloc courant. Le dispatch i entre dans la case i ou passe au dispatch $i + 1$; le dernier passe à la sortie. Chaque case est abaissée dans son propre bloc, sous un cadre push_loop(exit_block, None) qui donne sa cible au break ; une case qui finit sans break saute dans la suivante (3→4[U]), exactement comme en JavaScript. Figure 7.5 dessine le gabarit.

Les dispatchs portent la condition littérale true. Le commentaire parle de dispatch « opaque » : la condition l'est *parce que* le point fixe ne raffine pas sur un littéral, et que les deux successeurs reçoivent donc l'environnement entier. Un lecteur qui évaluerait cette condition concrètement élaguerait à tort la branche « suivant ». lower_try, lui, utilise la vraie sentinelle SummaryVal(Top).

Le commentaire du code rappelle l'ancienne forme, qui enchaînait les conséquences en ligne droite : c'était « an under-approximation twice over ». Entrer dans case 2 sans exécuter case 1 était irréprésentable, et le premier break scellait la chaîne, si bien que toutes les cases suivantes disparaissaient du CFG. La forme actuelle a trois propriétés : chaque case est atteignable sans les précédentes ; la sortie est atteignable sans entrer dans aucune case (un default est sur-approximé comme facultatif) ; le fall-through est fidèle.

Exemple 7.2: Pourquoi Warning et non Error

L'analyseur rapporte sur Status :

```
Status (1 hooks) e4_switch_try.tsx
warn  setter-in-render [hook:0] (line 7:6) setter `setMsg` called directly in the render body, move this
      ↪ call into a useEffect or an event handler
```

Un WARNING, pas une ERROR : aucun appel de setMsg n'est sur tous les chemins, puisque la sortie B1 du switch est atteignable sans entrer dans une case (0->5, 5->6, 6->1). Le prix de la sur-approximation du default est payé en force must, jamais en finding.

Trois détails complètent le tableau. Les tests case k: ne sont pas abaissés : en pratique des constantes, mais si k est une variable, sa lecture est perdue (section 7.14). Un default placé au milieu est dispatché dans l'ordre du texte. Enfin, la boucle d'instructions d'une case est recopiée (src/lowering/cfg_builder.rs, l. 826–831) au lieu d'appeler lower_stmts, avec le même arrêt au premier terminateur.

7.7.2 Le try : une branche sur l'inconnu

Extrait 7.27 – src/lowering/cfg_builder.rs, l. 492–510 — le try : un Branch sur T

```
492 fn lower_try(tr: &TryStatement, builder: &mut BlockBuilder) {
493     let span = builder.span_at(tr.span.start);
494     let try_block = builder.new_block();
495     // With no handler, a throw runs the finalizer and keeps propagating, so the
496     // throwing arm goes straight there.
497     let catch_block = builder.new_block();
498     let after = builder.new_block();
499
500     let branch = builder.seal_with(Terminator::Branch {
501         cond: Expr::SummaryVal(crate::ir::expr::SummaryValue::Top),
502         then_: try_block,
503         else_: catch_block,
504         span,
505     });
506     builder.add_edge(branch, try_block, EdgeKind::IfTrue);
507     builder.add_edge(branch, catch_block, EdgeKind::IfFalse);
508
509     builder.start_block(try_block);
510     lower_stmts(&tr.block.body, builder);
```

La suite de la fonction (src/lowering/cfg_builder.rs, l. 511–542) scelle le corps du try par un saut vers after ; ouvre catch_block, y lie le paramètre catch (e) à T — la valeur levée est inconnaisable, et la lier évite qu'e soit lu comme une capture —, abaisse le corps du catch et saute aussi vers after ; enfin ouvre after et y abaisse le finally. Sans catch, le bloc catch_block reste vide : « a throw runs the finalizer and keeps propagating ». Dans Status, c'est B1 → B7 ou B8 → B9.

Le commentaire de conception (src/lowering/cfg_builder.rs, l. 470–491) décrit l'ancien défaut avec précision. Le corps du try, celui du catch et le finally étaient enchaînés, chacun sous la garde !builder.is_terminated() ; un try dont le corps retourne terminait le bloc, et tout le catch et le finally n'étaient jamais abaissés — hooks et setters compris. Quand la garde passait, le catch était séquencé inconditionnellement après le corps, et une écriture faite seulement dans le catch passait pour faite sur tous les chemins. La forme actuelle dit les deux choses vraies à la fois : le catch est sur un chemin, le finally sur tous.

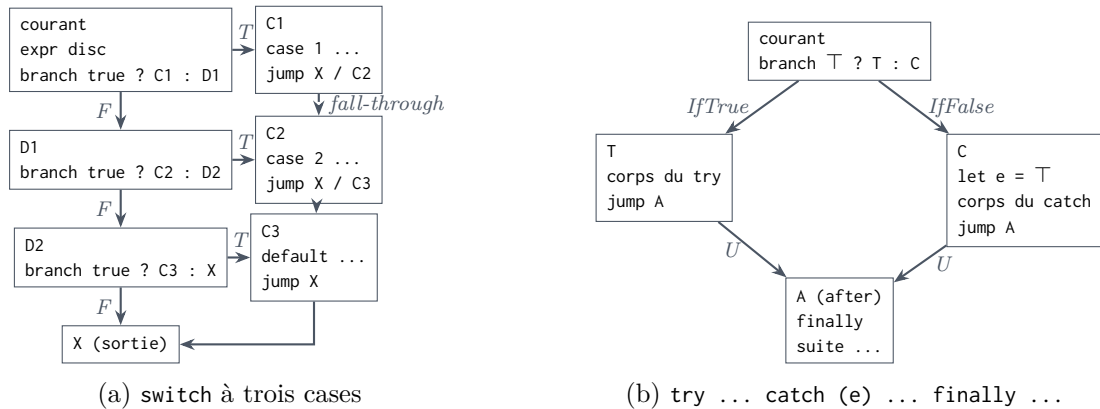


FIGURE 7.5 – Gabarits du switch et du try. Les dispatchs du switch portent Lit(true), que le point fixe ne raffine pas; les arêtes pointillées (fall-through) n'existent que pour une case sans break, les autres sautent à X. Le try branche sur la sentinelle $\top$ et place le finally sur la jonction.

Piège

Deux divergences du try. La première est documentée (src/lowering/cfg_builder.rs, l. 486–491) : un return dans le corps du try scelle son bloc, donc ce chemin ne passe pas par le finally, alors que JavaScript l'y ferait passer d'abord. Le finally reste atteignable par le bras du catch, donc ses hooks et ses écritures sont trouvés; seule sa présence sur le chemin qui retourne est perdue, ce qui coûte la force must (une ERROR rétrogradée en WARNING). La seconde ne l'est pas : le Branch sur $\top$ est placé *avant* le corps du try, donc le catch part de l'état d'avant le try. Le chemin « le try a partiellement exécuté, puis a levé » n'existe pas; section 7.14 en montre une conséquence observée.

7.8 Motifs, déstructurations et temporaires

7.8.1 Le principe : un temporaire, puis des projections

Le motif le plus fréquent de tout programme React est une déstructuration : `const [n, setN] = useState(0)`. Voici ce qu'en fait la construction du CFG, *avant* l'extraction des hooks (sortie -raw de la sonde, sans positions) :

```
=== raw (before extract_hooks) Counter (params []) ===
B0:
  let __arr_71 = useState(0)
  let n = __arr_71[0]
  let setN = __arr_71[1]
  => return Native<button>({#0 onClick: fn#1() { ... }})[n]
```

Un temporaire reçoit la source, puis chaque sous-motif est lié à une projection de ce temporaire, récursivement :

Extrait 7.28 – src/lowering/cfg_builder.rs, l. 869–894 — le motif tableau

```
869 BindingPattern::ArrayPattern(arr) => {
870     let temp = format!("__arr_{}", arr.span.start);
871     builder.push_stmt(Smt::Let {
872         var: temp.clone(),
873         rhs,
874         span,
875     });
```

```

876         for (i, elem) in arr.elements.iter().enumerate() {
877             let Some(elem) = elem else { continue };
878             let elem_rhs = Expr::IndexAccess {
879                 arr: Box::new(Expr::Var(temp.clone())),
880                 idx: Box::new(Expr::Lit(Prim::Int(i as i32))),
881             };
882             // Each sub-pattern sits somewhere; passing `None` here made a
883             // default's side effects report no line at all (#131).
884             let elem_span = builder.span_at(elem.span().start).or(span);
885             lower_binding_pattern(elem, elem_rhs, elem_span, builder);
886         }
887         // `const [a, ...rest] = xs`: bind `rest` to the source — the same
888         // sound over-approximation the object pattern uses for its rest.
889         // Leaving it unbound loses forwarded setters.
890         if let Some(rest) = &arr.rest {
891             let rest_span = builder.span_at(rest.span.start).or(span);
892             lower_binding_pattern(&rest.argument, Expr::Var(temp.clone()), rest_span, builder);
893         }
894     }

```

Le motif objet (`src/lowering/cfg_builder.rs`, l. 895–937) est construit de la même façon avec un temporaire `__obj_offset` : une clé statique ou chaîne donne un `FieldAccess` ; une clé calculée `[k]` est lue pour effet et la liaison reçoit `⊤` (« the key expression runs and `v` is bound — to an unknown value, not to nothing ») ; une clé numérique suit le même chemin. Dans les deux motifs, un reste (`...rest`) est lié au temporaire lui-même : c’est une sur-approximation *sound*, puisque le reste contient un sous-ensemble des champs, chacun avec la même valeur ; ne pas le lier perdait les setters transmis par les composants enveloppes (`(...props) => <X ...props/>`). Chaque sous-liaison prend la position de son sous-nœud, avec repli sur celle du motif (#131).

Le nom du temporaire est calculé à partir de l’*offset* du motif dans le fichier, pas tiré d’un compteur. Ce choix est figé :

Décision : ADR-20 §11 — des noms de temporaires à offset

ADR-20 §11 garde les noms `__arr_offset` et `__obj_offset` au lieu du compteur `fresh_temp`. Un offset est unique par position dans un fichier (pas de collision), déterministe, et les noms sont de toute façon α -renommés au splice (chapitre 8). L’offset n’est jamais relu ; en revanche le préfixe `__arr_` est porteur : `hook_extractor` teste `starts_with("__arr_")` pour résoudre la déstructuration de `useState`. Un renommage par compteur devrait garder le préfixe : du mouvement sans bénéfice.

Nom	Créé par	Unicité
<code>__tN</code>	<code>fresh_temp</code> (ternaire, court-circuit, hoist d’await)	par constructeur : chaque corps imbriqué repart de 0
<code>__arr_N</code> , <code>__obj_N</code>	motifs de liaison	offset du motif, unique par fichier
<code>__dstr_N</code>	cible de déstructuration en <i>affectation</i>	idem
<code>__pI</code>	paramètre déstructuré (<code>inject_param_preamble</code>)	index du paramètre, par corps
<code>__term_B</code>	hoist de terminateur (section 7.13)	identifiant du bloc

TABLE 7.3 – Les temporaires du lowering. Tous sont des noms réservés implicites : aucun code source n’est censé les employer.

7.8.2 Défauts et paramètres

Extrait 7.29 – `src/lowering/cfg_builder.rs`, l. 938–946 — un défaut de motif est émis, sans être modélisé

```

938     BindingPattern::AssignmentPattern(ap) => {
939         // The default value is not modeled (the binding keeps `rhs`), but
940         // it is evaluated when the source is undefined – emit it so its
941         // reads and side effects survive (`{ cb = () => setX(1) }`).
942         let default_span = builder.span_at(ap.right.span().start).or(span);
943         let default = lower_expr(&ap.right, builder);
944         builder.push_stmt(Stmt::ExprStmt(default, default_span));
945         lower_binding_pattern(&ap.left, rhs, span.or(default_span), builder);
946     }

```

Le défaut `x = d` est émis *inconditionnellement* — il ne s'exécute en vrai que si la source vaut `undefined` — et la liaison garde la projection : la valeur du défaut ne rejoint pas celle de la liaison. Ses lectures et ses effets sont visibles (invariant 7.2) ; sa valeur ne l'est pas. Pour `const cb = () => setX(1)` = props, la liaison `cb` vaut `props.cb` et non `props.cb` $\sqcup$ la flèche.

Un appel ultérieur à `cb` en hérite. Vérifié au commit `e67b10a` sur un couple témoin (`e23_cb_witness.tsx`, où `cb` est lié directement à la flèche) et fautif (`e24_cb_default.tsx`, où la même flèche n'est qu'un défaut de `props.cb`), tous deux passant `cb` en dépendance d'un effet qui l'appelle :

```

CbWitness (2 hooks) e23_cb_witness.tsx
  warn  always-unstable-deps [hook:1] (line 6:2) this effect depends on `cb`,
        a new reference every render, so `Object.is` always differs ...

CbDefault (2 hooks) e24_cb_default.tsx ✓
  verified  always-unstable-deps no deps array is defeated by an always-fresh reference

```

Le témoin est signalé parce que la liaison directe donne à `cb` un `FnLit` frais à chaque rendu. Le fautif est certifié : la liaison `cb = props.cb` n'est jamais qu'une prop, dont la stabilité ne doit rien à la flèche du défaut, et l'appel `cb()` ne peut donc pas être relié au `setX(1)` qu'il exécuterait réellement si l'appelant ne fournit pas la prop. C'est une sous-approximation : le même programme, au même risque de rendu perpétuel, est couvert dans un cas et ne l'est pas dans l'autre.

Les paramètres suivent la même mécanique, dans un préambule injecté en tête de corps :

Extrait 7.30 – `src/lowering/cfg_builder.rs`, l. 952–970 — le préambule des paramètres

```

952 pub(super) fn inject_param_preamble(
953     params: &FormalParameters,
954     builder: &mut BlockBuilder,
955 ) -> Vec<String> {
956     let mut names = Vec::new();
957     for (i, p) in params.items.iter().enumerate() {
958         match &p.pattern {
959             BindingPattern::BindingIdentifier(id) => {
960                 names.push(id.name.to_string());
961             }
962             other => {
963                 let temp = format!("_p{}", i);
964                 names.push(temp.clone());
965                 lower_binding_pattern(other, Expr::Var(temp), None, builder);
966             }
967         }
968     }
969     names
970 }

```

Un paramètre simple `x` ne produit *aucune* instruction : il est lié par l'appelant (le splice émet `let x' = arg`) ou par l'environnement initial du point fixe. Un paramètre déstructuré reçoit le nom synthétique `__pI` et son motif est abaissé sur `Var(__pI)`, sans position pour le `Let` de tête (argument `None`) : c'est la ligne `let __obj_56 = __p0` sans `@` de tous nos exemples. Un composant à props déstructurés a donc pour paramètre `__p0`. Un paramètre avec défaut (`x = 1`) est un motif et reçoit aussi un nom `__pI`. Le paramètre de reste (`...args`), lui, n'est pas lu du tout : seul `params.items` est parcouru (section 7.14).

7.8.3 Les cibles d'affectation

Une affectation à motif (`[i, max] = [max, i]`) doit *écrire* des liaisons existantes, pas en créer. `lower_assignment_target` (`src/lowering/expr_lower.rs`, l. 1023–1135) est le miroir de `lower_binding_pattern` : il émet des `Assign` au lieu de `Let`, via un temporaire `__dstr_offset`, et des `MemberWrite` pour les cibles membres imbriquées. Son commentaire de tête énonce le pendant de invariant 7.2 pour les écritures.

Invariant 7.3: Écrire chaque feuille

« Every leaf identifier is written on every path: a shape the walker cannot track precisely passes `opaque()` down rather than emitting nothing, because leaving a variable at its previous abstract value falsifies it. » (`src/lowering/expr_lower.rs`, l. 1027–1029.) Une *liaison périmée* — une variable laissée à sa valeur antérieure alors que le programme l'a écrite — n'est pas une ignorance : c'est une affirmation fausse, donc une sous-approximation.

Exemple 7.3: Déstructurations, composés, `delete`, gabarit, `void`

```
1 import { useState } from "react";
2
3 export function Destr({ a = 1, ...rest }: any) {
4   const [[min, max], setRange] = useState([0, 100]);
5   const { [rest.k]: v, w = min, ...others } = rest;
6   let i = 0;
7   [i, max] = [max, i];
8   i += 2;
9   i **= 2;
10  i |= 3;
11  delete others.x;
12  const s = `n=${i}`;
13  void track(s);
14  return <X {...others} title={s}><span>{v}</span></X>;
15 }
```

```
B0:
  let __obj_57 = __p0
  expr 1 @3:28
  let a = __obj_57.a @3:24
  let rest = __obj_57 @3:31
  let __arr_93 = StateVal(0) @4:9
  let min = __arr_93[0] @4:10
  let max = __arr_93[1] @4:15
  let setRange = StateSetter(0) @4:21
  let __obj_145 = rest @5:8
  expr rest.k @5:11
  let v = T @5:10
  expr min @5:27
  let w = __obj_145.w @5:23
  let others = __obj_145 @5:32
  let i = 0 @6:6
```

```

let __dstr_204 = [#1 max, i]<Exact(2)> @7:2
i := __dstr_204[0]
max := __dstr_204[1]
expr [#1 max, i]<Exact(2)> @7:2
i := (i Add 2) @8:2
expr i @8:2
i := (i Pow 2) @9:2
expr i @9:2
i := T @10:2
expr i @10:2
others.x <- undefined @11:2
expr true @11:2
let s = ("n=" Add i) Add "" @12:8
expr track(s) @13:2
expr undefined @13:2
=> return CompApp<X>({#3 ...4: others, title: s, children: [#5 Native<span>({#2 })[v]]<AtLeast(1)>})
hooks:
#0 State init=[#0 0, 100]<Exact(2)> @4:8

```

Tout le chapitre se lit dans ce bloc. Le défaut `a = 1` est émis (expr 1) sans que `a` le reçoive ; le reste `rest` est la source. Le motif imbriqué sur `useState` : le temporaire externe `a` disparu (c'est un temporaire d'état, section 7.11), le temporaire interne `__arr_93` reçoit `StateVal(0)` et ses projections restent des `IndexAccess`. La clé calculée donne la lecture `rest.k` puis `v = T`. La déstructuration en affectation écrit ses feuilles par des `Assign` sans position, puis l'`ExprStmt` final réutilise la même valeur : le même `ArrayLit` #1 apparaît deux fois, avec le même `ExprId`. `**=` est fidèle, `||=` met `i` à `T`. `delete` est un `MemberWrite` d'`undefined` suivi de `true`, le gabarit une chaîne d'additions, `void` une lecture suivie d'`undefined`. Enfin `<X ...others>` range le spread sous la clé synthétique `...4` et ses enfants dans `children`, d'arité `AtLeast(1)`.

7.9 Fonctions imbriquées, classes et sites d'allocation

7.9.1 Un corps par fonction

Une fonction imbriquée n'est pas « mise à plat » dans le CFG de son parent : elle est abaissée dans son propre `BlockBuilder`, et son CFG est emballé dans le littéral de fonction.

Extrait 7.31 – `src/lowering/expr_lower.rs`, l. 442–457 — une flèche : un corps à part

```

442     Expression::ArrowFunctionExpression(arrow) => {
443         let id = builder.next_expr_id();
444         let ctx = builder.ctx.clone();
445         // Concise body (`x => expr`) carries an implicit return; block body
446         // (`x => { ... }`) lowers like any function body.
447         let (params, body_cfg) = if arrow.expression {
448             build_expr_fn_body_cfg(&arrow.params, &arrow.body, &ctx)
449         } else {
450             build_fn_body_cfg(&arrow.params, &arrow.body, &ctx)
451         };
452         Expr::FnLit {
453             id,
454             params,
455             body_cfg: Arc::new(body_cfg),
456         }
457     }

```

Le contexte est cloné (`builder.ctx.clone()`) : le nouveau constructeur partage avec son parent la table des positions, les origines JSX et, surtout, le compteur d'identifiants d'allocation. Aucune notion de fermeture n'est stockée : les captures sont retrouvées plus tard, sur le corps,

par l'analyse de variables libres (`ir::free_vars::compute_free_vars`, chapitre 6). Une fonction expression suit le même chemin; une fonction sans corps (déclaration ambiante, surcharge TypeScript) reçoit `empty_cfg()`, un unique bloc `Unreachable`.

Une déclaration `function f()` devient un `Let f = FnLit` (`src/lowering/cfg_builder.rs`, l. 398–418). Le commentaire de ce bras dit « Hoisted declarations: bind name but emit no CFG node »; c'est l'inverse de ce que fait le code, qui émet le `Let` à la position textuelle. Le hissage de JavaScript n'est pas modélisé, avec une conséquence que la section 7.14 montrera.

Une classe (`lower_class`, `src/lowering/expr_lower.rs`, l. 58–135) devient un `ObjectLit` dont les membres sont rangés sous des clés synthétiques (`[method]N`, `[field]N`, `[accessor]N`, `[static]N`, `[extends]N`) qu'aucun `FieldAccess` ne peut atteindre; méthodes et blocs `static` deviennent des `FnLit`. Le motif est #77 : les corps de classes tombaient dans le bras vide de `lower_stmt`, et `class X m() setC(1)` se lisait comme un composant qui n'écrit jamais `c`. Une classe ne lie rien que l'environnement modélise, mais ses corps sont du code.

7.9.2 Le compteur d'allocation

Chaque nœud allouant (`ObjectLit`, `ArrayLit`, `FnLit`, `New`) porte un `ExprId`, son *site* d'allocation (définition 6.2) : c'est la clé du tas abstrait. D'où vient ce numéro ?

Extrait 7.32 – `src/lowering/cfg_builder.rs`, l. 20–58 — le compteur partagé et le contexte par fichier

```
20 // -- Allocation-site ids -----
21
22 /// The counter behind [ExprId], shared by every body lowered from one file.
23 ///
24 /// ExprId keys the abstract heap, and **one component's heap holds the
25 /// allocation sites of its render body, of every nested function body, and of
26 /// every callee spliced into it**. A counter per 'BlockBuilder' numbered all of
27 /// them from zero, so two unrelated objects shared a heap entry and the later
28 /// one silently answered member reads of the earlier – a false negative when
29 /// the impostor was the more stable of the two (#134).
30 ///
31 /// Cloning shares the counter, which is how it reaches a nested body without
32 /// changing what a builder owns. One counter per file; the splice gives an
33 /// inlined callee its own range (see ['crate::ir::Offsets']).
34 #[derive(Debug, Clone, Default)]
35 pub struct ExprIds(Rc<Cell<usize>>);
36
37 impl ExprIds {
38     fn next(&self) -> ExprId {
39         let id = self.0.get();
40         self.0.set(id + 1);
41         ExprId(id)
42     }
43 }
44
45 /// Everything one file's lowering shares with every body it builds: the span
46 /// table, the allocation-site counter, and the JSX callee origins.
47 ///
48 /// One carrier rather than three parameters. Nested bodies are lowered through
49 /// their own 'BlockBuilder', so a per-file fact only reaches a 'FnLit' body if
50 /// it travels with the builder that makes it; bundling is what keeps the next
51 /// such fact from needing its own threading pass.
52 #[derive(Debug, Clone)]
53 pub struct LowerCtx {
54     pub smap: SourceMap,
55     /// Shared with every other body of this file – see ['ExprIds'].
```

```

56   pub ids: ExprIds,
57   pub jsx: JsxOrigins,
58 }

```

Décision : ADR-10 et #134 — un compteur par fichier

ADR-10 adopte l'abstraction par sites d'allocation : un `ExprId` sur chaque nœud allouant, un `Arc<CFG>` dans `FnLit`, un tas monotone indexé par `ExprId`. À l'origine, le compteur vivait dans `BlockBuilder`. Comme chaque corps imbriqué a son propre constructeur, chacun numérotait ses sites depuis 0 ; or le tas d'un composant contient les sites de son rendu, de tous ses corps imbriqués et de tous les callés greffés. Deux objets sans rapport partageaient donc une entrée de tas, et le second répondait aux lectures de membres du premier — un faux négatif quand l'imposteur était le plus stable des deux (#134). Le compteur est désormais un `Rc<Cell<usize>` porté par `LowerCtx` ; cloner le contexte partage le compteur. Un compteur par fichier suffit, parce que le splice donne à un callé greffé sa propre plage d'identifiants (« One counter per file ; the splice gives an inlined callee its own range », `Offsets`, chapitre 8). Côté moteur, ADR-10 a refusé une carte d'environnement unique `EnvVal = Val | Loc`, dont l'extension écrasait la `Loc`, au profit de deux cartes.

Piège

« Un compteur par fichier » signifie un par famille. Chacun des trois points d'entrée crée son propre `LowerCtx` (`src/lowering/mod.rs`, l. 382–385, `src/lowering/mod.rs`, l. 446–449, `utility_lowerer.rs`) : tous les composants d'un fichier partagent un compteur, tous ses hooks custom un deuxième, ses utilitaires un troisième, chacun partant de 0. Ce n'est pas une collision, grâce au décalage du splice ; mais deux `ExprId` égaux dans un composant et dans un hook du même fichier ne désignent pas le même site.

Piège

L'ordre des identifiants dépend de la construction. Une flèche tire son identifiant *avant* d'abaisser son corps (liste 7.31) ; une déclaration de fonction abaisse son corps d'abord et tire son identifiant ensuite, si bien que les sites de son corps ont des identifiants inférieurs au sien. Une clé synthétique consomme un identifiant. Dans `Loader` (ci-dessous), la fonction `load` porte l'identifiant 1, la flèche `useless` le 3 et l'objet de props le 4 ; d'après l'ordre d'abaissement, 0 et 2 ont été consommés par la flèche de l'effet et par le tableau de deps, qui n'apparaissent plus dans le rendu. Rien ne doit dépendre de ces valeurs, seulement de leur unicité.

7.10 await, arêtes Await et positions synthétiques

7.10.1 Le problème : ce qui suit un await n'est plus dans la phase

Rappel React : asynchronisme et phases

Une fonction `async` s'exécute de façon synchrone jusqu'à son premier `await`, puis rend la main. La suite reprend à un tour ultérieur de la boucle d'événements, hors de toute phase de React : un `setState` écrit après un `await` dans un effet ne s'exécute ni pendant le rendu ni pendant le commit, mais plus tard, comme un callback de timer (chapitre 2).

Jusqu'à #117, `await e` s'abaissait en `e`, et un `setState` écrit après un `await` gardait la phase synchrone de sa région. ADR-35 a fendu le bloc.

Extrait 7.33 – Un effet asynchrone (/tmp/ch07/e6_async.tsx)

```

1 import { useState, useEffect } from "react";
2
3 export function Loader({ url }: { url: string }) {
4   const [data, setData] = useState(null);
5   useEffect(() => {
6     async function load() {
7       const res = await fetch(url);
8       setData(await res.json());
9     }
10    load();
11  }, [url]);
12  const useless = () => (url ? 1 : 2);
13  return <div>{String(data)}</div>;
14 }

```

```

#1 Effect deps=List[url]<Exact(1)> @5:2
B0:
  let load = fn#1() {
    B0:
      let __t0 = fetch(url) @7:24
      => jump B1
    B1:
      let res = __t0 @7:12
      let __t1 = res.json() @8:20
      => jump B2
    B2:
      expr setData(__t1) @8:6
      => return undefined
    edges: 0->1[Await] 1->2[Await]
  } @6:4
  expr load() @10:4
  => return undefined

```

Extrait 7.34 – src/lowering/expr_lower.rs, l. 577–606 — await : lier l'argument, puis couper

```

577 Expression::AwaitExpression(aw) => {
578   let value = lower_expr(&aw.argument, builder);
579   // The awaited expression is evaluated BEFORE the suspension:
580   // `await fetch(u)` calls `fetch` synchronously and only then
581   // yields. Returning it here would carry it into the enclosing
582   // statement, which the split moves into the post-await block – so
583   // the call would read as `deferred`, and `deferred` is a claim
584   // ("never inside a React phase"), not an over-approximation. Bind
585   // it on this side of the edge instead. A bare name or literal has
586   // nothing to evaluate, so it is passed through unchanged rather
587   // than given a binding no reader wants.
588   let value = match value {
589     v @ (Expr::Var(_) | Expr::Lit(_)) => v,
590     v => {
591       let tmp = builder.fresh_temp();
592       // The binding is synthetic, its position is not: everything
593       // the walk finds under `await fetch(...)` takes its witness
594       // from the statement it sits in, and a spanless hoist made
595       // every such row report no line at all (#131).
596       builder.push_stmt(Stmt::Let {
597         var: tmp.clone(),
598         rhs: v,
599         span: builder.span_at(aw.argument.span().start),
600       });
601       Expr::Var(tmp)
602     }
603   };

```

```

604         builder.split_at_await();
605         value
606     }

```

Chaque `await` fait deux choses. D'abord, l'argument est lié à un temporaire *de ce côté-ci* de la coupure : `await fetch(u)` appelle `fetch` de façon synchrone et ne suspend qu'ensuite. Rendre l'argument tel quel l'emporterait dans l'instruction englobante, que la coupure place dans le bloc suivant ; l'appel serait alors lu comme « différé », et « différé » est une affirmation (« jamais dans une phase React »), pas une sur-approximation. Un nom ou un littéral, qui n'ont rien à évaluer, passent sans temporaire. Ensuite, `split_at_await` (liste 7.5) scelle le bloc courant par un `Jump` et l'ouvre sur un bloc neuf, relié par une arête `Await`. Si le bloc est déjà scellé, rien n'est fait : un `return await f()` n'a plus rien à différer. Dans `load`, on voit les deux temporaires `__t0` et `__t1` liés avant chaque coupure, la liaison `res` et l'appel `setData` placés après. Le ternaire du `render helper useless` a ses propres `__t0` et ses propres blocs : autre constructeur, autre compteur de temporaires. Figure 7.6 dessine le gabarit.

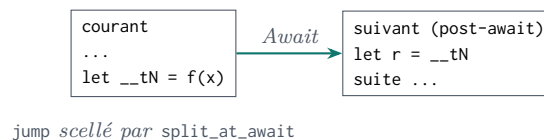


FIGURE 7.6 – Gabarit de `const r = await f(x)`. Le contrôle est inconditionnel — dominance et accessibilité sont inchangées —, mais le successeur s'exécute après une suspension.

Décision : ADR-35 — une nature d'arête, pas un champ de bloc

ADR-35 décide que l'`await` fend le bloc par une arête `EdgeKind::Await`. Pourquoi pas un champ « post-await » sur `BasicBlock` ? Parce que `BasicBlock` et `CFG` sont construits en plus de deux cents endroits, pour la plupart de l'IR de test écrite à la main : un champ serait un changement de deux cents sites pour un fait que ces fixtures n'ont pas. Une nature d'arête est additive, survit au remappage, et place le fait où il est : entre deux blocs. L'ensemble `post-await` est calculé par le consommateur (`CFG::post_await_blocks`, la fermeture des cibles d'arêtes `Await`) ; un corps imbriqué répond pour lui-même, et un `await` dans un callback ne diffère pas les instructions suivantes du corps englobant. Dominance et accessibilité ne changent pas, puisque l'arête est inconditionnelle. L'ADR dit que l'instruction en cours atterrit dans le successeur ; le code actuel (liste 7.34) en retire l'argument attendu, lié avant la coupure.

7.10.2 Positions synthétiques

Le lowering crée beaucoup d'instructions que la source n'écrit pas : temporaires de diamants et d'`await`, projections de motifs, lectures conservées. Quelle position leur donner ? Un diagnostic sans ligne est presque inutilisable.

Décision : ADR-39 — une liaison synthétique est synthétique, sa position ne l'est pas

ADR-39 (#131) décide que toute instruction synthétique prend la position de l'expression qu'elle lie : l'argument d'un `await`, la conséquence et l'alternative d'un ternaire, l'opérande droit d'un court-circuit, chaque sous-nœud d'un motif (élément, propriété, reste, clé calculée, défaut), avec repli sur la position du motif. Ce que la source ne nomme pas, le splice le nomme par son site d'appel ; un corps sans instruction hérite du témoin d'où l'on y est entré ; un `finding` prend la première position de sa chaîne de témoins. L'alternative refusée était

d'ajouter une position à `Terminator::Return` : « better by a line, at the cost of a hundred construction sites », laissée ouverte en #140. Mesure : 82 findings sans position ramenés à 0, ensemble de findings inchangé.

Quelques instructions restent sans position au commit e67b10a, et on les a toutes rencontrées : le `Let` de tête d'un paramètre déstructuré, l'`ExprStmt` de l'`init` (expression) et de l'`update` d'un `for`, le discriminant d'un `switch`, les feuilles d'une déstructuration en affectation, et le `Let __term_B` du hoist de terminateur quand le terminateur est un `Return` (section 7.13). Un `Stmt::span()` à `None` est documenté comme « a bug to fix, not a shape to route around » (`src/ir/stmt.rs`, l. 37–44).

7.11 L'extraction des hooks

7.11.1 Avant, après

Jusqu'ici, un appel `useState(0)` n'était qu'un `Call` parmi d'autres. La passe `extract_hooks` le reconnaît, lui attribue un label, crée l'entrée `HookEntry` correspondante et réécrit l'IR pour que les liaisons lisent des valeurs marquées. Figure 7.7 montre la transformation sur le compteur, d'après les deux sorties de la sonde.

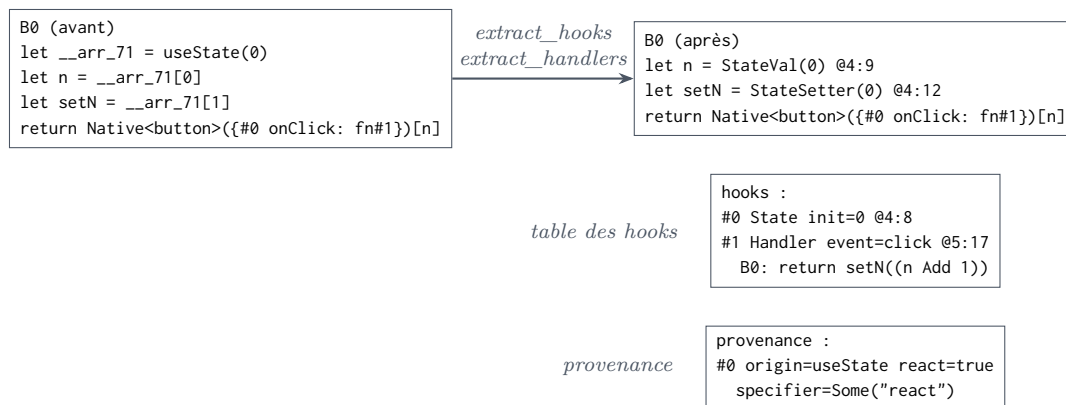


FIGURE 7.7 – Le compteur avant et après les passes d'extraction. Le `Let` du temporaire d'état disparaît ; ses deux projections sont réécrites en `StateVal(0)` et `StateSetter(0)` ; la table des hooks reçoit l'état (label 0) puis le handler (label 1) ; la provenance n'a qu'une ligne, car un handler n'en a jamais.

Extrait 7.35 – `src/lowering/hook_extractor.rs`, l. 264–304 — la passe `extract_hooks`

```
264 /// Walk `cfg` in block-id order, extract top-level hook calls, and rewrite
265 /// affected statements in-place. Returns `(hooks, provenance, next_label)`;
266 /// `provenance` holds one row per extracted hook call (never for handlers).
267 /// Destructuring is resolved: `__arr_N[0]` → `StateVal(L)`, `__arr_N[1]` → `StateSetter(L)`.
268 pub fn extract_hooks(
269     cfg: &mut CFG,
270     imports: &ImportCtx<'_,>,
271 ) -> (Vec<HookEntry>, Vec<HookProvenance>, HookLabel) {
272     let mut label: HookLabel = 0;
273     let mut hooks: Vec<HookEntry> = Vec::new();
274     let mut provenance: Vec<HookProvenance> = Vec::new();
275     // Maps array-destructuring temps (e.g. "__arr_42") → hook label, for useState/useReducer.
276     let mut state_temps: HashMap<String, HookLabel> = HashMap::new();
277
```

```

278 // Collected up front because the loop takes `&mut cfg.blocks` – the ids
279 // already come out in order (`CFG::blocks` is a `BTreeMap`).
280 hoist_terminator_hooks(cfg, imports);
281
282 let ids: Vec<BlockId> = cfg.blocks.keys().copied().collect();
283
284 for id in ids {
285     let old = std::mem::take(&mut cfg.blocks.get_mut(&id).unwrap().stmts);
286     let mut new: Vec<Stmt> = Vec::with_capacity(old.len());
287
288     for stmt in old {
289         process_stmt(
290             stmt,
291             &mut new,
292             &mut hooks,
293             &mut provenance,
294             &mut label,
295             &mut state_temps,
296             imports,
297         );
298     }
299
300     cfg.blocks.get_mut(&id).unwrap().stmts = new;
301 }
302
303 (hooks, provenance, label)
304 }

```

La passe commence par normaliser les terminateurs (`hoist_terminator_hooks`, section 7.13), puis parcourt les blocs par identifiant croissant et, dans chaque bloc, les instructions dans l'ordre ; chacune passe par `process_stmt`. La table `state_temps` associe à chaque temporaire `__arr_N` lié à un `useState` ou un `useReducer` le label de ce hook.

Extrait 7.36 – `src/lowering/hook_extractor.rs`, l. 369–417 — `process_stmt` : le cas `Let`

```

369 match stmt {
370     Stmt::Let {
371         var,
372         rhs,
373         span: stmt_span,
374     } => match try_consume_hook_call(rhs, imports) {
375         Ok((call, args)) => {
376             let lbl = *label;
377             *label += 1;
378             let is_state_like =
379                 call.is_react && matches!(call.origin_name.as_str(), "useState" | "useReducer");
380             let is_arr_temp = var.starts_with("__arr-");
381
382             let entry = make_hook_entry(&call, lbl, args, stmt_span);
383             let marker = hook_result_expr(&call, lbl, entry.as_ref());
384             provenance.push(call.provenance(lbl, stmt_span));
385             if let Some(entry) = entry {
386                 hooks.push(entry);
387             }
388
389             // Record the binding variable on Custom hooks (their import
390             // source and resolved file come from the classification).
391             if let Some(HookEntry::Custom { binding, .. }) = hooks.last_mut() {
392                 // `__arr_N` is a lowering temp, never a source name.
393                 if !is_arr_temp {
394                     *binding = Some(var.clone());

```

```

395         }
396     }
397
398     if is_state_like && is_arr_temp {
399         // Array-destructured useState/useReducer: drop the temp Let,
400         // subsequent IndexAccess stmts will be rewritten by rewrite_expr.
401         state_temps.insert(var, lbl);
402     } else {
403         out.push(Smt::Let {
404             var,
405             rhs: marker,
406             span: stmt_span,
407         });
408     }
409 }
410 Err(rhs) => {
411     out.push(Smt::Let {
412         var,
413         rhs: rewrite_expr(rhs, state_temps),
414         span: stmt_span,
415     });
416 }
417 },

```

Trois cas, lus dans le code :

- `Let v = <hook>(...)` : un label neuf, une entrée, une ligne de provenance ; la liaison est réécrite en `Let v = marqueur`. Exception : un `useState` ou `useReducer` déstructuré en tableau (le nom commence par `__arr_`) — le `Let` du temporaire est supprimé, et le temporaire enregistré dans `state_temps`.
- `ExprSmt(<hook>(...))` (src/lowering/hook_extractor.rs, l. 418–435) : même chose, et l'instruction devient `ExprSmt(HookMarker(L, marker_val))` — c'est le `expr HookMarker(1, Undefined)` du `useEffect` de `Profile`.
- tout le reste — `Assign`, `MemberWrite`, et tout `RHS` qui n'est pas un appel de hook — est seulement réécrit par `rewrite_expr`, qui remplace `Var(t)[0]` par `StateVal(L)`, `Var(t)[1]` par `StateSetter(L)` et `Var(t)[k]` ($k \geq 2$) par `Lit(Unit)` pour tout temporaire d'état `t` (src/lowering/hook_extractor.rs, l. 842–910).

Le mot important est *racine* : `try_consume_hook_call` (src/lowering/hook_extractor.rs, l. 504–525) ne reconnaît un hook que si l'appel est la racine du `RHS` d'un `Let` ou d'un `ExprSmt`, à une enveloppe `TSAnnotated` près (`useState<T>(x)`). Un appel de hook imbriqué dans une expression — `useContext(C).theme, f(useX())`, l'opérande droit d'un `&&` qui est un `Assign` — n'est pas extrait. Section 7.14 en mesure le coût. Le label de chaque hook étant attribué dans l'ordre des blocs, et les blocs étant numérotés à la réservation, *les labels ne suivent pas toujours l'ordre du texte* (section 7.14).

7.11.2 Qui est un hook ? La provenance

Avant de fabriquer une entrée, il faut décider si le callee est un hook, et lequel. `ImportCtx::classify_callee` (src/lowering/hook_extractor.rs, l. 572–636) applique, pour un nom `nu`, un ordre de priorité strict :

1. un hook défini localement dans le fichier masque tout (portée JavaScript) : hook custom, fichier courant ;
2. sinon, une origine d'import *prouvée* (`HookOrigin::React`, `File` ou `Package`, chapitre 5) fixe

l’identité, sous le nom *importé* et non sous l’alias local (`import useMemo as useM` donne `useMemo`);

3. sinon, un nom de forme hook non importé (`is_hook_name` : `use` suivi d’une majuscule ou d’un chiffre) est présumé être de React — c’est le cas des sources de test et des globales.

Pour un appel `ns.useX()`, le hook est de React si et seulement si `ns` est lié au module `react` (`import React from "react", import * as R from "react"`) ou s’appelle `React`; sinon c’est un hook custom de provenance inconnue (`store.useThing()`). La classification est « fail-closed » : un binding importé n’est classé que par ce que prouve son import. Le prédicat `is_hook_name` est partagé avec les détecteurs; l’ancien test `starts_with("use")` classait `userId()` comme hook.

Le résultat, un `ResolvedHookCall`, alimente à la fois l’entrée et la ligne de provenance `HookProvenance` (chapitre 6). Cette dernière existe pour une raison que l’on voit en pratique : un `useLayoutEffect` devient un `HookEntry::Effect` comme un `useEffect`, et seule sa provenance garde `origin_hook = "useLayoutEffect"`.

7.11.3 L’entrée et le marqueur

Deux fonctions décident ensemble de l’entrée et de ce que lit la liaison (tableau 7.4) : `make_hook_entry`, aux lignes 642 à 747 de `hook_extractor.rs`, et `hook_result_expr`, aux lignes 775 à 792 du même fichier.

Appel	HookEntry	la liaison lit
non React (local, fichier, paquet)	Customname, args, deps: Absent, binding, import_source, resolved_file	HookMarker(L, Unknown)
<code>useState(init)</code>	Stateinit (absent : Lit(Unit))	StateVal(L)
<code>useReducer(r, init)</code>	Stateinit (réducteur ignoré)	StateVal(L)
<code>useEffect</code> , <code>useLayoutEffect</code> , <code>useInsertionEffect</code>	Effectbody_cfg, deps	HookMarker(L, Undefined)
<code>useMemo(f, deps)</code>	Memobody_cfg, deps	MemoVal(L)
<code>useCallback(f, deps)</code>	Callbackbody_cfg, params, deps	CallbackVal(L)
<code>useRef(init)</code>	Refinit (absent : Lit(Null))	HookMarker(L, StableRef)
autre hook de React	Custom..., resolved_file: None	HookMarker(L, Unknown)

TABLE 7.4 – L’entrée créée pour chaque appel de hook, et la valeur marquée qui remplace l’appel (L est le label).

Le choix du marqueur est commenté longuement (`src/lowering/hook_extractor.rs`, l. 749–765). Les hooks de React que le moteur ne modélise pas (`useContext`, `useId`, `useTransition`) sont rangés en Custom, et leur liaison lit Unknown, c’est-à-dire `⊥`. Leur faire lire Undefined les rendait *prouvablement stables* et faisait taire toute règle de stabilité sur une valeur de contexte — un faux négatif. Undefined est réservé au seul hook réellement sans valeur, l’effet; un ref n’est pas sans valeur, c’est un conteneur d’identité constante, d’où `StableRef`.

Rappel React : les hooks et leurs corps

`useEffect(f, deps)` exécute `f` après le commit, quand un élément de `deps` a changé au sens d’`Object.is`; `useMemo(f, deps)` appelle `f` pendant le rendu et mémorise son résultat;

`useCallback(f, deps)` mémorise `f` elle-même. Un effet et un memo reçoivent une fonction sans argument ; un callback est appelé plus tard avec les arguments de son appelant (chapitre 2).

Le corps d'un hook à callback est calculé par `hook_body_cfg` :

Extrait 7.37 – `src/lowering/hook_extractor.rs`, l. 809–821 — le corps d'un hook : le `FnLit`, ou l'appel du callee

```
809 fn hook_body_cfg(arg: Option<Expr>) -> CFG {
810     let stmts = match arg {
811         Some(Expr::FnLit { body_cfg, .. }) => return unwrap_body(body_cfg),
812         Some(callee) => vec![Stmt::ExprStmt(
813             Expr::Call {
814                 fn_: Box::new(callee),
815                 args: vec![],
816             },
817             None,
818         )],
819         // No callback argument at all – not valid React; nothing runs.
820         None => vec![],
821     };
822 }
```

Un littéral `() => ...` apporte son CFG, *déplacé* hors du rendu (`Arc::try_unwrap`) : le `FnLit` disparaît de l'IR de rendu, qui ne voit plus que le marqueur. Ses paramètres sont jetés pour un effet ou un memo (corps sans argument, conformément à React), gardés dans `params` pour un callback. Toute autre expression — une variable, un membre, un appel qui rend une fonction — donne un corps qui est exactement l'*appel* de ce callee, que le moteur résoudra dans l'environnement comme n'importe quel appel. Le commentaire (`src/lowering/hook_extractor.rs`, l. 798–808) rappelle la version précédente : elle rendait un CFG `Unreachable`, c'est-à-dire « le callback ne fait rien » — $\perp$ là où $\top$ était requis —, et `useEffect(handler)` ressortait propre et *certifié* `verified infinite-loop`, alors que le même corps écrit en ligne était signalé.

Exemple 7.4: Hooks custom, `useRef`, `useMemo`, handler de composant

```
1 import { useState, useEffect, useMemo, useRef } from "react";
2 import { useQuery } from "@tanstack/react-query";
3
4 function useToggle(init: boolean) {
5     const [on, setOn] = useState(init);
6     const toggle = () => setOn(!on);
7     return [on, toggle] as const;
8 }
9
10 export function Panel({ id, opts }: { id: string; opts: object }) {
11     const { data } = useQuery({ queryKey: [id] });
12     const [on, toggle] = useToggle(false);
13     const ref = useRef(null);
14     const merged = useMemo(() => ({ ...opts, id }), [opts, id]);
15     useEffect(() => {
16         window.addEventListener("resize", () => toggle());
17     }, [merged]);
18     return <Child ref={ref} data={data} onPick={(x: number) => toggle()} {...merged} />;
19 }
```

Extrait de la sortie de la sonde (corps des hooks #4 à #6 élagués) :

```
B0:
    let __obj_279 = __p0
    let id = __obj_279.id @10:24
```

```

let opts = __obj_279.opts @10:28
let __obj_333 = HookMarker(0, Unknown) @11:8
let data = __obj_333.data @11:10
let __arr_382 = HookMarker(1, Unknown) @12:8
let on = __arr_382[0] @12:9
let toggle = __arr_382[1] @12:13
let ref = HookMarker(2, StableRef) @13:8
let merged = MemoVal(3) @14:8
expr HookMarker(4, Undefined) @15:2
=> return CompApp<Child>({#9 ref: ref, data: data, onPick: fn#10(x) {
  B0:
    => return toggle()
  edges:
    }, ...11: merged})
hooks:
#0 Custom useQuery({#0 queryKey: [#1 id]<Exact(1)>}) binding=Some("__obj_333") import_source=Some("@tanstack/react
  ↳ -query") resolved_file=None @11:8
#1 Custom useToggle(false) binding=None import_source=None resolved_file=Some("e5_hooks.tsx") @12:8
#2 Ref init=null @13:8
#3 Memo deps=List[opts, id]<Exact(2)> @14:8
  B0:
    => return {#3 ...4: opts, id: id}
#4 Effect deps=List[merged]<Exact(1)> @15:2
#5 Handler event=pick @18:9
#6 Handler event=resize @16:4

```

`useQuery` (un paquet) et `useToggle` (local) sont des Custom dont la liaison lit `HookMarker(L, Unknown)`. Un custom déstructuré en objet garde son temporaire `__obj_333` comme `binding` (seul le préfixe `__arr_` est exclu) ; déstructuré en tableau, `binding` vaut `None` et les projections `__arr_382[i]` restent des `IndexAccess` — la réécriture ne concerne que `useState`. `useRef` donne `StableRef` ; `useMemo` donne `MemoVal(3)`, avec pour corps la flèche concise déplacée. Les labels 5 et 6 sont un handler de composant (`onPick`, à la position de la balise ouvrante) et une subscription, que la section suivante explique. Le hook `useToggle`, abaissé par l'autre point d'entrée, a son propre compteur d'identifiants : dans sa sortie (non reproduite), la flèche `toggle` est `fn#0`.

Piège

Les formes de `useState` autres que `const [a, b] = useState()`. Le traitement d'état ne reconnaît que la déstructuration en tableau. `const t = useState(1)` lie `t` à `StateVal(1)` — le tuple entier lu comme la valeur —, et `t[1]` n'est plus reconnu comme setter : appeler `t[1](2)` dans le rendu (`e25_t_index_call.tsx`) donne `verified setter-in-render`, quand l'équivalent déstructuré (`e26_t_index_witness.tsx`, `setV(2)`) donne `error setter-in-render`. Le fautif appelle un `IndexAccess` sur `StateVal(1)`, une expression, jamais un `Var` du tableau des setters : c'est une sous-approximation vérifiée, pas seulement déduite du dump. Un `useState(2)` ; en instruction crée bien une entrée `State`, mais son marqueur vaut `Unknown`, ce qui déclenche un `analysis-limit` trompeur (« hook <hook:2> was not found in the registry »). `const [s, ...rest] = useState(0)` lie `rest` au temporaire supprimé, qui n'est plus lié nulle part. Le moteur n'a pas été vérifié au-delà de ces formes rares.

7.12 Handlers et subscriptions

7.12.1 La règle : par fuite, pas par nom

Un handler n'est pas un hook au sens de React, mais le moteur le traite comme une entrée de la table : un corps qu'un tiers (React, un composant enfant) peut invoquer zéro, une ou plusieurs fois, hors rendu. Oublier un handler, c'est oublier les écritures qu'il fait ; c'est donc une classe de faux négatifs. Le commentaire de `extract_handlers` fixe la règle :

Extrait 7.38 – `src/lowering/hook_extractor.rs`, l. 84–100 — les handlers se décident par fuite

```
84 /// Scan `cfg` for callback props and append each as `HookEntry::Handler`,
85 /// so their setter writes join the fixpoint (handlers run 0..N times –
86 /// under-approximating them is an FN class, TODO.md F4).
87 ///
88 /// Reachability is decided by ESCAPE, not by prop name:
89 /// - native elements: `on*` events and `ref` (the only native props React
90 ///   invokes);
91 /// - components: ANY function-valued prop (`onToggle`, `ref`, render props,
92 ///   `action={cb}` – the child may invoke whatever it receives);
93 /// - values resolve through `handler_body`: inline `FnLit`, a var bound to
94 ///   an `FnLit`, or a `useCallback` (`CallbackVal` – body from its hook
95 ///   entry); JSX inside render-helper closures is scanned too.
96 ///
97 /// Bare setters as props (`onOpenChange={setOpen}`) are handled by the
98 /// engine instead (unknown-child havoc in `eval_comp_app`): only the engine
99 /// knows whether the receiver is analyzable.
100 pub fn extract_handlers(cfg: &CFG, hooks: &mut Vec<HookEntry>, next_label: &mut HookLabel) {
```

Définition 7.4: Handler

Un *handler* est un corps de fonction qui *s'échappe* du rendu vers un tiers susceptible de l'appeler : sur un élément hôte, la valeur d'une prop `onX` ou `ref` ; sur un composant, la valeur de *n'importe quelle* prop. Il est extrait comme `HookEntry::HandlerLabel`, `event`, `body_cfg`, `span` quand sa valeur se résout en corps : un `FnLit` en ligne, une variable liée à un `FnLit`, ou un `CallbackVal(L)` dont le corps est celui du `useCallback`.

Rappel React : qui appelle quoi

React DOM n'invoque, sur un élément hôte, que les props d'événements `on[A-Z]...` et `ref` (appelé au montage et au démontage) ; toute autre prop est une donnée du DOM. Un composant, lui, peut appeler n'importe quelle fonction qu'il reçoit : `render props`, `children` en fonction, `action=cb` des formulaires de React 19.

L'algorithme a deux temps (`src/lowering/hook_extractor.rs`, l. 113–240). Une pré-passe construit une table `var_bodies` : nom de variable → corps, pour tout `Let` ou `Assign` du CFG de rendu dont le RHS est un `FnLit` ou un `CallbackVal` ; un seul corps par nom, le dernier rencontré gagne. Puis une descente dans toutes les expressions de premier niveau (instructions, `Return`, `Branch`) cherche les éléments JSX. Dans un `NativeElem`, une prop `onX/ref` résolue devient un handler, avec la position de la prop (tirée de `prop_spans`) et le nom d'événement de `prop_to_event` (`onClick` → `click`, `onMouseEnter` → `mouseenter`) ; les autres props et les enfants sont descendus. Dans un `CompApp`, toute prop résolue devient un handler, à la position de la balise ouvrante. Dans un `FnLit` encore présent dans le rendu (un `render helper`), la descente entre dans le corps.

Un setter nu passé en prop (`onChange=setN`) n'est *pas* un handler : que le receveur puisse l'appeler avec des arguments arbitraires dépend de ce que l'enfant est analysable, ce que seul le moteur sait (le *havoc* d'enfant inconnu, chapitre 16). Synthétiser ici une écriture $\top$ écraserait l'analyse inter-composants précise des enfants connus.

7.12.2 Les subscriptions

Extrait 7.39 – `src/lowering/hook_extractor.rs`, l. 18–41 — les subscriptions : `addEventListener` dans un corps d'effet

```
18 pub fn extract_subscriptions(hooks: &mut Vec<HookEntry>, next_label: &mut HookLabel) {
19     let mut new_handlers: Vec<HookEntry> = Vec::new();
```

```

20     for hook in hooks.iter() {
21         if let HookEntry::Effect { body_cfg, .. } = hook {
22             collect_subscriptions_in_cfg(body_cfg, &mut new_handlers, next_label);
23         }
24     }
25     hooks.extend(new_handlers);
26 }
27
28 fn collect_subscriptions_in_cfg(cfg: &CFG, out: &mut Vec<HookEntry>, next_label: &mut HookLabel) {
29     for block in cfg.blocks.values() {
30         for stmt in &block.stmts {
31             let (expr, span) = match stmt {
32                 Stmt::Let { rhs, span, .. } => (rhs, *span),
33                 Stmt::Assign { rhs, span, .. } => (rhs, *span),
34                 Stmt::MemberWrite { rhs, span, .. } => (rhs, *span),
35                 Stmt::ExprStmt(e, span) => (e, *span),
36             };
37             collect_subscriptions_in_expr(expr, span, out, next_label);
38         }
39         // Terminator::Return not scanned addEventListener is never a return expr.
40     }
41 }

```

Pour chaque `HookEntry::Effect`, la passe cherche dans les instructions du corps le motif `recv.addEventListener("<lit>", <FnLit>)` (où `<lit>` est une chaîne littérale) — le prédicat `Expr::subscription_listener`, partagé avec la relation des écrivains — et crée un handler dont l'événement est le littéral et le corps celui du listener. Le receveur et les arguments au-delà du deuxième sont redescendus; les corps de `FnLit` ne le sont pas (`for_each_child` ne les traverse pas), si bien qu'un listener enregistré dans un `setTimeout` n'est pas trouvé. Un nom d'événement non littéral, ou un listener passé par variable, ne donne rien (« acceptable FN » selon les tests). Dans `Panel`, c'est le label 6 (`event=resize`), à la position de l'instruction qui contient l'appel.

Piège

Le terminateur n'est pas lu. Le commentaire « `Terminator::Return` not scanned `addEventListener` is never a return expr » est faux pour un effet concis : dans `useEffect(() => el.addEventListener("x", f), [])`, l'appel est le `Return` du corps (section 7.13). La section 7.14 montre l'assurance fausse qui en résulte. `CFG::for_each_expr`, qui visite aussi les `Return` et les `Branch`, aurait couvert le cas.

7.13 Hoist de terminateur et flèches concises

Ces deux corrections forment une paire (#4 et #5, commit 30a00c5) : la seconde a rendu la première nécessaire.

Les flèches concises (#5). `oxc` représente `x => expr` comme un `FunctionBody` d'une seule `ExpressionStatement`, indiscernable de `x => expr`; sauf par le drapeau `expression`. Un consommateur qui oubliait ce drapeau abaissait le corps comme une instruction-expression, et la valeur de retour était perdue. La correction a deux parts : une fonction dédiée, et le point de dispatch unique `Candidate::build_cfg` (liste 7.2).

Extrait 7.40 – `src/lowering/cfg_builder.rs`, l. 985–1001 — le corps d'une flèche concise est un `Return`

```

985 /// Like [build_fn_body_cfg`] for concise arrow bodies (`x => expr`).
986 pub fn build_expr_fn_body_cfg(

```

```

987     params: &FormalParameters,
988     body: &FunctionBody,
989     ctx: &LowerCtx,
990 ) -> (Vec<String>, CFG) {
991     let mut builder = BlockBuilder::new(ctx);
992     builder.start_block(0);
993     let param_names = inject_param_preamble(params, &mut builder);
994     if let Some(Statement::ExpressionStatement(es)) = body.statements.first() {
995         let expr = lower_expr(&es.expression, &mut builder);
996         if !builder.is_terminated() {
997             builder.seal_with(Terminator::Return(expr));
998         }
999     }
1000     (param_names, builder.into_cfg(0))
1001 }

```

Le hoist de terminateur (#4). Mais alors, dans un hook custom concis comme `const useCount = () => useState(0)`, l'appel du hook est dans le terminateur `Return`, et `extract_hooks` ne réécrit que les instructions. Le hook ne produisait aucune entrée ; le composant déclarait zéro hook, et toutes les règles le laissaient passer en silence. Le commentaire (`src/lowering/hook_extractor.rs`, l. 306–320) explique le choix : apprendre à l'extracteur à parcourir aussi les terminateurs aurait dupliqué la déstructuration, l'étiquetage et la réécriture des marqueurs sur une seconde position ; on normalise donc le CFG.

Extrait 7.41 – `src/lowering/hook_extractor.rs`, l. 321–341 — sortir un appel de hook d'un terminateur

```

321 fn hoist_terminator_hooks(cfg: &mut CFG, imports: &ImportCtx<'_>) {
322     let ids: Vec<BlockId> = cfg.blocks.keys().copied().collect();
323     for id in ids {
324         let block = cfg.blocks.get_mut(&id).unwrap();
325         let (expr, span) = match &mut block.term {
326             Terminator::Return(e) => (e, None),
327             Terminator::Branch { cond, span, .. } => (cond, *span),
328             Terminator::Jump(_) | Terminator::Unreachable => continue,
329         };
330         if !contains_hook_call(expr, imports) {
331             continue;
332         }
333         let var = format!("__term_{id}");
334         let hoisted = std::mem::replace(expr, Expr::Var(var.clone()));
335         block.stmts.push(Smt::Let {
336             var,
337             rhs: hoisted,
338             span,
339         });
340     }
341 }

```

Tout `Return(e)` ou `Branchcond` dont l'expression *contient* un appel de hook est remplacé par `Var(__term_B)`, et `Let __term_B = e` est ajouté en fin de bloc — les instructions s'exécutent avant le terminateur, c'est donc la bonne place. L'extracteur voit ensuite un `Let` ordinaire. Le test est un test de *contenance* (`contains_hook_call`, `src/lowering/hook_extractor.rs`, l. 343–358), pour que l'expression entière se déplace d'un bloc. Le `Let` hissé d'un `Return` n'a pas de position, puisque `Return` n'en porte pas (#140) : le hook est rapporté, mais sans ligne — « it costs a position, never a finding », dit `docs/limitations.md`.

Exemple 7.5: Flèche concise, hoist, hooks conditionnels

```

1 import { useState, useEffect, useContext } from "react";
2
3 const useCount = () => useState(0);
4
5 export function Edge({ flag, Ctx }: { flag: boolean; Ctx: any }) {
6   const [c, setC] = useCount();
7   const theme = useContext(Ctx).theme;
8   flag && useEffect(() => {}, []);
9   const x = flag ? useState(1) : null;
10  return <div>{theme}{c}{x}</div>;
11 }

```

Extrait de la sortie de la sonde :

```

B0:
...
let __arr_170 = HookMarker(0, Unknown) @6:8
let c = __arr_170[0] @6:9
let setC = __arr_170[1] @6:12
let theme = useContext(Ctx).theme @7:8
let __t0 = flag @8:2
=> branch __t0 ? B1 : B2
B1:
__t0 := useEffect(fn#0() {
  B0:
    => return undefined
  edges:
}, [#1 ]<Exact(0)>) @8:10
=> jump B2
B2:
expr __t0 @8:2
=> branch flag ? B3 : B4
B3:
let __t1 = StateVal(1) @9:19
=> jump B5
...
hooks:
#0 Custom useCount() binding=None import_source=None resolved_file=Some("e7_edge.tsx") @6:8
#1 State init=1 @9:19
=== custom hook useCount (params []) ===
B0:
let __term_0 = StateVal(0)
=> return __term_0
hooks:
#0 State init=0

```

Dans `useCount`, `Return(useState(0))` a été normalisé en `let __term_0 = ...`, sans position, puis extrait. Dans `Edge`, le `useState(1)` d'un bras de ternaire est dans un `Let` : il est extrait, et l'analyseur rapporte `error conditional-hook [hook:1] (line 9:19)`. En revanche `useContext(Ctx).theme` (un hook imbriqué dans un `FieldAccess`) et `flag && useEffect(...)` (un hook dans un `Assign`) ne le sont pas : ils restent des `Call` ordinaires, et le composant annonce deux hooks.

Remarque

Le commentaire de `contains_hook_call` évoque `return cond ? useA() : useB()`. En réalité, un ternaire dans un `return` a déjà été fendu par `lower_ternary` : le terminateur ne lit que `Var(__tN)`, et les appels sont dans les `Let` des bras. Le hoist ne sert effectivement qu'aux appels directs ou imbriqués dans le terminateur ; et, pour un appel imbriqué (`return <div>useLabel()</div>`), le `Let` hissé n'a pas un appel pour racine, donc l'extracteur ne le reconnaît pas davantage.

7.14 Limites du lowering

7.14.1 Écarts vérifiés

Les écarts suivants ont été reproduits au commit e67b10a, chacun sur un couple « programme fautif / programme témoin » ; aucun n'avait d'issue sur le tracker au moment de la rédaction du dossier de notes. Tous sont des sous-approximations — une lecture, une écriture ou un chemin perdus —, sauf le paramètre de reste qui produit un faux positif. Tableau 7.5 les résume.

Source (extrait)	Cause, et verdict observé
<code>flag && useEffect(() => setN(1); , [])</code>	hook imbriqué (RHS d'un Assign) non extrait : AndHook (1 hooks), verified conditional-hook
<code>useContext(Ctx).theme; return <div>useLabel()</div></code>	hook non racine : aucune entrée, aucun analysis-limit
<code>const z = 0 ?? 5; if (z === 0) setX(1);</code>	?? comme + raffinement truthy : Coalesce ✓ verified setter-in-render ; le témoin <code>const z = 0</code> donne warn setter-in-render
<code>try obj = a:1; risky(); catch (e) cfg = obj;</code>	le catch part d'avant le try : Partial ✓ verified always-unstable-deps ; le témoin sans try donne warn always-unstable-deps
<code>useEffect(() => window.addEventListener("resize", ...), [])</code>	subscription dans le Return non lue : ConciseSub ✓ verified missing-cleanup ; la forme bloc donne warn missing-cleanup
<code>return <button onClick=handle/>; function handle() ...</code>	déclaration après return non abaissée : aucun handler, ✓
<code>onClick=flag ? () => setA(1) : () => setB(2)</code>	un corps par nom dans var_bodies : un seul handler (setB(2))
<code>useEffect(() => import(path); , [])</code>	repli <code>_ => opaque()</code> : lecture de path perdue, Lazy ✓ verified missing-deps ; le témoin <code>load(path)</code> donne warn missing-deps
<code>let x = 1; (x as any) = a: 1 ; (aussi x! = ..., x!++)</code>	cible enveloppée non écrite : TsTarget ✓ verified always-unstable-deps ; le témoin <code>x = a:1</code> donne warn
<code>for ([a, b] of pairs) ...</code>	cible motif pré-déclarée non écrite : a, b gardent leur valeur
<code>if (c) const [] = useState(0); (...args) => args.length</code>	aucun label laissé dans le CFG : verified conditional-hook reste non lu : args lu comme capture, warn missing-deps var:args (FP)

TABLE 7.5 – Écarts du lowering reproduits au commit e67b10a (fichiers /tmp/ch07/e8, e9, e13, e15–e21, v1–v4 ; sorties `-info -show-clean`).

Deux d'entre eux méritent d'être vus en entier, parce qu'ils produisent une assurance verified fausse. Le premier est le ?? :

```
Coalesce (1 hooks) e19_coalesce.tsx ✓
verified conditional-hook all hooks run unconditionally, in a stable order
verified lazy-init no useState/useRef initializer re-runs work on every render
verified setter-in-render no setter is called during render
verified state-mutation no state or prop object is mutated in place
Control (1 hooks) e19_coalesce.tsx
warn setter-in-render [hook:0] (line 16:4) setter `setX` called directly in the render body, ...
```

`0 ?? 5` vaut `0` ; le diamant l'envoie à la jonction par la branche `then_`, où le raffinement « truthy » retire `0` ; seul `5` survit, et le `setX(1)` gardé par `z === 0` devient inatteignable pour l'analyse. Le second est le hook conditionnel non extrait :

```
AndHook (1 hooks) e8_nested_hook.tsx
warn setter-in-render [hook:0] (line 5:28) setter `setN` is handed to a callee with no timing summary. ...
verified conditional-hook all hooks run unconditionally, in a stable order
```

L'appel `useEffect(...)` n'est pas extrait ; il reste un appel ordinaire à une fonction inconnue, à laquelle le setter est transmis (d'où l'avertissement `setter-in-render`), et `conditional-hook` certifie l'ordre des hooks sur le seul hook qu'il voit. Le composant est pourtant détecté : `hook_call_detect.rs` (la détection syntaxique sur l'AST, chapitre 5) voit ces formes, que l'extracteur ne voit pas.

Remarque

Trois écarts moins visibles complètent la liste. Les labels suivent l'ordre des blocs, non celui du texte : dans `if (a) if (b) ... const [x] = useState("x"); const [y] = useState("y");`, la jonction de l'if externe a été réservée avant les blocs de l'if interne, et y reçoit le label 0, x le label 1 (le verdict, `error conditional-hook [hook:1] (line 8:10)`, est juste). Les tests `case k` : d'un `switch` ne sont pas abaissés. Et `useReducer(r, init, initFn)` jette le réducteur et le troisième argument, avec leurs lectures.

Aucun de ces écarts ne demande de cas particulier dans une règle ; tous se corrigent dans le lowering, selon les principes du projet. Quelques pistes, relevées par le dossier de notes et non décidées par le projet : peler les enveloppes TypeScript d'une cible d'affectation avant `assign_target_ident`, comme `lower_expr` le fait pour les expressions ; lire les subscriptions par CFG : `for_each_expr` ; généraliser la normalisation du #4 en liant tout sous-appel de hook dans un `Let` au point de son évaluation ; donner à ?? une forme propre dont la branche de jonction ne raffine pas en `truthy`.

7.14.2 Pièges pour qui modifie ce code

Piège

- Ne jamais ajouter de bras `_ =>` dans `lower_stmt`, `lower_binop`, `for_each_child` ou `HookEntry::body_cfg` : l'exhaustivité est la garantie (#77).
- Toute nouvelle instruction synthétique porte une position (ADR-39) ; toute expression non représentable passe par `lower_for_effect` plutôt que d'être jetée (invariant 7.2) ; toute nouvelle forme de cible d'affectation écrit la liaison, au pire à $\top$ (invariant 7.3).
- Toujours `add_edge` après `seal_with(Jump|Branch)`, et tester `is_terminated()` avant de sceller une fin de branche (sinon le `debug_assert!` de `seal_with` se déclenche).
- `lower_expr` ne scelle jamais un bloc en `Return` ou `Unreachable` : ses seules fentes (diamants, `split_at_await`) laissent un bloc ouvert. C'est ce qui garantit qu'une jonction de diamant a toujours deux prédécesseurs.
- `Unreachable` ne se produit que si le contrôle ne continue pas (ADR-25).
- Le préfixe `__arr_` est lu par l'extracteur ; `__term_`, `__tN`, `__obj_`, `__dstr_` et `__pI` sont réservés.
- `expr_lower::is_event_prop_key` et `hook_extractor::is_event_prop` ont un corps identique : le premier décide quelles props reçoivent une position, le second lesquelles deviennent des handlers. Les modifier séparément désalignerait handlers et positions.
- `extract_hooks` *déplace* les corps d'effets, de memos et de callbacks hors du rendu : une passe qui les chercherait dans le rendu ne les trouvera pas. C'est pourquoi un `<button onClick>` construit dans un `useMemo` ne produit aucun `HookEntry::Handler`.

Résumé du chapitre

Le lowering traduit l'AST d'oxc en un CFG de blocs de base et une table de hooks ; après lui, rien ne voit plus l'AST. Il s'organise en trois passes : `Candidate::build_cfg` (instructions dans `cfg_builder.rs`, expressions dans `expr_lower.rs`), puis `extract_hooks`, `extract_handlers` et `extract_subscriptions` (`hook_extractor.rs`), dans cet ordre, qui fixe aussi l'ordre des labels.

- **La machine à blocs.** `BlockBuilder` réserve, scelle, ouvre et relie des blocs selon un protocole unique ; `lower_stmts` s'arrête au premier terminateur ; `into_cfg` scelle la chute en fin de corps en `Return(undefined)`, et `Unreachable` ne veut dire que « le contrôle ne continue pas » (ADR-25).
- **Les gabarits.** `if` : trois blocs, même sans `else` ; retour anticipé : une branche sans arête vers la jonction. Boucles : en-tête, corps, sortie (et `update` pour `for`), retour par une arête `Back`, seul lieu du widening ; `continue` est une arête `Back`, `break` une vraie arête. `switch` : chaîne de dispatchs `Branch(true)`, une case par bloc, fall-through fidèle (#1). `try` : `Branch` sur `⊥`, `finally` sur la jonction (#2). Diamants : un temporaire `__tN`, un `Branch` sur lui (ADR-20 §1). `await` : argument lié, puis arête `Await` (ADR-35).
- **Les expressions.** Ne perdre aucune lecture (`lower_for_effect`, clés synthétiques), écrire chaque feuille ; `⊥` (`opaque()`) pour ce qui n'est pas modélisé. Chaque nœud allouant reçoit un `ExprId` tiré d'un compteur partagé par fichier et par famille (ADR-10, #134).
- **Les hooks.** Reconnaissance d'un appel racine de `Let/ExprStmt`, par provenance d'import ; entrée `HookEntry`, marqueur, ligne de provenance ; destructuration d'état par `__arr_` ; corps déplacés hors du rendu. Handlers par fuite ; subscriptions dans les corps d'effets. Le hoist de terminateur (#4) rend les flèches concises (#5) sûres.
- **Les limites.** Hooks imbriqués non extraits, ?? lu comme `||`, `catch` partant d'avant le `try`, subscriptions d'effets concis, déclarations après `return`, cibles TypeScript : autant d'assurances fausses possibles, toutes corrigeables à la racine dans ce sous-système.

Fichiers rencontrés : `src/lowering/mod.rs`, `cfg_builder.rs`, `expr_lower.rs`, `hook_extractor.rs`. Le chapitre suivant (chapitre 8) greffe ces CFG les uns dans les autres : il lira les `Return` et les `Unreachable` que nous avons produits, et renommera les temporaires. Le moteur (chapitre 12) interprétera les arêtes `Back` et `Await` ; les relations (chapitre 14) liront les phases que l'`await` délimite.

Exercices

Exercice 7.1: Un `for` à la main

Dessinez le CFG que produit le lowering pour `for (let i = 0; i < n; i++) if (a[i]) continue; f();` : numéros de blocs, terminateurs, arêtes et leur nature. Comparez au test `continue_reaches_the_loop_header` (`src/lowering/cfg_builder.rs`, l. 1144–1169).

Solution

Le bloc courant reçoit `let i = 0` puis saute (U) à l'en-tête H ; les blocs réservés sont, dans l'ordre, H, le corps C, l'`update` U et la sortie X. H porte `branch (i < n) ? C : X (T/F)`. Dans C, l'`if` réserve trois blocs : le `then` contient le `continue`, soit `jump U` par une arête `Back` ; le `else` saute à la jonction, qui contient `expr f()` puis `jump U (U)`. U contient `i := (i Add 1)`, `expr i` sans position, puis `jump H` par une arête `Back`. Deux arêtes arrière aboutissent donc à U ou à H.

Exercice 7.2: Pourquoi continue est une arête Back

Écrivez une boucle dont le compteur n'avance que sur le chemin d'un `continue`, et expliquez pourquoi le point fixe ne convergerait pas si l'arête du `continue` était `Unconditional` (commentaire `src/lowering/cfg_builder.rs`, l. 390–393).

Solution

Par exemple `while (left < right) if (p(left)) left += 1; continue; right -= 1;`. Sur le chemin du `continue`, `left` croît à chaque tour ; si ce chemin revient à l'en-tête par une arête `Unconditional`, l'en-tête n'est pas un point de widening pour lui, et l'intervalle de `left` y croît d'une unité par itération du point fixe sans jamais se stabiliser. Avec une arête `Back`, le widening s'applique à l'en-tête : l'intervalle est étendu d'un coup (vers $+\infty$ ou vers le seuil suivant), et l'itération converge en un nombre fini de pas.

Exercice 7.3: Un switch et un niveau de diagnostic

Donnez le CFG d'un `switch` à trois cases dont la deuxième n'a pas de `break`, et justifiez que `setMsg` dans la première case ne peut être qu'un `WARNING` de `setter-in-render` (exemple 7.2).

Exercice 7.4: Garde-throw et splice

En vous appuyant sur ADR-25 §2 et le test `a_hook_after_a_guard_throw_is_not_conditional` (`tests/cfg_exit_integrity.rs`, l. 138–162), montrez que relier le `Unreachable` d'un `throw` à la jonction du `splice` ferait signaler comme conditionnel le `useState` qui suit la garde, au niveau `ERROR`.

Exercice 7.5: Un défaut de motif

Pour `const a = f() = props`, listez les instructions émises, dans l'ordre, et dites quelles lectures sont préservées et quelle valeur reçoit `a`.

Solution

`let __obj_N = props` (position du déclarateur), `expr f()` (position du défaut), `let a = __obj_N.a` (position de la propriété). Les lectures de `props` et de `f` sont préservées, et l'appel `f()` est visible comme effet ; `a` reçoit `props.a` seulement, sans le résultat de `f()`.

Exercice 7.6: Corriger à la racine

Sur `flag && useEffect(...)`, vérifiez avec l'analyseur que le hook n'est pas extrait. Proposez une correction conforme aux principes du projet (pas de cas particulier dans `process_stmt` ni dans une règle), en vous inspirant de la façon dont le #4 a été résolu.

Exercice 7.7: Le catch partiel

Reproduisez l'écart `Partial` de la tableau 7.5, puis proposez une modélisation sound du `try` : d'où devraient partir les arêtes vers le `catch` ? Quel coût en précision et en force `must` votre forme aurait-elle ?

Chapitre 8

Splice, remap et inlining

Objectifs du chapitre.

- Comprendre pourquoi `reactant-analyzer` analyse un hook custom ou un utilitaire *dans* chacun de ses appelants, en greffant son CFG, plutôt qu’une seule fois pour lui-même (ADR-5, ADR-24).
- Connaître les quatre espaces d’identités qu’une greffe doit décaler — blocs, labels de hooks, sites d’allocation, noms de variables — et les types qui les portent : `Offsets`, `SpliceIds`, la table de renommage.
- Savoir lire, étape par étape, la primitive unique `splice_callee_into_cfg` : coupe du bloc d’appel, `Return` réécrit en `Let + Jump`, `Unreachable` laissé coupé, arêtes, positions.
- Maîtriser l’hygiène de la greffe : α -renommage des liaisons de l’appelé, isolement de ses noms libres en collision (#141), substitution des arguments dans les initialiseurs d’état.
- Suivre l’expansion d’un hook custom (`expand_custom_hooks`) et l’inlining d’un utilitaire (`inline_in_cfg`), et les distinguer des deux mécanismes qui *ne* greffent pas : fermetures locales (B5/B6) et composants enfants.
- Savoir ce que fait un hook de bibliothèque sans source (`SummaryRegistry`, « connu n’est pas modélisé ») et comment un finding né dans un hook inliné est attribué (ADR-24).
- Connaître les cas limites et les défauts observés de la greffe, et savoir où les corriger.

Prérequis : chapitre 6 (CFG, définition 6.1 ; sites, définition 6.2 ; slots et labels, définition 6.3 ; unités `HookIR/FunctionIR`), chapitre 7 (`Return(undefined)` en fin de corps, `Unreachable`, `HookMarker`), chapitre 2 (ordre des hooks), chapitre 3 (`soundness`, définition 3.8 ; dominance, définition 3.9).

Le lowering (chapitre 7) produit trois sortes d’unités : des composants, des hooks custom, des utilitaires. Chacune a son CFG, ses labels, ses sites d’allocation et ses noms, tous numérotés à partir de zéro. Le moteur, lui, ne sait interpréter qu’une composant à la fois (chapitre 13). Ce chapitre décrit la couche qui fait le pont : avant le premier tour de point fixe, le moteur *greffe* dans le CFG de rendu du composant le corps de chaque hook custom et de chaque utilitaire qu’il appelle. Cette opération s’appelle un *splice* ; appliquée à un appel, c’est un *inlining*. Elle vit dans deux fichiers de l’IR, `src/ir/splice.rs` et `src/ir/remap.rs`, et deux fonctions du moteur, `expand_custom_hooks` et `expand_utility_calls` (`src/engine/fixpoint.rs`).

8.1 Pourquoi inliner

8.1.1 Un hook n’a pas d’analyse à lui

Partons d’un compteur qui délègue son état à un hook custom.

Extrait 8.1 – Un hook custom qui boucle (`/tmp/ch08/hook/Counter.tsx`)

```
1 import { useState, useEffect } from "react";
```

```

2
3 function useCounter(initial: number) {
4   const [count, setCount] = useState(initial);
5   useEffect(() => {
6     setCount(count + 1);
7   }, [count]);
8   return count;
9 }
10
11 export function Counter() {
12   const count = useCounter(0);
13   return <div>{count}</div>;
14 }

```

Le défaut est dans le hook : l'effet dépend de `count` et l'incrémente, donc chaque exécution en déclenche une autre. Mais c'est `Counter` qui boucle, et c'est lui que React rend. L'analyseur le rapporte sur le composant, en pointant la ligne du hook :

```

Counter (1 hooks) Counter.tsx
warn  infinite-loop [hook:1] (line 5:2) this effect keeps pushing state `count` (its deps do not provably gate it
↳ , so the effect can re-run every render) to new values on every run. Potential infinite render loop
→ state `count` is written here [hook:2] (line 5:2)
→ the abstract value of state `count` kept growing and was widened at iteration 3

```

Pourquoi ne pas analyser `useCounter` une fois pour toutes, et réutiliser le résultat dans chaque appelant ? Parce que les faits d'un hook dépendent de ses arguments. Changeons l'exemple : l'effet ajoute un pas `step` fourni par l'appelant.

Extrait 8.2 – Le même corps, deux faits incomparables (/tmp/ch08/step/Step.tsx)

```

1 function useStep(step: number) {
2   const [n, setN] = useState(0);
3   useEffect(() => {
4     setN(n + step);
5   }, [n]);
6   return n;
7 }
8
9 export function L() {
10   const n = useStep(1);
11   return <div>{n}</div>;
12 }
13
14 export function M() {
15   const n = useStep(0);
16   return <div>{n}</div>;
17 }

```

```

L (1 hooks) Step.tsx
warn  infinite-loop [hook:1] (line 5:2) this effect keeps pushing state `n` (its deps do not provably gate it, so
↳ the effect can re-run every render) to new values on every run. Potential infinite render loop
(2 trace step(s), rerun with --trace)

M (1 hooks) Step.tsx
warn  redundant-set-state [hook:1] (line 5:2) state `n` is set to a stable value it already holds, so the update
↳ is redundant
(1 trace step(s), rerun with --trace)

```

Même corps, même ligne, deux diagnostics différents : avec `step = 1` la valeur croît sans fin ; avec `step = 0` l'écriture réécrit la valeur courante. Une analyse unique de `useStep` avec un paramètre inconnu ($\top$) ne donnerait ni l'un ni l'autre avec certitude. ADR-24 le dit en une phrase : « A hook is not a component and has no analysis of its own ; its facts exist only relative

to the arguments a consumer supplies. » La conséquence architecturale est immédiate : on analyse le hook *dans* chaque appelant, avec les arguments de cet appelant.

Rappel React : les hooks d'un hook custom appartiennent à l'appelant

Pour React, un hook custom n'existe pas : c'est une fonction ordinaire dont le nom commence par `use`. Quand `Counter` appelle `useCounter(0)`, le `useState` et le `useEffect` exécutés à l'intérieur sont enregistrés dans la liste de hooks de la fibre de `Counter`, à la suite de ceux que `Counter` a déjà appelés. Deux composants qui appellent le même hook ont chacun leur état. C'est pourquoi la greffe *continue* la numérotation des labels de l'appelant (section 8.2) : le slot du `useState` de `useCounter` est un slot de `Counter`.

8.1.2 Cinq façons de franchir un appel

L'analyse est intra-procédurale au sens où le moteur n'interprète qu'un composant à la fois ; mais un composant appelle des choses. Le code distingue cinq cas, résumés au tableau 8.1. Trois sont des greffes ou leur substitut ; deux ne greffent rien.

Appel	Mécanisme	Où
hook custom dont la source est lue	greffe du corps entier	<code>expand_custom_hooks</code> , <code>HookRegistry</code> (section 8.5)
utilitaire de module appelé en position d'instruction	greffe du corps entier	<code>expand_utility_calls</code> , <code>FunctionRegistry</code> (section 8.6)
hook de bibliothèque sans source	résumé abstrait, pas de greffe	<code>SummaryRegistry</code> (section 8.7)
fermeture locale (<code>go()</code> , <code>setTimeout(cb)</code>)	exécution du corps de puis le tas, à l'interprétation	<code>B5/B6</code> , <code>exec_var_callback</code> (section 8.6.3)
composant enfant <code><Child/></code>	analyse séparée avec props abstraites	<code>eval_comp_app</code> (ADR-12, chapitre 16)

TABLE 8.1 – Les cinq traitements d'un appel. Seules les deux premières lignes modifient le CFG.

Décision : ADR-5 — intra-procédural, plus un registre de hooks

ADR-5 (mai 2026) pose le cadre. Phase 1 : chaque composant et chaque hook custom est analysé indépendamment, un hook non reconnu rend « Unknown » pour toutes ses valeurs. Phase 2, annoncée comme future : un inlining « call-string-1 » — à chaque appel `useX(args)`, substituer le corps du hook dans le CFG de l'appelant avec les arguments, à profondeur 1. Mécanisme central pour les hooks sans source : un registre en couches (hooks natifs, modules de bibliothèque, configuration utilisateur, repli « inconnu »). L'argument : l'inter-procédural est coûteux et inutile pour les premiers bugs visés.

Le code a dépassé cette ADR sur deux points qu'il faut connaître pour ne pas la lire comme une spécification. L'inlining existe et n'est pas limité à la profondeur 1 : un hook custom qui en appelle un autre est expansé récursivement (garde par nom, section 8.5), et les utilitaires sont greffés eux aussi (jusqu'à un budget de 8 par CFG). Le squelette de registre de l'ADR (`HookModel`, `HookResult`, fichiers `tanstack.rs`, `user_config.rs`, `reactant.toml`) n'existe pas sous cette forme : `src/registry/` contient `keyed.rs`, `mod.rs` et `summary.rs`, où le trait s'appelle `HookSummary` (section 8.7).

8.1.3 Où la greffe se place dans l'analyse d'un composant

Les deux passes d'expansion s'exécutent au début de `analyze_component_impl`, avant l'amorçage des slots et avant le premier tour de point fixe :

Extrait 8.3 – `src/engine/fixpoint.rs`, l. 197–229 — les deux passes d'expansion, dans l'ordre

```

197 // The identity supplies every splice in this component draws from.
198 let mut splice_salt = SpliceIds::for_component(&render_cfg, &hooks);
199
200 // Utility-function inlining. Runs before `expand_custom_hooks` so utility
201 // bodies containing hook calls become visible to the hook expansion pass.
202 if expand_utility_calls(
203     &mut render_cfg,
204     &mut hooks,
205     &config.function_registry,
206     &comp_file,
207     config.max_inline_depth,
208     &mut inline_origins,
209     &mut inline_regions,
210     &mut splice_salt,
211 ) && let Some(inter) = inter
212 {
213     inter
214         .stats
215         .borrow_mut()
216         .inline_budget_exhausted
217         .insert(comp_id);
218 }
219
220 // Expand Custom entries before seeding so inlined State entries are seeded.
221 expand_custom_hooks(
222     &mut hooks,
223     &mut hook_provenance,
224     &mut render_cfg,
225     inter,
226     &mut inline_origins,
227     &mut inline_regions,
228     &mut splice_salt,
229 );

```

Trois choses sont à retenir de cet extrait. Une réserve d'identités, `SpliceIds`, est créée *une fois par composant* et partagée par toutes les greffes (section 8.2). Les utilitaires sont greffés *avant* les hooks, « so utility bodies containing hook calls become visible to the hook expansion pass » ; l'ordre inverse a une conséquence que nous verrons (section 8.9). Enfin les hooks sont expansés « before seeding so inlined State entries are seeded » : après l'expansion, le `useState` de `useCounter` est une entrée State ordinaire de `Counter`, et le moteur l'amorce comme les autres.

8.2 Décaler les identités : le remap

8.2.1 Quatre espaces numérotés à partir de zéro

Greffer un CFG dans un autre, c'est copier des objets qui portent des identifiants. Or l'appelant et l'appelé ont été abaissés séparément, et chacun numérote ses choses depuis zéro. Il y a quatre espaces d'identités (tableau 8.2). Pour chacun, une collision serait silencieuse : deux blocs de même `BlockId` s'écraseraient dans la `BTreeMap` des blocs ; deux hooks de même label partageraient un slot ; deux littéraux de même `ExprId` partageraient une entrée de tas ; deux

variables de même nom partageraient une entrée d’environnement.

Espace	Décalé par	Règle
BlockId	splice_callee_into_cfg	blocs de l’appelé placés au-dessus du plus grand bloc de l’appelant ; jointure juste après
HookLabel	Offsets::labels, remap_*	labels de l’appelé placés au-dessus du plus grand label courant
ExprId (sites)	Offsets::ids, remap_*	plage fraîche tirée d’un curseur par composant (SpliceIds)
Var	table de renommage, sel	x devient x#s, un sel s par greffe (section 8.4)

TABLE 8.2 – Les quatre espaces d’identités d’une greffe et qui les décale.

Les BlockId sont renumérotés par la primitive de splice elle-même (section 8.3). Les noms de variables relèvent de l’hygiène (section 8.4). Les deux autres espaces, labels et sites, sont décalés par src/ir/remap.rs, avant la greffe, sur une copie du corps de l’appelé.

8.2.2 offsets : un seul porteur pour deux décalages

Extrait 8.4 – src/ir/remap.rs, l. 11–37 — ce qu’une greffe décale

```

11 /// What grafting a callee into a caller shifts.
12 ///
13 /// Both are identity supplies the splice hands the copy it makes, and both are
14 /// needed for the same reason: the caller and the callee number their own
15 /// things from zero. Labels were shifted from the start; allocation sites were
16 /// not, so a callee's `ObjectLit` shared a heap entry with the caller's and
17 /// answered its member reads (#134). One carrier, so a new splice site cannot
18 /// remember one and forget the other.
19 #[derive(Debug, Clone, Copy, PartialEq, Eq, Default)]
20 pub struct Offsets {
21     pub labels: HookLabel,
22     /// Added to every allocation-site [`ExprId`]. Unique per splice: the same
23     /// callee inlined twice is two allocation sites, not one.
24     pub ids: usize,
25 }
26
27 impl Offsets {
28     /// Shift labels only – for a remap that is not a graft (a hook table
29     /// renumbered in place keeps its own allocation sites).
30     pub fn labels(labels: HookLabel) -> Self {
31         Offsets { labels, ids: 0 }
32     }
33
34     fn is_identity(self) -> bool {
35         self.labels == 0 && self.ids == 0
36     }
37 }

```

Offsets a deux champs. labels est ajouté à chaque HookLabel ; ids à chaque ExprId de site d’allocation. Le commentaire explique pourquoi ils sont dans une même structure : historiquement, les labels étaient décalés depuis le début et les sites ne l’étaient pas, si bien qu’un ObjectLit de l’appelé partageait l’entrée de tas d’un objet de l’appelant et répondait à ses lectures de membres (#134). Réunir les deux dans un seul porteur rend impossible, pour un futur site de greffe, de penser à l’un et d’oublier l’autre — une application directe du principe

« corriger au centre ». Le constructeur `Offsets::labels` (labels seuls, pour renuméroter une table en place) n’a au e67b10a aucun appelant de production : seuls les tests de `remap.rs` l’emploient, et les deux sites de greffe construisent `Offsets labels, ids` explicitement.

8.2.3 La largeur d’une greffe : `alloc_id_span`

Pour tirer une plage de sites fraîche, il faut savoir combien de sites l’appelé occupe. C’est le rôle de `alloc_id_span` (`src/ir/remap.rs`, l. 39–67). La fonction rend un de plus que le plus grand `ExprId` de site rencontré (`ObjectLit`, `ArrayLit`, `New`, `FnLit`), en descendant dans les corps de `FnLit` imbriqués — un corps de fermeture est un CFG à part, que `for_each_child` ne parcourt pas. La profondeur de descente est bornée à 64 niveaux de fermetures, « and no real body approaches it ». La mesure suppose que les sites d’un corps forment l’intervalle $0..n$, ce que garantit le compteur par fichier du lowering (section 7.9) : on mesure un maximum, pas un nombre.

8.2.4 Appliquer le décalage : `remap_expr`, `remap_cfg`, `remap_hooks`

`remap_expr` (`src/ir/remap.rs`, l. 69–171) consomme l’expression (passage par valeur) et en rend une nouvelle. Les cinq expressions qui portent un label — `StateVal`, `StateSetter`, `MemoVal`, `CallbackVal`, `HookMarker` — reçoivent $1 + \text{offset}$. Les nœuds allouants (`FnLit`, et dans la suite du match, `ObjectLit`, `ArrayLit`, `New`) reçoivent $\text{id} + \text{off.ids}$. Le corps d’un `FnLit` est un `Arc<CFG>` : `Arc::try_unwrap` évite de le copier quand personne d’autre ne le partage. Les autres variantes se contentent de récuser ; les feuilles (`Lit`, `Var`, `SummaryVal`) sont rendues telles quelles. Si `off` est l’identité, la fonction court-circuite (test `remap_zero_is_identity`).

`remap_cfg` (`src/ir/remap.rs`, l. 221–252) applique `remap_expr` à toutes les instructions et à tous les terminateurs, *sans toucher aux* `BlockId` : ce n’est pas son rôle. `remap_hooks` (`src/ir/remap.rs`, l. 254–344) décale le label de chaque entrée de la table de hooks, et toutes les expressions et tous les corps qu’elle porte : `init` d’un `State` ou d’un `Ref`, `body_cfg` d’un `Effect`, `Memo`, `Callback` ou `Handler`, `args` d’un `Custom`, et les éléments des `deps` (via `map_exprs`, qui préserve l’arité et les positions de spread).

8.2.5 La réserve d’identités d’un composant : `SpliceIds`

Deux greffes dans un même composant ne doivent pas se marcher dessus non plus. Le moteur tire donc sel et sites d’une réserve unique, monotone :

Extrait 8.5 – `src/engine/fixpoint.rs`, l. 837–867 — la réserve de sels et de sites d’une composante

```
837 pub(crate) struct SpliceIds {
838     salt: u32,
839     next_alloc: usize,
840 }
841
842 impl SpliceIds {
843     fn for_component(render_cfg: &CFG, hooks: &[HookEntry]) -> Self {
844         SpliceIds {
845             salt: 0,
846             next_alloc: alloc_span(render_cfg, hooks),
847         }
848     }
849
850     /// One allocation site, for something the source did not allocate - a
851     /// library hook's per-member summary shape (#94).
852     fn alloc_one(&mut self) -> crate::ir::types::ExprId {
853         let id = self.next_alloc;
```

```

854     self.next_alloc += 1;
855     crate::ir::types::ExprId(id)
856 }
857
858 /// The salt and allocation-id offset for one graft `span` ids wide,
859 /// advancing both supplies past it.
860 fn take(&mut self, span: usize) -> (u32, usize) {
861     let salt = self.salt;
862     self.salt += 1;
863     let ids = self.next_alloc;
864     self.next_alloc += span;
865     (salt, ids)
866 }
867 }

```

`for_component` place le curseur `next_alloc` au-dessus des sites déjà présents dans le composant ; `take(span)` rend le sel courant et le début d’une plage de `span` sites, puis avance les deux ; `alloc_one` tire un site isolé pour une forme que la source n’alloue pas, l’objet résumé d’un hook de bibliothèque (section 8.7). La largeur d’une greffe est mesurée par `alloc_span(cfg, hooks)` (`src/engine/fixpoint.rs`, l. 869–882) : le maximum de `alloc_id_span` sur le corps et sur chaque `HookEntry::body_cfg()` de la table. Le corps d’un hook et les corps de ses entrées ont été numérotés par le *même* compteur de fichier : ils partagent une plage, d’où le `max` plutôt qu’une somme, et un seul décalage pour les deux.

Invariant 8.1: Plages disjointes (intention)

Dans un composant, le sel et le curseur de sites sont monotones : chaque greffe reçoit un sel neuf et une plage de sites $[c, c + w)$ située au-dessus de tout ce qui a été attribué avant elle. L’intention, énoncée dans la doc de `SpliceIds` (`src/engine/fixpoint.rs`, l. 828–836), est que deux greffes n’occupent jamais le même site, ni entre elles ni avec l’appelant ; c’est aussi ce qui fait que *le même* appelé greffé deux fois donne deux sites et non un seul.

Piège

L’invariant est une intention, pas une garantie. `alloc_span` ne mesure que les CFG : le corps et les `body_cfg()` des entrées. Or `HookEntry::body_cfg()` rend `None` pour `State`, `Ref` et `Custom`, si bien que les sites portés par un `init`, par les args d’un hook custom ou par des `deps` ne sont pas comptés — alors que `remap_hooks` les *décale*, et que les arguments d’un `Custom` finissent dans le CFG de rendu quand le hook est expansé. Nous montrerons une collision effective au section 8.9.

8.3 La primitive de splice

8.3.1 Un utilitaire à deux retours

Commençons par la greffe la plus simple à lire : un utilitaire de module, sans hook, dont le corps a plusieurs blocs.

Extrait 8.6 – Un utilitaire à deux return (`/tmp/ch08/clamp/Counter.tsx`)

```

1  import { useState } from "react";
2
3  function clamp(n: number) {
4    if (n > 10) {
5      return 10;
6    }
7    return n;

```

```

8 }
9
10 export function Counter() {
11   const [count, setCount] = useState(0);
12   const shown = clamp(count);
13   return <button onClick={() => setCount(count + 1)}>{shown}</button>;
14 }

```

Le lowering produit deux CFG séparés. Celui du rendu de Counter a un seul bloc, où l'appel est une instruction `let shown = clamp(count)`. Celui de `clamp` (un `FunctionIR`) a quatre blocs : le branch, le bras `then` qui retourne, le bras implicite d'un `if` sans `else` (B2), et la suite (B3), qui retourne `n`. La figure 8.1 reproduit, élaguées, les impressions de la sonde avant et après la greffe ; les arêtes observées après sont `B0->B1:Unconditional` `B1->B2:IfTrue` `B1->B3:IfFalse` `B3->B4:Unconditional` `B2->B5:Unconditional` `B4->B5:Unconditional`. Tout le reste de cette section explique comment on passe de l'une à l'autre.

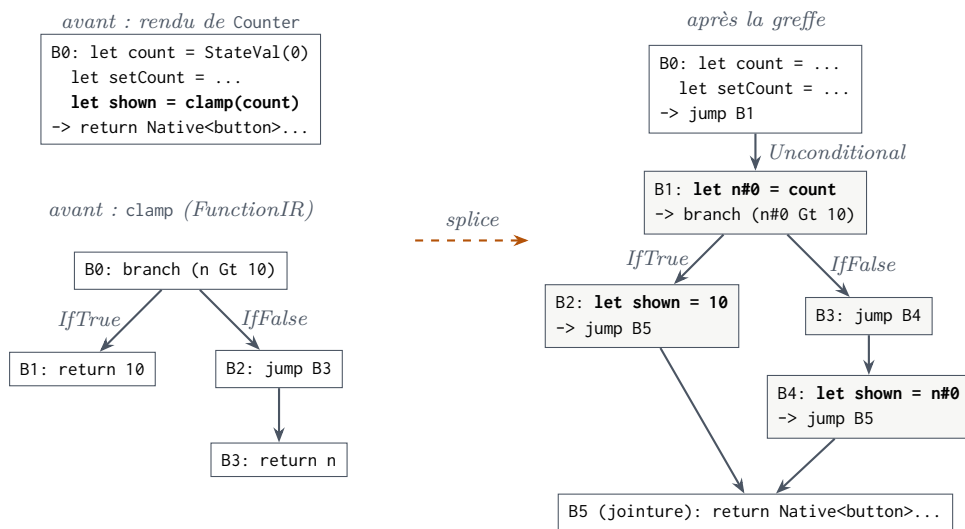


FIGURE 8.1 – Avant et après la greffe de `clamp` dans Counter. Les blocs de l'appelé (fond coloré) sont décalés de 1 (le plus grand bloc de l'appelant plus un) ; la jointure B5 reçoit les instructions qui suivaient l'appel et l'ancien terminateur ; chaque `return e` devient `let shown = e` suivi d'un saut vers la jointure ; le paramètre est lié par `let n#0 = count` en tête de l'entrée.

8.3.2 Le contrat : Splice

La primitive prend l'appelant, la position de l'instruction d'appel, et une description de l'appelé :

Extrait 8.7 – `src/ir/splice.rs`, l. 33–54 — le côté appelé d'une greffe

```

33 /// The callee side of a splice: what to graft in and how to bind it.
34 ///
35 /// - `params`/`args` are zipped into `let param = arg;` bindings prepended to
36 ///   the callee's entry block (extra params or args are ignored, as in JS).
37 /// - `bound_var` receives the callee's `Return(e)` value (`Some` for
38 ///   `let x = f(..)`, `None` to discard as in a bare `f(..)`).
39 /// - `rename` maps each callee-bound variable to its fresh name (build it with
40 ///   [`callee_rename_map`]). It must be built from the same params and body,
41 ///   and – for a hook whose effect/memo/callback bodies capture render-scope
42 ///   locals – the identical map must also be applied to those sub-hook bodies
43 ///   (via [`rename_vars_cfg`]) so captures stay linked to the renamed binding.

```



```

44 ///
45 /// `callee` must already have its `HookLabel`s remapped (see
46 /// [`crate::ir::remap::remap_cfg`]) when it carries hook slots; the splice only
47 /// rennumbers `BlockId`s and alpha-renames variables.
48 pub struct Splice<'a> {
49     pub callee: CFG,
50     pub params: &'a [Var],
51     pub args: &'a [Expr],
52     pub bound_var: Option<&'a Var>,
53     pub rename: &'a HashMap<Var, Var>,
54 }

```

Champ par champ :

- `callee` : le CFG à greffer, *possédé* (la primitive le consomme). S'il porte des slots de hooks, ses labels doivent déjà avoir été décalés par `remap_cfg` : la primitive ne renumérote que les blocs et ne renomme que les variables.
- `params`, `args` : les paramètres de l'appelé et les arguments de l'appel, appariés en `let param = arg` au début du bloc d'entrée.
- `bound_var` : la variable de l'appelant qui reçoit la valeur de retour (`Some("shown")` pour `let shown = clamp(...)`, ou `None` pour une instruction nue `f(...)`).
- `rename` : la table d' α -renommage, construite par `callee_rename_map` (section 8.4). Pour un hook dont les corps d'effets capturent des locales du rendu, *la même* table doit être appliquée à ces corps. (La doc renvoie à `rename_vars_cfg` ; le moteur passe en fait par `rename_hook_entry`, qui l'appelle.)

La fonction rend la plage demi-ouverte des `BlockId` où ont atterri les blocs de l'appelé, ou `None` si la greffe a été sautée. Ce n'est pas un détail : « the recorded pair every caller needs for provenance » (`src/ir/splice.rs`, l. 56–63). Le moteur enregistre cette plage comme région inlinée ; sans elle, une écriture de `setter` faite par un wrapper serait certifiée comme écrite directement par l'appelant (ADR-27 §4, section 8.5).

8.3.3 Étapes 1 à 4 : renommer, vérifier, couper, allouer

Extrait 8.8 – `src/ir/splice.rs`, l. 77–134 — renommage, préconditions, coupe du bloc d'appel, allocation des blocs et liaisons de paramètres

```

77 // 1. Alpha-rename callee-bound variables (params + locals) to fresh names.
78 let CFG {
79     entry: callee_entry_orig,
80     blocks: callee_block_map,
81     edges: callee_edges,
82 } = rename_vars_cfg(callee, rename);
83 let renamed_params: Vec<Var> = params.iter().map(|p| rename_one(rename, p)).collect();
84
85 // 1b. Both preconditions, checked before anything is mutated. Step 6 used
86 //     to bail here instead – after the caller's block had already been split
87 //     and its terminator replaced with `Unreachable`, so the bail left the
88 //     caller truncated: `post` dropped, the original terminator lost, and
89 //     no assertion or rollback to say so. Either way the splice is skipped;
90 //     the difference is that the caller now survives it intact.
91 if !callee_block_map.contains_key(&callee_entry_orig) {
92     return None; // headless callee: nothing to graft
93 }
94 if !caller.blocks.contains_key(&block_id) {
95     return None; // the call site named a block that is not there
96 }

```

```

97
98 // 2. Split the caller block at the call site: `pre` stays, the call stmt is
99 //   dropped, `post` moves to a fresh join block.
100 let block = caller.blocks.get_mut(&block_id).unwrap();
101 let mut post: Vec<Stmt> = block.stmts.split_off(stmt_idx);
102 // Everything the splice *synthesises* – the param bindings, and the
103 // assignment a callee `Return` becomes – executes here, at the call site,
104 // which is what inlining means. That is their position; without it they
105 // had none, and every finding a rule anchored on one reported no line
106 // (#131). The callee's own statements keep their own spans, so a finding
107 // inside the utility still names the utility.
108 let call_span = post.first().and_then(Stmt::span);
109 if !post.is_empty() {
110     post.remove(0); // the call statement itself
111 }
112 let old_term = std::mem::replace(&mut block.term, Terminator::Unreachable);
113
114 // 3. Fresh block-id allocation (offset past the caller's highest id).
115 let block_offset: BlockId = caller
116     .blocks
117     .keys()
118     .copied()
119     .max()
120     .map(|m| m + 1)
121     .unwrap_or(0);
122 let join_block_id: BlockId = block_offset + callee_block_map.len();
123
124 // 4. Param-binding prefix: `let renamed_param = arg;` (args are caller-side,
125 //   never renamed).
126 let param_lets: Vec<Stmt> = renamed_params
127     .iter()
128     .zip(args.iter())
129     .map(|(p, a)| Stmt::Let {
130         var: p.clone(),
131         rhs: a.clone(),
132         span: call_span,
133     })
134     .collect();

```

Étape 1. Toutes les variables du CFG de l'appelé sont renommées d'un coup par `rename_vars_cfg`, et les paramètres par `rename_one`.

Étape 1b : les préconditions avant toute mutation. Deux cas font sauter la greffe : un appelé sans bloc d'entrée, et un bloc d'appel absent de l'appelant. Le commentaire raconte le défaut corrigé (#13) : ces contrôles se faisaient à l'étape 6, *après* avoir coupé le bloc de l'appelant et remplacé son terminateur par `Unreachable`; sauter la greffe à ce moment laissait un appelant tronqué — instructions suivantes perdues, terminateur perdu — sans assertion ni retour arrière. Désormais une greffe sautée laisse l'appelant intact (tests `a_headless_callee_leaves_the_caller_untouched` et `a_missing_call_site_block_leaves_the_caller_untouched`, `src/ir/splice.rs`, l. 862–930).

Étape 2 : la coupe. Les instructions du bloc d'appel à partir de `stmt_idx` sont détachées dans `post` ; la première, l'appel lui-même, est retirée. Sa position est gardée dans `call_span` : c'est elle que porteront toutes les instructions *synthétisées* par la greffe (ADR-39 §2, #131). Le terminateur d'origine est mis de côté dans `old_term` ; le bloc reçoit provisoirement `Unreachable`.

Étapes 3 et 4 : blocs frais, liaisons de paramètres. Le décalage des blocs est « le plus grand `BlockId` de l'appelant, plus un » ; la jointure prend l'identifiant qui suit immédiatement les blocs de l'appelé, `block_offset + n`. Dans notre exemple, l'appelant n'a que `B0`, donc `block_offset`

= 1, les quatre blocs de `clamp` deviennent B1 à B4, et la jointure est B5.

Piège

Ce calcul suppose que les blocs de l'appelé sont numérotés sans trou, de 0 à $n - 1$. Si ce n'était pas le cas, `bid + block_offset` pourrait valoir `join_block_id`, et l'insertion de la jointure écraserait silencieusement un bloc de l'appelé. L'invariant de densité tient en pratique (section 6.2), mais il n'est ni documenté ni vérifié par `validate`.

Les liaisons de paramètres sont des `Let`, à la position de l'appel. Le zip s'arrête à la plus courte des deux listes. Un argument surnuméraire est ignoré, comme en JavaScript (hors arguments). Un paramètre sans argument, en revanche, n'est *pas* lié du tout : en JavaScript il vaudrait `undefined`; ici il reste libre et se lit comme une variable absente de l'environnement, c'est-à-dire $\top$ (chapitre 13). C'est une sur-approximation, pas un faux négatif.

8.3.4 Étape 5 : réécrire les terminateurs

Extrait 8.9 – `src/ir/splice.rs`, l. 136–186 — décalage des blocs et réécriture des retours

```

136 // 5. Offset callee blocks; rewrite each `Return(e)` into
137 // `[bound_var = e;] Jump(join)` .
138 let mut callee_blocks: Vec<(BlockId, BasicBlock)> = callee_block_map
139     .into_iter()
140     .map(|(bid, block)| (bid + block_offset, block))
141     .collect();
142 let mut return_blocks: Vec<BlockId> = Vec::new();
143 for (new_id, block) in callee_blocks.iter_mut() {
144     block.id = *new_id;
145     block.term = match std::mem::replace(&mut block.term, Terminator::Unreachable) {
146         Terminator::Jump(t) => Terminator::Jump(t + block_offset),
147         Terminator::Branch {
148             cond,
149             then_,
150             else_,
151             span,
152         } => Terminator::Branch {
153             cond,
154             then_: then_ + block_offset,
155             else_: else_ + block_offset,
156             span,
157         },
158         Terminator::Return(ret) => {
159             if let Some(var) = bound_var {
160                 // A `Let`, like the call statement it replaces: the
161                 // variable is bound here, and an `Assign` without a `Let`
162                 // is the IR's spelling of a write to an outer binding
163                 // (see `bound_vars`), which this is not.
164                 block.stmts.push Stmt::Let {
165                     var: var.clone(),
166                     rhs: ret,
167                     // `Terminator::Return` carries no span of its own, so
168                     // the call site is the only position available here.
169                     span: call_span,
170                 });
171             }
172             return_blocks.push(*new_id);
173             Terminator::Jump(join_block_id)
174         }
175     };
176     // Stays severed on purpose: `Unreachable` now means only what its
177     // name says – control does not continue (a `throw`, a stray

```

```

177      // `break`). A callee that merely falls off the end carries an
178      // explicit `Return(undefined)` from lowering and is handled by the
179      // arm above. Wiring `throw` to the join instead would invent a path
180      // that reaches the caller's exit without passing through the
181      // callee's hooks, which promoted a guard-throw custom hook
182      // (`if (!ctx) throw ...; useOptimistic(...)`) to an Error-tier
183      // `conditional-hook` – a false positive at the certain tier.
184      Terminator::Unreachable => Terminator::Unreachable,
185  };
186  }

```

Chaque bloc de l'appelé est décalé, et son terminateur réécrit selon sa forme :

- `Jump(t)` et `Branch` : les cibles sont décalées ; la condition et sa position restent.
- `Return(e)` : devient `Jump(join)`, précédé de `let bound = e` si l'appel a une variable de liaison. C'est un `Let` et non un `Assign` : le commentaire rappelle qu'un `Assign` sans `Let` est l'orthographe IR d'une écriture dans une liaison *extérieure* (section 6.2), ce qui n'est pas le cas ici — la variable est liée à cet endroit, comme dans l'instruction d'appel qu'on remplace. Sa position est `call_span`, faute de mieux : `Terminator::Return` ne porte pas de position (#140).
- `Unreachable` : *reste* `Unreachable`.

Le dernier bras mérite un encadré, parce qu'il a été tranché par une mesure.

Décision : ADR-25 — un corps qui tombe du bout rend `undefined`; `Unreachable` veut dire « le contrôle

Avant ADR-25, le lowering scellait la fin d'un corps sans `return` par `Unreachable`, comme un `throw`. La greffe laissait ce terminateur tel quel : un hook sans `return` produisait une jointure sans prédécesseur, et le composant était coupé de son propre `Return`. Sur les huit corpus, 208 composants avaient un CFG de rendu sectionné, dont 198 avec un `Return` sans état : l'environnement de sortie joignait sur moins de chemins que le programme n'en a — la direction interdite. **Décision 1** : le lowering écrit la sémantique JS, une chute en fin de corps est `Return(undefined)`, et le bras `Return` de la greffe la traite sans cas particulier (chapitre 7). **Décision 2** : `throw` garde `Unreachable`, et la greffe continue de le laisser coupé. L'alternative tentante, relier tout `Unreachable` de l'appelé à la jointure (« un chemin de plus n'est qu'une sur-approximation »), a été implémentée, mesurée et refusée : elle invente un chemin qui atteint la sortie de l'appelant *sans passer par les hooks de l'appelé*, et signalait l'idiome garde-throw comme `conditional-hook` au niveau `ERROR` sur du code conforme. Résultat mesuré : composants sectionnés 208 → 3, 12 findings révélés, aucun perdu. Nous rejouons ce cas au exemple 8.2.

8.3.5 Étapes 6 à 9 : insérer et raccorder

Extrait 8.10 – `src/ir/splice.rs`, l. 188–214 — liaisons en tête, insertion, jointure

```

188  // 6. Prepend param-binding Lets to the callee's entry block. Present by
189  //   step 1b – the caller is already half-rewritten here, so this cannot be
190  //   a bail-out point.
191  let callee_entry = callee_entry_orig + block_offset;
192  let (_, entry) = callee_blocks
193      .iter_mut()
194      .find(|(id, _)| *id == callee_entry)
195      .expect("callee entry block, checked before the caller was touched");
196  let mut new_stmts = param_lets;
197  new_stmts.extend(std::mem::take(&mut entry.stmts));
198  entry.stmts = new_stmts;

```

```

199
200 // 7. Insert callee blocks into the caller CFG.
201 for (id, block) in callee_blocks {
202     caller.blocks.insert(id, block);
203 }
204 // 8. The original block now jumps to the callee entry.
205 caller.blocks.get_mut(&block_id).unwrap().term = Terminator::Jump(callee_entry);
206 // 9. The join block carries the post-call stmts and the caller's old terminator.
207 caller.blocks.insert(
208     join_block_id,
209     BasicBlock {
210         id: join_block_id,
211         stmts: post,
212         term: old_term,
213     },
214 );

```

L'étape 6 préfixe le bloc d'entrée de l'appelé par les liaisons de paramètres ; le commentaire insiste : l'entrée est présente *par l'étape 1b*, et l'appelant étant déjà à moitié réécrit, ce point ne peut plus être un point de sortie — d'où `expect` et non un `return None`. Les étapes 7 à 9 insèrent les blocs de l'appelé, font sauter le bloc d'appel vers l'entrée de l'appelé, et créent la jointure qui porte `post` et `old_term`.

8.3.6 Étape 10 : les arêtes

La dernière étape (`src/ir/splice.rs`, l. 216–257) répare les arêtes. Le moteur lit les successeurs et prédécesseurs dans `CFG.edges`, pas dans les terminateurs : un bloc greffé sans arête serait un bloc que la *worklist* n'atteint jamais. Quatre retouches :

- les arêtes sortantes du bloc d'appel partent désormais de la jointure (elles suivent l'ancien terminateur) ;
- une arête `Unconditional` du bloc d'appel vers l'entrée de l'appelé ;
- les arêtes internes de l'appelé, décalées, *avec leur genre* : `Back` pilote le widening, `IfTrue`/`IfFalse` le raffinement par les gardes, `Await` les phases (chapitre 12) ;
- une arête `Unconditional` de chaque ancien bloc de retour vers la jointure.

En *build debug*, `validate` vérifie que tout terminateur a son arête ; c'est d'ailleurs le seul appel de `validate` du pipeline (section 6.2).

Invariant 8.2: Ce que garantit une greffe réussie

Après `splice_callee_into_cfg` rendant `Some(r)` : (i) *tous* les blocs et *toutes* les arêtes de l'appelé sont dans l'appelant, décalés dans `r` (test `every_callee_block_and_edge_is_spliced`) ; (ii) les genres d'arêtes internes sont préservés ; (iii) le contrôle entre dans l'appelé depuis le bloc d'appel et n'en sort que par un ancien `Return`, vers la jointure ; un `Unreachable` reste sans successeur ; (iv) les instructions de l'appelé gardent leurs positions (et donc leur fichier), les instructions synthétisées prennent la position de l'appel. Après `None`, l'appelant est inchangé.

Le point (i) est l'objet même de la primitive. Le module l'explique en tête (`src/ir/splice.rs`, l. 1–7) : la greffe existait en deux copies divergentes, et celle des hooks ne concaténait que le bloc d'*entrée* de l'appelé, perdant silencieusement tous les autres — un faux négatif sur tout corps de hook à plusieurs blocs. ADR-20 (Thème 1) a unifié les deux copies en une primitive partagée, avec α -renommage.

8.3.7 Déroulé complet : un utilitaire inter-fichiers

Le dépôt contient une fixture minimale de greffe, `tests/fixtures/splice_span/App.tsx`, conçue pour vérifier les positions.

Extrait 8.11 – `tests/fixtures/splice_span/App.tsx`, l. 1–6 — le composant appelant

```
1 import { normalize } from "../lib/heavy";
2
3 export function Page({ raw }: { raw: string }) {
4   const data = normalize(raw);
5   return <div>{data.name}</div>;
6 }
```

Extrait 8.12 – `tests/fixtures/splice_span/lib/heavy.ts`, l. 1–5 — l'utilitaire, dans un autre fichier

```
1 // A one-line utility: its whole body is a `Return` terminator, and the splice
2 // rewrites that into `let bound = JSON.parse(raw)` at the call site.
3 function normalize(raw: string) {
4   return JSON.parse(raw);
5 }
```

Avant la greffe (sonde, lancée depuis le répertoire de la fixture), le rendu de `Page` a un seul bloc `B0` : `let __obj_63 = __p0` (sans position), `let raw = __obj_63.raw` (3:23), `let data = normalize(raw)` (4:8), puis `return Native<div>(...)`; le corps de `normalize` est un seul bloc, `return JSON.parse(raw)`, sans arête. Après :

```
render_cfg:
  entry=B0
  B0:
    let __obj_63 = __p0 @-
    let raw = __obj_63.raw @FileId(0):3:23
    -> jump B1
  B1:
    let raw#0 = raw @FileId(0):4:8
    let data = JSON.parse(raw#0) @FileId(0):4:8
    -> jump B2
  B2:
    -> return Native<div>(Obj#0{}; [data.name]) prop_spans=[] @FileId(0):5:9
  edges: B0->B1:Unconditional B1->B2:Unconditional
  inline_origin: InlineOrigin { name: "normalize", from: "lib/heavy.ts", kind: Utility }
```

Lisons bloc par bloc.

B0 est le bloc d'appel. Ses deux premières instructions (*pre*) restent : la liaison sans position du paramètre déstructuré (section 7.8) et la lecture `raw = __obj_63.raw`. L'appel `let data = normalize(raw)` a disparu ; le terminateur est devenu `jump B1`. Il n'y avait pas d'instruction après l'appel, donc *post* est vide.

B1 est l'ancien `B0` de `normalize`, décalé de `max + 1 = 1`. En tête, la liaison de paramètre `let raw#0 = raw` : le paramètre de l'appelé est renommé `raw#0` (sel 0, première greffe du composant) ; l'argument `raw`, côté appelant, ne l'est pas. Sans renommage, les deux `raw` se confondraient dans l'environnement plat. Puis le `return JSON.parse(raw)` devenu `let data = JSON.parse(raw#0)`, un `Let` lié à `bound_var = data`. Les deux instructions synthétisées portent la position de l'appel, 4:8 dans le fichier 0 (`App.tsx`).

B2 est la jointure, `block_offset + 1 = 2`. Elle porte *post* (vide) et l'ancien terminateur `return Native<div>....`

arêtes `B0→B1` (étape 10b) et `B1→B2` (étape 10d) ; l'appelé n'avait pas d'arête interne, et `B0` pas d'arête sortante à déplacer.

La ligne `inline_origin` est la provenance de la greffe (ADR-19) : elle nourrit les pas « was inlined from » des chaînes de témoins (chapitre 17). La fixture charge un pack déclaratif avec une règle-sonde de niveau INFO, `probe/render-call`, qui signale les appels du rendu ; lancée depuis le répertoire de la fixture avec `-info`, elle voit l'appel greffé `JSON.parse` et le place à la position de l'appel :

```
$ cd tests/fixtures/splice_span && reactant check --info App.tsx lib/heavy.ts
```

```
Page (0 hooks) App.tsx
info probe/render-call (line 4:8) `parse`
```

```
✓ 2 file(s) no issues found.
```

Sans ADR-39, les deux instructions synthétisées n'auraient pas de position, et un finding ancré sur l'une d'elles ne nommerait aucune ligne (#131 : 82 findings sans position sur 7 146 dans le corpus, ramenés à 0).

8.4 Hygiène : α -renommage et capture

8.4.1 Un espace de noms plat

L'environnement abstrait du moteur est indexé par *nom* de variable, sans notion de portée (chapitre 11). C'est simple et rapide, mais cela rend la greffe dangereuse : si l'appelé a une locale *data* et l'appelant aussi, la greffe fait écrire l'une dans l'autre. Le remède classique des compilateurs est l' *α -renommage* : renommer les liaisons de l'appelé en noms frais avant de le copier. La question délicate est *lesquelles*.

Définition 8.1: Table de renommage d'une greffe

Soit un appelé de corps C , de table de hooks H et de paramètres P , greffé dans un CFG qui lie l'ensemble de noms B (les cibles de ses `Let`), avec le sel s . Notons $\text{bound}(C, P)$ l'union de P et des cibles des `Let` de C , et $\text{libres}(X)$ les variables libres d'un corps (définition 6.5). L'ensemble renommé est

$$R = \text{bound}(C, P) \cup \left\{ v \in \text{libres}(C) \cup \bigcup_{h \in H} \text{libres}(h.\text{body}) \mid v \in B \right\}$$

et le renommage est $\rho(v) = v\#s$ pour $v \in R$, l'identité sinon.

La première moitié est l'hygiène au sens strict : toute liaison de l'appelé est renommée, même sans collision. La seconde moitié traite les noms *libres* de l'appelé, et seulement ceux que l'appelant lie.

8.4.2 Le code : `bound_vars` et `callee_rename_map`

Extrait 8.13 – `src/ir/splice.rs`, l. 259–272 — les liaisons d'un corps

```
259 /// Every variable bound inside `cfg`: `params` plus every `let` target.
260 /// Assignments to variables never `let`-bound are left out – those are writes
261 /// to an outer/free binding, not locals.
262 pub fn bound_vars(cfg: &CFG, params: &[Var]) -> HashSet<Var> {
263     let mut bound: HashSet<Var> = params.iter().cloned().collect();
264     for block in cfg.blocks.values() {
265         for stmt in &block.stmts {
266             if let Stmt::Let { var, .. } = stmt {
267                 bound.insert(var.clone());
```

```

268     }
269   }
270 }
271 bound
272 }

```

`bound_vars` ne retient que les paramètres et les cibles de `Let`. Une cible d'`Assign` jamais liée par un `Let` dans le corps est une écriture dans une liaison extérieure, pas une locale.

Extrait 8.14 – `src/ir/splice.rs`, l. 302–330 — la table de renommage (sa doc, `src/ir/splice.rs`, l. 274–301, est résumée ci-dessous)

```

302 pub fn callee_rename_map(
303     cfg: &CFG,
304     hooks: &[HookEntry],
305     params: &[Var],
306     salt: u32,
307     caller_bound: &HashSet<Var>,
308 ) -> HashMap<Var, Var> {
309     let mut rename: HashSet<Var> = bound_vars(cfg, params);
310     // `hooks` matters: by this point a `useCallback` in the callee is a
311     // `HookEntry` with its own body CFG and only a marker left in `cfg`, so the
312     // names it reads – the ones most likely to be captured – are not in `cfg`
313     // at all. The same rename map is applied to those sub-bodies, so it has to
314     // be built from them too.
315     let bodies = std::iter::once(cfg).chain(hooks.iter()).filter_map(|h| h.body_cfg());
316     for body in bodies {
317         for free in crate::ir::free_vars::compute_free_vars(body) {
318             if !rename.contains(&free) && caller_bound.contains(&free) {
319                 rename.insert(free);
320             }
321         }
322     }
323     rename
324         .into_iter()
325         .map(|v| {
326             let fresh = format!("{v}#{salt}");
327             (v, fresh)
328         })
329         .collect()
330 }

```

La doc de la fonction justifie la seconde moitié par un cas du corpus (#141). Le hook `useOpenAskAiPageInSidePanel` du projet `twenty` lit un `t` qu’il importe de `@lingui/core/macro` : une liaison de module, qui ne change pas entre deux rendus. Greffé dans un composant qui écrit `const t = useLingui()`, ce `t` libre était *capturé* par la locale de l’appelant, et 208 lignes du corpus affirmaient que l’import était une valeur de hook à mettre dans des `deps`. En JavaScript, la portée est lexicale : un nom libre de l’appelé se résout dans le module de l’appelé, jamais dans une locale de son appelant.

Pourquoi ne pas renommer *tous* les noms libres ? Parce que plusieurs sont reconnus par leur nom en aval : `fetch`, `console`, `setTimeout`, un utilitaire frère que le registre résout. Les renommer en masse échangerait ce faux positif contre des faux négatifs. Quand l’appelant lie le nom, les deux liaisons sont prouvablement différentes, et celle de l’appelé doit être isolée ; qu’elle ne se résolve ensuite à rien est correct : une variable libre sans liaison se lit `⊥`, « already silent ».

Enfin `hooks` fait partie de l’entrée : au moment de la greffe, un `useCallback` de l’appelé est une entrée de table avec son propre corps, et il ne reste dans le CFG qu’un marqueur. Les noms que ces corps lisent — les plus susceptibles d’être capturés — ne sont donc pas dans `C` ; la table,

appliquée aussi à ces corps, doit être construite à partir d’eux.

Exemple 8.1: Isolement d’un import en collision

Un utilitaire lit une constante importée; le composant a une prop du même nom (/tmp/ch08/capture/) :

```
1 // k.ts
2 export const LIMIT = 3;
3 // util.ts
4 import { LIMIT } from "../k";
5 export function lim() { const v = LIMIT; return v; }
6 // App.tsx
7 export function C({ LIMIT }) {
8   const a = lim();
9   useEffect(() => { const b = lim(); }, []);
10  return <div>{a}</div>;
11 }
```

Dans le rendu, après greffe (sonde, extrait) :

```
let LIMIT = __obj_83.LIMIT @fFileId(0):3:20
let v#0 = LIMIT#0 @fFileId(2):3:8
let a = v#0 @fFileId(0):4:8
```

La locale *v* est renommée (première moitié); le *LIMIT* libre de *lim*, que le rendu lie, devient *LIMIT#0*, un nom qui ne se résout à rien : il ne lit plus la prop.

8.4.3 Renommer sans capturer : les fermetures imbriquées

Extrait 8.15 – src/ir/splice.rs, l. 431–447 — renommage d’une expression, cas de la fermeture

```
431 fn rename_vars_expr(expr: Expr, ren: &HashMap<Var, Var>) -> Expr {
432     match expr {
433         Expr::Var(v) => Expr::Var(rename_one(ren, &v)),
434         Expr::FnLit {
435             id,
436             params,
437             body_cfg,
438         } => {
439             // The closure's own params shadow same-named callee locals.
440             let inner = without(ren, &params);
441             let owned = Arc::try_unwrap(body_cfg).unwrap_or_else(|a| (*a).clone());
442             Expr::FnLit {
443                 id,
444                 params,
445                 body_cfg: Arc::new(rename_vars_cfg(owned, &inner)),
446             }
447         }
448     }
449 }
```

En entrant dans un *FnLit*, la table perd les paramètres de la fermeture : ils masquent les locales homonymes de l’appelé (test *rename_respects_fnlit_param_shadowing*). C’est la même discipline que *compute_free_vars*. *rename_vars_stmt* (src/ir/splice.rs, l. 383–411) renomme aussi les *cibles* (*Let.var*, *Assign.var*), de sorte qu’une liaison et ses lectures restent cohérentes. *rename_hook_entry* (src/ir/splice.rs, l. 540–632) applique la même table à chaque entrée de hook de l’appelé : *init*, *corps*, *deps* et *args*; pour un *Callback*, les *deps* (évaluées dans la portée du rendu) voient la table complète et le corps une table privée de ses paramètres.

8.4.4 Le sel et l’affichage : SPLICE_MARK

Extrait 8.16 – src/ir/splice.rs, l. 336–349 — le séparateur et le nom affiché

```

336 /// Character separating a spliced callee-local's source name from its unique
337 /// salt (`count#3`). A JS identifier never contains it, so it unambiguously
338 /// marks a variable alpha-renamed by [`splice_callee_into_cfg`].
339 pub const SPLICE_MARK: char = '#';
340
341 /// Recover the source-level name of a (possibly spliced-and-renamed) variable
342 /// for user-facing display: `count#3` → `count`, `count` → `count`. Never use
343 /// this for matching – only the full renamed name is unique.
344 pub fn source_name(var: &str) -> &str {
345     match var.split_once(SPLICE_MARK) {
346         Some((base, _)) => base,
347         None => var,
348     }
349 }

```

Le sel est un entier par greffe, tiré de `SpliceIds` ; deux greffes du même appelé dans un composant ont donc des noms frais distincts. Le séparateur `#` ne peut pas apparaître dans un identifiant JavaScript — sauf comme préfixe d’un champ privé `this.#x`, qui s’abaisse en `FieldAccess` et n’atteint jamais une `Var` : l’argument tient. Les messages ne doivent jamais montrer `count#0` : les chaînes de témoins et les règles passent par `source_name` (par exemple `src/rules/api/witness.rs`, l. 160–164), testé par `spliced_hook_locals_are_not_shown_with_rename_salt`. Inversement, `source_name` ne sert jamais à comparer : seul le nom complet est unique.

8.4.5 Substituer les arguments dans les initialiseurs d’état

Les liaisons `let param = arg` suffisent pour le corps greffé. Elles ne suffisent pas pour l’init d’un `useState` : le moteur évalue les initialiseurs dans un environnement séparé (constantes de module et props, chapitre 13), que les liaisons du rendu n’atteignent pas. Pour `useState(initial)`, il faut donc remplacer `initial` par l’argument concret `0` dans l’expression elle-même :

Extrait 8.17 – src/ir/splice.rs, l. 636–663 — substitution de lectures, attentive aux fermetures

```

636 /// Substitute variable *reads* with expressions, capture-aware. Unlike
637 /// `rename_vars_expr` this never touches binding sites – it is the honest,
638 /// exhaustive replacement for the old hand-rolled `subst_vars` that dropped
639 /// every composite expression through an `other => other` arm (so a param
640 /// nested in `useState({x: param})` or `useState() => param` never resolved).
641 pub fn subst_vars_expr(expr: Expr, subst: &HashMap<Var, Expr>) -> Expr {
642     if subst.is_empty() {
643         return expr;
644     }
645     match expr {
646         Expr::Var(v) => subst.get(&v).cloned().unwrap_or(Expr::Var(v)),
647         Expr::FnLit {
648             id,
649             params,
650             body_cfg,
651         } => {
652             let inner: HashMap<Var, Expr> = subst
653                 .iter()
654                 .filter(|(k, _)| !params.contains(k))
655                 .map(|(k, v)| (k.clone(), v.clone()))
656                 .collect();
657             let owned = Arc::try_unwrap(body_cfg).unwrap_or_else(|a| (*a).clone());
658             Expr::FnLit {

```

```

659         id,
660         params,
661         body_cfg: Arc::new(subst_vars_cfg(owned, &inner)),
662     }
663 }

```

Contrairement au renommage, la substitution ne touche jamais les sites de liaison. Le commentaire rappelle ce qu'elle remplace : une ancienne `subst_vars` écrite à la main, qui laissait tomber toute expression composée par un bras `other => other`, si bien qu'un paramètre niché dans `useState(x: param)` ou `useState(() => param)` n'était jamais résolu. Elle n'a qu'un appelant, `expand_custom_hooks`, pour les init des entrées `State` (section 8.5).

Piège

La substitution retire de sa table les *paramètres* d'un `FnLit` traversé, jamais ses `let` : `subst_vars_cfg` (`src/ir/splice.rs`, l. 755–821) substitue tous les membres droits et terminateurs sans regarder les cibles. Un initialiseur paresseux qui masque un paramètre du hook par une locale voit donc ses lectures remplacées par l'argument. Avec `useCounter(p)` contenant `useState(() => const p = 1; return p; )`, appelé comme `useCounter("x")` (`/tmp/ircheck/subst/`), l'entrée d'état devient (sonde) :

```

hook: State #1 init=Fn#1() {
  B0:
    let p#0 = 1 @fFileId(1):4:10
    -> return "x"
} @fFileId(1):3:8

```

L'état initial est abstrait par la valeur exacte "x" alors que le programme vaut 1 : ce n'est pas une sur-approximation mais une valeur fausse. Le cas est rare ; la correction naturelle, au niveau de la primitive, retirerait aussi de la table les noms `bound_vars`(`body`, `params`) du `FnLit` traversé, comme le fait déjà `compute_free_vars`.

8.5 Expansion des hooks custom

8.5.1 Ce que le lowering a laissé

Revenons au compteur du liste 8.1. Le lowering a produit deux unités. Dans `Counter`, l'appel `useCounter(0)` est une entrée `Custom` de la table de hooks, et sa valeur un marqueur opaque dans le rendu ; `useCounter` est un `HookIR` avec son corps et sa propre table, numérotée depuis 0 (sonde, lignes de provenance élaguées) :

```

ComponentIR Counter file=Counter.tsx param=props dom_props={} module_consts=[]
render_cfg:
  B0:
    let count = HookMarker(0, Unknown) @fFileId(0):12:8
    -> return Native<div>(Obj#0{}; [count]) prop_spans=[] @fFileId(0):13:9
hook: Custom #0 useCounter(0) deps=Absent binding=Some("count") import_source=None resolved_file=Some("Counter.tsx")
  ↳ @fFileId(0):12:8
HookIR useCounter params=["initial"] file=Counter.tsx
body_cfg:
  B0:
    let count = StateVal(0) @fFileId(0):4:9
    let setCount = StateSetter(0) @fFileId(0):4:16
    HookMarker(1, Undefined); @fFileId(0):5:2
    -> return count
hook: State #0 init=initial @fFileId(0):4:8
hook: Effect #1 deps=List[count] arity=Exact(1) spread_at=[] @fFileId(0):5:2
  B0:
    setCount((count Add 1)); @fFileId(0):6:4
    -> return undefined

```

L'expansion doit produire un composant où ces deux tables n'en font qu'une. Voici le résultat

:

```
render_cfg:
  B0:
    -> jump B1
  B1:
    let initial#0 = 0    @fFileId(0):12:8
    let count#0 = StateVal(1)  @fFileId(0):4:9
    let setCount#0 = StateSetter(1)  @fFileId(0):4:16
    HookMarker(2, Undefined);  @fFileId(0):5:2
    let count = count#0    @fFileId(0):12:8
    -> jump B2
  B2:
    -> return Native<div>(Obj#0{}; [count]) prop_spans=[] @fFileId(0):13:9
  edges: B0->B1:Unconditional B1->B2:Unconditional
  hook: State #1 init=0 @fFileId(0):4:8
  hook: Effect #2 deps=List[count#0] arity=Exact(1) spread_at=[] @fFileId(0):5:2
  B0:
    setCount#0((count#0 Add 1));  @fFileId(0):6:4
    -> return undefined
  prov: HookProvenance { label: 0, origin_hook: "useCounter", ..., inlined: false, ... }
  prov: HookProvenance { label: 1, origin_hook: "useState", ..., inlined: true, ... }
  prov: HookProvenance { label: 2, origin_hook: "useEffect", ..., inlined: true, ... }
  inline_origin: InlineOrigin { name: "useCounter", from: "Counter.tsx", kind: Hook }
```

Le tableau 8.3 récapitule ce qui est arrivé à chaque label.

Avant (unité, label)	Après	Dans le CFG	Provenance
Counter, Custom #0	disparue de la table	marqueur remplacé par la greffe	ligne 0 gardée, label pendant
useCounter, State #0	State #1, init=0	StateVal(1), StateSetter(1)	ligne 1, inlined: true
useCounter, Effect #1	Effect #2	HookMarker(2, ...)	ligne 2, inlined: true

TABLE 8.3 – Labels avant et après l'expansion de useCounter dans Counter : décalage offset = max + 1 = 1.

Tous les ingrédients des sections précédentes sont là : décalage des labels (offset = 1), greffe du corps entier au site du marqueur avec liaison du retour (let count = count#0), paramètre lié à la position de l'appel (let initial#0 = 0), la même table de renommage appliquée au corps de l'effet et à ses deps (count#0, setCount#0), et l'init de l'état substitué (init=initial devenu init=0).

8.5.2 L'algorithme

L'algorithme 4 parcourt la table par indice. Le point clé est la dernière ligne : l'entrée Custom est remplacée par les entrées de l'appelé, et *i* n'avance pas. Si la première entrée insérée est elle-même un Custom (un hook qui appelle un hook), elle est examinée au tour suivant : l'expansion est récursive sans être écrite récursivement.

8.5.3 Trouver le hook, se garder de la récursion

Extrait 8.18 – src/engine/fixpoint.rs, l. 925–942 — garde de récursion et recherche dans le registre

```
925 let custom_label = hooks[i].label();
```

Algorithme 4 : `expand_custom_hooks` (`src/engine/fixpoint.rs`, l. 889–1142), en pseudo-code

Input : la table `hooks` et le rendu d'un composant, la réserve `SpliceIds`

```

1  expanding  $\leftarrow \emptyset$ ; i  $\leftarrow 0$ ;
2  while i < |hooks| do
3      if hooks[i] n'est pas Custom then i  $\leftarrow i + 1$ ; continuer;
4      (n, args, src, f)  $\leftarrow$  hooks[i];  $\ell \leftarrow$  son label;
5      if n  $\in$  expanding then i  $\leftarrow i + 1$ ; continuer // garde de récursion;
6      h  $\leftarrow$  HookRegistry[(f, n)], sinon premier hook nommé n;
7      if h absent then
8          if SummaryRegistry connaît (n, src) then retaguer le marqueur  $\ell$ ;
9          i  $\leftarrow i + 1$ ; continuer;
10     o  $\leftarrow 1 + \max$  des labels de hooks;
11      $\sigma \leftarrow [p \mapsto a \mid (p, a) \in \text{zip}(h.params, args)]$ ;
12     (s, c)  $\leftarrow$  SpliceIds.take(alloc_span(h));
13      $\rho \leftarrow$  callee_rename_map(h, s, bound_vars(render));
14      $H' \leftarrow [\rho(\sigma_{init}(e)) \mid e \in \text{remap\_hooks}(h.hooks, \{o, c\})]$ ;
15     greffer remap_cfg(h.body, {o, c}) au marqueur  $\ell$ , lié à sa variable, avec  $\rho$ ; enregistrer
        la région;
16     ajouter les lignes de provenance de h, labels + o, inlined;
17     expanding  $\leftarrow$  expanding  $\cup \{n\}$ ;
18     remplacer hooks[i] par  $H'$  // sans avancer i
```

```

926
927     // Recursion guard: skip if we already started expanding this hook.
928     if expanding.contains(&name) {
929         i += 1;
930         continue;
931     }
932
933     // Prefer the resolved-file key; fall back to name-only when the import
934     // wasn't resolved OR the resolved file doesn't define the hook under
935     // that name (a one-level re-export: `./a` resolves but `useX` lives in
936     // `./b`). Same first-match caveat as the rest of `get_by_name`.
937     let hook_ir_opt = match &resolved_file {
938         Some(file) => reg
939             .get(&(file.clone(), name.clone()))
940             .or_else(|| reg.get_by_name(&name)),
941         None => reg.get_by_name(&name),
942     };

```

La garde est un ensemble de *noms* déjà expansés dans ce composant. Elle suffit à terminer sur un hook auto-récursif ou une chaîne $\text{useA} \rightarrow \text{useB} \rightarrow \text{useA}$; elle a un prix, que nous verrons plus bas. La recherche passe par le *HookRegistry*, un *KeyedRegistry*<*HookIR*> de clé (fichier, nom) (ADR-13, `src/engine/hook_registry.rs`, l. 6–13), construit une fois pour tout le programme. La clé complète vient du fichier résolu de l'import (*resolved_file*). À défaut — import non résolu, ou ré-export à un niveau où le fichier résolu ne définit pas le nom — le moteur retombe sur *get_by_name*, qui rend le premier hook de ce nom dans l'ordre des clés. C'est une asymétrie à connaître : la résolution des utilitaires, elle, refuse de deviner (section 8.6).

8.5.4 Préparer la copie

Extrait 8.19 – src/engine/fixpoint.rs, l. 996–1026 — décalage, substitution, sel, table de renommage

```

996      // Offset = first available label after all current entries.
997      let offset: HookLabel = hooks.iter().map(|h| h.label() + 1).max().unwrap_or(0);
998
999      // Param→arg substitution for State inits: they are seeded in a separate
1000      // env (module consts + props), which the render-body param bindings
1001      // never reach, so `useState(initial)` must inline the concrete arg.
1002      let param_subst: HashMap<String, Expr> = hook_ir
1003          .params
1004          .iter()
1005          .zip(call_args.iter())
1006          .map(|(p, a)| (p.clone(), a.clone()))
1007          .collect();
1008
1009      // One alpha-rename map for this expansion, shared by the body splice and
1010      // by every sub-hook body that captures a render-scope local – otherwise
1011      // an effect capturing `x` would desync from the body's renamed `x`.
1012      let (s, id_offset) = salt.take(alloc_span(&hook_ir.body_cfg, &hook_ir.hooks));
1013      let off = crate::ir::Offsets {
1014          labels: offset,
1015          ids: id_offset,
1016      };
1017      // The caller's own bindings, so a free name of the hook body cannot be
1018      // captured by one of them (#141).
1019      let caller_bound = crate::ir::bound_vars(render_cfg, &[]);
1020      let rename = crate::ir::callee_rename_map(
1021          &hook_ir.body_cfg,
1022          &hook_ir.hooks,
1023          &hook_ir.params,
1024          s,
1025          &caller_bound,
1026      );

```

Quatre calculs, dans cet ordre. Le décalage des labels est « le premier label libre après toutes les entrées courantes », y compris celles qu’une expansion précédente vient d’insérer. La substitution `param_subst` apparie paramètres et arguments *avant* renommage. La réserve fournit le sel et la plage de sites. La table de renommage protège les liaisons du rendu de l’appelant (`bound_vars(render_cfg, &[])` : les Let du rendu, ce qui inclut les liaisons issues du déstructurage des props, `let x = __obj_N.x`).

Extrait 8.20 – src/engine/fixpoint.rs, l. 1028–1047 — la table de l’appelé, copiée et réécrite

```

1028      let remapped: Vec<HookEntry> = remap_hooks(hook_ir.hooks.clone(), off)
1029          .into_iter()
1030          .map(|h| {
1031              let h = match h {
1032                  HookEntry::State { label, init, span } => HookEntry::State {
1033                      label,
1034                      init: crate::ir::subst_vars_expr(init, &param_subst),
1035                      span,
1036                  },
1037                  other => other,
1038              };
1039              crate::ir::rename_hook_entry(h, &rename)
1040          })
1041          .collect();

```

```

1042
1043     // Splice the hook's WHOLE body CFG at its call site (the HookMarker
1044     // binding), binding the return to the caller variable. Fixes the
1045     // multi-block-body FN (only the entry block used to survive) and the
1046     // destructuring rebind (the return is now actually bound).
1047     let body = remap_cfg(hook_ir.body_cfg.clone(), off);

```

L'ordre des trois réécritures d'une entrée est significatif : `remap_hooks` décale labels et sites ; `subst_vars_expr` remplace, dans l'init d'un State seulement, les noms de paramètres *d'origine* par les arguments ; `rename_hook_entry` renomme ce qui reste. Faire le renommage d'abord empêcherait la substitution de reconnaître les paramètres. Puis le corps est décalé par `remap_cfg` et greffé au site du marqueur, trouvé par `find_hook_marker` (`src/engine/fixpoint.rs`, l. 1144–1163) : le premier `let x = HookMarker(1, ...)` ou `HookMarker(1, ...)` ; de label ℓ , en ordre de blocs ; la variable `x` devient `bound_var`.

8.5.5 Enregistrer la région et la provenance

Une greffe réussie rend sa plage de blocs, que le moteur range comme région inlinée :

Extrait 8.21 – `src/engine/setters.rs`, l. 806–820 — une région greffée

```

806 /// One spliced-callee block range in a CFG: `start..end` are the callee's
807 /// blocks (the join past them belongs to the caller), `name` the callee's
808 /// EXPORTED name (aliased imports resolve – ADR-027 §3), `parent` the region
809 /// the call site itself sat in (nesting = the wrapper chain).
810 #[derive(Debug, Clone)]
811 pub struct InlineRegion {
812     pub start: BlockId,
813     pub end: BlockId,
814     pub name: Symbol,
815     /// File the spliced body came from – the import-resolution context for
816     /// calls the splice carried in (a wrapper's own helper call resolves
817     /// through the WRAPPER's file, not the component's).
818     pub from: std::path::PathBuf,
819     pub parent: Option<usize>,
820 }

```

`parent` est la région qui contenait le bloc d'appel : les régions s'emboîtent comme la chaîne des wrappers. C'est ce que lit la provenance des écrivains (ADR-27 §4, chapitre 14) : une écriture de setter située dans une région est *médiée par un wrapper*, et ne peut pas être certifiée comme écrite directement par le composant. D'où l'importance du cas défensif (`src/engine/fixpoint.rs`, l. 1079–1109) : si le marqueur est introuvable (« should not happen »), le moteur greffe les seules instructions du bloc d'entrée en tête du rendu et positionne `render_poisoned = true`, car ces instructions échappent à toute région ; aucune écriture du rendu ne pourra alors se dire directe. C'est un repli *fail-closed*.

Restent la provenance et le remplacement de l'entrée (`src/engine/fixpoint.rs`, l. 1111–1141). L'`InlineOrigin` (ADR-19) dit qu'un corps venu de `hook_ir.file` vit désormais dans ce CFG ; les lignes `HookProvenance` de l'appelé rejoignent celles du composant avec le même décalage de labels et `inlined: true` (ADR-23), si bien qu'un `useLayoutEffect` atteint à travers un wrapper reste distinguable d'un `useLayoutEffect` écrit dans le composant. La ligne de l'appel au wrapper est gardée : son label est *pendant* (aucune entrée de la table ne le porte plus), il faut donc passer par sa position.

8.5.6 Le garde-throw, rejoué

Exemple 8.2: Un hook qui lève une exception avant ses hooks

Le test `a_hook_after_a_guard_throw_is_not_conditional` (`tests/cfg_exit_integrity.rs`) fixe l’idiome suivant (`/tmp/ch08/throw/Cart.tsx`) :

```
1 function useCart() {
2   const ctx = useContext(CartContext);
3   if (ctx === undefined) {
4     throw new Error("useCart must be used within a CartProvider");
5   }
6   const [items, setItems] = useState(ctx.items);
7   return items;
8 }
9 export function CartModal() {
10  const items = useCart();
11  return <div>{items}</div>;
12 }
```

Après expansion (sonde, extrait) :

```
B1:
  let ctx#0 = HookMarker(1, Unknown) @fFileId(0):6:8
  -> branch (ctx#0 Eq undefined) ? B2 : B3 @fFileId(0):7:6
B2:
  new#1 Error("useCart must be used within a CartProvider"); @fFileId(0):8:4
  -> unreachable
B3:
  -> jump B4
B4:
  let items#0 = StateVal(2) @fFileId(0):10:9
  let setItems#0 = StateSetter(2) @fFileId(0):10:16
  let items = items#0 @fFileId(0):15:8
  -> jump B5
B5:
  -> return Native<div>(Obj#0{}; [items]) prop_spans=[] @fFileId(0):16:9
edges: B0->B1:Unconditional B1->B2:IfTrue B1->B3:IfFalse B3->B4:Unconditional B4->B5:Unconditional
hook: Custom #1 useContext(CartContext) deps=Absent binding=Some("ctx") ...
hook: State #2 init=ctx#0.items @fFileId(0):10:8
```

B2 reste unreachable, sans arête vers la jointure B5 : la seule sortie réelle passe par B4, où vit le `useState` (label 2), qui domine donc toutes les sorties (définition 3.9). Aucun `conditional-hook` n’est émis ; seule une INFO `analysis-limit` signale que `useContext`, entrée Custom de l’appelé devenue entrée du composant, n’est pas modélisé (#28). Si la greffe reliait B2 à B5, un chemin atteindrait la sortie sans passer par B4, et le hook deviendrait conditionnel au niveau ERROR. Deux détails de lecture : l’init de l’état est `ctx#0.items`, renommé mais évalué dans l’environnement d’entrée, où `ctx#0` n’existe pas, donc `⊥` ; et le champ `binding=Some("ctx")` de l’entrée Custom n’est *pas* renommé par `rename_hook_entry` — il n’est lu nulle part, il ne faut pas s’y fier après une greffe.

8.5.7 Le prix de la garde : un hook appelé deux fois

La garde de récursion est par nom et par composant. La doc de la fonction l’assume : « sound for the rare case of a hook called twice in the same component (second call stays opaque FN, not FP) » (`src/engine/fixpoint.rs`, l. 884–888). Vérifions :

Extrait 8.22 – Deux appels du même hook (`/tmp/ch08/step/ Twice.tsx`, `useStep` du liste 8.2)

```
1 export function Both() {
2   const a = useStep(0);
```



```

3   const b = useStep(1);
4   return <div>{a}{b}</div>;
5 }

```

Avec `-info` (l'INFO est sinon tue) :

```

Both (2 hooks) Twice.tsx
info  analysis-limit [hook:1] (line 13:8) hook `useStep` was not found in the registry. Pass its source file or
    ↪ add a HookSummary to analyse it (FN possible)
warn  redundant-set-state [hook:2] (line 5:2) state `n` is set to a stable value it already holds, so the update
    ↪ is redundant
      (1 trace step(s), rerun with --trace)
suspended analysis-limit 9 passing check(s) withheld: the analysis was truncated in this component, so they are not
    ↪ guaranteed

```

Seul le premier appel (`useStep(0)`) est greffé ; l'infinite-loop du second est manqué. Ce n'est pas un silence : l'INFO est émise et les assurances du composant sont suspendues. Mais le texte de l'INFO est trompeur — le hook *est* dans le registre — parce qu'il est produit par une seule règle pour toute entrée Custom restée opaque (`src/rules/impls/analysis_limit_info.rs`, l. 99–128). La limite ne figure pas dans `docs/limitations.md`, qui ne mentionne que son analogue pour les utilitaires (#53). Une garde par *pile d'expansion* (un nom interdit seulement dans sa propre greffe) terminerait aussi bien sur la récursion, en greffant les deux appels : c'est la piste centrale.

8.6 Inlining des utilitaires

8.6.1 Le registre des utilitaires

Un *utilitaire* est une fonction de module qui n'est ni un composant ni un hook (définition 5.1). Le lowering en fait un `FunctionIR` : fichier, nom, paramètres, corps — pas de table de hooks, puisque la règle des hooks interdit à une fonction ordinaire d'en appeler (`src/ir/function_ir.rs`, l. 1–22). Le registre les indexe par (fichier, nom) et garde en plus les arêtes d'import résolues :

Extrait 8.23 – `src/engine/function_registry.rs`, l. 51–63 — résolution fail-closed d'un utilitaire

```

51  /// Resolve a bare callee name at a call site, fail-closed (ADR-027 §3):
52  /// a definition in the caller's own file, else the caller's resolved
53  /// import edge for that local name – never a by-name guess across files
54  /// (the first-match fallback silently spliced the wrong body on a name
55  /// collision, and an aliased import did not resolve at all).
56  pub fn resolve(&self, caller_file: &Path, name: &str) -> Option<&FunctionIR> {
57      let key = (caller_file.to_path_buf(), name.to_string());
58      self.functions.get(&key).or_else(|| {
59          self.aliases
60              .get(&key)
61              .and_then(|target| self.functions.get(target))
62      })
63  }

```

ADR-27 §3 fixe la règle : une définition du fichier de l'appel, sinon l'arête d'import résolue de ce fichier pour ce nom local, *jamaïs* une devinette par nom à travers les fichiers. L'ancien repli « premier trouvé » greffait silencieusement le mauvais corps en cas d'homonymie, et un import renommé (`import putState as ps`) ne se résolvait pas du tout.

8.6.2 Trouver les appels, épuiser le budget

`expand_utility_calls` (`src/engine/fixpoint.rs`, l. 1554–1602) applique `inline_in_cfg` au CFG de rendu *et* au corps de chaque Effect, Memo, Callback et Handler, chacun avec son propre ensemble de noms en cours et son propre budget. Ni les init d'état, ni les args des hooks custom, ni les corps de fermetures imbriquées dans une expression ne sont parcourus.

La boucle de `inline_in_cfg` (`src/engine/fixpoint.rs`, l. 1604–1665) cherche une cible, la greffe, recommence. Chaque nom n'est inliné qu'une fois par CFG : c'est la garde de récursion (une chaîne $A \rightarrow B \rightarrow A$ termine après un aller-retour), et c'est aussi une limite (#53) — un deuxième appel du même utilitaire dans le même CFG reste un Call opaque. Le budget, `max_inline_depth = 8` greffes par CFG (`src/engine/fixpoint.rs`, l. 57), borne la croissance ; il est décrémenté même quand `splice_one_call` échoue. L'épuiser laisse les appels restants opaques — c'est sound, puisqu'un appel opaque rend $\top$ — mais c'est une troncature, mesurée (20 CFG dans le corpus excalidraw, 6 dans memos) : `ctx.truncated` devient une INFO `analysis-limit`, et le composant retient ses assurances `verified` : plutôt que de les publier sur du code jamais lu.

Extrait 8.24 – `src/engine/fixpoint.rs`, l. 1698–1718 — ce qu'est un appel greffable

```
1698 /// If `stmt` is `let _ = util(...)` or `util(...)` where `util` is a `Var`,
1699 /// return the callee name.
1700 fn utility_call_target(stmt: &Stmt) -> Option<String> {
1701     let call_expr = match stmt {
1702         Stmt::Let { rhs, .. } => rhs,
1703         Stmt::ExprStmt(expr, _) => expr,
1704         Stmt::Assign { .. } | Stmt::MemberWrite { .. } => return None,
1705     };
1706     let (fn_, _) = match call_expr {
1707         Expr::Call { fn_, args } => (fn_, args),
1708         Expr::TSAnnotated(inner) => match inner.as_ref() {
1709             Expr::Call { fn_, args } => (fn_, args),
1710             _ => return None,
1711         },
1712         _ => return None,
1713     };
1714     match fn_.as_ref() {
1715         Expr::Var(name) => Some(name.clone()),
1716         _ => None,
1717     }
1718 }
```

Seules deux formes sont des cibles : `let x = f(...)` et `f(...)`, avec `f` une Var. Un appel en position d'expression (`g(f(x))`), un bras de ternaire, une prop JSX n'est pas inliné (#52) ; un appel de méthode `o.f()` non plus. Le fichier de résolution est celui de la *région* qui contient l'instruction (`find_inlining_target`, `src/engine/fixpoint.rs`, l. 1667–1696) : un utilitaire greffé résout ses propres appels dans son propre fichier, ce qui fait marcher les chaînes `putState` → `helper` où le composant n'importe pas `helper`.

Extrait 8.25 – `src/engine/fixpoint.rs`, l. 1763–1790 — la greffe d'un appel d'utilitaire

```
1763 let (s, id_offset) = salt.take(alloc_span(&utility.body_cfg, &[]));
1764 // The caller's own bindings, so a free name of the utility cannot be
1765 // captured by one of them (#141).
1766 let caller_bound = crate::ir::bound_vars(cfg, &[]);
1767 let rename =
1768     crate::ir::callee_rename_map(&utility.body_cfg, &[], &utility.params, s, &caller_bound);
1769 let body = crate::ir::remap_cfg(
1770     utility.body_cfg,
1771     crate::ir::Offsets {
```

```

1772         labels: 0,
1773         ids: id_offset,
1774     },
1775 );
1776 let range = crate::ir::splice_callee_into_cfg(
1777     cfg,
1778     block_id,
1779     stmt_idx,
1780     crate::ir::Splice {
1781         callee: body,
1782         params: &utility.params,
1783         args: &call_args,
1784         bound_var: bound_var.as_ref(),
1785         rename: &rename,
1786     },
1787 )?;
1788 // The EXPORTED name (`utility.name`), not the local callee: an aliased
1789 // import must chain under the identity a team's rule names (ADR-027 §3).
1790 Some((range, utility.name, utility.file))

```

`splice_one_call` est un mince habillage de la primitive : un utilitaire n'a pas de hooks, donc Offsets `labels: 0`, `ids` et une table de hooks vide ; la table de renommage protège les liaisons *du CFG où l'on greffe*. Il rend le nom *exporté* (`utility.name`) et non le nom local de l'appel, pour que la chaîne des wrappers porte l'identité qu'une règle d'équipe nomme (ADR-27 §3).

Piège

Les utilitaires sont greffés *avant* les hooks (liste 8.3). Un utilitaire appelé *dans le corps d'un hook custom* n'est donc pas inliné : quand le corps du hook arrive dans le composant, la passe des utilitaires est terminée. Avec fonction `useThing()` `const [n, setN] = useState(0); bump(setN); return n;`, le rendu de l'appelant contient après expansion `bump(setN#0);`, un Call opaque (`/tmp/ch08/utilhook/U.tsx`). ADR-27 §5 reconnaît cet angle mort (« utilities inside custom-hook bodies ») ; il retire des lignes à la relation des écrivains, sans rendre incertaine une ligne présente.

8.6.3 Ce qui n'est pas une greffe : fermetures locales (B5/B6)

Une fermeture *locale* n'est pas dans le `FunctionRegistry` : elle n'est pas une fonction de module, c'est une valeur, un `FnLit` alloué dans le tas abstrait à son site (ADR-10). Son appel est traité à l'interprétation, sans toucher au CFG. Comparons les deux chemins sur un même défaut (`/tmp/ch08/local/Local.tsx`) :

```

1 export function Local() {
2   const [n, setN] = useState(0);
3   useEffect(() => {
4     const go = () => setN(n + 1);
5     go();
6   }, [n]);
7   return <div>{n}</div>;
8 }

```

Le même fichier définit `Module`, identique sauf que l'effet appelle `bump(setM, m)`, avec `bump` fonction de module qui fait `set(v + 1);`. Les deux composants reçoivent le même *infinite-loop* (« this effect keeps pushing state 'n' ... », puis « ... state 'm' ... »). Dans `Module`, `bump` est greffé dans le corps de l'effet. Dans `Local`, `go` est une `Var` liée à un site de tas ; l'appel `go()` a un *callee* de classe `Unknown`, et l'interpréteur l'exécute depuis le tas :

Extrait 8.26 – `src/domains/interp/interpreter.rs`, l. 521–527 — B6 : appel direct d’une fermeture locale

```

521      // B6: direct local call inlining Unknown callee that resolves to a heap Fn.
522      // External/imported functions have no Loc → skipped (no FP).
523      if class == TriggerClass::Unknown
524      && let Expr::Var(name) = fn_.as_ref()
525      {
526          exec_var_callback(transfer, name, env, ctx, depth);
527      }

```

`exec_var_callback` (`src/domains/interp/interpreter.rs`, l. 542–583) exécute *tous* les corps de fonction que la variable peut désigner (jointure sur les sites), dans l’environnement vivant du site d’appel complété par les captures de la fermeture. C’est aussi le mécanisme B5, pour une fermeture passée par variable à un appelant « dans le cycle » (`setTimeout(cb)`, `.then(cb)`). Trois différences avec une greffe sont à retenir : les *paramètres* de la fermeture sont liés à $\top$, pas aux arguments ; la profondeur est bornée par `MAX_INLINE_DEPTH = 3` (`src/domains/interp/interpreter.rs`, l. 21), avec une INFO en mode programme ; un callee sans site de tas (un import non greffé, un paramètre) n’est pas exécuté du tout, faux négatif accepté par ADR-10 (#19).

8.6.4 Ce qui n’est pas une greffe : les composants enfants

Un `<Child/>` n’est *jamaïs* greffé dans le CFG de son parent. ADR-12 a choisi une analyse descendante : le parent évalue les props de l’enfant en valeurs abstraites, et l’enfant est analysé à part avec ces props (`eval_comp_app`, chapitre 16). La différence est de fond : un hook s’exécute *dans* la fibre de son appelant, un enfant a sa propre fibre, son propre état, son propre cycle de rendu. Greffer un enfant confondrait ses slots avec ceux du parent. Nous verrons cependant au section 8.9 que la frontière parent–enfant partage un espace d’identités, les sites du tas, et qu’elle n’y applique aucun décalage.

8.7 Les hooks de bibliothèque : des résumés

8.7.1 Sans source, pas de greffe

Un hook importé d’un paquet npm n’a pas de source dans le run : `node_modules` n’est jamais abaissé (#51, fermée wontfix). Quand `expand_custom_hooks` ne trouve pas un Custom dans le HookRegistry, il consulte un second registre, de *résumés*. Un résumé implémente le trait `HookSummary` (`src/registry/summary.rs`, l. 10–48) et répond à quatre questions, par ordre de priorité : le résultat est-il une fonction qui navigue (`navigates`, `useNavigate`) ? est-il tenu immobile par la boucle de rendu automatique (`held_across_updates`, les hooks de routeur, #161) ? a-t-il des membres nommés porteurs d’un contrat (`members`, `useForm().setValue`) ? sinon, quelle est sa valeur abstraite globale (`summarize`, $\top$ par défaut) ? La recherche (`SummaryRegistry::get`, `src/registry/summary.rs`, l. 166–177) essaie la clé (`Some(paquet)`, `nom`), avec le paquet de l’import, puis (`None`, `nom`) ; le registre commun n’enregistre que des clés portées par un paquet, de sorte qu’un `useRouter` local ne prend pas le contrat de celui de `Next.js`.

8.7.2 Connu n’est pas modélisé

Extrait 8.27 – `src/registry/summary.rs`, l. 69–77 — le registre par défaut

```

69      /// Pre-populate with common TanStack Query, React Router and Next.js
70      /// hooks, scoped to their respective NPM packages so they only match

```

```

71    /// hooks that were actually imported from those packages.
72    ///
73    /// Registering a hook as  $\top$  is not modelling it — it is recording that the
74    /// hook is known, which is what separates a deliberate imprecision from
75    /// the `analysis-limit/unknown-hook` Info that means "we could not even
76    /// find this definition".
77    pub fn new_with_common() -> Self {

```

La phrase est à retenir : « Registering a hook as $\top$ is not modelling it — it is recording that the hook is *known* ». Beaucoup d'entrées du registre commun (TANSTACK_HOOKS, REACT_ROUTER_HOOKS, USE_DEBOUNCE_HOOKS..., src/registry/summary.rs, l. 336–399) rendent $\top$: elles n'apportent aucune précision, mais elles distinguent une imprécision *délibérée* de l'INFO `analysis-limit` qui signifie « nous n'avons même pas trouvé la définition ». Un $\top$ connu ne suspend pas les assurances ; un $\top$ inconnu les suspend. Et le choix de ne pas écrire de contrat est lui-même argumenté : pour `useDebounceCallback`, documenté comme renvoyant un callback mémorisé, le corpus n'offre qu'un site de vérification, « and a claim is the direction that loses findings » (src/registry/summary.rs, l. 142–148).

8.7.3 Ce que fait le moteur d'un résumé

Extrait 8.28 – src/engine/fixpoint.rs, l. 949–977 — le résumé d'un hook introuvable

```

949     if let Some(summary) = inter
950         .config
951         .summary_registry
952         .get(&name, import_source.as_deref())
953     {
954         // A per-member contract needs an allocation site of its own —
955         // one per call, so two `useForm()`'s are two objects. Drawn
956         // from the component's splice cursor, the same supply a graft
957         // uses (#134).
958         let sv = if summary.navigates() {
959             // Identity unpromised: react-router's `navigate` changes
960             // with the location outside a data router.
961             SummaryValue::Navigator { stable: false }
962         } else if summary.held_across_updates() {
963             SummaryValue::Held
964         } else if summary.members().is_empty() {
965             state_value_to_summary_value(summary.summarize(&[]))
966         } else {
967             SummaryValue::Shape {
968                 id: salt.alloc_one(),
969                 members: std::sync::Arc::new(
970                     summary
971                         .members()
972                         .iter()
973                         .map(|(k, v)| ((*k).to_string(), v.clone()))
974                         .collect(),
975                 ),
976             }
977         };

```

Le résumé devient un `SummaryValue` (chapitre 6) ; un contrat par membre reçoit un site d'allocation neuf, tiré de la *même* réserve que les greffes (`alloc_one`), une par appel, pour que deux `useForm()` soient deux objets. Puis `retag_marker` (src/engine/fixpoint.rs, l. 1247–1275) réécrit le marqueur du site d'appel en `MarkerVal::Summary(sv)`, et l'entrée `Custom` est *gardée*. Le commentaire du code explique pourquoi : autrefois la liaison devenait un `SummaryVal` nu et

l'entrée était retirée, ce qui effaçait le label du CFG et de la table — un `useAtom()` *conditionnel* était invisible à toutes les vérifications de la règle des hooks. Le marqueur est l'ancre de ces vérifications; le retagner par label atteint aussi un appel sans liaison et un appel dans un bloc autre que l'entrée.

Exemple 8.3: Trois hooks de bibliothèque

Extrait 8.29 – Hook résumé par membres, hook connu, hook inconnu (/tmp/ch08/summary/Form.tsx)

```
1 import { useForm } from "react-hook-form";
2 import { useQuery } from "@tanstack/react-query";
3 import { useWeird } from "weird-lib";
4
5 export function Form() {
6   const { setValue } = useForm();
7   const query = useQuery({ queryKey: ["k"] });
8   const weird = useWeird();
9   useEffect(() => {
10     setValue("name", query.data);
11   }, [query.data]);
12   return <div>{weird}</div>;
13 }
```

Après expansion, avec le registre commun (sonde, liste de membres élaguée) :

```
let __obj_200 = HookMarker(0, Summary(Shape { id: ExprId(5), members: [("register", StableRef), ..., ("
  ↳ setValue", StableRef), ..., ("handleSubmit", Wrapper { stable: true }), ("watch", StableRef), ("control",
  ↳ StableRef)] })) @fFileId(0):7:8
let setValue = __obj_200.setValue @fFileId(0):7:10
let query = HookMarker(1, Summary(Top)) @fFileId(0):8:8
let weird = HookMarker(2, Unknown) @fFileId(0):9:8
```

Form (4 hooks) Form.tsx

```
info analysis-limit [hook:2] (line 9:8) hook `useWeird` was not found in the registry. Pass its source file
  ↳ or add a HookSummary to analyse it (FN possible)
suspended analysis-limit 4 passing check(s) withheld: the analysis was truncated in this component, so they
  ↳ are not guaranteed
```

`useForm` a une forme : `setValue` est un membre stable, et l'effet qui l'appelle sans le déclarer n'est pas un oubli de deps. `useQuery` est connu et vaut `T` : aucune INFO. `useWeird` est inconnu : son marqueur reste `Unknown`, l'INFO est émise et les assurances sont suspendues. Retirons `useWeird` (/tmp/ch08/summary2/Form.tsx) : `-info -show-clean` publie alors `verified missing-deps every effect declares the variables it reads`, parmi quatre assurances — le `T` connu de `useQuery` ne suspend rien.

Piège

`Config::default()` a un `SummaryRegistry` *vide* (`src/engine/fixpoint.rs`, l. 55); c'est le driver qui installe `new_with_common()` (`src/driver/mod.rs`, l. 368–373), « so unit tests keep their baselines ». Un test ou un outil qui appelle le moteur avec la configuration par défaut ne voit donc aucun résumé : tous les hooks de bibliothèque y sont inconnus. La sonde du dossier 03 est dans ce cas; la sortie ci-dessus a été obtenue avec une variante qui installe le registre commun. Autre écart : le commentaire de tête de la branche (`src/engine/fixpoint.rs`, l. 944–948) dit encore que les hooks connus « are removed from the hooks vec », alors que le code, une trentaine de lignes plus bas, les garde délibérément.

8.8 Attribuer les findings des hooks inlinés

8.8.1 Un défaut, trois consommateurs

Une greffe copie les instructions de l'appelé avec leurs positions, donc avec leur `FileId`. Un finding né dans un hook inliné est ancré dans le fichier du hook, mais rapporté sous le composant. La fixture `tests/fixtures/shared_hook_repeat` pose le cas type :

Extrait 8.30 – `tests/fixtures/shared_hook_repeat/hooks/useShared.ts`, l. 6–12 — un hook partagé qui boucle

```
6 function useShared(initial) {
7   const [value, setValue] = useState(initial);
8   useEffect(() => {
9     setValue(value + 1);
10  }, [value]);
11  return value;
12 }
```

`App.tsx` y définit trois composants, Alpha, Beta et Gamma, qui appellent respectivement `useShared(0)`, `useShared(1)` et `useShared(2)`.

```
$ reactant check tests/fixtures/shared_hook_repeat --trace
```

```
Alpha (1 hooks) tests/fixtures/shared_hook_repeat/App.tsx
warn  infinite-loop [hook:1] (tests/fixtures/shared_hook_repeat/hooks/useShared.ts:8:2) this effect keeps pushing
  ↳ state `value` (its deps do not provably gate it, so the effect can re-run every render) to new values on every
  ↳ run. Potential infinite render loop [in 3 components]
    in: Alpha, Beta, Gamma
      → state `value` is written here [hook:2] (tests/fixtures/shared_hook_repeat/hooks/useShared.ts:8:2)
      → the abstract value of state `value` kept growing and was widened at iteration 3
    2 component(s) hidden. Every finding in them is a source line already reported above

! 1 warning(s) across 2 file(s), 3 component attribution(s).
```

La ligne principale nomme le fichier du hook, parce que l'ancre n'est pas dans le fichier du composant. Le rendu humain regroupe les trois findings identiques par localisation (`#129`) ; le JSON, lui, garde une ligne par consommateur, chacune avec `"component"` (`"Alpha"`, `"Beta"`, `"Gamma"`), `"file": "tests/fixtures/shared_hook_repeat/hooks/useShared.ts"`, `"component_file": "tests/fixtures/shared_hook_repeat/App.tsx"` et `"line": 8`.

Le rendu humain passe par une seule fonction, `position` (`src/driver/human.rs`, l. 15–35) : si le fichier de la position (résolu par la `FileTable` à partir de son `FileId`) diffère du fichier du composant, elle écrit (`chemin:L:C`), sinon (`line L:C`). La ligne principale et les pas de `-trace` l'empruntent tous deux, « so the two can never disagree about where a position points ».

8.8.2 La décision

Décision : ADR-24 — rendre l'origine, ne jamais fusionner les consommateurs

Contexte. Avant ADR-24, la ligne principale imprimait `line:col` de l'ancre sous l'en-tête du composant : un finding ancré dans un hook inliné montrait un numéro de ligne d'un *autre* fichier. Mesure sur les huit corpus avec le pack `guardrails` : 12 findings de règles custom sur 27 (44 %) étaient inexploitable ainsi (`presence-fade.tsx:17` pour `use-callback-ref.ts:17`), et le JSON était incohérent (`file` du composant, `line/col` de l'ancre). **Décision 1** : quand le fichier de l'ancre diffère de celui du composant, la ligne principale rend l'origine, et le `file` du JSON devient celui de l'ancre — un changement du driver seul. **Décision 2** : les findings

ne sont *jamaïs* dédoublés entre consommateurs, parce qu'ils sont incomparables, pas redondants : `useStep(1)` donne `infinite-loop` et `useStep(0)` donne `redundant-set-state` (liste 8.2). Supprimer les findings par consommateur au profit d'un finding de hook est un générateur de faux négatifs, et analyser le hook une fois avec des paramètres `T` ne subsume ni n'est subsumé par les résultats par consommateur. **Décision 3** : attribution correcte n'est pas actionnabilité — un finding bien placé dans une dépendance tierce reste dans du code qu'on ne peut pas changer ; la frontière first-party/dépendance n'est pas tranchée ici. **Alternative différée** à l'époque : regrouper pour l'*affichage*. Elle a été adoptée depuis dans le rendu humain (#129, « [in 3 components] »), le JSON gardant une ligne par composant ; les deux dangers que l'ADR signalait (un groupe n'a qu'une chaîne de témoins, et `[hook:N]` diverge entre membres) sont ceux qu'un tel regroupement doit respecter.

L'argument de soundness est explicite dans l'ADR : le rendu ne peut pas cacher un défaut, il ne crée ni ne retire aucun finding, et comptes, `-fail-on` et code de sortie sont calculés avant lui. C'est le refus de la décision 2 qui porte le contenu de soundness.

8.8.3 Un finding sans position

La greffe donne une position à tout ce qu'elle synthétise ; reste le cas d'un finding dont l'ancre n'en a pas (typiquement un `Return`, #140). ADR-39 §4 le règle une fois, dans le registre des règles, plutôt que dans chaque règle :

Extrait 8.31 – `src/rules/registry.rs`, l. 356–369 — un finding prend la première position de sa chaîne

```

356 /// A finding with no position of its own takes the first position its witness
357 /// chain names.
358 ///
359 /// The chain is the finding's evidence, and its first located step is where
360 /// that evidence starts – a rule that threaded a span into a `Step` but never
361 /// onto the diagnostic was withholding a position it already had. Applied here
362 /// rather than in each rule, because "a finding renders somewhere" is a
363 /// property of every finding (#131).
364 fn located(mut d: Diagnostic) -> Diagnostic {
365     if d.range.is_none() {
366         d.range = d.notes.iter().find_map(|n| n.range);
367     }
368     d
369 }
```

8.9 Cas limites et défauts observés

La greffe est l'endroit où l'analyse copie du code d'un contexte dans un autre ; c'est donc là que se logent les confusions d'identités. Le tableau 8.4 rassemble les cas relevés au e67b10a, dont plusieurs ne sont pas répertoriés dans le tracker. Nous détaillons ensuite les trois qui ont une conséquence observable.

8.9.1 Un return jeté

Le bras `Return` de l'étape 5 ne pousse `let bound = ret` que si `bound_var` est `Some` ; sinon l'expression `ret` disparaît, avec ses lectures et ses appels. La doc de Splice dit « `None` to discard as in a bare `f(..)` ; » : mais JavaScript *évalue* l'expression du `return` avant d'en jeter la valeur.

Cas	Mécanisme	Effet observé
bump(setN, n); avec return set(n + 1)	Return jeté sans bound_var	FN infinite-loop, « ✓ no issues found »
parent et enfant de fichiers différents	sites du tas non décalés à l'entrée de l'enfant	FN ERROR cross-setter-in-render
objet en argument d'un hook custom	alloc_span ignore args, init, deps	deux Obj#1 dans un rendu
hook appelé deux fois	garde par nom	2 ^e appel opaque, INFO trompeuse
utilitaire dans un hook custom	ordre des passes	Call opaque (ADR-27 §5)
local masquant un paramètre dans un init paresseux	substitution ignore les let d'un FnLit	valeur fausse
utilitaire qui <i>assigne</i> une variable de son module	Assign ni lié ni libre, non renommé	écrase la locale homonyme
utilitaire greffé dans un effet, import homonyme d'une prop	caller_bound limité au CFG greffé	capture de la prop

TABLE 8.4 – Défauts et limites de la greffe au e67b10a (reproductions sous /tmp/ch08/ et /tmp/ircheck/).

Extrait 8.32 – Un setter appelé dans un return (/tmp/ircheck/ret/Ret.tsx)

```

1 function bump(set, n) {
2   return set(n + 1);
3 }
4 export function C() {
5   const [n, setN] = useState(0);
6   useEffect(() => {
7     bump(setN, n);
8   });
9   return <div />;
10 }

```

Corps de l'effet après greffe (sonde) : l'appel du setter a disparu.

```

hook: Effect #1 deps=Absent @fFileId(0):7:2
B1:
  let set#0 = setN    @fFileId(0):8:4
  let n#0 = n        @fFileId(0):8:4
  -> jump B2
B2:
  -> return undefined

```

reactant check Ret.tsx répond « ✓ 1 file(s) no issues found. » ; la même fonction écrite set(n + 1); (Stmt.tsx) donne « warn infinite-loop [hook:0] (line 7:2) this effect keeps pushing state 'n' to new values on every run ». C'est un faux négatif, accompagné du « ✓ » réservé aux runs complets. Le même bras sert aux hooks appelés sans liaison : pour un HookMarker(1, ...); nu, find_hook_marker rend bound_var = None (elle-même rend Some), et le Return(e) du corps subit le même sort — joué sur useThing() { ...; return setN(n + 1); } appelé useThing(); sans liaison (/tmp/ircheck/ret/HookNoBind.tsx) : même « ✓ », un second FN de même cause. La correction est locale à la primitive et générale : quand bound_var vaut None, pousser Stmt::ExprStmt(ret, call_span), comme le lowering le fait déjà pour void e.

8.9.2 Des sites en collision

Dans un composant. Avec un hook `useBox` qui alloue (`const o = v ; return o;`) suivi de `useFoo( a: 1 )` (`/tmp/ircheck/alloc/App2.tsx`), le rendu après expansion contient :

```
let o#0 = Obj#1{v: v#0}   @fFileId(1):4:8
...
let opts#1 = Obj#1{a: 1}  @fFileId(0):7:8
```

Deux littéraux distincts portent le site `Obj#1`. Le curseur partait de `alloc_span = 1` (seul l'`Obj#0` du `<div/>` est visible), la greffe de `useBox` a reçu `[1,2)`, et l'objet `a: 1`, rangé dans les args de l'entrée `Custom`, avait déjà l'identifiant 1 : même classe de défaut que `#134`, le tas étant indexé par site et `Heap::insert` écrasant. Aucune conséquence sur un diagnostic n'a été exhibée. Correction centrale : que `alloc_span` mesure toutes les expressions d'une entrée, symétriquement à `remap_hook_entry`.

Entre un parent et un enfant. Le compteur de sites est par fichier. Quand un parent analyse un enfant d'un autre fichier, `eval_comp_app` copie dans le tas initial de l'enfant les entrées de tas de ses props — des sites *du fichier du parent* — alors que l'enfant alloue avec les sites *de son fichier*. Si les numéros coïncident, l'objet de l'enfant écrase la fermeture du parent (`/tmp/ch08/heap/`) :

```
1 // A.tsx : la fermeture passée en prop est Fn#1
2 export function Parent() {
3   const [n, setN] = useState(0);
4   return <Child onPick={() => setN(n + 1)} />;
5 }
6 // B.tsx, version 1 : `const b = 2;`; version 2 : `const b = { y: 2 };` (Obj#1)
7 export function Child({ onPick }) {
8   const a = { x: 1 };
9   const b = { y: 2 };
10  onPick();
11  return <div>{a.x}{b.y}</div>;
12 }
```

```
version 1 :
  Child (0 hooks) B.tsx
    error cross-setter-in-render var:onPick (line 4:2) prop `onPick` (a state setter of parent `Parent`) called
      ↳ during render of `Child`, which triggers a parent re-render on every render
        (1 trace step(s), rerun with --trace)
    1 clean component(s) hidden, rerun with --show-clean

! 1 error(s) across 2 file(s).
version 2 :
  1 clean component(s) hidden, rerun with --show-clean

✓ 2 file(s) no issues found.
```

Un `ERROR` disparaît parce qu'un objet sans rapport a été ajouté dans l'enfant : pas un défaut de la greffe, mais de la frontière qui *n'en est pas une* (chapitre 16) — `#134` a rendu les sites uniques par fichier et décale les greffes, mais pas l'entrée d'un enfant. Pistes : un `ExprId` unique à l'échelle du programme, ou un décalage des sites copiés à l'entrée de l'enfant.

8.9.3 Trois lectures fausses sans finding démontré

Un Assign non renommé. `bound_vars` ne compte que les `Let` : une variable seulement *assignée* dans l'appelé n'est ni liée ni libre au sens de `callee_rename_map`. Avec `util.ts = let mode = "a";`

export function setMode() mode = "b"; et un composant qui écrit let mode = s; setMode(); (/tmp/ch08/assign/), le rendu après greffe contient mode := "b" juste après let mode = s : une écriture forte de la locale de l'appelant par une variable du module de l'appelé — abstraitement "b", concrètement s. `infinite-loop` est quand même émis, pour un autre motif ; c'est le trou symétrique de #141, qui ne traite que les lectures.

caller_bound ne voit que le CFG greffé. Dans l'exemple du exemple 8.1, le même `lim()` greffé dans le corps de l'effet donne let v#1 = LIMIT : la table protège les liaisons *du corps de l'effet*, qui ne lie pas LIMIT ; la lecture de l'import devient une lecture de la prop capturée par l'effet. La protection de #141 ne s'étend pas à la portée lexicale englobante.

La substitution d'init qui ignore les let d'une fermeture, vue au section 8.4.

Remarque

Un dernier constat, révélé par la greffe mais hors de son périmètre. `bump(setN, n); dans le rendu` se greffe correctement (`set#0 = setN, set#0((v#0 Add 1));`), mais `setter-in-render` se tait, là où `setN(n + 1); direct` donne un ERROR — ADR-20 affirmant à tort qu'aucun alias de paramètre n'est émis. Cause côté règle : elle ne suit que let x = StateSetter(1) (src/rules/impls/setter_in_render.rs, l. 63–86), pas l'alias, alors que le mécanisme générique (resolve_setter_aliases, src/engine/setters.rs, l. 576–580, déjà utilisé par `infinite-loop`, `unnecessary-rerender`, `stale-closure`) existe pour exactement ce cas.

Résumé du chapitre

- Un hook custom n'a pas d'analyse propre : ses faits dépendent des arguments de chaque appelant (ADR-24). Le moteur greffe donc son corps dans chaque composant qui l'appelle, avant l'amorçage et le point fixe ; il fait de même pour les utilitaires de module appelés en position d'instruction, et greffe ceux-ci en premier.
- Une greffe décale quatre espaces d'identités : les blocs (dans la primitive), les labels et les sites (Offsets, remap_cfg, remap_hooks, plages tirées de SpliceIds), les noms (α -renommage x#s).
- `splice_callee_into_cfg` vérifie ses préconditions avant toute mutation, coupe le bloc d'appel, lie les paramètres par des Let à la position de l'appel, réécrit chaque Return(e) en let bound = e + saut vers une jointure qui porte la suite et l'ancien terminateur, laisse Unreachable coupé (ADR-25), préserve les genres d'arêtes et rend la plage de blocs pour la provenance.
- La table de renommage couvre toutes les liaisons de l'appelé et ses seuls noms libres que l'appelant lie (#141) ; elle est appliquée au corps et aux corps de ses hooks. Les init d'état reçoivent en plus une substitution des arguments.
- `expand_custom_hooks` décale les labels au-dessus de la table courante, garde la ligne de provenance du wrapper, marque les lignes inlinées, enregistre la région (ADR-27 §4) et réexamine la position insérée pour les hooks imbriqués ; sa garde par nom laisse opaque un deuxième appel du même hook.
- Fermetures locales (B5/B6) et composants enfants ne sont pas greffés ; les hooks de bibliothèque sont résumés (SummaryRegistry), et « connu n'est pas modélisé ».
- Un finding de hook inliné est rendu à son origine et jamais fusionné entre consommateurs (ADR-24).
- Défauts observés : return jeté sans liaison (FN démontré), sites en collision (dans un composant, et entre parent et enfant, FN ERROR démontré), garde par nom, et trois lectures fausses sans finding démontré.

Fichiers rencontrés : `src/ir/splice.rs`; `src/ir/remap.rs`; `src/engine/fixpoint.rs` (`SpliceIds`, `expand_custom_hooks`, `expand_utility_calls`); `src/engine/hook_registry.rs`; `src/engine/function_registry.rs`; `src/registry/summary.rs`; `src/driver/human.rs`. Le chapitre suivant ouvre la partie III : les treillis dans lesquels vivent les valeurs que le CFG greffé va propager (chapitre 9).

Exercices

Exercice 8.1: Numéroter une greffe

Un rendu a les blocs B0 à B3, l'appel est dans B2, et l'appelé a trois blocs B0 (entrée, branch), B1 (return a), B2 (throw). Donnez les identifiants des blocs de l'appelé et de la jointure après la greffe, et la liste des arêtes ajoutées. Quel bloc n'a pas de successeur ?

Solution

`block_offset` = 4 : l'appelé occupe B4, B5, B6 ; la jointure est $4 + 3 = 7$. Arêtes : les sorties de B2 partent de B7 ; B2→B4 ; B4→B5 et B4→B6 (genres préservés) ; B5→B7. B6 (Unreachable) n'a pas de successeur.

Exercice 8.2: Prédire la table de renommage

Un hook function `useX(a) const b = a + k; const [s] = useState(b); useEffect(() => log(s, t); ); return s;` est appelé dans un composant qui lie `t` et `s`, mais pas `k` ni `log`. Sel 2. Quels noms sont renommés, et en quoi ?

Solution

Liaisons de l'appelé : `a`, `b`, `s` (et les temporaires du déstructurage) → `a#2`, `b#2`, `s#2`. Libres : `k`, `log` (non liés par l'appelant, gardés) et `t`, lu dans le corps de l'effet et lié par l'appelant → `t#2`. D'où l'importance de passer hooks à `callee_rename_map`.

Exercice 8.3: Corriger le return jeté

Écrivez la modification du bras `Terminator::Return(ret)` de `splice_callee_into_cfg` qui supprime le faux négatif de `Ret.tsx`, et expliquez pourquoi elle ne peut pas créer de faux négatif ailleurs.

Solution

`match bound_var Some(var) => push Stmt::Let var, rhs: ret, span: call_span ), None => push Stmt::ExprStmt(ret, call_span))`. Elle ne fait qu'ajouter au modèle une évaluation que le programme concret effectue (comme le lowering de `void e`, `src/lowering/expr_lower.rs`, l. 227–232) : les comportements abstraits couvrent au moins ceux d'avant, donc aucune écriture réelle ne peut disparaître ; au pire apparaissent des faux positifs, tolérés.

Exercice 8.4: Deux appels du même hook

Proposez une garde de récursion pour `expand_custom_hooks` qui greffe les deux appels de `Twice.tsx` tout en terminant sur `function useA() useA();`. Quelle information faut-il ajouter aux entrées insérées ?

Solution

Interdire un nom seulement dans sa propre expansion : chaque entrée insérée porte la pile des hooks dont elle provient, et un `Custom` de nom `n` n'est sauté que si `n` figure dans sa

propre pile. Deux appels frères ont des piles disjointes ; un appel récursif retrouve son nom dans sa pile.

Troisième partie

Les domaines abstraits

Chapitre 9

Les treillis simples : plats, ensembles bornés, intervalles

Objectifs du chapitre.

- Lire le trait `AbstractDomain` comme un contrat : ce que chaque méthode doit garantir pour que l'analyse reste sound.
- Connaître les trois familles de treillis « simples » du code : les treillis plats (`BoolVal`, `SetterVal`), l'ensemble borné de chaînes (`StrConst`), les intervalles (`Interval`).
- Savoir calculer à la main toutes les opérations des intervalles : ordre, join, arithmétique (`+` `-` `*` `%` `**`), opérateurs bit à bit, widening simple et à seuils, raffinement sur gardes.
- Savoir prouver en quelques lignes la soundness de chacune de ces opérations, et reconnaître les endroits où le code s'en écarte.
- Connaître les limites observées : `NaN` et $\pm\infty$, $0 \times \pm\infty$, décalages sur des non-entiers, cycles finis.

Prérequis : chapitre 3 (treillis, concrétisation, widening, soundness : définition 3.2, définition 3.3, définition 3.6, définition 3.8) ; chapitre 6 pour les expressions `Expr`, `BinOp`, `Prim` ; chapitre 7 pour la forme des conditions de branche.

Le chapitre 3 a présenté les treillis « en un coup d'œil » et suivi un compteur jusqu'à son point fixe. Ce chapitre ouvre la boîte. Il décrit, opération par opération, les quatre domaines de base sur lesquels repose toute valeur abstraite de `reactant-analyzer`. Ces domaines sont petits (moins de 900 lignes à eux quatre), mais chaque ligne compte : une borne mal arrondie ou un cas $\perp$ oublié suffit à produire un faux négatif, et un faux négatif est interdit (définition 3.8).

Le chapitre 10 assemblera ces domaines en un produit, `StateValue`, et y ajoutera le treillis de stabilité référentielle. On peut déjà situer les pièces (figure 9.1) : sur les huit composantes de `StateValue`, quatre sont les treillis de ce chapitre, trois sont de simples booléens « cette sorte est-elle possible ? », et une est la stabilité.

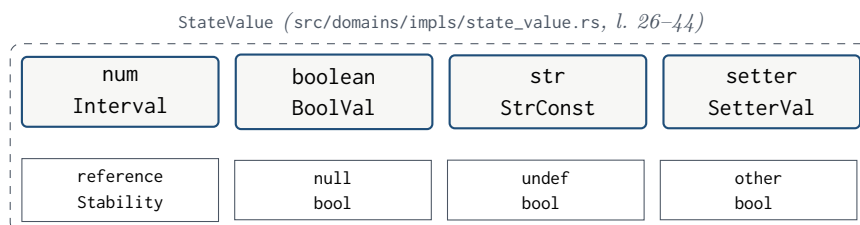


FIGURE 9.1 – Les huit composantes du produit `StateValue`. En bleu, les treillis étudiés dans ce chapitre ; en gris, ceux du chapitre 10 (la stabilité et les trois bits de sorte).

9.1 Le contrat d'un domaine : le trait `AbstractDomain`

9.1.1 Une question concrète

Prenons le programme qui servira de fil rouge à tout le chapitre.

Extrait 9.1 – Deux compteurs (`/tmp/ch09/counter.tsx`, repris de `tests/fixtures/widening.tsx`)

```
1 import { useState, useEffect } from "react";
2
3 export function Unbounded() {
4   const [count, setCount] = useState(0);
5   useEffect(() => {
6     setCount(count + 1);
7   }, [count]);
8   return <div>{count}</div>;
9 }
10
11 export function Guarded() {
12   const [count, setCount] = useState(0);
13   useEffect(() => {
14     if (count < 10) setCount(count + 1);
15   }, [count]);
16   return <div>{count}</div>;
17 }
```

`Unbounded` boucle : chaque exécution de l'effet écrit une nouvelle valeur dans `count`, ce qui provoque un nouveau rendu, donc une nouvelle exécution de l'effet. `Guarded` s'arrête à 10. Pour les distinguer, l'analyseur doit répondre à une question sur des *nombres* : l'ensemble des valeurs que `count` peut prendre est-il borné ? Il lui faut donc un domaine qui représente des ensembles de nombres (les intervalles), qui sache calculer `count + 1` (l'arithmétique), exploiter `count < 10` (le raffinement sur garde) et converger malgré une suite de valeurs infinie (le widening). L'analyseur répond ainsi :

```
$ reactant check /tmp/ch09/counter.tsx --show-clean
Guarded (2 hooks) /tmp/ch09/counter.tsx ✓
Unbounded (2 hooks) /tmp/ch09/counter.tsx
  warn infinite-loop [hook:0] (line 5:2) this effect keeps pushing state `count` (its deps do not provably gate it
    ↳ , so the effect can re-run every render) to new values on every run. Potential infinite render loop
      (2 trace step(s), rerun with --trace)

! 1 warning(s) across 1 file(s).
```

Toutes les pièces de cette réponse sont dans ce chapitre, sauf la règle elle-même (chapitre 19) et la boucle de point fixe (chapitre 13).

9.1.2 Le trait, méthode par méthode

Un domaine abstrait est, dans le code, un type qui implémente `AbstractDomain` (`src/domains/mod.rs`). La tête du trait a été présentée au chapitre 3 (liste 3.2) : `bottom`, `top`, `is_bottom`, `join`, `meet`, `widen`, et `widen_to` dont l'implémentation par défaut ignore les seuils. L'ordre du treillis n'est pas une méthode : c'est le supertrait `PartialOrd` (`a <= b` signifie $a \sqsubseteq b$, `partial_cmp` rend `None` pour deux éléments incomparables). La suite du trait est moins connue ; la voici.

Extrait 9.2 – `src/domains/mod.rs`, l. 40–85 — ponts vers `StateValue` et raffinements par défaut

```
40 /// Try to recover the underlying `StateValue`, if this domain IS `StateValue`.
41 /// Default returns `None` (other domains). Used when the heap needs to store
```



```

42  /// a captured environment with `StateValue` type (e.g. closure capture at FnLit creation).
43  fn as_state_value(&self) -> Option<StateValue> {
44      None
45  }
46
47  /// Convert a `StateValue` back to this domain.
48  /// Default returns `Self::bottom()` (other domains drop the value conservatively).
49  /// `StateValue` overrides to return itself. Used when restoring captured closure env.
50  fn from_state_value(sv: StateValue) -> Self {
51      let _ = sv;
52      Self::bottom()
53  }
54
55  // Branch narrowing: default = identity (sound, imprecise).
56  // Override for numeric domains to refine interval bounds on branch conditions.
57
58  // Nullability narrowing (ADR-015). The IR conflates `==`/`===` into `Eq`,
59  // so the refinements below are the sound envelope of both semantics.
60  /// Taken `x != null` (or false `x === null`): null impossible.
61  fn narrow_drop_null(self) -> Self {
62      self
63  }
64  /// Taken `x !== undefined` (or false `x === undefined`): undefined impossible.
65  fn narrow_drop_undef(self) -> Self {
66      self
67  }
68  /// Taken `x == null` / `x == undefined`: only null/undefined survive.
69  fn narrow_keep_nullish_only(self) -> Self {
70      self
71  }
72  /// Taken truthiness guard `if (x)`: excludes every falsy JS value
73  /// (null, undefined, 0, "", false).
74  fn narrow_truthy(self) -> Self {
75      self
76  }
77  /// Falsy branch (`else` of `if (x)`, taken `if (!x)`): only falsy values
78  /// survive (null, undefined, 0, "", false).
79  fn narrow_falsy(self) -> Self {
80      self
81  }
82
83  fn narrow_lt(self, _v: f64) -> Self {
84      self
85  }

```

Suivent `narrow_leq`, `narrow_gt`, `narrow_geq`, `narrow_eq` et `narrow_neq`, de la même forme (src/domains/mod.rs, l. 86–100). Trois groupes de méthodes :

- **Les ponts** `as_state_value` / `from_state_value` servent au tas, qui range l'environnement capturé par une fermeture sous forme de `StateValue`. Seul `StateValue` les redéfinit.
- **Les raffinements de sorte** (`narrow_drop_null`, `narrow_drop_undef`, `narrow_keep_nullish_only`, `narrow_truthy`, `narrow_falsy`) ne concernent que le produit, seul à savoir ce qu'est une sorte; ils sont étudiés au chapitre 10, sauf `narrow_truthy` dont l'effet sur les nombres est vu en section 9.8.
- **Les raffinements numériques** `narrow_lt` ... `narrow_neq` restreignent une valeur à celles qui satisfont $x < v$, $x \leq v$, etc., pour un littéral v . `Interval` les redéfinit, et `StateValue` les délègue à sa composante `num`.

Tous les raffinements valent l'identité par défaut, et le commentaire dit pourquoi c'est permis

: ne rien raffiner est toujours sound, seulement imprécis. Le tableau 9.1 récapitule qui redéfinit quoi.

Domaine	widen_to	ponts	narrow_lt ... narrow_neq	raffinements de sorte
BoolVal, SetterVal (macro)	défaut	défaut	identité	identité
StrConst	défaut	défaut	identité	identité
Stability	défaut	défaut	identité	identité
Interval	redéfini	défaut	redéfinis	identité
StateValue	redéfini	redéfinis	délégués à num	redéfinis

TABLE 9.1 – Les méthodes à implémentation par défaut de `AbstractDomain`, et les domaines qui les redéfinissent (relevé des `impl AbstractDomain`).

9.1.3 Le contrat

Le trait ne dit rien des propriétés que ses méthodes doivent avoir ; elles sont pourtant le fondement de la soundness. Écrivons-les une fois.

Définition 9.1: Contrat d'un domaine abstrait

Soit D un type qui implémente `AbstractDomain`, muni d'une concrétisation $\gamma : D \rightarrow \mathcal{P}(\mathcal{V})$ (définition 3.3), où $\mathcal{V}$ est l'ensemble des valeurs JavaScript concernées. Le domaine respecte le contrat si, pour tous $a, b \in D$:

1. *ordre* : $a \sqsubseteq b \Rightarrow \gamma(a) \subseteq \gamma(b)$;
2. *bottom et top* : $\gamma(\perp) = \emptyset$ et $\gamma(\top) = \mathcal{V}$; `is_bottom(a)` n'est vrai que si $\gamma(a) = \emptyset$;
3. *join* : $a \sqsubseteq a \sqcup b$ et $b \sqsubseteq a \sqcup b$, donc $\gamma(a) \cup \gamma(b) \subseteq \gamma(a \sqcup b)$;
4. *widening* : `widen` et `widen_to` (pour tout ensemble fini de seuils) majorent le `join` et rendent toute suite d'élargissements stationnaire (définition 3.6) ;
5. *refinement* : pour une condition c et son opérateur `narrow_c`, $\gamma(\text{narrow_c}(a)) \supseteq \gamma(a) \cap \{x \mid c(x)\}$;
6. *meet* : $\gamma(a) \cap \gamma(b) \subseteq \gamma(a \sqcap b)$.

Les conditions 3 à 5 portent la soundness : un `join` ou un `refinement` qui perdrait une valeur concrète ferait disparaître un comportement réel. Le `join` n'a pas besoin d'être la plus petite borne supérieure pour être sound (il suffit qu'il majore), mais il l'est pour les quatre domaines du chapitre, et c'est ce qui fait leur précision. La condition 6 est de pure forme aujourd'hui :

Piège

`meet` n'est appelé nulle part hors des tests et des délégations internes (`StateValue::meet` vers ses composantes). Le `refinement` de l'analyse passe exclusivement par les `narrow_*`, jamais par `meet`. Un contributeur qui voudrait raffiner un état par « intersection avec la garde » doit donc écrire un `narrow_*`, ou vérifier d'abord que le `meet` du domaine concerné respecte la condition 6 (ce n'est pas le cas de `Stability`, chapitre 10).

Piège

Dans `impl AbstractDomain for Interval`, les `raffinements` du trait sont écrits `Interval::narrow_lt(&self, v)` et non `self.narrow_lt(v)` (`src/domains/impls/interval.rs`, l. 362–365). La méthode du trait prend `self` par valeur, la méthode inhérente `&self`. Dans le corps du trait, `self.narrow_lt(v)` serait résolu dès l'étape « par valeur » de la recherche de méthode, donc vers la méthode du trait elle-même : récursion infinie. Le commentaire « fully-qualified » du code le signale.

Remarque

Le commentaire en tête du trait annonce encore Clone + Copy, alors que la borne réelle ne contient pas Copy (chapitre 3). Des quatre domaines de ce chapitre, BoolVal et Interval sont Copy ; SetterVal et StrConst ne dérivent pas Copy (le second contient un Arc), d'où les clone() de la macro ci-dessous.

9.2 Les treillis plats : BoolVal et SetterVal

9.2.1 Deux exemples React

Un booléen d'état est l'exemple le plus simple de valeur à suivre :

Extrait 9.3 – Un booléen d'état et un setter choisi dynamiquement (exemple d'illustration)

```
1 export function Panel({ admin }: { admin: boolean }) {
2   const [open, setOpen] = useState(false);
3   const [a, setA] = useState(0);
4   const [b, setB] = useState(0);
5   const set = admin ? setA : setB;
6   return <button onClick={() => { setOpen(!open); set(1); }}>{open ? "-" : "+"}</button>;
7 }
```

Que sait-on de open ? Au premier rendu, qu'il vaut false ; après un clic, qu'il vaut true ; en général, qu'il vaut l'un ou l'autre. Que sait-on de set ? Que c'est *un* setter de useState, mais lequel dépend de la prop : setA ou setB. Dans les deux cas, l'information utile est « une valeur précise, ou bien on ne sait pas laquelle ». C'est exactement ce que capture un treillis plat.

Définition 9.2: Treillis plat

Soit M un ensemble (les éléments « intermédiaires »). Le *treillis plat* sur M est $M \cup \{\perp, \top\}$ ordonné par $\perp \sqsubseteq m \sqsubseteq \top$ pour tout $m \in M$, deux éléments distincts de M étant incomparables. Son join vaut $a \sqcup a = a$, $\perp \sqcup x = x \sqcup \perp = x$, et $\top$ dans tous les autres cas ; son meet est le dual. Sa hauteur est 2 (plus longue chaîne stricte : $\perp \sqsubset m \sqsubset \top$).

Figure 9.2 dessine le treillis plat générique et indique, sur chaque arête, quel bras du code la justifie.

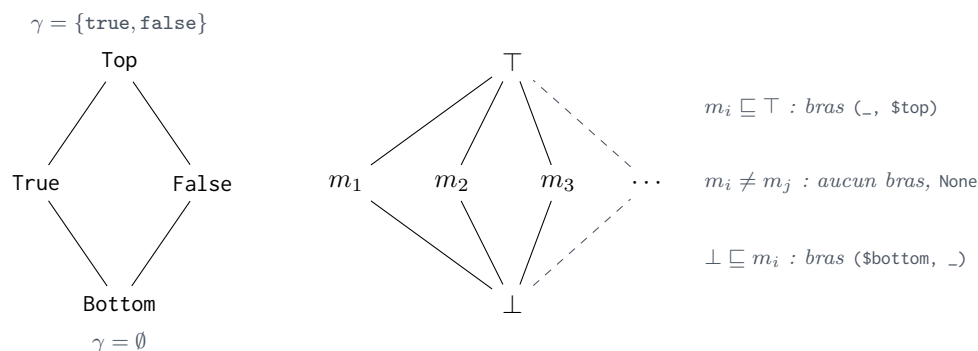


FIGURE 9.2 – À gauche, BoolVal, dont la concrétisation est une bijection avec $\mathcal{P}(\{\text{true}, \text{false}\})$. À droite, le treillis plat engendré par flat_lattice! pour un ensemble quelconque d'éléments intermédiaires, avec le bras de partial_cmp qui établit chaque relation.

9.2.2 Ce que fait le code : la macro `flat_lattice!`

Les deux treillis plats du code ne diffèrent que par la charge utile de leurs éléments intermédiaires. Leur ordre et leurs opérations sont donc engendrés par une seule macro, introduite lors de la campagne de dette technique (décision D5 d'ADR-20).

Extrait 9.4 – `src/domains/impls/mod.rs`, l. 1–53 — la macro `flat_lattice!`

```

1  /// Generate the flat-lattice `PartialOrd` + `AbstractDomain` impls for an enum
2  /// with a distinguished `$bottom` ( $\perp$ ) and `$top` ( $\top$ ) *unit* variant. Every
3  /// other variant is an incomparable "mid" element: `a  $\sqcup$  b = a` when `a == b`,
4  /// else ` $\top$ ` (dually for meet); ` $\perp < \text{mid} < \top$ `, two distinct mids incomparable.
5  /// Widen = join (a flat lattice has finite height). Requires `PartialEq + Clone`.
6  ///
7  /// Shared by [`bool_val::BoolVal`] and [`setter_val::SetterVal`] – the two flat
8  /// lattices in the product domain, whose only difference is the payload of
9  /// their mid elements.
10 macro_rules! flat_lattice {
11     ($ty:ident, bottom = $bottom:ident, top = $top:ident) => {
12         impl ::std::cmp::PartialOrd for $ty {
13             fn partial_cmp(&self, other: &Self) -> ::core::option::Option<::std::cmp::Ordering> {
14                 use ::std::cmp::Ordering;
15                 match (self, other) {
16                     (a, b) if a == b => Some(Ordering::Equal),
17                     ($ty::$bottom, _) | (_, $ty::$top) => Some(Ordering::Less),
18                     ($ty::$top, _) | (_, $ty::$bottom) => Some(Ordering::Greater),
19                     _ => None,
20                 }
21             }
22         }
23
24         impl $crate::domains::AbstractDomain for $ty {
25             fn bottom() -> Self {
26                 $ty::$bottom
27             }
28             fn top() -> Self {
29                 $ty::$top
30             }
31             fn is_bottom(&self) -> bool {
32                 matches!(self, $ty::$bottom)
33             }
34             fn join(&self, other: &Self) -> Self {
35                 match (self, other) {
36                     (a, b) if a == b => a.clone(),
37                     ($ty::$bottom, x) | (x, $ty::$bottom) => x.clone(),
38                     _ => $ty::$top,
39                 }
40             }
41             fn meet(&self, other: &Self) -> Self {
42                 match (self, other) {
43                     (a, b) if a == b => a.clone(),
44                     ($ty::$top, x) | (x, $ty::$top) => x.clone(),
45                     _ => $ty::$bottom,
46                 }
47             }
48             fn widen(&self, other: &Self) -> Self {
49                 self.join(other)
50             }
51         }
52     };
53 }

```

Lisons-la bras par bras.

- **Paramètres** (l. 11). La macro reçoit le type et les noms de ses variantes $\perp$ et $\top$; toutes les autres variantes sont les éléments intermédiaires. Elle exige `PartialEq + Clone` (les gardes `a == b` et les `clone()`).
- **partial_cmp** (l. 13–21). L'égalité d'abord ; puis « $\perp$ à gauche ou $\top$ à droite » donne `Less` ; puis « $\top$ à gauche ou $\perp$ à droite » donne `Greater` ; le reste, deux intermédiaires distincts, est incomparable. L'ordre des bras compte pour le couple $(\perp, \top)$, qui satisfait les deux motifs : le premier gagne, et c'est bien `Less`.
- **join** (l. 34–40). Égaux : on garde. L'un est $\perp$: on garde l'autre (le motif `($bottom, x) | (x, $bottom)` lie `x` des deux côtés). Sinon : $\top$. Ce « sinon » couvre aussi les cas où l'un est $\top$, ce qui est correct.
- **meet** (l. 41–47) est le dual : $\top$ est neutre, deux éléments différents donnent $\perp$.
- **widen** (l. 48–50) est le join : la hauteur est finie.

Proposition 9.1: flat_lattice! engendre un treillis

Pour tout type instancié par `flat_lattice!`, `partial_cmp` est l'ordre du treillis plat de la définition 9.2, `join` en est la plus petite borne supérieure, `meet` la plus grande borne inférieure, et `widen` est un widening.

Démonstration. L'ordre : les trois bras reproduisent exactement les trois clauses de la définition (réflexivité, $\perp$ minimum, $\top$ maximum), le dernier bras déclare incomparables deux intermédiaires distincts. Le join : si $a = b$, a est la plus petite borne de $\{a\}$; si $a = \perp$, b majore a et b et tout majorant de b en est un de $\{a, b\}$; sinon, soit l'un vaut $\top$ (seul majorant), soit a et b sont deux intermédiaires distincts, dont le seul majorant commun est $\top$. Le meet est dual. Le widening : `widen = join` majore le join, et toute suite croissante d'un treillis de hauteur 2 est stationnaire après au plus deux pas stricts. $\square$

La macro est définie *avant* les déclarations `pub mod bool_val` ; et `pub mod setter_val` ; (`src/domains/impls/mod.rs`, l. 55–60). C'est la portée *textuelle* des `macro_rules!` qui la rend visible dans ces sous-modules, sans `#[macro_export]` ni `use`. Déplacer la déclaration des modules au-dessus de la macro casserait la compilation.

9.2.3 BoolVal : un treillis plat exact

Extrait 9.5 – `src/domains/impls/bool_val.rs`, l. 1–13 — le treillis des booléens

```
1 /// Flat lattice for the boolean slot of `StateValue`:  $\perp < \{\text{True}, \text{False}\} < \top$ ,
2 /// with `True`/`False` incomparable. Lattice ops via the `flat_lattice!` macro.
3 #[derive(Debug, Clone, Copy, PartialEq, Eq)]
4 pub enum BoolVal {
5     ///  $\perp$  unreachable.
6     Bottom,
7     True,
8     False,
9     ///  $\top$  may be either.
10    Top,
11 }
12
13 flat_lattice!(BoolVal, bottom = Bottom, top = Top);
```

La concrétisation est $\gamma(\text{Bottom}) = \emptyset$, $\gamma(\text{True}) = \{\text{true}\}$, $\gamma(\text{False}) = \{\text{false}\}$, $\gamma(\text{Top}) = \{\text{true}, \text{false}\}$. C'est une bijection avec les parties de $\{\text{true}, \text{false}\}$, compatible avec l'ordre :

le treillis plat *est* ici le treillis des parties, et join et meet sont exacts ($\gamma(a \sqcup b) = \gamma(a) \cup \gamma(b)$). C'est le seul domaine du chapitre sans aucune perte.

Où naissent les booléens abstraits ? Un littéral `true` ou `false` donne `True` ou `False`, comme un littéral numérique donne `Interval::point` et une chaîne un singleton : trois cas d'un même `match` (`src/domains/transfer/state_value.rs`, l. 112–117).

La négation `!x` inverse exactement un booléen pur (`src/domains/transfer/state_value.rs`, l. 947–965). En revanche, *toute* comparaison (`=`, `<`, `in`, `instanceof...`) rend `BoolVal::Top`, même `1 < 2` (`src/domains/transfer/state_value.rs`, l. 797–806) : le domaine ne calcule pas la valeur d'une comparaison ; c'est le raffinement de branche (section 9.8) qui exploite la condition. Rendre un booléen pur, et non $\top$ tout entier, a son importance : les composantes `num` et `str` du résultat restent à $\perp$, ce qui permet d'enchaîner d'autres raffinements sur le résultat.

Piège

`!x` sur un x qui vaut $\perp$ rend `boolean(Top)` et non $\perp$: le test « booléen pur » de la ligne 949 est satisfait par $\perp$ (toutes ses composantes sont vides), puis le `match` tombe dans `_ => BoolVal::Top`. Un chemin mort redevient vivant pour la suite de l'évaluation. C'est imprécis, pas `unsound` : $\top$ majore tout.

9.2.4 SetterVal : un treillis plat à identité

Extrait 9.6 – `src/domains/impls/setter_val.rs`, l. 3–37 — le treillis plat des setters

```

3 /// Flat lattice for the component-setter slot of `StateValue`.
4 ///
5 /// ```text
6 ///      Top    (some setter, identity unknown)
7 ///      /  |  \
8 ///   One(a) One(b) One(c) ...
9 ///      \  |  /
10 ///      Bottom (not a setter)
11 /// ```
12 ///
13 /// React guarantees setter identity across renders, so any non-bottom
14 /// value maps to `Stability::Stable`.
15 #[derive(Debug, Clone, PartialEq, Eq)]
16 pub enum SetterVal {
17     ///  $\perp$  – no setter value possible.
18     Bottom,
19     /// Exactly this component's setter for this hook label.
20     One(ComponentId, HookLabel),
21     ///  $\top$  – some setter, but which one was lost at a join.
22     Top,
23 }
24
25 impl SetterVal {
26     /// Payload accessor: `Some` only when the setter identity is exact.
27     pub fn as_one(&self) -> Option<(&ComponentId, &HookLabel)> {
28         match self {
29             SetterVal::One(c, l) => Some((c, l)),
30             _ => None,
31         }
32     }
33 }
34
35 // Flat lattice:  $\perp < \text{One}(\dots) < \top$ , distinct One(..) incomparable. React
36 // guarantees setter identity across renders, so any non- $\perp$  value is `Stable`.
37 flat_lattice!(SetterVal, bottom = Bottom, top = Top);

```

- $\text{One}(\text{ComponentId}, \text{HookLabel})$ désigne le setter du slot HookLabel (définition 6.3) du composant ComponentId : $\gamma(\text{One}(c, l)) = \{\text{la fonction } \text{setState} \text{ du slot } l \text{ de } c\}$. L'identité est un ComponentId interné (ADR-40), plus un nom de composant comme dans les premières versions.
- Top : γ est l'ensemble de tous les setters de useState . On sait que la valeur est un setter, pas lequel.
- as_one (l. 27–32) est l'accessor qu'utilisent les règles et les relations pour savoir *quel* slot un appel écrit : il ne répond que si l'identité est exacte.

Dans le liste 9.3, les deux branches de $\text{admin ? setA : setB}$ donnent $\text{One}(c, 1)$ et $\text{One}(c, 2)$; leur join est Top . Ici le treillis plat *perd* de l'information : $\gamma(\text{Top})$ contient tous les setters, alors que la vérité est « setA ou setB ». La perte est sound (on majore) et c'est le prix de la hauteur 2.

Rappel React : identité des setters

React garantit que la fonction setState rendue par useState garde la même identité d'un rendu à l'autre. C'est pourquoi tout SetterVal non $\perp$ est projeté sur la stabilité Stable (commentaire des lignes 13–14 et 35–36) : on peut ignorer quel setter, on sait qu'il ne change pas.

9.3 Un ensemble borné de chaînes : StrConst

9.3.1 Le problème

Les chaînes d'état sont souvent des *énumérations* déguisées :

Extrait 9.7 – Un mode d'affichage (exemple d'illustration)

```
1 export function Gallery() {
2   const [mode, setMode] = useState("grid");
3   return (
4     <div className={"gallery-" + mode}>
5       <button onClick={() => setMode("list")}>list</button>
6       <button onClick={() => setMode("grid")}>grid</button>
7     </div>
8   );
9 }
```

Un treillis plat sur les chaînes dirait de $\text{mode} : \top$, puisque deux constantes différentes y sont écrites. On aimerait garder $\{\text{"grid"}, \text{"list"}\}$, et pour className , $\{\text{"gallery-grid"}, \text{"gallery-list"}\}$. Le treillis des parties de l'ensemble des chaînes le permettrait, mais il est de hauteur infinie : une chaîne qui grandit ($s + \text{"x"}$) engendrerait une suite d'ensembles strictement croissante. La solution classique borne la taille des ensembles.

Définition 9.3: Ensemble k -borné

Soit S un ensemble (ici les chaînes) et $k \geq 1$. Le treillis des *ensembles k -bornés* sur S est $\{\perp\} \cup \{X \subseteq S \mid 1 \leq |X| \leq k\} \cup \{\top\}$, ordonné par l'inclusion, $\perp$ en dessous de tout et $\top$ au-dessus. La concrétisation est $\gamma(\perp) = \emptyset$, $\gamma(X) = X$, $\gamma(\top) = S$. Le join de X et Y est $X \cup Y$ si $|X \cup Y| \leq k$, et $\top$ sinon ; le meet est $X \cap Y$ (et $\perp$ si l'intersection est vide). La hauteur est $k + 1$.

9.3.2 Ce que fait le code

Extrait 9.8 – src/domains/impls/str_const.rs, l. 7–39 — StrConst, ses constructeurs, $k = 4$

```

7  /// Max strings tracked before widening to Top.
8  const STR_WIDEN_THRESHOLD: usize = 4;
9
10 /// Abstract string-constant domain: a finite powerset lattice with widening.
11 ///
12 /// Bottom  $\leq$  Set(s)  $\leq$  Top. Set(a)  $\leq$  Set(b) iff  $a \subseteq b$ .
13 /// join(Set(a), Set(b)) widens to Top when  $|a \cup b| > \text{STR\_WIDEN\_THRESHOLD}$ .
14 #[derive(Debug, Clone, PartialEq)]
15 pub enum StrConst {
16     ///  $\perp$  no possible string value (unreachable path).
17     Bottom,
18     /// Finite known set of string constants.
19     Set(Arc<BTreeSet<String>>),
20     ///  $\top$  any string (widened beyond threshold).
21     Top,
22 }
23
24 impl StrConst {
25     pub fn singleton(s: String) -> Self {
26         let mut set = BTreeSet::new();
27         set.insert(s);
28         StrConst::Set(Arc::new(set))
29     }
30
31     pub(crate) fn from_set(set: BTreeSet<String>) -> Self {
32         if set.is_empty() {
33             StrConst::Bottom
34         } else if set.len() > STR_WIDEN_THRESHOLD {
35             StrConst::Top
36         } else {
37             StrConst::Set(Arc::new(set))
38         }
39     }

```

- STR_WIDEN_THRESHOLD vaut 4 : c'est le k de la définition. Ce seuil vient du domaine produit d'origine (ADR-8), qui l'a transmis tel quel à ADR-15.
- Set(Arc<BTreeSet<String>>) : l'ensemble est trié (BTreeSet), ce qui rend l'égalité et l'affichage déterministes, et partagé (Arc), ce qui rend le clone d'une valeur abstraite bon marché : les environnements sont copiés à chaque bloc.
- singleton fabrique $\{s\}$; from_set est le constructeur *canonique* : vide $\mapsto \perp$, plus de 4 éléments $\mapsto \top$, sinon l'ensemble.

Invariant 9.1: Forme canonique de StrConst

Toute valeur Set(s) construite par singleton ou from_set vérifie $1 \leq |s| \leq 4$. L'invariant n'est pas imposé par le type : la variante étant publique, StrConst::Set(Arc::new($\emptyset$)) reste constructible, et is_bottom y répondrait false. Tout code qui fabrique un ensemble doit passer par from_set.

L'ordre et les opérations :

Extrait 9.9 – src/domains/impls/str_const.rs, l. 42–62 — l'ordre de StrConst : l'inclusion

```

42 impl PartialOrd for StrConst {
43     fn partial_cmp(&self, other: &Self) -> Option<Ordering> {
44         match (self, other) {

```



```

45     (a, b) if a == b => Some(Ordering::Equal),
46     (StrConst::Bottom, _) => Some(Ordering::Less),
47     (_, StrConst::Bottom) => Some(Ordering::Greater),
48     (_, StrConst::Top) => Some(Ordering::Less),
49     (StrConst::Top, _) => Some(Ordering::Greater),
50     (StrConst::Set(a), StrConst::Set(b)) => {
51         let a_sub_b = a.is_subset(b);
52         let b_sub_a = b.is_subset(a);
53         match (a_sub_b, b_sub_a) {
54             (true, true) => Some(Ordering::Equal),
55             (true, false) => Some(Ordering::Less),
56             (false, true) => Some(Ordering::Greater),
57             (false, false) => None,
58         }
59     }
60 }
61 }
62 }

```

Extrait 9.10 – src/domains/impls/str_const.rs, l. 75–102 — join, meet et widen de StrConst

```

75 fn join(&self, other: &Self) -> Self {
76     match (self, other) {
77         (a, b) if a == b => a.clone(),
78         (StrConst::Bottom, x) | (x, StrConst::Bottom) => x.clone(),
79         (StrConst::Top, _) | (_, StrConst::Top) => StrConst::Top,
80         (StrConst::Set(a), StrConst::Set(b)) => {
81             let union: BTreeSet<String> = a.iter().cloned().chain(b.iter().cloned()).collect();
82             StrConst::from_set(union)
83         }
84     }
85 }
86
87 fn meet(&self, other: &Self) -> Self {
88     match (self, other) {
89         (a, b) if a == b => a.clone(),
90         (StrConst::Bottom, _) | (_, StrConst::Bottom) => StrConst::Bottom,
91         (StrConst::Top, x) | (x, StrConst::Top) => x.clone(),
92         (StrConst::Set(a), StrConst::Set(b)) => {
93             let inter: BTreeSet<String> = a.intersection(b).cloned().collect();
94             StrConst::from_set(inter)
95         }
96     }
97 }
98
99 fn widen(&self, other: &Self) -> Self {
100     // join already applies the threshold widening = join for this domain
101     self.join(other)
102 }

```

L'ordre est l'inclusion, avec les deux sentinelles aux extrémités ; les bras $\perp$ précèdent les bras $\top$, si bien que $(\perp, \top)$ donne Less. Le join calcule l'union puis la fait passer par `from_set`, qui la remplace par $\top$ si elle a plus de 4 éléments. Le meet calcule l'intersection, qui ne peut pas grossir, et que `from_set` transforme en $\perp$ si elle est vide.

Figure 9.3 en dessine un fragment.

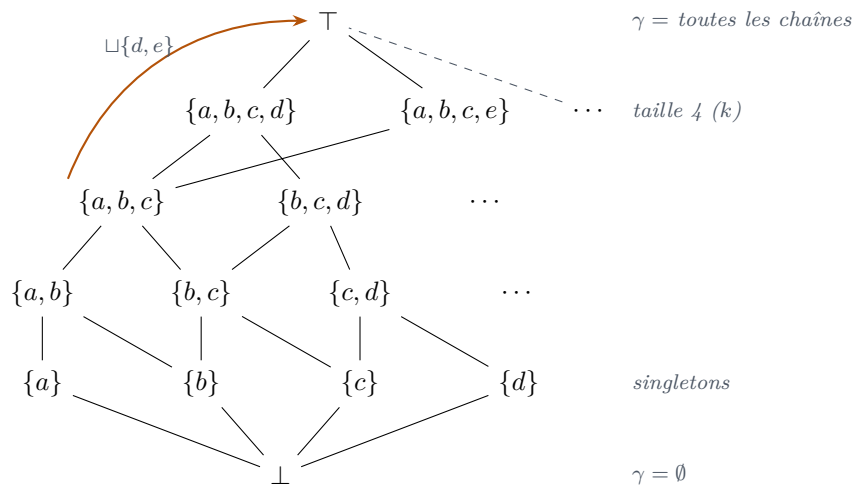


FIGURE 9.3 – Un fragment de `StrConst` (quelques arêtes seulement). Les ensembles de taille 1 à 4 sont ordonnés par inclusion ; au-delà, le join saute à $\top$: $\{a, b, c\} \sqcup \{d, e\} = \top$ (flèche), car l'union a cinq éléments.

Proposition 9.2: `StrConst` est un treillis de hauteur 5

`partial_cmp` est l'ordre des ensembles 4-bornés, join en est la plus petite borne supérieure, meet la plus grande borne inférieure. La plus longue chaîne stricte a six éléments ($\perp \sqsubset \{a\} \sqsubset \{a, b\} \sqsubset \{a, b, c\} \sqsubset \{a, b, c, d\} \sqsubset \top$), donc `widen` = join est un widening.

Démonstration. Pour le join de X et Y : tout majorant commun est soit $\top$, soit un ensemble Z de taille au plus 4 contenant X et Y , donc contenant $X \cup Y$. Si $|X \cup Y| \leq 4$, $X \cup Y$ est lui-même un élément et c'est le plus petit majorant. Sinon aucun ensemble représentable ne contient $X \cup Y$, et le seul majorant est $\top$. Pour le meet, tout minorant commun est $\perp$ ou un ensemble inclus dans X et dans Y , donc dans $X \cap Y$, qui a au plus 4 éléments : c'est le plus grand minorant (ou $\perp$ s'il est vide). La chaîne gagne au plus un élément par pas, et $\top$ est atteint au cinquième ajout. $\square$

La soundness du join découle de la proposition : il majore les deux opérandes, donc $\gamma(X) \cup \gamma(Y) \subseteq \gamma(X \sqcup Y)$. Mais l'égalité ne vaut plus au-delà du seuil : $\gamma(\{a, b, c\} \sqcup \{d, e\}) = S \supsetneq \{a, b, c, d, e\}$. Le « widening au seuil » du commentaire du code n'est donc pas un widening au sens de la définition 3.6 (c'est le join, qui suffit déjà), mais une perte de précision inscrite dans le join lui-même.

Piège

`StrConst` dérive `PartialEq`, et l'égalité d'un `Arc<BTreeSet<String>>` compare le *contenu*, pas les pointeurs (l'impl `PartialEq` for `Arc<T>` délègue à `τ`). Deux ensembles égaux alloués séparément sont égaux, ce qu'exige le test de convergence du point fixe. Remplacer cette égalité par `Arc::ptr_eq` « pour aller plus vite » empêcherait l'analyse de converger.

9.3.3 Où naissent les ensembles de chaînes

Un littéral chaîne donne un singleton. L'opérateur `+` sur deux chaînes pures calcule le produit cartésien des concaténations, puis le passe par `from_set` (`src/domains/transfer/state_value.rs`, l. 754–768, qui est le début de la table des opérateurs du liste 9.18). Dans le liste 9.7, "gallery-" + mode avec mode = {"grid", "list"} donne {"gallery-grid", "gallery-list"}. Si l'un des opérandes

est une chaîne inconnue ($\top$), le résultat est « une chaîne » (`StateValue::str_top()`), et non $\top$ tout entier. Enfin `typeof x` rend un singleton exact (`"number"`, `"string"...`) dès que x n'a qu'une sorte possible (`src/domains/transfer/state_value.rs`, l. 969–977).

Décision : ADR-20 — pas de combinateur `BoundedPowerset`

La campagne de dette technique a examiné l'extraction d'un combinateur générique `BoundedPowerset<T, N>` dont `StrConst` serait une instance (non-changement n° 5 d'ADR-20). Refusé : seul `StrConst` est un pur ensemble borné. `Stability`, qui ressemble à un ensemble borné de slots, mêle à ce fragment des éléments (`PerRender`, `Unknown`) aux joins croisés ; la faire entrer dans le combinateur restructurerait un type critique pour la soundness pour un seul consommateur réel. Abstraction prématurée.

9.4 Les intervalles : représentation, ordre, join

9.4.1 Le problème

Revenons à `Guarded` (liste 9.1). L'ensemble exact des valeurs de `count` est $\{0, 1, \dots, 10\}$. Pour `Unbounded`, c'est $\mathbb{N}$ tout entier. Un domaine de parties finies ne peut représenter le second, et le premier serait déjà coûteux avec une borne à 10^6 . Les *intervalles* gardent seulement la plus petite et la plus grande valeur : $[0, 10]$ et $[0, +\infty]$. On perd les trous (un intervalle est *convexe*), mais on garde exactement ce que la règle *infinite-loop* veut savoir : la valeur est-elle bornée ?

Définition 9.4: Intervalle abstrait

Un intervalle abstrait est un triplet (ℓ, h, ι) où $\ell, h \in \mathbb{R} \cup \{-\infty, +\infty\}$ et ι est un booléen (`is_int`). Sa concrétisation est

$$\gamma(\ell, h, \iota) = \{x \in \mathbb{R} \mid \ell \leq x \leq h\} \cap \begin{cases} \mathbb{Z} & \text{si } \iota \\ \mathbb{R} & \text{sinon.} \end{cases}$$

Tout triplet avec $\ell > h$ dénote $\emptyset$: c'est $\perp$. Le top est $[-\infty, +\infty]$ avec ι faux. La valeur JavaScript NaN n'appartient à la concrétisation d'*aucun* intervalle ; les bornes infinies signifient « pas de borne ».

La dernière phrase est une décision, pas une évidence : elle oblige toute opération qui peut produire NaN à sortir du domaine des intervalles (section 9.5).

9.4.2 Ce que fait le code

Extrait 9.11 – `src/domains/impls/interval.rs`, l. 5–27 — la structure `Interval`

```
5 /// Closed interval [lo, hi] over f64. `lo > hi` = bottom (empty).
6 ///
7 /// `is_int` records whether every concrete value the interval denotes is an
8 /// integer (`true` = proven all-integer; `false` = may contain non-integers).
9 /// It is a *precision* annotation only: it lets `narrow_lt`/`narrow_gt` apply
10 /// the integer tightening `x < 5 ==> x <= 4` (ADR-014 threshold widening relies on
11 /// this to bound counting loops) while staying sound over reals when the flag is
12 /// clear – a float state `x = 1.7` under `x < 2` must keep 1.7, not drop it.
13 /// Because it carries no set-membership information beyond `[lo, hi]`, it is
14 /// deliberately excluded from `PartialEq`/`PartialOrd`: two intervals with the
15 /// same bounds are equal (and the fixpoint converges) regardless of `is_int`.
16 #[derive(Debug, Clone, Copy)]
17 pub struct Interval {
```

```

18     pub lo: f64,
19     pub hi: f64,
20     pub is_int: bool,
21 }
22
23 impl PartialEq for Interval {
24     fn eq(&self, other: &Self) -> bool {
25         self.lo == other.lo && self.hi == other.hi
26     }
27 }

```

- `lo`, `hi` : les bornes, fermées, en `f64`. `f64::INFINITY` et `f64::NEG_INFINITY` codent l'absence de borne.
- `is_int` : « toute valeur concrète est entière ». Vrai veut dire *prouvé* entier ; faux veut dire « peut contenir des non-entiers ». Le bit n'ajoute aucune valeur à la concrétisation, il en retire (les non-entiers) : c'est une annotation de précision.
- `PartialEq` est écrit à la main (l. 23–27) pour *ignorer* `is_int` : deux intervalles de mêmes bornes sont égaux. Le point fixe converge ainsi quelle que soit l'évolution du bit.

Invariant 9.2: Sens du bit `is_int`

Si `is_int` est vrai, toute valeur concrète représentée est un entier. Toute opération qui produit un intervalle ne met `is_int` à vrai que si elle peut le prouver. Réciproquement, un `is_int` faux n'est jamais faux « à tort » : il n'enlève rien.

Les constructeurs élémentaires :

Extrait 9.12 – `src/domains/impls/interval.rs`, l. 29–67 — constructeurs et tests élémentaires

```

29 impl Interval {
30     pub fn point(v: f64) -> Self {
31         Interval {
32             lo: v,
33             hi: v,
34             is_int: v.is_finite() && v.fract() == 0.0,
35         }
36     }
37
38     pub fn top() -> Self {
39         Interval {
40             lo: f64::NEG_INFINITY,
41             hi: f64::INFINITY,
42             is_int: false,
43         }
44     }
45
46     /// Empty interval represents  $\perp$  for the numeric sub-lattice. `is_int` is
47     /// vacuously true (the empty set contains no non-integer) and never read:
48     /// every combinator short-circuits on `is_bottom` before touching it.
49     pub fn bottom() -> Self {
50         Interval {
51             lo: f64::INFINITY,
52             hi: f64::NEG_INFINITY,
53             is_int: true,
54         }
55     }
56
57     pub fn is_bottom(&self) -> bool {
58         self.lo > self.hi
59     }

```

```

60
61     pub fn is_point(&self) -> bool {
62         !self.is_bottom() && self.lo == self.hi
63     }
64
65     pub fn is_top(&self) -> bool {
66         self.lo == f64::NEG_INFINITY && self.hi == f64::INFINITY
67     }

```

- `point(v)` est l'abstraction d'une valeur unique. Son bit vaut vrai si v est fini et sans partie fractionnaire : `point(3.0)` est entier, `point(1.5)` ne l'est pas, `point(f64::INFINITY)` non plus.
- `bottom()` est le $\perp$ canonique $[+\infty, -\infty]$, avec `is_int` vrai par vacuité (l'ensemble vide ne contient aucun non-entier).
- `is_bottom` teste $\ell > h$, et non l'égalité au $\perp$ canonique : c'est le seul test fiable (voir le piège plus bas).

9.4.3 Ordre, join, meet

Extrait 9.13 – `src/domains/impls/interval.rs`, l. 312–328 — l'ordre des intervalles : l'inclusion des bornes

```

312 /// `[a,b] ≤ [c,d]` iff `[a,b] ⊆ [c,d]` (i.e.  $c \leq a$  &&  $b \leq d$ ). Bounds only – `is_int`
313 /// is a precision annotation and does not participate (see the struct docs).
314 impl PartialOrd for Interval {
315     fn partial_cmp(&self, other: &Self) -> Option<Ordering> {
316         if self == other {
317             return Some(Ordering::Equal);
318         }
319         let self_in_other = other.lo <= self.lo && self.hi <= other.hi;
320         let other_in_self = self.lo <= other.lo && other.hi <= self.hi;
321         match (self_in_other, other_in_self) {
322             (true, false) => Some(Ordering::Less),
323             (false, true) => Some(Ordering::Greater),
324             (true, true) => Some(Ordering::Equal),
325             (false, false) => None,
326         }
327     }
328 }

```

L'ordre est l'inclusion : $[a, b] \sqsubseteq [c, d]$ ssi $c \leq a$ et $b \leq d$. Le join est l'enveloppe convexe (hull, présentée au chapitre 3, liste 3.7) : les bornes extrêmes, entier si les deux opérandes le sont, $\perp$ neutre. L'implémentation du trait relie les deux et définit le meet :

Extrait 9.14 – `src/domains/impls/interval.rs`, l. 332–361 — `impl AbstractDomain for Interval` (début)

```

332 impl AbstractDomain for Interval {
333     fn bottom() -> Self {
334         Interval::bottom()
335     }
336     fn top() -> Self {
337         Interval::top()
338     }
339     fn is_bottom(&self) -> bool {
340         Interval::is_bottom(self)
341     }
342     fn join(&self, other: &Self) -> Self {
343         self.hull(other)

```

```

344 }
345 fn meet(&self, other: &Self) -> Self {
346     // Intersection: integer-valued if *either* operand is (the meet is a
347     // subset of each, so a subset of an all-integer set is all-integer).
348     let lo = self.lo.max(other.lo);
349     let hi = self.hi.min(other.hi);
350     Interval {
351         lo,
352         hi,
353         is_int: self.is_int || other.is_int,
354     } // bottom if lo > hi
355 }
356 fn widen(&self, other: &Self) -> Self {
357     Interval::widen(self, other)
358 }
359 fn widen_to(&self, other: &Self, thresholds: &[f64]) -> Self {
360     Interval::widen_to(self, other, thresholds)
361 }

```

Le meet est l'intersection des bornes. Son bit est un *ou* : si l'un des opérandes ne contient que des entiers, leur intersection non plus ; il n'y a pas de test $\perp$, car un résultat $\ell > h$ est $\perp$ par construction.

Proposition 9.3: Treillis des intervalles

Sur les intervalles non vides, *hull* est la plus petite borne supérieure et *meet* la plus grande borne inférieure pour l'ordre d'inclusion des bornes ; les deux respectent la concrétisation (conditions 3 et 6 de la définition 9.1).

Démonstration. Un intervalle $[c, d]$ majore $[a_1, b_1]$ et $[a_2, b_2]$ ssi $c \leq \min(a_1, a_2)$ et $d \geq \max(b_1, b_2)$; le plus petit est $[\min(a_1, a_2), \max(b_1, b_2)]$, c'est *hull*. Si $x \in \gamma(a_1, b_1, \iota_1)$, alors $\min(a_1, a_2) \leq x \leq \max(b_1, b_2)$; et si $\iota_1 \wedge \iota_2$, x est entier. Le raisonnement est dual pour le *meet* : x dans les deux concrétisations est entre les deux bornes basses et les deux bornes hautes, et entier dès que l'un des ι l'impose. $\square$

Figure 9.4 montre un fragment de ce treillis. Il est de hauteur infinie : la chaîne $[0, 0] \sqsubset [0, 1] \sqsubset [0, 2] \sqsubset \dots$ ne s'arrête jamais, ce qui imposera le widening (section 9.7).

Exemple 9.1: Le join perd les trous

Un handler écrit *setX(0)*, un autre *setX(10)*. Le slot vaut $[0, 0] \sqcup [10, 10] = [0, 10]$, qui contient aussi $1, \dots, 9$. Le join est pourtant la plus petite borne supérieure *dans ce treillis* : aucun intervalle plus petit ne contient 0 et 10. La perte vient du choix du domaine (convexe), pas de l'opération.

Piège

$\perp$ a plusieurs représentations. Le raffinement $[10, 20]$ sous $x < 5$ donne $[10, 4]$ (section 9.8), un $\perp$ non canonique. Les combinateurs (*hull*, *widen*, *add*...) testent *is_bottom()* d'abord et le traitent correctement. Mais *PartialEq* et *PartialOrd* comparent les bornes brutes : $[10, 4] == \text{Interval}::\text{bottom}()$ est faux, et *partial_cmp*($[10, 4]$, $[0, 3]$) rend *None* (aucune des deux inclusions de bornes n'est vraie, liste 9.13), alors qu'un $\perp$ devrait être *Less* que tout. Le chapitre 3 a relevé la conséquence pratique (une *worklist* qui revisite un bloc mort). Tout code nouveau qui compare des intervalles doit tester *is_bottom()* avant l'ordre.

L'effet sur le point fixe est précis. *StateStore::leq* (*src/domains/stores/state_store.rs*, l. 70–77) délègue à *leq_pointwise* (*src/domains/stores/mod.rs*, l. 23–25), qui compare par

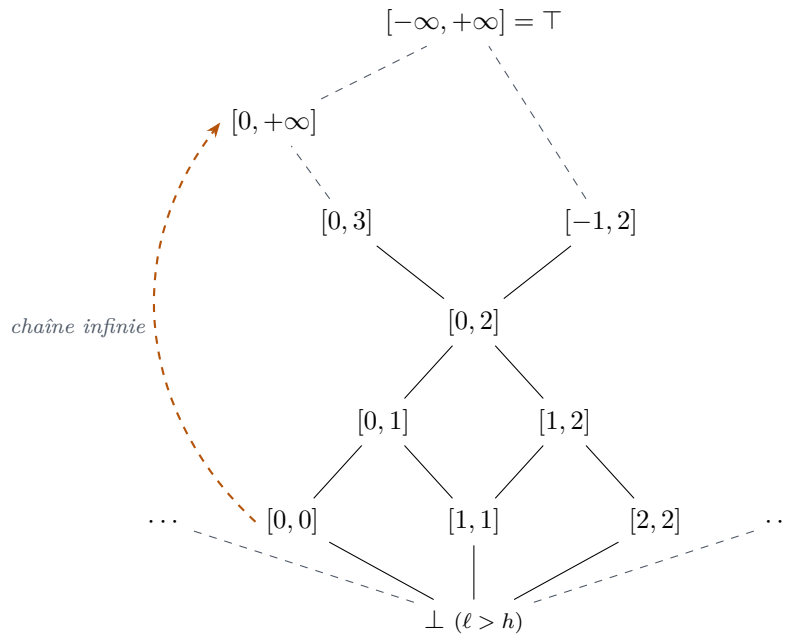


FIGURE 9.4 – Un fragment du treillis des intervalles. Les points sont les atomes ; le join de $[0, 0]$ et $[2, 2]$ est $[0, 2]$, qui contient aussi 1 (convexité). Toute chaîne $[0, 0] \sqsubset [0, 1] \sqsubset \dots$ monte sans fin vers $[0, +\infty]$: hauteur infinie.

`partial_cmp` brut ; `changed_labels` (`src/domains/stores/state_store.rs`, l. 87–95) compare de même par `!=` brut. Si un slot contient un $\perp$ non canonique dont la forme diffère d'un tour à l'autre (deux narrowings à des littéraux différents, par exemple), le test de convergence (`new_state.leq(&state)`, `src/engine/fixpoint.rs`, l. 495–495) peut rendre `false` alors que les deux valeurs dénotent $\emptyset$: la boucle externe continue un tour de plus qu'il ne faudrait, et `changed_labels` peut marquer le slot « changé » à tort, avançant son entrée dans `widen_trace`. Ce n'est ni un défaut de soundness (un tour de plus ne perd aucune valeur) ni un risque de non-terminaison : le garde-fou des 100 itérations (`src/engine/fixpoint.rs`, l. 499–512) force de toute façon un `widen` final. L'impact observable est une perte de précision possible (un widening avancé d'un tour) sur un slot qui n'était de toute façon plus atteignable.

Piège

Parce que `is_int` est hors de l'égalité, un tour de point fixe qui n'ajouterait qu'un non-entier à l'intérieur des bornes serait jugé convergé, et l'état retenu garderait un `is_int` vrai devenu faux. Le dossier de notes a tenté de reproduire ce scénario (un compteur par pas de 0,5) sans y parvenir : le bit `y` est bien faux à la convergence. Le risque reste théorique, mais c'est la seule brèche de l'invariant 9.2. Au commit `e67b10a`, ni une reproduction ni une preuve d'impossibilité n'existent : le scénario exige que les bornes se stabilisent *avant* qu'un opérande non entier n'entre en jeu à bornes inchangées, ce qu'aucun des exemples essayés (croissance directe, modulo) ne produit, faute d'avoir su construire un programme où l'un se produit sans l'autre.

9.5 Arithmétique d’intervalles

9.5.1 Addition, soustraction, multiplication, négation

Chaque écriture d’état d’un compteur passe par une opération arithmétique : $\text{count} + 1$, $\text{total} - \text{price}$, $n * 2$. Le domaine doit calculer, à partir des intervalles des opérandes, un intervalle qui contient tous les résultats possibles.

Extrait 9.15 – `src/domains/impls/interval.rs`, l. 148–187 — addition, soustraction, multiplication

```

148     pub fn add(&self, other: &Self) -> Self {
149         if self.is_bottom() || other.is_bottom() {
150             return Interval::bottom();
151         }
152         Interval {
153             lo: self.lo + other.lo,
154             hi: self.hi + other.hi,
155             is_int: self.is_int && other.is_int,
156         }
157     }
158
159     pub fn sub(&self, other: &Self) -> Self {
160         if self.is_bottom() || other.is_bottom() {
161             return Interval::bottom();
162         }
163         Interval {
164             lo: self.lo - other.hi,
165             hi: self.hi - other.lo,
166             is_int: self.is_int && other.is_int,
167         }
168     }
169
170     pub fn mul(&self, other: &Self) -> Self {
171         if self.is_bottom() || other.is_bottom() {
172             return Interval::bottom();
173         }
174         let products = [
175             self.lo * other.lo,
176             self.lo * other.hi,
177             self.hi * other.lo,
178             self.hi * other.hi,
179         ];
180         let lo = products.iter().cloned().fold(f64::INFINITY, f64::min);
181         let hi = products.iter().cloned().fold(f64::NEG_INFINITY, f64::max);
182         Interval {
183             lo,
184             hi,
185             is_int: self.is_int && other.is_int,
186         }
187     }

```

Extrait 9.16 – `src/domains/impls/interval.rs`, l. 189–198 — négation

```

189     pub fn neg(&self) -> Self {
190         if self.is_bottom() {
191             return Interval::bottom();
192         }
193         Interval {
194             lo: -self.hi,
195             hi: -self.lo,
196             is_int: self.is_int,

```



```

197     }
198 }
```

Ce sont les formules classiques de l'arithmétique d'intervalles :

$$\begin{aligned}
[a, b] + [c, d] &= [a + c, b + d], & [a, b] - [c, d] &= [a - d, b - c], \\
[a, b] \times [c, d] &= [\min P, \max P], \quad P = \{ac, ad, bc, bd\}, & -[a, b] &= [-b, -a].
\end{aligned}$$

Un opérande $\perp$ rend $\perp$: un chemin mort reste mort. Le bit entier est un *et* : la somme, la différence ou le produit de deux entiers est entier, mais un seul non-entier suffit à perdre la garantie (`test integrality_lost_through_float_arithmetic`, `src/domains/impls/interval.rs`, l. 608–613).

Théorème 9.1: Soundness de l'arithmétique d'intervalles

Pour $x \in \gamma(I)$ et $y \in \gamma(J)$ finis, $x + y \in \gamma(I + J)$, $x - y \in \gamma(I - J)$, $-x \in \gamma(-I)$, et, si aucun produit de coins n'est indéterminé ($0 \times \pm\infty$), $xy \in \gamma(I \times J)$.

Démonstration. Addition : $a \leq x \leq b$ et $c \leq y \leq d$ donnent $a + c \leq x + y \leq b + d$. Soustraction : $-d \leq -y \leq -c$, puis addition. Négation : multiplier par -1 renverse les inégalités. Multiplication : pour y fixé, $x \mapsto xy$ est affine, donc ses extremums sur $[a, b]$ sont en a ou b ; puis $y \mapsto ay$ et $y \mapsto by$ sont affines, extremums en c ou d : les extremums de xy sur le pavé sont parmi les quatre coins. Le bit : somme, différence, produit et opposé d'entiers sont entiers. $\square$

Le calcul se fait en `f64`, avec l'arrondi au plus proche. Comme JavaScript calcule les valeurs concrètes avec la même arithmétique et que l'arrondi est monotone ($u \leq v \Rightarrow \text{rd}(u) \leq \text{rd}(v)$), les bornes calculées encadrent encore les résultats concrets. Cet argument n'est écrit nulle part dans le code ; il est nécessaire, et il suffit.

La restriction « aucun produit indéterminé » du théorème n'est pas une précaution de style : le code la viole dans un cas, étudié à la section 9.10.

9.5.2 NaN, $\pm\infty$ et la division

Puisque NaN n'est dans aucun intervalle, une opération qui peut le produire doit rendre autre chose qu'un intervalle. La division en est l'exemple type : `x / 0` vaut `±Infinity`, et `0 / 0` vaut NaN. Le code ne cherche pas à distinguer les cas : `BinOp::Div` rend toujours `⊤` au niveau `StateValue` (liste 9.18). `docs/limitations.md` le documente.

La place de $\pm\infty$ est plus ambiguë. Une borne infinie signifie « non borné », mais un littéral `1e999` vaut concrètement `Infinity`, et `point(f64::INFINITY)` produit l'intervalle $[+\infty, +\infty]$. Le code traite de fait les infinis comme des *absences de borne*, pas comme des valeurs : les formules ci-dessus donneraient $+\infty - (+\infty) = \text{NaN}$ comme borne si les deux opérandes étaient le point $+\infty$. Aucun programme React raisonnable ne fait d'arithmétique sur `Infinity`, et l'écart n'a pas été observé sur un diagnostic ; il est signalé à la section 9.10.

9.5.3 Reste et puissance (commit 548f922)

Jusqu'au commit 548f922 (issue #73), `%` et `**` n'avaient pas de variante propre dans l'IR et tombaient dans l'opérateur opaque. Ils sont maintenant modélisés :

Extrait 9.17 – `src/domains/impls/interval.rs`, l. 200–253 — reste et puissance

```

200     /// `self % other` with JS semantics: the result takes the dividend's sign
201     /// and is strictly smaller in magnitude than both the divisor and the
```

```

202  /// dividend. `None` when the divisor may be zero – the result may then be
203  /// `NaN`, which no interval holds.
204  pub fn rem(&self, other: &Self) -> Option<Self> {
205      if self.is_bottom() || other.is_bottom() {
206          return Some(Interval::bottom());
207      }
208      if other.lo <= 0.0 && other.hi >= 0.0 {
209          return None;
210      }
211      let is_int = self.is_int && other.is_int;
212      let magnitude = other.lo.abs().max(other.hi.abs());
213      // Strictly below the divisor's magnitude: one less on integers, the
214      // closed bound otherwise.
215      let bound = if is_int { magnitude - 1.0 } else { magnitude };
216      Some(Interval {
217          lo: if self.lo >= 0.0 {
218              0.0
219          } else {
220              self.lo.max(-bound)
221          },
222          hi: if self.hi <= 0.0 {
223              0.0
224          } else {
225              self.hi.min(bound)
226          },
227          is_int,
228      })
229  }
230
231  /// `self ** other`. On the non-negative quadrant the function is monotone
232  /// in each argument, so the extremes sit at the corners of the operand
233  /// box. `None` outside it: a negative base with a fractional exponent is
234  /// `NaN`, and a negative exponent of zero is infinite.
235  pub fn pow(&self, other: &Self) -> Option<Self> {
236      if self.is_bottom() || other.is_bottom() {
237          return Some(Interval::bottom());
238      }
239      if self.lo < 0.0 || other.lo < 0.0 {
240          return None;
241      }
242      let powers = [
243          self.lo.powf(other.lo),
244          self.lo.powf(other.hi),
245          self.hi.powf(other.lo),
246          self.hi.powf(other.hi),
247      ];
248      Some(Interval {
249          lo: powers.iter().cloned().fold(f64::INFINITY, f64::min),
250          hi: powers.iter().cloned().fold(f64::NEG_INFINITY, f64::max),
251          is_int: self.is_int && other.is_int,
252      })
253  }

```

Les deux fonctions rendent une `Option` : `None` signifie « le résultat peut être NaN », et l'appelant le transforme en $\top$.

Le reste. En JavaScript, $x \% m$ a le signe du dividende x , et sa valeur absolue est strictement inférieure à $|m|$ et inférieure ou égale à $|x|$. Le code en tire :

- si le diviseur peut être nul ($c \leq 0 \leq d$), `None` ;

- sinon, $\mu = \max(|c|, |d|)$, la plus grande valeur absolue possible du diviseur, et la borne $\beta = \mu - 1$ sur des entiers (strictement sous μ), $\beta = \mu$ sinon (borne fermée, donc sur-approximation d'une borne ouverte);
- la borne basse est 0 si le dividende est positif, sinon $\max(a, -\beta)$; la borne haute est 0 si le dividende est négatif, sinon $\min(b, \beta)$.

Proposition 9.4: Soundness du reste

Si $0 \notin [c, d]$, pour tous $x \in \gamma([a, b])$, $m \in \gamma([c, d])$ finis, $x \% m \in \gamma(\text{rem}([a, b], [c, d]))$.

Démonstration. Soit $r = x \% m$. Si $x \geq 0$, $0 \leq r \leq x \leq b$ et $r < |m| \leq \mu$; sur des entiers $r \leq \mu - 1$. Donc $r \leq \min(b, \beta)$ et $r \geq 0 \geq$ la borne basse (qui vaut 0 ou $\max(a, -\beta) \leq 0$). Le cas $x \leq 0$ est symétrique. Si $a \geq 0$, tout x est positif et $r \geq 0$: la borne basse 0 est correcte; si $b \leq 0$, la borne haute 0 l'est de même. Le bit : le reste de deux entiers est entier. $\square$

Exemple 9.2: Un compteur modulo

`setI((i + 1) % 3)` à partir de `i = 0`. Au premier tour, $[0, 0] + [1, 1] = [1, 1]$ et $\text{rem}([1, 1], [3, 3]) = [0, \min(1, 2)] = [0, 1]$; le slot vaut $[0, 0] \sqcup [0, 1] = [0, 1]$. Au deuxième, $[1, 2] \% [3, 3] = [0, 2]$; au troisième, $[1, 3] \% [3, 3] = [0, 2]$: point fixe, sans widening. Avant le commit 548f922, le résultat était $\top$.

La puissance. Sur le quadrant $x \geq 0, y \geq 0$, la fonction x^y est croissante en x pour y fixé, et monotone en y pour x fixé (croissante si $x > 1$, constante si $x = 1$, décroissante si $x < 1$). Ses extremums sur un pavé sont donc aux coins, comme pour le produit. Hors du quadrant, un exposant fractionnaire d'une base négative donne NaN, un exposant négatif de zéro donne Infinity : None. On note que $0^0 = 1$ en JavaScript comme pour `f64::powf`.

Décision : commit 548f922 — une variante par opérateur

Le commit qui modélise `%` et `**` applique une règle prise avec #74 et #75 : chaque opérateur du langage a sa variante de `BinOp`, jamais un opérateur source porté par `Unknown`. Avec `Mod`, `Pow`, `In` et `InstanceOf`, `BinOp::Unknown` n'a plus de producteur et disparaît; `lower_binop` devient exhaustif, si bien qu'un opérateur nouveau du langage ne peut plus tomber silencieusement dans l'opaque. L'alternative (garder un joker `_` vers $\top$) était sound mais masquait les oublis. Le corpus, à 1494 diagnostics avant le commit, n'a pas bougé (`docs/corpus-baseline.json` n'est pas modifié par ce commit).

9.5.4 La table des opérateurs binaires

Les intervalles ne sont jamais utilisés seuls : l'évaluateur manipule des `StateValue`, et il décide, opérateur par opérateur, s'il peut se ramener aux intervalles.

Extrait 9.18 – `src/domains/transfer/state_value.rs`, l. 752–811 — la table des opérateurs binaires

```
752 fn eval_binop(op: &BinOp, lhs: StateValue, rhs: StateValue) -> StateValue {
753     match op {
754         BinOp::Add => {
755             if let (Some(a), Some(b)) = (as_arith(&lhs), as_arith(&rhs)) {
756                 return StateValue::number(a.add(&b));
757             }
758             match (as_str_only(&lhs), as_str_only(&rhs)) {
759                 (Some(StrConst::Set(a)), Some(StrConst::Set(b))) => {
760                     let product: BTreeSet<String> = a
761                         .iter()
762                         .flat_map(|s1| b.iter().map(move |s2| format!("{s1}{s2}")))
```

```

763         .collect();
764         StateValue::str_set(product)
765     }
766     (Some(_), Some(_)) => StateValue::str_top(),
767     _ => StateValue::top(),
768 }
769 }
770 BinOp::Sub => match (as_arith(&lhs), as_arith(&rhs)) {
771     (Some(a), Some(b)) => StateValue::number(a.sub(&b)),
772     _ => StateValue::top(),
773 },
774 BinOp::Mul => match (as_arith(&lhs), as_arith(&rhs)) {
775     (Some(a), Some(b)) => StateValue::number(a.mul(&b)),
776     _ => StateValue::top(),
777 },
778 BinOp::Div => StateValue::top(),
779 // `%` and `**` are exact on the numeric operands the interval domain
780 // can follow (`Interval::rem`, `Interval::pow`); where the result may
781 // be `NaN`, which no interval holds, they are  $\top$  like `/`.
782 BinOp::Mod => match (as_arith(&lhs), as_arith(&rhs)) {
783     (Some(a), Some(b)) => a
784         .rem(&b)
785         .map(StateValue::number)
786         .unwrap_or_else(StateValue::top),
787     _ => StateValue::top(),
788 },
789 BinOp::Pow => match (as_arith(&lhs), as_arith(&rhs)) {
790     (Some(a), Some(b)) => a
791         .pow(&b)
792         .map(StateValue::number)
793         .unwrap_or_else(StateValue::top),
794     _ => StateValue::top(),
795 },
796 BinOp::And | BinOp::Or => StateValue::top(),
797 // Always a boolean, whatever the operands: that alone keeps `num` and
798 // `str` at  $\perp$  so a guard over the result still narrows.
799 BinOp::Eq
800 | BinOp::Neq
801 | BinOp::Lt
802 | BinOp::Gt
803 | BinOp::Leq
804 | BinOp::Geq
805 | BinOp::In
806 | BinOp::InstanceOf => StateValue::boolean(BoolVal::Top),
807 BinOp::BitAnd | BinOp::BitOr | BinOp::BitXor | BinOp::Shl | BinOp::Shr | BinOp::UShr => {
808     eval_bitwise(op, &lhs, &rhs)
809 }
810 }
811 }

```

Le passage aux intervalles se fait par `as_arith` (`src/domains/transfer/state_value.rs`, l. 708–732), qui rend un intervalle si les seules sortes possibles de la valeur sont « nombre » et « null », puisque `ToNumber(null)` vaut 0; le chapitre 10 en détaille la logique. Il rend `None` dès qu’un `undefined`, un booléen, une chaîne, une référence, un setter ou la sorte résiduelle `other` est possible. La lecture de la table, ligne par ligne :

- `+` : numérique si les deux opérandes le sont ; concaténation exacte si les deux sont des chaînes pures ; « une chaîne » si ce sont des chaînes mal connues ; $\top$ sinon. `1 + "a"` donne donc $\top$ et non une chaîne : sûr, imprécis.
- `-` et `*` : numériques ou $\top$; `/` : toujours $\top$; `%` et `**` : l’intervalle, ou $\top$ si `None`.

- `&&` et `||` : $\top$. Leur valeur est rarement utilisée, car le lowering les transforme en diamant de CFG (chapitre 7).
- comparaisons, `in`, `instanceof` : un booléen inconnu (section 9.2).
- bit à bit : `eval_bitwise`, section suivante.

9.6 Les opérateurs bit à bit

Le code React utilise les opérateurs bit à bit pour des drapeaux (`flags & MASK`), des hachages (`hash >>> 0`) ou des index (`i << 1`). Jusqu'à #74, ils valaient $\top$; ils sont maintenant modélisés à partir d'une observation simple.

Remarque

JavaScript convertit les deux opérandes d'un opérateur bit à bit par `ToInt32` (`ToUint32` pour l'opérande gauche de `>>>`) : troncature vers zéro, puis réduction modulo 2^{32} dans $[-2^{31}, 2^{31} - 1]$. Le résultat est donc *toujours* un entier de cette plage, quels que soient les opérandes (chaînes, objets, `undefined`...), et un entier de $[0, 2^{32} - 1]$ pour `>>>`.

Extrait 9.19 – `src/domains/transfer/state_value.rs`, l. 813–844 — plages `int32/uint32` et utilitaires

```
813 const I32_MIN: f64 = i32::MIN as f64;
814 const I32_MAX: f64 = i32::MAX as f64;
815 const U32_MAX: f64 = u32::MAX as f64;
816
817 /// Every value `x` such that `x = ToInt32(_)` – the range every bitwise and
818 /// shift operator but `>>>` lands in.
819 fn int32_range() -> Interval {
820     Interval {
821         lo: I32_MIN,
822         hi: I32_MAX,
823         is_int: true,
824     }
825 }
826
827 /// The `>>>` range: unsigned, so a uint32.
828 fn uint32_range() -> Interval {
829     Interval {
830         lo: 0.0,
831         hi: U32_MAX,
832         is_int: true,
833     }
834 }
835
836 /// The constant an operand denotes, when it denotes exactly one number.
837 fn const_of(iv: &Option<Interval>) -> Option<f64> {
838     iv.filter(|i| !i.is_bottom() && i.is_point()).map(|i| i.lo)
839 }
840
841 /// A shift distance is taken mod 32, so only its low five bits matter.
842 fn shift_amount(k: f64) -> Option<u32> {
843     (k.is_finite() && k.abs() < 1e9).then(|_| (k as i64).rem_euclid(32) as u32)
844 }
```

Extrait 9.20 – `src/domains/transfer/state_value.rs`, l. 846–886 — `eval_bitwise` : plancher, `&`, `|` et `^`

```

846 /// Bitwise and shift operators.
847 ///
848 /// JS coerces *both* operands to int32 (uint32 for `>>>`) whatever they are, so
849 /// the result is always a number in a known range – which no operand check is
850 /// needed to claim. That floor alone beats  $\top$ : it puts the string, boolean and
851 /// reference components at  $\perp$ , so a downstream guard can still narrow on the
852 /// result. Constant operands tighten it further, exactly where React code
853 /// actually uses these (`flags & MASK`, `hash >>> 0`, `i << 1`).
854 fn eval_bitwise(op: &BinOp, lhs: &StateValue, rhs: &StateValue) -> StateValue {
855     // A  $\perp$  operand is an unreachable path: the result stays  $\perp$  rather than
856     // widening back to a live range.
857     if lhs.is_bottom_value() || rhs.is_bottom_value() {
858         return StateValue::bottom();
859     }
860     let l = as_arith(lhs);
861     let r = as_arith(rhs);
862     let in_i32 = |i: &Interval| !i.is_bottom() && i.lo >= I32_MIN && i.hi <= I32_MAX;
863
864     let refined = match op {
865         // `x & mask` can only keep bits the mask has: with a non-negative
866         // constant mask the result is in `[0, mask]`. Either side may be it.
867         BinOp::BitAnd => const_of(&r)
868             .or_else(|| const_of(&l))
869             .filter(|m| (0.0..=I32_MAX).contains(m))
870             .map(|m| Interval {
871                 lo: 0.0,
872                 hi: m,
873                 is_int: true,
874             }),
875         // `x | y` and `x ^ y` on non-negative operands cannot set a bit above
876         // the highest one either operand could have.
877         BinOp::BitOr | BinOp::BitXor => match (&l, &r) {
878             (Some(a), Some(b)) if in_i32(a) && in_i32(b) && a.lo >= 0.0 && b.lo >= 0.0 => {
879                 Some(Interval {
880                     lo: 0.0,
881                     hi: all_ones_above(a.hi.max(b.hi)),
882                     is_int: true,
883                 })
884             }
885             _ => None,
886         },

```

Extrait 9.21 – src/domains/transfer/state_value.rs, l. 887–939 — eval_bitwise : décalages, et all_ones_above

```

887     // `x << k` with a constant shift: multiply the bounds, and keep the
888     // result only when neither can wrap past int32 (a wrap makes the
889     // operation non-monotone, so the bounds would no longer bound it).
890     BinOp::Shl => match (&l, const_of(&r).and_then(shift_amount)) {
891         (Some(a), Some(k)) if in_i32(a) => {
892             let f = (1u64 << k) as f64;
893             let (lo, hi) = (a.lo * f, a.hi * f);
894             (lo >= I32_MIN && hi <= I32_MAX).then_some(Interval {
895                 lo,
896                 hi,
897                 is_int: true,
898             })
899         }
900         _ => None,
901     },
902     // `x >> k` is a floor-division by  $2^k$  on int32 – monotone, so the

```

```

903 // bounds carry straight through.
904 BinOp::Shr => match (&l, const_of(&r).and_then(shift_amount)) {
905     (Some(a), Some(k)) if in_i32(a) => {
906         let f = (1u64 << k) as f64;
907         Some(Interval {
908             lo: (a.lo / f).floor(),
909             hi: (a.hi / f).floor(),
910             is_int: true,
911         })
912     }
913     _ => None,
914 },
915 // `x >> k` is unsigned: whatever `x` is, the result fits in the low
916 // `32 - k` bits.
917 BinOp::UShr => const_of(&r).and_then(shift_amount).map(|k| Interval {
918     lo: 0.0,
919     hi: (u32::MAX >> k) as f64,
920     is_int: true,
921 }),
922 _ => None,
923 };
924
925 StateValue::number(refined.unwrap_or_else(|| match op {
926     BinOp::UShr => uint32_range(),
927     _ => int32_range(),
928 })))
929 }
930
931 /// The smallest `2^n - 1` that is at least `m` — the largest value any bitwise
932 /// OR/XOR of non-negative operands bounded by `m` can produce.
933 fn all_ones_above(m: f64) -> f64 {
934     if m <= 0.0 {
935         return 0.0;
936     }
937     let bits = (m.max(1.0).log2().floor() as u32 + 1).min(31);
938     ((1u64 << bits) - 1) as f64
939 }

```

Le principe : un résultat *plancher* (`int32_range` ou `uint32_range`, l. 925–928), qui ne demande aucune hypothèse sur les opérandes, et un résultat *raffiné* quand un opérande est constant ou borné. Rendre la plage `int32` plutôt que $\top$ est déjà un gain : les composantes chaîne, booléen et référence restent à $\perp$, et une garde sur le résultat reste raffnable. Cas par cas :

- $x \& m$, avec m une constante de $[0, 2^{31} - 1]$ (à gauche ou à droite) : le résultat n’a que des bits de m , donc est dans $[0, m]$.
- $x \mid y$, $x \wedge y$, les deux opérandes positifs dans `int32` : le résultat ne peut pas avoir de bit au-dessus du plus haut bit possible des opérandes, donc il est dans $[0, 2^n - 1]$ où $2^n - 1$ est le plus petit nombre de la forme « que des 1 » au moins égal au max (`all_ones_above`).
- $x \ll k$, k constant : multiplication des bornes par 2^k , gardée seulement si le résultat ne déborde pas d’`int32` (au-delà, la réduction modulo 2^{32} rend l’opération non monotone).
- $x \gg k$: division entière par 2^k arrondie vers le bas, monotone.
- $x \ggg k$: le résultat tient dans les $32 - k$ bits bas, quel que soit x .
- Un opérande $\perp$ rend $\perp$ (l. 857–859).

La négation bit à bit $\sim x$, qui vaut $-(\text{ToInt32}(x) + 1)$, est traitée à part dans les opérateurs unaires, avec soin :

Extrait 9.22 – `src/domains/transfer/state_value.rs`, l. 978–993 — la négation bit à bit

```

978 // `~x` is `-(ToInt32(x) + 1)` - decreasing, so the bounds swap. Outside
979 // int32 the coercion wraps and monotonicity is gone; the int32 range is
980 // still guaranteed.
981 UnaryOp::BitNot => {
982   if val.is_bottom_value() {
983     return StateValue::bottom();
984   }
985   let refined = as_arith(&val).filter(|i| !i.is_bottom()).and_then(|i| {
986     (i.lo >= I32_MIN && i.hi <= I32_MAX).then_some(Interval {
987       lo: -i.hi.ceil() - 1.0,
988       hi: -i.lo.floor() - 1.0,
989       is_int: true,
990     })
991   });
992   StateValue::number(refined.unwrap_or_else(int32_range))
993 }

```

La fonction est décroissante, les bornes s'échangent, et les arrondis `ceil` sur la borne haute et `floor` sur la borne basse encadrent la troncature de `ToInt32` sur des non-entiers : pour $x = 1,5$, $\text{ToInt32}(x) = 1$ et $\sim x = -2$, dans $[-\lceil 1,5 \rceil - 1, -\lfloor 1,5 \rfloor - 1] = [-3, -2]$.

Exemple 9.3: Calculs bit à bit

$[0, 100] \& 15 = [0, 15]$. $[0, 5] \mid [0, 9]$: le max est 9, $\lfloor \log_2 9 \rfloor + 1 = 4$ bits, résultat $[0, 15]$. $[3, 3] \ll 2 = [12, 12]$. $[0, 1000] \gg 2 = [0, 250]$. $x \ggg 28 \in [0, 15]$ quel que soit x . $\sim[-3, 4] = [-5, 2]$.

9.6.1 Un faux négatif : les décalages sur des non-entiers

Les bras `<<` et `>>` supposent implicitement que les bornes de l'opérande sont des entiers : ils multiplient ou divisent `a.lo` et `a.hi` tels quels. Or `ToInt32` tronque d'abord. Pour $x = 1,5$, JavaScript calcule $1,5 \ll 1 = \text{ToInt32}(1,5) \ll 1 = 1 \ll 1 = 2$, alors que le code calcule $[1,5 \times 2, 1,5 \times 2] = [3, 3]$, qui ne contient pas 2. De même pour $x = -1,5$: $-1,5 \gg 0 = \text{ToInt32}(-1,5) = -1$, alors que le code calcule $[\lfloor -1,5 \rfloor, \lfloor -1,5 \rfloor] = [-2, -2]$. Ce sont des sous-approximations, la direction interdite.

Le défaut est observable de bout en bout :

Extrait 9.23 – Décalages sur un état non entier (`/tmp/ch09/shift2.tsx`, élagué de sa ligne d'import)

```

1 export function ShlFloat() {
2   const [x] = useState(1.5);
3   const [count, setCount] = useState(0);
4   useEffect(() => {
5     const y = x << 1;
6     setCount(count + 3 - y);
7   }, [count]);
8   return <div>{count}</div>;
9 }
10
11 export function ShrFloat() {
12   const [x] = useState(-1.5);
13   const [count, setCount] = useState(0);
14   useEffect(() => {
15     const y = x >> 0;
16     setCount(count + y + 2);
17   }, [count]);
18   return <div>{count}</div>;
19 }
20

```



```

21 export function ShlInt() {
22   const [x] = useState(1);
23   const [count, setCount] = useState(0);
24   useEffect(() => {
25     const y = x << 1;
26     setCount(count + 3 - y);
27   }, [count]);
28   return <div>{count}</div>;
29 }

```

Concrètement, les trois composants bouclent : dans `ShlFloat`, `y` vaut 2 et l'effet écrit `count + 1` ; dans `ShrFloat`, `y` vaut -1 et l'effet écrit encore `count + 1` ; `ShlInt` est le témoin entier. L'analyseur :

```

$ reactant check /tmp/ch09/shift2.tsx --show-clean
ShlFloat (3 hooks) /tmp/ch09/shift2.tsx ✓
ShlInt (3 hooks) /tmp/ch09/shift2.tsx
  warn  infinite-loop [hook:1] (line 26:2) this effect keeps pushing state `count` (its deps do not provably gate
    ↳ it, so the effect can re-run every render) to new values on every run. Potential infinite render loop
    (2 trace step(s), rerun with --trace)
ShrFloat (3 hooks) /tmp/ch09/shift2.tsx ✓

! 1 warning(s) across 1 file(s).

```

Pour l'analyseur, `y` vaut $[3, 3]$ dans `ShlFloat` et $[-2, -2]$ dans `ShrFloat` : l'effet écrit `count + 0`, la valeur ne change pas, le point fixe converge sans widening, et les deux boucles infinies sont déclarées sans défaut. C'est un faux négatif ; aucune issue ne le suit au commit étudié. La correction naturelle, centrale et d'une ligne par bras, consiste à tronquer les bornes avant de décaler (`trunc` est monotone, et coïncide avec `ToInt32` sur la plage `int32`), ou à exiger `is_int` ; c'est l'exercice 9.8.

9.7 Widening : du saut à l'infini aux seuils

9.7.1 La chronologie d'un compteur

Le treillis des intervalles est de hauteur infinie : sans précaution, le point fixe de `Unbounded` ne termine pas. Suivons la boucle externe du moteur (chapitre 13) sur les deux compteurs du liste 9.1. Le slot est amorcé à $[0, 0]$ par `useState(0)`. À chaque tour, l'effet lit la valeur courante, écrit `count + 1`, et l'écriture est ajoutée au slot par `join` (mise à jour faible). Tant que le compteur de tours `iteration` reste sous `widen_threshold` (3 par défaut, `src/engine/fixpoint.rs`, l. 51–59), le nouvel état remplace l'ancien ; à partir de 3, il est *élargi* (`src/engine/fixpoint.rs`, l. 514–529, reproduit au liste 3.18) :

```
state = state.widen_to(&new_state, &thresholds)
```

Figure 9.5 dessine les deux histoires.

Détaillons `Guarded`. Tour 1 : l'effet lit $[0, 0]$, la garde `count < 10` le laisse intact, l'écriture vaut $[1, 1]$, le slot $[0, 0] \sqcup [1, 1] = [0, 1]$. Tour 2 : lecture $[0, 1]$, écriture $[1, 2]$, slot $[0, 2]$. Tour 3 : l'état candidat est $[0, 3]$; comme `iteration` atteint 3, on calcule $[0, 2] \nabla_T [0, 3]$ avec $T = \{0, 1, 10\}$ (les littéraux du composant) : la borne haute a grandi, elle prend le plus petit seuil ≥ 3 , soit 10. Tour 4 : la lecture vaut $[0, 10]$, la garde la raffine en $[0, 9]$ (section 9.8), l'écriture vaut $[1, 10]$, le candidat $[0, 10]$ est inclus dans l'état : c'est fini. Les valeurs finales $[0, +\infty]$ et $[0, 10]$ sont épinglées par les tests `unbounded_counter_is_flagged_and_grows_to_infinity` et `guarded_counter_converges_bounded` (`tests/widening_e2e.rs`, l. 79–122). La sortie `--info` en garde la trace :

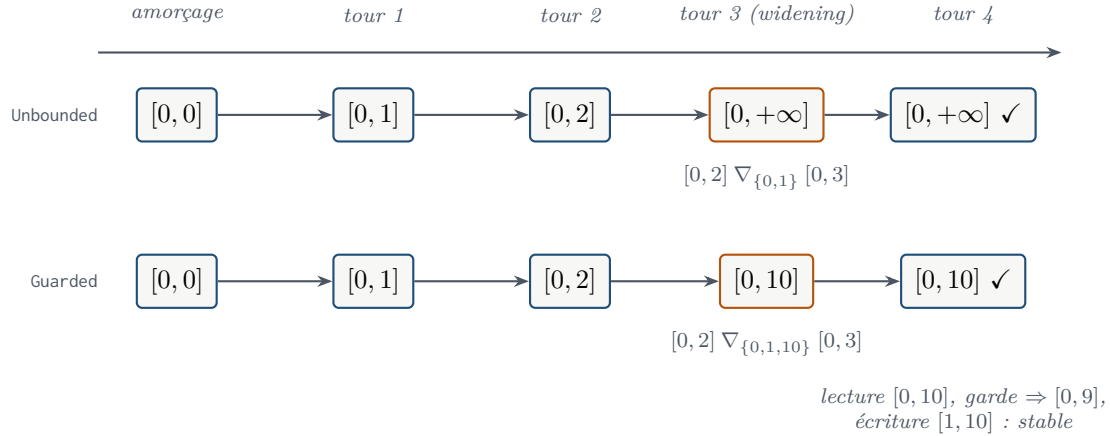


FIGURE 9.5 – Chronologie du slot count dans la boucle externe. Aux tours 1 et 2, le nouvel état remplace l’ancien ; au tour 3 (iteration = 3), il est élargi à seuils : la borne haute saute au plus petit seuil ≥ 3 , qui est $+\infty$ pour Unbounded ($T = \{0, 1\}$) et 10 pour Guarded ($T = \{0, 1, 10\}$). Au tour 4, le nouvel état est inclus dans l’ancien : post-point fixe ($\checkmark$).

```
$ reactant check /tmp/ch09/counter.tsx --info --trace
Guarded (2 hooks) /tmp/ch09/counter.tsx
  info widening-info (line 13:2) state `count` kept changing during analysis and was approximated to converge, so
    ↳ findings that depend on it may be imprecise
    ↳ state `count` is written here [hook:1] (line 13:2)
    ↳ the abstract value of state `count` kept growing and was widened at iteration 3
Unbounded (2 hooks) /tmp/ch09/counter.tsx
  warn infinite-loop [hook:0] (line 5:2) this effect keeps pushing state `count` (its deps do not provably gate it
    ↳ , so the effect can re-run every render) to new values on every run. Potential infinite render loop
    ↳ state `count` is written here [hook:1] (line 5:2)
    ↳ the abstract value of state `count` kept growing and was widened at iteration 3
  info widening-info (line 5:2) state `count` kept changing during analysis and was approximated to converge, so
    ↳ findings that depend on it may be imprecise
    ↳ state `count` is written here [hook:1] (line 5:2)
    ↳ the abstract value of state `count` kept growing and was widened at iteration 3
! 1 warning(s) across 1 file(s).
```

(Sortie élaguée des lignes `verified` : `-info` liste aussi, pour chaque composant, les règles qui ne signalent rien.) Les deux slots ont été élargis au tour 3 ; seul celui d’Unbounded est non borné, et c’est ce que lit `infinite-loop`.

9.7.2 Les deux opérateurs

Le widening simple (`src/domains/impls/interval.rs`, l. 85–106) et le widening à seuils (`src/domains/impls/interval.rs`, l. 108–146) ont été reproduits au chapitre 3 (listes 3.15 et 3.16), avec leur formule (définition 3.6). Rappelons celle du second : une borne qui n’a pas grandi est conservée ; une borne haute qui a grandi jusqu’à d devient $\min\{t \in T \cup \{+\infty\} \mid t \geq d\}$, une borne basse qui a baissé jusqu’à c devient $\max\{t \in T \cup \{-\infty\} \mid t \leq c\}$. Le code calcule ces extremums par `filter` puis `fold` à partir de $\pm\infty$, sans exiger que T soit trié.

Théorème 9.2: `widen_to` est un widening

Pour tout ensemble fini $T \subseteq \mathbb{R}$: (i) $I \sqcup J \sqsubseteq I \nabla_T J$ pour tous I, J ; (ii) pour toute suite $J_1, J_2, \dots$, la suite $I_0, I_{n+1} = I_n \nabla_T J_{n+1}$ est ultimement stationnaire.

Démonstration. (i) Si I ou J est $\perp$, le code rend l’autre, qui est le join. Sinon, la borne haute du résultat est h_I si $h_J \leq h_I$, et un $t \geq h_J$ sinon (seuil ou $+\infty$) : dans les deux cas elle est

$\geq \max(h_I, h_J)$. Symétriquement pour la borne basse. Le bit est $\iota_I \wedge \iota_J$, comme le join. (ii) La borne haute de I_n ne décroît jamais : elle est conservée ou remplacée par un $t \geq h_J > h_{I_n}$. Après son premier saut, elle appartient à $T \cup \{+\infty\}$, ensemble fini, et chaque saut ultérieur la fait croître strictement dans cet ensemble : au plus $|T| + 1$ sauts. Idem pour la borne basse. Une suite dont les deux bornes cessent de changer est stationnaire. $\square$

Le test `widen_to_is_sound_superset_of_hull` (`src/domains/impls/interval.rs`, l. 568–576) vérifie (i) sur un exemple. Le point (ii) explique pourquoi l'ensemble des seuils doit être *fini* et *fixé* avant le point fixe : des seuils qui dépendraient des valeurs calculées pourraient ne jamais s'épuiser.

Remarque

`widen_to` ne s'applique qu'à la composante `num` du produit. `StateValue::widen_to` le dit en deux lignes : seuils pour `num`, widening ordinaire (donc join, pour les composantes de hauteur finie) pour le reste.

Extrait 9.24 – `src/domains/impls/state_value.rs`, l. 670–676 — le widening à seuils du produit

```
670     fn widen_to(&self, other: &Self, thresholds: &[f64]) -> Self {
671         StateValue {
672             num: self.num.widen_to(&other.num, thresholds),
673             // Non-numeric slots have no notion of a threshold → plain widen.
674             ..self.widen(other)
675         }
676     }
```

9.7.3 D'où viennent les seuils

`collect_thresholds` (`src/engine/fixpoint.rs`, l. 1185–1206, reproduit au liste 3.17) récolte une fois par composant les littéraux numériques du rendu, des initialiseurs de `useState`, des corps d'effets et de handlers. La descente dans les expressions est la suivante :

Extrait 9.25 – `src/engine/fixpoint.rs`, l. 1212–1224 — la récolte des littéraux, fermetures comprises

```
1212 fn collect_lits_expr(expr: &Expr, out: &mut Vec<f64>) {
1213     use crate::ir::expr::Prim;
1214     match expr {
1215         Expr::Lit(Prim::Int(n)) => out.push(*n as f64),
1216         Expr::Lit(Prim::Float(f)) => out.push(*f),
1217         // `for_each_child` does not cross `FnLit`; thresholds inside closures
1218         // (event handlers, render helpers) still bound guarded growth.
1219         Expr::FnLit { body_cfg, .. } => collect_lits_cfg(body_cfg, out),
1220         _ => {}
1221     }
1222     // Structural descent (a no-op on `FnLit`, whose body was handled above).
1223     expr.for_each_child(&mut |c| collect_lits_expr(c, out));
1224 }
```

Le cas `FnLit` est notable : la descente structurelle ordinaire ne traverse pas les fermetures, mais un seuil écrit dans une fermeture (`onClick=() => n < 5 && setN(n + 1)`) doit pouvoir borner une croissance gardée. Le résultat est trié et dédoublé. Pour `Unbounded`, $T = \{0, 1\}$ (l'initialiseur et l'incrément) ; pour `Guarded`, $T = \{0, 1, 10\}$.

Piège

Tout littéral numérique devient un seuil, même sans rapport avec la croissance étudiée. Un padding: 12 dans le rendu d'un compteur non gardé ferait sauter la borne à 12 au tour 3, puis, au tour suivant, à $+\infty$ (aucun seuil ≥ 13). La précision peut changer, jamais la soundness (théorème ci-dessus), mais un seuil de plus peut retarder la convergence d'un tour, et changer le numéro d'itération qu'affiche widening-info.

9.7.4 Le même widening dans les boucles locales

La boucle externe n'est pas le seul endroit où une valeur peut croître sans fin : une boucle while dans un effet en est un autre. `analyze_cfg` applique le même opérateur, avec les mêmes seuils, sur les arcs arrière du CFG (`src/engine/cfg_analyzer.rs`, l. 97–112), à partir du troisième passage par un même en-tête (chapitre 12). Sur le composant `BoundedLocalLoop` de `tests/fixtures/widening.tsx` (`let i = 0; while (i < 5) i = i + 1; setTotal(i);`), l'en-tête de boucle voit $[0, 0]$, $[0, 1]$, $[0, 2]$, puis $[0, 2] \nabla_{\{0,1,5\}} [0, 3] = [0, 5]$; le corps, raffiné par $i < 5$, écrit $[1, 5]$, et l'en-tête est stable. La sortie de boucle, raffinée par la négation $i \geq 5$, vaut $[5, 5]$, et le slot total converge vers $[0, 5]$ (test `bounded_local_loop_is_precise`, `tests/widening_e2e.rs`, l. 125–137).

Décision : ADR-14 — seuils, signature inchangée, pas de narrowing

ADR-14 ajoute le widening à seuils « à la ASTRÉE » sur un ensemble fini récolté une fois par composant, appliqué à la boucle externe et aux arcs arrière. Deux choix de détail sont ceux du code actuel. D'abord, la signature de `AbstractDomain::widen` n'est pas modifiée (la faire porter des seuils aurait fait fuir un paramètre inutile dans tous les domaines et les stores) : on ajoute `widen_to`, avec une implémentation par défaut qui ignore les seuils. Ensuite, la partie 2 de l'ADR (un opérateur narrow et une phase descendante bornée) est abandonnée par la révision du 28 juin 2026 : le raffinement sur garde pendant la montée et les seuils récupèrent déjà les bornes littérales, et une phase descendante ne pourrait pas faire redescendre le store d'état, où les écritures s'accumulent par join. L'ADR prévoyait aussi de ne récolter que les littéraux des gardes et des initialiseurs ; le code en récolte davantage, ce qui est sans danger. Les éléments annexes de l'ADR (un type `WidenOutcome`) n'ont pas été réalisés : le nom n'apparaît nulle part dans `src/`.

Piège

Le mot « narrowing » a deux sens. En théorie de l'interprétation abstraite, c'est l'opérateur de la phase descendante, que `reactant-analyzer` n'implémente pas. Dans le code, les méthodes `narrow_*` sont des *raffinements sur garde* : elles restreignent une variable sous une condition de branche, pendant la montée. C'est l'objet de la section suivante. ADR-14 insiste sur la distinction.

9.8 Raffinement sur gardes

9.8.1 Le besoin

Sans raffinement, la garde de `Guarded` ne sert à rien : l'effet lit $[0, 10]$, écrit $[1, 11]$, et le widening fait sauter la borne à $+\infty$. Il faut que, *dans la branche* `if (count < 10)`, la variable `count` soit restreinte aux valeurs qui satisfont la condition. C'est le rôle des `narrow_*`. Leur contrat est la condition 5 de la définition 9.1 : garder au moins toutes les valeurs concrètes qui satisfont la condition.

9.8.2 Ce que fait le code : les raffinements numériques

Extrait 9.26 – src/domains/impls/interval.rs, l. 255–309 — les raffinements numériques des intervalles

```

255 // Narrowing: restrict interval to satisfy a comparison against a literal `v`.
256 //
257 // For `<`/`>` the sound bound over reals is `v` itself (the excluded
258 // endpoint is kept – a float state `x = 1.7` under `x < 2` would lose 1.7 if
259 // we used `v ± 1`, an unsound value FN). Integer tightening ( $x < 5 \implies x \leq 4$ )
260 // is applied only when `is_int` proves every value is an integer; that is
261 // what keeps ADR-014 threshold widening able to bound `i < N; i++` loops.
262 pub fn narrow_lt(&self, v: f64) -> Self {
263     let bound = if self.is_int { v.ceil() - 1.0 } else { v };
264     Interval {
265         lo: self.lo,
266         hi: self.hi.min(bound),
267         is_int: self.is_int,
268     }
269 }
270 pub fn narrow_leq(&self, v: f64) -> Self {
271     let bound = if self.is_int { v.floor() } else { v };
272     Interval {
273         lo: self.lo,
274         hi: self.hi.min(bound),
275         is_int: self.is_int,
276     }
277 }
278 pub fn narrow_gt(&self, v: f64) -> Self {
279     let bound = if self.is_int { v.floor() + 1.0 } else { v };
280     Interval {
281         lo: self.lo.max(bound),
282         hi: self.hi,
283         is_int: self.is_int,
284     }
285 }
286 pub fn narrow_geq(&self, v: f64) -> Self {
287     let bound = if self.is_int { v.ceil() } else { v };
288     Interval {
289         lo: self.lo.max(bound),
290         hi: self.hi,
291         is_int: self.is_int,
292     }
293 }
294 pub fn narrow_eq(&self, v: f64) -> Self {
295     if self.lo <= v && v <= self.hi {
296         Interval::point(v)
297     } else {
298         Interval::bottom()
299     }
300 }
301 /// Conservative: can't split an interval at an interior point; return self.
302 /// Exception: a point interval equal to `v` is exactly excluded  $\rightarrow \perp$ .
303 pub fn narrow_neq(&self, v: f64) -> Self {
304     if self.is_point() && self.lo == v {
305         Interval::bottom()
306     } else {
307         *self
308     }
309 }

```



```

226         (BinOp::Lt, true) => cur.narrow_lt(v),
227         (BinOp::Lt, false) => cur.narrow_geq(v),
228         (BinOp::Leq, true) => cur.narrow_leq(v),
229         (BinOp::Leq, false) => cur.narrow_gt(v),
230         (BinOp::Gt, true) => cur.narrow_gt(v),
231         (BinOp::Gt, false) => cur.narrow_leq(v),
232         (BinOp::Geq, true) => cur.narrow_geq(v),
233         (BinOp::Geq, false) => cur.narrow_lt(v),
234         (BinOp::Eq, true) => cur.narrow_eq(v),
235         (BinOp::Eq, false) => cur.narrow_neq(v),
236         (BinOp::Neq, true) => cur.narrow_neq(v),
237         (BinOp::Neq, false) => cur.narrow_eq(v),
238         _ => cur,
239     };
240     with_refined(x, refined)
241 }

```

Le paramètre `taken` distingue la branche « alors » (condition vraie) de la branche « sinon » (condition fausse, donc négation). La négation de $x < v$ est $x \geq v$, d'où les paires croisées. Le tableau 9.2 résume tous les motifs reconnus.

Condition	branche vraie	branche fausse
$x < v$ / $x \leq v$	<code>narrow_lt</code> / <code>narrow_leq</code>	<code>narrow_geq</code> / <code>narrow_gt</code>
$x > v$ / $x \geq v$	<code>narrow_gt</code> / <code>narrow_geq</code>	<code>narrow_leq</code> / <code>narrow_lt</code>
$x == v$, $x === v$	<code>narrow_eq</code>	<code>narrow_neq</code>
$x != v$, $x !== v$	<code>narrow_neq</code>	<code>narrow_eq</code>
$x == \text{null}$ (ou <code>undefined</code>)	<code>narrow_keep_nullish_only</code>	<code>narrow_drop_null</code> (<code>_undef</code>)
x	<code>narrow_truthy</code>	<code>narrow_falsy</code>
$!x$	<code>narrow_falsy</code>	<code>narrow_truthy</code>

TABLE 9.2 – Motifs reconnus par `narrow_env_for_branch` (`src/engine/cfg_analyzer.rs`, l. 212–293) ; x est une variable, v un littéral numérique. L'IR confond `==` et `===`.

Tout autre motif laisse l'environnement inchangé, ce qui est sûr : littéral à gauche (`10 > count`), comparaison entre deux variables (`count < limit`, le faux positif du liste 3.19), comparaison à une chaîne (`mode === "a"`), `typeof x === "string"`, conditions composées par `&&` ou `||` (qui sont des diamants de CFG, chaque membre étant alors une garde séparée).

Piège

La branche fausse de $x < 5$ applique `narrow_geq(5)`, la négation *numérique*. Or `undefined < 5` est faux : la branche « sinon » est prise par une valeur qui n'est pas ≥ 5 . C'est pourtant sûr, parce que `StateValue` ne délègue la comparaison qu'à sa composante `num` et garde toutes les autres telles quelles (`src/domains/impls/state_value.rs`, l. 604–629, avec le commentaire « `null < 5` est vrai en JS »). Un contributeur qui ferait « mourir » les sortes non numériques dans une branche de comparaison introduirait un faux négatif.

9.8.4 La véracité, et une garde impossible qui n'élague rien

`if (n)` ne compare à rien ; la branche vraie exclut les valeurs *falsy* de JavaScript : `null`, `undefined`, `0`, `""`, `false` (et `NaN`, qu'aucun intervalle ne contient). Côté nombres, `narrow_truthy` délègue à `narrow_neq(0.0)` :

Extrait 9.28 – `src/domains/impls/state_value.rs`, l. 565–582 — le raffinement de véracité (branche vraie)

```

565     fn narrow_truthy(mut self) -> Self {

```

```

566     self.null = false;
567     self.undef = false;
568     self.num = self.num.narrow_neq(0.0);
569     self.boolean = match self.boolean {
570         BoolVal::False => BoolVal::Bottom,
571         BoolVal::Top => BoolVal::True,
572         b => b,
573     };
574     if let StrConst::Set(set) = &self.str
575         && set.contains("")
576     {
577         let filtered: BTreeSet<String> =
578             set.iter().filter(|s| !s.is_empty()).cloned().collect();
579         self.str = StrConst::from_set(filtered); // empty set → ⊥
580     }
581     self
582 }

```

Sur un point $[0, 0]$, le résultat est $\perp$; sur $[0, 5]$, rien ne change. Les composantes booléenne et chaîne sont traitées de même (False meurt, "" est retiré de l'ensemble). Deux programmes le montrent :

Extrait 9.29 – Garde de véracité (/tmp/ch09/limits.tsx, extrait)

```

1  export function TruthyFromZero() {
2      const [n, setN] = useState(0);
3      useEffect(() => {
4          if (n) setN(n + 1);
5      }, [n]);
6      return <div>{n}</div>;
7  }
8
9  export function TruthyFromOne() {
10     const [n, setN] = useState(1);
11     useEffect(() => {
12         if (n) setN(n + 1);
13     }, [n]);
14     return <div>{n}</div>;
15 }

```

Dans TruthyFromZero, la branche raffine $[0, 0]$ en $\perp$, puis $n + 1$ vaut $\perp$ (l'addition absorbe $\perp$), et l'écriture ne change rien : pas de boucle, conformément à la réalité (test `truthy_guarded_increment_from_zero_never_starts`). Dans TruthyFromOne, $[1, 1]$ survit, et le compteur croît :

```

$ reactant check /tmp/ch09/limits.tsx
TruthyFromOne (2 hooks) /tmp/ch09/limits.tsx
  warn  infinite-loop [hook:0] (line 29:2) this effect keeps pushing state `n` (its deps do not provably gate it,
    ↳ so the effect can re-run every render) to new values on every run. Potential infinite render loop
    (2 trace step(s), rerun with --trace)
TruthyFromZero (2 hooks) /tmp/ch09/limits.tsx
  warn  redundant-set-state [hook:0] (line 21:2) state `n` is set to a stable value it already holds, so the update
    ↳ is redundant
    (1 trace step(s), rerun with --trace)

```

(Sortie élaguée aux deux composants concernés.) Le WARNING `redundant-set-state` sur TruthyFromZero est un faux positif : l'écriture n'a jamais lieu. La règle réévalue l'argument $n + 1$ dans l'environnement convergé *sans* le raffinement de la branche, obtient $[1, 1]$, et conclut « même valeur » parce que $[1, 1]$ et $[0, 0]$ sont tous deux des valeurs « stables » (chapitre 18).

Enfin, un raffinement qui donne $\perp$ ne rend pas la branche morte : `narrow_env_for_branch`

remplace la variable testée dans l'environnement, mais le bloc reste analysé. Si l'écriture ne dépend pas de la variable raffinée, elle a lieu. On le voit avec une première version du liste 9.23 où l'effet s'écrit `const y = x << 1; if (y === 2) setCount(count + 1); : y` vaut $[3, 3]$, la garde le raffine en $\perp$, et pourtant l'écriture `count + 1` est prise en compte (infinite-loop tire, fichier `/tmp/ch09/shift.tsx`). Ne pas élaguer est sûr ; c'est seulement moins précis qu'un élagage de bloc.

9.9 Bilan : la soundness opération par opération

Le tableau 9.3 rassemble les arguments du chapitre. Une opération *exacte* ne perd aucune information ($\gamma(a \sqcup b) = \gamma(a) \cup \gamma(b)$, ou l'analogue) ; une opération *sound* sur-approxime ; un *défaut* sous-approxime dans certains cas, ce qui peut produire un faux négatif.

Opération	Argument	Statut
BoolVal : join, meet	bijection avec $\mathcal{P}(\mathbb{B})$	exact
SetterVal : join, meet	proposition 9.1	sound
StrConst : join, meet	proposition 9.2	sound (join exact sous le seuil)
concaténation de chaînes	produit cartésien, puis <code>from_set</code>	exact sous le seuil
hull, meet des intervalles	proposition 9.3	sound
add, sub, neg	théorème 9.1	sound (bornes infinies)
mul	théorème 9.1, sauf $0 \times \pm\infty$	défaut
rem, pow	proposition 9.4, coins du quadrant	sound (valeurs finies)
/, &&,	$\top$	sound
comparaisons, instanceof	in, BoolVal::Top	sound
&, , ^, >>>, ~	plage int32/uint32, bits	sound
<<, >>	multiplication / division des bornes brutes	défaut (non-entiers)
widen, widen_to	théorème 9.2	widening
narrow_lt ... narrow_neq	théorème 9.3	sound

TABLE 9.3 – Statut de soundness des opérations des treillis simples.

Deux défauts sur quatorze lignes : c'est peu, et chacun tient à une hypothèse implicite (pas de NaN dans un repli min/max, des bornes entières dans un décalage). Tous deux se corrigent dans la couche intervalle, une fois pour toutes les règles, ce qui est l'esprit du projet (principes de CLAUDE.md : corriger le moteur, pas les règles).

9.10 Limites et défauts observés

9.10.1 $0 \times \pm\infty$: un $\perp$ inattendu

Le repli de `mul` (`src/domains/impls/interval.rs`, l. 180–181) calcule le minimum et le maximum des quatre produits de coins avec `f64::min` et `f64::max`. Or ces fonctions *ignorent* NaN : `f64::min(x, NaN)` vaut `x`. Un produit de coins $0 \times \pm\infty$, qui vaut NaN, est donc silencieusement écarté. C'est souvent sans conséquence ($[0, 5] \times [0, +\infty]$ donne bien $[0, +\infty]$, les autres coins suffisent), mais si les *quatre* coins sont NaN, les replis rendent leurs valeurs initiales, $+\infty$ et $-\infty$: l'intervalle $[+\infty, -\infty]$, c'est-à-dire $\perp$. Cela arrive pour $[0, 0] \times [-\infty, +\infty]$. Or, pour tout x fini, $0 \times x = 0$: la valeur concrète 0 est perdue.

Extrait 9.30 – Un produit par zéro d'un état non borné (`/tmp/ch09/limits.tsx`, extrait)

```

1 export function ZeroTimesUnbounded() {
2   const [n, setN] = useState(0);
3   const [k, setK] = useState(0);
4   const z = n * 0;
5   useEffect(() => {
6     setK(k + 1 + z);
7   }, [k]);
8   return (
9     <div>
10      <button onClick={() => setN(n + 1)}></button>
11      <button onClick={() => setN(n - 1)}></button>
12      {k}
13    </div>
14  );
15 }

```

Les deux handlers élargissent n dans les deux sens, vers $[-\infty, +\infty]$, et $z = n * 0$ est alors évalué à $\perp$. L'analyseur le laisse voir par un diagnostic contradictoire :

```

$ reactant check /tmp/ch09/limits.tsx
ZeroTimesUnbounded (5 hooks) /tmp/ch09/limits.tsx
warn infinite-loop [hook:1] (line 7:2) this effect keeps pushing state `k` (its deps do not provably gate it, so
  ↳ the effect can re-run every render) to new values on every run. Potential infinite render loop
    (2 trace step(s), rerun with --trace)
warn missing-deps [hook:2] var:z (line 7:2) `z` is used in this effect but not in its deps array, and its value
  ↳ never changes between renders
    (1 trace step(s), rerun with --trace)

```

« its value never changes » est la formulation commune aux valeurs de stabilité $\perp$ et *Stable* ; la règle, elle, ne se tait que sur *Stable*. Rejouée avec `const z = 0` ; au lieu de `const z = n * 0` ;, la même sortie ne porte plus que l'*infinite-loop* : c'est bien le $\perp$ de z qui produit ce *missing-deps*. Ici la conséquence est un faux positif, et la boucle sur k reste détectée. La raison n'est pas que n resterait borné : les deux gestionnaires font grandir son intervalle des deux côtés à chaque tour, et il atteint bien $[-\infty, +\infty]$ (c'est ce qui rend z inatteignable). Mais le *widen_trace* qui alimente *widening-info* n'enregistre que les élargissements de *state_from_render* et *state_from_effects* (*src/engine/fixpoint.rs*, l. 514–516, commentaire « handler widening is not a bug ») : l'élargissement de n , dû aux gestionnaires de clic, n'y apparaît pas, quand bien même le n final est bien non borné. Mais $\perp$ signifie « inatteignable » : toute écriture dont la valeur dépendrait de z deviendrait un no-op abstrait, et une boucle réelle pourrait disparaître. La correction naturelle (non appliquée au commit étudié) est de traiter $0 \times \pm\infty$ comme 0 (puisque $\pm\infty$ dénote l'absence de borne et non une valeur), ou de rendre $\top$ dès qu'un coin est *NaN* ; c'est l'exercice 9.7.

9.10.2 Infinity comme valeur

La définition 9.4 traite $\pm\infty$ comme absence de borne. Mais `useState(1e999)` amorce le slot à point(*f64::INFINITY*) = $[+\infty, +\infty]$, une vraie valeur. Suivons `setN(n % 3)` : le dividende est positif, le bit est faux (un infini n'est pas entier), donc la borne est $\mu = 3$ et `rem` rend $[0, 3]$ (liste 9.17). En JavaScript, `Infinity % 3` vaut *NaN*, qui n'y est pas. Le dossier de notes a confirmé ce calcul par une sonde, sans effet observable sur les diagnostics (*NaN* est `Object.is-égal` à lui-même, la boucle s'arrête concrètement). *docs/limitations.md* ne mentionne que les *NaN* de `/`, de `%` par un diviseur possiblement nul et de `**` hors du quadrant positif. L'écart est une limite de la représentation : un intervalle ne peut pas dire à la fois « non borné » et « contient *Infinity* ».

9.10.3 Les cycles finis

Le domaine représente l'ensemble des valeurs *atteignables*, pas les *transitions* entre valeurs. Un état qui cycle dans un ensemble fini a donc un point fixe abstrait fini, atteint sans widening :

Extrait 9.31 – Deux cycles finis (/tmp/ch09/limits.tsx, extrait)

```

1 export function ModCycle() {
2   const [i, setI] = useState(0);
3   useEffect(() => {
4     setI((i + 1) % 3);
5   }, [i]);
6   return <div>{i}</div>;
7 }
8
9 export function StrToggle() {
10  const [mode, setMode] = useState("a");
11  useEffect(() => {
12    setMode(mode === "a" ? "b" : "a");
13  }, [mode]);
14  return <div>{mode}</div>;
15 }

```

`i` converge vers $[0, 2]$ (exemple 9.2) et `mode` vers $\{"a", "b"\}$. Or les deux composants bouclent concrètement : chaque écriture change la valeur (au sens d'`Object.is`), donc la dep change, donc l'effet se réexécute. L'analyseur les déclare sans défaut :

```

$ reactant check /tmp/ch09/limits.tsx --show-clean
ModCycle (2 hooks) /tmp/ch09/limits.tsx ✓
StrToggle (2 hooks) /tmp/ch09/limits.tsx ✓
...

```

(Sortie élaguée aux deux composants concernés.) Le bras « point fixe » de `infinite-loop` exige que le slot ait été élargi et soit non borné, ce qui n'est pas le cas d'un cycle fini ; le bras « churn » ne concerne que les références (chapitres 15 et 19). Aucun treillis de ce chapitre ne peut porter l'information manquante, qui est une propriété de transition (« l'écriture diffère toujours de la valeur lue ») ; c'est une limite de conception. Au commit `e67b10a`, elle n'est documentée ni dans `docs/limitations.md`, ni par une issue ouverte (aucune ne correspond à une recherche sur « cycle »).

9.10.4 Imprécisions assumées

Pour mémoire, les imprécisions sûres rencontrées dans le chapitre : comparaisons jamais évaluées (`1 < 2` vaut `BoolVal::Top`) ; `narrow_neq` et `narrow_truthy` incapables de couper un intervalle (`[0, 5]` reste `[0, 5]` sous `if (n)`) ; raffinement limité aux motifs « variable contre littéral » ; garde impossible qui n'élague pas le bloc ; `1 + "a"` évalué à $\top$ au lieu d'une chaîne ; `!⊥` évalué à un booléen inconnu ; seuils récoltés sans discernement. Chacune peut produire un faux positif, aucune un faux négatif.

Résumé du chapitre

- Un domaine est un type qui implémente `AbstractDomain` (`src/domains/mod.rs`) ; l'ordre est `PartialOrd`. Le contrat implicite (définition 9.1) exige que `join`, `widening` et raffinement `majorant` ; `meet` n'est pas utilisé par l'analyse.

- `flat_lattice!` (`src/domains/impls/mod.rs`) engendre les treillis plats `BoolVal` (exact) et `SetterVal` (identité exacte ou $\top$). `StrConst` (`str_const.rs`) est un ensemble d'au plus 4 chaînes, $\top$ au-delà ; hauteur finie, `widen = join`.
- `Interval` (`interval.rs`) : bornes `f64`, bit `is_int` hors égalité, $\perp$ multiple (tester `is_bottom`), `NaN` jamais représenté. `Join` = enveloppe convexe.
- Arithmétique : formules classiques, `sound` grâce à la monotonie de l'arrondi ; `/` donne $\top$; `%` et `**` (commit 548f922) rendent `None` quand `NaN` est possible ; opérateurs bit à bit bornés par `int32/uint32` (`eval_bitwise`).
- Widening à seuils (ADR-14) : chaque borne saute au seuil le plus serré parmi les littéraux du composant ; terminaison en au plus $|T| + 1$ sauts par borne. Chronologie du compteur : $[0, 0]$, $[0, 1]$, $[0, 2]$, puis saut à 10 ou à $+\infty$ au tour 3.
- Raffinement sur garde : `narrow_*`, choisis par `narrow_env_for_branch` selon la forme « variable contre littéral » ; le resserrement entier n'a lieu que si `is_int` est prouvé. Pas de phase descendante.
- Défauts : $0 \times \pm\infty$ donne $\perp$; `<<` et `>>` sur des non-entiers sous-approximent (faux négatif observé) ; `Infinity` comme valeur ; cycles finis invisibles.

Le chapitre 10 assemble ces treillis en un produit sur les sortes JavaScript et y ajoute le treillis de stabilité référentielle ; le chapitre 11 décrit l'évaluateur qui les fait vivre.

Exercices

Exercice 9.1: Meet des treillis plats

(a) Montrer que le meet de `flat_lattice!` est la plus grande borne inférieure. (b) Calculer `True` $\sqcap$ `False` et vérifier la condition 6 du contrat pour `BoolVal`. (c) Pourquoi le bras (a, b) `if a == b` doit-il précéder les autres dans `join` ?

Solution

(a) Égaux : a est le plus grand minorant de $\{a\}$. L'un est $\top$: l'autre est minorant et tout minorant commun le minore. Sinon, deux intermédiaires distincts (ou un intermédiaire et $\perp$) n'ont que $\perp$ comme minorant commun. (b) $\perp$; $\gamma(\text{True}) \cap \gamma(\text{False}) = \emptyset = \gamma(\perp)$: exact. (c) Sans lui, $\top \sqcap \top$ ou $m \sqcap m$ tomberait dans le dernier bras et rendrait $\top$: correct pour $\top$, mais $m \sqcap m = \top$ serait une perte de précision gratuite (un booléen toujours `True` deviendrait inconnu au premier `join`).

Exercice 9.2: Chaînes

Calculer (a) $\{\text{"a"}, \text{"b"}\} \sqcup \{\text{"c"}\} \sqcup \{\text{"d"}\} \sqcup \{\text{"e"}\}$ en associant à gauche ; (b) $\{\text{"a"}, \text{"b"}\} \sqcap \{\text{"c"}\}$; (c) la valeur de `"x-" + s` pour $s = \{\text{"a"}, \text{"b"}\}$, puis pour $s = \{\text{"a"}, \text{"b"}, \text{"c"}\}$ et le préfixe $\{\text{"x"}, \text{"y"}\}$.

Solution

(a) $\{a, b, c\}$, puis $\{a, b, c, d\}$ (quatre éléments, encore représentable), puis $\top$ (cinq). (b) Intersection vide : `from_set` rend $\perp$. (c) $\{\text{"x-a"}, \text{"x-b"}\}$; le produit $\{x, y\} \times \{a, b, c\}$ a six éléments, `str_set` le remplace par « une chaîne » ($\top$ de `StrConst`).

Exercice 9.3: Widening à seuils

Avec $T = \{10, 20, 100\}$, calculer $[0, 5] \nabla_T [0, 12]$, puis $[0, 20] \nabla_T [0, 25]$, puis $[0, 100] \nabla_T [0, 101]$, puis $[0, 5] \nabla_T [-3, 5]$. Combien de sauts la borne haute peut-elle faire au plus ?

Solution

$[0, 20]$ (plus petit seuil ≥ 12) ; $[0, 100]$; $[0, +\infty]$ (aucun seuil ≥ 101) ; $[-\infty, 5]$ (aucun seuil ≤ -3). La borne haute prend ses valeurs dans $\{10, 20, 100, +\infty\}$ après le premier saut : au plus $|T| + 1 = 4$ sauts.

Exercice 9.4: Reste et puissance

Calculer avec le code : $[-7, 8] \% [3, 3]$; $[0, 9] \% [-4, -2]$; $[1, 2] \% [5, 5]$; $[0, 9] \% [0, 3]$; $[0, 2] ** [0, 3]$; $[-1, 2] ** [2, 2]$. Commenter la précision du troisième.

Solution

$[-2, 2]$ ($\mu = 3$, $\beta = 2$) ; $[0, 3]$ ($\mu = 4$, et en effet $7 \% -4 = 3$) ; $[0, 2]$; None, donc $\top$ (diviseur possiblement nul) ; coins $0^0 = 1$, $0^3 = 0$, $2^0 = 1$, $2^3 = 8$, donc $[0, 8]$; None (base possiblement négative), donc $\top$. Le troisième est sound mais imprécis : le résultat exact est $[1, 2]$, mais le code ne sait pas que $x \% 5 = x$ quand $0 \leq x < 5$ et met la borne basse à 0.

Exercice 9.5: Raffinements entiers

Sur $I = [0, 9]$ entier, calculer les raffinements sous $x \geq 2.5$, $x > 7$, $x == 12$, et sous $x != 9$ pour $I = [9, 9]$. Refaire le premier avec `is_int` faux. Prouver que `narrow_gt` est sound avec `is_int`.

Solution

$[3, 9]$ ($\lceil 2.5 \rceil$) ; $[8, 9]$ ($\lfloor 7 \rfloor + 1$) ; $\perp$ ($12 \notin [0, 9]$) ; $\perp$ (point égal à 9). Sans bit : $[2.5, 9]$. Preuve : un entier $x > v$ vérifie $x \geq \lfloor v \rfloor + 1$, car $\lfloor v \rfloor + 1$ est le plus petit entier strictement supérieur à v ; la nouvelle borne basse $\max(\ell, \lfloor v \rfloor + 1)$ est donc $\leq x$.

Exercice 9.6: Bit à bit

Calculer avec `eval_bitwise` : $[0, 300] \& 255$; $[0, 5] \wedge [0, 2]$; $[-4, 4] \mid [0, 1]$; $[100, 2^{30}] \ll 2$; $x \ggg 0$ pour x quelconque.

Solution

$[0, 255]$. $\max = 5$, $\lfloor \log_2 5 \rfloor + 1 = 3$ bits : $[0, 7]$. Opérande gauche possiblement négatif : pas de raffinement, plage `int32`. $2^{30} \times 4 = 2^{32}$ déborde d'`int32` : plage `int32`. $x \ggg 0 \in [0, 2^{32} - 1]$ (la plage `uint32` est le raffinement même, $k = 0$).

Exercice 9.7: Corriger la multiplication

Proposer une correction de `Interval::mul` pour $0 \times \pm\infty$ et prouver sa soundness sous l'hypothèse « $\pm\infty$ est une absence de borne ».

Solution

Calculer chaque produit de coins avec la convention $0 \times \pm\infty = 0$ (par exemple `if p.is_nan() 0.0 else p`). Preuve : soit $x \in [a, b]$, $y \in [c, d]$ finis. Remplaçons une borne infinie par une borne finie M assez grande pour contenir x et y : sur le pavé fini, le théorème 9.1 s'applique, et les coins valant $0 \times M = 0$ sont exactement ceux que la convention remplace ; les coins $u \times M$ avec $u \neq 0$ ne sont pas plus serrés que $u \times \infty$. L'intervalle corrigé contient donc xy . Pour $[0, 0] \times [-\infty, +\infty]$ il rend $[0, 0]$, exact. L'alternative plus simple, rendre $\top$ si un coin est `NaN`, est sound mais perd $[0, 5] \times [0, +\infty] = [0, +\infty]$.

Exercice 9.8: Corriger les décalages

Corriger les bras `Shl` et `Shr` d'`eval_bitwise` pour les opérandes non entiers, et vérifier sur `ShlFloat` et `ShrFloat` (liste 9.23).

Solution

Sur la plage `int32`, `ToInt32(x) = trunc(x)`, fonction croissante : l'image de $[\ell, h]$ est $[\text{trunc}(\ell), \text{trunc}(h)]$. Il suffit donc de remplacer `a.lo` et `a.hi` par `a.lo.trunc()` et `a.hi.trunc()` avant de multiplier ou de diviser, le reste de la preuve étant celle du cas entier (multiplication par 2^k sans débordement, division entière par 2^k arrondie vers le bas, toutes deux croissantes). `ShlFloat` : `trunc(1,5) = 1`, $y = [2, 2]$, l'effet écrit `count + 1` : la boucle est vue. `ShrFloat` : `trunc(-1,5) = -1`, $y = [-1, -1]$, idem. La correction est centrale : toutes les règles qui lisent une valeur issue d'un décalage en profitent.

Chapitre 10

Le produit `StateValue` et la stabilité référentielle

Objectifs du chapitre.

- Comprendre pourquoi la valeur abstraite de `reactant-analyzer` est un *produit* indexé par les sortes de JavaScript (ADR-015), et savoir calculer à la main son ordre, son join, son widening et sa concrétisation.
- Connaître le treillis `Stability` de l'ADR-017 : la trace de changement qu'il abstrait, les bornes `may` et `must`, les points `Stable`, `Versioned`, `VersionedTop`, `PerRender`, et savoir prouver la soundness de son join.
- Comprendre la double vue de l'état (le store range les valeurs écrites, la lecture rend une stabilité versionnée) et le traitement des littéraux, de `useRef`, `useMemo` et `useCallback`.
- Savoir quelle projection une règle doit employer (`to_stability`, `is_unstable_reference_only`, `is_unbounded`, `StabilityVerdict`), et pourquoi `PerRender` n'y a pas toujours le même sens.
- Connaître les contextes `AnalysisCtx`, `QueryContext`, `InterCtx` qui relient les domaines au moteur, et les défauts observés au commit e67b10a.

Prérequis : chapitre 9 (les quatre treillis de base, le widening à seuils) ; chapitre 3 (définition 3.2, définition 3.3, définition 3.7, définition 3.8) ; chapitre 2 (`Object.is`, `deps`, `bail-out`) ; chapitre 6 (définition 6.3, `Expr::StateVal`).

Le chapitre 9 a décrit quatre petits domaines : intervalles, booléens, ensembles bornés de chaînes, setters. Aucun ne suffit seul, parce qu'une variable JavaScript n'a pas de type fixe : `useState(null)` rend une valeur qui sera `null` au premier rendu et un nombre au suivant. Ce chapitre assemble ces domaines en une seule valeur abstraite, `StateValue`, et lui ajoute la pièce la plus propre à React : la *stabilité référentielle*, c'est-à-dire la réponse à la question « cette valeur sera-t-elle `Object.is`-égale à celle du rendu précédent ? ».

Deux questions guident tout le chapitre. La première porte sur les *valeurs* : dans le compteur `setN(n + 1)` amorcé à `null`, l'analyseur voit-il que `n` grandit sans borne ? Elle mène au produit ponctuel (sections 10.1 et 10.2). La seconde porte sur l'*identité* au fil des rendus : une `dep` qui lit un état objet change-t-elle à chaque rendu ? Elle mène au treillis de stabilité versionnée (sections 10.3 et 10.4) et à la double vue de l'état (section 10.5). Les sections suivantes montrent comment les mémos, les règles et le moteur lisent ces valeurs, puis recensent les limites observées.

10.1 Un enum plat perd `null` $\cup$ `number`

10.1.1 Le problème

Voici un compteur sans garde, amorcé à `null` au lieu de `0` :

Extrait 10.1 – Un compteur nullable (/tmp/ch10/nullable.tsx, extrait)

```

1 export function NullCounter() {
2   const [n, setN] = useState(null);
3   useEffect(() => {
4     setN(n + 1);
5   }, [n]);
6   return <div>{n}</div>;
7 }

```

Concrètement, React monte le composant avec $n = \text{null}$, puis exécute l'effet : $\text{null} + 1$ vaut 1 en JavaScript ($\text{ToNumber}(\text{null})$ vaut 0). La dep n passe de null à 1, l'effet se ré-exécute, écrit 2, puis 3, et ainsi de suite : une boucle de rendu infinie. Pour la détecter, l'analyseur doit voir la valeur de n *grandir*, c'est-à-dire garder l'intervalle numérique précis même quand null fait aussi partie des valeurs possibles.

Le motif n'a rien d'exotique : `useState(null)` est la manière idiomatique d'écrire « pas encore chargé ». Un domaine qui ne sait pas représenter « null ou un nombre » laisse passer toute une famille de boucles.

10.1.2 Le domaine d'ADR-008 : une somme plate

La première version du domaine des valeurs (ADR-8, juin 2026) était une somme : une valeur abstraite était *exactement* une sorte JavaScript.

Extrait 10.2 – L'ancien StateValue (docs/adr/ADR-008-value-domain.md, élagué des commentaires)

```

1 pub enum StateValue {
2   Bottom,
3   Null,
4   Undefined,
5   Number(Interval),
6   Boolean(BoolVal),
7   StrConst(Arc<BTreeSet<String>>),
8   Str,
9   Reference(Stability),
10  Top,
11 }

```

Dans une somme, le join de deux sortes différentes n'a qu'un majorant commun : $\top$. L'ADR le dit en une ligne de son tableau : $\text{Null} \sqcup \text{Number}(i) = \text{Top}$ (figure 10.1, à gauche). Sur `NullCounter`, le premier tour écrit $[1, 1]$ dans un slot qui vaut Null ; le join donne $\top$, et $\top$ est un point fixe. Le store ne s'élargit jamais, aucun widening n'a lieu, et le bras « point fixe » d'*infinite-loop* (qui cherche un slot élargi, chapitre 19) ne voit rien : un faux négatif (FN).

10.1.3 Trois rustines, et un FN qui résiste

L'ADR-008 a compensé la perte par trois mécanismes empilés (ADR-15, section « Context ») :

1. un `TypedStateStore` à cinq sous-stores (nombres, booléens, chaînes, références, inconnus), choisis label par label ;
2. une inférence `infer_state_type` qui devinait la sorte d'un slot à partir de son initialiseur ;
3. la lecture de l'annotation TypeScript `useState<number>(null)` : le lowering la capturait dans `HookEntry::State.type_hint`, et le point fixe remplaçait l'amorçage `Null` par `Number([0, 0])`.

Même avec les trois, `useState(null)` *sans* annotation restait un FN documenté, et un code

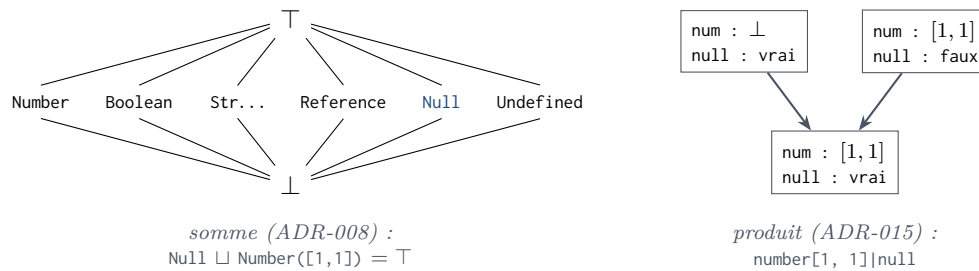


FIGURE 10.1 – Le même join dans les deux domaines. À gauche, la somme plate d’ADR-008 : deux sortes différentes n’ont que $\top$ pour majorant commun (les sous-treillis de chaque sorte sont repliés en un nœud). À droite, le produit d’ADR-015 : chaque sorte garde sa composante, le join se fait composante par composante (seules deux composantes sont montrées).

JavaScript pur qui mélange nombres et chaînes (`let a = 10; if (c) a = "s";`) perdait toute précision. Les rustines traitaient le symptôme (une sorte par slot) sans toucher à la cause (une sorte par valeur).

Décision : ADR-008 — domaine des valeurs en somme plate

Décidait : un StateValue en enum plat, tout join inter-sortes vaut $\top$; précision regagnée par un store typé et par les annotations TypeScript. **Statut** : remplacée par ADR-15 (14 juillet 2026). **Ce qui survit** : les sous-domaines Interval, BoolVal, StrConst (seuil 4), la machinerie de widening et de raffinement, et le signal « le store s’élargit, donc boucle possible ». **À ne pas croire** : la ligne du tableau d’ADR-008 « Boolean : oscillation true↔false détectée » ne correspond pas au comportement actuel (section 10.9).

10.2 Sortes disjointes : le produit StateValue

10.2.1 L’observation

Une valeur JavaScript n’est jamais à la fois un nombre et une chaîne, ni à la fois null et un objet : les sortes sont *disjointes*. Or ce que l’on veut abstraire, c’est un *ensemble* de valeurs possibles V . Un tel ensemble se découpe de manière unique selon les sortes :

$$V = (V \cap \text{Num}) \uplus (V \cap \text{Bool}) \uplus (V \cap \text{Str}) \uplus \dots$$

Chaque morceau s’abstrait dans le treillis de sa sorte, indépendamment des autres. Le domaine idéal (la *complétion disjonctive* de la somme, qui représente des disjonctions de valeurs abstraites) dégénère donc, pour une somme de sortes disjointes, en un *produit ponctuel* : une composante par sorte, chacune à $\perp$ quand la sorte est impossible. C’est l’observation d’ADR-015, et elle tient en une phrase, conformément au principe du paragraphe unique de CLAUDE.md.

Remarque

Dans une somme non disjointe (par exemple « entier » et « entier pair »), une même valeur pourrait relever de deux composantes, et un produit perdrait la corrélation entre elles. C’est précisément la disjonction des sortes qui rend le produit exact, sans opérateur de réduction entre composantes.

10.2.2 Le code

Extrait 10.3 – src/domains/impls/state_value.rs, l. 16–44 — le produit ponctuel des sortes JavaScript

```

16 /// Abstract JS value: pointwise product over the disjoint JS kinds (ADR-015).
17 ///
18 /// JS primitive kinds are mutually exclusive (a value is never a number AND a
19 /// string), so the disjunctive completion of the kind sum degenerates into a
20 /// product: one independent slot per kind, each  $\perp$  when that kind is impossible.
21 /// `join`/`meet`/`widen` are pointwise – a cross-kind join keeps BOTH kinds
22 /// (`number | null`, `number | string`) instead of collapsing to  $\top$ , which is
23 /// what enables infinite-loop detection through nullable states without a
24 /// TypeScript hint.
25 #[derive(Clone, PartialEq)]
26 pub struct StateValue {
27     /// Numeric kind – interval [lo, hi];  $\perp$  = cannot be a number.
28     pub num: Interval,
29     /// Boolean kind.
30     pub boolean: BoolVal,
31     /// String kind – finite constant set, threshold-widened to  $\top$ .
32     pub str: StrConst,
33     /// Object/array/function kind – reference stability;  $\perp$  = not a reference.
34     pub reference: Stability,
35     /// `null` possible?
36     pub null: bool,
37     /// `undefined` possible?
38     pub undef: bool,
39     /// Cross-component useState setter (flat lattice with identity payload).
40     pub setter: SetterVal,
41     /// Residual  $\top$  – kinds not modelled (symbol, bigint, ...). `true` means the
42     /// value may be something outside every other slot.
43     pub other: bool,
44 }

```

Les huit composantes, champ par champ :

- `num` : un `Interval` (section 9.4); $\perp$ signifie « ne peut pas être un nombre ».
- `boolean` : le treillis plat `BoolVal` (section 9.2).
- `str` : l'ensemble borné `StrConst`, élargi à $\top$ au-delà de quatre constantes (section 9.3).
- `reference` : la sorte « objet, tableau ou fonction ». Sa composante n'est pas un ensemble de valeurs mais une *stabilité* (section 10.4); $\perp$ signifie « ne peut pas être une référence ».
- `null`, `undef` : deux booléens Rust, `true` voulant dire « cette valeur est possible ». L'ordre de `bool` (`false` < `true`) est exactement le treillis à deux points $\perp \sqsubseteq \top$.
- `setter` : le treillis plat `SetterVal` des setters d'état identifiés par `(ComponentId, HookLabel)`.
- `other` : le *résidu*, « tout le reste » (symbol, bigint, et toute sorte non modélisée).

Définition 10.1: Le produit StateValue

Une valeur abstraite est un octuplet

$$v = (v.num, v.boolean, v.str, v.reference, v.null, v.undef, v.setter, v.other),$$

élément du produit des huit treillis ci-dessus. L'ordre, le join, le meet et le widening sont *ponctuels* (composante par composante). La concrétisation est l'union disjointe des concrétisations des composantes :

$$\begin{aligned}
 \gamma(v) = & \gamma_{num}(v.num) \cup \gamma_{bool}(v.boolean) \cup \gamma_{str}(v.str) \\
 & \cup \{o \text{ objet, tableau ou fonction} \mid \text{la trace de changement de } o \in \gamma_S(v.reference)\} \\
 & \cup \{\text{null} \mid v.null\} \cup \{\text{undefined} \mid v.undef\} \cup \gamma_{setter}(v.setter) \\
 & \cup \{\text{toute autre valeur (symbol, bigint, ...) } \mid v.other\}
 \end{aligned}$$

où γ_S est la concrétisation de la stabilité (définition 10.3).

Exemple 10.1: Lire une valeur du produit

La valeur affichée `number[1, inf]|null` a $num = [1, +\infty]$, $null = true$, et toutes les autres composantes à $\perp$. Sa concrétisation est $\{x \in \mathbb{R} \mid x \geq 1\} \cup \{null\}$ (avec le bit `is_int`, les seuls entiers). Elle ne contient ni `undefined`, ni chaîne, ni objet. L’affichage (Debug, écrit à la main, `src/domains/impls/state_value.rs`, l. 422–476) juxtapose les sortes peuplées séparées par `|`.

Piège

La composante `reference` ne ressemble pas aux autres. Pour `num` ou `str`, γ décrit des *valeurs* ; pour `reference`, elle décrit la *stabilité* des références possibles, sans rien dire de leur contenu. Le contenu des objets relève du tas par site d’allocation (chapitre 11). Une `StateValue ref(Stable)` dit « une référence qui ne change pas d’un rendu à l’autre », pas « l’objet `a: 1` ».

Et NaN ? Aucune composante n’en parle. Toute opération qui pourrait le produire (division, reste par un diviseur qui peut être nul, puissance, `undefined` en arithmétique) rend $\top$, dont le résidu `other` est vrai : c’est par là que NaN est couvert (section 9.5). Le code le dit explicitement pour `%` et `**` : « where the result may be NaN, which no interval holds, they are $\top$ like / » (`src/domains/transfer/state_value.rs`, l. 781).

10.2.3 $\perp$, $\top$ et les constructeurs

Extrait 10.4 – `src/domains/impls/state_value.rs`, l. 190–206 — $\perp$: toutes les composantes à $\perp$

```
190    /// `const` version of `AbstractDomain::bottom()` usable in struct-update syntax.
191    const fn bottom_value() -> Self {
192        StateValue {
193            num: Interval {
194                lo: f64::INFINITY,
195                hi: f64::NEG_INFINITY,
196                is_int: true,
197            },
198            boolean: BoolVal::Bottom,
199            str: StrConst::Bottom,
200            reference: Stability::Bottom,
201            null: false,
202            undef: false,
203            setter: SetterVal::Bottom,
204            other: false,
205        }
206    }
```

`bottom_value` est une `const fn` pour pouvoir servir dans la syntaxe de mise à jour de structure (`..Self::bottom_value()`) de chaque constructeur. L’intervalle vide y est écrit à la main avec ses bornes sentinelles $[+\infty, -\infty]$: tout prédicat qui lit `num.lo` ou `num.hi` doit donc d’abord tester `is_bottom()` (section 10.7). $\top$ met toutes les composantes à leur $\top$, *y compris* le résidu :

Extrait 10.5 – `src/domains/impls/state_value.rs`, l. 512–523 — $\top$: toutes les composantes à $\top$, résidu compris

```
512    fn top() -> Self {
513        StateValue {
514            num: Interval::top(),
```

```

515         boolean: BoolVal::Top,
516         str: StrConst::Top,
517         reference: Stability::Unknown,
518         null: true,
519         undef: true,
520         setter: SetterVal::Top,
521         other: true,
522     }
523 }

```

Les constructeurs `number`, `boolean`, `str_set`, `str_singleton`, `str_top`, `reference`, `null`, `undefined` et `component_setter` (`src/domains/impls/state_value.rs`, l. 125–188) peuplent une seule composante et laissent les autres à $\perp$:

Extrait 10.6 – `src/domains/impls/state_value.rs`, l. 125–130 — la forme commune des constructeurs

```

125     pub fn number(i: Interval) -> Self {
126         StateValue {
127             num: i,
128             ..Self::bottom_value()
129         }
130     }

```

Deux exceptions : `str_set` passe par `StrConst::from_set`, donc un ensemble vide donne $\perp$ et plus de quatre chaînes donnent `str: Top`; `component_setter(c, 1)` pose `SetterVal::One(c, 1)`.

10.2.4 KindMask : une seule énumération des sortes

Beaucoup de questions sur une valeur portent sur *quelles* sortes sont peuplées : la valeur est-elle $\perp$? n'est-elle qu'un setter ? combien de sortes mêle-t-elle ? Si chaque prédicat ré-énumérait les huit champs, l'ajout d'un neuvième champ serait silencieusement oublié quelque part, et un prédicat comme « la valeur n'est qu'une référence fraîche » deviendrait vrai à tort : un faux négatif latent. Le code centralise donc l'énumération dans un masque de bits.

Extrait 10.7 – `src/domains/impls/state_value.rs`, l. 46–78 — le masque des sortes peuplées

```

46 /// Bitset of which kind slots of a [ `StateValue` ] are non- $\perp$ .
47 ///
48 /// The product's kind enumeration lives in exactly one place -
49 /// [ `StateValue::populated_kinds` ], whose destructuring has no `..`. Adding a
50 /// kind field is therefore a compile error there, not a silent omission that
51 /// would quietly break every "is this  $\perp$  / only a setter / how many kinds"
52 /// predicate downstream (a latent false negative). Each such predicate derives
53 /// from this mask instead of re-listing the eight slots.
54 #[derive(Clone, Copy, PartialEq, Eq)]
55 struct KindMask(u8);
56
57 impl KindMask {
58     const NUM: u8 = 1 << 0;
59     const BOOL: u8 = 1 << 1;
60     const STR: u8 = 1 << 2;
61     const REF: u8 = 1 << 3;
62     const NULL: u8 = 1 << 4;
63     const UNDEF: u8 = 1 << 5;
64     const SETTER: u8 = 1 << 6;
65     const OTHER: u8 = 1 << 7;
66
67     fn is_empty(self) -> bool {

```

```

68     self.0 == 0
69 }
70 /// Number of populated (non-⊥) kinds.
71 fn count(self) -> u32 {
72     self.0.count_ones()
73 }
74 /// Every populated kind is within `{bit}` (holds when nothing is populated).
75 fn only(self, bit: u8) -> bool {
76     self.0 & !bit == 0
77 }
78 }

```

Extrait 10.8 – src/domains/impls/state_value.rs, l. 80–121 — l'unique énumération des sortes

```

80 impl StateValue {
81     /// Bitset of the non-⊥ kind slots – the single enumeration of the product's
82     /// kinds. The `is bottom` / `only setter` / `only reference` / `how many
83     /// kinds` predicates all read this instead of re-listing the slots.
84     fn populated_kinds(&self) -> KindMask {
85         let StateValue {
86             num,
87             boolean,
88             str,
89             reference,
90             null,
91             undef,
92             setter,
93             other,
94         } = self;
95         let mut m = 0u8;
96         if !num.is_bottom() {
97             m |= KindMask::NUM;
98         }
99         if *boolean != BoolVal::Bottom {
100             m |= KindMask::BOOL;
101         }
102         if *str != StrConst::Bottom {
103             m |= KindMask::STR;
104         }
105         if *reference != Stability::Bottom {
106             m |= KindMask::REF;
107         }
108         if *null {
109             m |= KindMask::NULL;
110         }
111         if *undef {
112             m |= KindMask::UNDEF;
113         }
114         if *setter != SetterVal::Bottom {
115             m |= KindMask::SETTER;
116         }
117         if *other {
118             m |= KindMask::OTHER;
119         }
120         KindMask(m)
121     }

```

Invariant 10.1: Une seule énumération des sortes

Tout prédicat de la forme « quelles sortes sont peuplées » se dérive de `populated_kinds`, dont la déstructuration n'a pas de `...`. Ajouter une composante à `StateValue` est donc une erreur de compilation dans `populated_kinds` (et dans `is_top_value`, qui utilise la même astuce pour la saturation à $\top$), pas un oubli silencieux.

Les consommateurs du masque sont `is_bottom_value` (`is_empty`), `typeof_name` (une seule sorte), `as_setter` et `is_unstable_reference_only` (`only`), et `to_stability` (`count() >= 2`). La protection vient d'ADR-020 (ADR-20, décision D4, commit c887c52).

Extrait 10.9 – `src/domains/impls/state_value.rs`, l. 270–287 — deux prédicats dérivés du masque

```

270     /// Exact setter identity, only when the value can be nothing else.
271     pub fn as_setter(&self) -> Option<(&ComponentId, &HookLabel)> {
272         // Only the setter slot may be populated (`as_one` still yields `None`
273         // if the setter itself is  $\perp/\top$ ).
274         if self.populated_kinds().only(KindMask::SETTER) {
275             self.setter.as_one()
276         } else {
277             None
278         }
279     }
280
281     /// True when the value is exactly a per-render-fresh reference (nothing
282     /// else) – a *must* fact: a new reference every render, guaranteed.
283     /// `Versioned` (changes only at setter events) deliberately returns false
284     /// (ADR-017).
285     pub fn is_unstable_reference_only(&self) -> bool {
286         self.reference == Stability::PerRender && self.populated_kinds().only(KindMask::REF)
287     }

```

Piège

`only(bit)` est *vrai* sur $\perp$: aucun bit n'est peuplé, donc tous les bits peuplés sont bien dans `bit`. Chaque consommateur doit écarter $\perp$ lui-même s'il le faut. `is_unstable_reference_only` le fait en exigeant en plus `reference == PerRender` ; `as_setter` le fait parce que `SetterVal::as_one` rend `None` sur $\perp$ et $\top$.

Piège

La protection est partielle. Les fonctions de transfert `as_arith`, `as_str_only` (`src/domains/transfer/state_value.rs`, l. 708–750) et la négation `!` (`src/domains/transfer/state_value.rs`, l. 947–965) ré-énumèrent les composantes à la main, sans passer par `KindMask` : une composante ajoutée ne les ferait pas échouer à la compilation (constat de lecture, dossier 04 §3.7).

`typeof_name` illustre la sémantique JavaScript encodée dans le masque : `typeof null` vaut "object", un setter est une "function", et la composante `reference` ne donne *pas* de réponse, parce qu'elle couvre à la fois les objets ("object") et les fonctions ("function") (`src/domains/impls/state_value.rs`, l. 240–268).

10.2.5 L'arithmétique : `as_arith` et `ToNumber(null)`

Le produit ne sert que si l'évaluateur sait calculer $n + 1$ quand n vaut `number[1, 1]` ou `null`. La règle est celle de JavaScript : `ToNumber(null)` vaut 0. L'évaluateur demande donc une *vue numérique* de chaque opérande :

Extrait 10.10 – src/domains/transfer/state_value.rs, l. 708–732 — la vue numérique d'un opérande

```

708 /// Numeric view of an operand for arithmetic, per JS `ToNumber` coercion.
709 ///
710 /// `Some` only when the value's active slots are within {number, null}:
711 /// `ToNumber(null) = 0`, so a nullable number stays a precise interval –
712 /// this is what lets `useState(null)` counters (`setN(n + 1)`) keep widening.
713 /// `undefined` coerces to NaN and every other kind is unpredictable → `None`.
714 fn as_arith(v: &StateValue) -> Option<Interval> {
715     if v.boolean == BoolVal::Bottom
716         && v.str == StrConst::Bottom
717         && v.reference == Stability::Bottom
718         && !v.undef
719         && v.setter == SetterVal::Bottom
720         && !v.other
721     {
722         // NB: a  $\perp$  interval stays Some( $\perp$ ) – a narrowed-dead path must produce
723         //  $\perp$  (joins as a no-op), not fall through to  $\top$ .
724         Some(if v.null {
725             v.num.hull(&Interval::point(0.0))
726         } else {
727             v.num
728         })
729     } else {
730         None
731     }
732 }

```

as_arith rend un intervalle si les seules sortes peuplées sont « nombre » et « null » ; la contribution de null est le point $[0, 0]$, joint à l'intervalle par hull. Toute autre sorte (undefined, qui donnerait NaN ; booléen ; chaîne ; référence ; setter ; résidu) rend None, et l'opération arithmétique vaut alors $\top$ (section 9.5).

Proposition 10.1: Soundness de la vue numérique

Si $\text{as_arith}(v) = \text{Some}(I)$, alors pour toute valeur concrète $x \in \gamma(v)$, $\text{ToNumber}(x) \in \gamma(I)$.

Démonstration. Some n'est rendu que si toutes les composantes autres que num et null sont à $\perp$; donc $\gamma(v) \subseteq \gamma(v.\text{num}) \cup \{\text{null}\}$. Un nombre est son propre ToNumber et appartient à $\gamma(v.\text{num}) \subseteq \gamma(I)$; null donne $0 \in \gamma([0, 0]) \subseteq \gamma(I)$ lorsque $v.\text{null}$ est vrai, puisque hull est un majorant. $\square$

Le commentaire « a $\perp$ interval stays Some($\perp$) » n'est pas un détail. Sur une branche morte, le raffinement de garde met num à $\perp$; l'addition rend alors $\perp$, et l'écriture de $\perp$ dans un store se joint sans effet. Si as_arith rendait None, la même écriture vaudrait $\top$ et polluerait le slot.

Exemple 10.2: NullCounter, tour par tour

Amorçage : $n \mapsto \{\text{null}\}$. Premier tour : l'effet évalue $n + 1$; as_arith(null) = $[0, 0]$, donc l'écriture vaut $[1, 1]$; le store fait le join ponctuel et contient number[1, 1]|null. Deuxième tour : as_arith rend $[0, 1]$, l'écriture $[1, 2]$, le store number[1, 2]|null. Au troisième tour, la boucle externe applique widen_to (section 9.7), qui n'agit que sur num ; la borne haute saute à $+\infty$, null reste vrai. La sonde relève la valeur convergée :

```

-- NullCounter
state[0] = number[1, inf]|null   widened=true   num=Interval { lo: 1.0, hi: inf, is_int: true }

```

et la ligne de commande signale la boucle :

```
$ reactant check /tmp/ch10/nullable.tsx --trace
NullCounter (2 hooks) /tmp/ch10/nullable.tsx
warn infinite-loop [hook:0] (line 5:2) this effect keeps pushing state `n` (its deps do not provably gate
  ↳ it, so the effect can re-run every render) to new values on every run. Potential infinite render loop
  ↳ state `n` is written here [hook:1] (line 5:2)
  ↳ the abstract value of state `n` kept growing and was widened at iteration 3
```

Le FN d'ADR-008 est devenu une détection, épinglée par le test `tests/narrowing.rs::null_init_without_hint_unbounded_is_flagged`.

La « sonde » citée ici et dans la suite du chapitre est un petit programme temporaire, hors du dépôt, qui appelle `lower_program` puis `analyze_component` sur un fichier et imprime les valeurs convergées des stores (`state[1]` pour le slot de label `l`, `memo[1]` pour un mémo), avec le drapeau `widened` (le slot est-il dans `widen_trace`?). Les identités de composant y apparaissent comme `ComponentId(4294967295)`, c'est-à-dire `ComponentId::SYNTHETIC` d'une analyse isolée.

Deux variantes du même fichier montrent les bords du mécanisme. Remplacer `useState(null)` par `useState(undefined)` (composant `UndefCounter`) fait échouer `as_arith : undefined + 1` vaut `NaN`, que l'intervalle ne sait pas représenter, donc l'écriture vaut `⊤` et le store aussi (la sonde affiche `state[0] = ⊤`). L'analyseur émet alors un `WARNING` plus vague :

```
UndefCounter (2 hooks) /tmp/ch10/nullable.tsx
warn infinite-loop [hook:0] (line 21:2) this effect may store a fresh reference into state `n` which its deps
  ↳ react to: possible infinite render loop
  ↳ a fresh value is written to state `n` here [hook:1] (line 22:4)
```

C'est un faux positif (FP) toléré : concrètement, `n` passe de `undefined` à `NaN`, puis `NaN + 1` vaut `NaN`, et `Object.is(NaN, NaN)` est vrai, donc React abandonne le rendu suivant. La valeur `⊤` a une composante `reference` à `Unknown`, que le graphe de `churn` lit comme « peut-être fraîche » (section 10.5). Le motif « chargement unique » `if (!user) setUser( name: "guest" )`, lui, converge sans bruit (le composant `FetchOnce` est propre) : la branche `!user` applique `narrow_falsy`, qui tue la composante `reference` (section 9.8), et l'écriture n'a lieu qu'une fois ; le store se stabilise sur `ref(PerRender)|null`.

10.2.6 Ordre, join, meet et widening ponctuels

Extrait 10.11 – `src/domains/impls/state_value.rs`, l. 479–501 — l'ordre produit

```
479 /// Combine two per-slot orderings; `None` when slots disagree in direction.
480 fn merge_ord(acc: Option<Ordering>, next: Option<Ordering>) -> Option<Ordering> {
481     match (acc?, next?) {
482         (Ordering::Equal, x) | (x, Ordering::Equal) => Some(x),
483         (Ordering::Less, Ordering::Less) => Some(Ordering::Less),
484         (Ordering::Greater, Ordering::Greater) => Some(Ordering::Greater),
485         _ => None,
486     }
487 }
488
489 impl PartialOrd for StateValue {
490     fn partial_cmp(&self, other: &Self) -> Option<Ordering> {
491         let mut ord = self.num.partial_cmp(&other.num);
492         ord = merge_ord(ord, self.boolean.partial_cmp(&other.boolean));
493         ord = merge_ord(ord, self.str.partial_cmp(&other.str));
494         ord = merge_ord(ord, self.reference.partial_cmp(&other.reference));
495         ord = merge_ord(ord, self.null.partial_cmp(&other.null));
496         ord = merge_ord(ord, self.undef.partial_cmp(&other.undef));
497         ord = merge_ord(ord, self.setter.partial_cmp(&other.setter));
498         ord = merge_ord(ord, self.other.partial_cmp(&other.other));
499         ord
```



```

500     }
501 }

```

`merge_ord` combine les comparaisons des composantes : `Equal` est neutre, deux `Less` (ou deux `Greater`) restent dans leur sens, et deux sens opposés, ou une composante incomparable (`None`, propagé par l'opérateur `?`), rendent tout le produit incomparable. On a donc $a \sqsubseteq b$ si et seulement si chaque composante de a est sous celle de b : c'est l'ordre produit (test `mixed_direction_slots_are_incomparable`).

Extrait 10.12 – `src/domains/impls/state_value.rs`, l. 631–676 — `join`, `meet`, `widen` et `widen_to` ponctuels

```

631     fn join(&self, other: &Self) -> Self {
632         StateValue {
633             num: self.num.hull(&other.num),
634             boolean: self.boolean.join(&other.boolean),
635             str: self.str.join(&other.str),
636             reference: self.reference.join(&other.reference),
637             null: self.null || other.null,
638             undef: self.undef || other.undef,
639             setter: self.setter.join(&other.setter),
640             other: self.other || other.other,
641         }
642     }
643
644     fn meet(&self, other: &Self) -> Self {
645         StateValue {
646             num: AbstractDomain::meet(&self.num, &other.num),
647             boolean: self.boolean.meet(&other.boolean),
648             str: AbstractDomain::meet(&self.str, &other.str),
649             reference: self.reference.meet(&other.reference),
650             null: self.null && other.null,
651             undef: self.undef && other.undef,
652             setter: AbstractDomain::meet(&self.setter, &other.setter),
653             other: self.other && other.other,
654         }
655     }
656
657     fn widen(&self, other: &Self) -> Self {
658         StateValue {
659             num: self.num.widen(&other.num),
660             boolean: self.boolean.join(&other.boolean),
661             str: self.str.widen(&other.str),
662             reference: self.reference.join(&other.reference),
663             null: self.null || other.null,
664             undef: self.undef || other.undef,
665             setter: self.setter.join(&other.setter),
666             other: self.other || other.other,
667         }
668     }
669
670     fn widen_to(&self, other: &Self, thresholds: &[f64]) -> Self {
671         StateValue {
672             num: self.num.widen_to(&other.num, thresholds),
673             // Non-numeric slots have no notion of a threshold → plain widen.
674             ..self.widen(other)
675         }
676     }

```

Trois remarques de lecture :

- le join de num est hull, celui des trois booléens est le « ou », celui des autres composantes est le join de leur treillis;
- le widening n'élargit que les composantes de hauteur infinie ou grande : num (Interval::widen) et str (StrConst::widen); les autres se contentent du join;
- widen_to ne passe les seuils qu'à num et hérite du reste par la syntaxe ..self.widen(other).

Théorème 10.1: Soundness du join et du widening du produit

Pour toutes valeurs $a, b : \gamma(a) \cup \gamma(b) \subseteq \gamma(a \sqcup b)$ et $\gamma(a) \cup \gamma(b) \subseteq \gamma(a \nabla b)$. De plus, toute suite $x_0, x_1 = x_0 \nabla y_1, x_2 = x_1 \nabla y_2, \dots$ est stationnaire.

Démonstration. γ est une union, composante par composante, de concrétisations disjointes (définition 10.1). Il suffit donc que chaque composante soit sound : $\gamma_k(a_k) \cup \gamma_k(b_k) \subseteq \gamma_k(a_k \sqcup b_k)$. C'est vrai pour hull, BoolVal, StrConst, SetterVal (section 9.9), pour les trois booléens (le « ou » est l'union de $\emptyset$ et du singleton) et pour Stability (théorème 10.2). Le widening de chaque composante majore son join, d'où la seconde inclusion. Terminaison : une suite de widenings est stationnaire composante par composante, parce que num a un vrai widening (le chapitre 9 le montre), que str saute à $\top$ au-delà de quatre constantes, et que les six autres composantes sont de hauteur finie (au plus 8 pour Stability). Un produit fini de suites stationnaires est stationnaire. $\square$

Le meet est lui aussi ponctuel; il n'est employé par aucun code de production (section 9.1), ce qui compte pour la stabilité (section 10.4).

Décision : ADR-015 — produit ponctuel sur les sortes disjointes

Décide : StateValue devient le produit de huit composantes; join, meet, widen, widen_to et partial_cmp sont ponctuels; ComponentSetter devient une composante (SetterVal); l'arithmétique applique ToNumber(null) = 0. **Supprime** : TypedStateStore, StateType, infer_state_type, HookEntry::State.type_hint; useState<T>(...) devient décoratif. **Refuse** : un opérateur de réduction ou un « query pool » à la MOPSA à l'intérieur du produit, puisque des sortes disjointes n'ont aucune information à échanger; un futur produit réduit (valeur $\times$ relationnel) se fera au niveau de l'analyse, avec QueryContext comme canal de lecture (section 10.8). **Épingle** : null_init_without_hint_unbounded_is_flagged, nullable_fetch_pattern_no_false_positive.

10.2.7 from_init : un nom trompeur

Extrait 10.13 – src/domains/impls/state_value.rs, l. 289–306 — la valeur syntaxique d'un initialiseur

```

289     /// Derive the best initial abstract value from a `useState(init)` expression.
290     pub fn from_init(init: &Expr) -> Self {
291         match init {
292             Expr::Lit(Prim::Int(n)) => StateValue::number(Interval::point(*n as f64)),
293             Expr::Lit(Prim::Float(f)) => StateValue::number(Interval::point(*f)),
294             Expr::Lit(Prim::Bool(b)) => {
295                 StateValue::boolean(if *b { BoolVal::True } else { BoolVal::False })
296             }
297             Expr::Lit(Prim::String(s)) => StateValue::str_singleton(s.to_string()),
298             Expr::Lit(Prim::Null) => StateValue::null(),
299             Expr::Lit(Prim::Unit) => StateValue::undefined(),
300             Expr::ObjectLit { .. }
301             | Expr::ArrayLit { .. }
302             | Expr::FnLit { .. }
303             | Expr::New { .. } => StateValue::reference(Stability::PerRender),
304             _ => StateValue::top(),

```

```

305     }
306   }

```

Piège

Malgré son nom et sa documentation, `from_init` n'est *pas* la fonction qui amorce les slots `useState`. L'amorçage évalue l'initialiseur avec l'évaluateur complet (`transfer.eval_expr`), et exécute le corps d'un initialiseur paresseux `useState(() => expr)` au lieu d'abstraire la fermeture en référence fraîche (`src/engine/fixpoint.rs`, l. 314–353, chapitre 13). Le seul appelant hors tests de `from_init` est `returns_value` (`src/engine/written.rs`, l. 108–121), qui calcule sans environnement la valeur qu'un updater `setX(prev => ...)` rend sur ses `return`. Pour cet usage purement syntaxique, un `FnLit` rendu vaut bien `PerRender` : un updater qui rend une fonction stocke une nouvelle fonction.

10.3 La stabilité référentielle : un premier treillis, et son impasse

10.3.1 Ce que React compare

Rappel React : Identité des valeurs d'un rendu à l'autre

React compare chaque entrée d'un tableau de deps à celle du rendu précédent avec `Object.is` (comme `===`, sauf `Object.is(NaN, NaN)` vrai et `Object.is(0, -0)` faux). Un effet, un `useMemo` ou un `useCallback` est ré-exécuté si *au moins une* dep diffère. Or :

- `, [], () => , new X(), arr.map(f)` écrits dans le corps du composant allouent une nouvelle référence à chaque évaluation, donc à chaque rendu ;
- un état garde son identité d'un rendu au suivant tant que son `setter` n'est pas appelé avec une valeur différente ;
- la fonction `setter` d'un `useState` et le conteneur rendu par `useRef` sont les mêmes à chaque rendu (garantie de React) ;
- `useMemo` et `useCallback` rendent la même référence tant que leurs deps sont `Object.is`-égales.

Une primitive, elle, est comparée par valeur : 1 est toujours `Object.is`-égal à 1, quelle que soit la façon dont on l'a calculé (chapitre 2).

La *stabilité référentielle* d'une expression est la réponse abstraite à la question « sa valeur est-elle la même, au sens de `Object.is`, que celle du rendu précédent ? ». C'est le domaine qu'ADR-2 plaçait au centre de l'analyse, dès la première semaine du projet.

10.3.2 Le treillis d'ADR-002

L'ADR-002 proposait un treillis plat à quatre points, de la forme étudiée au section 9.2 : $\perp \sqsubseteq \{\text{Stable}, \text{Unstable}\} \sqsubseteq \text{Unknown}$, avec `Stable` $\sqcup$ `Unstable` = `Unknown`. Une table de transfert statique en donnait la valeur : un littéral primitif, un `setter` et `useRef()` étaient `Stable` ; un littéral objet, tableau ou fonction était `Unstable` ; la valeur d'un `useState` était le join des arguments de ses `setters` ; `useMemo` et `useCallback`, le join de leurs deps.

Décision : ADR-002 — treillis de stabilité et trois stores

Décidait : le domaine central est la stabilité (`Object.is`), treillis à quatre points ; trois stores séparés (état soumis au point fixe, mémos fonctions de leurs deps, refs triviales) parce que les trois familles de hooks n'ont pas la même sémantique vis-à-vis du cycle de rendu ; widening après un seuil configurable. **Alternative refusée** : un store unifié, qui aurait forcé

tous les hooks dans le même point fixe. **Ce qui a changé depuis**, sans que le fichier porte la mention « Superseded » : `Unstable` est devenu `PerRender` et le treillis a gagné `Versioned` et `VersionedTop` (ADR-17); il n'existe pas de `RefStore` (`useRef` rend `reference(Stable)`); les fichiers `src/domains/stability.rs` et `src/domains/product.rs` annoncés n'existent pas; le seuil de widening par défaut est 3 et non 2.

10.3.3 Le faux positif F5

Le premier banc d'essai sur un vrai corpus (« memos », 15 juillet 2026) a laissé cinq `WARNING always-unstable-deps` de la même forme, un fournisseur de contexte qui tient un état objet :

Extrait 10.14 – Un état objet en dep (`/tmp/ch10/stability.tsx`, extrait)

```
1 export function ObjectStateDep() {
2   const [ctx, setCtx] = useState({ locale: "en" });
3   useEffect(() => { console.log(ctx); }, [ctx]);
4   return <button onClick={() => setCtx({ locale: "fr" })}>fr</button>;
5 }
```

L'initialiseur est un littéral objet, donc `Unstable`; le handler écrit un autre littéral, `Unstable` aussi; le store du slot contient `Unstable`, et chaque lecture de `ctx` rendait cette valeur. La règle concluait « `ctx` est une nouvelle référence à chaque rendu ». C'est faux : `ctx` ne change que lorsqu'on clique. Le domaine confondait deux faits différents :

- la *fraîcheur à l'allocation*, un fait sur un événement : la valeur *écrite* par `setCtx(...)` est un objet neuf;
- la *fraîcheur par rendu*, un fait entre deux rendus : la valeur *lue* par `ctx` change à chaque rendu.

La première est vraie, la seconde non, et l'ancien `Unstable` disait les deux à la fois.

10.3.4 Le faux négatif caché dessous

Corriger le FP « naïvement », en rendant « plutôt stable » toute lecture d'état, aurait ouvert un faux négatif réel. L'ADR-017 le montre avec ce composant :

Extrait 10.15 – Une boucle de churn d'objet (`/tmp/ch10/stability.tsx`, extrait)

```
1 export function ObjChurn() {
2   const [obj, setObj] = useState({ a: 1 });
3   useEffect(() => { setObj({ ...obj, b: 2 }); }, [obj]);
4   return <div />;
5 }
```

L'effet écrit à chaque exécution un objet neuf dans `obj`; la dep `obj` change donc, l'effet se ré-exécute, et ainsi de suite : une vraie boucle infinie. Or le bras point fixe d'`infinite-loop` ne la voyait pas, parce que le store ne grandit pas : `Unstable ⊔ Unstable = Unstable`, rien à élargir, contrairement à un intervalle. Le *seul* signal sur ce bug était le même `WARNING always-unstable-deps` qui était un FP sur `ObjectStateDep`. Faire taire l'un sans remplacer le signal faisait disparaître l'autre, et un FN est interdit (définition 3.8). La correction F5 devait donc être double : scinder le treillis, *et* ajouter une détection du churn qui ne repose pas sur la divergence du store.

10.4 Traces de changement, bornes may et must

10.4.1 L'objet concret : la trace de changement

Pour dire proprement ce que « stable » veut dire, il faut fixer ce que l'on abstrait. Ce n'est pas une valeur, mais le comportement d'une expression au fil des rendus.

Définition 10.2: Trace de changement

Soit une expression d'un composant (une dep, une variable) évaluée aux rendus successifs $r_0, r_1, \dots, r_n$, avec les valeurs $v_0, \dots, v_n$. Sa *trace de changement* est l'ensemble des rendus où `Object.is` échoue :

$$C = \{i \in \{1, \dots, n\} \mid \neg \text{Object.is}(v_i, v_{i-1})\}.$$

Pour un ensemble S de slots d'état, on note $\text{Set}(S)$ l'ensemble des rendus r_i ($i \geq 1$) précédés, depuis r_{i-1} , d'au moins un appel au setter d'un slot de S .

La trace de changement est exactement ce que React observe : les deps d'un effet, l'invalidation d'un mémo et la propagation d'un contexte ne dépendent que d'elle.

Exemple 10.3: Quatre traces

- `const r = useRef()` : $C = \emptyset$ (même conteneur à chaque rendu).
- `const o = a : 1` dans le corps : $C = \{1, \dots, n\}$ (un objet neuf à chaque rendu).
- `ctx` lu depuis `useState( locale: "en" )` dans `ObjectStateDep` : $C \subseteq \text{Set}(\{\text{ctx}\})$; il ne change qu'aux clics.
- `const v = flag ? a : 2` : `obj`, avec `obj` un état : tantôt neuf à chaque rendu, tantôt versionné; on ne peut rien borner.

10.4.2 Deux bornes, deux usages

Sur l'ensemble C , deux informations utiles coexistent. Elles ne sont pas opposées : ce sont deux *bornes* de nature différente, au sens de la définition 3.7.

- Une borne *may* (sur-approximation) : « $C \subseteq X$ », par exemple « ne change *qu'aux* sets de S ». C'est un fait de sûreté. Les règles s'en servent pour *se taire* à bon droit : une dep versionnée ne ré-exécute pas l'effet à chaque rendu.
- Une borne *must* (sous-approximation) : « $C \supseteq Y$ », ici « change à *chaque* rendu ». C'est un fait de certitude. Les règles s'en servent pour *tirer*, jusqu'au niveau ERROR.

Chaque borne prend trois valeurs, ordonnées : Never ($C = \emptyset$), OnSet ($\text{Set}(S)$), Every (tous les rendus). Une paire (may, must) cohérente vérifie $\text{must} \leq \text{may}$. Le figure 10.2 place les six paires possibles.

Les deux points écartés n'avaient qu'un consommateur prospectif, la détection du churn. Or celle-ci a de toute façon besoin d'une analyse d'atteignabilité du corps de l'effet (le set a-t-il lieu sur tous les chemins?), qui n'est pas une propriété de valeur. Elle vit donc dans la règle (chapitre 19) et le graphe de churn (chapitre 15), pas dans le treillis. L'ADR-017 garde le produit complet comme cadre d'extension si un second consommateur apparaît.

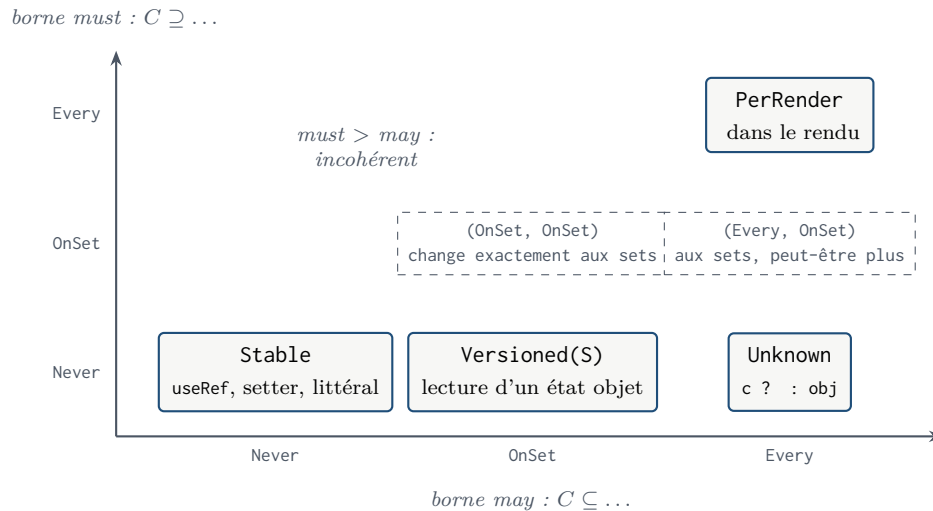


FIGURE 10.2 – Le carré may/must de l'ADR-017. Chaque point est une paire (borne may, borne must) sur la trace de changement C ; seules les paires avec $\text{must} \leq \text{may}$ ont un sens. Les quatre points encadrés en bleu sont retenus (avec $\perp$, non figuré) ; les deux points en pointillés, « doit changer aux sets », sont écartés. Le join affaiblit les deux bornes : il va vers la droite et vers le bas.

10.4.3 Le treillis Stability

Définition 10.3: Treillis de stabilité versionnée

Le treillis *Stability* a pour éléments $\perp$, *Stable*, *Versioned*(S) pour tout ensemble S de slots qualifiés avec $1 \leq |S| \leq 4$, *VersionedTop*, *PerRender* et *Unknown* ($\top$). Sa concrétisation, en termes de traces de changement (définition 10.2) :

$$\begin{aligned}
 \gamma(\perp) &= \emptyset \\
 \gamma(\text{Stable}) &= \{ \text{traces avec } C = \emptyset \} \\
 \gamma(\text{Versioned}(S)) &= \{ \text{traces avec } C \subseteq \text{Set}(S) \} \\
 \gamma(\text{VersionedTop}) &= \{ \text{traces avec } C \subseteq \text{Set}(S) \text{ pour un certain ensemble } S \} \\
 \gamma(\text{PerRender}) &= \{ \text{traces avec } C = \{1, \dots, n\} \} \\
 \gamma(\text{Unknown}) &= \text{toutes les traces}
 \end{aligned}$$

L'ordre est $\perp \sqsubseteq \text{Stable} \sqsubseteq \text{Versioned}(S) \sqsubseteq \text{Versioned}(T) \sqsubseteq \text{VersionedTop} \sqsubseteq \text{Unknown}$ pour $S \subseteq T$, et $\perp \sqsubseteq \text{PerRender} \sqsubseteq \text{Unknown}$; *PerRender* est incomparable avec *Stable*, les *Versioned* et *VersionedTop*.

Les quatre noms se lisent sur le carré : *Stable* est (Never, Never), *Versioned* est (OnSet, Never), *PerRender* est (Every, Every), *Unknown* est (Every, Never). *Versioned* est une borne may ; *PerRender* est une borne must. Le code transcrit la définition, avec un commentaire qui reproduit le diagramme de Hasse :

Extrait 10.16 – `src/domains/impls/stability.rs`, l. 9–52 — le treillis de stabilité versionnée

```

9  /// Threshold on `Versioned` label sets before widening to `VersionedTop`
10 /// (same pattern as `StrConst`).
11 pub const VERSIONED_LABELS_THRESHOLD: usize = 4;
12
13 /// Stability lattice: bounds on the *change trace* of a value – the set of
14 /// renders where `Object.is(v_i, v_{i-1})` fails (the only thing React observes).
15 ///
```

```

16 /// Two kinds of bounds coexist (ADR-017):
17 /// - may bound (over-approx): used by rules to stay silent soundly.
18 /// - must bound (under-approx): used by rules to fire without FPs.
19 ///
20 /// ```text
21 ///           Unknown  (⊤)
22 ///           /       \
23 ///   VersionedTop   PerRender
24 ///       |           |
25 ///   Versioned(S) ⊆-chains |
26 ///       |           |
27 ///     Stable        |
28 ///       \           /
29 ///       Bottom  (⊥)
30 /// ```
31 ///
32 /// `Versioned`, `VersionedTop` and `PerRender` are incomparable – not
33 /// opposites, different bounds: `join = Unknown` (guarantees nothing in
34 /// either direction).
35 #[derive(Debug, Clone, PartialEq, Eq)]
36 pub enum Stability {
37     /// ⊥ no information (unreachable path / uninitialized).
38     Bottom,
39     /// Never changes: the same reference on every render (safe as a dep).
40     Stable,
41     /// Changes only at setter events of these state slots (may bound).
42     /// Invariant: non-empty (canonicalised – `Versioned(⌀) ≡ Stable`) and
43     /// `len() ≤ VERSIONED_LABELS_THRESHOLD` (widened to `VersionedTop` above).
44     Versioned(BTreeSet<QualifiedSlot>),
45     /// Versioned by unknown state slots (threshold-widened `Versioned`).
46     VersionedTop,
47     /// A fresh reference every render, guaranteed (must bound).
48     /// For non-reference kinds via `to_stability`: "may change every render".
49     PerRender,
50     /// ⊤ no bound in either direction.
51     Unknown,
52 }

```

Un slot qualifié `QualifiedSlot` est un couple `(ComponentId, HookLabel)` (`src/ir/types.rs`, l. 6–9) : le label d'un hook n'est unique qu'au sein de son composant, et un `Versioned` peut traverser les frontières de composants par les props. L'ADR-017 écrit encore `(Symbol, HookLabel)`; le passage à `ComponentId` date de #7. Le figure 10.3 redessine le diagramme.

10.4.4 Canonisation

Extrait 10.17 – `src/domains/impls/stability.rs`, l. 54–70 — les constructeurs canonisants

```

54 impl Stability {
55     /// Canonicalising constructor:  $\emptyset \rightarrow \text{Stable}$ , over-threshold  $\rightarrow \text{VersionedTop}$ .
56     pub fn versioned(labels: BTreeSet<QualifiedSlot>) -> Self {
57         if labels.is_empty() {
58             Stability::Stable
59         } else if labels.len() > VERSIONED_LABELS_THRESHOLD {
60             Stability::VersionedTop
61         } else {
62             Stability::Versioned(labels)
63         }
64     }
65
66     /// Single-slot `Versioned`.

```

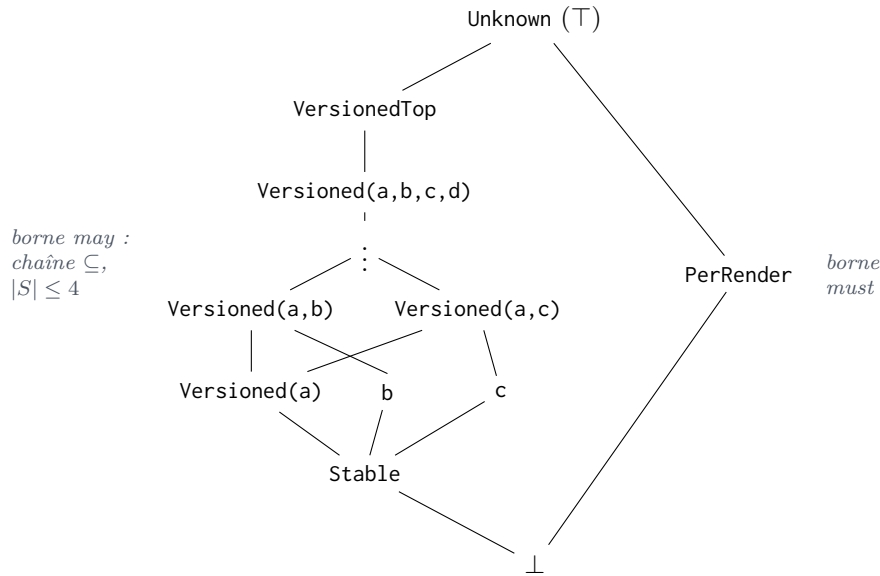


FIGURE 10.3 – Diagramme de Hasse de Stability (d’après le commentaire de `src/domains/impls/stability.rs`, l. 20–30). À gauche, la chaîne `may` ordonnée par inclusion des ensembles de slots, bornée par `VERSIONED_LABELS_THRESHOLD = 4`; à droite, le fait `must` `PerRender`. Les deux branches ne se rejoignent qu’en $\top$ et en $\perp$. Les étiquettes `b` et `c` abrègent `Versioned(b)` et `Versioned(c)`.

```

67 pub fn versioned_by(component: ComponentId, label: HookLabel) -> Self {
68     Stability::Versioned(BTreeSet::from([(component, label)]))
69 }
70 }

```

Invariant 10.2: Forme canonique de Versioned

Un `Versioned(S)` a toujours $1 \leq |S| \leq 4$. « Versionné par rien » veut dire « ne change jamais » : `Versioned(∅) ≡ Stable`; au-delà de quatre slots, la valeur est élargie à `VersionedTop`. L’invariant est garanti par `versioned` (qu’emploient `join` et `meet`) et par `versioned_by` (un singleton). Le variant étant public, rien n’empêche un appelant de construire `Versioned` à la main; aucune assertion ne le vérifie.

Le seuil reprend le motif de `StrConst` (section 9.3) : un ensemble borné, élargi à son $\top$ propre au-delà de quatre éléments.

10.4.5 Ordre, join, meet, widen

Extrait 10.18 – `src/domains/impls/stability.rs`, l. 72–101 — l’ordre de Stability

```

72 impl PartialOrd for Stability {
73     fn partial_cmp(&self, other: &Self) -> Option<Ordering> {
74         use Stability::*;
75         match (self, other) {
76             (a, b) if a == b => Some(Ordering::Equal),
77             (Bottom, _) => Some(Ordering::Less),
78             (_, Bottom) => Some(Ordering::Greater),
79             (_, Unknown) => Some(Ordering::Less),
80             (Unknown, _) => Some(Ordering::Greater),
81             // Stable ⊆ Versioned ⊆ VersionedTop (behaviour-set inclusion:

```



```

82      // "never changes"  $\subset$  "changes only at sets".
83      (Stable, Versioned(_) | VersionedTop) => Some(Ordering::Less),
84      (Versioned(_) | VersionedTop, Stable) => Some(Ordering::Greater),
85      (Versioned(s), Versioned(t)) => {
86          if s.is_subset(t) {
87              Some(Ordering::Less) // s  $\neq$  t here (equal case above)
88          } else if t.is_subset(s) {
89              Some(Ordering::Greater)
90          } else {
91              None
92          }
93      }
94      (Versioned(_, VersionedTop) => Some(Ordering::Less),
95      (VersionedTop, Versioned(_)) => Some(Ordering::Greater),
96      // PerRender vs Stable/Versioned/VersionedTop: incomparable
97      // (different bounds – may vs must).
98      _ => None,
99  }
100 }
101 }

```

L'ordre se lit de haut en bas : l'égalité d'abord, puis $\perp$ et Unknown aux extrémités, puis la chaîne may (Stable sous tout Versioned, inclusion stricte des ensembles, tout Versioned sous VersionedTop); tout le reste, c'est-à-dire PerRender face à la chaîne may, et deux ensembles non inclus l'un dans l'autre, est incomparable (None).

Extrait 10.19 – src/domains/impls/stability.rs, l. 108–146 — join, meet et widen de Stability

```

108  /// Least upper bound ( $\sqcup$ ).
109  pub fn join(&self, other: &Self) -> Self {
110      use Stability::*;
111      match (self, other) {
112          (a, b) if a == b => a.clone(),
113          (Bottom, x) | (x, Bottom) => x.clone(),
114          (Unknown, _) | (_, Unknown) => Unknown,
115          (Stable, v @ (Versioned(_) | VersionedTop))
116          | (v @ (Versioned(_) | VersionedTop), Stable) => v.clone(),
117          (Versioned(s), Versioned(t)) => Stability::versioned(s.union(t).cloned().collect()),
118          (VersionedTop, Versioned(_)) | (Versioned(_), VersionedTop) => VersionedTop,
119          // {Stable, Versioned, VersionedTop}  $\sqcup$  PerRender = Unknown
120          _ => Unknown,
121      }
122  }
123
124  /// Greatest lower bound ( $\sqcap$ ).
125  pub fn meet(&self, other: &Self) -> Self {
126      use Stability::*;
127      match (self, other) {
128          (a, b) if a == b => a.clone(),
129          (Unknown, x) | (x, Unknown) => x.clone(),
130          (Bottom, _) | (_, Bottom) => Bottom,
131          (Stable, Versioned(_) | VersionedTop) | (Versioned(_) | VersionedTop, Stable) => Stable,
132          (Versioned(s), Versioned(t)) => {
133              let inter: BTreeSet<_> = s.intersection(t).cloned().collect();
134              Stability::versioned(inter) //  $\emptyset$  canonicalises to Stable
135          }
136          (VersionedTop, v @ Versioned(_)) | (v @ Versioned(_), VersionedTop) => v.clone(),
137          // {Stable, Versioned, VersionedTop}  $\sqcap$  PerRender = Bottom
138          _ => Bottom,
139      }
140  }

```

```

141
142    /// Widening: join, whose `Versioned` union is already threshold-bounded -
143    /// chains have height  $\leq$  threshold + 4.
144    pub fn widen(&self, other: &Self) -> Self {
145        self.join(other)
146    }

```

Ces méthodes sont *inhérentes*; l'implémentation du trait `AbstractDomain` leur délègue (`src/domains/impls/stability.rs`, l. 149–168). Comme Rust préfère la méthode inhérente à la méthode de trait à la résolution de `self.join(other)`, il n'y a pas de récursion.

Théorème 10.2: Soundness du join de Stability

Pour tous $a, b : \gamma(a) \cup \gamma(b) \subseteq \gamma(a \sqcup b)$.

Démonstration. Par cas, dans l'ordre des bras du code. *Égalité* : $a \sqcup a = a$. $\perp : \gamma(\perp) = \emptyset$, donc $\gamma(\perp) \cup \gamma(x) = \gamma(x)$. *Unknown* : $\gamma(\text{Unknown})$ contient toutes les traces. *Stable et Versioned(S)* : une trace avec $C = \emptyset$ vérifie $C \subseteq \text{Set}(S)$; même argument avec *VersionedTop*. *Versioned(S) et Versioned(T)* : *Set* est croissant, donc $C \subseteq \text{Set}(S)$ ou $C \subseteq \text{Set}(T)$ entraîne $C \subseteq \text{Set}(S \cup T)$; si $|S \cup T| > 4$, *versioned* rend *VersionedTop*, dont la concrétisation contient celle de *Versioned(S \cup T)*; et $S \cup T$ n'est jamais vide. *VersionedTop et Versioned* : immédiat. *Dernier bras* (*PerRender* face à la chaîne *may*) : le résultat est *Unknown*, qui contient tout. $\square$

Proposition 10.2: Borne supérieure, hauteur et terminaison

Sous l'invariant 10.2, $a \sqcup b$ est la plus petite borne supérieure de a et b dans l'ordre de liste 10.18. Toute chaîne strictement croissante a au plus 8 éléments ($\perp \sqsubset \text{Stable} \sqsubset$ quatre *Versioned* de tailles 1 à 4 $\sqsubset \text{VersionedTop} \sqsubset \text{Unknown}$), donc *widen* = *join* termine.

Démonstration. Seuls deux cas ne sont pas évidents. *Stable $\sqcup$ PerRender* : le seul majorant commun est *Unknown*, puisque *PerRender* n'est sous rien d'autre. *Versioned(S) $\sqcup$ Versioned(T)* avec $|S \cup T| > 4$: un majorant *Versioned(U)* exigerait $U \supseteq S \cup T$, donc $|U| > 4$, ce que l'invariant interdit; restent *VersionedTop* et *Unknown*, et le plus petit est *VersionedTop*. $\square$

Piège

$\text{Versioned}(S) \sqcap \text{PerRender} = \perp$ est bien la borne inférieure *dans l'ordre abstrait*, mais les concrétisations s'intersectent : une trace où chaque rendu est précédé d'un set d'un slot de S change à chaque rendu *et* seulement aux sets. Le *meet* est donc strictement plus petit que l'intersection des concrétisations; l'utiliser pour raffiner une valeur ferait disparaître des comportements réels, un FN. Il n'est appelé par aucun code de production au commit e67b10a; ne pas s'en servir sans revoir ce point.

Décision : ADR-017 — stabilité versionnée, bornes may et must

Contexte : les cinq FP F5 sur des états objets, et le FN *ObjChurn* que leur correction naïve aurait ouvert. **Décide** : (1) le treillis ci-dessus; (2) une conversion « côté lecture » en un seul point, la lecture d'un slot (section 10.5); (3) un bras *churn* dans *infinite-loop* (ERROR si la dep *est* le slot, que le set a lieu sur tous les chemins et que la valeur écrite est *PerRender*; WARNING si la dep est *Versioned(S)* avec le slot dans S); (4) le recâblage des consommateurs (*always-unstable-deps* ne tire plus que sur *PerRender*). **Alternatives refusées** : encoder un compteur d'événements dans le store pour faire tirer le widening (« non-standard hack »); un nouveau nom de diagnostic (même classe de panne); garder les deux points « doit changer aux sets » (un seul consommateur, qui a de toute façon besoin

d'atteignabilité). **Coût accepté** : *Stability* n'est plus *Copy* (il possède un *BTreeSet*) ; les erreurs de compilation ont forcé l'audit de tous les consommateurs.

10.5 Deux vues d'un même état : la conversion côté lecture

10.5.1 Le problème

Le treillis sait maintenant dire « ne change qu'aux sets de *ctx* ». Reste à savoir *où* ce fait apparaît. Dans *ObjectStateDep*, le store du slot reçoit deux écritures : le littéral d'initialisation et celui du handler, deux objets neufs. Son contenu convergé est donc, à juste titre, *ref(PerRender)* : chaque valeur *écrite* est une allocation fraîche. La sonde le confirme :

```
-- ObjectStateDep
state[0] = ref(PerRender) widened=false num=Interval { lo: inf, hi: -inf, is_int: true }
```

Si la lecture *ctx* rendait cette valeur telle quelle, on retomberait sur le FP F5. Il faut que ce qu'un rendu *lit* dise autre chose que ce que les événements *écrivent*.

10.5.2 Le code : un seul point de conversion

Le fait React qui tranche est celui-ci : un slot d'état ne change *que* par son setter. Toute lecture de *StateVal(1)* a donc la borne may « ne change qu'aux sets de *l* », quelles que soient les valeurs écrites. L'évaluateur le traduit à un seul endroit :

Extrait 10.20 – *src/domains/transfer/state_value.rs*, l. 122–135 — la conversion côté lecture (ADR-017)

```
122 Expr::StateVal(label) => {
123     let mut val = ctx.state.get(*label);
124     // ADR-017 read-side conversion: the store holds the join of
125     // *written* values (event view); what a render *reads* can only
126     // change at setter events of this slot (cross-render view). The
127     // written value's allocation freshness (`PerRender`) must not
128     // leak into reads. Assumes sets happen outside render – the
129     // violation has its own diagnostic (`setter-in-render`).
130     if val.reference != Stability::Bottom {
131         val.reference = Stability::versioned_by(ctx.component, *label);
132     }
133     val
134 }
135 Expr::StateSetter(label) => StateValue::component_setter(ctx.component, *label),
```

Si la composante *reference* de la valeur stockée est peuplée, elle est remplacée par *Versioned((composant, 1))* ; les autres composantes passent inchangées. Le champ *ctx.component* de l'*AnalysisCtx* (section 10.8) fournit le propriétaire du slot. Une lecture dont la composante *reference* est $\perp$ (un état purement numérique, par exemple) n'est pas convertie : elle ne porte aucune étiquette de version, ce qui aura une conséquence au section 10.9. Tout ce qui est en aval hérite de la conversion gratuitement : liaisons d'environnement (*const o = obj*), évaluation des deps, props passées à un enfant.

Le chapitre 2 a énoncé l'invariant qui en découle, la *double vue de l'état* (invariant 2.1) ; le figure 10.4 le met en image.

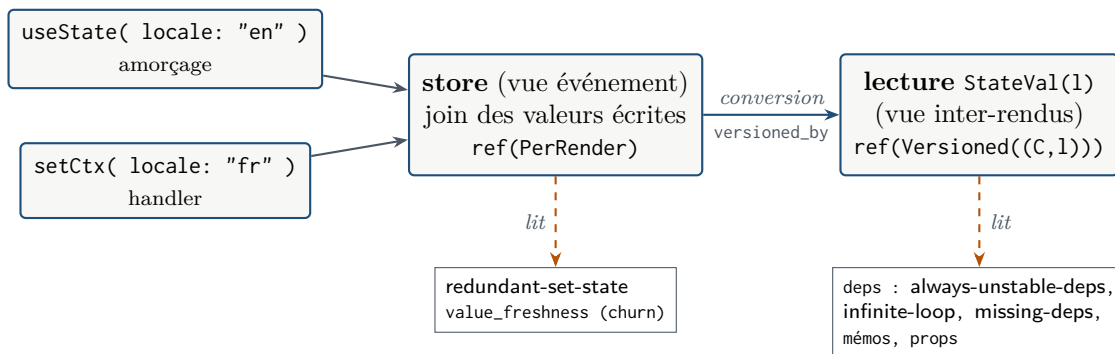


FIGURE 10.4 – La double vue de l’état sur ObjectStateDep. Le store range le join des valeurs écrites, deux allocations fraîches ; toute lecture du slot rend la vue *inter-rendus*, ce que Object.is compare d’un rendu à l’autre. Les consommateurs qui comparent des valeurs écrites lisent le store ; ceux qui raisonnent sur les deps lisent l’évaluation.

Proposition 10.3: Soundness de la conversion côté lecture

Sous l’hypothèse (H1) « aucun setter n’est appelé pendant le rendu », la trace de changement de toute lecture du slot l vérifie $C \subseteq \text{Set}(\{l\})$: la composante Versioned($\{l\}$) est une borne may correcte.

Démonstration. La valeur d’un état au rendu r_i est la valeur de son slot, que seuls les appels à son setter modifient (en file d’attente, traités au rendu suivant). Si r_i n’est précédé, depuis r_{i-1} , d’aucun appel au setter de l , et qu’aucun appel n’a eu lieu pendant r_{i-1} lui-même (H1), la valeur est la même référence, et Object.is réussit : $i \notin C$. $\square$

L’ADR-017 nomme les deux hypothèses dont dépend cette conversion, et elles sont couvertes chacune à part :

1. *H1, les sets ont lieu hors rendu.* Un setter appelé pendant le rendu avec une référence fraîche fait vraiment changer le slot à chaque rendu, ce que Versioned sous-estime. Cette violation a son propre diagnostic, **setter-in-render**, de niveau ERROR : c’est la soundness en couches du chapitre 2 (définition 2.2).
2. *H2, le couplage avec le bras churn.* infinite-loop accepte une dep Versioned comme « garde » qui empêche l’effet de tourner à chaque rendu. C’est sound *seulement* parce que le bras churn traite le cas où l’effet écrit lui-même une référence fraîche dans le slot qui le versionne. Retirer l’un sans l’autre rouvre le FN ObjChurn.

Exemple 10.4: Les cinq composants de stability.tsx

Le fichier /tmp/ch10/stability.tsx réunit ObjectStateDep, ObjChurn et trois composants de la section 10.6 (InlineObjectDep, MemoizedDep, CallbackDep) :

```

$ reactant check /tmp/ch10/stability.tsx --show-clean
CallbackDep (4 hooks) /tmp/ch10/stability.tsx ✓
InlineObjectDep (1 hooks) /tmp/ch10/stability.tsx
  warn  always-unstable-deps [hook:0] (line 5:2) this effect depends on `opts`, a new reference every render,
  ↳ so `Object.is` always differs and the effect re-runs on every render regardless of the other deps
  (1 trace step(s), rerun with --trace)
MemoizedDep (2 hooks) /tmp/ch10/stability.tsx ✓
ObjChurn (2 hooks) /tmp/ch10/stability.tsx
  error infinite-loop [hook:0] (line 30:2) this effect recreates object state `obj` it depends on. Every run
  ↳ stores a fresh reference (`Object.is` always fails) and re-triggers itself: infinite render loop
  (1 trace step(s), rerun with --trace)
ObjectStateDep (3 hooks) /tmp/ch10/stability.tsx ✓
  
```

```
! 1 error(s), 1 warning(s) across 1 file(s).
```

ObjectStateDep est propre : sa dep lit Versioned, et always-unstable-deps exige PerRender. Le FP F5 est mort, et ObjChurn est passé d'un WARNING noyé dans les FP à une ERROR.

10.5.3 Le churn d'objet, certifié par la structure

Sur ObjChurn, le store vaut aussi ref(PerRender), et la lecture obj vaut Versioned((C,0)). Rien ne s'élargit (sonde : widened=false), donc le bras point fixe d'infinite-loop ne peut rien voir. La certitude ne vient pas non plus de Versioned : c'est une borne may (« ne change *qu'aux* sets »), pas « change à chaque set ». Elle vient de la *structure* :

- la dep obj *est* le slot, sans accès de champ (must-change dès que la valeur écrite est fraîche) ;
- la valeur écrite ...obj, b: 2 est un ObjectLit, donc PerRender au sens must, et l'écriture a lieu sur tous les chemins du corps de l'effet (must-reach) ;
- React ré-exécute un effet dont une dep a changé (must-rerun).

Trois faits must, aucun fait may : une ERROR (chapitre 19). Côté moteur, c'est value_freshness qui classe la valeur écrite pour le graphe de churn (chapitre 15) ; elle lit la vue événement :

Extrait 10.21 – src/engine/written.rs, l. 173–200 — la fraîcheur d'une valeur écrite, lue sur la composante référence

```
173 /// Freshness of a value stored into `target`. Churn is about the REFERENCE
174 /// kind only: a widened numeric value (`count + 1`) changes but never fails
175 /// `Object.is` freshly.
176 ///
177 /// A reference versioned by the target slot alone is that slot's own content
178 /// – or something that moves only with it, the reading ADR-017 gives
179 /// `Versioned` – whatever residue the value carries: a state read of a T
180 /// store keeps every kind T beside the `Versioned` reference the read-side
181 /// conversion gives it, and written back it is not a change (#155). A value
182 /// versioned by *another* slot with a T residue stays `Maybe`, the reading
183 /// it had: copying slot X into slot Y changes Y whenever X moves, and #157 is
184 /// the precise account of that.
185 fn value_freshness(val: &StateValue, target: QualifiedSlot) -> Freshness {
186     match &val.reference {
187         Stability::PerRender => {
188             if val.is_unstable_reference_only() {
189                 Freshness::Fresh
190             } else {
191                 Freshness::Maybe // joined with other kinds
192             }
193         }
194         Stability::Unknown => Freshness::Maybe,
195         Stability::Versioned(by) if by.len() == 1 && by.contains(&target) => Freshness::Not,
196         // Stable / Versioned / ⊥ reference; residual T stays Maybe.
197         _ if val.other => Freshness::Maybe,
198         _ => Freshness::Not,
199     }
200 }
```

Le commentaire d'en-tête dit l'essentiel : le churn ne concerne que la sorte référence, parce qu'une valeur numérique élargie (count + 1) change mais n'échoue jamais Object.is « par fraîcheur ». Une valeur versionnée par le slot cible seul est son propre contenu : la réécrire n'est pas un changement (#155).

Piège

L'inverse n'est pas vrai : une valeur versionnée par le slot cible mais *dérivée* de lui (un mémo `useMemo(() => ( n: s.n + 1 ), [s])` réécrit dans `s`) est lue « non fraîche », alors qu'elle change exactement quand le cycle tourne. C'est un FN connu, ouvert sous l'étiquette `soundness-bug (#157)` ; sa correction demande un point « exactement le contenu du slot `q` » sous `Versioned({q})` dans le treillis, dégradé en `Versioned` par toute transformation. En attendant, la lecture d'ADR-017 tient.

10.5.4 Lire un champ d'un objet versionné

Un accès `obj.field` commence par le tas : si le récepteur désigne des objets alloués connus, la valeur est le join des champs lus (chapitre 11). Sinon, l'évaluateur garde les étiquettes de version du récepteur plutôt que de tomber à $\top$:

Extrait 10.22 – `src/domains/transfer/state_value.rs`, l. 617–628 — un champ d'objet versionné garde les étiquettes

```

617 // A field of a versioned object can change only at the object's own
618 // setter events: keep the version labels instead of degrading to  $\top$ 
619 // (the field's *kind* stays unknown – every other slot is  $\top$ ). Same
620 // no-mutation-during-render assumption as the `StateVal` read-side
621 // conversion above; in-place writes have their own diagnostics
622 // (`state-mutation`). ADR-017 §Limitations, member-deps.
623 let obj_val = eval_state_value(obj, env, ctx);
624 StateValue {
625     reference: obj_val.versioned_reference().unwrap_or(Stability::Unknown),
626     ..StateValue::top()
627 }
628 }
```

La *sorte* du champ reste inconnue (toutes les autres composantes sont à $\top$), mais sa borne may survit : un champ d'un état objet ne peut changer qu'aux sets de cet état, sous la même hypothèse « pas de mutation pendant le rendu » que la lecture d'un slot (les mutations en place ont leur diagnostic, `state-mutation`). Une `dep obj.field` ne peut toutefois donner qu'un `WARNING` dans le bras `churn : setObj(...obj, other: 2)` préserve la référence du champ, donc la `dep` peut ne pas changer.

10.6 Littéraux, `useRef`, `useMemo`, `useCallback`

10.6.1 Où naît une référence

La composante `reference` n'est peuplée que par un petit nombre de formes, toutes dans l'évaluateur :

Extrait 10.23 – `src/domains/transfer/state_value.rs`, l. 153–156 — les littéraux alloués et les éléments natifs

```

153 Expr::ObjectLit { .. } => StateValue::reference(Stability::PerRender),
154 Expr::ArrayLit { .. } => StateValue::reference(Stability::PerRender),
155 Expr::FnLit { .. } => StateValue::reference(Stability::PerRender),
156 Expr::NativeElem { .. } => StateValue::reference(Stability::Stable),
```

Extrait 10.24 – `src/domains/transfer/state_value.rs`, l. 176–181 — appels et `new`

```

176 Expr::Call { fn_, .. } if returns_fresh_reference(fn_) => {
```

```

177     StateValue::reference(Stability::PerRender)
178   }
179   Expr::Call { .. } => StateValue::top(),
180   // `new` allocates: a fresh reference, whose members stay  $\top$  (#158).
181   Expr::New { .. } => StateValue::reference(Stability::PerRender),

```

Le tableau 10.1 rassemble ces sources.

Expression	Valeur	Raison
..., [...], () => ...	ref(PerRender)	allocation à chaque évaluation (fait must)
new X(...)	ref(PerRender)	idem, depuis e67b10a (#158)
arr.map(f), filter, flat, split, structuredClone(x), JSON.parse...	ref(PerRender)	returns_fresh_reference
s.slice(...), concat	$\top$	(src/domains/transfer/state_value.rs, l. 238–259) peut rendre une chaîne, donc une primitive (#22)
autre appel	$\top$	rien n'est connu du résultat
élément JSX natif	ref(Stable)	non documenté, un FN (section 10.9.4)
useRef()	ref(Stable)	même conteneur à chaque rendu
setX (setter)	setter(...)	identité garantie par React
lecture StateVal(1)	Versioned(1) sur reference	conversion côté lecture
useMemo / useCallback	ref(...)	recompute_memo, ci-dessous

TABLE 10.1 – Les sources de la composante reference.

useRef passe par un marqueur de hook dont le commentaire explique pourquoi il est une référence et pas undefined :

Extrait 10.25 – src/domains/transfer/state_value.rs, l. 142–147 — les marqueurs de hooks

```

142   Expr::HookMarker(_, MarkerVal::Undefined) => StateValue::undefined(),
143   Expr::HookMarker(_, MarkerVal::Unknown) => StateValue::top(),
144   // `useRef` hands back the same container every render – a reference,
145   // and a stable one. Both halves matter: `undefined` was stable too, but
146   // it was not a reference, so the identity was invisible.
147   Expr::HookMarker(_, MarkerVal::StableRef) => StateValue::reference(Stability::Stable),

```

Le commentaire qui précède (src/domains/transfer/state_value.rs, l. 137–141) mérite d'être retenu : lire le résultat d'un hook personnalisé non résolu comme undefined le rendait *prouvablement stable* (to_stability joint Stable pour undef) et faisait taire toutes les règles de stabilité, un FN. Il vaut donc $\top$.

10.6.2 La valeur d'un mémo : recompute_memo

useMemo(f, deps) et useCallback(f, deps) rendent la même référence tant que leurs deps ne changent pas. Leur stabilité est donc le join des stabilités de leurs deps. Le moteur la recalcule par la méthode recompute_memo du trait Transfer :

Extrait 10.26 – src/domains/transfer/state_value.rs, l. 56–101 — la valeur d'un mémo, stabilité de ses deps

```

56   fn recompute_memo(
57     &self,
58     component: ComponentId,
59     deps: &DepsArg,
60     env: &AbstractEnv<StateValue>,

```

```

61     ctx: &mut AnalysisCtx<StateValue>,
62   ) -> StateValue {
63     // No readable deps array: the memo may recompute on any render and may
64     // equally never recompute –  $\perp$ . `Stable` here would be a *must* claim
65     // about a list the IR never saw, which is how `useMemo(fn, deps)` used
66     // to read as pinned forever.
67     let Some(deps) = deps.list() else {
68       return StateValue::reference(Stability::Unknown);
69     };
70     if deps.arity == Arity::Exact(0) {
71       // `[]` pins the memo – but only an array *known* to be empty.
72       return StateValue::reference(Stability::Stable);
73     }
74     if deps.is_empty() {
75       // Every entry is a spread whose source the fold cannot reach.
76       return StateValue::reference(Stability::Unknown);
77     }
78     let stability = deps.as_slice().iter().fold(Stability::Bottom, |acc, dep| {
79       // Structural projection (ADR-017): a dep that IS a state slot
80       // versions the memo by that slot regardless of the slot's kind –
81       // it changes only at the slot's setter events, `Versioned({1})`.
82       // `eval_state_value` can't express this: version labels live on
83       // the reference slot, so a numeric/bool state dep would collapse
84       // to a plain `Stable`/`PerRender` via `to_stability`. Not a store
85       // workaround – a genuine memo-side projection.
86       if let Expr::StateVal(1) = dep.peel_ts() {
87         return acc.join(&Stability::versioned_by(component, *1));
88       }
89       // Every other dep is evaluated through the normal path against the
90       // real fixpoint stores in `ctx` (so `MemoVal`, heap fields, and
91       // compound deps resolve instead of reading a fabricated  $\perp$  store).
92       let val = eval_state_value(dep, env, ctx);
93       // A dep whose reference kind is already versioned keeps its
94       // labels (`to_stability` would erase them if another slot is  $\perp$ ).
95       acc.join(
96         &val.versioned_reference()
97         .unwrap_or_else(|| val.to_stability()),
98       )
99     });
100     StateValue::reference(stability)
101   }

```

Étape par étape :

1. pas de tableau lisible (`deps.list()` vaut `None`) : `Unknown`. `Stable` serait une affirmation *must* sur une liste que l'IR n'a jamais vue ;
2. un tableau *connu* vide (`Arity::Exact(0)`, section 6.4.4) : `Stable`, le mémo est calculé une fois ;
3. un tableau dont toutes les entrées sont des spreads illisibles : `Unknown` ;
4. sinon, un pli (fold) par join sur les deps, depuis $\perp$: une dep qui *est* syntaxiquement `StateVal(1)` donne `versioned_by(component, 1)` quelle que soit la sorte du slot ; toute autre dep est évaluée normalement, et contribue sa composante versionnée si elle en a une, sa projection `to_stability` sinon.

Le résultat est rangé dans la composante `reference` d'une valeur dont tout le reste est $\perp$.

Exemple 10.5: Trois mémos

Sur `stability.tsx`, `MemoizedDep` écrit `useMemo(() => ( a: 1 ), [])`; `CallbackDep` écrit `useCallback(() => console.log(q), [q])` avec `q` un état objet mis à jour par un handler; et sur `memo.tsx`, `MemoOverObjState` écrit `useMemo(() => ( v: o.a ), [o])`. La sonde :

```
-- MemoizedDep
memo[0] (Memo) = ref(Stable)
-- CallbackDep
state[0] = ref(PerRender) widened=false num=Interval { lo: inf, hi: -inf, is_int: true }
memo[1] (Callback) = ref(Versioned({(ComponentId(4294967295), 0)}))
-- MemoOverObjState
state[0] = ref(PerRender) widened=false num=Interval { lo: inf, hi: -inf, is_int: true }
memo[1] (Memo) = ref(Versioned({(ComponentId(4294967295), 0)}))
```

Les deux derniers illustrent la double vue : le store de l'état objet vaut `ref(PerRender)`, mais la dep lue est versionnée, et le mémo hérite de l'étiquette. Un effet qui dépend de `onLoad` ou de `a` est donc « ne change qu'aux sets du slot 0 », et les trois composants sont propres.

La propagation des étiquettes le long d'une chaîne de mémos est gratuite : `Versioned({X})` $\sqcup$ `Stable = Versioned({X})`. L'ADR-017 exige seulement que `to_stability` reste hors de ce chemin, parce qu'il effacerait les étiquettes dès qu'une autre composante est $\top$: c'est le rôle de `versioned_reference` (section 10.7).

Piège

La valeur d'un mémo n'est *que* sa stabilité, logée dans la composante `reference`. La sorte réelle du résultat (un nombre, un objet) est perdue : `useMemo(() => 1, [])` vaut `ref(Stable)` et non le nombre 1. Avec une dep fraîche, la perte devient visible :

Extrait 10.27 – Un mémo primitif sur une dep fraîche (`/tmp/ch10/memo.tsx`, extrait)

```
1 export function MemoPrimitiveOverFreshDep() {
2   const cfg = { k: 1 };
3   const one = useMemo(() => 1, [cfg]);
4   useEffect(() => { console.log(one); }, [one]);
5   return <div>{one}</div>;
6 }
```

```
MemoPrimitiveOverFreshDep (2 hooks) /tmp/ch10/memo.tsx
warn  always-unstable-deps [hook:0] (line 19:8) this memo depends on `cfg`, a new reference every render, so
  ↳ `Object.is` always differs and the memo re-runs on every render regardless of the other deps
  ↳ the value flows through `cfg`, bound here (line 18:8)
warn  always-unstable-deps [hook:1] (line 20:2) this effect depends on `one`, a new reference every render,
  ↳ so `Object.is` always differs and the effect re-runs on every render regardless of the other deps
  ↳ the value flows through `one`, bound here (line 19:8)
```

Le premier WARNING est juste : le mémo est recalculé à chaque rendu. Le second est un FP : `one` vaut toujours 1, comparé par valeur, et l'effet ne tourne qu'une fois. La sonde donne `memo[0] (Memo) = ref(PerRender)`.

10.7 Projections vers les règles

10.7.1 Quatre questions différentes

Les règles ne lisent pas une `StateValue` en entier; elles lui posent des questions fermées, et chaque question a sa polarité :

- « cette dep est-elle, à coup sûr, une référence neuve à chaque rendu ? » (`must`; `always-unstable-deps`);

- « cette valeur est-elle prouvablement stable, donc omissible des deps ? » (une preuve de sûreté ; `missing-deps`) ;
- « cette écriture peut-elle croître sans borne ? » (may ; le bras point fixe d'infinite-loop) ;
- « cet état ne prend-il que quelques valeurs primitives ? » (`wasted-subtree-render`).

Le code fournit une projection par question. La plus ancienne, et la plus délicate, est `to_stability`.

10.7.2 `to_stability` : la règle « motion-wins »

Extrait 10.28 – `src/domains/impls/state_value.rs`, l. 320–369 — la projection vers la stabilité

```

320     pub fn to_stability(&self) -> Stability {
321         if self.other {
322             return Stability::Unknown;
323         }
324         let mut acc = Stability::Bottom;
325         let mut in_motion = false;
326         if !self.num.is_bottom() {
327             if self.num.is_point() {
328                 acc = acc.join(&Stability::Stable);
329             } else {
330                 in_motion = true;
331             }
332         }
333         match self.boolean {
334             BoolVal::Bottom => {},
335             BoolVal::True | BoolVal::False => acc = acc.join(&Stability::Stable),
336             BoolVal::Top => acc = acc.join(&Stability::Unknown),
337         }
338         match &self.str {
339             StrConst::Bottom => {}
340             StrConst::Set(set) if set.len() == 1 => acc = acc.join(&Stability::Stable),
341             _ => acc = acc.join(&Stability::Unknown),
342         }
343         match &self.reference {
344             Stability::PerRender => in_motion = true,
345             s => acc = acc.join(s),
346         }
347         if self.null || self.undef {
348             acc = acc.join(&Stability::Stable);
349         }
350         if self.setter != SetterVal::Bottom {
351             // React guarantees setter identity across renders.
352             acc = acc.join(&Stability::Stable);
353         }
354         // Each populated kind slot is individually "stable" when it holds a
355         // single point, but two populated kinds are two DISTINCT concrete
356         // values (`Object.is` never equates across kinds): the value can
357         // transition between them (`useState<string>()` + `setX("data")` in
358         // a handler → {undefined, "data"}). A cross-kind union is never
359         // definitely stable.
360         // `other` is already handled by the early return above, so it is ⊥ here
361         // and does not inflate the count.
362         if self.populated_kinds().count() >= 2 {
363             acc = acc.join(&Stability::Unknown);
364         }
365         if in_motion {
366             return Stability::PerRender;
367         }
368         acc

```

```
369 }
```

L'algorithme :

1. le résidu `other` force `Unknown` : une valeur opaque ne revendique jamais une (in)stabilité définie ;
2. chaque composante peuplée contribue : un point numérique, un booléen exact, une chaîne singleton, `null`, `undefined`, un setter contribuent `Stable` ; un booléen `⊤` ou plusieurs chaînes contribuent `Unknown` ; la référence contribue sa propre stabilité, sauf `PerRender`, qui lève le drapeau « en mouvement » ;
3. deux sortes peuplées ou plus donnent `Unknown` : deux sortes sont deux valeurs `Object.is`-distinctes, et la valeur peut passer de l'une à l'autre ;
4. *motion-wins* : un intervalle non ponctuel ou une référence `PerRender` rend `PerRender`, même si une autre composante a dit `Unknown`.

La sonde, sur des valeurs construites à la main :

```
null ⊔ 1 = number[1, 1]|null; to_stability = Unknown
widen -> number[1, inf]|null; to_stability = PerRender; is_unbounded=true
boolean ⊔ [0,5] to_stability = PerRender (motion wins over Unknown)
setter: setter(#4294967295#1) to_stability=Stable typeof=Some("function")
Versioned ⊔ Stable = Versioned(({ComponentId(4294967295), 0}))
Versioned ⊔ PerRender = Unknown
top-with-versioned-ref to_stability = Unknown; versioned_reference = Some(Versioned(({ComponentId(4294967295), 0})))
```

La première ligne est la règle des sortes (deux points, deux sortes : `Unknown`) ; la deuxième et la troisième sont *motion-wins* ; la dernière montre pourquoi `versioned_reference` existe : sur une valeur dont toutes les composantes sont `⊤` sauf une référence versionnée, `to_stability` rend `Unknown` et efface l'étiquette.

Piège

`PerRender` n'a pas le même sens selon sa provenance. Dans la composante `reference`, c'est une borne *must* : une nouvelle référence à chaque rendu, garantie. Rendu par `to_stability` pour un intervalle non ponctuel, c'est seulement « *peut* changer à chaque rendu » : l'ADR-017 l'assume (« the name reads oddly for a number but means 'may change every render', kind-agnostic »). Un consommateur qui a besoin du fait *must* doit lire la composante, pas la projection. Le message de `missing-deps` montre la confusion au grand jour :

Extrait 10.29 – Un nombre à deux valeurs (/tmp/ch10/limits.tsx, extrait)

```
1 export function Branchy({ flag }: { flag: boolean }) {
2   const x = flag ? 1 : 2;
3   useEffect(() => { console.log(x); }, []);
4   return <div>{x}</div>;
5 }
```

```
warn missing-deps [hook:0] var:x (line 13:2) `x` is used in this effect but not in its deps array, and it
↳ is recreated on every render
```

L'avertissement est fondé (`x` change si `flag` change), mais « *recreated on every render* » est faux : `x` vaut `[1,2]`, que `to_stability` projette sur `PerRender`, et `describe_value` (`src/rules/helpers/mod.rs`, l. 36–52) traduit `PerRender` par cette phrase.

10.7.3 Les prédicats

Le fait `must` a son propre prédicat, déjà rencontré (liste 10.9) : `is_unstable_reference_only` est vrai si la composante `reference` vaut `PerRender` *et* qu’aucune autre sorte n’est peuplée. C’est celui qu’appellent `always-unstable-deps` (`src/rules/impls/always_unstable_deps.rs`, l. 164), le classement de fraîcheur du `churn` (`src/engine/written.rs`, l. 188) et `returns_verdict_of` (`src/rules/api/query.rs`, l. 214). Les prédicats de croissance et de finitude :

Extrait 10.30 – `src/domains/impls/state_value.rs`, l. 371–401 — croissance non bornée et finitude

```

371  /// True if this value represents unbounded growth in the fixpoint.
372  ///
373  /// Used by `InfiniteLoop` to distinguish a setter that writes a bounded value
374  /// (branch narrowing held the growth) from one that truly diverges:
375  /// - Infinite interval bounds → numeric counter without a binding guard
376  /// - Per-render reference slot → new object/function literal every render
377  /// - Residual  $\top$  → precision lost, conservatively unbounded
378  pub fn is_unbounded(&self) -> bool {
379      (!self.num.is_bottom() && (self.num.lo.is_infinite() || self.num.hi.is_infinite()))
380      || self.reference == Stability::PerRender
381      || self.other
382  }
383
384  /// True if the value ranges over a few primitives only: booleans,
385  /// `null`/`undefined`, a finite set of string constants, a narrow integer
386  /// range. A state holding such a value re-renders only when a write
387  /// *changes* it (React bails out on `Object.is`-equal writes), so however
388  /// often it is written, it re-renders at the rate of its transitions.
389  pub fn is_finitely_valued(&self) -> bool {
390      const NARROW: f64 = 16.0;
391      let num_ok = self.num.is_bottom()
392          || (self.num.is_int
393              && self.num.lo.is_finite()
394              && self.num.hi.is_finite()
395              && self.num.hi - self.num.lo <= NARROW);
396      num_ok
397          && matches!(self.str, StrConst::Bottom | StrConst::Set(_))
398          && self.reference == Stability::Bottom
399          && self.setter == SetterVal::Bottom
400          && !self.other
401  }

```

`is_unbounded` est la garde n°2 du bras point fixe d’infinite-loop : une borne numérique infinie, une référence `PerRender` ou le résidu. Le test `!self.num.is_bottom()` n’est pas décoratif : l’intervalle $\perp$ a des bornes sentinelles infinies (liste 10.4), et sans ce test toute valeur non numérique serait « non bornée » (test `is_unbounded_requires_active_num_slot`). `is_finitely_valued` sert à `wasted-subtree-render` (`src/rules/impls/wasted_subtree_render.rs`, l. 167) : un état qui ne prend que quelques primitives ne provoque un rendu qu’à ses *transitions*, puisque React abandonne une écriture `Object.is`-égale. C’est le seul prédicat qui lise le bit `is_int` hors raffinement.

Extrait 10.31 – `src/domains/impls/state_value.rs`, l. 403–417 — garder les étiquettes, tester la stabilité

```

403  /// The reference slot's change-trace labels when it carries them
404  /// (`Versioned`/`VersionedTop`, else `None`. Lets a caller preserve the
405  /// version labels while degrading the value's *kind* – the recurring
406  /// ADR-017 "keep Versioned labels, don't erase them via  $\top$ " move.
407  pub fn versioned_reference(&self) -> Option<Stability> {
408      match &self.reference {
409          Stability::Versioned(_) | Stability::VersionedTop => Some(self.reference.clone()),

```

```

410     _ => None,
411   }
412 }
413
414 /// True if this value is definitively stable (won't cause a re-render).
415 pub fn is_stable(&self) -> bool {
416     matches!(self.to_stability(), Stability::Stable)
417 }

```

`is_stable` est « `to_stability` vaut `Stable` »; il sert de preuve de sûreté à `missing-deps` (`src/rules/impls/missing_deps.rs`, l. 88), `redundant-set-state` et `unnecessary-rerender`.

Exemple 10.6: Deux sortes, une seule sorte

Extrait 10.32 – Deux états optionnels (/tmp/ch10/kinds.tsx, extrait)

```

1 export function CrossKind() {
2   const [x, setX] = useState<string>();
3   useEffect(() => { console.log(x); }, []);
4   return <button onClick={() => setX("data")}>{x}</button>;
5 }
6
7 export function NeverWritten() {
8   const [x] = useState<string>();
9   useEffect(() => { console.log(x); }, []);
10  return <div>{x}</div>;
11 }

```

La sonde donne `string"data"` pour le premier état et `undefined` pour le second. Le premier mêle deux sortes, chacune un point : `to_stability` rend `Unknown`, `is_stable` est faux, et `missing-deps` tire à raison (`x` passe de `undefined` à `"data"`) :

```

CrossKind (3 hooks) /tmp/ch10/kinds.tsx
  warn missing-deps [hook:1] var:x (line 5:2) `x` is used in this effect but not in its deps array, and its
    ↳ value may change between renders

```

Le second n'a qu'une sorte, un point : `Stable`, et `NeverWritten` est propre. Aucune conversion `Versioned` n'a lieu, puisque la composante `reference` est $\perp$.

10.7.4 Côté règles : `StabilityVerdict`

La couche règles (chapitre 17) n'expose pas `Stability` directement, mais une projection totale qui replie $\perp$ et $\top$ sur le côté `may` :

Extrait 10.33 – `src/rules/api/query.rs`, l. 137–179 — le verdict de stabilité côté règles

```

137 /// Total stability classifier for a rule-facing value probe.
138 ///
139 /// Every [`Stability`] maps to exactly one variant (see [`StabilityVerdict::of`]);
140 /// `Unknown` ( $\top$ ) is a returned variant folded to the **may** side, so it cannot
141 /// be silently dropped. `Versioned` carries the change-driving slots (an empty
142 /// set means threshold-widened `VersionedTop`: "changes at some unknown slot").
143 #[derive(Debug, Clone, PartialEq, Eq)]
144 pub enum StabilityVerdict {
145     /// Same reference every render (the only Error-safe "stable" answer).
146     Stable,
147     /// Changes only at the named setter events (may bound). Empty = widened.
148     Versioned(BTreeSet<ComponentId, HookLabel>),
149     /// Changes on every render (must bound) – **kind-agnostic motion**, not
150     /// necessarily a fresh allocation (ADR-017): a numeric slot converged to a

```

```

151    /// non-point interval lands here alongside an object literal. A consumer
152    /// that needs "defeats `Object.is` because the identity is new" wants
153    /// [crate::domains::StateValue::is_unstable_reference_only`] instead.
154    PerRender,
155    ///  $\top$  – no bound in either direction. Folded to the may side.
156    Unknown,
157 }
158
159 impl StabilityVerdict {
160     /// Total projection of the six-variant [`Stability`] lattice onto the
161     /// rule-facing verdict. `Bottom` ( $\perp$ , unreachable/uninitialised) is not
162     /// provably stable, so it folds to `Unknown` (may side) – matching the
163     /// existing `is_stable` semantics (true only for `Stable`).
164     pub fn of(stability: Stability) -> Self {
165         match stability {
166             Stability::Stable => StabilityVerdict::Stable,
167             Stability::Versioned(slots) => StabilityVerdict::Versioned(slots),
168             Stability::VersionedTop => StabilityVerdict::Versioned(BTreeSet::new()),
169             Stability::PerRender => StabilityVerdict::PerRender,
170             Stability::Bottom | Stability::Unknown => StabilityVerdict::Unknown,
171         }
172     }
173
174     /// `true` iff the value is provably `Stable`. The sound gate: everything
175     /// else ( $\top$ , Versioned, PerRender) may change.
176     pub fn is_stable(&self) -> bool {
177         matches!(self, StabilityVerdict::Stable)
178     }
179 }

```

Trois points de lecture. Bottom ne vaut pas Stable : un $\perp$ n'est pas une preuve de stabilité, il part avec Unknown. VersionedTop devient un Versioned d'ensemble vide (« change à un slot inconnu »). Et la documentation de PerRender répète la mise en garde du piège ci-dessus. Deux fonctions libres s'appuient dessus : `stability_verdict_of` (`src/rules/api/query.rs`, l. 185) et `may_change_of` (`src/rules/api/query.rs`, l. 225), l'unique sonde « peut changer » sûre face à $\top$ (ADR-21).

Projection	Polarité	Consommateurs principaux
<code>is_unstable_reference_only</code>	must	<code>always-unstable-deps</code> , <code>returns_verdict_of</code> , <code>engine/fixpoint.rs</code> <code>value_freshness</code> , <code>helpers/jsx.rs</code>
<code>is_stable</code>	sûreté	<code>missing-deps</code> , <code>redundant-set-state</code> , <code>unnecessary-rerender</code>
<code>is_unbounded</code>	may	<code>infinite-loop</code> (bras point fixe)
<code>is_finitely_valued</code>	sûreté	<code>wasted-subtree-render</code>
<code>to_stability</code>	mixte	<code>StabilityVerdict</code> , <code>describe_value</code> , <code>recompute_memo</code>
<code>versioned_reference</code>	may	<code>recompute_memo</code> , <code>eval_field_access</code>
<code>as_setter</code>	exacte	transfert des setters, <code>engine/setters.rs</code>

TABLE 10.2 – Les projections de StateValue et leurs consommateurs (relevé grep au commit e67b10a, dossier 04 §4.7).

Décision : ADR-020 — ne pas supprimer `to_stability`

Lors de la campagne de dette technique, la suppression de `to_stability` au profit de prédicats de première classe a été proposée puis refusée (ADR-20, non-changement n°4) : c'est une projection légitime, documentée et testée (motion-wins, union de sortes vers Unknown, setter

stable), et aucun couplage avec perte d'information démontré n'y cause de FN ou de FP. La supprimer serait du remaniement ou un risque de soundness, pas une correction. La même campagne a refusé d'extraire un combinateur `BoundedPowerset<T,N>` commun à `StrConst` et `Stability` (non-changement n°5) : `Stability` est un treillis plus riche que l'ensemble borné.

10.8 Les contextes : la passerelle vers le moteur

10.8.1 Ce dont une fonction de transfert a besoin

Les fonctions de transfert ne manipulent pas seulement des `StateValue` isolées. Pour évaluer `StateVal(1)`, il leur faut le store d'état *et* l'identité du composant, sans laquelle l'étiquette `Versioned((C, 1))` ne peut pas être fabriquée. Pour `MemoVal`, le store des mémos; pour `obj.field`, le tas; pour un appel à travers une variable liée par `useCallback`, le CFG du corps; pour `<Child />`, toute la machinerie inter-composants. Le fichier `src/domains/context.rs` regroupe ces besoins en quelques types, seule flèche de dépendance `domains` → `engine` du périmètre.

10.8.2 AnalysisCtx : le paquet d'état mutable

Extrait 10.34 – `src/domains/context.rs`, l. 105–125 — le paquet d'état passé à chaque fonction de transfert

```

105 // -- AnalysisCtx -----
106
107 /// Bundle of mutable analysis state threaded through `Transfer` methods.
108 ///
109 /// Replaces the five separate `(env, state, memo, heap, ctx)` parameters.
110 /// `env` is kept as a separate parameter since its mutability and lifetime
111 /// differ between `eval_expr` (`&`) and `exec_stmt` (`&mut`).
112 pub struct AnalysisCtx<'a, D: AbstractDomain> {
113     /// The component under analysis. An analysis is always the analysis of
114     /// SOME component – intra or inter – so this is not an `Option`:
115     /// state-slot provenance (`Versioned` labels, `SetterVal`) always carries
116     /// the real owner. A hand-built IR analysed on its own carries
117     /// [`ComponentId::SYNTHETIC`].
118     pub component: ComponentId,
119     pub state: &'a mut StateStore<D>,
120     pub memo: &'a mut MemoStore<D>,
121     pub heap: &'a mut Heap,
122     pub query: &'a dyn QueryContext,
123     /// Optional inter-component context. `None` = intra-component analysis only.
124     pub inter: Option<&'a InterCtx<'a>>,
125 }
```

L'environnement local n'y figure pas : il est passé à part, parce que `eval_expr` le lit (&) alors que `exec_stmt` le modifie (&mut). Le champ `component` n'est pas une `Option`, et son commentaire dit pourquoi : une analyse est toujours l'analyse d'*un* composant, donc la provenance d'un slot (`Versioned`, `SetterVal`) porte toujours le vrai propriétaire ; une IR construite à la main et analysée seule porte `ComponentId::SYNTHETIC`, l'identité 4294967295 des sorties de la sonde. `analyze_cfg` construit un `AnalysisCtx` par bloc (chapitre 12).

Extrait 10.35 – `src/domains/context.rs`, l. 127–145 — un contexte sans requête ni inter-composants

```

127 impl<'a, D: AbstractDomain> AnalysisCtx<'a, D> {
128     /// Construct with `NullCtx` as the query context (tests, simple impls).
```

```

129     pub fn null(
130         component: ComponentId,
131         state: &'a mut StateStore<D>,
132         memo: &'a mut MemoStore<D>,
133         heap: &'a mut Heap,
134     ) -> Self {
135         static NULL: NullCtx = NullCtx;
136         AnalysisCtx {
137             component,
138             state,
139             memo,
140             heap,
141             query: &NULL,
142             inter: None,
143         }
144     }
145 }

```

Le static NULL donne une référence &'static dyn QueryContext sans allocation, NullCtx étant une structure unitaire. Le moteur l'emploie avec des stores d'amorçage vides pour évaluer les constantes de module, les arguments de hooks personnalisés et les initialiseurs de useState (src/engine/fixpoint.rs, l. 168, src/engine/fixpoint.rs, l. 245, src/engine/fixpoint.rs, l. 327).

10.8.3 QueryContext et ses deux implémentations

Extrait 10.36 – src/domains/context.rs, l. 27–49 — le canal de requête entre domaines

```

27 // -- QueryContext trait -----
28
29 /// Cross-domain context passed to every Transfer method for abstract-state queries.
30 pub trait QueryContext {
31     /// Body CFG of a `useCallback` hook, if the context knows it. Lets the
32     /// interpreter execute calls through a callback-bound variable
33     /// (`const cb = useCallback(...); ...; cb()`): the rewrite to
34     /// `CallbackVal(label)` moved the body out of the expression tree, so it
35     /// is not reachable through the heap like a plain FnLit.
36     fn callback_body(
37         &self,
38         _label: crate::ir::types::HookLabel,
39     ) -> Option<std::sync::Arc<crate::ir::cfg::CFG>> {
40         None
41     }
42 }
43
44 // -- NullCtx -----
45
46 /// No-op context: returns `Top` for every query. Used in tests and as recursion base.
47 pub struct NullCtx;
48
49 impl QueryContext for NullCtx {}

```

Le trait n'a plus qu'une requête, callback_body : le CFG du corps d'un useCallback, que la réécriture en CallbackVal(label) a sorti de l'arbre d'expressions et qui n'est donc pas atteignable par le tas comme un FnLit ordinaire. Sans réponse (None), l'interpréteur n'exécute pas le corps de l'appel (src/domains/interp/interpreter.rs, l. 579). NullCtx répond toujours None ; son commentaire « returns Top for every query » est daté. Pendant le point fixe, c'est FixpointCtx qui répond :

Extrait 10.37 – src/domains/context.rs, l. 147–171 — le contexte de requête du point fixe

```

147 // -- FixpointCtx -----
148
149 /// Mid-fixpoint context: evaluates `expr` against the current iteration's state.
150 ///
151 /// Used by `analyze_cfg` to give Transfer methods read access to the fixpoint
152 /// state accumulated so far. Local variables return `Bottom` (no per-block env
153 /// threaded here conservative but sound).
154 pub struct FixpointCtx<'a> {
155     pub state: &'a StateStore<StateValue>,
156     pub memo: &'a MemoStore<StateValue>,
157     /// `useCallback` body CFGs by hook label (see `QueryContext::callback_body`).
158     pub callbacks: &'a std::collections::HashMap<
159         crate::ir::types::HookLabel,
160         std::sync::Arc<crate::ir::cfg::CFG>,
161     >,
162 }
163
164 impl QueryContext for FixpointCtx<'_> {
165     fn callback_body(
166         &self,
167         label: crate::ir::types::HookLabel,
168     ) -> Option<std::sync::Arc<crate::ir::cfg::CFG>> {
169         self.callbacks.get(&label).cloned()
170     }
171 }

```

Piège

NullCtx sert aussi aux trois amorçages du section 10.8 qui précèdent la boucle de point fixe (constantes de module, retours d'arguments de hooks personnalisés, initialiseur paresseux d'un useState) : un appel à travers une variable liée par useCallback, rencontré pendant l'un de ces amorçages, n'y exécute donc pas son corps. Cela ne fuit dans aucun état retenu : le store et le tas passés à AnalysisCtx::null pour ces amorçages sont des copies jetables (src/engine/fixpoint.rs, l. 164–168, src/engine/fixpoint.rs, l. 240–244, src/engine/fixpoint.rs, l. 317–320), distinctes du store du point fixe et jamais fusionnées dedans. Ce qui reste ouvert est une question d'exécution de l'interpréteur, pas de domaine : est-ce qu'un effet de bord perdu de cette façon (un useCallback qui écrirait un *autre* slot depuis un initialiseur paresseux) pourrait échapper au point fixe par un autre chemin ; le chapitre 11 tranche.

Seul callbacks est lu (le clonage est celui d'un Arc, en temps constant) ; les champs state et memo ne sont lus par aucune méthode du trait, et la phrase « Local variables return Bottom » décrit un comportement disparu.

Décision : ADR-007 — requêtes entre domaines par un trait

Décide (« Decision A », solution B3) : un trait QueryContext passé en &dyn à chaque méthode de Transfer, ce qui garde le trait utilisable en objet (object-safe). **Écarte** : le ask générique de MOPSA, qui demande des GADT absents de Rust stable (et la spécialisation, instable). **Prévoit** (« Decision B », B1) : une migration vers un « Manager » générique à types marqueurs si le nombre de domaines dépassait cinq environ ; elle n'a pas eu lieu. **Daté** : l'ADR décrit une requête state_value_of et un troisième contexte AnalysisQueryCtx qui n'existent plus. L'ADR-015 a confirmé le rôle futur du trait : canal de lecture d'un éventuel produit réduit au niveau de l'analyse.

10.8.4 InterCtx et AnalyzeChildFn

Quand l'évaluateur rencontre un élément `<Child />` dont le composant est connu, il analyse l'enfant avec les props de l'appelant (chapitre 16). Cela crée un cycle de modules : le transfert (domains) doit appeler le point fixe (engine), qui appelle le transfert. Le cycle est cassé par un pointeur de fonction fourni par le moteur :

Extrait 10.38 – `src/domains/context.rs`, l. 15–25 — le pointeur qui casse le cycle transfert–point fixe

```

15 // -- AnalyzeChildFn -----
16
17 /// Function pointer for inlining a child component's analysis. Breaks the
18 /// circular dep between `domains::transfer` and `engine::fixpoint`.
19 pub type AnalyzeChildFn = fn(
20     &ComponentIR,
21     ComponentId,
22     AbstractEnv<StateValue>,
23     Heap,
24     &InterCtx<'_>,
25 ) -> AnalysisResult<StateValue>;

```

Extrait 10.39 – `src/domains/context.rs`, l. 51–103 — le contexte inter-composants

```

51 // -- InterCtx -----
52
53 /// Inter-component analysis context, threaded through the analysis when doing
54 /// top-down inlining across component boundaries.
55 ///
56 /// Uses `RefCell` for all mutable shared state so that `InterCtx` can be passed
57 /// as a shared `&InterCtx` reference avoiding nested `&mut` lifetime issues while
58 /// still allowing mutation through `borrow_mut()`.
59 pub struct InterCtx<'a> {
60     pub registry: &'a ComponentRegistry,
61     pub cache: &'a RefCell<ComponentCache>,
62     pub shared_state: &'a RefCell<SharedStateStore>,
63     pub call_graph: &'a RefCell<ComponentCallGraph>,
64     pub stats: &'a RefCell<AnalysisStats>,
65     /// All analysis results accumulated across the entire program analysis.
66     pub results: &'a RefCell<std::collections::HashMap<ComponentId, AnalysisResult<StateValue>>>>,
67     /// Components currently being analyzed (for recursion detection).
68     pub call_stack: RefCell<Vec<ComponentId>>,
69     /// The component being analyzed at this level.
70     pub component: ComponentId,
71     /// Analysis config (widen_threshold etc.).
72     pub config: &'a Config,
73     /// Callback provided by `engine::fixpoint` to inline a child component's analysis.
74     pub analyze_child: AnalyzeChildFn,
75     /// User-defined custom hook registry for inlining (None = no inlining).
76     pub hook_registry: Option<&'a HookRegistry>,
77 }
78
79 impl<'a> InterCtx<'a> {
80     /// Create a child context for inlining a nested component.
81     /// Shares all RefCell state; new call_stack with parent pushed.
82     pub fn child(&self, child: ComponentId) -> InterCtx<'a> {
83         let mut new_stack = self.call_stack.borrow().clone();
84         new_stack.push(self.component);
85         InterCtx {
86             registry: self.registry,
87             cache: self.cache,

```

```

88     shared_state: self.shared_state,
89     call_graph: self.call_graph,
90     stats: self.stats,
91     results: self.results,
92     call_stack: RefCell::new(new_stack),
93     component: child,
94     config: self.config,
95     analyze_child: self.analyze_child,
96     hook_registry: self.hook_registry,
97 }
98 }
99
100 pub fn is_recursive(&self, id: ComponentId) -> bool {
101     self.call_stack.borrow().contains(&id) || self.component == id
102 }
103 }

```

Tout l'état partagé (cache des analyses d'enfants, store d'état partagé entre composants, graphe d'appels, statistiques, résultats) passe par des `RefCell`, pour que `InterCtx` circule en référence partagée sans `&mut` imbriqués. `child` copie toutes les références et ne crée qu'une pile neuve, où il empile le composant *courant*.

Invariant 10.3: La pile d'appel contient les ancêtres

La `call_stack` d'un `InterCtx` contient les ancêtres du composant courant, pas le composant lui-même : il n'y est poussé qu'en descendant, par `child`. D'où le second disjoint `self.component == id` de `is_recursive`.

L'évaluateur appelle `is_recursive` avant `child` (`src/domains/transfer/state_value.rs`, l. 521, `src/domains/transfer/state_value.rs`, l. 578) : un composant récursif n'est pas ré-analysé, son élément vaut `ref(Stable)` et la coupure est comptée dans les statistiques. Les contextes racines sont construits par `analyze_program` avec une pile vide (`src/engine/fixpoint.rs`, l. 729–741).

10.9 Limites et défauts observés

Les défauts qui suivent ont été observés au commit `e67b10a` sur les fichiers de `/tmp/ch10/`. Ils sont présentés comme des constats ; ceux qui n'ont pas d'issue sont signalés comme tels.

10.9.1 Un mémo sur un état numérique : un FP de projection

Extrait 10.40 – Deux mémos sur un état (`/tmp/ch10/memo.tsx`, extrait)

```

1 export function MemoOverNumState() {
2     const [n, setN] = useState(0);
3     const doubled = useMemo(() => n * 2, [n]);
4     useEffect(() => { console.log(doubled); }, [doubled]);
5     return <button onClick={() => setN(5)}>{doubled}</button>;
6 }
7
8 export function MemoOverObjState() {
9     const [o, setO] = useState({ a: 1 });
10    const a = useMemo(() => ({ v: o.a }), [o]);
11    useEffect(() => { console.log(a); }, [a]);
12    return <button onClick={() => setO({ a: 2 })}>{a.v}</button>;
13 }

```

```

MemoOverNumState (4 hooks) /tmp/ch10/memo.tsx
  warn  always-unstable-deps [hook:2] (line 6:2) this effect depends on `doubled`, a new reference every render, so
    ↳ `Object.is` always differs and the effect re-runs on every render regardless of the other deps
      → the value flows through `doubled`, bound here (line 5:8)
MemoOverObjState (4 hooks) /tmp/ch10/memo.tsx ✓

```

Le WARNING est faux : `doubled` est un nombre, recalculé seulement quand `n` change. La sonde donne `state[0] = number[0, 5]` et `memo[1] (Memo) = ref(PerRender)`. La chaîne causale se lit dans le code :

1. la dep `n` n'est pas syntaxiquement `StateVal(0)` : le lowering lie `let n = StateVal(0)` (`src/lowering/hook_extractor.rs`, l. 783) et la dep est la variable `n` ; la projection structurelle de `recompute_memo` ne s'applique donc pas ;
2. la valeur de `n` est `number[0, 5]`, sans composante `reference` : la conversion côté lecture ne l'a pas étiquetée, et `versioned_reference` rend `None` ;
3. `to_stability` projette l'intervalle non ponctuel sur `PerRender`, au sens *may* de motion-wins ;
4. `recompute_memo` range ce `PerRender` dans la composante `reference` du mémo, où `is_unstable_reference_only` le lit comme un fait *must*.

Le glissement de sens de `PerRender` passe ainsi d'une projection *may* à un fait *must*. Avec un état objet, la lecture porte déjà `Versioned` dans sa composante référence, et le problème ne se pose pas. Aucune issue correspondante n'a été trouvée au commit `e67b10a` ; la même cause produit le FP de liste 10.27.

10.9.2 Les cycles finis

Extrait 10.41 – Un booléen qui bascule (`/tmp/ch10/kinds.tsx`, extrait)

```

1 export function Toggle() {
2   const [on, setOn] = useState(false);
3   useEffect(() => {
4     setOn(!on);
5   }, [on]);
6   return <div>{on ? 1 : 0}</div>;
7 }

```

C'est une vraie boucle infinie : chaque écriture change la valeur, la dep change, l'effet se ré-exécute. L'analyseur déclare pourtant le composant propre (la ligne `Toggle` (2 hooks) de la sortie de `-show-clean` porte la coche des composants sans diagnostic). Le store converge sans widening sur boolean (`⊤` de `BoolVal`) ; le bras point fixe d'*infinite-loop* exige un slot élargi, et le churn ne concerne que la sorte référence. Le produit est un domaine d'*ensembles de valeurs atteignables* : un cycle à travers un nombre fini de valeurs a un point fixe abstrait fini, et l'information « la valeur change à chaque exécution », une propriété de *transition*, n'y est pas représentable. Pour une référence, au contraire, le domaine conserve un fait de transition (`PerRender`, « neuve à chaque écriture »), ce qui permet l'ERROR sur `ObjChurn`. Le section 9.10 décrit la même famille pour $(i + 1) \% 3$ et les chaînes ; ce FN n'est ni dans une issue ni dans `docs/limitations.md`, et l'ADR-008 affirmait l'inverse.

10.9.3 Une chaîne qui grandit : `is_unbounded` ignore `str`

Le composant `StrGrow` de `/tmp/ch10/limits.tsx` écrit `setS(s + "x")` dans un effet de deps `[s]`. Le store passe de `"a"` à `"a"`, `"ax"`, etc., jusqu'à dépasser quatre constantes : `StrConst::Top` (sonde : `state[0] = string widened=true`). Mais `is_unbounded` (liste 10.30) ne regarde que `num`, la référence et le résidu, et la garde `n°2` du bras point fixe coupe :

```
StrGrow (2 hooks) /tmp/ch10/limits.tsx
info widening-info (line 5:2) state `s` kept changing during analysis and was approximated to converge, so
  ↳ findings that depend on it may be imprecise
...
verified infinite-loop no effect diverges into an infinite render loop
```

(sortie de `-show-clean -trace -info`, élaguée). « verified » sur une vraie boucle infinie : un FN. Parmi les composantes qu’ignore `is_unbounded`, `str` est la seule dont l’ensemble concret soit infini, donc la seule qui puisse grandir sans borne. Aucune issue ni mention dans `docs/limitations.md` n’a été trouvée.

10.9.4 Un élément JSX natif classé **Stable** : un FN sans trace

La ligne « élément JSX natif » du tableau 10.1 range la même valeur que `useRef()` : `ref(Stable)`. C’est pourtant la seule des huit lignes de ce tableau dont la colonne « Raison » ne renvoie à rien : ni fait `must` sur une allocation, ni garantie de React, ni commentaire dans le code (`src/domains/transfer/state_value.rs`, l. 153–156). Or un élément JSX natif est une valeur allouée : `<div/>` désucre en un appel (`React.createElement` ou `jsx`), qui rend un objet neuf à chaque évaluation, exactement comme `ou () =>` deux lignes au-dessus du même tableau.

Extrait 10.42 – Un élément natif en dep (`/tmp/ch10/limits.tsx`, extrait)

```
1 export function ElemDep() {
2   const el = <div />;
3   useEffect(() => { console.log(el); }, [el]);
4   return el;
5 }
```

```
ElemDep (1 hooks) /tmp/ch10/limits.tsx ✓
```

Composant propre, alors que `el` est `Object`.`is`-différent à chaque rendu et que l’effet devrait donc se ré-exécuter systématiquement. Comparer avec le même programme où `el` vaut `au lieu de <div/>` : `always-unstable-deps` tire, sans surprise (liste 10.14 donne le mécanisme). La cause est locale à la ligne citée plus haut : `Expr::ObjectLit`, `ArrayLit` et `FnLit` rendent `ref(PerRender)`, mais `Expr::NativeElem` juste en dessous rend `ref(Stable)`, sans qu’aucune des trois autres sources du produit (le tableau, un ADR, un test dédié) ne justifie l’écart. `missing-deps` hérite du même trou : omettre `el` des `deps` ne déclenche rien non plus, puisque `is_stable` le couvre.

Piège

Une `dep` qui est elle-même un élément JSX natif est un motif rare en pratique (on ne compare pas des arbres React par identité dans une application normale), ce qui explique sans doute que le défaut n’ait jamais atteint un corpus de test. Cela n’en fait pas moins un faux négatif au sens strict de la définition 3.8 : rien dans le code ne borne le domaine de `ElemDep` à des programmes « improbables ». Aucune issue ne suit ce défaut au commit `e67b10a`.

10.9.5 Limites connues et suivies

- **Slot jamais écrit** (#41, `precision-fp`) : un `useState(...)` dont aucun `setter` n’est appelé lit `Versioned`, pas `Stable`, et la `dep` ne peut pas être omise. Le raffinement demanderait un bit « slot jamais écrit » après le point fixe ; l’ADR-020 (non-changement n°3) garde `may_written_slots` syntaxique, parce qu’un bit observé pourrait sous-compter sur un chemin élagué.

- **Constantes de module opaques** (#34) : `const x = f()` vaut $\top$; le produit n'a pas d'encodage « sorte inconnue, identité stable ».
- **Méthodes ambiguës** (#22) : `slice` et `concat` restent $\top$; lever la limite demanderait la sorte du récepteur.
- **Écriture dérivée du slot écrit** (#157), vue au section 10.5.
- **Silence sur $\top$** : `const v = flag ? a : 2` : `obj` vaut `Unknown` (join de `PerRender` et de `Versioned`), et `always-unstable-deps` se tait sur une `dep v` (composant `MixedRef` de `kinds.tsx`, propre). C'est le choix documenté de la règle (« Top (precision lost) stays silent to avoid FPs », `src/rules/impls/always_unstable_deps.rs`, l. 161–163) : elle n'affirme que des faits `must`.

Piège

Plusieurs commentaires et ADR du périmètre sont datés, et il ne faut pas s'y fier sans relire le code : la documentation de `NullCtx` (« returns Top ») et de `FixpointCtx` (« Local variables return Bottom ») ; ADR-007 (`state_value_of`, `AnalysisQueryCtx`) ; ADR-015 et ADR-017 (`Symbol` au lieu de `ComponentId`) ; ADR-002 (fichiers inexistantes, seuil 2) ; ADR-008 (oscillation booléenne détectée) ; et le nom de `from_init`.

Résumé du chapitre

- Une valeur abstraite est un **produit ponctuel** sur les huit sortes disjointes de JavaScript (`StateValue`, `src/domains/impls/state_value.rs`) : les sortes étant disjointes, la complétion disjonctive dégénère en produit, sans opérateur de réduction (ADR-015, qui remplace la somme plate d'ADR-008). `Join`, `meet`, `widening` et `ordre` sont composante par composante ; γ est l'union des concrétisations (définition 10.1).
- `as_arith` applique `ToNumber(null) = 0`, ce qui fait détecter le compteur `useState(null)` ; `undefined`, les booléens et les chaînes rendent $\top$ en arithmétique. `KindMask` et `populated_kinds` sont l'unique énumération des sortes.
- La composante `reference` porte une **stabilité**, borne sur la *trace de changement* d'une valeur au fil des rendus. Le treillis `Stability` d'ADR-017 (définition 10.3) combine une borne `may` (`Stable` $\sqsubseteq$ `Versioned(S)` $\sqsubseteq$ `VersionedTop`, $|S| \leq 4$) et une borne `must` (`PerRender`), incomparables, jointes en `Unknown`. Son `join` est `sound` et de hauteur 8 ; son `meet` sous-estime l'intersection des γ et n'est pas utilisé.
- **Double vue** : le store range le join des valeurs écrites (`ref(PerRender)` pour un état objet) ; la lecture de `StateVal(1)` convertit la référence en `Versioned(1)`, en un seul point de l'évaluateur. C'est `sound` sous deux hypothèses couvertes ailleurs : pas de set pendant le rendu (`setter-in-render`) et le bras `churn` d'`infinite-loop`, qui certifie `ObjChurn`.
- Les littéraux, `new` et les allocateurs connus sont `PerRender` (`must`) ; `useRef` et le setter sont stables ; un `mémo` vaut le join des stabilités de ses `deps` (`recompute_memo`), rangé dans la composante `reference` : sa sorte réelle est perdue.
- Les règles lisent des **projections** : `is_unstable_reference_only` (`must`), `is_stable` (sûreté), `is_unbounded` (`may`), `is_finitely_valued`, `to_stability` (`motion-wins`, où `PerRender` ne veut dire que « peut changer à chaque rendu ») et `StabilityVerdict`.
- `AnalysisCtx`, `QueryContext`, `FixpointCtx`, `InterCtx` et `AnalyzeChildFn` (`src/domains/context.rs`) relient les domaines au moteur.
- Défauts observés : FP des `mémos` sur un état numérique, FN des cycles finis, des chaînes qui grandissent et de l'élément JSX natif classé `Stable` ; limites suivies #41, #34, #22, #157.

Le chapitre 11 décrit l'évaluateur complet qui fabrique ces valeurs, et les stores (définition 11.2)

qui les rangent.

Exercices

Exercice 10.1: Calculer dans le produit

Soit $a = \text{number}[0, 2]|\text{null}$ et $b = \text{string}"a"$. (a) Calculer $a \sqcup b$ et sa concrétisation. (b) Que valent `to_stability`, `is_unbounded` et `as_arith` sur $a \sqcup b$? (c) Les valeurs $c = \text{number}[0, 1]|\text{null}$ et $d = \text{number}[0, 2]$ sont-elles comparables?

Solution

(a) Composante par composante : $\text{number}[0, 2]|\text{string}"a"|\text{null}$, de concrétisation $[0, 2] \cup \{"a"\} \cup \{\text{null}\}$. (b) `to_stability` : l'intervalle n'est pas un point, donc le drapeau « en mouvement » est levé; trois sortes sont peuplées, ce qui joint `Unknown`; `motion-wins` rend `PerRender`. `is_unbounded` : faux (bornes finies, pas de référence `PerRender`, pas de résidu). `as_arith` : `None` (la chaîne est peuplée), donc $n + 1$ vaudrait $\top$. (c) Non : sur `num`, $c \sqsubset d$; sur `null`, $c \sqsupset d$; `merge_ord` rend `None`.

Exercice 10.2: Le meet trop petit

Exhiber un programme `React` dont une expression a une trace de changement à la fois dans $\gamma(\text{Versioned}(\{l\}))$ et dans $\gamma(\text{PerRender})$. Qu'en conclure sur `meet`?

Solution

La lecture `obj` dans `ObjChurn` (liste 10.15) : chaque rendu r_i ($i \geq 1$) est précédé d'un `setObj` avec un objet neuf, donc $C = \{1, \dots, n\} = \text{Set}(\{\text{obj}\})$. La trace appartient aux deux concrétisations, alors que $\text{Versioned}(\{l\}) \sqcap \text{PerRender} = \perp$. Le `meet` du code est une borne inférieure dans l'ordre abstrait mais pas une sur-approximation de l'intersection des γ : s'en servir pour raffiner éliminerait ce comportement réel, un FN.

Exercice 10.3: Casser l'invariant

Un contributeur construit à la main `Stability::Versioned(S5)` avec $|S_5| = 5$. (a) Le join reste-t-il `sound`? (b) Quelle propriété de la proposition 10.2 est perdue?

Solution

(a) Oui : la preuve du théorème 10.2 n'utilise pas la borne sur $|S|$; $\text{Versioned}(S_5)$ a simplement la concrétisation attendue. (b) La forme canonique : $\text{Versioned}(S_5) \sqcup \text{Versioned}(S_5)$ rend $\text{Versioned}(S_5)$ par le bras d'égalité, alors que la même information s'écrit canoniquement `VersionedTop`. La plus petite borne supérieure n'est plus garantie (un $\text{Versioned}(S_5)$ s'intercale entre les `Versioned` de taille 4 et `VersionedTop`), et la borne de hauteur 8 ne vaut plus pour les chaînes qui passent par des éléments non canoniques. La terminaison tient encore, parce que tout join de deux ensembles distincts repasse par `versioned`. C'est pourquoi les constructeurs `versioned` et `versioned_by` sont le seul chemin recommandé.

Exercice 10.4: Projeter à la main

Donner `to_stability` de : `[3, 3]; number[3, 3]|\text{null}; boolean (\top \text{ de BoolVal}); ref(\text{Versioned}(a))|\text{undefined}; setter(...); \top; number[0, \text{inf}]|\text{ref}(\text{Stable})`.

Solution

Stable (un point) ; Unknown (deux sortes) ; Unknown (booléen $\top$) ; Unknown (le join partiel vaut `Versioned(a)`, mais deux sortes sont peuplées) ; Stable (identité garantie par React) ; Unknown (résidu) ; PerRender (motion-wins, l'intervalle n'est pas un point).

Exercice 10.5: Corriger au bon niveau

Expliquer pourquoi `MemoOverObjState` est propre et `MemoOverNumState` ne l'est pas (liste 10.40). Pourquoi ajouter, dans `always-unstable-deps`, un cas « ignorer les deps qui sont des mémos » serait-il contraire aux principes du projet ? Où chercher une correction ?

Solution

La lecture de `o` porte `Versioned(o)` dans sa composante référence, que `versioned_reference` transmet au mémo ; la lecture de `n` n'a pas de composante référence, donc aucune étiquette, et `to_stability` projette $[0, 5]$ sur PerRender (sens may), que le mémo range comme un fait must. Un cas spécial dans la règle contournerait un défaut de la couche domaine (principe « pas de workarounds ») et masquerait des vrais positifs (un mémo sur une dep réellement fraîche). La correction doit se placer là où le sens glisse : soit dans `recompute_memo`, qui ne devrait pas convertir une projection may en fait must, soit dans le domaine, qui n'a pas d'endroit où loger une étiquette de version pour une sorte non référence. Au commit `e67b10a`, aucune issue ne suit ce défaut.

Exercice 10.6: is_unbounded et les chaînes

Écrire une version de `is_unbounded` qui ferme le FN `StrGrow`. Change-t-elle quelque chose pour `StrToggle` (`setMode(mode === "a" ? "b" : "a")`) ? Et pour un état écrit par des handlers avec cinq constantes distinctes ?

Solution

Ajouter le disjoint `self.str == StrConst::Top` (une chaîne de contenu inconnu peut grandir sans borne). `StrToggle` converge sur `string"a", "b"`, sous le seuil : inchangé (le FN des cycles finis demeure). Cinq constantes donnent aussi `StrConst::Top` : le prédicat deviendrait vrai alors que l'ensemble concret est fini. Qu'un diagnostic en résulte dépend des autres portes du bras point fixe (chapitre 19) ; le seuil de `StrConst` confond « beaucoup de constantes » et « croissance », et c'est le prix de la hauteur finie.

Exercice 10.7: Détecter Toggle

Pourquoi aucun domaine d'*ensembles de valeurs* ne peut-il détecter la boucle de liste 10.41 ? Quelle information faudrait-il ajouter ?

Solution

L'ensemble des valeurs atteignables de `on` est `{false, true}`, qui est aussi celui d'un composant qui écrit une seule fois `true` puis s'arrête ; les deux programmes ont le même point fixe abstrait. Ce qui les distingue est une propriété de *transition* : « la valeur écrite diffère toujours de la valeur lue » (`!on ≠ on`). Il faudrait un fait relationnel entre la lecture et l'écriture d'un même slot, analogue pour les primitives à ce que PerRender fournit pour les références.

Chapitre 11

Fonctions de transfert, interpréteur d'expressions et stores

Objectifs du chapitre.

- Savoir ce que calcule l'interpréteur abstrait d'un pas : la valeur d'une expression de l'IR (`eval_state_value`) et l'effet d'une instruction sur l'état abstrait (`exec_stmt`), et connaître le chemin exact des appels, du moteur jusqu'à l'évaluateur.
- Connaître l'environnement `AbstractEnv` et les quatre stores (`StateStore`, `MemoStore`, `SharedStateStore`, `Heap`) : leur clé, leur valeur par défaut ($\top$ ou $\perp$), leur mode de mise à jour (forte ou faible), leur portée.
- Savoir évaluer à la main, opération par opération, n'importe quelle expression de l'IR, et savoir ce que le lowering a déjà réécrit avant que l'évaluateur ne la voie.
- Comprendre comment un appel de setter, un *functional updater*, un callback `.then` ou `setTimeout`, une fonction locale ou un composant enfant font bouger l'état (ADR-009, ADR-010, ADR-012).
- Savoir où la soundness est garantie par construction, où elle est abandonnée par décision, et reconnaître les quatorze défauts observés au commit e67b10a (D1 à D14).

Prérequis : chapitre 3 (définition 3.2, définition 3.5, définition 3.6, définition 3.8, mises à jour fortes et faibles) ; chapitre 6 (définition 6.3, définition 6.2, définition 6.1) ; chapitre 7 (diamants, extraction des hooks, handlers) ; chapitre 9 (intervalles, `StrConst`, opérateurs) ; chapitre 10 (définition 10.1, définition 10.3, conversion côté lecture, `AnalysisCtx`).

Les trois chapitres précédents ont construit des *valeurs* abstraites : des intervalles, des ensembles de chaînes, un produit `StateValue` dont chaque composante suit une sorte de JavaScript, et un treillis de stabilité qui dit si une référence change d'un rendu à l'autre. Il manque la pièce qui les met en mouvement : la fonction qui, devant une expression `n + 1` ou une instruction `setN(n + 1)`, calcule la valeur abstraite ou fait évoluer l'état. C'est l'objet de ce chapitre, la « sémantique d'un pas » de l'analyse. Le chapitre 12 et le chapitre 13 décriront « l'itération des pas » : dans quel ordre on visite les blocs, quand on joint, quand on applique le *widening*, quand on s'arrête.

Le sous-système vit dans trois répertoires de `src/domains/`, qui correspondent à trois questions :

- `transfer/` : que *vaut* une expression ? C'est l'évaluateur `eval_state_value`, propre au domaine `StateValue`, avec le calcul des mémos et l'inlining des composants enfants ;
- `interp/` : que *fait* une instruction ? C'est l'interpréteur générique en $\top$: `Transfer` : liaisons, écritures de champ, appels de setter, callbacks, corps de fonctions ;
- `stores/` : où range-t-on ce que l'on sait ? L'environnement local, l'état React, les mémos, le tas, l'état partagé entre composants.

La progression suit la difficulté : on part d'un compteur, on apprend à évaluer ses expressions, puis à exécuter ses instructions, puis à descendre dans ses callbacks, son tas et ses enfants ; on termine par une étude de cas des défauts de soundness observés, qui mobilise tout le reste.

11.1 Le problème : évaluer $n + 1$ sans exécuter le programme

11.1.1 Deux compteurs

Voici deux composants presque identiques.

Extrait 11.1 – Deux compteurs (/tmp/ch11/counter.tsx)

```

1 import { useState, useEffect } from "react";
2
3 export function Counter() {
4   const [n, setN] = useState(0);
5   useEffect(() => {
6     setN(n + 1);
7   });
8   return <div>{n}</div>;
9 }
10
11 export function Settle() {
12   const [m, setM] = useState(0);
13   useEffect(() => {
14     setM(5);
15   });
16   return <div>{m}</div>;
17 }

```

Tous deux ont un effet sans tableau de deps, donc exécuté après chaque rendu, et cet effet appelle le setter de l'état. Counter boucle : chaque effet écrit une valeur neuve, React refait un rendu, l'effet se relance, indéfiniment. Settle ne boucle pas : le premier effet écrit 5, le rendu suivant relance l'effet, qui écrit encore 5, et React abandonne ce second rendu parce que `Object.is(5, 5)` est vrai (chapitre 2). L'analyseur fait bien la différence :

```

$ reactant check counter.tsx --trace --show-clean
Counter (2 hooks) counter.tsx
  warn  infinite-loop [hook:0] (line 5:2) this effect keeps pushing state `n` to new values on every run. Potential
    ↳ infinite render loop
      ↳ state `n` is written here [hook:1] (line 5:2)
      ↳ the abstract value of state `n` kept growing and was widened at iteration 3
Settle (2 hooks) counter.tsx ✓

! 1 warning(s) across 1 file(s).

```

La règle `infinite-loop` (chapitre 19) ne fait que lire un fait établi plus tôt : « la valeur abstraite de n a continué de croître et a subi un widening ». Ce chapitre explique comment ce fait est calculé. Pour cela, l'analyseur doit répondre à quatre questions, sans jamais exécuter le programme :

1. dans l'effet, que vaut n ? C'est une affaire d'*environnement* (section 11.2) ;
2. que vaut $n + 1$? C'est l'*évaluateur d'expressions* (section 11.3) ;
3. où va la valeur passée à `setN`, et comment se combine-t-elle avec les précédentes ? Ce sont les *stores* et la *mise à jour faible* (sections 11.5 et 11.6) ;
4. quand le corps de l'effet est-il exécuté, et que se passe-t-il s'il appelle un callback, une fonction locale ou un composant enfant ? Ce sont les *points d'entrée*, la *pré-passe* des callbacks, le *tas* et l'*inter-composants* (sections 11.8, 11.9 et 11.11).

11.1.2 Ce que voit l'interpréteur

L'interpréteur ne voit jamais le TSX : il voit l'IR produite par le lowering (chapitre 7). Voici celle de Counter, imprimée par la sonde d'impression du chapitre 7 :

```
=== component Counter (param props) ===
render_cfg:
  B0:
    let n = StateVal(0) @4:9
    let setN = StateSetter(0) @4:12
    expr HookMarker(1, Undefined) @5:2
    => return Native<div>({#1 })[n]
hooks:
  #0 State init=0 @4:8
  #1 Effect deps=Absent @5:2
  B0:
    expr setN((n Add 1)) @6:4
    => return undefined
```

Le rendu est un bloc unique de trois instructions et un terminateur Return. La déstructuration de `useState` a déjà été réécrite : `n` est lié à la lecture du slot 0 (`StateVal(0)`), `setN` à son setter (`StateSetter(0)`) ; l'appel `useEffect` n'a laissé qu'un marqueur de site d'appel (`HookMarker`). Le corps de l'effet est un CFG à part, rangé dans la table des hooks. L'IR ne connaît que quatre instructions, définies dans `src/ir/stmt.rs` : `Let` et `Assign` (lier une variable), `MemberWrite` (écrire un champ, `obj.f = v`) et `ExprStmt` (une expression évaluée pour ses effets, `expr e` dans l'impression). Il n'y a pas d'instruction `return` : le retour est le terminateur du bloc.

11.1.3 Le calcul, à la main

Donnons-nous, pour l'instant sans justification, trois règles : (i) la lecture `StateVal(0)` rend le contenu courant du store d'état ; (ii) $n + 1$ se calcule sur les intervalles, $[a, b] + [1, 1] = [a + 1, b + 1]$; (iii) un appel `setN(v)` *joint* v au contenu du slot, il ne l'écrase pas. Le store est amorcé par la valeur initiale, $[0, 0]$. Le moteur enchaîne alors des passes « rendu puis effets » jusqu'à ce que l'état ne bouge plus (tableau 11.1).

passee	store au début	n lu	$n + 1$	store après la passe
amorçage	—	—	—	$[0, 0]$
1	$[0, 0]$	$[0, 0]$	$[1, 1]$	$[0, 0] \sqcup [1, 1] = [0, 1]$
2	$[0, 1]$	$[0, 1]$	$[1, 2]$	$[0, 2]$
3	$[0, 2]$	$[0, 2]$	$[1, 3]$	$[0, 3]$, puis widening : $[0, +\infty]$
4	$[0, +\infty]$	$[0, +\infty]$	$[1, +\infty]$	$[0, +\infty]$: stable, arrêt

TABLE 11.1 – Le point fixe de Counter, calculé à la main. Le widening (définition 3.6) intervient à la troisième itération, c'est-à-dire au seuil par défaut `widen_threshold = 3` du moteur.

Pour `Settle`, la première passe donne $[0, 0] \sqcup [5, 5] = [0, 5]$ et la deuxième n'ajoute rien : le moteur s'arrête après une itération. La sonde temporaire du dossier de notes, qui appelle directement `analyze_component` et imprime les stores convergés, le confirme :

```
-- Counter (intra, iterations=3)
state[0] = number[0, inf] widened=true
env[n] = Val(number[0, inf])
-- Settle (intra, iterations=1)
state[0] = number[0, 5] widened=false
env[m] = Val(number[0, 5])
```

Rappel React : Le bail-out de `setState`

React compare la nouvelle valeur d'un état à l'ancienne par `Object.is` et n'ordonne pas de nouveau rendu si elles sont égales. C'est ce qui rend terminante la boucle de `Settle` : après le premier `setM(5)`, les suivants ne changent plus rien. Dans l'analyse, le point fixe abstrait est l'image de ce bail-out : une écriture qui n'ajoute rien au join ne relance pas la boucle.

Les règles (i) à (iii) sont exactement celles du code, avec des nuances que le chapitre va détailler. Retenons dès maintenant la troisième, parce qu'elle structure tout le reste : le store d'état contient le *join de toutes les valeurs écrites*, initialiseur compris. Il ne dit pas « ce que vaut l'état maintenant » mais « ce que l'état peut valoir à un moment quelconque ». Un appel de *setter peut* avoir lieu ; il ajoute un comportement possible, il n'en retire aucun.

11.1.4 Le contrat : le trait `Transfer`

Le moteur ne connaît pas `StateValue` : il est générique en un paramètre `T` : `Transfer` (`src/engine/cfg_analyzer.rs`, l. 34). Le trait fixe les quatre opérations dont il a besoin :

Extrait 11.2 – `src/domains/mod.rs`, l. 114–144 — le contrat entre le moteur et un domaine (la quatrième méthode, `recompute_memo`, suit jusqu'à la ligne 161)

```

114 pub trait Transfer {
115     type Domain: AbstractDomain;
116
117     /// Abstract evaluation of an expression in the current abstract state.
118     fn eval_expr(
119         &self,
120         expr: &Expr,
121         env: &AbstractEnv<Self::Domain>,
122         ctx: &mut context::AnalysisCtx<Self::Domain>,
123     ) -> Self::Domain;
124
125     /// Execute a statement, updating `env`, `state`, `memo`, and `heap` in place.
126     fn exec_stmt(
127         &self,
128         stmt: &Stmt,
129         env: &mut AbstractEnv<Self::Domain>,
130         ctx: &mut context::AnalysisCtx<Self::Domain>,
131     );
132
133     /// Fire the side effects of `expr` evaluated in *effect position* – a bare
134     /// `expr`; `statement` or a concise-arrow `Return` body: the callback
135     /// pre-pass, the setter-call weak-update, and the inter-component eval a
136     /// component application performs. This is the single definition of
137     /// "run an expression for its effects"; the fixpoint engine's `Return`
138     /// handling calls it instead of fabricating a throwaway `Stmt::ExprStmt`.
139     fn exec_expr_effects(
140         &self,
141         expr: &Expr,
142         env: &mut AbstractEnv<Self::Domain>,
143         ctx: &mut context::AnalysisCtx<Self::Domain>,
144     );

```

- `eval_expr` : la valeur abstraite d'une expression. L'environnement est pris en lecture seule (`&`), l'`AnalysisCtx` en écriture (`&mut`) ; le chapitre 10 a décrit ce paquet (`component`, `state`, `memo`, `heap`, `query`, `inter`) et pourquoi l'environnement en est séparé : sa mutabilité diffère d'une opération à l'autre.
- `exec_stmt` : l'effet d'une instruction, en place, sur l'environnement et les stores.

- `exec_expr_effects` : les effets d’une expression évaluée « pour ses effets », soit une instruction `expr`, soit le corps d’une flèche concise `() => setN(1)`, que le lowering range dans un terminateur `Return`. Le moteur l’appelle sur chaque terminateur `Return` qu’il rencontre (`src/engine/cfg_analyzer.rs`, l. 84–86).
- `recompute_memo` : la valeur d’un `useMemo` ou d’un `useCallback`, recalculée après chaque passe de rendu (section 11.10).

Il n’existe qu’une implémentation, `StateValueTransfer`, une structure vide : toute la variation vient du contexte.

Extrait 11.3 – `src/domains/transfer/state_value.rs`, l. 24–54 — l’unique implémentation du trait

```

24 pub struct StateValueTransfer;
25
26 impl Transfer for StateValueTransfer {
27     type Domain = StateValue;
28
29     fn eval_expr(
30         &self,
31         expr: &Expr,
32         env: &AbstractEnv<StateValue>,
33         ctx: &mut AnalysisCtx<StateValue>,
34     ) -> StateValue {
35         eval_state_value(expr, env, ctx)
36     }
37
38     fn exec_stmt(
39         &self,
40         stmt: &Stmt,
41         env: &mut AbstractEnv<StateValue>,
42         ctx: &mut AnalysisCtx<StateValue>,
43     ) {
44         exec_stmt_with_callbacks(self, stmt, env, ctx);
45     }
46
47     fn exec_expr_effects(
48         &self,
49         expr: &Expr,
50         env: &mut AbstractEnv<StateValue>,
51         ctx: &mut AnalysisCtx<StateValue>,
52     ) {
53         exec_expr_effects(self, expr, env, ctx, 0);
54     }

```

Le partage des rôles est net : l’évaluation des expressions est *propre au domaine* (`eval_state_value`, dans `transfer/`), l’exécution des instructions est *entièrement déléguée* à l’interpréteur générique de `interp/` (`exec_stmt_with_callbacks`, `exec_expr_effects`). Cet interpréteur n’est pourtant complet que pour `StateValue` : le tas et le store partagé rangent des `StateValue` quel que soit le domaine, et passent par les ponts `as_state_value / from_state_value` du trait `AbstractDomain` (chapitre 9), qui rendent `None` et $\perp$ pour tout autre domaine (`src/domains/interp/interpreter.rs`, l. 168). Un second domaine perdrait silencieusement les captures de fermetures et les écritures de champ.

11.1.5 Le chemin des appels

La figure 11.1 donne le chemin complet, du moteur à l’évaluateur. Il servira de carte pour tout le chapitre.

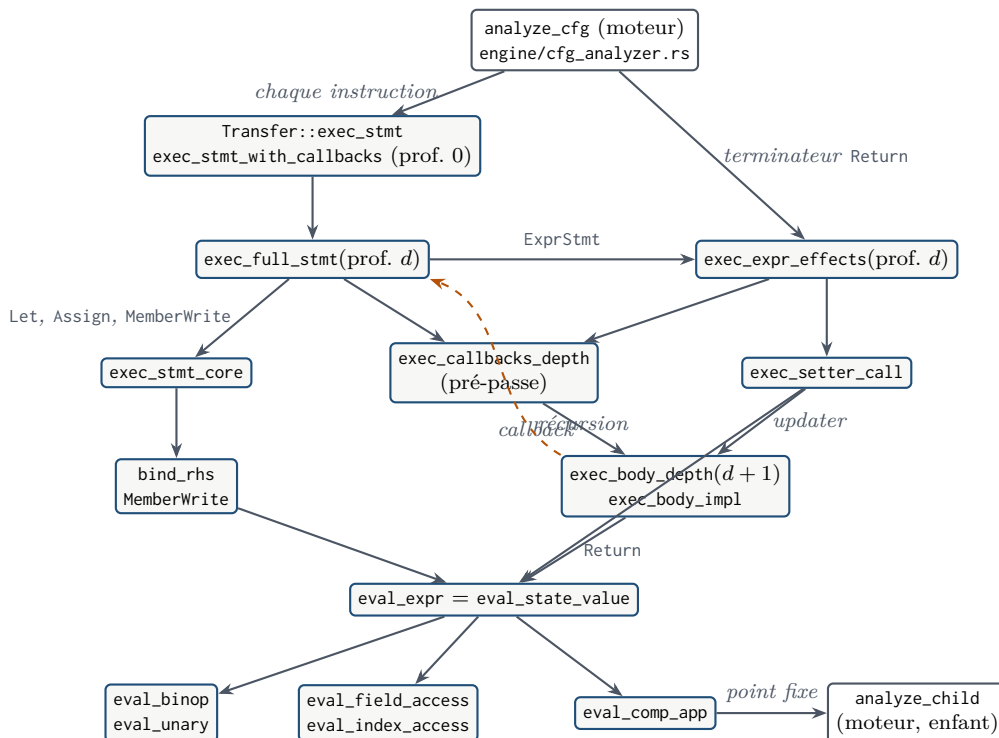


FIGURE 11.1 – Du moteur à l'évaluateur. Les boîtes blanches appartiennent au moteur (chapitres 12 et 16), les bleues à ce chapitre. La flèche tiretée est la récursion des corps imbriqués : un corps de callback ou de *functional updater* est exécuté instruction par instruction par le même `exec_full_stmt`, à la profondeur $d + 1$, plafonnée à `MAX_INLINE_DEPTH = 3`.

Invariant 11.1: L'évaluation est pure vis-à-vis de l'état React

`eval_state_value` ne met jamais à jour le store d'état. Les effets de bord (appel de setter, exécution d'un callback) sont tirés par l'interpréteur d'instructions, dans des positions syntaxiques précises (section 11.6). Deux exceptions assumées : `havoc_setter_props`, qui joint $\top$ dans un slot dont le setter part vers un enfant inanalysable, et `eval_comp_app`, qui lance l'analyse d'un enfant, laquelle écrit dans le store partagé (section 11.11).

Cet invariant vient de l'ADR-009 (« la pré-passe tire les effets, eval reste pur »). Il est commode : une même expression peut être évaluée plusieurs fois (par la pré-passe, puis par la liaison) sans que l'état n'enregistre deux écritures. Il a aussi un coût, que la section 11.12 chiffrera : un setter appelé ailleurs qu'en position d'effet n'écrit rien (défaut D2).

11.2 L'environnement abstrait

11.2.1 Un exemple : ce qu'on sait d'une variable

L'environnement répond à la question « que vaut la variable x en ce point du programme ? ». Prenons un composant qui mélange les cas courants : une prop, un appel à une fonction importée, une variable affectée sur deux branches, une autre sur une seule.

Extrait 11.4 – Ce qu'on sait d'une variable (/tmp/ch11/env.tsx)

```

1 import { useState } from "react";
2 import { format } from "../fmt";
3

```

```

4 export function Label({ flag }: { flag: boolean }) {
5   const [n] = useState(0);
6   const a = format(n);
7   let b;
8   if (flag) {
9     b = 1;
10  } else {
11    b = 2;
12  }
13  let c;
14  if (flag) {
15    c = 3;
16  }
17  return <div>{a}{b}{c}{n}</div>;
18 }

```

La sonde imprime l'environnement de sortie du rendu :

```

-- Label (intra, iterations=0)
  state[0] = number[0, 0]   widened=false
  env[flag] = Val(⊤)
  env[n] = Val(number[0, 0])
  env[a] = Val(⊤)
  env[b] = Val(number[1, 2])
  env[c] = Val(number[3, 3]|undefined)

```

Chaque ligne illustre une règle. `flag` est lu sur le paramètre de props, qui n'est lié à rien quand le composant est analysé seul : il vaut $\top$. `a` est le résultat d'un appel, que l'évaluateur n'exécute jamais : $\top$ aussi (section 11.3). `b` reçoit `[1, 1]` sur une branche, `[2, 2]` sur l'autre ; le bloc de jonction joint les deux environnements, d'où `[1, 2]`. `c` vaut `undefined` (le `let c` ; s'abaisse en `let c = undefined`) sur la branche qui ne l'affecte pas et `[3, 3]` sur l'autre : le join garde les deux sortes, ce que permet le produit (définition 10.1).

11.2.2 Quatre tables

Extrait 11.5 – `src/domains/stores/abstract_env.rs`, l. 46-60 — l'environnement abstrait

```

46 /// Per-variable abstract environment.
47 ///
48 /// `lookup` returns `D::top()` for unbound vars. `setter_bindings` is a
49 /// React-specific side-channel for setState. `locs` tracks heap allocation-site
50 /// `ExprId`s for FnLit/ObjectLit/ArrayLit vars; independent from `stabs`.
51 #[derive(Debug, Clone, PartialEq)]
52 pub struct AbstractEnv<D: AbstractDomain> {
53   stabs: HashMap<Var, D>,
54   locs: HashMap<Var, HashSet<ExprId>>,
55   setter_bindings: HashMap<Var, HookLabel>,
56   /// Side-channel for `useCallback` bindings: the body lives in the hook
57   /// entry (`QueryContext::callback_body`), not the heap, so calls through
58   /// the variable can still be executed for side effects.
59   callback_bindings: HashMap<Var, HookLabel>,
60 }

```

L'environnement n'est pas une seule table mais quatre, toutes privées (tableau 11.2). Le nom `stabs` est un vestige de l'ADR-002, quand la seule valeur abstraite était une stabilité.

Les deux dernières sont des *canaux latéraux* propres à React : savoir qu'une variable est le setter du slot 3, ou le callback du hook 5, permet d'exécuter ses appels (sections 11.6 et 11.8). La lecture principale est conservatrice :

table	contenu	écrite par	lue par
stabs	valeur abstraite de chaque variable	extend	lookup, lookup_env_val
locs	sites d'allocation que la variable peut désigner (section 11.9)	extend_loc (union)	lookup_env_val
setter_bindings	variable $\mapsto$ label du useState dont elle est le setter	bind_setter	setter_label
callback_bindings	variable $\mapsto$ label du useCallback qu'elle désigne	bind_callback	callback_label

TABLE 11.2 – Les quatre tables de AbstractEnv.

Extrait 11.6 – src/domains/stores/abstract_env.rs, l. 78–93 — la lecture d'une variable

```

78  /// Conservative lookup: `D::top()` for any variable not in the map.
79  pub fn lookup(&self, var: &str) -> D {
80      self.stabs.get(var).cloned().unwrap_or_else(D::top)
81  }
82
83  /// Returns heap location set for `var` if it was bound to an allocating expr.
84  /// Returns `None` when the variable has no Loc (external/imported function).
85  pub fn lookup_env_val(&self, var: &str) -> Option<EnvVal<D>> {
86      if let Some(ids) = self.locs.get(var) {
87          return Some(EnvVal::Loc {
88              ids: ids.clone(),
89              val: self.lookup(var),
90          });
91      }
92      self.stabs.get(var).map(|v| EnvVal::Val(v.clone()))
93  }

```

lookup rend $\top$ pour toute variable absente. lookup_env_val rend une *vue* EnvVal, qui porte la valeur et, si la variable a des sites, l'ensemble de ses sites ; cette vue sera détaillée avec le tas. L'écriture extend écrase la valeur et *préserve* les sites ; extend_loc *ajoute* un site, sans jamais en retirer (src/domains/stores/abstract_env.rs, l. 100–108).

Définition 11.1: Environnement abstrait

Un *environnement abstrait* ρ associe à chaque variable x une valeur $\rho(x) \in \text{StateValue}$, avec $\rho(x) = \top$ pour toute variable non liée. Sa concrétisation est l'ensemble des environnements concrets σ tels que $\sigma(x) \in \gamma(\rho(x))$ pour tout x . L'ordre, le join et le widening sont ponctuels.

Invariant 11.2: Une variable inconnue vaut $\top$

Les paramètres, les imports, les globales (fetch, window), toute liaison perdue se lisent $\top$. Perdre une liaison de *valeur* est donc sound. Les trois autres tables ne suivent pas cette règle : elles se lisent « rien par défaut ». Perdre une liaison de setter, de callback ou de site n'est *pas* sound, car ce qui n'est pas trouvé n'est pas exécuté.

Cette asymétrie est la racine de plusieurs défauts de la section 11.12 (D1, D10) : partout où une liaison se perd, la valeur retombe prudemment à $\top$ mais l'écriture de setter qui en dépendait disparaît.

11.2.3 Le join

Extrait 11.7 – src/domains/stores/abstract_env.rs, l. 130–173 — le join de deux environnements

```

130 fn join_stabs(a: &HashMap<Var, D>, b: &HashMap<Var, D>) -> HashMap<Var, D> {
131     let mut out = HashMap::new();
132     for (k, v) in a {
133         let w = map_get_or(b, k, D::top());
134         out.insert(k.clone(), v.join(&w));
135     }
136     for k in b.keys() {
137         if !a.contains_key(k) {
138             out.insert(k.clone(), D::top());
139         }
140     }
141     out
142 }
143
144 fn join_locs(
145     a: &HashMap<Var, HashSet<ExprId>>,
146     b: &HashMap<Var, HashSet<ExprId>>,
147 ) -> HashMap<Var, HashSet<ExprId>> {
148     let mut out = a.clone();
149     for (k, ids) in b {
150         out.entry(k.clone()).or_default().extend(ids);
151     }
152     out
153 }
154
155 /// Pointwise join. Variables present in only one side → `D::top()`.
156 pub fn join(&self, other: &Self) -> Self {
157     let stabs = Self::join_stabs(&self.stabs, &other.stabs);
158     let locs = Self::join_locs(&self.locs, &other.locs);
159     let mut setter_bindings = self.setter_bindings.clone();
160     for (k, &v) in &other.setter_bindings {
161         setter_bindings.entry(k.clone()).or_insert(v);
162     }
163     let mut callback_bindings = self.callback_bindings.clone();
164     for (k, &v) in &other.callback_bindings {
165         callback_bindings.entry(k.clone()).or_insert(v);
166     }
167     AbstractEnv {
168         stabs,
169         locs,
170         setter_bindings,
171         callback_bindings,
172     }
173 }

```

Variable par variable :

- présente des deux côtés : join de StateValue ;
- présente d'un seul côté : $\top$, ce qui est cohérent avec « absente vaut $\top$ » ;
- sites : union ensembliste ;
- liaisons de setter et de callback : union à *priorité gauche* (`entry(k).or_insert(v)`). Si une même variable désigne le setter de a sur un chemin et celui de b sur l'autre, le second est oublié (défaut D10).

Le widening `widen_to` (src/domains/stores/abstract_env.rs, l. 175–208) suit le même schéma, avec `widen_to` ponctuel sur les valeurs et les seuils de l'ADR-014 ; `analyze_cfg` l'emploie sur les arcs retour (src/engine/cfg_analyzer.rs, l. 97–111).

Décision : ADR-010 — deux tables plutôt qu'une

La tentation était une seule table $\text{Var} \rightarrow \text{EnvVal}$, où EnvVal vaut soit une valeur, soit un site. L'ADR-010 la refuse : « a single map $\text{EnvVal} = \text{Val} \mid \text{Loc}$ was tempting but $\text{env.extend}(\text{var}, \text{val})$ overwrote the Loc previously placed by $\text{env.extend_loc}(\text{var}, \text{id})$ ». La liaison $\text{const } f = () \Rightarrow \dots$ pose d'abord le site, puis la valeur ; avec une seule table, la seconde écriture efface la première et $f()$ n'est plus exécutable. Deux tables indépendantes suppriment le conflit. EnvVal subsiste comme *vue* de lecture, et comme type des champs d'un objet du tas.

Piège

$\text{AbstractEnv}::\text{bottom}()$ est l'environnement vide. C'est bien le plus petit élément pour leq , mais ce n'est *pas* l'élément neutre du join : comme une clé présente d'un seul côté donne $\top$, $\text{bottom}().\text{join}(\&e)$ rend un environnement où toutes les variables de e valent $\top$. En lecture, l'environnement vide se comporte comme « tout $\top$ ». Les appelants évitent le piège en ne joignant que des environnements calculés : analyze_cfg insère le premier environnement reçu sans join (`src/engine/cfg_analyzer.rs`, l. 98), exec_body_impl filtre les prédécesseurs non encore visités. Symétriquement, leq lit une clé absente de droite comme $\perp$ et non $\top$: l'asymétrie est sans conséquence aujourd'hui, car $\text{AbstractEnv}::\text{leq}$ n'est appelé qu'en tests ; le moteur compare les environnements par égalité structurelle (`src/engine/cfg_analyzer.rs`, l. 114).

11.3 L'évaluateur d'expressions

11.3.1 Un banc d'essai

Avant de lire le code, observons ce qu'il calcule sur une série d'expressions courtes.

Extrait 11.8 – Des opérateurs (`/tmp/ch11/ex1.tsx`)

```
1 import { useState, useEffect } from "react";
2
3 export function Ops() {
4   const [n, setN] = useState(0);
5   const [label, setLabel] = useState("a");
6   const masked = n & 7;
7   const kind = typeof n;
8   const text = `n=${n}`;
9   const neg = ~5;
10  const plus = +"5";
11  const both = label + "b";
12  const cmp = n < 3;
13  const notTrue = !true;
14  const notNum = !n;
15  const nul = null + 1;
16  useEffect(() => {
17    setLabel(both);
18  }, [both]);
19  return <div>{masked}{kind}{text}{neg}{plus}{cmp}{notTrue}{notNum}{nul}</div>;
20 }
```

```
-- Ops (intra, iterations=4)
state[0] = number[0, 0] widened=false
state[1] = string widened=true
env[n] = Val(number[0, 0])
env[label] = Val(string)
env[masked] = Val(number[0, 7])
env[kind] = Val(string{"number"})
```

```

env[text] = Val(⊤)
env[neg] = Val(number[-6, -6])
env[plus] = Val(number[5, 5])
env[both] = Val(string)
env[cmp] = Val(boolean)
env[notTrue] = Val(false)
env[notNum] = Val(⊤)
env[nul] = Val(number[1, 1])

```

Plusieurs réponses sont exactes (~ 5 vaut -6 , $+5$ vaut 5 , $\text{null} + 1$ vaut 1), d'autres sont des approximations sûres : un masque $n \& 7$ tient dans $[0, 7]$, une comparaison est « un booléen », et trois valent $\top$ alors que JavaScript saurait dire mieux (`text`, `notNum`). L'ensemble de chaînes de label grandit à chaque passe ("`a`", "`ab`", "`abb`", ...), dépasse le seuil de `StrConst` et devient `string`, le $\top$ de la composante chaîne (chapitre 9). Toutes ces réponses sortent d'une seule fonction.

11.3.2 La fonction `eval_state_value`

C'est une récursion structurelle sur l'arbre d'expression, un bras par variante de `Expr` (chapitre 6). Elle évalue les sous-expressions de gauche à droite, chacune une fois ; son coût est linéaire en la taille de l'expression, sauf pour une application de composant (une analyse complète de l'enfant) et pour la concaténation de deux ensembles de chaînes (un produit cartésien, borné par le seuil de `StrConst`). Voici le début :

Extrait 11.9 – `src/domains/transfer/state_value.rs`, l. 106–136 — l'évaluateur : littéraux, variables, hooks

```

106 fn eval_state_value(
107     expr: &Expr,
108     env: &AbstractEnv<StateValue>,
109     ctx: &mut AnalysisCtx<StateValue>,
110 ) -> StateValue {
111     match expr {
112         Expr::Lit(Prim::Int(n)) => StateValue::number(Interval::point(*n as f64)),
113         Expr::Lit(Prim::Float(f)) => StateValue::number(Interval::point(*f)),
114         Expr::Lit(Prim::Bool(b)) => {
115             StateValue::boolean(if *b { BoolVal::True } else { BoolVal::False })
116         }
117         Expr::Lit(Prim::String(s)) => StateValue::str_singleton(s.to_string()),
118         Expr::Lit(Prim::Null) => StateValue::null(),
119         Expr::Lit(Prim::Unit) => StateValue::undefined(),
120
121         Expr::Var(v) => env.lookup(v),
122         Expr::StateVal(label) => {
123             let mut val = ctx.state.get(*label);
124             // ADR-017 read-side conversion: the store holds the join of
125             // *written* values (event view); what a render *reads* can only
126             // change at setter events of this slot (cross-render view). The
127             // written value's allocation freshness (`PerRender`) must not
128             // leak into reads. Assumes sets happen outside render – the
129             // violation has its own diagnostic (`setter-in-render`).
130             if val.reference != Stability::Bottom {
131                 val.reference = Stability::versioned_by(ctx.component, *label);
132             }
133             val
134         }
135         Expr::StateSetter(label) => StateValue::component_setter(ctx.component, *label),
136         Expr::MemoVal(label) | Expr::CallbackVal(label) => ctx.memo.get(*label),

```

puis les marqueurs de hooks et les allocations :

Extrait 11.10 – `src/domains/transfer/state_value.rs`, l. 137–163 — l'évaluateur : marqueurs, allocations, éléments

```

137      // Call-site marker. A React hook with no tracked result really does
138      // return `undefined`; an unresolved custom hook returns T. Reading the
139      // latter as `undefined` made it *provably stable* (`to_stability`
140      // joins `Stable` for `undef`) and silenced every stability-gated rule
141      // on it – a false negative.
142      Expr::HookMarker(_, MarkerVal::Undefined) => StateValue::undefined(),
143      Expr::HookMarker(_, MarkerVal::Unknown) => StateValue::top(),
144      // `useRef` hands back the same container every render – a reference,
145      // and a stable one. Both halves matter: `undefined` was stable too, but
146      // it was not a reference, so the identity was invisible.
147      Expr::HookMarker(_, MarkerVal::StableRef) => StateValue::reference(Stability::Stable),
148      // A summarized library hook reads exactly as its summary; the marker
149      // is kept (rather than replaced by a bare `SummaryVal`) so the label
150      // stays anchored at the call site.
151      Expr::HookMarker(_, MarkerVal::Summary(sv)) => summary_value(sv),
152
153      Expr::ObjectLit { .. } => StateValue::reference(Stability::PerRender),
154      Expr::ArrayLit { .. } => StateValue::reference(Stability::PerRender),
155      Expr::FnLit { .. } => StateValue::reference(Stability::PerRender),
156      Expr::NativeElem { .. } => StateValue::reference(Stability::Stable),
157
158      Expr::CompApp {
159          name,
160          props,
161          origin,
162          ..
163      } => eval_comp_app(name, props, origin.as_deref(), env, ctx),

```

La suite (`src/domains/transfer/state_value.rs`, l. 165–190) évalue les opérandes d'un `BinOp` ou d'un `UnaryOp` puis délègue à `eval_binop` ou `eval_unary`, traite les appels, les `new`, les accès de membres et d'index, et traverse les annotations. La tableau 11.3 récapitule tous les bras ; on la détaille ensuite par familles.

11.3.3 Littéraux, variables et hooks

Les littéraux sont exacts. Un entier du source devient `Int` quand sa valeur est entière et petite, `Float` sinon ; les deux donnent un intervalle ponctuel. Une variable se lit dans l'environnement, `T` si elle n'y est pas.

Trois bras lisent des hooks. `StateVal(1)` lit le store d'état ; il applique la *conversion côté lecture* de l'ADR-017, que le chapitre 10 a démontrée et que la section 11.7 remplacera dans l'interpréteur. `StateSetter(1)` rend un setter *porteur de son propriétaire*, `SetterVal::One(composant, 1)`, même en analyse intra-composant : c'est ce qui permet de suivre un setter passé en prop jusqu'à l'enfant qui l'appelle (section 11.11), et c'est aussi l'origine du défaut D3. `MemoVal` et `CallbackVal` lisent le même `MemoStore` : un `useCallback` n'est qu'un mémo dont la valeur est une fonction.

Le marqueur `HookMarker` ancre le site d'appel d'un hook dont la valeur n'est pas suivie. Ses quatre formes ont chacune une histoire, racontée par les commentaires du code : un hook React sans valeur (`useEffect`) rend vraiment `undefined` ; un hook custom non résolu rendait autrefois `undefined` lui aussi, ce qui le faisait passer pour *prouvablement stable* et taisait toutes les règles de stabilité (un faux négatif corrigé par `b7bc459`) ; il rend désormais `T`. `useRef` rend le même conteneur à chaque rendu : une référence, et une référence `Stable`. Un hook de bibliothèque résumé se lit comme son résumé, par une table unique, `summary_value` (`src/domains/transfer/state_value.rs`,

expression de l'IR	valeur abstraite	remarque
Lit(Int(n)), Lit(Float(f))	number([n,n])	intervalle ponctuel
Lit(Bool(b))	boolean(True False)	
Lit(String(s))	str_singleton(s)	ensemble {s}
Lit(Null), Lit(Unit)	null(), undefined()	undefined s'abaisse en Lit(Unit)
Var(v)	env.lookup(v)	⊤ si non liée
StateVal(l)	ctx.state.get(l), référence convertie	section 11.7
StateSetter(l)	component_setter (ctx.component, l)	le setter porte son propriétaire
MemoVal(l), CallbackVal(l)	ctx.memo.get(l)	⊤ tant que non calculé
HookMarker(_, Undefined)	undefined()	hook React sans valeur suivie
HookMarker(_, Unknown)	⊤	hook custom ni inline ni résumé
HookMarker(_, StableRef)	ref(Stable)	useRef
HookMarker(_, Summary(s)), SummaryVal(s)	summary_value(s)	table unique des résumés
ObjectLit, ArrayLit, FnLit, New	ref(PerRender)	fait must : neuf à chaque rendu
NativeElem, CompApp	ref(Stable)	voir D8
BinOp, UnaryOp	eval_binop, eval_unary	opérandes d'abord
Call	ref(PerRender) ou ⊤	allocateurs connus seulement
FieldAccess, IndexAccess	lecture par le tas	section 11.9
TSAnotated(e)	valeur de e	l'annotation est ignorée

TABLE 11.3 – Les bras de eval_state_value, src/domains/transfer/state_value.rs, l. 106–190.

l. 192–222), « so the marker and the standalone SummaryVal can never disagree » : Top et Held donnent ⊤, StableRef donne ref(Stable), UnstableRef ref(PerRender), un Wrapper l'une ou l'autre selon son drapeau. Deux lignes méritent l'attention. Un résumé Shape (useForm(), dont seuls certains membres sont promis stables) vaut ⊤ : la promesse porte sur les membres, qui seront lus par le tas (section 11.9). Un Navigator non stable vaut ⊤ et non PerRender : on ne prouve pas sa fraîcheur.

11.3.4 Allocations et éléments

Un littéral d'objet, de tableau ou de fonction évalué pendant un rendu est une référence neuve à chaque rendu : ref(PerRender) est un fait *must* (définition 10.3). C'est le fait que la règle *always-unstable-deps* exploite. `new X()` alloue lui aussi (ajout du commit e67b10a, #158) ; ses membres restent ⊤. L'évaluation d'un littéral *ne peuple pas* le tas : seule la liaison d'une variable (`bind_rhs`) ou le passage en prop le fait.

Les éléments JSX, natifs (`<span/>`) ou composants (`<Child/>`), valent ref(Stable). C'est juste pour dire « la valeur d'un élément ne porte pas d'information utile », mais c'est faux au sens d'Object.is : en JavaScript, chaque rendu alloue un nouvel objet élément. Le chapitre 10 a relevé ce faux négatif ; on le retrouvera sous le nom D8.

11.3.5 Appels

Un appel n'est *jamaïs* exécuté par l'évaluateur : sa valeur est ⊤, sauf pour une liste blanche d'appels qui allouent toujours.

Extrait 11.11 – src/domains/transfer/state_value.rs, l. 238–259 — les appels dont le résultat est toujours neuf

```

238 fn returns_fresh_reference(callee: &Expr) -> bool {
239     match callee {
240         // `structuredClone(x)` always allocates a deep copy.
241         Expr::Var(v) => v == "structuredClone",
242         Expr::FieldAccess { obj, field } => match field.as_str() {
243             // Array methods returning a NEW array, array-only names (a
244             // string receiver has none of these).
245             "map" | "filter" | "flat" | "flatMap" | "toSorted" | "toReversed" | "toSpliced"
246             | "with" | "split" => true,
247             // Static allocators – receiver-restricted: a bare `.from`/`.keys`
248             // on an unknown object could be anything.
249             "from" | "of" => matches!(obj.as_ref(), Expr::Var(v) if v == "Array"),
250             "keys" | "values" | "entries" | "fromEntries" => {
251                 matches!(obj.as_ref(), Expr::Var(v) if v == "Object")
252             }
253             "parse" => matches!(obj.as_ref(), Expr::Var(v) if v == "JSON"),
254             _ => false,
255         },
256         Expr::TSAnnotated(inner) => returns_fresh_reference(inner),
257         _ => false,
258     }
259 }

```

La liste est volontairement courte. Elle exclut `slice` et `concat`, qui rendent une primitive sur une chaîne (`id.slice(0, 8)`) : prétendre `PerRender` serait une fausse preuve (#22, décision enregistrée). Elle exclut aussi les méthodes en place (`sort`, `reverse`, `fill`, `Object.assign`), qui rendent le receveur, soit le fait inverse. Les allocateurs statiques (`from`, `keys`, `parse`...) ne comptent que sur leur receveur connu (`Array`, `Object`, `JSON`).

Deux conséquences sont à garder en tête. D'abord, la *valeur de retour* d'une fonction locale appelée n'est jamais calculée : même quand l'interpréteur exécute son corps pour ses effets (section 11.8), il jette le retour. Ensuite, les *effets* d'un appel (un setter, un callback) ne passent pas par ici : ils sont tirés par l'interpréteur d'instructions.

11.3.6 Opérateurs

Le chapitre 9 a décrit en détail `eval_binop` et `eval_bitwise` (`src/domains/transfer/state_value.rs`, l. 752–929) ; on n'en garde ici que ce qu'il faut pour calculer à la main.

- `+`, `-`, `*` : arithmétique d'intervalles quand les deux opérandes sont dans `{nombre, null}`, avec `ToNumber(null) = 0` (vue `as_arith`, chapitre 10) ; pour `+`, concaténation exacte si les deux opérandes sont des ensembles finis de chaînes, « une chaîne » si ce sont des chaînes mal connues ; `⊥` dans tous les autres cas, y compris `"n" + n`.
- `/` : toujours `⊥` (un intervalle ne contient ni `NaN` ni `±Infinity`) ; `%` et `**` : exacts quand `NaN` est exclu, `⊥` sinon.
- comparaisons, `in`, `instanceof` : `boolean(Top)`. Rien n'est décidé, mais le résultat *n'est qu'un booléen*, ce qui garde les composantes nombre et chaîne à `⊥`.
- `&&`, `||` : `⊥` ; en pratique ces variantes n'arrivent jamais ici (section 11.4).
- bit à bit : toujours un nombre de la plage `int32` (`uint32` pour `»>`), raffiné pour un masque constant (`x & 7` dans `[0, 7]`) ou un décalage constant. Un opérande `⊥` rend `⊥` : un chemin tué par un raffinement doit rester mort.

Les opérateurs unaires sont propres à ce fichier :

Extrait 11.12 – `src/domains/transfer/state_value.rs`, l. 941–977 — les opérateurs unaires (début)

```

941 fn eval_unary(op: &UnaryOp, val: StateValue) -> StateValue {

```

```

942     match op {
943       UnaryOp::Neg => match as_arith(&val) {
944         Some(i) => StateValue::number(i.neg()),
945         None => StateValue::top(),
946       },
947       UnaryOp::Not => {
948         // Boolean-only operand inverts precisely; anything else is Top.
949         if val.num.is_bottom()
950           && val.str == StrConst::Bottom
951           && val.reference == Stability::Bottom
952           && !val.null
953           && !val.undef
954           && val.setter == SetterVal::Bottom
955           && !val.other
956         {
957           match val.boolean {
958             BoolVal::True => StateValue::boolean(BoolVal::False),
959             BoolVal::False => StateValue::boolean(BoolVal::True),
960             _ => StateValue::boolean(BoolVal::Top),
961           }
962         } else {
963           StateValue::top()
964         }
965       }
966       // `typeof x` is *always* a string, and an exact one whenever the operand
967       // has a single inhabited kind. That exactness is what makes
968       // `typeof x === "string"` a narrowable guard instead of `BoolVal::Top`.
969       UnaryOp::TypeOf => {
970         if val.is_bottom_value() {
971           return StateValue::bottom();
972         }
973         match val.typeof_name() {
974           Some(name) => StateValue::str_singleton(name.to_string()),
975           None => StateValue::str_top(),
976         }
977       }
978     }

```

- $-x$: négation d'intervalle, $\top$ hors {nombre, null};
- $!x$: exact sur un booléen pur, $\top$ sinon;
- `typeof x` : une chaîne, exacte ("number", "object" pour null, "function" pour un setter...) quand une seule sorte est habitée, string sinon. Cette exactitude fait de `typeof x === "string"` une garde raffinable;
- $\sim x$: $-(\text{ToInt32}(x) + 1)$, bornes échangées dans int32, plage int32 sinon;
- $+x$: ToNumber, pas l'identité; un booléen donne 0, 1 ou $[0, 1]$, un ensemble de chaînes donne l'enveloppe de leurs valeurs si *toutes* se lisent comme des nombres finis (`coerce_to_number`), $\top$ sinon;
- `UnaryOp::Unknown` : $\top$.

La fin de la fonction ($\sim$, $+$ et le cas inconnu) occupe `src/domains/transfer/state_value.rs`, l. 978–1004, et `coerce_to_number` `src/domains/transfer/state_value.rs`, l. 1006–1040.

Piège

Deux imprécisions surprennent à la lecture de la sonde. `!n` sur un nombre vaut $\top$ et non boolean : la politique « c'est toujours un booléen » des comparaisons n'a pas été appliquée à la négation. Un gabarit `'n=$n'` vaut $\top$ et non string : le lowering l'a réécrit en `"n=" + n`, et `+` entre une chaîne et un nombre tombe dans le cas « tout le reste ». Les deux

sont `sound` ; ils sont simplement plus lâches que JavaScript. La lecture des chaînes par `coerce_to_number` utilise l'analyseur de Rust (`s.trim().parse()`), plus strict que `ToNumber` : `+""` vaut 0 en JavaScript mais $\top$ ici. Là encore, c'est `sound`.

11.3.7 Membres et annotations

`o.f` et `xs[0]` se lisent dans le tas abstrait ; on y reviendra en section 11.9, une fois le tas présenté. Un index non constant (`xs[i]`) vaut $\top$. Une annotation de type (`TSAnnotated`) est transparente : l'ADR-020 a explicitement refusé de brancher les types TypeScript dans le domaine.

11.4 Ce que le lowering a déjà fait

L'évaluateur n'a aucun bras pour les gabarits, les ternaires, `&&`, `||`, `??`, le chaînage optionnel, les spreads, `++`, `void`, `delete`, `await` ou les séquences. Ces constructions n'arrivent jamais jusqu'à lui : le lowering (`src/lowering/expr_lower.rs`, chapitre 7) les a réécrites en constructions déjà traitées. Pour prédire une valeur abstraite, il faut donc connaître ces réécritures (tableau 11.4).

source	IR produite	conséquence pour la valeur
<code>'q\$er'</code>	<code>("q" + e) + "r"</code>	exact si <code>e</code> est une chaîne connue ; $\top$ si c'est un nombre
<code>tag'...\$e...'</code>	<code>Call(tag, [e, ...])</code>	$\top$ (appel), les lectures de <code>e</code> survivent
<code>a ? b : c</code>	diamant : <code>Branch(a)</code> , <code>let __tN = b /</code> <code>let __tN = c</code> , jonction qui lit <code>__tN</code>	join des deux branches <i>par le CFG</i> , avec raffinement sur <code>a</code>
<code>a && b, a b, a ?? b</code>	<code>let __tN = a; Branch(__tN); __tN</code> <code>:= b</code> dans le bloc droit	idem ; <code>b</code> est en position d'Assign (compte pour D2)
<code>a?.b, f?.(x)</code>	comme <code>a.b, f(x)</code>	la nullité du receveur n'est pas modélisée (compte pour D5)
<code>...o, k: v</code>	membre sous une clé synthétique <code>"...N"</code>	seuls les membres après le dernier spread sont suivis
<code>[...xs]</code>	<code>xs</code> gardé comme élément, arité <code>AtLeast(n)</code>	aucune influence sur la valeur
<code>f(...args)</code>	<code>args</code> émis comme instruction	un setter passé par spread n'est pas un argument (#76)
<code>[k]: v, get x()</code>	clés synthétiques <code>"[computed]N"</code> , <code>"[accessor]N"</code>	membre introuvable par son nom
<code>i++, -i</code>	<code>i := i + 1</code> (ou <code>- 1</code>), valeur <code>Var(i)</code>	post-écriture pour les deux formes
<code>o.f++, delete o.f</code>	<code>MemberWrite opaque / de undefined</code>	mise à jour faible du champ
<code>void e, (a, b, c)</code>	<code>ExprStmt</code> pour les effets, puis la valeur	effets conservés
<code>await e</code>	<code>let __tN = e</code> , puis coupure de bloc	la suite est dans un bloc successeur
<code>this, super</code>	<code>SummaryVal(Top)</code>	$\top$
classe	<code>ObjectLit</code> de ses membres	référence fraîche

TABLE 11.4 – Les réécritures du lowering qui déterminent la valeur calculée (d'après `src/lowering/expr_lower.rs`).

Exemple 11.1: Un ternaire n'est pas évalué par l'évaluateur

L'effet `setB((x) => (x < 10 ? x + 1 : x))` devient, dans l'IR, un corps de fonction à quatre blocs :


```

expr setB(fn#8(x) {
  B0:
    => branch (x Lt 10) ? B1 : B2
  B1:
    let __t0 = (x Add 1) @22:26
    => jump B3
  B2:
    let __t0 = x @22:34
    => jump B3
  B3:
    => return __t0
  edges: 0->1[T] 0->2[F] 1->3[U] 2->3[U]
}) @22:4

```

L'évaluateur ne voit que `x Lt 10`, `x Add 1`, `x` et `__t0`. Le « ternaire » est le join des environnements des blocs B1 et B2 au bloc B3. C'est à l'interpréteur de corps (section 11.8) de le calculer correctement ; on verra qu'il ne le fait pas (défaut D1).

Décision : ADR-020 n° 1 — garder le diamant de `&&/||`

On pourrait ajouter à l'IR un nœud plat `LogicalOp` évalué par l'évaluateur, comme `BinOp`. L'ADR-020 le refuse : le diamant modélise les *effets* d'un court-circuit (dans `flag && setA(1)`, l'appel n'a lieu que sur une branche) et permet au moteur de raffiner l'environnement sur la condition. Le coût accepté est une valeur moins précise quand l'expression est lue, et la dispersion du rhs dans un `Assign`, qui a une conséquence imprévue : défaut D2. Le chapitre 7 note en outre que la nature des deux arêtes d'un court-circuit encode l'opérateur, pas la valeur de vérité ; le point fixe n'en dépend pas, car il raffine d'après `then_/else_`.

11.5 Les stores

L'environnement décrit un *point* du programme : il change d'un bloc à l'autre, et le moteur en garde un par bloc. Ce que React conserve *d'un rendu à l'autre* (l'état, les mémos, les objets qui survivent) ne peut pas vivre là : il faut des structures dont la portée est le composant, voire le programme. Ce sont les *stores*.

11.5.1 StateStore : l'état React, sujet du point fixe

Extrait 11.13 – `src/domains/stores/state_store.rs`, l. 7–45 — le store d'état : lecture, mise à jour faible, fusion

```

7 /// Maps each `useState` / `useReducer` hook label to the current abstract value
8 /// of its state. Starts at `D::bottom()` and is refined by detected setter
9 /// calls during the worklist analysis.
10 #[derive(Debug, Clone, PartialEq)]
11 pub struct StateStore<D: AbstractDomain>(HashMap<HookLabel, D>);
12
13 impl<D: AbstractDomain> Default for StateStore<D> {
14     fn default() -> Self {
15         StateStore(HashMap::new())
16     }
17 }
18
19 impl<D: AbstractDomain> StateStore<D> {
20     pub fn new() -> Self {
21         Self::default()
22     }
23 }

```

```

24    /// Returns `D::bottom()` for labels not yet updated by any setter call.
25    pub fn get(&self, label: HookLabel) -> D {
26        map_get_or(&self.0, &label, D::bottom)
27    }
28
29    /// Monotone update: `self[label] = self[label]  $\sqcup$  val`.
30    pub fn update(&mut self, label: HookLabel, val: D) {
31        let current = self.get(label);
32        self.0.insert(label, current.join(&val));
33    }
34
35    /// Pointwise merge over the union of keys, absent keys read as `D::bottom`.
36    /// The per-key combinator `f` is the only thing `join`/`widen`/`widen_to`
37    /// differ by.
38    fn merge_with(&self, other: &Self, f: impl Fn(&D, &D) -> D) -> Self {
39        let mut out = self.0.clone();
40        for (k, v) in &other.0 {
41            let cur = map_get_or(&out, &k, D::bottom);
42            out.insert(k, f(&cur, v));
43        }
44        StateStore(out)
45    }

```

Le store associé à chaque label de `useState` (ou de `useReducer`) est une `StateValue`. Trois propriétés le définissent.

- **$\perp$ par défaut.** Un label jamais écrit se lit $\perp$, « impossible », à l'inverse de l'environnement. C'est correct parce que tout slot est *amorcé* avant la boucle par la valeur de son initialiseur (ci-dessous) : un $\perp$ ne peut provenir que d'un hook mal extrait.
- **Mise à jour faible.** `update` ne remplace jamais : elle joint. Un appel de setter ajoute un comportement possible (sémantique may). Le store contient donc, à tout instant, le join de toutes les valeurs écrites, initialiseur compris : c'est la *vue événement* de l'ADR-017.
- **Fusion sur l'union des clés.** `join`, `widen` et `widen_to` ne diffèrent que par le combinateur passé à `merge_with`.

L'amorçage se fait dans le moteur, une fois par analyse de composant (`src/engine/fixpoint.rs`, l. 314–353) ; pour chaque `HookEntry::State`, la valeur initiale est calculée ainsi, puis jointe au slot par `state.update(*label, init_val)` :

Extrait 11.14 – `src/engine/fixpoint.rs`, l. 336–348 — l'amorçage d'un slot par son initialiseur

```

336    match init {
337        // Lazy initializer `useState(() => expr)`: React runs
338        // the thunk once at mount and stores its RETURN value
339        // – the state is never the closure itself. Abstracting
340        // the FnLit (reference(Unstable)) made every lazy-init
341        // state slot an "unstable dep" (corpus FP, TODO.md F2).
342        Expr::FnLit {
343            params, body_cfg, ..
344        } if params.is_empty() => crate::domains::interp::exec_body(
345            transfer, body_cfg, &init_env, &mut ac,
346        ),
347        _ => transfer.eval_expr(init, &init_env, &mut ac),
348    }

```

Deux cas. Un initialiseur paresseux `useState(() => e)` est exécuté comme un corps de fonction (`exec_body`, section 11.8) et c'est son *retour* qui amorce le slot : React exécute la fonction une fois au montage et range son résultat, jamais la fonction elle-même. Tout autre initialiseur

est simplement évalué. Dans les deux cas, le contexte est `AnalysisCtx::null` avec un store vide, des mémos vides et un tas neuf : l’initialiseur voit l’environnement d’entrée du composant (constantes de module, props liées par un parent), mais ni l’état ni le tas du composant.

La boucle du point fixe (chapitre 13) n’emploie le store qu’à travers quelques méthodes : `bottom` pour l’état initial et les accumulateurs de passe, `join` pour combiner rendu, effets, handlers et tranche partagée, `leq` pour le test d’arrêt « le nouvel état est-il $\sqsubseteq$ l’ancien ? », `widen_to` à partir de `widen_threshold` itérations, `widen` (sans seuils) au garde-fou des cent itérations, et `changed_labels` pour savoir quels slots ont été soumis au widening (`src/engine/fixpoint.rs`, l. 486–530).

11.5.2 MemoStore : des valeurs dérivées, sans point fixe propre

Extrait 11.15 – `src/domains/stores/memo_store.rs`, l. 27–35 — le store des mémos : lecture et écriture

```
27    /// Returns `D::top()` for labels not yet computed (conservative).
28    pub fn get(&self, label: HookLabel) -> D {
29        map_get_or(&self.0, &label, D::top)
30    }
31
32    /// Store a precomputed domain value for a label.
33    pub fn set(&mut self, label: HookLabel, val: D) {
34        self.0.insert(label, val);
35    }
```

Le `MemoStore` range la valeur de chaque `useMemo` et de chaque `useCallback`. Il est l’opposé du store d’état sur les deux axes :

- **T par défaut** : un mémo pas encore calculé peut valoir n’importe quoi ;
- **mise à jour forte** : `set` écrase. Le mémo est recalculé en entier après chaque passe de rendu, à partir de l’environnement de sortie du rendu (`src/engine/fixpoint.rs`, l. 382–413). Ce n’est pas un sujet du point fixe : la convergence du store d’état entraîne celle des mémos, y compris d’une chaîne mémo $\rightarrow$ mémo, d’une itération à la suivante.

Pendant une passe, `analyze_cfg` travaille sur une copie jetée à chaque bloc (« Memo is fixed for this pass; local mutations are discarded », `src/engine/cfg_analyzer.rs`, l. 65–66).

11.5.3 SharedStateStore : l’état vu par tout le programme

Extrait 11.16 – `src/domains/stores/shared_state_store.rs`, l. 10–17 — le store inter-composants (`get` et `update` suivent, l. 24–34)

```
10    /// Cross-component state store: maps `(component, hook_label)`  $\rightarrow$  abstract value.
11    ///
12    /// Written when a `ComponentSetter` call is detected in a child component's analysis.
13    /// Read by each component's fixpoint loop to import mutations made by its children.
14    #[derive(Debug, Clone, Default)]
15    pub struct SharedStateStore {
16        entries: HashMap<ComponentId, HookLabel>, StateValue>,
17    }
```

Quand un enfant appelle un setter reçu en prop, l’écriture concerne un slot d’un *autre* composant. Elle ne peut pas aller dans le store d’état de l’enfant ; elle va dans ce store-ci, indexé par le couple (`ComponentId` du propriétaire, `label`). Il est non générique (fixé à `StateValue`), $\perp$ par défaut, mis à jour par `join`, exactement comme un `StateStore`. Il en existe *un seul par*

programme, créé par `analyze_program` (`src/engine/fixpoint.rs`, l. 714) et partagé par toutes les analyses, racines et enfants inlinés à toute profondeur, à toutes les itérations. Il n'est jamais remis à zéro : c'est un accumulateur monotone global. Chaque composant en importe sa part à chaque test de convergence :

Extrait 11.17 – `src/domains/stores/shared_state_store.rs`, l. 57–66 — la tranche d'un composant

```

57  /// Extract all entries for `comp` as a `StateStore` (for import into that component's fixpoint).
58  pub fn slice(&self, comp: ComponentId) -> StateStore<StateValue> {
59      let mut store = StateStore::new();
60      for ((c, label), val) in &self.entries {
61          if *c == comp {
62              store.update(*label, val.clone());
63          }
64      }
65      store
66  }
```

C'est tout le mécanisme de remontée de l'état : « No separate program-level fixpoint layer » (ADR-012). Pour le point fixe du parent, une écriture faite par un enfant est indiscernable d'une écriture locale.

11.5.4 Heap : le tas abstrait

Le quatrième store range les objets et les fermetures créés par le composant, indexés par leur site d'allocation (définition 6.2). Il mérite une section à lui seul (section 11.9) ; retenons pour l'instant qu'il a une entrée par `ExprId`, qu'il est mis à jour par écrasement (sauf les champs d'objets, joints), et qu'il n'entre pas dans le test de convergence.

11.5.5 Synthèse

Définition 11.2: Store

Un *store* est une table persistante d'une passe à l'autre, dont la clé désigne une entité du programme React plutôt qu'une variable : un slot d'état (`StateStore`), un mémo (`MemoStore`), un slot d'un composant quelconque (`SharedStateStore`) ou un site d'allocation (`Heap`). Un store est caractérisé par sa clé, sa valeur par défaut ($\perp$ si toute clé légitime est amorcée, $\top$ sinon), son mode de mise à jour (forte, par écrasement, ou faible, par join) et sa portée (un composant ou le programme). L'environnement `AbstractEnv`, indexé par variable et attaché à un point du programme, n'est pas un store.

Décision : ADR-002 — trois stores séparés, et ce qu'il en reste

L'ADR-002 a posé le principe : « A unified store would force all hooks into the same fixpoint ». L'état est seul sujet du point fixe, parce que seule sa mise à jour déclenche un nouveau rendu ; les mémos sont dérivés de leurs deps ; les refs sont triviales. Le principe tient, mais le détail a changé sans que l'ADR soit marquée remplacée : le domaine est devenu le produit `StateValue` (ADR-015) ; le `MemoStore` range une valeur et non un couple (deps, valeur), et il est recalculé à *chaque* itération, pas une fois après stabilisation ; le `RefStore` n'a jamais existé, `useRef` est un marqueur `StableRef` ; et deux stores sont apparus, le tas (ADR-010) et le store partagé (ADR-012). L'alternative refusée, un store unifié, aurait obligé les mémos et les refs à converger comme l'état.

	clé	défaut	mise à jour	point fixe ?	portée
AbstractEnv	Var	$\top$ (valeur); rien (setter, callback, site)	extend forte; extend_loc union	par bloc	un point de programme
StateStore	HookLabel	$\perp$	update = join	oui	un composant
MemoStore	HookLabel	$\top$	set forte	non, recalculé	un composant
SharedStateStore	(ComponentId, HookLabel)	$\perp$	update = join	via slice	le programme
Heap	ExprId	absent	insert forte; champs joints	non	un composant (copie aux enfants)

TABLE 11.5 – L’environnement et les quatre stores (définition 11.2).

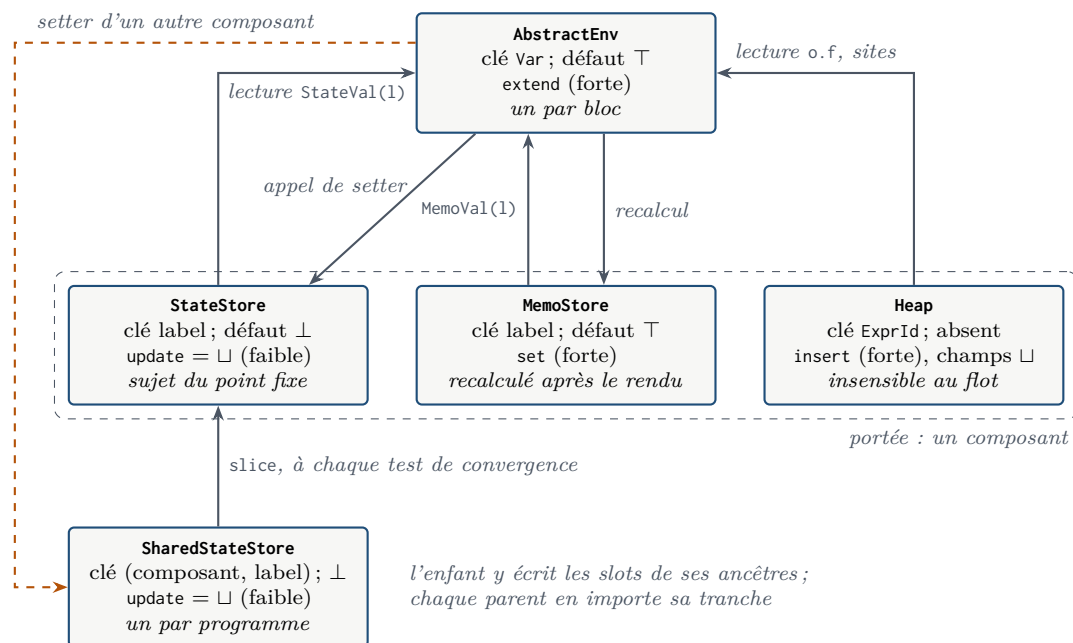


FIGURE 11.2 – La carte des stores. Les flèches vers l’environnement sont des lectures faites par l’évaluateur; la flèche « appel de setter » est l’écriture faible d’un setter local (section 11.6), la flèche tiretée celle d’un setter reçu d’un ancêtre; la flèche « recalcul » est la mise à jour des mémos depuis l’environnement de sortie du rendu. Le store partagé est le seul dont la portée dépasse le composant.

Piège

Les valeurs par défaut sont inversées d'un store à l'autre : l'environnement et les mémos rendent $\top$, l'état et le store partagé rendent $\perp$. La cohérence tient à l'amorçage. Une règle ou un contributeur qui lirait un label d'état jamais amorcé obtiendrait $\perp$, c'est-à-dire un chemin mort, et en tirerait des conclusions fausses. Autre confusion classique : `state_store.get(1)` (la vue événement, le join des écritures) n'est pas ce que lit un rendu ; le rendu lit `StateVal(1)`, converti (section 11.7).

11.6 L'instruction : liaisons, écritures de champ, setters

11.6.1 Le squelette : pré-passe, puis cœur

`Transfer::exec_stmt` délègue à `exec_stmt_with_callbacks`, qui appelle `exec_full_stmt` à la profondeur 0 (`src/domains/interp/interpreter.rs`, l. 29–36). Tout se décide là :

Extrait 11.18 – `src/domains/interp/interpreter.rs`, l. 69–99 — une instruction : pré-passe des callbacks, puis cœur

```

69 fn exec_full_stmt<T: Transfer>(<
70     transfer: &T,
71     stmt: &Stmt,
72     env: &mut AbstractEnv<T::Domain>,
73     ctx: &mut AnalysisCtx<T::Domain>,
74     depth: usize,
75 ) {
76     match stmt {
77         // An expression statement (`expr;`) contributes only its side effects.
78         // The same is true of a concise-arrow `Return` body, which the fixpoint
79         // engine feeds through here – both share the single definition in
80         // [ `exec_expr_effects` ] (callback pre-pass + setter + inter-component).
81         Stmt::ExprStmt(expr, _) => {
82             exec_expr_effects(transfer, expr, env, ctx, depth);
83         }
84         // Binding statements: callback pre-pass over the RHS parts, then the
85         // binding core.
86         Stmt::Let { rhs, .. } | Stmt::Assign { rhs, .. } => {
87             exec_callbacks_depth(transfer, rhs, env, ctx, depth);
88             exec_stmt_core(transfer, stmt, env, ctx);
89         }
90         Stmt::MemberWrite { obj, key, rhs, .. } => {
91             exec_callbacks_depth(transfer, obj, env, ctx, depth);
92             if let MemberKey::Index(idx) = key {
93                 exec_callbacks_depth(transfer, idx, env, ctx, depth);
94             }
95             exec_callbacks_depth(transfer, rhs, env, ctx, depth);
96             exec_stmt_core(transfer, stmt, env, ctx);
97         }
98     }
99 }

```

Une instruction s'exécute toujours en deux temps. D'abord la *pré-passe* `exec_callbacks_depth` parcourt les expressions de l'instruction et exécute les callbacks « dans le cycle » qu'elle y trouve (`.then`, `setTimeout`, `map...`, section 11.8). Ensuite le *cœur* fait ce que l'instruction dit : lier, écrire un champ, ou tirer les effets d'une expression. Une instruction `expr;` passe entièrement par `exec_expr_effects`, qui contient sa propre pré-passe.

11.6.2 Lier une variable : `bind_rhs`

Let et Assign partagent une seule séquence. Le commentaire du code rappelle pourquoi : « the arms had drifted, dropping the Assign aliases (FN) » (`src/domains/interp/interpreter.rs`, l. 143–149); `let s = setX` et `s = setX` doivent se comporter à l'identique.

Extrait 11.19 – `src/domains/interp/interpreter.rs`, l. 214–245 — lier une variable (1/2) : setters, callbacks, allocations

```

214 fn bind_rhs<T: Transfer>(<
215     transfer: &T,
216     var: &str,
217     rhs: &Expr,
218     env: &mut AbstractEnv<T::Domain>,
219     ctx: &mut AnalysisCtx<T::Domain>,
220 ) {
221     if let Expr::StateSetter(label) = rhs {
222         env.bind_setter(var.to_string(), *label);
223     }
224     if let Expr::CallbackVal(label) = rhs {
225         env.bind_callback(var.to_string(), *label);
226     }
227     if let Expr::FnLit {
228         id,
229         params,
230         body_cfg,
231     } = rhs
232     {
233         env.extend_loc(var.to_string(), *id);
234         ctx.heap.alloc_fn(*id, params, body_cfg, env);
235     }
236     // An object literal is a fresh container, but its members keep their own
237     // identities: `{ onClear }` where `onClear` is a `useCallback` is a new
238     // object every render holding the SAME function. Recording the per-member
239     // map on the heap is what lets `o.onClear` resolve to the member's own
240     // value instead of inheriting the container's `PerRender` (issue #88).
241     if let Expr::ObjectLit { id, fields } = rhs {
242         let members = obj_members(transfer, fields, env, ctx);
243         ctx.heap.insert(*id, HeapValue::Obj(members));
244         env.extend_loc(var.to_string(), *id);
245     }

```

Extrait 11.20 – `src/domains/interp/interpreter.rs`, l. 246–292 — lier une variable (2/2) : résumés, alias, chaînes de membres, valeur

```

246 // A library hook whose contract is per member (`useForm()`) promises
247 // `setValue` is the same function every render, and promises nothing about
248 // `formState`) reads as the same per-member map an object literal gets.
249 // The container stays T; only the named members carry a claim, so a
250 // destructured `const { setValue } = useForm()` resolves through the heap
251 // exactly as `const { onClear } = bag` does.
252 if let Expr::HookMarker(_, MarkerVal::Summary(SummaryValue::Shape { id, members })) = rhs {
253     let fields = members
254         .iter()
255         .map(|(k, v)| {
256             let val = transfer
257                 .eval_expr(&Expr::SummaryVal(v.clone()), env, ctx)
258                 .as_state_value()
259                 .unwrap_or_else(StateValue::top);
260             (k.clone(), EnvVal::Val(val))
261         })

```

```

262     .collect();
263     ctx.heap.insert(*id, HeapValue::Obj(fields));
264     env.extend_loc(var.to_string(), *id);
265 }
266 // Propagate heap locs for variable aliases (e.g. destructuring preamble:
267 // `let __obj = __p0` where __p0 carries the props heap location).
268 if let Expr::Var(src) = rhs {
269     if let Some(EnvVal::Loc { ids, .. }) = env.lookup_env_val(src) {
270         for &id in ids.iter().collect::<Vec<_>>() {
271             env.extend_loc(var.to_string(), id);
272         }
273     }
274     if let Some(label) = env.setter_label(src) {
275         env.bind_setter(var.to_string(), label);
276     }
277     if let Some(label) = env.callback_label(src) {
278         env.bind_callback(var.to_string(), label);
279     }
280 }
281 // Propagate heap locs from a member chain (e.g. `let f = props.onClick`
282 // where onClick is a FnLit stored in the parent's heap under the Obj).
283 if let Expr::FieldAccess { .. } = rhs
284     && let Some(ids) = resolve_locs(rhs, env, ctx.heap)
285 {
286     for id in ids {
287         env.extend_loc(var.to_string(), id);
288     }
289 }
290 let val = transfer.eval_expr(rhs, env, ctx);
291 env.extend(var.to_string(), val);
292 }

```

Pas à pas, pour `var = rhs` :

1. si `rhs` est un setter (`StateSetter(1)`), `var` est lié au label 1 dans la table des setters ; si c'est un `useCallback` (`CallbackVal(1)`), dans la table des callbacks ;
2. si `rhs` est un littéral de fonction, `var` reçoit son site, et la fermeture est allouée dans le tas avec ses captures (`alloc_fn`, section 11.9) ;
3. si `rhs` est un littéral d'objet, la carte de ses membres est rangée dans le tas sous son site, et `var` reçoit ce site ;
4. si `rhs` est un résumé `Shape` de bibliothèque (`const f = useForm()`), même chose avec les membres promis ;
5. si `rhs` est une autre variable (`let y = x`), `y` hérite des sites, de la liaison de setter et de la liaison de callback de `x` : c'est ainsi qu'un alias de setter reste un setter ;
6. si `rhs` est une chaîne de membres (`let f = props.onClick`), `var` reçoit les sites que `resolve_locs` trouve au bout de la chaîne ;
7. enfin, la valeur est évaluée et rangée (`env.extend`).

L'ordre a une conséquence subtile. Pour un littéral de fonction, le site est posé *avant* l'allocation, et `alloc_fn` capture l'environnement *avant* que `var` ne soit liée par valeur : une fonction récursive `const f = () => f()` capture `f` avec sa valeur antérieure, soit $\top$ à la première liaison. Autre conséquence : un tableau littéral n'est jamais rangé dans le tas, et un objet littéral n'y est rangé que s'il est *directement* lié ; un objet imbriqué (`inner: f`) n'a pas d'entrée propre (défaut D6).

11.6.3 Écrire un champ : une mise à jour faible

Extrait 11.21 – `src/domains/interp/interpreter.rs`, l. 155–200 — l'écriture de champ, mise à jour faible du tas

```

155     Stmt::MemberWrite { obj, key, rhs, .. } => {
156         if let Expr::FnLit {
157             id,
158             params,
159             body_cfg,
160         } = rhs
161         {
162             ctx.heap.alloc_fn(*id, params, body_cfg, env);
163         }
164         let val = transfer.eval_expr(rhs, env, ctx);
165         if let (Expr::Var(v), MemberKey::Field(field)) = (obj, key)
166             && let Some(EnvVal::Loc { ids, .. }) = env.lookup_env_val(v)
167         {
168             let Some(sv) = val.as_state_value() else {
169                 return;
170             };
171             let new_val = match rhs {
172                 Expr::FnLit { id, .. } => EnvVal::Loc {
173                     ids: std::collections::HashSet::from([*id]),
174                     val: sv,
175                 },
176                 _ => EnvVal::Val(sv),
177             };
178             for id in ids.iter().copied().collect::<Vec<_>>() {
179                 if let Some(HeapValue::Obj(fields)) = ctx.heap.get_mut(id) {
180                     // Values always join; the allocation sites survive only
181                     // when both sides have them (a plain value overwriting
182                     // a literal leaves nothing to chase).
183                     let joined = match (fields.get(field), &new_val) {
184                         (None, new) => new.clone(),
185                         (Some(old), new) => {
186                             let val = old.as_val().join(&new.as_val());
187                             match (old.locs(), new.locs()) {
188                                 (Some(a), Some(b)) => EnvVal::Loc {
189                                     ids: a.union(b).copied().collect(),
190                                     val,
191                                 },
192                                 _ => EnvVal::Val(val),
193                             }
194                         }
195                     };
196                     fields.insert(field.clone(), joined);
197                 }
198             }
199         }
200     }

```

`obj.f = v` ne change pas l'identité de `obj` : c'est ce qui en fait une mutation. L'écriture peut n'avoir lieu que sur un chemin, et `obj` peut désigner plusieurs sites : on *joint*, on ne remplace jamais. Les valeurs sont toujours jointes ; les sites ne survivent que si l'ancienne et la nouvelle valeur en ont (une valeur simple qui écrase une fermeture ne laisse rien à poursuivre). Trois limites : seul `Var.champ = v` est traité (pas `a.b.c = v`, pas `o[k] = v`) ; un champ *nouveau* (cas `(None, new)`) reçoit la seule nouvelle valeur, sans `undefined` ; et le tas n'étant pas sensible au flot, l'écriture faite sur une branche est vue sur l'autre. Les deux dernières se combinent dans le défaut D9.

11.6.4 Tirer les effets d'une expression

Extrait 11.22 – `src/domains/interp/interpreter.rs`, l. 114–126 — la définition unique de « exécuter une expression pour ses effets »

```
114 pub(crate) fn exec_expr_effects<T: Transfer>(
115     transfer: &T,
116     expr: &Expr,
117     env: &mut AbstractEnv<T::Domain>,
118     ctx: &mut AnalysisCtx<T::Domain>,
119     depth: usize,
120 ) {
121     exec_callbacks_depth(transfer, expr, env, ctx, depth);
122     exec_setter_call(transfer, expr, env, ctx, depth);
123     if let Expr::CompApp { .. } = expr {
124         transfer.eval_expr(expr, env, ctx);
125     }
126 }
```

Trois effets, dans cet ordre : la pré-passe des callbacks, l'appel de setter éventuel, et l'inlining d'un composant enfant si l'expression en est un (`return <Child/>`). C'est la définition unique de « exécuter une expression pour ses effets », partagée par `ExprStmt` et par le terminateur `Return` du moteur. Le cœur est l'appel de setter :

Extrait 11.23 – `src/domains/interp/interpreter.rs`, l. 341–392 — l'appel de setter : slot local, puis slot d'un propriétaire

```
341 fn exec_setter_call<T: Transfer>(
342     transfer: &T,
343     expr: &Expr,
344     env: &AbstractEnv<T::Domain>,
345     ctx: &mut AnalysisCtx<T::Domain>,
346     depth: usize,
347 ) {
348     if let Expr::Call { fn_, args } = expr
349         && let Expr::Var(name) = fn_.as_ref()
350         && let Some(label) = env.setter_label(name)
351     {
352         let arg_val = match args.first() {
353             Some(Expr::FnLit {
354                 params, body_cfg, ..
355             }) => {
356                 let mut sub_env = env.clone();
357                 if let Some(param) = params.first() {
358                     sub_env.extend(param.clone(), ctx.state.get(label));
359                 }
360                 exec_body_depth(transfer, body_cfg, &sub_env, ctx, depth + 1)
361             }
362             Some(a) => transfer.eval_expr(a, env, ctx),
363             None => T::Domain::top(),
364         };
365         ctx.state.update(label, arg_val);
366     }
367
368     // Cross-component ComponentSetter call.
369     // Handles fn_ = Var(name), FieldAccess { obj: Var, field }, or any other expr
370     // that evaluates to ComponentSetter { component, label }.
371     if let Expr::Call { fn_, args } = expr {
372         let comp_setter = transfer
373             .eval_expr(fn_, env, ctx)
374             .as_state_value()
```

```

375         .and_then(|sv| sv.as_setter().map(|(c, l)| (*c, *l)));
376     if let Some((component, label)) = comp_setter
377     && ctx.inter.is_some()
378     {
379         let arg_val = args
380             .first()
381             .map(|a| transfer.eval_expr(a, env, ctx))
382             .and_then(|v| v.as_state_value())
383             .unwrap_or(crate::domains::StateValue::top());
384         if let Some(inter) = &ctx.inter {
385             inter
386                 .shared_state
387                 .borrow_mut()
388                 .update(component, label, arg_val);
389         }
390     }
391 }
392 }

```

Premier bras : un setter lié localement. Si le callee est une variable liée à un setter (`env.setter_label(name)`), on calcule la valeur écrite et on la joint au slot (`ctx.state.update`). La valeur écrite dépend de l'argument :

- un littéral de fonction, le *functional updater* `setX(c => ...)` : son premier paramètre est lié à la valeur courante du store (`ctx.state.get(label)`), son corps est exécuté par `exec_body_depth` à la profondeur $d + 1$, et sa *valeur de retour* est la valeur écrite ;
- tout autre argument : sa valeur ;
- aucun argument : $\top$ (JavaScript écrirait `undefined` ; $\top$ est *sound*).

Second bras : un setter porteur de propriétaire. Si la valeur du callee est un setter `SetterVal::One(c, l)` et qu'une analyse inter-composants est en cours (`ctx.inter` présent), on joint la valeur du premier argument à `shared_state[(c, l)]`. C'est ainsi que `props.onChange(42)`, dans un enfant, écrit l'état du parent. Ce bras est *toujours* évalué, que le premier ait tiré ou non. Comme `StateSetter(1)` s'évalue en `SetterVal::One(composant courant, l)` (section 11.3), un setter local le tire aussi : pour un argument ordinaire, l'écriture dans le store partagé est redondante mais inoffensive ; pour un *functional updater*, elle ne l'est pas (défaut D3).

Rappel React : *Functional updater* et file d'attente

`setX(prev => f(prev))` reçoit la valeur la plus récente, après les mises à jour déjà en file, alors que `setX(x + 1)` lit la valeur capturée par la fermeture au moment du rendu. Plusieurs appels d'un même handler sont regroupés (*batching*). L'analyse lie le paramètre de l'updater au join du store, qui sur-approxime toute valeur en file ; elle ne cherche pas à modéliser l'ordre des mises à jour.

Exemple 11.2: Dérouler un *functional updater*

Le test unitaire `functional_updater_increments_state` (`src/domains/transfer/state_value.rs`, l. 1480–1521) construit à la main l'IR de `setN(c => c + 1)` sur un store où le slot 0 vaut `[5, 5]`. Le paramètre `c` est lié à `[5, 5]` ; le corps rend `[6, 6]` ; le slot devient `[5, 5]  $\sqcup$  [6, 6] = [5, 6]`. L'ancienne valeur reste : la mise à jour est faible. Sur un vrai programme, `useEffect(() => setA((x) => x + 1); )` mène le slot au widening comme le compteur de la section 11.1 ; la sonde, en mode intra-composant, donne `state[0] = number[0, inf] widened=true` pour `UpdaterNoDeps (/tmp/ch11/ex7.tsx)`. On verra en section 11.12 que le même programme,

analysé par le CLI, ne donne pas ce résultat.

Invariant 11.3: Où un setter tire

Un appel de setter écrit dans le store d'état en trois positions exactement : une instruction `expr; (ExprStmt)`, un terminateur `Return` de CFG analysé par le moteur, et le terminateur `Return` d'un corps exécuté par `exec_body_impl`. Nulle part ailleurs : un appel dans le membre droit d'une liaison, en argument d'un autre appel, ou dans le bras droit d'un `&&` (qui devient un `Assign`) n'écrit rien.

C'est une asymétrie avec le reste du moteur. La relation `slot_writers` du chapitre 14 voit un setter appelé dans *n'importe quelle* position d'expression depuis #130 ; le store d'état, lui, ne le voit qu'aux trois positions de l'invariant. Le défaut D2 en découle.

Remarque

`useReducer` passe par le même chemin : le lowering extrait `useReducer(reducer, init)` comme un état ordinaire, en sautant le réducteur, et `dispatch` est lié comme un setter. `dispatch(a)` écrit donc l'*action* `a` dans le slot, pas `reducer(s, a)` (défaut D12).

11.7 La conversion côté lecture

Le store range la vue *événement* : le join des valeurs écrites. Un rendu, lui, lit la vue *inter-rendus* : entre deux rendus, la valeur d'un état ne peut changer qu'aux appels de son setter. Le bras `StateVal(1)` de l'évaluateur passe de l'une à l'autre (`src/domains/transfer/state_value.rs`, l. 122–134) : si la composante `reference` de la valeur stockée est non- $\perp$, elle est *remplacée* par `Versioned((composant, 1))` ; les autres composantes passent intactes. Le chapitre 10 a démontré la soundness de cette conversion sous l'hypothèse « aucun setter n'est appelé pendant le rendu », dont la violation a son propre diagnostic (*setter-in-render*). Ce qui compte ici est sa *position* : c'est le seul point de conversion du projet (« The conversion happens in **one place** », ADR-017), et tout ce qui est en aval en hérite gratuitement : liaisons, deps, props, lectures de champs.

Exemple 11.3: Les deux vues sur un même slot

Dans `/tmp/ch11/ex17.tsx`, un état objet `const [obj, setObj] = useState( a: 1 )` est réécrit par un effet `setObj( a: 2 )`. La sonde imprime le store et la variable :

```
state[1] = ref(PerRender)   widened=false
env[obj] = Val(ref(Versioned(((ComponentId(4294967295), 1))))))
```

Le store dit vrai : on n'y a écrit que des littéraux frais. La variable dit vrai aussi : d'un rendu à l'autre, `obj` ne change qu'aux appels de `setObj`. Le composant est ici `ComponentId::SYNTHETIC` ($2^{32} - 1$), l'identité d'un composant analysé seul.

La lecture d'un champ prolonge la conversion : si le receveur est versionné et que le tas ne connaît pas le champ, `eval_field_access` garde les étiquettes de version au lieu de rendre un `T` complet (section 11.9). Un champ d'un état objet ne change, lui aussi, qu'aux écritures de cet état, sous l'hypothèse qu'on ne mute pas l'état en place (la règle *state-mutation* traite la violation).

11.8 Les callbacks : classe de déclenchement, pré-passe, corps

11.8.1 Le piège de l'onClick

Jusqu'ici, un effet n'écrivait l'état que par un appel direct. Le code réel passe par des callbacks :

Extrait 11.24 – Cinq façons d'appeler un setter depuis un effet (/tmp/ch11/ex9.tsx)

```
1 import { useState, useEffect } from "react";
2 import { myHelper } from "../lib";
3
4 export function Callbacks() {
5   const [user, setUser] = useState(null);
6   const [n, setN] = useState(0);
7   const [clicks, setClicks] = useState(0);
8   const [h, setH] = useState(0);
9   const [d, setD] = useState(0);
10  useEffect(() => {
11    fetch("/api").then((u) => setUser(u));
12    const tick = () => setN(42);
13    setTimeout(tick, 100);
14    window.addEventListener("click", () => setClicks(7));
15    myHelper(() => setH(9));
16    const f1 = () => setD(1);
17    const f2 = () => f1();
18    const f3 = () => f2();
19    const f4 = () => f3();
20    f4();
21  }, []);
22  return <div>{user}{n}{clicks}{h}{d}</div>;
23 }
```

Si l'interpréteur ignore les corps de fonctions, les cinq écritures sont invisibles : c'était la situation avant l'ADR-009, où seul le *functional updater* était descendu. Si au contraire il descend *uniformément* dans tous les callbacks, il exécute le handler de click comme s'il tournait à chaque effet, et un `addEventListener("click", () => setCount(c => c + 1))` parfaitement correct devient une « boucle infinie ».

Rappel React : Quand un callback s'exécute-t-il ?

`arr.map(cb)` exécute `cb` tout de suite. `.then(cb)`, `setTimeout(cb)`, `queueMicrotask(cb)`, `requestAnimationFrame(cb)` l'exécutent plus tard, mais *en conséquence* de l'effet qui les planifie : ils font partie du cycle automatique rendu, commit, effets. Un `addEventListener` ou un `onClick` ne l'exécute que sur un événement externe, hors de ce cycle.

11.8.2 La classe de déclenchement

La réponse de l'ADR-009 est de classer chaque appel par *ce qui déclenche* ses arguments fonctions :

Extrait 11.25 – `src/domains/interp/callbacks.rs`, l. 6–56 — la classe de déclenchement d'un appel

```
6 /// How a call's closure arguments should be treated by the side-effect pre-pass.
7 #[derive(Debug, Clone, Copy, PartialEq, Eq)]
8 pub enum TriggerClass {
9     /// Callee is a bound state setter handled by the core exec path (functional
10    /// updaters), so the pre-pass must NOT descend its closure.
```

```

11  Setter,
12  /// Runs as a consequence of the current render/effect: synchronous HOFs
13  /// (`map`, `forEach`, ...) and scheduled async (`then`/`.catch`/`.finally`,
14  /// `setTimeout`/`setInterval`, `queueMicrotask`, `requestAnimationFrame`).
15  /// Its closure arguments ARE descended into.
16  InCycle,
17  /// Event subscription (`addEventListener`/`removeEventListener`) triggered
18  /// externally, NOT part of the render→effect→render cycle. Not descended.
19  Subscription,
20  /// Unrecognized callee (custom helper/hook). Conservatively NOT descended
21  /// (FP-averse: avoids flagging custom subscription wrappers).
22  Unknown,
23 }
24
25 /// Classify a call's callee to decide whether its closure arguments run as a
26 /// consequence of the current render/effect (and so must be descended into for
27 /// their side effects).
28 pub fn classify_callee<D: AbstractDomain>(fn_: &Expr, env: &AbstractEnv<D>) -> TriggerClass {
29   match fn_ {
30     Expr::Var(name) => {
31       if env.setter_label(name).is_some() {
32         TriggerClass::Setter
33       } else {
34         match name.as_str() {
35           "setTimeout" | "setInterval" | "queueMicrotask" | "requestAnimationFrame" => {
36             TriggerClass::InCycle
37           }
38           _ => TriggerClass::Unknown,
39         }
40       }
41     }
42     Expr::FieldAccess { obj, field } => match field.as_str() {
43       "then" | "catch" | "finally" | "allSettled" | "any" => TriggerClass::InCycle,
44       "map" | "forEach" | "reduce" | "filter" | "find" | "flatMap" | "some" | "every"
45       | "findIndex" | "findLast" | "findLastIndex" | "reduceRight" | "sort" | "toSorted"
46       | "replace" | "replaceAll" => TriggerClass::InCycle,
47       // `Array.from(iterable, mapFn)`: the map callback runs
48       // synchronously. Receiver-restricted - a bare `.from` on an
49       // unknown object could be anything.
50       "from" if matches!(obj.as_ref(), Expr::Var(v) if v == "Array") => TriggerClass::InCycle,
51       "addEventListener" | "removeEventListener" => TriggerClass::Subscription,
52       _ => TriggerClass::Unknown,
53     },
54     _ => TriggerClass::Unknown,
55   }
56 }

```

- **Setter** : le callee est un setter lié. Son argument fonction est un *functional updater*, exécuté par `exec_setter_call`; la pré-passe ne doit pas le descendre, sinon il s'exécuterait deux fois.
- **InCycle** : fonctions d'ordre supérieur synchrones (`map`, `forEach`, `reduce`, `Array.from...`) et planifications asynchrones (`then`, `catch`, `finally`, les minuteurs). Leurs arguments fonctions sont exécutés.
- **Subscription** : `addEventListener`, `removeEventListener`. Non descendus ici; le lowering relève `addEventListener(nom, fn)` dans un effet comme un `HookEntry::Handler`, que le moteur analyse comme un *point d'entrée* séparé, exclu du signal de widening.
- **Unknown** : tout le reste, en particulier les helpers et hooks custom. Non descendus.

La reconnaissance est *nominale* : c'est le nom du callee qui décide, sans aucune information

de type. Un `x.map(cb)` dont `x` n'est pas un tableau est tout de même « dans le cycle » : on exécute un corps qui ne l'aurait peut-être pas été, ce qui peut produire un faux positif, jamais un faux négatif.

Décision : ADR-009 — Unknown n'est pas descendu

Descendre un callee inconnu serait le choix sound : rien ne prouve que `myHelper(cb)` n'appelle pas `cb` dans le cycle. L'ADR-009 le refuse : « for a linter, false positives are more costly than false negatives », car descendre produirait un FP sur chaque enveloppe de souscription custom (`useInterval`, `useEventCallback`). « We accept the FN » (#19) ; la table de politique est un bouton réglable. C'est l'un des rares cas documentés où la précision l'emporte sur l'invariant « faux négatifs interdits » (définition 3.8), et il n'est déclaré par aucune Info. Écarts avec l'ADR : elle distinguait `InCycleSync` et `InCycleDeferred`, que le code fusionne (même politique), et le code ajoute `Setter`.

11.8.3 La pré-passe

Extrait 11.26 – `src/domains/interp/interpreter.rs`, l. 477–540 — la pré-passe des callbacks

```

477 fn exec_callbacks_depth<T: Transfer>(
478     transfer: &T,
479     expr: &Expr,
480     env: &AbstractEnv<T::Domain>,
481     ctx: &mut AnalysisCtx<T::Domain>,
482     depth: usize,
483 ) {
484     if depth >= MAX_INLINE_DEPTH {
485         if let Some(inter) = ctx.inter {
486             inter
487                 .stats
488                 .borrow_mut()
489                 .callback_depth_capped
490                 .insert(inter.component);
491         }
492         return;
493     }
494     match expr {
495         Expr::Call { fn_, args } => {
496             let class = classify_callee(fn_, env);
497             // Descend the receiver: handles chains like `a.then(x).then(y)`.
498             exec_callbacks_depth(transfer, fn_, env, ctx, depth);
499             for arg in args {
500                 match arg {
501                     Expr::FnLit {
502                         params, body_cfg, ..
503                     } if class == TriggerClass::InCycle => {
504                         let mut sub_env = env.clone();
505                         for p in params {
506                             sub_env.extend(p.clone(), T::Domain::top());
507                         }
508                         let _ = exec_body_depth(transfer, body_cfg, &sub_env, ctx, depth + 1);
509                     }
510                     // B5: variable callback resolve Identifier to heap Fn and execute.
511                     Expr::Var(name) if class == TriggerClass::InCycle => {
512                         exec_var_callback(transfer, name, env, ctx, depth);
513                     }
514                     // Setter/Subscription/Unknown inline closures not descended.
515                     Expr::FnLit { .. } => {}
516                     other => {

```

```

517         exec_callbacks_depth(transfer, other, env, ctx, depth);
518     }
519 }
520 }
521 // B6: direct local call inlining Unknown callee that resolves to a heap Fn.
522 // External/imported functions have no Loc → skipped (no FP).
523 if class == TriggerClass::Unknown
524     && let Expr::Var(name) = fn_.as_ref()
525 {
526     exec_var_callback(transfer, name, env, ctx, depth);
527 }
528 }
529 Expr::CompApp { props, .. } => {
530     exec_callbacks_depth(transfer, props, env, ctx, depth);
531     // Fire inter-component inlining for CompApp inside children/fields.
532     transfer.eval_expr(expr, env, ctx);
533 }
534 // Bare FnLit outside a call: never runs by itself – not descended.
535 // Everything else: generic child descent.
536 other => {
537     other.for_each_child(&mut |c| exec_callbacks_depth(transfer, c, env, ctx, depth));
538 }
539 }
540 }

```

Pas à pas :

1. **Garde de profondeur.** À la profondeur `MAX_INLINE_DEPTH = 3`, on s'arrête. En analyse inter-composants, on l'enregistre dans `callback_depth_capped`, ce qui donnera une Info `analysis-limit` et suspendra les vérifications du composant ; en analyse intra, le plafond est silencieux.
2. **Appel.** On classe le callee, on descend le receveur (pour les chaînes `a.then(x).then(y)`), puis chaque argument :
 - un littéral de fonction, si l'appel est `InCycle` : son corps est exécuté par `exec_body_depth` à la profondeur $d + 1$, dans l'environnement *vivant* du site d'appel (capture naturelle d'une fermeture en ligne), paramètres à $\top$ (la valeur résolue d'une promesse, l'élément d'un tableau sont inconnus) ;
 - une variable, si l'appel est `InCycle` : c'est le cas *B5* de l'ADR-010 (`setTimeout(tick)`), résolu par le tas (`exec_var_callback`, section 11.9) ;
 - un littéral de fonction d'une autre classe : rien ;
 - tout autre argument : descente récursive (un appel imbriqué est traité à son tour).
3. **Appel direct d'une fonction locale.** Si le callee est `Unknown` et une variable, on tente de l'exécuter par le tas : c'est le cas *B6* (`load()`). Une fonction importée n'a pas de site et n'est pas exécutée.
4. **Composant enfant.** Pour un `CompApp` rencontré en profondeur (un enfant `JSX` dans un autre élément), pré-passe sur ses props puis évaluation, ce qui déclenche l'inlining (section 11.11).
5. **Tout le reste** : descente générique dans les sous-expressions, qui ne descend jamais dans un littéral de fonction.

Invariant 11.4: La pré-passe n'entre jamais dans un corps

La pré-passe ne descend pas *dans* un littéral de fonction ; un corps ne s'exécute que par `exec_body_depth`. Sinon, comme `exec_body_impl` rappelle `exec_full_stmt`, qui rappelle la pré-passe, chaque corps serait exécuté deux fois.

11.8.4 Exécuter un corps : `exec_body_impl`

Extrait 11.27 – `src/domains/interp/interpreter.rs`, l. 394–469 — l'exécution d'un corps de fonction imbriqué

```

394 /// Processes blocks in topological order. Back edges are ignored for env
395 /// propagation (forward-predecessor join only); statements still execute once so
396 /// loop-body setter side effects are captured. Return values from all exits are
397 /// joined; back-edge loops conservatively join return to Top.
398 fn exec_body_impl<T: Transfer>(<
399     transfer: &T,
400     cfg: &CFG,
401     entry_env: &AbstractEnv<T::Domain>,
402     ctx: &mut AnalysisCtx<T::Domain>,
403     depth: usize,
404 ) -> T::Domain {
405     let has_back_edge = cfg.edges.iter().any(|e| matches!(e.kind, EdgeKind::Back));
406
407     let order = topo_sort(cfg);
408     let mut env_at: HashMap<BlockId, AbstractEnv<T::Domain>> = HashMap::new();
409     env_at.insert(cfg.entry, entry_env.clone());
410
411     let mut return_val = T::Domain::bottom();
412
413     for bid in order {
414         let env = if bid == cfg.entry {
415             env_at
416                 .get(&bid)
417                 .cloned()
418                 .unwrap_or_else(AbstractEnv::bottom)
419         } else {
420             cfg.predecessors(bid)
421                 .iter()
422                 .filter_map(|p| env_at.get(p))
423                 .cloned()
424                 .reduce(|a, b| a.join(&b))
425                 .unwrap_or_else(AbstractEnv::bottom)
426         };
427         let mut env = env;
428
429         if let Some(block) = cfg.blocks.get(&bid) {
430             for stmt in &block.stmts {
431                 exec_full_stmt(transfer, stmt, &mut env, ctx, depth);
432             }
433             match &block.term {
434                 Terminator::Return(expr) => {
435                     // A concise arrow body `()` => EXPR` lowers EXPR to this
436                     // Return. EXPR may carry side effects `()` => setN(c => c+1)`,
437                     // `()` => arr.map(cb)`): fire them as a statement would, then
438                     // take EXPR's value as the return. `eval_expr` alone is pure
439                     // and would silently drop those effects.
440                     exec_callbacks_depth(transfer, expr, &env, ctx, depth);
441                     exec_setter_call(transfer, expr, &env, ctx, depth);
442                     let v = transfer.eval_expr(expr, &env, ctx);
443                     return_val = return_val.join(&v);
444                 }
445                 Terminator::Jump(next) => {
446                     env_at
447                         .entry(*next)
448                         .and_modify(|e| *e = e.join(&env))
449                         .or_insert(env);
450                 }
451             }
452         }
453     }
454     return_val
455 }
```

```

451         Terminator::Branch { then_, else_, .. } => {
452             for &next in &[*then_, *else_] {
453                 env_at
454                     .entry(next)
455                     .and_modify(|e| *e = e.join(&env))
456                     .or_insert_with(|| env.clone());
457             }
458         }
459         Terminator::Unreachable => {}
460     }
461 }
462 }
463
464 // Loop-carried return values can't be computed precisely in a single pass; be conservative.
465 if has_back_edge {
466     return_val = return_val.join(&T::Domain::top());
467 }
468 return_val
469 }

```

Tout corps imbriqué passe par là : callbacks, *functional updaters*, initialiseurs paresseux, arguments de hooks custom, fonctions locales inlinées. L'algorithme est volontairement simple :

1. ordonner les blocs par un tri topologique, post-ordre inversé d'un parcours en profondeur depuis l'entrée (`topo_sort`, `src/domains/interp/cfg.rs`, l. 5–11) ; les arcs retour sont ignorés, l'en-tête d'une boucle passe avant son corps, les blocs inatteignables ne sont jamais visités ;
2. pour chaque bloc, dans cet ordre, une seule fois : calculer son environnement comme le join de ceux de ses prédécesseurs, exécuter ses instructions par `exec_full_stmt` (à la même profondeur), puis traiter le terminateur ;
3. un `Return` tire les effets de son expression, comme une instruction, puis joint sa valeur à la valeur de retour ; un saut ou un branchement propage l'environnement vers les successeurs ;
4. s'il existe un arc retour, la valeur de retour est jointe à $\top$.

C'est le *second interpréteur de CFG* du projet. Le premier, `analyze_cfg` (chapitre 12), sert au rendu, aux effets et aux handlers : une worklist, avec raffinement sur les gardes et widening. Celui-ci n'a ni l'un ni l'autre : une seule passe, les deux branches d'un `if (true)` jointes, le corps d'une boucle exécuté une fois avec l'environnement d'avant la boucle. Les setters d'une boucle tirent donc bien, mais la valeur portée par la boucle n'est vue qu'à sa première itération (#21, limite déclarée dans `docs/limitations.md`).

Piège

Deux interpréteurs de force inégale (#12) : une même expression peut être évaluée avec une précision très différente selon qu'elle est écrite dans un effet ou dans le `setTimeout` de cet effet. Et le second a un défaut de propagation (D1) : l'étape 2 joint les environnements *d'entrée* des prédécesseurs, pas leurs sorties. On le démontrera en section 11.12.

11.8.5 Retour sur l'exemple

La sonde, en mode programme (le chemin du CLI), donne :

```

-- ComponentId(0) (iterations=1)
  state[0] =  $\top$    widened=false   shared= $\top$ 
  state[1] = number[0, 42]   widened=false   shared=number[42, 42]
  state[2] = number[0, 7]    widened=false   shared=number[7, 7]
  state[3] = number[0, 0]    widened=false   shared= $\perp$ 
  state[4] = number[0, 0]    widened=false   shared= $\perp$ 
stats: cache_hits=0 cache_misses=0 depth_capped={ComponentId(0)} unknown_refs={}

```

appel	mécanisme	slot
<code>fetch().then(u setUser(u)) setTimeout(tick, 100)</code>	<code>=></code> then est InCycle, le littéral est exécuté, u vaut $\top$ InCycle, argument variable : B5, le site de tick est dans le tas	<code>user = \top</code> <code>n = [0, 42]</code>
<code>addEventListener("click", ...) myHelper(() => setH(9))</code>	Subscription, non descendu; relevé en handler et analysé comme point d'entrée callee importé : Unknown, pas de site, pas de B6	<code>clicks = [0, 7]</code> <code>h = [0, 0]</code> (FN accepté)
<code>f4() → f3 → f2 → f1</code>	B6 à chaque niveau; profondeurs 1, 2, 3 : plafond	<code>d = [0, 0]</code>

TABLE 11.6 – Les cinq appels de liste 11.24.

La colonne `shared` montre que les écritures locales sont aussi versées au store partagé en mode programme (le second bras de `exec_setter_call`); c'est inoffensif ici, aucun argument n'étant un updater. Le CLI déclare le plafond et suspend les vérifications; il ne dit rien de `myHelper` :

```
$ reactant check ex9.tsx --info
Callbacks (7 hooks) ex9.tsx
info analysis-limit callback inlining reached the depth cap (3), so deeper HOF chains are not descended (FN
  ↳ possible on nested callbacks)
warn missing-cleanup [hook:5] (line 14:4) this effect calls `window.addEventListener` but returns no cleanup.
  ↳ The registration is repeated every time the effect re-runs (and on every mount, twice under StrictMode) and
  ↳ nothing ever undoes it; return a function that tears it down
suspended analysis-limit 10 passing check(s) withheld: the analysis was truncated in this component, so they are
  ↳ not guaranteed
```

11.9 Le tas par site d'allocation

11.9.1 Le besoin

L'environnement dit ce que *vaut* une variable. Deux questions lui échappent : « quel corps exécuter quand on appelle `tick` ? » (B5, B6) et « que vaut le membre `onClear` de l'objet `bag` ? ».

Extrait 11.28 – Membres, sites et fermetures (`/tmp/ch11/ex10.tsx`, extrait du corps de `Heapy`)

```
1  const [q, setQ] = useState("");
2  const onClear = useCallback(() => setQ(""), []);
3  const bag = { onClear, label: "x", n: 1 };
4  const fromBag = bag.onClear;
5  const { label } = bag;
6  const fresh = new Map();
7  const items = [1, 2].map((x) => x * 2);
8  const nested = { inner: { f: () => setQ("n") } };
9  const g = nested.inner.f;
10 useEffect(() => {
11   g();
12 }, [fromBag, items]);
```

`bag` est un objet neuf à chaque rendu, mais son membre `onClear` est *la même* fonction à chaque rendu (un `useCallback` aux deps vides). Si `bag.onClear` héritait de la stabilité du conteneur, la dep `fromBag` serait signalée à tort; c'était le faux positif de #88. Il faut donc une mémoire *par membre*.

11.9.2 Sites, valeurs du tas, vue EnvVal

Extrait 11.29 – src/domains/stores/heap.rs, l. 17–34 — le tas abstrait et ses valeurs

```

17 /// Value stored at a heap location (indexed by `ExprId` allocation site).
18 #[derive(Debug, Clone)]
19 pub enum HeapValue {
20     /// A function literal: its params, body CFG, and captured environment at creation site.
21     Fn {
22         params: Vec<Var>,
23         body_cfg: Arc<CFG>,
24         /// Free variables captured from the enclosing scope when this function was created.
25         captured: HashMap<Symbol, StateValue>,
26     },
27     /// An abstract object: fields may be plain values or heap locations (for FnLit props).
28     Obj(HashMap<Symbol, EnvVal<StateValue>>),
29 }
30
31 /// Abstract heap: maps allocation-site `ExprId`'s to `HeapValue`'s.
32 /// Populated by `eval_expr` for `FnLit`/`ObjectLit`/`ArrayLit` nodes.
33 #[derive(Debug, Clone, Default)]
34 pub struct Heap(HashMap<ExprId, HeapValue>);

```

La clé est le *site d'allocation* (définition 6.2) : l'ExprId que le lowering attribue à chaque ObjectLit, ArrayLit, FnLit et New, stable d'une itération à l'autre et unique dans un composant. Les sites synthétiques (l'objet de props d'un enfant) viennent de ExprId::fresh(), un compteur qui démarre à 10⁹. Une valeur du tas est soit une fermeture (Fn : paramètres, CFG du corps partagé par Arc, valeurs capturées de ses variables libres), soit un objet (Obj : une carte membre $\mapsto$ EnvVal).

EnvVal (src/domains/stores/abstract_env.rs, l. 10–27) est la vue d'une liaison : Val(v), une valeur seule, ou Loc ids, val, un ensemble de sites *et* une valeur. Le commentaire du code insiste : un site et une valeur ne sont pas des alternatives. Rendre T pour la valeur d'un Loc, comme autrefois, faisait lire une prop fonction comme « inconnue » dans l'enfant, alors que le parent l'avait prouvée PerRender.

11.9.3 Allouer une fermeture : un point unique

Extrait 11.30 – src/domains/stores/heap.rs, l. 45–70 — l'allocation d'une fermeture et de ses captures

```

45 /// Allocate a `HeapValue::Fn` at site `id`, snapshotting the closure's
46 /// captured `StateValue`s (the body's free vars resolved in `env`).
47 ///
48 /// The single place a FnLit becomes a heap closure – shared by every
49 /// statement arm (`Let`/`Assign`/`MemberWrite`) and by prop evaluation.
50 pub fn alloc_fn<D: AbstractDomain>(
51     &mut self,
52     id: ExprId,
53     params: &[Var],
54     body_cfg: &Arc<CFG>,
55     env: &AbstractEnv<D>,
56 ) {
57     let free = compute_free_vars(body_cfg);
58     let captured = free
59         .iter()
60         .filter_map(|v| env.lookup(v).as_state_value().map(|sv| (v.clone(), sv)))
61         .collect();
62     self.insert(

```

```

63         id,
64         HeapValue::Fn {
65             params: params.to_vec(),
66             body_cfg: Arc::clone(body_cfg),
67             captured,
68         },
69     );
70 }

```

`alloc_fn` est appelé par les trois bras qui peuvent lier un littéral de fonction (`Let/Assign`, membre d'objet, `MemberWrite`) et par l'évaluation des props d'un enfant. Il calcule les variables libres du corps et capture leur *valeur* dans l'environnement courant ; une variable libre non liée est capturée comme `⊤`. Ni les sites ni les liaisons de setter des variables capturées ne sont capturés ; un setter reste retrouvable parce que sa *valeur* est un `SetterVal::One(composant, label)`, que le second bras de `exec_setter_call` sait appeler.

11.9.4 Suivre une chaîne de membres : un walker unique

Extrait 11.31 – `src/domains/stores/heap.rs`, l. 134–165 — les sites que désigne une expression

```

134 /// The allocation sites `expr` may denote, chasing member chains through the
135 /// heap: `o.a.f` resolves as far as the objects on the way record their
136 /// members. `None` for anything the heap cannot place (a call, a primitive, an
137 /// unbound variable) – the caller then falls back to the expression's value.
138 ///
139 /// One walker so every consumer – the field evaluator, the `let` binder, the
140 /// object-literal member map – agrees on what a chain points to; the old
141 /// `Expr::Var` special cases each stopped at the first segment.
142 pub fn resolve_locs<D: AbstractDomain>(
143     expr: &Expr,
144     env: &AbstractEnv<D>,
145     heap: &Heap,
146 ) -> Option<HashSet<ExprId>> {
147     match expr {
148         Expr::Var(v) => env.lookup_env_val(v).and_then(|ev| ev.locs().cloned()),
149         Expr::TSAnnotated(inner) => resolve_locs(inner, env, heap),
150         Expr::FieldAccess { obj, field } => {
151             let ids: HashSet<ExprId> = resolve_locs(obj, env, heap)?
152                 .iter()
153                 .filter_map(|id| match heap.get(*id) {
154                     Some(HeapValue::Obj(fields)) => fields.get(field),
155                     _ => None,
156                 })
157                 .filter_map(|ev| ev.locs())
158                 .flatten()
159                 .copied()
160                 .collect();
161             (!ids.is_empty()).then_some(ids)
162         }
163         _ => None,
164     }
165 }

```

`resolve_locs` ne connaît que trois formes : une variable (ses sites), une annotation (transparente), et un accès de champ (les sites du membre, sur chaque objet que le receveur peut désigner). Un index, un appel, un littéral imbriqué rendent `None`. C'est le seul walker du projet : l'évaluateur de champs, la liaison `let` et la carte des membres d'un objet l'appellent tous, et le commentaire rappelle que les anciens cas particuliers `Expr::Var` « each stopped at the first

segment ».

11.9.5 Lire un membre

Extrait 11.32 – `src/domains/transfer/state_value.rs`, l. 598–628 — la lecture d'un membre par le tas

```

598 fn eval_field_access(
599     obj: &Expr,
600     field: &Symbol,
601     env: &AbstractEnv<StateValue>,
602     ctx: &mut AnalysisCtx<StateValue>,
603 ) -> StateValue {
604     if let Some(ids) = resolve_locs(obj, env, ctx.heap) {
605         let vals: Vec<StateValue> = ids
606             .iter()
607             .filter_map(|id| ctx.heap.get(*id))
608             .filter_map(|h| match h {
609                 HeapValue::Obj(fields) => fields.get(field).map(EnvVal::as_val),
610                 _ => None,
611             })
612             .collect();
613         if !vals.is_empty() {
614             return vals.into_iter().reduce(|a, b| a.join(&b)).unwrap();
615         }
616     }
617     // A field of a versioned object can change only at the object's own
618     // setter events: keep the version labels instead of degrading to T
619     // (the field's *kind* stays unknown – every other slot is T). Same
620     // no-mutation-during-render assumption as the `StateVal` read-side
621     // conversion above; in-place writes have their own diagnostics
622     // (`state-mutation`). ADR-017 §Limitations, member-deps.
623     let obj_val = eval_state_value(obj, env, ctx);
624     StateValue {
625         reference: obj_val.versioned_reference().unwrap_or(Stability::Unknown),
626         ..StateValue::top()
627     }
628 }

```

On résout les sites du receveur ; sur chaque site qui est un objet *possédant* le champ, on prend la valeur du membre ; on joint. Si aucun site ne répond, on retombe sur $\top$, en gardant les étiquettes de version du receveur (section 11.7). Un index entier constant se lit comme le membre de même nom (`xs[0]` comme le membre `"0"`, `src/domains/transfer/state_value.rs`, l. 641–651) : c'est ce qui permet à un hook résumé qui rend un tuple (`const [value, setValue] = useAtom(a)`) d'exposer ses positions.

Piège

L'étape « on joint les objets qui ont le champ » ignore ceux qui ne l'ont pas, au lieu d'y joindre `undefined` ou $\top$. Si le receveur peut désigner un objet sans ce membre (une autre branche, un objet issu d'un spread, un receveur nullable lu par `?.`), la réponse est sous-approximée. La doc de `obj_members` affirme le contraire (« an absent member falls back to the container's own value — sound, just imprecise ») ; ce n'est vrai que si *aucun* site n'a le membre. C'est le défaut D5.

La sonde confirme la figure 11.3 :

```

-- Heapy (intra, iterations=0)
state[0] = string{""} widened=false

```

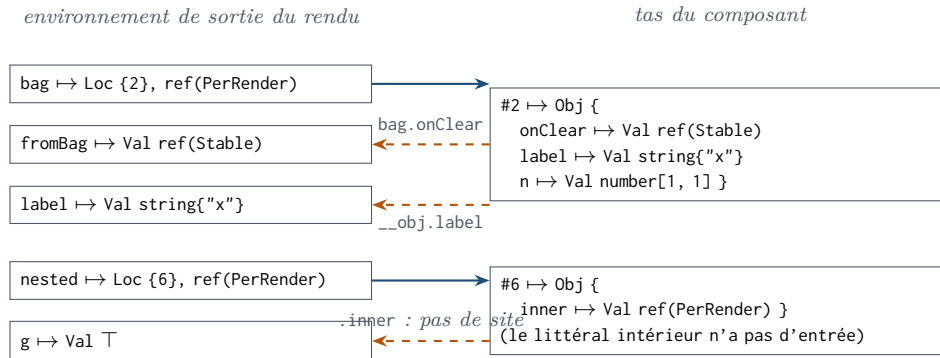


FIGURE 11.3 – Le tas de Heapy, d’après la sonde. Les flèches pleines vont d’une variable à ses sites ; les tiretées sont les lectures de membres qui ont produit une liaison. `bag.onClear` lit la valeur *propre* du membre, `Stable`, et non celle du conteneur ; `nested.inner.f` s’arrête faute de site pour le littéral intérieur, et `g` vaut `T`.

```
env[bag] = Loc { ids: {ExprId(2)}, val: ref(PerRender) }
env[fromBag] = Val(ref(Stable))
env[label] = Val(string{"x"})
env[fresh] = Val(ref(PerRender))
env[items] = Val(ref(PerRender))
env[nested] = Loc { ids: {ExprId(6)}, val: ref(PerRender) }
env[g] = Val(T)
```

et le CLI en tire deux avertissements justes, et un silence faux :

```
$ reactant check ex10.tsx
Heapy (4 hooks) ex10.tsx
warn  always-unstable-deps [hook:2] (line 13:2) this effect depends on `items`, a new reference every render, so
    ↳ `Object.is` always differs and the effect re-runs on every render regardless of the other deps
    (1 trace step(s), rerun with --trace)
warn  missing-deps [hook:2] var:g (line 13:2) `g` is used in this effect but not in its deps array, and its
    ↳ value may change between renders
    (1 trace step(s), rerun with --trace)
```

`fromBag` n’est pas signalé (sa stabilité est prouvée), `items` l’est (sa fraîcheur est certaine), `g` l’est parce qu’il vaut `T`. Mais `g()` n’a pas été exécuté dans l’effet : `setQ("n")` est manqué, `q` reste `""`, et le mémo `[q]` du composant est jugé stable alors qu’il changera (défaut D6).

11.9.6 Exécuter une fonction par son nom : B5 et B6

Extrait 11.33 – `src/domains/interp/interpreter.rs`, l. 544–583 — exécuter un callback désigné par une variable

```
544 fn exec_var_callback<T: Transfer>(<
545     transfer: &T,
546     name: &str,
547     env: &AbstractEnv<T::Domain>,
548     ctx: &mut AnalysisCtx<T::Domain>,
549     depth: usize,
550 ) {
551     if let Some(EnvVal::Loc { ids, .. }) = env.lookup_env_val(name) {
552         let ids: Vec<_> = ids.iter().copied().collect();
553         for id in ids {
554             if let Some(HeapValue::Fn {
555                 params,
556                 body_cfg,
557                 captured,
```

```

558     }) = ctx.heap.get(id)
559     {
560         let params = params.clone();
561         let body_cfg = Arc::clone(body_cfg);
562         let captured = captured.clone();
563         let mut sub_env = env.clone();
564         for (var, val) in captured {
565             sub_env.extend(var, T::Domain::from_state_value(val));
566         }
567         for p in &params {
568             sub_env.extend(p.clone(), T::Domain::top());
569         }
570         let _ = exec_body_depth(transfer, &body_cfg, &sub_env, ctx, depth + 1);
571     }
572 }
573 return;
574 }
575 // useCallback binding: the body lives in the hook entry, not the heap
576 // (the rewrite to `CallbackVal` moved it out of the expression tree).
577 // Params stay unbound → env-miss T, captures resolve in the live env.
578 if let Some(label) = env.callback_label(name)
579     && let Some(body_cfg) = ctx.query.callback_body(label)
580 {
581     let _ = exec_body_depth(transfer, &body_cfg, env, ctx, depth + 1);
582 }
583 }

```

C'est ici que le tas rejoint la pré-passe. Pour une variable liée à des sites, on exécute *tous* les corps possibles (« All bodies are executed and their effects joined — correct by over-approximation », ADR-010), dans l'environnement vivant, complété par les captures (qui l'emportent) et les paramètres à T . Pour une variable liée à un `useCallback`, le corps est demandé à `QueryContext::callback_body` (il vit dans la table des hooks, pas dans le tas). Pour une variable sans site ni callback (un import, un paramètre), on ne fait rien.

Trois programmes isolent la frontière (`/tmp/ch11/ex11.tsx`) : `Flat` lie `g = o.f` avec `o = f: () => setQ("n")` ; `Nested` lie `g = o.inner.f` avec un objet imbriqué ; `MethodCall` appelle directement `o.f()`. Seul le premier voit l'écriture :

```

-- Flat (intra, iterations=1)
state[0] = string{"", "n"} widened=false
-- Nested (intra, iterations=0)
state[0] = string{""} widened=false
-- MethodCall (intra, iterations=0)
state[0] = string{""} widened=false

```

Dans `Flat`, `resolve_locs(o.f)` trouve le site du littéral, `B6` exécute le corps. Dans `Nested`, le littéral intérieur n'a pas de site. Dans `MethodCall`, le callee est un `FieldAccess` : `B6` ne traite que les callees `Var`.

Décision : ADR-010 — un tas par site d'allocation

Pour relier un nom à un corps, l'ADR-010 choisit l'abstraction classique par site d'allocation : un `ExprId` par nœud qui alloue, une table `ExprId → valeur du tas`, un environnement à deux tables, et *une seule tas* partagé par toutes les passes et itérations d'un composant (sans quoi le tick alloué dans le rendu serait inconnu de l'effet). Limites déclarées : un callee `Unknown` sans site est un FN ; une profondeur au-delà de trois est un FN signalé ; les membres avant un `spread` et les accesseurs manquent à la carte. Écarts avec le code : les fermetures ont gagné leurs captures (ADR-012), les objets une carte d'`EnvVal` (#88), et la variante `HeapValue::Arr`, « reserved — future array domain », n'a jamais été écrite.

Piège

Le tas n'est pas un treillis qu'on itère. `Heap::join`, `widen` et `leq` existent (`src/domains/stores/heap.rs`, l. 80–131) mais ne sont appelés nulle part ; le moteur manipule un seul tas mutable par composant, passé à tous les blocs, toutes les passes, toutes les itérations. Trois conséquences. (1) Il est *insensible au flot* : une écriture de champ faite sur une branche est lue sur l'autre. (2) `insert` écrase : la ré-exécution de `const o = ...` remplace l'objet et efface les écritures de champ accumulées sur ce site ; une fermeture ré-allouée remplace ses captures. (3) Les sites d'une variable ne sont jamais retirés : `let f = () => a(); f = () => b();` fait exécuter les deux corps à `f()`, ce qui sur-approxime. Enfin, la doc de `Heap` (« Populated by `eval_expr` for `FnLit/ObjLit/ArrayLit` nodes ») est inexacte : c'est l'interpréteur qui peuple le tas, et aucun tableau n'y entre jamais.

11.10 La valeur d'un mémo

`useMemo(f, deps)` et `useCallback(f, deps)` rendent la même référence tant qu'aucune `dep` ne change au sens d'`Object.is`. Leur valeur abstraite n'est donc pas celle du corps `f`, qui n'est jamais évalué : c'est une *référence* dont la stabilité est le join des stabilités des `deps`. Le chapitre 10 a cité et commenté `recompute_memo` (`src/domains/transfer/state_value.rs`, l. 56–101) ; on en retient la table de décision, vérifiée sur `/tmp/ch11/ex17.tsx` (tableau 11.7).

mémo	deps	valeur (sonde)	raison
<code>useMemo(() => (k: 1), [])</code>	<code>[]</code> exact	<code>ref(Stable)</code>	épinglé au montage
<code>useMemo(() => (k: 1))</code>	absentes	<code>ref(Unknown)</code>	rien n'est borné
<code>useMemo(() => n * 2, [n])</code>	$n \in [0, 1]$	<code>ref(PerRender)</code>	projection <code>to_stability</code>
<code>useMemo(() => obj.a, [obj])</code>	état objet	<code>ref(Versioned(1))</code>	lecture convertie
<code>useMemo(() => p + 1, [p])</code>	prop	<code>ref(Unknown)</code>	<code>p</code> vaut $\top$
<code>useMemo(() => 1, [])</code>	littéral	<code>ref(PerRender)</code>	allocation
<code>useMemo(() => byObj, [byObj])</code>	un mémo	<code>ref(Versioned(1))</code>	étiquettes propagées
<code>useCallback(() => setN(1), [])</code>	<code>[]</code>	<code>ref(Stable)</code>	même calcul

TABLE 11.7 – La valeur de huit mémos de `/tmp/ch11/ex17.tsx`. Une `dep` qui est *syntactiquement* `StateVal(1)` donnerait `Versioned` quelle que soit sa sorte ; dans le code réel, `[n]` est un `Var("n")`, et un intervalle non ponctuel se projette en `PerRender` : c'est le faux positif de projection relevé au chapitre 10.

Deux décisions encadrent ce calcul. Des `deps` illisibles (absentes ou opaques) donnent `ref(Unknown)` et jamais `Stable` : le trait `Transfer` l'exige (« it must not share an answer with a written [], which pins the memo forever »). Et depuis l'ADR-020 (« `recompute_memo` gets the real stores »), les `deps` sont évaluées contre les vrais stores du point fixe : un mémo qui dépend d'un mémo lit sa valeur courante, et non un $\perp$ fabriqué.

11.11 L'inter-composants, vu du transfert

Le chapitre 16 décrira l'analyse inter-composants du point de vue du moteur (racines, registre, cache, graphe d'appels). Vue du transfert, elle se réduit à deux flux : les props descendent vers l'enfant par l'évaluation d'un `CompApp`, les écritures remontent vers le parent par le store partagé.

11.11.1 Un parent, deux enfants

Extrait 11.34 – Un enfant connu, un enfant inconnu (/tmp/ch11/ex13.tsx)

```

1 import { useState, useEffect } from "react";
2 import { Sheet } from "../ui";
3
4 function Child({ onChange }: { onChange: (n: number) => void }) {
5   useEffect(() => {
6     onChange(42);
7   }, []);
8   return <span />;
9 }
10
11 export function Parent() {
12   const [count, setCount] = useState(0);
13   const [open, setOpen] = useState(false);
14   return (
15     <div>
16       <Child onChange={setCount} />
17       <Sheet onOpenChange={setOpen} />
18       {count}{String(open)}
19     </div>
20   );
21 }

```

```

-- ComponentId(0) (iterations=0)
  env[onChange] = Val(setter(#1#0))
-- ComponentId(1) (iterations=1)
  state[0] = number[0, 42] widened=false shared=number[42, 42]
  state[1] =  $\top$  widened=false shared= $\perp$ 
stats: cache_hits=1 cache_misses=1 depth_capped={} unknown_refs={(ComponentId(1), "Sheet")}

```

ComponentId(0) est Child, ComponentId(1) est Parent. La chaîne causale :

1. la passe de rendu de Parent atteint le Return; `exec_expr_effects` lance la pré-passe, qui rencontre `<Child .../>` et l'évalue : `eval_comp_app`;
2. la prop `onChange` vaut `setter(##1##0)`, soit `SetterVal::One(Parent, 0)`;
3. Child est analysé jusqu'à son propre point fixe; son effet appelle `onChange(42)`, dont la valeur est un setter d'un autre composant : le second bras de `exec_setter_call` écrit `shared_state[(Parent, 0)]  $\sqcup$ = [42, 42]`;
4. au test de convergence, Parent importe sa tranche : le nouvel état n'est pas $\sqsubseteq$ l'ancien, on itère; à la seconde passe, l'enfant est trouvé dans le cache (`cache_hits=1`); `count` converge sur `[0, 42]`;
5. Sheet n'est pas dans le registre : son setter est *havoqué*, `open` reçoit $\top$, et une Info le déclare.

```

$ reactant check ex13.tsx --info
Parent (2 hooks) ex13.tsx
info analysis-limit component `Sheet` was not found in the analysis registry. Pass its file on the command line to
  ↳ analyse it (FN possible)
suspended analysis-limit 4 passing check(s) withheld: the analysis was truncated in this component, so they are not
  ↳ guaranteed

```

11.11.2 eval_comp_app, pas à pas

1. **Sans contexte inter** (analyse d'un composant seul) : tout enfant est inanalysable; on havoque les setters de ses props et on rend `ref(Stable)` (`src/domains/transfer/state_value.rs`,

1. 477–483).
2. **Résolution** du nom JSX par le registre, en un `ComponentId` (ADR-040). Un enfant inconnu ou ambigu est compté dans les statistiques (qui deviendront des `Info analysis-limit`), ses setters sont havoqués, et on rend `ref(Stable)` (`src/domains/transfer/state_value.rs`, l. 489–518).
3. **Récursion** : si l'enfant est déjà sur la pile d'appel, on compte une coupure et on rend `ref(Stable)`, *sans* havoc.
4. **Props** : `eval_props_map` rend une carte `prop ↦ EnvVal` ; une prop littéral de fonction est allouée dans le tas du parent pour que l'enfant puisse l'exécuter. Le code est en `src/domains/transfer/state_value.rs`, l. 653–689.
5. **Cache** : les props aplaties en `StateValue` servent de clé ; un succès enregistre l'arête du graphe d'appels et rend `ref(Stable)`, sans ré-analyser l'enfant.
6. **Contexte de l'enfant**, puis **analyse** :

Extrait 11.35 – `src/domains/transfer/state_value.rs`, l. 559–580 — le tas et l'environnement initiaux d'un enfant

```

559 // Build child initial env + heap:
560 // - copy heap entries for any Loc-valued props (FnLit bodies) into child's heap
561 // - insert the Obj (with full EnvVals) so the child can resolve FieldAccess → Loc
562 let mut child_env = AbstractEnv::bottom();
563 let props_id = ExprId::fresh();
564 let mut initial_heap = crate::domains::stores::Heap::new();
565 for ev in abstract_props_full.values() {
566     if let EnvVal::Loc { ids, .. } = ev {
567         for &id in ids {
568             if let Some(hv) = ctx.heap.get(id) {
569                 initial_heap.insert(id, hv.clone());
570             }
571         }
572     }
573 }
574 initial_heap.insert(props_id, HeapValue::Obj(abstract_props_full.clone()));
575 child_env.extend_loc(child_ir.param.clone(), props_id);
576
577 // Create child inter context and analyze
578 let child_inter = inter.child(child);
579 let analyze_child = inter.analyze_child;
580 let child_result = analyze_child(&child_ir, child, child_env, initial_heap, &child_inter);

```

L'enfant part d'un environnement vide où son paramètre de props est lié à un site synthétique frais ; son tas initial contient un objet sous ce site (la carte des props) et une copie des valeurs de tas des props qui ont des sites (fermetures, objets). `analyze_child` est un pointeur vers `analyze_component_inter` : une analyse complète, avec son propre point fixe, sous un `InterCtx` fils dont la pile d'appel contient le parent. Le résultat est rangé dans la table `results` et dans le cache, l'arête dans le graphe d'appels, et `eval_comp_app` rend, dans tous les cas, `ref(Stable)`.

Exemple 11.4: Une boucle croisée

Dans `/tmp/ch11/ex15.tsx`, Solo passe `onTick() => setB(b + 1)` à un `Ticker` dont l'effet sans `deps` appelle `onTick()`. La fermeture est allouée par `eval_props_map` (captures : `setB`, `b`) et copiée dans le tas de l'enfant ; dans `Ticker`, `onTick` a un site, et `onTick()` est un `B6` : le corps s'exécute avec ses captures, et `setB(b + 1)` tire le second bras de `exec_setter_call`, puisque le setter a pour propriétaire Solo. Le parent importe l'écriture, itère, et le CLI conclut :

```

Ticker (1 hooks) ex15.tsx
warn cross-component-infinite-loop [hook:0] (line 4:2) this effect calls `onTick`, a state setter of parent

```

↪ `Solo`. Parent re-renders → child re-renders → effect fires again: infinite loop

11.11.3 Le havoc des setters qui s'échappent

Un setter remis à un enfant *inanalysable* peut être appelé par lui avec n'importe quel argument, n'importe quand. Ne rien faire sous-approximerait l'état et fabriquerait des conclusions « l'état est stable ». D'où :

Extrait 11.36 – `src/domains/transfer/state_value.rs`, l. 265–297 — joindre $\top$ dans les slots dont le setter s'échappe

```

265 fn havoc_setter_props(
266     props_expr: &Expr,
267     env: &AbstractEnv<StateValue>,
268     ctx: &mut AnalysisCtx<StateValue>,
269 ) {
270     let Expr::ObjectLit { fields, .. } = props_expr else {
271         return;
272     };
273     let own = ctx.component;
274     let mut setters: Vec<(ComponentId, crate::ir::types::HookLabel)> = Vec::new();
275     for (_, v) in fields {
276         // Bare setter prop (<X onOpenChange={setOpen}/>): the value
277         // carries its owner (<StateSetter> always evals to a
278         // <component_setter>, intra included).
279         let val = eval_state_value(v, env, ctx);
280         if let Some((c, l)) = val.as_setter() {
281             setters.push((c, *l));
282         }
283         // Everything a function value can smuggle a setter through: spread
284         // objects, closures wrapping a setter call, heap-allocated FnLiits.
285         collect_escaping_setters(v, env, ctx.heap, own, &mut setters, &mut HashSet::new());
286     }
287     for (comp, label) in setters {
288         if comp == own {
289             ctx.state.update(label, StateValue::top());
290         } else if let Some(inter) = &ctx.inter {
291             inter
292                 .shared_state
293                 .borrow_mut()
294                 .update(comp, label, StateValue::top());
295         }
296     }
297 }

```

Un setter nu passé en prop est trouvé par sa valeur, qui porte son propriétaire. Les setters cachés (dans un spread, dans une fermeture qui appelle un setter, dans une fermeture du tas) sont cherchés par `collect_escaping_setters` (`src/domains/transfer/state_value.rs`, l. 299–367), qui parcourt les corps de fonctions atteignables depuis la valeur de chaque prop. Sa terminaison est garantie par *identité*, pas par un budget :

Extrait 11.37 – `src/domains/transfer/state_value.rs`, l. 385–392 — un corps n'est parcouru qu'une fois par environnement capturé

```

385     let key = (
386         cfg as *const _ as usize,
387         captured.map_or(0, |c| c as *const _ as usize),
388     );
389     if !walked.insert(key) {

```

```

390     return;
391   }
392   cfg.for_each_expr(&mut |e| setter_calls_in_expr(e, env, captured, heap, own, out, walked));

```

La clé est la paire (adresse du CFG, adresse des captures) : le même corps atteint sous d'autres captures résout ses callees autrement. Le commentaire du code garde la trace de l'ancienne version, un `depth > 4` qui était « a budget doing a cycle guard's job » : un setter caché derrière une cinquième fermeture était silencieusement manqué. Le parcours a néanmoins une limite, qui est le défaut D13.

Décision : ADR-012 — inlining descendant et store partagé

L'ADR-012 choisit l'*inlining descendant* : le parent analyse l'enfant au point d'application, avec les props abstraites, plutôt que de calculer des résumés paramétriques ascendants, jugés « without justified gain at the current stage ». La sensibilité au contexte passe par l'égalité abstraite des props (le cache). Le flux est bidirectionnel : props vers le bas, setters vers le haut par un `SharedStateStore` passé à toutes les analyses, sans couche de point fixe au niveau du programme. Écarts avec le code : `ComponentSetter` est devenu la composante setter du produit (ADR-015), les clés sont des `ComponentId` internés (ADR-040), la récursion rend `ref(Stable)` sans havoc. Limites acceptées : imports hors registre ($\top$ et Info), récursion (Info), composants dynamiques `const C = cond ? A : B` non analysés (#63, fermée *wontfix*).

Piège

Trois raccourcis de `eval_comp_app` ont des conséquences. (1) La valeur d'un élément est toujours `ref(Stable)`, y compris pour un enfant inconnu : elle n'encode pas l'échec. (2) La récursion ne havoque rien : un composant récursif qui passe à son instance intérieure un *autre* setter que celui qu'il a reçu sous-approxime ; l'Info de récursion le déclare (« recursive component reference Tree is not followed, so its props are treated as unknown »), mais son texte ne dit pas ce que fait le code, qui n'évalue ni ne havoque aucune prop. (3) La clé du cache et la table des résultats oublient une partie du contexte (défauts D4 et D7).

11.12 La soundness en pratique : garanties et défauts

11.12.1 Les garanties par construction

La soundness (définition 3.8) exige que chaque fonction de transfert sur-approxime la sémantique concrète. Le code la garantit par une poignée de mécanismes, que l'on peut maintenant récapituler (tableau 11.8).

À côté, des faux négatifs sont *acceptés et déclarés* : un callee `Unknown` non descendu (ADR-009, #19), le plafond de profondeur (Info), un enfant inconnu (Info), la récursion (Info), la valeur portée par une boucle dans un callback (#21). Le premier est le seul qu'aucune Info ne signale.

Reste ce que l'invariant du projet interdit : les faux négatifs *non déclarés*. Les quatorze défauts qui suivent ont été reproduits au commit e67b10a en rédigeant les notes de ce chapitre ; aucun n'était décrit dans le tracker à cette date (l'issue fermée #130 traitait l'équivalent de D2 pour la relation `slot_writers`, pas pour le store). Ce sont des constats, pas des diagnostics confirmés par le mainteneur. Chacun est une vraie boucle infinie ou une vraie dep manquante en React, que l'analyseur déclare sûre. Pour chacun, l'exercice est d'identifier l'hypothèse violée : c'est toujours l'une des garanties ci-dessus qui n'est pas tenue jusqu'au bout. Les quatorze cas ont été rejoués pour cette relecture, au commit e67b10a, sur des copies fraîches dans `/tmp/ch11/` : chacun reproduit exactement la valeur ou le silence du CLI annoncé plus bas. Restent constats,

mécanisme	garantie	où
variable inconnue $\Rightarrow \top$; clé d'un seul côté du join $\Rightarrow \top$	valeurs	AbstractEnv::lookup, join_stabs
setter $\Rightarrow$ join dans le slot	écritures may	StateStore::update
setter vers un enfant inconnu $\Rightarrow \top$ dans le slot	pas de « stable » fabriqué	havoc_setter_props
setters échappés cherchés par identité, pas par budget	pas de setter manqué par profondeur	collect_escaping_setters
appel $\Rightarrow \top$, sauf allocateurs certains	valeurs	returns_fresh_reference
/, et % ou ** à NaN possible $\Rightarrow \top$	pas de NaN dans un intervalle	eval_binop
opérande $\perp \Rightarrow \perp$	un chemin mort reste mort	eval_bitwise, as_arith
paramètres de callbacks $\Rightarrow \top$; retour d'un corps bouclant $\Rightarrow \top$	valeurs	exec_callbacks_depth, exec_body_impl
écriture de champ $\Rightarrow$ join	mutations may	MemberWrite
deps illisibles $\Rightarrow$ ref(Unknown), jamais Stable	mémos	recompute_memo

TABLE 11.8 – Où la soundness est garantie par construction.

non diagnostics : la question de savoir si D8 et D12 relèvent d'une décision de conception tacite plutôt que d'un défaut reste au mainteneur.

11.12.2 D1 : un corps qui voit l'état d'un bloc en retard

Extrait 11.38 – Trois callbacks à deux blocs (/tmp/ch11/ex3.tsx, extrait de l'effet)

```

1  setTimeout(() => {
2    const v = 5;
3    if (flag) {
4      setN(v);
5    }
6  }, 10);
7  setTimeout(() => {
8    const w = 7;
9    setM(w);
10 }, 10);
11 setTimeout(() => {
12   const s = setK;
13   if (flag) {
14     s(3);
15   }
16 }, 10);

```

```

-- OneBlockBehind (intra, iterations=1)
state[0] =  $\top$    widened=false
state[1] = number[0, 7]   widened=false
state[2] = number[0, 0]   widened=false

```

Le callback en ligne droite est exact ($m \in [0, 7]$). Les deux autres ne le sont pas : n lit v comme $\top$, et l'alias s est perdu, si bien que l'écriture de k *disparaît*. La cause est dans `exec_body_impl` (liste 11.27) : chaque terminateur écrit `env_at[next]`, qui est donc l'environnement *d'entrée* de `next` ; mais l'environnement d'un bloc non initial est calculé comme le join des `env_at[p]` de ses prédécesseurs p , c'est-à-dire de leurs environnements *d'entrée* (figure 11.4). On attendrait `env_at.get(&bid)`. Une valeur perdue retombe à $\top$ (sound) ; un alias de setter, un site ou une liaison de `useCallback` perdus suppriment une écriture (FN). C'est aussi ce qui met `UpdaterGuarded`

de l'exemple 11.1 à $\top$: le bloc de jonction lit `__t0` dans l'environnement d'entrée de B1 et B2, où il n'est pas lié. Les tests unitaires ne le voient pas : leurs corps à plusieurs blocs ne lisent que des variables de l'environnement initial.

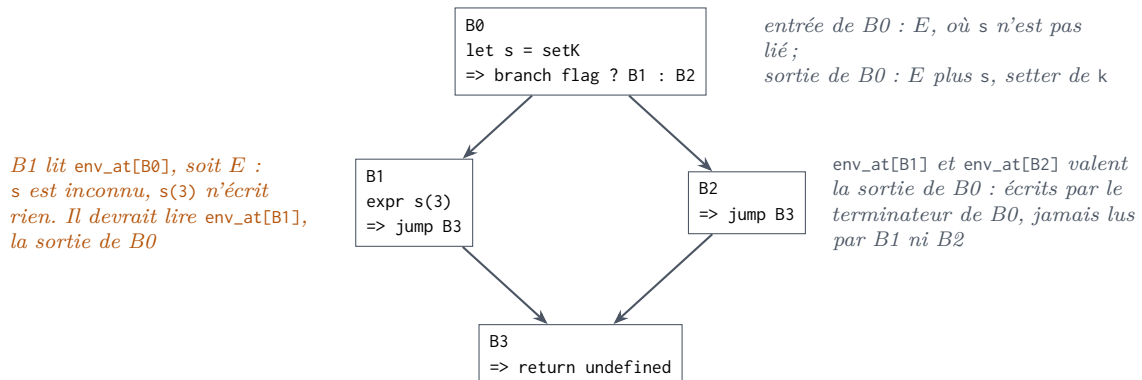


FIGURE 11.4 – Le défaut D1 sur le troisième callback de liste 11.38. `env_at` est indexé par le bloc *destinataire*, mais chaque bloc lit la table sous les noms de ses *prédécesseurs* : il voit l'entrée de son prédécesseur, soit l'état « d'un bloc en retard ».

11.12.3 D2 et D3 : où et comment un setter écrit

D2, le setter hors position d'effet. L'invariant 11.3 dit où un setter écrit. Trois effets sans deps, identiques au sens de React, le mettent à l'épreuve (`/tmp/ch11/ex6.tsx`) : `setA(a + 1)`; `(StmtPos), flag && setA(a + 1)`; `(AndPos), console.log(setA(a + 1))`; `(ArgPos)`.

```

-- StmtPos (intra, iterations=3)
  state[0] = number[0, inf] widened=true
-- AndPos (intra, iterations=0)
  state[0] = number[0, 0] widened=false
-- ArgPos (intra, iterations=0)
  state[0] = number[0, 0] widened=false

```

Le CLI signale `StmtPos` (infinite-loop) et déclare les deux autres propres. Dans `AndPos`, le lowering a rangé l'appel dans `__t0 := setA((a Add 1))` (un `Assign`) ; dans `ArgPos`, l'appel est un argument. Aucun des deux n'atteint `exec_setter_call`. Pire : si l'argument est un *functional updater* en position de liaison (`const r = setB((x) => ...)`), la pré-passe classe le callee `Setter` et ne descend pas le littéral, en comptant sur `exec_setter_call`, qui n'est pas appelé : ni l'écriture ni les effets du corps ne sont vus. `docs/limitations.md` affirme qu'un appel de setter « in any expression position [...] is seen » : c'est vrai pour la relation `slot_writers`, pas pour le store d'état.

D3, l'updater en mode programme. L'exemple 11.2 annonçait que le CLI ne retrouve pas le widening de `UpdaterNoDeps`. La sonde, en mode programme, le montre :

```

-- ComponentId(0) (iterations=1)
  state[0] = T widened=false shared=ref(PerRender)

```

Le premier bras de `exec_setter_call` exécute l'updater correctement ; le second, qui tire aussi pour un setter local dès que `ctx.inter` est présent, évalue l'argument `FnLit` comme une valeur : `ref(PerRender)`, la fermeture elle-même. Cette fermeture est écrite dans `shared_state[(composant, 0)]`, ré-importée par `slice` dans l'état ; le paramètre `x` devient « nombre ou référence », `x + 1` vaut $\top$, et plus aucun widening n'a lieu. Sur la copie du test `functional_updater_without_deps_is_flagged` (`/tmp/ch11/ex8.tsx, useEffect(() => setCount(c => c + 1); )`), le CLI répond `✓ 1 file(s) no issues found`. alors que le test d'intégration, qui

pas par le chemin intra-composant, est vert. C'est un défaut d'*interaction* : chaque bras est raisonnable isolément. Une correction possible — ne tirer le second bras que si le premier n'a pas tiré, ou si le propriétaire n'est pas `ctx.component` — reste à trancher par le mainteneur : elle touche la sémantique du second bras pour tout appel, pas seulement l'updater.

11.12.4 D4 et D7 : le contexte d'un enfant, oublié

Les deux défauts viennent de la même hypothèse : l'analyse d'un enfant ne dépendrait que des valeurs aplaties de ses props.

D4. Dans `/tmp/ch11/ex14.tsx`, `Board` rend deux `Ticker` (exemple 11.4) : `onTick={() => setA(1)}` puis `onTick={() => setB(b + 1)}`. Les deux fermetures valent `ref(PerRender)` : la clé du cache est la même, le second site d'appel touche le cache, et l'enfant n'est jamais analysé avec la fermeture qui boucle (`cache_hits=15` `cache_misses=1` pour tout le fichier). Le CLI est muet ; le même fichier où le second enfant est un composant `Ticker2` identique mais distinct donne `cross-component-infinite-loop`. La clé ignore les *corps* des fermetures, c'est-à-dire le tas.

D7. `Counter( step )` écrit `setN(n + step)` dans un effet sans deps. Rendu seul avec `step=1`, il est signalé (`infinite-loop`). Rendu deux fois, `<Counter step=1 /><Counter step=0 />` (`/tmp/ch11/ex22.tsx`), le CLI répond `Counter (2 hooks) ✓` : la seconde analyse a écrasé la première dans la table `results` (`src/domains/transfer/state_value.rs`, l. 583–586), et les règles ne lisent que la dernière. La table n'a qu'une entrée par `ComponentId`, alors qu'un composant a autant d'instances que de sites d'appel.

11.12.5 D5, D6, D9, D11, D13, D14 : le tas

Le tas concentre six défauts, tous des cas où un site, un membre ou un corps n'est pas suivi jusqu'au bout.

- **D5, lecture partielle d'un membre.** Avec `const o = flag ? f: 1 : ...p`, `o.f` est annoncé `number[1, 1]` alors qu'il peut valoir `p.f` ; même chose pour `q?.f` avec `q` nullable, et pour `r.f` quand l'autre branche est `g: 2` (`/tmp/ch11/ex18.tsx`). Conséquence observée, dans `/tmp/ch11/ex19.tsx` : un `useEffect(() => console.log(v), [])` où `v` vient d'un tel ternaire (`PartialDeps`) n'est pas signalé par `missing-deps`, alors que la variante sans ternaire l'est (`SpreadOnlyDeps`, même fichier).
- **D6, objets imbriqués et appels de méthodes.** `B6` ne traite que les calées `Var`, et un littéral imbriqué n'a pas de site : `o.f()` et `o.inner.f` manquent leur setter (vu en section 11.9). Avec `o = f: () => setN(n + 1)` dans un effet sans deps, `g = o.f` ; `g()` mène `n` au widening, `o.f()` non (`/tmp/ch11/ex12.tsx`).
- **D9, champ nouveau écrit sur une branche.** `if (flag) o.g = 2` `const v = o.g` ; donne `v = [2, 2]`, sans `undefined` (`/tmp/ch11/mw.tsx`) : le cas (`None`, `new`) de `MemberWrite` n'ajoute pas `undefined`, et le tas insensible au flot fait voir l'écriture au chemin qui ne l'a pas faite. Un champ *existant*, lui, est bien joint (`number[1, 1]|string"x"`).
- **D11, fermeture membre d'un objet prop.** Seuls les sites *directs* des props sont copiés dans le tas de l'enfant (liste 11.35). Avec `bag = run: () => setN(n + 1)` passé en prop, l'enfant qui appelle `bag.run` trouve un site sans valeur de tas et n'exécute rien ; la même fermeture passée directement (`run=run`) donne `cross-component-infinite-loop` (`/tmp/ch11/deep.tsx`).
- **D13, deux sauts de fermeture.** Dans un corps parcouru, `collect_escaping_setters` ne suit pas un appel vers une *autre* fermeture : avec `inner = (v) => setN(v)` ; `outer = (v) => inner(v)` passé dans `<Sheet cfg= cb: outer />`, `n` reste `[0, 0]`, alors qu'avec un seul saut il

vaut $\top$ (/tmp/ch11/esc2.tsx). Ce défaut-là est couvert globalement par l'Info « component Sheet was not found », qui suspend les vérifications ; la valeur calculée reste fausse.

- **D14, l'exécuteur de new Promise.** La pré-passe ne classe que les Call ; un New passe par la descente générique, qui n'entre pas dans un littéral de fonction. L'exécuteur de new Promise((resolve) => setN(n + 1); ...), que JavaScript exécute *synchroniquement*, n'est jamais exécuté : n reste [0, 0] et infinite-loop se déclare vérifiée (/tmp/ch11/np.tsx). Le cas fréquent new Promise(r => setTimeout(r, 100)).then(...) n'est pas touché.

11.12.6 D8, D10, D12 : trois modélisations

D8. const el = vaut ref(Stable) ; useEffect(..., [el]) n'est pas signalé par always-unstable-deps, alors que [obj] avec un littéral d'objet l'est (/tmp/ch11/ex24.tsx). En React, el est un objet neuf à chaque rendu. Ce pourrait être une décision implicite (éviter des FP sur children) ; aucun ADR ni commentaire ne la justifie.

D10. const s = c ? setA : setB ; s(1) ; n'écrit que a (/tmp/ch11/sj.tsx) : le join à priorité gauche des liaisons de setter garde setA, et la valeur jointe, SetterVal::Top, ne satisfait pas as_setter(), si bien que le second bras ne tire pas non plus. Le résultat dépend de l'ordre des opérandes du join.

D12. useReducer ignore le réducteur :

```
-- Red (intra, iterations=1)
state[0] = number[0, 1] widened=false
env[dispatch] = Val(setter(#4294967295#0))
```

Avec reducer = (s, a) => s + a et useEffect(() => dispatch(1);) (/tmp/ch11/red.tsx), l'action 1 est écrite telle quelle, le store converge sur [0, 1] et le CLI répond ✓. En React, s vaut 1, 2, 3, ... et l'effet relance le rendu indéfiniment. La cause est au lowering (le réducteur est sauté, chapitre 7), l'effet ici.

11.12.7 Synthèse

Remarque

Plusieurs de ces défauts se corrigent par une ligne ou par un déplacement de responsabilité, dans l'esprit du principe « pas de workaround » : lire env_at[bid] dans exec_body_impl (D1) ; tirer les setters depuis un point central qui voit toute position d'expression, sans double tir (D2) ; faire entrer les sites des props dans la clé du cache (D4). Ils sont proposés en exercice.

Résumé du chapitre

- L'interpréteur abstrait d'un pas tient en deux fonctions. La première, eval_state_value, est une récursion structurelle qui rend une StateValue sans jamais écrire l'état ; elle vit dans transfer/state_value.rs. La seconde, exec_full_stmt, dans interp/interpreter.rs, exécute une instruction en deux temps : pré-passe des callbacks, puis cœur. Le moteur les atteint par le trait Transfer, dont l'unique implémentation est StateValueTransfer.
- L'environnement AbstractEnv a quatre tables. Une variable inconnue vaut $\top$; une liaison de setter, de callback ou de site inconnue ne vaut rien, et sa perte coûte une écriture.
- Un appel vaut $\top$ sauf allocateur certain ; un littéral vaut ref(PerRender) ; un élément JSX ref(Stable). Le lowering a déjà transformé ternaires, &&, ||, ?? en diamants et les gabarits en additions.

	défaut	hypothèse violée	exemple
D1	exec_body_impl lit l'entrée des pré-décesseurs	propagation d'un CFG	ex3.tsx
D2	setter hors ExprStmt/Return	« tout appel qui peut avoir lieu est vu »	ex6.tsx
D3	second bras sur un updater local	un bras n'annule pas l'autre	ex8.tsx
D4	clé de cache aplatie	contexte = props aplaties	ex14.tsx
D5	membre joint sur les seuls sites qui l'ont	un site sans le membre compte	ex18.tsx
D6	B6 limité aux Var, littéraux imbriqués sans site	tout corps atteignable est suivi	ex11.tsx
D7	results écrasé par site d'appel	une instance par site	ex22.tsx
D8	élément JSX Stable	fraîcheur d'une allocation	ex24.tsx
D9	champ nouveau sans undefined	écriture may, lecture ailleurs	mw.tsx
D10	liaison de setter à priorité gauche	join des liaisons latérales	sj.tsx
D11	sites des membres de props non copiés	tas de l'enfant complet	deep.tsx
D12	réducteur ignoré	sémantique de dispatch	red.tsx
D13	un seul saut de fermeture	transitivité de l'échappement	esc2.tsx
D14	new jamais classé	exécuteur synchrone	np.tsx

TABLE 11.9 – Les quatorze défauts, tous reproduits sur /tmp/ch11/. D13 est couvert par une Info globale ; les autres sont des faux négatifs non déclarés.

- Quatre stores (définition 11.2) : `StateStore` ($\perp$, join, sujet du point fixe, amorcé par les initialiseurs), `MemoStore` ($\top$, écrasement, recalculé après chaque rendu), `SharedStateStore` ($\perp$, join, un par programme, importé par slice), `Heap` (par site, écrasement, insensible au flot).
- Un setter écrit par mise à jour faible, en trois positions seulement ; un *functional updater* lit le join du store. La lecture d'un état est convertie en `Versioned` en un seul point.
- Les callbacks sont classés par ce qui les déclenche (`Setter`, `InCycle`, `Subscription`, `Unknown`) ; seuls les `InCycle` sont exécutés, par un second interpréteur de CFG en une passe, jusqu'à la profondeur 3. `Unknown` non descendu est un FN accepté (ADR-009).
- Le tas relie un nom à un corps (B5, B6) et un objet à ses membres (#88) ; l'inlining d'un enfant copie les props dans un tas neuf, et les écritures remontent par le store partagé (ADR-012).
- Quatorze défauts (D1 à D14) montrent où ces garanties s'arrêtent : propagation des corps, positions des setters, contexte des enfants, membres et sites non suivis.

Le chapitre 12 décrira l'autre interpréteur de CFG, `analyze_cfg`, avec sa worklist, ses raffinements et ses arcs arrière ; le chapitre 13 la boucle qui enchaîne rendu, mémos, effets et handlers jusqu'au point fixe.

Exercices

Exercice 11.1: Calculer à la main

Donner la valeur abstraite de : (a) $a + 1$ avec $a = \text{number}[0, 3] \mid \text{null}$; (b) $x \ \& \ 7$ avec $x = \top$; (c) `typeof null` ; (d) `+s` avec $s = \text{string} "5", "6"$; (e) `!n` avec $n = \text{number}[0, 0]$; (f) `"a" + 1` ; (g) `+""`.

Solution

(a) `as_arith` accepte `null`, qui vaut 0 : l'opérande devient $[0, 3] \sqcup [0, 0] = [0, 3]$, et la somme $[1, 4]$. (b) Le masque est une constante positive : $[0, 7]$, quel que soit x . (c) Une seule sorte habitée, `null : string"object"`. (d) Les deux chaînes se lisent comme des nombres finis : `number[5, 6]`. (e) L'opérande n'est pas un booléen pur : $\top$ (et non `boolean`). (f) Chaîne plus nombre : $\top$. (g) `"".trim().parse::<f64>()` échoue en Rust : $\top$, alors que JavaScript donne `0`; c'est sound.

Exercice 11.2: Un gabarit

Pour `const t = 'id-$n'` avec n un état numérique, donner l'IR produite et la valeur de `t`. Proposer une amélioration sound.

Solution

La sonde d'impression donne `let t = ("id-" Add n) Add ""`, et la sonde de valeurs donne $\top$ pour `t` : `"id-" + n` tombe dans le cas « tout le reste » de `+`. Or en JavaScript, `+` dont un opérande est une chaîne rend toujours une chaîne : rendre `str_top()` dès qu'un côté est une chaîne pure serait sound et plus précis (la composante nombre resterait $\perp$).

Exercice 11.3: Dérouler un updater

Le slot `0` vaut $[0, 2]$. On exécute `setN(c => c + 1)`. Quelle valeur est écrite, que vaut le slot ensuite, et pourquoi contient-il encore `0` ?

Solution

`c` est lié à `ctx.state.get(0)`, soit $[0, 2]$; le corps rend $[1, 3]$, qui est la valeur écrite ; le slot devient $[0, 2] \sqcup [1, 3] = [0, 3]$. Il contient `0` parce que la mise à jour est faible : le store range le join de *toutes* les valeurs que l'état a pu prendre, initialiseur compris, pas sa valeur courante.

Exercice 11.4: Classer des callees

Donner la `TriggerClass` de `promise.then(f)`, `arr.forEach(f)`, `el.addEventListener("x", f)`, `myLib.subscribe(f)`, `Array.from(xs, f)`, `router.from(f)`, et dire si `f` est exécuté.

Solution

`then` : `InCycle`, exécuté. `forEach` : `InCycle`, exécuté. `addEventListener` : `Subscription`, non exécuté ici, mais relevé comme handler et analysé comme point d'entrée s'il est dans un effet. `subscribe` : `Unknown`, non exécuté (FN accepté, #19). `Array.from` : `InCycle`, le receveur étant `Array`. `router.from` : `Unknown`, car `from` n'est reconnu que sur le receveur `Array`.

Exercice 11.5: Le plafond de profondeur

Montrer que, dans liste 11.24, `f4()` n'atteint pas `setD`. Où l'utilisateur en est-il averti ? Pourquoi ne le serait-il pas si le composant était analysé seul ?

Solution

L'effet s'exécute à la profondeur 0 ; `f4()` est un `B6` qui exécute le corps de `f4` à la profondeur 1 , celui de `f3` à 2 , celui de `f2` à 3 . Dans ce dernier, la pré-passe de l'instruction `f1()` trouve $d \geq 3$ et s'arrête : `f1` n'est jamais exécuté. En mode programme, l'arrêt est enregistré dans `callback_depth_capped`, d'où l'Info `analysis-limit` et la suspension des vérifications. L'enregistrement est gardé par `if let Some(inter) = ctx.inter` : une analyse intra-composant (tests, phase 2 d'`analyze_program`) subit le plafond en silence.

Exercice 11.6: L'environnement vide

Calculer `AbstractEnv::bottom().join(&e)` pour $e = \{x \mapsto [1, 1]\}$. Pourquoi `analyze_cfg` n'est-il pas affecté ?

Solution

x n'est présent que du côté droit : il vaut $\top$. Le résultat est $\{x \mapsto \top\}$, pas e : l'environnement vide n'est pas neutre. `analyze_cfg` n'y est pas exposé parce qu'il insère le premier environnement reçu par un bloc sans `join` (`None => outgoing_env`), et ne joint qu'ensuite des environnements effectivement calculés.

Exercice 11.7: Corriger D1

Localiser dans liste 11.27 la ligne responsable de D1, écrire la correction, et décrire le test unitaire qui l'empêcherait de revenir.

Solution

Pour un bloc non initial, l'environnement est le `join` des `env_at.get(p)` des prédécesseurs (`src/domains/interp/interpreter.rs`, l. 419–425). Or les terminateurs écrivent déjà dans `env_at[next]` le `join` des sorties des prédécesseurs de `next` : il suffit de lire `env_at.get(&bid).cloned().unwrap_or_else(AbstractEnv::bottom)` pour tout bloc. Le test : un corps `() => const s = setK; if (c) s(3);` exécuté par `exec_body` doit écrire `[3, 3]` dans le slot de `setK` ; aujourd'hui, il n'écrit rien. Un second test sur un updater ternaire (exemple 11.1) doit rendre un intervalle et non $\top$.

Exercice 11.8: Une correction centrale de D2

Dans l'esprit du principe « pas de workaround » de `CLAUDE.md`, proposer un endroit unique où tirer les appels de `setter` pour qu'ils soient vus dans toute position d'expression, sans double tir.

Solution

Une réponse possible : la pré-passe visite déjà *tous* les appels, en toute position, et classe `Setter` les callees liés à un `setter`. C'est là qu'il faudrait appeler `exec_setter_call`, et le retirer de `exec_expr_effects` et du `Return` d'`exec_body_impl`, pour qu'un appel ne soit tiré qu'une fois. Le bras `Setter` y exécuterait aussi l'updater, ce qui corrige la variante « updater en position de liaison ». Il faut ensuite s'assurer qu'aucun appel n'est visité deux fois par la pré-passe au sein d'une même instruction, et que les rejeux post-convergence du moteur restent cohérents. Ce n'est qu'une piste, non implémentée ni testée ; le mainteneur tranchera.

Exercice 11.9: Une clé de cache qui voit D4

Sur `/tmp/ch11/ex14.tsx`, expliquer pourquoi la boucle n'est pas vue, et proposer une clé de cache qui la verrait.

Solution

Les deux props `onTick` valent `ref(PerRender)` une fois aplaties ; la clé est la même, le second site d'appel touche le cache. La carte `abstract_props_full` contient pourtant les *sites* des fermetures (`EnvVal::Loc`). Une clé qui inclurait, pour chaque prop, l'ensemble de ses sites distinguerait les deux appels. Il faudrait alors que `results` soit lui aussi indexé par site d'appel, sans quoi D7 masquerait la seconde analyse.

Quatrième partie

Le moteur

Chapitre 12

Interpréter un CFG : worklist, arcs arrière, dominance

Objectifs du chapitre.

- Savoir ce que calcule `analyze_cfg` sur un CFG : un environnement abstrait à la sortie de chaque bloc atteint et un store d'état enrichi de toutes les écritures de la passe. Savoir le dérouler à la main sur un bloc, un diamant et une boucle.
- Comprendre les trois opérateurs de fusion (première arrivée, join, *widening* à seuils), le compteur d'arcs arrière tenu par en-tête de boucle, et pourquoi l'analyse d'un CFG termine.
- Connaître le raffinement de garde (`narrow_env_for_branch`) : les motifs reconnus, ce qu'ils ne reconnaissent pas, et pourquoi une garde réfutée n'élimine pas sa branche.
- Savoir relire une passe terminée (`exit_env`, `entry_env_of`) et sonder une expression après convergence (`eval_in_stores`, `ConvergedEval`).
- Connaître les questions « sur tous les chemins » du moteur (`compute_dominators`, `DominatorTree`, `on_all_paths`, `ExitDominance`) et leurs conventions sur les sorties, les `throw` et les blocs inatteignables.

Prérequis : chapitre 3 (définition 3.2, définition 3.5, définition 3.6, définition 3.7, définition 3.9, algorithme 3); chapitre 6 (définition 6.1); chapitre 7 (définition 7.3, blocs de jonction); chapitre 9 (intervalles, `widen_to`, raffinements); chapitre 11 (définition 11.1, définition 11.2, `exec_stmt`).

Le chapitre 11 a donné la sémantique d'un *pas* : que vaut une expression, que fait une instruction. Le chapitre 13 donnera la boucle externe, celle qui enchaîne rendu, mémos, effets et handlers jusqu'à ce que l'état React ne bouge plus. Entre les deux, il manque une pièce de taille moyenne : exécuter abstraitement *un corps entier*, c'est-à-dire un CFG avec ses branchements et ses boucles. C'est le rôle de `analyze_cfg`, dans `src/engine/cfg_analyzer.rs`. Le rendu d'un composant, chacun de ses effets et chacun de ses handlers passent par elle, à chaque tour de la boucle externe.

Le chapitre 3 en a donné l'idée (algorithme 3) et un premier déroulé. Ce chapitre la reprend à la taille réelle du code, avec ses conventions : ce qui est suivi bloc par bloc et ce qui ne l'est pas, où l'on joint, où l'on accélère, ce que la condition d'un `if` apprend à l'analyse. Il présente ensuite trois services que le moteur et les règles construisent sur le même CFG une fois l'analyse finie : relire l'environnement d'un point du programme, évaluer une expression contre les stores convergés, et répondre aux questions de dominance (« ce bloc s'exécute-t-il à chaque rendu ? »).

12.1 Le problème : un corps, plusieurs chemins

Considérons un effet qui calcule une valeur par une boucle locale avant de l'écrire dans l'état. C'est le composant `BoundedLocalLoop` de `tests/fixtures/widening.tsx`, recopié sous un autre nom

:

Extrait 12.1 – Un effet avec une boucle locale (/tmp/ch12/loop.tsx)

```

1 import { useState, useEffect } from "react";
2
3 export function LocalLoop() {
4   const [total, setTotal] = useState(0);
5   useEffect(() => {
6     let i = 0;
7     while (i < 5) {
8       i = i + 1;
9     }
10    setTotal(i);
11  }, []);
12  return <div>{total}</div>;
13 }

```

Concrètement, l'effet s'exécute une fois (deps vides), la boucle fait cinq tours et setTotal écrit 5. L'analyseur ne dit rien de ce composant :

```

$ reactant check --verbose --info --no-color --project plain loop.tsx
[verbose] 1 components analyzed
[verbose] cache hits: 0, misses: 0
[verbose] LocalLoop: 1 iteration(s), widened: []
1 clean component(s) hidden, rerun with --show-clean

```

Pour arriver à ce silence, l'analyseur a dû répondre à une question qui n'a rien de spécifique à React : *quelles valeurs i peut-il avoir à chaque point du corps de l'effet ?* Il ne peut pas exécuter la boucle (en général, il ne connaît ni le nombre de tours ni les valeurs d'entrée). Il calcule donc, pour chaque bloc du CFG, un environnement abstrait qui sur-approxime toutes les valeurs concrètes possibles en ce point, sur tous les chemins qui y mènent. Trois formes de graphe suffisent à tout expliquer :

- la *séquence* : un bloc passe son environnement de sortie au suivant, sans perte ;
- le *diamant* : deux chemins se rejoignent, il faut un join ;
- la *boucle* : un chemin revient sur lui-même, la suite des environnements peut croître sans fin, il faut un widening.

Les sections 12.2 à 12.4 les traitent dans cet ordre.

12.1.1 Qui appelle analyze_cfg

analyze_cfg n'est pas appelée par les règles : c'est une brique du point fixe d'un composant. Ses cinq appelants sont tous dans analyze_component_impl (src/engine/fixpoint.rs) :

passe	CFG	environnement d'entrée	ligne
rendu (chaque tour)	render_cfg	initial_env (props liées)	366
effets (chaque tour)	corps de chaque effet	env_exit du rendu	429
handlers (chaque tour)	corps de chaque handler	env_exit du rendu	466
rafraîchissement	render_cfg	initial_env	548
écritures pures	corps de chaque effet	env_exit, état $\perp$	579

TABLE 12.1 – Les cinq appels d'analyze_cfg dans src/engine/fixpoint.rs. Les deux derniers ont lieu après convergence ; ils sont décrits au chapitre 13.

Voici la signature :

Extrait 12.2 – src/engine/cfg_analyzer.rs, l. 34–46 — signature d’analyze_cfg

```

34 pub fn analyze_cfg<'inter, T: Transfer>(
35     component: ComponentId,
36     cfg: &CFG,
37     entry_env: AbstractEnv<T::Domain>,
38     state: &StateStore<T::Domain>,
39     memo: &MemoStore<T::Domain>,
40     transfer: &T,
41     widen_threshold: usize,
42     thresholds: &[f64],
43     heap: &mut Heap,
44     ctx: &dyn QueryContext,
45     inter: Option<&'inter InterCtx<'inter>>,
46 ) -> (BlockEnvs<T::Domain>, StateStore<T::Domain>) {

```

Paramètre par paramètre :

- `component` : l’identité du composant analysé. Elle ne sert qu’à construire l’`AnalysisCtx` de chaque bloc (la provenance des slots, chapitre 10).
- `cfg` : le graphe à interpréter (définition 6.1).
- `entry_env` : l’environnement au début du bloc d’entrée.
- `state`, `memo` : les stores d’état et de mémos du tour courant de la boucle externe (définition 11.2), lus et non modifiés : la fonction en fait des copies.
- `transfer` : l’implémentation du trait `Transfer` ; en production c’est toujours `StateValueTransfer`.
- `widen_threshold`, `thresholds` : le nombre de passages par un arc arrière avant d’accélérer, et l’ensemble de seuils du widening (section 12.4).
- `heap` : le tas abstrait, emprunté mutablement et partagé avec les autres passes du composant.
- `ctx`, `inter` : l’accès en lecture aux entrées de hooks (`QueryContext`) et le contexte inter-composants (`None` en analyse isolée).

Le résultat est un couple : une table bloc $\mapsto$ environnement, et le store d’état enrichi des écritures de la passe.

12.2 Un bloc, puis une séquence

12.2.1 Deux tables, un store

La fonction commence par préparer son état de travail :

Extrait 12.3 – src/engine/cfg_analyzer.rs, l. 47–58 — initialisation

```

47 let mut entry_envs: HashMap<BlockId, AbstractEnv<T::Domain>> = HashMap::new();
48 let mut exit_envs: HashMap<BlockId, AbstractEnv<T::Domain>> = HashMap::new();
49 let mut state_out = state.clone();
50
51 entry_envs.insert(cfg.entry, entry_env);
52
53 let mut worklist: VecDeque<BlockId> = VecDeque::new();
54 let mut in_worklist: HashSet<BlockId> = HashSet::new();
55 worklist.push_back(cfg.entry);
56 in_worklist.insert(cfg.entry);
57
58 let mut back_edge_counts: HashMap<BlockId, usize> = HashMap::new();

```


Définition 12.1: Passe, environnement d'entrée, environnement de sortie

Une *passe* est une exécution d'`analyze_cfg` sur un CFG. Pendant une passe, chaque bloc b atteint porte un *environnement d'entrée* $In[b]$ (table `entry_envs`), valeur abstraite des variables avant sa première instruction, et un *environnement de sortie* $Out[b]$ (table `exit_envs`), valeur après son dernier transfert. Seul Out est rendu à l'appelant.

Trois choix se lisent déjà dans ces douze lignes.

- Seul le bloc d'entrée reçoit un environnement. Un bloc que rien n'atteint n'est jamais enfilé, donc jamais visité, et n'a pas de ligne dans `exit_envs` : l'absence de ligne joue le rôle de $\perp$ (« aucun état concret n'arrive ici »).
- `state_out` est *une seule* copie du store d'état pour toute la passe. L'état React n'est pas suivi bloc par bloc ; la section 12.6 en tire les conséquences.
- La worklist est une file FIFO (`VecDeque`) dédoublée par `in_worklist`. Ce n'est pas un parcours en ordre postfixe inverse : l'ordre de visite est l'ordre de découverte.

Piège

Le type de retour s'appelle `BlockEnvs`, et sa documentation dit « Per-block *entry* environments » (`src/engine/cfg_analyzer.rs`, l. 16–17). C'est faux : la fonction rend `exit_envs` (`src/engine/cfg_analyzer.rs`, l. 123), et sa propre documentation (`src/engine/cfg_analyzer.rs`, l. 21–22) le dit correctement. Tout consommateur de `block_states` lit des environnements de *sortie* ; celui qui veut l'entrée d'un bloc doit la reconstruire (section 12.7).

12.2.2 Le transfert d'un bloc

La boucle principale défile un bloc, applique ses instructions et range le résultat :

Extrait 12.4 – `src/engine/cfg_analyzer.rs`, l. 60–89 — visite d'un bloc

```

60  while let Some(b) = worklist.pop_front() {
61      in_worklist.remove(&b);
62
63      let env_in = entry_envs[&b].clone();
64      let mut env_out = env_in;
65      // Memo is fixed for this pass; local mutations are discarded.
66      let mut memo_local = memo.clone();
67
68      if let Some(block) = cfg.blocks.get(&b) {
69          let mut ac = AnalysisCtx {
70              component,
71              state: &mut state_out,
72              memo: &mut memo_local,
73              heap,
74              query: ctx,
75              inter,
76          };
77          for stmt in &block.stmts {
78              transfer.exec_stmt(stmt, &mut env_out, &mut ac);
79          }
80          // A concise-arrow body (`() => setN(1)`) lowers its expression to a
81          // `Return` terminator, not an `ExprStmt` – but its side effects must
82          // still fire. Run them through the shared effect-firing path instead
83          // of fabricating a throwaway `Stmt::ExprStmt`.
84          if let Terminator::Return(return_expr) = &block.term {
85              transfer.exec_expr_effects(return_expr, &mut env_out, &mut ac);
86          }

```

```

87     }
88
89     exit_envs.insert(b, env_out.clone());

```

Étape par étape :

1. `env_out` part d'une copie de $In[b]$.
2. Le store de mémos est *recopié à chaque bloc* (ligne 66) et la copie est jetée après le bloc : pendant une passe, les mémos sont figés. Ils ne sont recalculés qu'entre deux passes, par la boucle externe.
3. Un `AnalysisCtx` est construit pour le bloc. Il emprunte mutablement `state_out` et le tas : ce sont les deux canaux par lesquels un bloc communique avec les autres *sans* passer par les arêtes.
4. Chaque instruction passe par `exec_stmt` (chapitre 11).
5. Si le bloc se termine par `Return(e)`, les *effets de bord* de `e` sont exécutés (paragraphe suivant).
6. L'environnement obtenu devient $Out[b]$; une visite ultérieure du même bloc l'écrase.

Les tests unitaires du fichier épinglent les cas les plus simples. Sur un CFG d'un bloc vide, l'environnement d'entrée est rendu tel quel et l'état ne bouge pas (`empty_cfg_entry_env_preserved`, `src/engine/cfg_analyzer.rs`, l. 317–336). Un `let x = 42` donne $Out[0](x) = [42, 42]$ (`let_literal_extends_env`, `src/engine/cfg_analyzer.rs`, l. 338–363). Une séquence de deux blocs, `let x =` puis `return`, propage la valeur sans perte : $Out[1](x)$ est une référence `PerRender` (`two_block_exit_env_propagates`, `src/engine/cfg_analyzer.rs`, l. 592–649). Sur un arc `Jump`, la fonction de passage est l'identité.

12.2.3 Les effets de bord d'un `Return`

Un effet écrit avec une flèche concise n'a pas d'instruction :

Extrait 12.5 – Une flèche concise (`/tmp/ch12/concise.tsx`)

```

1 import { useState, useEffect } from "react";
2
3 export function Concise() {
4   const [n, setN] = useState(0);
5   useEffect(() => setN(n + 1));
6   return <div>{n}</div>;
7 }

```

Le lowering abaisse le corps `() => setN(n + 1)` en un seul bloc sans instruction, terminé par `Return(Call(setN, [n + 1]))`. Si `analyze_cfg` n'exécutait que les instructions, l'appel du setter serait invisible et cette boucle infinie passerait inaperçue : un faux négatif. D'où les lignes 80 à 86 de liste 12.4, qui confient l'expression du `Return` à `exec_expr_effects`, le même chemin que celui d'une instruction-expression `e`. Ce chemin (`src/domains/interp/interpreter.rs`, l. 114–126) fait trois choses : la pré-passe des callbacks, l'appel de setter (mise à jour faible), et l'évaluation d'une application de composant. La *valeur* de retour n'est pas calculée ici. Le diagnostic est bien produit :

```

$ reactant check --verbose --no-color --project plain concise.tsx
[verbose] Concise: 3 iteration(s), widened: [0]
Concise (2 hooks) concise.tsx
  warn  infinite-loop [hook:0] (line 5:2) this effect keeps pushing state `n` to new values on every run. Potential
  ↳ infinite render loop

```

Trois tests de `src/engine/cfg_analyzer.rs` fixent le comportement :

- un setter dans un Return écrit (l. 850–893, `setter_in_return_terminator_updates_state`);
- un `Return(Lit(Unit))` ne fait rien (l. 896–937, `unit_return_terminator_is_noop`);
- un *functional updater* passé dans un Return est exécuté (l. 940–1018, `functional_updater_in_return_terminator_fires`).

Le deuxième cas est le plus fréquent : depuis l’ADR-25, la fin d’un corps sans return est scellée `Return(Lit(Unit))`.

Remarque

Le commentaire des lignes 80 à 83 rappelle une correction : l’ancien code fabriquait une instruction `Stmt::ExprStmt` jetable pour réutiliser le chemin des instructions. Le trait `Transfer` expose désormais `exec_expr_effects` comme la définition unique de « exécuter une expression pour ses effets » (`src/domains/mod.rs`, l. 133–144).

12.2.4 L’environnement de fin d’un corps

Les passes d’effets et de handlers partent de `env_exit`, l’environnement de fin du rendu. Il est calculé à partir des sorties de la passe de rendu :

Extrait 12.6 – `src/engine/fixpoint.rs`, l. 1226–1237 — l’environnement de sortie d’un CFG

```
1226 fn exit_env<D: AbstractDomain>(<
1227     cfg: &CFG,
1228     block_states: &HashMap<BlockId, AbstractEnv<D>>,
1229 ) -> AbstractEnv<D> {
1230     cfg.blocks
1231         .values()
1232         .filter(|b| matches!(b.term, crate::ir::cfg::Terminator::Return(_)))
1233         .filter_map(|b| block_states.get(&b.id))
1234         .cloned()
1235         .reduce(|acc, env| acc.join(&env))
1236         .unwrap_or_else(AbstractEnv::bottom)
1237 }
```

C’est le join des `Out[b]` des blocs terminés par un `Return`. Un `Return` inatteignable n’a pas de ligne dans `block_states` et le `filter_map` l’écarte. La réduction se fait par `reduce`, pas par un `fold` partant de `AbstractEnv::bottom()`, et c’est voulu : le paragraphe suivant montre que la table vide n’est pas neutre pour le join. La même fonction existe en méthode publique sur `AnalysisResult` (`src/engine/analysis_result.rs`, l. 279–288) pour les règles.

12.3 Le diamant : joindre aux points de jonction

12.3.1 Ce que reçoit chaque successeur

Après la visite d’un bloc, `analyze_cfg` demande à `outgoing` la liste de ses successeurs, chacun avec l’environnement qu’il reçoit :

Extrait 12.7 – `src/engine/cfg_analyzer.rs`, l. 126–150 — l’environnement passé à chaque successeur

```
126 /// The env each successor of `b` receives when `b` exits with `env_out`: a
127 /// branch narrows on its condition, a jump passes the env on unchanged.
128 fn outgoing<D: AbstractDomain>(<
129     cfg: &CFG,
130     b: BlockId,
```

```

131   env_out: &AbstractEnv<D>,
132 ) -> Vec<(BlockId, AbstractEnv<D>>> {
133   if let Some(block) = cfg.blocks.get(&b) {
134     match &block.term {
135       Terminator::Branch {
136         cond, then_, else_, ..
137       } => vec![
138         (*then_, narrow_env_for_branch(env_out, cond, true)),
139         (*else_, narrow_env_for_branch(env_out, cond, false)),
140       ],
141       Terminator::Jump(succ) => vec![(*succ, env_out.clone())],
142       Terminator::Return(_) | Terminator::Unreachable => vec![],
143     }
144   } else {
145     cfg.successors(b)
146       .into_iter()
147       .map(|s| (s, env_out.clone()))
148       .collect()
149   }
150 }

```

outgoing lit le *terminateur* du bloc, pas la liste `cfg.edges`. Un Jump passe l'environnement tel quel ; un Branch le raffine selon sa condition, vers `then_` avec `taken = true` et vers `else_` avec `taken = false` (section 12.5) ; Return et Unreachable n'ont pas de successeur. La branche finale, qui lit `cfg.successors`, ne sert que pour un identifiant de bloc absent de `cfg.blocks`. Sur un CFG bien formé, les deux lectures coïncident : `CFG::validate` exige que toute cible de terminateur ait son arête (`src/ir/cfg.rs`, l. 181–227).

12.3.2 Un diamant, pas à pas

Le test `diamond_joins_at_merge_point` (`src/engine/cfg_analyzer.rs`, l. 651–753) construit à la main l'IR de ce fragment :

Extrait 12.8 – Le diamant du test, en source équivalent

```

1 let x;
2 if (true) { x = 1; } else { x = {}; }
3 // ici : x vaut 1 ou un objet frais

```

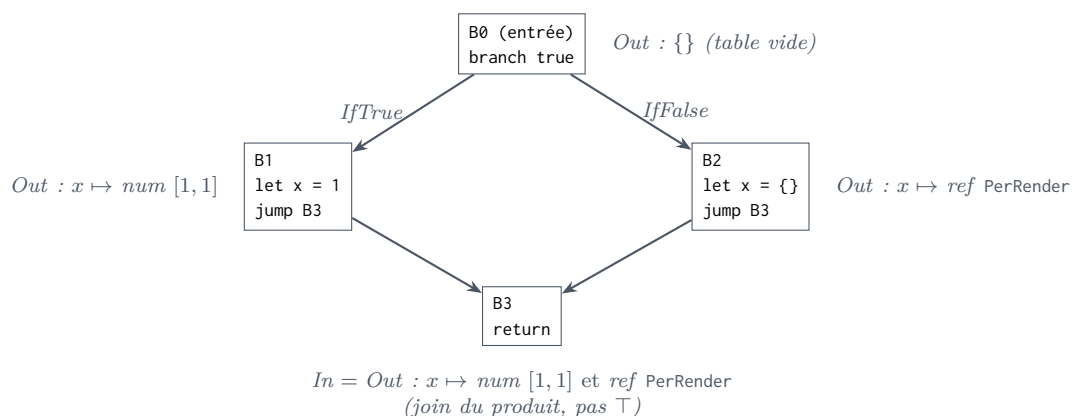


FIGURE 12.1 – Le diamant de `diamond_joins_at_merge_point`, annoté des environnements calculés par `analyze_cfg`.

La figure 12.1 montre le déroulé. La condition est le littéral `true` : `narrow_env_for_branch`

ne reconnaît pas ce motif (section 12.5), les deux branches reçoivent la table vide inchangée et sont visitées. La branche `else` est pourtant concrètement morte ; l’analyse l’explore quand même, ce qui est *sound*. B1 est visité en premier et installe $In[B3]$ par « première arrivée » (`None => outgoing_env`, ligne 98). B2 arrive ensuite et provoque le `join` (ligne 109). Le test vérifie que le `join` du produit `StateValue` garde *les deux* sortes : la composante numérique vaut $[1, 1]$, la composante référence `PerRender`, et la valeur n’est pas $\top$ (définition 10.1). Un domaine plat aurait perdu toute information ici, c’était la motivation de l’ADR-015.

12.3.3 Le join des environnements, et pourquoi $\perp$ n’y est pas neutre

Le `join` de deux `AbstractEnv` est point à point, avec une règle particulière pour une variable présente d’un seul côté :

Extrait 12.9 – `src/domains/stores/abstract_env.rs`, l. 130–142 — `join` des valeurs : une variable d’un seul côté vaut $\top$

```
130 fn join_stabs(a: &HashMap<Var, D>, b: &HashMap<Var, D>) -> HashMap<Var, D> {
131     let mut out = HashMap::new();
132     for (k, v) in a {
133         let w = map_get_or(b, k, D::top());
134         out.insert(k.clone(), v.join(&w));
135     }
136     for k in b.keys() {
137         if !a.contains_key(k) {
138             out.insert(k.clone(), D::top());
139         }
140     }
141     out
142 }
```

La règle suit l’invariant 11.2 : une variable absente d’un environnement s’y lit $\top$, donc son `join` avec n’importe quoi vaut $\top$. Dans le diamant ci-dessus, si seule la branche `then` liait `x`, B3 lirait `x` à $\top$. C’est *sound* : sur le chemin `else`, `x` a la valeur qu’il avait avant le `if`, que la table ne connaît pas.

Piège

La conséquence est que `AbstractEnv::bottom()`, la table vide, n’est *pas* un élément neutre du `join` : $\perp \sqcup \rho$ rend toutes les variables de ρ à $\top$. Écrire `fold(AbstractEnv::bottom(), join)` sur une liste d’environnements détruit toute l’information ; c’est pourquoi `exit_env` utilise `reduce`. De même, la première arrivée sur un bloc *installe* l’environnement reçu au lieu de le joindre à une entrée vide. En revanche l’ordre `leq` de la table lit une variable absente comme $\perp$ (`src/domains/stores/abstract_env.rs`, l. 216–239) : les deux conventions ne sont pas duales, et `analyze_cfg` ne se sert d’ailleurs pas de `leq`.

12.3.4 Détecter un changement

Une fois la nouvelle entrée calculée, le successeur n’est ré-enfilé que si elle diffère de l’ancienne (ligne 114, liste 12.10 plus bas). Le test est une égalité structurelle $!=$, pas $\neg(new \sqsubseteq old)$. Les deux sont équivalents ici, parce que la nouvelle entrée est toujours au-dessus de l’ancienne (un `join` ou un `widening` de l’ancienne) : pour $new \sqsupseteq old$, on a $new \sqsubseteq old \iff new = old$ dans un ordre partiel.

Remarque

L'égalité structurelle voit parfois un changement là où il n'y en a pas. Un intervalle vide n'a pas de représentation canonique : `Interval` compare ses bornes (`src/domains/impls/interval.rs`, l. 23–27), et `narrow_geq(5)` appliqué à $[0, 0]$ rend l'intervalle $\{lo = 5, hi = 0\}$, vide, tandis que le même raffinement de $[0, 1]$ rend $\{lo = 5, hi = 1\}$, vide aussi, mais différent pour `==`. La sonde du chapitre l'observe sur la boucle de la section suivante : la sortie de boucle, dont l'entrée reste $\perp$, est ré-enfilée et revisitée à chaque tour de la boucle (pas 7 et 10 du tableau 12.2). C'est un surcoût, borné par le nombre de changements de l'en-tête, sans effet sur le résultat.

12.4 La boucle : arcs arrière, compteur par en-tête, widening à seuils

12.4.1 La propagation

Voici la fin de la boucle principale, qui traite chaque successeur :

Extrait 12.10 – `src/engine/cfg_analyzer.rs`, l. 91–121 — fusion et propagation vers les successeurs

```

91     for (succ, outgoing_env) in outgoing(cfg, b, &env_out) {
92         let is_back = cfg
93             .edges
94             .iter()
95             .any(|e| e.from == b && e.to == succ && matches!(e.kind, EdgeKind::Back));
96
97         let new_entry = match entry_envs.get(&succ) {
98             None => outgoing_env,
99             Some(existing) => {
100                 if is_back {
101                     let cnt = back_edge_counts.entry(succ).or_insert(0);
102                     *cnt += 1;
103                     if *cnt >= widen_threshold {
104                         existing.widen_to(&outgoing_env, thresholds)
105                     } else {
106                         existing.join(&outgoing_env)
107                     }
108                 } else {
109                     existing.join(&outgoing_env)
110                 }
111             }
112         };
113
114         if entry_envs.get(&succ) != Some(&new_entry) {
115             entry_envs.insert(succ, new_entry);
116             if in_worklist.insert(succ) {
117                 worklist.push_back(succ);
118             }
119         }
120     }
121 }
```

Trois cas pour la nouvelle entrée ν d'un successeur s recevant ρ :

- s n'a pas encore d'entrée : $\nu = \rho$;
- l'arc $b \rightarrow s$ est de nature `EdgeKind::Back` : on incrémente le compteur de s ; tant qu'il est sous `widen_threshold`, $\nu = In[s] \sqcup \rho$, ensuite $\nu = In[s] \nabla_T \rho$ (`widen_to` avec les seuils T) ;
- sinon : $\nu = In[s] \sqcup \rho$.

Le compteur `back_edge_counts` est indexé par le bloc *cible*, c'est-à-dire l'en-tête de la boucle, et non par l'arc. Deux arcs arrière vers le même en-tête (la fin du corps et un `continue`) incrémentent le même compteur : une boucle qui contient un `continue` peut atteindre le seuil en moins de tours. Le compteur compte les *arrivées*, qu'elles changent l'entrée ou non. Il est local à la passe : chaque appel à `analyze_cfg` repart de zéro. Enfin, sa valeur de déclenchement `widen_threshold` est le même paramètre que celui de la boucle externe, 3 par défaut (`src/engine/fixpoint.rs`, l. 51–60).

La nature de l'arc est retrouvée en parcourant `cfg.edges` (lignes 92 à 95) : c'est le lowering qui décide où l'on accélère, en marquant `Back` les arêtes qui reviennent vers un en-tête (définition 7.3).

12.4.2 L'ensemble des seuils

Les seuils viennent du programme lui-même. Ils sont récoltés une fois par composant, avant la boucle externe :

Extrait 12.11 – `src/engine/fixpoint.rs`, l. 1185–1206 — récolte des seuils du widening

```
1185 /// Harvest the finite threshold set for "widening up-to" (see ADR-014).
1186 ///
1187 /// Collects numeric literals from the render CFG, all hook bodies, and useState
1188 /// init expressions – the constants against which guarded state growth is
1189 /// bounded. Over-collecting is harmless: the set stays finite (termination) and
1190 /// extra thresholds only add candidate bounds (precision, never unsoundness).
1191 fn collect_thresholds(render_cfg: &CFG, hooks: &[HookEntry]) -> Vec<f64> {
1192     let mut out: Vec<f64> = Vec::new();
1193     collect_lits_cfg(render_cfg, &mut out);
1194     for hook in hooks {
1195         match hook {
1196             HookEntry::State { init, .. } => collect_lits_expr(init, &mut out),
1197             HookEntry::Effect { body_cfg, .. } | HookEntry::Handler { body_cfg, .. } => {
1198                 collect_lits_cfg(body_cfg, &mut out)
1199             }
1200         }
1201     }
1202     out.sort_by(|a, b| a.partial_cmp(b).unwrap_or(std::cmp::Ordering::Equal));
1203     out.dedup();
1204     out
1205 }
1206 }
```

Tous les littéraux numériques du rendu, des corps d'effets et de handlers (y compris ceux des fonctions imbriquées, `src/engine/fixpoint.rs`, l. 1212–1224) et des initialiseurs de `useState` entrent dans T , triés et dédoublés. Le commentaire donne la justification : récolter trop est sans danger, l'ensemble reste fini et un seuil de plus n'ajoute qu'une borne candidate. Le même T est passé à toutes les passes du composant, donc aussi aux boucles locales. L'opérateur `widen_to` lui-même est présenté au chapitre 9 (théorème 9.2) : une borne qui croît saute au plus proche seuil qui l'englobe, ou à $\pm\infty$ s'il n'y en a pas. Sur `StateValue`, seul le composant numérique utilise les seuils ; les autres composantes prennent le widening ordinaire (`src/domains/impls/state_value.rs`, l. 670–676).

12.4.3 Le compteur de boucle, pas à pas

Deux tests de `src/engine/cfg_analyzer.rs` (l. 450–509),

- `loop_counter_unbounded_without_thresholds`,
- `loop_counter_bounded_by_threshold`,

analysent le même CFG, `counting_loop_cfg` (l. 365–448), l'IR de `let i = 0; while (i < 5) i = i + 1; .` Le premier passe $T = \emptyset$, le second $T = \{5\}$ (la constante de la garde, comme la récolterait `collect_thresholds`).

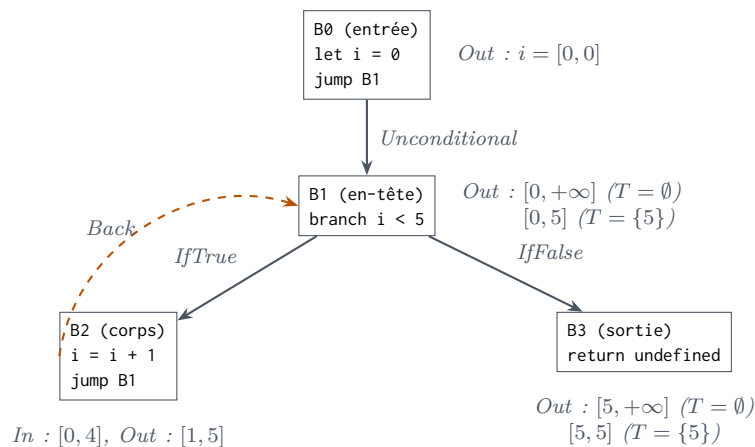


FIGURE 12.2 – `counting_loop_cfg` et ses environnements finaux (composante numérique de `i`). L’arc arrière est en pointillé.

Le tableau 12.2 donne chaque visite. Il a été relevé avec la réplique instrumentée de la sonde ; les environnements finaux qu’elle trouve sont identiques à ceux d’`analyze_cfg` et à ceux que les deux tests vérifient.

Quelques points méritent d’être soulignés.

- Au pas 1, `i` n’est pas lié dans la table d’entrée : sa lecture vaut $\top$ (invariant 11.2). Le `let` l’écrase.
- Au pas 2, la branche « faux » raffine $[0, 0]$ par `i >= 5` et obtient un intervalle vide. Le bloc B3 est quand même enfilé et visité (pas 4) : un raffinement vers $\perp$ ne rend pas un bloc inatteignable (section 12.5). Ici c’est sans conséquence, B3 ne fait rien.
- Les deux premiers retours par l’arc arrière joignent : l’en-tête voit $[0, 1]$ puis $[0, 2]$, exactement les valeurs concrètes des deux premiers tours. Le troisième retour déclenche le widening.
- Sans seuil, la borne supérieure saute à $+\infty$. Le raffinement de la garde rattrape une partie de la perte *sur les arcs sortants* : le corps reçoit $[0, 4]$ (`i` est entier, `i < 5` donne `i <= 4`), la sortie $[5, +\infty]$. Mais l’en-tête garde $[0, +\infty]$, et c’est ce que lit l’assertion du premier test.
- Avec le seuil 5, la borne saute à 5 au lieu de $+\infty$; la sortie de boucle reçoit alors exactement $[5, 5]$, la valeur concrète. C’est ce que vérifie le second test, et ce qui permet à `LocalLoop` d’écrire $[5, 5]$ dans `total`.
- Au pas 12, le corps renvoie $[1, 5]$, contenu dans l’entrée de l’en-tête : `widen_to` ne touche pas une borne qui ne croît pas, la nouvelle entrée est égale à l’ancienne, rien n’est enfilé. La passe se termine dès que la file est vide.

Décision : ADR-014

L’ADR-014 prévoyait deux mécanismes : le widening à seuils (partie 1) et un *narrowing* classique, c’est-à-dire une phase descendante après convergence (partie 2). La révision du 28 juin 2026 abandonne la partie 2. Constat : sur le compteur gardé, le raffinement de garde pendant la phase ascendante suffit ; sur la boucle locale ci-dessus, ce qui manquait était le widening à seuils *sur l’arc arrière interne* d’`analyze_cfg`, qui donne $[0, 5]$ en tête et $[5, 5]$ en sortie sans phase descendante. Les deux techniques ne récupèrent que des bornes littérales, et le widening à seuils les récupère déjà en montant ; une descente ne pourrait de toute

pas	bloc	$In \rightarrow Out$ (valeur de i)	mises à jour des successeurs
<i>pas communs aux deux variantes</i>			
1	B0	$\top \rightarrow [0, 0]$	$In[B1] := [0, 0]$ (première arrivée)
2	B1	$[0, 0]$	$In[B2] := [0, 0]; In[B3] := \perp$
3	B2	$[0, 0] \rightarrow [1, 1]$	arrière n° 1, join : $In[B1] := [0, 1]$
4	B3	$\perp$	–
5	B1	$[0, 1]$	$In[B2] := [0, 1]; In[B3]$ reste vide (ré-enfilé)
6	B2	$[0, 1] \rightarrow [1, 2]$	arrière n° 2, join : $In[B1] := [0, 2]$
7	B3	$\perp$	–
8	B1	$[0, 2]$	$In[B2] := [0, 2]; In[B3]$ reste vide (ré-enfilé)
<i>sans seuil, $T = \emptyset$</i>			
9	B2	$[0, 2] \rightarrow [1, 3]$	arrière n° 3, widening : $[0, 2] \nabla [1, 3] = [0, +\infty]$
10	B3	$\perp$	–
11	B1	$[0, +\infty]$	$In[B2] := [0, 4]; In[B3] := [5, +\infty]$
12	B2	$[0, 4] \rightarrow [1, 5]$	arrière n° 4 : $[0, +\infty] \nabla [1, 5]$ inchangé
13	B3	$[5, +\infty]$	– (file vide)
<i>avec seuil, $T = \{5\}$</i>			
9	B2	$[0, 2] \rightarrow [1, 3]$	arrière n° 3, widening : $[0, 2] \nabla_T [1, 3] = [0, 5]$
10	B3	$\perp$	–
11	B1	$[0, 5]$	$In[B2] := [0, 4]; In[B3] := [5, 5]$
12	B2	$[0, 4] \rightarrow [1, 5]$	arrière n° 4 : $[0, 5] \nabla_T [1, 5]$ inchangé
13	B3	$[5, 5]$	– (file vide)

TABLE 12.2 – Les treize visites d’analyze_cfg sur counting_loop_cfg (widen_threshold = 3). Seule la composante numérique de i est montrée; $\perp$ désigne un intervalle vide.

façon pas faire redescendre le store d’état, alimenté par des joins monotones. Alternatives refusées : la phase descendante bornée (narrow_iterations), la séparation WidenOutcome dans les résultats et la suppression des écritures pures (effect_setter_writes); aucune n’est implémentée au commit e67b10a. L’opérateur narrow reste une infrastructure différée, à reprendre seulement si l’on voulait récupérer des bornes symboliques.

Piège

Le mot « narrowing » désigne deux choses dans le projet. Les méthodes narrow_lt, narrow_truthy, etc. et narrow_env_for_branch sont du *raffinement de garde* : elles restreignent une variable sous la condition d’une branche. Le narrowing de l’ADR-014 est une phase descendante du point fixe ; il n’existe pas dans le code. Le texte de l’ADR fait lui-même la distinction.

12.4.4 L’algorithme complet

L’algorithme 5 récapitule analyze_cfg avec tous ses détails. Il raffine l’algorithme 3 sur trois points : le store d’état partagé, les effets de bord du Return, et le compteur par en-tête.

12.5 Raffiner par les gardes

12.5.1 Le problème

Reprenons le compteur gardé de l’ADR-014 :

Algorithme 5 : `analyze_cfg` (src/engine/cfg_analyzer.rs, l. 34–124)

Input : CFG d'entrée e , environnement ρ_0 , stores S (état) et M (mémos), tas H , seuil k , seuils T

Output : Out (bloc $\mapsto$ environnement de sortie), S'

- 1 $In \leftarrow \{e \mapsto \rho_0\}$; $Out \leftarrow \emptyset$; $S' \leftarrow S$; $C \leftarrow \emptyset$ (compteurs);
- 2 $W \leftarrow [e]$;
- 3 **while** W non vide **do**
- 4 $b \leftarrow \text{défiler}(W)$;
- 5 $\rho \leftarrow In[b]$; $M_b \leftarrow$ copie de M ;
- 6 **foreach** instruction i de b **do**
- 7 $\rho \leftarrow \text{exec_stmt}(i, \rho)$; // lit et écrit S' , H , M_b
- 8 **if** b se termine par $\text{Return}(x)$ **then** $\text{exec_expr_effects}(x, \rho)$;
- 9 $Out[b] \leftarrow \rho$;
- 10 **foreach** $(s, \rho_s) \in \text{outgoing}(b, \rho)$; // garde raffinée sur un Branch
- 11 **do**
- 12 **if** $s \notin \text{dom}(In)$ **then**
- 13 $\nu \leftarrow \rho_s$;
- 14 **else if** $b \rightarrow s$ est un arc Back **then**
- 15 $C[s] \leftarrow C[s] + 1$;
- 16 **if** $C[s] \geq k$ **then** $\nu \leftarrow In[s] \nabla_T \rho_s$ **else** $\nu \leftarrow In[s] \sqcup \rho_s$;
- 17 **else** $\nu \leftarrow In[s] \sqcup \rho_s$;
- 18 **if** $\nu \neq In[s]$ **then** $In[s] \leftarrow \nu$; enfile s s'il n'est pas dans W ;
- 19 **return** (Out, S')

Extrait 12.12 – Un compteur gardé (/tmp/ch12/guarded.tsx)

```

1 import { useState, useEffect } from "react";
2
3 export function Guarded() {
4   const [count, setCount] = useState(0);
5   useEffect(() => {
6     if (count < 10) setCount(count + 1);
7   }, [count]);
8   return <div>{count}</div>;
9 }

```

L'effet s'exécute, incrémente, se ré-exécute, jusqu'à `count = 10`, puis s'arrête. Pour le prouver, l'analyse doit utiliser la condition du `if` : dans la branche `then`, `count` est strictement inférieur à 10, donc `setCount` écrit au plus 10. Sans cette information, la valeur écrite suivrait la valeur lue et croîtrait sans fin. Le moteur l'utilise :

```

$ reactant check --verbose --info --no-color --project plain guarded.tsx
[verbose] Guarded: 3 iteration(s), widened: [0]
Guarded (2 hooks) guarded.tsx
info widening-info (line 5:2) state `count` kept changing during analysis and was approximated to converge, so
  ↳ findings that depend on it may be imprecise
...
verified infinite-loop no effect diverges into an infinite render loop

```

(sortie élaguée : les autres lignes `verified` sont omises.)

La boucle externe a dû accélérer (`widened: [0]`), mais le widening à seuils s'arrête sur le seuil 10 et le raffinement borne l'écriture; `infinite-loop` se déclare vérifiée. Le déroulé tour par tour appartient au chapitre 13. Il suffit de changer la forme de la garde pour perdre cette preuve :

Extrait 12.13 – La même garde, écrite `limit > count` (/tmp/ch12/unguarded.tsx)

```

1 import { useState, useEffect } from "react";
2
3 export function Unguarded({ limit }: { limit: number }) {
4   const [count, setCount] = useState(0);
5   useEffect(() => {
6     if (limit > count) setCount(count + 1);
7   }, [count, limit]);
8   return <div>{count}</div>;
9 }

```

Unguarded (2 hooks) unguarded.tsx

warn infinite-loop [hook:0] (line 5:2) this effect keeps pushing state `count` (its deps do not provably gate it $\hookrightarrow$, so the effect can re-run every render) to new values on every run. Potential infinite render loop

La condition compare deux variables. Aucun motif ne la reconnaît, aucun seuil ne vaut `limit : count` est widené jusqu'à $+\infty$. Le WARNING est un défaut *possible* (une prop `limit` valant Infinity bouclerait vraiment), mais pour toute limite finie c'est un faux positif, toléré par l'invariant de soundness; le corriger demanderait un domaine relationnel.

12.5.2 Ce que fait le code

Définition 12.2: Raffinement de garde

Le *raffinement de garde* d'un environnement ρ par une condition c sur la branche $t \in \{\text{vrai}, \text{faux}\}$ est un environnement ρ' qui ne diffère de ρ qu'en une variable x testée par c , avec $\rho'(x) \sqsubseteq \rho(x)$ et $\gamma(\rho'(x)) \supseteq \{v \in \gamma(\rho(x)) \mid c[x \mapsto v] \text{ peut s'évaluer à } t\}$. C'est `narrow_env_for_branch`.

La fonction commence par un utilitaire qui remplace une seule variable :

Extrait 12.14 – `src/engine/cfg_analyzer.rs`, l. 201–217 — `narrow_env_for_branch` : une variable à gauche, sinon rien

```

201 pub(crate) fn narrow_env_for_branch<D: AbstractDomain>(
202     env: &AbstractEnv<D>,
203     cond: &Expr,
204     taken: bool,
205 ) -> AbstractEnv<D> {
206     let with_refined = |x: &str, refined: D| {
207         let mut narrowed = env.clone();
208         narrowed.extend(x.to_string(), refined);
209         narrowed
210     };
211
212     match cond {
213         Expr::BinOp { op, lhs, rhs } => {
214             let Expr::Var(x) = lhs.as_ref() else {
215                 return env.clone();
216             };
217             match rhs.as_ref() {

```

Puis elle distingue la forme du membre droit. La comparaison numérique à un littéral a été montrée au chapitre 9 (liste 9.27); voici la suite, les comparaisons à `null` et `undefined` et les tests de vérité :

Extrait 12.15 – `src/engine/cfg_analyzer.rs`, l. 242–293 — raffinements de nullité et de vérité

```

242 Expr::Lit(Prim::Null) => {

```

```

243         let cur = env.lookup(x);
244         let refined = match (op, taken) {
245             (BinOp::Eq, true) | (BinOp::Neq, false) => cur.narrow_keep_nullish_only(),
246             (BinOp::Eq, false) | (BinOp::Neq, true) => cur.narrow_drop_null(),
247             _ => cur,
248         };
249         with_refined(x, refined)
250     }
251     Expr::Lit(Prim::Unit) => {
252         let cur = env.lookup(x);
253         let refined = match (op, taken) {
254             (BinOp::Eq, true) | (BinOp::Neq, false) => cur.narrow_keep_nullish_only(),
255             (BinOp::Eq, false) | (BinOp::Neq, true) => cur.narrow_drop_undef(),
256             _ => cur,
257         };
258         with_refined(x, refined)
259     }
260     _ => env.clone(),
261 }
262 }
263 // Truthiness guard `if (x)`: the taken branch excludes every falsy
264 // value (null, undefined, 0, "", false); the else branch keeps only
265 // the falsy ones (references are always truthy → ⊥ there).
266 Expr::Var(x) => with_refined(
267     x,
268     if taken {
269         env.lookup(x).narrow_truthy()
270     } else {
271         env.lookup(x).narrow_falsy()
272     },
273 ),
274 // `if (!x)`: branches swap – the ELSE branch is the truthy one.
275 Expr::UnaryOp {
276     op: crate::ir::expr::UnaryOp::Not,
277     arg,
278 } => {
279     if let Expr::Var(x) = arg.as_ref() {
280         with_refined(
281             x,
282             if taken {
283                 env.lookup(x).narrow_falsy()
284             } else {
285                 env.lookup(x).narrow_truthy()
286             },
287         )
288     } else {
289         env.clone()
290     }
291 }
292 _ => env.clone(),
293 }

```

Le tableau 12.3 résume la fonction ; il complète le tableau 9.2 par la colonne « ce qui est touché » et par les formes non reconnues.

12.5.3 Pourquoi ces raffinements sont sound

Trois précautions rendent ce tableau sound ; toutes sont écrites dans le code.

- *Le raffinement numérique ne touche que num.* Une comparaison JavaScript convertit ses

condition	branche vraie	branche fausse	composantes de StateValue touchées
$x < c$, $x \leq c$, $x > c$, $x \geq c$ (c littéral entier ou flottant)	<code>narrow_lt</code> , <code>_leq</code> , <code>_gt</code> , <code>_geq</code>	la négation : <code>_geq</code> , <code>_gt</code> , <code>_leq</code> , <code>_lt</code>	num seul
$x == c$, $x != c$ (c numérique)	<code>narrow_eq</code> <code>narrow_neq</code>	<code>narrow_neq</code> <code>narrow_eq</code>	num seul
$x == \text{null}$, $x != \text{null}$	<code>narrow_keep_nullish_only</code> <code>/ narrow_drop_null</code>	<code>narrow_drop_null</code> <code>narrow_keep_nullish_only</code>	tout (keep) ou null (drop)
$x == \text{undefined}$, $x != \text{undefined}$	<code>narrow_keep_nullish_only</code> <code>/ narrow_drop_undef</code>	<code>narrow_drop_undef</code> <code>narrow_keep_nullish_only</code>	tout ou undef
x $!x$	<code>narrow_truthy</code> <code>narrow_falsy</code>	<code>narrow_falsy</code> <code>narrow_truthy</code>	toutes sauf other toutes sauf other
$c < x$, $x < y$, $x.f < c$, $x === \text{"a"}$, $a \ \&\& \ b$, <code>true</code> , appel...	environnement inchangé		aucune

TABLE 12.3 – Motifs de `narrow_env_for_branch` (`src/engine/cfg_analyzer.rs`, l. 201–294). `==` et `===` sont confondus par l'IR.

opérandes : `null < 5` est vrai. Retirer `null` ou les chaînes de la branche « vrai » serait donc faux ; le commentaire de `src/domains/impls/state_value.rs`, l. 604–605 le dit.

- *L'IR confond `==` et `===`.* Le raffinement doit donc être correct pour les deux sémantiques à la fois (`src/engine/cfg_analyzer.rs`, l. 193–196). La branche où `x == null` est vrai garde `null` et `undefined` (vrai de `==`, sur-approximation de `===`) ; la branche où `x != null` est vrai ne retire que `null` (vrai de `!==` ; pour `!=`, qui retirerait aussi `undefined`, c'est une sur-approximation). `narrow_keep_nullish_only` réactive en outre `null` et `undefined` si la composante `other` est présente, car elle peut les cacher (`src/domains/impls/state_value.rs`, l. 551–558).
- *Un raffinement entier n'est appliqué qu'aux entiers.* Le passage de `i < 5` à `i <= 4` suppose que `i` est entier, ce que garantit le bit `is_int` ; sur un flottant la borne reste 5 (théorème 9.3).

Enfin, tout motif non reconnu rend l'environnement inchangé, ce qui est toujours *sound* : ne rien apprendre ne perd aucune valeur.

12.5.4 Une garde réfutée n'élimine pas sa branche

Que se passe-t-il quand le raffinement rend $\perp$? Le composant suivant est sain : `k` vaut 3, la condition `k > 10` est toujours fausse, l'effet n'écrit jamais.

Extrait 12.16 – Une branche morte (`/tmp/ch12/dead.tsx`)

```

1 import { useState, useEffect } from "react";
2
3 export function Dead() {
4   const [n, setN] = useState(0);
5   const k = 3;
6   useEffect(() => {
7     if (k > 10) {
8       setN(n + 1);
9     }
10  });
11  return <div>{n}</div>;
12 }
```

```
$ reactant check --verbose --no-color --project plain dead.tsx
[verbose] Dead: 5 iteration(s), widened: [0]
Dead (2 hooks) dead.tsx
  warn  infinite-loop [hook:0] (line 6:2) this effect keeps pushing state `n` to new values on every run. Potential
        ↪ infinite render loop
```

Sur la branche vraie, `narrow_gt(10)` appliqué à $k = [3, 3]$ donne bien un intervalle vide. Mais `outgoing` rend *toujours* les deux successeurs d'un `Branch` (lignes 135 à 140), le successeur est enfilé dès que son entrée change (lignes 114 à 118), et ni `analyze_cfg` ni le transfert ne testent « une variable vaut $\perp$, donc le bloc est inatteignable ». Le bloc est exécuté ; seule k y vaut $\perp$; n garde sa valeur et `setN(n + 1)` écrit une vraie valeur. Le `WARNING` est un faux positif : imprécis, *sound*.

Piège

Il serait tentant de « corriger » en ne propageant pas vers un successeur dont l'environnement raffiné contient une variable $\perp$. Ce correctif central n'est pas fait, et sa *soundness* n'est pas évidente : il faut que $\perp$ signifie « aucune valeur concrète » pour *tout* le produit `StateValue`, y compris quand la composante `other` ou le raffinement de vérité (qui laisse `other` intact) interviennent. Aucune issue ne suit ce point au commit e67b10a (recherche faite par le dossier 06, gh issue list sans résultat sur ce sujet).

12.5.5 Les autres utilisateurs du raffinement

`narrow_env_for_branch` est `pub(crate)` parce que deux preuves du moteur la réutilisent hors de toute passe : les preuves de convergence des gardes du graphe de `churn` (`src/engine/guards.rs`, l. 235 et `src/engine/guards.rs`, l. 771) appliquent les gardes d'un site d'écriture à l'environnement de sortie du rendu. Ce partage est la règle 3 du projet appliquée : une seule définition de ce qu'une garde apprend, au niveau central (chapitre 15).

12.6 L'état dans une passe : un store insensible au flot

Les environnements sont suivis bloc par bloc ; l'état `React`, le tas et les mémos ne le sont pas. Il faut comprendre pourquoi ce mélange est correct.

Le store d'état. `state_out` est créé par `state.clone()` (ligne 49), prêté à chaque bloc, et chaque `setter` y fait une mise à jour faible (un `join`, chapitre 11). Une lecture `StateVal(1)` dans un bloc voit donc les écritures de *tous* les blocs déjà visités pendant la passe, sur n'importe quel chemin. Le store est *insensible au flot* à l'intérieur d'un CFG.

Rappel React : l'état ne change pas au milieu d'un rendu

Pendant un rendu, `count` est une constante : un `setCount` n'agit que sur la file de mises à jour, et la nouvelle valeur n'est vue qu'au rendu suivant. Dans un effet ou un handler, la variable `count` est de même la valeur capturée par la closure. Seul un *functional updater* (`setCount(c => c + 1)`) lit l'état courant de la file. Un store unique, sans position, est donc proche de la sémantique de `React` : ce qui compte est l'ensemble des valeurs écrites, pas l'endroit du corps où on les écrit.

Invariant 12.1: Une passe ne fait que monter l'état

À tout moment d'une passe, `state_out` $\sqsupseteq$ `state`, et `state_out` ne fait que croître. Le store rendu contient donc l'état d'entrée et toutes les valeurs écrites pendant la passe.

L'ordre de visite a une conséquence : un bloc visité *avant* un bloc qui écrit ne voit pas cette écriture pendant la passe. Ce n'est pas un défaut de soundness, grâce à la boucle externe : le tour suivant repart d'un état qui inclut l'écriture, et le point fixe n'est atteint que lorsqu'aucune passe n'ajoute plus rien. À ce moment, `state_out` reste égal à `state` pendant toute la passe (il part de `state`, ne fait que croître, et finit $\sqsubseteq$ `state`), et chaque bloc a vu l'état final.

Le tas. Le tas est emprunté mutablement et n'est jamais recopié : il est partagé par toutes les passes d'un tour, et d'un tour à l'autre. C'est voulu : une closure allouée pendant le rendu doit être visible quand un effet l'appelle par une variable (le motif B5, commentaire `src/engine/cfg_analyzer.rs`, l. 25–28). Le test `heap_persists_across_render_and_effect_passes` du point fixe l'épingle (`src/engine/fixpoint.rs`, l. 2157–2232).

Les mémos. À l'inverse, le store de mémos est recopié par bloc et les écritures locales sont jetées : une passe lit les mémos du tour, elle ne les modifie pas. Leur recalcul est l'affaire de la boucle externe (chapitre 13).

12.7 Relire une passe terminée

12.7.1 Ce que garantit une passe

Quand la file est vide, que sait-on ?

Proposition 12.1: Une passe calcule un post-point fixe des équations de blocs

Supposons que `state_out` ne change pas pendant la passe (c'est le cas au point fixe de la boucle externe). À la fin d'`analyze_cfg`, pour tout bloc visité b : $Out[b] = f_b(In[b])$, où f_b est le transfert du bloc ; et pour tout successeur (s, ρ_s) rendu par `outgoing` depuis $Out[b]$: $\rho_s \sqsubseteq In[s]$. Si les transferts et les raffinements sont corrects, $\gamma(Out[b])$ contient tous les environnements concrets à la sortie de b .

Preuve. Toute modification de $In[b]$ enfile b ; la dernière visite de b a donc lieu avec l'entrée finale, et $Out[b]$ est le transfert de cette entrée. Lors de cette visite, chaque successeur a reçu $\nu \sqsupseteq \rho_s$ (première arrivée, join ou widening, tous au-dessus de ρ_s), et les entrées ne font ensuite que croître. Le système d'équations $In[s] \sqsupseteq \bigsqcup_p \rho_{p \rightarrow s}$, $Out[b] = f_b(In[b])$ est donc satisfait en post-point fixe, et le théorème 3.2 donne la sur-approximation. $\square$

L'hypothèse sur `state_out` n'est pas décorative : pendant un tour qui n'est pas le dernier, un bloc peut avoir été évalué contre un état plus petit que l'état final de la passe (section 12.6). Seule la boucle externe rend le résultat final sound.

12.7.2 `entry_env_of` : reconstruire l'entrée d'un bloc

Les règles et les relations ont parfois besoin de l'environnement *au milieu* d'un bloc, par exemple à l'instruction d'un appel de `setter`. La table `In` n'est pas publiée ; `entry_env_of` la reconstitue à partir des sorties :

Extrait 12.17 – `src/engine/cfg_analyzer.rs`, l. 152–185 — relire l'environnement d'entrée d'un bloc

```
152 /// The entry env of `block`, read back from a finished pass: `initial` for
153 /// the entry block, else the join of what each predecessor hands it from that
154 /// predecessor's exit env (ADR-042 §2).
155 ///
```

```

156 /// Joined, never widened. Every exit env over-approximates its block's
157 /// concrete exits, so their join over-approximates the concrete entry; the
158 /// widened entry the pass converged on is at least as large, never smaller.
159 /// A predecessor with no exit env was unreachable and contributes nothing.
160 pub(crate) fn entry_env_of<D: AbstractDomain>(
161     cfg: &CFG,
162     block: BlockId,
163     initial: &AbstractEnv<D>,
164     exits: &HashMap<BlockId, AbstractEnv<D>>,
165 ) -> AbstractEnv<D> {
166     if block == cfg.entry {
167         return initial.clone();
168     }
169     let mut env: Option<AbstractEnv<D>> = None;
170     for edge in cfg.edges.iter().filter(|e| e.to == block) {
171         let Some(exit) = exits.get(&edge.from) else {
172             continue;
173         };
174         for (succ, out) in outgoing(cfg, edge.from, exit) {
175             if succ != block {
176                 continue;
177             }
178             env = Some(match env {
179                 None => out,
180                 Some(e) => e.join(&out),
181             });
182         }
183     }
184     env.unwrap_or_else(AbstractEnv::bottom)
185 }

```

Pour le bloc d'entrée, c'est l'environnement initial. Pour un autre bloc, on rejoue outgoing depuis la sortie de chaque prédécesseur (le raffinement de garde est donc inclus) et l'on joint. Un prédécesseur sans sortie n'a pas été atteint et ne contribue rien. Le commentaire (lignes 156 à 159) donne l'argument de soundness : chaque sortie sur-approxime les sorties concrètes de son bloc, donc leur join sur-approxime l'entrée concrète. Ce join n'est jamais widené ; il est en général *plus précis* que l'entrée sur laquelle la passe a convergé (qui peut avoir sauté à un seuil), jamais moins sûr.

L'unique consommateur est la colonne `written` de la relation des écrivains, dans `src/engine/written.rs`, l. 255–274 : l'argument d'une écriture `y` est évalué dans l'environnement d'entrée de son bloc, puis on rejoue les instructions qui précèdent l'appel (chapitre 14).

Exemple 12.1: Relire l'en-tête de la boucle

Sur `counting_loop_cfg`, les prédécesseurs de l'en-tête B1 sont B0, qui lui passe $[0, 0]$, et B2, dont la dernière sortie vaut $[1, 5]$ dans les deux variantes (tableau 12.2). `entry_env_of` rend donc $[0, 0] \sqcup [1, 5] = [0, 5]$ pour B1, avec ou sans seuil. Avec $T = \{5\}$, c'est l'entrée finale de la passe. Sans seuil, l'entrée finale était $[0, +\infty]$: la relecture est strictement plus précise que l'entrée widenée, et reste sound (i vaut concrètement 0 à 5 en tête). Pour B2, elle rejoue la branche vraie depuis `Out[B1]` et rend $[0, 4]$ dans les deux cas : join est le hull des bornes (`src/domains/impls/interval.rs`, l. 71–78), et `narrow_lt` plafonne un entier à $v - 1$ (`src/domains/impls/interval.rs`, l. 262–268), ce qui rend ce calcul vérifiable directement sur ces deux définitions, sans exécuter `entry_env_of` elle-même.

12.8 Dominance : ce qui s'exécute sur tous les chemins

12.8.1 Le problème

Tout ce qui précède calcule des *valeurs*. Certaines règles posent une question de *structure* : ce bloc s'exécute-t-il à chaque passage dans le corps ? Trois composants la rendent concrète :

Extrait 12.18 – Trois questions de dominance (/tmp/ch12/early.tsx)

```

1 import { useState } from "react";
2
3 export function Panel({ ready }: { ready: boolean }) {
4   if (!ready) {
5     return null;
6   }
7   const [open, setOpen] = useState(false);
8   return <div onClick={() => setOpen(!open)}>{String(open)}</div>;
9 }
10
11 export function Tabs({ tab }: { tab: number }) {
12   const [seen, setSeen] = useState(0);
13   if (tab > 0) {
14     setSeen(tab);
15   }
16   return <div>{seen}</div>;
17 }
18
19 export function Always({ tab }: { tab: number }) {
20   const [seen, setSeen] = useState(0);
21   setSeen(tab);
22   return <div>{seen}</div>;
23 }

```

```

$ reactant check --no-color --project plain early.tsx
Always (1 hooks) early.tsx
  error setter-in-render [hook:0] (line 21:2) setter `setSeen` called directly in the render body, move this call
    ↳ into a useEffect or an event handler
Panel (2 hooks) early.tsx
  error conditional-hook [hook:0] (line 7:8) this hook is called conditionally (not on every render path)
Tabs (1 hooks) early.tsx
  warn setter-in-render [hook:0] (line 14:4) setter `setSeen` called directly in the render body, move this call
    ↳ into a useEffect or an event handler

```

Dans `Panel`, le bloc du `useState` ne domine pas la sortie du `return null` : le hook est sauté sur un chemin, **ERROR**. Dans `Tabs` et `Always`, la même écriture pendant le rendu reçoit deux niveaux différents : dans `Always`, le bloc de l'appel domine toutes les sorties, l'appel a lieu à chaque rendu et le défaut est certain (**ERROR**) ; dans `Tabs`, il ne les domine pas, le défaut n'est que possible (**WARNING**). La règle le dit en commentaire (`src/rules/impls/setter_in_render.rs`, l. 104–107). La dominance est donc l'un des outils qui *certifient* un niveau **ERROR** (chapitre 17).

12.8.2 compute_dominators

La définition 3.9 caractérise $\text{Dom}(b)$ comme la plus grande solution de $\text{Dom}(e) = \{e\}$, $\text{Dom}(b) = \{b\} \cup \bigcap_{p \in \text{pred}(b)} \text{Dom}(p)$. Le code est l'algorithme itératif classique :

Extrait 12.19 – `src/engine/dominance.rs`, l. 6–58 — le plus grand point fixe de la dominance

```

6 pub fn compute_dominators(cfg: &CFG) -> HashMap<BlockId, HashSet<BlockId>> {
7   let all_blocks: HashSet<BlockId> = cfg.blocks.keys().copied().collect();
8   let mut dom: HashMap<BlockId, HashSet<BlockId>> = HashMap::new();

```

```

9
10  dom.insert(cfg.entry, {
11      let mut s = HashSet::new();
12      s.insert(cfg.entry);
13      s
14  });
15
16  for &b in &all_blocks {
17      if b != cfg.entry {
18          dom.insert(b, all_blocks.clone());
19      }
20  }
21
22  let rpo = rpo(cfg);
23
24  let mut changed = true;
25  while changed {
26      changed = false;
27      for &b in &rpo {
28          if b == cfg.entry {
29              continue;
30          }
31          let preds = cfg.predecessors(b);
32          if preds.is_empty() {
33              continue;
34          }
35
36          let new_dom_base: HashSet<BlockId> = preds
37              .iter()
38              .filter_map(|p| dom.get(p))
39              .fold(None:::<HashSet<BlockId>>, |acc, pred_dom| {
40                  Some(match acc {
41                      None => pred_dom.clone(),
42                      Some(a) => a.intersection(pred_dom).copied().collect(),
43                  })
44              })
45              .unwrap_or_default();
46
47          let mut new_dom = new_dom_base;
48          new_dom.insert(b);
49
50          if dom[&b] != new_dom {
51              dom.insert(b, new_dom);
52              changed = true;
53          }
54      }
55  }
56
57  dom
58 }

```

- L'entrée est initialisée à $\{e\}$, tout autre bloc à l'ensemble de *tous* les blocs (lignes 10 à 20) : on part du haut du treillis $(\mathcal{P}(\text{Blocs}), \supseteq)$ et l'on descend.
- Les blocs sont parcourus en ordre postfixe inverse (RPO), jusqu'à ce qu'un tour complet ne change rien (*changed*).
- Le nouvel ensemble d'un bloc est l'intersection des ensembles de ses prédécesseurs, plus lui-même (lignes 36 à 48). Un bloc sans prédécesseur autre que l'entrée est sauté (ligne 32).
- Les prédécesseurs sont lus dans `cfg.edges` par `cfg.predecessors` (`src/ir/cfg.rs`, l. 173–179),

un parcours linéaire des arêtes.

L'ordre RPO vient d'un parcours en profondeur récursif depuis l'entrée :

Extrait 12.20 – `src/engine/dominance.rs`, l. 113–135 — ordre postfixe inverse

```
113 /// Reverse Post-Order traversal starting from `cfg.entry`.
114 pub fn rpo(cfg: &CFG) -> Vec<BlockId> {
115     let mut visited = HashSet::new();
116     let mut post_order = Vec::new();
117     dfs_post(cfg, cfg.entry, &mut visited, &mut post_order);
118     post_order.reverse();
119     post_order
120 }
121
122 fn dfs_post(
123     cfg: &CFG,
124     node: BlockId,
125     visited: &mut HashSet<BlockId>,
126     post_order: &mut Vec<BlockId>,
127 ) {
128     if !visited.insert(node) {
129         return;
130     }
131     for succ in cfg.successors(node) {
132         dfs_post(cfg, succ, visited, post_order);
133     }
134     post_order.push(node);
135 }
```

Un bloc inatteignable n'est pas dans le RPO. Il n'est jamais recalculé et garde « tous les blocs » : `dominates(a, b)` est vrai pour tout a . C'est neutre pour un fait `must` (le bloc ne s'exécute jamais), mais piégeux pour qui énumère les blocs sans test d'atteignabilité (plus bas). Quand un prédécesseur inatteignable participe à une intersection, son ensemble complet est l'élément neutre de l'intersection : il ne retire rien.

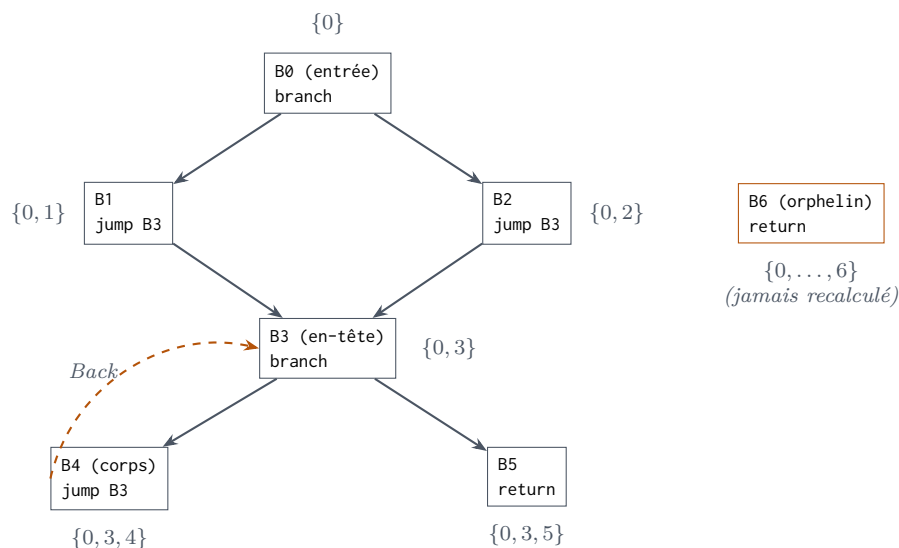


FIGURE 12.3 – Un losange suivi d'une boucle, et un bloc orphelin. Sous chaque bloc, son ensemble de dominateurs, tel que `compute_dominators` le rend (relevé à la sonde). RPO : $[0, 2, 1, 3, 5, 4]$.

Exemple 12.2: Dominateurs d'un losange suivi d'une boucle

Sur la figure 12.3 (un rendu schématique du type `if (fast) ... else ... while (delay > 10) ... return ...`, plus un `return` orphelin comme en laisse un `if/else` dont les deux branches retournent), la sonde donne le RPO $[0, 2, 1, 3, 5, 4]$. Premier tour : $\text{Dom}(2) = \{0, 2\}$, $\text{Dom}(1) = \{0, 1\}$; $\text{Dom}(3) = \{3\} \cup (\{0, 1\} \cap \{0, 2\} \cap \text{Tous}) = \{0, 3\}$ (B4, visité plus tard, vaut encore « tous » et ne retire rien); $\text{Dom}(5) = \{0, 3, 5\}$; $\text{Dom}(4) = \{0, 3, 4\}$. Second tour : $\text{Dom}(3) = \{3\} \cup (\{0, 1\} \cap \{0, 2\} \cap \{0, 3, 4\}) = \{0, 3\}$, inchangé; rien ne bouge, l'algorithme s'arrête. B6, inatteignable, garde $\{0, \dots, 6\}$.

12.8.3 DominatorTree et dominates

Recalculer la dominance à chaque question coûte un point fixe par question. `DominatorTree` mémorise le résultat :

Extrait 12.21 – `src/engine/dominance.rs`, l. 94–111 — la relation précalculée

```

94 /// Precomputed dominator relation for one CFG. Build once, query in O(1)-set -
95 /// the fix for `dominates()` driving per-rule quadratic recomputation.
96 pub struct DominatorTree {
97     dom: HashMap<BlockId, HashSet<BlockId>>,
98 }
99
100 impl DominatorTree {
101     pub fn new(cfg: &CFG) -> Self {
102         DominatorTree {
103             dom: compute_dominators(cfg),
104         }
105     }
106
107     /// `true` iff block `a` dominates block `b`.
108     pub fn dominates(&self, a: BlockId, b: BlockId) -> bool {
109         self.dom.get(&b).is_some_and(|dom_b| dom_b.contains(&a))
110     }
111 }

```

Malgré son nom, la structure ne contient pas un arbre de dominateurs immédiats mais la table complète des ensembles. La question `dominates(a, b)` est un test d'appartenance; un bloc `b` absent de la table (identifiant inconnu) répond faux. La fonction libre `dominates(cfg, a, b)` (`src/engine/dominance.rs`, l. 60–67) reconstruit tout à chaque appel, et sa documentation met en garde : « otherwise this is $O(\text{queries} \times \text{fixpoint})$ ».

12.8.4 Les sorties atteignables : ExitDominance

« S'exécute à chaque rendu » signifie « domine toutes les sorties ». Encore faut-il savoir ce qu'est une sortie. La couche règles en a un propriétaire unique :

Extrait 12.22 – `src/rules/api/query.rs`, l. 647–671 — les sorties sont les Return atteignables

```

647 pub struct ExitDominance {
648     domtree: DominatorTree,
649     exits: Vec<BlockId>,
650 }
651
652 impl ExitDominance {
653     pub fn of(cfg: &CFG) -> Self {
654         // Reachable exits only. A `Return` no path can arrive at is not an exit:
655         // lowering seals a fall-through tail as `Return(undefined)`, and an

```

```

656 // `if`/`else` whose both branches returned leaves that tail orphaned -
657 // counting it would make every hook before the branch fail to dominate
658 // "all exits" and report as conditional at the Error tier.
659 let reachable = cfg.reachable_blocks();
660 let exits = cfg
661   .blocks
662   .values()
663   .filter(|b| matches!(b.term, Terminator::Return(_)))
664   .map(|b| b.id)
665   .filter(|id| reachable.contains(id))
666   .collect();
667 ExitDominance {
668   domtree: DominatorTree::new(cfg),
669   exits,
670 }
671 }

```

Deux conventions s'y lisent. Une sortie est un bloc terminé par `Return` ; un `throw` (terminateur `Unreachable`) n'en est pas une. Et un `Return` que rien n'atteint n'est pas une sortie : le filtre passe par `CFG::reachable_blocks` (`src/ir/cfg.rs`, l. 153–163). Les méthodes `certify` et `may_be_skipped` qui suivent ont été présentées au chapitre 3 (liste 3.25).

Décision : ADR-025

Avant l'ADR-025, la fin d'un corps sans `return` était scellée `Unreachable`, comme un `throw`. Une greffe (chapitre 8) ne raccordait donc pas un appelé sans `return` à la suite de l'appelant : le `Return` de l'appelant n'avait plus de prédécesseur, `block_states` n'en avait pas de sortie, et `exit_env` joignait *moins de chemins que le programme n'en a* (208 composants touchés sur le corpus, dans la direction interdite). Décision : (1) une fin de corps est scellée `Return(Lit(Unit))` ; (2) `throw` garde `Unreachable` et la greffe continue de l'ignorer ; (3) un `Return` inatteignable n'est pas une sortie. Alternative refusée pour (2) : relier chaque `Unreachable` de l'appelé à la jonction. Elle invente un chemin qui contourne les hooks de l'appelé et produisait un `conditional-hook ERROR` sur l'idiome conforme `if (ctx === undefined) throw ...; useState(...)`. Le point (3) est la conséquence du point (1) : un `if/else` dont les deux branches retournent laisse désormais une queue `Return` orpheline, et la compter rendrait « conditionnel » tout hook placé avant.

Piège

B6, dans la figure 12.3, est un `Return` que rien n'atteint. La table le déclare dominé par *tous* les blocs, B4 compris, alors qu'aucun chemin ne passe par B4 avant B6 : ce n'est pas un fait sur le programme, c'est un artefact de l'initialisation. Toute réponse de `dominates(_, 6)` est donc vide de sens, et un consommateur qui énumère « tous les `Return` » sans `reachable_blocks` fait dépendre son verdict de cet artefact. `ExitDominance` ne pose jamais la question sur une sortie inatteignable ; c'est la seule façon correcte d'interroger la dominance des sorties.

Ce constat rend d'ailleurs difficile à rejouer, avec le code actuel, le mécanisme que décrit le point (3) de l'ADR-25 : une queue `Return` orpheline comptée parmi les sorties ferait *échouer* tout hook placé avant la branche à « dominer toutes les sorties ». Or, avec `compute_dominators` tel qu'il est écrit au commit e67b10a, c'est l'inverse qui se lit sur B6 : un bloc inatteignable est dominé par *tous* les blocs, donc `dominates(a, b)` vaut vrai pour tout *a* quand *b* est l'orphelin, et compter l'orphelin parmi les sorties ferait dominer *trivialement* cette sortie fantôme par tout hook, sans faire échouer `certify`. L'ADR-25 ne précise pas si `compute_dominators` avait, à sa date (29 juillet 2026), la même convention d'initialisation qu'au commit e67b10a ; le mécanisme exact qui aurait produit le `conditional-hook ERROR` qu'elle décrit n'est pas reconstituable depuis le seul code présent. Le point ne change rien à la décision retenue :

filtrer les sorties par `reachable_blocks` est correct indépendamment de ce mécanisme.

12.8.5 `on_all_paths` : un ensemble sur tous les chemins

La seconde question porte sur un *ensemble* de blocs : tout chemin de l'entrée à une sortie passe-t-il par l'un d'eux ? C'est ce que demande `infinite-loop` quand elle veut certifier qu'un effet écrit une valeur fraîche à chaque exécution, quelle que soit la branche prise (`src/rules/impls/infinite_loop.rs`, l. 454), ou le graphe de churn pour ses arêtes `Must` (`src/engine/churn.rs`, l. 538).

Extrait 12.23 – `src/engine/dominance.rs`, l. 69–92 — l'ensemble post-domine-t-il l'entrée ?

```

69 /// True when every entry→exit path of `cfg` passes through one of `blocks` –
70 /// the set post-dominates the entry. The must-reach fact behind
71 /// `must_on_all_paths` and the churn graph's `Must` edges (ADR-042 §6).
72 pub fn on_all_paths(cfg: &CFG, blocks: &HashSet<BlockId>) -> bool {
73     if blocks.contains(&cfg.entry) {
74         return true;
75     }
76     // BFS avoiding `blocks`; reaching an exit block means a path escapes.
77     let mut visited: HashSet<BlockId> = HashSet::new();
78     let mut queue = vec![cfg.entry];
79     visited.insert(cfg.entry);
80     while let Some(bid) = queue.pop() {
81         let succs = cfg.successors(bid);
82         if succs.is_empty() {
83             return false; // exit reached without hitting a block of the set
84         }
85         for succ in succs {
86             if !blocks.contains(&succ) && visited.insert(succ) {
87                 queue.push(succ);
88             }
89         }
90     }
91     true
92 }
```

L'idée est de chercher un contre-exemple : un chemin qui part de l'entrée, évite l'ensemble et atteint une feuille (un bloc sans successeur). S'il en existe un, la réponse est non ; sinon oui. Plusieurs détails comptent :

- *Toute feuille compte.* Un `Return` comme un `Unreachable (throw)` : `cfg.successors` est vide dans les deux cas. Un chemin d'exception qui évite l'ensemble suffit à dire non. C'est le choix prudent pour un fait `must`.
- *Le parcours est un DFS.* Le commentaire dit « BFS », mais `Vec::pop` dépile le dernier élément. Le résultat est le même, seul l'ordre de visite change.
- *Un cycle qui évite l'ensemble ne dit pas non.* Un chemin infini n'atteint aucune feuille. En particulier, sur un CFG qui ne contient aucune feuille atteignable (une boucle `while (true)` sans `break`), la fonction rend *vrai pour tout ensemble* ; la sonde l'a vérifié avec un ensemble réduit à un bloc inexistant.

Remarque

Cette vacuité contraste avec `ExitDominance`, dont la documentation dit qu'« a CFG with no reachable exit proves nothing in either direction » et qui rend `None`. Les deux quantificateurs n'ont pas la même convention sur un corps qui ne termine jamais, ni sur `throw` (une feuille

pour `on_all_paths`, pas une sortie pour `ExitDominance`). Chacune est défendable pour son usage ; il faut le savoir avant de remplacer l'une par l'autre.

`infinite-loop` appelle `must_on_all_paths` sur le CFG d'un effet réel, jamais sur un CFG construit à la main : la question est donc de savoir si un tel corps peut n'avoir aucune feuille atteignable. La réponse est non. `lower_while` (`src/lowering/cfg_builder.rs`, l. 603–632) ajoute *inconditionnellement* l'arête `IfFalse` vers le bloc de sortie de la boucle, que la condition soit ou non un littéral `true` ; la section 12.5 a montré que le raffinement de garde n'élague jamais une arête. Un `while (true)` lowé garde donc toujours une sortie structurellement atteignable, et le corps entier finit par un `Return` scellé par l'ADR-25. La vacuité de `on_all_paths` n'a donc pas d'incidence connue sur `infinite-loop` au commit `e67b10a` : elle reste une possibilité théorique du treillis, pas un cas observé, et un CFG où elle se manifesterait devrait être construit à la main comme celui de la sonde.

Sur la figure 12.3, la sonde donne : $\{1\}$ non (le chemin par B2 l'évite), $\{1,2\}$ oui, $\{3\}$ oui, $\{4\}$ non (on peut sortir sans entrer dans la boucle), $\{5\}$ oui. Sur le diamant seul (`src/engine/dominance.rs`, l. 192–258), $\{1\}$ non, $\{1,2\}$ oui, $\{3\}$ oui. On voit la différence avec la dominance : aucun bloc de $\{1,2\}$ ne domine B3, mais l'ensemble est sur tous les chemins.

`on_all_paths` n'est pas la seule implémentation du quantificateur « sur tous les chemins ». `must_setter_on_all_paths` (`src/rules/api/query.rs`, l. 527–622) porte sa propre analyse `must` en avant sur des booléens, et teste tous les blocs `Return` et `Unreachable`. La couche règles et sa façade typée `must_on_all_paths` (`src/rules/api/query.rs`, l. 624–640) sont présentées au chapitre 17. L'ADR-042 (§6) a déplacé `on_all_paths` des règles vers `src/engine/dominance.rs` : c'est une propriété du CFG, et le graphe de churn, qui est un produit du moteur, en a besoin.

12.9 Sonder après convergence : `eval_in_stores` et `ConvergedEval`

12.9.1 Le problème

Après le point fixe, les relations du moteur et les règles ont besoin de *valeurs* : que vaut la `dep_options` de cet effet ? l'argument de ce `setter` est-il un objet frais ? Elles ne relancent pas `analyze_cfg` : elles évaluent une expression dans un environnement donné (souvent `exit_env`), contre les stores du composant.

Définition 12.3: Sonde post-convergence

Une *sonde* est l'évaluation d'une expression de l'IR, dans un environnement donné, contre une *copie* des stores d'un composant, après son point fixe. Elle ne modifie ni les stores ni le tas du résultat.

12.9.2 Le cœur : `eval_in_stores`

Extrait 12.24 – `src/engine/eval.rs`, l. 31–46 — le cœur mécanique de toute sonde

```
31 pub fn eval_in_stores(
32     expr: &Expr,
33     env: &AbstractEnv<StateValue>,
34     component: ComponentId,
35     state: &StateStore<StateValue>,
36     memo: &MemoStore<StateValue>,
37     heap: &mut Heap,
38 ) -> StateValue {
39     let mut s = state.clone();
40     let mut m = memo.clone();
```

```

41     StateValueTransfer.eval_expr(
42         expr,
43         env,
44         &mut AnalysisCtx::null(component, &mut s, &mut m, heap),
45     )
46 }

```

La fonction clone l'état et les mémos, construit un AnalysisCtx nul (NullCtx, pas d'inter-composants) et appelle eval_expr. Le tas, lui, est emprunté mutablement : l'appelant passe un tas jetable, parce que l'évaluation *écrit* dans le tas (un ObjectLit sondé y crée une entrée). Le choix des stores est laissé à l'appelant, et la documentation (src/engine/eval.rs, l. 19–30) insiste : les stores convergés et un tas convergé, ou des stores vides pour une évaluation « au montage », ne sont pas interchangeables.

12.9.3 La couche commune : ConvergedEval

Extrait 12.25 – src/engine/eval.rs, l. 61–80 — le cas commun : stores et tas convergés

```

61 pub trait ConvergedEval {
62     /// A reusable evaluator holding one scratch heap. Use this wherever more
63     /// than one expression is probed – a loop over deps, over a path's
64     /// prefixes – so the clone happens once instead of once per call.
65     fn evaluator(&self) -> Eval<'_>;
66
67     /// One-shot probe. Same answer as [`Self::evaluator`], one clone.
68     fn eval_in(&self, env: &AbstractEnv<StateValue>, expr: &Expr) -> StateValue {
69         self.evaluator().at(env, expr)
70     }
71 }
72
73 impl ConvergedEval for AnalysisResult<StateValue> {
74     fn evaluator(&self) -> Eval<'_> {
75         Eval {
76             result: self,
77             heap: self.heap.clone(),
78         }
79     }
80 }

```

Extrait 12.26 – src/engine/eval.rs, l. 89–105 — un évaluateur réutilisable

```

89 pub struct Eval<'a> {
90     result: &'a AnalysisResult<StateValue>,
91     heap: Heap,
92 }
93
94 impl Eval<'_> {
95     pub fn at(&mut self, env: &AbstractEnv<StateValue>, expr: &Expr) -> StateValue {
96         eval_in_stores(
97             expr,
98             env,
99             self.result.component,
100             &self.result.state_store,
101             &self.result.memo_store,
102             &mut self.heap,
103         )
104     }
105 }

```


`result.evaluator()` rend un `Eval` qui possède *une* copie du tas convergé ; at évalue une expression contre l'état, les mémos et ce tas. `eval_in` est la version à un coup. Réutiliser le même `Eval` pour plusieurs sondes (toutes les deps d'un effet, par exemple `src/rules/impls/missing_deps.rs`, l. 155) évite un clone du tas par expression ; c'est sûr parce qu'un `ExprId` nomme un seul site d'allocation (définition 6.2, #134) : sonder deux fois le même site réécrit sa propre entrée, deux sites ne se télescopent pas.

Piège

L'histoire de cette couche est celle d'un faux négatif (#135). Le tas était un argument de l'appelant, au motif qu'un tas vide et un tas convergé étaient deux choix légitimes. Quatre sites sur six avaient choisi le tas vide, à tort : une lecture de membre (`obj.f`, `form.onSubmit`) ne se résout que par le tas, un tas vide répond $\top$, et $\top$ est le côté *silencieux* de presque tous les prédicats des règles (« je ne sais pas si c'est frais, je ne dis rien »). Le correctif a été central et non site par site : figer le cas commun dans `ConvergedEval` (`src/engine/eval.rs`, l. 48–60). Un site qui veut réellement des stores vides appelle `eval_in_stores` directement.

`eval_in_stores` vivait dans `rules/helpers` ; l'ADR-042 (§6) l'a descendue dans le moteur, parce que les relations dérivées après convergence (fraîcheur des écrivains, déclencheurs d'effets, graphe de churn) en ont besoin et ne peuvent dépendre de la couche règles. Le point fixe l'utilise lui-même pour `effect_triggers` (`src/engine/fixpoint.rs`, l. 647–667).

12.10 Terminaison et complexité

12.10.1 Pourquoi une passe termine

Invariant 12.2: Tout cycle porte un arc arrière

Dans un CFG produit par le lowering, tout cycle contient au moins une arête `EdgeKind::Back`. Le lowering marque ainsi la fin du corps d'un `while`, d'un `for`, d'un `for...of`, le retour d'un `do...while` et tout continue (définition 7.3 ; `src/lowering/cfg_builder.rs`, l. 350, `src/lowering/cfg_builder.rs`, l. 396, `src/lowering/cfg_builder.rs`, l. 629, `src/lowering/cfg_builder.rs`, l. 699, `src/lowering/cfg_builder.rs`, l. 768).

Théorème 12.1: Terminaison d'une passe

Sous l'invariant 12.2, `analyze_cfg` termine sur tout CFG fini.

Esquisse. Chaque entrée $In[s]$ ne fait que croître : elle est remplacée par un `join` ou un `widening` d'elle-même, tous deux au-dessus d'elle (`widen_to` $\sqsupseteq$ `join`, théorème 9.2). Un bloc n'est enfilé que si son entrée change strictement. Il suffit donc de montrer que chaque entrée ne change qu'un nombre fini de fois.

- Dans une valeur `StateValue`, seule la composante numérique est de hauteur infinie : booléens, nullité, stabilité, setters et `StrConst` sont de hauteur finie (chapitres 9 et 10). Une variable qui apparaît ou disparaît d'un côté passe à $\top$, qui est le sommet. Le nombre de variables est borné par le CFG.
- Sur l'entrée d'un en-tête, à partir de la `widen_threshold`-ième arrivée par un arc arrière, la composante numérique passe par `widen_to`, qui ne peut changer une borne qu'un nombre fini de fois (T est fini, puis $\pm\infty$).
- Un bloc qui n'est pas un en-tête reçoit un `join` d'un nombre fini de prédécesseurs. Retirer les arcs arrière rend le graphe acyclique (invariant) : par récurrence dans l'ordre topologique de ce graphe acyclique, une fois que les en-têtes ne changent plus, chaque bloc ne reçoit

plus qu'un nombre fini de valeurs différentes.

Le nombre total de changements est donc fini, et la file finit par se vider. $\square$

Il faut remarquer que les arrivées par un arc *non* arrière sur un en-tête (depuis le pré-en-tête) sont jointes, jamais widenées. C'est suffisant, parce que le pré-en-tête est hors de la boucle : dans une boucle imbriquée, l'entrée qu'il transmet ne croît qu'autant que la boucle extérieure, elle-même bornée par son propre arc arrière.

Piège

L'invariant est fragile, et sa violation ne se voit pas : un cycle dont aucune arête n'est marquée Back n'est jamais widené. Le cas s'est produit : un continue émis avec une arête Unconditional. Le commentaire de la fixture qui l'épingle le raconte : « Marked Unconditional, the join grew i by one per pass and the fixpoint never terminated » (tests/fixtures/widening.tsx, l. 51–54). Toute modification du lowering qui crée un cycle doit marquer son arête de retour ; toute greffe (chapitre 8) doit conserver la nature des arêtes.

12.10.2 Complexité

Soient N le nombre de blocs, E le nombre d'arêtes, h le nombre maximal de changements d'une entrée (borné par la hauteur des domaines après widening). Le tableau 12.4 résume les coûts, tels qu'ils se lisent dans le code.

fonction	coût	d'où il vient
<code>analyze_cfg</code>	$O(h \cdot N)$ visites ; chaque visite : transfert, clone de l'entrée et des mémos, $O(E)$ par successeur	<code>is_back</code> parcourt <code>cfg.edges</code> pour chaque successeur (lignes 92 à 95)
<code>compute_dominators</code>	$O(R \cdot N \cdot (E + N))$ pour R tours	<code>predecessors</code> en $O(E)$, intersections d'ensembles en $O(N)$
<code>dominates</code> (méthode)	$O(1)$ attendu	test d'appartenance dans un <code>HashSet</code>
<code>dominates</code> (libre)	un <code>compute_dominators</code> par appel	à éviter en boucle
<code>on_all_paths</code>	$O(N \cdot E)$	chaque bloc visité appelle <code>successors</code> en $O(E)$
<code>entry_env_of</code>	$O(E)$ + un outgoing par prédécesseur	filtre des arêtes entrantes

TABLE 12.4 – Coûts des fonctions du chapitre.

Aucun de ces coûts ne pose problème sur des corps de composants, qui ont typiquement quelques dizaines de blocs ; le seul piège documenté est la reconstruction de la dominance par question, qui a motivé `DominatorTree` (« the fix for `dominates()` driving per-rule quadratic recomputation », `src/engine/dominance.rs`, l. 94–95). Pour le nombre de tours R de `compute_dominators`, le parcours en RPO fait que, sur un CFG réductible, quelques tours suffisent ; l'exemple de la figure 12.3 en fait deux.

12.10.3 Deux interpréteurs de CFG

Remarque

`analyze_cfg` n'est pas le seul interpréteur de CFG du projet. `exec_body_impl` (`src/domains/interp/interpreter.rs`, chapitre 11) exécute les corps de fonctions imbriquées (callbacks, `.then`, `updaters`, initialiseurs) en une seule passe topologique, arcs arrière ignorés et sans widening : une valeur portée par une boucle n'y est vue qu'à sa première itération.

Les deux issues ouvertes #12 (« Two CFG interpreters of different strength, and nothing says which ran ») et #21 (faux négatif sur les valeurs portées par une boucle dans un callback) suivent cette asymétrie. Le handler de `onClick` passe par `analyze_cfg`; la flèche passée à `setTimeout` dans un effet passe par `exec_body_impl`.

Résumé du chapitre

- `analyze_cfg` (`src/engine/cfg_analyzer.rs`) interprète un CFG par worklist FIFO : environnements d'entrée et de sortie par bloc, un seul store d'état pour toute la passe (insensible au flot, mis à jour faiblement), un tas partagé entre passes, des mémos figés. Elle rend les environnements de *sortie* (malgré la documentation de `BlockEnvs`) et le store enrichi.
- Fusion : première arrivée, `join`, et `widen_to` à seuils sur un arc `Back` dès la `widen_threshold`-ième arrivée (3), compteur par en-tête local à la passe. Les seuils sont les littéraux numériques du composant. Le narrowing descendant de l'ADR-014 n'existe pas : le widening à seuils sur l'arc interne suffit ($[0,5]$ en tête, $[5,5]$ en sortie de la boucle `i < 5`).
- Les effets de bord d'un `Return` sont exécutés (`exec_expr_effects`) : une flèche concise écrit.
- `narrow_env_for_branch` raffine une variable comparée à un littéral, à `null/undefined`, ou testée pour sa vérité ; tout autre motif ne raffine rien. Une garde réfutée n'élimine pas sa branche (FP de `Dead`).
- Une passe finie est un post-point fixe des équations de blocs ; `entry_env_of` relit l'entrée d'un bloc par un `join` jamais `widené` ; `exit_env` joint les sorties des `Return` atteints (par `reduce` : la table vide n'est pas neutre).
- `compute_dominators` calcule les ensembles de dominateurs en RPO ; `DominatorTree` les mémorise ; `ExitDominance` ne compte que les `Return` atteignables (ADR-025) ; `on_all_paths` cherche un chemin qui évite un ensemble et atteint une feuille, `throw` compris.
- `eval_in_stores` est le cœur de toute sonde post-convergence ; `ConvergedEval/Eval` en figent le cas commun avec le tas convergé (#135).
- Fichiers : `src/engine/cfg_analyzer.rs`, `src/engine/dominance.rs`, `src/engine/eval.rs`, et dans `src/engine/fixpoint.rs` `exit_env` et `collect_thresholds`.

Le chapitre 13 enchaîne ces passes : rendu, mémos, effets, handlers, test de convergence et widening de l'état, puis les passes d'après convergence.

Exercices

Exercice 12.1: Dominateurs d'un losange

Soit le losange du test `diamond_cfg` (`src/engine/dominance.rs`, l. 192–258) : B0 se branche vers B1 et B2, qui sautent tous deux vers B3, terminé par `return`.

1. Donner un RPO possible, puis dérouler `compute_dominators` tour par tour.
2. Que rendent `dominates(1, 3)`, `on_all_paths` sur $\{1\}$ et `on_all_paths` sur $\{1, 2\}$?
3. On ajoute un bloc B4 terminé par `return`, sans arête entrante. Que devient `Dom(4)` ? `ExitDominance` compte-t-il B4 parmi les sorties ?

Solution

(1) Le DFS visite B1 puis B3, puis B2 : post-ordre $[3, 1, 2, 0]$, RPO $[0, 2, 1, 3]$. Premier tour : $\text{Dom}(2) = \{0, 2\}$, $\text{Dom}(1) = \{0, 1\}$, $\text{Dom}(3) = \{3\} \cup (\{0, 1\} \cap \{0, 2\}) = \{0, 3\}$. Second tour : rien ne change. (2) Faux (test `diamond_branch_does_not_dominates_join`) ; faux ; vrai (valeurs relevées à la sonde). (3) B4 n'est pas dans le RPO et garde tous les blocs, $\{0, 1, 2, 3, 4\}$;

ExitDominance l'écarte car il n'est pas dans `reachable_blocks`.

Exercice 12.2: Une boucle avec des seuils

Dérouler `analyze_cfg` sur l'IR de `let i = 0; while (i < 3) i = i + 2;` avec `widen_threshold = 3` et $T = \{0, 2, 3\}$ (les littéraux du programme). Quelles sont les valeurs finales de `i` en tête et en sortie de boucle? Comparer à la valeur concrète.

Solution

B0 installe $In[B1] = [0, 0]$. B1 passe $[0, 0]$ au corps et un intervalle vide à la sortie. Le corps renvoie $[2, 2]$; premier retour, join : $In[B1] = [0, 2]$. B1 passe au corps $[0, 2]$ (raffiné par $i < 3$, soit $i \leq 2$); le corps renvoie $[2, 4]$; deuxième retour, join : $In[B1] = [0, 4]$. B1 raffine $[0, 4]$ en $[0, 2]$ pour le corps, *égal* à son entrée : le corps n'est pas ré-enfilé, et il n'y aura pas de troisième retour. La sortie reçoit $[3, 4]$. Résultat : $[0, 4]$ en tête, $[3, 4]$ en sortie, sans aucun widening (le compteur de B1 s'arrête à 2). La valeur concrète en sortie est 4, contenue dans $[3, 4]$. Ce déroulé n'a pas été exécuté; il se relit directement sur `add` (`src/domains/impls/interval.rs`, l. 148–156), `hull` (`src/domains/impls/interval.rs`, l. 71–78) et `narrow_lt/narrow_geq` (`src/domains/impls/interval.rs`, l. 262–292).

Exercice 12.3: Lire un raffinement

Pour chaque condition, dire ce que reçoivent les deux branches si `x` vaut « nombre $[0, 10]$ ou `null` » : `x < 5`, `5 > x`, `x != null`, `!x`, `x === 0`.

Solution

`x < 5` : vrai $[0, 4]$ ou `null` (entier; `null` gardé car la comparaison le convertit), faux $[5, 10]$ ou `null`. `5 > x` : motif non reconnu, les deux branches reçoivent l'environnement inchangé. `x != null` : vrai $[0, 10]$ (`null` retiré), faux : `narrow_keep_nullish_only`, `null` seul. `!x` : vrai `narrow_falsy`, $[0, 0]$ ou `null`; faux `narrow_truthy`, `null` retiré et `narrow_neq(0)` sur $[0, 10]$, qui ne rend $\perp$ que sur le point $[0, 0]$ et laisse tout autre intervalle intact (`src/domains/impls/interval.rs`, l. 303–309) : $[0, 10]$. `x === 0` : `narrow_eq(0)`, $[0, 0]$ ou `null`; faux `narrow_neq(0)`, $[0, 10]$ ou `null`.

Exercice 12.4: Pourquoi `!=` suffit

Montrer que, dans `analyze_cfg`, la nouvelle entrée ν d'un bloc vérifie toujours $\nu \sqsupseteq In[s]$, et en déduire que le test `!=` (ligne 114) équivaut à $\nu \not\sqsubseteq In[s]$. Quelle hypothèse sur l'égalité de `AbstractEnv` ce raisonnement demande-t-il, et la remarque sur les intervalles vides de la section 12.3 la contredit-elle?

Solution

ν est $In[s] \sqcup \rho$ ou $In[s] \nabla_T \rho$, tous deux $\sqsupseteq In[s]$; pour $\nu \sqsupseteq In[s]$, $\nu \sqsubseteq In[s]$ équivaut à l'égalité dans l'ordre. Le raisonnement demande que l'égalité structurelle coïncide avec l'égalité d'ordre. Les intervalles vides la contredisent : deux représentations de $\perp$ sont égales pour l'ordre et différentes pour `==`. La conséquence n'est qu'un ré-enfilage superflu (le test voit un changement qui n'en est pas un), jamais un changement manqué : la terminaison n'est pas menacée, car ces représentations suivent l'entrée de l'en-tête, qui se stabilise.

Exercice 12.5: Une garde morte

Sur liste 12.16, expliquer pourquoi le raffinement $k := \perp$ n'empêche pas l'écriture. Proposer un test qui ferait de la branche une branche morte, dire où le placer pour qu'il profite à toutes les passes et à `entry_env_of`, et énumérer les composantes de `StateValue` pour lesquelles il

faut vérifier que $\perp$ signifie « aucune valeur concrète ».

Solution

Le raffinement ne porte que sur k ; n garde sa valeur et $\text{setN}(n + 1)$ écrit. Un test naturel serait dans `outgoing` : ne pas rendre un successeur dont l'environnement raffiné lie une variable à une valeur entièrement $\perp$, ce qui profite à `analyze_cfg` et à `entry_env_of`, qui partagent `outgoing`. Il faut vérifier toutes les composantes du produit (`num`, `boolean`, `str`, `reference`, `null`, `undef`, `setter`, `other`), et en particulier que `other`, jamais retiré par les raffinements, soit faux. Ce n'est pas fait au commit e67b10a.

Chapitre 13

Le point fixe d'un composant

Objectifs du chapitre.

- Comprendre ce que calcule `analyze_component_impl` : non pas une simulation de React, mais un sur-ensemble des états qu'un composant peut atteindre, sous la forme d'un post-point fixe du `StateStore`.
- Connaître la préparation d'un composant (constantes de module, inlining des utilitaires et des hooks personnalisés, seuils de widening, amorçage des `useState`) et ce qu'elle laisse dans les CFG.
- Savoir dérouler un tour de la boucle externe — rendu, mémos, effets, handlers, join, test — et expliquer chaque choix par un fait de React : batching, fermetures, deps, événements.
- Maîtriser la convergence : le test $N \sqsubseteq S$, le compteur global, le widening à seuils, `widen_trace`, le garde-fou à cent tours, et pourquoi tout cela termine.
- Connaître les deux passes post-convergence (rafraîchissement du rendu, `effect_setter_writes`) et la tranche où naissent les relations.
- Situer la boucle par rapport à React-tRace, en donner l'argument de soundness, et connaître, un par un, les trous observés au commit e67b10a.

Prérequis : chapitre 2 (le tour de React, React-tRace et sa lecture réduite de la boucle) ; chapitre 3 (définition 3.5, définition 3.6, définition 3.8) ; chapitre 10 (définition 10.3) ; chapitre 11 (environnements, définition 11.2, `exec_setter_call`, `recompute_memo`) ; chapitre 12 (`analyze_cfg`, sa passe et la monotonie du store qu'elle rend) ; chapitre 8 pour la mécanique des greffes.

Le chapitre 12 sait exécuter abstraitement *un* corps : le rendu, un effet, un handler. Mais un composant React n'est pas un corps, c'est un cycle : le rendu produit un environnement, les effets le lisent et écrivent l'état, l'état modifié provoque un nouveau rendu, et ainsi de suite jusqu'à ce que plus rien ne change — ou indéfiniment, ce qui est précisément l'un des défauts à détecter. Ce chapitre décrit la fonction qui referme ce cycle, `analyze_component_impl` (`src/engine/fixpoint.rs`, l. 130–698), et tout ce qu'elle appelle. C'est la pièce centrale du moteur : elle reçoit un `ComponentIR` et rend un `AnalysisResult`, l'objet que toutes les règles lisent.

Le chapitre 2 en a donné, sous forme d'algorithme, une lecture réduite, suffisante pour la correspondance avec React-tRace. Ici nous la reprenons à la taille du code, ligne à ligne : ce qui se passe une fois avant la boucle, ce que fait chacune des quatre passes d'un tour, comment on décide qu'on a fini, ce qu'on recalcule ensuite et pourquoi. La progression va du composant le plus simple (un `useState`, un effet) aux cas qui ont imposé les choix les moins évidents : handlers dans la boucle, mémos hors du test de convergence, ré-exécution des effets depuis $\perp$. Le chapitre se termine par un inventaire honnête de ce que le moteur garantit et de ce qu'il ne garantit pas, au commit e67b10a.

Toutes les sorties d'analyseur ont été observées avec le binaire construit à ce commit sur des fichiers placés sous `/tmp/ch13/`, par `reactant check -verbose -info -no-color -project plain <fichier>` (la ligne `[verbose] X: N iteration(s), widened: [...] vient de src/driver/mod.rs,`

l. 419–428 et affiche `iterations` et les clés de `widen_trace`). Les valeurs internes des stores, invisibles depuis la ligne de commande, ont été lues avec une petite sonde Rust hors dépôt qui lie la crate `reactant` par chemin et imprime les champs de `l'AnalysisResult`; sa notation est `s0=num[0,3]` pour « le slot 0 vaut l'intervalle $[0, 3]$ », `ref:PerRender` pour une référence fraîche à chaque rendu, `⊤` pour `StateValue::top()`.

13.1 Le problème : un ensemble d'états, pas une simulation

13.1.1 Un compteur qui ne s'arrête jamais

Le composant le plus court qui boucle à l'infini est celui-ci :

Extrait 13.1 – La boucle infinie canonique (`/tmp/ch13/runaway.tsx`)

```
1 import { useState, useEffect } from "react";
2
3 export function Runaway() {
4   const [x, setX] = useState(0);
5   useEffect(() => setX(x + 1));
6   return <div>{x}</div>;
7 }
```

Concrètement (un tour de React au sens du chapitre 2) : React rend `Runaway` avec $x = 0$, commit, exécute l'effet, qui appelle `setX(1)`; l'état ayant changé, React rend à nouveau avec $x = 1$, l'effet tourne encore (il n'a pas de tableau de deps), `setX(2)`, et ainsi de suite. La suite des valeurs de x est $0, 1, 2, 3, \dots$; le composant ne se stabilise jamais. L'analyseur le dit :

```
$ reactant check --verbose --info --no-color --project plain runaway.tsx
[verbose] symbol graph: 1 nodes, topo order = [Runaway@runaway.tsx]
[verbose] 1 components analyzed
[verbose] cache hits: 0, misses: 0
[verbose] Runaway: 3 iteration(s), widened: [0]
Runaway (2 hooks) runaway.tsx
  warn  infinite-loop [hook:0] (line 5:2) this effect keeps pushing state `x` to new values on every run. Potential
    ↳ infinite render loop
      (2 trace step(s), rerun with --trace)
  info  widening-info (line 5:2) state `x` kept changing during analysis and was approximated to converge, so
    ↳ findings that depend on it may be imprecise
      (2 trace step(s), rerun with --trace)
  verified conditional-hook all hooks run unconditionally, in a stable order
  [... six autres lignes verified ...]

! 1 warning(s) across 1 file(s).
```

Comment un programme qui ne fait que *trois* itérations peut-il conclure quelque chose sur une exécution qui n'en finit pas ? La réponse est le sujet du chapitre, mais l'idée tient en deux phrases. Le moteur ne suit pas la suite $0, 1, 2, \dots$; il calcule, pour le slot de x , un *ensemble* de valeurs qui contient toutes celles que x peut prendre, à n'importe quel tour. Cet ensemble est représenté par une valeur abstraite (ici un intervalle), il grossit tant que l'effet écrit quelque chose de neuf, et quand il grossit trop longtemps, un opérateur de widening (définition 3.6) le fait sauter à $[0, +\infty]$: « x peut prendre n'importe quelle valeur ≥ 0 ». Ce saut est le signal de divergence que la règle `infinite-loop` lit.

13.1.2 Pourquoi pas simuler

On pourrait vouloir exécuter le composant : rendre, appliquer les effets, rendre à nouveau, et déclarer une boucle infinie au bout de, disons, cent tours. Trois raisons rendent cette voie

impraticable pour un analyseur statique.

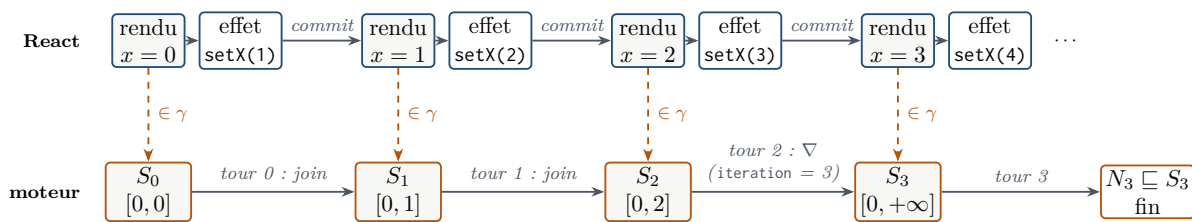
1. *Les entrées sont inconnues.* Un composant reçoit des props, lit des contextes, appelle des bibliothèques. Aucune valeur concrète n'est disponible ; il faut raisonner sur *toutes* les valeurs possibles à la fois, c'est-à-dire sur des ensembles.
2. *Les événements sont arbitraires.* Un `onClick` s'exécute zéro, une ou mille fois, dans n'importe quel ordre par rapport aux autres handlers. Aucune suite finie de tours ne représente toutes les exécutions.
3. *La terminaison n'est pas décidable*, et surtout, on veut une réponse *sound* (définition 3.8) : si l'analyseur dit « rien à signaler », aucune exécution concrète ne doit boucler. Une simulation bornée peut manquer une boucle qui n'apparaît qu'à la cent-unième itération ou qu'avec une entrée particulière.

L'interprétation abstraite répond aux trois points d'un coup : on remplace chaque valeur par un élément d'un treillis qui représente un ensemble de valeurs, on remplace la suite des tours par une suite *croissante* d'états abstraits, et on s'arrête à un post-point fixe (définition 3.5) qui, par le théorème du post-point fixe du chapitre 3, contient tout état concret atteignable.

Définition 13.1: État abstrait d'un composant, tour abstrait

L'*état abstrait* d'un composant est son `StateStore` (définition 11.2) : une fonction S des labels de ses hooks `useState` vers des `StateValue`, un label absent valant $\perp$. Un *tour abstrait* est une exécution de la boucle de `analyze_component_impl` : une passe de rendu depuis S , le recalcul des mémos, une passe par effet et une passe par handler, toutes depuis S , puis le join N des stores produits. La *suite des tours* est $S_0 \sqsubseteq S_1 \sqsubseteq \dots$, où S_0 est le store amorcé par les initialiseurs et S_{k+1} vaut N_k ou $S_k \nabla N_k$. La boucle s'arrête au premier k tel que $N_k \sqsubseteq S_k$; ce S_k est l'état abstrait convergé.

La figure 13.1 met en regard le temps de React et celui de l'analyseur pour Runaway. Le point important est que le tour abstrait k ne représente pas le k -ième tour de React : il représente *tous* les états atteignables en au plus k tours, et un tour de plus, quand il n'ajoute rien, prouve que l'ensemble est clos.



Chaque rendu concret de React lit une valeur de x contenue dans la concrétisation $\gamma(S_k)$ du store abstrait de même rang ; le store S_3 contient aussi tous les rendus suivants.

FIGURE 13.1 – Chronogramme de Runaway : les tours de React (en haut, infinis) et les tours de la boucle externe (en bas, quatre). Le store abstrait S_k sur-approxime toutes les valeurs de x atteignables en au plus k tours ; après le widening, S_3 les contient toutes.

13.1.3 Ce qu'un ensemble ne dit pas

Le prix de cette abstraction est visible sur un second exemple. Ici l'état est un objet, recréé par l'effet qui en dépend :

Extrait 13.2 – Un objet recréé dans ses propres deps (/tmp/ch13/fresh.tsx)


```

1 import { useState, useEffect } from "react";
2
3 export function Fresh() {
4   const [o, setO] = useState({ n: 0 });
5   useEffect(() => {
6     setO({ n: o.n + 1 });
7   }, [o]);
8   return <div>{o.n}</div>;
9 }

```

Concrètement, c'est une boucle infinie : chaque exécution de l'effet stocke un objet neuf, `Object.is` échoue sur le dep `[o]`, l'effet tourne encore. Abstraitemment, la valeur du slot est `ref:PerRender` (définition 10.3 : « une référence fraîche à chaque rendu ») dès l'amorçage, l'effet écrit `ref:PerRender`, et $PerRender \sqcup PerRender = PerRender$: le store ne bouge pas, la boucle s'arrête *au premier tour*, sans widening :

```

[verbose] Fresh: 0 iteration(s), widened: []
Fresh (2 hooks) fresh.tsx
error infinite-loop [hook:0] (line 5:2) this effect recreates object state `o` it depends on. Every run stores a
  ↳ fresh reference (`Object.is` always fails) and re-triggers itself: infinite render loop

```

L'ERROR est pourtant là, et c'est une ERROR, pas un WARNING : elle ne vient pas du point fixe mais des *relations* dérivées après lui (chapitre 14, chapitre 15) — ici la ligne `exact = true` de `effect_triggers` et la fraîcheur de l'écriture dans `slot_writers`. La leçon, que le chapitre 14 reprend à son compte (ADR-42), est celle-ci : le point fixe calcule un *ensemble de valeurs*, jamais une *suite de transitions*. La suite $0 \rightarrow 1 \rightarrow 0 \rightarrow 1$ et le fait « atteint $\{0, 1\}$ » ont exactement la même abstraction. Une question qui porte sur les transitions (« cette écriture relance-t-elle cet effet ? ») n'a pas de réponse dans le store ; elle en a une dans les relations, calculées à convergence sur les CFG. Ce chapitre s'arrête donc à la frontière : il montre comment le store converge et quelles données sont figées pour les relations, et laisse leur calcul aux chapitres suivants.

13.1.4 Les quatre portes d'entrée et la configuration

Le fichier `src/engine/fixpoint.rs` (2815 lignes, dont un millier de tests) expose quatre fonctions publiques, réexportées par `src/engine/mod.rs`, qui aboutissent toutes au cœur privé `analyze_component_impl`.

Extrait 13.3 – `src/engine/fixpoint.rs`, l. 83–118 — les deux entrées intra-composant

```

83 /// Public entry point intra-component analysis only (no inter-component context).
84 ///
85 /// The component is identified as [ComponentId::SYNTHETIC]: there is no
86 /// registry here to intern it, and a lone intra analysis compares its
87 /// component to nothing. Use [analyze_component_as`] when several such
88 /// results have to coexist in one program – they would otherwise all claim
89 /// the same identity.
90 pub fn analyze_component<T: Transfer<Domain = StateValue>>(
91   comp: ComponentIR,
92   transfer: &T,
93   config: &Config,
94 ) -> AnalysisResult<StateValue> {
95   analyze_component_as(comp, ComponentId::SYNTHETIC, transfer, config)
96 }
97
98 /// [analyze_component`] for a caller that has already interned the component.
99 ///
100 /// The id is what the result stamps into its own state labels and setter

```

```

101 /// owners, so a program built from several of these resolves each one's
102 /// provenance to the right component (#7).
103 pub fn analyze_component_as<T: Transfer<Domain = StateValue>>(<
104     comp: ComponentIR,
105     id: ComponentId,
106     transfer: &T,
107     config: &Config,
108 ) -> AnalysisResult<StateValue> {
109     analyze_component_impl(
110         comp,
111         id,
112         transfer,
113         config,
114         AbstractEnv::bottom(),
115         Heap::new(),
116         None,
117     )
118 }

```

- `analyze_program` (`src/engine/fixpoint.rs`, l. 704) est l'entrée de production, appelée par le driver : elle analyse les racines avec un `InterCtx` (phase 1), puis les composants non atteints sans (phase 2). Le section 13.5.4 en dit ce qui concerne la boucle ; le chapitre 16 fait le reste.
- `analyze_component` et `analyze_component_as` (liste 13.3) analysent un composant *isolé* : environnement d'entrée $\perp$ (toute variable inconnue vaut $\top$, chapitre 11), tas neuf, `inter = None`. La première l'identifie par `ComponentId::SYNTHETIC` (`src/ir/component_id.rs`, l. 38), la seconde par un identifiant déjà interné. Ce sont les entrées de la plupart des tests unitaires et de plusieurs tests d'intégration (`tests/widening_e2e.rs`, `tests/functional_updater.rs`).
- `analyze_component_inter` (`src/engine/fixpoint.rs`, l. 65–81) est la fonction de rappel que `eval_comp_app` appelle pour analyser un enfant rencontré dans un rendu ; elle existe pour casser la dépendance circulaire entre `domains::transfer` et `engine::fixpoint` (chapitre 11).

Piège

Une analyse *intra* (`analyze_component`) et une analyse *inter* (`analyze_program`) du même composant ne calculent pas la même chose. En *intra*, il n'y a pas de `HookRegistry` donc pas d'inlining des hooks personnalisés (section 13.2.2), pas de `SharedStateStore` (section 13.3.5), et un `SummaryRegistry` vide par défaut. Un test qui passe par `analyze_component` ne prouve rien du chemin suivi par la ligne de commande ; l'#14 relève que 36 des 42 fichiers de tests d'intégration construisent `Config::default()`, « which no user ever runs ». Nous en verrons une conséquence concrète au section 13.7 (l'updater fonctionnel).

Les paramètres du moteur tiennent dans une structure :

Extrait 13.4 – `src/engine/fixpoint.rs`, l. 38–60 — les paramètres du moteur et leurs défauts

```

38 pub struct Config {
39     pub widen_threshold: usize,
40     /// Known library hooks (TanStack, React Router, etc.) without source.
41     /// Used in `expand_custom_hooks` as a fallback when a hook is not in the `HookRegistry`.
42     pub summary_registry: SummaryRegistry,
43     /// Utility-function inlining registry. When non-empty, statement-level calls
44     /// to known utilities are spliced into the caller's CFG instead of opaque `Top`.
45     pub function_registry: FunctionRegistry,
46     /// Cap on transitive utility-inlining depth, to bound CFG growth when
47     /// utilities chain or recurse.
48     pub max_inline_depth: usize,
49 }

```

```

50
51 impl Default for Config {
52     fn default() -> Self {
53         Config {
54             widen_threshold: 3,
55             summary_registry: SummaryRegistry::new(),
56             function_registry: FunctionRegistry::new(),
57             max_inline_depth: 8,
58         }
59     }
60 }

```

`widen_threshold` sert *deux fois* : c’est le nombre de tours externes qui changent l’état avant que la boucle passe du join au widening (section 13.4), et c’est aussi le nombre de passages par un arc arrière avant le widening d’un en-tête de boucle dans `analyze_cfg` (chapitre 12). Les deux registres alimentent l’expansion (section 13.2.2) ; `max_inline_depth` borne le nombre de greffes d’utilitaires par CFG. Le driver n’utilise pas `Config::default()` tel quel : il remplace le `SummaryRegistry` vide par `SummaryRegistry::new_with_common()` (`src/driver/mod.rs`, l. 370–374), qui connaît les hooks des bibliothèques courantes.

La signature du cœur, avec le plan que son auteur en a écrit :

Extrait 13.5 — `src/engine/fixpoint.rs`, l. 120–138 — la doc d’en-tête et la signature de `analyze_component_impl`

```

120 /// Core fixpoint loop. Called by `analyze_component` and `analyze_component_inter`.
121 ///
122 /// Outer loop:
123 /// 1. Import cross-component state from `SharedStateStore` (if `inter` is set).
124 /// 2. Render pass: analyze `render_cfg`.
125 /// 3. Recompute memo store from exit env.
126 /// 4. Effect passes.
127 /// 5. Handler passes (in-cycle).
128 /// 6. Convergence check.
129 /// 7. Widen after `config.widen_threshold` iterations.
130 fn analyze_component_impl<T: Transfer<Domain = StateValue>>(<
131     comp: ComponentIR,
132     comp_id: ComponentId,
133     transfer: &T,
134     config: &Config,
135     initial_env: AbstractEnv<StateValue>,
136     initial_heap: Heap,
137     inter: Option<&InterCtx<'_>>,
138 ) -> AnalysisResult<StateValue> {

```

Un détail à corriger d’emblée : l’étape 1 du plan (« Import cross-component state ») n’a pas lieu en tête de boucle. L’import de la tranche du `SharedStateStore` se fait dans le test de convergence (`src/engine/fixpoint.rs`, l. 488–493), après les passes ; nous y reviendrons. Le reste du plan est fidèle, et la figure 13.2 le dessine en entier, préparation et post-convergence comprises. Les trois sections suivantes en parcourent les trois zones.

13.2 La préparation : ce qui se passe une fois

Avant le premier tour, `analyze_component_impl` transforme le `ComponentIR` qu’il reçoit (`src/engine/fixpoint.rs`, l. 139–353). Rien de ce qui suit n’est répété : les CFG sont modifiés en place par les greffes, les seuils sont récoltés une fois, l’état est amorcé une fois. La boucle n’a ensuite plus qu’à exécuter.

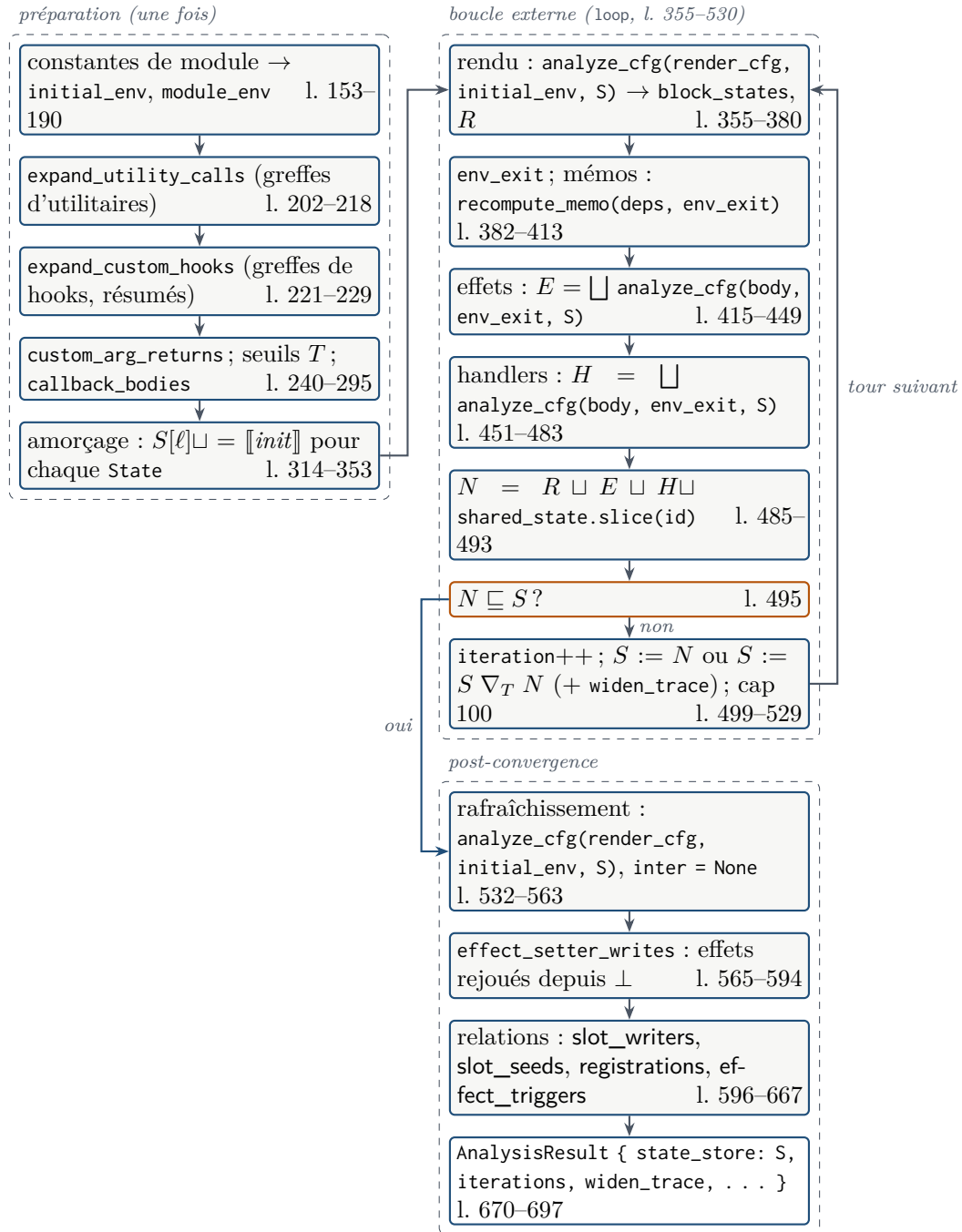


FIGURE 13.2 – Le flot de `analyze_component_impl` (`src/engine/fixpoint.rs`). Les numéros de ligne sont ceux du commit e67b10a. La préparation s'exécute une fois; la boucle externe répète les quatre passes jusqu'à $N \subseteq S$; la post-convergence recalcule ce que les règles liront.

13.2.1 Les constantes de module

Un composant lit souvent des constantes définies au niveau du module (`const LIMIT = 10;;`, `const EMPTY = [];`, `const Ctx = createContext(...)`). Sans traitement particulier, une lecture de `LIMIT` dans le rendu tomberait sur une variable absente de l’environnement, donc sur $\top$ (chapitre 11) — et une constante `EMPTY` passée en dep se lirait « peut changer à chaque rendu ».

Extrait 13.6 – `src/engine/fixpoint.rs`, l. 153–190 — les constantes de module amorcent deux environnements

```

153 // Seed module-level `const` bindings (TODO.md F7). A module const is
154 // evaluated once per module lifetime, so its value never changes across
155 // renders: a primitive literal seeds its exact value, a reference
156 // literal (`const D = {...}`) seeds a Stable reference. Without this,
157 // reads fall through to the env-miss default ( $\top$ ) and downstream rules
158 // see "may be fresh each render". Bindings already present in
159 // `initial_env` (props from a parent analysis) win.
160 let mut initial_env = initial_env;
161 // Module consts alone, in their own env: reused below as the entry env of
162 // custom-hook argument bodies – a module const has the same value at every
163 // program point, so it is the only binding that may be seeded there.
164 let module_env = {
165     let mut seed_state = StateStore::bottom();
166     let mut seed_memo: MemoStore<StateValue> = MemoStore::new();
167     let mut seed_heap = crate::domains::Heap::new();
168     let mut ac = AnalysisCtx::null(comp_id, &mut seed_state, &mut seed_memo, &mut seed_heap);
169     let empty_env = AbstractEnv::bottom();
170     let mut env = AbstractEnv::bottom();
171     for (name, init) in module_consts.iter() {
172         let val = match init {
173             ModuleConstInit::Prim(p) => {
174                 transfer.eval_expr(&Expr::Lit(p.clone()), &empty_env, &mut ac)
175             }
176             // A context object is a module-scoped reference like any
177             // literal – the role it also records is read by the rules layer.
178             ModuleConstInit::Ref | ModuleConstInit::Context(_) => {
179                 StateValue::reference(Stability::Stable)
180             }
181         };
182         // Bindings already present in `initial_env` (props from a parent
183         // analysis) win there; `module_env` keeps every const.
184         if !initial_env.contains(name) {
185             initial_env.extend(name.clone(), val.clone());
186         }
187         env.extend(name.clone(), val);
188     }
189     env
190 };

```

Le lowering a rangé ces constantes dans `comp.module_consts` (chapitre 6), sous la forme d’un `ModuleConstInit` : un littéral primitif (`Prim`), une référence (`Ref`) ou un objet de contexte (`Context`). La boucle les évalue une fois, contre des stores vides et un environnement vide, et les place dans deux environnements distincts :

- `initial_env`, l’environnement d’entrée du rendu, où une liaison déjà présente — une prop liée par le parent en analyse inter — l’emporte sur la constante de même nom ;
- `module_env`, qui ne contient *que* les constantes, et qui servira d’environnement d’entrée aux corps des arguments de hooks opaques (voir plus bas) : une constante de module a la même valeur en tout point du programme, c’est la seule liaison que l’on puisse y semer

sans risque.

Rappel React : Constantes de module

Un module ECMAScript est évalué une fois par vie du programme ; une `const` de module ne change donc jamais entre deux rendus, quel que soit le composant qui la lit. C'est ce qui autorise `Stability::Stable` pour une référence de module, alors qu'un littéral objet écrit *dans* le corps du composant vaut `PerRender` (définition 10.3). Un objet `createContext(...)` est une référence de module comme une autre ; le rôle `Context` qu'il porte en plus n'est lu que par les règles (`unstable-context-value`), pas par le moteur.

13.2.2 L'expansion : les utilitaires, puis les hooks personnalisés

Le composant que reçoit la boucle n'est pas toujours celui que le programmeur a écrit. Un `useTicker(5)` n'est pas un hook natif : l'IR le représente par une entrée `HookEntry::Custom` et un `HookMarker` au site d'appel, et rien dans le domaine ne sait ce qu'il fait. De même, un appel `helper(x)` ; en position d'instruction est un `Call` opaque, qui s'évalue à $\top$. La préparation remplace ce qu'elle peut par le corps correspondant, greffé dans le CFG de l'appelant — c'est l'*inlining* du chapitre 8, dont nous ne reprenons ici que ce que le point fixe en fait.

Extrait 13.7 – `src/engine/fixpoint.rs`, l. 192–229 — les deux expansions, dans cet ordre

```

192 // Provenance of every splice below (ADR-019) – ends up on the result.
193 let mut inline_origins: Vec<crate::engine::InlineOrigin> = Vec::new();
194 // Spliced-callee block ranges per CFG (ADR-027 §4): what decides whether
195 // a write site is caller-authored or wrapper-mediated.
196 let mut inline_regions = crate::engine::setters::InlineRegions::default();
197 // The identity supplies every splice in this component draws from.
198 let mut splice_salt = SpliceIds::for_component(&render_cfg, &hooks);
199
200 // Utility-function inlining. Runs before `expand_custom_hooks` so utility
201 // bodies containing hook calls become visible to the hook expansion pass.
202 if expand_utility_calls(
203     &mut render_cfg,
204     &mut hooks,
205     &config.function_registry,
206     &comp_file,
207     config.max_inline_depth,
208     &mut inline_origins,
209     &mut inline_regions,
210     &mut splice_salt,
211 ) && let Some(inter) = inter
212 {
213     inter
214         .stats
215         .borrow_mut()
216         .inline_budget_exhausted
217         .insert(comp_id);
218 }
219
220 // Expand Custom entries before seeding so inlined State entries are seeded.
221 expand_custom_hooks(
222     &mut hooks,
223     &mut hook_provenance,
224     &mut render_cfg,
225     inter,
226     &mut inline_origins,
227     &mut inline_regions,
228     &mut splice_salt,

```

```
229 );
```

L'ordre est voulu (commentaire des l. 200–201) : les utilitaires d'abord, pour qu'un hook appelé *dans* un utilitaire devienne visible à l'expansion des hooks ; les hooks ensuite, *avant* l'amorçage, pour que les `useState` d'un hook greffé soient amorcés comme les autres.

Un exemple. Le composant `Clock` appelle un hook personnalisé qui, à lui seul, boucle :

Extrait 13.8 – Un hook personnalisé qui boucle (`/tmp/ch13/ticker.tsx`)

```
1 import { useState, useEffect } from "react";
2
3 function useTicker(start: number) {
4   const [t, setT] = useState(start);
5   useEffect(() => {
6     setT(t + 1);
7   });
8   return t;
9 }
10
11 export function Clock() {
12   const t = useTicker(5);
13   return <div>{t}</div>;
14 }
```

La sonde montre les deux visages du même composant. En *intra* (`analyze_component`), l'entrée hook 0: Custom `useTicker` n'est pas expansée ; le store est vide et la boucle s'arrête au premier tour (`iterations = 0`). En *inter* (`analyze_program`, le chemin du CLI), l'entrée Custom de label 0 a été remplacée par un State de label 1 et un Effect de label 2, l'init `start` a été substitué par 5, et le point fixe trouve la divergence :

```
## engine result Clock
iterations = 3
state_store = s1=num[5,inf]
effect_setter_writes = s1=num[6,inf]
widen_trace = [(1, WidenEvent { iteration: 3, writers: [2] })]
```

```
[verbose] Clock: 3 iteration(s), widened: [1]
```

```
Clock (1 hooks) ticker.tsx
```

```
warn  infinite-loop [hook:1] (line 5:2) this effect keeps pushing state `t` to new values on every run. Potential
  ↳ infinite render loop
```

Le rapport dit (1 hooks) — le composant tel qu'écrit n'a qu'un appel de hook — mais désigne `[hook:1]`, le label du `useState` greffé, et pointe la ligne 5, dans le corps de `useTicker`. C'est la provenance (`inline_origins`, ADR-19) qui permet aux règles de retrouver ce fichier et cette ligne.

Ce que fait `expand_custom_hooks`. La fonction (`src/engine/fixpoint.rs`, l. 889–1142) parcourt hooks par indice et, pour chaque entrée Custom qu'elle sait résoudre, la remplace par les sous-entrées du hook. Les étapes, avec leurs lignes :

1. *Sortie immédiate* sans *inter* ou sans `HookRegistry` (l. 898–901) : c'est pourquoi l'analyse *intra* ne greffe rien.
2. *Garde de récursion par nom* (l. 927–931) : un ensemble *expanding* des noms déjà greffés ; une seconde occurrence du même nom est sautée et reste opaque.
3. *Résolution* (l. 933–942) : d'abord la clé (fichier résolu, nom), sinon le premier hook de ce nom, trié par clé (`get_by_name`). C'est plus permissif que la résolution des utilitaires,

- qui est fail-closed (voir plus bas).
4. *Repli sur un résumé* (l. 943–994) : un hook absent du registre mais connu du SummaryRegistry (TanStack, React Router...) n'est pas greffé ; son HookMarker est ré-étiqueté avec un SummaryValue (retag_marker), et l'entrée Custom est *conservée* pour que les règles des hooks voient toujours le site d'appel.
 5. *Décalage des labels* (l. 996–997) : offset est le premier label libre ; les sous-entrées du hook sont renumérotées à partir de lui, ce qui donne le label 1 au useState de Clock.
 6. *Substitution des paramètres dans les inits* (l. 999–1007) : les initialiseurs de useState sont évalués dans un environnement séparé qui ne voit pas les liaisons du rendu (section 13.2.5) ; useState(start) doit donc devenir useState(5) par substitution syntaxique.
 7. *Sel et identités* (l. 1012–1026) : salt.take(span) fournit un sel d'alpha-renommage (les locales du hook deviennent t#0, setT#0) et un décalage d'ExprId pour les sites d'allocation ; la table de renommage évite les liaisons de l'appelant (#141).
 8. *Grefe du corps entier* au site du HookMarker (l. 1043–1078), retour lié à la variable de l'appelant, et enregistrement de la région greffée (ADR-27 §4 : une écriture dans cette région est « via useTicker », pas directe). À défaut de marqueur, cas défensif, seules les instructions du bloc d'entrée sont greffées et la provenance du rendu est empoisonnée (render_poisoned, fail-closed, l. 1079–1109).
 9. *Provenance* (l. 1111–1130) : un InlineOrigin et les lignes HookProvenance du hook, décalées et marquées inlined: true (ADR-23).
 10. *Remplacement sans avancer* (l. 1132–1141) : l'entrée Custom est remplacée par les sous-entrées, et l'indice i n'est pas incrémenté, car la première sous-entrée peut elle-même être un Custom : l'expansion est récursive, bornée par la garde de récursion.

Les deux réserves d'identités que consomme chaque greffe vivent dans une petite structure :

Extrait 13.9 – src/engine/fixpoint.rs, l. 828–867 — les identités d'une greffe : sel et curseur d'ExprId

```

828 /// The identity supplies every splice in one component draws from: a salt that
829 /// alpha-renames the callee's locals, and a cursor over allocation-site ids.
830 ///
831 /// Both are monotonic for the same reason – the caller and the callee number
832 /// their own things from zero, and a graft has to land somewhere neither of
833 /// them already occupies. The salt has always been here; the id cursor is what
834 /// stops a callee's `ObjectLit` from sharing a heap entry with the caller's,
835 /// and stops the *same* callee inlined twice from being one allocation site
836 /// (#134).
837 pub(crate) struct SpliceIds {
838     salt: u32,
839     next_alloc: usize,
840 }
841
842 impl SpliceIds {
843     fn for_component(render_cfg: &CFG, hooks: &[HookEntry]) -> Self {
844         SpliceIds {
845             salt: 0,
846             next_alloc: alloc_span(render_cfg, hooks),
847         }
848     }
849
850     /// One allocation site, for something the source did not allocate – a
851     /// library hook's per-member summary shape (#94).
852     fn alloc_one(&mut self) -> crate::ir::types::ExprId {
853         let id = self.next_alloc;
854         self.next_alloc += 1;
855         crate::ir::types::ExprId(id)

```



```

856     }
857
858     /// The salt and allocation-id offset for one graft `span` ids wide,
859     /// advancing both supplies past it.
860     fn take(&mut self, span: usize) -> (u32, usize) {
861         let salt = self.salt;
862         self.salt += 1;
863         let ids = self.next_alloc;
864         self.next_alloc += span;
865         (salt, ids)
866     }
867 }

```

Invariant 13.1: Réserves monotones sur tout le composant

Le sel et le curseur d'allocation de `SpliceIds` ne font que croître pendant toute la préparation d'un composant, greffes d'utilitaires, greffes de hooks et formes de résumé (`alloc_one`) comprises. Deux greffes n'occupent donc jamais le même site d'allocation, et le même hook greffé deux fois ne partage pas une entrée du tas (#134).

Décision : ADR-005 — portée intra-procédurale et registre modulaire de hooks

Décision (ADR-5, 2026-05-29). Chaque composant est analysé indépendamment ; un hook inconnu rend `Unknown` ; un registre de modèles de hooks à quatre couches (natifs, bibliothèques, configuration utilisateur, repli) évite l'inlining ; une phase 2 ultérieure ajoutera un inlining *call-string-1*. *Ce qu'il en est au commit e67b10a*. L'inlining des hooks personnalisés existe (`expand_custom_hooks`), et il n'est pas limité à la profondeur 1 : il est récursif, gardé par nom. Le trait `HookModel` de l'ADR est devenu `HookSummary` (`src/registry/summary.rs`) et la configuration est `reactant.config.json`, pas `reactant.toml`. L'ADR n'a pas été amendé ; sa lecture doit se faire avec ces trois corrections.

Les utilitaires. `expand_utility_calls` (`src/engine/fixpoint.rs`, l. 1554–1602) parcourt le rendu puis chaque corps d'Effect, Memo, Callback et Handler — pas les inits de State ni les arguments des Custom — et appelle `inline_in_cfg` sur chacun :

Extrait 13.10 – `src/engine/fixpoint.rs`, l. 1611–1634 — la boucle de greffes d'un CFG et son budget

```

1611 fn inline_in_cfg(
1612     cfg: &mut CFG,
1613     ctx: &mut InlineCtx<'_,
1614     regions: &mut Vec<crate::engine::setters::InlineRegion>,
1615     expanding: &mut HashSet<String>,
1616 ) {
1617     let mut budget = ctx.max_depth;
1618     loop {
1619         let target = find_inlining_target(cfg, ctx.registry, ctx.caller_file, regions, expanding);
1620         if budget == 0 {
1621             // Exhausting the budget leaves the remaining calls opaque (T) -
1622             // sound, but a truncation, and one that measurably happens (20 CFGs
1623             // in the excalidraw corpus, 6 in memos). Record it so the component
1624             // withholds its assurances instead of publishing `verified:` over
1625             // utility bodies the analysis never read.
1626             ctx.truncated |= target.is_some();
1627             break;
1628         }
1629         let Some((block_id, stmt_idx, name)) = target else {
1630             break;

```

```

1631     };
1632     // Mark before splicing so a self-recursive call inside the spliced
1633     // body is skipped on the next scan.
1634     expanding.insert(name.clone());

```

Trois propriétés à retenir. La cible est une *instruction* (`let x = util(...)` ou `util(...);`, `utility_call_target`, l. 1698–1718) : un appel en position d'expression reste opaque (#52). La résolution du nom est fail-closed (`FunctionRegistry::resolve`, `src/engine/function_registry.rs`, l. 51–63) : une définition dans le fichier de la région où se trouve l'appel, sinon l'arête d'import résolue au lowering pour ce nom local, jamais une devinette par nom à travers les fichiers. Enfin le budget `max_inline_depth` est *par CFG* (`let mut budget = ctx.max_depth` au début de chaque `inline_in_cfg`), décrémenté même quand la greffe échoue, et chaque nom n'est greffé qu'une fois par CFG (`expanding`, neuf pour chaque CFG) : un second appel du même utilitaire dans le même corps reste un `Call` opaque.

Piège

Le drapeau `truncated`, qui signale qu'un appel est resté opaque faute de budget, n'est enregistré que sous `Some(inter)` (liste 13.7, l. 211) : il vit dans `AnalysisStats`, accessible seulement par `InterCtx`. En phase 2 — et en intra — il est jeté. Nous l'avons vérifié avec un composant A qui appelle dix utilitaires `u1(1); ... u10(1);` (budget 8), dans deux versions. Quand A est une racine (il rend un `<div>`), le CLI émet `info analysis-limit utility inlining ran out of splice budget here, so the remaining utility calls are treated as unknown (FN possible) [...] puis suspended analysis-limit 9 passing check(s) withheld`. Quand A et B se rendent mutuellement — aucune racine, tous deux en phase 2 —, la troncature a lieu de même, mais le CLI publie pour A neuf lignes `verified` et aucune `Info`. Les assurances couvrent alors du code que l'analyse n'a pas lu, contrairement à l'intention écrite au commentaire de `inline_in_cfg` (l. 1621–1625). Ce n'est pas une sous-approximation du store (les appels opaques valent `⊤`), mais un défaut de la couche de certification ; aucune issue ne le décrit au commit `e67b10a`. Le correctif naturel, au niveau central, serait de porter le drapeau sur `AnalysisResult` plutôt que sur les statistiques `inter`.

Piège

La garde de récursion par *nom* coupe aussi le second appel légitime d'un même hook. Avec `const a = useTicker(5); const b = useTicker(9);`, seul le premier est greffé ; pour le second le CLI émet `info analysis-limit [hook:1] (line 13:8) hook 'useTicker' was not found in the registry. Pass its source file or add a HookSummary to analyse it (FN possible)` puis suspend sept assurances. Le comportement est *sound* (le second appel vaut `⊤` et le composant retient ses assurances), mais le message est trompeur : le hook *est* dans le registre. Le commentaire de la garde (l. 884–888) assume ce cas comme « rare » et « FN, not FP » ; c'est un compromis, pas un défaut, au sens de `docs/limitations.md`.

Les arguments des hooks restés opaques. Un `useStore(s => s.count)` qui n'a été ni greffé ni résumé garde son entrée `Custom`. Le moteur exécute tout de même le corps de ses arguments `FnLit` (ou d'une variable dont la liaison à un littéral de fonction est certifiée syntaxiquement, `certified_fn_binding`), avec ses paramètres liés à `⊤` sur `module_env`, et range la valeur de retour jointe dans `custom_arg_returns`, clé (`label`, `indice`) (`src/engine/fixpoint.rs`, l. 231–277). L'argument de *soundness* tient en une phrase du commentaire : une capture non constante lit `⊤`, qui sur-approxime sa valeur en tout point du programme, donc quel que soit le moment où la bibliothèque appelle le sélecteur, la valeur stockée le couvre. Ce champ sert au vocabulaire déclaratif (ADR-23 §3, chapitre 22).

13.2.3 Les seuils de widening

Le widening des intervalles saute à $\pm\infty$ dès qu'une borne bouge ; avec lui, le compteur gardé if (`count < 10`) `setCount(count + 1)` vaudrait $[0, +\infty]$ alors que la garde le borne à 10. Le *widening à seuils* (définition 3.6) saute à la place au plus petit seuil qui englobe la nouvelle borne, et à $\pm\infty$ seulement au-delà du dernier seuil. Les seuils sont récoltés une fois, après expansion :

Extrait 13.11 – `src/engine/fixpoint.rs`, l. 1185–1224 — la récolte des seuils : tous les littéraux numériques, y compris dans les fermetures

```
1185 /// Harvest the finite threshold set for "widening up-to" (see ADR-014).
1186 ///
1187 /// Collects numeric literals from the render CFG, all hook bodies, and useState
1188 /// init expressions – the constants against which guarded state growth is
1189 /// bounded. Over-collecting is harmless: the set stays finite (termination) and
1190 /// extra thresholds only add candidate bounds (precision, never unsoundness).
1191 fn collect_thresholds(render_cfg: &CFG, hooks: &[HookEntry]) -> Vec<f64> {
1192     let mut out: Vec<f64> = Vec::new();
1193     collect_lits_cfg(render_cfg, &mut out);
1194     for hook in hooks {
1195         match hook {
1196             HookEntry::State { init, .. } => collect_lits_expr(init, &mut out),
1197             HookEntry::Effect { body_cfg, .. } | HookEntry::Handler { body_cfg, .. } => {
1198                 collect_lits_cfg(body_cfg, &mut out)
1199             }
1200         }
1201     }
1202     out.sort_by(|a, b| a.partial_cmp(b).unwrap_or(std::cmp::Ordering::Equal));
1203     out.dedup();
1204     out
1205 }
1206
1207 fn collect_lits_cfg(cfg: &CFG, out: &mut Vec<f64>) {
1208     cfg.for_each_expr(&mut |e| collect_lits_expr(e, out));
1209 }
1210
1211 fn collect_lits_expr(expr: &Expr, out: &mut Vec<f64>) {
1212     use crate::ir::expr::Prim;
1213     match expr {
1214         Expr::Lit(Prim::Int(n)) => out.push(*n as f64),
1215         Expr::Lit(Prim::Float(f)) => out.push(*f),
1216         // `for_each_child` does not cross `FnLit`; thresholds inside closures
1217         // (event handlers, render helpers) still bound guarded growth.
1218         Expr::FnLit { body_cfg, .. } => collect_lits_cfg(body_cfg, out),
1219         _ => {}
1220     }
1221 }
1222 // Structural descent (a no-op on `FnLit`, whose body was handled above).
1223 expr.for_each_child(&mut |c| collect_lits_expr(c, out));
1224 }
```

Décision : ADR-014 — widening à seuils, et pas de narrowing

Contexte (ADR-14, 2026-06-27). `Interval::widen` jette toute l'information numérique ; l'information qui bornerait la croissance (le littéral 10 de la garde) est pourtant dans le programme. *Décision, partie 1*. Un ensemble fini de seuils T par composant ; `widen_to` ajouté à côté de `widen` sans changer la signature du trait, en ne traitant que la composante numérique ; les seuils sont possédés par le pilote du point fixe et passés en argument, jamais stockés dans

les valeurs. *Partie 2, abandonnée* (révision du 2026-06-28). Le narrowing classique — une phase descendante après le post-point fixe — n'a pas été implémenté : le raffinement de branche pendant la montée plus le widening à seuils, y compris sur l'arc arrière interne d'`analyze_cfg`, retrouvent déjà toutes les bornes littérales ; et une descente ne pourrait de toute façon pas faire redescendre le store d'état, dont les écritures s'accumulent par join monotone. Trois conséquences annoncées ne sont donc pas réalisées : `WidenOutcome`, `Config::narrow_iterations`, et la suppression du recalcul `effect_setter_writes`, que l'ADR qualifie de « crude precision-recovery hack » et qui est resté (section 13.5.2).

Deux écarts entre le code et sa documentation sont à connaître. L'ADR annonçait pour seuils les littéraux des *gardes* et des *inits* ; le code prend *tous* les littéraux numériques du rendu, des *inits* de *State*, des corps d'*Effect* et de *Handler*, en descendant dans les *FnLit* (l. 1217–1219). C'est sans conséquence de soundness (« over-collecting is harmless », l. 1189–1190) : l'ensemble reste fini, un seuil de plus n'est qu'une borne candidate de plus. Inversement, la doc de la fonction annonce « all hook bodies » (l. 1187), mais les corps de *Memo* et de *Callback* et les arguments des *Custom* ne sont pas parcourus (le match des l. 1195–1201 les ignore). La tableau 13.1 donne les seuils des exemples du chapitre.

composant	littéraux du programme	T
Runaway	<code>useState(0), x + 1</code>	$\{0, 1\}$
Counter	<code>useState(0), count + 1</code> (dans le <code>onClick</code>)	$\{0, 1\}$
Guarded	<code>useState(0), count < 10, count + 1</code>	$\{0, 1, 10\}$
Lazy	<code>useState(() => 7), v < 20, v + 1</code>	$\{1, 7, 20\}$
Dead	<code>useState(0), const k = 3, k > 10, n + 1</code>	$\{0, 1, 3, 10\}$
Loader	<code>useState(0), setN(42)</code>	$\{0, 42\}$

TABLE 13.1 – Les seuils récoltés par `collect_thresholds` sur les exemples du chapitre (relevés par la sonde). Le 7 de *Lazy* vient du corps du *FnLit* initialiseur ; le 3 de *Dead* n'est pas une garde mais une constante locale, et bornera pourtant la croissance d'un palier (section 13.7).

13.2.4 Les corps de `useCallback`

Un `useCallback` est réécrit par le lowering en `CallbackVal(1)` : son corps quitte l'arbre d'expressions et vit dans l'entrée `HookEntry::Callback`. Pour qu'un appel à travers la variable liée (`const cb = useCallback(...); onLoad=(e) => cb(e)`) exécute encore le corps pour ses effets de bord, la préparation construit une table `callback_bodies` des corps par label, exposée à l'interpréteur par `QueryContext::callback_body` (src/engine/fixpoint.rs, l. 282–295, chapitre 11). C'est le seul rôle du `FixpointCtx` construit à chaque passe : ses deux autres champs, `state` et `memo`, ne sont lus par aucune méthode du trait, et semblent résiduels.

13.2.5 L'amorçage des slots

Le sujet du point fixe est déclaré, avec ses compagnons, juste avant la boucle :

Extrait 13.12 – src/engine/fixpoint.rs, l. 297–312 — le store sujet du point fixe et les accumulateurs de la boucle

```

297 // Fixpoint carrier: the product StateValue tracks every JS kind per label
298 // (ADR-015), so one store holds all slots – no per-type sub-store dispatch.
299 let mut state: StateStore<StateValue> = StateStore::bottom();
300 let mut memo_store: MemoStore<StateValue> = MemoStore::new();
301 let mut heap = initial_heap;
```

```

302 let mut widen_trace: HashMap<HookLabel, crate::engine::WidenEvent> = HashMap::new();
303 // Which effects wrote each slot during the current iteration – provenance
304 // for the widening events (ADR-019). Rebuilt every iteration.
305 let mut slot_writers: HashMap<HookLabel, Vec<HookLabel>> = HashMap::new();
306 let mut iteration: usize = 0;
307 let mut block_states: HashMap<BlockId, AbstractEnv<StateValue>>;
308 let mut env_exit: AbstractEnv<StateValue>;
309 let mut effect_block_states: HashMap<HookLabel, HashMap<BlockId, AbstractEnv<StateValue>>> =
310     HashMap::new();
311 let mut handler_block_states: HashMap<HookLabel, HashMap<BlockId, AbstractEnv<StateValue>>> =
312     HashMap::new();

```

Puis chaque `useState` reçoit la valeur de son initialiseur :

Extrait 13.13 – `src/engine/fixpoint.rs`, l. 314–353 — l’amorçage : un `join` par `useState`, l’initialiseur paresseux exécuté

```

314 // Seed each useState label with its init expression. The init runs in
315 // the component's entry scope: module consts (and parent-bound props,
316 // when analyzed inter) are visible to `useState(DEFAULT)`.
317 {
318     let init_env = initial_env.clone();
319     let init_memo = MemoStore::new();
320     let init_untyped = StateStore::bottom();
321     for hook in &hooks {
322         if let HookEntry::State { label, init, .. } = hook {
323             let init_val = {
324                 let mut init_untyped_mut = init_untyped.clone();
325                 let mut init_memo_mut = init_memo.clone();
326                 let mut heap = crate::domains::Heap::new();
327                 let mut ac = AnalysisCtx::null(
328                     comp_id,
329                     &mut init_untyped_mut,
330                     &mut init_memo_mut,
331                     &mut heap,
332                 );
333                 // A null/undefined init needs no TS-hint override: the product
334                 // value joins the null slot with whatever the setters write,
335                 // and the num slot widens independently.
336                 match init {
337                     // Lazy initializer `useState(() => expr)`: React runs
338                     // the thunk once at mount and stores its RETURN value
339                     // – the state is never the closure itself. Abstracting
340                     // the FnLit (reference(Unstable)) made every lazy-init
341                     // state slot an "unstable dep" (corpus FP, TODO.md F2).
342                     Expr::FnLit {
343                         params, body_cfg, ..
344                     } if params.is_empty() => crate::domains::interp::exec_body(
345                         transfer, body_cfg, &init_env, &mut ac,
346                     ),
347                     _ => transfer.eval_expr(init, &init_env, &mut ac),
348                 }
349             };
350             state.update(*label, init_val);
351         }
352     }
353 }

```

L’init est évalué dans `initial_env` (constantes et props venues du parent), contre des stores *vides* et un tas neuf : un initialiseur ne lit pas l’état, il le crée. Le cas particulier des l. 342–346 mérite un rappel de React.

Rappel React : Initialiseur paresseux

`useState(() => e)` appelle le thunk *une fois*, au montage, et stocke sa valeur de retour ; aux rendus suivants l'argument est ignoré (« React saves the initial state once and ignores it on the next renders », [Met26b]). L'état n'est jamais la fermeture elle-même. Abstraire le `FnLit` comme une référence (ce que faisait une version antérieure) faisait de chaque slot à init paresseux un « dep instable » — un faux positif de corpus, cité par le commentaire. Le moteur exécute donc le corps du thunk (`exec_body`) et amorce le slot avec son retour. La règle *lazy-init* exploite l'autre moitié du fait : un argument *non* paresseux est ré-évalué à chaque rendu pour rien.

Exemple 13.1: Un initialiseur paresseux et le seuil 20

Extrait 13.14 – /tmp/ch13/lazy.tsx

```
1 import { useState, useEffect } from "react";
2
3 export function Lazy() {
4   const [v, setV] = useState(() => 7);
5   useEffect(() => { if (v < 20) setV(v + 1); }, [v]);
6   return <div>{v}</div>;
7 }
```

L'IR de l'init est `FnLit { params: [], body_cfg: Return(Lit(Int(7))) }`. L'amorçage donne `s0=num[7,7]`, pas une référence. Les tours donnent `[7,8]`, `[7,9]` (joins), puis au troisième changement le widening à seuils cherche le plus petit seuil ≥ 10 dans $T = \{1, 7, 20\}$: c'est 20, d'où `[7,20]`, stable au tour suivant. Sonde : `iterations = 3`, `state_store = s0=num[7,20]`, `effect_setter_writes = s0=num[8,20]`, `widen_trace = [(0, WidenEvent { iteration: 3, writers: [1] })]`. Le CLI n'émet que l'Info `widening-info` et conclut `✓ 1 file(s) no issues found.` : le slot a été widené, mais la valeur que l'effet écrit est bornée (section 13.5.2).

Piège

Le cas paresseux n'est reconnu que pour un `FnLit` *sans paramètre écrit en place*. Avec `const makeInit = () => 7`; au niveau du module puis `useState(makeInit)`, l'init est un `Var` : il est évalué comme une expression ordinaire, pas exécuté. Et `makeInit` n'est pas une constante de module au sens du lowering. `ModuleConstInit` ne retient que les littéraux primitifs, les objets, tableaux, `new`, regex et JSX, et `createContext` (src/lowering/mod.rs, l. 314–336), pas une fonction fléchée. La lecture de `makeInit` manque donc l'environnement et vaut $\top$. Observé sur /tmp/ch13/makeinit.tsx (composant `LazyVar`) avec le même effet `if (v < 20) setV(v + 1)` : la sonde donne l'amorçage `s0= $\top$` , `iterations = 0`, et le CLI émet un `WARNING infinite-loop` (« may store a fresh reference into state 'v' ») : un faux positif. L'amorçage reste sound, car $\top$ contient 7 ; c'est seulement une perte de précision.

Invariant 13.2: Tout slot est présent dès l'amorçage

Après l'amorçage, chaque label `State` du composant (hooks greffés compris) est une clé du store, même si sa valeur est $\perp$ (un init qui s'évalue à $\perp$ est rarissime, mais `update` insère la clé de toute façon). Une passe qui part de `state.clone()` rend donc un store dont `labels()` liste *tous* les slots, qu'elle les ait écrits ou non. Ce fait anodin explique une imprécision de provenance décrite au section 13.4.3.

13.3 Un tour de la boucle externe

La boucle (`src/engine/fixpoint.rs`, l. 355–530) est une itération de Kleene *globale* (définition 3.5) : chaque tour ré-exécute tous les corps, dans un ordre fixe, à partir du même store S . Il n’y a pas de worklist à ce niveau — la worklist n’existe qu’à l’intérieur d’un CFG (chapitre 12). Chaque tour commence par `let mut state_store = state.clone();` (l. 356) : c’est cette copie, et non `state`, que toutes les passes du tour reçoivent et enrichissent.

13.3.1 La passe de rendu

Extrait 13.15 – `src/engine/fixpoint.rs`, l. 355–380 — la passe de rendu d’un tour

```

355 loop {
356     let mut state_store = state.clone();
357
358     // -- Render pass -----
359     // Use initial_env as entry: child analyses start with props bound.
360     let (bs, state_from_render) = {
361         let ctx = FixpointCtx {
362             state: &state_store,
363             memo: &memo_store,
364             callbacks: &callback_bodies,
365         };
366         analyze_cfg::<T>(
367             comp_id,
368             &render_cfg,
369             initial_env.clone(),
370             &state_store,
371             &memo_store,
372             transfer,
373             config.widen_threshold,
374             &thresholds,
375             &mut heap,
376             &ctx,
377             inter,
378         )
379     };
380     block_states = bs;

```

L’environnement d’entrée est `initial_env` : les constantes de module et, en analyse `inter`, les props que le parent a liées. `analyze_cfg` rend les environnements de sortie de chaque bloc (`bs`, rangé dans `block_states` malgré son nom, chapitre 12) et un store `state_from_render`, noté R , qui part de `state_store` et y joint chaque écriture rencontrée pendant le rendu. Comme `analyze_cfg` part d’une copie du store reçu et n’y fait que joindre (`src/engine/cfg_analyzer.rs`, l. 49, chapitre 12), $R \sqsupseteq S$ toujours. Deux sortes d’écritures peuvent survenir pendant un rendu : un setter appelé dans le corps (`setter-in-render` s’en occupe), et, quand `inter` est présent, l’analyse d’un enfant `<Child .../>` rencontré dans le JSX, lancée *pendant* le rendu par `eval_comp_app` (chapitre 11) ; l’enfant peut alors écrire dans le `SharedStateStore`, que le test de convergence importera.

13.3.2 Les mémos : une valeur qui n’est qu’une stabilité

Extrait 13.16 – `src/engine/fixpoint.rs`, l. 382–413 — le recalcul des mémos depuis l’environnement de sortie du rendu

```

382 // -- Recompute memo store from exit env -----

```



```

383 // Deps evaluate through the normal path against the real fixpoint stores
384 // (so a memo depending on another memo, or on a heap field, resolves
385 // instead of reading a fabricated  $\perp$  store). Sets are deferred to a
386 // second pass so each recompute reads a consistent snapshot – the
387 // fixpoint converges any memo-to-memo dependency across iterations.
388 env_exit = exit_env(&render_cfg, &block_states);
389 let memo_updates: Vec<(HookLabel, StateValue)> = {
390     let null_query = NullCtx;
391     let mut memo_ctx = AnalysisCtx {
392         component: comp_id,
393         state: &mut state_store,
394         memo: &mut memo_store,
395         heap: &mut heap,
396         query: &null_query,
397         inter,
398     };
399     hooks
400     .iter()
401     .filter_map(|hook| match hook {
402         HookEntry::Memo { label, deps, .. }
403         | HookEntry::Callback { label, deps, .. } => Some((
404             *label,
405             transfer.recompute_memo(comp_id, deps, &env_exit, &mut memo_ctx),
406         )),
407         _ => None,
408     })
409     .collect()
410 };
411 for (label, val) in memo_updates {
412     memo_store.set(label, val);
413 }

```

L'environnement de sortie du rendu, `env_exit`, est le join des environnements de sortie des blocs terminés par `Return` :

Extrait 13.17 – `src/engine/fixpoint.rs`, l. 1226–1237 — `exit_env` : le join des sorties, par `reduce` et non par `fold`

```

1226 fn exit_env<D: AbstractDomain>(<
1227     cfg: &CFG,
1228     block_states: &HashMap<BlockId, AbstractEnv<D>>,
1229 ) -> AbstractEnv<D> {
1230     cfg.blocks
1231     .values()
1232     .filter(|b| matches!(b.term, crate::ir::cfg::Terminator::Return(_)))
1233     .filter_map(|b| block_states.get(&b.id))
1234     .cloned()
1235     .reduce(|acc, env| acc.join(&env))
1236     .unwrap_or_else(AbstractEnv::bottom)
1237 }

```

Le `reduce` plutôt que `fold(bottom, join)` n'est pas une coquetterie : `AbstractEnv::bottom()` n'est pas neutre pour le join (une variable présente d'un seul côté devient $\top$, chapitre 12) ; un CFG sans `Return` atteint rend un environnement vide, dont chaque lecture vaut $\top$. C'est sound, et cette fonction est dupliquée à l'identique dans `AnalysisResult::exit_env` (`src/engine/analysis_result.rs`, l. 274–289).

Le recalcul lui-même est délégué au domaine (`Transfer::recompute_memo`, chapitre 11) : la valeur d'un `useMemo` ou d'un `useCallback` est *uniquement une stabilité de référence* — Stable pour des `deps []` connues vides, `Unknown` pour des `deps` illisibles, sinon le join des stabilités de ses

deps, un dep `StateVal(1)` donnant `Versioned((composant, 1))`. Le corps du mémo n'est *jamais* exécuté par le moteur : `useMemo(() => x + 1, [x])` n'a pas de composante numérique. Deux détails de la boucle en découlent.

- Les écritures dans le memo store sont *différées* (commentaire des l. 383–387) : chaque `recompute_memo` lit un instantané cohérent, et une chaîne mémo $\rightarrow$ mémo converge « à travers les itérations », avec un tour de retard par maillon.
- Le rendu du tour n lit les mémos calculés au tour $n - 1$, puisque le recalcul suit la passe de rendu. Au tour 0, il n'y a encore rien, et `MemoStore::get` d'un label absent vaut $\top$:

Extrait 13.18 – `src/domains/stores/memo_store.rs`, l. 27–30 — un mémo pas encore calculé vaut $\top$ — à l'inverse du `StateStore`, où l'absent vaut $\perp$

```
27  /// Returns `D::top()` for labels not yet computed (conservative).
28  pub fn get(&self, label: HookLabel) -> D {
29      map_get_or(&self.0, &label, D::top)
30  }
```

Exemple 13.2: Une boucle portée par un `useMemo`

Extrait 13.19 – `/tmp/ch13/memoloop.tsx`

```
1  import { useState, useEffect, useMemo } from "react";
2
3  export function MemoLoop() {
4      const [x, setX] = useState(0);
5      const d = useMemo(() => x + 1, [x]);
6      useEffect(() => {
7          setX(d);
8      }, [d]);
9      return <div>{x}</div>;
10 }
```

Concrètement le composant boucle ($0 \rightarrow 1 \rightarrow 2 \rightarrow \dots$). Abstraitement, au tour 0 le rendu lie `d` à `MemoVal(1)` avant tout recalcul, soit $\top$; l'effet exécute `setX( $\top$ )` et le slot saute à $\top$ en un tour ; le tour 1 ne change rien. La sonde donne `iterations = 1`, `state_store = s0= $\top$` , `widen_trace = []`, et après convergence le mémo vaut `ref:Versioned({( $C0$ , 0)})`, d'où `effect_setter_writes = s0=ref:Versioned(...)` et un trigger exact: `false`. Le CLI répond $\checkmark$ 1 file(s) no issues found. : un faux négatif, que nous rangerons au section 13.7 (#157). La leçon pour la boucle : *un slot qui atteint $\top$ par join n'a pas été widené*, donc n'est pas dans `widen_trace`, donc n'est pas un signal de divergence.

Piège

Le memo store n'entre pas dans le test de convergence : seul le `StateStore` y est comparé. Le memo store est recalculé à chaque tour et son dernier contenu est publié, mais si un mémo change au dernier tour sans que l'état change, aucun tour supplémentaire ne rejoue les effets avec sa nouvelle valeur. Les effets du tour ne la voient pas non plus : ils lisent la variable liée par le rendu, donc le mémo du recalcul précédent. Sur `MemoLoop`, l'effet du tour 0 écrit bien $\top$, pas la valeur `Versioned` calculée juste avant lui. La question est donc de savoir si une valeur de mémo peut *monter* entre deux tours sans que l'état bouge. Les deux sources de variation connues descendent. $\top$ au tour 0 laisse place à une stabilité plus précise. Dans une chaîne $m2 \rightarrow m1$, `m2` garde un tour de retard, mais sur une valeur de `m1` qui elle-même ne fait que descendre. Dans ces deux cas, la valeur périmée qu'une écriture a jointe dans l'état est au-dessus de la valeur finale, et l'état reste sound. Un dep dérivé de

l'état reste la seule voie de montée. Un dep qui *est* un slot n'en fait pas partie, car il vaut toujours Versioned (src/domains/transfer/state_value.rs, l. 79–88). Au commit e67b10a, nous n'avons pas construit d'instance d'une telle montée. La passe de rafraîchissement du section 13.5 corrige de toute façon ce que les *règles* lisent.

13.3.3 Les effets : tous, à chaque tour, depuis la sortie du rendu

Extrait 13.20 – src/engine/fixpoint.rs, l. 415–449 — les passes d'effets d'un tour

```

415 // -- Effect passes -----
416 let mut state_from_effects = StateStore::bottom();
417 slot_writers.clear();
418 for hook in &hooks {
419     if let HookEntry::Effect {
420         label, body_cfg, ..
421     } = hook
422     {
423         let (eff_bs, eff_state) = {
424             let ctx = FixpointCtx {
425                 state: &state_store,
426                 memo: &memo_store,
427                 callbacks: &callback_bodies,
428             };
429             analyze_cfg::<T>(
430                 comp_id,
431                 body_cfg,
432                 env_exit.clone(),
433                 &state_store,
434                 &memo_store,
435                 transfer,
436                 config.widen_threshold,
437                 &thresholds,
438                 &mut heap,
439                 &ctx,
440                 inter,
441             )
442         };
443         effect_block_states.insert(*label, eff_bs);
444         for slot in eff_state.labels() {
445             slot_writers.entry(slot).or_default().push(*label);
446         }
447         state_from_effects = state_from_effects.join(&eff_state);
448     }
449 }
```

Trois décisions tiennent dans ces trente lignes.

Tous les effets, quelles que soient leurs deps. Le motif `HookEntry::Effect { label, body_cfg, .. }` ignore deps. Un effet `[]` de montage, un effet `[count]`, un effet sans tableau sont exécutés de la même façon, à chaque tour. C'est une sur-approximation de la règle de React « l'effet tourne si et seulement si un dep a changé selon `Object.is` » (chapitre 2) : l'ensemble des écritures possibles est un sur-ensemble de ce que React ferait. La précision est récupérée *dans les règles*, qui relisent les deps : `infinite-loop` saute les effets à `[]` et ceux dont tous les deps sont prouvés stables (src/rules/impls/infinite_loop.rs, l. 96–113), et la relation `effect_triggers` dit lesquels un slot relance. L'ADR-4 décrivait au contraire « for each Effect whose decision = Effect » ; le code a choisi plus large, et il n'a pas de notion de décision.

Depuis `env_exit`. Chaque effet part de l’environnement de sortie du rendu de *ce* tour : il voit les valeurs que ce rendu a liées, exactement comme la fermeture que React a créée pendant le rendu.

Depuis le même `state_store`. Tous les effets du tour reçoivent la même copie `state_store` ; ce qu’un effet écrit n’est pas visible des effets suivants du même tour, seulement du tour suivant, via le join `state_from_effects`. L’ordre concret des effets (enfants d’abord, puis ordre syntaxique) est abstrait par un join commutatif.

Rappel React : Fermetures, file de mises à jour et batching

Un effet est une fermeture créée pendant le rendu : il lit les variables du rendu qui l’a créé, pas les valeurs courantes (EFF de React-tRace met en file $\langle \lambda_e, \sigma \rangle$ avec l’environnement σ du rendu, chapitre 2). Un `setX(v)` n’écrit pas l’état : il met v en file, et React applique la file au rendu suivant — depuis React 18, toutes les mises à jour d’un même effet ou d’un même événement sont regroupées (*batching*). Le moteur abstrait ce double mécanisme en un seul : la mise à jour faible $S[\ell] := S[\ell] \sqcup v$, qui contient tout résultat possible de la file (dernier gagne, updaters enchaînés, bail-out `Object.is`), au prix de l’ordre. Un effet qui appelle `setM(doubled)` puis `setM(prev => prev + 1)` voit, dans l’updater, un `prev` qui contient déjà la première écriture de la passe — c’est la file, abstraite.

Exemple 13.3: Un effet de montage, rejoué à chaque tour

Extrait 13.21 – `/tmp/ch13/loader.tsx`

```
1 import { useState, useEffect } from "react";
2
3 export function Loader() {
4   const [n, setN] = useState(0);
5   useEffect(() => {
6     setN(42);
7   }, []);
8   return <div>{n}</div>;
9 }
```

React exécute l’effet une fois. Le moteur l’exécute à chaque tour :

tour	R	effet 1	N	action
amorçage				$s0 = \text{num}[0, 0]$
0	$[0, 0]$	$[0, 42]$	$[0, 42]$	$\text{iteration} = 1 < 3 : S := N$
1	$[0, 42]$	$[0, 42]$	$[0, 42]$	$N \sqsubseteq S$: sortie

Résultat : $\text{iterations} = 1$, $s0 = \text{num}[0, 42]$ — l’enveloppe convexe de $\{0, 42\}$, que le domaine des intervalles ne peut pas distinguer de $\{0, \dots, 42\}$ —, $\text{effect_setter_writes} = s0 = \text{num}[42, 42]$, aucun trigger (deps []). La ré-exécution au tour 1 est sans conséquence ici (l’écriture est idempotente). Le CLI émet un `WARNING unnecessary-rerender` (« mount-only effect sets state ‘n’ to a constant different from its initial value ») et `verified infinite-loop`.

Exemple 13.4: Deux effets qui se nourrissent : le batching abstrait

Extrait 13.22 – `/tmp/ch13/mutual.tsx`

```
1 import { useState, useEffect } from "react";
2
3 export function Mutual() {
```

```

4  const [x, setX] = useState(0);
5  const [y, setY] = useState(0);
6  useEffect(() => { setY(x + 1); }, [x]);
7  useEffect(() => { setX(y + 1); }, [y]);
8  return <div>{x + y}</div>;
9  }

```

Au tour 0, les deux effets partent du même store $\{s_0 = [0, 0], s_1 = [0, 0]\}$:

```

iter 0: effect 2 out = s0=num[0,0], s1=num[0,1]
iter 0: effect 3 out = s0=num[0,1], s1=num[0,0]
iter 0: render out = s0=num[0,0], s1=num[0,0]
iter 0: new_state = s0=num[0,1], s1=num[0,1]

```

L'effet 3 ne voit pas l'écriture de l'effet 2 : il lit $y = [0, 0]$ et écrit $x \in [0, 1]$. Chaque tour n'ajoute donc que +1 à chaque slot, et le troisième changement déclenche le widening des deux : $s_0 = \text{num}[0, \text{inf}]$, $s_1 = \text{num}[0, \text{inf}]$, widen_trace sur les labels 0 et 1. Le CLI émet deux WARNING infinite-loop (un par slot) et deux derived-state.

13.3.4 Les handlers : dans le point fixe, hors du signal

Extrait 13.23 – src/engine/fixpoint.rs, l. 451–483 — les passes de handlers : même schéma que les effets, avec le commentaire décisif

```

451  // -- Handler passes (in-cycle) -----
452  // Handlers run 0..N times → include in fixpoint for sound range approx.
453  // NOT tracked in widened_labels (handler-caused widening ≠ InfiniteLoop).
454  let mut state_from_handlers = StateStore::bottom();
455  for hook in &hooks {
456      if let HookEntry::Handler {
457          label, body_cfg, ..
458      } = hook
459      {
460          let (h_bs, h_state) = {
461              let ctx = FixpointCtx {
462                  state: &state_store,
463                  memo: &memo_store,
464                  callbacks: &callback_bodies,
465              };
466              analyze_cfg::<T>(
467                  comp_id,
468                  body_cfg,
469                  env_exit.clone(),
470                  &state_store,
471                  &memo_store,
472                  transfer,
473                  config.widen_threshold,
474                  &thresholds,
475                  &mut heap,
476                  &ctx,
477                  inter,
478              )
479          };
480          handler_block_states.insert(*label, h_bs);
481          state_from_handlers = state_from_handlers.join(&h_state);
482      }
483  }

```

Le lowering a extrait chaque onClick=... et chaque addEventListener(ev, fn) d'un effet en

une entrée `HookEntry::Handler` avec son propre CFG (chapitre 7). Les handlers sont analysés exactement comme les effets — même `env_exit`, même `state_store` — et leur store H est joint dans N . Mais leur croissance n'entre pas dans `widen_trace`. Le commentaire des l. 452–453 dit tout : « Handlers run 0..N times $\rightarrow$ include in fixpoint for sound range approx. NOT tracked in widened_labels (handler-caused widening $\neq$ InfiniteLoop) ».

Rappel React : Événements

Un handler s'exécute sur une entrée extérieure, zéro, une ou N fois, dans un ordre arbitraire par rapport aux autres handlers ; en React-tRace c'est `STEPEVENT`, qui n'est franchi qu'en mode *event loop* (chapitre 2). Un compteur que l'utilisateur incrémente mille fois atteint mille : c'est une *valeur* que l'analyse doit admettre, mais ce n'est pas une boucle, puisque chaque tour exige un clic. Un effet qui incrémente, lui, relance le cycle sans intervention.

Décision : ADR-009 — points d'entrée et classes de déclenchement

Décision (ADR-9). Un composant est un ensemble de *points d'entrée* — rendu, effets, callbacks différés (`.then`, timers), handlers — analysés par la même machinerie, et qui ne diffèrent que sur deux axes : « dans le cycle automatique ? » et « un widening induit est-il un bug ? ». Le rendu *est* le cycle ; un effet ou un `.then y` sont (oui / oui) ; un handler n'y est pas (non / non : « clicking 1000 $\times$ isn't a bug »). La §5 de l'ADR (*multiplicity*) demande que les handlers soient tout de même dans la boucle de point fixe, pour la soundness des intervalles, avec widening, mais exclus du signal — c'est le code des l. 451–483 et le filtre `new_state_incycle` du test de convergence. *Alternative refusée* : descendre uniformément dans tout callback, qui produirait un faux positif *infinite-loop* sur `addEventListener('click', () => setCount(c => c + 1))`. *Un choix en tension* : pour un callee `Unknown`, l'ADR retient `skip` (« we accept the FN »), ce que l'invariant du projet interdit en principe ; le mécanisme B6 du chapitre 11 en réduit la portée.

Exemple 13.5: Un compteur par clic : widené, pas tracé

Extrait 13.24 – /tmp/ch13/counter.tsx

```
1 import { useState } from "react";
2
3 export function Counter() {
4   const [count, setCount] = useState(0);
5   return <button onClick={() => setCount(count + 1)}>{count}</button>;
6 }
```

IR : hook 0: State `init=Lit(Int(0))`, hook 1: Handler `click`. Seuils $\{0, 1\}$.

tour	R	handler 1	N	action
0	$[0, 0]$	$[0, 1]$	$[0, 1]$	iteration = 1 : $S := N$
1	$[0, 1]$	$[0, 2]$	$[0, 2]$	iteration = 2 : $S := N$
2	$[0, 2]$	$[0, 3]$	$[0, 3]$	iteration = 3 : $\text{widen_to} \rightarrow [0, +\infty]$; labels changés dans $R \sqcup E$: aucun
3	$[0, +\infty]$	$[0, +\infty]$	$[0, +\infty]$	$N \sqsubseteq S$: sortie

Sonde : iterations = 3, state_store = `s0=num[0,inf]`, widen_trace = []. CLI : Counter: 3 iteration(s), widened: [], aucun diagnostic. Le slot est bien widené (l'intervalle $[0, +\infty]$ est la seule valeur sound pour un compteur de clics), mais comme seul le handler l'a fait bouger, `changed_labels` de $R \sqcup E$ est vide et rien n'entre dans `widen_trace`.

L'inverse est tout aussi important : les handlers font partie du point fixe *pour rendre les effets sound*. Le test unitaire `handler_enables_infinite_loop_detection` (`src/engine/fixpoint.rs`, l. 2564–2694) analyse un effet `if (count > 1) setCount(count + 1)` avec `deps [count]`, plus un handler `setCount(count + 1)`. Sans les handlers dans la boucle, le slot resterait à `[0, 0]`. Dans la branche, `count` serait raffiné à $\perp$ (`narrow_gt(1)` sur `[0, 0]`), l'écriture `count + 1` vaudrait $\perp$, et la boucle infinie que déclenche le *deuxième* clic ne serait jamais vue : un faux négatif. Avec eux, le handler fait croître `count` tour après tour jusqu'à rendre la branche vivante ; l'effet écrit alors, et c'est *lui* qui fait entrer le label dans `widen_trace`. Les handlers participent à la *valeur* sans participer au *signal*.

13.3.5 Le join et le test de convergence

Extrait 13.25 – `src/engine/fixpoint.rs`, l. 485–497 — le join des quatre sources et le test d'arrêt

```
485 // -- Convergence check -----
486 let new_state_incycle = state_from_render.join(&state_from_effects);
487 // Include cross-component state updates made by child effects/callbacks.
488 let external_updates = inter
489     .map(|i| i.shared_state.borrow().slice(comp_id))
490     .unwrap_or_else(StateStore::bottom);
491 let new_state = new_state_incycle
492     .join(&state_from_handlers)
493     .join(&external_updates);
494
495 if new_state.leq(&state) {
496     break;
497 }
```

Le nouvel état est le join de quatre stores : *R* (rendu), *E* (effets), *H* (handlers) et la tranche du `SharedStateStore` qui concerne ce composant — les écritures faites, pendant ce tour ou un tour antérieur, par un setter de ce composant appelé *ailleurs* (un enfant qui a reçu `setCount` en prop). C'est ici, et non en tête de boucle, que l'étape 1 du plan de liste 13.5 a lieu. Le store partagé n'a qu'une opération d'extraction :

Extrait 13.26 – `src/domains/stores/shared_state_store.rs`, l. 57–66 — la tranche d'un composant dans le `SharedStateStore`

```
57 /// Extract all entries for `comp` as a `StateStore` (for import into that component's fixpoint).
58 pub fn slice(&self, comp: ComponentId) -> StateStore<StateValue> {
59     let mut store = StateStore::new();
60     for ((c, label), val) in &self.entries {
61         if *c == comp {
62             store.update(*label, val.clone());
63         }
64     }
65     store
66 }
```

et le test d'arrêt est l'ordre point à point des stores :

Extrait 13.27 – `src/domains/stores/state_store.rs`, l. 69–77 — $S \sqsubseteq S'$: chaque label de *S* est en dessous de sa valeur dans *S'*

```
69 /// `self`  $\sqsubseteq$  `other` : for every label, `self.get(L) ≤ other.get(L)`.
70 pub fn leq(&self, other: &Self) -> bool {
71     for &k in self.0.keys() {
72         if !leq_pointwise(&self.get(k), &other.get(k)) {
73             return false;
74         }
75     }
76     true
77 }
```

```

74         }
75     }
76     true
77 }

```

`leq_pointwise` s'appuie sur le `PartialOrd` de `StateValue` (`Less | Equal`) : deux valeurs incomparables font échouer le test, ce qui est la bonne réponse (le nouvel état n'est pas contenu dans l'ancien).

Proposition 13.1: Le test d'arrêt est un test d'égalité

À chaque tour, $R \sqsupseteq S$, $E \sqsupseteq S$ et $H \sqsupseteq S$, car chacune de ces passes part de `state.clone()` et n'y fait que joindre (`src/engine/cfg_analyzer.rs`, l. 49) ; donc $N \sqsupseteq S$. Par antisymétrie, `new_state.leq(&state)` équivaut à $N = S$ sur tous les labels : la boucle s'arrête exactement quand un tour de plus n'a rien ajouté. La suite (S_k) est croissante, et S_k à l'arrêt est un post-point fixe de la fonction « un tour » au sens de la définition 13.1.

La tranche partagée réserve une surprise que le section 13.7 détaillera : quand `inter` est présent, un composant écrit *ses propres* slots dans le `SharedStateStore`. Sur `Counter` analysé par `analyze_program`, la sonde lit `shared_state = {(ComponentId(0), 0): number[1, inf]}` — l'écriture du handler, rangée dans le store partagé par le second bras de `exec_setter_call` (`src/domains/interp/interpreter.rs`, l. 368–391), puis réimportée par `slice`. Ici c'est sans effet (la valeur est déjà dans H) ; avec un updater fonctionnel, ce ne l'est plus.

13.4 Convergence, widening et terminaison

13.4.1 Le compteur et les deux opérateurs

Quand $N \not\sqsubseteq S$, la boucle doit choisir comment avancer :

Extrait 13.28 – `src/engine/fixpoint.rs`, l. 499–530 — join tant que le compteur est bas, widening à seuils ensuite, garde-fou à cent

```

499     iteration += 1;
500     if iteration >= 100 {
501         // Pathological input: force widening on all labels to guarantee convergence.
502         for label in state.labels() {
503             widen_trace
504                 .entry(label)
505                 .or_insert_with(|| crate::engine::WidenEvent {
506                     iteration,
507                     writers: slot_writers.get(&label).cloned().unwrap_or_default(),
508                 });
509         }
510         state = state.widen(&new_state);
511         break;
512     }
513
514     if iteration >= config.widen_threshold {
515         // widen_trace: render+effects only handler widening is not a bug.
516         // `or_insert_with` keeps the FIRST widening iteration (the most
517         // informative one for the witness chain).
518         for label in new_state_incycle.changed_labels(&state) {
519             widen_trace
520                 .entry(label)
521                 .or_insert_with(|| crate::engine::WidenEvent {
522                     iteration,

```

```

523             writers: slot_writers.get(&label).cloned().unwrap_or_default(),
524         });
525     }
526     state = state.widen_to(&new_state, &thresholds);
527 } else {
528     state = new_state;
529 }
530 }

```

Lecture pas à pas.

1. *Le compteur* iteration n'est incrémenté que si l'état a changé. Au k -ième tour (compté de 0) qui change l'état, il passe à $k + 1$. Avec `widen_threshold = 3`, les deux premiers changements sont des joins ($S := N$), le troisième et les suivants des widenings. Le compteur est *global au composant*, pas par label : un slot qui commence à croître tard est widené dès son premier changement si le seuil est déjà atteint.
2. *Le widening* `widen_to(&new_state, &thresholds)` porte sur *tout* le store, y compris la croissance venue des handlers ; mais `widen_trace` ne retient que les labels changés dans `new_state_incycle =  $R \sqcup E$` (l. 518). C'est la traduction exacte de l'ADR-9 : la valeur est sound, le signal ne compte que le cycle.
3. *La première itération gagne* : `or_insert_with` garde l'événement du premier widening d'un label, le plus informatif pour la chaîne de témoins (ADR-19).
4. *Le garde-fou* : à iteration ≥ 100 , un widening *sans seuils* (`widen`, pas `widen_to`) sur tout le store, tous les labels du store dans `widen_trace` (y compris ceux qui n'ont pas bougé), et `break` immédiat.

Au niveau du store, les trois opérateurs ne diffèrent que par la fonction appliquée à chaque label :

Extrait 13.29 – `src/domains/stores/state_store.rs`, l. 35–62 — `join`, `widen`, `widen_to` : un seul parcours, trois combinateurs

```

35     /// Pointwise merge over the union of keys, absent keys read as `D::bottom`.
36     /// The per-key combinator `f` is the only thing `join`/`widen`/`widen_to`
37     /// differ by.
38     fn merge_with(&self, other: &Self, f: impl Fn(&D, &D) -> D) -> Self {
39         let mut out = self.0.clone();
40         for (&k, v) in &other.0 {
41             let cur = map_get_or(&out, &k, D::bottom);
42             out.insert(k, f(&cur, v));
43         }
44         StateStore(out)
45     }
46
47     /// Pointwise join of two stores.
48     pub fn join(&self, other: &Self) -> Self {
49         self.merge_with(other, |a, b| a.join(b))
50     }
51
52     /// Widening pointwise `D::widen`. Critical for interval domains where
53     /// widen  $\neq$  join (bounds jump to  $\pm\infty$  instead of hull).
54     pub fn widen(&self, other: &Self) -> Self {
55         self.merge_with(other, |a, b| a.widen(b))
56     }
57
58     /// Threshold widening pointwise `D::widen_to`. Domains without thresholds
59     /// (default impl) fall back to plain `widen`.
60     pub fn widen_to(&self, other: &Self, thresholds: &[f64]) -> Self {
61         self.merge_with(other, |a, b| a.widen_to(b, thresholds))

```



```
62 }
```

et au niveau de la valeur, seule la composante numérique a des seuils :

Extrait 13.30 – `src/domains/impls/state_value.rs`, l. 657–676 — le widening du produit : intervalles et chaînes ont un vrai widening, le reste est un join

```
657 fn widen(&self, other: &Self) -> Self {
658     StateValue {
659         num: self.num.widen(&other.num),
660         boolean: self.boolean.join(&other.boolean),
661         str: self.str.widen(&other.str),
662         reference: self.reference.join(&other.reference),
663         null: self.null || other.null,
664         undef: self.undef || other.undef,
665         setter: self.setter.join(&other.setter),
666         other: self.other || other.other,
667     }
668 }
669
670 fn widen_to(&self, other: &Self, thresholds: &[f64]) -> Self {
671     StateValue {
672         num: self.num.widen_to(&other.num, thresholds),
673         // Non-numeric slots have no notion of a threshold → plain widen.
674         ..self.widen(other)
675     }
676 }
```

L’algorithme 6 récapitule la boucle complète, garde-fou et provenance compris ; c’est la version détaillée de l’algorithme réduit du chapitre 2.

13.4.2 Trois déroulés

Les tableaux qui suivent ont été relevés avec la réplique de la sonde ; les valeurs finales coïncident avec celles du moteur.

Runaway (liste 13.1). Seuils $\{0, 1\}$. L’effet lit x dans `env_exit` (la valeur de fin de rendu) et écrit $x + 1$:

Guarded. Le compteur gardé, cas d’école de l’ADR-14 :

Extrait 13.31 – `/tmp/ch13/guarded.tsx`

```
1 import { useState, useEffect } from "react";
2
3 export function Guarded() {
4     const [count, setCount] = useState(0);
5     useEffect(() => {
6         if (count < 10) setCount(count + 1);
7     }, [count]);
8     return <div>{count}</div>;
9 }
```

Seuils $\{0, 1, 10\}$. L’effet a quatre blocs (Branch `count < 10`, la branche qui écrit, la branche vide, la jonction) et le raffinement de garde (`narrow_env_for_branch`, chapitre 12) borne `count` dans la branche *then* :

Algorithme 6 : La boucle externe de `analyze_component_impl` (`src/engine/fixpoint.rs`, 1. 297–530). W est la table locale `slot_writers` (« quels effets ont un store contenant ℓ »); ∇_T est `widen_to` avec les seuils T .

Entrées : `render_cfg`, `hooks` (après expansion), `initial_env`, seuils T , $t = \text{widen_threshold}$

Sorties : S convergé, `iteration`, `widen_trace`

```

1  $S \leftarrow \perp$ ; pour chaque State{ $\ell$ , init} faire
2    $S[\ell] \leftarrow S[\ell] \sqcup \llbracket \text{init} \rrbracket$ 
3  $i \leftarrow 0$ ;  $\text{trace} \leftarrow \emptyset$ ;
4 tant que vrai faire
5    $S' \leftarrow$  copie de  $S$ ;
6    $(bs, R) \leftarrow \text{analyze\_cfg}(\text{render\_cfg}, \text{initial\_env}, S')$ ;  $\text{env} \leftarrow \text{exit\_env}(bs)$ ;
7   pour chaque Memo/Callback{ $\ell$ , deps} faire
8      $\text{memo}[\ell] \leftarrow \text{recompute\_memo}(\text{deps}, \text{env})$ ; // écritures différées
9    $E \leftarrow \perp$ ;  $W \leftarrow \emptyset$ ;
10  pour chaque Effect{ $e$ , body} faire
11     $(\_, E_e) \leftarrow \text{analyze\_cfg}(\text{body}, \text{env}, S')$ ;  $E \leftarrow E \sqcup E_e$ ;  $W[\ell] += e$  pour tout
     $\ell \in \text{labels}(E_e)$ ;
12   $H \leftarrow \perp$ ; pour chaque Handler{ $h$ , body} faire
13     $(\_, H_h) \leftarrow \text{analyze\_cfg}(\text{body}, \text{env}, S')$ ;  $H \leftarrow H \sqcup H_h$ ;
14   $N_{\text{cycle}} \leftarrow R \sqcup E$ ;  $N \leftarrow N_{\text{cycle}} \sqcup H \sqcup \text{shared.slice}(\text{comp})$ ;
15  si  $N \sqsubseteq S$  alors
16    sortir;
17   $i \leftarrow i + 1$ ;
18  si  $i \geq 100$  alors
19     $\text{trace}[\ell] \leftarrow (i, W[\ell])$  pour tout  $\ell \in \text{labels}(S)$  non déjà tracé;  $S \leftarrow S \nabla N$ ; sortir;
20  si  $i \geq t$  alors
21    pour chaque  $\ell \in \text{changed\_labels}(N_{\text{cycle}}, S)$  non déjà tracé faire
22       $\text{trace}[\ell] \leftarrow (i, W[\ell])$ ;
23     $S \leftarrow S \nabla_T N$ ;
24  sinon
25     $S \leftarrow N$ ;

```

tour	<code>env_exit(x)</code>	effet 1	N et action
0	$[0, 0]$	$[0, 0] \sqcup ([0, 0] + 1) = [0, 1]$	$[0, 1]$; <code>iteration</code> = 1 : $S := [0, 1]$
1	$[0, 1]$	$[0, 1] \sqcup [1, 2] = [0, 2]$	$[0, 2]$; <code>iteration</code> = 2 : $S := [0, 2]$
2	$[0, 2]$	$[0, 3]$	$[0, 3]$; <code>iteration</code> = 3 : <code>widen_to</code> ($[0, 2]$, $[0, 3]$, $\{\emptyset, 1\}$) : la borne haute 3 dépasse 2 et aucun seuil n'est $\geq 3 : +\infty$; <code>widen_trace</code> [0] = {3, [1]}
3	$[0, +\infty]$	$[0, +\infty]$	$N \sqsubseteq S$: sortie, <code>iterations</code> = 3

TABLE 13.2 – Les tours de Runaway.

tour	count	count dans le <i>then</i>	effet	<i>N</i> et action
0	[0, 0]	[0, 0]	[0, 1]	join
1	[0, 1]	[0, 1]	[0, 2]	join
2	[0, 2]	[0, 2]	[0, 3]	iteration = 3 : plus petit seuil ≥ 3 : 10 ; $S := [0, 10]$; widen_trace[0]
3	[0, 10]	narrow_lt(10) : [0, 9]	[0, 10] $\sqcup$ [1, 10]	[0, 10] $\sqsubseteq S$: sortie

TABLE 13.3 – Les tours de Guarded : le seuil 10 arrête la croissance, et la garde raffinée rend l’écriture bornée.

Résultat : $s0=num[0,10]$, $effect_setter_writes = s0=num[1,10]$, un trigger {hook: 1, dep: 0, slot: (C0, 0), exact: true}. Le CLI n’émet que l’Info widening-info et verified infinite-loop : le label est bien dans widen_trace, mais l’écriture rejouée est bornée (section 13.5.2).

Dead. Un dernier déroulé montre ce qu’un seuil qui n’est pas une garde fait à la suite :

Extrait 13.32 – /tmp/ch13/dead.tsx

```

1 import { useState, useEffect } from "react";
2
3 export function Dead() {
4   const [n, setN] = useState(0);
5   const k = 3;
6   useEffect(() => {
7     if (k > 10) { setN(n + 1); }
8   });
9   return <div>{n}</div>;
10 }

```

Concrètement, $k > 10$ est toujours faux : l’effet n’écrit jamais. Abstraitement, narrow_env_for_branch raffine k à $\perp$ dans la branche, mais le bloc est tout de même exécuté (chapitre 12); setN($n + 1$) lit n , qui n’est pas raffiné, et écrit. Seuils {0, 1, 3, 10} : la suite est [0, 1], [0, 2] (joins), puis [0, 3] au seuil 3, puis [0, 10] au seuil 10, puis $[0, +\infty]$; iterations = 5, et widen_trace[0] vaut {iteration: 3, writers: [1]} (la première itération de widening, pas la dernière). Le CLI émet un WARNING infinite-loop : un faux positif, toléré par l’invariant, dont la cause est au chapitre 12 et pas dans la boucle externe.

13.4.3 iterations et widen_trace

Deux champs de l’AnalysisResult racontent la convergence. iterations est la valeur du compteur à la sortie : le nombre de tours qui ont *changé* l’état, pas le nombre de tours exécutés (il y en a toujours un de plus). Un composant dont l’état ne bouge pas au premier tour a iterations = 0 : c’est le cas de Fresh (liste 13.2), dont le store est à PerRender dès l’amorçage. widen_trace est la table des labels widenés par le cycle :

Extrait 13.33 – src/engine/analysis_result.rs, l. 18–28 — la provenance d’un widening

```

18 /// Provenance of a forced widening (ADR-019): which fixpoint iteration gave
19 /// up on convergence for a slot, and which effects were writing it then.
20 /// Feeds the `infinite-loop` / `widening-info` witness chains (`Step::Widen`).
21 #[derive(Debug, Clone, Default, PartialEq, Eq)]
22 pub struct WidenEvent {
23   /// Outer fixpoint iteration at which the slot was widened (first time).
24   pub iteration: usize,
25   /// Effects whose pass wrote this slot during the widening iteration.

```

```

26    /// Empty when the growth came from render or handlers only.
27    pub writers: Vec<HookLabel>,
28 }

```

Elle a deux lecteurs : `widening-info` (`src/rules/impls/widening_info.rs`, l. 20), qui émet une Info par label, et la première porte de `infinite-loop` (section 13.5.2). Le champ `iteration` est exact. Le champ `writers` ne l'est pas.

Piège

`writers` est censé lister « les effets dont la passe a écrit ce slot pendant l'itération du widening ». Il liste en réalité *tous les effets du composant*, pour chaque slot. La table locale `slot_writers` (liste 13.20, l. 444–446) est remplie pour chaque label de `eff_state.labels()`; or `eff_state` part de `state_store.clone()`, qui contient tous les slots amorcés (invariant 13.2). Sur `Mutual` (liste 13.22), la sonde donne `writers: [2, 3]` pour les deux slots, et la chaîne `-trace` du CLI attribue à l'effet 3 une écriture de `y` qu'il ne fait pas :

```

warn  infinite-loop [hook:1] (line 6:2) this effect keeps pushing state `y` (its deps do not provably gate
    ↳ it, so the effect can re-run every render) to new values on every run. Potential infinite render loop
    ↳ state `y` is written here [hook:2] (line 6:2)
    ↳ state `y` is written here [hook:3] (line 7:2)
    ↳ the abstract value of state `y` kept growing and was widened at iteration 3

```

Aucun verdict n'en dépend : c'est une imprécision de *provenance*, pas de soundness. Le correctif naturel serait de ne retenir que `eff_state.changed_labels(&state_store)`.

13.4.4 Terminaison

Théorème 13.1: Terminaison de la boucle externe

Pour tout composant et tout ensemble fini de seuils T , la boucle de `analyze_component_impl` s'arrête. Preuve. La suite (S_k) est croissante (proposition 13.1) et, à partir du tour t , chaque pas est un widening $S \nabla_T N$ point à point sur des valeurs `StateValue`. Il suffit donc que toute chaîne $v_0 \sqsubseteq v_0 \nabla_T v_1 \sqsubseteq \dots$ de `StateValue` se stabilise, composante par composante (liste 13.30) :

- `num` : `Interval::widen_to` ne fait que relâcher les bornes vers un seuil de T ou vers $\pm\infty$; chaque borne visite chaque seuil au plus une fois avant $\pm\infty$ (`src/domains/impls/interval.rs`, l. 112–146, ADR-14) : au plus $2(|T| + 1)$ pas;
- `str` : `StrConst::widen` bascule à $\top$ au-delà de `STR_WIDEN_THRESHOLD = 4` constantes (`src/domains/impls/str_const.rs`, l. 8);
- `reference` : `Stability::widen` est le join (`src/domains/impls/stability.rs`, l. 144–146) sur un treillis de hauteur finie, les ensembles `Versioned` étant plafonnés à `VERSIONED_LABELS_THRESHOLD = 4` labels (`src/domains/impls/stability.rs`, l. 11);
- `boolean`, `null`, `undef`, `other`, `setter` : treillis finis.

Le nombre de labels est fini (fixé par l'expansion), donc le store se stabilise, et le test $N \sqsubseteq S$ réussit. Le garde-fou à cent tours n'est pas nécessaire à l'argument; c'est une ceinture de sécurité.

Les $t - 1$ premiers pas sont des joins, en nombre borné par t ; le tas et le memo store ne participent pas au test et ne peuvent pas retarder l'arrêt. La complexité est celle de $O(k)$ tours, chacun coûtant une passe de `analyze_cfg` sur le rendu, chaque effet et chaque handler, avec k borné par t plus la hauteur effective ci-dessus. Le dépôt ne publie pas, au commit e67b10a, de distribution de iterations sur le corpus. Sur les exemples de ce chapitre, la valeur la plus haute est `iterations = 5` (Dead), atteinte en franchissant deux seuils avant $+\infty$.

Piège

Le garde-fou n'est jamais exercé : `grep 'iteration >= 100'` ne trouve que `src/engine/fixpoint.rs`, l. 500, aucun test. S'il était atteint, le résultat serait $S \nabla N$ sans re-vérification : un widening est au-dessus du join de ses opérandes, donc au-dessus de N , mais rien ne garantit qu'un tour de plus depuis ce nouvel état n'ajouterait rien ; le résultat pourrait sous-approximer d'un tour. Par le théorème 13.1, ce cas ne devrait pas se produire ; il n'est pas démontré qu'il ne se produit pas.

13.5 Après la convergence

Le store a convergé ; il reste à produire ce que les règles liront. Deux passes supplémentaires et une tranche de calculs relationnels y pourvoient (`src/engine/fixpoint.rs`, l. 532–698).

13.5.1 Rafraîchir le rendu

Extrait 13.34 – `src/engine/fixpoint.rs`, l. 532–563 — une passe de rendu de plus, sur les stores convergés

```

532 // -- Post-convergence: refresh the render pass -----
533 // Each iteration recomputes the memo store *after* its render pass, so the
534 // last pass – and everything the rules read from it: `env_exit`,
535 // `block_states`, and every object literal allocated into the heap – froze
536 // the memo values of the *previous* iteration. In a component whose state
537 // converges on the first pass that means  $\top$  for every `useCallback`, and a
538 // provably-stable callback then reads as "may change between renders". One
539 // more pass over the converged stores is what the rules actually want.
540 // `inter` is `None`: this re-evaluates and re-allocates, it does not
541 // re-analyze children (same choice as the effect re-run below).
542 {
543     let refresh_ctx = FixpointCtx {
544         state: &state,
545         memo: &memo_store,
546         callbacks: &callback_bodies,
547     };
548     let (bs, _) = analyze_cfg::

```

Le commentaire dit pourquoi. Dans la boucle, le recalcul des mémos *suit* la passe de rendu : la dernière passe — et tout ce que les règles en lisent, `env_exit`, `block_states`, les objets alloués dans le tas — a donc figé les mémos de l'itération *précédente*. Pour un composant qui converge au premier tour, cela signifie $\top$ pour chaque `useCallback`, et un callback prouvablement stable se lirait « peut changer entre deux rendus ». Une passe de plus sur les stores convergés donne

aux règles ce qu'elles veulent. Deux détails : `inter` est `None` (la passe ré-évalue et ré-alloue, elle ne ré-analyse pas les enfants), et le store de sortie de cette passe est jeté (`let (bs, _) = ...`) — l'état convergé est S , pas un S enrichi d'un tour de plus.

Invariant 13.3: D'où vient chaque champ de l'`AnalysisResult`

`state_store` est le S convergé de la boucle. `block_states` et `env_exit` (donc tout ce qui est évalué « dans l'environnement de sortie du rendu » : triggers, sites, sondes des règles) viennent de la passe de *rafraîchissement*. `effect_block_states` et `handler_block_states` viennent de la *dernière itération* de la boucle (écrasés à chaque tour, l. 443 et 480). `memo_store` est le dernier recalcul, effectué au dernier tour. `heap` a été muté en place par toutes les passes, rafraîchissement et ré-exécution des effets compris. `effect_setter_writes` vient de la passe décrite ci-dessous.

13.5.2 Les écritures pures des effets

Extrait 13.35 – `src/engine/fixpoint.rs`, l. 565–594 — rejouer les effets depuis un store $\perp$ pour isoler ce qu'ils écrivent

```

565 // -- Post-convergence: pure setter writes -----
566 // Re-run effects from  $\perp$  so `effect_setter_writes` contains only what setters
567 // actually wrote. InfiniteLoop uses this to distinguish bounded growth (narrowing
568 // held:  $[1,10]$ ) from true divergence ( $[1,+\infty]$ ).
569 let final_state = state;
570 let final_ctx = FixpointCtx {
571     state: &final_state,
572     memo: &memo_store,
573     callbacks: &callback_bodies,
574 };
575 let bottom_state: StateStore<StateValue> = StateStore::bottom();
576 let mut effect_setter_writes: StateStore<StateValue> = StateStore::bottom();
577 for hook in &hooks {
578     if let HookEntry::Effect { body_cfg, .. } = hook {
579         let (_, pure_writes) = analyze_cfg::<T>(
580             comp_id,
581             body_cfg,
582             env_exit.clone(),
583             &bottom_state,
584             &memo_store,
585             transfer,
586             config.widen_threshold,
587             &thresholds,
588             &mut heap,
589             &final_ctx,
590             None,
591         );
592         effect_setter_writes = effect_setter_writes.join(&pure_writes);
593     }
594 }
```

Le store convergé de *Guarded* est $[0, 10]$; celui de *Runaway* est $[0, +\infty]$; tous deux sont dans `widen_trace`. Pour les distinguer, il faut savoir non pas *ce que vaut* le slot, mais *ce que l'effet y écrit* : $[1, 10]$, borné par la garde, contre $[1, +\infty]$. C'est ce que calcule la ré-exécution de chaque effet depuis un store $\perp$, dans le même `env_exit` et contre les mêmes mémos : le store de sortie ne contient que les écritures des setters, sans la valeur préexistante. Les deux portes du bras intra de *infinite-loop* lisent les deux champs dans l'ordre :

Extrait 13.36 – `src/rules/impls/infinite_loop.rs`, l. 128–136 — les deux portes du bras intra

```

128         if let Some(&state_label) = local_setter_labels.get(&call.var) {
129             // -- Intra -----
130             if !comp_result.widen_trace.contains_key(&state_label) {
131                 continue; // state didn't diverge → bounded
132             }
133             let writes = comp_result.effect_setter_writes.get(state_label);
134             if !writes.is_bottom_value() && !writes.is_unbounded() {
135                 continue; // write bounded → narrowing held the growth
136             }

```

avec le prédicat de divergence :

Extrait 13.37 – `src/domains/impls/state_value.rs`, l. 371–382 — ce que « non borné » veut dire pour une valeur du produit

```

371     /// True if this value represents unbounded growth in the fixpoint.
372     ///
373     /// Used by `InfiniteLoop` to distinguish a setter that writes a bounded value
374     /// (branch narrowing held the growth) from one that truly diverges:
375     /// - Infinite interval bounds → numeric counter without a binding guard
376     /// - Per-render reference slot → new object/function literal every render
377     /// - Residual  $\top$  → precision lost, conservatively unbounded
378     pub fn is_unbounded(&self) -> bool {
379         (!self.num.is_bottom() && (self.num.lo.is_infinite() || self.num.hi.is_infinite()))
380         || self.reference == Stability::PerRender
381         || self.other
382     }

```

composant	state_store	tracé	effect_setter_writes	porte 2	verdict
Runaway	$[0, +\infty]$	oui	$[1, +\infty]$	non borné	WARNING
Guarded	$[0, 10]$	oui	$[1, 10]$	borné	absous
Lazy	$[7, 20]$	oui	$[8, 20]$	borné	absous
Dead	$[0, +\infty]$	oui	$[1, +\infty]$	non borné	WARNING (FP)
Loader	$[0, 42]$	non	$[42, 42]$	—	absous, porte 1
MemoLoop	$\top$	non	ref:Versioned	—	absous, porte 1 (FN)
Updater (intra)	$[0, +\infty]$	oui	$\perp$	$\perp$: ne pas absoudre	WARNING

TABLE 13.4 – Les deux portes de `infinite-loop` sur les exemples du chapitre (valeurs relevées par la sonde, verdicts du CLI). Updater est donné sur le chemin intra ; le chemin du CLI est au section 13.7.

L’ADR-14 qualifie ce bloc de « crude precision-recovery hack » et prévoyait de le supprimer avec le narrowing ; le narrowing ayant été abandonné, le bloc est resté, et il est le seul mécanisme du moteur qu’un ADR désigne ainsi. Il a un effet de bord à connaître.

Piège

Un updater fonctionnel rejoué depuis $\perp$ écrit $\perp$. Le premier bras de `exec_setter_call` (`src/domains/interp/interpreter.rs`, l. 352–360) lie le paramètre de `setX(c => c + 1)` à `ctx.state.get(label)`, qui vaut $\perp$ dans le store `bottom_state` ; $\perp + 1 = \perp$. C’est exactement ce que montre Updater en intra : `effect_setter_writes = s0=⊥` alors que `state_store = s0=num[0,inf]`. La règle en tient compte : la porte 2 n’absout que si `!writes.is_bottom_value()` — $\perp$ signifie « je ne sais pas ce qui a été écrit », pas « rien n’a été écrit ». Les tests

de `tests/functional_updater.rs` (`functional_updater_without_deps_is_flagged`, etc.) fixent ce comportement, sur le chemin `intra`.

13.5.3 La tranche des relations

Le reste de la fonction (`src/engine/fixpoint.rs`, l. 596–667) ne calcule plus rien sur le store : il dérive des *relations* sur les CFG, dans les environnements convergés. Le moteur n'est ici qu'un ordonnanceur ; le chapitre 14 détaille chaque relation. L'ordre compte : `collect_hook_calls`, `collect_effect_info`, `collect_handler_info` d'abord (l. 596–598) ; puis les setters *étrangers* (un setter reçu en prop d'un parent, qualifié par son propriétaire, l. 601–605) et les environnements par région (`SiteEnvs`, l. 608–618) dont `slot_writers` a besoin ; `slot_seeds` après, parce qu'elle replie les lignes de `slot_writers` et les deps des effets (l. 627–642) ; `registrations` (l. 646) ; enfin `effect_triggers` :

Extrait 13.38 – `src/engine/fixpoint.rs`, l. 647–667 — la relation `effect_triggers`, évaluée dans `env_exit` contre le store convergé

```

647 // The effect-trigger relation (ADR-042 §3): a dep evaluated in the render
648 // exit env, the one every hook body starts from.
649 let effect_triggers = {
650     let mut scratch = heap.clone();
651     crate::engine::triggers::collect_effect_triggers(
652         comp_id,
653         &render_cfg,
654         &hooks,
655         &memo_store,
656         |e| {
657             crate::engine::eval::eval_in_stores(
658                 e,
659                 &env_exit,
660                 comp_id,
661                 &final_state,
662                 &memo_store,
663                 &mut scratch,
664             )
665         },
666     )
667 };

```

Chaque dep d'un effet est évalué par `eval_in_stores` (chapitre 12) dans l'environnement de sortie du rendu rafraîchi, contre `final_state` et le memo store, sur une copie jetable du tas. Un dep qui *est* un slot donne une ligne exact: `true` — l'effet *doit* se relancer à toute écriture fraîche, un fait *must* —, un dep dont la stabilité est `Versioned(S)` donne une ligne exact: `false` par slot de `S` — l'effet *peut* se relancer, un fait *may* (définition 10.3, ADR-17). Sur `Guarded`, `Fresh`, `Lazy` la ligne est exacte (le dep est la variable d'état) ; sur `MemoLoop` elle est versionnée (le dep est le mémo) ; sur `Runaway` et `Loader` il n'y en a pas (pas de liste de deps, ou liste vide).

La fonction se termine en assemblant le résultat :

Extrait 13.39 – `src/engine/fixpoint.rs`, l. 670–698 — l'`AnalysisResult` rendu

```

670 AnalysisResult {
671     component: comp_id,
672     file: comp_file,
673     param: comp_param,
674     dom_props: comp_dom_props,
675     module_consts,
676     state_store: final_state,

```



```

677     memo_store,
678     block_states,
679     effect_block_states,
680     hook_calls,
681     effect_info,
682     handler_block_states,
683     handler_info,
684     widen_trace,
685     inline_origins,
686     effect_setter_writes,
687     custom_arg_returns,
688     render_cfg,
689     hooks: hooks_clone,
690     hook_provenance,
691     slot_writers,
692     slot_seeds,
693     registrations,
694     effect_triggers,
695     iterations: iteration,
696     heap,
697 }
698 }

```

13.5.4 Le programme : phases 1 et 2

`analyze_program` (`src/engine/fixpoint.rs`, l. 704–819) appelle `analyze_component_impl` sur chaque composant, en deux phases :

Extrait 13.40 – `src/engine/fixpoint.rs`, l. 757–794 — la photographie de la phase 1 et le balayage intra de la phase 2

```

757 // Everything phase 1 reached: the roots plus every component
758 // `eval_comp_app` analysed top-down under an `InterCtx`. Snapshotted HERE,
759 // before the sweep below adds intra-only results that record no call-graph
760 // edges and would otherwise be indistinguishable from genuine roots (#110).
761 let phase1_reached: std::collections::HashSet<ComponentId> =
762     results.borrow().keys().copied().collect();
763
764 // Phase 2: analyze unreachable components (props=T, intra only). Skip
765 // those already in `results` inter-component pass inserted precise results.
766 let remaining: Vec<ComponentId> = registry
767     .table()
768     .ids()
769     .filter(|id| !analysed.contains(id))
770     .collect();
771 for id in remaining {
772     let Some(key) = registry.key_of(id) else {
773         continue;
774     };
775     if let Some(ir) = registry.ir_for(&key) {
776         if results.borrow().contains_key(&id) {
777             continue;
778         }
779         // Phase 2 is intra-only, but the component still has to record its
780         // own identity – a `Versioned` label it mints here is compared
781         // against phase-1 results.
782         let result = analyze_component_impl(
783             ir,
784             id,
785             &StateValueTransfer,

```

```

786         config,
787         AbstractEnv::bottom(),
788         Heap::new(),
789         None,
790     );
791     stats.borrow_mut().components_analyzed += 1;
792     results.borrow_mut().insert(id, result);
793 }
794 }
```

En *phase 1*, chaque racine choisie par la `RootStrategy` (Heuristic : les composants qui n'apparaissent dans aucun `CompApp`; `-all-roots`; `-entry`) est analysée avec un `InterCtx`. Quand son rendu rencontre `<Child .../>`, `eval_comp_app` évalue les props, consulte un cache par égalité des props abstraites, et sur échec appelle `analyze_component_inter` sur l'enfant, dont le résultat remplace celui déjà présent dans `results` (chapitre 11). Le rendu du parent étant rejoué à chaque tour de *sa* boucle, l'enfant est analysé à chaque *cache miss*; c'est le résultat du dernier miss qui reste. Il n'y a pas de point fixe de niveau programme : « no separate program-level fixpoint layer » (ADR-12). Un enfant qui écrit un slot du parent le fait dans le `SharedStateStore`, et la boucle du parent l'importe au test de convergence suivant (liste 13.25).

En *phase 2*, chaque composant que la phase 1 n'a pas atteint est analysé *intra* (`inter = None`, props `⊥` par défaut d'environnement), avec son vrai `ComponentId`. L'ensemble `phase1_reached` est photographié *avant*, pour ne pas confondre un composant analysé en *intra* avec une racine (#110). Quatre conséquences de `inter = None`, toutes lues dans le code et vérifiées :

1. les hooks personnalisés ne sont pas greffés (l. 898) ;
2. le `SummaryRegistry` n'est pas consulté non plus (le repli est *dans* `expand_custom_hooks`, après ce `return`) : un `useQuery()` de phase 2 reste un marqueur opaque, signalé par une `Info analysis-limit` ;
3. tout `<Enfant .../>` est un enfant inconnu : `eval_comp_app` commence par `havoc_setter_props` (`src/domains/transfer/state_value.rs`, l. 477–483), donc tout `setter` passé en prop reçoit `⊥` — `sound` ;
4. les utilitaires sont bien greffés (cela ne dépend que de `config.function_registry`), mais la troncature de budget est silencieuse (section 13.2.2).

Le chapitre 16 reprend la phase 1 en détail : cache, graphe d'appels, récursion, dépendances de rendu.

13.6 Correspondance avec React-tRace et argument de *soundness*

13.6.1 Un tour abstrait, quatre transitions concrètes

L'ADR-1 fait de React-tRace la sémantique concrète du projet, avec une justification qui concerne directement ce chapitre : « the Tree Memory + render loop model (`StepInit` → `StepEffect` → `StepCheck`) directly corresponds to the fixpoint iteration of our abstract interpreter ». Le chapitre 2 a donné la correspondance règle par règle, dans sa table de correspondance ; la tableau 13.5 la donne *par passe*, avec ce que chaque passe abstrait.

Rappelons ce que le chapitre 2 a constaté : React-tRace est la référence *conceptuelle* du moteur, pas un oracle outillé. Aucune fonction de `src/engine/` ne cite une règle de l'article, et aucun test ne compare la boucle à l'interpréteur OCaml sur les exemples du papier. La correspondance de la tableau 13.5 est donc une lecture, faite ici, et non une propriété vérifiée

pas de la boucle	React-tRace	ce qui est abstrait
amorçage $S[\ell] \sqsubseteq \llbracket init \rrbracket$	STTBIND (phase Init)	rien : une fois, hors boucle
pas de rendu depuis S	STEPINIT puis les ré-évaluations SUCC de STEPCHECK (EVALONCE/EVALMULT)	tous les rendus possibles d'un coup ; STTREBIND devient la lecture du store joint, convertie en Versioned (src/domains/transfer/state_value.rs, l. 122)
recalcul des mémos	extension esquissée au §7 de l'article	valeur = stabilité seulement
passes d'effets depuis env_exit	STPEFFECT : COMMITTEFFSPATH sur les vues à décision Effect	<i>tous</i> les effets, ordre abstrait par le join ; fermeture = env_exit (EFF) ; APPSETNORMAL = mise à jour faible
passes de handlers	STPEVENT (mode <i>event loop</i>)	0..N exécutions, tout ordre ; hors widen_trace
tranche du SharedStateStore test $N \sqsubseteq S$	APPSETNORMAL vers la vue $m[p]$ d'un autre chemin STEPCHECK : CHECKNOEFFECT partout $\Rightarrow$ <i>event loop</i>	import différé au test de convergence « un état a changé » devient « le store a grossi »
widen_trace	le cycle rendered $\rightarrow$ check $\rightarrow$ rendered qui ne passe jamais par <i>event loop</i> (§3.1.1)	un signal abstrait : la croissance a dû être forcée à converger

TABLE 13.5 – Les passes de analyze_component_impl et les transitions de React-tRace (chapitre 2).

par le dépôt.

13.6.2 L'argument de soundness

Théorème 13.2: Soundness de la boucle externe (esquisse)

Soit un composant, et pour tout k soit ρ_k l'ensemble des stores d'état concrets (une valeur par slot) que React peut atteindre après au plus k tours — rendu, commit, effets, puis un nombre quelconque d'événements — depuis le montage, pour toutes les props et toutes les suites d'événements. Sous les hypothèses

- H1** les fonctions de transfert sont correctes (chapitre 11) et analyze_cfg calcule, pour chaque corps, un environnement de sortie et un store qui sur-approximent tout chemin (chapitre 12) ;
- H2** toute écriture d'un slot passe par exec_setter_call, qui la joint au store (mise à jour faible, src/domains/stores/state_store.rs, l. 29–33) ;
- H3** tout corps qui peut s'exécuter est l'un des corps passés : rendu, effets, handlers, callbacks descendus par l'interpréteur ;

on a $\alpha(\rho_k) \sqsubseteq S_k$ pour tout k (en notant S_k le store après k tours abstraits, prolongé par S_∞ à l'arrêt), donc $\alpha(\bigcup_k \rho_k) \sqsubseteq S_\infty$: le store convergé contient tout état atteignable.

Esquisse. Par récurrence sur k . Pour $k = 0$, ρ_0 est l'état au montage, produit des initialiseurs, et S_0 les évalue tous (liste 13.13). Supposons $\alpha(\rho_k) \sqsubseteq S_k$. Un tour concret de plus part d'un état $\sigma \in \rho_k$; le rendu lit σ , que S_k couvre, donc par H1 l'environnement de fin de rendu concret est dans $\gamma(\text{env_exit})$ et toute écriture du rendu est dans R . Les effets qui tournent (quels que soient leurs deps) partent de cet environnement et de σ ; chaque écriture, dans n'importe quel ordre et quel que soit le rejeu de la file, est une valeur que analyze_cfg sur le corps a jointe dans E (H1, H2), et l'updater lit un ctx.state.get qui couvre la file.

Les handlers, exécutés zéro ou plusieurs fois, ne produisent que des valeurs jointes dans H ; un événement supplémentaire après un tour part d'un état déjà couvert par N , et N est à nouveau un point de départ couvert au tour suivant, ce qui rend compte des suites d'événements de longueur quelconque. Un setter de ce composant appelé dans un autre composant écrit dans la tranche importée. Donc $\alpha(\rho_{k+1}) \sqsubseteq N_k \sqsubseteq S_{k+1}$, la dernière inégalité parce que S_{k+1} vaut N_k ou $S_k \nabla N_k \sqsupseteq N_k$ (`widen_to` $\sqsupseteq$ `join`, `src/domains/impls/interval.rs`, l. 112). À l'arrêt, $N \sqsubseteq S$ ferme la récurrence pour tout k ultérieur. $\square$

Les trois hypothèses ne sont pas gratuites, et c'est précisément là que se logent les trous du section 13.7 : H2 est violée par le second bras de `exec_setter_call` en `inter` et par le `lowering` de `useReducer` ; H3 est violée par les fonctions de nettoyage, que la boucle n'exécute pas. La tableau 13.6 liste, pour mémoire, les mécanismes sur lesquels l'argument s'appuie.

mécanisme	direction garantie	où
mise à jour faible des setters	$S \sqsupseteq$ toute valeur écrite	<code>src/domains/stores/state_store.rs</code> , l. 29–33
passes depuis <code>state.clone()</code> tous les effets à chaque tour	suite croissante aucun effet oublié à cause des deps	<code>src/engine/cfg_analyzer.rs</code> , l. 49 <code>src/engine/fixpoint.rs</code> , l. 418–449
handlers dans la boucle	valeurs atteignables par événements incluses	<code>src/engine/fixpoint.rs</code> , l. 451–483
environnement d'effet = <code>env_exit</code> <code>widen_to</code> $\sqsupseteq$ <code>join</code>	l'effet voit toute valeur de fin de rendu post-point fixe	<code>src/engine/fixpoint.rs</code> , l. 432 <code>src/domains/impls/interval.rs</code> , l. 112–146
<code>absent</code> = $\top$, <code>join</code> d'un seul côté = $\top$	pas de valeur inventée	chapitre 11
mémo à deps illisibles = <code>Unknown</code>	pas de mémo figé inventé	<code>src/domains/transfer/state_value.rs</code> , l. 67
enfant inconnu : havoc des setters en props	l'enfant peut appeler tout setter reçu	<code>src/domains/transfer/state_value.rs</code> , l. 477–483
budget épuisé $\rightarrow$ Info, assurances suspendues (phase 1)	pas de <code>verified</code> sur du code non lu	<code>src/engine/fixpoint.rs</code> , l. 1620–1628

TABLE 13.6 – Où la soundness de la boucle est tenue.

13.7 Ce que le moteur garantit, ce qu'il ne garantit pas

`docs/limitations.md` distingue deux registres : un *défaut* (« the analysis returns an under-approximation or reports something false ») se corrige ; un *compromis* (l'analyse reste sound, elle est seulement imprécise) se décide. Les constats qui suivent ont tous été observés au commit e67b10a sur les fichiers de `/tmp/ch13/`, et classés selon ce critère. Plusieurs ont déjà été rencontrés en chemin ; nous les rappelons pour que la liste soit complète.

Piège

L'updater fonctionnel, sur le chemin du CLI (défaut, FN).

Extrait 13.41 – `/tmp/ch13/updater.tsx`

```
1 import { useState, useEffect } from "react";
2
```

```

3 export function Updater() {
4   const [x, setX] = useState(0);
5   useEffect(() => {
6     setX((c) => c + 1);
7   });
8   return <div>{x}</div>;
9 }

```

Concrètement, c'est Runaway écrit avec un updater : la boucle est infinie. En intra, le moteur la voit (tableau 13.4). Par `analyze_program`, la sonde donne `iterations = 2`, `state_store = s0=⊤`, `widen_trace = []` et `shared_state = {(ComponentId(0), 0): ref(PerRender)}`, et le CLI :

```

[verbose] Updater: 2 iteration(s), widened: []
1 clean component(s) hidden, rerun with --show-clean

```

```

✓ 1 file(s) no issues found.

```

Le mécanisme est le défaut D3 du chapitre 11 : le second bras de `exec_setter_call` (`src/domains/interp/interpreter.rs`, l. 368–391) s'applique aussi au propre setter du composant dès que `inter` est présent, et il évalue l'argument comme une *valeur* — la fermeture `c => c + 1`, une référence `PerRender` — qu'il écrit dans le `SharedStateStore`. La tranche est jointe à N (liste 13.25), c devient « nombre ou référence », $c + 1$ vaut $\top$, et le slot atteint $\top$ en deux tours *par join* : pas de widening, `widen_trace` vide, porte 1 fermée. Deux causes se composent : le bras croisé ne devrait pas s'appliquer aux slots propres (ou devrait y exécuter l'updater comme le premier bras), et le signal de divergence ignore un slot qui atteint $\top$ sans widening. Les tests de `tests/functional_updater.rs` passent parce qu'ils appellent `analyze_component`. Aucune issue ouverte ne décrit ce cas au commit étudié.

Piège

$\top$ atteint par join n'est pas un signal (défaut, FN). `MemoLoop` (exemple 13.2) boucle concrètement et le CLI le déclare propre : le slot saute à $\top$ au tour 0 parce que le mémo n'est pas encore calculé, et la porte 1 de `infinite-loop` ne connaît que `widen_trace`. Le prédicat `is_unbounded`, qui compte pourtant `other` ($\top$ résiduel) comme non borné, n'est consulté qu'après la porte 1. Le bras `churn` lit l'écriture comme non fraîche (#157). Deux exemples du chapitre (`Updater` en `inter`, `MemoLoop`) montrent que « widené » et « a divergé » ne coïncident pas : un slot peut diverger vers $\top$ en un ou deux joins, et ce chemin ne laisse aucune trace.

Piège

Les fonctions de nettoyage n'écrivent pas dans le store (défaut).

Extrait 13.42 – `/tmp/ch13/cleanup.tsx`

```

1 import { useState, useEffect } from "react";
2
3 export function CleanupDeps({ id }: { id: string }) {
4   const [n, setN] = useState(0);
5   useEffect(() => {
6     return () => setN(7);
7   }, [id]);
8   return <div>{n}</div>;
9 }

```

À chaque changement de `id`, React exécute le nettoyage, donc `setN(7)`, puis rend avec $n = 7$. L'IR de l'effet est `Return(FnLit { ... setN(7) ... });` `exec_expr_effects` sur un `FnLit` ne l'exécute pas. Sonde : `iterations = 0`, `state_store = s0=num[0,0]` — la valeur

7 n'est pas dans le store. C'est une sous-approximation de l'état, non répertoriée dans docs/limitations.md. La relation `slot_writers` voit la ligne (`region: Effect(1)`, `phase: Cleanup`, valeur `number[7, 7]`, relevé par la sonde; chapitre 14), ce qui protège les règles qui la lisent. Sous `src/rules/`, trois règles lisent `state_store` directement : `redundant-set-state`, `always-unstable-deps` et `wasted-subtree-render`. La première écarte d'office un slot écrit depuis deux régions distinctes (l'ensemble contesté, `src/rules/impls/redundant_set_state.rs`, l. 57–81). Nous l'avons vérifié : un nettoyage `setN(7)` plus un `setN(0)` dans un autre effet ou dans un `onClick` ne produit aucun diagnostic. Au commit `e67b10a`, nous n'avons pas construit de cas où l'absence du 7 trompe l'une des deux autres. Le trou dans le store demeure.

Piège

useReducer : l'action à la place de l'état (défaut).

Extrait 13.43 – `/tmp/ch13/reducer.tsx`

```
1 import { useReducer, useEffect } from "react";
2
3 export function Reducer() {
4   const [s, dispatch] = useReducer((acc: number, a: number) => acc + a, 0);
5   useEffect(() => {
6     dispatch(1);
7   });
8   return <div>{s}</div>;
9 }
```

Le lowering traduit `useReducer` en `HookEntry::State` et *ignore le réducteur* (`let _reducer = it.next(); // skip reducer fn, src/lowering/hook_extractor.rs, l. 715–719`); `dispatch` est lié comme un setter. Le moteur joint donc l'action 1 au slot : `s0=num[0,1]`, `iterations = 1`, `widen_trace = []`. Concrètement `s` vaut 0, 1, 2, 3, ... et le composant boucle; le CLI répond ✓ 1 file(s) no issues found. Pour être sound, il faudrait exécuter le réducteur (`exec_body` avec (état, action)) ou écrire `⊥`. Aucune issue ne le décrit au commit étudié.

Piège

Trois compromis déjà rencontrés. Une branche dont la garde est réfutée est exécutée quand même (Dead, section 13.4.2) : faux positif, sound. `WidenEvent.writers` liste tous les effets (section 13.4.3) : provenance inexacte, aucun verdict touché. Un second appel du même hook personnalisé reste opaque, avec un message trompeur (section 13.2.2) : sound, assurances suspendues. À quoi s'ajoutent deux constats sur la couche de certification : la troncature d'inlining silencieuse en phase 2 (section 13.2.2, un défaut au sens des assurances publiées) et le garde-fou à cent tours jamais exercé (section 13.4.4).

Résumé du chapitre

- `analyze_component_impl` (`src/engine/fixpoint.rs`) ne simule pas React : elle calcule un post-point fixe du `StateStore`, un sur-ensemble des états atteignables par tous les tours et toutes les suites d'événements. Un tour abstrait représente tous les tours concrets de même rang.
- *Préparation*, une fois : constantes de module dans `initial_env` et `module_env`; greffes des utilitaires puis des hooks personnalisés (`SpliceIds` monotone, garde par nom, résumés par `retag_marker`); seuils = tous les littéraux numériques; `callback_bodies`; amorçage des `useState`, initialiseur paresseux exécuté.
- *Un tour* : rendu depuis `initial_env`; mémos recalculés depuis `env_exit` (valeur = stabilité,

constat	exemple	registre	référence
updater fonctionnel écrit comme fermeture dans le store partagé	Updater	défaut (FN)	chapitre 11 D3; aucune issue
$\top$ par join sans widening : porte 1 fermée	MemoLoop, Updater	défaut (FN)	#157, #144
nettoyages hors du store	CleanupDeps	défaut	non documenté
useReducer : action jointe, réducteur ignoré	Reducer	défaut	non documenté
troncature d'inlining silencieuse en phase 2	A/B	défaut (assurances)	non documenté
branche réfutée exécutée	Dead	compromis (FP)	chapitre 12
writers = tous les effets	Mutual	compromis (provenance)	section 13.4.3
second appel d'un hook personnalisé opaque	TwoClocks	compromis	src/engine/fixpoint.rs, l. 884
useState(makeInit) amorcé à $\top$	LazyVar	compromis (FP)	section 13.2.5
cap à 100 sans re-vérification ni test	—	risque théorique	src/engine/fixpoint.rs, l. 500

TABLE 13.7 – Les constats du chapitre, classés selon les deux registres de docs/limitations.md.

$\top$ au tour 0, hors test de convergence); tous les effets depuis env_exit et le même store (deps ignorées, batching abstrait); handlers de même, hors signal; $N = R \sqcup E \sqcup H \sqcup$ tranche partagée; arrêt si $N \sqsubseteq S$, ce qui équivaut à $N = S$.

- *Convergence* : compteur global des tours qui changent l'état; joins jusqu'à widen_threshold = 3, puis widen_to à seuils sur tout le store, widen_trace sur les seuls labels changés par $R \sqcup E$ (première itération retenue); garde-fou à 100. Terminaison par hauteur finie de chaque composante de StateValue.
- *Post-convergence* : passe de rafraîchissement (mémos à jour pour les règles, inter = None); effect_setter_writes depuis $\perp$ (porte 2 de infinite-loop; un updater y écrit $\perp$); tranche des relations, dont effect_triggers (exact = must, versionné = may).
- *Deux moteurs* : intra (analyze_component, phase 2) sans greffe de hooks ni résumés ni store partagé; inter (phase 1) avec.
- *Trous au commit e67b10a* : updater en inter (FN), $\top$ par join (FN), nettoyages et useReducer hors du store, troncature silencieuse en phase 2; compromis : branche réfutée exécutée, writers imprécis, second hook opaque.

Types et fichiers rencontrés : Config et SpliceIds (src/engine/fixpoint.rs); AnalysisResult et WidenEvent (src/engine/analysis_result.rs); StateStore, MemoStore et SharedStateStore (src/domains/stores/); EffectTrigger (src/engine/triggers.rs); HookRegistry et FunctionRegistry. Le chapitre 14 reprend la dernière tranche de la fonction : ce que sont slot_writers, slot_seeds, registrations et effect_triggers, et ce qu'une règle a le droit d'en certifier.

Exercices

Exercice 13.1: Un compteur gardé à pas de deux

Dérouler la boucle externe pour `useEffect(() => { if (count < 3) setCount(count + 2); }, [count])` avec `useState(0)`. Quels sont les seuils? Quelle valeur finale, quel `iterations`, quel `widen_trace`? Vérifier avec le CLI.

Solution

Seuils $\{0, 2, 3\}$. Tour 0 : dans la branche, `count` est raffiné à $[0, 0]$, l'effet écrit $[2, 2]$, $N = [0, 2]$. Tour 1 : `count = [0, 2]`, raffiné par < 3 à $[0, 2]$, écriture $[2, 4]$, $N = [0, 4]$. Tour 2 : `count = [0, 4]`, raffiné à $[0, 2]$, écriture $[2, 4]$, $N = [0, 4] \sqsubseteq S$. Arrêt sans widening : `iterations = 2`, `s0=num[0,4]`, `widen_trace = []`, `effect_setter_writes = s0=num[2,4]` (sonde, `/tmp/ch13/exo1.tsx`). CLI: Exo: 2 iteration(s), widened: [], aucun diagnostic. Le raffinement de garde a suffi; le seuil 3 n'a pas servi.

Exercice 13.2: Seuil bas

Refaire `Counter` (exemple 13.5) et `Runaway` avec `widen_threshold = 1`. Que devient chaque tableau? Comparer au test `widened_labels_triggered_with_low_threshold` (`src/engine/fixpoint.rs`, l. 2011).

Exercice 13.3: Égalité

La proposition 13.1 conclut que le test `new_state.leq(&state)` équivaut à $N = S$. Où l'implémentation `StateStore::leq` (liste 13.27) diffère-t-elle d'un test d'égalité structurelle, et pourquoi la différence est-elle sans effet dans la boucle? (Considérer un label présent dans S et absent de N , et deux valeurs incomparables.)

Exercice 13.4: Provenance

Construire un composant où deux effets écrivent deux slots distincts, observer `WidenEvent.writers` avec `-trace`, puis écrire le correctif de la l. 444 de `src/engine/fixpoint.rs` qui ne retient que les labels réellement changés par la passe. Quel test du dépôt faudrait-il ajouter?

Exercice 13.5: Zéro itération

Expliquer pourquoi `Fresh` (liste 13.2) converge avec `iterations = 0`, ce que valent `widen_trace` et `effect_setter_writes`, et par quelle chaîne de faits l'ERROR est tout de même produite (lire les chapitres 14 et 15).

Exercice 13.6: Intra contre inter

Reproduire `Updater` (liste 13.41) avec `analyze_component` puis avec `analyze_program`, et suivre la valeur écrite dans le `SharedStateStore` depuis `src/domains/interp/interpreter.rs`, l. 368–391 jusqu'à `src/engine/fixpoint.rs`, l. 488–493. Proposer un correctif au niveau central, et dire lequel des deux défauts composés il corrige.

Exercice 13.7: Terminaison

Écrire l'argument de terminaison complet en donnant, pour chaque composante de `StateValue`, une borne explicite sur la longueur d'une chaîne $v \sqsubseteq v \nabla_T v' \sqsubseteq \dots$ en fonction de $|T|$, `STR_WIDEN_THRESHOLD` et `VERSIONED_LABELS_THRESHOLD`. En déduire une borne sur `iterations`, et la comparer au cap de 100.

Chapitre 14

Les relations du moteur

Objectifs du chapitre.

- Comprendre pourquoi les faits que lisent les règles (« A écrit B », « A se ré-exécute quand B bouge ») sont calculés une fois, dans le moteur, après convergence, et jamais re-dérivés par une règle.
- Savoir lire la *polarité* d'une colonne (exact, must, may, $\top$) et en déduire ce qu'une règle a le droit de certifier.
- Suivre la marche des setters : région, classe de marche, phase ; appels directs, helpers locaux, IIFE, méthodes synchrones, await, cleanup.
- Connaître la ligne SlotWriter colonne par colonne, puis les relations written, effect_triggers, slot_seeds, registrations, slot_reads et body_calls.
- Comprendre la preuve mono-site converges_once_written : chaîne de gardes, dépliage des temporaires, trois arguments de mort.
- Situer la frontière entre relations et certification (ProgramRelations, primitives must_*, cliquet tests/layer_boundary.rs).

Prérequis : chapitre 6 (CFG, Expr, HookEntry, slots et sites), chapitre 7 (extraction des handlers, diamant &&/||, arêtes Await), chapitre 10 (Stability), chapitre 11 (environnements et stores), chapitre 13 (la dernière tranche de analyze_component_impl), chapitre 3 (must / may, dominance).

Le chapitre 13 s'est arrêté au moment où le point fixe d'un composant converge : on dispose alors d'un StateStore, d'un MemoStore, d'un tas, et d'un environnement abstrait par bloc de chaque corps. Ces objets décrivent des *valeurs*. Or les règles posent des questions d'une autre nature : « qui écrit ce slot, et quand ? », « cet effet relance-t-il celui-là ? », « ce listener est-il un jour retiré ? ». Ce sont des *relations* entre positions du programme, et ce chapitre explique comment le moteur les calcule, une fois, avec leur polarité écrite noir sur blanc, avant de les ranger dans l'AnalysisResult.

Le sous-système tient dans sept fichiers de src/engine/ : setters.rs (2 490 lignes, la marche des setters et la relation slot_writers), written.rs (415, ce qu'une écriture stocke), triggers.rs (110, effect_triggers), seeds.rs (552, slot_seeds), registrations.rs (683, la table des registrars et registrations), guards.rs (1 058, les preuves de gardes) et program_relations.rs (42, les relations programme-entier). Le graphe de churn, churn.rs, et la preuve multi-site de guards.rs sont l'objet du chapitre 15 ; nous n'en donnons ici que les entrées.

Les sorties d'analyseur de ce chapitre ont été observées avec le binaire construit au commit e67b10a sur des fichiers placés sous /tmp/reprobe/, /tmp/verif07/ et /tmp/ch14/, par des commandes de la forme reactant check -no-color -trace -show-clean [-info] <fichier>. Les *lignes de relation*, que la ligne de commande n'affiche pas, ont été lues par une petite sonde Rust (/tmp/reprobe/src/main.rs) qui lie la crate reactant par chemin, appelle lower_program puis analyze_program avec RootStrategy::Heuristic, et imprime les colonnes de chaque relation.

Son cœur, élagué, est le suivant :

Extrait 14.1 – la sonde de lecture des relations (élagué, /tmp/reprobe/src/main.rs)

```
1 let result = analyze_program(
2   ComponentRegistry::from_components(comps),
3   HookRegistry::new(), RootStrategy::Heuristic, &Config::default());
4 for (id, c) in &result.components {
5   for w in &c.slot_writers {
6     println!("slot={:?} owner={:?} setter={:?} line={:?} region={:?} phase={:?} \
7       via={:?} updater={:?} same_tick={:?} block={:?} guard_block={:?} \
8       fresh={:?} value.ref={:?} value.null={:?} expr={:?}", ...);
9   }
10  for t in &c.effect_triggers { ... } // hook, dep, slot, exact
11  for s in &c.slot_seeds { ... } // slot, path, normalized, sync, setter_escapes
12  for r in &c.registrations { ... } // effect, display, registrar, firing, timing,
13 } // handle, self_removing, block_id, pairing, event
14 let rel = ProgramRelations::new(&result);
15 for e in &rel.churn().edges { ... }
```

Dans les sorties de sonde, `line` est la ligne source du témoin (`span`), `value.ref` la composante référence de la valeur écrite et `expr` indique si la colonne `written.expr` est renseignée.

14.1 Pourquoi des relations

14.1.1 Un fait calculé deux fois est deux faits qui dérivent

Commençons par le programme qui a motivé la promotion des relations dans le moteur. Un effet sans tableau de deps enregistre une continuation de promesse qui écrit un objet neuf dans l'état :

Exemple 14.1: Une continuation dans un effet sans deps

Extrait 14.2 – /tmp/verif07/e12.tsx

```
1 import { useState, useEffect } from 'react';
2 export function NoDeps() {
3   const [x, setX] = useState({ n: 0 });
4   const [y, setY] = useState({ n: 0 });
5   useEffect(() => {
6     Promise.resolve().then(() => setX({ n: 1 }));
7     window.addEventListener('resize', () => setY({ n: 2 }));
8   });
9   return <div>{x.n}{y.n}</div>;
10 }
```

L'effet tourne après chaque rendu ; sa continuation `.then` stocke une référence neuve dans `x` ; le composant re-rend ; l'effet tourne à nouveau. C'est une boucle automatique. L'analyseur la voit :

```
NoDeps (4 hooks) /tmp/verif07/e12.tsx
warn infinite-loop [hook:2] (line 5:2) this effect has no dependency array and may store a fresh reference
  ↳ into state `x`, so it re-runs after every render and can re-trigger itself: possible infinite render loop
  ↳ a fresh value is written to state `x` here [hook:2] (line 6:4)
warn missing-cleanup [hook:2] (line 7:4) this effect calls `window.addEventListener` but returns no cleanup
  ↳ . The registration is repeated every time the effect re-runs (and on every mount, twice under StrictMode)
  ↳ and nothing ever undoes it; return a function that tears it down
```

Ce WARNING a été un faux négatif pendant toute la vie de l'issue #26, ouverte le 27 août et

close le 27 septembre 2026. La raison est instructive. Le moteur possédait déjà une marche des setters (`src/engine/setters.rs`) qui savait qu'un callback confié à `.then` s'exécute sur un tour *ultérieur* de la boucle d'événements. Mais le graphe de churn vivait dans `src/rules/helpers/churn.rs` et se servait d'une *seconde* marche, écrite pour lui, qui ignorait cette classification et approximait « imbriqué, donc jamais must-atteint » ; une écriture différée dans un effet sans deps n'y produisait aucune arête. Le texte de l'issue le disait déjà : « the event-vs-async callback classification lives in the engine, not in the syntactic collector ». Deux lectures d'un même fait, dont une seule était juste.

ADR-42 tire de ce constat une règle d'architecture qui se justifie en une phrase (principe du paragraphe unique) : *un fait calculé à deux endroits est deux faits qui dérivent*. La marche des règles a été supprimée, le graphe de churn est devenu un pli sur les relations du moteur, et le WARNING ci-dessus est apparu « par construction ».

14.1.2 Les trois couches

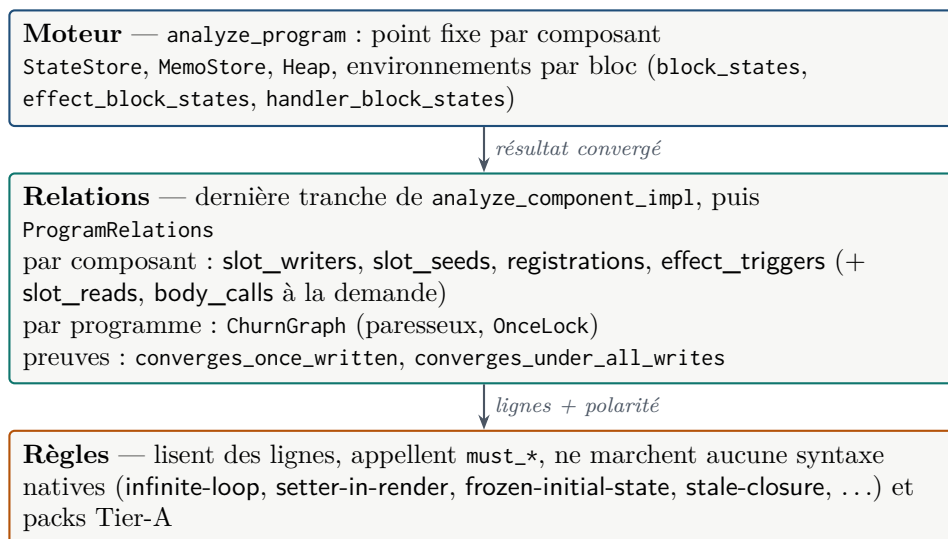


FIGURE 14.1 – Les trois couches d'ADR-42. Le moteur interprète ; la couche des relations dérive des faits du résultat convergé, chacun calculé une fois, avec sa polarité ; les règles transforment ces faits en diagnostics. La frontière basse est tenue par le cliquet `tests/layer_boundary.rs`.

La figure 14.1 résume l'architecture. Le catalogue `docs/relations.md` l'énonce dans ses premières lignes : « The engine interprets each component to a fixpoint. A layer of **relations** derives facts from the converged result — A writes B, A reads B, A re-runs when B moves, $A \rightarrow B$ — each computed once, in one place, with its polarity written down. The rules turn those facts into diagnostics and never walk syntax themselves. »

Décision : ADR-42 §1, la frontière et son cliquet

Une règle lit des lignes de relation et appelle des primitives `must_*` ; elle ne parcourt ni CFG ni expression. La frontière est tenue par un *test-cliquet*, `tests/layer_boundary.rs` : une liste explicite des fichiers de `src/rules/` qui touchent encore la syntaxe (`tests/layer_boundary.rs`, l. 20–31, dix entrées au commit `e67b10a`), et cinq marqueurs qui définissent « toucher la syntaxe » (`cfg.blocks`, `.blocks.values()`, `Stmt:::`, `for_each_child`, `Terminator:::`, `tests/layer_boundary.rs`, l. 35–41). Un changement peut retirer un fichier de la liste, jamais en ajouter un. Chaque entrée est « une promotion encore à faire, pas une exception ». L'alternative refusée était de laisser chaque règle garder sa propre lecture du programme, quitte à la corriger au cas

par cas : c'est précisément le *workaround* que le premier principe de CLAUDE.md interdit.

14.1.3 Le vocabulaire de polarité

Une relation est une table. Une *ligne* y décrit une occurrence dans le code (un site d'appel de setter, un enregistrement de callback, un couple slot-chemin de prop) et chaque *colonne* porte une affirmation dont la force est fixée à l'avance. Le catalogue donne quatre mots pour cette force, que nous adoptons tels quels.

Définition 14.1: Relation du moteur, polarité d'une colonne

Une *relation du moteur* est un vecteur de lignes calculé après convergence d'un composant (ou d'un programme), stocké sur l'AnalysisResult (ou construit paresseusement par ProgramRelations), et lu tel quel par les règles. Chaque colonne d'une ligne a une *polarité* :

- **exact** : un fait sur le code, vrai par construction (une région lexicale, une position, un nom tel qu'il est écrit) ;
- **must** : une sous-approximation ; quand la colonne dit oui, le programme concret le fait à chaque exécution. Une colonne must peut soutenir une ERROR ;
- **may** : une sur-approximation ; quand la colonne dit non, le programme concret ne le fait jamais. Une colonne may ne peut soutenir qu'un WARNING, et son négatif n'est pas une preuve d'absence sauf mention contraire ;
- **⊥-bearing** : la colonne a une valeur top (Unknown) qui satisfait toute requête ; l'analyse n'a pas pu trancher, et un lecteur doit prendre la branche qui *tire davantage*.

L'absence d'une ligne n'est jamais une preuve, sauf si l'entrée du catalogue le dit : une ligne existe pour ce que l'analyse a pu voir ; un plafond de profondeur, un callee non résolu ou une closure que rien n'appelle ne contribuent rien.

Cette définition prolonge celle des faits must et may du définition 3.7 : la nouveauté est qu'elle s'applique *colonne par colonne*, et qu'elle est écrite sur le type. Un lecteur de SlotWriter sait, avant toute règle, que *region* est exacte, que *phase* est may avec un $\perp$, et que *written.fresh* est must pour *Fresh*. C'est ce qui permet à la couche des règles de n'accorder une ERROR qu'à une preuve assemblée depuis des colonnes must ou exactes, par une primitive *must_** de *src/rules/api/query.rs* (chapitre 17).

Piège

« May » et « must » qualifient *la direction dans laquelle une colonne peut se tromper*, pas sa fiabilité. Une colonne may qui dit false ne promet rien : *same_tick = false* sur une ligne ne signifie pas que l'écriture est seule dans son tick, seulement que la marche n'en a pas vu d'autre. Symétriquement, une colonne must qui dit Fresh est une affirmation forte ; le code la refuse dès qu'un doute subsiste, quitte à produire Maybe sur du code correct. Toute la discipline du sous-système consiste à choisir, pour chaque colonne, dans quel sens elle a le droit d'être imprécise.

14.1.4 Où les relations naissent : la dernière tranche du point fixe

Les relations par composant sont calculées à un seul endroit : la fin de *analyze_component_impl* (*src/engine/fixpoint.rs*), après la boucle de point fixe et le rafraîchissement post-convergence décrits au chapitre 13. Toutes les voies d'analyse (*analyze_component*, *analyze_component_inter* pour un enfant inliné, les deux phases d'*analyze_program*) passent par cette tranche ; les relations existent donc sur *tout* AnalysisResult, intra comme inter-composants.

Extrait 14.3 – `src/engine/fixpoint.rs`, l. 596–646 — la dernière tranche : setters étrangers, environnements de site, puis les quatre relations

```

596 let hook_calls = collect_hook_calls(&hooks, &render_cfg);
597 let effect_info = collect_effect_info(&hooks);
598 let handler_info = collect_handler_info(&hooks);
599 // A write through a parent's setter prop is a row too, owner-qualified
600 // (ADR-042 §2) – the same resolution `setter-in-render` reads.
601 let foreign: HashMap<Var, crate::engine::setters::SetterProp> =
602     crate::engine::setters::collect_component_setter_vars(&render_cfg, &block_states, &heap)
603         .into_iter()
604         .filter(|(comp_id, prop)| prop.component != comp_id)
605         .collect();
606 // The envs each row's argument is evaluated in: the converged stores, and
607 // the per-block envs of the region the row sits in.
608 let site_envs = crate::engine::written::SiteEnvs {
609     component: comp_id,
610     state: &final_state,
611     memo: &memo_store,
612     heap: &heap,
613     render_entry: &initial_env,
614     render_exits: &block_states,
615     exit: &env_exit,
616     effect_exits: &effect_block_states,
617     handler_exits: &handler_block_states,
618 };
619 let slot_writers = crate::engine::setters::collect_slot_writers(
620     &render_cfg,
621     &hooks,
622     &inline_regions,
623     &hook_provenance,
624     &foreign,
625     &site_envs,
626 );
627 // The seed relation rides the same slice: it folds `slot_writers` rows and
628 // the effects' declared deps, so it must come after both (#106, ADR-031).
629 let slot_seeds = {
630     let mut labels = crate::engine::setters::setter_var_labels(&render_cfg);
631     for cfg in std::iter::once(&render_cfg).chain(hooks.iter().filter_map(|h| h.body_cfg())) {
632         labels = crate::engine::setters::resolve_setter_aliases(cfg, &labels);
633     }
634     crate::engine::seeds::collect_slot_seeds(
635         &render_cfg,
636         &hooks,
637         &comp_param,
638         &labels,
639         &slot_writers,
640         &effect_info,
641     )
642 };
643 // The registration relation rides the same slice (#111, ADR-034): one
644 // scan of the effect bodies, read by `stale-closure`, `missing-cleanup`
645 // and the Tier-A `registrations` anchor alike.
646 let registrations = crate::engine::registrations::collect_registrations(&render_cfg, &hooks);

```

Lisons cet extrait dans l'ordre, car l'ordre y est imposé.

1. `foreign` : les variables du rendu dont la valeur convergée est un `ComponentSetter` d'un *autre* composant (un setter reçu en prop), calculées par `collect_component_setter_vars` sur les environnements de bloc et le tas, puis filtrées sur `prop.component != comp_id`. Elles fourniront les lignes *étrangères* de `slot_writers` (section 14.4).

2. `site_envs` : un `SiteEnvs`, la collection des stores convergés et des environnements par bloc de chaque région, dans lesquels l’argument de chaque écriture sera évalué (section 14.5).
3. `collect_slot_writers` : la relation centrale, qui reçoit aussi les `InlineRegions` (les plages de blocs splicés, pour la provenance) et la table `hook_provenance`.
4. `collect_slot_seeds` : elle *replie* `slot_writers` et les deps déclarés des effets, et doit donc venir après (le commentaire cite #106 et ADR-31). La table d’alias `labels` qu’elle reçoit est construite par `setter_var_labels` puis `resolve_setter_aliases` sur le rendu et chaque corps de hook.
5. `collect_registrations` : indépendante, un seul balayage des corps d’effets, lu par `stale-closure`, `missing-cleanup` et l’ancre Tier-A `registrations`.

Quelques lignes plus bas (`src/engine/fixpoint.rs`, l. 649–667), `collect_effect_triggers` est appelée avec un évaluateur `eval_in_stores` fermé sur l’environnement de sortie du rendu et les stores finaux ; puis les quatre vecteurs sont rangés dans l’`AnalysisResult` (champs `slot_writers`, `slot_seeds`, `registrations`, `effect_triggers`, `src/engine/fixpoint.rs`, l. 670–697).

Deux points méritent d’être soulignés. D’abord, les relations lisent les CFG *post-expansion* : les utilitaires et les hooks personnalisés ont déjà été splicés (chapitre 8), si bien qu’une écriture faite dans un helper importé apparaît comme une ligne, avec sa provenance. Ensuite, elles ne modifient rien : `SiteEnvs::eval` clone les stores avant toute évaluation (`src/engine/written.rs`, l. 240), et le résultat convergé reste celui du point fixe.

14.1.5 Le catalogue

Le tableau 14.1 donne la vue d’ensemble que le reste du chapitre déplie. Il suit `docs/relations.md`, qui est la source de vérité documentaire du sous-système (le section 14.12 dira où ce document a un amendement de retard).

Relation	Stockage	Clé d’une ligne	Calculée par
<code>slot_writers</code> (<code>SlotWriter</code>)	<code>AnalysisResult</code>	un site d’appel de <code>setter</code>	<code>setters::collect_slot_writers</code>
<code>slot_seeds</code> (<code>SlotSeed</code>)	<code>AnalysisResult</code>	(slot, chemin de prop lu par l’initialiseur)	<code>seeds::collect_slot_seeds</code>
<code>registrations</code> (<code>Registration</code>)	<code>AnalysisResult</code>	un appel d’enregistrement dans un corps d’effet	<code>registrations::collect_registrations</code>
<code>effect_triggers</code> (<code>EffectTrigger</code>)	<code>AnalysisResult</code>	(effet, index de dep, slot qualifié)	<code>triggers::collect_effect_triggers</code>
<code>slot_reads</code> , <code>body_calls</code>	à la demande	un site de lecture / d’appel	<code>setters::collect_slot_reads</code> , <code>collect_body_calls</code>
<code>ChurnGraph</code>	<code>ProgramRelations</code> (<code>OnceLock</code>)	une arête (from, to, composant, effet)	<code>churn::ChurnGraph::build</code> (chapitre 15)

TABLE 14.1 – Les relations du moteur au commit e67b10a. S’y ajoutent deux *preuves* (des fonctions, pas des colonnes) : `guards::converges_once_written`, lue par `setter-in-render`, et `guards::converges_under_all_writes`, lue par le graphe de churn.

Le `HookLabel` qui identifie un slot est *local au composant* (définition 6.3) : le label 0 de `Parent` et le label 0 de `Child` sont deux slots différents. Quand une relation doit nommer un slot d’un autre composant, elle emploie un *slot qualifié*, `QualifiedSlot = (ComponentId, HookLabel)` (`src/ir/types.rs`, l. 9) ; c’est la clé d’un `EffectTrigger` et le nœud du graphe de churn. De même, un `BlockId` est local à un CFG : un identifiant de bloc d’un corps imbriqué ne désigne rien dans le CFG de la région, invariant que nous retrouverons à chaque colonne de position.

14.2 La marche des setters

14.2.1 Le problème : quatre écritures, quatre moments

Un corps d'effet est une région lexicale : tout ce qui est écrit entre ses accolades lui appartient. Mais le *moment* où une écriture s'exécute ne se lit pas à sa position. Voici un effet qui contient quatre appels de setter, dont aucun ne s'exécute au même moment que les autres.

Exemple 14.2: Un effet, quatre phases

Extrait 14.4 – /tmp/relprobe/ex2.tsx

```
1 import { useState, useEffect } from 'react';
2 export function Loader({ url }: { url: string }) {
3   const [data, setData] = useState<any>(null);
4   const [pos, setPos] = useState({ x: 0 });
5   useEffect(() => {
6     setData(null);
7     fetch(url).then(r => setData({ r }));
8     const id = setInterval(() => setPos({ x: 1 }), 100);
9     const h = (e: any) => setPos({ x: e.clientX });
10    window.addEventListener('mousemove', h);
11    return () => {
12      clearInterval(id);
13      window.removeEventListener('mousemove', h);
14    };
15  }, [url]);
16  return <div>{String(data)}{pos.x}</div>;
17 }
```

`setData(null)` s'exécute pendant la phase d'effet ; `setData( r )` sur une microtâche ultérieure ; `setPos( x: 1 )` sur un timer ; `setPos( x: e.clientX )` sur un événement souris. Le cleanup, lui, ne contient aucune écriture : il rend un handle et un listener, il n'appelle rien. La sonde donne :

```
-- slot_writers
slot=0 setter=setData line=Some(6) region=Effect(2) phase=Effect block=Some(0) guard_block=Some(0) fresh=Not
  ↳ value.ref=Bottom value.null=true
slot=0 setter=setData line=Some(7) region=Effect(2) phase=Deferred block=None guard_block=Some(0) fresh=Fresh
  ↳ value.ref=PerRender
slot=1 setter=setPos line=Some(10) region=Effect(2) phase=Handler block=None guard_block=Some(0) fresh=Fresh
  ↳ value.ref=PerRender
slot=1 setter=setPos line=Some(8) region=Effect(2) phase=Deferred block=None guard_block=Some(0) fresh=Fresh
  ↳ value.ref=PerRender
```

(colonnes `owner=None` `via=Direct` `updater=Unknown` `same_tick=false` omises). Quatre lignes, une par site ; même `region`, quatre `phase` ; et seule la première a un `block`. La ligne de commande, elle, ne signale rien : ce composant est correct.

La *marche des setters* (`SetterWalk`, `src/engine/setters.rs`) est l'algorithme qui produit ces lignes. Elle parcourt le CFG d'une région, descend dans les fonctions imbriquées selon des règles précises, et attribue à chaque appel de setter une *classe* qui deviendra sa phase. Tout le reste du chapitre repose sur elle : les relations `slot_reads` et `body_calls` sont deux autres canaux de la même marche, et `slot_seeds` replie ses lignes.

14.2.2 Région et phase : deux colonnes pour deux faits

Extrait 14.5 – `src/engine/setters.rs`, l. 641–648 — la région lexicale d'une écriture

```

641 #[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord)]
642 pub enum WriterRegion {
643     Render,
644     Effect(HookLabel),
645     Memo(HookLabel),
646     Callback(HookLabel),
647     Handler(HookLabel),
648 }

```

WriterRegion dit *quel corps du composant* contient l'appel : le rendu, ou le corps d'un hook identifié par son label (effet, memo, callback, handler). C'est un fait exact. Un `onClick=()` $\Rightarrow$ `set(1)` extrait par le lowering en `HookEntry::Handler` (chapitre 7) est de région `Handler(1)`. L'ordre dérivé (`Ord`) sert au tri déterministe des lignes, et `word()` (`src/engine/setters.rs`, l. 652–660) donne le mot affiché dans les messages.

Extrait 14.6 – `src/engine/setters.rs`, l. 681–701 — la phase d'exécution, un verdict may

```

681 /// Execution phase of a write – a MAY verdict (ADR-027 §1): a write
682 /// synchronous in its body carries that body's phase; a write inside a
683 /// nested `FnLit` is `Unknown` ( $\top$  – it may run in any phase) until a callee
684 /// summary sharpens it (ADR-027 §2). Classifying every nested callback as
685 /// "deferred" instead would under-approximate: `arr.forEach(x => setX(x))`
686 /// inside an effect runs synchronously in the effect phase.
687 #[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord)]
688 pub enum WriterPhase {
689     Render,
690     Effect,
691     Memo,
692     Callback,
693     Handler,
694     /// The callee summary proved deferral (ADR-027 §2): a timer, a microtask,
695     /// a promise continuation – the write never runs inside a React phase.
696     Deferred,
697     /// An effect's returned cleanup function.
698     Cleanup,
699     ///  $\top$  – the write may run in any phase. Satisfies every phase query.
700     Unknown,
701 }

```

WriterPhase dit *quand* l'écriture s'exécute. Le commentaire de tête fixe la polarité : c'est un verdict may. Une écriture synchrone dans son corps porte la phase de ce corps ; une écriture dans une `FnLit` imbriquée est `Unknown`, c'est-à-dire $\top$, tant qu'un résumé du callee ne l'affine pas. Les deux variantes `Deferred` (un timer, une microtâche, une continuation de promesse : l'écriture ne s'exécute dans aucune phase React) et `Cleanup` (la fonction retournée par un effet) sont des *preuves*, obtenues par contrat ; `Unknown` est l'absence de preuve.

La correspondance entre une région et la phase d'une écriture *synchrone* dans cette région est `sync_phase` (`src/engine/setters.rs`, l. 670–678) : `Render` $\rightarrow$ `Render`, `Effect(_)` $\rightarrow$ `Effect`, et ainsi de suite. Son commentaire (`src/engine/setters.rs`, l. 662–669) rappelle que cette égalité « lexis = exécution » a été faussée au-delà d'un `await` jusqu'à #117 : c'est la marche qui la rend vraie en basculant en `Deferred` à l'entrée d'un bloc `post-await`.

Décision : ADR-27 §1, région exacte et phase may

La question « lexical ou exécution ? » reçoit la réponse *les deux, en deux colonnes distinctes*, parce que ce sont deux faits distincts. L'alternative examinée, classer toute `FnLit` imbriquée comme différée, aurait été une sous-approximation : `arr.forEach(x => setX(x))` dans un effet s'exécute *synchroniquement* dans la phase d'effet. L'autre alternative, traiter $\top$ comme

« jamais dans cette phase », perdrait des findings. $\top$ satisfait donc toute requête includes : c'est « la sémantique may honnête du mot phase ».

14.2.3 La classe de marche

La marche ne manipule pas directement des `WriterPhase` mais une classe interne, `WalkClass`, décidée par *la manière dont la marche est entrée dans un CFG*.

Extrait 14.7 – `src/engine/setters.rs`, l. 1319–1344 — la classe interne de la marche

```

1319 /// The phase class a walk context assigns to writes found under it –
1320 /// decided by HOW the walk entered the CFG (ADR-027 §2).
1321 ///
1322 /// `Sync` is the only class with meaningful block ids. `Deferred` and
1323 /// `Handler` come from callee summaries and are context-free: a timer defers
1324 /// and a listener fires on events whatever phase registered them. Sync HOFs
1325 /// (`arr.map(fn)`) run their argument in the ENCLOSING class, so they
1326 /// propagate the current mode instead of assigning one. Everything unknown
1327 /// is  $\top$ . The summaries axiomatize host semantics (a bare `setTimeout` is the
1328 /// host timer unless a local binding shadows it, `.then` is a Promise, array
1329 /// HOFs are `Array.prototype`'s) – the same footing as the
1330 /// `addEventListener` subscription axiom.
1331 #[derive(Debug, Clone, Copy, PartialEq, Eq, Hash, PartialOrd, Ord)]
1332 pub(crate) enum WalkClass {
1333     /// Synchronous in the walked body (top level, or through a directly
1334     /// called local helper).
1335     Sync,
1336     /// Proved deferred by a callee summary.
1337     Deferred,
1338     /// A listener registered on an event target.
1339     Handler,
1340     /// An effect's returned cleanup function.
1341     Cleanup,
1342     /// Nested under an unknown callee – may run in any phase ( $\top$ ).
1343     Unknown,
1344 }

```

Le commentaire dit tout : `Sync` est la seule classe dont les identifiants de bloc ont un sens ; `Deferred` et `Handler` viennent de résumés de callees et sont *indépendants du contexte* (un timer diffère, un listener tire sur événement, quelle que soit la phase qui les a enregistrés) ; les HOF synchrones (`arr.map(fn)`) exécutent leur argument dans la classe *englobante* et la propagent au lieu d'en assigner une ; tout ce qui est inconnu est $\top$. Les résumés « axiomatisent la sémantique de l'hôte » : un `setTimeout` nu est le timer de l'hôte sauf si une liaison locale l'ombre, `.then` est une promesse, les HOF de tableau sont ceux d'`Array.prototype`.

La conversion vers la phase se fait au moment de pousser la ligne :

Extrait 14.8 – `src/engine/setters.rs`, l. 978–986 — de la classe à la phase

```

978 fn class_phase(class: WalkClass, region: WriterRegion) -> WriterPhase {
979     match class {
980         WalkClass::Sync => region.sync_phase(),
981         WalkClass::Deferred => WriterPhase::Deferred,
982         WalkClass::Handler => WriterPhase::Handler,
983         WalkClass::Cleanup => WriterPhase::Cleanup,
984         WalkClass::Unknown => WriterPhase::Unknown,
985     }
986 }

```

WalkClass	WriterPhase	Comment la marche y entre
Sync	<code>region.sync_phase()</code>	racine de la région ; corps d'un helper local appelé directement (B6) ; corps d'une IIFE ; callback d'un HOF synchrone (<code>map</code> , <code>forEach</code> , ...)
Deferred	Deferred	argument d'un registrar de timing = Deferred (<code>setTimeout</code> , <code>.then</code> , <code>queueMicrotask</code> , ...); bloc post-await d'une marche Sync
Handler	Handler	argument d'un registrar de timing = Handler (<code>addEventListener</code>); wrapper prouvé par résumé de bibliothèque (<code>form.handleSubmit(cb)</code>)
Cleanup	Cleanup	fonction retournée par un corps d'effet, à la racine, en mode Sync
Unknown	Unknown (T)	argument fonctionnel de tout autre callee, dont les registrars de timing = Unknown (<code>subscribe</code> , <code>on</code> , <code>addListener</code>)

TABLE 14.2 – De la classe de marche à la phase. Les régions et les classes sont orthogonales : une ligne `region = Effect(2)`, `phase = Handler` est un listener enregistré par l'effet 2.

Une projection publique plus grossière, `SetterCallPhase` (`src/engine/setters.rs`, l. 56–70), replie `Cleanup` sur `Deferred` et offre `may_run_in_body()`, vraie pour `Sync` et `Unknown` (`src/engine/setters.rs`, l. 72–79) : c'est la question qu'une règle comme `setter-in-render` pose. Jusqu'à #130, `Deferred` et `Unknown` étaient une seule variante `Other`.

Décision : ADR-38 §3, `Deferred` n'est pas `Unknown`

Replier la preuve de différé et l'absence de preuve dans une même variante coûtait à un consommateur *exactement* la distinction dont il a besoin : `Deferred` est une preuve que l'écriture ne s'exécute pas dans le passage du corps ; `Unknown` est `T`, et `T` inclut ce passage. Une règle qui doit taire une écriture prouvée différée mais signaler une écriture `T` ne pouvait pas les distinguer. Les deux variantes sont séparées et `may_run_in_body` est la seule question posée.

14.2.4 L'état d'une marche

Extrait 14.9 – `src/engine/setters.rs`, l. 1232–1283 — l'état d'une marche, champ par champ

```

1232 /// Threads the walk's fixed context (`setter_vars`, `fn_bindings`) and its
1233 /// expansion stack through the mutually recursive CFG/stmt/expr descent, so
1234 /// each step only takes what actually varies per call.
1235 ///
1236 /// `walking` is the set of CFGs on the current expansion *stack*, keyed by
1237 /// identity. It must stay a stack (pushed on entry, popped on exit), not a
1238 /// global visited set: a body first reached with no depth left and later with
1239 /// budget to spare has to be walked again, so a global set would lose findings.
1240 /// Skipping only re-entrant walks loses none – a cycle re-enters a body at a
1241 /// budget no larger than the one it is already being walked at, so the spliced
1242 /// cycle-free path reaches the same CFGs and `found` only ever grows.
1243 struct SetterWalk<'a> {
1244     setter_vars: &'a HashSet<Var>,
1245     fn_bindings: &'a HashMap<Var, Arc<CFG>>,
1246     walking: HashSet<usize>,
1247     /// The walk's outermost CFG (pointer identity) – where effect cleanup
1248     /// returns and reified listeners are recognized.
1249     root: usize,
1250     /// The walked root is an effect body: its top-level `addEventListener`
1251     /// listeners were reified as Handler entries (`extract_subscriptions`),
1252     /// and its returned function is its cleanup.

```

```

1253     effect_body: bool,
1254     /// Locally-bound names that disable a deferring-global summary
1255     /// (fail-closed: `let setTimeout = ...` anywhere in the component makes
1256     /// the bare name mean nothing).
1257     shadowed: &'a HashSet<Var>,
1258     /// The walk is currently inside a callback a synchronous higher-order
1259     /// function runs, so anything it finds may execute many times per tick.
1260     repeating: bool,
1261     /// Bodies of the component's `useCallback` hooks, by label – a memoized
1262     /// function is as proven a function literal as an inline one, and reading
1263     /// it as T fired the non-functional rules on correct code.
1264     callback_bodies: &'a HashMap<HookLabel, Arc<CFG>>,
1265     /// Names bound exactly once, to a function literal, in the walked root –
1266     /// the bar a `set(fn)` argument must clear before it counts as a proven
1267     /// functional updater. `collect_fn_bindings` is not that bar: it keeps the
1268     /// last binding of a re-bound name.
1269     certified_fns: &'a HashSet<Var>,
1270     /// Also record every non-setter call site (#126). Off for the setter
1271     /// consumers: a body with 200 calls would otherwise pay for 200 rows
1272     /// nobody reads, on a walk the rule pass runs once per component per rule.
1273     collect_calls: bool,
1274     /// Bindings whose *reads* to record (#127) – the slot-value names and
1275     /// their aliases. Empty for every consumer that does not ask; the
1276     /// traversal still runs, it just records nothing on this channel.
1277     read_vars: &'a HashSet<Var>,
1278     /// Callee spellings proven to wrap their function argument rather than run
1279     /// it – see [ `wrapper_callees` ]. Empty for every consumer that does not
1280     /// resolve summaries, which reads as "no wrapper is proven": T, the
1281     /// fire-more direction.
1282     wrappers: &'a HashSet<String>,
1283 }

```

Les champs se lisent en trois groupes. Le *contexte fixe* : `setter_vars`, les noms qui désignent un setter (alias résolus) ; `fn_bindings`, les `let f = FnLit` du corps ; `shadowed`, les noms liés localement qui désactivent un résumé de global (`let setTimeout = ...` n'importe où dans le composant fait perdre son sens au nom nu, fail-closed) ; `callback_bodies`, les corps des `useCallback` par label ; `certified_fns`, les noms liés *une seule fois* à un littéral de fonction et jamais re-liés, seule barre que doit franchir un argument `set(fn)` pour être un updater fonctionnel prouvé ; `wrappers`, les orthographes de callees prouvées envelopper leur argument sans l'exécuter. Puis la *pile* : `walking`, l'ensemble des CFG en cours d'expansion, par identité de pointeur, et `root`, la racine. Enfin les *modes* : `effect_body` (la racine est un corps d'effet : ses listeners de premier niveau ont été réifiés, sa fonction retournée est le `cleanup`), `repeating` (on est dans un callback qu'un HOF synchrone exécute plusieurs fois par tick), et les deux drapeaux de canaux `collect_calls` et `read_vars`.

Le commentaire sur `walking` mérite une lecture attentive. C'est une *pile*, pas un ensemble global de visités : un corps atteint d'abord sans budget de profondeur, puis plus tard avec du budget, doit être marché de nouveau, sinon on perdrait des lignes. N'ignorer que les entrées *ré-entrantes* ne perd rien, parce qu'un cycle ré-entre dans un corps avec un budget au plus égal à celui avec lequel il est déjà marché.

Quatre fonctions construisent une `SetterWalk`, avec des réglages différents (tableau 14.3). Toutes démarrent par `walk.cfg(cfg, max_depth, &mut found, WalkClass::Sync, None, None)` : mode Sync, pas de provenance, pas de témoin.

Point d'entrée (ligne)	effect_body	shadowed	callback_bodies	wrappers	collect_calls	read_vars
collect_setter_calls_with_extra (105)	false	vide	vide	prouvés	non	vide
collect_write_sites (189)	argument	argument	argument	prouvés	non	vide
collect_slot_reads (2062)	région Effect	vide	vide	vide	non	slots + alias
collect_body_calls (2153)	région Effect	vide	vide	vide	oui	vide

TABLE 14.3 – Les quatre points d'entrée de la marche (`src/engine/setters.rs`, constructions de `SetterWalk` l. 121, 213, 2081 et 2159). Seul `collect_write_sites`, appelé par `collect_slot_writers`, bénéficie de l'ombrage des registrars et des corps de `useCallback` ; « prouvés » désigne les wrappers calculés par `wrapper_callees` sur le CFG marché lui-même (`src/engine/setters.rs`, l. 1497), qui ne cherche les objets à résumer (`useForm()....`) que parmi les `Let` de ce CFG : dans un corps d'effet ou de handler extrait, l'ensemble est donc vide en pratique, direction sûre ; les deux canaux de lecture et d'appel reçoivent l'ensemble vide `NO_WRAPPERS` ; `collect_setter_calls` est la forme historique « une ligne par variable », encore appelée directement par plusieurs règles (section 14.11).

14.2.5 Parcourir un CFG

L'entrée dans un CFG (`SetterWalk::cfg`) commence par empiler le CFG dans `walking`, calculer `Reachability::of(cfg)` (un BFS par bloc, `src/engine/setters.rs`, l. 933–976) et `cfg.post_await_blocks()` (chapitre 7), puis parcourt les blocs en largeur depuis l'entrée :

Extrait 14.10 – `src/engine/setters.rs`, l. 1696–1716 — la boucle sur les blocs

```

1696     while let Some(bid) = queue.pop_front() {
1697         // Only a sync walk can be deferred by an await: a body already
1698         // classified `Deferred`, `Handler` or `T` does not become more so.
1699         let mode = if mode == WalkClass::Sync && post_await.contains(&bid) {
1700             WalkClass::Deferred
1701         } else {
1702             mode
1703         };
1704         let block_id = if mode == WalkClass::Sync {
1705             Some(bid)
1706         } else {
1707             None
1708         };
1709         let prov_block = if at_root { Some(bid) } else { prov };
1710         let outer_repeating = self.repeating;
1711         self.repeating = outer_repeating || reach.reaches(bid, bid);
1712         if let Some(block) = cfg.blocks.get(&bid) {
1713             for (i, stmt) in block.stmts.iter().enumerate() {
1714                 let at = block_id.map(|b| (b, i));
1715                 self.stmt(stmt, at, depth, found, mode, at_root, prov_block, witness);
1716             }

```

Trois idées tiennent dans ces vingt lignes.

1. **Le bloc n'a de sens qu'en mode Sync.** `block_id`, et donc la position `at = (bloc, index d'instruction)`, ne sont renseignés que pour une marche synchrone ; une écriture différée ou imbriquée n'a pas de position dans le CFG de la région.
2. **`prov_block` est toujours un bloc de la racine.** À la racine, chaque bloc est sa propre provenance ; en profondeur, on hérite du bloc d'où l'on est descendu. C'est la seule clé qui ait un sens dans le CFG de la région : l'appartenance à une région inlinee la lit, et la co-exécution (`same_tick`) aussi.
3. **Un bloc qui s'atteint lui-même rend tout ce qu'il contient `repeating`.**

`reach.reaches(bid, bid)` n'est vrai qu'à travers un vrai cycle, parce que le BFS part des *successeurs* du bloc et non du bloc lui-même (`src/engine/setters.rs`, l. 939–941).

La bascule `Sync` $\rightarrow$ `Deferred` sur un bloc `post-await` est le cœur d'ADR-35. Seule une marche `Sync` est affectée : « a body already classified `Deferred`, `Handler` or `T` does not become more so ».

Rappel React : microtâches, `await` et IIFE

Le code qui suit un `await` s'exécute dans une microtâche ultérieure, jamais pendant le passage courant du corps, même si la promesse est déjà résolue ; il en va de même pour `.then/.catch/.finally` et `queueMicrotask`. Une fonction `async` appelée *immédiatement* (IIFE) exécute en revanche *synchroniquement* son code jusqu'au premier `await`. C'est pourquoi le lowering coupe le bloc à chaque `await` avec une arête `EdgeKind::Await` (chapitre 7), et pourquoi la marche traite une IIFE comme un helper appelé au site.

Décision : ADR-35, la frontière `await`

ADR-35 choisit une *arête* `Await` plutôt qu'un champ de bloc : `BasicBlock` et `CFG` sont construits à plus de deux cents endroits (surtout de l'IR de test bâti à la main), un champ nouveau serait un changement à deux cents sites pour un fait que ces fixtures n'ont pas, tandis qu'un genre d'arête est additif et survit à `remap_cfg`. Le consommateur calcule la clôture `post-await_blocks` par successeurs (`src/ir/cfg.rs`, l. 121–144) ; un corps de boucle qui attend marque son propre en-tête, ce qui est juste (la seconde itération s'exécute bien après une suspension). Et la marche bascule `Sync` $\rightarrow$ `Deferred`, rien d'autre : cela *restaure* l'exactitude au lieu d'ajouter une approximation, un `post-await` n'est pas « phase inconnue » mais prouvablement un tour ultérieur. La mesure d'ADR-35 §4 a montré que ce découpage n'achetait *rien* tant que l'IIFE n'était pas marchée au site : la forme `(async () => ... )()` ne produisait aucune ligne du tout.

Le terminateur `Return` d'un corps d'effet reçoit un traitement à part (`src/engine/setters.rs`, l. 1720–1745) : si la racine est un corps d'effet et la marche en mode `Sync`, la fonction retournée, littéral ou nom lié dans `fn_bindings`, est descendue en classe `Cleanup` avec le bloc de provenance courant ; « without this the cleanup's writes are invisible to the relation ».

14.2.6 Un appel : le site, le helper local, l'IIFE

Le traitement d'une expression d'appel (`SetterWalk::expr`) est le passage décisif. Quand le callee est une variable, deux choses peuvent se produire : c'est un setter, et l'on enregistre un site ; c'est un helper local, et l'on descend dans son corps *dans le mode courant*.

Extrait 14.11 – `src/engine/setters.rs`, l. 1882–1919 — un site d'écriture, puis le bras B6

```
1882     if let Expr::Call { fn_, args } | Expr::New { fn_, args, .. } = expr {
1883         if let Expr::Var(name) = fn_.as_ref() {
1884             if self.setter_vars.contains(name) {
1885                 found.setters.push(FoundSite {
1886                     var: name.clone(),
1887                     class: mode,
1888                     span: stmt_span,
1889                     at,
1890                     prov_block: prov,
1891                     repeats: self.repeating,
1892                     updater: self.updater_of(args),
1893                     arg: args.first().map(|a| a.peel_ts().clone()),
1894                 });
1895             }
1896             // B6: direct call to a locally-bound function – its body runs
```

```

1897         // in the CURRENT mode, so sync-in-helper writes take this call
1898         // site's class and block id; writes the helper defers or
1899         // nests keep their own class (a `setTimeout` inside a sync
1900         // helper is not a sync write – lifting it was an
1901         // under-approximation).
1902         if depth > 0
1903             && let Some(body) = self.fn_bindings.get(name)
1904         {
1905             // Inner rows' provenance is this call site: the local
1906             // helper's definition is only reachable from code that
1907             // shares its region (salted names stay region-local).
1908             let mut inner = Found::default();
1909             self.cfg(
1910                 body,
1911                 depth - 1,
1912                 &mut inner,
1913                 WalkClass::Sync,
1914                 prov,
1915                 stmt_span,
1916             );
1917             found.absorb(inner, mode, at, prov);
1918         }
1919     }

```

Un `FoundSite` capture, au moment où la marche a encore tout sous la main, la variable, la classe, le span de l'instruction (le *témoin*), la position `at`, le bloc de provenance, le drapeau `repeats`, la classification de l'argument 0 (`updater_of`, section 14.4) et l'argument lui-même.

Le bras **B6** descend le corps d'un helper local appelé directement, en `Sync`, avec la provenance du site d'appel. Son résultat n'est pas ajouté tel quel : `Found::absorb` (`src/engine/setters.rs`, l. 1373–1408) donne aux lignes que la marche interne a classées `Sync` la classe *et le bloc du site d'appel*, et laisse aux autres la classe qu'elles ont gagnée. Le commentaire explique pourquoi : « a `setTimeout` inside a sync helper is not a sync write — lifting it was an under-approximation ». Les IIFE reçoivent exactement le même traitement (`src/engine/setters.rs`, l. 1920–1940) : le corps de `(async () => ... )()` est marché en `Sync`, puis absorbé.

Exemple 14.3: IIFE, `await`, `cleanup` : trois phases dans un seul effet

Extrait 14.12 – `/tmp/verif07/e11.tsx`

```

1 import { useState, useEffect } from 'react';
2 declare function load(): Promise<number>;
3 export function Phases() {
4     const [a, setA] = useState(0);
5     const [b, setB] = useState(0);
6     const [c, setC] = useState(0);
7     useEffect(() => {
8         (async () => {
9             setA(1);
10            const v = await load();
11            setB(v);
12        })();
13        return () => { setC(2); };
14    }, []);
15    return <div>{a}{b}{c}</div>;
16 }

```

```

slot=0 setter=setA line=Some(9) region=Effect(3) phase=Effect block=Some(0) guard_block=Some(0) fresh=Not
↳ value.ref=Bottom

```

```

slot=1 setter=setB line=Some(11) region=Effect(3) phase=Deferred block=None    guard_block=Some(0) fresh=Maybe
  ↪ value.ref=Unknown
slot=2 setter=setC line=Some(13) region=Effect(3) phase=Cleanup  block=None    guard_block=Some(0) fresh=Not
  ↪ value.ref=Bottom

```

setA(1), écrit dans une fonction imbriquée, prend la classe Sync et le bloc 0 de l'effet par absorption de l'IIFE. setB(v) est dans un bloc post-await de l'IIFE : la marche interne l'a classé Deferred, et absorb conserve cette classe; block = None, mais guard_block = Some(0), le bloc de l'instruction qui l'a planifié. Le cleanup retourné est marché en Cleanup. La valeur de v est le résultat d'un await sur un callee opaque, donc $\top$, d'où fresh = Maybe. La ligne de commande ne signale rien.

La figure 14.2 dessine ce que la marche voit : trois CFG, un seul prov_block.

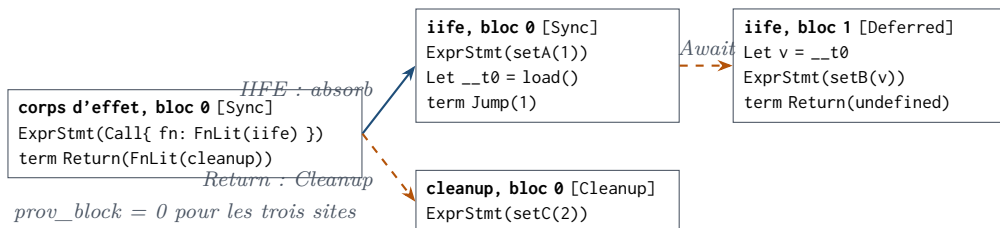


FIGURE 14.2 – La marche sur l'exemple 14.3. Le corps de l'IIFE est entré en Sync au site d'appel; const v = await load() est abaissé en Let __t0 = load(), fin de bloc, puis Let v = __t0 dans le bloc suivant (blocs tels que les imprime la sonde /tmp/verif07/irprobe), et l'arête EdgeKind::Await qui les relie fait basculer la marche en Deferred pour le bloc 1; la fonction retournée est marchée en Cleanup. Les trois sites héritent du bloc de provenance 0, seul identifiant qui ait un sens dans le CFG de l'effet.

14.2.7 Les arguments fonctionnels

Après le callee viennent les arguments. Chaque argument fonctionnel reçoit une classe décidée par arg_class :

Extrait 14.13 – src/engine/setters.rs, l. 1626–1651 — la classe d'un argument fonctionnel

```

1626 fn arg_class(&self, fn_: &Expr, mode: WalkClass) -> WalkClass {
1627     use crate::engine::registrations::Timing;
1628     if let Some((reg, _)) = crate::engine::registrations::match_registrar(fn_) {
1629         let shadowed = !reg.method_only
1630         && matches!(fn_.peel_ts(), Expr::Var(n) if self.shadowed.contains(n.as_str()));
1631         if !shadowed {
1632             return match reg.timing {
1633                 Timing::Deferred => WalkClass::Deferred,
1634                 Timing::Handler => WalkClass::Handler,
1635                 Timing::Unknown => WalkClass::Unknown,
1636             };
1637         }
1638     }
1639     // A proven wrapper hands its callback to the handler it returns, so
1640     // the callback does not run in this call's phase (#94). Proven is the
1641     // operative word: `wrapper_callees` resolved the spelling through a
1642     // library summary and checked this component never invokes the handler
1643     // itself.
1644     if call_free_key(fn_).is_some_and(|k| self.wrappers.contains(&k)) {
1645         return WalkClass::Handler;
1646     }

```



```

1647     match fn_ {
1648         Expr::FieldAccess { field, .. } if SYNC_HOF_METHODS.contains(&field.as_str()) => mode,
1649         _ => WalkClass::Unknown,
1650     }
1651 }

```

Trois sources, dans cet ordre : la table des registrars (section 14.3), dont le `timing` devient la classe, sauf si le nom nu est ombré localement ; un wrapper prouvé par résumé de bibliothèque (`form.handleSubmit(cb)`, #94), qui rend `Handler` ; enfin une méthode de `SYNC_HOF_METHODS` (`src/engine/setters.rs`, l. 1460–1474 : `map`, `forEach`, `filter`, `reduce`, `reduceRight`, `find`, `findIndex`, `findLast`, `findLastIndex`, `some`, `every`, `flatMap`, `sort`), qui propage le mode courant. Tout le reste est `T`.

La boucle sur les arguments (`src/engine/setters.rs`, l. 1941–1985) ajoute quatre règles :

- un listener `addEventListener` de *premier niveau* d'un corps d'effet a déjà été réifié en `HookEntry::Handler` par `extract_subscriptions` (`src/lowering/hook_extractor.rs`, l. 18) : on ne le redescend pas, sous peine de le compter deux fois ; ailleurs, le même motif est classé `Handler` ;
- un *teardown* (`is_teardown : removeEventListener, clearInterval, ...`) n'est jamais descendu : « a teardown takes the callback back, it does not call it » (ADR-34 §5) ;
- un callback d'un HOF synchrone rend ses écritures `repeating` ;
- une `FnLit` en argument coûte un niveau de profondeur ; une variable liée (bras **B5**) est résolue *sans* coût de profondeur. C'est le bras qui peut *cycler* (`const tick = t => raf(tick)`), et c'est la pile *walking* qui le termine (deux tests unitaires de terminaison, `src/engine/setters.rs`, l. 2401–2490).

La profondeur maximale est 2 (`collect_write_sites(cfg, ..., 2, ...)` dans `push_sites`, `src/engine/setters.rs`, l. 1077–1085) : un helper qui appelle un helper est vu, un troisième niveau ne l'est pas. Ce plafond est une des raisons pour lesquelles l'absence de ligne n'est pas une preuve.

14.2.8 Comment une `FnLit` est-elle entrée ?

C'est le point le plus souvent mal compris de la marche. `expr` visite tous les enfants d'une expression *sauf* les littéraux de fonction :

Extrait 14.14 – `src/engine/setters.rs`, l. 1870–1875 — les `FnLit` ne sont jamais traversées de l'extérieur

```

1870     let mut children: Vec<&Expr> = Vec::new();
1871     expr.for_each_child(&mut |c| {
1872         if !matches!(c.peel_ts(), Expr::FnLit { .. }) {
1873             children.push(c);
1874         }
1875     });

```

Une fonction littérale n'est descendue que par la *machinerie d'appel* : comme argument d'un `Call/New` (avec la classe de `arg_class`), comme callee d'une IIFE, comme corps d'un helper local appelé (B6) ou d'un nom passé en argument (B5), ou comme `cleanup` retourné. Le commentaire d'ADR-38 §1 le dit : « skipping `FnLit` children — those are entered by the machinery, which is the only place that knows what class the function runs in ».

Piège

Une `FnLit` placée en *prop JSX* (`onClick=() => setN(1)`) n'est donc jamais marchée depuis le CFG de rendu : elle n'y produit *aucune* ligne. La seule ligne de ce site vient du corps réifié en `HookEntry::Handler` par `extract_handlers` (`src/lowering/hook_extractor.rs`, l. 100), marché comme sa propre région. Sur `/tmp/verif07/inl.tsx` (`<button onClick=() => setN(1)>`), la sonde donne une seule ligne, `region=Handler(1) phase=Handler`. Le commentaire de `collect_slot_writers` (`src/engine/setters.rs`, l. 991–996) dit qu'une ligne `T` dupliquée « is dropped in favor of the handler row » : il décrit ce *résultat*, non un code ; aucune déduplication explicite n'existe dans la fonction (`src/engine/setters.rs`, l. 997–1229). C'est la règle structurelle « une `FnLit` n'est entrée que par la machinerie » qui l'assure.

Décision : ADR-38 §1, une traversée sans porte

Avant #130, `expr` ne traitait un appel qu'en position d'instruction ; un `setX(1)` dans un argument, un bras de ternaire ou une *prop JSX* était invisible. ADR-38 rend la traversée complète : chaque nœud est visité exactement une fois et la machinerie d'appel s'applique à tout `Call` rencontré, sur les trois canaux. « A write is a write wherever it is written. » Corpus : rien retiré, 37 lignes ajoutées, temps d'exécution inchangé sur 34 730 fichiers.

14.2.9 Le témoin

Une instruction sans *span* (un terminateur `Return`, un corps de flèche à expression) hérite du *span* du site par lequel le corps a été entré : c'est le paramètre *witness* que l'on voit circuler dans tous les appels, et un élément *JSX* rafraîchit le *span* pour son sous-arbre (`src/engine/setters.rs`, l. 1861–1869). ADR-39 §3 a introduit ce mécanisme parce que la marche passait `None` pour tout terminateur et toute descente, jetant la position sur laquelle elle venait de se tenir. Le résidu de ce mécanisme, une ligne *sans span* pour un handler à corps-expression marché comme racine de région, est traité au section 14.12.

14.2.10 Ce que la marche garantit**Invariant 14.1: Soundness de la marche des setters**

1. **`T` par défaut.** Tout argument fonctionnel d'un callee inconnu est `Unknown`, qui satisfait toute requête. Rétrécir hors de `T` n'est permis que sur *contrat* : un registrar de `timing` `Deferred` ou `Handler`, un wrapper prouvé par résumé de bibliothèque et non ré-invoqué, une arête `Await`.
2. **Functional n'est affirmé que sur un littéral ou un nom certifié** (`certified_fns`) : c'est une affirmation `must`, posée sur un chemin de suppression.
3. **`same_tick = false` n'est pas une promesse** : la profondeur est bornée et l'attribution se fait au site appelant.
4. **La granularité par site est monotone.** Ajouter des lignes ne fait perdre aucun appariement existentiel (ADR-28 §1).

14.3 La table des registrars : contrat contre devinette

La marche a besoin, pour chaque appel qui reçoit un callback, d'un *résumé* : ce callback sera-t-il exécuté maintenant, sur un tour ultérieur, sur un événement, ou ne sait-on pas ? Avant ADR-34, deux listes de noms coexistaient : les `DEFERRING_GLOBALS` / `DEFERRING_METHODS` de la marche, et le `REGISTRARS` des règles *stale-closure* et *missing-cleanup*, qui se recouvraient sur les

timers et les continuations de promesse et pouvaient dériver. Elles sont devenues *une* table, REGISTRARS, dans `src/engine/registrations.rs`, avec une colonne par lecteur.

Extrait 14.15 – `src/engine/registrations.rs`, l. 63–78 — une ligne de la table des registrars

```
63 #[derive(Debug)]
64 pub struct Registrar {
65     pub name: &'static str,
66     /// Index of the callback argument (`addEventListener('click', cb)` → 1).
67     pub cb_arg: usize,
68     pub firing: Firing,
69     pub method_only: bool,
70     pub timing: Timing,
71     /// Calls that undo this registration. Empty when the registration cannot
72     /// be taken back (`queueMicrotask`, a promise continuation).
73     pub teardown: &'static [&'static str],
74     /// What those calls are handed (#124). `clearInterval` takes the *handle*
75     /// the registration returned, `removeEventListener` takes the listener –
76     /// comparing the wrong one answers `none-seen` on correct code.
77     pub teardown_takes: TeardownArg,
78 }
```

`cb_arg` est l'indice de l'argument callback (`addEventListener('click', cb) → 1`); `firing` dit si le callback tire une fois (Once) ou indéfiniment (Repeating), un *axiome* de la table; `method_only` exige la forme `recv.name(...)` (un `then(...)` nu est trop ambigu pour affirmer quoi que ce soit); `timing` est le résumé de phase que lit la marche; `teardown` et `teardown_takes` servent à l'appariement (section 14.8).

name	cb_arg	firing	method_only	timing	teardown	teardown_takes
<code>setInterval</code>	0	Repeating	false	Deferred	<code>clearInterval</code>	Handle
<code>addEventListener</code>	1	Repeating	false	Handler	<code>removeEventListener</code>	Listener
<code>subscribe</code>	0	Repeating	true	Unknown	<code>unsubscribe</code>	Listener
<code>on</code>	1	Repeating	true	Unknown	<code>off</code> , <code>removeListener</code>	Listener
<code>addListener</code>	1	Repeating	true	Unknown	<code>removeListener</code>	Listener
<code>setTimeout</code>	0	Once	false	Deferred	<code>clearTimeout</code>	Handle
<code>setImmediate</code>	0	Once	false	Deferred	<code>clearImmediate</code>	Handle
<code>requestAnimationFrame</code>	0	Once	false	Deferred	<code>cancelAnimationFrame</code>	Handle
<code>requestIdleCallback</code>	0	Once	false	Deferred	<code>cancelIdleCallback</code>	Handle
<code>queueMicrotask</code>	0	Once	false	Deferred	—	Handle
<code>then</code>	0	Once	true	Deferred	—	Handle
<code>catch</code>	0	Once	true	Deferred	—	Handle
<code>finally</code>	0	Once	true	Deferred	—	Handle

TABLE 14.4 – Le contenu de REGISTRARS au commit e67b10a (`src/engine/registrations.rs`, l. 93–211), treize lignes. Trois registrars ne peuvent pas être défaits (`teardown` vide) : une continuation de promesse ne se retire pas.

La colonne `timing` n'est pas uniforme, et c'est là toute la décision.

Décision : ADR-34 §2, un contrat, pas un nom

Deferred couvre les timers, les microtâches et les continuations de promesse : le callback tourne sur un tour ultérieur de la boucle d'événements, prouvablement hors de toute phase React. Handler ne couvre que `addEventListener` : le DOM n'a pas de dispatch synchrone depuis un enregistrement, donc le listener ne tourne prouvablement pas pendant l'appel. Unknown couvre `subscribe`, `on`, `addListener` : un BehaviorSubject RxJS émet *sur-le-champ* au nouvel abonné, si bien qu'une exécution synchrone n'est pas exclue et que la marche doit

garder $\top$. L'alternative, classer `subscribe` en `Handler` par analogie de nom, aurait *perdu* des findings : `WriterPhase` est `may`, $\top$ satisfait toute requête, et rétrécir une ligne d'`Unknown` vers `Handler` est la seule direction qui puisse en faire disparaître. Cette direction n'est permise que là où le timing est un contrat de la plateforme, non une devinette de table de noms. La décision « faux positif accepté » de l'issue #42 (wontfix) achète de la sur-approximation ; elle n'achète pas cela.

Rappel React : dispatch DOM et observables

`addEventListener` n'appelle jamais le listener pendant l'appel d'enregistrement ; il faut un `dispatch`, donc un événement ou un `dispatchEvent` explicite, tous deux extérieurs au passage courant. À l'inverse, beaucoup de stores « `subscribe` » (RxJS `BehaviorSubject`, certains stores Redux-like) appellent immédiatement le callback avec la valeur courante. Le premier fait est un contrat de la plateforme ; le second est une propriété de chaque bibliothèque, que l'analyseur ne connaît pas. D'où `Handler` pour l'un et $\top$ pour les autres.

L'appariement d'un callee avec la table est `match_registrar` (`src/engine/registrations.rs`, l. 292–313) : le callee est un `Var(name)` (forme nue) ou un `FieldAccess obj, field` (forme méthode), et la première ligne dont `name` correspond et dont `method_only` est compatible avec la forme est retenue. Un registrar non `method_only` (`setInterval`, `addEventListener`, ...) est reconnu aussi bien nu qu'en position de méthode (`window.setTimeout`, `el.addEventListener`) ; la chaîne `display` rendue (`setInterval`, `socket.addEventListener`, `.then`) est celle qu'affichent les diagnostics. C'est le *seul* appariement par nom du sous-système, et c'est pourquoi l'ombrage existe : dans `arg_class` (liste 14.13), un nom nu lié localement (`let setTimeout = ...`) n'active pas le résumé. L'ombrage ne s'applique qu'à la forme nue : `!reg.method_only && matches!(fn_, Expr::Var(n) if shadowed...)`.

Enfin, `is_teardown` (`src/engine/registrations.rs`, l. 281–288) reconnaît un callee de la colonne `teardown`. Son commentaire souligne l'asymétrie : lire la table *rétrécit* ici (un `teardown` n'est pas descendu), alors qu'elle *élargit* côté registrar ; l'ensemble est donc volontairement limité aux partenaires de `teardown` des registrars déjà en table, « nothing joins it without a registration to undo ».

14.4 La ligne SlotWriter, colonne par colonne

14.4.1 Une ligne par site

Exemple 14.4: Deux écritures dans un handler

Extrait 14.16 – `/tmp/reprobe/ex1.tsx`

```
1 import { useState } from 'react';
2 export function Counter() {
3   const [count, setCount] = useState(0);
4   const inc = () => {
5     setCount(count + 1);
6     setCount(count + 1);
7   };
8   const safe = () => setCount(c => c + 1);
9   return <button onClick={inc} onDoubleClick={safe}>{count}</button>;
10 }
```

slot=0 setter=setCount line=Some(5) region=Handler(1) phase=Handler via=Direct updater=Unknown same_tick=true

```

↪ block=Some(0) fresh=Maybe
slot=0 setter=setCount line=Some(6) region=Handler(1) phase=Handler via=Direct updater=Unknown same_tick=true
↪ block=Some(0) fresh=Maybe
slot=0 setter=setCount line=None region=Handler(2) phase=Handler via=Direct updater=Functional same_tick=false
↪ block=Some(0) fresh=Not

```

Les deux appels de `inc` sont *deux lignes*, toutes deux `same_tick = true` : elles sont dans le même bloc du handler. La ligne de `safe` porte `updater = Functional` (le littéral `c => c + 1`) et `fresh = Not`. Elle n'a pas de `span` : le corps d'une flèche à expression est un simple terminateur `Return`, sans position, et marché comme *racine* de région il n'hérite d'aucun témoin (section 14.12). La ligne de commande ne dit rien : aucune règle native ne lit le motif « deux écritures non fonctionnelles du même slot dans un tick » (#61), mais un pack Tier-A peut l'exprimer depuis `same_tick` et `updater`.

Décision : ADR-28 §1, la granularité par site

Les lignes étaient autrefois clées (*region*, variable de *setter*, classe) : deux `setCount(count + 1)` dans un handler faisaient *une* ligne. Ce repli, délibéré et documenté, est renversé : une ligne par site d'appel. « A relation that cannot say there are two writes cannot express a rule about two writes », et aucun vocabulaire de garde ne recrée une distinction que la relation a jetée. Multiplier les lignes est un *raffinement monotone* (mêmes slots, mêmes phases, plus de témoins), et tout consommateur de la relation stockée la lit existentiellement : une requête qui apparait avant ne peut cesser d'apparier. Le seul consommateur qui veut l'ancienne forme, `collect_setter_calls`, replie *explicitement* à la fin (`src/engine/setters.rs`, l. 137–157), en préférant le site le plus synchrone.

14.4.2 La structure

Extrait 14.17 – `src/engine/setters.rs`, l. 728–786 — la ligne `SlotWriter`

```

728 /// One writer of a state slot, one row **per call site**.
729 ///
730 /// The granularity used to be one row per (region, alias-resolved setter
731 /// variable, phase class), collapsing two `setCount(count + 1)` calls in one
732 /// body into a single row. That is reversed here, and the reversal is the
733 /// point: the canonical stale-read shape is *two* non-functional writes of one
734 /// slot in one handler, and a relation that cannot say there are two cannot
735 /// express it. Multiplying rows is monotone – same slots, same phases, more
736 /// witnesses – and every shipped consumer of the stored relation reads it
737 /// existentially, so nothing that matched before stops matching (ADR-028 §2).
738 #[derive(Debug, Clone)]
739 pub struct SlotWriter {
740     pub slot: HookLabel,
741     /// The setter variable at the call site (an alias chain resolves it to
742     /// `slot`; a spliced wrapper's `setter#salt` param resolves too).
743     pub setter: Var,
744     /// Witness call-site span.
745     pub span: Option<SourceRange>,
746     pub region: WriterRegion,
747     pub phase: WriterPhase,
748     /// Whether the write is caller-authored or reached through inlined
749     /// wrappers (ADR-027 §4).
750     pub via: WriteProvenance,
751     /// Argument 0 of this write.
752     pub updater: Updater,
753     /// MAY: another write of the same slot in this region is CFG-reachable
754     /// from this one, so the two can land in the same tick – self-reachability
755     /// through a back edge included, which is how a single write inside a loop

```

```

756    /// pairs with itself.
757    ///
758    /// A per-row boolean, never a fold over the edge: precomputing it here is
759    /// what keeps a Tier-A rule about co-executing writes single-anchor and
760    /// existential, the same move that dissolved the effect+handler join in
761    /// ADR-027 §1. It is may-typed in one direction only – the walk is
762    /// depth-capped, so a write it never saw cannot make this `false` a
763    /// promise, which is why no guard may assert the negative.
764    pub same_tick: bool,
765    /// The component that owns `slot` when it is not this one: the write went
766    /// through a `ComponentSetter` prop (ADR-042 §2, the ADR-030 §1 device).
767    /// `None` for a local slot. A foreign row's `slot` is the OWNER's label,
768    /// so a reader that resolves it against this component's tables names an
769    /// unrelated slot (ADR-030 §3); every native reader filters on `owner`.
770    /// A foreign row is a may-write: the prop may be a closure that merely
771    /// carries the setter.
772    pub owner: Option<ComponentId>,
773    /// The block of the region's CFG the write runs in, for a write that runs
774    /// synchronously in the region, once per pass – what `must_on_all_paths`
775    /// takes. `None` for a nested, deferred, cleanup or repeating site.
776    pub block: Option<BlockId>,
777    /// The block of the region's CFG whose dominating guards the write runs
778    /// under: `block` for a synchronous write, the statement that scheduled
779    /// it for a nested, deferred, cleanup or repeating one – a continuation
780    /// runs only if the run that scheduled it took every branch above the
781    /// scheduling statement (#160). `None` for a row with no placeable
782    /// block.
783    pub guard_block: Option<BlockId>,
784    /// What the write stores (ADR-042 §2).
785    pub written: Written,
786 }

```

Définition 14.2: La ligne SlotWriter

Une ligne de `slot_writers` est un *écrivain* : un site d'appel de setter d'un slot, avec sa cible (`slot`, `owner`), sa position (`span`, `region`, `block`, `guard_block`), son moment (`phase`), sa provenance (`via`), son argument (`updater`, `written`) et sa co-exécution (`same_tick`). Le tableau 14.5 donne la polarité de chaque colonne.

14.4.3 La cible et le propriétaire

La table des cibles est construite en tête de `collect_slot_writers` : d'abord les setters locaux (`setter_var_labels` : chaque `let v = StateSetter(1)` du rendu, puis `resolve_setter_aliases` sur le rendu et chaque corps, un point fixe sur `let a = b / a = b`, `src/engine/setters.rs`, l. 581–619), puis les étrangers, *sans écraser* un local (`src/engine/setters.rs`, l. 1009–1020 : `targets.entry(v).or_insert(...)`, « a local binding wins a tie »). Les noms étrangers ne sont pas alias-résolus, parce que `cross_component_setters` lit déjà la valeur convergée de chaque liaison du rendu.

Un setter étranger est décrit par `SetterProp` :

Extrait 14.18 – `src/engine/setters.rs`, l. 245–259 — un setter reçu d'un autre composant

```

245 #[derive(Debug, Clone, PartialEq, Eq)]
246 pub(crate) struct SetterProp {
247     /// The component that owns the slot.
248     pub component: ComponentId,
249     pub label: HookLabel,
250     /// Calling the variable provably writes the slot – a must.

```

Colonne	Affirmation	Polarité	Remplie depuis
slot	label du slot, dans le composant propriétaire	exact	targets[var]
owner	None : slot local ; Some(parent) : écriture via une prop ComponentSetter	exact ; la ligne est une écriture may	foreign
setter	variable au site, alias résolu	exact	FoundSite::var
span	site témoin	exact ; None si non plaçable	span d'instruction, JSX, ou témoin hérité
region	corps lexical	exact	la région marchée
phase	quand ça s'exécute	may, T-bearing	class_phase(class, region)
via	Direct / Via(chaîne) / Unknown	exact ; Direct certifiable	InlineRegions::chain
updater	Functional(corps) ou T	must pour Functional	updater_of
same_tick	une autre écriture synchrone du même slot, même région, atteignable	may, unidirectionnel	Reachability
block	bloc de la région, écriture Sync non répétée	exact ; None sinon	site.at
guard_block	bloc dont les gardes dominantes s'appliquent	exact ; None sinon	site.prov_block
written.fresh	Fresh / Maybe / Not	must pour Fresh et Not	written::classify
written.value	valeur stockée	may	SiteEnvs::eval
written.expr	argument 0 tel qu'écrit	exact	FoundSite::arg

TABLE 14.5 – Les colonnes de slot_writers et leur polarité (d'après docs/relations.md et le code).

```

251  ///
252  /// True when the variable *is* the setter, and when it is a closure whose
253  /// own body calls the setter it captured (`reset={() => setCount(0)}`).
254  /// False when the capture is only carried: `renderFileTree`, a local render
255  /// helper closing over an `onFileClick` prop it merely puts inside a JSX
256  /// handler, writes nothing when called. Both are still writes for a *may*
257  /// reader – T over what the callee does – but only a must may be certified.
258  pub must_write: bool,
259  }

```

must_write distingue une variable qui *est* le setter (ou une closure dont le corps l'appelle, `reset={() => setCount(0)}`) d'une closure qui ne fait que le *porter* ; les deux sont des écritures pour un lecteur may, seule la première peut être certifiée (ADR-38 §4). Le calcul (`collect_component_setter_vars`, `src/engine/setters.rs`, l. 275–349) parcourt les noms liés du rendu dans l'ordre de `cfg.blocks` (un `BTreeMap`, donc déterministe, #120) et interroge la valeur convergée de chaque bloc : `ComponentSetter(c, 1) => must_write: true` ; un `EnvVal::Loc` vers une fonction du tas qui capture un setter $\Rightarrow$ `must_write = « son corps appelle-t-il ce setter en Sync ? »`. Le premier environnement qui résout un nom décide (la question « lequel devrait gagner » est #119).

Exemple 14.5: Une ligne étrangère

Dans `/tmp/verif07/e14.tsx`, `Seed` rend `<Child onSet=setS3 ... />` et `Child` appelle `onSet(v)` dans un `onClick`. `Child`, rendu par `Seed`, n'est pas une racine : il est analysé top-down (chapitre 16) et sa prop `onSet` vaut `ComponentSetter(Seed, 2)`. Sa relation contient :

```
=== component Child
```

```
slot=2 owner=Some("Seed") setter=onSet line=None region=Handler(0) phase=Handler via=Direct updater=Unknown
↳ same_tick=false block=Some(0) fresh=Maybe
```

Le slot est *nommé dans le propriétaire* : c'est le label 2 de Seed (s3), qui n'a rien à voir avec un éventuel label 2 de Child.

Décision : ADR-30, lignes qualifiées par leur propriétaire

Trois points. (§1) Les lignes étrangères viennent de la résolution du moteur lui-même (cross_component_setters), pas d'une seconde passe ; un local gagne l'égalité. (§2) Côté packs Tier-A, l'énumération *s'élargit sur la garde*, jamais sur la sorte : une règle qui ne nomme pas slot_ownership continue de lier exactement les lignes qu'elle liait ; l'alternative, élargir inconditionnellement, aurait changé en silence les findings de tous les packs déployés. (§3) Un HookLabel est par composant ; résoudre le label du parent dans la table de l'enfant nommerait un slot local sans rapport, « a wrong name, not a missing one ». D'où l'invariant : *tout lecteur natif filtre owner.is_none()* (par exemple src/engine/seeds.rs, l. 114–117).

14.4.4 same_tick : la co-exécution comme fait de ligne

Extrait 14.19 – src/engine/setters.rs, l. 1098–1143 — same_tick, block et written pour chaque site

```
1098     let mut sync_blocks: HashMap<(Option<ComponentId>, HookLabel), Vec<BlockId>> =
1099         HashMap::new();
1100     for s in &sites {
1101         if s.class == WalkClass::Sync
1102             && let (Some(&target), Some(b)) = (targets.get(&s.var), s.prov_block)
1103         {
1104             sync_blocks.entry(target).or_default().push(b);
1105         }
1106     }
1107     let reach = Reachability::of(cfg);
1108     for site in sites {
1109         let Some(&(owner, slot)) = targets.get(&site.var) else {
1110             continue;
1111         };
1112         let same_tick = site.repeats
1113             || match (site.class, site.prov_block) {
1114                 (WalkClass::Sync, Some(b)) => {
1115                     let blocks = sync_blocks
1116                         .get(&(owner, slot))
1117                         .map_or(&[.], Vec::as_slice);
1118                     // Co-execution is symmetric – "these two land in the
1119                     // same tick" does not care which runs first – so the
1120                     // question is reachability in EITHER direction, plus
1121                     // the same block, plus this block through a back edge.
1122                     blocks.iter().filter(|t| **t == b).count() > 1
1123                     || reach.reaches(b, b)
1124                     || blocks
1125                         .iter()
1126                         .any(|&t| reach.reaches(b, t) || reach.reaches(t, b))
1127                 }
1128                 _ => false,
1129             };
1130         let updater = site.updater.clone();
1131         // A position only a write that runs synchronously in this region,
1132         // once per pass, can claim: a repeating site runs 0..N times and
1133         // every other class runs on another turn or in another CFG.
1134         let at = (site.class == WalkClass::Sync && !site.repeats)
```



```

1135         .then_some(site.at)
1136         .flatten();
1137     let block = at.map(|(b, _)| b);
1138     let written = written::classify(
1139         site.arg.as_ref(),
1140         &site.updater,
1141         (owner.unwrap_or(envs.component), slot),
1142         |e| envs.eval(region, cfg, at, e),
1143     );

```

`same_tick` est vrai si le site est `repeats` (un callback de HOF synchrone, ou une écriture atteinte deux fois par un helper appelé deux fois), ou si, pour un site `Sync`, un autre site `Sync` du même slot est dans le même bloc, ou atteignable dans *l'une ou l'autre* direction, ou si le bloc s'atteint lui-même. La clé est `prov_block`, jamais le bloc interne. Le commentaire insiste : la co-exécution est symétrique. Juste avant, `dedup_source_sites` (`src/engine/setters.rs`, l. 913–931) a replié les copies d'un même site source atteint depuis deux appels d'un helper, en marquant la ligne survivante `repeats = true` : « being reached from two call sites is exactly co-execution ».

Décision : ADR-28 §3, un booléen par ligne, jamais un pli

Le fait « deux écritures du même slot dans un tick » est une relation *entre* deux lignes. La pousser dans un booléen *sur* chaque ligne est ce qui garde les règles Tier-A mono-ancre et existentielles (pas de $\forall$ sur `writers`, pas de seconde liaison), le même geste qui avait dissous la jointure effet + handler d'ADR-27 §1. Et le booléen est *may* dans une seule direction : la marche est bornée en profondeur, une écriture qu'elle n'a pas vue ne peut pas faire de fausse promesse, donc aucune garde ne peut affirmer le négatif (ADR-28 §5). Limite connue : deux écritures de branches mutuellement exclusives à l'intérieur d'un helper inline sont lues co-exécutées (#123).

14.4.5 Updater : l'argument 0 est-il une fonction prouvée ?

Extrait 14.20 – `src/engine/setters.rs`, l. 709–720 — la colonne `Updater`

```

709 #[derive(Debug, Clone)]
710 pub enum Updater {
711     /// Proven a function literal: an inline `set(prev => ...)` , or a variable
712     /// bound exactly once to one and never re-bound. The body is kept for the
713     /// consumers that need to look inside it.
714     Functional(Arc<CFG>),
715     /// T – everything else: a value expression, a call, an argument the walk
716     /// could not resolve to a literal, or no argument at all. Only a *proven*
717     /// function literal escapes this, so a rule keyed on "not functional"
718     /// over-reports rather than missing a write.
719     Unknown,
720 }

```

Extrait 14.21 – `src/engine/setters.rs`, l. 1303–1316 — comment l'argument est classé

```

1303 fn updater_of(&self, args: &[Expr]) -> Updater {
1304     match args.first().map(Expr::peel_ts) {
1305         Some(Expr::FnLit { body_cfg, .. }) => Updater::Functional(Arc::clone(body_cfg)),
1306         Some(Expr::CallbackVal(l)) => self
1307             .callback_bodies
1308             .get(l)
1309             .map_or(Updater::Unknown, |b| Updater::Functional(Arc::clone(b))),
1310         Some(Expr::Var(v)) if self.certified_fns.contains(v) => self
1311             .fn_bindings

```



```

1312         .get(v)
1313         .map_or(Updater::Unknown, |b| Updater::Functional(Arc::clone(b))),
1314         _ => Updater::Unknown,
1315     }
1316 }

```

Trois formes seulement sont `Functional` : un `FnLit` inline ; un `CallbackVal(1)` dont le corps de `useCallback` est connu ; une variable *certifiée*, c'est-à-dire liée une seule fois à un littéral et jamais re-liée dans aucune closure en dessous (`certified_fn_binding`, `src/ir/bindings.rs`, l. 151–160). `collect_fn_bindings` n'est pas cette barre : c'est un `HashMap::insert`, il garde la *dernière* liaison d'un nom re-lié. Tout le reste est `⊥`, de sorte qu'une règle qui tire sur « non fonctionnel » sur-rapporte plutôt que de manquer une écriture. ADR-28 §2 refuse deux colonnes (fonctionnel + pureté) : une seule colonne enregistre l'*expression*, chaque consommateur en dérive son verdict.

14.4.6 block et guard_block

Les deux colonnes de position ont des rôles distincts, et il est facile de les confondre avec les champs internes de la marche.

Piège

Quatre notions de bloc coexistent. `at` (interne) est (bloc, index d'instruction), seulement en `Sync`, et sert à l'environnement d'évaluation. `block` (colonne) vaut `at.0` si le site est `Sync` et non répété : « a position only a write that runs synchronously in this region, once per pass, can claim » ; c'est ce que `must_on_all_paths` prend. `guard_block` (colonne) vaut `prov_block` : toujours un bloc de la région, et pour un site différé, *l'instruction qui l'a planifié* ; une continuation ne s'exécute que si le passage qui l'a planifiée a pris toutes les branches au-dessus (#160). Ne jamais comparer un `BlockId` d'un corps imbriqué avec un bloc de la région. Notons qu'ADR-42 §2 décrit `block` comme « the walk's `prov_block`, exposed » : au commit e67b10a, c'est `guard_block` qui expose `prov_block`, et `block` vient de `at`.

Invariant 14.2: Positions d'une ligne

`block.is_some()` implique que `phase` est la phase synchrone de la région et que le site n'est pas `repeats`. `guard_block` est toujours un bloc du CFG de la région, ou `None` pour un site non plaçable. Sur l'exemple 14.2, seule `setData(null)` a un `block` ; les quatre lignes ont `guard_block = Some(0)`.

14.4.7 via : qui a écrit l'appel

Extrait 14.22 – `src/engine/setters.rs`, l. 789–804 — la provenance d'une écriture

```

789 #[derive(Debug, Clone, PartialEq, Eq)]
790 pub enum WriteProvenance {
791     /// Caller-authored: the site sits outside every recorded splice region.
792     /// This is a certainty, not a may-fact – regions are recorded at the
793     /// single splice primitive, exhaustively – which is what
794     /// `must_direct_write` certifies.
795     Direct,
796     /// Reached through inlined wrappers, outermost first
797     /// ([`putState`, `helper`] for `putState → helper → setX`). A write in
798     /// an inlined custom hook's body carries the hook's origin name first.
799     Via(Vec<Symbol>),
800     /// The site could not be placed (a defensive splice path grafted
801     /// statements outside any recordable range) – fails every provenance
802     /// guard and certifies nothing.

```

```
803   Unknown,
804 }
```

Le lowering splice les utilitaires et les hooks personnalisés (chapitre 8) en enregistrant, à la primitive de splice, une *InlineRegion* par callee : une plage de blocs *start..end*, le nom *exporté* du callee, le fichier d'origine et l'indice de la région englobante (*src/engine/setters.rs*, l. 806–834). Les plages sont disjointes (chaque splice alloue strictement au-dessus), donc au plus une région contient un bloc, et *chain* remonte les *parent* :

Extrait 14.23 – *src/engine/setters.rs*, l. 847–858 — de la plage de blocs à la chaîne de wrappers

```
847   // Ranges are disjoint (each splice allocates strictly above), so at
848   // most one region contains the block; parents encode the nesting.
849   let mut at = regions
850     .iter()
851     .position(|r| r.start <= block && block < r.end);
852   let mut chain: Vec<Symbol> = Vec::new();
853   while let Some(i) = at {
854     chain.push(regions[i].name.clone());
855     at = regions[i].parent;
856   }
857   chain.reverse();
858   chain
```

Le calcul de *via* (*src/engine/setters.rs*, l. 1144–1179) préfixe la chaîne par l'origine du hook si la région est le corps d'un hook custom inliné, puis : pas de *prov_block* et chaîne vide → *Unknown* (jamais *Direct* sur un site non plaçable) ; chaîne vide dans un CFG de rendu *render_poisoned* (un splice a dû greffer dans le bloc d'entrée) → *Unknown* ; sinon *Direct* ou *Via*(chaîne).

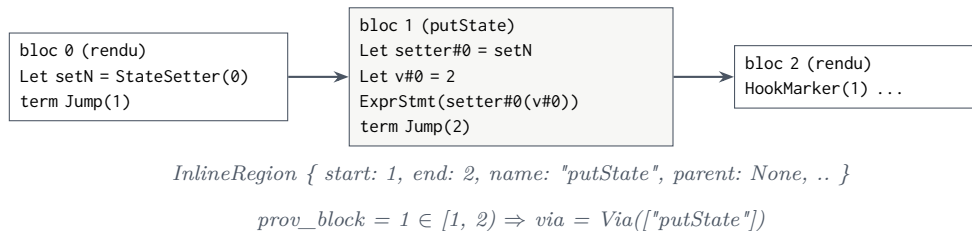


FIGURE 14.3 – Provenance d'une écriture splicée (exemple 14.6). Blocs tels que la sonde les imprime. Le splice émet le préfixe de paramètres `let setter#0 = setN` en tête du premier bloc du callee, avec le span du site d'appel (ligne 5 de *App.tsx*, ADR-39 §2), enregistre la plage de blocs du callee, et *resolve_setter_aliases* rattache *setter#0* au slot 0. Une chaîne *putState* → *helper* → *setX* donnerait *Via*(["putState", "helper"]).

Exemple 14.6: Un utilitaire importé écrit au rendu

Extrait 14.24 – */tmp/ch14/via/helpers.ts* et */tmp/ch14/via/App.tsx*

```
1 // helpers.ts
2 export function putState(setter, v) {
3   setter(v);
4 }
5 // App.tsx
6 import { putState } from "../helpers";
7 import { useState, useEffect } from "react";
8 export function App({ items }) {
9   const [n, setN] = useState(0);
```

```

10 putState(setN, 2);
11 useEffect(() => { putState(setN, items.length); }, [items]);
12 return <div>{n}</div>;
13 }

```

Une sonde multi-fichiers (`lower_files + analyze_lowered`, `/tmp/verif07/mprobe`) donne pour App :

```

slot=0 setter=setter line=Some(2) region=Render phase=Render via=Via(["putState"]) block=Some(1) fresh=Not
slot=0 setter=setter line=Some(2) region=Effect(1) phase=Effect via=Via(["putState"]) block=Some(1) fresh=Maybe

```

La colonne `setter` porte le paramètre renommé du callee (affiché par `source_name`, qui retire le sel de splice) et `line` est la ligne 2 *de* `helpers.ts` : le span propre de l’instruction du callee, dans son fichier. La ligne de commande, elle, ne signale rien sur `putState(setN, 2)` au rendu ; ce silence est analysé au section 14.11.

Le consommateur de `via` est la primitive `must_direct_write` :

Extrait 14.25 – `src/rules/api/query.rs`, l. 763–774 — la certification d’une écriture directe

```

763 pub fn must_direct_write(w: &crate::engine::SlotWriter) -> MustResult<DirectWrite> {
764     match w.via {
765         crate::engine::setters::WriteProvenance::Direct => MustResult::All(Certified::mint(
766             DirectWrite {
767                 setter: w.setter.clone(),
768                 slot: w.slot,
769             },
770             Provenance::at(w.span, Some(w.slot)),
771         )),
772         _ => MustResult::None,
773     }
774 }

```

Une ligne `Direct` est une certitude, non un fait *may* : les régions sont enregistrées à la primitive de splice unique, exhaustivement, et `Direct` n’est attribué qu’à un site dont le bloc de provenance est hors de toutes, dans un CFG qu’aucune greffe défensive n’a empoisonné. Les angles morts connus (appels de wrappers en position d’expression, #52 ; budget de splice, #54) retirent des *lignes*, ils ne rendent jamais incertain le `Direct` d’une ligne présente (`src/rules/api/query.rs`, l. 755–762).

14.4.8 Tri et déterminisme

Le tri final (`src/engine/setters.rs`, l. 1217–1228) par (`slot`, `region`, `phase`, `position`, `setter`) rend la relation déterministe et explique un détail de l’exemple 14.2 : la ligne `Handler` (ligne source 10) précède la ligne `Deferred` (ligne 8) parce que `Handler` < `Deferred` dans l’ordre dérivé de `WriterPhase`. Une ligne sans span trie en dernier (`u32::MAX`).

Piège

Le commentaire du champ `AnalysisResult::slot_writers` dit encore « one row per (region, alias-resolved setter variable, sync-vs-nested) » (`src/engine/analysis_result.rs`, l. 239–243) : c’est la granularité d’avant ADR-28, un commentaire périmé au commit `e67b10a`. La doc de la structure elle-même (`src/engine/setters.rs`, l. 728–737) dit juste : « one row **per call site** ». Même remarque pour la doc de `Found` (`src/engine/setters.rs`, l. 1346–1365), qui concatène deux commentaires dont le premier décrivait l’ancien type.

14.5 La colonne written : ce que l'écriture stocke

La colonne `written` porte trois faits sur l'argument 0 d'une écriture, dont le graphe de churn et les preuves de gardes ont besoin (`src/engine/written.rs`, doc de module, `src/engine/written.rs`, l. 1–16) : `fresh`, la jambe « must-change » d'une arête de churn ; `value`, la valeur abstraite stockée, pour les preuves qui rétrécissent dessus ; `expr`, l'argument tel qu'il est écrit, pour la preuve *relationnelle* (une garde compare le slot à l'expression même que l'écriture y stocke), « a claim about two spellings being one value, which no abstract value can carry ».

Extrait 14.26 – `src/engine/written.rs`, l. 37–58 — Freshness et Written

```
37 /// Does a write store a new reference every time it runs?
38 #[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord)]
39 pub enum Freshness {
40     Not,
41     /// May store a fresh reference (opaque value, imprecise updater).
42     Maybe,
43     /// Must store a fresh reference every call (`PerRender` argument).
44     Fresh,
45 }
46
47 /// Argument 0 of a write, as the churn proofs read it.
48 #[derive(Debug, Clone)]
49 pub struct Written {
50     pub fresh: Freshness,
51     /// The abstract value stored. A functional updater stores its return
52     /// value, approximated as a fresh reference: the proofs only ever read
53     /// the reference part, and a fresh-returning updater stores a truthy,
54     /// non-null one.
55     pub value: StateValue,
56     /// The argument as written, `None` for a bare `setX()`.
57     pub expr: Option<Expr>,
58 }
```

Freshness est un treillis à trois points, `Not < Maybe < Fresh` (figure 14.4), utilisé comme tel : `1.max(r).min(Freshness::Maybe)` dans le classement d'un opérateur binaire. `Fresh` est must (« à chaque appel, une référence neuve »), `Not` est must aussi (« jamais »), `Maybe` est l'inconnu.

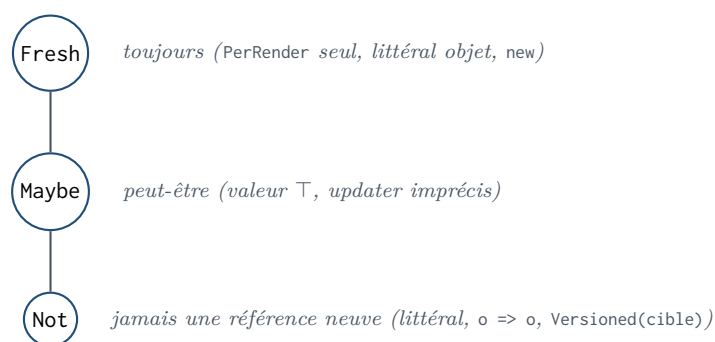


FIGURE 14.4 – Le treillis Freshness. Ce n'est pas un treillis d'approximation au sens du chapitre 3 : `Not` et `Fresh` sont deux affirmations must opposées, `Maybe` leur join.

Piège

Le commentaire du champ `value` (« approximated as a fresh reference ») est antérieur à `returns_value` : au commit `e67b10a`, un updater fonctionnel stocke la *jointure de ses return*, pas une référence de substitution. Le test `an_updater_stores_what_it_returns`

(src/engine/written.rs, l. 368) le vérifie; c'est ce qui permet à un `prev => null` d'un autre site de *ranimer* une garde `if (!s)` (chapitre 15).

14.5.1 Les trois cas de `classify`

Extrait 14.27 – src/engine/written.rs, l. 66–106 — le calcul de la colonne written

```

66 pub fn classify(
67     arg: Option<&Expr>,
68     updater: &Updater,
69     target: QualifiedSlot,
70     value_of: impl FnOnce(&Expr) -> StateValue,
71 ) -> Written {
72     let Some(arg) = arg else {
73         return Written {
74             fresh: Freshness::Not,
75             value: StateValue::top(),
76             expr: None,
77         };
78     };
79     let arg = arg.peel_ts();
80     let expr = Some(arg.clone());
81     match (arg, updater) {
82         (
83             Expr::FnLit {
84                 params, body_cfg, ..
85             },
86             _,
87         ) => Written {
88             fresh: returns_freshness(body_cfg, params),
89             value: returns_value(body_cfg),
90             expr,
91         },
92         (_, Updater::Functional(body)) => Written {
93             fresh: returns_freshness(body, &[]),
94             value: returns_value(body),
95             expr,
96         },
97         (other, Updater::Unknown) => {
98             let value = value_of(other);
99             Written {
100                 fresh: value_freshness(&value, target),
101                 value,
102                 expr,
103             }
104         }
105     }
106 }

```

1. **Pas d'argument** (`setX()`) : Not, valeur `⊥`.
2. **Updater fonctionnel** (littéral, ou `Updater::Functional`) : classé *sans environnement*, puisque l'updater s'exécute dans sa propre portée. `returns_freshness` donne Fresh si tous les return le sont, Not si aucun, Maybe sinon ou si rien ne retourne; `returns_value` (src/engine/written.rs, l. 113–122) est la jointure des `StateValue::from_init` de ses return : une allocation est `PerRender`, un littéral est lui-même, le reste (le paramètre compris) `⊥`.
3. **Valeur** : évaluée par `value_of` (c'est-à-dire `SiteEnvs::eval`) puis passée à `value_freshness`.

Par `return (classify_updater_return, src/engine/written.rs, l. 153–171)` : une allocation (`ObjectLit`, `ArrayLit`, `FnLit`, `New`) est `Fresh` ; l'identité `o => o` et un littéral sont `Not` ; un opérateur binaire retourne un primitif *sauf* les opérateurs logiques, qui retournent un opérande, d'où `l.max(r).min(Freshness::Maybe)` : « jamais must-frais, au plus `Maybe` » ; tout le reste est `Maybe`. `Expr::New` compte comme allocation depuis #158 (commit e67b10a).

Extrait 14.28 – `src/engine/written.rs`, l. 185–200 — fraîcheur d'une valeur stockée

```
185 fn value_freshness(val: &StateValue, target: QualifiedSlot) -> Freshness {
186     match &val.reference {
187         Stability::PerRender => {
188             if val.is_unstable_reference_only() {
189                 Freshness::Fresh
190             } else {
191                 Freshness::Maybe // joined with other kinds
192             }
193         }
194         Stability::Unknown => Freshness::Maybe,
195         Stability::Versioned(by) if by.len() == 1 && by.contains(&target) => Freshness::Not,
196         // Stable / Versioned / ⊥ reference; residual ⊤ stays Maybe.
197         _ if val.other => Freshness::Maybe,
198         _ => Freshness::Not,
199     }
200 }
```

Le churn ne regarde *que la sorte référence* de la valeur (chapitre 10) : `count + 1` change mais ne rate jamais `Object.is` « fraîchement ». `PerRender` seul (`is_unstable_reference_only`) est `Fresh` ; `PerRender` joint à d'autres sortes, `Unknown` et un résidu `⊤` sont `Maybe` ; `Versioned(cible)` est `Not`, parce qu'un contenu du slot réécrit dans le slot n'est pas un changement (#155). Une valeur versionnée par un *autre* slot avec un résidu `⊤` reste `Maybe`, et #157 est « le compte rendu précis » de cette limite : une valeur dérivée du slot écrit par un `useMemo` lit `Versioned(s)`, donc `Not`, et une boucle réelle reste silencieuse (section 14.12).

Rappel React : `Object.is` et le bail-out

React abandonne une mise à jour d'état dont la nouvelle valeur est `Object.is`-égale à l'actuelle. Un objet littéral neuf, un tableau, une fonction, un `new Map()` échouent *toujours* cette comparaison et provoquent un re-rendu ; un primitif égal ou la même référence, jamais. C'est pourquoi `Freshness` ne porte que sur la sorte référence, et pourquoi la preuve relationnelle du section 14.10 peut ignorer `NaN`.

14.5.2 Où évaluer l'argument : `SiteEnvs`

`SiteEnvs` (`src/engine/written.rs`, l. 213–226) rassemble les stores convergés (`state`, `memo`, `heap`), l'environnement d'entrée et les sorties par bloc du rendu (`render_entry`, `render_exits`), l'environnement de sortie du rendu (`exit`, qui est aussi l'entrée de tout corps de `hook`) et les sorties par bloc des effets et des handlers (`effect_exits`, `handler_exits`). Sa méthode `env_at` choisit l'environnement d'un site :

Extrait 14.29 – `src/engine/written.rs`, l. 248–274 — l'environnement d'un site

```
248 fn env_at(
249     &self,
250     region: WriterRegion,
251     cfg: &CFG,
252     at: Option<(BlockId, usize)>,
253     ac: &mut AnalysisCtx<'_, StateValue>,
254 ) -> AbstractEnv<StateValue> {
```

```

255     let (entry, exits) = match region {
256         WriterRegion::Render => (self.render_entry, Some(self.render_exits)),
257         WriterRegion::Effect(l) => (self.exit, self.effect_exits.get(&l)),
258         WriterRegion::Handler(l) => (self.exit, self.handler_exits.get(&l)),
259         // Memo and callback bodies keep no per-block envs.
260         WriterRegion::Memo(_) | WriterRegion::Callback(_) => (self.exit, None),
261     };
262     let (Some((block, stmt)), Some(exits)) = (at, exits) else {
263         return exits
264             .and_then(|ex| region_exit(cfg, ex))
265             .unwrap_or_else(|| self.exit.clone());
266     };
267     let mut env = entry_env_of(cfg, block, entry, exits);
268     if let Some(b) = cfg.blocks.get(&block) {
269         for s in b.stmts.iter().take(stmt) {
270             StateValueTransfer.exec_stmt(s, &mut env, ac);
271         }
272     }
273     env
274 }

```

Avec une position `at = (bloc, index)`, l'argument est évalué dans l'environnement d'entrée du bloc (`entry_env_of` : jointure des sorties des prédécesseurs filtrées par arête, chapitre 12) rejoué à travers les instructions *avant* l'appel. Le test `the_argument_is_read_in_the_env_before_the_call_not_after` (`tests/writer_columns.rs`) épingle la nuance : `let v = x; setX(v); v = a; n; n` n'est pas Fresh. Sans position (site imbriqué, différé, répété), on prend l'environnement de sortie de la région (jointure des blocs `Return`, `region_exit`), qui sur-approxime toute liaison capturée ; à défaut, la sortie du rendu. Les corps de memo et de callback ne gardent pas d'environnements par bloc.

Deux remarques. `eval` (`src/engine/written.rs`, l. 233–246) clone les trois stores à chaque appel : la relation ne perturbe jamais le résultat convergé, au prix de trois clones par ligne évaluée par valeur, coût non mesuré dans le dépôt. Et le gain de précision annoncé par ADR-42 §2 est réel : l'ancien collecteur des règles évaluait tout dans l'environnement de sortie du rendu, où un `const next = ... local` à l'effet est inconnu et lit `Maybe` ; le moteur a l'environnement de bloc (test `an_effect_local_allocation_is_a_fresh_write`). Ce que l'ADR annonce pour un site imbriqué (« `T` for a nested class with no env ») et pour value (« the reference part ») est en revanche dépassé : le code stocke la valeur entière et évalue dans l'environnement de sortie de la région ; c'est le churn qui projette sur la partie référence, et seulement pour la *propre* écriture d'un site (`reference_part`, `src/engine/written.rs`, l. 207–209).

14.6 La relation `effect_triggers` : identité, pas dépendance

Un effet se ré-exécute quand l'un de ses deps change au sens de `Object.is`. Pour savoir si une écriture dans un slot *relance* un effet, il faut donc savoir si un dep de l'effet *est* ce slot, ou seulement en dépend. C'est la relation `effect_triggers` (`src/engine/triggers.rs`).

Extrait 14.30 – `src/engine/triggers.rs`, l. 33–44 — une ligne de `effect_triggers`

```

33 /// One dep of an effect that a state slot moves.
34 #[derive(Debug, Clone, PartialEq, Eq)]
35 pub struct EffectTrigger {
36     pub hook: HookLabel,
37     /// Index of the dep in the effect's deps list.
38     pub dep: usize,
39     pub slot: QualifiedSlot,
40     /// The dep IS the slot value (`StateVal(1)` or an alias of it): the effect
41     /// must re-run whenever a fresh value is stored. `false`: the dep is

```

```

42    /// versioned by the slot, the effect may re-run.
43    pub exact: bool,
44  }

```

Le calcul, `collect_effect_triggers` (`src/engine/triggers.rs`, l. 50–102), examine chaque `dep` de chaque `HookEntry::Effect`. Une ligne par (effet, index de `dep`, slot qualifié) ; aucune pour un effet sans liste de `deps` ni pour `[]`. Un `dep` qui est `StateVal(1)` ou un alias résolu donne une ligne `exact = true` : l'effet *doit* se ré-exécuter quand une valeur fraîche est stockée dans le slot (`must`). Sinon, la valeur convergée du `dep` (lue dans le memo store pour un memo ou un callback, parce que sa valeur d'environnement est liée avant le recalcul du store et lirait un `T` périmé) est examinée : si sa référence est `Versioned(labels)`, une ligne `exact = false` par label (`may`). Un slot d'un *parent* peut apparaître, quand le `dep` est une prop que le parent a calculée depuis son propre état (test `a_prop_dep_is_versioned_by_the_parent_slot_that_feeds_it`, `tests/effect_triggers.rs`).

Colonne	Affirmation	Polarité
hook	l'effet	exact
dep	indice dans la liste des <code>deps</code>	exact
slot	(composant, label) ; le slot d'un parent quand le <code>dep</code> est une prop qu'il a calculée	exact
exact	<code>true</code> : le <code>dep</code> est la valeur du slot, une valeur fraîche relance l'effet ; <code>false</code> : le <code>dep</code> est versionné par le slot, l'effet <i>peut</i> se relancer	must si <code>true</code> , may si <code>false</code>

TABLE 14.6 – Les colonnes de `effect_triggers` (`docs/relations.md`).

Exemple 14.7: Deux triggers exacts, deux arêtes must

Extrait 14.31 – `/tmp/reprobe/ex4.tsx`

```

1 import { useState, useEffect } from 'react';
2 export function C() {
3   const [a, setA] = useState({ n: 0 });
4   const [b, setB] = useState({ n: 0 });
5   useEffect(() => { setB({ from: a.n }); }, [a]);
6   useEffect(() => { setA({ from: b.n }); }, [b]);
7   return <div>{a.n + b.n}</div>;
8 }

-- slot_writers
slot=0 setter=setA line=Some(6) region=Effect(3) phase=Effect via=Direct block=Some(0) guard_block=Some(0) fresh=
  ↳ Fresh value.ref=PerRender
slot=1 setter=setB line=Some(5) region=Effect(2) phase=Effect via=Direct block=Some(0) guard_block=Some(0) fresh=
  ↳ Fresh value.ref=PerRender
-- effect_triggers
hook=2 dep=0 slot=(C, 0) exact=true
hook=3 dep=0 slot=(C, 1) exact=true
=== churn graph
edge#0 (C, 0) -> (C, 1) strength=Must carrier=C effect=2 line=Some(5)
edge#1 (C, 1) -> (C, 0) strength=Must carrier=C effect=3 line=Some(6)
cycle edges=[0, 1] all_must=true cross_component=false

```

Les deux `deps` sont les slots (`exact = true`) ; les deux écritures stockent un littéral objet (`fresh = Fresh`) dans un bloc qui domine la sortie de son effet. Le graphe de churn (chapitre 15) en tire deux arêtes `Must` et un cycle `all_must` d'un seul composant : `must_effect_cycle` frappe le jeton et `infinite-loop` émet une `ERROR` pour chaque effet (extrait de la sortie `-trace`) :


```
error infinite-loop [hook:2] (line 5:2) these effects form a state-update cycle (`a` → `b` → `a`) where
  ↳ each step stores a fresh reference that re-runs the next effect: infinite render loop
  ↳ a fresh value is written to state `b` here [hook:2] (line 5:20)
  ↳ cycle continues: this effect freshly stores state `a` [hook:3] (line 6:20)
```

La seconde ERROR, ancrée sur [hook:3], est symétrique. Les deux WARNING `derived-state` que la ligne de commande ajoute relèvent d’une autre règle (chapitre 18). Sur l’exemple 14.2, au contraire, la relation est vide : le seul dep est une prop, et ce composant est une racine (props $\top$).

Décision : ADR-42 §3, identité et non dépendance

La relation n’est *pas* unifiée avec `render_deps::Deps` (chapitre 16). Celle-ci répond à la *dépendance* (count + 1 découle de count); celle-là à l’*identité* (le dep *est* count), et c’est l’identité que réclame une affirmation `must-rerun`. Les deux restent séparées. La sémantique est celle de l’ancien `classify_effect_deps` des règles, inchangée, mais calculée une fois. Notons qu’ADR-42 §3 annonce « one row per (hook, qualified slot) » : le code et docs/relations.md ont une ligne par (effet, index de dep, slot).

14.7 La relation `slot_seeds`

`useState(props.value)` lit son initialiseur au premier rendu seulement. Si la prop change ensuite, le slot garde la valeur de montage : c’est le défaut que `frozen-initial-state` (chapitre 18) signale. La relation `slot_seeds` (src/engine/seeds.rs) répond à deux questions par slot : *de quels chemins de props l’initialiseur lit-il ?* et *quelque chose re-synchronise-t-il visiblement le slot quand la prop bouge ?*

Exemple 14.8: NoneSeen contre Synced

Extrait 14.32 – /tmp/relprobe/ex3.tsx

```
1 import { useState, useEffect } from 'react';
2 export function Mirror({ value, other }: { value: string; other: string }) {
3   const [local, setLocal] = useState(value);
4   const [label, setLabel] = useState(other);
5   useEffect(() => { setLabel(other); }, [other]);
6   return <div>{local}{label}</div>;
7 }
```

```
-- slot_writers
slot=1 setter=setLabel line=Some(5) region=Effect(2) phase=Effect block=Some(0) fresh=Maybe
-- slot_seeds
slot=0 path=value normalized=["__p0.value"] sync=NoneSeen setter_escapes=false
slot=1 path=other normalized=["__p0.other"] sync=Synced setter_escapes=false
```

value est lu par l’initialiseur de local à travers le préambule de destructuration (let value = __p0.value), normalisé exactement en __p0.value; aucune écriture ne le re-synchronise : NoneSeen. label a une ligne d’effet de phase Effect dont les deps [other] couvrent exactement __p0.other : Synced. La ligne de commande (-info) affiche info frozen-initial-state [hook:0] var:value (line 3:8) state ‘local’ is seeded from ‘value’ and never re-synced. INFO et non WARNING : ce n’est pas la relation qui décide de la sévérité mais les strates de la règle native (ADR-31 §5); ici setLocal n’est jamais référencé, et la règle lit un instantané délibéré au montage (src/rules/impls/frozen_initial_state.rs, l. 241–247).

Extrait 14.33 – src/engine/seeds.rs, l. 35–78 — SeedSync et la ligne SlotSeed

```

35 ///
36 /// **MAY-typed in one direction.** `Synced` is a claim the relation makes from
37 /// a write it saw; `NoneSeen` is the absence of one, which is not a proof that
38 /// no sync exists – a setter the component handed out could be called from
39 /// anywhere (see [ `SlotSeed::setter_escapes` ]).
40 #[derive(Debug, Clone, Copy, PartialEq, Eq)]
41 pub enum SeedSync {
42     /// A render-time write, or an effect write that re-runs when this seed's
43     /// prop moves (its deps cover the seed path, or it declares none).
44     Synced,
45     /// No such write was seen.
46     NoneSeen,
47 }
48
49 /// One `(state slot, prop path its initializer reads)` row.
50 #[derive(Debug, Clone)]
51 pub struct SlotSeed {
52     pub slot: HookLabel,
53     /// The path as written at the seed site – what a message shows, and what
54     /// an env evaluation is run against. Exact.
55     pub path: AccessPath,
56     /// Props-param-rooted forms of [ `Self::path` ]: [ `value` ], [ `props.value` ]
57     /// and [ `props` ] must all cover a seed read as `value`. Non-empty by
58     /// construction – a path that does not normalize to the props param is not
59     /// a seed and produces no row.
60     ///
61     /// A may-set. Where the chase could not select through a binding it
62     /// widened to every path that binding reads, so a form here is one the
63     /// seed may denote (ADR-033 §3). The deps-coverage test is a must-claim
64     /// and therefore runs on the exact forms only, inside the relation – this
65     /// column is for consumers that want the possibilities.
66     pub normalized: Vec<AccessPath>,
67     pub sync: SeedSync,
68     /// An alias of this slot's setter is used somewhere other than a direct
69     /// call or a pure alias binding – passed as a prop, stored in an object,
70     /// handed to an opaque call.
71     ///
72     /// A separate column, not folded into [ `SeedSync` ], because it answers
73     /// a different question: not "is there a sync" but "could there be one we
74     /// cannot see". Folding it into `Synced` would erase the distinction the
75     /// native rule's Error tier is built on – a no-sync claim is certain only
76     /// when the setter stayed home.
77     pub setter_escapes: bool,
78 }

```

Colonne	Affirmation	Polarité
slot	le slot ensemencé	exact
path	le chemin tel qu'il est écrit au site de la graine	exact
normalized	les formes enracinées au paramètre props que la graine <i>peut</i> désigner	may-ensemble
sync	Synced : une écriture au rendu, ou d'effet relancé quand la prop bouge, a été vue ; NoneSeen : aucune	Synced may ; NoneSeen est une absence
setter_escapes	un alias du setter est allé là où la marche ne suit pas	may

TABLE 14.7 – Les colonnes de slot_seeds (docs/relations.md).

Trois colonnes portent une polarité explicite (tableau 14.7). `normalized` est un *may-ensemble* : là où la poursuite n'a pas pu sélectionner à travers une liaison, elle s'est élargie à tous les chemins lus, et une forme présente est une forme que la graine *peut* désigner. `sync` est *may* dans une seule direction : `Synced` est une affirmation faite depuis une écriture vue, `NoneSeen` une absence. `setter_escapes` est une colonne *séparée*, non repliée dans `sync`, parce qu'elle répond à une autre question : non pas « y a-t-il une `sync` » mais « pourrait-il y en avoir une que nous ne voyons pas » ; c'est la porte `!escaped` de `must_frozen_seed` (`src/rules/api/query.rs`, l. 904–916).

14.7.1 Le pli

Extrait 14.34 – `src/engine/seeds.rs`, l. 108–170 — les deux kills de `slot` et la couverture par graine

```

108
109     // The two slot-level kills, read off the writer relation: a
110     // render-time write (the adjust-during-render pattern – a sync path
111     // exists, its misuse is `setter-in-render`'s business), and an effect
112     // with no readable deps list, which re-runs after every render.
113     let rows = || {
114         slot_writers
115             .iter()
116             .filter(|w| w.owner.is_none() && w.slot == *label)
117     };
118     // **Phase, not region.** `region` is lexical, and a callback literal
119     // written inline in render lives in the render CFG – `useCallback(() =>
120     // setValue(x), ...)` would read as an adjust-during-render write and
121     // suppress the finding. Only `WriterPhase::Render` says the write
122     // actually runs during render; a nested write is T, and suppressing on
123     // T is the false negative this project does not trade.
124     let render_write = rows().any(|w| w.phase == WriterPhase::Render);
125     // The effect half asks "does this effect re-run and write the slot",
126     // so it takes the lexical region – but only for rows whose PHASE says
127     // the effect is what runs the write (#121). ADR-031 §2 kept this half
128     // purely lexical on the argument that a write nested in a `.then` the
129     // effect kicks off still counts, because the effect re-running is what
130     // re-runs it. That argument covers `Deferred` and `Cleanup`, both of
131     // which the effect schedules. It does not cover a callback the effect
132     // merely *hands* to an opaque callee – `manager.subscribe(setColor)`
133     // is a T-phase row, nothing establishes that the callee ever calls it,
134     // and suppressing on T is the false negative §3 already refuses for
135     // the render half. `Handler` is refused for the same reason a handler
136     // does not close a churn cycle: it needs an external event.
137     let effects: Vec<HookLabel> = rows()
138         .filter(|w| effect_triggered(w.phase))
139         .filter_map(|w| match w.region {
140             WriterRegion::Effect(l) => Some(l),
141             _ => None,
142         })
143         .collect();
144     let unconditional_effect = effects.iter().any(|l| {
145         effect_info
146             .get(l)
147             .is_some_and(|i| matches!(i.deps, DepsArg::Absent))
148     });
149
150     let escapes = setter_escapes(render_cfg, hooks, &aliases);
151
152     for seed in seeds {
153         // Per-seed: an effect whose declared deps cover THIS seed's path

```

```

154      // re-runs when this prop moves. A `DepsArg::Opaque` list gates the
155      // effect by something the engine cannot read, so it proves no sync
156      // and must not suppress one; a flattened spread declares its
157      // elements, not its source, so `covering()` is what may be
158      // credited.
159      let covered = effects.iter().any(|l| {
160          effect_info.get(l).is_some_and(|i| match &i.deps {
161              DepsArg::Absent => true,
162              DepsArg::Opaque => false,
163              DepsArg::List(d) => deps_cover_seed(&d.covering(), &seed, param, &bindings),
164          })
165      });
166      let sync = if render_write || unconditional_effect || covered {
167          SeedSync::Synced
168      } else {
169          SeedSync::NoneSeen
170      };

```

Le pli est *syntaxique à dessein* (ADR-20 item 3, doc de module) : il ne lit aucune valeur abstraite, il répond « une écriture se ré-exécute-t-elle quand cette prop bouge », non « produit-elle la bonne valeur » (question de *derived-state*). Il se déroule ainsi :

1. pour chaque `HookEntry::State`, `seed_paths(init, param, bindings)` donne les chemins lus par l'initialiseur, normalisés vers le paramètre `props` (`__p0`) ; un chemin qui ne se normalise pas n'est pas une graine ;
2. deux kills au niveau du slot, lus sur les lignes *locales* de `slot_writers` : une écriture de **phase** `Render`, ou un effet *sans deps* (`DepsArg::Absent`) qui écrit le slot ;
3. la moitié effet : région `Effect(1)` et phase dans `effect_triggered` (`src/engine/seeds.rs`, l. 192–197 : `Effect`, `Deferred`, `Cleanup`) ; un callback simplement *confié* à un callee opaque (`manager.subscribe(setColor)`, phase `T`) ne prouve pas de sync (`#121`) ;
4. par graine, la couverture par les deps déclarés (`deps_cover_seed`, `src/engine/seeds.rs`, l. 356–383) : un test syntaxique direct du chemin tel qu'écrit, puis la comparaison des formes normalisées, en ne créditant que les normalisations *exactes* des deux côtés ; `DepsArg::Opaque` ne couvre rien, `DepsArg::Absent` couvre tout ;
5. `setter_escapes` (`src/engine/setters.rs`, l. 2271–2334) : un alias du setter apparaît ailleurs qu'en position de callee direct ou d'alias pur (argument d'appel, champ d'objet, prop JSX, capture, écriture de membre).

Décision : ADR-31 §3 et ADR-33, phase et exactitude

Le kill de rendu lit la **phase**, pas la région : un callback littéral écrit au rendu (`useCallback(() => setValue(x), ...)`) est de région `Render` mais de phase `T`, et supprimer un finding sur `T` est « the false negative this project does not trade ». La première migration lisait la région et a fait perdre un WARNING réel (`use-provider-color-scheme` de mantine) ; la décision ne repose pas sur cette mesure, faite sous le flottement de `#120`, mais sur le principe. ADR-33 ajoute deux choses à la poursuite des liaisons : le garde-cycle *seen* est *cloné par branche* (le partager tuait toutes les branches sœurs sauf la première, dans l'ordre d'un `HashSet`) ; un littéral objet est *sélectionné* (`a: x .a` est `x`, exact) et non élargi ; et le bit exact interdit qu'une forme élargie soutienne l'affirmation *must* de couverture. Sans §3, restaurer les branches sœurs seules aurait élargi le côté déclaré et supprimé davantage : la direction interdite.

Exemple 14.9: Sélection à travers un littéral objet, setter échappé

Sur `/tmp/verif07/e14.tsx` (`const cfg = x: props.a ; useState(cfg.x); useState(props.b ?? 'd');` ; `useState(props.c)` dont le setter est passé en prop à Child) :

```
slot=0 path=cfg.x    normalized=["props.a"] sync=Synced  setter_escapes=false
slot=2 path=props.c  normalized=["props.c"] sync=NoneSeen setter_escapes=true
```

`cfg.x` est poursuivi jusqu'à `x: props.a`, sélectionné et exact ; les deps `[props.a]` couvrent : Synced. `s3` n'a aucune écriture vue et son setter s'échappe : `NoneSeen`, `setter_escapes = true`, et la règle native reste à `WARNING` (`warn frozen-initial-state [hook:2] var:props.c (line 6:8)`). Le slot 1 (`props.b ?? 'd'`) n'a *aucune ligne*. Le lowering traite `??` comme `||` (`src/lowering/expr_lower.rs, l. 754`) : Let `__tN = props.b`, un Branch sur `__tN`, puis Assign `__tN = 'd'` dans la branche fausse. L'initialiseur lit donc `__tN`, un nom à *deux* membres droits dans `local_bindings` (`src/ir/bindings.rs, l. 37–47`), et `normalize_to_prop` refuse de le poursuivre (« a multi-write binding is uncertain and not chased »).

Remarque

Ce refus contredit le commentaire de la même fonction, qui promet le cas `const v = props.a ?? d` (`src/engine/seeds.rs, l. 247–248`). Sur `/tmp/ch14/nullish.tsx`, trois enfants reçoivent une prop tirée d'un état du parent : `useState(props.b)` donne une ligne `NoneSeen` et `warn frozen-initial-state [hook:0] var:props.b (line 12:8); useState(props.b ?? 'd')` et `const v = props.a ?? 'd'; useState(v)` ne donnent ni ligne ni diagnostic. Au commit `e67b10a`, c'est un faux négatif de niveau `WARNING` qui ne figure pas dans le tracker (l'issue #25 liste d'autres résidus de `frozen-initial-state`).

14.8 La relation registrations et l'appariement

Un corps d'effet qui appelle `setInterval`, `addEventListener` ou `store.subscribe` confie un callback à quelque chose qui survit à l'effet. Deux règles natives (chapitre 20) posent des questions sur ces appels : `stale-closure` (« que capture le callback ? ») et `missing-cleanup` (« l'enregistrement est-il un jour défait ? »). La relation `registrations` (`src/engine/registrations.rs`) est leur lecture commune, calculée une fois sur les *corps d'effets seulement* (doc de module, `src/engine/registrations.rs, l. 15–17`).

Exemple 14.10: Quatre enregistrements, trois formes de teardown

Extrait 14.35 – `/tmp/ch14/regs.tsx`

```
1 import { useState, useEffect } from 'react';
2 declare const store: { subscribe(cb: () => void): () => void };
3 export function Regs() {
4   const [n, setN] = useState(0);
5   useEffect(() => {
6     const id = setInterval(() => setN(v => v + 1), 1000);
7     const u = store.subscribe(() => setN(0));
8     const h = () => setN(1);
9     window.addEventListener('resize', h);
10    window.addEventListener('scroll', () => setN(2), { once: true });
11    return () => {
12      clearInterval(id);
13      u();
14      window.removeEventListener('resize', () => setN(1));
15    };
16  }, []);
```

```

17   return <div>{n}</div>;
18 }

-- slot_writers
slot=0 setter=setN line=Some(9) region=Effect(1) phase=Handler updater=Unknown block=None guard_block=Some
  ↳ (0)
slot=0 setter=setN line=Some(6) region=Effect(1) phase=Deferred updater=Functional block=None guard_block=Some
  ↳ (0)
slot=0 setter=setN line=Some(7) region=Effect(1) phase=Unknown updater=Unknown block=None guard_block=Some
  ↳ (0)
slot=0 setter=setN line=None region=Handler(2) phase=Handler updater=Unknown block=Some(0) guard_block=Some
  ↳ (0)
-- registrations
effect=1 display=setInterval registrar=setInterval firing=Repeating timing=Deferred handle=Some("
  ↳ id") self_removing=false pairing=Paired event=None
effect=1 display=store.subscribe registrar=subscribe firing=Repeating timing=Unknown handle=Some("
  ↳ u") self_removing=false pairing=Paired event=None
effect=1 display=window.addEventListener registrar=addEventListener firing=Repeating timing=Handler handle=None
  ↳ self_removing=false pairing=Unpaired event=Some("resize")
effect=1 display=window.addEventListener registrar=addEventListener firing=Repeating timing=Handler handle=None
  ↳ self_removing=true pairing=Paired event=Some("scroll")

```

Côté écritures : le callback de subscribe est de phase Unknown (contrat inconnu) ; le listener h passé par nom (bras B5) est Handler ; le listener littéral de 'scroll', de premier niveau, a été réifié en Handler(2) ; et le setN(1) du cleanup ne produit *aucune* ligne, parce qu'il est l'argument d'un teardown, jamais descendu. Côté enregistrements : setInterval est Paired par le handle id ; subscribe par le disposer u() ; 'scroll' par once: true ; et 'resize' est Unpaired, parce que le cleanup retire un *autre* listener que h. La ligne de commande n'affiche pourtant rien pour ce fichier : missing-cleanup ne lit pas pairing, elle ne tire que sur un effet qui ne retourne *aucun* cleanup (cleanup_verdict(body_cfg) != CleanupVerdict::Absent ⇒ continue, src/rules/impls/missing_cleanup.rs, l. 63–65) ; la colonne pairing est le sujet de l'ancre Tier-A registrations (ADR-34 §7, chapitre 22).

La ligne Registration (src/engine/registrations.rs, l. 239–269) est décrite colonne par colonne dans le tableau 14.8. registrar est la clé stable de la table ; callback est l'expression confiée (un clone de FnLit ne copie qu'un Arc) ; handle est la liaison du *retour* de l'appel, celle que clearInterval(id) ou le disposer u() reprendront ; event est le nom littéral d'événement d'un addEventListener.

Colonne	Affirmation	Polarité
effect, span, display, registrar, callback, handle, event firing	l'appel tel qu'il est écrit et la ligne de table qu'il a appariée	exact
timing	le registrar appelle le callback Once ou Repeating	exact (axiome de la table)
self_removing	le callback tourne Deferred, en Handler, ou Unknown	may, T-bearing
block_id	l'enregistrement se retire lui-même après un dispatch	exact
pairing	bloc de premier niveau du corps d'effet ; None si imbriqué	exact
	Paired : le cleanup défait cet enregistrement ; Unpaired : prouvablement non ; Unknown	must pour Paired ; Unpaired affirmé sur les seules formes reconnues ; T-bearing

TABLE 14.8 – Les colonnes de registrations (docs/relations.md).

Deux détails de `collect_registrations` (`src/engine/registrations.rs`, l. 322–346) : la table des helpers d'un effet est complétée par les liaisons de fonctions du *rendu* (un helper défini au rendu et appelé dans l'effet est balayé, B6) ; et le `handle` est lié pour un `Let` ou un `Assign` au-dessus de l'appel (`src/engine/registrations.rs`, l. 606). Le balayage (`scan_cfg`, `src/engine/registrations.rs`, l. 589–683) va à profondeur 2 ; un callback `FnLit` est balayé avec `block_id = None` (« never must-reached ») ; l'ordre des lignes suit `cfg.blocks`, donc est déterministe.

14.8.1 L'appariement, un fait à trois valeurs

Extrait 14.36 – `src/engine/registrations.rs`, l. 218–236 — Pairing : seul `Paired` est une affirmation

```
218 #[derive(Debug, Clone, Copy, PartialEq, Eq)]
219 pub enum Pairing {
220     /// A cleanup this walk could read calls one of the registrar's teardown
221     /// names holding the same listener binding. The only value that is a claim.
222     Paired,
223     /// A readable cleanup with no such call, or no cleanup at all.
224     Unpaired,
225     /// A cleanup the walk cannot read, or a listener that is not a resolvable
226     /// binding – the teardown may be there and may not.
227     Unknown,
228 }
229
230 impl Pairing {
231     /// `true` unless the teardown is proven. What a rule that fires on a
232     /// missing teardown must ask: `Unknown` is a may-fact on the firing side.
233     pub fn may_be_unpaired(self) -> bool {
234         self != Pairing::Paired
235     }
236 }
```

`cleanup_bodies` (`src/engine/registrations.rs`, l. 360–385) lit les `Return` du corps d'effet : `return () => ...` ou `return nom` lié à un littéral unique (`fn_binding_in`) donnent des corps lisibles ; aucun retour non-undefined donne `None` ; un *seul* retour illisible rend le tout `Opaque`, même si d'autres sont lisibles (`(_, true) => Cleanups::Opaque`).

L'appariement lui-même, `pair` (`src/engine/registrations.rs`, l. 393–455), est un arbre de décision que la figure 14.5 reproduit.

Décision : ADR-34 §3 et §5, la liaison est le fait, le nom ne l'est pas

Apparier sur le seul nom de `teardown` certifierait exactement le bug que `subscribe-with-fresh-listener` existe pour attraper : un `cleanup` qui retire un *autre* listener. Le listener doit être la même *liaison*. L'amendement #124 a ajouté deux formes, après trois faux positifs mesurés sur le corpus dans les heures qui ont suivi la livraison : la forme *handle* (`clearInterval(id)` prend ce que l'enregistrement a *retourné*, une comparaison de listener ne peut jamais réussir) et la forme *disposer* (`const u = s.subscribe(f); return () => u()`), disponible pour tout registrar qui retourne quelque chose. `Paired` étant le verdict qui *supprime*, la forme `disposer` est un ensemble fermé de noms de méthodes (`unsubscribe`, `dispose`, `cancel`, `close`, `destroy`, `remove`, `off`, `abort`) : `s.emit('bye')` n'est pas un `teardown`. Le troisième argument booléen d'`addEventListener` est `capture`, pas `once`, et ne compte pas (`src/engine/registrations.rs`, l. 571–580). Enfin (§5), un `teardown` n'est jamais descendu par la marche : le descendre donnait à chaque listener enregistré une *seconde* ligne `⊥`, produite par le `cleanup` même qui le retire, et une ligne `⊥` satisfait `writer_phases includes`

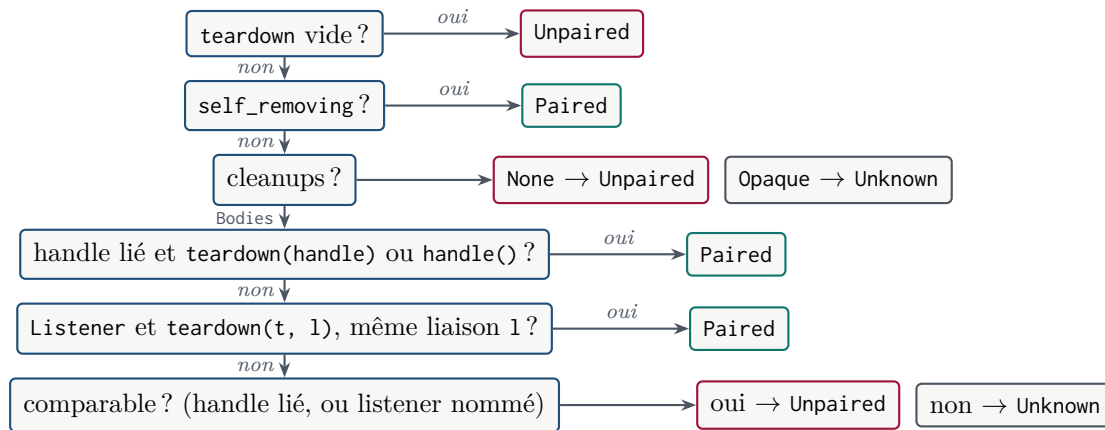


FIGURE 14.5 – L’arbre de décision de `pair` (`src/engine/registrations.rs`, l. 393–455). L’absence de cleanup est tranchée *avant* toute comparaison : « it is the one case where the absence is total ». Les disposers reconnus forment un ensemble fermé, `DISPOSERS` (`src/engine/registrations.rs`, l. 478–490).

<anything>.

14.9 Deux canaux de plus : `slot_reads` et `body_calls`

Deux relations de plus sortent de la même marche, sans être stockées : elles sont collectées à la demande par les règles ou les packs qui les lisent. La motivation est celle d’ADR-36 §1 : la marche répond déjà à la partie difficile (la classe d’un site, la greffe d’un helper ou d’une IIFE au site, la frontière `await`, le cleanup) ; « a second traversal would re-derive all of it and drift ».

Extrait 14.37 – `src/engine/setters.rs`, l. 2017–2051 — `BodyCall` et `SlotRead`

```

2017 /// One non-hook call site in a body (#126, ADR-036).
2018 ///
2019 /// The relation a pack reads through the `calls` edge. It is a may
2020 /// relation on two counts: the name is the callee as written and resolved to
2021 /// a binding, never a proof of which host primitive runs (the same footing as
2022 /// the registrar table, wontfix #42), and `phase` is the `WriterPhase` may
2023 /// verdict the writer rows already carry.
2024 #[derive(Debug, Clone)]
2025 pub struct BodyCall {
2026     /// The function name, or the method name of a member call.
2027     pub name: String,
2028     /// The receiver's root binding for a member call; `None` for a bare call.
2029     pub receiver: Option<String>,
2030     /// Where the call runs, on the lattice the writer rows use. A call in the
2031     /// effect's returned function is `Cleanup`; one after an `await` or inside
2032     /// a `then` is `Deferred`; one under an unresolved callee is `Unknown`.
2033     pub phase: WriterPhase,
2034     pub span: Option<SourceRange>,
2035 }
2036
2037 /// One read site of a state slot (#127, ADR-037).
2038 ///
2039 /// The write-side mirror image of [SlotWriter], and deliberately the same
2040 /// two columns: `region` is the lexical body – exact – and `phase` is the MAY
2041 /// verdict the walk assigned, so a read inside a `then` continuation or an
2042 /// effect's cleanup is distinguishable from one in the render body.
  
```



```

2043 #[derive(Debug, Clone)]
2044 pub struct SlotRead {
2045     pub slot: HookLabel,
2046     /// The binding the read went through: the slot's own name, or an alias.
2047     pub name: Var,
2048     pub region: WriterRegion,
2049     pub phase: WriterPhase,
2050     pub span: Option<SourceRange>,
2051 }

```

Found porte trois vecteurs, `setters`, `calls` et `reads` (`src/engine/setters.rs`, l. 1366–1371), activés respectivement toujours, par `collect_calls` et par `read_vars` non vide. `Found::absorb` (`src/engine/setters.rs`, l. 1373–1408) sert les trois canaux par construction : une ligne qu’une marche interne (B6, IIFE) a classée Sync prend la classe du site, une ligne différée ou imbriquée garde la sienne.

Une ligne de `body_calls` est (`name`, `receiver`, `phase`, `span`) : le nom d’un appel nu ou la *méthode* d’un appel de membre, et la racine du receveur ; `socket.join` et n’importe quel `.join` sont deux questions différentes et une règle doit dire laquelle elle pose. Les valeurs d’arguments restent en dehors (ADR-36 §2). Une ligne de `slot_reads` est (`slot`, `name`, `region`, `phase`, `span`), le miroir de `SlotWriter` avec les deux colonnes qui *transfèrent* ; `setter` et `via` ne transfèrent pas, ce sont des faits de provenance d’une écriture (ADR-37 §1).

Invariant 14.3: Ce que voient les canaux

Une lecture ou un appel dans une fonction imbriquée n’est enregistré que quand la marche *entre* dans cette fonction, jamais en traversant un `FnLit` de l’extérieur (où la classe serait `T` et la ligne un doublon). Une closure que rien n’appelle ne contribue donc aucune ligne ; « nothing may read the absence of a row as a proof that the slot is unread » (`src/engine/setters.rs`, l. 2052–2061). Dans `slot_reads`, la phase fait partie de l’identité d’une ligne (ADR-37 §3) : un même corps atteint comme callback de timer et comme cleanup donne deux lignes, et déduplication sur (`slot`, `span`, `region`, `name`) perdait la seconde.

Une différence de réglage mérite d’être connue (tableau 14.3) : `collect_slot_reads` et `collect_body_calls` n’ont pas de table d’ombrage (`shadowed` vide), si bien qu’un `setTimeout` lié localement y est lu comme le timer de l’hôte ; et pas de wrappers prouvés (`NO_WRAPPERS`), si bien qu’un callback confié à `form.handleSubmit` y est `T` et non `Handler`, ce qui est la direction sûre. Aucune règle native ne lit ces deux canaux ; ce sont les ancres et arêtes Tier-A `reads` et `calls` (chapitre 22) qui les consomment, plafonnées structurellement à `WARNING` (ADR-36 §4).

14.10 La preuve mono-site : une écriture qui éteint ses gardes

14.10.1 Le problème : écrire pendant le rendu, une seule fois

Appeler un setter pendant le rendu est interdit en général : l’écriture planifie un re-rendu, qui réexécute l’écriture, qui planifie un re-rendu. React tolère une exception documentée, « adjusting state when a prop changes » : une écriture *sous une garde que l’écriture elle-même éteint*.

Exemple 14.11: Deux écritures au rendu, une seule fautive

Extrait 14.38 – `/tmp/ch14/guarded.tsx`

```

1 import { useState } from 'react';
2 export function Adjust({ value }: { value: number }) {

```

```

3  const [prev, setPrev] = useState(value);
4  const [n, setN] = useState(0);
5  if (prev !== value) {
6    setPrev(value);
7  }
8  setN(1);
9  return <div>{prev}{n}</div>;
10 }

```

```

slot=0 setter=setPrev line=Some(6) region=Render phase=Render block=Some(1) guard_block=Some(1) fresh=Maybe
slot=1 setter=setN line=Some(8) region=Render phase=Render block=Some(3) guard_block=Some(3) fresh=Not

```

```

Adjust (2 hooks) /tmp/ch14/guarded.tsx
error setter-in-render [hook:1] (line 8:2) setter `setN` called directly in the render body, move this call
↳ into a useEffect or an event handler
→ `setN` is a state setter, so calling it writes state (line 8:2)

```

setN(1) est synchrone, dans un bloc qui domine toutes les sorties : ERROR certifiée. setPrev(value) est silencieuse : après l'écriture, prev vaut value, la garde prev !== value est fausse au rendu suivant, le composant converge en un rendu supplémentaire. Cette seconde conclusion est une *preuve*, converges_once_written (src/engine/guards.rs).

Rappel React : ajuster l'état pendant le rendu

La documentation de React admet `if (prev !== value) setPrev(value); setSelection(null);` dans le corps d'un composant : React réexécute immédiatement le composant avec le nouvel état, avant de rendre les enfants, et la garde étant devenue fausse, l'écriture ne se reproduit pas. La condition de validité est exactement celle de la preuve : la garde doit *mourir* sous l'écriture. `if (prev !== value) setPrev(value + 1)` bouclerait.

14.10.2 Les gardes d'un site

Extrait 14.39 – src/engine/guards.rs, l. 655–669 — la chaîne à prédécesseur unique

```

655 /// The blocks that run on every pass `call_block` runs on: itself and the
656 /// single-predecessor chain above it, up to a join point or the entry.
657 pub fn guard_chain(cfg: &CFG, call_block: BlockId) -> Vec<BlockId> {
658     let mut chain = vec![call_block];
659     let mut cur = call_block;
660     while cur != cfg.entry {
661         let mut preds = cfg.edges.iter().filter(|e| e.to == cur);
662         let (Some(edge), None) = (preds.next(), preds.next()) else {
663             break; // join point or entry: stop collecting dominators
664         };
665         cur = edge.from;
666         chain.push(cur);
667     }
668     chain
669 }

```

Extrait 14.40 – src/engine/guards.rs, l. 671–699 — les conjoints d'un site, dépliés

```

671 /// The conjunctive facts a site runs under: the branch constraints on the
672 /// single-predecessor chain above `call_block`, expanded through the lowered
673 /// short-circuit temps. Empty for an unguarded site.
674 pub fn site_guardchain(cfg: &CFG, call_block: BlockId) -> Vec<(&Expr, bool)> {
675     let mut guards: Vec<(&Expr, bool)> = Vec::new();
676     for pair in guard_chain(cfg, call_block).windows(2) {

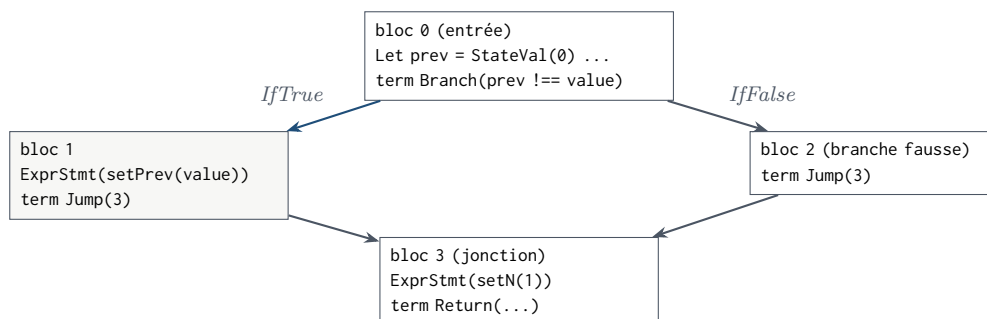
```

```

677     let (to, from) = (pair[0], pair[1]);
678     if let Some(edge) = cfg.edges.iter().find(|e| e.from == from && e.to == to)
679         && let Some(pb) = cfg.blocks.get(&from)
680         && let Terminator::Branch { cond, .. } = &pb.term
681     {
682         match edge.kind {
683             EdgeKind::IfTrue => guards.push((cond, true)),
684             EdgeKind::IfFalse => guards.push((cond, false)),
685             _ => {}
686         }
687     }
688 }
689
690 // Compound booleans (`a || b`, `a && b`) lower to a short-circuit temp
691 // (`__tN`) branched on directly – narrowing `__tN` alone proves nothing
692 // about the slot read inside an operand. Expand each guard into the
693 // conjunctive facts it implies over the operands.
694 let mut conjuncts: Vec<(&Expr, bool)> = Vec::new();
695 for (cond, taken) in guards {
696     expand_guard(cfg, cond, taken, 4, &mut conjuncts);
697 }
698 conjuncts
699 }

```

`guard_chain` remonte depuis le bloc de l'appel tant que le bloc courant a *un seul* prédécesseur, et s'arrête au premier point de jonction ou à l'entrée. C'est une sous-approximation des dominateurs (définition 3.9) : on collecte *moins* de gardes qu'il n'en domine réellement, donc on prouve moins de convergences, direction sûre. `site_guards` lit, pour chaque paire de la chaîne, la condition du Branch du prédécesseur et la polarité de l'arête (IfTrue $\rightarrow$ (cond, true), IfFalse $\rightarrow$ (cond, false)), puis déplie chaque garde par `expand_guard` (profondeur 4).



```

guard_chain(cfg, 1) = [1, 0], site_guards = [(prev != value, true)];
guard_chain(cfg, 3) = [3] : le bloc 3 a deux prédécesseurs, la chaîne s'arrête; site_guards = [],
la preuve répond false.

```

FIGURE 14.6 – Les chaînes de gardes des deux sites de l'exemple 14.11 (numéros de blocs de la sonde). Le site de `setN` n'a aucune garde : rien ne peut mourir, `converges_once_written` rend `false` et la règle passe à la dominance de sortie.

Le lowering abaisse `a || b` et `a && b` en un temporaire `__tN` branché directement (chapitre 7, diamant conservé par ADR-20 item 1) ; rétrécir `__tN` seul ne prouve rien sur le slot lu dans un opérande. `expand_guard` (`src/engine/guards.rs`, l. 953–1039) reconnaît le diamant (exactement un `Let` et un `Assign` du temporaire, le `Let` dans un bloc qui branche dessus, l'`Assign` dans un successeur direct) et déplie les deux polarités conjonctives : `t = a || b` pris FAUX $\Rightarrow$ `a` faux $\wedge$ `b` faux ; `t = a && b` pris VRAI $\Rightarrow$ `a` vrai $\wedge$ `b` vrai ; `!e` inverse ; `??` est abaissé comme `||`. Les polarités disjonctives passent telles quelles. Un `if (!b)` simple, lui, n'a pas de temporaire : sa

garde est (UnaryOp Not b, true), réécrite en (Var b, false).

14.10.3 Trois arguments de mort

Extrait 14.41 – src/engine/guards.rs, l. 714–780 — les trois bras, sur des gardes conjonctives

```

714 /// True when `guards`, the conjuncts of one site, are all dead once every
715 /// `rewrite` sits in its slot, starting from `env`.
716 ///
717 /// `shadowed` names what the site's body binds when `env` is not that
718 /// body's env – the render exit env under an effect body. Those names read
719 /// T: the render may bind the same name to something else (a lowered
720 /// short-circuit temp `__t0` most of all), and narrowing that value would
721 /// pronounce a guard dead on a name it does not test (#162).
722 fn dead_once_written(
723     guards: &[(&Expr, bool)],
724     shadowed: Option<&HashMap<&str, Option<&Expr>>>,
725     state_vals: &HashMap<Var, HookLabel>,
726     rewrites: &[Rewrite<'_>],
727     env: &AbstractEnv<StateValue>,
728     eval: &mut dyn FnMut(&Expr) -> StateValue,
729 ) -> bool {
730     let mut env = env.clone();
731     for name in shadowed.into_iter().flat_map(HashMap::keys) {
732         env.extend((name).to_string(), StateValue::top());
733     }
734     let slots_of = |label: HookLabel| -> HashSet<&Var> {
735         state_vals
736             .iter()
737             .filter(|(l, _)| **l == label)
738             .map(|(v, _)| v)
739             .collect()
740     };
741     for rewrite in rewrites {
742         for v in slots_of(rewrite.label) {
743             env.extend(v.clone(), rewrite.value.clone());
744         }
745     }
746
747     for &(cond, taken) in guards {
748         for rewrite in rewrites {
749             let slots = slots_of(rewrite.label);
750             // Relational arm: the guard compares the slot against an
751             // expression the write puts *into* the slot, so the two sides
752             // are the same value on the next render whatever that value is.
753             // An interval domain cannot say that – `x < y` after `x := y`
754             // needs the two to be related, not bounded – but the spellings
755             // can.
756             if let (Some(arg), Some(scope)) = (rewrite.expr, rewrite.scope)
757                 && write_settles_comparison(cond, taken, &slots, arg, scope, eval)
758             {
759                 return true;
760             }
761             // Member arm: the guard tests a *member* of the slot, so the
762             // value written at that member answers it – the whole-slot
763             // lookup below cannot, since the slot is one abstract value
764             // (#90).
765             if let Some(arg) = rewrite.expr
766                 && write_settles_member_truth(cond, taken, &slots, arg, eval)
767             {
768                 return true;

```

```

769     }
770   }
771   let narrowed = narrow_env_for_branch(&env, cond, taken);
772   if let Some(x) = guard_var(cond)
773     && narrowed.lookup(x).is_bottom_value()
774   {
775     return true;
776   }
777   env = narrowed;
778 }
779 false
780 }

```

Les gardes sont *conjonctives* : il suffit qu’une seule meure pour que le site ne puisse plus tirer, d’où le `return true` dès qu’un bras réussit. L’environnement de départ est celui du corps (ré-lié : chaque alias du slot reçoit la valeur écrite); `shadowed` n’intervient que pour la preuve multi-site sous l’environnement du rendu (#162), `converges_once_written` passe `None`.

1. **Le bras valeur.** On applique le narrowing de branche du moteur (`narrow_env_for_branch`, `src/engine/cfg_analyzer.rs`, l. 201–294) à l’environnement ré-lié; si la variable gardée devient $\perp$, la branche est morte. Sa portée est précise : il ne raffine que `Var(x)` (vérité), `!Var(x)`, et `Var(x) OP littéral` où le littéral est un nombre, `null` ou `undefined`. Toute autre forme laisse l’environnement inchangé, et ce bras ne prouve rien.
2. **Le bras relationnel** (`write_settles_comparison`, `src/engine/guards.rs`, l. 782–835) : la garde compare le slot à *l’expression même* que l’écriture y stocke; après l’écriture les deux côtés sont la même valeur, donc `<`, `>`, `!=`, `!==` sont faux et `==`, `===`, `<=`, `>=` vrais. « An interval domain cannot say that — $x < y$ after $x := y$ needs the two to be related, not bounded — but the spellings can. » L’égalité d’orthographe passe par `call_free_key`, qui refuse les appels (« a call may not return the same thing twice »), et le bras est refusé si l’un des côtés est une *orthographe fraîche* (`fresh_spelling`, `src/engine/guards.rs`, l. 837–842 : littéral objet, tableau, fonction, élément, `new`, nom lié à l’un d’eux, ou valeur convergée `PerRender`; #156, #158), puisqu’une allocation dénote une nouvelle référence à chaque exécution.
3. **Le bras membre** (`write_settles_member_truth`, `src/engine/guards.rs`, l. 908–937) : la garde teste un *membre* du slot, et l’écriture place à ce membre un *littéral* qui contredit la garde. Le slot étant une seule valeur abstraite, le narrowing sur le slot entier ne peut pas le voir (#90).

Exemple 14.12: Un exemple par bras

Valeur : `if (u === null) setU( id: 1 ) (/tmp/ch14/value_arm.tsx)`. L’écriture stocke `PerRender` non-`null`; `u` ré-lié, la garde `u === null` prise vraie rétrécit `u` à $\perp$. Aucun diagnostic. *Relationnel* : l’exemple 14.11, `prev !== value` contre `setPrev(value)`. *Membre* : `if (!id && sheet.leadId) setSheet( leadId: null, open: false ) (/tmp/ch14/member_arm.tsx)` : la garde `sheet.leadId` prise vraie est contredite par le littéral `null` placé à `leadId`. Aucun diagnostic non plus. Contre-exemple : `if (u === null && user) setU(user)` avec `user` une prop (`/tmp/ch14/nullguard.tsx`) : `user` est $\top$ et peut être `null`, la garde survit, et l’analyseur émet `warn setter-in-render [hook:0]` (line 5:4), un `WARNING` (conditionnel, pas de dominance) et non une `ERROR`.

14.10.4 La fonction et son appelant

Extrait 14.42 – `src/engine/guards.rs`, l. 60–89 — la preuve mono-site

```

60 /// True when the dominating guards of `call_block` provably kill the call

```

```

61 /// once `written` sits in state slot `label` – the set fires at most once.
62 ///
63 /// `exit_env` is the env the guards are narrowed from, and must be the env
64 /// of `cfg` itself (a render site); `eval` evaluates the literal the member
65 /// arm reads, and nothing else.
66 #[allow(clippy::too_many_arguments)]
67 pub fn converges_once_written(
68     cfg: &CFG,
69     call_block: BlockId,
70     state_vals: &HashMap<Var, HookLabel>,
71     label: HookLabel,
72     written: &StateValue,
73     written_expr: Option<&Expr>,
74     exit_env: &AbstractEnv<StateValue>,
75     eval: &mut dyn FnMut(&Expr) -> StateValue,
76 ) -> bool {
77     let guards = site_guards(cfg, call_block);
78     if guards.is_empty() {
79         return false;
80     }
81     let own = Rewrite {
82         label,
83         value: written.clone(),
84         expr: written_expr,
85         scope: Some(cfg),
86     };
87     // `exit_env` is this body's own env: nothing is shadowed.
88     dead_once_written(&guards, None, state_vals, &[own], exit_env, eval)
89 }

```

Pas de garde $\Rightarrow$ false. Sinon une seule Rewrite (la propre écriture du site, avec scope = Some(cfg) pour que le bras relationnel puisse lire les noms de l'argument dans le corps) et dead_once_written. Le seul appelant est setter-in-render (chapitre 18; src/rules/impls/setter_in_render.rs, l. 164–183) : la preuve n'est tentée que si aucune preuve d'ERROR n'a été obtenue, le site vient de collect_setter_calls (forme « une ligne par variable »), et la valeur écrite est évaluée par eval_in_exit_env dans l'environnement de sortie du rendu, non lue dans written.value.

Décision : ADR-42 §6, tranche 4 : une fonction, pas une colonne

La question « une écriture éteint-elle ses propres gardes ? » aurait pu devenir une colonne settles de SlotWriter. Elle reste une fonction sur les faits d'une ligne, parce que ses deux appelants lui donnent *deux valeurs différentes* : le graphe de churn la partie référence d'une écriture d'effet (une exécution qui stocke null ne stocke pas de référence fraîche), setter-in-render la valeur entière d'une écriture de rendu. Le chapitre 15 montre la forme forte de la même preuve, converges_under_all_writes, où les gardes doivent mourir sous *toutes* les écritures du slot qui s'exécutent dans la boucle.

14.11 ProgramRelations et la frontière de certification

14.11.1 Les relations programme-entier

Extrait 14.43 – src/engine/program_relations.rs — les relations programme-entier, paresseuses

```

1 /// Whole-program relations, computed once per program (ADR-042 §5).
2 ///
3 /// A per-component relation is computed at convergence and stored on
4 /// `AnalysisResult`. A relation that is a property of the *program* – the

```

```

5  /// churn graph reads every component's writes and triggers to find the cycles
6  /// a single component only participates in – has no such slice, and rebuilding
7  /// it inside every rule pass made the rules phase quadratic in component count
8  /// (#86). It lives here: one lazily-built entry per relation, bound to the
9  /// program it was built from, borrowed by every reader of that program.
10 ///
11 /// Lazy stays lazy: a rule that is disabled must not pay for a graph it never
12 /// reads. Adding a program-level relation means one more field here – never a
13 /// rebuild in a rule.
14
15 use std::sync::OnceLock;
16
17 use crate::engine::{ProgramAnalysisResult, churn::ChurnGraph};
18
19 /// Program-scoped, lazily-computed relations shared by every reader of one
20 /// [ `ProgramAnalysisResult` ].
21 pub struct ProgramRelations<'a> {
22     program: &'a ProgramAnalysisResult,
23     churn: OnceLock<ChurnGraph>,
24 }
25
26 impl<'a> ProgramRelations<'a> {
27     pub fn new(program: &'a ProgramAnalysisResult) -> Self {
28         ProgramRelations {
29             program,
30             churn: OnceLock::new(),
31         }
32     }
33
34     pub fn program(&self) -> &'a ProgramAnalysisResult {
35         self.program
36     }
37
38     /// The program's churn graph and its cycles, built on first request.
39     pub fn churn(&self) -> &ChurnGraph {
40         self.churn.get_or_init(|| ChurnGraph::build(self.program))
41     }
42 }

```

Une relation qui est une propriété du *programme* (le graphe de churn lit les écritures et les triggers de tous les composants pour trouver des cycles auxquels un composant seul ne fait que participer) n'a pas de tranche par composant. Elle vit ici : un champ `OnceLock` par relation, construit à la première demande, lié par sa durée de vie 'a au programme dont il est issu (impossible de lire le graphe contre un autre résultat). Deux invariants tiennent en une phrase chacun : « lazy stays lazy: a rule that is disabled must not pay for a graph it never reads », et ajouter une relation programme signifie un champ de plus ici, jamais une reconstruction dans une règle. Le graphe était autrefois reconstruit par composant, ce qui rendait la phase des règles quadratique en nombre de composants (#86 : dub et twenty ne finissaient pas) ; le test `churn_graph_is_built_once_per_program` (`src/rules/impls/infinite_loop.rs`, l. 630–684) le garde par un compteur `BUILDS`.

Côté règles, `ProgramCache` (`src/rules/api/cache.rs`, l. 26–31) *contient* un `ProgramRelations` et lui délègue `churn()`, à côté des trois structures programme-entier encore calculées dans `src/rules/helpers/` (`docs/relations.md`, section « Still built by the rules layer ») : l'index de montage (`mount.rs`), l'arbre des éléments (`render_tree.rs`, sur `render_deps`) et la relation `context_consumers` (`context_flow.rs`, ADR-32). Cette dernière est le troisième exemple de relation programme projetée : une post-passe sur les résultats convergés qui, pour chaque site `useContext`, dit si un composant susceptible de le rendre fournit la cellule lue ; son verdict est

une *absence* (« aucun provider sur aucun chemin »), et ADR-32 le protège par deux portes d'ascendance, parce qu'un composant que l'analyse n'a pas atteint est indistinguable d'une racine par `callers_of` seul (`src/rules/helpers/context_flow.rs`, l. 10–20). Elle est construite paresseusement, `OnceLock<ContextConsumers>` (`src/rules/api/cache.rs`, l. 29), exactement comme le graphe de churn, et n'attend qu'un lecteur moteur pour descendre d'une couche. `ProgramCache` est construit une fois par le driver. ADR-42 §5 annonçait que `ProgramRelations` *remplace* `ProgramCache` ; au commit e67b10a, il l'enveloppe.

14.11.2 La frontière de certification

Les relations ne produisent aucun diagnostic et n'attribuent aucune sévérité. Une `ERROR` n'existe qu'à travers un jeton `Certified<E>` frappé par une primitive `must_*` de `src/rules/api/query.rs` (définition 17.1), et chaque primitive *re-dérive* sa preuve depuis les colonnes `must` ou exactes plutôt que de faire confiance à un booléen. Le tableau 14.9 recense celles qui lisent ce chapitre.

Primitive	Ce qu'elle re-dérive	Lignes
<code>must_direct_write</code>	via <code>== Direct</code> sur une ligne <code>SlotWriter</code>	<code>src/rules/api/query.rs</code> , l. 763–774
<code>must_on_all_paths</code>	tout chemin entrée→sortie passe par un des blocs (<code>block</code>)	l. 627–640
<code>must_setter_on_all_paths</code>	un setter appelé sur tous les chemins (plus grand point fixe) ; re-balaye le CFG lui-même, sans lire <code>slot_writers</code>	l. 527–621
<code>must_effect_cycle</code>	toutes les arêtes <code>Must</code> et un seul composant	l. 1002–1031
<code>must_frozen_seed</code>	mouvement prouvé de la prop et <code>!escaped</code> (<code>setter_escapes</code>)	l. 904–916
<code>must_stale_capture</code>	une <code>Registration Repeating</code> , timing <code>!= Unknown</code> , sur tous les chemins	l. 942–989

TABLE 14.9 – Les primitives de certification qui consomment les relations de ce chapitre (`src/rules/api/query.rs`).

`must_effect_cycle` (`src/rules/api/query.rs`, l. 996–1031) est exemplaire : il reçoit les arêtes brutes et le cycle, recalcule `all_must` en lisant `strength` sur chaque arête et l'ensemble des composants impliqués (propriétaires des slots et porteurs des effets), et ne frappe le jeton que si tout est `Must` et qu'un seul composant apparaît ; « both facts are re-derived HERE from the raw edges ... not trusted from caller-supplied booleans ».

Deux nuances. `same_tick` et `body_calls` n'ont *aucun* chemin vers `ERROR` (ADR-28 §5, ADR-36 §4) ; les ancres Tier-A `registrations` et `seeds` sont plafonnées à `WARNING` ; mais les *règles natives* `stale-closure` et `frozen-initial-state` atteignent `ERROR` depuis ces deux relations, via `must_stale_capture` et `must_frozen_seed`. Un cycle de churn qui traverse deux composants est plafonné à `WARNING` : une dep de prop n'est jamais exactement le slot du parent.

14.11.3 Le cliquet

Extrait 14.44 – `tests/layer_boundary.rs`, l. 19–31 — les fichiers de règles qui touchent encore la syntaxe

```
19 /// The rule files still walking syntax, as of ADR-042 slice 5. Shrink only.
20 const ALLOWED: &[&str] = &[
21     "src/rules/api/query.rs",
```



```

22  "src/rules/api/witness.rs",
23  "src/rules/helpers/jsx.rs",
24  "src/rules/helpers/mod.rs",
25  "src/rules/helpers/mount.rs",
26  "src/rules/helpers/purity.rs",
27  "src/rules/impls/redundant_set_state.rs",
28  "src/rules/impls/setter_in_render.rs",
29  "src/rules/impls/state_mutation.rs",
30  "src/rules/impls/unnecessary_rerender.rs",
31 ];

```

Dix fichiers, et la liste ne peut que rétrécir. Le test lit chaque fichier de `src/rules/`, ignore la partie après `#[cfg(test)]`, cherche les cinq marqueurs, et échoue si un fichier hors liste en contient un *ou si un fichier de la liste n'en contient plus* (il doit alors être retiré). Notons qu'ADR-42 §1 cite `impls/conditional_hook.rs` dans la liste : au commit `e67b10a`, c'est `helpers/mod.rs` qui y figure à sa place ; l'ADR est un historique, le code fait foi.

14.11.4 Ce que la frontière ne garantit pas encore

Le cliquet interdit à une règle de *marcher la syntaxe* ; il n'interdit pas une *seconde lecture* d'un fait à travers une fonction du moteur. `collect_setter_calls`, la forme historique « une ligne par variable », est encore appelée directement par `setter-in-render`, `derived-state`, `infinite-loop`, `stale-closure`, `rules/helpers/mount.rs` et l'entité Tier-A. Voici ce que cela coûte.

Exemple 14.13: Un alias de setter au rendu

Extrait 14.45 – `/tmp/verif07/alias.tsx`

```

1  import { useState } from 'react';
2  export function A() {
3    const [n, setN] = useState(0);
4    const s = setN;
5    s(1);
6    return <div>{n}</div>;
7  }

```

La relation porte la ligne `slot=0 setter=s region=Render phase=Render via=Direct block=Some(0)` : une écriture synchrone au rendu, dans le bloc d'entrée. La ligne de commande `(-info -show-clean)` certifie pourtant l'absence :

```

A (1 hooks) /tmp/verif07/alias.tsx ✓
verified conditional-hook all hooks run unconditionally, in a stable order
verified lazy-init no useState/useRef initializer re-runs work on every render
verified setter-in-render no setter is called during render
verified state-mutation no state or prop object is mutated in place

```

Cause : la règle construit ses noms de setters à partir des seuls `let v = useState(1)` du rendu et des setters étrangers (`src/rules/impls/setter_in_render.rs`, l. 62–93) ; son unique appel à `resolve_setter_aliases` (`src/rules/impls/setter_in_render.rs`, l. 99–102) porte sur les *valeurs* d'état, pas sur les setters. Elle re-marche ensuite le rendu par `collect_setter_calls` (`src/rules/impls/setter_in_render.rs`, l. 109) au lieu de lire `slot_writers`. Le même silence frappe l'exemple 14.6 : `putState(setN, 2)` au rendu passe par l'alias de paramètre `setter#0`, et la ligne `Via(["putState"])` de phase `Render` ne produit aucun diagnostic.

Remarque

Au commit e67b10a, ce faux négatif de niveau ERROR ne figure pas dans le tracker. C'est, mot pour mot, la dérive entre deux lectures d'un même fait qu'ADR-42 §1 combat. `setter_in_render.rs` figure d'ailleurs encore dans la liste du cliquet (liste 14.44).

14.12 Limites, pièges et dette documentaire

14.12.1 Pièges de lecture

Piège

Région $\neq$ phase. Un callback littéral écrit au rendu est de *région* Render mais de *phase* $\top$. Conclure « écrit pendant le rendu » depuis la région a supprimé un vrai positif (ADR-31 §3). Réciproquement, `seeds.rs` utilise la région Effect *et* un filtre de phase (#121).

Piège

Les polarités sont à sens unique. `same_tick = false` n'est pas une preuve; `SeedSync::NoneSeen` est une absence, que `setter_escapes` qualifie; `Pairing::Unpaired` n'est une affirmation que sur les formes que l'appariement reconnaît; l'absence de ligne n'est jamais une preuve. Illustration (`/tmp/verif07/loc.tsx`) : `const put = (s, v) => s(v); put(setN, 2)` au rendu ne produit *aucune* ligne, parce que le bras B6 marche le corps de `put` sans lier ses paramètres aux arguments; `setter_escapes` vaut `true` pour ce slot, ce qui protège les règles qui affirment « personne d'autre n'écrit », mais une règle positive comme `setter-in-render` perd le site.

Piège

Deux WriteSite. `setters::WriteSite` (`pub(crate)`, le site brut de la marche) et `guards::WriteSite` (`pub`, le site de la preuve multi-site du chapitre 15) sont deux types sans rapport. De même, `build_edges` contient deux `invariance_of`, une fonction imbriquée puis une closure du même nom qui la masque.

Piège

Lignes sans span. Le corps d'une flèche à expression (`const safe = () => setCount(c => c + 1)`, exemple 14.4) est un terminateur Return, qui ne porte pas de span; marché comme racine de région, il est entré avec `witness = None` et n'hérite de rien. C'est le résidu connu de #140; ADR-39 §3 ne couvre que les corps entrés *depuis* un site d'appel. Conséquence : la ligne trie en dernier et `dedup_source_sites` ne peut pas l'identifier à une autre.

14.12.2 Une ligne étrangère lue comme écrivain local

La méthode `AnalysisResult::slot_written_outside(slot, except)` (`src/engine/setters.rs`, l. 2346–2364) répond « une autre région que `except` peut-elle écrire ce slot ? » par `self.slot_writers.iter().any(|w| w.slot == slot && w.region != except)`. L'invariant « tout lecteur natif filtre owner » (ADR-30 §3) a une exception : cette méthode ne filtre pas owner. Sur `/tmp/verif07/fo1.tsx`, un Child reçoit `onP=setP` d'un Parent dont le slot `p` a le label 0, et possède lui-même un slot local de label 0 (`a`) écrit par un effet `setA(b * 2)` sous [`b`]. La ligne étrangère `slot=0 owner=Some("Parent")` est prise pour un autre écrivain du slot local 0, et `derived-state` renonce : aucun diagnostic. Sur `fo2.tsx`, même programme avec un `useState` de plus dans Parent (le setter écrit alors le label 1), la règle émet `warn derived-state [hook:2]` (line 10:2). Même

programme à un renommage près, verdict différent : un faux négatif de niveau `WARNING`, dans la direction « retenue » (la doc de la méthode le dit : « both consumers use it to *withhold* a finding »), contraire à ADR-30 §3. Au commit e67b10a, ce défaut ne figure pas dans le tracker, et le « second consommateur » qu’annonce la doc n’existe pas : le seul appel de la méthode dans `src/` est `src/rules/impls/derived_state.rs`, l. 118.

14.12.3 Faux négatifs et faux positifs connus

Faux négatifs ouverts touchant ce chapitre : #157 (valeur dérivée du slot écrit lue `Not fraîche` : le motif « mirror state » rend le `fix` naïf `Versioned` $\Rightarrow$ `Maybe` inacceptable), #20 (un parent analysé seulement en `intra` ne donne pas de `ComponentSetter` à l’enfant, donc pas de ligne étrangère), #46 (setter appelé via un `index` ou une fonction retournée), #52 (utilitaires `inlinés` en position d’instruction seulement). S’y ajoutent les quatre constats non répertoriés de ce chapitre : l’alias de setter ignoré par `setter-in-render` (exemple 14.13, niveau `ERROR`) ; `slot_written_outside` sans filtre `owner` (niveau `WARNING`) ; le setter passé en paramètre à un helper local, sans ligne (piège ci-dessus) ; l’initialiseur `useState(p.x ?? d)` sans ligne `slot_seeds` (exemple 14.9, niveau `WARNING`). Faux positifs assumés, au plus `WARNING` : `same_tick` lu vrai sur des branches exclusives d’un helper `inliné` (#123), les résidus de la garde de synchronisation (#91 : garde disjonctive, garde de bornage), l’heuristique de nom des registrations (#42, `wontfix` : un `on/addListener` à deux arguments ou `subscribe` à un argument est une registration longue durée), et les résidus de la preuve multi-site (#159, #161) traités au chapitre 15.

14.12.4 Coûts et déterminisme

La marche paie `Reachability::of` à chaque CFG entré : un BFS par bloc, la table des successeurs coûtant elle-même un balayage linéaire des arêtes par bloc (`src/engine/setters.rs`, l. 933–937 rappelle que l’ancienne version par ligne était en $O(\text{lignes} \times \text{blocs} \times \text{arêtes})$). `SiteEnvs::eval` clone les trois stores à chaque ligne évaluée par valeur, coût non mesuré dans le dépôt. Le déterminisme repose sur CFG : `blocks`, un `BTreeMap` (`src/ir/cfg.rs`, l. 54–65) : toute marche qui itère `cfg.blocks` visite les blocs dans l’ordre des identifiants ; les sources de non-déterminisme restantes (les `HashMap` de résultats, les `HashSet` d’identifiants du tas) sont celles que #120 a dû trier (`collect_component_setter_vars`, `normalize_to_prop`, la déduplication des arêtes de `churn`).

14.12.5 Dette documentaire au commit e67b10a

Le lecteur qui compare les textes au code rencontrera ces écarts, tous vérifiés : le commentaire de `AnalysisResult::slot_writers` (granularité d’avant ADR-28) ; le commentaire de `Written::value` (antérieur à `returns_value`) ; ADR-42 §2 (`block = prov_block` ; `value =` partie référence ; `T` pour un site imbriqué), §3 (clé d’un trigger), §4 (« the convergence kill is unchanged », remplacé par les amendements de §6), §5 (`ProgramRelations` remplace `ProgramCache`) et ses « Consequences » (`rules/api/cache.rs` « is gone », #39 « untouched », contredits par le dernier paragraphe) ; ADR-20 item 9 (« the splice emits no param aliases », alors que l’exemple 14.6 montre `let setter#0 = setN`) ; le paragraphe « An edge is dropped when » de `docs/relations.md`, en retard d’un amendement ; et deux doc-comments déplacés dans `setters.rs` (la première ligne de la doc de `resolve_setter_aliases` est au-dessus de `setter_var_labels`, `src/engine/setters.rs`, l. 511–514 ; celle de `collect_fn_bindings` en tête de `certified_fn_names`, `src/engine/setters.rs`, l. 442). Ordre de foi : le code ; les doc-comments de module de `churn.rs` et `guards.rs`, réécrits au commit e67b10a ; le dernier amendement d’ADR-42 §6 ; `docs/relations.md` ; le reste d’ADR-42 comme récit historique.

Résumé du chapitre

Idées à retenir. Un fait calculé à deux endroits est deux faits qui dérivent (ADR-42) : les relations sont calculées une fois, dans la dernière tranche du point fixe, sur les CFG post-expansion, et rangées dans l'AnalysisResult; les règles les lisent et ne marchent aucune syntaxe (cliquet). Chaque colonne a une polarité (exact, must, may, $\top$ -bearing), et une ERROR ne s'assemble que depuis des colonnes must ou exactes, par une primitive `must_*` qui re-dérive sa preuve. La marche des setters décide la *classe* d'un site par la manière dont elle est entrée dans son CFG : synchrone à la racine, dans un helper appelé (B6) ou une IIFE; différée par un registrar, un await ou un cleanup; handler par `addEventListener`; $\top$ pour tout callee inconnu. Rétrécir hors de $\top$ n'est permis que sur contrat. Une `FnLit` n'est jamais traversée de l'extérieur.

Types et fichiers.

- `src/engine/setters.rs` : `WriterRegion` (exact), `WriterPhase` (may), `WalkClass`, `SetterWalk`, `Found`, `SlotWriter` et ses douze champs (quatorze colonnes une fois `written` déplié), `SetterProp`, `WriteProvenance`, `InlineRegions`, `Updater`;
- `written.rs` : `Freshness`, `Written`, `SiteEnvs`; `triggers.rs` : `EffectTrigger`; `seeds.rs` : `SeedSync`, `SlotSeed`;
- `registrations.rs` : `Registrar`, `REGISTRARS`, `Timing`, `Pairing`, `Registration`;
- `guards.rs` : `guard_chain`, `site_guards`, `expand_guard`, `dead_once_written`, `converges_once_written`; `program_relations.rs` : `ProgramRelations`;
- `src/rules/api/query.rs` : `must_direct_write` et `must_effect_cycle`;
- le cliquet : `tests/layer_boundary.rs`.

Ce que le chapitre suivant ajoute. Le chapitre 15 replie `slot_writers` et `effect_triggers` en un graphe programme-entier, cherche ses cycles, et généralise la preuve mono-site en `converges_under_all_writes` : les gardes d'un site doivent mourir sous *toutes* les écritures du slot qui s'exécutent dans la boucle automatique, les revivers déjà convergents étant exclus par un plus petit point fixe.

Exercices

Exercice 14.1: Prédire les lignes

Prédire les lignes `slot_writers` (région, phase, block, guard_block, same_tick, fresh) de

```
1 useEffect(() => {
2   items.forEach(i => setX(i));
3   setY(1);
4   setY(2);
5 }, [items]);
```

Solution

Observé sur `/tmp/ch14/exo1.tsx` : `setX` a `phase=Effect` (le HOF synchrone propage la classe englobante), `repeats` donc `same_tick=true`, `block=None`, `guard_block=Some(0)`, `fresh=Maybe` (i est $\top$); les deux `setY` ont `block=Some(0)`, `same_tick=true` (même bloc), `fresh=Not`.

Exercice 14.2: Un updater qui ranime

Pourquoi `setS(prev => null)` à un autre site ranime-t-il une garde `if (!s)` ? Quelle fonction de `written.rs` le garantit, et que dirait l'ancien commentaire du champ `value` ?

Exercice 14.3: Refuser Functional

Construire un composant où `setN(inc)` n'est pas classé `Updater::Functional` bien que `inc` soit lié à un littéral de fonction au rendu ; expliquer le rôle de `certified_fn_binding` et pourquoi `collect_fn_bindings` n'est pas la bonne barre.

Exercice 14.4: subscribe et BehaviorSubject

Donner un programme où classer `subscribe` en `Handler` produirait un faux négatif dans une règle Tier-A à garde `writer_phases includes effect`. Quelle colonne de `REGISTRARS` encode la décision ?

Exercice 14.5: once contre capture

Écrire un effet qui appelle `addEventListener(t, h, once: true)` sans `cleanup` et prédire `pairing` ; puis avec `true` en troisième argument. Vérifier avec la sonde.

Solution

Paired dans le premier cas (`self_removing`, tranché avant toute comparaison) ; Unpaired dans le second, si `h` est un nom (ligne comparable, `Cleanups::None`), conformément à la figure 14.5. Sur `/tmp/ch14/regs.tsx`, la ligne `'scroll'` est `self_removing=true pairing=Paired`.

Exercice 14.6: Aucune ligne de rendu pour un handler

Expliquer pourquoi `onClick=() => setN(1)` ne produit aucune ligne de région `Render`, alors qu'aucun code de `collect_slot_writers` ne déduplique. Que se passerait-il si `expr` traversait les `FnLit` ?

Solution

Le lowering réifie la flèche en `HookEntry::Handler (extract_handlers)`, marchée comme sa propre région ; depuis le CFG de rendu, la flèche n'est ni appelée, ni passée à un `callee`, ni retournée par un effet, donc la machinerie d'appel n'y entre jamais (liste 14.14). Si `expr` traversait les `FnLit`, le site serait vu une seconde fois depuis le rendu, avec la classe `T` : une ligne `region = Render`, `phase = Unknown` en double, qui satisfait toute requête de phase (une garde Tier-A `writer_phases includes render`, par exemple) sans correspondre à aucune exécution nouvelle.

Exercice 14.7: Réparer la dérivation

Proposer, en une phrase justifiée, la modification de `setter-in-render` qui ferait disparaître le faux négatif de l'exemple 14.13 sans réintroduire une marche dans la règle. Quelles colonnes de `SlotWriter` suffisent à reconstituer la preuve d'ERROR actuelle (dominance de sortie, phase synchrone, `must_write`) ?

Chapitre 15

Le graphe de churn et la preuve multi-site

Objectifs du chapitre.

- Comprendre pourquoi une boucle de rendu répartie sur plusieurs effets échappe au point fixe d'un composant et au bras « self-churn » de *infinite-loop*.
- Savoir lire et construire le *graphe de churn* : slots qualifiés, arêtes Must / May, auto-arêtes des effets sans deps, partition *self_slot*.
- Suivre la recherche de cycles (Tarjan en deux passes) et la stratification ERROR / WARNING qu'elle induit.
- Maîtriser le *kill de convergence* : de la condition « un seul écrivain » (ADR-18) à la preuve multi-site *converges_under_all_writes* (ADR-42), puis aux *sites* et aux *revivers* du commit e67b10a.
- Comprendre pourquoi l'ensemble des sites convergents est un *plus petit* point fixe, et pourquoi le plus grand serait insound.
- Situer la frontière de certification : ce que le moteur affirme (Must/May) et ce que la règle re-dérive avant de frapper une ERROR.

Prérequis : chapitre 14 (la relation *slot_writers*, ses colonnes *owner*, *block*, *guard_block*, *written* ; la relation *effect_triggers* ; la preuve mono-site *converges_once_written*, ses gardes et ses trois bras), chapitre 10 (le treillis *Stability*, *PerRender* et *Versioned*), chapitre 13 (environnements convergés, tranche finale du point fixe), chapitre 3 (*must / may*, dominance, plus petit point fixe).

Ce chapitre est le plus difficile de la partie moteur. Il traite d'une question qu'aucun des mécanismes vus jusqu'ici ne sait trancher seul : *deux* effets, chacun anodin, peuvent former ensemble une boucle de rendu infinie ; et, symétriquement, un effet qui *a l'air* de boucler peut converger parce qu'une garde meurt une fois sa propre écriture posée dans le slot. Le premier problème demande un graphe ; le second une preuve. Et les deux se rencontrent : une preuve de convergence peut être *ranimée* par une écriture voisine, si bien que la preuve doit connaître *tous* les sites qui écrivent le slot, et savoir lesquels ne tirent qu'une fois. Nous suivrons l'ordre historique, qui est aussi l'ordre de difficulté : le graphe sans kill, le kill mono-écrivain, la preuve multi-site, les sites, les revivers et le plus petit point fixe.

Tout ce chapitre vit dans deux fichiers du moteur, *src/engine/churn.rs* (927 lignes) et *src/engine/guards.rs* (1 058 lignes), tous deux nés au commit 05d3573 (ADR-42) et profondément remaniés au commit e67b10a. Les tests d'intégration sont dans *tests/effect_cycles.rs*. Toutes les sorties d'analyseur montrées ici ont été observées sur des fichiers de */tmp/ch15/* avec *reactant check* <fichier> -rule infinite-loop -rule cross-component-infinite-loop -info -show-clean -trace ; les tables de relations et d'arêtes proviennent d'une petite sonde qui appelle *analyze_program* puis *ProgramRelations::new(&result).churn()* et imprime les lignes (colonnes élaguées à l'affichage, jamais modifiées).

15.1 Le problème : deux effets qui se relancent

15.1.1 Un programme de huit lignes

Exemple 15.1: Deux effets miroirs

Chaque effet recopie dans *son* slot un champ de l'*autre* slot, dans un objet littéral neuf.

Extrait 15.1 – /tmp/ch15/two_effects.tsx

```
1 import { useState, useEffect } from 'react';
2 export function TwoEffects() {
3   const [a, setA] = useState({ n: 0 });
4   const [b, setB] = useState({ n: 0 });
5   useEffect(() => { setB({ from: a.n }); }, [a]);
6   useEffect(() => { setA({ from: b.n }); }, [b]);
7   return <div>{a.n + b.n}</div>;
8 }
```

Déroutons ce que fait React. Au montage, les deux effets tournent : `setB(...)` place une référence neuve dans `b`, `setA(...)` une référence neuve dans `a`. Le composant re-rend. Le premier effet compare son `dep a` par `Object.is` avec la valeur précédente : ce sont deux objets distincts, il re-tourne et écrit un *nouveau* `b`. Le second effet fait de même et écrit un nouveau `a`. Rien ne converge jamais : c'est une boucle de rendu infinie, que React interrompt avec l'erreur « Maximum update depth exceeded ».

Rappel React : `Object.is` sur les deps

Après un commit, React ré-exécute un effet si l'un de ses `deps` a changé au sens de `Object.is` par rapport au rendu précédent. Un objet littéral ... évalué dans un effet est une référence neuve à chaque exécution : `Object.is` échoue *toujours*. Une écriture d'état qui stocke une telle référence relance donc tout effet dont un `dep` *est* ce slot. Voir chapitre 2.

Voici ce que dit `reactant-analyzer` sur ce fichier.

```
TwoEffects (4 hooks) /tmp/ch15/two_effects.tsx
error infinite-loop [hook:2] (line 5:2) these effects form a state-update cycle (`a` → `b` → `a`) where each
  ↳ step stores a fresh reference that re-runs the next effect: infinite render loop
    → a fresh value is written to state `b` here [hook:2] (line 5:20)
    → cycle continues: this effect freshly stores state `a` [hook:3] (line 6:20)
error infinite-loop [hook:3] (line 6:2) these effects form a state-update cycle (`a` → `b` → `a`) where each
  ↳ step stores a fresh reference that re-runs the next effect: infinite render loop
    → a fresh value is written to state `a` here [hook:3] (line 6:20)
    → cycle continues: this effect freshly stores state `b` [hook:2] (line 5:20)

! 2 error(s) across 1 file(s).
```

Deux ERROR, une par effet porteur, et un chemin '`a`' → '`b`' → '`a`'. C'est le résultat que ce chapitre explique.

15.1.2 Pourquoi les autres bras sont aveugles

La règle `infinite-loop` (chapitre 19) a quatre bras. Deux d'entre eux ne peuvent *pas* voir ce programme, et comprendre pourquoi motive le graphe.

Le bras point fixe ne voit aucune divergence. Le point fixe d'un composant (chapitre 13) détecte une boucle par le *widening* : un compteur `setN(n + 1)` fait croître l'intervalle jusqu'au seuil, et l'événement de widening est le signal. Ici la valeur abstraite d'un slot objet est une *stabilité*

(définition 10.3) ; la référence stockée est `PerRender`, et $\text{PerRender} \sqcup \text{PerRender} = \text{PerRender}$. Rien ne croît, rien ne s'élargit : le point fixe converge en deux tours et ne dit rien. La sémantique de *valeur* ne distingue pas « $0 \rightarrow 1 \rightarrow 0 \rightarrow 1$ » de « atteint $\{0, 1\}$ » ; le churn est une abstraction de *transition*, et c'est l'argument d'ADR-42 (contexte) pour ne pas en faire un champ de `StateValue`.

Le bras self-churn exige le même slot. Le bras d'ADR-17 prouve `useEffect(() => setObj(...obj), [obj])` : le slot écrit *est* un dep du même effet (`must-rerun`), l'écriture est sur tous les chemins (`must-reach`), la valeur est `PerRender` (`must-change`). Dans l'exemple 15.1 chaque effet écrit un slot qu'il ne lit *pas* : `setB` sous `[a]`, `setA` sous `[b]`. La conjonction ne se referme qu'en composant les deux effets.

15.1.3 Les trois jambes d'une arête

Le graphe compose exactement les trois `must` du bras self-churn, mais *par arête* plutôt que par effet. Une arête $x \rightarrow y$ signifie : « un changement de x relance un effet qui stocke une référence fraîche dans y » (doc de module, `src/engine/churn.rs`, l. 3–6). Pour qu'elle soit certaine, il faut :

1. **must-rerun** : un dep de l'effet *est* le slot x (ligne `effect_triggers` avec `exact = true`) ;
2. **must-reach** : l'écriture dans y est dans un bloc sur tous les chemins du corps (`on_all_paths`) ;
3. **must-change** : la valeur écrite est `Fresh`, c'est-à-dire `PerRender` (colonne `written.fresh` de `slot_writers`).

Un cycle dont *toutes* les arêtes sont certaines est une boucle certaine : c'est la preuve d'ERROR. Une arête où l'une des jambes manque est `May` ; un cycle qui en contient une est un `WARNING`. Le reste du chapitre déplie cette phrase.

15.2 Le graphe : slots qualifiés, arêtes `Must` et `May`

15.2.1 Les nœuds : des slots qualifiés

Un nœud du graphe n'est pas un slot mais un *slot qualifié* : le couple `(ComponentId, HookLabel)`, type `QualifiedSlot` (`src/ir/types.rs`, l. 9). La qualification est indispensable dès qu'un effet écrit le slot d'un *autre* composant à travers un setter passé en `prop` : la ligne `slot_writers` porte alors `owner = Some(parent)` et son slot est le label *du propriétaire* (définition 14.2). La fonction `node_of` (`src/engine/churn.rs`, l. 153–156) fait cette traduction en une ligne : `(w.owner.unwrap_or(component), w.slot)`.

15.2.2 Les types

Extrait 15.2 – `src/engine/churn.rs`, l. 85–129 — les quatre types publics du graphe

```
85 #[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord)]
86 pub enum EdgeStrength {
87     /// Dep merely versioned by `from`, or the write is conditional/imprecise.
88     May,
89     /// Exact-slot dep  $\wedge$  must-fresh write on all paths.
90     Must,
91 }
92
93 #[derive(Debug, Clone)]
```



```

94 pub struct ChurnEdge {
95     pub from: QualifiedSlot,
96     pub to: QualifiedSlot,
97     pub strength: EdgeStrength,
98     /// Component whose effect carries this edge.
99     pub component: ComponentId,
100    pub effect_label: HookLabel,
101    pub write_span: Option<SourceRange>,
102    /// The carrying effect has no dependency array.
103    pub no_deps: bool,
104    /// A dep-driven edge from a slot into itself: the self-churn arm's
105    /// partition (ADR-020 item 2). Never part of a cycle the graph reports.
106    pub self_slot: bool,
107 }
108
109 /// One churn cycle: indices into the edge list, in cycle order
110 /// (`edges[i].to == edges[i+1].from`, last wraps to first).
111 #[derive(Debug, Clone)]
112 pub struct ChurnCycle {
113     pub edge_idx: Vec<usize>,
114     pub all_must: bool,
115     /// The cycle involves more than one component (slot owners or effect
116     /// carriers) – severity is capped at Warning: cross-component must-rerun
117     /// cannot be proven (prop deps are `Versioned`, never exact).
118     pub cross_component: bool,
119 }
120
121 /// The program's churn graph: every edge plus the cycles found in it.
122 ///
123 /// Whole-program data – `build` reads every component – so it is computed
124 /// once per program and shared by every component's rule pass
125 /// ([`crate::engine::ProgramRelations`]).
126 pub struct ChurnGraph {
127     pub edges: Vec<ChurnEdge>,
128     pub cycles: Vec<ChurnCycle>,
129 }

```

Quatre types, dont on retiendra :

- `EdgeStrength` dérive `Ord` avec `May < Must` : la déduplication garde « la plus forte » par simple comparaison ;
- `ChurnEdge` porte, outre ses extrémités et sa force, le *porteur* (`component`, `effect_label`), le span de l'écriture pour le témoin, et deux booléens de *partition* : `no_deps` (l'effet n'a pas de tableau de deps) et `self_slot` (arête guidée par deps d'un slot vers lui-même, réservée au bras self-churn) ;
- `ChurnCycle` est une liste d'indices d'arêtes *en ordre de cycle* (`edges[i].to == edges[i+1].from`, la dernière rebouclant sur la première) et deux faits exacts, `all_must` et `cross_component` ;
- `ChurnGraph` est une donnée *programme-entier*, construite une fois par programme et partagée par toutes les passes de règles via `ProgramRelations` (chapitre 14) ; le compteur `BUILDS` (`src/engine/churn.rs`, l. 131–137) n'existe qu'en test pour le garantir (#86).

Définition 15.1: Graphe de churn

Soit un programme dont les composants sont convergés. Le *graphe de churn* est le graphe orienté dont les nœuds sont les slots qualifiés (c, ℓ) et dont les arêtes $x \xrightarrow{e} y$ sont portées par un effet e (non mount-only, voir section 15.5) tel que : e possède une ligne d'écriture de y de phase autre que `Handler` et de fraîcheur autre que `Not` ; et soit e n'a pas de tableau de deps et $x = y$ (*auto-arête*), soit une ligne `effect_triggers` de e nomme x . L'arête est `Must` quand la

ligne de déclencheur est exacte, que l'écriture est *Fresh*, synchrone (*block = Some*) et que l'ensemble des blocs d'écritures fraîches de *y* dans le corps de *e* est sur tous les chemins ; elle est *May* sinon. Une arête tuée par la preuve de convergence (section 15.6) n'existe pas. Un *cycle* est un cycle simple de ce graphe ne passant par aucune arête *self_slot*.

La définition emprunte deux notions qui seront précisées plus loin (les sites et le kill) ; pour l'instant, on lit le graphe *sans kill*. Le sens d'une arête est celui de la doc de module : $x \rightarrow y$ dit qu'un *changement* de *x* relance un effet qui stocke une référence *fraîche* dans *y*. Le graphe parle donc de transitions, pas de valeurs.

15.2.3 La force d'une arête

Extrait 15.3 – `src/engine/churn.rs`, l. 530–543 — la jambe *must-reach* et la force

```

530     let fresh_blocks: HashSet<BlockId> = f
531         .writes
532         .iter()
533         .filter(|o| node_of(f.comp, o) == node && o.written.fresh == Freshness::Fresh)
534         .filter_map(|o| o.block)
535         .collect();
536     let must_write = w.written.fresh == Freshness::Fresh
537         && w.block.is_some()
538         && on_all_paths(f.body_cfg, &fresh_blocks);
539     let strength = if must_write {
540         EdgeStrength::Must
541     } else {
542         EdgeStrength::May
543     };

```

La force se décide en trois conjoints, dans l'ordre de la définition :

1. `w.written.fresh == Freshness::Fresh` : la colonne *written* (chapitre 14) dit que l'argument 0 est une référence *PerRender* à chaque appel ; *Maybe* (valeur opaque, updater imprécis) ne suffit pas ;
2. `w.block.is_some()` : la ligne porte un bloc, ce qui n'arrive que pour une écriture *synchrone, une fois par passe* de la région (définition 14.2) ; une écriture imbriquée dans une closure, différée, de *cleanup* ou répétée (*forEach*) a *block = None* et ne peut être que *May* ;
3. `on_all_paths(f.body_cfg, &fresh_blocks)` : l'ensemble de *tous* les blocs d'écritures fraîches du même nœud dans ce corps post-domine l'entrée. Prendre l'ensemble plutôt que le seul bloc courant est ce qui rend *Must* un *if (p) setX(...) else setX(...)* : aucun des deux blocs n'est sur tous les chemins, mais leur union l'est.

Extrait 15.4 – `src/engine/dominance.rs`, l. 72–92 — `on_all_paths`, la primitive *must-reach*

```

72 pub fn on_all_paths(cfg: &CFG, blocks: &HashSet<BlockId>) -> bool {
73     if blocks.contains(&cfg.entry) {
74         return true;
75     }
76     // BFS avoiding `blocks`; reaching an exit block means a path escapes.
77     let mut visited: HashSet<BlockId> = HashSet::new();
78     let mut queue = vec![cfg.entry];
79     visited.insert(cfg.entry);
80     while let Some(bid) = queue.pop() {
81         let succs = cfg.successors(bid);
82         if succs.is_empty() {
83             return false; // exit reached without hitting a block of the set
84         }

```

```

85     for succ in succs {
86         if !blocks.contains(&succ) && visited.insert(succ) {
87             queue.push(succ);
88         }
89     }
90 }
91 true
92 }

```

`on_all_paths` est un parcours depuis l'entrée qui *évite* les blocs de l'ensemble : si l'on atteint un bloc sans successeur (une sortie), un chemin s'échappe et la réponse est `false`. Le commentaire parle de BFS, mais la « file » est un `Vec` dont on retire le dernier élément (`queue.pop()`) : le parcours est en profondeur. L'ordre de visite est sans effet sur la réponse, qui ne dépend que de l'ensemble des blocs atteignables. C'est la même primitive que `must_on_all_paths` côté règles (chapitre 12), déplacée dans le moteur par ADR-42 §6 (« les preuves suivent les données »).

La jambe `must-rerun`, elle, n'apparaît pas dans cet extrait : elle est lue dans les lignes `effect_triggers` du porteur, partitionnées en `exact_local` (le dep *est* le slot local, `exact = true`) et `versioned` (le dep est seulement *versionné* par le slot qualifié, `exact = false`) à `src/engine/churn.rs`, l. 313–323. Une arête issue de `versioned` est `May` quelle que soit sa force d'écriture (`src/engine/churn.rs`, l. 592).

Phase de la ligne	Arête guidée par deps	Auto-arête (effet sans deps)
Effect (synchrone, porte un bloc)	Must si <code>exact ∧ Fresh ∧</code> tous les chemins, sinon <code>May</code>	idem, sans dep à tester
Deferred, Cleanup	<code>May</code>	<code>May (#26)</code>
Handler	aucune arête	aucune arête
Unknown ($\top$)	<code>May</code>	<code>May</code>

TABLE 15.1 – La table des phases d'ADR-42 §4 : ce qu'une écriture de corps d'effet peut soutenir selon sa colonne phase. Le filtre `Handler` est à `src/engine/churn.rs`, l. 304–309, le reste découle de `block` (liste 15.3).

Le tableau 15.1 n'est pas codé comme une table : il *découle* de deux faits. Une ligne de phase `Handler` est filtrée à la collecte des écritures de l'effet (`src/engine/churn.rs`, l. 304–309) : une écriture qu'un listener seul atteint demande un événement utilisateur par itération, la boucle n'est pas auto-entretenu (preuve par la table des registrars, ADR-34). Toute autre ligne entre, et sa colonne `block` décide seule si elle peut être `Must`. Le $\top$ (`Unknown`) est donc `May` : la direction « tirer plus ».

15.2.4 L'effet sans deps et son auto-arête

Exemple 15.2: Écriture fraîche sans tableau de deps

Extrait 15.5 – `/tmp/ch15/nodeps.tsx`

```

1 import { useState, useEffect } from 'react';
2 export function NoDeps() {
3     const [obj, setObj] = useState({ a: 1 });
4     useEffect(() => {
5         setObj({ a: 2 });
6     });
7     return <p>{obj.a}</p>;
8 }

```

```

9 export function NoDepsThen() {
10   const [obj, setObj] = useState({ a: 1 });
11   useEffect(() => {
12     Promise.resolve().then(() => setObj({ a: 2 }));
13   });
14   return <p>{obj.a}</p>;
15 }

```

```

NoDeps (2 hooks) /tmp/ch15/nodeps.tsx
  error infinite-loop [hook:1] (line 4:2) this effect has no dependency array and stores a fresh reference
    ↳ into state `obj`, so it re-runs after every render and re-triggers itself: infinite render loop
    ↳ a fresh value is written to state `obj` here [hook:1] (line 5:4)
NoDepsThen (2 hooks) /tmp/ch15/nodeps.tsx
  warn infinite-loop [hook:1] (line 11:2) this effect has no dependency array and may store a fresh reference
    ↳ into state `obj`, so it re-runs after every render and can re-trigger itself: possible infinite render
    ↳ loop
    ↳ a fresh value is written to state `obj` here [hook:1] (line 12:4)

```

Un effet sans tableau de deps re-tourne après *chaque* rendu, y compris celui que sa propre écriture provoque : il n’y a pas de dep à tester, l’auto-arête $y \rightarrow y$ est posée d’office.

Extrait 15.6 – `src/engine/churn.rs`, l. 569–576 — l’auto-arête d’un effet sans deps

```

569         // Re-runs after every render → its own write re-triggers it,
570         // whichever turn the write runs on (a handler row was dropped
571         // above).
572         if !killed {
573             push(node, strength, false);
574         }
575         continue;
576     }

```

Dans `NoDeps`, l’écriture est synchrone (`block = Some(0)`, l’entrée) et `Fresh` : l’arête est `Must`, le cycle de longueur 1 est tout-`Must`, `ERROR`. Dans `NoDepsThen`, la ligne a la phase `Deferred` (la table des registrars connaît `.then`) et donc `block = None` : l’arête est `May`, `WARNING`. C’est la cellule « `Deferred` / auto-arête = `May` » du tableau 15.1, et elle est une décision.

Décision : ADR-42 §4

Avant ADR-42, le graphe vivait dans `rules/helpers` avec sa propre marche des setters, sans classification de phase : une écriture imbriquée dans `.then` était lue « jamais auto-entretenu » et la boucle de `NoDepsThen` était silencieuse (#26, un FN). Plutôt que de corriger le second walker, ADR-42 le supprime : le graphe devient un *pli* sur `slot_writers` et `effect_triggers`, et la colonne phase décide. Deux cellules de la table changent le comportement : les auto-arêtes `Deferred/Unknown` apparaissent (côté « tirer plus ») et les arêtes guidées par deps d’une ligne `Handler` disparaissent (appuyées sur la preuve de la table des registrars). Mesure (`docs/precision-log.md`) : 1 493 → 1 494 sur le corpus, aucune ligne retirée, une ajoutée — un `WARNING FP` de la cellule T (une écriture sous un `useDebounceCallback` non résolu, dans `twenty`) ; la cellule #26 n’a rien changé sur ce corpus, elle est épinglée par `tests/effect_cycles.rs`.

Le champ `no_deps` est `deps.list().is_none()` (`src/engine/churn.rs`, l. 332) : un tableau de deps que le moteur ne sait pas lire (un `spread`, une variable) est traité comme *absent*, direction « tirer plus ».

15.2.5 Deux bras, une relation : `self_slot`

Extrait 15.7 – `src/engine/churn.rs`, l. 577–594 — les arêtes guidées par deps, exactes puis versionnées

```

577         for &l in &f.exact_local {
578             let x: QualifiedSlot = (f.comp, l);
579             let self_slot = x == node;
580             let dropped = killed || (self_slot && !write_can_retrigger(f, &ctx.state_vals, w));
581             if !dropped {
582                 push(x, strength, self_slot);
583             }
584         }
585         for &x in &f.versioned {
586             if f.exact_local.contains(&x.1) && x.0 == f.comp {
587                 continue; // already pushed as exact
588             }
589             let self_slot = x == node && x.0 == f.comp;
590             let dropped = killed || (self_slot && !write_can_retrigger(f, &ctx.state_vals, w));
591             if !dropped {
592                 push(x, EdgeStrength::May, self_slot);
593             }
594         }

```

Pour chaque slot local exact x des deps, une arête $x \rightarrow y$ de la force calculée; pour chaque slot qualifié versionné, une arête May (sauf s'il a déjà été poussé en exact). Le booléen `self_slot` marque l'arête $x \rightarrow x$ d'un effet *avec* deps : c'est le motif `setObj(...obj)` sous `[obj]` du bras `self-churn` (ADR-17), et le graphe la construit mais ne la cherche jamais dans un cycle.

Décision : ADR-20 item 2

Fusionner les deux bras (`self-churn` et graphe) a été proposé et refusé : le bras `self-churn` lit les *membres* du slot (`can_retrigger`, bras membre de la preuve) et produit un diagnostic ancré sur le slot avec sa propre stratification `ERROR/WARNING/INFO`; le bras graphe ne le peut pas (savoir si l'écriture de y par A change un dep de B est une propriété d'une *paire* d'arêtes). La partition est donc explicite : `self_slot` pour « même slot, avec deps » (bras `self-churn`), tout le reste (auto-arêtes sans deps comprises, poussées avec `self_slot = false` dans l'liste 15.6) pour le graphe. Une relation, deux lecteurs disjoints : aucun double rapport.

L'arête `self_slot` est abandonnée quand `write_can_retrigger` répond `false` (#90, `src/engine/churn.rs`, l. 828–852) : un updater fonctionnel `prev => (...prev, slug: f(prev))` laisse chaque membre non nommé `Object.is-égal`, et un dep qui ne lit que ces membres ne peut pas relancer l'effet. La preuve est syntaxique et *ferme par défaut* : `literal_overwrites` (`src/engine/churn.rs`, l. 882–899) n'accepte que `...prev, a: x, ...` avec le spread *en premier* et des clés nommées; toute autre forme répond « peut relancer ».

15.2.6 Déduplication et déterminisme

Une clé (`from`, `to`, `component`, `effect_label`) identifie une arête. Deux écritures du même nœud dans le même effet donnent deux candidates; on garde la plus forte, puis la plus tôt dans la source (`(strength, Reverse(pos))`, `src/engine/churn.rs`, l. 545–566), de sorte que la ligne sur laquelle un lecteur s'ancre ne dépende pas de l'ordre de la relation. Les arêtes sont enfin triées par (`component`, `effect_label`, `from`, `to`) (`src/engine/churn.rs`, l. 598–606) : `result.components` est une `HashMap` et l'ordre de visite varie d'une exécution à l'autre.

15.2.7 Trois graphes

Revenons à l'exemple 15.1. La sonde donne les deux lignes d'écriture, les deux déclencheurs et le graphe (colonnes élaguées) :

```
-- slot_writers
slot=0 setter=setA line=Some(6) region=Effect(3) phase=Effect block=Some(0) guard_block=Some(0) fresh=Fresh value.ref=
  ↳ PerRender
slot=1 setter=setB line=Some(5) region=Effect(2) phase=Effect block=Some(0) guard_block=Some(0) fresh=Fresh value.ref=
  ↳ PerRender
-- effect_triggers
hook=2 dep=0 slot=(TwoEffects, 0) exact=true
hook=3 dep=0 slot=(TwoEffects, 1) exact=true
=== churn graph
edge#0 (TwoEffects, 0) -> (TwoEffects, 1) strength=Must carrier=TwoEffects effect=2 line=Some(5) no_deps=false
  ↳ self_slot=false
edge#1 (TwoEffects, 1) -> (TwoEffects, 0) strength=Must carrier=TwoEffects effect=3 line=Some(6) no_deps=false
  ↳ self_slot=false
cycle edges=[0, 1] all_must=true cross_component=false
```

Le dep a de l'effet 2 est le slot 0 (exact=true) ; l'écriture de b est Fresh et dans le bloc 0, l'entrée, donc sur tous les chemins : arête $a \rightarrow b$ Must. Symétriquement $b \rightarrow a$. La figure 15.1 dessine ce graphe et deux voisins.

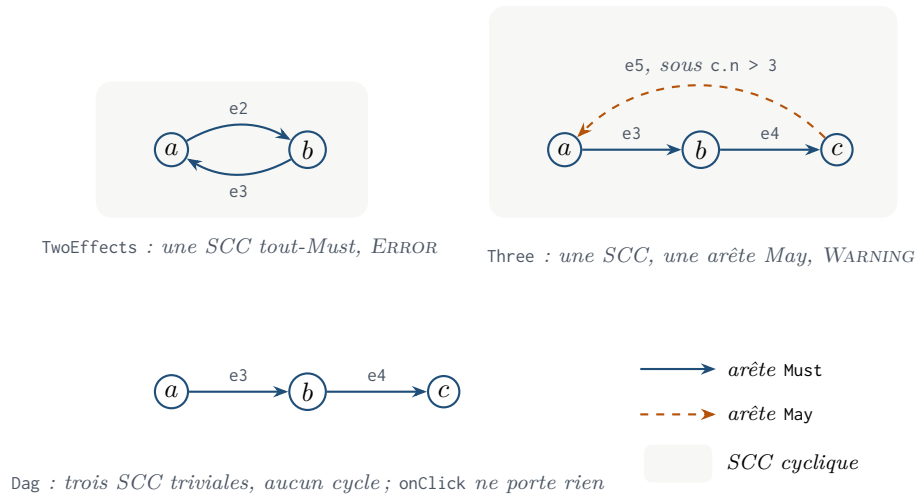


FIGURE 15.1 – Trois graphes de churn (two_effects.tsx, three.tsx, dag.tsx). Les nœuds sont des slots qualifiés du même composant ; eN est le label du hook porteur. Une arête pleine est Must, une arête pointillée May.

Exemple 15.3: Un cycle à trois, une garde

Extrait 15.8 – /tmp/ch15/three.tsx

```
1 import { useState, useEffect } from 'react';
2 export function Three() {
3   const [a, setA] = useState({ n: 0 });
4   const [b, setB] = useState({ n: 0 });
5   const [c, setC] = useState({ n: 0 });
6   useEffect(() => { setB({ from: a.n }); }, [a]);
7   useEffect(() => { setC({ from: b.n }); }, [b]);
8   useEffect(() => { if (c.n > 3) setA({ from: c.n }); }, [c]);
9   return <div>{a.n + b.n + c.n}</div>;
10 }
```

La sonde montre `block=Some(1)` pour `setA` (le bloc de la branche vraie, pas l'entrée) : `on_all_paths` échoue, l'arête $c \rightarrow a$ est `May`, le cycle n'est pas `all_must`. Trois `WARNING` « `may form a state-update cycle ('a'  $\rightarrow$  'b'  $\rightarrow$  'c'  $\rightarrow$  'a')` », un par effet porteur. Le même programme sans la garde (`three_must.tsx`) donne trois `ERROR`. Le lecteur notera que la garde `c.n > 3` ne *meurt* pas sous l'écriture `setA(...)` : elle teste `c`, pas `a` ; le kill de convergence de la section 15.6 n'a rien à dire ici.

Exemple 15.4: Une chaîne dérivée est un DAG

Dans `dag.tsx`, $a \rightarrow b \rightarrow c$ sans retour, et `setA` n'est appelé que dans `onClick`. Le graphe a deux arêtes `Must` et aucun cycle ; la ligne de `setA` a `region=Handler(5)` `phase=Handler` et n'entre pas. La règle n'émet que deux `INFO` (bras `self-churn` : « `freshly recreates state 'b' outside its deps. No update cycle was found` »), le marqueur d'imprécision résiduelle de la section 15.9.

15.3 Trouver les cycles : Tarjan en deux passes

15.3.1 Pourquoi deux passes

Le graphe une fois construit, il faut en extraire les cycles et leur attribuer une force. Une idée naïve serait de chercher les cycles dans le graphe entier et de déclarer `all_must` ceux dont toutes les arêtes sont `Must`. Elle rate un cas : un cycle tout-`Must` $a \rightarrow b \rightarrow a$ noyé dans une composante fortement connexe plus grande qui contient aussi une arête `May` $b \rightarrow c \rightarrow a$. Un seul cycle simple étant reconstruit par composante, on risquerait de ne rapporter que le `May`. D'où deux passes.

Extrait 15.9 – `src/engine/churn.rs`, l. 635–662 — `find_cycles` : sous-graphe `Must` d'abord, puis tout

```
635 pub fn find_cycles(edges: &[ChurnEdge]) -> Vec<ChurnCycle> {
636     let must_idx: Vec<usize> = (0..edges.len())
637         .filter(|&i| !edges[i].self_slot && edges[i].strength == EdgeStrength::Must)
638         .collect();
639     let all_idx: Vec<usize> = (0..edges.len()).filter(|&i| !edges[i].self_slot).collect();
640
641     let mut cycles = Vec::new();
642     let mut covered_nodes: HashSet<QualifiedSlot> = HashSet::new();
643
644     for cyc in cycles_in(edges, &must_idx) {
645         for &i in &cyc {
646             covered_nodes.insert(edges[i].from);
647             covered_nodes.insert(edges[i].to);
648         }
649         cycles.push(make_cycle(edges, cyc, true));
650     }
651     for cyc in cycles_in(edges, &all_idx) {
652         // A node already inside an all-must cycle: the Error already flags
653         // that loop region – don't re-report a weaker overlapping cycle.
654         if cyc.iter().any(|&i| {
655             covered_nodes.contains(&edges[i].from) || covered_nodes.contains(&edges[i].to)
656         }) {
657             continue;
658         }
659         cycles.push(make_cycle(edges, cyc, false));
660     }
661     cycles
662 }
```

1. **Passe Must.** On ne garde que les arêtes `Must` non `self_slot` (`must_idx`) et l'on cherche les composantes fortement connexes (SCC) cycliques ; chacune donne un cycle `all_must = true`, et ses nœuds sont marqués `covered_nodes`.
2. **Passe complète.** On reprend toutes les arêtes non `self_slot` (`all_idx`) ; un cycle dont un nœud est déjà couvert est *sauté* : l'ERROR signale déjà cette région, on ne rapporte pas un cycle plus faible qui la recoupe. Les autres donnent des cycles `all_must = false`.

Piège

La seconde passe saute un cycle `May` dès qu'un de ses nœuds est dans une SCC tout-Must. Un cycle $a \rightarrow b \rightarrow a$ tout-Must et un cycle $b \rightarrow d \rightarrow b$ `May` partagent b : seul le premier est rapporté. Ce n'est pas un FN, la boucle certaine couvre la région ; mais un lecteur qui compte les cycles de `graph.cycles` doit savoir qu'il ne compte pas tous les cycles du graphe.

15.3.2 Composantes et reconstruction

`cycles_in` (`src/engine/churn.rs`, l. 680–713) construit une table de nœuds *triée* (déterminisme), une liste d'adjacence nœud $\rightarrow$ indices d'arêtes, puis appelle `tarjan_sccs` (`src/engine/churn.rs`, l. 716–779), un Tarjan récursif classique — « les graphes sont minuscules », dit le commentaire. Une SCC est cyclique si elle a au moins deux nœuds ou si un de ses nœuds porte une auto-arête (`src/engine/churn.rs`, l. 700–703). Pour chaque SCC cyclique, `find_cycle_from` (`src/engine/churn.rs`, l. 783–814) reconstruit *un* cycle simple par un parcours en profondeur itératif depuis le plus petit nœud de la composante, en restant dans la SCC : il existe toujours, et son chemin d'arêtes devient `edge_idx`. Enfin `make_cycle` (`src/engine/churn.rs`, l. 664–676) pose `cross_component` : l'ensemble des composants impliqués (propriétaires de `from`, de `to`, et porteurs) a plus d'un élément.

Le coût est linéaire : $O(V + E)$ pour Tarjan, un parcours par SCC pour la reconstruction, le tout *une fois par programme*.

15.3.3 Ce que prouve un cycle tout-Must

Lemme 15.1: Composition des arêtes Must

Soit $x_0 \rightarrow x_1 \rightarrow \dots \rightarrow x_{n-1} \rightarrow x_0$ un cycle du graphe dont toutes les arêtes sont `Must`, portées par des effets $e_0, \dots, e_{n-1}$ d'un même composant. Dès que le composant est monté et que e_0 s'exécute une fois, chaque tour de rendu ré-exécute $e_0, \dots, e_{n-1}$ et pose une référence neuve dans chacun de $x_1, \dots, x_0$: la boucle de rendu est infinie.

Démonstration. Par récurrence sur les tours. Au montage, React exécute tous les effets, donc e_0 s'exécute. Supposons e_i exécuté au tour t . Son écriture de x_{i+1} est sur tous les chemins du corps (`must-reach`) : elle a lieu. Sa valeur est `PerRender` (`must-change`) : c'est une référence que `Object.is` distingue de la précédente, React planifie un rendu. Au tour $t + 1$, un `dep` de e_{i+1} est le slot x_{i+1} (`must-rerun`) : e_{i+1} s'exécute. La chaîne se referme sur e_0 au tour $t + n$. $\square$

C'est la version composée du triple `must` d'ADR-17 §3, et l'argument de *soundness* 1 d'ADR-18. Le lemme énonce une propriété du programme *concret* : c'est pourquoi la règle peut frapper une `ERROR` (« défaut certain dès que le code s'exécute »). Deux réserves, traitées plus loin : la preuve suppose que les jambes sont vraiment `must` (la section 15.8 montre pourquoi c'est faux au travers d'un `prop`), et qu'aucune garde ne meurt (section 15.6).

Voici la sortie observée pour `three_must.tsx`, la version sans garde de l'exemple 15.3 :

```
ThreeMust (6 hooks) three_must.tsx
error infinite-loop [hook:3] (line 6:2) these effects form a state-update cycle (`a` → `b` → `c` → `a`)
  ↳ where each step stores a fresh reference that re-runs the next effect: infinite render loop
    ↳ a fresh value is written to state `b` here [hook:3] (line 6:20)
    ↳ cycle continues: this effect freshly stores state `c` [hook:4] (line 7:20)
    ↳ cycle continues: this effect freshly stores state `a` [hook:5] (line 8:20)
error infinite-loop [hook:4] (line 7:2) these effects form a state-update cycle (`a` → `b` → `c` → `a`) ...
error infinite-loop [hook:5] (line 8:2) these effects form a state-update cycle (`a` → `b` → `c` → `a`) ...

! 3 error(s) across 1 file(s).
```

Le chemin affiché part toujours du plus petit nœud de la SCC (`find_cycle_from` démarre en `scc.iter().min()`), quel que soit l'effet porteur du diagnostic : les trois messages disent `'a' → 'b' → 'c' → 'a'`, et seule la première note (« a fresh value is written ... here ») change.

15.4 Le kill de convergence, première époque : un seul écrivain

15.4.1 Le motif « fetch once » en paire

Le graphe de la section précédente a un défaut immédiat : il voit des boucles là où le programme converge. Le motif le plus courant est la garde « si vide, remplir ».

Exemple 15.5: Deux chargements gardés

Extrait 15.10 – `/tmp/ch15/fetch_once_pair.tsx`

```
1 import { useState, useEffect } from 'react';
2 export function FetchOncePair() {
3   const [a, setA] = useState(null);
4   const [b, setB] = useState(null);
5   useEffect(() => { if (!b) setB({ src: 'e1' }); }, [a]);
6   useEffect(() => { if (!a) setA({ src: 'e2' }); }, [b]);
7   return <div>{a && b ? 'ok' : 'loading'}</div>;
8 }
```

```
FetchOncePair (4 hooks) /tmp/ch15/fetch_once_pair.tsx ✓
verified infinite-loop no effect diverges into an infinite render loop
```

Concrètement : au montage `b` est `null`, le premier effet écrit ... dans `b` ; `a` est `null`, le second écrit ... dans `a`. Au rendu suivant, les deux effets re-tournent (leurs deps ont changé) mais les deux gardes sont fausses : plus aucune écriture, le composant est stable en deux rendus. Sans kill, le graphe aurait pourtant deux arêtes May (les écritures sont sous une garde, `block=Some(1)` n'est pas sur tous les chemins) et un cycle $a \rightarrow b \rightarrow a$: un WARNING faux positif sur l'idiome le plus banal de React. La sonde confirme que le graphe est *vide* :

```
-- slot_writers
slot=0 setter=setA line=Some(6) region=Effect(3) phase=Effect block=Some(1) guard_block=Some(1) fresh=Fresh value.ref=
  ↳ PerRender
slot=1 setter=setB line=Some(5) region=Effect(2) phase=Effect block=Some(1) guard_block=Some(1) fresh=Fresh value.ref=
  ↳ PerRender
-- effect_triggers
hook=2 dep=0 slot=(FetchOncePair, 0) exact=true
hook=3 dep=0 slot=(FetchOncePair, 1) exact=true
=== churn graph
```

15.4.2 Pourquoi la garde meurt

L'argument est celui de la preuve mono-site du chapitre 14 (converges_once_written, section sur la preuve mono-site), que nous rappelons sur ce cas. Le corps du premier effet est abaissé en un bloc 0 terminé par Branch cond: !b, dont l'arête IfTrue mène au bloc 1 qui contient setB(...). guard_chain(cfg, 1) remonte la chaîne des prédécesseurs uniques : [1, 0]. site_guards lit la condition et la polarité de l'arête : (!b, true), que expand_guard réécrit en (b, false). Le bras *valeur* de dead_once_written ré-lie alors b à la valeur écrite — une référence PerRender, donc *truthy* et non nulle — et applique le rétrécissement de branche « b falsy » : $\perp$. La garde est morte sous sa propre écriture : une fois ... posé dans b, ce site ne peut plus tirer.

Remarque

Le kill porte sur le slot *écrit*, pas sur le slot lu par les deps. La garde !b du premier effet parle de b, que ce même effet écrit ; le dep a est sans importance pour la preuve. C'est ce qui permet de tuer l'arête $a \rightarrow b$: elle est « une écriture de b qui ne tire qu'une fois », d'où qu'elle soit déclenchée.

15.4.3 Le reviver : une écriture voisine ranime la garde

Exemple 15.6: Un troisième effet remet null

Extrait 15.11 – /tmp/ch15/multi_writer.tsx

```
1 import { useState, useEffect } from 'react';
2 export function MultiWriter() {
3   const [a, setA] = useState(null);
4   const [b, setB] = useState(null);
5   useEffect(() => { if (!b) setB({ src: 'e1' }); }, [a]);
6   useEffect(() => { setA({ src: 'e2' }); }, [b]);
7   useEffect(() => { setB(null); }, [a]);
8   return <div/>;
9 }
```

```
MultiWriter (5 hooks) /tmp/ch15/multi_writer.tsx
warn infinite-loop [hook:2] (line 5:2) these effects may form a state-update cycle (`a` → `b` → `a`)
  ↳ where each step may store a fresh reference that re-runs the next effect: possible infinite render loop
    → a fresh value is written to state `b` here [hook:2] (line 5:28)
    → cycle continues: this effect freshly stores state `a` [hook:3] (line 6:20)
warn infinite-loop [hook:3] (line 6:2) these effects may form a state-update cycle (`a` → `b` → `a`) ...
  ↳
```

La garde !b meurt toujours sous sa propre écriture. Mais le troisième effet, lui aussi relancé par a, remet null dans b au tour automatique suivant : la garde *tient de nouveau*, le premier effet peut réécrire ..., et la boucle vit. La sonde le montre : trois lignes d'écriture, dont setB(null) avec fresh=Not value.null=true, et le graphe garde ses deux arêtes.

```
slot=0 setter=setA line=Some(6) region=Effect(3) phase=Effect block=Some(0) fresh=Fresh value.ref=PerRender
slot=1 setter=setB line=Some(5) region=Effect(2) phase=Effect block=Some(1) fresh=Fresh value.ref=PerRender
slot=1 setter=setB line=Some(7) region=Effect(4) phase=Effect block=Some(0) fresh=Not value.ref=Bottom value.null=true
=== churn graph
edge#0 (MultiWriter, 0) -> (MultiWriter, 1) strength=May carrier=MultiWriter effect=2 line=Some(5) no_deps=false
  ↳ self_slot=false
edge#1 (MultiWriter, 1) -> (MultiWriter, 0) strength=Must carrier=MultiWriter effect=3 line=Some(6) no_deps=false
  ↳ self_slot=false
cycle edges=[0, 1] all_must=false cross_component=false
```

L'écriture setB(null) n'est pas fraîche : elle ne porte *aucune* arête (le continue de

src/engine/churn.rs, l. 503–504). Elle compte pourtant : c'est un *reviver*, un site qui ranime une garde qu'une autre écriture avait éteinte. Une preuve de convergence qui l'ignorerait serait insound.

15.4.4 La réponse d'ADR-018 : un seul écrivain

Décision : ADR-18

Le graphe de churn naît le 16 juillet avec un kill de convergence *conditionnel* : la preuve mono-site est appliquée à une arête seulement si le slot écrit a **un unique site d'écriture d'effet programme-entier**, quelle que soit la fraîcheur des autres (une écriture stable ranime aussi une garde). Les handlers ne sont pas comptés (ils demandent un événement). Argument de soundness (ADR-18, point 2) : le kill est appliqué strictement *moins* souvent qu'une preuve naïve, et retirer un kill ne peut qu'ajouter des arêtes, jamais perdre un cycle réel. Prix : un slot à plusieurs écrivains garde toutes ses arêtes, au pire un WARNING faux positif (#39).

Le faux positif de #39 est réel et fréquent : deux effets qui chargent chacun *le même* slot derrière la *même* garde.

Exemple 15.7: Deux écrivains gardés qui s'éteignent l'un l'autre

Extrait 15.12 – /tmp/ch15/settle_each_other.tsx

```
1 import { useState, useEffect } from 'react';
2 export function SettleEachOther() {
3   const [s, setS] = useState(null);
4   const [b, setB] = useState(null);
5   useEffect(() => { if (!s) setS({ from: 'e1' }); }, [b]);
6   useEffect(() => { if (!s) setS({ from: 'e2' }); }, [b]);
7   useEffect(() => { setB({ from: s }); }, [s]);
8   return <div>{s ? 'y' : 'n'}</div>;
9 }
```

Quelle que soit l'écriture qui tire la première, *s* devient *truthy* et *les deux* gardes meurent. Sous ADR-18, le slot *s* a deux écrivains : aucun kill, deux arêtes $b \rightarrow s$, une arête $s \rightarrow b$, un cycle, deux WARNING. Au commit e67b10a, la sonde ne garde que l'arête $s \rightarrow b$ (aucun cycle) et la règle n'émet qu'un INFO sur l'écriture de *b* hors deps. C'est le programme du test `two_guarded_writers_of_one_slot_that_settle_each_others_guard_converge` de `tests/effect_cycles.rs`.

Résoudre #39 exige de poser la *bonne* question : non pas « combien d'écrivains ? » mais « sous *chaque* écriture du slot, la garde meurt-elle ? ». C'est la preuve multi-site de la section 15.6. Avant de l'écrire, il faut savoir de quelles écritures on parle.

15.5 Qu'est-ce qu'un site ?

15.5.1 Qui peut ranimer une garde ?

ADR-18 ne comptait que les *écritures d'effet*. Mais tout ce que la boucle automatique exécute peut ranimer une garde : une écriture pendant le rendu, une écriture dans un `useMemo`, et même l'effet de montage d'un *enfant* que la boucle démonte et remonte. Trois faux négatifs de #162.

Exemple 15.8: Une écriture au rendu ranime une garde d'effet**Extrait 15.13 – /tmp/ch15/render_write.tsx**

```

1 import { useState, useEffect } from 'react';
2 export function RenderWrite({ cond }: { cond: boolean }) {
3   const [s, setS] = useState(null);
4   if (cond && s) setS(null);
5   useEffect(() => { if (!s) setS({ fresh: true }); }, [s]);
6   return <div>{s ? 'y' : 'n'}</div>;
7 }

```

Avec `cond` vrai : ... $\rightarrow$ rendu $\rightarrow$ null $\rightarrow$ effet $\rightarrow$... $\rightarrow$... La sonde montre la ligne `region=Render phase=Render block=Some(3) fresh=Not value.null=true` à côté de la ligne d'effet, et l'arête `self_slot s  $\rightarrow$  s` May survit ; la règle émet un WARNING « may store a fresh reference into state 's' which its deps react to ». Avant e67b10a, la liste des sites n'était construite que sur les lignes d'effet : l'écriture du rendu ne ranimait rien et l'arête était tuée. Test : `a_render_phase_write_revives_an_effect_guard`.

Définition 15.2: Site de convergence

Un *site* (au sens du graphe de churn) est une ligne `slot_writers` d'un composant, de phase autre que `Handler`, prise avec le corps (CFG) dans lequel elle s'exécute :

- région `Render` : le CFG de rendu ;
- région `Effect(1)` d'un effet *non* `mount-only`, ou `Memo(1)` : le corps du hook `l` ;
- région `Effect(1)` d'un effet *mount-only* (deps d'arité exactement 0) : seulement si la ligne est *étrangère* (`owner = Some`) et que le propriétaire ne garde pas ce composant monté à travers la boucle (`stays_mounted` faux) ; le corps du hook `l` ;
- régions `Callback(_)` et `Handler(_)` : jamais (les écritures d'un callback sont des lignes du corps qui l'appelle, la marche les a réifiées).

L'ensemble `foreign` est celui des slots qualifiés qu'un site d'un *autre* composant écrit.

Ce « site » n'est pas le site d'écriture de l'IR (définition 6.2, un `ExprId`) : c'est une ligne de relation *située* dans un corps, l'unité sur laquelle la preuve de convergence raisonne. Le code qui les collecte suit la définition mot pour mot.

Extrait 15.14 – src/engine/churn.rs, l. 259–287 — la collecte des sites

```

259 for w in comp_result
260   .slot_writers
261   .iter()
262   .filter(|w| w.phase != WriterPhase::Handler)
263 {
264   let body = match w.region {
265     WriterRegion::Render => Some(cfg),
266     WriterRegion::Effect(1) if mount_only(1) => match w.owner {
267       // The component itself stays mounted and fires it once.
268       None => None,
269       // A child the loop mounts and unmounts fires it every
270       // round.
271       Some(owner) if stays_mounted(owner) => None,
272       Some(_) => hook(1).and_then(HookEntry::body_cfg),
273     },
274     WriterRegion::Effect(1) | WriterRegion::Memo(1) => {
275       hook(1).and_then(HookEntry::body_cfg)
276     }
277   // A callback's writes are rows of the body that calls it.
278   WriterRegion::Callback(_) | WriterRegion::Handler(_) => None,

```

```

279     };
280     if let Some(body) = body {
281         all_sites.push(SiteRef {
282             comp,
283             cfg: body,
284             row: w,
285         });
286     }
287 }

```

Deux filtres distincts jouent ici, et il faut les distinguer (chapitre 14, région $\neq$ phase) : le filtre sur la *phase* ($\neq$ Handler) écarte une écriture qu'un listener non réifié atteint, où qu'elle soit écrite ; le filtre sur la *région* écarte les lignes de région Callback/Handler (un `setTimeout` dans un `onClick` est de phase Deferred mais de région Handler : exclu par la région). Le `expect("every effect write is a site")` de `src/engine/churn.rs`, l. 517 est l'invariant qui en découle : toute écriture non-handler d'un effet non mount-only est dans `all_sites`.

15.5.2 L'effet de montage d'un enfant

Un effet à deps [] tourne une fois par *montage*. Pour le composant lui-même, qui reste monté pendant la boucle, il ne peut rien ranimer indéfiniment : ses lignes ne sont pas des sites (cas `None => None`). Mais un enfant que le parent rend *conditionnellement* est démonté et remonté à chaque tour de la boucle, et son effet [] tire à chaque tour.

Exemple 15.9: Un enfant qui remonte

Extrait 15.15 – /tmp/ch15/remount.tsx

```

1 import { useState, useEffect } from 'react';
2 function Child({ onReady }: { onReady: (v: any) => void }) {
3     useEffect(() => { onReady(null); }, []);
4     return <span />;
5 }
6 export function Parent() {
7     const [s, setS] = useState(null);
8     useEffect(() => { if (!s) setS({ fresh: true }); }, [s]);
9     return <div>{s ? <Child onReady={setS} /> : null}</div>;
10 }

```

```

Child (1 hooks) /tmp/ch15/remount.tsx ✓
Parent (2 hooks) /tmp/ch15/remount.tsx
  warn  infinite-loop [hook:0] (line 8:2) this effect may store a fresh reference into state `s` which its
        ↳ deps react to: possible infinite render loop
        → a fresh value is written to state `s` here [hook:1] (line 8:28)

```

`s` vide $\rightarrow$ le parent écrit ... $\rightarrow$ `<Child/>` monte $\rightarrow$ son effet écrit `null` dans le slot du parent $\rightarrow$ `<Child/>` démonte, `s` vide, et ainsi de suite. La ligne de `Child` est étrange (`owner=Some("Parent")`, `fresh=Not`); `stays_mounted(Parent)` échoue car l'élément `<Child/>` est sous la garde `s` — le ternaire JSX est abaissé en `Branch` et l'élément vit dans le bloc `IfTrue` (`lower_ternary`, `src/lowering/expr_lower.rs`, l. 675–724) — et `s`, un état, ne *tient* pas ; la ligne est un site, donc `(Parent, 0)` est dans `foreign` et la preuve pour `setS(...)` répond `false` : l'arête `self_slot` reste, le bras `self-churn` émet un `WARNING`. Test : `a_child_the_loop_remounts_fires_its_mount_effect_every_round`.

La fonction `stays_mounted` (`src/engine/churn.rs`, l. 374–430) répond « oui » seulement si le parent rend au moins un élément de l'enfant, et si *chaque* occurrence est hors de toute closure

(un `items.map(i => <Child/>)` monte un enfant par élément que la boucle peut ajouter), sous des gardes qui *tiennent* (`Invariance::holds`, section 15.6) et sans `key`, ou avec une `key` qui tient. Un parent qui ne rend aucun tel élément — l'enfant est atteint via un composant intermédiaire — *échoue fermé*. Deux variantes du test le montrent : `stays_mounted.tsx`, où `<Child/>` est derrière `if (!ready) return null` avec `ready` un prop, est silencieuse (« verified »); `keyed.tsx`, où `<Child key=s ? 'a' : 'b' />` est rendu sur tous les chemins mais avec une clé qui lit le slot, est un `WARNING` : React remonte l'enfant à chaque écriture.

Rappel React : Montage, key, effets []

React identifie un enfant par sa position dans l'arbre et sa `key`. Si l'élément disparaît d'un rendu (branche null) ou change de `key`, l'instance est démontée (ses `cleanups` tournent) et une nouvelle est montée au rendu suivant : tous ses effets, [] compris, s'exécutent de nouveau. Un effet « mount-only » ne l'est donc que pour une instance qui reste montée.

Piège

L'étiquette du diagnostic ci-dessus est `[hook:0]` alors que l'effet a le label 1 : le bras self-churn ancre son diagnostic sur le *slot* (`.with_label(state_label)`, `src/rules/impls/infinite_loop.rs`, l. 524) et seule l'étape `Write` du témoin porte le label de l'effet. Le bras graphe, lui, ancre sur l'effet porteur.

15.5.3 La vue du site pour la preuve : `WriteSite`

La preuve ne lit pas les lignes brutes mais une projection, `WriteSite`, construite par `site_of` (`src/engine/churn.rs`, l. 431–443).

Extrait 15.16 – `src/engine/guards.rs`, l. 92–121 — `WriteSite`, la vue d'un site pour la preuve

```

92 pub struct WriteSite<'a> {
93     /// The component whose body the write sits in.
94     pub component: ComponentId,
95     /// The body the write sits in.
96     pub cfg: &'a CFG,
97     /// The slot written, qualified by its owner.
98     pub slot: QualifiedSlot,
99     /// The block of `cfg` whose dominating guards the write runs under: its
100     /// own for a synchronous write, the statement that scheduled it for a
101     /// nested, deferred or repeating one – every guard on that chain held
102     /// when the write was scheduled. `None` for a row with no position in
103     /// `cfg`.
104     pub guard_block: Option<BlockId>,
105     /// Its block when it runs synchronously, once per pass – what makes it
106     /// co-execute with the sites it dominates.
107     pub block: Option<BlockId>,
108     /// The write runs at most once per run of the statement that schedules
109     /// it, or on a timer of its own – an interval's later ticks are external
110     /// events, not rounds of the loop. False for a callback an unresolved
111     /// callee may keep and call again (WriterPhase::Unknown`): its guards
112     /// say when it was handed out, not how often it runs.
113     pub bounded: bool,
114     /// Already proven to fire at most once in the automatic loop, so it
115     /// revives nothing indefinitely (#160).
116     pub convergent: bool,
117     /// The value the write leaves in the slot.
118     pub value: &'a StateValue,
119     /// Argument 0 as written.
120     pub expr: Option<&'a Expr>,
121 }
```

Champ par champ :

component, **cfig**, **slot** le composant dont le corps contient l'écriture, ce corps, et le slot *qualifié* écrit (node_of).

guard_block le bloc dont les gardes dominantes conditionnent l'écriture : son propre bloc pour une écriture synchrone, *l'instruction qui l'a planifiée* pour une écriture imbriquée, différée ou répétée (une continuation .then ne tourne que si la passe qui l'a planifiée a pris toutes les branches au-dessus, #160). C'est la colonne guard_block de la ligne (définition 14.2).

block le bloc de l'écriture quand elle est synchrone, une fois par passe : ce qui la fait *co-exécuter* avec les sites qu'elle domine. None sinon.

bounded phase != Unknown : l'écriture tourne au plus une fois par exécution de l'instruction qui la planifie, ou sur son propre timer (les ticks suivants d'un setInterval sont des événements externes, pas des tours de boucle). Un callback confié à un callee inconnu peut être appelé indéfiniment : ses gardes disent quand il a été *remis*, pas combien de fois il tourne.

convergent déjà prouvé « tire au plus une fois dans la boucle automatique » : il ne ranime rien indéfiniment (section 15.7).

value, **expr** la valeur abstraite laissée dans le slot et l'argument 0 tel qu'écrit (colonne written).

L'ensemble foreign (src/engine/churn.rs, l. 344–350) se déduit des sites : un slot qualifié dont le propriétaire n'est pas le composant du site. Un tel slot n'est *jamaïs* tué : les gardes de l'autre site vivent dans des corps que l'environnement de ce composant ne sait pas lire.

15.6 La preuve multi-site : converges_under_all_writes

15.6.1 La question, en une phrase

Un site *converge* s'il tire au plus une fois dans la boucle automatique — la suite de rendus que les seules écritures d'état entretiennent, sans événement utilisateur. Pour le prouver, ses gardes doivent mourir :

- (a) sous *l'ensemble tueur* : sa propre écriture et toute écriture synchrone d'un slot local située sur sa chaîne de gardes, car ces écritures s'exécutent à *chaque* passe où lui-même s'exécute ;
- (b) sous *chaque autre écriture vivante* des slots de cet ensemble, prise avec les *faits invariants* sous lesquels cet autre site a tiré.

La condition (a) généralise la preuve mono-site ; la condition (b) est ce qui manquait à ADR-18 : au lieu de compter les écrivains, on les *interroge* un par un.

Définition 15.3: Ensemble tueur, reviver vivant, faits invariants

Soit p un site de composant C , de bloc de garde g et de chaîne $\text{chain}(g)$ (les blocs à prédécesseur unique au-dessus de g , lui compris).

- L'*ensemble tueur* de p est la famille de réécritures $\{\ell \mapsto v\}$ formée de la propre écriture de p et de celle de tout site q du *même corps*, écrivant un slot *local* de C , dont **block** est dans $\text{chain}(g)$. Deux écritures du même slot donnent le join de leurs valeurs.
- Un *reviver vivant* d'un slot ℓ de l'ensemble tueur est un site $q \neq p$ de C , hors ensemble tueur, écrivant (C, ℓ) , et non déjà prouvé convergent.
- Les *faits* d'un site q sont les conjoints de ses gardes qui *tiennent* à travers la boucle (définition 15.4).

p *converge* si ses gardes sont mortes sous l'ensemble tueur et, pour chaque reviver vivant q d'un slot ℓ de l'ensemble tueur, mortes sous l'environnement rétréci par les faits de q dans lequel ℓ reçoit la valeur écrite par q et les autres slots de l'ensemble tueur gardent leur

valeur tueuse s'ils n'ont aucun reviver vivant, $\top$ sinon. Si un slot de l'ensemble tueur est dans `foreign`, p ne converge pas.

15.6.2 Première moitié : préconditions et ensemble tueur

Extrait 15.17 – `src/engine/guards.rs`, l. 137–197 — `converges_under_all_writes` : signature et première moitié

```

137 #[allow(clippy::too_many_arguments)]
138 pub fn converges_under_all_writes(
139     peers: &[WriteSite<'>],
140     i: usize,
141     own_value: &StateValue,
142     foreign: &HashSet<QualifiedSlot>,
143     state_vals: &HashMap<Var, HookLabel>,
144     invariance: &Invariance<'>,
145     props_hold: bool,
146     exit_env: &AbstractEnv<StateValue>,
147     eval: &mut dyn FnMut(&Expr) -> StateValue,
148 ) -> bool {
149     let site = &peers[i];
150     let Some(guard_block) = site.guard_block else {
151         return false;
152     };
153     if !site.bounded || site.slot.0 != site.component {
154         return false;
155     }
156     let chain = guard_chain(site.cfg, guard_block);
157     let guards = site.guards(site.cfg, guard_block);
158     if guards.is_empty() {
159         return false;
160     }
161     // The kill set: this write, and every synchronous write of a local slot
162     // in a block of the chain – those run on every pass this site runs on.
163     // A slot written twice on the chain holds the join.
164     let co: Vec<usize> = (0..peers.len())
165         .filter(|&j| j != i)
166         .filter(|&j| {
167             let p = &peers[j];
168             std::ptr::eq(p.cfg, site.cfg)
169                 && p.slot.0 == site.component
170                 && p.block.is_some_and(|b| chain.contains(&b))
171         })
172         .collect();
173     let mut kill: Vec<Rewrite> = vec![Rewrite {
174         label: site.slot.1,
175         value: own_value.clone(),
176         expr: site.expr,
177         scope: Some(site.cfg),
178     }];
179     for &j in &co {
180         let p = &peers[j];
181         match kill.iter_mut().find(|r| r.label == p.slot.1) {
182             Some(r) => {
183                 r.value = r.value.join(p.value);
184                 r.expr = None;
185             }
186             None => kill.push(Rewrite {
187                 label: p.slot.1,
188                 value: p.value.clone(),

```



```

189         expr: p.expr,
190         scope: Some(site.cfg),
191     )),
192     }
193 }
194 let mine = let_bindings(site.cfg);
195 if !dead_once_written(&guards, Some(&mine), state_vals, &kill, exit_env, eval) {
196     return false;
197 }

```

La signature dit déjà tout le contrat : le site est désigné par son indice i dans `peers` (les sites d'un composant), sa valeur propre `own_value` est fournie par l'appelant, et `foreign`, `invariance`, `props_hold` sont les faits de contexte calculés une fois par `build_edges`. Les préconditions *échouent fermées* : pas de `guard_block` (ligne sans position plaçable), site non borné (`Unknown` : un callee inconnu peut rappeler le callback), slot étranger (`site.slot.0 != site.component`), aucune garde — dans les quatre cas la réponse est `false` et l'arête reste. Puis :

- `co` : les pairs du même corps (`ptr::eq(p.cfg, site.cfg)`), d'un slot local, dont `block` est sur la chaîne. Ils *co-exécutent* avec le site : toute passe qui atteint g traverse leur bloc.
- `kill` : la réécriture propre (avec `own_value`, la valeur que *l'appelant* attribue au site, voir section 15.7) puis celles des `co` ; un slot écrit deux fois sur la chaîne prend le join et perd son `expr` (le bras relationnel n'a plus d'orthographe unique à comparer).
- `mine = let_bindings(site.cfg)` : les noms que le corps lie. Sous l'environnement de sortie du rendu, ils lisent $\top$ (`shadowed`, #162) : le rendu peut lier le même nom — un temporaire `__t0` surtout — à autre chose, et une garde n'est jamais morte sur un nom qu'elle ne teste pas.

Le premier `dead_once_written` (chapitre 14, trois bras : valeur, relationnel, membre) répond à la condition (a). S'il échoue, inutile d'aller plus loin.

15.6.3 Seconde moitié : les revivers

Extrait 15.18 – `src/engine/guards.rs`, l. 198–258 — `converges_under_all_writes`, seconde moitié

```

198 let held: Vec<(&Expr, bool)> = guards
199     .iter()
200     .copied()
201     .filter(|(c, _)| invariance.holds(c, &mine, props_hold))
202     .collect();
203 // The other sites of a kill-set slot that may still fire: neither this
204 // site, nor one of its co-executing writes, nor one already proven
205 // convergent.
206 let live = |label: HookLabel| -> Vec<&WriteSite<'_>> {
207     (0..peers.len())
208         .filter(|&j| j != i && !co.contains(&j))
209         .map(|j| &peers[j])
210         .filter(|p| p.slot == (site.component, label) && !p.convergent)
211         .collect()
212 };
213 for k in &kill {
214     if foreign.contains(&(site.component, k.label)) {
215         return false;
216     }
217     for other in live(k.label) {
218         let same = std::ptr::eq(other.cfg, site.cfg);
219         let theirs = let_bindings(other.cfg);
220         let facts: Vec<(&Expr, bool)> = other
221             .guard_block

```

```

222         .map(|b| site_guardes(other.cfg, b))
223         .unwrap_or_default()
224         .into_iter()
225         .filter(|(c, _)| invariance.holds(c, &theirs, props_hold))
226         // Across bodies a name means the same thing only when neither
227         // body binds it: both then read the closure.
228         .filter(|(c, _)| same || names_unbound(c, &mine, &theirs))
229         .collect();
230     if contradicts(&held, &facts) {
231         continue;
232     }
233     let mut env = exit_env.clone();
234     for &(c, t) in &facts {
235         env = narrow_env_for_branch(&env, c, t);
236     }
237     // After the other site's write the slot holds its value. Every
238     // other slot of the kill set keeps its kill value when no live
239     // site can have moved it — a convergent site moves it finitely
240     // often, and each time this site's own run restores it — and
241     // reads  $\top$  otherwise.
242     let mut rewrites: Vec<Rewrite> = kill
243         .iter()
244         .filter(|r| r.label != k.label && live(r.label).is_empty())
245         .cloned()
246         .collect();
247     rewrites.push(Rewrite {
248         label: k.label,
249         value: other.value.clone(),
250         expr: other.expr,
251         scope: same.then_some(site.cfg),
252     });
253     if !dead_once_written(&guards, Some(&mine), state_vals, &rewrites, &env, eval) {
254         return false;
255     }
256 }
257 }
258 true

```

- `held` : les gardes du site qui *tiennent* (`Invariance::holds`) ; ce sont les faits sous lesquels *ce* site tire.
- `live(label)` : les revivers vivants du slot `label` — ni le site, ni un `co`, du même slot qualifié, non convergents.
- Pour chaque slot k de l'ensemble tueur : s'il est `foreign`, `false`. Pour chaque reviver `other` :
 1. ses faits : ses gardes qui tiennent, et, si les deux corps diffèrent, dont *aucun* nom n'est lié dans l'un ou l'autre corps (`names_unbound`) : un nom ne signifie la même chose dans deux corps que si tous deux le lisent dans la closure ;
 2. `contradicts(held, facts)` : un même conjoint (clé `call_free_key`) avec des polarités opposées — l'autre site a tiré sous `!p`, `p` tient, donc le nôtre (sous `p`) ne tire jamais à côté de lui : ce reviver est *sans objet*, on passe au suivant ;
 3. sinon on rétrécit l'environnement de sortie par les faits de `other`, on réécrit : les slots de l'ensemble tueur *autres* que k gardent leur valeur tueuse *s'ils n'ont aucun reviver vivant* (sinon ils lisent $\top$, par absence de réécriture), et k reçoit la valeur de `other` — son `expr` n'étant lisible par le bras relationnel que si les corps sont les mêmes (`scope`) ;
 4. si les gardes ne sont pas mortes dans cet environnement, `false`.

```

264 fn contradicts(held: &[(&Expr, bool)], facts: &[(&Expr, bool)]) -> bool {
265     held.iter().any(|(c, t)| {
266         let Some(k) = call_free_key(c) else {
267             return false;
268         };
269         facts
270             .iter()
271             .any(|(f, u)| t != u && call_free_key(f).as_deref() == Some(k.as_str()))
272     })
273 }

```

Piège

Le test de contradiction n'est sûr que parce qu'il porte sur des faits *invariants*. Deux gardes *s* et *!s* sur un état ne se contredisent pas d'un tour à l'autre : c'est justement *s* qui bouge. *held* et *facts* sont filtrés par `Invariance::holds` *avant* `contradicts`, et un état n'y entre jamais.

15.6.4 Ce qui tient à travers la boucle : Invariance

Extrait 15.20 – `src/engine/guards.rs`, l. 305–315 — les faits d'invariance d'un composant

```

305 pub struct Invariance<'a> {
306     /// The render's bindings ([`let_bindings`]).
307     pub render: &'a HashMap<&'a str, Option<&'a Expr>>,
308     pub state_vals: &'a HashMap<Var, HookLabel>,
309     pub memo_vals: &'a HashMap<Var, HookLabel>,
310     /// The names some body of the component writes or mutates.
311     pub mutated: &'a HashSet<Var>,
312     /// Some effect, memo or callback body of the program visibly navigates
313     /// ([`navigates`]), so a navigation-held value may move inside the loop.
314     pub navigates: bool,
315 }

```

Définition 15.4: Invariance à travers la boucle automatique

La boucle re-rend sur les seules écritures d'état. *Bouge* d'un tour à l'autre : un état, ce qui en dérive (un memo, un callback, le résultat d'un hook), une allocation fraîche, et tout nom qu'un corps du composant mute. *Tient* : un littéral; un nom lié une fois (dans le corps ou dans le rendu) à quelque chose qui tient; un nom que rien ne lie — un prop, un nom de module — *tant que les props tiennent* (`props_hold`); un appel sur des entrées qui tiennent; un résultat de hook dont le résumé dit qu'il ne bouge que sur navigation (`SummaryValue::Held`), sauf si un corps du programme navigue visiblement (`navigates`); un membre de littéral objet quand le membre tient. Profondeur bornée à 8, au-delà : ne tient pas.

La fonction `Invariance::holds` (`src/engine/guards.rs`, l. 321–390) suit cette définition cas par cas : `Lit` tient; `Var` ne tient pas s'il est dans `state_vals`, `memo_vals` ou `mutated`, sinon on suit sa liaison (binding, corps puis rendu), et un nom que rien ne lie répond `props_hold`; `FieldAccess/IndexAccess` sont résolus à travers un littéral objet ou un résultat de hook « *shaped* »; `BinOp`, `UnaryOp`, `Call` tiennent si leurs entrées tiennent; tout le reste (allocation, élément, autre hook) bouge. `mutated` est l'union des racines mutées de tous les corps du composant (`mutated_roots`) et des noms de module écrits par *n'importe quel* composant (`src/engine/churn.rs`, l. 202–224). `navigates` est calculé une fois, programme-entier, sur les rendus et tous les corps de hook *sauf* les handlers (`src/engine/churn.rs`, l. 206–216) : une navigation depuis un handler exige un événement utilisateur, elle n'est pas dans la boucle.

Piège

« Un appel sur des entrées qui tiennent tient » (`src/engine/guards.rs`, l. 294) est une *hypothèse*, pas une preuve : un callee opaque peut lire une horloge ou naviguer. Elle est documentée dans `docs/limitations.md` et portée par #161. C'est le seul point de la preuve où la soundness repose sur une hypothèse assumée plutôt que sur une sur-approximation (section 15.10).

15.6.5 Retour sur les exemples

multi_writer.tsx (exemple 15.6). Site `setB(...)` de l'effet 2 : gardes $[(b, \text{false})]$, aucun `co` (le corps n'a qu'une écriture), `kill` = $\{b \mapsto \text{réf. PerRender}\}$: morte, condition (a) satisfaite. `held` = $\square$ (`b` est un état). `live(b)` = $\{\text{setB}(\text{null})\}$ de l'effet 4 : non gardé, `facts` = $\square$; `rewrites` = $\{b \mapsto \text{null}\}$; la garde « `b falsy` » sur une valeur `null` n'est pas $\perp$: `false`. L'arête $a \rightarrow b$ reste, le cycle est rapporté.

null_updater.tsx (test a_null_returning_updater_at_another_site_revives_the_guard). Même schéma avec `if (s) setS(prev => null)` pour reviver : la sonde montre `updater=Functional` `fresh=Not` `value.null=true`. Un `updater` fonctionnel est lu comme le *join de ses retours* (chapitre 14, `written::classify`), ici `null` : la garde `!s` revit, l'arête `self_slot` reste, `WARNING`.

settle_each_other.tsx (exemple 15.7). Site `setS` de l'effet 2 : (a) morte sous $s \mapsto \text{réf. live}(s)$ = $\{\text{setS de l'effet 3}\}$: sa garde `!s` ne tient pas (état), `facts` = $\square$; réécriture $s \mapsto \text{réf. PerRender}$ de l'autre site : « `s falsy` » donne $\perp$, morte. `true`. Symétriquement pour l'effet 3. Les deux arêtes $b \rightarrow s$ sont tuées; seule $s \rightarrow b$ subsiste, sans cycle.

15.6.6 Ce que la preuve garantit**Lemme 15.2: Soundness du kill**

Supposons les hypothèses d'invariance de la définition 15.4 vraies du programme concret, et supposons que tout site marqué convergent parmi les pairs tire un nombre fini de fois dans la boucle automatique. Si `converges_under_all_writes` répond `true` pour un site p , alors p tire au plus une fois, puis un nombre fini de fois supplémentaires au plus égal au nombre de tirs des sites convergents, dans toute boucle automatique.

Esquisse. Les gardes de p sont conjonctives : il suffit qu'une soit fausse pour que p ne tire pas. Considérons un tour t après le premier tir de p et regardons, pour chaque slot ℓ de l'ensemble tueur, la dernière écriture de ℓ avant t . Trois cas. (i) C'est p lui-même ou un `co` : ℓ tient sa valeur tueuse, et les gardes sont mortes par (a). (ii) C'est un `reviver` vivant q : q a tiré, donc ses faits invariants étaient vrais lors de ce tir, et tiennent encore en t ; si l'un contredit un fait de p , p ne tire pas; sinon l'environnement de t est inclus dans celui que la preuve a construit pour q (faits de q , valeur de q dans ℓ , valeur tueuse ou $\top$ ailleurs), où les gardes sont mortes. (iii) C'est un site convergent : cela n'arrive qu'un nombre fini de fois, et après chaque tel tir un tir de p restaure la valeur tueuse. Un site étranger n'écrit jamais un slot tueur (`foreign exclu`). Donc p ne tire, hors les cas (iii), plus jamais. $\square$

Le lemme est conditionnel, et c'est voulu : la seconde hypothèse est *discharged* par le plus petit point fixe de la section suivante, la première reste une hypothèse du projet.

15.7 Revivers et plus petit point fixe des sites convergents

15.7.1 Un reviver qui tire une fois par requête

La preuve de la section précédente traite un reviver *vivant* comme éternel : s'il peut ranimer la garde une fois, il pourra la ranimer toujours. C'est faux pour un reviver qui, lui aussi, converge. Le motif vient du dépôt twenty (#160).

Exemple 15.10: Un drapeau de requête et sa continuation

Extrait 15.21 – /tmp/ch15/reviver.tsx

```
1 import { useState, useEffect } from 'react';
2 declare function refetch(): Promise<void>;
3 export function Reviver({ version }: { version: { id: string } }) {
4   const [req, setReq] = useState(false);
5   const [seeded, setSeeded] = useState<string | undefined>(undefined);
6   useEffect(() => {
7     if (!req) return;
8     setReq(false);
9     void refetch().then(() => { setSeeded(undefined); });
10  }, [req]);
11  useEffect(() => {
12    if (seeded === version.id) return;
13    setSeeded(version.id);
14  }, [version, seeded]);
15  return <div>{seeded}</div>;
16 }
```

Reviver (4 hooks) /tmp/ch15/reviver.tsx ✓
verified infinite-loop no effect diverges into an infinite render loop

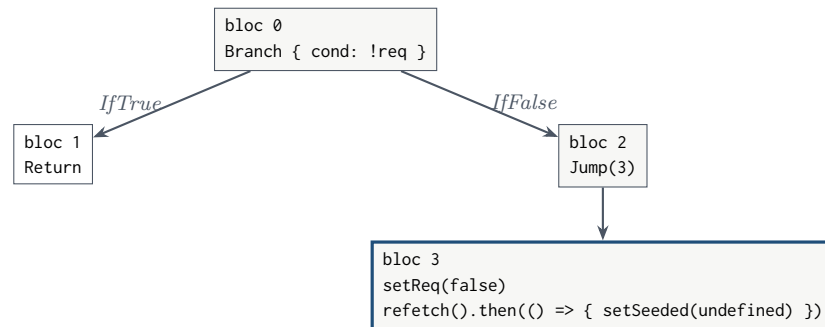
Test : a_reviver_that_resets_its_own_flag_fires_once_per_request.

Le second effet s'éteint *relationnellement* : la garde compare `seeded` à `version.id`, l'écriture pose `version.id` dans `seeded`, les deux côtés sont égaux au rendu suivant. Mais la continuation `.then` du premier effet remet `undefined` dans `seeded` : un reviver. Il n'est pourtant planifié que si `req` est vrai, et la *même passe* remet `req` à `false` : il tire une fois par requête externe, la paire converge en deux tours. Pour le voir, il faut trois choses.

```
slot=0 setter=setReq line=Some(8) region=Effect(2) phase=Effect block=Some(3) guard_block=Some(3) fresh=Not
slot=1 setter=setSeeded line=Some(9) region=Effect(2) phase=Deferred block=None guard_block=Some(3) fresh=Not
slot=1 setter=setSeeded line=Some(13) region=Effect(3) phase=Effect block=Some(3) guard_block=Some(3) fresh=Maybe
  ↳ value.ref=Unknown
-- effect_triggers
hook=2 dep=0 slot=(Reviver, 0) exact=true
hook=3 dep=1 slot=(Reviver, 1) exact=true
-- registrations
effect=2 display=.then registrar=then firing=Once timing=Deferred block_id=Some(3) line=Some(9) pairing=Unpaired
=== churn graph
```

1. **Une écriture différée a des gardes.** La ligne `setSeeded(undefined)` est `Deferred` (`block=None`) mais porte `guard_block=Some(3)` : le bloc de l'instruction `refetch().then(...)` qui l'a planifiée. La continuation ne tourne que si la passe qui l'a planifiée a pris toutes les branches au-dessus ; ses gardes sont donc celles du bloc 3 : `guard_chain(3) = [3, 2, 0]`, `site_guards = [(req, true)]` (la figure 15.2).
2. **L'ensemble tueur contient `setReq(false)`.** Cette écriture est synchrone, locale, dans le bloc 3 qui est sur la chaîne : c'est un `co`. Sous `req ↦ false`, la garde `(req, true)` est $\perp$: le reviver *converge*.

3. **Un reviver convergent n'est pas vivant.** Une fois `setSeeded(undefined)` prouvé, il sort de `live(seeded)` pour le site `setSeeded(version.id)`, qui converge à son tour.



`guard_chain(cfg, 3) = [3, 2, 0]` (blocs grisés);
`site_guards(cfg, 3) = [(req, true)]` : l'arête $0 \rightarrow 2$ est `IfFalse` sur `!req`, dépliée en `req` vrai;
 ensemble tueur du site `setSeeded(undefined)` : $\{\text{seeded} \mapsto \text{undefined}, \text{req} \mapsto \text{false}\}$ —
`setReq(false)` est un co (même corps, slot local, block = `Some(3)` sur la chaîne).

FIGURE 15.2 – Le corps du premier effet de l'exemple 15.10 après lowering (blocs de la sonde IR), la chaîne de gardes du bloc 3 et l'ensemble tueur de l'écriture différée. Le site différé n'a pas de block mais hérite du `guard_block` de l'instruction qui le planifie.

15.7.2 Le point fixe des sites convergents

Extrait 15.22 – `src/engine/churn.rs`, l. 450–485 — le plus petit point fixe des sites convergents

```

450 let mut convergent = vec![false; all_sites.len()];
451 loop {
452     let mut changed = false;
453     for (comp, idxs) in &by_comp {
454         let ctx = &ctxs[comp];
455         let invariance = invariance_of(ctx);
456         let mut evaluator = result.components[comp].evaluator();
457         let mut eval = |e: &Expr| evaluator.at(&ctx.exit, e);
458         let peers: Vec<WriteSite> = idxs
459             .iter()
460             .map(|&j| site_of(&all_sites[j], convergent[j]))
461             .collect();
462         for (k, &j) in idxs.iter().enumerate() {
463             if convergent[j] || all_sites[j].row.owner.is_some() {
464                 continue;
465             }
466             if converges_under_all_writes(
467                 &peers,
468                 k,
469                 peers[k].value,
470                 &foreign,
471                 &ctx.state_vals,
472                 &invariance,
473                 ctx.props_hold,
474                 &ctx.exit,
475                 &mut eval,
476             ) {
477                 convergent[j] = true;
478                 changed = true;
479             }
480         }
481     }
482     if !changed { break; }
483 }
  
```

```

480     }
481   }
482   if !changed {
483     break;
484   }
485 }

```

Le vecteur convergent part de « aucun ». À chaque tour, pour chaque composant, on projette les sites en *peers avec le drapeau courant*, puis on tente la preuve sur chaque site local non encore prouvé, avec sa *valeur entière* (`peers[k].value`). On s'arrête quand un tour ne marque plus rien. Les lignes étrangères (`owner.is_some()`) ne sont jamais tentées.

Théorème 15.1: Le plus petit point fixe des sites convergents

Soit S l'ensemble des sites d'un programme et, pour $X \subseteq S$, $F(X)$ l'ensemble des sites p tels que `converges_under_all_writes` répond `true` quand les pairs marqués convergents sont exactement ceux de X . Alors F est monotone ($X \subseteq Y \Rightarrow F(X) \subseteq F(Y)$), l'itération $X_0 = \emptyset$, $X_{n+1} = X_n \cup F(X_n)$ stationne en au plus $|S| + 1$ tours sur $\text{lfp}(X \mapsto X \cup F(X))$, et le résultat ne dépend pas de l'ordre de visite des composants ni des sites.

Démonstration. Marquer un site convergent ne fait que *retirer* des éléments de `live(label)` pour les autres : moins de revivers à réfuter, chaque condition de la preuve est monotone décroissante en `live`, donc F est monotone. La suite X_n est croissante dans un ensemble fini, d'où la terminaison ; un tour utile marque au moins un site, d'où la borne. L'indépendance de l'ordre est celle de tout plus petit point fixe d'un opérateur monotone sur un treillis fini (définition 3.5) ; le code le note à `src/engine/churn.rs`, l. 446–449. $\square$

Remarque

L'itération est *à la Jacobi* : `peers` est construit une fois au début de chaque tour à partir de convergent, avant la boucle interne. Un site prouvé au tour n n'est vu convergent par ses pairs qu'au tour $n + 1$. Le résultat n'en dépend pas (théorème), mais un déroulé à la main doit compter les tours ainsi.

Détaillons la case $\times$ du tour 1, la seule subtile. Le site `setSeeded(version.id)` satisfait (a) par le bras relationnel : la garde est `BinOp Eq, seeded, version.id` prise fausse, l'écriture pose *verbatim* `version.id` dans `seeded`, ni l'un ni l'autre n'est une orthographe fraîche (`version.id` est un membre de `prop`) : après l'écriture l'égalité est vraie, la branche fausse est morte. Pour (b), `live(seeded)` contient `setSeeded(undefined)` tant qu'il n'est pas marqué. Ses faits : (`req`, $\top$), mais `req` est un état, il ne tient pas, `facts = []`. On réécrit `seeded` $\mapsto$ `undefined` avec `scope = None` (autre corps) : le bras relationnel est inapplicable, le bras membre aussi (le chemin est le slot nu), et le bras valeur rétrécit `seeded == version.id` — dont le côté droit n'est pas un littéral : l'environnement est inchangé, rien n'est $\perp$. Échec. Au tour 2, le snapshot marque le reviver convergent : `live(seeded)` est vide, la boucle sur les revivers ne tourne pas, `true`.

15.7.3 Pourquoi pas le plus grand point fixe

On pourrait partir de « tous convergents » et retirer les sites que la preuve réfute. Ce plus grand point fixe est *insound*, et le contre-exemple est court.

Exemple 15.11: Deux écritures qui se raniment l'une l'autre

Extrait 15.23 – `/tmp/ch15/null_updater.tsx`

Site	Gardes	Tour 1	Tour 2
setReq(false) (effet 2, bloc 3)	(req, $\top$)	✓ (a) : req $\mapsto$ false tue; aucun reviver de req	—
setSeeded(undefined) (effet 2, .then)	(req, $\top$)	✓ (a) : co = {setReq} tue; (b) : le reviver setSeeded(version.id) réécrit seeded, req garde false	—
setSeeded(version.id) (effet 3, bloc 3)	(seeded $\equiv$ version.id, $\perp$)	× (a) tient; (b) : le reviver setSeeded(undefined) est encore vivant dans le snapshot	✓ live(seeded) = $\emptyset$
changed		true	true; tour 3 : false, sortie

TABLE 15.2 – Les tours du point fixe des sites sur l'exemple 15.10. $(c, \top)$ note une garde c prise vraie, $(c, \perp)$ prise fausse; (a) et (b) renvoient aux deux conditions de la définition 15.3. Le déroulé a été observé en instrumentant une copie de ChurnGraph::build (une trace par site et par tour) : les deux premiers sites sont marqués au tour 1, le troisième au tour 2, et le tour 3 ne change rien. Sur `reviver_unreset.tsx`, le même instrument montre un seul tour, sans aucun site marqué.

```

1 import { useState, useEffect } from 'react';
2 export function NullUpdater() {
3   const [s, setS] = useState(null);
4   useEffect(() => { if (!s) setS({ fresh: true }); }, [s]);
5   useEffect(() => { if (s) setS(prev => null); }, [s]);
6   return <div>{s ? 'y' : 'n'}</div>;
7 }

```

```

NullUpdater (3 hooks) null_updater.tsx
  warn infinite-loop [hook:0] (line 4:2) this effect may store a fresh reference into state `s` which its
    ↳ deps react to: possible infinite render loop
      ↳ a fresh value is written to state `s` here [hook:1] (line 4:28)

```

Appelons A le site `setS(...)` (garde `!s`) et B le site `setS(prev => null)` (garde `s`). Chacun meurt sous sa propre écriture : A pose une référence *truthy*, B pose `null`. Chacun est ranimé par l'autre : B remet `s` *falsy*, A le remet *truthy*. Si l'on suppose B *convergent*, A n'a plus de reviver vivant et est prouvé; si l'on suppose A *convergent*, B est prouvé. Le plus grand point fixe contient donc $\{A, B\}$: les deux arêtes seraient tuées et le programme — qui boucle $\dots \rightarrow \text{null} \rightarrow \dots \rightarrow \dots$ dès le montage — serait silencieux. Le plus petit point fixe ne marque ni l'un ni l'autre : au tour 1, chacun voit l'autre vivant et échoue; rien ne change; sortie. L'arête `self_slot s` $\rightarrow s$ reste et le bras `self-churn` émet le WARNING observé.

Théorème 15.2: Soundness du plus petit point fixe

Sous les hypothèses d'invariance, tout site de `lfp` tire un nombre fini de fois dans toute boucle automatique. Le plus grand point fixe de F n'a pas cette propriété.

Démonstration. Par récurrence sur le tour n où un site p entre dans X_n . Les pairs que la preuve de p tient pour convergents sont dans X_{n-1} et, par hypothèse de récurrence, tirent un nombre fini de fois; le lemme 15.2 conclut pour p . Pour la seconde partie, l'exemple 15.11 exhibe $\{A, B\} \subseteq \text{gfp}$ avec une boucle infinie. $\square$

Décision : ADR-42 §6, amendement #160

Le point fixe *minimal* est un choix argumenté en une phrase dans le code même : « a greatest fixpoint would read two sites that revive each other as convergent, which is exactly a loop » (src/engine/guards.rs, l. 41–42). L’alternative — une colonne *settles* sur la ligne, calculée une fois — avait déjà été refusée à la tranche 4 : deux appelants demandent deux valeurs (voir ci-dessous), et désormais la réponse dépend aussi de l’état des autres sites, ce qu’une colonne ne peut porter. Mesure du commit e67b10a (docs/precision-log.md) : 1 499 → 1 498 sur le corpus, trois lignes retirées et deux ajoutées, dont deux sont une même ligne reformulée.

Le programme frère `reviver_unreset.tsx` (test `a_reviver_that_keeps_its_flag_still_revives`), où `setReq(false)` est retiré et `req` vaut `true` pour toujours, donne un `WARNING` sur `seeded` : le `reviver` différé n’a plus de `co` qui tue sa garde (`req, ⊤`), il reste vivant, et `setSeeded(version.id)` n’est jamais prouvé. La preuve ne lit pas une garde comme mourant sous une écriture qui n’a pas lieu.

15.7.4 Le kill des arêtes : la partie référence

Le point fixe des sites fait, l’étape des arêtes réutilise la même preuve, mais avec une autre valeur propre.

Extrait 15.24 – src/engine/churn.rs, l. 502–529 — le kill d’une arête

```

502     for w in &f.writes {
503         if w.written.fresh == Freshness::Not {
504             continue;
505         }
506         let node = node_of(f.comp, w);
507         // Convergence proof: once a written value sits in the slot, do the
508         // dominating guards kill this write – under its own write and
509         // under every other site's? Edges claim reference churn, so the
510         // own write is read as the reference part of its value
511         // (references are truthy and non-nullish); another site's write
512         // revives with whatever it stores, `null` included.
513         let own_value = reference_part(&w.written.value);
514         let k = idxs
515             .iter()
516             .position(|&j| std::ptr::eq(all_sites[j].row, *w))
517             .expect("every effect write is a site");
518         let killed = convergent[idxs[k]]
519             || converges_under_all_writes(
520                 &peers,
521                 k,
522                 &own_value,
523                 &foreign,
524                 &ctx.state_vals,
525                 &invariance,
526                 ctx.props_hold,
527                 &ctx.exit,
528                 &mut eval,
529             );

```

Extrait 15.25 – src/engine/written.rs, l. 202–209 — reference_part

```

202 /// Projection of a written value onto its reference slot – what a
203 /// reference-churn loop can actually carry across renders. Every primitive
204 /// part (which cannot fail `Object.is` freshly) is dropped, so guard proofs
205 /// don't lose to residual  $\top$  noise. A  $\perp$  reference slot yields  $\perp$ : no

```

```

206 /// reference can ever be stored → the claimed reference churn is vacuous.
207 pub fn reference_part(written: &StateValue) -> StateValue {
208     StateValue::reference(written.reference.clone())
209 }

```

Une arête affirme un churn de *référence* : ce qu'elle transporte d'un rendu à l'autre est la partie référence de la valeur écrite. Le site est donc lu, pour sa propre écriture, comme `reference_part(w.written.value)` — les parties primitives, qui ne peuvent pas échouer `Object.is` fraîchement, sont abandonnées, ce qui évite de perdre la preuve à un $\top$ résiduel ; une partie référence $\perp$ donne $\perp$ (aucune référence ne peut être stockée, le churn prétendu est vide). Les autres sites, eux, raniment avec tout ce qu'ils stockent, null compris : dans la preuve, `other.value` est la valeur entière. Le point fixe des sites, à l'inverse, lit la valeur entière du site propre (`peers[k].value`) : il répond à la question « ce site tire-t-il une fois ? » et non « cette arête est-elle un churn ? ». Deux appelants, deux valeurs : c'est l'argument d'ADR-42 §6 (tranche 4) pour une fonction plutôt qu'une colonne.

Piège

Le `killed` de l'liste 15.24 est une disjonction : `convergent[idxs[k]] || converges_under_all_writes(..., &own_value, ...)`. Un site que le point fixe n'a pas marqué (parce que sa valeur *entière*, null compris, ne tue pas ses gardes) peut encore voir son *arête* tuée, parce que la partie référence de sa valeur, elle, les tue. Le premier disjoint, lui, est un raccourci : un site déjà prouvé convergent n'a pas besoin d'une seconde preuve, puisque ne pas tirer implique ne pas cherner. Ne pas confondre les deux questions en lisant une trace : « site non convergent » n'implique pas « arête présente ».

15.8 Plusieurs composants : lignes étrangères, `props_hold`, plafond Warning

15.8.1 Un setter passé en prop

Exemple 15.12: Un enfant qui réécrit l'état de son parent

Extrait 15.26 – /tmp/ch15/cross.tsx

```

1 import { useState, useEffect } from 'react';
2 export function Parent() {
3     const [data, setData] = useState({ n: 0 });
4     return <Child value={data} onUpdate={setData} />;
5 }
6 function Child({ value, onUpdate }) {
7     useEffect(() => { onUpdate({ n: value.n, seen: true }); }, [value]);
8     return <div/>;
9 }

```

```

Child (1 hooks) /tmp/ch15/cross.tsx
  warn cross-component-infinite-loop [hook:0] (line 7:2) this effect calls `onUpdate`, a state setter of
    ↳ parent `Parent` (its deps do not provably gate it, so the effect can re-run every render). Parent re-
      ↳ renders → child re-renders → effect fires again: infinite loop
Parent (1 hooks) /tmp/ch15/cross.tsx ✓

=== component Child (ComponentId(0))
  slot=0 owner=Some("Parent") setter=onUpdate line=Some(7) region=Effect(0) phase=Effect block=Some(0) fresh=Fresh
    ↳ value.ref=PerRender
  -- effect_triggers

```

```
hook=0 dep=0 slot=(Parent, 0) exact=false
=== churn graph
edge#0 (Parent, 0) -> (Parent, 0) strength=May carrier=Child effect=0 line=Some(7) no_deps=false self_slot=false
cycle edges=[0] all_must=false cross_component=true
```

Dans l'environnement de Child, `onUpdate` vaut `ComponentSetter(Parent, 0)` (passe descendante, chapitre 16) : la ligne d'écriture est *étrangère*, `owner=Some("Parent")`, et `node_of` la place sur le nœud `(Parent, 0)`. Le `dep` value est un prop dont la valeur est `Versioned((Parent, 0))` : le déclencheur est `exact=false`. Trois conséquences s'enchaînent :

1. l'arête est May (`versioned, src/engine/churn.rs, l. 592`);
2. elle n'est pas `self_slot` : la condition `x == node && x.0 == f.comp (src/engine/churn.rs, l. 589)` échoue car le slot appartient à Parent et l'effet à Child; elle entre donc dans la recherche de cycles, où elle forme un cycle de longueur 1;
3. `make_cycle` compte deux composants (propriétaire du nœud et porteur) : `cross_component = true`.

Décision : ADR-18, plafond Warning

Un cycle qui traverse deux composants est plafonné à WARNING et nommé `cross-component-infinite-loop`. La raison est structurelle : le `must-rerun` à travers un prop est *indémontrable* au commit e67b10a, parce qu'un `dep` qui est un prop est `Versioned`, jamais le slot exact — le composant enfant ne voit pas *quel* champ du parent a bougé. ADR-18 note la voie d'amélioration (une provenance de prop entier au site d'appel JSX) sans la prendre.

Le diagnostic observé ci-dessus n'est pourtant pas celui du bras graphe : c'est le *bras cross-component historique* de `infinite-loop` qui parle (« this effect calls 'onUpdate', a state setter of parent »). L'effet étant déjà dans `reported_effects`, le bras graphe le saute (`src/rules/impls/infinite_loop.rs, l. 277–279`) : un rapport par effet et par classe de panne.

15.8.2 Quand les props bougent : `props_hold`

La preuve de convergence lit des faits sur les props (« p tient ») : un prop n'est pas un état du composant. Mais un prop *est* un état du parent, et si la boucle passe par le parent, il bouge.

Exemple 15.13: Un cycle qui entre par les props

Extrait 15.27 – `/tmp/ch15/props_move.tsx`

```
1 import { useState, useEffect } from 'react';
2 function Child({ p, onChange }: { p: any; onChange: (v: any) => void }) {
3   const [x, setX] = useState(null);
4   const [s, setS] = useState(null);
5   useEffect(() => { setX({ p }); }, [p]);
6   useEffect(() => { if (p && !s) setS({ v: 1 }); }, [x]);
7   useEffect(() => { if (!p) setS(null); }, [x]);
8   useEffect(() => { onChange(s ? null : { k: 1 }); }, [s]);
9   return null;
10 }
11 export function Parent() {
12   const [p, setP] = useState<any>({ k: 1 });
13   return <Child p={p} onChange={setP} />;
14 }
```

Child (6 hooks) props_move.tsx

warn cross-component-infinite-loop [hook:2] (line 5:2) these effects may form a state-update cycle ('x' →

```

↪ `s` → `p` of `Parent` → `x`) where each step may store a fresh reference that re-runs the next effect:
↪ possible infinite render loop
  → a fresh value is written to state `x` here [hook:2] (line 5:20)
  → cycle continues: this effect freshly stores state `s` [hook:3] (line 6:33)
  → cycle continues: this effect freshly stores state `p` of `Parent` [hook:5] (line 8:20)
warn  cross-component-infinite-loop [hook:3] (line 6:2) these effects may form a state-update cycle (`x` →
↪ `s` → `p` of `Parent` → `x`) ...
warn  cross-component-infinite-loop [hook:5] (line 8:2) this effect calls `onChange`, a state setter of
↪ parent `Parent` (its deps do not provably gate it, so the effect can re-run every render). ...

```

Test : props_move_on_a_cycle_that_enters_through_another_edge.

Le cycle est $(\text{Parent}, p) \rightarrow x \rightarrow s \rightarrow (\text{Parent}, p)$: il *entre* dans Child par le prop p (arête versionnée vers x) et en *sort* par le setter-prop `onChange`. L'arête $x \rightarrow s$ est locale, et les deux sites de s tirent sous p et $!p$: si ces faits *tenaient*, ils se contrediraient et la garde `!s` mourrait — l'arête serait tuée et le cycle perdu. Or p bouge le long du cycle. La figure 15.3 dessine le graphe.

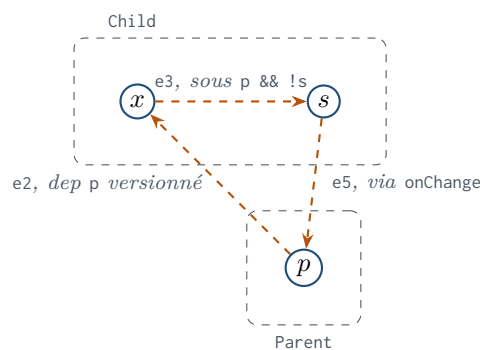


FIGURE 15.3 – Le graphe de `props_move.tsx` : un cycle de trois arêtes May dont un nœud appartient à Parent (cross_component). Child porte les trois arêtes ; comme un de ses déclencheurs nomme un slot du parent, `props_hold` est faux et les faits sur p ne sont pas lus.

La solution est un fait par composant, calculé avant toute preuve (`src/engine/churn.rs`, l. 232) : `props_hold = effect_triggers.iter().all(|t| t.slot.0 == comp)`. Aucun effet du composant ne réagit au slot d'un autre composant $\Rightarrow$ aucune boucle n'y entre par ses props $\Rightarrow$ ses props tiennent le long de toute boucle qui passe par ses arêtes. Un seul `dep` versionné par un slot du parent, et les props peuvent bouger sur *tout* cycle qui traverse le composant, quelle que soit l'arête empruntée : les preuves ne lisent alors plus aucun fait sur eux (`Invariance::holds` rend `props_hold` pour un nom que rien ne lie, `src/engine/guards.rs`, l. 353). Ici Child a un déclencheur sur $(\text{Parent}, \emptyset)$, donc `props_hold = false`, les faits p et $!p$ sont écartés, aucune contradiction, l'arête $x \rightarrow s$ survit, le cycle est rapporté.

Remarque

Le programme reçoit *trois* WARNING pour une seule boucle : deux du bras graphe (effets 2 et 3, porteurs d'arêtes) et un du bras cross-component historique (effet 5, qui appelle le setter du parent). La déduplication de `src/rules/impls/infinite_loop.rs`, l. 277 ne joue que par effet : l'effet 5 est sauté par le bras graphe parce que déjà rapporté, les effets 2 et 3 ne le sont pas. C'est un choix de ADR-18 (« un rapport par effet porteur »), pas un défaut.

15.8.3 Ce qui ne remonte pas

Le graphe ne voit une écriture étrangère que si le parent a été analysé *descendant* et que `ComponentSetter` a coulé jusqu'à l'enfant. Un parent analysé seulement en intra laisse l'enfant sans ligne étrangère, donc sans arête et sans `cross-component-infinite-loop` (#20, `precision-fn`).

Les Consequences d'ADR-42 l'enregistrent, en reprenant ADR-29, comme affaiblissement connu : « rows exist only for cycles the graph sees ; prop-mediated edges need the parent slot flowed top-down (#20) ».

15.9 De l'arête au diagnostic : la frontière de certification

Le graphe dit Must ou May ; il ne *frappe* rien. La sévérité est décidée côté règles, et jamais en faisant confiance à un booléen fourni par le graphe : elle est *re-dérivée* au point de connaissance (chapitre 17, définition 17.1).

15.9.1 Le bras graphe et `must_effect_cycle`

Extrait 15.28 – `src/rules/impls/infinite_loop.rs`, l. 281–292 — le bras multi-effets choisit la règle et demande la preuve

```

281      // Cross-component must-rerun is unprovable (prop deps are
282      // `Versioned`, never the exact slot) → Warning ceiling. An all-must
283      // intra-component cycle mints the proof – the only path to Error.
284      let rule = if cycle.cross_component {
285          "cross-component-infinite-loop"
286      } else {
287          "infinite-loop"
288      };
289      let cycle_proof = match must_effect_cycle(edges, cycle) {
290          MustResult::All(c) => Some(c),
291          _ => None,
292      };

```

Extrait 15.29 – `src/rules/api/query.rs`, l. 1002–1031 — `must_effect_cycle` re-dérive les deux faits

```

1002 pub(in crate::rules) fn must_effect_cycle(
1003     edges: &[crate::engine::ChurnEdge],
1004     cycle: &crate::engine::ChurnCycle,
1005 ) -> MustResult<EffectCycleProof> {
1006     use crate::engine::EdgeStrength;
1007     let all_must = cycle
1008         .edge_idx
1009         .iter()
1010         .all(|&i| edges[i].strength == EdgeStrength::Must);
1011     let mut comps: HashSet<ComponentId> = HashSet::new();
1012     for &i in &cycle.edge_idx {
1013         comps.insert(edges[i].from.0);
1014         comps.insert(edges[i].to.0);
1015         comps.insert(edges[i].component);
1016     }
1017     if all_must && comps.len() == 1 {
1018         // The cycle's first edge is where the proof starts: the write that
1019         // re-triggers the effect carrying it. Both are on the edge already, so
1020         // the token carries them rather than leaving the rule to re-derive
1021         // them from a `Vec` index.
1022         let first = cycle.edge_idx.first().map(|&i| &edges[i]);
1023         let provenance = match first {
1024             Some(e) => Provenance::at(e.write_span, Some(e.effect_label)),
1025             None => Provenance::default(),
1026         };
1027         MustResult::All(Certified::mint(EffectCycleProof, provenance))

```

```

1028     } else {
1029         MustResult::None
1030     }
1031 }

```

`must_effect_cycle` recalcule depuis les arêtes brutes ce que `ChurnCycle` porte déjà : toutes les arêtes `Must` et un seul composant impliqué. Si les deux tiennent, il *frappe* (`Certified::mint`) une preuve `EffectCycleProof` dont la provenance est la première arête du cycle — l’écriture qui relance l’effet porteur. Sinon `MustResult::None`, et le diagnostic est un `WARNING`. La règle choisit le nom selon `cycle.cross_component` et la sévérité selon la *preuve* : un cycle cross-component tout-`Must` est possible (deux arêtes `Must` locales et une auto-arête étrangère, par exemple) et reste un `WARNING`, et sa formulation suit la preuve (« may form »), pas `all_must` (`src/rules/impls/infinite_loop.rs`, l. 294–296).

cycle	no_deps	preuve	message (début)
1	oui	oui	« this effect has no dependency array and stores a fresh reference ... infinite render loop »
1	oui	non	« ... and may store a fresh reference ... possible infinite render loop »
1	non	oui	« this effect stores a fresh reference into state ... which its own deps react to »
1	non	non	« this effect may store a fresh reference ... which its own deps react to »
≥ 2	—	oui	« these effects form a state-update cycle (<i>chemin</i>) ... »
≥ 2	—	non	« these effects may form a state-update cycle (<i>chemin</i>) ... »

TABLE 15.3 – Les six formulations du bras graphe (`src/rules/impls/infinite_loop.rs`, l. 298–334). La colonne « preuve » est `cycle_proof.is_some()`, jamais `all_must`.

Le témoin (définition 17.2) est une étape `Write` (valeur `Fresh`) au `write_span` de l’arête portée, puis une étape `CycleEdge` par autre arête du cycle portée par le même composant : c’est la liste « cycle continues » des sorties de ce chapitre. Le chemin ‘a’ → ‘b’ → ‘a’ est rendu par `cycle_path` (`src/rules/helpers/cycles.rs`, l. 54–70), qui nomme un slot étranger ‘p’ of ‘Parent’ (`node_display`) ; ces deux fonctions vivent côté règles « parce qu’un nom d’affichage est un rendu, pas un fait » (`doc de module, src/rules/helpers/cycles.rs`, l. 1–8).

15.9.2 Le bras self-churn re-dérive must-reach

Extrait 15.30 – `src/rules/impls/infinite_loop.rs`, l. 441–460 — le bras self-churn ne fait pas confiance à `e.strength`

```

441     let (sev, churn_proof) = if e.self_slot {
442         // The Error anchors on the fresh write being on all paths –
443         // re-derived from the writer rows and minted by the
444         // must-primitive, never trusted from the edge's own strength.
445         let fresh_blocks: HashSet<BlockId> = comp_result
446             .slot_writers
447             .iter()
448             .filter(|w| w.owner.is_none() && w.slot == state_label)
449             .filter(|w| w.region == WriterRegion::Effect(*eff_label))
450             .filter(|w| w.written.fresh == Freshness::Fresh)
451             .filter_map(|w| w.block)
452             .collect();
453         if e.strength == EdgeStrength::Must && !fresh_blocks.is_empty() {
454             match must_on_all_paths(body_cfg, &fresh_blocks) {
455                 MustResult::All(c) => (Severity::Error, Some(c)),
456                 _ => (Severity::Warning, None),

```

```

457         }
458     } else {
459         (Severity::Warning, None)
460     }

```

Le bras self-churn (chapitre 19) lit sa partition — les arêtes du composant, de l'effet, non no_deps, de slot local vers slot local (src/rules/impls/infinite_loop.rs, l. 427–433) — et, pour une arête self_slot, *recalcule* l'ensemble des blocs d'écritures fraîches depuis slot_writers et demande must_on_all_paths. e.strength == Must est une condition nécessaire (elle évite un calcul inutile), jamais suffisante : la preuve est frappée par la must-primitive. Les arêtes non self_slot de la partition alimentent l'INFO « freshly recreates state ... outside its deps », supprimée pour les écritures covered par un cycle rapporté (exemple 15.4).

15.9.3 L'ancre churn_cycles des règles déclaratives

Extrait 15.31 – src/rules/helpers/cycles.rs, l. 72–93 — une ligne churn_cycles

```

72 /// One `churn_cycles` row: a cycle of the program graph, seen from the effect
73 /// of THIS component that carries one of its edges.
74 ///
75 /// Per-component attribution mirrors the native arm – an effect in another
76 /// component reports in that component's own pass – so the relation stays
77 /// single-anchored on the component under analysis.
78 #[derive(Debug, Clone, PartialEq, Eq)]
79 pub(in crate::rules) struct CycleRow {
80     /// The cycle rendered as `a → b → a`.
81     pub path: String,
82     /// The cycle spans more than one component (slot owners or effect
83     /// carriers). Exact: a fold of the node table the graph already built.
84     pub cross_component: bool,
85     /// Every edge of the cycle is a Must edge. Exact, and the graph's own
86     /// must-claim – it is what the native arm certifies an Error from.
87     pub all_must: bool,
88     /// The effect of this component that carries the edge.
89     pub effect: HookLabel,
90     /// The carrying edge's write site – the row's identity (ADR-024). A row
91     /// exists only when there is one.
92     pub span: SourceRange,
93 }

```

Décision : ADR-29

Les règles déclaratives (chapitre 22) sont *mono-ancre* : une règle parle d'un composant. Le graphe étant programme-entier, ADR-29 le *projette* : une ligne par arête de cycle *portée par un effet de ce composant* (collect_cycle_rows, src/rules/helpers/cycles.rs, l. 101–135), avec le chemin rendu, deux booléens exacts (cross_component, all_must — le may vit dans le graphe, pas dans la ligne) et le span de l'écriture comme identité (ADR-24). Une arête sans span ne donne *aucune* ligne : un canal de findings manqués enregistré, jamais un finding pointant nulle part. Refusés : un statut « couvert nativement, ne compte pas » ; fusionner les deux bras (ADR-20 item 2). L'ancre est plafonnée à WARNING structurellement : seule la règle native atteint ERROR, via must_effect_cycle.

15.9.4 Construit une fois

ChurnGraph::build lit tous les composants ; il est appelé une fois par programme, pares-

seusement, par l’accessor `ProgramRelations::churn` : un `OnceLock` rempli à la première demande par `get_or_init` (`src/engine/program_relations.rs`, l. 39–40, chapitre 14). Il est ensuite partagé par toutes les passes de règles. Un test le garantit par le compteur `BUILDS : churn_graph_is_built_once_per_program` (`src/rules/impls/infinite_loop.rs`, l. 631–684). C’est la correction de #86, où le graphe était reconstruit par composant et la phase des règles devenait quadratique en nombre de composants.

15.10 Limites connues au commit e67b10a

15.10.1 Faux négatifs connus

#157, valeur dérivée du slot écrit. `const d = useMemo(() => (...s), [s]); useEffect(() => setS(d), [d])` : la valeur écrite est `Versioned(s)`, que `written::classify` lit `Not` frais (lecture d’ADR-17) ; aucune arête, boucle silencieuse. Le correctif naïf (`Versioned => Maybe`) transforme le motif « mirror state » — recopier un prop dans un état — en `WARNING` massif ; l’issue reste ouverte, étiquetée `soundness-bug`.

#161, l’hypothèse d’invariance. Un appel sur des entrées qui tiennent est supposé rendre la même valeur ; une navigation cachée dans un `callee` opaque ou dans un objet routeur passé en prop n’est pas vue (`docs/limitations.md`).

#20, parent analysé en intra. Voir section 15.8.

15.10.2 Faux positifs assumés (Warning au plus)

Un slot convergent que *un autre composant* écrit aussi (résidu de #39 : `foreign` n’est jamais tué) ; une garde sur le résultat d’un hook non inliné ou d’un store externe, lu comme bougeant (#161) ; un setter-prop typé primitif dont l’argument `T` est lu comme référence possible (#159) ; une garde disjonctive ou arithmétique sur la valeur comparée (#91) ; un enfant rendu depuis un `.map` ou atteint via un composant intermédiaire, lu comme remontant (#162, côté `fail-closed`).

15.10.3 Granularité et forme du résultat

Le graphe est granulaire *au slot* : savoir si l’écriture de y par l’effet A change le *membre* de y que lit le `dep` de l’effet B est une propriété d’une paire d’arêtes, que ni une arête ni le `kill` ne portent ; seul le bras `self-churn` lit les membres (`can_retrigger`, bras membre de la preuve). Un seul cycle simple est reconstruit par SCC, et un cycle `May` qui recoupe une SCC `Must` n’est pas rapporté (section 15.3).

15.10.4 Dette documentaire

Le code fait foi ; trois textes sont en retard sur lui. Les « Consequences » d’ADR-42 disent encore que la condition d’écrivain unique est inchangée, alors que le dernier paragraphe et le §6 amendé la déclarent remplacée. Le paragraphe de `kill` de `docs/relations.md` (« under every other effect write of the slot in the component ») date de l’amendement #154 : il ignore les sites de rendu et de memo et l’exclusion des revivers convergents. Enfin le commentaire d’en-tête du bras `self-churn` (`src/rules/impls/infinite_loop.rs`, l. 393–394) dit de l’INFO que « cross-effect cycles are not analyzed », ce qui est faux depuis ADR-18 : l’INFO n’est plus émise que lorsqu’aucun cycle ne couvre l’écriture. Ordre de confiance : le code au commit e67b10a, puis les doc-comments de `module` de `churn.rs` et `guards.rs` (réécrits dans ce commit), puis le dernier amendement d’ADR-42 §6.

Résumé du chapitre

- Le **graphe de churn** (définition 15.1) est un pli sur `slot_writers` et `effect_triggers` : nœuds = slots qualifiés, arête $x \rightarrow y$ = « un changement de x relance un effet qui stocke une référence fraîche dans y ». `Must` = `dep exact` $\wedge$ `Fresh` $\wedge$ bloc synchrone sur tous les chemins ; `May` sinon ; la colonne phase décide (tableau 15.1), `Handler` ne porte rien. Un effet sans deps porte une auto-arête ; `self_slot` isole le bras `self-churn` (ADR-20 item 2).
- **Tarjan en deux passes** : sous-graphe `Must` (cycles `all_must`, `ERROR` via lemme 15.1) puis graphe entier (`WARNING`), en sautant les régions couvertes ; un cycle simple par `SCC`.
- Le **kill de convergence** retire une arête dont l'écriture tire au plus une fois. ADR-18 l'autorisait sous condition d'écrivain unique (FP #39) ; ADR-42 §6 le remplace par `converges_under_all_writes` : gardes mortes sous l'ensemble *tueur* (écriture propre et écritures synchrones co-exécutées sur la chaîne) et sous chaque *reviver vivant*, pris avec ses faits *invariants* (définitions 15.3 et 15.4).
- Un **site** (définition 15.2) est toute ligne non-handler d'un corps de rendu, d'effet non mount-only ou de memo, plus les lignes étrangères d'un effet mount-only d'un enfant que la boucle remonte (`stays_mounted`). Un slot `foreign` n'est jamais tué.
- Les sites convergents forment un **plus petit point fixe** (théorème 15.1) : un *reviver* prouvé n'est plus vivant. Le plus grand point fixe lirait deux sites qui se raniment comme convergents (théorème 15.2, exemple 15.11). L'arête est ensuite jugée sur la *partie référence* de sa valeur (`reference_part`).
- **Cross-component** : lignes `owner` = `Some`, deps `Versioned` donc jamais `Must`, `cross_component` plafonné à `WARNING` ; `props_hold` interdit de lire des faits sur les props d'un composant où une boucle peut entrer.
- **Frontière** : le graphe ne frappe rien ; `must_effect_cycle` et `must_on_all_paths` re-dérivent la preuve ; l'ancre `churn_cycles` (ADR-29) projette le graphe par composant.
- Fichiers : `src/engine/churn.rs` (`ChurnGraph::build`, `build_edges`, `find_cycles`, `stays_mounted`, `can_retrigger`), `src/engine/guards.rs` (`WriteSite`, `converges_under_all_writes`, `Invariance`, `contradicts`), `src/engine/written.rs` (`reference_part`), `src/rules/helpers/cycles.rs` (`CycleRow`), `tests/effect_cycles.rs`.

Le chapitre 16 explique d'où viennent les lignes étrangères et les valeurs `Versioned` des props que ce chapitre a supposées ; le chapitre 19 assemble les quatre bras de `infinite-loop`.

15.11 Exercices

Exercice 15.1: Prédire le graphe

Sans lancer l'analyseur, donner les arêtes (extrémités, force, `self_slot`, `no_deps`) et les cycles du programme suivant, puis la sévérité attendue.

```
1 export function Q({ p }: { p: boolean }) {
2   const [a, setA] = useState({});
3   const [b, setB] = useState({});
4   useEffect(() => { if (p) setB({ a }); else setB({}); }, [a]);
5   useEffect(() => { setA({ ...a, b }); }, [b, a]);
6   return null;
7 }
```

Solution

$a \rightarrow b$: deux écritures Fresh de b dans deux blocs dont l'union est sur tous les chemins, dep exact : Must. $b \rightarrow a$: Must (écriture à l'entrée, dep exact). $a \rightarrow a$: dep a exact et écriture de a : Must, `self_slot = true`, et `write_can_retrigger` répond true (l'argument n'est pas un updater fonctionnel). Le cycle $a \rightarrow b \rightarrow a$ est tout-Must, mono-composant : ERROR sur les deux effets par le bras graphe ; l'arête `self_slot` donne en plus une ERROR du bras `self-churn` sur le second effet.

Exercice 15.2: Un reviver stable

Dans l'exemple 15.6, remplacer `setB(null)` par `setB(CONST)` où `CONST` est un objet de module. L'arête $a \rightarrow b$ est-elle tuée ? Quelle ligne de `converges_under_all_writes` décide ?

Solution

Oui, elle est tuée. Le reviver est vivant (non gardé, non convergent) ; mais sa valeur est une référence Stable, truthy, et la garde « b falsy » devient bien $\perp$ sous cette réécriture : le second `dead_once_written` (`src/engine/guards.rs`, l. 253) réussit. Observé sur `/tmp/ch15/const_reviver.tsx` (`const CONST = src: 'const'` au niveau du module) : plus aucun cycle, seule reste l'INFO « freshly recreates state ' a ' outside its deps » sur l'effet qui écrit a . Piège inversé : l'exercice teste que le lecteur n'applique pas la condition d'écrivain unique d'ADR-18, qui aurait gardé l'arête ; la preuve multi-site l'enlève, à raison, puisque `CONST` ne remet jamais b à vide.

Exercice 15.3: Plus grand point fixe

Dérouler l'itération « tous convergents, puis retirer » sur `mutual.tsx` (`if (cond) setS(null); if (!s) setS(...)` dans un seul effet) et sur `null_updater.tsx`. Sur lequel des deux le plus grand point fixe diffère-t-il du plus petit ? Pourquoi ?

Solution

Sur `null_updater.tsx` seulement : chacun des deux sites y meurt sous sa propre écriture et n'est réfuté que par l'autre, donc $\text{gfp} = \{A, B\}$ et $\text{lfp} = \emptyset$. Dans `mutual.tsx`, la garde `cond` du site `setS(null)` est un prop qui tient : elle ne meurt sous aucune écriture de s , le site est réfuté quel que soit l'état de l'autre, puis il ranime l'autre ; les deux points fixes sont vides.

Exercice 15.4: Pourquoi jamais Must

Expliquer, à partir de la liste 15.7, pourquoi l'arête de l'exemple 15.12 ne peut pas être Must ni `self_slot`, puis proposer ce qu'il faudrait au moteur pour prouver un must-rerun à travers un prop.

Solution

Le `dep value` est un prop : sa valeur est `Versioned((Parent, 0))`, la ligne `effect_triggers` est `exact = false`, le slot entre donc dans `f.versioned` et la seule poussée possible est `push(x, EdgeStrength::May, ...)` (`src/engine/churn.rs`, l. 592). `self_slot` exige `x.0 == f.comp` (`src/engine/churn.rs`, l. 589) : le nœud appartient à Parent, l'effet à Child. Pour un must-rerun, il faudrait savoir que le prop est le slot entier du parent, et non une valeur seulement versionnée par lui : la voie qu'ADR-18 nomme sans la prendre, une provenance de prop entier au site d'appel JSX.

Exercice 15.5: Un enfant dans une liste

Modifier `stays_mounted.tsx` pour rendre `<Child/>` depuis `items.map(i => <Child key=i onReady=setS/>)` avec `items` un prop. Que répond `stays_mounted` et pourquoi? Est-ce un FP ou un FN potentiel?

Solution

false : l'élément est dans une closure (ok &= !closure, `src/engine/churn.rs`, l. 405) — la boucle peut ajouter un élément et monter un nouvel enfant. La ligne étrangère devient un site, `(Parent, 0)` est foreign, l'arête `self_slot` survit : WARNING. Côté fail-closed de #162, un FP possible, jamais un FN. Observé (`/tmp/ch15/map_child.tsx`) : le WARNING « this effect may store a fresh reference into state 's' which its deps react to », là où `stays_mounted.tsx` est « verified ».

Exercice 15.6: Mirror state

(Avancé.) Proposer une approche de #157 qui rende `setS(derived)` avec `derived = useMemo(() => (...), [s])` visible du graphe sans transformer `setLocal(propValue)` en WARNING. Indication : distinguer `Versioned` par un slot *du même composant* écrit par l'effet, et s'appuyer sur la preuve de convergence plutôt que sur la fraîcheur.

Chapitre 16

L'analyse inter-composants et inter-fichiers

Objectifs du chapitre.

- Comprendre pourquoi un analyseur *intra*-composant, qui lit les props comme $\top$, manque les défauts qui traversent une frontière de composant, et ce qu'il faut pour les voir.
- Connaître l'identité d'un composant : la clé (`file`, `name`), le `ComponentId` interné, le nom d'affichage qui n'est qu'un rendu, l'origine `CompOrigin` qu'un fichier prouve pour un élément JSX, et les trois réponses de `resolve_child` (ADR-40).
- Savoir comment les racines sont choisies (`RootStrategy`, `-all-roots`, `-entry`) et pourquoi une référence non résolue compte pour tous les homonymes.
- Suivre pas à pas l'inlining descendant (`eval_comp_app`, `InterCtx`, pile de récursion, props dans le tas) et le flux ascendant (setter porteur de propriétaire, store partagé monotone, `havoc`).
- Comprendre le cache de composants, pourquoi il n'est qu'un « déjà vu », et le faux négatif qui en découle ; les deux phases d'`analyze_program` et la discipline `phase1_reached`.
- Connaître ce que lit une règle (`AnalysisResult`, `ProgramAnalysisResult`, `RuleCtx`) et ce que le graphe de symboles d'ADR-13 est devenu.
- Maîtriser la *dépendance de rendu* (ADR-41) : le treillis `Deps`, la dépendance de contrôle, la séparation *genuine* / *sites*, *handlers*, contextes, bindings de module, fait `gated`, et sa composition sur l'arbre d'éléments (`uses`, `home_of`, `wasted_siblings`).

Prérequis : chapitre 13 (la boucle externe d'un composant, sa passe de rendu, le test de convergence, la passe de rafraîchissement) ; chapitre 11 (`eval_comp_app` vu du transfert, le `SharedStateStore`, le tas, le `havoc` des setters, définition 11.2) ; chapitre 10 (définition 10.1, la composante setter du produit, `InterCtx`) ; chapitre 6 (définition 6.3, définition 6.2, définition 6.1, `Expr::CompApp`) ; chapitre 3 (définition 3.2, définition 3.7, définition 3.8). Les règles qui consomment la dépendance de rendu sont traitées aux chapitres 18 et 21.

Jusqu'ici, le moteur a été décrit composant par composant : un CFG de rendu, des effets, des handlers, une boucle externe qui monte l'état jusqu'à un point fixe. Or un programme React est un *arbre* de composants, écrit dans des dizaines de fichiers, où l'état d'un parent descend en props et où ses setters descendent pour être appelés plus bas. Ce chapitre répond à deux questions qu'aucune analyse d'un composant isolé ne peut trancher.

1. **Qui rend qui, avec quelles props ?** C'est l'analyse *inter-composants* (ADR-12) : un parent est analysé d'abord, et quand son rendu rencontre `<Child a=x/>`, l'enfant est analysé avec les props *abstraites* évaluées dans l'environnement du parent. Elle demande de savoir quel composant un nom JSX désigne, à travers les fichiers (ADR-13, ADR-40), par où commencer, et comment ne pas ré-analyser un enfant pour des props identiques.
2. **Quelles parties de la sortie d'un composant peuvent être calculées à partir**

d'un état donné ? C'est la *dépendance de rendu* (ADR-41), une seconde analyse, séparée, lancée après convergence, qui alimente les règles de cascades de re-rendus.

Le sous-système tient dans sept fichiers de `src/engine/` : `component_registry.rs` (256 lignes), `root_detector.rs` (329), `component_cache.rs` (327), `symbol_graph.rs` (416), `analysis_result.rs` (289), `program_result.rs` (225) et `render_deps.rs` (1276). Le mécanisme lui-même est porté par quatre fichiers voisins : `src/ir/component_id.rs` (l'identité), `src/domains/context.rs` (InterCtx), `src/domains/transfer/state_value.rs` (eval_comp_app) et `src/engine/fixpoint.rs` (analyze_program). La composition des résumés de rendu sur l'arbre d'éléments vit dans la couche règles, `src/rules/helpers/render_tree.rs` (777 lignes). Toutes les sorties de l'analyseur montrées ici ont été observées avec le binaire du commit e67b10a sur des fichiers de `/tmp/ch16/` ou sur les fixtures du dépôt.

16.1 Le problème : des props inconnues, un setter qui remonte

16.1.1 Un badge qui relance son parent

Exemple 16.1: Un setter appelé pendant le rendu d'un enfant

Extrait 16.1 – `/tmp/ch16/intro.tsx`

```
1 import { useState } from "react";
2
3 function Badge({ onSeen }: { onSeen: (n: number) => void }) {
4   onSeen(1);
5   return <span>new</span>;
6 }
7
8 export function Inbox() {
9   const [seen, setSeen] = useState(0);
10  return <div>{seen}<Badge onSeen={setSeen} /></div>;
11 }
```

Badge appelle, pendant son rendu, la fonction que son parent lui passe. Or cette fonction est `setSeen`, le setter d'un slot d'`Inbox`. Chaque rendu de `Badge` écrit donc l'état de son parent pendant un rendu, ce que React signale en développement (« Cannot update a component while rendering a different component ») ; chaque écriture qui change la valeur re-rend le parent, donc l'enfant, qui écrit de nouveau. L'analyseur le voit :

```
$ reactant check intro.tsx --trace
Badge (0 hooks) intro.tsx
  error cross-setter-in-render var:onSeen (line 4:2) prop `onSeen` (a state setter of parent `Inbox`) called
    ↳ during render of `Badge`, which triggers a parent re-render on every render
    ↳ `onSeen` is a state setter, so calling it writes state (line 4:2)
1 clean component(s) hidden, rerun with --show-clean

! 1 error(s) across 1 file(s).
```

Le diagnostic est une **ERROR** : l'appel est inconditionnel dans le corps du rendu, et le message nomme le propriétaire du setter, `Inbox`. Ce nom n'est écrit nulle part dans `Badge`.

16.1.2 Ce qu'un analyseur intra ne voit pas

Retirons le parent. Le fichier `/tmp/ch16/badge-alone.tsx` ne contient que la fonction `Badge`, verbatim :

```
$ reactant check badge-alone.tsx --info
✓ 1 file(s) no issues found.
```

Aucune alarme, et à juste titre. Analysé seul, Badge reçoit un objet de props dont l'analyse ne sait rien : son paramètre n'est lié à rien dans l'environnement d'entrée, et une variable absente se lit $\top$ (`AbstractEnv::lookup, src/domains/stores/abstract_env.rs, l. 78–81`). `onSeen` est donc « une valeur quelconque » ; rien ne prouve que c'est un setter, et une règle `ERROR` doit prouver *toute* sa conclusion. Un analyseur intra-composant est *sound* (il ne conclut rien de faux), mais il est aveugle à toute une famille de défauts : un setter passé en prop et appelé pendant le rendu, un effet d'enfant qui relance le parent (`cross-component-infinite-loop`), un état qu'un parent possède pour le seul usage d'un arrière-petit-enfant.

Rappel React : props, setters, identité

Un setter renvoyé par `useState` a une identité stable pour toute la vie du composant, et il écrit toujours le slot de son *propriétaire*, quel que soit le composant qui l'appelle. Le passer en prop est l'idiome normal pour qu'un enfant remonte une valeur. Un *élément* `<Badge onSeen=setSeen/>` n'est qu'une description (`jsx(Badge, props)`) : Badge ne s'exécute que lorsque React monte l'élément, avec ces props (chapitre 2).

Pour conclure, l'analyse doit donc savoir que `<Badge/>` dans `Inbox` désigne *ce* Badge-là, analyser Badge avec la valeur abstraite que prend `onSeen` dans `Inbox` — un setter porteur de son propriétaire — puis faire remonter l'écriture de Badge dans l'état d'`Inbox`. C'est exactement ce que fait l'analyse inter-composants.

16.1.3 Deux questions, un sous-système

La seconde question naît d'un autre programme, la fixture `drill.tsx` du répertoire `tests/fixtures/state_lifted_too_high/` :

Extrait 16.2 – `tests/fixtures/state_lifted_too_high/drill.tsx`, l. 6–19 — un état tenu trois niveaux trop haut

```
6 function Content() { return <article>static content</article>; }
7 function Field({ value, onChange }: { value: string; onChange: (v: string) => void }) {
8   return <input id="f" value={value} onChange={(e) => onChange(e.target.value)} />;
9 }
10 function Sidebar({ value, onChange }: { value: string; onChange: (v: string) => void }) {
11   return <aside><h2>Filters</h2><Field value={value} onChange={onChange} /></aside>;
12 }
13 function Layout({ value, onChange }: { value: string; onChange: (v: string) => void }) {
14   return <main><Sidebar value={value} onChange={onChange} /><Content /></main>;
15 }
16 export default function App() {
17   const [value, setValue] = useState("");
18   return <Layout value={value} onChange={setValue} />;
19 }
```

Rien n'y est faux : c'est un problème de coût. À chaque frappe, `setValue` re-rend `App`, et donc tous les éléments de composant qu'`App` construit, récursivement : `Layout`, `Sidebar`, `Field`, et `Content` au passage. Seul `Field` utilise la valeur. Pour le dire, l'analyse doit savoir, pour chaque composant, ce qui dans sa sortie *dépend* de l'état et ce qui ne fait que le *transmettre* à un enfant. La valeur abstraite ne le dit pas (nous verrons pourquoi au section 16.10) : il faut une analyse de dépendance.

La figure 16.1 situe les deux mécanismes. Les points d'entrée exacts sont `resolver::analyze_lowered`

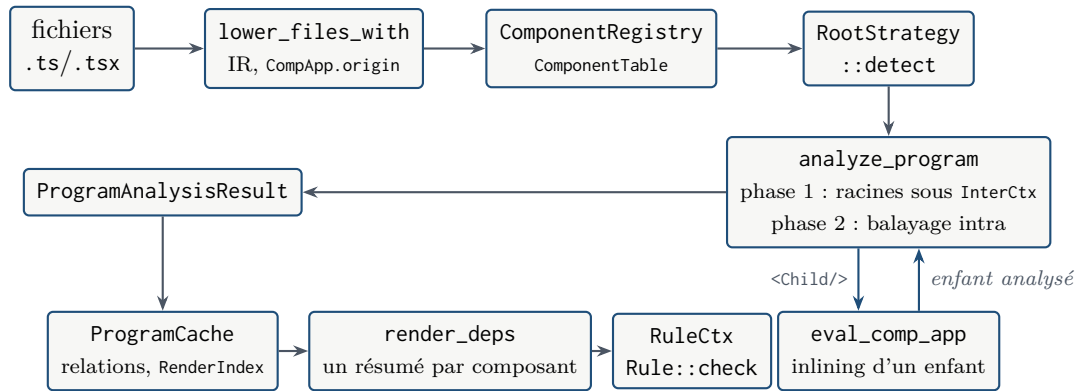


FIGURE 16.1 – Le pipeline programme. L'IR de tous les fichiers est abaissée d'abord ; le registre interne l'identité des composants ; la stratégie choisit les racines ; `analyze_program` analyse les racines en inlinant leurs enfants à la volée (`eval_comp_app`), puis balaie les composants non atteints. Les résumés de dépendance de rendu sont calculés à la demande, une fois par programme, dans la couche règles.

(`src/resolver/mod.rs`, l. 439–452), qui construit les registres et appelle `engine::analyze_program` (`src/engine/fixpoint.rs`, l. 704–819) ; puis le driver construit un `ProgramCache` partagé par tous les `RuleCtx`, dont la première demande de `render()` calcule les résumés `render_deps` de tous les composants. Retenons un point dès maintenant : dans le domaine des valeurs, un élément JSX de composant vaut *toujours* `StateValue::reference(Stability::Stable)` ; ce qui compte, ce sont les effets de bord de son évaluation — le résultat de l'enfant, les écritures dans le store partagé, les arêtes du graphe d'appel.

16.2 L'identité d'un composant

16.2.1 Trois façons de nommer un composant

Avant de pouvoir analyser un enfant, il faut savoir *lequel*. Considérons quatre fichiers (`/tmp/ch16/amb/`) :

Extrait 16.3 – Deux Widget, un alias, une référence nue (`/tmp/ch16/amb/`)

```

1 // a/Widget.tsx
2 import { useEffect, useState } from "react";
3 export function Widget() {
4   const [n, setN] = useState(0);
5   useEffect(() => { setN(n + 1); });
6   return <p>{n}</p>;
7 }
8 // b/Widget.tsx
9 export function Widget({ onReady }: { onReady: (v: number) => void }) {
10   onReady(1);
11   return <p>b</p>;
12 }
13 // App.tsx
14 import { useState } from "react";
15 import { Widget as Panel } from "../b/Widget";
16 export function App() {
17   const [v, setV] = useState(0);
18   return <Panel onReady={setV} />;
19 }
20 // Page.tsx (aucun import de Widget)

```

```

21 import { useState } from "react";
22 export function Page() {
23   const [v, setV] = useState(0);
24   return <Widget onReady={setV} />;
25 }

```

Trois « noms » coexistent. Le nom *écrit* au site d'appel (Panel, Widget); le couple (fichier de définition, nom exporté), qui seul désigne un composant sans ambiguïté ((b/Widget.tsx, Widget)); et le nom qu'un rapport *affiche*, qui doit distinguer deux Widget (Widget@b/Widget.tsx). L'histoire du projet est celle de la confusion entre ces trois orthographes, et de sa résolution par ADR-40.

16.2.2 ComponentId et ComponentTable

Extrait 16.4 – src/ir/component_id.rs, l. 23–29 — l'identité d'un composant

```

23 /// A component's identity, interned in a [`ComponentTable`].
24 ///
25 /// `Copy` and 4 bytes, so it sits inside a `BTreeSet` label or a store key
26 /// without the allocation a name costs, and two ids compare in one
27 /// instruction.
28 #[derive(Debug, Clone, Copy, PartialEq, Eq, Hash, PartialOrd, Ord)]
29 pub struct ComponentId(u32);

```

Un ComponentId est un index de 4 octets, Copy et Ord. Il tient dans une clé de store ou dans un label d'un BTreeSet sans allocation, et deux identités se comparent en une instruction. Seule ComponentTable::intern en fabrique en production (from_index est réservé aux tests, src/ir/component_id.rs, l. 44–47). Une valeur réservée, ComponentId::SYNTHETIC (u32::MAX, src/ir/component_id.rs, l. 38), sert aux IR construites à la main par les tests unitaires, qui n'analysent qu'un composant et n'en comparent jamais deux.

Extrait 16.5 – src/ir/component_id.rs, l. 61–74 — la table d'interning

```

61 #[derive(Debug, Default, Clone)]
62 pub struct ComponentTable {
63     /// A map, not a vector indexed by id: a result analysed before any table
64     /// existed carries [`ComponentId::SYNTHETIC`], and registering *that* id
65     /// is how such a result joins a program without its interior labels
66     /// having to be rewritten.
67     origins: HashMap<ComponentId, CompOrigin>,
68     by_origin: HashMap<CompOrigin, ComponentId>,
69     next: u32,
70     /// How many files define each bare name – the only input to
71     /// [`Self::display_name`], precomputed because that answer is asked once
72     /// per rendered finding.
73     name_counts: HashMap<String, usize>,
74 }

```

La table est une bijection origins / by_origin entre identités et CompOrigin file, name.intern est idempotent (src/ir/component_id.rs, l. 77–86); register accepte un identifiant existant, pour le seul cas qui ne peut pas passer par intern : un résultat analysé seul, qui porte SYNTHETIC dans ses labels et doit rejoindre un programme sans réécriture (src/ir/component_id.rs, l. 94–101). ids() rend les identifiants triés, et comme l'interning suit l'ordre trié des clés du registre, tout parcours de la table est reproductible d'une exécution à l'autre (src/ir/component_id.rs, l. 121–128).

C'est le registre qui interne, parce que c'est le seul endroit qui connaît l'ensemble des

composants :

Extrait 16.6 – `src/engine/component_registry.rs`, l. 42–55 — construction du registre, internant en ordre de clé trié

```
42 pub fn from_components(comps: Vec<ComponentIR>) -> Self {
43     let entries = KeyedRegistry::from_keyed(
44         comps
45             .into_iter()
46             .map(|comp| ((comp.file.clone(), comp.name.clone()), comp)),
47     );
48     // Interned in sorted key order, so an id is reproducible across runs:
49     // the analysis iterates ids in places a report is ordered by.
50     let mut table = ComponentTable::default();
51     for (file, name) in entries.all_keys() {
52         table.intern(CompOrigin { file, name });
53     }
54     Self { entries, table }
55 }
```

La table est ensuite clonée dans le résultat de programme (champ `component_table` de `ProgramAnalysisResult`), de sorte que règles et rendu résolvent contre la table même qui a indexé l'analyse. Le registre lui-même est une `KeyedRegistry`, une table (`PathBuf`, `Symbol`) → `ComponentIR` partagée avec les registres de hooks et d'utilitaires (`src/registry/keyed.rs`, l. 21–113).

16.2.3 Le nom d'affichage n'est qu'un rendu

Extrait 16.7 – `src/ir/component_id.rs`, l. 138–154 — le nom affiché, suffixé seulement en cas de collision

```
138 /// The name to *show* for `id`: the bare name when only one file defines
139 /// it, `name@<file>` when several do.
140 ///
141 /// A rendering concern and nothing else. It is content-dependent by
142 /// design – the suffix exists precisely to tell two same-named components
143 /// apart in a report – which is exactly why no table may be keyed by it.
144 /// `--entry` accepts the qualified form, and
145 /// [ `Self::resolve_display_name` ] is its inverse.
146 pub fn display_name(&self, id: ComponentId) -> Option<String> {
147     let origin = self.origin(id)?;
148     let count = self.name_counts.get(&origin.name).copied().unwrap_or(0);
149     Some(if count <= 1 {
150         origin.name.clone()
151     } else {
152         format!("{}", origin.name, origin.file.display())
153     })
154 }
```

Le nom d'affichage dépend du *contenu* du programme analysé : `Widget` devient `Widget@a/Widget.tsx` dès qu'un fichier sans rapport définit un second `Widget`. C'est voulu — le suffixe n'existe que pour distinguer deux homonymes dans un rapport —, et c'est précisément pourquoi aucune table ne doit être indexée par lui.

Décision : ADR-40 — l'identité est un id interné, le nom d'affichage un rendu

Avant ADR-40, le nom d'affichage était l'orthographe *sur laquelle tournait l'analyse* (ADR-38 §5 l'avait choisie pour unifier deux orthographes) : la table des résultats, le store partagé, le label d'un `Versioned`, le propriétaire d'un `setter`, le graphe d'appel et `RuleCtx` étaient

tous indexés par lui. Ajouter un fichier sans rapport renommait donc un composant et ré-indexait toutes ces tables à la fois. ADR-40 décide : (1) l'identité est un `ComponentId` interné, comme `FileId` l'est pour les chemins (ADR-19); (2) le nom d'affichage est minté au rendu, depuis la table, et rien ne le compare; (3) un élément JSX porte ce que son fichier a prouvé (`CompOrigin`); (4) la résolution vit en un seul endroit et refuse de deviner; (5) tout chemin est normalisé au lowering (`lower_files_with`), pour que clés du registre et réponses du résolveur s'écrivent pareil. Mesure : digest identique sur 35 541 fichiers (1 348 localisations), 873 s contre 858 s — ni régression, ni gain revendiqué.

Parmi les six tests du module (`src/ir/component_id.rs`, l. 184–274), celui qui « dit pourquoi pas » s'appelle

`interning_a_namesake_changes_an_existing_display_name_but_not_its_id`

et son nom dit tout : ajouter un homonyme change le nom d'affichage d'un composant existant, jamais son identifiant. La couche règles dit les deux moitiés à voix haute :

Extrait 16.8 – `src/rules/api/query.rs`, l. 393–404 — l'identité pour comparer, le nom pour écrire

```

393     /// The component this ctx is anchored on – its identity, which is
394     /// what a comparison or a table lookup wants.
395     pub fn component(&self) -> ComponentId {
396         self.component
397     }
398
399     /// The component's name as a report shows it, for a message. Never for a
400     /// comparison: the display name is content-dependent by design (#7), so
401     /// two runs over different file sets can spell one component two ways.
402     pub fn component_name(&self) -> String {
403         self.program().display_name(self.component)
404     }

```

Piège

Écrire `if ctx.component_name() == "Foo"` dans une règle est une faute, même quand un test passe : sur un autre ensemble de fichiers, le même composant s'appelle `Foo@src/a/Foo.tsx`. Toute comparaison ou toute clé passe par `component()` et par la table (`ProgramAnalysisResult::component_named` est l'inverse public du nom d'affichage, `src/engine/program_result.rs`, l. 79–81, et répond `None` pour un nom nu ambigu).

16.2.4 L'origine prouvée d'un élément JSX

Un élément de composant porte, depuis ADR-40, ce que *son propre fichier* a prouvé du nom qu'il écrit :

Extrait 16.9 – `src/ir/expr.rs`, l. 245–258 — l'élément de composant dans l'IR

```

245     CompApp {
246         name: Symbol,
247         props: Box<Expr>,
248         /// Span of the element's opening tag. A diagnostic about a JSX element
249         /// (a context provider's `value`, an identity-keyed prop) has nowhere
250         /// else to point: unlike `NativeElem`, whose handler props are reached
251         /// through `HookEntry::Handler`, a component element owns no hook.
252         span: Option<SourceRange>,
253         /// Which component `name` was proven to name, when the file's
254         /// imports (or its own declarations) settle it – the same fact

```

```

255     /// `HookEntry::Custom::resolved_file` records for a custom hook call.
256     /// `None` is ignorance, never a proven negative.
257     origin: Option<Arc<CompOrigin>>,
258 },

```

Extrait 16.10 – src/ir/expr.rs, l. 160–172 — l'origine : fichier de définition et nom exporté

```

160 /// The component a JSX callee was proven to name: the file that defines it and
161 /// the name that file knows it by.
162 ///
163 /// Both halves are needed, and they are exactly [`crate::engine::ComponentKey`]
164 /// spelled without the engine dependency. The file settles which of several
165 /// same-named components is meant; the name settles an alias, since
166 /// `import { Widget as W }` writes `` here while the origin file registers
167 /// `Widget`.
168 #[derive(Debug, Clone, PartialEq, Eq, Hash)]
169 pub struct CompOrigin {
170     pub file: std::path::PathBuf,
171     pub name: Symbol,
172 }

```

Les deux moitiés servent. Le fichier départage vingt composants `Form`; le nom résout un alias : `import Widget as Panel` écrit `<Panel/>`, mais l'origine vaut `(b/Widget.tsx, Widget)`. L'origine est calculée au lowering par `build_jsx_origins` (`src/lowering/import_resolution.rs`, l. 237–262) : chaque déclaration de premier niveau du fichier courant s'associe à elle-même, puis chaque import que le résolveur mappe vers un vrai fichier s'associe à (fichier résolu, nom importé). Un nom absent de la carte est *non résolu* — import d'espace de noms, barrel de ré-export, paquet npm — et origin vaut `None`, qui se lit « ignorance », jamais « absence prouvée ».

16.2.5 resolve_child : trois réponses

Extrait 16.11 – src/engine/component_registry.rs, l. 8–20 — la réponse du registre sur un nom JSX

```

8 /// The answer [`ComponentRegistry::resolve_child`] gives about one JSX callee.
9 #[derive(Debug, Clone, PartialEq, Eq)]
10 pub enum ChildLookup {
11     /// Exactly one component can be meant.
12     Resolved(ComponentKey),
13     /// No component of that name was lowered – an npm component, or a file
14     /// the run did not cover.
15     Unknown,
16     /// Several files define the name and nothing at the call site settles
17     /// which. Distinct from [`Self::Unknown`] because the fix differs: the
18     /// definition is in the run, only the reference to it is unresolvable.
19     Ambiguous,
20 }

```

Extrait 16.12 – src/engine/component_registry.rs, l. 114–127 — résoudre un élément, sans jamais deviner

```

114 pub fn resolve_child(&self, name: &Symbol, origin: Option<&CompOrigin>) -> ChildLookup {
115     if let Some(o) = origin {
116         let key = (o.file.clone(), o.name.clone());
117         if self.entries.contains(&key) {
118             return ChildLookup::Resolved(key);
119         }
120     }
121 }

```

```

121     let mut matches = self.entries.keys().filter(|(n)| n == name);
122     match (matches.next(), matches.next()) {
123         (Some(key), None) => ChildLookup::Resolved(key.clone()),
124         (Some(_), Some(_)) => ChildLookup::Ambiguous,
125         _ => ChildLookup::Unknown,
126     }
127 }

```

L'ordre est : (1) l'origine prouvée, si le registre contient ce couple ; (2) sinon le nom écrit, s'il est unique dans tout le registre ; (3) sinon Ambiguous si plusieurs fichiers le définissent, Unknown si aucun. Une origine qui pointe vers un fichier qui ne définit pas le composant (un barrel, limite d'un niveau de ré-export, #49) retombe sur le nom. Le balayage des clés est linéaire par appel.

Exemple 16.2: Alias, collision, ambiguïté

Sur le répertoire de la liste 16.3 :

```

$ reactant check . --info --verbose
[verbose] 3 components analyzed
[verbose] cache hits: 3, misses: 1
[verbose] App: 1 iteration(s), widened: []
[verbose] Page: 1 iteration(s), widened: []
[verbose] Widget@a/Widget.tsx: 3 iteration(s), widened: [0]
[verbose] Widget@b/Widget.tsx: 0 iteration(s), widened: []
Page (1 hooks) Page.tsx
info analysis-limit several analysed files define a component called `Widget` and this reference does not
  ↳ resolve to one of them, so the child is treated as unknown. Import it explicitly from its file (FN possible)
  ↳ )
suspended analysis-limit 4 passing check(s) withheld: the analysis was truncated in this component, so they
  ↳ are not guaranteed
Widget@a/Widget.tsx (2 hooks) a/Widget.tsx
warn infinite-loop [hook:0] (line 5:2) this effect keeps pushing state `n` to new values on every run.
  ↳ Potential infinite render loop
...
Widget@b/Widget.tsx (0 hooks) b/Widget.tsx
error cross-setter-in-render var:onReady (line 2:2) prop `onReady` (a state setter of parent `App`) called
  ↳ during render of `Widget@b/Widget.tsx`, which triggers a parent re-render on every render

```

(sortie élaguée : la ligne du graphe de symboles, les traces et les lignes verified sont omises.)
 <Panel> dans App a pour origine (b/Widget.tsx, Widget) : l'alias est résolu, l'enfant est inliné avec un setter d'App, et l'ERROR de b/Widget.tsx n'existe que grâce à cet inlining. <Widget> dans Page n'a pas d'origine (Page.tsx n'importe rien) et deux fichiers définissent le nom : réponse Ambiguous, enfant inanalysable, INFO sur Page. Les noms affichés sont suffixés, et ne servent qu'à l'affichage.

Décision : ADR-40 §4 — refuser de deviner

L'ancien `get_by_name` répondait par la première clé (file, name) dans l'ordre de tri. Ce n'était pas non déterministe, c'était *faux* : un leurre `a/Widget.tsx` remplaçait le `./b/Widget` importé, et l'ERROR qui dépendait du vrai corps disparaissait (« ✓ no issues found »). Désormais Ambiguous rend l'enfant inanalysable, comme un composant hors de l'exécution, et le parent porte une INFO analysis-limit. **Alternative refusée** : analyser chaque candidat d'un nom ambigu. C'est sound et plus précis, et cela a été écarté sur mesure : 1 347 références ambiguës sur les quatorze dépôts du corpus (huit n'en ont aucune), contre 24 500 références inconnues déjà rapportées (+5,5 %) ; <Button/> seul compte 1 453 sites ambigus quand le corpus est vu comme un seul arbre, et payer une analyse d'enfant par candidat à chacun est la forme du blocage quadratique de #86. Le résidu vient surtout d'alias `@workspace/*` non résolus (#48), dont la correction supprime la cause.

Piège

Le repli se fait sur `name`, le nom *écrit* au site, et non sur `o.name`, le nom exporté que porte l'origine. Un alias importé à travers un barrel (`import Widget as Panel from "../index"`, où `index.ts` ne fait que ré-exporter) cherche donc `Panel` et répond `Unknown`, même si un seul fichier définit `Widget` (lecture de `src/engine/component_registry.rs`, l. 115–126 ; le test `an_origin_pointing_at_no_component_falls_back_to_the_name` utilise le même nom des deux côtés). Direction : un enfant inanalysable, donc havoc et INFO — une perte de précision, pas de soundness. Inversement, un fichier voit toujours ses propres déclarations dans sa carte d'origines : un `<Child/>` défini dans le même fichier est résolu par l'origine, même si d'autres fichiers définissent un `Child`. Vérifié au e67b10a sur un répertoire à trois fichiers (`Widget.tsx`, un barrel `index.ts` qui ré-exporte `Widget`, et `App.tsx` qui importe `Widget as Panel from "../index"`) : `-info -verbose` répond INFO « component 'Panel' was not found in the analysis registry », pas « several analysed files define », bien qu'un seul fichier définisse `Widget`.

16.3 Les racines de l'analyse

L'inlining descendant a besoin d'un point de départ : les composants analysés sans parent.

Définition 16.1: Racine

Une *racine* est un composant analysé en tête de la phase 1, sans parent : son environnement d'entrée est vide (`AbstractEnv::bottom()`), donc son paramètre `props`, non lié, se lit `⊥`. Les enfants qu'elle rend sont inlinés avec les props qu'elle leur calcule.

Extrait 16.13 – `src/engine/root_detector.rs`, l. 25–36 — les trois stratégies

```
25 /// Strategy for selecting root components (entry points for top-down analysis).
26 pub enum RootStrategy {
27     /// Default: components that do not appear in any `CompApp` node.
28     Heuristic,
29     /// `--all-roots`: every component analyzed as a root (props = ⊥ if not inlined).
30     AllComponents,
31     /// `--entry Foo,Bar`: explicit list. A bare `Foo` makes every `(file, name)`
32     /// entry called `Foo` a root; the qualified `Foo@src/a/Foo.tsx` form – what
33     /// [crate::ir::ComponentTable::display_name`] mints for a collision, and
34     /// what the report prints back – selects exactly one.
35     Explicit(Vec<Symbol>),
36 }
```

Le driver choisit la stratégie d'après la ligne de commande : `-entry` l'emporte, puis `-all-roots`, sinon l'heuristique (`src/driver/mod.rs`, l. 282–288).

16.3.1 L'heuristique : ce que personne ne rend

Extrait 16.14 – `src/engine/root_detector.rs`, l. 57–91 — la détection des racines, stratégie par stratégie

```
57 RootStrategy::Heuristic => {
58     // A reference the lowering resolved rules out exactly one
59     // component; one it did not rules out every component of that
60     // name, because any of them could be the one meant. Marking by
61     // name alone made an aliased or renamed callee (`<Panel/>` for
62     // `Widget`) leave its target looking unreferenced, so the
63     // target was analysed a second time as a root and that pass
64     // overwrote the precise result its parent had produced (#7).
```

```

65     let mut refs: HashSet<CompAppRef> = HashSet::new();
66     for comp in registry.all_components() {
67         collect_compapp_in_component(comp, &mut refs);
68     }
69     let table = registry.table();
70     let mut referenced: HashSet<ComponentId> = HashSet::new();
71     for r in &refs {
72         match &r.origin {
73             Some(o) => referenced.extend(table.id_of(o)),
74             // Nothing settles the reference, so every component of
75             // that name may be the one meant and none of them is
76             // provably a root.
77             None => referenced.extend(table.ids_named(&r.name)),
78         }
79     }
80     table.ids().filter(|id| !referenced.contains(id)).collect()
81 }
82 RootStrategy::AllComponents => registry.table().ids().collect(),
83 RootStrategy::Explicit(names) => {
84     let mut ids: Vec<ComponentId> = names
85         .iter()
86         .flat_map(|name| explicit_matches(registry, name))
87         .collect();
88     ids.sort();
89     ids.dedup();
90     ids
91 }

```

Une racine heuristique est un composant qu'*aucun* `CompApp` du programme ne référence. La collecte, `collect_compapp_refs` (`src/engine/root_detector.rs`, l. 126–139), est syntaxique et sur-approximante : elle parcourt le rendu, les corps d'effets, de mémos, de callbacks et de handlers, et descend explicitement dans les `FnLit` imbriqués (un callback de `.map`, une `render prop`), que `for_each_child` ne traverse pas (`src/engine/root_detector.rs`, l. 145–160). Chaque référence garde le nom écrit *et* l'origine, parce que la même collecte sert aussi la relation des consommateurs de contexte (#115), qui a besoin de l'autre projection.

La règle de marquage mérite d'être lue lentement. Une référence résolue exclut exactement un composant. Une référence *non* résolue exclut *tous* les composants de ce nom, car n'importe lequel peut être celui qui est rendu. Le commentaire du code dit ce qui arrivait avant #7 : marquer par nom seul laissait la cible d'un alias (`<Panel/>` pour `Widget`) sans référence apparente ; elle était analysée une seconde fois comme racine, avec des props `⊥`, et cette passe *écrasait* le résultat précis que le parent avait produit. Nous retrouverons ce phénomène d'écrasement au section 16.6.

Exemple 16.3: Racines, récursion, composant inconnu

Extrait 16.15 – `/tmp/ch16/roots.tsx`

```

1  import { useState } from "react";
2
3  function Leaf({ n }: { n: number }) { return <span>{n}</span>; }
4  function Tree({ depth }: { depth: number }) {
5      return <Tree depth={depth - 1} />;
6  }
7  export function App() {
8      const [n, setN] = useState(0);
9      return (
10         <div>
11             <Leaf n={1} />

```

```

12     <Leaf n={1} />
13     <Tree depth={3} />
14     <Unknown onPick={setN} />
15   </div>
16 );
17 }
18 export function Orphan() { return <Leaf n={2} />; }

```

```

$ reactant check roots.tsx --info --verbose
[verbose] symbol graph: 4 nodes, topo order = [Tree@roots.tsx, Leaf@roots.tsx, Orphan@roots.tsx, App@roots.tsx]
[verbose] 2 components analyzed
[verbose] cache hits: 5, misses: 3
[verbose] App: 1 iteration(s), widened: []
[verbose] Leaf: 0 iteration(s), widened: []
[verbose] Orphan: 0 iteration(s), widened: []
[verbose] Tree: 0 iteration(s), widened: []
App (1 hooks) roots.tsx
  info analysis-limit component `Unknown` was not found in the analysis registry. Pass its file on the command
    ↳ line to analyse it (FN possible)
  suspended analysis-limit 4 passing check(s) withheld: the analysis was truncated in this component, so they
    ↳ are not guaranteed
Tree (0 hooks) roots.tsx
  info analysis-limit recursive component reference `Tree` is not followed, so its props are treated as unknown
    ↳ ; cross-component cycles are not fully analysed (FN possible)

✓ 1 file(s) no issues found.

```

Leaf est référencé; Tree se référence lui-même, il n'est donc pas racine; les racines sont App et Orphan. « 2 components analyzed » compte les deux racines et rien d'autre : le compteur `components_analyzed` n'est incrémenté que par les deux boucles d'`analyze_program` (`src/engine/fixpoint.rs`, l. 751, `src/engine/fixpoint.rs`, l. 791), pas par l'inlining des enfants — son commentaire (« including re-analyses due to fixpoint », `src/engine/program_result.rs`, l. 208) est inexact. App fait une itération de plus que nécessaire parce que `setN`, passé à l'inconnu `<Unknown>`, est havoqué (section 16.5). Les nombres de succès et de ratés de cache s'expliqueront au section 16.4.

16.3.2 -all-roots et -entry

Avec `AllComponents`, tout composant est une racine, analysée avec des props `T` si aucun parent ne l'a inliné avant. Avec `Explicit`, la liste vient de l'utilisateur : `explicit_matches` (`src/engine/root_detector.rs`, l. 44–50) sélectionne, pour un nom nu `Foo`, *tous* les composants écrits `Foo`, et pour la forme qualifiée `Foo@src/a/Foo.tsx` — celle-là même que le rapport imprime en cas de collision — un seul, via `resolve_display_name`. Une seule fonction sert au calcul des racines et à la détection des noms qui ne sélectionnent rien, pour que les deux ne puissent pas diverger :

Extrait 16.16 – `src/engine/root_detector.rs`, l. 101–110 — les noms d'entrée qui ne sélectionnent rien

```

101 pub fn unmatched(&self, registry: &ComponentRegistry) -> Vec<Symbol> {
102     match self {
103         RootStrategy::Explicit(names) => names
104             .iter()
105             .filter(|name| explicit_matches(registry, name).is_empty())
106             .cloned()
107             .collect(),
108         RootStrategy::Heuristic | RootStrategy::AllComponents => Vec::new(),
109     }
110 }

```


Le driver en fait une erreur d'usage (`src/driver/mod.rs`, l. 307–321) : une faute de frappe dans `-entry` réduisait autrefois l'exécution, en silence, à une analyse intra, et coûtait tous les résultats inter-composants avec un rapport par ailleurs propre.

```
$ reactant check intro.tsx --entry Nope
[error] --entry: no component named `Nope`
[error] name a component defined in the analysed files, `Name@path` to pick one of several with the same name
```

Piège

Une racine explicite mal choisie ne produit pas d'erreur, seulement moins de résultats. Sur l'liste 16.1, `-entry Badge` fait de `Badge` la racine, analysée avec des props `T` ; `Inbox`, non atteint, passe en phase 2 (intra) et y havoque le setter qu'il passe à `<Badge>`. L'ERROR disparaît (observé : « ✓ 1 file(s) no issues found », `Inbox`: 1 iteration(s)). À l'inverse, `-all-roots` la garde sur ce fichier, mais seulement parce que `Badge` (id 0) est analysé comme racine *avant* `Inbox` (id 1), qui l'inline ensuite et écrase le résultat imprécis : l'ordre des identifiants décide (section 16.6).

16.3.3 Un trou de la collecte, sans danger

`collect_compapp_refs` ne lit pas les arguments d'un `HookEntry::Custom` (branche `_ =>`, `src/engine/root_detector.rs`, l. 136). Un élément passé à un hook opaque — `useModal(<Dialog onClose=setS/>)`, `useModal` venant d'un paquet — ne marque donc pas `Dialog` comme référencé, et `Dialog` devient une racine, analysée avec des props `T`. Le dossier de notes l'a relevé sur la sonde (racines `App` et `Dialog`) ; le CLI montre les deux composants analysés hors inlining (`/tmp/ch16/hookarg.tsx`, « 2 components analyzed », « cache hits: 0, misses: 0 ») et la seule INFO sur le hook `useModal`. C'est une sur-approximation des racines, donc sûre ; on la note parce que `SymbolGraph`, lui, parcourt ces arguments (`src/engine/symbol_graph.rs`, l. 271–275) : deux collectes « syntaxiques » du même périmètre ne voient pas les mêmes choses.

16.4 L'inlining descendant

16.4.1 Le contexte partagé `InterCtx`

Une racine est analysée par `analyze_component_impl` (chapitre 13), avec un argument de plus : un contexte inter-composants.

Extrait 16.17 – `src/domains/context.rs`, l. 59–77 — le contexte d'une analyse inter-composants

```
59 pub struct InterCtx<'a> {
60     pub registry: &'a ComponentRegistry,
61     pub cache: &'a RefCell<ComponentCache>,
62     pub shared_state: &'a RefCell<SharedStateStore>,
63     pub call_graph: &'a RefCell<ComponentCallGraph>,
64     pub stats: &'a RefCell<AnalysisStats>,
65     /// All analysis results accumulated across the entire program analysis.
66     pub results: &'a RefCell<std::collections::HashMap<ComponentId, AnalysisResult<StateValue>>>,
67     /// Components currently being analyzed (for recursion detection).
68     pub call_stack: RefCell<Vec<ComponentId>>,
69     /// The component being analyzed at this level.
70     pub component: ComponentId,
71     /// Analysis config (widen_threshold etc.).
72     pub config: &'a Config,
73     /// Callback provided by `engine::fixpoint` to inline a child component's analysis.
74     pub analyze_child: AnalyzeChildFn,
75     /// User-defined custom hook registry for inlining (None = no inlining).
```



```
76   pub hook_registry: Option<&'a HookRegistry>,
77 }
```

Tout l'état mutable partagé — cache, store partagé, graphe d'appel, statistiques, table des résultats — est derrière des `RefCell`, elles-mêmes derrière des références partagées : on passe `&InterCtx` partout sans conflit d'emprunts mutables. Deux champs sont propres à un niveau : `component`, le composant en cours, et `call_stack`, la pile de ses ancêtres. `child(c)` fabrique le contexte d'un enfant en copiant toutes les références et en empilant le composant courant sur une pile neuve; `is_recursive(id)` teste `call_stack.contains(&id)` || `component == id` (`src/domains/context.rs`, l. 80–103; la pile ne contient que les ancêtres, d'où le second disjoint, chapitre 10). Enfin `analyze_child` est un pointeur de fonction, `analyze_component_inter` (`src/engine/fixpoint.rs`, l. 62–81), fourni par le moteur : il casse le cycle de modules entre le transfert (`domains`), qui rencontre l'élément, et le point fixe (`engine`), qui sait analyser un composant.

16.4.2 eval_comp_app, pas à pas

L'évaluation d'un `Expr::CompApp` dans le domaine des valeurs délègue à `eval_comp_app` (`src/domains/transfer/state_value.rs`, l. 158–163). La fonction tient en 125 lignes, que nous lisons en trois morceaux.

Extrait 16.18 – `src/domains/transfer/state_value.rs`, l. 476–518 — sans contexte `inter`, puis résolution de l'enfant

```
476 ) -> StateValue {
477     let Some(inter) = ctx.inter else {
478         // Intra-component analysis: every child is unanalyzable, so any
479         // setter escaping into props may be invoked with any argument -
480         // same argument as the unknown-child branch below (TODO.md B).
481         havoc_setter_props(props_expr, env, ctx);
482         return StateValue::reference(Stability::Stable);
483     };
484
485     // Registry lookup. The JSX callee name is only how the child was
486     // *written*; who it is, is the `ComponentId` the registry resolves it to,
487     // and that id is what the call stack, the call graph, the cache and the
488     // results map are all keyed by (ADR-040).
489     let child = match inter.registry.resolve_child(name, origin) {
490         ChildLookup::Resolved(key) => match inter.registry.id(&key) {
491             Some(id) => id,
492             // Unreachable: `resolve_child` only answers with a key this
493             // registry holds, and every one of those was interned.
494             None => return StateValue::reference(Stability::Stable),
495         },
496         // A child the analysis cannot pin down may invoke any setter it
497         // receives, with any argument, at any time (<Sheet
498         // onOpenChange={setOpen}>`). Havoc those state slots - leaving them
499         // untouched under-approximates state and fabricates "state is stable"
500         // conclusions (TODO.md F4). Resolved children don't need this: their
501         // setter calls are modeled precisely by the inter-component analysis.
502         //
503         // The two unresolved answers are recorded apart because they ask the
504         // user for different things: one to widen the run, one to disambiguate
505         // the reference (#7).
506         unresolved => {
507             {
508                 let mut stats = inter.stats.borrow_mut();
509                 let refs = match unresolved {
```

```

510         ChildLookup::Ambiguous => &mut stats.ambiguous_component_refs,
511         _ => &mut stats.unknown_component_refs,
512     };
513     refs.insert((inter.component, name.clone()));
514 }
515 havoc_setter_props(props_expr, env, ctx);
516 return StateValue::reference(Stability::Stable);
517 }
518 };

```

1. **Pas d'InterCtx** : analyse d'un composant seul, phase 2, ou passe de rafraîchissement (section 16.4.4). Tout enfant est alors inanalysable : on havocue les setters qui s'échappent dans ses props et on rend Stable.
2. **Résolution** par `resolve_child` (liste 16.12), puis `ComponentId`. `Unknown` et `Ambiguous` sont enregistrés séparément dans les statistiques — ils demandent à l'utilisateur deux remèdes différents, élargir l'exécution ou désambiguïser — et deviendront des `INFO analysis-limit` (`src/rules/impls/analysis_limit_info.rs`, l. 53–77) ; puis `havoc` et `Stable`.

Extrait 16.19 – `src/domains/transfer/state_value.rs`, l. 520–556 — récursion, props abstraites, cache

```

520 // Recursion guard – before the IR clone, which is not free.
521 if inter.is_recursive(child) {
522     let mut stats = inter.stats.borrow_mut();
523     stats.recursion_cutoffs += 1;
524     stats
525         .recursive_component_refs
526         .insert((inter.component, child));
527     return StateValue::reference(Stability::Stable);
528 }
529 // Unreachable – `resolve_child` just resolved it – but an unknown child may
530 // invoke any setter it received, so the fallback is the unknown-child one,
531 // never a silent skip.
532 let Some(child_ir) = inter.registry.ir_of(child).cloned() else {
533     havoc_setter_props(props_expr, env, ctx);
534     return StateValue::reference(Stability::Stable);
535 };
536
537 // Evaluate props → abstract map (EnvVals, preserving heap Locs for FnLit props)
538 let abstract_props_full = eval_props_map(props_expr, env, ctx);
539
540 // Flatten to StateValues for cache lookup and call graph recording
541 let abstract_props: HashMap<Symbol, StateValue> = abstract_props_full
542     .iter()
543     .map(|(k, ev)| (k.clone(), ev.as_val()))
544     .collect();
545
546 // Cache lookup (strict equality)
547 if inter
548     .cache
549     .borrow()
550     .lookup(child, &abstract_props)
551     .is_some()
552 {
553     inter.stats.borrow_mut().cache_hits += 1;
554     record_call_site(inter, child, abstract_props, None);
555     return StateValue::reference(Stability::Stable);
556 }

```

3. **Récursion** : si l'enfant est déjà sur la pile, on compte une coupure, on note la paire (courant, enfant) et on rend *Stable* — *avant* le clonage de l'IR, qui coûte. Il n'y a *pas* de havoc ici, contrairement aux branches Unknown/Ambiguous (section 16.4.5).
4. **Props** : `eval_props_map` évalue chaque champ de l'objet props dans l'environnement courant du parent et rend une carte `prop ↦ EnvVal`, qui garde les sites du tas d'une prop fermeture ou objet. Sa version aplatie en `StateValue` sert de clé au cache et au graphe d'appel.
5. **Cache** : un succès (égalité de treillis des props, section 16.6) enregistre une arête du graphe d'appel et rend *Stable sans ré-analyser l'enfant*.

Extrait 16.20 – `src/domains/transfer/state_value.rs`, l. 558–593 — l'environnement de l'enfant, son analyse, l'enregistrement

```

558
559 // Build child initial env + heap:
560 // - copy heap entries for any Loc-valued props (FnLit bodies) into child's heap
561 // - insert the Obj (with full EnvVals) so the child can resolve FieldAccess → Loc
562 let mut child_env = AbstractEnv::bottom();
563 let props_id = ExprId::fresh();
564 let mut initial_heap = crate::domains::stores::Heap::new();
565 for ev in abstract_props_full.values() {
566     if let EnvVal::Loc { ids, .. } = ev {
567         for &id in ids {
568             if let Some(hv) = ctx.heap.get(id) {
569                 initial_heap.insert(id, hv.clone());
570             }
571         }
572     }
573 }
574 initial_heap.insert(props_id, HeapValue::Obj(abstract_props_full.clone()));
575 child_env.extend_loc(child_ir.param.clone(), props_id);
576
577 // Create child inter context and analyze
578 let child_inter = inter.child(child);
579 let analyze_child = inter.analyze_child;
580 let child_result = analyze_child(&child_ir, child, child_env, initial_heap, &child_inter);
581
582 // Store result in the program-level results map and cache
583 inter
584     .results
585     .borrow_mut()
586     .insert(child, child_result.clone());
587 inter
588     .cache
589     .borrow_mut()
590     .insert(child, abstract_props.clone(), child_result);
591 record_call_site(inter, child, abstract_props, None);
592
593 StateValue::reference(Stability::Stable)

```

6. **Environnement de l'enfant** : un environnement vide, un site frais `props_id = ExprId::fresh()` sous lequel on range, dans un tas neuf, l'objet des props (`HeapValue::Obj`), plus une copie des valeurs de tas des props qui ont des sites; le paramètre de l'enfant est lié à ce site (`extend_loc`). C'est l'`AbstractObject` d'ADR-12 §9 : un accès `props.onSeen` dans l'enfant lit le tas et retrouve le setter du parent.
7. **Analyse** : `analyze_child` lance une analyse *complète* de l'enfant, avec son propre point fixe, sous le contexte fils.
8. **Enregistrement** : le résultat est rangé dans `results` (en *remplaçant* celui qui s'y trouvait)

et dans le cache, et une arête est ajoutée au graphe d'appel. La valeur de l'élément est, dans tous les cas, *Stable*.

La figure 16.2 résume ces échanges sous forme de diagramme de séquence.

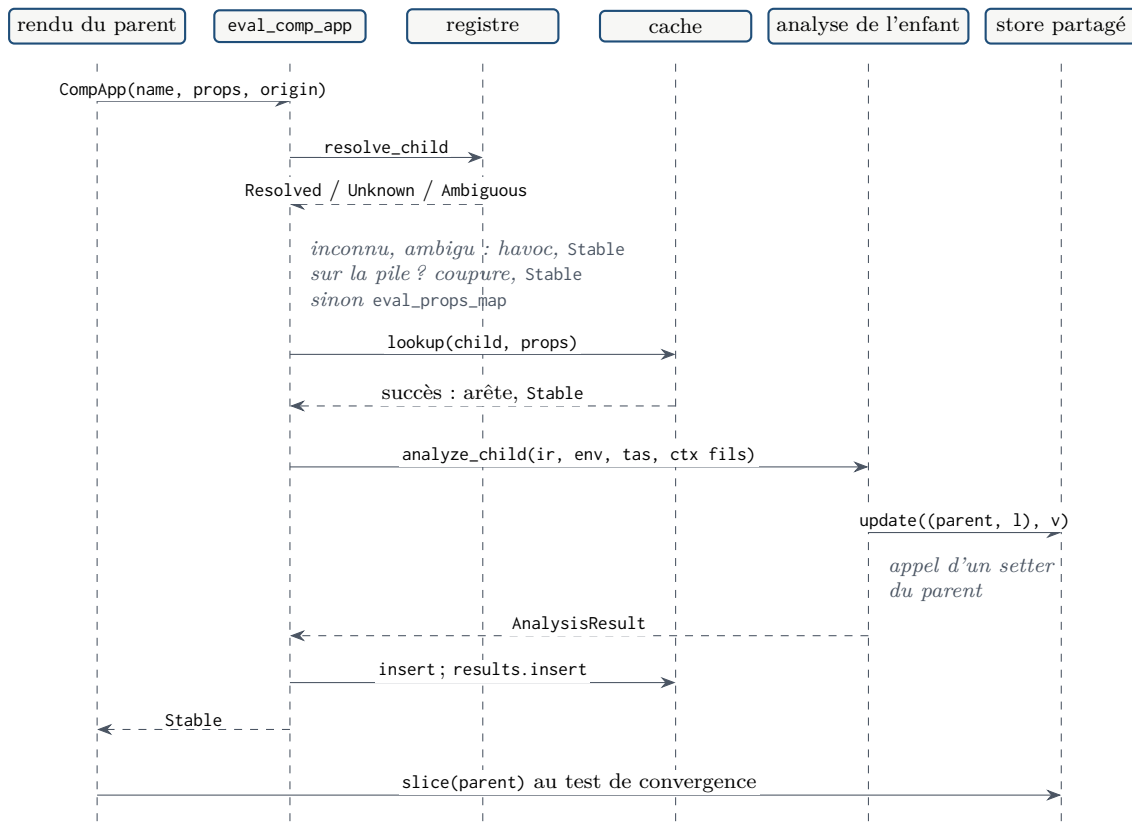


FIGURE 16.2 – Diagramme de séquence de l'évaluation d'un élément `<Child/>` sous un `InterCtx`. Les retours sont en pointillés. L'écriture de l'enfant dans le store partagé n'est lue par le parent qu'au test de convergence de sa boucle, qui relance un tour si elle fait monter son état.

16.4.3 Les props dans le tas

Pourquoi passer les props par le tas plutôt que par l'environnement ? Parce qu'un enfant doit pouvoir *exécuter* une fermeture reçue en prop. `eval_props_map` (`src/domains/transfer/state_value.rs`, l. 653–689) alloue dans le tas du parent un `FnLit` passé en ligne (`onTick=() => setB(b + 1)`), avec ses captures, et garde son site ; une prop qui aliasse un littéral déjà alloué garde aussi ses sites (`resolve_locs`). L'enfant reçoit une copie de ces entrées : quand son effet appelle `onTick()`, le corps s'exécute avec ses captures, et l'appel de `setB` a pour propriétaire le parent (chapitre 11, exemple « une boucle croisée »).

Deux limites de cette frontière sont traitées ailleurs, et il suffit ici de les situer. Seuls les sites *directs* des props sont copiés : une fermeture rangée comme *membre* d'un objet passé en prop arrive dans l'enfant sans son corps (défaut D11 du dossier 05, chapitre 11). Et les sites copiés gardent les `ExprId` du fichier parent, qui peuvent entrer en collision avec ceux que l'enfant alloue dans son propre fichier (chapitres 6 et 8).

16.4.4 Combien de fois un élément est-il évalué ?

Les compteurs de cache de l'exemple 16.3 (5 succès, 3 ratés) paraissent énigmatiques tant qu'on ne sait pas combien de fois un `CompApp` est évalué. Trois faits, vérifiés dans le code, suffisent.

1. La passe de rendu d'un composant tourne `iterations + 1` fois sous `InterCtx` : la boucle externe sort sur `new_state.leq(&state)` *avant* d'incrémenter `iteration` (`src/engine/fixpoint.rs`, l. 355–499).
2. Elle est suivie d'une passe de rafraîchissement *sans* `InterCtx` (`src/engine/fixpoint.rs`, l. 532–560), qui ne ré-inline rien : elle passe par la branche « pas d'`InterCtx` » d'`eval_comp_app`.
3. Dans une passe, un élément imbriqué dans un élément hôte est évalué *une* fois (bras `CompApp` d'`exec_callbacks_depth`, `src/domains/interp/interpreter.rs`, l. 529–533); un élément qui est *directement* l'expression du `return` l'est *deux* fois : par ce même bras, puis par l'appel d'`exec_expr_effects` sur le terminateur `Return` (`src/domains/interp/interpreter.rs`, l. 123–125, `src/engine/cfg_analyzer.rs`, l. 84–86). La seconde évaluation est un succès de cache.

Exemple 16.4: Le return direct compte double

Dans `/tmp/ch16/ret.tsx`, fonction `A()` `return <Leaf n=1 />`; et fonction `B()` `return <div><Leaf n=2 /></div>`; `-verbose` rapporte « cache hits: 1, misses: 2 ». Un raté par valeur de props, et un seul succès : la seconde évaluation du `return` direct d'`A`. Sur `roots.tsx`, `App` (`iterations = 1`, donc deux passes, éléments imbriqués dans `<div>`) évalue ses quatre éléments deux fois; `<Unknown>` ne touche pas le cache; `Orphan` (`return <Leaf n=2 />` direct) évalue son élément deux fois. On obtient trois ratés (`Leafn:1`, `Tree`, `Leafn:2`) et cinq succès (le second `<Leaf n=1/>` de la première passe, les trois éléments résolus de la seconde passe d'`App`, la seconde évaluation d'`Orphan`). Même raisonnement sur `drill.tsx` : « cache hits: 3, misses: 7 » (exercice 16.1).

Le coût d'un élément est donc au plus une analyse de l'enfant par valeur de props distincte, bornée par le cache, mais cette analyse est elle-même récursive (les enfants de l'enfant) : le coût total dépend de la profondeur de l'arbre et du nombre de passes de chaque ancêtre. Il n'y a pas de borne plus fine dans le code ; la pile interdit seulement les cycles.

16.4.5 La récursion : une coupure, pas un havoc

Un composant récursif est légitime en React — un arbre, un menu imbriqué — et chaque instance a son propre état. L'analyse, elle, ne suit pas la récursion : ADR-12 §11 prévoyait une coupure « façon MOPSA » avec un résultat `T` et une hypothèse `A_ignore_recursion` enregistrée.

Exemple 16.5: Un setter remis à l'instance intérieure

Extrait 16.21 – `/tmp/ch16/rec.tsx`

```
1 import { useState } from "react";
2 function Tree({ depth, onHit }: { depth: number; onHit?: (v: number) => void }) {
3   const [x, setX] = useState(0);
4   if (depth < 3) onHit?.(1);
5   return <div>{x}{depth > 0 ? <Tree depth={depth - 1} onHit={setX} /> : null}</div>;
6 }
7 export function App() {
8   return <Tree depth={3} />;
9 }
```

À l'exécution, l'instance intérieure appelle `onHit`, c'est-à-dire le `setX` de l'instance extérieure, pendant son rendu : un vrai `cross-setter-in-render`.

```
$ reactant check rec.tsx --info --show-clean
Tree (1 hooks) rec.tsx
  info analysis-limit recursive component reference `Tree` is not followed, so its props are treated as unknown
    ↳ ; cross-component cycles are not fully analysed (FN possible)
  suspended analysis-limit 4 passing check(s) withheld: the analysis was truncated in this component, so they
    ↳ are not guaranteed

✓ 1 file(s) no issues found.
```

L'analyse inline Tree une fois depuis App ; l'élément `<Tree .../>` intérieur tombe sur la coupure, qui rend Stable *sans* havoc : `x` n'est pas mis à $\top$, et l'écriture de l'instance intérieure n'est vue par personne. Le faux négatif est *déclaré* — INFO analysis-limit et suspension des assurances « verified » du composant tronqué — et c'est cette suspension, non une sur-approximation de l'état, qui protège les garanties publiées.

Piège

Le texte de l'INFO (« its props are treated as unknown ») décrit l'intention d'ADR-12, pas le code : l'instance récursive n'est pas analysée, ses props ne sont ni évaluées ni havoquées. De même, les commentaires de `AnalysisStats::recursive_component_refs` (« cut to $\top$ », `src/engine/program_result.rs`, l. 210) et de `ProgramAnalysisResult::recursive_components` (« received $\top$ result », `src/engine/program_result.rs`, l. 33) sont inexacts : la coupure rend Stable et ne met rien à $\top$ (`src/domains/transfer/state_value.rs`, l. 520–528). Havoquer ici, avec le même mécanisme que la branche Unknown, serait la correction centrale ; le choix n'est pas tranché au snapshot.

16.5 Le flux ascendant : setters, store partagé, havoc

16.5.1 Un setter qui porte son propriétaire

ADR-12 §7 introduisait une valeur `ComponentSetter component, label`, transposition du `Set_clos label, path` de React-tRace [LAY25] : un setter qui sait à quel composant il appartient. Elle est devenue la composante setter du produit `StateValue` (définition 10.1), dont `as_setter()` rend le couple `(ComponentId, HookLabel)`. Quand un parent passe `setSeen`, la prop `onSeen` de l'enfant vaut ce setter, propriétaire compris. L'interpréteur n'a plus qu'à regarder, à chaque appel, si le callee s'évalue en setter :

Extrait 16.22 – `src/domains/interp/interpreter.rs`, l. 368–391 — l'appel d'un setter porteur de propriétaire écrit le store partagé

```
368 // Cross-component ComponentSetter call.
369 // Handles fn_ = Var(name), FieldAccess { obj: Var, field }, or any other expr
370 // that evaluates to ComponentSetter { component, label }.
371 if let Expr::Call { fn_, args } = expr {
372     let comp_setter = transfer
373         .eval_expr(fn_, env, ctx)
374         .as_state_value()
375         .and_then(|sv| sv.as_setter().map(|(c, l)| (*c, *l)));
376     if let Some((component, label)) = comp_setter
377         && ctx.inter.is_some()
378     {
379         let arg_val = args
380             .first()
381             .map(|a| transfer.eval_expr(a, env, ctx))
382             .and_then(|v| v.as_state_value())
383             .unwrap_or(crate::domains::StateValue::top());
384         if let Some(inter) = &ctx.inter {
```

```

385         inter
386         .shared_state
387         .borrow_mut()
388         .update(component, label, arg_val);
389     }
390 }
391 }

```

L'argument (le premier, ou $\top$ s'il manque ou ne se lit pas) est joint dans l'entrée (propriétaire, label) du store partagé. Le bras est gardé par `ctx.inter.is_some()` : sans contexte `inter`, il ne fait rien — mais sans contexte `inter`, aucun setter étranger ne peut non plus arriver dans l'environnement.

16.5.2 Le store partagé, et pourquoi il n'y a pas de point fixe de programme

Extrait 16.23 – `src/domains/stores/shared_state_store.rs`, l. 29–34 — une mise à jour monotone

```

29  /// Monotone update: `self[(comp, label)] = self[(comp, label)]  $\sqcup$  val`.
30  pub fn update(&mut self, comp: ComponentId, label: HookLabel, val: StateValue) {
31      let key = (comp, label);
32      let current = map_get_or(&self.entries, &key, StateValue::bottom);
33      self.entries.insert(key, current.join(&val));
34  }

```

Le store partagé (définition 11.2) associe à chaque couple (`ComponentId`, `HookLabel`) une valeur, $\perp$ pour une clé inconnue, et ne fait que croître. La boucle externe du parent importe sa tranche à chaque test de convergence :

Extrait 16.24 – `src/engine/fixpoint.rs`, l. 486–497 — le parent importe les écritures de ses enfants

```

486  let new_state_incycle = state_from_render.join(&state_from_effects);
487  // Include cross-component state updates made by child effects/callbacks.
488  let external_updates = inter
489      .map(|i| i.shared_state.borrow()).slice(comp_id)
490      .unwrap_or_else(StateStore::bottom);
491  let new_state = new_state_incycle
492      .join(&state_from_handlers)
493      .join(&external_updates);
494
495  if new_state.leq(&state) {
496      break;
497  }

```

Décision : ADR-12 §1, §5, §8 — inlining descendant, flux bidirectionnel, pas de couche programme

Le parent est analysé d'abord, et l'enfant est analysé au point d'application avec les props abstraites ; pas de résumés paramétriques ascendants, qui demanderaient une couche d'abstraction supplémentaire « without justified gain at the current stage ». Le flux est bidirectionnel : les props descendent, les setters remontent. Et il n'y a « no separate program-level fixpoint layer » : les écritures d'un enfant sont des mutations du store partagé, « indistinguishable from local setters from the fixpoint's perspective ». La sensibilité au contexte est obtenue par le cache (une analyse par valeur de props). Écarts avec le texte : les clés sont des `ComponentId`, pas des `Symbol` (ADR-40) ; la récursion rend `Stable`, pas $\top$.

Proposition 16.1: L'import d'un store monotone préserve la convergence

Soit S_i l'état du parent au tour i et X_i sa tranche du store partagé lue à ce tour. Si $X_i \sqsubseteq X_{i+1}$ pour tout i (le store ne fait que croître) et si le nouvel état est $S'_i = F(S_i) \sqcup X_i$ avec F monotone, la boucle externe du parent, munie de son widening (chapitre 13), s'arrête sur un état S tel que $F(S) \sqcup X \sqsubseteq S$, où X est la tranche lue au dernier tour.

Esquisse. La suite des tranches est croissante dans un domaine où le widening force la stabilisation des slots ; le test `new_state.leq(&state)` est le même que pour un setter local, et une écriture d'enfant n'est, pour la boucle, qu'un terme de plus dans le join. La proposition ne dit rien de la tranche écrite *après* le dernier tour du parent (par un enfant analysé plus tard, sous une autre racine) : c'est l'une des raisons pour lesquelles la table des résultats n'est pas un point fixe de programme (section 16.6). $\square$

Sur l'liste 16.1, la chaîne est complète : `Inbox` rend `<Badge>`, qui est inliné ; le rendu de `Badge` appelle `onSeen(1)`, setter d'`Inbox` : `shared[(Inbox, 0)]  $\sqcup$  = [1, 1]`. Au test de convergence, `Inbox` importe sa tranche, son état monte, il fait un second tour (« `Inbox: 1 iteration(s)` » en `-verbose`), où `<Badge>` est un succès de cache (la prop `onSeen` n'a pas changé : « `cache hits: 1, misses: 1` »). La règle `cross-setter-in-render`, qui tourne sur `Badge`, voit dans son résultat un appel inconditionnel, pendant le rendu, d'une valeur setter dont le propriétaire est un autre composant.

16.5.3 Le havoc des setters qui s'échappent**Définition 16.2: Havoc**

*Havoc*er un slot, c'est y joindre $\top$ parce que son setter s'échappe vers un code que l'analyse ne peut pas suivre : un enfant inconnu, ambigu, ou toute analyse sans contexte inter. Un tel code peut appeler le setter n'importe quand, avec n'importe quel argument ; laisser le slot intact sous-approximerait l'état et fabriquerait des conclusions « l'état est stable ».

`havoc_setter_props` (`src/domains/transfer/state_value.rs`, l. 265–297, détaillé au chapitre 11) collecte, pour chaque champ de l'objet `props`, le setter nu (`as_setter()` sur la valeur) et les setters cachés derrière des fermetures, des objets du tas et des spreads (`collect_escaping_setters`). Puis :

Extrait 16.25 – `src/domains/transfer/state_value.rs`, l. 287–296 — un setter propre au composant ou d'un ancêtre

```

287     for (comp, label) in setters {
288         if comp == own {
289             ctx.state.update(label, StateValue::top());
290         } else if let Some(inter) = &ctx.inter {
291             inter
292                 .shared_state
293                 .borrow_mut()
294                 .update(comp, label, StateValue::top());
295         }
296     }

```

Un setter du composant courant est havocé dans son propre store ; un setter d'ancêtre, arrivé par une prop et transmis plus bas, l'est dans le store partagé, que l'ancêtre importera. Sur `roots.tsx`, c'est ce havoc de `setN`, remis à `<Unknown>`, qui fait faire à `App` un second tour.

Exemple 16.6: Un setter dans un littéral inline n'est pas havoqué

Extrait 16.26 – /tmp/ch16/hv.tsx

```

1 import { useState } from "react";
2 export function A() { const [n, setN] = useState(0); return <div>{n}<Unknown onPick={setN} /></div>; }
3 export function B() { const [n, setN] = useState(0); return <div>{n}<Unknown api={{ setN }} /></div>; }
4 export function C() { const [n, setN] = useState(0); const api = { setN }; return <div>{n}<Unknown api
   ↪   =>={api} /></div>; }
5 export function D() { const [n, setN] = useState(0); return <div>{n}<Unknown list={setN} /></div>; }

```

```

$ reactant check hv.tsx --verbose
[verbose] A: 1 iteration(s), widened: []
[verbose] B: 0 iteration(s), widened: []
[verbose] C: 1 iteration(s), widened: []
[verbose] D: 0 iteration(s), widened: []

```

(lignes [verbose] des composants seulement.) Le havoc, qui fait bouger `n` au premier tour et donc provoque un second tour, a lieu pour A (setter nu) et pour C (objet du tas), *pas* pour B ni D. `collect_escaping_setters` (`src/domains/transfer/state_value.rs`, l. 315–367) descend dans les champs d'un `ObjectLit` ou d'un `ArrayLit` syntaxique, mais n'y retient un champ `Var` que s'il désigne un site du tas; un setter nu en position de champ n'est poussé nulle part, et `havoc_setter_props` ne teste `as_setter()` que sur la valeur du champ de props entier. L'état reste à sa valeur initiale : c'est un trou de soundness du domaine des valeurs, dont le cas le plus courant est `<Ctx.Provider value= q, setQ >` — un provider est un composant inconnu du registre (section 16.10.6). Le dossier de notes n'a construit aucun diagnostic faux à partir de ce trou; il n'avait pas d'issue au snapshot.

16.6 Le cache de composants

16.6.1 Une analyse par valeur de props

Extrait 16.27 – `src/engine/component_cache.rs`, l. 19–27 — le cache, indexé par identité et props abstraites

```

19 /// Per-component analysis cache keyed by abstract props.
20 ///
21 /// Hit condition: strict lattice equality (`leq` in both directions).
22 /// On overflow: all entries are joined into one degraded entry (sound over-approximation).
23 #[derive(Debug)]
24 pub struct ComponentCache {
25     entries: HashMap<ComponentId, Vec<CacheEntry>>,
26     max_per_component: usize,
27 }

```

La clé est l'identité de l'enfant et ses props abstraites aplaties en `StateValue` (les sites du tas sont perdus à l'aplatissement). Chaque composant a au plus `DEFAULT_MAX_PER_COMPONENT = 5` entrées (`src/engine/component_cache.rs`, l. 10).

Extrait 16.28 – `src/engine/component_cache.rs`, l. 98–119 — égalité stricte : mêmes clés, $\sqsubseteq$ dans les deux sens

```

98
99 /// Strict equality: same keys, each value pair satisfies leq in both directions.
100 fn props_equal(a: &HashMap<Symbol, StateValue>, b: &HashMap<Symbol, StateValue>) -> bool {
101     if a.len() != b.len() {
102         return false;

```

```

103     }
104     for (k, va) in a {
105         let Some(vb) = b.get(k) else { return false };
106         let a_leq_b = matches!(
107             va.partial_cmp(vb),
108             Some(std::cmp::Ordering::Less | std::cmp::Ordering::Equal)
109         );
110         let b_leq_a = matches!(
111             vb.partial_cmp(va),
112             Some(std::cmp::Ordering::Less | std::cmp::Ordering::Equal)
113         );
114         if !(a_leq_b && b_leq_a) {
115             return false;
116         }
117     }
118     true
119 }

```

Le succès exige l'égalité de treillis, $a \sqsubseteq b \wedge b \sqsubseteq a$, champ par champ, après une comparaison des cardinaux. ADR-12 §2 l'explique : ne jamais réutiliser un résultat calculé pour une entrée moins précise (par exemple $\top$) quand l'entrée courante est plus précise.

Extrait 16.29 – `src/engine/component_cache.rs`, l. 65–92 — insertion, et éviction par join

```

65     /// Insert a new result. On overflow, evicts by joining all entries.
66     pub fn insert(
67         &mut self,
68         comp: ComponentId,
69         props: HashMap<Symbol, StateValue>,
70         result: AnalysisResult<StateValue>,
71     ) {
72         let entries = self.entries.entry(comp).or_default();
73         if entries.len() >= self.max_per_component {
74             // Evict: join all existing props + new props into a single degraded entry.
75             let all_props: Vec<&HashMap<Symbol, StateValue>> = entries
76                 .iter()
77                 .map(|e| &e.props)
78                 .chain(std::iter::once(&props))
79                 .collect();
80             let degraded_props = join_all_props(&all_props);
81             entries.clear();
82             entries.push(CacheEntry {
83                 props: degraded_props,
84                 result: Arc::new(result),
85             });
86         } else {
87             entries.push(CacheEntry {
88                 props,
89                 result: Arc::new(result),
90             });
91         }
92     }

```

Au-delà de cinq entrées, toutes les props sont jointes point à point, une clé absente d'une entrée comptant pour $\top$ (`join_all_props`, `src/engine/component_cache.rs`, l. 121–136), en une seule entrée dégradée.

Exemple 16.7: Égalité stricte et éviction

/tmp/ch16/cache1.tsx (<Leaf n=1/><Leaf n=1/> sous un parent sans état) : « cache hits: 1, misses: 1 » ; le second site est un succès. /tmp/ch16/cache.tsx (n vaut 1 à 6, puis encore 1) : « cache hits: 0, misses: 7 ». Au sixième insert, les cinq entrées et la nouvelle sont jointes en $n \in [1, 6]$; le septième site ($n = 1$) rate, car $[1, 1] \neq [1, 6]$, et il est ré-inséré. Le test unitaire `eviction_on_overflow` (`src/engine/component_cache.rs`, l. 281–299) vérifie qu’il ne reste qu’une entrée après débordement ; `top_props_match_anything_after_eviction`, malgré son nom, vérifie qu’un *Stable rate* contre une entrée $\top$ (`src/engine/component_cache.rs`, l. 301–326).

16.6.2 Un « déjà vu », pas une table de résultats

Voici la subtilité la plus importante du chapitre. `lookup` renvoie un `Arc<AnalysisResult>`, mais son seul appelant, `eval_comp_app`, ne regarde que `is_some()` (liste 16.19). Trois conséquences :

- le résultat stocké dans une entrée n’est *jamais relu* ;
- le résultat « de programme » d’un enfant est `results[child]`, remplacé à chaque raté par l’analyse du dernier site, jamais joint ;
- l’entrée dégradée associe des props jointes au résultat de la *dernière* insertion : la doc (« sound over-approximation ») ne vaut que pour la décision « sauter une analyse ».

Sauter une analyse est *sound* pour le *store partagé* : l’analyse sautée a déjà fait ses écritures, de façon monotone, lors du raté qui a rempli l’entrée. Elle ne l’est pas pour la table des résultats, que lisent toutes les règles intra-composant de l’enfant.

Exemple 16.8: Un résultat d’enfant écrasé : un faux négatif qui dépend de l’ordre

Extrait 16.30 – /tmp/ch16/overwrite.tsx

```
1 import { useState } from "react";
2
3 function Child({ onUpdate }: { onUpdate: (n: number) => void }) {
4   onUpdate(1);
5   return <div />;
6 }
7 export function A() {
8   const [n, setN] = useState(0);
9   return <Child onUpdate={setN} />;
10 }
11 export function B() {
12   return <Child onUpdate={() => {}} />;
13 }
```

```
$ reactant check overwrite.tsx
  2 clean component(s) hidden, rerun with --show-clean

✓ 1 file(s) no issues found.
```

Renommons A en Z, sans rien changer d’autre (/tmp/ch16/overwrite2.tsx) :

```
$ reactant check overwrite2.tsx
Child (0 hooks) overwrite2.tsx
  error cross-setter-in-render var:onUpdate (line 4:2) prop `onUpdate` (a state setter of parent `Z`) called
    ↳ during render of `Child`, which triggers a parent re-render on every render
```

(dernières lignes omises.) Les racines sont analysées dans l’ordre des identifiants, donc alphabétique dans un même fichier. Dans le premier cas, A inline Child avec un setter, puis

B l'inline avec une fonction vide — props différentes, raté — et *replace* results[Child]. La règle ne voit plus que le dernier site. Dans le second, Z passe en dernier et son site gagne.

L'erreur réelle d'A est perdue, sans INFO ni suspension : c'est un faux négatif non déclaré, qui viole l'invariant de soundness du projet. Il n'avait pas d'issue au snapshot (le dossier de notes l'a signalé). La même famille couvre `-all-roots` : chaque enfant y est ré-analysé comme racine, avec des props $\top$, et si son identifiant est plus grand que celui de son parent, ce dernier résultat l'emporte (la boucle de phase 1 ne saute pas une racine déjà présente dans results). La correction centrale n'est pas triviale : joindre les résultats de tous les sites, ou indexer les résultats par (identité, props) et faire lire aux règles l'ensemble des contextes d'un composant (exercice 16.3).

Piège

Deux résultats d'un même composant ne se joignent pas comme deux états : un AnalysisResult porte des CFG, des tables de hooks, des relations, des environnements par bloc. C'est pourquoi `exit_env` lui-même joint avec `reduce` et non avec `fold( $\perp$ , join)` : dans ce treillis d'environnements, $\perp$ n'est pas neutre pour le join, une clé absente de $\perp$ devenant $\top$ (`src/engine/analysis_result.rs`, l. 275–288).

16.7 Deux phases

Extrait 16.31 – `src/engine/fixpoint.rs`, l. 720–755 — phase 1 : les racines, sous un InterCtx

```

720     let roots = strategy.detect(&registry);
721     let mut analysed: std::collections::HashSet<ComponentId> = std::collections::HashSet::new();
722
723     // Phase 1: analyze roots top-down (children inlined via eval_comp_app).
724     for &root in &roots {
725         let Some(root_key) = registry.key_of(root) else {
726             continue;
727         };
728         if let Some(root_ir) = registry.ir_for(&root_key) {
729             let inter = InterCtx {
730                 registry: &registry,
731                 cache: &cache,
732                 shared_state: &shared_state,
733                 call_graph: &call_graph,
734                 stats: &stats,
735                 results: &results,
736                 call_stack: RefCell::new(vec![]),
737                 component: root,
738                 config,
739                 analyze_child: analyze_component_inter as AnalyzeChildFn,
740                 hook_registry: Some(&hook_registry),
741             };
742             let result = analyze_component_impl(
743                 root_ir,
744                 root,
745                 &StateValueTransfer,
746                 config,
747                 AbstractEnv::bottom(),
748                 Heap::new(),
749                 Some(&inter),
750             );
751             stats.borrow_mut().components_analyzed += 1;
752             results.borrow_mut().insert(root, result);

```

```

753         analysed.insert(root);
754     }
755 }

```

La phase 1 analyse chaque racine, dans l'ordre croissant des identifiants, avec un contexte inter neuf (pile vide) ; ses enfants sont inlinés à la volée, et leurs résultats s'accumulent dans `results`. La phase 2 (`src/engine/fixpoint.rs`, l. 764–794, commentée au chapitre 13) balaie tous les composants qui ne sont ni racines ni présents dans `results`, et les analyse *intra* : `inter = None`, `props T`, mais avec leur vrai `ComponentId`, parce qu'un label `Versioned` minté ici sera comparé à des résultats de phase 1. Entre les deux, une photographie :

Extrait 16.32 – `src/engine/fixpoint.rs`, l. 757–762 — ce que la phase 1 a atteint, photographié avant la phase 2

```

757 // Everything phase 1 reached: the roots plus every component
758 // `eval_comp_app` analysed top-down under an `InterCtx`. Snapshotted HERE,
759 // before the sweep below adds intra-only results that record no call-graph
760 // edges and would otherwise be indistinguishable from genuine roots (#110).
761 let phase1_reached: std::collections::HashSet<ComponentId> =
762     results.borrow().keys().copied().collect();

```

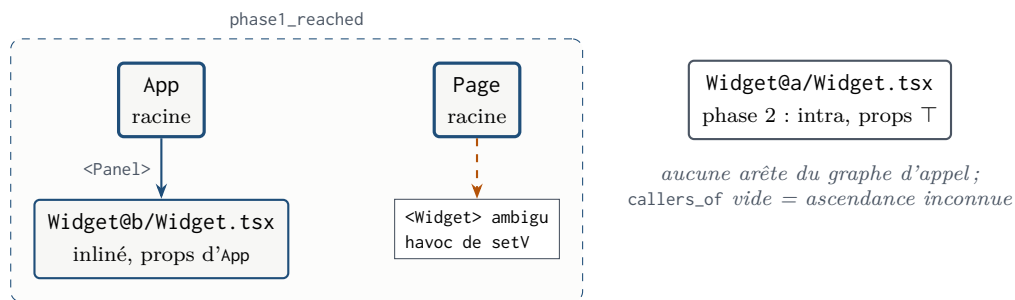


FIGURE 16.3 – Les deux phases sur l'exemple de la liste 16.3. En phase 1, les racines `App` et `Page` sont analysées ; `App` inline `Widget@b` par son origine ; la référence nue de `Page` est ambiguë, donc havoquée. Personne n'atteint `Widget@a`, que la phase 2 analyse seul. L'ensemble `phase1_reached` (cadre pointillé) est photographié entre les deux.

16.7.1 La discipline de lecture de `phase1_reached`

Un composant de phase 2 n'a pas de `InterCtx`, donc n'enregistre aucune arête du graphe d'appel : vu par `callers_of`, il est indiscernable d'une vraie racine. C'est tout le sens de la photographie (#110), dont la documentation énonce la règle :

Extrait 16.33 – `src/engine/program_result.rs`, l. 52–68 — ce que la phase 1 a atteint, et comment le lire

```

52 /// Components analysed in **phase 1** – the roots plus everything reached
53 /// top-down from them with an `InterCtx` (#110).
54 ///
55 /// **The reading discipline, and why the field exists.** Phase 2 sweeps
56 /// every component phase 1 did not reach and analyses it intra-only, with
57 /// no `InterCtx`, so it records no call-graph edges. Through
58 /// [ `ComponentCallGraph::callers_of` ] alone a phase-2 component is
59 /// therefore indistinguishable from a genuine root: both answer "no
60 /// callers". For a component NOT in this set, an empty `callers_of` means
61 /// **unknown ancestry**, never "proven root".
62 ///

```

```

63    /// Any relation that walks ancestry to conclude something is *absent* -
64    /// no provider above this consumer, no caller that could hold the setter -
65    /// must consult this set first, or it fails open on exactly the components
66    /// whose parents it cannot see. Empty for hand-built IR, which reads as
67    /// "nothing was inter-analysed": the conservative answer.
68    pub phase1_reached: HashSet<ComponentId>,

```

Invariant 16.1: Une ascendance vide n'est pas une ascendance prouvée

Pour un composant hors de `phase1_reached`, un `callers_of` vide signifie « ascendance inconnue », jamais « racine prouvée ». Toute relation qui remonte les appelants pour conclure à une *absence* — pas de provider au-dessus, pas d'appelant qui détienne le setter — consulte d'abord cet ensemble.

`complete_ancestry` (`src/engine/program_result.rs`, l. 134–151) rend la fermeture transitive des appelants, ou `None` dès qu'un composant du chemin n'a pas été analysé en phase 1. Son consommateur est la relation des consommateurs de contexte (`src/rules/helpers/context_flow.rs`, l. 100), qui écarte aussi une ascendance passant par une récursion coupée (`recursive_components`, `src/rules/helpers/context_flow.rs`, l. 104–108). La limite ouverte #20 en est la conséquence visible : `cross-component-infinite-loop` reste muette quand le parent n'est analysé qu'en intra.

Le chapitre 13 a listé ce que `inter = None` coûte en phase 2 : `hooks` personnalisés non greffés, `SummaryRegistry` non consulté, tout enfant havoqué, troncature du budget d'utilitaires silencieuse. Ajoutons-en une, symétrique de la dernière : `callback_depth_capped` (`src/domains/interp/interpreter.rs`, l. 484–491) et `inline_budget_exhausted` (`src/engine/fixpoint.rs`, l. 211–218) ne sont renseignés que sous `InterCtx`, si bien qu'un composant de phase 2 qui atteint ces plafonds ne porte pas l'INFO correspondante. Les résumés de dépendance de rendu, eux, restent précis en phase 2, parce qu'ils sont locaux (section 16.10).

16.8 Ce que lit une règle

16.8.1 `AnalysisResult` : le résultat d'un composant

Le résultat d'un composant (`src/engine/analysis_result.rs`, l. 171–272) est ce que lisent toutes les règles. Ses champs se rangent en cinq groupes (tableau 16.1) ; les chapitres précédents ont décrit comment chacun est calculé.

Un seul point est propre à ce chapitre : le résultat d'un enfant est celui de sa *dernière* analyse effective, pour les props de son dernier site analysé (exemple 16.8).

16.8.2 `ProgramAnalysisResult` et ses satellites

Le résultat de programme (`src/engine/program_result.rs`, l. 28–69) rassemble : la table `components` (`ComponentId` → `AnalysisResult`) ; le store partagé final ; le graphe d'appel ; les composants dont la récursion a été coupée ; les statistiques ; la `ComponentTable` ; la `FileTable` (ADR-19) ; le registre des utilitaires, exposé aux producteurs de témoins ; la `ModuleTable` (ADR-26) ; et `phase1_reached`. `file_table` et `module_table` sont laissés vides par `analyze_program` et remplis par `analyze_lowered` (`src/resolver/mod.rs`, l. 448–451). Default est « le programme vide », où chaque table répond « rien de connu » ; `single` (`src/engine/program_result.rs`, l. 91–114) fabrique un programme d'un seul composant déjà analysé en *enregistrant* son identifiant (souvent `SYNTHETIC`) au lieu d'en minter un que ses labels internes ne connaîtraient pas.

Le graphe d'appel (`src/engine/program_result.rs`, l. 166–201) associe à chaque appelant une liste de `CallSite` `callee`, `props`, `location`. Il est bruité, et il faut le savoir pour le lire

groupe	champs	rôle
identité	component, file, param, dom_props, module_consts	le composant (son ComponentId), son fichier (clé des témoins, ADR-19), le binding des props, les props typées DOM, les constantes de module (seule preuve qu'un <X.Provider> est un provider)
stores convergés	state_store, memo_store, block_states, effect_block_states, handler_block_states, heap, effect_setter_writes, custom_arg_returns	valeurs des slots et des mémos, environnement en sortie de chaque bloc (rendu, effets, handlers), tas final, écritures pures des effets (chapitre 13)
tables syn-taxiques	render_cfg, hooks, hook_provenance, hook_calls, effect_info, handler_info	le CFG après expansion, les entrées de hooks et leur provenance, les sites d'appel, les deps, les handlers
relations	slot_writers, slot_seeds, registrations, effect_triggers	calculées à convergence (chapitre 14)
traçabilité	widen_trace, inline_origins, iterations	widenings forcés, symboles inlinés, nombre d'itérations externes

TABLE 16.1 – Les champs d'AnalysisResult. Tout champ documenté « Empty for hand-built IR » se lit « non prouvé », jamais « absence prouvée ».

: location vaut toujours None (record_call_site(..., None)), une ligne est poussée à *chaque* évaluation — donc plusieurs par passe de rendu, et deux pour un return direct (section 16.4.4) —, et callers_of balaie toutes les arêtes et rend chaque appelant une fois, alors que callees_of rend les lignes telles quelles, doublons compris. L'information par site, avec sa position, est aujourd'hui portée par les ElementSite de la dépendance de rendu (section 16.10.4).

Les statistiques AnalysisStats (src/engine/program_result.rs, l. 203–225) comptent succès et ratés du cache, coupures de récursion et composants analysés, et rangent en ensembles les paires (appelant, appelé) coupées par récursion, les références inconnues et ambiguës (le callee y reste un nom, faute de résolution), les composants dont la profondeur de callbacks ou le budget d'utilitaires a été atteint. La règle analysis-limit (src/rules/impls/analysis_limit_info.rs, l. 39–97) les transforme en INFO et suspend les assurances du composant concerné.

16.8.3 RuleCtx et ProgramCache

Une règle ne reçoit ni l'un ni l'autre directement, mais un RuleCtx (src/rules/api/query.rs, l. 318–323) : l'identité du composant, son AnalysisResult résolu une fois, sa configuration, et un ProgramCache partagé par tous les composants du programme. ctx.comp() rend le résultat du composant, ctx.program() celui du programme, ctx.component() l'identité, ctx.component_name() le nom à écrire (liste 16.8). Le ProgramCache (src/rules/api/cache.rs, l. 26–31) compose les relations de programme du moteur (ProgramRelations, le graphe de churn, ADR-42 §5) avec trois structures que la couche règles construit encore elle-même, chacune dans un OnceLock calculé à la première demande : la relation des consommateurs de contexte, l'index des montages et l'index de rendu (RenderIndex). Demander les montages construit l'index de rendu, puisque MountIndex lit ses conditions de montage sur ElementSite::guard (src/rules/api/cache.rs, l. 62–68, #149).

16.9 Le graphe de symboles : prévu, réalisé

ADR-13 (analyse inter-fichiers) prévoyait un pipeline *eager* en cinq étapes, dont la dernière était « Lower + analyze in topo order » : un graphe de symboles (composants, hooks, utilitaires — pas de fichiers, parce que les imports circulaires entre fichiers sont courants alors que les dépendances circulaires entre fonctions React sont quasi inexistantes), trié topologiquement pour analyser les feuilles d'abord. Le graphe existe (`src/engine/symbol_graph.rs`) : `build` (`src/engine/symbol_graph.rs`, l. 60–118) crée un nœud par composant et par hook et une arête $A \rightarrow B$ quand A appelle syntaxiquement B — précise quand le lowering a résolu le callee (origine d'un élément, `resolved_file` d'un hook), sinon biaisée vers le même fichier puis vers le premier nœud homonyme inséré. `topo_sort` (`src/engine/symbol_graph.rs`, l. 195–255) est un Kahn inversé : les appelés d'abord, les nœuds pris dans des cycles ajoutés à la fin.

Décision : ADR-13 §4, §6 — un ordre topologique qui n'ordonne rien

L'ordre effectif de l'analyse est celui des racines (identifiants croissants) et de l'inlining descendant ; ni `analyze_program` ni le driver ne lisent l'ordre topologique. Son unique consommateur est une ligne de `-verbose` (`src/driver/mod.rs`, l. 324–336) — c'est la ligne « symbol graph: ... topo order = [...] » des sorties de ce chapitre. #7 le relevait déjà. Le reste d'ADR-13 est bien réalisé : clés composites (`PathBuf`, `String`) pour tous les registres (d'où `ComponentKey`), traits `FileDiscoverer` et `ImportResolver`, entrée par répertoire, `FunctionIR` et inlining des utilitaires en position d'instruction, imports non résolus lus `⊥` avec une `INFO`. Le repli `get_by_name`, qu'ADR-13 acceptait comme limite, a été retiré pour les composants par ADR-40.

Piège

La doc de `build` affirme un graphe « over-approximate but never under-approximate ». C'est faux pour un élément construit dans un `FnLit` en ligne du rendu (callback de `.map`, `render prop`) : `collect_callees_in_expr` ne traverse pas les `FnLit` (`src/engine/symbol_graph.rs`, l. 284–304) et `CFG::for_each_expr` ne visite que les expressions de premier niveau. Sur `/tmp/ch16/sg1.tsx` (`function Z() return <ul>[1].map((i) => <A key=i /></ul>;`), `-verbose` affiche « topo order = [Z@sg1.tsx, A@sg1.tsx] » : aucune arête $Z \rightarrow A$, alors que la détection de racines, qui descend dans les `FnLit`, voit bien A (« 1 components analyzed »). Sans conséquence sur les résultats, puisque le graphe n'est qu'affiché.

16.10 La dépendance de rendu

16.10.1 Pourquoi une seconde analyse

Exemple 16.9: Une valeur calculée, une condition, un voisin coûteux

Extrait 16.34 – `/tmp/ch16/ctrl1.tsx`

```
1 import { useState } from "react";
2 function Row({ i }: { i: number }) { return <li>row {i}</li>; }
3 function Heavy() { return <ul>[1, 2, 3].map((i) => <Row key={i} i={i} /></ul>; }
4 function Hint({ msg }: { msg: string }) { return <p>{msg}</p>; }
5 function Clear() { return <button>clear</button>; }
6 export function Search() {
7   const [text, setText] = useState("");
8   let msg = "";
9   if (text.length > 3) msg = "long";
10  return (
```



```

11     <div>
12       <input value={text} onChange={(e) => setText(e.target.value)} />
13       <Hint msg={msg} />
14       {text ? <Clear /> : null}
15     <Heavy />
16   </div>
17 );
18 }

```

```

$ reactant check ctrl.tsx --rule wasted-subtree-render --trace
Search (2 hooks) ctrl.tsx
warn wasted-subtree-render [hook:0] (line 12:26) each `change` event writes state `text` and re-renders `<
  ↳ Heavy>` (at least 2 component renders, including a list) although none of its inputs depends on it. Move `
  ↳ text` and the elements that use it into their own component, or build the unrelated elements higher up and
  ↳ pass them in as `children`
  ↳ → `<Heavy>` re-renders (at least 2 component renders, including a list) although none of its props depends
  ↳ on it (line 15:6)

```

Chaque frappe re-rend les trois éléments de Search. `<Heavy>` ne dépend de rien : c'est du gaspillage. `<Hint>` reçoit `msg`, une chaîne littérale choisie sous une condition sur `text` : sa valeur ne contient pas `text`, mais elle en *dépend*. `<Clear>` n'a aucune prop, mais son *existence* dépend de `text`. Le domaine des valeurs ne sait dire aucune de ces deux choses : `Stability::Versioned(S)` (définition 10.3) vit sur le seul slot de référence, si bien qu'un booléen, `text.length` ou une allocation `a: text` (qui est `PerRender`) n'en portent rien, et `MountIndex::Guard` contournait le manque par un balayage syntaxique des `StateVal`.

Décision : ADR-41 §1 — une analyse avant séparée, pas un champ de `StateValue`

La dépendance de rendu est une analyse avant, lancée sur un composant *convergé*, à la demande. Elle n'est *pas* une composante du produit `StateValue` : comme champ, elle changerait toutes les comparaisons de valeurs existantes — `unnecessary-re-render` compare `arg == init`, la clé du `ComponentCache` compare des props — et la dépendance est un autre fait que la valeur (un `a: text` frais dépend de `text`). Le résumé est *local* au composant (une prop se lit `Prop(name)`, quoi que passe le parent) et se compose sur l'arbre d'éléments à origine prouvée, ce qui garde précis le résumé d'un composant analysé seulement en phase 2. Cette décision a retiré un contournement (le balayage syntaxique de `MountIndex`, #149) en corrigeant la cause : c'est le premier principe du projet appliqué.

16.10.2 Les sources et le treillis `Deps`

Extrait 16.35 – `src/engine/render_deps.rs`, l. 53–84 — une entrée de rendu, dans le repère d'un composant

```

53 /// A render input a value may be computed from, in the frame of one component.
54 #[derive(Debug, Clone, PartialEq, Eq, PartialOrd, Ord, Hash)]
55 pub enum Source {
56     /// The value of the component's own state slot (inlined custom hooks'
57     /// slots included).
58     Slot(HookLabel),
59     /// The setter of one of its state slots: a *write capability*. It never
60     /// changes, but whoever uses it has to sit below the slot's owner.
61     Setter(HookLabel),
62     /// One top-level prop.
63     Prop(Symbol),
64     /// The props object as a whole (`{...props}`, `props[k]`, `f(props)`).
65     AllProps,
66     /// A ref's contents.

```

```

67   Ref(HookLabel),
68   /// The result of a hook the engine does not model (an unresolved custom
69   /// hook, a library hook, ...): a reactive source outside the model.
70   Hook(HookLabel),
71   /// A proven React context: the context object itself, and what
72   /// `useContext` reads from it. Its value comes from the nearest provider
73   /// above, which the element tree pairs it with.
74   Context(ContextId),
75   /// A name the component does not bind (a module binding, an import, a
76   /// global) that some function of the program writes: the one channel a
77   /// handler can change besides state (`cache.x = ...`, `counter++`,
78   /// `seen.add(k)`), so a write to it travels with the write to the slot
79   /// ([`Writes`]). A name nothing writes is not a render input and reads
80   /// as nothing, which keeps the sets small. Frame-free: the same name in
81   /// every component, which is what lets it be carried down the element
82   /// tree untranslated.
83   Module(Var),
84 }

```

Une source est une entrée du rendu *dans le repère d'un composant* : $\text{Slot}(\emptyset)$ du parent et $\text{Slot}(\emptyset)$ de l'enfant sont deux choses différentes, et $\text{Prop}(\text{name})$ est relatif au composant courant. Seules $\text{Context}(\text{id})$ et $\text{Module}(\text{name})$ sont *indépendantes du repère* : le même contexte et le même nom de module dans tous les composants, ce qui leur permet de descendre l'arbre d'éléments sans traduction.

Définition 16.3: Dépendance de rendu

Soit Src l'ensemble (fini pour un programme) des sources d'un composant. La *dépendance de rendu* d'une valeur v du rendu est un élément de $\mathcal{P}(\text{Src}) \cup \{\top\}$ ordonné par inclusion, $\top$ au sommet : un ensemble *may* des sources dont v peut être calculée, directement (opérande, capture) ou par contrôle (condition d'une branche qui décide de son affectation). $\top$ signifie « peut dépendre de n'importe quoi ». Le join est l'union, $\top$ absorbant. Une source absente de la dépendance est une source dont v est *prouvée* indépendante.

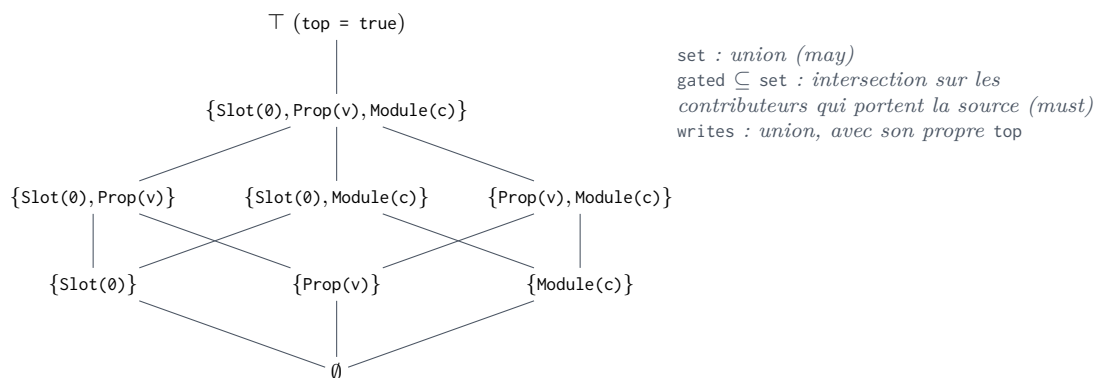


FIGURE 16.4 – Le treillis Deps pour trois sources : parties ordonnées par inclusion, plus un sommet $\top$ qui vide set et gated. Le treillis est fini, donc la montée s'arrête sans widening. La composante gated va dans le sens dual : elle ne peut que rétrécir quand on joint.

Extrait 16.36 – `src/engine/render_deps.rs`, l. 119–131 — un ensemble may de sources, et deux compléments

```

119 /// May-set of sources. `top` is "may depend on anything".
120 #[derive(Debug, Clone, Default, PartialEq, Eq)]
121 pub struct Ddeps {

```

```

122     pub top: bool,
123     pub set: BTreeSet<Source>,
124     /// The part of `set` a function value reaches, when called, only behind
125     /// a test of its own arguments (`e => { if (e.key === "Enter") f() }`)
126     /// has `f` here). A must-fact: one ungated contributor takes a source
127     /// out.
128     pub gated: BTreeSet<Source>,
129     /// For a function value: what calling it writes besides state.
130     pub writes: Writes,
131 }

```

Deps porte l'ensemble `set`, le drapeau `top`, et deux compléments utiles aux valeurs fonctions. `gated` est la partie de `set` qu'une fonction n'atteint, quand on l'appelle, que derrière un test de ses propres arguments (section 16.10.5); `writes` est l'ensemble des noms de module qu'appeler la fonction peut écrire (`Writes top, names, src/engine/render_deps.rs`, l. 90–117). Leur `join` cohabite dans une même fonction :

Extrait 16.37 – `src/engine/render_deps.rs`, l. 153–170 — le `join` : `may` pour `set`, `must` pour `gated`

```

153     pub fn union_with(&mut self, other: &Deps) {
154         self.top |= other.top;
155         self.writes.union_with(&other.writes);
156         if self.top {
157             self.set.clear();
158             self.gated.clear();
159             return;
160         }
161         let ungated: BTreeSet<Source> = self
162             .set
163             .difference(&self.gated)
164             .chain(other.set.difference(&other.gated))
165             .cloned()
166             .collect();
167         self.gated.extend(other.gated.iter().cloned());
168         self.gated.retain(|s| !ungated.contains(s));
169         self.set.extend(other.set.iter().cloned());
170     }

```

`set` croît par union. Une source reste `gated` seulement si *aucun* contributeur ne l'apporte non gardée : on calcule les sources non gardées des deux côtés, on réunit les `gated`, puis on retire les non gardées. `top` absorbe tout, sauf `writes`, qui garde son propre `top` (`Deps::top()` pose `writes: Writes::top()`, `src/engine/render_deps.rs`, l. 141–147). Enfin `ungated()` (`src/engine/render_deps.rs`, l. 172–180) efface `gated` en gardant `writes` : appliquée au résultat d'un appel, elle dit que la valeur n'est plus la fonction décrite (`debounce(fn)` rend une fonction qui appelle peut-être `fn`), si bien que le fait `may` survit et le fait `must` non.

Piège

`Deps::is_empty()` est `!top && set.is_empty()` (`src/engine/render_deps.rs`, l. 149) : il *ignore* `writes`. Une fermeture qui n'écrit qu'un binding de module est « vide » pour ce test. Ce n'est pas un défaut — les questions de rendu passent par touches, et les écritures par `co_writes` — mais un appel à `is_empty` pour décider qu'une fonction « ne fait rien » serait faux.

Une question se pose dans un repère sous la forme d'une Relevance `any_prop`, sources (`src/engine/render_deps.rs`, l. 207–250), et la réponse élémentaire est touches (`src/engine/render_deps.rs`, l. 185–205) : une dépendance touche une question si elles partagent une source, si la dépendance est $\top$ et la question non vide, ou si l'une porte l'objet props entier (`AllProps`) et l'autre une prop. `gated_for(rel)` répond vrai quand toutes les sources de `rel` que

la dépendance porte sont gardées (chapitre 21 montre ces deux fonctions).

16.10.3 L'analyse avant d'un composant

`render_deps` (`src/engine/render_deps.rs`, l. 417–503) prend un `AnalysisResult` convergé — donc le CFG *après* greffe des hooks personnalisés et des utilitaires — et l'ensemble `written` des noms qu'une fonction quelconque du programme écrit. La valeur abstraite interne est un `DVal` `deps`, `shape` (`src/engine/render_deps.rs`, l. 333–379) : une forme `Props` marque l'objet `props` (un `.field` dessus donne `Prop(field)`), une forme `Members` garde les sources membre par membre d'un littéral local (ce dont un retour de hook déstructuré a besoin). L'environnement d'entrée lie le paramètre `props` à `DVal AllProps`, `Props` (`src/engine/render_deps.rs`, l. 447–454).

Évaluation. `eval` (`src/engine/render_deps.rs`, l. 815–948) suit la tableau 16.2. Deux lignes méritent l'attention : un élément de composant s'évalue à $\emptyset$ — ses `props` sont les entrées de l'enfant, attribuées au site, et l'élément lui-même n'est pas un usage —, alors qu'un élément hôte prend l'union de ses `props` et de ses enfants.

expression	dépendance
<code>Lit</code> , <code>SummaryVal</code>	$\emptyset$
<code>StateVal(1)</code> , <code>StateSetter(1)</code>	<code>Slot(1)</code> , <code>Setter(1)</code> (respectivement)
<code>HookMarker(1)</code>	<code>ref</code> : <code>Ref(1)</code> ; <code>hook opaque</code> : <code>Hook(1) $\cup$ arguments</code> (dé-gardés) ; sinon $\emptyset$
<code>MemoVal(1)</code> , <code>CallbackVal(1)</code>	variables libres du corps (fermeture pour un callback) $\cup$ liste de <code>deps</code> ; entrée introuvable : $\top$
<code>FnLit</code>	fermeture : captures (gardées si derrière un test des paramètres) + <code>writes</code>
<code>FieldAccess</code>	sur <code>Props</code> : <code>Prop(field)</code> ; sur <code>Members</code> : le membre ; sinon la base
<code>IndexAccess</code> , <code>BinOp</code>	union des opérandes
<code>Call</code> , <code>New</code>	callee et arguments, dé-gardés ; un callee <code>use/useX</code> lève <code>inline_hook</code>
<code>ObjectLit</code>	union des champs ; forme <code>Members</code> si aucun n'est un <code>spread</code>
<code>ArrayLit</code>	union des éléments ; jamais de forme <code>Members</code>
<code>CompApp</code>	$\emptyset$
<code>NativeElem</code>	<code>props $\cup$ enfants</code>

TABLE 16.2 – L'évaluation de la dépendance de rendu (`src/engine/render_deps.rs`, l. 815–948).

Lecture d'un nom. Un nom non lié dans le composant — binding de module, import, global — n'est pas forcément une constante :

Extrait 16.38 – `src/engine/render_deps.rs`, l. 603–625 — un nom non lié : contexte prouvé, binding écrit, ou rien

```

603     fn var(&self, env: &Env, v: &Var) -> DVal {
604         if v == self.param {
605             return env.get(v).cloned().unwrap_or(DVal {
606                 deps: Deps::one(Source::AllProps),
607                 shape: Shape::Props,
608             });
609         }
610         // Not bound here: module scope, an import or a global. A proven
611         // context object reads as its context, whose value its provider
612         // sets; a name some function writes as the name itself; anything
613         // else is constant across renders and reads as nothing.
614         env.get(v)

```

```

615         .cloned()
616         .unwrap_or_else(|| match self.consts.get(v) {
617             Some(ModuleConstInit::Context(id)) => {
618                 DVal::of(Deps::one(Source::Context(id.clone())))
619             }
620             _ if self.module_written.contains(v) => {
621                 DVal::of(Deps::one(Source::Module(v.clone())))
622             }
623             _ => DVal::empty(),
624         })
625     }

```

Un objet contexte prouvé (`ModuleConstInit::Context`) se lit `Context(id)` ; un nom que quelque fonction du programme écrit se lit `Module(name)` ; tout le reste est constant d'un rendu à l'autre et ne dépend de rien. Restreindre `Module` aux noms écrits garde les ensembles petits : sur le dépôt `twenty`, lire `Module` pour *tout* nom libre coûtait 70 % (301 s à 514 s) ; avec la restriction, l'exécution mesure 366 s (plan de campagne `docs/campaign/rerender-cascade-plan.md`, §10, « Corpus effect of module writes »).

Transfert et dépendance de contrôle. `run_cfg` itère en ordre RPO jusqu'à stabilité, au plus `MAX_ROUNDS = 64` tours. Au début de chaque bloc, il calcule sa *condition de chemin* `pc` : la dépendance des conditions des branches qui le contrôlent, évaluées dans l'environnement de sortie du bloc de branche.

Extrait 16.39 – `src/engine/render_deps.rs`, l. 689–704 — la condition de chemin d'un bloc, puis le transfert

```

689         let mut p = outer.pc.clone();
690         for br in controllers.get(&b).into_iter().flatten() {
691             if let (Some(env), Some(Terminator::Branch { cond, .. })) =
692                 (out_env.get(br), cfg.blocks.get(br).map(|x| &x.term))
693             {
694                 p.union_with(&self.eval(cond, env).deps);
695             }
696         }
697         if pc.get(&b) != Some(&p) {
698             pc.insert(b, p.clone());
699             changed = true;
700         }
701         let mut env = in_env.get(&b).cloned().unwrap_or_default();
702         for stmt in &block.stmts {
703             self.transfer(stmt, &mut env, &p);
704         }

```

Une branche *contrôle* un bloc quand exactement un de ses deux côtés l'atteint : c'est la différence symétrique des ensembles atteignables depuis `then_` et `else_` (`controlling_branches`, `src/engine/render_deps.rs`, l. 1240–1276), la même définition que `MountIndex`. Le transfert (`src/engine/render_deps.rs`, l. 777–813) ajoute `pc` à toute affectation : `env[x] = eval(rhs) ∪ pc`. Dans l'exemple 16.9, `msg = "long"` est dans le bloc contrôlé par `text.length > 3` : `msg` prend `Slot(0)`, et `<Hint>` n'est pas gaspillé. Un `MemberWrite obj.k = v` est une mise à jour *faible* de la racine, qui perd sa forme ; un appel de méthode en instruction (`xs.push(v)`) ajoute ses arguments au récepteur — sans `pc`, et pour toute méthode, pas seulement les méthodes mutantes. Si la boucle ne converge pas en 64 tours, le drapeau `top` passe à vrai et *tout* le résumé devient `⊤` : le plafond est une garde, pas un bouton de précision, et il dégrade vers le haut.

16.10.4 La collecte : utiliser ou transmettre

Après le point fixe, une passe de collecte parcourt les blocs dans le même ordre et produit le résumé :

Extrait 16.40 – src/engine/render_deps.rs, l. 301–322 — le résumé d'un composant

```

301 /// Render dependence summary of one component.
302 #[derive(Debug, Clone, Default)]
303 pub struct RenderDeps {
304     pub genuine: Deps,
305     pub sites: Vec<ElementSite>,
306     pub handlers: Vec<HostHandler>,
307     /// The component may read a context the analysis cannot name: a
308     /// `useContext` of an unproven object or reached through an inlined hook,
309     /// or a hook of user code the engine could not see into.
310     pub any_context: bool,
311     /// Per effect, the module names its body (callbacks it registers
312     /// included) may write: what a write it makes to a slot travels with.
313     pub effect_writes: BTreeMap<HookLabel, Writes>,
314 }
315
316 impl RenderDeps {
317     /// Whether the component uses `rel`: its own output, effects or hooks
318     /// depend on it, or `rel` names a context it may read.
319     pub fn uses(&self, rel: &Relevance) -> bool {
320         self.genuine.touche(rel) || (self.any_context && rel.contexts().next().is_some())
321     }
322 }

```

C'est le cœur de la dichotomie : *genuine* est ce que le composant *utilise*, *sites* ce qu'il *transmet*. Entrent dans *genuine* : les appels en position d'instruction, les écritures de membre pendant le rendu et la valeur rendue, chacun avec son pc ; tout élément *hôte* (*NativeElem*), qui est sortie du composant qui le construit, même passé en *children* à un enfant (`<Modal><input value=text/></Modal>`) ; les effets (variables libres du corps et liste de deps, évaluées dans l'environnement de sortie du bloc d'appel, src/engine/render_deps.rs, l. 466–497) ; les arguments d'un hook resté opaque après expansion. Un *Let* ou un *Assign* n'est jamais un usage en soi ; une condition de branche n'en est un qu'à travers le pc des blocs qu'elle contrôle.

Chaque élément de composant devient un site :

Extrait 16.41 – src/engine/render_deps.rs, l. 252–277 — un élément de composant construit par le rendu

```

252 /// A component element built by the render (`<Child a={x} />`).
253 #[derive(Debug, Clone)]
254 pub struct ElementSite {
255     pub name: Symbol,
256     pub origin: Option<Arc<CompOrigin>>,
257     pub span: Option<SourceRange>,
258     /// Per explicit prop, what it may depend on (`children` included).
259     pub props: Vec<(Symbol, Deps)>,
260     /// What the spread entries (`{...rest}`) may depend on: any prop of the
261     /// child may carry it.
262     pub spread: Deps,
263     /// The conditions the element is built under.
264     pub guard: Deps,
265     /// Built in a callback the render passes to a call (`map`): possibly
266     /// many instances, possibly none.
267     pub in_list: bool,
268     /// The element is the provider of a proven context (`<Ctx.Provider>`, or

```

```

269    /// `` since React 19): its `value` reaches the context's consumers
270    /// below, and its children render unchanged.
271    pub provides: Option<ContextId>,
272    /// Index (in [RenderDeps::sites]) of the element this one is nested in
273    /// as a prop or child: `<Provider value={v}><Row /></Provider>` gives
274    /// `Row` the provider's index. What reaches the enclosing element may
275    /// reach this one through it (a context, a clone).
276    pub parent: Option<usize>,
277 }

```

Extrait 16.42 – `src/engine/render_deps.rs`, l. 961–979 — la garde d’un site : le chemin, et le type d’élément s’il est une valeur

```

961    // The element type can be a value of this render (`const {
962    // Modal } = useModal()`, `<Modal />`): what it depends on
963    // decides which component mounts, like a condition does.
964    let mut guard = ctx.pc.clone();
965    let root = name.split('.').next().unwrap_or(name);
966    if let Some(ty) = env.get(root) {
967        guard.union_with(&ty.deps);
968    }
969    let mut site = ElementSite {
970        name: name.clone(),
971        origin: origin.clone(),
972        span: *span,
973        props: Vec::new(),
974        spread: Deps::default(),
975        guard,
976        in_list: ctx.in_list,
977        provides: self.context_of(name.strip_suffix(".Provider").unwrap_or(name), env),
978        parent: ctx.parent,
979    };

```

La garde d’un site est le pc de son bloc, plus — si le nom de type est une valeur du rendu (`const Modal = useModal(); <Modal />`) — la dépendance de cette valeur : quel composant se monte en dépend, comme d’une condition (ce correctif a retiré 8 FP sur le dépôt dub lors de la campagne). Chaque prop explicite garde sa dépendance ; les spreads vont dans `spread` ; `provides` est renseigné si le nom, sans `.Provider`, désigne un contexte prouvé non masqué ; `parent` est l’indice du site dans lequel celui-ci est imbriqué comme prop ou enfant. Chaque prop `onX` (ou `spread`) d’un élément hôte devient un `HostHandler` (`src/engine/render_deps.rs`, l. 282–299), avec la balise, l’attribut type littéral et la dépendance de la valeur du handler : c’est là qu’un setter « atterrit ».

Callbacks synchrones. Les `FnLit` passés à un appel (`.map`, `.forEach`) tournent pendant le rendu : collect les analyse par `nested` (`src/engine/render_deps.rs`, l. 1079–1109), avec des paramètres alimentés par le récepteur et les autres arguments, `in_list` vrai, et une profondeur bornée par `MAX_NESTING = 8` (au-delà, `T`). Le corps d’un `useMemo` lu pendant le rendu est parcouru de même, sans `in_list`. Un `FnLit` qui n’est pas passé à un appel ne tourne pas pendant le rendu : seules ses captures comptent, comme dépendance de la valeur fonction.

Exemple 16.10: Une écriture dans un `forEach` est perdue

Extrait 16.43 – `/tmp/ch16/foreach.tsx` (extrait)

```

1 export function App() {
2   const [sel, setSel] = useState(0);

```



```

3   let label = "";
4   [1, 2, 3].forEach((i) => { if (i === sel) label = String(i); });
5   return (
6     <div>
7       <input value={sel} onChange={(e) => setSel(Number(e.target.value))} />
8       <Child label={label} />
9       <Heavy />
10    </div>
11  );
12 }

```

```
$ reactant check foreach.tsx --rule wasted-subtree-render
```

```
App (2 hooks) foreach.tsx
```

```

warn wasted-subtree-render [hook:0] (line 10:25) each `change` event writes state `sel` and re-renders `<
  ↳ Child>` and `<Heavy>` (2 component renders) although none of their inputs depends on it. Move `sel` and
  ↳ the elements that use it into their own component, or build the unrelated elements higher up and pass them
  ↳ in as `children`

```

L'affirmation est fausse pour `<Child>` : `label` est calculé à partir de `sel`. Deux faits du code se combinent. `nested` lance `run_cfg(body, ..., false)` et *jette* l'environnement de sortie du callback : une écriture du callback dans un binding local du rendu ne remonte pas. Et `compute_free_vars` retire `label`, cible d'un `Assign`, des variables libres du callback (`src/ir/free_vars.rs`, l. 96–106). `label` garde donc la dépendance de `""`, `∅`. C'est une source manquante, donc un faux positif d'une règle d'absence — la direction qu'ADR-41 §2 veut exclure ; il n'était ni documenté dans `docs/limitations.md` ni suivi par une issue au snapshot.

16.10.5 Handlers gardés : le fait `gated`

Une frappe dans un champ texte écrit à chaque touche ; un `onKeyDown` qui n'écrit que sur `Enter` écrit une fois par validation. La différence n'est pas une question de preuve mais de *fréquence*, et la règle `wasted-subtree-render` ne signale par défaut que les déclencheurs continus (option `continuousOnly`, ADR-41, chapitre 21). `param_gated_vars` (`src/engine/render_deps.rs`, l. 1176–1238) calcule, pour une fonction, les variables libres qu'elle n'atteint que derrière un test de ses paramètres : il propage les variables *teintées* par les paramètres jusqu'à un point fixe, et les blocs contrôlés par une condition qui en lit une ; une variable libre est gardée si elle n'apparaît que dans de tels blocs. Sur la fixture `tests/fixtures/wasted_subtree_render/keyed.tsx`, le handler de `Submit` (`if (e.key !== "Enter") return; setQ(...)`) a la dépendance `Setter(∅)` avec `gated = Setter(∅)` (résumé relevé par la sonde du dossier 08) ; `KeyLog` (`onKeyDown=(e) => setLast(e.key)`) ne garde rien. Par défaut, seul `KeyLog` est signalé ; avec `continuousOnly=false`, `Escape`, `Handed`, `KeyLog` et `Submit` le sont (observé). Le fait est *must* : un seul contributeur non gardé l'annule (liste 16.37), et il ne peut que classer un déclencheur comme discret, jamais changer une preuve.

16.10.6 Contextes

Extrait 16.44 – `tests/fixtures/state_lifted_too_high/context.tsx`, l. 6–21 — un état qui atteint son consommateur par contexte

```

6   const Query = createContext({ q: "", setQ: (_, string) => {} });
7   function Search() {
8     const { q, setQ } = useContext(Query);
9     return <input value={q} onChange={(e) => setQ(e.target.value)} />;
10  }
11  function Header() { return <h1>Title</h1>; }
12  function Page() { return <main><Header /><Search /></main>; }
13  export function App() {

```



```

14  const [q, setQ] = useState("");
15  return (
16    <Query.Provider value={{ q, setQ }}>
17      <Header />
18      <Page />
19    </Query.Provider>
20  );
21 }

```

Dans le résumé d'App (sonde du dossier 08), le provider est un site provides = Some(Query) dont la prop value a la dépendance Slot(0), Setter(0); <Header> et <Page> ont parent = Some(0); children vaut $\emptyset$, puisqu'un élément de composant s'évalue à $\emptyset$. Dans Search, useContext(Query) est un hook opaque : sa dépendance est Hook(0) $\cup$ celle de son argument, Context(Query). Le drapeau any_context reste faux, parce que Query est prouvé et écrit dans le composant ; il passe à vrai pour un useContext d'un objet non prouvé ou atteint par un hook inliné, et pour tout hook de code utilisateur que l'analyse n'a pas pu inliner (reads_unnamed_context, src/engine/render_deps.rs, l. 645–670). Un hook de paquet, lui, ne peut atteindre un contexte des modules utilisateur que si on le lui passe, ce que ses arguments montrent.

```

$ reactant check tests/fixtures/state_lifted_too_high/context.tsx --rule state-lifted-too-high --trace
App (1 hooks) tests/fixtures/state_lifted_too_high/context.tsx
  warn state-lifted-too-high [hook:0] (line 14:8) state `q` is only used inside `<Search>`, 2 levels below `App`.
    ↳ Every write re-renders `App`, `Page` and 2 other components they render although none of them uses it; move
    ↳ the state into `Search`, with the `Query` provider that hands it on
      → `App` renders `<Page>` inside the `Query` provider without using it itself (line 18:6)
      → `Page` renders `<Search>` inside the `Query` provider without using it itself (line 12:41)

```

Côté domaine des valeurs, Query.Provider est un composant *inconnu* du registre : il compte dans unknown_component_refs, d'où les INFO analysis-limit sur les X.Provider, et c'est le cas courant du trou de havoc de l'exemple 16.6. Dans Guarded, plus bas dans la même fixture, un <Tooltip/> non résolu sous le provider compte comme consommateur possible : pas de finding.

16.10.7 Bindings de module

Un handler peut changer autre chose qu'un état : cache.hits += 1, counter++, seen.set(k, v). Si un autre composant lit ce binding, il n'est pas gaspillé. written_names (src/engine/render_deps.rs, l. 406–412) rassemble les racines écrites du rendu et de chaque corps de hook ; written_roots (src/engine/render_deps.rs, l. 1125–1171) retient la cible d'un Assign (l'IR épelle ainsi l'écriture d'un binding extérieur), la racine d'un MemberWrite, le récepteur d'une méthode mutante (la liste d'ADR-28 : push, pop, ..., set, et Object.assign), fermetures imbriquées comprises, moins ce que le corps lie. L'union sur tous les composants est l'ensemble written de la lecture d'un nom. Puis writes_of (src/engine/render_deps.rs, l. 580–601) traduit les racines écrites d'un corps en noms de module à travers les valeurs qu'elles peuvent aliaser (const c = cache; c.x = 1 écrit cache) ; une racine de valeur T donne Writes::top().

```

$ reactant check tests/fixtures/wasted_subtree_render/module_write.tsx --rule wasted-subtree-render --rule-option wasted-
  ↳ subtree-render:continuousOnly=false
Control (2 hooks) ... each `change` event writes state `text` and re-renders `<Hits>`, `<Count>`, `<Seen>`, `<
  ↳ Stats>` and `<Chart>` (6 component renders) ...
Debounce (2 hooks) ... each `change` event writes state `text` and re-renders `<Chart>` (2 component renders) ...
Handed (1 hooks) ... each `change` event in `<Field>` writes state `text` and re-renders `<Chart>` (2 component
  ↳ renders) ...
Typing (2 hooks) ... each `change` event writes state `text` and re-renders `<Chart>` (2 component renders) ...

```

(lignes élaguées aux en-têtes et au début des messages.) Control n'écrit que son état : les cinq voisins sont gaspillés. Le handler de Typing écrit cache, counter et seen à côté de setText : Hits, Count, Seen et Stats (qui lit cache à travers l'utilitaire describe() inliné) cessent d'être

gaspillés, seul Chart reste. Dans Handed, le setter descend vers Field, dont la fermeture écrit `last : writes` s'accumule le long du trajet. Dans Debounced, `debounce(...)` est opaque, mais `ungated()` garde `writes`.

Remarque

Les noms de module s'apparient *par orthographe*, à travers les fichiers : deux `cache` sans rapport n'en font qu'un, ce qui donne moins de findings, jamais un faux. Une hypothèse reste, énoncée par ADR-41 §2 : un appel que l'analyse ne voit pas ne lit ni n'écrit de binding de module (#51, #52).

16.11 Composer sur l'arbre d'éléments

16.11.1 L'index de rendu et la résolution d'un site

Les résumés sont locaux ; les questions sont globales (« où cet état est-il utilisé ? »). La couche règles les compose dans `RenderIndex` (`src/rules/helpers/render_tree.rs`), construit une fois par programme : `build` (`src/rules/helpers/render_tree.rs`, l. 117–137) calcule `written`, puis un résumé par composant, puis le nombre de sites qui nomment chaque composant. Passer d'un site à un composant se fait par `resolve` (`src/rules/helpers/render_tree.rs`, l. 767–777), qui n'accepte que l'origine prouvée (`table.id_of(origin)`).

Piège

Il y a donc deux notions de « résolu ». Le moteur (`resolve_child`) accepte l'origine, sinon un nom unique ; l'arbre d'éléments n'accepte que l'origine. Un site sans origine dont le nom est unique est inliné par le moteur mais opaque pour les cascades. Chacun est prudent dans sa polarité : l'inliner qui trouverait un enfant par son nom analyse au pire un corps de trop, alors que la règle de cascade qui se tromperait d'enfant *cacherait* les usages du vrai.

16.11.2 `uses` : descendre en traduisant la question

`uses(comp, rel)` (`src/rules/helpers/render_tree.rs`, l. 516–592) construit un arbre d'usages. Le nœud est *utilisateur* si le résumé `uses(rel)` (genuine touche la question, ou `any_context` et la question nomme un contexte). Pour chaque site : une garde touchée fait du nœud un utilisateur ; un provider est sauté, sa valeur étant suivie par contexte ; sinon on calcule les props qui portent la question (`forwarded`, qui laisse passer un spread de l'objet props sous les mêmes noms et traite tout autre spread comme « n'importe quelle prop »), les contextes qui la portent (`contexts_at`, qui remonte les parent jusqu'aux providers dont la value la touche) et les modules. Un site dans une liste, non résolu, trop profond (`MAX_DEPTH = 64`) ou déjà visité fait du nœud un utilisateur. Sinon on descend avec la question *traduite dans le repère de l'enfant* : `Prop(p)` pour chaque prop qui la porte, `any_prop` si un spread peut renommer, contextes et modules tels quels.

16.11.3 `home_of` : le plus petit sous-arbre

`home_of(owner, label)` (`src/rules/helpers/render_tree.rs`, l. 195–257) rend `None` si un écrivain du slot peut écrire n'importe quoi (`co_writes.top`), si personne ne lit le slot ou si personne ne l'écrit. Sinon il construit l'arbre d'usages de la question `Slot(1)`, `Setter(1) ∪ co_writes` et descend tant qu'exactly un enfant a des usages :

Extrait 16.45 – `src/rules/helpers/render_tree.rs`, l. 226–244 — descendre tant qu'un seul enfant utilise

```

226     let mut visiting = HashSet::from([owner]);
227     let mut node = self.uses(owner, &both, program, &mut visiting, 0);
228     let mut path = Vec::new();
229     loop {
230         if node.user {
231             break;
232         }
233         let mut used: Vec<(Hop, UseTree)> = node
234             .kids
235             .into_iter()
236             .filter(|(t)| t.has_users())
237             .collect();
238         if used.len() != 1 {
239             break;
240         }
241         let (hop, next) = used.pop().expect("one used child");
242         path.push(hop);
243         node = next;
244     }

```

Le chemin parcouru est la liste des composants qui re-rendent pour rien; siblings compte les autres sites (hors providers) que construisent les composants du chemin. `co_writes` (src/rules/helpers/render_tree.rs, l. 154–178) réunit les `writes` des *landings* du setter et les `effect_writes` des effets qui écrivent le slot; `landings` (src/rules/helpers/render_tree.rs, l. 379–512) suit la capacité `Setter(1)` de `prop` en `prop` jusqu'aux handlers hôtes dont la dépendance la touche — une `prop` à la fois, car « a prop that lands below must not hide one that does not » —, et une `prop onX` qui n'atterrit nulle part plus bas atterrit sur l'élément lui-même.

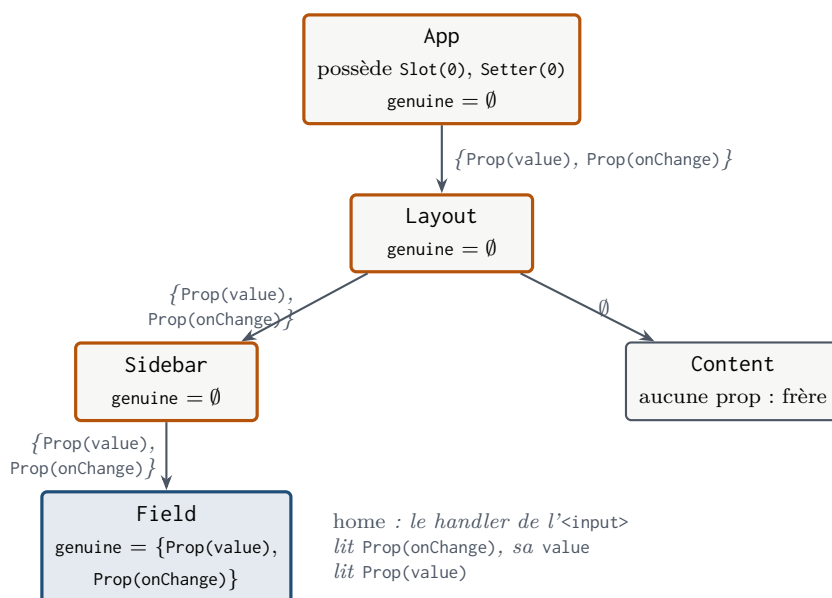


FIGURE 16.5 – L'arbre d'éléments de `drill.tsx` et la question `Slot(0)`, `Setter(0)` d'App, traduite à chaque saut dans le repère de l'enfant. Les trois composants du chemin (bord orangé) ne font que transmettre; Field (bleu) est le *home*; Content est un frère qui re-rend aussi (`siblings = 1`).

Exemple 16.11: Le `home` de `value` dans `drill.tsx`

Les résumés (sonde du dossier 08) : App, Layout et Sidebar ont `genuine = ∅`, et chacun transmet `value` et `onChange` à son unique enfant utilisateur; Field a `genuine = Prop(onChange)`,

Prop(value) et un handler change sur `<input>` de dépendance Prop(onChange). `home_of(App, 0)` descend `App` → `Layout` → `Sidebar` → `Field` (figure 16.5), avec un frère, `Content` :

```
$ reactant check tests/fixtures/state_lifted_too_high/drill.tsx --rule state-lifted-too-high --trace
App (1 hooks) tests/fixtures/state_lifted_too_high/drill.tsx
  warn state-lifted-too-high [hook:0] (line 17:8) state `value` is only used inside `

```

La règle elle-même, ses seuils et ses options sont au chapitre 18.

`wasted_siblings(owner, rel)` (`src/rules/helpers/render_tree.rs`, l. 266–326) répond à la question complémentaire, celle de `wasted-subtree-render` (chapitre 21) : les sites qu'aucune garde, prop ou spread (hors provider), ni aucun site englobant, ne relie à la question ; seuls les composants résolus sont candidats, un enfant qui utilise un contexte porté ou un module écrit est écarté, et le coût est une borne inférieure (une liste compte une fois).

Remarque

Tout le fichier `render_tree.rs` obéit à la règle que son en-tête énonce (`src/rules/helpers/render_tree.rs`, l. 11–13) : l'absence d'usage y est une preuve, donc tout inconnu est un usage — élément non résolu, récursion, profondeur, résumé $\top$, contexte non nommé, écrivain de racine $\top$ (ADR-41 §2). Le chapitre 21 en fait un invariant.

16.12 Soundness : ce qui est garanti, ce qui fuit

Les deux mécanismes du chapitre n'ont pas la même polarité. L'inlining alimente des règles qui prouvent une *présence* (un setter appelé, une boucle) : il doit sur-approximer l'état. La dépendance de rendu alimente des règles qui prouvent une *absence* (« aucune entrée ne dépend de l'état ») : pour elles, une source manquante est un faux positif, et « sound » veut dire « tout inconnu est un usage ». La tableau 16.3 récapitule les garanties et les fuites vérifiées au snapshot.

mécanisme	garantie	fuite vérifiée
résolution (<code>resolve_child</code>)	jamais de devinette : ambigu = inanalysable	repli sur le nom écrit après un barrel (précision seulement)
enfant inanalysable	havoc des setters échappés	setter champ d'un littéral inline (exemple 16.6)
récursion	coupure déclarée, assurances suspendues	aucun havoc (exemple 16.5)
store partagé	monotone, importé par join	—
cache	saut d'analyse sound pour le store	<code>results[child]</code> écrasé : FN non déclaré (exemple 16.8)
élément argument d'un hook opaque	racine en trop, INFO sur le hook	ni inliné ni havoqué : <code>App: 0 iteration(s)</code> sur <code>hookarg.tsx</code>
phase 2	<code>phase1_reached</code> , <code>complete_ancestry</code>	plafonds non enregistrés hors <code>InterCtx</code>
dépendance de rendu	plafonds $\Rightarrow \top$; inconnu = usage	écriture d'un callback synchrone perdue (exemple 16.10)
bindings de module	<code>Module(name)</code> , <code>writes</code>	appel opaque supposé muet (#51, #52)

TABLE 16.3 – Garanties et fuites du sous-système inter-composants.

La ligne « élément argument d'un hook opaque » demande un mot. Sur `/tmp/ch16/hookarg.tsx` (`const m = useModal(<Dialog onClose=setS />)`, avec un `Dialog` qui appelle `onClose(1)` pendant son rendu), `-verbose` rapporte « cache hits: 0, misses: 0 » et « App: 0 iteration(s) » : l'élément argument n'est jamais évalué par le domaine des valeurs, donc ni inliné, ni havoqué, et `s` reste à sa valeur initiale, alors qu'un `useModal` qui rend l'élément ferait appeler `setS`. Le faux négatif est déclaré par l'INFO du hook inconnu (« hook 'useModal' was not found in the registry ... (FN possible) »), qui suspend les assurances d'App. Vérifié au `e67b10a` au-delà des seuls compteurs : `make_hook_entry` (`src/lowering/hook_extractor.rs`, l. 641–660) retire les arguments d'un hook Custom du CFG et les range dans `HookEntry::Custom::args`, sans laisser de `Expr::Call` qui les contiendrait encore ; `expand_custom_hooks` (`src/engine/fixpoint.rs`, l. 889–992) ne les lit que pour une substitution syntaxique (`param_subst`, seuls les inits `State` du hook expansé) ; `custom_arg_returns` (`src/engine/fixpoint.rs`, l. 240–275) n'exécute qu'un argument `FnLit` ou une variable dont la liaison est certifiée fonction (`certified_fn_binding`) — un `Expr::CompApp` ne correspond à aucun des deux et retombe dans le continue. Aucun autre point du moteur n'évalue les arguments d'un hook Custom : le `Dialog` de l'exemple n'est donc bien ni inliné ni havoqué.

Enfin, trois commentaires de code sont en retard sur lui : `components_analyzed` (« including re-analyses »), `recursive_component_refs` et `recursive_components` (« `T` » au lieu de `Stable`), et la doc du `ComponentCache` (« sound over-approximation » pour une entrée dont le résultat est celui de la dernière insertion).

Résumé du chapitre

- Un analyseur intra lit les props comme `T` : sound, mais aveugle aux défauts qui traversent une frontière (un setter du parent appelé par l'enfant). L'analyse inter-composants (ADR-12) inline chaque enfant au point où le parent construit l'élément, avec les props abstraites du parent, et fait remonter ses écritures.
- **Identité** (ADR-40) : clé (`file`, `name`), `ComponentId` interné par le registre dans une `ComponentTable` portée par le résultat ; le nom d'affichage (`Name@file` en cas de collision) n'est qu'un rendu, jamais une clé (`RuleCtx::component()` contre `component_name()`). Un élément porte l'origine que son fichier a prouvée ; `resolve_child` répond `Resolved`, `Unknown` ou `Ambiguous`, et refuse de deviner.
- **Racines** : heuristique (ce qu'aucun `CompApp` ne référence, une référence non résolue comptant pour tous les homonymes), `-all-roots`, `-entry` (un nom qui ne sélectionne rien est une erreur d'usage).
- **Inlining** : `eval_comp_app` résout, coupe la récursion (sans havoc), évalue les props dans le tas, consulte le cache, analyse l'enfant sous un `InterCtx` fils, range son résultat. Un élément vaut toujours `Stable` ; un `return` direct est évalué deux fois par passe.
- **Flux ascendant** : un setter porte son propriétaire ; l'appeler écrit le `SharedStateStore`, monotone, que la boucle du parent importe à chaque test de convergence ; pas de point fixe de programme. Un setter qui s'échappe vers un enfant inanalysable est havoqué (trou : setter champ d'un littéral inline).
- **Cache** : égalité de treillis, cinq entrées, éviction par join. Ce n'est qu'un « déjà vu » : `results[child]` est le dernier raté, jamais un join, d'où un FN dépendant de l'ordre des racines.
- **Deux phases** : racines sous `InterCtx`, puis balayage intra ; `phase1_reached` photographié entre les deux ; hors de lui, une ascendance vide est inconnue (`complete_ancestry`).
- **Graphe de symboles** (ADR-13) : construit, trié, affiché en `-verbose`, n'ordonne rien.
- **Dépendance de rendu** (ADR-41, définition 16.3) : analyse avant séparée sur le composant convergé ; `Deps = P(Src) ∪ {T}`, may, avec `gated` (must) et `writes` ; dépendance de contrôle

par les branches qui contrôlent un bloc ; `genuine` (utilisé) contre `sites` (transmis) ; `handlers`, contextes (`Context(id)`, `any_context`), bindings de module (`Module(name)`). Composition sur l'arbre à origine prouvée : `uses`, `home_of`, `landings`, `wasted_siblings` ; tout inconnu est un usage.

- Fichiers : `component_registry.rs`, `root_detector.rs`, `component_cache.rs`, `symbol_graph.rs`, `analysis_result.rs`, `program_result.rs`, `render_deps.rs` ; `component_id.rs`, `context.rs`, `state_value.rs`, `fixpoint.rs` ; `rules/helpers/render_tree.rs`.

La partie suivante passe aux règles : le chapitre 17 montre comment elles lisent ces résultats par des requêtes typées et un sceau de sévérité ; `state-lifted-too-high` et `wasted-subtree-render` (chapitres 18 et 21) sont les deux consommateurs de la dépendance de rendu.

Exercices

Exercice 16.1: Compter les passages dans le cache

Sur `drill.tsx` (liste 16.2), `-verbose` affiche « cache hits: 3, misses: 7 » et « App: 1 iteration(s) ». Retrouver ces nombres à partir des trois faits du section 16.4.4.

Solution

App fait deux passes (une itération, due au setter appelé par `Field` via le store partagé). Son `return <Layout .../>` est direct : deux évaluations par passe. Passe 1 : raté (`Layout` analysé, donc `Sidebar`, `Field` et `Content`), puis succès. Passe 2 : `value` a changé, raté (`Layout` ré-analysé, et avec lui `Sidebar` et `Field`, dont les props ont changé), puis succès. `Content` (props vides) rate la première fois et réussit à la seconde analyse de `Layout`. Ratés : 2 (`Layout`) + 2 (`Sidebar`) + 2 (`Field`) + 1 (`Content`) = 7 ; succès : 2 (`App`) + 1 (`Content`) = 3.

Exercice 16.2: Pourquoi tous les homonymes

Dans l'heuristique des racines, une référence sans origine marque comme référencés *tous* les composants de ce nom. Que se passerait-il si elle n'en marquait aucun ? si elle n'en marquait qu'un, le premier par tri ?

Solution

N'en marquer aucun ferait de chaque homonyme une racine analysée avec des props `⊥` ; si l'un d'eux est aussi inliné par un parent qui l'importe, l'ordre des identifiants peut faire gagner le résultat imprécis (écrasement de `results`). N'en marquer qu'un reproduirait la devinette qu'ADR-40 a supprimée : la référence ne désigne pas forcément celui-là. Marquer tous les homonymes ne coûte que des racines en moins, jamais une conclusion.

Exercice 16.3: Corriger l'écrasement à la racine

Reproduire l'exemple 16.8, puis proposer une correction de `results[child]` qui se justifie en un paragraphe (« règle du paragraphe unique »).

Solution

Deux pistes. Indexer les résultats par (identité, props abstraites) — la clé du cache — et faire tourner les règles intra-composant sur chaque contexte d'un composant : c'est la sensibilité au contexte qu'ADR-12 §2 revendique, poussée jusqu'aux règles ; le coût est un multiplicateur borné par la taille du cache. Ou joindre les résultats d'un même composant, ce qui exige un join d'`AnalysisResult` qui n'existe pas (CFG, relations et témoins ne se joignent

pas comme des états, section 16.6). La première se justifie en une phrase (« une règle doit voir chaque contexte d'appel analysé »); la seconde non. Aucune n'est implémentée au snapshot.

Exercice 16.4: Perdre le fait `gated`

Écrire un handler `onKeyDown` qui appelle `setQ` derrière `if (e.key === "Enter")` et ailleurs sans test. Que devient `gated`? Relier la réponse à liste 16.37.

Solution

Par exemple `(e) => if (e.key === "Enter") setQ(e.currentTarget.value); log(setQ);`. `param_gated_vars` trouve `setQ` dans un bloc contrôlé par un test de `e` et dans un bloc non contrôlé : `setQ` est dans `open`, donc pas gardé. Plus généralement, dans `union_with`, une source apportée non gardée par un seul contributeur est retirée de `gated` : le fait est `must`. Le déclencheur redevient continu. Observé sur `/tmp/ch16/gated.tsx`, avec un `<Heavy/>` voisin : sans option, `wasted-subtree-render` ne signale que le composant `Mixed` (« each 'keydown' event writes state 'q' and re-renders '<Heavy>' »), pas `Keyed`, dont le seul appel est gardé.

Exercice 16.5: Repères

Expliquer pourquoi `Module(name)` et `Context(id)` descendent l'arbre d'éléments sans traduction, alors que `Slot(l)` et `Prop(p)` doivent être traduits à chaque saut.

Solution

`Slot(l)` désigne le slot *l* du composant courant : le même label dans l'enfant est un autre slot. `Prop(p)` désigne la prop *p* reçue : la prop value du parent devient, chez l'enfant, la prop sous laquelle il la transmet. Un nom de module ou un contexte prouvé désigne le même objet dans tous les composants (appariement par orthographe pour le premier, `ContextId` canonique pour le second) : uses les recopie tels quels dans la question traduite.

Exercice 16.6: Un havoc complet

Sur l'exemple 16.6, où faut-il corriger pour que B et D soient havoqués, et pourquoi pas dans une règle?

Solution

Dans `collect_escaping_setters`, au niveau où il descend dans les champs d'un `ObjectLit` ou les éléments d'un `ArrayLit` : un champ dont la valeur est un setter (`as_setter()`) doit être poussé, comme l'est le setter nu au niveau de la prop. C'est le mécanisme partagé de soundness du domaine des valeurs : toutes les règles lisent l'état qu'il calcule, et un correctif par règle serait le contournement que le premier principe du projet interdit.

Exercice 16.7: Ce qui manque à `nested`

Sur l'exemple 16.10, quel fait faudrait-il remonter de l'analyse du callback pour que `<Child>` ne soit pas signalé? Pourquoi la copie des seules variables libres aggrave-t-elle le problème?

Solution

Il faudrait joindre, dans l'environnement du rendu, la dépendance des bindings *extérieurs* que le callback affecte (ici `label`, avec le pc du test `i === sel`). Or `nested` jette l'environnement de sortie, et `compute_free_vars` retire les cibles d'Assign des variables libres : `label` n'est même pas copié dans l'environnement du callback, et rien ne dit qu'il est écrit. Les racines

écrites existent pourtant déjà (`written_roots`) : c'est le fait à réutiliser.

Exercice 16.8: Le bottom qui n'est pas neutre

Pourquoi `AnalysisResult::exit_env` joint-il les environnements de sortie avec `reduce` et non avec `fold(AbstractEnv::bottom(), join)` ?

Solution

Dans le `join` d'environnements, une clé présente d'un seul côté prend $\top$ (chapitre 11). Partir de l'environnement vide ferait donc passer à $\top$ toute variable de l'autre environnement : $\perp$ n'est pas l'élément neutre du `join`. `reduce` part du premier environnement réel, et seul le cas « aucun bloc `Return` » rend l'environnement vide.

Cinquième partie

Les règles

Chapitre 17

La couche règles : requêtes typées, sévérité certifiée, témoins

Objectifs du chapitre.

- Situer la couche règles dans le pipeline : ce qu'elle lit (le point fixe convergé et les relations du moteur), ce qu'elle produit (des diagnostics gradués, localisés, expliqués) et ce qu'elle s'interdit (parcourir la syntaxe).
- Connaître le contrat d'une règle (trait `Rule`, `RuleCtx`, `SafeCheck`) et savoir écrire la plus simple d'entre elles.
- Comprendre comment le système de types de Rust garantit qu'une `ERROR` ne peut naître que d'une preuve : le sceau de `Diagnostic`, le jeton `Certified`, les verdicts polarisés `MustResult` et `May`.
- Lire les primitives `must` qui frappent ces jetons (dominance sur les sorties, plus grand point fixe, quantificateur $\forall$ -stable, frappe au point de connaissance) et savoir en ajouter une en respectant le contrat.
- Connaître le vocabulaire fermé des témoins (`Step`, `Note`, `FileId`) et la manière dont une chaîne devient une sortie humaine ou JSON.
- Suivre la passe par composant du registre (`check`, `safe_check`, `located`, `clamp`, `suspension`, `filtres`, `tri`), le cache programme et la frontière « walk-free » qui sépare les règles du moteur.

Prérequis : chapitre 3 (faits `must` et `may`, définition 3.7; `soundness`, définition 3.8; dominance, définition 3.9); chapitre 10 (treillis `Stability`, définition 10.3); chapitre 12 (analyse d'un CFG, `compute_dominators`); chapitre 14 (relations du moteur, définition 14.1) et chapitre 15 (graphe de `churn`, définition 15.1). Les chapitres suivants (chapitre 18, chapitre 19, chapitre 20, chapitre 21, chapitre 22) supposent ce chapitre lu.

Les chapitres précédents ont construit, composant par composant, un point fixe abstrait et un ensemble de relations nommées. Il reste à en tirer ce qu'un développeur veut lire : « ce hook est appelé conditionnellement, ligne 5 », avec un niveau de confiance juste et une explication. C'est le travail de la *couche règles*, le répertoire `src/rules/`. Son doc-comment de tête résume la découpe :

Extrait 17.1 – `src/rules/mod.rs`, l. 1–16 — la découpe de la couche règles

```
1  /// The rules layer (ADR-006): pure post-passes over the converged fixpoint.
2  ///
3  /// Layout:
4  /// - [`api`] – the typed surface a rule programs against (ADR-021): query
5  ///   primitives, the sealed [`Diagnostic`], the witness vocabulary, and the
6  ///   [`ProgramCache`] that holds whole-program derived data.
7  /// - [`impls`] – the rule implementations, one file per rule.
8  /// - [`helpers`] – shared analysis machinery (setter/churn/eval/scans).
9  /// - [`docs`] – static documentation for every diagnostic name.
```

```

10 /// - ['declarative'] - the declarative pack frontend (ADR-022 Tier A):
11 /// loader, validator, executor.
12 ///
13 /// This module keeps only the ['Rule'] trait, ['SafeCheck'], the native rule
14 /// set (['all_rules']), the dynamic ['registry'] (ADR-022) and the public
15 /// façade - every name re-exported here is at its historical path, so
16 /// consumers never reach into submodules.

```

Ce chapitre ne présente pas les règles une à une (c'est l'objet des chapitres 18 à 21) ; il présente la *surface* contre laquelle toutes sont écrites, et la manière dont cette surface rend certaines erreurs de raisonnement impossibles à compiler. On part d'un faux négatif réel, qui a motivé toute la conception (ADR-21).

17.1 Le problème : dire vrai, avec la bonne certitude

Une règle ne se contente pas de dire « il y a peut-être un problème ». Elle choisit un niveau parmi trois (chapitre 1) : **ERROR** quand le défaut est certain dès que le code s'exécute, **WARNING** quand il n'est que possible (ou certain mais d'un coût incertain), **INFO** quand elle avoue une limite de l'analyse ou un motif qui a l'air voulu. Se tromper de niveau vers le haut est un mensonge (une **ERROR** sur un fait seulement possible) ; se taire sur un défaut possible est un faux négatif (FN), interdit par l'invariant de soundness (définition 3.8). Les deux fautes naissent au même endroit : là où une règle traduit une valeur abstraite en décision.

17.1.1 Deux prédicats qui n'étaient pas complémentaires

Considérons un composant qui écrit son propre état dans un effet, sans condition :

Extrait 17.2 – Un effet qui s'auto-relance tant que data bouge (/tmp/ch17/topdep.tsx)

```

1 import { useState, useEffect } from "react";
2
3 export function Poller({ data }: { data: unknown }) {
4   const label = "fixed";
5   const [n, setN] = useState(0);
6   useEffect(() => {
7     setN(n + 1);
8   }, [label, data]);
9   return <div>{n}</div>;
10 }

```

`data` est une prop : l'analyse n'en sait rien, sa valeur abstraite est $\top$. Chaque exécution de l'effet écrit $n + 1$, donc provoque un nouveau rendu ; si `data` change d'identité à chaque rendu (un objet recréé par le parent, par exemple), l'effet se relance, et la boucle ne s'arrête jamais. La règle *infinite-loop* doit donc au minimum émettre un **WARNING**.

Avant ADR-21, elle ne le faisait pas. Le domaine offrait deux prédicats sur la stabilité d'une valeur, `is_stable()` (vrai seulement pour `Stability::Stable`) et `is_unstable()` (vrai seulement pour `PerRender`). Un helper partagé, `all_deps_unstable`, était construit sur le second, et la règle sautait l'effet par `if !all_deps_unstable(deps) continue`. Or pour une `dep` à $\top$, les deux prédicats rendent `false` : `is_unstable() == false`, donc `all_deps_unstable == false`, donc la règle passe à l'effet suivant, et la boucle possible n'est jamais rapportée. L'ADR le qualifie sans détour : « a real soundness FN — the cardinal sin — written by the maintainer, shipped, in a *shared* helper ».

valeur Stability	is_stable()	is_unstable()	ce que la valeur permet
Stable	vrai	faux	ne change jamais
PerRender	faux	vrai	change à chaque rendu
Versioned, VersionedTop	faux	faux	change quand un état change
Unknown ($\top$)	faux	faux	peut changer à chaque rendu

TABLE 17.1 – Les deux prédicats historiques (d’après ADR-21, section « The false negative hiding underneath »). Les deux dernières lignes tombent dans le « trou » : ni stables, ni instables. Une porte construite sur `is_unstable` les traite comme stables.

Le tableau 17.1 montre le trou. Il n’y a là aucune subtilité mathématique : $\top$ se concrétise en « n’importe quelle valeur », donc en particulier en une valeur qui change ; le prédicat qu’il fallait était la *négation* de `is_stable`, pas `is_unstable`. Mais la convention « $\top$ se replie du côté `may` » vivait dans la tête de chaque auteur de règle, et une seule inattention suffisait.

17.1.2 Deux corrections, et la leçon

La correction s’est faite en deux temps (ADR-21, section « Hardening round »). D’abord `is_unstable` a été retiré de la surface des règles et remplacé par une sonde dont le nom dit la polarité, `may_change` ($\top$, `Versioned`, `PerRender` $\mapsto$ vrai). Ensuite, le quantificateur a été corrigé : la première version sautait l’effet dès qu’une dep était prouvée stable, ce qui faisait réapparaître le FN sur `[label, data]` — exactement le liste 17.2. React relance un effet si *une* dep a changé (sémantique « ou ») ; la seule situation où le tableau de deps protège vraiment est celle où *toutes* les deps sont prouvées stables (section 17.5.3). Au commit e67b10a, l’analyseur rapporte bien ce composant :

```

Poller (2 hooks) topdep.tsx
warn  infinite-loop [hook:0] (line 6:2) this effect keeps pushing state `n` (its deps do not provably gate it, so
    ↳ the effect can re-run every render) to new values on every run. Potential infinite render loop
    ↳ state `n` is written here [hook:1] (line 6:2)
    ↳ the abstract value of state `n` kept growing and was widened at iteration 3
warn  missing-deps [hook:1] var:n (line 6:2) `n` is used in this effect but not in its deps array, and it is
    ↳ recreated on every render
    ↳ `n` is read here [hook:1] (line 6:2)

```

Trois tests épinglent le comportement dans `tests/effect_cycles.rs` :

- `top_prop_dep_does_not_silence_self_write_loop`, la même boucle avec la seule dep `data` ;
- `stable_dep_alongside_top_dep_does_not_gate_self_write_loop`, notre composant, à un nom près ;
- leur complément `all_stable_deps_gate_self_write_effect`, qui vérifie que des deps toutes constantes font bien taire la règle.

La leçon tirée par l’ADR va plus loin que le bug. Une étude y a prototypé cinq façons d’écrire des règles (Rust, JSON déclaratif, Datalog, rappel JavaScript, Starlark) et conclu que « the frontend is not the soundness lever — the query surface is » : tant que des sondes sûres et des sondes dangereuses coexistent, la discipline de l’auteur est la seule barrière. D’où le programme de ce chapitre : faire porter la polarité par les *types*, de sorte que confondre `must` et `may`, ou attacher `Severity::Error` à un booléen quelconque, soit une erreur de compilation.

Décision : ADR-021 — une surface de requête typée

Décision. La polarité (`must`, `may`, $\top$) est un type ; la sévérité `ERROR` est certifiée par un jeton que seules des primitives du module `src/rules/api/query.rs` peuvent frapper ; $\top$ est

une variante *retournée* de chaque classifieur, repliée du côté may à l'intérieur de la primitive ; toutes les règles se lient à un objet unique, RuleCtx. **Alternatives refusées.** (1) Faire du choix du frontend d'écriture le levier de soundness : l'étude montre que chaque frontend, tel que prototypé, plafonnait à WARNING pour un auteur non fiable. (2) Une migration progressive : « a transition window would leave the FN vector open » ; la migration a été faite d'un bloc, avec les quatorze règles de l'époque. (3) Un All(T) nu dans MustResult : le jeton vit *dans* All pour que toutes les primitives partagent une seule histoire de frappe. **Confiance résiduelle.** L'annotation de polarité de chaque primitive : une primitive may étiquetée must rouvrirait l'ERROR-sur-may. Elle est désormais localisée dans un seul fichier.

Remarque

ADR-6 avait posé la frontière moteur/règles : les règles sont des post-passes pures sur le point fixe convergé, et le moteur n'émet aucun diagnostic (la seule exception historique, l'enregistrement du widening, est devenue widen_trace). ADR-21 ne remet pas cette frontière en cause ; il installe une barrière à *l'intérieur* du code des règles. Quant à ADR-7, qui organisait les requêtes entre domaines pendant le transfert (QueryContext), il concerne le moteur ; ADR-21 dit en réaliser l'idée côté règles, sous la forme d'un contexte typé. Au commit e67b10a, seuls QueryContext, NullCtx et FixpointCtx subsistent de cet ADR dans src/domains/context.rs.

17.2 Le trait Rule et une première règle

17.2.1 Le contrat

Une règle est une structure sans état qui implémente le trait Rule :

Extrait 17.3 – src/rules/mod.rs, l. 75–106 — le trait Rule

```

75 /// Post-pass analysis rule operating on a fully-computed `AnalysisResult`.
76 ///
77 /// Rules are stateless; adding a new rule = new struct + `impl Rule`.
78 ///
79 /// Both methods bind to [ `RuleCtx` ] (ADR-021 §4): the caller resolves the
80 /// component once ( `RuleCtx::new` ), and the ctx is the stable anchor the
81 /// future external frontends bind to.
82 pub trait Rule {
83     /// Rule id. Borrowed from `self` so dynamically loaded rules (ADR-022) can
84     /// own their `pack/rule` id; native impls keep returning `&'static str`.
85     fn name(&self) -> &str;
86     fn check(&self, ctx: &RuleCtx) -> Vec<Diagnostic>;
87
88     /// When this rule is *applicable* to the ctx's component but `check` found
89     /// nothing, the positive assurance to surface under `--info`.
90     ///
91     /// Only consulted after `check` returned no diagnostics for the component,
92     /// so implementations decide *applicability* only – they need not re-check.
93     /// Default `None`: the rule opts out (e.g. Info-limitation rules, which have
94     /// no "safe" state to report).
95     fn safe_check(&self, _ctx: &RuleCtx) -> Option<SafeCheck> {
96         None
97     }
98
99     /// The options a built-in rule accepts ( `rules.<name>.options` in the
100     /// config, `--rule-option <name>:<key>=<value>` on the CLI). Default: none,
101     /// and the registry refuses any option addressed to the rule. Pack rules
102     /// keep their own, schema-free options (ADR-022 §4).

```

```

103     fn options(&self) -> &'static [OptionSpec] {
104         &[]
105     }
106 }

```

Trois méthodes, dont une seule obligatoire en plus du nom :

- `name()` rend l'*identifiant de règle*. Il est distinct du *nom de diagnostic* que porte chaque finding (`Diagnostic::rule`) : une règle peut émettre sous plusieurs noms. `InfiniteLoop` émet aussi `cross-component-infinite-loop`, `SetterInRender` émet aussi `cross-setter-in-render`. Cette distinction réapparaîtra au registre (section 17.7), où elle protège d'un FN.
- `check(&RuleCtx)` rend la liste des diagnostics du composant. Elle est pure : elle lit le contexte et ne modifie rien. Sa signature d'origine (ADR-6) prenait un `&AnalysisResult` et rendait des `Warning` ; ADR-21 (§4) l'a remplacée par un `&RuleCtx`, l'ancre à laquelle se lieront aussi les frontends externes.
- `safe_check` décrit l'*assurance positive* affichée sous `-info` (« `verified: ...` ») quand la règle était applicable et n'a rien trouvé. Elle ne décide que de l'*applicabilité* : le registre ne l'appelle que si `check` a rendu une liste vide.
- `options()` déclare les options typées qu'accepte une règle native (section 17.7.5).

Le trait n'est pas `Send + Sync`, alors que la signature prévue par ADR-6 l'exigeait : la passe des règles est séquentielle (section 17.8). L'assurance est une petite structure de deux chaînes statiques :

Extrait 17.4 – `src/rules/mod.rs`, l. 61–73 — `SafeCheck`, une assurance positive

```

61 /// A check that was *applicable* to a component and found nothing wrong -
62 /// surfaced under `--info` as positive assurance ("verified: ...").
63 ///
64 /// Distinct from an absent diagnostic: emptiness alone cannot tell "the
65 /// infinite-loop check ran and the component is safe" from "there was no
66 /// useState/useEffect for it to check". A `SafeCheck` records the former only.
67 #[derive(Debug, Clone, PartialEq, Eq)]
68 pub struct SafeCheck {
69     /// Diagnostic name the assurance corresponds to (matches `RuleDoc::name`).
70     pub rule: &'static str,
71     /// Present-tense assurance, e.g. "no effect diverges into an infinite loop".
72     pub message: &'static str,
73 }

```

Le doc-comment insiste sur la différence entre « aucune alerte » et « vérifié » : un composant sans `useEffect` n'a aucune boucle infinie, mais il n'a pas non plus été *vérifié* contre elle. Seul le premier cas mérite une ligne `verified`.

17.2.2 L'objet de contexte `RuleCtx`

Tout ce qu'une règle peut lire passe par `RuleCtx` :

Extrait 17.5 – `src/rules/api/query.rs`, l. 314–340 — `RuleCtx` et sa référence au cache

```

314 /// The single object a rule's `check`/`safe_check` binds to: the program result,
315 /// the component under analysis, its resolved [`AnalysisResult`], and the typed
316 /// query primitives (methods, added in the primitives section). The stable anchor
317 /// the future external frontends bind to (ADR-021 §4).
318 pub struct RuleCtx<'a> {
319     cache: CacheRef<'a>,
320     component: ComponentId,
321     comp: &'a AnalysisResult<StateValue>,

```

```

322     config: RuleConfig,
323 }
324
325 /// The ctx's [`ProgramCache`]: shared with every other component of the same
326 /// program when the dispatcher supplies one, private when a caller builds a
327 /// one-off ctx (single-component callers, tests).
328 enum CacheRef<'a> {
329     Shared(&'a ProgramCache<'a>),
330     Own(Box<ProgramCache<'a>>),
331 }
332
333 impl<'a> CacheRef<'a> {
334     fn get(&self) -> &ProgramCache<'a> {
335         match self {
336             CacheRef::Shared(c) => c,
337             CacheRef::Own(c) => c,
338         }
339     }
340 }

```

Quatre champs, tous privés. `component` est l'identité du composant analysé (un `ComponentId`, à comparer) ; `comp` est son `AnalysisResult<StateValue>`, résolu *une fois* à la construction (`build` indexe `program.components[&component]` et panique si le composant est absent : le dispatcher ne construit un contexte que pour un composant analysé) ; `config` porte les options de la règle ; `cache` donne accès au programme entier et aux structures qui en dérivent (section 17.8).

Les accesseurs publics sont `program()`, `component()` (l'identité), `component_name()` (le nom d'affichage, « never for a comparison » : il dépend du contenu, #7), `comp()` et `config()` ; `cache()` est `pub(in crate::rules)`, invisible d'un frontend externe. Trois constructeurs existent (`src/rules/api/query.rs`, l. 342–382) : `RuleCtx::new` et `with_config`, qui créent un cache *privé*, pour les tests et les appels isolés ; `RuleCtx::cached`, qu'emploie le registre, qui *partage* le cache de tout le programme.

17.2.3 Exemple minimal : conditional-hook

La règle la plus simple du dépôt tient en trente lignes. Elle signale un hook qui n'est pas appelé à chaque rendu, ce qui casse l'association par ordre d'appel entre hooks et slots d'état.

Rappel React : l'ordre des hooks

React n'identifie pas un `useState` par son nom mais par son *rang* dans la suite des appels de hooks du rendu. Si un rendu saute un hook, tous les suivants se décalent d'un cran et lisent l'état d'un autre. D'où la règle « n'appellez les hooks qu'au niveau supérieur » [Met26d].

Extrait 17.6 – Un hook sous condition (/tmp/ch17/profile.tsx)

```

1  import { useState } from "react";
2
3  export function Profile({ visible }: { visible: boolean }) {
4      if (visible) {
5          const [n, setN] = useState(0);
6          return <div onClick={() => setN(n + 1)}>{n}</div>;
7      }
8      return null;
9  }

```

Extrait 17.7 – `src/rules/impls/conditional_hook.rs`, l. 14–46 — la règle `conditional-hook` en entier

```

14 impl Rule for ConditionalHook {
15     fn name(&self) -> &'static str {
16         Self::NAME
17     }
18
19     fn safe_check(&self, ctx: &RuleCtx) -> Option<crate::rules::SafeCheck> {
20         let (result, component) = (ctx.program(), ctx.component());
21         // Applicable as soon as the component calls any hook at all.
22         result
23             .components
24             .get(&component)
25             .is_some_and(|c| !c.hook_calls.is_empty())
26             .then_some(crate::rules::SafeCheck {
27                 rule: Self::NAME,
28                 message: "all hooks run unconditionally, in a stable order",
29             })
30     }
31
32     fn check(&self, ctx: &RuleCtx) -> Vec<Diagnostic> {
33         // The dominance  $\forall$ -exits check + guard witness live in the primitive; a
34         // conditional hook yields a `Certified`, the only path to `error()`.
35         ctx.hook_is_conditional()
36             .into_iter()
37             .map(|proof| {
38                 Diagnostic::error(
39                     "conditional-hook",
40                     proof,
41                     "this hook is called conditionally (not on every render path)",
42                 )
43             })
44             .collect()
45     }
46 }

```

Toute l'analyse est dans la primitive `ctx.hook_is_conditional()`, qui rend un vecteur de *preuves* `Certified<ConditionalHookCall>`; la règle transforme chaque preuve en `ERROR` par `Diagnostic::error`, et c'est la seule manière d'en construire une (section 17.4). Le `safe_check` déclare la règle applicable dès que le composant appelle au moins un hook. L'analyseur produit :

```

Profile (2 hooks) profile.tsx
error conditional-hook [hook:0] (line 5:10) this hook is called conditionally (not on every render path)
→ guarded by a condition evaluated here, so some render paths skip the hook (line 4:6)

```

La ligne principale porte le label du hook (`[hook:0]`) et sa position ; la ligne `→`, visible avec `-trace`, est un *témoin* : l'étape qui explique pourquoi le hook est conditionnel, ici la condition visible de la ligne 4. Ni la position ni le témoin n'ont été écrits par la règle : ils voyagent *dans* la preuve (section 17.5.1). Le « (2 hooks) » compte aussi le handler `onClick`, que la primitive filtre. Le détail de la règle (formulation par dominance, cas limites, FN connus) est au section 20.3.

17.2.4 Les dix-neuf règles natives

`all_rules()` instancie les règles natives dans un ordre fixe, qui est aussi leur ordre d'exécution :

Extrait 17.8 – `src/rules/mod.rs`, l. 108–131 — les règles natives

```

108 /// Instantiate all built-in rules.

```



```

109 pub fn all_rules() -> Vec<Box<dyn Rule>> {
110     vec![
111         Box::new(ConditionalHook),
112         Box::new(MissingDeps),
113         Box::new(MissingCleanup),
114         Box::new(AlwaysUnstableDeps),
115         Box::new(LazyInit),
116         Box::new(RedundantSetState),
117         Box::new(UnnecessaryRerender),
118         Box::new(SetterInRender),
119         Box::new(ServerComponentHook),
120         Box::new(StaleClosure),
121         Box::new(StateMutation),
122         Box::new(StateLiftedTooHigh),
123         Box::new(WastedSubtreeRender),
124         Box::new(InfiniteLoop),
125         Box::new(DerivedState),
126         Box::new(FrozenInitialState),
127         Box::new(UnstableContextValue),
128         Box::new(WideningInfo),
129         Box::new(AnalysisLimitInfo),
130     ]
131 }

```

#	Rule::name()	autres noms émis	constructeurs présents
1	conditional-hook	—	error
2	missing-deps	—	warn
3	missing-cleanup	—	warn
4	always-unstable-deps	—	warn
5	lazy-init	—	error, warn, info
6	redundant-set-state	—	warn
7	unnecessary-rerender	—	warn
8	setter-in-render	cross-setter-in-render	error, warn
9	server-component-hook	—	warn
10	stale-closure	—	error, warn
11	state-mutation	—	error, warn
12	state-lifted-too-high	—	warn
13	wasted-subtree-render	—	warn
14	infinite-loop	cross-component-infinite-loop	error, warn, info
15	derived-state	—	warn
16	frozen-initial-state	—	error, warn, info
17	unstable-context-value	—	warn
18	widening-info	—	info
19	analysis-limit	—	info

TABLE 17.2 – Les règles natives dans l’ordre d’all_rules(). La dernière colonne relève les constructeurs Diagnostic::* présents dans chaque fichier de src/rules/impls/ : c’est le niveau le plus haut qu’une règle *peut* atteindre avant toute surcharge. Quinze règles implémentent safe_check ; les quatre autres sont analysis-limit, widening-info, state-lifted-too-high et wasted-subtree-render.

Le tableau 17.2 se lit comme une carte des preuves : sept règles seulement contiennent un appel à Diagnostic::error, donc sept seulement consomment un jeton Certified. Toutes les autres plafonnent à WARNING *par construction*. La table des docs compte vingt et une entrées, parce qu’elle est indexée par nom de diagnostic et que deux règles émettent un second nom.

Piège

Deux commentaires du registre annoncent « the 14 native rules and the 16-entry doc table » et « 16 native entries » (src/rules/registry.rs, l. 121–130) : ils datent de la migration d'ADR-21. Au commit e67b10a, il y a dix-neuf règles et vingt et une docs. Le test `natives_match_all_rules_order` (src/rules/registry.rs, l. 452–459) vérifie l'ordre et le compte réels, pas les commentaires.

17.3 Diagnostic et Severity : le sceau

17.3.1 Trois niveaux, deux ordres

Extrait 17.9 – src/rules/api/diagnostic.rs, l. 24–52 — les niveaux et leur rang

```

24 /// Confidence level of a diagnostic.
25 ///
26 /// - `Error` the defect is certain whenever the flagged code runs. Built only
27 /// from a proof of the whole claim, not of one of its conjuncts (#142).
28 /// - `Warning` a possible defect (conditional path, over-approximation), or a
29 /// certain fact whose cost is not (a fresh reference, one wasted render).
30 /// - `Info` not actionable without context: a known analysis limitation
31 /// (widening, depth cap), or a pattern that looks intentional (a seed-once
32 /// prop name, a cheap pure initializer). Hidden by default; show with --info.
33 #[derive(Debug, Clone, Copy, PartialEq, Eq, Default)]
34 pub enum Severity {
35     Error,
36     #[default]
37     Warning,
38     Info,
39 }
40
41 impl Severity {
42     /// Lattice rank for the `⊔` clamp (ADR-022 §3): Error > Warning > Info.
43     /// Distinct from the enum discriminant, whose order (Error < Warning <
44     /// Info) the deterministic output sort depends on.
45     fn rank(self) -> u8 {
46         match self {
47             Severity::Error => 2,
48             Severity::Warning => 1,
49             Severity::Info => 0,
50         }
51     }
52 }

```

Le doc-comment est la définition normative des trois niveaux, la même que celle du `CLAUDE.md` du dépôt : une `ERROR` se construit « only from a proof of the whole claim, not of one of its conjuncts » (#142, section 17.5.6) ; un `WARNING` est un défaut possible *ou* un fait certain dont le coût ne l'est pas ; une `INFO` n'est pas actionnable sans contexte et reste cachée sans `-info`.

Deux ordres coexistent sur ce type, et il ne faut pas les confondre. Le *rang* (rank, privé) est l'ordre de confiance `INFO < WARNING < ERROR` ; c'est sur lui que se calcule le meet du clamp (section 17.7.2). Le *discriminant* de l'énumération suit l'ordre de déclaration, `Error = 0`, `Warning = 1`, `Info = 2` ; c'est lui que le tri de sortie utilise (`a.severity() as u8`), pour que les `ERROR` viennent en tête. Le type ne dérive volontairement pas `PartialOrd` : un ordre dérivé serait celui du discriminant, faux pour le clamp.

Piège

Parce que `rank` est privé au module `feuille`, `src/rules/impls/infinite_loop.rs` en ré-implémente une copie (`src/rules/impls/infinite_loop.rs`, l. 472–477, commentaire « `Severity has no Ord: rank Error > Warning > Info manually` ») pour choisir le meilleur `finding` par slot. Deux sources de vérité pour le même ordre : si un quatrième niveau apparaissait, il faudrait penser aux deux.

17.3.2 La structure et son champ scellé

Extrait 17.10 – `src/rules/api/diagnostic.rs`, l. 54–73 — `Diagnostic`, dont seule la sévérité est privée

```
54 /// Finding produced by a rule against the fixpoint analysis result.
55 #[derive(Debug, Clone, PartialEq)]
56 pub struct Diagnostic {
57     /// Private (ADR-021): the seal. Set only by `error`/`warn`/`info`; read
58     /// through [ `Diagnostic::severity` ]. A `pub` field here would let any code
59     /// forge an Error by mutation or struct literal.
60     severity: Severity,
61     /// Diagnostic name. `Cow`: native rules pass string literals (borrowed,
62     /// zero-cost), dynamically loaded rules own their `pack/rule` id.
63     pub rule: Cow<'static, str>,
64     pub message: String,
65     /// Hook label most directly involved, if any.
66     pub hook_label: Option<HookLabel>,
67     /// Variable name most directly involved, if any.
68     pub var: Option<Var>,
69     /// Source location of the primary finding, if available.
70     pub range: Option<SourceRange>,
71     /// Secondary evidence items explaining the causal chain.
72     pub notes: Vec<Note>,
73 }
```

Tous les champs sont publics sauf un. `rule` est un `Cow<'static, str>` : les natives passent des littéraux (empruntés, sans coût), les règles de packs possèdent leur identifiant `pack/rule`. `hook_label`, `var` et `range` désignent ce que le `finding` pointe ; `notes` est la chaîne de témoins (section 17.6). Le champ `severity`, lui, est privé, et le commentaire dit pourquoi : « a `pub` field here would let any code forge an Error by mutation or struct literal ».

17.3.3 Pourquoi un module `feuille`

Rendre le champ privé ne suffit pas, et c'est une leçon de Rust. La visibilité privée est *descendante* : un item privé d'un module est visible de ce module *et de tous ses descendants*. Quand `Diagnostic` vivait dans `src/rules/mod.rs`, chaque règle, sous-module de `rules`, pouvait encore appeler le constructeur « privé » et écrire `Diagnostic::new(..).with_severity(Severity::Error)`. Le premier jet d'ADR-21 avait ce défaut ; la revue l'a trouvé en compilant des sondes de forge. Le correctif a été de déplacer le type dans un module *feuille*, `src/rules/api/diagnostic.rs`, dont les règles sont des sœurs et non des descendantes :

Extrait 17.11 – `src/rules/api/diagnostic.rs`, l. 1–12 — le raisonnement du module `feuille`

```
1 /// The diagnostic type and its severity seal (ADR-021 §2).
2 ///
3 /// `Diagnostic` lives in this leaf module so that its `severity` field and
4 /// the free constructors (`new`/`with_severity`) are invisible to every rule
5 /// module. Rust privacy is downward: an item private to a module is visible to
```

```

6 /// that module's *descendants* – so when `Diagnostic` lived in `rules/mod.rs`,
7 /// every rule submodule could still call `Diagnostic::new(..).with_severity`
8 /// (Severity::Error)` or write `d.severity = Severity::Error` through the
9 /// then-`pub` field, silently bypassing the proof discipline. In a leaf module
10 /// the rules are siblings, not descendants: the only doors are
11 /// [ `Diagnostic::error` ] (requires a [ `Certified` ] proof), [ `Diagnostic::warn` ]
12 /// and [ `Diagnostic::info` ]. Severity is read through [ `Diagnostic::severity` ].

```

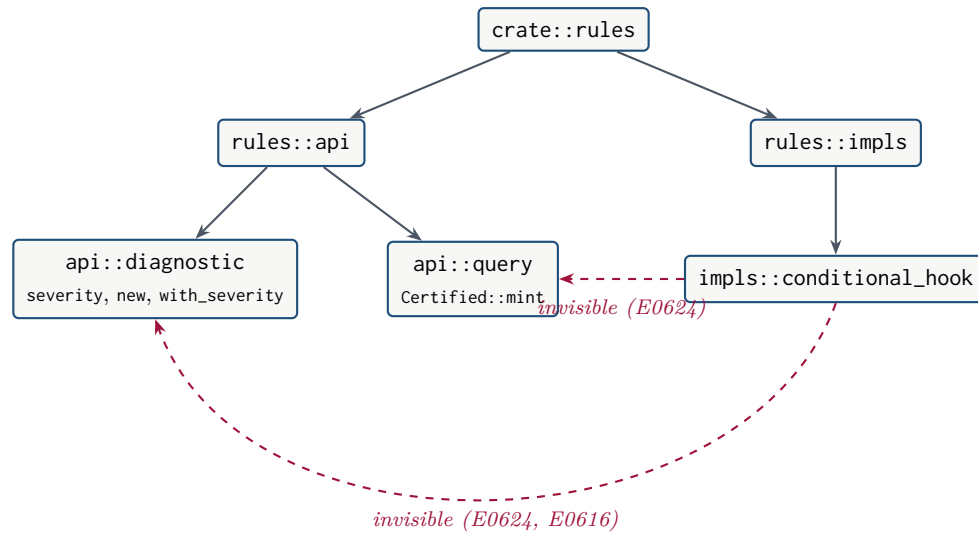


FIGURE 17.1 – L’arbre des modules de `src/rules/`. Un item privé est visible de son module et de ses descendants, jamais de ses sœurs ni de leurs enfants : ni la règle `conditional_hook` ni aucune autre ne voit le champ `severity`, les constructeurs privés de `Diagnostic` ou `Certified::mint`. Si `Diagnostic` était défini dans `crate::rules` lui-même, toute règle, descendante, les verrait.

La figure 17.1 résume la situation. Hors du module, trois portes seulement fabriquent un `Diagnostic` :

Extrait 17.12 – `src/rules/api/diagnostic.rs`, l. 157–190 — les trois portes

```

157 /// The only constructor of an `Error` (ADR-021 §2). Builds the finding
158 /// *from* a proof: the certified evidence's span/label/witness ride along, so
159 /// they need not be re-threaded by hand. A `May<_>`/`MustResult::Some` value
160 /// has no `Certified` to pass here – Error-on-may is a type error.
161 ///
162 /// Further `.with_*` builders may still refine/override the absorbed fields.
163 pub fn error<E>(<
164     rule: impl Into<Cow<'static, str>>,
165     proof: Certified<E>,
166     message: impl Into<String>,
167 ) -> Self {
168     let prov = proof.provenance();
169     Diagnostic {
170         severity: Severity::Error,
171         rule: rule.into(),
172         message: message.into(),
173         hook_label: prov.hook_label,
174         var: None,
175         range: prov.range,
176         notes: prov.notes.clone(),
177     }
178 }

```

```

179
180     /// A Warning: a safe over-claim (conditional path / over-approximation). Free
181     /// to construct – a Warning asserts no MUST fact.
182     pub fn warn(rule: impl Into<Cow<'static, str>>, message: impl Into<String>) -> Self {
183         Diagnostic::new(rule, message).with_severity(Severity::Warning)
184     }
185
186     /// An Info: a known analysis limitation, or a pattern that looks
187     /// intentional. Makes no must/may claim; hidden unless `--info`.
188     pub fn info(rule: impl Into<Cow<'static, str>>, message: impl Into<String>) -> Self {
189         Diagnostic::new(rule, message).with_severity(Severity::Info)
190     }

```

warn et info sont libres : un WARNING n'affirme aucun fait must, une INFO n'affirme rien. error est la seule porte vers une ERROR, et elle exige un argument de type Certified<E>. Elle absorbe la provenance de la preuve (position, label du hook, témoins) : la règle n'a pas à les recopier à la main, et ne peut pas les oublier. Le var reste vide ; la règle l'ajoute si elle le souhaite.

méthode	visibilité	effet
new(rule, message)	privée	sévérité par défaut (WARNING), champs optionnels vides
with_severity(sev)	privée	seule écriture de severity hors des portes
severity()	pub	lecture seule
clamp(ceiling)	pub	abaissement par le consommateur, jamais de montée
with_label, with_var, with_range	pub	remplissent hook_label, var, range (with_range écrase la position absorbée d'une preuve)
with_step(step, label, range, name)	pub	ajoute une Note dont le texte est step.render(name)
with_notes(notes)	pub	ajoute (extend) des notes déjà construites
error(rule, proof, message)	pub	seule porte ERROR ; absorbe range, hook_label, notes
warn, info	pub	new(..).with_severity(..)

TABLE 17.3 – La surface complète de Diagnostic (src/rules/api/diagnostic.rs, l. 74–191).

Le tableau 17.3 donne la surface complète. Une seule méthode publique touche la sévérité après construction, clamp, et elle ne sait que descendre (section 17.7.2).

17.4 Le typestate : Certified, MustResult, May

Le sceau interdit d'écrire Severity::Error dans un diagnostic. Il reste à décider *qui* a le droit de fournir l'argument de Diagnostic::error. C'est le rôle d'un *typestate* : un type dont l'existence même d'une valeur témoigne qu'un certain calcul a eu lieu.

17.4.1 La provenance et le jeton

Une preuve se compose de deux choses : ce qui a été prouvé (l'*évidence*, dont le type dépend de la primitive) et l'endroit où la preuve se trouve (la *provenance*).

Extrait 17.13 – src/rules/api/query.rs, l. 52–57 — la provenance d'une preuve

```

52 #[derive(Debug, Clone, Default, PartialEq)]
53 pub struct Provenance {
54     pub range: Option<SourceRange>,
55     pub hook_label: Option<HookLabel>,

```

```
56     pub notes: Vec<Note>,
57 }
```

range est la position de la preuve dans le source, hook_label le hook concerné, notes une chaîne de témoins déjà rendue (section 17.6). C'est un *défaut*, pas un verrou : Diagnostic::error l'absorbe, et la règle peut ensuite raffiner la position par with_range (state-mutation pointe ainsi le site de mutation plutôt que l'écriture qui prouve le cycle). Plusieurs primitives laissent la provenance vide, et le doc-comment (src/rules/api/query.rs, l. 37–51) le revendique : must_init_calls_setter reçoit une Expr, qui ne porte pas de position ; must_same_ref_mutation reçoit des identifiants de conteneurs ; must_on_all_paths un ensemble de blocs. Une provenance vide y est « honest, not an omission ».

Extrait 17.14 – src/rules/api/query.rs, l. 74–109 — le jeton de preuve Certified

```
74 /// A certified MUST fact: the evidence plus its provenance.
75 ///
76 /// The constructor (`Certified::mint`) is private to this module. Rule code in
77 /// sibling modules can only [`Certified::evidence`]/[`Certified::provenance`] –
78 /// it can never forge a token. This is the enforcement: an `Error` is reachable
79 /// only from a `Certified`, and a `Certified` only from a must-primitive here.
80 #[derive(Debug, Clone, PartialEq)]
81 pub struct Certified<E> {
82     evidence: E,
83     provenance: Provenance,
84 }
85
86 impl<E> Certified<E> {
87     /// Mint a token. **Private to the query module** – the whole point.
88     fn mint(evidence: E, provenance: Provenance) -> Self {
89         Certified {
90             evidence,
91             provenance,
92         }
93     }
94
95     pub fn evidence(&self) -> &E {
96         &self.evidence
97     }
98
99     /// Discard the proof and keep the evidence – a *downgrade* (the safe
100     /// direction: the fact loses its Error eligibility, never gains one).
101     /// The demotion path of [`must_frozen_seed`]’s gate check.
102     pub fn into_evidence(self) -> E {
103         self.evidence
104     }
105
106     pub fn provenance(&self) -> &Provenance {
107         &self.provenance
108     }
109 }
```

Définition 17.1: Jeton certifié, sceau de sévérité

Un *jeton certifié* est une valeur de type Certified<E> : une évidence de type E accompagnée d’une Provenance. Ses champs sont privés et son unique constructeur, Certified::mint, est privé au module src/rules/api/query.rs. Hors de ce module, on peut *lire* un jeton (evidence, provenance) ou le *dégrader* (into_evidence, qui rend l’évidence et détruit la preuve), jamais en fabriquer un.

Une primitive *must* est une fonction de ce module qui rend un verdict dont au moins une variante contient un jeton ; elle *frappe* (*mint*) ce jeton quand elle a établi le fait sur tous les chemins. Un *verdict polarisé* est l'une des formes `MustResult<T>` (`All(Certified<T>)`, `Some(T)`, `None`), `May<T>` ou un classifieur total dont `T` est une variante.

Le *sceau de sévérité* est la conjonction de deux faits de visibilité : le champ `severity` de `Diagnostic` n'est écrit que par `error`, `warn`, `info` et `clamp` ; `Diagnostic::error` exige un `Certified<E>`. Il s'ensuit que tout diagnostic de niveau `ERROR` a été construit à partir d'un jeton frappé par une primitive *must*.

Invariant 17.1: Les points de frappe

Au commit `e67b10a`, `src/rules/api/query.rs` contient exactement dix appels à `Certified::mint` : dans `hook_is_conditional` (`src/rules/api/query.rs`, l. 498), `must_setter_on_all_paths` (l. 618), `must_on_all_paths` (l. 631), `ExitDominance::certify` (l. 698), `must_init_calls_setter` (l. 736), `must_direct_write` (l. 765), `classify_motion` (l. 858), `must_stale_capture` (l. 982), `must_effect_cycle` (l. 1027) et `must_same_ref_mutation` (l. 1053). Le périmètre de confiance de toutes les `ERROR` de l'outil est la correction de ces dix sites.

`must_frozen_seed` n'apparaît pas dans la liste : il ne frappe pas, il *consomme* une preuve venue de `classify_motion` et la rend telle quelle ou la dégrade (section 17.5.4).

17.4.2 Les verdicts polarisés

Extrait 17.15 – `src/rules/api/query.rs`, l. 111–135 — `MustResult` et `May`

```
111 /// A three-valued MUST verdict. `All` carries the certified token (the only way
112 /// to obtain a `Certified` for a single-verdict primitive); `Some`/`None` are
113 /// MAY facts with no path to an `Error`.
114 ///
115 /// (Deviate from the ADR-021 §1 literal `All(T)`: the token lives *inside* `All`
116 /// so `must_*` and the `Vec<Certified<_>>` primitives share one minting story.)
117 #[derive(Debug, Clone, PartialEq)]
118 pub enum MustResult<T> {
119     /// Proven on all paths – carries the minted proof.
120     All(Certified<T>),
121     /// Proven on some but not all paths – a MAY fact (raw payload).
122     Some(T),
123     /// No qualifying evidence at all.
124     None,
125 }
126
127 /// A MAY fact. There is no path from `May<_>` to an `Error`.
128 #[derive(Debug, Clone, Copy, PartialEq, Eq)]
129 pub struct May<T>(pub T);
130
131 impl<T> May<T> {
132     pub fn into_inner(self) -> T {
133         self.0
134     }
135 }
```

Vu d'une règle, `MustResult` est un petit treillis à trois points ordonné par la force de la preuve, `None` $\sqsubseteq$ `Some` $\sqsubseteq$ `All`, dont seul le sommet transporte un jeton. `Some(T)` dit « établi sur certains chemins » : un fait *may*, dont la charge est brute. `May<T>` a un champ public : on en construit librement, puisque ce n'est pas une preuve, et aucune fonction ne transforme un `May` en `Certified`. Une règle typique s'écrit donc :

Extrait 17.16 – Forme usuelle de consommation d'un `MustResult` (pseudo-code)

```

1 match must_primitive(...) {
2   MustResult::All(proof) => Diagnostic::error(NAME, proof, "..."),
3   MustResult::Some(evidence) => Diagnostic::warn(NAME, "..."),
4   MustResult::None => /* rien, ou un autre bras */,
5 }

```

Piège

`MustResult::Some` n'a rien à voir avec `Option::Some` : c'est un fait *may*, pas « une valeur présente ». Un `if let Some(x) = ...` écrit par réflexe sur un `MustResult` ne compile pas, mais un `MustResult::Some(x) => error(...)` ne compile pas non plus, faute de jeton : l'homonymie ne peut donc pas faire émettre une `ERROR` à tort. Elle peut en revanche faire lire un code de travers.

17.4.3 Les classifieurs totaux

La deuxième moitié de la leçon d'ADR-21 concerne $\top$. Au lieu de prédicats booléens qui laissent un trou, la surface expose des *classifieurs* dont $\top$ est une variante que la règle doit traiter :

Extrait 17.17 – `src/rules/api/query.rs`, l. 143–179 — `StabilityVerdict`, classifieur total

```

143 #[derive(Debug, Clone, PartialEq, Eq)]
144 pub enum StabilityVerdict {
145     /// Same reference every render (the only Error-safe "stable" answer).
146     Stable,
147     /// Changes only at the named setter events (may bound). Empty = widened.
148     Versioned(BTreeSet<(ComponentId, HookLabel)>),
149     /// Changes on every render (must bound) – **kind-agnostic motion**, not
150     /// necessarily a fresh allocation (ADR-017): a numeric slot converged to a
151     /// non-point interval lands here alongside an object literal. A consumer
152     /// that needs "defeats Object.is`" because the identity is new" wants
153     /// [`crate::domains::StateValue::is_unstable_reference_only`] instead.
154     PerRender,
155     ///  $\top$  – no bound in either direction. Folded to the may side.
156     Unknown,
157 }
158
159 impl StabilityVerdict {
160     /// Total projection of the six-variant [`Stability`] lattice onto the
161     /// rule-facing verdict. `Bottom` ( $\perp$ , unreachable/uninitialised) is **not**
162     /// provably stable, so it folds to `Unknown` (may side) – matching the
163     /// existing `is_stable` semantics (true only for `Stable`).
164     pub fn of(stability: Stability) -> Self {
165         match stability {
166             Stability::Stable => StabilityVerdict::Stable,
167             Stability::Versioned(slots) => StabilityVerdict::Versioned(slots),
168             Stability::VersionedTop => StabilityVerdict::Versioned(BTreeSet::new()),
169             Stability::PerRender => StabilityVerdict::PerRender,
170             Stability::Bottom | Stability::Unknown => StabilityVerdict::Unknown,
171         }
172     }
173
174     /// `true` iff the value is provably `Stable`. The sound gate: everything
175     /// else ( $\top$ , Versioned, PerRender) *may* change.
176     pub fn is_stable(&self) -> bool {
177         matches!(self, StabilityVerdict::Stable)
178     }
179 }

```


179 }

`StabilityVerdict::of` est une projection totale du treillis à six éléments `Stability` (définition 10.3) sur quatre verdicts : `Bottom` et `Unknown` fusionnent en `Unknown`, `VersionedTop` devient `Versioned` d'ensemble vide. `Bottom` ($\perp$, inatteignable ou non initialisé) n'est pas « prouvé stable » : il part du côté `may`. `is_stable` est la seule porte sûre, et la sonde « peut changer » en est la négation typée :

Extrait 17.18 – `src/rules/api/query.rs`, l. 221–227 — la sonde $\top$ -sûre

```
221 /// The sole  $\top$ -safe stability-reachability probe (ADR-021 §3): `true` unless the
222 /// value is provably `Stable` (`T`/`Versioned`/`PerRender` → `true`). Replaces
223 /// the withdrawn StateValue::is_unstable`, whose PerRender`-only test let a
224 /// T`/`Versioned` value read as "not changing" – the shipped false negative.
225 pub fn may_change_of(val: &StateValue) -> May<bool> {
226     May(!stability_verdict_of(val).is_stable())
227 }
```

Deux autres classifieurs suivent le même modèle. `ReturnsVerdict` (`src/rules/api/query.rs`, l. 189–219) pose une question d'*identité* et non de stabilité (un sélecteur de store qui rend une référence neuve à chaque appel) : `Stable`, `FreshReference` ou `Unknown` ; son doc-comment précise « a classifier, not a must-primitive: it mints nothing ». `CleanupVerdict` (`src/rules/api/query.rs`, l. 1136–1178) dit ce qu'un corps d'effet retourne : `Present`, `Absent` ou `Unknown`, où `Unknown` se replie du côté « il y a peut-être un cleanup » ; seul `Absent` est une affirmation exploitable, et `missing-cleanup` (WARNING) ne tire que sur lui.

Piège

`may_change` et `may_change_of` n'ont, au commit e67b10a, aucun appelant hors de `query.rs` (recherche dans `src/` et `tests/`). La porte effectivement utilisée par `infinite-loop` est `all_deps_provably_stable` (section 17.5.3), construite directement sur `stability_verdict_of(..).is_stable()`. La « seule sonde $\top$ -sûre » d'ADR-21 existe donc, mais ce n'est pas elle qui a fermé le FN.

17.4.4 Le graphe des types

La figure 17.2 rassemble le dispositif. Chaque flèche rouge est un chemin qu'une règle pourrait vouloir emprunter par erreur, et que le compilateur refuse.

Exemple 17.1: Essayez de forger une Error

Écrivons, dans une caisse extérieure qui dépend de la bibliothèque `reactant`, les quatre tentatives qu'un auteur pressé pourrait faire (`/tmp/ch17probe/src/lib.rs`) :

```
1 use reactant::rules::{Certified, Diagnostic, May, Provenance, Severity};
2
3 pub fn forge_token() -> Certified<()> {
4     Certified::mint(), Provenance::default()
5 }
6 pub fn forge_ctor() -> Diagnostic {
7     Diagnostic::new("x", "m").with_severity(Severity::Error)
8 }
9 pub fn forge_field(mut d: Diagnostic) -> Diagnostic {
10     d.severity = Severity::Error;
11     d
12 }
13 pub fn forge_may() -> Diagnostic {
```

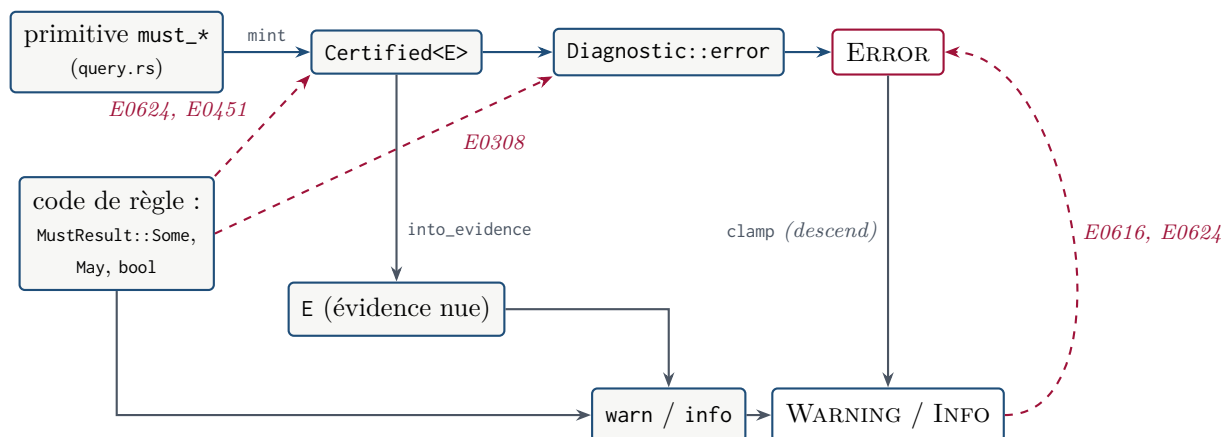


FIGURE 17.2 – Le graphe des types de la sévérité. En bleu, le seul chemin vers une `ERROR` : une primitive `must` frappe un jeton, que `Diagnostic::error` consomme. En gris, les chemins libres : un fait `may` ou une évidence dégradée ne mène qu'à `warn` ou `info`, et `clamp` ne fait que descendre. En rouge, les flèches interdites, avec le code d'erreur que `rustc` rend réellement (exemple 17.1) : passer un fait `may` à `error` (E0308, types incompatibles), frapper un jeton hors de `query.rs` (E0624, fonction privée ; E0451 pour un littéral de structure), remonter une sévérité en écrivant le champ (E0616, champ privé) ou par `with_severity` (E0624).

```
14 Diagnostic::error("x", May(()), "m")
15 }
```

`rustc` refuse les quatre (sortie élaguée : les en-têtes d'erreur, puis le début du détail de la dernière ; la ligne 21 est celle du fichier de sonde, dont chaque fonction est précédée d'un commentaire) :

```
error[E0624]: associated function `mint` is private
error[E0624]: associated function `new` is private
error[E0624]: method `with_severity` is private
error[E0616]: field `severity` of struct `Diagnostic` is private
error[E0308]: mismatched types
--> src/lib.rs:21:28
|
21 | Diagnostic::error("x", May(()), "m")
| ----- ^^^^^^^ expected `Certified<_>`, found `May<()>`
```

et une cinquième sonde, le littéral `Certified` `evidence: ()`, `provenance: Provenance::default()`, donne « `error[E0451]: fields 'evidence' and 'provenance' of struct 'Certified' are private` ». Depuis une règle de `src/rules/impls/`, sœur des modules `api::diagnostic` et `api::query`, les messages sont les mêmes : pour la visibilité privée, un module frère et une caisse extérieure sont logés à la même enseigne. ADR-21 cite les mêmes codes E0616 et E0624 pour ses propres sondes.

Piège

Le typestate empêche de *forger* une preuve, pas de la *transférer*. `Certified` dérive `Clone`, et tous les champs de `Diagnostic` sauf la sévérité sont publics : une règle pourrait cloner une preuve réelle sur le fait `A` pour construire une `ERROR` sur le fait `B`, ou changer `d.rule`, `d.message` et `d.range` après `Diagnostic::error`. ADR-21 affirme que « `certified evidence for finding A cannot build an Error about B` » ; c'est vrai du type de l'évidence (un `Certified<ConditionalHookCall>` ne s'obtient que pour un hook conditionnel), mais aucun type ne lie la preuve au *message*. Cette dernière garantie repose sur la revue, et le noyau de

confiance reste la polarité des primitives.

17.5 Les primitives must

Le `typestate` déplace la question « cette `ERROR` est-elle justifiée ? » vers une autre : « cette primitive ne frappe-t-elle que sur un fait établi sur tous les chemins ? ». Le fichier `query.rs` énonce son contrat en tête de la section des primitives :

Extrait 17.19 – `src/rules/api/query.rs`, l. 417–421 — le contrat normatif des primitives

```
417 // -- Query primitives (ADR-021 §3) -----
418 //
419 // Contract (normative): every primitive returns a polarity-typed verdict;  $\top$ 
420 // folds to may inside; only must-primitives mint `Certified`. Adding one means
421 // following the contract – no new ADR.
```

Trois clauses : un verdict typé par sa polarité, $\top$ replié du côté *may* à l'intérieur de la primitive, et la frappe réservée aux primitives `must`. La dernière phrase compte : ajouter une primitive qui respecte le contrat ne demande pas de nouvel ADR. Cette section présente les primitives de la plus simple à la plus subtile ; chacune illustre une manière différente d'obtenir un « pour tout » correct.

17.5.1 Dominance sur les sorties : `hook_is_conditional`

La question de `conditional-hook` se formule par dominance (définition 3.9) : un `hook` est appelé à chaque rendu si et seulement si son bloc domine chacune des sorties du CFG de rendu. La relation « domine toutes les sorties » a un propriétaire unique, `ExitDominance` :

Extrait 17.20 – `src/rules/api/query.rs`, l. 652–683 — `ExitDominance` : les sorties atteignables, et leur négation

```
652 impl ExitDominance {
653     pub fn of(cfg: &CFG) -> Self {
654         // Reachable exits only. A `Return` no path can arrive at is not an exit:
655         // lowering seals a fall-through tail as `Return(undefine)`, and an
656         // `if`/`else` whose both branches returned leaves that tail orphaned –
657         // counting it would make every hook before the branch fail to dominate
658         // "all exits" and report as conditional at the Error tier.
659         let reachable = cfg.reachable_blocks();
660         let exits = cfg
661             .blocks
662             .values()
663             .filter(|b| matches!(b.term, Terminator::Return(_)))
664             .map(|b| b.id)
665             .filter(|id| reachable.contains(id))
666             .collect();
667         ExitDominance {
668             domtree: DominatorTree::new(cfg),
669             exits,
670         }
671     }
672
673     /// `true` when `block` may be skipped on some render path – the rule-facing
674     /// negation of [Self::certify], and deliberately not its MustResult::None
675     /// case: a CFG with no reachable exit proves nothing in either direction, so
676     /// nothing is skippable there.
677     pub fn may_be_skipped(&self, block: BlockId) -> bool {
```

```

678     !self.exits.is_empty()
679     && self
680     .exits
681     .iter()
682     .any(|&exit| !self.domtree.dominates(block, exit))
683 }

```

Le point décisif est le filtre `reachable.contains(id)`. Le lowering scelle toujours la queue d'un corps par un `Return(undefined)` (chapitre 7) ; quand les deux branches d'un `if/else` retournent, cette queue est orpheline. La compter comme une sortie ferait échouer, pour tout hook placé avant la branche, le test « domine toutes les sorties », et la règle tirerait au niveau `ERROR` sur un hook parfaitement inconditionnel. Le commentaire le dit en toutes lettres.

`may_be_skipped` est la négation *côté règle* de `certify`, et pas la simple lecture de son cas `None` : un CFG sans sortie atteignable ne prouve rien dans aucun sens, donc rien n'y est « sautable » (`may_be_skipped` rend faux) et rien n'y est certifié dominant (`certify` rend `None`). Formellement, avec X l'ensemble des sorties atteignables et b le bloc du hook :

$$\begin{aligned} \text{may_be_skipped}(b) &\iff X \neq \emptyset \wedge \exists x \in X, \neg \text{dom}(b, x), \\ \text{certify}(b) = \text{All} &\iff X \neq \emptyset \wedge \forall x \in X, \text{dom}(b, x). \end{aligned}$$

La primitive de la règle applique cette négation à chaque appel de hook et frappe une preuve par hook conditionnel :

Extrait 17.21 – `src/rules/api/query.rs`, l. 472–511 — `hook_is_conditional`, la négation $\forall$ -sorties par hook

```

472    /// Every conditionally-called hook in the component: a hook whose block does
473    /// not dominate every render exit. Packages the whole dominance  $\forall$ -exits
474    /// check so a rule cannot under-quantify (ADR-021 §3).
475    pub fn hook_is_conditional(&self) -> Vec<Certified<ConditionalHookCall>> {
476        let cfg = &self.comp.render_cfg;
477        // One owner for "what the exits are": this used to keep its own copy of
478        // the enumeration and its own `DominatorTree`, which could disagree with
479        // `ExitDominance` about reachability.
480        let dominance = ExitDominance::of(cfg);
481        cfg_hook_calls(self.comp)
482            .filter(|call| !matches!(call.kind, HookKind::Handler))
483            .filter(|call| dominance.may_be_skipped(call.block_id))
484            .map(|call| {
485                let mut notes = Vec::new();
486                if let Some((), guard_span) = guard_site(cfg, call.block_id) {
487                    let step = Step::Branch {
488                        desc: "a condition evaluated here, so some render paths skip the hook"
489                            .to_string(),
490                    };
491                    notes.push(Note {
492                        message: step.render(&super::witness::fallback_name),
493                        step,
494                        hook_label: None,
495                        range: guard_span,
496                    });
497                }
498                Certified::mint(
499                    ConditionalHookCall {
500                        label: call.label,
501                        span: call.span,
502                    },
503                    Provenance {
504                        range: call.span,

```

```

505             hook_label: Some(call.label),
506             notes,
507         },
508     )
509 })
510 .collect()
511 }

```

Trois remarques.

- Les handlers (`HookKind::Handler`, les props `onX` que le lowering range parmi les hooks) sont filtrés : ce ne sont pas des hooks au sens de React.
- Le fait prouvé est un *existantiel* (« il existe un chemin qui saute le hook »), et c’est pourtant un fait *must* : il affirme que le hook *est* sauté sur un chemin réel du CFG, pas qu’il *pourrait* l’être. L’ordre des hooks est alors cassé pour ce rendu-là, quel que soit le coût. D’où l’`ERROR`.
- C’est la seule primitive dont la preuve transporte déjà un témoin : la note `Step::Branch` est construite ici, avec la position de la garde que calcule `guard_site` (`src/rules/api/query.rs`, l. 1092–1123) — parmi les dominateurs stricts du bloc terminés par un `Branch` dont un successeur n’atteint pas le bloc, le plus profond. Toutes les autres primitives laissent la règle construire sa chaîne ; `Provenance::with_notes` n’a d’ailleurs aucun appelant au commit `e67b10a`.

L’étude de la règle elle-même, avec le schéma des deux situations typiques et les FN connus de l’extraction des hooks, est au section 20.3. La dominance sert aussi à `setter-in-render`, dont l’`ERROR` exige que l’appel domine toutes les sorties (chapitre 12) ; la primitive à usage unique `must_dominates_all_exits` (`src/rules/api/query.rs`, l. 708–712) construit un `ExitDominance` et interroge un bloc.

17.5.2 Une analyse `must` avant : `must_setter_on_all_paths`

`derived-state` veut savoir si un effet appelle un setter *sur tous les chemins* de son corps (« `this effect always sets...` »). La dominance d’un seul bloc ne suffit pas : l’appel peut se trouver dans plusieurs blocs, sur des branches différentes qui se rejoignent. La primitive porte donc sa propre analyse de flot de données. Elle commence par collecter les sites d’appel :

Extrait 17.22 – `src/rules/api/query.rs`, l. 527–552 — collecte des sites d’appel du setter

```

527 pub fn must_setter_on_all_paths(
528     cfg: &CFG,
529     setter_vars: &HashSet<Var>,
530     restrict_to: Option<&HashSet<BlockId>>,
531 ) -> MustResult<SetterCall> {
532     let mut call_sites: Vec<(BlockId, Var, Expr, Option<SourceRange>>) = vec![];
533     for (bid, block) in &cfg.blocks {
534         if restrict_to.is_some_and(|set| !set.contains(bid)) {
535             continue;
536         }
537         for stmt in &block.stmts {
538             if let Stmt::ExprStmt(expr, span) = stmt
539                 && let Some((var, arg)) = try_extract_setter_call(expr, setter_vars)
540             {
541                 call_sites.push((*bid, var, arg.clone(), *span));
542             }
543         }
544         if let Terminator::Return(expr) = &block.term
545             && let Some((var, arg)) = try_extract_setter_call(expr, setter_vars)

```

```

546     {
547         call_sites.push((*bid, var, arg.clone(), None));
548     }
549 }
550 if call_sites.is_empty() {
551     return MustResult::None;
552 }

```

Les préconditions suivent (src/rules/api/query.rs, l. 553–575) : au moins un site ; tous les sites visent le *même* setter ; tous les arguments sont sans appel, au sens de `arg_is_call_free`, qui suit les liaisons locales (une variable n’est « sans appel » que si *toutes* ses liaisons le sont). Si l’une échoue, le verdict est `None`. Vient ensuite le calcul proprement dit :

Extrait 17.23 – src/rules/api/query.rs, l. 576–621 — l’analyse must avant et sa conclusion

```

576 // must_in[B] =  $\wedge$  must_out[preds]; must_out[B] = must_in[B]  $\vee$  called_in[B].
577 let called_in: HashMap<BlockId, bool> = cfg
578     .blocks
579     .keys()
580     .map(|&bid| (bid, call_sites.iter().any(|(b, _, _, _)| b == &bid)))
581     .collect();
582 let mut must_out: HashMap<BlockId, bool> = cfg.blocks.keys().map(|&bid| (bid, true)).collect();
583 match must_out.get_mut(&cfg.entry) {
584     Some(e) => *e = called_in[&cfg.entry],
585     None => return MustResult::None,
586 }
587 let mut changed = true;
588 while changed {
589     changed = false;
590     for &bid in cfg.blocks.keys() {
591         if bid == cfg.entry {
592             continue;
593         }
594         let preds = cfg.predecessors(bid);
595         if preds.is_empty() {
596             continue;
597         }
598         let must_in = preds.iter().all(|&p| *must_out.get(&p).unwrap_or(&false));
599         let new_val = must_in || called_in[&bid];
600         if must_out[&bid] != new_val {
601             must_out.insert(bid, new_val);
602             changed = true;
603         }
604     }
605 }
606 let exit_blocks: Vec<_> = cfg
607     .blocks
608     .values()
609     .filter(|b| matches!(b.term, Terminator::Return(_) | Terminator::Unreachable))
610     .collect();
611 if exit_blocks.is_empty() {
612     return MustResult::None;
613 }
614 if exit_blocks
615     .iter()
616     .all(|b| *must_out.get(&b.id).unwrap_or(&false))
617 {
618     MustResult::All(Certified::mint(evidence, Provenance::at(target_span, None)))
619 } else {
620     MustResult::Some(evidence)
621 }

```

Les équations, écrites en commentaire à la ligne 576, sont celles d’une analyse must avant sur le treillis booléen, dont le meet est la conjonction :

$$out(entry) = called(entry), \quad in(B) = \bigwedge_{P \in pred(B)} out(P), \quad out(B) = in(B) \vee called(B).$$

L’itération part de true partout (sauf l’entrée) et ne peut que descendre : elle calcule le *plus grand* point fixe. C’est la bonne solution d’une analyse must, et c’est l’inverse de ce que l’on fait pour une analyse may (définition 3.5) : partir de false ferait tomber l’information à tort dans toute boucle, parce qu’un arc arrière transmettrait un false initial que rien ne corrigerait.

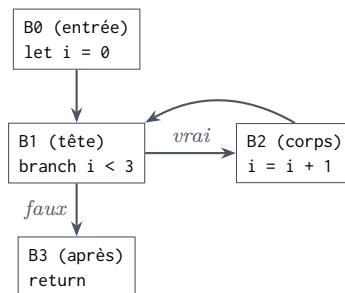


FIGURE 17.3 – Un CFG idéalisé de corps d’effet avec une boucle. L’appel `setB(a * 2)` est placé soit dans B0 (avant la boucle), soit dans B2 (dans la boucle), soit dans B3 (après). Le lowering réel produit davantage de blocs, mais la même forme.

appel dans	passe	$out(B0)$	$out(B1)$	$out(B2)$	$out(B3)$	verdict
B0	init	vrai	vrai	vrai	vrai	
	1	vrai	vrai	vrai	vrai	stable : All
B0, départ à false (à ne pas faire)	init	vrai	faux	faux	faux	
	1	vrai	faux	faux	faux	stable : Some, à tort
B2	init	faux	vrai	vrai	vrai	
	1	faux	faux	vrai	faux	
	2	faux	faux	vrai	faux	stable : Some
B3	init	faux	vrai	vrai	vrai	
	1	faux	faux	faux	vrai	
	2	faux	faux	faux	vrai	stable : All

TABLE 17.4 – Itérations de `must_out` sur le CFG de la figure 17.3, dans l’ordre de `cfg.blocks.keys()` (B1, B2, B3 ; l’entrée n’est pas recalculée). En gras, les valeurs qui changent. La seule sortie est B3 ; le verdict est All si $out(B3)$ vaut vrai. La deuxième ligne montre ce que donnerait un plus petit point fixe : dès la première passe, $out(B1) = out(B0) \wedge out(B2) = vrai \wedge faux$, et la valeur fausse venue de l’arc arrière ne se corrige jamais.

Le tableau 17.4 déroule l’algorithme. Un appel dans le corps de boucle (B2) n’est pas sur tous les chemins, puisque la boucle peut ne pas s’exécuter : Some. Un appel avant ou après la boucle l’est : All. L’analyseur confirme ces trois verdicts sur trois composants de cette forme (`/tmp/ch17/mustloop2.tsx`, où `a` est un état et l’appel est `setB(a * 2)`) : `derived-state`, qui n’agit que sur `MustResult::All`, signale `Before` et `After` et donne l’assurance `verified` à `Inside` :

```

After (4 hooks) mustloop2.tsx
  warn derived-state [hook:2] (line 27:2) this effect always sets `setB` to a call-free expression of `a` replace
    ↳ with `useMemo` or compute during render
Before (4 hooks) mustloop2.tsx

```

```
warn derived-state [hook:2] (line 6:2) this effect always sets `setB` to a call-free expression of `a` replace
  ↳ with `useMemo` or compute during render
Inside (4 hooks) mustloop2.tsx ✓
verified derived-state no effect merely mirrors other state
```

(Élagué et filtré sur `derived-state`, avec `-ignore-rule infinite-loop -ignore-rule widening-info` : parce que `a` est un état rouvert par un `setA` du gestionnaire de clic, sa valeur est widenée à l'itération du point fixe, ce qui rend aussi `b` instable et fait porter aux trois composants un `WARNING infinite-loop` et une `INFO widening-info` — un fait réel mais sans rapport avec la primitive `must` ici étudiée. `Inside` n'est donc pas un composant sans diagnostic, seulement sans diagnostic `derived-state`.)

Le fait est certain (A11), et la règle n'en tire pourtant qu'un `WARNING` : le coût d'un état dérivé (un rendu de plus) n'est pas un défaut certain. C'est l'autre moitié de la définition des niveaux, « `a` certain fact whose cost is not ».

Piège

`must_setter_on_all_paths` et `ExitDominance` n'ont pas la même notion de « sortie ». La première prend tous les blocs `Return` et `Unreachable`, sans filtre d'atteignabilité ; la seconde, seulement les `Return` atteignables. Les deux sont *sound* pour leur usage : dans l'analyse `must`, un bloc sans prédécesseur est sauté (`preds.is_empty()`) et garde sa valeur initiale vraie, donc une sortie orpheline n'invalidé pas le « pour tout ». Mais un contributeur qui voudrait unifier les deux doit le savoir. Autre détail : un appel trouvé dans le terminateur `Return` d'une flèche concise (`() => setX(v)`) est enregistré sans position (`None`, ligne 547), faute de position sur `Terminator::Return (#140)`.

17.5.3 Le bon quantificateur : `all_deps_provably_stable`

Revenons au FN de la section 17.1. La porte qui décide si le tableau de `deps` d'un effet l'empêche de boucler vit dans `src/rules/helpers/mod.rs` :

Extrait 17.24 – `src/rules/helpers/mod.rs`, l. 211–235 — la porte $\forall$ -stable

```
211 /// `true` when every dep in `deps` is provably `Stable` in the render-exit
212 /// env – the only situation where a deps array genuinely gates an effect for
213 /// good: React re-runs an effect when any dep changed (OR semantics), so a
214 /// single dep that may change ( $\top$ /Versioned`/`PerRender`) keeps the effect
215 /// live no matter how stable its neighbours are. Empty `deps` returns `true`
216 /// ([] is mount-only – gated by definition; infinite-loop` handles it
217 /// upstream anyway).
218 ///
219 /// ADR-021 §5 + quantifier fix: the first cut of the  $\top$ -fix
220 /// (all_deps_may_change` still used the wrong quantifier – it skipped when
221 /// *one* dep was provably stable, silently dropping the mixed-deps case
222 /// ([stableConst, topProp`), the same FN family one stable dep away. The
223 /// sound gate quantifies  $\forall$ -stable, keyed on [query::stability_verdict_of`
224 /// ( $\top$  is a returned variant folded to the may side).
225 pub(in crate::rules) fn all_deps_provably_stable(
226     deps: &[Expr],
227     result: &AnalysisResult<StateValue>,
228 ) -> bool {
229     let exit_env = result.exit_env();
230     let mut eval = result.evaluator();
231     deps.iter().all(|dep| {
232         let val = eval.at(&exit_env, dep);
233         query::stability_verdict_of(&val).is_stable()
234     })
235 }
```


Rappel React : la sémantique « ou » des deps

Après chaque rendu, React compare chaque dep à sa valeur précédente par `Object.is`, et relance l'effet si *au moins une* diffère. Un tableau `[a, b]` protège donc l'effet seulement si *a et b* restent identiques ; `[]` ne le relance jamais après le montage ; l'absence de tableau le relance à chaque rendu.

Proposition 17.1: Soundness de la porte

Soit un effet de deps $d_1, \dots, d_k$ évaluées dans l'environnement de sortie du rendu. Si `all_deps_provably_stable` rend faux, la règle examine l'effet ; si elle rend vrai, chaque d_i a pour valeur abstraite `Stable`, dont la concrétisation ne contient que des valeurs identiques d'un rendu à l'autre, donc aucune dep ne change et React ne relance pas l'effet. Sauter l'effet dans ce cas ne perd aucun comportement concret : la porte ne crée pas de FN.

Le quantificateur est la clef. Sauter dès qu'une dep est stable (la première correction) rendrait l'implication fausse pour `[label, data]`. Sauter tant qu'aucune dep n'est « instable » (le code d'origine) la rendrait fausse pour toute dep à `⊤` ou `Versioned`. Le seul consommateur, `src/rules/impls/infinite_loop.rs`, l. 110, exige en plus une arité exacte (`dep_exprs.arity.exact().is_some()`) : un tableau contenant un spread aplati pourrait cacher des éléments mobiles. Un deps vide rend vrai, ce que la règle traite en amont.

17.5.4 Frapper au point de connaissance : `classify_motion` puis `must_frozen_seed`

La règle `frozen-initial-state` signale un état initialisé depuis une prop qui change : `useState` ne lit son initialiseur qu'au premier rendu, et l'état reste figé sur la première valeur.

Extrait 17.25 – Un état qui gèle une prop mobile (`/tmp/ch17/frozen.tsx`)

```
1 import { useState } from "react";
2
3 function Parent() {
4   const [user, setUser] = useState({ name: "a" });
5   return <Child user={user} onRename={() => setUser({ name: "b" })} />;
6 }
7
8 function Child({ user }) {
9   const [local, setLocal] = useState(user);
10  return <button onClick={() => setLocal({ name: "c" })}>{local.name}</button>;
11 }
```

Pour une `ERROR`, il faut prouver que la prop `user` bouge vraiment. Sa valeur abstraite dans `Child` est `Versioned(Parent, 0)` : elle change quand le slot 0 de `Parent` est écrit (définition 10.3). Reste à vérifier que ce slot est effectivement écrit quelque part. C'est ce que fait `classify_motion` :

Extrait 17.26 – `src/rules/api/query.rs`, l. 839–890 — `classify_motion` frappe là où il constate

```
839 pub fn classify_motion(val: &StateValue, result: &ProgramAnalysisResult) -> Motion {
840   // Version labels live on the reference slot only (`to_stability` erases
841   // them when another kind slot is  $\top$ ) – check it first, like
842   // `recompute_memo` does.
843   if let Stability::Versioned(labels) = &val.reference {
844     let mut unverifiable = false;
845     for (owner, slot) in labels {
846       let Some(owner_result) = result.components.get(owner) else {
847         unverifiable = true;
848         continue;
849       }
850     }
851   }
```

```

849     };
850     let (writable, write_span) = slot_write_evidence(owner_result, *slot);
851     if writable {
852         let owner_states = crate::rules::state_val_labels(&owner_result.render_cfg);
853         let display = format!(
854             "state {} of `{}`",
855             crate::rules::state_slot_name(*slot, &owner_states),
856             result.display_name(*owner)
857         );
858         return Motion::Proven(Certified::mint(
859             MovingFeeder {
860                 owner: *owner,
861                 slot: *slot,
862                 display,
863                 write_span,
864             },
865             // The write that moves the feeding slot *is* the proof, and
866             // this is where it was found. The consuming rule anchors
867             // the finding at the seed site instead – a better place to
868             // point a user – but the token now says where the proof
869             // lives without anyone reading back into the evidence.
870             Provenance::at(write_span, None),
871         ));
872     }
873 }
874 return if unverifiable {
875     Motion::Unproven
876 } else {
877     // Every feeding slot is owned by an analyzed component and its
878     // setter is never referenced there: the prop provably never
879     // changes (React state moves only through its setter).
880     Motion::Still
881 };
882 }
883 if val.reference == Stability::VersionedTop {
884     return Motion::Unproven;
885 }
886 match val.to_stability() {
887     Stability::Bottom | Stability::Stable => Motion::Still,
888     _ => Motion::Unproven,
889 }
890 }

```

Si un setter du slot nourricier est réellement référencé dans son propriétaire (`slot_write_evidence`, `src/rules/api/query.rs`, l. 810–834, qui s’appuie sur `may_written_slots`, un calcul syntaxique), la primitive frappe un `Certified<MovingFeeder>` *ici*, avec pour provenance la position de l’écriture. Si tous les propriétaires sont analysés et qu’aucun setter n’est référencé, la prop ne bouge jamais : `Motion::Still`, et la règle se tait. Si un propriétaire n’a pas été analysé, rien n’est prouvé : `Unproven`.

La preuve n’est pas encore une `ERROR`. Quatre portes peuvent la dégrader, et `must_frozen_seed` est leur seul juge :

Extrait 17.27 – `src/rules/api/query.rs`, l. 904–916 — `must_frozen_seed` rend ou dégrade la preuve

```

904 pub fn must_frozen_seed(
905     feeder: Certified<MovingFeeder>,
906     escaped: bool,
907     all_seed_named: bool,
908     locally_written: bool,
909     mount: MountCoupling,

```

composante	val.reference	condition	verdict
Versioned(labels)		un (owner, slot) a un propriétaire analysé et un setter référencé	Proven (premier label)
Versioned(labels)		aucun label écrivable, au moins un propriétaire absent	Unproven
Versioned(labels)		tous les propriétaires analysés, aucun setter référencé	Still
VersionedTop		—	Unproven
autre		val.to_stability() vaut Bottom ou Stable	Still
autre		sinon (PerRender, Unknown, ...)	Unproven

TABLE 17.5 – La table de décision de `classify_motion`.

```

910 ) -> MustResult<MovingFeeder> {
911   if !escaped && !all_seed_named && locally_written && mount == MountCoupling::Free {
912     MustResult::All(feeder)
913   } else {
914     MustResult::Some(feeder.into_evidence())
915   }
916 }

```

Le setter ne doit pas s'échapper; les props de seed ne doivent pas toutes porter un nom qui annonce un seed (`initial*`, `default*`); le slot doit être écrit localement; et le couplage de montage doit être `Free` (section 17.9). Toute porte qui échoue rend `MustResult::Some(feeder.into_evidence())` : le fait survit comme fait may, son éligibilité à l'ERROR disparaît. L'effet se voit en ajoutant une key construite depuis la prop, qui fait remonter l'enfant quand elle change (`/tmp/ch17/frozenkey.tsx`, `<Child key=user .../>`) :

```

Child (2 hooks) frozen.tsx
  error frozen-initial-state [hook:0] var:user (line 9:8) state `local` is seeded from `user`, which is fed by
    ↳ state `user` of `Parent` and changes. `useState` reads its initializer on the first render only and nothing
    ↳ here re-syncs it, so `local` stays frozen at the first `user` value
    ↳ `user` is read here [hook:0] (line 9:8)
    ↳ state `local` reads its initializer on the first render only, so later renders ignore it [hook:0] (line 9:8)
    ↳ state state `user` of `Parent` is written here
Child (2 hooks) frozenkey.tsx
  info frozen-initial-state [hook:0] var:user (line 9:8) state `local` is seeded from `user`, ...

```

(Deux sorties concaténées, la seconde élaguée.) Avec la key, le couplage vaut `Reseeds` et la règle rend une `INFO` : dégradation, jamais suppression. Deux défauts de surface sont visibles dans la dernière note : « `state state` » (le display du `MovingFeeder` est pré-qualifié « `state 'user' of 'Parent'` » et le gabarit de `Step::Write` préfixe à nouveau « `state` ») et l'absence de position (l'écriture est dans une flèche concise, #140).

Décision : ADR-021, durcissement — pas de distributeur de jetons

Constat. Dans la première version, `must_frozen_seed(bool, ...)` et `must_effect_cycle(bool, bool)` ne faisaient aucune analyse : tout appelant qui passait `true` obtenait un `Certified`. L'ADR les appelle des « token vending machines ». **Décision.** La frappe se fait « au point de connaissance » (la discipline d'ADR-19) : là où le fait est *constaté*. `classify_motion` vit dans `query.rs` et frappe en constatant l'écriture; `must_frozen_seed` consomme la preuve et ne peut que la dégrader; `must_effect_cycle` re-dérive ses deux faits des arêtes brutes. **Conséquence.** Des portes mal réglées peuvent sur-étiqueter un nourricier *réellement* prouvé, jamais en fabriquer un. C'est la surface de confiance résiduelle que l'ADR nomme.

Piège

`Motion::Still` ne dégrade pas : il *supprime* le finding. C'est une preuve d'immobilité, fondée sur la syntaxe (`may_written_slots`, qu'ADR-20, item 3, interdit de remplacer par un bit observé pendant le point fixe, qui pourrait sous-compter) et sur le fait que l'état React ne bouge que par son setter. Autre asymétrie à connaître : `StabilityVerdict::of` envoie `Bottom` du côté `may`, mais `classify_motion` le compte comme `Still`, via `to_stability()`. Pour un lecteur soucieux de soundness, c'est l'endroit le plus délicat de la primitive.

17.5.5 Re-dériver plutôt que croire : `must_effect_cycle`

Le graphe de churn (définition 15.1) marque chacun de ses cycles de deux booléens, `all_must` et `cross_component`. La primitive qui certifie la boucle infinie *ne les lit pas* : elle recompte la force de chaque arête et l'ensemble des composants impliqués (propriétaires des slots *et* porteurs d'effets), puis ne frappe que si toutes les arêtes sont `Must` et qu'un seul composant est en jeu (`src/rules/api/query.rs`, l. 1002–1031, étudié au section 15.9). Sa provenance est la première arête du cycle, l'écriture qui relance l'effet porteur. Elle est `pub(in crate::rules)` : seule la règle native l'appelle. Sur deux effets qui se renvoient des objets neufs :

```
PingPong (4 hooks) pingpong.tsx
error infinite-loop [hook:2] (line 6:2) these effects form a state-update cycle (`a` → `b` → `a`) where each
  ↳ step stores a fresh reference that re-runs the next effect: infinite render loop
  ↳ a fresh value is written to state `b` here [hook:2] (line 6:20)
  ↳ cycle continues: this effect freshly stores state `a` [hook:3] (line 7:20)
```

(Extrait : la même sortie contient une seconde `ERROR` ancrée sur l'autre effet, et deux `WARNING derived-state`.) Un cycle qui traverse deux composants reste un `WARNING` : une dep de prop n'est jamais le slot exact.

17.5.6 La preuve de toute la conclusion : `must_stale_capture`

`stale-closure` signale un callback de longue durée (un `setInterval`, un `addEventListener`) qui capture un état au montage et le réécrit à chaque tir depuis cette valeur figée. L'affirmation « l'état ne peut plus avancer » est une conjonction, et la version antérieure à #142 certifiait sur l'un de ses conjoints. La primitive actuelle les exige tous :

Extrait 17.28 – `src/rules/api/query.rs`, l. 942–959 — `must_stale_capture` : les premières portes

```
942 pub fn must_stale_capture(
943     reg: &crate::engine::registrations::Registration,
944     deps: &crate::ir::hooks::DepsArg,
945     effect_body: &CFG,
946     cb_body: &CFG,
947     slot_setters: &HashSet<Var>,
948 ) -> MustResult<StaleCapture> {
949     use crate::engine::registrations::{Firing, Timing};
950     let mount_only = matches!(deps.list(), Some(1) if 1.arity == crate::ir::hooks::Arity::Exact(0));
951     if !mount_only || reg.firing != Firing::Repeating || reg.timing == Timing::Unknown {
952         return MustResult::None;
953     }
954     let Some(reg_block) = reg.block_id else {
955         return MustResult::None;
956     };
957     if !crate::engine::dominance::on_all_paths(effect_body, &HashSet::from([reg_block])) {
958         return MustResult::None;
959     }
```

L'effet est *mount-only* (deps [] exactes, Arity::Exact(0)), donc ne ré-enregistre jamais ; le registrar tire de manière répétée (Firing::Repeating) *et* avec un timing connu — les registrars reconnus par leur seul nom (on, subscribe, addListener) ont Timing::Unknown, parce que l'appelé pourrait exécuter le callback une seule fois, de façon synchrone ; l'enregistrement est sur tous les chemins du corps d'effet ; et, dans la suite de la fonction (src/rules/api/query.rs, l. 961–989), le callback appelle un setter du slot sur tous ses chemins, en ne comptant que les écritures de premier niveau. C'est l'application littérale de la règle du CLAUDE.md : « Error = preuve de *toute* la conclusion, pas d'un seul conjoint ». La règle et ses sorties sont étudiées au chapitre 20.

17.5.7 Les autres primitives

primitive	fait certifié (évidence)	consommateurs
hook_is_conditional	hook sauté sur un chemin (ConditionalHookCall, avec témoin)	conditional-hook
ExitDominance::certify, must_dominates_all_exits must_on_all_paths	bloc qui domine toutes les sorties atteignables (DominatesAllExits) un ensemble de blocs coupe tout chemin entrée-sortie (OnAllPaths); face typée de engine::dominance::on_all_paths	setter-in-render, frontend déclaratif infinite-loop
must_setter_on_all_paths	un seul setter, arguments sans appel, sur tous les chemins (SetterCall)	derived-state, frontend déclaratif
must_init_calls_setter	l'initialiseur d'un useState appelle un setter (InitSetterCall)	lazy-init
must_direct_write	la ligne SlotWriter est d'origine Direct, hors de toute région greffée (DirectWrite, ADR-27)	frontend déclaratif
classify_motion	le slot nourricier d'une prop est écrit (MovingFeeder)	frozen-initial-state
must_stale_capture	callback qui fige le slot qu'il écrit (StaleCapture)	stale-closure
must_effect_cycle	cycle de churn tout-Must, mono-composant (EffectCycleProof)	infinite-loop
must_same_ref_mutation	mutation et appel du setter dans le même conteneur (SameRefMutation)	state-mutation

TABLE 17.6 – Les dix primitives qui frappent (invariant 17.1) et leurs consommateurs. must_frozen_seed consomme la preuve de classify_motion.

Le tableau 17.6 complète la liste. must_direct_write (src/rules/api/query.rs, l. 763–774) mérite une mention : sa certitude est structurelle et par ligne. Les angles morts connus de la relation slot_writers (#52, #54) *retirent des lignes* ; ils ne rendent jamais incertain un Direct présent. Une ligne absente est un finding manqué, compensé par le canal analysis-limit (section 17.7.4), pas une ERROR fausse.

17.6 Les témoins

Un diagnostic sans explication oblige le développeur à refaire l'analyse de tête. reactant-analyzer accompagne chaque finding d'une *chaîne de témoins* : une suite d'étapes typées, chacune ancrée à un endroit du source, qui reconstitue le raisonnement. ADR-19 en fixe la forme.

17.6.1 Un exemple entre deux fichiers

Reprenons lazy-init sur un initialiseur qui appelle une fonction importée (/tmp/ch17/cfg/) :

Extrait 17.29 – Card.tsx et api.ts

```

1 // Card.tsx
2 import { useState } from "react";
3 import { loadUser } from "../api";
4
5 export function Card({ id }: { id: string }) {
6   const [user] = useState(loadUser(id));
7   return <div>{user.id}</div>;
8 }
9
10 // api.ts
11 export function loadUser(id: string) {
12   fetch("/api/user/" + id);
13   return { id };
14 }

```

Card (1 hooks) Card.tsx

warn lazy-init [hook:0] (line 5:8) this useState is initialised by a direct function call. The call runs on
 ↳ every render but the result is only used on mount; wrap as `useState(() => ...)` to defer it
 → `loadUser` resolves to an import from api.ts
 → `fetch` has side effects (subscriptions/requests/timers re-fire on every call) (api.ts:2:2)

La première étape résout le nom `loadUser` ; la seconde pointe l'appel `fetch` *dans un autre fichier*, et l'affichage le nomme (`api.ts:2:2`) au lieu d'écrire (line 2:2), qui désignerait à tort la ligne 2 de `Card.tsx`. Le niveau reste **WARNING** : la chaîne explique, elle ne prouve rien de plus.

17.6.2 Le vocabulaire fermé

Une étape de témoin est une *Note*, qui porte un jugement typé *Step* et son ancre :

Extrait 17.30 – `src/rules/api/witness.rs`, l. 25–41 — *Note*, une étape ancrée

```

25 /// One step of a diagnostic's witness chain (ADR-019).
26 ///
27 /// `message` is the pre-rendered prose ([`Step::render`] is the single
28 /// rendering point – rules never format trace text); `step` is the typed
29 /// judgment JSON consumers read.
30 #[derive(Debug, Clone, PartialEq)]
31 pub struct Note {
32   pub message: String,
33   /// The typed witness step this note carries.
34   pub step: Step,
35   /// Hook label this note points to, if any.
36   pub hook_label: Option<HookLabel>,
37   /// Source location this note points to, if available. Carries the
38   /// [`crate::ir::FileId`] of the file it points into (may differ from the
39   /// component's file after cross-file inlining).
40   pub range: Option<SourceRange>,
41 }

```

Extrait 17.31 – `src/rules/api/witness.rs`, l. 81–130 — *Step*, le vocabulaire des jugements

```

81 /// One typed step of a finding's witness chain.
82 ///
83 /// Each step is attached to a [`Note`] carrying its `(hook_label,
84 /// range)` anchor; the step itself holds only the judgment.
85 #[derive(Debug, Clone, PartialEq)]
86 pub enum Step {
87   /// Value flowed through this binding: `const x = f(props)`.
88   Binding { var: Var },

```

```

89  /// Name resolution: `f` → import / local fn / state setter / unknown.
90  Resolve { name: String, target: ResolveTarget },
91  /// A call and its effect class.
92  Call { callee: String, class: EffectClass },
93  /// State write, with the written value's class.
94  Write { slot: HookLabel, value: ValueClass },
95  /// Read of a reactive value (deps rules: the read site).
96  Read { what: String },
97  /// The site is guarded by this branch condition.
98  Branch { desc: String },
99  /// Escapes into an event handler that re-triggers the cycle.
100 Handler { event: String, slot: HookLabel },
101 /// Churn-graph edge: the cycle continues through this write.
102 /// `from`/`to` are qualified display names (possibly cross-component).
103 CycleEdge { from: String, to: String },
104 /// Fixpoint evidence: the slot's abstract value grew until widening.
105 Widen { slot: HookLabel, iteration: u32 },
106 /// In-place mutation: the object's contents change, its reference doesn't.
107 Mutate { target: String },
108 /// A long-lived callback closes over this value at registration time –
109 /// later firings keep the captured value, not the current one.
110 Capture { what: String },
111 /// `useState`/`useRef` evaluate their initializer on the first render
112 /// only – later renders ignore it.
113 InitOnce { slot: HookLabel },
114 /// A component hands the value down to a child element without using it.
115 /// `from`/`to` are component display names, `props` the props that carry
116 /// it (empty: through a spread, or through `context` when it names one).
117 Forward {
118     from: String,
119     to: String,
120     props: Vec<String>,
121     context: Option<String>,
122 },
123 /// An element re-renders with its parent although none of its inputs
124 /// changed. `renders` is a lower bound on the component renders it costs.
125 Rerender {
126     component: String,
127     renders: usize,
128     list: bool,
129 },
130 }

```

Définition 17.2: Chaîne de témoins

La *chaîne de témoins* d'un diagnostic est la suite `notes: Vec<Note>`. Chaque `Note` associe un jugement `Step` (l'une des quatorze variantes de l'énumération fermée de `src/rules/api/witness.rs`), une ancre (`hook_label`, `range`) dont le `range` porte l'identité du fichier (`FileId`), et un texte `message` produit une fois pour toutes par `Step::render`. L'énumération n'a pas de variante de texte libre : une règle qui a besoin d'une nouvelle sorte de justification étend le vocabulaire.

L'énumération est fermée par choix, comme le dit le doc-comment de tête du module (`src/rules/api/witness.rs`, l. 1–9) : une variante `Text(String)` éroderait le vocabulaire, chaque règle finissant par y écrire sa prose. Le discriminant stable `Step::kind()` est défini aux lignes 134 à 151 du même fichier ; les consommateurs JSON le lisent : `binding`, `resolve`, `call`, `write`, `read`, `branch`, `handler`, `cycle-edge`, `widen`, `mutate`, `capture`, `init-once`, `forward`, `rerender`. ADR-19 en prévoyait neuf ; les cinq dernières ont été ajoutées depuis, chaque fois qu'une règle en avait besoin.

`Step::render(&dyn Fn(HookLabel) -> String)` (`src/rules/api/witness.rs`, l. 157–271) est l’*unique* point où naît la prose des témoins. Son paramètre traduit un label de slot en nom source : une fermeture sur la table `state_slot_name` de la règle (qui choisit le plus petit nom source non temporaire, pour que le résultat ne dépende pas de l’aléa de hachage), ou `fallback_name`, qui rend `state #N`. Les variables passent par `crate::ir::source_name`, qui retire le sel d’alpha-renommage des variables greffées (chapitre 8). Quelques gabarits, verbatim :

variante	texte produit
Resolve Import(path)	<code>{n}</code> resolves to an import from {path}
Call Effectful	<code>{callee}</code> has side effects (subscriptions/requests/timers re-fire on every call)
Write Fresh	a fresh value is written to state {n} here
Branch desc	guarded by {desc}
CycleEdge from: _, to	cycle continues: this effect freshly stores state {to} (from n’est exporté qu’en JSON)
Widen slot, iteration	the abstract value of state {n} kept growing and was widened at iteration {iteration}
InitOnce slot	state {n} reads its initializer on the first render only, so later renders ignore it
Forward props, ..	<code>{from}</code> passes it to <code><{to}></code> {via} without using it itself

TABLE 17.7 – Quelques gabarits de `Step::render` (`src/rules/api/witness.rs`, l. 157–271); {n} est le nom du slot rendu par la fermeture.

Des classes auxiliaires raffinent certains jugements : `ResolveTarget` (`Import`, `LocalFn`, `Setter`, `Unknown`), `EffectClass` (`Setter`, `Effectful`, `PureCheap`, `Unknown`; « the class refines wording/severity, never soundness ») et `ValueClass` (`Fresh`, `SameAsCurrent`, `Unknown`).

17.6.3 L’identité du fichier : `FileId`

Pour que l’étape `fetch` de l’exemple puisse nommer `api.ts`, chaque position porte son fichier :

Extrait 17.32 – `src/ir/source_range.rs`, l. 36–49 — une position porte son fichier

```

36 /// A `(file, line, col)` source position. `line` is 1-indexed, `col` 0-indexed.
37 #[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
38 pub struct SourceRange {
39     pub file: FileId,
40     pub line: u32,
41     pub col: u32,
42 }
43
44 impl SourceRange {
45     /// `(line, col)` sort key for ordering diagnostics within a file.
46     pub fn pos_key(&self) -> (u32, u32) {
47         (self.line, self.col)
48     }
49 }
```

`FileId` (`src/ir/source_range.rs`, l. 3–9) est un `u32` interné dans une `FileTable` : quatre octets et `Copy`, de sorte que `SourceRange` reste `Copy`. C’est le premier pilier d’ADR-19, qui corrige une limite d’ADR-11 : une position dans un CFG inliné depuis un autre fichier ne pouvait pas désigner ce fichier. Une note issue d’un hook greffé porte le `FileId` de son fichier d’origine, sans qu’aucun code de splice ait eu à s’en occuper. La table voyage du programme abaissé jusqu’au

rapport final.

17.6.4 Les producteurs partagés

Certaines chaînes se construisent de la même manière dans plusieurs règles. ADR-19 (§4) les regroupe dans des producteurs de `witness.rs` :

- `resolve_and_classify(registry, component_file, name)` (`src/rules/api/witness.rs`, l. 413–457) résout un nom d'un niveau dans le `FunctionRegistry` : `ResolveUnknown` s'il est introuvable, `LocalFn` s'il est dans le même fichier, `Import(path)` sinon ; puis un `CallEffectful` *seulement* si le corps résolu contient un appel à effet (`find_effectful_call`). Le doccomment en donne la règle de soundness : « the scan only *refines* — an unresolved or effect-free body adds no step, never a weaker verdict ».
- `chase_value` (`src/rules/api/witness.rs`, l. 463–494) remonte d'un saut de liaison (une liaison unique) et produit un `Binding`, puis résout le premier appelé.
- `slot_history` (`src/rules/api/witness.rs`, l. 529–560) raconte le widening d'un slot : un `Write` par effet écrivain enregistré dans `widen_trace`, puis un `Widen` avec l'itération. C'est la chaîne du `Poller` de la section 17.1.

La classification par nom (`EFFECTFUL`, neuf noms comme `fetch`, `setInterval` ou `addEventListener`) garde un ensemble « pur et bon marché » restreint aux builtins en temps constant (`Math.*`, `Date.now`, `parseInt...`), « so a `PureCheap` verdict never hides expensive work ». La provenance moteur, deuxième pilier d'ADR-19, est enregistrée au point de connaissance, pendant l'analyse : `widen_trace` pour le widening, les origines d'inlining pour les greffes.

Piège

Le vocabulaire n'est pas entièrement employé. `ResolveTarget::Setter` n'est construit nulle part (seuls le gabarit et la sérialisation JSON le mentionnent), et `Step::Binding` n'est produit que par `chase_value`. Conséquence visible : quand l'initialiseur appelle un setter, `resolve_and_classify`, qui passe par le `FunctionRegistry` et ignore les setters, écrit une note trompeuse (`/tmp/ch17/setterinit.tsx`, `useState(setUser(null))`) :

```
error lazy-init [hook:1] (line 5:8) this useState init calls a state setter, so it runs a state write on
  ↳ every render (the result is discarded after mount); move the call into an effect or event handler
  ↳ `setUser` could not be resolved, treated as opaque
```

L'ERROR est juste (`must_init_calls_setter`) ; seul le témoin est faux. La correction centrale serait dans le producteur, pas dans la règle (exercice 17.5).

17.6.5 Du témoin à la position : `located`

Certaines règles placent une position dans une étape du témoin mais pas sur le diagnostic lui-même. Plutôt que de corriger chaque règle, le registre applique une fonction à tous les findings :

Extrait 17.33 – `src/rules/registry.rs`, l. 356–369 — `located` : la première position du témoin

```
356 /// A finding with no position of its own takes the first position its witness
357 /// chain names.
358 ///
359 /// The chain is the finding's evidence, and its first located step is where
360 /// that evidence starts – a rule that threaded a span into a `Step` but never
361 /// onto the diagnostic was withholding a position it already had. Applied here
362 /// rather than in each rule, because "a finding renders somewhere" is a
363 /// property of every finding (#131).
364 fn located(mut d: Diagnostic) -> Diagnostic {
```

```

365     if d.range.is_none() {
366         d.range = d.notes.iter().find_map(|n| n.range);
367     }
368     d
369 }

```

C'est une illustration directe du troisième principe du CLAUDE.md : un problème qui touche plusieurs règles se corrige une fois, au niveau central (#131). Deux tests l'épinglent : `a_finding_with_no_range_takes_its_first_witness_position` et `a_finding_that_has_a_range_keeps_it` (`src/rules/registry.rs`, l. 401–450) ; le second vérifie qu'une position posée par la règle n'est jamais écrasée.

17.6.6 Le rendu

La prose est déjà dans chaque note ; le driver n'a qu'à la disposer. En sortie humaine (`src/driver/human.rs`), chaque note devient une ligne `→ message [hook:N] (position)`, avec au plus huit étapes, suivies de « ... n more step(s) » au-delà (`src/driver/human.rs`, l. 207–234) ; sans `-trace`, une ligne « (k trace step(s), rerun with `-trace`) » les résume. La position passe par une fonction unique :

Extrait 17.34 – `src/driver/human.rs`, l. 23–35 — la ligne principale et les étapes partagent la même position

```

23 fn position(
24     range: SourceRange,
25     comp_file: Option<&Path>,
26     files: &FileTable,
27     display: &dyn Fn(&Path) -> String,
28 ) -> String {
29     match files.path(range.file) {
30         Some(f) if comp_file != Some(f) => {
31             format!("({}:{:})", display(f), range.line, range.col)
32         }
33         _ => format!("(line {:})", range.line, range.col),
34     }
35 }

```

En JSON, chaque note garde ses champs structurés en plus du message, avec son `kind` (`src/driver/json.rs`, l. 148–227). Pour `Card.tsx` :

```

{
  "message": "`fetch` has side effects (subscriptions/requests/timers re-fire on every call)",
  "kind": "call",
  "hook_label": null,
  "file": "api.ts",
  "line": 2,
  "col": 2,
  "callee": "fetch",
  "effect_class": "effectful"
}

```

Décision : ADR-024 — attribuer sans jamais fusionner

Décision. L'identité d'un finding est son ancre `SourceRange` ; l'endroit où il s'affiche est une vue. Quand le fichier de l'ancre diffère de celui du composant, la ligne principale nomme le fichier d'origine, et le champ `file` du JSON devient celui de l'ancre. Et les findings ne sont *jamais* dédupliqués entre consommateurs d'un même hook. **Alternative refusée.** Fusionner les dix findings identiques qu'un hook partagé produit dans dix composants.

Les faits par consommateur sont incomparables, pas redondants : le même corps donne infinite-loop sous useStep(1) et redundant-set-state sous useStep(0). Fusionner ferait perdre l'un des deux, un FN.

17.7 Le registre

Les règles produisent des diagnostics bruts. Entre elles et l’affichage, une étape décide de ce qui est montré, à quel niveau et dans quel ordre : RuleRegistry (src/rules/registry.rs, ADR-22 §8). Le registre contient les natives, dans l’ordre d’all_rules(), puis les règles de packs dans leur ordre d’enregistrement, avec une doc par nom de diagnostic et les surcharges du consommateur. Il est partagé par tous les frontends (CLI, WASM), qui exécutent ainsi exactement la même composition.

17.7.1 La passe par composant

Le driver construit un cache pour tout le programme, trie les composants par nom d’affichage, puis appelle registry.check_component(&rule_cache, id) pour chacun (src/driver/mod.rs, l. 397–434). Le cœur de la passe est une boucle sur les règles :

Extrait 17.35 – src/rules/registry.rs, l. 254–294 — la passe par composant, jusqu’aux filtres

```

254     pub fn check_component<'a>(<
255         &self,
256         cache: &'a ProgramCache<'a>,
257         component: ComponentId,
258     ) -> ComponentFindings {
259         let program = cache.program();
260         let mut diags: Vec<Diagnostic> = Vec::new();
261         let mut safe_checks: Vec<SafeCheck> = Vec::new();
262         for r in &self.rules {
263             let ctx = RuleCtx::cached(cache, component, self.options_for(r.name()));
264             let produced = r.check(&ctx);
265             // `safe_check` is consulted on the `raw` output: a rule whose
266             // findings were all filtered away still ran and found something –
267             // it is not "verified safe".
268             if produced.is_empty()
269                 && let Some(sc) = r.safe_check(&ctx)
270             {
271                 safe_checks.push(sc);
272             }
273             diags.extend(produced.into_iter().map(located).map(|d| self.clamped(d)));
274         }
275
276         // A component where the analyzer admits it truncated (`analysis-limit`
277         // says "FN possible") must not also publish `verified: ...` universals
278         // that the very same limit could falsify – an opaque hook may hide the
279         // conditional call, the missing dep, the diverging effect. Read off the
280         // `unfiltered` diagnostics on purpose: silencing the Info with
281         // `--ignore-rule` hides the notice, it does not restore the guarantee.
282         //
283         // The count survives the clear and is reported as
284         // `suspended_safe_checks`: withholding the assurances is right, but
285         // withholding them `silently` is what made a truncated component
286         // indistinguishable from one that simply had nothing to check.
287         let mut suspended_safe_checks = 0;
288         if diags.iter().any(|d| d.rule == AnalysisLimitInfo::NAME) {
289             suspended_safe_checks = safe_checks.len();

```

```

290     safe_checks.clear();
291 }
292
293 diags.retain(|d| self.visible(&d.rule));
294 safe_checks.retain(|s| self.visible(s.rule));

```

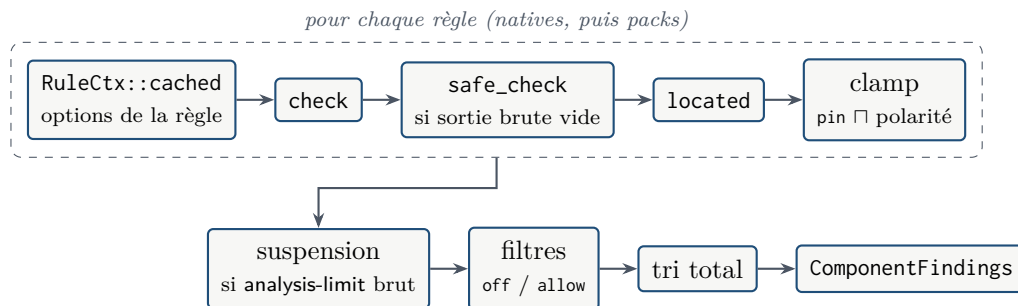


FIGURE 17.4 – La séquence de `check_component`. `safe_check` ne voit que la sortie *brute* de `check`, avant tout filtre ; la suspension lit les diagnostics *non filtrés* ; le clamp précède le tri, dont la sévérité est une clé. Le filtre `-info` vient ensuite, dans le driver.

La figure 17.4 montre l'ordre, qui n'est pas arbitraire. Trois points de soundness en dépendent.

- `safe_check` est consulté sur la sortie *brute* : une règle dont tous les findings seront filtrés par `-ignore-rule` a tout de même trouvé quelque chose ; elle n'est pas « vérifiée sûre ».
- La suspension des assurances lit les diagnostics *avant* les filtres : masquer l'INFO `analysis-limit` cache l'avis, pas la troncature (section 17.7.4).
- Le clamp est appliqué avant le tri, puisque la sévérité est une clé de tri.

Le reste de la fonction (`src/rules/registry.rs`, l. 296–340) établit un *ordre total*. Les règles itèrent des `HashMap` ; sans tri, deux findings de même clé reviendraient dans un ordre dépendant de l'aléa de hachage, et deux exécutions ne produiraient pas des sorties identiques octet pour octet. La clé est le n-uplet (`rule`, discriminant de sévérité, `position`, `message`, `var`, `hook_label`), où la position est (absente ?, chemin du fichier, ligne, colonne) : le chemin, parce qu'un composant dont les hooks sont inlinés depuis plusieurs fichiers mélange leurs ancres (ADR-13) ; « absente ? » en tête, pour que les findings sans position restent en dernier. Les assurances sont triées par nom de règle.

17.7.2 Surcharges et clamp

Un utilisateur peut régler une règle par fichier de configuration ou en ligne de commande. Le frontend résout d'abord précédences (la CLI l'emporte sur la configuration, ADR-22 §5) et remet au registre des surcharges déjà résolues :

Extrait 17.36 – `src/rules/registry.rs`, l. 29–49 — les surcharges du consommateur

```

29 /// Consumer overrides, already precedence-resolved by the frontend (ADR-022)
30 /// §5: CLI beats config; the registry never sees raw flags).
31 #[derive(Debug, Clone, Default)]
32 pub struct RuleOverrides {
33     /// Keyed by diagnostic name (severity/off) – options entries additionally
34     /// require the key to be a pack rule id (natives declare no params in v1).
35     pub entries: BTreeMap<String, OverrideEntry>,
36     /// `--rule` allowlist; `None` = everything visible.
37     pub allow: Option<BTreeSet<String>>,
38 }

```

```

39
40 #[derive(Debug, Clone, Default)]
41 pub struct OverrideEntry {
42     /// Drop this diagnostic name at emission.
43     pub off: bool,
44     /// Severity pin: findings are clamped down to this ceiling (never up -
45     /// [`Diagnostic::clamp`] is downgrade-only by construction).
46     pub ceiling: Option<Severity>,
47     /// Per-rule options, delivered to the rule via [`RuleConfig`].
48     pub options: serde_json::Map<String, serde_json::Value>,
49 }

```

Le champ `ceiling` est un *plafond* : le finding est abaissé à ce niveau s'il le dépasse, jamais relevé.

Extrait 17.37 – `src/rules/api/diagnostic.rs`, l. 104–114 — le clamp ne sait que descendre

```

104 /// Consumer override (ADR-022 §3): lower this finding to `ceiling` when its
105 /// constructed severity exceeds it. Downgrade-only, hence sound to expose:
106 /// Error is constructible only through [`Diagnostic::error`], so the
107 /// constructed severity IS the polarity ceiling and `pin` polarity`
108 /// reduces to a min – an upgrade request is a structural no-op.
109 pub fn clamp(mut self, ceiling: Severity) -> Self {
110     if ceiling.rank() < self.severity.rank() {
111         self.severity = ceiling;
112     }
113     self
114 }

```

Le raisonnement du doc-comment est court et complet. Une `ERROR` ne se construit que par `D diagnostic::error`, donc la sévérité construite *est* le plafond que permet la polarité. Le niveau final vaut $pin \sqcap polarité$ dans l'ordre `INFO` $\sqsubseteq$ `WARNING` $\sqsubseteq$ `ERROR`, et ce meet se réduit à un minimum : une demande de montée est un no-op structurel, que le test `clamp_upgrade_is_a_no_op` (« the soundness case ») vérifie.

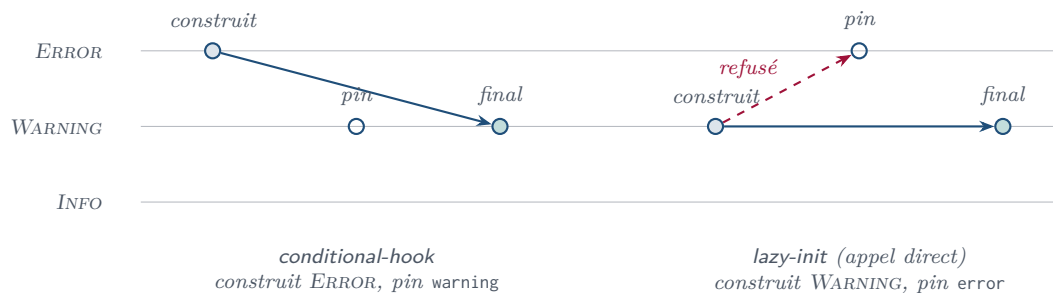


FIGURE 17.5 – Le pipeline du clamp : $final = pin \sqcap construit$, calculé sur le rang. À gauche, une descente honorée ; à droite, une montée demandée et refusée par construction. Les deux cas sont ceux de la configuration `/tmp/ch17/cfg/reactant.config.json`.

La figure 17.5 illustre les deux cas sur le répertoire `/tmp/ch17/cfg/`, qui contient `Profile` (liste 17.6) et `Card`, avec cette configuration :

```

1 {
2   "rules": {
3     "conditional-hook": "warning",
4     "lazy-init": "error"
5   }
6 }

```

```

Card (1 hooks) Card.tsx
  warn lazy-init [hook:0] (line 5:8) this useState is initialised by a direct function call. The call runs on
    ↳ every render but the result is only used on mount; wrap as `useState(() => ...)` to defer it
    (2 trace step(s), rerun with --trace)
Profile (2 hooks) Profile.tsx
  warn conditional-hook [hook:0] (line 5:10) this hook is called conditionally (not on every render path)
    (1 trace step(s), rerun with --trace)

! 2 warning(s) across 3 file(s).

```

Décision : ADR-022 — *pin* $\square$ *polarity*, évalué par finding

Décision. Le niveau épinglé par le consommateur est combiné, finding par finding, avec le niveau que la polarité autorise, par un meet. Un pin ne peut qu'abaisser. **Alternative refusée.** Rejeter statiquement les configurations dont le pin dépasse ce qu'une règle peut prouver. Cela aurait interdit la stratification gratuite : une règle comme *stale-closure* émet des ERROR et des WARNING, et un pin error est légitime pour les premières sans être une promesse sur les secondes.

17.7.3 off, allow et la discipline des noms

Deux autres surcharges filtrent. *off* supprime un nom de diagnostic, *allow* (le drapeau *-rule*) restreint l'affichage à une liste; *off* l'emporte toujours (visible, *src/rules/registry.rs*, l. 220–232). Le doc-comment du module fixe une discipline de nommage : sévérité et *off* sont indexés par *nom de diagnostic*, les options par *identifiant de règle*.

Extrait 17.38 – *src/rules/registry.rs*, l. 11–17 — pourquoi *off* filtre à l'émission

```

11 //! Naming discipline: severity/off overrides are keyed by **diagnostic name**
12 //! ( `Diagnostic::rule`, the namespace of `--rule`/`--ignore-rule` and
13 //! `RuleDoc`); options are keyed by **rule id** ( `Rule::name()` ). One rule may
14 //! emit several diagnostic names ( `setter-in-render` also emits
15 //! `cross-setter-in-render` ), which is why "off" filters findings at
16 //! emission and never skips a rule's execution – skipping by rule id would
17 //! silently swallow the *other* diagnostic name, a forbidden false negative.

```

Si *off* sautait l'exécution de la règle, désactiver *setter-in-render* ferait aussi disparaître *cross-setter-in-render*, que l'utilisateur n'a pas demandé à taire : un FN. Le test *off_on_one_diagnostic_name_keeps_the_other* (*src/rules/registry.rs*, l. 667) l'épingle avec une règle qui émet deux noms.

17.7.4 La suspension des assurances

La règle *analysis-limit* émet une INFO quand l'analyse a dû tronquer, par exemple sur un hook de bibliothèque inconnu. Un composant ainsi tronqué ne doit pas publier en même temps des assurances universelles (« all hooks run unconditionally ») que la limite même pourrait falsifier : un hook opaque peut cacher un appel conditionnel, une dep manquante, un effet divergent. Sur */tmp/ch17/info.tsx*, où Clean n'appelle que des hooks connus et Truncated appelle *useVendor* importé de "vendor-lib", avec *-info -show-clean* :

```

Clean (3 hooks) info.tsx ✓
  verified always-unstable-deps no deps array is defeated by an always-fresh reference
  verified conditional-hook all hooks run unconditionally, in a stable order
  ...
  verified state-mutation no state or prop object is mutated in place
Truncated (3 hooks) info.tsx
  info analysis-limit [hook:0] (line 11:8) hook `useVendor` was not found in the registry. Pass its source file or
    ↳ add a HookSummary to analyse it (FN possible)

```

```
suspended analysis-limit 4 passing check(s) withheld: the analysis was truncated in this component, so they are not
↪ guaranteed
```

(Élagué : Clean reçoit dix lignes `verified`.) Le compte des assurances retenues survit à leur effacement (`suspended_safe_checks`) : les retenir est juste, les retenir *en silence* rendait un composant tronqué indiscernable d'un composant qui n'avait rien à vérifier. Avec en plus `-ignore-rule analysis-limit`, la ligne info disparaît, mais la ligne `suspended` reste et aucune assurance ne revient : c'est le comportement du test `ignoring_the_limit_hides_the_notice_but_not_the_suspension` (`src/rules/registry.rs`, l. 775).

Piège

La suspension est *par composant*, pas par couple (type de limite, vérification) : un enfant inconnu coûte au parent des garanties qui ne portent que sur son propre corps (#31, ouverte). Elle ne touche que les lignes `verified`, jamais un diagnostic ni le code de sortie. Et la coche ✓ d'un composant signifie « aucun diagnostic visible », pas « vérifié » : sous `-ignore-rule analysis-limit`, `Truncated` s'affiche avec ✓ tout en portant sa ligne `suspended`.

17.7.5 Options et validation bruyante

Deux natives déclarent des options typées : `state-lifted-too-high` (`minDepth`, `minWastedRenders`) et `wasted-subtree-render` (`minWastedRenders`, `continuousOnly`). Une option est un `OptionSpec` :

Extrait 17.39 – `src/rules/api/query.rs`, l. 276–288 — la déclaration typée d'une option

```
276 #[derive(Debug, Clone, Copy, PartialEq, Eq)]
277 pub struct OptionSpec {
278     pub name: &'static str,
279     pub kind: OptionKind,
280     /// One line, shown by `reactant explain`.
281     pub doc: &'static str,
282 }
283
284 #[derive(Debug, Clone, Copy, PartialEq, Eq)]
285 pub enum OptionKind {
286     UInt { default: u64, min: u64, max: u64 },
287     Bool { default: bool },
288 }
```

La règle lit sa valeur par `ctx.config().uint(&SPEC)` ou `flag(&SPEC)`, qui rend le défaut en l'absence de réglage. `RuleConfig` n'a pas à valider : `set_overrides` (`src/rules/registry.rs`, l. 160–200) l'a fait avant toute analyse, et de manière *bruyante*. Chaque clé doit nommer une doc existante ; des options doivent viser un identifiant de règle, pas un nom de diagnostic seul ; une native doit déclarer l'option, et la valeur doit passer `OptionSpec::check`. La moindre erreur arrête l'outil (code de sortie 2) :

```
$ reactant check --rule-option conditional-hook:foo=1 .
[error] rule `conditional-hook` is built-in and declares no options
$ reactant check --rule-option state-lifted-too-high:minDepth=0 .
[error] option `minDepth` of rule `state-lifted-too-high` must be an integer between 1 and 64
$ reactant check --rule-option cross-setter-in-render:x=1 .
[error] `cross-setter-in-render` is a diagnostic name, not a rule id. Options must target the rule
$ reactant check --ignore-rule no-such .
[error] unknown rule `no-such`. Run `reactant rules` for the list of valid names
```

Ignorer un réglage mal orthographié serait l'analogue, côté configuration, d'un faux négatif : l'utilisateur croirait une règle désactivée ou réglée alors qu'elle ne l'est pas. La validation est de plus atomique : les surcharges ne sont remplacées qu'après le succès de toutes les vérifications.

L'enregistrement d'une règle de pack (`register`, `src/rules/registry.rs`, l. 142–158) applique la même discipline : l'identifiant doit être de la forme `pack/rule` (les noms nus sont réservés aux natives), n'entrer en collision ni avec une règle ni avec une doc, et la doc fournie doit porter ce même nom. Le dernier cas remonte sous la variante `UnknownRule(doc.name)`, faute de variante dédiée, ce qui donne à l'auteur de pack un message peu parlant.

Piège

Deux commentaires affirment qu'aucune native ne déclare d'options (« `natives declare no params in v1` », `src/rules/registry.rs`, l. 33–34; « `no native rule declares params in v1` », `src/rules/api/query.rs`, l. 229–232). Ils datent d'avant les deux règles de cascade de rendu et sont périmés.

17.7.6 La documentation générée

Chaque nom de diagnostic a une entrée `RuleDoc` :

Extrait 17.40 – `src/rules/docs.rs`, l. 11–24 — la documentation d'un nom de diagnostic

```
11 /// Documentation entry for one diagnostic name.
12 #[derive(Debug, Clone, PartialEq)]
13 pub struct RuleDoc {
14     /// Matches `Diagnostic::rule` exactly.
15     pub name: Cow<'static, str>,
16     /// One line, shown by `reactant rules`.
17     pub summary: Cow<'static, str>,
18     /// What the rule detects and why it matters, shown by `reactant explain`.
19     pub explanation: Cow<'static, str>,
20     /// Minimal buggy snippet.
21     pub example: Cow<'static, str>,
22     /// How to fix it.
23     pub fix: Cow<'static, str>,
24 }
```

La table statique `RULE_DOCS` (vingt et une entrées, construites par une `const fn`) documente les natives ; les packs fournissent leurs docs au chargement (`RuleDoc::new`, chaînes possédées, d'où les `Cow`). `reactant rules` affiche le nom et le `summary` ; `reactant explain <nom>` affiche le reste, et les options déclarées :

```
$ NO_COLOR=1 reactant rules
always-unstable-deps      a dep is a fresh reference every render, so the deps array never matches
analysis-limit            the analyzer deliberately truncated analysis here (potential false negatives)
conditional-hook          hook called inside a conditional branch
...
$ NO_COLOR=1 reactant explain state-lifted-too-high
...
Options:
  minDepth (default 1): report only when the state's home is at least this many levels below its owner
  minWastedRenders (default 2): report only when each write re-renders at least this many components for nothing (the
    ↳ components above the home plus the other elements they build)
$ NO_COLOR=1 reactant explain lazy
[error] unknown rule `lazy`
did you mean: lazy-init?
```

La doc sert aussi de référentiel des noms valides : `set_overrides` refuse toute clé qui n'en nomme aucune. Des tests tiennent la table (`src/rules/docs.rs`, l. 385–426) : `every_rule_name_has_a_doc`, `multi_name_rules_have_docs` (pour les deux noms secondaires), `docs_are_sorted_and_unique`, `no_empty_fields`. Le fichier `tests/docs_drift.rs`, lui, ne teste pas `RULE_DOCS` mais le vocabulaire des packs déclaratifs : chaque ancre, arête et garde que le schéma JSON accepte doit apparaître entre accents graves dans `docs/custom-rules.md` et dans la

référence du skill `skills/reactant-rules/REFERENCE.md` (`tests/docs_drift.rs`, l. 53–66).

17.8 Le cache programme

Une règle s'exécute composant par composant, mais certaines ont besoin d'une structure du programme entier : `infinite-loop` lit le graphe de churn, qui parcourt tous les composants. Tant que ce graphe était construit à l'intérieur de `check`, la phase des règles était quadratique en nombre de composants, au point de bloquer l'analyse de deux gros dépôts du corpus (#86). La correction, amendement d'ADR-21, est un cache programme partagé :

Extrait 17.41 – `src/rules/api/cache.rs`, l. 26–41 — `ProgramCache`

```
26 pub struct ProgramCache<'a> {
27     relations: ProgramRelations<'a>,
28     mounts: OnceLock<MountIndex>,
29     consumers: OnceLock<ContextConsumers>,
30     render: OnceLock<RenderIndex>,
31 }
32
33 impl<'a> ProgramCache<'a> {
34     pub fn new(program: &'a ProgramAnalysisResult) -> Self {
35         ProgramCache {
36             relations: ProgramRelations::new(program),
37             mounts: OnceLock::new(),
38             consumers: OnceLock::new(),
39             render: OnceLock::new(),
40         }
41     }
42 }
```

Chaque entrée est construite au premier usage (`OnceLock::get_or_init`) puis réutilisée par tous les composants du run. La durée de vie 'a lie le cache au programme dont il dérive : « a cache can never be read against a different analysis result ». Quatre structures y vivent (`src/rules/api/cache.rs`, l. 43–76) :

- le graphe de churn, via `ProgramRelations`, la structure du moteur qui le construit (chapitre 15) ;
- la relation `context_consumers` (`ContextConsumers`, ADR-32) ;
- l'index de montage (`MountIndex`), qui dépend lui-même de l'arbre de rendu : `mounts()` appelle `render()` ;
- l'arbre de rendu (`RenderIndex`), construit sur les résumés de dépendance de rendu (définition 16.3).

Les trois dernières sont encore construites par la couche règles, dans `src/rules/helpers/` ; `docs/relations.md` les liste sous « Still built by the rules layer », et ADR-42 (§5) prévoit leur descente dans le moteur. Seuls `new` et `program` sont publics ; les quatre accesseurs sont `pub(in crate::rules)`. Un frontend externe peut donc construire un cache et le passer au registre, jamais lire ses entrées.

Le contexte tient le cache par l'énumération `CacheRef` (liste 17.5) : `Shared(&ProgramCache)` quand le registre fournit le cache du run, `Own(Box<ProgramCache>)` quand un appelant construit un contexte isolé avec `RuleCtx::new`. Le second cas est celui des tests et du harnais `tests/catalogue.rs` : correct, mais il reconstruit les structures du programme pour chaque contexte, donc reproduirait la forme quadratique de #86 sur un gros programme. Un frontend ne doit pas l'imiter.

Remarque

ADR-42 (§5) annonce que « ProgramRelations replaces ProgramCache » et que `rules/api/cache.rs` disparaît. Au commit e67b10a, le fichier existe et ProgramCache *compose* ProgramRelations avec les trois structures restantes (chapitre 14). Le cache emploie OnceLock plutôt que OnceCell, ce qui le rend partageable entre fils d'exécution ; la passe des règles reste pourtant séquentielle, une boucle for sur les composants dans le driver.

17.9 Les helpers programme

Le répertoire `src/rules/helpers/` contient la machinerie partagée entre règles. `helpers/mod.rs` rassemble de petits outils : le nommage (`state_slot_name`, `hook_kind_word`, `describe_value`, qui traduit une valeur abstraite en phrase et interdit d'imprimer un domaine avec `?`), l'applicabilité des assurances (`has_hook_kind`), des balayages (`arg_is_call_free`, `collect_callees`) et la porte $\forall$ -stable (section 17.5.3). Sept sous-modules calculent des *relations de programme*. Chacun répond à une question dont la réponse, pour être sound, doit pencher d'un côté précis ; le tableau 17.8 donne ce côté.

module	question et principe de soundness	consommateurs
<code>render_tree.rs</code>	où l'état « habite » (<code>home_of</code>), quels éléments re-rendent pour rien, où un setter transmis est appelé depuis un événement. L' <i>absence</i> d'usage est une preuve, donc toute inconnue compte comme un usage	<code>state-lifted-too-high</code> , <code>wasted-subtree-render</code>
<code>mount.rs</code>	ce que les sites d'appel prouvent sur la durée de montage d'un consommateur (<code>MountCoupling</code>). Le verdict ne fait que <i>dégrader</i> , jamais supprimer	<code>frozen-initial-state</code>
<code>context_flow.rs</code>	un <code>useContext</code> a-t-il un provider au-dessus de lui ? Une absence n'est affirmée que sur des chemins entièrement vus	ancre Tier A <code>context_consumers</code>
<code>jsx.rs</code> , <code>providers.rs</code>	la valeur passée à un élément est-elle une référence neuve à chaque rendu (<code>site_identity</code>) ? Verdict must à deux valeurs, <code>FreshEveryRender</code> ou <code>Unknown</code>	<code>unstable-context-value</code> , relations JSX du Tier A
<code>cycles.rs</code>	projection par composant des cycles du graphe de churn ; la position d'écriture est l'identité de la ligne (ADR-24)	ancre Tier A <code>churn_cycles</code>
<code>purity.rs</code>	un corps de fonction est-il impur (mutation d'un objet venu d'ailleurs, appel de setter) ? Impure n'est affirmé que sur un site prouvé	garde Tier A <code>updater_body</code>

TABLE 17.8 – Les helpers programme de `src/rules/helpers/` et le côté vers lequel chacun penche.

L'absence est une preuve : `RenderIndex::home_of`. `state-lifted-too-high` affirme qu'un état n'est utilisé que profondément dans un seul sous-arbre, et que les composants au-dessus ne font que le transmettre. C'est une affirmation d'*absence* d'usage. Le doc-comment du module en tire la conséquence (`src/rules/helpers/render_tree.rs`, l. 5–13) : un élément dont le composant ne se résout pas, un composant ré-entré par récursion, un résumé qui a atteint un plafond (`T`) « all stop the descent » et comptent comme usage. `home_of` (`src/rules/helpers/render_tree.rs`, l. 201–256) exige au moins un lecteur et au moins un écrivain dans l'arbre, puis descend tant que le nœud courant n'utilise pas la valeur et qu'un seul enfant a des usagers. `resolve` n'accepte qu'une origine prouvée, jamais une correspondance par nom, qui pourrait choisir un homonyme d'un autre fichier et cacher les usages du vrai. Sur un état `q` de App utilisé seulement par Search, deux niveaux plus bas (`/tmp/ch17/lifted.tsx`) :

```
| App (1 hooks) lifted.tsx
```

```
warn state-lifted-too-high [hook:0] (line 4:8) state `q` is only used inside `

```

Chaque saut du chemin devient une étape `Step::Forward`. Le niveau est `WARNING` : les rendus en trop sont certains, leur coût ne l'est pas. La règle est étudiée au chapitre 21.

Dégrader, jamais tuer : `MountIndex`. La key de la section 17.5.4 a transformé une `ERROR` en `INFO`. Le verdict vient de `MountIndex::coupling` (`src/rules/helpers/mount.rs`, l. 123–148) : `Reseeds` si *chaque* site d'appel porte une key ou une garde qui lit au moins ce que lit le seed ; `WriterCoupled` si chaque site rend le consommateur sous une garde d'état écrite dans le même commit que le nourricier ; `Free` sinon. Le doc-comment de `Reseeds` (`src/rules/helpers/mount.rs`, l. 44–67) explique pourquoi ce n'est pas une preuve : une garde qui passe d'une valeur vraie à une autre garde l'enfant monté, et une key objet se convertit en une chaîne constante. Les deux formes gèlent réellement, et l'analyseur ne sait pas les distinguer de l'idiome : il dégrade donc, au lieu de supprimer. Un appelé non résolu est attribué à *tous* les composants de ce nom, sans danger puisque la relation ne peut que dégrader, et seulement si tous les sites remontent.

Une absence sur des chemins vus : `ContextConsumers`. La relation `context_consumers` dit, pour chaque `useContext`, si un composant qui peut le rendre fournit la même cellule de contexte. Son verdict négatif s'appelle `NoneOnAnalyzedPaths` et non « aucun provider » : `build` (`src/rules/helpers/context_flow.rs`, l. 87–139) abandonne une ligne dès que l'ascendance n'est pas complète (#110), qu'une récursion a été coupée, ou qu'un composant non analysé mentionne un membre de la chaîne et pourrait donc se trouver au-dessus.

L'identité d'un site : `site_identity`. Une valeur passée à `<Ctx.Provider value=...>` est jugée dans l'environnement du bloc, sur le tas convergé (`src/rules/helpers/jsx.rs`, l. 270) : `FreshEveryRender` si elle n'est qu'une référence neuve, `Unknown` sinon. Une variable n'est jugée que si elle est liée *exactement une fois* dans le corps : zéro liaison signifie que le parent la possède, deux liaisons que la valeur de sortie du bloc n'est peut-être pas celle que l'élément a reçue. Les sites de providers (`collect_provider_sites`, `providers.rs`) exigent un nom de la forme `X.Provider` où `X` est une cellule `createContext` prouvée.

Piège

`collect_provider_sites` est plus étroit que la dépendance de rendu du moteur. Sur `/tmp/ch17/ctx.tsx`, qui passe un objet neuf à trois providers de la même cellule `Ctx`, seul `<Ctx.Provider value= user, setUser >` au niveau supérieur du rendu reçoit le `WARNING unstable-context-value`. La forme React 19 `<Ctx value=...>` (composant `P19`) et un provider construit dans un `items.map(...)` (composant `Rows`) ne sont pas vus : le premier parce que `provider_context` exige le suffixe `.Provider`, le second parce que `each_component_element` ne traverse pas les corps de fonction (la famille de #30). Dans les deux cas, l'élément est aussi traité comme un composant inconnu, et une `INFO analysis-limit` (« component 'Ctx' was not found », « component 'Ctx.Provider' was not found ») avoue la limite et suspend les assurances : le FN n'est pas silencieux.

Les deux derniers. `collect_cycle_rows` (`src/rules/helpers/cycles.rs`, l. 101–135) projette les cycles du graphe de churn sur un composant : une arête portée par un effet d'un autre composant sera rapportée dans la passe de cet autre composant, et une arête sans position ne

produit pas de ligne (un finding manqué, jamais faux). `classify_body` (`purity.rs`) rend `Impure` si un site de mutation s'enracine hors du corps ou si un setter est appelé; tout ce que la poursuite des liaisons ne sait pas placer compte comme possédé par le corps, pour que `Impure` reste une affirmation.

17.10 La frontière « walk-free »

Les helpers qui calculent des relations dans la couche règles sont une dette, et ADR-42 (§1) la nomme : « a fact computed in two places is two facts that drift ». La décision fixe la frontière entre moteur et règles en une phrase : une règle lit des lignes de relation et appelle des primitives `must`; elle ne parcourt ni CFG ni expression. Chaque parcours vit dans le moteur et produit une relation nommée dont les colonnes déclarent leur polarité (définition 14.1).

La frontière est tenue par un test à *cliquet* (*ratchet*) :

Extrait 17.42 – `tests/layer_boundary.rs`, l. 19–41 — la liste qui ne peut que rétrécir

```
19 /// The rule files still walking syntax, as of ADR-042 slice 5. Shrink only.
20 const ALLOWED: &[&str] = &[
21     "src/rules/api/query.rs",
22     "src/rules/api/witness.rs",
23     "src/rules/helpers/jsx.rs",
24     "src/rules/helpers/mod.rs",
25     "src/rules/helpers/mount.rs",
26     "src/rules/helpers/purity.rs",
27     "src/rules/impls/redundant_set_state.rs",
28     "src/rules/impls/setter_in_render.rs",
29     "src/rules/impls/state_mutation.rs",
30     "src/rules/impls/unnecessary_rerender.rs",
31 ];
32
33 /// What counts as touching syntax: iterating a CFG's blocks, matching
34 /// statements or terminators, or descending an expression tree.
35 const MARKERS: &[&str] = &[
36     "cfg.blocks",
37     ".blocks.values()",
38     "Stmt::",
39     "for_each_child",
40     "Terminator::",
41 ];
```

La mécanique est simple (`tests/layer_boundary.rs`, l. 54–59) : `walks_syntax` lit un fichier, le coupe au premier `#[cfg(test)]` (un test a le droit de construire un CFG à la main) et cherche l'un des cinq marqueurs. Deux tests encadrent la liste. `no_rule_file_outside_the_list_walks_syntax` refuse qu'un fichier hors liste parcoure la syntaxe; son message d'échec dit quoi faire (« compute the fact in the engine as a relation (ADR-042 §1) instead of adding them to the list »). `the_list_only_shrinks` exige qu'un fichier qui a cessé de parcourir la syntaxe quitte la liste. Chaque entrée est une promotion à faire, pas une exception. Un troisième test vérifie que `docs/relations.md` nomme chaque relation stockée.

Décision : ADR-042 — les relations sont des produits du moteur

Décision. Une règle ne parcourt pas la syntaxe; le graphe de churn descend dans le moteur (`engine/churn.rs`); les preuves suivent les données (`on_all_paths`, l'évaluateur convergé); la frontière de certification ne bouge pas, `must_effect_cycle` reste dans `query.rs`. **Alternatives refusées.** Fondre le churn dans `StateValue` (déjà refusé par ADR-41 pour la dépendance

de rendu) ; unifier `effect_triggers` avec les deps de la dépendance de rendu, parce qu'une dépendance n'est pas une identité.

Piège

Le cliquet est une heuristique textuelle. Il documente un engagement, il ne le prouve pas. Un parcours déguisé (`cfg.blocks.iter()` au lieu de `.values()`, ou une fonction de parcours définie ailleurs) échapperait aux marqueurs, et c'est ainsi que `render_tree.rs`, `context_flow.rs`, `providers.rs` et `cycles.rs` sont hors liste : ils lisent des résumés du moteur ou délèguent le parcours (`jsx::top_level_exprs`, `root_detector::collect_compapp_refs`). La liste réelle diffère d'ailleurs de celle qu'écrit ADR-42 : `helpers/mod.rs` y est entré, `impls/conditional_hook.rs` en est sorti depuis que sa logique vit dans `hook_is_conditional`.

17.11 La façade, le Tier A et l'ouverture

`src/rules/mod.rs` ré-exporte tout ce qui est public à son chemin historique (`src/rules/mod.rs`, l. 25–59), de sorte que les déplacements successifs (découpe en `api/impls/helpers`, descente des setters et de l'évaluateur dans le moteur) n'ont cassé aucun chemin `crate::rules::X`. La façade se lit en trois cercles : `pub` pour la surface stable (trait, contexte, verdicts, jetons, primitives, Diagnostic, vocabulaire des témoins, docs, registre, les dix-neuf structures de règles) ; `pub(crate)` pour le vocabulaire partagé avec le moteur et le driver (tables d'alias de setters, nommage, balayages, évaluateur) ; `pub(in crate::rules)` pour ce qui ne sort pas de la couche (`all_deps_provably_stable`, `fn_lit_binding...`). `must_effect_cycle` n'est pas ré-exporté du tout.

Cette surface est aussi celle contre laquelle s'écrivent les règles déclaratives du Tier A (ADR-22, chapitre 22) : un pack JSON s'ancore sur une relation (`writers`, `churn_cycles`, `context_consumers...`) et ne peut atteindre `ERROR` que par des gardes `must_*` qui appellent les primitives de ce chapitre (`must_setter_on_all_paths`, `must_dominates_all_exits`, `must_direct_write...`, `src/rules/declarative/exec.rs`). `tests/catalogue.rs` mesure ce que le Tier A sait exprimer : vingt-deux classes de règles, dont `EXPRESSIBLE_NOW` = 21 sont prouvées chargées, tirant sur un fixture fautif et muettes sur un fixture conforme (`tests/catalogue.rs`, l. 1094) ; la vingt-deuxième est exclue par conception (#101). Une issue ouverte montre que la leçon d'#142 vaut aussi là : une règle de pack épinglée `error` atteint `ERROR` sur une seule garde `must_*` alors qu'une garde `may` de la même conjonction n'est pas prouvée (#143).

Remarque

Le principe de la couche tient en une phrase : *la polarité est un type, et l'ERROR est un jeton*. Tout le reste en découle : les primitives frappent là où elles constatent, les classifieurs rendent `⊤` comme une variante, le registre ne peut qu'abaisser, les témoins voyagent avec les preuves, et les parcours de syntaxe quittent la couche un par un. Ce qui reste de confiance tient dans dix appels à `Certified::mint`.

Résumé du chapitre

- La couche règles (`src/rules/`) transforme le point fixe et les relations du moteur en diagnostics gradués, localisés et expliqués. Une règle implémente `Rule` (`name`, `check`, `safe_check`, `options`) et ne lit que son `RuleCtx`. Dix-neuf règles natives émettent vingt et un noms de diagnostic.
- Le FN historique d'ADR-21 venait de deux prédicats non complémentaires (`is_stable`, `is_unstable`) et d'un mauvais quantificateur. La réponse est structurelle : la polarité est

un type, et $\top$ une variante retournée.

- `Diagnostic` vit dans un module `feuille`; sa sévérité est privée; `Diagnostic::error` exige un `Certified<E>`, dont le constructeur est privé à `query.rs` (définition 17.1). Forger une `ERROR` est une erreur de compilation (E0624, E0616, E0451, E0308). Transférer une vraie preuve reste possible et relève de la revue.
- Les primitives `must` frappent au point de connaissance : dominance sur les sorties *atteignables* (`hook_is_conditional`), plus grand point fixe booléen (`must_setter_on_all_paths`), preuve d'écriture du nourricier (`classify_motion`) que `must_frozen_seed` ne peut que dégrader, re-dérivation depuis les arêtes brutes (`must_effect_cycle`), preuve de tous les conjoints (`must_stale_capture`). La porte des `deps` quantifie $\forall$ -stable.
- Les témoins sont des `Note` typées par un vocabulaire fermé de quatorze `Step` (définition 17.2); `Step::render` est l'unique point de prose; `FileId` permet de nommer le fichier d'une étape; `located` donne à tout `finding` la première position de sa chaîne.
- Le registre exécute, par composant : contexte, `check`, `safe_check` sur sortie brute, `located`, `clamp` *pin* $\sqcap$ *polarité* (ne peut qu'abaisser), suspension des assurances sous `analysis-limit`, filtres `off/allow` à l'émission, tri total. Toute surcharge invalide est une erreur bruyante.
- `ProgramCache` construit une fois par run les structures du programme (#86). Les helpers programme penchent tous d'un côté sûr : l'absence d'usage est une preuve (donc l'inconnu est un usage), le couplage de montage ne fait que dégrader, l'absence de `provider` n'est affirmée que sur des chemins vus.
- La frontière `walk-free` (ADR-42) est tenue par un cliquet textuel; dix fichiers y figurent encore.
- Fichiers rencontrés : `src/rules/mod.rs`, `registry.rs`, `docs.rs`, `api/query.rs`, `api/diagnostic.rs`, `api/witness.rs`, `api/cache.rs`, `helpers/*.rs`, `tests/layer_boundary.rs`. Les chapitres suivants appliquent cette surface règle par règle, en commençant par les règles sur l'état (chapitre 18).

Exercices

Exercice 17.1: Un distributeur de jetons

On veut une primitive `must_effect_writes(body: &CFG, setters: &HashSet<Var>) -> MustResult<>` qui certifie qu'un corps d'effet écrit un état sur tous ses chemins. (a) Écrivez-la en respectant le contrat de la section 17.5, en réutilisant une primitive existante. (b) Un collègue propose plutôt `must_effect_writes(all_paths: bool) -> MustResult<>`, l'analyse étant faite par la règle. Pourquoi ADR-21 appelle-t-il cela un distributeur de jetons ?

Solution

(a) Collecter les blocs qui contiennent un appel à l'un des `setters`, puis déléguer à `must_on_all_paths(body, &blocks)` et, sur `All`, frapper `Certified::mint()`, `Provenance::default()` (ou mieux, `Provenance::at` avec la position du premier appel); sur `None`, rendre `None` ou `Some` selon qu'au moins un appel existe. La fonction vit dans `query.rs`, seul endroit où `mint` est visible. (b) La version à booléen ne fait aucune analyse : tout appelant qui passe `true` obtient un jeton, et la polarité redevient une affaire de discipline dans chaque règle, exactement la situation qui a produit le FN d'origine.

Exercice 17.2: Itérer `must_out`

Un corps d'effet est un losange : `B0` (entrée, branch `c`) mène à `B1` et `B2`, qui rejoignent `B3` (`return`). `B1` et `B2` appellent tous deux `setx(0)`, `B0` et `B3` n'appellent rien. Déroulez

l'algorithme du liste 17.23 dans l'ordre B1, B2, B3. Quel est le verdict ? Même question si seul B1 appelle `setX(0)`.

Solution

Initialisation : $out(B0) = \text{faux}$, les autres à vrai. Passe 1 : $out(B1) = out(B0) \vee \text{vrai} = \text{vrai}$; de même $out(B2) = \text{vrai}$; $out(B3) = (\text{vrai} \wedge \text{vrai}) \vee \text{faux} = \text{vrai}$. Rien n'a changé : stable, la sortie B3 vaut vrai, verdict All, et le témoin désigne le site de B1, premier dans l'ordre des blocs. Si seul B1 appelle : passe 1, $out(B2) = \text{faux}$ (changé), $out(B3) = \text{vrai} \wedge \text{faux} = \text{faux}$ (changé) ; passe 2 sans changement ; verdict Some.

Exercice 17.3: Pourquoi off n'arrête pas une règle

Un utilisateur écrit "setter-in-render": "off" dans sa configuration. Que se passerait-il si le registre sautait l'exécution de la règle `SetterInRender` au lieu de filtrer ses findings à l'émission ? Quel test l'empêche ?

Solution

La règle émet aussi `cross-setter-in-render`, un nom distinct que l'utilisateur n'a pas désactivé. Sauter la règle ferait disparaître ces findings en silence : un FN. C'est pourquoi `off` est indexé par nom de diagnostic et appliqué par visible après check. Le test `off_on_one_diagnostic_name_keeps_the_other` l'épingle.

Exercice 17.4: Tromper le cliquet

Proposez une modification d'une règle hors liste qui parcourt les blocs d'un CFG sans déclencher `layer_boundary`. Pourquoi serait-elle contraire à ADR-42, même si le test passe ?

Solution

Les marqueurs sont des sous-chaînes. `for (_, b) in body.blocks.iter()` ne contient ni `cfg.blocks` (la variable s'appelle `body`) ni `.blocks.values()` ; un `use crate::ir::stmt::Stmt as S` ; suivi de `S::Let .. évite Stmt::`. Le test passe, mais le fait est calculé dans la couche règles : un second calcul du même fait, qui dérivera du premier. ADR-42 demande de produire une relation dans le moteur ; le cliquet n'est qu'une alarme, pas la définition de la frontière.

Exercice 17.5: Un témoin faux, une correction centrale

Le WARNING « 'setUser' could not be resolved, treated as opaque » de la section 17.6 accompagne une ERROR juste. Proposez une correction qui ne soit pas un cas particulier dans `lazy-init`. Même question pour « state state » dans le témoin de `frozen-initial-state`.

Solution

Le vocabulaire a déjà la bonne variante, `ResolveTarget::Setter`, jamais construite. La correction centrale est dans le producteur partagé : `resolve_and_classify` (ou son appelant) doit connaître les setters du composant et rendre `ResolveSetter`, suivi de `CallSetter`, avant de consulter le `FunctionRegistry`. Toute règle qui résout un nom en profite. Pour « state state », le défaut est un contrat implicite entre la fermeture `name` et le gabarit de `Step::Write`, qui préfixe « state » : soit `MovingFeeder::display` ne doit pas être pré-qualifié par « state », soit le gabarit doit recevoir un nom qualifié par un type distinct. Corriger la chaîne dans la seule règle serait un contournement.

Exercice 17.6: Dédupliquer, un FN

Un hook `useStep(step)` contient `useEffect(() => setN(n + step); , [n])`. Il est appelé par `L` avec `useStep(1)` et par `M` avec `useStep(0)`. Expliquez pourquoi fusionner les findings des deux consommateurs, au motif qu'ils ont la même ancre dans le fichier du hook, produirait un FN (ADR-24).

Solution

Après greffe, les deux corps ne sont pas analysés dans le même contexte : dans `L`, l'effet écrit `n + 1`, une valeur qui croît, d'où `infinite-loop` ; dans `M`, il écrit `n + 0`, la valeur déjà détenue, d'où `redundant-set-state`. L'analyseur le confirme sur `/tmp/ch17/step.tsx` : `L` reçoit un `WARNING infinite-loop` et `M` un `WARNING redundant-set-state`, tous deux à la ligne 5:2 du hook. Les deux findings ont la même ancre mais affirment des faits différents. Garder un représentant supprimerait l'autre fait, que rien ne signalerai plus : un FN, que l'invariant de soundness interdit.

Chapitre 18

Les règles sur l'état

Objectifs du chapitre.

- Connaître les sept règles natives qui parlent de l'état `useState` hors boucles infinies : `lazy-init`, `unstable-context-value`, `redundant-set-state`, `setter-in-render`, `derived-state`, `state-mutation`, `frozen-initial-state` — le bug React que chacune vise et les conditions exactes de son déclenchement.
- Savoir dire, pour chacune, quelle primitive `must` lui ouvre l'ERROR, et ce qui la retient au WARNING ou à l'INFO.
- Distinguer les règles qui *signalent un défaut possible* (leur silence doit être une preuve) de celles qui *affirment un fait* (elles se taisent tant que le fait n'est pas établi).
- Savoir quelles relations et quelles tables de noms chaque règle lit, et reconnaître les écarts entre ces lectures, source de plusieurs faux négatifs observés.
- Reproduire chaque verdict sur un programme court avec l'analyseur, et connaître les faux positifs assumés et les faux négatifs relevés au commit e67b10a.

Prérequis : chapitre 17 (le trait `Rule`, le jeton `Certified`, définition 17.1, les primitives `must`, les témoins, définition 17.2) ; chapitre 14 (relations du moteur, définition 14.1, en particulier `slot_writers` et `slot_seeds`) ; chapitre 10 (treillis de stabilité, définition 10.3, et la double vue de l'état) ; la dominance (définition 3.9). Les deux règles d'état les plus difficiles, `infinite-loop` et `state-lifted-too-high`, sont l'objet du chapitre 19.

Le chapitre 17 a décrit la surface contre laquelle toute règle est écrite : un contexte `RuleCtx`, des verdicts polarisés, un sceau qui interdit de fabriquer une `ERROR` sans preuve. Ce chapitre ouvre la partie consacrée aux règles elles-mêmes, en commençant par la famille la plus nombreuse, celle qui parle de l'état local d'un composant. Neuf fichiers de `src/rules/impls/` en relèvent ; sept sont traités ici, les deux restants au chapitre suivant.

L'ordre suit la difficulté. `lazy-init` ne regarde qu'une expression, syntaxiquement ; `unstable-context-value` lit un seul fait `must` du domaine ; `redundant-set-state` lit le store d'état ; `setter-in-render` raisonne par dominance et par phase ; `derived-state` exige un fait « sur tous les chemins » et interroge la relation des écrivains ; `state-mutation` poursuit une identité de référence à travers des portées ; `frozen-initial-state`, enfin, a besoin de l'analyse descendante entre composants et d'une stratification en trois niveaux. Chaque section suit la même grille : le bug React visé, les conditions exactes, le niveau et la preuve requise, les relations lues, des exemples gradués avec leur sortie réelle, les faux positifs (FP) assumés et les faux négatifs (FN) interdits ou observés, les pièges.

Toutes les sorties ont été obtenues au commit e67b10a avec le binaire `target/debug/reactant` sur des fichiers écrits pour ce chapitre sous `/tmp/ch18/`. Sauf mention contraire, la commande est :

```
reactant check /tmp/ch18/<fichier>.tsx --all-roots --no-color --fail-on never
```

complétée de `-trace` (notes de témoin), `-info` (niveau INFO et assurances verified), `-show-clean` (composants sans diagnostic) et `-rule <nom>` (filtre sur un nom de diagnostic) selon le besoin. Les numéros de ligne des sorties renvoient aux fichiers reproduits, dont les listings portent les vrais numéros.

18.1 Le socle commun

Avant d'entrer dans les règles, fixons ce qu'elles ont en commun : la manière dont elles retrouvent un slot à partir d'un nom, la manière dont elles déclarent leur applicabilité, et les deux attitudes qu'une règle peut avoir face à l'incertitude.

18.1.1 Du nom au slot

Une règle ne voit pas le code source, elle voit l'IR (chapitre 6). Un `const [n, setN] = useState(0)` y devient deux liaisons du CFG de rendu, `Let n = StateVal(0)` et `Let setN = StateSetter(0)`, où `0` est le label du slot (définition 6.3). Pour savoir qu'un appel `setN(1)` écrit le slot `0`, il faut donc une table « variable → label ». Le moteur en fournit plusieurs, de portée croissante :

Extrait 18.1 – `src/engine/setters.rs`, l. 515–528 — les deux tables syntaxiques du rendu

```
515 pub(crate) fn setter_var_labels(cfg: &CFG) -> HashMap<Var, HookLabel> {
516     state_binding_labels(cfg, |rhs| match rhs {
517         Expr::StateSetter(label) => Some(*label),
518         _ => None,
519     })
520 }
521
522 /// `var → state label` for every `let var = useState(...)[0]` (the value) in `cfg`.
523 pub(crate) fn state_val_labels(cfg: &CFG) -> HashMap<Var, HookLabel> {
524     state_binding_labels(cfg, |rhs| match rhs {
525         Expr::StateVal(label) => Some(*label),
526         _ => None,
527     })
528 }
```

`setter_var_labels` et `state_val_labels` ne lisent que les liaisons *directes* du CFG de rendu. Un alias `const s = setN` n'y figure pas. La fermeture par alias est l'affaire de `resolve_setter_aliases`, et sa forme complète, appliquée au rendu *et* à tous les corps de hooks, est `all_setter_labels` :

Extrait 18.2 – `src/engine/setters.rs`, l. 621–635 — la table fermée par alias, sur tous les corps

```
621 /// Alias-resolved `setter var → state label` across the render body and every
622 /// hook body. Utility inlining binds setter params via aliases (`let setter =
623 /// setX`) inside spliced bodies; rules matching a setter by name must follow
624 /// those aliases through every body or a spliced setter call goes unseen (false
625 /// negative). The shared recipe of `derived-state`, `state-mutation`,
626 /// `stale-closure` and `frozen-initial-state`.
627 pub(crate) fn all_setter_labels(comp: &AnalysisResult<StateValue>) -> HashMap<Var, HookLabel> {
628     let mut labels = setter_var_labels(&comp.render_cfg);
629     for cfg in
630         std::iter::once(&comp.render_cfg).chain(comp.hooks.iter().filter_map(|h| h.body_cfg()))
631     {
632         labels = resolve_setter_aliases(cfg, &labels);
633     }
634     labels
635 }
```

Son doc-comment nomme la raison d’être de la fonction : l’inlining d’un utilitaire lie les paramètres setter par des alias (`let setter = setX`) dans les corps greffés (chapitre 8), et une règle qui cherche un setter par son nom sans suivre ces alias rate l’appel — un faux négatif. Le nom affiché d’un slot, enfin, vient de `state_slot_name` (`src/rules/helpers/mod.rs`, l. 94–105), qui choisit le plus petit nom source non technique et se replie sur « `state ##N` ».

Les sept règles n’utilisent pas toutes la même table. Le tableau 18.1 en fait l’inventaire ; nous verrons que deux FN observés viennent précisément de règles qui lisent une table plus étroite que leurs voisines.

règle	reconnaissance d’un setter	reconnaissance d’une valeur d’état
lazy-init	setter_var_labels (rendu, sans alias)	—
unstable-context-value	—	—
redundant-set-state	env.setter_label (environnement abstrait)	store state
setter-in-render	liaisons <code>Let ... = StateSetter</code> du rendu, plus les setters reçus en props	state_val_labels, fermée par alias sur le rendu
derived-state	all_setter_labels	state_val_labels
state-mutation	all_setter_labels	state_val_labels
frozen-initial-state	all_setter_labels	state_val_labels

TABLE 18.1 – Les tables de noms lues par chaque règle (lecture des fichiers de `src/rules/impls/` au commit e67b10a).

18.1.2 Applicabilité et assurance

Chaque règle implémente `safe_check` (section 17.2) : si `check` n’a rien trouvé et que la règle est *applicable* au composant, elle publie une assurance positive, affichée sous `-info` par une ligne `verified`. L’applicabilité se décide presque toujours par `has_hook_kind` (`src/rules/helpers/mod.rs`, l. 107–118), qui teste la présence d’un appel de hook d’un genre donné. Le registre suspend ces assurances dans un composant qui porte un `analysis-limit` (section 17.7.4) ; nous en verrons un exemple à la section 18.3.

18.1.3 Signaler ou affirmer

Les sept règles ne prennent pas le même risque quand l’analyse ne sait pas. Deux attitudes coexistent, et les confondre fait mal lire le code.

Définition 18.1: Règle de signalement, règle d’affirmation

Une *règle de signalement* rapporte un *défaut possible* : un appel coûteux répété, un setter qui peut s’exécuter au rendu, un objet d’état muté, un état figé. L’invariant de soundness (définition 3.8) s’y applique pleinement : elle ne peut se taire que sur une *preuve* d’absence du défaut, et une incertitude se traduit en `WARNING` ou en `INFO`, jamais en silence.

Une *règle d’affirmation* rapporte un *fait* : « cette écriture stocke la valeur déjà présente », « cet effet recopie un autre état », « cette valeur est une nouvelle référence à chaque rendu ». Elle ne parle que si le fait est établi, et elle utilise des faits `may` pour *retenir* son diagnostic. Son silence sur un cas incertain est une perte de précision.

Le code dit lui-même dans quelle famille il se range. Le doc-comment de `InitEffect` (`src/rules/impls/lazy_init.rs`, l. 50–54) garantit que « the trigger stays "a call is present" (sound: no false negative) » ; celui d’`UnstableContextValue` assume qu’un cas non prouvé est sauté, « precision lost, never soundness » (`src/rules/impls/unstable_context_value.rs`, l. 19–24) ; le commentaire des slots contestés de `redundant-set-state` résume la seconde attitude en

une phrase : « Both are may-facts used to *withhold*, so over-approximating costs a warning instead of inventing one » (src/rules/impls/redundant_set_state.rs, l. 46–56). lazy-init, setter-in-render, state-mutation et frozen-initial-state sont des règles de signalement ; unstable-context-value, redundant-set-state et derived-state des règles d'affirmation.

Piège

« Règle d'affirmation » ne veut pas dire « règle qui peut atteindre l'ERROR ». Les trois règles d'affirmation de ce chapitre plafonnent au WARNING : le fait est certain, son coût ne l'est pas (un re-rendu de plus, une valeur recopiée). À l'inverse, les ERROR du chapitre viennent toutes de règles de signalement, et chacune passe par une primitive must qui re-dérive le fait qu'elle certifie (invariant 17.1).

18.2 lazy-init : un appel dans l'initialiseur

Rappel React : l'initialiseur de useState

useState(init) n'utilise init qu'au premier rendu ; aux rendus suivants, React rend la valeur courante du slot et ignore l'argument. Mais l'argument est une expression JavaScript ordinaire : elle est *évaluée* à chaque appel du composant, avant que useState ne soit appelé. useState(buildTree()) construit donc un arbre à chaque rendu pour le jeter aussitôt. La forme paresseuse useState(() => buildTree()) passe une fonction, que React n'appelle qu'au montage [Met26b]. useRef(init) a le même défaut, mais pas de forme paresseuse : useRef(() => x) stocke la fonction elle-même, et le correctif est l'idiome if (ref.current === null) ref.current =

18.2.1 La condition : un appel syntaxiquement présent

La règle parcourt les hooks du composant et ne retient que les entrées HookEntry::State et HookEntry::Ref. Son critère de déclenchement tient en une ligne :

Extrait 18.3 – src/rules/impls/lazy_init.rs, l. 107–115 — le déclencheur : un appel dans l'initialiseur, sans poursuite des liaisons

```

107      // Fire only on a call that is syntactically part of the init. We do
108      // NOT chase calls through local bindings: after custom-hook inlining
109      // an already-lazy `useState(() => f())` is flattened to a temp bound
110      // to `f()`, which is indistinguishable from an eager
111      // `const x = f(); useState(x)` – chasing would flag correct lazy code
112      // (corpus FP: `useMediaQuery`). See git history / notes.
113      if init.is_call_free() {
114          continue;
115      }

```

Expr::is_call_free (src/ir/expr.rs, l. 513–528) est faux dès que l'arbre de l'expression contient un nœud Call, New, CompApp ou NativeElem ; les corps de FnLit ne sont pas traversés, ce qui laisse passer la forme paresseuse. Un appel imbriqué (useState(1 + f()), useState({ key: getValue() })) déclenche ; un littéral, une lecture de champ (useState(props.value)) ou une arithmétique sans appel ne déclenchent pas.

La décision la plus importante est ce que la règle *ne fait pas* : elle ne suit pas les liaisons locales. const initial = buildTree(); useState(initial) est muet.

Décision : pas de poursuite des liaisons

Constat. Après l'inlining d'un hook utilisateur (chapitre 8), une forme déjà paresseuse `useState(() => f())` écrite dans le corps du hook est aplatie en un temporaire lié à `f()` dans le composant appelant ; l'IR devient indiscernable de `const x = f(); useState(x)`. **Décision.** Ne déclencher que sur un appel syntaxiquement présent dans l'initialiseur. **Alternative refusée.** Poursuivre la liaison, ce qui signalait du code correct : le commentaire cite un faux positif du corpus, `useMediaQuery`. **Coût.** Un appel caché derrière un `const` n'est pas signalé.

Piège

Deux commentaires du fichier disent le contraire du code. Le doc-comment du type (`src/rules/impls/lazy_init.rs`, l. 19–24) présente comme premier atout la poursuite de l'appel « through local bindings » ; le commentaire en tête de `check` (`src/rules/impls/lazy_init.rs`, l. 89–93) décrit une poursuite « only when that binding is used exactly once ». Les deux décrivent un comportement retiré. C'est le code (l. 107–115), et le test `call_behind_binding_is_not_chased`, qui font foi.

18.2.2 Graduer par la nature de l'appel

Déclencher est une décision binaire ; le niveau, lui, dépend de ce que fait l'appel. La règle classe les callees de l'initialiseur :

Extrait 18.4 – `src/rules/impls/lazy_init.rs`, l. 206–234 — classification de l'initialiseur, avec sa précedence

```

206 fn classify_init_effect(init: &Expr, setters: &HashSet<Var>) -> InitEffect {
207     let mut callees: Vec<&Expr> = Vec::new();
208     collect_callees(init, &mut callees);
209
210     let mut effectful: Option<String> = None;
211     let mut pure_name: Option<String> = None;
212     let mut has_unknown = false;
213
214     for callee in callees {
215         match classify_callee(callee, setters) {
216             Callee::Setter => return InitEffect::Setter,
217             Callee::Effectful(n) => effectful.get_or_insert(n),
218             Callee::PureCheap(n) => pure_name.get_or_insert(n),
219             Callee::Other => {
220                 has_unknown = true;
221                 continue;
222             }
223         };
224     }
225
226     match (effectful, pure_name) {
227         (Some(n), _) => InitEffect::Effectful(n),
228         // A pure-cheap verdict needs an actual pure call AND no unknown call.
229         // `has_unknown` also covers the no-`Call`-callee case (e.g. a `CompApp` /
230         // `NativeElem` init that is not call-free but has no plain callee).
231         (None, Some(n)) if !has_unknown => InitEffect::PureCheap(n),
232         _ => InitEffect::Unknown,
233     }
234 }

```

`collect_callees` (`src/rules/helpers/mod.rs`, l. 173–195) rassemble le callee de chaque `Call` et `New`, et l'élément lui-même pour un `CompApp` ou un `NativeElem` (travail réel de coût inconnu).

Chaque callee est classé par `classify_callee` (`src/rules/impls/lazy_init.rs`, l. 246–265) : un `StateSetter` ou une variable de la table `setters` est un setter ; sinon la classification par nom, partagée avec les témoins (`witness::classify_callee_name`, `src/rules/api/witness.rs`, l. 324–348), reconnaît une courte liste d'appels à effet de bord (`fetch`, `subscribe`, `addEventListener`, `setTimeout`, `setInterval`, `requestAnimationFrame`, `postMessage...`, `src/rules/api/witness.rs`, l. 309–319) et quelques fonctions pures et bon marché (`Math.*`, `Date.now`, `performance.now`, `parseInt`, `Number...`). La précedence est `Setter` > `Effectful` > `Unknown` > `PureCheap` : un seul appel inconnu suffit à perdre le verdict « tout est bon marché ».

Cette classification est une heuristique de *nommage*. Elle ne peut que changer le niveau et la formulation, jamais le déclenchement : c'est ce qui la rend inoffensive pour la soundness. Le tableau 18.2 donne la matrice complète, lue sur `src/rules/impls/lazy_init.rs`, l. 122–181.

InitEffect	useState	useRef
Setter	ERROR (via <code>must_init_calls_setter</code>)	ERROR (idem)
Effectful(n)	WARNING « calls n, which has side effects »	WARNING
Unknown	WARNING « initialised by a direct function call »	INFO
PureCheap(n)	INFO « wrapping ... is optional »	silence (continue)

TABLE 18.2 – Niveau de `lazy-init` selon la nature de l'appel et le hook. Pour `useRef`, le correctif coûte trois lignes au lieu d'un jeton : la règle ne le recommande en `WARNING` que si l'appel répété nuit (écriture d'état, effet de bord).

18.2.3 La seule Error : un setter dans l'initialiseur

`useState(setN(1))` exécute une écriture d'état à chaque rendu : c'est une boucle de rendu, pas une simple perte de temps. La règle passe alors par la primitive `must_init_calls_setter` :

Extrait 18.5 – `src/rules/api/query.rs`, l. 722–740 — la primitive qui certifie un setter dans l'initialiseur

```

722 pub fn must_init_calls_setter(init: &Expr, setters: &HashSet<Var>) -> MustResult<InitSetterCall> {
723     let mut callees = Vec::new();
724     crate::rules::collect_callees(init, &mut callees);
725     let calls_setter = callees.iter().any(|c| {
726         let c = match c {
727             Expr::TSAnnotated(inner) => inner.as_ref(),
728             other => other,
729         };
730         matches!(c, Expr::StateSetter(_)) || matches!(c, Expr::Var(v) if setters.contains(v))
731     });
732     if calls_setter {
733         // No provenance: an `Expr` carries no span, so the position of the
734         // offending call is something the *caller* holds and this primitive
735         // never sees.
736         MustResult::All(Certified::mint(InitSetterCall, Provenance::default()))
737     } else {
738         MustResult::None
739     }
740 }
```

Extrait 18.6 – `src/rules/impls/lazy_init.rs`, l. 148–158 — le bras `Setter` de `useState`

```

148     (InitEffect::Setter, false) => {
149         let msg = "this useState init calls a state setter, so it runs a state write on \
150             every render (the result is discarded after mount); move the call \
```

```

151         into an effect or event handler";
152     match must_init_calls_setter(init, &setters) {
153         MustResult::All(proof) => Diagnostic::error("lazy-init", proof, msg),
154         // Detection matches classify's `Setter` branch, so this is
155         // unreachable; fall back to Warning (still reported – no FN).
156         _ => Diagnostic::warn("lazy-init", msg),
157     }
158 }

```

La primitive refait la détection de `classify_callee` sur les seuls setters ; comme la règle n'entre dans ce bras que si la classification a rendu `Setter`, le repli `Diagnostic::warn` est inatteignable, ce que dit le commentaire (l. 154–155). Garder ce repli est la discipline du chapitre 17 : le niveau `ERROR` ne s'obtient que par le jeton, et une incohérence future entre la règle et la primitive coûterait un `WARNING`, jamais une `ERROR` non prouvée. La primitive mint sans provenance (« an Expr carries no span ») ; la règle ancre le diagnostic sur la plage du hook.

18.2.4 Exemples

Extrait 18.7 – Initialiseurs gradués (/tmp/ch18/lazy.tsx)

```

1 import { useState, useRef } from "react";
2
3 declare function buildTree(): object;
4
5 export function Lazy() {
6     const [t] = useState(buildTree());
7     const [now] = useState(Date.now());
8     const [data] = useState(fetch("/api"));
9     const [ok] = useState(() => buildTree());
10    return <p>{String(t)}{now}{String(data)}{String(ok)}</p>;
11 }
12
13 export function SetterInInit() {
14     const [n, setN] = useState(0);
15     const [m] = useState(setN(1));
16     return <p>{n}{String(m)}</p>;
17 }
18
19 export function Refs() {
20     const cache = useRef(new Map());
21     const seed = useRef(Math.random());
22     const req = useRef(fetch("/x"));
23     return <p>{String(cache.current)}{seed.current}{String(req.current)}</p>;
24 }

```

Sortie avec `-info -trace -rule lazy-init` (composant `Behind` omis, il est étudié plus bas) :

```

Lazy (4 hooks) lazy.tsx
warn lazy-init [hook:0] (line 6:8) this useState is initialised by a direct function call. The call runs on
  ↳ every render but the result is only used on mount; wrap as `useState(() => ...)` to defer it
  ↳ `buildTree` could not be resolved, treated as opaque
warn lazy-init [hook:2] (line 8:8) this useState init calls `fetch`, which has side effects, on every render,
  ↳ and the result is only used on mount, so every later render repeats the effect (duplicate subscriptions/requests
  ↳ /timers, not just wasted work); wrap as `useState(() => ...)`
  ↳ `fetch` could not be resolved, treated as opaque
info lazy-init [hook:1] (line 7:8) this useState init calls `Date.now` on every render; the call is cheap and
  ↳ pure, so wrapping as `useState(() => ...)` is optional
Refs (3 hooks) lazy.tsx
warn lazy-init [hook:2] (line 22:8) this useRef init calls `fetch`, which has side effects, on every render, and
  ↳ only the first render's value is kept, so every later render repeats the effect (duplicate subscriptions/
  ↳ requests/timers); initialise lazily with `if (ref.current === null) ref.current = ...`

```



```

→ `fetch` could not be resolved, treated as opaque
info lazy-init [hook:0] (line 20:8) this useRef is initialised by a direct function call. The call runs on every
  ↳ render but the result is only kept from the first; if it is not cheap, initialise lazily with `if (ref.current
    ↳ === null) ref.current = ...`
SetterInInit (2 hooks) lazy.tsx
error lazy-init [hook:1] (line 15:8) this useState init calls a state setter, so it runs a state write on every
  ↳ render (the result is discarded after mount); move the call into an effect or event handler
→ `setN` could not be resolved, treated as opaque

```

Chaque ligne se lit sur la matrice. `buildTree()` est un appel inconnu (WARNING); `Date.now()` est pur et bon marché (INFO); `fetch` est à effet (WARNING au message distinct); la forme paresseuse est muette. Pour les refs, `new Map()` est un New de callee inconnu (INFO pour `useRef`), `Math.random()` est classé pur et bon marché (silence pour `useRef`), `fetch` garde son WARNING. `[hook:N]` est ici le label du slot ou de la ref. La note de témoin vient de `witness::chase_value`, qui tente de résoudre le callee par le registre de fonctions; pour `setN`, elle affirme « could not be resolved » alors que le setter est parfaitement connu de la règle — la note décrit la résolution de fonction, pas la classification. Sur le même composant, `setter-in-render` est muet : l'initialiseur d'un `HookEntry::State` n'est pas une instruction du CFG de rendu que cette règle parcourt (section 18.5).

18.2.5 Ce que la règle ne voit pas

Extrait 18.8 – Un appel caché, deux fois (/tmp/ch18/lazy.tsx, suite)

```

26 export function Behind({ c }: { c: boolean }) {
27   const initial = buildTree();
28   const [a] = useState(initial);
29   const [b] = useState(c ? buildTree() : 0);
30   const [d] = useState(1 + Number(buildTree()));
31   return <p>{String(a)}{String(b)}{d}</p>;
32 }

```

```

Behind (3 hooks) lazy.tsx
warn lazy-init [hook:2] (line 30:8) this useState is initialised by a direct function call. The call runs on
  ↳ every render but the result is only used on mount; wrap as `useState(() => ...)` to defer it
→ `Number` could not be resolved, treated as opaque

```

Le slot a est muet par décision. Le slot b l'est aussi, et c'est plus surprenant : l'appel est bien écrit dans l'argument de `useState`. L'impression de l'IR (sonde du chapitre 7) en donne la raison :

```

B0:
  let initial = buildTree() @27:8
  let a = StateVal(0) @28:9
  => branch c ? B1 : B2

B1:
  let __t0 = buildTree() @29:27
  => jump B3

B2:
  let __t0 = 0 @29:41
  => jump B3

...
hooks:
  #0 State init=initial @28:8
  #1 State init=__t0 @29:8
  #2 State init=(1 Add Number(buildTree())) @30:8

```

(Élagué.) Le lowering abaisse la conditionnelle en diamant avec un temporaire `__t0` (chapitre 7), et l'initialiseur du hook #1 devient `Var(__t0)`, sans appel. Le refus de poursuivre les liaisons, pensé pour les temporaires de l'inlining, frappe aussi ceux du lowering. `useState(c`

&& buildTree()) se comporte de même. C'est un FN d'une règle de signalement : sous -info, le composant reçoit même l'assurance `verified lazy-init`. Aucune issue ne le suit au commit `e67b10a`. La correction naturelle, dans l'esprit « général d'abord », distinguerait un temporaire introduit par le lowering (préfixe `__t`) d'une liaison écrite par l'auteur (exercice 18.2).

18.3 unstable-context-value : une valeur neuve à chaque rendu

Rappel React : contexte et identité

Un composant qui appelle `useContext(Ctx)` est re-rendu chaque fois que la valeur du provider le plus proche change, et React compare par `Object.is` — indépendamment de `React.memo` sur le consommateur. Écrire `<Ctx.Provider value={{ a, b }}>` alloue un nouvel objet à chaque rendu du provider : *tous* les consommateurs re-rendent alors à chaque rendu du provider, même quand `a` et `b` n'ont pas bougé. Le correctif est `useMemo` sur la valeur. React 19 accepte aussi `<Ctx value>` comme provider, en plus de `<Ctx.Provider value>` [Met26b].

Cette règle est la plus courte du chapitre (72 lignes) parce que tout son travail est délégué : à un helper qui trouve les providers, et à un fait `must` du domaine qui dit si la valeur est neuve.

18.3.1 Trouver les providers prouvés

Extrait 18.9 – `src/rules/helpers/providers.rs`, l. 47–86 — les sites de providers prouvés, dans un ordre déterministe

```
47 /// Every proven context provider in the component's **render body**, in a
48 /// deterministic order. Render-only for the reason given in [`super::jsx`].
49 pub(crate) fn collect_provider_sites(comp: &AnalysisResult<StateValue>) -> Vec<ProviderSite<'_>> {
50     let contexts: HashMap<&Var, &ContextId> = comp
51         .module_consts
52         .iter()
53         .filter_map(|(name, init)| match init {
54             ModuleConstInit::Context(id) => Some((name, id)),
55             _ => None,
56         })
57         .collect();
58     if contexts.is_empty() {
59         return Vec::new();
60     }
61
62     let cfg = &comp.render_cfg;
63     let bindings = local_bindings(cfg);
64     let mut found: Vec<((BlockId, usize), ProviderSite<'_>>) = Vec::new();
65     for (block_id, exprs) in top_level_exprs(cfg) {
66         let env = comp.block_states.get(&block_id);
67         for (idx, expr) in exprs.into_iter().enumerate() {
68             each_component_element(expr, &mut |name, props, span| {
69                 let Some((context, context_id)) = provider_context(name, &contexts) else {
70                     return;
71                 };
72                 found.push((
73                     (block_id, idx),
74                     ProviderSite {
75                         context,
76                         context_id,
77                         identity: site_identity(prop_value(props), env, &bindings, comp),
78                         span,
79                     },
79                 ));
80             });
81         }
82     }
83     found
```

```

80         ));
81     });
82 }
83 }
84 found.sort_by_key(|a| a.0);
85 found.into_iter().map(|(_, site)| site).collect()
86 }

```

Un élément est un provider de cette règle si son nom est `X.Provider` et si `X` est une constante de module initialisée par un `createContext` atteint par un `import React (ModuleConstInit::Context)`. `provider_context (src/rules/helpers/providers.rs, l. 88–99)` retire le suffixe `.Provider` et cherche la base parmi ces contextes; l'identité canonique `ContextId (#109)` sert aux relations qui appariant consommateurs et providers entre fichiers. Seuls les expressions de premier niveau du corps de rendu sont parcourues (`top_level_exprs`) : un provider construit dans une flèche passée à `.map` ou dans un `useCallback` n'est pas vu (#30), ce que le code motive par le fait qu'un provider bâti dans un `useMemo` est précisément la forme corrigée. Les sites sont triés par (bloc, index d'expression) pour que la sortie ne dépende pas de l'itération d'une table.

18.3.2 Le fait `must : FreshEveryRender`

Extrait 18.10 – `src/rules/helpers/jsx.rs`, l. 267–298 — l'identité d'une valeur à un site du rendu

```

267 /// Is the expression a fresh reference at *this* site, on every render?
268 ///
269 /// The one reader of the identity fact, shared by both JSX relations.
270 pub(crate) fn site_identity(
271     value: Option<&Expr>,
272     env: Option<&AbstractEnv<StateValue>>,
273     bindings: &HashMap<&str, Vec<&Expr>>,
274     comp: &AnalysisResult<StateValue>,
275 ) -> ValueIdentity {
276     let (Some(value), Some(env)) = (value, env) else {
277         return ValueIdentity::Unknown;
278     };
279     // A `Var` is read from the block's *exit* env, which equals the value at
280     // the element only when the variable is bound exactly once in this body:
281     // a second binding (a rebind after the JSX, a branch temp) makes the exit
282     // value a different object than the one the element received. Zero
283     // bindings means the value is owned by a parent – its freshness is the
284     // parent's, not this component's, so it is not this rule's finding.
285     if let Expr::Var(v) = value.peel_ts()
286         && bindings.get(v.as_str()).map_or(0, Vec::len) != 1
287     {
288         return ValueIdentity::Unknown;
289     }
290     // The converged heap, not an empty one: it resolves a props-rooted
291     // `FieldAccess` instead of degrading it to  $\top$ .
292     let val = comp.eval_in(env, value);
293     if val.is_unstable_reference_only() {
294         ValueIdentity::FreshEveryRender
295     } else {
296         ValueIdentity::Unknown
297     }
298 }

```

Deux étapes. D'abord une garde syntaxique : si la valeur est une variable, elle n'est lue dans l'environnement du bloc que si elle est liée *exactement une fois* dans le corps. Zéro liaison signifie

que la valeur vient du parent (sa fraîcheur est l'affaire du parent); deux liaisons signifient que la valeur en sortie de bloc n'est pas forcément celle que l'élément a reçue. Ensuite l'évaluation avec le tas convergé, et le prédicat :

Extrait 18.11 – src/domains/impls/state_value.rs, l. 281–287 — un fait must : une référence neuve, et rien d'autre

```
281    /// True when the value is exactly a per-render-fresh reference (nothing
282    /// else) – a *must* fact: a new reference every render, guaranteed.
283    /// `Versioned` (changes only at setter events) deliberately returns false
284    /// (ADR-017).
285    pub fn is_unstable_reference_only(&self) -> bool {
286        self.reference == Stability::PerRender && self.populated_kinds().only(KindMask::REF)
287    }
```

Le verdict `Stability::PerRender` seul ne suffirait pas : pour une sorte non référence, `to_stability` rend aussi `PerRender` quand la valeur « peut changer à chaque rendu » (un nombre qui bouge, chapitre 10), et un nombre qui change n'est pas un problème d'identité. `is_unstable_reference_only` exige que la seule sorte peuplée soit la référence. Une valeur tirée d'un état (`Versioned`, définition 10.3) ou d'un memo n'est pas `PerRender` : elle rend `Unknown`, et la règle se tait.

Extrait 18.12 – src/rules/impls/unstable_context_value.rs, l. 50–71 — la règle entière

```
50    fn check(&self, ctx: &RuleCtx) -> Vec<Diagnostic> {
51        collect_provider_sites(ctx.comp())
52        .into_iter()
53        .filter(|site| site.identity == ValueIdentity::FreshEveryRender)
54        .map(|site| {
55            let context = site.context;
56            let mut d = Diagnostic::warn(
57                Self::NAME,
58                format!(
59                    "`{context}.Provider` is given a newly allocated value on every render. \
60                    `Object.is` fails for every consumer, so each `useContext({context})` \
61                    re-renders whenever this component does, even when nothing in the value \
62                    changed; wrap the value in `useMemo`"
63                ),
64            );
65            if let Some(span) = site.span {
66                d = d.with_range(span);
67            }
68            d
69        })
70        .collect()
71    }
```

Décision : Warning, jamais Error

La référence neuve est certaine; ce qu'elle coûte ne l'est pas. Le doc-comment (src/rules/impls/unstable_context_value.rs, l. 30–31) en donne l'exemple : un provider dont les deux consommateurs re-rendent de toute façon ne paie rien. C'est la définition même du WARNING pour « un fait certain au coût incertain » (CLAUDE.md, niveaux de diagnostic). Le diagnostic n'a ni label ni variable : il est ancré sur la balise ouvrante du provider.

18.3.3 Exemples

Extrait 18.13 – Cinq providers (/tmp/ch18/ctx.tsx)

```

1 import { createContext, useMemo, useState } from "react";
2
3 const Theme = createContext({ dark: false });
4
5 export function Inline({ dark }: { dark: boolean }) {
6   return (
7     <Theme.Provider value={{ dark }}>
8       <p />
9     </Theme.Provider>
10  );
11 }
12
13 export function ViaLocal({ dark }: { dark: boolean }) {
14   const value = { dark, size: 2 };
15   return (
16     <Theme.Provider value={value}>
17       <p />
18     </Theme.Provider>
19   );
20 }
21
22 export function Memoized({ dark }: { dark: boolean }) {
23   const value = useMemo(() => ({ dark }), [dark]);
24   return (
25     <Theme.Provider value={value}>
26       <p />
27     </Theme.Provider>
28   );
29 }
30
31 export function FromState() {
32   const [value] = useState({ dark: true });
33   return (
34     <Theme.Provider value={value}>
35       <p />
36     </Theme.Provider>
37   );
38 }
39
40 export function React19({ dark }: { dark: boolean }) {
41   return (
42     <Theme value={{ dark }}>
43       <p />
44     </Theme>
45   );
46 }

```

Sortie avec `-info -show-clean` :

```

FromState (1 hooks) ctx.tsx
  info analysis-limit component `Theme.Provider` was not found in the analysis registry. Pass its file on the
    ↳ command line to analyse it (FN possible)
  suspended analysis-limit 5 passing check(s) withheld: the analysis was truncated in this component, so they are not
    ↳ guaranteed
Inline (0 hooks) ctx.tsx
  info analysis-limit component `Theme.Provider` was not found in the analysis registry. Pass its file on the
    ↳ command line to analyse it (FN possible)
  warn unstable-context-value (line 7:4) `Theme.Provider` is given a newly allocated value on every render. `Object
    ↳ .is` fails for every consumer, so each `useContext(Theme)` re-renders whenever this component does, even when
    ↳ nothing in the value changed; wrap the value in `useMemo`
Memoized (1 hooks) ctx.tsx
  info analysis-limit component `Theme.Provider` was not found in the analysis registry. Pass its file on the

```

```

    ↪ command line to analyse it (FN possible)
suspended analysis-limit 4 passing check(s) withheld: the analysis was truncated in this component, so they are not
    ↪ guaranteed
React19 (0 hooks) ctx.tsx
info analysis-limit component `Theme` was not found in the analysis registry. Pass its file on the command line to
    ↪ analyse it (FN possible)
ViaLocal (0 hooks) ctx.tsx
info analysis-limit component `Theme.Provider` was not found in the analysis registry. Pass its file on the
    ↪ command line to analyse it (FN possible)
warn unstable-context-value (line 16:4) `Theme.Provider` is given a newly allocated value on every render. `
    ↪ Object.is` fails for every consumer, so each `useContext(Theme)` re-renders whenever this component does, even
    ↪ when nothing in the value changed; wrap the value in `useMemo`

```

Inline et ViaLocal déclenchent : la valeur est un littéral objet, directement ou à travers une variable liée une seule fois. Memoized et FromState se taisent : une valeur de memo ou d'état n'est pas PerRender. La sortie montre au passage deux mécanismes du registre. L'élément JSX Theme.Provider est aussi, pour le moteur, un composant inconnu, d'où l'INFO analysis-limit ; et dans les composants où aucune règle n'a rien trouvé, cet aveu de troncature *suspend* les assurances verified (section 17.7.4) — on lit « 4 passing check(s) withheld » au lieu de « verified unstable-context-value ». React19 se tait : la forme <Theme value> n'est pas reconnue, faute du suffixe .Provider. C'est un FN de précision que la règle assume.

Piège

Le doc-comment du type affirme qu'« an imported context is not proven here and is skipped » (src/rules/impls/unstable_context_value.rs, l. 21–24). Ce n'est vrai que si le fichier qui définit le contexte n'est pas analysé. Quand il l'est, le résolveur recopie ModuleConstInit::Context dans les constantes de module de l'importateur, et le provider devient prouvé. Vérifié sur deux fichiers (/tmp/rs-verif/ctx/, app.tsx important ThemeCtx de ./ctx) : reactant check . rapporte ThemeCtx.Provider ligne 8, reactant check app.tsx ne rapporte que le contexte local. Le périmètre de la règle dépend donc des fichiers passés sur la ligne de commande (chapitre 23).

18.4 **redundant-set-state** : écrire ce que l'état contient déjà

Rappel React : bail-out

Quand un setter reçoit une valeur Object.is-égale à la valeur courante, React abandonne la mise à jour (*bail-out*) : pas de re-rendu des enfants, pas d'effets. L'écriture n'en reste pas moins du code mort, souvent le reste d'une refonte, et React peut devoir rendre le composant une fois de plus pour s'apercevoir qu'elle ne change rien [Met26b].

C'est la première règle qui lit le *store d'état* convergé (définition 11.2) plutôt qu'une expression. Son principe tient en une comparaison ; tout l'intérêt est dans ce que la comparaison approxime, et dans les cas où la règle se déclare incompétente.

18.4.1 Le test : stable et stable

La règle examine deux familles de corps. Dans le corps de rendu, chaque bloc est lu avec son environnement d'entrée (block_states) ; dans chaque corps d'effet, l'environnement de sortie du rendu. Handlers, callbacks et memos ne sont pas examinés. Dans ces corps, seules les instructions de premier niveau de la forme setX(e) sont testées :

Extrait 18.14 – src/rules/impls/redundant_set_state.rs, l. 253–279 — le test d'un appel de setter

```

253   for stmt in stmts {
254       if let Stmt::ExprStmt(Expr::Call { fn_, args }, call_span) = stmt
255           && let Expr::Var(name) = fn_.as_ref()
256           && let Some(label) = env.setter_label(name)
257       {
258           if skip_labels.contains(&label) || contested.contains(&label) {
259               continue;
260           }
261
262           let arg_val = args
263               .first()
264               .map(|a| crate::rules::eval_in_stores(a, env, component, state, memo, heap))
265               .unwrap_or(StateValue::top());
266
267           let current_val = state.get(label);
268
269           if arg_val.is_stable() && current_val.is_stable() {
270               diags.push(
271                   Diagnostic::warn(
272                       "redundant-set-state",
273                       format!(
274                           "state {} is set to a stable value it already holds, so the update is \
275                           redundant",
276                           name_of(label)
277                       ),
278                   )
279               .with_label(label)

```

Le setter est reconnu par `env.setter_label`, la table de l'environnement abstrait (tableau 18.1). La valeur écrite est évaluée dans les stores ; la valeur courante est lue dans le store d'état, *pas* par l'évaluation de `StateVal`. La règle déclenche si les deux sont `Stable` au sens de `is_stable` (`src/domains/impls/state_value.rs`, l. 414–417), c'est-à-dire si leur projection `to_stability` vaut `Stable`.

Pourquoi « stable et stable » approximerait-il « égal » ? Parce que le store contient la *jointure des valeurs écrites*, y compris celle que l'on teste. Pour un nombre, `useState(42)` puis `setN(42)` donne un store `[42, 42]`, ponctuel, donc stable ; `useState(0)` puis `setN(42)` donne `[0, 42]`, qui n'est pas ponctuel : la projection rend `PerRender` et la règle se tait. Le même raisonnement vaut pour les chaînes (ensemble de constantes), les booléens et le mélange de deux sortes. C'est la double vue de l'état du chapitre 10 : le store est la vue « événementielle » de ce qui a été écrit, et c'est la seule qui puisse répondre à la question posée ; l'évaluation de `StateVal(1)` rend `Versioned`, qui n'est jamais `Stable`.

Piège

Pour les *références*, l'argument s'effondre. `Stability::Stable` ne porte aucune identité : deux constantes de module distinctes sont toutes deux `Stable`, et leur jointure aussi. « stable et stable » ne prouve donc pas « même référence ». La documentation explique la règle parle de « stable and equal » (`src/rules/docs.rs`, l. 223) ; le code ne teste que la stabilité. L'exemple `TwoConsts` ci-dessous est un faux positif assumé de niveau `WARNING`.

18.4.2 Les slots contestés

Affirmer « l'état contient déjà cette valeur » suppose que l'écriture testée est toute l'histoire du slot. Deux situations ôtent ce droit, et la règle les calcule avant tout test :

Extrait 18.15 – `src/rules/impls/redundant_set_state.rs`, l. 57–81 — les slots sur lesquels la règle n'a pas qualité pour parler

```

57     let contested: HashSet<HookLabel> = {
58         let mut regions: HashMap<HookLabel, WriterRegion> = HashMap::new();
59         let mut multi: HashSet<HookLabel> = HashSet::new();
60         for w in &result.slot_writers {
61             if w.owner.is_some() {
62                 continue; // a foreign label is the owner's (ADR-042 §2)
63             }
64             match regions.entry(w.slot) {
65                 std::collections::hash_map::Entry::Vacant(e) => {
66                     e.insert(w.region);
67                 }
68                 std::collections::hash_map::Entry::Occupied(e) => {
69                     if *e.get() != w.region {
70                         multi.insert(w.slot);
71                     }
72                 }
73             }
74         }
75         let escaping = result.escaping_slots();
76         regions
77             .keys()
78             .copied()
79             .filter(|s| multi.contains(s) || escaping.contains(s))
80             .collect()
81     };

```

Un slot est *contesté* s'il est écrit dans plus d'une région lexicale (`WriterRegion`, colonne exacte de `slot_writers`, section 14.4), ou si son setter s'échappe du composant (`escaping_slots`, `src/engine/setters.rs`, l. 2383–2398). Les lignes étrangères (owner non vide, écriture d'un slot du parent par un setter reçu en prop) sont ignorées, car leur label est celui du propriétaire (ADR-42 §2). La lecture de la relation remplace un ancien balayage du rendu et des autres effets, qui manquait les écrivains des handlers, des `useCallback` et des continuations `.then()` (#92).

Une seconde exclusion vise les corps d'effets. La fonction `collect_transition_setters`, définie en `src/rules/impls/redundant_set_state.rs`, l. 172–191, écarte tout label dont deux appels, dans le même corps et y compris dans des `FnLit` imbriqués, reçoivent des arguments abstraits différents — le motif « machine à états » `if (ok) setMode("idle") else setMode("busy")`, où chaque écriture pourrait bien changer la valeur.

18.4.3 Exemples

Extrait 18.16 – Écritures redondantes ou non (`/tmp/ch18/redundant.tsx`)

```

1  import { useState, useEffect } from "react";
2
3  export function Redundant() {
4      const [n, setN] = useState(42);
5      useEffect(() => {
6          setN(42);
7      }, []);
8      return <p>{n}</p>;
9  }
10
11 export function NotRedundant() {
12     const [n, setN] = useState(0);
13     useEffect(() => {

```

```

14     setN(42);
15   }, []);
16   return <p>{n}</p>;
17 }
18
19 export function Contested() {
20   const [n, setN] = useState(0);
21   useEffect(() => {
22     setN(0);
23   }, []);
24   return <button onClick={() => setN(1)}>{n}</button>;
25 }
26
27 export function Transition({ ok }: { ok: boolean }) {
28   const [mode, setMode] = useState("idle");
29   useEffect(() => {
30     if (ok) setMode("idle");
31     else setMode("busy");
32   }, [ok]);
33   return <p>{mode}</p>;
34 }
35
36 const A = { k: "a" };
37 const B = { k: "b" };
38
39 export function TwoConsts() {
40   const [v, setV] = useState(A);
41   useEffect(() => {
42     setV(B);
43   }, []);
44   return <p>{v.k}</p>;
45 }
46
47 export function InRender() {
48   const [n, setN] = useState(0);
49   setN(0);
50   return <p>{n}</p>;
51 }

```

Sortie avec `-trace -rule redundant-set-state -show-clean` :

```

Contested (3 hooks) redundant.tsx ✓
InRender (1 hooks) redundant.tsx
  warn redundant-set-state [hook:0] (line 49:2) state `n` is set to a stable value it already holds, so the update
    ↳ is redundant
    → the value written to state `n` is the value it already holds [hook:0] (line 49:2)
NotRedundant (2 hooks) redundant.tsx ✓
Redundant (2 hooks) redundant.tsx
  warn redundant-set-state [hook:0] (line 5:2) state `n` is set to a stable value it already holds, so the update
    ↳ is redundant
    → the value written to state `n` is the value it already holds [hook:0] (line 6:4)
Transition (2 hooks) redundant.tsx ✓
TwoConsts (2 hooks) redundant.tsx
  warn redundant-set-state [hook:0] (line 41:2) state `v` is set to a stable value it already holds, so the update
    ↳ is redundant
    → the value written to state `v` is the value it already holds [hook:0] (line 42:4)

```

- Redundant : store [42, 42], argument [42, 42]. Le diagnostic est ancré sur l'effet (ligne 5), la note sur l'appel (ligne 6) : pour les effets, la règle recopie la plage de l'effet sur ses diagnostics a posteriori (src/rules/impls/redundant_set_state.rs, l. 127–133).
- NotRedundant : store [0, 42], silence. Sans filtre, c'est unnecessary-rerender qui parle (« mount-only effect sets state 'n' to a constant different from its initial value »).

- **Contested** : le slot est écrit par l'effet et par un handler, deux régions ; silence, bien que l'effet écrive 0 dans un état initialisé à 0.
- **Transition** : deux arguments différents dans le même corps, label écarté.
- **TwoConsts** : le faux positif annoncé. `v` passe de A à B, un vrai changement de référence, mais les deux sont Stable.
- **InRender** : dans le corps de rendu, la règle ne pose jamais de plage elle-même ; la position (line 49:2) vient de `located`, qui recopie la plage de la première note (section 17.6). Sans filtre, le même appel reçoit aussi une **ERROR setter-in-render** : aucune déduplication entre règles.

Remarque

Deux écritures identiques et redondantes dans un même effet (`/tmp/ch18/redundant2.tsx`, `if (ok) setMode("idle"); else setMode("idle");` avec un état initialisé à "idle") produisent deux diagnostics de même message et de même plage (celle de l'effet), qui ne diffèrent que par la position de leur note. Le rendu humain les regroupe et affiche « [in 2 components] » suivi de « in: Twice, Twice » : le regroupement du rendu compte des attributions, et en trouve deux pour un seul composant. La sortie JSON montre bien deux diagnostics distincts. Défaut cosmétique du rendu, relevé au commit e67b10a.

Piège

La règle ne voit que les appels en *position d'instruction* de premier niveau : un `setN(0)` dans un `.then(() => ...)` d'un effet, ou dans l'argument d'un autre appel, n'est pas testé. Pour une règle d'affirmation de niveau **WARNING**, c'est une perte de précision et non une atteinte à la soundness ; mais c'est une troisième lecture des écritures, à côté de `slot_writers` (qui voit toutes les positions depuis ADR-38) et de `collect_setter_calls`. `redundant_set_state.rs` figure pour cette raison dans la liste du cliquet « walk-free » (section 17.10).

18.5 setter-in-render : écrire pendant le rendu

Rappel React : mise à jour pendant le rendu

Un setter appelé *pendant* le rendu d'un composant planifie un nouveau rendu de ce même composant avant même le commit. Inconditionnel, l'appel se répète à chaque passe : React interrompt la boucle par l'erreur « Too many re-renders » au-delà de 25 passes (`RE_RENDER_LIMIT` de `ReactFiberHooks.js`, constante depuis les premières versions des Hooks, hors du dépôt `reactant-analyzer` et non vérifiée sur le code source de React lui-même dans ce manuscrit). Un setter d'un *autre* composant (reçu en prop) appelé pendant le rendu déclenche l'avertissement « Cannot update a component while rendering a different component ». React documente pourtant une exception, l'idiome « ajuster l'état pendant le rendu » : `if (prev !== value) setPrev(value)`, dont la garde s'éteint dès que l'écriture a eu lieu, coûte un rendu supplémentaire et converge [Met26b].

La règle doit donc séparer trois cas : l'écriture certaine à chaque rendu (**ERROR**), l'écriture possible (**WARNING**), et l'écriture gardée qui converge (silence). Elle émet deux noms de diagnostic, **setter-in-render** pour un setter local et **cross-setter-in-render** pour le setter d'un parent.

18.5.1 Quels noms sont des setters

Extrait 18.17 – `src/rules/impls/setter_in_render.rs`, l. 63–97 — les setters locaux et étrangers

```

63     let local_setter_info: HashMap<Var, (HookLabel, Option<crate::ir::SourceRange>)> =
64         comp_result
65             .render_cfg
66             .blocks
67             .values()
68             .flat_map(|b| b.stmts.iter())
69             .filter_map(|stmt| {
70                 if let Stmt::Let {
71                     var,
72                     rhs: Expr::StateSetter(label),
73                     ..
74                 } = stmt
75                 {
76                     let span = comp_result
77                         .hook_calls
78                         .iter()
79                         .find(|c| c.label == *label)
80                         .and_then(|c| c.span);
81                     Some((var.clone(), (*label, span)))
82                 } else {
83                     None
84                 }
85             })
86             .collect();
87
88     // Cross-component setters: ComponentSetter-valued props, excluding self-references.
89     let cs_vars: HashMap<Var, crate::engine::setters::SetterProp> =
90         crate::rules::cross_component_setters(comp_result, component);
91
92     let mut all_setter_vars: HashSet<Var> = local_setter_info.keys().cloned().collect();
93     all_setter_vars.extend(cs_vars.keys().cloned());
94
95     if all_setter_vars.is_empty() {
96         return vec![];
97     }

```

Les setters locaux sont les seules liaisons directes `Let x = StateSetter(1)` du rendu, sans fermeture d'alias. Les setters étrangers viennent de `cross_component_setters` (`src/engine/setters.rs`, l. 432–440) : les props dont la valeur est un setter d'un autre composant, lue dans l'environnement des blocs (`SetterProp`, chapitre 14). Chaque `SetterProp` porte un booléen `must_write` : vrai si appeler la variable écrit prouvablement, faux pour une fonction qui ne fait que *capturer* un setter (ADR-38 §4). Sans aucun setter, la règle rend immédiatement.

18.5.2 Les sites et leur phase

Les appels sont collectés par `collect_setter_calls(render_cfg, all_setter_vars, 2)` : la marche des écritures du moteur, qui descend dans les `FnLit` passés en argument ou liés à une variable jusqu'à deux niveaux, et classe chaque site par phase (chapitre 14). La règle pose une seule question à cette phase, « l'écriture peut-elle s'exécuter pendant le passage du corps ? » :

Extrait 18.18 – `src/rules/impls/setter_in_render.rs`, l. 109–121 — une ligne `Handler` ou `Deferred` est une preuve de non-rendu

```

109     collect_setter_calls(&comp_result.render_cfg, &all_setter_vars, 2)
110         .into_iter()
111         .filter_map(|call| {
112             // The walk classified when this write runs, and this rule is
113             // the question "during the render pass?". A `Handler` or

```

```

114         // `Deferred` row is a *proof* of no – a user event or a known
115         // deferring registrar stands between the render pass and the
116         // write – so it is not this rule's finding. `Unknown` is  $\top$  and
117         // stays, because  $\top$  includes the render pass and dropping it
118         // would be the false negative #130 is about.
119         if !call.class.may_run_in_body() {
120             return None;
121         }

```

SetterCallPhase	exemple	may_run_in_body	effet sur la règle
Sync	setN(1) au rendu, wrap(setN(1))	vrai	retenu ; ERROR possible
Handler	listener enregistré, handler JSX réifié	faux	écarté (preuve)
Deferred	setTimeout(() => setN(1)), cleanup	faux	écarté (preuve)
Unknown	compose(() => setN(1)) sans résumé	vrai ($\top$)	retenu ; jamais ERROR

TABLE 18.3 – Les quatre phases d’un site et ce qu’en fait `setter-in-render` (src/engine/setters.rs, l. 56–79).

Écarter Unknown serait le faux négatif que corrige ADR-38 (#130) : $\top$ inclut le passage du rendu.

18.5.3 La preuve : deux must, puis la dominance

Extrait 18.19 – src/rules/impls/setter_in_render.rs, l. 147–162 — l’unique chemin vers l’ERROR

```

147         // Unconditional call  $\impl$  a `Certified` proof – the only path to
148         // Error, and it takes two musts. The phase must be `Sync`: a  $\top$ 
149         // row is never certain, whatever block it sits in, because the
150         // proof would be about the block, not about the timing. And
151         // calling the variable must provably write: a local helper that
152         // only *carries* a captured setter into a JSX handler writes
153         // nothing when called, and dominance cannot upgrade a may.
154         let certain_write = call.class == SetterCallPhase::Sync
155             && cs_vars.get(&call.var).is_none_or(|p| p.must_write);
156         let proof = certain_write
157             .then_some(call.block_id)
158             .flatten()
159             .and_then(|bid| match exit_dom.certify(bid) {
160                 MustResult::All(c) => Some(c),
161                 _ => None,
162             });

```

Trois conditions, toutes nécessaires. La phase doit être Sync : sur une ligne $\top$, une preuve de dominance porterait sur le bloc, pas sur le moment de l’écriture. La variable doit écrire à coup sûr : un setter local, ou un SetterProp dont `must_write` est vrai — la dominance prouve que l’appel a lieu, pas que l’écriture a lieu. Enfin le bloc de l’appel doit dominer toutes les sorties atteignables du CFG de rendu : `ExitDominance::certify` (liste 17.20) frappe alors un `Certified<DominatesAllExits>`. Un appel trouvé dans un `FnLit` imbriqué a `block_id = None` et ne peut jamais atteindre l’ERROR. La figure 18.1 montre la différence sur les deux programmes les plus simples.

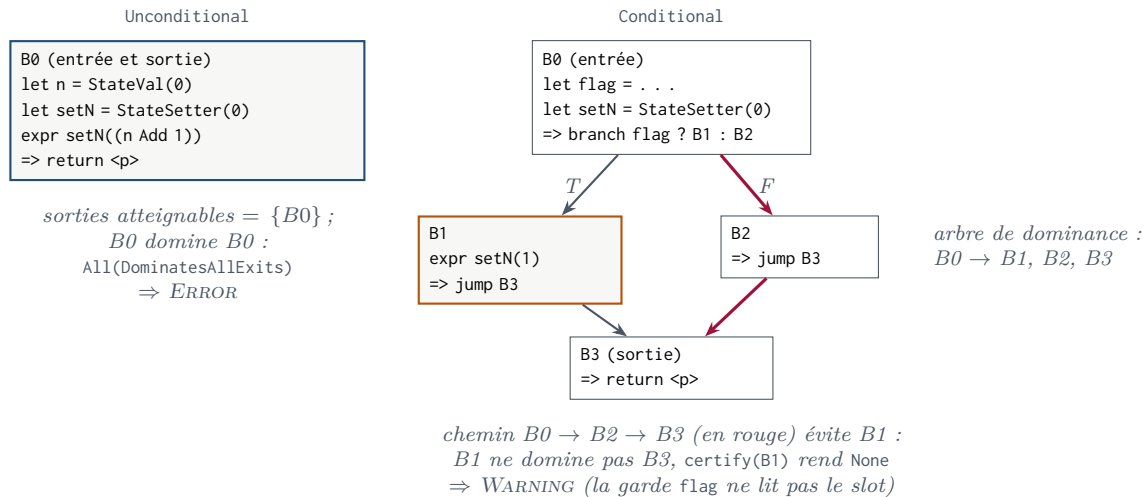


FIGURE 18.1 – CFG de rendu de Unconditional et Conditional (liste 18.21), tels que les imprime la sonde d'IR (élagués). La preuve d'ERROR demande que le bloc de l'appel domine *toutes* les sorties atteignables. Dans Conditional, B0 est le dominateur immédiat de B1, B2 et B3; B1 ne domine aucune sortie.

18.5.4 L'idiome d'ajustement : se taire sur une convergence prouvée

Sans preuve d'ERROR, un appel de setter local gardé peut encore être silencieux, si ses gardes meurent une fois la valeur écrite :

Extrait 18.20 – src/rules/impls/setter_in_render.rs, l. 164–183 — le silence de l'idiome « adjust during render »

```

164      // Sanctioned adjust-during-render idiom: conditional local call
165      // whose guards die once the written value is in the slot -
166      // converges after one extra render, no diagnostic.
167      if proof.is_none()
168          && let Some(bid) = call.block_id
169          && let Some(&(label, _)) = local_setter_info.get(&call.var)
170          && let Some(arg) = setter_call_arg(&comp_result.render_cfg, bid, &call.var)
171          && converges_once_written(
172              &comp_result.render_cfg,
173              bid,
174              &state_vals,
175              label,
176              &eval_in_exit_env(arg, comp_result),
177              Some(arg),
178              &comp_result.exit_env(),
179              &mut |e| eval_in_exit_env(e, comp_result),
180          )
181      {
182          return None;
183      }

```

`converges_once_written` (src/engine/guards.rs, l. 60–89) est la preuve « mono-site » étudiée au section 14.10 : elle remonte la chaîne de prédécesseurs uniques du bloc d'appel, collecte les gardes qui le dominent, et tente trois arguments de mort — le bras *valeur* (après l'écriture, le narrowing rend la garde $\perp$), le bras *relationnel* (la garde compare le slot à l'expression même qu'on y écrit) et le bras *membre* (le littéral écrit contredit la garde sur un champ). C'est une preuve, donc un silence légitime pour une règle de signalement. Le premier argument de

l'appel est retrouvé par `setter_call_arg` (`src/rules/impls/setter_in_render.rs`, l. 289–304), qui ne regarde que les instructions de premier niveau du bloc.

L’idiome n’est jamais appliqué au setter d’un parent, et le doc-comment dit pourquoi : « setting another component’s state during render is a runtime error regardless of guards » (`src/rules/impls/setter_in_render.rs`, l. 29–35).

18.5.5 Messages et ancrages

La formulation suit le verdict de phase (ADR-38 §3) : « called directly in the render body » est une phrase Sync, et une ligne $\top$ reçoit `unknown_phase_message` (`src/rules/impls/setter_in_render.rs`, l. 276–287), qui n’affirme que ce que la marche sait :

- local Sync : « setter ‘x’ called directly in the render body, move this call into a `useEffect` or an event handler » ;
- local $\top$: « setter ‘x’ is handed to a callee with no timing summary. If that callee runs it during render, this re-renders on every render » ;
- étranger Sync : « prop ‘x’ (a state setter of parent ‘P’) called during render of ‘C’, which triggers a parent re-render on every render » ;
- étranger $\top$: la même prop « is handed to a callee with no timing summary ».

Un diagnostic local porte le label du slot (`[hook:N]`) ; un diagnostic étranger porte la variable (`var:x`) et pas de label. La troisième branche du choix de message, « setter ‘x’ called in the render body » (`src/rules/impls/setter_in_render.rs`, l. 231–245), est inatteignable : `all_setter_vars` est exactement l’union des setters locaux et étrangers, et tout appel collecté relève donc d’une des deux premières branches.

Reste une réfutation propre aux setters étrangers (`src/rules/impls/setter_in_render.rs`, l. 122–146). La table des setters étrangers est un existentiel sur les points du programme ; si le nom n’est setter *que* par elle et que le bloc même de l’appel le réassigne, avant l’appel, à une fonction qui ne mentionne aucun setter, il n’y a pas d’écriture à rapporter (#119). Le test est délibérément syntaxique : à la jonction de `if (c) { f = onUpdate; } f()`, l’environnement contient le join de la flèche et du setter, et ce join perd la qualité de setter ; réfuter sur l’environnement supprimerait le vrai positif avec le faux.

18.5.6 Exemples

Extrait 18.21 – Six manières d’appeler un setter depuis le rendu (`/tmp/ch18/setter.tsx`)

```

1 import { useState } from "react";
2
3 declare function compose(f: () => void): () => void;
4 declare function wrap(x: unknown): void;
5
6 export function Unconditional() {
7   const [n, setN] = useState(0);
8   setN(n + 1);
9   return <p>{n}</p>;
10 }
11
12 export function Conditional({ flag }: { flag: boolean }) {
13   const [n, setN] = useState(0);
14   if (flag) setN(1);
15   return <p>{n}</p>;
16 }
17
```

```

18 export function Adjust({ open }: { open: boolean }) {
19   const [wasOpen, setWasOpen] = useState(open);
20   if (open !== wasOpen) setWasOpen(open);
21   return <p>{String(wasOpen)}</p>;
22 }
23
24 export function Handler() {
25   const [n, setN] = useState(0);
26   return <button onClick={() => setN(n + 1)}>{n}</button>;
27 }
28
29 export function Timing() {
30   const [n, setN] = useState(0);
31   setTimeout(() => setN(1), 0);
32   return <button onClick={compose(() => setN(2))}>{n}</button>;
33 }
34
35 export function Nested() {
36   const [n, setN] = useState(0);
37   wrap(setN(1));
38   return <p>{n}</p>;
39 }
40
41 function Child({ onMeasure }: { onMeasure: (w: number) => void }) {
42   onMeasure(42);
43   return <div />;
44 }
45
46 export function Parent() {
47   const [width, setWidth] = useState(0);
48   return <Child onMeasure={setWidth} />;
49 }

```

Sortie avec `-trace -rule setter-in-render -rule cross-setter-in-render -show-clean` :

```

Adjust (1 hooks) setter.tsx ✓
Child (0 hooks) setter.tsx
  error cross-setter-in-render var:onMeasure (line 42:2) prop `onMeasure` (a state setter of parent `Parent`)
    ↳ called during render of `Child`, which triggers a parent re-render on every render
    ↳ `onMeasure` is a state setter, so calling it writes state (line 42:2)
Conditional (1 hooks) setter.tsx
  warn setter-in-render [hook:0] (line 14:12) setter `setN` called directly in the render body, move this call
    ↳ into a useEffect or an event handler
    ↳ `setN` is a state setter, so calling it writes state (line 14:12)
Handler (2 hooks) setter.tsx ✓
Nested (1 hooks) setter.tsx
  error setter-in-render [hook:0] (line 37:2) setter `setN` called directly in the render body, move this call into
    ↳ a useEffect or an event handler
    ↳ `setN` is a state setter, so calling it writes state (line 37:2)
Parent (1 hooks) setter.tsx ✓
Timing (1 hooks) setter.tsx ✓
Unconditional (1 hooks) setter.tsx
  error setter-in-render [hook:0] (line 8:2) setter `setN` called directly in the render body, move this call into
    ↳ a useEffect or an event handler
    ↳ `setN` is a state setter, so calling it writes state (line 8:2)

```

- Unconditional : Sync, bloc d'entrée qui est aussi la seule sortie : ERROR.
- Conditional : le bloc de l'appel ne domine pas la sortie ; la garde flag est une prop qui ne lit pas le slot, aucun des trois bras de mort ne s'applique : WARNING.
- Adjust : bras relationnel. La garde `open !== wasOpen` compare le slot à l'expression même que l'on écrit ; une fois `wasOpen` égal à `open`, la garde est fausse. Silence.
- Handler : le handler JSX est extrait en hook (2 hooks), la phase est Handler : silence.

- **Nested** : `wrap(setN(1))` n'est pas en position d'instruction, mais depuis ADR-38 la marche visite toute position d'expression ; la ligne est `Sync` et le bloc domine la sortie : `ERROR`.
- **Child** : `onMeasure` est le setter lui-même, `must_write` vaut vrai : `ERROR cross-setter-in-render`, attribuée à l'enfant.
- **Timing** : silencieux. On attendait au moins un `WARNING` pour `compose(() => setN(2))`, un callee sans résumé. L'explication est un défaut du repli par variable, étudié ci-dessous.

Décision : ADR-38 : une écriture est une écriture où qu'elle soit écrite

Constat. La marche des écritures ne voyait que les appels en position d'instruction : `wrap(setN(1))` était muet, sans même un `analysis-limit`, et la règle ne lisait pas la phase que la marche calculait. **Décision.** Une seule traversée de toutes les positions d'expression, partagée par les trois canaux (écritures, appels, lectures) ; la règle lit `may_run_in_body` ; `Deferred` et `Unknown` deviennent deux variantes ; l'`ERROR` exige `Sync` et `must_write`. **Alternative refusée.** Garder la formulation certaine sur les lignes `T` : ce serait affirmer « la seule chose que la marche n'a pas pu établir ». **Mesure.** Sur 34 730 fichiers, 27 diagnostics ajoutés, tous des `WARNING` dont 25 de la classe `T` (`handleSubmit(onSubmit)...)`, aucune `ERROR` nouvelle.

18.5.7 Ce que la règle ne voit pas

Trois limites tiennent aux données que lit la règle, et non à son raisonnement.

Les alias de setter. `const s = setN` ; `s(n + 1)` au rendu est une boucle certaine, et la règle donne pourtant l'assurance `verified setter-in-render` : sa table des setters locaux ne ferme pas les alias (tableau 18.1), alors que `slot_writers` porte la ligne. L'exemple 14.13 du chapitre 14 en fait l'analyse complète ; le correctif naturel est de lire la table fermée, ou mieux la relation, au niveau central (exercice 18.5).

Le repli par variable. `collect_setter_calls` est la forme historique « une ligne par variable » de la marche ; elle replie les sites explicitement, à la fin :

Extrait 18.22 – `src/engine/setters.rs`, l. 136–157 — le repli : un site par variable, le plus petit dans l'ordre de `WalkClass`

```

136     let mut found = found.setters;
137     // Collapse to the historical one-row-per-var shape, preferring the sync
138     // site: its block id serves dominance, where any other class records
139     // `None`. The walk itself no longer collapses, so this is the one place
140     // the old granularity still exists – and the only consumer that wants it.
141     found.sort_by_key(|s| s.class); // WalkClass derives Ord with Sync first
142     let mut by_var: HashMap<Var, (Option<SourceRange>, Option<BlockId>, WalkClass)> =
143         HashMap::new();
144     for site in &found {
145         by_var
146             .entry(site.var.clone())
147             .or_insert((site.span, site.at.map(|(b, _)| b), site.class));
148     }
149     by_var
150         .into_iter()
151         .map(|(var, (span, block_id, class))| SetterCall {
152             var,
153             span,
154             block_id,
155             class: class.into(),
156         })
157         .collect()

```

L'ordre de WalkClass est celui de sa déclaration (src/engine/setters.rs, l. 1331–1344) : Sync < Deferred < Handler < Cleanup < Unknown. Le repli garde le premier site de la plus petite classe. Deux conséquences, observées au commit e67b10a :

Extrait 18.23 – Deux effets du repli (/tmp/ch18/timing.tsx et /tmp/ch18/tworender.tsx, élagués)

```

1 export function OnlyUnknown() {           // timing.tsx, l. 5
2   const [n, setN] = useState(0);
3   return <button onClick={compose(() => setN(2))}>{n}</button>;
4 }
5
6 export function DeferredThenUnknown() {    // timing.tsx, l. 10
7   const [n, setN] = useState(0);
8   setTimeout(() => setN(1), 0);
9   return <button onClick={compose(() => setN(2))}>{n}</button>;
10 }
11
12 export function TwoRenderCalls({ flag }: { flag: boolean }) { // tworender.tsx, l. 3
13   const [n, setN] = useState(0);
14   if (flag) setN(1);           // l. 5
15   setN(2);                     // l. 6
16   return <p>{n}</p>;
17 }

```

DeferredThenUnknown (1 hooks) timing.tsx ✓

OnlyUnknown (1 hooks) timing.tsx

warn setter-in-render [hook:0] (line 7:9) setter `setN` is handed to a callee with no timing summary. If that
 ↳ callee runs it during render, this re-renders on every render

TwoRenderCalls (1 hooks) tworender.tsx

warn setter-in-render [hook:0] (line 5:12) setter `setN` called directly in the render body, move this call into
 ↳ a useEffect or an event handler

(Deux exécutions, notes omises.) Dans DeferredThenUnknown, le site Deferred du setTimeout est plus petit que le site Unknown du compose : c'est lui que le repli garde, la règle l'écarte comme preuve de non-rendu, et le site T disparaît avec lui. Le composant est déclaré propre alors que OnlyUnknown, qui ne diffère que par l'absence du setTimeout, reçoit son WARNING. C'est un faux négatif de soundness pour une règle de signalement. Dans TwoRenderCalls, les deux sites sont Sync ; le repli garde le premier, conditionnel, et la règle rend un WARNING ligne 5 alors que l'appel de la ligne 6 domine la sortie et vaudrait une ERROR. Le composant est signalé, mais au mauvais niveau et au mauvais endroit. Les deux défauts ont la même cause : l'ordre du repli est un ordre de *synchronicité*, pas l'ordre de la question may_run_in_body, et une règle qui a besoin de tous les sites lit une projection qui n'en garde qu'un. slot_writers, lui, a une ligne par site (ADR-28) : la règle y trouverait les deux. Aucune issue ne suit ces deux cas au commit e67b10a.

L'initialiseur. useState(setN(1)) (liste 18.7) n'est vu que par lazy-init : l'initialiseur d'un hook n'est pas une instruction du CFG de rendu.

Piège

« La règle est muette, donc aucun setter ne s'exécute au rendu » est faux dans les trois cas ci-dessus, et la ligne verified n'y change rien : l'assurance dit seulement que check n'a rien trouvé sur ce qu'il lit. Quand on étend la règle, la bonne question n'est pas « quel cas ajouter ? » mais « quelle relation lire ? » — le principe « modulaire et général d'abord » de CLAUDE.md.

18.6 derived-state : un effet qui recopie un état

Rappel React : état dérivé

`useEffect(() => { setB(a * 2) }, [a])` maintient `b` égal à une fonction de `a`. Chaque changement de `a` coûte deux rendus : le premier affiche l'ancien `b`, périmé, l'effet écrit, le second affiche la bonne valeur. La documentation React range ce motif parmi les effets inutiles : une valeur calculable à partir de l'état se calcule pendant le rendu, éventuellement dans un `useMemo` [Met26b].

C'est une règle d'affirmation : elle dit « cet effet *toujours* recopie une expression sans appel de `a` », et ne parle que si chaque mot de cette phrase est établi.

18.6.1 Cinq conditions

Extrait 18.24 – `src/rules/impls/derived_state.rs`, l. 76–128 — les conditions de `derived-state`

```

76      // Dep array must be exactly 1 state variable: one visible element,
77      // and an arity that does not refute there being only one. An
78      // elision refutes it (`[a, ,]` declares two); a flattened spread
79      // cannot, so `[...tags]` stays in – withholding the finding there
80      // would be the false-negative direction.
81      if deps.len() != 1 || !deps.arity.may_be(1) {
82          continue;
83      }
84      let dep_var = match &deps.as_slice()[0] {
85          Expr::Var(v) if state_var_names.contains(v) => v.clone(),
86          _ => continue,
87      };
88
89      // Unconditional single call-free setter on all paths (the promoted
90      // must-forward). derived-state stays Warning, so it reads the fact
91      // off the verdict; only the `All` (all-paths) case qualifies.
92      let (setter_name, write_span) =
93          match must_setter_on_all_paths(body_cfg, &setter_vars, None) {
94              MustResult::All(proof) => (proof.evidence().var.clone(), proof.evidence().span),
95              _ => continue,
96          };
97
98      // Reject self-referential updates (e.g. `setX(x+1)`) accumulation, not derivation.
99      if let (Some(set_lbl), Some(dep_lbl)) = (
100          setter_label.get(&setter_name),
101          state_val_label.get(&dep_var),
102      ) && set_lbl == dep_lbl
103      {
104          continue;
105      }
106
107      // Nothing else may write this slot – read off the shared writer
108      // relation, not off a scan of the render body plus the other
109      // effect bodies (#92). Those two places are not where the missed
110      // writers live: a handler bound to a JSX prop, a `useCallback`
111      // body, a write inside a `.then()` the effect kicked off. The
112      // relation already carries all of them, and every consumer that
113      // claims "no other writer" should ask the same question.
114      //
115      // And if the setter left the component at all, the claim is not
116      // ours to make: something we cannot see may write the slot.
117      if let Some(&slot) = setter_label.get(&setter_name) {

```

```

118         if result.slot_written_outside(slot, WriterRegion::Effect(*eff_label)) {
119             continue;
120         }
121         if result.slot_setter_escapes(slot) {
122             continue;
123         }
124     } else if render_setters.contains(&setter_name) {
125         // No label for this name – fall back to the old render scan
126         // rather than claiming anything.
127         continue;
128     }

```

1. L'effet a une liste de deps d'*un seul* élément visible, dont l'arité ne réfute pas « un seul » (`deps.arity.may_be(1)`) : une éliision `[a, ,]` déclare deux éléments et réfute; un spread aplati ne réfute pas, et le garder est la direction sûre (chapitre 6, arité des deps).
2. Ce dep est une variable liée directement à un `StateVal` du rendu (`state_val_labels`, sans alias) : état vers état, et non prop vers état.
3. `must_setter_on_all_paths(body_cfg, setter_vars, None)` rend `All` : un seul setter, des arguments sans appel, sur tous les chemins du corps. C'est l'analyse `must` en avant du section 17.5.2 (`must_out[B]` est le meet des prédécesseurs, ou vrai si le bloc appelle). La règle lit le fait dans la preuve (`proof.evidence()`) mais reste au `WARNING`: le fait établit le motif, pas un défaut certain.
4. Le setter n'écrit pas le slot du dep : `setX(x + 1)` est une accumulation, laissée à `infinite-loop`.
5. Personne d'autre n'écrit le slot, et son setter ne s'échappe pas. Cette question est posée à la relation `slot_writers` (`slot_written_outside`) et à `slot_setter_escapes`, au lieu de l'ancien balayage du rendu et des autres effets, qui manquait les handlers, les `useCallback` et les continuations (#92) : « every consumer that claims "no other writer" should ask the same question ».

La branche `else if render_setters.contains(...)` (l. 124–128) n'est atteinte que si le setter retenu n'a pas de label dans `all_setter_labels`; comme `setter_vars` en est issu, le cas est pratiquement impossible — un filet hérité de l'ancien balayage. Le diagnostic est ancré sur l'effet, porte son label, et deux notes : `Step::Read` du dep, `Step::Write` du slot dérivé. Le message a une coquille (« expression of 'a' replace with », sans ponctuation, l. 133–134).

18.6.2 Exemples

Extrait 18.25 – Dérivations, vraies et fausses (/tmp/ch18/derived.tsx)

```

1 import { useState, useEffect } from "react";
2
3 export function Derived({ start }: { start: number }) {
4     const [a] = useState(start);
5     const [b, setB] = useState(0);
6     useEffect(() => {
7         setB(a * 2);
8     }, [a]);
9     return <p>{b}</p>;
10 }
11
12 export function WithCall({ start }: { start: number }) {
13     const [a] = useState(start);
14     const [b, setB] = useState(0);
15     useEffect(() => {
16         setB(Math.abs(a));

```

```

17   }, [a]);
18   return <p>{b}</p>;
19 }
20
21 export function Editable({ start }: { start: number }) {
22   const [a] = useState(start);
23   const [b, setB] = useState(0);
24   useEffect(() => {
25     setB(a * 2);
26   }, [a]);
27   return <button onClick={() => setB(0)}>{b}</button>;
28 }
29
30 export function Partial({ start }: { start: number }) {
31   const [a] = useState(start);
32   const [b, setB] = useState(0);
33   useEffect(() => {
34     if (a > 0) setB(a * 2);
35   }, [a]);
36   return <p>{b}</p>;
37 }
38
39 export function TwoDeps({ start }: { start: number }) {
40   const [a] = useState(start);
41   const [c] = useState(start);
42   const [b, setB] = useState(0);
43   useEffect(() => {
44     setB(a + c);
45   }, [a, c]);
46   return <p>{b}</p>;
47 }

```

```

Derived (3 hooks) derived.tsx
  warn derived-state [hook:2] (line 6:2) this effect always sets `setB` to a call-free expression of `a` replace
    ↳ with `useMemo` or compute during render
      ↳ `a` is read here [hook:0] (line 6:2)
      ↳ state `b` is written here [hook:2] (line 7:4)
Editable (4 hooks) derived.tsx ✓
Partial (3 hooks) derived.tsx ✓
TwoDeps (4 hooks) derived.tsx ✓
WithCall (3 hooks) derived.tsx ✓

```

(Commande sans filtre, avec `-trace -show-clean`.) Seul `Derived` satisfait les cinq conditions. `WithCall` échoue à la troisième (un appel dans l'argument : peut-être pas pur, peut-être pas dérivable), `Partial` aussi (l'appel n'est pas sur tous les chemins : `MustResult::Some`), `TwoDeps` à la première, `Editable` à la cinquième (un handler écrit aussi `b`). Les états `a` ne sont jamais écrits ici ; quand un handler écrit `a`, `a` est élargi par le point fixe et `b` avec lui, et le composant reçoit en plus un `WARNING infinite-loop` qui est un faux positif du bras point fixe (chapitre 17, discussion de `must_setter_on_all_paths` ; chapitre 19).

18.6.3 Un faux négatif par collision de labels

La cinquième condition interroge `slot_written_outside` :

Extrait 18.26 – `src/engine/setters.rs`, l. 2356–2364 — « quelqu'un d'autre écrit-il ce slot ? »

```

2356   pub fn slot_written_outside(
2357       &self,
2358       slot: crate::ir::types::HookLabel,
2359       except: crate::engine::setters::WriterRegion,

```

```

2360     ) -> bool {
2361         self.slot_writers
2362             .iter()
2363             .any(|w| w.slot == slot && w.region != except)
2364     }

```

La fonction compare le label du slot à celui de chaque ligne de `slot_writers`, sans regarder la colonne `owner`. Or une ligne étrangère — l'écriture d'un slot du parent par un setter reçu en prop — porte le label *du propriétaire* (ADR-42 §2), dans l'espace de labels du parent. Si ce numéro coïncide avec celui du slot local, la règle croit à un autre écrivain.

Extrait 18.27 – Le slot 1 du parent masque le slot 1 de l'enfant (/tmp/ch18/derived_foreign.tsx)

```

1  import { useState, useEffect } from "react";
2
3  export function Parent() {
4      const [p0] = useState(0);
5      const [p1, setP1] = useState(0);
6      return <Child onX={setP1} label={String(p0 + p1)} />;
7  }
8
9  function Child({ onX, label }: { onX: (n: number) => void; label: string }) {
10     const [a, setA] = useState(1);
11     const [b, setB] = useState(0);
12     useEffect(() => {
13         setB(a * 2);
14     }, [a]);
15     return <button onClick={() => { onX(1); setA(a + 1); }}>{b}{label}</button>;
16 }

```

```

Child (4 hooks) derived_foreign.tsx ✓
  verified derived-state no effect merely mirrors other state
Parent (2 hooks) derived_foreign.tsx ✓

```

(Commande sans `-all-roots`, avec `-rule derived-state -info -show-clean`.) Le handler de `Child` appelle `onX`, setter du slot 1 de `Parent`; la ligne correspondante a `slot = 1` et `owner = Some(Parent)`. `b` est le slot 1 de `Child` : la règle se tait, et publie même l'assurance `verified`. Le même composant sans `onX(1)`, ou avec le setter du slot 0 du parent, reçoit le `WARNING` (vérifié sur `/tmp/rs-verif/derived_foreign.tsx` et `derived_foreign2.tsx`). `redundant-set-state` ne souffre pas du défaut : elle filtre `w.owner.is_some()` (liste 18.15). Le correctif relève du « général d'abord » : filtrer les lignes étrangères dans `slot_written_outside` elle-même, dont `derived-state` est le seul appelant (exercice 18.6). Aucune issue ne suit ce FN au commit `e67b10a`.

Piège

Deux espaces de labels cohabitent dans `slot_writers` d'un composant : les lignes `owner = None` parlent des slots locaux, les lignes `owner = Some(p)` des slots de `p`. « Every native reader filters on owner », dit la documentation de la relation ; un lecteur qui l'oublie ne plante pas, il compare des numéros qui n'ont rien à voir.

18.7 state-mutation : muter l'état en place

Rappel React : identité et mutation

React compare l'ancienne et la nouvelle valeur d'un slot par `Object.is`. `items.push(x)`; `setItems(items)` passe au setter la *même* référence : React voit que rien n'a changé et saute le re-rendu, l'interface reste figée alors que les données ont bougé. La forme correcte crée une nouvelle référence (`setItems([...items, x])`), y compris dans un updater (`setItems(prev => [...prev, x])`). Muter un objet reçu en props écrit dans des données possédées par le parent. Les refs, elles, sont faites pour être mutées : `ref.current.push(x)` est légitime et ne planifie aucun rendu.

La règle a deux bras : le bras A (état muté puis passé tel quel à son setter, ERROR ou WARNING) et le bras B (objet de props muté, WARNING). Elle est la seule du chapitre à poursuivre une *identité de référence* à travers les liaisons et les portées, et elle fait ce travail elle-même : `state_mutation.rs` est dans la liste du cliquet « walk-free » (section 17.10).

18.7.1 Racines, sites et portées

Extrait 18.28 – `src/rules/impls/state_mutation.rs`, l. 37–65 — racine d'identité, site enregistré, portée lexicale

```
37 /// Where an object expression's reference identity roots.
38 #[derive(Debug, Clone, Copy, PartialEq, Eq)]
39 enum MutRoot {
40     State(HookLabel),
41     Props,
42     Other,
43 }
44
45 /// One recorded site (mutation or same-identity setter call).
46 #[derive(Debug, Clone)]
47 struct Site {
48     span: Option<SourceRange>,
49     /// Top-level container the site runs under (render body, one effect, one
50     /// handler...). A mutation and a set in the same container run on the same
51     /// trigger → the bug is certain (Error); across containers the pairing is
52     /// conservative (Warning).
53     container: usize,
54     /// Display text of the mutated object expression (`items`, `user.tags`).
55     desc: String,
56 }
57
58 /// One lexical scope of the chase: local `let`/`assign` bindings, plus names
59 /// that must NOT resolve outward (function params shadow their surroundings),
60 /// plus updater params known to BE the current slot value (`setX(p => ...)``).
61 struct Scope<'a> {
62     bindings: HashMap<&'a str, Vec<&'a Expr>>,
63     shadowed: HashSet<&'a str>,
64     param_roots: HashMap<&'a str, HookLabel>,
65 }
```

La question centrale est « à quoi cette expression est-elle identique ? », et la réponse est une `MutRoot` : un slot d'état, l'objet props, ou autre chose. Chaque site enregistré (une mutation ou un appel de setter à même identité) garde sa plage, un texte d'affichage et un *conteneur* : l'identifiant de la portée de déclenchement où il s'exécute. Le rendu est le conteneur 0, le hook d'indice `i` le conteneur `1 + i` :

Extrait 18.29 – `src/rules/impls/state_mutation.rs`, l. 384–407 — un conteneur par corps de déclenchement

```

384 // Container 0 = render body; hooks get 1 + their position.
385 let render_scope = Scope::from_cfg(&result.render_cfg);
386 collector.walk_cfg(&result.render_cfg, &mut vec![], 0);
387 for (i, hook) in result.hooks.iter().enumerate() {
388     let (body_cfg, params): (&CFG, &[Var]) = match hook {
389         HookEntry::Effect { body_cfg, .. }
390         | HookEntry::Memo { body_cfg, .. }
391         | HookEntry::Handler { body_cfg, .. } => (body_cfg, &[]),
392         HookEntry::Callback {
393             body_cfg, params, ..
394         } => (body_cfg, params.as_slice()),
395         _ => continue,
396     };
397     let mut scopes = vec![
398         Scope {
399             bindings: render_scope.bindings.clone(),
400             shadowed: HashSet::new(),
401             param_roots: HashMap::new(),
402         },
403         Scope::params(params),
404     ];
405     // Hook bodies close over the render scope; container = 1 + i.
406     collector.walk_cfg(body_cfg, &mut scopes, 1 + i);
407 }

```

Les corps d'effets, de memos, de handlers et de callbacks héritent des liaisons du rendu, qu'ils capturent. Un `FnLit` imbriqué reste dans le conteneur de son corps englobant (« a nested function runs on the same trigger family », `src/rules/impls/state_mutation.rs`, l. 283–290), avec ses paramètres en ombrage : un paramètre ne remonte jamais vers l'état ni vers les props.

Un *site de mutation* est une instruction `MemberWrite` (`obj.f = v`, `arr[i] = v`) ou un appel reconnu par `mutation_receiver` (`src/ir/expr.rs`, l. 143–158) : une méthode de la liste `MUTATING_METHODS` (`push`, `pop`, `shift`, `unshift`, `splice`, `sort`, `reverse`, `fill`, `copyWithin`, `add`, `delete`, `clear`, `set`, `src/ir/expr.rs`, l. 121–135) ou `Object.assign(target, ...)`, dont le receveur est le premier argument.

Décision : ADR-28 §4 : un seul reconnaisseur de sites de mutation

La liste des formes qui mutent est partagée par `state-mutation`, par la garde de pureté du Tier A et par la dépendance de rendu : « the one list every reader of "is this a mutation site" shares » (`src/ir/expr.rs`, l. 115–120). Ce qui reste propre à la règle est la question de la racine (« ce receveur est-il ce slot ? »). La migration s'est faite « diagnostics unchanged » (ADR-28, Consequences). Dupliquer la liste dans chaque lecteur, c'était préparer deux lectures d'un même fait qui dérivent.

Piège

Le commentaire source (`src/ir/expr.rs`, l. 115–120) attribue lui-même ce partage à « ADR-028 §2 », mais le texte cité vient de la §4 de cette ADR (« `ImpureBody` is a second derived verdict over the same column »), la §2 traitant d'une tout autre décision (une seule colonne `Updater`, partagée entre le classifieur fonctionnel/non-fonctionnel et le classifieur de pureté). Une dérive de numérotation dans le commentaire, au commit `e67b10a`.

18.7.2 La poursuite d'identité

Extrait 18.30 – src/rules/impls/state_mutation.rs, l. 112–167 — chase : remonter une référence jusqu'à sa racine

```

112     fn chase(&self, expr: &'a Expr, scopes: &[Scope<'a>], seen: &mut HashSet<&'a str>) -> MutRoot {
113         match expr {
114             Expr::TSAnnotated(inner) => self.chase(inner, scopes, seen),
115             // A path through `current` is ref semantics – mutation sanctioned.
116             Expr::FieldAccess { field, .. } if field == "current" => MutRoot::Other,
117             // A path through a DOM-only field is DOM manipulation.
118             Expr::FieldAccess { field, .. } if DOM_FIELDS.contains(&field.as_str()) => {
119                 MutRoot::Other
120             }
121             // A DOM-typed prop read directly off the props param
122             // (`props.canvas....`) is a DOM node handed down for imperative use.
123             Expr::FieldAccess { obj, field }
124                 if self.dom_props.contains(field.as_str())
125                 && matches!(obj.as_ref(), Expr::Var(v) if v == self.param) =>
126             {
127                 MutRoot::Other
128             }
129             // An element/field shares the container's identity for our
130             // purposes: mutating `items[0]` mutates what `items` holds.
131             Expr::FieldAccess { obj, .. } => self.chase(obj, scopes, seen),
132             Expr::IndexAccess { arr, .. } => self.chase(arr, scopes, seen),
133             Expr::StateVal(l) => MutRoot::State(*l),
134             Expr::Var(v) => {
135                 if !seen.insert(v.as_str()) {
136                     return MutRoot::Other;
137                 }
138                 for scope in scopes.iter().rev() {
139                     if let Some(l) = scope.param_roots.get(v.as_str()) {
140                         return MutRoot::State(*l);
141                     }
142                     if scope.shadowed.contains(v.as_str()) {
143                         return MutRoot::Other;
144                     }
145                     if let Some(rhss) = scope.bindings.get(v.as_str()) {
146                         let mut best = MutRoot::Other;
147                         for rhs in rhss {
148                             match self.chase(rhs, scopes, seen) {
149                                 s @ MutRoot::State(_) => return s,
150                                 MutRoot::Props => best = MutRoot::Props,
151                                 MutRoot::Other => {}
152                             }
153                         }
154                         return best;
155                     }
156                 }
157                 if let Some(l) = self.state_val_label.get(v.as_str()) {
158                     return MutRoot::State(*l);
159                 }
160                 if v == self.param {
161                     return MutRoot::Props;
162                 }
163                 MutRoot::Other
164             }
165             _ => MutRoot::Other,
166         }
167     }

```

La poursuite descend les accès : un champ ou un élément partage l'identité de son conteneur (muter `items[0]` mute ce que tient `items`). Trois exclusions sont posées *par construction*, avant

toute remontée : un chemin par `.current` (sémantique de ref), par `style`, `classList` ou `dataset` (manipulation du DOM, `DOM_FIELDS`), et une prop dont le type TypeScript déclaré est une interface DOM lue sur le paramètre `props`. Une variable se résout de la portée la plus intérieure vers l'extérieur : paramètre d'updater (`param_roots`, c'est le slot), nom ombré (autre chose), liaisons locales — la première qui remonte à un slot gagne, c'est une analyse may — puis les valeurs d'état du rendu, puis le paramètre `props` lui-même. Tout ce qui racine sur une allocation (`[...arr]`, `new Map(arr)`, un littéral) tombe dans `Other`. L'ensemble `seen` coupe les cycles de liaisons.

Un appel de `setter` est à même identité si son argument remonte au slot du `setter`, ou si son argument est un `updater` :

Extrait 18.31 – `src/rules/impls/state_mutation.rs`, l. 295–334 — l'updater : son paramètre *est* le slot

```

295  /// Functional updater `setX(p => ...)` : React passes the slot's current
296  /// value as `p`, so `p` IS the slot reference. A mutation of `p` is a
297  /// state mutation; returning `p` afterwards is a same-identity set.
298  fn walk_updater(
299      &mut self,
300      label: HookLabel,
301      params: &'a [Var],
302      body_cfg: &'a CFG,
303      scopes: &mut Vec<Scope<'a>>,
304      span: Option<SourceRange>,
305      container: usize,
306  ) {
307      let mut scope = Scope::params(params);
308      if let Some(p) = params.first() {
309          scope.shadowed.remove(p.as_str());
310          scope.param_roots.insert(p.as_str(), label);
311      }
312      scopes.push(scope);
313      self.walk_cfg(body_cfg, scopes, container);
314      // Same-identity set = some return path yields the parameter itself
315      // (possibly through a local alias – chase with the body's bindings).
316      scopes.push(Scope::from_cfg(body_cfg));
317      for block in body_cfg.blocks.values() {
318          if let Terminator::Return(e) = &block.term
319              && self.chase(e, scopes, &mut HashSet::new()) == MutRoot::State(label)
320          {
321              self.ident_sets.push((
322                  label,
323                  Site {
324                      span,
325                      container,
326                      desc: display_expr(e),
327                  },
328              ));
329              break;
330          }
331      }
332      scopes.pop();
333      scopes.pop();
334  }

```


18.7.3 Les deux bras et la preuve

Pour chaque slot muté qui a au moins un set de même identité, la règle calcule les conteneurs des mutations et ceux des sets, et appelle la primitive :

Extrait 18.32 – `src/rules/api/query.rs`, l. 1039–1057 — la primitive : un conteneur commun

```
1039 /// `All` iff a mutation site and a setter-call site share a container (the same
1040 /// event/trigger scope) – the certain `state-mutation` Error (ADR-021 §3);
1041 /// `None` otherwise (a MAY: mutation and set on different triggers).
1042 pub fn must_same_ref_mutation(
1043     mutation_containers: &HashSet<usize>,
1044     setter_containers: &HashSet<usize>,
1045 ) -> MustResult<SameRefMutation> {
1046     if mutation_containers
1047         .intersection(setter_containers)
1048         .next()
1049         .is_some()
1050     {
1051         // No provenance: the inputs are container ids, so neither the mutation
1052         // site nor the setter call reaches this primitive. The rule anchors it.
1053         MustResult::All(Certified::mint(SameRefMutation, Provenance::default()))
1054     } else {
1055         MustResult::None
1056     }
1057 }
```

Extrait 18.33 – `src/rules/impls/state_mutation.rs`, l. 433–440 — la seule voie vers l'ERROR de state-mutation

```
433 // same_trigger (mutation and set share a container) ==> certain Error;
434 // routed through the must-primitive so the proof is the only path there.
435 let mut_containers: HashSet<usize> = mut_sites.iter().map(|m| m.container).collect();
436 let set_containers: HashSet<usize> = sets.iter().map(|s| s.container).collect();
437 let proof = match must_same_ref_mutation(&mut_containers, &set_containers) {
438     MustResult::All(c) => Some(c),
439     _ => None,
440 };
```

Intersection non vide : ERROR ; sinon WARNING. Un diagnostic par slot, ancré sur le premier site de mutation dans l'ordre des positions, avec deux notes (`Step::Mutate`, `Step::Write` de valeur `SameAsCurrent`) ; la note d'écriture prend un set du même conteneur que ce site quand il y en a un (`src/rules/impls/state_mutation.rs`, l. 447–451). Le bras B émet un WARNING par site de mutation racinée dans les props, dédoublonné par position, sans label.

18.7.4 Exemples

Extrait 18.34 – Mutations d'état et de props (`/tmp/ch18/mutation.tsx`)

```
1 import { useState, useEffect, useRef } from "react";
2
3 export function PushThenSet() {
4     const [items, setItems] = useState<string[]>([]);
5     const add = (x: string) => {
6         items.push(x);
7         setItems(items);
8     };
9     return <button onClick={() => add("a")}>{items.length}</button>;
10 }
```

```

11
12 export function UpdaterMutates() {
13   const [items, setItems] = useState<string[]>([]);
14   return (
15     <button onClick={() => setItems((prev) => { prev.push("a"); return prev; })}>
16       {items.length}
17     </button>
18   );
19 }
20
21 export function CopyThenSet() {
22   const [items, setItems] = useState<string[]>([]);
23   return <button onClick={() => setItems([...items, "a"])}>{items.length}</button>;
24 }
25
26 export function CrossContainer() {
27   const [items, setItems] = useState<string[]>([]);
28   useEffect(() => {
29     items.push("x");
30   }, []);
31   return <button onClick={() => setItems(items)}>{items.length}</button>;
32 }
33
34 export function PropMutation(props: { user: { name: string } }) {
35   props.user.name = "x";
36   return <p>{props.user.name}</p>;
37 }
38
39 export function RefIsFine() {
40   const ref = useRef<string[]>([]);
41   ref.current.push("x");
42   return <p>{ref.current.length}</p>;
43 }
44
45 export function ExclusiveBranches({ flag }: { flag: boolean }) {
46   const [items, setItems] = useState<string[]>([]);
47   const onClick = () => {
48     if (flag) {
49       items.push("x");
50     } else {
51       setItems(items);
52     }
53   };
54   return <button onClick={onClick}>{items.length}</button>;
55 }

```

Sortie avec `-trace -rule state-mutation -show-clean` :

```

CopyThenSet (2 hooks) mutation.tsx ✓
CrossContainer (3 hooks) mutation.tsx
  warn state-mutation [hook:0] var:items (line 29:4) `items` is mutated in place and `setItems` is called with
    ↳ the same reference. React compares with `Object.is`, sees no change, and skips the re-render
    ↳ `items` is mutated in place here, so its reference identity is unchanged [hook:0] (line 29:4)
    ↳ the value written to state `items` is the value it already holds [hook:0]
ExclusiveBranches (2 hooks) mutation.tsx
  error state-mutation [hook:0] var:items (line 49:6) `items` is mutated in place and `setItems` is called with
    ↳ the same reference. React compares with `Object.is`, sees no change, and skips the re-render
    ↳ `items` is mutated in place here, so its reference identity is unchanged [hook:0] (line 49:6)
    ↳ the value written to state `items` is the value it already holds [hook:0] (line 51:6)
PropMutation (0 hooks) mutation.tsx
  warn state-mutation var:props.user (line 35:2) `props.user` roots in this component's props, so mutating it
    ↳ writes into an object owned by the parent; copy it before changing
    ↳ `props.user` is mutated in place here, so its reference identity is unchanged (line 35:2)
PushThenSet (2 hooks) mutation.tsx

```

```

error state-mutation [hook:0] var:items (line 6:4) `items` is mutated in place and `setItems` is called with
  ↳ the same reference. React compares with `Object.is`, sees no change, and skips the re-render
  ↳ `items` is mutated in place here, so its reference identity is unchanged [hook:0] (line 6:4)
  ↳ the value written to state `items` is the value it already holds [hook:0] (line 7:4)
RefIsFine (1 hooks) mutation.tsx ✓
UpdaterMutates (2 hooks) mutation.tsx
error state-mutation [hook:0] var:prev (line 15:48) `items` is mutated in place and `setItems` is called with
  ↳ the same reference. React compares with `Object.is`, sees no change, and skips the re-render
  ↳ `prev` is mutated in place here, so its reference identity is unchanged [hook:0] (line 15:48)
  ↳ the value written to state `items` is the value it already holds [hook:0]

```

- **PushThenSet** : mutation et set dans la flèche add, liée au rendu, donc tous deux dans le conteneur 0. **ERROR**.
- **UpdaterMutates** : prev est le slot (param_roots) ; le push est une mutation d'état, le return prev un set de même identité, dans le conteneur du handler. **ERROR**. La note d'écriture n'a pas de position : le corps concis () => setItems(...) est abaissé en terminateur Return, et walk_cfg parcourt les terminateurs sans plage (src/rules/impls/state_mutation.rs, l. 216–221). Même cause pour la note de CrossContainer.
- **CopyThenSet** : le spread est une allocation, Other. **Silence**.
- **CrossContainer** : mutation dans l'effet (conteneur 2), set dans le handler (conteneur 3) : pas d'intersection, **WARNING**. Le défaut est pourtant réel — le clic ne re-rendra pas.
- **PropMutation** : bras B, **WARNING** sans label.
- **RefIsFine** : le chemin passe par .current. **Silence**.
- **ExclusiveBranches** : **ERROR**, alors que la mutation et le set sont dans les deux branches exclusives d'un même handler et ne s'exécutent jamais dans le même clic.

Piège

La preuve « même conteneur » établit que la mutation et le set ont le même *déclencheur*, pas qu'ils s'exécutent sur un même *chemin*. Dans ExclusiveBranches, l'état est bel et bien corrompu (un premier clic mute sans re-rendre, un second passe la même référence et React l'ignore), mais la phrase du message, « 'setItems' is called with the same reference », n'est pas vraie sur le chemin de la mutation. Au regard de la doctrine « Error = preuve de toute la conclusion » (#142), le point reste ouvert : au commit e67b10a, aucune issue ne tranche ExclusiveBranches pour state-mutation. La colonne same_tick de slot_writers existe déjà et pourrait fonder une primitive qui exigerait la co-exécution — c'est l'objet de l'exercice 18.7 — mais elle est elle-même une approximation may dans un sens, dont un défaut de précision voisin (une indirection par helper local qui fait paraître co-exécutantes des branches exclusives) est suivi par #123.

Remarque

Le nom du setter affiché dans le message est choisi par setter_label.iter().find(...) sur une HashMap (src/rules/impls/state_mutation.rs, l. 442–446). Quand un slot a plusieurs noms de setter (un alias), le choix dépend de l'ordre d'itération de la table, qui varie d'une exécution à l'autre en Rust. state_slot_name documente précisément ce risque pour le nom du slot et prend le minimum (src/rules/helpers/mod.rs, l. 90–105) ; le nom du setter n'a pas reçu le même traitement. Le défaut se reproduit : sur un composant où setItems a trois alias s, t, u qui reçoivent tous items après un items.push (/tmp/ch18/mutalias.tsx), deux séries de douze exécutions de l'analyseur (chaque exécution est un nouveau processus, donc une nouvelle graine de hachage) ont chacune produit une répartition différente entre les quatre noms — par exemple t:4, s:3, u:4, setItems:1 sur une série, s:5, t:3, setItems:3, u:1 sur l'autre — sans qu'aucun nom ne domine de façon stable. La sortie n'est donc pas déterministe, ce qui gêne toute comparaison de corpus (chapitre 25).

Les limites documentées de la règle (`docs/limitations.md`) sont un FN sur un alias échappé (`ref.current = arr`, puis mutation par la `ref`, #23) et un FP sur une prop typée par une interface DOM *importée* (#38). La campagne « state » a relevé un autre FN : une valeur de module mutée puis copiée superficiellement (`filters.tags.push(t); setFilters({...filters})`) — la racine du set est une allocation, la mutation porte sur un objet partagé avec l'état initial.

18.8 frozen-initial-state : un état figé sur la première prop

Rappel React : initialiseur et remontage

`useState(props.value)` lit la prop au premier rendu seulement. Si le parent change ensuite `value` et que rien ne resynchronise le slot, l'enfant garde la valeur du montage : c'est le classique « mon champ ne se met pas à jour quand les props changent ». Le gel n'est observable que si l'instance survit au changement : changer la `key` d'un élément démonte l'ancienne instance et en monte une nouvelle, dont l'initialiseur relit la prop ; un rendu conditionnel démonte aussi. Un nom de prop comme `initialValue` ou `defaultTab` annonce l'idiotisme « non contrôlé avec valeur par défaut », où le gel est voulu [Met26b].

C'est la règle la plus stratifiée du chapitre : elle rend une `ERROR`, un `WARNING` ou une `INFO` selon une chaîne de faits dont un seul, le mouvement de la prop, exige l'analyse inter-composants. Le chapitre 17 (section 17.5.4) a présenté ses deux primitives, `classify_motion` et `must_frozen_seed`, comme l'exemple type d'une frappe « au point de connaissance » ; on regarde ici la règle entière.

18.8.1 Le seed et sa synchronisation : une relation du moteur

Le point de départ est la relation `slot_seeds` (section 14.7) : une ligne `SlotSeed` par chemin de prop qu'un initialiseur `useState` lit, avec un verdict de synchronisation et une colonne d'échappement du setter.

Extrait 18.35 – `src/engine/seeds.rs`, l. 34–47 — le verdict de synchronisation, `may` dans un sens

```
34 /// Whether anything visibly re-syncs a prop-seeded slot when the prop moves.
35 ///
36 /// **MAY-typed in one direction.** `Synced` is a claim the relation makes from
37 /// a write it saw; `NoneSeen` is the absence of one, which is not a proof that
38 /// no sync exists – a setter the component handed out could be called from
39 /// anywhere (see [SlotSeed::setter_escapes`]).
40 #[derive(Debug, Clone, Copy, PartialEq, Eq)]
41 pub enum SeedSync {
42     /// A render-time write, or an effect write that re-runs when this seed's
43     /// prop moves (its deps cover the seed path, or it declares none).
44     Synced,
45     /// No such write was seen.
46     NoneSeen,
47 }
```

Extrait 18.36 – `src/rules/impls/frozen_initial_state.rs`, l. 106–115 — pas de seed, ou un chemin de synchronisation : silence

```
106         let seeds: Vec<&SlotSeed> = comp.seeds_of(*label).collect();
107         if seeds.is_empty() {
108             continue;
109         }
110         // A render-time write, or an effect that re-runs when this prop
```

```
111 // moves: a sync path exists. Its quality is `derived-state`'s
112 // business, and the render-time case belongs to `setter-in-render`.
113 if seeds.iter().any(|s| s.sync == SeedSync::Synced) {
114     continue;
115 }
```

Synced signifie qu’une écriture de rendu, ou une écriture d’effet qui se ré-exécute quand la prop bouge (ses deps couvrent le chemin du seed, ou il n’en déclare pas), a été vue. La qualité de cette synchronisation n’est pas la question de la règle : c’est celle de **derived-state**, et l’écriture de rendu relève de **setter-in-render**. **NoneSeen** n’est qu’une absence, que la colonne **setter_escapes** qualifie.

Décision : ADR-31 : un pli promu au moteur

Constat. La règle possédait sa propre machinerie de seeds (chemins, normalisation vers le paramètre props, couverture par les deps, échappement), dont un second consommateur avait besoin. **Décision.** Déplacer ces plis dans `src/engine/seeds.rs`, sans les changer, et en faire une relation calculée à la convergence ; la moitié « effet » de la synchronisation se dérive de `slot_writers`, jamais d'un second balayage. Deux points d'attention : la moitié « rendu » lit la *phase* et non la région (un callback littéral du rendu est de région `Render` mais de phase `⊤`, et y voir une synchronisation supprimait un vrai gel), et l'échappement est une colonne, pas un verdict Synced — « a no-sync claim is certain only when the setter stayed home ». **Ce qui reste natif.** La preuve de mouvement, les strates `INFO` et la rétrogradation par le montage (#95).

18.8.2 La prop bouge-t-elle ?

Extrait 18.37 – `src/rules/impls/frozen_initial_state.rs`, l. 140–174 — classer le mouvement de chaque seed

```

140   for seed in &seeds {
141       let mut expr = Expr::Var(seed.path.root.clone());
142       for seg in &seed.path.segments {
143           expr = Expr::FieldAccess {
144               obj: Box::new(expr),
145               field: seg.clone(),
146           };
147       }
148       let val = eval_with_heap(&expr, comp);
149       match classify_motion(&val, result) {
150           Motion::Still => continue,
151           Motion::Proven(proof) => {
152               all_seed_named &= is_seed_named(&seed.path);
153               note_moving(seed, &mut moving_props);
154               if proven.is_none() {
155                   proven = Some((seed.path.to_string(), proof));
156               }
157           }
158           Motion::Unproven => {
159               all_seed_named &= is_seed_named(&seed.path);
160               note_moving(seed, &mut moving_props);
161               // Several members of one object can seed the same
162               // slot (`const input = action.settings.input`, then a
163               // dozen reads of it): name the handle they share.
164               unproven = Some(match &unproven {
165                   Some(prev) => prev.common_prefix(&seed.path),
166                   None => seed.path.clone(),

```

```

167         });
168     }
169 }
170 }
171 let unproven_path = unproven.map(|p| p.to_string());
172 if proven.is_none() && unproven_path.is_none() {
173     continue; // every seeding prop provably never changes
174 }

```

Pour chaque seed, la règle reconstruit l'expression du chemin et l'évalue avec le *tas convergé* (eval_with_heap, src/rules/impls/frozen_initial_state.rs, l. 325–336), pour qu'un props.a.b se résolve au lieu de tomber à $\top$. classify_motion rend alors l'un de trois verdicts (src/rules/api/query.rs, l. 839–890) :

- Proven(Certified<MovingFeeder>) si la partie référence est Versioned(S) et que, pour un slot (owner, 1) de S, le propriétaire est analysé et référence le setter de 1 : le prop est nourri par un état qui peut réellement bouger. C'est le seul endroit où ce jeton est frappé.
- Still si la valeur est Stable ou $\perp$, ou si tous les propriétaires sont analysés et qu'aucun ne référence le setter : un état React ne bouge que par son setter.
- Unproven dans tous les autres cas : un propriétaire non analysé, VersionedTop, une valeur $\top$ ou PerRender.

Une valeur n'est Versioned que si le parent a été analysé *avant* l'enfant et lui a transmis ses props (analyse descendante, chapitre 16). Sous -all-roots, un composant exporté est aussi analysé comme racine, avec des props $\top$: au mieux Unproven, donc au mieux WARNING.

Tous les seeds Still : silence. Sinon la règle retient le premier Proven et, pour les Unproven, le préfixe commun de leurs chemins (plusieurs membres d'un même objet nomment la poignée qu'ils partagent). Elle note aussi les noms de props par lesquels arrivent les seeds mouvants : ils servent au raisonnement sur le montage.

18.8.3 Le montage

Extrait 18.38 – src/rules/helpers/mount.rs, l. 44–67 — ce que les sites d'appel prouvent du montage

```

44 /// What the call sites prove about a consumer's mount lifetime, relative to
45 /// the state slot feeding one of its seeding props.
46 #[derive(Debug, Clone, Copy, PartialEq, Eq)]
47 pub enum MountCoupling {
48     /// Every call site looks like it re-seeds the consumer when a seeding prop
49     /// moves: the element carries a `key` built from that prop, or is rendered
50     /// only under a guard built from it – either way a change is expected to
51     /// arrive on a fresh instance, whose initializer reads it.
52     ///
53     /// Deliberately *not* a proof, so it downgrades rather than kills: a guard
54     /// moving between two truthy values keeps the child mounted
55     /// (`if (!data) return <Spinner/>` over a refetched object), and a `key`
56     /// holding an object stringifies to a constant. Both shapes really do
57     /// freeze, and the analyzer cannot tell them from the idiom.
58     Reseeds,
59     /// Every call site renders the consumer under a state guard, and every
60     /// scope that writes the feeding slot writes that guard's slot too: the
61     /// prop moves in the same commit that mounts or unmounts the consumer, so
62     /// no mounted instance need ever observe the change. Not a proof of
63     /// stillness – it only costs the Error tier its certainty.
64     WriterCoupled,

```

```

65     /// Nothing proven – a mounted instance may observe the change.
66     Free,
67 }

```

MountIndex::coupling (src/rules/helpers/mount.rs, l. 116–148) examine tous les sites JSX quiinstancient le composant. Si *chacun* porte une key ou une garde qui couvre chaque prop de seed, le couplage est Reseeds. Si chacun est gardé par un état du propriétaire que toute écriture du slot nourricier écrit aussi, il est WriterCoupled. Sinon, ou sans site visible, Free. La couverture n'est pas symétrique (src/rules/helpers/mount.rs, l. 152–174) : key={item} couvre un seed lu comme item.name, mais key={item.id} ne couvre pas un seed lu comme item, car un autre champ peut bouger seul. Ni Reseeds ni WriterCoupled ne sont des preuves que l'enfant ne survit pas au changement (une key objet se convertit en une chaîne constante) : ils *dégradent*, ils ne suppriment jamais.

18.8.4 La stratification

Extrait 18.39 – src/rules/impls/frozen_initial_state.rs, l. 195–206 — le niveau de base d'une preuve de mouvement

```

195     let (mut severity, message, proven_evidence) = match (&proven, &unproven_path) {
196         (Some((path, proof)), _) => {
197             let feeder = proof.evidence();
198             // `escaped`: something outside this component holds the
199             // setter – an unseen sync path may exist. `WriterCoupled`:
200             // the mount condition moves with the feeder, so no mounted
201             // instance need ever observe the change.
202             let severity = if escaped || mount == MountCoupling::WriterCoupled {
203                 Severity::Warning
204             } else {
205                 Severity::Error
206             };

```

Extrait 18.40 – src/rules/impls/frozen_initial_state.rs, l. 231–255 — les trois rétrogradations

```

231     // Seed-once intent named on every seeding prop (`initialValue`)
232     // downgrades one level: the pattern is idiomatic, the finding
233     // stays visible.
234     if all_seed_named {
235         severity = match severity {
236             Severity::Error => Severity::Warning,
237             _ => Severity::Info,
238         };
239     }
240
241     // Local slot never written (setter never even referenced –
242     // `const [{ snap }] = useState(...)`): the author never intended
243     // this state to move, it is a deliberate mount-time snapshot.
244     // Surfaced as advice only.
245     if !locally_written.contains(label) {
246         severity = Severity::Info;
247     }
248
249     // Every call site re-seeds on change (`key={seed}`, or a render
250     // guarded by the seed): the freeze is real but expected to arrive
251     // on a fresh instance. Advice only – and never a kill, since
252     // neither shape proves the child cannot survive the change.
253     if mount == MountCoupling::Reseeds {
254         severity = Severity::Info;

```

255

}

Un seed prouvé part de l'ERROR, sauf si le setter s'échappe (une synchronisation invisible est possible) ou si le montage est WriterCoupled; un seed seulement Unproven part du WARNING. Viennent ensuite trois rétrogradations, dans l'ordre : tous les seeds mouvants nommés initial* ou default* (is_seed_named teste le dernier segment du chemin, src/rules/impls/frozen_initial_state.rs, l. 60–66) font descendre d'un cran; un slot jamais écrit localement (const [snap] = useState(user), un instantané délibéré) tombe à l'INFO; un montage Reseeds aussi. La figure 18.2 résume le chemin de décision.

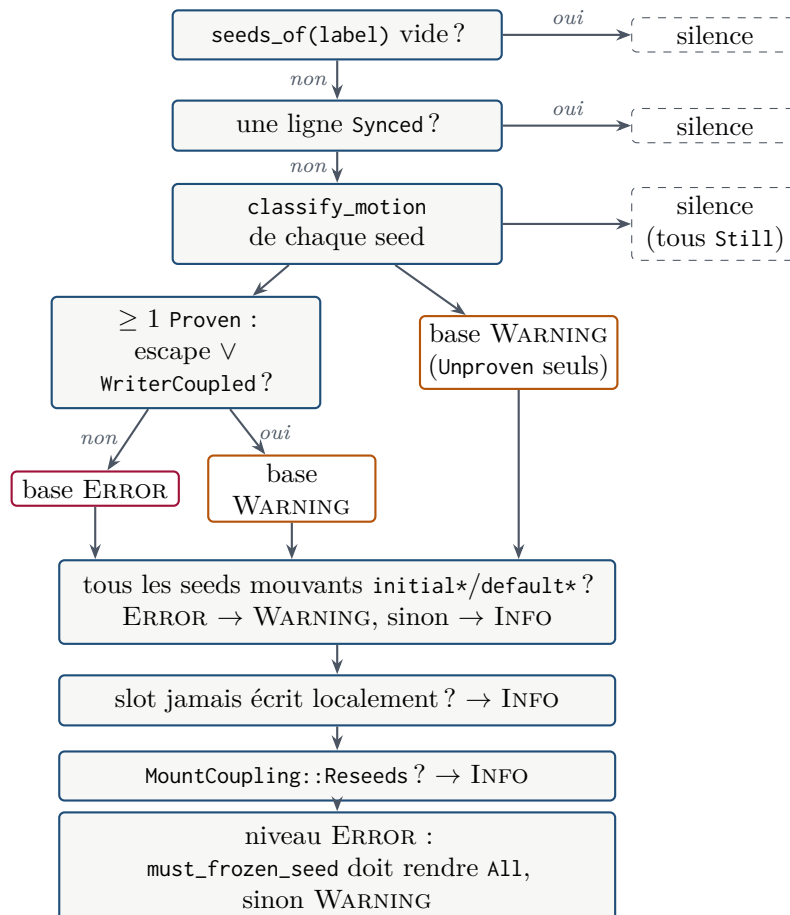


FIGURE 18.2 – Les strates de frozen-initial-state (src/rules/impls/frozen_initial_state.rs, l. 97–283). Les rétrogradations ne font que descendre; seule la dernière étape consomme le jeton de classify_motion. Le message ne dépend que de la branche Proven/Unproven, pas des rétrogradations.

Le calcul du niveau est fait deux fois, et c'est voulu. La stratification ci-dessus produit une valeur de Severity; mais une Severity::Error ne suffit pas à construire une ERROR (section 17.3) :

Extrait 18.41 – src/rules/impls/frozen_initial_state.rs, l. 263–285 — l'ERROR consomme le jeton, que must_frozen_seed rend ou dégrade

```

263 // The Error tier consumes the `Certified<MovingFeeder>` minted by
264 // `classify_motion` (the point of knowledge); `must_frozen_seed`
265 // demotes it when an idiomatic downgrade applies, so `severity ==
266 // Error`  $\impl$  `All`. Warning/Info are safe over-claims.
267 let feeder_proof = proven.map(|(_, proof)| proof);

```



```

268     let mut d = match (severity, feeder_proof) {
269         (Severity::Error, Some(proof)) => match must_frozen_seed(
270             proof,
271             escaped,
272             all_seed_named,
273             locally_written.contains(label),
274             mount,
275         ) {
276             MustResult::All(proof) => {
277                 Diagnostic::error("frozen-initial-state", proof, message)
278             }
279             _ => Diagnostic::warn("frozen-initial-state", message),
280         },
281         (Severity::Info, _) => Diagnostic::info("frozen-initial-state", message),
282         _ => Diagnostic::warn("frozen-initial-state", message),
283     }
284     .with_label(*label)
285     .with_var(primary_path.clone());

```

`must_frozen_seed` (liste 17.27) re-vérifie les quatre portes (pas d'échappement, pas tous nommés, écrit localement, montage Free) sur le jeton frappé par `classify_motion`, et rend `Some(into_evidence())` dès que l'une échoue. D'où l'invariant du commentaire : « `severity == Error` $\implies$ `All` ». Une erreur future dans la stratification coûterait un `WARNING`, jamais une `ERROR` sans preuve.

18.8.5 Exemples

Extrait 18.42 – Un parent, six enfants (/tmp/ch18/frozen.tsx)

```

1  import { useState, useEffect } from "react";
2
3  function Child({ user }: { user: { name: string } }) {
4      const [local, setLocal] = useState(user);
5      return <button onClick={() => setLocal({ name: "c" })}>{local.name}</button>;
6  }
7
8  function Keyed({ user }: { user: { name: string } }) {
9      const [local, setLocal] = useState(user);
10     return <button onClick={() => setLocal({ name: "k" })}>{local.name}</button>;
11 }
12
13 function Snapshot({ user }: { user: { name: string } }) {
14     const [snap] = useState(user);
15     return <p>{snap.name}</p>;
16 }
17
18 function Synced({ user }: { user: { name: string } }) {
19     const [local, setLocal] = useState(user);
20     useEffect(() => {
21         setLocal(user);
22     }, [user]);
23     return <button onClick={() => setLocal({ name: "s" })}>{local.name}</button>;
24 }
25
26 function Reseeded({ user }: { user: { name: string } }) {
27     const [local, setLocal] = useState(user);
28     return <button onClick={() => setLocal({ name: "r" })}>{local.name}</button>;
29 }
30

```

```

31 function Primitive({ title }: { title: string }) {
32   const [t, setT] = useState(title);
33   return <input value={t} onChange={(e) => setT(e.target.value)} />;
34 }
35
36 function Parent() {
37   const [user, setUser] = useState({ name: "a" });
38   const [title, setTitle] = useState("x");
39   return (
40     <div onClick={() => { setUser({ name: "b" }); setTitle("y"); }}>
41       <Child user={user} />
42       <Keyed key={user.name} user={user} />
43       <Reseeded key={user} user={user} />
44       <Snapshot user={user} />
45       <Synced user={user} />
46       <Primitive title={title} />
47     </div>
48   );
49 }
50
51 export function Root() {
52   return <Parent />;
53 }

```

Sortie *sans* `-all-roots`, avec `-trace -info -rule frozen-initial-state -show-clean` (notes élaguées sauf pour Child) :

```

Child (2 hooks) frozen.tsx
  error frozen-initial-state [hook:0] var:user (line 4:8) state `local` is seeded from `user`, which is fed by
    ↳ state `user` of `Parent` and changes. `useState` reads its initializer on the first render only and nothing
    ↳ here re-syncs it, so `local` stays frozen at the first `user` value
    ↳ `user` is read here [hook:0] (line 4:8)
    ↳ state `local` reads its initializer on the first render only, so later renders ignore it [hook:0] (line 4:8)
    ↳ state state `user` of `Parent` is written here
Keyed (2 hooks) frozen.tsx
  error frozen-initial-state [hook:0] var:user (line 9:8) state `local` is seeded from `user`, which is fed by
    ↳ state `user` of `Parent` and changes. ...
Parent (3 hooks) frozen.tsx ✓
Primitive (2 hooks) frozen.tsx
  warn frozen-initial-state [hook:0] var:title (line 32:8) state `t` is seeded from `title` and never re-synced.
    ↳ `useState` reads its initializer on the first render only, so if `title` changes, `t` keeps the mount-time
    ↳ value
Reseeded (2 hooks) frozen.tsx
  info frozen-initial-state [hook:0] var:user (line 27:8) state `local` is seeded from `user`, which is fed by
    ↳ state `user` of `Parent` and changes. ...
Snapshot (1 hooks) frozen.tsx
  info frozen-initial-state [hook:0] var:user (line 14:8) state `snap` is seeded from `user`, which is fed by
    ↳ state `user` of `Parent` and changes. ...
Synced (3 hooks) frozen.tsx ✓
  verified frozen-initial-state no state slot freezes a changing prop's first value

```

- Child : user vaut `Versioned({(Parent, 0)})` ; setUser est référencé dans Parent : Proven. Pas d'échappement, pas de nom de seed, slot écrit localement, montage Free : ERROR. La chaîne de témoins donne la lecture, la sémantique « une seule fois » et l'écriture qui fait bouger la source.
- Keyed : key={user.name} ne couvre pas le seed lu comme user (asymétrie de la couverture) : Free, ERROR.
- Reseeded : key={user} couvre le seed : Reseeds, INFO. Le message est identique à celui de l'ERROR.
- Snapshot : le setter n'est pas même déstructuré, le slot n'est jamais écrit : INFO.
- Synced : l'effet de deps [user] écrit le slot, ligne Synced : silence, avec assurance.

- **Primitive** : `title` est une chaîne. La conversion côté lecture de `StateVal` ne pose `Versioned` que sur la partie référence (section 10.5) ; une chaîne n'en a pas, et la valeur jointe `{ "x", "y" }` se projette en `PerRender` : `Unproven`, `WARNING`. C'est la limite documentée « props primitifs » (#25) : le gel est signalé, mais jamais au niveau `ERROR`.

Deux autres fichiers complètent la gradation. Avec `-all-roots` et des composants exportés sans parent (`/tmp/ch18/frozen_roots.tsx`) :

Extrait 18.43 – Props `⊤` (`/tmp/ch18/frozen_roots.tsx`, élagué de l'import)

```
3 export function FrozenWarn({ value }: { value: { n: number } }) {
4   const [v, setV] = useState(value);
5   return <button onClick={() => setV({ n: 1 })}>{v.n}</button>;
6 }
7
8 export function Seeded({ initialValue }: { initialValue: string }) {
9   const [v, setV] = useState(initialValue);
10  return <input value={v} onChange={(e) => setV(e.target.value)} />;
11 }
```

```
FrozenWarn (2 hooks) frozen_roots.tsx
warn frozen-initial-state [hook:0] var:value (line 4:8) state `v` is seeded from `value` and never re-synced.
  ↳ `useState` reads its initializer on the first render only, so if `value` changes, `v` keeps the mount-time
  ↳ value
Seeded (2 hooks) frozen_roots.tsx
info frozen-initial-state [hook:0] var:initialValue (line 9:8) state `v` is seeded from `initialValue` and
  ↳ never re-synced. ...
```

Props `⊤` : `Unproven`, `WARNING` ; nommé `initialValue` : `INFO`. Et un setter qui s'échappe vers un petit-enfant (`/tmp/ch18/frozen_escape.tsx`, `<Editor onChange={setLocal} />` dans l'enfant) rend `warn frozen-initial-state [hook:0] var:user (line 8:8)` avec le message « prouvé » : le mouvement est certain, l'absence de synchronisation ne l'est plus.

Piège

Trois lectures trompeuses de cette règle. (1) Le message ne change pas avec les rétrogradations : une `INFO Reseeded` dit « stays frozen » comme une `ERROR` ; c'est le niveau qui porte la nuance. (2) Un `WARNING` sous `-all-roots` ne dit pas que le parent ne bouge pas la prop : il dit que l'analyse n'a pas vu le parent. Pour obtenir l'`ERROR`, il faut que le parent soit atteint en analyse descendante (chapitre 16). (3) La dernière note affiche « state state 'user' of 'Parent' » : le display du `MovingFeeder` est déjà qualifié (« state 'user' of 'Parent' »), et le gabarit de `Step::Write` préfixe à nouveau « state ». Défaut cosmétique, déjà relevé au chapitre 17.

Les limites documentées de la règle sont un FN sur les props primitifs et les seeds qui passent par un memo (#25) et un FP sur un enfant toujours remonté par une machinerie que l'analyse ne voit pas (#136), que `Reseeds` ne rattrape que pour les formes `key` et garde.

18.9 Synthèse

Le tableau 18.4 rassemble les sept règles sur la même grille. Deux colonnes méritent l'attention : la preuve qui ouvre l'`ERROR`, qui est chaque fois une primitive must distincte, et la famille, qui dit comment lire un silence.

Trois leçons générales s'en dégagent.

règle	noms émis	niveaux	preuve de l'ERROR	données lues	applicable si
lazy-init	idem	E, W, I	must_init_calls_setter (un setter dans l'initialiseur)	initialiseurs State/Ref, setter_var_labels, registre de fonctions (témoin)	State ou Ref
unstable-context- value	idem	W	—	module_consts, rendu, tas convergé, is_unstable_reference_only	un provider prouvé
redundant-set- state	idem	W	—	store d'état, block_states, slot_writers (régions), escaping_slots	State et Effect
setter-in-render	idem, cross-setter- in-render	E, W	ExitDominance::certify + phase Sync + must_write	collect_setter_calls, cross_component_setters, converges_once_written	State
derived-state	idem	W	— (lit must_setter_on_all_paths)	deps, all_setter_labels, slot_writers, slot_setter_escapes	State et Effect
state-mutation	idem	E, W	must_same_ref_mutation (conteneur commun)	tous les corps, all_setter_labels, mutation_receiver	State
frozen-initial- state	idem	E, W, I	classify_motion puis must_frozen_seed	slot_seeds, tas convergé, MountIndex, may_written_slots	slot_seeds non vide

TABLE 18.4 – Les sept règles d'état de ce chapitre (E = ERROR, W = WARNING, I = INFO). La dernière colonne est la condition de `safe_check`.

Une Error est toujours une conjonction re-dérivée. `setter-in-render` conjugue la phase, la certitude de l'écriture et la dominance ; `frozen-initial-state` le mouvement prouvé et quatre portes ; `state-mutation` la même identité et le même déclencheur. Chaque fois, la règle calcule un niveau, puis doit présenter un jeton qu'elle n'a pas pu forger ; un désaccord entre les deux coûte un WARNING (invariant 17.1). Le cas `ExclusiveBranches` montre la limite de la méthode : le jeton prouve exactement ce que la primitive vérifie, et la question est alors de savoir si ce fait suffit à « toute la conclusion ».

Les faits `may` retiennent, les preuves font taire. Les règles de signalement se taisent sur des preuves : une phase `Handler` ou `Deferred`, une garde qui meurt, un `seed` `Still` ou `Synced`, un appel absent. Les règles d'affirmation se taisent sur des faits `may` qui leur ôtent le droit de parler : un slot contesté, un setter échappé, un second écrivain. Confondre les deux, c'est soit inventer des diagnostics, soit en perdre.

Les FN observés viennent des lectures, pas des raisonnements. Au commit e67b10a, aucun des défauts relevés dans ce chapitre n'est une erreur de logique dans une preuve. Tous tiennent à la donnée qu'une règle lit : une table sans alias, une projection « une ligne par variable », un filtre `owner` oublié, un temporaire du `lowering` pris pour une liaison d'auteur. Le tableau 18.5 les récapitule. C'est l'argument même d'ADR-42 : une relation promue au moteur, lue par tous, ne peut pas dériver d'une relecture locale ; les trois fichiers de ce chapitre encore dans la liste du cliquet « walk-free » (`redundant_set_state.rs`, `setter_in_render.rs`, `state_mutation.rs`) sont autant de promotions à faire.

Résumé du chapitre

- Sept règles parlent de l'état local : `lazy-init` (un appel dans l'initialiseur), `unstable-context-value` (une valeur de provider neuve à chaque rendu), `redundant-set-state` (écrire la valeur déjà présente), `setter-in-render` et `cross-setter-in-render` (écrire pendant le rendu), `derived-state` (un effet qui recopie un état), `state-mutation` (muter l'état ou les props en place),

règle	constat	nature	cause
lazy-init	<code>useState(c ? f() : 0), useState(c && f())</code> muets	FN (signalement)	temporaire __tN non poursuivi
unstable-context-value	<code><Ctx value></code> (React 19) muet	FN de précision	suffixe .Provider exigé
redundant-set-state	deux constantes objet distinctes	FP (WARNING)	Stable sans identité
redundant-set-state	« [in 2 components] ... Twice, Twice »	cosmétique	regroupement du rendu
setter-in-render	<code>alias const s = setN</code> au rendu	FN (ERROR)	table sans alias
setter-in-render	<code>setTimeout</code> puis <code>compose</code> : site T perdu	FN (WARNING)	repli par variable
setter-in-render	appel conditionnel puis inconditionnel	ERROR rendue WARNING	repli par variable
derived-state	setter de parent de même label	FN (WARNING)	owner non filtré
state-mutation	branches exclusives	ERROR discutable	preuve de conteneur
frozen-initial-state	prop primitive (#25)	plafond WARNING	Versioned sur la référence seule

TABLE 18.5 – Constats relevés au commit e67b10a sur les exemples de ce chapitre, hors limites déjà suivies par une issue (sauf #25, rappelée pour mémoire).

frozen-initial-state (un état figé sur la première valeur d'une prop).

- Deux familles : les règles de *signalement* ne se taisent que sur une preuve ; les règles d'*affirmation* ne parlent que sur un fait établi et retiennent sur des faits may (définition 18.1). Les trois règles d'affirmation plafonnent au WARNING.
- Quatre ERROR, quatre primitives : `must_init_calls_setter` ; `ExitDominance::certify` après les deux `must Sync` et `must_write` ; `must_same_ref_mutation` ; `classify_motion` consommé par `must_frozen_seed`. La règle calcule un niveau, le jeton le confirme ou le dégrade.
- Les données lues : les tables `setter_var_labels`, `state_val_labels`, `all_setter_labels` (tableau 18.1) ; les relations `slot_writers` et `slot_seeds` ; le store d'état, le tas convergé, `collect_setter_calls`, `converges_once_written`, `MountIndex`.
- Fichiers : les sept fichiers de règles de `src/rules/impls/` (un par règle, nommé d'après elle) ; `providers.rs`, `jsx.rs` et `mount.rs` dans `src/rules/helpers/` ; `setters.rs`, `seeds.rs` et `guards.rs` dans `src/engine/` ; les primitives de `src/rules/api/query.rs`. ADR : ADR-21, ADR-27, ADR-28, ADR-31, ADR-38, ADR-42.
- Constats (tableau 18.5) : les faux négatifs observés tiennent tous à la donnée lue — table sans alias, repli par variable, owner non filtré, temporaire du lowering —, ce qui plaide pour la lecture des relations du moteur.
- Le chapitre 19 traite les deux règles d'état restantes, `infinite-loop` et `state-lifted-too-high`, qui mobilisent le graphe de churn et la dépendance de rendu.

18.10 Exercices

Exercice 18.1: Lire la matrice de lazy-init

Pour chacun des initialiseurs suivants, prédire le verdict de `lazy-init` pour `useState` puis pour `useRef`, en justifiant par `classify_init_effect : parseInt(s), Math.max(a, compute()), new Date(), { at: Date.now() }, <Row />`. Vérifier ensuite avec l'analyseur.

Solution

`parseInt(s)` : PureCheap $\Rightarrow$ INFO / silence. `Math.max(a, compute())` : un PureCheap et un Other, `has_unknown` $\Rightarrow$ Unknown $\Rightarrow$ WARNING / INFO. `new Date()` : le callee `Date` d'un New n'est pas `now`, classé Unknown $\Rightarrow$ WARNING / INFO. `{ at: Date.now() }` : l'appel est dans un `ObjectLit`, déclencheur actif, PureCheap $\Rightarrow$ INFO / silence. `<Row />` : `CompApp` poussé par `collect_callees`, classé Other $\Rightarrow$ WARNING / INFO. Les dix verdicts sont confirmés par l'analyseur sur `/tmp/ch18/exo1.tsx` (trois WARNING, cinq INFO, deux silences pour les refs `parseInt` et `Date.now`).

Exercice 18.2: Le temporaire du lowering

Expliquer pourquoi `useState(c ? compute() : 0)` échappe à `lazy-init` alors que `useState(1 + compute())` est signalé, en vous appuyant sur l'impression d'IR de la section 18.2. Proposer une correction qui ne rouvre pas le faux positif `useMediaQuery`, et dire où elle doit vivre (règle, helper, lowering).

Solution

Le lowering abaisse la conditionnelle en diamant et l'initialiseur devient `var(__t0)` ; la règle refuse de poursuivre les liaisons. Le faux positif `useMediaQuery` vient d'un temporaire d'*inlining* qui cache une forme paresseuse. Une correction consiste à poursuivre les seules liaisons introduites par le lowering pour une expression de l'initialiseur (temporaires `__tN` dont toutes les affectations sont dans le diamant de cet initialiseur), en rassemblant les

callees de toutes les branches. Pour rester générale, la distinction « temporaire de lowering » / « liaison d’auteur ou d’inlining » devrait être un fait de l’IR (une marque sur la liaison) plutôt qu’un test de préfixe dans la règle.

Exercice 18.3: Égalité par stabilité

Montrer, avec `TwoConsts`, que `is_stable(arg) ∧ is_stable(store)` n’implique pas l’égalité pour la sorte référence, alors qu’elle l’implique pour un intervalle. Proposer une restriction qui élimine ce faux positif sans en créer d’autre, et discuter de ce qu’elle coûte, sachant que la règle est une règle d’affirmation de niveau `WARNING`.

Exercice 18.4: Réparer le repli

Pour `DeferredThenUnknown` et `TwoRenderCalls` (liste 18.23), écrire la liste des sites que produit la marche avant repli, avec leur classe et leur bloc. (a) Montrer que le premier cas est un FN au sens de l’invariant de soundness, et le second une perte de niveau. (b) Proposer deux corrections : changer l’ordre du repli, ou faire lire à la règle une ligne par site. Laquelle respecte le principe « modulaire et général d’abord » ? Quels autres appelants de `collect_setter_calls` seraient affectés par la première ?

Solution

(a) `DeferredThenUnknown` : deux sites (`setN`, `Deferred`, `None`) et (`setN`, `Unknown`, `None`) ; le repli garde `Deferred`, que la règle écarte ; le site `⊤`, qui peut s’exécuter au rendu, n’est jamais examiné : silence sur un défaut possible, FN. `TwoRenderCalls` : deux sites `Sync`, blocs `B1` (conditionnel) et `B3` (qui domine la sortie) ; le premier est gardé : `WARNING` au lieu d’`ERROR`, le composant est signalé. (b) Changer l’ordre (`Unknown` avant `Deferred`) corrige le premier cas mais pas le second, et modifie les verdicts de `derived-state`, `infinite-loop`, `stale-closure`, de `rules/helpers/mount.rs` et de l’entité `Tier A`, qui appellent aussi `collect_setter_calls`. Lire une ligne par site — `slot_writers` ou la marche non repliée — corrige les deux et fait sortir `setter_in_render.rs` de la liste du cliquet : c’est la correction au niveau central.

Exercice 18.5: Fermer les alias

Écrire un composant où `const s = setN` ; `s(n + 1)` est appelé au rendu, constater l’assurance `verified setter-in-render`, puis indiquer quelle table du tableau 18.1 la règle devrait lire. Écrire le test d’intégration qui verrouillerait la correction (un composant fautif, un composant sain de même forme).

Exercice 18.6: Filtrer owner au bon endroit

Reproduire le FN de `derived-state` de la liste 18.27. Expliquer pourquoi une ligne `SlotWriter` d’`owner = Some(Parent)` porte le label du parent. Écrire la correction dans `slot_written_outside` et vérifier qu’elle ne change pas le verdict de `Editable` (liste 18.25).

Solution

L’écriture `onX(1)` dans le handler de `Child` écrit le slot 1 de `Parent` ; la relation la range dans les écrivains de `Child` (c’est là qu’est le site) avec le label du propriétaire, seul label qui désigne ce slot (ADR-42 §2). La correction ajoute `w.owner.is_none()` au prédicat de `slot_written_outside`. Dans `Editable`, le second écrivain est un handler local (`owner = None`, région `Handler`), toujours compté : le verdict (silence) ne change pas.

Exercice 18.7: Co-exécution

Pour `ExclusiveBranches`, discuter si l'`ERROR` est justifiée au regard de « `Error` = preuve de toute la conclusion ». Proposer une primitive `must_*` plus forte qui exige que la mutation et le `set` s'exécutent sur un même chemin (par exemple : le bloc du `set` est dominé par celui de la mutation, dans le même corps), et évaluer les cas où elle ferait descendre une `ERROR` réelle au `WARNING`.

Exercice 18.8: Strates à la main

Pour chacun des enfants de la liste 18.42, suivre la figure 18.2 et retrouver le niveau observé. Puis prédire le niveau de `Child` (a) si la prop s'appelle `initialUser`, (b) si l'enfant passe `setLocal` à un petit-enfant, (c) si le fichier est analysé avec `-all-roots` et `Child` exporté sans être rendu par `Parent`.

Solution

(a) `Proven`, base `ERROR`, tous les seeds nommés $\Rightarrow$ `WARNING`; `must_frozen_seed` n'est pas consulté (le niveau n'est plus `ERROR`). (b) `setter_escapes` $\Rightarrow$ base `WARNING` (observé sur `frozen_escape.tsx`). (c) Props $\top$ en analyse de racine : `Unproven` $\Rightarrow$ `WARNING`.

Chapitre 19

Étude de cas : infinite-loop et state-lifted-too-high

Objectifs du chapitre.

- Connaître les quatre bras de `infinite-loop` — point fixe, cross-component, cycles du graphe de churn, self-churn —, l'ordre dans lequel ils s'exécutent, ce que chacun lit dans l'`AnalysisResult` et dans les relations du moteur, et la manière dont ils se dédoublonnent.
- Savoir pourquoi le widening est un *signal* et non une preuve (plafond WARNING, #144), et pourquoi seul un cycle tout-Must d'un seul composant, ou une écriture fraîche sur tous les chemins vers le slot exact d'un dep, atteint l'ERROR.
- Lire la garde $\forall$ -stable sur les deps, le filtre des écritures Handler, la revivification multi-écrivains et le re-déclenchement sensible aux champs (#90) comme autant de décisions de soundness ou de précision, chacune adossée à un test.
- Reproduire chaque verdict sur un programme court, lire les témoins produits et connaître les faux positifs assumés et les faux négatifs observés au commit e67b10a.
- Comprendre `state-lifted-too-high` : la dépendance de rendu, le foyer d'un slot, le principe « l'absence d'usage vaut preuve » et le plafond WARNING d'une règle de coût.

Prérequis : chapitre 13 (la boucle externe, `widen_trace`, `effect_setter_writes`) ; chapitre 15 (le graphe de churn, définition 15.1, les sites, la preuve multi-site et le plus petit point fixe des sites) ; chapitre 16 (le store partagé, la dépendance de rendu, définition 16.3, et `home_of`) ; chapitre 17 (le sceau de sévérité, définition 17.1, les primitives must et les témoins, définition 17.2) ; chapitre 18 (la grille de lecture d'une règle d'état, les tables de noms).

Le chapitre 18 a parcouru sept règles d'état en suivant une grille fixe. Ce chapitre applique la même grille aux deux règles restantes, mais en changeant d'échelle : `infinite-loop` est la règle la plus riche du catalogue, celle qui consomme le plus de produits du moteur — le point fixe et sa trace de widening, le store partagé, la relation des écrivains, celle des déclencheurs, le graphe de churn — et celle dont l'histoire (ADR-8, ADR-15, ADR-17, ADR-18, ADR-42) raconte le mieux comment le projet a appris à séparer un signal d'une preuve. `state-lifted-too-high`, plus jeune et plus courte, illustre la situation inverse : une règle de coût qui n'atteindra jamais l'ERROR, construite sur une analyse qui prouve des *absences*.

La progression suit celle du code de `src/rules/impls/infinite_loop.rs` (1667 lignes, tests compris), bras par bras et du plus simple au plus difficile : le compteur qui s'élargit, le setter d'un parent, les cycles du graphe, le self-churn. Chaque bras est introduit par un programme React court et sa sortie observée, puis lu ligne à ligne, puis confronté à ses cas limites. Les mécanismes du moteur que ces bras consomment ont été construits dans les chapitres précédents ; nous les rappelons d'une phrase et renvoyons à leur définition, sans les redémontrer.

Toutes les sorties ont été obtenues au commit e67b10a avec le binaire `target/debug/reactant`

sur des fichiers écrits pour ce chapitre sous /tmp/ch19/, ou sur les fixtures du dépôt. Sauf mention contraire, la commande est :

```
reactant check /tmp/ch19/<fichier>.tsx --all-roots --no-color --fail-on never
```

complétée de `-trace` (notes de témoin), `-info` (niveau INFO et assurances verified), `-show-clean`, `-verbose` (itérations et labels élargis) et `-rule <nom>` selon le besoin. Les numéros de ligne des sorties renvoient aux listings reproduits, qui portent les vrais numéros des fichiers (en-tête import compris).

19.1 Le bug visé et l'anatomie de la règle

19.1.1 Une boucle que React n'arrête pas

Rappel React : la boucle rendu → commit → effets → setState

Après avoir rendu un composant et commité le résultat, React exécute les effets `useEffect` dont un `dep` a changé : la comparaison est `Object.is`, élément par élément, et l'effet ré-exécute dès qu'un élément diffère (sémantique « ou ») ; un effet sans tableau ré-exécute après chaque commit. Un `setState` dans l'effet planifie un nouveau rendu. Si la nouvelle valeur fait à nouveau échouer `Object.is` sur un `dep` — un nombre qui a grandi, un objet fraîchement alloué —, l'effet repart, et ainsi de suite (la chronologie du chapitre 2). Contrairement à un setter appelé *pendant* le rendu, que React interrompt par « Too many re-renders » (setter-in-render, chapitre 18), une boucle portée par des effets passifs n'est pas interrompue. En développement, React compte les mises à jour planifiées pendant le vidage des effets passifs et, au-delà de 50 (`NESTED_PASSIVE_UPDATE_LIMIT` dans `ReactFiberWorkLoop.js`, branche main du dépôt de React consultée pour ce manuscrit), écrit dans la console « Maximum update depth exceeded. This can happen when a component calls `setState` inside `useEffect`, but `useEffect` either doesn't have a dependency array, or one of the dependencies changes on every render » — un `console.error`, pas une exception : la boucle continue. En production, rien : l'onglet gèle. C'est la classe « outage » du catalogue.

Deux mécanismes distincts font échouer `Object.is` d'un rendu au suivant, et ils demandent deux analyses différentes. Une *valeur qui grandit* (`setCount(count + 1)`) se voit dans le domaine numérique : l'intervalle du slot ne converge pas et le point fixe doit l'élargir. Une *référence recrée* (`setObj( ...obj )`) ne se voit pas ainsi : `PerRender` joint à `PerRender` donne `PerRender`, rien ne grandit, le point fixe converge au premier tour (chapitre 10). La certitude vient alors de la *structure* des `deps` et des écritures, c'est-à-dire des relations du moteur et du graphe de churn. La règle est la réunion de ces deux lectures, plus deux extensions : le setter d'un parent, et les cycles qui traversent plusieurs effets.

19.1.2 Les quatre bras

Le type et son doc-comment ouvrent le fichier :

Extrait 19.1 – `src/rules/impls/infinite_loop.rs`, l. 26–37 — le type de la règle et un doc-comment en retard sur le code

```
26 /// Fires when an effect causes an infinite render loop.
27 ///
28 /// - ``infinite-loop`` local state widens in fixpoint.
29 /// - ``cross-component-infinite-loop`` parent state widens via ComponentSetter prop.
30 ///
```

```

31 /// Effects with `deps: []` (mount-only) are excluded. Effects with all-unstable deps
32 /// are treated as no-deps. If parent not in results, cross fires as Warning.
33 pub struct InfiniteLoop;
34
35 impl InfiniteLoop {
36     const NAME: &'static str = "infinite-loop";
37 }

```

Piège

Ce doc-comment décrit la règle telle qu'elle était avant ADR-21 §5 et avant ADR-18. Deux affirmations sont fausses au commit e67b10a : « Effects with all-unstable deps are treated as no-deps » (le filtre actuel est le quantificateur inverse, $\forall$ -stable, section 19.2.5) et « If parent not in results, cross fires as Warning » (le code *saut* l'appel quand l'écriture partagée est $\perp$, section 19.3). Un lecteur qui apprend la règle par ses commentaires apprend une règle qui n'existe plus ; le manuscrit lit le code.

`safe_check` déclare la règle applicable dès qu'un composant a au moins un `useState` et un `useEffect` — une boucle rendu $\rightarrow$ effet $\rightarrow$ `setState` a besoin des deux :

Extrait 19.2 – `src/rules/impls/infinite_loop.rs`, l. 44–54 — l'applicabilité et l'assurance publiée sous `-info`

```

44 fn safe_check(&self, ctx: &RuleCtx) -> Option<crate::rules::SafeCheck> {
45     let (result, component) = (ctx.program(), ctx.component());
46     use crate::engine::HookKind;
47     // A render→effect→setState cycle needs both a state slot and an effect.
48     (crate::rules::has_hook_kind(result, component, HookKind::State)
49      && crate::rules::has_hook_kind(result, component, HookKind::Effect))
50     .then_some(crate::rules::SafeCheck {
51         rule: Self::NAME,
52         message: "no effect diverges into an infinite render loop",
53     })
54 }

```

La méthode `check` commence par construire ses tables de noms puis boucle sur les effets ; les deux premiers bras vivent dans cette boucle, les deux autres la suivent :

Extrait 19.3 – `src/rules/impls/infinite_loop.rs`, l. 56–83 — le prélude de `check` : tables de setters, retour anticipé, ensemble de déduplication

```

56 fn check(&self, ctx: &RuleCtx) -> Vec<Diagnostic> {
57     let (result, component) = (ctx.program(), ctx.component());
58     let comp_result = &result.components[&component];
59
60     // Authoritative setter map: base setters plus alias chains
61     // (`const s1 = setX`, `s2 = s1`) resolved across the render and every
62     // hook body – a setter called in an effect through an alias must still
63     // be attributed to its state slot (else a real loop goes unflagged).
64     let local_setter_labels: HashMap<Var, HookLabel> = all_setter_labels(comp_result);
65     let state_names = state_val_labels(&comp_result.render_cfg);
66
67     // ComponentSetter props, excluding self-references.
68     let cs_vars: HashMap<Var, crate::engine::setters::SetterProp> =
69         crate::rules::cross_component_setters(comp_result, component);
70
71     let mut all_setter_vars: HashSet<Var> = local_setter_labels.keys().cloned().collect();
72     all_setter_vars.extend(cs_vars.keys().cloned());
73
74     if all_setter_vars.is_empty() {

```

```

75     return vec![];
76 }
77
78 let render_fn_bindings: HashMap<Var, Arc<CFG>> =
79     collect_fn_bindings(&comp_result.render_cfg);
80 let mut diags = Vec::new();
81 // Effects already reported by the fixpoint/cross arms – the churn
82 // cycle arm skips them (same outage class, one report per effect).
83 let mut reported_effects: HashSet<HookLabel> = HashSet::new();

```

Trois tables sont lues. `all_setter_labels` (le socle commun du chapitre 18) est la table la plus large : les setters du rendu, fermés par alias sur le rendu *et* sur tous les corps de hooks, ce qui permet de suivre `const s = set0` jusque dans un effet. `cross_component_setters` (`src/engine/setters.rs`, l. 426–440) donne les props reconnus comme setters d’un *autre* composant, sous la forme d’un `SetterProp` (`src/engine/setters.rs`, l. 244–259) qui porte le composant propriétaire et le label. `collect_fn_bindings` enfin liste les fonctions liées dans le rendu, pour suivre `const cb = () => setN(n + 1)` appelé depuis l’effet. Si aucune variable de setter n’existe, ni locale ni prop, la règle rend `vec![]` avant même de consulter le graphe de churn ; un effet qui n’écrit rien ne porte de toute façon aucune arête.

Le tableau 19.1 résume les quatre bras ; la figure 19.1 montre le chemin d’un effet à travers eux. Le reste du chapitre les détaille dans cet ordre.

bras	condition de déclenchement	ce qu’il lit	niveau, nom émis
1. point fixe (intra)	setter local appelé dans le corps hors Handler ; slot dans <code>widen_trace</code> ; écriture rejouée $\perp$ ou non bornée	<code>widen_trace</code> , <code>effect_setter_writes</code> , <code>all_setter_labels</code> , marche des setters	WARNING au plus (#144) ; infinite-loop
2. cross-component	prop setter d’un autre composant appelé dans le corps ; écriture partagée ni $\perp$ ni bornée	<code>cross_component_setters</code> , <code>shared_state</code>	WARNING au plus ; cross-component-infinite-loop
3. graphe de churn	une arête d’un cycle du graphe portée par un effet de ce composant, effet non déjà signalé	<code>ChurnGraph::cycles</code> , <code>must_effect_cycle</code>	ERROR si cycle tout-Must mono-composant, sinon WARNING ; l’un ou l’autre nom selon <code>cross_component</code>
4. self-churn	arête pilotée par deps d’un slot du composant vers un slot du composant ; <code>self_slot</code> pour l’ERROR/WARNING, autre slot pour l’INFO	arêtes <code>!no_deps</code> du graphe, <code>slot_writers</code> , <code>must_on_all_paths</code>	ERROR (triple must), WARNING, INFO ; infinite-loop

TABLE 19.1 – Les quatre bras de `infinite-loop` (`src/rules/impls/infinite_loop.rs`, l. 56–241). Les bras 1 et 2 sont évalués dans la boucle sur les effets ; les bras 3 et 4 après. Les niveaux sont des plafonds : aucun bras n’atteint l’ERROR sans passer par une primitive `must`.

La déduplication se fait en deux temps, visibles à la fin de check :

Extrait 19.4 – `src/rules/impls/infinite_loop.rs`, l. 230–241 — les bras 3 et 4, après la boucle sur les effets

```

230 // -- F5b: multi-effect churn cycles (see churn_graph.rs) -----
231 // The graph is whole-program data: read it from the ctx cache, which
232 // builds it once for the run (issue #86).

```

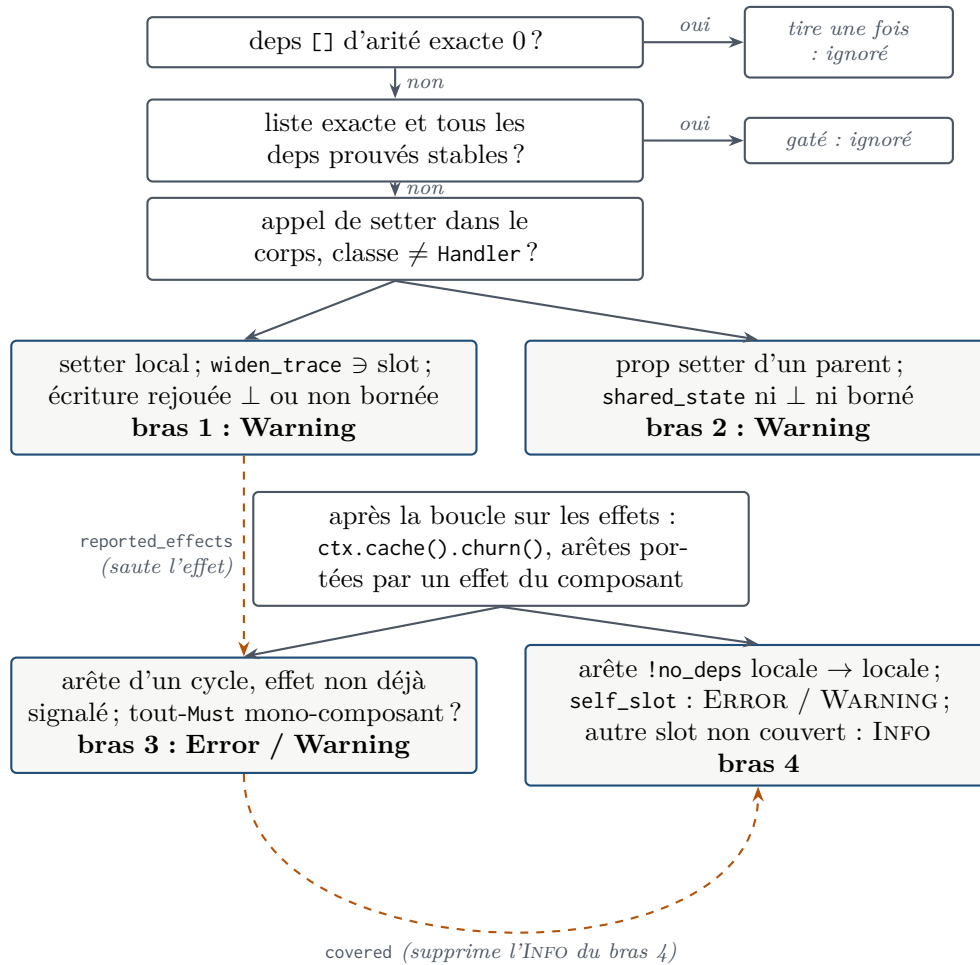


FIGURE 19.1 – Le chemin d'un effet à travers les quatre bras. Les deux premiers filtres (montage seul, $\forall$ -stable) ne concernent que les bras 1 et 2 ; le graphe de churn a appliqué ses propres filtres au moment de construire ses arêtes (chapitre 15). Les flèches pointillées sont les deux canaux de déduplication.

```

233     let graph = ctx.cache().churn();
234     let (cycle_diags, covered) =
235         check_multi_effect_cycles(graph, result, component, &reported_effects);
236     diags.extend(cycle_diags);
237     // Self-churn arm last: its Info branch skips writes a cycle covers.
238     diags.extend(check_object_churn(graph, result, component, &covered));
239
240     diags
241 }

```

`reported_effects` contient les labels d'effets signalés par les bras 1 et 2; le bras 3 les saute (« same outage class, one report per effect »). Le bras 3 rend en retour `covered`, les paires (effet, slot local) qu'un cycle signalé recouvre; le bras 4 s'en sert pour taire son INFO. Le graphe lui-même vient du cache programme (`ctx.cache().churn()`, le cache programme du chapitre 17) : il est construit une fois par exécution, jamais par composant — c'est la leçon de l'#86, où une construction par `check` rendait la phase des règles quadratique, et le test `churn_graph_is_built_once_per_program` (`src/rules/impls/infinite_loop.rs`, l. 634–684) la verrouille par un compteur thread-local.

19.2 Bras 1 : le widening comme signal

19.2.1 Trois compteurs

Exemple 19.1: Un compteur, un compteur borné, un compteur de montage

Extrait 19.5 – /tmp/ch19/counter.tsx

```

1  import { useState, useEffect } from "react";
2
3  export function Counter() {
4      const [count, setCount] = useState(0);
5      useEffect(() => {
6          setCount(count + 1);
7      }, [count]);
8      return <p>{count}</p>;
9  }
10
11 export function Bounded() {
12     const [count, setCount] = useState(0);
13     useEffect(() => {
14         if (count < 10) setCount(count + 1);
15     }, [count]);
16     return <p>{count}</p>;
17 }
18
19 export function MountOnly() {
20     const [count, setCount] = useState(0);
21     useEffect(() => {
22         setCount(count + 1);
23     }, []);
24     return <p>{count}</p>;
25 }

```

Concrètement, `Counter` boucle (chaque écriture change `count`, dep de l'effet), `Bounded` s'arrête à 10, `MountOnly` écrit une fois. Sortie (`-trace -verbose`, lignes `verified` omises) :

```
[verbose] Bounded: 3 iteration(s), widened: [0]
```

```

[verbose] Counter: 3 iteration(s), widened: [0]
[verbose] MountOnly: 3 iteration(s), widened: [0]
Counter (2 hooks) /tmp/ch19/counter.tsx
  warn infinite-loop [hook:0] (line 5:2) this effect keeps pushing state `count` (its deps do not provably
    ↳ gate it, so the effect can re-run every render) to new values on every run. Potential infinite render loop
    ↳ state `count` is written here [hook:1] (line 5:2)
    ↳ the abstract value of state `count` kept growing and was widened at iteration 3
MountOnly (2 hooks) /tmp/ch19/counter.tsx
  warn missing-deps [hook:1] var:count (line 21:2) `count` is used in this effect but not in its deps array,
    ↳ and it is recreated on every render
    ↳ `count` is read here [hook:1] (line 21:2)

```

Trois observations guident la section. D'abord, les *trois* slots sont élargis (widened: [0] partout) : le point fixe exécute chaque effet à chaque tour quels que soient ses deps (chapitre 13), donc MountOnly et Bounded font grandir count exactement comme Counter. Le widening seul ne distingue donc rien ; il faut deux filtres de plus. Ensuite, Bounded est absous par la valeur que son effet *écrit* : [1,10], bornée. Enfin MountOnly est absous par ses deps avant toute lecture du point fixe. Le niveau est WARNING, pas ERROR : nous y reviendrons (section 19.2.7).

19.2.2 La frise des itérations

La figure 19.2 rejoue les tours de la boucle externe pour Counter et Bounded, d'après les déroulés vérifiés du chapitre 13 (les compteurs emballé et gardé y sont déroulés tour par tour ; ce sont les mêmes programmes à la présence des deps près, qui ne changent pas les valeurs puisque le point fixe exécute chaque effet à chaque tour). Seuil de widening `widen_threshold = 3` (src/engine/fixpoint.rs, l. 54) ; seuils de valeur {0,1} pour Counter, {0,1,10} pour Bounded (la constante de la garde est collectée comme seuil, même chapitre).

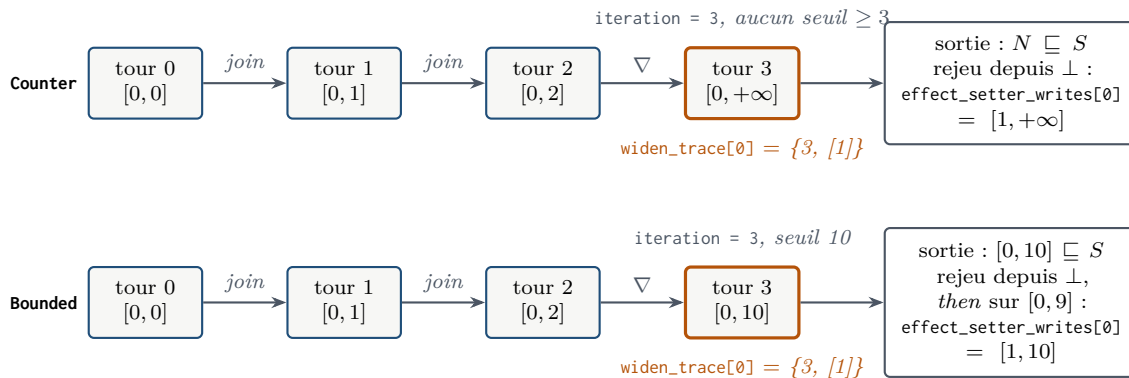


FIGURE 19.2 – Frise des itérations de la boucle externe pour Counter et Bounded. Les deux slots entrent dans `widen_trace` au même tour ; seule la valeur rejouée depuis $\perp$ après convergence les distingue : [1, +∞] est non bornée, [1, 10] est bornée.

19.2.3 Ce que le point fixe laisse à la règle

Deux champs de l'AnalysisResult portent le signal. Le premier est la trace de widening :

Extrait 19.6 – src/engine/analysis_result.rs, l. 18–28 — la provenance d'un widening forcé

```

18 /// Provenance of a forced widening (ADR-019): which fixpoint iteration gave
19 /// up on convergence for a slot, and which effects were writing it then.
20 /// Feeds the `infinite-loop` / `widening-info` witness chains (`Step::Widen`).
21 #[derive(Debug, Clone, Default, PartialEq, Eq)]

```

```

22 pub struct WidenEvent {
23     /// Outer fixpoint iteration at which the slot was widened (first time).
24     pub iteration: usize,
25     /// Effects whose pass wrote this slot during the widening iteration.
26     /// Empty when the growth came from render or handlers only.
27     pub writers: Vec<HookLabel>,
28 }

```

Elle est remplie par la boucle externe, à la fin de chaque tour, au test de convergence :

Extrait 19.7 – src/engine/fixpoint.rs, l. 485–529 — le test de convergence et l’enregistrement du widening, sur render $\sqcup$ effets seulement

```

485     // -- Convergence check -----
486     let new_state_incycle = state_from_render.join(&state_from_effects);
487     // Include cross-component state updates made by child effects/callbacks.
488     let external_updates = inter
489         .map(|i| i.shared_state.borrow().slice(comp_id))
490         .unwrap_or_else(StateStore::bottom);
491     let new_state = new_state_incycle
492         .join(&state_from_handlers)
493         .join(&external_updates);
494
495     if new_state.leq(&state) {
496         break;
497     }
498
499     iteration += 1;
500     if iteration >= 100 {
501         // Pathological input: force widening on all labels to guarantee convergence.
502         for label in state.labels() {
503             widen_trace
504                 .entry(label)
505                 .or_insert_with(|| crate::engine::WidenEvent {
506                     iteration,
507                     writers: slot_writers.get(&label).cloned().unwrap_or_default(),
508                 });
509         }
510         state = state.widen(&new_state);
511         break;
512     }
513
514     if iteration >= config.widen_threshold {
515         // widen_trace: render+effects only handler widening is not a bug.
516         // `or_insert_with` keeps the FIRST widening iteration (the most
517         // informative one for the witness chain).
518         for label in new_state_incycle.changed_labels(&state) {
519             widen_trace
520                 .entry(label)
521                 .or_insert_with(|| crate::engine::WidenEvent {
522                     iteration,
523                     writers: slot_writers.get(&label).cloned().unwrap_or_default(),
524                 });
525         }
526         state = state.widen_to(&new_state, &thresholds);
527     } else {
528         state = new_state;
529     }

```

Trois détails de cet extrait comptent pour la règle. D’abord, `changed_labels` est pris sur `new_state_incycle`, le join du rendu et des effets *sans* les handlers ni les écritures des enfants

(l. 486) : un slot que seul un clic fait grandir s'élargit dans le store (le `widen_to` porte sur `new_state` entier, l. 491–493) mais n'entre pas dans `widen_trace` — « handler widening is not a bug ». Ensuite, `or_insert_with` garde la *première* itération de widening, la plus informative pour le témoin. Enfin, le garde-fou des 100 itérations (l. 500–512) inscrit *tous* les labels du store dans la trace avant de forcer la convergence : sur une entrée pathologique, la première porte du bras 1 est ouverte pour chaque slot. Le second champ est la valeur écrite par les effets, recalculée après convergence en rejouant chaque corps d'effet depuis un store $\perp$ (chapitre 13, où ce jeu « pur » est lu ligne à ligne) : elle ne contient que ce que les setters écrivent, pas la valeur préexistante. Le prédicat qui la lit vit dans le domaine :

Extrait 19.8 – `src/domains/impls/state_value.rs`, l. 371–382 — `is_unbounded` : les trois formes de la croissance non bornée

```
371     /// True if this value represents unbounded growth in the fixpoint.
372     ///
373     /// Used by `InfiniteLoop` to distinguish a setter that writes a bounded value
374     /// (branch narrowing held the growth) from one that truly diverges:
375     /// - Infinite interval bounds → numeric counter without a binding guard
376     /// - Per-render reference slot → new object/function literal every render
377     /// - Residual  $\top$  → precision lost, conservatively unbounded
378     pub fn is_unbounded(&self) -> bool {
379         (!self.num.is_bottom() && (self.num.lo.is_infinite() || self.num.hi.is_infinite()))
380         || self.reference == Stability::PerRender
381         || self.other
382     }
```

Définition 19.1: Signal de divergence d'un slot

Dans un `AnalysisResult` convergé, un slot ℓ porte un *signal de divergence* lorsque $\ell \in \text{dom}(\text{widen_trace})$ et que `effect_setter_writes.get( $\ell$ )` est soit $\perp$ (aucune écriture rejouée n'a produit de valeur), soit non bornée au sens de `is_unbounded` : une borne d'intervalle infinie, une référence `PerRender`, ou un résidu $\top$.

Le signal est un *fait may* au sens de la définition 3.7 : un widening n'est qu'un abandon de précision, forcé par un seuil d'itérations, sur un store qui joint aussi ce que les handlers et les enfants écrivent. Il inclut les vraies boucles ; il n'inclut qu'elles seulement si aucune autre source de croissance n'existe.

19.2.4 Les deux portes

Le bras intra tient en une vingtaine de lignes. Après avoir collecté les appels de setter du corps (`collect_setter_calls_with_extra`, profondeur 1, avec les fonctions liées dans le rendu), il applique trois filtres :

Extrait 19.9 – `src/rules/impls/infinite_loop.rs`, l. 115–136 — le filtre `Handler` et les deux portes du bras intra

```
115     let calls =
116         collect_setter_calls_with_extra(body_cfg, &all_setter_vars, 1, &render_fn_bindings);
117
118     for call in &calls {
119         // A write only a registered event listener reaches does not
120         // close the loop: it needs a user event per iteration, which
121         // is the churn graph's own reason for excluding handlers.
122         // Before ADR-034 §4 the walk computed this class and the
123         // collapse threw it away, so `addEventListener('keydown', h)`
124         // read as an effect-body write (#93).
```

```

125         if call.class == SetterCallPhase::Handler {
126             continue;
127         }
128         if let Some(&state_label) = local_setter_labels.get(&call.var) {
129             // -- Intra -----
130             if !comp_result.widen_trace.contains_key(&state_label) {
131                 continue; // state didn't diverge → bounded
132             }
133             let writes = comp_result.effect_setter_writes.get(state_label);
134             if !writes.is_bottom_value() && !writes.is_unbounded() {
135                 continue; // write bounded → narrowing held the growth
136             }

```

La première porte demande que le slot soit dans `widen_trace`; la seconde, que l'écriture rejouée ne soit pas à la fois non- $\perp$ et bornée. Sur `Bounded`, la première passe (`widened: [0]`) et la seconde absout : `[1,10]` est bornée. Le commentaire du code le dit en une phrase, « *narrowing held the growth* » — le mot est celui de l'ADR-14, dont la partie *narrowing* n'a jamais été implémentée : c'est le *raffinement de garde* pendant la phase ascendante et le *widening* à seuils qui bornent l'écriture (chapitre 13).

Piège

La seconde porte traite $\perp$ comme « inconnu, ne pas absoudre », pas comme « rien n'est écrit ». La raison est un effet de bord du rejeu depuis $\perp$: un updater fonctionnel `setX(c => c + 1)` y lie `c` à la valeur du store $\perp$, donc écrit $\perp$ (chapitre 13). Absoudre sur $\perp$ serait un faux négatif certain sur la forme fonctionnelle de `Counter`. Le test unitaire `is_unbounded_requires_active_num_slot` (`src/domains/impls/state_value.rs`, l. 807) fixe la frontière : un intervalle $\perp$ n'est pas « non borné », c'est la règle qui en fait un cas à part.

Le filtre qui précède les portes est le filtre de *classe* : une écriture que seul un écouteur d'événement enregistré atteint (`addEventListener('keydown', h)` dans le corps de l'effet) a la classe `SetterCallPhase::Handler` et ne ferme pas la boucle, puisqu'il faut un événement utilisateur par itération. C'est la même raison qui exclut les handlers du graphe de *churn* (chapitre 15). Le commentaire rappelle l'§93 : avant ADR-34 §4, la marche calculait cette classe et le repli la jetait, si bien qu'un écouteur `keydown` se lisait comme une écriture de corps d'effet.

19.2.5 La garde $\forall$ -stable sur les deps

Avant les portes, deux filtres portent sur les *deps* de l'effet :

Extrait 19.10 – `src/rules/impls/infinite_loop.rs`, l. 96–113 — montage seul, puis suppression seulement si *tous* les *deps* sont prouvés stables

```

96         // Mount-only: fires once, no loop. Only an array the engine knows
97         // is empty says so.
98         if matches!(deps.list(), Some(d) if d.arity == Arity::Exact(0)) {
99             continue;
100         }
101         // A deps array gates the effect for good only when EVERY dep is
102         // provably stable (React re-runs on ANY changed dep – OR
103         // semantics). One stable dep among moving ones gates nothing, and
104         // a T/`Versioned` dep is never provably stable (ADR-021 §5: the
105         // shipped T FN, plus its all-vs-any quantifier sibling).
106         if let Some(dep_exprs) = deps.list()
107             // A  $\forall$  that suppresses must range over the whole list: a
108             // flattened spread hides elements that may well move.
109             && dep_exprs.arity.exact().is_some()
110             && all_deps_provably_stable(dep_exprs.as_slice(), comp_result)

```

```

111      {
112          continue;
113      }

```

Le premier reconnaît un tableau vide, mais seulement si le moteur *sait* qu'il est vide : `Arity::Exact(0)`. Un `[...xs]` dont l'aplatissement ne laisse rien n'a pas cette arité et n'est pas `mount-only`. Le second est le filtre qui a coûté un faux négatif au projet, et dont le quantificateur a été corrigé deux fois.

Décision : ADR-021 §5 : le dep $\top$ et son jumeau de quantificateur

Constat. La première version du filtre s'appuyait sur `is_unstable` (PerRender seulement) : un `dep prop`, évalué $\top$, se lisait « ne change pas », l'effet était sauté, et `useEffect(() => setN(n + 1), [data])` bouclait en silence — le faux négatif livré. Le premier correctif (`all_deps_may_change`) a réparé $\top$ mais gardé le mauvais quantificateur : il sautait dès qu'*un* `dep` était prouvé stable, ce qui rouvrait le FN une constante plus loin (`[label, data]`). **Décision.** React ré-exécute un effet dès qu'*un* `dep` change (sémantique « ou ») ; la suppression doit donc quantifier $\forall$: l'effet n'est ignoré que si *tous* ses `deps` sont prouvés stables, sur une liste d'arité exacte (un `spread` aplati cache des éléments qui peuvent bouger), $\top$ et `Versioned` n'étant jamais stables. **Alternatives refusées** : garder « un `dep` stable gate » (FN), ou ne plus filtrer du tout (FP massifs sur les effets à `deps` constants).

Le prédicat lui-même est dans les helpers, et son doc-comment raconte la même histoire :

Extrait 19.11 – `src/rules/helpers/mod.rs`, l. 225–235 — `all_deps_provably_stable` : $\forall$ -stable, clé sur `stability_verdict_of`

```

225 pub(in crate::rules) fn all_deps_provably_stable(
226     deps: &[Expr],
227     result: &AnalysisResult<StateValue>,
228 ) -> bool {
229     let exit_env = result.exit_env();
230     let mut eval = result.evaluator();
231     deps.iter().all(|dep| {
232         let val = eval.at(&exit_env, dep);
233         query::stability_verdict_of(&val).is_stable()
234     })
235 }

```

`stability_verdict_of` (chapitre 17) replie $\top$ du côté `may` ; `Versioned` n'est pas stable non plus. Deux tests d'intégration verrouillent les deux FN, `top_prop_dep_does_not_silence_self_write_loop` et `stable_dep_alongside_top_dep_does_not_gate_self_write_loop` (`tests/effect_cycles.rs`, l. 367–417) ; leur complément `all_stable_deps_gate_self_write_effect` (l. 419) vérifie que le filtre agit encore.

Exemple 19.2: Un `dep` stable, puis un `dep` stable à côté d'un `dep` $\top$

Extrait 19.12 – `/tmp/ch19/deps.tsx`

```

1 import { useState, useEffect } from "react";
2
3 export function StableGate({ label }: { label: string }) {
4     const [n, setN] = useState(0);
5     const limit = 10;
6     useEffect(() => {
7         setN(n + 1);
8     }, [limit]);

```

```

9   return <p>{label} {n}</p>;
10 }
11
12 export function MixedDeps({ data }: { data: unknown }) {
13   const [n, setN] = useState(0);
14   const limit = 10;
15   useEffect(() => {
16     setN(n + 1);
17   }, [limit, data]);
18   return <p>{n}</p>;
19 }

```

Sortie (`-trace -show-clean -rule infinite-loop`; sans le filtre `-rule`, `missing-deps` signale en plus la lecture de `n` dans les deux effets, et `StableGate` n'est plus propre) :

```

MixedDeps (2 hooks) /tmp/ch19/deps.tsx
  warn  infinite-loop [hook:0] (line 15:2) this effect keeps pushing state `n` (its deps do not provably gate
    ↳ it, so the effect can re-run every render) to new values on every run. Potential infinite render loop
    ↳ state `n` is written here [hook:1] (line 15:2)
    ↳ the abstract value of state `n` kept growing and was widened at iteration 3
StableGate (2 hooks) /tmp/ch19/deps.tsx ✓

```

`StableGate` : `limit` est une constante, l'effet tire une fois après le montage, la règle se tait (les deux slots sont pourtant élargis, comme au exemple 19.1). `MixedDeps` : `data` est un prop, `⊤` ; le quantificateur $\forall$ échoue, l'effet est examiné, le signal est là. Concrètement, un parent qui passe un `data` neuf à chaque rendu fait bien boucler `MixedDeps`.

19.2.6 Le diagnostic et ses témoins

Quand les deux portes passent, la règle émet un `WARNING` ancré sur le *slot* et sur la plage de l'effet, respectivement par `with_label(state_label)` et par `with_range` :

Extrait 19.13 – `src/rules/impls/infinite_loop.rs`, l. 143–156 — le `WARNING` du bras intra

```

143         let mut diag = Diagnostic::warn(
144             "infinite-loop",
145             format!(
146                 "this effect keeps pushing state {} to new values \
147                 on every run. Potential infinite render loop",
148                 state_slot_name(state_label, &state_names),
149                 deps_note
150             ),
151         )
152         .with_label(state_label);
153
154         if let Some(r) = comp_result.effect_info.get(eff_label).and_then(|i| i.span) {
155             diag = diag.with_range(r);
156         }

```

La chaîne `deps_note`, calculée juste avant (l. 138–142), ajoute la parenthèse « its deps do not provably gate it, so the effect can re-run every render » dès que `deps.is_declared()` (`src/ir/hooks.rs`, l. 145–150) : l'auteur a écrit un tableau, lisible ou non ; un effet sans tableau n'a pas besoin qu'on le dise. Suivent deux séries de notes. Pour chaque handler du composant qui appelle aussi un setter du slot, une note `Step::Handler` (`src/rules/impls/infinite_loop.rs`, l. 158–183) : l'événement qui relance la boucle. Puis l'historique du slot, pris directement dans la provenance du moteur :

Extrait 19.14 – `src/rules/api/witness.rs`, l. 529–560 — `slot_history` : un `Write` par effet écrivain au tour du widening, puis le `Widen`

```

529 pub fn slot_history(
530     result: &AnalysisResult<StateValue>,
531     slot: HookLabel,
532     name: &dyn Fn(HookLabel) -> String,
533 ) -> Vec<Note> {
534     let Some(event) = result.widen_trace.get(&slot) else {
535         return vec![];
536     };
537     let mut notes = Vec::new();
538     for writer in &event.writers {
539         let span = result.effect_info.get(writer).and_then(|i| i.span);
540         notes.push(note(
541             Step::Write {
542                 slot,
543                 value: ValueClass::Unknown,
544             },
545             Some(*writer),
546             span,
547             name,
548         ));
549     }
550     notes.push(note(
551         Step::Widen {
552             slot,
553             iteration: event.iteration as u32,
554         },
555         None,
556         None,
557         name,
558     ));
559     notes
560 }

```

C'est exactement ce qu'affiche la sortie de l'exemple 19.1 : « state 'count' is written here [hook:1] » (l'effet 1 était dans `writers`), puis « kept growing and was widened at iteration 3 ». La règle `widening-info` (`src/rules/impls/widening_info.rs`, 41 lignes) émet, sous `-info`, une INFO portant la même chaîne de témoins (`slot_history`) pour *chaque* label de `widen_trace`, sans porte : c'est le marqueur de perte de précision du chapitre 20, et la raison pour laquelle `Bounded` et `MountOnly` affichent une INFO là où `infinite-loop` se tait (`-info -rule widening-info -show-clean`) :

```

Bounded (2 hooks) /tmp/ch19/counter.tsx
info widening-info (line 13:2) state `count` kept changing during analysis and was approximated to converge, so
  ↳ findings that depend on it may be imprecise
  (2 trace step(s), rerun with --trace)
Counter (2 hooks) /tmp/ch19/counter.tsx
info widening-info (line 5:2) state `count` kept changing during analysis and was approximated to converge, so
  ↳ findings that depend on it may be imprecise
  (2 trace step(s), rerun with --trace)
MountOnly (2 hooks) /tmp/ch19/counter.tsx
info widening-info (line 21:2) state `count` kept changing during analysis and was approximated to converge, so
  ↳ findings that depend on it may be imprecise
  (2 trace step(s), rerun with --trace)

```

Piège

Dans « [hook:0] », le label désigne le *slot* pour les bras 1 et 4 (`with_label(state_label)`), mais l'*effet* pour les bras 2 et 3 (`with_label(*eff_label)`, `with_label(e.effect_label)`). Sur `Counter`, [hook:0] est `count` et la note [hook:1] est l'effet. La section 19.5 montre les deux

conventions côte à côte sur NoDeps et ObjChurn.

19.2.7 Pourquoi Warning et pas Error

Le bras 1 n'a aucune voie vers l'ERROR : le constructeur est `Diagnostic::warn`, et aucune primitive `must` ne frappe de jeton pour lui. L'#144 (ouverte, *precision-fn*) constate l'asymétrie : la divergence de *valeur* plafonne à WARNING alors que la divergence de *référence* (bras 4) atteint l'ERROR. La raison tient à ce que le signal prouve.

Décision : ADR-008 puis ADR-015 : le widening comme signal de boucle

Constat. L'ADR-8 pose le principe : « If `StateStore<StateValue>` widens on a label $\rightarrow$ potential infinite loop », avec une table de précision par type ; sa limite documentée est `useState(null)` sans annotation, dont le join `Null  $\sqcup$  Number` donnait $\top$ et convergeait immédiatement — un FN. **Décision.** L'ADR-15 remplace l'énumération `plate` par un *produit* de sortes disjointes (chapitre 10) et choisit la coercition `JS ToNumber(null) = 0` dans l'arithmétique : le compteur `useState(null)` non gardé s'élargit et est signalé ; `if (!user) setUser(...)` converge par raffinement de nullité. Ce que l'ADR-15 ne change pas : le signal reste un widening, c'est-à-dire un fait *may*.

Un widening dit « l'analyse n'a pas su borner cette valeur en trois tours ». Ce n'est pas « cette valeur change à chaque rendu concret ». Trois causes autres qu'une boucle le produisent : une garde que l'analyse n'exploite pas (Dead au chapitre 13 : une branche morte quand même exécutée), une croissance venue d'un *autre* slot, et la coercition ou le join qui perdent l'information. Le programme suivant montre la deuxième cause.

Exemple 19.3: Un miroir nourri par un handler : faux positif du bras 1

Extrait 19.15 – `/tmp/ch19/handlerfp.tsx`

```
1 import { useState, useEffect } from "react";
2
3 export function Mirror() {
4   const [a, setA] = useState(0);
5   const [b, setB] = useState(0);
6   useEffect(() => {
7     setB(a * 2);
8   }, [a]);
9   return <button onClick={() => setA(a + 1)}>{b}</button>;
10 }
```

```
[verbose] Mirror: 3 iteration(s), widened: [1]
Mirror (4 hooks) /tmp/ch19/handlerfp.tsx
warn  derived-state [hook:2] (line 6:2) this effect always sets `setB` to a call-free expression of `a`
  ↳ replace with `useMemo` or compute during render
    ↳ `a` is read here [hook:0] (line 6:2)
    ↳ state `b` is written here [hook:2] (line 7:4)
warn  infinite-loop [hook:1] (line 6:2) this effect keeps pushing state `b` (its deps do not provably gate
  ↳ it, so the effect can re-run every render) to new values on every run. Potential infinite render loop
    ↳ state `b` is written here [hook:2] (line 6:2)
    ↳ the abstract value of state `b` kept growing and was widened at iteration 3
```

Concrètement, `Mirror` ne boucle pas : l'effet ne re-tire qu'à chaque clic. Abstraitemment, le store joint les écritures des handlers (`new_state = new_state_incycle.join(&state_from_handlers)`, liste 19.7) : `a` grandit à cause du clic, `b = a * 2` grandit avec lui *par l'effet*, donc `b` change dans `new_state_incycle` et entre dans `widen_trace` ; au rejeu, `a` vaut déjà un intervalle élargi,

donc $a * 2$ est non borné. Les deux portes passent (le diagnostic observé l'atteste). C'est un faux positif toléré par l'invariant de soundness, au niveau WARNING ; le derived-state voisin est, lui, un vrai positif.

Ce FP est la démonstration par l'exemple de l'#144 : promouvoir le bras 1 en ERROR demanderait un fait de plus, par exemple « l'effet est re-déclenché par le slot qu'il écrit » (un trigger exact du slot écrit vers l'effet écrivain), et ce fait n'est pas dans le signal. L'exercice 19.3 y revient.

Piège

Le chemin CLI a un faux négatif que le chemin analyze_component n'a pas. Sur useEffect(() => setX((c) => c + 1);) (/tmp/ch19/updater.tsx), -verbose affiche « Updater: 2 iteration(s), widened: [] » et verified infinite-loop : aucun diagnostic, alors que le composant boucle. En intra, l'updater rejoué donne $\perp$ et la porte laisse passer ; en inter (analyze_program), le slot vaut $\top$ dès le deuxième tour par le store partagé, widen_trace reste vide et la première porte ferme : le composant écrit ses propres slots dans le SharedStateStore, qui les rejoint au tour suivant par external_updates (l. 488–493 de liste 19.7), et le join atteint $\top$ sans passer par le widening (chapitre 13). Aucune issue ne décrit ce cas au commit e67b10a.

19.3 Bras 2 : le setter d'un parent

Exemple 19.4: L'enfant réécrit l'état du parent

Extrait 19.16 – /tmp/ch19/cross.tsx

```
1 import { useState, useEffect } from "react";
2
3 export function Parent() {
4   const [data, setData] = useState({ n: 0 });
5   return <Child value={data} onUpdate={setData} />;
6 }
7
8 function Child({ value, onUpdate }: { value: { n: number }; onUpdate: (v: { n: number; seen: boolean })
   ↪ => void }) {
9   useEffect(() => { onUpdate({ n: value.n, seen: true }); }, [value]);
10  return <div />;
11 }
```

```
Child (1 hooks) /tmp/ch19/cross.tsx
  warn cross-component-infinite-loop [hook:0] (line 9:2) this effect calls `onUpdate`, a state setter of
    ↪ parent `Parent` (its deps do not provably gate it, so the effect can re-run every render). Parent re-
    ↪ renders → child re-renders → effect fires again: infinite loop
Parent (1 hooks) /tmp/ch19/cross.tsx ✓
```

Concrètement : onUpdate pose un objet neuf dans data, Parent re-rend, Child reçoit un value neuf, son effet repart. Le diagnostic est émis dans Child, ancré sur l'effet ([hook:0] est le label de l'effet, le seul hook de Child).

Le bras cross est la branche else if du même for :

Extrait 19.17 – src/rules/impls/infinite_loop.rs, l. 195–226 — le bras cross-component

```
195   } else if let Some(prop) = cs_vars.get(&call.var) {
196     // -- Cross-component -----
197     let (parent_comp, parent_label) = (prop.component, prop.label);
198     let shared_write = result.shared_state.get(parent_comp, parent_label);
```

```

199         if shared_write.is_bottom_value() {
200             continue; // setter not reached in semantic analysis
201         }
202         if !shared_write.is_unbounded() {
203             continue; // write is bounded → no divergence
204         }
205
206         let deps_note = if deps.is_declared() {
207             " (its deps do not provably gate it, so the effect can re-run every render)"
208         } else {
209             ""
210         };
211         let msg = format!(
212             "this effect calls `{}`, a state setter of parent `{}`. \
213             Parent re-renders → child re-renders → effect fires again: infinite loop",
214             call.var,
215             result.display_name(parent_comp),
216             deps_note
217         );
218         let mut diag = Diagnostic::warn("cross-component-infinite-loop", msg)
219             .with_label(*eff_label);
220
221         if let Some(r) = comp_result.effect_info.get(eff_label).and_then(|i| i.span) {
222             diag = diag.with_range(r);
223         }
224         reported_effects.insert(*eff_label);
225         diags.push(diag);
226     }

```

`cs_vars` associe la variable `onUpdate` à un `SetterProp {component: Parent, label: 0}` : la reconnaissance vient de l'analyse descendante, qui a passé à Child un `StateSetter` qualifié par son propriétaire (chapitre 16). La valeur lue est `result.shared_state.get(parent_comp, parent_label)`, la tranche du `SharedStateStore` où les enfants déposent ce qu'ils écrivent dans les slots de leurs parents. Deux `continue` : $\perp$ (le setter n'a jamais été atteint par l'analyse sémantique) et une écriture bornée. Ici l'écriture est un littéral objet, `PerRender`, donc `is_unbounded`.

Le niveau est `WARNING` sans voie vers l'`ERROR`, pour une raison structurelle que le bras 3 partagera : le `dep value` de l'effet enfant est *versionné* par le slot du parent, jamais le slot exact (un `prop` n'est pas un `StateVal`), donc le `must-rerun` est improuvable. Le filtre de `deps` de la section 19.2.5 s'applique tel quel : `value` évalue `Versioned((Parent, 0))`, qui n'est pas stable, donc l'effet n'est pas gâté.

Piège

L'ordre des bras décide de la formulation. L'effet de Child entre dans `reported_effects`; le bras 3, qui aurait aussi trouvé l'auto-boucle $(\text{Parent}, \text{data}) \rightarrow (\text{Parent}, \text{data})$ portée par cet effet (cycle `cross_component` de longueur 1), le saute. Le même programme, si le bras 2 se taisait, recevrait un `WARNING` du bras 3 avec la formulation « may store a fresh reference into state 'data' of 'Parent' ». Deux bras, un même plafond, deux textes. Le test `cross_component_object_churn_warns` (`tests/effect_cycles.rs`, l. 211–244), écrit sur ce même programme, ne verrouille que le nom, le nombre et le niveau (un `WARNING` qui nomme 'Parent') : les deux formulations le satisfont. Son commentaire attribue encore le diagnostic au graphe (« The Versioned dep gates the old cross arm »), ce qui était vrai tant que le filtre de `deps` tenait `Versioned` pour stable; depuis le quantificateur $\forall$ -stable (section 19.2.5), c'est le bras 2 qui parle, comme le montre la sortie observée.

La limite de ce bras est celle de l'analyse descendante. Un parent qui n'est pas atteint en phase 1 (parce qu'il n'est pas une racine, ou parce que l'élément `<Child/>` ne se résout pas) laisse

shared_state à $\perp$ pour son slot : continue, silence. C'est l'#20 (*precision-fn*, docs/limitations.md) : « cross-component rules need the parent to be reached top-down ». Le graphe de churn a la même dépendance, puisque ses lignes owner viennent de la même résolution.

19.4 Bras 3 : les cycles du graphe de churn

19.4.1 Quatre programmes, quatre verdicts

Le bras 3 ne calcule rien : il lit les cycles que le moteur a trouvés dans le graphe de churn (définition 15.1) et décide, pour chaque arête portée par un effet du composant courant, d'un niveau et d'un texte. Les quatre composants suivants couvrent les verdicts possibles ; le chapitre 15 a construit leurs graphes ligne à ligne, nous ne retenons ici que ce que la règle en fait.

Exemple 19.5: Cycle tout-Must, revivification, paire fetch-once, DAG

Extrait 19.18 – /tmp/ch19/cycles.tsx

```

1 import { useState, useEffect } from "react";
2
3 export function TwoEffects() {
4   const [a, setA] = useState({ n: 0 });
5   const [b, setB] = useState({ n: 0 });
6   useEffect(() => { setB({ from: a.n }); }, [a]);
7   useEffect(() => { setA({ from: b.n }); }, [b]);
8   return <div>{a.n + b.n}</div>;
9 }
10
11 export function MultiWriter() {
12   const [a, setA] = useState<object | null>(null);
13   const [b, setB] = useState<object | null>(null);
14   useEffect(() => { if (!b) setB({ src: 'e1' }); }, [a]);
15   useEffect(() => { setA({ src: 'e2' }); }, [b]);
16   useEffect(() => { setB(null); }, [a]);
17   return <div />;
18 }
19
20 export function FetchOncePair() {
21   const [a, setA] = useState<object | null>(null);
22   const [b, setB] = useState<object | null>(null);
23   useEffect(() => { if (!b) setB({ src: 'e1' }); }, [a]);
24   useEffect(() => { if (!a) setA({ src: 'e2' }); }, [b]);
25   return <div>{a && b ? 'ok' : 'loading'}</div>;
26 }
27
28 export function Dag() {
29   const [a, setA] = useState({ n: 0 });
30   const [b, setB] = useState({ n: 0 });
31   const [c, setC] = useState({ n: 0 });
32   useEffect(() => { setB({ from: a.n }); }, [a]);
33   useEffect(() => { setC({ from: b.n }); }, [b]);
34   return <div onClick={() => setA({ n: 1 })}>{c.n}</div>;
35 }

```

Sortie (-trace -info -show-clean -rule infinite-loop -rule cross-component-infinite-loop)
:

Dag (6 hooks) /tmp/ch19/cycles.tsx

```

info  infinite-loop [hook:1] (line 32:2) this effect depends on object state but freshly recreates state `b`
  ↳ outside its deps. No update cycle was found, but deps may be too imprecise to rule one out
  ↳ a fresh value is written to state `b` here [hook:3] (line 32:20)
info  infinite-loop [hook:2] (line 33:2) this effect depends on object state but freshly recreates state `c`
  ↳ outside its deps. No update cycle was found, but deps may be too imprecise to rule one out
  ↳ a fresh value is written to state `c` here [hook:4] (line 33:20)
FetchOncePair (4 hooks) /tmp/ch19/cycles.tsx ✓
verified infinite-loop no effect diverges into an infinite render loop
MultiWriter (5 hooks) /tmp/ch19/cycles.tsx
warn  infinite-loop [hook:2] (line 14:2) these effects may form a state-update cycle (`a` → `b` → `a`)
  ↳ where each step may store a fresh reference that re-runs the next effect: possible infinite render loop
  ↳ a fresh value is written to state `b` here [hook:2] (line 14:28)
  ↳ cycle continues: this effect freshly stores state `a` [hook:3] (line 15:20)
warn  infinite-loop [hook:3] (line 15:2) these effects may form a state-update cycle (`a` → `b` → `a`)
  ↳ where each step may store a fresh reference that re-runs the next effect: possible infinite render loop
  ↳ a fresh value is written to state `a` here [hook:3] (line 15:20)
  ↳ cycle continues: this effect freshly stores state `b` [hook:2] (line 14:28)
TwoEffects (4 hooks) /tmp/ch19/cycles.tsx
error infinite-loop [hook:2] (line 6:2) these effects form a state-update cycle (`a` → `b` → `a`) where
  ↳ each step stores a fresh reference that re-runs the next effect: infinite render loop
  ↳ a fresh value is written to state `b` here [hook:2] (line 6:20)
  ↳ cycle continues: this effect freshly stores state `a` [hook:3] (line 7:20)
error infinite-loop [hook:3] (line 7:2) these effects form a state-update cycle (`a` → `b` → `a`) where
  ↳ each step stores a fresh reference that re-runs the next effect: infinite render loop
  ↳ a fresh value is written to state `a` here [hook:3] (line 7:20)
  ↳ cycle continues: this effect freshly stores state `b` [hook:2] (line 6:20)

```

Les lignes de la relation `slot_writers` (imprimées par une petite sonde hors dépôt, écrite pour le chapitre 15, qui parcourt la relation ; colonne via élaguée) expliquent les forces des arêtes de la figure 19.3 :

```

=== TwoEffects
slot=0 setter=setA line=Some(7) region=Effect(3) phase=Effect block=Some(0) fresh=Fresh
slot=1 setter=setB line=Some(6) region=Effect(2) phase=Effect block=Some(0) fresh=Fresh
=== MultiWriter
slot=0 setter=setA line=Some(15) region=Effect(3) phase=Effect block=Some(0) fresh=Fresh
slot=1 setter=setB line=Some(14) region=Effect(2) phase=Effect block=Some(1) fresh=Fresh
slot=1 setter=setB line=Some(16) region=Effect(4) phase=Effect block=Some(0) fresh=Not
=== FetchOncePair
slot=0 setter=setA line=Some(24) region=Effect(3) phase=Effect block=Some(1) fresh=Fresh
slot=1 setter=setB line=Some(23) region=Effect(2) phase=Effect block=Some(1) fresh=Fresh
=== Dag
slot=0 setter=setA line=None region=Handler(5) phase=Handler block=Some(0) fresh=Fresh
slot=1 setter=setB line=Some(32) region=Effect(3) phase=Effect block=Some(0) fresh=Fresh
slot=2 setter=setC line=Some(33) region=Effect(4) phase=Effect block=Some(0) fresh=Fresh

```

Résumons ce que le moteur a décidé avant que la règle ne lise (chapitre 15). `TwoEffects` : deux écritures `Fresh` au bloc 0, deux deps exacts, deux arêtes `Must`, une SCC $\{a, b\}$ dans le sous-graphe `Must`. `MultiWriter` : l'écriture de `e2` est au bloc 1, sous `if (!b)`, donc `May` ; elle mourrait sous sa propre écriture, mais le site `setB(null)` de `e4` la ranime à chaque tour, `converges_under_all_writes` échoue, l'arête reste (`test_multi_writer_revival_is_warning`, `tests/effect_cycles.rs`, l. 184–209). `FetchOncePair` : chaque garde meurt sous toutes les écritures de son slot, les deux arêtes sont tuées, le graphe est vide. `Dag` : deux arêtes `Must` sans retour ; la ligne de `setA` a la phase `Handler` et n'est ni un site ni une arête.

Remarque

Le commentaire du test `multi_writer_revival_is_warning` dit du cycle $a \rightarrow b \rightarrow a$ qu'il est « real ». Une exécution concrète le nuance : quand `a` change, `e2` puis `e4` tirent dans le même vidage des effets ; l'écriture de `e4 (null)` est traitée la dernière, et si `b` valait déjà `null` l'état final de `b` ne change pas, `e3` ne repart pas. L'analyse ne modélise pas l'ordre des écritures d'un même vidage ; elle suppose que la revivification peut avoir lieu, et le `WARNING` est la réponse `sound` — au pire un faux positif, jamais un silence.

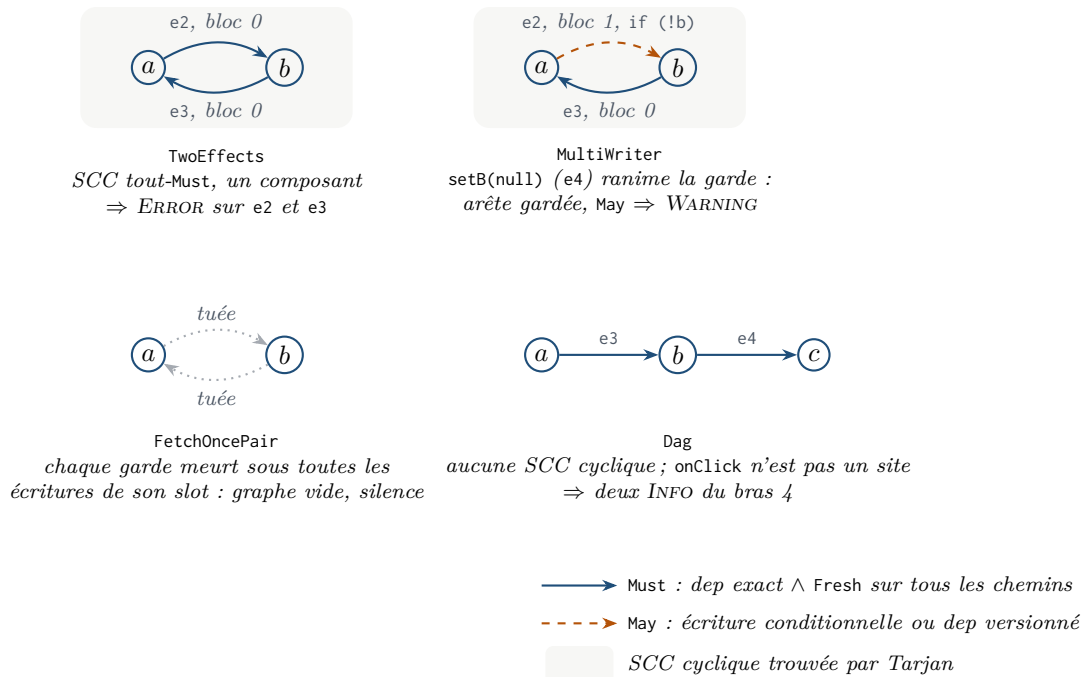


FIGURE 19.3 – Les graphes de churn des quatre composants de liste 19.18, avec le verdict du bras 3. *eN* est le label du hook porteur ; le bloc est celui de l'écriture dans le corps de l'effet (bloc 0 = entrée = tous les chemins). La ligne *setB(null)* de *MultiWriter* n'est pas fraîche et ne porte pas d'arête, mais elle est un *site* qui empêche le kill de l'arête $a \rightarrow b$.

19.4.2 Ce que la règle lit et ce qu'elle re-dérive

La boucle du bras 3 parcourt les cycles, puis les arêtes de chaque cycle portées par un effet du composant :

Extrait 19.19 – *src/rules/impls/infinite_loop.rs*, l. 267–296 — une arête du cycle, un diagnostic ; la preuve est re-dérivée

```

267     for cycle in cycles {
268         let cyc: Vec<&ChurnEdge> = cycle.edge_idx.iter().map(|&i| &edges[i]).collect();
269         let path = cycle_path(edges, cycle, component, result, &mut names);
270         for e in &cyc {
271             if e.component != component {
272                 continue;
273             }
274             if e.to.0 == component {
275                 covered.insert((e.effect_label, e.to.1));
276             }
277             if reported_effects.contains(&e.effect_label) {
278                 continue; // fixpoint/cross arm already flagged this effect
279             }
280
281             // Cross-component must-rerun is unprovable (prop deps are
282             // `Versioned`, never the exact slot)  $\rightarrow$  Warning ceiling. An all-must
283             // intra-component cycle mints the proof – the only path to Error.
284             let rule = if cycle.cross_component {
285                 "cross-component-infinite-loop"
286             } else {
287                 "infinite-loop"
288             };
289             let cycle_proof = match must_effect_cycle(edges, cycle) {

```

```

290         MustResult::All(c) => Some(c),
291         _ => None,
292     };
293
294     // The wording follows the proof, not `all_must`: a cross-component
295     // cycle can be all-must and still carry no proof (Warning).
296     let certain = cycle_proof.is_some();

```

Trois choses se passent avant le texte. La paire (effet, slot cible) entre dans `covered` si le slot est local — *avant* le test de `reported_effects`, si bien qu'un effet déjà signalé par le bras 1 ou 2 couvre quand même son écriture pour le bras 4. Le nom du diagnostic suit `cycle.cross_component` : c'est un booléen exact du `ChurnCycle` (`src/engine/churn.rs`, l. 109–119), calculé par `make_cycle` sur l'ensemble des composants propriétaires et porteurs. Et la preuve vient de `must_effect_cycle` (`src/rules/api/query.rs`, l. 1002–1031, chapitre 17), qui *re-dérive* les deux faits sur les arêtes brutes — toutes `Must`, un seul composant — au lieu de faire confiance à `cycle.all_must`. C'est la frappe « au point de connaissance » de l'ADR-21 : le graphe dit `Must` ou `May`, il ne frappe rien.

Invariant 19.1: Toute `Error` d'infinite-loop passe par une primitive `must`

Au commit `e67b10a`, `infinite_loop.rs` appelle `Diagnostic::error` en deux points seulement : l. 337 avec le jeton de `must_effect_cycle` (bras 3) et l. 496 avec celui de `must_on_all_paths` (bras 4). Les bras 1 et 2 n'appellent que `Diagnostic::warn`. Le sceau de sévérité (définition 17.1) rend impossible une `ERROR` construite autrement ; la règle n'a donc pas à *vérifier* `all_must`, elle ne peut simplement pas en abuser.

Six formulations suivent, choisies par (longueur du cycle = 1 ?, `no_deps` ?, preuve ?) :

Extrait 19.20 – `src/rules/impls/infinite_loop.rs`, l. 298–334 — les six textes du bras 3 ; le texte suit la preuve, pas `all_must`

```

298     let msg = if cyc.len() == 1 && e.no_deps && certain {
299         format!(
300             "this effect has no dependency array and stores a fresh \
301             reference into state {to_name}, so it re-runs after every \
302             render and re-triggers itself: infinite render loop"
303         )
304     } else if cyc.len() == 1 && e.no_deps {
305         format!(
306             "this effect has no dependency array and may store a fresh \
307             reference into state {to_name}, so it re-runs after every \
308             render and can re-trigger itself: possible infinite render loop"
309         )
310     } else if cyc.len() == 1 && certain {
311         format!(
312             "this effect stores a fresh reference into state {to_name} \
313             which its own deps react to, so the re-render runs it again: \
314             infinite render loop"
315         )
316     } else if cyc.len() == 1 {
317         format!(
318             "this effect may store a fresh reference into state {to_name} \
319             which its own deps react to, so the re-render can run it again: \
320             possible infinite render loop"
321         )
322     } else if certain {
323         format!(
324             "these effects form a state-update cycle ({path}) where each \
325             step stores a fresh reference that re-runs the next \
326             effect: infinite render loop"
327         )

```

```

328         } else {
329             format!(
330                 "these effects may form a state-update cycle ({path}) where \
331                 each step may store a fresh reference that re-runs the \
332                 next effect: possible infinite render loop"
333             )
334         };

```

longueur	no_deps	preuve	texte (début) et niveau
1	oui	oui	« this effect has no dependency array and stores a fresh reference into state ... re-triggers itself: infinite render loop » — ERROR
1	oui	non	« ... and may store a fresh reference ... can re-trigger itself: possible infinite render loop » — WARNING
1	non	oui	« this effect stores a fresh reference into state ... which its own deps react to ... infinite render loop » — ERROR
1	non	non	« this effect may store a fresh reference ... can run it again: possible ... » — WARNING
≥ 2	—	oui	« these effects form a state-update cycle (<i>chemin</i>) where each step stores a fresh reference ... » — ERROR
≥ 2	—	non	« these effects may form a state-update cycle (<i>chemin</i>) ... possible infinite render loop » — WARNING

TABLE 19.2 – Les six formulations du bras 3 (src/rules/impls/infinite_loop.rs, l. 298–334). « preuve » signifie `must_effect_cycle = All`; un cycle `cross_component` peut être tout-Must et n’avoir pas de preuve, il reçoit alors le texte « may ». Le chemin est rendu par `cycle_path` (src/rules/helpers/cycles.rs, l. 54–69), les slots d’un autre composant s’affichant « ‘x’ of ‘Parent’ ».

La longueur 1 avec `no_deps` est le cas de `NoDeps` (section 19.5) : un effet sans tableau qui écrit une référence fraîche est une auto-arête *non self_slot* du graphe, donc un cycle de longueur 1 du bras 3 — et non du bras 4, qui saute les effets sans tableau. La longueur 1 sans `no_deps` est plus rare : une arête pilotée par `deps` d’un slot vers lui-même est *self_slot* et jamais dans un cycle ; il reste le cas `cross`, où le slot écrit appartient au parent et le `dep` est versionné par lui — l’auto-boucle de `Parent/Child` que le bras 2 a devancée.

Le diagnostic est ancré sur l’*effet* porteur et porte deux sortes de notes, construites aux l. 336–377 du même fichier : un `Step::Write` avec `ValueClass::Fresh` à la page de l’écriture, dont le nom est le nom qualifié de la cible (`node_display`), puis un `Step::CycleEdge` par autre arête du cycle portée par un effet *du même composant* — les arêtes des autres composants seront racontées dans leur propre passe. C’est ce qu’on lit sur `TwoEffects` : « a fresh value is written to state ‘b’ here [hook:2] », puis « cycle continues: this effect freshly stores state ‘a’ [hook:3] ».

19.4.3 Deux passes de Tarjan, un plafond pour le cross

Décision : ADR-018 : un graphe de churn multi-effets, et ce qui en a bougé

Constat. L'ADR-17 avait laissé une limite explicite : deux effets, chacun écrivant l'état que l'autre lit, forment une boucle qu'aucun bras ne voit — ni le point fixe (rien ne grandit), ni le self-churn (aucun effet ne réécrit *son* dep). **Décision.** Un graphe sur les slots qualifiés (composant, label), arête $x \rightarrow y$ « un changement de x ré-exécute un effet qui écrit une référence fraîche dans y », force Must ou May ; un effet sans deps porte une auto-arête ; les écritures de handlers ne portent rien ; Tarjan en deux passes, sous-graphe Must d'abord (ERROR), graphe complet ensuite (WARNING) en sautant les régions déjà couvertes ; les cycles qui traversent plusieurs composants sont plafonnés à WARNING sous le nom **cross-component-infinite-loop**, parce qu'un dep prop est Versioned, jamais le slot exact. **Alternatives refusées** : une nouvelle règle (même classe de panne, un seul nom), un compteur d'événements encodé dans le store pour faire tirer le widening (« non-standard hack », déjà refusé par l'ADR-17 §3). **Dérive.** L'ADR situe le graphe dans `src/rules/helpers/churn_graph.rs` et conditionne le kill de convergence à un *unique* écrivain d'effet. Au commit e67b10a le graphe est un produit du moteur (`src/engine/churn.rs`, ADR-42 §4) et le kill est la preuve multi-site sur un plus petit point fixe des sites (chapitre 15, #154, #160, #162). Le commentaire « see `churn_graph.rs` » de liste 19.4 est un vestige.

Le double passage de Tarjan (`find_cycles`, `src/engine/churn.rs`, l. 635–662, chapitre 15) a une conséquence de lecture : un nœud pris dans un cycle tout-Must n'est jamais re-signalé par un cycle May qui le traverserait aussi. L'ERROR « couvre » la région. Symétriquement, les arêtes `self_slot` n'entrent dans aucune des deux passes : c'est la partition du bras 4.

Décision : ADR-029 : exposer les cycles aux packs sans schéma programme

Les règles déclaratives (chapitre 22) lisent le *même* graphe par une ancre `churn_cycles` sans arête, projetée sur le composant ancré : une ligne par arête du cycle portée par un effet de ce composant, colonnes exactes `{path, cross_component, all_must}`. Aucun jeton Certified n'y est atteignable : le plafond WARNING est structurel. La duplication avec le bras 3 est délibérée, et seul le bras graphe est exposé — l'ADR-20 item 2 interdit de fusionner les deux bras churn.

19.5 Bras 4 : le self-churn

19.5.1 Une référence recrée dans ses propres deps

Exemple 19.6: Objet recréé, sans deps, gardé, conditionnel

Extrait 19.21 – `/tmp/ch19/churn.tsx`

```
1 import { useState, useEffect } from "react";
2
3 export function ObjChurn() {
4   const [obj, setObj] = useState({ a: 1 });
5   useEffect(() => {
6     setObj({ ...obj, b: 2 });
7   }, [obj]);
8   return <p>{obj.a}</p>;
9 }
10
11 export function NoDeps() {
```

```

12  const [obj, setObj] = useState({ a: 1 });
13  useEffect(() => {
14    setObj({ a: 2 });
15  });
16  return <p>{obj.a}</p>;
17 }
18
19 export function FetchOnce() {
20   const [user, setUser] = useState<{ id: number } | null>(null);
21   useEffect(() => {
22     if (user === null) setUser({ id: 1 });
23   }, [user]);
24   return <p>{user?.id}</p>;
25 }
26
27 export function Conditional() {
28   const [obj, setObj] = useState({ a: 1 });
29   useEffect(() => {
30     if (Math.random() > 0.5) setObj({ ...obj, b: 2 });
31   }, [obj]);
32   return <p>{obj.a}</p>;
33 }

```

```

Conditional (2 hooks) /tmp/ch19/churn.tsx
  warn infinite-loop [hook:0] (line 29:2) this effect may store a fresh reference into state `obj` which its
    ↳ deps react to: possible infinite render loop
    → a fresh value is written to state `obj` here [hook:1] (line 30:29)
FetchOnce (2 hooks) /tmp/ch19/churn.tsx ✓
  verified infinite-loop no effect diverges into an infinite render loop
NoDeps (2 hooks) /tmp/ch19/churn.tsx
  error infinite-loop [hook:1] (line 13:2) this effect has no dependency array and stores a fresh reference
    ↳ into state `obj`, so it re-runs after every render and re-triggers itself: infinite render loop
    → a fresh value is written to state `obj` here [hook:1] (line 14:4)
ObjChurn (2 hooks) /tmp/ch19/churn.tsx
  error infinite-loop [hook:0] (line 5:2) this effect recreates object state `obj` it depends on. Every run
    ↳ stores a fresh reference (`Object.is` always fails) and re-triggers itself: infinite render loop
    → a fresh value is written to state `obj` here [hook:1] (line 6:4)

```

Quatre programmes, quatre bras différents ou presque. ObjChurn est le cas nominal du bras 4 : ERROR, ancrée sur le slot ([hook:0]). NoDeps est le bras 3, cycle de longueur 1, ancré sur l'effet ([hook:1]) ; les deux conventions d'ancrage sont côte à côte. Conditional est le bras 4 sans preuve : WARNING. FetchOnce est le silence du kill de convergence : la garde `user === null` meurt une fois `id: 1` posé dans le slot, l'arête est tuée par le moteur, le bras 4 ne voit rien.

Rappel React : Object.is et les références

`Object.is` compare deux objets par identité. Un littéral `...obj, b: 2` alloue un objet neuf à chaque évaluation, même si ses champs sont égaux aux précédents : `Object.is(prev, next)` est faux à chaque rendu, le dep a changé, l'effet ré-exécute. C'est ce que capture `Stability::PerRender` (définition 10.3), et ce qui rend le bras 1 aveugle : le join de deux `PerRender` est `PerRender`, le store converge au premier tour, rien n'entre dans `widen_trace`. L'ADR-17 l'énonce comme le point de départ de ce bras.

19.5.2 La partition et le verdict

Le bras 4 saute d'emblée les effets sans tableau de deps (ils appartiennent au bras 3), puis lit sa partition du graphe :

Extrait 19.22 – `src/rules/impls/infinite_loop.rs`, l. 417–433 — la partition du bras 4 : arêtes pilotées par deps, locales de bout en bout

```

417     if deps.list().is_none() {
418         continue; // no dependency array: the graph arm's self-edge
419     }
420
421     // This arm's partition of the relation (ADR-020 item 2, ADR-042 §4):
422     // the effect's dep-driven edges from this component's own slots into
423     // this component's own slots. A `self_slot` edge is the loop; every
424     // other one feeds the Info branch. Freshness, the convergence kill and
425     // the field-sensitive re-trigger test (#90) were applied by the engine
426     // when it built the edge.
427     let edges = graph.edges.iter().filter(|e| {
428         e.component == component
429         && e.effect_label == *eff_label
430         && !e.no_deps
431         && e.from.0 == component
432         && e.to.0 == component
433     });

```

Le commentaire énumère ce que le moteur a déjà appliqué en construisant l'arête : la fraîcheur de l'écriture, le kill de convergence, le re-déclenchement sensible aux champs. La règle ne refait rien de tout cela ; elle stratifie :

Extrait 19.23 – `src/rules/impls/infinite_loop.rs`, l. 439–471 — le verdict par arête : la preuve est re-dérivée des lignes d'écrivains

```

439     for e in edges {
440         let state_label = e.to.1;
441         let (sev, churn_proof) = if e.self_slot {
442             // The Error anchors on the fresh write being on all paths –
443             // re-derived from the writer rows and minted by the
444             // must-primitive, never trusted from the edge's own strength.
445             let fresh_blocks: HashSet<BlockId> = comp_result
446                 .slot_writers
447                 .iter()
448                 .filter(|w| w.owner.is_none() && w.slot == state_label)
449                 .filter(|w| w.region == WriterRegion::Effect(*eff_label))
450                 .filter(|w| w.written.fresh == Freshness::Fresh)
451                 .filter_map(|w| w.block)
452                 .collect();
453             if e.strength == EdgeStrength::Must && !fresh_blocks.is_empty() {
454                 match must_on_all_paths(body_cfg, &fresh_blocks) {
455                     MustResult::All(c) => (Severity::Error, Some(c)),
456                     _ => (Severity::Warning, None),
457                 }
458             } else {
459                 (Severity::Warning, None)
460             }
461         } else {
462             // Sets a different object state freshly while depending on
463             // object state: a multi-effect cycle candidate. The churn
464             // graph (F5b) analyzes those; when it reported a cycle for
465             // this write the Info would be a duplicate – skip. Otherwise
466             // keep it: deps may be too imprecise to close a real cycle.
467             if covered.contains(&(*eff_label, state_label)) {
468                 continue;
469             }
470             (Severity::Info, None)
471         };

```


Une arête `self_slot` est *la* boucle : un dep de l'effet réagit au slot même que l'effet écrit. Le moteur la pousse de deux façons (`src/engine/churn.rs`, l. 577–594) : depuis un dep *exact* (le dep est le slot), avec la force calculée (`Must` si l'écriture est `Fresh` sur tous les chemins), ou depuis un dep seulement *versionné* par le slot (un alias, un memo, un champ), toujours `May`. Pour l'`ERROR`, la règle ne fait pas confiance à `e.strength == Must` seul : elle recollecte, dans `slot_writers`, les blocs des écritures `Fresh` de ce slot dans la région de cet effet (lignes `owner.is_none()`), et demande à `must_on_all_paths` (chapitre 17) de frapper le jeton. Si la force est `May` (dep versionné, écriture conditionnelle), ou si le jeton n'est pas frappé (écriture imbriquée, différée), c'est un `WARNING`. Une arête vers un *autre* slot local est un candidat à cycle multi-effets : si le bras 3 a couvert la paire, `silence` ; sinon une `INFO`.

Proposition 19.1: L'Error du self-churn est un triple must

Quand le bras 4 émet une `ERROR` pour l'effet e et le slot ℓ , les trois faits suivants sont établis, chacun par une primitive ou une colonne `must` : (i) *must-rerun* : un dep de e est le slot ℓ (`EffectTrigger.exact`, chapitre 14) — toute écriture qui change ℓ ré-exécute e , par la sémantique « ou » des deps ; (ii) *must-change* : l'écriture est `Freshness::Fresh`, c'est-à-dire un argument de sorte référence `PerRender` seulement (chapitre 14) — chaque exécution pose une référence que `Object.is` distingue de la précédente ; (iii) *must-reach* : les blocs des écritures fraîches sont sur tous les chemins du corps (`must_on_all_paths`). La composition (i)◦(ii)◦(iii) est une boucle certaine dès le premier `commit`. Aucun fait `may` n'entre dans la chaîne ; en particulier la garde $\forall$ -stable n'y joue aucun rôle, puisqu'un dep `exact` n'est jamais stable.

C'est mot pour mot la stratification de l'ADR-17 §3, et l'argument de `soundness 3` de la même ADR.

Décision : ADR-017 §3 et §4 : un bras, pas une règle ; un gate couplé à ce bras

Constat. La scission d'`Unstable` en `Versioned/PerRender` (chapitre 10) tuait un faux positif massif d'`always-unstable-deps` sur l'état objet, mais rouvrait un faux négatif : un effet dont le dep est `Versioned(X)` était considéré *gaté*, alors que l'effet lui-même peut être ce qui change X . **Décision.** Un nouveau bras d'`infinite-loop` plutôt qu'une nouvelle règle (même classe de panne) et plutôt qu'un compteur d'événements dans le store ; la certitude vient de la *structure* des deps — « *Certainty cannot come from Versioned* », qui est une borne `may` — avec la stratification `ERROR/WARNING/silence` ci-dessus. Et une dépendance explicite : le fait que `Versioned` gate un effet ailleurs n'est `sound` *que* couplé à ce bras (« *load-bearing soundness dependency* »). **Dérive.** L'ADR nomme le gate `all_deps_unstable`, avec le quantificateur « un dep stable gate » ; le code actuel est `all_deps_provably_stable`, $\forall$ -stable (section 19.2.5), depuis ADR-21 §5.

Décision : ADR-020 item 2 : les deux bras churn restent séparés

Le bras 4 et le graphe ne sont pas deux implémentations parallèles : ils couvrent deux partitions disjointes de la même relation d'arêtes — « *single-effect same-slot with deps* » pour le self-churn, « *length-1 self-edges for effects without deps and cross-slot cycles* » pour le graphe —, tenues par des *continue* explicites (liste 19.22, `src/rules/impls/infinite_loop.rs`, l. 417). Supprimer le self-churn serait un FN sur `useEffect(() => setObj(...obj), [obj])` et la perte de l'`INFO`. À ne pas re-tenter.

19.5.3 Meilleur verdict par slot, et les trois textes

Un même effet peut porter plusieurs arêtes vers le même slot (deux écritures, ou un dep `exact` et un dep versionné). La règle garde le meilleur verdict par slot, rang `ERROR > WARNING > INFO`, départagé sur la position source la plus tôt pour que l'ancrage ne dépende pas de

l'ordre de la relation (src/rules/impls/infinite_loop.rs, l. 472–491). Les trois textes :

Extrait 19.24 – src/rules/impls/infinite_loop.rs, l. 493–523 — les trois strates du bras 4

```

493     for (state_label, (sev, call_span, proof)) in best {
494         let eff_span = comp_result.effect_info.get(eff_label).and_then(|i| i.span);
495         let mut diag = match (sev, proof) {
496             (Severity::Error, Some(proof)) => Diagnostic::error(
497                 "infinite-loop",
498                 proof,
499                 format!(
500                     "this effect recreates object state {state} it depends on. \
501                     Every run stores a fresh reference (`Object.is` always fails) \
502                     and re-triggers itself: infinite render loop",
503                     state = state_slot_name(state_label, &state_vals)
504                 ),
505             ),
506             (Severity::Info, _) => Diagnostic::info(
507                 "infinite-loop",
508                 format!(
509                     "this effect depends on object state but freshly recreates \
510                     state {state} outside its deps. No update cycle was found, \
511                     but deps may be too imprecise to rule one out",
512                     state = state_slot_name(state_label, &state_vals)
513                 ),
514             ),
515             _ => Diagnostic::warn(
516                 "infinite-loop",
517                 format!(
518                     "this effect may store a fresh reference into state \
519                     {state} which its deps react to: possible infinite render loop",
520                     state = state_slot_name(state_label, &state_vals)
521                 ),
522             ),
523         }

```

L'INFO mérite une phrase. Elle marque une *imprécision résiduelle* : l'effet dépend d'un état objet et recrée fraîchement un *autre* état objet, mais le graphe n'a fermé aucun cycle par cette écriture. Sur Dag, c'est exact — il n'y a pas de cycle. Sur un programme où les deps seraient trop imprécis pour que le graphe voie l'arête retour, ce serait le seul indice. Le bloc de commentaires qui précède la fonction (src/rules/impls/infinite_loop.rs, l. 383–394) dit encore, pour l'INFO, « cross-effect cycles are not analyzed (FN-flavor limit) » : c'est faux depuis l'ADR-18, ils le sont, et l'INFO n'est émise que quand aucun cycle ne couvre l'écriture. Second doc-comment en retard du fichier.

19.5.4 Le re-déclenchement sensible aux champs

Exemple 19.7: Un updater qui préserve le membre lu par le dep

Extrait 19.25 – /tmp/ch19/fields.tsx

```

1  import { useState, useEffect } from "react";
2
3  export function Slugify() {
4      const [data, setData] = useState({ name: "", slug: "" });
5      useEffect(() => {
6          setData((prev) => ({ ...prev, slug: prev.name.toLowerCase() }));
7      }, [data.name]);
8      return <p>{data.slug}</p>;

```

```

9  }
10
11 export function SlugifyWhole() {
12   const [data, setData] = useState({ name: "", slug: "" });
13   useEffect(() => {
14     setData((prev) => ({ ...prev, slug: prev.name.toLowerCase() }));
15   }, [data]);
16   return <p>{data.slug}</p>;
17 }

```

```

Slugify (2 hooks) /tmp/ch19/fields.tsx ✓
verified infinite-loop no effect diverges into an infinite render loop
SlugifyWhole (2 hooks) /tmp/ch19/fields.tsx
error infinite-loop [hook:0] (line 13:2) this effect recreates object state `data` it depends on. Every run
  ↳ stores a fresh reference (`Object.is` always fails) and re-triggers itself: infinite render loop
  ↳ a fresh value is written to state `data` here [hook:1] (line 14:4)

```

Concrètement, `Slugify` ne boucle pas : l'écriture change `data` (objet neuf) mais préserve `data.name`, le seul membre que le `dep` lit ; `Object.is("", "")` est vrai, l'effet ne repart pas. `SlugifyWhole`, dont le `dep` est l'objet entier, boucle.

Avant l'#90 (*precision-fp*, fermée), les deux composants recevaient l'ERROR : le graphe était granulaire au slot. Le correctif est dans le moteur, au moment de pousser une arête `self_slot` (`src/engine/churn.rs`, l. 577–594, `write_can_retrigger`), et son cœur est une fonction de trente lignes :

Extrait 19.26 — `src/engine/churn.rs`, l. 828–852 — `can_retrigger` : `false` seulement si chaque `dep` réactif lit un membre préservé

```

828 fn can_retrigger(
829   dep_exprs: &[Expr],
830   triggers: &[EffectTrigger],
831   component: ComponentId,
832   label: HookLabel,
833   state_vals: &HashMap<Var, HookLabel>,
834   written_expr: Option<&Expr>,
835 ) -> bool {
836   let Some(overwritten) = updater_overwrites(written_expr) else {
837     return true;
838   };
839   for (i, dep) in dep_exprs.iter().enumerate() {
840     let reacts = triggers
841       .iter()
842       .any(|t| t.dep == i && t.slot == (component, label));
843     if !reacts {
844       continue;
845     }
846     match slot_member(dep, label, state_vals) {
847       Some(m) if !overwritten.contains(&m) => {}
848       _ => return true,
849     }
850   }
851   false
852 }

```

`updater_overwrites` (`src/engine/churn.rs`, l. 856–876) n'accepte qu'un `updater` à un paramètre dont chaque `return` est soit `prev` lui-même, soit un littéral `...prev, k: ...` avec le *spread en premier* et des clés nommées ; il rend l'ensemble des clés écrasées. `slot_member` (l. 910–927) donne le premier membre qu'un `dep` lit sur le slot, `None` pour le slot nu. La réponse est `true` —

« peut re-déclencher », direction sound — dès qu'un dep réactif n'est pas placé sous un membre préservé. Le dep `data.name` de `Slugify` lit `name`, l'updater n'écrase que `slug : false`, l'arête est abandonnée. Le dep `data` de `SlugifyWhole` est le slot nu : `true`.

Piège

Ce raffinement ne joue que pour l'arête `self_slot`, et seulement pour un updater fonctionnel de cette forme exacte. Un `setData( ...data, slug )` (sans updater), un spread en seconde position, une clé calculée ou un second spread rendent `None`, donc `true` : l'arête reste, et la règle signale. C'est un choix de précision borné par la soundness, pas une analyse générale des champs ; la limite « granularité slot vs membre du graphe multi-effets » de `docs/limitations.md` le rappelle pour les cycles à plusieurs effets, qui ne bénéficient pas de ce test.

19.5.5 Un faux négatif de soundness : l'écriture d'une valeur dérivée

Exemple 19.8: Le memo qui se réécrit

Extrait 19.27 — `/tmp/ch19/derived157.tsx`

```
1 import { useState, useEffect, useMemo } from "react";
2
3 export function DerivedWrite() {
4   const [items, setItems] = useState<number[]>([]);
5   const sorted = useMemo(() => [...items].sort(), [items]);
6   useEffect(() => {
7     setItems(sorted);
8   }, [sorted]);
9   return <p>{items.length}</p>;
10 }
```

DerivedWrite (3 hooks) /tmp/ch19/derived157.tsx ✓
verified infinite-loop no effect diverges into an infinite render loop

Sonde : slot=0 setter=setItems line=Some(7) region=Effect(2) phase=Effect block=Some(0) fresh=Not.

Concrètement, ce composant boucle : `setItems(sorted)` pose un tableau neuf (le memo recalcule `[...items].sort()` dès que `items` change), `items` change, le memo recalcule, `sorted` est une référence neuve, l'effet repart. L'analyseur affirme le contraire. La cause est dans la colonne `written.fresh` : la valeur de `sorted` évalue `Versioned(items)` — le memo porte les labels de ses deps (ADR-17 §4) —, et `value_freshness` (`src/engine/written.rs`, l. 185–200, chapitre 14) lit « `Versioned` par le seul slot cible » comme `Freshness::Not` : « le slot ne change qu'à ses propres sets ». C'est vrai du slot, faux de la *valeur dérivée* qu'on y écrit. Une écriture `Not` ne porte aucune arête ; le bras 4 ne voit rien, ni le bras 1 (rien ne s'élargit). C'est l'#157 (*soundness-bug*, ouverte), documentée dans `docs/limitations.md` : « a loop carried by such a write is silent ». C'est, au commit `e67b10a`, le seul faux négatif de classe `soundness` que le projet reconnaisse à cette règle.

19.6 Synthèse : témoins, pièges, écarts

19.6.1 Les témoins produits

Chaque bras ancre son diagnostic et raconte sa preuve avec un sous-ensemble fermé des pas de la définition 17.2 :

bras	ancres (with_label, plage)	notes, dans l'ordre
1	slot ; plage de l'effet	un <code>Step::Handler{event, slot}</code> par <code>handler</code> qui écrit aussi le <code>slot</code> ; puis <code>slot_history</code> : un <code>Step::Write{Unknown}</code> par effet écrivain au tour du widening, un <code>Step::Widen{iteration}</code>
2	effet ; plage de l'effet	aucune
3	effet ; plage de l'effet	<code>Step::Write{Fresh}</code> à la plage de l'écriture, nom qualifié ; un <code>Step::CycleEdge{from, to}</code> par autre arête du cycle portée par ce composant
4	slot ; plage de l'effet	<code>Step::Write{Fresh}</code> à la plage de l'écriture retenue

TABLE 19.3 – Les témoins de `infinite-loop` par bras (`src/rules/impls/infinite_loop.rs`, l. 158–191, 348–376, 528–538 ; `Step` dans `src/rules/api/witness.rs`, l. 85–131).

19.6.2 Faux positifs assumés, faux négatifs observés

Le tableau qui suit rassemble ce que le chapitre a rencontré, avec l'issue ou le test qui le fixe. Les faux positifs sont tous au niveau `WARNING` ou `INFO`, conformément à l'invariant ; les faux négatifs sont le vrai coût.

sorte	programme	bras	cause, référence
FP WARNING	Mirror (exemple 19.3) : un slot nourri par un handler en fait grandir un autre par l'effet	1	le store joint les handlers ; <code>widen_trace</code> est par label, pas par déclencheur ; #144
FP WARNING	Dead (chapitre 13) : branche morte exécutée	1	la garde réfutée n'élimine pas sa branche
FP WARNING	une garde qui converge mais dont la preuve échoue : reviver qui ne tire qu'une fois par événement externe, fait d'un hook lu comme mouvant	3, 4	kill refusé faute de preuve de convergence ; #160, #161 (ouvertes)
FN	Updater : <code>setX(c => c + 1)</code> sans deps, chemin CLI	1	T par le store partagé ; <code>widen_trace</code> vide ; non répertorié
FN	DerivedWrite (exemple 19.8)	4	valeur dérivée lue <code>Not</code> ; #157
FN	parent atteint seulement en intra	2, 3	<code>shared_state</code> $\perp$, lignes <code>owner</code> absentes ; #20
FN (hors règle)	alias de setter appelé au rendu	—	<code>setter-in-render</code> ne suit pas les alias ; <code>infinite-loop</code> les suit via <code>all_setter_labels</code> (chapitre 18)

TABLE 19.4 – Faux positifs et faux négatifs d'`infinite-loop` rencontrés au commit e67b10a.

Piège

Deux vues de l'état coexistent (chapitre 10) : le *store*, où une écriture `PerRender` rend le slot `PerRender`, et l'évaluation de `StateVal(ℓ)`, qui rend `Versioned({ℓ})` parce qu'un slot React ne change qu'à ses sets. Le bras 1 lit le store (`effect_setter_writes`) ; les deps et la fraîcheur lisent l'évaluation. Confondre les deux produit soit le FP d'`always-unstable-deps` sur l'état objet, soit le FN `ObjChurn` : c'est précisément la tension que l'ADR-17 résout en couplant le gate `Versioned` au bras 4.

19.6.3 Mesure sur le corpus

Sur le corpus de référence (`docs/corpus-baseline.json`, 1 498 diagnostics), `infinite-loop` en compte 35 et `cross-component-infinite-loop` 18 (dossier 10, §5.5). Le chapitre 25 décrit la méthode ; retenons qu'une modification d'un bras se mesure d'abord là, avant de se justifier.

19.7 state-lifted-too-high : un état porté trop haut

19.7.1 Le bug visé : le *prop drilling*

Rappel React : un parent qui re-rend re-rend ses enfants

Quand un composant re-rend, React re-rend par défaut tous les éléments qu'il retourne, que leurs props aient changé ou non (sauf barrière `React.memo`). Un état possédé par `App` et utilisé seulement dans `Field`, trois niveaux plus bas, fait donc re-rendre `App`, chaque intermédiaire et tous leurs autres enfants à chaque frappe de clavier, pour la seule raison que la valeur *traverse* ces composants en props. La correction est de descendre l'état au plus près de son usage — la recommandation « *lift state up* » lue à l'envers.

Exemple 19.9: Trois niveaux de transmission

Extrait 19.28 – `tests/fixtures/state_lifted_too_high/drill.tsx`, l. 6–19 — la fixture `drill.tsx`

```

6 function Content() { return <article>static content</article>; }
7 function Field({ value, onChange }: { value: string; onChange: (v: string) => void }) {
8   return <input id="f" value={value} onChange={e => onChange(e.target.value)} />;
9 }
10 function Sidebar({ value, onChange }: { value: string; onChange: (v: string) => void }) {
11   return <aside><h2>Filters</h2><Field value={value} onChange={onChange} /></aside>;
12 }
13 function Layout({ value, onChange }: { value: string; onChange: (v: string) => void }) {
14   return <main><Sidebar value={value} onChange={onChange} /><Content /></main>;
15 }
16 export default function App() {
17   const [value, setValue] = useState("");
18   return <Layout value={value} onChange={setValue} />;
19 }

```

```

$ reactant check tests/fixtures/state_lifted_too_high/drill.tsx --no-color --fail-on never --trace --rule state-
  ↳ lifted-too-high
App (1 hooks) tests/fixtures/state_lifted_too_high/drill.tsx
warn state-lifted-too-high [hook:0] (line 17:8) state `value` is only used inside `<Field>`, 3 levels
  ↳ below `App`. Every write re-renders `App`, `Layout`, `Sidebar` and 1 other component they render only to
  ↳ pass it down; move the state into `Field`
  ↳ `App` passes it to `<Layout>` as `value`, `onChange` without using it itself (line 18:9)
  ↳ `Layout` passes it to `<Sidebar>` as `value`, `onChange` without using it itself (line 14:15)

```

→ `Sidebar` passes it to `` as `value`, `onChange` without using it itself (line 11:32)

La version corrigée, `drill_fixed.tsx`, où `Field` possède l'état, est silencieuse (test `the_fixed_version_is_silent`, `tests/state_lifted_too_high.rs`, l. 58).

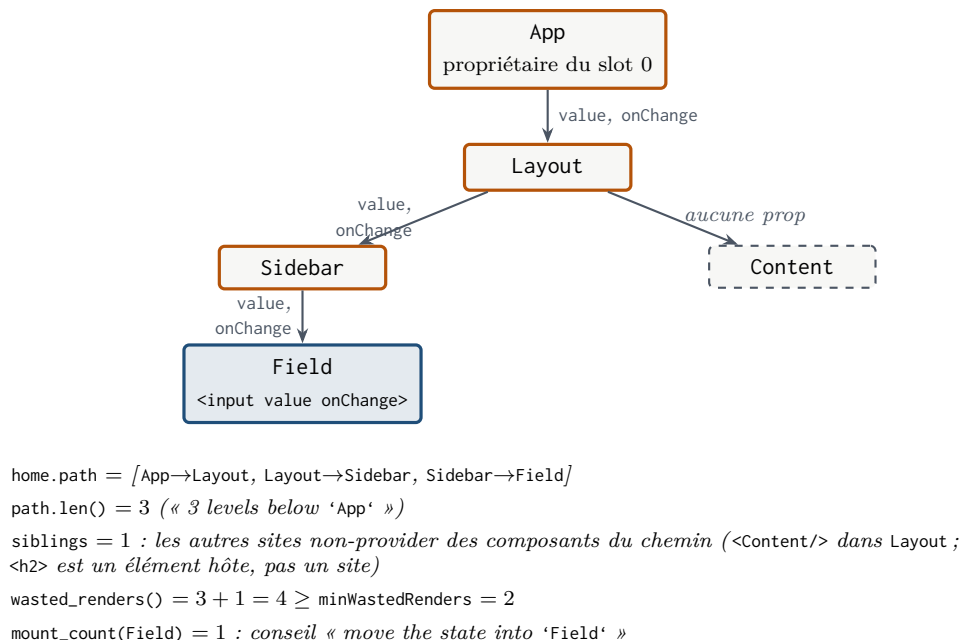


FIGURE 19.4 – L'arbre d'éléments de `drill.tsx` vu par `state-lifted-too-high` : le chemin du foyer (bord orangé) descend d'`App` à `Field` (bleu), le foyer ; `Content` (pointillé) est un frère compté dans `siblings`. En dessous, les quantités que la règle calcule et le seuil qu'elles franchissent. La construction du chemin par `home_of` est détaillée au chapitre 16.

19.7.2 Le foyer d'un slot

La règle repose sur la dépendance de rendu (définition 16.3, chapitre 16) : pour chaque composant, un résumé local dit quelles *sources* (slots, setters, props, contextes, noms de module) atteignent sa sortie hôte, ses effets, ses conditions de montage — ce qu'il *utilise* —, et pour chaque élément qu'il construit, quelles sources chaque prop transporte — ce qu'il *transmet*. L'index de rendu (`RenderIndex`, `src/rules/helpers/render_tree.rs`) compose ces résumés sur l'arbre d'éléments, en traduisant la question à chaque saut dans le repère de l'enfant : `Slot(0)` d'`App` devient `Prop(value)` dans `Layout`.

Définition 19.2: Foyer d'un slot

Soit un slot ℓ possédé par un composant o . Un composant *utilise* ℓ s'il utilise la valeur du slot, son setter, ou un nom de module que les écrivains de ℓ écrivent aussi (« co-écritures »). Le *foyer* de ℓ est le composant atteint depuis o en descendant l'arbre d'éléments tant que le composant courant n'utilise pas ℓ lui-même et qu'*exactement un* de ses enfants (au sens du sous-arbre) l'utilise ; le *chemin* est la suite des sauts parcourus. Le foyer est indéfini (`None`) si un écrivain de ℓ peut écrire n'importe quel nom de module, si rien ne lit la valeur de ℓ , ou si rien n'appelle son setter.

C'est exactement `home_of` (`src/rules/helpers/render_tree.rs`, l. 195–257, lu au chapitre 16), qui rend un `Home` :

Extrait 19.29 – `src/rules/helpers/render_tree.rs`, l. 34–64 — Hop, Home et le compte des rendus gaspillés

```

34 /// One step down the element tree: `from` builds `<to>` and hands it the
35 /// relevant value through `props`.
36 #[derive(Debug, Clone)]
37 pub(in crate::rules) struct Hop {
38     pub from: ComponentId,
39     pub to: ComponentId,
40     pub span: Option<SourceRange>,
41     /// The props that carry it (`None`: through a spread).
42     pub props: Option<Vec<Symbol>>,
43     /// No prop carries it, the context of this name does: `<to>` is nested
44     /// in its provider.
45     pub context: Option<String>,
46 }
47
48 /// Where a slot's uses live.
49 pub(in crate::rules) struct Home {
50     /// From the owner down to the home component (empty: the owner itself).
51     pub path: Vec<Hop>,
52     /// Component elements the components on the path build besides the next
53     /// hop: they re-render on every write too, and none of them uses the
54     /// slot (a list site counts once, a lower bound).
55     pub siblings: usize,
56 }
57
58 impl Home {
59     /// Component renders each write triggers for nothing: the components
60     /// above the home, and the other elements they build.
61     pub(in crate::rules) fn wasted_renders(&self) -> usize {
62         self.path.len() + self.siblings
63     }
64 }

```

Un Hop porte les props qui transportent la valeur (`None` pour un spread) ou, à défaut, le nom du contexte qui la porte. `siblings` est une *borne inférieure* : un site de liste compte une fois. `wasted_renders` additionne les composants du chemin et leurs frères.

19.7.3 L'absence d'usage vaut preuve

Le foyer prouve une *absence* : « aucun composant au-dessus du foyer n'utilise le slot ». Pour une règle qui signale sur cette base, la polarité est inversée par rapport à `infinite-loop` : une source *manquante* dans un résumé serait un faux positif, et « sound » veut dire ici « ne jamais affirmer une absence qu'on n'a pas vue ». D'où le principe de l'ADR-41 §2, que l'en-tête de `render_tree.rs` énonce et que la descente applique à chaque site :

Extrait 19.30 – `src/rules/helpers/render_tree.rs`, l. 531–590 — la descente d'uses : garde touchée, liste, élément non résolu, profondeur, récursion — tout inconnu est un usage — puis la question traduite dans le repère de l'enfant

```

531     for (i, site) in summary.sites.iter().enumerate() {
532         // Whether the element exists, or which component it is, depends
533         // on it: deciding that is a use, whatever the props carry.
534         if site.guard.touche(rel) {
535             tree.user = true;
536             continue;
537         }
538         // A provider's value is followed to the elements it wraps, which
539         // it renders unchanged.

```



```

540     if site.provides.is_some() {
541         continue;
542     }
543     let (props, all) = forwarded(site, rel);
544     let contexts = contexts_at(summary, i, rel);
545     let modules: Vec<Source> = rel.modules().cloned().collect();
546     if props.is_empty() && !all && contexts.is_empty() && modules.is_empty() {
547         continue;
548     }
549     // One of many instances: the owner of the site holds the state.
550     if site.in_list {
551         tree.user = true;
552         continue;
553     }
554     let Some(child) = resolve(site, program) else {
555         tree.user = true;
556         continue;
557     };
558     if depth >= MAX_DEPTH || !visiting.insert(child) {
559         tree.user = true;
560         continue;
561     }
562     // Only the prop-less hop names its context: a prop is what the
563     // message has to show when there is one.
564     let context = match contexts.first() {
565         Some(Source::Context(c)) if props.is_empty() && !all => Some(c.origin_name.clone()),
566         _ => None,
567     };
568     let child_rel = Relevance {
569         any_prop: all,
570         sources: props
571             .iter()
572             .cloned()
573             .map(Source::Prop)
574             .chain(contexts)
575             .chain(modules)
576             .collect(),
577     };
578     let sub = self.uses(child, &child_rel, program, visiting, depth + 1);
579     visiting.remove(&child);
580     tree.kids.push((
581         Hop {
582             from: comp,
583             to: child,
584             span: site.span,
585             props: (!all).then_some(props),
586             context,
587         },
588         sub,
589     ));
590 }

```

Invariant 19.2: Tout inconnu est un usage

Dans la descente de `uses`, un composant sans résumé de rendu (`UseTree::user()`, `src/rules/helpers/render_tree.rs`, l. 525), un site dont la garde de montage touche la question, un site en liste, un élément dont l'origine ne se résout pas à un composant unique enregistré (`resolve`, `src/rules/helpers/render_tree.rs`, l. 767–777 : une enveloppe `memo`, un composant de bibliothèque, un import non résolu), un composant déjà en cours de visite

(récursion) ou une profondeur ≥ 64 marquent le composant courant comme *utilisateur*. La descente s'arrête donc *au-dessus* de l'inconnu, et le foyer ne peut jamais être placé sous un composant dont on ignore ce qu'il fait de la valeur.

Exemple 19.10: Trois arrêts de la descente

Sur les fixtures du dépôt (tests/fixtures/state_lifted_too_high/), commande avec `-rule state-lifted-too-high -show-clean` :

- `unresolved_child.tsx` : `Field` vient de `some-ui-lib`. `Layout` construit un élément non résolu et devient utilisateur ; la descente s'arrête à lui, un niveau sous `App`, et ne descend jamais dans `Field`. Avec les seuils par défaut le résultat est le silence (test `what_cannot_be_proven_is_not_claimed`, l. 137), mais pour une raison de *seuil* : `App` ne construit qu'un élément, donc `siblings = 0` et `wasted_renders() = 1 < 2`. Avec `-rule-option state-lifted-too-high:minWastedRenders=1`, on observe bien « state 'value' is only used inside '<Layout>', 1 level below 'App' » : l'invariant a placé le foyer au-dessus de l'inconnu, pas plus bas.
- `list_items.tsx` : `Row` est rendu dans un `.map`. Le site en liste fait de `Wrapper` l'utilisateur ; le foyer est `Wrapper`, « 1 level below 'App' », avec `Side` pour frère : WARNING « move the state into 'Wrapper' », jamais `Row` — un état par ligne changerait le sens du programme (test `list_items_keep_the_state_in_the_component_that_maps_them`, l. 158).
- `intermediate_uses.tsx` : `Layout` lit `value` dans un attribut hôte (`<main title=value>`). Il est utilisateur ; le foyer est `Layout`, « 1 level below 'App' », frère `Other` : WARNING « move the state into 'Layout' » (test `the_home_stops_at_the_first_component_that_uses_it`, l. 96).

19.7.4 La règle, ligne à ligne

Le fichier fait 146 lignes ; en voici la moitié utile. Les deux options d'abord :

Extrait 19.31 – `src/rules/impls/state_lifted_too_high.rs`, l. 22–43 — les deux seuils, déclarés comme options typées

```
22 impl StateLiftedTooHigh {
23     const NAME: &'static str = "state-lifted-too-high";
24     const MIN_DEPTH: OptionSpec = OptionSpec {
25         name: "minDepth",
26         kind: OptionKind::UInt {
27             default: 1,
28             min: 1,
29             max: 64,
30         },
31         doc: "report only when the state's home is at least this many levels below its owner",
32     };
33     const MIN_WASTED: OptionSpec = OptionSpec {
34         name: "minWastedRenders",
35         kind: OptionKind::UInt {
36             default: 2,
37             min: 1,
38             max: 1000,
39         },
40         doc: "report only when each write re-renders at least this many components for \
41             nothing (the components above the home plus the other elements they build)",
42     };
43 }
```

`minDepth` (défaut 1, bornes [1,64]) est la profondeur minimale du foyer ; `minWastedRenders` (défaut 2, bornes [1,1000]) le nombre minimal de rendus gaspillés par écriture. Ce sont des options au sens de l'ADR-41 §4 : le registre valide la configuration contre la spécification, bruyamment. Une valeur hors bornes est refusée avec le code de sortie 2 (observé) :

```
$ reactant check tests/fixtures/state_lifted_too_high/drill.tsx --no-color --fail-on never --rule-option state-lifted-too-high:minDepth=99
[error] option `minDepth` of rule `state-lifted-too-high` must be an integer between 1 and 64
```

Un nom d'option inconnu l'est aussi : c'est ce que verrouille le test `a_bad_option_is_a_usage_error` (`tests/state_lifted_too_high.rs`, l. 187), avec `depth=2`, le code 2 et le message « has no option 'depth' ». Puis le cœur de check :

Extrait 19.32 – `src/rules/impls/state_lifted_too_high.rs`, l. 54–77 — un foyer par slot `useState`, trois filtres

```
54 fn check(&self, ctx: &RuleCtx) -> Vec<Diagnostic> {
55     let program = ctx.program();
56     let owner = ctx.component();
57     let Some(result) = program.components.get(&owner) else {
58         return Vec::new();
59     };
60     let min_depth = ctx.config().uint(&Self::MIN_DEPTH) as usize;
61     let min_wasted = ctx.config().uint(&Self::MIN_WASTED) as usize;
62     let index = ctx.cache().render();
63     let names = state_val_labels(&result.render_cfg);
64     let mut diags = Vec::new();
65     for hook in &result.hooks {
66         let HookEntry::State { label, span, .. } = hook else {
67             continue;
68         };
69         let Some(home) = index.home_of(owner, *label, program) else {
70             continue;
71         };
72         if home.path.is_empty()
73             || home.path.len() < min_depth
74             || home.wasted_renders() < min_wasted
75         {
76             continue;
77         }
78     }
79 }
```

Pour chaque `HookEntry::State` du composant courant (le propriétaire), la règle demande le foyer à l'index de rendu du cache programme (`ctx.cache().render()`, construit une fois par exécution comme le graphe de churn). Trois continue : un chemin vide (le propriétaire est lui-même utilisateur, ou plusieurs enfants le sont), un chemin plus court que `minDepth`, un gaspillage sous `minWastedRenders`. Sur `drill.tsx` avec `minDepth=4` ou `minWastedRenders=5`, la règle se tait (observé) ; avec les défauts, $3 \geq 1$ et $4 \geq 2$ passent.

Le message est assemblé ensuite (l. 78–124) : la liste des composants re-rendus est le chemin plus « N other component(s) they render » pour les frères ; le conseil dépend de `mount_count` :

Extrait 19.33 – `src/rules/impls/state_lifted_too_high.rs`, l. 96–117 — le conseil suit le nombre de sites qui montent le foyer, et le contexte éventuel

```
96     let home_id = home.path.last().expect("non-empty").to;
97     // A component rendered from several places is shared: moving the
98     // state into it would change it for every other caller, so the
99     // advice is a wrapper that owns the state and renders it.
100     let fix = if index.mount_count(home_id) > 1 {
101         format!(
```

```

102         ``{target}` is rendered from other places too, so wrap this \
103         <{target}>` in a small component that owns the state"
104     )
105   } else {
106     format!("move the state into `{target}`")
107   };
108   // Reached by context, the state takes its provider along, and the
109   // components on the way do not even pass it down.
110   let context = home.path.iter().find_map(|h| h.context.as_deref());
111   let (why, fix) = match context {
112     Some(ctx) => (
113       "although none of them uses it",
114       format!("{fix}, with the `{ctx}` provider that hands it on"),
115     ),
116     None => ("only to pass it down", fix),
117   };

```

Un foyer monté depuis plusieurs endroits est *partagé* : y déplacer l'état le changerait pour tous les autres appelants, d'où le conseil d'envelopper *ce* `<Target>` dans un petit composant propriétaire. Un chemin qui passe par un contexte change le « pourquoi » (« although none of them uses it » : les composants intermédiaires ne transmettent même pas la valeur, le provider la porte) et le « comment » (« with the 'Ctx' provider that hands it on »). Le diagnostic est un WARNING ancré sur le slot et la plage du `useState`, avec une note `Step::Forward{from, to, props, context}` par saut (l. 125–142) : c'est la chaîne de la sortie du exemple 19.9.

Exemple 19.11: Un état qui voyage par contexte

Extrait 19.34 – `tests/fixtures/state_lifted_too_high/context.tsx`, l. 6–21 — la fixture `context.tsx`, première moitié

```

6  const Query = createContext({ q: "", setQ: (_, string) => {} });
7  function Search() {
8    const { q, setQ } = useContext(Query);
9    return <input value={q} onChange={(e) => setQ(e.target.value)} />;
10 }
11 function Header() { return <h1>Title</h1>; }
12 function Page() { return <main><Header /><Search /></main>; }
13 export function App() {
14   const [q, setQ] = useState("");
15   return (
16     <Query.Provider value={{ q, setQ }}>
17       <Header />
18       <Page />
19     </Query.Provider>
20   );
21 }

```

```

App (1 hooks) tests/fixtures/state_lifted_too_high/context.tsx
  warn state-lifted-too-high [hook:0] (line 14:8) state `q` is only used inside ``, 2 levels below `
    ↳ App`. Every write re-renders `App`, `Page` and 2 other components they render although none of them uses
    ↳ it; move the state into `Search`, with the `Query` provider that hands it on
    ↳ `App` renders `` inside the `Query` provider without using it itself (line 18:6)
    ↳ `Page` renders `` inside the `Query` provider without using it itself (line 12:41)
Guarded (1 hooks) tests/fixtures/state_lifted_too_high/context.tsx ✓
OtherSearch (2 hooks) tests/fixtures/state_lifted_too_high/context.tsx ✓
Search (2 hooks) tests/fixtures/state_lifted_too_high/context.tsx ✓

```

(Commande : `-trace -show-clean -rule state-lifted-too-high`.) Le site `<Query.Provider value= q, setQ >` a provides = `Some(Query)`; il est sauté par la descente (liste 19.30, « A

provider's value is followed to the elements it wraps »), et les éléments qu'il enveloppe reçoivent Context(Query) dans leur question. Header ne consomme pas le contexte, Page non plus ; Search l'appelle par useContext(Query). Les deux frères comptés sont Header dans App et Header dans Page. La seconde moitié du fichier, Guarded, place un <Tooltip/> importé d'une bibliothèque sous le provider : élément non résolu, donc utilisateur possible, donc silence — l'invariant en action (le suivi des contextes est au chapitre 16).

19.7.5 Pourquoi Warning, et ce qui arrête la cascade

Décision : ADR-041 §3 : deux règles de cascade, toutes deux Warning

Constat. Les cascades de rendu (state-lifted-too-high, wasted-subtree-render) constatent un fait certain — ces composants re-rendent pour rien — dont le *coût* est incertain : un composant statique re-rendu trois fois par seconde ne coûte rien. **Décision.** Aucune des deux règles n'atteint l'ERROR : c'est la définition du WARNING de CLAUDE.md, « fait certain au coût incertain ». Les seuils sont des options parce que le coût acceptable est l'affaire d'une équipe, pas de l'analyseur (§4 de la même ADR). **Alternative refusée** : porter la dépendance de rendu dans StateValue comme un champ de plus — elle aurait changé toutes les comparaisons de valeurs existantes (unnecessary-rerender, clé du cache de composants) pour un fait qui n'est pas une valeur (§1).

Ce qui arrête la descente arrête aussi la règle, et c'est voulu. Une enveloppe React.memo ou useRef n'est pas dans le registre des composants : l'élément est opaque, le composant qui le construit est utilisateur, le foyer s'arrête là. L'#64 (*precision-fn*, ouverte) note que cela tronque les cascades ; le chapitre 21 y revient pour wasted-subtree-render. Symétriquement, un état que *personne* n'écrit, ou dont le setter n'atterrit nulle part, n'a pas de foyer : la règle n'a alors rien à dire sur des écritures qui n'existent pas.

Piège

La règle n'a pas de safe_check : aucune ligne verified state-lifted-too-high n'apparaît sous -info. La raison est la polarité : une règle d'affirmation qui se tait n'a pas prouvé l'absence de défaut, elle a seulement manqué une preuve de présence (chapitre 18, « signaler ou affirmer »). Sur la fixture owner_effect.tsx (le tableau kanban de makeplane/plane), le propriétaire enregistre ses écouteurs dans un effet sur une ref : l'effet est un usage, le chemin est vide, silence — pas d'assurance.

Résumé du chapitre

infinite-loop est une règle à quatre bras qui partagent un nom, une classe de panne et deux canaux de déduplication (reported_effects, covered), mais pas leurs preuves. Le *bras point fixe* lit un *signal* (définition 19.1) : un slot dans widen_trace dont l'écriture rejouée depuis $\perp$ est $\perp$ ou non bornée ; c'est un fait may, plafonné à WARNING (#144), filtré par le montage seul, par la garde $\forall$ -stable sur les deps (ADR-21 §5) et par la classe Handler des écritures. Le *bras cross-component* lit le store partagé pour un prop setter d'un autre composant, WARNING aussi, muet si le parent n'est pas atteint en descendant (#20). Le *bras graphe* lit les cycles du graphe de churn et frappe l'ERROR par must_effect_cycle quand le cycle est tout-Must dans un seul composant, WARNING sinon ; six textes, un nom par cross_component. Le *bras self-churn* lit les arêtes pilotées par deps locales de bout en bout : ERROR par must_on_all_paths sur une arête self_slot (triple must, proposition 19.1), WARNING sans preuve, INFO pour une écriture fraîche vers un autre slot qu'aucun cycle ne couvre. Le moteur a déjà appliqué la fraîcheur, le kill de

convergence et le re-déclenchement sensible aux champs (#90) en construisant les arêtes. Deux invariants tiennent tout : toute `ERROR` passe par une primitive `must` (invariant 19.1), et le `widening` n'est jamais une preuve.

`state-lifted-too-high` lit le foyer d'un slot (définition 19.2) dans l'index de rendu, filtre par `minDepth` et `minWastedRenders`, et émet un `WARNING` dont le conseil suit `mount_count` et le contexte. Sa soundness est celle d'une preuve d'*absence* : tout inconnu est un usage (invariant 19.2), la descente s'arrête au-dessus.

Types et fichiers rencontrés.

- `InfiniteLoop`, `check_multi_effect_cycles` et `check_object_churn`, dans `src/rules/impls/infinite_loop.rs`.
- `src/engine/analysis_result.rs`, `src/engine/fixpoint.rs`, `src/domains/impls/state_value.rs` : `WidenEvent`, `widen_trace`, `effect_setter_writes`, `is_unbounded`.
- `src/engine/setters.rs` : `SetterProp`, `cross_component_setters`, `all_setter_labels`.
- `src/engine/churn.rs` : `ChurnEdge`, `ChurnCycle`, `ChurnGraph`, `find_cycles`, `can_retrigger`.
- `src/rules/api/query.rs`, `src/rules/api/witness.rs` : `must_effect_cycle`, `must_on_all_paths`, `slot_history`, `Step`.
- `src/rules/impls/state_lifted_too_high.rs` : `StateLiftedTooHigh`, `OptionSpec`.
- `src/rules/helpers/render_tree.rs` : `RenderIndex`, `Home`, `Hop`, `home_of`, `uses`, `resolve`.

Faux négatifs observés. `DerivedWrite` (#157); `Updater` sur le chemin `CLI` (non répertorié); parent atteint seulement en `intra` (#20).

Doc-comments en retard sur le code. La tête d'`InfiniteLoop` (l. 26–32); le bloc du `self-churn` (l. 390–394); le renvoi « `see churn_graph.rs` » (l. 230).

Le chapitre 20 passe aux règles sur les effets, dont `widening-info` rencontrée ici, et aux primitives de dominance et de couverture qu'elles partagent avec les bras 3 et 4.

Exercices

Exercice 19.1: Les portes à la main

Pour `Bounded` (liste 19.5), calculer les intervalles successifs du slot, l'itération d'élargissement et `effect_setter_writes`, puis dire quelle porte absout et pourquoi `widening-info` parle quand même. Refaire l'exercice avec la garde `if (count < limit)` où `limit` est un `prop` : quelle porte change de réponse ?

Exercice 19.2: Le quantificateur

Montrer qu'un filtre « un `dep` prouvé stable suffit à gater l'effet » introduit un faux négatif; construire le contre-exemple à partir de `MixedDeps` (liste 19.12) et le relier au test `stable_dep_alongside_top_dep_does_not_gate_self_write_loop`. Pourquoi l'arité exacte de la liste est-elle nécessaire en plus du quantificateur ?

Exercice 19.3: Promouvoir le bras 1 ?

Reproduire le faux positif `Mirror` (exemple 19.3). Proposer une condition supplémentaire du bras 1 fondée sur `effect_triggers` (« l'effet est re-déclenché par le slot qu'il écrit ») et dire si elle suffirait à promouvoir le bras en `ERROR` au sens de l'#144. Vérifier qu'elle ne rouvre pas le faux négatif des effets sans `deps`, puis celui d'`Updater` : la condition lit-elle le bon store ?

Exercice 19.4: Ordre des bras

Sur Parent/Child (liste 19.16), donner l'arête du graphe de churn que le bras 3 aurait rapportée si le bras 2 s'était tu, sa force, son `cross_component` et le texte du tableau 19.2 qu'elle aurait reçu. Que se passe-t-il si l'on inverse l'ordre des bras 2 et 3 dans `check` ?

Exercice 19.5: Deux sites qui se raniment

Écrire un composant où deux effets écrivent le même slot sous des gardes que l'écriture de l'autre ranime, et prédire le verdict du bras 3 à l'aide du théorème du plus petit point fixe des sites (chapitre 15) : pourquoi un plus grand point fixe des sites convergents les déclarerait-il convergents à tort ? Vérifier avec le test `two_writes_of_one_slot_that_revive_each_other_are_a_loop`.

Exercice 19.6: Le membre préservé

Transformer `Slugify` (liste 19.25) en écrivant `setData( ...data, slug )` sans `updater` ; prédire le verdict à partir de `updater_overwrites`, puis observer. Proposer une extension de `can_retrigger` à cette forme et dire quel fait supplémentaire elle exige pour rester `sound`.

Exercice 19.7: Réparer `DerivedWrite`

Expliquer, colonne par colonne, pourquoi la ligne de `setItems(sorted)` est `Freshness::Not` (exemple 19.8). Proposer, dans l'esprit « général d'abord » de `CLAUDE.md`, où placer le correctif de l'#157 : dans `value_freshness`, dans l'évaluation des `memos`, ou dans la construction des arêtes ? Quel FP chaque emplacement risque-t-il de créer ?

Exercice 19.8: Un foyer sans preuve

Réécrire `drill.tsx` pour que `state-lifted-too-high` se taise sans déplacer l'état (indice : faire lire `value` par `Layout`, ou rendre `<Field/>` depuis deux endroits). Pour chaque variante, dire lequel des trois filtres de liste 19.32 se ferme, et quel conseil la règle donnerait si elle parlait.

Chapitre 20

Les règles sur les effets

Objectifs du chapitre.

- Connaître les huit règles natives qui portent sur les effets et les hooks à deps : `widening-info`, `analysis-limit`, `conditional-hook`, `always-unstable-deps`, `missing-deps`, `unnecessary-rerender`, `missing-cleanup` et `stale-closure`.
- Savoir, pour chacune, quel défaut React elle vise, quelles données du moteur elle lit, à quel niveau elle peut conclure et quelle preuve ce niveau exige.
- Comprendre les mécanismes transverses que ces règles partagent, dont : la polarité d'une liste de deps (lire pour tirer, lire pour se taire), la relation d'enregistrement `registrations` (ADR-34) et le jeton Certified qui seul ouvre la porte de l'ERROR (ADR-21).
- Lire une sortie de `reactant-analyzer` sur ces règles, y compris les lignes `verified` et `suspended`, et reconnaître les faux positifs assumés (#40, #42) et les faux négatifs connus au commit e67b10a.

Prérequis : chapitre 17 (trait Rule, sceau de sévérité, Certified, témoins); chapitre 14 (relations du moteur, dont `registrations`); chapitre 10 (treillis `Stability`, `Versioned`, `PerRender`); chapitre 6 (`DepsArg`, `AccessPath`, couverture par préfixe); chapitre 12 (dominance). Le chapitre 18 applique la même grille aux règles sur l'état.

20.1 Vue d'ensemble : sept familles, une grille

Un effet React est un morceau de code que React exécute *après* le commit, et que le composant garde sous contrôle au moyen d'un tableau de dépendances. Les `useEffect`, `useMemo` et `useCallback` ont en commun ce tableau : c'est lui qui décide, rendu après rendu, si le corps doit être ré-exécuté ou si la valeur mémorisée peut être réutilisée. Presque tous les défauts de ce chapitre naissent d'un désaccord entre ce que le corps *lit* et ce que le tableau *déclare*, ou entre ce que l'effet *démarre* et ce qu'il *défait*.

Ce chapitre étudie huit règles natives, regroupées en sept familles et présentées de la plus simple à la plus riche. Toutes sont des post-passes pures (ADR-6) : elles ne modifient rien, elles lisent l'`AnalysisResult` convergé d'un composant (chapitre 13) et produisent des `Diagnostic`. Les règles `wasted-subtree-render` et `server-component-hook`, qui figurent dans le même dossier de notes, relèvent de la dépendance de rendu et des projets Next.js; elles sont traitées au chapitre 21.

20.1.1 La carte des règles

Le tableau 20.1 donne, pour chaque règle, le niveau le plus élevé qu'elle puisse atteindre, ce qu'elle lit et, le cas échéant, le jeton de preuve qui l'autorise à produire une ERROR. Le niveau maximal n'est pas une convention : il découle de la construction, puisque `Diagnostic::error`

exige un `Certified<E>` que seul le module `src/rules/api/query.rs` peut frapper (définition 17.1). Deux règles seulement détiennent un tel jeton.

Règle	Niveau max	Données du moteur lues	Jeton d'ERROR
widening-info	INFO	widen_trace	—
analysis-limit	INFO	stats, hook_calls[].opaque, effect_info	—
conditional-hook	ERROR (tous jours)	hook_calls, render_cfg	Certified<ConditionalHookCall>
always-unstable-deps	WARNING	hooks (deps), env de sortie, stores et tas convergés	—
missing-deps	WARNING	effect_info, env de sortie, tas convergé	—
unnecessary-rerender	WARNING	hooks (init, corps), stores convergés	—
missing-cleanup	WARNING	registrations, corps d'effets	—
stale-closure	ERROR (sous preuve)	registrations, corps, alias de setters	Certified<StaleCapture>

TABLE 20.1 – Les huit règles du chapitre. Fichiers : `src/rules/impls/widening_info.rs` (41 lignes), `analysis_limit_info.rs` (158), `conditional_hook.rs` (581), `always_unstable_deps.rs` (489), `missing_deps.rs` (701), `unnecessary_rerender.rs` (446), `missing_cleanup.rs` (113), `stale_closure.rs` (469).

Aucune de ces règles n'a d'état : ce sont des structures unitaires (`pub struct MissingDeps;`) qui implémentent le trait `Rule` du chapitre 17. Aucune ne déclare d'option. Six d'entre elles fournissent un `safe_check`, l'assurance positive « `verified ...` » affichée sous `-info` quand la règle s'appliquait et n'a rien trouvé ; les deux règles INFO n'en ont pas, puisqu'une limite n'a pas d'« état sûr » à publier.

20.1.2 La grille de lecture

Chaque règle est présentée avec la même grille, celle du chapitre 18 :

1. le *défaut React visé*, dans un encadré de rappel ;
2. un *programme court* qui déclenche la règle, et sa sortie réellement observée ;
3. les *conditions exactes*, lues dans le code (extraits verbatim) ;
4. le *niveau* et la *preuve* qu'il exige ;
5. les *silences*, que les commentaires du code appellent des *kills* ;
6. les *faux positifs assumés* et les *faux négatifs connus*, présentés comme des constats au commit e67b10a.

Définition 20.1: Kill

Un *kill* est une condition qui supprime un finding candidat. Dans une analyse sound, un kill n'est admissible que s'il est une *preuve* que le défaut ne peut pas se produire (« la capture ne peut pas être périmée », « le slot n'est jamais écrit ») ; un kill heuristique est un faux négatif potentiel, donc interdit (définition 3.8).

Les sorties de ce chapitre ont été obtenues avec le binaire construit au commit e67b10a, sur des fichiers placés sous `/tmp/ch20/`, par des commandes de la forme :

```
NO_COLOR=1 reactant check /tmp/ch20/timer.tsx --trace [--info] [--show-clean] [--rule R]
```

Deux conventions aident à les lire. Les numéros [hook:N] sont les HookLabel attribués par le lowering dans l'ordre des appels (0 est le premier hook, les handlers JSX onX comptent aussi). Le registre trie les diagnostics d'un composant d'abord par nom de règle, puis par sévérité et par position (src/rules/registry.rs, l. 307–334) : une sortie est donc groupée par règle en ordre alphabétique, non par ligne.

20.1.3 Ce que React fait d'un tableau de deps

Rappel React : tableau de dépendances

useEffect(f, deps) exécute f après le commit du premier rendu, puis après chaque rendu où *au moins une* entrée de deps diffère de l'entrée de même rang au rendu précédent, selon Object.is. Sans deuxième argument, l'effet tourne après chaque rendu ; avec [], il ne tourne qu'au montage (et son cleanup au démontage). useMemo et useCallback utilisent la même comparaison pour décider de recalculer leur valeur. Trois conséquences structurent ce chapitre :

- une valeur lue par le corps mais absente des deps n'est pas comparée : le corps peut continuer à voir la copie d'un ancien rendu (missing-deps, stale-closure) ;
- une entrée qui est une référence neuve à chaque rendu fait échouer Object.is à chaque rendu, et donc tout le tableau (always-unstable-deps) ;
- un effet qui démarre quelque chose de durable est ré-exécuté à chaque changement de deps (et deux fois au premier montage sous StrictMode) ; sans cleanup, chaque exécution s'ajoute aux précédentes (missing-cleanup).

20.1.4 Trois états d'un argument de deps, deux manières de le lire

Le chapitre 6 a présenté la façon dont l'IR garde l'argument de deps d'un hook. Elle distingue trois faits, que le code refuse de confondre :

Extrait 20.1 – src/ir/hooks.rs, l. 102–107 — les trois états de l'argument de deps

```
102 #[derive(Debug, Clone)]
103 pub enum DpsArg {
104     Absent,
105     Opaque,
106     List(DepsList),
107 }
```

Absent signifie qu'aucun argument n'a été écrit : le hook tourne à chaque rendu et rien de ce qu'il capture ne peut se périmer. Opaque signifie qu'un argument a été écrit mais que l'IR ne peut pas le lire (useMemo(fn, deps), où deps est une variable) : le hook *est* gardé par une liste, dont aucun élément n'est visible. List porte un tableau littéral sous la forme d'une DpsList : les éléments visibles elems, l'arité Exact(n) ou AtLeast(n), et les positions spread_at des éléments venus d'un spread. Lire Opaque comme Absent sauterait le hook ; le lire comme [] affirmerait qu'il ne déclare rien. Les deux lectures sont fausses, en sens opposés.

Un spread pose un problème plus fin. Dans useEffect(f, [...rows]), React compare rows[0], rows[1], ..., jamais rows lui-même. Le lowering garde pourtant rows dans elems, parce que c'est ce qu'il peut voir. Selon qu'une règle lit la liste pour *tirer* ou pour *se taire*, cet élément n'a pas la même valeur de vérité.

Définition 20.2: Polarité d'une lecture de deps

Une règle lit une liste de deps *pour tirer* quand la présence d'un élément peut la faire émettre un diagnostic (« cet élément est instable ») ; elle la lit *pour se taire* quand la présence d'un élément peut supprimer un diagnostic (« cette lecture est déclarée »). Lire `elems` pour tirer est toujours sound, car chaque élément sur-approxime ce qu'il représente ; lire `elems` pour se taire ne l'est pas, car la source d'un spread ne couvre pas sa propre identité.

Le code centralise cette distinction dans une seule méthode, `DepsList::covering`. Son commentaire (`src/ir/hooks.rs`, l. 192–202) est l'énoncé même de la définition : lire `elems` pour faire *tirer* une règle est sound quelle que soit la troncature de la liste, parce que chaque élément sur-approxime ce qu'il représente ; le lire pour la faire *taire* ne l'est pas, parce que React compare `rows[0]`, `rows[1]`, etc., et jamais `rows`. Les éléments écrits à côté du spread couvrent toujours leurs propres lectures, d'où un filtre plutôt qu'un refus de toute la liste :

Extrait 20.2 – `src/ir/hooks.rs`, l. 203–215 — la liste qui couvre une lecture (le commentaire de tête, l. 192–202, est résumé dans le texte)

```

203 pub fn covering(&self) -> std::borrow::Cow<'_, [Expr]> {
204     if self.spread_at.is_empty() {
205         return std::borrow::Cow::Borrowed(&self.elems);
206     }
207     std::borrow::Cow::Owned(
208         self.elems
209             .iter()
210             .enumerate()
211             .filter(|(i, _)| !self.spread_at.contains(i))
212             .map(|(_, e)| e.clone())
213             .collect(),
214     )
215 }
```

Invariant 20.1: Polarité des deps

Une règle qui *tire* sur un élément lit `declared_deps()` ou `as_slice()` (tous les éléments visibles) ; une règle qui *se tait* parce qu'une lecture est déclarée lit `covering_deps()` (tous les éléments sauf les sources de spread). Dans ce chapitre, `always-unstable-deps` est du premier type, `missing-deps` et `stale-closure` du second.

Les deux accesseurs vivent sur `EffectInfo`, la vue « deps » qu'une passe de collecte du moteur construit à la convergence pour chaque `useEffect`, `useMemo` et `useCallback` (`src/engine/analysis_result.rs`, l. 95–120). Outre `deps`, elle porte `free_paths`, les chemins d'accès lus par le corps et non définis localement, et `deps_pinned`, les lectures qui n'apparaissent que dans une sous-expression nommée mot pour mot par les deps ; `missing-deps` en fera l'usage à la section 20.5.

20.1.5 L'environnement de sortie du rendu

Plusieurs règles évaluent une expression « telle que le rendu la voit à la fin ». Le moteur leur fournit pour cela l'*environnement de sortie*, le join ($\sqcup$) des environnements abstraits de tous les blocs Return du CFG de rendu :

Extrait 20.3 – `src/engine/analysis_result.rs`, l. 279–288 — l'environnement de sortie du rendu

```

279 pub fn exit_env(&self) -> AbstractEnv<D> {
280     self.render_cfg
281         .blocks
```

```

282     .values()
283     .filter(|b| matches!(b.term, Terminator::Return(_)))
284     .filter_map(|b| self.block_states.get(&b.id))
285     .cloned()
286     .reduce(|acc, env| acc.join(&env))
287     .unwrap_or_else(AbstractEnv::bottom)
288 }

```

Le commentaire de la méthode (`src/engine/analysis_result.rs`, l. 275–278) explique pourquoi la réduction ne part pas de `bottom` : `bottom.join(env)` envoie toute clé absente de `bottom` sur `top`. La même propriété vaut entre deux environnements de sortie : une variable liée dans l’un et absente de l’autre vaut `top` dans leur `join`. La section 20.5 montrera la conséquence concrète de ce détail sur un composant à retour anticipé.

20.2 Les règles Info : annoncer ce que l’on n’a pas vu

Les deux premières règles ne signalent aucun défaut du programme : elles signalent un défaut de *l’analyse*. Elles sont le moyen par lequel `reactant-analyzer` tient la promesse de soundness du chapitre 3 sans mentir sur sa portée. Un analyseur sound peut perdre de la précision (widening), ou renoncer à regarder une partie du programme (un hook dont il n’a pas le source) ; dans le second cas, un silence ne vaut plus preuve d’absence. Les règles INFO le disent, et le registre en tire une conséquence mécanique.

20.2.1 Un composant sur lequel l’analyse abdique

Extrait 20.4 – `/tmp/ch20/limits.tsx` : un hook de bibliothèque, des deps illisibles, un compteur qui boucle

```

1 import { useState, useEffect, useMemo } from "react";
2 import { useVendorThing } from "vendor-lib";
3
4 export function Counter({ deps }: { deps: any[] }) {
5   const [n, setN] = useState(0);
6   const thing = useVendorThing();
7   const v = useMemo(() => ({ thing }), deps);
8   useEffect(() => {
9     setN(n + 1);
10  }, [n]);
11  return <div>{n}{String(v)}</div>;
12 }

```

Trois choses échappent ici à l’analyse : le corps de `useVendorThing` (le paquet `vendor-lib` n’est pas analysé), les éléments du tableau `deps` (une variable, pas un littéral), et la valeur exacte de `n`, qui croît sans borne. La sortie, obtenue avec `-info -trace` :

```

Counter (4 hooks) limits.tsx
info analysis-limit [hook:1] (line 6:8) hook `useVendorThing` was not found in the registry. Pass its source
  ↳ file or add a HookSummary to analyse it (FN possible)
info analysis-limit [hook:2] (line 7:8) the deps argument here is not a written array, so its entries cannot be
  ↳ enumerated, and deps checks run with nothing declared (FP possible, and FN on whatever the list does gate)
warn infinite-loop [hook:0] (line 8:2) this effect keeps pushing state `n` (its deps do not provably gate it, so
  ↳ the effect can re-run every render) to new values on every run. Potential infinite render loop
  ↳ state `n` is written here [hook:3] (line 8:2)
  ↳ the abstract value of state `n` kept growing and was widened at iteration 3
warn missing-deps [hook:2] var:thing (line 7:8) `thing` is used in this memo but not in its deps array, and its
  ↳ value may change between renders
  ↳ `thing` is read here [hook:2] (line 7:8)

```

```

info widening-info (line 8:2) state `n` kept changing during analysis and was approximated to converge, so
  ↳ findings that depend on it may be imprecise
  ↳ state `n` is written here [hook:3] (line 8:2)
  ↳ the abstract value of state `n` kept growing and was widened at iteration 3
suspended analysis-limit 8 passing check(s) withheld: the analysis was truncated in this component, so they are not
  ↳ guaranteed

```

On y lit, dans l'ordre alphabétique des règles, deux INFO `analysis-limit`, le WARNING `infinite-loop` (étudié au chapitre 19), un WARNING `missing-deps` sur `thing` qui n'existe que *parce que* les `deps` sont illisibles, l'INFO `widening-info`, et une dernière ligne `suspended` que nous expliquerons à la section 20.2.4. Sans `-info`, le driver retire les INFO de la sortie (`src/driver/mod.rs`, l. 438) ; la suspension, elle, a lieu avant ce filtre.

20.2.2 widening-info : la trace du widening

Le chapitre 13 a décrit la boucle externe du point fixe : au-delà de `widen_threshold` tours (3 par défaut, `src/engine/fixpoint.rs`, l. 54), le store d'état n'est plus remplacé par sa nouvelle valeur : on lui applique le *widening* (∇), avec les seuils du composant (ADR-14). C'est à cet endroit que le moteur note, pour chaque slot qui change encore, le premier tour de widening et les effets qui l'écrivaient :

Extrait 20.5 – `src/engine/fixpoint.rs`, l. 514–529 — l'enregistrement d'un widening dans `widen_trace`

```

514     if iteration >= config.widen_threshold {
515         // widen_trace: render+effects only handler widening is not a bug.
516         // `or_insert_with` keeps the FIRST widening iteration (the most
517         // informative one for the witness chain).
518         for label in new_state_incycle.changed_labels(&state) {
519             widen_trace
520                 .entry(label)
521                 .or_insert_with(|| crate::engine::WidenEvent {
522                     iteration,
523                     writers: slot_writers.get(&label).cloned().unwrap_or_default(),
524                 });
525         }
526         state = state.widen_to(&new_state, &thresholds);
527     } else {
528         state = new_state;
529     }

```

Trois détails comptent pour la règle. D'abord, `or_insert_with` garde le *premier* tour de widening, le plus instructif pour un témoin. Ensuite, seul `new_state_incycle` est comparé, c'est-à-dire l'état produit par le rendu et les effets : un widening causé par les seuls handlers n'est pas tracé (le commentaire l'annonce : « handler widening is not a bug »). Enfin, au-delà de cent tours, le widening est forcé sur tous les labels, tous inscrits dans la trace (`src/engine/fixpoint.rs`, l. 499–512). Le type enregistré est minimal :

Extrait 20.6 – `src/engine/analysis_result.rs`, l. 20–28 — un événement de widening

```

20 /// Feeds the `infinite-loop` / `widening-info` witness chains (`Step::Widen`).
21 #[derive(Debug, Clone, Default, PartialEq, Eq)]
22 pub struct WidenEvent {
23     /// Outer fixpoint iteration at which the slot was widened (first time).
24     pub iteration: usize,
25     /// Effects whose pass wrote this slot during the widening iteration.
26     /// Empty when the growth came from render or handlers only.
27     pub writers: Vec<HookLabel>,
28 }

```

La règle elle-même est la plus courte du catalogue (41 lignes) : elle trie les labels de `widen_trace` et émet une INFO par slot, dont les notes sont produites par le producteur de témoins partagé `slot_history` (`src/rules/api/witness.rs`, l. 529), un `Step::Write` par effet écrivain puis un `Step::Widen`.

Extrait 20.7 – `src/rules/impls/widening_info.rs`, l. 15–40 — le corps de `check`, tout ce que fait la règle

```

15 fn check(&self, ctx: &RuleCtx) -> Vec<Diagnostic> {
16     let (result, component) = (ctx.program(), ctx.component());
17     let result = &result.components[&component];
18     let state_names = state_val_labels(&result.render_cfg);
19     let name_of = |l| state_slot_name(l, &state_names);
20     let mut labels: Vec<_> = result.widen_trace.keys().copied().collect();
21     labels.sort_unstable();
22     labels
23         .into_iter()
24         .map(|label| {
25             Diagnostic::info(
26                 "widening-info",
27                 format!(
28                     "state {} kept changing during analysis and was \
29                     approximated to converge, so findings that depend on it \
30                     may be imprecise",
31                     name_of(label)
32                 ),
33             )
34             // Witness (ADR-019): the engine's own record of the widening.
35             .with_notes(crate::rules::api::witness::slot_history(
36                 result, label, &name_of,
37             ))
38         })
39         .collect()
40 }

```

Le diagnostic ne porte aucune position : `Diagnostic::info` n'en reçoit pas. C'est le registre qui lui en donne une, par la fonction `located`, appliquée à tout diagnostic sans position (ADR-19, #131) :

Extrait 20.8 – `src/rules/registry.rs`, l. 364–369 — un diagnostic sans position prend celle de sa première note

```

364 fn located(mut d: Diagnostic) -> Diagnostic {
365     if d.range.is_none() {
366         d.range = d.notes.iter().find_map(|n| n.range);
367     }
368     d
369 }

```

D'où le (line 8:2) de la sortie : c'est la position de l'effet écrivain, portée par la note `Step::Write`.

Remarque

`widening-info` n'a pas de NAME associé : son nom est le littéral `"widening-info"` (`src/rules/impls/widening_info.rs`, l. 11–13). Sa documentation (`src/rules/docs.rs`) parle d'un intervalle qui « saute à $+\infty$ » ; avec les seuils d'ADR-14, le saut peut s'arrêter à une

constante du programme. Le message du diagnostic, lui, reste vrai dans les deux cas.

20.2.3 analysis-limit : sept renoncements

analysis-limit rassemble tous les endroits où le moteur a choisi de renoncer à une partie du programme pour rester sound. Chacun est une source de FN possibles, et chacun a son message. Le tableau 20.2 les énumère.

Cas	Source dans le moteur	Début du message
réursion coupée	stats.recursive_component_refs	« recursive component reference ... is not followed »
composant inconnu	stats.unknown_component_refs	« component ... was not found in the analysis registry »
composant ambigu	stats.ambiguous_component_refs	« several analysed files define a component called ... »
profondeur de call-backs	stats.callback_depth_capped	« callback inlining reached the depth cap (3) »
budget d'inlining	stats.inline_budget_exhausted	« utility inlining ran out of splice budget »
hook inconnu	hook_calls[].opaque	« hook ... was not found in the registry »
deps illisibles	EffectInfo::deps_are_opaque()	« the deps argument here is not a written array »

TABLE 20.2 – Les sept cas d'analysis-limit (src/rules/impls/analysis_limit_info.rs). La profondeur 3 est MAX_INLINE_DEPTH (src/domains/interp/interpreter.rs, l. 21).

Les cinq premiers cas lisent les statistiques du programme, AnalysisStats, présentées au chapitre 16 ; ils sont émis sans position, sans label et sans note, si bien que located n'a rien à leur donner et qu'ils s'affichent sans ligne. Les deux derniers sont propres au composant et méritent qu'on les lise.

Extrait 20.9 – src/rules/impls/analysis_limit_info.rs, l. 99–108 — le hook opaque : la condition

```

99      // Unknown custom hooks survived expand_custom_hooks (not in HookRegistry or SummaryRegistry).
100      if let Some(comp_result) = result.components.get(&component) {
101          for call in &comp_result.hook_calls {
102              // Not `kind == Custom`: a summarized library hook is a custom
103              // hook whose abstraction is known, and it keeps its row so
104              // rules-of-hooks checks can see the call site. `opaque` is the
105              // fact this Info is about.
106              if !call.opaque {
107                  continue;
108              }

```

Le message nomme le hook par sa HookEntry::Custom, ou à défaut <hook:N> (l. 109–115), et porte son label et sa position. La condition est call.opaque et non « c'est un hook personnalisé » : un hook de bibliothèque que le SummaryRegistry résume (chapitre 8) est un hook personnalisé dont l'abstraction est connue ; il garde sa ligne dans hook_calls (pour que conditional-hook voie l'appel) mais n'est pas une limite. Le cas des deps illisibles, ajouté plus tard, commente lui-même son rôle dans le système :

Extrait 20.10 – src/rules/impls/analysis_limit_info.rs, l. 131–154 — les deps illisibles

```

131      // A deps argument the engine could not read (`useMemo(fn, deps)`). The

```

```

132 // hook is gated by a list of which not one element is visible, so
133 // every deps-based rule is running blind on it – and `registry.rs`
134 // keys the suspension of "verified:" assurances off this Info, which
135 // is how a component stops publishing a universal over a hook nobody
136 // could check.
137 if let Some(comp_result) = result.components.get(&component) {
138     for info in comp_result.effect_info.values() {
139         if !info.deps_are_opaque() {
140             continue;
141         }
142         let mut d = Diagnostic::info(
143             "analysis-limit",
144             "the deps argument here is not a written array, so its entries \
145             cannot be enumerated, and deps checks run with nothing declared \
146             (FP possible, and FN on whatever the list does gate)",
147         )
148         .with_label(info.label);
149         if let Some(span) = info.span {
150             d = d.with_range(span);
151         }
152         diags.push(d);
153     }
154 }

```

Le message est remarquable par sa double annonce : « FP possible, and FN on whatever the list does gate ». Les règles qui lisent les deps pour se taire (*missing-deps*, *stale-closure*) vérifient un tel hook « rien couvert », ce qui ne peut que produire trop de findings (le thing de la sortie ci-dessus); celles qui lisent les deps pour tirer (*always-unstable-deps*) n'ont aucun élément à examiner, et c'est là que se logent les FN possibles.

Piège

Le commentaire de tête d'*analysis_limit_info.rs* annonce « Six cases » (*src/rules/impls/analysis_limit_info.rs*, l. 6–22) et la documentation générée (*reactant explain analysis-limit*) n'en cite que quatre (*src/rules/docs.rs*, l. 79–91); le code en émet sept. Pour savoir ce qu'une règle fait, on lit *check*, pas son commentaire.

20.2.4 La suspension des assurances

Sous *-info*, une règle qui s'appliquait à un composant et n'y a rien trouvé publie une *assurance* (*SafeCheck*, *src/rules/mod.rs*, l. 61–73) : une ligne *verified* au présent, « every effect declares the variables it reads ». Une assurance est un énoncé universel. Or un composant dont l'analyse a été tronquée ne peut pas en publier honnêtement : le hook opaque peut cacher l'appel conditionnel, la dep manquante, l'effet qui diverge. Le registre applique donc la règle suivante, sur la liste des diagnostics *non filtrée* :

Extrait 20.11 – *src/rules/registry.rs*, l. 287–291 — un seul *analysis-limit* retire toutes les assurances du composant

```

287 let mut suspended_safe_checks = 0;
288 if diags.iter().any(|d| d.rule == AnalysisLimitInfo::NAME) {
289     suspended_safe_checks = safe_checks.len();
290     safe_checks.clear();
291 }

```

Le commentaire qui précède ce code (*src/rules/registry.rs*, l. 276–286) donne les deux raisons de sa forme. La lecture se fait sur la liste non filtrée : *-ignore-rule analysis-limit*

masque l'avis, il ne rétablit pas la garantie (test `tests/cli.rs`, l. 330–340). Et le compte survit à l'effacement : retirer les assurances en silence rendait un composant tronqué indiscernable d'un composant qui n'avait simplement rien à vérifier. C'est la ligne `suspended ... 8 passing check(s) withheld` de la sortie.

Exemple 20.1: Des deps illisibles suffisent

```
1 export function Grid({ items, deps }: { items: number[]; deps: any[] }) {
2   const total = useMemo(() => items.length, deps);
3   return <i>{total}</i>;
4 }
```

```
Grid (1 hooks) opaque.tsx
info  analysis-limit [hook:0] (line 3:8) the deps argument here is not a written array, so its entries
      ↳ cannot be enumerated, and deps checks run with nothing declared (FP possible, and FN on whatever the list
      ↳ does gate)
warn  missing-deps [hook:0] var:items (line 3:8) `items` is used in this memo but not in its deps array,
      ↳ and its value may change between renders
      ↳ `items` is read here [hook:0] (line 3:8)
suspended analysis-limit 1 passing check(s) withheld: the analysis was truncated in this component, so they
      ↳ are not guaranteed
```

Le WARNING sur `items` est peut-être un FP (`deps` contient peut-être `items`); l'assurance qu'aurait publiée une autre règle applicable est retenue.

Décision : niveau Info et suspension par composant

INFO est, dans la doctrine du projet (CLAUDE.md), le niveau des « limites de l'analyse », affichées derrière `-info`. Une limite n'est ni un défaut certain ni un défaut possible du programme : l'afficher en WARNING aurait noyé les vrais findings dans les 28 230 « hook not found » mesurés sur le corpus (`docs/campaign/AUDIT.md`, mesure au commit `0c45de8`), pour l'essentiel dus au périmètre `node_modules` (wontfix #51). La suspension se fait par composant et non par couple (limite, règle) : un enfant non analysé coûte au parent toutes ses assurances, y compris sur son propre corps. C'est grossier mais sûr ; l'affinement est suivi par #31. Les deux règles INFO n'ont pas de `safe_check` : une limite n'a pas d'« état sûr » à publier (`src/rules/mod.rs`, l. 88–97).

20.3 conditional-hook : la règle des hooks par dominance

`conditional-hook` est la seule règle du catalogue dont l'ERROR ne dépend d'aucune valeur abstraite : elle ne lit que la *forme* du CFG de rendu. C'est aussi la plus ancienne (`src/rules/impls/conditional_hook.rs`, 581 lignes dont 535 de tests), et la meilleure introduction aux preuves `must` par dominance.

Rappel React : règles des hooks

React ne connaît pas les hooks par leur nom mais par leur *rang* : le premier `useState` d'un composant lit le premier slot de sa fibre, le deuxième hook le deuxième, et ainsi de suite (section 2.5). Si un rendu saute un hook, tous les hooks suivants lisent le slot de leur voisin ; React le détecte quand le nombre de hooks change (« Rendered fewer hooks than expected ») mais pas quand deux hooks de même forme échangent leurs slots. D'où la règle d'`eslint-plugin-react-hooks` : un hook s'appelle au niveau supérieur du composant, jamais sous une condition, dans une boucle, après un retour anticipé ou dans un callback.

20.3.1 Le problème : un retour anticipé

Extrait 20.12 – /tmp/ch20/profile.tsx : un effet placé après un retour anticipé

```
1 import { useState, useEffect } from "react";
2
3 export function Profile({ user }: { user: { name: string } | null }) {
4   const [open, setOpen] = useState(false);
5   if (!user) {
6     return <p>anonymous</p>;
7   }
8   useEffect(() => {
9     document.title = user.name;
10  }, [user.name]);
11  return <button onClick={() => setOpen(!open)}>{user.name}</button>;
12 }
```

Aucun hook n'est ici écrit dans un `if` ; c'est le `return` de la ligne 6 qui rend le `useEffect` conditionnel. Un linter qui chercherait les hooks « à l'intérieur d'une accolade de `if` » le manquerait. `reactant-analyzer` le trouve :

```
Profile (3 hooks) profile.tsx
  error conditional-hook [hook:1] (line 8:2) this hook is called conditionally (not on every render path)
    → guarded by a condition evaluated here, so some render paths skip the hook (line 5:6)
```

Le compteur « (3 hooks) » inclut le handler `onClick`, qui reçoit le label 2 : le lowering range les props JSX `onX` parmi les `HookEntry` (variante `Handler`), sans en faire des hooks au sens de React. La note pointe la condition `!user` (ligne 5, colonne 6).

20.3.2 La formulation par dominance

La question « ce hook est-il appelé à chaque rendu ? » se traduit exactement en une question de dominance (définition 3.9) : un rendu est un chemin de l'entrée du CFG à l'un de ses blocs `Return` ; le hook s'exécute sur tous les rendus si et seulement si son bloc se trouve sur tous ces chemins, c'est-à-dire s'il *domine* chacune des sorties.

Définition 20.3: Hook conditionnel

Soit G le CFG de rendu d'un composant, X l'ensemble de ses blocs `Return` *atteignables* depuis l'entrée, et b le bloc qui porte l'appel d'un hook. Le hook est *conditionnel* si $X \neq \emptyset$ et s'il existe $x \in X$ tel que b ne domine pas x . Il est *inconditionnel* si b domine tout $x \in X$.

La figure 20.1 montre les deux situations typiques. À gauche, le composant `Profile` : le bloc d'entrée domine les deux sorties, donc `useState` est inconditionnel ; le bloc du `useEffect` ne domine pas la sortie `return <p>`, donc le hook est conditionnel. À droite, un losange (`if/else` qui se referme) suivi d'un `useState` : le bloc de jonction domine l'unique sortie, et le hook est inconditionnel alors même qu'il suit une condition. Le composant `Badge` de /tmp/ch20/diamond.tsx, de cette forme, reçoit l'assurance « verified conditional-hook all hooks run unconditionally, in a stable order ».

20.3.3 Ce que fait le code

La règle délègue tout à une primitive de la surface typée (chapitre 17). Son check tient en quelques lignes :

Extrait 20.13 – src/rules/impls/conditional_hook.rs, l. 32–45 — la règle : une `ERROR` par jeton

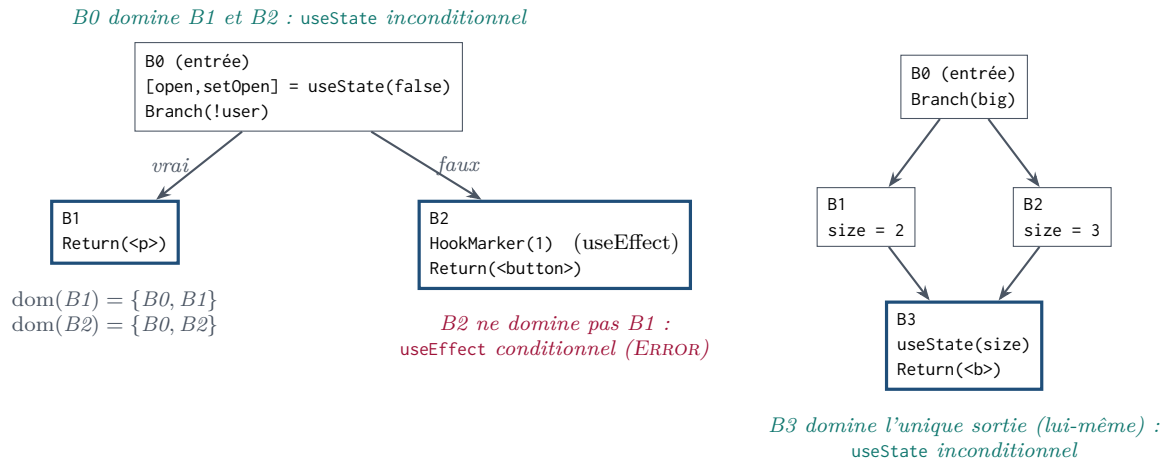


FIGURE 20.1 – Dominance et hooks conditionnels. À gauche, Profile (liste 20.12) ; à droite, un losange qui se referme avant le hook. Les sorties (blocs Return atteignables) sont en trait épais. La numérotation des blocs est illustrative : aucune sortie de la CLI n'expose les identifiants réels.

```

32 fn check(&self, ctx: &RuleCtx) -> Vec<Diagnostic> {
33     // The dominance ∇-exits check + guard witness live in the primitive; a
34     // conditional hook yields a `Certified`, the only path to `error()`.
35     ctx.hook_is_conditional()
36         .into_iter()
37         .map(|proof| {
38             Diagnostic::error(
39                 "conditional-hook",
40                 proof,
41                 "this hook is called conditionally (not on every render path)",
42             )
43         })
44         .collect()
45 }

```

Le seul constructeur d'ERROR, `Diagnostic::error`, exige un `Certified<E>` (définition 17.1) ; ici `E = ConditionalHookCall`, un label et une position (`src/rules/api/query.rs`, l. 435–441). Ce jeton, seul le module `src/rules/api/query.rs` peut le frapper, et il ne le fait que dans `hook_is_conditional` :

Extrait 20.14 – `src/rules/api/query.rs`, l. 475–511 — la primitive `hook_is_conditional`

```

475 pub fn hook_is_conditional(&self) -> Vec<Certified<ConditionalHookCall>> {
476     let cfg = &self.comp.render_cfg;
477     // One owner for "what the exits are": this used to keep its own copy of
478     // the enumeration and its own `DominatorTree`, which could disagree with
479     // `ExitDominance` about reachability.
480     let dominance = ExitDominance::of(cfg);
481     cfg_hook_calls(&self.comp)
482         .filter(|call| !matches!(call.kind, HookKind::Handler))
483         .filter(|call| dominance.may_be_skipped(call.block_id))
484         .map(|call| {
485             let mut notes = Vec::new();
486             if let Some((), guard_span) = guard_site(cfg, call.block_id) {
487                 let step = Step::Branch {
488                     desc: "a condition evaluated here, so some render paths skip the hook"
489                 }
490                 .to_string(),

```

```

490         };
491         notes.push(Note {
492             message: step.render(&super::witness::fallback_name),
493             step,
494             hook_label: None,
495             range: guard_span,
496         });
497     }
498     Certified::mint(
499         ConditionalHookCall {
500             label: call.label,
501             span: call.span,
502         },
503         Provenance {
504             range: call.span,
505             hook_label: Some(call.label),
506             notes,
507         },
508     )
509 })
510 .collect()
511 }

```

Trois étapes. (1) `ExitDominance::of` construit une fois la relation de dominance et l'ensemble des sorties. (2) Les `HookCallInfo` de kind `Handler` sont écartés : un handler JSX n'est pas un hook. (3) Pour chaque hook dont le bloc « peut être sauté », on cherche la condition à blâmer (`guard_site`), on en fait une note `Step::Branch`, et on frappe le jeton. `ExitDominance` porte la définition des sorties :

Extrait 20.15 — `src/rules/api/query.rs`, l. 652–683 — les sorties atteignables et le test `may_be_skipped`

```

652 impl ExitDominance {
653     pub fn of(cfg: &CFG) -> Self {
654         // Reachable exits only. A `Return` no path can arrive at is not an exit:
655         // lowering seals a fall-through tail as `Return(undefiend)`, and an
656         // `if`/`else` whose both branches returned leaves that tail orphaned -
657         // counting it would make every hook before the branch fail to dominate
658         // "all exits" and report as conditional at the Error tier.
659         let reachable = cfg.reachable_blocks();
660         let exits = cfg
661             .blocks
662             .values()
663             .filter(|b| matches!(b.term, Terminator::Return(_)))
664             .map(|b| b.id)
665             .filter(|id| reachable.contains(id))
666             .collect();
667         ExitDominance {
668             domtree: DominatorTree::new(cfg),
669             exits,
670         }
671     }
672
673     /// `true` when `block` may be skipped on some render path - the rule-facing
674     /// negation of [Self::certify], and deliberately not its MustResult::None
675     /// case: a CFG with no reachable exit proves nothing in either direction, so
676     /// nothing is skippable there.
677     pub fn may_be_skipped(&self, block: BlockId) -> bool {
678         !self.exits.is_empty()
679         && self
680             .exits

```

```

681         .iter()
682         .any(|&exit| !self.domtree.dominates(block, exit))
683     }

```

Piège

Pourquoi « atteignables » ? Le lowering scelle toute fin de fonction par un `Return(undefined)` (ADR-25, chapitre 7). Dans un `if/else` dont les deux branches retournent, ce bloc de fin n'a aucun prédécesseur : ce n'est pas une sortie d'un rendu réel. Le commentaire d'`ExitDominance::of` (`src/rules/api/query.rs`, l. 654–658) motive le filtre par le risque inverse : compter ce bloc rendrait tous les hooks placés avant le `if` non dominants pour cette sortie orpheline, donc conditionnels, au niveau `ERROR`. Avec l'algorithme actuel, ce risque ne se matérialiserait pas : un bloc inatteignable n'est pas visité par l'ordre post-ordre inverse et garde pour dominateurs *tous* les blocs (`src/engine/dominance.rs`, l. 16–22), si bien que tout bloc le « domine ». Le filtre n'en est pas moins juste : il rend la définition des sorties indépendante de cette convention de calcul. Le composant `Both` de `/tmp/ch20/diamond.tsx` (un `useState` puis un `if/else` qui retourne des deux côtés) est déclaré « verified ». Symétriquement, un CFG sans sortie atteignable ne prouve rien : `may_be_skipped` y répond `false`.

Où est le hook ? Le bloc d'un hook, `HookCallInfo::block_id`, est calculé à la convergence par `collect_hook_calls` : c'est le premier bloc, dans l'ordre des identifiants (les blocs vivent dans un `BTreeMap`), qui contient une expression porteuse du label (`StateVal`, `MemoVal`, `CallbackVal`, `HookMarker`...); un label absent du CFG retombe sur le bloc d'entrée (`src/engine/fixpoint.rs`, l. 1324–1354). Les hooks « sans valeur » comme `useEffect` ou `useRef` n'auraient aucune expression de ce type; le lowering les ancre par un `Expr::HookMarker`, et c'est ce marqueur qui place le `useEffect` de `Profile` dans `B2`. Avant l'introduction du marqueur, ces hooks tombaient sur le bloc d'entrée, qui domine tout : un FN historique, corrigé au commit 4f61e6d et épinglé par `tests/conditional_hook_e2e.rs`.

Quelle condition blâmer ? Le témoin choisit, parmi les dominateurs du bloc du hook terminés par un `Branch`, ceux dont au moins un successeur n'atteint *pas* le bloc, et retient le plus profond (celui qui a le plus de dominateurs) :

Extrait 20.16 – `src/rules/api/query.rs`, l. 1092–1105 — le choix de la garde (début de `guard_site`)

```

1092 fn guard_site(cfg: &CFG, block: BlockId) -> Option<(BlockId, Option<SourceRange>)> {
1093     let doms = compute_dominators(cfg);
1094     let dominators = doms.get(&block)?;
1095     dominators
1096         .iter()
1097         .filter(|&d| {
1098             d != block
1099             && matches!(
1100                 cfg.blocks.get(&d).map(|b| &b.term),
1101                 Some(Terminator::Branch { .. })
1102             )
1103             && cfg.successors(d).iter().any(|&s| !reaches(cfg, s, block))
1104         })
1105     .max_by_key(|&d| doms.get(&d).map_or(0, |s| s.len()))

```

Le filtre « un successeur qui n'atteint pas le hook » évite de blâmer un losange qui se referme *avant* le hook : ses deux branches atteignent le hook, il n'y est pour rien (test `rejoining_diamond_between_guard_and_hook_is_not_blamed`, `src/rules/impls/conditional_hook.rs`, l. 207–336). La position vient du `Branch` lui-même ou, à défaut, de la dernière instruction du

bloc garde ; si aucune n'est connue, la note est imprimée sans ligne.

Coût. `DominatorTree::new` appelle `compute_dominators`, l'algorithme itératif par ensembles en ordre post-ordre inverse (chapitre 12) ; chaque requête `dominates` n'est ensuite qu'un test d'appartenance à un ensemble de dominateurs (`src/engine/dominance.rs`, l. 107–110). `guard_site`, lui, recalcule les dominateurs et lance un parcours `reaches` par successeur candidat : c'est une double construction de la même relation, négligeable sur des CFG de rendu de quelques dizaines de blocs, mais à connaître.

20.3.4 Niveau et preuve

La règle émet toujours une `ERROR`, et c'est la seule du chapitre à le faire sans condition. La preuve est structurelle : il existe un chemin du CFG de rendu, de l'entrée à une sortie atteignable, qui ne passe pas par le hook. Le jeton garantit que la règle ne peut pas conclure sur une quantification plus faible : `hook_is_conditional` empaquette le « pour toute sortie » pour qu'aucune règle ne puisse le sous-quantifier (ADR-21 §3).

Remarque

La preuve porte sur les chemins du CFG, pas sur leur faisabilité : un `if (true) return;` placé avant un hook donnerait le même verdict qu'un `if (!user)`. C'est la lecture de la règle des hooks elle-même, qui est syntaxique ; `eslint-plugin-react-hooks` ne raisonne pas autrement.

20.3.5 Cas limites observés

Exemple 20.2: Boucle, exception, branches qui retournent

Le fichier `/tmp/ch20/diamond.tsx` regroupe quatre composants. `Badge` (losange puis `useState`) et `Both` (`useState` puis `if/else` qui retournent) sont « verified ». `Thrower` commence par `if (!on) throw new Error("off");` puis appelle `useState` : il est « verified » lui aussi, parce qu'un bloc qui lève n'est pas une sortie `Return` ; un rendu qui lève n'a pas de suite, et ne peut donc pas décaler les hooks d'un rendu ultérieur. C'est exact pour `hook_is_conditional`, dont la définition ne porte que sur les `Return` atteignables ; que ce choix s'aligne sur la lecture qu'`eslint-plugin-react-hooks` fait d'un chemin qui lève reste, au commit e67b10a, une affirmation non vérifiée côté `eslint`. `Loop` appelle `useEffect` dans un `for (const x of xs)` :

```
Loop (1 hooks) diamond.tsx
  error conditional-hook [hook:0] (line 30:4) this hook is called conditionally (not on every render path)
    → guarded by a condition evaluated here, so some render paths skip the hook
```

Le Branch de la boucle n'a pas de position : la note sort sans ligne.

Piège

Un hook que l'IR ne voit pas n'est pas vérifié. Le fichier `/tmp/ch20/s4.tsx` compare quatre écritures du même défaut :

```
1 export function A({ on }: { on: boolean }) {
2   let v = 0;
3   if (on) { const w = useMemo(() => 1, []); v = w; }
4   return <i>{v}</i>;
5 }
6 export function B({ on }: { on: boolean }) {
7   let v = 0;
8   if (on) { v = useMemo(() => 1, []); }
9   useEffect(() => {}, []);
```

```

10   return <i>{v}</i>;
11 }
12 export function C({ on }: { on: boolean }) {
13   if (on) { useEffect(() => {}, []); }
14   return <i />;
15 }
16 export function D({ on }: { on: boolean }) {
17   const v = on && useMemo(() => 1, []);
18   useEffect(() => {}, []);
19   return <i>{String(v)}</i>;
20 }

```

A et C reçoivent l'ERROR attendue. B et D affichent « (1 hooks) » et l'assurance « verified conditional-hook » : le `useMemo` placé en membre droit d'une *affectation* ou d'un `&&` ne produit aucun `HookEntry`. C'est un FN sur une règle de niveau ERROR, constaté au commit e67b10a ; sa cause est dans l'extraction des hooks (chapitre 7), pas dans la règle, qui ne peut juger que ce que l'IR lui donne. Le dossier de notes signale la même absence pour un hook dans une branche de ternaire, et ne trouve pas d'issue qui la suive.

L'assurance. Le `safe_check` déclare la règle applicable dès que `hook_calls` n'est pas vide (`src/rules/impls/conditional_hook.rs`, l. 19–30). Comme `hook_calls` contient aussi les handlers, un composant sans aucun hook mais avec un `onClick` reçoit « verified conditional-hook all hooks run unconditionally, in a stable order » (composant `OnlyHandler`, `/tmp/ch20/onlyhandler.tsx`) : une assurance vide de contenu, vraie mais sans objet.

Décision : ADR-021 : la dominance comme primitive

Avant ADR-21, `conditional_hook.rs` gardait sa propre énumération des sorties et son propre `DominatorTree`, qui pouvaient diverger d'`ExitDominance` sur l'atteignabilité (le commentaire de `src/rules/api/query.rs`, l. 477–479 en garde la trace). La décision : un seul propriétaire de « ce que sont les sorties », une primitive qui rend des `Certified<ConditionalHookCall>`, et une règle qui ne fait plus que les convertir en diagnostics. L'alternative, laisser chaque règle calculer sa dominance, rendait possible une règle qui teste « domine *une* sortie » au lieu de « domine *toutes* les sorties », c'est-à-dire une ERROR sur une quantification fausse.

20.4 always-unstable-deps : quand Object.is échoue à chaque rendu

Avec `always-unstable-deps`, on quitte la forme du CFG pour les valeurs : la règle interroge le domaine de stabilité du chapitre 10. C'est aussi la première règle à lire une liste de deps, et elle le fait dans la polarité « tirer » (définition 20.2).

Rappel React : Object.is et références fraîches

React compare chaque entrée de deps à l'entrée de même rang du rendu précédent par `Object.is`, et relance le hook dès qu'*une* entrée diffère : c'est un « ou » logique sur les entrées. Les primitifs (nombres, booléens, chaînes) sont comparés par valeur ; un littéral objet, tableau ou fonction évalué pendant le rendu est une référence neuve à chaque rendu, et fait donc échouer la comparaison à chaque rendu, quels que soient ses voisins. `useMemo`, `useCallback` et `useRef` existent pour rendre une identité stable ; le setter d'un `useState` et l'objet d'un `useRef` sont stables par contrat, et un état objet garde son identité tant que son setter n'écrit pas.

20.4.1 Le problème : un objet frais à côté d'un memo

Extrait 20.17 – /tmp/ch20/feed.tsx : la même dep, fraîche puis mémorisée

```
1 import { useState, useEffect, useMemo } from "react";
2
3 export function Feed({ userId }: { userId: string }) {
4   const [count, setCount] = useState(0);
5   const opts = { userId, limit: 20 };
6   useEffect(() => {
7     fetchPosts(opts);
8   }, [opts, count]);
9   const stable = useMemo(() => ({ userId }), [userId]);
10  useEffect(() => {
11    fetchPosts(stable);
12  }, [stable, count]);
13  return <button onClick={() => setCount(count + 1)}>{count}</button>;
14 }
```

Feed (5 hooks) feed.tsx

```
warn always-unstable-deps [hook:1] (line 6:2) this effect depends on `opts`, a new reference every render, so `
  ↳ Object.is` always differs and the effect re-runs on every render regardless of the other deps
  ↳ the value flows through `opts`, bound here (line 5:8)
```

Le premier effet dépend de `opts`, un littéral objet évalué à chaque rendu : il tourne donc à chaque rendu, et la présence de `count` à côté n'y change rien. Le second dépend de `stable`, la valeur d'un `useMemo` : le memo absorbe la fraîcheur de l'allocation, et l'effet est muet. `count` est un nombre, comparé par valeur. La note, produite par le producteur de témoins `chase_value` (`src/rules/api/witness.rs`, l. 463), remonte d'une liaison vers l'origine de la référence.

20.4.2 La borne `must PerRender`

Le treillis *Stability* (ADR-17, définition 10.3) distingue deux sortes de bornes sur la *trace de changement* d'une valeur, c'est-à-dire l'ensemble des rendus où `Object.is` avec le rendu précédent échoue :

Extrait 20.18 – `src/domains/impls/stability.rs`, l. 35–52 — le treillis de stabilité référentielle

```
35 #[derive(Debug, Clone, PartialEq, Eq)]
36 pub enum Stability {
37   ///  $\perp$  no information (unreachable path / uninitialized).
38   Bottom,
39   /// Never changes: the same reference on every render (safe as a dep).
40   Stable,
41   /// Changes only at setter events of these state slots (may bound).
42   /// Invariant: non-empty (canonicalised – `Versioned(()) ≡ Stable`) and
43   /// `len() ≤ VERSIONED_LABELS_THRESHOLD` (widened to `VersionedTop` above).
44   Versioned(BTreeSet<QualifiedSlot>),
45   /// Versioned by unknown state slots (threshold-widened `Versioned`).
46   VersionedTop,
47   /// A fresh reference every render, guaranteed (must bound).
48   /// For non-reference kinds via `to_stability`: "may change every render".
49   PerRender,
50   ///  $\top$  no bound in either direction.
51   Unknown,
52 }
```

`Versioned(S)` est une borne *may* : la valeur ne change *qu'aux* écritures des slots de *S*, peut-être moins. Elle sert à se taire. `PerRender` est une borne *must* : la référence change à *chaque*

rendu. Elle sert à tirer. La règle ne tire que sur ce dernier fait, et seulement pour une valeur qui n'est *que* une référence :

Extrait 20.19 – src/domains/impls/state_value.rs, l. 281–287 — le prédicat de la règle

```

281    /// True when the value is exactly a per-render-fresh reference (nothing
282    /// else) – a *must* fact: a new reference every render, guaranteed.
283    /// `Versioned` (changes only at setter events) deliberately returns false
284    /// (ADR-017).
285    pub fn is_unstable_reference_only(&self) -> bool {
286        self.reference == Stability::PerRender && self.populated_kinds().only(KindMask::REF)
287    }

```

La seconde conjonction (`populated_kinds().only(KindMask::REF)`) exclut les valeurs mixtes du produit `StateValue` (définition 10.1) : une dep qui vaut « un objet ou null » n'est pas une référence neuve à coup sûr.

20.4.3 Ce que fait le code

Extrait 20.20 – src/rules/impls/always_unstable_deps.rs, l. 71–109 — la boucle de la règle

```

71    for hook in &result.hooks {
72        let (label, deps, kind, span) = match hook {
73            HookEntry::Effect {
74                label, deps, span, ..
75            } => (*label, deps, HookKind::Effect, *span),
76            HookEntry::Memo {
77                label, deps, span, ..
78            } => (*label, deps, HookKind::Memo, *span),
79            HookEntry::Callback {
80                label, deps, span, ..
81            } => (*label, deps, HookKind::Callback, *span),
82            _ => continue,
83        };
84        let Some(deps_ref) = deps.list().map(DepsList::as_slice) else {
85            continue;
86        };
87
88        if deps_ref.is_empty() {
89            continue;
90        }
91
92        let unstable: Vec<&Expr> = deps_ref
93            .iter()
94            .filter(|dep| {
95                eval_dep_is_unstable(
96                    result.component,
97                    dep,
98                    &env_exit,
99                    &result.state_store,
100                    &result.memo_store,
101                    &mut scratch,
102                    &transfer,
103                )
104            })
105            .collect();
106
107        if unstable.is_empty() {
108            continue;
109        }

```

On y lit quatre décisions.

1. *Pas de liste, pas de finding.* `deps.list()` ne rend rien pour `Absent` (le hook tourne déjà à chaque rendu) ni pour `Opaque` (aucun élément à examiner). Ce second cas est l'un des FN annoncés par `analysis-limit`.
2. *Pas de finding sur []*. Un effet de montage n'a aucune entrée à comparer.
3. *Tous les éléments visibles.* `DepsList::as_slice` rend `elems`, y compris la source d'un spread : la règle *tire* sur un élément, c'est la polarité où lire `elems` est `sound` (invariant 20.1).
4. *Un tas convergé.* La dep est évaluée par `eval_in_stores` (l. 160 du même fichier) dans l'environnement de sortie, avec les stores convergés et une copie du tas convergé, faite une fois par composant (`result.heap.clone()`, l. 69). Un tas vide répondrait `top` à toute lecture de membre (`form.onSubmit`), que la règle lit comme un silence : c'était le FN de #135 (test `a_member_dep_holding_a_fresh_function_fires`, `tests/always_unstable_deps.rs`, l. 330).

Le message nomme les deps par leur expression, jamais par leur position (`fmt_deps`, `src/rules/impls/always_unstable_deps.rs`, l. 167–181) : une position dans `elems` cesse d'être une position dans le tableau source dès que le lowering retire une élision ou aplatit un spread (#118).

20.4.4 Ce que la règle tait, et pourquoi

Les primitifs. Un compteur convergé vers $[0, 10]$, ou porté par le widening à $[0, +\infty)$, n'est jamais signalé : sa valeur varie, mais `Object.is` compare des nombres par valeur. Le commentaire de tête de la règle l'explique : traiter un large intervalle comme instable confondrait la variance du point fixe avec la nouveauté référentielle, et ferait tirer la règle sur la dep canonique `[count]` (`src/rules/impls/always_unstable_deps.rs`, l. 20–25 ; test `wide_numeric_state_dep_not_flagged`).

L'état objet. C'est le cas qui a motivé ADR-17.

Exemple 20.3: Un état objet n'est pas une référence fraîche

```
1 export function Locale() {
2   const [ctx, setCtx] = useState({ locale: "en" });
3   useEffect(() => {
4     applyLocale(ctx);
5   }, [ctx]);
6   return <button onClick={() => setCtx({ locale: "fr" })}>{ctx.locale}</button>;
7 }
```

Le store d'état contient le join des valeurs *écrites* : deux littéraux objets, donc des références `PerRender` au moment de leur allocation. Mais un rendu *lit* le slot, et React garantit que son identité ne change qu'aux écritures du setter. La conversion côté lecture d'ADR-17 (chapitre 11) transforme donc la lecture de `ctx` en `Versioned((Locale, 0))`. `is_unstable_reference_only` rend `false`, et reactant `check /tmp/ch20/objstate.tsx -info` publie « verified always-unstable-deps no deps array is defeated by an always-fresh reference ». L'effet ne tourne en effet qu'aux clics.

top. Une dep dont la valeur est inconnue (une prop, par défaut) ne tire pas : `top` n'est pas une borne `must`. C'est une perte de précision assumée ; elle ne coûte rien en soundness parce que la règle est un conseil (`WARNING`) et que la seule conséquence grave d'une dep toujours neuve, la boucle de rendu, est l'affaire d'infinite-loop.

Décision : ADR-017 : Versioned se tait, et ce silence est couplé

Le banc du corpus du 15 juillet 2026 (« F5 ») laissait cinq WARNING `always-unstable-deps`, tous de la forme de l'exemple 20.3 : un provider de contexte à état objet. La cause était une confusion entre fraîcheur *à l'allocation* (un fait sur une écriture) et fraîcheur *par rendu* (un fait inter-rendus sur un slot). ADR-17 les sépare, introduit `Versioned(S)` et fait tirer la règle sur `PerRender` seulement (ADR-17 §4). Deux alternatives ont été refusées : rendre les lectures d'état « à peu près stables » sans nouvelle valeur du treillis (cela créait un FN, le composant `ObjChurn` qui réécrit son état avec un objet neuf à chaque effet), et compter les événements d'écriture dans le store. Le point subtil est l'argument de soundness n° 2 de l'ADR : la garde « une dep `Versioned` borne l'effet », utilisée par `infinite-loop`, n'est sound *que* couplée au bras *churn* de cette règle, qui couvre exactement le cas `setX(...x)` (chapitre 15, chapitre 19). Retirer l'un sans l'autre rouvre le FN.

20.4.5 Niveau : un fait certain au coût incertain

La règle plafonne à WARNING, par construction : elle n'appelle que `Diagnostic::warn`. Le fait qu'elle affirme est pourtant certain (la référence est neuve à chaque rendu, c'est une borne `must`) ; ce qui ne l'est pas, c'est le coût. Un `useMemo` recalculé à chaque rendu gaspille du travail, un effet relancé à chaque rendu peut n'être qu'un `console.log`. La doctrine du projet range précisément ce cas en WARNING : « fait certain au coût incertain ». L'escalade vers l'ERROR, quand l'effet réécrit ce qui alimente sa propre dep, appartient au bras *churn* d'`infinite-loop` (ADR-17 §3).

Piège

Tirer sur la source d'un spread peut être un FP. « Lire `elems` pour tirer est sound » signifie « ne produit pas de FN », pas « ne produit pas de FP » :

```
1 export function Rows({ a, b }: { a: number; b: number }) {
2   const rows = [a, b];
3   useEffect(() => { console.log(rows); }, [...rows]);
4   return <i />;
5 }
```

```
Rows (1 hooks) spread2.tsx
  warn  always-unstable-deps [hook:0] (line 4:2) this effect depends on `rows`, a new reference every render,
    ↳ so `Object.is` always differs and the effect re-runs on every render regardless of the other deps
    ↳ the value flows through `rows`, bound here (line 3:8)
  warn  missing-deps [hook:0] var:rows (line 4:2) `rows` is used in this effect but not in its deps array,
    ↳ and it is recreated on every render
    ↳ `rows` is read here [hook:0] (line 4:2)
```

React compare ici `a` et `b`, deux nombres ; l'effet ne tourne pas à chaque rendu, et le WARNING `always-unstable-deps` est un FP (observé au commit `e67b10a` sur `/tmp/ch20/spread2.tsx`). Il est toléré : un WARNING, sur un motif rare, dans la direction permise. Le `missing-deps`, lui, est juste : le corps lit `rows` entier, que `[...rows]` ne couvre pas (section 20.5).

L'assurance. Elle est publiée quand un `EffectInfo` au moins a des deps visibles non vides (!`declared_deps().is_empty()`, `src/rules/impls/always_unstable_deps.rs`, l. 44–59) : « no deps array is defeated by an always-fresh reference ».

20.5 missing-deps : ce que la closure lit et que les deps taisent

missing-deps est la règle la plus bavarde du catalogue (4128 findings sur le corpus au commit 451ed75, docs/campaign/AUDIT.md) et celle dont les silences sont les plus travaillés. Elle est l'analogue d'exhaustive-deps d'eslint-plugin-react-hooks, avec une différence de fond : là où eslint exige toute valeur réactive, missing-deps pardonne ce qu'elle peut *prouver* stable.

Rappel React : closures et deps

Une fonction créée pendant un rendu capture les const de ce rendu. Le corps d'un `useEffect(f, deps)` est une telle fonction : quand les deps ne changent pas, React garde l'effet du rendu précédent, qui continue de voir les valeurs de *ce* rendu. Une valeur lue par le corps et absente des deps n'est pas comparée ; si elle change, le corps travaille sur une copie périmée. Il en va de même pour la fonction mémorisée par `useCallback` et la valeur calculée par `useMemo`.

20.5.1 Le problème : trois lectures, trois verdicts

Extrait 20.21 – /tmp/ch20/search.tsx : une prop oubliée, un ref atteint par un objet frais, un objet non déclaré

```
1 import { useState, useEffect, useRef, useCallback } from "react";
2
3 export function Search({ url, settings }: { url: string; settings: any }) {
4   const [n, setN] = useState(0);
5   const r = useRef(0);
6   const bag = { r };
7   useEffect(() => {
8     fetch(url + n);
9   }, [n]);
10  const log = useCallback(() => {
11    console.log(bag.r.current);
12    setN(0);
13  }, []);
14  useEffect(() => {
15    render(settings.halfnote, settings.material);
16  }, []);
17  return <button onClick={log}>{n}</button>;
18 }
```

```
Search (6 hooks) search.tsx
warn  missing-deps [hook:2] var:url (line 7:2) `url` is used in this effect but not in its deps array, and its
    ↳ value may change between renders
    ↳ `url` is read here [hook:2] (line 7:2)
warn  missing-deps [hook:4] var:settings (line 14:2) `settings` is used in this effect but not in its deps array
    ↳ , and its value may change between renders
    ↳ `settings` is read here [hook:4] (line 14:2)
```

Le premier effet lit `fetch`, `url` et `n` ; `n` est déclaré, `fetch` est une globale, `url` est une prop (top, donc « may change ») : un finding. Le callback lit `bag.r.current` et `setN` ; `bag` est un objet neuf à chaque rendu, et pourtant la règle se tait. Le dernier effet lit deux membres de `settings` et la liste ne nomme rien sous `settings` : un seul finding, sur la racine. Ces trois verdicts mobilisent les trois notions de cette section : les chemins d'accès, la couverture par préfixe, et les kills de stabilité.

20.5.2 Chemins d'accès et couverture par préfixe

La granularité de la règle n'est pas la variable mais le *chemin d'accès* (chapitre 6) :

Extrait 20.22 – `src/ir/free_vars.rs`, l. 6–23 — un chemin d'accès : une racine et une suite de champs

```

6  /// A read access rooted at a variable, refined by a chain of field names.
7  ///
8  /// `x` → `{root: x, segments: []}`; `x.a.b` → `{root: x, segments: [a, b]}`.
9  /// This is the granularity `missing-deps` matches at (TODO.md F1b): a dep
10 /// `[x.b]` covers a use of `x.a` only if it is a *prefix* of it, so `use(x.a)`
11 /// with deps `[x.b]` is (correctly) reported while `use(x.b)` with `[x.b]`
12 /// and `use(x.a)` with `[x]` are not.
13 ///
14 /// Dynamic member access (`x[i]`) cannot name the touched element, so on the
15 /// *use* side the path stops at the last named handle above the index:
16 /// `x.a[i].b` → `{root: x, segments: [a]} - "touches all of `x.a`", coverable
17 /// by a whole-`x` or whole-`x.a` dep but not by `x.a.b`. On the *dep* side such
18 /// an access declares nothing (see [ `dep_paths` ]).
19 #[derive(Clone, PartialEq, Eq, PartialOrd, Ord, Hash, Debug)]
20 pub struct AccessPath {
21     pub root: Var,
22     pub segments: Vec<String>,
23 }

```

Côté lecture, un index dynamique coupe le chemin au-dessus de l'index : `x.a[i].b` devient `x.a`, « touche tout `x.a` ». Côté deps, il ne déclare rien. La couverture est un test de préfixe :

Extrait 20.23 – `src/ir/free_vars.rs`, l. 253–278 — les chemins déclarés et la couverture

```

253 /// Access paths *declared* by a deps array. A dep with a dynamic index
254 /// (`x[i]`) declares nothing coverable – we can't prove which element it
255 /// pins – so it is dropped (leaning to a false positive, never a negative).
256 /// Non-variable deps (literals, calls) yield no path.
257 pub fn dep_paths(deps: &[Expr]) -> Vec<AccessPath> {
258     deps.iter()
259         .filter_map(|e| {
260             let mut side = Vec::new();
261             match extract_path(e, &mut side) {
262                 Some((root, segments, opaque)) if !opaque => Some(AccessPath { root, segments }),
263                 _ => None,
264             }
265         })
266         .collect()
267 }
268
269 /// A used path is covered when some declared path is a *prefix* of it:
270 /// `[x]` covers `x.a`, `[x.a]` covers `x.a` and `x.a.b`, but `[x.b]` covers
271 /// neither `x.a` nor whole `x`.
272 pub fn path_covered(used: &AccessPath, declared: &[AccessPath]) -> bool {
273     declared.iter().any(|d| {
274         d.root == used.root
275         && d.segments.len() <= used.segments.len()
276         && used.segments[..d.segments.len()] == d.segments[..]
277     })
278 }

```

Exemple 20.4: Préfixes et index

Le fichier `/tmp/ch20/paths.tsx` lit `x.a` sous quatre listes. P1 : `[x.b]` ne couvre pas `x.a`; P2 : `[x]` couvre `x.a`; P3 : `[x.a.b]` est plus long que `x.a` et ne le couvre pas; P4 lit `x.a[i].b` sous `[x.a, i]`, coupé en `x.a` (et `i`), couvert.

P1 (1 hooks) paths.tsx

```

warn missing-deps [hook:0] var:x (line 4:2) `x.a` is used in this effect but not in its deps array, and
  ↳ its value may change between renders
  ↳ `x.a` is read here [hook:0] (line 4:2)
P2 (1 hooks) paths.tsx ✓
P3 (1 hooks) paths.tsx
warn missing-deps [hook:0] var:x (line 12:2) `x.a` is used in this effect but not in its deps array, and
  ↳ its value may change between renders
  ↳ `x.a` is read here [hook:0] (line 12:2)
P4 (1 hooks) paths.tsx ✓

```

Les tests unitaires `sibling_field_mismatch_warns` et `whole_var_dep_covers_field_use` (`src/rules/impls/missing_deps.rs`, l. 403, l. 437) épinglent les deux premiers cas.

Les chemins lus viennent de l'EffectInfo du hook, construit par le moteur à la convergence (`collect_effect_info`, `src/engine/fixpoint.rs`, l. 1425–1495) :

Extrait 20.24 – `src/engine/analysis_result.rs`, l. 95–120 — la vue « deps » d'un effet, d'un memo ou d'un callback

```

95 /// Captured information about a hook body with deps (useEffect, useMemo, useCallback)
96 /// for dep-checking rules.
97 #[derive(Debug, Clone)]
98 pub struct EffectInfo {
99     pub label: HookLabel,
100     /// Which hook this info came from (Effect / Memo / Callback).
101     pub kind: HookKind,
102     /// Access paths read in the body but not locally defined within it –
103     /// member-chain granular (`x.a`, not just `x`) so `missing-deps` matches
104     /// a dep against the exact field used (TODO.md F1b).
105     pub free_paths: HashSet<AccessPath>,
106     /// The subset of `free_paths` that occurs only inside a sub-expression
107     /// the deps array names verbatim – `searchParams.get` where the list holds
108     /// `searchParams.get("sort")`. Covered by construction, and kept apart from
109     /// `free_paths` because a consumer of this hook's value asks the other
110     /// question: what the closure holds still includes a pinned read.
111     pub deps_pinned: HashSet<AccessPath>,
112     /// Deps argument as the IR could read it – see ['DepsArg'], whose three
113     /// states are three different facts. One field rather than a list plus a
114     /// present-flag: the two could disagree, and the reading they used to
115     /// disagree about (an unreadable argument shown as an empty array that is
116     /// definitely present) was wrong in both directions.
117     pub deps: DepsArg,
118     /// Source location of the hook call site, if available.
119     pub span: Option<SourceRange>,
120 }

```

Deux champs demandent une explication. `free_paths` est calculé par `compute_free_paths`, qui retire les racines définies dans le corps et résout ses renommages : `const c = cond` n'est pas une lecture de `cond`, mais `const c = pick(cond)` en est une. `deps_pinned` contient les lectures qui n'apparaissent *que* dans une sous-expression écrite mot pour mot dans les deps : `searchParams.get("sort")` lu dans le corps et déclaré tel quel dans la liste. Une telle lecture est comparée par le tableau lui-même, elle ne peut pas être périmée. Le mot pour mot est toute la condition : `[JSON.stringify(o)]` ne *pin* pas `o`, parce qu'une sérialisation perd de l'information (`src/ir/free_vars.rs`, l. 120–137).

```

Lossy (1 hooks) pin.tsx
warn missing-deps [hook:0] var:o (line 9:2) `o` is used in this effect but not in its deps array, and its value
  ↳ may change between renders
  ↳ `o` is read here [hook:0] (line 9:2)
Sorted (1 hooks) pin.tsx ✓

```

20.5.3 L'algorithme

Voici la boucle principale de check, élaguée de ses commentaires (les commentaires originaux contiennent le caractère « ... », que la chaîne de compilation du manuscrit ne sait pas encore rendre dans un listing) :

Extrait 20.25 – La boucle de missing-deps, élaguée des commentaires (src/rules/impls/missing_deps.rs, l. 49–98)

```

1      for (label, info) in &result.effect_info {
2          if !info.has_deps_array() {
3              continue;
4          }
5
6          let declared: Vec<AccessPath> = dep_paths(&info.covering_deps());
7
8          let declared_roots: HashSet<&Var> = declared.iter().map(|d| &d.root).collect();
9          let mut reported: BTreeSet<AccessPath> = BTreeSet::new();
10
11         for path in &info.free_paths {
12             if path_covered(path, &declared) || info.deps_pinned.contains(path) {
13                 continue;
14             }
15             if !env_exit.contains(&path.root) {
16                 continue;
17             }
18             if !env_exit.lookup(&path.root).is_stable()
19                 && !member_is_stable(path, &env_exit, result)
20                 && !closure_is_behaviorally_stable(path, result, &env_exit, &mut HashSet::new())
21             {
22                 reported.insert(if declared_roots.contains(&path.root) {
23                     path.clone()
24                 } else {
25                     AccessPath::root(path.root.clone())
26                 });
27             }
28         }

```

Sa structure est celle de l'algorithme 7. Chaque ligne qui fait continue est un kill (définition 20.1), et chacun doit être une preuve que la capture ne peut pas être périmée.

Algorithme 7 : missing-deps (lecture de src/rules/impls/missing_deps.rs, l. 49–128)

Input : R : l'AnalysisResult convergé d'un composant ; E : son environnement de sortie

```

1  foreach hook  $h$  de  $R$  muni d'un argument de deps (List ou Opaque) do
2       $D \leftarrow \text{dep\_paths}(\text{covering}(h.\text{deps}))$  ; // Opaque :  $D = \emptyset$ 
3       $\text{rapport} \leftarrow \emptyset$ ;
4      foreach chemin  $p \in h.\text{free\_paths}$  do
5          if  $p$  couvert par  $D$  ou  $p \in h.\text{deps\_pinned}$  then continuer;
6          if  $\text{racine}(p) \notin \text{dom}(E)$  then continuer (globale);
7          if  $E(\text{racine}(p))$  stable then continuer (kill 1);
8          if un préfixe non vide de  $p$  s'évalue stable then continuer (kill 2);
9          if  $p$  nomme une closure dont les captures sont stables then continuer (kill 3);
10         ajouter à  $\text{rapport}$  :  $p$  si  $D$  nomme sa racine, sinon  $\text{racine}(p)$ ;
11     émettre un WARNING par élément de  $\text{rapport}$  (ensemble ordonné);

```

Polarité. La liste déclarée est `covering_deps()`, pas `declared_deps()` : elle décide de ce qu'on *ne* rapporte *pas* (invariant 20.1). Le composant `Spread` de `/tmp/ch20/beh.tsx` lit `rows` sous `[...rows]`; `covering` retire la source du `spread`, et la règle tire, comme `React` le ferait :

```
Spread (1 hooks) beh.tsx
warn missing-deps [hook:0] var:rows (line 10:2) `rows` is used in this effect but not in its deps array, and
  ↳ its value may change between renders
    ↳ `rows` is read here [hook:0] (line 10:2)
```

Deps illisibles. Un argument `opaque` passe le premier test (`has_deps_array()` vaut `true` pour tout argument écrit) et `covering_deps()` est vide : le hook est vérifié « rien couvert ». C'est le items de l'exemple 20.1, un FP possible, jamais un FN. Le lire comme `Absent` aurait sauté un hook qui peut se périmer.

Globales. Une racine absente de l'environnement de sortie (`fetch`, `console`, `document`) n'est pas une valeur du composant : elle n'est pas rapportée.

Granularité du rapport. Si la liste nomme au moins un chemin sous la racine de p , c'est p lui-même qui est rapporté (le correctif est par membre); sinon c'est la racine (le correctif est de déclarer l'objet). D'où `x.a` pour `P1` et `settings`, une seule fois, pour `Search`.

20.5.4 Les trois kills de stabilité

Kill 1 : la racine est stable. `env_exit.lookup(root).is_stable()` : un setter, un ref, un littéral, une constante de module. Une valeur qui ne change jamais ne peut pas être périmée. Attention : `Versioned` *n'est pas* `Stable`. Un état reste une dep requise, comme dans `eslint` (ADR-17 §4).

Kill 2 : un préfixe est stable. C'est le cas de `bag.r.current`.

Extrait 20.26 – `src/rules/impls/missing_deps.rs`, l. 139–157 — un préfixe stable suffit

```
139 fn member_is_stable(
140     path: &AccessPath,
141     env_exit: &AbstractEnv<StateValue>,
142     result: &crate::engine::AnalysisResult<StateValue>,
143 ) -> bool {
144     // Every prefix, not just the whole path. A read is stale only when every
145     // handle it passes through can change between renders: `bag.ref.current`
146     // reaches a stable ref at `bag.ref`, so the stale copy of `bag` a closure
147     // holds still reaches that ref and reads its current value. Stopping at
148     // the full path answers True for any current tail (a ref cell is not
149     // heap-modelled), and stopping at the root is what the caller already did.
150     //
151     // Same direction as the root check it backs up: this only ever removes
152     // findings, and it removes them for the same reason the root check does –
153     // the capture is provably not stale (#88, and the 2,010 corpus rows where
154     // the container was fresh and the member was not).
155     let mut eval = result.evaluator();
156     (1..=path.segments.len()).any(|n| eval.at(env_exit, &path.prefix_expr(n)).is_stable())
157 }
```

Le raisonnement du commentaire mérite d'être déplié, et la figure 20.2 le fait sur l'exemple. Une lecture n'est périmée que si *chaque* poignée qu'elle traverse peut changer entre rendus.

bag est un objet neuf à chaque rendu (PerRender) ; mais la copie de bag que tient une closure périmée contient le *même* ref r, puisque bag.r est stable, et .current lit la valeur courante de ce ref. S'arrêter au chemin entier ne suffit pas : une queue .current répond top, la cellule d'un ref n'étant pas modélisée dans le tas. S'arrêter à la racine, c'est ce que le kill 1 faisait déjà. Le kill parcourt donc tous les préfixes non vides.

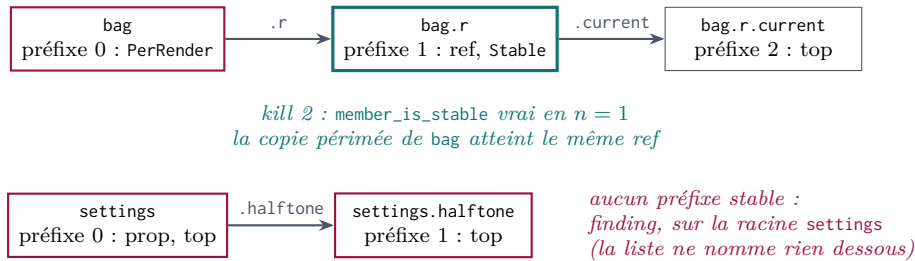


FIGURE 20.2 – Les préfixes de deux chemins lus par Search (liste 20.21), annotés de la stabilité de leur valeur dans l’environnement de sortie. Un cadre vert est un préfixe stable, qui suffit à éteindre le finding ; un cadre rouge, un préfixe qui peut changer. Le préfixe 0 (la racine) relève du kill 1 ; le kill 2 examine les préfixes $n = 1, \dots, |p|$.

L’évaluation passe par `result.evaluator()`, l’évaluateur post-convergence du chapitre 12, avec le tas convergé : c’est le même correctif (#135) que pour `always-unstable-deps`. Le commentaire cite l’ordre de grandeur qui l’a motivé : 2010 rangées du corpus où le conteneur était frais et le membre ne l’était pas (#88, #133).

Kill 3 : la closure se comporte de façon stable. C’est le plus subtil, et il sépare nettement cette règle de la précédente.

Extrait 20.27 – `src/rules/impls/missing_deps.rs`, l. 159–181 — identité contre comportement

```

159 /// Identity vs behavior (ADR-017 framing): this rule guards against *stale
160 /// closures*, so what matters is whether the values a captured function
161 /// closes over can change between renders – not whether the function's
162 /// identity does. `const cb = () => setX(1)` is a fresh reference every
163 /// render (PerRender), yet omitting it from a deps array is harmless when
164 /// every value it captures is Stable: the stale copy behaves identically.
165 /// Identity-based rules (`always-unstable-deps`, the `infinite-loop` churn
166 /// arm) must keep reading PerRender – deps arrays compare by `Object.is`.
167 fn closure_is_behaviorally_stable(
168     path: &AccessPath,
169     result: &crate::engine::AnalysisResult<StateValue>,
170     env_exit: &AbstractEnv<StateValue>,
171     seen: &mut HashSet<AccessPath>,
172 ) -> bool {
173     if !seen.insert(path.clone()) {
174         // Cycle between closures: recursion only descends through captures
175         // whose env value is non-stable *because* they are closures – a cycle
176         // adds no new evidence of instability.
177         return true;
178     }
179     let Some(caps) = closure_captures(path, result) else {
180         return false;
181     };

```

Une closure créée au rendu est PerRender : son *identité* change à chaque rendu. Mais si toutes ses captures sont stables, sa copie périmée se *comporte* exactement comme la copie courante, et l’omettre des deps est inoffensif. La fonction descend récursivement dans les captures qui ne

sont pas stables *parce qu’elles* sont elles-mêmes des closures ; un cycle entre closures n’apporte aucune preuve d’instabilité nouvelle et répond `true`. Les captures viennent de `closure_captures` (`src/rules/impls/missing_deps.rs`, l. 204–224), qui connaît les deux orthographes d’une closure dans l’IR : un `FnLit` lié dans le rendu (`captures = racines` libres, par `compute_free_vars`, pour ne jamais en oublier) et un `useCallback` extrait dans la table des hooks (`captures = ses free_paths`). Le chemin peut atteindre la closure à travers un conteneur (`api.bump`, pour un hook personnalisé qui renvoie `bump`), par `closure_binding_of`, qui échoue fermé dès qu’une liaison est multiple ou conditionnelle (`src/ir/bindings.rs`, l. 58–94).

Exemple 20.5: Identité contre comportement

```
1 export function Beh() {
2   const [x, setX] = useState(0);
3   const reset = () => setX(0);
4   useEffect(() => { reset(); }, []);
5   useEffect(() => { reset(); }, [reset]);
6   return <i>{x}</i>;
7 }
```

Beh (3 hooks) beh.tsx

warn always-unstable-deps [hook:2] (line 6:2) this effect depends on `reset`, a new reference every render,
 ↳ so `Object.is` always differs and the effect re-runs on every render regardless of the other deps
 → the value flows through `reset`, bound here (line 4:8)

Premier effet : `reset` n’est pas déclaré, mais sa seule capture, `setX`, est stable ; la copie périmée de `reset` fait la même chose que la courante, et `missing-deps` se tait (kill 3). Second effet : la même closure, lue cette fois en *identité* par `always-unstable-deps`, défait le tableau. Les deux règles ont raison, chacune pour sa question : l’une demande « la copie tenue peut-elle se comporter autrement ? », l’autre « `Object.is` peut-il réussir ? ». Le commentaire de `closure_is_behaviorally_stable` l’écrit : les règles d’identité (`always-unstable-deps`, le bras `churn` d’infinite-loop) doivent continuer de lire `PerRender`.

20.5.5 Niveau, message et assurance

La règle n’émet que des `WARNING`. Une omission peut être voulue (un effet qu’on veut vraiment ne lancer qu’au montage, #32), et une valeur qui « peut changer » ne change peut-être jamais. Le message est construit par `src/rules/impls/missing_deps.rs`, l. 100–127 : le chemin, le mot `effect`, `memo` ou `callback`, et une description de la valeur de la racine, `describe_value` :

Extrait 20.28 – `src/rules/helpers/mod.rs`, l. 42–52 — des stabilités aux mots

```
42 pub(crate) fn describe_value(val: &StateValue) -> &'static str {
43   use crate::domains::Stability;
44   match val.to_stability() {
45     Stability::Bottom | Stability::Stable => "its value never changes between renders",
46     Stability::PerRender => "it is recreated on every render",
47     Stability::Versioned(_) | Stability::VersionedTop => {
48       "its value changes when state is updated"
49     }
50     Stability::Unknown => "its value may change between renders",
51   }
52 }
```

Le diagnostic porte le label du hook, la racine comme `var`, la position du hook et une note `Step::Read`. L’assurance est publiée dès qu’un `EffectInfo` a un argument de `deps` : « every effect declares the variables it reads » (`src/rules/impls/missing_deps.rs`, l. 30–41).

Piège

« **it is recreated on every render** » pour un nombre. Dans le Timer de la section 20.8, missing-deps dit de `n`, un compteur : « it is recreated on every render ». Ce n'est pas une référence ; mais `n` a subi le widening vers $[0, +\infty)$, et `to_stability` projette un intervalle non ponctuel sur `PerRender`, au sens « peut changer à chaque rendu » (ADR-17 §4, qui assume ce nom étrange pour un nombre). La *décision* de tirer est juste ; seul le *texte* est imprécis. `describe_value` n'ayant qu'un appelant natif, la règle, un correctif qui distinguerait références et primitifs y resterait local.

20.5.6 Faux positifs assumés**Décision : wontfix #40 : un test d'existence lit l'objet entier**

```
1 export function Locale({ cfg }: { cfg: { locale: string } | null }) {
2   useEffect(() => {
3     if (!cfg) return;
4     applyLocale(cfg.locale);
5   }, [cfg?.locale]);
6   return <div />;
7 }
```

```
Locale (1 hooks) wontfix.tsx
warn  missing-deps [hook:0] var:cfg (line 6:2) `cfg` is used in this effect but not in its deps array, and
      ↳ its value may change between renders
      ↳ `cfg` is read here [hook:0] (line 6:2)
```

Le test `!cfg` lit la référence entière, que `[cfg?.locale]` ne couvre pas. Distinguer « usage de la valeur » de « test d'existence » demanderait de suivre *comment* une référence est consommée ; le WARNING est sound et aligné sur eslint. L'issue #40 est fermée wontfix : trois findings de ce type sur le corpus, réouverture seulement avec un design de suivi du type de consommation.

Piège

Un retour anticipé rend top ce qui suit. L'environnement de sortie est le join des environnements des blocs `Return` (section 20.1) ; une variable liée après un retour anticipé est absente de l'environnement du premier `Return`, et le join la rend top.

```
1 export function E({ on }: { on: boolean }) {
2   if (!on) return <b />;
3   const k = 5;
4   useEffect(() => { console.log(k); }, []);
5   return <i />;
6 }
7 export function F({ on }: { on: boolean }) {
8   const k = 5;
9   useEffect(() => { console.log(k); }, []);
10  return <i />;
11 }
```

`reactant check /tmp/ch20/ee.tsx -rule missing-deps` donne, pour `E`, « `'k'` is used in this effect but not in its deps array, and its value may change between renders », et laisse `F` muet. C'est un FP sound (dans la direction « tirer »), qui frappe tout composant à retour anticipé ; `E` reçoit d'ailleurs aussi l'ERROR `conditional-hook`, légitime.

20.5.7 Faux négatifs observés

La règle hérite des limites du moteur : elle ne voit une valeur changer que si le store d'état a vu l'écriture.

Exemple 20.6: Une écriture que le store ne voit pas

```

1 export function C1({ bus2 }: { bus2: any }) {
2   const [n, setN] = useState(0);
3   useEffect(() => {
4     window.addEventListener("x", () => setN(5));
5   }, []);
6   useEffect(() => { console.log(n); }, []);
7   return <div>{n}</div>;
8 }
9 export function C2({ bus2 }: { bus2: any }) {
10  const [n, setN] = useState(0);
11  useEffect(() => {
12    bus2.foo(() => setN(5));
13  }, [bus2]);
14  useEffect(() => { console.log(n); }, []);
15  return <div>{n}</div>;
16 }

```

```

C1 (4 hooks) em3.tsx
  warn missing-deps [hook:2] var:n (line 7:2) `n` is used in this effect but not in its deps array, and it
    ↳ is recreated on every render
    ↳ → `n` is read here [hook:2] (line 7:2)
C2 (3 hooks) em3.tsx ✓
C3 (3 hooks) em3.tsx ✓

```

Dans C1, le listener `addEventListener` est une registration de timing Handler : l'écriture `setN(5)` atteint le store, `n` vaut `{0, 5}` et la règle tire. Dans C2 (callback remis à une méthode inconnue) et C3 (le même avec `bus2.subscribe`, registrar de timing Unknown), `n` reste la constante 0 dans le store convergé, donc Stable : la règle est muette alors que `n` change vraiment. Le fichier `/tmp/ch20/asy.tsx` montre le même silence pour une écriture placée dans une IIFE asynchrone, (`async () => const v = await load(); setN(v); )()`, avec, sous `-info`, une assurance « verified missing-deps » *fausse*.

Ce n'est pas un défaut de la règle, qui lit correctement ce que le moteur lui donne ; c'est la famille FN des « callees inconnus » (#19, docs/limitations.md) et le « gap 4 » de docs/campaign/triage-effects.md pour l'IIFE. ADR-34 §5 et la marche des écrivains (chapitre 14) traitent un argument fonction d'un appel inconnu comme un écrivain de phase top ; que cette information n'atteigne pas la valeur du slot lue par la règle est un constat au commit e67b10a, dont la cause précise relève du moteur (chapitre 13).

20.6 unnecessary-render : l'effet de montage qui écrase l'initialisation

`unnecessary-render` est une règle de conseil, courte et locale. Elle introduit pourtant une idée qui reviendra : une même expression ne s'évalue pas de la même façon selon le *moment* du cycle React qu'on interroge.

Rappel React : effet de montage et rendu supplémentaire

const [x, setX] = useState(A); useEffect(() => setX(B); , []); : au montage, React rend avec A, commite, exécute l'effet, qui écrit B, puis rend une seconde fois. Si A et B sont deux constantes, le premier rendu est perdu et l'utilisateur peut voir A un instant. Initialiser directement à B supprime ce rendu. Une exception : en rendu serveur, le premier rendu client doit reproduire le rendu serveur (hydratation); l'idiome « drapeau de montage » (useState(false) passé à true dans un effet []) veut précisément ce second rendu.

20.6.1 Le problème et sa sortie

Extrait 20.29 – /tmp/ch20/width.tsx, composant Theme : deux écritures de montage

```
1 export function Theme() {
2   const [mode, setMode] = useState("light");
3   const [mounted, setMounted] = useState(false);
4   useEffect(() => {
5     setMode("dark");
6     setMounted(true);
7   }, []);
8   return <div className={mode}>{mounted ? "client" : "server"}</div>;
9 }
```

Theme (3 hooks) width.tsx

```
warn unnecessary-re-render [hook:1] (line 14:2) mount-only effect flips state `mounted` from `false` to `true`.
  ↳ The SSR mount-flag idiom costs one extra rerender on every mount; prefer `useSyncExternalStore` for client
  ↳ detection
    ↳ state `mounted` is written here [hook:2] (line 14:2)
warn unnecessary-re-render [hook:0] (line 14:2) mount-only effect sets state `mode` to a constant different from
  ↳ its initial value, which costs one extra rerender on mount; consider initialising directly with the target value
    ↳ state `mode` is written here [hook:2] (line 14:2)
```

Deux WARNING, deux conseils. Le label du diagnostic est celui du *slot* (0 pour mode, 1 pour mounted), celui de la note est celui de l'effet (2). Les deux findings ont la même position et le même nom de règle; le tri du registre les départage par message, ce qui explique que mounted passe avant mode.

20.6.2 Deux évaluations, deux moments

La règle compare la valeur initiale d'un slot et la valeur qu'y écrit un effet de montage. La valeur initiale est évaluée *au montage*, c'est-à-dire sans rien de ce que le point fixe a accumulé :

Extrait 20.30 – src/rules/impls/unnecessary_rerender.rs, l. 62–82 — la valeur initiale, évaluée avec des stores et un tas vides

```
62     let init_values: HashMap<HookLabel, StateValue> = result
63         .hooks
64         .iter()
65         .filter_map(|h| {
66             if let HookEntry::State { label, init, .. } = h {
67                 // Mount-time init eval: empty stores + empty heap, NOT the
68                 // converged bundle.
69                 let val = crate::rules::eval_in_stores(
70                     init,
71                     &empty_env,
72                     result.component,
73                     &empty_state,
74                     &empty_memo,
```

```

75         &mut crate::domains::Heap::new(),
76     );
77     Some((*label, val))
78 } else {
79     None
80 }
81 })
82 .collect();

```

Évaluer l’initialiseur dans les stores convergés serait une erreur de moment : ce qui compte est ce que voit le *premier* rendu. L’argument du setter, lui, est évalué par `result.eval_in(&empty_env, a)` (`src/rules/impls/unnecessary_rerender.rs`, l. 133–139) : un environnement *vide*, mais les stores et le tas *convergés* (`ConvergedEval`). La boucle de recherche est un balayage plat :

Extrait 20.31 – `src/rules/impls/unnecessary_rerender.rs`, l. 110–146 — le balayage des écritures de montage

```

110     // Order-independent scan: any const setter call anywhere in a
111     // mount-only effect forces the extra render, so a flat pass over
112     // every block suffices (no need for control-flow ordering).
113     for block in body_cfg.blocks.values() {
114         for stmt in &block.stmts {
115             let Stmt::ExprStmt(Expr::Call { fn_, args }, _) = stmt else {
116                 continue;
117             };
118             let Expr::Var(setter_name) = fn_.as_ref() else {
119                 continue;
120             };
121
122             let Some(&state_label) = setters.get(setter_name) else {
123                 continue;
124             };
125
126             let Some(init_val) = init_values.get(&state_label) else {
127                 continue;
128             };
129             if !init_val.is_stable() {
130                 continue;
131             }
132
133             let arg_val = args
134                 .first()
135                 .map(|a| {
136                     use crate::rules::ConvergedEval;
137                     result.eval_in(&empty_env, a)
138                 })
139                 .unwrap_or(StateValue::top());
140
141             if !arg_val.is_stable() {
142                 continue;
143             }
144             if arg_val == *init_val {
145                 continue; // same as init → redundant-set-state, not this rule
146             }

```

Quatre conditions doivent tenir pour chaque instruction : c’est un appel direct `setter(...)` en position d’instruction, le setter (alias compris) désigne un slot, sa valeur initiale est `Stable`, l’argument est `Stable` et différent de la valeur initiale. L’égalité est l’égalité structurelle de `StateValue` ; une écriture égale à l’initialisation est l’affaire de `redundant-set-state` (chapitre 18).

Exemple 20.7: Ce que « constante » veut dire

```

1  const MOD = "dark";
2  export function A() {
3    const [m, setM] = useState("light");
4    const local = "dark";
5    useEffect(() => { setM(local); }, []);
6    return <i>{m}</i>;
7  }
8  export function B() {
9    const [m, setM] = useState("light");
10   useEffect(() => { setM(MOD); }, []);
11   return <i>{m}</i>;
12 }
13 export function C() {
14   const [m, setM] = useState("light");
15   useEffect(() => { if (Math.random() > 2) setM("dark"); }, []);
16   return <i>{m}</i>;
17 }
18 export function D() {
19   const [m, setM] = useState(0);
20   useEffect(() => { setM(1 + 1); }, []);
21   return <i>{m}</i>;
22 }

```

```

A (2 hooks)  ur.tsx  ✓
B (2 hooks)  ur.tsx  ✓
C (2 hooks)  ur.tsx
  warn  unnecessary-re-render  [hook:0] (line 16:2)  mount-only effect sets state `m` to a constant different
    ↳ from its initial value, which costs one extra rerender on mount; consider initialising directly with the
    ↳ target value
    ↳ state `m` is written here [hook:1] (line 16:2)
D (2 hooks)  ur.tsx
  warn  unnecessary-re-render  [hook:0] (line 21:2)  mount-only effect sets state `m` to a constant different
    ↳ from its initial value, which costs one extra rerender on mount; consider initialising directly with the
    ↳ target value
    ↳ state `m` is written here [hook:1] (line 21:2)

```

A et B sont muets : `local` et `MOD` ne sont pas dans l'environnement vide, valent `top`, et la règle se tait. D tire : `1 + 1` est un littéral calculé, la constante 2. C tire alors que l'écriture n'a lieu que sur un chemin (qui, ici, n'est jamais pris) : le balayage est plat, sans regard pour le chemin. Le commentaire du code le justifie (« any const setter call anywhere in a mount-only effect forces the extra render »), ce qui n'est vrai que pour une écriture inconditionnelle. Sur C, le rendu supplémentaire n'est que possible.

Ce sont des pertes de précision, pas de soundness : la règle est un conseil. Elle ne compte pas non plus un setter appelé dans un `return`, dans un ternaire ou dans un callback imbriqué de l'effet, faute de `Stmt::ExprStmt(Expr::Call ...)`.

20.6.3 L'idiome SSR

Extrait 20.32 – `src/rules/impls/unnecessary_rerender.rs`, l. 148–156 — le drapeau de montage

```

148 // SSR mount-flag idiom: `useState(false)` flipped to
149 // `true` on mount (hasMounted/isClient/isHydrated). The
150 // extra render is the point – the server/hydration pass
151 // must see `false` – so "initialise with the target
152 // value" would break hydration. Keep the warning (the
153 // idiom has a modern replacement) but give advice that
154 // doesn't break the component.

```

```

155         let is_mount_flag = *init_val == StateValue::boolean(BoolVal::False)
156         && arg_val == StateValue::boolean(BoolVal::True);

```

L’idiome reçoit un autre conseil : « initialiser à la valeur cible » casserait l’hydratation. Le WARNING est conservé, parce que l’idiome a un remplaçant moderne (`useSyncExternalStore`). Le conseil s’applique à *toute* bascule `false` vers `true` de montage, y compris une animation d’entrée qui n’a rien à voir avec la détection du client (`isVisible` dans un `ScrollEntrance.tsx` du corpus, noté par `docs/campaign/triage-2026-09-03-untriaged-clusters.md`) : le message `y` est inexact, pas le fait (un rendu en plus).

20.6.4 Niveau, alias et assurance

WARNING toujours. Sur une écriture inconditionnelle, le fait est certain (un rendu en plus au montage) et son coût incertain ; sur une écriture conditionnelle (`C`), le fait lui-même n’est que possible. Les deux lectures mènent au même niveau.

Décision : ADR-020 §6 : une résolution des setters par corps d’effet

Un utilitaire inline dans l’effet lie son paramètre `setter` par un alias (`let setter = setX`) ; la règle suit ces alias avec `resolve_setter_aliases(body_cfg, &setter_to_label)`, *par corps d’effet* (`src/rules/impls/unnecessary_rerender.rs`, l. 108). ADR-20 a examiné, et refusé, une résolution unique calculée pour tout le composant et partagée par toutes les règles : un alias défini dans l’effet A fuirait dans l’effet B, et attribuerait une écriture au mauvais slot. Les résolutions divergent entre règles *à dessein* ; une résolution globale serait soit insound, soit vide. `stale-closure` applique la même discipline.

Enfin, `src/rules/impls/unnecessary_rerender.rs`, l. 36–53 : l’assurance exige un `useState` et un effet dont les `deps` ont l’arité exacte 0, et s’énonce « no mount effect overwrites its initial state ».

20.7 missing-cleanup : un enregistrement que rien ne défait

Avec `missing-cleanup`, on entre dans la seconde famille de défauts annoncée en tête de chapitre : non plus ce que l’effet *lit*, mais ce qu’il *démarre* et ne défait pas. La règle est courte (113 lignes) parce que tout le travail de reconnaissance est fait en amont, par le moteur, dans la relation `registrations` (ADR-34, section 14.8).

Rappel React : cleanup d’un effet

Une fonction renvoyée par le corps d’un effet est son *cleanup*. React l’appelle avant chaque ré-exécution de l’effet (quand ses `deps` changent) et au démontage ; en `StrictMode` et en développement, il monte, démonte et remonte chaque composant au premier montage, donc exécute l’effet deux fois (section 2.6). Un effet qui enregistre un listener, un timer répétitif ou un abonnement sans le défaire en accumule un de plus à chaque exécution ; les anciens continuent de tirer contre un état que le composant n’a plus.

20.7.1 Le problème et sa sortie

Extrait 20.33 – `/tmp/ch20/width.tsx`, composant `Width` : un listener jamais retiré

```

1 export function Width() {
2   const [w, setW] = useState(0);
3   useEffect(() => {

```



```

4   window.addEventListener("resize", () => setW(window.innerWidth));
5   }, []);
6   return <div>{w}</div>;
7 }

```

Width (3 hooks) width.tsx

warn missing-cleanup [hook:1] (line 6:4) this effect calls `window.addEventListener` but returns no cleanup. The
 ↳ registration is repeated every time the effect re-runs (and on every mount, twice under StrictMode) and nothing
 ↳ ever undoes it; return a function that tears it down
 verified stale-closure no long-lived callback captures a stale state value

La position est celle de la registration, non de l'effet. L'assurance de *stale-closure* (le callback ne lit aucun état, seulement `setW` et `window`) montre au passage que les deux règles lisent la même relation.

20.7.2 La relation d'enregistrement

Une *registration* est un appel, dans le corps d'un effet, qui remet un callback à quelque chose qui survit à l'appel : un timer, la cible d'un événement, un émetteur, une promesse. Le moteur les reconnaît par une table unique de *registrars* (section 14.3), dont chaque ligne porte deux colonnes qui intéressent ce chapitre :

Extrait 20.34 – `src/engine/registrations.rs`, l. 31–57 — comment et quand un callback enregistré tire

```

31 /// How a registered callback re-fires after the registering call returns.
32 #[derive(Debug, Clone, Copy, PartialEq, Eq)]
33 pub enum Firing {
34     /// Fires an unbounded number of times (timer tick, event, subscription).
35     Repeating,
36     /// Fires once, shortly after registration (timeout, promise, rAF).
37     Once,
38 }
39
40 /// When a registered callback can run, relative to the React phases.
41 ///
42 /// This is the phase *summary* ADR-027 §2 promised and never shipped: a
43 /// registration argument used to fall to T in the slot-writer walk.
44 #[derive(Debug, Clone, Copy, PartialEq, Eq)]
45 pub enum Timing {
46     /// A timer, a microtask, a promise continuation: the callback runs on a
47     /// later turn of the event loop, provably outside every React phase.
48     Deferred,
49     /// An external event. The DOM has no synchronous dispatch from
50     /// `addEventListener`, so the callback provably does not run during the
51     /// registering call.
52     Handler,
53     /// The registrar may invoke the callback synchronously – an RxJS
54     /// `BehaviorSubject` emits to a new subscriber on the spot – so nothing
55     /// about the timing is proven and the walk keeps T.
56     Unknown,
57 }

```

`Firing` dit combien de fois le callback tire : sans borne (`Repeating`) ou une fois (`Once`). `Timing` dit quand il peut tirer par rapport à l'appel qui l'enregistre, et c'est une question de *contrat* : un timer ou une continuation de promesse tire sur un tour ultérieur de la boucle d'événements (`Deferred`) ; le DOM n'a pas de dispatch synchrone depuis `addEventListener` (`Handler`) ; un émetteur reconnu à son seul nom peut appeler le callback sur-le-champ, comme un `BehaviorSubject` de RxJS

(Unknown). La figure 20.3 range les lignes de la table REGISTRARS (src/engine/registrations.rs, l. 93–211) selon ces deux colonnes.

	Deferred	Handler	Unknown
Repeating	setInterval teardown clearInterval (par le handle)	addEventListener teardown removeEventListener (par le listener)	recv.subscribe, recv.on, recv.addListener (méthodes seulement)
Once	setTimeout, setImmediate, requestAnimationFrame, requestIdleCallback, queueMicrotask, .then, .catch, .finally	(aucune ligne)	(aucune ligne)

tirets bleus : périmètre de missing-cleanup (Repeating)

cadres rouges : seules lignes où stale-closure peut atteindre l'ERROR (Repeating et timing prouvé)

FIGURE 20.3 – La table REGISTRARS en matrice Firing × Timing. Les lignes « méthodes seulement » (method_only) ne valent que sous la forme recv.nom(...); then, catch, finally aussi. queueMicrotask et les continuations de promesse n'ont pas de teardown.

La relation est calculée une fois par composant, à la convergence (collect_registrations, src/engine/registrations.rs, l. 322–346, appelée en src/engine/fixpoint.rs, l. 646), par un balayage de profondeur 2 : une registration placée dans un helper local appelé par l'effet, ou dans un callback de premier niveau, compte encore ; rien de plus profond. Chaque ligne est une Registration ; ses derniers champs portent ce que les deux règles liront :

Extrait 20.35 – src/engine/registrations.rs, l. 257–269 — la fin de la ligne Registration

```

257  /// The registration takes itself back: `addEventListener(t, h, { once:
258  /// true })` removes its own listener after one dispatch, so no teardown is
259  /// needed or possible.
260  pub self_removing: bool,
261  /// Top-level block of the effect body carrying the registration; `None`
262  /// when nested in another callback (then never must-reached).
263  pub block_id: Option<BlockId>,
264  pub span: Option<SourceRange>,
265  pub pairing: Pairing,
266  /// The event name, when the registrar takes one as a string literal
267  /// before the callback (`addEventListener("mousemove", h)`).
268  pub event: Option<String>,
269 }
```

Les premiers champs (src/engine/registrations.rs, l. 238–256) sont l'effet porteur (effect), le nom affiché (display, par exemple window.addEventListener), la ligne de table (registrar), firing, timing, le callback tel qu'écrit (callback) et la liaison qui reçoit la valeur de retour (handle, le id de const id = setInterval(...)). block_id obéit à un invariant que stale-closure exploitera : il n'est renseigné que pour une registration placée au niveau supérieur du corps de l'effet (ou dans un helper appelé en ligne depuis ce niveau) ; dans un callback imbriqué il vaut None, et une telle registration n'est jamais prouvée atteinte (src/engine/registrations.rs, l. 584–588).

Décision : ADR-034 : une table, une relation

Avant ADR-34, trois lecteurs posaient la même question aux mêmes sites sans partager la réponse : `stale-closure` et `missing-cleanup` partageaient un balayage dans `rules::helpers::registrations`, avec sa propre table ; la marche des écrivains gardait deux listes `DEFERRING_*`, libres de diverger ; et la « phase » promise par ADR-27 §2 pour un callback enregistré n'existait pas, si bien qu'un callback remis à `addEventListener` tombait à top. La décision : une seule table dans `engine::registrations`, une colonne par lecteur, et une relation du moteur que les règles lisent au lieu de rebalayer les corps (ADR-42). Le point de soundness est la colonne `timing` : réduire une ligne de `Unknown` à `Handler` est la seule direction qui peut faire *perdre* un finding, et elle n'est permise que là où le `timing` est un contrat, pas une devinette sur un nom (ADR-34 §2).

20.7.3 Le cleanup, fait trois-valué

Extrait 20.36 – `src/rules/api/query.rs`, l. 1147–1178 — le verdict de cleanup (le type `CleanupVerdict`, l. 1135–1145, a trois variantes : `Present`, `Absent`, `Unknown`)

```

1147 /// Classify an effect body's teardown. `fn_lit_binding`'s certainty bar is
1148 /// reused for `return unsubscribe`: a variable bound to exactly one function
1149 /// literal counts as a cleanup, a re-bound or imported one is `Unknown`.
1150 pub fn cleanup_verdict(body: &CFG) -> CleanupVerdict {
1151     let mut verdict = CleanupVerdict::Absent;
1152     for block in body.blocks.values() {
1153         let Terminator::Return(expr) = &block.term else {
1154             continue;
1155         };
1156         match classify_returned(expr, body) {
1157             // One cleanup on one path is a cleanup: the rule is about the
1158             // author forgetting teardown entirely, not about a path missing it.
1159             CleanupVerdict::Present => return CleanupVerdict::Present,
1160             CleanupVerdict::Unknown => verdict = CleanupVerdict::Unknown,
1161             CleanupVerdict::Absent => {}
1162         }
1163     }
1164     verdict
1165 }
1166
1167 fn classify_returned(expr: &Expr, body: &CFG) -> CleanupVerdict {
1168     match expr.peel_ts() {
1169         Expr::FnLit { .. } => CleanupVerdict::Present,
1170         // `return;` and `return undefined;` – the author returned nothing.
1171         Expr::Lit(Prim::Unit) => CleanupVerdict::Absent,
1172         Expr::Var(v) => match crate::rules::fn_lit_binding(v, body) {
1173             Some(_) => CleanupVerdict::Present,
1174             None => CleanupVerdict::Unknown,
1175         },
1176         _ => CleanupVerdict::Unknown,
1177     }
1178 }

```

Chaque bloc `Return` du corps de l'effet est classé. Un littéral fonction, ou une variable liée à exactement un littéral fonction (la barre de certitude de `fn_lit_binding`), est un cleanup (`Present`) ; un `return;` nu, ou la fin de corps que le lowering scelle en `Return(undefined)`, n'en est pas un (`Absent`) ; tout le reste (un appel, une variable importée ou re-liée) est `Unknown`. Et un seul `Present` suffit.

Invariant 20.2: Seule l'absence est une affirmation

CleanupVerdict::Unknown se replie du côté may : il signifie « il y a peut-être un cleanup » et ne peut jamais être lu comme une absence. Absent, et lui seul, est une affirmation sur laquelle une règle peut agir.

20.7.4 La règle

Extrait 20.37 – src/rules/impls/missing_cleanup.rs, l. 56–84 — un effet sans cleanup qui enregistre quelque chose de répétitif

```

56     for hook in &result.hooks {
57         let HookEntry::Effect {
58             label, body_cfg, ..
59         } = hook
60         else {
61             continue;
62         };
63         if cleanup_verdict(body_cfg) != CleanupVerdict::Absent {
64             continue;
65         }
66
67         // The engine's registration relation (ADR-034), never a second
68         // scan of the same bodies.
69         // Deterministic: the scan walks blocks in id order, but two
70         // registrations in one block tie – order by position, then by name.
71         let mut repeating: Vec<_> = result
72             .registrations
73             .iter()
74             .filter(|r| r.effect == *label && r.firing == Firing::Repeating)
75             .collect();
76         repeating.sort_by_key(|r| {
77             (
78                 r.span.map_or((u32::MAX, u32::MAX), |s| s.pos_key()),
79                 r.display.clone(),
80             )
81         });
82         let Some(first) = repeating.first() else {
83             continue;
84         };

```

Le commentaire de tête (src/rules/impls/missing_cleanup.rs, l. 15–30) donne les deux décisions qui bornent le bruit. *Registrars répétitifs seulement* : un setTimeout ou un .then sans cleanup tire au pire une fois, en retard, contre un composant démonté ; c'est un vrai problème, mais un autre (un drapeau d'annulation, pas un teardown), et le signaler sur chaque chaîne de promesses enterrerait le signal. *Absence prouvable seulement* : un effet qui renvoie quelque chose a un auteur qui a écrit un teardown, ou quelque chose d'inclassable ; dans les deux cas le conseil serait faux. Le message nomme chaque registrar, dans l'ordre des positions (src/rules/impls/missing_cleanup.rs, l. 86–101) ; il est émis en WARNING, jamais en ERROR : la registration et le teardown peuvent exister tous deux et vivre dans un helper que la règle ne voit pas.

Exemple 20.8: Un émetteur, trois noms

```

1 export function M({ a, b }: { a: any; b: any }) {
2     useEffect(() => {
3         a.on("x", () => {});
4         b.subscribe(() => {});

```

```

5   a.on("y", () => {});
6   }, [a, b]);
7   return <i />;
8 }

```

```

M (1 hooks) mc.tsx
warn  missing-cleanup [hook:0] (line 4:4) this effect calls `a.on`, `b.subscribe`, `a.on` but returns no
      ↳ cleanup. The registration is repeated every time the effect re-runs (and on every mount, twice under
      ↳ StrictMode) and nothing ever undoes it; return a function that tears it down

```

Les trois lignes sont Repeating, donc toutes nommées. `a.on` apparaît deux fois : `names.dedup()` ne retire que les doublons *consécutifs*, et le tri par position intercale `b.subscribe`. Défaut cosmétique du message.

20.7.5 Ce que la règle ne voit pas

Piège

Un cleanup sur un chemin suffit. Le composant `Cond` de `/tmp/ch20/sc2.tsx` enregistre un `setInterval` sous `if (live)` et ne renvoie son `cleanup` que dans cette branche ; `missing-cleanup` publie « verified » (sortie de la section 20.8). Ici c'est juste : le chemin sans `cleanup` n'enregistre rien. Mais le verdict n'est pas sensible au chemin (« one cleanup on one path is a cleanup », `src/rules/api/query.rs`, l. 1157–1159) : `if (tz === "UTC") return () => clearInterval(id)` ; tait aussi la règle quand l'autre chemin fuit (scénario S-EFF-1 de la campagne à l'aveugle, `docs/campaign/triage-effects.md`, gap 3). Ce FN est hors du contrat de la règle, qui est « ne renvoie aucun `cleanup` ».

Décision : `cleanup_verdict` plutôt que `pairing`

Chaque `Registration` porte pourtant un fait plus fin, `pairing` (`Paired`, `Unpaired`, `Unknown` : le `cleanup` défait-il *cette* registration, avec le même listener ou le même handle ? ADR-34 §3, amendé par #124). La règle native ne le lit pas, et le dossier de notes établit que c'est voulu au sens du contrat, sans phrase d'ADR qui le tranche. Trois arguments : la question « le `cleanup` ne défait pas cette registration » est celle d'une autre règle, déclarative, qui s'appuie sur l'ancre `registrations` (chapitre 22) ; `Unpaired` couvre aussi « un `cleanup` lisible sans le bon `teardown` », et lire `pairing` ferait tirer la règle sur des effets qui renvoient un `cleanup`, ce que sa documentation exclut ; enfin ADR-34 réserve `Paired` à la suppression et replie `Unknown` du côté `may`, sans promettre de preuve d'absence.

Deux autres limites sont observées : la table n'a pas d'entrée pour les observateurs (`ResizeObserver.observe`, `MutationObserver.observe` ; scénario S-ASYNC-8), et l'assurance est publiée pour tout composant qui a un `useEffect`, même sans aucune registration (`src/rules/impls/missing_cleanup.rs`, l. 42–50) : « every effect that starts something long-lived also tears it down » est alors vrai sans objet.

20.8 stale-closure : prouver toute la conclusion

`stale-closure` est l'aboutissement du chapitre. Elle réunit tout ce qui précède : la polarité des deps, la stabilité versionnée d'ADR-17, la relation d'enregistrement d'ADR-34, la dominance, et la discipline du jeton `Certified`. C'est aussi la règle dont l'histoire illustre le mieux ce que veut dire, dans ce projet, « prouver *toute* la conclusion ».

Rappel React : closure périmée d'un callback durable

Un callback remis dans un effet à `setInterval`, `addEventListener`, `subscribe` ou `.then` survit au rendu qui l'a créé. Il lit les valeurs capturées à ce rendu, et continue de les lire tant que l'effet n'est pas relancé. Trois correctifs canoniques : déclarer la valeur dans les deps (l'effet se relance et réenregistre un callback frais), passer par l'updater fonctionnel (`setN(v => v + 1)` reçoit la valeur courante au moment où React applique la mise à jour), ou lire un miroir tenu dans un ref (`r.current = n` au rendu, `r.current` dans le callback).

20.8.1 Le problème : le compteur figé

Extrait 20.38 – `/tmp/ch20/timer.tsx` : la forme canonique et deux variantes

```

1 import { useState, useEffect } from "react";
2
3 export function Timer() {
4   const [n, setN] = useState(0);
5   useEffect(() => {
6     const id = setInterval(() => setN(n + 1), 1000);
7     return () => clearInterval(id);
8   }, []);
9   return <div>{n}</div>;
10 }
11
12 export function Delayed() {
13   const [n, setN] = useState(0);
14   useEffect(() => {
15     const id = setTimeout(() => setN(n + 1), 500);
16     return () => clearTimeout(id);
17   }, []);
18   return <div>{n}</div>;
19 }
20
21 export function Fixed() {
22   const [n, setN] = useState(0);
23   useEffect(() => {
24     const id = setInterval(() => setN((v) => v + 1), 1000);
25     return () => clearInterval(id);
26   }, []);
27   return <div>{n}</div>;
28 }

```

La figure 20.4 déroule `Timer` dans le temps. Le callback enregistré au montage a capturé `n = 0`. Chaque tick calcule `setN(0 + 1)` : le premier fait passer l'état à 1 et provoque un rendu ; l'effet, de deps `[]`, n'est pas relancé, et le callback garde sa capture. Les ticks suivants écrivent encore 1, une valeur `Object.is`-égale à l'état courant ; l'état n'avance plus jamais.

La sortie, restreinte à deux règles avec `-rule stale-closure -rule missing-deps -show-clean :`

```

Delayed (2 hooks) timer.tsx
  warn missing-deps [hook:1] var:n (line 14:2) `n` is used in this effect but not in its deps array, and it is
    ↳ recreated on every render
    ↳ `n` is read here [hook:1] (line 14:2)
  warn stale-closure [hook:1] var:n (line 15:10) `n` is captured by the `setTimeout` callback registered in this
    ↳ mount-only effect, so the callback outlives the render and keeps reading the mount-time value after `n` changes
    ↳ `setTimeout` has side effects (subscriptions/requests/timers re-fire on every call) [hook:1] (line 15:10)
    ↳ `n` is captured at registration time, so the callback keeps this value, not the latest one [hook:0] (line
    ↳ 15:10)
    ↳ state `n` is written here [hook:0]
Fixed (2 hooks) timer.tsx ✓
Timer (2 hooks) timer.tsx

```

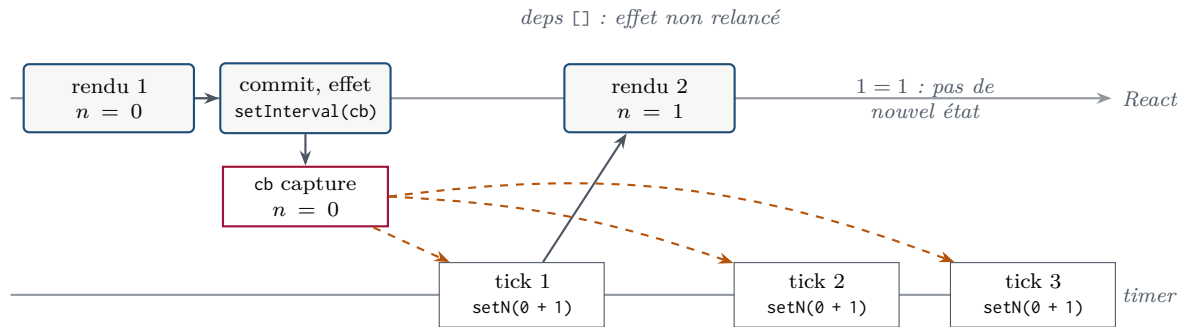


FIGURE 20.4 – La capture figée de Timer (liste 20.38). Le callback `cb` est créé une fois, au rendu 1 ; toutes ses exécutions (flèches pointillées) lisent la même copie `n = 0`. L'état passe à 1 au premier tick, puis n'avance plus.

```

warn  missing-deps [hook:1] var:n (line 5:2) `n` is used in this effect but not in its deps array, and it is
  ↳ recreated on every render
  ↳ `n` is read here [hook:1] (line 5:2)
error stale-closure [hook:1] var:n (line 6:10) the `setInterval` callback registered by this mount-only effect
  ↳ reads `n` and writes it back. `n` was captured once at mount, so every firing recomputes from the same frozen
  ↳ value and the state can never advance past its first update
  ↳ `setInterval` has side effects (subscriptions/requests/timers re-fire on every call) [hook:1] (line 6:10)
  ↳ `n` is captured at registration time, so the callback keeps this value, not the latest one [hook:0] (line 6:10)
  ↳
  ↳ state `n` is written here [hook:0]
  
```

Trois composants, trois niveaux. Timer reçoit l'ERROR ; Delayed, dont le callback ne tire qu'une fois, un WARNING ; Fixed, qui passe par l'updater fonctionnel, rien. `missing-deps` signale les deux premiers aussi, mais elle ne dit que « `n` n'est pas déclaré » ; `stale-closure` dit ce qui s'ensuit. La note Write n'a pas de position : l'écriture est le Return du corps concis `() => setN(n + 1)`, qui ne porte pas de position d'instruction.

20.8.2 Prouver la conséquence

Le commentaire de tête de la règle (`src/rules/impls/stale_closure.rs`, l. 26–64) la sépare nettement de `missing-deps` : celle-ci a la parité eslint, *toute* capture non couverte ; `stale-closure` prouve la *conséquence*. Trois faits s'enchaînent. Le domaine de versions dit que le slot ne change qu'aux écritures de son setter ; les `deps` disent que l'effet n'est jamais relancé à ces écritures ; la registration dit que la copie périmée continue de s'exécuter. Et quand le callback *écrit* le slot qu'il lit, le gel est auto-infligé et certain.

20.8.3 Le chemin d'un finding

`check` (`src/rules/impls/stale_closure.rs`, l. 215–468) se lit en cinq étapes.

1. Préparation, par composant. Les alias d'état du rendu (`resolve_setter_aliases` sur `state_val_labels`), ceux des memos, la table setter vers slot (`all_setter_labels`), l'ensemble des slots *peut-être écrits* (`may_written_slots`), les fonctions liées du rendu et la table des corps de `useCallback` (l. 220–239). `may_written_slots` (`src/engine/setters.rs`, l. 2194–2207) rend les slots dont le setter, ou un alias, est référencé quelque part dans le composant : appelé, passé en prop, capturé.

2. Par effet. Un effet sans argument de deps est sauté : il se relance à chaque rendu, chaque registration reçoit une capture fraîche (la fuite de l'ancienne est un problème de cleanup, pas de péremption). Un argument illisible *n'est pas* ce cas : il est vérifié « rien couvert ». La liste déclarée est la vue covering (invariant 20.1), et mount_only vaut `arity == Exact(0)` (l. 253–266). Les registrations sont celles de la relation du moteur, filtrées sur l'effet : pas de second balayage (l. 272–280). Enfin les alias sont *réensemencés par corps d'effet* (l. 284), pour qu'un `const cur = n` de l'effet A ne fuie pas dans l'effet B (ADR-20 §6).

3. Résoudre le callback. La registration porte le callback *tel qu'écrit* ; il faut en trouver le corps.

Extrait 20.39 – `src/rules/impls/stale_closure.rs`, l. 76–114 — du callback écrit à un corps

```

76 fn resolve_callback<'a>(<
77     cb: &'a Expr,
78     declared: &[AccessPath],
79     effect_body: &'a CFG,
80     render_cfg: &'a CFG,
81     memo_vars: &HashMap<Var, HookLabel>,
82     callback_hooks: &HashMap<HookLabel, (&'a [Var], &'a CFG)>,
83 ) -> Option<(Option<&'a str>, &'a [Var], &'a CFG)> {
84     match cb.peel_ts() {
85         Expr::FnLit {
86             params, body_cfg, ..
87         } => Some((None, params, body_cfg)),
88         Expr::Var(v) => {
89             let as_path = AccessPath {
90                 root: v.clone(),
91                 segments: vec![],
92             };
93             if path_covered(&as_path, declared) {
94                 return None; // identity-covered: re-registered on change
95             }
96             if let Some((params, body)) = fn_lit_binding(v, effect_body) {
97                 return Some((Some(v.as_str()), params, body));
98             }
99             if let Some((params, body)) = fn_lit_binding(v, render_cfg) {
100                 return Some((Some(v.as_str()), params, body));
101             }
102             if let Some(label) = memo_vars.get(v.as_str())
103                 && let Some((params, body)) = callback_hooks.get(label)
104             {
105                 return Some((Some(v.as_str()), params, body));
106             }
107             None
108         }
109         Expr::CallbackVal(label) => callback_hooks
110             .get(label)
111             .map(|(params, body)| (None, *params, *body)),
112         _ => None,
113     }
114 }

```

Un littéral fonction se résout à lui-même. Une variable se résout par `fn_lit_binding`, d'abord dans le corps de l'effet, puis dans le rendu, puis dans la table des `useCallback` ; tout autre cas (un `import`, une liaison conditionnelle `let cb = a ? f : g`) rend `None`, et la registration est *sautée*. Une exception, décisive : une variable callback *elle-même déclarée dans les deps* rend `None` exprès (« identity-covered »). L'effet se relance dès que l'identité de la fonction change, et une closure

du rendu change d'identité chaque fois que ses captures changent : la réinscription porte une capture fraîche.

4. Chaque capture. Les captures sont les chemins libres du corps du callback, moins ses paramètres, triés (l. 300–304). Pour chacune qui n'est pas couverte :

- sa racine est résolue en slots par `resolve_root_slots` (`src/rules/impls/stale_closure.rs`, l. 126–172), d'abord *syntactiquement* par les alias d'état (c'est ce qui fait marcher les slots primitifs : un compteur a une composante référence `Bottom`, la conversion `Versioned` ne s'y applique pas), puis par la valeur évaluée : `Versioned(labels)` donne les slots locaux et un drapeau `foreign` pour ceux d'un autre composant, `VersionedTop` donne `foreign` ;
- une racine qui ne se résout en aucun slot (un ref, une valeur `Stable`, un setter) est ignorée ;
- les slots locaux jamais écrits sont retirés ; s'il ne reste rien et rien d'étranger, la capture est ignorée (kill « jamais écrit ») ;
- on cherche, pour le témoin, si le callback écrit lui-même un slot capturé : c'est `self_write`, calculé par `collect_setter_calls_with_extra` à profondeur 2 ;
- on demande la preuve d'`ERROR` à `must_stale_capture`.

Extrait 20.40 – `src/rules/impls/stale_closure.rs`, l. 343–354 — la sévérité vient du jeton, et de lui seul

```

343         // The whole Error claim is one proof (#142): a proof of
344         // one conjunct (the registration's reach) is not enough.
345         let proof =
346             match must_stale_capture(reg, deps, body_cfg, cb_body, &slot_setters) {
347                 MustResult::All(c) => Some(c),
348                 _ => None,
349             };
350         let severity = if proof.is_some() {
351             Severity::Error
352         } else {
353             Severity::Warning
354         };

```

5. Dédoublonner et émettre. Les candidats sont rangés par chemin dans un `BTreeMap` ; pour un même chemin on garde la meilleure sévérité, puis la registration la plus tôt dans le source (l. 356–386). Chaque finding reçoit l'un de trois gabarits de message (`ERROR` ; `WARNING` d'un effet de montage ; `WARNING` d'un effet à deps non vides), la position de la registration et un témoin : `Step::Call` (le registrar), `Step::Resolve` si le callback est une variable nommée, `Step::Capture`, et `Step::Write` si une auto-écriture a été trouvée (l. 390–463).

20.8.4 La preuve d'Error : cinq faits must

Extrait 20.41 – `src/rules/api/query.rs`, l. 942–989 — la primitive `must_stale_capture`

```

942 pub fn must_stale_capture(
943     reg: &crate::engine::registrations::Registration,
944     deps: &crate::ir::hooks::DepsArg,
945     effect_body: &CFG,
946     cb_body: &CFG,
947     slot_setters: &HashSet<Var>,
948 ) -> MustResult<StaleCapture> {
949     use crate::engine::registrations::{Firing, Timing};
950     let mount_only = matches!(deps.list(), Some(1) if 1.arity == crate::ir::hooks::Arity::Exact(0));

```

```

951     if !mount_only || reg.firing != Firing::Repeating || reg.timing == Timing::Unknown {
952         return MustResult::None;
953     }
954     let Some(reg_block) = reg.block_id else {
955         return MustResult::None;
956     };
957     if !crate::engine::dominance::on_all_paths(effect_body, &HashSet::from([reg_block])) {
958         return MustResult::None;
959     }
960
961     let mut write_blocks: HashSet<BlockId> = HashSet::new();
962     let mut write_spans: Vec<SourceRange> = Vec::new();
963     for (bid, block) in &cb_body.blocks {
964         for stmt in &block.stmts {
965             if let Stmt::ExprStmt(expr, span) = stmt
966                 && try_extract_setter_call(expr, slot_setters).is_some()
967             {
968                 write_blocks.insert(*bid);
969                 write_spans.extend(*span);
970             }
971         }
972         // A concise arrow body (`() => setN(n + 1)`) returns the call.
973         if let Terminator::Return(expr) = &block.term
974             && try_extract_setter_call(expr, slot_setters).is_some()
975         {
976             write_blocks.insert(*bid);
977         }
978     }
979     if write_blocks.is_empty() || !crate::engine::dominance::on_all_paths(cb_body, &write_blocks) {
980         return MustResult::None;
981     }
982     MustResult::All(Certified::mint(
983         StaleCapture {
984             registrar: reg.registrar,
985             write_span: write_spans.into_iter().min_by_key(|r| r.pos_key()),
986         },
987         Provenance::at(reg.span, Some(reg.effect)),
988     ))
989 }

```

La primitive re-dérive tout depuis la registration brute et les corps, plutôt que de faire confiance à des faits calculés par la règle : une preuve d'un conjoint ne peut ainsi pas tenir lieu de la conclusion. Elle frappe un `Certified<StaleCapture>` si et seulement si cinq faits tiennent, tous must :

1. l'effet est de montage : `deps []` exactement, il ne réenregistre jamais ;
2. le registrar est `Repeating` : le callback tire sans borne ;
3. son `timing` n'est pas `Unknown` : il tire *après* l'appel qui l'enregistre, donc seuls `setInterval` et `addEventListener` qualifient (figure 20.3) ;
4. la registration est au niveau supérieur du corps (`block_id` renseigné) et sur tous ses chemins (`on_all_paths`) ;
5. le callback appelle un setter d'un slot capturé, par une instruction de niveau supérieur ou le `Return` d'un corps concis, sur tous ses chemins.

`on_all_paths` (`src/engine/dominance.rs`, l. 69–92) est un parcours depuis l'entrée qui évite l'ensemble de blocs donné ; atteindre un bloc sans successeur en dehors de l'ensemble réfute la propriété. La figure 20.5 dessine la conjonction, avec pour chaque feuille (flèche pointillée) le test d'intégration qui la réfute seule et vérifie que le finding retombe en `WARNING`.

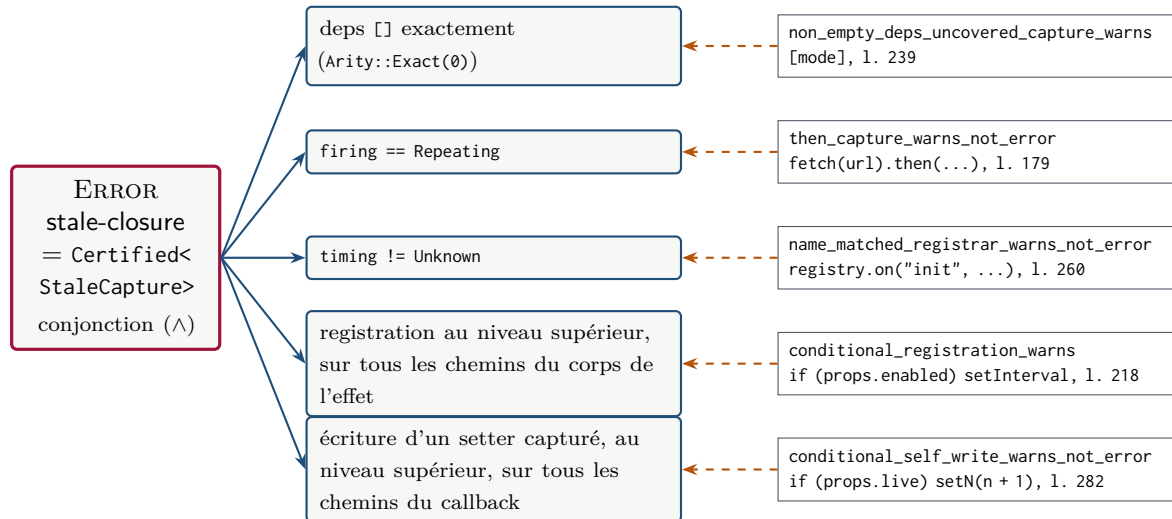


FIGURE 20.5 – La conjunction de `must_stale_capture` (liste 20.41). À droite, pour chaque feuille, le test de `tests/stale_closure.rs` qui la rend fausse (toutes les autres restant vraies) et exige un WARNING au lieu de l'ERROR. La feuille 2 est aussi réfutée par `set_timeout_self_write_warns` (l. 200).

Décision : #142 : une preuve de *toute* la conclusion

Jusqu'à #142, l'ERROR de `stale-closure` était frappée d'un jeton `Certified<OnAllPaths>` prouvant seulement que la registration était atteinte. Les deux autres conjoints étaient des faits may : le registrar pouvait n'être reconnu qu'à son nom (`on`, `subscribe`, `addListener`, tous `Repeating` dans la table), et l'auto-écriture pouvait être *n'importe quelle* écriture, conditionnelle ou imbriquée. Deux programmes l'ont montré : `const registry = on(name, fn) fn(); ;` dont la méthode `on` appelle le callback une fois, synchronement, et jamais plus ; et un intervalle dont l'écriture est sous `if (Math.random() > 2)`. Les deux recevaient l'ERROR. La cause racine : le `typestate` garantissait qu'une ERROR tient *une* preuve, pas que la preuve couvre l'affirmation. Le correctif suit le patron déjà appliqué par `must_frozen_seed` et `must_effect_cycle` : une primitive `must` *par affirmation d'ERROR*, qui ne frappe que si tous les conjoints sont prouvés. L'alternative, garder une preuve partielle et documenter la limite, contredisait l'invariant du projet : une ERROR est la preuve de toute la conclusion, pas d'un seul conjoint.

Décision : wontfix #42 : l'heuristique de nom d'émetteur

Une méthode maison nommée `on`, `addListener` ou `subscribe` sur un objet qui n'est pas un émetteur peut donner un WARNING `stale-closure` (et `missing-cleanup`). L'issue #42 est fermée wontfix au motif d'un coût borné : plafond WARNING par construction. Ce motif était faux avant #142 ; il est vrai depuis, parce que ces lignes sont `Timing::Unknown` et que `must_stale_capture` les refuse. La décision couvre aussi l'ancre déclarative `registrations` (ADR-34 §6, #116).

```

1 const bus = { on(name: string, fn: () => void) { fn(); } };
2 export function Emitter() {
3   const [n, setN] = useState(0);
4   useEffect(() => {
5     bus.on("init", () => setN(n + 1));
6   }, []);
7   return <div>{n}</div>;

```

```
8 }
```

```
Emitter (2 hooks) wontfix.tsx
warn missing-cleanup [hook:1] (line 16:4) this effect calls `bus.on` but returns no cleanup. The
  ↳ registration is repeated every time the effect re-runs (and on every mount, twice under StrictMode) and
  ↳ nothing ever undoes it; return a function that tears it down
warn stale-closure [hook:1] var:n (line 16:4) `n` is captured by the `bus.on` callback registered in this
  ↳ mount-only effect, so the callback outlives the render and keeps reading the mount-time value after `n`
  ↳ changes
    ↳ → `bus.on` has side effects (subscriptions/requests/timers re-fire on every call) [hook:1] (line 16:4)
    ↳ → `n` is captured at registration time, so the callback keeps this value, not the latest one [hook:0] (line
    ↳ → 16:4)
    ↳ → state `n` is written here [hook:0]
```

La forme de Emitter est exactement celle de Timer, et c'est justement le cas où l'ERROR serait fausse : ici bus.on appelle fn une fois, sur-le-champ.

Remarque

Le témoin et la preuve n'ont pas la même exigence. self_write, qui nourrit la note Step::Write, accepte une écriture conditionnelle ou à profondeur 2; must_stale_capture n'accepte qu'une écriture de niveau supérieur sur tous les chemins. Un WARNING peut donc montrer « state n is written here » sans être une ERROR : c'est le composant Live de /tmp/ch20/variants.tsx, dont l'écriture est sous if (live). On retrouve la leçon d'ADR-38 : une écriture est une écriture où qu'elle soit écrite (pour un lecteur may), et une certitude seulement quelquefois (pour un lecteur must).

20.8.5 Les silences, et ce qui les justifie

Chacun des kills listés par le commentaire de tête (src/rules/impls/stale_closure.rs, l. 53–64) est une preuve que la capture ne peut pas se périmier.

Exemple 20.9: Quatre composants, quatre verdicts

```
1 export function Scroll() {
2   const [n, setN] = useState(0);
3   useEffect(() => {
4     const h = () => setN(n + 1);
5     window.addEventListener("scroll", h);
6     return () => window.removeEventListener("scroll", h);
7   }, []);
8   return <i>{n}</i>;
9 }
10 export function Mirror() {
11   const [n, setN] = useState(0);
12   const r = useRef(n);
13   r.current = n;
14   useEffect(() => {
15     const id = setInterval(() => console.log(r.current), 1000);
16     return () => clearInterval(id);
17   }, []);
18   return <button onClick={() => setN(n + 1)}>{n}</button>;
19 }
20 export function Never() {
21   const [n] = useState(0);
22   useEffect(() => {
23     const id = setInterval(() => console.log(n), 1000);
24     return () => clearInterval(id);
```

```

25   }, []);
26   return <i>{n}</i>;
27 }
28 export function Cond({ live }: { live: boolean }) {
29   const [n, setN] = useState(0);
30   useEffect(() => {
31     if (live) { const id = setInterval(() => setN(n + 1), 1000); return () => clearInterval(id); }
32   }, []);
33   return <i>{n}</i>;
34 }

```

Avec `-rule stale-closure -rule missing-cleanup -info -show-clean -trace` (sortie abrégée aux lignes de ces deux règles) :

```

Cond (2 hooks) sc2.tsx
  warn stale-closure [hook:1] var:n (line 32:22) `n` is captured by the `setInterval` callback registered
    ↳ in this mount-only effect, so the callback outlives the render and keeps reading the mount-time value after
    ↳ `n` changes
    ↳ `setInterval` has side effects (subscriptions/requests/timers re-fire on every call) [hook:1] (line
    ↳ 32:22)
    ↳ `n` is captured at registration time, so the callback keeps this value, not the latest one [hook:0] (line
    ↳ 32:22)
    ↳ state `n` is written here [hook:0]
  verified missing-cleanup every effect that starts something long-lived also tears it down
Mirror (4 hooks) sc2.tsx ✓
  verified missing-cleanup every effect that starts something long-lived also tears it down
  verified stale-closure no long-lived callback captures a stale state value
Never (2 hooks) sc2.tsx ✓
  verified missing-cleanup every effect that starts something long-lived also tears it down
  verified stale-closure no long-lived callback captures a stale state value
Scroll (2 hooks) sc2.tsx
  error stale-closure [hook:1] var:n (line 6:4) the `window.addEventListener` callback registered by this
    ↳ mount-only effect reads `n` and writes it back. `n` was captured once at mount, so every firing recomputes
    ↳ from the same frozen value and the state can never advance past its first update
    ↳ `window.addEventListener` has side effects (subscriptions/requests/timers re-fire on every call) [hook:1]
    ↳ (line 6:4)
    ↳ `h` is a function defined in this file
    ↳ `n` is captured at registration time, so the callback keeps this value, not the latest one [hook:0] (line
    ↳ 6:4)
    ↳ state `n` is written here [hook:0]
  verified missing-cleanup every effect that starts something long-lived also tears it down

```

Scroll : le listener est une variable, résolue dans le corps de l'effet (d'où la note « h is a function defined in this file », un `Step::Resolve`); `addEventListener` a le timing `Handler`; les cinq faits tiennent, `ERROR`. Mirror : le callback lit `r`, un `ref Stable`, qui ne se résout en aucun slot; silence, c'est le correctif du miroir. Never : `n` est un état, mais son setter n'est jamais référencé; le slot ne peut pas changer, silence (kill « jamais écrit »). Cond : la registration est dans la branche `then`, `on_all_paths` échoue sur la feuille 4, `WARNING`.

Trois autres silences complètent la liste. L'*updater fonctionnel* (Fixed) : le callback ne lit plus `n`, seulement le paramètre `v`. L'*effet sans tableau de deps* : le composant `NoDeps` de `/tmp/ch20/variants.tsx`, un `setInterval(() => setN(n + 1))` dans un `useEffect` sans deuxième argument, est muet; chaque rendu réenregistre une capture fraîche. La *couverture par identité* mérite un exemple, car elle met en jeu trois règles à la fois.

Exemple 20.10: tick dans les deps, puis hors des deps

```

1 export function Tick() {
2   const [n, setN] = useState(0);
3   const tick = () => setN(n + 1);
4   useEffect(() => {
5     const id = setInterval(tick, 1000);

```

```

6   return () => clearInterval(id);
7 }, [tick]);
8   return <i>{n}</i>;
9 }

```

Avec `[tick]`, `stale-closure` publie « `verified` » : `resolve_callback` voit que la variable `callback` est une `dep`, et rend `None`. Parmi les règles de ce chapitre, le seul finding de ce composant est un `WARNING always-unstable-deps` sur `tick`, une closure neuve à chaque rendu (sortie de `Tick` dans `/tmp/ch20/beh.tsx`) : l'effet se réenregistre à chaque rendu, ce qui est précisément ce qui rend la capture fraîche. `infinite-loop` (chapitre 19) tire aussi : une `dep` qui échoue à chaque rendu équivaut, pour cette règle, à une absence de garde, et `n` diverge pendant l'analyse comme dans le `Counter` du section 20.2. Avec `[]` à la place (`/tmp/ch20/tick2.tsx`) :

```

Tick (2 hooks) tick2.tsx
warn  missing-deps [hook:1] var:tick (line 5:2) `tick` is used in this effect but not in its deps array,
  ↳ and it is recreated on every render
  ↳ `tick` is read here [hook:1] (line 5:2)
error stale-closure [hook:1] var:n (line 6:10) the `setInterval` callback registered by this mount-only
  ↳ effect reads `n` and writes it back. `n` was captured once at mount, so every firing recomputes from the
  ↳ same frozen value and the state can never advance past its first update
  ↳ `setInterval` has side effects (subscriptions/requests/timers re-fire on every call) [hook:1] (line 6:10)
  ↳ `tick` is a function defined in this file
  ↳ `n` is captured at registration time, so the callback keeps this value, not the latest one [hook:0] (line
  ↳ 6:10)
  ↳ state `n` is written here [hook:0]

```

`tick` est résolu dans le rendu par `fn_lit_binding`; son corps concis écrit `n` sur tous ses chemins; `ERROR. always-unstable-deps` se tait (`deps` vides); `missing-deps` dit ici « `recreated on every render` » à juste titre, `tick` étant une fonction.

20.8.6 Niveaux, limites et assurance

Tout ce qui survit aux `kills` tire au moins un `WARNING` : un registrar à tir unique (fenêtre de péremption bornée), un registrar reconnu au nom, des `deps` non vides (le gel dure jusqu'au changement d'une `dep` sans rapport, composant `Mode` de `/tmp/ch20/variants.tsx`), une registration ou une auto-écriture conditionnelle ou imbriquée, un slot étranger ou inconnu. Un slot étranger (`foreign`) plafonne à `WARNING` et ne bénéficie pas du `kill` « `jamais écrit` » : rien de local ne permet de prouver quoi que ce soit contre lui (`src/rules/impls/stale_closure.rs`, l. 116–124).

Piège

Un callback non résolu est sauté. Un callback importé, lié conditionnellement ou caché derrière un wrapper non inliné fait rendre `None` à `resolve_callback`, et la registration est ignorée : c'est un FN documenté (#24), le prix de la barre de certitude de `fn_lit_binding`. Le `kill` « `jamais écrit` » est, lui, syntaxique : le domaine n'a pas de bit « `jamais écrit` » (#41), si bien que `missing-deps`, qui n'a pas ce `kill`, exige encore la `dep` d'un état jamais écrit.

L'assurance est publiée quand un effet à `deps` déclarées porte au moins une ligne de registrations (`src/rules/impls/stale_closure.rs`, l. 198–213) : « `no long-lived callback captures a stale state value` ». Le corpus ne contient aucun `stale-closure` (`docs/campaign/AUDIT.md`) ; l'audit juge ce zéro sain, les bases maintenues passant par l'updater fonctionnel.

20.9 Synthèse : ce que les règles d'effets se partagent

Les huit règles de ce chapitre sont indépendantes : aucune n'appelle une autre, chacune lit l'`AnalysisResult` convergé à sa façon. Elles partagent pourtant cinq mécanismes, qui sont la vraie matière du chapitre ; le tableau 20.3 les récapitule règle par règle, avec les constats faits pendant la rédaction.

1. Une liste de deps se lit dans une polarité. Lue pour tirer, elle rend tous ses éléments visibles (`always-unstable-deps`) ; lue pour se taire, elle rend seulement ceux qui couvrent vraiment une lecture (`missing-deps`, `stale-closure`). `DepsList::covering` est le seul endroit du code où la différence est faite, et un argument `Opaque` n'est ni `Absent` ni `[]` (invariant 20.1).

2. Une valeur s'évalue à un moment. L'environnement de sortie du rendu, joint sur tous les `Return`, sert aux règles qui demandent « ce que le rendu voit » ; les stores et le tas convergés servent à évaluer une dep ou un membre ; des stores vides servent à évaluer ce que voit le *premier* rendu (`unnecessary-rerender`). Se tromper de moment est une erreur de sémantique, pas de précision : un tas vide rendait `always-unstable-deps` et `missing-deps` muettes sur les membres (#135).

3. Un fait du moteur n'est calculé qu'une fois. `registrations` (ADR-34) est lue par `missing-cleanup` et `stale-closure`, qui balayaient autrefois les mêmes corps chacune de son côté ; `EffectInfo` est lu par quatre règles ; `widen_trace` par `widening-info` et `infinite-loop`. C'est la frontière d'ADR-42 : les règles ne parcourent pas l'IR pour y retrouver un fait que le moteur sait produire.

4. Une `Error` est la preuve de toute la conclusion. Deux règles seulement peuvent en produire, et chacune reçoit son jeton d'une primitive unique de `src/rules/api/query.rs` qui quantifie à sa place : « pour toute sortie » (`hook_is_conditional`), « les cinq faits » (`must_stale_capture`). L'histoire de #142 montre que le `typestate` seul ne suffit pas : il garantit qu'une `ERROR` tient une preuve ; c'est la discipline « une primitive par affirmation » qui garantit que la preuve couvre l'affirmation.

5. Un silence n'est une preuve que si l'analyse a regardé. Chaque `kill` est une preuve d'impossibilité (définition 20.1), et là où l'analyse renonce, `analysis-limit` le dit et retire les assurances du composant. Les FN constatés de ce chapitre sont presque tous en amont des règles : `hooks` non extraits, écritures que le store ne voit pas.

Règle	Deps lues	Kills (preuves de silence)	Constats au commit e67b10a
widening-info	—	—	handlers non tracés
analysis-limit	—	—	doc : 6 puis 4 cas annoncés, 7 émis
conditional-hook	—	bloc qui domine toutes les sorties atteignables	FN : hook en membre droit d'affectation ou de &&; assurance vide pour un composant sans hook
always-unstable-deps	tirer	primitif, Versioned, top, []	FP : source d'un spread de primitifs
missing-deps	se taire	couvert, pin, globale, racine, préfixe, comportement	FP : retour anticipé (top par join), #40; FN : écritures invisibles au store (#19); message « recreated » pour un nombre
unnecessary-rerender	arité 0	init ou argument non Stable, argument égal	écriture conditionnelle comptée; variable en argument ignorée
missing-cleanup	—	cleanup présent ou inconnu, registrar Once	non sensible au chemin (S-EFF-1); pas d'observateurs; doublon non consécutif
stale-closure	se taire	couvert (y compris par identité), ref ou Stable, updater, pas de deps, jamais écrit	FN : callback non résolu (#24); FP WARNING : #42

TABLE 20.3 – Synthèse des huit règles : polarité de lecture des deps, kills, et constats observés pendant la rédaction.

Résumé du chapitre

- Huit règles natives portent sur les effets et les hooks à deps. Deux sont des INFO de limite (widening-info, analysis-limit), deux peuvent produire une ERROR (conditional-hook, toujours; stale-closure, sous preuve), quatre plafonnent à WARNING par construction.
- analysis-limit n'est pas qu'un avis : un seul de ses diagnostics retire toutes les assurances verified du composant, et le compte en est publié (suspended).
- conditional-hook est une question de dominance : un hook est conditionnel si son bloc ne domine pas toutes les sorties *atteignables* du CFG de rendu. Le jeton Certified<ConditionalHookCall> vient de hook_is_conditional.
- always-unstable-deps tire sur la borne must PerRender d'une valeur qui n'est qu'une référence; Versioned se tait (ADR-17), et ce silence est couplé au bras churn d'infinite-loop.
- missing-deps raisonne sur des chemins d'accès, couvre par préfixe, lit les deps pour se taire (covering) et ne se tait que sur preuve : racine, préfixe ou comportement stables.
- unnecessary-rerender compare la valeur initiale évaluée *au montage* à une écriture constante d'un effet [].
- missing-cleanup et stale-closure lisent la relation registrations (Firing, Timing, block_id); le cleanup est trois-valué et seule l'absence est une affirmation.
- L'ERROR de stale-closure exige cinq faits must, re-dérivés par must_stale_capture : deps [], Repeating, timing prouvé, registration et auto-écriture sur tous les chemins (#142).

Fichiers rencontrés : src/rules/impls/, les huit fichiers du tableau 20.1, src/rules/api/query.rs (ExitDominance, cleanup_verdict, must_stale_capture), src/engine/registrations.rs, src/ir/hooks.rs (DepsArg, DepsList::covering), src/ir/free_vars.rs (AccessPath, path_covered), src/rules/registry.rs (suspension, located, tri). Le chapitre 21 quitte le domaine des valeurs pour la dépendance de rendu et le graphe de modules de Next.js.

20.10 Exercices

Exercice 20.1: Préfixes

Un effet lit $x.a$ (`useEffect(() => use(x.a); , D)`, avec x une prop). Prédire la sortie de `missing-deps` pour $D = [x.b], [x], [x.a.b]$, puis pour un corps qui lit $x.a[i].b$ sous $[x.a, i]$.

Solution

$[x.b]$: tire sur $x.a$ (le préfixe $x.b$ n'est pas un préfixe de $x.a$, et la liste nomme la racine x , donc le chemin est rapporté tel quel). $[x]$: muet. $[x.a.b]$: tire, un chemin déclaré plus long que le chemin lu ne le couvre pas. $x.a[i].b$: coupé en $x.a$ par l'index dynamique, couvert par $x.a$; muet. Sortie observée sur `/tmp/ch20/paths.tsx` (exemple 20.4).

Exercice 20.2: Polarité en une phrase

Pourquoi `[...rows]` ne couvre-t-il pas une lecture de `rows`, alors qu'`always-unstable-deps` peut tirer sur `rows`? Répondre en une phrase, puis dire si le tir d'`always-unstable-deps` est toujours juste.

Solution

React compare `rows[0]`, `rows[1]`, etc., jamais `rows` : pour se taire, il faut une entrée qui couvre la lecture, ce que la source d'un spread n'est pas ; pour tirer, chaque élément visible sur-approxime ce qu'il représente, ce qui ne crée jamais de FN. Le tir n'est pas toujours juste : si les éléments de `rows` sont des primitifs, le WARNING est un FP (`/tmp/ch20/spread2.tsx`, section 20.4).

Exercice 20.3: Faire varier Timer

Partir de `Timer` (liste 20.38) et obtenir successivement : un WARNING en changeant seulement les `deps` ; un WARNING en ne touchant que le corps du callback ; une ERROR avec un callback qui contient un `if` ; un silence sans toucher aux `deps`. Pour chaque cas, nommer la feuille de la figure 20.5 en jeu.

Solution

`Deps [mode] : WARNING` (feuille 1, composant `Mode`). `() => if (live) setN(n + 1); : WARNING` (feuille 5, `Live`). `() => if (up) setN(n + 1); else setN(n - 1); : ERROR`, l'écriture est sur tous les chemins du callback (`UpDown`). Silence : `setN(v) => v + 1` (`updater` fonctionnel, `Fixed`) ou un miroir `r.current` (`Mirror`). Sorties observées sur `/tmp/ch20/variants.tsx`, `timer.tsx` et `sc2.tsx` ; les tests `non_empty_deps_uncovered_capture_warns`, `conditional_self_write_warns_not_error` et `self_write_on_every_branch_errors` de `tests/stale_closure.rs` épinglent les trois premiers.

Exercice 20.4: La sortie orpheline

Soit un composant qui appelle `useState`, puis un `if/else` dont les deux branches retournent. (a) Quel bloc `Return` du CFG n'est pas atteignable ? (b) Si `ExitDominance::of` ne filtrait pas les sorties atteignables, `conditional-hook` tirerait-elle sur le `useState`, avec le `compute_dominators` actuel ? (c) Et avec un algorithme de dominance qui donnerait à un bloc inatteignable un ensemble de dominateurs vide ?

Solution

(a) Le `Return(undefined)` qui scelle la fin de la fonction (ADR-25) : aucune branche n'y arrive. (b) Non : ce bloc n'est pas dans l'ordre post-ordre inverse, il garde pour `dominateurs`

l'ensemble de tous les blocs (`src/engine/dominance.rs`, l. 16–22), donc le bloc du `useState` le domine. (c) Oui : le bloc du `useState` ne le dominerait plus, `may_be_skipped` rendrait `true`, et l'on aurait une `ERROR` fausse sur tout hook placé avant le `if`. C'est le cas que décrit le commentaire de `src/rules/api/query.rs`, l. 654–658 ; le filtre sur l'atteignabilité rend le verdict indépendant de la convention (composant `Both` de `/tmp/ch20/diamond.tsx`, « `verified` »).

Exercice 20.5: Trois règles sur un callback nommé

Dans `Tick` (exemple 20.10), expliquer pourquoi `stale-closure` se tait avec `[tick]` alors que `n` est capturé, et pourquoi c'est `always-unstable-deps` qui tire. Prédire ensuite les trois règles avec `[]`.

Solution

Avec `[tick]`, `resolve_callback` constate que la variable `callback` est une `dep` (`path_covered`) et rend `None` : l'effet se relance chaque fois que `tick` change d'identité, donc chaque fois que ses captures changent ; la capture est toujours fraîche. Mais `tick` est une closure neuve à *chaque* rendu : `PerRender`, d'où le `WARNING always-unstable-deps`, l'effet se réenregistrant à chaque rendu. Avec `[]` : `missing-deps` tire sur `tick`, `stale-closure` donne une `ERROR` sur `n` (`tick` résolu dans le rendu, corps concis qui écrit `n`), `always-unstable-deps` se tait (`deps` vides). Sortie de `/tmp/ch20/tick2.tsx`.

Exercice 20.6: Un hook invisible

Dans le composant `B` de `/tmp/ch20/s4.tsx` (`if (on) v = useMemo(() => 1, []);`), quelles règles du chapitre voient le `useMemo` ? Remplacer `1` par une prop `p` et vérifier la réponse.

Solution

Aucune. Le `useMemo` ne produit pas de `HookEntry` : il n'a ni ligne `hook_calls` (`conditional-hook` ne le voit pas), ni `EffectInfo` (`missing-deps` et `always-unstable-deps` non plus). Avec `() => p` (`/tmp/ch20/s4b.tsx`), le composant `B2` n'apparaît même pas sous `-show-clean`, faute de hook, alors que la forme `const w = useMemo(() => p, []); v = w;` (`A2`) reçoit l'`ERROR conditional-hook` et le `WARNING missing-deps` sur `p`. Le défaut est dans l'extraction des hooks, et la leçon générale : une règle ne peut pas être plus complète que l'IR qu'elle lit.

Chapitre 21

Rendu et Server Components : wasted-subtree-render, server-component-hook

Objectifs du chapitre.

- Comprendre pourquoi deux règles du catalogue ne lisent presque rien du domaine des valeurs.
- Suivre `wasted-subtree-render` de bout en bout : déclencheurs, fréquence, arbre d'éléments, compte des rendus, options.
- Savoir lire une affirmation d'*absence* (« aucune entrée ne dépend de l'état ») et la règle de soundness qui l'accompagne : tout inconnu est un usage.
- Suivre `server-component-hook` : faits de module, graphe serveur de Next.js, frontière "use client", preuves de hook client.
- Savoir expliquer pourquoi ces deux règles plafonnent à WARNING, et ce que chacune refuse de conclure.

Prérequis : la dépendance de rendu et l'arbre d'éléments (chapitre 16, définition 16.3); la surface typée des règles et le sceau de sévérité (chapitre 17, définition 17.1); la détection et les faits de module (chapitre 5); les projets Next.js vus par le driver (chapitre 23). Les relations `slot_writers` et `registrations` (chapitre 14) servent à la première règle.

21.1 Deux règles à côté du domaine des valeurs

Les règles des chapitres précédents interrogent toutes, d'une façon ou d'une autre, le résultat du point fixe : la valeur abstraite d'un slot, la stabilité d'une référence, les lignes `SlotWriter` d'un setter. Les deux règles de ce chapitre sont différentes. Elles posent des questions que le domaine des valeurs ne sait pas trancher, et elles s'appuient sur des faits *calculés à côté* de lui.

- `wasted-subtree-render` demande : « quand cet état change, quels sous-arbres re-rendent alors qu'aucune de leurs entrées n'en dépend ? » La réponse vient d'une analyse de flot séparée, la *dépendance de rendu* (ADR-41), composée sur l'arbre d'éléments du programme.
- `server-component-hook` demande : « ce hook est-il appelé dans un module que Next.js rend côté serveur ? » La réponse vient de deux faits qui ne sont pas du tout abstraits : une convention de noms de fichiers et le graphe des imports (ADR-26).

Le tableau 21.1 résume ce qui les rapproche et ce qui les oppose. Le point commun le plus important est le niveau : aucune des deux ne peut émettre une ERROR. Le point de divergence le plus instructif est la *forme* de l'affirmation. La première affirme une *absence* (« aucune entrée

ne dépend de l'état ») ; la seconde affirme une *présence* (« ce module est serveur, et il appelle un hook client »). Nous verrons que ces deux formes n'imposent pas la même discipline de soundness.

	wasted-subtree-render	server-component-hook
Bug visé	rendus répétés de sous-arbres dont la sortie ne change pas	hook appelé dans un Server Component : le rendu lève une erreur
Faits consommés	render_deps (par composant), RenderIndex, slot_writers, registrations, state_store	ModuleTable (directives, arêtes d'import), HookEntry, HookProvenance
Forme de l'affirmation	absence d'usage	présence d'un fait de convention
Règle de prudence	tout inconnu est un usage	rien n'est serveur sans "use client" dans le programme ; seuls les hooks prouvés client comptent
Niveau	WARNING (rendus certains, coût incertain)	WARNING (faits hors du domaine)
Options	minWastedRenders, continuousOnly	aucune
safe_check	aucun	sur les Server Components seulement
Fichier	src/rules/impls/wasted_subtree_render.rs (333 l.)	src/rules/impls/server_component_hook.rs (308 l.)
Aides	src/rules/helpers/render_tree.rs, src/engine/render_deps.rs	src/project/nextjs.rs, src/ir/module.rs
Tests	tests/wasted_subtree_render.rs (13)	5 tests unitaires, tests/nextjs_project.rs

TABLE 21.1 – Les deux règles du chapitre en un coup d'œil.

Le chapitre suit l'ordre de difficulté croissante pour la première règle (le problème, les faits, l'algorithme, les options, les cas limites, la soundness), puis traite la seconde, plus courte mais plus délicate à justifier : c'est l'exemple type d'une règle qui *sait* quelque chose de vrai sans pouvoir le prouver dans le domaine abstrait.

21.2 Le rendu gaspillé : le problème

Rappel React : re-rendu en cascade

Quand un état change, React re-rend le composant qui le possède, puis *tous les éléments de composant que ce rendu construit*, récursivement, que leurs props aient changé ou non. La cascade ne s'arrête qu'à trois endroits :

1. un élément qui n'a pas été reconstruit par ce rendu : un children reçu d'un parent garde son identité d'objet, React le reconnaît et saute son rendu ;
2. un composant enveloppé dans memo dont toutes les props sont Object.is-égales aux précédentes ;
3. la frontière du composant propriétaire : si l'état et les éléments qui le lisent vivent dans un composant plus petit, le reste de l'arbre n'est pas re-rendu du tout.

Les écritures faites par un même handler sont regroupées en un seul rendu (batching), et une écriture d'une valeur Object.is-égale à la valeur courante est abandonnée sans rendu (bail-out). Voir chapitre 2 pour le modèle complet.

Le cas qui coûte cher est celui d'un état écrit *très souvent* : à chaque frappe dans un champ texte, à chaque mouvement de souris, à chaque pas de défilement, à chaque tick de minuterie.

Si le composant qui le possède construit aussi un sous-arbre lourd qui n'en dépend pas, ce sous-arbre produit la même sortie des dizaines de fois par seconde.

Exemple 21.1: Un champ de recherche à côté d'une barre latérale

Le fichier `/tmp/ch21/search.tsx` :

Extrait 21.1 – Un état de saisie au-dessus d'un sous-arbre qui ne le lit pas

```

1 import { useState } from "react";
2
3 function Results({ query }: { query: string }) {
4   return <p>results for {query}</p>;
5 }
6 function Sidebar() {
7   return <nav>{["a", "b", "c"].map((k) => <Link key={k} to={k} />)}</nav>;
8 }
9 function Link({ to }: { to: string }) { return <a href={to}>{to}</a>; }
10
11 export default function SearchPage() {
12   const [query, setQuery] = useState("");
13   return (
14     <main>
15       <input value={query} onChange={(e) => setQuery(e.target.value)} />
16       <Results query={query} />
17       <Sidebar />
18     </main>
19   );
20 }

```

Chaque frappe écrit `query`. `SearchPage` re-rend ; `<Results>` re-rend à juste titre, sa prop `query` a changé ; `<Sidebar>` et ses trois `<Link>` re-rendent pour rien. La commande `cargo run -q -check /tmp/ch21/search.tsx -rule wasted-subtree-render -trace` donne :

```

SearchPage (2 hooks) /tmp/ch21/search.tsx
warn  wasted-subtree-render [hook:0] (line 15:27) each `change` event writes state `query`
and re-renders `<Sidebar>` (at least 2 component renders, including a list) although none of
its inputs depends on it. Move `query` and the elements that use it into their own component,
or build the unrelated elements higher up and pass them in as `children`
→ `<Sidebar>` re-renders (at least 2 component renders, including a list) although none of
its props depends on it (line 17:6)

```

(Les lignes longues sont repliées ici ; l'analyseur les imprime sur une seule ligne.) Le finding pointe le handler `onChange` (ligne 15, colonne 27), désigne l'élément gaspillé (`<Sidebar>`, ligne 17) et donne un coût : *au moins* deux rendus de composant, `Sidebar` et un `Link` compté une fois pour toute la liste.

Trois difficultés se cachent derrière ce diagnostic d'apparence simple.

Le domaine des valeurs ne répond pas à la question. La question est « la sortie de `<Sidebar>` peut-elle dépendre de `query` ? ». Le fait le plus proche dans le domaine est la stabilité versionnée d'ADR-17 (`Versioned(S)`, définition 10.3) : il dit de quelles écritures d'état une *référence* peut provenir. Mais il ne vit que dans la composante *reference* du produit `StateValue` : un booléen `open`, un nombre `text.length` n'y portent rien, et un objet alloué (`{ a: text }`) est `PerRender` quelle que soit son origine (ADR-41, contexte). La dépendance est un fait différent de la valeur ; ADR-41 §1 en tire une analyse séparée (section 21.3).

La question est inter-composants. Savoir que `SearchPage` passe `query` à `<Results>` ne suffit pas : il faut savoir que `Results` l'*utilise*, que `Sidebar` ne reçoit rien, et que rien, sous `Sidebar`, ne

lit `query` par un autre canal (un contexte, une variable de module). Il faut donc composer des résumés par composant le long de l'arbre d'éléments.

L'affirmation est une absence. Un finding dit « aucune entrée de `<Sidebar>` ne dépend de `query` ». Une dépendance oubliée par l'analyse produirait donc un *faux positif*, pas un faux négatif. C'est l'inverse de la plupart des règles, où une sur-approximation fait tirer davantage. Toute la conception de la règle découle de ce renversement de polarité (section 21.7).

21.3 Ce que la règle consomme : dépendance de rendu et arbre d'éléments

La règle ne calcule presque rien elle-même : elle interroge un index programme, le `RenderIndex`, bâti sur les résumés de dépendance de rendu de chaque composant. Ces deux objets sont exposés en détail au chapitre 16 (définition 16.3) ; on en rappelle ici ce que la règle lit, dans l'ordre où elle le lit.

21.3.1 Le résumé d'un composant

`src/engine/render_deps.rs` exécute, sur le CFG convergé d'un composant (hooks personnalisés et utilitaires déjà greffés), une analyse avant qui associe à chaque variable un *may-ensemble* de sources. Une `Source` (`src/engine/render_deps.rs`, l. 53–84) est une entrée du rendu *dans le repère du composant* : `Slot(1)` (la valeur d'un slot), `Setter(1)` (son setter, une capacité d'écriture qui ne change jamais), `Prop(name)` et `AllProps`, `Ref(1)`, `Hook(1)` (le résultat d'un hook non modélisé), `Context(id)` (un contexte prouvé) et `Module(var)` (un nom non lié ici que quelque fonction du programme écrit). Seules les deux dernières sont indépendantes du repère : elles traversent l'arbre d'éléments sans traduction.

Les ensembles se joignent par union ; le treillis est fini, donc pas de widening, et le plafond de 64 tours dégrade le résumé en top, jamais en plus petit (ADR-41 §1). Une `Deps` (`src/engine/render_deps.rs`, l. 119–131) porte l'ensemble `set`, un drapeau `top`, une partie `gated` (les sources qu'une valeur fonction n'atteint que derrière un test de ses propres arguments, fait *must*) et `writes` (les noms de module qu'appeler la valeur peut écrire). La question que pose une règle se formule comme une `Relevance` (`src/engine/render_deps.rs`, l. 207–214) : un ensemble de sources dans le repère d'un composant, plus un drapeau `any_prop` (« n'importe quelle prop », après un `spread`). La réponse élémentaire est `Deps::touches` (`src/engine/render_deps.rs`, l. 182–205, détaillée au chapitre 16) : une dépendance `top` touche toute question non vide ; `AllProps` (l'objet `props` pris en entier) touche toute prop nommée ; une question `any_prop` touche toute prop ; sinon il faut une source commune.

Le résumé d'un composant, `RenderDeps`, sépare deux choses que tout le reste du chapitre distingue :

Extrait 21.2 – `src/engine/render_deps.rs`, l. 301–322 — le résumé : ce qui est utilisé, ce qui est construit

```
301 /// Render dependence summary of one component.
302 #[derive(Debug, Clone, Default)]
303 pub struct RenderDeps {
304     pub genuine: Deps,
305     pub sites: Vec<ElementSite>,
306     pub handlers: Vec<HostHandler>,
307     /// The component may read a context the analysis cannot name: a
308     /// `useContext` of an unproven object or reached through an inlined hook,
```

```

309     /// or a hook of user code the engine could not see into.
310     pub any_context: bool,
311     /// Per effect, the module names its body (callbacks it registers
312     /// included) may write: what a write it makes to a slot travels with.
313     pub effect_writes: BTreeMap<HookLabel, Writes>,
314 }
315
316 impl RenderDeps {
317     /// Whether the component uses `rel`: its own output, effects or hooks
318     /// depend on it, or `rel` names a context it may read.
319     pub fn uses(&self, rel: &Relevance) -> bool {
320         self.genuine.touches(rel) || (self.any_context && rel.contexts().next().is_some())
321     }
322 }

```

- *genuine* rassemble ce que le composant *utilise* : sa sortie hôte (y compris les closures de handlers et les éléments hôtes qu'il passe en *children*), ses effets, ses appels en position d'instruction, ses écritures de membres, les arguments de ses hooks opaques, et les conditions et *types* des éléments qu'il rend.
- *sites* liste chaque élément de composant qu'il construit, avec, prop par prop, ce dont la valeur peut dépendre. Une prop est *transmise*, pas utilisée : c'est pourquoi un élément de composant s'évalue à l'ensemble vide ($\text{CompApp} \mapsto \emptyset$, chapitre 16).
- *handlers* liste les props d'événement des éléments hôtes (*onChange* sur un `<input>...`) avec la balise, l'attribut type littéral et les dépendances de la closure : c'est là qu'un setter « atterrit ».
- *effect_writes* donne, par effet, les noms de module que son corps peut écrire.

Chaque élément de composant est un `ElementSite` :

Extrait 21.3 – `src/engine/render_deps.rs`, l. 252–277 — un élément de composant construit par le rendu

```

252 /// A component element built by the render (`<Child a={x} />`).
253 #[derive(Debug, Clone)]
254 pub struct ElementSite {
255     pub name: Symbol,
256     pub origin: Option<Arc<CompOrigin>>,
257     pub span: Option<SourceRange>,
258     /// Per explicit prop, what it may depend on (`children` included).
259     pub props: Vec<(Symbol, Deps)>,
260     /// What the spread entries (`{...rest}`) may depend on: any prop of the
261     /// child may carry it.
262     pub spread: Deps,
263     /// The conditions the element is built under.
264     pub guard: Deps,
265     /// Built in a callback the render passes to a call (`map`): possibly
266     /// many instances, possibly none.
267     pub in_list: bool,
268     /// The element is the provider of a proven context (`Ctx.Provider`, or
269     /// `Ctx` since React 19): its `value` reaches the context's consumers
270     /// below, and its children render unchanged.
271     pub provides: Option<ContextId>,
272     /// Index (in [RenderDeps::sites]) of the element this one is nested in
273     /// as a prop or child: `<Provider value={v}><Row /></Provider>` gives
274     /// `Row` the provider's index. What reaches the enclosing element may
275     /// reach this one through it (a context, a clone).
276     pub parent: Option<usize>,
277 }

```

Quatre champs comptent pour la règle : `guard` (les conditions sous lesquelles l'élément est construit), `props` et `spread` (ce que ses entrées peuvent porter), `in_list` (construit dans un callback passé à un appel, typiquement `.map` : zéro, une ou plusieurs instances) et `parent` (l'élément de composant dans lequel celui-ci est imbriqué comme prop ou enfant, ce qui permet de suivre un provider de contexte jusqu'aux éléments qu'il enveloppe).

21.3.2 L'index programme

`RenderIndex::build` (`src/rules/helpers/render_tree.rs`, l. 117–137) calcule d'abord l'ensemble programme des noms écrits (un nom libre n'est un canal que si quelque chose l'écrit), puis un résumé `render_deps` par composant, puis le nombre de sites qui nomment chaque composant. L'index est construit une seule fois par exécution, à la première demande (`ProgramCache::render`, `src/rules/api/cache.rs`, l. 70–76, un `OnceLock`) et partagé par `wasted-subtree-render` et `state-lifted-too-high` (chapitre 19).

Passer d'un site à un composant se fait par `resolve`, qui n'accepte qu'une origine prouvée :

Extrait 21.4 – `src/rules/helpers/render_tree.rs`, l. 766–777 — un élément ne se résout que par son origine prouvée

```
766 /// The component and element names, when exactly one.
767 pub(in crate::rules) fn resolve(
768     site: &ElementSite,
769     program: &ProgramAnalysisResult,
770 ) -> Option<ComponentId> {
771     let table = &program.component_table;
772     // Only a proven origin: a name match could pick a same-named component of
773     // another file, and a wrong child hides the real one's uses. An element
774     // the registry does not hold (a `memo` wrapper, a library component, an
775     // unresolved import) is opaque.
776     site.origin.as_deref().and_then(|o| table.id_of(o))
777 }
```

Le moteur de valeurs, lui, accepte aussi un nom unique (`resolve_child`, ADR-40). Les deux choix sont prudents, chacun dans sa polarité : l'inliner qui retrouverait un enfant par son seul nom analyserait un corps de plus (au pire du travail inutile) ; la règle de cascade qui se tromperait d'enfant *cachera* les usages du vrai.

21.3.3 L'arbre d'éléments de `typing.tsx`

La fixture de référence de la règle est `tests/fixtures/wasted_subtree_render/typing.tsx`, mesurée à l'exécution par l'oracle `scripts/rerender-bench/` (React 19, `jsdom`, en-tête de `tests/wasted_subtree_render.rs`) :

Extrait 21.5 – `tests/fixtures/wasted_subtree_render/typing.tsx`, l. 5–20 — un champ texte à côté d'un sous-arbre coûteux

```
5 function Row({ i }: { i: number }) { return <li>row {i}</li>; }
6 function ExpensiveTree() {
7     return <ul>{[0, 1, 2].map((i) => <Row key={i} i={i} />)}</ul>;
8 }
9 function Preview({ text }: { text: string }) { return <p>{text}</p>; }
10
11 export default function App() {
12     const [text, setText] = useState("");
13     return (
14         <div>
```



```

15   <input id="q" value={text} onChange={(e) => setText(e.target.value)} />
16   <Preview text={text} />
17   <ExpensiveTree />
18 </div>
19 );
20 }

```

Le résumé de App (relevé par le dossier de notes 08, §6.3, sur une sonde de débogage) a `genuine = {Slot(0), Setter(0)}` (l'`<input>` lit `text` et sa closure capture `setText`), un handler `change` sur `<input>` sans type dont les dépendances sont `{Setter(0)}`, et deux sites : `<Preview>` avec `text = {Slot(0)}`, et `<ExpensiveTree>` sans prop ni garde. Le résumé d'`ExpensiveTree` a un seul site, `<Row>`, marqué `in_list`. La figure 21.1 montre comment la question Relevance `{Slot(0)}` se propage sur cet arbre.

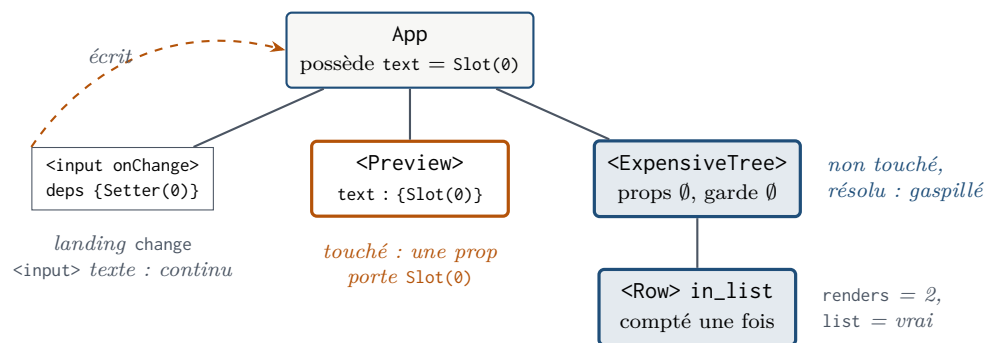


FIGURE 21.1 – L'arbre d'éléments de `typing.tsx` et la question Relevance `{Slot(0)}`. Le handler de l'`<input>` écrit le slot (flèche pointillée) ; `<Preview>` est touché (bord orangé) ; `<ExpensiveTree>` ne l'est pas, se résout, et coûte son propre rendu plus un `<Row>` compté une fois pour toute la liste (bleu).

La propagation s'arrête dès qu'un élément est touché : `<Preview>` n'est pas un candidat, et ce qui serait imbriqué dans lui ne le serait pas davantage (parent). Elle s'arrête aussi, dans l'autre sens, dès qu'un élément est *inconnu*. C'est l'invariant qui gouverne tout le fichier `render_tree.rs`, énoncé dès son en-tête (`src/rules/helpers/render_tree.rs`, l. 11–13) :

Invariant 21.1: Tout inconnu est un usage

Dans les règles de cascade, l'absence d'usage est une preuve. Donc tout ce que l'analyse ne voit pas compte comme un usage : un élément dont le composant ne se résout pas (bibliothèque, enveloppe `memo` ou `forwardRef`, import non résolu, nom ambigu), un composant ré-entré par récursion, la profondeur au-delà de `MAX_DEPTH = 64`, un résumé plafonné à `top`, un contexte que l'analyse ne sait pas nommer (`any_context`), un écrivain dont la racine écrite est `top` (ADR-41 §2).

21.4 L'algorithme de wasted-subtree-render pas à pas

La méthode `check` de `src/rules/impls/wasted_subtree_render.rs` tient en deux cents lignes (`src/rules/impls/wasted_subtree_render.rs`, l. 60–260). Elle s'exécute une fois par composant, qui joue le rôle de *propriétaire* (owner) des états qu'il déclare. Son squelette est donné par l'algorithme 8 ; on en suit les étapes dans l'ordre.

Algorithme 8 : WastedSubtreeRender::check, schématisé

Input : un composant propriétaire o , l'index `RenderIndex`, les options m (`minWastedRenders`) et c (`continuousOnly`)

Output : au plus un `WARNING` par déclencheur

```

1  $T \leftarrow \emptyset$  ; // déclencheurs, groupés par clé
2 foreach slot  $l$  d'un HookEntry::State de  $o$  do
3   foreach Landing  $h$  de index.landings(o, l) do
4     ajouter  $l$  au déclencheur Handler(h) de  $T$ 
5 foreach écrivain local  $w$  de slot_writers dans un effet  $e$  do
6   ajouter  $w.slot$  au déclencheur Effect(e), avec les enregistrements de  $e$  qui peuvent appeler le setter
7 foreach déclencheur  $t \in T$  do
8   if  $t.writes$  est top then passer au suivant;
9   if aucun événement, ou  $c$  et aucun événement continu d'un slot à valeurs nombreuses then passer;
10   $rel \leftarrow \{Slot(l) \mid l \in t.slots\} \cup \{Module(x) \mid x \in t.writes\}$ ;
11   $W \leftarrow index.wasted\_siblings(o, rel)$ ;
12  if  $W = \emptyset$ , ou  $\sum_{w \in W} w.renderers < m$  sans liste then passer;
13  émettre un WARNING avec une note Step::Rerender par élément de  $W$ ;
```

21.4.1 Étape 1 : collecter les déclencheurs**Rappel React : batching**

Toutes les écritures d'état faites pendant un même handler d'événement, ou pendant un même callback de minuterie, sont regroupées par React en *un seul* rendu. Deux setters appelés l'un après l'autre dans un `onChange` ne coûtent qu'un rendu du propriétaire.

La règle ne raisonne donc pas slot par slot mais *déclencheur* par déclencheur : un handler d'événement, ou l'ensemble des listeners et minuteries qu'un effet enregistre. Un déclencheur écrit un *ensemble* de slots, qui re-rendent ensemble. La clé qui décide que deux écritures appartiennent au même lot est `TriggerKey`, et ce qu'on accumule pour chaque clé est un `Trigger` :

Extrait 21.6 – `src/rules/impls/wasted_subtree_render.rs`, l. 263–292 — ce qui fait de deux écritures un seul lot

```

263 type Events = BTreeSet<(String, bool)>;
264
265 /// What makes two writes one batch.
266 #[derive(PartialEq)]
267 enum TriggerKey {
268     /// One event handler: the owner's element the setter leaves through (two
269     /// instances of one child are two handlers), the component building the
270     /// handler's element, the handler prop's span, the event.
271     Handler(
272         Option<SourceRange>,
273         ComponentId,
274         Option<SourceRange>,
275         String,
276     ),
277     /// The listeners and timers one effect registers.
278     Effect(HookLabel),
279 }
280
```

```

281 struct Trigger {
282     key: TriggerKey,
283     slots: BTreeSet<HookLabel>,
284     /// Each event, and whether it is continuous.
285     events: Events,
286     /// Where the finding points: in the owner.
287     span: Option<SourceRange>,
288     /// The component the handler sits in, when not the owner.
289     place: Option<ComponentId>,
290     /// The module names the batch writes beside the slots.
291     writes: Writes,
292 }

```

Un déclencheur Handler est identifié par quatre choses : l'élément du propriétaire par lequel le setter sort (via), le composant qui construit l'élément hôte portant le handler, le span de la prop de handler et le nom de l'événement. Deux instances d'un même enfant sont donc deux déclencheurs, parce que leurs via diffèrent. Un déclencheur Effect est identifié par le label de l'effet.

Les handlers : où le setter atterrit. Un setter n'est pas forcément appelé dans le propriétaire : il peut être passé en prop, traverser plusieurs composants, et n'être appelé que par un `<input>` trois niveaux plus bas. `RenderIndex::landings` répond à la question « où ce setter peut-il être appelé par un événement ? » en posant la question `Relevance {Setter(1)}` et en descendant l'arbre d'éléments. Chaque réponse est un `Landing` :

Extrait 21.7 – `src/rules/helpers/render_tree.rs`, l. 77–97 — l'endroit où une capacité d'écriture est appelée

```

77 /// Where a write capability (a setter, or a closure calling one) handed
78 /// down from an owner is called: an event handler of a host element, or an
79 /// event prop of an element the analysis cannot see into.
80 #[derive(Debug, Clone)]
81 pub(in crate::rules) struct Landing {
82     /// The component whose render builds the element.
83     pub component: ComponentId,
84     pub event: String,
85     pub target: HandlerTarget,
86     /// The handler prop on a host element; the element on an opaque one.
87     pub span: Option<SourceRange>,
88     /// The owner's element the capability leaves through ('None': it lands
89     /// in the owner's own output).
90     pub via: Option<SourceRange>,
91     /// Some hop on the way calls the capability only behind a test of its
92     /// event argument ('if (e.key == "Enter")').
93     pub keyed: bool,
94     /// The module names the handler, and every closure it was handed
95     /// through, may write beside the slot: what reads them changes too.
96     pub writes: Writes,
97 }

```

La descente (`land`, `src/rules/helpers/render_tree.rs`, l. 406–512) examine d'abord les handlers hôtes du composant courant dont les dépendances touchent la question : chacun donne un atterrissage, avec la balise et l'attribut `type`. Elle suit ensuite chaque prop d'un élément de composant qui porte la capacité, en traduisant la question dans le repère de l'enfant (`Setter(0)` du parent devient `Prop(onChange)` de l'enfant). Si une prop d'événement (`onX`) entre dans un composant sans atterrir plus bas (composant opaque, enfant qui ne l'appelle que dans un effet), l'atterrissage est posé sur l'élément lui-même, avec une cible `HandlerTarget::Component`.

Au passage, la descente accumule keyed (un saut n'appelle la capacité que derrière un test de l'argument d'événement) et writes (les noms de module que les closures traversées peuvent écrire).

Exemple 21.2: Un setter passé à un enfant

La fixture tests/fixtures/wasted_subtree_render/child_setter.tsx :

Extrait 21.8 – tests/fixtures/wasted_subtree_render/child_setter.tsx, l. 5–22 — le setter descend dans <Field>

```

5 function Field({ onChange }: { onChange: (v: string) => void }) {
6   return <input onChange={e => onChange(e.target.value)} />;
7 }
8 function Row({ i }: { i: number }) { return <li>row {i}</li>; }
9 function Heavy() {
10   return <ul>[1, 2, 3].map((i) => <Row key={i} i={i} />)</ul>;
11 }
12
13 export function Form() {
14   const [v, setV] = useState("");
15   return (
16     <div>
17       <p>{v}</p>
18       <Field onChange={setV} />
19       <Heavy />
20     </div>
21   );
22 }
```

```

Form (1 hooks) tests/fixtures/wasted_subtree_render/child_setter.tsx
warn wasted-subtree-render [hook:0] (line 18:6) each `change` event in `<Field>` writes
state `v` and re-renders `<Field>` and `<Heavy>` (at least 3 component renders, including a
list) although none of their inputs depends on it. [...]
→ `<Field>` re-renders (1 component render) although none of its props depends on it (line 18:6)
→ `<Heavy>` re-renders (at least 2 component renders, including a list) although none of its
props depends on it (line 19:6)
```

(Sortie repliée, conseil final éliminé en « [...] ».) L'atterrissage est dans Field, donc le finding pointe l'élément <Field> du propriétaire (ligne 18) et le message dit « in <Field> » (champ place du Trigger, src/rules/impls/wasted_subtree_render.rs, l. 101–123). Plus surprenant : <Field> lui-même est déclaré gaspillé. C'est exact. Sa seule prop, onChange, porte {Setter(0)}, et la question est {Slot(0)} : un setter ne change jamais d'identité, donc Field re-rend à chaque frappe pour produire la même sortie.

Les effets : listeners et minuteries. La seconde boucle parcourt la relation slot_writers du composant (définition 14.2 ; src/rules/impls/wasted_subtree_render.rs, l. 124–146). Elle ne garde que les écrivains locaux (w.owner vide : une écriture à travers le setter d'un autre composant est comptée chez cet autre), qui écrivent un slot d'état du composant, depuis une région WriterRegion::Effect(e). Pour chacun, elle retient les lignes de la relation registrations (ADR-34) de l'effet e dont le callback peut appeler le setter. Deux prédicats syntaxiques en décident :

Extrait 21.9 – src/rules/impls/wasted_subtree_render.rs, l. 315–333 — peut-il appeler le setter ? derrière un test ?

```

315 /// Whether a registered callback may call `setter`: an inline function that
316 /// reads it, or anything else (a named listener this walk does not follow).
317 fn may_call(callback: &Expr, setter: &str) -> bool {
318   match callback.peel_ts() {
319     Expr::FnLit { body_cfg, .. } => compute_free_vars(body_cfg).contains(setter),
```

```

320     _ => true,
321   }
322 }
323
324 /// Whether a registered callback calls `setter` only behind a test of its
325 /// event argument.
326 fn keyed_call(callback: &Expr, setter: &str) -> bool {
327   match callback.peel_ts() {
328     Expr::FnLit {
329       params, body_cfg, ..
330     } => param_gated_vars(params, body_cfg).contains(setter),
331     _ => false,
332   }
333 }

```

may_call penche vers « oui » : un callback qui n'est pas une fonction littérale (un listener nommé que ce parcours ne suit pas) est supposé écrire. keyed_call penche vers « non gardé ». Les deux erreurs possibles vont donc dans le même sens : plus de déclencheurs, et des déclencheurs plus fréquents. Le nom de l'événement est celui de l'enregistrement ("resize" dans addEventListener("resize", f)), à défaut le nom du registrar (setInterval).

Exemple 21.3: Une minuterie qui écrit deux slots

Le fichier /tmp/ch21/clock.tsx :

Extrait 21.10 – Deux écritures dans un même tick

```

1 function Row({ i }: { i: number }) { return <li>{i}</li>; }
2 function Agenda() { return <ul>[1, 2, 3].map((i) => <Row key={i} i={i} />)</ul>; }
3
4 export default function Dashboard() {
5   const [now, setNow] = useState(Date.now());
6   const [ticks, setTicks] = useState(0);
7   useEffect(() => {
8     const id = setInterval(() => { setNow(Date.now()); setTicks((t) => t + 1); }, 1000);
9     return () => clearInterval(id);
10  }, []);
11  return (<main><time>{now}</time><Agenda /></main>);
12 }

```

Dashboard (3 hooks) clock.tsx

warn wasted-subtree-render [hook:0] (line 9:2) each `setinterval` event writes states `now` and `ticks` and re-renders ``<Agenda>`` (at least 2 component renders, including a list) although none of its inputs depends on them. Move `now` and `ticks` and the elements that use them into their own component, or build the unrelated elements higher up and pass them in as `children`
→ ``<Agenda>`` re-renders (at least 2 component renders, including a list) although none of its props depends on it (line 16:6)

Les deux écritures appartiennent au déclencheur Effect(1) : un seul finding, qui nomme les deux slots (« states now and ticks »), positionné sur l'effet (ligne 9). L'événement s'appelle setInterval : c'est le nom du registrar, mis en minuscules.

Quand un effet vient d'un hook personnalisé greffé dans le composant, le correctif n'appartient pas au composant : tous les appelants du hook paient le même prix. region_hook (src/rules/impls/wasted_subtree_render.rs, l. 294–313) retrouve le nom du hook par la provenance (hook_provenance) de l'effet greffé, et le message change de conseil. Sur la fixture tests/fixtures/wasted_subtree_render/hook_trigger/, où useWindowSize écrit un objet frais à chaque resize :

```

Page (1 hooks) tests/fixtures/wasted_subtree_render/hook_trigger/Page.tsx
warn wasted-subtree-render [hook:1] (tests/fixtures/wasted_subtree_render/hook_trigger/use-size.ts:5:2)
each `resize` event writes state `size` and re-renders `

` (at least 2 component renders,
including a list) although none of its inputs depends on it. The writes come from
`useWindowSize`, which does this to every component that calls it: write only when a value its
callers read changes, or call it from a smaller component
→ `

` re-renders (at least 2 component renders, including a list) although none of its
props depends on it (line 9:48)

```

La position est dans l'autre fichier (use-size.ts, ligne 5) : c'est celle de l'effet greffé.

21.4.2 Étape 2 : la fréquence, un classement et non une preuve

Un clic qui ouvre une boîte de dialogue re-rend la page une fois. Une frappe dans un champ la re-rend à chaque caractère. Par défaut, la règle ne parle que du second cas. La fréquence d'un événement est calculée par `event_frequency` :

Extrait 21.11 – `src/rules/helpers/render_tree.rs`, l. 720–751 — la fréquence d'un événement sur une cible

```

720 pub(in crate::rules) fn event_frequency(
721     event: &str,
722     target: Option<&HandlerTarget>,
723     keyed: bool,
724 ) -> Frequency {
725     let e = event.to_ascii_lowercase();
726     if MOTION_EVENTS.contains(&e.as_str()) || e == "setinterval" {
727         return Frequency::Continuous;
728     }
729     if !TYPING_EVENTS.contains(&e.as_str()) || (keyed && KEY_EVENTS.contains(&e.as_str())) {
730         return Frequency::Discrete;
731     }
732     let typing = match target {
733         Some(HandlerTarget::Host { tag, input_type }) => match tag.as_str() {
734             "textarea" => true,
735             "input" => input_type
736                 .as_deref()
737                 .is_none_or(|t| TEXT_INPUT_TYPES.contains(&t)),
738             // `contenteditable` hosts: `input` is typing, `change` never fires.
739             _ => e == "input" || e == "beforeinput",
740         },
741         Some(HandlerTarget::Component { name }) => {
742             TEXT_COMPONENT_HINTS.iter().any(|h| name.contains(h))
743         }
744         None => e == "input" || e == "keydown" || e == "keyup",
745     };
746     if typing {
747         Frequency::Continuous
748     } else {
749         Frequency::Discrete
750     }
751 }

```

Elle s'appuie sur quatre tables (`src/rules/helpers/render_tree.rs`, l. 677–713) résumées dans la tableau 21.2. L'arbre de décision est le suivant.

1. Un événement de mouvement (mousemove, scroll, resize, drag...) ou `setinterval` est continu, quelle que soit la cible.
2. Un événement qui n'est pas de frappe (click, mouseover, select...) est discret; un événement clavier (keydown, keyup, keypress) dont l'écriture est gardée par un test de l'événement

(keyed) aussi.

3. Reste un événement de frappe ; la cible tranche. Un `<textarea>` est continu ; un `<input>` l'est si son `type` est absent ou textuel (number et range compris) ; un autre hôte (`contenteditable`) l'est pour `input` et `beforeinput`. Un élément de composant l'est si son nom contient un indice (`Input`, `Search`, `Editor`, `Slider`...). Un listener d'effet, sans cible, l'est pour `input`, `keydown` et `keyup`.

Table	Contenu
MOTION_EVENTS	mousemove, pointermove, touchmove, scroll, wheel, drag, dragover, resize, selectionchange
TYPING_EVENTS	change, input, keydown, keyup, keypress, beforeinput
TEXT_INPUT_TYPES	text, search, email, password, url, tel, number, range
TEXT_COMPONENT_HINTS	Input, TextArea, Textarea, Search, Editor, Slider
KEY_EVENTS	keydown, keyup, keypress

TABLE 21.2 – Les tables de classement de `render_tree.rs`.

Un événement continu ne suffit pas. Un `scroll` qui écrit `window.scrollY > 50` écrit à chaque pas, mais React abandonne toute écriture `Object.is-égale` à la valeur courante : le booléen ne re-rend qu'à ses basculements. La règle exige donc qu'au moins un des slots du déclencheur ait des valeurs *nombreuses*, au sens du domaine des valeurs :

Extrait 21.12 – `src/domains/impls/state_value.rs`, l. 384–401 — un slot à peu de valeurs re-rend au rythme de ses transitions

```

384  /// True if the value ranges over a few primitives only: booleans,
385  /// `null`/`undefined`, a finite set of string constants, a narrow integer
386  /// range. A state holding such a value re-renders only when a write
387  /// *changes* it (React bails out on `Object.is`-equal writes), so however
388  /// often it is written, it re-renders at the rate of its transitions.
389  pub fn is_finitely_valued(&self) -> bool {
390      const NARROW: f64 = 16.0;
391      let num_ok = self.num.is_bottom()
392          || (self.num.is_int
393              && self.num.lo.is_finite()
394              && self.num.hi.is_finite()
395              && self.num.hi - self.num.lo <= NARROW);
396      num_ok
397          && matches!(self.str, StrConst::Bottom | StrConst::Set(_))
398          && self.reference == Stability::Bottom
399          && self.setter == SetterVal::Bottom
400          && !self.other
401  }
```

Un slot est « à valeurs finies » s'il ne contient que des booléens, `null` ou `undefined`, un ensemble fini de chaînes constantes ou un intervalle entier de largeur au plus 16, et aucune référence ni setter. C'est la seule lecture du domaine des valeurs que fait la règle, et elle porte sur le store d'état convergé (`result.state_store`). La fixture `scroll_flag.tsx` (un `scroll` qui écrit un booléen) est muette par défaut, et tire avec `continuousOnly=false`.

Remarque

Le texte de `keyed` vient du fait `gated` de la dépendance de rendu : la part de l'ensemble qu'une valeur fonction n'atteint que derrière un test de ses propres arguments (`Deps::gated_for`, `src/engine/render_deps.rs`, l. 189–198). C'est un fait `must` : un seul contributeur non gardé retire la source. Sur `keyed.tsx`, les trois handlers qui n'écrivent que sur `Enter` ou `Escape` sont

discrets (muets par défaut) ; seul KeyLog, qui écrit e.key à chaque touche, tire.

Le point important est ailleurs : *la fréquence ne prouve rien*. Elle ne décide que de ce qui est montré par défaut. Classer un événement continu comme discret masque un finding que continuousOnly=false rendrait visible ; l'inverse montre un finding par défaut. Aucune des deux erreurs ne change l'affirmation du finding, qui porte sur les entrées des éléments, pas sur la fréquence (ADR-41, conséquences).

21.4.3 Étape 3 : les frères gaspillés

Pour un déclencheur retenu, la question est $rel = \{\text{Slot}(l)\}_l \cup \{\text{Module}(x)\}_x$: les slots du lot, et les noms de module que le lot écrit à côté d'eux (un handler qui fait `cache.x = v` ; `setV(v)` change aussi ce que lisent les lecteurs de cache). `wasted_siblings` y répond :

Extrait 21.13 – `src/rules/helpers/render_tree.rs`, l. 266–326 — les éléments que le lot ne peut pas changer

```

266 pub(in crate::rules) fn wasted_siblings(
267     &self,
268     owner: ComponentId,
269     rel: &Relevance,
270     program: &ProgramAnalysisResult,
271 ) -> Vec<Wasted> {
272     let Some(summary) = self.summaries.get(&owner) else {
273         return Vec::new();
274     };
275     // A provider hands its `value` on by context, which is followed below
276     // (`carried`), and renders its children unchanged: only its mount
277     // condition reaches what it wraps.
278     let touched: Vec<bool> = summary
279         .sites
280         .iter()
281         .map(|s| {
282             s.guard.touches(rel)
283             || (s.provides.is_none()
284                 && (s.spread.touches(rel) || s.props.iter().any(|(_, d)| d.touches(rel))))
285         })
286         .collect();
287     let reached = |mut i: usize| -> bool {
288         loop {
289             if touched[i] {
290                 return true;
291             }
292             match summary.sites[i].parent {
293                 Some(p) => i = p,
294                 None => return false,
295             }
296         }
297     };
298     let mut out = Vec::new();
299     for (i, site) in summary.sites.iter().enumerate() {
300         if reached(i) {
301             continue;
302         }
303         let Some(child) = resolve(site, program) else {
304             continue;
305         };
306         // A consumer of a context the write changes re-renders anyway,
307         // and so does a reader of a module name it writes.

```



```

308         let carried = Relevance::of(
309             contexts_at(summary, i, rel)
310                 .into_iter()
311                 .chain(rel.modules().cloned()),
312         );
313         if self.summaries.get(&child).is_none_or(|s| s.uses(&carried)) {
314             continue;
315         }
316         let mut visiting = HashSet::from([owner]);
317         let (renders, list) = self.subtree_renders(child, &carried, program, &mut visiting, 0);
318         out.push(Wasted {
319             child,
320             span: site.span,
321             renders,
322             list: list || site.in_list,
323         });
324     }
325     out
326 }

```

On la lit en quatre temps.

1. **Touché.** Un site est touché si sa garde, l'une de ses props ou son spread dépend de la question. Pour un provider de contexte (provides), seule la garde compte : sa value ne va pas à ses enfants par les props, elle est suivie autrement.
2. **Atteint.** Un site est écarté s'il est touché *ou* si l'un des éléments dans lesquels il est imbriqué (la chaîne des parent) l'est. Un `<Panel value={text}><Heavy /></Panel>` ne rend pas `<Heavy>` candidat : Panel pourrait lui transmettre la valeur, par contexte ou par `cloneElement`.
3. **Résolu.** Seul un site dont `resolve` trouve le composant (liste 21.4) est candidat. Un élément opaque n'est jamais déclaré gaspillé.
4. **Non utilisateur.** La question transportée (`carried`) rassemble les contextes qui portent la valeur jusqu'à ce site (`contexts_at`) et les noms de module écrits. Si l'enfant *utilise* l'un d'eux, ou s'il n'a pas de résumé, il est écarté : un consommateur du contexte re-rend de toute façon, à bon droit.

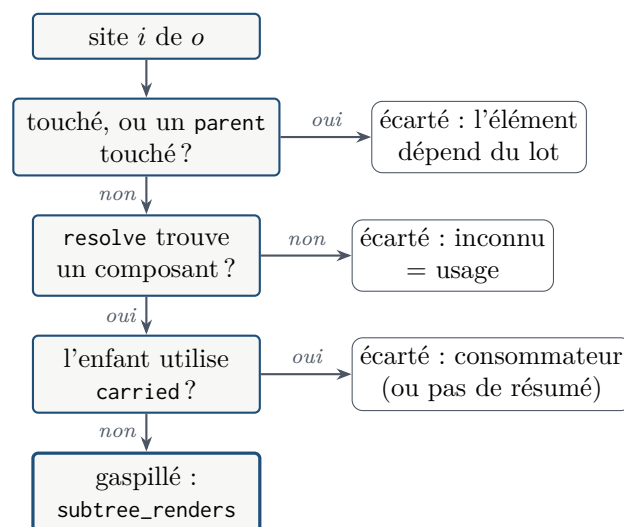


FIGURE 21.2 – Le filtre de `wasted_siblings` sur un site du propriétaire. Toute sortie à droite est un refus de conclure ; seule la sortie du bas produit une affirmation.

La figure 21.2 résume ce filtre. Chaque question y a deux issues, et l'issue prudente est toujours celle qui écarte le site. C'est l'invariant 21.1, écrit en code.

21.4.4 Étape 4 : compter les rendus

Pour un candidat, `subtree_renders` compte les rendus de composant qu'il coûte :

Extrait 21.14 – `src/rules/helpers/render_tree.rs`, l. 328–372 — le coût d'un sous-arbre gaspillé

```

328     /// Component renders one render of `comp` costs for nothing: itself and
329     /// every component element below it, counting a list's item once (a
330     /// lower bound; `list` says the real number is unknown). `carried`: the
331     /// contexts the write changes. A consumer of one re-renders anyway, and
332     /// so may an element the analysis cannot see into; neither is counted,
333     /// nor is anything below them.
334     fn subtree_renders(
335         &self,
336         comp: ComponentId,
337         carried: &Relevance,
338         program: &ProgramAnalysisResult,
339         visiting: &mut HashSet<ComponentId>,
340         depth: usize,
341     ) -> (usize, bool) {
342         let mut renders = 1;
343         let mut list = false;
344         if depth >= MAX_DEPTH || !visiting.insert(comp) {
345             return (renders, list);
346         }
347         if let Some(s) = self.summaries.get(&comp) {
348             for site in &s.sites {
349                 let child = resolve(site, program);
350                 let consumer = match child {
351                     Some(c) => self.summaries.get(&c).is_none_or(|s| s.uses(carried)),
352                     None => !carried.is_empty(),
353                 };
354                 // A provider is not a component render.
355                 if consumer || site.provides.is_some() {
356                     continue;
357                 }
358                 list |= site.in_list;
359                 match child {
360                     Some(child) => {
361                         let (r, l) =
362                             self.subtree_renders(child, carried, program, visiting, depth + 1);
363                         renders += r;
364                         list |= l;
365                     }
366                     None => renders += 1,
367                 }
368             }
369         }
370         visiting.remove(&comp);
371         (renders, list)
372     }

```

Le candidat compte pour un. Chaque élément de composant qu'il construit ajoute le coût de son propre sous-arbre, sauf s'il consomme ce que le lot change (il re-rend à bon droit, et ce qui est sous lui aussi) et sauf s'il est un provider (qui n'est pas un rendu de composant). Un `site in_list` ne compte *qu'une fois* et allume le drapeau `list` : le vrai nombre d'instances est inconnu, peut-être grand, peut-être nul. La récursion et la profondeur (`MAX_DEPTH`) arrêtent la

descente en comptant le composant pour un. Le compte est donc présenté comme une borne inférieure (« at least N component renders, including a list »), avec une exception discutée à la section 21.11.

21.4.5 Étape 5 : seuil et message

Le seuil est appliqué en `src/rules/impls/wasted_subtree_render.rs`, l. 185–190 : le déclencheur produit un finding si au moins un élément est gaspillé et si le total atteint `minWastedRenders`, ou si l'un des sous-arbres contient une liste. Le message assemble le déclencheur (premier événement continu, sinon le premier), les éléments gaspillés, le coût, puis l'un des deux correctifs structurels : déplacer l'état et ses usagers dans leur propre composant, ou construire les éléments indépendants plus haut et les passer en `children`. Chaque élément gaspillé reçoit une note `Step::Rerender`, visible avec `-trace`. Le finding est porté par le label du premier slot et par le span du handler (ou de l'élément `via`, ou de l'effet).

21.5 Des options sur une règle native

Jusqu'à ADR-41, aucune règle native n'acceptait de paramètre (ADR-22 §4). Une règle classée par coût change la donne : savoir à partir de combien de rendus inutiles un sous-arbre mérite d'être signalé, ou si un clic compte, est un choix d'équipe, pas un fait que l'analyseur peut établir. La règle déclare donc deux options typées :

Extrait 21.15 – `src/rules/impls/wasted_subtree_render.rs`, l. 33–48 — les deux options de la règle

```

33     const MIN_WASTED: OptionSpec = OptionSpec {
34         name: "minWastedRenders",
35         kind: OptionKind::UInt {
36             default: 2,
37             min: 1,
38             max: 1000,
39         },
40         doc: "report only when each write re-renders at least this many components for \
41             nothing (a subtree holding a list always qualifies)",
42     };
43     const CONTINUOUS_ONLY: OptionSpec = OptionSpec {
44         name: "continuousOnly",
45         kind: OptionKind::Bool { default: true },
46         doc: "report only states written from a continuous event (typing, pointer motion, \
47             scroll, drag, a timer); `false` also reports clicks and other discrete events",
48     };

```

Le type `OptionSpec` (`src/rules/api/query.rs`, l. 272–288) ne connaît que deux formes, un entier non signé borné et un booléen. Le registre valide chaque valeur configurée *avant* l'analyse, et bruyamment : une clé inconnue ou une valeur hors bornes est une erreur d'usage, jamais un réglage ignoré. Les accesseurs `RuleConfig::uint` et `flag` (`src/rules/api/query.rs`, l. 247–269) peuvent alors rendre la valeur ou le défaut sans vérifier.

```

reactant check typing.tsx --rule wasted-subtree-render \
  --rule-option wasted-subtree-render:minWastedRenders=0

```

```
[error] option `minWastedRenders` of rule `wasted-subtree-render` must be an integer between 1 and 1000
```

Le code de sortie est 2 (erreur d'usage). Les options se donnent aussi dans le fichier de

configuration; `-rule-option` l'emporte sur la valeur de même clé (ADR-41 §4). Avec, dans `/tmp/ch21/cfg/reactant.config.json` :

```
1 { "rules": { "wasted-subtree-render":
2   { "options": { "continuousOnly": false, "minWastedRenders": 1 } } } }
```

le composant `HoverCard` suivant, muet par défaut, produit deux findings :

Extrait 21.16 – Un survol à côté d'un sous-arbre (`/tmp/ch21/cfg/hover_inline.tsx`)

```
1 function Heavy() { return <section>[0, 1, 2].map((i) => <span key={i}>{i}</span>)</section>; }
2 export default function HoverCard() {
3   const [hover, setHover] = useState(false);
4   return (
5     <div className={hover ? "on" : "off"}
6       onMouseOver={() => setHover(true)} onMouseOut={() => setHover(false)}>
7       <Heavy />
8     </div>
9   );
10 }
```

```
HoverCard (3 hooks) hover_inline.tsx
warn  wasted-subtree-render [hook:0] (line 8:9) each `mouseover` event writes state `hover`
and re-renders `<Heavy>` (1 component renders) although none of its inputs depends on it. [...]
warn  wasted-subtree-render [hook:0] (line 8:44) each `mouseout` event writes state `hover`
and re-renders `<Heavy>` (1 component renders) although none of its inputs depends on it. [...]
```

Il fallait les deux options. `mouseover` n'est pas un événement continu et `hover` est un booléen : `continuousOnly=false` est nécessaire. Le `.map` de `Heavy` ne construit que des éléments hôtes (`<span>`), donc aucun site `in_list`, et le coût est d'un seul rendu : `minWastedRenders=1` est nécessaire aussi. Les deux handlers sont deux déclencheurs distincts, donc deux findings.

Décision : ADR-041 §4

Décision. Une règle native peut déclarer des options typées (`Rule::options`), que le registre valide; `-rule-option <rule>:<key>=<value>` bat la configuration. Les défauts (2 et true) viennent d'un balayage du corpus. **Ce qui est amendé.** ADR-22 §4 (« natives declare no params in v1 ») : les seuils d'une règle classée par coût sont l'affaire de l'équipe. Une règle native qui ne déclare rien continue de refuser toute option.

Piège

Une option ne peut qu'ajuster *ce qui est montré*, jamais ce qui est affirmé. Baisser `minWastedRenders` ou couper `continuousOnly` fait apparaître des findings dont l'affirmation (« aucune entrée ne dépend de l'état ») est calculée de la même façon. Aucune option ne relâche l'invariant 21.1.

21.6 Cas limites : ce que les fixtures épinglent

Les fixtures de `tests/fixtures/wasted_subtree_render/` sont le contrat de la règle. Cinq d'entre elles (`typing`, `composer`, `dropzone`, `click`, `children`) viennent de `scripts/rerender-bench/`, où leur nombre de rendus a été mesuré sous React 19 (en-tête de `tests/wasted_subtree_render.rs`). On en retient quatre familles.

Le correctif « children ». C'est le cas limite le plus instructif, parce qu'il illustre le premier point d'arrêt de la cascade (encadré du début du chapitre) :

Extrait 21.17 – tests/fixtures/wasted_subtree_render/children.tsx, l. 3–16 — l'élément lourd est construit plus haut

```

3 // Fix ("lift content up"): the caller builds <Heavy/> and passes it as
4 // children. The element object is created in App's render, which does not
5 // re-run, so React bails out on it.
6 function Heavy() { return <section>[0, 1, 2].map((i) => <span key={i}>{i}</span>)</section>; }
7 function HoverCard({ children }: { children: ReactNode }) {
8   const [hover, setHover] = useState(false);
9   return (
10     <div id="card" className={hover ? "on" : "off"}
11       onMouseOver={() => setHover(true)} onMouseOut={() => setHover(false)}>
12       {children}
13     </div>
14   );
15 }
16 export default function App() { return <HoverCard><Heavy /></HoverCard>; }
```

`<Heavy />` est construit dans le rendu d'`App`, qui ne se ré-exécute pas quand `hover` change ; `HoverCard` reçoit l'objet élément par sa prop `children` et le renvoie tel quel, et React saute son rendu. Côté analyse, c'est immédiat : le résumé de `HoverCard` n'a *aucun* site (l'élément est un site d'`App`), donc `wasted_siblings` n'a rien à examiner. La fixture est muette même avec `continuousOnly=false` et `minWastedRenders=1` (sortie observée : « ✓ 1 file(s) no issues found. »), alors que la variante où `HoverCard` construit lui-même `<Heavy />` (liste 21.16) tire. La dépendance de rendu distingue ainsi, sans règle spéciale, « construire » et « transmettre » un élément.

Le provider. Dans `provider.tsx`, `App` rend `<Ctx.Provider value={text}>` autour de `<Panel><Reader /></Panel>`, et `Reader` lit le contexte. Le provider n'est pas touché (seule sa garde compterait), mais `contexts_at` ajoute `Context(Ctx)` à la question transportée pour tout site imbriqué dans lui. `<Reader>`, site d'`App` imbriqué dans `<Panel>`, lui-même imbriqué dans le provider, utilise ce contexte : il est écarté, rien n'est affirmé sur lui. `<Panel>`, en revanche, ne lit rien : il *est* gaspillé, pour un rendu. La fixture n'est muette par défaut qu'à cause du seuil ; avec `minWastedRenders=1` et `continuousOnly=false`, on observe « re-renders `<Panel>` (1 component renders) ». Le nom du test qui l'épingle, `children_from_above_and_provider_subtrees_are_not_claimed`, décrit donc le sort de `<Reader>`, pas celui de `<Panel>`. La fixture `context.tsx` montre l'autre face (ADR-41, #145) : les éléments qu'un provider enveloppe *peuvent* être gaspillés quand ils ne consomment pas le contexte, et le test `a_provider_is_followed_to_its_consumers` vérifie qu'un consommateur, direct ou à travers un hook, n'est jamais compté.

La garde. Dans `click.tsx`, `{open && <Dialog onClose={...} />}` a une garde `{Slot(0)}` : l'élément existe ou non selon l'état, il est touché, jamais candidat. Seul `<BigList>` est déclaré gaspillé, et seulement avec `continuousOnly=false` :

```

Page (3 hooks) tests/fixtures/wasted_subtree_render/click.tsx
warn  wasted-subtree-render [hook:0] (line 14:24) each `click` event writes state `open` and
      re-renders `<BigList>` (at least 2 component renders, including a list) [...]
warn  wasted-subtree-render [hook:0] (line 15:15) each `click` event in `<Dialog>` writes
      state `open` and re-renders `<BigList>` (at least 2 component renders, including a list) [...]
```

Deux déclencheurs : le bouton d'ouverture du propriétaire, et le bouton de fermeture de `Dialog`, où le setter a atterri à travers la closure `onClose`.

L'élément opaque. Enveloppons `Sidebar` de la liste 21.1 dans `memo (/tmp/ch21/memo.tsx)` : la règle devient muette. Ce n'est pas qu'elle reconnaisse la barrière `memo` ; c'est que `memo(function Sidebar() {...})` ne se résout vers aucun composant enregistré (#64), et qu'un élément opaque

n'est jamais candidat. Le résultat est juste ici, mais pour une raison qui vaut aussi pour une bibliothèque *sans* memo : un sous-arbre lourd importé d'un paquet, à côté d'un champ rapide, n'est pas signalé (docs/limitations.md, « Render cascades stop at what the analysis cannot see into »).

Fixture	Ce qu'elle épingle	Verdict par défaut
typing.tsx	frappe à côté d'un sous-arbre avec liste	WARNING
typing_fixed.tsx	état déplacé dans <Editor>	muet
child_setter.tsx	setter passé à un enfant (#146)	WARNING, <Field> compris
keyed.tsx	écriture derrière e.key : discret (#148)	WARNING sur KeyLog seul
click.tsx	clic discret, garde	muet ; 2 WARNING
scroll_flag.tsx	booléen écrit par scroll	sans continuousOnly
children.tsx	élément reçu en children	muet ; WARNING sans continuousOnly
provider.tsx	consommateur sous un provider	muet dans tous les cas
context.tsx	provider suivi jusqu'aux consommateurs (#145)	muet (seuil : <Panel> coûte 1)
hook_trigger/	effet venu d'un hook personnalisé	2 WARNING
module_write.tsx	nom de module écrit à côté de l'état (#147)	WARNING nommant le hook
		3 WARNING ; 4 sans continuousOnly

TABLE 21.3 – Quelques fixtures de la règle et leur verdict, rejoués au commit photographié.

21.7 Niveau et soundness d'une affirmation d'absence

Ce que dit un finding. Un finding de wasted-subtree-render est la conjonction de trois affirmations : (a) le déclencheur écrit les slots nommés ; (b) chaque écriture re-rend le propriétaire, donc chaque élément gaspillé ; (c) aucune entrée de ces éléments (props, spread, garde, éléments englobants, contextes portés, noms de module écrits) ne dépend de ce que le lot change. La première vient des relations du moteur ; la deuxième est la sémantique de React (chapitre 2) ; la troisième est l'affirmation d'absence.

Polarité renversée. Pour une affirmation d'absence, sur-approximer les *dépendances* est la direction prudente : une dépendance de trop fait taire la règle (un finding de moins), une dépendance oubliée fabriquerait une affirmation fausse. C'est exactement ce que garantit l'analyse avant de render_deps.rs : un may-ensemble, joint par union, dégradé en top au plafond, jamais en plus petit. Et c'est ce que prolonge l'invariant 21.1 au niveau de l'arbre : tout ce qui n'est pas vu compte comme un usage.

Théorème 21.1: Correction de l'affirmation d'absence

Supposons que, pour chaque composant C , le résumé $\text{render_deps}(C)$ sur-approxime les sources dont dépendent la sortie, les effets et les éléments de C (hypothèse de définition 16.3), et qu'aucun appel opaque ne lise ni n'écrive un binding de module (ADR-41 §2). Alors tout élément e rapporté par $\text{wasted_siblings}(o, \text{rel})$ a des props, un spread et une garde indépendants de rel , n'est imbriqué dans aucun élément qui en dépend, se résout vers un

composant *C* qui n'utilise pas ce qui est porté jusqu'à lui, et tout sous-arbre qu'il compte est fait d'éléments résolus non consommateurs.

Esquisse. Chaque exclusion de la figure 21.2 teste touches sur un ensemble qui contient les vraies dépendances ; un test négatif sur un sur-ensemble est négatif sur l'ensemble. Les cas où l'ensemble n'est pas connu (non résolu, top, récursion, profondeur, contexte innommable) sont tous rangés du côté « usage ».

Ce qui fuit. Deux hypothèses restent, toutes deux écrites dans ADR-41. La première est l'appel opaque : une fonction que l'analyse ne voit pas est supposée ne lire ni écrire aucun binding de module (#51, #52). La seconde est la correspondance par orthographe des noms de module entre fichiers : une collision donne *moins* de findings, jamais un faux. Si la première est violée, la règle peut affirmer à tort qu'un élément est gaspillé : c'est un faux positif, toléré au niveau WARNING, et c'est la seule direction où elle peut se tromper sur l'affirmation.

Et les faux négatifs ? Le prix de la prudence est une sous-déclaration : un sous-arbre memo ou de bibliothèque, un élément de nom ambigu, une cascade à travers un contexte importé d'un paquet ne sont jamais signalés. Le projet les classe comme limites de précision (#64 porte l'étiquette *precision-fn*, « sound but incomplete ») et non comme bogues de soundness : l'invariant « faux négatifs interdits » (définition 3.8) porte sur les défauts que l'analyse *prétend* exclure, et cette règle ne prétend rien sur un élément qu'elle n'a pas pu résoudre.

Décision : ADR-041 §1–§3

Décision. La dépendance de rendu est une analyse avant séparée, pas un champ de StateValue. **Alternatives refusées.** (1) Réutiliser Versioned(S) (ADR-17) : ce fait ne vit que sur la composante *reference*, un booléen ou *text.length* n'y portent rien, et une allocation est *PerRender* quelle que soit son origine. (2) Ajouter la dépendance comme composante du produit : toutes les comparaisons de valeurs existantes en seraient changées (*unnecessary-rerender* compare *arg == init*, la clé du *ComponentCache* compare les props). **Niveau.** WARNING : les rendus supplémentaires sont certains, leur coût ne l'est pas (définition du WARNING dans CLAUDE.md).

Remarque

Pourquoi pas ERROR quand le sous-arbre est énorme ? Parce que le niveau ERROR exige un *défaut* certain (définition 17.1). Un rendu inutile n'est pas un défaut : c'est un coût, et la règle n'a aucun moyen de le chiffrer (un composant peut être trivial à rendre). La règle n'a d'ailleurs pas de *safe_check* : une assurance « aucun rendu gaspillé » reposerait sur la même borne inférieure, et n'aurait pas de sens pour un élément non résolu.

21.8 Server Components : le graphe serveur de Next.js

Rappel React : Server Components et "use client"

Sous l'App Router de Next.js, les fichiers réservés d'un dossier *app/* (*page.tsx*, *layout.tsx*...) sont des *Server Components* : React les rend une fois, côté serveur, et envoie le résultat. Il n'y a ni état, ni commit, ni effets ; React n'a aucun hook à leur fournir, et un *useState* y fait échouer le rendu. Une directive "use client", écrite dans le *prologue* d'un module (avant toute instruction), ouvre une frontière : ce module, et tout ce qu'il importe, est compilé pour le client. La propriété « être serveur » n'est donc pas celle d'un fichier isolé, mais de *la*

façon dont on l'atteint par les imports. Un module importé des deux côtés est compilé deux fois, et sa copie serveur doit pouvoir s'exécuter. Voir chapitre 2.

`server-component-hook` répond à une question que le domaine abstrait ignore complètement : dans quel environnement ce module s'exécute-t-il ? Elle s'appuie sur deux faits de nature différente. Le premier est une *convention de nommage* : quels fichiers sont des entrées serveur. Le second est un *graphe* : les arêtes d'import résolues, que le front-end enregistre dans les `ModuleFacts` de chaque fichier (chapitre 5, liste 5.32) : le prologue de directives, verbatim, et les fichiers importés que le résolveur a pu associer à un vrai fichier, `import type` exclu.

21.8.1 Les entrées : une convention de noms

Extrait 21.18 – `src/project/nextjs.rs`, l. 13–38 — les entrées App Router

```

13 /// App Router files React renders on the server unless they opt out.
14 ///
15 /// `error` and `global-error` are deliberately absent: Next *requires* them to
16 /// be Client Components, so they are never server entries.
17 const SERVER_ENTRY_STEMS: &[&str] = &[
18     "page",
19     "layout",
20     "template",
21     "default",
22     "not-found",
23     "loading",
24 ];
25
26 /// The App Router entry kind of `file` (`"page"`, `"layout"`, ...), or `None`.
27 ///
28 /// Requires both halves of the convention: the stem is a reserved entry name
29 /// *and* the file sits under a directory named `app`. The stem alone would
30 /// claim any `components/page.tsx`.
31 pub fn server_entry_kind(file: &Path) -> Option<&'static str> {
32     let stem = file.file_stem()?.to_str()?;
33     let kind = SERVER_ENTRY_STEMS.iter().find(|s| **s == stem)?;
34     file.parent()?
35         .ancestors()
36         .any(|d| d.file_name().and_then(|n| n.to_str()) == Some("app"))
37         .then_some(*kind)
38 }

```

Il faut les deux moitiés de la convention : un nom réservé *et* un ancêtre nommé `app`. Le nom seul désignerait n'importe quel `components/page.tsx`. `error` et `global-error` sont absents à dessein : Next.js exige qu'ils soient des Client Components. Les groupes de routes entre parenthèses passent sans traitement particulier : le test `entry_kinds_need_stem_and_app_dir` accepte `/r/src/app/(marketing)/x/layout.tsx`.

21.8.2 Le graphe serveur : une atteignabilité avec frontière

Extrait 21.19 – `src/project/nextjs.rs`, l. 40–55 — les modules du graphe serveur

```

40 /// Modules Next compiles into the **server** graph: every App Router server
41 /// entry that does not opt out, plus everything they import, stopping at each
42 /// "use client" boundary.
43 ///
44 /// Reachability is what decides it, not the absence of a directive: a module

```



```

45 /// with no directive that nothing server-side imports is simply not part of
46 /// this graph, and a shared module imported from both sides *is* - Next
47 /// compiles it twice, and the server copy is the one that has to run.
48 pub fn server_modules(table: &ModuleTable) -> HashSet<PathBuf> {
49     let seeds: Vec<&Path> = table
50         .paths()
51         .filter(|p| server_entry_kind(p).is_some())
52         .map(PathBuf::as_path)
53         .collect();
54     table.reachable_from(seeds, Some(USE_CLIENT))
55 }

```

`server_modules` part de toutes les entrées (les seeds du parcours), puis appelle le mécanisme unique de la table, `ModuleTable::reachable_from(seeds, Some("use client"))` (liste 5.33, `src/ir/module.rs`, l. 91–126). C'est un parcours en largeur sur les arêtes d'import ; un module qui déclare la frontière n'est ni visité ni traversé, qu'il soit un point de départ ou non. Une page qui porte elle-même "use client" n'est donc pas serveur, et un module que seul un module client importe ne l'est pas non plus. Le coût est $O(V + E)$ sur le graphe de modules.

Définition 21.1: Module serveur

Un module m est *module serveur* s'il existe un chemin d'arêtes d'import résolues $e = m_0 \rightarrow m_1 \rightarrow \dots \rightarrow m_k = m$ tel que e est une entrée App Router (`server_entry_kind(e) \neq \text{None}`) et qu'aucun m_i ($0 \leq i \leq k$) ne déclare "use client" dans son prologue.

Exemple 21.4: Un module partagé et une frontière

Le projet `/tmp/ch21/nx/` (un `next.config.js`, un `tsconfig.json`) contient cinq modules :

Extrait 21.20 – Le projet `/tmp/ch21/nx/` (fichiers concaténés)

```

1 // app/page.tsx
2 import { Search } from "../components/search";
3 import { Price } from "../components/price";
4 export default function Home() { return <main><Search /><Price value={3} /></main>; }
5
6 // app/shop/page.tsx
7 import { useSession } from "next-auth/react";
8 import { useFormat } from "../lib/format";
9 export default function Shop() {
10     const session = useSession();
11     const total = useFormat(10);
12     return <p>{total}{String(session)}</p>;
13 }
14
15 // components/search.tsx
16 "use client";
17 import { useState } from "react";
18 import { Price } from "../price";
19 export function Search() {
20     const [q, setQ] = useState("");
21     return <div><input value={q} onChange={e => setQ(e.target.value)} /><Price value={q.length} /></div>
22     <=> >;
23 }
24
25 // components/price.tsx
26 import { useId } from "react";
27 import { useFormat } from "../lib/format";
28 export function Price({ value }: { value: number }) {
29     const id = useId();

```

```

29   const text = useFormat(value);
30   return <span id={id}>{text}</span>;
31 }
32
33 // lib/format.ts
34 export function useFormat(n: number) { return n.toFixed(2); }

```

La figure 21.3 montre le graphe et la frontière. Les points de départ sont les deux `page.tsx`. Depuis `app/page.tsx`, le parcours atteint `price.tsx`, mais s'arrête à `search.tsx`, qui déclare la frontière. `price.tsx` est importé des deux côtés : il est serveur (par `app/page.tsx`), quoique `search.tsx` l'importe aussi.

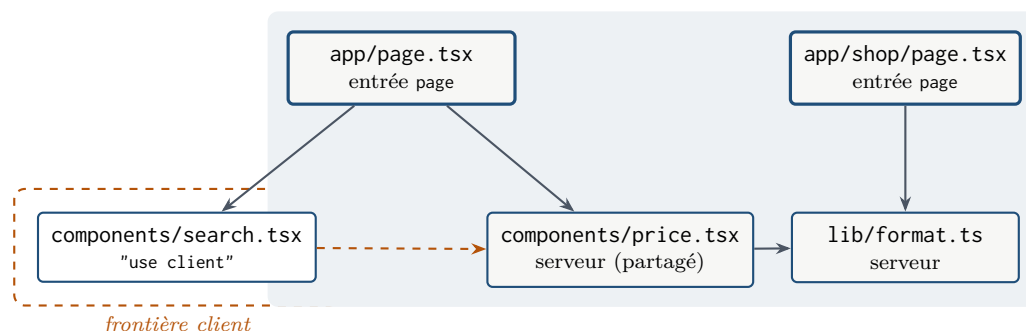


FIGURE 21.3 – Le graphe de modules de `/tmp/ch21/nx/`. Fond bleu : `server_modules`. La frontière "use client" (pointillés orangés) arrête le parcours : `search.tsx` n'est pas visité. L'arête pointillée de `search.tsx` vers `price.tsx` n'est jamais suivie depuis le serveur ; `price.tsx` est serveur par l'arête directe de la page.

Piège

Une directive n'en est une que dans le prologue. Dans `/tmp/ch21/nx3/components/toggle.tsx`,

```

1 import { useState as useLocal } from "react";
2 "use client";
3 export function Toggle() { const [on, setOn] = useLocal(false); /* ... */ }

```

la chaîne `"use client"` suit une instruction d'import : `oxc` ne la range pas dans `Program::directives`, et `Next.js` ne la lit pas davantage. Le module est serveur (il est importé par `app/page.tsx`) et la règle tire, en nommant le hook par son origine malgré l'alias :

```

Toggle (2 hooks) nx3/components/toggle.tsx
warn  server-component-hook [hook:0] (line 5:8) `useState` is called in a Server Component.
      this module is imported into the App Router's server graph and no `use client` directive
      covers it, [...]

```

21.9 L'algorithme de server-component-hook

La règle tient en trois portes et un message (`src/rules/impls/server_component_hook.rs`, l. 129–193).

Porte 1 : le programme utilise-t-il la directive ? Sans un seul `"use client"` dans les modules abaissés (`ModuleTable::any_declares`), la règle se tait. Un code qui n'écrit jamais la directive n'est pas un code Server Components, et qualifier ses modules de « serveur » serait une

fiction (src/ir/module.rs, l. 82–89). Le projet /tmp/ch21/nx2/ est la copie de nx/ sans la directive de search.tsx : aucun finding (« ✓ 6 file(s) no issues found. »), alors que sous Next.js tous ses hooks seraient fautifs.

Porte 2 : le fichier du composant est-il serveur ? server_modules(table) doit contenir comp.file. Dans nx/, Search est client, il n'est pas examiné.

Porte 3 : quels hooks sont des preuves ? Chaque HookEntry du composant passe par client_only_hook, avec sa ligne de provenance (HookProvenance) :

Extrait 21.21 – src/rules/impls/server_component_hook.rs, l. 67–111 — qu'est-ce qui prouve un hook client ?

```

67 /// The client-only hook this entry calls, as it should be named in the
68 /// message; `None` when the entry may legally run on the server.
69 fn client_only_hook(entry: &HookEntry, prov: Option<&HookProvenance>) -> Option<String> {
70     let named = |fallback: &str| {
71         prov.map(|p| p.origin_hook.clone())
72             .unwrap_or_else(|| fallback.to_string())
73     };
74     match entry {
75         // Modelled React hooks: the entry kind *is* the proof, whichever
76         // surface name produced it (`useReducer` files as `State`,
77         // `useLayoutEffect` as `Effect`).
78         HookEntry::State { .. } => Some(named("useState")),
79         HookEntry::Effect { .. } => Some(named("useEffect")),
80         HookEntry::Memo { .. } => Some(named("useMemo")),
81         HookEntry::Callback { .. } => Some(named("useCallback")),
82         HookEntry::Ref { .. } => Some(named("useRef")),
83         // An opaque `useX` is not evidence on its own – plenty of `use`-named
84         // helpers call no hook at all. Only the ones whose origin is a hook
85         // documented as client-only count.
86         HookEntry::Custom {
87             name,
88             import_source,
89             ..
90         } => {
91             let origin = prov
92                 .map(|p| p.origin_hook.as_str())
93                 .unwrap_or(name.as_str());
94             let is_react = prov.is_some_and(|p| p.react);
95             let specifier = prov
96                 .and_then(|p| p.specifier.as_deref())
97                 .or(import_source.as_deref());
98             let react_hit = is_react && REACT_CLIENT_HOOKS.contains(&origin);
99             let package_hit = specifier.is_some_and(|s| {
100                 PACKAGE_CLIENT_HOOKS
101                     .iter()
102                     .any(|(pkg, hooks)| *pkg == s && hooks.contains(&origin))
103             });
104             (react_hit || package_hit).then(|| origin.to_string())
105         }
106         // A DOM handler in a server module is a different Next error
107         // ("event handlers cannot be passed to Client Component props") and
108         // not a hook call.
109         HookEntry::Handler { .. } => None,
110     }
111 }

```

Les cinq hooks modélisés (State, Effect, Memo, Callback, Ref) sont des preuves par leur *genre* : un `useReducer`, rangé en State, en est une, et il est nommé par son origine (`origin_hook`), pas par l'alias local ni par le genre. Un hook Custom n'est une preuve que si son origine figure dans l'une de deux listes fermées :

Extrait 21.22 – `src/rules/impls/server_component_hook.rs`, l. 10–47 — les hooks documentés comme client

```

10 /// React's own hooks that only exist while rendering on the client. The
11 /// engine models the first five as dedicated `HookEntry` kinds; the rest
12 /// reach the IR as `Custom` rows with a `react` provenance.
13 const REACT_CLIENT_HOOKS: &[&str] = &[
14     "useContext",
15     "useId",
16     "useTransition",
17     "useDeferredValue",
18     "useSyncExternalStore",
19     "useImperativeHandle",
20     "useDebugValue",
21     "useOptimistic",
22     "useActionState",
23     "useFormStatus",
24     "useEffectEvent",
25 ];
26
27 /// Client-only hooks of the packages a Next app routes and submits through,
28 /// keyed by the import specifier they must have come from. Package-scoped on
29 /// purpose: a project's own `useRouter` proves nothing.
30 const PACKAGE_CLIENT_HOOKS: &[(&str, &[&str])] = &[
31     (
32         "next/navigation",
33         &[
34             "useRouter",
35             "usePathname",
36             "useSearchParams",
37             "useParams",
38             "useSelectedLayoutSegment",
39             "useSelectedLayoutSegments",
40             "useLinkStatus",
41             "useServerInsertedHTML",
42         ],
43     ),
44     ("next/router", &["useRouter"]),
45     ("next/compat/router", &["useRouter"]),
46     ("react-dom", &["useFormStatus", "useFormState"]),
47 ];

```

La liste des paquets est indexée par le spécificateur d'import : un `useRouter` du projet ne prouve rien, celui de `next/navigation` si (test `the_package_scope_is_load_bearing`). Un `HookEntry::Handler` (un `onClick` sur un élément hôte) n'est pas un hook ; c'est une autre erreur de Next.js, que la règle ne traite pas.

Exemple 21.5: Ce qui n'est pas une preuve

Reprenons `/tmp/ch21/nx/` (liste 21.20) :

```

Price (2 hooks) nx/components/price.tsx
warn  server-component-hook [hook:0] (line 5:8) `useId` is called in a Server Component. this
      module is imported into the App Router's server graph and no `use client` directive covers
      it, so React renders it on the server, where hooks do not exist; add `use client` at the
      top of the file, or move the stateful part into a child component that declares it

```

```
Search (2 hooks) nx/components/search.tsx ✓
Shop (2 hooks) nx/app/shop/page.tsx ✓
```

(Commande : `check nx -rule server-component-hook -show-clean`; les lignes `suspended analysis-limit` de `-info` sont omises.) `useId` est dans `REACT_CLIENT_HOOKS` : `Price`, module partagé, est signalé. `Shop` ne l'est pas : `useSession` vient de `next-auth/react`, paquet absent de la liste, et `useFormat` est un utilitaire au nom en `use` qui n'appelle aucun hook. Le premier cas est un faux négatif réel (`useSession` est un hook client) ; le second est exactement le faux positif que la liste fermée évite.

Le message. Les noms sont dédoublonnés dans l'ordre de première apparition : trois `useState` sont une seule directive manquante. Il y a *un* finding par composant, qui nomme au plus trois hooks puis « and N more », et qui distingue une entrée (« this file is an App Router page ») d'un module atteint transitivement. Sur la fixture `tests/fixtures/next_project/` :

```
HomePage (2 hooks) tests/fixtures/next_project/src/app/page.tsx
warn server-component-hook [hook:0] (line 7:8) `useState` is called in a Server Component.
  this file is an App Router `page` and no `use client` directive covers it, [...]
Sidebar (2 hooks) tests/fixtures/next_project/src/components/sidebar.tsx
warn server-component-hook [hook:0] (line 8:8) `usePathname`, `useState`, `useMemo` are
  called in a Server Component. this module is imported into the App Router's server graph
  and no `use client` directive covers it, [...]
```

`Sidebar` n'a que deux hooks à la surface (`usePathname`, `useNavItems`) ; `useState` et `useMemo` viennent de `useNavItems`, greffé par inlining. Le `Counter` client de la même fixture garde son finding `infinite-loop` : les autres règles ne sont pas supprimées dans un projet Next (`test a_client_component_is_not_reported`).

L'assurance. `safe_check` (`src/rules/impls/server_component_hook.rs`, l. 118–127) ne publie « this Server Component calls no client-only hook » que pour un composant qui passe les portes 1 et 2. Sur un composant client, l'assurance serait vide : il n'y avait rien à vérifier.

21.10 Pourquoi Warning et pas Error

Un `useState` dans une page sans directive fait échouer le rendu à coup sûr. Pourquoi la règle ne produit-elle pas une `ERROR` ? Parce qu'une `ERROR` doit être prouvée dans le domaine, par un jeton `Certified` que seules les primitives `must` savent frapper (définition 17.1), et que les deux faits dont dépend la conclusion n'appartiennent pas au domaine.

- **L'ensemble des entrées** vient d'une convention de noms de fichiers. Un projet peut être configuré autrement ; un dossier `app/` peut ne pas être celui du routeur.
- **Le graphe** vient des arêtes que le résolveur a trouvées. Une arête manquante (alias non résolu) laisse son sous-graphe non classé ; une arête de trop (un `import { type T }` au niveau du spécificateur, compté comme arête de valeur, chapitre 5) pourrait rattacher au serveur un module qui n'y est pas.

Le projet appelle « incertain » un fait qui ne repose pas sur le domaine ; c'est la définition du `WARNING`. On remarquera que les deux règles du chapitre plafonnent au même niveau pour des raisons opposées : `wasted-subtree-render` a la preuve mais pas le défaut (un coût n'est pas un bogue) ; `server-component-hook` a le défaut mais pas la preuve.

Décision : ADR-026 §4

Décision. Les modules serveur sont *analysés*, comme tout composant. La règle est un WARNING, gardée par l'usage de la directive, alimentée par les seuls hooks prouvés client, un finding par composant. **Alternatives refusées.** (1) *Sauter* les Server Components : la qualité de serveur dépend d'arêtes peut-être manquantes, et sauter un module à tort efface ses bogues, un faux négatif. Les analyser ne coûte rien en soundness : sans état, un Server Component est sur-approximé comme tout composant aux props top. (2) Une primitive must qui frapperait un Certified : « it would dress a path convention as a domain proof ». (3) Supprimer les findings des autres règles dans un module serveur : toute erreur de classement deviendrait un faux négatif. (4) Accepter tout `useX` comme preuve : nombre d'utilitaires nommés `use...` n'appellent aucun hook. **Résultat.** Zéro finding sur les corpus Next.js (qui compilent), et un finding correct sous mutation (directive retirée d'une page de ai-chatbot).

La direction de l'erreur est la bonne. Une arête manquante *retire* des modules du graphe serveur, donc des findings : la règle sous-déclare (#29), elle ne sur-déclare pas. Et comme aucun module n'est sauté, cette sous-déclaration ne cache rien aux autres règles.

21.11 Pièges et limites connues

Piège

Le compte des rendus n'est pas toujours une borne inférieure. `subtree_renders` compte *un* rendu pour un élément non résolu *sous* un candidat (`None => renders += 1`, liste 21.14). Dans `/tmp/ch21/unres.tsx`, fonction `Tree() { return <div><Lib /></div>; }` avec `Lib` importé de `some-lib` donne « re-renders `<Tree>` (2 component renders) ». Si `Lib` est un memo, React ne le re-rend pas, et le vrai compte est 1. L'affirmation d'absence n'est pas en cause (`Lib` ne reçoit rien) ; seul le seuil `minWastedRenders` peut être franchi à tort. Au premier niveau, en revanche, le `<Lib />` du même fichier n'est jamais candidat.

Piège

Un `.map` ne fait pas toujours une liste. Seul un site de *composant* construit dans un callback est `in_list`. Un `.map` qui ne produit que des éléments hôtes (`<span>`, `<tr>`) ne rend pas le sous-arbre « liste », et son coût reste d'un rendu (liste 21.16).

Écarts de texte observés. Trois défauts de gabarit, sans effet sur les décisions : « (1 component renders) » au pluriel dans le message principal (`src/rules/impls/wasted_subtree_render.rs`, l. 212–216), alors que la note `Step::Rerender` dit « 1 component render » ; « each setinterval event » pour une minuterie ; « Server Component. this file... » en minuscule après un point (`src/rules/impls/server_component_hook.rs`, l. 179–187). Le commentaire de `REACT_CLIENT_HOOKS` (« The engine models the first five as dedicated HookEntry kinds ») est ambigu : les cinq premiers de la liste (`useContext...`) n'ont pas de genre dédié ; il faut lire « les cinq hooks modélisés, traités par les bras dédiés de `client_only_hook` ».

Limites connues de server-component-hook.

- **Imports non résolus** (#29, `precision-fn`) : un sous-graphe atteint par un alias non résolu n'est pas classé.
- **Run rétréci** : reactant `check tests/fixtures/next_project/src/app` n'abaisse pas `counter.tsx`, seul porteur de "use client". La porte 1 ferme la règle : aucun finding, mais

le blind spot « 3 imported file(s) resolved outside the analysed set and were never read » retient le « clean bill » (chapitre 23).

- **Listes fermées** : un hook client d'un paquet non listé (next-auth/react, exemple 21.5) n'est pas une preuve.
- **Hook en position return** : un hook personnalisé réduit à `return useMemo(...)` est vu, mais le finding sort sans numéro de ligne (#140).
- **Coût** : `server_modules` est recalculé à chaque appel, dans `check` et dans `safe_check`, pour chaque composant : $O(C \cdot (V + E))$ par exécution, là où le `RenderIndex` est mis en cache dans `ProgramCache`. Observation de coût, pas de justesse.

Limites connues de `wasted-subtree-render`. `memo` et `forwardRef` opaques (#64) ; appel opaque supposé sans lecture ni écriture de module (#51, #52) ; une cascade à travers un contexte importé d'un paquet n'est pas suivie (`docs/limitations.md`).

Résumé du chapitre

- Les deux règles du chapitre lisent des faits calculés *à côté* du domaine des valeurs : la dépendance de rendu (ADR-41) pour l'une, les faits de module et une convention de noms (ADR-26) pour l'autre. Toutes deux plafonnent à `WARNING`, l'une faute de défaut (un coût), l'autre faute de preuve dans le domaine.
- `wasted-subtree-render` groupe les écritures par déclencheur (handler, ou enregistrements d'un effet), classe la fréquence (classement, jamais preuve), puis demande à `RenderIndex::wasted_siblings` quels éléments résolus aucune écriture du lot n'atteint, et compte leurs rendus (`subtree_renders`, liste comptée une fois).
- Son affirmation est une absence ; d'où l'invariant 21.1 : non résolu, récursion, profondeur, top, contexte innommable comptent comme usage. Le prix est une sous-déclaration assumée (#64).
- Deux options typées (`minWastedRenders`, `continuousOnly`) ajustent ce qui est montré, jamais ce qui est affirmé.
- `server-component-hook` : porte `any_declares("use client")`, graphe `server_modules` (entrées App Router, atteignabilité arrêtée à la frontière), preuves de hook client en listes fermées, un finding par composant. Les modules serveur restent analysés par toutes les règles.
- Types : `RenderDeps`, `ElementSite`, `Relevance`, `Landing`, `Wasted`, `TriggerKey`, `OptionSpec`, `ModuleFacts`, `ModuleTable`. Fichiers :
`src/rules/impls/wasted_subtree_render.rs`,
`src/rules/helpers/render_tree.rs`, `src/engine/render_deps.rs`,
`src/rules/impls/server_component_hook.rs`, `src/project/nextjs.rs`.
- Le chapitre suivant (chapitre 22) quitte les règles natives pour les règles déclaratives des packs.

21.12 Exercices

Exercice 21.1: Construire ou transmettre

Dans liste 21.1, proposez deux réécritures de `SearchPage` qui font taire `wasted-subtree-render`, l'une par extraction d'état, l'autre par `children`. Pour chacune, dites quel résumé `RenderDeps` change et pourquoi `wasted_siblings` n'a plus de candidat.

Solution

(1) Un composant `SearchBox` possède `query`, rend l'`<input>` et `<Results>`; `SearchPage` rend `<SearchBox />` et `<Sidebar />`. Le slot appartient désormais à `SearchBox`, dont le seul site (`<Results>`) est touché. `SearchPage` n'a plus d'état : aucun déclencheur. (2) `SearchPage({ sidebar })` rend `{sidebar}` et reçoit `<Sidebar />` de son parent : l'élément devient un site du parent, et le résumé de `SearchPage` n'a plus de site `Sidebar` (comme `children.tsx`).

Exercice 21.2: Deux setters, un lot

Un `onChange` appelle `setA(e.target.value)` puis `setB(true)`. Le composant rend `<X a={a} />` et `<Y b={b} />`, plus `<Z />`. Combien de findings, et quels éléments sont nommés ? Même question si `setB` est appelé depuis un `onBlur`.

Solution

Un seul déclencheur (même handler) : question `{Slot(a), Slot(b)}`, `X` et `Y` touchés, seul `<Z>` est candidat ; le finding tire si `Z` coûte au moins `minWastedRenders` rendus ou contient une liste. Avec `onBlur`, il y a deux déclencheurs ; celui de `onBlur` est discret (`blur` n'est pas un événement de frappe) et `b` est un booléen : muet par défaut. Celui du `onChange` a pour question `{Slot(a)}` : `<Y>` et `<Z>` sont candidats. Rejoué (`/tmp/ch21/lot.tsx`, `Z` construisant deux `<Row>`) : « each change event writes states a and b and re-renders `<Z>` (at least 2 component renders, including a list) » pour la première version, « each change event writes state a and re-renders `<Y>` and `<Z>` (at least 3 component renders, including a list) » pour la seconde.

Exercice 21.3: La frontière

Dans figure 21.3, on ajoute "use client" en tête de `price.tsx`. Quels modules restent serveur ? Que devient le finding sur `Price` ? Et si l'on retire la directive de `search.tsx` au lieu d'en ajouter une ?

Solution

`price.tsx` devient une frontière : il n'est plus visité, et `server_modules` vaut `{app/page.tsx, app/shop/page.tsx, lib/format.ts}`. Plus de finding sur `Price` (rejoué : « ✓ 6 file(s) no issues found. »). Sans directive dans `search.tsx`, le programme n'en contient plus aucune : la porte 1 ferme la règle, aucun finding (c'est `/tmp/ch21/nx2/`).

Exercice 21.4: Pourquoi pas une Error ?

Un contributeur propose une primitive `must_proven_server_module(file)` qui rendrait un `Certified` quand le fichier est une entrée page sans directive et sans aucun import. Le cas semble certain. Qu'en dit ADR-26 ? Quel fait reste hors du domaine ?

Solution

Même sans import, le fait « ce fichier est une entrée serveur » reste une convention de chemin (`server_entry_kind`) : le routeur peut être configuré autrement, et le dossier `app/` n'être pas celui de Next.js. ADR-26 §4 refuse précisément cette primitive, qui « habillerait une convention de chemin en preuve de domaine ».

Chapitre 22

Les règles déclaratives et les packs

Objectifs du chapitre.

- Comprendre pourquoi `reactant-analyzer` ouvre ses règles aux équipes par un langage *déclaratif* ancré sur les relations du moteur, et pourquoi il refuse toute échappatoire syntaxique (ADR-22).
- Savoir lire et écrire une règle de pack : ancre, navigation `forEach`, gardes, paramètres, gabarit de message, documentation obligatoire.
- Connaître le système de sortes qui type chaque arête, chaque garde et chaque champ, et savoir prédire le rejet d'un pack mal typé.
- Suivre le chargement en trois étages (`load_pack`) jusqu'à la représentation résolue `ResolvedRule`, et connaître le catalogue des erreurs et des avertissements.
- Suivre l'exécution d'une règle (`TierARule`, `EntityCtx`, conjonction court-circuitée) et le calcul de la sévérité effective « `pin` $\sqcap$ polarité ».
- Maîtriser les trois quantificateurs `any_of`, `every`, `none`, et la place qu'ils réservent à $\top$.
- Connaître l'histoire du vocabulaire (de 3/21 à 21/22 classes exprimables) et ses limites, dont la brèche de soundness ouverte #143.

Prérequis : chapitre 17 (trait `Rule`, `RuleCtx`, jeton `Certified`, définition 17.1 ; chaîne de témoins, définition 17.2 ; clamp du registre) ; chapitre 14 (relations du moteur, définition 14.1 ; `slot_writers`, `registrations`, lectures et appels de corps) ; chapitre 18 et chapitre 20 pour comparer les règles de pack à leurs cousines natives. Les notions de fait `must/may` (définition 3.7), de soundness (définition 3.8) et de stabilité (définition 10.3) sont supposées acquises.

Les chapitres précédents ont décrit les dix-neuf règles *natives* de `reactant-analyzer`, écrites en Rust contre la surface typée du chapitre 17. Une équipe qui adopte l'outil veut pourtant très vite ajouter les siennes : bannir un hook retiré, fixer un budget de dépendances, repérer un motif maison. Ce chapitre décrit la réponse du projet, le *Tier A* : un langage de règles *déclaratif*, écrit en JSON, chargé au démarrage, validé par le cœur, puis exécuté exactement comme une règle native. Le module l'annonce dès son en-tête :

Extrait 22.1 – `src/rules/declarative/mod.rs`, l. 1–16 — le contrat du sous-système

```
1 /// Tier A (ADR-022): declarative rule packs over semantic anchors.
2 ///
3 /// A pack is inert JSON evaluated by the trusted engine: anchors bind rows of
4 /// engine-resolved relations (post alias-resolution/inlining/fixpoint), guards
5 /// are predicates over polarity-typed verdicts, and there is no syntax
6 /// position anywhere in the schema** – a rule that cannot be expressed
7 /// semantically is refused, never emulated (the ADR's scope principle).
8 ///
9 /// This module lives under `src/rules/` on purpose: the executor inherits the
10 /// ADR-021 typestate (it cannot mint `Certified`, and `Diagnostic::error` is
11 /// the only Error door) and reaches the `pub(crate)` helper relations without
12 /// any visibility widening.
```

```

13 ///
14 /// [`load_pack`] is the whole public surface: JSON in (from *any* host – the
15 /// core re-validates every pack it receives, ADR-022 §6), executable rules +
16 /// owned docs out.

```

Tout le chapitre développe les quatre phrases de cet en-tête : le pack est *inerte* (du JSON, pas du code) ; ses ancres sont des *relations déjà résolues* par le moteur ; ses gardes sont des prédicats sur des verdicts *polarisés* ; et il n'existe *aucune position syntaxique* dans le schéma. La distribution des packs (paquet npm, cœur WASM, compilation JS/TS vers JSON, action GitHub) est laissée au chapitre 24.

22.1 Pourquoi un langage de règles, et pourquoi pas ESLint

22.1.1 Deux demandes d'équipe

Partons de deux demandes réalistes. La première : « signaler tout effet qui écrit un état qu'il liste dans ses propres deps ». La seconde : « interdire le hook `useLegacyStore`, retiré par l'équipe ». Chacune s'écrit en quelques lignes de règle ESLint : chercher un appel `setX(...)` dans le corps d'un `useEffect` dont le tableau contient `x` ; chercher un appel dont le callee s'appelle `useLegacyStore`. Voici deux programmes qui mettent ces règles syntaxiques en échec.

Extrait 22.2 – Un setter aliasé (/tmp/ch22/alias.tsx)

```

1 import { useEffect, useState } from "react";
2
3 export function Ticker() {
4   const [count, setCount] = useState(0);
5   const bump = setCount;
6   useEffect(() => {
7     bump(count + 1);
8   }, [count]);
9   return <div>{count}</div>;
10 }

```

Extrait 22.3 – Un hook retiré caché derrière un wrapper (/tmp/ch22/wrapper.tsx)

```

1 import { useLegacyStore } from "legacy";
2
3 function useCartStore() {
4   return useLegacyStore();
5 }
6
7 export function Cart() {
8   const store = useCartStore();
9   return <div>{String(store)}</div>;
10 }

```

Dans le premier, l'appel écrit `bump(count + 1)` : aucun motif « set suivi d'une majuscule » ne le reconnaît, et reconnaître `bump` demande de suivre la liaison `const bump = setCount`. Dans le second, le composant `Cart` n'appelle jamais `useLegacyStore` par son nom ; il faut savoir que `useCartStore` l'appelle, c'est-à-dire inliner le wrapper. Les deux règles de pack `guardrails/self-retriggering-effect` et `guardrails/banned-hook`, que nous lirons en détail plus loin, trouvent les deux défauts (deux exécutions, chacune filtrée par `-rule` sur la règle concernée) :

```

Ticker (2 hooks) alias.tsx
  error guardrails/self-retriggering-effect [hook:1] (line 7:4) this effect writes `count`, which is listed in its
    ↳ own dependency array

```

```
Cart (1 hooks) wrapper.tsx
warn  guardrails/banned-hook [hook:1] `useLegacyStore` is withdrawn by team policy
```

Elles n'ont rien fait de spécial pour cela : la résolution des alias, l'inlining des hooks personnalisés et le point fixe ont tourné *avant* les règles (chapitres 8 et 13). Une règle qui part d'une relation du moteur hérite de ce travail ; une règle qui part de la syntaxe le perd.

Remarque

Le second finding n'a pas de position (« line » vaut null dans la sortie JSON). La cause tient au corps de `useCartStore` : il ne lie pas l'appel par un `let`, il le *retourne* nu, `return useLegacyStore();`. Ce site est un `Terminator::Return`, seul terminator à ne porter aucun champ `span` — `Terminator::Branch`, lui, en a un (`src/ir/cfg.rs`, l. 13–31). L'extracteur de hooks ne regarde jamais les terminators ; `hoist_terminator_hooks` déplace donc l'appel dans un `Stmt::Let` synthétique pour que l'extraction ordinaire le voie, mais ce `Let` hérite de `span: None` faute de mieux (`src/lowering/hook_extractor.rs`, l. 321–339). La ligne de `hook_origins` ainsi produite — puis recopiée telle quelle par l'inlining de `useCartStore` dans `Cart` — porte donc `span: None`, et `Candidate::range` (`src/rules/declarative/exec.rs`, l. 193–219, bras `Origin`) la restitue sans position. Le défaut est connu et suivi (#140) : donner un `span` à `Terminator::Return` est un changement mécanique mais qui touche une quarantaine de sites IR, jugé hors périmètre au moment du fix du terminator-hoist lui-même.

22.1.2 Le principe de périmètre

La leçon est érigée en principe normatif par ADR-22 :

*« reactant does semantics only. It does not compete with ESLint or any syntax-level linter: a rule that cannot be expressed against the engine's semantic relations is **refused**, never emulated with a syntactic fallback. »*

La justification tient en une phrase, que l'ADR donne telle quelle : « *the full pipeline — alias resolution, inlining, fixpoint — runs before rules, so every semantic anchor inherits the engine's soundness for free, while any AST escape hatch would forfeit it* ». Il suffit d'une seule ancre syntaxique pour réintroduire le *plancher de faux négatifs* que toute la surface typée existe pour supprimer : un filtre de syntaxe ne voit pas l'alias, ne voit pas le wrapper, ne voit pas la prop qui transporte un setter, et il ne le saura jamais.

Décision : ADR-22 §1 — des ancrs sémantiques, jamais de syntaxe

Décision. Une règle déclarative commence toujours par choisir une *relation que le moteur a déjà résolue* (la table des appels de hooks, les écrivains d'un slot, les éléments JSX construits par le rendu...); les noms n'y apparaissent que comme *filtres sur des entités résolues*, jamais comme motifs textuels sur la syntaxe d'appel. Corollaire : le schéma est dérivable de `src/rules/api/` — une relation sélectionnable est une sorte d'entité, une garde est une primitive publique.

Alternatives refusées. Une échappatoire AST « pour les cas simples » (elle réintroduit le plancher de FN) ; une règle « bannir `moment()` dans les composants » formulée comme filtre d'import externe (refusée en v1, ADR-22 §Limitations) ; laisser l'outil concurrencer ESLint, qui fait très bien la syntaxe.

22.1.3 Trois tiers, un seul est livré

ADR-21 (chapitre 17) esquissait trois niveaux de frontends pour les règles. ADR-22 et ADR-23 les ont tranchés :

- **Tier A** : des règles déclaratives JSON, livrées, objet de ce chapitre ;
- **Tier B** : un frontend programmable. Starlark a été rejeté (ADR-23 §5 : la crate ne compile ni pour l'hôte ni pour wasm32, et elle alourdit une distribution de moins d'un mégaoctet) ; la voie retenue pour la communauté est d'*écrire* un pack en JS ou TS, puis de le compiler en JSON committé (chapitre 24) ;
- **Tier C** : les règles Rust de première main, seules à pouvoir créer une primitive et donc une autorité ERROR.

Le mot « inerte » de l'en-tête a donc un sens précis : un pack ne contient *aucun code*. Même écrit en JS, il est évalué une fois, sur la machine de l'auteur, et c'est le JSON résultant qui s'exécute en CI.

22.1.4 Place dans le pipeline

Le sous-système intervient à deux moments (figure 22.1). *Au chargement*, avant toute analyse, le texte JSON est désérialisé, validé et « cuit » en une représentation typée `ResolvedRule`, emballée dans un objet `TierARule` qui implémente le trait `Rule`. *À l'exécution des règles*, après le point fixe, le registre appelle `TierARule::check` pour chaque composant, comme il appelle une règle native ; la règle construit un adaptateur `EntityCtx` vers les relations, énumère les lignes de l'ancre, évalue les gardes et émet des `Diagnostic`.

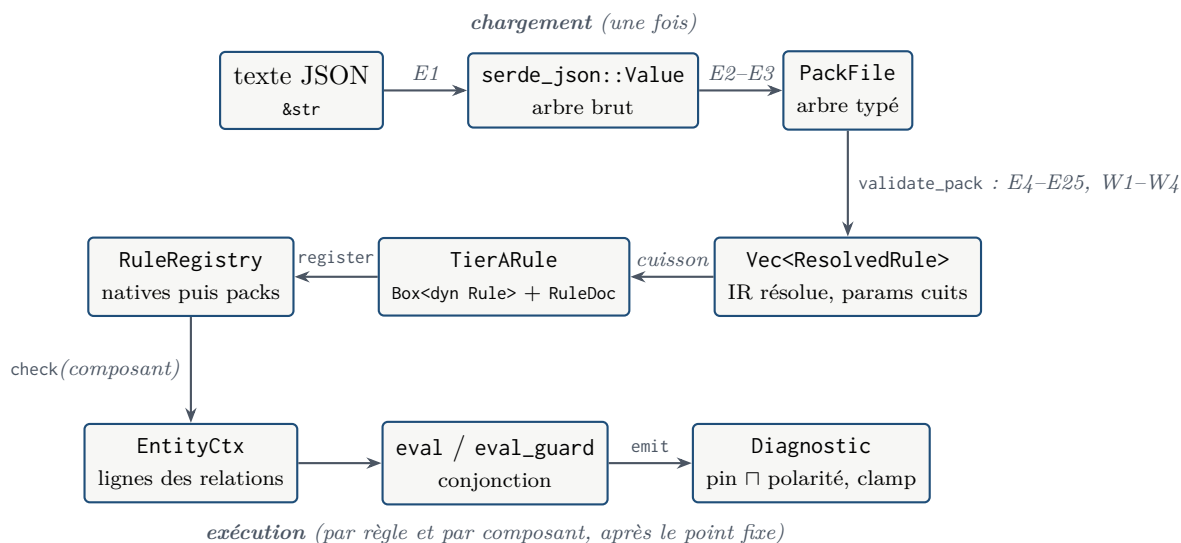


FIGURE 22.1 – Le flot d'une règle déclarative, de son texte à son diagnostic. Les étiquettes E_n et W_n renvoient au catalogue des erreurs et avertissements de la section 22.4 : chaque étage rejette ce qu'il sait rejeter, avec un chemin JSON exact.

Côté CLI, le chargement est fait par `load_config_and_registry` (`src/cli/config_load.rs`, l. 55–89), commun aux sous-commandes `check`, `rules` et `explain` : pour chaque pack, *dans l'ordre de la configuration*, on résout le chemin, on lit le fichier, on appelle `declarative::load_pack`, on imprime les avertissements et on enregistre chaque règle. Tout échec est bruyant et termine avec le code de sortie 2. Le commentaire en donne la raison : « *ignoring a configured pack would run fewer rules than the config asks for — the config-level analogue of a false negative* ». Le registre

refuse en outre un nom sans /, un doublon et une documentation dont le nom ne coïncide pas avec l'identifiant (`src/rules/registry.rs`, l. 142–158).

22.2 Anatomie d'une règle

22.2.1 La plus petite règle utile

Le pack `guardrails`, livré avec l'outil (`packs/guardrails.json`), contient cinq règles. La première signale un effet sans tableau de `deps` :

Extrait 22.4 – `packs/guardrails.json`, l. 6–18 — la règle `effect-without-deps-array`

```

6  {
7    "id": "effect-without-deps-array",
8    "docs": {
9      "description": "an effect declares no dependency array at all",
10     "why": "Without a deps array the effect re-runs after every single render, including renders caused
    ↪ by unrelated state. It is the one deps-array state no native rule inspects: `always-unstable-deps`
    ↪ and `missing-deps` both need an array to reason about, so this shape is invisible today. Note that
    ↪ the omission is sometimes deliberate – the latest-ref idiom (`useInsertionEffect(() => { ref.
    ↪ current = fn })`, a userland `useEffectEvent`) requires it – so treat this as a review trigger,
    ↪ not a defect claim.",
11     "fix": "Add the array: `[ ]` if the effect really is mount-only, otherwise what the body reads. If
    ↪ the omission is the latest-ref idiom, turn this rule off for the hook that implements it.",
12     "example": "useEffect(() => { document.title = title; }) // runs after every render"
13   },
14   "severity": "warning",
15   "anchor": { "relation": "hook_calls", "kind": "effect" },
16   "guards": [{ "kind": "deps_declared", "of": "anchor", "eq": false }],
17   "message": "this effect declares no dependency array – it re-runs after every render"
18 },

```

Extrait 22.5 – Un titre synchronisé après chaque rendu (`/tmp/ch22/ex1_no_deps.tsx`)

```

1  import { useEffect } from "react";
2
3  export function Title({ title }: { title: string }) {
4    useEffect(() => {
5      document.title = title;
6    });
7    return <h1>{title}</h1>;
8  }

```

```

Title (1 hooks) ex1_no_deps.tsx
warn  guardrails/effect-without-deps-array [hook:0] (line 4:2) this effect declares no dependency array – it re-
    ↪ runs after every render

! 1 warning(s) across 1 file(s).

```

Lisons la règle champ par champ.

- `id` : l'identifiant *nu* de la règle ; elle est adressée `guardrails/effect-without-deps-array`, le nom du pack servant d'espace de noms.
- `docs` : obligatoire. `description` devient le résumé affiché par `reactant rules`, `why` l'explication de `reactant explain`, `fix` le remède ; `example` est facultatif. Le commentaire du type l'impose sans détour : « *a custom rule without an explanation is exactly the diagnostic a team learns to ignore* » (`src/rules/declarative/schema.rs`, l. 58–73).

- `severity` : un *plafond* (`error`, `warning` ou `info`), pas une sévérité. Nous verrons en section 22.6 que la sévérité effective se calcule par `finding`.
- `anchor` : la relation dont chaque ligne est un candidat, ici `hook_calls` filtrée aux effets. Il y a une ligne par appel d'effet du composant.
- `guards` : une liste de prédicats, lue comme une *conjonction*. Ici une seule garde, `deps_declared` avec `eq: false` : « l'appel n'a pas reçu de tableau de deps ».
- `message` : un gabarit, ici sans champ interpolé.

À l'exécution, l'unique ligne est le `useEffect` de label 0 ; la garde évalue `row.effect.is_some_and(|i| i.has_deps_array()) == false`, vraie car l'argument est absent (`src/rules/declarative/exec.rs`, l. 682–687) ; le plafond `warning` donne un `WARNING`, placé sur le site du hook (« `[hook:0]` (line 4:2) »).

Rappel React : Le tableau de deps

Sans tableau, un effet se ré-exécute après *chaque* commit ; avec `[]`, seulement au montage ; avec `[a, b]`, quand l'un des éléments n'est plus `Object.is`-égal à sa valeur du rendu précédent (chapitre 2). La garde `deps_declared` ne regarde que la présence de l'argument : `useEffect(f, rest)` (une variable), `[]` ou `[rest]` as `const` déclarent tous un tableau, ce que vérifie le test `deps_declared_asks_whether_an_argument_was_passed_at_all` (`tests/declarative.rs`, l. 2047–2091).

22.2.2 Le fichier de pack

Le modèle `serde` du fichier est la source unique du JSON Schema que publie le dépôt ; la commande `reactant schemas` en tire le fichier `docs/schemas/pack.schema.json`. Voici ce modèle :

Extrait 22.6 – `src/rules/declarative/schema.rs`, l. 17–56 — `PackFile` et `RuleDef`

```

17 #[derive(Debug, Deserialize)]
18 #[cfg_attr(feature = "schema-gen", derive(JsonSchema))]
19 #[serde(deny_unknown_fields)]
20 pub struct PackFile {
21     /// Editor-facing schema URL; not interpreted. Accepted for the same reason
22     /// `reactant.config.json` accepts it: a published schema is only useful if
23     /// the file is allowed to point at it.
24     #[serde(rename = "$schema", default)]
25     pub schema: Option<String>,
26     /// Format version; only `1` exists.
27     #[serde(rename = "schemaVersion")]
28     pub schema_version: u32,
29     /// Pack name: the namespace of every rule id (`<name>/<rule>`).
30     pub name: String,
31     pub rules: Vec<RuleDef>,
32 }
33
34 #[derive(Debug, Deserialize)]
35 #[cfg_attr(feature = "schema-gen", derive(JsonSchema))]
36 #[serde(deny_unknown_fields)]
37 pub struct RuleDef {
38     /// Bare rule id (no `/`); addressed as `

```

```

46  #[serde(default)]
47  pub params: BTreeMap<String, ParamDecl>,
48  pub anchor: Anchor,
49  #[serde(rename = "forEach", default)]
50  pub for_each: Option<ForEach>,
51  #[serde(default)]
52  pub guards: Vec<Guard>,
53  /// Message template interpolating navigated entities ( `{setter.slot}` )
54  /// and params ( `{param.maxDeps}` ); `{ `/{ `}` ` escape braces.
55  pub message: String,
56  }

```

schemaVersion doit valoir 1 ; name ne doit être ni vide, ni contenir /, ni coïncider avec un nom de diagnostic natif ; les identifiants de règles sont nus et uniques dans le pack ; l'ordre des règles est l'ordre d'enregistrement. Toutes les structures « plates » portent deny_unknown_fields : une clef inconnue est une erreur, jamais un silence. Les deux énumérations à étiquette interne (Anchor, étiquetée par relation, et Guard, étiquetée par kind) ne le permettent pas avec serde ; le validateur refait donc la vérification sur le JSON brut (check_keys, src/rules/declarative/validate.rs, l. 970–990).

Définition 22.1: Pack, règle déclarative

Un *pack* est un fichier JSON {schemaVersion, name, rules}. Une *règle déclarative* y est la donnée d'une *ancree* (une relation du moteur), d'une *navigation* facultative le long d'au plus une arête typée (forEach), d'une conjonction de *gardes*, d'un *gabarit* de message, d'une documentation et d'un *pin* de sévérité. Une ligne de l'ancree, éventuellement accompagnée de l'élément navigué, est un *candidat* ; un candidat qui passe toutes les gardes produit un finding.

22.2.3 Une grammaire

Le JSON Schema publié compte près de deux mille lignes. Une grammaire abrégée, écrite à la main à partir de schema.rs (ce n'est pas un artefact du dépôt), en donne la structure :

Extrait 22.7 – Grammaire abrégée d'un pack (pseudo-EBNF, rédigée pour ce chapitre)

```

1  Pack    ::= { "schemaVersion": 1, "name": Ident, "rules": [ Rule* ] }
2  Rule    ::= { "id": Ident, "docs": Docs, "severity": Pin,
3              "params"? : { Ident: { "type": PType, "default": Json } * },
4              "anchor": Anchor,
5              "forEach"? : { "edge": Edge, "as": Ident },
6              "guards"? : [ Guard* ],
7              "message": Template }
8  Docs    ::= { "description": Str, "why": Str, "fix": Str, "example"? : Str }
9  Pin     ::= "error" | "warning" | "info"
10 PType   ::= "number" | "string" | "boolean" | "string[]"
11 Anchor  ::= { "relation": "hook_calls", "kind"? : HookKind }
12          | { "relation": "jsx_props" | "elements", "elements"? : "component" | "host" | "any" }
13          | { "relation": "registrations", "firing"? : "repeating" | "once" }
14          | { "relation": "render_setter_calls" | "render_calls" | "hook_origins"
15              | "context_providers" | "churn_cycles" | "context_consumers" }
16 Edge    ::= "deps" | "body_setter_calls" | "args" | "writers"
17          | "calls" | "props" | "reads" | "seeds"
18 Guard   ::= Filter                                (24 kinds)
19          | Must                                    (5 kinds, "else"? : "keep" | "drop")
20          | { "kind": "any_of", "guards": [ Guard, Guard+ ] }
21          | { "kind": "every", "of": "anchor.deps", "as": Ident, "guards": [ Guard+ ] }
22          | { "kind": "none", "of": "anchor." Edge, "as": Ident, "guards": [ Guard+ ] }

```



```

23 Leaf      ::= Value | { "$param": Ident }           (leaf constant positions only)
24 Template::= ( Text | "{" Ident "." Ident "}" | "{" " | "}" ) *
```

Trois traits de cette grammaire sont des décisions. Il n'y a *qu'une* ancre et *au plus une* arête : pas de seconde variable libre, pas de jointure (ADR-22 §2, qui constatait que toutes les règles natives de l'époque avaient cette forme, sauf le bras inter-composants de *stale-closure*, d'où la limite #68). Les paramètres ne se glissent qu'aux feuilles : « *Parameters are values, not structure* » (ADR-22 §4). Enfin la liste de gardes est plate et conjonctive ; la disjonction et les quantificateurs sont des gardes comme les autres.

22.2.4 Paramètres et options du consommateur

Deux autres règles de guardrails montrent les paramètres. `guardrails/oversized-effect` fixe un budget de deps, `guardrails/banned-hook` une liste de hooks retirés :

Extrait 22.8 – `packs/guardrails.json`, l. 65–71 — `oversized-effect` (docs omises)

```

65   "severity": "warning",
66   "params": { "maxDeps": { "type": "number", "default": 5 } },
67   "anchor": { "relation": "hook_calls", "kind": "effect" },
68   "guards": [
69     { "kind": "count", "of": "anchor.deps", "more_than": { "$param": "maxDeps" } }
70   ],
71   "message": "this effect declares more than {param.maxDeps} dependencies – split it by concern"
```

Extrait 22.9 – `packs/guardrails.json`, l. 81–87 — `banned-hook` (docs omises)

```

81   "severity": "warning",
82   "params": { "banned": { "type": "string[]", "default": [] } },
83   "anchor": { "relation": "hook_origins" },
84   "guards": [
85     { "kind": "name", "of": "anchor", "one_of": { "$param": "banned" } }
86   ],
87   "message": "{anchor.name} is withdrawn by team policy"
```

Un paramètre se déclare avec un type et un défaut *obligatoire* ; il s'utilise par `{"$param": "maxDeps"}` en position de feuille, ou par `{param.maxDeps}` dans le gabarit. Le consommateur le surcharge dans `reactant.config.json`, sous la clef `rules`. La configuration de ce chapitre (`/tmp/ch22/reactant.config.json`) est :

Extrait 22.10 – La configuration des exemples du chapitre

```

1 {
2   "packs": [
3     "/home/rboudrouss/reactant-analyzer/packs/guardrails.json",
4     "/home/rboudrouss/reactant-analyzer/packs/community/wave2.json"
5   ],
6   "rules": {
7     "guardrails/banned-hook": { "options": { "banned": ["useLegacyStore"] } }
8   }
9 }
```

Sur un composant qui importe le hook sous un alias et déclare sept deps,

Extrait 22.11 – Un hook retiré importé sous alias, un effet trop large (`/tmp/ch22/ex4_banned.tsx`)

```

1 import { useEffect } from "react";
2 import { useLegacyStore as useStore } from "legacy";
```



```

3
4 export function Cart({ a, b, c, d, e, f }: Record<string, number>) {
5   const store = useStore();
6   useEffect(() => {
7     console.log(store, a, b, c, d, e, f);
8   }, [store, a, b, c, d, e, f]);
9   return <div />;
10 }

```

l'analyseur répond :

```

Cart (2 hooks)  ex4_banned.tsx
warn  guardrails/banned-hook [hook:0] (line 5:8) `useLegacyStore` is withdrawn by team policy
warn  guardrails/oversized-effect [hook:1] (line 6:2) this effect declares more than 5 dependencies – split it by
      ↳ concern

! 2 warning(s) across 1 file(s).

```

Trois faits sont à retenir. D'abord la *cuisson* : les valeurs effectives des paramètres (défaut, écrasé par l'option) sont substituées *au chargement* dans la représentation résolue et dans le gabarit ; {param.maxDeps} est devenu le littéral « 5 » avant la première analyse, et l'exécuteur ne voit plus jamais de paramètre. Ensuite, la ligne de hook_origins porte l'identité *résolue* du hook, useLegacyStore, malgré l'alias useStore : la garde name filtre une entité, pas un texte. Enfin la validation des options est bruyante. Avec une option mal typée (/tmp/ch22/opt/reactant.config.json, "maxDeps": "eight"), tout le pack est refusé :

```

[error] pack `/home/rboudrouss/reactant-analyzer/packs/guardrails.json` (/home/rboudrouss/reactant-analyzer/packs/
      ↳ guardrails.json): at `rules[3].options.maxDeps`: option value "eight" does not match declared type `number`
exit=2

```

Le type générique qui porte ces positions est PVal<T>, « valeur ou paramètre », doté d'un Deserialize manuel :

Extrait 22.12 – src/rules/declarative/schema.rs, l. 899–936 — PVal, une feuille qui accepte une valeur ou une référence de paramètre

```

899 /// Value-or-parameter: a leaf constant position that accepts either a JSON
900 /// value of type `T` or `{ "$param": "<name>" }` (ADR-022 §4, parameters are
901 /// values, never structure).
902 #[derive(Debug, Clone, PartialEq)]
903 pub enum PVal<T> {
904     Value(T),
905     Param(String),
906 }
907
908 impl<'de, T: serde::de::DeserializeOwned> Deserialize<'de> for PVal<T> {
909     fn deserialize<D>(deserializer: D) -> Result<Self, D::Error>
910     where
911         D: serde::Deserializer<'de>,
912     {
913         use serde::de::Error;
914         let v = serde_json::Value::deserialize(deserializer)?;
915         if let serde_json::Value::Object(m) = &v
916             && let Some(p) = m.get("$param")
917         {
918             if m.len() != 1 {
919                 return Err(D::Error::custom(
920                     "a { \"$param\": ... } reference takes no other key",
921                 ));
922             }
923             return match p {
924                 serde_json::Value::String(s) => Ok(PVal::Param(s.clone())),

```

```

925         other => Err(D::Error::custom(format!(
926             "\"$param\" expects a parameter name string, got {other}\"
927         )),
928     };
929 }
930 T::deserialize(v).map(PVal::Value).map_err(|e| {
931     D::Error::custom(format!(
932         "expected a value or {{\"$param\": \"<name>\"}}: {e}\"
933     ))
934 })
935 }
936 }

```

Piège

Le JSON Schema publié est *plus permissif* que le validateur. Toutes les listes de noms de verdicts y sont typées `PVal<Vec<...Name>`, si bien qu'un éditeur accepte `"is": {"$param": "v"}` dans une garde `stability`. Mais le validateur résout ces positions avec `expected = None` (« cette position ne prend aucun paramètre »), et le chargement échoue : « this position does not accept a {"\$param": ...} reference » (`src/rules/declarative/validate.rs`, l. 1129–1168). Les positions réellement paramétrables sont les listes de noms (`one_of`, `origin.hook`, `provenance.through`), les préfixes, les booléens (`origin.direct`, `cycle.*`, `deps_declared.eq`) et les seuils de `count`. Un paramètre est une constante « métier », jamais un choix de polarité.

Piège

Un paramètre `number` accepte un négatif ou un flottant (`value_matches` ne teste que `is_number`), mais la position `count` déséréalise en `u64`. Un défaut ou une option `-1` passe donc la déclaration puis échoue à la résolution (« param 'n' value is unusable here: invalid value: integer '-1', expected u64 »), ce qui rejette le pack entier.

22.2.5 Le gabarit de message

Le gabarit interpole des champs des entités liées, `{anchor.name}` ou `{setter.slot}`, et des paramètres, `{param.maxDeps}`; `{{` et `}}` échappent les accolades. L'analyse du gabarit (`src/rules/declarative/validate.rs`, l. 2392–2484) le parcourt caractère par caractère, exige la forme `liaison.champ`, substitue immédiatement les `param.x`, et type chaque couple (`liaison`, `champ`) contre la table des champs, que nous verrons en section 22.3. Le résultat est une liste de `Segment : Lit(String)` ou `Field(BindRef, Field)`. Un gabarit vide est refusé. Au rendu, les identifiants source sont mis entre backquotes, les mots de verdict restent nus, et une entité sans nom source reçoit une forme anonyme (`effect #2`, `state #0`) : *jamais* de chaîne vide (`src/rules/declarative/entity.rs`, l. 932–961).

Le nom affiché passe toujours par une table de nommage résolue, jamais par un label brut. Quand plusieurs variables désignent le même slot, on prend la plus petite, et non la première, pour que la sortie ne dépende pas de l'ordre d'un `HashMap` :

Extrait 22.13 – `src/rules/declarative/entity.rs`, l. 1069–1078 — le nom affiché d'un label

```

1069 /// The smallest non-temp source name bound to `label`. Smallest, not first:
1070 /// `HashMap` order is seed-dependent and the name is user-visible.
1071 fn pick_name(names: &HashMap<Var, HookLabel>, label: HookLabel) -> Option<String> {
1072     names
1073     .iter()
1074     .filter(|(var, l)| **l == label && !var.starts_with("__"))
1075     .map(|(var, _)| var)

```

```

1076     .min()
1077     .map(|var| crate::ir::source_name(var).to_string())
1078 }

```

22.3 Le système de sortes

22.3.1 Un pack qui compile en apparence

Un auteur veut signaler un hook personnalisé qui reçoit un argument recréé à chaque rendu. Il écrit, par analogie avec les deps :

Extrait 22.14 – /tmp/ch22/bad/bad-sort.json — une garde *stability* sur un argument

```

5  {
6    "id": "stable-arg",
7    "docs": { "description": "d", "why": "w", "fix": "f" },
8    "severity": "warning",
9    "anchor": { "relation": "hook_calls", "kind": "custom" },
10   "forEach": { "edge": "args", "as": "a" },
11   "guards": [{ "kind": "stability", "of": "a", "is": ["per-render"] }],
12   "message": "{a.returns}"
13 }

```

Le JSON est bien formé et chaque mot existe dans le schéma. Le chargement échoue pourtant :

```

[error] pack `./bad-sort.json` (./bad-sort.json): at `rules[0].guards[0].of`: guard `stability` applies to a deps entry,
    ↳ but the subject binds a call-site argument
exit=2

```

Le message dit ce qui était attendu (« a deps entry »), ce qui a été obtenu (« a call-site argument ») et où (`rules[0].guards[0].of`). Il est le produit d'un *système de types* : chaque entité liée a une *sorte*, et chaque arête, chaque garde et chaque champ est typé contre elle.

Définition 22.2: Sorte

La *sorte* d'une entité liée est son type statique dans le langage de packs. L'ancre fixe la sorte de *anchor* ; une arête *forEach* fixe la sorte de la liaison ; une garde admet une ou plusieurs sortes pour son sujet *of* ; un champ de gabarit est porté par certaines sortes seulement. Un pack est bien typé si chacune de ces contraintes est satisfaite ; il est rejeté au chargement sinon.

L'univers des sortes est une énumération de seize variantes :

Extrait 22.15 – src/rules/declarative/validate.rs, l. 59–98 — les seize sortes

```

59 /// The static type of a bound entity – the universe of the schema's type
60 /// system. Every edge and guard is typed against these.
61 #[derive(Debug, Clone, Copy, PartialEq, Eq)]
62 pub(crate) enum Sort {
63     /// A `hook_calls` row; `None` = any kind.
64     Hook(Option<HookKindFilter>),
65     /// An alias-resolved setter call in the render CFG.
66     SetterRender,
67     /// An alias-resolved setter call in the anchor's body CFG.
68     SetterBody,
69     /// One declared deps-array entry.
70     Dep,
71     /// One call-site argument of a custom-hook anchor.

```

```

72   Arg,
73   /// One `hook_provenance` row (ADR-027 §7): a resolved hook identity.
74   /// Kind-less and edge-less by design – the row survives inlining, so
75   /// there may be no `hook_calls` row (and no body, no deps) behind it.
76   HookOrigin,
77   /// One writer of a state-hook anchor's slot (ADR-027 §1).
78   Writer,
79   /// One proven context-provider element (#71).
80   Provider,
81   /// One prop of one resolved component element (#71 step 2).
82   JsxProp,
83   /// One render-loop cycle of the program churn graph, carried by an effect
84   /// of this component (#108).
85   ChurnCycle,
86   /// One prop seed of a state-hook anchor's slot (#106).
87   Seed,
88   /// One `useContext` call site with complete ancestry (#115).
89   ContextConsumer,
90   /// One callback registration in an effect body (#111).
91   Registration,
92   /// One non-hook call site in a body (#126).
93   Call,
94   /// One read site of a state-hook anchor's slot (#127).
95   Read,
96   /// One element the render body builds (#126).
97   Element,
98 }

```

La sorte Hook est paramétrée par le filtre de genre de l'ancre : le genre est une information de *typage*. La garde `cleanup`, par exemple, n'est admise que sur `Hook(Some(Effect))` ; une ancre `hook_calls` sans `kind` lie `Hook(None)`, qui ne l'admet pas.

22.3.2 Ancres et arêtes

Les dix ancres et la sorte qu'elles lient sont résumées au tableau 22.1.

relation	sorte liée	une ligne représente...
<code>hook_calls</code> (<code>kind?</code>)	<code>Hook(kind)</code>	un appel de hook, joint à son <code>HookEntry</code> et à son <code>EffectInfo</code> par label; <code>kind</code> parmi <code>state</code> , <code>effect</code> , <code>memo</code> , <code>callback</code> , <code>ref</code> , <code>custom</code> , <code>handler</code>
<code>render_setter_calls</code>	<code>SetterRender</code>	un appel de setter du corps de rendu, résolu à travers les alias
<code>render_calls</code>	<code>Call</code>	un appel non-hook du corps de rendu (garde <code>name</code> obligatoire)
<code>hook_origins</code>	<code>HookOrigin</code>	une identité de hook résolue, qui <i>survit</i> à l' <i>inlining</i>
<code>context_providers</code>	<code>Provider</code>	un <code><Ctx.Provider value=...></code> prouvé
<code>jsx_props</code> (<code>elements?</code>)	<code>JsxProp</code>	un prop d'un élément construit par le rendu
<code>elements</code> (<code>elements?</code>)	<code>Element</code>	un élément construit par le rendu (arête props)
<code>churn_cycles</code>	<code>ChurnCycle</code>	un cycle de rendu du graphe de churn programme, porté par un effet de ce composant
<code>context_consumers</code>	<code>ContextConsumer</code>	un <code>useContext</code> dont toute l'ascendance a été vue
<code>registrations</code> (<code>firing?</code>)	<code>Registration</code>	un enregistrement de callback dans un corps d'effet

TABLE 22.1 – Les dix ancres (`src/rules/declarative/schema.rs`, l. 106–211). Le filtre `elements` vaut `component` par défaut ; `handler` désigne un gestionnaire d'événement DOM passé en JSX, que le lowering modélise comme une entrée `HookEntry::Handler` (chapitre 7).

Depuis l'ancre, la navigation `forEach` suit au plus une arête, sous un nom de liaison :

Extrait 22.16 – `src/rules/declarative/schema.rs`, l. 238–247 — `ForEach`, une arête et une liaison

```
238 /// Typed navigation from the anchor (ADR-022 §2): at most one edge, one
239 /// binding, no joins.
240 #[derive(Debug, Deserialize)]
241 #[cfg_attr(feature = "schema-gen", derive(JsonSchema))]
242 #[serde(deny_unknown_fields)]
243 pub struct ForEach {
244     pub edge: EdgeName,
245     #[serde(rename = "as")]
246     pub bind: String,
247 }
```

Les huit arêtes ne partent que de certaines sortes, et chacune produit une sorte d'élément. La figure 22.2 en donne le diagramme : c'est le « schéma entité-relation » du langage. Une seule fonction, `edge_element_sort`, implémente ce typage ; son commentaire en fait un invariant : « *the one typing table the `forEach` navigation and the `none` quantifier both read, so an edge cannot be navigable in one and not the other* » (`src/rules/declarative/validate.rs`, l. 138–141).

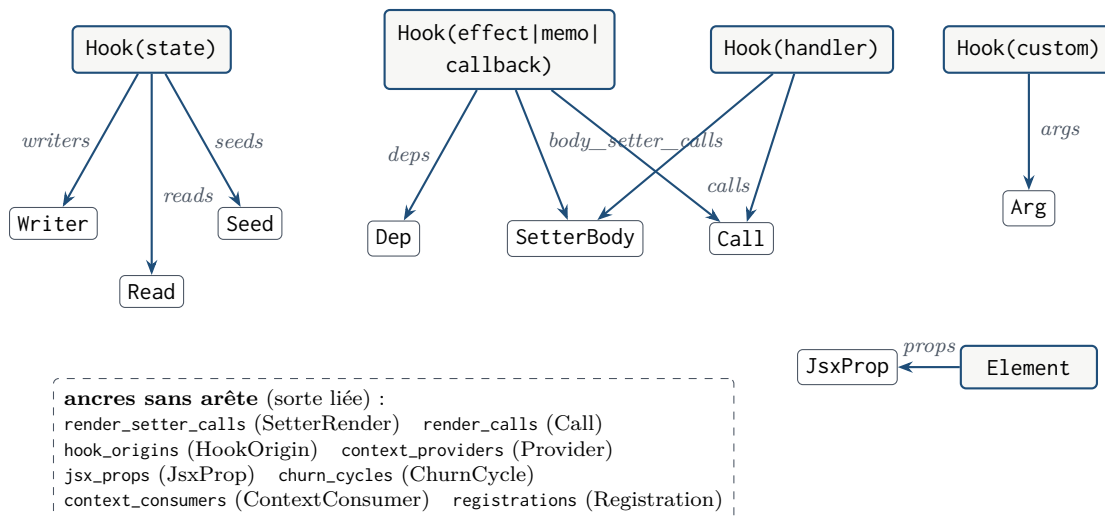


FIGURE 22.2 – Le diagramme entité-relation des sortes. En bleu, les sortes que lie l'ancre `hook_calls` (selon `kind`) ou `elements` ; en blanc, les sortes d'éléments ; chaque flèche est une arête navigable par `forEach` ou quantifiable par `none` (`every` ne quantifie que `deps`). Les sortes `Hook(ref)` et `Hook(None)` n'ont aucune arête. Une même sorte d'élément peut être atteinte par deux chemins : `Call` est aussi la sorte de l'ancre `render_calls`, `JsxProp` celle de `jsx_props`.

Deux remarques sur ces arêtes. Les relations `calls`, `reads`, `body_setter_calls` viennent de marches du corps bornées en profondeur (budget 2, `src/rules/declarative/entity.rs`, l. 385–428) : elles peuvent *sous-énumérer*, ce qui comptera pour les quantificateurs (section 22.7). Et `writers` lit la relation `slot_writers` du moteur, une ligne par *site* d'écriture du slot (définition 6.2), filtrée aux lignes non étrangères (`owner.is_none() && slot == label`, `src/rules/declarative/entity.rs`, l. 362–370).

22.3.3 Gardes

Le `Guard` du schéma compte trente-deux variantes : vingt-sept gardes *filtrantes* et cinq gardes *certifiantes*, préfixées `must_`. Le commentaire de l'énumération fixe la convention :

Extrait 22.17 – `src/rules/declarative/schema.rs`, l. 305–308 — le préfixe `must_` rend la polarité visible

```
305 /// A guard: a predicate over an engine verdict. `must_*` guards certify
306 /// (attach the `Certified` proof on `All`); the others filter. The `must_`
307 /// prefix makes polarity visible in the JSON, and the $3 load-time warning is
308 /// a prefix scan.
```

Le tableau 22.2 les range par sorte de sujet. La colonne « polarité » dit ce que la garde affirme : un fait *exact* (sans approximation), un fait *may* (sur-approximé : la garde peut passer à tort, jamais échouer à tort), ou une *certification* *must*.

TABLE 22.2 – Les trente-deux gardes (`src/rules/declarative/schema.rs`, l. 305–727 ; typage dans `validate_guard`).

kind	sujet admis	valeurs, polarité
stability	Dep	is not parmi stable, versioned, per-render, unknown ; verdict lu en sortie de rendu
returns	Arg	ce que <i>retourne</i> un argument fonctionnel : stable, fresh-reference, unknown
identity	Provider, JsxProp, Arg, Registration	fresh-every-render (fait prouvé) ou unknown
cleanup	Hook(effect)	present, absent, unknown ; seul absent est une affirmation
in_deps	SetterBody	le slot écrit figure dans les deps de l'ancre ; negate facultatif
name, source, receiver, prop	toute sorte portant le champ	one_of prefix ; filtre textuel sur une entité résolue, <i>positive-only</i>
origin	Hook(_), HookOrigin	hook, direct : identité résolue, appel direct ou via wrapper
phase	Call, Read	is parmi huit phases dont unknown ; may, positive-only
provenance	Writer	through, direct : chaîne de wrappers exportés menant à l'écriture
writer_phases	ancre Hook(state)	includes : existentiel may sur <i>tous</i> les écrivains du slot ; $\top$ satisfait
updater, updater_body	Writer	functional unknown ; impure unknown
same_tick	Writer	sans valeur : fait may à sens unique
provider	ContextConsumer	provider-seen, none-on-analyzed-paths
teardown	Registration	paired, none-seen
registers	ancre Hook(effect)	firing : existentiel may sur les enregistrements
seed_sync	Seed	synced, none-seen
slot_ownership	SetterRender	local, foreign ; <i>élargit</i> l'énumération
cycle	ChurnCycle	cross_component, all_must : booléens exacts
count	ancre à deps	more_than less_than equals sur <code>anchor.deps</code>
deps_declared	ancre à deps	eq : un argument deps a-t-il été passé
any_of	—	disjonction d'au moins deux gardes
every	ancre à deps	$\forall$ may sur <code>anchor.deps</code> , sans preuve
none	ancre admettant l'arête	$\neg\exists$ sur <code>anchor.<edge></code> , sans preuve

kind	sujet admis	valeurs, polarité
<code>must_setter_on_all_paths</code>	<code>SetterBody</code>	certifie : le setter s'exécute sur tous les chemins du corps
<code>must_dominates_all_exits</code>	<code>SetterRender</code>	certifie : l'appel domine toutes les sorties du rendu
<code>must_init_calls_setter</code>	ancree <code>Hook(state ref)</code>	certifie : l'initialiseur appelle un setter
<code>must_hook_is_conditional</code>	ancree <code>Hook(_)</code>	certifie : l'appel de hook est conditionnel
<code>must_direct_write</code>	<code>Writer</code>	certifie : l'écriture est directe (aucun wrapper)

Chaque garde de verdict prend une liste de noms tirés d'une énumération qui *miroite totalement* un verdict du moteur : `T y` porte un nom, `unknown`, pour pouvoir être nommé mais jamais oublié. Les conversions moteur $\rightarrow$ schéma sont des `match` totaux dans `entity.rs` (« *a new phase is a compile error here* », `src/rules/declarative/entity.rs`, l. 1080–1092). Certaines sont volontairement asymétriques : le fait moteur `Pairing` a trois valeurs, mais `Unpaired` et `Unknown` se replient tous deux sur `none-seen` — « *an absence of evidence, and the vocabulary says so* » (`src/rules/declarative/entity.rs`, l. 1149–1160).

Définition 22.3: Garde positive-only

Une garde est *positive-only* si elle n'a pas de forme négative et échoue quand le champ qu'elle teste est absent. Les gardes textuelles, `phase`, `writer_phases`, `updater`, `same_tick`, `provider`, `teardown`, `registers`, `seed_sync` le sont. Seules les gardes dont le verdict est un miroir total incluant `unknown` admettent `not` (`stability`, `returns`, `identity`, `cleanup`).

La raison est un argument de soundness d'ADR-23 : une négation sur un verdict `may` inverserait le sens de l'approximation. « Pas dans le rendu » n'est pas le complément de « peut-être dans le rendu » ; pour l'écrire, on énumère les phases acceptées, `unknown` compris si l'on veut rester du côté sûr. La ligne `(None, ..) => false` de `text_matches` (`src/rules/declarative/exec.rs`, l. 907–918) est la forme exécutable de la règle « absent $\Rightarrow$ échec ».

22.3.4 Champs

Un champ est ce que rend `{liaison.champ}` et ce que filtre une garde textuelle. Une seule énumération sert les deux :

Extrait 22.18 – `src/rules/declarative/validate.rs`, l. 448–477 — la table unique des champs

```

448 /// A field of a bound entity – what `{binding.field}` renders and what a
449 /// text guard matches.
450 ///
451 /// This enum is the single table both projections read: [`Field::token`] names
452 /// it in the schema, [`Field::admits`] says which sorts carry it, and
453 /// `EntityCtx::field_raw` computes it. All three are total matches on `Field`,
454 /// so a new field cannot be half-added – the previous split between `field_for`
455 /// and `render_field`, each ending in a catch-all, let a field validate and
456 /// then render as the empty string.
457 #[derive(Debug, Clone, Copy, PartialEq, Eq)]
458 pub(crate) enum Field {
459     Kind,
460     Name,
461     Source,
462     Slot,

```

```

463     Setter,
464     Path,
465     Stability,
466     Returns,
467     Region,
468     Phase,
469     Via,
470     Identity,
471     Cleanup,
472     Prop,
473     Cycle,
474     Owner,
475     Firing,
476     Receiver,
477 }

```

Trois projections sont des match totaux sur `Field : Field::token` (le mot du gabarit), `Field::admits` (quelles sortes portent le champ) et `EntityCtx::field_raw` (sa valeur). Un nouveau champ ne peut donc pas être ajouté à moitié. Le commentaire rappelle la dette que cette table a remboursée : deux fonctions distinctes, chacune terminée par un cas « attrape-tout », laissaient un champ valider puis se rendre comme la chaîne vide. La garde textuelle est typée par la même table (« *a field can never be renderable but unguardable (or the reverse)* », `src/rules/declarative/validate.rs`, l. 1632–1634). Le tableau 22.3 en donne la matrice.

champ	sortes qui le portent
kind	Hook(_), JsxProp, Element
name	Hook(None), Hook(k) pour $k \in \{\text{state, memo, callback, ref, custom}\}$, HookOrigin, Provider, JsxProp, ContextConsumer, Registration, Call, Read, Element
receiver	Call
source	Hook(custom), HookOrigin
setter	SetterRender, SetterBody, Writer
slot	SetterRender, SetterBody, Writer, Read
path	Dep, Seed
stability	Dep
returns	Arg
phase	Writer, Call, Read
via	Writer
region	Writer, Read
identity	Provider, JsxProp, Arg, Registration
prop	JsxProp
cleanup	Hook(effect)
firing	Registration
owner	SetterRender
cycle	ChurnCycle

TABLE 22.3 – La matrice champ $\times$ sorte, lue dans `Field::admits` (`src/rules/declarative/validate.rs`, l. 530–911). Ni un effet ni un handler ne portent `name` : rien ne les nomme.

22.3.5 Le refus du point de programme

Revenons au pack rejeté du début. La matrice dit que `stability` est portée par `Dep` et par rien d'autre ; le commentaire de `Field::Path` en donne la raison :

Extrait 22.19 – `src/rules/declarative/validate.rs`, l. 669–671 — là où le refus est appliqué

```
669 // `stability` stays a deps-entry fact: reading it for a call-site
670 // argument is the program-point error ADR-023 §2 refuses – this
671 // table is where the refusal is enforced.
```

Décision : ADR-23 §2 — un verdict se lit au point de programme de son entité

Décision. Un verdict n'est lisible que sur l'entité pour laquelle il a été calculé, au point de programme où il l'a été. La stabilité d'un dep est un verdict *de sortie de rendu* (`RuleCtx::stability_verdict` sur l'environnement de sortie); un argument d'appel est évalué *au point d'appel*. Dans `let x = {}; useThing(x); x = props.stable;`, `x` vaut `Stable` en sortie et `PerRender` à l'appel. Lire la stabilité de sortie pour un argument serait un faux négatif silencieux. L'ADR le dit : « *This is the single easiest way to introduce a silent false negative while believing the change is free, and it is why "just point the existing guard at a new edge" is refused.* »

Mise en œuvre. Sur un argument, on utilise `returns` (ce que retourne un argument fonctionnel, calculé *pendant* le point fixe, ADR-23 §3 amendé) ou `identity`, lue dans l'environnement convergé *du bloc de l'appel* (`src/rules/declarative/entity.rs`, l. 244–271).

Alternative refusée. « Pointer la garde existante vers une nouvelle arête », qui semble gratuit et ne l'est pas.

22.4 La validation en trois étages

22.4.1 Le point d'entrée

Toute la surface publique du sous-système est une fonction, `load_pack` : du JSON en entrée, des règles exécutables et leur documentation en sortie.

Extrait 22.20 – `src/rules/declarative/mod.rs`, l. 47–87 — `load_pack`, trois étages puis la cuisson

```
47 pub fn load_pack(
48     json: &str,
49     options_by_full_id: &BTreeMap<String, serde_json::Map<String, serde_json::Value>>,
50 ) -> Result<PackLoad, PackError> {
51     let raw: serde_json::Value = serde_json::from_str(json).map_err(|e| PackError {
52         path: String::new(),
53         message: format!("not valid JSON: {e}"),
54     });
55     // Typed deserialization with exact paths (`rules[1].guards[0].kind`).
56     let pack: schema::PackFile = serde_path_to_error::deserialize(&raw).map_err(|e| PackError {
57         path: e.path().to_string(),
58         message: e.inner().to_string(),
59     });
60
61     let (resolved, warnings) = validate::validate_pack(&raw, &pack, options_by_full_id)?;
62
63     let rules = resolved
64         .into_iter()
65         .zip(&pack.rules)
66         .map(|(def, src)| {
67             let doc = RuleDoc::new(
68                 def.id.clone(),
69                 src.docs.description.clone(),
70                 src.docs.why.clone(),
71                 src.docs.example.clone().unwrap_or_default(),
72                 src.docs.fix.clone(),
73             );
```

```

74         LoadedRule {
75             id: def.id.clone(),
76             rule: Box::new(exec::TierARule { def }) as Box<dyn Rule>,
77             doc,
78         }
79     })
80     .collect();
81
82     Ok(PackLoad {
83         pack_name: pack.name,
84         rules,
85         warnings,
86     })
87 }

```

Les trois étages correspondent aux trois flèches du haut de la figure 22.1.

1. **Syntaxe JSON.** Le texte devient un `serde_json::Value` brut. Un échec donne une erreur sans chemin, « not valid JSON: ... ».
2. **Désérialisation typée.** Le brut devient un `PackFile` par `serde_path_to_error`, qui fournit le chemin exact de la faute. Sont rejetés ici un champ inconnu dans une structure `deny_unknown_fields`, une variante inconnue, un champ requis manquant, un mauvais type, une référence `$param` mal formée.
3. **Validation sémantique.** `validate_pack` reçoit à la fois le brut et le typé : le brut sert à vérifier les clefs inconnues à l'intérieur des ancres et des gardes, là où `serde` ne le peut pas. Il produit les `ResolvedRule` et une liste d'avertissements non bloquants.

Vient enfin la *cuisson* : chaque `ResolvedRule` est appariée à sa `RuleDef` source pour construire la `RuleDoc`, puis emballée dans un `TierARule`. Les options du consommateur ont été substituées pendant la validation : le `RuleConfig` que le registre passe au `RuleCtx` n'est jamais lu par une règle de pack (le registre ne valide d'ailleurs les options que des règles natives, `if !name.contains('/')`, `src/rules/registry.rs`, l. 162–200).

Une faute de frappe dans le nom d'une garde s'arrête au deuxième étage, avec la liste des noms possibles (`/tmp/ch22/rej/p_typo.json`, `"kind": "stabilty"`) :

```

[error] pack `./p_typo.json` (./p_typo.json): at `rules[0].guards[0].kind`: unknown variant `stabilty`, expected one of
  ↳ `stability`, `returns`, `origin`, `in_deps`, `name`, `receiver`, `phase`, `prop`, `source`, `identity`, `
  ↳ cleanup`, `provenance`, `writer_phases`, `updater`, `updater_body`, `same_tick`, `provider`, `teardown`, `
  ↳ registers`, `seed_sync`, `slot_ownership`, `cycle`, `every`, `none`, `count`, `deps_declared`, `
  ↳ must_setter_on_all_paths`, `must_dominates_all_exits`, `must_init_calls_setter`, `must_hook_is_conditional`, `
  ↳ must_direct_write`, `any_of`
exit=2

```

22.4.2 L'ordre des vérifications

`validate_pack` (`src/rules/declarative/validate.rs`, l. 2489–2544) contrôle d'abord l'identité du pack — `schemaVersion` égal à 1, nom non vide, sans /, sans collision avec un nom de diagnostic natif — puis l'unicité des identifiants, et délègue chaque règle à `validate_rule` (`src/rules/declarative/validate.rs`, l. 1195–1435), qui procède dans cet ordre :

1. identifiant non vide et sans / ; `description`, `why`, `fix` non vides après `trim` ;
2. clefs de l'ancre (`check_keys`) : c'est ce qui rejette `{"relation": "hook_origins", "kind": "custom"}`, l'ancre des origines étant sans genre ;
3. ancre → (`ResolvedAnchor`, `Sort`) ;
4. `forEach` : nom de liaison interdit s'il vaut `anchor`, `param`, la chaîne vide ou contient un point, puis typage de l'arête par `edge_element_sort` ;

5. environnement de paramètres (`ParamEnv::build`) : types des défauts, options inconnues, types des options;
6. gardes, dans l'ordre, par la fonction récursive `validate_guard`;
7. contrôles transverses sur l'arbre de gardes résolu;
8. gabarit du message;
9. avertissements.

La validation s'arrête à la *première* faute (chaque étape est suivie d'un ?) : un pack porte au plus une `PackError`. Chaque erreur est un couple { `path`, `message` }, affiché « at 'chemin': message », et chaque message dit ce qui était attendu et ce qui a été obtenu. Ce format est pensé pour une boucle d'écriture itérative, humaine ou outillée : le dépôt livre une *skill* pour agent (`skills/reactant-rules/SKILL.md`) qui repose sur ces messages pour converger.

22.4.3 Typer une garde

Toutes les branches de `validate_guard` suivent le même schéma : contrôle des clefs admises (ce qui rend `negate` sur `stability` aussi bruyant qu'un `deny_unknown_fields`), résolution du sujet `of` en (`BindRef`, `Sort`), test de sorte, arité des champs, résolution des `PVal` au type de la position, listes non vides. L'exemple canonique est `stability` :

Extrait 22.21 – `src/rules/declarative/validate.rs`, l. 1514–1539 — le typage de la garde `stability`

```

1514     Guard::Stability { of, is, not } => {
1515         let (of, sort) = cx.resolve_of(of, g_path)?;
1516         if sort != Sort::Dep {
1517             return Err(PackError::new(
1518                 format!("{g_path}.of"),
1519                 format!(
1520                     "guard `stability` applies to a deps entry, but the subject binds {}",
1521                     sort.describe()
1522                 ),
1523             ));
1524         }
1525         let (names, negated) = match (is, not) {
1526             (Some(pv), None) => (cx.env.resolve(pv, None, &format!("{g_path}.is"))?, false),
1527             (None, Some(pv)) => (cx.env.resolve(pv, None, &format!("{g_path}.not"))?, true),
1528             _ => {
1529                 return Err(PackError::new(
1530                     g_path,
1531                     "guard `stability` takes exactly one of `is` / `not`",
1532                 ));
1533             }
1534         };
1535         if names.is_empty() {
1536             return Err(PackError::new(g_path, "the verdict list must not be empty"));
1537         }
1538         ResolvedGuard::Stability { of, names, negated }
1539     }

```

Le sujet se résout par `GuardCx::resolve_of` (`src/rules/declarative/validate.rs`, l. 1481–1498) : "anchor" désigne l'ancree, le nom de la liaison `forEach` désigne l'élément lié, tout autre nom est une erreur qui liste les liaisons disponibles. La référence résolue, `BindRef`, n'a que deux valeurs, `Anchor` et `Bound` : il n'y a jamais plus de deux entités visibles.

22.4.4 Les contrôles transverses

Certaines contraintes ne portent pas sur une garde mais sur l'arbre entier :

Extrait 22.22 – `src/rules/declarative/validate.rs`, l. 1321–1366 — trois contrôles sur l'arbre de gardes résolu

```

1321 // A may-typed quantifier can select a row on the strength of  $\top$  elements
1322 // alone, so a rule that uses one must not also carry Error authority: with
1323 // no `must_*` anywhere, the finding stratifies to Warning structurally
1324 // rather than by policy (ADR-023 §4's amendment).
1325 if has_must && guards.iter().any(quantifies) {
1326     return Err(PackError::new(
1327         format!("{path}.guards"),
1328         "a rule using `every` cannot also use a `must_*` guard: the quantifier is \
1329         may-typed, so a row it selected cannot carry a certified claim",
1330     ));
1331 }
1332
1333 // #126: the `calls` relation is the first unbounded one – it enumerates
1334 // EVERY call, so a rule with no `name` guard on the call row fires on all
1335 // of them. The guard is required as a top-level conjunct: inside `any_of`
1336 // one branch could leave the row unnamed, which is the same nuisance one
1337 // disjunct at a time.
1338 let call_subject = if bound_sort == Some(Sort::Call) {
1339     Some(BindRef::Bound)
1340 } else if anchor_sort == Sort::Call {
1341     Some(BindRef::Anchor)
1342 } else {
1343     None
1344 };
1345 if let Some(subject) = call_subject
1346     && !guards.iter().any(
1347         |g| matches!(g, ResolvedGuard::Text { of, field: Field::Name, .. } if *of == subject),
1348     )
1349 {
1350     return Err(PackError::new(
1351         format!("{path}.guards"),
1352         "a rule over `calls` needs a `name` guard on the call row: the relation \
1353         enumerates every call in the body, so without one the rule fires on all of \
1354         them. Put it at the top level, not inside `any_of`",
1355     ));
1356 }
1357
1358 // #107: the render-setter enumeration widens ONLY for a rule that names
1359 // ownership. Anywhere in the tree, `any_of` included – a foreign row a
1360 // disjunct can select must exist for that disjunct to see it.
1361 let anchor = match anchor {
1362     ResolvedAnchor::RenderSetterCalls { .. } => ResolvedAnchor::RenderSetterCalls {
1363         foreign: guards.iter().any(names_ownership),
1364     },
1365     other => other,
1366 };

```

Le premier interdit de combiner un quantificateur `may` et une garde certifiante (nous y reviendrons en section 22.7); la fonction `quantifies` reconnaît `every` et `none`, même si le message ne parle que d'`every` :

```

[error] pack `./p_every_must.json` (./p_every_must.json): at `rules[0].guards`: a rule using `every` cannot also use a `
  ↳ must_*` guard: the quantifier is may-typed, so a row it selected cannot carry a certified claim
exit=2

```

Le second exige une garde `name` au niveau supérieur sur toute ligne de la relation `calls` ou `render_calls`, la première relation *non bornée* du langage : sans elle, la règle tirerait sur chaque appel du corps.

```
[error] pack `./p_calls.json` (./p_calls.json): at `rules[0].guards`: a rule over `calls` needs a `name` guard on the
  ↳ call row: the relation enumerates every call in the body, so without one the rule fires on all of them. Put it
  ↳ at the top level, not inside `any_of`
exit=2
```

Le troisième n'est pas un rejet : il pose le drapeau `foreign` de l'ancre `render_setter_calls` si et seulement si la règle nomme `slot_ownership` quelque part dans son arbre. Nous verrons en section 22.8 pourquoi l'énumération s'élargit *sur la garde*.

22.4.5 Le catalogue des erreurs et des avertissements

Le tableau 22.4 regroupe les classes d'erreurs par étage. La numérotation E1–E25 est celle du dossier de notes, pas celle du code, qui ne numérote rien ; les messages sont cités verbatim ou sous forme de gabarit.

TABLE 22.4 – Les classes d'erreurs du chargement d'un pack (toutes fatales, code de sortie 2).

#	étage	message (verbatim ou gabarit)
E1	JSON	not valid JSON: {e}
E2	serde	unknown field 'bogus', expected ...; unknown variant ...; missing field ...
E3	serde / PVal	a {"\$param": ...} reference takes no other key; expected a value or {"\$param": "<name>": ...
E4	pack	unsupported schemaVersion {n}, only 1 exists
E5	pack	pack name is empty; ... must not contain '/'; ... collides with a built-in diagnostic name
E6	pack	duplicate rule id 'x' in this pack
E7	règle	rule id is empty; rule id 'a/b' contains '/'. The pack name is the namespace, ids are bare
E8	docs	docs are mandatory: 'why' is empty
E9	clefs	{what} does not accept field 'k'. Allowed: ...
E10	forEach	'x' is not a usable binding name
E11	arête	edge 'deps' needs an effect/memo/callback anchor, but the anchor binds ... (et sept variantes)
E12	params	default {v} does not match declared type 't'
E13	options	unknown option 'k'. Declared params: ...; option value {v} does not match declared type 't'
E14	\$param	this position does not accept a {"\$param": ...} reference; reference to undeclared param 'p'; param 'p' has type 't', but this position expects 'u'; param 'p' value is unusable here: ...
E15	liaison	unknown binding 'x'. Available: anchor[, b]
E16	sorte	guard 'k' applies to ..., but the subject binds ...; pour une garde textuelle, la liste des champs que la sorte porte
E17	ancre requise	guard 'in_deps' needs an anchor with a deps array ...; idem count, deps_declared, must_init_calls_setter, must_hook_is_conditional, writer_phases
E18	arité	... takes exactly one of 'is' / 'not'; ... one_of / prefix; ... more_than / less_than / equals; ... at least one of 'hook' / 'direct'

#	étage	message
E19	liste vide	the verdict list must not be empty et ses variantes
E20	sujet d'arête	guard 'count' counts 'anchor.deps' (got 'x'); idem every; guard 'none' quantifies over an edge of the anchor, spelled 'anchor.<edge>'
E21	composition	guard 'any_of' needs at least two alternatives; guard 'every' needs at least one guard to quantify; idem none
E22	polarité	a 'must_*' guard cannot appear inside 'every' (idem none)
E23	polarité	a rule using 'every' cannot also use a 'must_*' guard ...
E24	calls	a rule over 'calls' needs a 'name' guard on the call row ...
E25	gabarit	unclosed '{'; template placeholder ... must be 'binding.field' or 'param.name'; unknown binding 'x' in template; ... has no field 'f'; message template is empty

Quatre avertissements (LoadWarning) n'empêchent pas le chargement ; ils sont imprimés sur la sortie d'erreur, préfixés « [warn] rule 'pack/rule': ».

- W1** Un pin error sans aucune garde `must_*` : « severity is pinned "error" but no `must_*` guard is used, so findings can only emit as warnings ». C'est *le seul* contrôle statique de sévérité (ADR-22 §3, `src/rules/declarative/validate.rs`, l. 1384–1393).
- W2** Un paramètre déclaré jamais référencé.
- W3** Une ancre `kind: "custom"` sans arête, dont toutes les gardes portent sur l'identité de l'ancre : elle ne voit que les hooks que le moteur n'a *pas* su résoudre (#6) ; la règle devrait aller sur `hook_origins`.
- W4** Une branche `must_*` laissée en "else": "keep" dans un `any_of` : elle passe toujours, la disjonction est toujours vraie.

Exemple 22.1: Un pin inatteignable

La règle `demo/no-deps` (`/tmp/ch22/rej/p_error_nomust.json`) reprend `effect-without-deps-array` en l'épinglant `error`. Elle se charge, avertit, et ne produit que des `WARNING` :

```
[warn] rule `demo/no-deps`: severity is pinned "error" but no must_* guard is used, so findings can only emit as
↳ warnings
Title (1 hooks) ../ex1_no_deps.tsx
warn demo/no-deps [hook:0] (line 4:2) no deps array
```

Décision : ADR-22 §3 et §6 — une validation dynamique, refaite par le cœur

Décision. Aucun rejet statique ne compare le pin aux gardes : la sévérité est calculée par finding (section 22.6), et la seule vérification statique est un avertissement (W1). Et le cœur Rust *revalide* chaque pack qu'il reçoit, quel que soit l'hôte (« *from any host — the core re-validates every pack it receives* », liste 22.1) : l'hôte npm est un transport, jamais une frontière de confiance.

Alternatives refusées. Un rejet statique des conflits pin/polarité, qui aurait interdit la *stratification* (une même règle `ERROR` là où c'est prouvé, `WARNING` ailleurs) ; une validation confiée à l'hôte JS.

Piège

Le nom as d'un quantificateur `every` ou `none` n'est pas contrôlé comme celui d'un `forEach`. Comme `resolve_of` teste "anchor" en premier, un "as": "anchor" rend l'élément quantifié inaccessible : "of": "anchor" désigne toujours l'ancre, et une garde `stability y` est rejetée

(E16). Le défaut est bruyant, donc sans danger.

Deux tests gardent la documentation alignée sur le schéma : `tests/schemas.rs` exige que le schéma sur disque égale la sortie de `reactant schemas`, et `tests/docs_drift.rs` que chaque ancre, arête et garde apparaisse dans `docs/custom-rules.md` et dans la référence de la skill.

22.5 L'exécution d'une règle

22.5.1 Ce que voit l'exécuteur

À la sortie du chargement, l'exécuteur ne connaît plus ni JSON ni paramètre. Il reçoit une représentation entièrement typée :

Extrait 22.23 – `src/rules/declarative/validate.rs`, l. 953–964 — l'IR résolue d'une règle

```

953 /// A fully-typed, param-baked rule – the executor never sees `PVal` or raw
954 /// JSON again.
955 #[derive(Debug, Clone, PartialEq)]
956 pub(crate) struct ResolvedRule {
957     /// Full id, `pack/rule`.
958     pub id: String,
959     pub pin: SeverityPin,
960     pub anchor: ResolvedAnchor,
961     pub for_each: Option<EdgeName>,
962     pub guards: Vec<ResolvedGuard>,
963     pub message: Vec<Segment>,
964 }
```

Le nom de la liaison `forEach` a disparu, seule l'arête survit : les références aux entités ont été résolues en `BindRef`. Les gardes forment un arbre `ResolvedGuard` de vingt-cinq variantes, plus compact que le schéma : les quatre gardes textuelles (`name`, `source`, `receiver`, `prop`) y fusionnent en une variante `Text { of, field, one_of, prefix }` (« *the schema keeps a distinct kind per field so the JSON says what it matches, the executor runs one arm* ») et les cinq `must_*` en `Must { kind, of, els }`. Le gabarit est une liste de `Segment` où les paramètres sont déjà des littéraux.

La règle elle-même est un emballage minimal, accompagné de l'énumération des preuves qu'une garde certifiante peut détenir :

Extrait 22.24 – `src/rules/declarative/exec.rs`, l. 37–50 — `TierARule` et `Proof`

```

37 pub(crate) struct TierARule {
38     pub def: ResolvedRule,
39 }
40
41 /// A held certification for the finding under evaluation. The enum exists
42 /// because different must-guards certify different evidence types; emission
43 /// matches to reach the generic `Diagnostic::error`.
44 enum Proof {
45     Setter(Certified<SetterCall>),
46     Dominates(Certified<DominatesAllExits>),
47     Init(Certified<InitSetterCall>),
48     Conditional(Certified<ConditionalHookCall>),
49     Direct(Certified<DirectWrite>),
50 }
```

`TierARule` implémente `Rule : name()` rend l'identifiant complet `pack/rule`, `check` fait le travail. Elle ne redéfinit pas `safe_check`, dont le défaut du trait s'applique (« *Custom rules have*

no safe_check in v1 », `src/rules/declarative/exec.rs`, l. 1–11). Surtout, le module vit sous `src/rules/` comme toute règle native : il ne peut pas frapper de `Certified` (constructeur privé à `api::query`) et ne peut construire une `ERROR` que par `Diagnostic::error`, qui exige un tel jeton (chapitre 17).

22.5.2 L'adaptateur vers les relations

Pour chaque couple (règle, composant), `check` construit un `EntityCtx`, l'adaptateur entre les entités du schéma et les primitives du moteur :

Extrait 22.25 – `src/rules/declarative/entity.rs`, l. 123–153 — `EntityCtx`

```
123 pub(crate) struct EntityCtx<'a> {
124     pub ctx: &'a RuleCtx<'a>,
125     pub comp: &'a AnalysisResult<StateValue>,
126     /// Canonical alias-resolved setter → slot relation (`all_setter_labels`).
127     pub setter_labels: HashMap<Var, HookLabel>,
128     pub setter_vars: HashSet<Var>,
129     /// State-value bindings in the render CFG (alias-resolved) – the naming
130     /// table for slots.
131     pub state_names: HashMap<Var, HookLabel>,
132     /// `var → label` for every bound hook result, whatever the kind – the
133     /// naming table for `{anchor.name}`. Direct bindings only: unlike a slot
134     /// name, a hook's name is the variable the call itself binds, not an alias
135     /// of it.
136     pub hook_names: HashMap<Var, HookLabel>,
137     exit_dom: OnceCell<ExitDominance>,
138     conditional: OnceCell<HashMap<HookLabel, Certified<ConditionalHookCall>>>,
139     /// Index into `comp.hook_provenance` by label (indices, not references:
140     /// `OnceCell` is invariant, so a borrowed map would freeze `a`).
141     provenance: OnceCell<HashMap<HookLabel, usize>>,
142     /// `ComponentSetter`-valued props (#107), resolved on first use by the
143     /// ownership-aware enumeration.
144     cross_setters: OnceCell<HashMap<Var, crate::engine::setters::SetterProp>>,
145     /// The slot → readers relation (#127), computed on first use: unlike the
146     /// writers, it is not needed by any native rule, so a component no pack
147     /// asks about never walks for it.
148     slot_reads: OnceCell<Vec<crate::engine::SlotRead>>,
149     /// Body calls per hook (#126), all bodies at once on first use. A rule
150     /// that navigates `calls` and then asks `none of anchor.calls` would
151     /// otherwise walk the body once per row – quadratic in the body's size.
152     body_calls: OnceCell<HashMap<HookLabel, Vec<crate::engine::BodyCall>>>,
153 }
```

Les quatre premières tables sont calculées d'emblée par le constructeur : la relation `setter → slot` résolue à travers les alias (`all_setter_labels`), l'ensemble des variables `setters`, la table de nommage des slots (valeurs d'état et leurs alias) et celle des hooks (liaisons directes). Les autres relations sont des `OnceCell` *paresseuses*, partagées entre les lignes et calculées au premier usage : dominance des sorties du rendu, carte des hooks conditionnels certifiés, index de provenance, `setters` étrangers, lectures de slot, appels de corps. Le commentaire de `slot_reads` explique l'intérêt : aucune règle native ne lit cette relation, si bien qu'un composant sur lequel aucun pack ne la demande ne la calcule jamais. Celui de `body_calls` en donne un autre : sans cache, une règle qui navigue `calls` puis demande `none of anchor.calls` re-marcherait le corps pour chaque ligne. Les relations globales au programme (graphe de churn, consommateurs de contexte) viennent du `ProgramCache` du chapitre 17, construites une fois par programme.

Chaque ancre et chaque arête a sa méthode d'énumération sur `EntityCtx` : `hook_rows(kind)`, `render_setters(foreign)`, `origin_rows()`... pour les ancres ; `deps(row)`, `body_setters(row)`,

writers(row), reads(row)... pour les arêtes. Aucune ne marche la syntaxe : elles lisent les relations que le moteur a produites (ADR-42) ou appellent les mêmes lecteurs que les règles natives (collect_setter_calls, cleanup_verdict, ProgramCache::churn).

22.5.3 Énumérer les candidats

check est un aiguillage sur l'ancree. Pour hook_calls, la boucle externe parcourt les lignes, la boucle interne l'arête du forEach :

Extrait 22.26 – src/rules/declarative/exec.rs, l. 227–245 — le début de TierARule::check

```

227 fn check(&self, ctx: &RuleCtx) -> Vec<Diagnostic> {
228     let e = EntityCtx::new(ctx);
229     let mut out = Vec::new();
230     match self.def.anchor {
231         ResolvedAnchor::HookCalls(kind) => {
232             for row in e.hook_rows(kind) {
233                 match self.def.for_each {
234                     Some(EdgeName::Deps) => {
235                         for dep in e.deps(&row) {
236                             self.eval(
237                                 &e,
238                                 &Candidate::Hook {
239                                     row: &row,
240                                     bound: Some(Bound::Dep(&dep)),
241                                 },
242                                 &mut out,
243                             );
244                         }
245                     }
246                 }
247             }
248         }
249     }
250 }

```

Chaque combinaison (ligne d'ancree, élément navigué éventuel) devient un Candidate, évalué par eval. Candidate a une variante par ancree, et Hook { row, bound } porte l'élément lié sous forme d'un Bound (Setter, Dep, Arg, Writer, Seed, Call, Read). Le commentaire du type rappelle une dette réparée : « *The two anchors used to have a guard match each, so a guard could be handled on one and silently fall into an unreachable! catch-all on the other* » (src/rules/declarative/exec.rs, l. 77–112). Un seul évaluateur récursif de gardes sert désormais toutes les ancrees.

22.5.4 Une conjonction court-circuitée

Extrait 22.27 – src/rules/declarative/exec.rs, l. 408–428 — évaluer un candidat

```

408 /// Evaluate every guard against one candidate; emit on a full pass. Guards
409 /// run in author order and short-circuit on the first failure.
410 fn eval(&self, e: &EntityCtx<'_>, cand: &Candidate<'_, '_>, out: &mut Vec<Diagnostic>) {
411     let mut proofs: Vec<Proof> = Vec::new();
412     for guard in &self.def.guards {
413         if !self.eval_guard(e, cand, guard, &mut proofs) {
414             return;
415         }
416     }
417
418     let message: String = self
419         .def
420         .message
421         .iter()
422         .map(|seg| match seg {
423             Segment::Lit(s) => s.clone(),

```

```

424         Segment::Field(r, f) => e.render_field(&cand.entity_at(*r), *f),
425     })
426     .collect();
427     out.push(self.emit(message, proofs, cand.label(), cand.range()));
428 }

```

La sémantique d'une règle tient dans ces vingt lignes. Les gardes sont évaluées *dans l'ordre de l'auteur* et la première qui échoue arrête l'évaluation du candidat ; chaque garde certifiante qui certifie pousse sa *Proof* ; si toutes passent, le message est rendu et *emit* construit le diagnostic, avec le label et la position du candidat. *eval_guard* (*src/rules/declarative/exec.rs*, l. 434–787) est un *match* exhaustif sur *ResolvedGuard* ; ses bras sont eux-mêmes exhaustifs sur *Bound* (« *a new edge would otherwise validate, load, and silently emit nothing* »), et les combinaisons que le validateur exclut aboutissent à des *unreachable!*.

Proposition 22.1: Sémantique d'une règle

Soit R une règle d'ancre A , d'arête facultative e et de gardes $g_1, \dots, g_n$. Pour un composant C , l'ensemble des findings de R est

$$\{ (a, b) \mid a \in A(C), b \in e(a) \text{ (ou } b = \emptyset \text{ sans arête)}, \bigwedge_{i=1}^n g_i(a, b) \}$$

où $A(C)$ est l'énumération de l'ancre et $e(a)$ celle de l'arête depuis la ligne a . L'ordre des g_i n'influe que sur le coût et sur les preuves collectées, jamais sur l'ensemble des findings : un g_i est une fonction pure du candidat.

22.5.5 Exemple : écrivains d'un même tick

La règle communautaire *community-state/same-tick-slot-collapse* utilise l'arête *writers* et deux gardes *may* :

Extrait 22.28 – *packs/community/state.json*, l. 14–21 — *same-tick-slot-collapse* (docs omises)

```

14     "severity": "warning",
15     "anchor": { "relation": "hook_calls", "kind": "state" },
16     "forEach": { "edge": "writers", "as": "w" },
17     "guards": [
18       { "kind": "same_tick", "of": "w" },
19       { "kind": "updater", "of": "w", "is": ["unknown"] }
20     ],
21     "message": "{w.setter} writes {w.slot} in `{w.region}` alongside another write of the same slot in
    ↳ the same tick, and its argument is not a proven functional updater – the writes collapse onto one
    ↳ snapshot"

```

Extrait 22.29 – Deux incréments qui n'en font qu'un (*/tmp/ch22/ex7_same_tick.tsx*)

```

1  import { useState } from "react";
2
3  export function Qty() {
4    const [qty, setQty] = useState(0);
5    const addPair = () => {
6      setQty(qty + 1);
7      setQty(qty + 1);
8    };
9    const addPairOk = () => {
10     setQty((q) => q + 1);
11     setQty((q) => q + 1);
12   };
13   return (

```

```

14     <div>
15       <button onClick={addPair}>{qty}</button>
16       <button onClick={addPairOk}>ok</button>
17     </div>
18   );
19 }

```

Qty (3 hooks) ex7_same_tick.tsx

```

warn community-state/same-tick-slot-collapse [hook:0] (line 6:4) `setQty` writes `qty` in `handler` alongside
  ↳ another write of the same slot in the same tick, and its argument is not a proven functional updater - the
  ↳ writes collapse onto one snapshot
warn community-state/same-tick-slot-collapse [hook:0] (line 7:4) `setQty` writes `qty` in `handler` alongside
  ↳ another write of the same slot in the same tick, and its argument is not a proven functional updater - the
  ↳ writes collapse onto one snapshot

```

Déroulons. L'ancre lie une ligne, le `useState` de `label 0`. L'arête `writers` donne quatre lignes `SlotWriter`, une par site (ADR-28) : lignes 6, 7, 10 et 11. Toutes sont `same_tick` : deux écritures synchrones du même slot, dans la même région, atteignables l'une depuis l'autre (chapitre 14). La garde `updater is ["unknown"]` échoue sur les deux updaters fonctionnels `q => q + 1` (`Updater::Functional`) et passe sur les deux `qty + 1`. Restent deux findings, placés sur le site de chaque écriture. Le gabarit rend `{w.setter}` et `{w.slot}` entre backquotes, et `{w.region}` nu, la règle ajoutant elle-même les backquotes. Le compte « (3 hooks) » inclut les deux handlers `onClick`, que le lowering modélise comme des entrées de hook.

Rappel React : Mises à jour groupées et updater fonctionnel

Dans un handler, React *groupe* les mises à jour d'état : les deux appels `setQty(qty + 1)` lisent le même instantané `qty` du rendu courant et écrivent la même valeur, si bien que le compteur n'avance que d'une unité. La forme fonctionnelle `setQty(q => q + 1)` reçoit la valeur en attente et compose. La règle ne dit pas « cet argument lit le slot » : les positions d'argument de `setter` ne portent pas de verdict d'expression (#67) ; `updater` ne sait dire que « pas un updater fonctionnel prouvé ».

22.5.6 Position, label, ordre

La position d'un finding est celle du certificat quand il sort en `ERROR` (la provenance « voyage » avec la preuve passée à `Diagnostic::error`), sinon celle que donne `Candidate::range` : le site de l'élément navigué s'il en a un, sinon la ligne de l'ancre (`src/rules/declarative/exec.rs`, l. 193–219). Le label de hook attaché est celui de l'ancre, ou celui de l'effet porteur pour un cycle ou un enregistrement ; un élément JSX n'en a pas, d'où l'absence de `[hook:N]` dans certaines sorties.

Piège

« L'ordre des packs est l'ordre de sortie » (ADR-22 §8) vaut pour l'*exécution* et pour *reactant rules*, pas pour l'affichage. Le registre trie les diagnostics d'un composant par (règle, sévérité, position, message, variable, label) (`src/rules/registry.rs`, l. 316–333) : le nom de règle domine, si bien qu'une règle `community-effects/...` s'affiche avant la native `missing-cleanup`. Le tri garantit le déterminisme, testé par `runs_are_deterministic`.

22.6 La sévérité : pin $\sqcap$ polarité

22.6.1 Une règle, deux niveaux

La règle `guardrails/self-retriggering-effect` est la seule du pack `guardrails` épinglée `error` :

Extrait 22.30 – packs/guardrails.json, l. 48–55 — self-retriggering-effect (docs omises)

```

48   "severity": "error",
49   "anchor": { "relation": "hook_calls", "kind": "effect" },
50   "forEach": { "edge": "body_setter_calls", "as": "setter" },
51   "guards": [
52     { "kind": "in_deps", "of": "setter" },
53     { "kind": "must_setter_on_all_paths", "of": "setter" }
54   ],
55   "message": "this effect writes {setter.slot}, which is listed in its own dependency array"

```

Appliquons-la à deux composants qui écrivent leur propre dep, l'un sans condition, l'autre sous une garde :

Extrait 22.31 – Deux effets qui s'auto-relancent (/tmp/ch22/ex2_self_retrigger.tsx)

```

1  import { useEffect, useState } from "react";
2
3  export function Unconditional() {
4    const [n, setN] = useState(0);
5    useEffect(() => {
6      setN(n + 1);
7    }, [n]);
8    return <div>{n}</div>;
9  }
10
11 export function Guarded() {
12   const [n, setN] = useState(0);
13   useEffect(() => {
14     if (n < 5) setN(n + 1);
15   }, [n]);
16   return <div>{n}</div>;
17 }

```

```

Guarded (2 hooks) ex2_self_retrigger.tsx
  warn  guardrails/self-retriggering-effect [hook:1] (line 14:15) this effect writes `n`, which is listed in its
        ↳ own dependency array
Unconditional (2 hooks) ex2_self_retrigger.tsx
  error  guardrails/self-retriggering-effect [hook:1] (line 6:4) this effect writes `n`, which is listed in its own
        ↳ dependency array

! 1 error(s), 1 warning(s) across 1 file(s).

```

(sortie filtrée par `-rule guardrails/self-retriggering-effect`; sans filtre, la native infinite-loop ajoute un WARNING sur Unconditional, chapitre 19).

La même règle émet une ERROR et un WARNING. Déroulons. Dans les deux composants, l'ancre est l'effet (label 1) et l'arête `body_setter_calls` lie un setter `setN` de slot 0. `in_deps` passe : le dep `n`, résolu par la table de nommage des slots, désigne le slot 0. Vient `must_setter_on_all_paths`, en `else: keep` par défaut. Dans `Unconditional`, la primitive rend `MustResult::All(c)` : le setter s'exécute sur tous les chemins du corps ; la garde pousse `Proof::Setter(c)` et passe. Dans `Guarded`, la primitive ne certifie pas ; en `keep`, la garde passe *sans preuve*. À l'émission, le pin `error` avec une preuve donne une ERROR, placée au site certifié `setN(...)` (6:4) ; le pin `error` sans preuve donne un WARNING, placé au site du setter lié (14:15).

Rappel React : Un effet qui écrit son dep

L'écriture change un élément du tableau de deps ; au commit suivant, React compare les deps par `Object.is`, constate le changement et relance l'effet, qui écrit à nouveau. Sans condition de sortie, le composant se re-rend indéfiniment ; avec `if (n < 5)`, la boucle

s'arrête d'elle-même. La différence entre les deux programmes est exactement ce que certifie `must_setter_on_all_paths`.

22.6.2 Garde certifiante, keep et drop

Le bras d'`eval_guard` qui traite les `must_*` tient en huit lignes :

Extrait 22.32 – `src/rules/declarative/exec.rs`, l. 688–695 — une garde certifiante

```

688     ResolvedGuard::Must { kind, els, .. } => match (self.certify(e, cand, *kind), els) {
689         (Some(p), _) => {
690             proofs.push(p);
691             true
692         }
693         (None, ElseBehavior::Keep) => true,
694         (None, ElseBehavior::Drop) => false,
695     },

```

`certify` appelle la primitive du moteur qui correspond au genre de la garde, sur le sujet du candidat (`src/rules/declarative/exec.rs`, l. 790–866) : `must_setter_on_all_paths` sur le CFG du corps de l'ancre et l'ensemble des alias du slot écrit ; la dominance des sorties du rendu pour `must_dominates_all_exits` ; `must_init_calls_setter` sur l'initialiseur d'un `useState` ou d'un `useRef` ; la carte des `Certified<ConditionalHookCall>` pour `must_hook_is_conditional` ; `must_direct_write` sur une ligne `writers`, qui certifie si et seulement si l'écriture est `Direct`. Ce sont les primitives du chapitre 17, pas des réimplémentations. Pour la première, le commentaire précise pourquoi l'exécuteur ne se contente pas du bloc du setter : « *the primitive's own must-forwarding handles multi-site/branchy writes, which the deduplicated SetterCall.block_id could not* ».

Il en résulte deux usages très différents d'une même garde :

- en "else": "keep" (le défaut), la garde *ne filtre rien* : elle passe toujours, et ne fait qu'élever le finding quand elle certifie. C'est la stratification de `self-retriggering-effect` ;
- en "else": "drop", elle se comporte comme un filtre « certifié seulement » : un candidat non prouvé est abandonné. C'est le bon choix quand le critère de la règle *est* la certification (`community-state/effect-mirrors-render-value`).

22.6.3 L'émission

Extrait 22.33 – `src/rules/declarative/exec.rs`, l. 868–904 — `emit`, pin $\sqcap$ polarité

```

868     /// `effective = pin  $\sqcap$  polarity`: Error iff pinned Error AND a proof is
869     /// held; the proof's provenance (range/label/notes) rides automatically.
870     /// Downgraded/unproven findings still carry the proofs' trace notes.
871     fn emit(
872         &self,
873         message: String,
874         mut proofs: Vec<Proof>,
875         label: Option<HookLabel>,
876         range: Option<SourceRange>,
877     ) -> Diagnostic {
878         let id = self.def.id.clone();
879         let d = match (self.def.pin, proofs.is_empty()) {
880             (SeverityPin::Error, false) => {
881                 let first = proofs.remove(0);
882                 match first {
883                     Proof::Setter(c) => Diagnostic::error(id, c, message),
884                     Proof::Dominates(c) => Diagnostic::error(id, c, message),

```

```

885         Proof::Init(c) => Diagnostic::error(id, c, message),
886         Proof::Conditional(c) => Diagnostic::error(id, c, message),
887         Proof::Direct(c) => Diagnostic::error(id, c, message),
888     }
889 }
890 (SeverityPin::Error | SeverityPin::Warning, _) => Diagnostic::warn(id, message),
891 (SeverityPin::Info, _) => Diagnostic::info(id, message),
892 };
893 let d = proofs
894     .iter()
895     .fold(d, |d, p| d.with_notes(p.provenance().notes.clone()));
896 let d = match (d.range, range) {
897     (None, Some(r)) => d.with_range(r),
898     _ => d,
899 };
900 match (d.hook_label, label) {
901     (None, Some(l)) => d.with_label(l),
902     _ => d,
903 }
904 }

```

Définition 22.4: Pin, polarité, sévérité effective

Le *pin* d'une règle est la sévérité déclarée par son auteur. La *polarité* d'un finding vaut ERROR si au moins une garde certifiante a produit une preuve pour ce candidat, WARNING sinon. Sur la chaîne $\text{INFO} \sqsubseteq \text{WARNING} \sqsubseteq \text{ERROR}$, la sévérité émise est $\text{pin} \sqcap \text{polarité}$; le registre lui applique ensuite le plafond du consommateur c (`Diagnostic::clamp`), d'où la sévérité affichée

$$s = \text{pin} \sqcap \text{polarité} \sqcap c.$$

pin	preuve détenue ?	sévérité émise
error	oui	ERROR, par <code>Diagnostic::error(id, Certified, msg)</code> : position, label et notes du certificat
error	non	WARNING
warning	indifférent	WARNING
info	indifférent	INFO

TABLE 22.5 – La table de emit. La première preuve détenue sert de certificat; les suivantes n'ajoutent que leurs notes de témoin.

Le plafond du consommateur ne peut que descendre :

Extrait 22.34 – `src/rules/api/diagnostic.rs`, l. 104–114 — `Diagnostic::clamp`

```

104    /// Consumer override (ADR-022 §3): lower this finding to `ceiling` when its
105    /// constructed severity exceeds it. Downgrade-only, hence sound to expose:
106    /// Error is constructible only through [Diagnostic::error], so the
107    /// constructed severity IS the polarity ceiling and  $\text{pin} \sqcap \text{polarity}$ 
108    /// reduces to a min – an upgrade request is a structural no-op.
109    pub fn clamp(mut self, ceiling: Severity) -> Self {
110        if ceiling.rank() < self.severity.rank() {
111            self.severity = ceiling;
112        }
113        self
114    }

```

Avec l'entrée `"guardrails/self-retriggering-effect": "warning"` sous la clef `rules` de la

configuration /tmp/ch22/opt2/reactant.config.json, l'ERROR d'Unconditional sort en WARNING (Guarded était déjà un WARNING, inchangé) :

```
Guarded (2 hooks) ex2_self_retrigger.tsx
warn   guardrails/self-retriggering-effect [hook:1] (line 14:15) this effect writes `n`, which is listed in its
        ↳ own dependency array
Unconditional (2 hooks) ex2_self_retrigger.tsx
warn   guardrails/self-retriggering-effect [hook:1] (line 6:4) this effect writes `n`, which is listed in its own
        ↳ dependency array
```

Une demande de promotion ("error" sur une règle épinglée warning) est, symétriquement, sans effet.

Invariant 22.1: Une Error de pack porte un certificat

Un diagnostic de pack est une ERROR seulement si (i) la règle est épinglée error, (ii) une garde `must_*` a obtenu, pour *ce* candidat, un `Certified<_>` frappé par une primitive du moteur, et (iii) aucun plafond consommateur ne l'a abaissé. La condition (ii) n'est pas une politique : `emit` n'a pas d'autre chemin vers `Diagnostic::error`, et `Certified` ne se construit pas hors de `api::query`.

Décision : ADR-22 §3 — la sévérité par finding, à l'émission

Décision. La sévérité est `pin` $\sqcap$ `polarity`, calculée par finding et à l'émission. La *stratification* en découle gratuitement : une règle épinglée error émet ERROR là où la preuve existe et WARNING ailleurs, sans qu'on l'écrive deux fois. Le pin est un plafond, jamais une promesse ; la surcharge du consommateur est soumise au même $\sqcap$.

Alternatives refusées. Un rejet statique d'une règle error « insuffisamment prouvée » (qui aurait interdit la stratification) ; une sévérité prise telle quelle dans le JSON (qui aurait donné l'autorité ERROR à un auteur de pack, en contradiction avec ADR-21).

Remarque

La position de l'ERROR vient du certificat, celle du WARNING du candidat. Dans l'exemple, les deux coïncident avec un site d'appel du setter, mais ce n'est pas une règle générale : une preuve peut désigner un autre site que la ligne navigée, et c'est elle qui fait foi. En JSON (`-format json`), les deux diagnostics ont `"notes": []` : la preuve de `must_setter_on_all_paths` ne porte pas ici de témoin à afficher.

22.7 Les quantificateurs

Une liste de gardes est une conjonction sur *un* candidat. Trois gardes composées étendent ce pouvoir d'expression : `any_of` (une disjonction sur le même candidat), `every` (un $\forall$ sur les deps de l'ancre) et `none` (un $\neg\exists$ sur une arête de l'ancre). Chacune a été l'occasion d'une décision de soundness.

22.7.1 any_of : toutes les branches sont évaluées

Extrait 22.35 – `src/rules/declarative/exec.rs`, l. 696–707 — la disjonction

```
696         // Every branch is evaluated: short-circuiting would make whether a
697         // `must_*` branch contributes its proof – and therefore whether the
698         // finding can reach Error – depend on the order the author wrote
699         // the branches in.
700         ResolvedGuard::AnyOf(children) => {
701             // Not `.any()`: it short-circuits, and each call pushes proofs.
```

```

702         let mut passed = false;
703         for child in children {
704             passed |= self.eval_guard(e, cand, child, proofs);
705         }
706         passed
707     }

```

On attendrait `children.iter().any(...)`. Ce serait faux : `any` court-circuite, et chaque appel peut pousser une preuve. Selon l'ordre dans lequel l'auteur a écrit les branches, une branche `must_*` contribuerait ou non sa preuve, et le finding atteindrait ou non l'ERROR. L'évaluation complète rend la sévérité indépendante de l'ordre des branches (test `any_of_severity_is_branch_order_independent`). Les preuves d'une branche certifiante sont gardées même si la disjonction échoue — sans effet, puisque `eval` abandonne alors le candidat.

Exemple 22.2: Une branche `must_*` qui rend la disjonction toujours vraie

La règle de sonde `chk5/w4` (`/tmp/ch22/w4/p_w4.json`), épinglée `error`, lie les setters du corps d'un effet et exige `any_of` [`must_setter_on_all_paths` of `s`, `in_deps` of `s`], la branche certifiante étant laissée en `keep`. Sur le programme du liste 22.31 :

```

[warn] rule `chk5/w4`: at `rules[0].guards[0].guards[0]`: a `must_*` branch of `any_of` with the default `else`:
  ↳ "keep" always passes, so the disjunction is always true. Add `else`: "drop" if the branch is meant to
  ↳ be a condition
Guarded (2 hooks) ../ex2_self_retrigger.tsx
warn  chk5/w4 [hook:1] (line 14:15) writes `n`
Unconditional (2 hooks) ../ex2_self_retrigger.tsx
error  chk5/w4 [hook:1] (line 6:4) writes `n`

```

(sortie filtrée par `-rule chk5/w4`; sans filtre, la native `infinite-loop` ajoute un WARNING sur `Unconditional`, chapitre 19.)

Trois faits en une sortie. L'avertissement W4 est émis au chargement, avec le chemin de la branche fautive. La disjonction passe sur *tout* setter du corps, la branche `in_deps` étant morte. Et parce que toutes les branches sont évaluées, la preuve est collectée quand elle existe : ERROR pour `Unconditional`, WARNING pour `Guarded`.

Piège

La liste des genres qui déclenchent W4 (`src/rules/declarative/validate.rs`, l. 2169–2173) omet `MustDirectWrite`. Un `must_direct_write` laissé en `keep` dans un `any_of` rend lui aussi la disjonction toujours vraie, sans avertissement : le dossier de notes l'a vérifié par une sonde où chaque écrivain `direct` tire en ERROR alors que l'autre branche, `same_tick`, n'est jamais vraie. Ce n'est pas une brèche de soundness — la disjonction *est* prouvée par sa branche certifiée — mais un défaut de diagnostic d'auteur.

22.7.2 every : un refus, puis un amendement

La règle `guardrails/inert-single-dep` signale un effet dont toutes les deps sont prouvées stables : il ne se relancera jamais après le montage.

Extrait 22.36 – `packs/guardrails.json`, l. 27–38 — `inert-single-dep` (docs omises)

```

27     "severity": "warning",
28     "anchor": { "relation": "hook_calls", "kind": "effect" },
29     "guards": [
30         { "kind": "count", "of": "anchor.deps", "more_than": 0 },
31         {
32             "kind": "every",
33             "of": "anchor.deps",

```



```

34     "as": "dep",
35     "guards": [{ "kind": "stability", "of": "dep", "is": ["stable"] }]
36   }
37 ],
38   "message": "every dependency of this effect is provably stable – it can never re-run after mount"

```

Extrait 22.37 – Des deps prouvées stables, et une dep inconnue (/tmp/ch22/ex3_every.tsx)

```

1  import { useEffect, useRef, useState } from "react";
2  import { useOpaque } from "some-uninstalled-pkg";
3
4  export function Inert() {
5    const ref = useRef<HTMLDivElement>(null);
6    const [n, setN] = useState(0);
7    useEffect(() => {
8      console.log(ref.current, n);
9    }, [ref, setN]);
10   return <div ref={ref}>{n}</div>;
11 }
12
13 export function Opaque() {
14   const ref = useRef<HTMLDivElement>(null);
15   const thing = useOpaque();
16   useEffect(() => {
17     console.log(ref.current, thing);
18   }, [ref, thing]);
19   return <div ref={ref} />;
20 }

```

```

Inert (3 hooks) ex3_every.tsx
  warn  guardrails/inert-single-dep [hook:2] (line 7:2) every dependency of this effect is provably stable – it can
    ↳ never re-run after mount
1 clean component(s) hidden, rerun with --show-clean

```

Dans `Inert`, le conteneur `useRef` et le setter sont `Stable` (chapitre 10) : `every` passe. Dans `Opaque`, `thing` vient d'un hook non résolu, dont le verdict est `⊤ (unknown)`; `is: ["stable"]` échoue sur cet élément, `every` échoue, la règle se tait. Ce silence est une *décision*, et elle a une histoire.

Décision : ADR-23 §4 et son amendement — le quantificateur universel

Refus initial. ADR-23 §4 refusait tout $\forall$ en v1, pour deux raisons. (1) « $\top$ *folds to "violates"* » : un pli booléen ferait d'un seul `unknown` la suppression du finding pour toute la ligne. (2) « $\forall$ *is vacuously true over an edge we cannot enumerate exactly* » : spreads et élisions perdus, deps non littéraux lus comme `[]`.

Amendement (2026-09-01) : « **the condition is discharged; every ships** ». Le tableau littéral porte désormais un bit exact, et un deps non littéral ne donne plus de liste du tout : la raison (2) tombe. Pour la raison (1), la solution proposée — faire satisfaire $\top$ par le quantificateur — a été *essayée et rejetée à la mesure* : `inert-single-dep` tirait alors sur tout effet dépendant d'une prop, les props d'une racine étant $\top$. Réponse retenue : « *Whether $\top$ satisfies is the body's decision, not the quantifier's* ». Une règle qui accepte les listes *qui pourraient* convenir l'écrit `is: ["stable", "unknown"]`.

Alternative refusée. Épingler l'arité (`count equals 1`, puis `2`, puis...) pour simuler le $\forall$: « *exactly the per-rule hack the project forbids* ».

L'implémentation reflète ces trois points :

Extrait 22.38 – `src/rules/declarative/exec.rs`, l. 757–785 — `every`

```

757 ResolvedGuard::Every(body) => {

```

```

758         let Some(row) = cand.row() else {
759             unreachable!("validated: `every` quantifies over a deps-bearing anchor")
760         };
761         // Quantifying needs a domain. A written array supplies one
762         // even when a spread hides part of it – the fold then ranges
763         // over the elements the engine can see and refutes  $\forall$  as soon
764         // as one of them violates. An absent or unreadable argument
765         // supplies no element at all, and a claim about nothing is not
766         // a claim the engine may make.
767         if row.effect.and_then(|i| i.deps.list()).is_none() {
768             return false;
769         }
770         // `proofs` is deliberately not threaded in: a may-typed
771         // quantifier must never contribute Error authority. The
772         // validator already refuses a `must_*` inside the body, so
773         // this scratch vector stays empty – it is the second lock.
774         let mut scratch: Vec<Proof> = Vec::new();
775         let all = e.deps(row).into_iter().all(|dep| {
776             let elem = Candidate::Hook {
777                 row,
778                 bound: Some(Bound::Dep(&dep)),
779             };
780             body.iter()
781                 .all(|g| self.eval_guard(e, &elem, g, &mut scratch))
782         });
783         debug_assert!(scratch.is_empty(), "`every` must not mint a proof");
784         all
785     }

```

Un domaine est requis : un argument absent ou illisible ne fournit aucun élément, et « *a claim about nothing is not a claim the engine may make* », d'où l'échec. Un tableau écrit fournit un domaine même quand un spread en cache une partie : un seul élément visible qui viole réfute le $\forall$. Une liste *connue vide* quantifie vacuement vrai — d'où le count `more_than 0` en tête de `inert-single-dep`. Enfin `every` n'a aucune autorité `ERROR`.

Invariant 22.2: Aucune preuve ne sort d'un quantificateur

Un finding sélectionné par `every` ou `none` ne peut pas être une `ERROR`. Deux verrous l'assurent. Le validateur rejette un `must_*` à l'intérieur du quantificateur (E22) et dans toute règle qui en utilise un (E23). À l'exécution, le corps du quantificateur écrit dans un vecteur `scratch` jamais fusionné dans `proofs` ; pour `every`, un `debug_assert!` vérifie qu'il reste vide.

La garde `count`, qui accompagne souvent `every`, répond sur une borne inférieure ce qu'elle *réfute* :

Extrait 22.39 – `src/rules/declarative/exec.rs`, l. 649–681 — `count` sur une arité exacte ou minorée

```

649         ResolvedGuard::Count(cmp) => {
650             let Some(arity) = cand
651                 .row()
652                 .and_then(|r| r.effect)
653                 .and_then(|i| i.deps.list())
654                 .map(|l| l.arity)
655             else {
656                 return false;
657             };
658             match arity {
659                 Arity::Exact(m) => {
660                     let m = m as u64;

```

```

661         match cmp {
662             CountCmp::MoreThan(n) => m > *n,
663             CountCmp::LessThan(n) => m < *n,
664             CountCmp::Equals(n) => m == *n,
665         }
666     }
667     // A flattened spread leaves only a lower bound, so the
668     // guard answers what that bound *refutes* and passes
669     // otherwise. Refusing outright deleted findings: the arity
670     // of `[a, ..., g, ...rest]` provably exceeds any budget its
671     // visible elements already exceed.
672     Arity::AtLeast(m) => {
673         let m = m as u64;
674         match cmp {
675             CountCmp::MoreThan(_) => true,
676             CountCmp::LessThan(n) => m < *n,
677             CountCmp::Equals(n) => m <= *n,
678         }
679     }
680 }
681 }

```

Une règle de sonde `count equals 2` (/tmp/ch22/count/) le montre sur cinq effets de deps respectives `[a, ...rest]`, `[a, b, c, ...rest]`, `[a, ,]`, `rest` et `[a, b]` (lignes 4 à 8) :

```

Deps (5 hooks) count.tsx
warn  demo/two-deps [hook:0] (line 4:2) this effect has exactly two deps
warn  demo/two-deps [hook:2] (line 6:2) this effect has exactly two deps
warn  demo/two-deps [hook:4] (line 8:2) this effect has exactly two deps

```

`[a, ...rest]` est au moins 1, ce qui ne réfute pas « égal à 2 » : la garde passe (faux positif accepté). `[a, b, c, ...rest]` est au moins 3 : réfuté. L'élision de `[a, ,]` compte comme un élément, l'arité exacte est 2. La variable `rest` n'a pas d'arité : échec (c'est `deps_declared` qui en parle). Refuser toute borne inférieure, comme le faisait une version antérieure, supprimait des findings : l'arité de `[a, ..., g, ...rest]` dépasse prouvablement tout budget que ses éléments visibles dépassent déjà.

Piège

Deux commentaires du code décrivent encore l'ancien `count` qui « refuse » sur un tableau aplati (`src/rules/declarative/exec.rs`, l. 640–644 et le doc-commentaire de `EntityCtx::deps`, `src/rules/declarative/entity.rs`, l. 430–434). Le bras `Arity::AtLeast` les contredit : c'est lui qui fait foi.

22.7.3 none : la négation d'un existentiel

Un `forEach` est un existentiel : la règle tire s'il existe un élément de l'arête qui passe les gardes. Ce que les auteurs demandaient sans pouvoir l'écrire, c'est l'inverse : « acquiert une ressource et n'en libère aucune », « a un `prop value` et pas de `onChange` ». Le commentaire de la garde, livrée avec la relation des appels (ADR-36), en fait l'argument de `soundness` :

Extrait 22.40 – `src/rules/declarative/schema.rs`, l. 641–659 — `none` : la direction non sûre est ici la bonne

```

641     /// Negated existential over an edge of the anchor (#126): passes when
642     /// **no** row satisfies the nested guards.
643     ///
644     /// The one thing the guard language could not say, and what the wish-list
645     /// kept asking for: "acquires a resource and releases none", "has a `value`"

```

```

646    /// prop and no `onChange`, "subscribes and never reads the current value".
647    /// A `forEach` is the existential; this is its negation, and neither can be
648    /// written as the other.
649    ///
650    /// **The unsound direction is the safe one here.** Every relation it can
651    /// quantify over may under-enumerate, a depth-capped walk or a callee it
652    /// could not resolve, and a missing row makes `none` pass, so the rule
653    /// fires where it should not. That is a false positive, which this project
654    /// accepts; the direction it never takes is losing a finding.
655    ///
656    /// Never mints a proof, exactly like `every`: a `must_*` anywhere in a rule
657    /// that uses it is rejected at load time.
658    #[serde(rename = "none")]
659    NoneOf {

```

Définition 22.5: Quantificateur none

La garde `{ "kind": "none", "of": "anchor.e", "as": "q", "guards": [h1, ..., hm] }` passe sur un candidat d'ancre a si et seulement si aucun élément $q \in e(a)$ ne satisfait $h_1 \wedge \dots \wedge h_m$. L'arête e est typée par la même table que `forEach`. Si l'énumération $e(a)$ sous-estime les éléments concrets, `none` peut passer à tort : c'est un faux positif, jamais un faux négatif ; et comme la garde ne produit aucune preuve, le finding est au plus un `WARNING`.

Extrait 22.41 – `src/rules/declarative/exec.rs`, l. 727–756 — `none` sur une arête d'une ligne de `hook`

```

727    ResolvedGuard::NoneOf { edge, body } => {
728        let Some(row) = cand.row() else {
729            unreachable!("validated: `none` quantifies over an anchor's edge")
730        };
731        // Same second lock as `every`: the validator refuses a
732        // `must_*` inside, so this scratch vector stays empty and no
733        // Error authority can leak out of a negated existential.
734        let mut scratch: Vec<Proof> = Vec::new();
735        let mut hit = |bound: Bound<'_, '_>| {
736            let elem = Candidate::Hook {
737                row,
738                bound: Some(bound),
739            };
740            body.iter()
741                .all(|g| self.eval_guard(e, &elem, g, &mut scratch))
742        };
743        let any = match edge {
744            EdgeName::Deps => e.deps(row).iter().any(|d| hit(Bound::Dep(d))),
745            EdgeName::BodySetterCalls => {
746                e.body_setters(row).iter().any(|s| hit(Bound::Setter(s)))
747            }
748            EdgeName::Args => e.args(row).iter().any(|a| hit(Bound::Arg(a))),
749            EdgeName::Writers => e.writers(row).iter().any(|w| hit(Bound::Writer(w))),
750            EdgeName::Seeds => e.seeds(row).iter().any(|s| hit(Bound::Seed(s))),
751            EdgeName::Calls => e.body_calls(row).iter().any(|c| hit(Bound::Call(c))),
752            EdgeName::Reads => e.reads(row).iter().any(|r| hit(Bound::Read(r))),
753            EdgeName::Props => unreachable!("handled above"),
754        };
755        !any
756    }

```

Un candidat interne remplace la liaison : la ligne ancre reste la même, l'élément lié devient la ligne quantifiée. Le bras précédent (`src/rules/declarative/exec.rs`, l. 708–726) traite de la

même façon `none of anchor.props` pour l'ancre `elements`.

Exemple 22.3: Un input contrôlé sans écrivain

Extrait 22.42 – `packs/community/wave2.json`, l. 349–388 — `controlled-input-without-a-writer` (docs et params omis)

```

349     "anchor": {
350       "relation": "elements",
351       "elements": "host"
352     },
353     "forEach": {
354       "edge": "props",
355       "as": "p"
356     },
357     "guards": [
358       {
359         "kind": "name",
360         "of": "anchor",
361         "one_of": {
362           "$param": "tags"
363         }
364       },
365       {
366         "kind": "prop",
367         "of": "p",
368         "one_of": [
369           "value",
370           "checked"
371         ]
372       },
373       {
374         "kind": "none",
375         "of": "anchor.props",
376         "as": "q",
377         "guards": [
378           {
379             "kind": "prop",
380             "of": "q",
381             "one_of": {
382               "$param": "writers"
383             }
384           }
385         ]
386       }
387     ],
388     "message": "<{anchor.name}> is driven by {p.prop} with nothing on the same element that can
    ↪ change it"

```

Les paramètres omis valent `tags = ["input", "textarea", "select"]` et `writers = ["onChange", "onInput", "readOnly", "disabled", "defaultValue"]`. Sur

Extrait 22.43 – `/tmp/ch22/ex5_input.tsx`

```

1  import { useState } from "react";
2
3  export function Broken() {
4    const [name] = useState("");
5    return <input value={name} />;
6  }
7

```

```

8 export function Fine() {
9   const [name, setName] = useState("");
10  return <input value={name} onChange={(e) => setName(e.target.value)} />;
11 }

```

l'analyseur répond :

```

Broken (1 hooks) ex5_input.tsx
warn  community-wave2/controlled-input-without-a-writer (line 5:9) <input> is driven by `value` with
      ↳ nothing on the same element that can change it
1 clean component(s) hidden, rerun with --show-clean

```

L'ancre `elements` (filtre `host`) lie l'élément `<input>`; `forEach` `props` as `p` lie son `prop value`; `none of anchor.props` as `q` parcourt les `props` du *même* élément et n'en trouve aucun parmi `writers` dans `Broken`. Le gabarit `<{anchor.name}>` donne `<'input'>`, les noms source étant rendus entre backquotes. Pas de `[hook:N]` : un élément n'a pas de label de hook.

Exemple 22.4: Une ressource acquise et jamais rendue

Extrait 22.44 — `packs/community/wave2.json`, l. 93–131 — `acquired-resource-not-released` (docs et params omis)

```

93   "anchor": {
94     "relation": "hook_calls",
95     "kind": "effect"
96   },
97   "forEach": {
98     "edge": "calls",
99     "as": "c"
100  },
101  "guards": [
102    {
103      "kind": "name",
104      "of": "c",
105      "one_of": {
106        "$param": "acquire"
107      }
108    },
109    {
110      "kind": "phase",
111      "of": "c",
112      "is": [
113        "effect"
114      ]
115    },
116    {
117      "kind": "none",
118      "of": "anchor.calls",
119      "as": "r",
120      "guards": [
121        {
122          "kind": "name",
123          "of": "r",
124          "one_of": {
125            "$param": "release"
126          }
127        }
128      ]
129    }
130  ],

```

```
131 "message": "this effect calls {c.name} and nothing in it gives that back – no release call
    ↳ appears anywhere in the effect or its cleanup"
```

Par défaut acquies vaut ["observe", "createObjectURL", "lock", "acquies"] et release liste disconnect, unobserve, revokeObjectURL, leave, unlock, release, stop, abort, close. Sur un composant qui observe un élément sans jamais se déconnecter, et sur son jumeau correct (/tmp/ch22/ex6_resource.tsx, -show-clean) :

Extrait 22.45 – /tmp/ch22/ex6_resource.tsx

```
1 export function LeakFires({ cb }: { cb: ResizeObserverCallback }) {
2   const el = useRef<HTMLDivElement>(null);
3   useEffect(() => {
4     const ro = new ResizeObserver(cb);
5     ro.observe(el.current!);
6   }, [cb]);
7   return <div ref={el} />;
8 }
9
10 export function ReleaseSilent({ cb }: { cb: ResizeObserverCallback }) {
11   const el = useRef<HTMLDivElement>(null);
12   useEffect(() => {
13     const ro = new ResizeObserver(cb);
14     ro.observe(el.current!);
15     return () => ro.disconnect();
16   }, [cb]);
17   return <div ref={el} />;
18 }
```

```
LeakFires (2 hooks) ex6_resource.tsx
  warn community-wave2/acquired-resource-not-released [hook:1] (line 7:4) this effect calls `observe` and
    ↳ nothing in it gives that back – no release call appears anywhere in the effect or its cleanup
ReleaseSilent (2 hooks) ex6_resource.tsx ✓
```

(la ligne d'import de react est omise du listing ; les numéros de ligne de la sortie sont ceux du fichier). Les appels du corps (BodyCall) portent un nom, un receveur et une phase : ro.observe(...) est de phase effect, ro.disconnect() de phase cleanup, puisqu'il est dans la fonction retournée. La garde phase restreint l'acquisition au corps de l'effet ; le none ne filtre pas la phase, si bien que la libération dans le cleanup est vue. La règle exige un name au niveau supérieur sur c (E24). Sa limite, documentée dans son why : sans valeur des arguments (#67), elle ne peut pas vérifier que c'est la *même* ressource qui est libérée.

Rappel React : Ce que React ne libère pas

Au démontage, React exécute la fonction de cleanup de chaque effet, et rien d'autre. Un ResizeObserver, une URL d'objet, un verrou, une socket restent vivants tant que le code ne les libère pas explicitement ; en Strict Mode, le double montage de développement rend la fuite visible deux fois.

La règle community-wave2/state-never-read-during-render combine writer_phases et none sur l'arête reads : « le slot a au moins un écrivain, dans n'importe quelle phase, et aucune lecture de phase render, memo ou unknown n'est visible » (packs/community/wave2.json, l. 282–318). Sur la fixture tests/fixtures/community_wave2/state7.tsx, où un scrollY est écrit à chaque défilement sans jamais être rendu :

```
State7Fires (2 hooks) state7.tsx
  warn community-wave2/state-never-read-during-render [hook:0] (line 4:8) state `scrollY` is written but no render
    ↳ -phase read of it is visible – every write costs a render that changes nothing
```

```
1 clean component(s) hidden, rerun with --show-clean
```

Le unknown de la liste des phases de lecture est essentiel : une lecture de phase $\top$ *pourrait* être une lecture de rendu, elle doit donc suffire à faire taire la règle. La doc de l’arête le rappelle : « *the ABSENCE of rows is not a proof that the slot is unread* » (src/rules/declarative/schema.rs, l. 286–295).

22.8 La croissance du vocabulaire

22.8.1 Mesurer ce qu’on sait dire

À sa naissance (ADR-22), le langage était beaucoup plus petit qu’aujourd’hui. ADR-23 a mesuré sa portée sur un catalogue de vingt et une classes de règles tirées de huit corpus : trois seulement étaient exprimables, et encore sous une forme affaiblie. Les bloqueurs se rangeaient en cinq classes : entités de position d’expression manquantes (8 classes), ancre unique (4), faits que le moteur ne calcule pas (5), programme entier (3), identité de hook (1). La conclusion tient en une phrase : « *No rule language can invent a fact the engine never computed* ».

Décision : ADR-23 §1 et §5 — croître par les entités, rester déclaratif

Décision. (§1) Le langage grandit par les *entités*, pas par les gardes : une entité est admissible si elle nomme une position dans une relation déjà résolue ; « *Entities add positions, never verdicts* ». (§5) La voie communautaire est l’écriture de packs en JS/TS compilés en JSON committé : « *a JS pack is arbitrary code execution at analysis time, as it is for ESLint. Committing the generated JSON confines that to the authoring machine[...]* »

Alternatives refusées. Starlark comme Tier B (crate non compilable pour wasm32, arbre de dépendances contre une distribution légère, alors qu’un parseur JS, oxc, est déjà embarqué) ; des gardes génériques qui « pointent » un verdict existant vers une nouvelle position (ADR-23 §2).

Le test tests/catalogue.rs matérialise ce catalogue (rebasé à vingt-deux classes par ADR-27 §6) : une entrée Expressible est *prouvée* (sa règle de pack se charge, tire sur une fixture fautive, se tait sur une fixture conforme) ; une entrée Blocked nomme le vocabulaire manquant. La constante EXPRESSIBLE_NOW vaut 21 (tests/catalogue.rs, l. 1094). L’en-tête du fichier retrace la courbe : 3/21 (ADR-22) → 5/21 (provenance des hooks et garde origin, arête args et garde returns) → 6/21 (arête writers, writer_phases) → 7/22 (provenance des écritures, must_direct_write, rebasage) → 8/22 (context_providers, identity) → ... → 21/22 (registrations). Le tableau 22.6 résume les ADR qui ont fait grandir le langage.

22.8.2 L’élargissement nommé

Chaque extension pose la même question : que devient un pack déjà publié ? ADR-27 a formulé l’argument de séquençement : changer ce qu’une sorte déjà livrée énumère, ou affiner un $\top$ après que des packs existent, change les findings que ces packs émettent. D’où une règle de conception, appliquée par ADR-30 §2 aux setters étrangers (un setter de parent reçu en prop) :

« *The enumeration widens on the GUARD, never on the sort. [...] naming ownership is what makes owner-qualified rows exist.* »

L’ancre render_setter_calls n’énumère les lignes étrangères que si la règle nomme slot_ownership quelque part, any_of compris (liste 22.22) : un pack écrit avant l’extension lie

ADR	apport au langage
ADR-27	arête writers, writer_phases, provenance, must_direct_write, ancre hook_origins (réparation de #6)
ADR-28	une ligne writers par site; updater, updater_body, same_tick
ADR-29	ancre churn_cycles, garde cycle (booléens exacts, jamais de Certified)
ADR-30	lignes de setters de rendu qualifiées par propriétaire, slot_ownership
ADR-31	arête seeds, seed_sync
ADR-32	ancre context_consumers, provider
ADR-34	ancre registrations, teardown, registers
ADR-35	phase deferred après un await
ADR-36	calls, render_calls, receiver, phase, none, elements/props, filtre host
ADR-37	arête reads
ADR-42	les relations sont des produits du moteur; la couche règles ne marche aucune syntaxe

TABLE 22.6 – Les ADR qui ont étendu le vocabulaire du Tier A après ADR-22 et ADR-23.

exactement les lignes qu’il liait. Le filtre `elements: host` de l’ancre `elements`, dont le défaut reste `component`, suit le même principe. Et le nom d’un slot étranger est cherché dans le composant *propriétaire* (`src/rules/declarative/entity.rs`, l. 969–977) : les labels sont locaux à un composant et entreraient en collision.

22.8.3 Les packs livrés

fichier	pack	règles
<code>packs/guardrails.json</code>	<code>guardrails</code>	5 : les règles de ce chapitre
<code>packs/community/async.json</code>	<code>community-async</code>	5 (continuations asynchrones, listeners, sélecteurs de store)
<code>packs/community/effects.json</code>	<code>community-effects</code>	4 (enregistrements sans <code>teardown</code> , liens d’effets)
<code>packs/community/render.json</code>	<code>community-render</code>	3 (props instables, snapshots de store)
<code>packs/community/state.json</code>	<code>community-state</code>	3 (<code>same-tick-slot-collapse...</code>)
<code>packs/community/wave2.json</code>	<code>community-wave2</code>	8 (ressources, navigation en rendu, inputs contrôlés...)

TABLE 22.7 – Les packs livrés au commit `e67b10a`.

Les packs communautaires ont un statut particulier, fixé par l’en-tête de `tests/community_packs.rs` : « *These are NOT first-party rules. Several are proxies with known false positives — the campaign’s point was to measure the vocabulary against demand, and a proxy that had to stand in for a missing fact is evidence, not a recommendation.* » Aucun n’est épinglé `error` (test `no_community_rule_claims_an_error`). Les docs de `controlled-input-without-a-writer` le disent crûment : sur un corpus de 34 730 fichiers, tous les hits étaient des `<input type="hidden" value=.../>`, que la règle ne peut exclure faute de voir la *valeur* du prop `type` (#67); c’est « une démonstration que la forme est exprimable, pas une règle à activer telle quelle ».

22.9 Limites, et une brèche ouverte

22.9.1 La brèche de conjonction (#143)

Reprenons l’issue ouverte #143, étiquetée `soundness-bug`. Un auteur veut une règle `ERROR` « état écrit pendant le rendu » et l’écrit ainsi :

Extrait 22.46 – /tmp/ch22/i143/pack.json — la règle de l'issue #143

```

5   {
6     "id": "render-writer",
7     "docs": { "description": "d", "why": "w", "fix": "f" },
8     "severity": "error",
9     "anchor": { "relation": "hook_calls", "kind": "state" },
10    "forEach": { "edge": "writers", "as": "w" },
11    "guards": [
12      { "kind": "writer_phases", "of": "anchor", "includes": ["render"] },
13      { "kind": "must_direct_write", "of": "w", "else": "drop" }
14    ],
15    "message": "state {anchor.name} is written during render ({w.region}, phase {w.phase}, via {w.via})"
16  }

```

Extrait 22.47 – Un setter confié à une bibliothèque, un autre à un handler (/tmp/ch22/i143/form.tsx)

```

1  import { useState } from "react";
2  import { handleSubmit } from "some-form-lib";
3
4  export function Form() {
5    const [n, setN] = useState(0);
6    return (
7      <form onSubmit={handleSubmit(() => setN(1))}>
8        <button onClick={() => setN(2)}>{n}</button>
9      </form>
10    );
11  }

```

```

Form (2 hooks) i143/form.tsx
error demo/render-writer [hook:0] (line 5:8) state `n` is written during render (handler, phase handler, via
  ↳ direct)
error demo/render-writer [hook:0] (line 7:4) state `n` is written during render (render, phase unknown, via
  ↳ direct)
warn setter-in-render [hook:0] (line 7:4) setter `setN` is handed to a callee with no timing summary. If that
  ↳ callee runs it during render, this re-renders on every render
    (1 trace step(s), rerun with --trace)

! 2 error(s), 1 warning(s) across 1 file(s).

```

Deux ERROR, dont l'une sur une écriture de handler, là où la règle native `setter-in-render` n'émet qu'un WARNING. Suivons l'évaluation. `writer_phases` a pour sujet l'*ancree* : c'est un existentiel sur *tous* les écrivains du slot, qui passe grâce à la ligne T (le callback confié à `handleSubmit`, de phase `unknown`, qui satisfait toute requête de phase). Il ne restreint donc pas la ligne liée `w. must_direct_write` certifie ensuite chaque ligne, y compris celle du `onClick`, de phase handler. La preuve détenue dit « cette écriture est directe », pas « cette écriture a lieu pendant le rendu » : elle prouve un conjoint, pas la conclusion. C'est exactement ce que la définition d'ERROR du projet exclut (« preuve de *toute* la conclusion, pas d'un seul conjoint », définition 3.8). Détail annexe : la première ligne (5:8) est l'écriture `setN(2)` du handler, dont la ligne `writers` n'a pas de position propre et retombe sur le site du `useState`.

Proposition 22.2: Ce que prouve réellement une Error de pack

Si une règle épinglée `error` émet une ERROR sur un candidat, alors la garde certifiante qui a fourni la preuve est vraie de ce candidat *dans toutes les exécutions*. Si de plus chaque autre garde de la conjonction de premier niveau est *exacte* ou certifiante, la conjonction entière est prouvée. Sans cette hypothèse, une garde *may* de la conjonction peut être vraie par sur-approximation seulement, et l'ERROR excède ce qui est prouvé.

Esquisse. La première partie est l'invariant 22.1 : le Certified est frappé par une primitive must sur le sujet du candidat. Une garde exacte (name, prop, count sur arité exacte, cycle...) est vraie du candidat concret si elle l'est de l'abstrait ; une garde certifiante l'est par construction ; la conjonction de faits vrais sur toutes les exécutions l'est aussi. L'exemple ci-dessus fournit le contre-exemple quand une garde may (writer_phases) s'y mêle. □

La direction de correctif proposée par l'issue est une *polarité par garde* calculée à la validation : exacte (name, source, prop, count, cycle...), must (must_*, identity is fresh-every-render, stability is stable...) ou may (phase, writer_phases, provider, registers, same_tick, toute liste de valeurs contenant unknown...). Une règle mêlant une garde may à un must_* serait plafonnée à WARNING, avec un avertissement de chargement semblable à W1. Le correctif exige un amendement d'ADR-22 §3, qui supposait que le verdict certifié *est* celui du finding. La cause est la même que celle de #142 dans les règles natives (chapitre 17) : c'est au niveau central, dans la sémantique de la conjonction, qu'elle se corrige, pas par une interdiction ad hoc de la paire writer_phases/must_direct_write.

Piège

writer_phases porte sur le slot, pas sur la ligne navigée. Combiné à un forEach writers, il ne restreint pas l'écrivain lié ; pour filtrer la phase d'un écrivain, il n'existe pas aujourd'hui de garde phase sur la sorte Writer (elle n'admet que Call et Read, tableau 22.2), seul le champ {w.phase} peut être affiché.

22.9.2 Les limites connues

- **Ancre unique** (#68, ouverte) : une règle ne joint pas deux entités indépendantes, donc les règles inter-composants restent inexprimables, « *never by a syntactic bypass* ». Les relations programme (churn, consommateurs de contexte) sont contournées par *projection* sur le composant ancré.
- **Peu de verdicts d'expression** (#67, ouverte) : seuls les deps et les arguments de hooks personnalisés portent un verdict ; ni la valeur d'un prop, ni celle d'un provider, ni l'argument d'un setter. D'où les proxies des packs communautaires.
- **Pas de verdict d'atteignabilité du rendu** (#132, ouverte) : une lecture pendant le rendu n'est pas une preuve que la valeur atteint la sortie ; state-never-read-during-render se tait sur un slot lu puis jeté.
- **Un plafond de 21/22** (#101, fermée wontfix) : la classe nullable-return-unguarded est exclue par conception. Son ancre serait un site de déréréfencement, une entité de forme syntaxique, inadmissible par ADR-23 §1 ; et ADR-20 refuse de câbler les types TypeScript dans le domaine. Réponse à l'équipe qui la réclame : activer strictNullChecks.
- **kind: "custom" ne voit que les hooks non résolus** (#6, fermée) : un hook inline perd sa ligne custom. Les règles d'identité vont sur hook_origins (d'où W3).
- **Relations may** : calls, render_calls, reads et registrations sont des correspondances de noms résolus, jamais des preuves de primitive hôte ; aucune garde must_* n'accepte ces sortes, leur plafond WARNING est structurel (ADR-36 §4, dans l'esprit de la décision #42).

22.9.3 Écarts constatés entre documentation et code

Au commit photographié, plusieurs textes sont en retard sur le code ; le lecteur qui les croise doit suivre le code. skills/reactant-rules/SKILL.md parle encore de deux ancres et affirme qu'aucun quantificateur universel n'existe ; docs/custom-rules.md dit qu'une collision d'*identifiant*

avec un nom natif rejette le pack, alors que c'est le *nom du pack* qui est comparé, et décrit le refus du $\forall$ sans mentionner `every` ; le `fix` de `community-render/unstable-prop-to-child` affirme qu'aucune garde ne porte sur le nom d'un prop, alors que `prop` existe (ADR-36 §9) ; le commentaire de tête de `schema.rs` nomme une fonction `validate::check_unknown_keys` qui s'appelle `check_keys` ; `Sort::describe` produit « a effect hook call » ; et la section « What a pack can name in a body » de `docs/limitations.md` tient pour incroyable « un élément hôte avec un prop value et pas de `onChange` », que `controlled-input-without-a-writer` exprime désormais. Les six écarts ont été relus au commit `e67b10a` : aucun n'était corrigé.

Résumé du chapitre

- Un **pack** est un fichier JSON inerte de règles déclaratives (Tier A). Une règle est une **ancree** (une relation déjà résolue par le moteur, jamais un motif syntaxique), au plus une **arête** `forEach`, une conjonction de **gardes**, un **gabarit**, une doc obligatoire et un **pin** de sévérité. Parce qu'`alias`, `inlining` et `point fixe` ont tourné avant, une règle de pack voit ce qu'un linter syntaxique ne voit pas (ADR-22).
- Un **système de seize sortes** type chaque arête (`edge_element_sort`, partagé par `forEach` et `none`), chaque garde et chaque champ (la table unique `Field`). Un verdict ne se lit qu'au point de programme de son entité (ADR-23 §2).
- `load_pack` charge en **trois étages** (JSON, `serde` typé, validation sémantique), s'arrête à la première faute avec un chemin exact, et cuit paramètres et options dans une `ResolvedRule` ; le cœur revalide tout pack, quel que soit l'hôte.
- L'exécuteur `TierARule` construit un `EntityCtx` par (règle, composant), énumère les candidats et évalue une **conjonction court-circuitée** ; `any_of` évalue toutes ses branches.
- La sévérité est **pin** $\square$ **polarité** $\square$ **plafond consommateur** : `ERROR` seulement avec un `Certified` frappé par une primitive `must_*` ; `keep` stratifie, `drop` filtre.
- `every` et `none` sont `may`, sans preuve (deux verrous) ; $\top$ est l'affaire du corps, pas du quantificateur ; `none` transforme une sous-énumération en faux positif, jamais en faux négatif.
- Le vocabulaire a crû par les entités de 3/21 à 21/22 classes exprimables ; toute extension élargit une énumération seulement si la règle la nomme.
- Une brèche reste ouverte : une `ERROR` de pack peut ne prouver qu'un conjoint (#143).

Fichiers rencontrés : les cinq fichiers de `src/rules/declarative/` (`mod.rs`, `schema.rs`, `validate.rs`, `entity.rs`, `exec.rs`), `packs/guardrails.json`, `packs/community/*.json`, `src/cli/config_load.rs`, `tests/declarative.rs`, `tests/catalogue.rs`. Le chapitre 23 montre comment la configuration et le registre s'insèrent dans la commande `check` ; le chapitre 24 décrit la distribution des packs (`npm`, `WASM`, `packs build`, action GitHub).

Exercices

Exercice 22.1: Un mémo qui dépend d'une valeur recréée

Écrire une règle qui signale un `useMemo` dont un `dep` est `per-render`, avec le chemin du `dep` dans le message.

Solution

```
Ancre {"relation": "hook_calls", "kind": "memo"}, {"forEach": {"edge": "deps", "as": "d"}},
garde {"kind": "stability", "of": "d", "is": ["per-render"]}, message "{d.path} is
```

recreated on every render". C'est, à la formulation près, la règle `team/no-per-render-memo-dep` de la fixture `tests/fixtures/packs/team.json`. Le `pin warning` suffit : aucune garde n'est certifiante.

Exercice 22.2: Deux quantificateurs, un même fait

Sur le composant `Opaque` du liste 22.37, que répondent `every ... is ["stable"]` et `every ... is ["stable", "unknown"]`? Pourquoi ADR-23 a-t-il laissé ce choix au corps?

Solution

Le premier échoue (`thing` est $\top$), le second passe. Faire satisfaire $\top$ par le quantificateur lui-même aurait fait tirer `inert-single-dep` sur tout effet dépendant d'une prop de racine (mesuré puis rejeté), et aurait donné à `every` et à `forEach` deux lectures différentes du même verdict.

Exercice 22.3: Compter sur une borne

Que répond `count less_than 3` sur `[a, ...rest]`, `[a, b, c, ...rest]`, `[a, ,]` et `rest`? Et `more_than 1`?

Solution

`less_than 3` : passe ($1 < 3$), échoue ($3 \not< 3$), passe (arité exacte 2), échoue (pas d'arité).
`more_than 1` : passe (une borne inférieure ne réfute jamais « $> n$ »), passe, passe ($2 > 1$), échoue (liste 22.39).

Exercice 22.4: Le W4 manquant

Une règle épinglée `error`, ancre `state`, `forEach writers as w`, gardes `any_of [must_direct_write of w, same_tick of w]` (branche certifiante en `keep`). Prédire, sur un composant dont l'unique écrivain est `direct` et non `same_tick`, les avertissements de chargement et les diagnostics.

Solution

Aucun avertissement : la liste de W4 omet `MustDirectWrite`. La branche `keep` passe toujours, la disjonction est vraie ; la preuve de l'écriture directe est collectée, et le `pin error` donne une `ERROR` sur l'écrivain, alors que `same_tick` est faux. L'`ERROR` est justifiée par la disjonction telle qu'elle est écrite ; l'auteur, lui, n'a pas été prévenu que sa seconde branche est morte.

Exercice 22.5: Réparer #143

Attribuer une polarité (exacte, `must`, `may`) à chacune des gardes `writer_phases`, `must_direct_write`, `name`, `stability is ["stable"]`, `stability is ["stable", "unknown"]`, `identity is ["fresh-every-render"]`. Appliquer la règle « une garde `may` à côté d'un `must_*` plafonne à `WARNING` » à la liste 22.46 ; que devient la sortie ?

Solution

`may`, `must`, exacte, `must`, `may` (la liste contient `unknown`), `must`. La règle de l'issue mêle `writer_phases` (`may`) et `must_direct_write` : elle serait plafonnée à `WARNING` et le chargement avvertirait ; les deux findings sortiraient en `WARNING`, comme la native `setter-in-render`. Le correctif est central (la polarité de la conjonction), pas propre à cette règle.

Exercice 22.6: Élargir sur la garde

Pourquoi `slot_ownership` *élargit*-il l'énumération de `render_setter_calls` au lieu d'être un simple filtre sur une énumération qui contiendrait toujours les setters étrangers ?

Solution

Si l'énumération contenait toujours les lignes étrangères, tout pack écrit avant ADR-30 lierait soudain de nouvelles lignes et émettrait de nouveaux findings sans que son auteur ait rien changé. En n'élargissant que pour une règle qui nomme la propriété, un pack publié garde exactement son comportement (ADR-30 §2, argument de séquençement d'ADR-27).

Exercice 22.7: Un schéma trop accueillant

Pourquoi `{"kind": "stability", "of": "d", "is": {"$param": "v"}}` est-il accepté par un éditeur qui lit `docs/schemas/pack.schema.json`, mais rejeté au chargement ? Quelle décision de conception le rejet protège-t-il ?

Solution

Le schéma est généré depuis les types `serde`, où toute liste de verdicts est un `PVal<Vec<...>` ; le validateur, lui, résout ces positions avec `expected = None`. Le rejet protège « *parameters are values, not structure* » : un paramètre de consommateur ne doit pas pouvoir changer la polarité d'une garde (par exemple ajouter `unknown` à une liste).

Sixième partie

L'outil et sa méthode

Chapitre 23

Projets, configuration, driver et CLI

Objectifs du chapitre.

- Savoir invoquer `reactant-analyzer` et lire sa sortie : sous-commandes, forme historique, codes de sortie, anatomie d’une ligne de finding.
- Connaître le fichier `reactant.config.json`, la précedence « drapeaux avant configuration » et la résolution des overrides (off, allowlist, plafond de sévérité *pin* $\sqcap$ *polarité*).
- Suivre `run_check`, la fonction unique qui compose tout un check pour l’hôte natif comme pour l’hôte WASM, et ses points de sortie.
- Comprendre comment un run choisit ses fichiers (`.gitignore`, exclusions) et comment il résout les imports d’un projet (Vite, Next.js, `tsconfig extends/references/paths/baseUrl`).
- Maîtriser le contrat central du sous-système : *pas de « clean bill » pour du code non lu* (blind spots, `-follow-imports`).
- Connaître les deux rendus (humain et JSON v2), leurs deux comptages, et le registre des résumés de hooks de bibliothèques.

Prérequis : chapitre 5 (parse oxc, détection des composants et des hooks, faits de module) ; chapitre 8 (inlining des hooks, résumés de bibliothèques) ; chapitre 16 (racines, analyse d’un programme) ; chapitre 17 (registre de règles, clamp, sceau de sévérité, définition 17.1 ; témoins, définition 17.2) ; chapitre 22 pour les packs ; chapitre 21 pour `server-component-hook`. La notion de soundness (définition 3.8) est supposée acquise.

Les chapitres précédents ont décrit le cœur de l’analyseur : l’IR, le moteur, les relations, les règles. Ce chapitre décrit sa *coquille* : tout ce qui se passe avant le parsing (quels fichiers lire, avec quelle résolution d’imports, sous quelle configuration) et tout ce qui se passe après les règles (tri, groupage, comptage, rendu, code de retour). Rien de ce code ne manipule un treillis. Il porte pourtant une part de la soundness du système, car un fichier qu’on oublie de lire est un faux négatif aussi sûrement qu’une fonction de transfert trop optimiste. Le fil rouge du chapitre est cette phrase de `src/driver/blind_spots.rs` : « « *No issues found* » *is a claim about the code, and the analyzer may only make it about code it actually read* ».

On procède du plus visible au plus enfoui : l’invocation telle que l’utilisateur la tape (section 23.1), la configuration (section 23.2), le driver qui compose le tout (section 23.3), puis les étapes du driver dans l’ordre où il les exécute : découverte des fichiers, projets et résolution d’imports, parse, blind spots, Next.js, rendu, et enfin le registre des résumés de bibliothèques.

23.1 L'invocation vue de l'utilisateur

23.1.1 Deux premiers runs

Le fichier `tests/fixtures/clean.tsx` contient quatre composants sans défaut. Le premier est un compteur ordinaire :

Extrait 23.1 – `tests/fixtures/clean.tsx`, l. 7–20 — un compteur sans défaut

```

7 function Counter() {
8   const [count, setCount] = useState(0);
9
10  useEffect(() => {
11    document.title = `Count: ${count}`;
12  }, [count]); // count is in deps ✓
13
14  return (
15    <div>
16      <p>{count}</p>
17      <button onClick={() => setCount((n) => n + 1)}></button>
18    </div>
19  );
20 }
```

Lancé sans sous-commande, l'analyseur répond :

```
$ reactant tests/fixtures/clean.tsx ; echo "exit=$?"
```

```
4 clean component(s) hidden, rerun with --show-clean
```

```
✓ 1 file(s) no issues found.
exit=0
```

Trois choses se lisent déjà. La forme `reactant <chemin>` est une forme *historique*, équivalente à `reactant check <chemin>`. Les composants propres sont masqués par défaut et seulement comptés. Et la ligne finale « ✓ ... no issues found », que nous appellerons le *clean bill*, est une affirmation que l'outil ne fait que sous conditions (section 23.7).

Le second fixture, `tests/fixtures/config_project/App.tsx`, contient deux défauts connus :

Extrait 23.2 – `tests/fixtures/config_project/App.tsx`, l. 7–14 — un setter en plein rendu et une dep manquante

```

7 export function App() {
8   const [n, setN] = useState(0);
9   setN(1);
10  useEffect(() => {
11    console.log(n);
12  }, []);
13  return <div>{n}</div>;
14 }
```

```
App (2 hooks) tests/fixtures/config_project/App.tsx
```

```
warn missing-deps [hook:1] var:n (line 10:2) `n` is used in this effect but not in its deps array, and it is
  ↳ recreated on every render
```

```
(1 trace step(s), rerun with --trace)
```

```
error setter-in-render [hook:0] (line 9:2) setter `setN` called directly in the render body, move this call into
  ↳ a useEffect or an event handler
```

```
(1 trace step(s), rerun with --trace)
```

```
! 1 error(s), 1 warning(s) across 1 file(s).
```

```
exit=1
```

Anatomie d’une ligne. L’en-tête de composant donne le nom affiché, le nombre de hooks *abaissés* (`useState`, `useEffect`; les handlers JSX comptent aussi, ce sont des `HookEntry::Handler`), et le fichier de définition. Chaque finding porte ensuite, dans cet ordre : une étiquette de sévérité de largeur fixe (`error`, `warn`, `info`) ; le nom de la règle ; le label du hook concerné [`hook:N`] ; la variable en cause `var:x` s’il y en a une ; la position (`line L:C`), avec une ligne comptée à partir de 1 et une colonne à partir de 0 ; le message ; enfin le nombre de pas de témoin (définition 17.2), visibles sous `-trace`. L’ordre des deux findings (`missing-deps` avant `setter-in-render`) n’est pas celui du fichier : c’est le tri total du registre de règles, par nom de règle d’abord (`src/rules/registry.rs`, l. 296–333).

Masquage. Trois classes de composants ne s’impriment pas (`src/driver/human.rs`, l. 113–148). Un composant *trivial* (zéro hook, zéro finding) n’est jamais imprimé, même sous `-show-clean`. Un composant *propre* (des hooks, aucun finding) est masqué et compté, sauf sous `-show-clean`. Un composant dont *tous* les findings répètent une localisation déjà imprimée est masqué et compté (section 23.9). Sous `-show-clean -info`, un composant propre imprime la liste des vérifications applicables qui ont tourné sans rien trouver :

```
Counter (3 hooks) tests/fixtures/clean.tsx ✓
  verified always-unstable-deps no deps array is defeated by an always-fresh reference
  verified conditional-hook all hooks run unconditionally, in a stable order
  verified derived-state no effect merely mirrors other state
  verified infinite-loop no effect diverges into an infinite render loop
  ...
```

(extrait ; la liste complète compte dix lignes pour `Counter`). Les trois hooks de `Counter` sont `useState`, `useEffect` et le handler `onClick`.

23.1.2 Sous-commandes et forme historique

Le binaire tient en cinq lignes : `main` appelle `cli::run` et sort avec le code rendu (`src/main.rs`). La bibliothèque (`src/lib.rs`) déclare dix modules publics, `config`, `domains`, `driver`, `engine`, `ir`, `lowering`, `project`, `registry`, `resolver` et `rules` ; le module `cli` n’en fait *pas* partie. `clap` et le parsing d’`argv` restent du côté binaire (ADR-16 §1), si bien que tout ce qui a un sens pour un autre hôte, le build WASM en particulier, vit dans la bibliothèque.

Extrait 23.3 – `src/cli/mod.rs`, l. 31–48 — la structure `clap` de la ligne de commande

```
31 #[derive(Parser)]
32 #[command(
33     name = "reactant",
34     version,
35     about = "Static analyzer for React hook bugs, based on abstract interpretation",
36     args_conflicts_with_subcommands = true,
37     // `help` is ours (`driver::run_help`), shared with the WASM frontend;
38     // clap keeps `-h`/`--help`, which stays the exhaustive flag reference.
39     disable_help_subcommand = true
40 )]
41 struct Cli {
42     #[command(subcommand)]
43     command: Option<Command>,
44
45     /// Legacy form: `reactant src/` behaves like `reactant check src/`.
46     #[command(flatten)]
47     check: check::CheckArgs,
```

```
48 }
```

Cli juxtapose une sous-commande optionnelle et les arguments de check aplatis (flatten). Les sous-commandes sont check, rules, explain <rule>, schemas [-out DIR] et help (src/cli/mod.rs, l. 50–76). L'aide help n'est pas celle de clap (disable_help_subcommand) : c'est driver::run_help, une page partagée avec l'hôte WASM ; -h/-help restent la référence exhaustive générée par clap. Le dispatch :

Extrait 23.4 – src/cli/mod.rs, l. 110–127 — le dispatch des sous-commandes

```
110 /// Parse arguments, dispatch, and return the process exit code.
111 pub fn run() -> i32 {
112     let cli = Cli::parse();
113     match cli.command {
114         Some(Command::Check(args)) => check::run(args),
115         Some(Command::Rules { config }) => rules_cmd::run(config.as_deref()),
116         Some(Command::Explain { rule, config }) => explain::run(&rule, config.as_deref()),
117         Some(Command::Schemas { out }) => schemas_cmd::run(out.as_deref()),
118         Some(Command::Help) => {
119             print!("{}", reactant::driver::run_help(color::enabled(false)));
120             EXIT_OK
121         }
122         // Bare `reactant` and the legacy `reactant src/` form are both a
123         // check; `check` itself defaults its paths to the current directory,
124         // so nothing here has to know about that default.
125         None => check::run(cli.check),
126     }
127 }
```

Le cas None couvre à la fois reactant nu et reactant src/ : check::run remplace une liste de chemins vide par "." (src/cli/check.rs, l. 99–102), si bien que reactant seul analyse le répertoire courant.

Piège

L'attribut args_conflicts_with_subcommands = true interdit de placer un drapeau *avant* une sous-commande. docs/usage.md dit que reactant -info check src/ est « rejected ». C'est vrai, mais pas par clap : clap lit la forme historique, avec check et src comme chemins, et c'est le driver qui refuse le premier argument.

```
$ reactant --info check src
[error] no command or path named `check`

reactant: static analyzer for React hook bugs, based on abstract interpretation
...
exit=2
```

Pour la même raison, un répertoire qui s'appelle littéralement rules ou check doit s'écrire ./rules. Sinon, la sous-commande l'emporte.

rules liste les noms de diagnostics que le registre peut émettre. Au commit étudié, on en compte 21 pour les natives : dix-neuf règles, dont deux émettent un second nom (cross-component-infinite-loop, cross-setter-in-render), suivies de ceux des packs chargés (src/driver/mod.rs, l. 624–642). explain <rule> imprime la documentation d'un nom : résumé, explication, Example:, Fix: et Options: (src/driver/mod.rs, l. 644–704). Un nom inconnu donne une suggestion calculée par sous-chaîne et par segment séparé par -. Ce n'est pas une distance d'édition :

```
$ reactant explain loop
[error] unknown rule `loop`
did you mean: cross-component-infinite-loop, infinite-loop?
```

```
exit=2
```

schemas émet les JSON Schemas de `pack.json` et de `reactant.config.json`, dérivés des types Rust par `schemars` (`src/cli/schemas_cmd.rs`, feature `cargo schema-gen`). `tests/schemas.rs` vérifie que les fichiers versionnés sous `docs/schemas/` sont identiques à cette sortie.

23.1.3 Codes de sortie

Trois constantes suffisent (`src/driver/mod.rs`, l. 36–38) : `EXIT_OK = 0`, `EXIT_FINDINGS = 1`, `EXIT_USAGE = 2`. Le code 1 dépend du seuil `-fail-on` (`error`, `warning` par défaut, ou `never`). Le code 2 couvre toute erreur d’usage ou d’entrée-sortie ; en voici quatre, observées :

```
$ reactant chekc                                     # stdout vide, aide sur stderr
[error] no command or path named `chekc`
$ reactant check does/not/exist
[error] no such file or directory: does/not/exist
$ reactant check tests/fixtures/vite_project --entry Bogus
[error] --entry: no component named `Bogus`
[error] name a component defined in the analysed files, `Name@path` to pick one of several with the same name
$ reactant check --bogus
error: unexpected argument '--bogus' found
```

La dernière erreur vient de clap, qui sort lui aussi avec le code 2 (observé ici). Les autres sont produites par le driver ou par le chargement de la configuration. Ce qui *ne change jamais* le code de sortie : les diagnostics INFO, les blind spots (section 23.7) et les findings masqués parce qu’ils vivent hors des chemins nommés (section 23.7.2).

Décision : ADR-016 — une CLI à sous-commandes, un pipeline unique

Avant ADR-16, la CLI était une commande plate dont `src/main.rs` dupliquait le pipeline d’analyse, et cela *sans* résolveur d’imports : le binaire ne voyait pas les hooks définis dans d’autres fichiers, alors que l’API plugin les voyait. L’ADR décide : (1) des sous-commandes, avec `check` par défaut et la forme historique conservée ; (2) un pipeline dédoublé, `lower_files` → `LoweredProgram` → `analyze_lowered`, ce qui donne enfin au binaire la résolution cross-file ; (3) des métadonnées de règles indexées par *nom de diagnostic*, parce que onze structures de règles émettaient treize noms ; (4) une sortie JSON par DTO dédiés, le type `Diagnostic` restant sans `serde` ; (5) le contrat 0/1/2 et `-fail-on`, un `tsconfig` manquant n’étant qu’un avertissement, jamais un code 2 ; (6) les types de projet (section 23.5). Alternative refusée pour (3) : étendre le trait `Rule`, ce qui aurait cassé les implémentations externes sans couvrir les noms doubles. Les points (3) et (4) ont depuis été absorbés par le `RuleRegistry` dynamique et par le driver partagé (ADR-22 §6 et §8).

23.2 La configuration

23.2.1 Un fichier découvert à la racine

Le fixture `tests/fixtures/config_discover` contient le même `App.tsx` que ci-dessus et ce fichier de configuration, commentaire compris :

Extrait 23.5 – `tests/fixtures/config_discover/reactant.config.json` (verbatim)

```
1 {
2   // Discovered at the project root without --config; proves discovery+merge.
3   "failOn": "never"
4 }
```

reactant check tests/fixtures/config_discover imprime les deux mêmes findings, mais sort avec le code 0 : la clé failOn a comblé l'absence du drapeau -fail-on. Le fichier est du JSONC, c'est-à-dire du JSON avec commentaires et virgules finales, comme les tsconfig.json. Il passe par le même strip_jsonc que ces derniers (section 23.5.3).

Extrait 23.6 – src/config.rs, l. 26–67 — le type de la configuration

```

26 #[derive(Debug, Default, Deserialize, PartialEq)]
27 #[cfg_attr(feature = "schema-gen", derive(JsonSchema))]
28 #[serde(deny_unknown_fields, rename_all = "camelCase")]
29 pub struct ReactantConfig {
30     /// Editor-facing schema URL; not interpreted.
31     #[serde(rename = "$schema", default)]
32     pub schema: Option<String>,
33     /// Rule packs to load: npm package names or relative paths, in order
34     /// (ADR-022 §8: pack order is output order). Consumed by the pack loader.
35     #[serde(default)]
36     pub packs: Vec<String>,
37     /// Per-diagnostic overrides: `off` | `<severity>` | { severity?, options? }`.
38     #[serde(default)]
39     pub rules: BTreeMap<String, RuleSetting>,
40
41     // -- `check` flag equivalents (CLI takes precedence, ADR-022 §5) -----
42     #[serde(default)]
43     pub entry: Vec<String>,
44     #[serde(default)]
45     pub all_roots: Option<bool>,
46     #[serde(default)]
47     pub fail_on: Option<FailOnConfig>,
48     #[serde(default)]
49     pub project: Option<ProjectConfig>,
50     #[serde(default)]
51     pub format: Option<FormatConfig>,
52     #[serde(default)]
53     pub info: Option<bool>,
54     #[serde(default)]
55     pub show_clean: Option<bool>,
56     #[serde(default)]
57     pub trace: Option<bool>,
58     /// Directory names never walked, matched at any depth. Non-empty replaces
59     /// the default policy (`.gitignore` if the tree has one, else
60     /// `dist`/`build`/`.next`); `node_modules` is excluded regardless.
61     #[serde(default)]
62     pub exclude_dirs: Vec<String>,
63     /// Analyze the files the named paths import, transitively. The report
64     /// still covers only the named paths. Not a speed optimization.
65     #[serde(default)]
66     pub follow_imports: Option<bool>,
67 }

```

Chaque champ a un rôle précis :

- schema (\$schema en JSON) sert à l'éditeur et n'est pas interprété ;
- packs donne la liste ordonnée des packs de règles déclaratives (chapitre 22), noms npm ou chemins ; l'ordre des packs est l'ordre de sortie (ADR-22 §8) ;
- rules associe un nom de diagnostic à un réglage (voir plus bas). C'est une BTreeMap, pour une itération déterministe ;
- les champs suivants sont les équivalents des drapeaux de check : entry, allRoots, failOn, project, format, info, showClean, trace, excludeDirs, followImports (rename_all = "camelCase").

`-verbose`, `-rule`, `-ignore-rule`, `-rule-option` et `-no-color` n'ont pas d'équivalent de configuration ; les options de règles passent par `rules`.

Validation bruyante. L'en-tête du module pose le principe : « a present but broken config is always an error, never silently degraded to defaults (silent config loss would silently drop packs and overrides) » (`src/config.rs`, l. 1–10). L'attribut `deny_unknown_fields` en est la première application :

```
$ reactant check tests/fixtures/config_project --config tests/fixtures/config_project/badkey.json
[error] invalid tests/fixtures/config_project/badkey.json: unknown field `rulez`, expected one of `schema`, `packs`, `
  ↳ rules`, `entry`, `allRoots`, `failOn`, `project`, `format`, `info`, `showClean`, `trace`, `excludeDirs`, `
  ↳ followImports` at line 2 column 9
exit=2
```

La position « line 2 column 9 » est exacte parce que `strip_jsonc` remplace chaque commentaire en conservant ses retours à la ligne.

Trois formes de réglage. Une entrée de règles s'écrit `"off"`, `"<sévérité>"` ou `{ "severity": ..., "options": {...} }`. Les trois se normalisent dans un seul type :

Extrait 23.7 – `src/config.rs`, l. 96–104 — un réglage de règle normalisé

```
96 /// One `rules` entry, normalized from its three JSON forms:
97 /// `off`, ``, `{ "severity": ..., "options": {...} }`.
98 #[derive(Debug, Clone, Default, PartialEq)]
99 pub struct RuleSetting {
100     pub off: bool,
101     /// Severity ceiling (ADR-022 §3: `pin ⊔ polarity` – downgrade-only).
102     pub severity: Option<Severity>,
103     pub options: serde_json::Map<String, serde_json::Value>,
104 }
```

La désérialisation est écrite à la main (`src/config.rs`, l. 124–177), avec un `Visitor` qui accepte une chaîne ou un objet. Le commentaire donne la raison : « an untagged enum would report "data did not match any variant", useless for the loud-validation contract — every message must say what was expected and what was found ». Le résultat, observé :

```
[error] invalid tests/fixtures/config_project/badsetting.json: unknown rule setting `warn`, expected "off", "error", "
  ↳ warning" or "info" at line 2 column 35
```

23.2.2 Chargement : configuration, registre, packs

`check::run` commence par déterminer une *racine* : le premier argument qui est un répertoire, sinon `"."` (`src/cli/check.rs`, l. 104–112). Le driver recalcule la même racine pour détecter le projet (section 23.3). Puis `load_config_and_registry` (`src/cli/config_load.rs`, l. 16–91) procède en quatre temps :

1. Un `-config <p>` explicite est chargé, et son absence est une erreur d'usage (« An explicit `-config` must exist »). Sinon `config::discover(root)` cherche *seulement* `<root>/reactant.config.json`, sans remonter vers les ancêtres ni se replier sur le répertoire courant (`src/config.rs`, l. 334–340). Sinon, la configuration par défaut s'applique.
2. Les chemins de packs relatifs se résolvent contre le répertoire du fichier de configuration (« the `tsconfig-extends` contract »), si bien qu'une configuration veut dire la même chose depuis n'importe quel répertoire courant.

3. Le registre part des règles natives, `RuleRegistry::natives()`. Les options non vides de règles sont transmises au chargeur de packs, indexées par identifiant complet.
4. Les packs sont chargés dans l'ordre de la configuration :

Extrait 23.8 – `src/cli/config_load.rs`, l. 55–91 — chargement des packs, chaque échec est bruyant

```

55 // Packs in config order, rules in pack order (ADR-022 §8). Every failure
56 // is loud: ignoring a configured pack would run fewer rules than the
57 // config asks for – the config-level analogue of a false negative.
58 for spec in &cfg.packs {
59     let path = match resolve_pack_path(spec, &config_dir) {
60         Ok(p) => p,
61         Err(msg) => {
62             eprintln!("[error] pack `{spec}`: {msg}");
63             return Err(EXIT_USAGE);
64         }
65     };
66     let json = match std::fs::read_to_string(&path) {
67         Ok(s) => s,
68         Err(e) => {
69             eprintln!("[error] pack `{spec}`: cannot read {}: {e}", path.display());
70             return Err(EXIT_USAGE);
71         }
72     };
73     let load = match declarative::load_pack(&json, &options) {
74         Ok(l) => l,
75         Err(e) => {
76             eprintln!("[error] pack `{spec}` ({}): {e}", path.display());
77             return Err(EXIT_USAGE);
78         }
79     };
80     for w in &load.warnings {
81         eprintln!("[warn] rule `{}`: {}", w.rule, w.message);
82     }
83     for rule in load.rules {
84         if let Err(e) = registry.register(rule.rule, rule.doc) {
85             eprintln!("[error] pack `{spec}`: {e}");
86             return Err(EXIT_USAGE);
87         }
88     }
89 }
90 Ok((cfg, registry))
91 }

```

Le commentaire des lignes 55–57 formule une idée qui reviendra souvent : ignorer un pack configuré exécuterait moins de règles que la configuration ne le demande, « the config-level analogue of a false negative ». La résolution d'une spécification de pack (`src/cli/config_load.rs`, l. 93–128) est volontairement simple. Une spécification qui commence par `.` ou `/`, ou qui finit par `.json`, est un chemin. Toute autre est un nom npm, cherché dans `<base>/node_modules/<nom>/`, où l'on prend le champ `"reactant"` du package.json, à défaut `pack.json`. Il n'y a pas d'algorithme de résolution Node complet. Ce module lit le disque avec `std::fs` et non à travers la couture `FileSystem` : c'est du code d'hôte natif, et l'hôte WASM résout les packs de son côté (chapitre 24).

```

[error] pack `@team/react-rules`: not installed: tests/fixtures/config_project/node_modules/@team/react-rules does not
    ↪ exist
exit=2

```

`rules` et `explain` réutilisent la même fonction avec `root = "."` (`src/cli/rules_cmd.rs`, `src/cli/explain.rs`) : leur configuration est celle du répertoire courant, alors que celle de `check`

est celle du premier répertoire argument.

23.2.3 Précédence : les drapeaux comblent, la configuration complète

Un même réglage peut venir de deux sources, par exemple `-fail-on` et `"failOn"`. La règle d'ADR-22 §5 est que les drapeaux l'emportent. Pour que la configuration puisse combler un drapeau *absent*, il faut pouvoir distinguer « drapeau absent » de « drapeau à sa valeur par défaut ». C'est pourquoi les trois options énumérées de `CheckArgs` n'ont pas de valeur par défaut clap :

Extrait 23.9 – `src/cli/check.rs`, l. 62–75 — trois Option sans défaut clap

```
62  /// Output format (default: human)
63  // No clap default_value on this and the two Options below: a default
64  // would always yield `Some`, making "flag absent" indistinguishable from
65  // "flag = default" and killing config precedence (ADR-022 §5).
66  #[arg(long, value_enum)]
67  pub format: Option<OutputFormat>,
68
69  /// Findings severity that makes the exit code non-zero (default: warning)
70  #[arg(long, value_enum)]
71  pub fail_on: Option<FailOn>,
72
73  /// Project kind: auto-detect (default), force vite/next conventions, or plain walk
74  #[arg(long, value_enum)]
75  pub project: Option<ProjectMode>,
```

Invariant 23.1: Absent n'est pas défaut

Tant que la fusion avec la configuration n'a pas eu lieu, un drapeau non passé est représenté par `None` (options énumérées) ou par une liste vide (`entry`, `exclude_dir`). Les défauts finaux (`Human`, `Warning`, `Auto`) ne sont appliqués qu'*après* la fusion (`src/cli/check.rs`, l. 174–188). Un `default_value` clap produirait toujours `Some` et rendrait la configuration inopérante sur ces trois clés.

La fusion elle-même vit dans la bibliothèque, sur une forme neutre des drapeaux, `CheckArgsPartial` (`src/config.rs`, l. 231–248). Chaque hôte y projette son `argv`, fusionne, puis en tire des `CheckOptions`. Il y a donc un seul mécanisme de précédence pour deux hôtes.

Extrait 23.10 – `src/config.rs`, l. 250–269 — la fusion drapeaux / configuration

```
250 impl CheckArgsPartial {
251     /// Config values fill only the holes the flags left. Boolean flags are
252     /// turn-on-only – no `--no-info` exists to detect.
253     pub fn merge(&mut self, cfg: &ReactantConfig) {
254         if self.entry.is_empty() {
255             self.entry = cfg.entry.clone();
256         }
257         if self.exclude_dirs.is_empty() {
258             self.exclude_dirs = cfg.exclude_dirs.clone();
259         }
260         self.all_roots |= cfg.all_roots.unwrap_or(false);
261         self.info |= cfg.info.unwrap_or(false);
262         self.show_clean |= cfg.show_clean.unwrap_or(false);
263         self.trace |= cfg.trace.unwrap_or(false);
264         self.follow_imports |= cfg.follow_imports.unwrap_or(false);
265         self.fail_on = self.fail_on.or(cfg.fail_on);
266         self.format = self.format.or(cfg.format);
267         self.project = self.project.or(cfg.project);
```



```
268 }
269 }
```

Régime	Champs	Règle	Conséquence
liste	entry, exclude_dirs	drapeau non vide remplace	pas de concaténation
booléen	all_roots, info, show_clean, trace, follow_imports	OU logique	la config ne peut pas être éteinte en CLI
option	fail_on, format, project	Option::or, drapeau d'abord	défaut appliqué après
sans équivalent	verbose, rule, ignore_rule, rule_option, no_color	drapeau seul	

TABLE 23.1 – Les trois régimes de précedence de CheckArgsPartial::merge.

Tableau 23.1 résume les trois régimes. Pour les listes, un test de `tests/discovery_exclusions.rs` vérifie que `-exclude-dir` remplace `excludeDirs` en entier :
`the_flag_beats_the_config_exclude_dirs`.

Piège

Les booléens sont « turn-on-only » : aucun `-no-info` n'existe, et une configuration qui pose `"info": true` ne peut pas être contredite depuis la ligne de commande. Pour un run ponctuel sans `-info`, il faut passer une autre configuration avec `-config`.

23.2.4 Overrides de règles : off, allowlist, options

Les réglages de règles et les filtres `-rule/-ignore-rule` se combinent en un seul ensemble d'overrides :

Extrait 23.11 – `src/config.rs`, l. 271–293 — la résolution des overrides

```
271 /// Resolve the config `rules` map and the CLI filters into one override set
272 /// (ADR-022 §5): `--ignore-rule` always denies; an explicit `--rule X`
273 /// resurrects a config-`"off"` X; severity pins come from config only.
274 pub fn resolve_overrides(
275     cfg: &ReactantConfig,
276     rule: &[String],
277     ignore_rule: &[String],
278 ) -> crate::rules::RuleOverrides {
279     let mut overrides = crate::rules::RuleOverrides::default();
280     for (name, setting) in &cfg.rules {
281         let entry = overrides.entries.entry(name.clone()).or_default();
282         entry.off = setting.off && !rule.contains(name);
283         entry.ceiling = setting.severity;
284         entry.options = setting.options.clone();
285     }
286     for name in ignore_rule {
287         overrides.entries.entry(name.clone()).or_default().off = true;
288     }
289     if !rule.is_empty() {
290         overrides.allow = Some(rule.iter().cloned().collect());
291     }
292     overrides
293 }
```

Trois phrases résument la fonction. `-ignore-rule` refuse toujours. Un `-rule X` explicite *ressuscite* un `X` que la configuration avait mis à `"off"`. Et dès qu'un `-rule` est présent, il définit une allowlist. Sur le fixture, avec `off.json = {"rules": {"missing-deps": "off"}}`, on observe :

config	-rule	-ignore-rule	missing-deps	setter-in-render
"off"	—	—	masqué	visible (exit 1)
"off"	missing-deps	—	visible (résurrection)	masqué (allow-list)
"off"	missing-deps	missing-deps	masqué	masqué; « ✓ no issues found »

TABLE 23.2 – Table de vérité observée des overrides sur `tests/fixtures/config_project`; la colonne « config » est la valeur de `rules.missing-deps` dans `off.json`.

La dernière ligne de la tableau 23.2 mérite attention. Le composant `App` n’a plus de `finding` visible : il est compté comme « clean component » et le `clean bill` est imprimé. Cette ligne est une affirmation sur les règles *demandées*, pas sur le code. Le filtrage agit après les règles, par `RuleRegistry::visible` dans `check_component` (`src/rules/registry.rs`, l. 293–294). Il n’ôte donc rien au calcul, seulement à l’affichage et au comptage.

Options de règles. `-rule-option <rule>:<key>=<value>` est découpé au premier : puis au premier `=`. La valeur est lue comme du JSON si elle en est (3, true), comme une chaîne sinon (`src/config.rs`, l. 295–319). `apply_rule_options` la pose ensuite par-dessus la valeur de configuration de même clé (`src/config.rs`, l. 321–332). La validation n’est pas faite ici mais dans `RuleRegistry::set_overrides` (`src/rules/registry.rs`, l. 160–200) : nom de diagnostic connu, règle qui accepte des options, clé déclarée, type et bornes. Un seul échec suffit pour sortir avec le code 2 :

```
[error] rule `missing-deps` is built-in and declares no options
[error] option `minDepth` of rule `state-lifted-too-high` must be an integer between 1 and 64
[error] `--rule-option badformat`: expected `

```

Ce sont quatre runs distincts, dans l’ordre :

```
-rule-option missing-deps:foo=1;
-rule-option state-lifted-too-high:minDepth=abc;
-rule-option badformat;
-config vers badrule.json, qui nomme no-such-rule. Un nom inconnu dans rules, -rule ou
-ignore-rule est refusé, parce qu’une faute de frappe dans un "off" ne ferait rien en silence, et
une faute dans un -rule masquerait tout en silence.
```

23.2.5 Sévérité : un plafond, jamais un plancher

Le champ `severity` d’un réglage n’est pas une sévérité imposée mais un *plafond*, que la configuration seule peut poser. `Diagnostic::clamp` ne fait que baisser :

Extrait 23.12 – `src/rules/api/diagnostic.rs`, l. 104–114 — le `clamp`, qui ne fait que baisser

```
104    /// Consumer override (ADR-022 §3): lower this finding to `ceiling` when its
105    /// constructed severity exceeds it. Downgrade-only, hence sound to expose:
106    /// Error is constructible only through [Diagnostic::error`], so the
107    /// constructed severity IS the polarity ceiling and `pin` polarity`
108    /// reduces to a min – an upgrade request is a structural no-op.
109    pub fn clamp(mut self, ceiling: Severity) -> Self {
110        if ceiling.rank() < self.severity.rank() {
111            self.severity = ceiling;
112        }
113        self
```

```
114 }
```

Observons les deux sens. Avec `downgrade.json` (`"setter-in-render": "warning"`) et `-fail-on-error`, les deux lignes deviennent `warn`, le résumé est « ! 2 warning(s) across 1 file(s) » et le code de sortie vaut 0. Avec `upgrade.json` (`"missing-deps": "error"`), le JSON garde `"severity": "warning"` : la demande de promotion est un no-op structurel (`test config_upgrade_is_a_no_op, tests/config.rs`).

Décision : ADR-022 §3 et §5 — $pin \sqcap polarité$, pour les natives comme pour les packs

Une sévérité `ERROR` est une preuve (définition 17.1) : elle ne se construit que par `Diagnostic::error`, derrière le sceau de la couche règles (chapitre 17). La sévérité effective d'un finding est donc le meet du plafond demandé (pin) et de la sévérité construite (la $polarité$), pris dans l'ordre `INFO  $\sqsubseteq$  WARNING  $\sqsubseteq$  ERROR`. Baisser un `ERROR` natif est permis : le consommateur en prend le risque. Promouvoir un finding de polarité `may` en `ERROR` ne peut pas arriver, et c'est silencieux et structurel. `"off"` désactive. La même règle vaut pour les natives et pour les packs, ce qui évite un second régime (chapitre 22). L'alternative, une sévérité libre dans la configuration, aurait permis à un fichier JSON de fabriquer des `ERROR` que le moteur n'a pas prouvés, contre l'invariant des niveaux de diagnostic.

Remarque

Deux ordres coexistent sur `Severity` (`src/rules/api/diagnostic.rs`, l. 41–52). Le *rang* (`ERROR = 2 > WARNING = 1 > INFO = 0`) sert au clamp. L'ordre de l'énumération (`ERROR < WARNING < INFO`) sert au tri déterministe de la sortie. Les confondre inverserait l'un des deux.

23.3 Le driver : run_check comme composition

23.3.1 Hôte et driver

Une fois la configuration fusionnée et les overrides installés dans le registre, `check::run` n'a plus qu'un appel à faire :

Extrait 23.13 – `src/cli/check.rs`, l. 192–202 — l'hôte natif appelle le driver, imprime, sort

```
192 let out = driver::run_check(
193     Arc::new(OsFileSystem),
194     &args.paths,
195     &registry,
196     &opts,
197     &|p| super::display_relative(p),
198 );
199 eprint!("{}", out.stderr);
200 print!("{}", out.stdout);
201 out.exit_code
202 }
```

Tout le reste est dans `driver::run_check`, dont l'en-tête de module définit la frontière :

Extrait 23.14 – `src/driver/mod.rs`, l. 1–10 — le contrat du driver

```
1 /// The shared frontend driver (ADR-022 §6): the whole `check` composition -
2 /// project context, discovery, lowering, analysis, rule pass, counting,
3 /// exit policy, rendering - behind one function used verbatim by the native
4 /// CLI and the WASM entry point, so their behavior cannot fork.
5 ///
```

```

6  /// The host owns everything OS-flavored: argv parsing, config discovery and
7  /// override resolution (done *before* calling in, on the registry),
8  /// tty/`NO_COLOR` sniffing (resolved into [`CheckOptions::color`]), stream
9  /// writing and the process exit. The driver only reads through the
10 /// [`FileSystem`] seam and returns buffered streams.

```

Définition 23.1: Hôte, driver

Le *driver* est la fonction `run_check` et ses renderers : la composition complète d'un check, du contexte de projet au texte rendu. Il ne lit le monde qu'à travers la couture `FileSystem` et rend des flux tamponnés. Un *hôte* est un frontend (le binaire natif, ou le wrapper WASM/JS) : il parse `argv`, découvre et fusionne la configuration, résout les overrides sur le registre, décide des couleurs (tty, `NO_COLOR`, `-no-color`), écrit les flux et termine le processus.

Extrait 23.15 – `src/driver/mod.rs`, l. 61–90 — options résolues et sortie tamponnée

```

61 /// Resolved `check` options – CLI flags and config values already merged by
62 /// the host (flags win, ADR-022 §5).
63 pub struct CheckOptions {
64     pub info: bool,
65     pub show_clean: bool,
66     pub trace: bool,
67     pub verbose: bool,
68     pub all_roots: bool,
69     pub entry: Vec<String>,
70     /// Directory names never walked (`--exclude-dir`). Non-empty replaces the
71     /// default policy; see [`crate::resolver::EXCLUDED_DIRS`].
72     pub exclude_dirs: Vec<String>,
73     /// `--follow-imports`: analyse the files the named ones import, instead of
74     /// treating discovery as the sole producer of lowered files (#138). The
75     /// report still covers only the components defined in the named paths.
76     pub follow_imports: bool,
77     pub format: ReportFormat,
78     pub fail_on: FailOn,
79     pub project: ProjectOverride,
80     /// Resolved by the host (tty + NO_COLOR + --no-color); the driver never
81     /// sniffs the environment.
82     pub color: bool,
83 }
84
85 /// Buffered run result: the host writes the streams and exits.
86 pub struct CheckOutput {
87     pub stdout: String,
88     pub stderr: String,
89     pub exit_code: i32,
90 }

```

La signature est `run_check(fs: Arc<dyn FileSystem>, paths: &[String], registry: &RuleRegistry, opts: &CheckOptions, display: &dyn Fn(&Path) -> String) -> CheckOutput` (`src/driver/mod.rs`, l. 102–112). Le registre arrive prêt, avec les natives, les packs et les overrides installés. Quelques champs appellent un commentaire. `color` est déjà résolu par l'hôte ; côté natif, c'est `src/cli/color.rs` (couleurs actives si et seulement s'il n'y a pas de `-no-color`, si `NO_COLOR` est absent ou vide, et si `stdout` est un terminal). Le driver ne « renifle » jamais l'environnement. `display` rend un chemin pour l'affichage : la CLI passe un relativiseur au répertoire courant, `display_relative` (`src/cli/mod.rs`, l. 21–29), et WASM passe l'identité. `CheckOutput` est tamponné, et l'on sait ce que porte chaque flux : `stdout` contient le rapport, humain ou *un seul* document JSON ; `stderr` contient les avertissements, les erreurs de parse et

le bavardage `-verbose`.

Ce découpage a une conséquence testable. Le crate `WASM` appelle le même `run_check` avec un `MemFileSystem` (chapitre 24), et `tests/memfs_parity.rs` exige une sortie octet par octet identique entre `OsFileSystem` et `MemFileSystem` sur les fixtures `vite_project`, `next_project` et `cross_file_hook`. La couture elle-même est minuscule :

Extrait 23.16 – `src/resolver/filesystem.rs`, l. 12–19 — la couture en lecture seule

```
12 pub trait FileSystem: Send + Sync {
13     fn read_to_string(&self, path: &Path) -> Result<String, String>;
14     fn is_file(&self, path: &Path) -> bool;
15     fn is_dir(&self, path: &Path) -> bool;
16     /// Immediate children of `dir`; empty when unreadable or absent.
17     /// Order is not part of the contract – callers sort.
18     fn read_dir(&self, dir: &Path) -> Vec<PathBuf>;
19 }
```

`OsFileSystem` relaie simplement `std::fs`. `MemFileSystem` est une `BTreeMap<PathBuf, String>` dont les clés sont normalisées lexicalement à l’insertion. Un répertoire y « existe » si et seulement si une clé lui est strictement préfixée, composante par composante (`src/resolver/filesystem.rs`, l. 75–82). Découverte, détection de projet, chaînes `tsconfig` et résolution d’alias tournent ainsi *dans* le moteur au-dessus de la table, et aucune sémantique de l’analyseur n’est réimplémentée côté hôte (`src/resolver/filesystem.rs`, l. 1–7).

23.3.2 Le flot de `run_check`

La figure 23.1 montre l’ordre exact des étapes (`src/driver/mod.rs`, l. 106–508), avec les points de sortie `EXIT_USAGE` à droite et, à gauche, les étapes qui alimentent la liste des blind spots.

1. Commande mal tapée. C’est la première garde :

Extrait 23.17 – `src/driver/mod.rs`, l. 115–129 — un premier argument qui ne nomme rien

```
115 // -- Mistyped command -----
116 // A first argument naming nothing on disk is a mistyped path when it looks
117 // like one, and a mistyped command otherwise (`reactant chekc`), which the
118 // command list answers better than an IO error. Only the first argument:
119 // that is the only position a command can occupy.
120 if let Some(first) = paths.first()
121     && !first.contains(['/', '\\', '.'])
122     && !fs.is_dir(Path::new(first))
123     && !fs.is_file(Path::new(first))
124 {
125     let _ = writeln!(err, "[error] no command or path named `{first}`");
126     let _ = writeln!(err);
127     err.push_str(&run_help(opts.color));
128     return CheckOutput::usage(err);
129 }
```

Un premier argument sans `/`, sans `\` et sans `.`, qui n’existe pas sur le disque, ressemble à une commande. La page d’aide lui répond mieux qu’une erreur d’entrée-sortie (test `an_unknown_command_prints_the_help_not_an_io_error`, `tests/cli.rs`). Seul le premier argument est examiné, parce que c’est la seule position qu’une commande puisse occuper. Un argument en forme de chemin garde son « no such file or directory ».

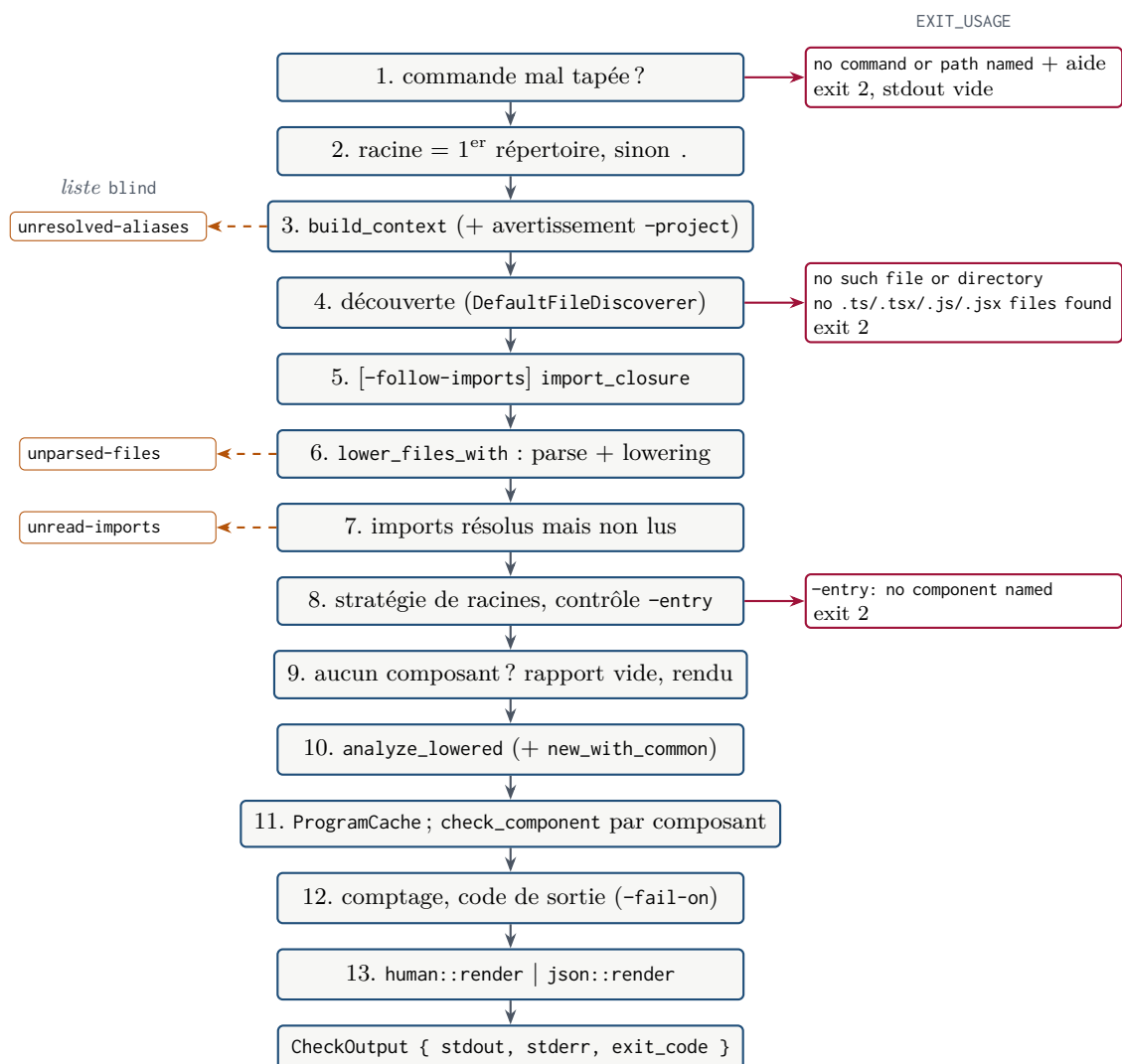


FIGURE 23.1 – Le flot de `run_check`. À droite, en rouge, les trois points de sortie `EXIT_USAGE` du driver (les autres codes 2 naissent avant, dans l'hôte : clap, configuration, overrides). À gauche, en pointillés, les trois étapes qui alimentent la liste des blind spots.

2–3. Racine et contexte. La racine de projet est le premier argument qui est un répertoire à travers la couture, sinon `.` (`src/driver/mod.rs`, l. 131–139). C’est la même règle que dans `check.rs`, mais évaluée par `fs.is_dir`. Puis `build_context` calcule le type de projet, la racine de découverte et le résolveur (section 23.5). Un `alias_warning` est imprimé et devient un blind spot. Sous `-verbose`, une ligne résume le contexte.

4–7. Fichiers. La découverte (section 23.4), la fermeture optionnelle des imports (section 23.7.2), le lowering et le canal d’erreurs (section 23.6), puis le calcul des imports non lus (section 23.7).

8. Racines. La stratégie est `Explicit` si `-entry` est non vide (noms nettoyés par `trim`), sinon `AllComponents` sous `-all-roots`, sinon `Heuristic` (`src/driver/mod.rs`, l. 282–288; la notion de racine est au chapitre 16). Une table « origine → (fichier, nombre de hooks) » est construite *avant* l’analyse, qui consomme les composants. Elle est indexée par l’origine (fichier, nom) du composant et jamais par son nom affiché, dont l’orthographe dépend des autres fichiers présents dans le run (#7, ADR-40). Puis vient le contrôle de `-entry` :

Extrait 23.18 – `src/driver/mod.rs`, l. 307–321 — un `-entry` qui ne sélectionne rien est une erreur

```
307 // An `--entry` name that matches nothing selects no root, which silently
308 // collapses the run to intra-component analysis – every cross-component
309 // finding lost to a typo, with an otherwise clean report. Fail instead.
310 let unmatched = strategy.unmatched(&temp_registry);
311 if !unmatched.is_empty() {
312     for name in &unmatched {
313         let _ = writeln!(err, "[error] --entry: no component named `{name}`");
314     }
315     let _ = writeln!(
316         err,
317         "[error] name a component defined in the analysed files, `Name@path` \
318         to pick one of several with the same name"
319     );
320     return CheckOutput::usage(err);
321 }
```

Le commentaire donne le raisonnement : un nom qui ne correspond à rien ne sélectionne aucune racine et réduit silencieusement le run à l’analyse intra-composant. Chaque finding inter-composants serait alors perdu sur une faute de frappe, avec un rapport par ailleurs propre.

9–10. Analyse. S’il n’y a aucun composant, un rapport vide est rendu tout de suite, avec le code 0. Sinon :

Extrait 23.19 – `src/driver/mod.rs`, l. 366–374 — le seul branchement de `new_with_common` dans un run réel

```
366 // Ship with the common library-hook summaries (TanStack Query, React Router)
367 // enabled: they resolve these hooks to T as *known* hooks, so a real
368 // `useQuery`/`useNavigate` no longer emits `analysis-limit/unknown-hook`
369 // noise. `Config::default()` stays empty so unit tests keep their baselines.
370 let config = Config {
371     summary_registry: crate::registry::SummaryRegistry::new_with_common(),
372     ..Config::default()
373 };
374 let program_result = analyze_lowered(lowered, strategy, config);
```

Le driver est le *seul* endroit du dépôt où le registre commun de résumés de bibliothèques est installé dans un run réel (section 23.10). `Config::default()` garde un registre vide pour que les tests unitaires conservent leurs lignes de base. Sous `-verbose`, le driver imprime ensuite `components_analyzed` et les compteurs `cache hits/misses`. Il s'agit du cache inter-composants du moteur (chapitre 16), qui indexe le résultat d'un enfant par ses props abstraites. Ce n'est pas un cache disque : chaque invocation repart de zéro.

11. Règles. Les composants sont parcourus dans l'ordre de leur nom affiché. Un *seul* `ProgramCache` sert tout le run : les règles qui ont besoin de structures programme, comme le graphe de churn de `infinite-loop` (chapitre 15), le construisent une fois au lieu d'une fois par composant, ce qui rendait la phase des règles quadratique (#86, `src/driver/mod.rs`, l. 407–410). Pour chaque composant, `registry.check_component` fait tout le passage des règles : `check`, `repli` sur `safe_check`, `clamp`, `filtres off/allow`, `tri total` (chapitre 17). Le driver ne garde que le filtre d'affichage des INFO (`-info`) et la mise à l'écart des composants définis hors des chemins nommés (section 23.7.2).

Extrait 23.20 – `src/driver/mod.rs`, l. 480–484 — la politique de sortie

12. Code de sortie

```
480 let exit_code = match opts.fail_on {
481     FailOn::Error if errors > 0 => EXIT_FINDINGS,
482     FailOn::Warning if errors + warnings > 0 => EXIT_FINDINGS,
483     _ => EXIT_OK,
484 };
```

Invariant 23.2: Ce qui fait le code de sortie

Le code 1 est décidé par les seuls comptes `errors` et `warnings` des composants *rapportés*, ligne par ligne, après `clamp` et `filtres`. Les INFO n'y entrent jamais, les blind spots non plus (`test a_blind_spot_is_not_a_finding`, `tests/blind_spots.rs`), ni les findings retenus hors des chemins nommés. Les comptes sont faits *par ligne composant*, sans groupage par localisation (section 23.9).

Déterminisme. Deux runs consécutifs doivent être identiques octet par octet (`tests consecutive_runs_are_byte_identical` et sa variante `-trace`, `tests/cli.rs`; variante avec configuration dans `tests/config.rs`). Plusieurs dispositifs y concourent : le tri de `discover` (l'ordre de `read_dir` ne fait pas partie du contrat), les `BTreeMap/BTreeSet` de la configuration et des imports non lus, le tri des arêtes vers les utilitaires, le tri des composants par nom affiché, le tri total des diagnostics dans `check_component` et le groupage par localisation dans l'ordre du rapport. Les notes d'un diagnostic ne sont pas triées : c'est leur production qui doit être déterministe, d'où le test `-trace`.

23.4 La découverte des fichiers

23.4.1 Un exemple : `scripts/build` n'est pas du `build`

Construisons sous `/tmp/ch23/gi` un petit dépôt : un `package.json` vide, un `.gitignore` de deux lignes (`dist` et `/src/generated`), le même composant `Looper` à boucle infinie dans `scripts/build/Tool.tsx` et dans `dist/Out.tsx`, un module `src/generated/api.ts`, et `src/Uses.tsx` qui l'importe par `./generated/api`. Depuis `/tmp/ch23` :


```
$ reactant gi
  Looper (2 hooks) gi/scripts/build/Tool.tsx
    warn infinite-loop [hook:0] (line 4:2) this effect keeps pushing state `n` (its deps do not provably gate it, so
      ↳ the effect can re-run every render) to new values on every run. Potential infinite render loop
        (2 trace step(s), rerun with --trace)

! 1 warning(s) across 2 file(s).
not analyzed:
  • 1 imported file(s) resolved outside the analysed set and were never read. Pass them on the command line to analyse
    ↳ them (gi/src/generated/api.ts)
exit=1
```

`scripts/build/` est parcouru, parce que le `.gitignore` ne mentionne pas `build`. `dist/` est sauté, par le motif non ancré `dist`. `src/generated/` est sauté lui aussi, par le motif ancré `/src/generated`. Mais comme `Uses.tsx` importe un fichier de ce répertoire, le run le *nomme* dans un blind spot et retient le clean bill (section 23.7). `Uses`, sans hook ni finding, est trivial et ne s'imprime pas. Avec `-exclude-dir vendor`, la liste configurée remplace *toute* la politique par défaut :

```
$ reactant gi --exclude-dir vendor
  Looper@gi/dist/Out.tsx (2 hooks) gi/dist/Out.tsx
    warn infinite-loop [hook:0] (line 4:2) ...
  Looper@gi/scripts/build/Tool.tsx (2 hooks) gi/scripts/build/Tool.tsx
    warn infinite-loop [hook:0] (line 4:2) ...

! 2 warning(s) across 4 file(s).
```

(messages élidés). Quatre fichiers sont lus et le blind spot disparaît. Les deux `Looper` homonymes sont désambiguïsés sous la forme `Name@path`.

23.4.2 Quels fichiers, quels répertoires

Un fichier est *source* s'il a une extension de `SOURCE_EXTENSIONS = ["ts", "tsx", "js", "jsx"]` et n'est ni un `*.d.ts`, ni un `*.test.*`, ni un `*.spec.*` (`src/resolver/mod.rs`, l. 535–557). `.mts` et `.cts` ne sont donc jamais découverts, bien que `source_type_for` sache les parser. Les répertoires obéissent à trois listes (`src/resolver/mod.rs`, l. 506–533, commentaires élidés) :

Extrait 23.21 – Les constantes de la découverte (élagué : quatre lignes non contiguës, `src/resolver/mod.rs`, l. 509, 515, 526, 533)

```
1 pub const SOURCE_EXTENSIONS: &[&str] = &["ts", "tsx", "js", "jsx"];
2 pub const ALWAYS_EXCLUDED_DIRS: &[&str] = &["node_modules", ".git"];
3 pub const EXCLUDED_DIRS: &[&str] = &["node_modules", "dist", "build", ".next"];
4 pub const HOST_PRUNED_DIRS: &[&str] = &["node_modules", ".git", ".next"];
```

`HOST_PRUNED_DIRS` sert à l'hôte WASM, qui charge un sur-ensemble de fichiers et laisse le moteur refiltrer (chapitre 24). La politique elle-même tient dans une fonction et une récursion :

Extrait 23.22 – `src/resolver/mod.rs`, l. 559–575 — la politique de répertoires

```
559 /// Whether the walk descends into `path`, whose bare name is `name`.
560 ///
561 /// One precedence order, three sources (#137): the list the user configured,
562 /// else the repository's own `.gitignore`s, else the built-in names – with
563 /// [ `ALWAYS_EXCLUDED_DIRS` ] short-circuiting all three.
564 fn skip_dir(name: &str, path: &Path, configured: &[String], ignores: &GitignoreStack) -> bool {
565     if ALWAYS_EXCLUDED_DIRS.contains(&name) {
566         return true;
567     }
568     if !configured.is_empty() {
569         return configured.iter().any(|c| c == name);
570     }
```

```

571     if !ignores.is_empty() {
572         return ignores.ignores_dir(path);
573     }
574     EXCLUDED_DIRS.contains(&name)
575 }

```

La marche `walk` (`src/resolver/mod.rs`, l. 577–599) empile le `.gitignore` du répertoire courant s’il y en a un. Pour chaque enfant, elle descend dans un répertoire que `skip_dir` ne saute pas, retient un fichier si `is_source_file` l’accepte, puis dépile en remontant.

L’ordre de précedence est le suivant. `ALWAYS_EXCLUDED_DIRS` court-circuite tout : `-exclude-dir` restreint ce qu’on saute en plus, il ne fait pas entrer la marche dans un arbre de paquets. Vient ensuite la liste configurée, si elle est non vide. Puis les `.gitignore` du dépôt, s’il y en a au moins un. Enfin, les noms intégrés. `discover` (`src/resolver/mod.rs`, l. 601–623) renvoie seul un `root` qui est un fichier source, amorce la pile des `.gitignore` situés *au-dessus* de `root`, marche, puis *trie*.

Décision : #137 — lire ce que le dépôt déclare généré

L’ancienne politique appliquait les quatre noms d’`EXCLUDED_DIRS` à toute profondeur. Elle sautait donc `scripts/build/`, du code *d’outillage* et non de la sortie de `build`, ainsi que tout répertoire fonctionnel appelé `dist`. Le commentaire de `EXCLUDED_DIRS` la qualifie de « soundness bug ». Le correctif ne consiste pas à allonger ou raccourcir la liste, ce qui aurait été un patch local, mais à lire la source de vérité du dépôt, le `.gitignore`. Les noms intégrés ne restent que comme réponse quand il n’y a rien à lire.

23.4.3 Un lecteur de `.gitignore` pour une seule question

Le module `src/resolver/gitignore.rs` ne répond qu’à une question : « ce répertoire est-il de la sortie générée ? ». Il n’est jamais interrogé sur un fichier, ce qui dispense des subtilités propres aux fichiers (un / final restreint aux répertoires, ré-inclusion d’un fichier sous un répertoire exclu). La pile se construit ainsi :

Extrait 23.23 – `src/resolver/gitignore.rs`, l. 98–114 — les `.gitignore` au-dessus de la racine de marche

```

98     pub fn seed(fs: &dyn FileSystem, root: &Path) -> GitignoreStack {
99         let work_tree = root.ancestors().find(|dir| is_project_root(fs, dir));
100         let mut above: Vec<&Path> = root
101             .ancestors()
102             .skip(1)
103             .take_while(|dir| match work_tree {
104                 Some(top) => dir.starts_with(top),
105                 None => false,
106             })
107             .collect();
108         above.reverse();
109         let mut stack = GitignoreStack::default();
110         for dir in above {
111             stack.push_for(fs, dir);
112         }
113         stack
114     }

```

Le *work tree* est le premier ancêtre qui porte un `.git` (répertoire, ou fichier dans le cas d’un `worktree` ou d’un sous-module) ou un `package.json` (`src/resolver/gitignore.rs`, l. 149–153). Sans lui, rien n’est lu au-dessus de la racine : un `.gitignore` égaré dans `$HOME` ne doit pas décider de ce que sont les sources d’un projet. La marche empile ensuite un `.gitignore` par répertoire

rencontré et le dépile en remontant, grâce au booléen rendu par `push_for`. Chaque couche rend un verdict à trois valeurs, où la dernière règle qui s'applique gagne :

Extrait 23.24 – `src/resolver/gitignore.rs`, l. 59–75 — verdict d'un fichier : la dernière règle gagne

```
59  /// `Some(true)` if this file ignores `path`, `Some(false)` if it
60  /// explicitly re-includes it, `None` if no rule mentions it.
61  ///
62  /// Last match wins, which is git's rule and the reason `!` works at all.
63  fn matches_dir(&self, path: &Path) -> Option<bool> {
64      let rel = relative_posix(&self.dir, path)?;
65      let name: Vec<char> = rel.rsplit('/').next().unwrap_or(&rel).chars().collect();
66      let full: Vec<char> = rel.chars().collect();
67      let mut verdict = None;
68      for rule in &self.rules {
69          let target = if rule.anchored { &full } else { &name };
70          if glob_match(&rule.pattern, target) {
71              verdict = Some(!rule.negated);
72          }
73      }
74      verdict
75  }
```

Puis, dans la pile, la couche la plus profonde qui a une opinion gagne (`find_map` sur les couches en ordre inverse, `src/resolver/gitignore.rs`, l. 138–146). C'est ce qui permet à un `!generated` placé dans `packages/b/.gitignore` de ré-inclure ce que la racine excluait (un test de `tests/discovery_exclusions.rs` le vérifie). L'analyse d'une ligne (`src/resolver/gitignore.rs`, l. 155–192) suit git : blancs finaux coupés sauf s'ils sont échappés, lignes vides et `#...` ignorées, `!` pour la négation, `/` final retiré, et toute `/` restante *ancree* le motif au répertoire du fichier. Le `glob` (`src/resolver/gitignore.rs`, l. 211–255) traite `**`, qui traverse les `/` (et `a/**/b` reconnaît `a/b`), `*` et `?`, qui ne les traversent pas, les classes `[...]` et l'échappement.

Invariant 23.3: Syntaxe incomprise, répertoire lu

Ignorer un répertoire qu'il fallait lire est un faux négatif. Toute syntaxe que le lecteur ne comprend pas, par exemple une classe `[` non terminée, *ne reconnaît rien* (`src/resolver/gitignore.rs`, l. 235–237) : le doute fait lire davantage, jamais moins.

Piège

L'invariant ne couvre que la syntaxe. Un répertoire *réellement* listé, comme le `src/generated/` de l'exemple, n'est plus parcouru, alors qu'il contient du code que l'application importe. L'ancienne liste de noms l'aurait lu. La parade est le blind spot `unread-imports`, qui nomme tout fichier importé mais non lu. Le lecteur n'est pas git non plus : il ignore `.git/info/exclude` et les exclues globaux (`docs/limitations.md`).

23.5 Projets et résolution d'imports

23.5.1 Un projet Vite

Le fixture `tests/fixtures/vite_project` reproduit l'échafaudage de Vite. `src/App.tsx` importe un hook par alias :

Extrait 23.25 – `tests/fixtures/vite_project/src/App.tsx`, l. 5–10 — un import par l'alias `@/*`

```
5 import { useData } from "@hooks/useData";
```

```

6
7 function App() {
8   const data = useData(0);
9   return <div>{data}</div>;
10 }

```

Extrait 23.26 – tests/fixtures/vite_project/src/hooks/useData.ts, l. 4–10 — le hook, qui boucle

```

4 function useData(initial) {
5   const [value, setValue] = useState(initial);
6   useEffect(() => {
7     setValue(value + 1);
8   }, [value]);
9   return value;
10 }

```

La racine tsconfig.json ne contient que des references (JSONC, virgules finales comprises). C'est tsconfig.app.json qui déclare "baseUrl": "." et "paths": { "@/*": ["/src/*"] }.

```

$ reactant check tests/fixtures/vite_project --verbose
[verbose] vite project: discovery root tests/fixtures/vite_project/src, tsconfig aliases loaded
[verbose] symbol graph: 2 nodes, topo order = [useData@tests/fixtures/vite_project/src/hooks/useData.ts, App@tests/
  ↳ fixtures/vite_project/src/App.tsx]
[verbose] 1 components analyzed
[verbose] cache hits: 0, misses: 0
[verbose] App: 3 iteration(s), widened: [1]
App (1 hooks) tests/fixtures/vite_project/src/App.tsx
  warn  infinite-loop [hook:1] (tests/fixtures/vite_project/src/hooks/useData.ts:6:2) this effect keeps pushing
    ↳ state `value` (its deps do not provably gate it, so the effect can re-run every render) to new values on every
    ↳ run. Potential infinite render loop
      (2 trace step(s), rerun with --trace)

! 1 warning(s) across 2 file(s).

```

Quatre décisions du sous-système se lisent dans cette sortie. vite.config.ts désigne un projet Vite. La découverte est rétrécie à src/, si bien que vite.config.ts n'est pas analysé (« 2 file(s) »). L'alias @/hooks/useData est résolu vers src/hooks/useData.ts après un saut par references. Enfin, le hook est inliné dans App, et la position, qui pointe dans un autre fichier que l'en-tête, s'imprime avec son chemin (section 23.9).

23.5.2 Type de projet, racine de configuration, racine de découverte

Extrait 23.27 – src/project/mod.rs, l. 26–37 — les trois conventions reconnues

```

26 /// Build-tool convention detected at a project root.
27 #[derive(Debug, Clone, Copy, PartialEq, Eq)]
28 pub enum ProjectKind {
29     /// No recognized build tool: walk the given path as-is, relative imports only.
30     Plain,
31     /// Vite project: sources under `src/`, tsconfig `paths` aliases.
32     Vite,
33     /// Next.js project: sources under `src/` when the router lives there,
34     /// tsconfig `paths` aliases plus bare `baseUrl` resolution, and RSC
35     /// ``"use client"`` boundaries (ADR-026).
36     NextJs,
37 }

```

La détection repose sur des fichiers marqueurs : next.config.{ts,js,mjs,cjs,mts} désigne un projet Next.js, vite.config.{ts,js,mjs,mts} un projet Vite (src/project/mod.rs, l. 39–67). Next est testé *avant* Vite, parce qu'une application Next peut porter un vite.config.* pour

son exécuteur de tests (vite). Le marqueur est cherché en remontant les ancêtres :

Extrait 23.28 – `src/project/mod.rs`, l. 80–105 — la remontée vers le marqueur le plus proche

```
80 pub fn locate(from: &Path, fs: &dyn FileSystem) -> Option<(ProjectKind, PathBuf)> {
81     nearest_ancestor(from, |dir| match detect(dir, fs) {
82         ProjectKind::Plain => None,
83         kind => Some(kind),
84     })
85 }
86
87 /// The nearest ancestor of `from` (itself included) for which `probe` yields
88 /// a value, and that ancestor's path.
89 ///
90 /// The empty path is probed rather than skipped: `Path::new("src").parent()`
91 /// is `""`, which is the working directory, and `reactant check src` from a
92 /// project root is the common case this exists for.
93 fn nearest_ancestor<T>(
94     from: &Path,
95     mut probe: impl FnMut(&Path) -> Option<T>,
96 ) -> Option<(T, PathBuf)> {
97     let mut cur = Some(from);
98     while let Some(dir) = cur {
99         if let Some(found) = probe(dir) {
100             return Some((found, dir.to_path_buf()));
101         }
102         cur = dir.parent();
103     }
104     None
105 }
```

Définition 23.2: Racine de configuration, racine de découverte

La *racine de configuration* (`config_root`) est l'ancêtre le plus proche de la racine de projet, elle-même comprise, qui porte un marqueur; c'est là que commence la recherche des `tsconfig`. La *racine de découverte* (`discovery_root`) est le répertoire effectivement parcouru pour trouver les sources : la racine de projet, ou `<racine>/src` dans les cas de rétrécissement.

Extrait 23.29 – `src/project/mod.rs`, l. 171–215 — `build_context`, première moitié

```
171 pub fn build_context(
172     root: &Path,
173     forced: Option<ProjectKind>,
174     fs: Arc<dyn FileSystem>,
175 ) -> ProjectContext {
176     // Where the build-tool marker and the tsconfig live – walked upward, so
177     // pointing at a subdirectory still resolves the project's aliases (#9).
178     // `--project` names the kind, not the place, so a forced kind reuses the
179     // located root when there is one and falls back to `root` when there is
180     // not: that is what keeps `--project vite` working on an unmarked tree.
181     let located = locate(root, fs.as_ref());
182     let config_root = located
183         .as_ref()
184         .map(|(_, r)| r.clone())
185         .unwrap_or_else(|| root.to_path_buf());
186     let kind = forced.unwrap_or_else(|| located.map_or(ProjectKind::Plain, |(k, _)| k));
187
188     // Discovery walks what the user named. Narrowing to `<root>/src` is a
189     // convenience for "analyse this project" and applies only when the path
190     // given *is* the project root: pointing inside is already a narrowing, and
191     // widening it back out would analyse files nobody asked for.
```

```

192 let discovery_root = if config_root == *root {
193     match kind {
194         ProjectKind::Plain => root.to_path_buf(),
195         ProjectKind::Vite => {
196             let src = root.join("src");
197             if fs.is_dir(&src) {
198                 src
199             } else {
200                 root.to_path_buf()
201             }
202         }
203         ProjectKind::NextJs => next_discovery_root(root, fs.as_ref()),
204     }
205 } else {
206     root.to_path_buf()
207 };
208 if kind == ProjectKind::Plain {
209     return ProjectContext {
210         kind,
211         discovery_root,
212         resolver: Box::new(DefaultImportResolver::new(fs)),
213         alias_warning: None,
214     };
215 }

```

La seconde moitié (`src/project/mod.rs`, l. 217–253) choisit le résolveur et l’avertissement selon le résultat de `locate_tsconfig_paths`. Elle a trois bras, et deux d’entre eux posent un `alias_warning` dont le texte nomme vite.config ou next.config; on en verra un en section 23.8.

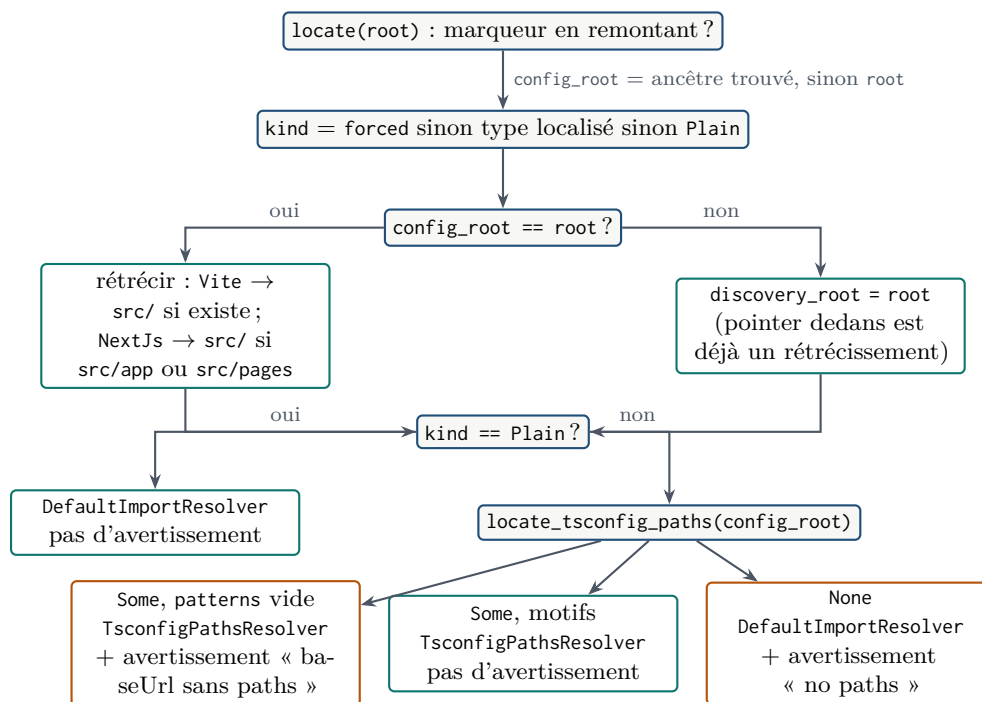


FIGURE 23.2 – L’arbre de décision de `build_context`. En orange, les deux feuilles qui posent un `alias_warning`, que le driver transforme en blind spot `unresolved-aliases`.

La figure 23.2 résume la fonction. Deux choix y méritent d’être justifiés. Le premier : `-project` nomme un *type*, pas un *lieu*. Un type forcé réutilise la racine localisée s’il y en a une, et se replie sur la racine donnée sinon, ce qui garde `-project` vite utilisable sur un

arbre sans marqueur. Le driver prévient alors, en appliquant la même remontée pour ne pas prétendre absent un marqueur situé un niveau plus haut (`src/driver/mod.rs`, l. 147–163 ; `tests forcing_a_kind_{from_inside_the_project_does_not,on_an_unmarked_tree_still}_warn` dans `tests/blind_spots.rs`) :

```
[warn] --project vite: no vite.config.* found in ..., still trying tsconfig paths
```

Le second : le rétrécissement à `src/` est une commodité pour « analyser ce projet ». Il ne s'applique que si le chemin donné *est* la racine de configuration. Pointer dans un sous-répertoire est déjà un rétrécissement, et ré-élargir analyserait des fichiers que personne n'a demandés. Pour Next.js, le rétrécissement exige `src/app` ou `src/pages` (`src/project/mod.rs`, l. 135–149), car Next peuple exactement l'une des deux dispositions.

Décision : ADR-016 §6 — des alias lus dans `tsconfig` seulement

Les alias d'un projet Vite peuvent aussi se déclarer dans `resolve.alias` de `vite.config.*`. Les lire exigerait d'*exécuter* du JavaScript, et l'ADR refuse l'alternative d'une expression régulière « au mieux » : « a best-effort regex would risk *wrong* resolutions, i.e. silent FNs ». `jsconfig.json` et les cartes `exports` de `package.json` sont hors périmètre. Le manque est dit et non caché : quand un type Vite ou Next ne trouve pas de `paths`, un avertissement est imprimé *hors -info*, parce que c'est un caveat de soundness et pas du bruit. Il nomme le fichier de configuration dont les alias ne sont pas lus (« Aliases declared only in vite.config are not read »).

Remarque

L'avertissement est sound mais grossier. Il est émis même si le projet n'écrit aucun `@/...`, ce qui retient à tort le clean bill (section 23.8). Ce faux positif de caveat ne coûte qu'un « ✓ » ; il ne peut pas produire de faux négatif.

23.5.3 Les `tsconfig` : fichier, répertoire, ancêtres

La recherche des `paths` emboîte trois niveaux, résumés en figure 23.3. Le résultat est un `TsconfigPaths` :

Extrait 23.30 – `src/project/tsconfig.rs`, l. 16–27 — `baseUrl` et `paths`, en forme absolue

```
16 /// `compilerOptions.baseUrl` + `paths`, resolved to absolute form.
17 #[derive(Debug, Clone)]
18 pub struct TsconfigPaths {
19     /// Absolute base for path substitutions. When `paths` is declared without
20     /// a `baseUrl`, this is the directory of the declaring config (TS 4.1+).
21     pub base_url: PathBuf,
22     /// `(pattern, targets)` pairs in declaration order, e.g.
23     /// `("@/*", [".src/*"])`. Patterns contain at most one `*`. Empty when
24     /// the config declares a `baseUrl` and no `paths` – `base_url` alone
25     /// still resolves non-relative specifiers.
26     pub patterns: Vec<(String, Vec<String>>),
27 }
```

Invariant 23.4: « `baseUrl` seul » n'est pas « rien »

`Some(TsconfigPaths)` avec `patterns` vide signifie qu'une configuration déclare un `baseUrl` et aucun `paths`. Les spécificateurs nus se résolvent alors contre la base, mais aucun alias n'existe. `None` signifie que rien n'a été trouvé. Cette distinction choisit la feuille de la figure 23.2.

Niveau fichier : `paths_from_config`. Un fichier qui déclare ses propres `paths` non vides gagne. Sans `baseUrl`, sa base est son propre répertoire, selon la sémantique de TypeScript 4.1 et suivants. Sinon, on hérite par `extends`, chaîne ou tableau :

Extrait 23.31 – `src/project/tsconfig.rs`, l. 221–245 — l’héritage par `extends`

```

221 // A patternless parent (bare `baseUrl`) does not end the search – a
222 // later entry in an `extends` array may still declare the aliases – but
223 // it is kept, so an inherited base is not thrown away.
224 let mut inherited_base: Option<TsconfigPaths> = None;
225 for spec in specs {
226     if let Some(parent) = resolve_config_ref(&dir, spec, fs)
227       && let Some(mut found) = paths_from_config(&parent, visited, fs)
228     {
229         // A `baseUrl` declared in THIS config overrides the inherited one.
230         if let Some(base) = &own_base {
231             found.base_url = base.clone();
232         }
233         if !found.patterns.is_empty() {
234             return Some(found);
235         }
236         inherited_base.get_or_insert(found);
237     }
238 }
239 own_base
240     .map(|base_url| TsconfigPaths {
241         base_url,
242         patterns: Vec::new(),
243     })
244     .or(inherited_base)
245 }
```

Si le fichier n’a ni `paths` ni `extends`, il rend son `baseUrl` seul avec une liste de motifs vide, ou `None` (`src/project/tsconfig.rs`, l. 206–215).

Un parent qui n’a qu’un `baseUrl` ne clôt pas la recherche, puisqu’une entrée suivante du tableau peut encore déclarer les alias. Il est seulement mis de côté. Un `baseUrl` déclaré dans le fichier courant l’emporte sur celui qui est hérité. Un ensemble `visited` partagé coupe les cycles. `resolve_config_ref` (`src/project/tsconfig.rs`, l. 137–154) ajoute `.json` si l’extension manque, fait viser `<dir>/tsconfig.json` à une référence de répertoire, et ne suit pas les `extends` vers un paquet. Un nom nu sans `/` est écarté d’emblée. Un nom scopé comme `"@tsconfig/vite-react"` contient un `/` : il est tenté comme chemin relatif au fichier déclarant, puis écarté faute de fichier. Un projet Vite dont le `tsconfig.json` se réduit à `{ "extends": "@tsconfig/vite-react" }`, sans `node_modules`, donne ainsi l’avertissement « no tsconfig ‘paths’ found » et le blind spot correspondant (observé sous `/tmp/ch23/ext`). Un `tsconfig` illisible ou non parsable donne `None` en silence à ce niveau. Le silence est rattrapé par `alias_warning`, jamais par un code 2 (ADR-16 §5).

Niveau répertoire : `load_tsconfig_paths` (`src/project/tsconfig.rs`, l. 247–295). On part de `<root>/tsconfig.json` et de sa chaîne `extends`. Si aucun `paths` n’y figure, on parcourt `references[].path` dans l’ordre : c’est la disposition de Vite, où les `paths` sont dans `tsconfig.app.json`. Un résultat « `baseUrl` seul » est retenu, mais ne court-circuite *pas* ce saut (`test_a_bare_base_url_does_not_shortcut_the_references_hop`).

Extrait 23.32 – `src/project/mod.rs`, l. 121–133 — le plus proche ancêtre qui déclare de vrais `paths`

Niveau ancêtres : locate_tsconfig_path

```

121 fn locate_tsconfig_paths(config_root: &Path, fs: &dyn FileSystem) -> Option<TsconfigPaths> {
122     let mut base_only: Option<TsconfigPaths> = None;
123     nearest_ancestor(config_root, |dir| match load_tsconfig_paths(dir, fs) {
124         Some(found) if !found.patterns.is_empty() => Some(found),
125         Some(found) => {
126             base_only.get_or_insert(found);
127             None
128         }
129         None => None,
130     })
131     .map(|(paths, _)| paths)
132     .or(base_only)
133 }

```

Le répertoire du marqueur est l'endroit où la recherche *commence*, pas celui où elle s'arrête (#139). Un monorepo qui garde vite.config.mts dans une sous-application et les paths à la racine perdait sinon tous ses alias. Le commentaire de la fonction cite excalidraw, qui analysait 37 fichiers sans rien rapporter pour exactement cette raison. La même discipline que pour references s'applique : un ancêtre à baseUrl seul n'est retenu qu'à défaut de mieux.

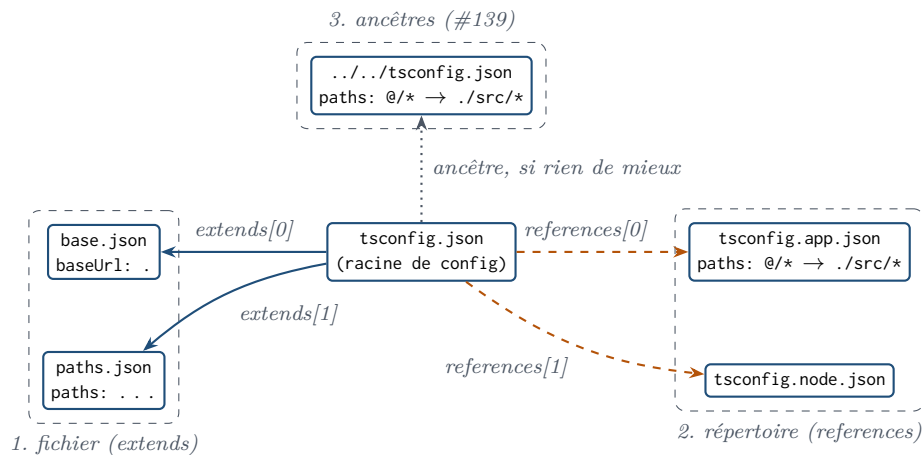


FIGURE 23.3 – Les trois niveaux de la recherche des paths. Traits pleins : la chaîne extends, suivie en premier, où un parent à baseUrl seul est mis de côté. Tirets : le saut references, tenté si la chaîne n'a pas de motifs. Pointillés : la remontée vers les ancêtres de la racine de configuration. Un ensemble visited partagé coupe les cycles.

Le JSONC est ôté par `strip_jsonc` (`src/project/tsconfig.rs`, l. 29–128), en deux passes attentives aux chaînes. La première supprime `// ...` et `/* ... */` en conservant les retours à la ligne, pour que les positions d'erreur restent justes. La seconde supprime toute virgule dont le prochain caractère non blanc est `]` ou `}`. La configuration de `reactant-analyzer` réutilise cette fonction.

23.5.4 Résoudre un spécificateur

Un résolveur implémente un trait d'une méthode (`src/resolver/mod.rs`, l. 42–47). Sa documentation, « Resolve a relative specifier », date d'ADR-13 : depuis ADR-16 et ADR-26, les résolveurs `tsconfig` répondent aussi à des spécificateurs non relatifs. `DefaultImportResolver` (`src/resolver/mod.rs`, l. 650–677) ne traite que les spécificateurs relatifs. Il essaie `<base>.<ext>` pour chaque extension source, puis `<base>/index.<ext>`, et rend un chemin normalisé. `TsconfigPathsResolver` ajoute les alias :

Extrait 23.33 – `src/project/paths_resolver.rs`, l. 78–116 — résolution TypeScript : exacts, puis long préfixe, puis `baseUrl`

```

78 impl ImportResolver for TsconfigPathsResolver {
79     fn resolve(&self, from: &Path, specifier: &str) -> Option<PathBuf> {
80         if specifier.starts_with('.') {
81             return self.fallback.resolve(from, specifier);
82         }
83
84         for (pattern, targets) in &self.exacts {
85             if pattern == specifier {
86                 for t in targets {
87                     if let Some(hit) = self.probe(t) {
88                         return Some(hit);
89                     }
90                 }
91             }
92         }
93
94         for (prefix, suffix, targets) in &self.wildcards {
95             let Some(rest) = specifier.strip_prefix(prefix.as_str()) else {
96                 continue;
97             };
98             let Some(captured) = rest.strip_suffix(suffix.as_str()) else {
99                 continue;
100             };
101             for t in targets {
102                 let substituted = t.replacen('*', captured, 1);
103                 if let Some(hit) = self.probe(&substituted) {
104                     return Some(hit);
105                 }
106             }
107             // Longest matching prefix consumed the specifier: per TS
108             // semantics, do not fall through to shorter patterns – nor to the
109             // baseUrl probe, which TypeScript also skips once `paths` matched.
110             return None;
111         }
112
113         // TypeScript's last resort for a non-relative specifier: `baseUrl`.
114         self.probe(specifier)
115     }
116 }

```

Les motifs exacts (sans `*`) passent d'abord. Viennent ensuite les motifs à joker, triés à la construction par longueur de préfixe décroissante (« `@/generated/* must beat @/*` », `src/project/paths_resolver.rs`, l. 44). Un préfixe qui reconnaît le spécificateur le *consomme*. Si aucune cible n'existe, la réponse est `None`, sans retomber sur un motif plus court ni sur `baseUrl`, comme le fait TypeScript. En dernier recours, le spécificateur est sondé tel quel contre `baseUrl`. La sonde (`src/project/paths_resolver.rs`, l. 55–75) essaie `<base>/<cible>`, puis `.with_extension(ext)`, puis `/index.<ext>`.

Proposition 23.1: La sonde `baseUrl` ajoute, elle ne redirige pas

Soit s un spécificateur non relatif. Si s est reconnu par un motif de `paths`, la sonde `baseUrl` n'est pas consultée. Sinon, elle ne rend un chemin que si un fichier source existe à `<baseUrl>/s`, avec une extension ou un index ajoutés. Elle ne peut donc qu'*ajouter* une arête d'import là où l'on avait `None`, jamais changer une résolution existante.

Preuve. Les deux premières boucles rendent avant la dernière ligne dès qu'un motif reconnaît s (`src/project/paths_resolver.rs`, l. 84–111). La dernière ligne n'est atteinte qu'en l'absence de

reconnaissance, et `probe` ne rend `Some` qu'après un `fs.is_file` vrai. □

Un paquet npm (`react`, `@tanstack/react-query`) retombe ainsi sur `None`, sauf si un fichier homonyme existe sous `baseUrl`. C'est l'argument de soundness d'ADR-26 §2–3 : admettre les spécificateurs non relatifs dans les arêtes d'import ne peut qu'en ajouter.

Combinateurs. `ChainResolver` (le premier qui répond gagne) et `ScopedResolver` (la portée dont la racine est le plus long préfixe du fichier *importateur* gagne, pour les monorepos) existent pour l'API plugin (`src/resolver/mod.rs`, l. 96–160; `docs/plugins.md`). Le driver ne s'en sert pas : un run n'a qu'un résolveur, celui de `build_context`.

23.5.5 Pièges de la détection

Piège

La remontée de `locate` est *lexicale* (`Path::parent`) et non canonique. Depuis un chemin relatif, elle s'arrête donc au répertoire courant. `cd tests/fixtures/vite_project/src && reactant . -verbose` n'imprime aucune ligne « vite project » : le run est Plain. Avec le chemin absolu, `reactant "$PWD"`, Vite est détecté. Sur ce fixture, le finding survit dans les deux cas. Le hook `y` est découvert dans le même run et trouvé par son nom seul, `get_by_name` du registre à clé (fichier, nom) (`src/registry/keyed.rs`, l. 61–67, ADR-13), même sans alias résolu.

Piège

Si *seuls des fichiers* sont passés, la racine de projet est `"."` (`src/driver/mod.rs`, l. 134–139) et la détection n'utilise pas le répertoire du fichier. Depuis la racine du dépôt, observé :

```
$ reactant tests/fixtures/vite_project/src/App.tsx
  1 clean component(s) hidden, rerun with --show-clean

✓ 1 file(s) no issues found.
exit=0
```

Le run est Plain. L'alias `@/hooks/useData` n'est pas résolu, aucune arête n'est créée, donc aucun blind spot unread-imports, et le hook n'est pas dans le run. Le clean bill est imprimé alors qu'un hook qui boucle n'a pas été lu. Seul `-info` révèle « analysis-limit ... hook 'useData' was not found in the registry ... (FN possible) ». Le comportement est cohérent avec le contrat affiché (section 23.7 : un spécificateur non résolu est indistinguishable d'un paquet npm), mais c'est un piège d'usage. `reactant tests/fixtures/vite_project/src` (le répertoire) détecte Vite et trouve le finding. Ce n'est pas le cas qu'`#9` a fermé : sa correction fait remonter *la racine donnée* vers l'ancêtre marqué (`src/project/mod.rs`, l. 171–178), alors qu'ici aucun argument n'est un répertoire, si bien que la racine vaut `"."` avant même que `locate` ne s'exécute (`src/driver/mod.rs`, l. 131–139) : il n'y a pas de chemin à remonter. Au commit `e67b10a`, ce cas précis, un seul fichier passé en argument, n'a pas d'issue ouverte qui le décrive.

Piège

Avec plusieurs répertoires en argument, seul le *premier* décide du type de projet, de la configuration et du rétrécissement à `src/`. Les autres sont parcourus tels quels avec le même résolveur, ce qui peut résoudre un alias d'un paquet vers un fichier *réel* d'un autre paquet, sans blind spot (exercice 23.7). Un répertoire sans source donne un code 2 même si les autres en ont. La liste des fichiers n'est pas dédoublée entre arguments : `reactant dir dir/App.tsx` compte deux fois le même fichier dans « across N file(s) », sans dupliquer les findings.

23.6 Parse, lowering et canal d'erreurs

Sous `/tmp/ch23/parse`, `Recovered.tsx` contient la ligne `const z = a ?? b || c;` (une erreur de syntaxe dont `oxc` se rétablit), suivie d'un composant qui boucle. `Broken.tsx` commence par `const = ;`, qui fait paniquer le parser.

```
$ reactant parse
[skipped] parse/Broken.tsx: Unexpected token, the file was not analyzed
[parse error] parse/Recovered.tsx: Logical expressions and coalesce expressions cannot be mixed
App (2 hooks) parse/Recovered.tsx
  warn  infinite-loop [hook:0] (line 6:2) this effect keeps pushing state `n` (its deps do not provably gate it, so
    ↳ the effect can re-run every render) to new values on every run. Potential infinite render loop
      (2 trace step(s), rerun with --trace)

! 1 warning(s) across 1 file(s).
not analyzed:
  • 1 file(s) could not be parsed and were dropped, so nothing in them was analysed
exit=1
```

Le fichier récupéré est *analysé* : son finding existe malgré l'erreur de syntaxe. Le fichier abandonné est signalé deux fois, par une ligne `[skipped]` et par un blind spot. Les deux cas n'ont pas la même gravité, et le type d'erreur les distingue :

Extrait 23.34 – `src/resolver/mod.rs`, l. 164–178 — deux erreurs de parse qui ne se valent pas

```
164 /// A file the parser or the filesystem complained about.
165 ///
166 /// The two cases are not equally serious, and the caller cannot tell them apart
167 /// from the message alone: a recovered syntax error is noise, while a dropped
168 /// file means every finding it held is a silent false negative – the direction
169 /// the project forbids. Hence `analyzed`.
170 #[derive(Debug, Clone, PartialEq, Eq)]
171 pub struct ParseError {
172     pub file: PathBuf,
173     pub message: String,
174     /// `true` when the parser recovered and the file was still lowered;
175     /// `false` when the file was dropped from the run (read error, or a parser
176     /// panic that leaves the program empty).
177     pub analyzed: bool,
178 }
```

La boucle de `lower_files_with` décide entre les deux :

Extrait 23.35 – `src/resolver/mod.rs`, l. 270–312 — lecture, parse, et le seul critère d'abandon

```
270 for path in files {
271     // One spelling for every registry key. `discover` hands back paths
272     // rooted the way the user typed them (`./b/W.tsx` for `reactant .`)
273     // while `ImportResolver::resolve` answers in normalized form, so a
274     // `(file, name)` lookup built from a resolved import missed every
275     // time the run was invoked with a `.`-prefixed root – silently, and
276     // for imports, hooks, contexts and utilities alike.
277     let path = &normalize(path);
278     let source = match fs.read_to_string(path) {
279         Ok(s) => s,
280         Err(e) => {
281             lowered.parse_errors.push(ParseError {
282                 file: path.clone(),
283                 message: e,
284                 analyzed: false,
285             });
286             continue;
287         }
288     }
```

```

288     };
289     let alloc = Allocator::default();
290     let ret = OxcParser::new(&alloc, &source, source_type_for(path))
291         .with_options(ParseOptions::default())
292         .parse();
293     // `panicked` is oxc's only "this AST is unusable" signal (the program
294     // is empty). The parser recovers from every other syntax error and
295     // still returns a lowerable program, so skipping on a non-empty
296     // diagnostic list would drop whole files from the analysis – a
297     // forbidden false negative, and a growing one: oxc keeps moving TS
298     // semantic checks into the parser.
299     if ret.panicked || !ret.diagnostics.is_empty() {
300         lowered.parse_errors.push(ParseError {
301             file: path.clone(),
302             message: ret
303                 .diagnostics
304                 .first()
305                 .map(|d| d.message.to_string())
306                 .unwrap_or_else(|| "the parser produced no usable program".to_string()),
307             analyzed: !ret.panicked,
308         });
309     }
310     if ret.panicked {
311         continue;
312     }

```

Trois points de ce code comptent. (i) Chaque chemin est normalisé *avant* de servir de clé. La découverte rend les chemins tels que l'utilisateur les a tapés (`./b/W.tsx` pour `reactant`), le résolveur les rend normalisés, et une recherche (fichier, nom) construite depuis un `import` manquait donc toutes ses cibles, en silence, sur un run lancé avec un chemin préfixé par `..` `normalize` (`src/resolver/mod.rs`, l. 625–648) est une réduction lexicale de `.` et `...` `canonicalize` n'est pas employé, parce qu'il suivrait les liens symboliques et produirait des chemins UNC sous Windows. (ii) `ret.panicked` est le seul signal d'oxc qui dise « cet AST est inutilisable ». Sauter un fichier dès qu'une liste de diagnostics est non vide abandonnerait des fichiers entiers, ce qui est un faux négatif interdit, et de plus en plus fréquent à mesure qu'oxc déplace des vérifications sémantiques TypeScript dans le parser (commit 7d1957c). (iii) Le type source dépend de l'extension :

Extrait 23.36 – `src/resolver/mod.rs`, l. 193–200 — le type source, JSX partout sauf dans `.ts`

```

193 pub fn source_type_for(path: &Path) -> oxc_span::SourceType {
194     use oxc_span::SourceType;
195     match path.extension().and_then(|e| e.to_str()) {
196         Some("tsx") => SourceType::tsx(),
197         Some("ts") | Some("mts") | Some("cts") => SourceType::ts(),
198         _ => SourceType::unambiguous().with_jsx(true),
199     }
200 }

```

`.js` et `.jsx` acceptent le JSX, parce que l'écosystème React en met dans des `.js` (Babel, CRA, Docusaurus). Le commentaire rapporte que trois composants réels d'excalidraw étaient perdus quand un `.js` était parsé sans JSX et faisait paniquer le parser. `.ts` reste sans JSX, puisque `<T>expr` y est une assertion de type.

Après le parse, chaque fichier subit quatre passes de lowering : composants, hooks personnalisés, utilitaires, faits de module (chapitre 5). Deux collectes s'y ajoutent, noms de contextes et imports résolus. Après la boucle, deux passes inter-fichiers ont besoin que tous les fichiers aient été vus : `resolve_imported_contexts` (un contexte importé d'un autre fichier analysé devient

un contexte ici, sur un niveau seulement, #49) et `resolve_imported_utilities` (arêtes vers les utilitaires, triées pour le déterminisme). Le produit est un `LoweredProgram` (`src/resolver/mod.rs`, l. 202–233) : composants, hooks, utilitaires, nombre de fichiers effectivement abaissés, erreurs de parse, `FileTable` (ADR-19), `ModuleTable` (directives et arêtes d'import résolues, ADR-26 §1) et arêtes vers les utilitaires (ADR-27 §3). C'est la frontière « `parse+lower` / analyse » qu'ADR-16 §2 a exposée.

Reste le canal de rapport :

Extrait 23.37 – `src/driver/mod.rs`, l. 236–260 — bruit récupéré contre perte silencieuse

```

236 // -- Lower -----
237 let mut lowered = lower_files_with(fs.as_ref(), &files, ctx.resolver.as_ref());
238 // A file the parser recovered from is noise, and stays on the human
239 // channel. A *dropped* file is not: everything it held is a silent false
240 // negative, so it is reported whatever the format – stderr is a separate
241 // stream, so stdout stays exactly one JSON document.
242 let mut dropped = 0usize;
243 for e in &lowered.parse_errors {
244     if e.analyzed {
245         if opts.format == ReportFormat::Human {
246             let _ = writeln!(err, "[parse error] {}: {}", e.file.display(), e.message);
247         }
248     } else {
249         dropped += 1;
250         let _ = writeln!(
251             err,
252             "[skipped] {}: {}, the file was not analyzed",
253             e.file.display(),
254             e.message
255         );
256     }
257 }
258 if dropped > 0 {
259     blind.push(BlindSpot::unparsed_files(dropped));
260 }

```

Invariant 23.5: Un fichier perdu est toujours dit

Une erreur récupérée (`[parse error]`) n'est imprimée qu'en format humain : c'est du bruit, et le fichier a été analysé. Un fichier abandonné (`[skipped]`) est imprimé *quel que soit le format*, sur `stderr`, si bien que `stdout` reste un seul document JSON. Il alimente de plus le `blind spot` `unparsed-files`. En JSON, `parse_errors[].analyzed` porte la même distinction :

```

"files_analyzed": 1,
"parse_errors": [
  { "file": "parse/Broken.tsx", "message": "Unexpected token", "analyzed": false },
  { "file": "parse/Recovered.tsx", "message": "Logical expressions and coalesce expressions cannot be mixed", "
    ↪ analyzed": true }
],

```

(JSON reformaté sur moins de lignes ; contenu observé avec `-format json`, dont le `stderr` ne contient que la ligne `[skipped]`).

23.7 Blind spots : pas de clean bill pour du code non lu

23.7.1 Un run rétréci

Le fixture `tests/fixtures/blind_spots/outside_root` contient `app/App.tsx`, qui importe `../shared/useThing` (un hook qui boucle), et `app/Broken.tsx`, qui a sa propre boucle. Pointons l'analyseur sur `app` seulement :

```
$ reactant tests/fixtures/blind_spots/outside_root/app
Broken (2 hooks) tests/fixtures/blind_spots/outside_root/app/Broken.tsx
  warn  infinite-loop [hook:0] (line 7:2) this effect keeps pushing state `n` to new values on every run. Potential
        ↳ infinite render loop
        (2 trace step(s), rerun with --trace)
  1 clean component(s) hidden, rerun with --show-clean

! 1 warning(s) across 2 file(s).
not analyzed:
  • 1 imported file(s) resolved outside the analysed set and were never read. Pass them on the command line to analyse
    ↳ them (tests/fixtures/blind_spots/outside_root/shared/useThing.ts)
exit=1
```

App est « propre » par ignorance : `useThing` est opaque, parce que son fichier n'est pas dans le run. Le résolveur savait exactement où était ce code, et le run le dit.

Définition 23.3: Blind spot

Un *blind spot* est une raison pour laquelle le silence d'un run n'est pas une preuve de correction : une source vers laquelle le pipeline a été dirigé et qu'il n'a pas lue. Il en existe trois espèces, de clés stables `unresolved-aliases` (les alias du projet n'ont pas pu être chargés), `unparsed-files` (des fichiers ont été abandonnés au parse) et `unread-imports` (des imports résolus vers de vrais fichiers que la découverte n'a pas atteints).

Extrait 23.38 – `src/driver/blind_spots.rs`, l. 22–31 — une raison pour laquelle le silence n'est pas une preuve

```
22 /// One reason this run's silence is not evidence of correctness.
23 pub struct BlindSpot {
24     /// Stable machine key for JSON consumers: `unresolved-aliases`,
25     /// `unparsed-files`, `unread-imports`.
26     pub kind: &'static str,
27     /// How many occurrences this entry aggregates.
28     pub count: usize,
29     /// One sentence naming what was not read.
30     pub detail: String,
31 }
```

L'en-tête du module (`src/driver/blind_spots.rs`, l. 1–13) fixe l'architecture : les blind spots sont collectés « once, at the driver level », et une liste non vide interdit le clean bill « in every output format ». Les entrées sont agrégées par espèce : un run qui abandonne deux cents fichiers le dit en une ligne, et le détail reste où il est déjà (le tableau `parse_errors`, les diagnostics `-info`). Le calcul des imports non lus est remarquable par ce qu'il *ne fait pas* :

Extrait 23.39 – `src/driver/mod.rs`, l. 262–280 — les arêtes résolues qui sortent de l'ensemble analysé

```
262 // An import the resolver mapped to a real file that discovery never
263 // reached: the pipeline knew where the code was and did not read it (#9).
264 // Read off the resolved edges rather than re-resolving, so this cannot
265 // disagree with what lowering actually did.
266 let analysed: std::collections::HashSet<PathBuf> = files
```

```

267     .iter()
268     .map(|p| crate::resolver::normalize(p))
269     .collect();
270     let unread: std::collections::BTreeSet<&PathBuf> = lowered
271     .module_table
272     .paths()
273     .filter_map(|p| lowered.module_table.facts(p))
274     .flat_map(|f| f.imports.iter())
275     .filter(|dep| !analysed.contains(*dep))
276     .collect();
277     if !unread.is_empty() {
278         let examples: Vec<String> = unread.iter().take(3).map(|p| display(p)).collect();
279         blind.push(BlindSpot::unread_imports(&examples, unread.len()));
280     }

```

Il lit les arêtes déjà résolues par le lowering (la `ModuleTable`) au lieu de re-résoudre les imports, et ne peut donc pas contredire ce que le lowering a fait. Le `BTreeSet` rend les trois exemples déterministes.

Invariant 23.6: Pas de clean bill pour du code non lu

La ligne « ✓ ... no issues found. » n'est imprimée que si le run n'a ni `ERROR` ni `WARNING` visibles et n'a aucun blind spot (`src/driver/human.rs`, l. 265–283). Sinon, la ligne devient « ! ... no findings, but parts of this run were not analyzed, so this is not a clean bill. », suivie de la liste `not analyzed`. La règle vaut dans tous les formats : en JSON, un `blind_spots` non vide signifie que `summary.errors` et `summary.warnings` sont un minorant et non un verdict (`src/driver/json.rs`, l. 23–25). Un nouveau blind spot s'ajoute en poussant sur la liste, pas en modifiant chaque `render`.

Décision : Couverture seulement : `analysis-limit` n'est pas un blind spot

Une INFO `analysis-limit` décrit du code que l'analyseur *a* lu et abstrait soundement en $\top$, par exemple un hook de paquet npm introuvable. Elle se déclenche sur presque tous les runs : le module mesure 370 sites sur une application de 209 fichiers, presque tous des composants et des hooks npm. « A caveat printed every time is a caveat nobody reads » (`src/driver/blind_spots.rs`, l. 15–20). Ce qui retient le clean bill est plus étroit et décisif : une source que le run a été chargé de lire et qu'il n'a pas lue. Le prix de ce choix se voit au piège du fichier seul (section 23.5) : un spécificateur que le résolveur n'a pas su résoudre est indistinguable d'un paquet npm, donc ne crée pas d'arête, donc pas de blind spot.

23.7.2 -follow-imports : lire ce que les chemins nommés importent

La parade au run rétréci est opt-in :

```

$ reactant tests/fixtures/blind_spots/outside_root/app --follow-imports
App (1 hooks) tests/fixtures/blind_spots/outside_root/app/App.tsx
  warn  infinite-loop [hook:1] (tests/fixtures/blind_spots/outside_root/shared/useThing.ts:6:2) this effect keeps
    ↪ pushing state `n` to new values on every run. Potential infinite render loop
    (2 trace step(s), rerun with --trace)
Broken (2 hooks) ...

! 2 warning(s) across 3 file(s).
  followed 1 imported file(s) (tests/fixtures/blind_spots/outside_root/shared/useThing.ts)

```

La fermeture est calculée *avant* le lowering, pour coûter un parse de plus par fichier plutôt qu'une passe de lowering complète par niveau de profondeur (`src/driver/mod.rs`, l. 216–234) :

Extrait 23.40 – `src/resolver/closure.rs`, l. 44–66 — la fermeture transitive des imports


```

44 pub fn import_closure(
45     fs: &dyn FileSystem,
46     seeds: &[PathBuf],
47     resolver: &dyn ImportResolver,
48 ) -> Vec<PathBuf> {
49     let mut seen: HashSet<PathBuf> = seeds.iter().map(|p| normalize(p)).collect();
50     let mut queue: Vec<PathBuf> = seen.iter().cloned().collect();
51     let mut found: Vec<PathBuf> = Vec::new();
52
53     while let Some(file) = queue.pop() {
54         for dep in edges_of(fs, &file, resolver) {
55             let dep = normalize(&dep);
56             if !followable(&dep) || !seen.insert(dep.clone()) {
57                 continue;
58             }
59             found.push(dep.clone());
60             queue.push(dep);
61         }
62     }
63
64     found.sort();
65     found
66 }

```

C'est un parcours en profondeur avec un ensemble *seen*, qui termine sur les cycles. Chaque fichier y est parsé une fois pour ses seules arêtes, par `collect_module_facts`, la fonction même qui produit le blind spot `unread-imports`. La fermeture ne peut donc pas diverger de ce qui est déclaré non lu. `followable` exclut tout chemin qui contient `node_modules`, même si un alias `tsconfig` y pointe : le code des paquets n'est jamais abaissé, et le `SummaryRegistry` est le point d'extension prévu pour lui (#51, fermée *wontfix*). Les exclusions de la découverte (`.gitignore`, `EXCLUDED_DIRS`) ne s'appliquent *pas* : elles décident de ce qu'on parcourt à l'aveugle, alors qu'une arête d'import explicite prouve que le code s'exécute. Les `import type` ne sont pas des arêtes. La documentation du drapeau prévient que ce n'est pas une optimisation : « the closure often approaches the whole project ».

Définition 23.4: Chemins nommés, fichiers suivis, findings retenus

Les *chemins nommés* sont les fichiers issus des arguments. Ils fixent la *portée du rapport*. Les *fichiers suivis* sont ceux que la fermeture ajoute, et ils ne servent qu'à rendre l'analyse des fichiers nommés correcte. Un finding d'un composant défini dans un fichier suivi est calculé mais *retenu* : il est compté dans `withheld` et non affiché.

Extrait 23.41 – `src/driver/mod.rs`, l. 440–451 — un composant hors des chemins nommés : compté, pas montré

```

440     // A component defined outside the named paths exists in this run only
441     // to make the named ones analysable. Its findings are real, and saying
442     // how many there are is the honest half of not showing them.
443     if let Some((file, _)) = meta
444         && !named.contains(&crate::resolver::normalize(file))
445     {
446         if !diags.is_empty() {
447             withheld += diags.len();
448             withheld_files.insert(display(file));
449         }
450         continue;
451     }

```

L'exemple `/tmp/ch23/wh` rend ce mécanisme visible. `app/App.tsx` rend `<Panel />`, défini dans `shared/Panel.tsx`, qui boucle.

```
$ reactant wh/app

! 1 file(s), no findings, but parts of this run were not analyzed, so this is not a clean bill.
not analyzed:
  • 1 imported file(s) resolved outside the analysed set and were never read. Pass them on the command line to analyse
    ↳ them (wh/shared/Panel.tsx)
exit=0
$ reactant wh/app --follow-imports

✓ 2 file(s) no issues found.
  followed 1 imported file(s) (wh/shared/Panel.tsx)
  1 finding(s) in those file(s) are not shown. Name the path(s) to report them (wh/shared/Panel.tsx)
exit=0
```

Piège

Sous `--follow-imports`, le clean bill peut coexister avec un finding réel. Le « ✓ » porte sur les chemins nommés, le finding de `Panel` est retenu, et le code de sortie vaut 0. La ligne jaune qui suit est la « moitié honnête » de ce choix : le finding existe, le run dit combien il y en a et où. En JSON, `followed.withheld` vaut 1 et `summary.warnings` vaut 0. Pour le rapporter, il faut nommer `wh/shared`.

23.8 Next.js et le graphe serveur

Rappel React : Server Components et App Router

Sous l'App Router de Next.js, un module est un *Server Component* sauf si une directive `"use client"` ouvre une frontière client au-dessus de lui dans le graphe d'imports. Un Server Component est rendu une fois, sur le serveur, et React ne lui fournit aucun hook. Les fichiers réservés `page`, `layout`, `template`, `default`, `not-found` et `loading` sous un répertoire `app/` sont des entrées serveur. `error` et `global-error` doivent être des Client Components. Un module importé des deux côtés est compilé deux fois, et la copie serveur doit s'exécuter elle aussi.

Le type `NextJs` ajoute trois choses aux conventions de Vite : le rétrécissement conditionnel à `src/` (section 23.5), la sonde `baseUrl` (proposition 23.1) et le calcul du graphe serveur, qui nourrit la règle `server-component-hook` (chapitre 21) :

Extrait 23.42 – `src/project/nextjs.rs`, l. 26–55 — entrées serveur et graphe serveur

```
26 /// The App Router entry kind of `file` (`"page"`, `"layout"`, ...), or `None`.
27 ///
28 /// Requires both halves of the convention: the stem is a reserved entry name
29 /// *and* the file sits under a directory named `app`. The stem alone would
30 /// claim any `components/page.tsx`.
31 pub fn server_entry_kind(file: &Path) -> Option<&'static str> {
32     let stem = file.file_stem()?.to_str()?;
33     let kind = SERVER_ENTRY_STEMS.iter().find(|s| **s == stem)?;
34     file.parent()?
35         .ancestors()
36         .any(|d| d.file_name().and_then(|n| n.to_str()) == Some("app"))
37         .then_some(*kind)
38 }
39
40 /// Modules Next compiles into the **server** graph: every App Router server
41 /// entry that does not opt out, plus everything they import, stopping at each
42 /// `use client` boundary.
43 ///
```

```

44 /// Reachability is what decides it, not the absence of a directive: a module
45 /// with no directive that nothing server-side imports is simply not part of
46 /// this graph, and a shared module imported from both sides *is* – Next
47 /// compiles it twice, and the server copy is the one that has to run.
48 pub fn server_modules(table: &ModuleTable) -> HashSet<PathBuf> {
49     let seeds: Vec<&Path> = table
50         .paths()
51         .filter(|p| server_entry_kind(p).is_some())
52         .map(PathBuf::as_path)
53         .collect();
54     table.reachable_from(seeds, Some(USE_CLIENT))
55 }

```

La liste `SERVER_ENTRY_STEMS` (`src/project/nextjs.rs`, l. 13–24) contient les six noms du rappel ci-dessus. `error` et `global-error` en sont délibérément absents.

Le calcul exige les deux moitiés de la convention (un nom réservé *et* un ancêtre `app`), faute de quoi n'importe quel `components/page.tsx` serait une entrée. L'appartenance au graphe est décidée par l'atteignabilité et non par l'absence de directive. `reachable_from` est l'unique mécanisme partagé sur la `ModuleTable`, une table non interprétée de chaînes et de chemins (ADR-26 §1).

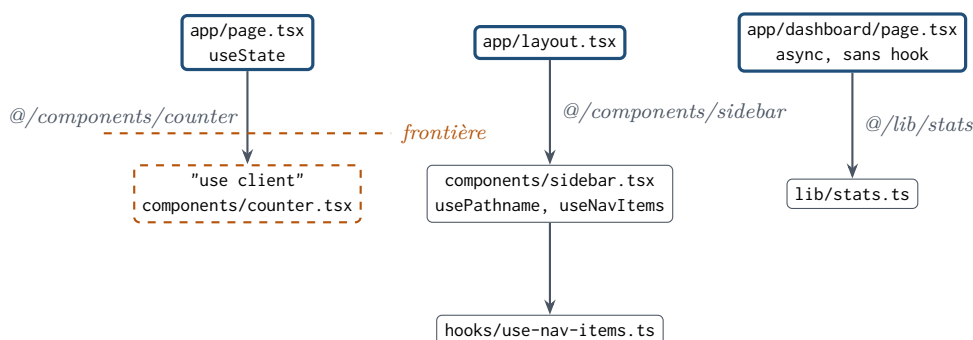


FIGURE 23.4 – Le graphe d'imports de `tests/fixtures/next_project/src` (chemins relatifs à `src/`). En gras, les trois entrées serveur. Le graphe serveur est `{page, layout, dashboard/page, sidebar, use-nav-items, stats}`; il s'arrête à `counter.tsx`, qui porte "use client".

Le fixture `tests/fixtures/next_project` (figure 23.4) comprend une page serveur qui appelle `useState`, un layout qui importe une barre latérale sans directive, un composant client et une page asynchrone sans hook.

```

$ reactant tests/fixtures/next_project --verbose --show-clean
[verbose] next.js project: discovery root tests/fixtures/next_project/src, tsconfig aliases loaded
...
[verbose] 3 components analyzed
[verbose] cache hits: 1, misses: 2
...
Counter (3 hooks) tests/fixtures/next_project/src/components/counter.tsx
  warn  infinite-loop [hook:1] (line 7:2) this effect has no dependency array and may store a fresh reference into
  ↳ state `n`, so it re-runs after every render and can re-trigger itself: possible infinite render loop
  (1 trace step(s), rerun with --trace)
HomePage (2 hooks) tests/fixtures/next_project/src/app/page.tsx
  warn  server-component-hook [hook:0] (line 7:8) `useState` is called in a Server Component. this file is an App
  ↳ Router `page` and no `"use client"` directive covers it, ...
Sidebar (2 hooks) tests/fixtures/next_project/src/components/sidebar.tsx
  warn  server-component-hook [hook:0] (line 8:8) `usePathname`, `useState`, `useMemo` are called in a Server
  ↳ Component. this module is imported into the App Router's server graph ...

! 3 warning(s) across 7 file(s).

```

(lignes éliminées par ...). `src/app` existe, donc la découverte se fait dans `src/`, et ni `next.config.ts` ni `tsconfig.json` ne sont analysés. Les « 3 components analyzed » sont les trois racines (`HomePage`,

RootLayout, DashboardPage), les enfants étant analysés par descente. RootLayout et DashboardPage, sans hook ni finding, ne s'impriment pas même sous `--show-clean`. `usePathname`, importé de `next/navigation`, est un hook *connu* du registre de résumés (section 23.10) et ne donne pas d'INFO « unknown hook ».

```
$ reactant tests/fixtures/next_project/src/app
1 clean component(s) hidden, rerun with --show-clean

! 3 file(s), no findings, but parts of this run were not analyzed, so this is not a clean bill.
not analyzed:
  • 3 imported file(s) resolved outside the analysed set and were never read. Pass them on the command line to analyse
    ↳ them (tests/fixtures/next_project/src/components/counter.tsx, tests/fixtures/next_project/src/components/
    ↳ sidebar.tsx, tests/fixtures/next_project/src/lib/stats.ts)
exit=0
```

`server-component-hook` ne se déclenche plus sur `HomePage`. `counter.tsx`, le seul fichier qui porte "use client", n'a pas été abaissé, et la règle est gardée par la présence d'au moins un "use client" dans le programme (ADR-26 §4). La règle sous-rapporte, conformément à son contrat, et le blind spot retient le clean bill : il n'y a pas de faux négatif silencieux.

Un projet à `baseUrl` seul. Sous `/tmp/ch23/nb` : un `next.config.js`, un `tsconfig.json` réduit à `{ "compilerOptions": { "baseUrl": "." } }`, puis `app/page.tsx` qui importe `components/widget`, `components/widget.tsx` ("use client") qui importe `useCounter` depuis `lib/counter`, et `lib/counter.ts` qui boucle.

```
$ reactant nb --verbose
[warn] tsconfig declares `baseUrl` but no `paths`, so bare specifiers resolve against it, but `@/...`-style aliases stay
  ↳ unresolved and their targets are NOT analyzed (possible false negatives). Aliases declared only in next.config
  ↳ are not read.
[verbose] next.js project: discovery root nb, tsconfig aliases unavailable
...
Widget (1 hooks) nb/components/widget.tsx
warn infinite-loop [hook:1] (nb/lib/counter.ts:4:2) this effect keeps pushing state `n` (its deps do not
  ↳ provably gate it, so the effect can re-run every render) to new values on every run. Potential infinite render
  ↳ loop
    (2 trace step(s), rerun with --trace)

! 1 warning(s) across 4 file(s).
not analyzed:
  • tsconfig declares `baseUrl` but no `paths`, ...
```

La sonde `baseUrl` résout `components/widget` et `lib/counter`, ce qui donne l'arête `<Widget />` et l'inlining de `useCounter`. Sans `src/app`, la découverte reste à la racine et `next.config.js` est analysé (« 4 file(s) »). Le blind spot `unresolved-aliases` est posé alors que ce projet n'écrit aucun `@/...`. C'est le caveat grossier de la section 23.5. De plus, `--verbose` dit « aliases unavailable » alors qu'un résolveur `baseUrl` est bien chargé, parce que le libellé ne teste que `alias_warning.is_none()` (`src/driver/mod.rs`, l. 181–185).

Décision : ADR-026 — analyser les Server Components quand même

L'alternative naturelle, *sauter* les modules serveur, est refusée. Le caractère serveur n'est pas une propriété d'un fichier mais de la façon dont il est atteint, à travers des alias que le résolveur a pu manquer. Décider « serveur, on saute » sur des arêtes incomplètes perdrait en silence les défauts du module. Les analyser ne coûte rien en soundness : un Server Component n'a pas d'état, et l'interprétation abstraite le sur-approxime comme tout composant aux props `⊥`. Deux autres alternatives sont refusées : fabriquer un Certified à partir de la convention de chemins (« it would dress a path convention as a domain proof »), d'où un

WARNING ; et supprimer les autres findings d'un module serveur, ce qui transformerait chaque erreur de classification en faux négatif. L'ADR ajoute la détection `next.config.*` avant Vite, le rétrécissement conditionnel, la sonde `baseUrl` et des résumés `next/navigation`. Un point de son §5 est dépassé : `usePathname` y est dit « typé string », alors qu'au commit étudié c'est un résumé `Held` (#161).

23.9 Le rendu : humain et JSON

23.9.1 Nommer le bon fichier

Un hook inline depuis un autre fichier porte des positions de ce fichier. Imprimer (line 6:2) sous l'en-tête d'`App.tsx` désignerait une ligne dans le mauvais fichier. ADR-24 §1 l'a mesuré : 44 % des findings de règles personnalisées étaient dans ce cas. Une seule fonction rend toutes les positions, celle de la ligne principale comme celles des pas `-trace` :

Extrait 23.43 – `src/driver/human.rs`, l. 23–35 — (line L:C) ou (chemin:L:C)

```

23 fn position(
24     range: SourceRange,
25     comp_file: Option<&Path>,
26     files: &FileTable,
27     display: &dyn Fn(&Path) -> String,
28 ) -> String {
29     match files.path(range.file) {
30         Some(f) if comp_file != Some(f) => {
31             format!("{}", display(f), range.line, range.col)
32         }
33         _ => format!("(line {}:{})", range.line, range.col),
34     }
35 }

```

Décision : ADR-011 — positions et notes

ADR-11 a introduit `SourceRange { line, col }` (ligne comptée à partir de 1), calculé par une table des débuts de ligne construite une fois en $O(n)$ puis interrogée par recherche binaire en $O(\log n)$. Il a aussi introduit des `span: Option<SourceRange>` sur les instructions et les `HookEntry`, et des `Note` portées par `Diagnostic` pour exprimer une chaîne causale, affichées « $\rightarrow \dots$ » par la CLI. Sans cela, un diagnostic ne donnait que `[hook:0]`, sans ligne et sans le handler qui avait élargi un état. L'ADR a été dépassée sur deux points : ADR-19 a ajouté un `file: FileId` aux positions et typé les notes en `Step`, et ADR-24 fait nommer le fichier d'origine sur la ligne principale. Malgré son nom, `SourceRange` est une position de *début* (`src/ir/source_range.rs`, l. 36–42) : aucun `render` n'imprime de fin de plage.

23.9.2 Localisations et attributions

Trois composants Alpha, Beta et Gamma appellent `useShared(k)`, défini dans `hooks/useShared.ts` avec la boucle du section 23.5 (`fixture tests/fixtures/shared_hook_repeat`) :

```

$ reactant check tests/fixtures/shared_hook_repeat --trace
Alpha (1 hooks) tests/fixtures/shared_hook_repeat/App.tsx
  warn  infinite-loop [hook:1] (tests/fixtures/shared_hook_repeat/hooks/useShared.ts:8:2) this effect keeps pushing
    ↳ state `value` (its deps do not provably gate it, so the effect can re-run every render) to new values on every
    ↳ run. Potential infinite render loop [in 3 components]
      in: Alpha, Beta, Gamma
        ↳ state `value` is written here [hook:2] (tests/fixtures/shared_hook_repeat/hooks/useShared.ts:8:2)
        ↳ the abstract value of state `value` kept growing and was widened at iteration 3

```

```
2 component(s) hidden. Every finding in them is a source line already reported above

! 1 warning(s) across 2 file(s), 3 component attribution(s).
```

En JSON, le même run a trois lignes de diagnostic, une par composant, toutes avec "file": ".../hooks/useShared.ts" et "line": 8, puis "summary": { "errors": 0, "warnings": 3, ..., "exit_code": 1 }.

Définition 23.5: Localisation, attribution

Une *attribution* est une ligne « (composant, diagnostic) » du rapport. Une *localisation* est une classe d'attributions de même clé (rule, (fichier, ligne, colonne), message). Une attribution sans position forme sa propre classe. Le composant *canonique* d'une localisation est le premier dans l'ordre du rapport, c'est-à-dire par nom affiché.

Extrait 23.44 – src/driver/locations.rs, l. 24–40 — la clé de groupage et l'index

```
24 /// The key a finding is grouped by – exactly the key the corpus measurement
25 /// used. `None` for the position when the diagnostic carries no range: a row
26 /// whose location is unknown is never claimed to be the same as another's.
27 type Key<'a> = (&'a str, Option<(FileId, u32, u32)>, &'a str);
28
29 /// Which components share each finding, and where each row should render.
30 pub struct LocationIndex {
31     /// `roles[component][diagnostic]` – `Some(group)` when this row is the
32     /// canonical one for its location, `None` when it repeats one printed
33     /// earlier under a component sorted before it.
34     roles: Vec<Vec<Option<usize>>>,
35     /// Component indices sharing a location, canonical one first.
36     groups: Vec<Vec<usize>>,
37     /// Distinct-location counts – what the human summary reports.
38     pub errors: usize,
39     pub warnings: usize,
40 }
```

La construction (src/driver/locations.rs, l. 42–89) fait une passe unique dans l'ordre du rapport. Une clé déjà vue avec une position connue ajoute le composant au groupe existant et marque la ligne comme répétition (None). Une clé nouvelle ouvre un groupe et compte une ERROR ou un WARNING. Le résumé humain compte des localisations, le JSON et -fail-on comptent des attributions. La queue « , N component attribution(s) » n'apparaît que si les deux nombres diffèrent (src/driver/human.rs, l. 284–310). Le module cite sa mesure : 6 322 findings pour 1 170 localisations distinctes sur le corpus du 2 septembre 2026, soit 81 % de répétition (src/driver/locations.rs, l. 8–10).

Décision : ADR-024 §2 et #129 — grouper pour l'affichage, jamais dédupliquer

ADR-24 §2 refuse de dédupliquer les findings entre les consommateurs d'un hook. Ils ne sont pas redondants mais *incomparables* : le même corps de hook, analysé sous useStep(1) et sous useStep(0), donne des faits différents. Un hook n'a pas d'analyse propre, et supprimer les findings par consommateur au profit d'un finding de niveau hook serait un générateur de faux négatifs. Le groupage d'affichage, forme faible, était alors différé. #129 l'a adopté, *uniquement* dans le renderer humain : le JSON reste une ligne par composant, et -fail-on lit les comptes non groupés. Ce que chaque finding *est* ne change pas.

Piège

ADR-24 §2 nommait deux risques du groupage : un groupe n'a qu'un témoin, et le `hook_label` peut différer d'un membre à l'autre. L'implémentation imprime le `[hook:N]` et la trace du composant *canonique* seulement. Les autres chaînes de témoins et les autres labels ne sont visibles qu'en JSON. Deux membres d'une localisation ont même règle, même position et même message, mais pas nécessairement même label ni même chemin de témoin.

Le canal d'assurance. Sous `-info` seulement, chaque composant imprimé liste ses `verified` (section 23.1). Quand une `analysis-limit` a tronqué l'analyse, le registre vide cette liste, et le `render` imprime à la place ce qui a été retenu (`src/driver/human.rs`, l. 37–66) :

```
$ reactant tests/fixtures/vite_project/src/App.tsx --info
App (1 hooks) tests/fixtures/vite_project/src/App.tsx
info analysis-limit [hook:0] (line 8:8) hook `useData` was not found in the registry. Pass its source file or
  ↳ add a HookSummary to analyse it (FN possible)
suspended analysis-limit 1 passing check(s) withheld: the analysis was truncated in this component, so they are not
  ↳ guaranteed
```

La ligne `suspended` n'est *pas* soumise aux filtres de règles : `-ignore-rule analysis-limit` masque l'avis, pas l'absence de garantie.

23.9.3 Le schéma JSON v2

`-format json` produit un seul document sur `stdout` (`to_string_pretty` suivi d'un retour à la ligne). Sa racine :

Extrait 23.45 – `src/driver/json.rs`, l. 17–31 — la racine du rapport JSON

```
17 #[derive(Serialize)]
18 struct JsonReport<'a> {
19     version: u32,
20     files_analyzed: usize,
21     parse_errors: Vec<JsonParseError>,
22     diagnostics: Vec<JsonDiagnostic<'a>>,
23     /// What the run knows it did not read. Non-empty means `summary.errors` and
24     /// `summary.warnings` are a lower bound, not a verdict (#9, #47).
25     blind_spots: Vec<JsonBlindSpot<'a>>,
26     /// Present only under `--follow-imports` (#138): what the run read beyond
27     /// the named paths, and what it found there and is not reporting.
28     #[serde(skip_serializing_if = "Option::is_none")]
29     followed: Option<JsonFollowed<'a>>,
30     summary: JsonSummary,
31 }
```

Chaque diagnostic sépare deux fichiers. `to_json_diag` (`src/driver/json.rs`, l. 229–254) calcule `file: anchor_file.or_else(|| file.map(display))`, où `anchor_file` est le fichier de la position du diagnostic, et `component_file: file.map(display)` :

```
$ reactant check tests/fixtures/cross_file_hook --format json      # extrait
{"component": "Page",
 "file": "tests/fixtures/cross_file_hook/hooks/useData.ts",
 "component_file": "tests/fixtures/cross_file_hook/page.tsx",
 "line": 7,
 "col": 2,
 "hook_label": 1,
 ...
 { "message": "state `value` is written here", "kind": "write", "hook_label": 2,
   "file": "tests/fixtures/cross_file_hook/hooks/useData.ts", "line": 7, "col": 2,
   "slot": 1, "value_class": "unknown" },
 { "message": "the abstract value of state `value` kept growing and was widened at iteration 3",
```



```
"kind": "widen", "hook_label": null, "file": null, "line": null, "col": null,
"slot": 1, "iteration": 3 }
```

(notes reformatées sur moins de lignes). En v1, `file` était le fichier du composant et se trouvait donc apparié à un numéro de ligne d'un autre fichier. La v2 fait de `file` le fichier de l'ancre, avec `repli` sur `component_file`, qui garde le sens v1 (ADR-24 §1). Les autres points à connaître :

- `notes[].kind` est `Step::kind()` : `binding`, `resolve`, `call`, `write`, `read`, `branch`, `handler`, `cycle-edge`, `widen`, `mutate`, `capture`, `init-once`, `forward`, `rerender` (`src/rules/api/witness.rs`, l. 134–151). Les champs propres à un genre sont omis quand ils sont vides. `file`, `line`, `col` et `hook_label` sont toujours présents, à `null` au besoin ;
- `summary.errors/warnings` comptent des attributions. `components_analyzed` est le nombre de composants *rapportés* et non le compteur du moteur. `exit_code` reflète le `-fail-on` actif ;
- `blind_spots` est toujours présent, éventuellement vide ; `followed` n'apparaît que sous `-follow-imports` ;
- aucun JSON Schema n'est généré pour le rapport : seuls `pack.json` et `reactant.config.json` en ont un, et les types TypeScript du rapport, côté npm, sont écrits à la main.

Remarque

La documentation n'a pas suivi le code sur trois points. La doc du variant `OutputFormat::Json`, visible dans `reactant -help`, dit « `schema v1` » (`src/cli/mod.rs`, l. 82), alors que le code émet `version: 2` (`src/driver/json.rs`, l. 268). La liste des `kind` de `docs/usage.md` et du commentaire `src/driver/json.rs`, l. 91–92 omet `forward` et `rerender`. Enfin, la page `reactant help` n'affiche pas `-rule-option`, que `-help` de `clap` connaît.

23.10 Le registre des résumés de bibliothèques

23.10.1 Connu n'est pas modélisé

Le fichier `/tmp/ch23/sum/Nav.tsx` contient deux composants identiques, à la provenance du hook près :

Extrait 23.46 – Deux `useQuery`, deux provenances (`/tmp/ch23/sum/Nav.tsx`)

```
1 import { useEffect } from "react";
2 import { useQuery } from "@tanstack/react-query";
3 import { useQuery as useOther } from "my-query-lib";
4
5 export function Known() {
6   const q = useQuery({ queryKey: ["a"] });
7   useEffect(() => {
8     console.log(q);
9   }, [q]);
10  return <div />;
11 }
12
13 export function Unknown() {
14   const q = useOther({ queryKey: ["a"] });
15   useEffect(() => {
16     console.log(q);
17   }, [q]);
18  return <div />;
19 }
```

```
$ reactant Nav.tsx --info
Unknown (2 hooks) Nav.tsx
```



```

info analysis-limit [hook:0] (line 14:8) hook `useQuery` was not found in the registry. Pass its source file or
↳ add a HookSummary to analyse it (FN possible)
suspended analysis-limit 4 passing check(s) withheld: the analysis was truncated in this component, so they are not
↳ guaranteed
1 clean component(s) hidden, rerun with --show-clean

✓ 1 file(s) no issues found.

```

Dans les deux cas, la valeur de `q` est `⊤`. Mais `Known` importe un hook *connu* : il n'émet pas d'INFO, et ses assurances ne sont pas suspendues. `Unknown` importe un hook *introuvable*, et l'analyseur admet la limite. Le message nomme le nom importé (`useQuery`), pas l'alias local. La recherche se fait par provenance, (`paquet`, `nom`) :

Extrait 23.47 – `src/registry/summary.rs`, l. 52–60 — la clé des résumés

```

52 /// Key: `(package, hook_name)`. `package = None` for unscoped registrations
53 /// that match any import source (or hooks defined locally).
54 type SummaryKey = (Option<String>, String);
55
56 /// Registry mapping library hooks to their abstract summaries.
57 /// Lookup: `(package, name)` exact match first, then `(None, name)` fallback.
58 pub struct SummaryRegistry {
59     summaries: HashMap<SummaryKey, Box<dyn HookSummary>>,
60 }

```

Extrait 23.48 – `src/registry/summary.rs`, l. 166–177 — la recherche : paquet exact, puis non scopé

```

166 /// Look up a summary for `name` imported from `import_source`.
167 /// Tries the scoped entry first, then the unscoped fallback.
168 pub fn get(&self, name: &str, import_source: Option<&str>) -> Option<&dyn HookSummary> {
169     if let Some(src) = import_source {
170         let scoped: SummaryKey = (Some(src.to_string()), name.to_string());
171         if let Some(s) = self.summaries.get(&scoped) {
172             return Some(s.as_ref());
173         }
174     }
175     let unscoped: SummaryKey = (None, name.to_string());
176     self.summaries.get(&unscoped).map(|s| s.as_ref())
177 }

```

23.10.2 Le trait et le registre commun

Extrait 23.49 – `src/registry/summary.rs`, l. 10–31 — le contrat abstrait d'un hook sans source (début)

```

10 pub trait HookSummary: Send + Sync {
11     fn name(&self) -> &str;
12
13     /// Compute the abstract return value of this hook given abstract arg values.
14     /// Default: `Top` (most conservative completely unknown).
15     fn summarize(&self, _args: &[StateValue]) -> StateValue {
16         StateValue::top()
17     }
18
19     /// Named members of the returned object that carry a contract of their own.
20     ///
21     /// This is the shape libraries actually publish: `useForm()` promises
22     /// `setValue` is the same function at every render and promises nothing

```

```

23    /// about `formState`. Everything not listed reads  $\top$ , so a member added to
24    /// a library after this table was written is never credited with a
25    /// stability nobody wrote down.
26    ///
27    /// Empty means the hook's return has no per-member contract, and
28    /// [ `Self::summarize` ] alone answers for it.
29    fn members(&self) -> &'static [(&'static str, SummaryValue)] {
30        &[]
31    }

```

Le trait se termine par deux prédicats, `held_across_updates` et `navigates`, tous deux `false` par défaut (`src/registry/summary.rs`, l. 33–47).

Un résumé répond à quatre questions. Le moteur les lit dans cet ordre (`src/engine/fixpoint.rs`, l. 958–977) : `navigates` (le résultat est une fonction qui navigue, comme `useNavigate()`); `held_across_updates` (le résultat ne bouge que sur navigation et reste immobile à travers la boucle de re-rendu automatique, #161); `members` (contrats par membre nommé ou par position de tuple); enfin `summarize`, $\top$ par défaut. La consommation dans `expand_custom_hooks`, avec le retagage du marqueur d'appel qui garde la `HookEntry` visible aux règles des hooks, est décrite au chapitre 8. Le commentaire de tête de cette branche (`src/engine/fixpoint.rs`, l. 944–948) dit encore que les hooks connus « are removed from the hooks vec », alors que le code les garde délibérément quelques lignes plus bas.

Le contenu de `new_with_common()` (`src/registry/summary.rs`, l. 77–150), dans l'ordre d'insertion :

1. des hooks connus, en $\top$, pour `@tanstack/react-query`, `react-router-dom/react-router`, `next/navigation`, `next/router` et `next/compat/router` (`useRouter`);
2. des résumés Held : `useSearchParams`, `useParams`, `usePathname`, `useSelectedLayoutSegment(s)` de `next/navigation`; `useParams`, `useLocation`, `useMatch` de `React Router`;
3. pour `React Router`, `useSearchParams` en tuple (`"0" → Held`, `"1" → Navigator { stable: false }`) et `useNavigate` en `NavigatorSummary`;
4. des formes par membre : `react-hook-form` (`useForm/useFormContext`, treize membres `StableRef` et `handleSubmit` en `Wrapper { stable: true }`); `useRouter` de `Next` (`push`, `replace`, `back`, `forward` en `Navigator { stable: true }`; `refresh`, `prefetch` en `StableRef`); `swr` (`mutate stable`); `@mantine/form` (`onSubmit` en `Wrapper { stable: false }`); `jotai` (position "1" de `useAtom stable`, `useSetAtom stable`, `useAtomValue/useStore` en $\top$);
5. `use-debounce`, connu et $\top$.

Rappel React : identité des valeurs de bibliothèque

Les deps d'un hook sont comparées par `Object.is`. Ce qu'une bibliothèque *documente*, c'est qu'un membre garde son identité d'un rendu à l'autre (`setValue` de `react-hook-form`, `router.push` de `Next`, `mutate` de `SWR`, le setter de `jotai`), et c'est pourquoi les applications l'omettent des deps. Une valeur d'URL (`useSearchParams`, `useParams`, `usePathname`) ne change que sur navigation, que la boucle de re-rendu automatique ne peut pas provoquer, sauf si un corps qu'elle exécute navigue lui-même.

Décision : Revendiquer une stabilité est le sens dangereux

Trois politiques découlent de la soundness. (1) Enregistrer un hook en $\top$ n'est pas le modéliser. C'est déclarer qu'il est *connu*, ce qui distingue une imprécision délibérée de l'INFO « hook introuvable » (`src/registry/summary.rs`, l. 69–76). (2) `useContext` et les hooks React non modélisés (`useActionState`, `useOptimistic`, `useTransition`, `useId`, `useFormStatus`) sont *volontairement* laissés inconnus : l'INFO signale un trou du moteur qu'il faut combler

en les modélisant, et un résumé $\top$ le cacherait (ADR-26 §5). (3) Un contrat par membre ne crédite que les membres listés. Les autres restent $\top$, si bien qu'un membre ajouté à la bibliothèque après l'écriture de la table n'hérite jamais d'une stabilité que personne n'a écrite. Une revendication de stabilité est ce qui fait *perdre* des findings. Faute de preuve suffisante, `useDebounceCallback` reste donc $\top$: le corpus n'offrait qu'un site pour vérifier la stabilité documentée.

Piège

Un `HookSummary` écrit par un plugin et qui renverrait une valeur précise par `summarize`, `null` par exemple, est projeté par le moteur sur trois classes seulement, `StableRef`, `UnstableRef` ou $\top$ (`src/engine/fixpoint.rs`, l. 1169–1181). De plus, l'argument `args` reçoit toujours `&[]`. Enfin, `new_with_common()` insère certaines clés deux fois (`(next/navigation, usePathname)` en $\top$ puis en `Held`, `useRouter` en $\top$ puis en forme) : le dernier `HashMap::insert` gagne, et l'ordre des insertions fait partie du contrat.

Le registre à clé composite. Les registres de composants, de hooks et de fonctions partagent une primitive, `KeyedRegistry<V>`, une table $(\text{PathBuf}, \text{Symbol}) \rightarrow V$ (`src/registry/keyed.rs`, l. 15–23). La clé composite évite les collisions entre deux fichiers qui définissent le même nom (ADR-13). La recherche héritée par nom seul, `get_by_name`, prend la première clé dans l'ordre trié. C'est elle qui retrouve `useData` dans le run `-project plain` du section 23.5, où l'alias n'est pas résolu mais où le hook a été découvert. Entre deux homonymes, ce repli choisit la première clé triée, qui n'est pas forcément la bonne (exercice 23.7). Ce fichier n'a pas de tests (#17), et il ne faut pas confondre ce registre avec le `RuleRegistry` des règles.

Résumé du chapitre

- La **CLI** est une coquille mince (`src/cli/`) : clap, fusion de la configuration, overrides, écriture des flux. Toute la composition d'un check vit dans `driver::run_check`, partagée octet pour octet avec l'hôte WASM à travers la couture `FileSystem`. Les codes de sortie sont 0, 1 (seuil `-fail-on`) et 2 (usage ou entrée-sortie).
- La **configuration** `reactant.config.json` est du JSONC, validé bruyamment. Les drapeaux comblent en premier et la configuration complète (`CheckArgsPartial::merge` : les listes remplacent, les booléens se combinent par OU, les options par `or`). Les overrides combinent "off", la résurrection par `-rule`, l'`allowlist`, `-ignore-rule` (qui refuse toujours) et un plafond de sévérité qui ne fait que baisser (*pin* $\sqcap$ *polarité*).
- La **découverte** lit le `.gitignore` du dépôt pour savoir quels répertoires sont générés (#137). La syntaxe incomprise fait lire davantage. `node_modules` et `.git` ne sont jamais parcourus.
- Les **projets** Vite et Next.js sont détectés par marqueur en remontant les ancêtres. Les alias viennent des seuls `tsconfig`, par `extends`, puis `references`, puis les ancêtres. Le résolveur suit la sémantique de TypeScript (exacts, plus long préfixe, puis `baseUrl`, qui ne peut qu'ajouter).
- Un fichier récupéré au parse est **analysé** ; un fichier perdu est toujours dit.
- **Pas de clean bill pour du code non lu** : alias non chargés, fichiers perdus et imports résolus hors de l'ensemble analysé retiennent le « ✓ » dans tous les formats. `-follow-imports` lit la fermeture, mais le rapport reste limité aux chemins nommés, les findings des fichiers suivis étant comptés et non montrés. Les spécificateurs non résolus et les `analysis-limit` ne sont pas des blind spots.

- Le **rendu humain** nomme le fichier de chaque position et groupe par localisation. Le **JSON v2** reste une ligne par composant ; `-fail-on` compte des attributions.
- Le **SummaryRegistry** déclare des hooks de bibliothèques *connus* (Held, Navigator, contrats par membre). Il n'est installé que par le driver.

Types et fichiers rencontrés :

- `src/cli/` : `Cli`, `CheckArgs`; `src/config.rs` : `ReactantConfig`, `RuleSetting`, `CheckArgsPartial`;
- `src/driver/` : `CheckOptions`, `CheckOutput`, `CheckReport`, `BlindSpot`, `Followed`, `LocationIndex`;
- `src/project/` : `ProjectKind`, `ProjectContext`, `TsconfigPaths`, `TsconfigPathsResolver`;
- `src/resolver/` : `FileSystem`, `GitignoreStack`, `ParseError`, `LoweredProgram`, `import_closure`;
- `src/registry/` : `HookSummary`, `SummaryRegistry`, `KeyedRegistry`.

Le chapitre suivant (chapitre 24) montre l'autre hôte du driver : le cœur WASM, le paquet npm et l'action GitHub.

23.11 Exercices

Exercice 23.1: Fichier ou répertoire

Depuis la racine du dépôt, prédire les fichiers analysés, les blind spots et le code de sortie de reactant `tests/fixtures/vite_project/src/App.tsx`, puis de reactant `tests/fixtures/vite_project/src`. Expliquer l'écart.

Solution

Dans le premier cas, seul un fichier est passé : la racine de projet est `."`, qui ne porte pas de marqueur, donc le run est `Plain`. L'alias `@/hooks/useData` n'est pas résolu et aucune arête n'est créée, donc il n'y a pas de blind spot. Le hook est absent du run. On obtient « ✓ 1 file(s) no issues found. » et le code 0, et seul `-info` montre l'`analysis-limit`. Dans le second cas, `src` est un répertoire : `locate` remonte à `vite_project`, qui porte `vite.config.ts`, puis charge les paths. Pas de rétrécissement, puisque la racine donnée n'est pas la racine de configuration. Deux fichiers sont lus, un `WARNING infinite-loop` est émis, et le code vaut 1. La détection utilise le premier *répertoire*, jamais le répertoire d'un fichier.

Exercice 23.2: Faire réapparaître missing-deps

Avec une configuration `"missing-deps": "off"`, donner des drapeaux qui rendent `missing-deps` visible. Donner ensuite une combinaison où il ne peut pas l'être, quoi que dise la configuration.

Solution

`-rule missing-deps` le ressuscite (`entry.off = setting.off && !rule.contains(name)`), mais pose une allowlist qui masque toutes les autres règles. Tout `-ignore-rule missing-deps` le masque de façon définitive, puisque la boucle sur `ignore_rule` vient après et force `off = true` (liste 23.11). Voir la tableau 23.2.

Exercice 23.3: Un .gitignore illisible

Écrire un `.gitignore` dont une ligne contient une classe non terminée (`gen[`) et qui devrait, selon l'auteur, exclure `gen1/`. Que fait `reactant-analyzer` ? Pourquoi est-ce le bon sens d'erreur ?

Solution

`class_end` ne trouve pas de `]`, et `glob_match` rend `false` : la règle ne reconnaît rien, et `gen1/` est parcouru (sauf si une autre règle l'exclut). L'erreur conduit à lire un répertoire de trop. Au pire, cela produit des faux positifs sur du code généré, jamais un faux négatif sur du code source (invariant 23.3).

Exercice 23.4: extends en tableau

Un `tsconfig.json` déclare `"baseUrl": "./a"` et `"extends": [". /p1.json", ". /p2.json"]`. `p1.json` ne déclare qu'un `"baseUrl": ". /b"`, `p2.json` déclare `"baseUrl": ". /c"` et des `paths`. Quel `TsconfigPaths paths_from_config` rend-il ?

Solution

Sans `paths` propres, on parcourt `extends`. `p1.json` rend `(b, [])`, dont la base est écrasée par la base propre `a` (`src/project/tsconfig.rs`, l. 229–232), puis le résultat est mis de côté dans `inherited_base`. `p2.json` rend `(c, motifs)`, dont la base est de même écrasée par `a`, et comme les motifs sont non vides, la fonction les rend. Résultat : `base_url = <dir>/a`, motifs de `p2.json`. Le `baseUrl` déclaré dans le fichier courant l'emporte toujours sur l'hérité (liste 23.31).

Exercice 23.5: Deux comptages

Sur `tests/fixtures/shared_hook_repeat`, pourquoi le résumé humain dit-il « 1 warning(s) » et le JSON `"warnings": 3` ? Lequel pilote `-fail-on` ? Pourquoi ADR-24 interdit-il de faire le groupage dans les règles ?

Solution

L'humain compte des localisations (`LocationIndex`), et les trois attributions partagent la clé (`infinite-loop`, `useShared.ts:8:2`, `message`). Le JSON et `-fail-on` comptent les attributions ligne par ligne (invariant 23.2). Dédupliquer dans les règles supprimerait des findings par consommateur, qui sont incomparables (`useStep(1)` contre `useStep(0)`) : ce serait un générateur de faux négatifs.

Exercice 23.6: Un résumé maison

Écrire (en Rust) un `HookSummary` pour un hook `useStableThing` qui renvoie une référence stable, l'enregistrer sans paquet, et expliquer pourquoi un `summarize` qui rendrait `StateValue::null()` serait lu comme `⊥` par le moteur.

Solution

```
1 struct StableThing;
2 impl HookSummary for StableThing {
3     fn name(&self) -> &str { "useStableThing" }
4     fn summarize(&self, _args: &[StateValue]) -> StateValue {
5         StateValue::reference(Stability::Stable)
6     }
7 }
8 registry.register(Box::new(StableThing)); // clé (None, "useStableThing")
```

Le registre doit ensuite être placé dans `Config::summary_registry` par l'appelant : le driver n'installe que `new_with_common()`. `state_value_to_summary_value` ne distingue que la référence stable, la référence instable et le reste. `null` tombe dans « le reste », c'est-à-dire `⊥` (`src/engine/fixpoint.rs`, l. 1172–1181). C'est `sound`, puisque `⊥` sur-approxime `null`, mais

imprécis.

Exercice 23.7: Un monorepo à deux tsconfig

Sous `/tmp/ch23/mono2`, `apps/web` porte un `vite.config.ts` et un `tsconfig.json` (`@/*` $\rightarrow$ `./src/*`), et `packages/ui` un `tsconfig.json` (`@/*` $\rightarrow$ `./lib/*`). Chaque paquet définit un hook `useThing` : celui de `apps/web/src` ne fait rien, celui de `packages/ui/lib` boucle. `App` et `Button` importent chacun `"@/useThing"`. On observe :

```
$ reactant packages/ui          # ! 1 warning(s) across 2 file(s).  exit=1 (Button)
$ reactant apps/web packages    # ✓ 4 file(s) no issues found.      exit=0
$ reactant apps packages        # ✓ 5 file(s) no issues found.      exit=0
```

(lignes de résumé seulement). Expliquer les trois résultats. Comment `ScopedResolver` réglerait-il le problème, et pourquoi le driver ne peut-il pas l'utiliser seul ?

Solution

Premier run. `packages/ui` n'a pas de marqueur, donc le run est `Plain` et l'alias n'est pas résolu. Le seul `useThing` du run est trouvé par son nom seul (`get_by_name`), et c'est le bon. *Deuxième run.* Le premier répertoire, `apps/web`, fixe un contexte `Vite`, avec un résolveur, celui des paths de `apps/web`. Le `"@/useThing"` de `Button.tsx` est résolu vers `apps/web/src/useThing.ts`, un fichier qui existe, si bien que la résolution est *fausse* et non *absente*. Le hook qui boucle n'est pas inliné dans `Button`, et aucun blind spot n'est posé, puisque le fichier ciblé est lu. *Troisième run.* `apps` n'a pas de marqueur, donc le run est `Plain`. Les deux alias restent non résolus, et `get_by_name` prend la première clé triée, (`apps/web/src/useThing.ts`, `useThing`), pour les deux composants. Les deux derniers runs impriment un clean bill sur un programme qui boucle : ce sont des faux négatifs silencieux. `ScopedResolver` routerait par fichier *importateur*, la portée de plus long préfixe gagnant (`scope("packages/ui", ...)`, `scope("apps/web", ...)`), ce qui corrige le deuxième run. Mais `build_context` ne construit qu'un `ProjectContext`, avec un seul `resolver`, et le driver n'offre aucun moyen d'en composer plusieurs. Il faut passer par l'API plugin (`docs/plugins.md`). Le troisième run relève en plus du repli par nom (ADR-13), qui choisit arbitrairement entre homonymes. Au commit `e67b10a`, aucune issue ouverte ne décrit ce faux négatif silencieux propre au premier répertoire qui fixe l'unique résolveur d'un run multi-répertoires : `#48` porte sur les spécificateurs `@workspace/*`, non résolus faute d'algorithme de `workspace`, et `#59` (fermée) documente seulement l'absence d'un `ImportResolver` par fichier, sans constater le cas où un alias se résout *à tort* vers un fichier réel d'un autre paquet.

Chapitre 24

Distribution : WASM, npm, GitHub Action, CI

Objectifs du chapitre.

- Comprendre comment un même cœur Rust est livré sous deux formes : un binaire natif (`cargo`) et un module WebAssembly enveloppé dans un paquet npm, exécutable sous Node comme dans un navigateur.
- Savoir à quoi servent les features `Cargo cli` et `schema-gen`, et pourquoi le build WASM s'en passe.
- Connaître le principe « l'hôte n'est jamais une frontière de confiance » et sa mise en œuvre dans `crates/reactant-wasm/src/lib.rs`.
- Parcourir le paquet npm : `bin`, API programmatique, chargement du cœur, `reactant packs build`, `types pack.d.ts` générés.
- Savoir comment la parité native/WASM est testée, et connaître les divergences qui subsistent.
- Utiliser l'Action GitHub en connaissant ses pièges (épingle de version, `args`, angles morts), et lire les jobs de la CI du dépôt.

Prérequis : chapitre 23 (la CLI, la configuration, `run_check`, la couture `FileSystem` et `MemFileSystem`, les blind spots, le rapport JSON v2) ; chapitre 22 (`packs`, `load_pack`, `validation`). Le chapitre 25 reprend la mesure corpus, dont on ne voit ici que le workflow.

Les chapitres précédents décrivent un analyseur qu'on lance par `cargo run - check`. Ce n'est pas ainsi que ses utilisateurs le rencontrent. Un développeur React tape `npx reactant-analyzer` ; une équipe ajoute trois lignes à un workflow GitHub ; un outil en ligne veut analyser le tampon d'un éditeur dans le navigateur. Aucun de ces hôtes n'a de chaîne Rust installée, et deux d'entre eux n'ont même pas de système de fichiers.

La question de ce chapitre est donc : comment livrer *le même* analyseur à des hôtes aussi différents, sans que son comportement bifurque ? Car la soundness (définition 3.8) est une propriété d'un programme précis. Si l'hôte npm réimplémentait la découverte des fichiers ou la validation des packs, un faux négatif pourrait exister sous `npx` et pas sous `cargo`, et aucun test du moteur ne le verrait. La réponse du projet tient en une phrase de l'ADR-22 §6 : un seul artefact WASM, un hôte JS réduit au transport, un cœur qui revalide tout ce qu'il reçoit.

On procède du plus bas au plus haut : la compilation (section 24.1), le cœur WASM (section 24.2), le paquet npm (section 24.3), les tests de parité (section 24.4), l'Action GitHub (section 24.5), la CI (section 24.6), et enfin ce qui est versionné et stable (section 24.7).

24.1 Deux artefacts, un cœur : les features Cargo

24.1.1 Un workspace, deux crates

Le dépôt est un workspace Cargo à deux membres : la crate racine `reactant` (bibliothèque et binaire) et `crates/reactant-wasm`. La crate racine sépare nettement ce qui est propre à l'hôte natif de ce qui a un sens partout :

Extrait 24.1 – `Cargo.toml`, l. 13–33 — workspace, features et cibles de la crate racine

```

13 [workspace]
14 members = [".", "crates/reactant-wasm"]
15
16 [features]
17 default = ["cli", "schema-gen"]
18 # The CLI (clap and friends) is binary-only; the wasm build uses
19 # `default-features = false` to keep the dependency tree to oxc + serde.
20 cli = ["dep:clap"]
21 # JSON Schema generation (ADR-022 §6): only the `reactant schemas` emitter
22 # needs schemars – validation is serde + hand checks, so the wasm build
23 # drops it.
24 schema-gen = ["dep:schemars"]
25
26 [[bin]]
27 name = "reactant"
28 path = "src/main.rs"
29 required-features = ["cli"]
30
31 [lib]
32 name = "reactant"
33 path = "src/lib.rs"
```

Deux features, toutes deux actives par défaut. La feature `cli` tire `clap`, le parseur d'arguments, et le binaire `reactant` l'exige (`required-features`). La feature `schema-gen` tire `schemars`, qui ne sert qu'à émettre les JSON Schemas (commande `reactant schemas`) : la validation elle-même est faite par `serde` et par des vérifications écrites à la main (chapitre 22), elle n'a pas besoin de `schemars`. Dans le code, `schema-gen` n'apparaît que sous forme d'attributs conditionnels, `#[cfg_attr(feature = "schema-gen", derive(JsonSchema))]`, dans `src/config.rs` et `src/rules/declarative/schema.rs`. La feature `cli` n'apparaît nulle part dans la bibliothèque : le module `cli` est déclaré par `src/main.rs`, pas par `src/lib.rs` (chapitre 23). C'est cette séparation, voulue par l'ADR-16 §1, qui rend possible un build sans CLI.

La crate WASM est minuscule :

Extrait 24.2 – `crates/reactant-wasm/Cargo.toml`, l. 1–15 — le manifeste de la crate WASM

```

1 [package]
2 name = "reactant-wasm"
3 version = "0.6.0"
4 edition = "2024"
5 publish = false
6
7 [lib]
8 crate-type = ["cdylib"]
9
10 [dependencies]
11 reactant = { path = "../..", default-features = false }
12 wasm-bindgen = "0.2"
13 serde = { version = "1", features = ["derive"] }
14 serde_json = "1"
```



```
15 console_error_panic_hook = "0.1"
```

`crate-type = ["cdylib"]` produit une bibliothèque dynamique, qui devient un module `.wasm` pour la cible `wasm32-unknown-unknown`. La dépendance `reactant = { path = "../..", default-features = false }` retire `clap` et `schemars`. Il ne reste que ce que le commentaire de `Cargo.toml` promet : « *oxc + serde* ». On le vérifie avec `cargo tree` :

```
$ cargo tree -p reactant-wasm -e normal --depth 2 --offline
```

```
reactant-wasm v0.6.0 (/home/rboudrouss/reactant-analyzer/crates/reactant-wasm)
+-- console_error_panic_hook v0.1.7
|   +-- cfg-if v1.0.4
|   +-- wasm-bindgen v0.2.127
+-- reactant v0.6.0 (/home/rboudrouss/reactant-analyzer)
|   +-- oxc_allocator v0.138.0
|   +-- oxc_ast v0.138.0
|   +-- oxc_parser v0.138.0
|   +-- oxc_span v0.138.0
|   +-- serde v1.0.228
|   +-- serde_json v1.0.150
|   +-- serde_path_to_error v0.1.20
+-- serde v1.0.228 (*)
+-- serde_json v1.0.150 (*)
+-- wasm-bindgen v0.2.127 (*)
```

Le cœur ne dépend d’aucune bibliothèque d’entrées-sorties, de threads ou de `mmap`. C’est ce que l’ADR-22 §6 rappelle pour justifier la faisabilité : `oxc` est du Rust pur, et toutes les lectures de fichiers passent par les traits de l’ADR-13, dont la couture `FileSystem` (chapitre 23).

Piège

Rien, dans la suite de tests ordinaire, ne compile la bibliothèque sans ses features par défaut : `cargo test` active `cli` et `schema-gen`. Un `use clap::...` égaré dans `src/` passerait donc tous les tests et ne casserait que le build WASM. C’est pourquoi la CI a un job dédié, `no-default-features`, qui lance `cargo clippy -lib -no-default-features` (section 24.6).

24.1.2 Construire le paquet : `npm/build.sh`

Le script `npm/build.sh` fabrique tout ce que le paquet `npm` publie et que le dépôt ne versionne pas (`npm/dist/`, `npm/schemas/` et `npm/LICENSE` sont listés dans `npm/.gitignore`) :

Extrait 24.3 – `npm/build.sh`, l. 13–34 — les étapes de construction du paquet `npm`

```
13 rustup target add wasm32-unknown-unknown
14
15 # 8 MiB stack: the analysis is recursive (bounded inline depth, oxc's
16 # recursive descent) and the wasm default of 1 MiB is tighter than native.
17 RUSTFLAGS='-C link-arg=-zstack-size=8388608' \
18 cargo build -p reactant-wasm --release --target wasm32-unknown-unknown
19
20 # `--target web`, not `nodejs`, for BOTH hosts: the emitted `.wasm` is
21 # byte-identical across wasm-bindgen targets, and the web glue runs under Node
22 # as well (the bytes come from readFileSync + initSync instead of fetch – see
23 # npm/lib/core-node.js). One artifact, one glue, both hosts; a second target
24 # would ship a second copy of the same 3.5 MB module.
25 wasm-bindgen target/wasm32-unknown-unknown/release/reactant_wasm.wasm \
26   --target web --out-dir npm/dist
27
28 # Schemas from the native binary – same commit compiles the shipped core and
29 # the published schemas, so they validate the same types by construction.
```

```

30 cargo run --quiet --release -- schemas --out npm/schemas
31
32 # lib/pack.d.ts from the freshly-written pack.schema.json (ADR-023 §5): the
33 # authoring types compile from the same schemars output as the validator.
34 node npm/scripts/gen-pack-dts.js

```

Quatre décisions se lisent dans les commentaires.

1. **Une pile de 8 Mio.** L'analyse est récursive (descente récursive d'oxc, inlining à profondeur bornée), et la pile par défaut d'un module WASM, 1 Mio, est plus étroite que celle d'un processus natif. Sans ce drapeau d'édition de liens, un fichier profondément imbriqué pourrait faire déborder la pile sous WASM et pas en natif.
2. **Une seule cible `wasm-bindgen`, `web`, pour les deux hôtes.** Le `.wasm` émis est identique octet pour octet quelle que soit la cible, seule la glue JS diffère, et la glue `web` fonctionne aussi sous Node à condition de lui fournir les octets (section 24.3). Une seconde cible aurait livré une seconde copie du même module.
3. **Les schémas viennent du binaire natif du même commit.** `cargo run - schemas -out npm/schemas` les engendre à partir des types Rust que le validateur utilise ; schémas publiés et validation ne peuvent donc pas diverger.
4. **`lib/pack.d.ts` est engendré à partir de ce schéma** (section 24.3.6).

La commande `schemas` du binaire natif est la seule consommatrice de `schemars`. Lancée avec `-out /tmp/ch24/schemas`, elle écrit `pack.schema.json` et `reactant-config.schema.json` ; le premier est identique (vérifié avec `cmp`) à celui de `npm/schemas/` et à l'instantané versionné `docs/schemas/pack.schema.json`.

Remarque

Le commentaire de `npm/build.sh` parle d'un module de « 3.5 MB », l'ADR-23 §5 de 788 Ko compressés ; reconstruit à `e67b10a`, le `.wasm` pèse 4 389 894 octets, 1 164 013 après `gzip`. Aucun test ne borne cette taille.

24.2 Le cœur WASM et l'hôte non fiable

24.2.1 Le problème : un hôte qui pourrait mentir

Considérons une équipe dont le `reactant.config.json` déclare `"packs": ["@acme/react-rules"]` et règle la sévérité d'une règle de ce pack. Sous `npx`, c'est du JavaScript qui trouve le paquet `@acme/react-rules` dans `node_modules`, lit son JSON et le transmet au cœur. Si ce code JS « aidait » en validant lui-même le pack, ou en interprétant la configuration JSONC, il y aurait deux validateurs. Le jour où ils divergent, un pack refusé en natif serait accepté sous npm, ou une clé de configuration ignorée d'un côté seulement. L'ADR-22 §6 tranche par un principe, que l'en-tête du crate énonce :

Extrait 24.4 – `crates/reactant-wasm/src/lib.rs`, l. 1–10 — l'en-tête du cœur WASM

```

1  /// WASM entry point for the reactant analyzer (ADR-022 §6).
2  ///
3  /// The JS host is pure transport: it reads argv, the raw config text, the
4  /// pack bytes (resolved via `require.resolve`) and a superset file map, and
5  /// writes back the returned streams and exit code. **The host is never a
6  /// trust boundary**: the config and every pack are re-parsed and re-validated
7  /// here, and discovery/project detection/tsconfig chains/alias resolution all
8  /// run inside the engine over the in-memory filesystem – the exact same
9  /// `driver::run_check` composition as the native CLI, so behavior cannot
10 /// fork.

```

Définition 24.1: Hôte de transport

Un *hôte* de reactant-analyzer est un frontend qui invoque le driver : le binaire natif, ou le wrapper JS du paquet npm (chapitre 23). Un hôte est dit *de transport* lorsqu'il se borne à *acheminer des octets* : argv, texte brut de la configuration, texte des packs, contenu des fichiers ; et à restituer les flux et le code de sortie qu'on lui rend. Il ne décide d'aucune sémantique : ni quels fichiers analyser, ni quel projet détecter, ni si un pack ou une configuration est valide.

Invariant 24.1: L'hôte n'est jamais une frontière de confiance

Tout ce que le cœur WASM reçoit de l'hôte JS (enveloppe, configuration, packs, options) est reparsé et revalidé dans le cœur, par les mêmes fonctions que la CLI native : `config::parse`, `declarative::load_pack`, `RuleRegistry::register`, `RuleRegistry::set_overrides`, puis `driver::run_check`. La découverte des fichiers, la détection du projet, les chaînes `tsconfig` et la résolution d'alias tournent dans le moteur au-dessus d'un `MemFileSystem`. Un hôte altéré peut rendre un run indisponible (erreur d'usage), pas le rendre moins sound.

La contrepartie est assumée : l'hôte doit envoyer un *sur-ensemble* des fichiers que le moteur lira, puisqu'un fichier absent de la table est, pour le moteur, un fichier qui n'existe pas.

24.2.2 Les cinq exports

Le crate expose cinq fonctions à JavaScript, par `#[wasm_bindgen]`. Toutes échangent des chaînes JSON : aucune structure Rust ne traverse la frontière.

Nom JS	Fonction Rust	Rôle
<code>hostConstants()</code>	<code>host_constants</code>	sert <code>prunedDirs</code> , <code>sourceExtensions</code> , <code>configFileName</code> à l'hôte (crates/reactant-wasm/src/lib.rs, l. 24–39)
<code>helpPage(color)</code>	<code>help_page</code>	la page d'aide, mêmes octets que <code>reactant help</code> (l. 41–48)
<code>packSpecs(text)</code>	<code>pack_specs</code>	la liste packs d'une configuration, extraite par le vrai parseur (l. 50–60)
<code>validatePack(json)</code>	<code>validate_pack</code>	<code>load_pack</code> sans analyse, pour packs build (l. 62–83)
<code>run(inputJson)</code>	<code>run → run_inner</code>	les commandes <code>check</code> , <code>rules</code> , <code>explain</code> (l. 163–260)

TABLE 24.1 – Les exports `wasm_bindgen` du cœur.

Les trois premiers existent pour que l'hôte n'ait *rien* à savoir par lui-même. `hostConstants` lui dit quels répertoires il peut ignorer sans jamais cacher un fichier au moteur (`HOST_PRUNED_DIRS` : `node_modules`, `.git`, `.next`, `src/resolver/mod.rs`, l. 533) et quelles extensions sont des sources. `packSpecs` répond à la seule question que l'hôte doit se poser *avant* l'analyse, à savoir quels packs aller chercher, sans qu'il ait à parser du JSONC :

Extrait 24.5 – `crates/reactant-wasm/src/lib.rs`, l. 50–60 — l'hôte ne parse jamais la configuration

```

50 /// Extract the `packs` list from raw config text – the one config field the
51 /// host needs *before* calling [`run`] (to resolve pack files). Parsed by
52 /// the same validator as the real run, so the host never interprets JSONC.
53 /// Returns `{ "ok": [specs...] }` or `{ "error": "..."}`.
54 #[wasm_bindgen(js_name = packSpecs)]
55 pub fn pack_specs(config_text: &str) -> String {
56     match config::parse(config_text, Path::new(config::CONFIG_FILE_NAME)) {
57         Ok(cfg) => serde_json::json!({ "ok": cfg.packs }).to_string(),
58         Err(e) => serde_json::json!({ "error": e.to_string() }).to_string(),
59     }

```

```
60 }
```

24.2.3 L’enveloppe d’entrée

run reçoit un document JSON, l’enveloppe, désérialisé dans le type Input :

Extrait 24.6 – crates/reactant-wasm/src/lib.rs, l. 85–105 — l’enveloppe d’entrée

```
85 #[derive(Deserialize)]
86 #[serde(rename_all = "camelCase", deny_unknown_fields)]
87 struct Input {
88     /// "check" | "rules" | "explain"
89     command: String,
90     #[serde(default)]
91     explain_rule: Option<String>,
92     /// Positional path arguments, replayed identically to a CLI invocation.
93     #[serde(default)]
94     paths: Vec<String>,
95     /// Superset file map (path → text), cwd-relative POSIX paths.
96     #[serde(default)]
97     files: BTreeMap<String, String>,
98     /// Raw `reactant.config.json` text, if the host found one.
99     #[serde(default)]
100    config: Option<String>,
101    /// Resolved packs: bytes in, validation here.
102    #[serde(default)]
103    packs: Vec<PackInput>,
104    options: Options,
105 }
```

Champ par champ : command vaut "check", "rules" ou "explain"; explain_rule nomme la règle à documenter; paths rejoue les arguments positionnels d’une invocation CLI; files est la table « chemin relatif POSIX → texte » qui deviendra le MemFileSystem; config est le texte brut de reactant.config.json; packs est une liste de PackInput { name, json }, où name est la spécification écrite dans la configuration (pour les messages d’erreur) et json le texte du pack. options est un miroir un pour un des drapeaux de check (crates/reactant-wasm/src/lib.rs, l. 114–145) : info, showClean, trace, verbose, allRoots, entry, excludeDir, followImports, format, failOn, project, rule, ignoreRule, color.

Input et Options portent deny_unknown_fields : une clé mal orthographiée est une erreur d’usage, pas une option silencieusement ignorée. PackInput ne le porte pas. Le champ options est obligatoire (il n’a pas de #[serde(default)]), les autres ont une valeur par défaut. La sortie est Output { exitCode, stdout, stderr }, sérialisée elle aussi en JSON.

24.2.4 run_inner : tout refaire dans le cœur

Extrait 24.7 – crates/reactant-wasm/src/lib.rs, l. 170–212 — la revalidation de la configuration et des packs

```
170 fn run_inner(input_json: &str) -> Output {
171     let input: Input = match serde_json::from_str(input_json) {
172         Ok(i) => i,
173         Err(e) => return usage(format!("[error] invalid input envelope: {e}\n")),
174     };
175
176     // -- Config: re-parsed and validated here, never trusted from the host ----
177     let cfg = match &input.config {
```

```

178     Some(text) => match config::parse(text, Path::new(config::CONFIG_FILE_NAME)) {
179         Ok(c) => c,
180         Err(e) => return usage(format!("[error] {e}\n")),
181     },
182     None => config::ReactantConfig::default(),
183 };
184
185 // -- Registry: natives + re-validated packs (config order, §8) -----
186 let mut registry = RuleRegistry::natives();
187 let options_by_id: BTreeMap<String, serde_json::Map<String, serde_json::Value>> = cfg
188     .rules
189     .iter()
190     .filter(|(k, s)| !s.options.is_empty())
191     .map(|(k, s)| (k.clone(), s.options.clone()))
192     .collect();
193 let mut warnings = String::new();
194 for pack in &input.packs {
195     let load = match declarative::load_pack(&pack.json, &options_by_id) {
196         Ok(l) => l,
197         Err(e) => return usage(format!("[error] pack `{}`: {e}\n", pack.name)),
198     };
199     for w in &load.warnings {
200         warnings.push_str(&format!("[warn] rule `{}`: {}{}\n", w.rule, w.message));
201     }
202     for rule in load.rules {
203         if let Err(e) = registry.register(rule.rule, rule.doc) {
204             return usage(format!("[error] pack `{}`: {e}\n", pack.name));
205         }
206     }
207 }
208 let overrides =
209     config::resolve_overrides(&cfg, &input.options.rule, &input.options.ignore_rule);
210 if let Err(e) = registry.set_overrides(overrides) {
211     return usage(format!("[error] {e}\n"));
212 }

```

On reconnaît, étape par étape, la boucle native (src/cli/config_load.rs, l. 45–89, chapitre 23) : registre des natives, table des options par règle extraite de la configuration, packs dans l'ordre de la configuration (ADR-22 §8), load_pack sur chacun, avertissements préfixés [warn] rule, enregistrement, puis overrides. Une seule différence de forme : le cœur ne lit pas de fichier, il reçoit les octets. La commande check construit ensuite le système de fichiers en mémoire et appelle le driver commun :

Extrait 24.8 – crates/reactant-wasm/src/lib.rs, l. 237–251 — la commande check sous WASM (extrait)

```

237     let fs = Arc::new(MemFileSystem::from_map(
238         input
239             .files
240             .iter()
241             .map(|(p, s)| (PathBuf::from(p), s.clone()))),
242     ));
243     let paths = if input.paths.is_empty() {
244         vec![".".to_string()]
245     } else {
246         input.paths.clone()
247     };
248     // Identity display: the host already sends cwd-relative paths.
249     let out = driver::run_check(fs, &paths, &registry, &opts, &|p: &Path| {
250         p.display().to_string()
251     });

```

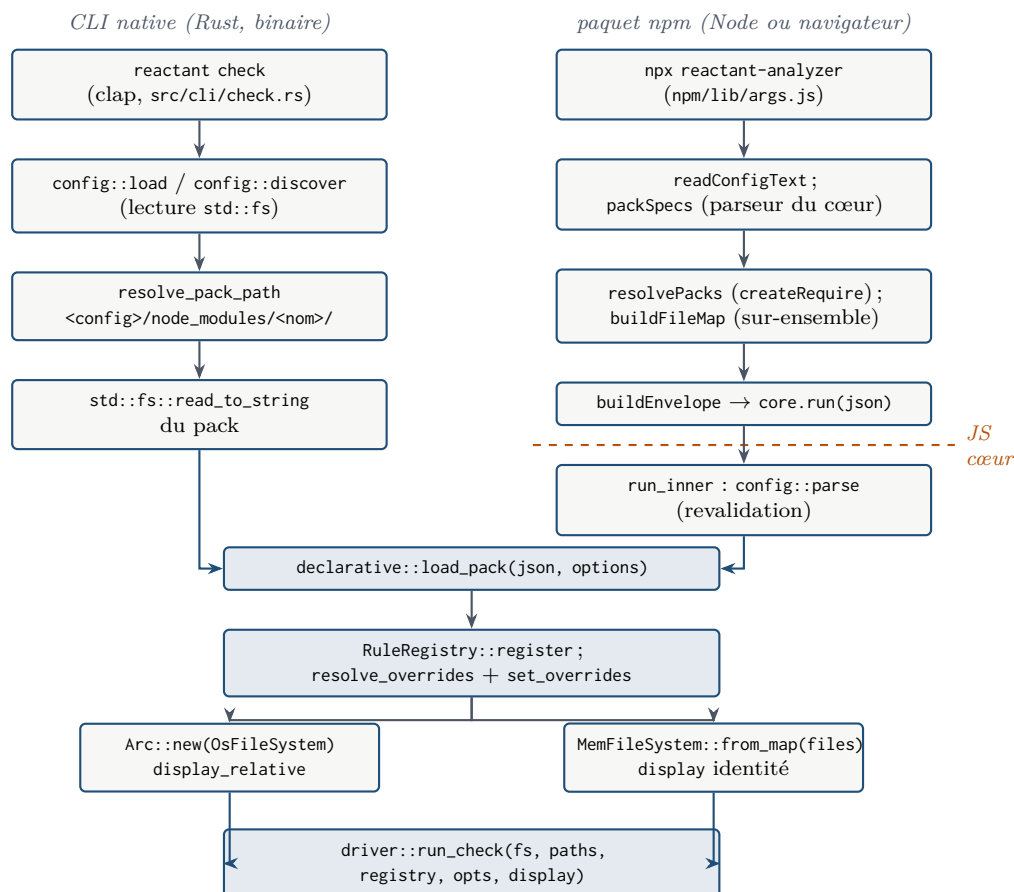


FIGURE 24.1 – Séquences de chargement native (à gauche) et WASM (à droite). Au-dessus du tronc commun, chaque hôte ne fait que trouver et lire des octets ; le tronc commun (en bleu) est le même code Rust, appelé depuis `src/cli/` d’un côté et depuis `run_inner` de l’autre. Seul le choix de l’implémentation de `FileSystem` et de la fonction d’affichage diffère.

Deux détails comptent pour la parité. La fonction `display` est l’identité, là où la CLI native relativise les chemins au répertoire courant : l’hôte JS envoie déjà des chemins relatifs au répertoire courant, si bien que les deux affichages coïncident à condition de lancer les deux hôtes depuis le même répertoire. Et `check_options` (`crates/reactant-wasm/src/lib.rs`, l. 262–323) convertit les chaînes `format`, `fail0n`, `project` (une valeur inconnue donne `[error] unknown format ‘x’, code 2`), remplit un `CheckArgsPartial` et le fusionne avec la configuration par `partial.merge(cfg)`, le même mécanisme de précedence que la CLI native.

Enfin, `run` installe `console_error_panic_hook` avant `run_inner` (`crates/reactant-wasm/src/lib.rs`, l. 163–168) : une panique du cœur apparaît dans la console JS avec son message, au lieu d’une instruction `unreachable` opaque.

24.2.5 Deux séquences de chargement, un point de convergence

La figure 24.1 met côte à côte les deux chemins, de l’invocation à l’analyse. Ils ne partagent rien au-dessus de la ligne pointillée de droite, qui sépare le JavaScript du cœur, et tout en dessous.

Décision : ADR-022 §6 — WASM seul dans npm, résolution par l'hôte, validation par le cœur

Décision. Le paquet npm contient un seul artefact `.wasm` et un wrapper JS mince (« le modèle prettier ») : `npx reactant` a le même comportement, bit pour bit, sur toutes les plateformes. L'hôte résout les noms de packs npm (champ `"reactant"` de leur `package.json`, via `require.resolve`) et transmet des chaînes JSON ; le cœur revalide chaque pack reçu ; les schémas sont engendrés depuis les types Rust.

Alternatives refusées. Des binaires natifs par plateforme (le modèle esbuild/biome) : ce serait une optimisation *additive* et future, car les publier plus tard ne casse rien alors que les retirer casserait. Un algorithme de résolution Node complet en Rust : la CLI native se contente de `node_modules/<nom>/` et des chemins relatifs. Une validation côté JS : ce serait un second validateur, donc une bifurcation possible.

24.3 Le paquet npm

24.3.1 Ce que publie npm/package.json

Le paquet s'appelle `reactant-analyzer`, version 0.6.0, en modules ESM (`"type": "module"`), et exige Node 20.19 ou plus récent (engines). Il publie quatre répertoires, `bin/`, `lib/`, `dist/` et `schemas/`, et expose :

Extrait 24.9 – `npm/package.json`, l. 9–21 — points d'entrée du paquet npm

```

9   "bin": { "reactant": "bin/reactant.js" },
10  "main": "./lib/index.js",
11  "types": "./lib/index.d.ts",
12  "exports": {
13    ".": {
14      "browser": { "types": "./lib/browser.d.ts", "default": "./lib/browser.js" },
15      "default": { "types": "./lib/index.d.ts", "default": "./lib/index.js" }
16    },
17    "./browser": { "types": "./lib/browser.d.ts", "default": "./lib/browser.js" },
18    "./wasm": "./dist/reactant_wasm.js",
19    "./lib/pack": { "types": "./lib/pack.d.ts" },
20    "./package.json": "./package.json"
21  },

```

La condition `browser` de l'export `"."` fait qu'un bundler (Vite, webpack) choisit automatiquement `lib/browser.js`, tandis que Node prend `lib/index.js`. `./wasm` expose la glue brute, `./lib/pack` n'expose que des types (pour l'auteur d'un pack), et `bin` fait de `npx reactant-analyzer` (ou `npx reactant` une fois le paquet installé) une commande.

24.3.2 L'architecture en couches de npm/lib/

Les fichiers de `npm/lib/` forment trois couches. La règle qui les organise est qu'une option ne puisse pas exister pour un hôte JS et pas pour l'autre.

La CLI JS n'est pas un chemin parallèle à l'API : elle *consomme* l'API. `lib/cli.js` parse `argv`, appelle `projectInput` (partagé avec `analyzeProject`) puis le même `run` qu'un script appellerait. Et `run` passe toujours par `buildEnvelope`. L'en-tête de `npm/lib/envelope.js` précise que sa validation est seulement *ergonomique* : une option mal orthographiée doit produire un message lisible plutôt qu'une erreur serde venue du WASM, mais « *It is NOT a trust boundary* ». C'est l'invariant 24.1, vu du côté JS.

Fichier	Rôle
<i>Entrées</i>	
bin/reactant.js	process.exitCode = await main(argv) : le code de sortie est affecté, pas process.exit, pour que la sortie tamponnée soit vidée.
lib/cli.js	help, schemas, packs build; sinon projectInput puis api.run.
lib/index.js	entrée Node : l'API, plus analyzeProject (lire un projet sur disque).
lib/browser.js	entrée navigateur : la même API, plus initWasm, sans aucun import node:.
<i>Cœur de l'API, sans node:</i>	
lib/api.js	makeApi(loadCore) : run, analyze, rules, explain, help, packSpecs, validatePack, hostConstants.
lib/envelope.js	buildEnvelope : construit l'enveloppe, refuse les clés inconnues; UsageError.
lib/args.js	argv → options, un pour un avec l'enveloppe; drapeau inconnu ⇒ code 2.
<i>Transport disque et chargement du cœur</i>	
lib/project.js	projectInput : configuration, packs résolus, table des fichiers.
lib/host.js	readConfigText, resolvePacks, buildFileMap, toPosix.
lib/core-node.js	import() de la glue, puis initSync sur les octets lus par readFileSync.
lib/core-web.js	init() : fetch du .wasm voisin de la glue, ou source fournie par initWasm.

TABLE 24.2 – Les modules de npm/lib/ (hors fichiers .d.ts).

24.3.3 Un seul module, deux façons de l'amorcer

La glue `-target web` de `wasm-bindgen` attend normalement un `fetch`. Sous Node, le module `core-node.js` lui donne directement les octets :

Extrait 24.10 – `npm/lib/core-node.js`, l. 45–55 — amorcer la glue web sous Node

```

45 async function tryImport() {
46   let glue;
47   try {
48     glue = await import(GLUE);
49   } catch (e) {
50     if (e && (e.code === "ERR_MODULE_NOT_FOUND" || e.code === "MODULE_NOT_FOUND")) return null;
51     throw e;
52   }
53   glue.initSync({ module: readFileSync(WASM) });
54   return glue;
55 }
```

Le résultat est mis en cache une fois par processus. `tryLoadCore` renvoie `null` au lieu de lever une exception si le bundle manque (arbre de développement où `npm/build.sh` n'a pas tourné); seul `packs build` s'en sert, pour dégrader en « écrire sans valider ». Dans le navigateur, `npm/lib/core-web.js` appelle `init()`, qui va chercher le `.wasm` posé à côté de la glue, ce que Vite, webpack et un `<script type="module">` résolvent correctement; `initWasm(source)` permet de fournir une URL, une `Response`, des octets ou un `WebAssembly.Module` déjà compilé.

Exemple 24.1: Analyser un tampon d'éditeur dans le navigateur

Le navigateur n'a pas de disque : son seul mode est la table de fichiers virtuelle. `docs/usage.md` donne cet usage :

Extrait 24.11 – Usage navigateur de l'API (extrait de `docs/usage.md`, élagué)

```

1 import { analyze } from "reactant-analyzer/browser";
2
3 const { report } = await analyze({
4   files: { "src/App.tsx": editorBuffer }, // cwd-relative POSIX keys
```



```

5   config: { rules: { "derived-state": "off" } },
6 });

```

config peut être un objet : `normalizeConfig` le sérialise, et le cœur reste le seul à le parser. `report` est le document JSON v2 tel quel (chapitre 23). Les findings sont un résultat, jamais une exception ; seule une erreur d'usage (code 2) lève une `UsageError`. Le commentaire de `npm/lib/browser.js` ajoute que l'analyse est synchrone une fois le module prêt et monopolise le thread : un hôte interactif doit la lancer dans un Worker.

24.3.4 La table des fichiers : un sur-ensemble

L'hôte Node doit construire `files` sans rien décider. `buildFileMap` parcourt les chemins donnés et retient toute source (extensions servies par le cœur), plus les fichiers qui orientent la découverte et la détection de projet :

Extrait 24.12 – `npm/lib/host.js`, l. 95–103 — les fichiers que l'hôte charge

```

95  const wanted = (name) =>
96    constants.sourceExtensions.some((ext) => name.endsWith(`.${ext}`)) ||
97    name === ".gitignore" ||
98    // Not analyzed: it is one of the two markers that bound the upward
99    // ".gitignore" search, and ".git" never reaches the map.
100   name === "package.json" ||
101   /^tsconfig[/\\]*\.json$/.test(name) ||
102   /^vite\.config\.(ts|js|mjs|mts)$/.test(name) ||
103   /^next\.config\.(ts|js|mjs|cjs|mts)$/.test(name);

```

Il n'élague que `prunedDirs`. Le commentaire des lignes 81–85 en donne la raison : depuis #137, les exclusions du moteur dépendent du `.gitignore` de l'arbre, et un hôte qui appliquerait d'avance une liste de noms cacherait des fichiers que le moteur voulait lire, « *a superset walk that is not a superset* ». Sous `-follow-imports`, le parcours s'élargit au projet englobant (l'ancêtre le plus proche portant un `package.json` ou un `.git`), pour la raison de l'#138 : sous WASM, le moteur ferme les arêtes d'import à l'intérieur de sa vue du système de fichiers, et un fichier que l'hôte n'a pas chargé est indiscernable d'un fichier inexistant. La fermeture reviendrait vide, « *followed 0* », sans le dire.

24.3.5 Résoudre les packs : deux implémentations

La résolution d'un nom de pack est la seule sémantique que l'ADR-22 laisse à l'hôte, et c'est là qu'on trouve la divergence la plus nette entre les deux hôtes. Côté natif :

Extrait 24.13 – `src/cli/config_load.rs`, l. 98–108 — la résolution native d'un nom de pack

```

98  fn resolve_pack_path(spec: &str, base: &Path) -> Result<PathBuf, String> {
99    if spec.starts_with('.') || spec.starts_with('/') || spec.ends_with(".json") {
100      return Ok(base.join(spec));
101    }
102    let pkg_dir = base.join("node_modules").join(spec);
103    if !pkg_dir.is_dir() {
104      return Err(format!(
105        "not installed: {} does not exist",
106        pkg_dir.display()
107      ));
108    }

```

Côté npm :

Extrait 24.14 – npm/lib/host.js, l. 55–74 — la résolution npm d'un nom de pack

```

55 export function resolvePacks(specs, configDir) {
56   return specs.map((spec) => {
57     let packPath;
58     if (spec.startsWith(".") || path.isAbsolute(spec) || spec.endsWith(".json")) {
59       packPath = path.resolve(configDir, spec);
60     } else {
61       const req = createRequire(path.join(path.resolve(configDir), "package.json"));
62       let pkgPath;
63       try {
64         pkgPath = req.resolve(`${spec}/package.json`);
65       } catch {
66         throw new Error(`pack \`${spec}\`: not installed (require.resolve failed)`);
67       }
68       const pkg = JSON.parse(fs.readFileSync(pkgPath, "utf8"));
69       const rel = pkg.reactant ?? "pack.json";
70       packPath = path.join(path.dirname(pkgPath), rel);
71     }
72     return { name: spec, json: fs.readFileSync(packPath, "utf8") };
73   });
74 }

```

Les deux traitent comme un chemin toute spécification qui commence par ., qui est absolue ou qui finit par .json, relative au répertoire du fichier de configuration. Pour un nom npm, les deux lisent le champ "reactant" du package.json du paquet, à défaut pack.json. Mais le natif ne regarde que <répertoire de la config>/node_modules/<nom>/, alors que createRequire(...).resolve applique l'algorithme de Node, qui remonte les node_modules des répertoires parents.

Piège

Dans un monorepo dont le gestionnaire de paquets « hisse » les dépendances à la racine, un pack installé seulement dans node_modules/ de la racine est trouvé par npx reactant lancé avec la configuration d'un sous-paquet, mais pas par le binaire natif. Celui-ci s'arrête sur une erreur d'usage, qu'on observe sur un arbre sans node_modules :

```
[error] pack `@acme/react-rules`: not installed: a/node_modules/@acme/react-rules does not exist
```

(code de sortie 2, observé). Le comportement npm dans le monorepo, lui, est déduit du code (Node absent), non exécuté. L'ADR-22 §6 assume ce choix (« pas de résolution Node complète en Rust »). La direction de l'écart est la bonne : le natif refuse bruyamment, il n'exécute pas moins de règles en silence.

Les messages diffèrent aussi d'un détail. Le natif ajoute le chemin résolu, [error] pack 'spec' (chemin): ... (src/cli/config_load.rs, l. 76); le cœur WASM, qui n'a reçu que des octets, écrit [error] pack 'spec': ... (crates/reactant-wasm/src/lib.rs, l. 197). Le message interne, celui de PackError, est le même. On observe la forme native sur un pack incomplet :

```
[error] pack `./bad.pack.json` (b/./bad.pack.json): at `rules[0]`: missing field `docs`
```

24.3.6 reactant packs build : écrire un pack en JS, committer du JSON

Un pack est un fichier JSON inerte (chapitre 22). Pour une équipe, écrire à la main dix règles presque identiques est fastidieux ; elle voudrait une boucle sur une table. L'ADR-23 §5 accorde cette commodité sans rien céder sur l'inertie : le pack peut être *écrit* en JS ou TS, il est évalué *au moment de l'écriture*, et c'est le JSON produit qu'on committe. L'analyseur, natif ou WASM, ne consomme jamais que du JSON. Le fixture npm/test/fixtures/team.pack.cjs montre

la forme visée :

Extrait 24.15 – `npm/test/fixtures/team.pack.cjs`, l. 12–29 — engendrer une règle par ligne d’une table

```
12 // One banned-hook rule per entry – the table is the source of truth.
13 const BANNED = [
14   { hook: "useLayoutEffect", wrapper: "useSafelayoutEffect" },
15   { hook: "useInsertionEffect", wrapper: "useStyleEffect" },
16 ];
17
18 const bannedRules = BANNED.map(({ hook, wrapper }) => ({
19   id: `no-direct-${hook.replace(/[A-Z]/g, (c) => `-${c.toLowerCase()}`).slice(4)}`,
20   docs: {
21     description: `${hook} called directly instead of ${wrapper}`,
22     why: `${hook} warns during SSR; ${wrapper} swaps it for useEffect on the server`,
23     fix: `import ${wrapper} and call it instead`,
24   },
25   severity: "warning",
26   anchor: { relation: "hook_calls", kind: "effect" },
27   guards: [{ kind: "origin", of: "anchor", hook: [hook], direct: true }],
28   message: `${hook} is called directly – use ${wrapper}`,
29 }));
```

La commande n’existe que dans le paquet npm : elle a besoin de Node pour évaluer le module. Le binaire natif la mentionne dans son aide (« compile a JS rule pack to pack.json (npm package only) ») et, donné hors du dépôt, répond [error] no command or path named ‘packs’ (code 2, observé) — mais à la racine du dépôt, où `packs/` existe, il l’analyse comme un chemin (`src/driver/mod.rs`, l. 115–123) et répond tout autre chose. L’algorithme de `npm/lib/packs.js` tient en trois temps. `evaluate` importe le module par `import(pathToFileURL(abs))`, qui charge ESM comme CommonJS (et `.ts` sur un Node qui retire les types) ; il déballe un éventuel double default de l’interopérabilité ESM/CJS, appelle la valeur exportée si c’est une fonction, et exige un objet. Puis vient l’écriture :

Extrait 24.16 – `npm/lib/packs.js`, l. 72–91 — valider par le cœur avant d’écrire

```
72 const json = JSON.stringify(pack, null, 2) + "\n";
73
74 if (wasm && typeof wasm.validatePack === "function") {
75   const verdict = JSON.parse(wasm.validatePack(json));
76   if (verdict.error) {
77     io.err.write(`[error] ${input}: ${verdict.error}\n`);
78     return 2;
79   }
80   for (const w of verdict.ok.warnings) {
81     io.err.write(`[warning] ${w}\n`);
82   }
83   const target = outputPath ?? defaultOut(input);
84   fs.mkdirSync(path.dirname(path.resolve(target)), { recursive: true });
85   fs.writeFileSync(target, json);
86   io.out.write(
87     `wrote ${target}: pack \`${verdict.ok.name}\`, ${verdict.ok.rules.length} rule(s): ` +
88     `${verdict.ok.rules.join(", ")}\n`,
89   );
90   return 0;
91 }
```

`validatePack` est ici le `load_pack` du cœur (tableau 24.1) : le générateur ne peut pas bénir un pack que le cœur refuserait. Si le bundle est trop ancien pour exposer `validatePack`, le fichier est écrit quand même, mais avec la mention `not validated` : un silence se lirait « validé ». Le

JSON est indenté, pour que le fichier committé soit lisible dans un diff.

Piège

`validate_pack` appelle `load_pack` avec une table d'options *vide* (`crates/reactant-wasm/src/lib.rs`, l. 70). `packs build` ne peut donc pas détecter une option consommateur invalide écrite dans `reactant.config.json` ; c'est le check suivant qui la refusera.

Décision : ADR-023 §5 — JS/TS compilé en JSON, Starlark rejeté

Décision. La voie communautaire pour composer des règles est d'écrire le pack en JS/TS et de committer le JSON engendré. Un pack JS est de l'exécution de code arbitraire, comme une configuration ESLint ; committer le JSON confine ce risque à la machine de l'auteur, et la CI n'exécute que des données.

Alternatives refusées. Un Tier B en Starlark : la crate `starlark` ne compilait ni pour l'hôte ni pour `wasm32`, et elle alourdissait une distribution qui se veut « `oxc + serde` ». Exécuter le pack JS à chaque analyse : ce serait réintroduire du code d'auteur dans le run, et faire bifurquer les deux hôtes, puisque le natif n'a pas de moteur JS.

Les types de l'auteur. Le fichier `npm/lib/pack.d.ts` (448 lignes) est *engendré* par le script `gen-pack-dts.js` de `npm/scripts/` à partir de `schemas/pack.schema.json`, donc des mêmes types Rust que le validateur. Le générateur traduit le JSON Schema 2020-12 de `schemars` : `$ref` en nom de type, `enum/const` en unions de littéraux, `oneOf/anyOf` en unions dédoublonnées, objets en interfaces avec `?` pour les champs non requis. L'auteur annote son module par `/** @type {import("reactant-analyzer/lib/pack").Pack} */` et son éditeur complète les ancres et les gardes. Le mode `-check` échoue si le fichier committé n'est plus à jour ; la CI l'exécute (section 24.4).

24.4 La parité native/WASM, testée

Le principe de l'invariant 24.1 ne vaut que s'il est vérifié. Il l'est à trois niveaux, du plus interne au plus externe.

1. Les deux systèmes de fichiers (Rust). `tests/memfs_parity.rs` lance `driver::run_check` une fois sur `OsFileSystem`, une fois sur un `MemFileSystem` chargé des mêmes fichiers (y compris `tsconfig` et `vite.config`, « *the superset walk the WASM host performs* »), et exige l'égalité de `stdout`, `stderr` et du code de sortie sur trois fixtures, `vite_project`, `next_project` et `cross_file_hook` (`tests/memfs_parity.rs`, l. 44–117). Ce test tourne dans `cargo test` : il ne demande ni WASM ni Node.

2. Le binaire contre le paquet publié (shell). `npm/test/smoke.sh` définit une fonction `compare` qui lance la même ligne de commande avec le binaire natif (`target/release/reactant`) puis avec `node npm/bin/reactant.js`, depuis le même répertoire courant et sous `NO_COLOR=1`, et exige l'égalité octet pour octet de la sortie combinée et du code de sortie (`npm/test/smoke.sh`, l. 13–26). Elle est appliquée à treize invocations :

Extrait 24.17 – `npm/test/smoke.sh`, l. 28–40 — les treize invocations comparées

```
28 compare "check human"  check tests/fixtures/vite_project --fail-on never
29 compare "check json"   check tests/fixtures/vite_project --format json --fail-on never
30 compare "check trace"  check tests/fixtures/vite_project --trace --info --show-clean --fail-on never
31 compare "exit code"    check tests/fixtures/vite_project --fail-on warning
```

```

32 compare "next human"    check tests/fixtures/next_project --fail-on never
33 compare "next json"     check tests/fixtures/next_project --format json --info --fail-on never
34 compare "pack project"  check tests/fixtures/pack_project --format json --fail-on never
35 compare "config off"    check tests/fixtures/config_project --config tests/fixtures/config_project/off.json
    ↪ --format json --fail-on never
36 compare "rules"         rules
37 compare "explain"       explain infinite-loop
38 compare "help"          help
39 compare "unknown cmd"   chekc
40 compare "explain pack"  explain team/effect-writes-own-dep --config tests/fixtures/pack_project/reactant.
    ↪ config.json

```

La liste couvre les deux rendus, `-trace`, le code de sortie sous `-fail-on warning`, un projet Next.js, un pack, une configuration qui éteint des règles, les commandes `rules`, `explain` et `help`, une commande mal tapée (`chekc`, à laquelle les deux hôtes doivent répondre par la même page d'aide précédée de `[error] no command or path named 'chekc'`) et l'`explain` d'une règle de pack. Le script enchaîne sur `npm/test/packs.sh` et `npm/test/api.js`.

3. L'écriture des packs et l'API (Node). `npm/test/packs.sh` compile `team.pack.cjs` puis `team.pack.mjs` (qui réexporte le même objet) et exige un JSON identique octet pour octet à `team.pack.expected.json`, puis lance `gen-pack-dts.js -check`. De son côté, le test Rust `js_authored_pack_output_loads_in_the_core` charge ce JSON attendu par `load_pack` et vérifie ses trois identifiants et l'absence d'avertissement (`tests/declarative.rs`, l. 953–967). `npm/test/api.js` vérifie d'abord que `analyzeProject` et `check -format json` produisent le même document, puis couvre ce que la CLI n'exerce jamais : table virtuelle, erreurs d'usage levées, absence d'import `node` : dans le graphe de l'entrée navigateur.

Piège

Comparer des *comptes* ne suffit pas. Lors du travail sur l'§138, une vérification de parité comparait le nombre de findings des deux hôtes, égal des deux côtés ; comparer les fichiers nommés montrait immédiatement la divergence (« *comparing counts is not comparing behaviour* »). D'où la comparaison octet pour octet de `smoke.sh`, et le déterminisme du binaire, testé par ailleurs (chapitre 25), sans lequel une telle comparaison serait impossible.

Divergences connues. Trois écarts subsistent, tous documentés dans le code ou constatés à sa lecture.

- Un fichier illisible est simplement absent de la table WASM, alors que le natif enregistre une erreur d'entrée-sortie pour un fichier passé explicitement (« *documented v1 divergence* », `npm/lib/host.js`, l. 109).
- La résolution des noms de packs et la forme des messages d'erreur de pack (section 24.3.5).
- **Le drapeau `-rule-option` n'existe qu'en natif.** Le commit 6e45e83 l'a ajouté à la CLI (`src/cli/check.rs`, l. 85–88, appliqué l. 149–159 par `apply_rule_options`) et à `docs/usage.md`, mais ni l'enveloppe `Options` du cœur, ni `npm/lib/args.js`, ni `npm/lib/envelope.js` ne le connaissent. Sous `npx`, `-rule-option state-lifted-too-high:minDepth=2` tombe donc dans la branche `unknown flag d'args.js` (code 2), et `smoke.sh` ne contient aucune invocation qui l'utilise. La page d'aide commune ne le mentionne pas non plus. Les options d'une règle restent réglables sous `npm` par la configuration. Constat de lecture, non exécuté (Node absent) ; aucune issue ouverte sur cet écart.

24.5 L'Action GitHub

24.5.1 Trois lignes dans un workflow

Le dépôt est aussi une Action GitHub. Le README.md en donne l'usage :

Extrait 24.18 – Usage de l'Action (extrait du README.md, élagué)

```
1   - uses: actions/checkout@v4
2   - uses: rboudrouss/reactant-analyzer@v0.6.0
3     with:
4       path: .           # default: the whole checkout
5       fail-on: error    # warnings annotate the PR without failing it
```

action.yml déclare une action *composite* à une seule étape, qui exécute `node "${{ github.action_path }}/scripts/gh-action.mjs"` en lui passant les entrées par des variables `INPUT_*`. Cinq entrées : `path` (défaut `.`, chemins séparés par des espaces), `fail-on` (défaut `warning`), `config` (défaut vide : découverte automatique), `version` (défaut `latest`) et `args` (drapeaux supplémentaires). Six sorties : `errors`, `warnings`, `infos`, `exit-code`, `blind-spots` et `json` (chemin du rapport complet).

24.5.2 Ce que fait scripts/gh-action.mjs

Extrait 24.19 – scripts/gh-action.mjs, l. 18-25 — l'invocation de npx

```
18 const args = [
19   "--yes", `reactant-analyzer@${version}`,
20   "check", ...paths, "--format", "json", "--fail-on", failOn,
21 ];
22 if (config) args.push("--config", config);
23 args.push(...extra);
24
25 const res = spawnSync("npx", args, { encoding: "utf8", maxBuffer: 64 * 1024 * 1024 });
```

Le script ne lance pas le code du dépôt : il lance le *paquet npm publié*, `npx -yes reactant-analyzer@<version> check ... -format json -fail-on <niveau>`. Les packs viennent donc de la configuration du dépôt analysé, résolus par l'hôte npm, et leurs `ERROR` font échouer `-fail-on error` exactement comme celles des règles natives. Si le lancement échoue, le script émet `::error::` et sort avec 2 ; si la sortie standard n'est pas un document JSON (erreur d'usage, code 2), il émet `::error::reactant exited ... without a JSON report` et relaie le code.

Il transforme ensuite le rapport v2 en *commandes de workflow*, les lignes `::error ...::` que GitHub affiche comme annotations sur le fichier et la ligne :

Extrait 24.20 – scripts/gh-action.mjs, l. 46-62 — une annotation par erreur de parse et par diagnostic

```
46 for (const pe of report.parse_errors ?? []) {
47   console.log(`::warning file=${escProp(pe.file)},title=parse error::${escData(pe.message)} - file skipped,
48     ↳ findings inside it are not proven absent`);
49 }
50 for (const d of report.diagnostics ?? []) {
51   const props = [`title=${escProp(d.rule)}`];
52   if (d.file) props.push(`file=${escProp(d.file)}`);
53   if (d.line != null) {
54     props.push(`line=${d.line}`);
55     props.push(`col=${(d.col ?? 0) + 1}`); // JSON col is 0-indexed, annotations are 1-indexed
```

```

56   }
57   const msg = [
58     `[$${d.component}] $${d.message}`,
59     ...(d.notes ?? []).map((n) => `→ $${n.message}`),
60   ].join("\n");
61   console.log(`::${level[d.severity]} ?? "notice" $${props.join(",")}:$${escData(msg)}`);
62 }

```

Trois détails. La sévérité info devient notice, seul troisième niveau que GitHub connaisse. La colonne est incrémentée, car le JSON est indexé à partir de 0 et les annotations à partir de 1. Les notes de la chaîne de témoins (définition 17.2) deviennent des lignes préfixées par →. Les valeurs sont échappées (% , retour chariot et saut de ligne dans le message; en plus : et , dans les propriétés).

Exemple 24.2: Les annotations du fixture pack_project

Sur tests/fixtures/pack_project, le rapport JSON observé (reactant check tests/fixtures/pack_project -format json -fail-on never) contient deux diagnostics : un WARNING infinite-loop en "line": 7, "col": 2 avec deux notes, et un ERROR team/effect-writes-own-dep en "line": 8, "col": 4 sans note. Appliquées à la main, les lignes 50–62 du script produiraient les deux commandes suivantes (messages coupés) :

Extrait 24.21 – Annotations reconstituées à la main à partir de scripts/gh-action.mjs (non exécuté, élagué)

```

1 ::warning title=infinite-loop,file=tests/fixtures/pack_project/App.tsx,line=7,col=3::[App] this effect
  ↳ keeps pushing state `n` (...) Potential infinite render loop%A→ state `n` is written here%0
  ↳ A→ the abstract value of state `n` kept growing and was widened at iteration 3
2 ::error title=team/effect-writes-own-dep,file=tests/fixtures/pack_project/App.tsx,line=8,col=5::[App]
  ↳ this effect writes `n`, which is in its own deps array

```

Les colonnes 2 et 4 du JSON deviennent 3 et 5; reconstruction manuelle vérifiée contre escData/escProp et la boucle des l. 41–62, sur le JSON ci-dessus.

24.5.3 Les angles morts dans une CI

Le contrat central du chapitre 23, pas de « clean bill » pour du code non lu, doit survivre au passage par GitHub, où l'on ne lit souvent que le résumé du job. Le script émet donc un ::warning title=not analyzed:: par angle mort, et ajoute au résumé du job un bloc « **Not a clean bill** » qui dit que les comptes sont une borne inférieure (scripts/gh-action.mjs, l. 64–77). La sortie blind-spots le formule en toutes lettres :

Extrait 24.22 – action.yml, l. 53–60 — la sortie blind-spots

```

53 blind-spots:
54   description: >-
55     Number of things the run knows it did not read (unloadable aliases,
56     dropped files, imports resolved outside the analysed set). Non-zero
57     means `errors` and `warnings` are a lower bound, not a verdict. It never
58     affects the exit code – gate on it yourself if your project wants a run
59     that reads everything or fails.
60   value: ${{ steps.check.outputs.blind-spots }}

```

Un angle mort n'est pas un finding : il ne change jamais le code de sortie. Une équipe qui veut un run exhaustif ou rien doit tester steps.<id>.outputs.blind-spots elle-même.

24.5.4 Pièges de l’Action

Piège

L’épingle ne fixe pas l’analyseur. `uses: rboudrouss/reactant-analyzer@v0.6.0` fixe la version de `action.yml` et de `gh-action.mjs`, pas celle de l’analyseur. L’analyseur vient de `npx reactant-analyzer@${version}`, avec version à latest par défaut (`scripts/gh-action.mjs`, l. 14, l. 18–21). Un workflow épinglé au tag, sans entrée `version:`, exécute donc la dernière version publiée, et ses résultats peuvent changer du jour au lendemain sans aucun commit. Pour un résultat reproductible, il faut écrire aussi `version: 0.6.0`.

Piège

L’entrée `args` est coupée sur les espaces (`input("args").split(/\s+/)`) : une valeur ne peut pas contenir d’espace, ce que la description de l’entrée dit (`action.yml`, l. 33–38). Il en va de même de `path`. Et comme l’Action lance le paquet npm, `args` n’accepte que les drapeaux de l’hôte npm : `-rule-option`, propre au natif (section 24.4), y serait refusé.

24.6 La CI du dépôt

Le dépôt a deux workflows et une configuration Dependabot. `.github/workflows/ci.yml` tourne sur chaque push vers main, chaque pull request et à la demande, avec un groupe de concurrency par ref qui annule le run précédent. Ses sept jobs tournent tous sur `ubuntu-latest` :

Job	Commande	Ce qu’il garde
fmt	<code>cargo fmt --all --check</code>	le style ; « <i>the cheapest gate — fail fast</i> »
clippy	<code>cargo clippy --workspace</code> <code>--all-targets --all-features --locked</code> <code>-D warnings</code>	les lints, tests compris
test	<code>cargo test --workspace --all-features</code> <code>--locked, RUSTFLAGS=-D warnings</code>	toute la suite Rust, dont <code>tests/memfs_parity.rs</code>
no-default-features	<code>cargo clippy --lib</code> <code>--no-default-features --locked -D warnings</code>	la configuration du build WASM, sans clap ni schemars
docs	<code>cargo doc --workspace --no-deps</code> <code>--all-features --locked,</code> <code>RUSTDOCFLAGS=-D warnings</code>	les liens de documentation interne, qu’un renommage casse en silence
wasm-parity	<code>cargo build --release --locked;</code> <code>npm/build.sh; npm/test/smoke.sh</code>	l’artefact publié : parité octet pour octet, packs build, <code>pack.d.ts</code> , API
action	<code>node --check scripts/gh-action.mjs;</code> <code>actionlint</code>	la syntaxe du script de l’Action et la validité des workflows

TABLE 24.3 – Les jobs de `.github/workflows/ci.yml`.

Le job `wasm-parity` est le seul à construire ce que les utilisateurs npm exécutent réellement : son commentaire le dit, « *The published artifact is the wasm core, not the native binary* » (`.github/workflows/ci.yml`, l. 77–79). Il installe `wasm-bindgen-cli` à la version exacte que fixe `Cargo.lock` :

Extrait 24.23 – `.github/workflows/ci.yml`, l. 94–106 — épingler `wasm-bindgen-cli` sur `Cargo.lock`

```

94     # Must match the wasm-bindgen version in Cargo.lock, or the generated
95     # glue and the .wasm disagree at load time.
96     - name: Install wasm-bindgen-cli
97       run: |
```



```

98     version=$(grep -A1 '^name = "wasm-bindgen"$' Cargo.lock \
99         | grep '^version' | head -1 | cut -d'"' -f2)
100     echo "pinning wasm-bindgen-cli $version"
101     cargo install wasm-bindgen-cli --version "$version" --locked
102     - run: cargo build --release --locked
103     - run: npm/build.sh
104     # Byte-for-byte wasm↔native comparison, plus `packs build` byte-identity
105     # and `gen-pack-dts.js --check` (npm/test/packs.sh).
106     - run: npm/test/smoke.sh

```

Faute de quoi la glue engendrée et le `.wasm` ne s'accorderaient pas au chargement. Le job action existe parce que rien d'autre ne lit `action.yml` : c'est une action composite consommée par d'autres dépôts.

Le workflow corpus. `.github/workflows/corpus.yml` (job `measure`) rejoue l'analyseur sur les quatorze dépôts épinglés du corpus et compare le résultat à `docs/corpus-baseline.json`. Il ne tourne ni sur les pull requests ni la nuit : un run complet dure environ 13 minutes sur environ 40 000 fichiers, et le corpus, épinglé commit par commit, ne peut pas dériver seul. Il se déclenche sur les pushes vers `main` qui touchent l'analyseur (`src/**`, `crates/**`, `packs/**`, `Cargo.lock`, la baseline, le script de clonage ou le workflow lui-même) et à la demande. Ce workflow est un instrument de mesure plus qu'une porte de livraison ; il est décrit au chapitre 25. Enfin, `.github/dependabot.yml` propose chaque mois les mises à jour Cargo (les crates `oxc_*` groupées) et celles des actions GitHub.

24.7 Versions et stabilité

24.7.1 Une version, cinq endroits

La version 0.6.0 est écrite à la main à cinq endroits, que le commit de release `adfe677` (« *release: v0.6.0* ») modifie ensemble : `Cargo.toml`, `crates/reactant-wasm/Cargo.toml`, `npm/package.json`, `.claude-plugin/plugin.json` et l'épingle de l'Action dans `README.md` (plus `Cargo.lock`, par voie de conséquence). Les tags `v0.1.0` à `v0.6.0` marquent les releases. Aucun test du dépôt ne vérifie que ces numéros restent alignés (ni `CARGO_PKG_VERSION` ni `npm/package.json` n'apparaissent dans `tests/` ou `src/`). Aucun workflow n'exécute `npm publish` : `wasm-parity` construit le paquet et le teste, sans le publier.

24.7.2 Ce qui est un contrat

Pour un consommateur, trois formats sont des contrats versionnés, et le reste ne l'est pas.

- **Le rapport JSON** porte `"version": 2` (`src/driver/json.rs`, l. 268). L'ADR-16 §4 en fait des DTO propres à la sortie, précisément pour que le schéma reste stable à travers les refontes de l'IR ; la `v2` a changé le sens de `file` et ajouté `blind_spots`, qu'un consommateur `v2` plus ancien peut ignorer. L'API JS ne définit pas de second format : `analyze` rend ce document tel quel, car « *the wire schema is the stability contract* » (`npm/lib/api.js`).
- **Les codes de sortie** : 0 propre, 1 findings au niveau de `-fail-on` ou au-dessus, 2 erreur d'usage ou d'entrée-sortie (ADR-16 §5). Un `tsconfig` manquant n'est qu'un avertissement, et un `INFO` ne change jamais le code. L'Action et le workflow corpus s'appuient sur cette distinction (le second lance `-fail-on never` pour que seul un code 2 le fasse échouer).
- **Le format de pack** porte `schemaVersion`, dont « *only '1' exists* » ; les schémas publiés dans `npm/schemas/` décrivent ce format et celui de la configuration.

Les findings eux-mêmes ne sont pas stables : chaque amélioration du moteur les déplace, et c'est la baseline du corpus qui mesure ces déplacements (chapitre 25).

Piège

Homonymie : l'ADR-017 ne parle pas de versions. L'ADR-17, « *Versioned reference stability* », n'a rien à voir avec les versions du logiciel. Elle raffine la composante de stabilité référentielle du domaine abstrait : *Versioned* y désigne une référence qui ne change que lorsque certaines deps changent (définition 10.3). La stabilité *de l'interface* (CLI, JSON, codes de sortie) relève de l'ADR-16 ; la distribution relève de l'ADR-22 §6 et de l'ADR-23 §5.

Résumé du chapitre

- Un seul cœur Rust, deux artefacts. La feature `cli` (`clap`) et la feature `schema-gen` (`schemars`) sont actives par défaut et retirées par la crate `crates/reactant-wasm` (`default-features = false`), dont les dépendances se réduisent à `oxc`, `serde` et `wasm-bindgen`. Le job CI `no-default-features` garde cette configuration.
- `npm/build.sh` construit un unique `.wasm` (pile de 8 Mio, cible web pour Node comme pour le navigateur), les schémas par le binaire natif du même commit, et `lib/pack.d.ts` depuis ces schémas.
- L'hôte JS est un hôte *de transport* (définition 24.1) et jamais une frontière de confiance (invariant 24.1) : `run_inner` reparse la configuration, recharge chaque pack par `load_pack`, installe les overrides et appelle le même driver : `run_check` que la CLI, sur un `MemFileSystem`. L'hôte envoie un sur-ensemble des fichiers.
- Le paquet npm `reactant-analyzer` a une CLI qui consomme son API, une entrée Node et une entrée navigateur au même comportement, et `packs build`, qui évalue un pack JS au moment de l'écriture, le valide par le cœur et écrit le JSON qu'on committe.
- La parité se teste à trois niveaux : `tests/memfs_parity.rs` (deux systèmes de fichiers), `npm/test/smoke.sh` (treize invocations comparées octet pour octet), `packs.sh` et `api.js`. Divergences restantes : fichier illisible, résolution des noms de packs dans un monorepo, forme des messages d'erreur de pack, et `-rule-option` absent de l'hôte npm.
- L'Action GitHub exécute le paquet npm publié, transforme le rapport v2 en annotations (colonne +1, `info` → `notice`) et rend visibles les angles morts sans toucher au code de sortie. Son épingle ne fixe pas l'analyseur : il faut une entrée `version`.
- Les contrats stables sont le JSON v2, les codes de sortie 0/1/2 (ADR-16) et le format de pack `schemaVersion: 1`. L'ADR-17 n'en fait pas partie, c'est une homonymie.

Fichiers rencontrés : `Cargo.toml` ; le crate `crates/reactant-wasm/` ; dans `npm/`, `package.json`, `build.sh`, `bin/`, `lib/`, `scripts/` et `test/` ; `action.yml` et `scripts/gh-action.mjs` ; `.github/` ; `tests/memfs_parity.rs`. Le chapitre 25 décrit la méthode de mesure qu'outille le workflow corpus : `tests`, `corpus`, `baseline` et `campagnes`.

24.8 Exercices

Exercice 24.1: Ajouter un drapeau à check

On veut ajouter un drapeau booléen `-quiet` à `check`. Énumérer tous les endroits à modifier pour que les deux hôtes le connaissent, et dire quel test échouerait si l'on oubliait le côté npm. Appliquer la même grille à `-rule-option`.

Solution

Côté natif : la structure clap de `src/cli/check.rs`, puis `CheckArgsPartial` et `CheckOptions` (et la configuration si le drapeau `y` a un équivalent). Côté cœur WASM : `Options` et `check_options` dans `crates/reactant-wasm/src/lib.rs`. Côté JS : `BOOL_FLAGS` dans `npm/lib/args.js`, `BOOL_OPTIONS` dans `npm/lib/envelope.js` et les types de `npm/lib/types.d.ts`. Côté aide : `driver::run_help`. Aucun test n'échoue automatiquement si le drapeau manque côté npm, tant que `npm/test/smoke.sh` ne contient pas d'invocation qui l'utilise : c'est exactement la situation de `-rule-option`, présent dans `src/cli/check.rs` et absent de `Options`, `d'args.js` et de `smoke.sh`. Il faut donc ajouter aussi une ligne `compare` à `smoke.sh`.

Exercice 24.2: Pourquoi la cible web sous Node ?

`npm/build.sh` n'engendre que la glue `-target web`, alors que `wasm-bindgen` propose une cible `nodejs`. Expliquer ce que l'on gagne et ce que `npm/lib/core-node.js` doit faire en échange.

Solution

Le `.wasm` est identique pour toutes les cibles ; seule la glue diffère. Une cible unique évite de publier deux copies d'un module de plusieurs mégaoctets, et garantit que Node et navigateur exécutent littéralement la même glue. En échange, sous Node, la glue ne peut pas faire de `fetch` d'un chemin de fichier : `core-node.js` importe la glue par `import()`, lit les octets du `.wasm` avec `readFileSync` et appelle `initSync({ module })`.

Exercice 24.3: Un workflow qui change tout seul

Un workflow contient `uses: rboudrouss/reactant-analyzer@v0.6.0` sans bloc `with`. Une version 0.7.0 est publiée sur npm. Que se passe-t-il au prochain run ? Que faut-il écrire pour que le run soit reproductible ?

Solution

`gh-action.mjs` est celui du tag `v0.6.0`, mais il lance `npx -yes reactant-analyzer@latest`, donc l'analyseur 0.7.0. Les findings, et donc le succès du job sous `fail-on: warning`, peuvent changer sans aucun commit du dépôt analysé. Il faut ajouter `with: { version: 0.6.0 }` (idéalement la même version que le tag).

Exercice 24.4: Pack hissé dans un monorepo

Un monorepo a `node_modules/@acme/react-rules/` à la racine et `apps/web/reactant.config.json` qui déclare `"packs": ["@acme/react-rules"]`. On lance `reactant check apps/web` (binaire natif) puis `npx reactant-analyzer check apps/web`. Prédire les deux issues, et dire pourquoi l'écart n'est pas une faute de `soundness`.

Solution

Le natif cherche `apps/web/node_modules/@acme/react-rules/`, ne le trouve pas et sort en code 2 avec `[error] pack '@acme/react-rules': not installed: ....` L'hôte npm, par `createRequire(...).resolve`, remonte jusqu'au `node_modules` de la racine, trouve le pack et l'analyse a lieu. L'écart est une différence de disponibilité, pas de résultat : le natif ne lance jamais une analyse avec moins de règles que la configuration n'en demande, il refuse bruyamment (`src/cli/config_load.rs`, l. 55–57). On le contourne en installant le pack dans le sous-paquet ou en le désignant par un chemin relatif.

Chapitre 25

Tests, corpus et campagnes : la méthode de mesure

Objectifs du chapitre.

- Comprendre pourquoi la promesse « faux négatifs interdits » impose une manière particulière d'écrire les tests : on teste le silence *et* le témoin qui prouve que le silence n'est pas une panne.
- Connaître les six altitudes de mesure du projet, des tests unitaires à l'oracle runtime, et le point d'entrée exact de chacune.
- Savoir écrire un test par paires « tire encore » / « ne tire plus », et lire un test de graduation de sévérité.
- Connaître le corpus de quatorze dépôts épinglés, la métrique en localisations distinctes, le digest, l'empreinte, et la porte `scripts/corpus-baseline.py` avec ses trois codes de sortie.
- Savoir suivre le cycle de vie d'un chiffre, du push rouge au commit chore(corpus), et comprendre l'erreur de comptage du 3 septembre 2026 qui a fait naître cette procédure.
- Connaître les deux types de campagnes (FP classique, wish-list à l'aveugle), le banc de rendu jsdom, et la manière dont le projet classe ce qu'il ne corrige pas.

Prérequis : chapitre 3 (soundness, must/may) ; chapitre 17 (sévérité certifiée, Certified, témoins) ; chapitre 23 (CLI, rapport JSON, blind spots, localisations). Les chapitres de la partie V (règles) fournissent les exemples mais ne sont pas indispensables. La livraison (CI de chaque changement, parité WASM, Action GitHub) est au chapitre 24.

Les chapitres précédents décrivent une chaîne de production : un fichier `.tsx` entre, `oxc` le parse, le lowering en fait un CFG, le moteur calcule un point fixe et des relations, les règles émettent des diagnostics, le driver les trie et les imprime. Ce chapitre ne décrit aucune étape de plus. Il décrit l'*instrument* qui observe cette chaîne et qui dit si elle tient sa promesse.

Cette promesse est inhabituelle. La plupart des linters s'évaluent au taux de faux positifs : un outil qui crie trop est désinstallé. `reactant-analyzer` accepte des faux positifs (FP) et s'interdit les faux négatifs (FN) (définition 3.8). Or un FN est un silence. On ne « voit » pas un silence dans une sortie, et un test qui vérifie qu'une règle se tait passe aussi bien quand la règle est juste que quand elle est cassée. Toute la méthode de ce chapitre découle de cette asymétrie. Elle tient en une phrase, écrite en tête du workflow corpus :

The unit tests guard mechanisms ; the corpus guards outcomes.

(`.github/workflows/corpus.yml`, l. 5.) On procède du plus petit au plus grand : pourquoi mesurer (section 25.1), la sévérité comme contrat (section 25.2), la pyramide des tests (section 25.3), la discipline des paires (section 25.4), les angles morts (section 25.5), le corpus (section 25.6), sa métrique (section 25.7), une erreur de comptage devenue étude de cas (section 25.8), le cycle de

vie d'un chiffre (section 25.9), les campagnes (section 25.10), l'oracle runtime (section 25.11) et enfin la façon de classer ce que l'on ne corrige pas (section 25.12).

25.1 Pourquoi mesurer : le silence et son témoin

25.1.1 Un test qui passe pour de mauvaises raisons

Voici le premier composant de la fixture `tests/fixtures/clean.tsx`, un compteur sans défaut :

Extrait 25.1 – `tests/fixtures/clean.tsx`, l. 7–20 — un compteur correct

```
7 function Counter() {
8   const [count, setCount] = useState(0);
9
10  useEffect(() => {
11    document.title = `Count: ${count}`;
12  }, [count]); // count is in deps ✓
13
14  return (
15    <div>
16      <p>{count}</p>
17      <button onClick={() => setCount((n) => n + 1)}></button>
18    </div>
19  );
20 }
```

Lancé sur ce fichier, l'analyseur ne dit rien et affiche la coche verte :

```
$ reactant check tests/fixtures/clean.tsx ; echo exit=$?
```

```
4 clean component(s) hidden, rerun with --show-clean
```

```
✓ 1 file(s) no issues found.
exit=0
```

Supposons maintenant qu'une modification du lowering fasse disparaître *tous* les `useEffect` du CFG. La sortie serait la même, au caractère près : aucun effet, donc aucune dépendance manquante, donc « no issues found ». Un test qui se contenterait de vérifier le silence sur `clean.tsx` passerait. Il y a pire : c'est exactement la forme de plusieurs défauts réels du projet. Avant #2, un composant dont le `try` contenait un `return` voyait son `catch` entier disparaître du CFG, et il était déclaré « 1 hook, ✓ » (section 25.4). Avant #9, un projet Vite dont les alias n'étaient pas chargés finissait sur « ✓ N file(s) no issues found. » sans avoir lu une seule cible d'import (section 25.5).

Définition 25.1: Silence témoigné

Un test de silence est *témoigné* lorsqu'il est accompagné d'un test voisin, sur un programme qui ne diffère que par le défaut, qui vérifie que la même règle *tire*. Le témoin établit que l'information nécessaire à la règle atteint bien la règle ; le silence n'a alors plus qu'une explication, la bonne.

Le projet écrit ce témoin partout où il vérifie un silence : c'est la « discipline des paires » (section 25.4). À l'échelle du corpus, le même principe devient : on ne se réjouit pas d'une baisse du nombre de findings tant que chaque localisation retirée n'a pas été relue à la source.

25.1.2 Six altitudes, six portes d'entrée

La méthodologie observe la chaîne à six altitudes, de la plus fine à la plus large. Chacune a sa porte d'entrée dans le code, résumée dans le tableau 25.1.

Altitude	Ce qui est observé	Point d'entrée
1. unitaire	un domaine, un transfert, une fonction	<code>#[cfg(test)] mod tests et crate::test_support</code>
2. intégration lib	source $\rightarrow$ règles, sans CLI	<code>Parser $\rightarrow$ lower_program $\rightarrow$ analyze_component_as ou analyze_program $\rightarrow$ RuleCtx::new $\rightarrow$ Rule::check</code>
3. intégration binaire	le comportement visible (texte, code de sortie)	<code>env!("CARGO_BIN_EXE_reactant") $\rightarrow$ cli $\rightarrow$ driver::run_check</code>
4. parité	deux hôtes, une seule sortie	<code>run_check sur OsFileSystem et sur MemFileSystem; npm/test/smoke.sh (WASM contre natif)</code>
5. corpus	les résultats sur 14 applications réelles	<code>reactant -format json test-repo $\rightarrow$ scripts/corpus-baseline.py</code>
6. oracle runtime	la vérité concrète (rendus comp-tés)	<code>scripts/render-bench/run.mjs (React 19 et jsdom)</code>

TABLE 25.1 – Les six altitudes de mesure et leur point d'entrée (d'après `notes/14-tests-corpus-methodo.md`, §1.1).

À côté de ces mesures automatiques, il y a des *mesures humaines* : les campagnes (`docs/campaign/`), le journal de précision (`docs/precision-log.md`), la page des limites (`docs/limitations.md`), la procédure de triage publiée (`skills/reactant-triage/`) et le tracker GitHub. Elles produisent des documents datés et des issues étiquetées, pas des verdicts verts ou rouges, et c'est leur articulation avec les mesures automatiques que ce chapitre veut rendre claire.

Quelques ordres de grandeur, mesurés au commit `e67b10a` : `cargo test -workspace` exécute 1 529 tests en 73 suites, dont 632 tests unitaires sous `src/` et 897 tests d'intégration répartis dans 70 fichiers `tests/*.rs` (26 212 lignes) ; le corpus compte 40 164 fichiers source ; la baseline committée vaut 1 498 localisations.

25.2 La sévérité comme contrat

Avant de tester quoi que ce soit, il faut savoir *ce qu'un test peut affirmer*. Dans `reactant-analyzer`, un diagnostic ne dit pas seulement « il y a peut-être un problème ici » : son niveau est une affirmation logique, et les tests vérifient cette affirmation autant que la présence du diagnostic.

25.2.1 Trois niveaux, une définition chacun

Extrait 25.2 – `src/rules/api/diagnostic.rs`, l. 24–39 — les trois niveaux et leur contrat

```

24 /// Confidence level of a diagnostic.
25 ///
26 /// - `Error` the defect is certain whenever the flagged code runs. Built only
27 /// from a proof of the whole claim, not of one of its conjuncts (#142).
28 /// - `Warning` a possible defect (conditional path, over-approximation), or a
29 /// certain fact whose cost is not (a fresh reference, one wasted render).
```

```

30 /// - `Info`    not actionable without context: a known analysis limitation
31 ///   (widening, depth cap), or a pattern that looks intentional (a seed-once
32 ///   prop name, a cheap pure initializer). Hidden by default; show with --info.
33 #[derive(Debug, Clone, Copy, PartialEq, Eq, Default)]
34 pub enum Severity {
35     Error,
36     #[default]
37     Warning,
38     Info,
39 }

```

Le commentaire est la spécification. Une ERROR dit « le défaut est certain dès que ce code s'exécute », et elle doit être construite à partir d'une preuve de *toute* l'affirmation, pas d'un de ses conjoints (#142). Un WARNING dit « défaut possible », ou « fait certain au coût incertain ». Un INFO marque une limite de l'analyse ou un motif qui a l'air voulu. Le chapitre 17 a montré comment cette hiérarchie est rendue infalsifiable : une ERROR ne se fabrique qu'à partir d'un Certified (définition 17.1), jeton dont le constructeur est privé au module des primitives must, et la seule transformation de sévérité exposée aux consommateurs, clamp, ne peut que descendre :

Extrait 25.3 – src/rules/api/diagnostic.rs, l. 104–114 — le seul changement de niveau exposé descend

```

104 /// Consumer override (ADR-022 §3): lower this finding to `ceiling` when its
105 /// constructed severity exceeds it. Downgrade-only, hence sound to expose:
106 /// Error is constructible only through [ `Diagnostic::error` ], so the
107 /// constructed severity IS the polarity ceiling and `pin ⊔ polarity`
108 /// reduces to a min – an upgrade request is a structural no-op.
109 pub fn clamp(mut self, ceiling: Severity) -> Self {
110     if ceiling.rank() < self.severity.rank() {
111         self.severity = ceiling;
112     }
113     self
114 }

```

Invariant 25.1: Un faux positif ne porte jamais d'Error

Puisqu'une ERROR n'existe qu'adossée à une preuve must de toute l'affirmation, un diagnostic de niveau ERROR ne peut être faux que si le moteur lui-même est faux (un soundness-bug). Tout faux positif « ordinaire », dû à une sur-approximation, est de niveau WARNING ou moins. docs/limitations.md l'écrit en tête de sa liste de FP : « Every entry here is Warning or below by construction. A false positive never carries an Error. » (docs/limitations.md, l. 110–113).

Cet invariant a une conséquence directe sur l'écriture des tests. Un test qui vérifie une ERROR vérifie une *preuve*. Un test qui vérifie qu'une ERROR est devenue un WARNING vérifie qu'une preuve induite a été *retirée*. Un test qui vérifie un WARNING vérifie une possibilité. Les trois sont des tests différents, et un test qui se contente de « la règle tire » n'en est aucun.

25.2.2 Un exemple gradué : trois formes, trois verdicts

Les tests f5_object_churn_unconditional_is_error, f5_object_churn_conditional_is_warning et f5_fetch_once_guard_converges de tests/corpus_fp_fixes.rs fixent trois formes voisines d'un même programme. On les réunit dans un fichier :

Extrait 25.4 – Trois formes de churn d’objet (/tmp/ch25/churn.tsx, construit d’après tests/corpus_fp_fixes.rs, l. 472–521)

```

1 import { useEffect, useState } from "react";
2
3 export function Unconditional() {
4   const [obj, setObj] = useState({ a: 1 });
5   useEffect(() => {
6     setObj({ ...obj, b: 2 });
7   }, [obj]);
8   return <div>{obj.a}</div>;
9 }
10
11 export function Conditional({ cond }) {
12   const [obj, setObj] = useState({ a: 1 });
13   useEffect(() => {
14     if (cond()) {
15       setObj({ ...obj, b: 2 });
16     }
17   }, [obj]);
18   return <div>{obj.a}</div>;
19 }
20
21 export function FetchOnce() {
22   const [user, setUser] = useState(null);
23   useEffect(() => {
24     if (user === null) {
25       setUser({ name: "guest" });
26     }
27   }, [user]);
28   return <div>{user}</div>;
29 }

```

Rappel React : Object.is sur les deps et sur les écritures

Après chaque commit, React compare chaque entrée du tableau de deps d’un effet à sa valeur précédente avec `Object.is`. Un objet littéral est une référence neuve à chaque évaluation : `{ ...obj, b: 2 }` n’est jamais `Object.is`-égal à `obj`. Symétriquement, un `setState` dont l’argument est `Object.is`-égal à la valeur courante est abandonné sans rendu. Un effet qui écrit un objet neuf dans le slot dont il dépend se relance donc à chaque commit, sans fin.

La sortie observée, élaguée des lignes `verified` sauf une :

```
$ reactant check churn.tsx --all-roots --info --show-clean --trace
```

```

Conditional (2 hooks) churn.tsx
warn  infinite-loop [hook:0] (line 13:2) this effect may store a fresh reference into state `obj` which its deps
  ↳ react to: possible infinite render loop
    ↳ a fresh value is written to state `obj` here [hook:1] (line 15:6)
warn  missing-deps [hook:1] var:cond (line 13:2) `cond` is used in this effect but not in its deps array, and
  ↳ its value may change between renders
    ↳ `cond` is read here [hook:1] (line 13:2)
FetchOnce (2 hooks) churn.tsx ✓
verified infinite-loop no effect diverges into an infinite render loop
Unconditional (2 hooks) churn.tsx
error  infinite-loop [hook:0] (line 5:2) this effect recreates object state `obj` it depends on. Every run stores
  ↳ a fresh reference (`Object.is` always fails) and re-triggers itself: infinite render loop
    ↳ a fresh value is written to state `obj` here [hook:1] (line 6:4)

! 1 error(s), 2 warning(s) across 1 file(s).
exit=1

```


Les trois verdicts forment une échelle. Unconditional est une ERROR : l'écriture est sur tous les chemins de l'effet, la valeur écrite est fraîche à coup sûr, et le slot écrit est dans les deps ; le commentaire du test résume « Triple must → Error ». Conditional est un WARNING : la garde `cond()` peut devenir fausse, la boucle n'est que possible. `FetchOnce` est *prouvé* convergent : l'écriture rend la garde `user === null` fausse au rendu suivant, et l'analyseur délivre l'assurance `verified` sous `-info`. Le test `f5_object_churn_conditional_is_warning` vérifie les deux côtés de sa cellule : il exige un WARNING *et* l'absence d'ERROR (`tests/corpus_fp_fixes.rs`, l. 494–521).

Remarque

Le commentaire de `f5_object_churn_unconditional_is_error` note que ce FN, découvert en concevant la correction F5, « never widens (references converge), only catchable through dep structure » (`tests/corpus_fp_fixes.rs`, l. 474–475). Le point fixe des valeurs converge, puisqu'une référence fraîche reste une référence fraîche ; c'est la structure des deps, via le graphe de churn (chapitre 15), qui voit la boucle. Un test de graduation fixe donc aussi *quel mécanisme* doit porter le verdict.

25.3 La pyramide des tests

Figure 25.1 dispose les six altitudes en pyramide. La base est large et rapide, le sommet est étroit, lent et manuel. Les quatre étages du bas tournent à chaque changement (job test de la CI, chapitre 24) ; le corpus tourne à chaque push sur main qui touche l'analyseur ; l'oracle runtime n'est lancé qu'à la main.

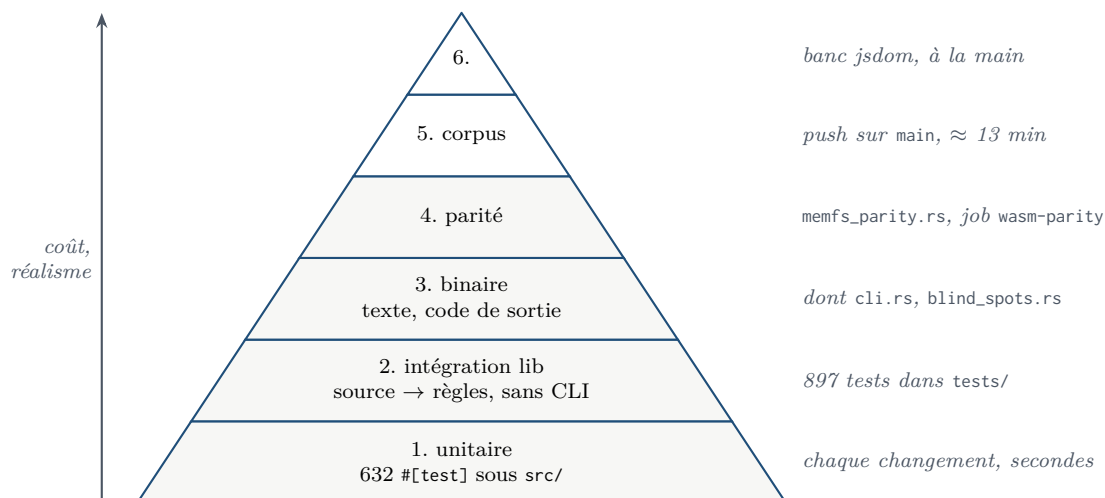


FIGURE 25.1 – La pyramide des mesures. Les quatre étages grisés tournent dans cargo test ; les deux du haut mesurent des résultats plutôt que des mécanismes.

25.3.1 Étage 1 : les tests unitaires et `src/test_support.rs`

Les 632 tests unitaires vivent dans des modules `#[cfg(test)]` à côté du code qu'ils testent ; les plus fournis sont `src/domains/impls/state_value.rs` (54 tests) et `src/domains/transfer/state_value.rs` (44). Pour construire un CFG ou un résultat d'analyse à la main, ils partagent un petit module de constructeurs :

Extrait 25.5 – `src/test_support.rs`, l. 26–52 — les constructeurs de CFG et d'identités

```
26 /// One-block render CFG terminated by `return unit` - the canonical trivial
```

```

27 /// body used by most rule/engine tests (entry 0, block id 0, no edges).
28 pub(crate) fn single_block_cfg(stmts: Vec<Stmt>) -> CFG {
29     single_block_cfg_term(stmts, Terminator::Return(Expr::Lit(Prim::Unit)))
30 }
31
32 /// One-block CFG with a caller-chosen terminator (entry 0, block id 0, no edges).
33 pub(crate) fn single_block_cfg_term(stmts: Vec<Stmt>, term: Terminator) -> CFG {
34     let mut blocks = std::collections::BTreeMap::new();
35     blocks.insert(0, BasicBlock { id: 0, stmts, term });
36     CFG {
37         entry: 0,
38         blocks,
39         edges: vec![],
40     }
41 }
42
43 /// The id a hand-built [analysis_result] carries, and therefore the one
44 /// [prog] registers it under – what a test passes to
45 /// [crate::rules::RuleCtx::new].
46 pub(crate) const C: crate::ir::ComponentId = crate::ir::ComponentId::SYNTHETIC;
47
48 /// A second, third ... distinct component, for a fixture that needs more than
49 /// one identity and builds no table.
50 pub(crate) const fn cid(i: u32) -> crate::ir::ComponentId {
51     crate::ir::ComponentId::from_index(i)
52 }

```

`single_block_cfg` fabrique le corps trivial d'un composant : un seul bloc, d'identifiant 0, terminé par `return unit`, sans arête. Il est utilisé 49 fois sous `src/`. La constante `C` est l'identité `ComponentId::SYNTHETIC`, qui vaut `ComponentId(u32::MAX)` (`src/ir/component_id.rs`, l. 38) : c'est celle qu'estampille `analyze_component`, et donc celle qu'un test doit passer à `RuleCtx::new.cid(i)` fournit une deuxième, une troisième identité quand un test en a besoin sans construire de table. Le reste du module (`src/test_support.rs`, l. 54–142) ajoute `named`, qui interne un nom dans une table globale au binaire de test (« Process-wide and monotone: one name is one id for the life of the test binary »), `prog`, qui enregistre un résultat sous l'identité *qu'il porte déjà*, et `analysis_result`, un `AnalysisResult` à valeurs par défaut destiné à la syntaxe `AnalysisResult { hooks, ..analysis_result(cfg) }`.

Piège

Le module est déclaré `#[cfg(test)] mod test_support;` dans `src/lib.rs` : il n'existe que pour les tests *de la bibliothèque* et reste invisible depuis `tests/`, qui voit la `crate` comme un utilisateur externe. C'est pourquoi le harnais de l'étage 2 est recopié dans une trentaine de fichiers (issue ouverte #14).

25.3.2 Étape 2 : le harnais « source → règles »

L'essentiel des tests d'intégration suit un même motif : parser un programme TSX écrit en ligne, l'abaisser, analyser chaque composant, lancer toutes les règles natives, et renvoyer la liste des couples (règle, message). Sa version canonique ouvre `tests/corpus_fp_fixes.rs` :

Extrait 25.6 – `tests/corpus_fp_fixes.rs`, l. 14–69 — le harnais source → règles

```

14 fn diagnostics(src: &str) -> Vec<(String, String)> {
15     let alloc = Allocator::default();
16     let ret = Parser::new(&alloc, src, SourceType::tsx())
17         .with_options(ParseOptions::default())
18         .parse();

```

```

19     assert!(
20         ret.diagnostics.is_empty(),
21         "parse errors: {:?}",
22         ret.diagnostics
23     );
24     let components = lower_program(
25         &ret.program,
26         src,
27         std::path::Path::new("test.tsx"),
28         &mut Default::default(),
29     );
30     assert!(!components.is_empty(), "no component detected");
31
32     let mut components_map = std::collections::HashMap::new();
33     let mut names = Vec::new();
34
35     let mut component_table = reactant::ir::ComponentTable::default();
36     for comp in components {
37         let id = component_table.intern(reactant::ir::CompOrigin {
38             file: comp.file.clone(),
39
40             name: comp.name.clone(),
41         });
42
43         let result = reactant::engine::analyze_component_as(
44             comp,
45             id,
46             &StateValueTransfer,
47             &Config::default(),
48         );
49
50         components_map.insert(id, result);
51
52         names.push(id);
53     }
54     let prog = reactant::engine::ProgramAnalysisResult {
55         components: components_map,
56         component_table,
57         ..Default::default()
58     };
59
60     let mut out = Vec::new();
61     for name in &names {
62         for rule in all_rules() {
63             for d in rule.check(&RuleCtx::new(&prog, *name)) {
64                 out.push((d.rule.to_string(), d.message.clone()));
65             }
66         }
67     }
68     out
69 }

```

L'ordre est celui du pipeline. Une erreur de parse fait échouer le test (l. 19–23) : un programme de test mal écrit ne doit jamais passer pour un programme silencieux. De même, un programme où aucun composant n'est détecté fait échouer le test (l. 30). Chaque composant reçoit une identité internée (l. 35–41), puis est analysé seul par `analyze_component_as` avec `Config::default()` (l. 43–48). Un `ProgramAnalysisResult` est assemblé à la main (l. 54–58), et chaque règle de `all_rules()` est appliquée à chaque composant (l. 60–67). Quand le défaut vit dans l'analyse inter-composants, le fichier offre une variante, `program_rules_fired`, qui remplace la boucle intra par `analyze_program(..., RootStrategy::AllComponents, ...)` (`tests/corpus_fp_fixes.rs`,

l. 391–422).

Piège

`Config::default()` n'est pas la configuration d'un utilisateur. Le driver construit son registre de résumés par `SummaryRegistry::new_with_common()` (`src/driver/mod.rs`, l. 371), qui connaît les contrats de bibliothèques courantes ; `Config::default()` a un registre vide. Au commit étudié, 47 fichiers de `tests/` écrivent `Config::default()` ; l'issue #14 en tire la conclusion : « most of the integration suite validates a degraded engine ». Les tests de l'étage 3, qui passent par le binaire, ne sont pas concernés.

25.3.3 Étage 3 : le binaire, et le déterminisme

Certains comportements n'existent que dans la sortie : la ligne de résumé, le groupage des localisations, le code de sortie. Les tests de cet étage lancent le binaire compilé, que Cargo expose par `env!("CARGO_BIN_EXE_reactant")`, avec `NO_COLOR=1` et le répertoire du manifeste comme répertoire courant (`tests/blind_spots.rs`, l. 14–21). Parmi eux, deux tests ont un rôle méthodologique particulier, parce que la mesure corpus repose sur eux :

Extrait 25.7 – `tests/cli.rs`, l. 360–398 — la sortie doit être identique octet pour octet d'un run à l'autre

```

360 fn assert_byte_identical_across_runs(args: &[&str]) {
361     let first = stdout(&reactant(args));
362     assert!(!first.is_empty(), "nothing to compare for {args:?}");
363     for _ in 0..3 {
364         assert_eq!(
365             stdout(&reactant(args)),
366             first,
367             "output must be byte-identical across runs: {args:?}"
368         );
369     }
370 }
371
372 const DETERMINISM_ARGS: &[&str] = &[
373     "check",
374     "tests/fixtures",
375     "--all-roots",
376     "--info",
377     "--fail-on",
378     "never",
379 ];
380
381 #[test]
382 fn consecutive_runs_are_byte_identical() {
383     // Rules iterate HashMaps internally; the output layer's total ordering
384     // must absorb that – consecutive runs over a fixture set with many
385     // same-severity diagnostics (analysis-limit Infos) must not reorder.
386     assert_byte_identical_across_runs(DETERMINISM_ARGS);
387 }
388
389 #[test]
390 fn consecutive_traced_runs_are_byte_identical() {
391     // The total order above covers diagnostics, not the notes *inside* one:
392     // `Diagnostic::notes` is not sorted, so a witness chain built by iterating
393     // a HashMap (a churn-graph `CycleEdge`, say) would reorder here and
394     // nowhere else. Without `--trace` no test walks that output at all.
395     let mut args = DETERMINISM_ARGS.to_vec();
396     args.push("--trace");

```

```

397     assert_byte_identical_across_runs(&args);
398 }

```

La commande de référence est `check tests/fixtures -all-roots -info -fail-on never` : toutes les fixtures du dépôt, y compris huit fichiers de démonstration qu’aucun autre test ne cite nommément (`bugs.tsx`, `dashboard.tsx`, `search.tsx`...). Elle est exécutée quatre fois (une, puis trois comparaisons). La variante `-trace` existe parce que l’ordre total du rapport couvre les diagnostics mais pas les notes à l’intérieur d’un diagnostic : une chaîne de témoins construite en itérant une `HashMap` se réordonnerait là et nulle part ailleurs. Le journal de précision étend l’affirmation au corpus : « Four runs of a frozen binary on one repository, and two on the whole corpus, produce *bit-identical* JSON files » (`docs/precision-log.md`, l. 33–35).

Proposition 25.1: Un écart est toujours un signal

Si l’analyse est déterministe au niveau de l’octet, alors deux mesures différentes du même corpus par deux binaires ne peuvent différer que par un changement de comportement du binaire, ou par une erreur de comptage de celui qui mesure. Il n’y a pas de bruit.

Ce n’est pas un détail. La section 25.8 raconte comment, un jour où les chiffres ne se reproduisaient pas, c’est cette proposition qui a permis de conclure à une erreur humaine plutôt qu’à une dérive.

25.3.4 Étage 4 : la parité, et les méta-tests

La parité vérifie qu’un même programme donne la même sortie sur deux hôtes. `tests/memfs_parity.rs` lance `driver::run_check` une fois sur le système de fichiers réel, une fois sur un `MemFileSystem` chargé des mêmes fichiers (y compris `tsconfig.json` et `vite.config`), et compare `stdout`, `stderr` et `exit_code` (`tests/memfs_parity.rs`, l. 44–117). La parité entre le binaire natif et le module WASM publié est vérifiée hors de cargo `test`, par `npm/test/smoke.sh` ; elle est décrite au chapitre 24.

À côté, une famille de *méta-tests* ne teste aucune règle : ils testent la mesure elle-même, ou une frontière d’architecture. Ils transforment un nombre ou une liste en invariant compilé, de sorte qu’on ne puisse pas « éditer un nombre » pour faire avancer une courbe.

- `tests/corpus_baseline.rs` vérifie la cohérence interne de la baseline du corpus (section 25.7).
- `tests/catalogue.rs` fige la mesure d’expressivité du vocabulaire déclaratif : 22 entrées, dont `EXPRESSIBLE_NOW = 21` (`tests/catalogue.rs`, l. 1093–1094). Chaque entrée exprimable doit tirer sur sa fixture `fires_on` et se taire sur `silent_on` : pour faire passer la constante à 22, il faut écrire la règle et ses deux fixtures.
- `tests/community_packs.rs` vérifie que les packs issus de la campagne à l’aveugle chargent, gardent leur nombre de règles et n’émettent jamais d’`ERROR` (section 25.10).
- `tests/layer_boundary.rs` tient la frontière moteur/règles de l’ADR-42 par un *ratchet*, une liste d’exceptions qui ne peut que rétrécir.
- `tests/docs_drift.rs` et `tests/schemas.rs` vérifient que la documentation du vocabulaire et les JSON Schemas publiés ne dérivent pas du code.

Le ratchet mérite un regard, parce que sa technique est réutilisable :

Extrait 25.8 – `tests/layer_boundary.rs`, l. 19–41 — la liste qui ne peut que rétrécir, et ce qui compte comme parcourir la syntaxe

```

19 /// The rule files still walking syntax, as of ADR-042 slice 5. Shrink only.
20 const ALLOWED: &[&str] = &[
21     "src/rules/api/query.rs",

```

```

22     "src/rules/api/witness.rs",
23     "src/rules/helpers/jsx.rs",
24     "src/rules/helpers/mod.rs",
25     "src/rules/helpers/mount.rs",
26     "src/rules/helpers/purity.rs",
27     "src/rules/impls/redundant_set_state.rs",
28     "src/rules/impls/setter_in_render.rs",
29     "src/rules/impls/state_mutation.rs",
30     "src/rules/impls/unnecessary_rerender.rs",
31 ];
32
33 /// What counts as touching syntax: iterating a CFG's blocks, matching
34 /// statements or terminators, or descending an expression tree.
35 const MARKERS: &[&str] = &[
36     "cfg.blocks",
37     ".blocks.values()",
38     "Stmt::",
39     "for_each_child",
40     "Terminator::",
41 ];

```

Extrait 25.9 – tests/layer_boundary.rs, l. 54–59 — la détection, purement textuelle

```

54 /// `true` when the non-test part of the file touches syntax.
55 fn walks_syntax(path: &Path) -> bool {
56     let text = fs::read_to_string(path).expect("readable file");
57     let body = text.split("#[cfg(test)]").next().unwrap_or("");
58     MARKERS.iter().any(|m| body.contains(m))
59 }

```

Deux tests l’exploitent : `no_rule_file_outside_the_list_walks_syntax` (aucun nouveau contrevenant) et `the_list_only_shrinks` (un fichier assaini doit quitter la liste). La détection est textuelle et ignore tout ce qui suit le premier `#[cfg(test)]`. Elle peut signaler à tort un fichier qui cite `Stmt::` dans un commentaire, jamais laisser passer un parcours écrit avec l’un des motifs ; un parcours écrit autrement, par un helper renommé par exemple, lui échapperait. C’est un garde-fou contre la dérive ordinaire, pas une preuve.

Décision : ADR-042 §1 — une frontière tenue par un test

Une règle lit des lignes de relation et appelle des primitives `must` ; elle ne parcourt aucun CFG. L’ADR-42 en donne la raison : « a fact computed in two places is two facts that drift ». L’alternative, laisser chaque règle recalculer ses faits localement, est refusée ; la migration étant progressive, le ratchet fige l’état atteint au lieu d’exiger une migration d’un bloc.

25.4 La discipline des paires

25.4.1 Le problème : une correction de précision retire des findings

Une correction de précision *retire* des diagnostics. C’est son but, et c’est aussi son danger : la façon la plus simple de retirer un FP est de faire taire la règle sur une forme trop large, qui englobe aussi de vrais défauts. Le test qui vérifie « ne tire plus » sur la forme corrigée passe dans les deux cas. Pour distinguer la correction juste de la correction trop large, il faut un second test, sur la forme voisine qui *est* un défaut.

Définition 25.2: Paire fires / silent

Une *paire fires / silent* est un couple de programmes qui ne diffèrent que par ce qui fait le défaut : l'un doit faire tirer une règle (*fires*), l'autre, un quasi-voisin volontairement difficile, doit la laisser muette (*silent*). Une correction de précision est accompagnée d'au moins un test « tire encore » sur la forme voisine qui reste un vrai positif.

Les noms des tests l'annoncent : `..._still_warns`, `..._still_loops`, `..._still_fires`, `..._still_error`, ou le commentaire « TP preservation: the widening arm must survive the churn addition » (tests/corpus_fp_fixes.rs, l. 693). Le fichier tests/corpus_fp_fixes.rs, 1973 lignes et 79 tests, est le registre de ces paires : chaque cas y est un reproducteur minimal extrait d'un dépôt réel du corpus.

25.4.2 Un exemple : une écriture qui règle sa propre garde (#91)

React documente un idiome, « ajuster l'état pendant le rendu » : un composant compare un slot à une valeur dérivée des props et, s'ils diffèrent, écrit la valeur dans le slot. L'écriture a lieu pendant le rendu, mais elle ne boucle pas, parce qu'au rendu suivant la comparaison est fausse. Le corpus en contenait une occurrence dans `CurrencyInput` de `twenty`, signalée à tort par `setter-in-render`. Le test qui fixe la correction :

Extrait 25.10 – tests/corpus_fp_fixes.rs, l. 1694–1713 — la forme corrigée : la garde est réglée par l'écriture

```

1694 /// React's documented "adjust state during render" idiom: the guard compares
1695 /// the slot against the very expression the write stores in it, so the two are
1696 /// equal on the next render and the branch is dead (twenty `CurrencyInput`).
1697 #[test]
1698 fn a_write_that_settles_its_own_guard_is_not_a_render_setter() {
1699     let fired = rules_fired(
1700         r#"
1701         function C({ value, decimals }) {
1702             const [scale, setScale] = useState(0);
1703             const scaleForCurrentValue = getSafeScale(value, decimals);
1704             if (scale < scaleForCurrentValue) { setScale(scaleForCurrentValue); }
1705             return <div>{scale}</div>;
1706         }
1707         "#,
1708     );
1709     assert!(
1710         !fired.iter().any(|r| r == "setter-in-render"),
1711         "the guard cannot hold again once the slot holds what it compares against: {fired:?}"
1712     );
1713 }

```

L'affirmation relationnelle qui fonde la correction est que « $x < y$ after $x := y$ is false for *all* x and y » (docs/precision-log.md, entrée « a write that settles its own guard »). Elle est vraie à une condition : que l'expression écrite et l'expression comparée dénotent la même valeur au rendu suivant, ce que le moteur n'admet que pour deux orthographes identiques sans appel. Où passe alors la frontière ? Les deux tests témoins la dessinent :

Extrait 25.11 – tests/corpus_fp_fixes.rs, l. 1760–1800 — les deux formes voisines qui doivent tirer encore

```

1760 /// The loop it must not swallow: the written value reads the slot, so each
1761 /// write flips the guard back on (dub's `setUseAsync(Boolean(g && !useAsync))`).
1762 #[test]
1763 fn a_write_derived_from_the_slot_still_loops() {

```

```

1764     let fired = rules_fired(
1765         r#"
1766         function C({ groups }) {
1767             const [useAsync, setUseAsync] = useState(false);
1768             useEffect(() => {
1769                 setUseAsync(Boolean(groups && !useAsync));
1770             }, [groups, useAsync]);
1771             return <div>{String(useAsync)}</div>;
1772         }
1773         "#,
1774     );
1775     assert!(
1776         fired.iter().any(|r| r == "infinite-loop"),
1777         "`!useAsync` in the written value oscillates: {fired:?}"
1778     );
1779 }
1780
1781 /// A comparison the write does NOT settle: the guard tests one expression and
1782 /// the write stores another.
1783 #[test]
1784 fn a_write_of_something_else_still_fires() {
1785     let fired = rules_fired(
1786         r#"
1787         function C({ a, b }) {
1788             const [slot, setSlot] = useState(null);
1789             useEffect(() => {
1790                 if (slot !== a) { setSlot(b); }
1791             }, [slot, a, b]);
1792             return <div>{slot}</div>;
1793         }
1794         "#,
1795     );
1796     assert!(
1797         fired.iter().any(|r| r == "infinite-loop"),
1798         "`slot !== a` says nothing about a write of `b`: {fired:?}"
1799     );
1800 }

```

Dans `a_write_derived_from_the_slot_still_loops`, la valeur écrite *lit* le slot : chaque écriture réarme la garde, et la boucle est réelle (quatre composants de `dub`, et l'analyseur a raison sur les quatre). Dans `a_write_of_something_else_still_fires`, la garde compare `a` et l'écriture stocke `b` : rien ne dit que la garde devient fausse. Réunies dans un fichier, les trois formes donnent :

Extrait 25.12 – La paire #91 et ses deux témoins (`/tmp/ch25/settle.tsx`, d'après `tests/corpus_fp_fixes.rs`, l. 1698–1800)

```

1  export function Settles({ value, decimals }) {
2      const [scale, setScale] = useState(0);
3      const scaleForCurrentValue = getSafeScale(value, decimals);
4      if (scale < scaleForCurrentValue) { setScale(scaleForCurrentValue); }
5      return <div>{scale}</div>;
6  }
7
8  export function Oscillates({ groups }) {
9      const [useAsync, setUseAsync] = useState(false);
10     useEffect(() => {
11         setUseAsync(Boolean(groups && !useAsync));
12     }, [groups, useAsync]);
13     return <div>{String(useAsync)}</div>;
14 }
15

```



```

16 export function Elsewhere({ a, b }) {
17   const [slot, setSlot] = useState(null);
18   useEffect(() => {
19     if (slot !== a) { setSlot(b); }
20   }, [slot, a, b]);
21   return <div>{slot}</div>;
22 }

```

```

Elsewhere (2 hooks) settle.tsx
  warn  infinite-loop [hook:0] (line 20:2) this effect may store a fresh reference into state `slot` which its deps
    ↳ react to: possible infinite render loop
      → a fresh value is written to state `slot` here [hook:1] (line 21:22)
Oscillates (2 hooks) settle.tsx
  warn  infinite-loop [hook:0] (line 12:2) this effect may store a fresh reference into state `useAsync` which its
    ↳ deps react to: possible infinite render loop
      → a fresh value is written to state `useAsync` here [hook:1] (line 13:4)
1 clean component(s) hidden, rerun with --show-clean

! 2 warning(s) across 1 file(s).
exit=1

```

Settles est le composant propre caché ; les deux témoins tirent, en WARNING, parce que leurs gardes pourraient converger dans des cas que l'analyse ne tranche pas. Le journal a mesuré l'effet de la correction sur le corpus de l'époque : 1 359 → 1 343, seize localisations retirées et aucune ajoutée. Cette ligne est antérieure à l'épinglage du corpus et ne peut plus être vérifiée (section 25.8).

25.4.3 Retirer une preuve induite : try/catch/finally (#2)

La paire peut porter sur le *niveau* plutôt que sur la présence. Le lowering de try/catch/finally avait deux défauts dans une même branche : un try qui retourne scellait le bloc, de sorte que le catch et le finally n'étaient pas abaissés du tout ; et quand ils l'étaient, ils étaient séquencés *inconditionnellement* après le corps du try, ce qui faisait croire au raisonnement « sur tous les chemins » qu'une écriture présente seulement dans le catch avait lieu à chaque rendu. La fixture tests/fixtures/try_catch_finally/App.tsx contient une forme par cellule :

Extrait 25.13 – tests/fixtures/try_catch_finally/App.tsx, l. 29–49 — la rétrogradation et son témoin

```

29 // The other half of #2: when the gate did pass, catch and finally were
30 // sequenced *unconditionally* after the try body, so all-paths reasoning was
31 // told a catch-only write happens on every path. It does not – this is a
32 // Warning, not an Error.
33 export function CatchOnlyWrite() {
34   const [n, setN] = useState(0);
35   try {
36     console.log("ok");
37   } catch (e) {
38     setN(1);
39   }
40   return <div>{n}</div>;
41 }
42
43 // The control for that: a write with no `try` around it really is on every
44 // path, and must stay an Error.
45 export function AlwaysWrite() {
46   const [n, setN] = useState(0);
47   setN(1);
48   return <div>{n}</div>;
49 }

```

```

AlwaysWrite (1 hooks) tests/fixtures/try_catch_finally/App.tsx
  error setter-in-render [hook:0] (line 47:2) setter `setN` called directly in the render body, move this call into
    ↳ a useEffect or an event handler
CatchOnlyWrite (1 hooks) tests/fixtures/try_catch_finally/App.tsx
  warn setter-in-render [hook:0] (line 38:4) setter `setN` called directly in the render body, move this call into
    ↳ a useEffect or an event handler
FinallyAfterReturn (1 hooks) tests/fixtures/try_catch_finally/App.tsx
  warn setter-in-render [hook:0] (line 25:4) setter `setN` called directly in the render body, move this call into
    ↳ a useEffect or an event handler
ReturnInTry (2 hooks) tests/fixtures/try_catch_finally/App.tsx
  error conditional-hook [hook:1] (line 13:4) this hook is called conditionally (not on every render path)
  warn infinite-loop [hook:0] (line 13:4) this effect keeps pushing state `n` to new values on every run.
    ↳ Potential infinite render loop

! 2 error(s), 3 warning(s) across 1 file(s).

```

(Sortie de `check tests/fixtures/try_catch_finally -all-roots`, lignes « trace step(s) » élaguées.)

Avant #2, `CatchOnlyWrite` était une `ERROR` « claimed all-paths », et `ReturnInTry` était invisible (« 1 hook, ✓ »). Le test `a_catch_only_write_is_not_certain` exige désormais "warning" pour `CatchOnlyWrite`; son témoin, `a_write_on_every_path_is_still_certain`, exige "error" pour `AlwaysWrite`, avec ce commentaire : « the demotion must be caused by the try, not by breaking the Error tier » (`tests/try_catch_finally.rs`, l. 94–103). Sans ce témoin, une modification qui aurait rétrogradé *toutes* les `ERROR` de `setter-in-render` aurait fait passer le test de rétrogradation.

Remarque

`FinallyAfterReturn` est un `WARNING` alors que JavaScript exécute toujours le `finally`. C'est une divergence acceptée et documentée : le `return` du `try` scelle son bloc, le chemin qui retourne n'atteint donc pas le `finally` dans le CFG ; celui-ci reste atteint par le bras qui lève une exception, si bien que ses hooks et ses écritures sont trouvés. « What is lost is its presence on the returning path, which costs must strength (an Error demoted to a Warning) and never a finding » (`docs/precision-log.md`, l. 840–846). La perte est dans le sens toléré.

25.4.4 Le test par retrait (*gate-by-removal*)

Le journal de précision emploie une seconde vérification, manuelle : désactiver le correctif et constater que *exactement* le test attendu devient rouge. « The false negative is real (unit test plus *gate-by-removal*: putting `Heap::new()` back turns exactly the member-dep test red) » (`docs/precision-log.md`, l. 560–562). La procédure établit deux choses que le test seul n'établit pas : le test dépend bien du correctif (il n'est pas vert par accident) et le correctif n'a pas d'autre effet observable dans la suite (aucun autre test ne rougit). Elle n'est pas automatisée ; elle est consignée dans l'entrée du journal.

Piège

Écrire le test « ne tire plus » *après* le correctif, sur le programme que l'on vient de corriger, ne prouve rien : on observe ce que l'on a construit. L'ordre sain est celui du journal : écrire la paire, constater que la moitié *silent* est rouge sans le correctif et que la moitié *fires* est verte, appliquer le correctif, constater que les deux sont vertes, puis retirer le correctif pour vérifier que seul le test attendu rougit.

25.5 Le silence n'est une preuve que si l'on a regardé

25.5.1 Un défaut que la sortie cachait

La fixture `tests/fixtures/blind_spots/outside_root/` contient un répertoire `app/` et un répertoire `shared/`. Le composant `App` de `app/App.tsx` appelle `useThing`, défini dans `shared/useThing.ts` :

Extrait 25.14 – `tests/fixtures/blind_spots/outside_root/shared/useThing.ts`, l. 1–10 — une boucle infinie que personne ne voit quand on analyse `app/` seul

```
1 import { useEffect, useState } from "react";
2
3 // An infinite loop nobody sees when the run is pointed at `app/` alone.
4 export function useThing() {
5   const [n, setN] = useState(0);
6   useEffect(() => {
7     setN(n + 1);
8   });
9   return n;
10 }
```

Le fichier `app/Broken.tsx` contient la même boucle, écrite directement. Analysons `app/` seul :

```
Broken (2 hooks) tests/fixtures/blind_spots/outside_root/app/Broken.tsx
warn infinite-loop [hook:0] (line 7:2) this effect keeps pushing state `n` to new values on every run. Potential
  ↳ infinite render loop
    (2 trace step(s), rerun with --trace)
1 clean component(s) hidden, rerun with --show-clean

! 1 warning(s) across 2 file(s).
not analyzed:
  • 1 imported file(s) resolved outside the analysed set and were never read. Pass them on the command line to analyse
    ↳ them (tests/fixtures/blind_spots/outside_root/shared/useThing.ts)
exit=1
```

`App` est déclaré propre (« 1 clean component(s) hidden »), mais la ligne « not analyzed » dit pourquoi ce silence ne vaut rien : l'import de `useThing` a été résolu vers un vrai fichier que le run n'a pas lu. C'est un *blind spot* (définition 23.3), et le JSON le porte dans un tableau dédié : `"blind_spots": [{"kind": "unread-imports", "count": 1, ...}]`. Sur le répertoire parent, le finding apparaît, attribué à `App` mais localisé dans le fichier du hook :

```
App (1 hooks) tests/fixtures/blind_spots/outside_root/app/App.tsx
warn infinite-loop [hook:1] (tests/fixtures/blind_spots/outside_root/shared/useThing.ts:6:2) this effect keeps
  ↳ pushing state `n` to new values on every run. Potential infinite render loop
```

25.5.2 Trois propriétés tenues ensemble

`tests/blind_spots.rs` pilote le binaire, parce que « the summary line *is* the behaviour under test » (`tests/blind_spots.rs`, l. 10). Ses tests tiennent trois propriétés, et c'est leur conjonction qui a du sens :

Extrait 25.15 – `tests/blind_spots.rs`, l. 31–51 — le témoin et le retrait de la coche

```
31 /// The control: a run with nothing unread still gets the green tick. Without
32 /// this the fix could "pass" by never claiming a clean bill at all.
33 #[test]
34 fn a_run_that_read_everything_keeps_its_clean_bill() {
35   let out = reactant(&["tests/fixtures/clean.tsx"]);
36   let s = stdout(&out);
```

```

37     assert!(s.contains("no issues found"), "{s}");
38     assert!(!s.contains("not analyzed:"), "{s}");
39     assert_eq!(out.status.code(), Some(0));
40 }
41
42 /// Aliases the resolver cannot load: every `@/...` target is unlowered, so the
43 /// run has no basis for a clean bill.
44 #[test]
45 fn unloadable_aliases_withhold_the_clean_bill() {
46     let out = reactant(&["tests/fixtures/blind_spots/vite_no_paths"]);
47     let s = stdout(&out);
48     assert!(!s.contains("no issues found"), "{s}");
49     assert!(s.contains("not a clean bill"), "{s}");
50     assert!(s.contains("no tsconfig `paths` found"), "{s}");
51 }

```

1. Un run qui a tout lu garde sa coche (`a_run_that_read_everything_keeps_its_clean_bill`). Le commentaire donne la raison : sans ce témoin, le correctif « passerait » en ne certifiant plus jamais rien.
2. Un angle mort retire la coche *et nomme* ce qui n'a pas été lu. Tests : `unloadable_aliases_withhold_the_clean_bill` et `an_import_resolved_outside_the_run_is_named`.
3. Un angle mort n'est pas un finding et ne change pas le code de sortie. Test : `a_blind_spot_is_not_a_finding`.

Le dernier test du groupe prouve que l'angle mort n'est pas décoratif :

Extrait 25.16 – `tests/blind_spots.rs`, l. 63–80 — ce que l'angle mort cachait, et ce qu'il ne change pas

```

63 /// ...and the finding it was hiding is real: pass the directory above and the
64 /// `infinite-loop` shows up. The blind spot is not decoration.
65 #[test]
66 fn the_unread_import_was_hiding_a_finding() {
67     let out = reactant(&["tests/fixtures/blind_spots/outside_root"]);
68     let s = stdout(&out);
69     assert!(s.contains("infinite-loop"), "{s}");
70     assert!(!s.contains("not analyzed:"), "{s}");
71 }
72
73 /// A blind spot is not a finding: it says the counts are a lower bound, not
74 /// that something is wrong. The exit code keeps meaning "findings were
75 /// reported", so a CI pipeline's contract does not silently change.
76 #[test]
77 fn a_blind_spot_is_not_a_finding() {
78     let out = reactant(&["tests/fixtures/blind_spots/vite_no_paths"]);
79     assert_eq!(out.status.code(), Some(0), "{s}", stdout(&out));
80 }

```

La troisième variante de la fixture, `vite_no_paths`, déclare ses alias `@/` seulement dans `vite.config.ts`. Le run n'émet aucun finding et sort avec 0, mais il refuse la coche :

```

! 1 file(s), no findings, but parts of this run were not analyzed, so this is not a clean bill.
not analyzed:
  • no tsconfig `paths` found, so aliased imports (e.g. `@/...`) stay unresolved and their targets are NOT analyzed (
    ↳ possible false negatives). Aliases declared only in vite.config are not read.
exit=0

```

Décision : #9, #47 — un angle mort n'est pas un finding

Deux politiques étaient possibles : faire échouer un run incomplet (code de sortie non nul), ou le laisser réussir en retirant l'assurance. Le projet choisit la seconde, parce que le code de sortie signifie « des findings ont été rapportés » et qu'en changer le sens casserait en silence le contrat des pipelines CI. L'Action GitHub expose le nombre d'angles morts dans une sortie séparée, `blind-spots`, en précisant qu'il « never affects the exit code — gate on it yourself » (`action.yml`, l. 53–60).

25.5.3 L'angle mort vu par la mesure

Ces deux runs sont aussi la plus petite démonstration de l'outil de comparaison du corpus, `scripts/corpus-diff.py`, que la section 25.7 détaille. On enregistre les deux runs en JSON, puis on les compare :

```
$ reactant check tests/fixtures/blind_spots/outside_root/app --format json --fail-on never > /tmp/ch25/
  ↪ before.json
$ reactant check tests/fixtures/blind_spots/outside_root --format json --fail-on never > /tmp/ch25/after.
  ↪ json
$ scripts/corpus-diff.py /tmp/ch25/before.json /tmp/ch25/after.json --show 5
```

```
before: 1
after: 2
delta: +1 (0 removed, 1 added)

ADDED (1):
  1 infinite-loop
  tests/fixtures/blind_spots/outside_root/shared/useThing.ts:6:2 this effect keeps pushing state `n` to new values on
  ↪ every run. Potential infinite render loop
```

Un ajout qui n'est la suppression d'aucun défaut préexistant a une signature reconnaissable : c'est celle d'une correction de *soundness*. Le journal de précision l'énonce en tête : « A precision correction only removes locations ; a *soundness* correction adds some, and those additions are findings a bug had been silencing » (`docs/precision-log.md`, l. 13–16).

25.6 Le corpus : quatorze dépôts épinglés

25.6.1 Pourquoi un corpus

Les fixtures sont écrites par ceux qui écrivent les règles. Elles contiennent les formes auxquelles on a pensé. Le corpus contient les autres : quatorze applications et bibliothèques React réelles, 40 164 fichiers `.ts`, `.tsx`, `.js` et `.jsx` hors `node_modules`, 1,1 Go. Le répertoire `test-repo/` est ignoré par Git et reconstruit par `scripts/setup-test-repo.sh`.

Le choix n'est pas le volume. Le script le dit pour chaque groupe : les trois monorepos sont « the ones that actually exercise the scaling paths — they are where the utility-inlining budget is exhausted and where the infinite-loop $O(C^2)$ hang (#86) shows up » (`scripts/setup-test-repo.sh`, l. 33–35) ; les quatre projets Next sont « chosen to cover the four layouts that change how the analyzer resolves a project, not just to add volume » (`scripts/setup-test-repo.sh`, l. 40–46).

25.6.2 Épingler : une mesure sur des sources fixes

Extrait 25.17 – `scripts/setup-test-repo.sh`, l. 8–17 — pourquoi chaque entrée est épinglée

Dépôt	Fichiers source	Pourquoi il est là
twenty	25 033	monorepo (CRM)
mantine	5 490	monorepo (bibliothèque UI)
dub	4 327	monorepo (Next.js)
chakra-ui	2 758	bibliothèque UI
excalidraw	664	application Vite, alias dans vite.config
memos	569	application
bulletproof-react	425	gabarit d'architecture
next-shadcn-dashboard-starter	300	Next, src/app/, @/* → ./src/*
shadcn-admin	235	application
ai-chatbot	154	Next, app/ à la racine, @/* → ./*
commerce	65	Next, baseUrl sans paths
novel	61	éditeur
zustand	48	bibliothèque d'état
precedent	35	Next, alias multiples

TABLE 25.2 – Les quatorze dépôts du corpus et leur nombre de fichiers source (recompté au commit e67b10a).

```

8 # Chaque entrée est ÉPINGLEE à un commit. Sans cela le corpus suit la branche
9 # par défaut de quatorze dépôts tiers, et un chiffre corpus change parce que
10 # quelqu'un d'autre a poussé – la mesure ne veut plus rien dire, et une CI qui
11 # la surveillance échoue pour des raisons qui ne sont pas des régressions (#15).
12 # `degit` était plus rapide mais jetai `.git`, donc le corpus n'était même pas
13 # vérifiable après coup ; un clone superficiel sur un SHA garde l'identité.
14 #
15 # Pour bouger le corpus : changer un SHA ici, re-cloner avec --force, puis
16 # régénérer la ligne de base avec scripts/corpus-baseline.py. Les trois vont
17 # ensemble, dans un commit qui dit pourquoi.

```

Chaque entrée du manifeste est un triplet « source GitHub, nom local, SHA », et le clonage récupère exactement ce commit, en profondeur 1 :

Extrait 25.18 – scripts/setup-test-repo.sh, l. 102–107 — un clone superficiel sur un SHA

```

102 echo ">> $src@${sha:0:12} -> test-repo/$name"
103 mkdir -p "$target"
104 git -C "$target" init -q
105 git -C "$target" remote add origin "https://github.com/$src.git"
106 git -C "$target" fetch -q --depth 1 origin "$sha"
107 git -C "$target" checkout -q FETCH_HEAD

```

Le mode `-verify` ne clone rien : pour chaque dépôt, il imprime `ok`, `missing`, `unpinned` (un ancien clone sans `.git`) ou `DRIFT have=... want=...`, et sort avec 1 au moindre écart (scripts/setup-test-repo.sh, l. 65–84). Sur la machine de rédaction, les quatorze lignes sont `ok`.

Décision : #15 — épingler le corpus

Avant le 4 septembre 2026, le script clonait la branche par défaut de chaque dépôt avec `degit`, qui jette `.git`. Le corpus suivait donc les pushes d'autrui, et deux mesures prises à deux dates ne portaient pas sur les mêmes sources ; pire, on ne pouvait même pas vérifier après coup sur quoi une mesure avait porté. L'épinglage par SHA avec un clone superficiel coûte un peu de temps de clonage et garde l'identité. Le prix a été une rupture de série (section 25.8) : re-cloner a fait passer le corpus de 34 747 à 40 164 fichiers.

25.6.3 Un seul arbre, et ce que cela coûte

La mesure officielle lance l'analyseur sur *test-repo* comme *un seul arbre* : `reactant -format json -fail-on never test-repo`. Ce choix a deux conséquences qu'il faut connaître avant de lire un chiffre du corpus. D'abord, le répertoire racine n'est pas un projet : aucun des quatorze `tsconfig.json` n'est traité comme racine de projet et *aucun alias n'est chargé* (#139 : « fourteen projects do not run as one »). Ensuite, des composants homonymes de dépôts différents se rencontrent ; lors de #7, 84 % des références ambiguës venaient de collisions de noms *entre* dépôts (« Eighty-four per cent of it is the instrument »). La baseline mesure donc une configuration qu'aucun utilisateur ne lance. Les mesures « par projet » existent, mais elles ne sont pas comparables à la baseline : pour le même binaire, 1 318 localisations projet par projet contre 1 346 en arbre unique (`docs/campaign/rerender-cascade-plan.md`, l. 300–303).

Piège

Le run corpus complet demande plus de 8 Go de mémoire : « twenty alone peaks at 6.6 GB » (`docs/campaign/rerender-cascade-plan.md`, l. 295–296). Sur une machine de développement, il peut être tué par le système avant d'écrire son JSON. C'est une des raisons pour lesquelles la baseline est normalement régénérée depuis l'artefact de la CI, et non depuis un run local (section 25.9).

Figure 25.2 résume le chemin d'une mesure, du manifeste épinglé au verdict. La section suivante en détaille l'unité de compte et la porte.

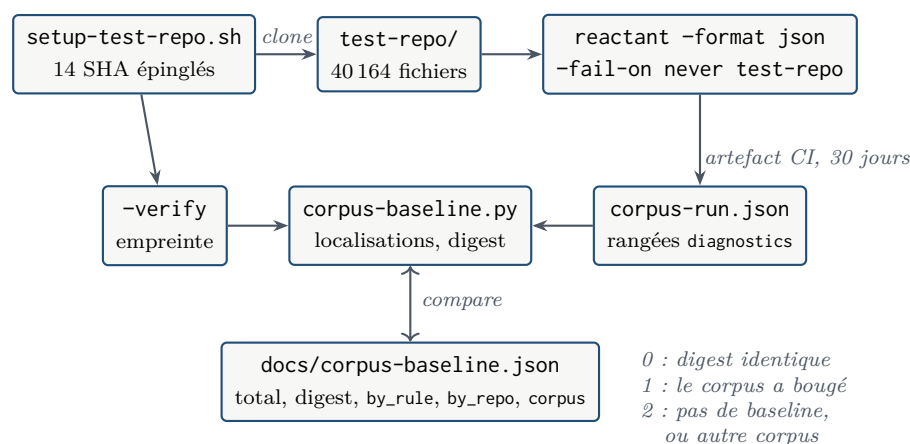


FIGURE 25.2 – Le flot de la mesure corpus : un manifeste épinglé, un run JSON, une porte à trois codes de sortie. L'empreinte du corpus est vérifiée *avant* le digest.

25.7 La métrique : rangées, localisations, digest

25.7.1 Une rangée n'est pas un défaut

Un hook partagé défectueux, inline dans trois composants, produit trois diagnostics : un par composant qui le consomme (ADR-24). La fixture `tests/fixtures/shared_hook_repeat/` en est l'illustration :

```

Alpha (1 hooks) tests/fixtures/shared_hook_repeat/App.tsx
warn  infinite-loop [hook:1] (tests/fixtures/shared_hook_repeat/hooks/useShared.ts:8:2) this effect keeps pushing
  ↳ state `value` (its deps do not provably gate it, so the effect can re-run every render) to new values on every
  ↳ run. Potential infinite render loop [in 3 components]
      (2 trace step(s), rerun with --trace)
  
```

```
2 component(s) hidden. Every finding in them is a source line already reported above
! 1 warning(s) across 2 file(s), 3 component attribution(s).
```

La sortie humaine groupe (définition 23.5) ; le JSON, lui, garde *trois* rangées dans `diagnostics`, chacune avec son `component`. Compter les rangées reviendrait à compter trois fois le même défaut, et à faire bouger le compteur chaque fois qu'un composant de plus consomme le hook. À l'échelle du corpus, l'écart est énorme : « 6,322 produced findings resolve to 1,170 distinct source locations » (`docs/limitations.md`, l. 235–236), soit 81 % de répétition.

Définition 25.3: Localisation distincte (corpus)

Une *localisation distincte* est un quadruplet (`file`, `line`, `col`, `message`) pris sur les rangées du tableau `diagnostics` d'un rapport JSON. Le total d'un run est le cardinal de l'ensemble de ses localisations distinctes.

Extrait 25.19 – `scripts/corpus-baseline.py`, l. 36–40 — l'unité comparable, dupliquée à l'identique dans `corpus-diff.py`

```
36 def locations(path):
37     """Distinct (file, line, col, message) tuples -> rule. The comparable unit."""
38     with open(path) as fh:
39         diags = json.load(fh)["diagnostics"]
40     return {(d["file"], d["line"], d["col"], d["message"]): d["rule"] for d in diags}
```

Chaque composante a un rôle. `file`, `line` et `col` localisent ; `message` distingue deux findings au même endroit. La *valeur* du dictionnaire, la règle, ne sert qu'aux ventilations. Ce qui n'entre pas dans la clef est aussi important : `component` (c'est voulu, la métrique compte des défauts, pas des attributions), `severity` et `notes`.

Remarque

La clef du corpus n'est pas tout à fait celle du groupage du driver, qui inclut la règle : (`rule`, (`fichier`, `ligne`, `colonne`), `message`) (définition 23.5). Deux règles qui émettraient le même message au même endroit feraient deux groupes à l'affichage et une seule localisation au corpus. Le cas est théorique, les messages étant propres à chaque règle, mais il vaut d'être su si l'on compare le résumé humain et le chiffre du corpus.

Sur la fixture, `scripts/corpus-diff.py /tmp/ch25/shr.json` répond « 1 distinct locations », pour un JSON de trois rangées.

25.7.2 La baseline commitée

Le chiffre du corpus vit dans un fichier, `docs/corpus-baseline.json`, produit par l'outil et jamais tapé :

Extrait 25.20 – `docs/corpus-baseline.json`, l. 1–23 — le total, le digest et la ventilation par règle

```
1 {
2   "total": 1498,
3   "digest": "e3f8016ea8848a5c25b1942dff2e512957ca960795ea4e4dd952699bac158081",
4   "by_rule": {
5     "always-unstable-deps": 381,
6     "conditional-hook": 9,
7     "cross-component-infinite-loop": 18,
8     "cross-setter-in-render": 6,
9     "frozen-initial-state": 81,
10    "infinite-loop": 35,
```



```

11     "lazy-init": 222,
12     "missing-cleanup": 3,
13     "missing-deps": 476,
14     "redundant-set-state": 6,
15     "server-component-hook": 1,
16     "setter-in-render": 35,
17     "stale-closure": 2,
18     "state-lifted-too-high": 26,
19     "state-mutation": 15,
20     "unnecessary-rerender": 10,
21     "unstable-context-value": 51,
22     "wasted-subtree-render": 121
23 },

```

Suivent `by_repo` (de 1 pour `precedent` à 535 pour `dub`) et, à la dernière ligne, l’empreinte `00e5fa79fa000c85`. Les cinq champs sont calculés par les fonctions `summarize` et `fingerprint` de `scripts/corpus-baseline.py` (l. 65–88) :

- `total` est le nombre de localisations distinctes ;
- `digest` est un SHA-256 de la liste *triée* des localisations ;
- `by_rule` et `by_repo` sont des ventilations, qui disent *où* une dérive s’est produite ; un fichier hors de `test-repo/` est rangé dans (`other`), une localisation sans fichier dans (`unlocated`) ;
- `corpus` est l’empreinte du corpus mesuré, ou `"unverified"`.

Extrait 25.21 – `scripts/corpus-baseline.py`, l. 53–79 — le digest et l’empreinte

```

53 def digest(index):
54     """Exact-match hash over the sorted location list.
55
56     The counts alone would let an equal number of removals and additions pass
57     unnoticed – which is exactly the shape the 2026-09-03 error took.
58     """
59     h = hashlib.sha256()
60     for key in sorted(index, key=lambda k: (str(k[0]), k[1] or 0, k[2] or 0, k[3])):
61         h.update(f"{key[0]}|{key[1]}|{key[2]}|{key[3]}\n".encode())
62     return h.hexdigest()
63
64
65 def fingerprint():
66     """Corpus identity, from the pinned manifest. `None` when unverifiable."""
67     try:
68         out = subprocess.run(
69             [str(ROOT / "scripts" / "setup-test-repo.sh"), "--verify"],
70             capture_output=True,
71             text=True,
72             timeout=120,
73         )
74     except (OSError, subprocess.SubprocessError):
75         return None
76     lines = [ln.strip() for ln in out.stdout.splitlines() if ln.strip()]
77     if not lines or any("ok" not in ln for ln in lines):
78         return None
79     return hashlib.sha256("\n".join(sorted(lines)).encode()).hexdigest()[:16]

```

Le *digest* existe parce que les totaux ne suffisent pas : un run qui retire trois localisations et en ajoute trois autres garde le même total et les mêmes ventilations si les règles et les dépôts coïncident. Le tri utilise `k[1] or 0` parce qu’un finding sans position porte `line: null`, et que `None < int` lèverait une exception en Python. L’*empreinte* est le SHA-256, tronqué à seize chiffres hexadécimaux, des lignes que `setup-test-repo.sh --verify` imprime ; une seule ligne qui n’est pas

ok la rend None.

Définition 25.4: Baseline, empreinte

La *baseline* est le fichier docs/corpus-baseline.json : total, digest, ventilations et *empreinte* du corpus sur lequel ils ont été mesurés. Deux runs sont *comparables* si et seulement s'ils ont la même empreinte ; ils sont *égaux* si et seulement s'ils ont de plus le même digest.

25.7.3 La porte : comparer, ou refuser de comparer

Le mode par défaut de scripts/corpus-baseline.py compare un run à la baseline. L'ordre des vérifications compte :

Extrait 25.22 – scripts/corpus-baseline.py, l. 114–130 — l'identité du corpus avant le digest

```

114     if not BASELINE.exists():
115         print(f"no baseline at {BASELINE.relative_to(ROOT)} – run with --generate", file=sys.stderr)
116         return 2
117     base = json.loads(BASELINE.read_text())
118
119     if base.get("corpus") != summary["corpus"]:
120         print(
121             f"corpus mismatch: baseline={base.get('corpus')} run={summary['corpus']}\n"
122             "Comparing runs from two different corpora is meaningless. Re-clone with\n"
123             "scripts/setup-test-repo.sh --force, or regenerate the baseline.",
124             file=sys.stderr,
125         )
126         return 2
127
128     if base["digest"] == summary["digest"]:
129         print(f"corpus unchanged: {summary['total']} distinct locations")
130         return 0

```

L'empreinte est vérifiée *avant* le digest : une mesure sur d'autres sources n'est jamais présentée comme un delta, elle est refusée avec le code 2. Si le digest est identique, le script imprime « corpus unchanged » et sort avec 0. Sinon, il imprime le total, le delta et les ventilations des écarts, puis sort avec 1 (scripts/corpus-baseline.py, l. 132–159). Voici la porte lancée, en mode comparaison seulement, sur le petit run /tmp/ch25/after.json de la section 25.5 ; le corpus local est bien le corpus épinglé, l'empreinte concorde, le digest diffère :

```

baseline: 1498
run:      2
delta:    -1496

by rule:
-476 missing-deps
-381 always-unstable-deps
-222 lazy-init
...
by repo:
-535 dub
-426 twenty
...
+2 (other)
-1 precedent

Digest changed. For the moved locations themselves, keep the previous
run and use: scripts/corpus-diff.py before.json /tmp/ch25/after.json --show 3

The corpus moved. If that is the point of the change, regenerate with
scripts/corpus-baseline.py <run.json> --generate
and say in the commit message what moved and why.
exit=1

```

(Sortie élaguée aux « ... ». Les deux localisations du petit run tombent dans (other) parce que leur chemin ne commence pas par `test-repo/`. Sans `-generate`, le script n'écrit rien : `git status docs/corpus-baseline.json` est resté vide.)

La baseline ne contient que le *hachage* de la liste des localisations, pas la liste. Pour nommer ce qui a bougé, il faut deux runs complets et le second script.

25.7.4 corpus-diff.py : compter les deux côtés

Extrait 25.23 – `scripts/corpus-diff.py`, l. 84–97 — deux ensembles, trois nombres, une réconciliation

```

84     before, after = locations(args.before), locations(args.after)
85     removed, added = set(before) - set(after), set(after) - set(before)
86
87     print(f"before: {len(before)}")
88     print(f"after: {len(after)}")
89     print(f"delta: {len(after) - len(before):+d} ({len(removed)} removed, {len(added)} added)")
90     fmt("REMOVED", removed, before, args.show)
91     fmt("ADDED", added, after, args.show)
92
93     # The endpoint is `after`, counted. Never `before` minus removals.
94     if len(before) - len(removed) + len(added) != len(after):
95         print("\nBUG: counts do not reconcile", file=sys.stderr)
96         return 2
97     return 0

```

Le script calcule les deux différences ensemblistes, imprime *trois* nombres, *before*, *after* et *delta* avec (*removed*, *added*), puis ventile chaque côté par règle et, avec `-show N`, liste les premières localisations. Le commentaire de la l. 93 est la règle de méthode : « The endpoint is *after*, counted. Never *before* minus removals. »

Proposition 25.2: La réconciliation est une tautologie

Pour deux ensembles finis A et B , $|A| - |A \setminus B| + |B \setminus A| = |B|$.

Démonstration. A est l'union disjointe de $A \cap B$ et de $A \setminus B$, donc $|A| - |A \setminus B| = |A \cap B|$. De même B est l'union disjointe de $A \cap B$ et de $B \setminus A$, donc $|A \cap B| + |B \setminus A| = |B|$. $\square$

La vérification des l. 94–96 ne peut donc jamais échouer tant que *before* et *after* sont des ensembles (les clefs d'un dictionnaire). Elle protège le script contre une réécriture future qui manipulerait des listes avec doublons, pas contre des données incohérentes. La vraie protection contre l'erreur qui a fait naître le script est ailleurs : *imprimer les deux côtés*, et ne jamais écrire un point d'arrivée à la main. C'est l'objet de la section suivante.

25.7.5 Ce que la métrique ne voit pas

Piège

- **La sévérité n'est pas dans le digest.** Une `ERROR` rétrogradée en `WARNING` à message constant, comme `CatchOnlyWrite` en section 25.4, laisse « corpus unchanged ». Les commits qui changent des niveaux le vérifient à part ; celui de `a7387ba` l'écrit : « The JSON has the same 9,332 rows and the same severities before and after ».
- **Un changement de libellé est un retrait plus un ajout.** Le PR de `e67b10a` : trois retraits, deux ajouts, dont deux sont « one line reworded ». Toujours lire le diff,

jamais seulement le delta.

- **Les positions nulles comptent** (line or 0) : les 25 findings sans position apparus avec #4 et #5 sont des localisations comme les autres (défaut ouvert #140).
- **Le corpus en arbre unique n'est pas quatorze projets** (section 25.6).

Enfin, les totaux sont eux-mêmes gardés par un méta-test qui ne lance pas l'analyseur :

Extrait 25.24 – tests/corpus_baseline.rs, l. 33–67 — la baseline doit être cohérente avec elle-même

```

33 /// Every finding belongs to exactly one rule, so the per-rule counts must add
34 /// up to the total. Editing the total alone is the shape of the error.
35 #[test]
36 fn the_per_rule_counts_sum_to_the_total() {
37     let b = baseline();
38     let total = b["total"].as_u64().expect("total");
39     assert_eq!(sum(&b, "by_rule"), total, "by_rule does not sum to total");
40 }
41
42 /// ...and so does the per-repo breakdown, which is the independent check: a
43 /// consistent edit would have to touch all three.
44 #[test]
45 fn the_per_repo_counts_sum_to_the_total() {
46     let b = baseline();
47     let total = b["total"].as_u64().expect("total");
48     assert_eq!(sum(&b, "by_repo"), total, "by_repo does not sum to total");
49 }
50
51 /// The digest is what makes an equal number of removals and additions visible –
52 /// the counts alone would let that pass, which is close to the shape the
53 /// 2026-09-03 error took. Its absence would silently weaken the gate.
54 #[test]
55 fn the_baseline_carries_a_digest_and_a_corpus_identity() {
56     let b = baseline();
57     let digest = b["digest"].as_str().expect("digest must be present");
58     assert_eq!(
59         digest.len(),
60         64,
61         "sha256 hex digest expected, got {digest:?}"
62     );
63     assert!(
64         b["corpus"].as_str().is_some(),
65         "corpus identity must be recorded, even when it is `unverified`"
66     );
67 }

```

Le raisonnement est dans les commentaires. Qui voudrait faire verdier un build rouge en éditant le total à la main devrait aussi éditer `by_rule` et `by_repo`, deux ventilations indépendantes qui doivent chacune sommer au total (au commit étudié, les deux somment à 1 498) ; et le digest de 64 caractères, qu'on ne peut pas recalculer sans la liste des localisations, doit être présent. Le test ne prouve pas que le chiffre est juste ; il rend coûteuse la seule falsification facile.

25.8 Étude de cas : l'erreur de comptage du 3 septembre 2026

Les outils des deux sections précédentes ne sont pas nés d'une réflexion abstraite. Ils sont nés d'une erreur, que le journal de précision raconte en détail sous le titre « Table correction (2026-09-03) » (docs/precision-log.md, l. 664–752). Elle mérite d'être lue comme une étude de

cas de méthode.

25.8.1 Les faits

Le 3 septembre 2026, plusieurs corrections de précision sont consignées dans le journal, chacune avec son delta et son point d'arrivée, dans une colonne cumulative : chaque ligne part du point d'arrivée de la précédente. Quand on relance les binaires figés de chaque étape, les chiffres ne se reproduisent pas :

Étape	Enregistré	Mesuré
avant #90	1 343	1 348
#90, a member is not the slot	1 340 (−3)	1 345 (−3), delta correct
#134, the identity of an allocation site	1 348 (+8 : 6 retirées, 14 ajoutées)	1 334 (−11 : 26 retirées, 15 ajoutées)
#94, moitié valeurs	1 332 (−16)	1 326 (−8)
#94, moitié timing	1 314 (−18)	1 325 (−1)

TABLE 25.3 – Enregistré contre mesuré, d'après docs/precision-log.md, l. 686–694.

25.8.2 Le raisonnement

La première question est : dérive ou erreur ? La proposition 25.1 y répond. Quatre runs d'un binaire figé sur un dépôt, deux sur le corpus entier, donnent des JSON identiques bit pour bit ; les huit runs de la correction voient le même corpus (34 747 fichiers, 14 016 composants). Il n'y a pas de bruit, donc l'écart est humain. Un détail de procédure a son importance : le binaire de chaque étape a été identifié par une *sonde comportementale* (un cas `handleSubmit` pour la moitié timing, un cas `const { setValue } = useForm()` pour la moitié valeurs), pas par sa date ; un premier étiquetage par horodatage s'était trompé.

La seconde question est : où est l'erreur ? Le journal écarte d'abord l'hypothèse naturelle (retraits comptés en rangées, bornes en localisations) : testée, elle ne tient pas. Puis il dégage un motif. Le delta de #90 est exact, mais la colonne porte déjà un écart de +5 hérité du 2 septembre. Pour #134, les ajouts sont presque justes (15 contre 14) mais les retraits sont faux (26 contre 6), et l'entrée nomme précisément une famille, les `$errors.<member>` de mantine : l'explication qui colle est que la famille relue à la main a été prise pour le total. Le signe du delta s'est inversé. Pour #94, les deux moitiés valent ensemble −9 localisations, pas −34.

Counting what you re-read is not counting what changed. It is the same mistake as the subtracted endpoint, at the other end of the calculation.

25.8.3 Deux erreurs symétriques

L'erreur a deux faces. À une extrémité, les *retraits* ont été comptés en relisant une famille, au lieu de comparer deux runs. À l'autre, le *point d'arrivée* a été déduit (départ moins retraits) au lieu d'être compté. Et parce que la colonne est cumulative, une seule ligne fausse décale toutes les suivantes. Ce qui n'est pas remis en cause, le journal le dit aussi : *le sens* de chaque correction, tenu par un test à retrait et par la relecture de chaque localisation retirée. C'est la *taille* annoncée qui était fausse.

Invariant 25.2: Un point d'arrivée est compté, jamais déduit

Tout chiffre du corpus qui figure dans le dépôt est produit par un outil à partir d'un run complet : `corpus-diff.py` imprime `before`, `after`, `removed` et `added` ; `corpus-baseline.py -generate` écrit la baseline. Aucun total n'est obtenu par soustraction, et ce qui a été relu à la source est énoncé séparément de ce qui a changé.

Décision : #15 — le nombre est produit, pas tapé

Après l'erreur, la règle aurait pu rester une consigne. Le commit 386e285 (« `infra: the corpus number is produced, not typed` ») en fait un mécanisme : la baseline committée avec son digest et l'empreinte du corpus, la porte `corpus-baseline.py`, le méta-test `tests/corpus_baseline.rs` et le workflow `corpus` qui rejoue la mesure à chaque push sur `main`. L'alternative, une consigne dans le journal, est exactement ce qui avait échoué.

25.8.4 Les ruptures de série

L'épinglage du lendemain (4 septembre) a coupé la colonne une seconde fois : même binaire 3a068ed, 1 344 sur l'ancien corpus, 1 358 sur le nouveau. Le journal l'écrit sans ambiguïté : « `Do not extend the column across this break` » (`docs/precision-log.md`, l. 876–877). Les lignes du 2 septembre, dont aucun binaire n'a survécu, sont laissées telles quelles et marquées non vérifiables, « `rather than presented as checked` ». Et la section de correction garde côte à côte l'enregistré et le mesuré, parce que « `erasing the gap by rewriting the column would do exactly what this log exists to prevent` ».

Piège

Le tableau récapitulatif du journal saute de #7 (1317 → 1348, 5 septembre) à #151 (1493 → 1494, 27 septembre). Les mouvements intermédiaires (1348 → 1346 → 1500 → 1493 → 1494) sont consignés dans les commits `chore(corpus)` et dans `docs/campaign/rerender-cascade-plan.md`, pas dans le journal. Lire la colonne du journal seule donne une série à trous.

Le #151 lui-même part de 1493, une unité sous la baseline committée de 1494 : ce n'est pas une coquille du journal. Le job `corpus` ne tournant pas sur les pull requests (section 25.9), le changement de e257bcc n'a été mesuré pour la première fois qu'après la fusion sur `main`, et le run a échoué (« `run: 1493 / baseline: 1494 / delta: -1` », `-1 state-lifted-too-high` sur `twenty`, run 36263786207). La baseline committée, elle, est restée figée à 1494 — la valeur du dernier `chore(corpus)` avant e257bcc, jamais corrigée pour ce delta de `-1` — jusqu'à sa régénération par 0f576bb (section 25.9) : le #151 part donc à bon droit du 1493 mesuré, pas du 1494 resté écrit dans le dépôt.

25.9 Le cycle de vie d'un chiffre

25.9.1 Le workflow `corpus`

Le workflow `.github/workflows/corpus.yml` a un seul job, `measure`. Il tourne sur les pushes vers `main` qui touchent `src/**`, `crates/**`, `packs/**`, `Cargo.lock`, la baseline, le manifeste du corpus ou le workflow lui-même, et à la demande ; ni sur les pull requests, ni la nuit, pour la raison écrite en tête : « `a full run is ~13 minutes over ~40k files [...]` The corpus is pinned commit by commit, so it cannot drift on its own between those pushes ; a nightly would re-measure an unchanged input » (`.github/workflows/corpus.yml`, l. 11–15).

Extrait 25.25 – `.github/workflows/corpus.yml`, l. 60–86 — vérifier le corpus, mesurer, comparer, garder l'artefact

```

60     # Refuses to go on against a corpus that is not the pinned one – a delta
61     # measured on different sources is not a delta.
62     - name: Verify the corpus is the pinned one
63       run: ./scripts/setup-test-repo.sh --verify
64
65     - run: cargo build --release --locked
66
67     # `--fail-on never` because exit 1 means "findings were reported", and a
68     # corpus of fourteen real apps always has some. This step should fail only
69     # when the analyzer could not run – a usage error still exits 2 and still
70     # fails. Whether the *number* is acceptable is the next step's job.
71     - name: Run the analyzer over the corpus
72       run: ./target/release/reactant --format json --fail-on never test-repo > corpus-run.json
73
74     - name: Compare against the committed baseline
75       run: ./scripts/corpus-baseline.py corpus-run.json
76
77     # Kept whatever the outcome: on a failure this is the "before" a human
78     # needs for `corpus-diff.py`, which is the only thing that names the
79     # locations that moved.
80     - name: Upload the run
81       if: always()
82       uses: actions/upload-artifact@v4
83       with:
84         name: corpus-run
85         path: corpus-run.json
86         retention-days: 30

```

Trois choix se lisent dans les commentaires. L'analyseur tourne avec `--fail-on never`, parce qu'un corpus de quatorze applications réelles a toujours des findings et que « `exit 1 means findings were reported` » : ce pas ne doit échouer que si l'analyseur n'a pas pu tourner. C'est la porte, au pas suivant, qui juge le nombre. Enfin, le JSON est téléversé `if: always()` et gardé trente jours : en cas d'échec, c'est le « avant » dont un humain aura besoin pour `corpus-diff.py`. Le cache du corpus a pour clef le hachage du manifeste (`.github/workflows/corpus.yml`, l. 47–54) : puisque chaque dépôt est épinglé, le corpus n'est re-cloné que si quelqu'un le déplace délibérément.

25.9.2 Rouge, puis vert : une séquence réelle

Un changement qui fait bouger le corpus rend donc main rouge, à *dessein*. La baseline n'est régénérée qu'ensuite, par un commit séparé qui dit ce qui a bougé et pourquoi. Figure 25.3 reconstitue la séquence du 27 septembre 2026 à partir de `gh run list --workflow corpus.yml` et des messages de commit.

Le message de `0f576bb` dit exactement ce qui s'est passé : « Regenerated from the CI artifact of `548f922` (run `36329437512`): the count main has produced since `ADR-042` (`#152`) landed, six additions all attributed in `docs/precision-log.md`. The corpus job on main compared against the `1,494` of `432277e` and failed on that delta alone. » La PR `#163`, fusionnée dans `e67b10a`, montre la variante : le job corpus ne tournant pas sur les PR, il n'existe pas d'artefact avant la fusion, et la baseline a été régénérée *dans la PR* depuis un run local du binaire final (« the analysis is byte-deterministic, and the run on main after the merge is the check »). Le run sur `main` après fusion est vert : la prédiction locale était juste.

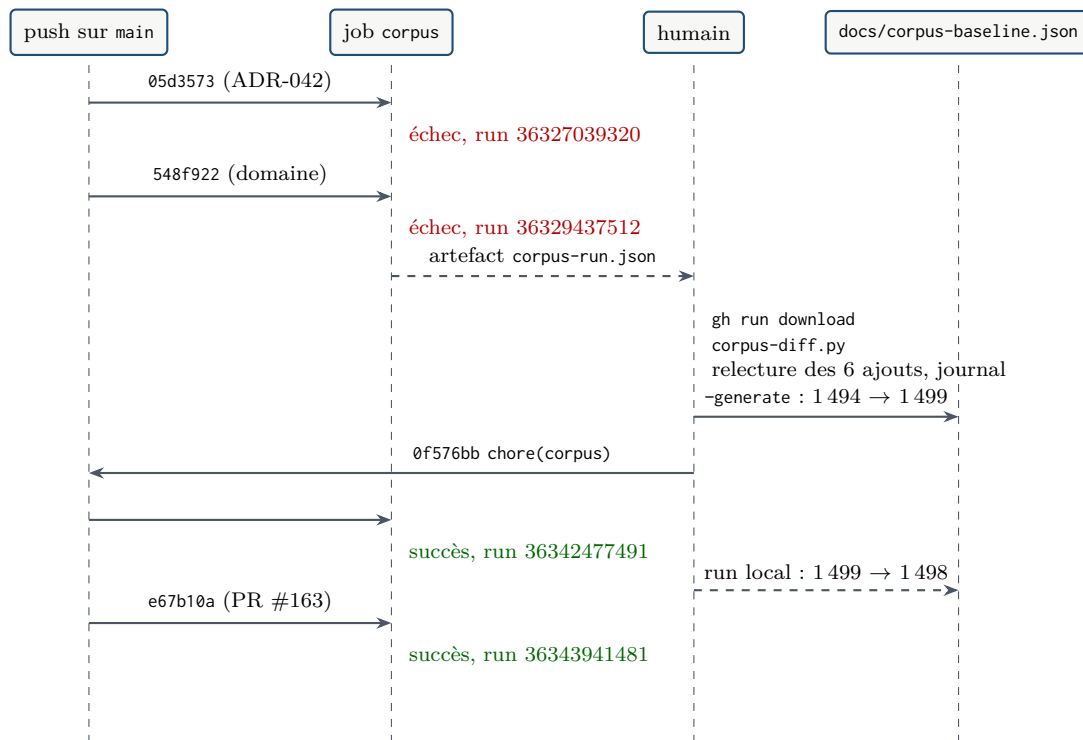


FIGURE 25.3 – Le cycle de vie d’un chiffre, avec les runs réels du 27 septembre 2026. Deux pushes rouges, un chore(corpus) régénéré depuis l’artefact de 548f922, puis un vert ; la PR #163 régénère sa baseline elle-même, faute d’artefact avant fusion.

Commit	Date	Transition	Motif (message de commit)
386e285	2026-09-04	création (1 317)	the corpus number is produced, not typed (#15)
806d114	2026-09-05	1 317 → 1 348	a JSX callee is resolved by the file that writes it (#7)
a7387ba	2026-09-24	1 348 → 1 346	a naming merge, not a finding lost
ba09a62	2026-09-24	1 346 → 1 500	the two render-cascade rules
61610ad	2026-09-24	1 500 → 1 493	keyed handlers are discrete
432277e	2026-09-24	1 493 → 1 494	context values
0f576bb	2026-09-27	1 494 → 1 499	the relations engine and the multi-site proof
e67b10a	2026-09-27	1 499 → 1 498	the convergence sites (#162, #160, #158, #161)

TABLE 25.4 – L’historique de la baseline (git log - docs/corpus-baseline.json).

25.9.3 Ce que fait l'humain entre le rouge et le vert

Le rouge n'est pas une faute; c'est une demande de justification. Entre le rouge et le chore(corpus), la procédure est toujours la même : télécharger l'artefact; comparer avec corpus-diff.py le run d'avant et celui d'après, localisation par localisation; relire à la source *chaque* localisation déplacée; classer chaque retrait (FP retiré, ou vrai positif perdu, ce qui serait un FN) et chaque ajout (défaut qu'un bug faisait taire, ou nouveau FP); rédiger l'entrée du journal; régénérer.

Piège

- **Un delta positif n'est pas une régression.** Les corrections de soundness font monter le compteur : #4/#5 (+26), #2 (+4), #7 (+31).
- **Un delta négatif n'est pas un progrès** tant que chaque retrait n'a pas été relu : un retrait peut être un vrai positif perdu, c'est-à-dire un FN, la direction interdite.
- **Un delta nul n'est pas une correction inutile.** #135 ne bouge rien sur le corpus (0 retiré, 0 ajouté), et pourtant le FN qu'il corrige est réel : « a soundness correction is justified by what it makes impossible, not by what it moves today » (docs/precision-log.md, l. 564–565).

25.10 Les campagnes

Le workflow corpus dit *si* le chiffre a bougé. Il ne dit pas ce qu'il faudrait corriger, ni ce qui manque. Ce travail est celui des *campagnes*, des mesures humaines datées. Le projet en pratique deux sortes : la campagne FP classique, qui part du corpus, et la campagne « wish-list à l'aveugle », qui part de la demande.

25.10.1 La campagne FP classique

Elle suit sept étapes, que chaque entrée du journal de précision reprend.

1. **Mesurer.** Un run corpus, puis un audit de ce qui tire. L'audit du 2 septembre 2026 (docs/campaign/AUDIT.md) compte 6 322 findings, dont « three rules are 92% of them » (missing-deps, always-unstable-deps, lazy-init), et découvre que 81 % de la sortie est une même ligne répétée par composant consommateur, ce qui a donné #129 et la métrique de la section 25.7.
2. **Échantillonner et relire à la source.** Un cluster par règle, un verdict par localisation : vrai positif, FP, ou « not worth fixing ». La passe du 3 septembre sur deux clusters jamais triés a trouvé unstable-context-value à 53 vrais positifs sur 53, et frozen-initial-state à 12 FP sur 79, tous d'une même famille (#136).
3. **Nommer la forme.** Une famille de FP a une cause dans le moteur ; elle devient une issue étiquetée (section 25.12).
4. **Corriger à la racine.** Pas de cas spécial dans une règle : c'est le premier principe de CLAUDE.md. L'entrée de #135 le dit en une phrase : « The fix is central, not per rule ».
5. **Encoder en paires,** dans tests/corpus_fp_fixes.rs ou dans le fichier de la règle, avec test par retrait (section 25.4).
6. **Re-mesurer** avec corpus-diff.py, relire chaque retrait à la source, caractériser les ajouts, ou les déclarer non triés : « They lie in the direction the project's invariant tolerates, not in the forbidden one », et ils « deserve a triage pass ».
7. **Consigner :** une entrée datée du journal (la forme, l'affirmation en une phrase, le delta before → after (N removed, M added), ce qui tire encore et pourquoi, ce qui n'est pas prouvé),

une ligne dans docs/limitations.md si une limite subsiste, et le commit chore(corpus).

Décision : 87f87b5 — une ADR décide, le journal mesure

Sept ADR de l'époque avaient la même forme : une règle était imprécise sur une forme, voici le mécanisme, N localisations retirées. « That is a measurement, not a decision. » Elles sont devenues des entrées du journal, qui le rappelle en tête : une ADR enregistre ce que le reste du système doit respecter (un domaine, une relation, un invariant, une alternative refusée), une correction de précision enregistre une *mesure* (docs/precision-log.md, l. 6–11).

La même grille de lecture est publiée pour les utilisateurs. La procédure de triage skills/reactant-triage/SKILL.md demande, pour chaque WARNING, de tenter de *falsifier* la chaîne de témoins (définition 17.2) étape par étape : « Every step true means a true positive, even when the code looks fine at a glance. One step false means a false positive, and that step names the cause. » (l. 60–61 du même fichier). Et deux interdits : « Silence a finding to make the run green », « Call a component clean when it also carries analysis-limit or suspended ».

25.10.2 La campagne « wish-list à l'aveugle »

tests/catalogue.rs prouve que 21 des 22 entrées du catalogue sont exprimables dans le vocabulaire déclaratif (chapitre 22). La question est juste, mais circulaire : on mesure le vocabulaire contre les règles pour lesquelles on l'a conçu. La campagne du 2 septembre 2026 (#128, docs/campaign/README.md) pose l'autre question : « can it express what someone who has never seen it would ask for ? »

Quatre agents, présentés comme ingénieurs React confirmés, reçoivent une description sommaire de la cible et l'*interdiction de lire le dépôt*. Chacun écrit quinze scénarios dans un domaine (état, effets, rendu, asynchrone), dans un format fixe : *What it flags*, *Why it matters*, *Severity intent*, *Fires on* (la fixture qui doit tirer), *Silent on* (le quasi-voisin difficile qui doit rester muet), *Semantic facts required*. On reconnaît la paire fires/silent de la définition 25.2, écrite cette fois par quelqu'un qui ignore ce que l'outil sait faire. Quatre autres agents trient les soixante scénarios contre le vocabulaire livré, écrivent des règles de pack pour ce qu'ils déclarent exprimable, et *exécutent chaque règle sur sa paire* : « A rule that could not be demonstrated firing was downgraded on the spot » (docs/campaign/README.md, l. 23–26).

Figure 25.4 en montre le déroulé. Le premier passage donne 16 scénarios déjà couverts par une règle native, 1 exprimable par un pack démontré, 16 partiels (un proxy utile, avec un manque nommé) et 27 inexprimables. Chaque fichier de triage finit par une liste de manques « phrased as an issue a maintainer could open ». Deux d'entre eux deviennent des issues puis des ADR : #126 ajoute la relation *calls* (ADR-36), #127 la relation *reads* (ADR-37). Le re-triage donne 16 / 9 / 17 / 18 : huit scénarios basculent vers « exprimable », soit 25 sur 60 atteignables par une règle que quelqu'un peut écrire (docs/campaign/triage-2026-09-02-wave2.md, l. 14–23).

Les huit règles nouvelles sont commitées avec leurs paires, et un test les garde exécutables. Voici la paire du scénario S-STATE-7 :

Extrait 25.26 – tests/fixtures/community_wave2/state7.tsx, l. 1–21 — un slot écrit mais jamais lu au rendu, et son quasi-voisin

```
1 import { useState, useEffect } from "react";
2
3 export function State7Fires() {
4   const [scrollY, setScrollY] = useState(0);
5   useEffect(() => {
6     const onScroll = () => setScrollY(window.scrollY);
7     window.addEventListener("scroll", onScroll);
8     return () => window.removeEventListener("scroll", onScroll);
```

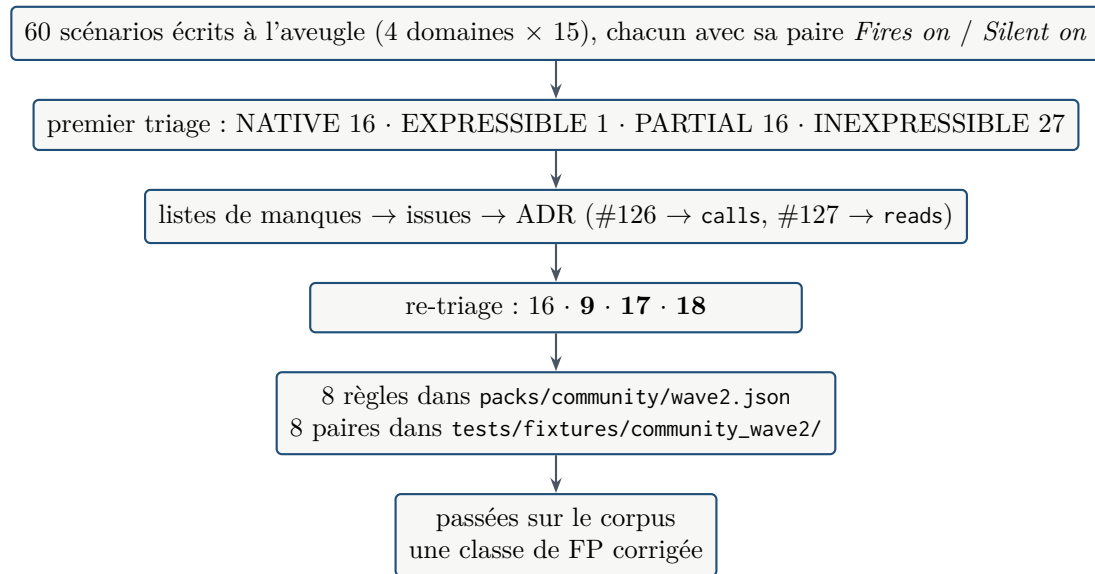


FIGURE 25.4 – L’entonnoir de la campagne à l’aveugle (docs/campaign/README.md, triage-2026-09-02-wave2.md). Chaque étage est une mesure datée ; les triages ne sont pas réécrits quand le vocabulaire change.

```

9   }, []);
10   return <div>static</div>;
11 }
12
13 export function State7Silent() {
14   const [scrollY, setScrollY] = useState(0);
15   useEffect(() => {
16     const onScroll = () => setScrollY(window.scrollY);
17     window.addEventListener("scroll", onScroll);
18     return () => window.removeEventListener("scroll", onScroll);
19   }, []);
20   return <div>{scrollY}</div>;
21 }

```

On copie la paire sous /tmp/ch25/wave2/, à côté d’un fichier de configuration reactant.config.json qui charge le pack packs/community/wave2.json. Le run donne :

```

State7Fires (2 hooks) wave2/state7.tsx
  warn  community-wave2/state-never-read-during-render [hook:0] (line 4:8) state `scrollY` is written but no render
        ↳ -phase read of it is visible – every write costs a render that changes nothing
  1 clean component(s) hidden, rerun with --show-clean

! 1 warning(s) across 1 file(s).
exit=1

```

La règle est un verdict d’*absence* (aucune lecture en phase de rendu n’est visible), et sa propre documentation avoue que « this over-reports and never certifies » : d’où un WARNING, jamais une ERROR. C’est exactement ce que vérifie tests/community_packs.rs, dont le test every_wave2_rule_discriminates_its_fixture_pair exige que chaque règle de la vague 2 tire sur au moins un composant ...Fires et sur aucun ...Silent (tests/community_packs.rs, l. 66–141).

25.11.2 Une paire, mesurée puis analysée

Extrait 25.28 – `scripts/rerender-bench/scenarios/02-drill-before.tsx`, l. 1–21 — prop drilling : un état possédé trois niveaux trop haut

```
1 import { useState } from "react";
2
3 // Prop drilling: App owns `value` only to hand it down through Layout and
4 // Sidebar to Field, which is also the only writer. Every keystroke re-renders
5 // the whole chain plus Layout's other child (Content).
6 function Content() { return <article>static content</article>; }
7 function Field({ value, onChange }: { value: string; onChange: (v: string) => void }) {
8   return <input id="f" value={value} onChange={(e) => onChange(e.target.value)} />;
9 }
10 function Sidebar({ value, onChange }: { value: string; onChange: (v: string) => void }) {
11   return <aside><h2>Filters</h2><Field value={value} onChange={onChange} /></aside>;
12 }
13 function Layout({ value, onChange }: { value: string; onChange: (v: string) => void }) {
14   return <main><Sidebar value={value} onChange={onChange} /><Content /></main>;
15 }
16 export default function App() {
17   const [value, setValue] = useState("");
18   return <Layout value={value} onChange={setValue} />;
19 }
20 export const interaction = "type 5 chars";
21 export async function interact(ui: any) { await ui.type("#f", "hello"); }
```

Le banc a mesuré, pour cinq caractères tapés : « App, Layout, Sidebar, Content x5 each » de rendus gaspillés avant, « Field only » après (`docs/campaign/rerender-cascade-plan.md`, l. 47 ; non relancé ici, Node.js étant absent de la machine de rédaction). L'analyse statique du même fichier :

```
App (1 hooks) scripts/rerender-bench/scenarios/02-drill-before.tsx
warn state-lifted-too-high [hook:0] (line 17:8) state `value` is only used inside `<Field>`, 3 levels below `
  ↳ App`. Every write re-renders `App`, `Layout`, `Sidebar` and 1 other component they render only to pass it down;
  ↳ move the state into `Field`
    ↳ `App` passes it to `<Layout>` as `value`, `onChange` without using it itself (line 18:9)
    ↳ `Layout` passes it to `<Sidebar>` as `value`, `onChange` without using it itself (line 14:15)
    ↳ `Sidebar` passes it to `<Field>` as `value`, `onChange` without using it itself (line 11:32)
1 clean component(s) hidden, rerun with --show-clean

! 1 warning(s) across 1 file(s).
exit=1
```

Sur `02-drill-after.tsx`, l'analyseur répond « ✓ 1 file(s) no issues found. ». Le runtime (quatre composants gaspillés, cinq fois chacun) et le statique (« App, Layout, Sidebar and 1 other ») disent la même chose : c'est en ce sens que le banc est un oracle. Le banc reste un outil manuel, lancé ni par `cargo test` ni par la CI ; ses mesures sont figées dans les fixtures `tests/fixtures/state_lifted_too_high/` et `tests/fixtures/wasted_subtree_render/`, dont les tests annoncent qu'elles encodent des comptes « measured at runtime (React 19, jsdom) ».

25.11.3 Les onze paires au commit étudié

Le banc contient onze paires `-before/-after`. On a relancé l'analyseur sur les vingt-deux fichiers (options par défaut) ; aucun fichier `-after` ne produit de finding. Côté `-before` :

La lecture du tableau 25.5 demande de la prudence : un silence de l'analyseur face à un gaspillage mesuré n'est pas forcément un FN au sens de la soundness, puisque ces règles ne certifient jamais l'absence de gaspillage. Plusieurs silences sont des choix documentés. `wasted-`

Scénario	Gaspillage mesuré au runtime	Findings statiques (défaut)
01 colocate	ExpensiveTree ×5, Row ×15	wasted-subtree-render
02 drill	App, Layout, Sidebar, Content ×5	state-lifted-too-high
03 children	Heavy ×6	aucun
04 toggle	BigList ×2, Item ×8	aucun
05 context	Header ×5	aucun (3 INFO analysis-limit sous -info)
06 memo	List ×5, Row ×15	aucun (1 INFO sous -info)
07 plane	KanBan ×4, Column ×12, Card ×20	aucun
08 dub	Table ×3, TableRow ×9	state-lifted-too-high, wasted-subtree-render
09 Expensify	3 frères ×5	wasted-subtree-render
10 gutenber	ListViewBlock (memo) ×8	aucun (3 INFO sous -info)
11 outline	SortableTable, Table (memo) ×2	unstable-context-value seulement

TABLE 25.5 – Le banc de rendu face à l’analyseur. Colonne du milieu : docs/campaign/rerender-cascade-plan.md, l. 46–56 ; colonne de droite : runs de target/debug/reactant au commit e67b10a, options par défaut ; les INFO ont été observés avec -all-roots -info.

subtree-render ne considère par défaut que les déclencheurs *continus* (frappe, déplacement de souris) ; les déclencheurs discrets exigent l’option `continuousOnly=false`, ce que fixe le test `discrete_triggers_need_the_option` (tests/wasted_subtree_render.rs, l. 138–151). Avec cette option, 04 et 07 tirent bien `wasted-subtree-render` (observé). Le scénario 03 reste muet même avec `continuousOnly=false` (observé) : l’unique composant gaspillé, `Heavy`, n’atteint pas le second seuil de la règle, `minWastedRenders` (src/rules/impls/wasted_subtree_render.rs, l. 32–40), qui n’accepte par défaut que les cas où chaque écriture gaspille *au moins deux* rendus de composant. Avec `minWastedRenders=1` en plus de `continuousOnly=false`, la règle tire deux fois sur `HoverCard` (observé), une fois par handler (`mouseover`, `mouseout`) : le silence par défaut de 03 est donc le seuil, choisi pour ignorer un gaspillage d’un seul composant, pas un défaut lié à la nature booléenne de l’état. Les scénarios 06 et 10 reposent sur `memo`, que le `lowering` ne détecte pas encore (#64, section 25.12), et 05 sur une séparation des fréquences de mise à jour d’un contexte que le plan de campagne classe parmi les règles non écrites.

Décision : ADR-001 — deux oracles concrets, dont un seul a produit des nombres

L’ADR-1 adopte la sémantique de `React-tRace` [LAY25] comme sémantique concrète de référence, et annonce des tests de régression vérifiant que l’analyseur sur-approxime les traces de son interpréteur. Au commit étudié, aucun fichier de tests/ ni de src/ ne mentionne `React-tRace`, et le document docs/semantics.md que l’ADR annonce n’existe pas. L’oracle effectivement utilisé, et qui a produit des nombres versionnés, est le banc jsdom : `React 19` réel, mais sur des scénarios choisis, sans prétention de couverture.

25.12 Classer ce que l’on ne corrige pas

Toutes les mesures précédentes produisent des constats. Une partie devient une correction ; le reste doit être *classé*, de façon à rester citable et à ne pas être rediscuté à chaque fois.

25.12.1 Défaut ou compromis

docs/limitations.md commence par distinguer deux registres qui ne doivent pas être confondus :

Définition 25.5: Défaut, compromis

Un *défaut* (*defect*) est une analyse qui renvoie une sous-approximation ou qui affirme quelque chose de faux : « Those get fixed. » Un *compromis* (*trade-off*) est une analyse qui reste saine et n'est qu'imprécise : « Those get decided. » (docs/limitations.md, l. 16–22).

Le tracker traduit ce partage en labels. `soundness-bug` (« the analysis is wrong, not imprecise ») est un défaut ; `precision-fp` (« known false positive — sound but imprecise ») et `precision-fn` (« known false negative — sound but incomplete ») sont des compromis. S'y ajoutent `infra` (tests, CI, mesure du corpus), `rule-proposal`, `ux`, `blocked`, les tailles `size/S` (quelques heures), `size/M` (un ou deux jours), `size/L` (plusieurs jours, avec un passage de conception), et les zones `area/*`.

Piège

« `precision-fn` » semble contredire « faux négatifs interdits ». Il n'en est rien, parce que le FN d'un `precision-fn` est relatif à l'*intention* de la règle, et que le silence correspondant ne revendique rien : la règle ne certifie pas l'absence, ou l'absence est signalée comme `analysis-limit` ou comme angle mort. Un FN qui produit une assurance fausse, une coche ou un `verified` injustifiés, une `ERROR` manquante là où la preuve existait, est un `soundness-bug`. Le journal le dit de #5 : « An honest "I don't know" turned into four unearned guarantees: the forbidden direction ».

25.12.2 Les issues fermées `wontfix`

Une limite tranchée « on ne corrige pas » est une issue *fermée* avec le label `wontfix`, « so the reasoning stays citable and nobody proposes the fix again » (docs/TODO.md, l. 15–17). Au commit étudié, il y en a six :

#	Titre	Raison et condition de réouverture
#101	<code>nullable-return-unguarded</code> exclu du catalogue	l'ancre serait un site de déréférencement, entité syntaxique interdite ; plafond honnête 21/22 ; aucune réouverture
#65	exports par défaut anonymes	nom générique cosmétique, identité gérée par (fichier, nom)
#63	composants dynamiques (cond ? A : B)	aucun <code>CompApp</code> généré ; rouvrir avec un design de résolution à travers un <code>join</code>
#51	<code>node_modules</code> jamais abaissé	périmètre voulu, le <code>SummaryRegistry</code> est le point d'extension
#42	heuristique de nom d'émetteur de <code>stale-closure</code>	FP par décision, plafond <code>WARNING</code> ; rouvrir si le corpus montre du bruit
#40	lecture de tout l'objet par une garde ou ??	FP par décision, aligné sur <code>eslint</code> ; rouvrir avec un suivi du mode de consommation

TABLE 25.6 – Les six issues fermées `wontfix` (`gh issue list -state closed -label wontfix`).

Chaque fermeture porte, sauf #101 et #65, une *condition de réouverture* : ce qu'il faudrait apporter pour que la décision soit reprise. Ce n'est pas une formule de politesse, comme le montre l'exemple suivant.

Exemple 25.1: La réouverture de #64

L'issue #64 (« `React.memo / forwardRef` wrappers ») avait été fermée wontfix le 27 août 2026, avec la condition « `Reopen with a corpus count of how many components are missed this way` ». La campagne `render cascade` a produit ce chiffre : « `201 hook-bearing components across the corpus are invisible, and a memo barrier cannot be seen` » (`docs/campaign/rerender-cascade-plan.md`, l. 112–114). L'issue a été rouverte le 6 septembre 2026 et le label wontfix retiré; elle est aujourd'hui ouverte, étiquetée `precision-fn` et `area/lowering`. Les documents datés antérieurs, comme `trriage-2026-09-02-wave2.md`, disent encore « wontfix #64 » : ce sont des preuves datées, pas l'état courant.

Décision : ADR-020 — les non-changements délibérés

L'ADR-20 consigne onze refactorisations « évidentes » qu'il ne faut pas tenter, parce que la déduplication introduirait un FN; sa règle transverse est « `never introduce a false negative to remove a duplication` ». Elle doit cependant être lue à la lumière du journal : son item 8 recommandait de garder le tas initial (le seed) de `eval_in` comme argument par appelant, et #135 a ensuite supprimé ce paramètre (« `eval_in now seeds self.heap.clone()` »), parce que quatre des six appelants passaient un tas vide, ce qui produisait un FN. L'ADR n'a pas été réécrite : « `an ADR is a historical record that does not get rewritten` » (`docs/TODO.md`, l. 19–20).

25.12.3 La dette du sous-système lui-même

La méthodologie a ses propres issues infra ouvertes. #14 : 36 des 42 fichiers d'intégration de l'époque construisent `Config::default()`, qu'aucun utilisateur ne lance, et le harnais réel (avec `new_with_common`) devrait être exposé aux tests par une feature ou un `tests/common/mod.rs`. #17 : six fichiers sans tests; au commit étudié, `src/engine/component_registry.rs` en porte désormais six, les cinq autres (`src/registry/keyed.rs`, `src/domains/stores/heap.rs`, `src/ir/source_range.rs`, `src/lowering/import_resolution.rs`, `crates/reactant-wasm/src/lib.rs`) n'en ont toujours aucun, le dernier étant couvert indirectement par `npm/test/smoke.sh`. #18 : une liste de tests à écrire en priorité, dont l'un, le déterminisme sous `-trace`, existe maintenant (`consecutive_traced_runs_are_byte_identical`). Les limites de l'analyse elle-même sont l'objet du chapitre 27, et la façon dont cette discipline s'est formée celui du chapitre 26.

Résumé du chapitre

- La promesse « faux négatifs interdits » rend le silence suspect : tout test de silence a un témoin qui tire (définition 25.1), toute correction de précision une paire `fires/silent` (définition 25.2), et une coche n'est délivrée que si tout a été lu.
- Le niveau d'un diagnostic est une affirmation : `ERROR` = preuve de toute l'affirmation. Un FP ne porte jamais d'`ERROR` (invariant 25.1); les tests fixent le niveau, pas seulement la présence.
- Six altitudes de mesure : unitaire (`src/test_support.rs`), harnais source → règles (`tests/corpus_fp_fixes.rs`), binaire (`tests/blind_spots.rs`, `tests/cli.rs`), parité, corpus, oracle runtime. Le déterminisme octet pour octet fait de tout écart un signal.
- Le corpus : quatorze dépôts épinglés par SHA (`scripts/setup-test-repo.sh`), lancés comme un seul arbre. L'unité est la localisation distincte (`file`, `line`, `col`, `message`) (définition 25.3), pas la rangée JSON.
- La baseline `docs/corpus-baseline.json` porte total, digest, ventilations et empreinte; `scripts/corpus-baseline.py` refuse de comparer deux corpus différents (code 2), valide

un digest identique (0), signale tout mouvement (1). `scripts/corpus-diff.py` nomme ce qui a bougé et imprime toujours les deux côtés.

- L'erreur du 3 septembre 2026 (retraits comptés en relisant une famille, point d'arrivée déduit) a donné l'invariant « un point d'arrivée est compté, jamais déduit » (invariant 25.2) et le mécanisme de #15.
- Un chiffre vit ainsi : push rouge, artefact, diff, relecture, journal, chore(corpus), vert. Les campagnes (FP classique, wish-list à l'aveugle) et le banc jsdom fournissent ce que le chiffre ne dit pas ; ce qui n'est pas corrigé est classé (défaut ou compromis, wontfix avec condition de réouverture).

Le chapitre suivant (chapitre 26) retrace comment ces règles de méthode se sont formées au fil de l'histoire du projet.

25.13 Exercices

Exercice 25.1: Retrouver le témoin

Dans `tests/corpus_fp_fixes.rs`, choisissez trois tests dont le nom dit qu'une règle ne tire plus. Pour chacun, retrouvez le test « tire encore » qui l'accompagne, et énoncez en une phrase la différence de programme qui sépare les deux.

Solution

Pour #91 : `a_write_that_settles_its_own_guard_is_not_a_render_setter` est témoigné par `a_write_derived_from_the_slot_still_loops` (la valeur écrite lit le slot) et par `a_write_of_something_else_still_fires` (la garde compare a, l'écriture stocke b). Les autres se trouvent de la même façon, en cherchant `still` dans les noms et « TP preservation » dans les commentaires.

Exercice 25.2: Même total, digest différent

Construisez deux petits programmes, et donc deux runs JSON, qui ont le même nombre de localisations distinctes, les mêmes ventilations par règle et par dépôt, et des digests différents. Que répondrait `corpus-baseline.py` si l'un était la baseline ? Qu'aurait répondu une porte fondée sur les seuls totaux ?

Solution

Il suffit de déplacer un finding d'une ligne dans le même fichier (par exemple insérer une ligne vide au-dessus d'un effet qui boucle) : le total et les ventilations sont inchangés, la clef (`file`, `line`, `col`, `message`) change, donc le digest aussi. La porte sortirait avec 1 et des ventilations vides ; une porte aux totaux seuls aurait conclu « inchangé », ce qui est la forme de l'erreur du 3 septembre.

Exercice 25.3: Une vérification qui peut échouer

La proposition 25.2 montre que la réconciliation de `corpus-diff.py` ne peut pas échouer. Proposez une vérification qui, elle, pourrait échouer sur des données incohérentes, et dites ce qu'elle détecterait.

Solution

Par exemple : vérifier que `summary.errors + summary.warnings + summary.infos` du JSON égale le nombre de rangées de `diagnostics` ; ou vérifier que le nombre de rangées est au moins le nombre de localisations. Le driver filtre les INFO une seule fois, avant de compter *et*

avant de peupler les diagnostics du composant (`src/driver/mod.rs`, l. 438–438) : l'égalité est donc exacte, avec ou sans `-info` (observé sur `tests/fixtures -all-roots` : $23 + 111 + 0 = 134$ rangées sans `-info`, $23 + 111 + 69 = 203$ avec). La première vérification détecterait plutôt un rapport tronqué ou un résumé incohérent avec son contenu — un compteur mis à jour sans que la liste le soit, par exemple.

Exercice 25.4: Un quatrième consommateur

Sur `tests/fixtures/shared_hook_repeat`, prédissez le nombre de rangées JSON et de localisations distinctes si l'on ajoute un quatrième composant qui appelle le même hook. Vérifiez sur une copie sous `/tmp`.

Solution

Quatre rangées, une localisation : la clef ne contient pas le composant. La sortie humaine dirait « [in 4 components] » et « 4 component attribution(s) ».

Exercice 25.5: Rendre la sévérité visible

Proposez une extension du digest qui rende visibles les changements de sévérité. Argumentez : cela changerait-il la politique de régénération de la baseline ? Qu'aurait montré la correction #2 (section 25.4) ?

Solution

Hacher (`file`, `line`, `col`, `message`, `severity`). La correction #2 aurait alors fait bouger le digest (une `ERROR` devenue `WARNING`), et le workflow aurait exigé un `chore(corpus)` justifiant la rétrogradation, ce qui est souhaitable puisqu'une rétrogradation est une affirmation de moins. Le coût : des régénérations plus fréquentes, et une série où une même localisation peut « bouger » sans changer de place.

Exercice 25.6: Écrire un scénario à l'aveugle

Écrivez, au format des scénarios de campagne (*What it flags*, *Why it matters*, *Severity intent*, *Fires on*, *Silent on*, *Semantic facts required*), un scénario pour un défaut React qu'aucune règle du catalogue ne couvre. Puis triez-le : `NATIVE`, `EXPRESSIBLE`, `PARTIAL` ou `INEXPRESSIBLE`, en vous appuyant sur les relations du chapitre 14.

Chapitre 26

Histoire et doctrine du projet

Objectifs du chapitre.

- Connaître la chronologie du projet, du prototype jeté du 31 mai 2026 au commit `e67b10a` du 27 septembre 2026, et savoir situer chaque ADR dans une phase.
- Comprendre les quatre lieux où le projet range une décision (ADR, journal de précision, issue fermée `wontfix`, `test-cliquet`) et savoir choisir le bon.
- Connaître les principes de `CLAUDE.md` et leur histoire : écrits *après* dix-neuf ADR, puis imposés à tous les suivants.
- Savoir expliquer les onze non-changements d'ADR-020 et, plus généralement, pourquoi une simplification apparente peut être un faux négatif déguisé.
- Suivre les grandes bascules de conception : la polarité devenue un type, les relations devenues des produits du moteur, l'identité séparée de son rendu.
- Savoir lire l'argument de *soundness* d'un changement : dans quelle direction il déplace les findings, et quelle preuve cette direction exige.

Prérequis : ce chapitre est une synthèse : il suppose lus les chapitres techniques, en particulier chapitre 3 (*soundness*, *must/may*), chapitre 10 (*StateValue*, *Stability*), chapitre 14 et chapitre 15 (relations, graphe de *churn*), chapitre 17 (*Certified*, témoins) et chapitre 25 (corpus et mesure). Chaque ADR cité a sa fiche en chapitre A.

Les chapitres précédents décrivent le code tel qu'il est au commit `e67b10a`. Ils répondent à la question « comment ça marche ? ». Ce chapitre répond à une autre question : « pourquoi est-ce fait ainsi, et pas autrement ? ». Presque chaque type central du dépôt porte dans son commentaire le numéro de l'ADR qui le justifie : *StateValue* renvoie à ADR-015, *Certified* à ADR-021, *EdgeKind::Await* à ADR-035, *ProgramRelations* à ADR-042. Lire ces types sans leur histoire, c'est lire la réponse sans la question.

Il y a une raison plus pratique. Un contributeur qui découvre le dépôt voit des duplications, des marches séparées, des niveaux de sévérité qui semblent trop prudents. Il est tenté de « nettoyer ». Or beaucoup de ces formes ont déjà été examinées, mesurées, et conservées parce que leur simplification introduisait un faux négatif (FN). Le projet a donc appris à *écrire ses refus*. Connaître ces refus est la condition pour modifier le code dans l'esprit du projet.

Le plan suit l'ordre chronologique, qui se trouve être aussi l'ordre logique : le problème de départ (section 26.2), les premières semaines (section 26.3), le tournant du corpus (section 26.4), la codification des principes (section 26.5), la polarité devenue un type (section 26.6), les relations (section 26.7), l'identité et l'attribution (section 26.8), les cascades de rendu (section 26.9), la méthode (section 26.10) et enfin les fils rouges (section 26.11).

26.1 Un sous-système qui n'est pas du code

26.1.1 Quatre lieux pour une décision

Prenons un problème concret. Sur le corpus, la règle *infinite-loop* signale un composant qui ne boucle pas. Un contributeur trouve la cause, corrige le moteur, relance le corpus : un finding de moins, aucun de plus. Où écrire ce qui s'est passé ? Et si, au contraire, il décide de ne *pas* corriger, parce que la correction coûterait un FN ailleurs, où écrire ce refus pour que le suivant ne recommence pas ?

Le projet répond par quatre lieux distincts, chacun avec son critère d'entrée.

Définition 26.1: ADR, entrée de journal, wontfix, cliquet

Une décision du projet est rangée dans exactement l'un de quatre lieux :

1. un *ADR* (*Architecture Decision Record*, docs/adr/ADR-NNN-*.md) si elle contraint le reste du système : « un domaine, une relation, un invariant, une alternative refusée » (docs/adr/README.md, l. 3-4) ;
2. une entrée du *journal de précision* (docs/precision-log.md) si c'est une correction de précision mesurée : une règle sur-signalait une forme, le moteur a été corrigé là où l'information se perdait, le corpus l'a mesuré ;
3. une *issue fermée wontfix* sur le tracker si c'est une limite tranchée « on ne corrige pas » ;
4. un *test-cliquet* si la décision doit être rendue exécutoire : un test dont une liste ne peut que rétrécir.

Le journal de précision énonce lui-même la frontière avec les ADR, en gras : « **This is not architecture**, so it does not live in adr/ » (docs/precision-log.md, l. 6). Une correction de précision enregistre une *mesure* : la forme, l'argument qui la tranche, le delta sur le corpus. Une décision enregistre une *contrainte* que les changements futurs doivent respecter.

Exemple 26.1: Ranger quatre situations

- Le domaine des valeurs devient un produit sur les sortes JS (ADR-015) : tout transfert, toute règle qui compare des valeurs en dépend. C'est un ADR.
- La règle *infinite-loop* cesse de signaler une garde dont la borne était mal lue, et le corpus perd vingt findings : c'est une entrée du journal de précision.
- Un composant dynamique `const C = cond ? A : B` n'est pas résolu, et le projet décide de ne pas le faire sans un design pour résoudre une référence de composant à travers un `join` : c'est l'issue #63, fermée wontfix.
- Les règles ne doivent plus parcourir de CFG : la liste des fichiers qui le font encore est figée dans `tests/layer_boundary.rs` et ne peut que rétrécir. C'est un cliquet.

26.1.2 Un corpus de décisions

Au commit e67b10a, ce sous-système compte quarante-deux ADR (6 457 lignes au total, hors index), le fichier `CLAUDE.md` (40 lignes), le journal de précision (1 509 lignes), six issues fermées wontfix et deux cliquets principaux (`tests/layer_boundary.rs` et `tests/catalogue.rs`). Il n'a pas de type Rust propre, mais il est cité par le code dans les deux sens : les types portent le numéro de leur ADR, et les sections « Consequences » des ADR nomment les fichiers touchés. C'est cette double citation qui permet de reconstituer l'histoire depuis le code, et c'est elle que ce chapitre exploite.

Remarque

Les ADR sont écrits en anglais depuis le 6 juin 2026 : les ADR 001 à 013 étaient d'abord rédigés en français et ont été traduits par le commit 4647b85. CLAUDE.md, lui, est resté en français. Les citations de ce chapitre sont données dans leur langue d'origine.

26.2 Le problème de départ

26.2.1 Un prototype, puis une réécriture

Le premier commit, 5b44b77 first commit, date du 8 mai 2026. Il pose un analyseur organisé autour d'un *walker* sur l'AST : `src/engine/walker.rs` (800 lignes), `src/core/aval.rs`, des règles comme `rules/dead_state.rs`. Le 29 mai, le commit 002579b écrit les six premiers ADR, un PRD et un `docs/ir.md`; le 31 mai, f0319bc feat: remove old logic jette le prototype. Le projet commence donc par une *décision de réécriture* motivée par la conception, pas par un bug.

Pourquoi jeter un analyseur qui fonctionnait ? Parce qu'une fonction de transfert écrite directement sur l'AST d'oxc doit traiter des dizaines de formes équivalentes. Considérons ce composant :

Extrait 26.1 – Trois formes syntaxiques pour un même flot de contrôle

```
1 function Badge({ user }) {
2   const [open, setOpen] = useState(false);
3   if (!user) return null;           // retour précoce
4   const label = user.name ?? "anon"; // court-circuit
5   open && setOpen(false);           // effet de bord conditionnel
6   return <span>{label}</span>;
7 }
```

Un retour précoce, un `??` et un `&&` à effet de bord sont trois syntaxes pour une seule chose : un branchement. Un walker sur l'AST doit apprendre les trois ; un CFG n'en connaît qu'une.

Décision : ADR-003 — une IR dédiée à base de CFG

Décision. Abaisser l'AST vers une IR propre, représentée comme un CFG de blocs de base terminés par un Terminator (définition 6.1). **Pourquoi.** « A tree-shaped IR (React-tRace style) requires a CPS-transform pass for early returns and cannot represent loops (while/for) without back-edges. » Le CFG représente nativement les retours précoces et les boucles, place le widening aux en-têtes de boucle et rend la dominance (nécessaire à `conditional-hook`) standard. **Refusé.** Une IR arborescente à la React-tRace. Voir ADR-3.

26.2.2 Deux CFG par composant, et une sémantique de référence

La deuxième décision fondatrice découpe le composant selon les phases de React.

Rappel React : phases

Le corps d'un composant s'exécute pendant le rendu ; les fonctions passées à `useEffect` s'exécutent après le commit. Un `setState` appelé dans un effet programme un nouveau rendu, qui recalcul le corps, puis relance les effets dont un `dep` a changé au sens d'`Object.is`.

Décision : ADR-004 — `render_cfg` et CFG d'effets séparés

Décision. Chaque composant porte un CFG de rendu et, par hook, le CFG de son corps. **Pourquoi.** La séparation rendu/effet est un invariant de React ; autant la rendre structurelle

plutôt que de la laisser implicite. **Refusé.** Un méta-CFG unique « render + effects in a single graph with render→effect→check back-edges », plus expressif mais qui « makes the render-time / effect-time semantic separation implicit » et « risks incorrect modeling of React’s execution order ». Voir ADR-4.

La troisième décision fixe la sémantique concrète dont l’analyse est l’abstraction.

Décision : ADR-001 — React-tRace comme sémantique concrète

Décision. La sémantique opérationnelle React-tRace [LAY25] sert de sémantique concrète de référence ; les fonctions de transfert en dérivent. **Pourquoi.** C’est « the only publicly available React formalization with a conformance proof ». **Limite reconnue.** Elle ne couvre que `useState` et `useEffect` sans tableau de deps ; les extensions devaient être spécifiées dans `docs/semantics.md`. Voir ADR-1.

Piège

ADR-001 annonce trois conséquences : un `docs/semantics.md` qui spécifie les extensions, des fonctions de transfert qui citent la règle React-tRace correspondante, et des tests de régression contre l’interpréteur OCaml. Aucune des trois n’existe au commit `e67b10a` : `docs/semantics.md` n’a jamais été créé. La sémantique concrète du projet est donc celle que les chapitres 2 et 11 reconstituent à partir du code, pas un document. Un lecteur qui cherche « la spécification » ne la trouvera pas ; c’est l’écart le plus ancien entre ADR et code (section 26.10.6).

26.2.3 La frise

La figure 26.1 résume les cinq mois du projet. Le volume de commits (322 au total) se répartit ainsi : mai 5, juin 79, juillet 85, août 40, septembre 113. Quatre pauses nettes découpent le travail en campagnes. Le tableau 26.1 nomme les onze phases que la frise numérote.

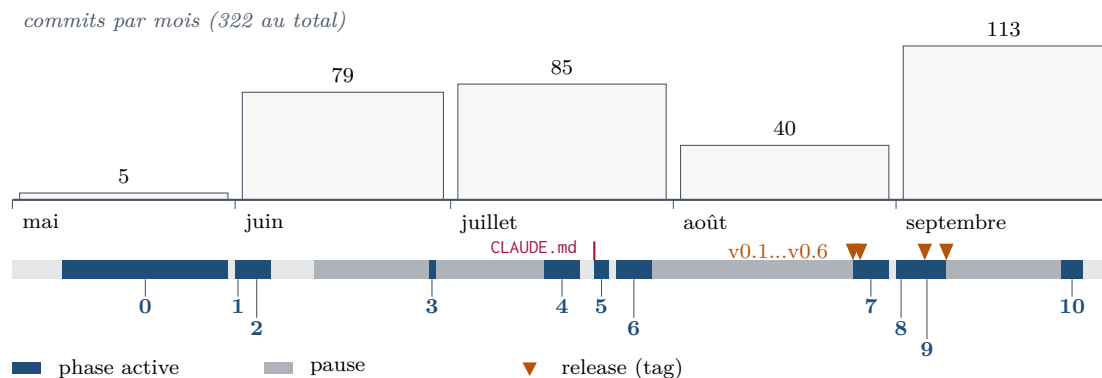


FIGURE 26.1 – Chronologie du projet, du 8 mai au 27 septembre 2026 : histogramme des commits par mois (git log au commit `e67b10a`), phases numérotées comme au tableau 26.1, pauses, releases `v0.1.0` à `v0.6.0`, et arrivée de `CLAUDE.md` le 21 juillet.

#	Dates (2026)	Phase et ADR
0	05-08 → 05-31	prototype sur l'AST, six premiers ADR (001–006), puis f0319bc jette le prototype
1	06-01	le pipeline en un jour : onze commits dans l'ordre du pipeline, de 55d76f6 (types IR) à 3df3cca (CLI)
2	06-02 → 06-06	domaines et moteur : ADR-007 à 013, sévérité à trois paliers, inter-composants, cross-file
3	06-28	précision numérique : ADR-014 (widening à seuils ; narrowing écrit puis abandonné)
4	07-14 → 07-19	le corpus entre en scène : ADR-015 à 019
5	07-21 → 07-23	CLAUDE.md, campagne de dette technique close par ADR-020
6	07-24 → 07-29	surface typée et packs : ADR-021 à 025
7	08-26 → 08-31	distribution (Action, npm, v0.1 à v0.3), ADR-026, le TODO devient le tracker
8	09-01 → 09-02	la vague des relations : treize ADR en deux jours (027–039), catalogue Tier-A 5/21 → 21/22
9	09-02 → 09-08	campagne de précision, ADR de précision rétrogradés en journal, ADR-040 (identité), v0.4 à v0.6
10	09-24 → 09-27	doctrine de sévérité (#142), ADR-041 (cascades de rendu), ADR-042 (relations du moteur), PR #163

TABLE 26.1 – Les onze phases du projet (dates des commits, `git log -date=short`).

26.3 Les premières semaines : un pipeline en un jour

26.3.1 L'ordre des commits est l'ordre du manuscrit

Le 1^{er} juin 2026, onze commits reconstruisent tout le pipeline, dans l'ordre même où les données le traversent : types IR et CFG (55d76f6), détection des composants (cbab629), constructeur de CFG (898621b), lowering des expressions (62f3f7c), extraction des hooks (667cbb8), domaines (ebed38a), stores et trait Transfer (daa8d67), moteur (dd02a31), règles (a1b8db1), registre de hooks (3af23bc), CLI et tests de bout en bout (3df3cca). À la fin de la journée, `src/` compte 6 723 lignes en 41 fichiers. Ce n'est pas un hasard si les parties II à V de ce manuscrit suivent le même ordre : la réécriture a été conçue comme une chaîne, et elle a été écrite comme on la lit.

La semaine suivante (phase 2) ajoute les couches qui font de ce pipeline un analyseur : les requêtes inter-domaines (ADR-7), un premier domaine de valeurs (ADR-8), la traversée des callbacks (ADR-9), le tas par site d'allocation (ADR-10), les positions source (ADR-11), l'analyse inter-composants par inlining descendant (ADR-12) et l'analyse cross-file (ADR-13). Le 6 juin, `src/` a triplé (21 986 lignes, 74 fichiers).

26.3.2 Un choix pragmatique qui a survécu : `Unknown` → `skip`

Parmi ces premiers ADR, un choix mérite l'attention, parce qu'il contredit l'invariant que le projet adoptera un mois plus tard. Considérons :

Extrait 26.2 – Un callback passé à une fonction inconnue

```
1 useEffect(() => {
2   myUtil(() => setCount(count + 1)); // myUtil : helper utilisateur non résolu
3 }, [count]);
```

L'analyse ne sait pas si `myUtil` appelle son argument. Descendre dans la closure serait le choix sound : on ne peut pas prouver qu'elle ne s'exécute pas. ADR-009 a fait l'autre choix, et l'a écrit :

« **Choice of `Unknown` → `skip` default:** for a linter, false positives are more costly than

false negatives. Descending an unknown callee would be the *sound* choice (over-approx: we can't prove cb doesn't run as a consequence), but it would produce an FP on every custom subscription wrapper [...] We accept the FN » (docs/adr/ADR-009-callback-traversal.md, l. 74–78).

Ce choix vit encore dans l'interpréteur du point fixe, sous la forme d'une variante d'énumération :

Extrait 26.3 – src/domains/interp/callbacks.rs, l. 6–23 — les quatre classes de déclencheurs d'ADR-009

```

6  /// How a call's closure arguments should be treated by the side-effect pre-pass.
7  #[derive(Debug, Clone, Copy, PartialEq, Eq)]
8  pub enum TriggerClass {
9      /// Callee is a bound state setter handled by the core exec path (functional
10     /// updaters), so the pre-pass must NOT descend its closure.
11     Setter,
12     /// Runs as a consequence of the current render/effect: synchronous HOFs
13     /// (`map`, `forEach`, ...) and scheduled async (`then`/`.catch`/`.finally`,
14     /// `setTimeout`/`setInterval`, `queueMicrotask`, `requestAnimationFrame`).
15     /// Its closure arguments ARE descended into.
16     InCycle,
17     /// Event subscription (`addEventListener`/`removeEventListener`) triggered
18     /// externally, NOT part of the render→effect→render cycle. Not descended.
19     Subscription,
20     /// Unrecognized callee (custom helper/hook). Conservatively NOT descended
21     /// (FP-averse: avoids flagging custom subscription wrappers).
22     Unknown,
23 }
```

La variante Unknown documente elle-même sa raison : « Conservatively NOT descended (FP-averse) ». Le mot « conservatively » est ici trompeur : du point de vue de la soundness, ne pas descendre est la direction *non* conservatrice.

Piège

TriggerClass::Unknown est la plus grande exception de principe encore présente dans le code. Elle date de juin, CLAUDE.md de juillet. Depuis, la marche des relations a pris l'autre choix : ADR-034 §5 écrit « The walk descends any function argument of an unknown call, at T, because an unknown callee may invoke it ». Le moteur contient donc deux interpréteurs de force différente ; l'issue #12 le dit sans détour (« two CFG interpreters of different strength, and nothing says which ran ») et #19 suit le FN résiduel. Il ne faut ni le cacher ni le « corriger » localement dans une règle : la direction qu'indique la doctrine est d'aligner le point fixe sur la marche des relations, dans le moteur. Au commit e67b10a, aucune ADR ne tranche encore cette correction : #12 reste OPEN.

26.4 Le tournant du corpus

Du 12 juin au 14 juillet, le dépôt ne reçoit presque aucun commit (deux pauses, interrompues par ADR-014 le 28 juin). Quand le travail reprend, il change de méthode. ADR-016 (15 juillet) déclare comme premier objectif « **Corpus benchmarking**. Measuring the real FP/FN rate on public React repos » (docs/adr/ADR-016-cli-projects-json.md, l. 20–21), et le commit b0c5d9f ajoute le script qui génère le répertoire test-repo/. À partir de là, presque chaque décision cite un chiffre. Trois ADR de cette semaine montrent comment la mesure change la forme du code.

26.4.1 ADR-015 : un FN documenté devient une détection

Exemple 26.2: Un compteur nullable

Extrait 26.4 – /tmp/ch26/ex1.tsx

```
1 import { useState, useEffect } from "react";
2
3 export function NullCounter() {
4   const [n, setN] = useState(null);
5   useEffect(() => {
6     setN(n + 1);
7   }, [n]);
8   return <div>{n}</div>;
9 }
```

Au premier tour, n vaut `null`; `null + 1` vaut 1 en JavaScript, puis 2, 3... L'effet dépend de n et l'écrit à chaque fois : boucle infinie.

Sous ADR-008, le domaine des valeurs était une énumération plate : `Bottom`, `Null`, `Number(Interval)`, ..., `Top`. Le join de `Null` et de `Number([1,1])` donnait `Top`, et `Top` ne croît pas : pas de widening, pas de signal. ADR-008 documentait ce FN, et trois rustines s'étaient empilées pour le contourner (un `TypedStateStore`, une fonction `infer_state_type`, et la lecture de l'annotation `useState<T>`).

ADR-015 remplace l'énumération par un produit sur les sortes JS disjointes (définition 10.1) :

Extrait 26.5 – `src/domains/impls/state_value.rs`, l. 16–24 — la justification du produit, en tête du type

```
16 /// Abstract JS value: pointwise product over the disjoint JS kinds (ADR-015).
17 ///
18 /// JS primitive kinds are mutually exclusive (a value is never a number AND a
19 /// string), so the disjunctive completion of the kind sum degenerates into a
20 /// product: one independent slot per kind, each  $\perp$  when that kind is impossible.
21 /// `join`/`meet`/`widen` are pointwise – a cross-kind join keeps BOTH kinds
22 /// (`number | null`, `number | string`) instead of collapsing to  $\top$ , which is
23 /// what enables infinite-loop detection through nullable states without a
24 /// TypeScript hint.
```

Le join de `null` et d'un nombre garde maintenant les deux sortes, la composante numérique croît jusqu'au widening, et la boucle est trouvée :

```
NullCounter (2 hooks) ex1.tsx
  warn  infinite-loop [hook:0] (line 5:2) this effect keeps pushing state `n` (its deps do not provably gate it, so
    ↳ the effect can re-run every render) to new values on every run. Potential infinite render loop
    ↳ state `n` is written here [hook:1] (line 5:2)
    ↳ the abstract value of state `n` kept growing and was widened at iteration 3

! 1 warning(s) across 1 file(s).
```

Le commit `c32e1b0` touche 40 fichiers (+1 674 / -1 198) et *supprime* les trois rustines. C'est le premier exemple, dans l'histoire du projet, de ce que CLAUDE.md appellera plus tard « corriger la cause racine, jamais le symptôme » : la cause était l'abstraction, pas l'absence d'indice de type.

Remarque

Le niveau est `WARNING`, pas `ERROR`, alors que la boucle est certaine. Le bras « divergence de valeur » d'`infinite-loop` n'a pas de `must-primitive` qui prouve qu'une valeur diffère à chaque tour ; l'issue #144 est ouverte pour cela. La sévérité suit la preuve disponible, pas l'intuition (section 26.6).

26.4.2 ADR-017 : un couple FP/FN

Le premier banc sur le corpus « memos » produit cinq faux positifs de `always-unstable-deps`, baptisés F1 à F5. Leur forme commune est celle de `Provider` ci-dessous ; et sous ces FP se cachait un FN, celui d'`ObjChurn`.

Extrait 26.6 – /tmp/ch26/ex2.tsx

```
1 import { useState, useEffect } from "react";
2
3 export function ObjChurn() {
4   const [obj, setObj] = useState({ a: 1 });
5   useEffect(() => {
6     setObj({ ...obj, b: 2 });
7   }, [obj]);
8   return <div>{obj.a}</div>;
9 }
10
11 export function Provider() {
12   const [ctx, setCtx] = useState({ locale: "en" });
13   useEffect(() => {
14     console.log(ctx);
15   }, [ctx]);
16   return <button onClick={() => setCtx({ locale: "fr" })}>x</button>;
17 }
```

Sous ADR-002, le treillis de stabilité avait quatre points, $\perp \sqsubseteq \{\text{Stable}, \text{Unstable}\} \sqsubseteq \top$. Un objet d'état est une référence recrée par le setter : `Unstable`. Le dep `ctx` de `Provider` était donc « instable », d'où le FP ; et l'effet d'`ObjChurn`, qui dépend du même genre de valeur, ne pouvait pas être distingué d'un effet sain.

ADR-017 scinde `Unstable` en deux bornes (définition 10.3) : `PerRender`, borne *must* (« une référence fraîche à chaque rendu, garanti »), et `Versioned(S)`, borne *may* (« ne change qu'aux setters des slots de *S* »). Lire un état donne `Versioned({slot})` : le setter est stable et l'état ne change qu'au setter.

```
ObjChurn (2 hooks) ex2.tsx
  error infinite-loop [hook:0] (line 5:2) this effect recreates object state `obj` it depends on. Every run stores
    ↳ a fresh reference (`Object.is` always fails) and re-triggers itself: infinite render loop
    ↳ a fresh value is written to state `obj` here [hook:1] (line 6:4)
Provider (3 hooks) ex2.tsx ✓
```

```
! 1 error(s) across 1 file(s).
```

Les deux lignes sont les deux moitiés d'une même décision. `Provider` se tait parce que `ctx` est `Versioned`, pas `PerRender`. `ObjChurn` devient une `ERROR` parce que trois faits *must* se conjuguent : le dep *est* le slot, l'écriture a lieu sur tous les chemins de l'effet, et la valeur écrite est `PerRender`. ADR-017 §4 nomme ce couplage : « This coupling is a load-bearing soundness dependency ». Retirer le FP sans le bras churn aurait rendu le FN définitif.

Décision : ADR-017 — stabilité versionnée

Décision. Remplacer `Unstable` par `PerRender (must)` et `Versioned(S)/VersionedTop (may)`; convertir la lecture d'un état en `Versioned`. **Coût assumé.** `Stability` possède désormais un ensemble et perd `Copy` : « the compile errors force an exhaustive audit of every consumer — desirable for a semantics change ». **Refusé.** Un compteur d'événements pour faire diverger le store : un hack non standard, alors que la certitude vient de la structure des deps. Voir ADR-17.

26.4.3 ADR-018 et ADR-019 : le graphe de churn et les témoins

La même semaine, ADR-018 construit le graphe de churn (définition 15.1) : les cycles de deux effets ou plus, qui n'étaient que des `INFO`, deviennent des `ERROR` quand toutes leurs arêtes sont `must`. ADR-019 remplace les notes de diagnostic en texte libre par des chaînes de témoins typées (définition 17.2), avec un vocabulaire fermé de variantes `Step` et l'interdiction explicite d'une variante `Text(String)` : « a free-text escape hatch would erode the vocabulary back to prose ». ADR-019 ajoute aussi le `FileId` aux positions, parce qu'après l'inlining cross-file une note pouvait pointer dans le mauvais fichier ; le bug « disappears rather than being patched » (`docs/adr/ADR-019-witness-chain.md`, l. 50).

```
error infinite-loop [hook:2] (line 6:2) these effects form a state-update cycle (`a` → `b` → `a`) where each
  ↳ step stores a fresh reference that re-runs the next effect: infinite render loop
  ↳ a fresh value is written to state `b` here [hook:2] (line 6:20)
  ↳ cycle continues: this effect freshly stores state `a` [hook:3] (line 7:20)
```

(Extrait de la sortie sur `/tmp/ch26/ex3.tsx`, deux effets qui s'écrivent l'un l'autre un objet frais ; la même exécution émet une seconde `ERROR` pour l'autre effet et deux `WARNING derived-state`. Le mécanisme est au chapitre 15.)

Remarque

Le vocabulaire `Step` comptait neuf variantes dans ADR-019 ; il en compte quatorze au commit `e67b10a` (`Mutate`, `Capture`, `InitOnce`, `Forward`, `Rerender` se sont ajoutées). Il a crû par la voie sanctionnée, l'extension de l'énumération, sans jamais gagner de variante texte libre. Une décision qui interdit une échappatoire n'interdit pas la croissance.

26.5 La codification : CLAUDE.md et ADR-020

26.5.1 Des principes écrits après coup

Le 21 juillet 2026, le commit `6805689` ajoute `CLAUDE.md` à la racine. Dix-neuf ADR existent déjà. Le fichier ne crée donc pas une doctrine : il nomme celle qui était déjà à l'œuvre (ADR-013 §7 disait dès le 5 juin « `FPs` possible, `FNs` forbidden »), puis l'impose à tout ce qui suit. En voici le cœur, tel qu'il est au commit `e67b10a` :

Extrait 26.7 – `CLAUDE.md`, l. 5–28 — les trois principes et les deux invariants

```
5 ## Principes de conception – NON NÉGOCIABLES
6
7 1. **Pas de workarounds.** Corriger la cause racine, jamais le symptôme. Si un fix
8   contourne un problème au lieu de le résoudre (patch local, cas spécial, hack
9   rule-par-rule pour un défaut du moteur), c'est la mauvaise solution : corriger
10  le moteur / l'IR / la couche centrale à la place.
11
12 2. **Règle du paragraphe unique.** Si une décision demande plus d'un paragraphe
13   pour être justifiée, c'est une mauvaise idée. Chercher la solution plus simple
14   qui se justifie en une ou deux phrases.
15
```

```

16 3. **Modulaire et général d'abord.** Toujours préférer une solution générale et
17 réutilisable (mécanisme partagé, abstraction dans la couche commune) à une
18 solution ad hoc dupliquée dans chaque règle. Un problème qui touche plusieurs
19 règles se corrige une fois, au niveau central.
20
21 ## Invariants du projet
22
23 - **Soundness** : l'interprétation abstraite calcule un sur-ensemble des
24 comportements. Faux positifs tolérés, faux négatifs INTERDITS.
25 - **Niveaux de diagnostic** : Error (défaut certain dès que le code
26 s'exécute, preuve de toute la conclusion, pas d'un seul conjoint),
27 Warning (défaut possible, ou fait certain au coût incertain), Info (limite
28 de l'analyse ou motif qui a l'air voulu, derrière --info).

```

Chaque principe répond à une tentation précise.

- **Pas de workarounds.** La tentation est de corriger une règle pour un défaut du moteur (« si la valeur est $\top$, ne rien dire »). Le principe renvoie la correction à la couche où l'information se perd.
- **Paragraphe unique.** La tentation est une décision qui empile les cas particuliers. Le principe fait de la longueur de la justification un signal d'alarme.
- **Général d'abord.** La tentation est la copie par règle. Le principe exige qu'un problème qui touche plusieurs règles soit corrigé une fois, au niveau central.

Définition 26.2: Les deux invariants du projet

1. **Soundness** (définition 3.8) : l'analyse sur-approxime les comportements ; un FP est toléré, un FN interdit.
2. **Niveaux de diagnostic** : une ERROR est un défaut certain dès que le code s'exécute, prouvé pour *toute* la conclusion ; un WARNING est un défaut possible, ou un fait certain au coût incertain ; une INFO est une limite de l'analyse ou un motif qui a l'air voulu, affichée derrière `-info`.

26.5.2 L'histoire du fichier lui-même

CLAUDE.md a été modifié quatre fois, et chaque modification enregistre une leçon.

Commit	Date	Changement
6805689	07-21	création : trois principes, soundness, niveaux « Error (certain), Warning (incertain), Info (limitations) »
d0fbdb1	07-22	pointeur vers docs/tech-debt.md (le registre de la campagne de dette)
f72d113	07-23	le registre devient ADR-020 : « ne pas re-tenter les fixes qui y sont refusés »
835e0e4	08-27	le backlog devient le tracker GitHub ; convention des issues fermées wontfix
67b9824	09-24	niveaux réécrits : preuve de <i>toute</i> la conclusion ; « fait certain au coût incertain »

TABLE 26.2 – Historique de CLAUDE.md (`git log - CLAUDE.md`).

La dernière réécriture mérite un mot. Elle suit #142, corrigée par 307f6c7 (« the Error tier needs a proof of the whole claim ») : la règle *stale-closure* émettait une ERROR dont la preuve ne couvrait qu'un des deux conjoints de sa conclusion. Et elle prépare ADR-041, dont les deux règles restent au WARNING parce que « the extra renders are certain, their cost is not » (docs/adr/ADR-041-render-dependence.md, l. 90). La définition des niveaux n'a pas été inventée d'un coup ; elle a été resserrée deux fois par des cas réels.

26.5.3 La campagne de dette et ses onze refus

Entre le 21 et le 23 juillet, une campagne de nettoyage parcourt un registre de dette technique (docs/tech-debt.md : 18 workarounds, 36 constats d’architecture, un inventaire de code répété). Beaucoup de changements sont appliqués — le splice unifié, la macro `flat_lattice!`, le `KindMask`... Mais la leçon durable est ailleurs, et ADR-020 la formule ainsi :

« the recurring and more important finding was that many high-effort items rested on an **optimistic false premise**: what looked like accidental duplication was in fact deliberate, load-bearing variation » (docs/adr/ADR-020-tech-debt-cleanup-decisions.md, l. 13–16).

Définition 26.3: Non-changement

Un *non-changement* est une simplification apparente (déduplication, unification, suppression d’une passe) examinée puis refusée parce qu’elle coûterait de la soundness, de la précision mesurée ou de la simplicité. Il est écrit avec son argument, pour que la même simplification ne soit pas re-tentée. La charge de la preuve est inversée : « If any non-change above is revisited, the burden is to show the replacement is *exactly* soundness-equivalent, not merely cleaner » (docs/adr/ADR-020-tech-debt-cleanup-decisions.md, l. 155–156).

Le premier des onze montre la forme de l’argument.

Exemple 26.3: Le diamant && est une duplication porteuse

ADR-003 prévoyait d’abaisser `a && b` en un nœud d’expression `If(a, b, false)`. Le lowering a choisi un diamant de blocs : on évalue `a` dans un temporaire, on branche, et `b` n’est évalué que dans le bloc de droite (chapitre 7). Pourquoi ne pas revenir au nœud plat, plus simple ? Parce que

```
1 open && setOpen(false);
```

n’exécute `setOpen(false)` que si `open` est vrai. Un nœud plat oblige l’évaluateur soit à rejouer cette exécution conditionnelle, soit à perdre l’appel — un FN sur toute règle qui compte les écritures (docs/adr/ADR-020-tech-debt-cleanup-decisions.md, l. 60). Le coût accepté est la perte du raffinement relationnel à travers `&&`, reconstruit par une fonction dédiée.

Le tableau 26.3 donne les onze refus, chacun en une phrase ; les arguments complets sont dans ADR-20.

Décision : ADR-020 — cartographier avant de « corriger »

Décision. Clore le registre de dette par un ADR qui enregistre ce qui a été délibérément *laissé*, et supprimer le registre. **Règle transverse.** « **map before “fixing”**; **never introduce a false negative to remove a duplication** » (l. 151–152). **Conséquence.** CLAUDE.md renvoie à ADR-020 avec la consigne de ne pas re-tenter ses fixes refusés. Voir ADR-20.

Piège

Le principe 3 (« modulaire et général d’abord ») semble contredire ADR-020 : deux bras churn, une résolution par règle, n’est-ce pas de la duplication ? Non. Le principe condamne la copie d’un *même* traitement dans chaque règle ; ADR-020 défend des traitements *différents* qui se ressemblent. La généralité n’est pas l’unification aveugle : on unifie ce qui calcule le même fait, pas ce qui a la même forme.

#	On garde...	... parce que la simplification serait
1	le diamant <code>&&/ </code>	un FN : l'effet de bord conditionnel du court-circuit se perd
2	deux bras <code>churn</code> séparés	un FN sur <code>useEffect(() => setObj({...obj})), [obj])</code> : les bras couvrent des partitions disjointes
3	<code>may_written_slots</code> syntaxique	un FN : un bit « observé par le point fixe » sous-compte les écritures d'un chemin élagué
4	la projection <code>to_stability</code>	une perte : projection légitime, documentée, testée, dont des appelants ont besoin
5	pas de <code>BoundedPowerSet<T,N></code>	une abstraction prématurée : un seul vrai consommateur, <code>Stability</code> est plus riche
6	une résolution de composant par règle	un FN ou un FP : les résolutions divergent délibérément (rendu seul, par effet, contre un parent)
7	le chemin « position de retour » d' <code>exec_body_impl</code>	un FN : il faut la <i>valeur</i> de l'expression
8	la graine de tas d' <code>eval_in</code> par site d'appel	une régression : tas vide et tas convergé ne sont pas interchangeables
9	la passe <code>resolve_setter_aliases</code>	un FN sur les alias écrits par l'utilisateur, <code>const s1 = setX</code>
10	<code>TSType</code> hors du domaine	un FN : les types sont effacés et non imposés, <code>useState<number>()</code> peut valoir <code>undefined</code>
11	les noms temporaires par offset	du <code>churn</code> sans bénéfice : le préfixe <code>__arr_</code> est porteur

TABLE 26.3 – Les onze non-changements d'ADR-020, d'après `docs/adr/ADR-020-tech-debt-cleanup-decisions.md`, l. 60–143.

26.6 La polarité devient un type

26.6.1 Un FN corrigé deux fois

Jusqu'au 24 juillet, chaque règle choisissait elle-même son niveau. Rien n'empêchait une règle d'émettre une `ERROR` sur un fait `may`, ni de se taire sur un fait mal quantifié. Le programme suivant montre le second défaut.

Extrait 26.8 – `/tmp/ch26/ex8.tsx`

```

1 import { useState, useEffect } from "react";
2
3 export function C({ data }: { data: unknown }) {
4   const label = "fixed";
5   const [n, setN] = useState(0);
6   useEffect(() => {
7     setN(n + 1);
8   }, [label, data]);
9   return <div>{n}</div>;
10 }
11
12 export function D() {
13   const label = "fixed";
14   const [n, setN] = useState(0);
15   useEffect(() => {
16     setN(n + 1);
17   }, [label]);
18   return <div>{n}</div>;
19 }

```

Rappel React : sémantique OU des deps

React relance un effet si *un* de ses deps a changé au sens d'Object.is. Un tableau de deps ne bloque définitivement l'effet que si *tous* ses deps restent identiques.

La garde d'infinite-loop doit sauter un effet que ses deps bloquent. Sa première version testait « la valeur est-elle instable ? » ; or `is_stable()` et `is_unstable()` ne sont pas complémentaires, $\top$ et `Versioned` tombent entre les deux, et un effet dont un dep était $\top$ était sauté. La première correction (a9b91f7) introduisit `all_deps_may_change`, mais avec le mauvais quantificateur : elle sautait l'effet dès qu'un dep était prouvé stable. Dans C, `label` est stable et `data` (une prop de racine) est $\top$: l'effet était sauté, FN. Le durcissement du lendemain (eb5cb93) pose le bon quantificateur :

Extrait 26.9 – src/rules/helpers/mod.rs, l. 211–235 — la garde quantifiée $\forall$ -stable

```

211 /// `true` when every dep in `deps` is provably `Stable` in the render-exit
212 /// env – the only situation where a deps array genuinely gates an effect for
213 /// good: React re-runs an effect when any dep changed (OR semantics), so a
214 /// single dep that may change ( $\top$ /Versioned/PerRender) keeps the effect
215 /// live no matter how stable its neighbours are. Empty `deps` returns `true`
216 /// ([] is mount-only – gated by definition; infinite-loop handles it
217 /// upstream anyway).
218 ///
219 /// ADR-021 §5 + quantifier fix: the first cut of the  $\top$ -fix
220 /// (all_deps_may_change) still used the wrong quantifier – it skipped when
221 /// one dep was provably stable, silently dropping the mixed-deps case
222 /// ([stableConst, topProp]), the same FN family one stable dep away. The
223 /// sound gate quantifies  $\forall$ -stable, keyed on [query::stability_verdict_of]
224 /// ( $\top$  is a returned variant folded to the may side).
225 pub(in crate::rules) fn all_deps_provably_stable(
226     deps: &[Expr],
227     result: &AnalysisResult<StateValue>,
228 ) -> bool {
229     let exit_env = result.exit_env();
230     let mut eval = result.evaluator();
231     deps.iter().all(|dep| {
232         let val = eval.at(&exit_env, dep);
233         query::stability_verdict_of(&val).is_stable()
234     })
235 }
```

```

C (2 hooks) ex8.tsx
warn  infinite-loop [hook:0] (line 6:2) this effect keeps pushing state `n` (its deps do not provably gate it, so
  ↳ the effect can re-run every render) to new values on every run. Potential infinite render loop
    ↳ state `n` is written here [hook:1] (line 6:2)
    ↳ the abstract value of state `n` kept growing and was widened at iteration 3
warn  missing-deps [hook:1] var:n (line 6:2) `n` is used in this effect but not in its deps array, and it is
  ↳ recreated on every render
    ↳ `n` is read here [hook:1] (line 6:2)
D (2 hooks) ex8.tsx
warn  missing-deps [hook:1] var:n (line 15:2) `n` is used in this effect but not in its deps array, and it is
  ↳ recreated on every render
    ↳ `n` is read here [hook:1] (line 15:2)
```

C est examiné et signalé. D, dont tous les deps sont prouvés stables, ne peut tourner qu'une fois après le montage : la garde tue légitimement l'`infinite-loop`, et il ne reste que le `missing-deps`, correct.

Cet épisode donne sa forme opérationnelle à l'invariant de soundness.

Invariant 26.1: La direction d'un changement

Tout changement déclare la direction dans laquelle il déplace les findings. Un ajout (une ligne de relation de plus, une arête de plus, un $\top$ conservé) va du côté toléré et n'exige qu'un argument de précision. Une suppression (un kill, une garde, un narrowing, une déduplication) peut créer un FN : elle n'est admise que sur une preuve must. Sauter un effet est une suppression ; elle exige « $\forall$ dep, stable prouvé ».

On retrouve cette phrase, sous des formes variées, dans presque tous les ADR à partir de 025 : « Row multiplication cannot lose a match » (ADR-028), « Every part of this change moves suppression in one direction: less of it » (ADR-033), « The unsound direction is the safe one » (ADR-036), « The two behavioural changes go in opposite, argued directions » (ADR-042).

26.6.2 Le sceau : Certified

ADR-021 fait de la polarité un type (définition 17.1). Une `ERROR` ne peut se construire qu'à partir d'un jeton `Certified`, et un `Certified` ne peut être frappé que dans le module des requêtes :

Extrait 26.10 – `src/rules/api/query.rs`, l. 74–93 — le jeton de preuve et son constructeur privé

```

74 /// A certified MUST fact: the evidence plus its provenance.
75 ///
76 /// The constructor (`Certified::mint`) is private to this module. Rule code in
77 /// sibling modules can only [`Certified::evidence`]/[`Certified::provenance`] –
78 /// it can never forge a token. This is the enforcement: an `Error` is reachable
79 /// only from a `Certified`, and a `Certified` only from a must-primitive here.
80 #[derive(Debug, Clone, PartialEq)]
81 pub struct Certified<E> {
82     evidence: E,
83     provenance: Provenance,
84 }
85
86 impl<E> Certified<E> {
87     /// Mint a token. Private to the query module – the whole point.
88     fn mint(evidence: E, provenance: Provenance) -> Self {
89         Certified {
90             evidence,
91             provenance,
92         }
93     }

```

La première version n'était pas scellée, et l'ADR le raconte dans un encadré « Hardening round » : « Rust privacy is downward », si bien que les constructeurs « privés » de `rules/mod.rs` restaient appelables depuis chaque sous-module de règle, et un champ `pub severity` permettait de forger une `ERROR` (`docs/adr/ADR-021-typed-query-surface.md`, l. 28–36). `Diagnostic` a été déplacé dans le module feuille `src/rules/api/diagnostic.rs`, et les trois sondes de contrefaçon sont devenues des erreurs de compilation. Le même encadré note que deux primitives étaient des « token vending machines » : elles frappaient un `Certified` pour quiconque passait `true`.

Décision : ADR-021 — sévérité certifiée par le moteur

Décision. `must/may/T` deviennent des types ; `Diagnostic::error` est le seul constructeur d'`ERROR` et exige un `Certified` ; les quatorze règles migrent en une fois. **Refusé.** Une fenêtre de transition où l'ancienne API resterait disponible : elle laisserait ouvert le vecteur de FN. Voir ADR-21.

Piège

Un Certified prouve *un* fait. Si la conclusion d'un diagnostic est une conjonction, la preuve d'un conjoint ne suffit pas. C'est exactement #142 (stale-closure, corrigée) et #143 (l'exécuteur des packs Tier-A, ouverte, soundness-bug) : une primitive `must_*` certifie un conjoint pendant qu'une garde `may` voisine reste non prouvée. Le type garantit qu'aucune `ERROR` ne naît d'un fait `may` isolé ; il ne garantit pas que le fait certifié soit *toute* la conclusion. D'où la réécriture de `CLAUDE.md` du 24 septembre.

26.6.3 Les packs, et la même idée appliquée aux règles des utilisateurs

ADR-022 (26 juillet) ouvre l'analyseur aux règles d'équipe : des packs JSON déclaratifs (chapitre 22), un fichier de configuration et une distribution WASM. La sévérité effective d'un finding de pack est `pin` $\sqcap$ `polarity` : le niveau demandé par l'auteur, borné par la polarité du verdict qui a produit ce finding. Le refus notable est celui d'un rejet statique des packs dont le `pin` dépasse la polarité : il aurait interdit la stratification `finding` par `finding`. Le lendemain, ADR-023 remplace le Starlark prévu par ADR-022 §7 : Starlark avait été choisi pour une propriété — l'absence d'itération non bornée, qui force toute décision `must/may` à passer par une primitive vérifiée — et cette propriété n'est pas propre au langage ; ADR-023 la garde et adopte une compilation JS/Ts vers le JSON Tier-A. Il refuse d'abord le quantificateur $\forall$ — refus levé plus tard, quand l'IR a su dire qu'un littéral de tableau est exact.

Remarque

Le catalogue de référence Tier-A mesure l'expressivité des packs : combien de règles d'un catalogue de 21, puis 22, le vocabulaire sait écrire. La courbe va de 3/21 à 5/21 fin août, puis de 5/21 à 8/22 (aa0dbf3, 1^{er} septembre) et, commit par commit, jusqu'à 21/22 (0c45de8, 2 septembre). La 22^e règle est exclue par décision (#101, fermée wontfix). Le cliquet est la constante `EXPRESSIBLE_NOW` de `tests/catalogue.rs`.

26.7 Les relations : un fait, un endroit

26.7.1 Le symptôme : deux marches qui divergent

Exemple 26.4: Une continuation `.then` dans un effet sans deps

Extrait 26.11 — `/tmp/ch26/ex7.tsx`

```
1 import { useState, useEffect } from "react";
2
3 function load() { return fetch("/api"); }
4
5 export function Poller() {
6   const [data, setData] = useState({ v: 0 });
7   useEffect(() => {
8     load().then(() => setData({ v: 1 }));
9   });
10  return <div>{data.v}</div>;
11 }
```

L'effet n'a pas de tableau de deps : il tourne après chaque rendu. Sa continuation stocke un objet frais, qui provoque un rendu, qui relance l'effet.

ADR-018 avait listé ce cas dans ses limitations : « Auto-run nested callbacks (`.then(() => set(fresh))`) in **no-deps** effects: no self-edge ». En septembre, la relation d'écriture `slot_writers`

savait déjà que l'écriture existe et qu'elle est différée. Mais le graphe de churn était construit par sa propre marche, dans `rules/helpers/`, qui ne voyait pas les écritures imbriquées. Deux calculs du même fait avaient divergé : c'est l'issue #26. Au commit `e67b10a` :

```

Poller (2 hooks) ex7.tsx
warn  infinite-loop [hook:1] (line 7:2) this effect has no dependency array and may store a fresh reference into
    ↪ state `data`, so it re-runs after every render and can re-trigger itself: possible infinite render loop
    → a fresh value is written to state `data` here [hook:1] (line 8:4)

```

La ligne d'écriture est `Deferred` ; ADR-042 §4 en fait une auto-arête `may`, donc un `WARNING` (chapitre 15).

26.7.2 La vague de septembre

Du 1^{er} au 2 septembre, treize ADR (027 à 039) promeuvent une à une dans le moteur les marches que les règles avaient accumulées depuis juin (définition 14.1). La figure 26.2 en donne la carte.

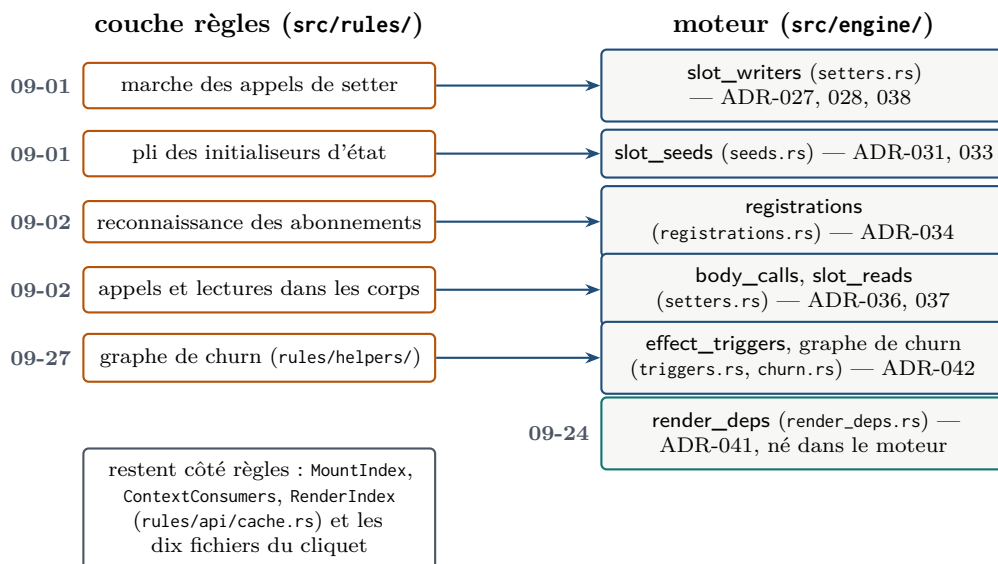


FIGURE 26.2 – La migration des faits de la couche règles vers le moteur (dates des commits d'ADR). Chaque flèche remplace une marche propre à une ou plusieurs règles par une relation calculée une fois, à la fin de la convergence, avec des colonnes polarisées (`docs/relations.md`).

Trois décisions de cette vague illustrent chacune un aspect de la doctrine.

Le contrat, pas le nom (ADR-034). La colonne phase d'une ligne d'écriture est un verdict `may` où $\top$ satisfait toute requête. La rétrécir de `Unknown` à `Handler` est la seule direction qui puisse *perdre* un finding. ADR-034 ne l'admet que là où le moment d'exécution est un contrat :

Extrait 26.12 – `src/engine/registrations.rs`, l. 40–57 — les trois moments d'exécution d'un callback enregistré

```

40 /// When a registered callback can run, relative to the React phases.
41 ///
42 /// This is the phase *summary* ADR-027 §2 promised and never shipped: a
43 /// registration argument used to fall to  $\top$  in the slot-writer walk.
44 #[derive(Debug, Clone, Copy, PartialEq, Eq)]
45 pub enum Timing {
46     /// A timer, a microtask, a promise continuation: the callback runs on a

```

```

47   /// later turn of the event loop, provably outside every React phase.
48   Deferred,
49   /// An external event. The DOM has no synchronous dispatch from
50   /// `addEventListener`, so the callback provably does not run during the
51   /// registering call.
52   Handler,
53   /// The registrar may invoke the callback synchronously – an RxJS
54   /// `BehaviorSubject` emits to a new subscriber on the spot – so nothing
55   /// about the timing is proven and the walk keeps  $\top$ .
56   Unknown,
57 }

```

`addEventListener` a un contrat (le DOM ne rappelle jamais le listener pendant l'enregistrement); `subscribe` n'en a pas (un `BehaviorSubject` RxJS émet sur-le-champ). Le refus de classer `subscribe` en `Handler` est donc une application directe de l'invariant 26.1.

Une écriture est une écriture où qu'elle soit écrite (ADR-038). Avant 7607ac9, la marche d'écriture ne visitait que certaines positions syntaxiques. Un setter appelé en argument n'avait « no row in the writer relation at all » (ADR-38, l. 25 du fichier) :

Extrait 26.13 – `/tmp/ch26/ex5.tsx`, composant `Wrapped` (extrait)

```

1  function wrap(x) { return x; }
2
3  export function Wrapped() {
4    const [n, setN] = useState(0);
5    wrap(setN(1));
6    return <div>{n}</div>;
7  }

```

```

Wrapped (1 hooks) ex5.tsx
error  setter-in-render [hook:0] (line 13:2) setter `setN` called directly in the render body, move this call into
  ↳ a useEffect or an event handler
  ↳ `setN` is a state setter, so calling it writes state (line 13:2)

```

La correction n'est pas une règle de plus : c'est la suppression d'une porte de traversée dans la marche. Sur le corpus, elle ajoute 27 findings et en retire 9 (des doublons de composants) — une correction de soundness peut réduire le nombre de findings.

Un champ qui manque se remplit une fois (ADR-039). Quand une liaison synthétique n'avait pas de position, chaque règle qui s'en apercevait bricolait un repli. ADR-039 remplit la position « once, for every rule, rather than in each rule that happens to notice ». C'est le principe 3 appliqué à une métadonnée.

26.7.3 ADR-042 : la frontière et son cliquet

ADR-042 (26 septembre) tire la conclusion de la vague et la justifie en une phrase :

« A rule reads relation rows and calls must-primitives. It walks no CFG and no expression. [...] One sentence justifies it: a fact computed in two places is two facts that drift, and #26 is the drift. » (docs/adr/ADR-042-relations-are-engine-products.md, l. 64-67)

La frontière n'est pas laissée à la bonne volonté. Elle est tenue par un test :

Extrait 26.14 – `tests/layer_boundary.rs`, l. 19–41 — la liste qui ne peut que rétrécir, et les marqueurs de syntaxe

```

19 /// The rule files still walking syntax, as of ADR-042 slice 5. Shrink only.
20 const ALLOWED: &[&str] = &[
21     "src/rules/api/query.rs",
22     "src/rules/api/witness.rs",
23     "src/rules/helpers/jsx.rs",
24     "src/rules/helpers/mod.rs",
25     "src/rules/helpers/mount.rs",
26     "src/rules/helpers/purity.rs",
27     "src/rules/impls/redundant_set_state.rs",
28     "src/rules/impls/setter_in_render.rs",
29     "src/rules/impls/state_mutation.rs",
30     "src/rules/impls/unnecessary_rerender.rs",
31 ];
32
33 /// What counts as touching syntax: iterating a CFG's blocks, matching
34 /// statements or terminators, or descending an expression tree.
35 const MARKERS: &[&str] = &[
36     "cfg.blocks",
37     ".blocks.values()",
38     "Stmt::",
39     "for_each_child",
40     "Terminator::",
41 ];

```

Définition 26.4: Cliquet

Un *cliquet* (*ratchet*) est un test qui fige une liste d'exceptions à une décision et échoue dans les deux sens : si un fichier hors de la liste enfreint la décision (la liste ne peut pas grossir), et si un fichier de la liste ne l'enfreint plus (la liste doit rétrécir). `tests/layer_boundary.rs` en porte les deux tests, `no_rule_file_outside_the_list_walks_syntax` et `the_list_only_shrinks`.

Chaque entrée de la liste est, selon l'ADR, « a promotion still to do, not an exception ». Le cliquet transforme une décision d'architecture en contrainte que la CI fait respecter.

Piège

Le cliquet est textuel : il cherche les chaînes `cfg.blocks`, `Stmt::`, `Terminator::`... dans la partie non-test de chaque fichier. Un marqueur dans un commentaire compte, et un parcours écrit autrement passe. C'est un cliquet de discipline, pas une preuve. Par ailleurs, la liste écrite dans ADR-042 §1 nomme `impls/conditional_hook.rs` et pas `helpers/mod.rs`, alors que le test fait l'inverse : quand l'ADR et le cliquet divergent, c'est le cliquet qui dit l'état du code.

Décision : ADR-042 — les relations sont des produits du moteur

Décision. Le graphe de churn quitte `rules/helpers/`; la ligne d'écriture gagne les colonnes `owner`, `block`, `written`; une relation `effect_triggers` naît; `ProgramRelations` remplace `ProgramCache` côté moteur; la frontière est tenue par un cliquet. **Refusés.** Unifier `effect_triggers` et `render_deps` (identité n'est pas dépendance : le `must-rerun` a besoin de l'identité); et, dans l'amendement du 27 septembre sur les sites de convergence, un plus grand point fixe sur les sites, qui « would read mutual revival as convergence ». Voir ADR-42.

26.8 Identité et attribution

Quatre ADR, étalés sur deux mois, corrigent le même défaut sous quatre formes : une chaîne d'affichage servait de clé.

Exemple 26.5: Deux composants du même nom

Le répertoire `/tmp/ch26/ex10/` contient deux fichiers `a/Widget.tsx` et `b/Widget.tsx`, qui définissent chacun un `Widget`. Celui de `b/` appelle dans un effet la prop `onChange` avec un objet frais; `App.tsx` importe `Widget` depuis `./b/Widget` et lui passe `setQ`.

```
Widget@ex10/b/Widget.tsx (1 hooks) ex10/b/Widget.tsx
warn cross-component-infinite-loop [hook:0] (line 4:2) this effect calls `onChange`, a state setter of
  ↳ parent `App` (its deps do not provably gate it, so the effect can re-run every render). Parent re-renders
    ↳ → child re-renders → effect fires again: infinite loop
1 clean component(s) hidden, rerun with --show-clean

! 1 warning(s) across 3 file(s).
```

Le nom `Widget@ex10/b/Widget.tsx` n'apparaît que parce qu'un second fichier définit `Widget` : c'est un *rendu*, pas une identité. La résolution suit l'import d'`App.tsx` vers `b/`.

Avant ADR-040, l'identité d'un composant était son nom d'affichage (choix d'ADR-038 §5), et la résolution d'un callee JSX prenait le premier candidat trié — ici `a/Widget.tsx`, qui n'est pas celui que le programme rend. ADR-040 le dit sans euphémisme : « The old first-match was not non-deterministic, it was **wrong** » (docs/adr/ADR-040-component-identity-is-an-interned-id.md, l. 100–101). Une référence qui ne se résout à aucun fichier unique devient *Ambiguous*, et le parent porte une INFO *analysis-limit* plutôt qu'un inlining deviné. L'identité, elle, devient un entier interné :

Extrait 26.15 – `src/ir/component_id.rs`, l. 23–29 — l'identité d'un composant

```
23 /// A component's identity, interned in a [`ComponentTable`].
24 ///
25 /// `Copy` and 4 bytes, so it sits inside a `BTreeSet` label or a store key
26 /// without the allocation a name costs, and two ids compare in one
27 /// instruction.
28 #[derive(Debug, Clone, Copy, PartialEq, Eq, Hash, PartialOrd, Ord)]
29 pub struct ComponentId(u32);
```

Le même motif s'était présenté trois fois auparavant.

- **ADR-019** : `SourceRange` n'avait pas de fichier ; après l'inlining cross-file, une note pouvait pointer dans le mauvais fichier. On ajoute un `FileId`.
- **ADR-024** : un finding né dans un hook inline est attribué au fichier d'*origine*, et les findings de consommateurs différents ne sont pas fusionnés, parce que leurs faits dépendent des arguments de chacun ; le regroupement (« [in 2 components] ») n'est qu'un affichage (#129).
- **ADR-030** : le nom d'un slot écrit par un composant étranger se résout dans le composant *propriétaire*, jamais dans celui qui lit.

Piège

Une chaîne d'affichage ne doit jamais servir de clé : elle dépend du contenu du programme (`Widget` devient `Widget@...` dès qu'un autre fichier le définit). Le coût de la correction est à la mesure de la faute : le commit 806d114 touche 107 fichiers (+2 959 / −1 454). Le digest du corpus reste identique, et la résolution passe de 1 317 à 1 348 composants (29 perdus, 60 gagnés) : ce qui disparaît était inline à tort.

26.9 Les cascades de rendu : une seconde analyse plutôt qu'un champ

La dernière grande décision avant le snapshot concerne les rendus inutiles : un état tenu trop haut, qui re-rend un sous-arbre qui ne le lit pas. Pour les détecter, il faut savoir de quoi dépend la sortie de chaque composant. L'idée la plus directe serait d'ajouter à `StateValue` un champ « dépend de ». ADR-041 la refuse :

« It is **not** a field of `StateValue`. As a field it would change every existing value comparison [...]
] Dependence is also a different fact from value. » (docs/adr/ADR-041-render-dependence.md, l. 44–47)

Décision : ADR-041 — la dépendance de rendu

Décision. Une analyse en avant séparée, sur un composant déjà convergé, composée le long de l'arbre d'éléments d'origine prouvée (définition 16.3) ; deux règles, `state-lifted-too-high` et `wasted-subtree-render` ; des options typées sur les règles natives (amende ADR-022 §4).
Niveau. WARNING au plus : « the extra renders are certain, their cost is not ». **Refusé.** La dépendance comme champ du domaine des valeurs. Voir ADR-41 et chapitre 21.

La décision tient en une phrase, comme l'exige le principe 2 : dépendance et valeur sont deux faits, donc deux analyses. Sa mesure aussi : la baseline du corpus passe de 1 346 à 1 500 localisations avec l'arrivée des deux règles.

26.10 La méthode

26.10.1 Le cycle d'une décision

Les messages de commit, les sections « Consequences » des ADR et le journal de précision laissent voir un processus stable, résumé par la figure 26.3.

Exemple 26.6: Un raccourci mesuré puis refusé (ADR-025)

Jusqu'au 29 juillet, un corps qui « tombe » à la fin était scellé `Unreachable`, comme un `throw`. Le splice concluait que le contrôle ne revenait jamais, et 198 composants du corpus étaient coupés de leur propre `Return` (le commentaire de `src/lowering/cfg_builder.rs`, l. 222–230 le raconte). ADR-025 scelle la chute en `Return(undefined)`. Un raccourci plus large était tentant : relier *tout* `Unreachable` au bloc de jonction. Il a été mesuré, puis refusé, sur ce genre de code :

Extrait 26.16 – `/tmp/ch26/ex4.tsx` (extrait)

```
1 function useCart() {
2   const ctx = useContext(CartContext);
3   if (ctx === undefined) throw new Error("useCart outside provider");
4   const [items] = useState(ctx.items);
5   return items;
6 }
```

Relier le `throw` à la jonction invente un chemin vers la sortie qui contourne `useState` : `conditional-hook` en `ERROR` sur du code conforme. Au commit `e67b10a`, `Cart` est propre (✓ 1 file(s) no issues found.) ; avec `-info`, seule apparaît l'INFO « hook `useContext` was not found in the registry », qui relève de la limite #28. Mesure de l'ADR : composants sectionnés 208 → 3 ; 12 findings révélés, aucun perdu.

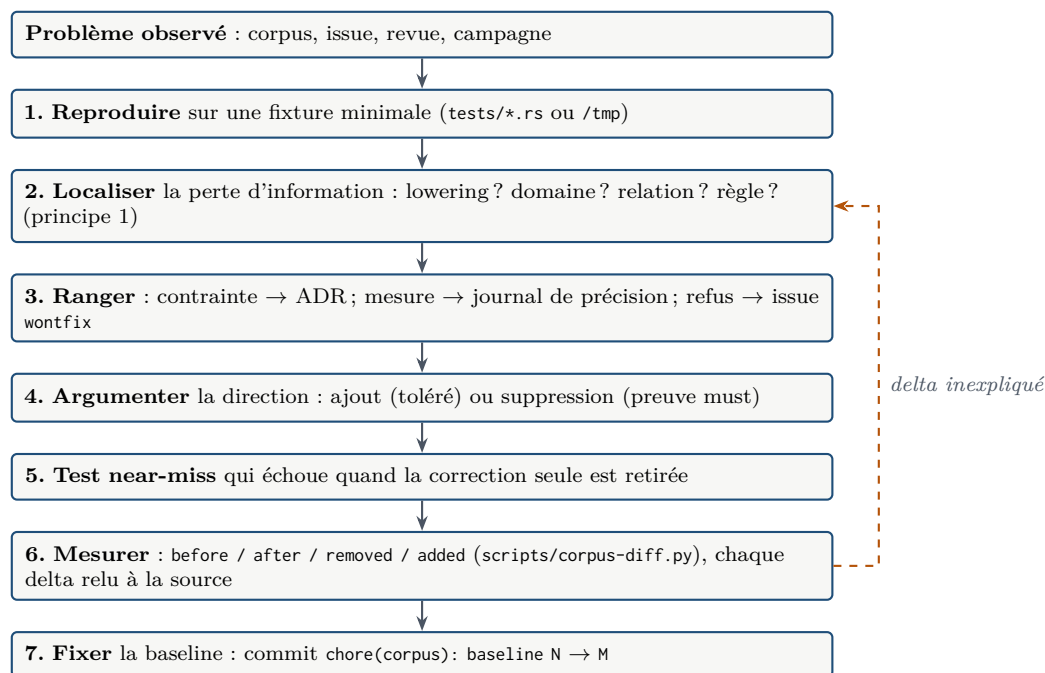


FIGURE 26.3 – Le cycle d’une décision, reconstitué à partir de l’historique (notes du dossier 15, §4.1). Un delta de corpus qu’on ne sait pas attribuer renvoie à la localisation.

26.10.2 La mesure comme juge, et la mesure corrigée

Depuis ADR-016, chaque décision est confrontée au corpus, y compris les refus (ADR-025, ADR-040 « dropped on measurement »). La métrique, la procédure et l’erreur de comptage du 3 septembre qui l’a fait naître sont au chapitre 25. Deux traits concernent l’histoire.

- **La mesure peut être fausse, et elle se corrige là où elle est écrite.** ADR-031 porte une mention « CORRECTED » : sa comparaison reposait sur un run par côté alors que l’analyse n’était pas encore déterministe (#120). Le journal de précision porte une « Table correction » où un signe s’était inversé.
- **Une correction de précision peut révéler un défaut de soundness.** La migration d’ADR-031 a fait apparaître le non-déterminisme (#120) et une suppression fondée sur une coïncidence, que corrige ADR-033.

26.10.3 Les ADR rétrogradés et les numéros réutilisés

Le 2 septembre, en pleine campagne de précision, sept ADR (040 à 046) sont écrits pour des corrections de précision. Le lendemain, le commit 87f87b5 (« ADRs are for decisions, precision fixes get a log ») les supprime et verse leur contenu dans docs/precision-log.md. C’est la naissance de la frontière de la définition 26.1.

Piège

Les numéros 040, 041 et 042 ont ensuite été *réattribués* : identité des composants (5 septembre), dépendance de rendu (24 septembre), relations du moteur (26 septembre). Un vieux message de commit qui cite « ADR-042 » du début septembre peut désigner une correction de précision disparue. Deux autres pièges de lecture de git log : les ADR 001–013 ont été traduits et plusieurs portent des lignes updated successives, si bien que leur champ Date n’est pas la date du texte actuel ; et certaines issues restent OPEN alors qu’un commit

les nomme comme corrigées (elles restent ouvertes sur un résidu). Ne pas conclure de l'état d'une issue sans lire son dernier commentaire.

26.10.4 Les refus citables

Depuis le 27 août (835e0e4), le backlog vit sur le tracker GitHub et les limites tranchées « on ne corrige pas » deviennent des issues fermées wontfix, créées puis fermées aussitôt pour que le raisonnement soit citable. Le tableau 26.4 les liste.

Issue	Titre (tracker) et raison résumée
#40	« FP by decision — whole-object read via guard/nullish is flagged » : distinguer usage de valeur et test d'existence demanderait de suivre le type de consommation
#42	« FP by decision — stale-closure emitter-name heuristic » : un appel on/addListener est traité comme un enregistrement durable, plafond WARNING
#51	« By design — node_modules [...] are never lowered » : périmètre ; les résumés de hooks sont le point d'extension
#63	« Out of scope — dynamic components » : pas de design pour résoudre une référence de composant à travers un join
#65	« Out of scope — anonymous default exports get a generic name » : reste cosmétique
#101	« Catalogue — nullable-return-unguarded is excluded by design » : ancre syntaxique et refus de TSType (ADR-020 n° 10)

TABLE 26.4 – Les six issues fermées wontfix (gh issue list -state closed -label wontfix).

26.10.5 Le graphe des ADR

Un ADR n'est presque jamais effacé ; il est remplacé en tout ou partie, amendé, ou ses conditions sont levées. La figure 26.4 dessine ces relations. Un seul ADR a été entièrement remplacé (008 par 015) ; les autres remplacements portent sur une section.

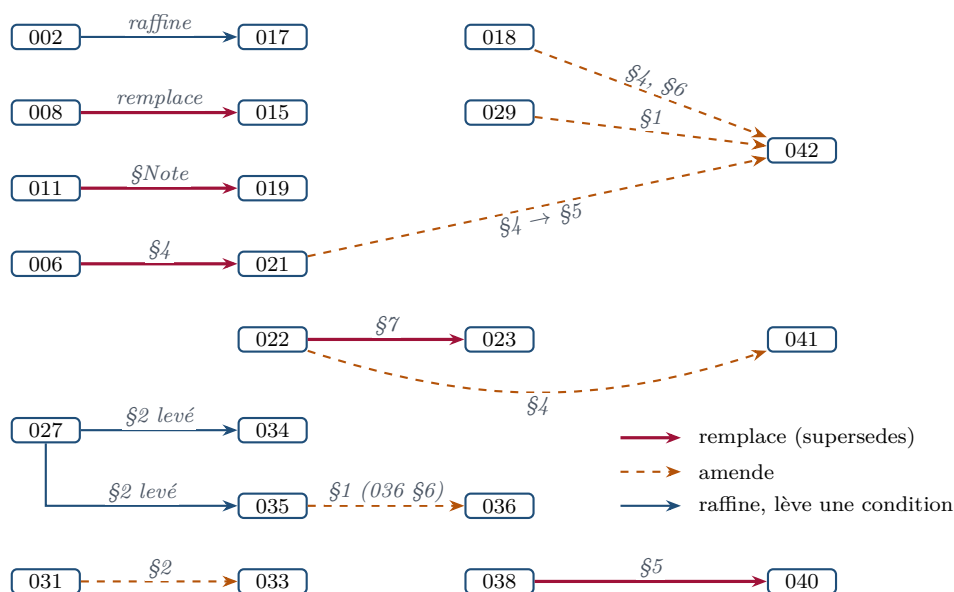


FIGURE 26.4 – Remplacements et amendements entre ADR (notes du dossier 15, §5.3 ; en-têtes Supersedes/Amends des ADR). Une flèche va de l'ADR ancien vers celui qui le modifie ; l'étiquette nomme la section touchée.

26.10.6 Quand l'ADR et le code divergent

La double citation ADR $\leftrightarrow$ code s'est désynchronisée par endroits. Le lecteur doit connaître ces écarts, constatés au commit e67b10a, pour ne pas prendre un ADR pour une description du code (tableau 26.5 ; ils sont repris, avec leur piste de correction, au chapitre 27).

ADR	Annonce	État au commit e67b10a
001	docs/semantics.md, citations React-tRace, tests contre l'interpréteur OCaml	aucun des trois
003	docs/ir.md ; a && b $\rightarrow$ nœud If	ir.md supprimé ; diamant de blocs (ADR-020 n° 1)
005	src/registry/user_config.rs, reactant.toml	absents
016	schéma JSON v1	sortie en "version": 2 (ADR-024 §1)
019	neuf variantes de Step	quatorze (croissance par la voie sanctionnée)
039	span sur Return : « a hundred construction sites »	le commentaire de src/ir/cfg.rs, l. 25 dit « a 40-site IR change »
042 §1	liste du cliquet avec impls/conditional_hook.rs	le test liste helpers/mod.rs à la place
042	« rules/api/cache.rs is gone »	présent : ProgramCache enveloppe ProgramRelations et trois structures

TABLE 26.5 – Écarts entre ADR et code (notes du dossier 15, §8.3, revérifiés par `grep` pour 042).

26.11 Les fils rouges

Relue d'un bout à l'autre, l'histoire du projet se ramène à huit fils, qui reviennent d'ADR en ADR.

1. **Soundness : la direction d'abord.** FP tolérés, FN interdits, sous la forme opérationnelle de l'invariant 26.1. Les exceptions historiques (`TriggerClass::Unknown`) sont antérieures à la formulation et sont suivies par des issues.
2. **Les niveaux comme typage.** De la convention (trois paliers, 4 juin) à la typestate (Certified, ADR-021), à l'exécuteur de packs (pin $\sqcap$ polarity, ADR-022), puis au plafond structurel : une sorte qu'aucune `must_*` ne lie ne peut pas atteindre l'ERROR. Point ouvert : la conjonction (#143).
3. **Produits du moteur plutôt que marches de règles.** ADR-006 sépare moteur et règles ; de juin à août les règles réaccumulent des marches ; de 027 à 042 elles sont promues une à une (figure 26.2).
4. **Généralité, pas unification aveugle.** « one central relation, never a second bespoke one » (ADR-027 §4), « one column, not two » (ADR-028), une table unique des registrars (ADR-034) — et, en contrepois, les duplications porteuses d'ADR-020.
5. **Le paragraphe unique.** Les ADR postérieurs à juillet portent des justifications explicitement « en une phrase » (ADR-022 §1, ADR-030 §2, ADR-042 §1). Les ADR longs (021, 022, 023, 042, près de 280 lignes chacun) ne contredisent pas la règle en esprit : chaque décision y a sa phrase, la longueur vient du nombre de décisions.
6. **La mesure plutôt que l'argument,** et la mesure corrigée quand elle est fausse.
7. **Identité n'est pas rendu.** `FileId`, `ComponentId`, l'identité d'un finding par sa position d'ancre, le nom d'un slot résolu chez son propriétaire.
8. **Déterminisme.** `BTreeMap` pour les blocs de CFG, tri avant récursion dans la chasse aux liaisons (ADR-033), digest de la baseline : un résultat qui change d'un run à l'autre ne peut pas juger une décision.

Remarque

Ces fils sont aussi une grille de revue. Devant un changement proposé, on peut poser huit questions : dans quelle direction déplace-t-il les findings ? son niveau est-il porté par une preuve ? calcule-t-il un fait déjà calculé ailleurs ? unifie-t-il deux traitements différents ? se justifie-t-il en une phrase ? a-t-il été mesuré ? utilise-t-il un affichage comme clé ? est-il déterministe ?

Résumé du chapitre

- Le projet range chaque décision dans un lieu : ADR (contrainte), journal de précision (mesure), issue fermée wontfix (refus citable), cliquet (décision exécutoire). La frontière ADR/journal date du 3 septembre (87f87b5).
- Onze phases en cinq mois (figure 26.1) : prototype jeté, pipeline en un jour, domaines, corpus, codification, surface typée, distribution, relations, précision et identité, doctrine de sévérité et cascades.
- CLAUDE.md a été écrit après dix-neuf ADR : il nomme une doctrine existante (pas de workaround, paragraphe unique, général d'abord ; soundness ; niveaux) et l'impose ensuite. Sa définition des niveaux a été resserrée par #142.
- ADR-020 enseigne à cartographier avant de corriger : onze simplifications apparentes sont refusées parce qu'elles coûtent un FN.
- L'invariant opérationnel : tout changement déclare sa direction ; une suppression exige une preuve must. Certified et Diagnostic::error en font un type ; le quantificateur $\forall$ -stable en est un exemple.
- « A fact computed in two places is two facts that drift » : les marches des règles sont promues en relations du moteur, et tests/layer_boundary.rs empêche le retour en arrière.
- Une chaîne d'affichage n'est jamais une clé (FileId, ComponentId, attribution d'origine).
- Types et fichiers rencontrés : TriggerClass, StateValue, Stability, Certified, Timing, ComponentId ; CLAUDE.md, docs/adr/, docs/precision-log.md, tests/layer_boundary.rs, tests/catalogue.rs.
- Le chapitre suivant (chapitre 27) rassemble ce qui reste ouvert : les limites assumées, les défauts observés au commit e67b10a et les pistes de correction à la racine.

26.12 Exercices

Exercice 26.1: Onze phrases

Pour trois des non-changements d'ADR-020 au choix (tableau 26.3), écrire un programme React de quelques lignes qui deviendrait un FN si l'on appliquait la simplification refusée.

Solution

Pour le n° 2 : `useEffect(() => setObj({...obj}), [obj])`, le cas même que cite l'ADR, disparaîtrait avec le bras self-churn. Pour le n° 9 : `const s1 = setX; s1(1)` ; dans le corps de rendu est la forme que la passe d'alias doit servir. Pour le n° 10 : `const [v, setV] = useState<number[]>(); useEffect(() => setV([...(v ?? [])]), [v])` ; — l'annotation laisserait croire à une valeur typée, alors que la valeur initiale est undefined puis une référence fraîche à chaque tour.

Passés à l'analyseur au commit e67b10a (fichier /tmp/ch26/exo1.tsx), le premier et le troisième programme donnent chacun une `ERROR infinite-loop` (« this effect recreates object state ... »). Le deuxième, en revanche, est *silencieux* : l'alias s1 d'un setter appelé dans le rendu

n'est pas signalé par `setter-in-render`. C'est un FN observé, qui montre qu'un non-changement protège un mécanisme sans garantir que chaque règle s'en serve ; il est repris au chapitre 27.

Exercice 26.2: Le mauvais quantificateur

Reprendre `/tmp/ch26/ex8.tsx`. Remplacer mentalement `all_deps_provably_stable` par « il existe un `dep` prouvé stable ». Quel composant devient silencieux ? Pourquoi est-ce un FN, et quelle ligne de la définition 26.1 faudrait-il pour enregistrer ce retour en arrière s'il était voulu ?

Solution

C devient silencieux : `label` est stable, donc la garde « existentielle » saute l'effet, alors que `data` peut changer et relancer l'effet (sémantique OU). C'est un FN car la boucle reste possible. Un tel changement est une suppression sans preuve `must` : il contredit l'invariant 26.1 et ne pourrait être enregistré nulle part ; il ne peut être que refusé.

Exercice 26.3: Ranger une décision

Pour chacune des situations suivantes, dire où elle doit être enregistrée : (a) `missing-deps` cesse de signaler une forme de lecture mal attribuée, le corpus perd douze findings ; (b) on décide que les composants de classe ne seront pas analysés ; (c) on introduit une nouvelle relation `slot_resets` lue par deux règles ; (d) on veut empêcher toute nouvelle règle d'appeler directement l'évaluateur.

Solution

(a) Journal de précision. (b) Issue fermée `wontfix` (avec le raisonnement), et une ligne dans `docs/limitations.md`. (c) ADR : une relation est une contrainte que le reste du système doit respecter, et `docs/relations.md` doit la nommer (`test every_relation_is_in_the_catalogue`). (d) Un cliquet, sur le modèle de `tests/layer_boundary.rs`, accompagné d'un ADR qui énonce la frontière.

Exercice 26.4: Le raccourci d'ADR-025

Dessiner le CFG de `useCart` inline dans `Cart` (exemple 26.6), d'abord tel que le produit le code, puis avec le raccourci refusé (tout `Unreachable` relié au bloc de jonction). Montrer quel hook cesse de dominer toutes les sorties atteignables.

Exercice 26.5: Deux ADR contradictoires

Rédiger, au format du projet (Contexte, Décision, Alternatives refusées, Conséquences), un ADR qui refuse de faire descendre le point fixe dans les callees inconnus, puis l'ADR inverse. Lequel respecte `CLAUDE.md` ? Quel serait le coût mesurable du second, et comment le mesurer ?

Exercice 26.6: Numéros réutilisés

Expliquer comment les numéros d'ADR 040 à 042 ont pu désigner deux documents différents à trois semaines d'écart, et proposer une règle simple qui l'aurait évité.

Solution

Les ADR de précision 040–046 ont été supprimés le 3 septembre (87f87b5) sans laisser de trace dans `docs/adr/` ; les numéros ainsi libérés ont été repris par les ADR suivants, 040 dès le 5 septembre. Une règle possible : un numéro d'ADR n'est jamais réutilisé ; un ADR retiré laisse un fichier-stub « `Withdrawn, see precision-log.md` ».

Chapitre 27

Limites connues, défauts observés et perspectives

Objectifs du chapitre.

- Savoir où `reactant-analyzer` déclare lui-même ne pas voir, et distinguer une limite *assumée* (un compromis décidé, souvent une issue fermée `wontfix`) d'un *défaut* (une sous-approximation qui viole la soundness).
- Connaître, programme minimal et sortie à l'appui, les défauts observés au commit e67b10a pendant la rédaction de ce manuscrit : faux négatifs, faux positifs, écarts entre ADR et code.
- Pour chacun, savoir retrouver la cause dans le code et la piste de correction « à la racine » que la doctrine du projet impose.
- Voir que ces défauts se regroupent en quelques causes communes, et en tirer la dette et les perspectives de recherche du projet.

Prérequis : tout le manuscrit ; plus particulièrement la définition de la soundness (définition 3.8), la distinction défaut/compromis (définition 25.5), le point fixe d'un composant (chapitre 13), les relations (chapitre 14) et l'API des règles (chapitre 17).

Les chapitres précédents ont décrit ce que fait l'analyseur. Celui-ci dit ce qu'il ne fait pas. Le ton est celui d'un relevé : chaque constat a été reproduit au commit e67b10a (27 septembre 2026) sur un programme minimal, et la sortie citée est celle du binaire `target/debug/reactant` construit à ce commit. Aucun constat n'est un jugement sur le projet ; plusieurs sont d'ailleurs le produit direct de sa méthode, qui consiste à écrire ce que l'on ne sait pas encore faire.

Le chapitre avance en quatre temps. La section 27.1 range les limites que le projet documente lui-même. La section 27.2 présente les défauts observés pendant la rédaction, du plus simple à reproduire au plus subtil, et les récapitule dans un tableau. La section 27.3 regroupe ces défauts par cause et dresse l'état de la dette. La section 27.4 ouvre sur les questions qui dépassent la correction d'un bogue.

27.1 Deux registres, et ce que le projet documente déjà

27.1.1 Défaut ou compromis : la grille de lecture

Partons de deux phrases de `docs/limitations.md`. La première décrit un tableau de dépendances :

```
By decision: arr.slice() and arr.concat() in a deps array are not proven fresh, because the same method on a string returns a primitive and the proof would be false #22.
```

La seconde décrit un hook atteint seulement par un `return` :

Reported, but with no line or column, since `Terminator::Return` carries no span #140. It costs a position, never a finding.

La première est un *compromis* : l'analyse reste sound (elle refuse de prouver une fraîcheur qu'elle ne peut pas garantir), elle est seulement moins précise ; le prix est un faux négatif *de précision*, rangé sous l'étiquette `precision-fn`. La seconde est un *défaut* : l'analyse perd une information (la position) sur laquelle le rapport s'appuie. La page les sépare explicitement (« Two registers, and they must not be confused ») et le chapitre 25 en a fait une définition (définition 25.5). Nous l'employons telle quelle : un *défaut* se corrige, un *compromis* se décide.

Définition 27.1: Constat

Un *constat* est un comportement de `reactant-analyzer` observé au commit e67b10a sur un programme minimal, avec sa sortie, rangé dans l'un des trois types suivants :

- **FN** (faux négatif) : le programme a un défaut React certain ou possible que la règle concernée devrait signaler, et la sortie est muette (ou la ligne ✓ ... no issues found s'affiche) ;
- **FP** (faux positif) : la sortie affirme un défaut absent, ou l'affirme à un niveau que la preuve ne porte pas (une `ERROR` sur une conclusion partiellement prouvée) ;
- **écart** : un texte du dépôt (ADR, doc-comment, page de docs/) décrit autre chose que le code.

Un FN dont la cause est une sous-approximation du moteur est un défaut de soundness au sens de définition 3.8 ; un FP au niveau `ERROR` viole la définition des niveaux (définition 1.3) ; un FP au niveau `WARNING` est toléré par construction.

27.1.2 La page `docs/limitations.md`

La page des limites (370 lignes au commit étudié) est le résumé, le tracker est le détail : « Every entry links to its issue, which carries the repro, the cause and the shape of the fix » (l. 4–5). Sa structure suit la grille ci-dessus :

- **Confirmed defects** : une seule entrée, #140, déjà citée. La section est courte à dessein : un défaut connu est corrigé, pas documenté.
- **What reactant may miss** : les FN de précision (callees non atteints, `useContext` non modélisé, hooks React non modélisés, résidus par règle, hypothèses de la preuve de convergence).
- **Why reactant may warn wrongly** : les FP, précédés d'une promesse forte, « Every entry here is Warning or below by construction. A false positive never carries an Error. » (l. 112–113). Plusieurs constats de la section 27.2 la mettent en défaut.
- **Reading the output, Cross-file limits, What a pack can name in a body, Writing declarative packs, Out of scope.**

Tableau 27.1 range ces entrées par couche, dans l'ordre du pipeline décrit au chapitre 4. Les numéros renvoient au tracker ; le détail de chaque mécanisme est dans le chapitre indiqué.

27.1.3 Les limites tranchées « on ne corrige pas »

Le projet ne supprime pas une limite qu'il a décidé de garder : il la ferme en `wontfix`, pour que le raisonnement reste citable et que personne ne repopose le correctif (`docs/TODO.md`, l. 15–17). Au 29 septembre 2026, six issues portent ce label.

Couche (chapitre)	FN documentés	FP documentés (WARNING au plus)	Hors périmètre / wontfix
Détection (chapitre 5)	memo/forwardRef #64; utilitaires par défaut #55; closures imbriquées #56	—	composants dynamiques #63; défaut anonyme #65; node_modules #51
IR, lowering (chapitres 9 et 10)	spreads, clés calculées #76; inlining en position d’instruction #52, une fois par récursion #53, FnLit retourné #57	—	—
Domaines (chapitres 9 et 10)	useContext $\top$ #28; sept hooks React $\top$ #27; NaN #73; slice/concat #22	slot jamais écrit lu Versioned #41; constantes de module $\top$ #34, #36	—
Moteur (chapitres 13 et 16)	callees sans Loc #19; valeurs portées par une boucle #21; parent analysé en intra #20; setters échappés à un niveau #46	—	—
Relations, churn (chapitres 14 et 15)	valeur dérivée du slot #157; invariance supposée #161; remontage #162	arêtes sur slot convergent #39, #159, #160, #161; same_tick #123; garde disjonctive #91	—
Règles (chapitres 18, 20 et 21)	résidus #23, #24, #25, #30; server-component-hook #29; divergence jamais ERROR #144	missing-deps #32, #35; state-mutation #38; frozen-initial-state #136; canal d’assurance #31	heuristique de nom #42; objet entier lu par garde #40
Packs (chapitre 22)	verdicts d’expression #67; ancre unique #68; atteignabilité de rendu #132	—	une entrée du catalogue #101
Projet, CLI (chapitre 23)	alias hors tsconfig #47; @workspace/* #48; ré-exports #49, #50; run rétréci #138	—	TanStack Router #66

TABLE 27.1 – Les limites que docs/limitations.md documente, rangées par couche (état du tracker au 29 septembre 2026).

- #63 (composants dynamiques `const C = cond ? A : B`) et #65 (exports par défaut anonymes) : hors périmètre. Le premier laisse un `analysis-limit` sur le composant qui rend l'élément dynamique (composant `Dyn` de la section 27.2.11).
- #51 : `node_modules` n'est jamais abaissé ; le point d'extension supporté est le `SummaryRegistry` (section 23.10).
- #101 : l'entrée `nullable-return-unguarded` du catalogue Tier A est exclue « by design », car elle pose une question de flot de types et non de sémantique des hooks.
- #42 : `stale-closure` reconnaît une souscription à son nom (`on`, `addListener`, `subscribe`) ; l'heuristique ne mène jamais à une `ERROR`.
- #40 : une `dep` déclarée comme champ (`[x?.locale]`) ne couvre pas un test de véracité de l'objet entier ; le `Warning` est `sound` et aligné sur `eslint`.

À ces fermetures s'ajoutent les « *deliberate non-changes* » d'ADR-20, onze simplifications apparentes que l'on ne doit pas faire sans refaire l'argument de *soundness* (section 26.5). L'une d'elles pèsera plus loin : l'item 9 garde `resolve_setter_aliases` comme passe propre à chaque règle, parce que « *Removing it is an FN* ».

Remarque

Une partie des limites documentées ne sont ni des défauts ni des compromis de précision, mais des *hypothèses nommées* : la *soundness* de la preuve de convergence suppose qu'un appel sur des entrées tenues rend la même valeur, et qu'une navigation se voit (`docs/limitations.md`, l. 67–75, #161) ; la dépendance de rendu suppose qu'un appel opaque ne lit ni n'écrit de `binding` de module (ADR-41 §2). Ces hypothèses sont écrites ; c'est ce qui les distingue d'un défaut.

27.2 Les défauts observés au commit e67b10a

Les seize dossiers de notes qui ont servi à écrire ce manuscrit ont chacun été relus et vérifiés contre le code. Chaque relecture a produit une section « Subtilités, pièges, limites » et, souvent, des constats nouveaux, reproduits sur des programmes minimaux mais absents du tracker à la date de la relecture. Cette section les rassemble. Sauf pour L4, dont la trace vient d'une sonde du moteur (dossier 06), tous les programmes ci-dessous ont été rejoués pour ce chapitre, sous `/tmp/r27/`, avec `reactant check . -show-clean -no-color -trace` lancé depuis le répertoire de l'exemple (et `-info` quand c'est indiqué). Chaque constat est accompagné d'un *témoin* : une variante voisine que l'analyseur traite correctement, ce qui localise le défaut.

La figure 27.1 place les constats dans le pipeline ; le tableau 27.2 (page 1064) les récapitule.

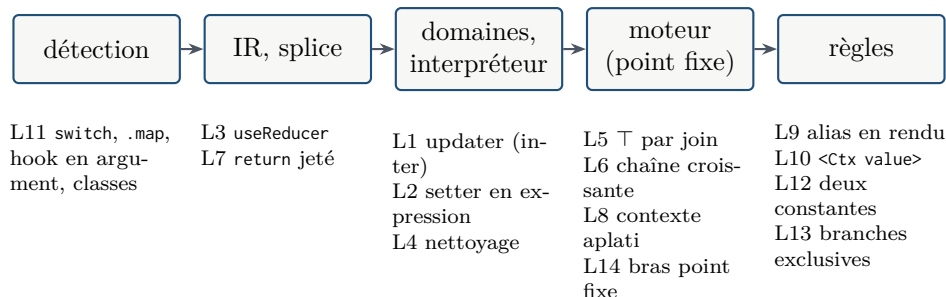


FIGURE 27.1 – Où naissent les constats de ce chapitre. Chaque étiquette renvoie à une ligne du tableau 27.2. Un constat est placé dans la couche qui contient sa *cause*, pas celle où il devient visible : presque tous se voient dans les règles.

27.2.1 L1 — L’updater fonctionnel sur le chemin programme

Commençons par le plus simple à écrire, et le plus inattendu.

Extrait 27.1 – /tmp/r27/updater/Updater.tsx : la même boucle, deux écritures

```
1 export function Updater() {
2   const [count, setCount] = useState(0);
3   useEffect(() => {
4     setCount((c) => c + 1);
5   }, [count]);
6   return <p>{count}</p>;
7 }
8
9 export function Direct() {
10  const [count, setCount] = useState(0);
11  useEffect(() => {
12    setCount(count + 1);
13  }, [count]);
14  return <p>{count}</p>;
15 }
```

Les deux composants bouclent en React : l’effet écrit une valeur nouvelle, le rendu suivant change count, l’effet se relance. L’analyseur ne signale que le second.

```
Direct (2 hooks) Updater.tsx
  warn  infinite-loop [hook:0] (line 13:2) this effect keeps pushing state `count` (its deps do not provably gate
    ↳ it, so the effect can re-run every render) to new values on every run. Potential infinite render loop
    ↳ state `count` is written here [hook:1] (line 13:2)
    ↳ the abstract value of state `count` kept growing and was widened at iteration 3
Updater (2 hooks) Updater.tsx ✓
```

Pourtant le test de tests/functional_updater.rs, l. 62–79 décrit exactement ce programme, attend un finding, et passe. La différence est le chemin : le test appelle analyze_component (tests/functional_updater.rs, l. 53), l’analyse intra sans InterCtx ; la CLI appelle analyze_program, qui en a un (section 13.5.4). Sur le second chemin, exec_setter_call a deux bras indépendants.

Le premier bras, src/domains/interp/interpreter.rs, l. 348–366, exécute l’updater avec c lié à l’état courant. Le second, destiné aux setters d’un *autre* composant, s’applique dès que le callee s’évalue en setter porteur de propriétaire :

Extrait 27.2 – src/domains/interp/interpreter.rs, l. 368–391 — le second bras

```
368 // Cross-component ComponentSetter call.
369 // Handles fn_ = Var(name), FieldAccess { obj: Var, field }, or any other expr
370 // that evaluates to ComponentSetter { component, label }.
371 if let Expr::Call { fn_, args } = expr {
372   let comp_setter = transfer
373     .eval_expr(fn_, env, ctx)
374     .as_state_value()
375     .and_then(|sv| sv.as_setter().map(|(c, l)| (*c, *l)));
376   if let Some((component, label)) = comp_setter
377     && ctx.inter.is_some()
378   {
379     let arg_val = args
380       .first()
381       .map(|a| transfer.eval_expr(a, env, ctx))
382       .and_then(|v| v.as_state_value())
383       .unwrap_or(crate::domains::StateValue::top());
384     if let Some(inter) = &ctx.inter {
385       inter
386         .shared_state
```



```

387         .borrow_mut()
388         .update(component, label, arg_val);
389     }
390 }
391 }

```

Or un setter local s'évalue lui aussi en setter porteur de propriétaire : `Expr::StateSetter(label)` rend `StateValue::component_setter(ctx.component, *label)` (`src/domains/transfer/state_value.rs`, l. 135). Aucun test `component != ctx.component` n'écarte le composant lui-même, et l'argument est évalué par `eval_expr` : c'est la *fonction* `c => c + 1`, une référence `PerRender`, qui entre dans le `SharedStateStore`. La tranche partagée est jointe au nouvel état (`src/engine/fixpoint.rs`, l. 487–492) ; le slot vaut alors nombre $\sqcup$ référence, `c + 1` n'est plus arithmétique et rend $\top$, et $\top$ ne croît plus. Le slot atteint $\top$ en deux tours *par join*, sans widening : `widen_trace` reste vide et le bras point fixe d'infinite-loop n'est jamais consulté (section 27.2.5). La valeur écrite par l'updater est un nombre, donc pas fraîche : le bras churn se tait aussi.

Piège

Un test vert sur `analyze_component` ne prouve rien du CLI. Deux différences au moins séparent les chemins : pas de `SharedStateStore` en intra, et un `Config::default()` dont le `SummaryRegistry` est vide. L'issue #14 le dit du second point (« which no user ever runs ») ; au commit étudié, 47 fichiers de `tests/` construisent `Config::default()`.

Piste de correction. Deux causes se composent, et chacune se corrige au centre : le second bras ne devrait pas s'appliquer aux slots du composant courant (ou devrait exécuter l'updater comme le premier), et le signal de divergence ne devrait pas ignorer un slot qui atteint $\top$ sans widening (L5). La première correction relève d'une même question que L2 et L3 : *qu'est-ce qu'écrire un slot ?* (section 27.3.1).

27.2.2 L2 — Un appel de setter hors position d'instruction

Extrait 27.3 – `/tmp/r27/misc/Misc.tsx` et son témoin `/tmp/r27/ctrl/Ctrl.tsx` (extraits)

```

1 export function AndSetter({ flag }: { flag: boolean }) {
2   const [a, setA] = useState(0);
3   useEffect(() => {
4     flag && setA(a + 1);
5   });
6   return <p>{a}</p>;
7 }
8 export function IfSetter({ flag }: { flag: boolean }) {
9   const [a, setA] = useState(0);
10  useEffect(() => {
11    if (flag) setA(a + 1);
12  });
13  return <p>{a}</p>;
14 }

```

AndSetter (2 hooks) Misc.tsx ✓

IfSetter (2 hooks) Ctrl.tsx

warn infinite-loop [hook:0] (line 17:2) this effect keeps pushing state `a` to new values on every run.

↪ Potential infinite render loop

Le lowering abaisse `flag && setA(a + 1)` en diamant : `Let __tN = flag`, puis, dans le bras droit, `Assign __tN = setA(a + 1)` (`src/lowering/expr_lower.rs`, l. 782–789, section 7.5). L'interpréteur

ne passe par `exec_setter_call` que pour un `ExprStmt` ou un `Return`; un `Let` ou un `Assign` ne reçoit que la pré-passe des callbacks et le cœur de liaison :

Extrait 27.4 – `src/domains/interp/interpreter.rs`, l. 81–89 — seul `ExprStmt` déclenche l'écriture d'état

```
81     Stmt::ExprStmt(expr, _) => {
82         exec_expr_effects(transfer, expr, env, ctx, depth);
83     }
84     // Binding statements: callback pre-pass over the RHS parts, then the
85     // binding core.
86     Stmt::Let { rhs, .. } | Stmt::Assign { rhs, .. } => {
87         exec_callbacks_depth(transfer, rhs, env, ctx, depth);
88         exec_stmt_core(transfer, stmt, env, ctx);
89     }
```

L'écriture n'atteint donc jamais le `StateStore`. `docs/limitations.md` affirme pourtant (l. 49–50) : « A *setter call* in any expression position, `wrap(setN(1))`, a ternary arm, a JSX prop, is seen ». C'est vrai de la relation `slot_writers` depuis #130, pas du store que lit le bras point fixe. La piste centrale est la même que pour L1 : un seul endroit où « appeler un setter » est interprété, atteint par toute évaluation d'un `Call`, quelle que soit sa position.

27.2.3 L3 — `useReducer` : l'action écrite à la place de l'état

Rappel React : `useReducer`

`const [s, dispatch] = useReducer(reducer, init)` tient un état comme `useState`, mais `dispatch(a)` ne l'écrit pas directement : React appelle `reducer(s, a)` et stocke le résultat, avec le même bail-out `Object.is` qu'un setter. Un réducteur qui renvoie `s` inchangé ne provoque donc pas de rendu.

Extrait 27.5 – `/tmp/r27/reducer/Reducer.tsx`

```
1  export function Grows() {
2      const [s, dispatch] = useReducer((acc: number, a: number) => acc + a, 0);
3      useEffect(() => {
4          dispatch(1);
5      });
6      return <p>{s}</p>;
7  }
8
9  function keep(s: { n: number }, a: { type: string }) {
10     return a.type === "noop" ? s : { n: s.n + 1 };
11 }
12
13 export function Settles() {
14     const [state, dispatch] = useReducer(keep, { n: 0 });
15     useEffect(() => {
16         dispatch({ type: "noop" });
17     }, [state]);
18     return <p>{state.n}</p>;
19 }
```

`Grows` boucle (0, 1, 2...); `Settles` ne boucle pas (le réducteur rend `s`, React sort par bail-out). L'analyseur répond exactement l'inverse :

```
Grows (2 hooks) Reducer.tsx ✓
Settles (2 hooks) Reducer.tsx
```

```
error infinite-loop [hook:0] (line 17:2) this effect recreates object state `state` it depends on. Every run
  ↳ stores a fresh reference (`Object.is` always fails) and re-triggers itself: infinite render loop
  ↳ a fresh value is written to state `state` here [hook:1] (line 18:4)
```

La cause est dans le lowering, qui traduit le hook en slot d'état ordinaire en sautant le réducteur :

Extrait 27.6 — `src/lowering/hook_extractor.rs`, l. 715–719 — `useReducer` devient un `HookEntry::State`

```
715     "useReducer" => {
716         let _reducer = it.next(); // skip reducer fn
717         let init = it.next().unwrap_or(Expr::Lit(Prim::Unit));
718         Some(HookEntry::State { label, init, span })
719     }
```

`dispatch` est lié comme un setter : l'interpréteur écrit l'*action* dans le slot. Pour *Grows*, l'action est le point 1 : le store vaut $[0, 1]$, converge sans widening, et le composant est déclaré propre. Pour *Settles*, l'action est un littéral objet frais : le bras *self-churn* en tire une *ERROR* certifiée sur un programme sain, ce que la promesse « A false positive never carries an Error » exclut. Ni *docs/limitations.md* ni le tracker ne mentionnent `useReducer` (recherche au 28 septembre 2026). Une modélisation sound exécute le réducteur comme un updater à deux paramètres (`exec_body` avec $(state, action)$), ou, à défaut, écrit $\top$; dans les deux cas le changement est local au lowering du hook et à l'interprétation des setters, pas aux règles.

27.2.4 L4 — Les écritures d'un nettoyage n'entrent pas dans le store

Rappel React : nettoyage d'effet

La fonction renvoyée par un effet est son *nettoyage* : React l'exécute avant de relancer l'effet (quand ses *deps* changent) et au démontage. Un setter appelé là est une écriture d'état réelle, faite pendant la phase de commit.

Dans `useEffect(() => { return () => setN(7); }, [id])`, l'IR de l'effet se termine par `Return(FnLit { ... setN(7) })`. L'analyseur de CFG ne fait, sur l'expression d'un `Return`, que tirer ses effets de bord (`src/engine/cfg_analyzer.rs`, l. 84–86) ; une `FnLit` est une valeur, son corps n'est pas exécuté. La CLI n'expose pas le store, mais la primitive publique qui le calcule, `analyze_component` (la même que `tests/functional_updater.rs`, l. 53–53 appelle), le fait : rejouée au commit e67b10a sur ce composant, elle donne `state_store = StateStore({0: number[0, 0]})`, ce qui confirme la sonde du dossier 06 (§6.11). La valeur 7, que `n` prend à chaque changement de `id`, n'est pas dans le store, alors que la relation `slot_writers` porte bien la ligne (phase: *Cleanup*, dossier 06 §6.11). Aucun finding faux n'a été exhibé : c'est une sous-approximation *latente*, qui fausse toute règle qui lirait la valeur du slot plutôt que la relation (section 13.7). La piste est d'exécuter le nettoyage renvoyé comme un corps de l'effet, joint au store au même titre que le corps lui-même.

27.2.5 L5 — $\top$ par join n'est pas un signal

Extrait 27.7 — `/tmp/r27/memo/MemoLoop.tsx`

```
1 export function MemoLoop() {
2   const [x, setX] = useState(0);
3   const d = useMemo(() => x + 1, [x]);
4   useEffect(() => {
5     setX(d);
```

```

6   }, [d]);
7   return <p>{x}</p>;
8 }

```

Concrètement, x vaut 0, 1, 2... : chaque écriture change d , qui relance l'effet. La sortie est MemoLoop (3 hooks) MemoLoop.tsx ✓ puis ✓ 1 file(s) no issues found. Le mécanisme tient en deux faits. D'abord, au tour 0, le rendu lit le mémoire avant tout calcul, et un label absent du MemoStore vaut $\top$:

Extrait 27.8 – src/domains/stores/memo_store.rs, l. 27–30 — un mémoire pas encore calculé vaut $\top$

```

27   /// Returns `D::top()` for labels not yet computed (conservative).
28   pub fn get(&self, label: HookLabel) -> D {
29       map_get_or(&self.0, &label, D::top)
30   }

```

L'effet écrit donc $\top$ dans x dès le premier tour ; le tour suivant est stable. Ensuite, widen_trace, le seul signal de divergence que lit le bras point fixe (src/rules/impls/infinite_loop.rs, l. 130), n'est rempli qu'au moment du widening :

Extrait 27.9 – src/engine/fixpoint.rs, l. 514–526 — widen_trace n'enregistre que les slots élargis

```

514   if iteration >= config.widen_threshold {
515       // widen_trace: render+effects only handler widening is not a bug.
516       // `or_insert_with` keeps the FIRST widening iteration (the most
517       // informative one for the witness chain).
518       for label in new_state_incycle.changed_labels(&state) {
519           widen_trace
520               .entry(label)
521               .or_insert_with(|| crate::engine::WidenEvent {
522                   iteration,
523                   writers: slot_writers.get(&label).cloned().unwrap_or_default(),
524               });
525       }
526       state = state.widen_to(&new_state, &thresholds);

```

Un slot qui saute à $\top$ avant le seuil converge « proprement » et la porte 1 le déclare borné. Le prédicat is_unbounded, qui compterait ce $\top$ comme non borné, n'est consulté qu'*après* cette porte. Le bras churn, lui, lit l'écriture comme non fraîche (Versioned), ce qui est le cas documenté #157. La leçon, énoncée par le dossier 06, est générale : *atteindre $\top$ par join est une divergence au même titre qu'être élargi*. La piste centrale est d'en faire un seul prédicat de divergence, lu par toutes les portes, plutôt qu'un effet de bord de l'étape de widening.

27.2.6 L6 — Une chaîne qui grandit sans borne

Extrait 27.10 – /tmp/r27/misc/Misc.tsx, composant StrGrow

```

1   export function StrGrow() {
2       const [s, setS] = useState("");
3       useEffect(() => {
4           setS(s + "x");
5       }, [s]);
6       return <p>{s}</p>;
7   }

```

Ici le widening a bien lieu, et l'analyseur le dit, mais seulement sous -info :

```

StrGrow (2 hooks) Misc.tsx
info widening-info (line 13:2) state `s` kept changing during analysis and was approximated to converge, so
  ↳ findings that depend on it may be imprecise
  → state `s` is written here [hook:1] (line 13:2)
  → the abstract value of state `s` kept growing and was widened at iteration 3

```

`infinite-loop` se tait, parce que la seconde porte demande que l'écriture soit non bornée, et que le prédicat ne regarde pas les chaînes :

Extrait 27.11 – `src/domains/impls/state_value.rs`, l. 371–382 — `is_unbounded` ignore l'emplacement `str`

```

371     /// True if this value represents unbounded growth in the fixpoint.
372     ///
373     /// Used by `InfiniteLoop` to distinguish a setter that writes a bounded value
374     /// (branch narrowing held the growth) from one that truly diverges:
375     /// - Infinite interval bounds → numeric counter without a binding guard
376     /// - Per-render reference slot → new object/function literal every render
377     /// - Residual  $\top$  → precision lost, conservatively unbounded
378     pub fn is_unbounded(&self) -> bool {
379         (!self.num.is_bottom() && (self.num.lo.is_infinite() || self.num.hi.is_infinite()))
380         || self.reference == Stability::PerRender
381         || self.other
382     }

```

Parmi les emplacements que `is_unbounded` ne lit pas (booléen, chaîne, `null`, `undefined`, setter), seule la chaîne a un ensemble concret infini (définition 9.3) : c'est le seul oubli qui puisse cacher une croissance. La correction est locale au domaine : `StrConst::Top` atteint par widening est une croissance non bornée.

27.2.7 L7 — Le `return` d'un appelé greffé sans liaison

Extrait 27.12 – `/tmp/r27/ret/Ret.tsx`; le témoin `/tmp/r27/ctrl/Ctrl.tsx` écrit `set(n + 1)`; sans `return`

```

1  function bump(set: (n: number) => void, n: number) {
2    return set(n + 1);
3  }
4
5  export function C() {
6    const [n, setN] = useState(0);
7    useEffect(() => {
8      bump(setN, n);
9    });
10   return <div />;
11 }

```

La sortie est C (2 hooks) `Ret.tsx` ✓ ; le témoin reçoit `warn infinite-loop [hook:0]` (line 9:2) `this effect keeps pushing state 'n'...` La seule différence est le mot `return`. Le splice d'un appelé (section 8.3) remplace chaque `Return(e)` de l'appelé par un saut vers la jonction, et ne garde `e` que si l'appel était lié à une variable :

Extrait 27.13 – `src/ir/splice.rs`, l. 158–174 — le bras `Return` du splice

```

158   Terminator::Return(ret) => {
159       if let Some(var) = bound_var {
160           // A `Let`, like the call statement it replaces: the
161           // variable is bound here, and an `Assign` without a `Let`
162           // is the IR's spelling of a write to an outer binding

```

```

163         // (see `bound_vars`), which this is not.
164         block.stmts.push(Sstmt::Let {
165             var: var.clone(),
166             rhs: ret,
167             // `Terminator::Return` carries no span of its own, so
168             // the call site is the only position available here.
169             span: call_span,
170         });
171     }
172     return_blocks.push(*new_id);
173     Terminator::Jump(join_block_id)
174 }

```

Pour une instruction `bump(setN, n);`, `bound_var` vaut `None` et l'expression est jetée avec ses effets. Le commentaire du paramètre dit « `None to discard as in a bare f(..); » (src/ir/splice.rs, l. 37–38); mais JavaScript évalue l'expression du return avant de rendre la main : jeter la valeur n'autorise pas à jeter l'évaluation. La correction est d'une ligne et centrale : pousser Sstmt::ExprSstmt(ret, call_span) quand bound_var est None, comme le lowering le fait déjà pour void e.`

27.2.8 L8 — Deux instances, un seul résultat

Extrait 27.14 – `/tmp/r27/persp/Persp.tsx` (extrait); le témoin `/tmp/r27/persp2/One.tsx` ne garde que la première instance

```

1  function Counter({ step }: { step: number }) {
2    const [n, setN] = useState(0);
3    useEffect(() => {
4      setN(n + step);
5    }, [n]);
6    return <p>{n}</p>;
7  }
8
9  export function Two() {
10   return (
11     <div>
12       <Counter step={1} />
13       <Counter step={0} />
14     </div>
15   );
16 }

```

La première instance boucle. Avec elle seule, l'analyseur le voit (`warn infinite-loop [hook:0]` (line 5:2) dans `One.tsx`); avec les deux, il affiche `Counter` (2 hooks) `Persp.tsx` ✓ et `-verbose` donne `Counter: 1 iteration(s), widened: []`. Chaque site d'appel d'un enfant est analysé par `eval_comp_app`, qui range le résultat dans la table du programme par identité d'enfant :

Extrait 27.15 – `src/domains/transfer/state_value.rs`, l. 582–590 — le résultat d'un enfant écrase le précédent

```

582     // Store result in the program-level results map and cache
583     inter
584         .results
585         .borrow_mut()
586         .insert(child, child_result.clone());
587     inter
588         .cache
589         .borrow_mut()

```

```
590 .insert(child, abstract_props.clone(), child_result);
```

Le second site (`step = 0`, point fixe immédiat) remplace le résultat du premier. C'est le prix d'une sensibilité au contexte décidée par ADR-12 (section 16.8) : la table `results` suppose que l'analyse d'un enfant ne dépend que de ses props aplaties, et garde la dernière. Joindre les résultats des sites, ou garder un résultat par (enfant, props abstraites), rendrait la table `sound`; le dossier 08 relève aussi que le cache ne sert que de « déjà vu » (section 16.6).

27.2.9 L9 — Un alias de setter appelé pendant le rendu

Extrait 27.16 – /tmp/r27/alias/Alias.tsx

```
1 export function AliasInRender() {
2   const [n, setN] = useState(0);
3   const s = setN;
4   s(n + 1);
5   return <p>{n}</p>;
6 }
7
8 export function DirectInRender() {
9   const [n, setN] = useState(0);
10  setN(n + 1);
11  return <p>{n}</p>;
12 }
```

```
AliasInRender (1 hooks) Alias.tsx ✓
DirectInRender (1 hooks) Alias.tsx
error setter-in-render [hook:0] (line 12:2) setter `setN` called directly in the render body, move this call into
  ↳ a useEffect or an event handler
  ↳ `setN` is a state setter, so calling it writes state (line 12:2)
```

Les deux composants lèvent « Too many re-renders » en React. Sous `-info`, `AliasInRender` affiche même `verified setter-in-render no setter is called during render`, une assurance fausse, à côté d'un `widening-info` sur `n` : le moteur a bien vu l'écriture, c'est la règle qui ne la voit pas. La règle construit sa table des setters à la main, en ne retenant que les `Let x = StateSetter(1)` du rendu :

Extrait 27.17 – `src/rules/impls/setter_in_render.rs`, l. 63–86 — les setters locaux de `setter-in-render`

```
63 let local_setter_info: HashMap<Var, (HookLabel, Option<crate::ir::SourceRange>>)> =
64   comp_result
65     .render_cfg
66     .blocks
67     .values()
68     .flat_map(|b| b.stmts.iter())
69     .filter_map(|stmt| {
70       if let Stmt::Let {
71         var,
72         rhs: Expr::StateSetter(label),
73         ..
74       } = stmt
75       {
76         let span = comp_result
77           .hook_calls
78           .iter()
79           .find(|c| c.label == *label)
80           .and_then(|c| c.span);
81         Some((var.clone(), (*label, span)))
```

```

82         } else {
83             None
84         }
85     })
86     .collect();

```

`const s = setN` est un `Let s = Var(setN)` et n'y entre pas. Les autres règles passent par `all_setter_labels` ou `resolve_setter_aliases`, et `infinite-loop` voit l'alias équivalent dans un effet (dossier 10, §6.12).

Décision : ADR-020, item 9

« `resolve_setter_aliases` stays a per-rule pass », parce que les alias écrits à la main existent dans le CFG de rendu : « Removing it is an FN. » L'ADR a vu le risque ; c'est une règle qui ne l'applique pas. Alternative plus profonde, conforme à ADR-42 : `setter_in_render.rs` figure dans la liste du cliquet `tests/layer_boundary.rs` des fichiers de règles qui marchent encore la syntaxe, alors que `slot_writers` porte déjà la ligne `phase=Render` de cet appel. Lire la relation, c'est hériter de la résolution des alias faite une fois dans le moteur.

27.2.10 L10 — Le provider React 19

Rappel React : `<Ctx value>`

Depuis React 19, un contexte se fournit directement par `<Ctx value={v}>` ; `<Ctx.Provider value={v}>` reste accepté. Les deux formes ont le même coût : une valeur fraîche à chaque rendu re-rend tous les consommateurs.

Extrait 27.18 – `/tmp/r27/ctx19/Ctx.tsx`

```

1  const Ctx = createContext(null);
2
3  export function P19({ children }) {
4      const [user, setUser] = useState(null);
5      return <Ctx value={{ user, setUser }}>{children}</Ctx>;
6  }
7
8  export function P18({ children }) {
9      const [user, setUser] = useState(null);
10     return <Ctx.Provider value={{ user, setUser }}>{children}</Ctx.Provider>;
11 }

```

P18 (1 hooks) Ctx.tsx

warn unstable-context-value (line 12:9) `Ctx.Provider` is given a newly allocated value on every render. `Object`
 ↳ is` fails for every consumer, so each `useContext(Ctx)` re-renders whenever this component does, even when
 ↳ nothing in the value changed; wrap the value in `useMemo`

P19 (1 hooks) Ctx.tsx ✓

La couche règles reconnaît un provider à son suffixe :

Extrait 27.19 – `src/rules/helpers/providers.rs`, l. 88–99 — un provider se reconnaît à `.Provider`

```

88 /// `TabsContext.Provider` → the proven context binding it names, with the
89 /// cell that binding resolves to.
90 fn provider_context<'a>(<
91     name: &str,
92     contexts: &HashMap<&'a Var, &'a ContextId>,
93 ) -> Option<(&'a Var, &'a ContextId)> {
94     let base = name.strip_suffix(".Provider")?;
95     contexts

```



```

96     .iter()
97     .find(|(c, _)| c.as_str() == base)
98     .map(|(c, id)| (*c, *id))
99 }

```

Le moteur, lui, accepte les deux formes (`strip_suffix(".Provider").unwrap_or(name), src/engine/render_deps.rs, l. 977`). Deux définitions d'un même fait divergent donc : `render_tree` suit le contexte d'un `<Ctx>`, `unstable-context-value` et la relation des consommateurs ne le voient pas. Sous `-info`, le seul signal est un `analysis-limit` (« component Ctx was not found... »), qui suspend les assurances : le canal d'assurance reste honnête, le finding manque. La correction naturelle est la descente de `providers.rs` dans le moteur que prévoit ADR-42 §5 : un seul endroit dit ce qu'est un provider.

27.2.11 L11 — Les formes que la détection ne reconnaît pas

Le front-end décide, par des tests syntaxiques, quelles fonctions sont des composants, des hooks ou des utilitaires (définition 5.1). Une fonction refusée n'est pas sur-approximée par $\top$: elle est *absente*. Une heuristique trop étroite est donc un risque de soundness, pas de précision ; c'est la requalification que #122 a faite (étiquettes `soundness-bug` et `precision-fn`) en ajoutant la règle 4.

Exemple 27.1: Un même défaut, deux formes de retour

Extrait 27.20 – `/tmp/r27/switch/App.tsx`

```

1 function ViaIf({ k, onChange }: { k: number; onChange: (n: number) => void }) {
2   onChange(1);
3   if (k === 1) return <a />;
4   return <b />;
5 }
6 function ViaSwitch({ k, onChange }: { k: number; onChange: (n: number) => void }) {
7   onChange(1);
8   switch (k) {
9     case 1: return <a />;
10    default: return <b />;
11  }
12 }
13 export function App() {
14   const [n, setN] = useState(0);
15   return (
16     <div>
17       <ViaIf k={n} onChange={setN} />
18       <ViaSwitch k={n} onChange={setN} />
19     </div>
20   );
21 }

```

Avec `-info` (lignes verified omises) :

```

App (1 hooks) App.tsx
info analysis-limit component `ViaSwitch` was not found in the analysis registry. Pass its file on the
  ↳ command line to analyse it (FN possible)
suspended analysis-limit 4 passing check(s) withheld: the analysis was truncated in this component, so they
  ↳ are not guaranteed
ViaIf (0 hooks) App.tsx
error cross-setter-in-render var:onChange (line 4:2) prop `onChange` (a state setter of parent `App`)
  ↳ called during render of `ViaIf`, which triggers a parent re-render on every render
  ↳ `onChange` is a state setter, so calling it writes state (line 4:2)

```

La même ERROR certaine est trouvée dans `ViaIf` et manquée dans `ViaSwitch`; la perte n'est avouée qu'en INFO, masquée par défaut, avec un conseil trompeur (le fichier est déjà sur la ligne de commande).

La cause est une énumération à la main des formes de retour :

Extrait 27.21 – `src/lowering/jsx_detect.rs`, l. 14–45 — les formes de retour qui comptent pour la règle 2

```

14 fn stmt_has_jsx_return(stmt: &Statement) -> bool {
15     match stmt {
16         Statement::ReturnStatement(ret) => {
17             ret.argument.as_ref().is_some_and(|e| expr_contains_jsx(e))
18         }
19         // Expression-body arrows (`() => <div/>`) store the expression as an ExpressionStatement
20         Statement::ExpressionStatement(es) => expr_contains_jsx(&es.expression),
21         Statement::BlockStatement(block) => body_returns_jsx(&block.body),
22         Statement::IfStatement(if_) => {
23             stmt_has_jsx_return(&if_.consequent)
24             || if_
25                 .alternate
26                 .as_ref()
27                 .is_some_and(|alt| stmt_has_jsx_return(alt))
28         }
29         Statement::WhileStatement(w) => stmt_has_jsx_return(&w.body),
30         Statement::ForStatement(f) => stmt_has_jsx_return(&f.body),
31         Statement::LabeledStatement(l) => stmt_has_jsx_return(&l.body),
32         Statement::TryStatement(tr) => {
33             body_returns_jsx(&tr.block.body)
34             || tr
35                 .handler
36                 .as_ref()
37                 .is_some_and(|h| body_returns_jsx(&h.body.body))
38             || tr
39                 .finalizer
40                 .as_ref()
41                 .is_some_and(|f| body_returns_jsx(&f.body))
42         }
43     } => false,
44 }
45 }
```

Il n'y a pas de bras `SwitchStatement`, et `expr_contains_jsx` (même fichier, l. 47–63) ne descend ni dans un appel ni dans un tableau : `return items.map(i => <li/>)`, `return createPortal(<div/>, e1)` et `return [<a/>, <b/>]` ne comptent pas. La règle 4 (« it calls a React hook », `src/lowering/component_detector.rs`, l. 66–72) rattrape une partie des composants, mais `expr_calls_hook` n'inspecte que le callee d'un appel (`src/lowering/hook_call_detect.rs`, l. 60–63) : un hook qui n'apparaît qu'en argument, `console.log(useState(0))`, n'est pas vu. Enfin `detect_fns` ne visite que les déclarations de fonctions, les `const` initialisés à une fonction et leurs exports (`src/lowering/detector.rs`, l. 41–71) : une classe extends `React.Component` n'est jamais candidate. Le fichier `/tmp/r27/detect/Limits.tsx` réunit ces formes ; sur huit définitions, seule `Dyn` est analysée, et la sortie se termine par ✓ 1 file(s) no issues found.

Piège

La ligne ✓ ... no issues found. promet qu'un run « read everything it was pointed at » (`docs/limitations.md`, l. 37–38) : elle porte sur les *fichiers* lus, pas sur les fonctions reconnues. Une fonction que la détection ignore ne laisse aucun angle mort.

La piste centrale est de remplacer les `match` à bras par défaut `_ => false` par un parcours exhaustif de l'AST (un visiteur `oxc`), comme le `lowering` s'interdit déjà tout bras `_ => {}` (#77). Les classes, elles, relèvent d'une décision de périmètre qui n'est écrite nulle part : ni `docs/limitations.md` ni une issue ne les mentionnent.

27.2.12 L12 à L14 — Trois faux positifs

Les faux positifs sont tolérés au niveau `WARNING` ; deux des trois constats suivants le sont, le troisième non.

Extrait 27.22 – /tmp/r27/fp/Fp.tsx, composant TwoConsts

L12 — Deux constantes, une seule stabilité

```
1 const A = { k: "a" };
2 const B = { k: "b" };
3
4 export function TwoConsts() {
5   const [v, setV] = useState(A);
6   useEffect(() => {
7     setV(B);
8   }, []);
9   return <p>{v.k}</p>;
10 }
```

```
TwoConsts (2 hooks) Fp.tsx
warn  redundant-set-state [hook:0] (line 8:2) state `v` is set to a stable value it already holds, so the update
      ↳ is redundant
```

`v` passe de `A` à `B`, et React re-rend. La règle conclut « already holds » de ce que la valeur écrite et la valeur courante sont toutes deux *stables* (`src/rules/impls/redundant_set_state.rs`, l. 269) ; or `StateValue::is_stable` teste seulement `to_stability() == Stable` (`src/domains/impls/state_value.rs`, l. 415–417). Pour un primitif ponctuel, stable et égal coïncident ; pour deux références, la stabilité ne dit rien de l'identité. La piste est de demander l'égalité, pas la stabilité : même point pour un primitif, même site d'allocation (définition 6.2) pour une référence.

Extrait 27.23 – /tmp/r27/fp/Fp.tsx, composant ExclusiveBranches

L13 — Une Error sur deux branches exclusives

```
1 export function ExclusiveBranches({ flag }: { flag: boolean }) {
2   const [items, setItems] = useState<string[]>([]);
3   const onClick = () => {
4     if (flag) {
5       items.push("x");
6     } else {
7       setItems(items);
8     }
9   };
10  return <button onClick={onClick}>{items.length}</button>;
11 }
```

```
ExclusiveBranches (2 hooks) Fp.tsx
error state-mutation [hook:0] var:items (line 18:6) `items` is mutated in place and `setItems` is called with
  ↳ the same reference. React compares with `Object.is`, sees no change, and skips the re-render
  → `items` is mutated in place here, so its reference identity is unchanged [hook:0] (line 18:6)
  → the value written to state `items` is the value it already holds [hook:0] (line 20:6)
```

La mutation et le `set` ne s'exécutent jamais dans le même clic. L'état est bien corrompu (une

mutation sans re-rendu), mais la phrase affirmée, « `setItems` is called with the same reference » après la mutation, est fausse sur tout chemin. La primitive qui certifie l'ERROR ne regarde que les *conteneurs* :

Extrait 27.24 – `src/rules/api/query.rs`, l. 1039–1057 — la preuve « même conteneur » de `state-mutation`

```

1039 /// `All` iff a mutation site and a setter-call site share a container (the same
1040 /// event/trigger scope) – the certain `state-mutation` Error (ADR-021 §3);
1041 /// `None` otherwise (a MAY: mutation and set on different triggers).
1042 pub fn must_same_ref_mutation(
1043     mutation_containers: &HashSet<usize>,
1044     setter_containers: &HashSet<usize>,
1045 ) -> MustResult<SameRefMutation> {
1046     if mutation_containers
1047         .intersection(setter_containers)
1048         .next()
1049         .is_some()
1050     {
1051         // No provenance: the inputs are container ids, so neither the mutation
1052         // site nor the setter call reaches this primitive. The rule anchors it.
1053         MustResult::All(Certified::mint(SameRefMutation, Provenance::default()))
1054     } else {
1055         MustResult::None
1056     }
1057 }

```

Deux sites dans le même handler partagent un déclencheur, pas un chemin. C'est une ERROR tirée d'une preuve partielle, la classe même que #142 a corrigée pour *stale-closure* (« Error tier minted from a partial proof ») et que CLAUDE.md exclut depuis (une ERROR prouve *toute* la conclusion, pas un seul conjoint) : la conjonction « mutation *et* set sur le même passage » n'est prouvée que pour son premier conjoint. La primitive doit demander la co-exécution (un chemin qui contient les deux sites, ou la dominance de l'un par l'autre, définition 3.9) ; à défaut, plafonner à WARNING.

Extrait 27.25 – `/tmp/r27/fp/Fp.tsx`, composant Derived
L14 — Le bras point fixe sur un état dérivé

```

1 export function Derived() {
2   const [a, setA] = useState(1);
3   const [b, setB] = useState(0);
4   useEffect(() => {
5     setB(a * 2);
6   }, [a]);
7   return <button onClick={() => setA(a + 1)}>{b}</button>;
8 }

```

Derived (4 hooks) Fp.tsx

```

warn  derived-state [hook:2] (line 29:2) this effect always sets `setB` to a call-free expression of `a` replace
  ↳ with `useMemo` or compute during render
warn  infinite-loop [hook:1] (line 29:2) this effect keeps pushing state `b` (its deps do not provably gate it,
  ↳ so the effect can re-run every render) to new values on every run. Potential infinite render loop
    → state `b` is written here [hook:2] (line 29:2)
    → the abstract value of state `b` kept growing and was widened at iteration 3

```

`derived-state` est juste ; `infinite-loop` ne l'est pas. `a` croît par le handler, le store le joint, et `b = a * 2` croît donc dans la partie « rendu + effets » du tour : `widen_trace` contient `b`, l'écriture $[2, +\infty)$ est non bornée, la dep `a` n'est pas prouvée stable, et le bras point fixe tire. Mais l'effet ne dépend pas de `b` et ne peut pas se relancer lui-même : la croissance est nourrie par un

événement utilisateur, que le moteur exclut précisément de `widen_trace` (« handler widening is not a bug », `src/engine/fixpoint.rs`, l. 515) quand elle est *directe*. Le bras demande « ce slot a-t-il été élargi ? » au lieu de « la boucle rendu $\rightarrow$ effet $\rightarrow$ rendu nourrit-elle cette croissance ? ». #144 et #91 sont voisines mais distinctes ; aucune issue ne décrit ce cas.

27.2.13 Écarts entre ADR, documentation et code

Le chapitre 26 a relevé les écarts les plus visibles entre ADR et code (tableau 26.5) : `docs/semantics.md` annoncé par ADR-1 et jamais créé, `docs/ir.md` d'ADR-3, `rules/api/cache.rs` déclaré disparu par ADR-42. Les dossiers de vérification en ajoutent d'autres, que voici avec leur état au commit.

- **ADR-3 et la règle 4.** L'ADR énumère trois priorités de détection (l. 50–52 : préfixe `use`, retour JSX, annotation). Le code en a quatre ; la quatrième, « it calls a React hook (#122) », est commentée dans `src/lowering/component_detector.rs`, l. 66–72.
- **ADR-13 et `symbol_extractor.rs`.** La section « Consequences » annonce un fichier nouveau, « lightweight pre-pass » (l. 165) : `src/lowering/symbol_extractor.rs`. Le fichier n'a jamais existé sur main ; le graphe de symboles se construit depuis l'IR déjà abaissée (`src/engine/symbol_graph.rs`).
- **ADR-18.** L'ADR place le bras graphe dans `src/rules/helpers/churn_graph.rs` (l. 37) et fonde le kill de convergence sur « the single-writer condition » (l. 67). Le fichier a été supprimé par la promotion d'ADR-42 (le graphe vit dans `src/engine/churn.rs`), et la condition mono-écrivain a été remplacée par la preuve multi-site (`converges_under_all_writes`, `src/engine/guards.rs`, l. 138 ; #154, section 15.6).
- **ADR-42 §4 et §5.** §4 affirme encore « The convergence kill is unchanged: it applies only to a slot with a single effect write row program-wide » (l. 145–146), démenti par ses propres amendements de §6 (« Amended 2026-09-27 (#154) », l. 184). §5 et les Consequences annoncent que `ProgramRelations` *remplace* `ProgramCache` ; au commit, `src/rules/api/cache.rs` (77 lignes) existe et `ProgramCache` *compose* `ProgramRelations` avec les trois structures que la couche règles construit encore (`MountIndex`, `ContextConsumers`, `RenderIndex`).
- **ADR-20 item 9.** « the splice α -renames and emits no param aliases » : le splice émet `let setter#0 = setN` pour un paramètre (dossier 07, exemple 14).
- **Doc-comments en retard.** Trois renvois à une méthode `ImportCtx::callee_is_react` qui n'existe plus (`src/lowering/hook_detector.rs`, l. 44, `src/lowering/mod.rs`, l. 164, `src/lowering/mod.rs`, l. 279 ; la méthode est `classify_callee`) ; « The 14 native rules and the 16-entry doc table » (`src/rules/registry.rs`, l. 130) pour 19 objets `Rule` ; « schema v1 » dans l'aide de `-format json` (`src/cli/mod.rs`, l. 82) pour une sortie en version: 2 (`src/driver/json.rs`, l. 268) ; doc-comments déplacés au-dessus de la mauvaise fonction dans `src/engine/setters.rs` et `src/ir/expr.rs` (rustdoc les attache à la fonction suivante).
- **Tracker.** #158 (`new X()`) est encore ouverte alors que e67b10a la cite et que `Expr::New` existe ; #7 (identité de composant) l'est aussi alors qu'ADR-40 déclare l'implémenter. Le dossier 15 met en garde : une issue ouverte peut l'être pour ses résidus, et seul son dernier commentaire le dit.

Aucun de ces écarts ne change un diagnostic. Ils comptent parce que le projet fait du texte une source (section 26.10) : l'ordre de confiance que le dossier 07 a dû établir pour ADR-42 (le code, puis les doc-comments de module, puis le dernier amendement, puis `docs/relations.md`, puis le reste de l'ADR comme historique) est un symptôme, pas une règle du projet.

27.2.14 Récapitulatif

TABLE 27.2 – Défauts observés au commit e67b10a. « Niveau » est celui du finding manqué (FN) ou émis à tort (FP). Aucune issue ne décrit L1 à L14 au 28 septembre 2026 ; la colonne donne l’issue voisine.

	Constat	Type	Couche	Niveau	Cause	Issue
L1	updater <code>c => c + 1</code> ignoré en inter	FN	interpréteur	WARNING	<code>interpreter.rs</code> l. 368–391	#14
L2	setter hors position d’instruction (<code>&&</code> , ternaire)	FN	interpréteur	WARNING	<code>interpreter.rs</code> l. 81–89	#130
L3	<code>useReducer</code> : action écrite dans le slot	FN et FP	lowering	WARNING ; ERROR	<code>hook_extractor.rs</code> l. 715–719	—
L4	écritures d’un nettoyage hors store	latent	moteur	—	<code>cfg_analyzer.rs</code> l. 84–86	—
L5	slot à <code>T</code> par join, jamais signalé	FN	moteur	WARNING	<code>fixpoint.rs</code> l. 514–526	#157
L6	chaîne croissante non bornée ignorée	FN	domaines	WARNING	<code>state_value.rs</code> l. 378–382	—
L7	<code>return e</code> d’un appelé greffé sans liaison	FN	splice	WARNING	<code>splice.rs</code> l. 158–174	—
L8	deux instances d’un enfant, un résultat	FN	inter-composants	WARNING	<code>state_value.rs</code> l. 582–586	—
L9	alias de setter appelé au rendu	FN	règle	ERROR	<code>setter_in_render.rs</code> l. 63–86	—
L10	provider <code><Ctx value></code> (React 19)	FN	règle	WARNING	<code>providers.rs</code> l. 94	#30
L11	<code>switch</code> , <code>.map</code> , hook en argument, classe	FN	détection	tous	<code>jsx_detect.rs</code> , <code>hook_call_detect.rs</code> , <code>detector.rs</code>	#122
L12	deux constantes distinctes « already holds »	FP	règle	WARNING	<code>redundant_set_state.rs</code> l. 269	—
L13	mutation et set sur branches exclusives	FP	primitive	ERROR	<code>query.rs</code> l. 1039–1057	#142
L14	bras point fixe sur état dérivé	FP	règle	WARNING	<code>infinite_loop.rs</code> l. 130–136	#144
E1	ADR-3 : trois priorités, le code en a quatre	écart	ADR	—	<code>component_detector.rs</code> l. 66–72	#122
E2	ADR-13 : <code>symbol_extractor.rs</code> jamais créé	écart	ADR	—	<code>symbol_graph.rs</code>	—
E3	ADR-18 : <code>churn_graph.rs</code> , kill mono-écrivain	écart	ADR	—	<code>churn.rs</code> , <code>guards.rs</code>	#154
E4	ADR-42 §4 « kill unchanged », §5 <code>cache.rs</code> « gone »	écart	ADR	—	<code>rules/api/cache.rs</code>	#151
E5	<code>docs/semantics.md</code> (ADR-1), <code>docs/ir.md</code> (ADR-3) absents	écart	docs	—	—	—
E6	doc-comments en retard (<code>callee_is_react</code> , « 14 native rules », « schema v1 »)	écart	code	—	<code>hook_detector.rs</code> , <code>registry.rs</code> , <code>cli/mod.rs</code>	—

D’autres constats figurent dans les dossiers de notes sans avoir été rejoués ici ; ils sont signalés dans les chapitres correspondants : collision d’`ExprId` entre le fichier du parent et celui de l’enfant (section 6.9), `alloc_span` qui ne mesure pas les expressions des hooks sans corps, ?? lu comme `||` par le narrowing (section 7.14), `catch` qui repart de l’état d’avant le `try`, hooks imbriqués dans une expression (`a && useX()`) non extraits et déclarés inconditionnels, setter

perdu au join de deux alias (section 11.12), havoc incomplet d'un setter dans un littéral objet remis à un enfant inconnu (section 16.12), `slot_written_outside` qui ne filtre pas le propriétaire (section 18.6).

27.3 Causes communes et dette

27.3.1 Quelques racines pour quatorze constats

La doctrine du projet interdit le correctif local (`CLAUDE.md`, principes 1 et 3) : un problème qui touche plusieurs règles se corrige une fois, au niveau central. Relus sous cet angle, les constats de la section précédente se regroupent (figure 27.2).

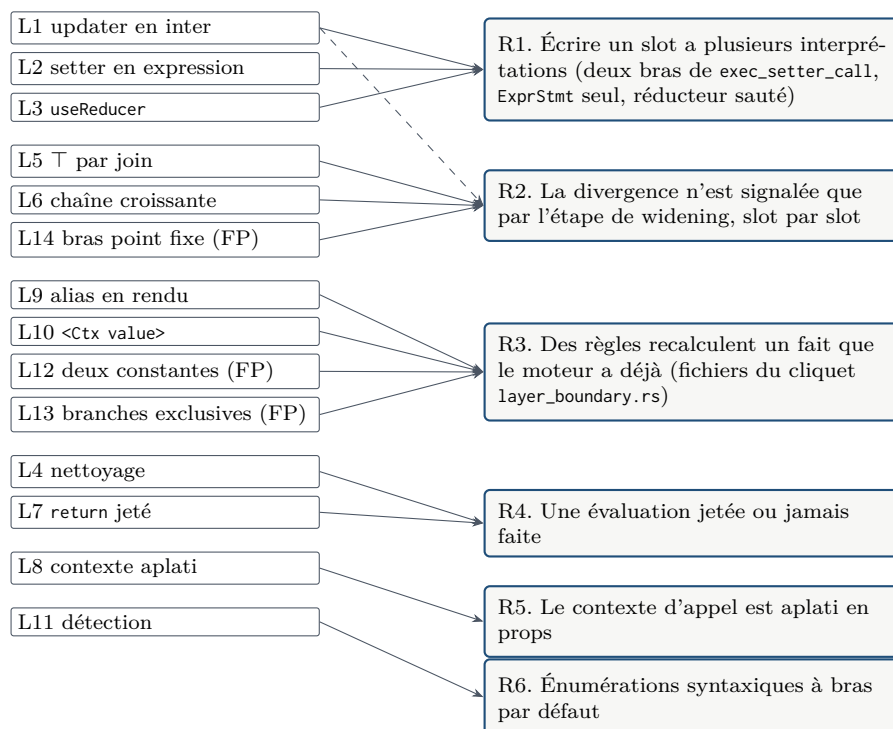


FIGURE 27.2 – Les constats L1–L14 et leurs racines. La flèche pointillée rappelle que L1 cumule deux causes. Le regroupement est une lecture de ce chapitre, pas une décision du projet.

R1 — Qu'est-ce qu'écrire un slot ? L1, L2 et L3 sont trois réponses différentes à la même question : un updater exécuté par le premier bras et pas par le second, un appel vu en instruction et pas en expression, une action prise pour une valeur. Une seule fonction de transfert « écriture d'état », appelée par toute évaluation de `Call` et qui sait exécuter un updater ou un réducteur, supprimerait les trois. C'est la situation que la relation `slot_writers` a déjà atteinte (#130 a rendu visible un appel en toute position) ; le store, lui, garde trois chemins.

R2 — Un seul prédicat de divergence. L5 et L6 sont des oublis dans ce qu'est « diverger » ; L14 est l'excès inverse, une divergence attribuée au slot et non à la boucle. Le dossier 04 ajoute les cycles finis (un booléen qui bascule, un $(i + 1) \% k$) qui convergent sans widening ; nous y revenons en section 27.4. Tous plaident pour un prédicat de divergence unique, calculé par le moteur, que les deux portes du bras point fixe liraient.

R3 — La frontière sans marche. Les fichiers de L9, L12 et L13 (`setter_in_render.rs`, `redundant_set_state.rs`, `state_mutation.rs`) figurent tous trois dans la liste `ALLOWED` de `tests/layer_boundary.rs`, l. 20, qui recense les fichiers de règles marchant encore la syntaxe; L10 naît d'un helper de la couche règles (`providers.rs`) que ADR-42 §5 prévoit de descendre. Chacun recalcule localement un fait (qui est un setter, ce qui est égal, ce qui co-exécute, ce qu'est un provider) que le moteur connaît mieux. La corrélation n'est pas une preuve, mais elle confirme la direction d'ADR-42 : « Each entry is a promotion still to do, not an exception ».

R4 à R6. L4 et L7 jettent une évaluation que JavaScript fait ; la règle générale est qu'une sous-expression non représentable passe par une évaluation pour effet plutôt que d'être supprimée, principe que le lowering s'applique déjà (`src/lowering/expr_lower.rs`, l. 229–232 pour `void e`). L8 relève de la sensibilité au contexte d'ADR-12. L11 relève de l'exhaustivité (#77).

27.3.2 L'état de la dette

`docs/TODO.md` n'est plus qu'une redirection : le backlog vit sur le tracker depuis le 27 août 2026, une issue par limite. Au 29 septembre 2026, 59 issues sont ouvertes, dont 5 soundness-bug (#7, #143, #157, #158, #162), 25 precision-fn, 13 precision-fp et 5 infra. Trois familles méritent d'être nommées.

- **L'instrument de test.** #14 : la plupart des tests d'intégration construisent `Config::default()`, dont le registre de résumés est vide, et beaucoup passent par l'analyse intra ; L1 en est la conséquence directe. #17 : cinq fichiers restent sans test (`registry/keyed.rs`, `stores/heap.rs`, `ir/source_range.rs`, `import_resolution.rs` du lowering et le `lib.rs` de la crate `WASM`) ; #18 : les tests à écrire en premier. `src/engine/render_deps.rs` n'a aucun test unitaire.
- **Les deux interpréteurs.** #12 : « Two CFG interpreters of different strength, and nothing says which ran ». `analyze_cfg` (`worklist`, `widening`, `narrowing`) tourne sur le rendu, les effets et les handlers ; `exec_body_impl` (une passe, arcs arrière ignorés) sur tout corps imbriqué. Une même expression est évaluée avec une précision très différente selon qu'elle est écrite dans un effet ou dans le `setTimeout` de cet effet ; #21 en est une conséquence documentée.
- **La politique Unknown.** Un callback passé à un callee inconnu n'est pas exécuté par le point fixe (ADR-9, « Unknown → skip ») ; la relation des écrivains, elle, le descend à $\top$ (ADR-34 §5). Le dossier 15 y voit « la plus grande exception de principe encore dans le code » au regard de l'invariant « faux négatifs interdits » : un setter dans `myUtil()` => `setX(x + 1)` ne fait pas bouger le store.

Décision : ADR-009 et ADR-034

ADR-9 a choisi de ne pas exécuter un callback remis à un callee de classe `Unknown` : « for a linter, false positives are more costly than false negatives. Descending an unknown callee would be the *sound* choice [...] We accept the FN » ; l'ADR le présente comme un réglage (« It's a knob »). ADR-34 §5 a choisi l'inverse pour la relation des écrivains : « The walk descends any function argument of an unknown call, at $\top$ ». Les deux décisions coexistent. La première est un compromis au sens de définition 25.5 tant qu'aucune règle ne lit l'absence d'écriture dans le store comme une preuve ; c'est précisément ce que fait le bras point fixe d'infinite-loop.

27.4 Perspectives de recherche

Les pistes de la section précédente sont des corrections. Celles qui suivent sont des questions ouvertes, qui demandent un travail de conception avant d’être codées.

27.4.1 Une sémantique concrète écrite

ADR-1 promettait un `docs/semantics.md` qui spécifie les extensions du modèle (effets, handlers, inter-composants), des fonctions de transfert qui citent la règle React-tRace correspondante [LAY25], et des tests de régression contre l’interpréteur de référence. Aucun des trois n’existe (section 26.10.6). Or plusieurs constats de ce chapitre (L3, L4, L7) sont des écarts à une sémantique concrète que personne n’a écrite : que fait `dispatch`, quand court un nettoyage, qu’évalue un `return`. Une sémantique concrète écrite, même partielle, serait l’oracle qui transforme ces constats en tests. Le Strict Mode (double montage en développement) n’a, lui, aucune position écrite dans le dépôt, hors du message de `missing-cleanup`.

27.4.2 Réducteurs, contexte et cycle de vie

`useReducer` (L3) et `useContext` (#28, « the single largest source of analysis limits », 363 sites sur les huit corpus mesurés) ont un point commun : leur valeur est produite par du code que le composant ne contient pas, un réducteur ou le provider le plus proche. Le premier se modélise comme un updater à deux paramètres ; le second demande un store de programme indexé par contexte, alimenté par les valeurs des providers que `render_deps` sait déjà trouver (section 16.10.6). Le cycle de vie (nettoyage, L4 ; remontage, #162 ; Strict Mode) demande, lui, que le modèle abstrait de React (chapitre 2) représente le démontage comme une transition, ce qu’il ne fait pas aujourd’hui.

27.4.3 Précision relationnelle et information de transition

Extrait 27.26 – `/tmp/r27/persp/Persp.tsx`, composant `Toggle`

```
1 export function Toggle() {
2   const [on, setOn] = useState(false);
3   useEffect(() => {
4     setOn(!on);
5   }, [on]);
6   return <p>{String(on)}</p>;
7 }
```

`Toggle` boucle : chaque écriture change `on`, qui relance l’effet. L’analyseur répond `Toggle` (2 hooks) `Persp.tsx` ✓ (`-verbose : 1 iteration(s)`, `widened: []`). Le domaine abstrait sait que `on` est un booléen ; il ne sait pas que chaque exécution de l’effet *le fait changer*. Les domaines non relationnels de `reactant-analyzer` (intervalles, ensembles bornés, chapitre 9) décrivent des valeurs, pas des transitions : un cycle fini (booléen qui bascule, compteur modulo k , deux chaînes qui alternent) converge sans widening, et ni le bras point fixe ni le churn (qui ne regarde que les références) ne le voient. Le dossier 04 range ce FN parmi les limites de conception. Deux directions sont ouvertes : un domaine de transitions (la valeur écrite est-elle toujours différente de la valeur lue ?), ou un domaine relationnel entre valeur lue et valeur écrite. La même absence de relations explique que `narrow_env_for_branch` ne raffine que les gardes `Var op Lit`, `Var` et `!Var` : une garde `x < y` ne dit rien.

27.4.4 Une Error pour la divergence de valeur

Le bras point fixe est plafonné à WARNING (#144) parce que le widening n'est pas une preuve : il dit que l'analyse n'a pas su borner, pas que la valeur croît. Obtenir une ERROR demande une primitive `must` de divergence, par exemple une fonction de rang : une quantité qui croît strictement à chaque tour de la boucle automatique et qu'aucune garde ne borne. C'est l'analogue, pour les valeurs, de ce que la preuve multi-site (chapitre 15) fait pour les références fraîches.

27.4.5 Le vocabulaire des packs

#68 (une seule ancre par règle déclarative) et #67 (pas de verdict d'expression sur une prop, une valeur de provider ou un argument de setter) bornent ce que Tier A peut dire (section 22.9). La position du projet est constante : « Never by a syntactic bypass ». Chaque extension doit donc passer par un nouveau fait du moteur, avec sa polarité ; #143 rappelle que la polarité d'une conjonction de gardes est elle-même à typer.

Résumé du chapitre

- `reactant-analyzer` distingue le *défaut* (sous-approximation, affirmation fausse : se corrige) du *compromis* (imprécision sound : se décide). `docs/limitations.md` ne liste qu'un défaut confirmé (#140) ; les limites tranchées sont des issues fermées `wontfix` (#40, #42, #51, #63, #65, #101) et les non-changements d'ADR-20.
- Au commit e67b10a, quatorze constats (tableau 27.2) : dix FN, dont deux ERROR manquées (un alias de setter en rendu, un enfant à `switch` que la détection fait disparaître) ; trois FP, dont une ERROR sur branches exclusives ; une sous-approximation latente (les nettoyages) ; `useReducer` est à la fois FN et FP ERROR. Aucun n'avait d'issue.
- Chaque constat a une cause localisée, fichier et lignes (de l'interpréteur des domaines à la détection du front-end, en passant par le splice, le point fixe et quatre fichiers de règles), et une piste centrale : une seule interprétation de l'écriture d'état, un seul prédicat de divergence, des règles qui lisent les relations au lieu de marcher la syntaxe, aucune évaluation jetée, des énumérations exhaustives.
- Les écarts `ADR ↔ code` (ADR-3 règle 4, ADR-13, ADR-18, ADR-42 §4-§5, `docs/semantics.md`, `doc-comments`) ne changent aucun diagnostic mais obligent à lire le code comme première source.
- Les perspectives de recherche : une sémantique concrète écrite, la modélisation des réducteurs, du contexte et du cycle de vie, une information de transition pour les primitifs, une primitive `must` de divergence, un vocabulaire de packs étendu par des faits du moteur.

Les annexes qui suivent rassemblent les fiches ADR, le glossaire, le catalogue des règles et la carte de la codebase.

27.5 Exercices

Exercice 27.1: Classer un constat

Pour chacun des cas suivants, dire s'il s'agit d'un défaut ou d'un compromis, et justifier en une phrase : (a) #22, `slice/concat` jamais prouvés frais ; (b) L6, `is_unbounded` qui ignore les chaînes ; (c) L12, `redundant-set-state` sur deux constantes ; (d) L13, `state-mutation` en ERROR sur branches exclusives.

Solution

(a) Compromis : refuser la preuve reste sound, le prix est un FN de précision décidé. (b) Défaut : une croissance réelle est déclarée bornée, une boucle certaine reste silencieuse. (c) Compromis toléré en l'état (FP de niveau WARNING), mais sa cause est une confusion de notions (stable pour égal) qui mérite correction. (d) Défaut : une ERROR affirme une conclusion dont un seul conjoint est prouvé, ce que la définition des niveaux interdit.

Exercice 27.2: Un témoin pour L2

Écrire deux variantes de `AndSetter` qui, d'après le code cité, devraient elles aussi être muettes (le setter dans le bras d'un ternaire, en argument d'un appel), et une variante qui devrait être signalée. Vérifier avec `reactant check`.

Solution

Muettes d'après `src/domains/interp/interpreter.rs`, l. 81–89 : `const r = flag ? setA(a + 1) : 0;` (chaque bras du ternaire est un `Let` du temporaire) et `wrap(setA(a + 1));` avec un `wrap` non inline (le callee de `l'ExprStmt` est `wrap`, pas le setter). Signalée : `if (flag) setA(a + 1);`, un `ExprStmt` dans une branche. Rejoué (`/tmp/r27/exo/Exo.tsx`) : Ternary et Arg sont affichés ✓, Branch reçoit `warn infinite-loop [hook:0]`.

Exercice 27.3: Corriger L7 à la racine

Proposer le changement de `src/ir/splice.rs`, l. 158–174 qui corrige L7, et dire pourquoi il ne peut pas produire de faux négatif nouveau.

Solution

Dans le bras `Terminator::Return(ret)`, quand `bound_var` est `None`, pousser `Stmt::ExprStmt(ret, call_span)` avant le saut vers la jonction. L'instruction ajoutée n'écrit aucune liaison (c'est une évaluation pour effet) ; elle ne peut qu'ajouter des effets (lectures, appels de setter) que JavaScript exécute réellement, donc sur-approximer davantage, jamais retirer un comportement.

Exercice 27.4: Un prédicat de divergence

Écrire en pseudo-code un prédicat `diverges(slot)` qui couvre L5 et L6 sans produire L14. Quelle information le moteur doit-il conserver que `widen_trace` n'a pas aujourd'hui ?

Solution

Une piste : `diverges(s)` vrai si la valeur de `s` a été élargie *ou* a atteint $\top$ par `join` pendant la partie « rendu + effets » d'un tour, *et* si la croissance est entretenue par la boucle automatique, c'est-à-dire si un effet qui écrit `s` dépend (par ses `deps`) d'un slot dont la croissance vient de cette même boucle. Il faut conserver, par slot, l'origine de la croissance (rendu/effets contre handlers) et le graphe « écrit par / dépend de » entre effets et slots, que `effect_triggers` fournit déjà (section 14.6). `widen_trace` ne garde que l'itération et la liste des effets écrivains (et le dossier 06 note que cette liste est surestimée).

Septième partie

Annexes

Annexe A

Fiches des décisions d'architecture (ADR)

Objectifs du chapitre.

- Disposer d'une fiche de référence pour chacune des 42 décisions d'architecture (docs/adr/ADR-001 à ADR-042) : contexte, décision, alternatives refusées, conséquences, statut, ADR liés, fichiers, chapitre du manuscrit qui l'explique.
- Savoir retrouver, pour un type ou un mécanisme du code, l'ADR qui le justifie, et inversement.
- Distinguer un ADR toujours en vigueur d'un ADR remplacé (*superseded*) ou amendé, et suivre la chaîne jusqu'à la décision actuelle.

Prérequis : aucun chapitre n'est nécessaire pour *consulter* une fiche isolée, mais chaque fiche suppose connu le vocabulaire du chapitre 3 (treillis, must/may, soundness) ; pour une lecture suivie plutôt qu'un usage de référence, lire d'abord chapitre 26, qui raconte l'histoire que ces fiches documentent.

Cette annexe n'est pas un résumé narratif : c'est un catalogue. Chaque *ADR* (*Architecture Decision Record*) y a une section numérotée ADR-NN, avec le label `adr:NN` auquel renvoie la macro `\adr{NN}` employée dans tout le manuscrit. La règle d'entrée d'un ADR, redite ici, est celle de docs/adr/README.md : « une décision que le reste du système doit respecter : un domaine, une relation, un invariant, une alternative refusée ». Les corrections de précision — une règle sur-signalait une forme, le moteur a été corrigé, le corpus l'a mesuré — ne sont pas des ADR et vivent dans docs/precision-log.md, hors du périmètre de cette annexe.

Chaque fiche suit huit champs fixes. **Contexte** pose le problème concret qui a motivé la décision. **Décision** résume ce qui a été tranché, souvent avec une citation de l'ADR original (en anglais : les ADR de *reactant-analyzer* sont rédigés en anglais). **Refusé** liste les alternatives examinées et leur raison de rejet — c'est la partie la plus utile à qui voudrait « nettoyer » le code sans relire l'ADR. **Conséquences** nomme les fichiers et types touchés, tels qu'observés à e67b10a (certains fichiers annoncés par un ADR n'ont jamais existé, ou ont depuis disparu : le dire est plus utile que de le taire). **Statut** reprend le vocabulaire de docs/adr/README.md : *Accepted* (décidé, en vigueur), *Implemented* (décidé et dont l'implémentation complète est constatée), *Superseded by ADR-NN* (remplacé). **ADR liés** pointe vers les amendements et remplacements, dans les deux sens. **Fichiers** donne les chemins qui incarnent la décision aujourd'hui. **Chapitre** renvoie au chapitre technique qui explique le mécanisme en détail — cette annexe résume, elle n'explique pas à nouveau.

Quinze ADR sont dits *structurants* : ils fixent une pièce du système que presque tout le reste présuppose (ADR-001, 002, 003, 009, 010, 012, 014, 015, 017, 018, 021, 022, 027, 041, 042). Leur fiche est plus longue et cite du code Rust vérifié dans le dépôt ; les autres tiennent en une dizaine de lignes. L'ordre suit la numérotation des ADR, qui est aussi, à quelques regroupements de

journée près, l'ordre chronologique (chapitre 26 donne la frise complète).

A.1 ADR-001 — La sémantique concrète : React-tRace

29 mai 2026. Statut : Accepted.

Contexte. Une interprétation abstraite calcule une abstraction α d'une sémantique concrète C (définition 3.3). Sans C explicite, la soundness (définition 3.8) ne peut pas être établie formellement, et les fonctions de transfert sont écrites « by guesswork ».

Décision. Adopter comme sémantique concrète de référence l'article *React-tRace* [LAY25] (Lee, Ahn, Yi, OOPSLA 2025), seule formalisation publique des hooks `useState/useEffect` avec preuve de conformité empirique. Le modèle *Tree Memory* et sa boucle de rendu (`StepInit` $\rightarrow$ `StepEffect` $\rightarrow$ `StepCheck`) correspond directement à l'itération de point fixe de `reactant-analyzer` (chapitre 13) ; les règles `SttReBind`, `CheckEffect`, `CheckNoEffect` définissent exactement les conditions de re-render que l'analyser doit détecter.

Refusé. Rien d'alternatif n'est débattu : la décision est fondatrice. Ses limites sont assumées explicitement : React-tRace ne couvre que `useState/useEffect` sans tableau de dépendances ; les extensions (`useMemo`, `useCallback`, `useRef`, objets) devaient être spécifiées « as extensions of React-tRace », sans garantie formelle équivalente.

Conséquences. Trois artefacts étaient annoncés : `docs/semantics.md` spécifiant les extensions ; des fonctions de transfert citant en commentaire la règle React-tRace correspondante ; des tests de régression contre l'interpréteur OCaml du papier. **Aucun des trois n'existe à e67b10a** : ni `docs/semantics.md`, ni occurrence de « React-tRace » ou « SttReBind » dans `src/` ou `tests/` (vérifié : `ls docs/semantics.md` échoue, `grep -r` est vide). La sémantique concrète reste un repère de conception dans les têtes des contributeurs, jamais un artefact exécutable ou vérifié mécaniquement.

ADR liés. Aucun (fondateur) ; le README du projet cite toujours le papier.

Fichiers. Aucun des fichiers annoncés n'a été livré. Le cycle qui en dérive est `src/engine/fixpoint.rs`, l. 704–704 (`analyze_program`).

Chapitre. chapitre 2 (le modèle React abstrait) et chapitre 13 (le cycle qui en dérive).

A.2 ADR-002 — Treillis de stabilité et trois stores

29 mai 2026. Statut : Accepted, raffiné par ADR-17.

Rappel React : Object.is et la stabilité référentielle

React compare les dépendances d'un `useEffect/useMemo` et les props d'un composant mémorisé par `Object.is` (identité de référence pour un objet, une fonction ou un tableau). Une fonction ou un objet littéral recréé à chaque rendu est donc *toujours* « différent » de sa version précédente, même si son contenu est identique. C'est cette propriété — la *stabilité référentielle* — que le domaine doit suivre, pas la valeur elle-même.

Contexte. Il faut un domaine abstrait pour les références (objets, tableaux, fonctions) qui capture exactement ce dont les règles ont besoin : « cette référence est-elle la même à chaque rendu ? ».

Décision. Un treillis à quatre points $\perp \sqsubseteq \{\text{Stable}, \text{Unstable}\} \sqsubseteq \top$; une table de transfert statique ; trois stores séparés plutôt qu'un store unifié : `StateStore` (soumis au point fixe),

MemoStore (dérivé), RefStore (trivial, un ref n'entre jamais dans une comparaison de dépendances). Un widening après deux itérations.

Refusé. Aucune alternative n'est débattue dans cet ADR fondateur.

Conséquences. Fichiers annoncés : `stability.rs`, `ref_store.rs`, `product.rs`. À e67b10a, `Stability` vit dans `src/domains/impls/stability.rs`; **`src/domains/stores/ref_store.rs` et `src/domains/product.rs` n'existent plus** (vérifié : ls échoue sur les deux) — ADR-14 notait déjà que `ProductDomain::widen` était « a never-wired cartesian product ». Le treillis à quatre points lui-même est **raffiné par ADR-17** : `Unstable` devient `PerRender` (borne must) et gagne `Versioned(S)/VersionedTop` (borne may) — voir ?? pour la forme actuelle.

ADR liés. Raffiné par ADR-17 ; les stores fusionnent plus tard dans le domaine produit d'ADR-15.

Fichiers. `src/domains/impls/stability.rs`; `src/domains/stores/state_store.rs`.

Chapitre. chapitre 9 (le treillis, avant raffinement) et chapitre 11 (les stores).

A.3 ADR-003 — Une IR dédiée à base de CFG

29 mai 2026. Statut : Accepted.

Contexte. L'AST produit par `oxc` a des dizaines de formes syntaxiquement différentes pour une même situation sémantique (`if/else` vs opérateur ternaire, boucle `for` vs `while`, etc.). Faire porter les domaines abstraits directement sur cet AST les obligerait à traiter chaque forme séparément.

Décision. Abaisser (*lowering*) l'AST vers une IR propre, représentée comme un CFG (définition 6.1) plutôt qu'un arbre : les retours précoces et les boucles s'y expriment nativement, les composantes fortement connexes donnent les points de widening, la dominance (nécessaire pour détecter les hooks conditionnels, `conditional-hook`) y est standard. Les hooks sont des nœuds de première classe de l'IR (pas une convention de nommage retrouvée après coup) ; les composants sont identifiés par un ordre de priorités : préfixe `use` $\Rightarrow$ hook ; JSX retourné $\Rightarrow$ composant ; annotation explicite.

Refusé. Une IR arborescente à la `React-tRace` : elle exigerait une transformation en *style de passage de continuation* (CPS) pour représenter les retours précoces, et ne peut pas représenter de boucle sans arête arrière — un choix incompatible avec la dominance dont ADR-25 aura besoin.

Conséquences. Les domaines abstraits ne voient jamais l'AST `oxc`, seulement l'IR. `docs/ir.md`, annoncé, a disparu ; la table de désucre elle-même a dérivé depuis (le diamant de blocs qui désucre `&&/||`, conservé par ADR-20 item 1 — section 26.5).

ADR liés. Sa table de désucre dérive au fil du temps (constat d'ADR-20) ; le CFG qu'il pose est raffiné par ADR-25 (`fall-through`) et ADR-35 (arête `Await`).

Fichiers. `src/ir/` (`mod.rs`, `cfg.rs`, `expr.rs`, `stmt.rs`, `hooks.rs`) ; `src/lowering/` (abaissement en une passe).

Chapitre. chapitre 6.

A.4 ADR-004 — `render_cfg` et `effect_cfg` séparés

29 mai 2026. Statut : Accepted.

Contexte. Le rendu et les effets ont des sémantiques distinctes vis-à-vis du `StateStore` : un rendu lit l'état, un effet peut l'écrire et en déclencher un nouveau. Les confondre dans un seul

graphe rendrait cette distinction implicite.

Décision. `ComponentIR{ render_cfg, hooks: Vec<HookEntry> }`; chaque effet porte son propre `body_cfg`. Le cycle d'analyse est rendu $\rightarrow$ effets $\rightarrow$ join $\rightarrow$ test de convergence $\rightarrow$ widening (chapitre 13).

Refusé. Un méta-CFG unifié, avec des arêtes arrière rendu $\rightarrow$ effet $\rightarrow$ vérification. Plus expressif, mais jugé « hard to maintain » et rendant implicite la séparation render/effet, qui est un invariant de React : autant la rendre structurelle. Les bugs visés (boucles infinies, état incohérent) sont des propriétés de l'état au point fixe, pas des propriétés de chemins dans un graphe combiné.

Conséquences. Aucun changement depuis : c'est toujours la forme du code à e67b10a.

ADR liés. Aucun amendement direct ; présuppose ADR-3 (l'IR CFG) et conditionne ADR-9 (traversée des callbacks pendant les effets).

Fichiers. `src/ir/component.rs` (`ComponentIR`) ; `src/engine/fixpoint.rs` (le cycle rendu/effets).

Chapitre. chapitre 6 (la structure) et chapitre 13 (le cycle).

A.5 ADR-005 — Portée intra-procédurale et registre de hooks

29 mai 2026. Statut : *Accepted, dépassé de fait.*

Contexte. Un composant appelle des hooks personnalisés et des hooks de bibliothèques (`react-router`, `TanStack Query`...). L'analyse ne peut pas tous les inliner.

Décision. Une première phase intra-procédurale : un hook inconnu vaut $\top$ (Unknown). Un registre `HookModel` en couches (hooks natifs, modules de bibliothèques connus, configuration utilisateur `reactant.toml`, repli Unknown).

Refusé. Rien de débattu ; la portée intra-procédurale est un choix d'étape, explicitement provisoire.

Conséquences. `src/registry/user_config.rs` et `reactant.toml` étaient annoncés — **jamais créés** (vérifié : `src/registry/user_config.rs` n'existe pas). À e67b10a, `src/registry/` contient `keyed.rs`, `mod.rs`, `summary.rs`. Le registre en couches est dépassé de fait par l'inlining inter-composants (ADR-12), l'inlining cross-file (ADR-13) et le `SummaryRegistry` — des résumés packagés pour des bibliothèques précises (`TanStack`, `React Router`, `next/navigation`) plutôt qu'une configuration utilisateur.

ADR liés. Dépassé par ADR-12, ADR-13 et le `SummaryRegistry`.

Fichiers. `src/registry/keyed.rs`, `src/registry/summary.rs`.

Chapitre. chapitre 5.

A.6 ADR-006 — Les règles en post-passe

29 mai 2026. Statut : *Accepted.*

Contexte. Une règle peut être écrite *inline*, couplée aux fonctions de transfert du point fixe, ou en *post-passe* sur le résultat convergé.

Décision. Post-passe pure sur `AnalysisResult`. Seule exception « inline » tolérée : enregistrer `widened_labels` pendant le point fixe lui-même (une information que la post-passe ne pourrait pas reconstituer). « The engine [...] doesn't know about rules ».

Refusé. Le couplage inline généralisé : il romprait la frontière moteur/règles et forcerait chaque nouvelle règle à comprendre l'algorithme de point fixe.

Conséquences. La signature (`&AnalysisResult`) $\rightarrow$ `Vec<Warning>` est remplacée par `check(&RuleCtx)` (ADR-21 §4); la frontière est durcie plus tard par ADR-42 (les règles ne marchent plus aucune syntaxe, seulement des relations).

ADR liés. Sa signature est remplacée par ADR-21; sa frontière est durcie par ADR-42.

Fichiers. `src/rules/registry.rs`, l. 254–254 (`RuleRegistry::check_component`).

Chapitre. chapitre 17.

A.7 ADR-007 — Requêtes inter-domaines

2 juin 2026. Statut : Accepted.

Contexte. Un domaine (par exemple un futur `SetterEffect`) doit parfois lire l'état d'un autre domaine (la `Stability` d'une référence). Référence prise chez MOPSA, dont le *Manager* résout ce problème avec des GADT.

Décision (A, implémentée). Un trait `QueryContext` en `&dyn` (object-safe); `AnalysisCtx<D>` regroupe (`state`, `memo`, `heap`, `query`); trois implémentations (`NullCtx`, `FixpointCtx`, `AnalysisQueryCtx`).

Décision (B, différée). Des types marqueurs `Queryable<Q>` en Rust stable, la spécialisation (`#[feature(specialization)]`, RFC #31844) étant instable.

Refusé. Le *Manager* générique à la MOPSA : les GADT sont absents de Rust stable et la spécialisation est instable — la décision B ne peut pas être prise tout de suite.

Conséquences. ADR-15 refuse ensuite de généraliser en « query pool » n -aire; ADR-21 réalise le « typed Manager later » promis ici, sous une autre forme (la surface de requêtes typée des règles).

ADR liés. La généralisation en pool est refusée par ADR-15; le « Manager typé plus tard » est réalisé par ADR-21.

Fichiers. `src/domains/mod.rs` (`AnalysisCtx`, trait `QueryContext`).

Chapitre. chapitre 11.

A.8 ADR-008 — `StateValue` : enum plat et `TypedStateStore`

2 juin 2026. Statut : Superseded by ADR-15.

Contexte. La `Stability` seule ne distingue pas `setState(count + 1)` (croît sans borne) de `setState(42)` (converge immédiatement) : il faut une vraie valeur abstraite pour la partie état.

Décision. Un enum plat `StateValue` { `Bottom`, `Null`, `Undefined`, `Number(Interval)`, ..., `Top` }; des sous-stores typés par sorte inférée statiquement; puis (4 juin) un indice `useState<T>` pour contourner le problème que `join(Null, Number) = Top` fait perdre toute borne.

Refusé (limite résiduelle assumée). `useState(null)` sans annotation de type reste un faux négatif : sans indice, la sorte `Null` et la sorte `Number` restent incomparables, le `join` retombe à `⊥`.

Conséquences. **Superseded by ADR-15** le 14 juillet : les trois mécanismes (l'enum plat, `TypedStateStore`, l'indice `useState<T>`) sont *supprimés* en bloc; `Interval`, `BoolVal`, `StrConst` survivent comme composantes du domaine produit.

ADR liés. **Superseded by ADR-15** (le seul ADR entièrement remplacé plutôt qu'amendé, section 26.4).

Fichiers. Aucun : `TypedStateStore` n'existe plus à e67b10a.

Chapitre. chapitre 10 (pour la forme qui l'a remplacé).

A.9 ADR-009 — Traversée sémantique des callbacks

2 juin 2026, complétée le 4 juin. Statut : Accepted — complete.

Contexte. `fetch().then(u => setUser(u))` est invisible pour un point fixe qui ne regarde que les instructions du corps direct. Mais descendre uniformément dans *tout* callback armerait un faux positif : un `addEventListener` de clic verrait son handler exécuté à chaque itération du point fixe, faisant croître un état qui ne croît en réalité qu'au clic.

Décision. Classifier chaque appelé : `TriggerClass` distingue `Setter` (le setter lui-même, jamais descendu), `InCycle` (conséquence directe du rendu ou de l'effet courant : `map`, `forEach`, `.then`, `setTimeout`... — descendu), `Subscription` (abonnement externe, *non* descendu) et `Unknown` (appelé non reconnu, *non* descendu). Les *handlers* deviennent des points d'entrée séparés de l'analyse (`HookEntry::Handler`, `extract_handlers`, `extract_subscriptions`); un widening provoqué par un handler est exclu de `widened_labels`.

Extrait A.1 – `src/domains/interp/callbacks.rs`, l. 6–23 — `TriggerClass` — la classification qui décide si un appelé est descendu

```
6 // How a call's closure arguments should be treated by the side-effect pre-pass.
7 #[derive(Debug, Clone, Copy, PartialEq, Eq)]
8 pub enum TriggerClass {
9     // Callee is a bound state setter handled by the core exec path (functional
10    // updaters), so the pre-pass must NOT descend its closure.
11    Setter,
12    // Runs as a consequence of the current render/effect: synchronous HOFs
13    // ('map', 'forEach', ...) and scheduled async ('.then'/'catch'/'finally',
14    // 'setTimeout'/'setInterval', 'queueMicrotask', 'requestAnimationFrame').
15    // Its closure arguments ARE descended into.
16    InCycle,
17    // Event subscription ('addEventListener'/'removeEventListener') triggered
18    // externally, NOT part of the render→effect→render cycle. Not descended.
19    Subscription,
20    // Unrecognized callee (custom helper/hook). Conservatively NOT descended
21    // (FP-averse: avoids flagging custom subscription wrappers).
22    Unknown,
23 }
```

Refusé — choix assumé non sound. `Unknown → skip` : « for a linter, false positives are more costly than false negatives. Descending an unknown callee would be the *sound* choice [...] We accept the FN ». C'est un choix *antérieur* à `CLAUDE.md` (juin, contre juillet) et il contredit aujourd'hui l'invariant « faux négatifs interdits ».

Piège

Ce choix a survécu, mais pas partout : la **marche des relations** (ADR-34 §5) a pris la décision inverse — elle descend tout argument fonction d'un appel inconnu, à $\top$, « because an unknown callee may invoke it ». Deux traversées du même code, deux politiques différentes sur les appelés inconnus : c'est le résidu tracé par les issues #19 (FN, appelés inconnus sans `Loc`) et #12 (deux interpréteurs de force différente). Ne pas supposer qu'une correction faite d'un côté vaut pour l'autre.

Conséquences. `src/domains/interp/callbacks.rs` porte `classify_callee` et `TriggerClass`. `src/domains/interp/interpreter.rs` porte `exec_callbacks_depth` (appelé dans `exec_full_stmt`) et `exec_body/exec_body_impl`, la descente « effets de bord seulement, valeur de retour ignorée ». L'API publique (`Transfer`, `règles`, `AnalysisResult`) est inchangée pour ce premier incrément.

ADR liés. Le choix `Unknown → skip` est inversé pour la marche des relations par ADR-34 §5.

Fichiers. `src/domains/interp/callbacks.rs`; `src/domains/interp/interpreter.rs`.

Chapitre. chapitre 13.

A.10 ADR-010 — Modèle de tas par site d'allocation

3 juin 2026. Statut : *Accepted* — *complete*.

Contexte. Deux formes échappaient à l'analyse : un callback stocké dans une variable puis passé ailleurs (`const cb = () => . . . ; setTimeout(cb)`), et un appel direct à une fonction locale (`load()` définie plus haut dans le composant).

Décision. Un identifiant `ExprId` sur chaque nœud allouant (littéral objet, tableau, fonction) ; un tas `Heap: ExprId -> HeapValue` monotone ; l'environnement abstrait porte deux cartes séparées, `stabs` (valeurs) et `locs` (localisations), car une carte unique `Val | Loc` s'écrasait sur elle-même dès qu'une variable changeait de rôle. Une profondeur d'inlining plafonnée :

Extrait A.2 – `src/domains/interp/interpreter.rs`, l. 21–21 — la profondeur d'inlining des appels locaux

```
21 pub const MAX_INLINE_DEPTH: usize = 3;
```

Refusé — limite connue assumée. Au-delà de cette profondeur, ou en cas de récursion, l'appel n'est pas descendu : un diagnostic `INFO analysis-limit` le signale plutôt que de le passer sous silence.

Conséquences. ADR-23 §3 révélera plus tard un défaut de conception laissé par cet ADR : `locs` est monotone et n'est *jamais invalidé* à la réaffectation d'une variable — un raffinement futur, pas un non-changement décidé.

ADR liés. Défaut révélé (non corrigé ici) par ADR-23 §3.

Fichiers. `src/ir/types.rs` (`ExprId`) ; `src/domains/stores/abstract_env.rs` (`locs`) ; `src/domains/stores/heap.rs` ; `src/engine/cfg_analyzer.rs` et `src/engine/fixpoint.rs` (le tas créé une fois, partagé entre passes de rendu et d'effets).

Chapitre. chapitre 6 (`ExprId`) et chapitre 11 (le tas).

A.11 ADR-011 — `SourceRange` et notes

3 juin 2026. Statut : *Accepted*, §Note superseded by ADR-19.

Contexte. Sans position, un diagnostic ne peut pas être affiché à côté du code fautif ; et un seul type de note limitait l'analyse à un seul participant nommé par finding.

Décision. `SourceRange { line, col }` ; une table `line_starts` construite en $O(n)$, offrant une conversion en $O(\log n)$ par recherche dichotomique. `Note { message, hook_label, range }` ; `Option<SourceRange>` pour ne pas casser les tests d'IR écrits à la main sans position réelle.

Refusé. Rien de débattu au-delà de la structure retenue.

Conséquences. La structure `Note` et sa limitation « pas de fichier » (un seul projet ne pouvait alors désigner qu'un fichier implicite) sont **superseded by ADR-19**, qui ajoute `FileId` après l'arrivée de l'inlining cross-file (ADR-13) : une note pouvait sinon pointer dans le mauvais fichier.

ADR liés. §Note superseded by ADR-19.

Fichiers. `src/ir/source_range.rs`, l. 36–43.

Chapitre. chapitre 6.

A.12 ADR-012 — Analyse inter-composants

4 juin 2026. Statut : Accepted.

Contexte. Un setter passé en *prop* à un enfant, puis appelé là-bas, ferme une boucle qui traverse la frontière entre deux composants — invisible à une analyse strictement intra-composant.

Décision. Inlining **descendant** (*top-down*) : le parent est analysé d'abord, jamais de résumés remontants (*bottom-up*), « without justified gain at the current stage ». Un cache indexé par égalité abstraite des props (double leq, dans les deux sens) évite de ré-analyser un enfant avec des props équivalentes. `ComponentSetter` (l'analogue du `Set_clos` de `React-tRace`) ; `SharedStateStore` pour l'état partagé entre composants inlinés ; un `RootDetector` modulaire (`-all-roots`, `-entry`) pour décider par où commencer. La récursion entre composants donne $\top$ (même politique que MOPSA pour la récursion de fonctions).

Refusé. Les résumés remontants (*bottom-up*) : pas de gain démontré à ce stade du projet.

Conséquences. Limites reconnues : les imports non résolus, les composants dynamiques (`const C = cond ? A : B`, #63, plus tard `wontfix`, section 26.10). `src/engine/fixpoint.rs` reçoit `SharedStateStore` en paramètre et propage les appels `ComponentSetter` ; `src/engine/` gagne `ComponentRegistry`, `RootDetector`, `ProgramAnalysisResult` ; les règles migrent vers cette dernière.

ADR liés. L'inlining *bottom-up* refusé ici reste refusé jusqu'à e67b10a.

Fichiers. `src/domains/stores/` (`SharedStateStore`) ; `src/engine/` (`ComponentRegistry`, `RootDetector`) ; `src/resolver/mod.rs`, l. 439 (`analyze_lowered`).

Chapitre. chapitre 16.

A.13 ADR-013 — Analyse cross-file

5 juin 2026. Statut : Accepted — phases 1–4.

Contexte. Trois problèmes à la fois : des collisions de noms (`Page()` défini dans plusieurs fichiers) ; une découverte de fichiers manuelle ; des utilitaires opaques qui masquent un setter (`doOrNot(setX(...))` devient un faux positif si `doOrNot` n'est jamais regardé).

Décision. Une clé (`PathBuf`, `String`) plutôt qu'un nom seul ; deux traits, `FileDiscoverer` et `ImportResolver` ; une entrée par répertoire ; un graphe de **symboles** (pas de fichiers) trié topologiquement pour ordonner l'analyse ; `FunctionIR` et l'inlining des fonctions utilitaires ; une analyse *eager* (tout le graphe, pas de paresse) ; un import non résolu vaut $\top$ plus un INFO — « FPs possible, FNs forbidden », déjà la politique de soundness du projet.

Refusé. Rien d'alternatif débattu pour la structure retenue ; les limites sont assumées et listées, pas contournées : `alias tsconfig`, chaînes de ré-export, `node_modules` (jamais abaissé, #51, `wontfix`), inlining seulement en position d'instruction (#52).

Conséquences. `src/resolver/mod.rs` porte `lower_files` (`src/resolver/mod.rs`, l. 237–237) et `lower_files_with` (`src/resolver/mod.rs`, l. 247–247).

ADR liés. Le périmètre `node_modules` refusé ici est confirmé `wontfix` par l'issue #51 (section 26.10).

Fichiers. `src/resolver/mod.rs` ; `src/ir/function.rs` (`FunctionIR`).

Chapitre. chapitre 16.

A.14 ADR-014 — Widening à seuils ; narrowing abandonné

27–28 juin 2026. Statut : Accepted (partie 1) ; partie 2 superseded.

Contexte. Le widening standard d’un intervalle saute directement à $\pm\infty$: count borné par une garde `count < 10` converge vers $[0, \infty)$, perdant l’information `count ∈ [0, 10]` que la garde donnait pourtant.

Décision (partie 1, implémentée). Récolter des *seuils* dans les littéraux des gardes et des initialisations rencontrées dans le CFG ; `widen_to(&self, other, &[f64])` qui élargit vers le seuil le plus proche au lieu de $\pm\infty$, sans changer la signature de `widen` elle-même ; la même technique s’applique à l’arête arrière interne d’`analyze_cfg`.

Refusé — partie 2, abandonnée le lendemain de sa proposition. Un *narrowing* descendant, pour affiner après convergence. Révision interne : « narrowing and threshold widening both recover only concrete literal bounds » — le narrowing n’aurait rien apporté que le widening à seuils n’apportait déjà, et une phase descendante ne peut de toute façon pas faire redescendre un store dont les join sont monotones (une valeur ne peut que croître au sens du treillis). L’ADR exigeait des tests de propriété pour cette partie 2 : ils n’ont jamais été écrits, faute d’objet.

Conséquences. `src/domains/impls/interval.rs` (`widen_to`) ; `src/domains/impls/state_value.rs` route les seuils vers la composante `num` ; `src/engine/fixpoint.rs` calcule les seuils avant la boucle externe et les transmet à `widen_to` ; `src/engine/analysis_result.rs` : `widened_labels` : `HashSet` devient plus tard une `HashMap` d’un enum `WidenOutcome`.

ADR liés. Aucun amendement ultérieur.

Fichiers. `src/domains/impls/interval.rs` ; `src/engine/fixpoint.rs` ; `src/engine/cfg_analyzer.rs`.

Chapitre. chapitre 12.

A.15 ADR-015 — Domaine produit sur sortes disjointes

14 juillet 2026. Statut : Implemented ; supersedes ADR-8.

Contexte. Après ADR-8, chaque join entre deux sortes JS différentes (un `null` et un `number`) retombait à $\top$: trois rustines empilées (`TypedStateStore`, inférence de type, indice `useState<T>`) tentaient de compenser sans traiter la cause.

Décision. « disjoint sum $\Rightarrow$ union = product » : les sortes primitives JS sont mutuellement exclusives (une valeur n’est jamais à la fois un nombre et une chaîne), donc l’union disjonctive des sortes dégénère en un *produit* — un emplacement indépendant par sorte, chacun $\perp$ quand cette sorte est impossible.

Extrait A.3 – `src/domains/impls/state_value.rs`, l. 16–44 — `StateValue` — le domaine produit sur sortes disjointes

```
16 /// Abstract JS value: pointwise product over the disjoint JS kinds (ADR-015).
17 ///
18 /// JS primitive kinds are mutually exclusive (a value is never a number AND a
19 /// string), so the disjunctive completion of the kind sum degenerates into a
20 /// product: one independent slot per kind, each  $\perp$  when that kind is impossible.
21 /// `join`/`meet`/`widen` are pointwise – a cross-kind join keeps BOTH kinds
22 /// (`number | null`, `number | string`) instead of collapsing to  $\top$ , which is
23 /// what enables infinite-loop detection through nullable states without a
24 /// TypeScript hint.
25 #[derive(Clone, PartialEq)]
26 pub struct StateValue {
27     /// Numeric kind – interval [lo, hi];  $\perp$  = cannot be a number.
```

```

28   pub num: Interval,
29   /// Boolean kind.
30   pub boolean: BoolVal,
31   /// String kind – finite constant set, threshold-widened to  $\top$ .
32   pub str: StrConst,
33   /// Object/array/function kind – reference stability;  $\perp$  = not a reference.
34   pub reference: Stability,
35   /// `null` possible?
36   pub null: bool,
37   /// `undefined` possible?
38   pub undef: bool,
39   /// Cross-component useState setter (flat lattice with identity payload).
40   pub setter: SetterVal,
41   /// Residual  $\top$  – kinds not modelled (symbol, bigint, ...). `true` means the
42   /// value may be something outside every other slot.
43   pub other: bool,
44 }

```

Les opérations (join, meet, widen) sont ponctuelles, sorte par sorte : un join entre un état null et un état number garde *les deux* sortes (number|null) au lieu de s'effondrer sur $\top$. C'est précisément ce qui rend détectable un compteur initialisé à null : la coercition JS `ToNumber(null)` = 0 est câblée dans les opérateurs binaires, et un narrowing de nullabilité et de véracité s'ajoute sur les gardes (if (x), x ?? d, x !== null).

Exemple A.1: Le FN d'ADR-008 devient une détection

Extrait A.4 – Compteur initialisé à null

```

1 import { useState, useEffect } from "react";
2
3 function Counter() {
4   const [n, setN] = useState(null);
5   useEffect(() => {
6     setN(n + 1);
7   }, [n]);
8   return <span>{n}</span>;
9 }

```

Sous ADR-8, n valait $\top$ dès l'initialisation (`join(Null, Number) = Top`), et un $\top$ ne peut jamais prouver de croissance : silence. Sous ADR-15, n porte la sorte null *et* la sorte num = [0,0]; $n + 1$ coerce le null en 0 et produit num = [1,1], qui croît à chaque itération du point fixe. Sortie observée (`-no-color, e67b10a`) :

```

Counter (2 hooks) null-counter.tsx
  warn  infinite-loop [hook:0] (line 5:2) this effect keeps pushing state `n` (its deps do not provably gate
    ↳ it, so the effect can re-run every render) to new values on every run. Potential infinite render loop
      (2 trace step(s), rerun with --trace)

! 1 warning(s) across 1 file(s).

```

Le niveau observé est WARNING, pas ERROR : les deps [n] ne sont pas *prouvablement* une garde (chapitre 19 explique la distinction entre le bras *déps* et le bras *churn* d'infinite-loop).

Refusé. Un opérateur de réduction inter-sortes, ou un « query pool » n -aire à la MOPSA (repris d'ADR-7) : « the slots describe disjoint kinds », rien à réduire puisque rien ne se recouvre.

Conséquences. `src/domains/impls/state_value.rs` (la struct produit, les opérations ponctuelles, `to_stability` « motion-wins »); `src/domains/impls/setter_val.rs` (SetterVal, l'ancienne variante ComponentSetter d'ADR-12 §7 devenue une sorte à treillis plat); `src/ir/hooks.rs` : `HookEntry::State` perd son `type_hint`. Un ancien FN devient une détection épinglée

(tests/narrowing.rs, l. 221–221null_init_without_hint_unbounded_is_flagged); un non-FP est épinglé symétriquement (tests/narrowing.rs, l. 326–326nullable_fetch_pattern_no_false_positive). Commit c32e1b0 : 40 fichiers, +1 674/−1 198 lignes.

ADR liés. Supersedes ADR-8; KindMask (state_value.rs, « the product’s kind enumeration lives in exactly one place ») est un apport ultérieur d’ADR-20 (item D4, section 26.5); ADR-20 item 10 interdit d’y câbler TSType.

Fichiers. src/domains/impls/state_value.rs; src/domains/impls/setter_val.rs; src/ir/hooks.rs.

Chapitre. chapitre 10.

A.16 ADR-016 — CLI, sortie JSON, détection de projet Vite

15 juillet 2026. Statut : Implemented.

Contexte. Le binaire dupliquait le pipeline *sans* le résolveur d’imports (un faux négatif silencieux sur tout code inatteignable depuis un point d’entrée mal détecté); il n’y avait pas de sortie machine; les projets Vite n’étaient pas analysables (structure de répertoires non reconnue).

Décision. Des sous-commandes (check par défaut, rules, explain); un pipeline *unique* partagé (lower_files, analyze_lowered) entre le binaire et les tests; les métadonnées des règles indexées par nom de *diagnostic* (onze structures de règles émettaient treize noms distincts à l’époque); un DTO JSON côté binaire; un contrat de codes de sortie 0/1/2 (succès, warning-seul, error); ProjectKind::{Plain, Vite}; support tsconfig JSONC avec extends et references.

Refusé. Évaluer vite.config.* pour résoudre les alias : « a best-effort regex would risk *wrong* resolutions, i.e. silent FNs » — le risque de mal résoudre un chemin (et donc de rater du code) l’emporte sur le confort d’une détection plus riche.

Conséquences. Le schéma JSON est passé en v2 depuis (#129, section 26.10).

ADR liés. Le schéma JSON évolue par la suite (v2, #129) sans ADR dédié (correction de précision de l’ergonomie, pas de la soundness).

Fichiers. src/driver/mod.rs, l. 106–106 (run_check); src/driver/json.rs, src/driver/human.rs.

Chapitre. chapitre 23.

A.17 ADR-017 — Stabilité versionnée

15 juillet 2026. Statut : Implemented.

Contexte. Un couple faux positif/faux négatif noué ensemble : un état objet stable en pratique (recréé seulement au setState) était lu comme Unstable, produisant des FP « always-unstable-deps » sur les context providers du corpus *memos*; et « any FP fix that silences that warning without a replacement signal silently drops a true infinite loop » — l’ObjChurn correspondant restait invisible.

Décision. Introduire des bornes *may* (« ne change qu’aux set ») et *must* (« change à chaque rendu »), au lieu du seul point Unstable.

Extrait A.5 — src/domains/impls/stability.rs, l. 35–64 — Stability — le treillis versionné : Versioned/VersionedTop (may) et PerRender (must)

```
35 #[derive(Debug, Clone, PartialEq, Eq)]
36 pub enum Stability {
37     /// ⊥ no information (unreachable path / uninitialized).
38     Bottom,
```

```

39    /// Never changes: the same reference on every render (safe as a dep).
40    Stable,
41    /// Changes only at setter events of these state slots (may bound).
42    /// Invariant: non-empty (canonicalised — `Versioned(∅) ≡ Stable`) and
43    /// `len() ≤ VERSIONED_LABELS_THRESHOLD` (widened to `VersionedTop` above).
44    Versioned(BTreeSet<QualifiedSlot>),
45    /// Versioned by unknown state slots (threshold-widened `Versioned`).
46    VersionedTop,
47    /// A fresh reference every render, guaranteed (must bound).
48    /// For non-reference kinds via `to_stability`: "may change every render".
49    PerRender,
50    /// T no bound in either direction.
51    Unknown,
52 }
53
54 impl Stability {
55     /// Canonicalising constructor:  $\emptyset \rightarrow \text{`Stable`}$ , over-threshold  $\rightarrow \text{`VersionedTop`}$ .
56     pub fn versioned(labels: BTreeSet<QualifiedSlot>) -> Self {
57         if labels.is_empty() {
58             Stability::Stable
59         } else if labels.len() > VERSIONED_LABELS_THRESHOLD {
60             Stability::VersionedTop
61         } else {
62             Stability::Versioned(labels)
63         }
64     }

```

La conversion `Versioned` $\rightarrow$ lecture d'état se fait **côté lecture**, en un seul endroit du transfert (le bras `Expr::StateVal(1)`) : le store garde une vue *événementielle* de l'état (une valeur par `set`), l'évaluation en tire une vue *inter-rendus*. Un nouveau bras « churn » d'infinite-loop est stratifié `ERROR/WARNING` en plus du bras « deps » historique.

Refusé. Un compteur d'événements pour faire diverger artificiellement le store à chaque `set` : « a non-standard hack », alors que la certitude doit venir de la structure des dépendances, pas d'un artifice de comptage. Deux points intermédiaires du produit complet (*may*-seul, *must*-seul) ont aussi été envisagés puis écartés au profit du couple complet.

Couplage de soundness. Le *gating* par `Versioned` n'est sound *qu'avec* le bras churn actif : sans lui, un état « seulement *may*-changeant » laisserait passer un cas qui change en réalité à chaque rendu.

Conséquences. Cinq FP « memos » supprimés ; les bugs de la classe `ObjChurn` passent de « noyés dans le bruit » à `ERROR` propre. `Stability` perd `Copy` (elle porte désormais un ensemble) : « the compile errors force an exhaustive audit of every consumer — desirable for a semantics change ». Commit `f31acae` : 26 fichiers, +2364 lignes.

ADR liés. Raffine `ADR-2` ; couplé au graphe de churn d'`ADR-18` pour la soundness du *gating*.

Fichiers. `src/domains/impls/stability.rs`.

Chapitre. chapitre 10 et chapitre 19 (le bras churn).

A.18 ADR-018 — Graphe de churn multi-effets

16 juillet 2026. Statut : *Implemented* ; amendé par *ADR-42*.

Contexte. Une boucle répartie sur *deux* effets — l'effet A écrit un slot lu par l'effet B, qui écrit à son tour un slot lu par A — est invisible à un bras *infinite-loop* qui ne regarde qu'un effet à la fois.

Décision. Un graphe sur les slots qualifiés (`component`, `label`) ; une arête « un changement de `x` relance un effet qui stocke une référence fraîche dans `y` » ; une arête *must* exige une dépendance exacte *et* une fraîcheur (`Fresh`) sur tous les chemins ; une auto-arête pour les effets sans tableau de deps ; une « convergence kill » conditionnée à un **écrivain unique** du slot ; l'algorithme de Tarjan en deux passes pour les composantes fortement connexes. Le graphe est construit une fois par programme (résolvant un problème de performance quadratique, #86) ; les cycles inter-composants sont plafonnés à `WARNING`.

Refusé. Rien de débattu au-delà de la construction retenue ; la condition d'écrivain unique est une simplification *assumée*, pas débattue ici — elle sera remplacée plus tard.

Conséquences. Cycles objets sur deux ou N effets : `INFO` → `ERROR` avec le chemin du cycle. Écritures fraîches sans tableau de deps (l'oubli classique) : `silence` → `ERROR`. Churn objet inter-composants (sous condition `Versioned`) : `silence` → `WARNING`. Corpus (4 dépôts) : inchangé, aucun nouveau FP.

ADR liés. Amendé par ADR-42 §4 et §6 : la construction migre dans le moteur, et la condition d'écrivain unique est remplacée par la preuve multi-sites de l'issue #154.

Fichiers. À e67b10a, le graphe vit dans `src/engine/churn.rs` (déplacé depuis `rules/helpers/` par ADR-42 — voir cette fiche).

Chapitre. chapitre 15.

A.19 ADR-019 — Chaînes de témoins typées

17 juillet 2026. Statut : *Implemented* ; *supersedes ADR-11 §Note*.

Contexte. Les notes de diagnostic étaient du texte libre, sans fichier associé, et la provenance d'un fait (pourquoi le moteur croit-il cela ?) était jetée dès qu'elle avait servi.

Décision. `FileId/FileTable` ; la provenance est enregistrée « au point de connaissance » (`widen_trace`, `inline_origins`) plutôt que reconstruite après coup ; un vocabulaire *fermé* `Step` (neuf variantes à l'origine : `Binding`, `Resolve`, `Call`, `Write`, `Read`, `Branch`, `Handler`, `CycleEdge`, `Widen`), **sans** variante `Text(String)` ; un rendu centralisé, une bibliothèque de témoins partagée entre règles ; des bornes (un niveau de résolution, huit étapes maximum).

Refusé. Un domaine de provenance complet, à la MOPSA : « touches every transfer function and multiplies memory for witness depth no rule needs ». Une variante `Step::Text(String)` : « a free-text escape hatch would erode the vocabulary back to prose within months ».

Conséquences. À e67b10a, l'enum `Step` compte cinq variantes de plus (`Mutate`, `Capture`, `InitOnce`, `Forward`, `Rerender`) : le vocabulaire a crû par la voie sanctionnée — l'extension de l'enum — jamais par une variante texte libre. Commit 146a86a : 55 fichiers, +1 737/−322 lignes ; tous les littéraux `SourceRange` du dépôt sont touchés.

ADR liés. Supersedes ADR-11 §Note.

Fichiers. `src/rules/api/witness.rs` (enum `Step`, ligne 86).

Chapitre. chapitre 17.

A.20 ADR-020 — Dette technique : onze non-changements délibérés

23 juillet 2026. Statut : *Accepted*.

Contexte. Une campagne d'audit sur dix-huit « workarounds » suspects et trente-six constats d'architecture a trouvé que beaucoup de « duplications » apparentes étaient en réalité des variations porteuses de soundness — les retirer aurait introduit un faux négatif.

Décision. Une liste de changements réellement appliqués (splice unifié avec α -renommage, `recompute_memo` corrigé pour utiliser les vrais stores, vocabulaire churn hissé dans un module partagé, une sentinelle opaque typée, la macro `flat_lattice!`, `KindMask...`) et **onze non-changements**, chacun avec sa phrase de justification (reproduits en détail au section 26.5, tableau 26.3) : garder le diamant `&&/||` plutôt qu'un `LogicalOp` plat ; garder les deux bras du churn séparés ; garder `may_written_slots` syntaxique ; garder la projection `to_stability` ; ne pas généraliser en `BoundedPowerset<T,N>` ; ne pas partager une résolution de composant unique entre règles ; garder séparé le chemin « position de retour » ; garder le seed de tas par site d'appel ; garder `resolve_setter_aliases` comme passe séparée ; ne pas câbler `TSType` dans le domaine ; garder les noms temporaires par *offset*.

Refusé. Chacun des onze changements listés ci-dessus, précisément parce qu'il coûtait un faux négatif identifiable sur un programme concret.

Conséquences. `CLAUDE.md` cite désormais cet ADR comme lecture obligatoire avant toute nouvelle tentative de « nettoyage ». La leçon est répétée mot pour mot dans le dépôt : « map before “fixing” ; never introduce a false negative to remove a duplication ».

ADR liés. Réinvoqué explicitement par ADR-29 (item 2, refus de fusionner les deux bras churn) et ADR-31 (item 3, `may_written_slots`).

Fichiers. Aucun fichier propre : c'est un ADR de non-changement, sa trace est dans `docs/adr/ADR-020-tech-debt-clean-up-decisions.md` lui-même.

Chapitre. chapitre 26, section 26.5 (table complète des onze non-changements).

A.21 ADR-021 — Surface de requêtes typée

24 juillet 2026. Statut : Accepted — implemented.

Contexte. Trois défauts distincts : un FN réel dans un *helper* partagé (`is_unstable` n'était pas exactement le complément de `is_stable`) ; `Severity::Error` attachable en pratique à n'importe quel booléen, sans lien démontrable avec une preuve `must` ; le « must-forward » (propager une preuve certaine à travers un appel) dupliqué dans chaque règle qui en avait besoin. Étude préalable : cinq *frontends* de règles prototypés, avec la conclusion « the frontend is not the soundness lever — the query surface is ».

Décision. La polarité devient un **type** plutôt qu'une convention :

Extrait A.6 – `src/rules/api/query.rs`, l. 111–129 — `MustResult<T>` — le verdict `must` à trois valeurs, et `May<T>`, sans chemin vers une `ERROR`

```
111 /// A three-valued MUST verdict. `All` carries the certified token (the only way
112 /// to obtain a `Certified` for a single-verdict primitive); `Some`/`None` are
113 /// MAY facts with no path to an `Error`.
114 ///
115 /// (Deviates from the ADR-021 §1 literal `All(T)`: the token lives *inside* `All`
116 /// so `must_*` and the `Vec<Certified<T>>` primitives share one minting story.)
117 #[derive(Debug, Clone, PartialEq)]
118 pub enum MustResult<T> {
119     /// Proven on **all** paths – carries the minted proof.
120     All(Certified<T>),
121     /// Proven on **some** but not all paths – a MAY fact (raw payload).
122     Some(T),
123     /// No qualifying evidence at all.
```

```

124     None,
125 }
126
127 /// A MAY fact. There is no path from `May<_>` to an `Error`.
128 #[derive(Debug, Clone, Copy, PartialEq, Eq)]
129 pub struct May<T>(pub T);

```

Diagnostic::error n'est atteignable que depuis un Certified :

Extrait A.7 – src/rules/api/query.rs, l. 74–84 — Certified<E> — le jeton scellé : constructeur privé au module

```

74 /// A certified MUST fact: the evidence plus its provenance.
75 ///
76 /// The constructor (`Certified::mint`) is private to this module. Rule code in
77 /// sibling modules can only [`Certified::evidence`]/[`Certified::provenance`] –
78 /// it can never forge a token. This is the enforcement: an `Error` is reachable
79 /// only from a `Certified`, and a `Certified` only from a must-primitive here.
80 #[derive(Debug, Clone, PartialEq)]
81 pub struct Certified<E> {
82     evidence: E,
83     provenance: Provenance,
84 }

```

Des primitives « graines » produisent les premiers Certified; RuleCtx regroupe l'accès aux relations. Migration « big-bang » des quatorze règles existantes en un seul changement : « a transition window would leave the FN vector open » — laisser cohabiter ancienne et nouvelle API, même temporairement, aurait laissé un chemin non vérifié vers un faux négatif.

Piège

La première version du scellement n'était *pas* étanche : la visibilité Rust étant descendante (un module voit ses parents publics), Diagnostic::new restait appellable depuis les sous-modules de règles, et un champ pub severity permettait de *forger* une Error sans passer par un Certified. Le « hardening round » a déplacé Diagnostic dans un module feuille dédié, src/rules/api/diagnostic.rs (commit eb5cb93), pour qu'aucun module frère ne puisse plus le construire directement. Retenir la leçon : un scellement de type ne protège que ce que la visibilité du module protège réellement.

Refusé. Une fenêtre de transition où l'ancienne signature (&AnalysisResult) -> Vec<Warning> resterait disponible en parallèle de la nouvelle : elle laisserait ouvert le vecteur de FN le temps de la migration.

Conséquences. Positif : aucune Error-sur-may ni aucun abandon silencieux de T ne survit à la compilation ; le must-forward est partagé et à trois valeurs ; la surface de requêtes devient l'ancre stable sur laquelle le futur *frontend* de règles se lie ; le FN vérifié est corrigé et épinglé par un test. **Coût** : les quatorze règles et la signature du trait migrent en un seul changement ; la rigueur de relecture exigée sur les annotations de polarité augmente.

ADR liés. Remplace la signature d'ADR-6 ; un amendement du 1^{er} septembre introduit ProgramCache, repris ensuite par ADR-42 §5.

Fichiers. src/rules/api/query.rs ; src/rules/api/diagnostic.rs.

Chapitre. chapitre 17.

A.22 ADR-022 — Frontends de règles, packs déclaratifs, WASM

25 juillet 2026. Statut : *Accepted*, §7 superseded by ADR-23 §5, §4 amendé par ADR-41 §4.

Contexte. Les cinq questions laissées ouvertes par l'étude préalable d'ADR-21 : comment une équipe écrit-elle sa propre règle sans toucher au moteur ni en perdre la soundness ?

Décision. Principe normatif : « reactant does *semantics only* » — une règle personnalisée ne lit jamais l'AST ni le texte source, seulement des relations sémantiques déjà résolues par le moteur (les « ancrs » Tier-A). Une règle est une ancre plus une navigation typée dans les relations qu'elle expose. La sévérité est $\sqcap$ polarity — le minimum entre un plafond choisi par l'auteur de la règle et la polarité du fait lui-même — évaluée **par finding**, ce qui donne la stratification ERROR/WARNING « gratuitement » (sans code supplémentaire par règle). Paramètres restreints à une position de constante feuille (jamais une expression arbitraire). Espace de noms pack/rule ; documentation obligatoire par règle ; distribution **WASM-only** sur npm ; le cœur Rust revalide systématiquement tout pack chargé, même compilé — « the host JS is not a trust boundary ».

Refusé. Le rejet statique des conflits pin/polarity : il aurait interdit la stratification par finding, qui est précisément ce que le $\sqcap$ rend possible. Les callbacks JS bruts avec autorité d'ERROR : un callback JS ne peut jamais être certifié par construction.

Conséquences. Positif : les règles personnalisées héritent de la soundness du moteur par construction ; un seul mécanisme de sévérité ($\sqcap$) sert auteurs de règles, consommateurs et règles natives ; le schéma JSON dérive de la surface api/, donc la croissance du vocabulaire Tier-C se propage gratuitement ; WASM-only garantit un comportement identique partout. **Coût** : il faut construire une couche « entité-arête » par-dessus api/ (les primitives prennent des types IR bruts — &CFG, &Expr — non JSON-appelables directement) ; les messages d'erreur du validateur doivent être assez bons pour nourrir une boucle d'auteur assistée par un LLM.

ADR liés. §7 (une syntaxe déclarative en Starlark) superseded by ADR-23 §5 ; §4 (la portée des options de règles) amendé par ADR-41 §4.

Fichiers. src/rules/api/ (surface) ; distribution WASM (chapitre 24).

Chapitre. chapitre 22 et chapitre 24.

A.23 ADR-023 — Croissance du vocabulaire Tier-A

26 juillet 2026. Statut : *Accepted* ; supersedes ADR-22 §7.

Contexte. Mesuré sur le catalogue : seules 3 règles sur 21 étaient alors exprimables avec les seules ancrs et primitives Tier-A disponibles.

Décision. Croître le vocabulaire par **entités** — des positions dans des relations déjà résolues — plutôt que par des gardes ad hoc. Un verdict doit se lire *au point de programme de son entité* (sinon c'est un nouveau fait non prouvé, donc un FN potentiel). Première entité : args, avec une primitive returns_verdict — amendée le lendemain pour être calculée **pendant** le point fixe, car le contexte nécessaire ne survit pas à la post-passe.

Refusé. Le quantificateur universel $\forall$ sur les éléments d'un tableau : un $\top$ y plierait en « viole », et une vacuité sur une arête tronquée serait un faux positif silencieux. **Amendement du 1^{er} septembre** : la condition est levée par un bit exact porté par les littéraux ArrayLit ; la primitive every est livrée, mais seulement au positif, sans autorité d'ERROR. Starlark comme langage de règles déclaratives est rejeté : il ne compile pas nativement dans l'arbre de dépendances du projet, qui a déjà oxc sous la main pour parser du JS/TS et le compiler en JSON.

Conséquences. Le catalogue passe de 5/21 à 8/22 (le dénominateur change avec l’entité args), puis continue de croître ADR après ADR jusqu’à 21/22 (ADR-34).

ADR liés. Supersedes ADR-22 §7 (Starlark) ; le refus du quantificateur $\forall$ est partiellement levé par l’amendement du 1^{er} septembre.

Fichiers. src/rules/api/ (entité args) ; le calcul de returns_verdict vit dans le point fixe (src/engine/fixpoint.rs, l. 130–130analyze_component_impl).

Chapitre. chapitre 22.

A.24 ADR-024 — Attribution des findings inlinés

26 juillet 2026. Statut : Accepted.

Contexte. Sur un échantillon de packs, 12 findings sur 27 (44 %) s’affichaient sous le fichier du composant *consommateur* avec la ligne du hook *inliné* — une attribution qui pointait le mauvais fichier pour un contributeur qui voudrait corriger le hook lui-même.

Décision. La ligne primaire d’un finding rend l’**origine** (le fichier du hook) quand elle diffère du fichier consommateur ; en JSON, le champ *file* devient le fichier de l’ancre plutôt que celui du composant affiché.

Refusé. Dédupliquer les findings entre consommateurs d’un même hook inliné (useStep(1) déclenchant infinite-loop et useStep(0) déclenchant redundant-set-state : ce sont des faits *incomparables*, fusionner perdrait de l’information). Un groupement d’affichage différé : repris plus tard, autrement, par l’issue #129, qui groupe seulement à l’*affichage* tout en gardant une ligne par composant en JSON.

Conséquences. Aucune régression du nombre de findings ; l’attribution change, pas le compte.

ADR liés. Le groupement d’affichage refusé ici revient sous une forme différente avec l’issue #129.

Fichiers. src/driver/json.rs, src/driver/human.rs.

Chapitre. chapitre 17.

A.25 ADR-025 — Fall-through = return undefined

29 juillet 2026. Statut : Accepted.

Contexte. Terminator::Unreachable confondait trois situations distinctes : la fin normale d’un corps de fonction sans return explicite, un throw, et un break orphelin. Le *splice* (greffe d’un CFG dans un autre, chapitre 8) laissait alors le bloc de jointure sans prédécesseur valide. Mesuré : 208 composants « sectionnés » (l’environnement de sortie, exit_env, incomplet — un faux négatif).

Décision. Un corps qui « tombe » à sa fin sans return explicite est scellé Return(Lit(Unit)) — un return undefined implicite, exactement la sémantique JS. throw garde Unreachable. ExitDominance devient le seul propriétaire de l’ensemble des sorties **atteignables** d’un CFG.

Extrait A.8 – src/ir/cfg.rs, l. 12–30 — Terminator — Return sans span, Unreachable réservé à l’arrêt de contrôle

```
12 #[derive(Debug, Clone)]
13 pub enum Terminator {
14     Jump(BlockId),
15     Branch {
```

```

16     cond: Expr,
17     then_: BlockId,
18     else_: BlockId,
19     /// Where the condition is evaluated in the source (None for
20     /// synthetic branches and manual-IR tests).
21     span: Option<crate::ir::SourceRange>,
22 },
23 /// Unlike [Terminator::Branch], this carries no span: nothing needed the
24 /// position of a return until a hook could be extracted from one (#4),
25 /// and adding it now is a 40-site IR change tracked separately. The cost is
26 /// that a hook reached only through a return yields findings with no line
27 /// number – visible but unlocated, which is still strictly better than the
28 /// silence it replaced.
29 Return(Expr),
30 Unreachable,

```

Piège

Le champ `span` de `Terminator::Return` est `None` par construction (contrairement à `Branch`, qui en porte un) : « nothing needed the position of a return until a hook could be extracted from one ». Un hook atteint seulement par un chemin qui finit sur un `return` produira donc un finding *sans numéro de ligne* — visible, mais non localisé. C'est un choix assumé (issue #140 encore ouverte), pas un oubli. Le commentaire du code chiffre ce coût à « a 40-site IR change » ; six semaines plus tard, refusant le même ajout pour une autre raison (nommer l'appelant plutôt que l'appelé lors d'un *splice*), ADR-39 le chiffre à « a hundred construction sites ». Les deux estimations, écrites à deux moments par le même changement refusé, ne se recoupent pas — signe qu'il s'agit d'ordres de grandeur à main levée, pas d'une mesure, et qu'aucune des deux ne doit être citée comme un chiffre d'audit.

Refusé — mesuré. Relier tout `Unreachable` au bloc de jointure : cela aurait fait naître une `ERROR conditional-hook` sur l'idiome courant « garde puis `throw` » :

Exemple A.2: L'idiome guard-throw, non signalé

Extrait A.9 – `useCart` : garde par exception avant les hooks

```

1 function useCart(items: unknown[]) {
2   if (!items) {
3     throw new Error("items required");
4   }
5   const [open, setOpen] = useState(false);
6   return { open, setOpen };
7 }

```

Sortie observée (e67b10a) : *aucun* finding.

```

1 clean component(s) hidden, rerun with --show-clean

✓ 1 file(s) no issues found.

```

Relier le `throw` au `join` aurait rendu `useState` atteignable par deux chemins de hauteur différente dans l'arbre des appels de hooks, un motif que `conditional-hook` confondrait avec un hook réellement conditionnel.

Conséquences. Douze findings révélés, zéro perdu ; les composants « sectionnés » tombent de 208 à 3. Commit 34ce48b : 7 fichiers, +395 lignes.

ADR liés. Aucun amendement direct ; ADR-39 révisé le calcul des positions manquantes en aval.

Fichiers. `src/ir/cfg.rs` (Terminator); `src/engine/dominance.rs` (ExitDominance).

Chapitre. chapitre 7 et chapitre 12.

A.26 ADR-026 — Projets Next.js

27 août 2026. Statut : *Implemented*.

Contexte. Les projets Next.js n'étaient analysés « only by accident » (aucune reconnaissance de leur structure). Question de fond : que signifie un *Server Component* pour un analyseur bâti sur le rendu/l'état/les effets, des notions qui n'existent pas côté serveur ?

Décision. Ne pas sauter les modules serveur — sauter sur la base d'arêtes incomplètes du graphe d'imports serait un FN. `ModuleFacts { directives, imports }` : les directives (`"use client"`, `"use server"`) et les imports sont enregistrés *sans être interprétés* comme une preuve de domaine; `reachable_from` calcule l'atteignabilité dans ce graphe. `ProjectKind::NextJs` testé avant vite (un projet Next.js a aussi un `vite.config`, rarement, mais la priorité doit être sans ambiguïté). `baseUrl` comme résolveur de chemins additionnel. La règle `server-component-hook` est plafonnée WARNING : les faits qu'elle utilise (convention de nommage, graphe d'imports) sont hors du domaine sémantique de l'analyse. Des résumés pour `next/navigation`.

Refusé. Une *must-primitive* certifiant qu'un module est un Server Component : « would dress a path convention as a domain proof » — la convention `"use client"` n'est pas un fait sémantique prouvé par interprétation abstraite, seulement une convention de fichier. Supprimer les autres findings dans un module serveur : cela masquerait de vrais bugs d'état/effets dans le code qui s'exécute malgré tout côté serveur au premier rendu.

Conséquences. Zéro FP sur cinq corpus Next.js testés; 46 INFO `unknown-hook` supprimés (les hooks de Next.js désormais reconnus); quatre findings révélés par la nouvelle règle.

ADR liés. Aucun amendement.

Fichiers. `src/resolver/mod.rs` (`ProjectKind::NextJs`); la règle `server-component-hook`.

Chapitre. chapitre 21 et chapitre 23.

A.27 ADR-027 — Relation slot-writer, résumés de phase, provenance

1^{er} septembre 2026. Statut : *Accepted*, §2 amendé par ADR-34 et ADR-35.

Contexte. Des règles de type « wrapper enforcement » (« l'état ne s'écrit que via `putState` », une politique d'équipe) étaient inexprimables : rien ne disait où, ni dans quelle phase, un slot était écrit. Un audit du code contre les ADR a motivé une relation dédiée.

Décision. Une ligne d'écrivain à deux colonnes distinctes : *region* (*exacte* — `Render`, `Effect`, etc.) et *phase* (*may*, `⊤` pour un `FnLit` imbriqué non encore résolu).

Extrait A.10 – `src/engine/setters.rs`, l. 681–701 — `WriterPhase` — la phase d'exécution d'une écriture, un verdict `may`

```
681 /// Execution phase of a write – a MAY verdict (ADR-027 §1): a write
682 /// synchronous in its body carries that body's phase; a write inside a
683 /// nested `FnLit` is `Unknown` (⊤ – it may run in any phase) until a callee
684 /// summary sharpens it (ADR-027 §2). Classifying every nested callback as
685 /// "deferred" instead would under-approximate: `arr.forEach(x => setX(x))`
686 /// inside an effect runs synchronously in the effect phase.
687 #[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord)]
688 pub enum WriterPhase {
```

```

689   Render,
690   Effect,
691   Memo,
692   Callback,
693   Handler,
694   /// The callee summary proved deferral (ADR-027 §2): a timer, a microtask,
695   /// a promise continuation – the write never runs inside a React phase.
696   Deferred,
697   /// An effect's returned cleanup function.
698   Cleanup,
699   /// T – the write may run in any phase. Satisfies every phase query.
700   Unknown,
701 }

```

La relation est calculée à *convergence* du point fixe et **déplacée des règles vers le moteur** (première étape du fil rouge « relations = produits du moteur », section 26.7). Les résumés de phase des appelés sont *whitelisted* avant que le vocabulaire ne soit publié — affiner $\top$ après coup changerait des findings déjà livrés, ce qui violerait la stabilité du catalogue. Résolution d'imports des fonctions utilitaires « fail-closed » (import non résolu $\rightarrow \top$). Provenance enregistrée au site unique du splice. `must_direct_write` donne l'autorité d'ERROR aux règles de politique d'équipe. Catalogue re-basé à 22 entrées.

Refusé. Classifier tout `FnLit` imbriqué comme `Deferred` plutôt que `T : arr.forEach(x => setX(x))` à l'intérieur d'un effet s'exécute *synchrone* dans la phase de l'effet, pas « en différé » — classifier systématiquement en `Deferred` sous-approximerait.

Conséquences. La note d'ADR-23 §4 sur l'ordonnancement de `any_of/eval_guard` est déclarée périmée. Le test-cliquet `catalogue_has_21_entries` est remplacé par le pin à 22 entrées quand §6 atterrit. `docs/custom-rules.md` et les compétences `skills/reactant-rules/` gagnent un test anti-dérive.

ADR liés. §2 (résumés de phase, `await`) amendé par ADR-34 et ADR-35, qui lèvent les points laissés en suspens.

Fichiers. `src/engine/setters.rs` (`WriterPhase`, tête de `SlotWriter`).

Chapitre. chapitre 14.

A.28 ADR-028 — writers par site, Updater, same_tick

1^{er} septembre 2026. Statut : *Accepted*.

Contexte. `stale-update` (deux écritures non fonctionnelles du même slot dans un même tick d'événement, où seule la dernière compte) était inexprimable : la relation d'écriture précédente effondrait plusieurs appels en une seule ligne.

Décision. Renverser l'effondrement documenté par la ligne précédente : une ligne par *site d'appel*, pas par région.

Extrait A.11 – `src/engine/setters.rs`, l. 728–745 — `SlotWriter` — une ligne par site, et pourquoi le renversement est monotone

```

728 /// One writer of a state slot, one row **per call site**.
729 ///
730 /// The granularity used to be one row per (region, alias-resolved setter
731 /// variable, phase class), collapsing two `setCount(count + 1)` calls in one
732 /// body into a single row. That is reversed here, and the reversal is the
733 /// point: the canonical stale-read shape is *two* non-functional writes of one
734 /// slot in one handler, and a relation that cannot say there are two cannot
735 /// express it. Multiplying rows is monotone – same slots, same phases, more

```



```

736 /// witnesses – and every shipped consumer of the stored relation reads it
737 /// existentially, so nothing that matched before stops matching (ADR-028 §2).
738 #[derive(Debug, Clone)]
739 pub struct SlotWriter {
740     pub slot: HookLabel,
741     /// The setter variable at the call site (an alias chain resolves it to
742     /// `slot`; a spliced wrapper's `setter#salt` param resolves too).
743     pub setter: Var,
744     /// Witness call-site span.
745     pub span: Option<SourceRange>,

```

Une **unique** colonne Updater d'où chaque consommateur dérive son propre verdict (fonctionnel ou non, pureté du corps de l'Updater) plutôt que plusieurs colonnes spécialisées. Un booléen `same_tick` par ligne, jamais un pli agrégé — la relation « deux écritures dans le même tick » reste *may* par construction.

Refusé. Deux colonnes spécifiques au lieu d'un Updater partagé : aurait dupliqué la même information sous deux formes qui pourraient diverger. Une forme *niée* de `same_tick` (« pas dans le même tick ») : la relation est *may*, sa négation ne serait pas *must* et n'apporterait rien.

Conséquences. Le catalogue Tier-A passe de 14/22 à 16/22 ; #61 reste ouverte (elle ne couvre que la moitié asynchrone du problème).

ADR liés. L'alternative à deux colonnes, refusée ici, est explicitement rappelée par ADR-27 §4 (« one central relation, never a second bespoke one »).

Fichiers. `src/engine/setters.rs`.

Chapitre. chapitre 14.

A.29 ADR-029 — Ancre churn_cycles

1^{er} septembre 2026. Statut : Accepted, §1 amendé par ADR-42.

Contexte. `cross-component-effect-cycle` était bloquée par l'issue #68 : le catalogue Tier-A n'acceptait alors qu'une seule ancre par règle, et un cycle de churn est une propriété du *programme*, pas d'un composant.

Décision. Projeter la relation *programme* (le graphe de churn) sur le composant « ancré » : une ligne par arête de cycle portée par l'un de ses effets. Les colonnes sont booléennes et *exactes* — le caractère *may* du fait reste dans le graphe lui-même, pas dans la ligne projetée. Une arête sans position source (*span*) ne produit pas de ligne.

Refusé. Un statut « couvert nativement, ne compte pas » pour sortir cette classe de bug du calcul de couverture Tier-A : la mesure porte sur le *vocabulaire* disponible aux règles personnalisées, pas sur ce que les règles natives couvrent déjà par ailleurs. Fusionner les deux bras du churn (repris et refusé par ADR-20 item 2).

Conséquences. Le catalogue passe de 16/22 à 17/22 ; l'issue #68 cesse d'être bloquante pour cette classe de règles.

ADR liés. §1 (la construction du graphe) amendé par ADR-42, qui la déplace dans le moteur.

Fichiers. L'ancre `churn_cycles` lit `src/engine/churn.rs` (après le déplacement d'ADR-42 ; côté règles à l'époque de cet ADR).

Chapitre. chapitre 15.

A.30 ADR-030 — Lignes render-setter qualifiées par propriétaire

1^{er} septembre 2026. Statut : Accepted.

Contexte. `setter-called-in-child-render` était traitée comme une jointure ad hoc entre deux relations, au lieu de lire une relation dédiée.

Décision. Des lignes « étrangères » dérivées de la résolution `cross_component_setters` déjà existante, avec une colonne `owner` (le composant *propriétaire* du slot, pas celui qui appelle le setter). L'énumération des cas s'élargit *sur la garde* qui sélectionne les lignes (`slot_ownership`), jamais *sur la sorte* de la ligne elle-même — élargir la sorte changerait la forme des findings déjà livrés par les packs. Le nom du slot étranger est affiché dans le composant **propriétaire**, pas dans celui qui le lit.

Refusé. Élargir la sorte de ligne inconditionnellement : changerait les findings des packs déjà distribués. Restreindre le scan au seul bloc du site d'appel : supprimerait des findings dans les cas où l'environnement abstrait est imprécis — un coût en soundness pour un gain de bruit seulement.

Conséquences. Le catalogue passe de 17/22 à 18/22. Aveu explicite : l'attribution du propriétaire n'était pas exacte au moment de cet ADR (issue #119, ouverte puis corrigée par le commit `ce3b080`).

ADR liés. Aucun amendement direct au-delà de la correction `ce3b080`.

Fichiers. `src/engine/setters.rs` (colonne `owner`, voir aussi ADR-42 §2, où elle est formalisée sur `SlotWriter`).

Chapitre. chapitre 14.

A.31 ADR-031 — Relation `slot_seeds`

1^{er} septembre 2026. Statut : Accepted, §2 amendé par l'issue #121 et ADR-33.

Contexte. `state-mirrors-prop-without-sync` était vue comme une jointure ad hoc entre une prop et un slot d'état, sans relation propre pour la relier.

Décision. « A fold promoted to the engine » : les fonctions utilitaires de `frozen_initial_state.rs` migrent dans `engine/seeds.rs`. La moitié « effet » de la relation est **dérivée** de `slot_writers` (aucun second parcours du CFG) ; la moitié « render » lit la **phase prouvée** du moteur, pas la région lexicale du code source. L'échappement d'une prop hors de sa synchro initiale est représenté comme une *colonne* de la relation, jamais comme un verdict de synchronisation déjà tranché.

Refusé. Plier l'échappement dans un champ `synced` binaire : effacerait la distinction précise dont dépend le palier `ERROR` de la règle.

Piège

La première mesure de cet ADR affirmait une sortie native « inchangée sur 14 corpus ». C'était **faux** : un seul run avait été fait par côté de la comparaison, alors que l'analyse n'est pas rigoureusement déterministe à ce stade (issue #120, corrigée ensuite par ADR-33). Recompté honnêtement : 12 corpus identiques, `twenty` −1 (un FP retiré, une amélioration), `mantine` −1 (un *vrai* positif perdu par accident, restauré ensuite par ADR-33). La leçon : une mesure « inchangé » sur un pipeline non déterministe doit être répétée avant d'être publiée.

Conséquences. Amendement de l'issue #121 : un filtre `effect_triggered(phase)` affine la moitié effet.

ADR liés. §2 amendé par l'issue #121 ; le déterminisme requis pour la mesure est livré par ADR-33.

Fichiers. `src/engine/seeds.rs`.

Chapitre. chapitre 14.

A.32 ADR-032 — Relation context_consumers

1^{er} septembre 2026. Statut : Accepted.

Contexte. `consumer-without-provider` était bloquée sur l'issue #28 (comment modéliser `useContext` sans domaine dédié à part entière?).

Décision. Une post-passe purement diagnostique : le verdict de cette règle est une **absence** (aucun provider trouvé), donc toute la conception se ramène à une porte d'ascendance fiable. *Gate 1* : ascendance complète prouvée dans l'arbre d'éléments (issue #110). *Gate 2* : une passe syntaxique de complétion pour les composants non atteints par l'ascendance sémantique. Appariement sur la cellule canonique `ContextId` (issue #109). Cas toléré : le provider d'un contexte compte lui-même comme un « hit » pour son propre `useContext` éventuel. Le verdict est nommé `none-on-analyzed-paths`, délibérément pas `no-provider` — le nom dit la limite de l'analyse, pas une absence prouvée dans l'absolu.

Refusé. Rien d'alternatif débattu pour la structure des deux portes.

Conséquences. Le catalogue passe de 19/22 à 20/22.

ADR liés. Aucun amendement direct.

Fichiers. La relation `context_consumers` (moteur, résolution d'ascendance dans l'arbre d'éléments).

Chapitre. chapitre 14.

A.33 ADR-033 — Bit d'exactitude de la chasse aux liaisons

2 septembre 2026. Statut : Accepted.

Contexte. La sortie de l'analyseur n'était pas rigoureusement déterministe (issue #120) : `normalize_to_prop` pouvait répondre un chemin de prop différent à chaque exécution sur le même code.

Décision. L'ensemble `seen` de la chasse aux liaisons (qui évite les cycles de résolution) est cloné *par branche* plutôt que partagé — clé sur les racines, récursion indépendante par chemin. La sélection à travers un objet littéral partiel réutilise `members_after_last_spread`, déjà partagée avec l'interpréteur. `NormPath { path, exact }` : un chemin *élargi* (par exemple parce qu'une des branches passait par une propriété dynamique) ne peut plus soutenir une revendication `must`.

Refusé. Corriger uniquement §1 (le clonage par branche) sans ajouter le bit `exact` de §3 : aurait seulement *augmenté* la suppression de findings sans garantir qu'aucun nouveau chemin élargi ne serait pris pour un `must`. §3 est « la moitié qui garde l'autre moitié `sound` ».

Conséquences. Le corpus `mantine` gagne +1 (le vrai positif perdu par ADR-31 est restauré) ; 13 corpus restent identiques. Effet de bord positif : `collect_component_setter_vars` devient déterministe au passage.

ADR liés. Corrige le défaut de déterminisme révélé par ADR-31.

Fichiers. La chasse aux liaisons (`normalize_to_prop`, `NormPath`) dans le moteur des relations.

Chapitre. chapitre 14.

A.34 ADR-034 — Relation d'enregistrement, une table de registrars

2 septembre 2026. Statut : Accepted.

Contexte. Trois lecteurs différents entretenaient chacun sa propre liste (`REGISTRARS`, `DEFERRING_GLOBALS`, `DEFERRING_METHODS`) ; le résumé de phase promis par ADR-27 §2 n'avait jamais été écrit.

Décision. Une seule table `[Registrar]`, avec une colonne `timing` :

Extrait A.12 – `src/engine/registrations.rs`, l. 40–57 — Timing — « le contrat, pas le nom »

```
40 /// When a registered callback can run, relative to the React phases.
41 ///
42 /// This is the phase *summary* ADR-027 §2 promised and never shipped: a
43 /// registration argument used to fall to T in the slot-writer walk.
44 #[derive(Debug, Clone, Copy, PartialEq, Eq)]
45 pub enum Timing {
46     /// A timer, a microtask, a promise continuation: the callback runs on a
47     /// later turn of the event loop, provably outside every React phase.
48     Deferred,
49     /// An external event. The DOM has no synchronous dispatch from
50     /// `addEventListener`, so the callback provably does not run during the
51     /// registering call.
52     Handler,
53     /// The registrar may invoke the callback synchronously – an RxJS
54     /// `BehaviorSubject` emits to a new subscriber on the spot – so nothing
55     /// about the timing is proven and the walk keeps T.
56     Unknown,
57 }
```

`Handler` est réservé à `addEventListener` *seulement* (contrat DOM : aucun appel synchrone possible) ; `Unknown` couvre `subscribe/on/addListener`, dont le contrat n'exclut pas un appel synchrone (RxJS `BehaviorSubject` émet parfois immédiatement au nouvel abonné). Un appariement tri-valué entre enregistrement et *teardown* (`Paired` seul est une revendication exploitable) — amendement de l'issue #124 : trois formes de *teardown* distinguées. Une écriture faite depuis une classe `Handler` ne ferme pas un cycle de churn (issue #93). Un *teardown* n'est jamais compté comme un rappel du callback lui-même.

Rappel React : le contrat DOM de `addEventListener`

Le DOM ne dispatch jamais un événement de façon synchrone depuis l'appel `addEventListener` lui-même : le callback enregistré ne peut probablement pas s'exécuter *pendant* l'appel qui l'enregistre. C'est ce contrat, et lui seul, qui autorise `Timing::Handler` ; une bibliothèque d'observables comme RxJS ne l'offre pas.

Refusé. `Handler` pour `subscribe/on` : RxJS peut émettre *synchroniquement* à l'abonnement, donc aucun contrat ne garantit l'absence d'exécution immédiate — classer ces registrars en `Handler` serait le seul sens où « rétrécir T » pourrait perdre un vrai positif sans preuve.

Conséquences. Le catalogue passe de 20/22 à **21/22**, décrit dans l'ADR lui-même comme « the honest ceiling ». Amendement de l'issue #116 : une ancre publique `registrations` est ajoutée — la décision `wontfix` de l'issue #42 (heuristique de nom d'émetteur pour *stale-closure*)

couvre désormais une surface plus large grâce à cette ancre.

ADR liés. Livre le résumé de phase promis par ADR-27 §2 ; amendé par les issues #116 et #124.

Fichiers. `src/engine/registrations.rs`.

Chapitre. chapitre 14.

A.35 ADR-035 — Frontière de phase `await`

2 septembre 2026. Statut : Accepted, §1 amendé par ADR-36 §6.

Contexte. `AwaitExpression` était effacée au lowering, sans trace dans le CFG. La supposition « `sync_phase : lexis = execution`, provably » — c’est-à-dire que la position lexicale d’une instruction suffit à connaître sa phase d’exécution — devenait **fausse** après un `await` : le code qui suit dans le texte s’exécute en réalité sur un tour ultérieur de la boucle d’événements.

Décision. Scinder le bloc du CFG par une **arête** `EdgeKind::Await` (une variante d’arête, pas un champ ajouté à `BasicBlock`) :

Extrait A.13 – `src/ir/cfg.rs`, l. 32–45 — `EdgeKind` — l’arête `Await`, contrôle inchangé, phase rompue

```
32
33 #[derive(Debug, Clone, PartialEq, Eq)]
34 pub enum EdgeKind {
35     Unconditional,
36     IfTrue,
37     IfFalse,
38     Back,
39     /// The split at an `await` (#117, ADR-035). Control is the same – the edge
40     /// is unconditional – but the successor runs on a later turn of the event
41     /// loop, so `sync_phase`'s "lexis = execution, provably" stops holding
42     /// across it. Consumers ask [ `CFG::post_await_blocks` ] rather than reading
43     /// this variant directly.
44     Await,
45 }
```

`post_await_blocks` calcule la fermeture des blocs atteignables après un `await` ; la seule transition de phase permise est `Sync` → `Deferred`, jamais l’inverse. Décision annexe cruciale : les **IIFE** (fonctions immédiatement invoquées, `(async () => { . . . })()`) s’exécutent **à leur site d’appel** — sans cette règle, aucune ligne de relation n’était jamais produite pour ce motif très courant en React (`useEffect` avec une IIFE asynchrone interne).

Refusé. Un champ de bloc pour représenter l’`await`, plutôt qu’une variante d’arête : `BasicBlock` et CFG sont construits en plus de deux cents endroits du code ; une variante d’arête est *additive* (tout match existant sur `EdgeKind` a déjà un bras `_`), un champ de bloc aurait exigé de revoir chacun des deux cents sites.

Conséquences. Aucune régression mesurée ; le fait « la suite d’un `await` est différée » devient disponible aux relations en aval (`writers`, `registrations`).

ADR liés. §1 (l’endroit exact où la scission a lieu) amendé par ADR-36 §6 : l’expression attendue s’évalue *avant* la suspension, pas après.

Fichiers. `src/ir/cfg.rs` (`EdgeKind::Await`).

Chapitre. chapitre 6.

A.36 ADR-036 — Relation d'appels

2 septembre 2026. Statut : Accepted.

Contexte. Le corps d'un effet n'était lisible par les règles qu'à travers trois relations existantes (écriture, lecture, enregistrement) : aucune ne disait simplement « cette fonction a été appelée ici ».

Décision. Un second **canal** de la *même* marche (« the setter walk's second output, not a second walk ») : une ligne (name, receiver, phase) par appel, *sans* les valeurs des arguments (question distincte, laissée à l'issue #67). Le nom de l'appelé est une garde **obligatoire** : c'est la première relation du catalogue non bornée en cardinalité, et un appel sans nom résolu est rejeté même s'il est caché dans un combinateur any_of. Plafond WARNING. Ancre render_calls ; quantificateur none (une ligne *manquante* est la direction non sound, donc la direction sûre à choisir : mieux vaut une règle qui tire à tort qu'une qui se tait à tort). Option elements: component | host | any ; ancre elements.

Refusé. Exposer les valeurs des arguments dans cette même relation : une question à part (#67), qui mériterait sa propre relation plutôt qu'un élargissement de celle-ci.

Conséquences. Les règles de « calls » (audit d'API interdite, politique d'équipe) deviennent exprimables sans marcher le CFG elles-mêmes.

ADR liés. Prépare le terrain pour ADR-35 §1 (la position exacte de la scission await, amendée par cet ADR §6).

Fichiers. src/engine/setters.rs (second canal de la même marche).

Chapitre. chapitre 14.

A.37 ADR-037 — Relation de lecture de slot

2 septembre 2026. Statut : Accepted.

Contexte. Toute la relation d'écriture (slot_writers) n'avait pas de miroir côté lecture : aucune règle ne pouvait exprimer « ce slot est lu ici ».

Décision. Une image miroir exacte de slot_writers : mêmes colonnes region et phase. Troisième canal de la même marche unique. reads_in ne traverse *pas* un FnLit imbriqué (sinon T serait compté deux fois, une fois côté lecture et une fois côté appelant). La phase fait partie de l'*identité* de la ligne – sinon une ligne Cleanup distincte d'une ligne Effect disparaîtrait par fusion accidentelle. La marche devient une traversée « réelle » (et non un simple accumulateur) dès qu'un canal quelconque est actif.

Refusé — reporté, pas tranché. La moitié « la lecture atteint-elle le rendu ? » : une question de *taint* (propagation de provenance) distincte, laissée à ADR-41 (dépendance de rendu).

Piège

La même cécité que celle du côté écriture existe côté lecture : « the same blindness on the setter side is an FN », déposée comme issue #130. Ne pas supposer que corriger un côté de la marche corrige l'autre.

Conséquences. La lecture d'un slot devient une relation de première classe, symétrique à l'écriture.

ADR liés. La moitié « taint » refusée ici est reprise par ADR-41.

Fichiers. src/engine/setters.rs (troisième canal).

Chapitre. chapitre 14.

A.38 ADR-038 — Une écriture est une écriture où qu'elle soit écrite

2 septembre 2026. Statut : *Accepted*, §5 superseded by ADR-40.

Contexte. `wrap(setN(1))` passait *silencieusement* à travers la marche d'écriture, alors qu'un `setN(1)` nu au même endroit déclenchait `ERROR setter-in-render` : un faux négatif de pure position syntaxique — le setter était bien appelé, seulement enveloppé dans un appel de fonction.

Décision. Une traversée unique, *sans porte* conditionnelle sur la forme syntaxique de l'appel. `setter-in-render` lit enfin la phase *calculée* par le moteur (`Handler/Deferred` valent preuve de non-appartenance au rendu) plutôt que la position lexicale. `Deferred` est séparé de `T` dans `SetterCallPhase`. Deux conditions *must* avant de certifier une écriture en rendu : la phase est `Sync`, *et* la variable appelée *est* le setter ou une fermeture qui l'appelle (`must_write`). Une seule orthographe retenue pour l'identité d'un composant (le *display name*) — ce choix particulier sera remplacé par ADR-40.

Exemple A.3: Le FN de position corrigé

Extrait A.14 – Setter enveloppé, appelé en rendu

```
1 function wrap(f: () => void) {
2   f();
3 }
4
5 function Counter() {
6   const [n, setN] = useState(0);
7   wrap(() => setN(n + 1));
8   return <span>{n}</span>;
9 }
```

Sortie observée (e67b10a) :

```
Counter (1 hooks) wrapped-render-setter.tsx
error setter-in-render [hook:0] (line 4:2) setter `setN` called directly in the render body, move this call
  ↳ into a useEffect or an event handler
    (1 trace step(s), rerun with --trace)

! 1 error(s) across 1 file(s).
```

Avant ADR-038, cette forme — setter passé en fermeture à une fonction d'ordre supérieur — n'avait « no row in the writer relation at all » (citation de l'ADR).

Refusé. Rien d'alternatif à la suppression de la porte : la porte elle-même était le défaut.

Conséquences. Mesure sur corpus : 6 322 → 6 340 findings sur 34 730 fichiers (27 nouveaux WARNING, 9 retraits qui n'étaient pas de vrais findings mais des doublons de comptage). Sans précalcul de la marche, la règle aurait tourné 43 fois plus lentement sur le corpus *ai-chatbot* — justifiant que la relation soit calculée une fois à convergence plutôt qu'à la demande de chaque règle.

ADR liés. §5 (l'identité de composant par *display name*) superseded by ADR-40.

Fichiers. `src/engine/setters.rs` (`must_write`, `SetterCallPhase`).

Chapitre. chapitre 14.

A.39 ADR-039 — Une liaison synthétique a une position réelle

2 septembre 2026. Statut : Accepted.

Contexte. Mesuré : 82 findings sur 7 146 (1,1 %) n'avaient *aucune* position affichable — le lowering crée des liaisons synthétiques (des variables temporaires introduites par le désucre) qui n'existent pas dans le texte source.

Décision. Chaque instruction synthétique prend la position de l'expression qu'elle lie. Ce que le code source ne nomme nulle part (par exemple, un paramètre introduit par un splice) prend le **site d'appel** du splice qui l'a introduit. Une marche de relation porte un witness *hérité* par défaut, jamais en surcharge d'un témoin déjà présent. Un finding sans *range* du tout prend la première position de sa chaîne de témoins — calculé une seule fois, dans le registre partagé, pour toutes les règles à la fois.

Refusé. Ajouter un *span* à Terminator::Return (reprenant la question laissée ouverte par ADR-25) : « not worth it for what it buys » au regard du nombre de sites de construction à mettre à jour.

Conséquences. 82 → 0 finding sans position ; l'ensemble des findings lui-même reste inchangé (seule l'attribution de position change).

ADR liés. Referme, sans le résoudre entièrement (#140 reste ouverte), le même point qu'ADR-25 avait laissé de côté.

Fichiers. Le point de splice (chapitre 8) et src/rules/api/witness.rs (le registre de positions par défaut).

Chapitre. chapitre 8.

A.40 ADR-040 — Identité de composant = ComponentId

5 septembre 2026. Statut : Accepted ; supersedes ADR-38 §5.

Contexte. Trois représentations coexistaient pour « le même » composant : une clé (PathBuf, Symbol), un *display name* qui dépendait du *contenu* du fichier (deux fichiers définissant chacun un Widget produisaient des collisions), et un Symbol nu venant de CompApp. Deux défauts se terminaient tous deux par un rassurant « ✓ ... no issues found » à tort : un premier appariement trié par hasard, et des chemins de fichiers orthographiés différemment selon l'OS.

Décision. Une identité **interne**, indépendante du contenu :

Extrait A.15 – src/ir/component_id.rs, l. 23–29 — ComponentId — interné, comparable en une instruction

```
23 /// A component's identity, interned in a [`ComponentTable`].
24 ///
25 /// `Copy` and 4 bytes, so it sits inside a `BTreeSet` label or a store key
26 /// without the allocation a name costs, and two ids compare in one
27 /// instruction.
28 #[derive(Debug, Clone, Copy, PartialEq, Eq, Hash, PartialOrd, Ord)]
29 pub struct ComponentId(u32);
```

Le *display name* n'est plus qu'un **rendu**, calculé seulement à l'affichage, jamais utilisé comme clé. CompApp.origin porte le fichier et le nom exporté. resolve_child répond l'une de trois réponses, dont Ambiguous explicite (plutôt qu'un choix arbitraire silencieux). Tous les chemins de fichiers du dépôt sont normalisés à la même forme.

Piège

Le même motif structural revient deux fois dans l'histoire du projet : une identité *interne*, comparable et indépendante du contenu, à côté d'un *rendu* calculé seulement à l'affichage. `FileId/ADR-19` l'avait déjà fait pour les fichiers; `ComponentId/ADR-40` le refait pour les composants. Chaque fois qu'une chaîne de caractères sert de *clé* plutôt que d'étiquette d'affichage, un bug d'identité guette (fil rouge du chapitre 26, « identité $\neq$ rendu »).

Refusé — mesuré. Analyser chaque candidat d'un nom de composant ambigu, plutôt que de répondre Ambiguous : 1 347 références concernées sur le corpus, une forme quadratique en nombre de composants (rappel de la contrainte de performance déjà posée par #86 à ADR-18).

Conséquences. Le *digest* du corpus (l'empreinte agrégée des findings) reste **identique** sur 35 541 fichiers pour le seul changement d'identité — preuve que le renommage n'a, par lui-même, rien changé au comportement observable. La résolution de composants ambigus passe de 1 317 à 1 348 lignes résolues (29 retraits, 60 ajouts). Commit 806d114 : 107 fichiers, +2 959/−1 454 lignes ; temps de corpus 873 s contre 858 s avant.

ADR liés. Supersedes ADR-38 §5. L'ADR se déclare lui-même « Implements: #7 » et son texte reprend le triple défaut d'identité que l'issue décrivait ; le tracker, à e67b10a, marque pourtant toujours #7 ouverte — un oubli de fermeture, pas un doute sur la correction : l'ADR corrige bien ce que l'issue décrivait.

Fichiers. `src/ir/component_id.rs`.

Chapitre. chapitre 16.

A.41 ADR-041 — Dépendance de rendu

24 septembre 2026. Statut : Accepted ; amende ADR-22 §4.

Contexte. Quelles parties de la sortie JSX d'un composant dépendent réellement d'un slot d'état, et lesquelles ne font que le *transmettre* sans jamais l'utiliser pour décider quoi rendre ? Stability (chapitre 10) ne répond pas à cette question : elle est booléenne (stable/instable), alors qu'un `text.length` ou une allocation imbriquée demandent une notion plus fine.

Décision. Une analyse **avant séparée** (`engine/render_deps.rs`), sur un composant déjà convergé, calculant des ensembles *may* de Source (l'origine d'une valeur) ; distinction *genuine* (usage réel dans une décision de rendu) contre *sites* (propagation pure). Composition le long de l'arbre d'éléments *d'origine prouvée* : « absence of a use is a proof, so every unknown is a use » — tout ce que l'analyse ne peut pas exclure compte comme un usage, jamais l'inverse. Deux règles WARNING : `state-lifted-too-high` et `wasted-subtree-render`. Des options *typées* sur les règles natives (`Rule::options()`, `-rule-option`).

Rappel React : fréquence de déclenchement : continue ou discrète

Un événement `onMouseMove` déclenche à haute fréquence (continu) ; un `onClick` déclenche rarement (discret). `Deps::gated` enregistre, pour une valeur fonction, les captures que son corps n'atteint que sous un test de l'argument événementiel (issue #148) — un événement dont chaque écriture n'est atteinte qu'à travers un tel test est discret. Un échec de cette détection classe par défaut en discret : cela ne change jamais un statut `must` en `may`, seulement le classement d'un WARNING.

Refusé. Un champ de plus sur le domaine des valeurs (`StateValue`) : cela changerait *toute* comparaison de valeurs, y compris la clé du cache d'inlining (`ComponentCache`) — un coût de performance et de complexité disproportionné pour une propriété qui n'est utile qu'à deux règles.

Conséquences. Un provider de contexte *prouvé* est suivi, pas un simple usage (issue #145) : sa valeur atteint, comme `Context(id)`, tout élément imbriqué — un consommateur peut donc gaspiller son sous-arbre, et un état peut être « chez lui » juste au-dessus de son provider. Un handler qui écrit un binding de module à côté d'un état (issue #147) est suivi jusqu'aux lecteurs de ce binding. Mesure sur corpus : la baseline passe de 1 346 à 1 500 (commit `ba09a62`), puis se stabilise à 1 493, 1 494 après corrections de précision.

ADR liés. Amende ADR-22 §4 (portée des options de règles natives).

Fichiers. `src/engine/render_deps.rs`.

Chapitre. chapitre 16 (définition 16.3) et chapitre 21 (les deux règles).

A.42 ADR-042 — Les relations sont des produits du moteur

26–27 septembre 2026. Statut : Accepted.

Contexte. Une revue externe a repéré que le graphe de churn vivait encore dans `rules/helpers/` avec sa *propre* marche de setters, indépendante de `slot_writers` et sans classification de phase — exactement la situation qu'ADR-27 §1 avait commencé à corriger pour les écritures, mais jamais généralisée. L'issue #26 est le symptôme direct de cette dérive.

Décision. Une frontière normative, tenue par un **cliquet** (un test qui casse la compilation ou la CI si elle est franchie) : « une règle lit des lignes et appelle des must-primitives, elle ne marche aucune syntaxe ».

Extrait A.16 – `src/engine/program_relations.rs`, l. 21–44 — `ProgramRelations` — le graphe de churn construit une fois, en aval du point fixe

```

21 pub struct ProgramRelations<'a> {
22     program: &'a ProgramAnalysisResult,
23     churn: OnceLock<ChurnGraph>,
24 }
25
26 impl<'a> ProgramRelations<'a> {
27     pub fn new(program: &'a ProgramAnalysisResult) -> Self {
28         ProgramRelations {
29             program,
30             churn: OnceLock::new(),
31         }
32     }
33
34     pub fn program(&self) -> &'a ProgramAnalysisResult {
35         self.program
36     }
37
38     /// The program's churn graph and its cycles, built on first request.
39     pub fn churn(&self) -> &ChurnGraph {
40         self.churn.get_or_init(|| ChurnGraph::build(self.program))
41     }
42 }

```

`SlotWriter` gagne trois colonnes : `owner`, `block`, `written` (déjà vues aux fiches ADR-30 et ADR-28, désormais formalisées ici) :

Extrait A.17 – `src/engine/setters.rs`, l. 770–786 — Les colonnes `owner`, `block`, `written` de `SlotWriter`

```

770     /// A foreign row is a may-write: the prop may be a closure that merely
771     /// carries the setter.

```

```

772     pub owner: Option<ComponentId>,
773     /// The block of the region's CFG the write runs in, for a write that runs
774     /// synchronously in the region, once per pass – what `must_on_all_paths`
775     /// takes. `None` for a nested, deferred, cleanup or repeating site.
776     pub block: Option<BlockId>,
777     /// The block of the region's CFG whose dominating guards the write runs
778     /// under: `block` for a synchronous write, the statement that scheduled
779     /// it for a nested, deferred, cleanup or repeating one – a continuation
780     /// runs only if the run that scheduled it took every branch above the
781     /// scheduling statement (#160). `None` for a row with no placeable
782     /// block.
783     pub guard_block: Option<BlockId>,
784     /// What the write stores (ADR-042 §2).
785     pub written: Written,
786 }

```

et Written lui-même porte la fraîcheur et la valeur écrite :

Extrait A.18 – src/engine/written.rs, l. 37–58 — Freshness et Written — ce qu’une écriture stocke, pour les preuves de churn

```

37 /// Does a write store a new reference every time it runs?
38 #[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord)]
39 pub enum Freshness {
40     Not,
41     /// May store a fresh reference (opaque value, imprecise updater).
42     Maybe,
43     /// Must store a fresh reference every call (`PerRender` argument).
44     Fresh,
45 }
46
47 /// Argument 0 of a write, as the churn proofs read it.
48 #[derive(Debug, Clone)]
49 pub struct Written {
50     pub fresh: Freshness,
51     /// The abstract value stored. A functional updater stores its return
52     /// value, approximated as a fresh reference: the proofs only ever read
53     /// the reference part, and a fresh-returning updater stores a truthy,
54     /// non-null one.
55     pub value: StateValue,
56     /// The argument as written, `None` for a bare `setX()`.
57     pub expr: Option<Expr>,
58 }

```

Une relation `effect_triggers` nouvelle (**identité**, pas dépendance de rendu — délibérément *non* unifiée avec `render_deps` d’ADR-41) :

Extrait A.19 – src/engine/triggers.rs, l. 33–44 — EffectTrigger — une ligne par dépendance d’effet qu’un slot fait bouger

```

33 /// One dep of an effect that a state slot moves.
34 #[derive(Debug, Clone, PartialEq, Eq)]
35 pub struct EffectTrigger {
36     pub hook: HookLabel,
37     /// Index of the dep in the effect's deps list.
38     pub dep: usize,
39     pub slot: QualifiedSlot,
40     /// The dep IS the slot value (`StateVal(1)` or an alias of it): the effect
41     /// must re-run whenever a fresh value is stored. `false`: the dep is
42     /// versioned by the slot, the effect may re-run.
43     pub exact: bool,

```

44 }

engine/churn.rs devient un simple *pli* sur deux relations, combinées par la table de phases (section 26.7 pour le détail de cette table) ; les preuves suivent désormais les données : « sur tous les chemins » vit dans engine/dominance.rs, les gardes dans engine/guards.rs. Un catalogue docs/rerelations.md documente toutes les relations publiées.

Refusé. Unifier effect_triggers avec render_deps : « identité $\neq$ dépendance » — le *must-rerun* d'un effet a besoin de savoir *quelle* dépendance a bougé (identité), pas seulement *si* le rendu en dépend. Dans un amendement du 27 septembre sur les sites de convergence, un *plus grand* point fixe sur les sites de revivification est refusé : il « would read mutual revival as convergence » — deux effets qui se relancent mutuellement à l'infini semblent « converger » vus par le plus grand point fixe, alors qu'ils ne convergent pas du tout. Le point fixe retenu est le **plus petit** (définition 3.5).

Conséquences. rules/helpers/churn.rs et rules/helpers/churn_graph.rs sont supprimés ; engine/churn.rs, engine/guards.rs, engine/triggers.rs et engine/program_relations.rs existent désormais ; rules/api/cache.rs a disparu (vérifié : absent à e67b10a). L'issue #26 est fermée *par construction* (plus de seconde marche à faire dériver), avec un test de régression dans tests/effect_cycles.rs. Séquence livrée en cinq commits séparés, chacun mesuré sur corpus, l'arbre restant vert entre chaque étape : refactor sans changement de comportement, ajout de owner/block/written, ajout d'effect_triggers, migration de engine/churn.rs avec re-mesure, puis le test-cliquet et docs/rerelations.md. Faiblesse assumée, héritée d'ADR-29 : les lignes de cycle n'existent que pour les cycles *vus* par le graphe — les arêtes médiées par une prop auraient besoin de faire remonter le slot parent (#20, encore ouverte).

ADR liés. Amende ADR-18 (construction et condition d'écrivain unique), ADR-29 §1 (les lignes viennent désormais du moteur), ADR-21 §4 (ProgramCache $\rightarrow$ ProgramRelations).

Fichiers. src/engine/program_relations.rs ; src/engine/churn.rs ; src/engine/guards.rs ; src/engine/triggers.rs ; src/engine/setters.rs ; src/engine/written.rs.

Chapitre. chapitre 14 et chapitre 15.

Résumé du chapitre

Quarante-deux ADR, quarante-et-une décisions en vigueur et un seul remplacement complet (ADR-8, superseded by ADR-15). Trois formes reviennent tout au long du catalogue et valent d'être retenues au-delà de la fiche qui les introduit chacune :

- **Une relation est promue, jamais dupliquée.** À partir d'ADR-27, chaque nouvelle relation (slot_seeds, context_consumers, calls, reads, registrations, le graphe de churn et effect_triggers) est calculée *une fois*, par le moteur, sur la *même* marche qu'une relation déjà existante — jamais par un second parcours du CFG. ADR-42 donne la formule qui clôt cette série : « a fact computed in two places is two facts that drift ».
- **Un non-changement est une décision, pas un oubli.** ADR-20 (onze fois) et la table dispersée du chapitre 26 (trente-six fois) documentent des simplifications apparentes refusées parce qu'elles coûtaient un faux négatif mesurable. Avant de « nettoyer » une duplication dans ce code, chercher d'abord si un ADR l'a déjà justifiée.
- **L'identité n'est jamais un rendu.** FileId (ADR-19), ComponentId (ADR-40), la cellule canonique ContextId (ADR-32) répètent le même motif : une clé interne, comparable et indépendante du contenu, à côté d'une chaîne calculée seulement pour l'affichage. Chaque défaut d'identité rencontré dans l'histoire du projet venait d'une chaîne de caractères utilisée comme clé.

Cette annexe est un catalogue de référence, pas un récit : pour l'histoire suivie — pourquoi ces décisions sont venues dans cet ordre, quels fils rouges les traversent, ce que la mesure sur corpus a changé à la méthode — voir chapitre 26. Pour une décision citée depuis un chapitre technique, la macro `\adr{NN}` renvoie directement à la fiche correspondante de cette annexe.

Exercices

Exercice A.1: Direction d'un refus

Pour cinq ADR de votre choix parmi ADR-9, ADR-17, ADR-28, ADR-34 et ADR-42, identifiez dans le champ « Refusé » de la fiche correspondante si l'alternative écartée aurait *ajouté* des findings, en aurait *retiré*, ou n'aurait rien changé au compte total. Pour chacune, écrivez la phrase unique qui justifie le refus.

Exercice A.2: Reconstituer une chaîne de remplacement

En partant d'ADR-8, suivez la chaîne « supersedes »/« amends » jusqu'à la décision en vigueur à e67b10a pour le domaine des valeurs d'état. Combien d'ADR faut-il traverser ? Faites de même en partant d'ADR-11 pour la provenance des diagnostics.

Exercice A.3: Une écriture qui échappe encore

ADR-38 corrige le cas `wrap(setN(1))`. Construisez, sur le modèle de l'exemple de cette fiche, une variante à deux niveaux d'enveloppe (`wrap(( ) => wrap2(( ) => setN(n + 1))`) et vérifiez avec `cargo run -q - check` si elle est toujours détectée. Si oui, quelle propriété de la décision (« une traversée, sans porte ») l'explique ?

Exercice A.4: Écrire un ADR

Au format des fiches de cette annexe (Contexte, Décision, Refusé, Conséquences), rédigez un ADR-043 fictif qui autoriserait le point fixe à descendre dans les appelés `Unknown` d'ADR-9 — puis un second ADR qui refuse ce changement. Lequel des deux respecte l'invariant « faux négatifs interdits » de `CLAUDE.md` ? Lequel respecte la politique historique « FP-averse » d'ADR-9 ? Les deux réponses sont-elles la même décision ?

Annexe B

Glossaire

Objectifs du chapitre.

- Disposer d'un point d'entrée alphabétique unique pour tout terme, type ou identifiant rencontré dans le manuscrit ou dans le code de `reactant-analyzer`.
- Distinguer, pour chaque terme, un rappel d'une phrase de la définition formelle qui vit dans un chapitre (renvoi `def:...`) de la simple mention d'un type ou d'une ligne de code sans théorie associée.
- Repérer les quelques mots qui portent deux sens différents selon le chapitre (`site`, `région`, `ancre`) et savoir lequel est en jeu.

Prérequis : aucun : cette annexe se consulte à tout moment, dans n'importe quel ordre, comme un dictionnaire. Elle suppose seulement le vocabulaire de base de chapitre 3 (`treillis`, `must/may`, `soundness`) pour les entrées mathématiques.

Cette annexe fusionne les glossaires des seize dossiers de notes qui ont servi à écrire les chapitres techniques, harmonisés avec le « Lexique imposé » de `PLAN.md`. Elle ne réexplique rien : quand un terme a déjà une définition formelle numérotée ailleurs dans le manuscrit (définition 3.2, définition 6.3...), l'entrée ici tient en une phrase et renvoie au chapitre. Pour tout le reste — un type `Rust`, une fonction, un champ de structure — l'entrée dit ce que fait la chose, en une à trois phrases, avec l'endroit du dépôt où la vérifier (`\src` ou `\src\line`) au commit `e67b10a`. Les entrées sont classées alphabétiquement sans distinguer majuscule et minuscule ; un identifiant du code (`StateValue`, `ExprId`) est classé à sa première lettre, comme un mot ordinaire.

Piège

Trois mots changent de sens selon le chapitre où on les lit. `site` désigne un site d'allocation (chapitre 6, `ExprId`, une clé du tas abstrait) *ou* un site d'écriture de la preuve de convergence (chapitre 15, une ligne non-handler d'un corps de rendu ou d'effet) — et *aussi* un `ElementSite` de rendu (chapitre 16). `région` désigne le corps lexical d'une écriture (`WriterRegion`, chapitre 14) *ou* la plage de blocs d'une greffe (`InlineRegion`, chapitre 8) : la première est un des cinq valeurs `Render/Effect/Memo/Callback/Handler` qu'une écriture habite, la seconde une plage `start..end` de `BlockId` que la greffe enregistre pour décider si une écriture est directe ou passée par un wrapper. Enfin `ancre` nomme la position qui fait l'identité d'un diagnostic (chapitre 17) *et*, dans le vocabulaire des packs déclaratifs, la relation sur laquelle une règle Tier-A itère (chapitre 22). Le contexte tranche à chaque fois ; ce glossaire signale les deux sens plutôt que d'en choisir un.

Glossaire

- AccessPath (chemin d'accès)** Racine plus chaîne de champs d'une lecture, utilisée pour épingle une lecture aux deps déclarées d'un hook. `src/ir/free_vars.rs`, l. 19 — chapitre 6.
- ADR** *Architecture Decision Record* : décision que le reste du système doit respecter (un domaine, une relation, un invariant, une alternative refusée) ; les 42 fiches sont à chapitre A, l'histoire du projet à chapitre 26. `docs/adr/README.md`.
- alias de setter** Constante liée une seule fois à un setter (`const s = setX`), résolue par fermeture sur le CFG plutôt que par égalité syntaxique. `src/engine/setters.rs`, l. 581 — chapitre 14.
- alias (tsconfig)** Motif du champ `paths` d'un `tsconfig.json` (`@/*`) ; « exact » sans `*`, « wildcard » avec. `src/project/paths_resolver.rs`, l. 22 — chapitre 23.
- ambigu / Ambiguous** Réponse de la résolution d'un nom JSX quand plusieurs fichiers définissent ce nom sans qu'aucune preuve d'import ne tranche ; le composant se lit alors `T`. `src/engine/component_registry.rs`, l. 114 — chapitre 16.
- analysis-limit** INFO émise quand l'analyse a coupé volontairement (composant introuvable, budget d'inlining épuisé, récursion) ; jamais un défaut, toujours un faux négatif possible signalé. `src/rules/impls/analysis_limit_info.rs` — chapitre 25.
- ancre (anchor)** Voir l'encadré ci-dessus : position qui fait l'identité d'un diagnostic (chapitre 17) ou relation sur laquelle itère une règle Tier-A (chapitre 22). ADR-024, ADR-022 §2.
- arête Await** Arête de CFG inconditionnelle dont le successeur s'exécute après une suspension (`await`) ; les blocs qu'elle atteint sont « post-await », où une marche Sync devient Deferred. `src/ir/cfg.rs`, l. 39 — chapitre 7.
- arête Back (arc arrière)** Arête qui referme une boucle dans un CFG ; seul endroit où le point fixe applique le widening. `src/ir/cfg.rs`, l. 38 — chapitre 12.
- arité (Arity)** Longueur connue de la source d'un tableau de deps : `Exact(n)` (exactement n) ou `AtLeast(n)` (au moins n , en présence d'un spread). `src/ir/hooks.rs`, l. 52 — chapitre 6.
- assurance (verified:) / SafeCheck** Ligne affichée sous `-info` quand une règle applicable n'a rien trouvé ; retirée (« suspended ») pour un composant tronqué par un `analysis-limit`. `src/rules/mod.rs`, l. 61 — chapitre 17.
- bail-out** Abandon par React d'un rendu déjà planifié quand `Object.is(ancienne, nouvelle)` est vrai pour l'état concerné ; la borne de stabilité `Stable/Versioned` sert précisément à prédire ce cas. chapitre 2, chapitre 10.
- baseline (corpus)** Nombre de localisations distinctes commité dans `docs/corpus-baseline.json`, produit par un script et jamais tapé à la main. chapitre 25.
- baseUrl-only** Résolveur `TsconfigPaths` sans motif (`patterns` vide) : résout les spécificateurs nus relativement à `baseUrl`, sans `alias`. `src/project/tsconfig.rs`, l. 22 — chapitre 23.
- blind spot (angle mort)** Section « not analyzed » du rapport (`unresolved-aliases`, `unparsed-files`, `unread-imports`) : ce que le run sait ne pas avoir lu, jamais un finding. `src/driver/blind_spots.rs`, l. 22 — chapitre 23.
- bloc de base (BasicBlock)** Suite linéaire d'instructions (`Stmt`) terminée par un `Terminator` ; unité du CFG. `src/ir/cfg.rs`, l. 6 — chapitre 6 (définition 6.1).
- bloc de jonction (join block)** Bloc où convergent les branches d'un `if`, d'un ternaire ou d'un `&&/||` ; porte les instructions qui suivent la structure de contrôle. chapitre 7.
- bloc orphelin** Bloc présent dans le CFG mais non atteignable depuis l'entrée (par exemple la jonction de deux `return`) ; `CFG::reachable_blocks` l'exclut des passes d'analyse. `src/ir/cfg.rs`, l. 146 — chapitre 6.
- boucle automatique** Suite `rendu` → effets → `setState` → `rendu` qui s'entretient sans événement externe ; seul objet de la preuve de convergence du churn. `src/engine/churn.rs`, l. 26 — chapitre 15.

- budget d'inlining** Nombre maximal de greffes d'utilitaires par CFG (`max_inline_depth`, 8 par défaut); au-delà, les appels restants sont opaques. `src/engine/fixpoint.rs`, l. 1617 — chapitre 8.
- canal (*channel*)** Sortie de la marche des écrivains : `setters` (toujours produite), `calls` (`collect_calls`), `reads` (`read_vars` non vide). chapitre 14.
- candidat (*Candidate*)** Fonction de premier niveau retenue par un détecteur (nom, paramètres, corps, drapeau expression), avant classification en composant, hook ou utilitaire. `src/lowering/mod.rs`, l. 133 — chapitre 5.
- canonisation (stabilité)** Réduction `Versioned(∅) ≡ Stable`, et passage à `VersionedTop` au-delà de 4 slots suivis. `src/domains/impls/stability.rs`, l. 55 — chapitre 10.
- catalogue Tier A** Les 22 classes de règles de référence utilisées pour mesurer l'expressivité des packs déclaratifs; 21 sont exprimables, 1 exclue par construction. `tests/catalogue.rs` — chapitre 22, chapitre C.
- certificat de liaison** Preuve syntaxique qu'un nom désigne, dans tout le corps analysé, une unique valeur ou fermeture (jamais re-lié plus bas); condition pour qu'un Updater soit lu Functional plutôt que T. `src/engine/setters.rs`, l. 442 — chapitre 14.
- Certified** Jeton de preuve *must*, frappé seulement dans `src/rules/api/query.rs`, seul passage vers `ERROR`. Voir définition 17.1. `src/rules/api/query.rs`, l. 80.
- CFG (graphe de flot de contrôle)** Graphe de blocs de base et d'arêtes typées produit par le lowering. Voir définition 6.1. `src/ir/cfg.rs`, l. 60.
- churn** Réécriture d'une référence fraîche dans un slot dont dépend l'effet qui l'écrit, relançant la boucle automatique par échec d'`Object.is`. chapitre 15, ADR-017.
- cible d'affectation** Côté gauche d'une affectation ou d'un motif de déstructuration abaissé par le lowering : identifiant, membre, ou temporaire de motif. `src/lowering/cfg_builder.rs`, l. 950 — chapitre 7.
- clamp / pin / ceiling** Abaissement de sévérité par la configuration ou un pack, jamais de relèvement : la sévérité effective est `pin ⊓ polarité`. `src/rules/api/diagnostic.rs`, l. 104 — chapitre 17.
- classe de déclenchement (*TriggerClass*)** Ce qui déclenche un callback exécuté « dans le cycle » : `Setter`, `InCycle`, `Subscription`, `Unknown`. `src/domains/interp/callbacks.rs`, l. 6 — chapitre 11.
- clean bill** Ligne ✓ ... no issues found. du rapport humain, émise seulement en l'absence de tout finding *et* de tout blind spot. `src/driver/human.rs`, l. 266 — chapitre 23.
- cliquet (*ratchet*)** Liste d'exceptions (fichiers de règles qui touchent encore la syntaxe, scénarios de campagne) qui ne peut que rétrécir d'un commit à l'autre. Voir définition 26.4. `tests/layer_boundary.rs`.
- ClosureBinding** Les deux orthographes possibles d'une liaison fonctionnelle : littéral `FnLit` ou `CallbackVal(label)` référant à un `useCallback`. `src/ir/bindings.rs`, l. 53 — chapitre 6.
- ComponentId** Identité internée d'un composant sur 4 octets; `SYNTHETIC (u32::MAX)` en réserve l'analyse hors registre. `src/ir/component_id.rs`, l. 28 — chapitre 6.
- composant** Fonction de premier niveau capitalisée satisfaisant l'un des critères 2 à 4 du détecteur (retour JSX, annotation de type, appel direct de hook). Voir définition 5.1. `src/lowering/component_detector.rs`, l. 47.
- config root / discovery root** Ancêtre le plus proche portant un marqueur d'outil de build (racine de configuration, point de départ de la recherche de `tsconfig`) et répertoire effectivement parcouru (parfois `<root>/src`). `src/project/mod.rs`, l. 181 — chapitre 23.
- constante de module (*ModuleConstInit*)** `const` de niveau supérieur à initialiseur de nature certaine (primitive, référence, contexte); sert notamment à prouver un `<X.Provider>`. `src/ir/component.rs`, l. 12 — chapitre 6.
- container (*state-mutation*)** Portée dans laquelle une mutation d'objet est signalée : 0 pour le

- rendu, $1+i$ pour le hook i . `src/rules/impls/state_mutation.rs`, l. 46 — chapitre 18.
- contested (slot)** Slot écrit dans plusieurs régions lexicales, ou dont le setter s'échappe; `redundant-set-state` se tait sur un slot contesté. `src/rules/impls/redundant_set_state.rs`, l. 57 — chapitre 18.
- convergence kill** Suppression d'une arête du graphe de churn dont l'écriture est prouvée tirer au plus une fois dans la boucle automatique; calculée par plus petit point fixe (ADR-042 §6). `src/engine/churn.rs`, l. 26 — chapitre 15.
- corpus (épinglé)** Ensemble de dépôts `test-repo/` clonés sur des SHA fixes, référence de mesure des campagnes de précision. `scripts/setup-test-repo.sh` — chapitre 25.
- couche (Tour d'architecture)** Bloc de code délimité qui ne connaît que les couches en-dessous de lui (front-end, IR, domaines, moteur, règles, driver). Voir définition 4.1. chapitre 4.
- couture (seam) FS** Trait `FileSystem` par lequel passe toute lecture du moteur, point d'entrée d'un système de fichiers en mémoire pour les tests et le WASM. `src/resolver/filesystem.rs`, l. 1 — chapitre 23.
- couverture (covering)** Éléments d'une liste de deps qui peuvent *taire* un finding (à l'exclusion des sources de spread); à distinguer des éléments qui peuvent *tirer* une règle. Voir définition 6.6. `src/ir/hooks.rs`, l. 203.
- deps** Argument optionnel d'un hook React qui borne quand il se ré-exécute (`useEffect`, `useMemo`, `useCallback`); modélisé par `DepsArg` (`Absent/Opaque/List`). `src/ir/hooks.rs`, l. 103 — chapitre 6.
- diamant** Motif de CFG d'un `?:`, `&&`, `||` ou `??`: un Branch, deux blocs qui affectent un temporaire, un bloc de jonction qui le lit. chapitre 7.
- directive / prologue** Chaîne littérale en tête d'un module (`"use client"`), lue depuis le prologue du programme `oxc`. `src/ir/module.rs`, l. 22 — chapitre 6.
- dispatch opaque** Chaîne de `Branch(true)` par laquelle le lowering encode le choix d'une case de switch. `src/lowering/cfg_builder.rs`, l. 786 — chapitre 7.
- dominance** Un bloc a domine un bloc b si tout chemin de l'entrée à b passe par a . Voir définition 3.9. `src/engine/dominance.rs`, l. 6.
- drift (corpus)** État d'un dépôt du corpus qui n'est plus au SHA épinglé: signal d'alerte de la méthode de mesure. `scripts/setup-test-repo.sh`, l. 80 — chapitre 25.
- driver** Composition de check partagée entre le binaire natif et le cœur WASM, indépendante d'argv et des flux d'entrée-sortie. Voir définition 23.1. `src/driver/mod.rs`, l. 1.
- écrivain (writer)** Voir `SlotWriter` / définition 14.2.
- élargissement voir widening** Voir définition 3.6.
- ElementSite** Élément de composant construit par le rendu, avec ses deps par prop, sa garde, sa liste, son éventuel Provider et son parent; unité de base de `render_deps`. `src/engine/render_deps.rs`, l. 252 — chapitre 16.
- emplacement (slot de StateValue)** Une composante du produit `StateValue` (nombre, booléen, chaîne, référence, null, undefined, setter, other); à ne pas confondre avec un slot d'état (définition 6.3). `src/domains/impls/state_value.rs`, l. 26 — chapitre 10.
- entité / arête / garde (vocabulaire Tier A)** Position dans une relation résolue par le moteur (`Anchor`, ligne); navigation typée vers une entité voisine (`EdgeName`); prédicat sur un verdict ou un champ d'entité (`Guard`). `src/rules/declarative/schema.rs`, l. 108 — chapitre 22.
- env de sortie (exit_env)** Jointure des environnements de sortie des blocs `Return` d'un rendu; sert d'environnement d'entrée aux effets et aux handlers. `src/engine/fixpoint.rs`, l. 1226 — chapitre 13.
- ExitDominance** Relation « ce bloc domine toutes les sorties `Return` atteignables » d'un CFG, construite une fois et servant de preuve `must` à plusieurs règles. `src/rules/api/query.rs`,

l. 647 — chapitre 17.

ExprId (site d'allocation) Identité stable d'un nœud allouant (`ObjectLit`, `ArrayLit`, `FnLit`, `New`) entre les itérations du point fixe ; clé du tas abstrait. Voir définition 6.2. `src/ir/types.rs`, l. 11.

fail-closed Principe : n'enregistrer comme preuve que ce qui est syntaxiquement établi ; l'absence de preuve se lit $\top$ (inconnu), jamais comme un fait négatif. chapitre 5, chapitre 4 (définition 4.2).

faux positif / faux négatif (FP / FN) Un finding sans défaut réel (toléré) contre un défaut réel non signalé (interdit par l'invariant de soundness). Voir définition 1.4.

feeder / nourricier Slot d'un autre composant dont une prop de graine (`seed`) dépend ; `MovingFeeder` porte cette relation. `src/rules/api/query.rs`, l. 779 — chapitre 18.

fixpoint / point fixe Le plus petit état stable pour une fonction de transfert monotone ; le point fixe d'un composant est ce que calcule la boucle de `fixpoint.rs` (avec widening, donc un post-point fixe). Voir définition 3.5. chapitre 13.

flèche concise Fonction fléchée à corps expression (`x => expr`) ; le lowering l'abaisse en `Return(expr)` et porte le drapeau expression sur le `Candidate`. `src/lowering/mod.rs`, l. 137 — chapitre 5.

foreign (slot) Slot qualifié qu'un site d'un autre composant écrit (via une prop `ComponentSetter`) ; jamais tué par la preuve de convergence du churn. `src/engine/churn.rs`, l. 342 — chapitre 15.

freshness (fraîcheur) Ordre `Not < Maybe < Fresh` : l'écriture d'un slot stocke-t-elle une référence neuve à chaque appel. `src/engine/written.rs`, l. 37 — chapitre 14.

frontière (boundary) Directive de module qui arrête et exclut un parcours `reachable_from` (typiquement `"use client"`). `src/ir/module.rs`, l. 91 — chapitre 6.

garde (guard) Condition de branche qui domine une écriture ou un élément monté ; en Tier A, aussi un filtre de règle sur un verdict. `src/engine/guards.rs`, l. 655 — chapitre 14, chapitre 15.

graphe d'appels Arêtes `parent → CallSite{callee, props, location}` enregistrées à chaque évaluation d'un `<Child/>`. `src/engine/program_result.rs`, l. 196 — chapitre 11, chapitre 16.

graphe de churn Graphe programme-entier $x \rightarrow y$: un changement de x relance un effet qui écrit une référence fraîche dans y . Voir définition 15.1. `src/engine/churn.rs`, l. 94.

guard_block Bloc dont les gardes dominantes s'appliquent : celui qui a planifié une écriture différée. `src/engine/setters.rs`, l. 777 — chapitre 14.

handler Corps qu'un tiers (React, un enfant) peut invoquer de 0 à N fois : prop `JSX onX` ou callback `addEventListener` de premier niveau d'un effet. Voir définition 7.4. `src/ir/hooks.rs`, l. 306.

hauteur (d'un treillis) Longueur maximale d'une chaîne strictement croissante ; finie pour `StrConst` (5) ou `Stability` (au plus 8), infinie pour les intervalles. chapitre 9.

havoc Jointure de $\top$ dans les slots dont un setter s'échappe vers un enfant inanalysable. Voir définition 16.2. `src/domains/transfer/state_value.rs`, l. 265.

HOF synchrone Méthode dont le callback s'exécute dans la phase appelante (`.map`, `.forEach`...), listée dans la table `SYNC_HOF_METHODS` du moteur. chapitre 14, chapitre 22.

home / hop (arbre de rendu) Racine du plus petit sous-arbre contenant tous les usages d'un slot, et une étape `parent → enfant` de transmission d'une valeur par props ou contexte. `src/rules/helpers/render_tree.rs`, l. 36 — chapitre 16, chapitre 21.

hook custom Fonction de premier niveau dont le nom satisfait `is_hook_name`. Voir définition 5.1. `src/lowering/hook_detector.rs`, l. 46.

HookEntry Ligne de la table des hooks d'un corps : `State`, `Effect`, `Memo`, `Callback`, `Ref`, `Custom`, `Handler`. `src/ir/hooks.rs`, l. 247 — chapitre 6.

HookLabel (label) Rang `usize` d'un hook dans l'ordre de déclaration d'un composant ; indice de

- slot local au composant. Voir définition 6.3. `src/ir/types.rs`, l. 2.
- HookMarker / MarkerVal** Expression qui ancre, dans le CFG, le site d'appel d'un hook sans valeur suivie ; `MarkerVal` dit ce que lit sa liaison (`Undefined`, `StableRef`, `Unknown`, `Summary`). `src/ir/expr.rs`, l. 287 — chapitre 6.
- HookOrigin / HookProvenance** Ce que la déclaration d'import prouve d'une liaison de hook (`React`, `File`, `Package`) ; ligne « `label` → (hook d'origine, spécificateur, fichier, inliné?) ». `src/lowering/import_resolution.rs`, l. 41 — chapitre 5.
- hôte (*host*)** Frontend qui parse `argv`, découvre la configuration, décide des couleurs et écrit les flux (CLI native ou emballage WASM/JS) ; jamais frontière de confiance. Voir définition 23.1, définition 24.1.
- hull** Le plus petit intervalle contenant deux intervalles : leur *join*. `src/domains/impls/interval.rs`, l. 69 — chapitre 9.
- IIFE** Fonction immédiatement appelée sur place ; marchée au site d'appel comme un helper de la marche des écrivains. `src/engine/setters.rs`, l. 1920 — chapitre 14.
- in-cycle** Callback qui s'exécute en conséquence directe du rendu ou de l'effet courant (HOF synchrone, promesse, minuteur) ; s'oppose aux handlers. ADR-009 — chapitre 11.
- InlineRegion / région d'inlining** Plage de `BlockId` d'une greffe (avec le fichier d'origine), qui décide si une écriture est `Direct` ou `Via` un wrapper. ADR-027 §4 — chapitre 8, chapitre 14.
- inlining (top-down)** Analyse d'un composant enfant au site `<Child/>` avec les props abstraites évaluées par le parent. `src/domains/transfer/state_value.rs`, l. 470 — chapitre 16.
- InterCtx** Contexte d'analyse inter-composants partagé : registre, cache, store partagé, graphe d'appels, statistiques, pile d'appel. `src/domains/context.rs`, l. 59 — chapitre 11.
- IR** Représentation intermédiaire possédée, sans AST, seule lue par les domaines, les relations et les règles. Voir définition 6.1 et chapitre 6.
- k-ensemble (ensemble *k-borné*)** Ensemble d'au plus 4 constantes littérales ; au-delà, l'emplacement devient $\top$ (widening par seuil de cardinal). Voir définition 9.3.
- kill / *kill set* (ensemble tueur)** Écriture du site plus les écritures synchrones de slots locaux sur sa chaîne de gardes, dont la mort est requise pour prouver la convergence. Voir définition 20.1 (règles), définition 15.3 (moteur).
- KindMask** Masque des sortes JS peuplées d'un `StateValue`, énumération protégée par déstructuration exhaustive. `src/domains/impls/state_value.rs`, l. 46 — chapitre 10.
- label (voir HookLabel)** Voir définition 6.3.
- landing** Handler (hôte, ou prop `onX opaque`) où une capacité d'écriture transmise à un enfant finit par être appelée depuis un événement. `src/rules/helpers/render_tree.rs`, l. 80 — chapitre 16.
- liaison périmée (*stale binding*)** Variable laissée, dans l'IR, à sa valeur antérieure alors que le programme concret l'a réécrite entre-temps ; une sous-approximation assumée du lowering, jamais du domaine. `src/lowering/expr_lower.rs`, l. 1027 — chapitre 7.
- localisation distincte** Tuple (fichier, ligne, colonne, message), unité de compte de la méthode de mesure sur corpus. Voir définition 25.3.
- lowering (abaissement)** Traduction, en une passe, de l'AST `oxc` vers l'IR (CFG et table des hooks), indépendante des domaines abstraits. chapitre 7, ADR-003.
- marcheur (*walker*)** Parcours linéaire commun aux trois détecteurs de chapitre 5 sur `Program::body`. `src/lowering/detector.rs`, l. 37.
- may / must** Fait vrai sur au moins une exécution (sur-approximation, sert à se taire soudainement) contre fait vrai sur toutes les exécutions (sous-approximation, seul à ouvrir l'ERROR). Voir définition 3.7.
- mint (frapper)** Construire un `Certified` ; fonction privée `Certified::mint`, jamais exposée hors de `query.rs`. `src/rules/api/query.rs`, l. 88 — chapitre 17.
- module_written / mutated** Noms de module écrits par un composant programme-entier, et ra-

- cines mutées d'un corps ; ces noms ne « tiennent » pas à travers la boucle automatique. `src/engine/churn.rs`, l. 202 — chapitre 15.
- Motion** Verdict de mouvement d'une prop de graine : `Still`, `Proven(Certified)`, `Unproven`. `src/rules/api/query.rs`, l. 792 — chapitre 18.
- motion-wins** Règle de `to_stability` : dès qu'un emplacement du produit `StateValue` est en mouvement, la stabilité globale devient `PerRender`. `src/domains/impls/state_value.rs`, l. 308 — chapitre 10.
- mount-only** Effet dont les deps ont une arité exacte de 0 (`[]`) : une seule exécution certaine, au montage. `src/rules/impls/infinite_loop.rs`, l. 96 — chapitre 19, chapitre 15.
- MustResult** Verdict `All(Certified)` / `Some(T)` / `None` d'une primitive `must`. `src/rules/api/query.rs`, l. 117 — chapitre 17.
- narrowing (raffinement de garde)** Raffinement de l'environnement abstrait sur une branche selon sa condition ; à ne pas confondre avec l'opérateur descendant classique de l'interprétation abstraite (non implémenté ici). Voir définition 12.2.
- navigates** Fait qu'une navigation visible quelque part dans le programme fait bouger toute valeur tenue (`held`) par cette navigation. `src/engine/churn.rs`, l. 206 — chapitre 15.
- nom de diagnostic (vs id de règle)** Clé de `Diagnostic::rule`, espace de noms de `off`, du plafond de sévérité et de `-rule` ; distincte de `l'id Rule::name()`, espace de noms des options. `src/rules/registry.rs`, l. 11 — chapitre 17, chapitre C.
- Object.is** Égalité de référence utilisée par React pour comparer deux valeurs de deps, et pour décider un bail-out de `setState`. chapitre 2.
- opaque (T)** Valeur abstraite « n'importe quoi », repli des expressions ou des hooks que l'analyse ne peut ni évaluer ni résumer. `src/lowering/expr_lower.rs`, l. 24 — chapitre 9.
- opaque (hook)** Hook qui n'a pu être ni inline ni résumé par un contrat de bibliothèque ; sa valeur de retour est `T`. `src/engine/analysis_result.rs`, l. 73 — chapitre 8.
- pack** Fichier JSON `{schemaVersion, name, rules}`, espace de noms de règles déclaratives Tier A. Voir définition 22.1.
- passe / itération** Une exécution d'`analyze_cfg` sur un CFG (rendu, effet, ou handler) ; un tour de la boucle de point fixe d'un composant. Voir définition 12.1, définition 13.1.
- PerRender** Référence garantie fraîche à chaque rendu (borne *must*) ; pour une valeur non-référence, « peut changer à chaque rendu ». Voir définition 10.3.
- phase 1 / phase 2** Analyse top-down des composants racines sous `InterCtx` (phase 1) puis balayage intra des composants non atteints, `inter = None` (phase 2). `src/engine/fixpoint.rs`, l. 723 — chapitre 16.
- phase (WriterPhase)** Moment où une écriture s'exécute (`may`) : `Sync`, `Deferred`, `Handler`, `Cleanup`, `Unknown`. Fait partie de la ligne `SlotWriter` (définition 14.2). `src/engine/setters.rs`, l. 687.
- point d'entrée** CFG analysé séparément avec son propre déclencheur : rendu, effet, handler. ADR-009 §3 — chapitre 13.
- polarité** Annotation `must/may/exact` d'une primitive ou d'une colonne de relation ; seul noyau de confiance résiduel du moteur des règles. ADR-021 « Residual trust » — chapitre 17.
- post-await** Voir arête `Await`.
- post-point fixe** État S tel que $F(S) \sqsubseteq S$; ce qui calcule la boucle du point fixe avec widening (pas nécessairement le plus petit point fixe). `src/engine/fixpoint.rs`, l. 495 — chapitre 13.
- primitive (query primitive)** Fonction de `query.rs` qui rend un verdict polarisé (`may`, `must`, ou un jeton `Certified`) à partir des relations du moteur. `src/rules/api/query.rs`, l. 417 — chapitre 17.
- profondeur d'inlining** Nombre de corps imbriqués en cours d'expansion pendant l'interprétation, plafonné (`MAX_INLINE_DEPTH = 3`). `src/domains/interp/interpreter.rs`, l. 21 — chapitre 11.
- ProgramCache / ProgramRelations** Structures programme-entier paresseuses (`OnceLock`), partagées

- par tous les contextes de règles d'un run (index de rendu, index de montage, churn...).
- `src/rules/api/cache.rs`, l. 26 — chapitre 17.
- projection mémo** Valeur d'un `useMemo` : une référence dont la stabilité est le join des stabilités de ses deps. `src/domains/transfer/state_value.rs`, l. 56 — chapitre 11.
- provenance (*via*)** Chaîne de wrappers épissés qui mène à une écriture, ou `Direct` si l'appelant écrit lui-même le slot. Fait partie de la ligne `SlotWriter` (définition 14.2). `src/engine/setters.rs`, l. 789.
- QualifiedSlot (slot qualifié)** Paire (`ComponentId`, `HookLabel`) : nom d'un slot en dehors de son composant. Voir définition 6.3. `src/ir/types.rs`, l. 9.
- QueryContext / NullCtx / FixpointCtx** Canal de requête inter-domaines (une seule requête : `callback_body`) ; `FixpointCtx` le sert depuis la table des corps de `useCallback`, `NullCtx` répond toujours `None`. `src/domains/context.rs`, l. 30 — chapitre 11.
- racine (*root*)** Composant analysé en tête de phase 1, sans parent connu : son paramètre `props` se lit `T`. Voir définition 16.1. `src/engine/root_detector.rs`, l. 25.
- ratchet (voir cliquet)** Voir définition 26.4.
- région (WriterRegion)** Corps lexical d'une écriture : `Render`, `Effect(1)`, `Memo(1)`, `Callback(1)`, `Handler(1)` ; fait exact, partie de définition 14.2. `src/engine/setters.rs`, l. 641.
- registration / registrar** Appel d'un corps d'effet qui confie un callback à quelque chose qui lui survit (`setInterval`, `addEventListener`) ; le *registrar* est la ligne de table qui le reconnaît. `src/engine/registrations.rs`, l. 63 — chapitre 14.
- relation (du moteur)** Fait nommé, calculé une fois par le moteur, à colonnes de polarité déclarée (`slot_writers`, graphe de `churn...`). Voir définition 14.1. `docs/relations.md`.
- remap** Décalage des labels et des `ExprId` d'une copie de CFG greffée par un splice, pour éviter toute collision avec l'appelant. `src/ir/remap.rs`, l. 1 — chapitre 8.
- render dep (dépendance de rendu)** Ensemble *may* des entrées de rendu (`slot`, `setter`, `prop`, `contexte`, `module`) dont une valeur calculée par un composant peut dépendre. Voir définition 16.3. `src/engine/render_deps.rs`, l. 1.
- RenderIndex / MountIndex** Vue programme des résumés `render_deps` de tous les composants (plus les comptes de montage), et index composant $\rightarrow$ sites JSX de montage. `src/rules/helpers/render_tree.rs`, l. 28 — chapitre 21.
- résumé (SummaryValue / HookSummary)** Contrat abstrait, grossier mais sound, d'un hook de bibliothèque sans code source disponible (`TanStack`, `React Router...`). `src/registry/summary.rs`, l. 10 — chapitre 8.
- reviver** Écriture, sur un autre site, qui peut ranimer la garde d'un site déjà tué ; un site prouvé tirer au plus une fois n'est reviver de personne (plus petit point fixe). Voir définition 15.3, #160.
- RuleCtx** Objet unique auquel une règle se lie pour un composant : programme, composant, résultat convergé, configuration, cache. `src/rules/api/query.rs`, l. 318 — chapitre 17.
- salt (sel)** Suffixe `#n` d' α -renommage des liaisons locales d'un CFG greffé par splice, pour garantir l'hygiène du renommage. `src/ir/splice.rs`, l. 300 — chapitre 8.
- same_tick** Fait *may* : une autre écriture synchrone du même slot, dans la même région, est atteignable dans le même tick. `src/engine/setters.rs`, l. 753 — chapitre 14.
- sceller (*seal*)** Fermer un bloc de base courant avec son terminateur et l'insérer dans la table des blocs du CFG. Voir définition 7.1. `src/lowering/cfg_builder.rs`, l. 173.
- seed (graine)** Chemin de `prop` lu par l'initialiseur d'un `useState`, relation `slot_seeds`. `src/engine/seeds.rs`, l. 49 — chapitre 14, ADR-031.
- setter** Fonction d'écriture d'un slot (`Expr::StateSetter(1)` en IR), ou valeur abstraite `SetterVal` portant l'identité (composant, label) de son propriétaire. `src/domains/impls/setter_val.rs`, l. 15 — chapitre 11.
- setter qui s'échappe (*escaping*)** Alias d'un setter utilisé autrement qu'un appel direct ou un

- alias pur ; déclenche un havoc sur le slot correspondant s'il fuit vers un enfant inanalysable. `src/engine/setters.rs`, l. 2261 — chapitre 11, chapitre 14.
- shape (*member contract*)** Contrat par membre nommé d'un objet retourné par un hook de bibliothèque, ou par position d'un tuple. `src/ir/expr.rs`, l. 353 — chapitre 8.
- site** Deux sens distincts : site d'*allocation* (`ExprId`, clé du tas abstrait, définition 6.2) et site d'*écriture* (ligne non-handler d'un corps de rendu ou d'effet, unité de la preuve de convergence du churn, définition 15.2). Voir l'encadré en tête de cette annexe.
- site convergent** Site d'écriture prouvé tirer au plus une fois ; exclu des revivers d'un autre site. `src/engine/churn.rs`, l. 446 — chapitre 15.
- slot** Emplacement d'état d'un `useState/useReducer`, indexé par son `HookLabel`, local au composant. Voir définition 6.3. `src/ir/types.rs`, l. 6.
- SlotWriter (*writer, ligne d'écriture*)** Ligne de la relation « écrivains d'un slot » : un site d'appel de `setter`, avec sa région, sa phase, sa fraîcheur et sa provenance. Voir définition 14.2. `src/engine/setters.rs`, l. 738.
- soundness** Propriété de l'interprétation abstraite : l'ensemble des comportements calculés sur-approxime les comportements réels ; faux positifs tolérés, faux négatifs interdits. Voir définition 3.8.
- source (entrée de rendu)** Ce dont une valeur calculée au rendu peut dépendre : `Slot`, `Setter`, `Prop`, `AllProps`, `Ref`, `Hook`, `Context`, `Module`. `src/engine/render_deps.rs`, l. 53 — chapitre 16.
- splice (greffe)** Insertion du CFG d'un callee (hook custom ou utilitaire) dans l'appelant, au site d'appel, avec α -renommage hygiénique et décalage des identités. Voir définition 8.1. `src/ir/splice.rs`, l. 64.
- StateValue** Valeur abstraite produit ponctuel de huit emplacements, une par sorte JS observable (nombre, booléen, chaîne, référence, null, undefined, `setter`, `other`). Voir définition 10.1. `src/domains/impls/state_value.rs`, l. 26.
- stays_mounted** Fait qu'un parent monte un enfant une fois et le garde monté (gardes et key invariants) : le `[]` de cet enfant ne tire qu'une fois dans la boucle automatique. `src/engine/churn.rs`, l. 368 — chapitre 15.
- store** Table `label` $\rightarrow$ valeur abstraite, sujette au point fixe, mise à jour par *weak update* (« join » plutôt que remplacement). Voir définition 11.2. `src/domains/stores/state_store.rs`, l. 1.
- teardown / handle / disposer** Appel qui défait une registration, tel `clearInterval` ou `removeEventListener`. *handle* est la valeur renvoyée par la registration, jamais descendue par la marche des écrivains. `src/engine/registrations.rs`, l. 271 — chapitre 14.
- temporaire (lowering)** Variable synthétique introduite par le lowering (`__tN`, `__arr_N`, `__obj_N`, `__dstr_N`, `__pN`, `__term_N`) pour porter une sous-expression sans nom source. chapitre 7.
- terminateur (Terminator)** Fin d'un bloc de base : `Jump`, `Branch`, `Return`, `Unreachable`. `src/ir/cfg.rs`, l. 12 — chapitre 6.
- Tier A / B / C** Trois familles de règles : A, déclarative JSON ; B, frontend programmable (abandonné, ADR-023) ; C, Rust de première main, seule à créer des primitives et l'autorité `ERROR`. ADR-021 *Future direction* — chapitre 22.
- treillis** Ensemble ordonné muni d'un plus petit élément $\perp$, d'un plus grand $\top$, et des opérations `join` ($\sqcup$) et `meet` ($\sqcap$). Voir définition 3.2.
- trigger (déclencheur)** Ligne (effet, dep, slot, exact) de la relation `effect_triggers` : tel changement de slot fait bouger telle dep, avec ou sans garantie de relance (exact). `src/engine/triggers.rs`, l. 33 — chapitre 14.
- unbounded (*is_unbounded*)** Prédicat « peut croître sans borne » : intervalle numérique infini, référence `PerRender`, ou emplacement `other` peuplé. `src/domains/impls/state_value.rs`, l. 378 — chapitre 10.
- utilitaire** Fonction de premier niveau non-hook, non-capitalisée, sans retour JSX et non exportée par défaut. Voir définition 5.1. `src/lowering/utility_detector.rs`, l. 17.

Versioned(S) Borne *may* : la référence ne change qu’aux écritures des slots de l’ensemble *S*. Voir définition 10.3.

vue événement / vue inter-rendus Le store d’un slot tient le *join* de toutes les valeurs écrites (vue événement) ; une lecture de l’état convertit la référence en Versioned du slot lu (vue inter-rendus). src/domains/transfer/state_value.rs, l. 122 — chapitre 11, ADR-017.

WalkClass Classe de phase d’un site dans la marche des écrivains : Sync, Deferred, Handler, Cleanup, Unknown (T). src/engine/setters.rs, l. 1332 — chapitre 14.

weak update (mise à jour faible) Écriture d’un slot par *join* ($\text{state}[l] := \text{state}[l] \sqcup v$) plutôt que par remplacement, seule façon de garder la monotonie du point fixe. src/domains/stores/state_store.rs, l. 29 — chapitre 13.

widening Opérateur d’accélération de la convergence : une borne qui grandit à chaque itération saute à $\pm\infty$ (ou au seuil englobant le plus proche, *widening à seuils*). Voir définition 3.6.

widen_trace Trace des slots élargis pendant le point fixe : itération du premier élargissement, et effets « écrivains » incriminés — signal de divergence potentielle pour *infinite-loop*. src/engine/analysis_result.rs, l. 18 — chapitre 13, chapitre 19.

witness (chaîne de témoins) Suite typée de Note (chacune un Step) qui explique, étape par étape, comment un diagnostic a été établi. Voir définition 17.2. src/rules/api/witness.rs, l. 86.

wontfix Issue fermée qui enregistre un refus citable de corriger une limite : à lire avant de re-proposer le même correctif. Voir définition 26.1. CLAUDE.md.

worklist File des blocs dont l’environnement d’entrée a changé, dédoublée, amorcée au bloc d’entrée du CFG ; moteur de analyze_cfg. src/engine/cfg_analyzer.rs, l. 53 — chapitre 12.

wrapper (prouvé) Membre de bibliothèque qui renvoie un handler autour de son argument (form.handleSubmit(cb)), prouvé par un résumé plus un contrôle d’échappement. src/engine/setters.rs, l. 1476 — chapitre 14, chapitre 23.

Résumé du chapitre

- Cette annexe rassemble en une seule liste alphabétique tout le vocabulaire technique du manuscrit : termes mathématiques du chapitre 3, types et fonctions du code, vocabulaire des relations et des packs déclaratifs.
- Un terme déjà formalisé ailleurs (définition 3.2, définition 6.3, définition 14.2, définition 15.1, définition 17.1, définition 17.2...) n’est pas redéfini ici : l’entrée renvoie au chapitre qui porte la définition et sa justification.
- Trois mots changent de sens selon le chapitre — *site*, *région*, *ancre* — et ce glossaire signale systématiquement les deux lectures plutôt que d’en imposer une.
- Les entrées propres au code (un type, un champ, une fonction) portent une localisation vérifiée au commit e67b10a : elles se lisent comme un index de fichiers autant que comme un dictionnaire de concepts.

B.1 Exercices

Exercice B.1: Deux sens d’un même mot

En une phrase chacun, distinguez le *site* du chapitre 6 (définition 6.2) du *site* du chapitre 15 (définition 15.2). Un site d’écriture porte-t-il toujours un ExprId de site d’allocation ? Justifiez à l’aide de la ligne SlotWriter (définition 14.2).

Exercice B.2: Suivre un terme jusqu'à sa définition

Partez de l'entrée « `PerRender` » de ce glossaire. Sans consulter le chapitre 10, énoncez ce que doit être `Versioned(S)` par opposition, en vous appuyant seulement sur le vocabulaire *must/may* de définition 3.7. Vérifiez ensuite votre réponse à la définition renvoyée.

Exercice B.3: Un mot manquant

Choisissez un identifiant Rust public d'un fichier de `src/rules/impls/` qui n'apparaît pas dans ce glossaire. Écrivez, au même format qu'une entrée ci-dessus (terme, une à trois phrases, `\srcline`, chapitre), la fiche qui devrait l'accompagner.

Annexe C

Catalogue des règles

Objectifs du chapitre.

- Disposer d'une fiche de référence compacte pour chacune des dix-neuf règles natives (vingt et un noms de diagnostic) : niveau maximal, primitive must qui ouvre l'ERROR, relations et données du moteur lues, fichier, tests, chapitre où elle est expliquée en détail.
- Disposer de la même fiche pour les règles déclaratives des packs livrés (`guardrails`, `community/*`), et savoir ce que leur statut « non first-party » signifie.
- Comprendre la distinction entre *id de règle* (`Rule::name()`, namespace des options) et *nom de diagnostic* (`Diagnostic::rule`, namespace de `-rule`, `off`, `RuleDoc`), et pourquoi elle empêche un faux négatif silencieux.
- Savoir lire et reproduire les sorties de `reactant rules` et `reactant explain`, et retrouver dans le code la table qui les alimente.

Prérequis : chapitre 17 (le trait `Rule`, le jeton `Certified`, définition 17.1, les primitives `must`, les témoins, définition 17.2) ; chapitre 18, chapitre 19, chapitre 20, chapitre 21 et chapitre 22, dont cette annexe résume et indexe la matière sans la reprendre.

Cette annexe n'ajoute aucun mécanisme nouveau : c'est un catalogue, à consulter plutôt qu'à lire d'un trait. Chaque ligne d'une fiche est une reformulation compacte d'un fait déjà établi et sourcé dans le chapitre correspondant ; le renvoi `chap: . . .` en tête de chaque tableau dit où trouver le bug React visé, les exemples gradués avec leur sortie observée, les faux positifs assumés et les faux négatifs connus. Toutes les sorties de cette annexe ont été observées au commit `e67b10a` avec le binaire `target/debug/reactant`, depuis `/tmp/manuscrit-check/`.

C.1 Lire ce catalogue

C.1.1 Deux espaces de noms

Une règle a un *id* (`Rule::name()`) et émet un ou plusieurs *noms de diagnostic* (`Diagnostic::rule`). La plupart du temps les deux coïncident ; deux règles seulement en émettent un second : `infinite-loop` émet aussi `cross-component-infinite-loop`, `setter-in-render` émet aussi `cross-setter-in-render`. Le module qui compose les frontends (CLI et WASM) l'explique dans son commentaire de tête :

Extrait C.1 – `src/rules/registry.rs`, l. 11–17 — la discipline de nommage : deux espaces de noms, une seule raison

```
11 /// Naming discipline: severity/off overrides are keyed by **diagnostic name**
12 /// (Diagnostic::rule`, the namespace of --rule`/`--ignore-rule` and
13 /// RuleDoc`); options are keyed by **rule id** (Rule::name()`). One rule may
```

```

14 /// emit several diagnostic names (`setter-in-render` also emits
15 /// `cross-setter-in-render`), which is why `off` filters findings at
16 /// emission and never skips a rule's execution – skipping by rule id would
17 /// silently swallow the other diagnostic name, a forbidden false negative.

```

Piège

Les options d'une règle (`minDepth`, `maxDeps`...) s'adressent à l'*id* ; les surcharges de sévérité, `off` et les filtres `-rule/-ignore-rule` s'adressent au *nom de diagnostic*. Adresser des options à `cross-setter-in-render` lève `RegistryError::OptionsOnDiagnosticOnly` : ce nom n'est pas un id de règle, seulement une étiquette de sortie. Couper une règle par son id plutôt que par ses noms de diagnostic ferait taire le nom *et* son cousin `cross-*`, alors qu'`off` ne doit couper que ce qui est explicitement nommé — un faux négatif interdit par construction.

C.1.2 Ce que `reactant rules` et `reactant explain` affichent

Les deux sous-commandes lisent la même table statique, `RULE_DOCS` (`src/rules/docs.rs`, l. 64–), triée alphabétiquement par nom de diagnostic ; les règles chargées dynamiquement depuis un pack (section C.3) y ajoutent leur propre `RuleDoc`, construit depuis leur JSON. Une entrée porte cinq champs : le nom, un résumé d'une ligne (`rules`), une explication, un exemple fautif et une piste de correction (les trois derniers, affichés par `explain`) :

Extrait C.2 – `src/rules/docs.rs`, l. 165–178 — une entrée de `RULE_DOCS` : le résumé, l'explication, l'exemple et la correction d'`infinite-loop`

```

165     doc(
166         "infinite-loop",
167         "effect sets state that re-triggers the effect, and the state diverges",
168         "The render → effect → setState → render cycle never stabilizes: the \
169             abstract value of the state slot required widening and the effect's \
170             write stays unbounded, the structural signature of an infinite render \
171             loop. Detected through the fixpoint (guards that bound the state, e.g. \
172             `if (n < 5) setN(n+1)`, converge and are not flagged), including \
173             through `.then(...)` / `setTimeout(...)` callbacks and inlined \
174             cross-file custom hooks.",
175         "useEffect(() => { setCount(count + 1); }, [count]);",
176         "Add a converging condition, use a deps array that doesn't include the state \
177             being set, or move the update out of the effect.",
178     ),

```

Le test `every_rule_name_has_a_doc` garantit qu'aucune règle native n'est sans `RuleDoc` ; il n'assure rien sur leur fraîcheur — section 19.1 montre un doc-comment de `infinite_loop.rs` en retard de deux ADR sur le code qu'il décrit, quand ce catalogue-ci décrit la règle au commit `e67b10a`.

C.2 Les dix-neuf règles natives

Dix-neuf fichiers de `src/rules/impls/` (tous sauf `mod.rs`) implémentent le trait `Rule` ; ils émettent vingt et un noms de diagnostic une fois comptés `cross-component-infinite-loop` et `cross-setter-in-render`. Les quatre tableaux qui suivent reprennent, groupe par groupe, la même grille : niveau maximal, `primitive must` qui ouvre l'`ERROR` (« — » si la règle ne peut jamais en produire), données et relations du moteur lues, fichier avec sa longueur et le nombre de tests qui l'exercent. Le tableau 17.6 donne, pour chaque `primitive must` citée ici, le fait exact qu'elle certifie.

C.2.1 Les règles sur l'état (chapitre 18)

règle	niveaux	preuve de l'ERROR	données du moteur lues	fichier (lignes, tests)
lazy-init	E,W,I	must_init_calls_setter	initialiseurs State/Ref, setter_var_labels, registre de fonctions	lazy_init.rs (644 l., 16 tests)
unstable-context-value	W	—	module_consts, rendu et tas convergés, is_unstable_reference_only	unstable_context_value.rs (72 l., 14 tests)
redundant-set-state	W	—	store d'état, block_states, slot_writers (régions), escaping_slots	redundant_set_state.rs (629 l., 8 tests)
setter-in-render, cross-setter-in-render	E,W	ExitDominance::certify + phase Sync + must_write	collect_setter_calls, cross_component_setters, converges_once_written	setter_in_render.rs (586 l., 7 tests)
derived-state	W	— (lit must_setter_on_all_paths sans en tirer d'ERROR)	deps, all_setter_labels, slot_writers, slot_setter_escapes	derived_state.rs (169 l., 17 tests)
state-mutation	E,W	must_same_ref_mutation	tous les corps, all_setter_labels, mutation_receiver	state_mutation.rs (521 l., 19 tests)
frozen-initial-state	E,W,I	classify_motion → must_frozen_seed	slot_seeds, tas convergé, MountIndex, may_written_slots	frozen_initial_state.rs (336 l., 39 tests)

TABLE C.1 – Les sept règles d'état du chapitre 18 (E = ERROR, W = WARNING, I = INFO).

C.2.2 Le point fixe et le prop drilling (chapitre 19)

règle	niveaux	preuve de l'ERROR	données du moteur lues	fichier (lignes, tests)
infinite-loop, cross-component-infinite-loop	E,W,I	must_effect_cycle (bras 3, cycle tout-Must mono-composant) ou must_on_all_paths (bras 4, self-slot)	widen_trace, effect_setter_writes, all_setter_labels, cross_component_setters, ChurnGraph::cycles, slot_writers	infinite_loop.rs (1667 l., 19 tests) + tests/effect_cycles.rs (40)
state-lifted-too-high	W	— (jamais d'ERROR: le coût des re-rendus n'est pas certain, leur nombre l'est)	render_deps, RenderIndex::home_of, Home::wasted_renders, Home::siblings, mount_count; options minDepth, minWastedRenders	state_lifted_too_high.rs (146 l., 13 tests)

TABLE C.2 – Les deux règles du chapitre 19 : quatre bras pour la première (tableau 19.1), la dépendance de rendu pour la seconde.

Piège

`infinite-loop` est la seule règle de ce catalogue dont le doc-comment (`src/rules/impls/infinite_loop.rs`, l. 26–37) contredit le code au commit `e67b10a` sur deux points précis (le filtre sur les deps tout-instables, le traitement du cas cross-component sans état partagé connu) — détaillé au section 19.1. Une fiche de catalogue ne remplace jamais une lecture du code pour qui modifie la règle.

règle	niveaux	preuve de l'ERROR	données du moteur lues	fichier (lignes, tests)
widening-info	I	—	widen_trace	widening_info.rs (41 l.) + tests/widening_e2e.rs (4, partagés avec infinite-loop)
analysis-limit	I	—	stats, hook_calls[], opaque, effect_info	analysis_limit_info.rs (158 l.) ; tests dispersés dans les suites e2e des limites (pas de fichier dédié)
conditional-hook	E (tous-jours)	hook_is_conditional → Certified<ConditionalHookCall>	hook_calls, render_cfg (dominance de tous les blocs de sortie atteignables)	conditional_hook.rs (581 l., 9 tests) + tests/conditional_hook_e2e.rs (1)
always-unstable-deps	W	—	hooks (deps), environnement de sortie, stores et tas convergés	always_unstable_deps.rs (489 l., 19 tests)
missing-deps	W	—	effect_info (covering_deps), environnement de sortie, tas convergé	missing_deps.rs (701 l., 30 tests) + tests/deps_exactness.rs (14)
unnecessary-rerender	W	—	hooks (initialiseur, corps), stores convergés au montage	unnecessary_rerender.rs (446 l., 7 tests)
missing-cleanup	W	—	registrations, corps d'effet (cleanup_verdict)	missing_cleanup.rs (113 l., 9 tests)
stale-closure	E (sous preuve), sinon W	must_stale_capture (cinq conjoints) → Certified<StaleCapture>	registrations, corps de callback, alias de setters	stale_closure.rs (469 l., 21 tests)

TABLE C.3 – Les huit règles du chapitre 20 (tableau 20.1 dans ce chapitre-là).

règle	niveaux	preuve de l'ERROR	données du moteur lues	fichier (lignes, tests)
wasted-subtree-render	W	— (une absence d'usage certaine, un coût non chiffré)	render_deps (par composant), RenderIndex, slot_writers, registrations, state_store ; options minWastedRenders, continuousOnly	wasted_subtree_render.rs (333 l., 13 tests)
server-component-hook	W	— (fait de convention, hors du domaine des valeurs)	ModuleTable (directives, arêtes d'import), HookEntry, HookProvenance	server_component_hook.rs (308 l., 5 tests) + tests/nextjs_project.rs (13, partagés)

TABLE C.4 – Les deux règles du chapitre 21 (tableau 21.1 dans ce chapitre-là).

C.2.3 Les règles sur les effets et les hooks à deps (chapitre 20)

C.2.4 Le rendu et les Server Components (chapitre 21)

Remarque

Sept règles seulement plafonnent au-dessus du WARNING: quatre peuvent atteindre l'ERROR sans condition supplémentaire au-delà de leur primitive must (lazy-init si l'initialiseur écrit, setter-in-render, state-mutation, frozen-initial-state), deux sous condition structurelle stricte (conditional-hook, toujours; stale-closure, sous cinq preuves conjointes), et infinite-loop sous deux preuves distinctes selon le bras. Les huit autres — unstable-context-value, redundant-set-state, derived-state, state-lifted-too-high, always-unstable-deps, missing-deps, unnecessary-rerender, missing-cleanup, wasted-subtree-render, server-component-hook — plafonnent au WARNING par construction : soit le fait est certain mais son coût ne l'est pas, soit la question posée sort du domaine que le point fixe a prouvé. widening-info et analysis-limit sont les deux seules règles d'INFO pure : elles ne portent aucun verdict sur le code, seulement sur les limites de l'analyse elle-même.

C.3 Les règles des packs livrés

Le chapitre 22 décrit le langage : une règle déclarative ancre une entité dans une relation du moteur, l'énumère au besoin (forEach), lui applique une conjonction de gardes typées, et déclare une sévérité clampée par pin □ polarité (ADR-22, ADR-23). Cette section n'en répète pas la mécanique ; elle catalogue les règles que les packs livrés au commit e67b10a en tirent.

fichier	pack	règles	thème
packs/guardrails.json	guardrails	5	deps absentes, deps toutes stables, self-retriggering, budget de taille, hooks bannis (tableau C.6)
packs/community/async.json	community-async	5	continuations asynchrones sans teardown, écriture hors phase React, doubles écritures, sélecteurs de store instables
packs/community/effects.json	community-effects	4	enregistrements et teardowns non appariés
packs/community/render.json	community-render	3	props instables, snapshots de store
packs/community/state.json	community-state	3	collisions d'écriture dans le même tick, effets miroir
packs/community/wave2.json	community-wave2	8	ressources non libérées, navigation et travail coûteux en rendu, inputs non contrôlés. . .

TABLE C.5 – Les six packs livrés (packs/), vingt-huit règles déclaratives au total en plus des dix-neuf natives.

C.3.1 guardrails : le pack de démonstration

guardrails est le seul pack dont les règles sont écrites comme des garde-fous d'équipe directement activables ; c'est celui que le chapitre 22 commente ligne à ligne.

self-retriggering-effect est la seule règle déclarative épinglée error parmi les six packs livrés (test no_community_rule_claims_an_error, tests/community_packs.rs) : elle réutilise directement must_setter_on_all_paths, la primitive du tableau 17.6 que derived-state consomme côté natif, ce qui illustre la promesse du langage — « aucune sévérité déclarative au-delà de ce qu'une primitive du moteur a déjà prouvée ».

id (guardrails/. . .)	sévérité	ancre et gardes
effect-without-deps-array	warning	effet dont <code>deps_declared = false</code>
inert-single-dep	warning	effet à ≥ 1 dep, dont chaque dep est prouvée stable (every)
self-retriggering-effect	error	effet dont un setter du corps est dans les deps <i>et</i> <code>must_setter_on_all_paths</code>
oversized-effect	warning	effet dont le nombre de deps dépasse le paramètre <code>maxDeps</code> (défaut 5)
banned-hook	warning	appel de hook dont le nom est dans le paramètre <code>banned</code> (défaut [])

TABLE C.6 – Les cinq règles de `packs/guardrails.json`, lues directement dans le JSON.

C.3.2 Les packs `community/*` : des preuves de vocabulaire, pas des règles à activer

Les vingt-trois règles des cinq packs `community/*` viennent de la campagne « liste de souhaits à l’aveugle » du 2026-09-02 (#128) : chacune traduit en JSON déclaratif une demande formulée sans connaître le vocabulaire disponible. Leur statut est écrit noir sur blanc en tête du test qui les protège :

« These are NOT first-party rules. Several are proxies with known false positives — the campaign’s point was to measure the vocabulary against demand, and a proxy that had to stand in for a missing fact is evidence, not a recommendation. » (`tests/community_packs.rs`, l. 4–7)

Piège

Charger un pack `community/*` dans un projet réel sans relire ses règles une à une expose à des faux positifs assumés par construction : la doc de `controlled-input-without-a-writer`, par exemple, rapporte que sur un corpus de 34 730 fichiers tous les résultats étaient des `<input type="hidden">`, que la règle ne peut pas exclure faute de lire la *valeur* du prop `type` (#67). Aucune de ces règles n’atteint `error` ; c’est la seule garantie que le test leur impose.

C.3.3 La sortie de `reactant rules`

Chargé avec le pack `guardrails` (fichier de configuration minimal, `{ "packs": [ . . . ] }`), `reactant rules` affiche les vingt et un noms natifs puis les cinq noms qualifiés `guardrails/. . .`, triés alphabétiquement dans chaque bloc :

```
$ reactant rules --config reactant.config.json
```

always-unstable-deps	a dep is a fresh reference every render, so the deps array never matches
analysis-limit	the analyzer deliberately truncated analysis here (potential false negatives)
conditional-hook	hook called inside a conditional branch
cross-component-infinite-loop	child effect sets parent state, parent re-renders child, effect refires
cross-setter-in-render	parent's setter (received as prop) called during render
derived-state	effect only mirrors another state, so compute it during render
frozen-initial-state	useState seeded from a prop that changes, so the state freezes at the first value
infinite-loop	effect sets state that re-triggers the effect, and the state diverges
lazy-init	useState initializer calls a function on every render
missing-cleanup	effect starts something long-lived and returns no teardown
missing-deps	effect body captures a variable not listed in its deps array
redundant-set-state	setState called with the value the state already holds
server-component-hook	a hook is called in a Next.js Server Component
setter-in-render	setState called during the render body
stale-closure	long-lived callback keeps a state value frozen at registration time
state-lifted-too-high	state is used only deep in one child subtree; the components above re-render to
↪ pass it down	
state-mutation	state or prop object mutated in place, same reference, no re-render
unnecessary-rerender	mount-only effect immediately overwrites the initial state
unstable-context-value	context provider hands consumers a new object every render

wasted-subtree-render ↳ not depend on it	a state written on every keystroke or pointer move re-renders subtrees that do
widening-info	state slot required widening to converge (precision lost here)
guardrails/effect-without-deps-array	an effect declares no dependency array at all
guardrails/inert-single-dep	every dependency of an effect is provably stable, so it can never re-run
guardrails/self-retriggering-effect	an effect unconditionally writes a state slot listed in its own dependency array
guardrails/oversized-effect	an effect's dependency array exceeds the team's size budget
guardrails/banned-hook	a custom hook the team has withdrawn is still being called

Avec les cinq packs `community/*` ajoutés à la configuration, la même commande produit quarante-neuf lignes au lieu de vingt-six — vingt-trois règles déclaratives de plus (les vingt-huit du tableau C.5 moins les cinq déjà chargées de `guardrails`), chacune sous son préfixe de pack (`community-async/. . .`, `community-wave2/. . . . .`), sans qu'aucun nom natif ne bouge : les deux espaces de noms ne se recouvrent jamais, par construction du préfixe `pack/`.

Résumé du chapitre

- Ce catalogue index dix-neuf règles natives (vingt et un noms de diagnostic, deux règles en émettant chacune deux) et vingt-huit règles déclaratives réparties sur six packs livrés (`guardrails`, cinq `community/*`).
- Deux espaces de noms coexistent et ne se confondent pas : l'*id* de règle (`Rule::name()`, namespace des options) et le *nom de diagnostic* (`Diagnostic::rule`, namespace de `off/-rule/RuleDoc`) ; reactant rules et reactant explain lisent la table statique `RULE_DOCS` plus les `RuleDoc` générés par pack.
- Sept règles natives seulement peuvent atteindre l'`ERROR`, chacune derrière une primitive `must` distincte du tableau 17.6 (chapitre 17) ; les douze autres plafonnent au `WARNING` ou à l'`INFO` par construction, pas par prudence éditoriale.
- `guardrails` est le seul pack directement activable ; les cinq packs `community/*` sont des preuves d'expressivité issues d'une campagne à l'aveugle (#128), assumées comme porteuses de faux positifs et jamais épinglées `error`.
- Pour chaque règle, le tableau de son groupe donne le chemin vers le fichier, le nombre de tests, et le chapitre qui explique le bug React visé, l'algorithme et les cas limites ; cette annexe ne s'y substitue pas.

C.4 Exercices

Exercice C.1: Deux noms, une règle

Un projet configure `"off": ["infinite-loop"]`. Le composant suivant, où un effet écrit l'état d'un parent reçu en `prop`, est-il encore signalé ? Justifier en citant le nom de diagnostic concerné et la raison pour laquelle `off` ne coupe pas l'exécution de la règle elle-même (section C.1).

Solution

Oui : `off` vise le nom de diagnostic `infinite-loop`, pas l'`id` de règle `InfiniteLoop`. Le bras 2 (`cross-component`) de la même règle émet `cross-component-infinite-loop`, un nom distinct que la configuration ne mentionne pas ; le composant reste signalé sous ce nom. Couper l'`id` aurait éteint les deux noms d'un coup, un faux négatif que la discipline de nommage de `registry.rs` interdit précisément d'introduire.

Exercice C.2: Plafond de sévérité

Parmi `state-lifted-too-high`, `conditional-hook` et `guardrails/self-retriggering-effect`, laquelle peut produire une `ERROR`, et à quelle primitive `must` le doit-elle ?

Solution

`conditional-hook` le peut toujours, via `hook_is_conditional` qui frappe un `Certified<ConditionalHookCall>` dès qu'un hook ne domine pas toutes les sorties atteignables du rendu (tableau C.3). `guardrails/self-retriggering-effect` le peut aussi, mais par une garde déclarative `must_setter_on_all_paths`, la même primitive que `derived-state` consomme côté natif sans jamais en tirer d'`ERROR` — deux lectures différentes du même fait. `state-lifted-too-high` ne le peut jamais : le nombre de re-rendus gaspillés est certain, leur coût ne l'est pas, et la règle plafonne au `WARNING` par construction (tableau C.2).

Bibliographie

- [CC77] Patrick Cousot and Radhia Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)*, pages 238–252. ACM, 1977.
- [CC79] Patrick Cousot and Radhia Cousot. Systematic design of program analysis frameworks. In *Proceedings of the 6th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)*, pages 269–282. ACM, 1979.
- [LAY25] Jay Lee, Joongwon Ahn, and Kwangkeun Yi. React-trace: A semantics for understanding react hooks. *Proceedings of the ACM on Programming Languages*, 9(OOPSLA2), 2025. Sémantique formelle des hooks React ; référence d’ADR-001.
- [Met26a] Meta Platforms, Inc. eslint-plugin-react-hooks. <https://www.npmjs.com/package/eslint-plugin-react-hooks>, 2026.
- [Met26b] Meta Platforms, Inc. React — the library for web and native user interfaces (documentation officielle). <https://react.dev/>, 2026. Consulté en septembre 2026.
- [Met26c] Meta Platforms, Inc. Render and commit. <https://react.dev/learn/render-and-commit>, 2026.
- [Met26d] Meta Platforms, Inc. Rules of hooks. <https://react.dev/reference/rules/rules-of-hooks>, 2026.
- [rbo26] rboudrouss. reactant-analyzer — static analyzer for react hook bugs based on abstract interpretation. <https://github.com/rboudrouss/reactant-analyzer>, 2026.
- [RY20] Xavier Rival and Kwangkeun Yi. *Introduction to Static Analysis: An Abstract Interpretation Perspective*. MIT Press, 2020.
- [The26] The Oxc Project Developers. Oxc — the javascript oxidation compiler. <https://oxc.rs/>, 2026.

Index

abstraction, 73
ADR, 1022, 1071
alias locaux, 207
analyse syntaxique, 5
arène, 132
arête arrière, 237
assurance, 794
attribution, 952

baseline, 1004
blind spot, 945
bloc de base, 177
bloc de jonction, 226
bloc orphelin, 226
blocs orphelins, 178
bottom, 70

candidat, 873
CFG, 177
chaîne ascendante, 70
chaîne de témoins, 679, 681
chemins nommés, 947
clean bill, 915
cliquet, 694, 1038
clé synthétique, 231
complétion disjonctive, 347
composant, 131, 148
compromis, 1017
concrétisation, 73
conditionnel, 796
constat, 1047
correcte, 73
correspondance de Galois, 73
couche, 116
couche règles, 652
couvert, 208
cuisson, 875

de transport, 965
diagramme de Hasse, 69

diamant, 233
domine, 92
driver, 926
déclencheur, 844
défaut, 1017
dépendance de rendu, 606, 636

empreinte, 1004
en couches, 46
en-tête, 237
ensembles k -bornés, 313
enveloppe, 966
environnement abstrait, 394
environnement d'entrée, 443
environnement de sortie, 443, 789

fail-closed, 120
faits syntaxiques de module, 135
faux négatif, 15, 90
faux positif, 15, 90
fichiers suivis, 947
fonction de transfert, 77
frappe, 665

graphe de churn, 568

handler, 259
hauteur, 70
Havoquer, 626
hook custom, 131, 148
hôte, 926, 965

identifiant de règle, 656
inliner, 95
inlining, 267
inter-procédurale, 95
internement, 200
interprétation abstraite, 9, 68
intra-procédurale, 94
issue fermée wontfix, 1022
itération de Kleene, 78

- jeton certifié, 664
- join, 70
- journal de précision, 1022
- kill, 787
- label, 189
- liaison périmée, 247
- localisation, 952
- localisation distincte, 1002
- lowering, 214
- may, 88, 359
- meet, 70
- mise à jour faible, 33, 96
- mise à jour forte, 96
- module serveur, 859
- monotone, 78
- motion-wins, 373
- must, 88, 359, 665
- mémoire d'arbre, 55
- nom de diagnostic, 656
- non-changement, 1031
- ordre partiel, 70
- Ouvrir, 219
- pack, 873
- paire fires / silent, 993
- passe, 443
- pin, 896
- plus petit point fixe, 78
- point fixe, 10, 78
- polarité, 89, 515, 518, 896
- positive-only, 881
- post-point fixe, 78
- pour se taire, 789
- pour tirer, 789
- produit ponctuel, 347
- pré-en-tête, 237
- racine, 615
- racine de configuration, 935
- racine de découverte, 935
- raffinement de garde, 76, 453
- registration, 819
- relation du moteur, 518
- reviver, 581
- règle d'affirmation, 701
- règle de signallement, 701
- règle déclarative, 873
- Réserver, 219
- résidu, 348
- sceau de sévérité, 665
- Sceller, 219
- site, 172, 185
- slot, 172, 189
- slot qualifié, 520, 570
- sonde, 465
- sorte, 877
- sound, 15, 90
- sous-approximation, 68
- splice, 267
- stabilité référentielle, 357
- stale closure, 4
- store, 406
- structurants, 1071
- sur tous les chemins, 92
- sur-approximation, 68
- sémantique collectrice, 68
- sémantique concrète, 26, 68
- terminateur, 177
- test-cliquet, 1022
- Tier A, 867
- top, 70
- tour, 28
- trace de changement, 359
- treillis, 70
- treillis plat, 309
- typestate, 663
- témoigné, 983
- updater, 30
- utilitaire, 131, 148, 291
- valeur abstraite, 10
- variables libres, 205
- verdict polarisé, 665
- widening, 9, 83
- widening à seuils, 83
- worklist, 79
- écrivain, 535
- épinglage, 208
- α -renommage, 281